PROGRAMLAMA · DERS KİTABI

C Dillerinin Tarihi ve Öğrenimi

C · C++ · C#

YUSUF BARLAK & AHMET Ş. KART

virenta & brave raven

C Dillerinin Tarihi ve Öğrenimi

C · C++ · C# — Sıfırdan başlayanlar ve hatırlamak isteyenler için

YUSUF BARLAK & AHMET Ş. KART

virenta & brave raven

Birinci basım · 2026
Kitaptaki örnekler C17, C++17 ve C# 12 (.NET 8) ile denenmiştir.

İçindekiler

Önsöz **20**

KISIM 0 — Başlarken

Bölüm 1 Bilgisayar, Program ve Programlama Dilleri **25**

1.1 Bilgisayar Nedir? 25
1.2 Bit, Bayt ve Sayı Sistemleri 27
1.3 Algoritma 29
1.4 Makine Dili, Assembly ve Yüksek Seviyeli Diller 33
1.5 Derleyici, Yorumlayıcı ve Sanal Makine 34
1.6 Programlama Paradigmaları 36
1.7 Programlama Hataları 37
1.8 Bu Kitabın Düzeni 39

Bölüm 2 C Ailesinin Tarihi: Bell Labs'tan .NET'e **45**

2.1 Öncüller: 1950'ler ve 1960'lar 45
2.2 BCPL ve B: C'nin Doğrudan Ataları 46
2.3 C'nin Doğuşu ve UNIX (1969–1973) 46
2.4 K&R ve Taşınabilirlik (1978) 47
2.5 C'nin Standartları 48
2.6 C++: Sınıflı C'den Modern C++'a 50
2.7 Java ve C#'ın Doğuşu 52
2.8 .NET'in Evrimi ve C# Sürümleri 53
2.9 İsimlerin Hikâyesi 54
2.10 Soy Ağacı 55
2.11 Kullanım Alanları ve İlk Karşılaştırma 56
2.12 Bugün Neden Hâlâ Önemliler? 57

Bölüm 3 Geliştirme Ortamının Kurulumu **61**

3.1 Neye İhtiyacımız Var? 61
3.2 Windows'ta gcc ve g++: MSYS2 ile MinGW-w64 63
3.3 Linux ve macOS 65
3.4 VS Code Kurulumu ve Eklentiler 66
3.5 Visual Studio Community 68
3.6 .NET SDK Kurulumu 69
3.7 İlk Programlar: Üç Dilde Merhaba, Dünya! 71
3.8 Çevrim İçi Derleyiciler 74
3.9 Sık Kurulum Sorunları 74
3.10 Hata Ayıklayıcıya İlk Bakış 76

KISIM I — C

Bölüm 4 C'ye Giriş: İlk Program ve Derleme Süreci **83**

4.1 C'nin Felsefesi 83
4.2 İlk Program: Merhaba Dünya 84
4.3 Derleme ve Çalıştırma 85
4.4 Derleme Süreci: Kaynak Koddan Programa 87
4.5 Derleyici Uyarıları 90
4.6 Yorum Satırları 91

4.7 Kod Düzeni ve Okunabilirlik ... 92
4.8 Kaçış Dizileri ... 93
4.9 Hata Mesajlarını Okumak ... 95
4.10 main Fonksiyonu ve Çıkış Kodu ... 100
4.11 Aynı Program Üç Dilde ... 101

Bölüm 5 Değişkenler, Veri Tipleri ve Operatörler ... 107
5.1 Değişken Nedir? ... 107
5.2 Tanımlama, İlk Değer Verme ve Atama ... 108
5.3 Değişken Adları ve Anahtar Sözcükler ... 111
5.4 Tamsayı Tipleri ... 113
5.5 Kayan Noktalı Tipler ... 116
5.6 Sabitler ... 118
5.7 printf ile Değer Yazdırma: Biçim Belirteçleri ... 123
5.8 Aritmetik Operatörler ... 125
5.9 Atama Operatörleri, Artırma ve Azaltma ... 128
5.10 İlişkisel ve Mantıksal Operatörler ... 130
5.11 Tip Dönüşümleri ... 133
5.12 Taşma ve Kayan Nokta Hassasiyeti ... 137
5.13 Operatör Önceliği ve Birleşme Yönü ... 141
5.14 Örnek Programlar ... 143

Bölüm 6 Klavyeden Girdi ve Biçimli Çıktı ... 149
6.1 printf'in Yapısı ... 149
6.2 scanf ile Klavyeden Okuma ... 156
6.3 scanf'in Dönüş Değeri ... 160
6.4 Girdi Arabelleği ve %c Tuzağı ... 161
6.5 Metin Okumak: %s, Boşluklu Metin ve fgets ... 164
6.6 Girdi Doğrulama ... 165
6.7 Örnek Programlar ... 167
6.8 Türkçe Karakterler ve Hizalama ... 170

Bölüm 7 Koşullu İfadeler ... 175
7.1 if Deyimi ... 175
7.2 if-else ... 177
7.3 else if Merdiveni ... 178
7.4 Blok ve Süslü Parantez ... 178
7.5 İç İçe if ve Sarkan else ... 180
7.6 Mantıksal Operatörlerle Bileşik Koşullar ... 181
7.7 = ile == Karışıklığı ... 183
7.8 Ondalıklı Sayıları Koşulda Karşılaştırmak ... 184
7.9 ! Operatörünün Önceliği ... 185
7.10 Üçlü Operatör ?: ... 186
7.11 switch Deyimi ... 187
7.12 Örnek Programlar ... 191

Bölüm 8 Döngüler ... 199
8.1 while Döngüsü ... 199
8.2 do-while Döngüsü ... 201
8.3 for Döngüsü ... 202
8.4 Sayaç, Toplayıcı ve Bayrak ... 204
8.5 break, continue ve Sonsuz Döngüler ... 205
8.6 İç İçe Döngüler ve Desenler ... 208
8.7 İzleme Tablosu ve Off-by-One Hatası ... 210
8.8 Klasik Algoritmalar ... 211
8.9 Kullanıcı Girdisiyle Çalışan Döngüler ... 216

8.10 Ondalık Sayaçla Döngü Kurmak ... 219
8.11 goto Üzerine ... 219

Bölüm 9 Fonksiyonlar ... 225
9.1 Neden Fonksiyon? ... 225
9.2 Fonksiyonun Anatomisi ... 226
9.3 void Fonksiyonlar ve Erken Çıkış ... 227
9.4 Prototip (Bildirim) ... 228
9.5 Parametre ve Argüman ... 230
9.6 Değerle Geçirme ... 231
9.7 Çağrı Yığını ... 232
9.8 Kapsam ... 233
9.9 static Yerel Değişken ... 235
9.10 return Olmadan Biten Fonksiyon ... 236
9.11 Hazır Fonksiyonlar ... 237
9.12 Algoritmaları Fonksiyona Dönüştürmek ... 242
9.13 Fonksiyonlarla Program Tasarımı: Hesap Makinesi ... 243
9.14 Sık Yapılan Hatalar ... 246

Bölüm 10Diziler ... 253
10.1 Neden Dizi? Tanımlama ve Bellek ... 253
10.2 İlk Değer Verme ... 255
10.3 İndeks ve Sınır Dışı Erişim ... 256
10.4 Eleman Sayısını Bulmak: sizeof ... 258
10.5 Diziyi Doldurmak ve Dolaşmak ... 259
10.6 Temel Dizi Algoritmaları ... 259
10.7 Diziler ve Fonksiyonlar ... 261
10.8 Kopyalama ve Karşılaştırma ... 263
10.9 Ters Çevirme ve Kaydırma ... 264
10.10 Doğrusal ve İkili Arama ... 266
10.11 Sıralama: Kabarcık ve Seçmeli ... 267
10.12 İki Boyutlu Diziler ... 270
10.13 Değişken Uzunluklu Diziler ... 273

Bölüm 11Karakter Dizileri (String) ... 279
11.1 String Nedir? ... 279
11.2 Tanımlama ve İlk Değer ... 281
11.3 Yazdırma ve Karakter Karakter Dolaşma ... 283
11.4 Klavyeden String Okumak ... 283
11.5 string.h Fonksiyonları ... 286
11.6 == ile Karşılaştırma ve = ile Atama Tuzağı ... 290
11.7 Kendi String Fonksiyonlarımızı Yazmak ... 291
11.8 ctype.h: Karakter Denetimi ve Dönüşümü ... 292
11.9 Metin ile Sayı Arasında Dönüşüm ... 293
11.10 Güvenlik: Arabellek Taşması ... 295
11.11 Türkçe Karakterler ve UTF-8 ... 296
11.12 String Dizisi ... 297
11.13 Klasik String Algoritmaları ... 298

Bölüm 12İşaretçiler (Pointers) ... 305
12.1 Bellek ve Adres ... 305
12.2 İşaretçi Tanımlama ve Dolaylama ... 306
12.3 NULL ve Başlatılmamış İşaretçi ... 308
12.4 Adresle Geçirme ... 309
12.5 İşaretçi Aritmetiği ... 311
12.6 Diziler ve İşaretçiler ... 313

12.7 İşaretçiler ve Stringler ... 315
12.8 const ve İşaretçiler ... 317
12.9 Yerel Değişkenin Adresini Döndürmek ... 318
12.10 İşaretçinin İşaretçisi ... 318
12.11 İşaretçi Dizisi ve Diziye İşaretçi ... 319
12.12 Komut Satırı Argümanları ... 320
12.13 void İşaretçi ve Fonksiyon İşaretçisi ... 322
12.14 Sık Yapılan İşaretçi Hataları ... 324

Bölüm 13Dinamik Bellek Yönetimi ... 329
13.1 Neden Dinamik Bellek? ... 329
13.2 Bellek Bölgeleri ... 330
13.3 malloc ve free ... 330
13.4 calloc ... 332
13.5 Boyutu Kullanıcıdan Alıp Dizi Oluşturmak ... 333
13.6 realloc ile Büyüyen Dizi ... 334
13.7 Fonksiyondan Dinamik Dizi Döndürmek ... 337
13.8 Dinamik String ... 338
13.9 Dinamik İki Boyutlu Dizi ... 339
13.10 Bellek Hataları ... 341
13.11 Bellek Hatalarını Bulan Araçlar ... 343
13.12 sizeof ve Ayrılan Boyut ... 345
13.13 Üç Dilde Bellek Yönetimi ... 345

Bölüm 14Yapılar: struct, union, enum ve typedef ... 351
14.1 Neden Yapı? ... 351
14.2 Yapı Tanımı ve Üyelere Erişim ... 352
14.3 typedef ile Tip Takma Adı ... 354
14.4 Atama ve Karşılaştırma ... 355
14.5 Yapı Dizileri ... 356
14.6 Yapılar ve Fonksiyonlar ... 359
14.7 İç İçe Yapılar ... 361
14.8 Yapılar ve Dinamik Bellek ... 362
14.9 Bellek Yerleşimi ve Dolgu ... 363
14.10 union ... 366
14.11 enum ... 369
14.12 Uygulama: Sınıf Kayıt Sistemi ... 371

Bölüm 15Önişlemci, Makrolar ve Çok Dosyalı Programlar ... 377
15.1 Önişlemci Nedir? ... 377
15.2 Nesne Benzeri Makrolar: #define ... 378
15.3 Parametreli Makrolar ... 379
15.4 Metne Çevirme (#) ve Birleştirme (##) ... 383
15.5 Koşullu Derleme ... 383
15.6 Önceden Tanımlı Adlar ve assert ... 386
15.7 Programı Dosyalara Bölmek ... 387
15.8 Derleme ve Bağlama ... 389
15.9 Başlık Koruyucusu ... 391
15.10 Başlıkta Neler Bulunur? ... 393
15.11 extern ve Dosya Kapsamlı static ... 393
15.12 make ile Derlemeyi Otomatikleştirmek ... 395
15.13 Üç Dilde: Önişlemci ve Program Yapısı ... 396
15.14 Sık Yapılan Hatalar ... 397

Bölüm 16Bit Düzeyinde İşlemler ... 401
16.1 İkilik Gösterim ve Bit Numaralama ... 401

16.2 Bit Operatörleri ... 402
16.3 Kaydırma Operatörleri ... 403
16.4 Bitleri Ekrana Yazmak ... 404
16.5 Bit Maskeleri ... 405
16.6 Bayraklar ve İzin Sistemi ... 407
16.7 Sık Kullanılan Küçük Bit Teknikleri ... 409
16.8 İşaretli Sayılar ve İkiye Tümleyen ... 410
16.9 Küçük Tiplerde Tip Yükseltmesi Tuzağı ... 412
16.10 Önceliğe Dikkat ... 413

Bölüm 17Dosya İşlemleri ... 419
17.1 Dosya Nedir? Metin ve İkili Dosyalar ... 419
17.2 FILE, fopen ve fclose ... 420
17.3 Metin Dosyasına Yazmak ... 422
17.4 Metin Dosyasından Okumak ... 423
17.5 EOF ve feof Tuzağı ... 425
17.6 Metin Dosyası Programları ... 427
17.7 Standart Akımlar ve Yönlendirme ... 428
17.8 İkili Dosyalar: fwrite ve fread ... 430
17.9 Dosyada Gezinmek: fseek, ftell, rewind ... 432
17.10 Dosya Hatalarını Yönetmek ... 434
17.11 Platform Farkları: Satır Sonu, Yollar, Kodlama ... 435
17.12 Uygulama: Dosyada Telefon Rehberi ... 436

Bölüm 18Özyineleme (Recursion) ... 443
18.1 Özyineleme Nedir? ... 443
18.2 Temel Durum ve Özyinelemeli Durum ... 444
18.3 Çağrı Yığını ve Faktöriyel ... 445
18.4 Sayılarla Özyineleme ... 447
18.5 Fibonacci ve Bellekleme ... 449
18.6 Dizi ve String Üzerinde Özyineleme ... 451
18.7 Hanoi Kuleleri ... 453
18.8 Özyinelemeli İkili Arama ... 454
18.9 Yazdırmayı Çağrıdan Önce mi Sonra mı Yapmalı? ... 455
18.10 Temel Durumu Unutmak: Yığın Taşması ... 457
18.11 Özyineleme mi, Döngü mü? ... 458

Bölüm 19Veri Yapılarına Giriş: Bağlı Liste, Yığın ve Kuyruk ... 463
19.1 Veri Yapısı Nedir? ... 463
19.2 Düğüm ve Tek Yönlü Bağlı Liste ... 464
19.3 Liste Başını Değiştiren Fonksiyonlar ... 466
19.4 Silme ve Sıralı Ekleme ... 469
19.5 Yığın (Stack) ... 471
19.6 Kuyruk (Queue) ... 476
19.7 Çift Yönlü Bağlı Liste ... 480
19.8 Karmaşıklık: Büyük O Gösterimine Giriş ... 483

Bölüm 20C Uygulama Projesi: Öğrenci Kayıt Sistemi ... 489
20.1 Gereksinimler ... 489
20.2 Tasarım ... 490
20.3 Girdi Okuma Yardımcısı ... 491
20.4 Liste İşlemleri ... 492
20.5 Ekrana Yazan İşlevler ... 493
20.6 Menü Komutları ... 494
20.7 Dosyaya Kaydetme ve Yükleme ... 495
20.8 Ana Program ... 496

20.9 Tam Program ... 497
20.10 Örnek Çalışma ... 501

KISIM II — C++

Bölüm 21C'den C++'a: Tarih, Felsefe ve İlk Program ... 507
21.1 C++ Neden Doğdu? ... 507
21.2 C++'ın Tasarım Felsefesi ... 509
21.3 C ile Uyumluluk ... 511
21.4 İlk C++ Programı ... 516
21.5 Ad Alanları ve using namespace std; ... 519
21.6 Derleme, Dosya Uzantıları ve Başlıklar ... 521
21.7 Standartlar ve Kitabın Kullandığı Sürüm ... 522
21.8 Üç Dilde: Aynı Program ... 523

Bölüm 22Tipler, Değişkenler ve Operatörler: C++'ın Getirdikleri ... 529
22.1 C'den Gelenler: Hızlı Bakış ... 529
22.2 bool: Gerçek Bir Tip ... 531
22.3 İlk Değer Verme Biçimleri ve Daraltma Koruması ... 532
22.4 auto ile Tip Çıkarımı ... 533
22.5 Sabitler: const ve constexpr ... 535
22.6 C++ Tarzı Tip Dönüştürmeleri ... 537
22.7 nullptr ve NULL ... 539
22.8 Sayısal Sınırlar: numeric_limits ... 540
22.9 Referanslara İlk Bakış ... 542
22.10 Kapsam, Ad Alanları ve :: Operatörü ... 543
22.11 Operatörler: C++'ta Değişenler ... 545
22.12 Hepsini Bir Araya Getirelim ... 546

Bölüm 23iostream ile Girdi ve Çıktı ... 551
23.1 Akış Kavramı ... 551
23.2 cout ile Yazdırma ... 552
23.3 cin ile Okuma ... 554
23.4 Satır Okumak: getline ... 556
23.5 Biçimli Çıktı: iomanip ... 558
23.6 Hatalı Girdi ve Akışın Kilitlenmesi ... 562
23.7 Hata İletileri İçin cerr ... 566
23.8 printf ile Karşılaştırma ... 567
23.9 Örnek Uygulama: Fatura ... 568
23.10 Türkçe Karakterler ve Hizalama ... 570

Bölüm 24Kontrol Yapıları: Koşullar ve Döngüler ... 575
24.1 C'den Gelenler: Hızlı Bakış ... 575
24.2 Karşılaştırma Sonuçları bool'dur ... 576
24.3 Başlatıcılı if ve switch (C++17) ... 578
24.4 switch: Düşme ve case Etiketinde Tanımlama ... 580
24.5 Küçük Farklar: ?: ve Ondalık Karşılaştırma ... 582
24.6 while, do-while ve Klasik for ... 583
24.7 Aralık Tabanlı for (C++11) ... 584
24.8 Döngü Algoritmalarının C++ Sürümleri ... 587
24.9 Örnek: Not İstatistikleri ... 589

Bölüm 25Fonksiyonlar: C++'ın Getirdikleri ... 595
25.1 C'den Gelenler: Hızlı Bakış ... 595
25.2 Referansla Geçirme ... 596
25.3 const & ile Geçirme ... 601

25.4 Varsayılan Parametreler ... 603
25.5 Fonksiyon Aşırı Yükleme ... 605
25.6 inline ve constexpr Fonksiyonlar ... 609
25.7 Dönüş Tipi auto ve [[nodiscard]] ... 611
25.8 Fonksiyon Şablonuna İlk Bakış ... 612
25.9 Fonksiyon Gibi Davranan Değerler ... 613
25.10 Üç Dilde Parametre Geçirme ... 615
25.11 Örnek: Fonksiyonlarla Puan Özeti ... 617
25.12 Sık Yapılan Hatalar ... 618

Bölüm 26Diziler, std::array ve std::vector ... 623
26.1 C Dizileri C++'ta ... 623
26.2 std::array: Boyutu Bilen Sabit Dizi ... 626
26.3 std::vector: Büyüyen Dizi ... 628
26.4 Eleman Ekleme, Silme ve Yeniden Boyutlandırma ... 630
26.5 Boyut, Kapasite ve Bellekte Büyüme ... 632
26.6 Vector'ü Fonksiyona Geçirmek ... 635
26.7 vector<vector<int>> ile Matris ... 637
26.8 Algoritmalara İlk Bakış ... 638
26.9 Hangisini Seçmeli? ... 639

Bölüm 27std::string ve Karakter İşlemleri ... 645
27.1 C Stringlerinden std::string'e ... 645
27.2 Oluşturma ve İlk Değer ... 646
27.3 Birleştirme ve Karşılaştırma ... 647
27.4 Uzunluk ve Karaktere Erişim ... 649
27.5 Dolaşma ... 650
27.6 Arama: find ve npos ... 651
27.7 Parçalama ve Değiştirme ... 653
27.8 Sayı ile Metin Arasında Dönüşüm ... 654
27.9 Satır Okumak: getline ... 657
27.10 cctype: Karakter Sınıflandırma ve Dönüştürme ... 658
27.11 Klasik Metin İşleri ... 659
27.12 C ile Birlikte Çalışma: c_str ve Ham Diziler ... 661
27.13 Türkçe Karakterler ve Bayt Uzunluğu ... 662
27.14 Üç Dilde Metin ... 663

Bölüm 28İşaretçiler ve Referanslar ... 669
28.1 C İşaretçileri C++'ta ... 669
28.2 Referans: Kurallar ve Davranış ... 671
28.3 Referans mı, İşaretçi mi? ... 673
28.4 nullptr ile "Değer Yok" ... 674
28.5 const Doğruluğu ... 675
28.6 Referans Döndürmek ... 677
28.7 this İşaretçisi: Küçük Bir Önizleme ... 679
28.8 lvalue ve rvalue: Sezgisel Giriş ... 680
28.9 Üç Dilde Referanslar ... 682
28.10 Sık Yapılan Hatalar ... 683

Bölüm 29Dinamik Bellek, RAII ve Akıllı İşaretçiler ... 689
29.1 new ve delete ... 689
29.2 new[] ve delete[]: Dinamik Dizi ... 692
29.3 Yapıcı ve Yıkıcı: new Ne Yapar? ... 693
29.4 Elle Yönetimin Zorluğu ... 694
29.5 RAII: Kaynağı Bir Nesneye Emanet Etmek ... 695
29.6 std::unique_ptr: Tek Sahipli İşaretçi ... 697

29.7 std::shared_ptr: Paylaşımlı Sahiplik ... 701
29.8 std::weak_ptr ve Döngüsel Referans ... 703
29.9 Ne Zaman Hangisi? ... 705

Bölüm 30Sınıflar ve Nesneler ... 713
30.1 Neden Sınıf? ... 713
30.2 İlk Sınıf: Üyeler ve Üye Fonksiyonlar ... 714
30.3 Erişim Belirteçleri ve Kapsülleme ... 715
30.4 Yapıcılar ... 717
30.5 Yıkıcı ... 720
30.6 this: Nesnenin Kendisi ... 722
30.7 const Üye Fonksiyonlar ... 723
30.8 static Üyeler ... 724
30.9 Nesneleri vector'de Tutmak ve Fonksiyonlara Geçirmek ... 725
30.10 Kopyalama: Kopya Yapıcı ve Kopya Atama ... 727
30.11 Sınıfı .hpp ve .cpp Dosyalarına Bölmek ... 730
30.12 struct ile class ve friend ... 731
30.13 Hepsini Bir Araya Getirelim: Kesir Sınıfı ... 732

Bölüm 31Kalıtım ve Polimorfizm ... 741
31.1 Kalıtım Nedir? ... 741
31.2 Erişim Denetimi: public, protected, private ... 743
31.3 Yapıcı ve Yıkıcı Sırası ... 745
31.4 Fonksiyon Gizleme ... 747
31.5 virtual, override ve Dinamik Bağlama ... 748
31.6 Perde Arkası: Sanal Tablo ... 751
31.7 Saf Sanal Fonksiyon ve Soyut Sınıf ... 752
31.8 Sanal Yıkıcı Neden Gerekli? ... 755
31.9 Çok Biçimli Koleksiyon ... 757
31.10 Nesne Dilimleme Tuzağı ... 758
31.11 Aşağı Dönüşüm: dynamic_cast ... 760
31.12 Çalışan Hiyerarşisi ... 760
31.13 Çoklu Kalıtım ... 762

Bölüm 32Operatör Aşırı Yükleme ... 769
32.1 Neden Operatör Aşırı Yükleme? ... 769
32.2 Kurallar ve Sınırlar ... 770
32.3 Üye Fonksiyon Olarak Operatörler ... 771
32.4 Üye Olmayan Operatörler ... 773
32.5 Akış Operatörleri: << ve >> ... 775
32.6 Karşılaştırma Operatörleri ... 777
32.7 Kesir Sınıfı Bir Arada ... 778
32.8 İndeks ve Fonksiyon Çağrı Operatörleri ... 780
32.9 Artırma ve Azaltma Operatörleri ... 782
32.10 Atama Operatörleri ... 783
32.11 Aşırı Yüklenmemesi Gerekenler ve İyi Uygulamalar ... 784
32.12 Üç Dilde Operatör Aşırı Yükleme ... 785

Bölüm 33Şablonlar (Templates) ... 791
33.1 Neden Şablon? C'deki Çözümler ... 791
33.2 Fonksiyon Şablonu ... 792
33.3 Sınıf Şablonu ... 794
33.4 Birden Fazla Parametre, Tip Olmayan Parametreler ... 797
33.5 Şablon Özelleştirme ... 799
33.6 Perde Arkası: Örnekleme ve Başlık Dosyaları ... 800
33.7 Şablon Hata Mesajları ... 802

Bölüm 34İstisna Yönetimi ... 809
34.1 Dönüş Koduyla Hata Bildiriminin Sorunları ... 809
34.2 throw, try ve catch ... 810
34.3 Standart İstisna Hiyerarşisi ... 812
34.4 Birden Çok catch ve Sıra ... 813
34.5 catch (...) ve Yeniden Fırlatma ... 815
34.6 Kendi İstisna Sınıfımız ... 816
34.7 Yığın Çözülmesi ... 818
34.8 noexcept ... 820
34.9 İstisna mı, Dönüş Değeri mi? ... 821
34.10 Üç Dilde Hata Yönetimi ... 822

Bölüm 35STL Kapsayıcıları ... 827
35.1 STL'nin Üç Ayağı ... 827
35.2 Sıralı Kapsayıcılar ... 829
35.3 pair, tuple ve Yapısal Bağlama ... 832
35.4 set ve multiset: Sıralı Kümeler ... 833
35.5 map ve multimap: Anahtar–Değer Çiftleri ... 836
35.6 Sırasız Kapsayıcılar ... 840
35.7 Uyarlayıcılar: stack, queue, priority_queue ... 842
35.8 Yineleyici Geçersizleşmesi ... 845
35.9 Hangi Kapsayıcı, Ne Zaman? ... 845

Bölüm 36STL Algoritmaları, Yineleyiciler ve Lambda İfadeleri ... 851
36.1 Yineleyici Nedir? ... 851
36.2 Hazır Algoritmalar ... 855
36.3 Temel Algoritmalar ... 856
36.4 Lambda İfadeleri ... 858
36.5 Koşul Alan Algoritmalar ... 861
36.6 Sıralama Ölçütleri ... 864
36.7 std::function ve Çağrılabilir Nesneler ... 865
36.8 Örnek: Öğrenci Listesini Süzmek ve Özetlemek ... 866
36.9 Hangi Algoritma, Hangi Kapsayıcı? ... 868

Bölüm 37Dosya İşlemleri ve Akışlar ... 873
37.1 fstream Sınıfları ve İlk Program ... 873
37.2 Açma Kipleri ve Kapatma ... 875
37.3 Dosyadan Okumak ... 876
37.4 Döngü Koşulunda eof Tuzağı ... 878
37.5 Akış Durumu: fail ve clear ... 879
37.6 Dosyaya Ekleme: ios::app ... 880
37.7 CSV Dosyaları ... 881
37.8 RAII: Dosya Kendiliğinden Kapanır ... 883
37.9 Konum ve Atlama: seekg, seekp, tellg ... 885
37.10 İkili Dosyalar: write ve read ... 885
37.11 Dosya Yolu ve Çalışma Klasörü ... 888
37.12 filesystem Kitaplığı (C++17) ... 888

Bölüm 38Özyineleme C++'ta ... 895
38.1 Bölüm 18'den Taşıdıklarımız ... 895
38.2 vector ile Özyineleme ... 895
38.3 Bellekleme: std::map ile Tablo ... 897
38.4 Geri İzleme: Permütasyonlar ... 898
38.5 Alt Kümeler ve Geri Alma ... 899
38.6 Birleştirmeli Sıralama (Merge Sort) ... 900

38.7 Hızlı Sıralama (Quicksort) ... 901

Bölüm 39Modern C++: Taşıma Semantiği ve Ötesi ... 909

39.1 Neden Taşıma? Kopyalamanın Maliyeti ... 909
39.2 lvalue, rvalue ve std::move ... 911
39.3 Taşıma Yapıcısı ve Taşıma Ataması ... 911
39.4 Beşler Kuralı ve Sıfır Kuralı ... 915
39.5 auto ve decltype ... 916
39.6 std::optional ... 918
39.7 std::variant ... 919
39.8 std::string_view ... 920
39.9 constexpr Fonksiyonlar ... 921
39.10 enum class ... 923
39.11 C++20'ye Bakış ... 923
39.12 Modern C++ Yazım Önerileri ... 926
39.13 Üç Dilde: Aynı Sorunlar, Farklı Çözümler ... 927

Bölüm 40C++ Uygulama Projesi: Kütüphane Yönetim Sistemi ... 933

40.1 Gereksinimler ... 933
40.2 Tasarım ... 934
40.3 Hata Sınıfı ve Yardımcılar ... 934
40.4 Yayın Hiyerarşisi ... 935
40.5 Kütüphane Sınıfı ... 936
40.6 Dosyaya Kaydetme ve Yükleme ... 939
40.7 Menü ve Ana Program ... 940
40.8 Tam Program ... 941
40.9 Örnek Çalışma ... 947

KISIM III — C#

Bölüm 41.NET Dünyası ve C#'a Giriş ... 953

41.1 C#'ın Tasarım Hedefleri ... 953
41.2 .NET Platformu: Kaynak Koddan Makine Koduna ... 954
41.3 SDK, Çalışma Zamanı ve Sınıf Kütüphanesi ... 956
41.4 Yönetilen Bellek ve Çöp Toplayıcı ... 957
41.5 İlk Program: Main ve Top-Level Statements ... 958
41.6 Ad Alanları ve using ... 961
41.7 Konsola Yazmak, Yorumlar ve Derleyici Mesajları ... 962
41.8 Proje Yapısı ve dotnet Araçları ... 964
41.9 C# Nerede Kullanılır? ... 966

Bölüm 42Değişkenler, Veri Tipleri ve Operatörler ... 971

42.1 Değişkenler ve İsimlendirme ... 971
42.2 Değer Tipleri ve Referans Tipleri ... 974
42.3 Sayısal Tipler ve Sabit Boyutları ... 976
42.4 Gerçek Sayılar: float, double ve decimal ... 979
42.5 char, bool ve string ... 981
42.6 Sabit Değerler (Literal) ... 984
42.7 var, const ve readonly ... 986
42.8 Aritmetik Operatörler ... 988
42.9 Atama, Artırma ve Azaltma ... 990
42.10 Karşılaştırma ve Mantıksal Operatörler ... 993
42.11 Bit Operatörleri ve Operatör Önceliği ... 995
42.12 Tip Dönüşümleri ve Taşma ... 998
42.13 Nullable Değer Tipleri ... 1004

Bölüm 43Konsol Girdi/Çıktı ve Biçimlendirme 1011
43.1 Console Sınıfı ve Yazdırma 1011
43.2 Klavyeden Okuma 1014
43.3 Birleşik Biçimlendirme 1015
43.4 Dize İçine Değer Yerleştirme 1016
43.5 Biçim Belirteçleri 1017
43.6 Genişlik ve Hizalama 1019
43.7 Kültür Ayarları 1020
43.8 Metni Sayıya Çevirmek ve Güvenli Girdi 1023
43.9 Türkçe Karakterler 1025
43.10 Örnek Programlar 1025

Bölüm 44Koşullu İfadeler ve Desen Eşleme 1033
44.1 if, else if ve else 1033
44.2 switch Deyimi 1035
44.3 switch İfadesi 1039
44.4 Desen Eşleme 1040
44.5 Üçlü Operatör ve null Operatörleri 1045
44.6 Örnek Uygulama: Menülü Banka Programı 1047
44.7 Örnek Uygulama: Kargo Ücreti 1048

Bölüm 45Döngüler 1055
45.1 while Döngüsü 1055
45.2 do-while Döngüsü 1056
45.3 for Döngüsü 1057
45.4 break, continue ve Sonsuz Döngü 1058
45.5 foreach Döngüsü 1059
45.6 İç İçe Döngüler ve Desenler 1061
45.7 Klasik Algoritmaların C# Sürümleri 1062

Bölüm 46Metotlar 1069
46.1 Neden Metot? 1069
46.2 Metodun Anatomisi 1070
46.3 void ve Değer Döndüren Metotlar 1073
46.4 Parametre Geçirme: Varsayılan Davranış 1075
46.5 ref: Değişkenin Kendisini Geçirmek 1076
46.6 out: Değer Döndürmenin İkinci Yolu 1077
46.7 in: Salt Okunur Referans 1079
46.8 params: Değişken Sayıda Argüman 1080
46.9 Varsayılan Değerli ve Adlandırılmış Parametreler 1081
46.10 Aşırı Yükleme 1082
46.11 İfade Gövdeli Metotlar 1084
46.12 Yerel Fonksiyonlar 1085
46.13 Birden Çok Değer Döndürmek: Demetler 1086
46.14 Math Sınıfı 1087
46.15 Random Sınıfı ve Rastgele Sayılar 1088
46.16 Örnek: Metotlarla Küçük Bir Hesap Kütüphanesi 1090
46.17 Üç Dilde Metotlar 1091

Bölüm 47Diziler 1097
47.1 Dizi Tanımlama ve Oluşturma 1097
47.2 Length, İndeks ve Sınır Denetimi 1100
47.3 Diziyi Dolaşmak 1101
47.4 İndeksler ve Aralıklar 1103
47.5 Diziler Referans Tipidir 1104

47.6 Diziyi Kopyalamak ... 1105
47.7 Diziyi Metoda Geçirmek ... 1106
47.8 Array Sınıfı ... 1108
47.9 Temel Dizi Algoritmaları ... 1109
47.10 Çok Boyutlu Diziler ... 1113
47.11 Düzensiz Diziler ... 1115

Bölüm 48String, StringBuilder ve Karakterler ... 1123
48.1 string Tipinin Yapısı ... 1123
48.2 Değiştirilemezlik ... 1125
48.3 Temel Üyeler ... 1127
48.4 Bölme ve Birleştirme ... 1129
48.5 Karşılaştırma ve Kültür ... 1131
48.6 Çok Satırlı ve Ham Metin Sabitleri ... 1133
48.7 StringBuilder ... 1134
48.8 char: Karakterlerle Çalışmak ... 1137
48.9 Örnek Programlar ... 1141

Bölüm 49Sınıflar ve Nesneler ... 1149
49.1 Neden Sınıf? ... 1149
49.2 İlk Sınıfımız: Alanlar ve Metotlar ... 1150
49.3 Erişim Belirleyiciler ve Kapsülleme ... 1152
49.4 Özellikler (Property) ... 1153
49.5 Yapıcılar ... 1155
49.6 this Anahtar Sözcüğü ... 1158
49.7 static Üyeler ve static Sınıflar ... 1159
49.8 ToString ve Equals ... 1161
49.9 Referans Semantiği ... 1163
49.10 null ve NullReferenceException ... 1165
49.11 Nesne Dizileri ... 1166
49.12 Örnek: BankaHesabi ... 1167
49.13 Üç Dilde: Sınıf ... 1168

Bölüm 50Kalıtım ... 1175
50.1 Neden Kalıtım? ... 1175
50.2 Neler Devralınır? ... 1177
50.3 Tekli Kalıtım Kuralı ... 1179
50.4 Yapıcılar ve base(...) ... 1180
50.5 Metot Ezme: virtual ve override ... 1183
50.6 base ile Taban Metodunu Çağırmak ... 1185
50.7 Gizleme ve new ... 1186
50.8 sealed: Ezilemez, Türetilemez ... 1187
50.9 Yukarı ve Aşağı Dönüşüm ... 1188
50.10 object: Her Sınıfın Atası ... 1191
50.11 Kalıtım mı, Kompozisyon mu? ... 1193
50.12 Örnek: Çalışan Hiyerarşisi ... 1194
50.13 C++'tan Gelenler İçin Farklar ... 1196

Bölüm 51Polimorfizm, Soyut Sınıflar ve Arayüzler ... 1203
51.1 Çok Biçimlilik: Aynı Çağrı, Farklı Davranış ... 1203
51.2 Sanal Çağrıların Tuzakları ... 1206
51.3 Soyut Sınıflar ... 1208
51.4 Arayüzler (interface) ... 1211
51.5 Soyut Sınıf mı, Arayüz mü? ... 1216
51.6 .NET'in Hazır Arayüzleri ... 1217
51.7 Bağımlılığı Arayüzle Tersine Çevirmek ... 1221

51.8 Tam Örnek: Ödeme Sistemi ... 1223

Bölüm 52Değer Tipleri: struct, enum ve record ... 1233

52.1 struct: Değer Semantiği ... 1233
52.2 struct mı, class mı? ... 1236
52.3 readonly struct ... 1236
52.4 enum: Adlandırılmış Sabitler ... 1238
52.5 Metinden Enum Üretmek ... 1240
52.6 [Flags]: Bit Bayrağı Olarak Enum ... 1241
52.7 record: Veri Taşıyan Tipler ... 1242
52.8 Kutulama ve Kutudan Çıkarma ... 1246

Bölüm 53İstisna Yönetimi ... 1253

53.1 İstisna Nedir? ... 1253
53.2 try ve catch ... 1254
53.3 İstisna Hiyerarşisi ... 1256
53.4 Birden Çok catch ve Sıra ... 1257
53.5 finally: Her Durumda Çalışan Blok ... 1258
53.6 throw ile İstisna Fırlatmak ... 1260
53.7 throw; ile throw ex; Farkı ... 1262
53.8 Kendi İstisna Sınıfımız ve when Süzgeci ... 1264
53.9 using ve IDisposable ... 1265
53.10 İstisna mı, Dönüş Değeri mi? ... 1267
53.11 Üç Dilde Hata Yönetimi ... 1268

Bölüm 54Koleksiyonlar ve Generics ... 1275

54.1 Dizilerden Koleksiyonlara ... 1275
54.2 List: Büyüyen Dizi ... 1277
54.3 Dolaşırken Değiştirmek ... 1279
54.4 Arama ve Sıralama ... 1280
54.5 Liste Bir Referans Tipidir ... 1281
54.6 Dictionary: Anahtar–Değer Çiftleri ... 1282
54.7 HashSet ve SortedSet: Kümeler ... 1285
54.8 Queue ve Stack ... 1285
54.9 LinkedList ... 1287
54.10 Sıralı Sözlük: SortedDictionary ve SortedList ... 1288
54.11 Kendi Generic Sınıfımız ... 1289
54.12 Generic Metotlar ... 1291
54.13 Kısıtlar: where ... 1292
54.14 Koleksiyon Arayüzleri ... 1294
54.15 Doğru Koleksiyonu Seçmek ... 1295

Bölüm 55Delegate, Event ve Lambda İfadeleri ... 1301

55.1 Delegate: Metodu Değişkende Tutmak ... 1301
55.2 Func, Action ve Predicate ... 1303
55.3 Lambda İfadeleri ... 1305
55.4 Çok Noktaya Yayın (Multicast) ... 1307
55.5 event: Yayıncı ve Abone ... 1308
55.6 Üç Dilde: Geri Çağırma ... 1312
55.7 Sık Hatalar ve İyi Uygulamalar ... 1312

Bölüm 56LINQ ... 1317

56.1 LINQ Nedir? ... 1317
56.2 Örnek Veri: Öğrenci Listesi ... 1318
56.3 Sıralama ... 1319
56.4 Sorgu Sözdizimiyle Aynı Sorgu ... 1320

56.5 Gruplama: GroupBy ... 1321
56.6 Tek Eleman Seçmek: First, FirstOrDefault, Single ... 1322
56.7 Soru Soran ve Sayan Metotlar ... 1323
56.8 Tekrarsız Yapmak, Dilimlemek ve Sonucu Saklamak ... 1325
56.9 Ertelenmiş Yürütme ... 1325
56.10 Üç Dilde: Aynı İş, Üç Yol ... 1327
56.11 Gerçekçi Bir Örnek: Ürün Stok Raporu ... 1329

Bölüm 57Dosya İşlemleri ... 1333
57.1 File Sınıfı ile Yazma ve Okuma ... 1333
57.2 Dosya Yolları, Çalışma Klasörü ve Path ... 1335
57.3 Hataları Yakalamak ... 1336
57.4 StreamWriter, StreamReader ve using ... 1337
57.5 Klasörler ve FileInfo ... 1339
57.6 CSV Dosyalarını Okumak ve Yazmak ... 1341
57.7 JSON ile Kaydetme ... 1343
57.8 Asenkron Dosya İşlemleri ... 1344
57.9 Sık Yapılan Hatalar ... 1346

Bölüm 58Özyineleme C#'ta ... 1349
58.1 ve 38'den Taşıdıklarımız ... 1349
58.2 Klasör Ağacını Dolaşmak ... 1350
58.3 Dictionary ile Hafızalama: Para Bozma ... 1351
58.4 İkili Ağaç: Dugum Sınıfı ... 1352
58.5 Yığın Taşması ve StackOverflowException ... 1354
58.6 Kuyruk Özyinelemesi ... 1356

Bölüm 59Modern C#: Nullable Referans Tipleri, async/await ve Diğerleri ... 1361
59.1 Nullable Referans Tipleri ... 1361
59.2 Null Operatörleri: ?., ??, ??= ve ! ... 1363
59.3 Tuple ve Ayrıştırma ... 1365
59.4 Desen Eşlemenin İleri Biçimleri ... 1367
59.5 Kodu Kısaltan Özellikler ... 1368
59.6 Eşzamansız Programlama: Neden Gerekli? ... 1370
59.7 async, await ve Task ... 1370
59.8 Birden Çok Görev: WhenAll ... 1372
59.9 İstisnalar ve await ... 1373
59.10 Eşzamanlılık: Üç Dile Genel Bakış ... 1375
59.11 C# Sürüm Sürüm: Yeniliklerin Özeti ... 1376

Bölüm 60C# Uygulama Projesi: Envanter Takip Sistemi ... 1381
60.1 Gereksinimler ... 1381
60.2 Tasarım ... 1382
60.3 Ürün ve Hata Sınıfı ... 1382
60.4 Envanter Sınıfı ... 1382
60.5 LINQ ile Raporlar ... 1383
60.6 Depo Arayüzü ve CSV ... 1384
60.7 Menü ve Ana Program ... 1384
60.8 Tam Program ... 1384

KISIM IV — Karşılaştırma ve Kapanış

Bölüm 61Üç Dil Yan Yana ... 1395
61.1 Vaka 1: Dosyadan Sayı Okuyup İstatistik Çıkarmak ... 1395
61.2 Vaka 2: Öğrenci Sınıfı ... 1399
61.3 Vaka 3: Kelime Frekansı ... 1403

61.4 Üç Vakadan Çıkan Genel Resim ... 1406
61.5 Ortak Miras: Aynı Sözdiziminin İzleri ... 1406
61.6 Karşılaştırma Tabloları ... 1406
61.7 Geçiş Tuzakları ... 1410
61.8 Hangi Dilde Hangi Çözüm? ... 1411

Bölüm 62 Hangi Dil, Ne Zaman? Kullanım Alanları ve Yol Haritası ... 1415
62.1 Ortak Miras, Farklı Hedefler ... 1415
62.2 C'nin Güçlü ve Zayıf Yönleri ... 1416
62.3 C++'ın Güçlü ve Zayıf Yönleri ... 1416
62.4 C#'ın Güçlü ve Zayıf Yönleri ... 1417
62.5 Alanlara Göre Dil Seçimi ... 1418
62.6 Dil Seçimi İçin Bir Karar Rehberi ... 1421
62.7 Sektörde Kullanım Örnekleri ... 1422
62.8 Bu Kitaptan Sonra: Genel Yol Haritası ... 1423
62.9 Her Dil İçin Sonraki Adımlar ... 1424
62.10 Kapanış ... 1425

EKLER — Ekler

Ek A ASCII Tablosu ve Sayı Sistemleri ... 1428
A.1 Sayı sistemleri kısaca ... 1428
A.2 Onluktan diğer tabanlara çevirme ... 1428
A.3 Gruplama yoluyla çevirme ... 1429
A.4 Önek notasyonu ... 1429
A.5 ASCII kontrol karakterleri (0–31 ve 127) ... 1429
A.6 Kaçış dizileri ... 1430
A.7 Boşluk ve noktalama (32–47) ... 1431
A.8 Rakamlar ve ara semboller (48–64) ... 1431
A.9 Büyük harfler ve semboller (65–96) ... 1432
A.10 Küçük harfler ve semboller (97–126) ... 1433
A.11 Silme karakteri (127) ... 1434
A.12 Sayı sistemleri çevirme tablosu (0–32) ... 1434

Ek B Operatör Öncelik Tabloları ... 1436
B.1 C operatör öncelik tablosu ... 1436
B.2 C++ operatör öncelik tablosu ... 1437
B.3 C# operatör öncelik tablosu ... 1438
B.4 Üç dil arasındaki farklar ... 1439
B.5 Sık yapılan hatalar ... 1440

Ek C Anahtar Kelimeler ... 1441
C.1 C Anahtar Sözcükleri ... 1441
C.2 C++ Anahtar Sözcükleri ... 1443
C.3 C# Ayrılmış Sözcükleri ... 1445
C.4 C# Bağlamsal Anahtar Sözcükleri ... 1448

Ek D Sık Görülen Derleyici ve Çalışma Zamanı Hataları ... 1450
D.1 C Hataları ... 1450
D.2 C++ Hataları ... 1454
D.3 C# Hataları ... 1457

Ek E Terimler Sözlüğü ... 1460
E.1 A ... 1460
E.2 B ... 1461
E.3 Ç ... 1461

E.4 D ... 1461
E.5 E ... 1462
E.6 F ... 1462
E.7 G ... 1463
E.8 H ... 1463
E.9 İ ... 1463
E.10 K ... 1464
E.11 L ... 1464
E.12 M ... 1464
E.13 N ... 1465
E.14 O ... 1465
E.15 Ö ... 1465
E.16 P ... 1465
E.17 R ... 1465
E.18 S ... 1466
E.19 Ş ... 1466
E.20 T ... 1466
E.21 U ... 1467
E.22 Ü ... 1467
E.23 V ... 1467
E.24 Y ... 1467
E.25 Z ... 1468

Ek F Alıştırmaların Cevapları ... 1470

Önsöz

Bu kitap, programlama dünyasının en köklü ailelerinden birini, **C ailesini**, doğduğu sırayla anlatır. Önce 1972'de Bell Laboratuvarları'nda ortaya çıkan C'yi, ardından aynı çatı altında C'nin üzerine inşa edilen C++'ı, son olarak da 2000'li yılların başında Microsoft'un .NET platformuyla birlikte sunduğu C#'ı öğreneceğiz.

Üç dili bir arada anlatmanın bir nedeni var. Bu diller birbirinden habersiz doğmadı. C++, C'nin sözdizimini ve felsefesini devraldı ve onu sınıflarla, şablonlarla genişletti. C# ise hem C'nin sade sözdizimini hem de C++'ın nesne yönelimli fikirlerini aldı, bunlara yönetilen bellek ve zengin bir standart kütüphane ekledi. Bir dili öğrenirken diğerinin neden öyle tasarlandığını anlamak, üçünü de daha derinden kavramayı sağlar. `printf` ile `std::cout` ve `Console.WriteLine` arasındaki farkı gören biri, yalnızca üç ayrı komut ezberlemiş olmaz; programlama dillerinin zaman içinde hangi sorunlara nasıl çözümler ürettiğini de görmüş olur.

Bu kitap kimler için?

Kitap iki tür okur düşünülerek yazıldı. Birincisi programlamaya **hiç başlamamış** olanlar. Onlar için ilk bölümler bilgisayarın ne olduğundan, bir programın nasıl çalıştığından ve gerekli araçların nasıl kurulacağından başlar. İkincisi bir zamanlar bu dillerle uğraşmış, aradan zaman geçmiş ve bilgilerini **tazelemek** isteyenler. Onlar için her bölüm tanım kutuları, kritik noktalar ve bölüm özetleriyle hızlıca taranabilecek biçimde düzenlendi.

Kitabın düzeni

Kitap dört kısım ve eklerden oluşur:

- **Kısım 0 — Başlarken:** Bilgisayar ve programlama kavramları, C ailesinin tarihçesi, geliştirme ortamlarının kurulumu.
- **Kısım I — C:** Değişkenlerden işaretçilere, dinamik bellekten dosya işlemlerine ve temel veri yapılarına kadar C dili.
- **Kısım II — C++:** C'den C++'a geçiş, sınıflar, kalıtım, şablonlar, STL ve modern C++.
- **Kısım III — C#:** .NET platformu, nesne yönelimli programlama, koleksiyonlar, LINQ ve modern C#.
- **Kısım IV — Karşılaştırma ve Kapanış:** Üç dilin yan yana incelenmesi, güçlü ve zayıf yönleri, hangi işte hangisinin seçileceği.
- **Ekler:** ASCII tablosu, operatör öncelikleri, anahtar kelimeler, sık görülen hata mesajları, terimler sözlüğü ve alıştırmaların cevapları.

C++ kısmı, okurun C kısmını okuduğunu varsayar ve "C'de böyleydi, C++'ta böyle" diyerek ilerler. C# kısmı da önceki iki dile sık sık geri döner. Yine de yalnızca C#'ı öğrenmek isteyen biri doğrudan Kısım III'ten başlayabilir; gerekli yerlerde önceki bölümlere atıf yapılmıştır.

Kutular ve işaretler

Metin boyunca aşağıdaki kutularla karşılaşacaksınız:

- **Tanım** — Bir kavramın kısa ve kesin tanımı.

- **Kritik Nokta** — Kaçırılmaması gereken, çoğu zaman hataların kaynağı olan önemli bilgi.
- **Sık Yapılan Hata** — Yeni başlayanların sıkça düştüğü bir tuzak ve ondan nasıl kaçınılacağı.
- **İpucu** — İşi kolaylaştıran pratik bir öneri.
- **Üç Dilde** — Aynı konunun C, C++ ve C#'taki karşılıklarının kısa bir karşılaştırması.
- **Tarihçe** — Bir özelliğin ne zaman ve neden dile eklendiğine dair not.

Kod örnekleri, üst köşesinde dosya adı yazan açık renkli kutularda verilir. Programın ekrana yazdıkları hemen altındaki koyu renkli **Çıktı** kutusunda görünür. Çıktı kutusunda sarı renkle vurgulanmış metinler, programı çalıştıran kişinin klavyeden yazdığı girdilerdir.

Nasıl çalışmalı?

Programlama okuyarak değil, yazarak öğrenilir. Her örneği kendi bilgisayarınızda yazmanızı, derlemenizi ve çalıştırmanızı öneririz. Kopyala-yapıştır yerine elle yazmak, sözdizimini çok daha hızlı yerleştirir. Ardından örneği değiştirin: bir sayıyı büyütün, bir koşulu tersine çevirin, bir satırı silin ve ne olduğunu gözlemleyin. Derleyicinin verdiği hata mesajlarını okumayı alışkanlık hâline getirin; bu mesajlar ilk başta korkutucu görünse de zamanla en iyi yardımcınız olur.

Her bölümün sonunda bir özet ve alıştırmalar bulunur. Alıştırmaların cevapları kitabın sonundaki Ek F'dedir. Cevaba bakmadan önce soruyu gerçekten denemek, öğrenmenin en etkili yoludur.

Kitaptaki bütün tam programlar C17 (`gcc`), C++17 (`g++`) ve C# 12 (.NET 8) ile derlenip çalıştırılarak denenmiştir.

İyi çalışmalar.

KISIM 0

Başlarken

Bilgisayar, programlama ve C ailesinin hikâyesi

Kod yazmaya başlamadan önce zemini hazırlayacağız. Bu kısımda bilgisayarın bir programı nasıl çalıştırdığını, programlama dillerinin neden ortaya çıktığını ve C ailesinin yarım asrı aşan hikâyesini göreceğiz. Son olarak C, C++ ve C# için gerekli derleyicileri ve editörleri kuracak, üç dilde de ilk programımızı çalıştıracağız.

BÖLÜM 1

Bilgisayar, Program ve Programlama Dilleri

BU BÖLÜMDE

- Bir bilgisayarın ana bileşenlerini (işlemci, bellek, depolama, giriş-çıkış) ve bir programın bellekte nasıl çalıştığını
- Bit, bayt, ikilik ve onaltılık sayı sistemlerini; sayılar arasında dönüşüm yapmayı
- Algoritma kavramını; akış şeması ve sözde kod (pseudocode) ile bir algoritmayı anlatmayı
- Makine dili, assembly ve yüksek seviyeli dil arasındaki farkı; derleyici, yorumlayıcı ve sanal makine mantığını
- Yordamsal, nesne yönelimli ve fonksiyonel programlama yaklaşımlarını ve C, C++, C#'ın bu yaklaşımlardaki yerini
- Sözdizimi, çalışma zamanı ve mantık hatalarını birbirinden ayırmayı
- Bu kitabın neden C, C++, C# sırasıyla ilerlediğini ve kitabın işaretlerini nasıl okuyacağımızı

Programlamaya başlamadan önce üç şeyi bilmek gerekir: bilgisayar neyle çalışır, bir program ne demektir ve insanın yazdığı metin makinenin anlayacağı hâle nasıl gelir. Bu bölüm, kitabın geri kalanının dayanacağı bu zemini hazırlar. Henüz tek satır C yazmayacağız; ama sonraki bölümlerde karşımıza çıkacak "bayt", "bellek", "derleme" ve "algoritma" gibi sözcüklerin anlamını burada oturtacağız.

Konuları bilen okur için bölüm hızlı taranabilecek biçimde düzenlendi. Hiç ön bilgisi olmayan okur için ise her kavram en basit örnekten başlar. Bölümün sonunda küçük algoritmaları kâğıt üzerinde tasarlayabilecek, sayıları ikilik ve onaltılık tabana çevirebilecek ve C, C++, C# dillerinin hangi yoldan çalıştırılabilir programa dönüştüğünü açıklayabileceksiniz.

1.1 Bilgisayar Nedir?

Bilgisayar, verileri alıp saklayan, kurallara göre işleyen ve sonucu geri veren bir makinedir. Kurallar bir program olarak verilir; aynı makine, yüklenen programa göre hesap makinesi, oyun konsolu ya da metin düzenleyici olur. Makinenin kendisine **donanım** (hardware), üzerinde çalışan programlara **yazılım** (software) denir.

Temel donanım bileşenleri

Bir bilgisayarın içinde birçok parça vardır, ama programcı için dört görev önemlidir: işlemek, geçici saklamak, kalıcı saklamak ve dış dünyayla iletişim kurmak.

Bileşen	Görevi	Örnek
İşlemci (CPU, central processing unit)	Komutları tek tek okuyup yürütür; aritmetik ve mantık işlemlerini yapar	Masaüstü ve dizüstü bilgisayar işlemcileri, telefon işlemcileri
Bellek (RAM, random access memory)	Çalışan programı ve verisini geçici olarak tutar; çok hızlıdır, elektrik kesilince içeriği silinir	8 GB, 16 GB RAM
Depolama (storage)	Dosyaları ve programları kalıcı saklar; RAM'den yavaş ama çok daha büyüktür	SSD, sabit disk, USB bellek
Giriş-çıkış aygıtları (I/O)	Kullanıcı ve dış dünyayla veri alışverişi	Klavye, fare, ekran, yazıcı, ağ kartı

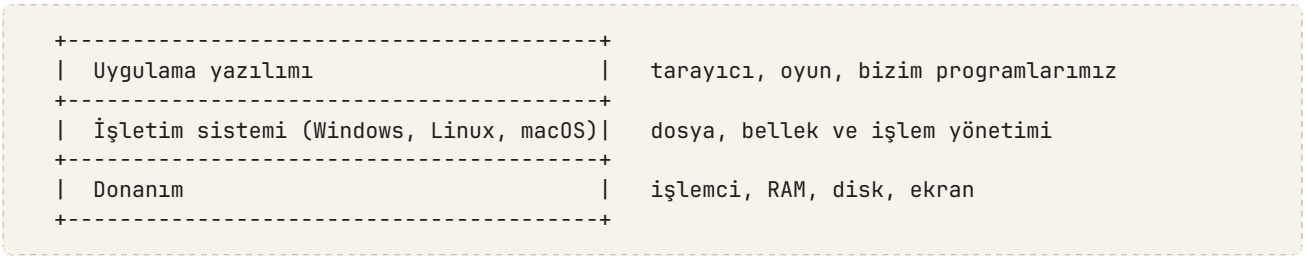
İşlemci bu dört parçanın içinde en çok işi yapanıdır, ama tek başına hiçbir şey saklamaz. Her işlem için verinin bellekte olması gerekir. İşlemcinin içinde yalnızca **yazmaç** (register) denen çok az sayıda, çok hızlı küçük alan bulunur; asıl çalışma alanı RAM'dir.

Bu parçalar arasındaki hız farkı büyüktür. İşlemci saniyede milyarlarca komut yürütebilir; RAM bu hıza yetişmeye çalışır; depolama ise ona göre çok yavaştır. Bu yüzden program önce depolamadan RAM'e yüklenir, işlemci de verisini oradan alır. İşlemcinin içinde ayrıca RAM'den daha hızlı, küçük bir **önbellek** (cache) bulunur ve sık kullanılan verileri tutar; ama bu ayrıntıyı yönetmek donanımın işidir, programcının değil.

Modern işlemciler birden çok **çekirdek** (core) içerir. Her çekirdek bağımsız bir komut akışı yürütebilir, yani aynı anda birden çok iş yapılabilir. Kitabın temel bölümlerinde programlarımız tek bir komut akışıyla çalışacak; çok çekirdekli programlama ileri konulardandır.

Yazılım katmanları

Donanımla doğrudan konuşmak zordur. Bu yüzden yazılım katmanlar hâlinde düzenlenir:



İşletim sistemi (operating system), donanımı yönetir ve programlara ortak hizmetler sunar: dosya açmak, ekrana yazmak, bellek ayırmak gibi. Bizim yazacağımız programlar en üstteki katmandadır ve bu hizmetleri işletim sistemi aracılığıyla alır. Aynı kaynak kodun Windows'ta ve Linux'ta farklı komutlarla derlenmesinin nedeni de budur; bunu 3. Bölümde göreceğiz.

Bir program bellekte nasıl çalışır?

Diskte duran bir program yalnızca bir dosyadır. Çalıştırdığımızda işletim sistemi dosyanın içeriğini RAM'e yükler ve işlemciye "komutları buradan okumaya başla" der. Programın komutları da verileri de aynı bellekte durur; bilgisayarın bu temel düzenine **depolanan program** fikri denir ve bugünkü bilgisayarların neredeyse tamamı onu kullanır.

İşlemci sonsuz bir döngüde şu üç adımı tekrarlar:

1. **Getir** (fetch): sıradaki komutu bellekten okur.
2. **Çöz** (decode): komutun ne yapmak istediğini anlar.

3. **Yürüt** (execute): komutu uygular; sonucu yazmaca ya da belleğe yazar.

Bir sonraki komuta geçilir ve döngü yeniden başlar. Bu döngü saniyede milyarlarca kez döner. Komutlar çok basittir: iki sayıyı topla, bir değeri bellekten yazmaca taşı, iki sayıyı karşılaştır, belirli bir komuta atla. Karmaşık görünen her program, bu basit komutların çok sayıda ve doğru sırayla dizilmesinden oluşur.

Çalışan bir programın belleği kabaca iki bölümden oluşur: komutların durduğu bölge ve verilerin durduğu bölge. Basit bir çizimle:

```
      RAM
+-------------------+
|  program komutları|  <--- işlemci buradan okur (getir-çöz-yürüt)
+-------------------+
|  veriler          |  <--- değişkenlerin değerleri burada durur
|  (sayılar, metin) |
+-------------------+
```

Bellek, art arda dizilmiş sıra numaralı küçük gözlerden oluşur. Her gözün numarasına **adres** denir. İşlemci bir veriye ihtiyaç duyduğunda "şu adresteki değeri getir" der. Programlama dillerindeki **değişken** kavramı, bu adresleri insanın anlayacağı adlarla anmanın yoludur; bunu 5. Bölümde ayrıntılı işleyeceğiz.

> ◆ **Kritik Nokta · İşlemci yalnızca makine dilini anlar**
>
> İşlemci, kendi komut kümesindeki ikilik sayı kalıplarından başka hiçbir şeyi çalıştıramaz. Bizim yazdığımız `printf("Merhaba");` gibi bir satır, çevrilmeden hiçbir işlemcinin işine yaramaz. Kitabın büyük bölümü, bu çeviri düzeneğinin nasıl işlediğiyle ilgilidir.

1.2 Bit, Bayt ve Sayı Sistemleri

Bilgisayar her şeyi iki durumlu devrelerle temsil eder: akım var ya da yok, gerilim yüksek ya da düşük. Bu iki durumu 0 ve 1 rakamlarıyla yazarız. Metin, fotoğraf, müzik, program; bilgisayarın içinde hepsi 0 ve 1 dizisidir.

> **Tanım: Bit ve bayt**
>
> **Bit** (binary digit), 0 ya da 1 değerini tutan en küçük bilgi birimidir. **Bayt** (byte), 8 bitin oluşturduğu gruptur ve belleğin adreslenebilen en küçük birimidir.

`n` bit, 2^n farklı kalıp üretir. Bu çok önemli bir sonuçtur, çünkü bir tipin saklayabileceği değer sayısını belirler:

Bit sayısı	Farklı değer sayısı	Örnek
1	2	0, 1
4	16	tek bir onaltılık rakam
8 (1 bayt)	256	0 … 255
16 (2 bayt)	65.536	0 … 65.535
32 (4 bayt)	4.294.967.296	yaklaşık 4,3 milyar
64 (8 bayt)	yaklaşık $1{,}8 \times 10^{19}$	çok büyük sayılar

Bellek ve depolama miktarları bayt cinsinden ölçülür. Günlük kullanımda 1 KB yaklaşık bin bayt, 1 MB yaklaşık bir milyon bayt, 1 GB yaklaşık bir milyar bayttır. Bellek üreticileri ve işletim sistemleri bu önekleri bazen 1000,

bazen 1024 katı olarak kullanır; kesin ayrım gerektiğinde 1024 katları için KiB, MiB, GiB adları kullanılır. Bizim için sıra büyüklüğünü bilmek yeterlidir.

İkilik sistem

Günlük hayatta **onluk** (decimal) sistemi kullanırız: on rakam (0–9) vardır ve her basamak bir önceki basamağın on katıdır. 352 sayısı şu anlama gelir: 3×100 + 5×10 + 2×1.

İkilik (binary) sistem aynı mantıkla çalışır; yalnızca iki rakamı (0 ve 1) vardır ve her basamak bir öncekinin iki katıdır. Sağdan sola basamak değerleri 1, 2, 4, 8, 16, 32, 64, 128… biçimindedir:

Basamak değeri	128	64	32	16	8	4	2	1
Bit	0	0	1	0	1	1	0	1

Bu tabloda 1 olan basamakların değerlerini toplarız: 32 + 8 + 4 + 1 = **45**. Yani ikilik `00101101`, onluk 45'tir. Başa eklenen sıfırlar değeri değiştirmez; `101101` de aynı sayıdır.

Onlukten ikiliğe çevirmek için sayıyı sürekli 2'ye böler, kalanları yazar ve sondan başa doğru okuruz. 45 için:

```
45 / 2 = 22  kalan 1
22 / 2 = 11  kalan 0
11 / 2 =  5  kalan 1
 5 / 2 =  2  kalan 1
 2 / 2 =  1  kalan 0
 1 / 2 =  0  kalan 1

Kalanları aşağıdan yukarı oku: 1 0 1 1 0 1  ->  101101
```

Sonucu doğrulayalım: 32 + 8 + 4 + 1 = 45. İkilik sayılar da onluk sayılar gibi toplanır; fark, 1 + 1 işleminin 10 (iki) vermesi ve elde taşımasıdır:

```
  0110   (6)
+ 0011   (3)
------
  1001   (9)
```

Onaltılık sistem

İkilik sayılar makine için uygundur, ama insan için uzun ve okunması zordur: 255 sayısı `11111111` yazılır. **Onaltılık** (hexadecimal) sistem bu sorunu çözer. 16 rakamı vardır: 0–9 ve ardından A, B, C, D, E, F harfleri (A = 10, … F = 15).

Dört bit tam 16 farklı değer taşıdığı için bir onaltılık rakam, tam dört bite karşılık gelir. Dönüşüm bu yüzden kolaydır: ikilik sayıyı sağdan başlayarak dörderli gruplarız, her grubu bir onaltılık rakama çevirirız.

Onluk	İkilik	Onaltılık	Onluk	İkilik	Onaltılık
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C

Onluk	İkilik	Onaltılık	Onluk	İkilik	Onaltılık
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Örnek: `1011 0111` ikiliğini çevirelim. `1011` = B, `0111` = 7 olduğundan sonuç `B7`'dir. Onluğa çevirmek için her rakamı 16'nın kuvvetiyle çarparız: B7 = 11 × 16 + 7 = **183**. Ters yönde, `D6A` sayısını ikiliğe çevirmek için her rakamı dört bitle yazarız: D = 1101, 6 = 0110, A = 1010; sonuç `1101 0110 1010`.

Onlukten onaltılığa çevirmek için 16'ya bölme yöntemi kullanılır. 156 sayısı için: 156 / 16 = 9, kalan 12 (C). Sonuç `9C`'dir. Doğrulama: 9 × 16 + 12 = 156.

C'de onaltılık sayılar başına `0x` konarak yazılır (`0xB7`). Bu yazımı 5. Bölümde sabitleri anlatırken kullanacağız. Onaltılık sayılar bellek adreslerinde, renk kodlarında ve bit düzeyinde işlemlerde sürekli karşımıza çıkar.

Sayıların dışındaki veriler

Bilgisayar yalnızca sayı saklayabilir; metin, renk ya da ses de sayıya çevrilerek saklanır. Karakterler için bir **karakter tablosu** kullanılır. Bunun en eski ve en yalın örneği ASCII'dir: büyük `A` harfi 65, küçük `a` harfi 97, `0` rakamı karakter olarak 48 numarayla tutulur. Yani `A` harfi bellekte `01000001` bit kalıbıdır. Aynı bit kalıbı, hangi tipin gözlüğüyle bakıldığına göre 65 sayısı ya da `A` harfi olarak yorumlanır. **Tip** kavramının özü budur: bit kalıpları kendiliğinden anlam taşımaz, anlamı yorumlama verir. Türkçe karakterleri de kapsayan UTF-8 gibi tablolar 11. Bölümde yeniden karşımıza çıkacak; ASCII tablosunun tamamı kitabın ekindedir.

> **▲ Sık Yapılan Hata · Sayı ile rakam karakteri aynı şey değildir**
>
> Bellekteki `'7'` karakteri 55 numaralı bit kalıbıdır; 7 sayısı ise `00000111`'dir. Bir programın "7" yazısını okuyup onunla hesap yapabilmesi için önce bu çeviriyi yapması gerekir. Klavyeden okunan girdilerde bu ayrım sürekli işimize çıkacak.

1.3 Algoritma

Bir programı yazmadan önce yapılacak işin adımlarını düşünmek gerekir. Bu adımların kendisine **algoritma** denir.

> **Tanım: Algoritma**
>
> Bir problemi çözmek için izlenen, sonlu sayıda, açık ve sıralı adımlardan oluşan işlem dizisidir. Her adım belirsizliğe yer bırakmayacak kadar net olmalı ve dizi sonunda bir sonuç üretmelidir.

Algoritma yalnızca bilgisayara özgü değildir. Çay demlemek, bir yemek tarifi ya da kütüphanede kitap bulma yöntemi de algoritmadır:

```
Çay demleme algoritması
1. Su ısıtıcısını suyla doldur ve çalıştır.
2. Su kaynayınca çaydanlığa boşalt.
```

```
3. Demliğe çay koy ve üzerine sıcak su ekle.
4. Beş dakika bekle.
5. Bardağa önce demlikten, sonra çaydanlıktan koy.
```

İyi bir algoritmanın dört özelliği vardır:

- **Belirli**: Her adım tek bir anlam taşır; "biraz bekle" yerine "beş dakika bekle" denir.
- **Sonlu**: Belirli bir adım sayısından sonra biter. Bitmeyen bir işlem algoritma değildir.
- **Girdi ve çıktısı vardır**: Dışarıdan veri alabilir ve bir sonuç verir.
- **Etkin**: Her adım gerçekten yapılabilir bir işlemdir.

Aynı problemi çözen birden çok algoritma olabilir. Birinin diğerinden daha az adımda sonuç vermesi mümkündür; bu fark veri büyüdükçe önem kazanır ve ileri bölümlerde (arama ve sıralama algoritmalarında) karşımıza çıkacak.

> ◆ **Kritik Nokta · Önce çözüm, sonra kod**
>
> Programcılığın en sık yapılan hatası, problemi anlamadan klavyenin başına geçmektir. Önce problemi küçük parçalara böleriz, adımları kâğıt üzerinde yazarız, birkaç sayıyla elle deneriz; ancak sonra kodlarız. Dil ne olursa olsun yöntem aynıdır. Bu yüzden algoritma düşüncesi, C, C++ ve C# için ortak temeldir.

Sözde kod ve akış şeması

Algoritmayı bilgisayar diline geçirmeden önce iki araçla ifade ederiz.

Sözde kod (pseudocode), gündelik dil ile programlama dilinin arasında bir yazımdır. Belirli bir dilin kurallarına bağlı değildir; amaç adımları sade ve açık yazmaktır. Bu kitapta şu biçimi kullanacağız: değerleri `←` işaretiyle atar, koşulları `EĞER ... İSE`, tekrarları `TEKRARLA` ile yazarız.

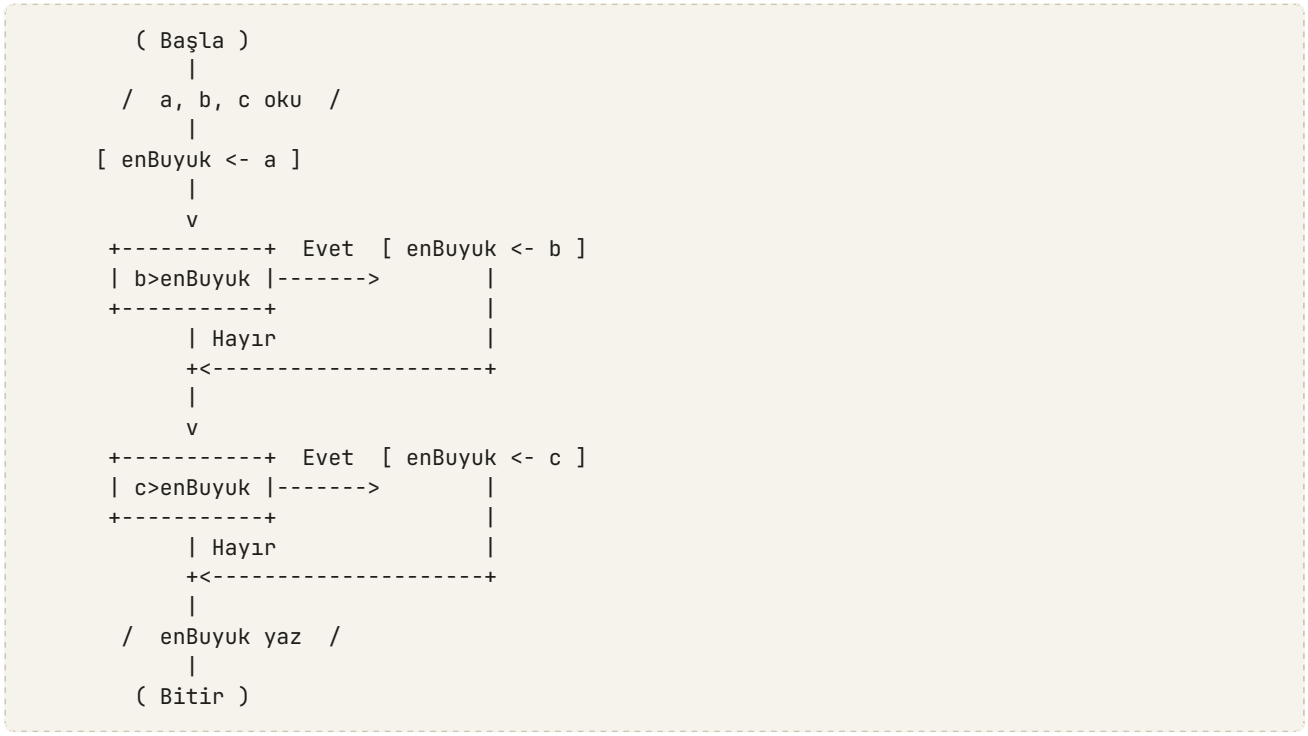
Akış şeması (flowchart), algoritmayı şekillerle çizer. Başlangıç ve bitiş yuvarlatılmış dikdörtgenle, işlem dikdörtgenle, giriş-çıkış paralelkenarla, karar ise eşkenar dörtgenle gösterilir. Oklar akışın yönünü verir. Bu kitapta çizimleri metinle yapacağız, ama kâğıt üzerinde şekilleri çizmek de aynı işi görür.

Algoritma 1: Üç sayıdan en büyüğünü bulmak

Klavyeden üç sayı geldiğini ve en büyüğünü bulmak istediğimizi düşünelim. Fikir şudur: ilk sayıyı geçici olarak "en büyük" sayalım, sonra diğerlerini sırayla bununla karşılaştıralım; daha büyük bir sayı görürsek en büyüğü güncelleyelim.

```
BAŞLA
  OKU a, b, c
  enBuyuk ← a
  EĞER b > enBuyuk İSE
      enBuyuk ← b
  EĞER c > enBuyuk İSE
      enBuyuk ← c
  YAZ enBuyuk
BİTİR
```

Akış şeması şöyle çizilir:



Algoritmayı elle çalıştırmaya **izleme** (trace) diyoruz. Sayılar 4, 9, 6 olsun:

Adım	a	b	c	enBuyuk	Açıklama
Başlangıç	4	9	6	4	`enBuyuk ← a`
`b > enBuyuk`	4	9	6	9	9 > 4 doğru, güncellenir
`c > enBuyuk`	4	9	6	9	6 > 9 yanlış, değişmez

Sonuç 9'dur. İzleme tablosu hata ayıklamanın da en temel yöntemidir: algoritmanın her adımında değişkenlerin değerini yazarak ilerleriz.

Algoritma 2: Bir listedeki en büyük sayı

Aynı fikir sayıların sayısına bağlı değildir. Liste kaç elemanlı olursa olsun, en büyüğü bulmak için tek yol listeyi baştan sona gezmektir. Bu kez bir **tekrar** yapısına ihtiyacımız var:

```
BAŞLA
  OKU n, sayilar[1..n]
  enBuyuk ← sayilar[1]
  i ← 2
  i <= n OLDUKÇA TEKRARLA
      EĞER sayilar[i] > enBuyuk İSE
          enBuyuk ← sayilar[i]
      i ← i + 1
  YAZ enBuyuk
BİTİR
```

Liste 7, 3, 9, 4 olsun (n = 4):

i	sayilar[i]	enBuyuk (sonra)
(başlangıç)		7

i	sayilar[i]	enBuyuk (sonra)
2	3	7
3	9	9
4	4	9

`i` 5 olunca `i <= n` koşulu bozulur ve döngü biter. Sonuç 9'dur. Bu algoritmayı 8. Bölümde `for` döngüsüyle, 10. Bölümde dizilerle C'ye çevireceğiz.

Algoritma 3: 1'den n'e kadar toplam

Bu kez bir **toplayıcı** (accumulator) kullanacağız: başlangıçta sıfır olan, her adımda yeni bir değer eklenen değişken.

```
BAŞLA
  OKU n
  toplam ← 0
  sayac ← 1
  sayac <= n OLDUKÇA TEKRARLA
      toplam ← toplam + sayac
      sayac ← sayac + 1
  YAZ toplam
BİTİR
```

n = 5 için izleme:

Tur	sayac (eklemeden önce)	toplam (eklemeden sonra)
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15

Beşinci turdan sonra `sayac` 6 olur, koşul bozulur ve döngü biter; toplam 15'tir. `toplam ← toplam + sayac` satırı matematikte anlamsızdır, ama bir **atamadır**: sağ taraf hesaplanır, sonuç sol taraftaki değişkene yazılır. C'de aynı kalıbı `toplam = toplam + sayac;` biçiminde göreceğiz.

Algoritma 4: Ortalama

Ortalama, toplamın eleman sayısına bölünmesidir. Önceki algoritmayı küçük bir değişiklikle kullanabiliriz; bu tür "bildiğimiz parçaları birleştirme" programcılığın günlük işidir.

```
BAŞLA
  OKU n
  toplam ← 0
  i ← 1
  i <= n OLDUKÇA TEKRARLA
      OKU sayi
      toplam ← toplam + sayi
      i ← i + 1
```

```
    ortalama ← toplam / n
    YAZ ortalama
  BİTİR
```

Girdiler 8, 6, 10, 4 ise (n = 4), toplam 28 olur ve ortalama 28 / 4 = 7'dir. Burada akla bir soru gelmeli: n sıfır girilirse ne olur? Sıfıra bölme tanımsızdır. İyi bir algoritma bu durumu da düşünür ve bölmeden önce `EĞER n > 0 İSE` denetimi koyar. Hata türlerini anlatırken bu duruma geri döneceğiz.

> **İpucu · Her algoritmayı iki üç örnekle deneyin**
>
> Bir algoritmayı yazdıktan sonra en az üç girdiyle elle çalıştırmak iyi bir alışkanlıktır: sıradan bir değer, bir uç değer (en küçük/en büyük) ve bir özel durum (sıfır, boş liste, eşit sayılar). Çoğu mantık hatası bu denemede kodlanmadan yakalanır.

1.4 Makine Dili, Assembly ve Yüksek Seviyeli Diller

Algoritmayı kâğıtta tasarladık. Şimdi onu işlemcinin çalıştırabileceği hâle getirmemiz gerekiyor. Programlama dilleri, bu yolculuğun farklı noktalarında durur ve **seviyeleriyle** sınıflandırılır: bir dil makineye ne kadar yakınsa o kadar düşük seviyelidir.

Makine dili

İşlemcinin doğrudan anladığı tek dil **makine dilidir** (machine language). Komutlar ikilik sayılardan oluşur; hangi bit kalıbının hangi işlemi yaptığını işlemcinin üreticisi belirler. Her işlemci ailesinin kendi komut kümesi vardır; bir ailenin makine kodu başka bir ailede çalışmaz.

x86-64 işlemcilerde (çoğu masaüstü ve dizüstü bilgisayarda) iki sayıyı toplayan küçük bir kod parçası, onaltılık yazımla şöyle görünür:

```
89 F8        ; eax yazmacına edi yazmacındaki değeri kopyala
01 F0        ; eax'e esi yazmacındaki değeri ekle
C3           ; fonksiyondan dön
```

Bu baytları ezberlemek ya da elle yazmak, tek bir hatanın tüm programı bozduğu çok yorucu bir iştir. İlk bilgisayarlar böyle programlanıyordu.

Assembly

Bir sonraki adım, her makine komutuna insanın okuyabileceği kısa bir ad vermektir. **Assembly dili** bunu yapar: `89 F8` yerine `mov eax, edi` yazarız. Assembly'yi makine koduna çeviren programa **çevirici** (assembler) denir.

```
mov  eax, edi     ; eax <- edi
add  eax, esi     ; eax <- eax + esi
ret               ; geri dön
```

Assembly okunabilir olsa da hâlâ işlemciye bağlıdır: her komut tek bir makine komutuna karşılık gelir, bir işlemci ailesi için yazılan kod diğerinde çalışmaz ve en basit iş bile onlarca satır tutar. Assembly bugün de kullanılır; ancak yalnızca çok küçük ve performans ya da donanım denetimi açısından kritik parçalarda (işletim sistemlerinin başlatma kodu, sürücüler, bazı gömülü sistem rutinleri).

Yüksek seviyeli diller

Yüksek seviyeli dillerde (high-level language) aynı işi insanın düşünme biçimine yakın yazarız. İki sayının toplamını döndüren bir fonksiyon C'de şöyledir:

```c
int topla(int a, int b) {
    return a + b;
}
```

Bu kodda yazmaç adı, bellek adresi, işlemci komutu yoktur. Yalnızca ne yapmak istediğimizi söyleriz; nasıl yapılacağına derleyici karar verir. Aynı kaynak kod, derleyici hedefi değiştirilerek farklı işlemci aileleri için makine koduna çevrilebilir. Yüksek seviyeli dillerin üç büyük avantajı şunlardır:

- **Okunabilirlik**: Kod, algoritmanın sözde kodundan çok uzak değildir.
- **Taşınabilirlik**: Aynı kaynak kod farklı işlemcilerde ve işletim sistemlerinde yeniden derlenebilir.
- **Verimlilik**: Programcı, işlemciyi değil problemi düşünür; aynı iş çok daha az satırla yazılır.

C, C++ ve C# bu gruba girer. Yine de dillerin "seviyesi" kesin bir çizgi değildir. C, yüksek seviyeli olmasına karşın belleğe ve donanıma çok yakın çalışmaya izin verdiği için sık sık "orta seviyeli" diye anılır. C#'ta bellek yönetiminin büyük kısmı çalışma ortamına bırakılmıştır; bu yüzden daha "yukarıdadır". Bu farkı kitap boyunca birçok yerde göreceğiz.

> **Tanım: Kaynak kod**
>
> Programcının yüksek seviyeli bir dille yazdığı, düz metin dosyasında duran program metnidir (source code). C için dosya uzantısı `.c`, C++ için `.cpp`, C# için `.cs` 'dir.

1.5 Derleyici, Yorumlayıcı ve Sanal Makine

Kaynak kod, işlemci tarafından doğrudan çalıştırılamaz; bir çeviri gerekir. Çeviri iki klasik yöntemle yapılır.

Derleyici

Derleyici (compiler), kaynak kodun tamamını çalıştırmadan önce okur ve hedef işlemcinin makine koduna çevirir. Sonuç, ayrı bir **çalıştırılabilir dosyadır** (executable; Windows'ta `.exe`). Program çalıştırılırken artık kaynak koda ya da derleyiciye ihtiyaç yoktur.

```
merhaba.c  --->  [ Derleyici ]  --->  merhaba (çalıştırılabilir dosya)  --->  işlemci
(kaynak)                              (makine kodu)
```

Derlemenin avantajları şunlardır: çeviri çalışma öncesinde bir kez yapıldığı için program hızlıdır, dilin sözdizimi hataları program çalışmadan yakalanır ve program derleyicisiz bir bilgisayara dağıtılabilir. Dezavantajı, çıkan dosyanın derlendiği işlemciye ve işletim sistemine özgü olmasıdır: Windows için derlenen program Linux'ta çalışmaz; kaynak kod ise o platformda yeniden derlenerek çalıştırılabilir.

Yorumlayıcı

Yorumlayıcı (interpreter), kaynak kodu çalışırken satır satır (daha doğrusu deyim deyim) okur ve o anda yürütür. Ayrı bir çalıştırılabilir dosya üretmez; kod her çalıştırıldığında yeniden yorumlanır. Klasik örnek olarak

Python gösterilir (bugünkü Python gerçekte kaynak kodu önce kendine özgü ara koda çevirir; ama en basit anlatımda bir yorumlayıcıdır).

Yorumlayıcıda yazılan kod hemen çalıştırılabilir, taşınması kolaydır ve denemek için pratiktir. Karşılığında çeviri işi her çalıştırmada tekrarlandığı için çoğunlukla daha yavaştır; hata da yalnızca o satıra gelindiğinde ortaya çıkar.

Özellik	Derleyici	Yorumlayıcı
Çalışma biçimi	Programın tamamını önceden çevirir	Kodu çalışırken okur ve yürütür
Çıktı	Ayrı çalıştırılabilir dosya	Ayrı dosya yok
Hata yakalama	Sözdizimi hataları çalışmadan önce	Hataya sıra gelince
Hız	Genellikle yüksek	Genellikle daha düşük
Örnek diller	C, C++	Python (basitleştirilmiş anlatımda)

Ara kod ve sanal makine

İki yöntemin arasında bir yol daha vardır. Kaynak kod önce **ara koda** (intermediate code) çevrilir; bu kod belirli bir işlemciye değil, hayali bir makineye göre yazılmıştır. Ara kodu çalıştıran program **sanal makinedir** (virtual machine). Sanal makine ara kodu ya yorumlar ya da çalışma sırasında işlemcinin makine koduna çevirir. İkinci yönteme **JIT** (just-in-time, tam zamanında) derleme denir.

C#'ta işleyiş böyledir:

```
Merhaba.cs --> [ C# derleyicisi ] --> Merhaba.dll (IL, ara kod)
                                            |
                     program çalışırken     v
                             [ CLR + JIT derleyicisi ] --> makine kodu --> işlemci
```

C# derleyicisi kaynak kodu **IL**'e (intermediate language, ara dil) çevirir. Program çalıştırılınca **CLR** (Common Language Runtime, ortak dil çalışma zamanı) devreye girer ve JIT derleyicisi, her metodu ilk kez çağrıldığında IL'den o bilgisayarın makine koduna çevirir. Bunu 41. Bölümde ayrıntısıyla göreceğiz. Java da benzer bir düzen kullanır: ara kodu Java sanal makinesi çalıştırır.

Bu yöntemin kazancı taşınabilirliktir: aynı ara kod, sanal makinenin bulunduğu her platformda çalışır. Bedeli ise sanal makinenin kurulu olması gereksinimidir ve çalışmanın başlangıcında JIT çevirisinin bir miktar zaman almasıdır. Günümüzde .NET, ara kodu önceden makine koduna çeviren seçenekler de sunar; bu ayrıntı ileri bölümlere kalır.

Üç dilin bu bakımdan konumunu özetleyelim:

Dil	Çeviri yolu	Sonuç
C	Derleyici	Doğrudan makine kodu (platforma özgü çalıştırılabilir dosya)
C++	Derleyici	Doğrudan makine kodu (platforma özgü çalıştırılabilir dosya)
C#	Derleyici + JIT	Önce IL, çalışırken makine kodu

◆ Kritik Nokta · Derleyici, bir dili "hızlı" ya da "yavaş" yapan şeyin tek başına kendisi değildir

Derlenen dillerin çoğu zaman hızlı olması, makine kodunun çalışmadan önce üretilmesinden ve bellek üzerindeki doğrudan denetimden gelir. Ama dilin hızı, programın yazılış biçimine ve derleyicinin ayarlarına da bağlıdır. "Derlenen dil hızlı, yorumlanan dil yavaştır" cümlesi bir eğilimi anlatır, kesin bir kural değildir.

Derleme aşamaları

"Derlemek" sözcüğü günlük dilde iki ayrı şeyi karıştırır: dar anlamda derleyicinin yaptığı çeviri ve kaynak dosyadan çalıştırılabilir dosyaya giden bütün yol. C ve C++'ta bu yol aşamalıdır: önişleme, derleme, assembly'ye çeviri ve bağlama (linking). Her aşamanın ne yaptığını 4. Bölümde, ilk C programımızı derlerken göreceğiz. Şimdilik şunu bilmemiz yeterli: derleyici hataları bulursa çalıştırılabilir dosya üretilmez.

1.6 Programlama Paradigmaları

Bir programı yazmanın tek bir yolu yoktur. Problemi nasıl parçalayacağımızı ve kodu nasıl düzenleyeceğimizi belirleyen yaklaşıma **programlama paradigması** denir. Üç büyük yaklaşım vardır.

Yordamsal programlama

Yordamsal (procedural) programlamada program, sırayla çalışan komutlardan ve bu komutları gruplayan **fonksiyonlardan** (yordamlardan) oluşur. Veri ayrıdır, veri üzerinde işlem yapan fonksiyonlar ayrıdır. Program "ilk önce şunu yap, sonra bunu yap" diye ilerler. C, bu yaklaşımın en bilinen dillerinden biridir.

Nesne yönelimli programlama

Nesne yönelimli programlamada (OOP, object-oriented programming) veri ve o veriyi işleyen fonksiyonlar tek bir **nesnede** birleştirilir. Program, birbiriyle mesajlaşan nesnelerden oluşur. Banka hesabı bir nesnedir: bakiyesi vardır (veri), para yatırabilir ve çekebilir (davranış). Nesnelerin şablonu **sınıftır** (class). Büyük programları düzenlemeyi kolaylaştırdığı için yaygındır. C++ ve C# bu yaklaşımı kuran dillerdendir; C++ ve C# kısımlarında (Bölüm 30–31 ve 49–51) ayrıntılı işleyeceğiz.

Fonksiyonel programlama

Fonksiyonel programlamada hesaplamalar, matematikteki gibi girdisinden çıktı üreten fonksiyonların birbirine bağlanmasıyla yapılır. Değişkenleri sürekli değiştirmek yerine, yeni değerler üreten fonksiyonlar kullanılır. Fonksiyonlar da başka fonksiyonlara parametre olarak verilebilir. C++ (C++11'den itibaren lambda ifadeleri ve standart algoritmalar) ve C# (lambda ifadeleri ve LINQ) bu yaklaşımın bazı özelliklerini benimsemiştir.

Üç Dilde · Aynı iş, üç yaklaşım *C · C++ · C#*

Dört sayının toplamını bulalım. Yordamsal yaklaşımda döngü ve toplayıcı değişken kullanırız; fonksiyonel yaklaşımda ise hazır bir fonksiyona listeyi veririz.

C (yordamsal):

```c
int sayilar[] = {7, 3, 9, 4};
int toplam = 0;
for (int i = 0; i < 4; i++) {
    toplam += sayilar[i];
}
```

C++ (standart algoritma, `<numeric>` başlığı ile):

```cpp
std::vector<int> sayilar = {7, 3, 9, 4};
int toplam = std::accumulate(sayilar.begin(), sayilar.end(), 0);
```

C# (LINQ ile):

```csharp
int[] sayilar = { 7, 3, 9, 4 };
int toplam = sayilar.Sum();
```

Üç kod da 23 sonucunu verir. Bu kodların hiçbirini şimdi anlamamız gerekmiyor; yalnızca dillerin farklı yaklaşımlara nasıl yer verdiğini görmek yeterli.

Nesne yönelimli yaklaşımda ise veriyi işleyen kodu veriyle birlikte tutarız:

```
C   : struct Daire { double yaricap; };   +  ayrı fonksiyon  alan(daire)
C++ : class Daire  { double yaricap; double alan() ... };      daire.alan()
C#  : class Daire  { double Yaricap;  double Alan() ... };     daire.Alan()
```

C'de veri (`struct`) ve onu işleyen fonksiyon ayrı durur. C++ ve C#'ta fonksiyon sınıfın içine girer ve nesne üzerinden çağrılır.

Üç dil nereye düşüyor?

Dil	Ağırlıklı yaklaşım	Notlar
C	Yordamsal	Sınıf yoktur; düzen fonksiyonlar ve `struct`'larla kurulur
C++	Çok paradigmalı	Yordamsal kod, nesne yönelimli kod, şablonlarla genel programlama ve fonksiyonel öğeler bir arada kullanılabilir
C#	Ağırlıklı nesne yönelimli	Fonksiyonel öğeler (lambda, LINQ) de güçlü biçimde desteklenir

Hiçbir paradigma diğerinden üstün değildir; her biri bazı problemlere daha uygundur. Kitap, aynı kavramları üç dilde yan yana görmenizi sağlayarak bu yaklaşımların her birini kullanma imkânı verir.

1.7 Programlama Hataları

Hata yapmak, programcılığın doğal bir parçasıdır. Önemli olan hatanın türünü tanımak ve nerede arayacağını bilmektir. Hatalar üç sınıfa ayrılır.

Sözdizimi hataları

Sözdizimi hatası (syntax error), dilin yazım kurallarının bozulmasıdır: noktalı virgülü unutmak, parantezi kapatmamak, bir sözcüğü yanlış yazmak. Derleyici bu hataları program çalışmadan önce yakalar ve bildirir; hata varsa çalıştırılabilir dosya üretilmez.

▸ hataNoktaliVirgul.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba")
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
hataNoktaliVirgul.c: In function 'main':
hataNoktaliVirgul.c:4:22: error: expected ';' before 'return'
    4 |     printf("Merhaba")
      |                      ^
      |                      ;
    5 |     return 0;
      |     ~~~~~~
```

Derleyici hatanın dosya adını, satır ve sütun numarasını verir ve ne beklediğini söyler. Burada 4. satırın sonunda `;` beklenmişti. Derleyici bazen hatayı bir satır sonra fark eder, çünkü eksik noktalı virgülü ancak sonraki sözcüğü görünce anlar. Bu yüzden bir hata mesajında gösterilen satırın hemen öncesine de bakmak gerekir. Birden çok hata varsa ilk hatadan başlayıp düzeltmek ve yeniden derlemek en doğru yoldur; sonraki mesajların çoğu ilk hatanın sonucudur.

Çalışma zamanı hataları

Program derlenir ve başlar, ancak çalışırken bir işlemi yapamaz ve durur. Buna **çalışma zamanı hatası** (runtime error) denir. Tipik nedenler: sıfıra bölme, var olmayan bir dosyayı açmak, geçersiz bir bellek adresine erişmek.

Daha önceki ortalama algoritmasında `n` sıfır olursa, programın C'ye çevrilmiş hâli şöyle bir sonuç verebilir:

TERMİNAL

```
$ ./ortalama
Floating point exception (core dumped)
```

Mesajın sözcükleri sisteme göre değişir ve "kayan nokta" ifadesine bakıp yanılmamak gerekir: tamsayı bölmesindeki sıfıra bölme de bu mesajla sonlanabilir. Önemli olan, derleyicinin bu hatayı önceden göstermemiş olmasıdır; çünkü `n`'in değeri ancak program çalışırken bilinir.

Mantık hataları

Mantık hatası (logic error), programın sorunsuz derlenip çalıştığı, ama yanlış sonuç ürettiği durumdur. Derleyici de çalışma ortamı da bir şey söylemez; program yazılanı eksiksiz yapar, ancak yazılan istenen şey değildir. Üç sayıdan en büyüğünü bulan algoritmamızda `b > enBuyuk` yerine `b < enBuyuk` yazsak, program çalışır ama en küçüğü bulurdu.

Mantık hatalarını bulmak en zor iştir. Bu yüzden şu alışkanlıklar işe yarar:

- Programı yazmadan önce algoritmayı kâğıtta tasarlamak ve izleme tablosuyla denemek,
- Programı sonucu önceden bilinen girdilerle sınamak,
- Ara değerleri ekrana yazdırmak ya da hata ayıklayıcı (debugger) kullanmak. Hata ayıklayıcıyı 3. Bölümde tanıyacağız.

Hata türü	Ne zaman ortaya çıkar?	Kim bildirir?	Örnek
Sözdizimi	Derlerken	Derleyici	Eksik `;`, kapatılmamış parantez
Çalışma zamanı	Program çalışırken	İşletim sistemi / çalışma ortamı	Sıfıra bölme, geçersiz bellek erişimi
Mantık	Sonuca bakınca	Kimse; programcı fark eder	Yanlış formül, ters koşul

Derleyiciler bunlara ek olarak **uyarı** (warning) verir. Uyarı, kodun derlenebildiğini ama şüpheli bir yer içerdiğini söyler. Uyarıları görmezden gelmeyip düzeltmek iyi bir alışkanlıktır; 4. Bölümde derleyiciyi bu bakımdan nasıl daha titiz çalıştıracağımızı göreceğiz.

> *Tarihçe · Hataya neden "bug" deniyor?*
>
> Yazılım hatalarına İngilizcede "bug" (böcek) denir. Terim bilgisayarlardan öncesine uzanır; mühendislikte arızaları anlatmak için kullanılırdı. Bilgisayar tarihinde ise 1947'de Harvard Mark II adlı makinenin bir rölesine sıkışmış bir güvenin bulunması ünlüdür: güve bulunduğu gün tutulan kayıt defterine yapıştırılmış, yanına gerçekten bir "bug" bulunduğu yazılmıştır.

1.8 Bu Kitabın Düzeni

Yeni bir şeye başlarken kitabın nasıl kurulduğunu bilmek zaman kazandırır. Bu bölümün son başlığında kitabın izlediği yolu ve işaretlerini tanıtıyoruz.

Neden C, sonra C++, sonra C#?

Kitap üç dili **tarih sırasıyla** öğretir: C (1972), C++ (1985) ve C# (2000). Bu sıralamanın üç nedeni vardır:

- **Her dil bir öncekinin üzerine kurulmuştur.** C++ kendini C'nin devamı olarak geliştirdi ve C programlarının büyük bölümünü derleyebilir. C#'ın söz dizimi (süslü parantezler, `if`, `for`, `while`) C ve C++'tan miras kalmıştır. Tarih sırasıyla ilerlemek, her yeni özelliğin hangi eksikten doğduğunu görmemizi sağlar.
- **C, bilgisayarın nasıl çalıştığını gizlemez.** Bellek, adres ve işaretçi gibi kavramları C'de açıkça görürüz. Bunları öğrenmek, sonraki iki dilde "arka planda ne olduğunu" anlamayı kolaylaştırır.
- **Karşılaştırma öğretir.** C++ bölümleri "C'de böyleydi, C++'ta böyle" diye, C# bölümleri ise ikisini de bilen okura "farklar şunlar" diye ilerler. Aynı kavramı üç kez sıfırdan öğrenmek yerine, bildiğimizin üstüne ekleme yaparız.

Kitabın bölümleri şöyle kümelenir:

Kısım	Bölümler	İçerik
0. Başlarken	1–3	Bilgisayar ve programlama temelleri, üç dilin tarihi, geliştirme ortamının kurulumu
I. C	4–20	Değişkenlerden dinamik belleğe, dosyalardan veri yapılarına; uygulama projesi

Kısım	Bölümler	İçerik
II. C++	21–40	C'ye eklenenler: nesneler, şablonlar, STL, modern C++; uygulama projesi
III. C#	41–60	.NET dünyası, nesne yönelimli programlama, koleksiyonlar, LINQ; uygulama projesi
IV. Karşılaştırma	61–62	Üç dil yan yana, hangi dilin ne zaman seçileceği
Ekler		ASCII ve sayı sistemleri, operatör öncelikleri, anahtar sözcükler, hata rehberi, terimler sözlüğü, alıştırma cevapları

Hiç programlama bilmeyen okur kitabı baştan sona sırayla okumalıdır. Daha önce bir dil öğrenmiş ve hatırlamak isteyen okur, bölüm özetlerini ve Kritik kutularını tarayarak hızla ilerleyebilir. Yalnızca bir dil isteyen okur için de kısımlar büyük ölçüde kendi içinde bütündür; ancak C++ kısmı C bilindiğini, C# kısmı ise ikisini de bildiğini varsayar.

Örnek kodlar

Kitaptaki programlar C17 (`gcc -std=c17`), C++17 (`g++ -std=c++17`) ve .NET 8 / C# 12 ile yazılmıştır; daha yeni sürümlerin özellikleri ilgili bölümlerde ayrıca belirtilir. Tam programlar dosya adıyla verilir ve hemen ardından gerçek çıktısı gelir. Programları okumakla yetinmemek gerekir: her örneği yazıp çalıştırmak, değiştirip ne olduğuna bakmak kavramların yerleşmesinin en etkili yoludur. Geliştirme ortamının kurulumu 3. Bölümdedir.

Çıktılarda klavyeden girilen değerler `{{...}}` içinde gösterilir. Yani aşağıdaki gibi bir çıktıda `42`'yi kullanıcı yazmıştır:

```
ÇIKTI
Bir sayı girin: 42
Sayının karesi: 1764
```

Kutuların anlamı

Metnin içinde belirli işaretli kutular bulunur:

Kutu	Anlamı
Tanım	Bir terimin kısa ve kesin tanımı
Kritik	Anlaşılmadığında sonraki konuları da zorlaştıran temel nokta
Dikkat	Sık yapılan bir hata ve ondan nasıl kaçınılacağı
İpucu	Pratik öneri ya da kestirme yol
Üç Dilde	Aynı konunun C, C++ ve C#'taki karşılığı
Tarihçe	Bir özelliğin ne zaman ve neden ortaya çıktığı
Not	Genel bilgi ya da ayrıntı

Her bölüm hedeflerle başlar, bölüm özetiyle ve alıştırmalarla biter. Alıştırmaların cevapları kitabın sonundaki cevap anahtarında yer alır. Cevaba bakmadan önce soruyu çözmeye çalışmak, öğrenmenin büyük kısmını oluşturur.

Bölüm Özeti

- **Bilgisayar** dört temel işlev yürütür: işlemci komutları yürütür, RAM çalışan programı ve verisini geçici tutar, depolama kalıcı saklar, giriş-çıkış aygıtları dış dünyayla iletişim kurar.
- Program diskte dosya olarak durur; çalıştırılınca RAM'e yüklenir ve işlemci komutları sırayla **getir-çöz-yürüt** döngüsüyle işler.
- **Bit** 0 ya da 1 tutar; **bayt** 8 bittir. `n` bit 2^n farklı değer ifade eder (8 bit için 256).
- **İkilik** sistemde basamak değerleri 1, 2, 4, 8, 16… biçimindedir; onluğa çevirmek için 1 olan basamak değerleri toplanır. Onluktan ikiliğe çevirmek için sayı sürekli 2'ye bölünür, kalanlar sondan başa okunur.
- **Onaltılık** sistem (0–9, A–F) dört biti tek rakamla gösterir; ikilik sayı dörtlü gruplara ayrılarak kolayca çevrilir. C'de `0x` ile yazılır.
- Bellekteki bit kalıpları kendi başlarına anlam taşımaz; anlamı **tip** belirler. `01000001` hem 65 sayısı hem de `A` karakteri olarak yorumlanabilir.
- **Algoritma**, bir problemi çözen sonlu, belirli ve sıralı adımlardır. Sözde kod ve akış şemasıyla anlatılır; **izleme tablosu** ile elle denenir.
- **Makine dili** işlemcinin tek anladığı dildir; **assembly** ona okunabilir adlar verir; **yüksek seviyeli diller** (C, C++, C#) insanın düşünme biçimine yakındır ve farklı platformlar için yeniden derlenebilir.
- **Derleyici** programı çalışmadan önce makine koduna çevirir; **yorumlayıcı** çalışırken yürütür; **sanal makine** ara kodu çalıştırır. C ve C++ doğrudan makine koduna, C# önce IL'e, çalışırken JIT ile makine koduna çevrilir.
- **Paradigmalar:** C ağırlıklı olarak yordamsaldır; C++ çok paradigmalıdır; C# ağırlıklı nesne yönelimlidir, fonksiyonel öğeleri de güçlüdür.
- **Hata türleri:** sözdizimi hatalarını derleyici bildirir, çalışma zamanı hataları program çalışırken ortaya çıkar, mantık hatalarını yalnızca programcı fark edebilir.
- Kitap C, C++, C# sırasıyla ilerler; her dil bir öncekinin üzerine kurulduğu için tarih sırası öğrenmeyi kolaylaştırır.

Alıştırmalar

1. Aşağıdakilerden hangisi RAM için doğrudur?

   A) Elektrik kesilse de içeriğini korur.
   B) Depolama aygıtlarından daha yavaştır.
   C) Çalışan programı ve verisini geçici olarak tutar; elektrik kesilince içeriği silinir.
   D) Komutları yürüten birimdir.
   E) Yalnızca ekrana veri gönderir.

2. 8 bit ile kaç farklı değer ifade edilebilir?

   A) 8
   B) 64
   C) 128

D) 256
E) 512

3. Aşağıdaki dönüşümleri yapın.

a) İkilik `110101` sayısı onluk olarak kaçtır? b) Onluk 37 sayısının ikilik karşılığı nedir? c) Onaltılık `2F` sayısı onluk olarak kaçtır? d) İkilik `1010 1110` sayısının onaltılık karşılığı nedir? e) Onluk 200 sayısının onaltılık ve ikilik karşılığı nedir?

4. Derleyici ve yorumlayıcı hakkında hangisi doğrudur?

A) Derleyici kodu satır satır çalıştırırken çevirir.
B) Yorumlayıcı programı önceden tek seferde çevirir ve ayrı bir çalıştırılabilir dosya üretir.
C) Derleyici programı çalışmadan önce çevirir; ortaya çıkan çalıştırılabilir dosya derleyici olmayan bir bilgisayarda da çalışabilir.
D) C, yorumlayıcıyla çalışan bir dildir.
E) Yorumlayıcı hiçbir zaman hata bildirmez.

5. Bir C# programı hangi yoldan geçerek çalışır?

A) Kaynak kod doğrudan makine koduna çevrilir.
B) Kaynak kod IL'e çevrilir; program çalışırken CLR'ın JIT derleyicisi IL'i makine koduna çevirir.
C) Kaynak kod önce assembly'ye, sonra IL'e çevrilir.
D) Kaynak kod yalnızca yorumlanır, hiçbir aşamada makine koduna çevrilmez.
E) IL, kaynak koda geri çevrilir ve sonra çalıştırılır.

6. Aşağıdaki sözde kodun çıktısı nedir?

```
x ← 2
y ← 5
x ← x + y
y ← x - y
x ← x - y
YAZ x, y
```

7. Aşağıdaki sözde kod çalıştığında `toplam` değeri kaç olur?

```
toplam ← 0
sayac ← 2
sayac <= 8 OLDUKÇA TEKRARLA
    toplam ← toplam + sayac
    sayac ← sayac + 2
YAZ toplam
```

8. Aşağıdaki algoritma bir listedeki en büyük sayıyı bulmayı amaçlıyor. Girdi -5, -2, -9 olduğunda yanlış sonuç verir. Hatayı bulun ve düzeltin.

```
OKU n, sayilar[1..n]
enBuyuk ← 0
i ← 1
i <= n OLDUKÇA TEKRARLA
    EĞER sayilar[i] > enBuyuk İSE
```

```
        enBuyuk ← sayilar[i]
    i ← i + 1
YAZ enBuyuk
```

9. Aşağıdaki durumların her biri hangi hata türüne girer (sözdizimi, çalışma zamanı, mantık)?

a) Programda `printf("Merhaba"` yazılıp parantez kapatılmamıştır. b) Program çalışırken kullanıcı 0 girmiş ve program sıfıra bölme yüzünden durmuştur. c) Dairenin alanını bulmak için yarıçap iki ile çarpılmış; program sorunsuz derlenip çalışmış ama sonuç yanlıştır.

10. Hangisi doğrudur?

    A) C nesne yönelimli bir dildir ve sınıf içerir.
    B) C++ yalnızca nesne yönelimli programlamayı destekler.
    C) C ağırlıklı olarak yordamsal, C++ ise çok paradigmalı bir dildir.
    D) C# yalnızca fonksiyonel bir dildir.
    E) Yordamsal programlamada fonksiyon kullanılmaz.

11. Klavyeden okunan bir sayının çift mi tek mi olduğunu söyleyen algoritmayı sözde kodla yazın.

12. Klavyeden önce eleman sayısı `n`, sonra `n` tane sayı okuyup bunların en küçüğünü bulan algoritmayı sözde kodla yazın. Girdi 5, 2, 8, 2 için izleme tablosunu çıkarın.

13. 1'den 100'e kadar olan çift sayıların toplamını bulan algoritmayı sözde kodla yazın. Sonuç kaç olmalıdır?

BÖLÜM 2

C Ailesinin Tarihi: Bell Labs'tan .NET'e

BU BÖLÜMDE

- C'nin BCPL ve B üzerinden nasıl ortaya çıktığını, UNIX ile ilişkisini ve K&R kitabının rolünü
- C'nin standartlarını (C89/C90, C99, C11, C17, C23) ve her birinin başlıca yeniliklerini
- C++'ın "sınıflı C"den ISO standardına ve C++11 sonrası modern döneme uzanan yolunu
- Java'nın etkisini, .NET platformunun ve C# dilinin ortaya çıkışını; .NET Framework'ten .NET 10'a evrimi
- "C", "C++" ve "C#" adlarının kökenini
- Üç dilin birbirine ve diğer dillere olan akrabalığını; tipik kullanım alanlarını ve temel farklarını

Bir programlama dilinin kuralları çoğu zaman tuhaf görünür: neden C'de dizi indeksi 0'dan başlar, neden C++'ta hem `struct` hem `class` vardır, neden C# bu kadar çok sürüm geçirmiştir? Bu soruların cevabı genellikle dilin yazıldığı dönemin donanımında, çözmeye çalıştığı sorunda ve önceki dillerden devraldığı mirastadır. Tarihi bilmek, bir kuralı ezberlemek yerine nedenini anlamamızı sağlar.

Bu bölümde 1950'lerden 2020'lere uzanan bir çizgi izleyeceğiz: ilk yüksek seviyeli dillerden C'nin doğuşuna, oradan C++'a ve son olarak C# ile .NET platformuna. Kod yazmayacağız; yalnızca birkaç küçük örnekle dilin nasıl değiştiğine bakacağız. Bölümün sonunda üç dilin ortak kökünü, farklarını ve hangi iş için hangisinin seçildiğini söyleyebileceksiniz.

2.1 Öncüller: 1950'ler ve 1960'lar

İlk bilgisayarlar makine diliyle, yani doğrudan sayılarla programlanıyordu. Ardından her işlemci komutuna kısa bir ad veren **assembly** dili geldi (Bölüm 1). Assembly insan için daha okunaklıydı, ama hâlâ tek bir işlemciye bağlıydı: bir makine için yazılmış program başka bir makinede çalışmazdı. Çözüm, makineden bağımsız, matematiğe ve insan diline yakın **yüksek seviyeli diller** olarak görüldü.

1950'lerin sonunda üç dil ortaya çıktı ve sonraki on yılların yönünü belirledi:

Dil	Yıl	Amaç ve katkı
FORTRAN	1957	IBM'de John Backus'un ekibi geliştirdi. Bilimsel ve sayısal hesaplar için tasarlandı; derleyicinin elle yazılmış assembly'ye yakın hızda kod üretebileceğini gösterdi.
LISP	1958	John McCarthy'nin tasarımı. Yapay zekâ çalışmalarında kullanıldı; işlevsel programlamanın ve çöp toplamanın öncüsü oldu.
COBOL	1959	İş ve ticaret uygulamaları için, bir komite eliyle tasarlandı. İngilizceye yakın söz dizimi vardı. Grace Hopper'ın çalışmalarından etkilendi.

Bu üçü arasında C ailesinin doğrudan atası **ALGOL**'dür. ALGOL 58'in ardından 1960'ta yayımlanan ALGOL 60, uluslararası bir ekibin ortak çalışmasıydı ve programlama dillerine kalıcı fikirler kazandırdı:

- **Blok yapısı**: `begin ... end` arasına alınan deyim grupları (C ailesinde bunlar `{ ... }` olur).
- **Özyineleme** (recursion): fonksiyonun kendini çağırabilmesi.
- **Yerel değişkenler ve kapsam kuralları**.
- Dilin söz dizimini resmî olarak tanımlayan bir gösterim (Backus-Naur biçimi, BNF). Raporun yazımında Peter Naur'un büyük payı vardır.

ALGOL hiçbir zaman ticari olarak geniş çaplı kullanılmadı, ama yüzlerce sonraki dil onun fikirlerini taşıdı. Bugün `if`, `while` ve bloklarla programlama yapıyorsak bunun bir kısmını ALGOL'e borçluyuz.

> **Not · Bu dönemin ortak sorunu**
>
> 1960'ların dilleri ya bilimsel hesap, ya iş yazılımı, ya da araştırma için yazılmıştı. İşletim sistemi gibi donanıma yakın yazılımlar ise hâlâ assembly ile yazılıyordu. C'nin hikâyesi, bu boşluğu doldurma çabasıdır.

2.2 BCPL ve B: C'nin Doğrudan Ataları

ALGOL 60'ın öğrettikleri 1960'ların başında Britanya'da büyük bir dil tasarımına dönüştü: **CPL** (Combined Programming Language), Cambridge ve Londra'daki ekiplerin ortak çalışmasıydı ve Christopher Strachey'in öncülüğünde 1963 dolaylarında tasarlandı. CPL çok zengin, ama derleyicisinin yazılması zor bir dildi.

Cambridge'den Martin Richards, CPL'in yalın bir çekirdeğini 1967'de **BCPL** (başta "Basic CPL") adıyla geliştirdi. BCPL'in iki özelliği önemliydi:

- **Tipsizdi**: Tek bir veri türü vardı, bellek sözcüğü (word). Bu sözcük bir tamsayı, bir adres ya da bir karakter olarak yorumlanabilirdi.
- Derleyicisi küçüktü ve kolayca başka makinelere taşınabiliyordu.

Bell Laboratuvarları'nda (Bell Labs) çalışan Ken Thompson, 1969 civarında BCPL'den esinlenerek **B** dilini yazdı. Amaç, o sırada geliştirilmekte olan yeni işletim sistemi için küçük bir bilgisayarda (DEC PDP-7) çalışabilen ve assembly'den rahat bir dil elde etmekti. B de BCPL gibi tipsizdi: her şey makine sözcüğüydü.

```
ALGOL 60 (1960) --> CPL (~1963) --> BCPL (1967) --> B (~1969) --> C (~1972)
```

Bu ardışık soy zincirinde her halka bir öncekini sadeleştirir ya da uyarlar. C, zincirin son halkasıdır ve bu zincire ilk kez güçlü bir şey ekler: veri tipleri.

2.3 C'nin Doğuşu ve UNIX (1969–1973)

Bell Labs ve UNIX

Bell Labs, 1960'ların sonunda MIT ve General Electric ile birlikte **Multics** adlı büyük bir işletim sistemi projesine katılıyordu. Bell Labs, 1969'da projeden çekildi. Ken Thompson ve Dennis Ritchie, Multics'in iyi fikirlerinden esinlenerek çok daha küçük bir sistem yazmaya koyuldular. Bu sistem, adıyla Multics'e gönderme yapan bir söz oyunuyla **UNIX** olarak anıldı.

UNIX'in ilk sürümleri assembly ile yazıldı: önce PDP-7, sonra 1970–71'de daha güçlü olan DEC **PDP-11** için. PDP-11 iki yeni sorun çıkardı:

- Bellek **bayt adreslenebilir**di; yani tek bir karakter ile 16 bitlik bir sayı ayrı birimlerdi. Her şeyi "sözcük" sayan B burada yetersiz kalıyordu.
- Makine kayan noktalı sayıları da destekliyordu ve B bunu ifade edemiyordu.

C ortaya çıkıyor

Çözüm, B'ye **tip** eklemekti. Dennis Ritchie 1971–72 yıllarında B'yi geliştirerek `char`, `int` gibi tipleri getirdi, işaretçi ve dizi kavramlarını bu tiplerle yeniden düzenledi. Ortaya çıkan dilin adı, B'den sonraki harf olduğu için **C** oldu. Dilin çekirdeği 1972'de oluştu; kitaplarda "C 1972'de doğdu" denmesinin nedeni budur.

> *Tarihçe · UNIX'in C ile yeniden yazılması (1973)*
>
> 1973 civarında UNIX çekirdeğinin büyük bölümü C ile yeniden yazıldı. O dönemde işletim sistemlerinin assembly dışında bir dille yazılması alışılmış bir şey değildi; ama sonuç büyük bir fark yarattı. Sistem kaynak kodu hem okunaklıydı, hem de başka bir makineye taşımak için yalnızca derleyiciyi ve küçük bir donanıma bağlı kısmı uyarlamak yeterliydi. 1970'lerin sonuna doğru UNIX farklı donanımlara taşınmaya başladı. Bu, "taşınabilir işletim sistemi" fikrinin ilk büyük başarısıydı ve C'nin yaygınlaşmasının asıl nedeni oldu.

Bu iki gelişme birbirini besledi: UNIX yayıldıkça C de yayıldı; C yayıldıkça UNIX farklı makinelere taşındı. Ken Thompson ile Dennis Ritchie, UNIX'e yaptıkları katkılar nedeniyle 1983'te bilgisayar biliminin en saygın ödülü sayılan **Turing Ödülü**'nü paylaştılar.

C'nin tasarım anlayışı

Ritchie'nin dili, ortaya çıktığı koşulları yansıtır:

- **Küçüktür**: Dilin kendisi az sayıda anahtar sözcük ve kuraldan oluşur. Giriş/çıkış gibi işler dile değil, standart kütüphaneye bırakılmıştır (Bölüm 4).
- **Donanıma yakındır**: Bellek adresleriyle doğrudan çalışmaya (işaretçiler, Bölüm 12), bit düzeyinde işlemlere (Bölüm 16) izin verir.
- **Hızlıdır**: Derleyici, kodunuzu araya fazladan denetim katmanı koymadan makine koduna çevirir.
- **Programcıya güvenir**: Dizinin dışına yazmak ya da geçersiz bir adresi kullanmak gibi hataları çalışma zamanında denetlemez. Bu özgürlük C'nin hem gücü hem tehlikesidir.

Bu dörtlü, sonraki bölümlerde sık sık karşımıza çıkacak. C++ bu mirasın üzerine kurulur; C# ise bazı noktalarda bilerek bunun karşıtı bir yol izler.

2.4 K&R ve Taşınabilirlik (1978)

1970'lerin ortasına kadar C'yi Bell Labs dışındaki programcılar, kaynak kodlarla birlikte dağıtılan notlardan ve UNIX dokümantasyonundan öğreniyordu. 1978'de Brian Kernighan ve Dennis Ritchie'nin birlikte yazdığı ***The C Programming Language*** kitabı yayımlandı. Kitap, C'nin ilk tam anlatımıydı ve yıllarca dilin fiilî tanımı olarak kullanıldı. Yazarların baş harflerinden dolayı bu kitaba ve kitaptaki dil sürümüne **K&R C** denir.

Kitabın ilk programı, bugün de her programlama kitabının ilk örneği olan "merhaba dünya" idi. Kitabın en tanınmış ayrıntılarından biri, fonksiyon tanımlarının bugünkünden farklı yazılmasıdır. K&R C'de parametre tipleri parantezin içinde değil, altında belirtilirdi:

```c
/* K&R tarzı: parametre tipleri parantezin dışında */
int toplam(a, b)
int a, b;
{
    return a + b;
}
```

Bugünkü yazımda ise tipler parametre listesinde yer alır:

```c
/* ANSI tarzı: prototip biçimi */
int toplam(int a, int b)
{
    return a + b;
}
```

İkinci biçimin avantajı, derleyicinin çağrıdaki argümanların sayısını ve tipini kontrol edebilmesidir. K&R biçiminde bu denetim yapılamazdı; yanlış sayıda argümanla çağrı sessizce derlenirdi. Bölüm 9'da fonksiyon prototiplerini ayrıntılı göreceğiz.

1980'lerin başında C, büyük şirketlerin, üniversitelerin ve yeni ortaya çıkan kişisel bilgisayar yazılım firmalarının tercih ettiği dil hâline geldi. Her derleyici dili biraz farklı yorumluyordu; aynı program bir derleyicide çalışırken diğerinde derlenmiyordu. Bu durum standartlaşmayı kaçınılmaz kıldı.

2.5 C'nin Standartları

ANSI C (1989) ve ISO C (1990)

1983'te Amerikan Ulusal Standartlar Enstitüsü (ANSI), C için bir standart komitesi (X3J11) kurdu. Komite, mevcut uygulamaları inceleyip ortak bir dil tanımı çıkarmaya çalıştı. Standart 1989'da onaylandı ve **ANSI C** ya da **C89** adıyla bilindi. Uluslararası Standartlar Örgütü (ISO) bunu 1990'da neredeyse aynen kabul etti; bu sürüme **C90** denir. C89 ve C90 teknik olarak aynı dildir.

Bu standardın getirdikleri arasında şunlar vardır:

- Fonksiyon prototipleri (yukarıda gördüğümüz tip bildirimli parametre listesi), `void`, `const`. Bunların birkaçı C++ geliştirilirken doğmuş ve C'ye oradan aktarılmıştı.
- `void *` genel işaretçi tipi.
- Standart kütüphanenin ilk tam tanımı (`<stdio.h>`, `<string.h>`, `<stdlib.h>` ve diğerleri).
- Dilin tamamı için ortak, açık kurallar.

Kernighan ve Ritchie kitabın ikinci baskısını 1988'de ANSI C'ye göre yeniden yazdılar. 1995'te "Düzeltme 1" adıyla küçük bir eklenti yayımlandı (geniş karakter desteği gibi), ama bu ayrı bir dil sürümü olarak kabul görmez.

Sonraki standartlar

C'nin sonraki sürümleri ISO tarafından yayımlanır. Her biri bir öncekini kapsar; dil geriye dönük uyumluluğu bilinçli olarak korur. Eski C89 programlarının büyük bölümü bugün de derlenir.

Standart	Yıl	Başlıca yenilikler
C89 / C90	1989 / 1990	İlk resmî standart: prototipler, `void`, `const`, standart kütüphane
C99	1999	`//` yorumları, `long long`, `_Bool` ve `<stdbool.h>`, değişkeni bloğun ortasında tanımlama, `<stdint.h>`, `inline`, değişken uzunluklu diziler (VLA), `snprintf`
C11	2011	Çoklu iş parçacığı desteği (`<threads.h>`), `_Atomic`, `_Generic`, `_Static_assert`; tehlikeli `gets` işlevinin kaldırılması; VLA'nın isteğe bağlı olması
C17	2018	C11'in hata düzeltmeleri; yeni özellik yok (C18 olarak da anılır). Bu kitabın standardı
C23	2024	`bool`, `true`, `false` ve `nullptr` anahtar sözcük; `constexpr`; `typeof`; ikilik sabit (`0b1010`) ve rakam ayırıcı (`1'000'000`); `#embed`

C17'nin adı onaylandığı yıldan, ISO yayımı ise 2018'den gelir; bu yüzden aynı sürüm C17 ve C18 olarak anılır. C23 için de benzer durum geçerlidir: çalışma 2023 hedefiyle yürütüldü, resmî yayım 2024'te yapıldı. Bu kitapta derlemeler `gcc -std=c17` ile yapılır; çıktıları C17'ye göre doğrulanmıştır.

> *Tarihçe · C99 uzun süre tam benimsenmedi*
>
> C99 çok sayıda yenilik getirdi, ama bazı derleyiciler, özellikle Microsoft'unki, bunları yıllarca tam desteklemedi. Bu yüzden 2000'li yıllarda yazılmış birçok C programı bilinçli olarak C89'a uygun yazılmıştır. Bugün gcc ve clang dahil bütün büyük derleyiciler C99, C11 ve C17'yi desteklemektedir.

Standart sürümlerin farkı gerçektir. Aşağıdaki program C99'da geçerli olan `//` yorumunu kullanıyor; ama C89 kurallarıyla derlemeye çalışırsak derleyici reddeder:

▸ yorum89.c

```c
#include <stdio.h>

int main(void) {
    // yorum satırı: C89'da yok
    printf("Merhaba\n");
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
yorum89.c: In function 'main':
yorum89.c:4:5: error: C++ style comments are not allowed in ISO C90
    4 |     // yorum satırı: C89'da yok
      |     ^
yorum89.c:4:5: note: (this will be reported only once per input file)
```

Derleyiciye hangi standardı kullanacağını `-std=` seçeneğiyle söyleriz (`-std=c89`, `-std=c99`, `-std=c17`, `-std=c2x`). Mesajdaki "C++ style comments" ifadesi ilginçtir: `//` yorumu aslında C++'ın buluşu değil, BCPL'den gelir; C++ onu yeniden canlandırmış ve C99 bunu C++'tan almıştır. Şimdi o dile, C++'a geçelim.

2.6 C++: Sınıflı C'den Modern C++'a

Stroustrup ve "C with Classes" (1979)

Danimarkalı bilgisayar bilimci **Bjarne Stroustrup**, doktora çalışmasını Cambridge'de, dağıtık sistemler üzerine yaptı ve bu çalışmada **Simula** dilini kullandı. Simula 67, Norveçli Ole-Johan Dahl ve Kristen Nygaard'ın tasarladığı bir dildi ve **sınıf** (class) ile **nesne** (object) kavramlarını ilk kez programlama diline getirmişti. Stroustrup, büyük bir programı sınıflarla parçalara ayırmanın düzen sağladığını bu çalışmada gördü; ama Simula programları çok yavaş çalışıyordu.

1979'da Bell Labs'a geçtiğinde karşısındaki sorun tersine bir ikilemdi: C hızlıydı ve donanıma yakındı, ama büyük programları düzenlemek için sınıf benzeri bir araç sunmuyordu. Stroustrup çözümü, C'ye Simula'dan alınan sınıfları eklemekte buldu. Ortaya çıkan dile **"C with Classes"** (sınıflı C) adı verildi. Dil, C programlarını olduğu gibi derleyebilecek şekilde tasarlandı; böylece mevcut C kodu ve C bilgisi çöpe gitmiyordu.

İlk "C with Classes" derleyicisi, kodu önce C'ye çeviren, sonra C derleyicisine veren bir çevirici (translator) olarak yazıldı. Bu fikir, bir sonraki aşamada **Cfront** adlı araca dönüştü.

C++ adı ve ilk yayımlar (1983–1985)

1983'te dil önemli yeni özellikler kazandı: sanal fonksiyonlar (çok biçimlilik), fonksiyon aşırı yükleme, referanslar, `const` ve `//` yorumları. Bu noktada dilin adı da değişti: **C++**. Adın şakasını biraz sonra anlatacağız.

C++ Stroustrup'un tek başına geliştirdiği bir araştırma projesi olmaktan çıkıp ticari bir ürün oldu:

- **1985**: Cfront'un ilk ticari sürümü yayımlandı ve Stroustrup'un ***The C++ Programming Language*** kitabının birinci baskısı çıktı. Bu kitap, uzun süre C++'ın fiilî tanımı oldu; tıpkı K&R'ın C için yaptığı gibi.
- **1989**: Dilin 2.0 sürümü çoklu kalıtımı (multiple inheritance) ve soyut sınıfları getirdi.
- **1990 dolayları**: Şablonlar (templates) ve istisnalar (exceptions) dile eklendi. Bunlar, dilin sonraki standardına girdi.

Standart Şablon Kütüphanesi ve ISO standardı (1998)

C++'ın bir diğer önemli kilometre taşı, 1990'ların başında Alexander Stepanov ve arkadaşlarının geliştirdiği **STL**'dir (Standard Template Library). STL, `vector` gibi veri yapılarını ve sıralama gibi algoritmaları şablonlarla genel biçimde yazma fikrine dayanır. 1994'te standart taslağına alındı ve C++'ın standart kütüphanesinin omurgası hâline geldi.

C++ standartlaştırma çalışması ANSI ve ISO komiteleri altında 1989'dan itibaren yürütüldü ve ilk uluslararası standart **1998'de** yayımlandı: **C++98**. 2003'te küçük bir düzeltme sürümü (C++03) geldi. Ardından yaklaşık on üç yıllık bir sessizlik başladı. Yeni sürüm uzun süre "C++0x" kod adıyla anıldı (adındaki x'in 200x yılını göstermesi bekleniyordu); sonunda 2011'de **C++11** olarak yayımlandı.

Modern C++ (2011'den bugüne)

C++11, dile öylesine köklü yenilikler getirdi ki bundan sonrasına **modern C++** denir. Standart komitesi o tarihten sonra üç yılda bir yeni sürüm yayımlama düzenine geçti:

Standart	Yıl	Başlıca yenilikler
C++98	1998	İlk ISO standardı: sınıflar, şablonlar, istisnalar, STL, `std::string`, akışlar (`cin`, `cout`)

Standart	Yıl	Başlıca yenilikler
C++03	2003	C++98'in hata düzeltmeleri
C++11	2011	`auto`, aralık tabanlı `for`, lambda ifadeleri, taşıma semantiği (move), akıllı işaretçiler (`unique_ptr`, `shared_ptr`), `nullptr`, `constexpr`, çoklu iş parçacığı
C++14	2014	C++11'in rötuşu: genel lambda ifadeleri, dönüş tipi çıkarımı, `make_unique`
C++17	2017	`if` içinde ilk değer, yapısal bağlama (structured bindings), `optional`, `variant`, `string_view`, dosya sistemi kütüphanesi. Bu kitabın standardı
C++20	2020	Kavramlar (concepts), aralıklar (ranges), modüller, eş yordamlar (coroutines), üç yönlü karşılaştırma `<=>`, `std::format`
C++23	2023	`std::print`, `std::expected`, `import std;` ve başka eklemeler

Modern C++ yalnızca yeni özellik eklemedi, programların yazılma biçimini de değiştirdi. Eskiden elle yönetilen bellek ve `new` / `delete` yerine akıllı işaretçiler ve kaplar (container) önerilir. Aynı işi iki dönemin üslubuyla yazarak farkı görelim: bir `vector`'ün elemanlarını toplayalım.

▸ tarihCpp.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> sayilar;
    sayilar.push_back(4);
    sayilar.push_back(8);
    sayilar.push_back(15);

    // C++98 üslubu: yineleyici (iterator) ile
    int toplam98 = 0;
    for (std::vector<int>::iterator it = sayilar.begin();
         it != sayilar.end(); ++it) {
        toplam98 += *it;
    }

    // C++11 üslubu: aralık tabanlı for
    int toplam11 = 0;
    for (int s : sayilar) {
        toplam11 += s;
    }

    std::cout << "98 üslubu: " << toplam98 << "\n";
    std::cout << "11 üslubu: " << toplam11 << "\n";
    return 0;
}
```

ÇIKTI

```
98 üslubu: 27
11 üslubu: 27
```

İki döngü aynı sonucu verir; ikincisi çok daha kısa ve okunaklıdır. Ayrıntılarını C++ bölümlerinde göreceğiz. Şimdilik bilmemiz gereken, C++'ın C gibi yerinde saymadığı, düzenli aralıklarla değişen ve eski yazım biçimlerini koruyarak büyüyen bir dil olduğudur.

Yeni özelliklerin eski standartla derlenmediğini yine derleyiciden görebiliriz. `nullptr` anahtar sözcüğü C++11 ile geldi:

bosIsaretci.cpp

```cpp
#include <iostream>

int main() {
    int *p = nullptr;
    if (p == nullptr) {
        std::cout << "bos\n";
    }
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
bosIsaretci.cpp: In function 'int main()':
bosIsaretci.cpp:4:14: error: 'nullptr' was not declared in this scope
    4 |     int *p = nullptr;
      |              ^~~~~~~
```

Mesajda derleyici `nullptr`'yi tanımıyor; çünkü `-std=c++98` seçeneğiyle derleme yaptık. C++17 ile (`g++ -std=c++17`) aynı kod sorunsuz derlenir.

> *Tarihçe · C++ neden C'yi kapsıyor?*
>
> Stroustrup'un en önemli tasarım kararlarından biri, C++'ın C ile büyük ölçüde **uyumlu** olmasıydı. Bu sayede C programcıları yeni dili adım adım öğrenebildi ve mevcut C kütüphaneleri C++ programlarından doğrudan kullanılabildi. Uyumluluk bugün tam değildir: C ve C++ zamanla ayrı yönlere gelişti ve her ikisinde de diğerinde olmayan özellikler var. "Her C programı geçerli bir C++ programıdır" cümlesi ne yazık ki doğru değildir.

2.7 Java ve C#'ın Doğuşu

Java'nın etkisi (1995)

1990'ların ortasında C++ çok yaygındı, ama aynı zamanda zor bir dil olarak da biliniyordu: elle bellek yönetimi, işaretçi hataları, platforma bağımlı derleme. Sun Microsystems'in James Gosling önderliğindeki ekibi 1995'te **Java** dilini yayımladı. Java'nın sözdizimi C++'a çok benziyordu, ama temel tasarım kararları farklıydı:

- Kod doğrudan makine koduna değil, **bayt koduna** (bytecode) derlenir; bu kodu her platformdaki **Java Sanal Makinesi** (JVM) çalıştırır. Slogan: "bir kez yaz, her yerde çalıştır."
- Bellek **çöp toplayıcı** (garbage collector) tarafından otomatik yönetilir; `free` ya da `delete` yoktur.
- Elle işaretçi aritmetiği yoktur.

Java, kurumsal yazılım dünyasında hızla yer buldu ve "C benzeri söz dizimi, ama güvenli ve sanal makine üzerinde çalışan" bir dil modeli oluşturdu. Microsoft da Java'yı Windows için kullanmaya çalıştı (Visual J++), ama Sun ile lisans konusunda yasal bir anlaşmazlık yaşadı. Bu anlaşmazlığın, Microsoft'un kendi yönetilen (managed) çalışma ortamını ve dilini geliştirme kararında rol oynadığı sık anlatılır.

.NET ve C# (2000–2002)

Microsoft, **.NET** platformunu ve **C#** dilini 2000 yılında duyurdu. Geliştirme sürecinde dil, "COOL" kod adıyla anılmıştır. Dilin başmimarı **Anders Hejlsberg**'dir. Hejlsberg, Danimarkalı bir yazılım mühendisidir ve daha önce Borland için hızlı derleyicisiyle ünlenen **Turbo Pascal**'ı yazmış ve **Delphi**'nin başmimarı olmuştu; 1996'da Microsoft'a geçti. C#'ın Delphi'den, Java'dan ve C++'tan izler taşıması bu geçmişin sonucudur.

C# 1.0 ve onu çalıştıran **.NET Framework 1.0**, 2002'de yayımlandı. Platformun mimarisi Java'ya benzerdi:

1. C# derleyicisi, kaynak kodu **IL** (Intermediate Language, ara dil) denen ara koda çevirir.
2. Program çalışırken **CLR** (Common Language Runtime) içindeki **JIT** derleyicisi bu IL'yi makine koduna dönüştürür.
3. Bellek, çöp toplayıcı ile yönetilir.

Bu mimariyi Bölüm 1'de derleyici ve yorumlayıcı farkıyla birlikte görmüştük. Önemli bir fark vardı: .NET, tek bir dille sınırlı değildi. Platform üzerinde Visual Basic .NET gibi birçok dil aynı kütüphaneleri ve çalışma ortamını paylaşıyordu. C# bunların arasında platformun "ana dili" oldu.

C# dilinin kendisi, 2002–2003'te uluslararası standartlara bağlandı (ECMA-334, ardından ISO/IEC 23270). Yani dilin tanımı bir şirketin değil, bağımsız bir standardın elindedir; ama dilin gelişimini fiilen Microsoft'un yönettiği ekip belirler.

2.8 .NET'in Evrimi ve C# Sürümleri

Üç ayrı .NET kolu, bir birleşme

.NET'in tarihi iki döneme ayrılır:

- **.NET Framework (2002–)**: Yalnızca Windows'ta çalışan, kapalı kaynaklı platform. Son ana sürümü 4.8'dir; yeni özellik almaz, yalnızca bakım görür. Eski kurumsal uygulamaların büyük bölümü hâlâ üzerinde çalışır.
- **Mono**: 2000'lerin başında, Miguel de Icaza'nın öncülüğünde başlayan bağımsız ve açık kaynaklı bir .NET uyarlaması. Linux ve macOS'ta .NET programlarını çalıştırmayı mümkün kıldı. Daha sonra Unity oyun motorunun C# desteğinin temeli oldu.
- **.NET Core (2016–)**: Microsoft'un sıfırdan yazdığı, açık kaynaklı ve **çapraz platform** (Windows, Linux, macOS) yeni platform. Daha hızlı ve modülerdi. Bu dönüşümün hazırlığı 2014'te başladı: Microsoft .NET'in büyük bölümünü açık kaynak olarak yayımladı ve C# derleyicisi **Roslyn**'i herkese açtı.

Bu kolların birleşimi **.NET 5** (2020) ile sağlandı: ad "Core" ekini bıraktı ve tek bir .NET çatısı oluştu. O günden beri her yıl Kasım ayında yeni bir ana sürüm çıkar. Çift numaralı sürümler **LTS** (Long-Term Support, uzun süreli destek) alır: .NET 6 (2021), .NET 8 (2023) ve .NET 10 (2025). Bu kitap .NET 8 ve C# 12 ile doğrulanmıştır; sonraki sürümlerin özelliklerine de gerektiğinde değineceğiz.

C# sürümleri ve öne çıkan özellikleri

C#, bu kitaptaki üç dil arasında en çok sürüm alan ve dil düzeyinde en hızlı değişenidir. Aşağıdaki tablo, her sürümün öne çıkan özelliğini gösterir. Bunların çoğunu ilgili C# bölümlerinde göreceğiz.

Sürüm	Yıl	Öne çıkan özellikler
C# 1.0	2002	İlk sürüm: sınıflar, arayüzler, özellikler (property), olaylar (event), temsilciler (delegate)
C# 2.0	2005	Jenerikler (generics), boş değer alabilen tipler (`int?`), yineleyiciler (`yield`), anonim metotlar
C# 3.0	2007	LINQ, lambda ifadeleri, `var` , uzantı metotları, anonim tipler
C# 4.0	2010	`dynamic` , adlandırılmış ve isteğe bağlı parametreler
C# 5.0	2012	`async` / `await` ile eşzamansız programlama
C# 6.0	2015	Metin enterpolasyonu (`$"..."`), `?.` operatörü, ifade gövdeli üyeler, `nameof`

Sürüm	Yıl	Öne çıkan özellikler
C# 7.0	2017	Demetler (tuples), örüntü eşleme (pattern matching) başlangıcı, `out var`, yerel fonksiyonlar
C# 8.0	2019	Boş değer alamayan referans tipleri (nullable reference types), `switch` ifadeleri, aralık ve indeks (`^1` , `..`)
C# 9.0	2020	Kayıtlar (`record`), `init` erişimcisi, üst düzey ifadeler (top-level statements)
C# 10	2021	Genel `using` , dosya kapsamlı ad alanı (file-scoped namespace), `record struct`
C# 11	2022	Ham metin sabitleri (`"""`), liste örüntüleri, `required` üyeler
C# 12	2023	Birincil kurucular (primary constructors), koleksiyon ifadeleri (`[1, 2, 3]`). Bu kitabın sürümü
C# 13	2024	`params` ile koleksiyon tipleri, yeni `Lock` tipi
C# 14	2025	Uzantı üyeleri (extension members), `field` anahtar sözcüğü, `?.` ile atama

Bu hızlı değişimi iki dönemin üslubuyla yazılmış küçük bir programda görelim. Aynı işi yapıyoruz: bir ad listesinden dört harften uzun olanları bulmak. Önce C# 1.0 dönemindeki `ArrayList` ile, sonra C# 3.0'la gelen jenerik koleksiyon ve LINQ ile:

surumlerCs.cs

```
using System.Collections;

// C# 1.0 üslubu: tipsiz ArrayList, elle dönüştürme
ArrayList eski = new ArrayList();
eski.Add("Ayşe");
eski.Add("Mehmet");
eski.Add("Elif");
eski.Add("Zeynep");

string eskiSonuc = "";
foreach (object nesne in eski) {
    string ad = (string)nesne;
    if (ad.Length > 4) {
        eskiSonuc += ad + " ";
    }
}
Console.WriteLine("Eski: " + eskiSonuc.Trim());

// C# 2.0 ve 3.0 sonrası: tipli liste ve LINQ
var yeni = new List<string> { "Ayşe", "Mehmet", "Elif", "Zeynep" };
var uzunlar = yeni.Where(ad => ad.Length > 4);
Console.WriteLine("Yeni: " + string.Join(" ", uzunlar));
```

ÇIKTI

```
Eski: Mehmet Zeynep
Yeni: Mehmet Zeynep
```

Birinci bölümde her elemanı `object` olarak alıp elle `string`'e dönüştürmek zorundaydık; yanlış tipte bir eleman eklenseydi hata ancak çalışırken ortaya çıkardı. İkincisinde jenerik liste her elemanın `string` olduğunu derleme sırasında bilir ve LINQ ile filtre tek satıra iner. Jenerikleri ve LINQ'yu C# bölümlerinde ayrıntılı işleyeceğiz.

2.9 İsimlerin Hikâyesi

Bu üç adın her biri kendi içinde küçük bir şakadır.

C

Ad, dilin **B**'den türemiş olmasından gelir: B'nin ardından gelen harf C'dir.

C++

Bjarne Stroustrup'un dili 1983'te "C++" olarak adlandırması, C'nin `++` operatörüne göndermedir. Bu operatör bir değişkeni **bir artırır**. Dolayısıyla `C++`, "C'nin bir artırılmışı" anlamına gelir. Stroustrup'un aktardığına göre adı Bell Labs'taki meslektaşı Rick Mascitti önerdi.

Şakanın ince bir yanı da var: `++` **sonek** (postfix) biçiminde, yani `c++` biçiminde kullanıldığında ifadenin değeri artırmadan **önceki** değerdir. Bunu küçük bir programda görelim (operatörleri Bölüm 5'te ayrıntılı işlemiştik):

▸ cArtirma.c

```c
#include <stdio.h>

int main(void) {
    int c = 5;
    int eski = c++;
    printf("eski = %d, c = %d\n", eski, c);
    return 0;
}
```

ÇIKTI

```
eski = 5, c = 6
```

`c++` ifadesi, `c`'yi 6 yapar ama ifadenin değeri hâlâ eski 5'tir. Önek biçimi `++c` kullansaydık `eski` de 6 olurdu.

C

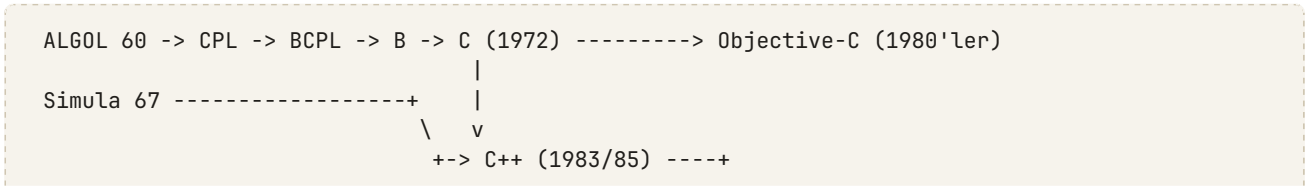
Adın okunuşu "si şarp"tır (C sharp). **Sharp** müzikte bir notayı yarım ton yükselten **diyez** (♯) işaretidir; yani C# müzik notası olarak "do diyez"dir. Yine bir artış fikri: C++ "bir artır" ise, C# "bir yarım ton yukarı çık" der.

İkinci bir okuma da vardır: `#` işaretine yakından bakıldığında, dört adet `+` işaretinin iki satır iki sütun hâlinde birleşmiş hâli gibi görünür. C#, bu bakışla "C++'ın C++'ı"dır. Bu yorum espri olarak sık anlatılır; Microsoft'un resmî açıklaması ise müzikteki diyez üzerinedir.

> **Not · Dosya adlarında "#"**
>
> Klavyede `#` yazmak ve dosya adlarında kullanmak sorun çıkardığından C# dosyalarının uzantısı `.cs`, C++ dosyalarının uzantısı `.cpp` (bazen `.cc`) olur. Komutlarda da `g++`, `dotnet` gibi adlar kullanılır; `c#` ya da `c++` yazılmaz.

2.10 Soy Ağacı

Üç dil birbirinden bağımsız doğmadı. Aşağıdaki şema ana akrabalık hatlarını gösterir (oklar "esinlendi" ya da "türedi" anlamındadır):



```
                                    |              |
                                    v              |
                                Java (1995)        |
                                    |              |
Delphi (Hejlsberg) --+              v              |
                     +-----------> C# (2000/02) <--+
```

C ve C++'tan sonra gelen birçok dil, doğrudan türemese bile C'nin **söz dizimini** (süslü parantezler, `if` / `for` / `while`, `;` ile biten deyimler) miras aldı:

Dil	Yıl	C ile ilişkisi
Objective-C	1980'ler	C'ye Smalltalk tarzı nesne yönelimli özellikler ekler; uzun yıllar Apple platformlarında kullanıldı
Java	1995	C++ söz dizimini temel alır
JavaScript	1995	Adı Java'ya benzese de ayrı bir dildir; söz dizimi C benzeridir
PHP	1995	Web programlamada, C benzeri söz dizimiyle
Go	2009	Google'da tasarlandı; tasarımcılarından biri Ken Thompson'dır; C'nin sadeliğini modern özelliklerle birleştirir
Rust	2015 (sürüm 1.0)	Mozilla'da doğdu; C ve C++'ın alanında bellek güvenliğini derleyici denetimiyle sağlamayı hedefler

Python'un standart yorumlayıcısı da (CPython) C ile yazılmıştır; yani yüksek seviyeli dillerin pek çoğu altta C kullanır.

2.11 Kullanım Alanları ve İlk Karşılaştırma

Üç dil aynı kökten gelse de, tasarım hedefleri farklı olduğu için farklı alanlarda öne çıkar.

C nerede kullanılır?

- İşletim sistemi çekirdekleri ve aygıt sürücüleri (Linux çekirdeği başta olmak üzere).
- Gömülü sistemler: mikrodenetleyiciler, otomobil ve ev aleti yazılımları.
- Diğer dillerin yorumlayıcı ve çalışma ortamları, veritabanı motorları, ağ yazılımları.

C++ nerede kullanılır?

- Oyun motorları ve yüksek başarımlı grafik uygulamaları.
- Tarayıcılar, veritabanı yazılımları, finans ve borsa sistemleri.
- Ses, video ve bilimsel hesaplama yazılımları; masaüstü uygulamaları.

C# nerede kullanılır?

- Kurumsal iş uygulamaları ve web servisleri (ASP.NET Core).
- Windows masaüstü uygulamaları (WPF, Windows Forms) ve çapraz platform arayüzleri.
- Unity oyun motoru ile oyun geliştirme.
- Bulut hizmetleri ve mobil uygulamalar.

Üç dilin başlıca özelliklerini bir arada görelim:

Özellik	C	C++	C#
Çıkış	1972	1979 / 1983 (adı) / 1985 (ilk sürüm)	2000 duyuruldu, 2002 ilk sürüm
Tasarımcı	Dennis Ritchie (Bell Labs)	Bjarne Stroustrup (Bell Labs)	Anders Hejlsberg ve ekibi (Microsoft)
Ana paradigma	Yordamsal	Çok paradigmalı: yordamsal, nesne yönelimli, jenerik, işlevsel öğeler	Nesne yönelimli; jenerik ve işlevsel öğeler
Bellek yönetimi	Elle (`malloc` , `free`)	Elle ya da RAII ve akıllı işaretçilerle	Çöp toplayıcı (GC)
Derleme modeli	Doğrudan makine koduna	Doğrudan makine koduna	IL'ye derlenir, çalışırken JIT ile makine koduna çevrilir
Standart	ISO/IEC 9899	ISO/IEC 14882	ECMA-334 ve ISO/IEC 23270
Tipik alan	Sistem, gömülü	Oyun, performans, sistem	Kurumsal, web, Unity oyunları, masaüstü

Bu tablodaki her satırı kitap ilerledikçe ayrıntılandıracağız. Şimdilik akılda tutulacak iki ilke: C ve C++ programcıya kontrol verir ve bunun bedelini hata riskiyle öder; C# ise bazı kontrolleri çalışma ortamına devrederek güvenlik ve üretkenlik kazandırır.

Üç Dilde · Aynı işin ilk görünümü *C · C++ · C#*

Üç dilde ekrana "5 + 3 = 8" yazdıran satırlar, aynı iş için üç farklı yaklaşım gösterir. C'de biçim metni ve değerler ayrı verilir; C++'ta akış operatörü zincirlenir; C#'ta ise metin enterpolasyonu kullanılabilir.

```c
printf("5 + 3 = %d\n", 5 + 3);
```

```cpp
std::cout << "5 + 3 = " << 5 + 3 << "\n";
```

```csharp
Console.WriteLine($"5 + 3 = {5 + 3}");
```

C kısmı Bölüm 4 ve 6'da, C++ kısmı Bölüm 23'te, C# kısmı Bölüm 43'te ayrıntılı işlenecek.

2.12 Bugün Neden Hâlâ Önemliler?

Üç dil de yarım asırdan yaşlı ya da ona yakın; yine de her biri kendi alanında etkin biçimde kullanılmaya devam ediyor. Bunun birkaç nedeni var:

- **Mevcut kod tabanı**: İşletim sistemleri, veritabanları, tarayıcılar ve oyun motorları milyonlarca satır C ve C++ kodundan oluşur. Bu kod yeniden yazılmaz, bakımı yapılır ve geliştirilir.
- **Performans ve denetim**: Belleğe ve donanıma doğrudan erişim gereken, gecikmenin kritik olduğu yerlerde C ve C++ halen en doğal seçimdir.
- **Sürekli yenilenme**: C++ üç yılda bir, C# her yıl yeni bir sürümle güncellenir; C de belirli aralıklarla yeni standartlar yayımlar.
- **Bilginin aktarılabilirliği**: C'yi öğrenmek bellek, adres ve derleme kavramlarını gösterir; C++ bunların üzerine soyutlamayı; C# ise yönetilen bir ortamın nasıl çalıştığını ekler. Üçünü birlikte bilen biri başka dillere de çabuk uyum sağlar.

Dillerin ne kadar yaygın olduğunu ölçmeye çalışan birkaç düzenli çalışma vardır: TIOBE endeksi, IEEE Spectrum sıralaması ve Stack Overflow geliştirici anketi gibi. Bu listeler kullandıkları yönteme göre farklı sonuçlar verir ve yıldan yıla değişir; bu yüzden burada sıra ya da yüzde vermiyoruz. Genel olarak şunu söyleyebiliriz: C, C++ ve C# onlarca yıldır bu listelerin üst bölümlerinde yer almaktadır.

> ◆ **Kritik Nokta · Üç dil, tek ailenin üç dalı**
>
> C bu ailenin kökü: küçük, hızlı, programcıya güvenen. C++ onun üzerine kurulu ve onu kapsayan bir genişleme: nesneler, şablonlar, zengin kütüphane. C# ise ailenin yönetilen üyesi: C++ ve Java'dan etkilenmiş, çöp toplayıcılı ve .NET üzerinde çalışan bir dil. Bu kitap üçünü bu sırayla öğretir, çünkü her adım bir öncekinin üzerine kurulur.

Bölüm Özeti

- 1950'lerin sonunda FORTRAN, LISP ve COBOL ortaya çıktı; 1960'ta ALGOL 60 blok yapısı ve özyineleme gibi fikirlerle C ailesinin söz diziminin atası oldu.
- C'nin soy zinciri ALGOL 60 ⟶ CPL ⟶ BCPL (Martin Richards, 1967) ⟶ B (Ken Thompson, ~1969) ⟶ C şeklindedir.
- C, 1972 civarında Bell Labs'ta Dennis Ritchie tarafından, B'ye veri tipleri eklenerek geliştirildi; adı B'den sonraki harften gelir.
- UNIX çekirdeğinin 1973 civarında C ile yeniden yazılması, işletim sistemlerinin taşınabilirliğini mümkün kıldı ve C'yi yaydı.
- Kernighan ve Ritchie'nin 1978 tarihli kitabı C'nin fiilî tanımı oldu (K&R C); fonksiyon tanımları eski biçimde, parametre tipleri parantezin dışında yazılırdı.
- C standartları: C89 (ANSI, 1989) / C90 (ISO, 1990), C99, C11, C17 (yayımı 2018) ve C23 (2024). Bu kitap C17'yi esas alır; derleyiciye standardı `-std=` ile söyleriz.
- C++, 1979'da Bell Labs'ta Bjarne Stroustrup tarafından "C with Classes" adıyla başladı; Simula'nın sınıf fikrini C'nin hızıyla birleştirdi. Adı 1983'te C++ oldu, ilk ticari sürüm ve kitap 1985'te geldi.
- İlk ISO C++ standardı 1998'dedir (C++98); C++11 modern C++ dönemini başlattı ve ardından C++14, 17, 20, 23 üç yıllık düzenle geldi. Bu kitap C++17'yi esas alır.
- Java (Sun, 1995), C++ benzeri söz dizimini sanal makine ve çöp toplayıcı ile birleştirdi; bu model Microsoft'un .NET ve C# yaklaşımına zemin hazırladı.
- C#, Anders Hejlsberg liderliğinde Microsoft'ta geliştirildi; 2000'de duyuruldu, C# 1.0 ve .NET Framework 1.0 2002'de yayımlandı.
- .NET Framework yalnızca Windows'a özgüydü; .NET Core (2016) açık kaynaklı ve çapraz platform oldu; .NET 5 (2020) ile tek çatı altında birleşildi. Sonrasında her yıl yeni bir .NET ve C# sürümü çıkar.
- "C++" adı `++` artırma operatörüne, "C#" adı ise müzikteki diyez işaretine gönderme yapar.
- Üç dilin kökeni ortaktır: C'nin söz dizimi Java, JavaScript, Go ve başka dillere de geçmiştir.
- C sistem ve gömülü yazılımda, C++ oyun ve yüksek başarımlı uygulamalarda, C# kurumsal, web ve Unity oyunlarında öne çıkar.

Alıştırmalar

1. C dilini Bell Labs'ta geliştiren kişi kimdir?

   A) Ken Thompson
   B) Dennis Ritchie
   C) Bjarne Stroustrup
   D) Martin Richards
   E) Anders Hejlsberg

2. Aşağıdakilerden hangisi C'nin soy zincirini doğru sırayla verir?

   A) B → BCPL → CPL → C
   B) CPL → BCPL → B → C
   C) BCPL → CPL → B → C
   D) ALGOL → C → B → BCPL
   E) CPL → B → BCPL → C

3. B dilinin PDP-11 gibi yeni makinelerde yetersiz kalmasının temel nedeni nedir? Birkaç cümleyle açıklayın ve C'nin bu soruna getirdiği çözümü belirtin.

4. UNIX çekirdeğinin C ile yeniden yazılması neden önemli bir gelişmedir?

   A) UNIX'i daha büyük yaptığı için
   B) Assembly'den vazgeçildiği için bellek kullanımı sıfıra indiği için
   C) İşletim sisteminin farklı makinelere taşınmasını kolaylaştırdığı için
   D) UNIX'i ücretsiz yaptığı için
   E) C'yi yalnızca UNIX'e özgü kıldığı için

5. K&R tarzı ile ANSI tarzı fonksiyon tanımı arasındaki fark nedir? Aşağıdaki iki tanımı karşılaştırıp hangisinde derleyicinin argüman sayısını denetleyebildiğini yazın.

```
int a(x, y) int x, y; { return x + y; }
int b(int x, int y) { return x + y; }
```

6. Aşağıdaki dosya `-std=c89` ile derlenirse ne olur? Neden?

```
#include <stdio.h>

int main(void) {
    // selam
    printf("Merhaba\n");
    return 0;
}
```

7. Aşağıdaki standartlardan hangisi yeni dil özelliği getirmeyen, yalnızca düzeltme içeren sürümdür?

   A) C99
   B) C11
   C) C17

D) C23
E) C89

8. Bjarne Stroustrup'un "C with Classes" fikri hangi dilden hangi kavramı C'ye aktarmayı hedefliyordu? Dilin adı hangi yılda "C++" oldu?
9. Aşağıdaki özelliklerden hangisi C++11 ile gelmemiştir?

   A) `auto`
   B) aralık tabanlı `for`
   C) lambda ifadeleri
   D) `nullptr`
   E) `std::print`

10. Bu programın çıktısı nedir?

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    int y = x++;
    printf("%d %d\n", x, y);
    return 0;
}
```

11. C# 1.0 hangi yıl yayımlanmıştır ve hangi platformla birlikte çıkmıştır? C# dilinin ilk tasarımcısı kimdir?
12. .NET Framework ile .NET Core arasındaki iki temel fark nedir? Bu iki platform hangi sürümle birleşmiştir?
13. Aşağıdaki özelliklerden hangisi C# 2.0'da gelen jeneriklerin sağladığı bir avantajdır?

    A) Programın daha hızlı derlenmesi
    B) Koleksiyon elemanlarının tipinin derleme sırasında bilinmesi ve elle dönüştürme gereksiniminin kalkması
    C) Çöp toplayıcının kaldırılması
    D) Koleksiyonlara farklı tiplerde eleman eklenebilmesi
    E) İşaretçilerin kullanılabilmesi

14. "C#" adının iki farklı yorumunu açıklayın.
15. Aşağıdaki alanların her biri için C, C++ ve C#'tan en uygun adayı eşleştirin ve kısaca gerekçelendirin: (a) Linux çekirdeği için bir aygıt sürücüsü, (b) Unity ile bir mobil oyun betiği, (c) yüksek başarımlı bir oyun grafik motoru, (d) kurumsal bir web servisi.
16. Üç dilin bellek yönetimi yaklaşımını karşılaştıran kısa bir paragraf yazın (en fazla beş cümle).

BÖLÜM 3

Geliştirme Ortamının Kurulumu

BU BÖLÜMDE

- Derleyici, editör, IDE ve SDK kavramlarını birbirinden ayırmayı; her dil için nelere ihtiyaç olduğunu
- Windows'ta MSYS2 ve MinGW-w64 ile gcc/g++ kurmayı, PATH ayarını yapmayı ve kurulumu doğrulamayı
- Linux ve macOS'ta derleyici kurmayı; macOS'ta `gcc` komutunun aslında clang olduğunu
- VS Code'u, C/C++ ve C# Dev Kit eklentileriyle birlikte kurmayı; Türkçe karakterler için UTF-8 ayarını
- Visual Studio Community'nin hangi iş yükleriyle kurulacağını ve ne zaman tercih edileceğini
- .NET SDK'yı (güncel LTS sürümü .NET 10) kurmayı; `dotnet new`, `dotnet run` ve proje yapısını tanımayı
- Üç dilde ilk programı derleyip çalıştırmayı; sık kurulum sorunlarını çözmeyi
- Hata ayıklayıcıda kesme noktası koyup programı adım adım izlemeyi

Programlamayı öğrenmenin en kısa yolu program yazıp çalıştırmaktan geçer. Bunun için bilgisayarımızda birkaç aracın kurulu olması gerekir. Kurulum çoğu zaman bir kez yapılır, ama en sık takılınan yer de burasıdır; sorun kitabın konularında değil, PATH ayarında ya da yanlış klasörde çıkar.

Bu bölümde C, C++ ve C# için çalışma ortamını adım adım kuracağız. Bölümün sonunda her üç dilde de "Merhaba, Dünya!" programını yazıp derleyecek, çalıştıracak ve hata ayıklayıcıyla (debugger) satır satır izleyecek durumda olacağız. Sonraki bölümlerdeki bütün örnekler bu ortamda denenebilir.

Kurulum adımları zamanla küçük değişiklikler gösterir: sürüm numaraları, düğme adları ve indirme sayfalarının düzeni değişebilir. Bu bölümde menü ve düğme adlarını yalnızca kararlı olanlarla anıyoruz; bir pencere anlatılandan farklı görünürse adımın amacı aynı kalır.

3.1 Neye İhtiyacımız Var?

Kaynak kod, insanın okuyabildiği düz bir metin dosyasıdır. Bilgisayar bu metni doğrudan çalıştıramaz; önce makine koduna çevrilmesi gerekir (Bölüm 1). Dolayısıyla en az iki araca ihtiyaç vardır: kodu yazacağımız bir **editör** ve onu çeviren bir **derleyici**.

> **Tanım: Derleyici**
>
> Bir programlama dilinde yazılmış kaynak kodu okuyup başka bir biçime, genellikle makine koduna ya da bir ara koda çeviren programdır (compiler). Hataları da bildirir.

Tanım: Editör

Düz metin dosyalarını yazmaya ve düzenlemeye yarayan programdır. Kod editörleri buna ek olarak sözdizimini renklendirir, girintiyi düzenler ve kod tamamlama sunar.

Tanım: IDE

Editör, derleyici çağırma, hata ayıklayıcı ve proje yönetimini tek pencerede toplayan bütünleşik geliştirme ortamıdır (Integrated Development Environment).

Bir de **SDK** (Software Development Kit) kavramı vardır. C# için derleyici tek başına dağıtılmaz; derleyiciyi, çalışma zamanını (runtime), standart kütüphaneyi ve `dotnet` komut satırı aracını içeren bir paket olan .NET SDK kurulur. C ve C++ için ise gcc ya da clang gibi bir derleyici paketi yeterlidir.

Üç dil için gereken araçlar ve bu kitapta izleyeceğimiz yol şöyledir:

Dil	Derleyici / SDK	Önerilen editör	Alternatif ortam
C	gcc (Windows'ta MinGW-w64, macOS'ta clang)	VS Code + C/C++ eklentisi	Visual Studio (MSVC derleyicisi)
C++	g++ (gcc ile birlikte gelir)	VS Code + C/C++ eklentisi	Visual Studio (MSVC derleyicisi)
C#	.NET SDK (10 LTS)	VS Code + C# Dev Kit	Visual Studio Community

Kitabın çıktıları C için `gcc -std=c17`, C++ için `g++ -std=c++17` ve C# için C# 12 (.NET 8) ile alınmıştır; örnekler .NET 10'da da aynı çalışır. Aynı sürümlerle çalışırsak ekranda kitaptakiyle aynı çıktıları ve aynı uyarı mesajlarını görürüz.

Tarihçe · gcc'nin adı

gcc, 1980'lerin sonunda GNU projesi kapsamında geliştirilmeye başlandığında yalnızca C dilini derliyordu ve adı "GNU C Compiler" idi. Zamanla C++ dahil birçok dili desteklemeye başladı ve ad "GNU Compiler Collection" olarak anlamlandırıldı. Bugün `gcc` komutu C derleyicisini, `g++` komutu C++ derleyicisini çalıştırır.

Terminalle ilk tanışma

Kitap boyunca derleme ve çalıştırma işini **terminalden** yapacağız. Terminal, komutları yazı olarak verdiğimiz penceredir; içinde çalışan ve komutları yorumlayan programa **kabuk** (shell) denir. Windows'ta PowerShell ya da Komut İstemi (cmd), Linux ve macOS'ta ise genellikle bash ya da zsh kullanılır.

Fare ile yapabildiğimiz her şeyi komutla da yapabiliriz. Başlangıç için beş komut yeter:

İş	Windows (PowerShell / cmd)	Linux ve macOS
Bulunduğumuz klasörü görme	`pwd` (PowerShell), `cd` (cmd)	`pwd`
Klasörün içeriğini listeleme	`dir`	`ls`
Klasöre girme	`cd klasorAdi`	`cd klasorAdi`
Bir üst klasöre çıkma	`cd ..`	`cd ..`
Yeni klasör oluşturma	`mkdir klasorAdi`	`mkdir klasorAdi`

Çalışma klasörümüzü şimdiden oluşturalım. Adında boşluk, Türkçe karakter ya da çok derin bir yol bulunmayan basit bir klasör ileride sorun çıkarmaz:

```
TERMİNAL
$ mkdir cdersleri
$ cd cdersleri
$ pwd
```

Bu kitaptaki terminal örneklerinde satır başındaki `$` işareti komut isteminin gösterimidir; siz yalnızca `$`'dan sonrasını yazarsınız. Windows'ta komut isteminin görünümü farklıdır (`PS C:\Users\...>` gibi), ama komutlar aynıdır.

> **Not · Çalışma klasörünü seçmek**
>
> Windows'ta Masaüstü ve Belgeler klasörleri bulut yedekleme hizmetleriyle eşitleniyor olabilir; derleme sırasında oluşan dosyalar bu hizmetlerle çakışabilir. Kullanıcı klasörünüzün altında, `C:\Users\Adiniz\cdersleri` gibi sade bir klasör daha sorunsuz çalışır. Kullanıcı adınız Türkçe karakter ya da boşluk içeriyorsa bunun yaratabileceği sorunlara "Sık Kurulum Sorunları" başlığında döneceğiz.

3.2 Windows'ta gcc ve g++: MSYS2 ile MinGW-w64

gcc, kökeni itibarıyla Unix benzeri sistemlerin derleyicisidir. Windows'ta kullanabilmek için **MinGW-w64** adlı bir derleme ortamına ihtiyaç vardır. MinGW-w64, gcc'yi Windows'un programlama arayüzleriyle derleyen ve Windows'ta doğrudan çalışan `.exe` dosyaları üreten bir projedir. Bu ortamı kurmanın en kolay yolu **MSYS2** kullanmaktır. MSYS2, Windows için bir paket yöneticisi (`pacman`) ve bir dizi hazır araç sunar; MinGW-w64 derleyicisini de bu yöneticiyle kurarız.

Adım 1: MSYS2'yi kurmak

1. Tarayıcıda `msys2.org` adresine gidin ve ana sayfadaki kurulum dosyasını (`msys2-x86_64-` ile başlayıp tarih içeren `.exe`) indirin.
2. Dosyayı çalıştırın. Önerilen kurulum klasörü `C:\msys64`'tür; değiştirmeyin. Klasör yolunda boşluk ya da Türkçe karakter bulunmamalıdır.
3. Diğer ekranlarda varsayılan seçenekleri kabul edin. Kurulum bitince MSYS2'nin kendi terminalini açmayı öneren bir seçenek çıkabilir.

MSYS2 yalnızca 64 bitlik, güncel Windows sürümlerinde çalışır. Kurulum sayfasındaki güncel gereksinimleri kontrol etmekte yarar vardır.

Adım 2: Doğru terminali açmak

MSYS2 kurulunca Başlat menüsünde birden fazla terminal kısayolu görünür (MSYS2 MSYS, UCRT64, MINGW64, CLANG64 gibi). Bunların her biri farklı bir "ortam"dır ve farklı derleyici paketleriyle çalışır. Bizim kullanacağımız ortam **UCRT64**'tür. UCRT, Windows 10 ve sonrasında işletim sisteminin kendisiyle gelen modern C çalışma zamanı kütüphanesidir (Universal C Runtime); bu sayede derlediğimiz programlar ek kütüphane aramadan çalışır.

Başlat menüsünden **MSYS2 UCRT64** kısayolunu açın. Siyah (ya da koyu) bir terminal penceresi görünür.

> **▲ Sık Yapılan Hata · Yanlış MSYS2 terminali**
>
> Başlat menüsündeki ilk MSYS2 kısayolu "MSYS2 MSYS"dir. Bu ortam, MSYS2'nin kendi araçları içindir; burada kurduğumuz gcc, Windows programı değil, MSYS2'ye bağımlı bir program üretir. Derleyici paketini kuracağımız ve gcc'yi deneyeceğimiz terminal her zaman **UCRT64** olmalıdır. Pencerenin başlığında ve komut isteminde `UCRT64` yazdığını kontrol edin.

Adım 3: Sistemi güncellemek ve derleyiciyi kurmak

UCRT64 terminalinde önce paket veritabanını ve temel paketleri güncelleyelim:

```
TERMİNAL
$ pacman -Syu
```

`pacman` ne yapacağını anlatır ve onay ister (`Proceed with installation? [Y/n]`). Enter tuşu varsayılanı, yani "evet"i seçer. Güncelleme sırasında terminal penceresini kapatması istenebilir; kapatıp UCRT64 terminalini yeniden açın ve aynı komutu bir kez daha çalıştırarak güncelleme bitene kadar tekrarlayın.

Ardından derleme araç zincirini kuralım:

```
TERMİNAL
$ pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

Komut hangi paketlerin kurulacağını sorar (`Enter a selection (default=all)`); Enter tuşuyla hepsini seçmek en doğrusudur. Ardından kuruluma onay verilir. Bu işlem gcc'yi, g++'ı, `gdb` hata ayıklayıcısını, `make` aracını ve standart kütüphaneyi indirip kurar. Bağlantı hızına göre birkaç dakika sürebilir.

Komuttaki parçalar şunlardır:

Parça	Anlamı
`pacman`	MSYS2'nin paket yöneticisi
`-S`	Paket kur (sync)
`--needed`	Zaten kurulu ve güncel olan paketleri yeniden kurma
`base-devel`	Derleme için gereken temel araçların grubu
`mingw-w64-ucrt-x86_64-toolchain`	UCRT ortamı için 64 bitlik gcc, g++, gdb ve ilgili araçların grubu

Adım 4: gcc'yi PATH'e eklemek

MSYS2 terminalinde gcc'yi çalıştırabiliriz. Ancak ileride VS Code'un terminalinden, PowerShell'den ya da Komut İstemi'nden de çalıştırmak isteyeceğiz. Bunun için gcc'nin bulunduğu klasörü **PATH**'e eklemeliyiz.

> **Tanım: PATH**
>
> İşletim sisteminin, yazdığımız bir komutun hangi klasörlerde aranacağını belirlemek için kullandığı klasör listesidir. Bu liste bir **ortam değişkeni**dir (environment variable). `gcc` yazdığımızda sistem, PATH'teki klasörleri sırayla tarar ve `gcc.exe` dosyasını bulduğu ilk yerde çalıştırır. Bulamazsa "komut tanınmıyor" ya da "komut bulunamadı" der.

Ekleyeceğimiz klasör, MSYS2'yi varsayılan yere kurduysak şudur:

```
C:\msys64\ucrt64\bin
```

Adımlar:

1. Windows aramasına "ortam değişkenleri" (environment variables) yazın ve hesabınız için ortam değişkenlerini düzenlemeyi sunan sonucu açın.
2. Üstteki "Kullanıcı değişkenleri" (User variables) listesinde `Path` satırını seçip Düzenle'ye (Edit) basın.
3. Yeni (New) ile yukarıdaki klasör yolunu yeni bir satır olarak ekleyin.
4. Açık pencereleri Tamam (OK) ile kapatın.

PATH değişikliği, değişiklikten sonra açılan terminallere yansır. Açık duran terminalleri, PowerShell pencerelerini ve VS Code'u kapatıp yeniden açmak gerekir.

> ◆ **Kritik Nokta · PATH değişikliğinden sonra terminali yeniden açın**
>
> PATH'e eklemeyi yapmak yetmez; değişiklik, yalnızca sonradan başlatılan programlara aktarılır. Kurulumdan sonra "gcc tanınmıyor" hatasının en yaygın nedeni, eski bir terminal ya da editör penceresinin açık bırakılmasıdır.

Adım 5: Kurulumu doğrulamak

Yeni bir PowerShell ya da Komut İstemi penceresi açıp sürümleri sorgulayalım:

```
TERMİNAL
$ gcc --version
$ g++ --version
$ gdb --version
```

Her komut, sürüm bilgisini içeren birkaç satır yazar. İlk satırda araç adını (`gcc.exe`, `g++.exe`) ve bir sürüm numarasını görmeliyiz:

```
TERMİNAL
gcc.exe (Rev..., Built by MSYS2 project) ...
Copyright (C) ... Free Software Foundation, Inc.
```

Sürüm numaralarının bizimkinden farklı olması normaldir; MSYS2 depoları sürekli güncellenir ve her kurulum o günün sürümünü getirir. Önemli olan komutların bulunmasıdır; güncel MSYS2 paketleri kitabın kullandığı C17 ve C++17 standartlarını rahatça destekler.

> *Tarihçe · MinGW'nin devamı: MinGW-w64*
>
> İlk MinGW projesi (Minimalist GNU for Windows), gcc'yi Windows'a taşımak için başlatılmıştı. 64 bit Windows desteği ve yeni Windows arayüzlerini kapsama gereksinimi ile ortaya çıkan MinGW-w64 projesi bunun bağımsız bir devamıdır. Bugün "MinGW" denildiğinde çoğunlukla MinGW-w64 kastedilir.

3.3 Linux ve macOS

Linux

Linux dağıtımlarında gcc, dağıtımın paket yöneticisiyle birkaç saniyede kurulur. Kurulumdan sonra ekstra bir PATH ayarı gerekmez.

Ubuntu, Debian ve türevlerinde `build-essential` paketi gcc'yi, g++'ı ve `make`'i birlikte kurar:

```
TERMİNAL
$ sudo apt update
$ sudo apt install build-essential gdb
```

Fedora'da karşılığı şudur:

```
TERMİNAL
$ sudo dnf install gcc gcc-c++ make gdb
```

`sudo` komutu yönetici yetkisiyle çalıştırır ve parolamızı ister; parola yazarken ekranda bir şey görünmemesi normaldir. Ardından kurulumu doğrularız:

```
TERMİNAL
$ gcc --version
$ g++ --version
```

Dağıtımın depolarındaki gcc sürümü dağıtımın sürümüne göre değişir; yaygın dağıtımlarda kitabın gerektirdiği C17 ve C++17 desteği zaten vardır.

macOS

macOS'ta derleyici, Apple'ın komut satırı geliştirici araçlarıyla (Command Line Tools) gelir. Terminal uygulamasını açıp şunu yazalım:

```
TERMİNAL
$ xcode-select --install
```

Açılan pencerede kurulumu onaylarız. Bu işlem derleyiciyi, `make`'i, `git`'i ve hata ayıklayıcıyı (`lldb`) kurar; Xcode'un tamamını indirmek gerekmez.

> **▲ Sık Yapılan Hata · macOS'ta gcc aslında clang'dir**
>
> macOS'ta `gcc` ve `g++` komutları GNU gcc'yi değil, Apple'ın clang derleyicisini çalıştırır. `gcc --version` çıktısında "Apple clang" ifadesi görünür. Bu derleyici kitaptaki bütün programları derler; ancak uyarı ve hata mesajlarının sözcükleri gcc'den biraz farklı olabilir. Mesajın düzeni (dosya, satır, sütun, açıklama) aynıdır. GNU gcc'nin kendisini kullanmak isteyenler Homebrew paket yöneticisiyle kurabilir; bu durumda komutun adı sürüm numarası taşır (ör. `gcc-15`).

3.4 VS Code Kurulumu ve Eklentiler

Visual Studio Code (VS Code), Microsoft'un ücretsiz ve çok platformlu kod editörüdür. Windows, Linux ve macOS'ta aynı şekilde çalışır. Birkaç eklentiyle üç dilin ihtiyacını da karşıladığı için kitap boyunca onu kullanacağız.

Kurulum

1. `code.visualstudio.com` adresinden işletim sisteminize uygun yükleyiciyi indirin ve çalıştırın.
2. Windows'ta kurulum sırasında "PATH'e ekle" (Add to PATH) seçeneği işaretliyse VS Code'u terminalden `code .` komutuyla da açabilirsiniz. Bu seçenek varsayılan olarak işaretlidir.

3. VS Code'u açın. Dil ve görünüm seçenekleri istenirse varsayılanları kabul edin.

Eklentiler

VS Code, yalnızca düz metin düzenler; dil desteğini **eklentiler** (extensions) ekler. Sol taraftaki çubuktan Extensions görünümünü açalım (Ctrl+Shift+X; macOS'ta Cmd+Shift+X) ve arama kutusuna adını yazarak eklentiyi bulup Install düğmesine basalım.

Eklenti	Yayıncı	Ne sağlar?	Hangi dil için?
C/C++	Microsoft	Renklendirme, kod tamamlama, hata altı çizme, `gdb` / `lldb` ile hata ayıklama	C ve C++
C# Dev Kit	Microsoft	C# desteği, proje gezgini, hata ayıklama (C# eklentisini otomatik getirir)	C#

> **▲ Sık Yapılan Hata · Eklenti derleyici değildir**
>
> C/C++ eklentisi kodu renklendirir ve hatalarını gösterir, ama derleme yapmaz. Derleyiciyi daha önce kurduk (MSYS2, `build-essential` ya da Command Line Tools). Aynı şekilde C# Dev Kit de .NET SDK yerine geçmez; SDK ayrı kurulur. Yayıncı adının "Microsoft" olduğunu kontrol edin; benzer adlı başka eklentiler de vardır.

C# Dev Kit'in kullanım koşulları eklentinin sayfasında yazılıdır. Bireysel öğrenme amacıyla kullanım için ek bir ücret gerekmez; kurumsal kullanımın koşulları farklı olabilir. Dev Kit istemeyen okurlar yalnızca "C#" eklentisiyle de kod yazıp çalıştırabilir; bu kitaptaki örnekler için yeterlidir.

Klasör açma ve ilk dosya

VS Code, tek tek dosyalardan çok **klasörlerle** çalışır. Önceki başlıkta oluşturduğumuz `cdersleri` klasörünü açalım:

1. File menüsünden Open Folder'ı seçelim ve `cdersleri` klasörünü gösterelim. "Bu klasördeki dosyalara güveniyor musunuz?" diye sorulursa onaylayalım.
2. Sol taraftaki Explorer görünümünde yeni dosya düğmesine basıp `merhaba.c` adını verelim. Uzantı önemlidir: C için `.c`, C++ için `.cpp`, C# için `.cs`.
3. Terminal menüsünden New Terminal'i seçelim (kısayolu Ctrl+`). Alt kısımda, açık klasörde başlayan bir terminal paneli görünür.

Windows'ta VS Code terminali varsayılan olarak PowerShell açar. gcc'yi PATH'e eklediysek bu terminalden `gcc --version` çalışır.

Türkçe karakterler ve UTF-8

Programlarımız ekrana "Merhaba, Dünya!" gibi Türkçe karakterli metinler yazdıracak. Bu karakterlerin sağlıklı görünmesi için iki halkanın uyumlu olması gerekir: kaynak dosyanın kodlaması ve terminalin kodlaması.

- **Kaynak dosya**: VS Code dosyaları varsayılan olarak UTF-8 kaydeder. Pencerenin sağ altındaki durum çubuğunda kodlamanın `UTF-8` olduğunu görebiliriz. Değilse oradaki kodlama adına tıklayıp "Save with Encoding" ile UTF-8'i seçeriz.
- **Terminal (Windows)**: Komut İstemi ve PowerShell geleneksel olarak Türkçe karakterleri farklı bir kod sayfasıyla gösterebilir. Çıktıda `ü`, `ş` yerine bozuk karakterler görürsek terminalin kod sayfasını UTF-8'e alırız:

```
TERMİNAL
```

```
$ chcp 65001
```

Bu komut yalnızca açık olan terminal oturumunu etkiler; yeni bir terminal açtığımızda yeniden yazmak gerekir. Çıktıda `Active code page: 65001` benzeri bir satır görürüz (Türkçe Windows'ta ifade Türkçe olabilir).

Windows Terminal ya da VS Code'un tümleşik terminali kullanılırken yazı tipi sorunu çıkmaz; eski konsol penceresi bazı karakterleri yazı tipinin eksikliğinden gösteremeyebilir. Linux ve macOS terminalleri varsayılan olarak UTF-8 kullanır, orada ek ayar gerekmez.

C# için ek bir not: .NET konsol uygulamaları çıktıyı terminalin kod sayfasıyla yazar. `chcp 65001` bu durumda da çözümdür. Alternatif olarak programın başına `Console.OutputEncoding = System.Text.Encoding.UTF8;` satırı eklenebilir; ama kitabın örneklerinde buna gerek bırakmayacak bir ortam kurmayı tercih ediyoruz.

> **İpucu · VS Code'un çalıştır düğmesi ve -Wall**
>
> VS Code'un sağ üstündeki Run düğmesi bir C/C++ dosyasını derleyip çalıştırabilir. Ancak varsayılan olarak `-Wall` gibi uyarı seçeneklerini eklemez ve hangi C standardıyla derleneceğini bize sormaz. Bu kitapta derlemeyi terminalde elle yapacağız: komutu kendimiz yazarsak her bilgisayarda aynı sonucu alırız ve derleyiciye ne istediğimizi tam olarak söylemiş oluruz. Düğmeyi kullanmak isteyenler, eklentinin oluşturduğu `.vscode/tasks.json` dosyasındaki `args` listesine `-std=c17`, `-Wall` ve `-Wextra` seçeneklerini ekleyebilir.

3.5 Visual Studio Community

Visual Studio, Microsoft'un yalnızca Windows için hazırladığı tam donanımlı IDE'sidir. Editörü, derleyicisi, hata ayıklayıcısı, proje yönetimi ve tasarım araçları tek yükleyiciyle gelir. **Community** sürümü bireysel geliştiriciler, öğrenciler ve açık kaynak projeler için ücretsizdir; kurumsal kullanımın lisans koşulları farklıdır ve yükleyici sayfasında yazılıdır.

Visual Studio'yu `visualstudio.microsoft.com` adresinden indiririz. Yükleyici, kurulacak bileşenleri **iş yükleri** (workloads) olarak gruplar. Bizim iki iş yüküne ihtiyacımız var:

İş yükü	Ne için?	Ne getirir?
C++ ile masaüstü geliştirme (Desktop development with C++)	C ve C++	MSVC derleyicisi, Windows SDK, C++ proje şablonları
.NET masaüstü geliştirme (.NET desktop development)	C#	.NET SDK ve konsol uygulaması şablonları

Yükleyicideki iş yükü adları kurulum diline göre Türkçe ya da İngilizce görünür. Hangisini seçeceğimizi karıştırmamak için yukarıdaki parantez içi İngilizce adlara bakabiliriz.

Visual Studio ile gelen C/C++ derleyicisi **MSVC**'dir (`cl.exe`); gcc değildir. İki derleyici de C ve C++ standartlarını destekler, ancak uyarı ve hata mesajları farklıdır; ayrıca MSVC'nin C desteği gcc'ye göre bazı yönlerden daha kısıtlıdır. Bu yüzden kitaptaki C ve C++ çıktılarını birebir görmek istiyorsak gcc ile çalışmak daha uygundur. Visual Studio'nun C# tarafında ise böyle bir ayrım yoktur; C# derleyicisi .NET SDK ile aynıdır.

Visual Studio ne zaman tercih edilir?

- **C# öğrenirken Windows kullanıyorsak**: Proje oluşturma, derleme ve hata ayıklama tek tuşla yapılır; hata listesi, kod tamamlama ve düzeltme önerileri çok gelişmiştir.

- **Büyük çözümlerde ve Windows masaüstü uygulamalarında**: Formlar, tasarımcılar ve profil oluşturma araçları gerektiğinde.
- **Hata ayıklayıcıyı görsel olarak güçlü bir ortamda öğrenmek istediğimizde.**

Hafif, çok platformlu ve her üç dilde tek tip çalışan bir ortam istiyorsak VS Code daha uygundur. Bu kitapta iki seçenek de geçerlidir; terminal komutları ise her ikisinde aynı çalışır.

Visual Studio'da ilk C# projesi

1. Başlangıç penceresinden yeni bir proje oluşturmayı seçelim (Create a new project).
2. Şablon listesinde **Console App** şablonunu seçip ileri gidelim. Adında ".NET Framework" geçen şablon yalnızca Windows'a özgü eski .NET Framework içindir; onu seçmeyin.
3. Proje adına `Merhaba` yazalım.
4. Framework olarak listedeki en yeni LTS sürümü (**.NET 10.0**) seçelim. Listede yoksa .NET SDK ayrıca kurulmalıdır (bir sonraki başlık).
5. "Do not use top-level statements" kutusunu işaretlemeyelim. Böylece dosya doğrudan kod satırıyla başlar; kitabın C# örnekleri bu biçimdedir.
6. Create'e basalım. Açılan `Program.cs` dosyasına kodu yazıp Ctrl+F5 ile çalıştıralım.

Ctrl+F5 programı hata ayıklayıcı olmadan çalıştırır ve bittiğinde konsol penceresini bir tuşa basılana kadar açık tutar. F5 ise hata ayıklayıcıyla çalıştırır. Visual Studio'da menü adları kurulum diline göre değişir, ama adımların sırası aynıdır.

C ya da C++ için aynı şablon listesinde "Console App" adlı C++ şablonunu seçeriz. Ancak yukarıda belirttiğimiz nedenlerle bu kitabın C ve C++ derlemelerini gcc ile yapacağız.

3.6 .NET SDK Kurulumu

C# programlarını derlemek ve çalıştırmak için .NET SDK gerekir. Visual Studio'nun .NET iş yükü SDK'yı kendisi getirir. VS Code ile çalışacaksak (ya da terminalden derlemek istiyorsak) SDK'yı ayrı kurmalıyız.

Bu kitabın C# örnekleri **C# 12** ile yazılmış ve **.NET 8** üzerinde denenmiştir. .NET 8'in desteği Kasım 2026'da sona erdiği için yeni kurulumlarda güncel uzun süreli destekli (LTS) sürüm olan **.NET 10**'u öneriyoruz; kitaptaki bütün örnekler .NET 10'da da değişiklik gerekmeden çalışır.

Kurulum

- **Windows**: `dotnet.microsoft.com/download` adresinde .NET 10 için "SDK" yükleyicisini (Runtime değil) indirip çalıştırın. Komut satırından kurmayı tercih edenler Windows'un paket yöneticisini kullanabilir: `winget install Microsoft.DotNet.SDK.10`.
- **macOS**: Aynı sayfadaki macOS SDK yükleyicisini (işlemci mimarisine uygun olanı) çalıştırın.
- **Linux**: Dağıtımın paket yöneticisi genellikle yeterlidir. Ubuntu için `sudo apt install dotnet-sdk-8.0`, Fedora için `sudo dnf install dotnet-sdk-8.0` komutlarını kullanabiliriz. Dağıtımın deposunda paket yoksa Microsoft'un dağıtıma özel kurulum yönergelerine bakılır.

SDK ve **Runtime** farklı şeylerdir: Runtime yalnızca hazır bir .NET programını çalıştırır; SDK ise derleyiciyi ve geliştirme araçlarını da içerir. Biz SDK'ya ihtiyaç duyarız.

Tanım: SDK

Yazılım geliştirme seti (Software Development Kit). Bir platform için program geliştirmeye yarayan derleyici, kütüphane ve komut satırı araçlarının bir arada dağıtılan paketidir. .NET SDK, C# derleyicisini ve `dotnet` komutunu içerir.

Doğrulama

Yeni bir terminal açıp sürümü soralım:

```
TERMİNAL
$ dotnet --version
8.0.xxx
```

Çıktının sonundaki sayılar kurulan güncellemeye göre değişir; ilk sayının 8 olması yeterlidir. Makinede birden çok SDK kuruluysa hangilerinin bulunduğunu şu komut gösterir:

```
TERMİNAL
$ dotnet --list-sdks
```

Not · Sürümler hakkında

Microsoft her yıl Kasım ayında yeni bir .NET sürümü yayımlar; çift numaralı sürümler (8, 10, ...) uzun süreli desteklidir. Güncel destek takvimi `dotnet.microsoft.com` adresinde yayımlanır. `dotnet new console` komutu kurulu SDK'nın sürümünü hedefleyen bir proje oluşturur (ör. `net10.0`). Kitabın örnekleri bu durumda da çalışır, çünkü C# yeni sürümlerde eski kodları kırmayacak şekilde ilerler.

dotnet komutları ve proje yapısı

.NET dünyasında hemen her iş `dotnet` komutuyla yapılır. Başlangıçta üç komut işimizi görür:

Komut	Ne yapar?
`dotnet new console -o Merhaba`	`Merhaba` klasöründe yeni bir konsol projesi oluşturur
`dotnet build`	Projeyi derler
`dotnet run`	Gerekirse derler, ardından programı çalıştırır

`dotnet new console -o Merhaba` komutu bir klasör ve içinde iki dosya üretir:

```
Merhaba/
    Merhaba.csproj
    Program.cs
```

`Program.cs` programın kaynak kodudur. `Merhaba.csproj` ise **proje dosyasıdır**: derleyiciye projenin nasıl derleneceğini anlatan bir XML dosyasıdır. .NET 10 şablonunun ürettiği içerik şöyledir:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
```

```
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Bu dosyada üç ayar önemlidir:

- `TargetFramework`: Hangi .NET sürümü için derleneceğini belirtir (`net10.0`).
- `ImplicitUsings`: `System`, `System.Linq` gibi sık kullanılan ad alanlarını (namespace) her dosyada kendiliğinden tanımlı kılar. Bu yüzden `Console.WriteLine` yazarken dosyanın başında `using System;` bulunmaz. Kitabın örneklerinde `using` satırı yazılması da zararsızdır.
- `Nullable`: Boş (null) değerlerle ilgili olası hataları derleyicinin uyarmasını sağlar. Bu konuyu C# kısmında ayrıntılı göreceğiz.

`dotnet build` ya da `dotnet run` komutu çalıştırıldığında klasörde iki yeni klasör oluşur:

```
Merhaba/
    Merhaba.csproj
    Program.cs
    bin/
        Debug/net10.0/
            Merhaba.dll
            Merhaba.exe      (Windows'ta; Linux ve macOS'ta uzantısız Merhaba)
            ...
    obj/
```

`bin` klasörü derlemenin ürünlerini, yani çalıştırılabilir çıktıyı içerir. `obj` ise derleme sırasındaki ara dosyalar içindir. İkisi de otomatik üretilir; elle düzenlemeyiz ve silinirse bir sonraki derlemede yeniden oluşur. `Debug` klasör adı, varsayılan yapılandırmanın hata ayıklama için uygun olan "Debug" olduğunu gösterir.

> **▲ Sık Yapılan Hata · dotnet komutlarını doğru klasörde çalıştırın**
>
> `dotnet run` ve `dotnet build`, komutun yazıldığı klasörde bir `.csproj` dosyası arar. Proje klasörünün bir üstündeyken çalıştırırsak şuna benzer bir hata alırız:
>
> ```
> TERMİNAL
> $ dotnet run
> MSBUILD : error MSB1003: Specify a project or solution file. The current working directory
> does not contain a project or solution file.
> ```
>
> Çözüm basittir: `cd Merhaba` ile proje klasörüne girip komutu yeniden çalıştırırız.

3.7 İlk Programlar: Üç Dilde Merhaba, Dünya!

Geleneksel ilk program ekrana bir selam yazar. Kurulumumuzu sınamak için aynı programı üç dilde yazalım. Her dilin ayrıntılarını ilgili bölümlerde göreceğiz (C için Bölüm 4); şimdilik amacımız derleme ve çalıştırma düzenini tanımaktır.

C

Çalışma klasörümüzde `merhaba.c` dosyasını oluşturup şunu yazalım:

merhaba.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba, Dünya!\n");
    return 0;
}
```

Derlemek için terminalde şu komutu kullanırız:

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra merhaba.c -o merhaba
```

Komutun parçaları şunlardır:

Parça	Anlamı
`gcc`	C derleyicisini çalıştır
`-std=c17`	C17 standardına göre derle
`-Wall -Wextra`	Önerilen uyarıların çoğunu aç
`merhaba.c`	Derlenecek kaynak dosya
`-o merhaba`	Çıktı dosyasının adı (output)

Komut sessizce biter ve klasörde yeni bir dosya belirir: Windows'ta `merhaba.exe`, Linux ve macOS'ta `merhaba`. Çalıştırmak için dosya adını yazarız. Windows PowerShell'de ve Linux/macOS'ta geçerli klasördeki programı çalıştırmak için başına `.\` ya da `./` koymak gerekir; çünkü geçerli klasör güvenlik nedeniyle komut aramasına kendiliğinden dahil edilmez.

TERMİNAL

```
$ ./merhaba
Merhaba, Dünya!
```

Windows PowerShell'de aynı komut `.\merhaba` biçiminde yazılır; Komut İstemi'nde (cmd) yalnızca `merhaba` yazmak yeter.

C++

C++ için dosyanın uzantısı `.cpp` olur ve derleyici `g++`'tır:

merhabaCpp.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Merhaba, Dünya!\n";
    return 0;
}
```

TERMİNAL

```
$ g++ -std=c++17 -Wall -Wextra merhabaCpp.cpp -o merhabaCpp
$ ./merhabaCpp
Merhaba, Dünya!
```

Komutlar `gcc` komutuna çok benzer; yalnızca derleyici adı `g++` ve standart `-std=c++17` olur.

C#

C# programı bir proje ile yaşar. Önce proje oluşturalım, ardından klasöre girip çalıştıralım:

```
TERMİNAL
$ dotnet new console -o MerhabaCs
$ cd MerhabaCs
```

Şablon `Program.cs` dosyasına tek bir `Console.WriteLine` satırı yazar. Bu dosyanın içeriğini silip yerine şunu yazalım:

▸ **Program.cs**

```
Console.WriteLine("Merhaba, Dünya!");
```

```
TERMİNAL
$ dotnet run
Merhaba, Dünya!
```

Satırı yazmak için `using System;` ya da bir `Main` metodu gerekmedi: C# 9'dan itibaren kullanılabilen **üst düzey ifadeler** (top-level statements) sayesinde program doğrudan ifadelerle başlayabilir. Bunu C# kısmında ayrıntılı göreceğiz.

> **Not · Kitaptaki C# örneklerini çalıştırmak**
>
> Kitabın C# örnekleri `Program.cs` içeriğidir. Bir örneği denemek için `dotnet new console -o Deneme` ile yeni bir proje oluşturur, `Program.cs` dosyasının içeriğini örnekle değiştirir ve proje klasöründe `dotnet run` komutunu çalıştırırız. Aynı projeyi birçok örnek için tekrar tekrar kullanabiliriz; her yeni örnek eskisinin yerine yazılır. Dosya adı başlığında `Program.cs` dışında bir ad görünen örnekler tek dosyalık program olarak düşünülmelidir; içeriği `Program.cs` 'e yazmak yeterlidir.

> **Üç Dilde · Aynı iş, üç ortam** *C · C++ · C#*
>
> Üç dilin ilk programı, yazılış ve çalıştırılış biçimiyle derleme modellerini de yansıtır.

	C	C++	C#
Dosya	`merhaba.c`	`merhabaCpp.cpp`	`Program.cs` (bir proje içinde)
Derleme	`gcc ... -o merhaba`	`g++ ... -o merhabaCpp`	`dotnet build` (ya da `dotnet run` ile birlikte)
Çalıştırma	`./merhaba`	`./merhabaCpp`	`dotnet run`
Sonuç	Makine kodu (`.exe`)	Makine kodu (`.exe`)	IL kodu (`.dll`), çalışırken JIT ile makine koduna
Başlık	`#include <stdio.h>`	`#include <iostream>`	Gerekmez (`ImplicitUsings`)

C ve C++ doğrudan işlemcinin çalıştırdığı bir dosya üretir. C# ise önce ara koda (IL) derlenir; bu kod .NET çalışma zamanı tarafından çalıştırılır (Bölüm 1). `dotnet run` bu iki adımı arka planda birleştirir.

Konsol penceresi açılıp kapanıyor mu?

IDE'lerde ya da çift tıklayarak çalıştırılan `.exe` dosyalarında program kısa bir pencere açıp hemen kapanabilir. Program işini bitirip çıktığı için pencere de kapanır. Programları terminalde çalıştırmak bu sorunu kökten çözer: pencere bizim, program bittiğinde çıktı ekranda kalır. Visual Studio'da ise Ctrl+F5 kullanılırsa pencere bir tuşa basılana kadar açık kalır.

3.8 Çevrim İçi Derleyiciler

Hiçbir şey kurmadan bir kodu denemek istediğimiz zamanlar olur: kısa bir fikri sınamak, başkasının bilgisayarından çalışmak ya da kurulum sorununu çözerken yedek bir yol bulmak. Bu durumlarda tarayıcıda çalışan **çevrim içi derleyiciler** kullanılabilir. Bir dil seçilir, kod yazılır, Run düğmesine basılır ve çıktı sayfada görünür.

Çevrim içi derleyiciler kısa denemeler için pratiktir, ama sınırlıdır:

- Derleyici sürümünü ve seçeneklerini (`-std`, `-Wall`) site belirler; kitabın çıktılarından farklı sonuç görebiliriz.
- Klavyeden girdi veren programlar çoğu sitede ayrı bir girdi kutusu ister.
- Dosya işleme, çok dosyalı programlar ve büyük projeler için uygun değildir.
- Çoğunda zaman ve bellek sınırı vardır; uzun süren programlar kesilir.

Asıl çalışmamızı kendi bilgisayarımızda kurduğumuz araçlarla yapmalıyız. Böylece derleme seçeneklerini, hata mesajlarını ve dosya düzenini kontrol edebilir, internet bağlantısına bağlı kalmayız.

3.9 Sık Kurulum Sorunları

Kurulum sırasında karşılaşılan sorunların çoğu birkaç nedene dayanır. Aşağıda en sık görülenleri, belirtileri ve çözümleriyle derledik.

"Komut bulunamadı" ya da "tanınmıyor" hatası

Belirti: `gcc --version` ya da `dotnet --version` yazınca terminal komutu bulamadığını bildirir.

```
TERMİNAL
$ gcc --version
gcc : The term 'gcc' is not recognized as the name of a cmdlet, function, script file, or opera
ble program.
```

Komut İstemi'nde mesaj `'gcc' is not recognized as an internal or external command...`, Linux ve macOS'ta ise `gcc: command not found` biçimindedir. Türkçe Windows'ta mesaj Türkçe de olabilir; anlamı aynıdır. Nedenlerini sırayla kontrol edelim:

1. **Terminal PATH'ten önce açılmıştı.** Terminali (ve VS Code'u) kapatıp yeniden açalım.
2. **PATH'e yanlış klasör eklenmiş.** `C:\msys64\ucrt64\bin` klasörünü Dosya Gezgini'nde açıp içinde `gcc.exe` dosyasının bulunduğunu doğrulayalım. Başka bir klasör (örneğin `C:\msys64\mingw64\bin`) eklendiyse o ortamın araçları kurulu olmayabilir.

3. **Derleyici hiç kurulmamış.** MSYS2 yüklendi ama `pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain` komutu çalıştırılmamış olabilir. Bu durumda `C:\msys64\ucrt64\bin` klasörü ya yoktur ya da `gcc.exe` içermez.
4. **Komut yanlış terminalde çalıştırılıyor.** `pacman` komutu yalnızca MSYS2 terminalinde vardır; PowerShell'de çalışmaz. Tersine, PATH ayarı yapılmadan gcc yalnızca MSYS2 UCRT64 terminalinde bulunur.

Dosya bulunamıyor

Belirti: Derleme komutu dosyayı bulamadığını söyler.

```
TERMİNAL
$ gcc merhaba.c -o merhaba
gcc: error: merhaba.c: No such file or directory
gcc: fatal error: no input files
compilation terminated.
```

Terminal, dosyanın bulunduğu klasörde değildir ya da dosya adı yanlıştır. `pwd` ile bulunduğumuz klasörü, `dir` ya da `ls` ile klasörün içeriğini kontrol edelim. Dosya adı görünür ama komut yine hata veriyorsa uzantıların gizli olup olmadığına bakalım: `merhaba.c.txt` gibi çift uzantılı bir dosya oluşmuş olabilir. Windows'ta Dosya Gezgini'nde dosya uzantılarını göstermeyi açmak bu karışıklığı önler.

Türkçe karakterli ya da boşluklu klasör yolu

Kullanıcı adı veya klasör adı `ş`, `ı`, `ğ` gibi karakterler ya da boşluk içeriyorsa bazı araçlar yolu yanlış yorumlayabilir; derleme sırasında "dosya bulunamadı" benzeri anlaşılması zor hatalar çıkabilir. Çözüm, çalışma klasörünü sade bir yolda tutmaktır: `C:\kitap\c`, `C:\Users\Adiniz\cdersleri` gibi yalnızca ASCII harf ve rakamdan oluşan, boşluksuz bir yol. Kullanıcı adının kendisi Türkçe karakter içeriyorsa projeleri `C:\` altında bir klasöre koymak en güvenlisidir.

Antivirüs programı ve çalıştırılabilir dosyalar

Yeni derlenen, imzasız `.exe` dosyaları bazı güvenlik yazılımlarınca şüpheli bulunup karantinaya alınabilir ya da çalıştırılması engellenebilir. Belirtisi, derleme başarılı olduğu halde `.exe` dosyasının birden kaybolması ya da "erişim engellendi" hatasıdır. Böyle bir durumda güvenlik yazılımının kayıtlarına bakarız. Çalışma klasörünü güvenlik yazılımının taramasından hariç tutmak bir çözüm olabilir; ancak bu ayar yalnızca kendi oluşturduğumuz ve güvendiğimiz bir klasör için yapılmalıdır. Bir program hâlâ çalışıyorsa (pencere açık ya da arka planda) aynı adlı `.exe` dosyası yeniden derlenemez; önce programı kapatmak gerekir.

Program çalışıp hemen kapanıyor

Bir önceki başlıkta açıkladığımız gibi, çift tıklanan programın penceresi iş bitince kapanır. Programı terminalden çalıştırın. Visual Studio'da Ctrl+F5 kullanın.

Türkçe karakterler bozuk görünüyor

Çıktıda `Dünya` yerine `DÃ¼nya` ya da `D�nya` görüyorsak kodlama uyuşmazlığı vardır. Kaynak dosyanın UTF-8 olduğunu doğrulayalım, ardından terminalde `chcp 65001` komutunu çalıştıralım (yukarıdaki UTF-8 başlığına bakınız).

dotnet restore hatası

Belirti: `dotnet run` ya da `dotnet new` sırasında "restore failed" benzeri bir hata ve ağ bağlantısına işaret eden bir mesaj. .NET projeleri ilk derlemede bazı paketleri geri yükler (restore) ve bu adım internet bağlantısı gerektirebilir. Bağlantıyı, kurum ağı kısıtlamalarını ve vekil sunucu (proxy) ayarlarını kontrol edip komutu yeniden çalıştırmak çoğu zaman yeterlidir.

> **İpucu · Sorun giderirken ilk hatayı okuyun**
>
> Derleyici birden çok hata verirse yalnızca ilkine bakın. Sonraki hataların çoğu, ilkinin yan etkisidir. Hata mesajının ilk satırını sözcüğü sözcüğüne okumak, çoğu kurulum sorununu çözer: mesaj hangi dosyayı, hangi satırı ve neyi bulamadığını söyler.

3.10 Hata Ayıklayıcıya İlk Bakış

Programlar ilk seferde çoğu zaman beklediğimiz gibi çalışmaz. Derleme hatasını derleyici gösterir, ancak programın derlenip yanlış sonuç vermesi daha zor yakalanır (Bölüm 1'deki mantık hataları). Bu durumda en güçlü araç **hata ayıklayıcıdır** (debugger).

> **Tanım: Kesme noktası**
>
> Hata ayıklayıcıya "programı şu satırda durdur" demek için koyduğumuz işarettir (breakpoint). Program bu satıra gelince çalışmayı bırakır; değişkenlerin o anki değerlerini inceleyebilir ve programı adım adım ilerletebiliriz.

Hata ayıklayıcıyı denemek için kasıtlı bir hata içeren küçük bir program yazalım. Program 1'den 6'ya kadar sayıların ortalamasını hesaplamak istiyor (beklenen sonuç 3,5):

▸ ortalamaHata.c

```c
#include <stdio.h>

int main(void) {
    int toplam = 0;
    for (int i = 1; i <= 6; i++) {
        toplam += i;
    }
    int adet = 6;
    double ortalama = toplam / adet;
    printf("Ortalama: %.2f\n", ortalama);
    return 0;
}
```

ÇIKTI

```
Ortalama: 3.00
```

Program derlendi, uyarı vermedi, ama sonuç 3,50 olmalıydı. Hatanın nerede olduğunu bulmak için bu programı hata ayıklayıcıyla izleyelim.

Hazırlık: hata ayıklama bilgisi

Hata ayıklayıcının satırlarla ve değişken adlarıyla çalışabilmesi için derleyicinin çıktıya bu bilgiyi eklemesi gerekir. gcc'de bunu `-g` seçeneği yapar:

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra -g ortalamaHata.c -o ortalamaHata
```

VS Code'da C/C++ eklentisinin hata ayıklama için oluşturduğu derleme görevi bu seçeneği zaten ekler. .NET projeleri varsayılan olarak Debug yapılandırmasıyla derlenir, ek bir ayar gerekmez.

Adım adım izlemek

VS Code'da dosyayı açıp aşağıdaki sırayı izleyelim:

1. **Kesme noktası koyalım.** `int toplam = 0;` satırının sol kenarına, satır numarasının yanına tıklarız; kırmızı bir nokta belirir. Aynı iş imleç satırdayken F9 tuşuyla da yapılır.
2. **Hata ayıklamayı başlatalım.** F5 tuşuna basarız. İlk seferde VS Code hangi derleyici ve yapılandırmayla çalışılacağını sorar; gcc'yi seçeriz. Program kesme noktasında durur ve satır vurgulanır.
3. **Adım adım ilerleyelim.** F10 ile satırı çalıştırıp bir sonrakine geçeriz.
4. **Değişkenlere bakalım.** Sol taraftaki hata ayıklama görünümünde değişkenler (Variables) bölümünde `toplam`, `adet` ve `ortalama` ile değerleri görünür. Bir değişkenin üzerine fareyi getirmek de değerini gösterir.

`for` döngüsü bittiğinde `toplam` 21 olur; `adet` ise 6'dır. `double ortalama = toplam / adet;` satırını bir adım çalıştırdığımızda `ortalama` 3 olur, çünkü iki `int` değeri bölünce sonuç da `int` çıkar ve ondalık kısım atılır. Hatanın kaynağı bu satırdır: `double` değişkene atama, bölme tamsayı olarak yapıldıktan sonra gerçekleşir. Bölüm 5'te bu davranışı ayrıntılı göreceğiz. Düzeltme için bölünenlerden birini `double`'a çevirmek yeterlidir:

```
double ortalama = (double)toplam / adet;   // ondalıklı bölme
```

Hata ayıklayıcıyla aynı bilgiyi `printf` satırları ekleyerek de bulabilirdik. Ancak hata ayıklayıcı, kodu değiştirmeden, programın herhangi bir anında tüm değişkenleri göstermesi bakımından üstündür.

Temel hata ayıklama komutları

Hata ayıklama komutlarının kısayolları VS Code'da ve Visual Studio'da aynıdır:

Eylem	Kısayol	Anlamı
Kesme noktası koy / kaldır	F9	İmlecin olduğu satıra işaret koyar ya da kaldırır
Hata ayıklamayı başlat / sürdür	F5	Programı başlatır; durmuşsa bir sonraki kesme noktasına kadar çalıştırır
Adım atla (Step Over)	F10	Satırı çalıştırır; satırda fonksiyon çağrısı varsa içine girmez
İçine gir (Step Into)	F11	Satırdaki fonksiyonun içine girer
Dışarı çık (Step Out)	Shift+F11	Geçerli fonksiyonu bitirip çağıran yere döner
Durdur	Shift+F5	Hata ayıklamayı sonlandırır
Hata ayıklayıcı olmadan çalıştır	Ctrl+F5	Programı normal biçimde çalıştırır

Bazı dizüstü bilgisayarlarda ve macOS'ta işlev tuşları ses ve parlaklık düğmesi olarak çalışır; bu durumda F tuşuyla birlikte Fn tuşuna basmak gerekebilir.

C# projelerinde yöntem aynıdır. VS Code'da C# Dev Kit kurulu bir projede F5 hata ayıklamayı başlatır; Visual Studio'da F5 ya da araç çubuğundaki başlat düğmesi aynı işi görür. Kesme noktası koymak, adım atlamak ve değişkenleri incelemek için aynı kısayollar geçerlidir. Visual Studio'da değişkenler, hata ayıklama sırasında açılan **Locals** (yerel değişkenler) ve **Watch** (izleme) pencerelerinde görünür; VS Code'da aynı bilgi Variables ve Watch bölümlerindedir. **Call Stack** (çağrı yığını) penceresi ise hangi fonksiyondan hangisine gelindiğini listeler; fonksiyonları işleyeceğimiz Bölüm 9'da bu pencere de işimize yarayacak.

> ◆ **Kritik Nokta · Hata ayıklayıcı yolu gösterir, hatayı düzeltmez**
>
> Hata ayıklayıcı programın ne yaptığını görmemizi sağlar; hatayı bulmak yine bize düşer. İyi bir alışkanlık, çalıştırmadan önce her değişkenin satır satır alması gereken değeri kafamızda ya da kâğıtta yazmak ve hata ayıklayıcıdaki gerçek değerlerle karşılaştırmaktır. İlk ayrıldıkları nokta, hatanın bulunduğu yerdir.

Bölüm Özeti

- Program yazmak için bir editör ve bir derleyici (ya da SDK) gerekir; IDE bunları ve hata ayıklayıcıyı tek pencerede toplar.
- C ve C++ için gcc ve g++, C# için .NET SDK kullanıyoruz. Kitabın çıktıları C17, C++17 ve C# 12 (.NET 8) ile alınmıştır; yeni kurulumlar için .NET 10 önerilir.
- Windows'ta gcc, MSYS2 ile kurulan MinGW-w64'tür: MSYS2 UCRT64 terminalinde `pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain` komutu çalıştırılır.
- Derleyiciyi her terminalden kullanabilmek için `C:\msys64\ucrt64\bin` klasörü PATH'e eklenir; sonrasında terminaller yeniden açılır.
- Kurulum `gcc --version`, `g++ --version` ve `dotnet --version` komutlarıyla doğrulanır.
- Linux'ta `build-essential` (Debian/Ubuntu) ya da `gcc gcc-c++` (Fedora), macOS'ta `xcode-select --install` yeterlidir. macOS'ta `gcc` komutu aslında clang'dir.
- VS Code, C/C++ ve C# Dev Kit eklentileriyle üç dilin editörü olur; eklentiler derleyici yerine geçmez.
- Türkçe karakterler için kaynak dosya UTF-8 olmalı; Windows terminalinde gerekirse `chcp 65001` çalıştırılır.
- Visual Studio Community, C# için en rahat Windows ortamıdır; "C++ ile masaüstü geliştirme" ve ".NET masaüstü geliştirme" iş yüklerini içerir. Derleyicisi gcc değil MSVC'dir.
- `dotnet new console -o Ad`, `dotnet build` ve `dotnet run` ile C# projesi oluşturulur, derlenir ve çalıştırılır; `bin` ve `obj` klasörleri otomatik üretilir.
- C ve C++ programları `gcc -std=c17 -Wall -Wextra dosya.c -o program` ve `g++ -std=c++17 -Wall -Wextra dosya.cpp -o program` komutlarıyla derlenir.
- "Komut bulunamadı" hatasının en sık nedenleri PATH ayarı, terminalin yeniden açılmaması ve yanlış klasördür.
- Hata ayıklayıcıda kesme noktası (F9), başlat/sürdür (F5), adım atla (F10) ve içine gir (F11) ile program satır satır izlenir; gcc ile `-g` seçeneği gerekir.

Alıştırmalar

1. Windows'ta MSYS2 ile derleyici kurulumunda hangi terminal kullanılmalı ve hangi klasör PATH'e eklenmelidir?

   A) MSYS2 MSYS; `C:\msys64\usr\bin`
   B) MSYS2 UCRT64; `C:\msys64\ucrt64\bin`
   C) PowerShell; `C:\Windows\System32`
   D) MSYS2 MSYS; `C:\msys64`
   E) Komut İstemi; `C:\Program Files`

2. `gcc --version` yazınca "gcc tanınmıyor" hatası alınıyor. PATH'e doğru klasör eklenmiş. Hata hâlâ sürüyorsa ilk olarak ne yapılmalıdır?

   A) Windows yeniden yüklenmelidir
   B) Dosya adı `gcc.txt` olarak değiştirilmelidir
   C) Açık terminal ve VS Code kapatılıp yeniden açılmalıdır
   D) .NET SDK kaldırılmalıdır
   E) Antivirüs programı kapatılmalıdır

3. macOS'ta `gcc --version` çıktısında "Apple clang" görünüyor. Bu ne anlama gelir?

   A) Kurulum bozuktur
   B) `gcc` komutu aslında clang derleyicisini çalıştırmaktadır
   C) Programlar derlenemez
   D) GNU gcc ayrıca kurulmuştur
   E) Xcode'un tamamı kurulmalıdır

4. .NET'te SDK ile Runtime arasındaki fark nedir?

   A) Aynı şeylerdir
   B) SDK yalnızca programı çalıştırır, Runtime derleyiciyi içerir
   C) Runtime hazır bir .NET programını çalıştırır; SDK ek olarak derleyiciyi ve geliştirme araçlarını içerir
   D) SDK yalnızca Windows'ta vardır
   E) Runtime yalnızca C için gereklidir

5. `dotnet new console -o Merhaba` komutu çalıştırıldıktan sonra hangi dosyalar oluşur? Hangi dosya kaynak kodu, hangisi proje ayarlarını içerir?
6. Aşağıdaki komutun her parçasının ne işe yaradığını açıklayın:

```
TERMİNAL
$ g++ -std=c++17 -Wall -Wextra hesap.cpp -o hesap
```

7. Windows PowerShell'de `merhaba.exe` derlendi, ama `merhaba` yazınca program çalışmıyor. Bu durumda yazılması gereken komut nedir ve neden?
8. Bir okur `dotnet run` komutunu yazınca "Specify a project or solution file" hatası alıyor. Büyük olasılıkla nedeni nedir ve nasıl çözülür?
9. Kısa program: `merhaba.c` dosyasını oluşturup ekrana "Selam, Dünya!" yazdıran bir C programı yazın. Bu programı derleyip çalıştıracak komutları (Linux/macOS biçiminde) sırasıyla yazın.

10. Kısa program: Terminalde `Selam` adında yeni bir C# konsol projesi oluşturmak, içindeki `Program.cs` dosyasına ekrana "Selam, Dünya!" yazan tek satırlık kodu koymak ve projeyi çalıştırmak için gereken komutları sırasıyla yazın.

KISIM I

C

1972 · Bell Laboratuvarları

C, 1972'de Dennis Ritchie tarafından Unix işletim sistemini yazmak için geliştirildi. Küçük, hızlı ve donanıma yakın bir dildir. Programcıya geniş bir özgürlük tanır ve karşılığında dikkat bekler. Bugün işletim sistemi çekirdeklerinden gömülü cihazlara, veritabanlarından başka dillerin yorumlayıcılarına kadar sayısız yazılımın temelinde C vardır.

Bu kısımda C'yi baştan sona öğreneceğiz: değişkenler, kontrol yapıları, fonksiyonlar, diziler, işaretçiler, dinamik bellek, yapılar, dosyalar ve temel veri yapıları. C'de öğrenilen kavramların neredeyse tamamı C++ ve C#'ta da karşımıza çıkacak.

BÖLÜM 4

C'ye Giriş: İlk Program ve Derleme Süreci

BU BÖLÜMDE

- C'nin tasarım felsefesini ve dili neden "küçük, hızlı ve donanıma yakın" diye tanımladığımızı
- İlk C programını satır satır okumayı: `#include`, `main`, `printf`, `return 0`, noktalı virgül ve süslü parantezler
- Bir kaynak dosyanın çalıştırılabilir programa dönüşme aşamalarını (önişleme, derleme, assembly, bağlama) ve `gcc -E`, `-S`, `-c`, `-o` seçeneklerini
- Derleyici uyarısı ile hata arasındaki farkı ve `-Wall -Wextra` alışkanlığını
- Yorum satırlarını, kod düzenini ve kaçış dizilerini (`\n`, `\t`, `\\`, `\"`, `%%`)
- gcc'nin hata mesajlarını okumayı ve en sık yapılan ilk hataları tanımayı
- `main` fonksiyonunun dönüş değerini ve programın çıkış kodunu

Bölüm 1'de programın, derleyicinin ve algoritmanın ne olduğunu, Bölüm 2'de C'nin Bell Labs'tan bugüne uzanan hikâyesini, Bölüm 3'te de geliştirme ortamının kurulumunu gördük. Artık elimizde çalışan bir derleyici var. Bu bölümde onu ilk kez gerçekten kullanacağız.

Amacımız yalnızca ekrana bir yazı yazdırmak değil. Bu küçük programı bahane ederek bir C programının yaşam döngüsünü baştan sona izleyeceğiz: kaynak dosyanın nasıl yazıldığını, hangi aşamalardan geçtiğini, derleyicinin bize nasıl uyarı ve hata bildirdiğini ve programın bittiğinde işletim sistemine ne söylediğini. Sonraki bölümlerde karşılaşacağımız her hata mesajını bu bölümde edinilen okuma alışkanlığıyla çözeceğiz.

4.1 C'nin Felsefesi

Bir dili öğrenmeden önce neyi hedeflediğini bilmek, kurallarını anlamayı kolaylaştırır. C'nin tasarımı birkaç ilkeye dayanır:

- **Küçük bir dil.** C'nin anahtar sözcükleri azdır ve çekirdek dil bilinçli olarak dar tutulmuştur. Giriş-çıkış, matematik, metin işleme gibi işlerin hiçbiri dilin kendisinde değil, **standart kütüphanede** (standard library) bulunur. Dil küçük olduğu için tamamı kısa sürede öğrenilebilir ve çok farklı donanımlara kolayca taşınabilir.
- **Hızlı çalışan kod.** C programı doğrudan işlemcinin makine koduna derlenir. Araya sanal makine, çöp toplayıcı ya da yorumlayıcı girmez. Yazdığımız kod ile makine komutları arasındaki ilişki öngörülebilirdir.
- **Donanıma yakınlık.** C, belleği adresleriyle, sayıları bit ve baytlarıyla görmemize izin verir. İşletim sistemi çekirdekleri, aygıt sürücüleri ve mikrodenetleyici yazılımları bu yüzden C ile yazılır.

- **Programcıya güven.** C, programcının ne yaptığını bildiğini varsayar. Bir dizinin sınırlarının dışına yazmak istersek bizi durdurmaz; kullanılmamış belleği otomatik temizlemez; her işlemin sonucunu denetleme sorumluluğunu bize bırakır.

Son ilke hem C'nin gücü hem de en büyük tehlikesidir. Güvenlik kontrollerinin çoğunu yapan diller kolay ve güvenli, ama daha yavaş ve donanımdan uzaktır. C ise kontrolü bize verir, bedeli de dikkatli yazmaktır. Kitap boyunca bu dengeyi sık sık hissedeceğiz: C'de bir şey mümkünse genellikle serbesttir, doğru olup olmadığını denetlemek bize aittir.

> **Not · C'nin kısa bir tanımı**
>
> C, **derlenen**, **yordamsal** (procedural) ve **statik tipli** bir dildir. Program fonksiyonlardan oluşur, her değişkenin tipi derleme sırasında bellidir ve kod, çalışmadan önce bir kez makine koduna çevrilir. C++ (Bölüm 21) bu temelin üzerine nesne yönelimini, C# (Bölüm 41) ise yönetilen bir çalışma ortamını ekleyecek.

4.2 İlk Program: Merhaba Dünya

Programlama geleneğine uyup her yeni dilde ilk programı ekrana bir selam yazdırmak üzere kuruyoruz. Bir metin düzenleyicide `merhaba.c` adında bir dosya oluşturalım ve içine şunu yazalım:

`merhaba.c`

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba Dünya!\n");
    return 0;
}
```

```
ÇIKTI
Merhaba Dünya!
```

Dosya uzantısının `.c` olması gerekir; derleyici dosyanın hangi dil olduğunu uzantıdan anlar. Windows varsayılan olarak uzantıları gizler, bu yüzden Not Defteri ile kaydedilen bir dosya `merhaba.c.txt` adını alabilir. Dosyayı doğrudan kod düzenleyicisinde `merhaba.c` adıyla oluşturmak bu sorunu önler.

Beş satırlık bu program C'nin neredeyse bütün temel kurallarını içerir. Şimdi satır satır inceleyelim.

`#include <stdio.h>`

`#` ile başlayan satırlar **önişlemci** (preprocessor) komutlarıdır. Bunlar derleme başlamadan önce çalışan ayrı bir adımın işidir ve C deyimi olmadıkları için sonlarına noktalı virgül konmaz. `#include` komutu, adı verilen **başlık dosyasının** (header file) içeriğini bulunduğu yere kopyalar.

`stdio.h` adı "standard input/output" ifadesinden gelir. Bu dosyada `printf` gibi giriş-çıkış fonksiyonlarının **bildirimleri** vardır: fonksiyonun adı, aldığı değerler ve döndürdüğü tip. Derleyici bir fonksiyon çağrısı gördüğünde onu nasıl çağıracağını bilmek zorundadır; bu bilgiyi başlık dosyasından alır. `<` ve `>` işaretleri, dosyanın derleyicinin standart başlık klasörlerinde aranacağını söyler. Kendi yazdığımız başlık dosyalarını tırnakla çağırmayı Bölüm 15'te göreceğiz.

`int main(void)`

`main` programın **başlangıç noktasıdır**. Program çalıştırıldığında işletim sistemi ilk olarak bu fonksiyonu çağırır. Bir C programında tam olarak bir tane `main` bulunur. Satırdaki `int`, fonksiyonun işini bitirdiğinde bir tamsayı döndüreceğini, `(void)` ise dışarıdan hiçbir değer almadığını belirtir. Bu satırı ayrıntılı olarak bölümün sonunda inceleyeceğiz; fonksiyonların kendisi ise Bölüm 9'un konusudur. Şimdilik bu kalıbı her programda aynen yazalım.

`{ ... }`

Süslü parantezler bir **blok** oluşturur. Burada `main` fonksiyonunun gövdesini çevreler: çalışmasını istediğimiz tüm satırlar `{` ile `}` arasında yer alır. Her açılan süslü parantez kapatılmak zorundadır.

`printf("Merhaba Dünya!\n");`

`printf` (print formatted), parantez içine yazılan metni ekrana basan bir **standart kütüphane fonksiyonudur**. Çift tırnak içindeki metne **metin sabiti** (string literal) denir. Sondaki `\n` yeni satır karakteridir; imleci alt satıra geçirir. Bu, fonksiyon çağrısını sonlandıran **noktalı virgüllü** bir deyimdir.

`return 0;`

`main` fonksiyonunu bitirir ve işletim sistemine `0` değerini iletir. Gelenek gereği `0`, "program başarıyla tamamlandı" anlamına gelir. Bunun ayrıntılarını bölümün sonunda göreceğiz.

Noktalı virgül

C'de bir **deyim** (statement) noktalı virgülle biter. Satır sonu derleyici için bir anlam taşımaz; deyimi bitiren işaret `;`'tür. Noktalı virgül unutulduğunda derleyici bir sonraki satırı da aynı deyimin devamı sanar ve şaşırır. Bu klasik hatayı hata mesajlarıyla birlikte göreceğiz.

Aşağıdaki tablo, programdaki her parçayı özetler:

Kod	Anlamı
`#include <stdio.h>`	Önişlemci komutu; `printf` gibi fonksiyonların bildirimlerini programa ekler
`int main(void)`	Programın başladığı fonksiyon; bir `int` döndürür, parametre almaz
`{ ... }`	Fonksiyon gövdesini çevreleyen blok
`printf("...")`	Metni ekrana yazar
`\n`	Yeni satır karakteri
`;`	Deyimi bitirir
`return 0;`	Fonksiyonu bitirir, işletim sistemine "başarılı" bilgisi verir

4.3 Derleme ve Çalıştırma

Kaynak dosya yalnızca bir metin dosyasıdır; işlemci onu doğrudan çalıştıramaz. Dosyayı çalıştırılabilir bir programa çevirmek için derleyiciye başvururuz. Terminali `merhaba.c` dosyasının bulunduğu klasörde açıp şu komutu yazalım:

```
TERMİNAL
```

```
$ gcc -std=c17 -Wall -Wextra merhaba.c -o merhaba
```

Komutu parçalarına ayıralım:

Parça	Anlamı
`gcc`	Derleyiciyi çağırır
`-std=c17`	C17 standardının kurallarına göre derle. gcc 15 ile birlikte varsayılan standart C23 oldu; kitaptaki davranışı görmek için C17'yi açıkça isteriz
`-Wall`	Yaygın şüpheli durumlar için uyarıları aç (Warnings: all)
`-Wextra`	`-Wall` 'ın kapsamadığı ek uyarıları da aç
`merhaba.c`	Derlenecek kaynak dosya
`-o merhaba`	Çıktı dosyasının adını belirle (output)

Komut sessizce biterse derleme başarılıdır; gcc sorun olmadığında bir şey söylemez. Klasörde artık `merhaba` adlı bir dosya vardır (Windows'ta `merhaba.exe`). `-o` seçeneğini yazmazsak Linux ve macOS'ta dosyanın adı `a.out`, Windows'ta `a.exe` olur.

Programı çalıştırma komutu işletim sistemine göre değişir:

Ortam	Komut
Linux, macOS	`./merhaba`
Windows PowerShell	`.\merhaba`
Windows Komut İstemi (cmd)	`merhaba`

Linux, macOS ve PowerShell güvenlik gereği geçerli klasörü komut aramasına dahil etmez. `./` ön eki "şu anki klasördeki `merhaba`" anlamına gelir. Tam oturum şöyle görünür:

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra merhaba.c -o merhaba
$ ./merhaba
Merhaba Dünya!
```

İpucu · Her seferinde aynı komutu yazmayın

Terminalde yukarı ok tuşu, son girilen komutu geri getirir. Kaynak dosyada her değişiklikten sonra "kaydet, derle, çalıştır" üçlüsünü tekrarlayacağız; bu üç adımı yukarı ok ve Enter ile hızlıca yapabiliriz. Kaydetmeyi unutmak, eski programı çalıştırıp "değişiklik etki etmiyor" yanılgısına düşmenin en yaygın nedenidir.

▲ Sık Yapılan Hata · Önce derle, sonra çalıştır

Kaynak dosyayı değiştirmek çalıştırılabilir dosyayı değiştirmez. Değişikliğin etkisini görmek için dosyayı kaydedip yeniden derlemeliyiz. Derleme hata verirse eski çalıştırılabilir dosya yerinde kalır ve `./merhaba` eski sürümü çalıştırmaya devam eder. Bu yüzden her çalıştırmadan önce derlemenin hatasız bittiğinden emin olmalıyız.

4.4 Derleme Süreci: Kaynak Koddan Programa

`gcc` komutu tek bir komut gibi görünse de arka planda dört ayrı aşamayı sırayla çalıştırır:

```
 merhaba.c        merhaba.i         merhaba.s         merhaba.o        merhaba
(kaynak kod) --> (genişletilmiş) --> (assembly) -----> (nesne -------> (çalıştırılabilir
                  kaynak)             kodu)             dosyası)       dosya)
        önişlemci       derleyici         çevirici          bağlayıcı
       (preprocessor)   (compiler)        (assembler)        (linker)
                                                          + standart kütüphane
```

> **Tanım: Derleme süreci**
>
> Bir C kaynak dosyasını çalıştırılabilir programa dönüştüren zincirdir: **önişleme**, **derleme**, **assembly'ye çevirme** ve **bağlama**. Günlük dilde bütün zincire "derleme" deriz; dar anlamda derleyici yalnızca ikinci aşamadır.

Aşama	Aracı	Ne yapar?	Çıktı
1. Önişleme	önişlemci	`#` satırlarını işler: `#include` satırını dosya içeriğiyle değiştirir, yorumları siler	genişletilmiş kaynak (`.i`)
2. Derleme	derleyici	Sözdizimini denetler ve kodu assembly diline çevirir	assembly kodu (`.s`)
3. Çevirme	çevirici (assembler)	Assembly komutlarını makine koduna çevirir	nesne dosyası (`.o` , Windows'ta `.obj`)
4. Bağlama	bağlayıcı (linker)	Nesne dosyalarını ve kütüphanelerin kodunu birleştirir	çalıştırılabilir dosya

Bazı kaynaklar çeviriciyi derleyiciye dahil edip üç aşamadan söz eder. Aşamaların sayısı bir adlandırma tercihidir; yaptıkları iş aynıdır.

gcc'ye hangi aşamada duracağını söyleyebiliriz. Böylece her aşamanın çıktısını kendi gözümüzle inceleyebiliriz. Şimdi bunu sırayla yapalım.

Aşama 1: Önişleme (`-E`)

`-E` seçeneği gcc'yi önişlemeden sonra durdurur ve sonucu ekrana yazar. `-P` seçeneği de satır numarası işaretlerini bastırarak çıktıyı sadeleştirir:

```
TERMİNAL
$ gcc -E -P merhaba.c
```

Çıktı yüzlerce satır uzunluğundadır. Tek satırlık `#include <stdio.h>` , `stdio.h` dosyasının tüm içeriğiyle değişmiştir. Çıktının içinde şuna benzer satırlar bulunur:

```
extern int printf (const char *__restrict __format, ...);
```

Bu, `printf` fonksiyonunun **bildirimidir**: adı `printf` 'tir, ilk parametresi bir metindir, ardından istenen sayıda başka değer gelebilir (`...`), sonuç olarak bir `int` döndürür. Başlık dosyası fonksiyonun kodunu değil, yalnızca bu tanıtımı taşır. Çıktının sonunda ise kendi yazdığımız `main` fonksiyonu aynen durur:

```
int main(void) {
    printf("Merhaba Dünya!\n");
    return 0;
}
```

Önişlemci kodun anlamıyla ilgilenmez; yalnızca metin düzeyinde bir "kes ve yapıştır" yapar. Yorumların silindiği de bu aşamadadır.

Aşama 2: Derleme (`-S`)

`-S` seçeneği derleyiciyi assembly aşamasından sonra durdurur ve `merhaba.s` dosyasını üretir:

```
TERMİNAL
$ gcc -S merhaba.c
```

Dosyanın içinde işlemcinin komutları okunabilir metin olarak yazılıdır. 64 bitlik bir Linux sisteminde, gcc 13 ile üretilen dosyadan seçilmiş satırlar şöyledir (bilgi amaçlı satırlar çıkarıldı):

```
main:
        pushq   %rbp
        movq    %rsp, %rbp
        leaq    .LC0(%rip), %rax
        movq    %rax, %rdi
        call    puts@PLT
        movl    $0, %eax
        popq    %rbp
        ret
```

Bu çıktı işlemciye, gcc sürümüne, işletim sistemine ve optimizasyon ayarlarına göre değişir; amacımız onu okumak değil, **var olduğunu görmektir**. Yine de bir ayrıntı dikkat çekicidir: `printf("Merhaba Dünya!\n")` çağrısı `puts` çağrısına dönüşmüştür. Biçim belirteci içermeyen ve `\n` ile biten bir metin için `puts` aynı işi daha ucuza yaptığından gcc bu değişikliği kendiliğinden yapar. Programın davranışı değişmez. `return 0;` satırının `movl $0, %eax` komutuna karşılık geldiğini de görebiliriz: işlemcinin `eax` yazmacına `0` konur, çağıran bu yazmaçtan dönüş değerini okur.

Aşama 3: Çevirme (`-c`)

`-c` seçeneği, önişleme, derleme ve çevirme aşamalarını yapıp bağlamadan önce durur:

```
TERMİNAL
$ gcc -c merhaba.c
```

Sonuçta `merhaba.o` adında bir **nesne dosyası** (object file) oluşur. Bu dosya artık metin değil, ikilik makine kodudur; bir metin düzenleyicide açılırsa anlamsız karakterler görürüz. Nesne dosyası henüz çalıştırılabilir değildir, çünkü içinde `puts` fonksiyonunun kodu yoktur. Yalnızca "burada `puts` kullanıldı" şeklinde bir not taşır. Linux'ta `nm` aracıyla bu notu görebiliriz:

```
TERMİNAL
$ nm merhaba.o
0000000000000000 T main
                 U puts
```

`T` işareti `main`'in bu dosyada tanımlı olduğunu, `U` (undefined) işareti ise `puts`'ın tanımının başka bir yerden bulunması gerektiğini söyler.

Aşama 4: Bağlama

Son aşamada **bağlayıcı** (linker) devreye girer. `puts`'ın makine kodu `stdio.h` içinde değil, C'nin standart kütüphanesindedir. Bağlayıcı, nesne dosyasındaki eksik referansı kütüphanedeki kodla eşleştirir ve her şeyi tek bir çalıştırılabilir dosyada birleştirir:

```
TERMİNAL
$ gcc merhaba.o -o merhaba
```

Böylece yazmadığımız bir fonksiyonu kullanabiliriz: bildirimi başlık dosyası, kodu kütüphane sağlar. Bağlayıcı bir fonksiyonun kodunu bulamazsa program oluşmaz; bu hata türünü bu bölümün ilerisinde göreceğiz.

Tüm aşamaları yaptıktan sonra sonuç, tek komutla elde ettiğimiz programın aynısıdır:

```
TERMİNAL
$ gcc -E -P merhaba.c -o merhaba.i
$ gcc -S merhaba.i -o merhaba.s
$ gcc -c merhaba.s -o merhaba.o
$ gcc merhaba.o -o merhaba
$ ./merhaba
Merhaba Dünya!
```

Ara dosyaların hepsini tek seferde görmek isteseydik `gcc -save-temps merhaba.c -o merhaba` komutu da işe yarardı. Günlük işte ara dosyalarla uğraşmayız; ama büyük projelerde `-c` seçeneğini sürekli kullanırız. Her `.c` dosyası ayrı derlenir, nesne dosyaları sonda bağlanır. Böylece değişmeyen dosyaları yeniden derlemek gerekmez. Bunu Bölüm 15'te çok dosyalı programlarla göreceğiz.

> **◆ Kritik Nokta · #include kodu getirmez, tanıtım getirir**
>
> `#include <stdio.h>` yalnızca `printf`'in **bildirimini** programa ekler. Fonksiyonun asıl kodu standart kütüphanededir ve bağlayıcı tarafından programa eklenir. Başlık dosyasını unutmak derleyicinin fonksiyonu tanımamasına, kütüphaneyi bulamamak ise bağlayıcının hata vermesine yol açar. Hataların hangi aşamada ortaya çıktığı, hangisini düzelteceğimizi belirler.

Aşağıda sık kullanacağımız gcc seçeneklerini toplu olarak görüyoruz:

Seçenek	Anlamı
`-o ad`	Çıktı dosyasının adı
`-std=c17`	Kullanılacak C standardı
`-Wall`, `-Wextra`	Uyarıları aç
`-Werror`	Uyarıları hata say; uyarı varsa derleme başarısız olur
`-E`	Yalnızca önişle
`-S`	Assembly'ye kadar derle
`-c`	Nesne dosyasına kadar derle, bağlama
`-g`	Hata ayıklayıcı için ek bilgi üret (Bölüm 3)
`-O2`	Kodu hız için optimize et

4.5 Derleyici Uyarıları

Derleyici bize iki türde ileti verir: **hata** (error) ve **uyarı** (warning). İkisinin sonucu çok farklıdır.

> **Tanım: Hata ve uyarı**
>
> **Hata**, dilin kurallarına aykırı bir durumdur; derleyici çalıştırılabilir dosya üretmez. **Uyarı** ise kodun kurallara uyduğunu, ama büyük olasılıkla bir yanlışlık içerdiğini bildirir; derleme sürer ve program oluşur.

Uyarıların program oluşmasını engellememesi onları önemsiz yapmaz. Çoğu uyarı, çalışma anında ortaya çıkacak gerçek bir hatanın erken işaretidir. Aşağıdaki program geçerlidir, ama içinde hiç kullanılmayan bir değişken vardır:

▸ kullanilmayan.c

```c
#include <stdio.h>

int main(void) {
    int kullanilmayan = 5;
    printf("Merhaba\n");
    return 0;
}
```

Bu programı `-Wall -Wextra` ile derlediğimizde gcc şunu bildirir:

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra kullanilmayan.c -o kullanilmayan
kullanilmayan.c: In function 'main':
kullanilmayan.c:4:9: warning: unused variable 'kullanilmayan' [-Wunused-variable]
    4 |     int kullanilmayan = 5;
      |         ^~~~~~~~~~~~~
```

ÇIKTI

```
Merhaba
```

Uyarıya rağmen program oluştu ve çalıştı. Mesajın sonundaki köşeli parantezli `[-Wunused-variable]` etiketi, uyarıyı hangi seçeneğin ürettiğini gösterir. Seçenekler olmadan derleseydik gcc bu programı sessizce derlerdi ve değişkeni fark etmezdik. Böyle bir değişken, kodun bir kısmının unutulduğuna ya da yanlış değişken adının yazıldığına işaret edebilir.

Bu nedenle kitap boyunca iki kural uygulayacağız:

1. Derlemeyi her zaman `gcc -std=c17 -Wall -Wextra` ile yapacağız.
2. Uyarı veren kodu çalışıyor diye bırakmayacağız; **uyarısız derlenene kadar** düzelteceğiz.

> **İpucu · Uyarıları hata yapmak**
>
> `-Werror` seçeneği her uyarıyı hataya çevirir. Uyarı üreten kodun derlenmesini engellemek istediğimiz ekip projelerinde sık kullanılır. Öğrenirken zorunlu değildir, ama "uyarısız derle" ilkesini zorlamak için denemeye değer.

> *Tarihçe · Neden C17?*
>
> C standardı zamanla değişti (Bölüm 2). Yeni gcc sürümleri varsayılan olarak daha yeni bir standardı seçebilir; örneğin gcc 15 ile varsayılan C23 oldu. Kitaptaki programlar ve hata mesajları tek bir standarda göre yazıldığı için

`-std=c17` seçeneğini açıkça veriyoruz. Böylece hangi gcc sürümünü kurmuş olursak olalım aynı davranışı görürüz.

4.6 Yorum Satırları

Yorumlar (comments), derleyicinin tümüyle yok saydığı, kodu okuyan insanlara yönelik notlardır. C'de iki tür yorum vardır:

Tür	Yazım	Kullanım
Tek satırlık	`// ...`	Satırın sonuna kadar geçerlidir. C99 ile gelmiştir
Çok satırlık	`/* ... */`	Arada kalan her şey, satır sınırı gözetmeden yok sayılır

▸ yorumlar.c

```c
/* Bu program yorum türlerini gösterir.
   Çok satırlı yorum birkaç satıra yayılabilir. */
#include <stdio.h>

int main(void) {
    // Tek satırlık yorum satır sonuna kadar sürer
    printf("Merhaba\n"); // kodun yanına da yazılabilir

    /* Aşağıdaki satır yorum içinde olduğu için çalışmaz
    printf("Bu satır çalışmaz\n");
    */
    printf("Bitti\n");
    return 0;
}
```

ÇIKTI

```
Merhaba
Bitti
```

Yorum işaretleri metin sabitinin içinde yorum başlatmaz. `printf("a // b\n");` ekrana `a // b` yazar; `//` ancak tırnakların dışında yorum işaretidir.

> *Tarihçe · // yorumu C99 ile geldi*
>
> C'nin ilk standardında (C89) yalnızca `/* */` yorumu vardı. `//` yorumu, C++'ta ve C'nin atalarından BCPL'de bulunuyordu; C99 standardıyla C'ye de eklendi. Bu yüzden çok eski derleyiciler `//` yorumunu tanımaz. Kullandığımız C17'de ikisi de geçerlidir.

İç içe `/* */` tuzağı

`/* */` yorumları **iç içe yazılamaz**. Yorum, ilk gördüğü `*/` işaretinde kapanır:

▸ icIceYorum.c

```c
#include <stdio.h>

int main(void) {
    /* Başlangıç yorumu
    printf("Bir\n"); /* iç yorum */
    printf("İki\n");
```

```
    */
    printf("Üç\n");
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

icIceYorum.c: In function 'main':
icIceYorum.c:5:22: warning: "/*" within comment [-Wcomment]
    5 |     printf("Bir\n"); /* iç yorum */
      |
icIceYorum.c:7:6: error: expected expression before '/' token
    7 |     */
      |      ^
```

Dıştaki yorum, içteki `*/` işaretinde kapandı. Bundan sonra `printf("İki\n");` normal kod gibi okundu; ardından gelen yalnız `*/` ise derleyiciye anlamsız geldi. gcc önce `"/*" within comment` uyarısıyla "yorumun içinde yorum mu açtın?" diye sordu. Bir kod bloğunu geçici olarak devre dışı bırakmak istiyorsak ve içinde `/* */` varsa satırları tek tek `//` ile yorumlamalıyız. Çoğu kod düzenleyicide satırları seçip `Ctrl+/` (macOS'ta `Cmd+/`) tuşlarına basmak bunu otomatik yapar.

İyi yorum nasıl olur?

Yorum, kodun **ne yaptığını** tekrar etmemeli; **neden** öyle yapıldığını açıklamalıdır:

```
x = x + 1;           // x'e 1 ekle                  (gereksiz)
indeks = indeks + 1; // başlık satırını atla        (anlamlı)
```

Birinci yorum kodun zaten söylediğini yineler. İkincisi kodun okuyucuya vermediği bilgiyi, yani amacı sunar. Fazla yorum da az yorum kadar zararlıdır; kod değiştiğinde yorum güncellenmezse okuyucuyu yanıltır. Önce anlaşılır adlar ve düzenli bir kod yazmaya çalışmalı, yorumu geriye kalan boşluklar için kullanmalıyız.

4.7 Kod Düzeni ve Okunabilirlik

C'de boşluk, girinti ve satır sonu derleyici için çoğunlukla anlam taşımaz. Aşağıdaki program, ilk programımızla aynı işi yapan ama okunması zor bir hâlidir:

▸ daginik.c

```
#include <stdio.h>
int main(void){printf("Bir ");
    printf("iki ")
;printf("Üç\n"
);return
0;}
```

```
ÇIKTI
Bir iki Üç
```

Derleyici bu düzensizliği sorunsuz kabul eder. Ancak insan okuyucu için aynı programı şöyle yazmak çok daha iyidir:

```c
#include <stdio.h>

int main(void) {
    printf("Bir ");
    printf("iki ");
    printf("üç\n");
    return 0;
}
```

Kitap boyunca şu düzen kurallarını izleyeceğiz:

- Her deyim ayrı bir satırda yazılır.
- Süslü parantezin içindeki satırlar **4 boşluk** içeriden başlar. Açan `{` karakteri, bloğu başlatan satırın sonundadır; kapatan `}` ise bloğu başlatan satırla aynı hizada tek başına durur.
- Mantıksal bölümler arasında boş satır bırakılır; `#include` satırları ile fonksiyon arasında da boş satır olur.
- Bir satırı yaklaşık 80 karakterde tutarız; daha uzun ifadeleri uygun yerde bölebiliriz.
- Değişken ve fonksiyon adları `camelCase` yazılır (Bölüm 5). Türkçe karakterler kod adlarında kullanılmaz; yalnızca ekrana yazılan metinlerde bulunur.

> **Not · Türkçe karakter nerede olur?**
>
> `printf("Merhaba Dünya!\n")` içindeki `ü` bir metin sabitinde olduğu için sorun yaratmaz; çoğu sistemde doğru görünür. Windows konsolunda bozuk görünürse nedeni karakter kodlamasıdır ve Bölüm 3'te anlatılan `chcp 65001` komutuyla çözülür. Ancak `int yaş = 5;` gibi bir **ad** taşınabilir değildir; adlarda yalnızca İngilizce harfler, rakam ve alt çizgi kullanırız.

4.8 Kaçış Dizileri

`printf` ekrana yalnızca yazdığımız metni basar; kendiliğinden alt satıra geçmez. Bunu üç ardışık çağrıyla görelim:

▸ satirSonu.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba");
    printf("Dünya");
    printf("!\n");
    return 0;
}
```

```
ÇIKTI
MerhabaDünya!
```

Üç çağrı, aralarında hiçbir boşluk olmadan arka arkaya yazdı. İmleci alt satıra taşımak için metne `\n` eklemek gerekir. `\n` kaynak kodda iki karakter gibi görünse de C için **tek bir karakterdir**. Ters bölü ile başlayan bu özel yazımlara **kaçış dizisi** (escape sequence) denir.

> **Tanım: Kaçış dizisi**
>
> Metin sabiti ya da karakter sabiti içinde, ters bölü (`\`) ile başlayan ve yazılması zor ya da özel anlamı olan bir karakteri temsil eden iki karakterlik yazımdır. Örneğin `\n` yeni satır, `\t` sekme karakteridir.

Yazım	Anlamı
`\n`	Yeni satır; imleci alt satırın başına geçirir
`\t`	Sekme (tab); imleci bir sonraki sekme durağına atlatır
`\\`	Ters bölü karakterinin kendisi
`\"`	Çift tırnak karakteri
`\'`	Tek tırnak karakteri
`\r`	Satır başı; imleci aynı satırın başına götürür
`\0`	Boş karakter (null); metinlerin sonunu işaretler (Bölüm 11)
`%%`	Yüzde işaretinin kendisi (`printf` 'e özgü bir kuraldır)

Bu yazımların neden gerektiğini düşünelim. Çift tırnak, metin sabitinin sınırını belirler; metnin içinde çift tırnak yazmak için `"` yerine `\"` kullanırız, yoksa derleyici metnin orada bittiğini sanır. Ters bölü kaçış dizilerini başlattığı için, ters bölünün kendisini yazdırmak üzere `\\` yazarız. Hepsini tek bir programda görelim:

▸ **kacis.c**

```c
#include <stdio.h>

int main(void) {
    printf("Ad     Not\n");
    printf("Ayşe   85\n");
    printf("Mehmet 90\n");
    printf("Elif dedi ki: \"Hazırım.\"\n");
    printf("Dosya yolu: C:\\kitap\\merhaba.c\n");
    printf("Başarı oranı: %%95\n");
    printf("Birinci satır\nİkinci satır\n");
    return 0;
}
```

```
ÇIKTI
Ad      Not
Ayşe    85
Mehmet  90
Elif dedi ki: "Hazırım."
Dosya yolu: C:\kitap\merhaba.c
Başarı oranı: %95
Birinci satır
İkinci satır
```

Sekme karakteri sütunları hizalamak için kullanışlıdır. Sekme durakları belirli aralıklarla (çoğu terminalde 8 karakterde bir) bulunduğundan, `Ad` ile `Ayşe` farklı uzunlukta olsa da ardından gelen sütun aynı yerden başlar. Düzgün sütunlar için `printf` 'in genişlik ayarını Bölüm 6'da kullanacağız.

`%%` bir kaçış dizisi değil, `printf` fonksiyonunun kuralıdır. `printf` içinde `%` işareti özeldir: `%d` gibi **biçim belirteçlerini** (format specifier) başlatır. Bunlar, değişkenlerin değerini metnin içine yerleştirmek için kullanılır

ve Bölüm 5 ile 6'da ayrıntılı işlenir. Şimdilik bilmemiz gereken tek şey, ekrana gerçekten bir `%` yazmak istediğimizde `%%` yazmamız gerektiğidir.

> **▲ Sık Yapılan Hata · Ters bölü, eğik çizgi ve yüzde**
>
> Kaçış dizileri **ters bölü** `\` ile yazılır. `printf("Merhaba/n");` ekrana olduğu gibi `Merhaba/n` basar ve alt satıra geçmez. Yüzde işareti için `\%` biçiminde bir kaçış dizisi de yoktur: `%%` yazılır. Tek başına bırakılan bir `%` işareti `printf` tarafından bir biçim belirtecinin başlangıcı sanılır; geçersiz biçim belirteci ise tanımsız davranıştır (Bölüm 5).

> **◆ Kritik Nokta · İmleç yalnızca \n ile alt satıra iner**
>
> Çıktıyı tahmin ederken imlecin hangi satırda olduğunu adım adım izlemek gerekir. İlk programdaki `printf("Merhaba\n")` ile `printf("Merhaba")` arasındaki tek fark alt satıra geçip geçmemektir. Programın son `printf`'inde `\n` yoksa imleç o satırda kalır ve birçok terminalde bir sonraki komut istemi çıktının hemen yanına yapışır.

4.9 Hata Mesajlarını Okumak

Her programcı derleyiciden hata alır; fark, mesajı okuyabilip okuyamamaktadır. gcc'nin hata mesajları ilk bakışta karmaşık görünür, ama hepsi aynı düzene sahiptir. Unutulmuş bir noktalı virgülle başlayalım:

▸ noktaliVirgul.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba\n")
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
noktaliVirgul.c: In function 'main':
noktaliVirgul.c:4:24: error: expected ';' before 'return'
    4 |     printf("Merhaba\n")
      |                        ^
      |                        ;
    5 |     return 0;
      |     ~~~~~~
```

Mesajın ikinci satırı hatanın kendisidir ve her zaman `dosya:satır:sütun: tür: açıklama` düzenindedir:

Parça	Örnekteki değeri	Anlamı
Dosya	`noktaliVirgul.c`	Hata hangi dosyada
Satır	`4`	Kaçıncı satırda
Sütun	`24`	Satırın kaçıncı karakterinde
Tür	`error`	`error` derlemeyi durdurur; `warning` durdurmaz
Açıklama	`expected ';' before 'return'`	`return`'den önce `;` bekleniyordu

Altındaki satırlarda gcc sorunlu satırı gösterir, `^` ile noktayı işaretler ve çoğu kez çözümü de önerir (burada `;` eklemeyi). İlk satırdaki `In function 'main':` ifadesi hatanın hangi fonksiyonun içinde olduğunu bildirir.

Mesajın bir inceliğine dikkat edelim: gcc, noktalı virgülün eksik olduğunu `return` satırına geldiğinde fark etti. `printf("Merhaba\n")` satırı kendi başına eksiksiz görünür; derleyici ancak sonraki sözcüğü görünce deyimin bitmediğini anlar. Bu yüzden gcc yerini satırın sonu olarak işaretlese de, hatayı bildirdiği satırda bir şey göremezsek **bir önceki satıra** da bakmalıyız.

> **İpucu · İlk hatadan başlayın**
>
> Tek bir hata, ardından başka hata mesajlarını da getirebilir. Örneğin kapanmayan bir `{` yüzünden derleyici dosyanın sonuna kadar şikâyet eder. Mesaj listesi uzunsa yalnızca **en üstteki** hatayı düzeltip yeniden derlemek en iyisidir; çoğu zaman diğerleri kendiliğinden kaybolur.

Sık görülen derleme hataları

Başka sık hatalara bakalım. Kapanmayan bir süslü parantez:

▸ kapanmayan.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba\n");
    return 0;
```

DERLEYİCİ ÇIKTISI

```
kapanmayan.c: In function 'main':
kapanmayan.c:5:5: error: expected declaration or statement at end of input
    5 |     return 0;
      |     ^~~~~~
```

"Girdinin sonunda bir deyim ya da bildirim bekleniyordu": derleyici dosyanın sonuna ulaştı, ama `main`'in bloğu hâlâ açıktı. Her `{` için bir `}` bulunmalıdır. Düzenli girinti, eksik parantezi bulmayı çok kolaylaştırır.

Kapanmayan bir çift tırnak daha çok mesaj üretir:

▸ tirnak.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba\n);
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

tirnak.c: In function 'main':
tirnak.c:4:12: warning: missing terminating " character
    4 |     printf("Merhaba\n);
      |            ^
tirnak.c:4:12: error: missing terminating " character
    4 |     printf("Merhaba\n);
      |            ^~~~~~~~~~
tirnak.c:5:5: error: expected expression before 'return'
    5 |     return 0;
      |     ^~~~~~
tirnak.c:5:14: error: expected ';' before '}' token
    5 |     return 0;
      |              ^
      |              ;
    6 | }
      | ~
```

Bu, "ilk hatadan başla" ilkesinin tipik bir örneğidir. Gerçek sorun tek bir satırdadır: metin sabiti kapanmamıştır. Derleyici metnin satırın sonuna kadar sürdüğünü sanınca sonraki mesajlar bunun yan etkisi olarak çıkar. Yalnızca ilk hatayı düzeltip yeniden derlemek yeterli olacaktır.

Üçüncü klasik hata, başlık dosyasını unutmaktır. Aşağıdaki kodda `#include <stdio.h>` satırı yoktur:

```c
int main(void) {
    printf("Merhaba\n");
    return 0;
}
```

gcc 13 şu iletiyi verir (gcc 14 ve sonrasında `warning` yerine `error` yazar):

```
DERLEYİCİ ÇIKTISI

includeYok.c: In function 'main':
includeYok.c:2:5: warning: implicit declaration of function 'printf'
    2 |     printf("Merhaba\n");
      |     ^~~~~~
includeYok.c:1:1: note: include '<stdio.h>' or provide a declaration of 'printf'
  +++ |+#include <stdio.h>
    1 | int main(void) {
```

"Dolaylı bildirim" (implicit declaration), derleyicinin `printf`'i daha önce hiç tanıtılmamış bulduğu anlamına gelir. Mesajın devamı çözümü kendisi söylüyor: `include '<stdio.h>' or provide a declaration`. gcc eklenecek satırı bile `+++ |+#include <stdio.h>` ile gösteriyor. Hata mesajlarının çoğu çözümü taşır; yeter ki sonuna kadar okuyalım.

Dördüncüsü, bir fonksiyon adının yanlış yazılması:

```c
#include <stdio.h>

int main(void) {
    prinft("Merhaba\n");
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

yazimHata.c: In function 'main':
yazimHata.c:4:5: warning: implicit declaration of function 'prinft'; did you mean 'printf'?
    4 |     prinft("Merhaba\n");
      |     ^~~~~~
      |     printf
/usr/bin/ld: .../ccXXXX.o: in function `main':
yazimHata.c:(.text+0x18): undefined reference to `prinft'
collect2: error: ld returned 1 exit status
```

gcc `did you mean 'printf'?` diye tahmin yürütür ve haklıdır. Bu örnekte iki farklı aşamanın mesajı birlikte görünüyor: derleyici uyarı verdi ve çalışmaya devam etti, ama bağlayıcı `prinft` adında bir fonksiyon bulamadığı için hata verdi. gcc 14 ve sonrasında ilk mesaj doğrudan `error` olur ve bağlama aşamasına hiç gelinmez. Aynı hatanın sürüme göre farklı mesajlarla görünebileceğini bilmek, yabancı bir bilgisayarda şaşırmamamızı sağlar.

> **▲ Sık Yapılan Hata · "Sadece uyarı" diye geçmeyin**
>
> Başlık dosyasını unutan program gcc 13'te bir uyarıyla yine de derlenir ve çalışır; çünkü gcc `printf`'i tanıdığı için tahmin yürütür. Başka bir fonksiyonda aynı tahmin yanlış olabilir ve program çalışma anında rastgele davranır. Yeni gcc sürümlerinin bu durumu doğrudan hata saymasının nedeni budur. Her uyarıyı okumak ve gidermek bu yüzden önemlidir.

Bağlama hatası

Bir de kodun yazımı doğru olduğu hâlde bağlama aşamasında çıkan hata vardır. Tipik örnek, `main` yerine `Main` yazmaktır:

▸ mainYanlis.c

```c
#include <stdio.h>

int Main(void) {
    printf("Merhaba\n");
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

/usr/bin/ld: .../Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
```

C büyük ve küçük harfi ayırt eder; `Main` sıradan bir fonksiyon adıdır ve derleme aşaması sorunsuz geçer. Ancak bağlayıcı, programın başlayacağı `main` fonksiyonunu bulamaz. Windows'ta (MinGW) mesaj benzerdir, ama `undefined reference to 'WinMain'` ifadesi geçer; macOS'ta ise `Undefined symbols for architecture ... "_main"` ile başlar. Hepsinin ortak noktası `undefined reference` ya da `undefined symbols` ifadesidir.

Bağlama hatalarının iki ayırt edici özelliği vardır: - Mesaj `ld returned 1 exit status` ile biter (`ld`, bağlayıcının adıdır). - Genellikle kaynak dosyanızdaki bir satırı gösteren `dosya.c:satır:sütun:` konumu yoktur, çünkü bağlayıcı kaynak satırlarına değil nesne dosyalarına bakar.

Bağlama hatalarının başka bir sık nedeni, çağrılan fonksiyonun kodunun kütüphanede bulunamamasıdır. Matematik fonksiyonlarını Linux'ta kullanırken `-lm` seçeneğinin gerekmesini Bölüm 9'da göreceğiz.

Dört tür hata

Programlama hataları ortaya çıktıkları anlara göre dört gruba ayrılır (Bölüm 1'deki sınıflandırmanın gcc ile somutlaşmış hâli):

Hata türü	Ne zaman ortaya çıkar?	Program çalışır mı?	Örnek
Derleme hatası	Derleme aşamasında	Hayır, çalıştırılabilir dosya oluşmaz	Eksik `;`, kapanmayan `{`, yanlış yazım
Bağlama hatası	Bağlama aşamasında	Hayır, çalıştırılabilir dosya oluşmaz	`main` yerine `Main`, bulunamayan fonksiyon
Çalışma zamanı hatası	Program çalışırken	Başlar; çökebilir ya da yanlış davranabilir	Sıfıra bölme, geçersiz bellek erişimi
Mantık hatası	Sonuca bakınca	Evet, ama yanlış sonuç verir	Yanlış formül, yanlış sıra

Derleme ve bağlama hatalarını ayırt etmenin yolu mesajdır: `undefined reference` geçiyorsa bağlama hatasıdır, `dosya.c:satır:sütun: error` biçimindekiler derleme hatasıdır.

Çalışma zamanı hatasına örnek olarak sıfıra bölmeyi düşünelim:

```
int bolen = 0;
int sonuc = 10 / bolen;     // derlenir, ama çalışırken sorun çıkar
```

Bu kod derlenir. Çalıştırıldığında C bu durumu **tanımsız davranış** (undefined behavior) sayar, yani standart sonucu vaat etmez. Pek çok Linux sisteminde program `Floating point exception` iletisiyle çöker; başka sistemlerde sessizce kapanabilir. Kullanıcıya hata mesajı vermeyen, yalnızca beklenmedik biçimde sonlanan programlar bu türdendir.

Mantık hatası en sinsisidir, çünkü derleyici de çalışma ortamı da şikâyet etmez:

▸ geriSayim.c

```
#include <stdio.h>

int main(void) {
    // Amaç: 3, 2, 1, Başla! şeklinde geri sayım yapmak
    printf("1\n");
    printf("2\n");
    printf("3\n");
    printf("Başla!\n");
    return 0;
}
```

ÇIKTI

```
1
2
3
Başla!
```

Program kusursuz çalıştı, ama geri sayım ters çıktı. Bu tür hataları ancak çıktıyı beklediğimiz sonuçla karşılaştırarak yakalarız. İlerleyen bölümlerde değişkenlerin ara değerlerini yazdırarak ya da hata ayıklayıcıyla (Bölüm 3) adım adım izleyerek mantık hatalarını arayacağız.

4.10 main Fonksiyonu ve Çıkış Kodu

Her C programı `main` fonksiyonundan başlar. İlk satırı yeniden parçalayalım:

```
int main(void)
```

Parça	Anlamı
`int`	`main` işini bitirince işletim sistemine bir tamsayı döndürür
`main`	Fonksiyonun adı. Bağlayıcı tam olarak bu adı arar
`(void)`	`main` dışarıdan hiçbir değer almaz

`return 0;` ifadesi bu tamsayıyı döndürür. Bu değere programın **çıkış kodu** (exit status) denir.

> **Tanım: Çıkış kodu**
>
> Bir program bittiğinde işletim sistemine ilettiği tamsayıdır. Gelenek gereği `0` "program başarıyla tamamlandı", sıfırdan farklı bir değer "bir sorun oluştu" anlamına gelir. Sıfırdan farklı değerlerin neyi anlattığına programı yazan karar verir.

Ekranda bir fark görmeyiz, ama bu değer başka programlar için çok değerlidir. Derleme sistemleri ve betikler bir komutun çıkış koduna bakarak devam edip etmeyeceklerine karar verir. Aşağıdaki program bilerek `3` döndürüyor:

▸ donus3.c

```c
#include <stdio.h>

int main(void) {
    printf("Bir sorun oluştu!\n");
    return 3;
}
```

ÇIKTI

```
Bir sorun oluştu!
```

Çıkış kodunu son çalıştırılan komuttan sonra şu komutlarla öğrenebiliriz:

Ortam	Komut
Linux, macOS (bash, zsh)	`echo $?`
Windows PowerShell	`$LASTEXITCODE`
Windows Komut İstemi (cmd)	`echo %ERRORLEVEL%`

TERMİNAL

```
```

```
$ gcc -std=c17 -Wall -Wextra donus3.c -o donus3
$ ./donus3
Bir sorun oluştu!
$ echo $?
3
```

Kabuklarda `&&` işleci, soldaki komut başarılı (`0`) bittiyse sağdakini çalıştırır. Çıkış kodunun pratik kullanımını bu iki program gösteriyor:

```
TERMİNAL
$ ./merhaba && echo "Program başarıyla bitti"
Merhaba Dünya!
Program başarıyla bitti
$ ./donus3 && echo "Program başarıyla bitti"
Bir sorun oluştu!
```

İkinci komutta `echo` hiç çalışmadı, çünkü `donus3` sıfırdan farklı bir kodla bitti.

main'in yazılışına ilişkin ayrıntılar

- **`return 0;` yazmasak da olur.** C99'dan beri `main` fonksiyonunun sonuna gelindiğinde derleyici kendiliğinden `return 0;` ekler. Bu kolaylık yalnızca `main` için geçerlidir. Biz programın nasıl bittiğini açıkça görmek için yine de `return 0;` yazacağız.
- **`int main(void)` ile `int main()`.** İkisi de derlenir. İşlev tanımında boş parantez de "parametre yok" demektir. Fark, gövdesiz bildirimlerde (Bölüm 9'daki prototipler) ortaya çıkar: orada `(void)` "hiç parametre yok" derken boş `()` "parametreler belirtilmedi" anlamına gelir. Bu belirsizlikten kaçınmak için `int main(void)` yazacağız.
- **Komut satırı argümanları.** `main` bir de `int main(int argc, char *argv[])` biçiminde yazılabilir ve programı çalıştırırken verilen değerleri alır. Bunu işaretçileri öğrendikten sonra, Bölüm 12'de göreceğiz.

> *Tarihçe · C23'te main()*
>
> C23 standardında boş parantez `()` her bağlamda "parametre yok" anlamına gelir; yani `int main()` ile `int main(void)` eşdeğer olur. Bu kitap C17'yi temel aldığı ve `(void)` biçimi her standartta kesin olduğu için `int main(void)` kullanır.

> **◆ Kritik Nokta · Derle, uyarılara bak, çalıştır**
>
> Bu bölümün pratik özeti şudur: kodu yazarken `.c` uzantısını ve düzenli girintiyi kullanırız; `gcc -std=c17 -Wall -Wextra` ile derleriz; hata ve uyarıları **ilk mesajdan başlayarak** okuruz; uyarı kalmayınca programı çalıştırırız. Bu döngü kitabın geri kalanında hiç değişmeyecek.

4.11 Aynı Program Üç Dilde

Merhaba Dünya programını C++ ve C#'ta da görelim. Henüz bu dillerin ayrıntısını bilmiyoruz; amacımız yalnızca aynı programın nasıl göründüğünü karşılaştırmak.

Üç Dilde · Merhaba Dünya *C · C++ · C#*

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba Dünya!\n");
    return 0;
}
```

```cpp
#include <iostream>

int main() {
    std::cout << "Merhaba Dünya!" << std::endl;
    return 0;
}
```

```csharp
Console.WriteLine("Merhaba Dünya!");
```

Üç programın yaptığı iş aynı, ama yazılışları farklıdır. C'de `printf` bir kütüphane fonksiyonudur ve `\n` ile satır sonu yazarız. C++'ta (Bölüm 21 ve 23) `<iostream>` başlığı gelir; çıktı `std::cout` akışına `<<` işleciyle gönderilir. C#'ta (Bölüm 41) ise `Main` fonksiyonu ve `#include` satırı yazmaya gerek yoktur; yukarıdaki tek satır, C# 9'dan beri "üst düzey deyimler" (top-level statements) sayesinde tam bir programdır.

Derleme ve çalıştırma komutları da dillere göre değişir. Bu yapının nedeni, derleme modelleridir:

Üç Dilde · Derleme modeli *C · C++ · C#*

Dil	Derleme komutu	Üretilen	Nasıl çalışır?
C	`gcc merhaba.c -o merhaba`	Makine kodu (yerel çalıştırılabilir dosya)	Doğrudan işlemcide
C++	`g++ merhaba.cpp -o merhaba`	Makine kodu (yerel çalıştırılabilir dosya)	Doğrudan işlemcide
C#	`dotnet run` (veya `dotnet build`)	Ara dil (IL) içeren derleme (assembly)	.NET çalışma ortamı ara dili makine koduna çevirir

C ve C++ doğrudan işlemcinin dilini üretir; bu dört aşamalı zincir (önişleme, derleme, çevirme, bağlama) ikisi için de geçerlidir. C# derleyicisi ise önce makineden bağımsız bir **ara dil** üretir; bu ara dil, program çalışırken .NET'in JIT derleyicisi tarafından makine koduna çevrilir. C#'ta `#include` ve başlık dosyaları yoktur; yerlerini ad alanları (namespace) ve derleme referansları alır.

Bölüm Özeti

- C küçük, hızlı ve donanıma yakın bir dildir; programcıya güvenir ve denetimi ona bırakır.
- Bir C programı `main` fonksiyonundan başlar. İlk programımız `#include <stdio.h>`, `int main(void)`, `printf` ve `return 0;` öğelerinden oluşur.
- C'de deyimler noktalı virgülle biter; `#` ile başlayan önişlemci satırları bunun dışındadır.
- `gcc -std=c17 -Wall -Wextra dosya.c -o program` komutu kitabın standart derleme komutudur. Çalıştırma komutu Linux/macOS'ta `./program`, PowerShell'de `.\program`, cmd'de `program` biçimindedir.

- Derleme süreci dört aşamadır: önişleme (`-E`), derleme (`-S`), çevirme (`-c`) ve bağlama. Her aşamanın çıktısı ayrı olarak incelenebilir.
- `#include` bir fonksiyonun yalnızca bildirimini getirir; kodu standart kütüphaneden bağlayıcı ekler.
- Hata, derlemeyi durdurur; uyarı durdurmaz. Uyarısız derlemek temel alışkanlıktır.
- Yorumlar `//` (C99'dan beri) ve `/* */` ile yazılır. `/* */` yorumları iç içe geçemez.
- Boşluk ve girinti derleyiciyi ilgilendirmez, ama okunabilirlik için 4 boşluklu girinti, her satırda bir deyim ve anlamlı adlar kullanırız.
- Kaçış dizileri: `\n` yeni satır, `\t` sekme, `\\` ters bölü, `\"` çift tırnak; yüzde işareti için `%%` yazılır.
- gcc hata mesajı `dosya:satır:sütun: tür: açıklama` düzenindedir. Her zaman ilk hatadan başlanır; hata bildirilen satırda sorun görünmüyorsa bir önceki satıra bakılır.
- Bağlama hataları (`undefined reference`) kaynak satırı göstermez ve `ld returned 1 exit status` ile biter.
- `main`'in döndürdüğü değer çıkış kodudur: `0` başarı, sıfırdan farklı değer sorundur. Çıkış kodu `echo $?` (Linux/macOS), `$LASTEXITCODE` (PowerShell) ve `echo %ERRORLEVEL%` (cmd) ile görülür.
- C ve C++ doğrudan makine kodu üretir; C# önce ara dil üretir ve çalışma ortamı onu makine koduna çevirir.

Alıştırmalar

1. Aşağıdakilerden hangisi `#include <stdio.h>` satırını işleyen derleme aşamasıdır?

   A) Bağlama
   B) Önişleme
   C) Çevirme (assembler)
   D) Programın çalıştırılması
   E) Optimizasyon

2. gcc'nin, programı yalnızca assembly kodu üretene kadar işleyip durmasını sağlayan seçenek hangisidir?

   A) `-E`
   B) `-c`
   C) `-o`
   D) `-S`
   E) `-Wall`

3. Aşağıdaki ifadelerden hangisi doğrudur?

   A) Uyarı veren bir kod derlenmez.
   B) Hata veren bir kod derlenir, ama çalıştırılamaz.
   C) Uyarı veren kod derlenir; ama uyarı çoğu zaman gerçek bir sorunun işaretidir.
   D) `-Wall` seçeneği kodu hızlandırır.
   E) Uyarılar ve hatalar aynı şeydir.

4. Ekrana `C:\temp\not.txt` yazdırmak için hangi satır doğrudur?

   A) `printf("C:\temp\not.txt\n");`
   B) `printf("C:/temp/not.txt/n");`
   C) `printf("C:\\temp\\not.txt\n");`

D) `printf('C:\temp\not.txt\n');`
E) `printf("C:\"temp\"not.txt\n");`

5. Programın çıktısı nedir?

```
printf("Bir\n");
printf("İki\n\n");
printf("Üç");
printf("Dört\n");
```

6. Programın çıktısı nedir?

```
printf("a"); // printf("b");
/* printf("c"); */ printf("d\n");
printf("%%e\n");
```

7. Aşağıdaki program Linux'ta derlenip çalıştırıldıktan hemen sonra `echo $?` yazılırsa ekrana ne gelir?

```
#include <stdio.h>

int main(void) {
    printf("Tamam\n");
    return 7;
}
```

A) `Tamam`
B) `0`
C) `7`
D) `1`
E) Hiçbir şey

8. Aşağıdaki durumlardan hangisi bir **bağlama hatasıdır**?

A) Bir deyimin sonundaki noktalı virgülün unutulması
B) Hiç kullanılmayan bir değişken
C) Tanımlı olmayan bir fonksiyonun çağrılması ve mesajda `undefined reference` görünmesi
D) Programın ekrana yanlış sırayla yazı yazması
E) Bir tamsayının sıfıra bölünmesi

9. Aşağıdaki durumları derleme hatası, bağlama hatası, çalışma zamanı hatası ya da mantık hatası olarak sınıflandırın.

a) `printf("Merhaba")` satırından sonra `;` konmamış. b) `main` fonksiyonunun adı `Main` olarak yazılmış. c) Program, 3 ile 5'in toplamını yazdırması gerekirken ekrana yanlışlıkla 35 yazıyor. d) Program çalışırken bir sayıyı sıfıra bölmeye çalışıp çöküyor. e) `#include` satırı unutulduğu için gcc `printf`'i tanıyamıyor.

10. "Programcıya güven" ilkesi C için ne anlama gelir?

A) Derleyici hiçbir hatayı yakalamaz.
B) Dil, programcının ne yaptığını bildiğini varsayar ve çoğu denetimi (örneğin dizi sınırı kontrolü) ona bırakır.
C) C programları her zaman güvenli çalışır.
D) Programcı yalnızca standart kütüphaneyi kullanabilir.
E) C'de değişkenlerin tipi çalışma anında belirlenir.

11. `gcc -c merhaba.c` komutu sonunda klasörde hangi dosya oluşur?

    A) `merhaba` (çalıştırılabilir)
    B) `merhaba.s`
    C) `merhaba.i`
    D) `merhaba.o`
    E) `a.out`

12. Aşağıdaki program derlenirken birkaç hata ve bir bağlama sorunu çıkarmaktadır. Hataları bulun, her birinin türünü söyleyin ve düzeltin.

```c
#include <stdio.h>

int Main(void) {
    printf("Birinci\n")
    prinft("İkinci\n");
    return 0
}
```

13. Programın çıktısı nedir?

```c
printf("A\\nB\n");
printf("%%d\n");
printf("\"Merhaba\"\n");
```

14. Ekrana aşağıdaki üç satırı yazdıran bir program yazın. İkinci satırdaki tırnaklar ve yüzde işareti çıktıda da görünmelidir.

```
Kitap: C Dillerinin Tarihi ve Öğrenimi
"Merhaba" dedi ve %100 emin oldu.
Dosya: C:\kitap\ornek.c
```

15. Yalnızca `printf` ve `\n` kullanarak, `*` karakterlerinden oluşan 4 satırlık bir üçgen çizen program yazın (birinci satırda 1, dördüncü satırda 4 yıldız).

16. Hiçbir şey yazdırmayan, ama çıkış kodu 42 olan bir program yazın. Derleyip çalıştırdıktan sonra çıkış kodunu Linux'ta ve Windows PowerShell'de görmek için gereken komutları da belirtin.

BÖLÜM 5

Değişkenler, Veri Tipleri ve Operatörler

BU BÖLÜMDE

- Değişkenin ne olduğunu; tanımlamayı, ilk değer vermeyi ve atamayı birbirinden ayırmayı
- Geçerli değişken adı kurallarını, anahtar sözcükleri ve adlandırma geleneklerini
- `char`, `short`, `int`, `long`, `long long`, `float`, `double` tiplerini; `sizeof`, `<limits.h>`, `<float.h>` ve `<stdint.h>` ile boyut ve sınırları bulmayı
- Sabitleri (literal, `const`, `#define`) ve `printf` biçim belirteçlerini
- Aritmetik, atama, artırma/azaltma, ilişkisel ve mantıksal operatörleri; tamsayı bölmesini ve `%` işlecini
- Örtük ve açık tip dönüşümünü, taşmayı, `unsigned` tuzaklarını ve kayan noktalı sayıların hassasiyetini
- Operatör önceliğini ve birleşme yönünü okuyarak bir ifadeyi adım adım çözmeyi

Bir program, hangi dilde yazılmış olursa olsun, sonunda veriyle çalışır: bir ürünün fiyatı, bir öğrencinin notu, bir sensörden gelen sıcaklık. Bu verilerin program çalıştığı sürece bir yerde tutulması, üzerlerinde işlem yapılması ve sonucun gösterilmesi gerekir. Bu bölüm, bunun üç temel aracını anlatır: verinin saklandığı yer olan **değişken**, saklanan verinin türünü belirleyen **veri tipi** ve veriler üzerinde işlem yapan **operatör**.

Bölümün sonunda küçük ama gerçek hesaplamalar yapan programlar yazabilecek, daha önemlisi bu hesapların bilgisayarda neden bazen beklenmedik sonuç verdiğini açıklayabileceksiniz. `7 / 2` ifadesinin neden 3 olduğunu, `0.1 + 0.2` toplamının neden tam 0,3 etmediğini ve `-1 < 1u` karşılaştırmasının neden yanlış çıktığını bu bölümde göreceğiz. Bunların hiçbiri hata değildir; her biri dilin ve donanımın tanımlı davranışıdır, yeter ki bilinsin.

5.1 Değişken Nedir?

Bilgisayarın belleği (RAM), her biri bir numaraya sahip çok sayıda küçük kutudan oluşur. Bu kutulara **bayt** (byte) denir ve her bayt 8 bit tutar (Bölüm 1). Bir program bir sayıyı saklamak istediğinde belleğin bir bölgesini ayırır ve sayıyı oraya yazar. Bu bölgeyi her seferinde sayısal adresiyle anmak çok güç ve hataya açıktır; bu nedenle programlama dilleri bölgeye bir ad verme olanağı sunar.

> **Tanım: Değişken**
>
> Bellekte belirli bir büyüklükte yer kaplayan, bir adı ve bir tipi olan, içindeki değer program çalışırken değiştirilebilen adlandırılmış bellek alanıdır.

Her değişkenin üç özelliği vardır:

- **Ad**: Değişkene koda yazarken başvurduğumuz etiket (`yas`, `toplam`).

- **Tip**: Alanın kaç bayt yer kaplayacağını ve içindeki bitlerin nasıl yorumlanacağını belirler (`int`, `double`, `char`).
- **Değer**: Alanın o andaki içeriği.

Bir değişkenin bir de **adresi** vardır: ayrılan alanın bellekteki konumu. Adresleri Bölüm 12'de işaretçileri (pointer) anlatırken ayrıntılı göreceğiz. Şimdilik değişkeni, üzerinde etiketi yazılı bir kutu olarak düşünmek yeterlidir:

```
  yas            boy               harf
+--------+   +------------+     +-------+
|   19   |   |    1.75    |     |  'A'  |
+--------+   +------------+     +-------+
  int           double             char
 4 bayt         8 bayt            1 bayt
```

"Değişken" adı, değerinin program boyunca değişebilmesinden gelir: kutudaki sayıyı çıkarıp yerine başkasını koyabiliriz. Değişmeyen tek şey tiptir; `int` olarak açılan kutu sonuna kadar `int` kalır.

Aynı kutuyu belleğin gerçek yüzüyle çizersek şöyle görünür. `yas` değişkeninin `int` olduğunu, `int`'in bu makinede 4 bayt yer kapladığını ve değerin 19 olduğunu varsayalım (adresler örnek olarak seçilmiştir):

```
Adres     Bayt içeriği (ikilik)
0x1000    00010011  \
0x1001    00000000   |  yas: 4 bayt, değeri 19
0x1002    00000000   |
0x1003    00000000  /
```

19 sayısı ikilikte `10011`'dir (16 + 2 + 1). Çoğu bilgisayar (x86 ve ARM işlemciler) çok baytlı sayıları "en düşük bayt önce" düzeniyle, yani küçük sonlu (little-endian) biçimde saklar; bu yüzden değerin düşük baytı ilk adreste durur. Bu ayrıntıyı bilmek, ileride belleği bayt bayt incelediğimizde işimize yarayacak; şimdilik 4 baytlık bir alanın tek bir sayı tuttuğunu bilmemiz yeter.

5.2 Tanımlama, İlk Değer Verme ve Atama

Bir değişkeni kullanabilmek için önce derleyiciye "şu tipte, şu adda bir alan ayır" demeliyiz. Buna **tanımlama** diyoruz. Teknik olarak bir fonksiyon içindeki `int sayi;` satırı hem bir bildirimdir (declaration; adı ve tipi duyurur) hem de bir tanımdır (definition; yer ayırır). Kitapta bu işlem için kısaca "tanımlama" diyeceğiz.

Değişkenle ilgili üç ayrı kalıp vardır:

```c
int sayi;          // tanımlama
int adet = 10;     // tanımlama + ilk değer verme (initialization)
adet = 25;         // atama (assignment)
```

> **Tanım: İlk değer verme ve atama**
>
> **İlk değer verme** (initialization), değişken yaratılırken içine ilk değerin konmasıdır. **Atama** (assignment), var olan bir değişkenin içeriğinin sonradan değiştirilmesidir. İkisi de `=` ile yazılır, ama ilki tanımlamanın parçası, ikincisi bağımsız bir ifadedir.

Hepsini bir programda görelim:

ilkDegisken.c

```c
#include <stdio.h>

int main(void) {
    int yas = 19;
    double boy = 1.75;
    char harf = 'A';

    printf("Yaş: %d\n", yas);
    printf("Boy: %f\n", boy);
    printf("Boy: %.2f\n", boy);
    printf("Harf: %c\n", harf);
    return 0;
}
```

ÇIKTI

```
Yaş: 19
Boy: 1.750000
Boy: 1.75
Harf: A
```

`printf` içindeki `%d`, `%f` ve `%c` birer **yer tutucudur**: virgülden sonra yazılan değişkenin değeri bu yerlere sırayla yerleşir. Bu bölümde sık kullanacağımız belirteçleri ayrı bir başlıkta toplayacağız; şimdilik şu kadarı yeter: `%d` tamsayı, `%f` ondalıklı sayı (virgülden sonra altı basamakla), `%.2f` virgülden sonra iki basamaklı ondalıklı sayı, `%c` tek karakterdir. Kodda ondalık ayırıcı virgül değil **noktadır** (`1.75`). Tek karakter tek tırnakla (`'A'`), metin çift tırnakla (`"Yaş"`) yazılır.

Atama operatörü matematikteki eşitlik değildir

C'de `=` işareti "eşittir" anlamına gelmez; **atama operatörüdür**. Önce sağ taraf hesaplanır, sonuç soldaki değişkene yazılır. Bu yüzden `sayi = sayi + 1;` ifadesi matematikte saçma olsa da C'de tamamen geçerlidir: "kutudaki değeri al, 1 ekle, sonucu aynı kutuya geri koy."

atamaOrnek.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 10;
    sayi = 25;              // eski değer silinir
    sayi = sayi + 1;        // önce sağ taraf: 25 + 1
    printf("sayi = %d\n", sayi);

    int a = 3, b = 8;       // aynı tipte birden çok tanım
    int gecici = a;         // takas: a'nın değerini sakla
    a = b;
    b = gecici;
    printf("a = %d, b = %d\n", a, b);

    int x, y, z;
    x = y = z = 0;          // sağdan sola: önce z, sonra y, sonra x
    printf("%d %d %d\n", x, y, z);
    return 0;
}
```

```
ÇIKTI
sayi = 26
a = 8, b = 3
0 0 0
```

Bu programdan üç şey öğreniyoruz:

1. Aynı tipte birden çok değişken virgülle ayrılarak tek satırda tanımlanabilir ve her birine ayrı ilk değer verilebilir.
2. İki değişkenin değerini değiştirmek (takas, swap) için üçüncü bir kutu gerekir. `a = b; b = a;` yazarsak `a`'nın eski değeri kaybolur ve iki değişkende de 8 kalır.
3. Atama bir ifadedir ve değer üretir: `z = 0` ifadesinin değeri 0'dır, bu değer `y`'ye atanır, onun değeri de `x`'e. Zincirleme atama bu yüzden sağdan sola çalışır.

> *Tarihçe · Bloğun ortasında tanımlama*
>
> C'nin ilk standardında (C89) tüm yerel değişkenlerin bloğun en başında tanımlanması gerekirdi. C99'dan itibaren bir değişken bloğun herhangi bir yerinde tanımlanabilir; tek koşul değişkeni kullanmadan önce tanımlamaktır. Bu kitaptaki örnekler değişkeni ihtiyaç duyulan yerde tanımlar.

İlk değeri olmayan değişken: çöp değer

> **Tanım: Tanımsız davranış**
>
> C standardının sonucu hakkında hiçbir söz vermediği kod davranışıdır (undefined behavior). Program çökebilir, yanlış sonuç verebilir ya da tesadüfen doğru çalışabilir; derleyici de bunu hata olarak bildirmek zorunda değildir. Bu kitapta çıktısı tanımsız davranışa bağlı programları göstermeyeceğiz; yalnızca nedenini açıklayacağız.

Bir değişkeni tanımlayıp içine hiçbir şey yazmazsak ne olur? Fonksiyon içindeki (yerel) değişkenler için C, kutunun içini temizlemez. Kutuda, o bellek bölgesinde daha önce çalışan kodlardan kalmış rastgele bir değer bulunur. Buna **çöp değer** (garbage value) denir.

```c
int sayi;                  // ilk değer verilmedi
printf("%d\n", sayi);      // çöp değeri okur
```

İlk değeri verilmemiş bir yerel değişkeni okumak **tanımsız davranıştır**. Ekrana 0 da yazılabilir, 21845 de, her çalıştırmada farklı bir sayı da. En kötü ihtimal, tesadüfen 0 yazıp programın doğru çalışıyormuş izlenimi vermesidir; hata başka bir bilgisayarda ya da başka bir derleyici ayarında ortaya çıkar. Bu yüzden böyle bir kodun çıktısını göstermiyoruz. İyi haber, `-Wall` seçeneğinin bu hatayı çoğu zaman yakalamasıdır:

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra cop.c -o cop
cop.c: In function 'main':
cop.c:5:5: warning: 'sayi' is used uninitialized [-Wuninitialized]
    5 |     printf("%d\n", sayi);
      |     ^~~~~~~~~~~~~~~~~~~~
cop.c:4:9: note: 'sayi' was declared here
    4 |     int sayi;
      |         ^~~~
```

◆ Kritik Nokta · Her değişkene ilk değerini tanımlarken verin

Değeri sonradan klavyeden okuyacak olsak bile `int sayi = 0;` yazmak iyi bir alışkanlıktır. İlk değeri belli olmayan değişken, hatası bulunması en zor programların kaynağıdır.

Bir istisna: **global** değişkenler (fonksiyonların dışında tanımlananlar) ve `static` değişkenler (Bölüm 9) program başlarken otomatik olarak sıfırlanır. Bu garanti yalnızca onlar içindir.

▲ Sık Yapılan Hata · int a, b, c = 0; kalıbı

`int a, b, c = 0;` satırı üç değişkeni de sıfırlamaz; yalnızca `c`'ye ilk değer verir. `a` ve `b` çöp değerle kalır. Her değişkenin ilk değeri ayrı ayrı yazılmalıdır: `int a = 0, b = 0, c = 0;`.

5.3 Değişken Adları ve Anahtar Sözcükler

Değişkene istediğimiz adı veremeyiz. C'de ad (identifier) şu kurallara uymalıdır:

1. Ad yalnızca harf (`a-z`, `A-Z`), rakam ve alt çizgiden (`_`) oluşur.
2. Rakamla başlayamaz.
3. Boşluk, `-`, `+`, `.` gibi özel karakterler içeremez.
4. **Anahtar sözcük** (keyword) olamaz.
5. Büyük ve küçük harf farklıdır: `sayi`, `Sayi` ve `SAYI` üç ayrı değişkendir.

Türkçe karakterler (`ç`, `ğ`, `ı`, `ö`, `ş`, `ü`) ad olarak kullanılmamalıdır. Bazı derleyiciler kabul etse de bu davranış taşınabilir değildir ve dosya kodlaması değiştiğinde program derlenmez hâle gelebilir. Kodda adları Türkçe sözcüklerin ASCII karşılıklarıyla yazarız (`ogrenciSayisi`, `toplamTutar`); Türkçe karakterler yalnızca ekrana yazılan metinlerde (tırnak içinde) kullanılır.

Ad	Geçerli mi?	Neden
`ogrenciSayisi`, `sayi2`, `toplam_tutar`	Evet	Harf, rakam ve alt çizgi; rakam başta değil
`2sayi`	Hayır	Rakamla başlıyor
`ogrenci sayisi`	Hayır	Boşluk içeriyor
`ortalama-not`	Hayır	`-` çıkarma işlecidir; `ortalama - not` diye okunur
`int`, `for`, `return`	Hayır	Anahtar sözcük
`Sayi` ve `sayi`	Evet, ayrı iki ad	Büyük/küçük harf duyarlı

Hatalı bir ad kullandığımızda derleyicinin ne dediğine bakalım:

adRakam.c

```c
#include <stdio.h>

int main(void) {
    int 2sayi = 5;
    printf("%d\n", 2sayi);
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

adRakam.c: In function 'main':
adRakam.c:4:9: error: invalid suffix "sayi" on integer constant
    4 |     int 2sayi = 5;
      |         ^~~~
adRakam.c:4:9: error: expected identifier or '(' before numeric constant
adRakam.c:5:20: error: invalid suffix "sayi" on integer constant
    5 |     printf("%d\n", 2sayi);
      |                    ^~~~
```

Derleyici `2sayi`'yi "2 sayısına yapışmış `sayi` eki" olarak okudu; bu yüzden ilk mesaj tuhaf görünür. Asıl ipucu ikinci mesajdadır: bir ad beklenen yerde sayı geldi. Anahtar sözcük kullanıldığında da benzer bir mesaj alırız (`int for = 5;` için `expected identifier or '(' before 'for'`).

Anahtar sözcükler

Anahtar sözcükler dilin kendisine ayrılmış sözcüklerdir ve başka bir amaçla kullanılamaz. C17'de 44 tane vardır:

Grup	Anahtar sözcükler
Tipler	`char` `short` `int` `long` `float` `double` `signed` `unsigned` `void` `_Bool` `_Complex` `_Imaginary`
Karar ve döngü	`if` `else` `switch` `case` `default` `for` `while` `do` `break` `continue` `goto`
Depolama ve niteleyiciler	`auto` `register` `static` `extern` `const` `volatile` `restrict` `inline` `_Thread_local` `_Atomic`
Tip tanımlama	`struct` `union` `enum` `typedef`
Diğer	`return` `sizeof` `_Alignas` `_Alignof` `_Generic` `_Noreturn` `_Static_assert`

Hepsini ezberlemek gerekmez; kod düzenleyiciler bunları ayrı renkte gösterir ve kitap ilerledikçe her birini tanıyacağız. `main` ve `printf` anahtar sözcük değildir, ama onların adını da değişkene vermeyiz.

> *Tarihçe · Anahtar sözcüklerin sayısı*
>
> C'nin ilk standardında (C89/C90) 32 anahtar sözcük vardı. C99 beş tane ekledi (`inline`, `restrict`, `_Bool`, `_Complex`, `_Imaginary`), C11 yedi tane daha (`_Alignas`, `_Alignof`, `_Atomic`, `_Generic`, `_Noreturn`, `_Static_assert`, `_Thread_local`). Yeni sözcüklerin alt çizgiyle ve büyük harfle başlaması, eski programlarda aynı adı kullanan değişkenlerin bozulmaması içindi. C23'te `bool`, `true`, `false` gibi bu sözcüklerin bir kısmı alt çizgisiz yazılabilen gerçek anahtar sözcükler oldu.

Alt çizgiyle başlayan adlar da dikkat ister. Alt çizgi ile büyük harf ya da iki alt çizgi ile başlayan adlar (`_Toplam`, `__x`) derleyici ve standart kütüphane için ayrılmıştır. Bu kitapta alt çizgiyle başlayan ad kullanmayacağız.

Adlandırma alışkanlıkları

Kurallar derleyiciyi memnun eder; alışkanlıklar ise kodu okuyan insanı. Kitap boyunca şu düzeni izleyeceğiz:

- Değişken ve fonksiyon adları **camelCase** yazılır: ilk sözcük küçük, sonrakilerin baş harfi büyük (`ogrenciSayisi`, `ortalamaNot`).
- Sabitler **BÜYÜK_HARFLE** ve sözcükler alt çizgiyle ayrılarak yazılır (`PI`, `MAX_NOT`, `HAFTA_GUN`).
- Ad, değişkenin ne tuttuğunu söylemelidir: `x` yerine `yas`, `n` yerine `ogrenciSayisi`.
- `i`, `j`, `k` gibi tek harfli adlar yalnızca kısa ömürlü döngü sayaçlarında kabul görür (Bölüm 8).

5.4 Tamsayı Tipleri

Bellek bit düzeyinde çalışır ve `n` bitle 2^n farklı değer ifade edilebilir. Tip, bir değişkene kaç bit ayrılacağını ve bu bitlerin nasıl yorumlanacağını belirler. Bir tamsayı tipinin kaç farklı değeri saklayabildiği, ayrılan bit sayısına bağlıdır: 8 bit 256 farklı değere, 16 bit 65.536 farklı değere, 32 bit yaklaşık 4,3 milyara izin verir.

C'nin tamsayı tipleri küçükten büyüğe şunlardır:

Tip	Tipik boyut	Tipik aralık (işaretli)	printf
`char`	1 bayt	-128 … 127	`%c` ya da `%d`
`short`	2 bayt	-32.768 … 32.767	`%hd`
`int`	4 bayt	yaklaşık ±2,1 milyar	`%d`
`long`	4 ya da 8 bayt	platforma göre değişir	`%ld`
`long long`	8 bayt	yaklaşık $\pm 9{,}2 \times 10^{18}$	`%lld`

Tabloda "tipik" sözcüğü bilinçli kullanıldı. C standardı boyutları kesin olarak vermez, yalnızca alt sınırlar koyar ve sıralamayı garanti eder:

- `char` en az 8 bit, `short` ve `int` en az 16 bit, `long` en az 32 bit, `long long` en az 64 bit olmalıdır.
- Boyutlar `char ≤ short ≤ int ≤ long ≤ long long` sıralamasına uyar.
- Kesin olarak bilinen tek değer `sizeof(char) == 1`'dir.

> *Tarihçe · int neden 4 bayt?*
>
> C geliştirilirken kullanılan PDP-11 bilgisayarında `int` 16 bitti. İşlemciler 32 bitlik sayıları doğrudan işleyebilir hâle geldikçe çoğu derleyici `int`'i 32 bite çıkardı. `long`'un 32 mi 64 bit mi olacağı ise işletim sistemi ailelerine göre ayrıştı; bu ayrışmayı birazdan göreceğiz.

sizeof ile boyut öğrenme

Bir tipin ya da değişkenin bellekte kaç bayt yer kaplayacağını `sizeof` operatörü söyler. Aşağıdaki program bu bilgisayardaki boyutları ve `<limits.h>` başlık dosyasının sunduğu sınır sabitlerini yazdırır:

▸ boyutlar.c

```c
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("char      : %zu\n", sizeof(char));
    printf("short     : %zu\n", sizeof(short));
    printf("int       : %zu\n", sizeof(int));
    printf("long      : %zu\n", sizeof(long));
    printf("long long : %zu\n", sizeof(long long));
    printf("INT_MIN   : %d\n", INT_MIN);
    printf("INT_MAX   : %d\n", INT_MAX);
    printf("UINT_MAX  : %u\n", UINT_MAX);
    printf("LLONG_MAX : %lld\n", LLONG_MAX);
    return 0;
}
```

```
ÇIKTI
char      : 1
short     : 2
int       : 4
long      : 8
long long : 8
INT_MIN   : -2147483648
INT_MAX   : 2147483647
UINT_MAX  : 4294967295
LLONG_MAX : 9223372036854775807
```

Bu çıktı 64 bitlik bir Linux sisteminde alındı. Aynı program 64 bitlik Windows'ta çalıştırıldığında `long` satırında 4 yazar; Linux ve macOS'ta 8'dir. Standart bunu da garanti etmediği için aynı kaynak kod farklı platformlarda farklı boyut bildirebilir. `long long` ise her yerde en az 64 bittir. Taşınabilir kod yazarken 4 bayttan büyük bir tamsayı gerekiyorsa `long`'a değil `long long`'a güvenilir.

`sizeof` bir fonksiyon değil **operatördür**. Tip adıyla (`sizeof(int)`) ya da bir değişkenle (`sizeof(yas)`) kullanılabilir; değişkenle kullanılırsa parantez zorunlu değildir ama okunurluk için yazılır. Sonucu `size_t` adlı işaretsiz bir tamsayı tipidir ve `printf` ile `%zu` kullanılarak yazdırılır. Bölüm 10'da dizilerin eleman sayısını bulurken de `sizeof`'u kullanacağız.

> **İpucu · Ekranda sayı yerine "zu" mu görüyorsunuz?**
>
> Çok eski Windows derleyicileri `%zu` belirtecini tanımaz ve ekrana sayı yerine `zu` yazar. Çözüm, ya `printf("%u\n", (unsigned)sizeof(int));` biçimini kullanmak ya da Bölüm 3'te kurduğumuz güncel derleyiciye geçmektir. Aynı eski derleyiciler `%lld` belirtecinde de sorun çıkarabilir.

İşaretli ve işaretsiz tipler

Tamsayı tipleri varsayılan olarak **işaretlidir** (signed): hem negatif hem pozitif değer tutabilir. Tipin başına `unsigned` (işaretsiz) yazarsak negatif değer tutmaz, ama pozitif tarafın üst sınırı yaklaşık iki katına çıkar:

Tip	Aralık	printf
`unsigned char`	0 … 255	`%u` ya da `%d`
`unsigned short`	0 … 65.535	`%hu`
`unsigned int` (kısaca `unsigned`)	0 … 4.294.967.295	`%u`
`unsigned long`	0 … 2^{64}-1 (Linux/macOS), 0 … 2^{32}-1 (Windows)	`%lu`
`unsigned long long`	0 … 2^{64}-1	`%llu`

n bitlik işaretli bir tamsayının aralığı **-2^{n-1} … 2^{n-1}-1**'dir; işaretsiz olan ise **0 … 2^n-1** aralığını kapsar. İşaretli tarafta negatif uç, pozitif uçtan bir fazladır: 8 bitte -128 … 127. Bu asimetri, negatif sayıların ikiye tümleyen (two's complement) biçiminde saklanmasından gelir; bunu Bölüm 16'da bit düzeyinde inceleyeceğiz.

Tip adlarını kısaltabiliriz: `short int` yerine `short`, `long int` yerine `long`, `unsigned int` yerine `unsigned` yazmak aynı anlama gelir. Kitapta okunurluk için çoğunlukla kısa biçimleri kullanacağız.

Bir tamsayı tipini seçerken pratik kurallar şunlardır:

- Genel amaçlı sayaç ve hesaplar için `int`.
- 2 milyarı aşabilecek büyüklükler (saniye cinsinden zaman, dosya boyutu, büyük çarpımlar) için `long long`.
- Bellek boyutu ve eleman sayısı için standart kütüphanenin kullandığı `size_t`.

- Negatif olamayacak bir değer olması, tipi `unsigned` yapmak için tek başına yeterli sebep değildir; `unsigned` tipler aşağıda göreceğimiz tuzakları da beraberinde getirir.

Sabit genişlikli tipler: stdint.h

Boyutun platforma göre değişmesi bazı programlarda kabul edilemez: bir ağ paketinin başlığını ya da bir dosya biçimini tanımlarken "tam olarak 32 bit" demek isteriz. C99 ile gelen `<stdint.h>` başlık dosyası bu ihtiyacı karşılar:

Tip	Anlamı
`int8_t`, `int16_t`, `int32_t`, `int64_t`	Tam olarak 8, 16, 32, 64 bitlik işaretli tamsayı
`uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`	Aynı genişliklerde işaretsiz tamsayı
`INT32_MAX`, `UINT8_MAX` vb.	Bu tiplerin sınır sabitleri

▸ sabitGenislik.c

```c
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

int main(void) {
    int32_t a = 2000000000;
    uint8_t b = 255;
    int64_t c = 9000000000000000000;

    printf("a = %" PRId32 "\n", a);
    printf("b = %u\n", b);
    printf("c = %" PRId64 "\n", c);
    printf("sizeof(int64_t) = %zu\n", sizeof(int64_t));
    printf("UINT8_MAX = %d\n", UINT8_MAX);
    return 0;
}
```

ÇIKTI

```
a = 2000000000
b = 255
c = 9000000000000000000
sizeof(int64_t) = 8
UINT8_MAX = 255
```

Burada alışılmadık bir söz dizimi var: `"%" PRId32 "\n"`. `<inttypes.h>` içindeki `PRId32` ve `PRId64`, o platformda doğru `printf` belirtecini üreten makrolardır. C'de yan yana yazılan metin sabitleri birleştirilir; derleyici bu ifadeyi `"a = %d\n"` ya da `"c = %ld\n"` gibi platforma uygun tek bir metne dönüştürür. Alışkanlık olarak tam genişlikli tipleri, boyutun gerçekten önemli olduğu yerlerde (dosya biçimleri, ağ, gömülü sistemler) kullanırız; sıradan hesaplarda `int` yeterlidir.

char: küçük bir tamsayı

`char` adı "karakter"den gelir, ama bellekte karakterin kendisini değil **kodunu** saklar. Örneğin `'A'` için 65, `'a'` için 97, `'0'` için 48 saklanır. Bu eşleştirmeye ASCII tablosu denir (tamamı Ek A'da verilmiştir). Dolayısıyla `char`, 1 baytlık küçük bir tamsayıdır ve aynı değeri `%c` ile karakter, `%d` ile sayı olarak yazdırabiliriz:

charTamsayi.c

```c
#include <stdio.h>

int main(void) {
    char harf = 'A';
    printf("%c %d\n", harf, harf);
    printf("%c %d\n", harf + 1, harf + 1);
    printf("%d\n", 'a' - 'A');

    char rakam = '7';
    printf("%d\n", rakam - '0');

    char kucuk = 'g';
    char buyuk = kucuk - 'a' + 'A';
    printf("%c -> %c\n", kucuk, buyuk);
    return 0;
}
```

ÇIKTI

```
A 65
B 66
32
7
g -> G
```

Bu programdan çıkan sonuçlar, ileride metinlerle çalışırken sürekli karşımıza çıkacak:

- Büyük harfle küçük harf arasında ASCII'de hep 32 fark vardır. Küçük harfi büyüğe çevirmek `kucuk - 'a' + 'A'` ile yapılır.
- `'7'` karakteri ile 7 sayısı aynı şey değildir: `'7'` ASCII'de 55'tir. Bir rakam karakterinin sayı değerini bulmak için `'0'` çıkarılır: `'7' - '0'` ifadesi 7 eder.
- Aritmetikte `char` önce `int`'e dönüştürülür. `harf + 1` bir `int`'tir; `%c` ile yazdırırsak karaktere (`B`), `%d` ile yazdırırsak koda (`66`) çevrilir.

> **Not · char işaretli mi, işaretsiz mi?**
>
> `char`'ın işaretli mi işaretsiz mi olduğunu standart derleyiciye bırakır. x86 işlemcilerde gcc `char`'ı işaretli (-128 … 127) kabul eder; bazı ARM platformlarında ise işaretsizdir (0 … 255). Bir `char`'ı sayı olarak kullanıp 127'den büyük değerlerle hesap yapacaksak tipi açıkça `signed char` ya da `unsigned char` yazmak gerekir. Metin işlerinde bu fark ortaya çıkmaz.

5.5 Kayan Noktalı Tipler

Tamsayılar sayma ve sıralama işleri için idealdir, ama bir odanın alanını, bir ürünün ağırlığını ya da bir ortalamayı hesaplarken kesirli sayılara ihtiyaç duyarız. C'de bunun için **kayan noktalı** (floating-point) tipler vardır. Adlarını, sayıdaki ondalık noktanın "kayabilmesinden" alırlar: sayı, bilimsel gösterimdeki gibi bir anlamlı kısım ve bir üsle saklanır (`1.75 = 1.75 × 10⁰`, `0.00032 = 3.2 × 10⁻⁴`).

Tip	Tipik boyut	Yaklaşık anlamlı basamak	Yaklaşık en büyük değer	printf
`float`	4 bayt	6-7	$3{,}4 \times 10^{38}$	`%f`
`double`	8 bayt	15-16	$1{,}8 \times 10^{308}$	`%f`

Tip	Tipik boyut	Yaklaşık anlamlı basamak	Yaklaşık en büyük değer	printf
`long double`	8, 12 ya da 16 bayt (platforma göre)	18-19 (x86 genişletilmiş)	platforma göre	`%Lf`

Günümüzde işlemcilerin neredeyse tamamı bu tipleri **IEEE 754** standardına göre saklar (standart ilk kez 1985'te yayımlandı). Standart `float` için "tek duyarlık", `double` için "çift duyarlık" der. C standardı IEEE 754'ü zorunlu kılmaz, ama pratikte karşılaşacağımız tüm derleyiciler onu kullanır.

> ◆ **Kritik Nokta · Ondalıklı sayı için varsayılan tip double'dır**
>
> Kodda `3.14` yazdığımızda sabit `double` tipindedir. Bellek kısıtı gibi özel bir sebep yoksa ondalıklı sayılar için `double` kullanırız. `float` yarı bellek kullanır, ama yalnızca yaklaşık 7 anlamlı basamak tutar; bu, günlük hesaplarda şaşırtıcı hatalara yol açabilir.

Ondalıklı sabitleri yazmak

Ondalıklı bir sabit, nokta içeren ya da bilimsel gösterimle yazılan sabittir. Bilimsel gösterimde `e` (ya da `E`) harfi "10'un kuvveti" anlamına gelir:

Yazılış	Değeri	Tipi
`3.14`	3,14	`double`
`3.14f`	3,14	`float`
`3.14L`	3,14	`long double`
`2.5e3`	$2{,}5 \times 10^{3} = 2500$	`double`
`1.6e-19`	$1{,}6 \times 10^{-19}$	`double`
`.5` ve `5.`	0,5 ve 5,0	`double`

Sabitin tipini `f` ya da `L` sonekiyle değiştirebiliriz. Bir `float` değişkene `0.1` yazarsak değer önce `double` olarak oluşur, sonra `float`'a çevrilir; bu çoğu zaman zararsızdır, ama `0.1f` yazmak niyeti açıkça gösterir.

Sınırları float.h ile öğrenmek

Tamsayı sınırları `<limits.h>` içindeydi; kayan noktalı tiplerin sınırları `<float.h>` başlık dosyasındadır. Bu dosyadaki `FLT_DIG` ve `DBL_DIG` sabitleri, ondalık yazılıp saklanıp tekrar okunduğunda değişmeden kalan basamak sayısını verir; `FLT_MAX` ve `DBL_MAX` ise en büyük sonlu değerlerdir. `FLT_EPSILON` ve `DBL_EPSILON`, 1,0 ile ondan büyük en küçük sayı arasındaki farktır; yani 1,0 civarında ayırt edilebilen en küçük aralıktır.

▸ kayanTipler.c

```c
#include <stdio.h>
#include <float.h>

int main(void) {
    printf("float  boyut : %zu bayt\n", sizeof(float));
    printf("double boyut : %zu bayt\n", sizeof(double));
    printf("FLT_DIG      : %d\n", FLT_DIG);
    printf("DBL_DIG      : %d\n", DBL_DIG);
    printf("FLT_MAX      : %e\n", FLT_MAX);
    printf("DBL_MAX      : %e\n", DBL_MAX);
    printf("FLT_EPSILON  : %e\n", FLT_EPSILON);
```

```
    printf("DBL_EPSILON  : %e\n", DBL_EPSILON);
    return 0;
}
```

```
ÇIKTI
float  boyut : 4 bayt
double boyut : 8 bayt
FLT_DIG      : 6
DBL_DIG      : 15
FLT_MAX      : 3.402823e+38
DBL_MAX      : 1.797693e+308
FLT_EPSILON  : 1.192093e-07
DBL_EPSILON  : 2.220446e-16
```

`%e` belirteci sayıyı bilimsel gösterimle yazar: `3.402823e+38`, 3,402823 × 10^{38} demektir. Çok büyük ya da çok küçük sayıları `%f` ile yazdırmak onlarca basamaklı okunmaz çıktılar üretir; bu durumda `%e` (ya da `%g`) tercih edilir.

Sonsuz ve tanımsız sonuçlar

Kayan noktalı sayılarda sıfıra bölme programı çökertmez. IEEE 754 kuralları gereği sonuç, sonsuz (`inf`) ya da "sayı değil" anlamına gelen `nan` (not a number) olur. Bu, tamsayı bölmesinden köklü bir farktır; ona aritmetik başlığında döneceğiz.

▸ sonsuz.c

```c
#include <stdio.h>

int main(void) {
    double sifir = 0.0;
    double sonuc = 1.0 / sifir;
    printf("%f\n", sonuc);
    printf("%f\n", -1.0 / sifir);
    printf("%d\n", sonuc > 1e308);
    return 0;
}
```

```
ÇIKTI
inf
-inf
1
```

Sıfıra bölmeyi sabit bir sayıyla (`1.0 / 0.0`) yazmak yerine bir değişken üzerinden yaptık; çünkü derleyici sabit ifadeyi görürse bir uyarı verebilir. `0.0 / 0.0` gibi anlamsız işlemler `nan` üretir ve `nan` hiçbir sayıya, kendisine bile eşit değildir. `<math.h>` içindeki `isnan` ve `isinf` makroları bu durumları test eder (Bölüm 9).

5.6 Sabitler

Bir değer program boyunca değişmeyecekse bunu koda açıkça belirtmek hem hatayı önler hem de kodu okunur kılar. π sayısı, haftadaki gün sayısı, vergi oranı bu türdendir. Bu değerleri kodun içine çıplak sayılar olarak gömmek kötü bir alışkanlıktır; bunlara **sihirli sayı** (magic number) denir. Kodda `0.20` yazan yüz yerin hangisinin vergi oranı olduğunu kimse hatırlamaz. Sabit kullanmanın üç yolu vardır: sabit değerler (literal), `const` değişkenler ve `#define` ile tanımlanan makrolar.

Sabit değerler (literal)

Koda doğrudan yazdığımız değerlere **sabit değer** (literal) denir. Yazılış biçimi, sabitin tipini belirler:

Yazılış	Tip	Açıklama
`5`	`int`	Onluk tamsayı
`5L` , `5LL`	`long` , `long long`	Sonek tipi seçer
`5U` , `5UL` , `5ULL`	`unsigned` türevleri	`U` işaretsiz anlamına gelir
`5.0`	`double`	Noktalı sayı varsayılan olarak `double`
`5.0f`	`float`	`f` soneki
`'A'`	`int` (C'de)	Tek tırnak, tek karakter; değeri karakter kodudur (65)
`"A"`	karakter dizisi	Çift tırnak; `'A'` ve `'\0'` olmak üzere iki karakterlik dizi (Bölüm 11)
`0x1F`	`int`	Onaltılık (hexadecimal): 31
`017`	`int`	Sekizlik (octal), başında `0` : 15

Onluk bir tamsayı sabitinin tipi, içine sığdığı ilk tiptir: sırasıyla `int` , `long` , `long long` . Bu kural bir ayrıntıyı gösterir: `2147483647` bir `int` 'tir, ama `2147483648` bir `int` olamaz çünkü sığmaz; derleyici onu bir üst tipe yükseltir.

▸ sabitTipler.c

```c
#include <stdio.h>

int main(void) {
    printf("%zu %zu %zu\n", sizeof(5), sizeof(5.0), sizeof(5.0f));
    printf("%zu %zu\n", sizeof(2147483647), sizeof(2147483648));
    printf("%zu\n", sizeof('A'));
    printf("%d %d %d\n", 0x1F, 017, 010);
    printf("%x %X %o\n", 255, 255, 8);
    return 0;
}
```

```
ÇIKTI
4 8 4
4 8
4
31 15 8
ff FF 10
```

Bu çıktıda iki tuzak gizli:

- C'de `'A'` bir `char` değil, `int` tipindedir; bu yüzden `sizeof('A')` 4'tür. C++'ta aynı ifade `char` olduğu için 1 verir (C++ kısmında karşılaştıracağız).
- Başına `0` yazılan bir tamsayı **sekizlik** kabul edilir. `010` , on değil sekiz eder. Bu sebeple sayıların başına görsel hizalama için sıfır koymayız.

Son satırda `printf` 'in onaltılık (`%x` , küçük harflerle; `%X` , büyük harflerle) ve sekizlik (`%o`) yazdırma belirteçlerini kullandık. 255'in onaltılıkta `ff` , 8'in sekizlikte `10` olduğunu hatırlayalım (Bölüm 1'deki sayı sistemleri).

Karakter sabitlerinde, çıktıda gördüğümüz kaçış dizileri (`\n`, `\t`, `\\`, `\'`, `\"`, `\0`) de birer tek karakterdir ve `char` türünde saklanır. `'\n'` ASCII'de 10, `'\0'` ise 0 değerindedir.

> *Tarihçe · İkilik, ayraç ve yeni sabit yazımları*
>
> Sekizlik yazım C'nin çıkış dönemindeki makinelerin bellek yapısından (3 bitlik gruplar) gelir ve bu yüzden başında `0` vardır. İkilik sabitler (`0b1010`) uzun süre standart dışı bir derleyici eklentisiydi; C23 bunları resmen dile ekledi. C23 ayrıca büyük sayıları okunur kılmak için basamak ayracını da getirdi (`1'000'000`). Bu kitaptaki örnekler C17'ye uygun olduğundan bu iki yazımı kullanmayacağız.

const ile sabit

Bir değişkenin tanımına `const` niteleyicisini (qualifier) eklersek değişken **salt okunur** olur: değeri tanımlarken veririz, sonra değiştiremeyiz.

▸ constAlan.c

```c
#include <stdio.h>

int main(void) {
    const double PI = 3.14159;
    const int HAFTA_GUN = 7;
    double yaricap = 2.0;

    printf("Çevre: %.2f\n", 2 * PI * yaricap);
    printf("Haftada %d gün var.\n", HAFTA_GUN);
    return 0;
}
```

ÇIKTI

```
Çevre: 12.57
Haftada 7 gün var.
```

Bir sabite sonradan değer atamaya çalışırsak derleyici programı reddeder:

▸ constHata.c

```c
#include <stdio.h>

int main(void) {
    const int HAFTA_GUN = 7;
    HAFTA_GUN = 8;
    printf("%d\n", HAFTA_GUN);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
constHata.c: In function 'main':
constHata.c:5:15: error: assignment of read-only variable 'HAFTA_GUN'
    5 |     HAFTA_GUN = 8;
      |               ^
```

`const` değişkenin bir tipi vardır, bellekte yer kaplar ve derleyici ona yanlış tipte değer atandığında uyarı verebilir. Bu, bir sonraki bölümde göreceğimiz `#define` yöntemine göre önemli bir üstünlüktür.

> *Tarihçe · const'un kökeni*
>
> `const` niteleyicisi C'nin ilk sürümlerinde yoktu. Fikir 1980'lerin başında Bjarne Stroustrup'un "C with Classes" dilinde doğdu ve ANSI C'ye (C89) oradan geçti. Yani `const`, C'den C++'a değil, tersine C++'ın öncüsünden C'ye geçmiş bir özelliktir.

#define ile sabit

`#define` bir **önişlemci** komutudur (Bölüm 4'te derleme sürecinin ilk adımı olarak gördük). Önişlemci, derleme başlamadan önce kaynak metni tarar ve tanımlanan adı karşılığıyla "bul ve değiştir" mantığıyla yer değiştirir. `#define` bir değişken yaratmaz, bellekte yer kaplamaz.

▸ defineSabit.c

```c
#include <stdio.h>

#define PI 3.14159
#define MAX_NOT 100

int main(void) {
    printf("Alan: %.2f\n", PI * 2 * 2);
    printf("En yüksek not: %d\n", MAX_NOT);
    return 0;
}
```

ÇIKTI

```
Alan: 12.57
En yüksek not: 100
```

Önişlemci `PI * 2 * 2` ifadesini `3.14159 * 2 * 2` olarak yeniden yazdı; derleyici hiç `PI` adını görmedi. Söz dizimi `#define AD DEGER` biçimindedir: sonunda `;` yoktur, `=` yoktur, tip yoktur. Makrolar Bölüm 15'te ayrıntılı işlenecek.

▲ Sık Yapılan Hata · #define satırına ; koymak

`#define` düz metin değiştirmedir; satırın sonundaki `;` de değiştirilen metne dahildir. Aşağıdaki program masum görünür, ama `KDV_ORANI` yerine `0.20;` yazılır:

▸ defNokta.c

```c
#include <stdio.h>

#define KDV_ORANI 0.20;

int main(void) {
    double fiyat = 100 * (1 + KDV_ORANI);
    printf("%.2f\n", fiyat);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
defNokta.c: In function 'main':
defNokta.c:3:23: error: expected ')' before ';' token
    3 | #define KDV_ORANI 0.20;
      |                       ^
defNokta.c:6:31: note: in expansion of macro 'KDV_ORANI'
    6 |     double fiyat = 100 * (1 + KDV_ORANI);
      |                               ^~~~~~~~
defNokta.c:6:26: note: to match this '('
    6 |     double fiyat = 100 * (1 + KDV_ORANI);
      |                          ^
```

Satır `100 * (1 + 0.20;);` biçimine döner. Hata mesajı `#define` satırını gösterir ve `in expansion of macro` notunu düşer. Aynı mantıkla `#define PI = 3.14` de yanlıştır: `=` işareti de metne karışır.

`const` ile `#define` arasındaki farkı özetleyelim:

Özellik	const	#define
Kim işler?	Derleyici	Önişlemci
Tipi var mı?	Evet (`const int`)	Hayır, yalnızca metin
Bellekte yer kaplar mı?	Evet, bir değişkendir	Hayır
Sonunda `;`	Var	Yok
Kapsam kuralları	Değişkenin kapsamı (blok, dosya)	Tanımdan dosya sonuna kadar
Hata ayıklayıcıda görünür mü?	Evet	Hayır

İkisi de kullanılır. Tip denetimi sağladığı için yeni yazılan kodda `const` tercih edilir; `#define` ise eski kodlarda ve koşullu derlemede sık görülür.

Not · C'de const değişken "sabit ifade" değildir

Sık düşülen bir yanılgı: `const int N = 10;` yazınca `N`'yi dizi boyutu olarak her yerde kullanabileceğimizi sanmaktır. C'de `const` ile tanımlanan değişken, derleyici açısından hâlâ bir değişkendir; "derleme zamanı sabiti" sayılmaz. Dizi boyutları ve `case` etiketleri için `#define` ya da `enum` kullanılır (Bölüm 10 ve Bölüm 14). C++ bu konuda farklıdır: orada `const int N = 10;` gerçek bir sabittir.

5.7 printf ile Değer Yazdırma: Biçim Belirteçleri

Değişkenlerle çalışmaya başladığımıza göre, değerlerini ekrana doğru biçimde yazdırmayı bilmemiz gerekir. `printf` ilk argümanı olan **biçim metnindeki** (format string) `%` ile başlayan belirteçleri, sonraki argümanlarla sırayla değiştirir. Her belirteç hem argümanın tipini hem de gösterim biçimini söyler. Ayrıntıları Bölüm 6'da işleyeceğiz; bu bölümdeki örnekleri yazıp okuyabilmek için gerekenleri burada toplayalım.

Belirteç	Argüman tipi	Örnek çıktı
`%d` ya da `%i`	`int`	`-42`
`%u`	`unsigned int`	`4294967295`
`%ld` / `%lld`	`long` / `long long`	`9223372036854775807`
`%lu` / `%llu`	`unsigned long` / `unsigned long long`	`18446744073709551615`
`%hd` , `%hhd`	`short` , `char` (sayı olarak)	`-32768`
`%zu`	`size_t` (`sizeof` sonucu)	`8`
`%f`	`double` (ve `float`)	`3.141593`
`%.2f`	`double` , 2 ondalık basamak	`3.14`
`%e`	`double` , bilimsel gösterim	`3.141593e+00`
`%g`	`double` , kısa gösterim	`3.14159`
`%Lf`	`long double`	`3.141593`
`%c`	`int` / `char` , karakter olarak	`A`
`%s`	metin (karakter dizisi)	`Merhaba`
`%x` , `%X`	`unsigned` , onaltılık	`ff` , `FF`
`%o`	`unsigned` , sekizlik	`377`
`%p`	işaretçi (adres)	`0x7ffd5c3a` (örnek)
`%%`	`%` karakterinin kendisi	`%`

Birkaç ayrıntı kolayca kaçırılır:

- `float` argümanlar `printf` 'e `double` olarak geçer; bu yüzden `float` için de `%f` kullanılır. `%lf` belirteci `printf` 'te `%f` ile aynı anlama gelir (C99'dan beri), ama gereksizdir. `scanf` ile okurken bu fark önemli hâle gelecek (Bölüm 6).
- `%f` varsayılan olarak noktadan sonra **6 basamak** yazar. Basamak sayısını `%.Nf` biçimiyle ayarlarız: `%.0f` , `%.1f` , `%.3f` . Sayı gerektiğinde **yuvarlanır**.
- `%d` ile yazdırılan değerin gerçekten `int` olması gerekir. `long` bir değeri `%d` ile yazdırmak yanlış sonuç verir.
- `%` karakterini yazmak için `%%` yazılır.

Genişlik ve hizalama için belirteçlerin arasına sayı eklenebilir: `%5d` sayıyı en az 5 karakterlik alana sağa yaslı yazar, `%-5d` sola yaslı yazar, `%05d` boşlukları sıfırla doldurur, `%8.3f` toplam 8 karakter genişliğinde ve 3 ondalık basamakla yazar. `%+d` pozitif sayılarda da işareti gösterir.

printfOrnek.c

```c
#include <stdio.h>

int main(void) {
    int adet = 42;
    double fiyat = 19.5;
    long long buyuk = 9000000000LL;
    unsigned int isaretsiz = 4000000000u;
    char harf = 'k';

    printf("[%d] [%5d] [%-5d] [%05d] [%+d]\n", adet, adet, adet, adet, adet);
    printf("[%f] [%.1f] [%8.3f] [%e] [%g]\n", fiyat, fiyat, fiyat, fiyat, fiyat);
    printf("%lld %u\n", buyuk, isaretsiz);
    printf("%c %d %x %o\n", harf, harf, 255, 255);
    printf("Başarı oranı: %d%%\n", 95);
    return 0;
}
```

ÇIKTI

```
[42] [   42] [42   ] [00042] [+42]
[19.500000] [19.5] [  19.500] [1.950000e+01] [19.5]
9000000000 4000000000
k 107 ff 377
Başarı oranı: 95%
```

Her çıktıyı elle doğrulamak iyi bir alıştırmadır. Köşeli parantezler, genişlik ve hizalamanın etkisini görünür kılmak için kullanıldı.

▲ Sık Yapılan Hata · Belirteç ile argüman uyuşmazlığı

Belirteç ile argümanın tipi uyuşmazsa sonuç tanımsızdır. `printf("%f", sayi)` ifadesinde `sayi` bir `int` ise çıktı anlamsız bir sayı olur; `printf("%d", 3.5)` de öyle. `printf`, argümanların gerçek tipini bilmez; yalnızca biçim metnine güvenir. gcc'nin `-Wall` seçeneği bu uyuşmazlıkları yakalar, bu yüzden uyarıları okumak önemlidir:

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra fmt.c -o fmt
fmt.c: In function 'main':
fmt.c:6:14: warning: format '%f' expects argument of type 'double', but argument 2 has
type 'int' [-Wformat=]
    6 |     printf("%f\n", sayi);
      |             ~^     ~~~~
      |              |     |
      |              |     int
      |              double
      |             %d
fmt.c:7:14: warning: format '%d' expects argument of type 'int', but argument 2 has type '
double' [-Wformat=]
    7 |     printf("%d\n", oran);
      |             ~^     ~~~~
      |              |     |
      |              int   double
      |             %f
```

5.8 Aritmetik Operatörler

> **Tanım: Operatör ve işlenen**
>
> **Operatör** (operator), bir işlemi gösteren simgedir. İşlemin uygulandığı değerlere **işlenen** (operand) denir. `a + b` ifadesinde `+` operatör, `a` ve `b` işlenendir. Bir veya daha fazla işlenen ile operatörün birleşimine **ifade** (expression) denir; her ifadenin bir değeri vardır.

C'nin beş aritmetik operatörü vardır: toplama `+`, çıkarma `-`, çarpma `*`, bölme `/` ve bölümden kalanı veren `%`. Ayrıca işareti belirleyen tekli (unary) `-` ve `+` operatörleri bulunur (`-x`, `+x`). C'de üs alma operatörü **yoktur**; bunun için `<math.h>` içindeki `pow` fonksiyonu kullanılır (Bölüm 9).

Önce hepsini birlikte görelim, sonra bölmeye ve `%`'ye daha yakından bakalım:

▸ aritmetik.c

```c
#include <stdio.h>

int main(void) {
    int a = 17, b = 5;

    printf("%d + %d = %d\n", a, b, a + b);
    printf("%d - %d = %d\n", a, b, a - b);
    printf("%d * %d = %d\n", a, b, a * b);
    printf("%d / %d = %d\n", a, b, a / b);
    printf("%d %% %d = %d\n", a, b, a % b);
    printf("1 / 2   = %d\n", 1 / 2);
    printf("-7 / 2  = %d\n", -7 / 2);
    printf("7 / 2.0 = %.1f\n", 7 / 2.0);
    return 0;
}
```

ÇIKTI

```
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3
17 % 5 = 2
1 / 2   = 0
-7 / 2  = -3
7 / 2.0 = 3.5
```

Tamsayı bölmesi

`17 / 5` neden 3,4 değil de 3? Çünkü C'de iki işlenen de tamsayıysa bölmenin sonucu da tamsayıdır: kesirli kısım atılır. Bu **yuvarlama değil, kesmedir**. `1 / 2` bu yüzden 0'dır. İşlenenlerden en az biri ondalıklıysa (`7 / 2.0`) bölme ondalıklı yapılır ve sonuç `double` olur.

Negatif sayılarda kesme **sıfıra doğrudur**: `-7 / 2` sonucu -4 değil, -3'tür (-3,5 sıfıra doğru -3'e kesilir).

> *Tarihçe · Negatif sayıların bölmesi*
>
> C89 standardı, negatif işlenenler için bölmenin sıfıra mı yoksa aşağıya mı yuvarlanacağını derleyiciye bırakmıştı. C99, davranışı sıfıra doğru kesme olarak sabitledi. Bu kural C++11'de ve C#'ta da aynıdır. Bazı diller (örneğin Python) `-7 // 2` için -4 verir; bu ayrım diller arası taşınan kodda hataya yol açabilir.

Sıfıra bölme

Tamsayıyı sıfıra bölmek tanımsız davranıştır. Pratikte programın çalışma zamanında çökmesine yol açar; Linux'ta bu `Floating point exception` mesajıyla görünür (adı yanıltıcıdır, tamsayı bölmesi için de aynı mesaj çıkar).

```
int pay = 10;
int payda = 0;
int sonuc = pay / payda;     // çalışma zamanı hatası
int kalan = pay % payda;     // aynı şekilde
```

Bölenin sıfır olabileceği her yerde bölmeden önce kontrol etmek zorundayız. Bunu yapmak için koşul ifadelerine (Bölüm 7) ihtiyacımız olacak. Kayan noktalı bölme ise, daha önce gördüğümüz gibi, çökmez; `inf` ya da `nan` üretir.

Kalan operatörü: %

`a % b`, `a`'yı `b`'ye böldüğümüzde artan **kalandır**. Bölme sıfıra doğru kesildiği için şu eşitlik her zaman sağlanır:

```
a == (a / b) * b + (a % b)
```

Bundan, kalanın işaretinin **bölünenin işaretiyle aynı** olduğu sonucu çıkar:

▸ modNegatif.c

```
#include <stdio.h>

int main(void) {
    printf("7 %% 2   = %d\n", 7 % 2);
    printf("-7 %% 2  = %d\n", -7 % 2);
    printf("7 %% -2  = %d\n", 7 % -2);
    printf("-7 %% -2 = %d\n", -7 % -2);
    printf("2 %% 5   = %d\n", 2 % 5);
    return 0;
}
```

ÇIKTI

```
7 % 2   = 1
-7 % 2  = -1
7 % -2  = 1
-7 % -2 = -1
2 % 5   = 2
```

Küçük sayıyı büyüğe bölünce kalan, sayının kendisidir: `2 % 5` sonucu 2'dir. Başka bazı dillerde `-7 % 2` sonucu 1 çıkar; C'de -1'dir. Negatif sayılarla `%` kullanan kod başka dile taşınırken bu fark akılda tutulmalıdır.

`%` operatörünün üç sık kullanım yeri vardır ve programlamada sürekli karşımıza çıkar:

- **Çift/tek ayrımı:** `sayi % 2` çift sayılarda 0, pozitif tek sayılarda 1 verir.
- **Basamak ayırma:** `1453 % 10` son basamağı (3), `1453 / 10` son basamağı atılmış sayıyı (145) verir.
- **Başa dönen sayaçlar:** saat gibi 24'te başa dönen değerler için `(22 + 5) % 24` sonucu 3'tür.

İlk ikisini birleştiren küçük bir örnek, saniye cinsinden bir süreyi saat, dakika ve saniyeye ayırır:

▸ sureAyir.c

```c
#include <stdio.h>

int main(void) {
    int toplamSaniye = 7384;

    int saat = toplamSaniye / 3600;
    int kalan = toplamSaniye % 3600;
    int dakika = kalan / 60;
    int saniye = kalan % 60;

    printf("%d saniye = %d saat %d dakika %d saniye\n",
           toplamSaniye, saat, dakika, saniye);

    int sayi = 1453;
    printf("Son basamak: %d\n", sayi % 10);
    printf("Son basamaksız: %d\n", sayi / 10);
    printf("Saat 22 iken 5 saat sonra: %d\n", (22 + 5) % 24);
    return 0;
}
```

ÇIKTI

```
7384 saniye = 2 saat 3 dakika 4 saniye
Son basamak: 3
Son basamaksız: 145
Saat 22 iken 5 saat sonra: 3
```

Önce 3600'e bölerek tam saatleri bulduk; geriye kalan saniyeyi 60'a bölerek dakikaları, onun kalanını da saniyeleri elde ettik. Bu "bölüm ve kalan" ikilisi, bir ölçüyü daha küçük birimlere ayırmanın standart yoludur.

`%` yalnızca tamsayılarla çalışır. İşlenenlerden biri `double` ise program derlenmez; ondalıklı sayılar için `<math.h>` içindeki `fmod` kullanılır (Bölüm 9):

▸ modDouble.c

```c
#include <stdio.h>

int main(void) {
    double a = 7.5;
    printf("%f\n", a % 2);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
modDouble.c: In function 'main':
modDouble.c:5:22: error: invalid operands to binary % (have 'double' and 'int')
    5 |     printf("%f\n", a % 2);
      |                     ^
```

◆ **Kritik Nokta · Bölme ve kalan için üç kural**

1. İki tamsayının bölmesi tamsayıdır; kesirli kısım atılır.
2. `%` işleminin sonucu bölünenin işaretini taşır: `-17 % 5` sonucu -2, `17 % -5` sonucu 2'dir.
3. Aynı önceliğe sahip operatörler soldan sağa işlenir: `7 / 2 * 2.0` önce `7 / 2 = 3`, sonra `3 * 2.0 = 6.0` olur; 7,0 değil.

Not · Bölme ve kalan üç dilde

Tamsayı bölmesi ve işaret kuralı üç dilde de aynıdır: `7 / 2` her birinde 3, `-7 / 2` her birinde -3, `-7 % 2` her birinde -1 verir.

Dil	7 / 2	7 / 2.0	a % b ondalıklı sayıda
C	3	3.5	Hata, `fmod` gerekir
C++	3	3.5	Hata, `std::fmod` gerekir
C#	3	3.5	Çalışır: `7.5 % 2` sonucu 1.5

C#'ta `%` operatörü `double` ile de kullanılabilir ve işareti bölünenden alır. Ayrıca C#'ta tamsayıyı sıfıra bölmek bir `DivideByZeroException` istisnası fırlatır; program hatayı yakalayabilir.

5.9 Atama Operatörleri, Artırma ve Azaltma

Bileşik atama

`x = x + 5` gibi "değişkeni hesapla ve aynı değişkene geri yaz" kalıbı o kadar sık geçer ki C'de kısaltması vardır. **Bileşik atama** (compound assignment) operatörleri `x op= y` biçimindedir ve `x = x op y` anlamına gelir: `+=`, `-=`, `*=`, `/=`, `%=`.

▸ bilesik.c

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    x += 5;                 // 15
    x -= 3;                 // 12
    x *= 2;                 // 24
    x /= 5;                 // 24 / 5: tamsayı bölmesi, 4
    x %= 3;                 // 4 % 3 = 1
    printf("x = %d\n", x);

    int y = 4;
    y *= 2 + 3;             // y = y * (2 + 3)
    printf("y = %d\n", y);
    return 0;
}
```

ÇIKTI

```
x = 1
y = 20
```

Dikkat edilecek nokta: `y *= 2 + 3` ifadesi `y = y * 2 + 3` demek değildir. Sağ taraf bir bütün olarak hesaplanır; ifade `y = y * (2 + 3)` olarak çalışır ve sonuç 11 değil 20'dir.

Artırma ve azaltma: ++ ve --

Bir değişkeni 1 artırmak o kadar yaygındır ki bunun ayrı bir operatörü vardır: `x++`, `x = x + 1` anlamına gelir; `x--` ise `x = x - 1` anlamındadır. Operatör değişkenin önüne (ön ek, prefix) ya da arkasına (son ek, postfix)

yazılabilir. Tek başına bir satırda kullanıldığında ikisi arasında fark yoktur. Bir ifadenin içinde kullanıldığında fark ortaya çıkar:

- **Son ek** (`x++`): ifadenin değeri `x`'in **eski** değeridir; ardından `x` bir artar.
- **Ön ek** (`++x`): önce `x` bir artar; ifadenin değeri **yeni** değerdir.

`--` için mantık aynıdır.

onSonEk.c

```c
#include <stdio.h>

int main(void) {
    int n = 5;
    printf("%d\n", n++);    // 5 yazar, sonra n = 6
    printf("%d\n", n);      // 6
    printf("%d\n", ++n);    // önce n = 7, sonra 7 yazar
    printf("%d\n", n--);    // 7 yazar, sonra n = 6
    printf("%d\n", --n);    // önce n = 5, sonra 5 yazar
    return 0;
}
```

ÇIKTI

```
5
6
7
7
5
```

Adım adım izleyelim: `n++` eski değer olan 5'i verir, sonra `n` 6 olur. Ardından `n`'yi yazdırınca 6 görürüz. `++n` önce `n`'yi 7 yapar ve 7'yi yazdırır. `n--` eski değer olan 7'yi verir, sonra `n` 6 olur. `--n` önce `n`'yi 5 yapar ve 5'i yazdırır.

`++` ve `--` yalnızca değişkenlere uygulanır; `5++` ya da `(a + b)++` yazılamaz.

▲ Sık Yapılan Hata · Aynı ifadede bir değişkeni birden fazla değiştirmek

Bir değişkeni tek bir ifadede iki kez değiştirmek (ya da `++` / `--` ile değiştirip aynı ifadede bir de okumak) tanımsız davranıştır. Derleyici hangi `i++`'ın önce yapılacağını garanti etmez:

```c
int i = 5;
int x = i++ + i++;          // tanımsız
i = i++;                    // tanımsız
printf("%d %d", i, i++);    // tanımsız
```

gcc bu tür kodları `-Wall` ile yakalar:

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra seq.c -o seq
seq.c: In function 'main':
seq.c:5:7: warning: operation on 'i' may be undefined [-Wsequence-point]
    5 |     i = i++ + ++i;
      |     ~~^~~~~~~~~~
```

Sonuç derleyici sürümüne, hatta optimizasyon ayarına göre değişebilir. Kural basittir: bir değişkeni bir ifadede en fazla bir kez değiştirin ve `++` / `--` kullanan satırı sade tutun.

> **Not · Diğer dillerde ++ ve --**
>
> `++` ve `--` operatörleri C++ ve C#'ta da vardır ve ön ek/son ek farkı aynıdır. Değerlendirme sırasında fark vardır: C#'ta bir ifadenin işlenenleri kesin olarak soldan sağa değerlendirilir, bu nedenle `i++ + i++` gibi bir ifadenin sonucu tanımlıdır. C ve C++'ta bu tür yazımlardan kaçınılır.

5.10 İlişkisel ve Mantıksal Operatörler

Karşılaştırma operatörleri

İki değeri karşılaştırıp sonuç olarak doğru ya da yanlış üreten operatörlere **ilişkisel** (relational) operatörler denir:

Operatör	Anlamı	Örnek	Sonuç
`==`	eşit	`5 == 5`	1
`!=`	eşit değil	`5 != 5`	0
`<`	küçük	`3 < 7`	1
`>`	büyük	`3 > 7`	0
`<=`	küçük veya eşit	`4 <= 4`	1
`>=`	büyük veya eşit	`3 >= 4`	0

C'nin bu konuda önemli bir özelliği vardır: karşılaştırma sonucu için **ayrı bir doğru/yanlış tipi yoktur** (`_Bool`'a birazdan değineceğiz). Sonuç bir `int`'tir: ifade doğruysa **1**, yanlışsa **0** üretir. Ters yönde de kural basittir: bir koşul beklenen yerde **sıfır yanlış, sıfırdan farklı her değer doğru** kabul edilir.

> **Tanım: Doğru ve yanlış (C'de)**
>
> C'de yanlış, sıfır değerinin (`0`, `0.0`, `'\0'`, boş işaretçi) kendisidir. Sıfırdan farklı her değer doğru sayılır. İlişkisel ve mantıksal operatörlerin ürettiği doğru değeri her zaman `1`'dir.

Mantıksal operatörler

Birden fazla koşulu birleştirmek için üç **mantıksal** (logical) operatör vardır:

Operatör	Adı	Ne zaman doğru?
`&&`	VE (AND)	İki işlenen de doğruysa
`||`	VEYA (OR)	En az biri doğruysa
`!`	DEĞİL (NOT)	İşlenen yanlışsa (tekli operatör)

Doğruluk tablosu:

a	b	a VE b	a VEYA b	DEĞİL a
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0

a	b	a VE b	a VEYA b	DEĞİL a
1	1	1	1	0

Sonuçları bir `printf` ile görmek, kuralları pekiştirir:

▸ karsilastirma.c

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 3;

    printf("a > b         : %d\n", a > b);
    printf("a == b        : %d\n", a == b);
    printf("a != b        : %d\n", a != b);
    printf("a >= 7        : %d\n", a >= 7);
    printf("a > 5 && b > 5 : %d\n", a > 5 && b > 5);
    printf("a > 5 || b > 5 : %d\n", a > 5 || b > 5);
    printf("!(a > b)      : %d\n", !(a > b));
    printf("!5            : %d\n", !5);
    printf("!0            : %d\n", !0);
    printf("5 && -2       : %d\n", 5 && -2);
    return 0;
}
```

```
ÇIKTI
a > b         : 1
a == b        : 0
a != b        : 1
a >= 7        : 1
a > 5 && b > 5 : 0
a > 5 || b > 5 : 1
!(a > b)      : 0
!5            : 0
!0            : 1
5 && -2       : 1
```

Son üç satır C'nin "sıfır yanlış, gerisi doğru" kuralını gösterir: `!5` 0 olur (5 doğru sayıldığı için tersi yanlıştır), `!0` 1 olur ve `5 && -2` ifadesinde iki işlenen de sıfırdan farklı olduğundan sonuç 1'dir; sonucun 5 ya da -2 olmadığına, yalnızca 1 ya da 0 olduğuna dikkat edelim.

Kısa devre değerlendirme

`&&` ve `||` operatörleri soldan sağa işlenir ve sonuç belli olur olmaz durur. Buna **kısa devre** (short-circuit) değerlendirme denir:

- `a && b` ifadesinde `a` yanlışsa sonuç ne olursa olsun yanlıştır; `b` hiç hesaplanmaz.
- `a || b` ifadesinde `a` doğruysa sonuç doğrudur; `b` hiç hesaplanmaz.

Bu yalnızca bir hız iyileştirmesi değildir; yan etkisi olan ifadelerde davranışı değiştirir:

▸ kisaDevre.c

```c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0, c = 0;
```

```
    int r1 = (a != 0) && (++b > 0);   // sol taraf yanlış: ++b hiç çalışmaz
    int r2 = (a == 0) || (++c > 0);   // sol taraf doğru: ++c hiç çalışmaz

    printf("r1 = %d, b = %d\n", r1, b);
    printf("r2 = %d, c = %d\n", r2, c);

    int pay = 10, payda = 0;
    int guvenli = (payda != 0) && (pay / payda > 2);
    printf("guvenli = %d\n", guvenli);
    return 0;
}
```

```
ÇIKTI
r1 = 0, b = 0
r2 = 1, c = 0
guvenli = 0
```

`b` ve `c` hiç artmadı, çünkü sağ taraflar çalıştırılmadı. Son örnek, kısa devrenin güvenlik amacıyla kullanımıdır: `payda` sıfırsa `pay / payda` hiç değerlendirilmediği için sıfıra bölme hatası oluşmaz. Bu kalıbı koşul ifadelerinde (Bölüm 7) çok kullanacağız.

> **▲ Sık Yapılan Hata · = ile == karıştırmak ve zincirleme karşılaştırma**
>
> İki yaygın hata ilişkisel operatörlerle ilgilidir.
>
> **Birincisi**, `==` yerine `=` yazmaktır. `if (x = 5)` ifadesi `x`'e 5 atar ve atamanın değeri 5 (sıfırdan farklı, yani doğru) olduğu için koşul her zaman doğru çıkar. gcc `-Wall` ile parantez eksikliği uyarısı verir; Bölüm 7'de koşullarla birlikte yeniden göreceğiz.
>
> **İkincisi**, matematikteki `1 < x < 10` yazımını kullanmaktır. C'de bu `(1 < x) < 10` demektir: önce `1 < x` hesaplanır, 0 ya da 1 çıkar; bu sonuç 10 ile karşılaştırılır ve her zaman doğru olur. Doğru yazım `1 < x && x < 10`'dur.

_Bool ve stdbool.h

C'nin ilk standardında doğru/yanlış için ayrı bir tip yoktu; programcılar `int` kullanırdı. C99, `_Bool` tipini getirdi: yalnızca 0 ve 1 tutan bir tamsayı tipi. Diğer her değer ona atanırken 0'a ya da 1'e dönüştürülür: sıfır 0 olur, sıfırdan farklı her değer 1 olur. Alt çizgili ve büyük harfle başlayan bu garip ad, eski programlarda `bool` adını kullananlarla çakışmamak için seçildi. Bu yüzden C99, `<stdbool.h>` başlık dosyasını da ekledi: bu dosya `bool`, `true` (1) ve `false` (0) adlarını makro olarak tanımlar.

▸ boolOrnek.c

```
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    bool yagmurVar = true;
    bool sicak = false;
    bool kopya = 42;          // sıfırdan farklı: 1 olur

    printf("yagmurVar = %d\n", yagmurVar);
    printf("sicak = %d\n", sicak);
    printf("kopya = %d\n", kopya);
    printf("sizeof(bool) = %zu\n", sizeof(bool));

    bool disaridaGezilir = !yagmurVar && !sicak;
```

```
    printf("disaridaGezilir = %d\n", disaridaGezilir);
    return 0;
}
```

```
ÇIKTI
yagmurVar = 1
sicak = 0
kopya = 1
sizeof(bool) = 1
disaridaGezilir = 0
```

`bool` değerlerini `printf` ile yazdırırken `%d` kullanırız; `bool` için ayrı bir belirteç yoktur ve değer önce `int`'e yükseltilir. `sizeof(bool)` genellikle 1'dir.

> *Tarihçe · bool'un yolu*
>
> C++ ilk standardından (C++98) beri yerleşik bir `bool` tipine sahiptir. C ise bu kavramı 1999'da `_Bool` ile getirdi ve `bool` yazabilmek için `<stdbool.h>` eklemek gerekti. C23 standardı bu farkı kapattı: `bool`, `true` ve `false` artık gerçek anahtar sözcüklerdir ve başlık dosyası eklemek gerekmez. Bu kitapta C17 kullandığımız için `<stdbool.h>` yazmaya devam edeceğiz.

Üç Dilde · Doğru/yanlış ve tip boyutları üç dilde *C · C++ · C#*

Yukarıdaki iki konu, üç dil arasındaki en görünür farklardandır:

Konu	C	C++	C#
Doğru/yanlış tipi	`_Bool` (`bool` için `<stdbool.h>`)	`bool` (yerleşik)	`bool` (yerleşik)
`5 > 3` ifadesinin tipi	`int` (1)	`bool` (`true`)	`bool` (`true`)
`if (sayi)` yazmak (sayi `int`)	Geçerli: sıfırdan farklı doğru	Geçerli	**Derleme hatası**: `bool` gerekir
`int` boyutu	En az 16 bit, genelde 32	En az 16 bit, genelde 32	Her zaman 32 bit
`long` boyutu	32 ya da 64 bit	32 ya da 64 bit	Her zaman 64 bit
`char` boyutu	1 bayt	1 bayt	2 bayt (UTF-16)
Ondalıklı sabit	`double`	`double`	`double`

C#'ın tip boyutlarını sabit tutması, aynı kodun her platformda aynı davranmasını sağlar. C#'ta ayrıca işaretsiz ve işaretli bayt için `byte` ve `sbyte`, 128 bitlik, para hesaplarına uygun bir ondalıklı tip için `decimal` vardır. C#'ın `if (sayi)` yazımını reddetmesi ise `=` ve `==` karışıklığından doğan hataları ortadan kaldırır.

5.11 Tip Dönüşümleri

Bir ifadede farklı tipten değerler karışırsa ne olur? `5 + 2.5` işleminde bir `int` ile bir `double` toplanıyor. Bilgisayar tek bir tipteki iki değeri toplayabilir; bu yüzden değerlerden biri önce dönüştürülmelidir. Dönüşüm iki şekilde olur: derleyici kendiliğinden yapar (**örtük**) ya da biz isteriz (**açık**).

Örtük dönüşüm

İki farklı tipte işlenen bir operatörde buluştuğunda C, küçük tipi büyük tipe yükseltir. Sıra kabaca şöyledir:

```
char, short  ->  int  ->  long  ->  long long  ->  float  ->  double  ->  long double
```

Yani `int + double` işleminin sonucu `double` olur. **Atamada** ise sağdaki değer, soldaki değişkenin tipine çevrilir. `int`'ten `double`'a atama sorunsuzdur. `double`'dan `int`'e atarken ise ondalık kısım atılır; sayı yuvarlanmaz, **sıfıra doğru kesilir**.

▸ ortukDonusum.c

```c
#include <stdio.h>

int main(void) {
    int tamsayi = 5;
    double toplam = tamsayi + 2.5;   // int, double'a yükselir
    int a = 3.99;                    // ondalık kısım atılır
    int b = -3.99;                   // sıfıra doğru: -3
    double c = 7 / 2;                // önce tamsayı bölmesi (3), sonra 3.0
    double d = 7 / 2.0;              // ondalıklı bölme

    printf("toplam = %.1f\n", toplam);
    printf("a = %d, b = %d\n", a, b);
    printf("c = %.1f, d = %.1f\n", c, d);
    printf("5 / 2 + 0.5 = %.1f\n", 5 / 2 + 0.5);
    return 0;
}
```

ÇIKTI

```
toplam = 7.5
a = 3, b = -3
c = 3.0, d = 3.5
5 / 2 + 0.5 = 2.5
```

Dönüşümler **işlem işlem** yapılır; derleyici ifadenin tamamına bakıp "sonuç double olacak, o hâlde bölmeyi ondalıklı yapayım" demez. `double c = 7 / 2;` bu yüzden klasik bir tuzaktır: değişkenin `double` olması, bölmenin tamsayı olarak yapılmasını engellemez; çünkü bölme atamadan önce yapılır. Benzer şekilde `5 / 2 + 0.5` ifadesinde önce `5 / 2` iki tamsayı olduğu için 2 çıkar; `double` yalnızca sonraki toplamada devreye girer ve sonuç 3,0 değil 2,5 olur. gcc bu satırlar için uyarı vermez; farkı görmek bize kalır.

> ◆ **Kritik Nokta · Ortalamayı tamsayı bölmesiyle hesaplamak**
>
> `toplam` ve `adet` birer `int` iken `double ortalama = toplam / adet;` yazarsak `ortalama` değişkeni `double` olsa bile sonuç kesilmiş çıkar: `toplam` 145, `adet` 2 ise sonuç 72,5 değil 72,0 olur. Çözüm, işlenenlerden birini bölmeden önce `double`'a çevirmektir; yolu hemen aşağıda.

Açık dönüşüm (cast)

Dönüşümü kendimiz yapmak istersek ifadenin önüne parantez içinde tipi yazarız: `(tip) ifade`. Buna **tip dönüştürme** (type casting) ya da kısaca **cast** denir.

castFark.c

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 2;
    printf("a / b           : %d\n", a / b);
    printf("(double)a / b   : %.2f\n", (double)a / b);
    printf("(double)(a / b) : %.2f\n", (double)(a / b));
    printf("(int)(-3.99)    : %d\n", (int)(-3.99));
    return 0;
}
```

ÇIKTI

```
a / b           : 3
(double)a / b   : 3.50
(double)(a / b) : 3.00
(int)(-3.99)    : -3
```

İlk iki ifade arasındaki fark çok önemlidir. `(double)a / b` ifadesinde cast, `/` operatöründen önce uygulanır (tekli operatörler önceliklidir): `a` önce 7,0 olur, sonra `7.0 / 2` ondalıklı bölme olarak 3,5 verir. `(double)(a / b)` ifadesinde ise parantez içi önce hesaplanır: `7 / 2` tamsayı bölmesidir ve 3 verir; bu 3 sonradan 3,0'a çevrilir. Kesilen 0,5 geri gelmez.

Cast, değişkenin kendisini değiştirmez; yalnızca o ifadede kullanılan değerin başka tipte bir kopyasını üretir. `a` hâlâ bir `int`'tir.

> ◆ **Kritik Nokta · Cast'in yeri sonucu belirler**
>
> `(double)toplam / adet` doğru ortalamadır, `(double)(toplam / adet)` yanlıştır. Cast, kesilmeyi önlemek istediğimiz bölmeden **önce**, bölünene (ya da böleni) uygulanmalıdır.
>
> Pozitif bir sayıyı en yakın tamsayıya yuvarlamak için `(int)(x + 0.5)` kalıbı kullanılır; negatif sayılarda çalışmaz. Genel çözüm `<math.h>` içindeki `round` fonksiyonudur (Bölüm 9).

Tamsayı yükseltmesi

`char` ve `short` gibi `int`'ten küçük tipler aritmetikte kendi boyutlarıyla işlem görmez: işlemden önce otomatik olarak `int`'e yükseltilir. Buna **tamsayı yükseltmesi** (integer promotion) denir. İki `char`'ın toplamı `char` değil `int`'tir; bu yüzden `char`'a sığmayan bir sonuç bile doğru hesaplanır:

yukseltme.c

```c
#include <stdio.h>

int main(void) {
    char c = 100;
    short s = 30000;

    printf("c + c = %d\n", c + c);
    printf("s * 2 = %d\n", s * 2);
    printf("sizeof(c) = %zu, sizeof(c + c) = %zu\n", sizeof(c), sizeof(c + c));
    return 0;
}
```

```
ÇIKTI
c + c = 200
s * 2 = 60000
sizeof(c) = 1, sizeof(c + c) = 4
```

`c` en çok 127 tutabilen bir `char`'tır, ama `c + c` ifadesi `int` olarak hesaplandığı için 200 çıktı. `sizeof` farkı açıkça gösterir: `c` 1 bayt, `c + c` 4 bayttır. Sonucu tekrar bir `char` değişkene atarsak sığmayan değer bozulur; bu yüzden ara hesapları `int` değişkende tutmak daha güvenlidir. `printf`'e verilen `char` de `int`'e yükseltilir; hem `%d` hem `%c` ile çalışmasının sebebi budur.

Olağan aritmetik dönüşümler

İki farklı tipteki işlenen bir operatörde karşılaştığında C'nin ortak tipi belirleme kuralları kabaca şöyledir:

1. Önce her iki işlenene tamsayı yükseltmesi uygulanır (`char` ve `short` `int` olur).
2. İşlenenlerden biri `long double` ise ötekini `long double`'a, değilse `double` ise `double`'a, değilse `float` ise `float`'a çevirir.
3. Her ikisi de tamsayıysa ve tipleri farklıysa: işaretsiz olan işaretliden küçük değilse (aynı boyutta ya da daha büyükse) **işaretsiz olan kazanır**; işaretli olan daha büyükse ve işaretsizin tüm değerlerini tutabiliyorsa işaretli olan kazanır.

Üçüncü kuralın sonucu, işaretli ile işaretsiz tamsayının aynı işlemde karşılaşmasıdır. Buradaki sürpriz yaygın bir hata kaynağıdır.

İşaretli ile işaretsiz karşılaştırma tuzağı

`int` ile `unsigned int` aynı işlemde buluşursa C işaretliyi işaretsize çevirir, tersini değil. `-1`, işaretsize çevrilince en büyük değere, 4294967295'e dönüşür. Karşılaştırma bu yüzden şaşırtıcı sonuç verir:

▸ isaretKarsilastirma.c

```c
#include <stdio.h>

int main(void) {
    int a = -1;
    unsigned int b = 1;

    printf("a'nın işaretsiz hali: %u\n", (unsigned int)a);
    printf("a < b      : %d\n", a < b);
    printf("a < (int)b : %d\n", a < (int)b);
    return 0;
}
```

```
ÇIKTI
a'nın işaretsiz hali: 4294967295
a < b      : 0
a < (int)b : 1
```

`a < b` ifadesinde önce `a` işaretsize çevrildi: `4294967295 < 1` yanlıştır, sonuç 0'dır. Matematiksel olarak -1, 1'den küçüktür; ama C bu iki tipi öyle karşılaştırmaz. gcc bu tuzağı yalnızca `-Wextra` ile (`-Wsign-compare`) uyarır; `-Wall` tek başına susar. Bu yüzden derlerken ikisini birlikte kullanmayı alışkanlık edinelim:

```
TERMİNAL
```

```
$ gcc -std=c17 -Wall -Wextra isaretKarsilastirma.c -o isaret
isaretKarsilastirma.c: In function 'main':
isaretKarsilastirma.c:8:35: warning: comparison of integer expressions of different signedness:
'int' and 'unsigned int' [-Wsign-compare]
    8 |     printf("a < b      : %d\n", a < b);
      |                                   ^
```

Çözüm, iki tarafı aynı işaretlilikte tutmaktır: değişkenleri baştan aynı tipte tanımlamak ya da değerin `int`'e sığdığından eminsek `(int)b` gibi bir cast kullanmaktır. `<`, `>`, `<=`, `>=` ve `==` aynı kuralı izler.

> **▲ Sık Yapılan Hata · unsigned ile geriye sayan döngü**
>
> `unsigned int i` için `i >= 0` ifadesi **her zaman doğrudur**, çünkü işaretsiz sayı sıfırın altına inemez: `0 - 1` işlemi -1 değil 4294967295 üretir. Bu yüzden `unsigned` bir sayaçla yazılan geriye sayan döngüler (`i >= 0` koşuluyla) hiç bitmez. Döngüleri Bölüm 8'de göreceğiz; kuralı şimdiden not edelim: geriye sayan sayaçlar işaretli tipte olmalıdır. Aynı tuzağa `strlen` fonksiyonunun işaretsiz `size_t` döndürmesi yüzünden Bölüm 11'de yeniden rastlayacağız.

> **Üç Dilde · Tip dönüşümü üç dilde** *C · C++ · C#*
>
> Üç dilde de `7 / 2` tamsayı bölmesidir ve 3 verir; `(double)7 / 2` gibi cast ile 3,5 elde edilir. Asıl fark, **daraltan** dönüşümlerdedir:
>
> ```c
> int x = 3.99;          // C: geçerli, x = 3 (gcc -Wall uyarmaz)
> ```
>
> ```cpp
> int x = 3.99;          // C++: geçerli, x = 3
> int y{3.99};           // C++11: süslü parantezli ilk değer vermede
>                        //   daraltıcı dönüşüm DERLEME HATASIDIR
> ```
>
> ```csharp
> int x = 3.99;          // C#: DERLEME HATASI (CS0266): double'dan int'e
>                        //   örtük dönüşüm yok
> int y = (int)3.99;     // C#: açık cast gerekir, y = 3
> ```
>
> C# veri kaybı olabilecek her dönüşüm için açık cast ister; C ise bunu sessizce yapar. C++11 ile gelen süslü parantezli ilk değer verme (`{}`), C++'ta bu korumayı isteğe bağlı olarak getirir.

5.12 Taşma ve Kayan Nokta Hassasiyeti

Bilgisayarda sayıların sonsuz hassasiyeti ve sınırsız büyüklüğü yoktur. Tamsayılar sonlu bir aralığa sığmak zorundadır; ondalıklı sayılar ise ikilik tabanda saklandıkları için bazı değerleri yalnızca yaklaşık olarak ifade eder. Bu iki sınırlamanın sonuçlarını bilmek, "program neden yanlış sonuç verdi?" sorusunun cevabı çoğu zaman buradadır.

Tamsayı taşması

Her tamsayı tipinin bir aralığı vardır. Bir hesabın sonucu bu aralığın dışına çıkarsa **taşma** (overflow) olur. C bu durumda iki türlü davranır.

İşaretli taşma tanımsız davranıştır. En büyük `int` değerine 1 eklemek bunun örneğidir:

```
int enBuyuk = INT_MAX;       // 2147483647
enBuyuk = enBuyuk + 1;       // tanımsız davranış
```

Birçok bilgisayarda sonuç -2147483648 çıkar ("başa sarma"), ama derleyici bunu garanti etmez; optimizasyon açıkken bambaşka bir şey de yapabilir. Çıktısını bu yüzden göstermiyoruz. Taşmanın en sık görüldüğü yer büyük sayıların çarpımıdır ve çoğu zaman fark edilmez:

```
int a = 100000;
int b = a * a;                 // 10 milyar, int'e sığmaz: tanımsız
long long c = a * a;           // bu da hatalı: çarpma yine int ile yapılır
long long d = (long long)a * a;   // doğru: çarpmadan ÖNCE büyük tipe çevir
```

Üçüncü satır özellikle sinsidir. Atama soldaki değişkenin tipine bakmaz; `a * a` çarpması önce `int` ile yapılır ve taşar, sonuç ancak ondan sonra `long long`'a çevrilir. Çözüm, çarpmanın işlenenlerinden birini çarpmadan önce büyük tipe dönüştürmektir:

▸ carpimTasma.c

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    int a = 100000;
    long long d = (long long)a * a;
    printf("100000 * 100000 = %lld\n", d);

    int x = INT_MAX - 5;
    int y = 10;
    printf("x + y taşar mı? %d\n", y > 0 && x > INT_MAX - y);
    printf("x + 3 taşar mı? %d\n", 3 > 0 && x > INT_MAX - 3);
    return 0;
}
```

ÇIKTI

```
100000 * 100000 = 10000000000
x + y taşar mı? 1
x + 3 taşar mı? 0
```

Programın ikinci yarısı, toplamı yapmadan taşıp taşmayacağını önceden sınayan bir kalıp gösterir: `x + y` ifadesi ancak `x > INT_MAX - y` ise taşar. Çıkarma taşmaya yol açmadığı için bu denetim güvenlidir.

İşaretsiz taşma tanımlıdır. Kuralı bellidir: sonuç, tipin alabileceği değer sayısına göre modu alınır; sayaç başa sarar.

▸ unsignedSarma.c

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    unsigned int u = UINT_MAX;
    u = u + 1;
    printf("%u\n", u);
    u = u - 1;
    printf("%u\n", u);

    unsigned char kucuk = 250;
    kucuk = kucuk + 10;
```

```
    printf("%d\n", kucuk);

    unsigned int negatif = -1;
    printf("%u\n", negatif);
    return 0;
}
```

```
ÇIKTI
0
4294967295
4
4294967295
```

`UINT_MAX + 1` sıfıra sarar; sıfırdan 1 çıkarınca en büyük değere dönülür. `unsigned char` 0 … 255 tutar; 250 + 10 = 260 olunca 260 mod 256 = 4 kalır. `unsigned` bir değişkene negatif değer atamak da aynı mantıkla büyük bir pozitif sayı üretir (-1 ⟶ 4294967295). İşaretsiz bir değişken sıfırın altına inemez; `0 - 1` işlemi -1 değil, 4294967295 verir.

Ondalıklı sayıların hassasiyeti

Bilgisayar sayıları ikilik tabanda saklar. 0,5 ve 0,25 ikilikte sonlu sayıda basamakla yazılır, ama 0,1 yazılamaz (tıpkı 1/3'ün onluk tabanda 0,333… diye sürüp gitmesi gibi). Sayı, kendisine en yakın temsil edilebilir değere yuvarlanır. Hatayı `%.20f` ile (virgülden sonra 20 basamak) görebiliriz. Aynı programda `float` ile `double`'ın kaç basamağı doğru tuttuğuna da bakalım:

▸ hassasiyet.c

```c
#include <stdio.h>

int main(void) {
    double toplam = 0.1 + 0.2;
    float kucuk = 0.1f;
    float f = 123456789.0f;
    double d = 123456789.0;

    printf("%.1f\n", toplam);
    printf("%.20f\n", toplam);
    printf("%.20f\n", kucuk);
    printf("%d\n", toplam == 0.3);
    printf("float : %.1f\n", f);
    printf("double: %.1f\n", d);
    return 0;
}
```

```
ÇIKTI
0.3
0.30000000000000004441
0.10000000149011611938
0
float : 123456792.0
double: 123456789.0
```

`%.1f` ile yuvarlanınca 0.3 görünür, ama gerçek değer 0,30000000000000004…'tür. `float` olan 0,1'in hatası ise çok daha büyüktür. Dördüncü satırda `==` operatörünü kullandık: `0.1 + 0.2` ile `0.3` **eşit çıkmadı**, ifade 0 verdi. Son iki satır hassasiyetin sınırını gösterir: `float` yaklaşık 7, `double` yaklaşık 15-16 anlamlı basamak tutar. Dokuz basamaklı sayıyı `float` yaklaşık olarak saklayıp 123456792 yazdı; `double` ise tam tuttu.

Bu yüzden **ondalıklı sayılar `==` ile karşılaştırılmaz.** Bunun yerine aralarındaki farkın çok küçük olup olmadığına bakılır. `<math.h>` içindeki `fabs` bir `double`'ın mutlak değerini verir:

▸ epsilonKarsilastir.c

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double a = 0.1 + 0.2;
    double b = 0.3;
    double es = 1e-9;                  // kabul edilen en büyük fark

    printf("a == b            : %d\n", a == b);
    printf("fabs(a - b) < es  : %d\n", fabs(a - b) < es);

    double x = 0.1;
    double on = x + x + x + x + x + x + x + x + x + x;
    printf("10 kez 0.1 toplamı: %.17f\n", on);
    printf("on == 1.0         : %d\n", on == 1.0);
    return 0;
}
```

ÇIKTI

```
a == b            : 0
fabs(a - b) < es  : 1
10 kez 0.1 toplamı: 0.99999999999999989
on == 1.0         : 0
```

Eşik değeri (`es`) bir kural değil, bir tercihtir: karşılaştırılan değerlerin büyüklüğüne ve hesabın hassasiyetine göre seçilir. Çok büyük sayılarda göreceli bir fark ölçüsü gerekebilir. Burada anlaşılması gereken ilke, kayan noktalı eşitliğin "yaklaşık eşitlik" olarak ele alınması gerektiğidir.

Aynı hassasiyet sınırı `float` ile büyük tamsayılar arasında da kendini gösterir. `float` yalnızca 24 bitlik bir anlamlı kısım tutar; bu yüzden 16.777.216'dan (2^{24}) sonra her tamsayıyı gösteremez:

▸ floatSinir.c

```c
#include <stdio.h>

int main(void) {
    int n = 16777217;                  // 2^24 + 1
    float f = n;
    printf("int   : %d\n", n);
    printf("float : %.1f\n", f);
    printf("%d\n", 16777216.0f + 1.0f == 16777216.0f);
    return 0;
}
```

ÇIKTI

```
int   : 16777217
float : 16777216.0
1
```

`int`'ten `float`'a dönüşümde 16.777.217 sayısı 16.777.216'ya yuvarlandı; hatta `16777216.0f + 1.0f` ifadesi, 16.777.216'ya hiçbir şey eklememiş gibi davrandı.

◆ Kritik Nokta · Para hesabında kayan nokta kullanmayın

Para ya da sayma gibi tam olması gereken hesaplarda `double` yerine tamsayı kullanmak daha güvenlidir. Örneğin tutarları kuruş cinsinden `int` ya da `long long` olarak tutarız: 10,20 TL değil 1020 kuruş. Toplama ve çıkarma kesin olur; ekrana yazarken `tutar / 100` ve `tutar % 100` ile lira ve kuruşu ayırırız. `double`, ölçüm ve bilimsel hesaplar için uygundur; küçük hatalar kabul edilebilir ve birikimleri bilinçli yönetilir.

Üç Dilde · Taşma ve hassasiyet üç dilde *C · C++ · C#*

- **İşaretli taşma:** C ve C++'ta tanımsız davranıştır. C#'ta varsayılan olarak sessizce başa sarar (tanımlı bir davranıştır); `checked` anahtar sözcüğüyle taşma anında `OverflowException` fırlatılır.
- **İşaretsiz taşma:** C ve C++'ta tanımlı ve başa sarar. C# `unchecked` bağlamda aynı şekilde sarar.
- **Kayan nokta:** Üç dil de IEEE 754 kullanır; `0.1 + 0.2 == 0.3` üçünde de yanlıştır. C#'ta ayrıca `decimal` tipi vardır: onluk tabanda 28-29 basamak hassasiyetiyle çalışır ve para hesapları için tasarlanmıştır.

5.13 Operatör Önceliği ve Birleşme Yönü

Bir ifadede birden çok operatör varsa hangisinin önce çalışacağını **öncelik** (precedence) belirler. Matematikteki alışkanlığımız burada da geçerlidir: çarpma, bölme ve kalan, toplama ve çıkarmadan önce yapılır. Aynı öncelikteki operatörlerin hangi yönden işleneceğini ise **birleşme yönü** (associativity) belirler: `20 - 6 - 4` ifadesi soldan sağa `(20 - 6) - 4 = 10` olarak hesaplanır; atama gibi bazı operatörler sağdan sola işlenir.

Aşağıdaki tablo, bu bölümde gördüğümüz operatörleri ve ileride tanışacaklarımızı en yüksek öncelikten en düşüğe doğru gösterir:

Seviye	Operatörler	Birleşme
1	`()` fonksiyon çağrısı, `[]`, `.`, `->`, son ek `++` `--`	soldan sağa
2	Ön ek `++` `--`, tekli `+` `-`, `!`, `~`, `(tip)` cast, `*` (içerik), `&` (adres), `sizeof`	sağdan sola
3	`*` `/` `%`	soldan sağa
4	`+` `-`	soldan sağa
5	`<<` `>>`	soldan sağa
6	`<` `<=` `>` `>=`	soldan sağa
7	`==` `!=`	soldan sağa
8	`&` (bit düzeyinde VE)	soldan sağa
9	`^`	soldan sağa
10	`|`	soldan sağa
11	`&&`	soldan sağa
12	`||`	soldan sağa
13	`?:` (koşul operatörü)	sağdan sola
14	`=` `+=` `-=` `*=` `/=` `%=` ve diğer atamalar	sağdan sola
15	`,` (virgül operatörü)	soldan sağa

Tablonun tamamını ezberlemek gerekmez (C, C++ ve C# tabloları Ek B'de bulunur). Çok sık işimize yarayacak birkaç sonuç şunlardır:

- Aritmetik operatörler karşılaştırmadan, karşılaştırma mantıksal operatörlerden, mantıksal operatörler atamadan **önce** işlenir. `a + 1 > b && c` beklediğimiz gibi `((a + 1) > b) && c` olarak okunur.
- `&&`, `||`'dan önceliklidir: `a || b && c`, `a || (b && c)` demektir.
- Bit operatörleri (`&`, `^`, `|`) karşılaştırma operatörlerinden **düşük** önceliklidir. `x & 1 == 0` ifadesi bu yüzden `x & (1 == 0)` olarak okunur ve amaçlanan sonucu vermez; doğrusu `(x & 1) == 0`'dır. Bit operatörlerini Bölüm 16'da inceleyeceğiz.
- Tekli operatörler ikili aritmetik operatörlerden önce çalışır: `-a * b`, `(-a) * b` demektir; cast da böyledir: `(double)a / b` ifadesinde cast önce uygulanır.

▸ oncelik.c

```c
#include <stdio.h>

int main(void) {
    printf("2 + 3 * 4       = %d\n", 2 + 3 * 4);
    printf("(2 + 3) * 4     = %d\n", (2 + 3) * 4);
    printf("20 - 6 - 4      = %d\n", 20 - 6 - 4);
    printf("100 / 10 * 5    = %d\n", 100 / 10 * 5);
    printf("7 + 8 %% 3 * 2  = %d\n", 7 + 8 % 3 * 2);
    printf("1 + 2 > 2 && 3 < 4 = %d\n", 1 + 2 > 2 && 3 < 4);
    return 0;
}
```

```
ÇIKTI
2 + 3 * 4       = 14
(2 + 3) * 4     = 20
20 - 6 - 4      = 10
100 / 10 * 5    = 50
7 + 8 % 3 * 2   = 11
1 + 2 > 2 && 3 < 4 = 1
```

`100 / 10 * 5` ifadesi `100 / 50 = 2` değildir: `/` ile `*` aynı seviyededir, soldan sağa gidilir ve ifade `(100 / 10) * 5 = 50` olur. `7 + 8 % 3 * 2` ifadesinde önce `%` (sonuç 2), sonra `*` (sonuç 4), en son `+` çalışır ve sonuç 11 çıkar. Son satırdaki `1 + 2 > 2 && 3 < 4` ifadesi `((1 + 2) > 2) && (3 < 4)` olarak okunur ve 1 verir.

Atama sağdan sola işlendiği için `x = y = z = 0` zincirinde önce `z`, sonra `y`, sonra `x` sıfırlanır.

Emin değilseniz parantez kullanın

Parantez önceliği değiştirir ve kodu okuyana niyetimizi gösterir. Parantezi unutmak, çoğu zaman hata mesajı vermeyen, makul görünen yanlış sonuçlar üreten bir mantık hatasıdır. Örneğin iki ara not üzerinden ortalama hesaplayan şu programa bakalım:

▸ ortalama.c

```c
#include <stdio.h>

int main(void) {
    int ilkNot = 65;
    int ikinciNot = 80;

    double hatali = ilkNot + ikinciNot / 2;
```

```
    double yarim = (ilkNot + ikinciNot) / 2;
    double dogru = (ilkNot + ikinciNot) / 2.0;

    printf("Hatalı : %.2f\n", hatali);
    printf("Yarım  : %.2f\n", yarim);
    printf("Doğru  : %.2f\n", dogru);
    return 0;
}
```

```
ÇIKTI
Hatalı : 105.00
Yarım  : 72.00
Doğru  : 72.50
```

`ilkNot + ikinciNot / 2` ifadesinde `/` önce yapıldığı için yalnızca `ikinciNot` ikiye bölünür (65 + 40 = 105). Parantez eklenince toplam doğru bölünür; ancak 145 / 2 iki tamsayının bölmesi olduğu için sonuç 72 çıkar. Böleni `2.0` yapınca bölme ondalıklı olur ve 72,50 elde edilir. Program hiçbir hata vermedi; ekranda her seferinde makul görünen bir sayı vardı. Mantık hatası böyle gizlenir.

İpucu · Karmaşık ifadeyi adım adım açın

Uzun bir ifadeyi çözerken kâğıda parantezlerle yeniden yazmak en güvenli yoldur. `2 + 3 * 4 % 5` için önce `3 * 4` ⟶ 12, sonra `12 % 5` ⟶ 2, en son `2 + 2` ⟶ 4. Her adımı alt alta yazın; bir adımı zihinden atlamaya çalışmak hataya yol açar. Bir ifadeyi birkaç satıra bölüp ara sonuçları adlandırılmış değişkenlerde tutmak da hem hata ayıklamayı hem de okumayı kolaylaştırır.

Not · Operand sırası ile öncelik aynı şey değildir

Öncelik, hangi operatörün hangi işlenenlere uygulanacağını belirler; işlenenlerin hangi sırayla *hesaplanacağını* değil. `f() + g()` gibi bir ifadede `f` ve `g`'den hangisinin önce çağrılacağını C belirtmez. Fonksiyonları Bölüm 9'da işlediğimizde, yan etkisi olan ifadelerin aynı ifadede birleştirilmemesi gerektiğini bu yüzden yeniden hatırlayacağız.

5.14 Örnek Programlar

Bu bölümde öğrendiklerimizi iki küçük programda birleştirelim. Klavyeden girdi okumayı Bölüm 6'da öğreneceğimiz için değerleri şimdilik kodun içine yazıyoruz.

İlk programda Celsius'tan Fahrenheit'a dönüşüm yapalım. Formül `F = C × 9/5 + 32`'dir. Tamsayı bölmesi tuzağının burada nasıl ortaya çıktığına dikkat edelim:

`sicaklik.c`

```
#include <stdio.h>

int main(void) {
    double celsius = 36.6;
    double fahrenheit = celsius * 9 / 5 + 32;
    printf("%.1f C = %.1f F\n", celsius, fahrenheit);

    int tam = 37;
    int dogru = tam * 9 / 5 + 32;        // önce çarpma: 333 / 5 = 66
    int hatali = tam * (9 / 5) + 32;     // 9 / 5 tamsayı bölmesi: 1
    printf("Doğru : %d\n", dogru);
```

```
    printf("Hatalı: %d\n", hatali);
    return 0;
}
```

```
ÇIKTI
36.6 C = 97.9 F
Doğru : 98
Hatalı: 69
```

`celsius * 9 / 5` ifadesi soldan sağa işlenir: önce `celsius * 9` (bir `double`), sonra `/ 5`; ondalıklı bölme olduğu için sorun yoktur. Tamsayı versiyonunda ise `9 / 5` ifadesini parantez içine alınca 1'e kesildi ve sıcaklık 69'a düştü; çarpmayı bölmeden önce yapmak (`tam * 9 / 5`) kesmenin etkisini azalttı. Tamsayıyla kalmak zorundaysak, kesmeden doğan kaybı en aza indirmek için bölmeyi hesabın en sonuna bırakırız.

İkinci programda bir fatura hesaplayalım. Birim fiyat, adet ve KDV oranını sabit ve değişken olarak tanımlayalım; tutarları hem `double` hem kuruş cinsinden tamsayı olarak hesaplayarak iki yaklaşımı yan yana görelim:

▸ **fatura.c**

```c
#include <stdio.h>

int main(void) {
    const double KDV_ORANI = 0.20;
    const int KDV_YUZDE = 20;

    int adet = 3;
    double birimFiyat = 249.90;

    double araToplam = adet * birimFiyat;
    double kdv = araToplam * KDV_ORANI;
    double genelToplam = araToplam + kdv;

    printf("Ara toplam  : %8.2f TL\n", araToplam);
    printf("KDV (%%%d)   : %8.2f TL\n", KDV_YUZDE, kdv);
    printf("Genel toplam: %8.2f TL\n", genelToplam);

    long long birimKurus = 24990;
    long long araKurus = adet * birimKurus;
    long long kdvKurus = araKurus * KDV_YUZDE / 100;
    long long toplamKurus = araKurus + kdvKurus;

    printf("Kuruş hesabı: %lld TL %02lld kuruş\n",
           toplamKurus / 100, toplamKurus % 100);
    return 0;
}
```

```
ÇIKTI
Ara toplam  :   749.70 TL
KDV (%20)   :   149.94 TL
Genel toplam:   899.64 TL
Kuruş hesabı: 899 TL 64 kuruş
```

`%8.2f` belirteci sayıyı toplam 8 karakterlik alana, iki ondalıkla yazdığı için tutarlar alt alta hizalanır. Kuruş hesabında `%02lld` belirteci kuruşun tek haneli olması durumunda başına sıfır koyar (`5` yerine `05`). İki yaklaşım bu veride aynı sonucu verdi; fakat kuruş hesabı kayan noktalı hatalardan bağımsızdır ve bu yüzden para işlerinde tercih edilir.

Bölüm Özeti

- **Değişken**, bellekte adı, tipi ve değeri olan alandır. `tip ad = deger;` ilk değer verir, `ad = deger;` sonradan atar. Yerel değişkenlere tanımlarken ilk değer verilmelidir; ilk değeri olmayan değişkeni okumak çöp değer verir ve tanımsız davranıştır.
- **Ad kuralları:** harf, rakam ve `_` kullanılır; rakamla başlanmaz; boşluk, özel karakter ve anahtar sözcük olmaz; büyük/küçük harf farklıdır. Kitabın geleneği: değişkenler `camelCase`, sabitler `BUYUK_HARF`.
- **Tamsayı tipleri:** `char` (1 bayt), `short` (tipik 2), `int` (tipik 4), `long` (4 ya da 8), `long long` (8). Boyutlar platforma göre değişebilir; `sizeof` bayt sayısını verir (`%zu`). Tam genişlik gerekiyorsa `<stdint.h>` tipleri (`int32_t`, `uint8_t` ...) kullanılır.
- **Sınırlar:** `INT_MAX` 2147483647, `INT_MIN` -2147483648'dir. n bitlik işaretli aralık -2^{n-1} ... $2^{n-1}-1$, işaretsiz aralık 0 ... 2^n-1'dir.
- **Kayan noktalı tipler:** `float` (~7 basamak), `double` (~15-16 basamak), `long double`. Ondalıklı sabit `double`'dır (`3.14`), `float` için `f` soneki gerekir (`3.14f`).
- **Sabitler:** `const double PI = 3.14;` tipli, bellekte yer kaplayan, `;` ile biten bir salt okunur değişkendir. `#define PI 3.14` ise önişlemcinin metin değiştirmesidir; tipi yoktur, `;` ile bitmez. C'de `'A'` bir `int`'tir, `5.0` bir `double`'dır, başında `0` olan sayı sekizliktir.
- **printf belirteçleri:** `%d` (`int`), `%u`, `%ld`, `%lld`, `%zu`, `%f`, `%.2f`, `%e`, `%g`, `%c`, `%s`, `%x`, `%o`, `%%`. Belirteç ile argüman tipinin uyuşması gerekir.
- **Tamsayı bölmesi** kesirli kısmı atar (sıfıra doğru): `7 / 2` = 3, `-7 / 2` = -3. `%` kalanı verir ve işareti bölünenin işaretidir (`-7 % 2` = -1); yalnızca tamsayılarla çalışır. Tamsayıyı sıfıra bölmek tanımsızdır.
- **Atama ve artırma:** `x += 5` ≡ `x = x + 5`; `y *= 2 + 3` ≡ `y = y * (2 + 3)`. `x++` ifadede eski değeri, `++x` yeni değeri verir. Bir değişkeni aynı ifadede iki kez değiştirmek tanımsızdır.
- **Doğru/yanlış:** karşılaştırma ve mantıksal operatörler `int` 1 ya da 0 üretir; koşulda 0 yanlış, sıfırdan farklı her değer doğrudur. `&&` ve `||` kısa devre yapar. `_Bool` ve `<stdbool.h>` (`bool`, `true`, `false`) C99'dan beri vardır; C23'te gerçek anahtar sözcüktür.
- **Tip dönüşümü:** karışık ifadede küçük tip büyüğe yükselir; `char` ve `short` aritmetikte `int` olur. `double`'dan `int`'e atama kesir kısmı atar. Cast'in yeri önemlidir: `(double)a / b` doğru, `(double)(a / b)` kesilmiş sonuçtur.
- **İşaretli/işaretsiz:** `int` ile `unsigned` karşılaşırsa işaretli olan işaretsize çevrilir; `-1 < 1u` yanlıştır. `-Wall -Wextra` ile derleyin.
- **Taşma:** işaretsiz taşma tanımlıdır (modül alır); işaretli taşma tanımsızdır. Büyük çarpımlarda çarpmadan önce büyük tipe çevirin: `(long long)a * a`.
- **Hassasiyet:** `0.1 + 0.2` tam 0,3 değildir. Ondalıklı sayılar `==` ile değil, farkın küçüklüğüyle karşılaştırılır. Para için tamsayı (kuruş) kullanın.
- **Öncelik:** `( )` > tekli operatörler (`++`, `-`, `!`, cast) > `* / %` > `+ -` > karşılaştırma > `&&` > `||` > atama. Aynı seviyede soldan sağa (atama ve tekliler hariç). Emin değilseniz parantez kullanın.

Alıştırmalar

1. Aşağıdaki kod parçasından sonra ekrana ne yazılır?

```c
int a = 17, b = 5;
double c = a / b;
printf("%.1f", c);
```

A) 3.4
B) 3.0
C) 3.5
D) 4.0
E) Program derlenmez

2. `printf("%d", -17 % 5);` satırı ekrana ne yazdırır?

A) 3
B) 2
C) -3
D) 0
E) -2

3. Aşağıdakilerden hangisi geçerli bir C değişken adıdır?

A) 3toplam
B) toplam-not
C) _toplam2
D) double
E) ogrenci sayisi

4. `int` tipinin 4 bayt olduğu bir platformda C17 ile derlenen `printf("%zu", sizeof('A'));` satırı ne yazdırır?

A) 1
B) 2
C) 4
D) 8
E) Program derlenmez

5. `int x = 5; int y = x++ + 2;` satırlarından sonra `x` ve `y` değerleri nedir?

A) x = 5, y = 7
B) x = 6, y = 7
C) x = 6, y = 8
D) x = 5, y = 8
E) Sonuç tanımsızdır

6. `printf("%d", 0.1 + 0.2 == 0.3);` satırı neden 0 yazdırır?

A) `==` operatörü yalnızca tamsayılarla çalışır.
B) 0,1 ve 0,2 ikilik tabanda tam olarak gösterilemez; toplam 0,3'e tam eşit değildir.
C) `printf` ondalıklı karşılaştırmaları yazdıramaz.
D) `double` tipi 0,1 değerini saklayamaz ve derleyici hata verir.
E) C'de ondalıklı sayılar toplanamaz.

7. Programın çıktısı nedir?

```c
int a = 10, b = 4;
double r = a / b;
printf("%.1f %.1f %d %d\n", r, (double)a / b, a % b, -a / b);
```

8. Programın çıktısı nedir?

```c
int x = 0, y = 5;
int r = (x != 0) && (++y > 0);
int s = (x == 0) || (++y > 0);
printf("%d %d %d\n", r, s, y);
```

9. Programın çıktısı nedir? (ASCII'de `'a'` = 97, `'A'` = 65)

```c
char c = 'a';
printf("%c %d %c\n", c + 2, c + 2, c - 32);
```

10. Programın çıktısı nedir?

```c
unsigned char k = 255;
k = k + 1;
unsigned int u = 3;
printf("%d %d\n", k, u - 5 > 0);
```

11. Programın çıktısı nedir?

```c
int a = 4;
int b = ++a * 2;
int c = a-- + 1;
printf("%d %d %d\n", a, b, c);
```

12. Aşağıdaki program derlenmiyor. Hatalı satırları bulun, her birinin nedenini açıklayın ve düzeltin.

```c
#include <stdio.h>

int main(void) {
    const int N = 5;
    int 2adet = 3;
    double oran = 7.5 % 2;
    N = N + 1;
    printf("%d %f\n", 2adet, oran);
    return 0;
}
```

13. Aşağıdaki program iki sayının ortalamasını hesaplamak için yazılmıştır. 7 ve 8 için 7.5 yazması beklenirken ekrana 7.0 yazılıyor. Hatanın nedenini açıklayın ve iki ayrı yoldan düzeltin.

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 8;
    double ort = (a + b) / 2;
    printf("%.1f\n", ort);
    return 0;
}
```

14. Yarıçapı 5 olan bir dairenin alanını ve çevresini hesaplayıp iki ondalık basamakla yazdıran bir program yazın. π değerini `const` ile tanımlayın (3.14159 kullanın).
15. Değeri değişkende tutulan üç basamaklı bir sayının (ör. 472) yüzler, onlar ve birler basamağını `/` ve `%` operatörleriyle ayırın; basamakların toplamını ve sayının tersini (ör. 274) yazdırın.
16. Bir tutarı (ör. 786 TL) en az sayıda banknotla ödemek için 200, 100, 50, 20, 10, 5 ve 1 TL'lik kaç parça (banknot ya da madeni para) gerektiğini hesaplayıp yazdıran bir program yazın. Bölme ve kalan operatörlerini ve bileşik atamayı kullanın.

BÖLÜM 6

Klavyeden Girdi ve Biçimli Çıktı

BU BÖLÜMDE

- `printf` biçim belirtecinin bayrak, genişlik, hassasiyet ve dönüşüm harfinden oluşan yapısını okumayı ve yazmayı
- Sayıları, metinleri ve adresleri `%e`, `%g`, `%x`, `%o`, `%c`, `%s`, `%p`, `%zu` ile istenen görünümde yazdırmayı; hizalı tablolar üretmeyi
- `scanf` ile `int`, `double`, `char` değerleri okumayı; `&` işlecinin neden gerektiğini açıklamayı
- `scanf`'in dönüş değerini denetlemeyi ve arabellekte kalan `\n`, `%c`, `%s` ve taşma tuzaklarından kaçınmayı
- `getchar`, `putchar`, `puts` ve `fgets` ile karakter ve satır düzeyinde giriş çıkış yapmayı
- Hatalı girdiyi yakalayıp kullanıcıdan yeniden isteyen bir girdi doğrulama kalıbı kurmayı
- Aldığı veriyle hesap yapıp sonucu düzgün biçimlendiren küçük programlar yazmayı

Şimdiye kadarki programlarımız tek yönlüydü: değerler kodun içine yazılıyor, program sonucu ekrana basıp bitiyordu. Gerçek programlar ise kullanıcıya soru sorar, cevabı alır ve ona göre hesap yapar. Bu bölümde çıktı tarafını (`printf`) ayrıntılı olarak, girdi tarafını (`scanf` ve yardımcı fonksiyonlar) ise sıfırdan öğreneceğiz.

Bölüm 5'te `printf`'i yalnızca gerektiği kadar tanıdık. Şimdi belirteçlerin tam yapısını, hizalamayı ve yuvarlamayı işleyeceğiz. Ardından klavyeden okumaya geçip girdinin neden bu kadar kolay yanlış gidebildiğini, yani arabellek (buffer) mantığını göreceğiz. Bölüm sonundaki üç örnek program, hepsini birleştirir. Henüz `if` ve döngüleri (Bölüm 7 ve 8) işlemedik; girdi doğrulama kısmında bunların yalnızca küçük birer kalıbını kullanacak ve ayrıntısını sonraya bırakacağız.

6.1 printf'in Yapısı

`printf` (print formatted), ilk argümanı olan **biçim metnini** (format string) soldan sağa tarar. Düz karakterleri olduğu gibi ekrana yazar; `%` ile başlayan her **dönüşüm belirtecini** (conversion specification) ise sonraki argümanlardan sırayla birinin değeriyle değiştirir. Belirteç sayısı ile argüman sayısı ve tipleri uyuşmak zorundadır.

Tanım: Dönüşüm belirteci

`printf` biçim metninde `%` ile başlayıp bir dönüşüm harfiyle biten ve karşılığındaki argümanın nasıl yazılacağını tarif eden parçadır. Genel biçimi şöyledir:

`%[bayraklar][genişlik][.hassasiyet][uzunluk]harf`

Köşeli parantezli parçaların hepsi isteğe bağlıdır; yalnızca `%` ve dönüşüm harfi zorunludur.

Parçaların görevi şöyledir:

Parça	Örnek	Görevi
Bayrak	`-`, `0`, `+`, boşluk, `#`	Hizalama, doldurma ve işaret davranışını değiştirir
Genişlik	`8`	Çıktının kaplayacağı **en az** karakter sayısı
Hassasiyet	`.3`	Ondalık basamak sayısı (`%f`), en çok karakter sayısı (`%s`)
Uzunluk	`h`, `l`, `ll`, `z`, `L`	Argümanın tipinin boyutunu söyler (`short`, `long`, `long long`, `size_t`, `long double`)
Dönüşüm harfi	`d`, `f`, `s`	Değerin hangi türde yorumlanıp yazılacağını belirler

Bölüm 5'te bu harflerin çoğunu bir tabloda görmüştük. Şimdi hepsini tek programda, köşeli parantezler içinde çalıştıralım; parantezler boşlukların görünür olmasını sağlar:

▸ belirtecler.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 42;
    unsigned int buyuk = 3000000000u;
    long long cokBuyuk = 9000000000LL;
    double pi = 3.14159;
    char harf = 'A';

    printf("[%d] [%i]\n", sayi, sayi);
    printf("[%u] [%lld]\n", buyuk, cokBuyuk);
    printf("[%f] [%e] [%g]\n", pi, pi, pi);
    printf("[%g] [%g]\n", 1234567.0, 0.00001);
    printf("[%c] [%d]\n", harf, harf);
    printf("[%s]\n", "metin");
    printf("[%x] [%X] [%o] [%#x] [%#o]\n", 255, 255, 8, 255, 8);
    printf("[%zu]\n", sizeof(int));
    printf("Başarı: %d%%\n", 85);
    return 0;
}
```

```
ÇIKTI
[42] [42]
[3000000000] [9000000000]
[3.141590] [3.141590e+00] [3.14159]
[1.23457e+06] [1e-05]
[A] [65]
[metin]
[ff] [FF] [10] [0xff] [010]
[4]
Başarı: 85%
```

Çıktıdan şunları çıkarıyoruz:

- `%f` her zaman noktadan sonra altı basamak yazar. `%e` bilimsel gösterimdir: `3.141590e+00`, 3,14159 × 10^0 demektir.
- `%g`, `%f` ile `%e`'den kısa olanı seçer ve gereksiz sıfırları atar. Varsayılan olarak toplam altı anlamlı basamak yazar; bu yüzden `1234567.0` sayısı `1.23457e+06` olur. Çok büyük ve çok küçük değerlerde `%g` işe yarar.

- `%d` ile `%i` `printf`'te aynıdır. `scanf`'te farklıdırlar (bunu aşağıda göreceğiz); kitapta hep `%d` kullanacağız.
- `%x` küçük, `%X` büyük harfli onaltılık yazar; `%o` sekizliktir. `#` bayrağı önek ekler: `0xff` ve `010`.
- `%c` ile `%d` aynı `char` değerini karakter ve sayı olarak gösterir (Bölüm 5).

Genişlik, hizalama ve doldurma

Genişlik, çıktının kaplayacağı **en az** karakter sayısını belirler. Değer daha kısaysa soldan boşluk eklenir, yani varsayılan hizalama sağadır. Değer genişlikten uzunsa asla kesilmez. Bayraklar bu davranışı değiştirir:

Bayrak	Etkisi	Örnek	Sonuç (değer 42)
(yok)	Sağa yaslı, boşlukla doldur	`%6d`	`[ 42]`
`-`	Sola yaslı	`%-6d`	`[42 ]`
`0`	Boşluk yerine sıfırla doldur	`%06d`	`[000042]`
`+`	Pozitif sayıda da işaret yaz	`%+d`	`[+42]`
boşluk	Pozitif sayının önüne boşluk koy	`% d`	`[ 42]`

▸ genislik.c

```c
#include <stdio.h>

int main(void) {
    printf("[%6d]\n", 42);
    printf("[%-6d]\n", 42);
    printf("[%06d]\n", 42);
    printf("[%+d] [%+d]\n", 42, -42);
    printf("[% d] [% d]\n", 42, -42);
    printf("[%2d]\n", 123456);
    printf("[%8.3f]\n", 3.14159);
    printf("[%-8.1f]\n", 3.14159);
    printf("[%08.2f]\n", 3.14159);
    printf("[%08.2f]\n", -3.14159);
    printf("[%10s]\n", "Elif");
    printf("[%-10s]\n", "Elif");
    return 0;
}
```

ÇIKTI

```
[    42]
[42    ]
[000042]
[+42] [-42]
[ 42] [-42]
[123456]
[   3.142]
[3.1     ]
[00003.14]
[-0003.14]
[      Elif]
[Elif      ]
```

Satır satır inceleyelim:

1. `%2d` ile `123456` yazdırdık: genişlik "en az" demektir, sayı sığmasa da kesilmez.

2. `%8.3f` değerinde `3.142` noktayla birlikte beş karakterdir; genişlik 8 olduğu için önüne üç boşluk gelir. **Nokta da genişliğe dahildir.**
3. `%08.2f` boşlukları sıfırla doldurur; negatif sayıda işaret en başta kalır (`-0003.14`).
4. `%10s` ve `%-10s` metni sağa ya da sola yaslar.

> **▲ Sık Yapılan Hata · 0 bayrağı ile - bayrağı birlikte olmaz**
>
> `%-05d` yazarsak `-` bayrağı `0`'ı geçersiz kılar ve sağ taraf sıfırla değil boşlukla dolar. Sola yaslı bir sayının sağına sıfır eklemek anlamsız olurdu. Ayrıca `0` bayrağı `%s` ve `%c` ile kullanılırsa sonucu standart tanımlamaz; metin doldurmak için kullanmayalım.

Genişliği çalışma anında belirlemek

Genişliği ya da hassasiyeti kodda sabit yazmak yerine bir argümandan alabiliriz. Bunun için sayı yerine `*` yazılır ve `*` için bir `int` argüman, değerden **önce** verilir:

▸ yildizGenislik.c

```c
#include <stdio.h>

int main(void) {
    int genislik = 8;
    int hassasiyet = 2;
    double pi = 3.14159265;

    printf("[%*d]\n", genislik, 42);
    printf("[%-*d]\n", genislik, 42);
    printf("[%*.*f]\n", genislik, hassasiyet, pi);
    printf("[%.*s]\n", 3, "Merhaba");
    return 0;
}
```

ÇIKTI

```
[      42]
[42      ]
[    3.14]
[Mer]
```

Son satırdaki `%.*s`, metnin yalnızca ilk 3 karakterini yazar. Hassasiyetin `%s` ile kullanımı "en çok bu kadar karakter" anlamına gelir.

Yuvarlama

`%.2f` sayıyı iki basamağa **yuvarlar**, kesmez. Tamsayıya `(int)` ile dönüştürmek ise ondalık kısmı keser (Bölüm 5). İkisinin farkı aşağıdaki çıktıda görülüyor. Son iki satır beklenmedik olabilir:

▸ yuvarlama.c

```c
#include <stdio.h>

int main(void) {
    printf("%.2f\n", 3.14159);
    printf("%.2f\n", 3.999);
    printf("%.0f\n", 2.7);
    printf("%d\n", (int)2.7);
    printf("%.2f\n", 2.675);
```

```
    printf("%.2f\n", 1.005);
    return 0;
}
```

```
ÇIKTI
3.14
4.00
3
2
2.67
1.00
```

Elle yuvarlasak `2.675` için `2.68`, `1.005` için `1.01` bulurduk. Ancak bu sayılar ikilik tabanda tam saklanamaz (Bölüm 5'teki `0.1 + 0.2` örneğiyle aynı neden); bellekteki değer `2.67499999...` biçimindedir ve `printf` gerçek değeri yuvarlar. Bu bir hata değil, ondalık kesirlerin ikilik tabandaki doğal sonucudur.

Hizalı tablo yazdırmak

Genişlik ve hassasiyeti birleştirince düzgün sütunlu çıktılar elde ederiz. Kural basit: başlık, ayırıcı çizgi ve veri satırlarında **aynı genişlikleri** kullanmak. İsimleri sola, sayıları sağa yaslayalım:

notTablosu.c

```c
#include <stdio.h>

int main(void) {
    printf("%-8s %6s %6s %8s\n", "Ad", "Yazili", "Uygul.", "Ortalama");
    printf("-------- ------ ------ --------\n");
    printf("%-8s %6d %6d %8.2f\n", "Deniz", 70, 85, (70 + 85) / 2.0);
    printf("%-8s %6d %6d %8.2f\n", "Mehmet", 95, 90, (95 + 90) / 2.0);
    printf("%-8s %6d %6d %8.2f\n", "Elif", 55, 62, (55 + 62) / 2.0);
    return 0;
}
```

```
ÇIKTI
Ad       Yazili Uygul. Ortalama
-------- ------ ------ --------
Deniz        70     85    77.50
Mehmet       95     90    92.50
Elif         55     62    58.50
```

`printf` argüman olarak verilen ifadeyi önce hesaplar, sonra yazar. Bu yüzden `(70 + 85) / 2.0` doğrudan verilebilir. Bölme işleminde `2.0` kullandığımıza dikkat edelim; `2` yazsaydık tamsayı bölmesi olurdu (`155 / 2` = 77) ve `%8.2f` ile yazdırmak da tanımsız davranış olurdu.

Kalan belirteçler: %s, %p, %zu

Kalan üç belirteç için kısa notlar:

- `%s`, sonu `\0` ile biten karakter dizisini (string) yazar. Şimdilik yalnızca çift tırnaklı metin sabitleriyle kullanıyoruz; karakter dizilerini Bölüm 11'de işleyeceğiz.
- `%p`, bir **adresi** yazar. Argüman `void *` tipinde olmalıdır, bu yüzden dönüştürme (cast) yazarız. Adres her çalıştırmada değişebileceği için çıktıyı doğrulayamayız. İşaretçileri Bölüm 12'de ayrıntılı göreceğiz.
- `%zu`, `size_t` tipini yazar; `sizeof` bu tipi üretir.

adresYaz.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 7;
    printf("sayi değişkeninin adresi: %p\n", (void *)&sayi);
    printf("int boyutu: %zu bayt\n", sizeof(int));
    return 0;
}
```

ÇIKTI

```
sayi değişkeninin adresi: 0x7ffd2c8e4a3c
int boyutu: 4 bayt
```

Adres sistemden sisteme ve çalıştırmadan çalıştırmaya farklıdır; sizin ekranınızda başka bir değer görünecektir.

Yanlış belirteç

`printf` argümanların tipini bilmez; yalnızca biçim metnindeki harfe güvenir. `%d` görürse belleği `int` gibi, `%f` görürse `double` gibi okur. Argümanın gerçek tipi başkaysa sonuç anlamsız bir sayıdır, üstelik bu bir **tanımsız davranıştır**. Bu yüzden böyle bir programın çıktısını göstermiyoruz. Neyse ki `-Wall`, biçim metnini argümanlarla karşılaştırır:

```c
// yanlisBelirtec.c içindeki main'in gövdesi
int adet = 5;
double fiyat = 2.5;
printf("%f\n", adet);          // satır 6
printf("%d\n", fiyat);         // satır 7
printf("%d %d\n", adet);       // satır 8
printf("%d\n", adet, fiyat);   // satır 9
```

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra yanlisBelirtec.c -o yanlisBelirtec
yanlisBelirtec.c: In function 'main':
yanlisBelirtec.c:6:14: warning: format '%f' expects argument of type
'double', but argument 2 has type 'int' [-Wformat=]
yanlisBelirtec.c:7:14: warning: format '%d' expects argument of type
'int', but argument 2 has type 'double' [-Wformat=]
yanlisBelirtec.c:8:17: warning: format '%d' expects a matching 'int'
argument [-Wformat=]
yanlisBelirtec.c:9:12: warning: too many arguments for format
[-Wformat-extra-args]
```

Program bu uyarılarla yine de derlenir; çünkü bunlar hata değil uyarıdır. Ama dört satırın dördü de yanlıştır. Kural: uyarı varsa programı çalıştırmadan önce uyarıyı düzeltmek.

> **▲ Sık Yapılan Hata · Tamsayı bölmesini %f ile yazdırmak**
>
> `printf("%.2f\n", 5 / 2);` satırında `5 / 2` iki `int`'in bölümüdür ve sonucu `int`'tir (2); `%.2f` ise `double` bekler. Tersi de yanlıştır: `printf("%d\n", 5 / 2.0);`. Belirteci seçmeden önce **ifadenin tipini** bulalım: iki tamsayıdan çıkan sonuç tamsayıdır, işleme bir ondalıklı sayı katılıyorsa sonuç ondalıklıdır.

printf'in dönüş değeri

`printf` da bir değer döndürür: yazdığı karakter sayısını (hata olursa negatif bir değer). Bu değer çoğunlukla kullanılmaz, ama bir metnin ekranda kaç karakter yer kapladığını öğrenmek için işe yarar:

printfDonus.c

```c
#include <stdio.h>

int main(void) {
    int n = printf("Merhaba\n");
    printf("%d karakter yazıldı\n", n);
    return 0;
}
```

ÇIKTI

```
Merhaba
8 karakter yazıldı
```

`Merhaba` yedi harf, `\n` de bir karakter olduğu için dönüş değeri 8'dir.

Tek karakter ve satır yazdırma: putchar ve puts

Her yazdırma işi için `printf` gerekmez. İki küçük fonksiyon daha vardır:

- `putchar(c)`, tek bir karakter yazar.
- `puts(metin)`, metni yazar ve **sonuna kendiliğinden `\n` ekler**.

putsOrnek.c

```c
#include <stdio.h>

int main(void) {
    puts("Birinci satır");
    puts("İkinci satır");
    putchar('A');
    putchar('\n');
    printf("Birinci satır\n");
    return 0;
}
```

ÇIKTI

```
Birinci satır
İkinci satır
A
Birinci satır
```

`puts`, biçim belirteci yorumlamaz. Bu özellik bir güvenlik açısından da önemlidir: kullanıcıdan gelen bir metni `printf(metin)` biçiminde doğrudan biçim metni olarak vermek, içinde `%` varsa tehlikelidir. Metni olduğu gibi yazmak istiyorsak `puts(metin)` ya da `printf("%s\n", metin)` kullanırız.

> **Üç Dilde · Çıktı yazdırma üç dilde** *C · C++ · C#*
>
> Üç dilde de ekrana yazmanın birkaç yolu vardır. Aşağıdaki satırların hepsi aynı çıktıyı üretir: `Toplam: 12.50 TL`

```c
// C: biçim metni + argümanlar
printf("Toplam: %7.2f TL\n", 12.5);
```

```cpp
// C++: akış (stream) ve << operatörü; C'nin printf'i de kullanılabilir
#include <iostream>
#include <iomanip>
std::cout << "Toplam: " << std::setw(7) << std::fixed
          << std::setprecision(2) << 12.5 << " TL\n";
```

```csharp
// C#: dize enterpolasyonu ($"...") ile biçim belirteci
Console.WriteLine($"Toplam: {12.5,7:F2} TL");
```

C'de biçim ve değerler ayrıdır ve tip uyuşmazlığını yalnızca derleyici uyarısı yakalar. C++'ın `cout`'u tipi kendisi bilir, bu yüzden "yanlış belirteç" hatası yoktur; bunun bedeli, `setw` ve `setprecision` gibi daha uzun yazımdır. C#'ta `{değer,genişlik:biçim}` sözdizimi, C'nin `%genişlik.hassasiyet` mantığının karşılığıdır: `12.5,7:F2` ifadesi "12,5 değerini 7 karakter genişliğinde, 2 ondalıkla yaz" demektir. Aynı konunun C++ ve C# ayrıntılarını ilgili bölümlerde göreceğiz.

6.2 scanf ile Klavyeden Okuma

`scanf` (scan formatted), `printf`'in tersini yapar: klavyeden yazılan metni okur, biçim belirtecine göre sayıya ya da karaktere çevirir ve değişkene yerleştirir. Genel kalıp şöyledir:

```c
scanf("biçim metni", &degisken1, &degisken2, ...);
```

İlk örnekte kullanıcıdan yaşını alıp bir yıl sonraki yaşını hesaplayalım. Çıktı bloğunda `{{...}}` içindeki kısım kullanıcının klavyeden yazdığı değerdir:

▸ yasOku.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    printf("Yaşınızı girin: ");
    scanf("%d", &yas);
    printf("Gelecek yıl %d yaşında olacaksınız.\n", yas + 1);
    return 0;
}
```

```
ÇIKTI
Yaşınızı girin: 19
Gelecek yıl 20 yaşında olacaksınız.
```

`scanf` çağrıldığında program durur ve kullanıcı Enter'a basana kadar bekler. Kullanıcı neyin istendiğini bilmediği için `scanf`'ten önce `printf` ile bir **istem** (prompt) yazarız. İstemin sonuna `\n` koymadığımıza dikkat edelim; böylece kullanıcının yazdığı cevap soruyla aynı satırda görünür.

& işleci neden gerekli?

`printf` değeri **kullanır**; `scanf` ise değeri bir yere **yazar**. Yazabilmesi için değişkenin değerini değil, bellekteki yerini, yani **adresini** bilmesi gerekir. `&yas` ifadesi "`yas` değişkeninin adresi" demektir.

> **Tanım: Adres işleci (&)**
>
> Tekli `&` işleci, işlenenin bellekteki adresini verir. `&yas`, `yas` değişkeninin ilk baytının adresidir. Adres alınabilen değerlerin (işaretçiler) ayrıntısını Bölüm 12'de göreceğiz.

Belleğin durumunu şöyle düşünebiliriz:

```
scanf("%d", &yas);

  yas  (adres 0x1000)
+----------+
|    ?     |   <-- scanf, okuduğu 19'u bu adrese yazar
+----------+
```

Bu yüzden kural basittir: `scanf`'te sayı ve karakter değişkenlerinin önüne `&` koyarız, `printf`'te koymayız. Bölüm 11'de karakter dizilerini (string) okurken tek bir istisna ile karşılaşacağız.

Belirteçler

`scanf` belirteçleri `printf`'inkilere benzer, ancak bir fark önemlidir: `scanf` değişkenin bellekteki **kaç baytını** yazacağını belirteçten anlar. `printf`'te `float` sessizce `double`'a çevrilir; `scanf`'te böyle bir çeviri yoktur, bu yüzden `float` ile `double` için belirteçler farklıdır.

Değişken tipi	`scanf` belirteci	`printf` belirteci
`short`	`%hd`	`%hd` ya da `%d`
`int`	`%d`	`%d`
`unsigned int`	`%u`	`%u`
`long`	`%ld`	`%ld`
`long long`	`%lld`	`%lld`
`float`	`%f`	`%f`
`double`	**`%lf`**	`%f`
`char`	`%c`	`%c`

`lf` içindeki `l`, "long float", yani `double` anlamına gelir. Yanlış seçim belleği bozar: `short` için `%d` kullanırsak `scanf` 2 baytlık değişkene 4 bayt yazar; `double` için `%f` kullanırsak 8 baytlık değişkenin yalnızca 4 baytı yazılır ve sonuç bozuk bir sayı olur.

> **◆ Kritik Nokta · double okurken %lf, yazarken %f**
>
> `double` bir değişkeni `printf` ile yazdırırken `%f`, `scanf` ile okurken `%lf` kullanılır. `float` için ikisinde de `%f` yazılır. Bu tabloyu bir kez doğru öğrenmek, birçok sessiz hatayı önler.

`scanf`'te `%i` ve `%d` de farklıdır: `%d` yalnızca onluk sayı okur; `%i` ise girdideki öneke bakarak `0x1F` gibi onaltılık ya da `017` gibi sekizlik sayıyı da yorumlar. Bu bölümde yalnızca `%d` kullanacağız.

Birden çok değer okumak

Tek `scanf` çağrısıyla birden çok değer okuyabiliriz; her belirteç için bir `&degisken` veririz. Girdiler boşlukla ya da Enter ile ayrılabilir: `scanf("%d %d", &a, &b)` için `12 30` yazmakla `12`, Enter, `30` yazmak aynıdır. Bunun nedeni `scanf`'in boşluk kurallarıdır:

Biçimde	Girdide ne olur?
`%d`, `%f`, `%lf`	Sayıdan önceki boşluk, sekme ve Enter'ları kendisi atlar
`%c`	**Hiçbir şey atlamaz**; sıradaki karakteri, boşluk ya da Enter olsa bile alır
Biçimdeki boşluk ( )	Girdideki sıfır ya da daha fazla boşluk, sekme ve Enter'ı atlar
Başka karakter (`/`, `:`, `,`)	Girdide aynen bulunmalıdır; bulunmazsa okuma orada durur

Farklı tipte üç değeri aynı anda okuyalım:

▸ ucDeger.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    double boy = 0.0;
    char grup = ' ';

    printf("Yaş, boy (m) ve kan grubu harfi: ");
    scanf("%d %lf %c", &yas, &boy, &grup);
    printf("Yaş: %d, boy: %.2f m, grup: %c\n", yas, boy, grup);
    return 0;
}
```

```
ÇIKTI
Yaş, boy (m) ve kan grubu harfi: 19 1.78 A
Yaş: 19, boy: 1.78 m, grup: A
```

Biçimdeki `%c`'nin önündeki boşluk önemlidir: `1.78` ile `A` arasındaki boşluğu atlar. Bunun neden gerekli olduğunu "arabellek" başlığında ayrıntılı göreceğiz.

Kalıplı girdi

Biçim metnindeki sıradan karakterler girdide aynen beklenir. Bu sayede `5/3/2005` ya da `09:30` gibi girdileri tek hamlede ayrıştırabiliriz:

▸ tarih.c

```c
#include <stdio.h>

int main(void) {
    int gun = 0, ay = 0, yil = 0;
    printf("Tarih (g/a/yyyy): ");
    scanf("%d/%d/%d", &gun, &ay, &yil);
```

```
    printf("Tarih: %02d.%02d.%d\n", gun, ay, yil);
    return 0;
}
```

```
ÇIKTI
Tarih (g/a/yyyy): 5/3/2005
Tarih: 05.03.2005
```

`scanf` üç sayıyı `/` karakterlerini atlayarak okudu. Yazdırırken kullandığımız `%02d`, günü ve ayı iki haneli, başına sıfır koyarak yazdı. Kullanıcı kalıba uymazsa (`5 3 2005` yazarsa) `scanf` ilk `/` beklediği yerde durur ve kalan değişkenler değişmez. Değişkenlere baştan `= 0` vermemiz bu yüzden önemlidir; vermeseydik çöp değer yazdırırdık.

Sık hatalar

Biçim metninin argümanlarla uyuşmasını `-Wall`, `scanf` için de denetler. Başlangıçta en çok yapılan iki hata, `&` unutmak ve `double`'ı `%f` ile okumaktır:

```
int sayi = 0;
double boy = 0.0;
scanf("%d", sayi);       // & unutuldu
scanf("%f", &boy);       // double için %f
```

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra amp.c -o amp
amp.c:6:13: warning: format '%d' expects argument of type 'int *',
but argument 2 has type 'int' [-Wformat=]
amp.c:7:13: warning: format '%f' expects argument of type 'float *',
but argument 2 has type 'double *' [-Wformat=]
```

İlk uyarı `&` unutulduğunu söylüyor: `scanf` bir `int *` (int'in adresi) bekliyor, biz `int` verdik. Bu durumda `scanf`, `sayi`'nın değerini (burada 0) bir adres sanıp oraya yazmaya çalışır. Sonuç tanımsız davranıştır ve program çoğunlukla çöker. İkinci uyarı `double` için `%f` yazdığımızı gösteriyor ve çözümü de verir: `%lf`. Bu hatada program çökmeyebilir, yalnızca `boy` bozuk bir değer alır; sessiz olduğu için daha da tehlikelidir.

> **▲ Sık Yapılan Hata · scanf'in biçim metnine soru yazmak**
>
> `scanf("Yaşınız: %d", &yas);` yazmayalım. `scanf`'in biçim metni ekrana bir şey yazdırmaz; girdide aranacak **şablondur**. Bu satırda kullanıcı `Yaşınız: 19` yazmadıkça okuma başarısız olur. Soruyu `printf` ile soralım, `scanf`'e yalnızca belirteç verelim. Bir de `scanf("%d\n", &x);` yazmayalım: biçimin sonundaki `\n` bir boşluk gibi çalışır ve boşluk olmayan bir karakter gelene kadar okumaya devam eder; program takılmış gibi görünür.

> **▲ Sık Yapılan Hata · Ondalık sayıda virgül**
>
> Türkçe bölge ayarlarında sayı tuş takımı virgül yazabilir, ama C ondalık ayırıcı olarak **noktayı** bekler. `scanf("%lf", &boy)` ile `1,78` girilirse `boy`'a yalnızca `1` okunur, `,78` arabellekte kalır. Çıktıda da C her zaman nokta yazar (`1.78`).

6.3 scanf'in Dönüş Değeri

`scanf` bir değer döndürür: başarıyla okuyup değişkene yerleştirdiği **öğe sayısı**. Kullanıcının beklenen girdiyi yazıp yazmadığını anlamanın yolu budur.

Dönüş değeri	Anlamı
1, 2, 3, …	O kadar öğe başarıyla okundu
0	Hiçbir öğe okunamadı (girdi beklenen biçime uymuyor)
`EOF` (genellikle -1)	Girdi bitti ya da okunamadı (Linux/macOS'ta Ctrl+D, Windows'ta Ctrl+Z ve Enter)

Sayı beklenirken harf yazılırsa ne olduğuna bakalım:

▸ harfGirdim.c

```c
#include <stdio.h>

int main(void) {
    int sayi = -1;
    printf("Bir sayı girin: ");
    int okunan = scanf("%d", &sayi);
    printf("scanf %d öğe okudu, sayi = %d\n", okunan, sayi);
    return 0;
}
```

```
ÇIKTI
Bir sayı girin: abc
scanf 0 öğe okudu, sayi = -1
```

Kullanıcı `abc` yazdı; `%d` bir rakam bekliyordu, `a` rakam değil. `scanf` hiçbir şey okuyamadı ve 0 döndürdü; `sayi` olduğu gibi (-1) kaldı. Program ne çöktü ne uyarı verdi; `scanf` sessizce vazgeçti. Dönüş değerine bakmazsak hatayı fark edemeyiz.

Birden çok değer okurken dönüş değeri, kaç öğenin başarıyla okunduğunu verir. `scanf("%d %d", &a, &b)` çağrısına `7 abc` yazılırsa `7` okunur, `abc` yüzünden ikinci okuma başarısız olur: dönüş değeri 1'dir, `a` değişir, `b` değişmez. Ayrıca başarısız olunca `scanf` geçersiz karakterlerin (`abc`) üzerinden atlamaz; onlar arabellekte bekler ve sonraki bir `scanf("%d")` aynı yerde yine takılır. Bunu hemen aşağıda ele alacağız.

Dönüş değerini kullanmanın standart kalıbı `if` ile `!= 1` denetimidir (`if`'i Bölüm 7'de işleyeceğiz; şimdilik kalıbı olduğu gibi okuyalım):

```c
if (scanf("%d", &yas) != 1) {
    printf("Geçersiz girdi!\n");
    return 1;
}
```

Bu kod, "`scanf` bir öğe okuyamadıysa hata mesajı ver ve programı bitir" demektir. Böylece programın çöp değerle devam etmesini engelleriz.

◆ Kritik Nokta · scanf'in dönüş değerini denetleyin

`scanf`'in `0` döndürmesi hata değil, "okunamadı" demektir; program çalışmaya devam eder. Girdiye güvenemeyeceğimiz her yerde, okuduğumuz öğe sayısını beklediğimiz sayıyla karşılaştırmalıyız. Sayımı her zaman ilk uyuşmazlığın olduğu yerde keseriz: `scanf("%d-%d", &a, &b)` çağrısına `5 3` yazılırsa `5` okunur, `-` beklenirken boşluk görülür; dönüş değeri 1'dir.

6.4 Girdi Arabelleği ve %c Tuzağı

Klavyede yazdığımız karakterler programa tek tek ulaşmaz. İşletim sistemi yazılanları bir **arabellekte** (buffer) biriktirir ve biz Enter'a basınca satırı programa verir. Enter'a basıldığında arabelleğe `'\n'` (yeni satır) karakteri de eklenir. `scanf` bu arabellekten ihtiyacı kadarını alır, kalanı arabellekte bırakır.

```
Klavyede yazılan:   2  0  [Enter]
Arabelleğin içi:   '2' '0' '\n'

scanf("%d", &yas)  okur:   '2' '0'      ->  yas = 20
Arabellekte kalan:                 '\n'
```

`%d` sayıyı okuyunca durur; Enter'ın bıraktığı `'\n'` arabellekte kalır. Sonraki `scanf("%d")` için bu sorun değildir, çünkü `%d` baştaki boşlukları ve Enter'ları kendisi atlar. Ama `%c` hiçbir şey atlamaz: arabellekte ne varsa onu alır. Klasik tuzak budur:

▸ tuzak.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    char harf = ' ';

    printf("Yaş: ");
    scanf("%d", &yas);
    printf("Baş harfiniz: ");
    scanf("%c", &harf);
    printf("yas = %d, harf kodu = %d\n", yas, harf);
    return 0;
}
```

ÇIKTI

```
Yaş: 20
Baş harfiniz: yas = 20, harf kodu = 10
```

Kullanıcı `Baş harfiniz:` sorusuna cevap yazamadı; `scanf("%c")` arabellekte hazır bekleyen `'\n'` karakterini okudu ve program devam etti. Harfin kodunu `%d` ile yazdırdık: 10, yani yeni satır karakterinin ASCII kodu.

Çözüm 1: `%c`'nin önüne bir boşluk koymak. Biçimdeki boşluk, arabellekte kalan boşluk, sekme ve Enter karakterlerini atlar; sonra gerçek karakteri okur:

▸ cozumBosluk.c

```c
#include <stdio.h>

int main(void) {
```

```
    int yas = 0;
    char harf = ' ';

    printf("Yaş: ");
    scanf("%d", &yas);
    printf("Baş harfiniz: ");
    scanf(" %c", &harf);
    printf("yas = %d, harf = %c\n", yas, harf);
    return 0;
}
```

```
ÇIKTI
Yaş: 20
Baş harfiniz: E
yas = 20, harf = E
```

Bu sefer `scanf(" %c")` önce `'\n'` karakterini atladı, arabelleğin boşaldığını görünce yeni bir girdi bekledi ve `E`'yi okudu.

> ◆ **Kritik Nokta · Karakter okurken " %c" yazın**
>
> Bir önceki okumanın arabellekte ne bıraktığından bağımsız çalışması için `%c`'yi her zaman başında boşlukla, yani `" %c"` biçiminde yazmak güvenli bir alışkanlıktır.

getchar ve putchar

Tek bir karakter okumak için `scanf` yerine `getchar()` kullanabiliriz. `getchar` arabellekten bir karakter alır ve `int` olarak döndürür. `%c` gibi, hiçbir şey atlamaz: arabellekte ne varsa onu verir. Sonucu `char` değil `int` bir değişkende saklarız; çünkü girdi bittiğinde `getchar` özel bir değer olan `EOF` döndürür ve bu değer `char`'a sığmayabilir.

▸ tekKarakter.c

```c
#include <stdio.h>

int main(void) {
    printf("Bir karakter yazın: ");
    int c = getchar();
    printf("Yazdığınız karakter: ");
    putchar(c);
    putchar('\n');
    printf("ASCII kodu: %d\n", c);

    int sonraki = getchar();
    printf("Sıradaki karakterin kodu: %d\n", sonraki);
    return 0;
}
```

```
ÇIKTI
Bir karakter yazın: K
Yazdığınız karakter: K
ASCII kodu: 75
Sıradaki karakterin kodu: 10
```

Kullanıcı yalnızca `K` yazıp Enter'a bastı, ama arabellekte iki karakter vardı: `K` ve `'\n'`. İkinci `getchar()` Enter'ı aldı; kodu 10'dur. Enter da arabellekte sıradan bir karakterdir.

Arabelleği temizlemek

Bazen arabellekte işimize yaramayan karakterler kalır: `scanf("%d")`'den sonra `'\n'`, ya da kullanıcının sayı yerine yazdığı `abc`. Eski kaynaklarda bunu `fflush(stdin)` ile temizlemek önerilir. Bunu yapmayalım:

> **▲ Sık Yapılan Hata · fflush(stdin) kullanmayın**
>
> `fflush`, C standardında yalnızca **çıktı** akımları için tanımlıdır. `stdin` bir girdi akımıdır; `fflush(stdin)` standartta tanımsız davranıştır. Eski Microsoft kütüphanesi bunu bir eklenti olarak sunduğu için Windows'ta çalışıyormuş gibi görünür; Linux ve macOS'ta genellikle arabelleği temizlemez ve hata kalır. `fflush(stdout)` doğrudur, `fflush(stdin)` yanlıştır.

Standarda uygun temizleme kalıbı, `'\n'` görene kadar karakterleri tek tek okuyup atmaktır:

```c
int c;
while ((c = getchar()) != '\n' && c != EOF) {
    // karakteri at
}
```

Bu kod, "bir karakter oku; yeni satır ya da girdi sonu değilse bir tane daha oku" demektir. `while` döngüsünü Bölüm 8'de ayrıntılı işleyeceğiz; şimdilik tek satırlık bir kalıp olarak alalım. `c != EOF` denetimi, girdinin tamamen bittiği durumda döngünün sonsuza kadar dönmesini engeller. Bu kalıbı her seferinde yazmak yerine kısa bir fonksiyona sarmak mümkündür; fonksiyonları Bölüm 9'da öğrenince bunu yapacağız.

Şimdi `%c` tuzağının ikinci çözümünü görelim. Önce sayıyı okuyoruz, sonra satırın kalanını atıyoruz, sonra karakteri okuyoruz:

▸ tamponTemizle.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 0;
    char harf = ' ';
    int c;

    printf("Sayı: ");
    scanf("%d", &sayi);
    while ((c = getchar()) != '\n' && c != EOF) {
        // satırın kalanını at
    }

    printf("Harf: ");
    scanf("%c", &harf);
    printf("sayi = %d, harf = %c\n", sayi, harf);
    return 0;
}
```

```
ÇIKTI
Sayı: 20
Harf: E
sayi = 20, harf = E
```

İki çözümün farkı şudur:

Çözüm	Ne yapar?	Ne zaman?
`scanf(" %c", &harf);`	Okumadan önce boşluk ve Enter'ları atlar	Karakter okumadan hemen önce; en kolayı
`while` ile temizleme	Satırın kalanını tamamen atar	Satırda fazlalık (`20 abc`) kalmış olabilirse

6.5 Metin Okumak: %s, Boşluklu Metin ve fgets

Karakter dizilerini (string) Bölüm 11'de ayrıntılı işleyeceğiz; ancak kullanıcıdan bir ad ya da cümle okumak çok yaygın bir ihtiyaç olduğundan şimdiden kısa bir önizleme yapalım. Metin saklamak için `char` tipinde bir **dizi** (array) tanımlarız: `char ad[20];` bu kutu 20 karakterlik yer ayırır. Metnin sonuna ise otomatik olarak bir `'\0'` (sıfır karakteri) eklenir; yani 20 karakterlik dizide en çok 19 harf saklanabilir.

%s boşluğa kadar okur

`scanf("%s", ad)` boşluk, sekme ya da Enter görünceye kadar okur. Dikkat: burada `&` **yok**, çünkü bir dizinin adı zaten dizinin adresi gibi davranır (Bölüm 10 ve 12).

▸ adOku.c

```c
#include <stdio.h>

int main(void) {
    char ad[20] = "";
    printf("Adınız: ");
    scanf("%19s", ad);
    printf("Merhaba, %s!\n", ad);
    return 0;
}
```

ÇIKTI

```
Adınız: Mehmet Ali
Merhaba, Mehmet!
```

Kullanıcı `Mehmet Ali` yazdı, ama `%s` boşlukta durdu: `ad` yalnızca `Mehmet` oldu, `Ali` arabellekte kaldı. `scanf("%s")` ile boşluklu metin okunamaz.

Taşma tehlikesi ve %19s

Biçimdeki `19` sayısı, `%s`'in en çok kaç karakter okuyacağını sınırlar. Bu sınır kritiktir: `ad` dizisi 20 baytlıktır ve `'\0'` için 1 bayt gerekir. Sınır koymazsak ve kullanıcı 30 harflik bir ad yazarsa, `scanf` dizinin ötesindeki belleğe yazar.

> ▲ **Sık Yapılan Hata · Sınırsız %s kullanmayın**
>
> `scanf("%s", ad);` yazılan metnin uzunluğunu denetlemez. Dizi kısaysa `scanf` onun dışındaki belleği ezer; buna **arabellek taşması** (buffer overflow) denir ve tanımsız davranıştır. Bu hata, tarihte birçok güvenlik açığının kaynağı olmuştur. Dizi boyutundan bir eksik sınır yazalım: `char ad[20]` için `%19s`.

Boşluklu metin için fgets

Boşluk içeren bir satırın tamamını okumanın güvenli yolu `fgets`'tir. İmzası şöyledir:

```
fgets(dizi, boyut, stdin);
```

`fgets` en çok `boyut - 1` karakter okur, sona `'\0'` ekler ve satır sonunda (`'\n'`) durur. `'\n'` karakterini de diziye **dahil eder**. Taşma olmaz, çünkü boyutu biz veririz. `stdin`, klavyeyi temsil eden standart girdi akımının adıdır; dosya işlemlerinde Bölüm 17'de başka akımlarla tanışacağız.

▸ satirOku.c

```c
#include <stdio.h>

int main(void) {
    char ad[30] = "";
    printf("Ad soyad: ");
    fgets(ad, sizeof ad, stdin);
    printf("[%s]\n", ad);
    return 0;
}
```

ÇIKTI

```
Ad soyad: Mehmet Ali
[Mehmet Ali
]
```

Köşeli parantezler, `fgets`'in satır sonunu da dizide sakladığını gösteriyor: kapanış `]` bir alt satıra düştü. Sondaki `'\n'`'i atmak için Bölüm 11'de `strcspn` ve benzeri fonksiyonları göreceğiz. Şimdilik bilmemiz gereken, `fgets`'in satır sonunu atmadığıdır.

`scanf` ile `fgets`'i karıştırmak ise aynı `%c` tuzağını doğurur: `scanf("%d")`'den sonra arabellekte kalan `'\n'`'i `fgets` hemen okur ve boş bir satır döndürür. Önce arabelleği temizlemeli ya da tüm girdiyi `fgets` ile okuyup sayıya sonradan çevirmeliyiz. Bu ikinci yaklaşıma bölüm sonunda döneceğiz.

> *Tarihçe · gets neden kaldırıldı?*
>
> Eskiden satır okumak için `gets(dizi)` kullanılırdı. Bu fonksiyon boyut parametresi almadığı için taşmayı engellemenin hiçbir yolu yoktu. C99'da eskimiş (deprecated) ilan edildi, C11'de standarttan tamamen çıkarıldı. Eski kitaplarda ve örneklerde görürseniz kullanmayın; yerine `fgets` vardır.

6.6 Girdi Doğrulama

Şimdiye kadar kullanıcının doğru girdi yazacağını varsaydık. Gerçek programlar böyle çalışmaz: kullanıcı sayı yerine harf yazar, aralık dışı değer girer ya da yanlışlıkla Enter'a basar. Girdiyi kullanmadan önce denetlemek ve geçersizse yeniden istemek gerekir. Bunun için üç parçaya ihtiyacımız var:

1. `scanf`'in dönüş değeri (okuma başarılı mı?),
2. değerin aralığı (anlamlı mı?),
3. başarısız okumadan sonra arabelleği temizlemek (aynı hatada takılmamak için).

İlk iki parçayı `if` ile, tekrarı ise `while` döngüsüyle kuracağız. Bunları Bölüm 7 ve 8'de ayrıntılı işleyeceğiz; burada yalnızca kalıp olarak kullanıyoruz: `while (koşul) { ... }` "koşul doğru olduğu sürece bloğu tekrarla" demektir.

pozitifSayi.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 0;
    int tamam = 0;

    while (!tamam) {
        printf("Pozitif bir sayı girin: ");
        int okunan = scanf("%d", &sayi);

        // satırın kalanını at
        int c;
        while ((c = getchar()) != '\n' && c != EOF) {
        }

        if (okunan == EOF) {
            return 1;
        }
        if (okunan == 1 && sayi > 0) {
            tamam = 1;
        } else {
            printf("Geçersiz girdi, tekrar deneyin.\n");
        }
    }
    printf("Girilen sayı: %d\n", sayi);
    return 0;
}
```

ÇIKTI

```
Pozitif bir sayı girin: abc
Geçersiz girdi, tekrar deneyin.
Pozitif bir sayı girin: -5
Geçersiz girdi, tekrar deneyin.
Pozitif bir sayı girin: 12
Girilen sayı: 12
```

`tamam` bir **bayrak** (flag) değişkenidir: geçerli girdi alınana kadar 0 kalır. Her turda sırasıyla şunlar olur: `scanf` okumayı dener; ardından satırın kalanı, yani `abc` ya da Enter, arabellekten atılır; son olarak dönüş değeri ve aralık denetlenir. Arabelleği temizlemeseydik `abc` yerinde kalır ve her turda `scanf` aynı yerde takılarak sonsuz döngüye girerdik. `okunan == EOF` denetimi, girdi tamamen bittiğinde (örneğin kullanıcı Ctrl+D'ye bastığında) programın sonsuza dek dönmesini engeller.

Daha sağlam yöntem: fgets ile sscanf

Girdiyi `scanf` ile doğrudan okumak yerine önce satırı `fgets` ile bir metin olarak alıp sonra `sscanf` ile sayıya çevirmek, arabellek karmaşasını ortadan kaldırır. `sscanf`, `scanf` ile aynı biçim belirteçlerini kullanır, yalnızca klavye yerine **bir metinden** okur:

sscanfIle.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    int tamam = 0;
    char satir[64];
```

```
    while (!tamam) {
        printf("Yaşınız (1-120): ");
        if (fgets(satir, sizeof satir, stdin) == NULL) {
            printf("\nGirdi bitti.\n");
            return 1;
        }
        if (sscanf(satir, "%d", &yas) == 1 && yas >= 1 && yas <= 120) {
            tamam = 1;
        } else {
            printf("Geçersiz girdi, tekrar deneyin.\n");
        }
    }
    printf("Yaşınız: %d\n", yas);
    return 0;
}
```

```
ÇIKTI
Yaşınız (1-120): abc
Geçersiz girdi, tekrar deneyin.
Yaşınız (1-120): 150
Geçersiz girdi, tekrar deneyin.
Yaşınız (1-120): 25
Yaşınız: 25
```

Her tur bir satırı tamamen tüketir; arabellekte artık karakter kalmaz. `fgets` girdi bitince `NULL` döndürür (Bölüm 12'de `NULL`'ı tanıyacağız), bu da sonsuz döngüyü önler. Bu yöntemin tek zayıflığı, `25abc` gibi girdilerin kabul edilmesidir: `sscanf` baştaki `25`'i okur ve geri kalanı yok sayar. Bunu katı biçimde denetlemek için ileride `strtol` fonksiyonunu göreceğiz.

> **İpucu · Hangi yöntemi seçmeli?**
>
> Küçük deneme programlarında `scanf` yeterlidir. Kullanıcıdan gelen girdinin hatalı olabileceği her yerde, önce `fgets` ile satırı almak ve `sscanf` ile çözümlemek daha öngörülebilirdir; çünkü her zaman bir satır tüketilir ve arabellekte artık kalmaz.

6.7 Örnek Programlar

Bölümün başından beri gördüklerimizi üç küçük uygulamada birleştirelim. Her birinde önce ne yapacağımızı, sonra kodu, sonra çıktıyı inceliyoruz.

Dairenin alanı ve çevresi

Yarıçapı kullanıcıdan alıp alan (πr^2) ve çevre ($2 \pi r$) hesaplayalım. Değerler ondalıklı olabileceği için `double` ve `%lf` kullanıyoruz; π için `const` bir sabit tanımlıyoruz:

▸ daire.c

```
#include <stdio.h>

int main(void) {
    const double PI = 3.14159265358979;
    double yaricap = 0.0;

    printf("Yarıçap (cm): ");
    if (scanf("%lf", &yaricap) != 1 || yaricap < 0) {
        printf("Geçersiz yarıçap.\n");
```

```
        return 1;
    }

    double alan = PI * yaricap * yaricap;
    double cevre = 2 * PI * yaricap;

    printf("Alan  : %8.2f cm^2\n", alan);
    printf("Çevre : %8.2f cm\n", cevre);
    return 0;
}
```

```
ÇIKTI
Yarıçap (cm): 2.5
Alan  :    19.63 cm^2
Çevre :    15.71 cm
```

`||` işlecinin kısa devre yaptığını Bölüm 5'ten hatırlayalım: `scanf` başarısız olursa (`!= 1` doğru) `yaricap < 0` hiç hesaplanmaz. Başarısız okumadan sonra `yaricap` zaten 0.0 olarak kalır, ama denetimi yine de doğru sırayla yazmak iyi bir alışkanlıktır. Her iki sonuç da `%8.2f` ile aynı genişlikte yazıldığı için ondalık noktalar alt alta gelir.

Sıcaklık çevirme

Celsius cinsinden bir sıcaklığı Fahrenheit ve Kelvin'e çevirelim: `F = C × 9 / 5 + 32` ve `K = C + 273.15`. Bölüm 5'te gördüğümüz tamsayı bölmesi tuzağından kaçınmak için sıcaklığı `double` tutuyoruz:

▸ sicaklikCevir.c

```c
#include <stdio.h>

int main(void) {
    double celsius = 0.0;

    printf("Sıcaklık (Celsius): ");
    if (scanf("%lf", &celsius) != 1) {
        printf("Geçersiz girdi.\n");
        return 1;
    }

    double fahrenheit = celsius * 9 / 5 + 32;
    double kelvin = celsius + 273.15;

    printf("%10.2f C\n", celsius);
    printf("%10.2f F\n", fahrenheit);
    printf("%10.2f K\n", kelvin);
    return 0;
}
```

```
ÇIKTI
Sıcaklık (Celsius): 25.5
     25.50 C
     77.90 F
    298.65 K
```

Üç satırın genişliği aynı olduğundan sayıların sağ kenarı hizalanır ve birimler bir sütunda toplanır. Aynı programı tamsayı değişkenlerle yazsaydık `9 / 5` ifadesi 1'e kesilirdi; `celsius * 9 / 5` ifadesinde ise `celsius` `double` olduğu için çarpma ondalıklı yapılır ve sorun çıkmaz.

Fatura hesabı

Son programda birim fiyatı, adedi, iskonto yüzdesini ve KDV yüzdesini okuyup hizalı bir fatura çıktısı üretelim. Dört değeri tek `scanf` ile okuyoruz ve dönüş değerinin 4 olup olmadığını denetliyoruz:

▸ fatura.c

```c
#include <stdio.h>

int main(void) {
    double birimFiyat = 0.0;
    int adet = 0;
    int iskontoYuzde = 0;
    int kdvYuzde = 0;

    printf("Birim fiyat, adet, iskonto %%, KDV %%: ");
    int okunan = scanf("%lf %d %d %d", &birimFiyat, &adet,
                       &iskontoYuzde, &kdvYuzde);
    if (okunan != 4) {
        printf("Eksik ya da geçersiz girdi.\n");
        return 1;
    }

    double araToplam = birimFiyat * adet;
    double iskonto = araToplam * iskontoYuzde / 100;
    double net = araToplam - iskonto;
    double kdv = net * kdvYuzde / 100;
    double genel = net + kdv;

    printf("\n%-16s %10.2f TL\n", "Ara toplam", araToplam);
    printf("%-16s %10.2f TL\n", "Iskonto", -iskonto);
    printf("%-16s %10.2f TL\n", "Net tutar", net);
    printf("%-16s %10.2f TL\n", "KDV", kdv);
    printf("%-16s %10.2f TL\n", "Genel toplam", genel);
    return 0;
}
```

```
ÇIKTI
Birim fiyat, adet, iskonto %, KDV %: 249.90 3 10 20

Ara toplam           749.70 TL
Iskonto              -74.97 TL
Net tutar            674.73 TL
KDV                  134.95 TL
Genel toplam         809.68 TL
```

Etiketlerde bilerek `İskonto` yerine `Iskonto` yazdık; bir önceki başlıktaki bayt sayma kuralı burada da geçerlidir. İstem metninde `%` işaretini yazdırmak için biçim metnine `%%` yazdık. Ara toplam `749.70`, iskonto `749.70 × 10 / 100 = 74.97`, net `674.73`, KDV `674.73 × 20 / 100 = 134.946` ve genel toplam `809.676`'dır; `%.2f` yuvarlayarak `134.95` ve `809.68` yazdırır. Bu programda para birimi için `double` kullandık. Gerçek ticari yazılımlarda yuvarlama hataları birikebileceği için kuruş cinsinden tamsayı tutmak daha güvenlidir (Bölüm 5).

6.8 Türkçe Karakterler ve Hizalama

Konsolda doğru görünen Türkçe karakterler bile sütunlu çıktıda sürpriz çıkarabilir. Kaynak dosyamız UTF-8 olduğunda `a`, `b`, `1` gibi karakterler 1 bayt, `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` gibi Türkçe harflerin her biri 2 bayt yer kaplar. `printf`'in genişlik değeri ise **karakter değil bayt** sayar:

▸ turkceGenislik.c

```c
#include <stdio.h>

int main(void) {
    printf("[%-8s]\n", "Ekmek");
    printf("[%-8s]\n", "Çorba");
    printf("[%-8s]\n", "Süt");
    return 0;
}
```

ÇIKTI

```
[Ekmek   ]
[Çorba  ]
[Süt    ]
```

`Ekmek` 5 baytlık yer kaplar, 3 boşlukla 8'e tamamlanır. `Çorba` 5 karakterdir ama `Ç` iki bayt olduğu için 6 bayttır; yalnızca 2 boşluk eklenir ve ekranda bir karakter kısa kalır. Aynı durum `Süt` için de geçerlidir. Sütunlu çıktılarda Türkçe karakterli metinler hizayı bozar; çözüm genişlikleri bu farka göre ayarlamak ya da sütun başlıklarında ve ürün adlarında ASCII karakterlerle yetinmektir.

> **Not · Windows konsolunda Türkçe karakterler**
>
> Konsolun kod sayfası UTF-8 değilse `ç`, `ş`, `ğ` gibi harfler bozuk görünür. Çözüm, Bölüm 3'te gördüğümüz gibi çalıştırmadan önce `chcp 65001` komutunu vermek ve kaynak dosyayı UTF-8 olarak kaydetmektir. Bozulma yazarken değil, **gösterirken** olur: program doğru baytları üretir, konsol onları yanlış yorumlar.

> **Üç Dilde · Klavyeden okuma üç dilde** *C · C++ · C#*
>
> Aynı işi üç dil nasıl yapıyor? Her birinde kullanıcıdan bir tamsayı ve bir ad okuyoruz.

```c
// C: biçim metni ve adres (&); hata denetimi bize ait
int yas = 0;
char ad[30] = "";
scanf("%d", &yas);
scanf("%29s", ad);
```

```cpp
// C++: cin ve >> işleci; tipi derleyici bilir, & ve belirteç yok
int yas = 0;
std::string ad;
std::cin >> yas;
std::cin >> ad;            // C'deki %s gibi boşlukta durur
std::getline(std::cin, ad); // boşluklu satırın tamamı
```

```cs
// C#: Console.ReadLine her zaman metin (string) döndürür
int yas = int.Parse(Console.ReadLine()!);
string ad = Console.ReadLine()!;
```

```
// Hatalı girdiye karşı:
if (int.TryParse(Console.ReadLine(), out int deger)) { /* geçerli */ }
```

C'de dönüşümü `scanf` yapar ve tip uyuşmazlığı çoğunlukla yalnızca uyarıdır. C++'ın `cin >> yas` ifadesi tipi bilir, bu yüzden `&` ve `%d` gerekmez; başarısız olursa akım hata durumuna geçer ve bunu denetlemek yine programcıya kalır. C#'ta `ReadLine` satırı metin olarak döndürür; sayıya çevirmek `int.Parse` (hatada istisna fırlatır) ya da `int.TryParse` (başarıyı `bool` döndürür) ile yapılır. `TryParse` kalıbı, C'deki `scanf(...) == 1` denetiminin karşılığıdır. Bu yapıları ilgili bölümlerde ayrıntısıyla göreceğiz.

Bölüm Özeti

- **Biçim belirteci** `%[bayrak][genişlik][.hassasiyet][uzunluk]harf` yapısındadır; yalnızca `%` ve dönüşüm harfi zorunludur.
- **Genişlik** en az karakter sayısıdır, değer sığmazsa kesilmez. Varsayılan hizalama sağadır; `-` sola yaslar, `0` sıfırla doldurur, `+` işareti zorlar. `%8.3f`'te nokta da genişliğe dahildir. `*` ile genişlik argümandan alınır.
- **Hassasiyet** `%f`'te ondalık basamak sayısını, `%s`'te en çok karakter sayısını belirler. `%.2f` yuvarlar, kesmez; ikilik tabanda tam saklanamayan sayılar (`2.675`) beklenenden farklı yuvarlanabilir.
- **Diğer belirteçler:** `%e` bilimsel, `%g` kısa gösterim, `%x` / `%X` / `%o` onaltılık ve sekizlik (`#` önek ekler), `%p` adres, `%zu` `size_t`, `%%` yüzde işareti.
- Belirteç ile argüman tipi uyuşmazsa sonuç tanımsız davranıştır; `-Wall` bunu uyarı olarak bildirir ve uyarılar giderilmelidir.
- `puts` metni ve sonuna `\n` yazar; `putchar` tek karakter yazar. `printf` yazdığı karakter sayısını döndürür.
- `scanf` adres ister: sayı ve karakter değişkenlerinin önüne `&` konur. `double` için `%lf`, `float` için `%f` kullanılır.
- `%d`, `%f`, `%lf` baştaki boşlukları atlar; `%c` **atlamaz**. Karakter okurken `" %c"` yazılır.
- `scanf` **dönüş değeri** başarıyla okunan öğe sayısıdır (0: okunamadı, `EOF`: girdi bitti). Girdiye güvenilemeyen her yerde denetlenmelidir. Ondalık ayırıcı nokta olmalıdır.
- Klavye girdisi **arabellekte** tutulur; `scanf` Enter'ın bıraktığı `'\n'`'i arabellekte bırakır. Bu, sonraki `%c`, `getchar` ya da `fgets` çağrısını bozar. Arabellek `while ((c = getchar()) != '\n' && c != EOF)` kalıbıyla temizlenir; `fflush(stdin)` tanımsız davranıştır.
- `getchar` sonucu `int` değişkende saklanır, çünkü `EOF` `char`'a sığmayabilir.
- `scanf("%s")` boşlukta durur ve sınırsız kullanılırsa taşar; `%19s` gibi bir sınır koyulmalıdır. Boşluklu satır okumak için `fgets(dizi, boyut, stdin)` kullanılır; satır sonunu (`'\n'`) dizide bırakır. `gets` C11'de kaldırılmıştır.
- **Girdi doğrulama:** okuma başarısını, değer aralığını denetlemek ve geçersizse yeniden istemek gerekir. `fgets` ile satırı alıp `sscanf` ile çözümlemek en öngörülebilir yöntemdir.
- Genişlik bayt sayar; UTF-8'de Türkçe harfler 2 bayttır ve sütunlu çıktıda hizayı bozar.

Alıştırmalar

1. `printf("[%6.2f]", 3.14159);` satırı ekrana ne yazdırır?

A) `[3.14]`
B) `[ 3.14]`
C) `[ 3.14]`
D) `[3.142]`
E) `[ 3.142]`

2. `double` tipinde bir değişkeni `scanf` ile okumak için hangi belirteç kullanılır?

A) `%d`
B) `%f`
C) `%lf`
D) `%c`
E) `%s`

3. `int a = 0, b = 0;` tanımlıdır. `int n = scanf("%d %d", &a, &b);` çağrısına `7 abc` yazılırsa `n` kaç olur?

A) 0
B) 1
C) 2
D) -1
E) Tanımsızdır

4. Aşağıdaki program parçasının çıktısı nedir?

```
printf("[%-5d][%05d][%+d]\n", 42, 42, 42);
```

5. Aşağıdaki programda ikinci `scanf` kullanıcıdan bir şey beklemeden neden geçer?

```
int yas = 0;
char harf = ' ';
scanf("%d", &yas);
scanf("%c", &harf);
```

A) `%c` yalnızca büyük harf okuyabilir.
B) `%d` girdinin tamamını tüketir ve arabelleği boşaltır.
C) `%c` arabellekte kalan `'\n'` karakterini okur.
D) `harf` başlangıç değeri verilmediği için.
E) `scanf` aynı anda yalnızca bir kez çağrılabilir.

6. Aşağıdaki kodda hata nedir ve nasıl düzeltilir?

```
int sayi = 0;
scanf("%d", sayi);
```

7. Aşağıdaki kodda hata nedir ve nasıl düzeltilir?

```
double boy = 0.0;
scanf("%f", &boy);
```

8. Aşağıdaki program parçasının çıktısı nedir?

```
printf("%x %X %o %#x\n", 255, 255, 255, 255);
```

9. Aşağıdakilerden hangisi arabelleği temizlemenin C standardına uygun yoludur?

   A) `fflush(stdin);`
   B) `while ((c = getchar()) != '\n' && c != EOF) { }`
   C) `clear(stdin);`
   D) `scanf("%d");`
   E) `gets(arabellek);`

10. Aşağıdaki program parçasının çıktısı nedir? (`|` karakterleri sütun sınırlarını gösterir.)

    ```
    printf("%5.1f|%-8s|%3d\n", 7.86, "ab", 12345);
    ```

11. `char satir[20];` tanımlıdır ve kullanıcı `Ece` yazıp Enter'a basar. `fgets(satir, sizeof satir, stdin);` çağrısından sonra `satir` dizisinde `Ece` harflerinden sonra ne bulunur?

    A) Hiçbir şey, yalnızca `'\0'`
    B) `'\n'` karakteri ve ardından `'\0'`
    C) Boşluk karakteri
    D) `EOF`
    E) Dizi taşar

12. Kullanıcıdan iki tamsayı okuyan, bunların toplamını, farkını, çarpımını ve bölümünü (ondalıklı, iki basamakla) aynı genişlikte sütun hâlinde yazdıran bir program yazın. Bölen 0 ise bölümü yazdırmayın, "tanımsız" yazsın. Okuma başarısını denetleyin.

13. Kullanıcıdan saniye cinsinden bir süre okuyup `SS:DD:ss` biçiminde (ör. `01:23:45`) yazdıran bir program yazın.

14. Kullanıcıdan 1 ile 10 arasında bir not alana kadar tekrar soran, geçerli notu aldıktan sonra "Not: N" yazdıran bir program yazın. `fgets` ve `sscanf` kullanın; geçersiz girdide "Geçersiz not." yazsın.

BÖLÜM 7

Koşullu İfadeler

BU BÖLÜMDE

- Bir programın akışını koşula göre dallandırmak için `if`, `if-else` ve `else if` merdivenini kurmayı
- Süslü parantez kullanmanın neden gerekli olduğunu, `if (koşul);` ve sarkan else (dangling else) tuzaklarını
- Mantıksal operatörlerle bileşik koşullar yazmayı; `=` / `==` karışıklığını, `0 < x < 10` ve `x == 1 || 2` hatalarını tanımayı
- Ondalıklı sayıları koşulda doğru karşılaştırmayı
- Üçlü operatör `?:` ile iki değerden birini seçmeyi
- `switch-case` deyimini; `break`, `default` ve düşme (fall-through) davranışını ve `case` etiketlerinin kısıtlarını
- Bu araçlarla artık yıl, üçgen türü, not hesaplama ve menü gibi gerçek karar problemlerini çözmeyi

Şimdiye kadar yazdığımız programlar yukarıdan aşağıya, her satırı sırayla çalıştırıyordu. Gerçek programlar ise karar verir: yaş 18 ya da büyükse bir şey yap, değilse başka bir şey yap; girilen şifre doğruysa devam et, yanlışsa uyar; paranın yetip yetmediğine göre işlemi onayla ya da reddet. Bu bölüm, akışı koşullara göre dallandıran dil yapılarını anlatır.

Karar vermenin hammaddesini Bölüm 5'te gördük: ilişkisel ve mantıksal operatörler, "sıfır yanlıştır, sıfırdan farklı her değer doğrudur" kuralı ve kısa devre değerlendirme. Burada bu bilgiyi kullanacağız; gerektiğinde kısa hatırlatmalar yapıp ayrıntı için Bölüm 5'e göndereceğiz. Küçük bir `=` ya da `;` programın anlamını tamamen değiştirebildiği için bölümün bir kısmı, hataların nasıl oluştuğunu ve derleyicinin onları nasıl yakaladığını göstermeye ayrılmıştır.

7.1 if Deyimi

Bir programın bazı satırları yalnızca belirli durumlarda çalışmalıdır. `if` deyimi, parantez içindeki koşul doğruysa ardındaki bloğu çalıştırır; koşul yanlışsa bloğu atlar ve program bir sonraki satırdan devam eder.

Tanım: if deyimi

Bir koşulu değerlendirip yalnızca doğruysa (sıfırdan farklıysa) kendisine bağlı deyimi ya da bloğu çalıştıran karar yapısıdır.

```
if (koşul) {
    // koşul doğruysa çalışır
}
```

Akışı şöyle düşünebiliriz:

```
          +-----------+
          |  koşul?   |
          +-----------+
           doğru |  \ yanlış
                 v   \
          +----------+ \
          |   blok   |  |
          +----------+  |
                 \     /
                  v   v
              (sonraki satır)
```

Koşul olarak sıfırdan farklı her ifade doğru sayıldığı için ilişkisel bir karşılaştırma yazmak zorunda değiliz. Önce bu kuralı bir `if` içinde görelim:

▸ dogruYanlis.c

```c
#include <stdio.h>

int main(void) {
    int sifir = 0;
    int bes = 5;
    int eksiBir = -1;

    if (bes) {
        printf("5 doğru sayılır\n");
    }
    if (eksiBir) {
        printf("-1 doğru sayılır\n");
    }
    if (sifir) {
        printf("0 doğru sayılır\n");
    }
    printf("Bitti\n");
    return 0;
}
```

ÇIKTI

```
5 doğru sayılır
-1 doğru sayılır
Bitti
```

Yalnızca `0` için blok çalışmadı. Özellikle `-1` doğrudur: "negatif sayı yanlıştır" diye düşünmek yaygın bir yanlıştır. Aynı mantıkla `if (5)` her zaman, `if (0)` hiçbir zaman çalışır. `if (sayi != 0)` yerine kısaca `if (sayi)` yazıldığını da göreceğiz; ikisi aynı anlama gelir.

Şimdi gerçek bir karşılaştırma içeren ilk programımızı yazalım. Kullanıcıdan yaşını alıp yetişkin olup olmadığını söyleyecek. `scanf` ile girdi okumayı Bölüm 6'da ayrıntılı işleyeceğiz; burada kalıbı olduğu gibi kullanıyoruz.

▸ yetiskinMi.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    printf("Yaşınız: ");
    scanf("%d", &yas);
```

```
    if (yas >= 18) {
        printf("Yetişkinsiniz.\n");
    }
    printf("Program sona erdi.\n");
    return 0;
}
```

```
ÇIKTI
Yaşınız: 25
Yetişkinsiniz.
Program sona erdi.
```

Girdi `16` olsaydı yalnızca son satır yazılırdı: `yas >= 18` yanlış olacağından blok atlanır, `Program sona erdi.` ise `if`'e bağlı olmadığı için her durumda çalışır.

7.2 if-else

Koşul yanlışken de bir şey yapmak istiyorsak `if`'in yanına `else` ekleriz. İki bloktan tam biri çalışır; hangisi çalışırsa çalışsın program ikisinden sonraki satırdan devam eder.

```
if (koşul) {
    // koşul doğruysa çalışır
} else {
    // koşul yanlışsa çalışır
}
```

Sayının tek mi çift mi olduğunu bulmak klasik bir örnektir. Kalan operatörü işe yarar: çift sayıların 2'ye bölümünden kalan 0'dır.

▸ tekCift.c

```
#include <stdio.h>

int main(void) {
    int sayi = 0;
    printf("Bir tamsayı girin: ");
    scanf("%d", &sayi);

    if (sayi % 2 == 0) {
        printf("%d çifttir.\n", sayi);
    } else {
        printf("%d tektir.\n", sayi);
    }
    return 0;
}
```

```
ÇIKTI
Bir tamsayı girin: -7
-7 tektir.
```

Negatif sayılarda dikkatli olmalıyız. Bölüm 5'te gördüğümüz gibi `%` sonucu bölünenin işaretini taşır: `-7 % 2` ifadesi `-1` verir. Bu yüzden tek sayıyı `sayi % 2 == 1` ile aramak negatif sayılarda yanılır. Yukarıdaki gibi çifti arayıp gerisini `else`'e bırakmak ya da `sayi % 2 != 0` yazmak güvenlidir.

7.3 else if Merdiveni

İkiden fazla yol olduğunda `else if` ile bir merdiven kurarız. Koşullar yukarıdan aşağıya sırayla denenir; ilk doğru çıkan koşulun bloğu çalışır ve merdivenin geri kalanı atlanır. Hiçbiri doğru değilse en sondaki `else` çalışır.

▸ harfNotu.c

```c
#include <stdio.h>

int main(void) {
    int puan = 0;
    printf("Puan (0-100): ");
    scanf("%d", &puan);

    if (puan >= 90) {
        printf("Harf notu: A\n");
    } else if (puan >= 80) {
        printf("Harf notu: B\n");
    } else if (puan >= 70) {
        printf("Harf notu: C\n");
    } else if (puan >= 60) {
        printf("Harf notu: D\n");
    } else {
        printf("Harf notu: F\n");
    }
    return 0;
}
```

ÇIKTI

```
Puan (0-100): 74
Harf notu: C
```

74 için sırayla bakıldı: `>= 90` yanlış, `>= 80` yanlış, `>= 70` doğru. `C` yazıldı ve `>= 60` koşuluna hiç bakılmadı. Bu yüzden üçüncü koşulu `puan >= 70 && puan < 80` diye yazmaya gerek yoktur: `puan >= 80` zaten yukarıda elenmiştir. Merdivenin **sırası** önemlidir; `puan >= 60` koşulunu en üste koysaydık 95 alan da `D` alırdı.

> **▲ Sık Yapılan Hata · Ayrı if'ler ile else if aynı şey değildir**
>
> Her koşulu ayrı bir `if` ile yazarsak her biri bağımsız olarak denenir ve birden fazlası çalışabilir. Yukarıdaki merdiveni `else` olmadan, dört ayrı `if` olarak yazıp `puan = 95` verirsek `A`, `B`, `C` ve `D` birlikte yazılır; çünkü 95 her koşulu sağlar. Tek sonuç istiyorsak `else if` kullanırız.

Merdiveni tasarlarken iki kural işimizi kolaylaştırır:

- Koşulları en özelden en genele ya da bir uçtan diğerine doğru, birbirini dışlayacak biçimde sıralarız.
- Hiçbir koşulun karşılamadığı durumu (bilinmeyen girdi, geçersiz değer) son `else` ile ele alırız.

7.4 Blok ve Süslü Parantez

`{ }` ile çevrili deyimler topluluğuna **blok** denir. Bir blok, tek bir deyim gibi davranır. `if`'in gövdesinde tek deyim varsa süslü parantez yazmak zorunlu değildir; ancak bu durumda koşula yalnızca **ilk deyim** bağlıdır. Girinti derleyiciyi ilgilendirmez, yalnızca okuyanı yanıltır:

▸ suslusuz.c

```c
#include <stdio.h>

int main(void) {
    int puan = 80;
    if (puan < 50)
        printf("Kaldınız.\n");
        printf("Tekrar deneyin.\n");
    return 0;
}
```

Derleyici girintinin gerçek akışla uyuşmadığını fark edip uyarır:

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra suslusuz.c -o suslusuz
suslusuz.c: In function 'main':
suslusuz.c:5:5: warning: this 'if' clause does not guard... [-Wmisleading-indentation]
    5 |     if (puan < 50)
      |     ^~
suslusuz.c:7:9: note: ...this statement, but the latter is misleadingly indented as if it were g
uarded by the 'if'
    7 |         printf("Tekrar deneyin.\n");
      |         ^~~~~~
```

Puan 80 olduğu için koşul yanlıştır ve `Kaldınız.` yazılmaz; ama ikinci `printf` `if`'e bağlı değildir, bu yüzden yine de çalışır:

ÇIKTI

```
Tekrar deneyin.
```

> **İpucu · Süslü parantezleri her zaman yazın**
>
> Gövde tek satır bile olsa `{ }` kullanmak, sonradan araya satır eklendiğinde bu hataya düşmemizi engeller. Kitaptaki örneklerde `if`, `else` ve döngü gövdeleri her zaman süslü parantezlidir.

Koşulun ardına konan noktalı virgül

İkinci tuzak, koşulun hemen ardına yanlışlıkla `;` koymaktır. C'de tek başına `;` bir **boş deyimdir** (empty statement): hiçbir şey yapmaz. Yani `if (sayi > 5);` ifadesi "koşul doğruysa hiçbir şey yap" demektir; alttaki blok ise artık `if`'in gövdesi olmayıp sıradan bir blok olarak her zaman çalışır:

▸ bosGovde.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 3;
    if (sayi > 5);
    {
        printf("Sayı 5'ten büyük\n");
    }
    return 0;
}
```

`gcc`, `-Wextra` seçeneğiyle bu alışkanlığı yakalar:

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra bosGovde.c -o bosGovde
bosGovde.c: In function 'main':
bosGovde.c:5:18: warning: suggest braces around empty body in an 'if' statement [-Wempty-body]
    5 |     if (sayi > 5);
      |                  ^
bosGovde.c:5:5: warning: this 'if' clause does not guard... [-Wmisleading-indentation]
    5 |     if (sayi > 5);
      |     ^~
bosGovde.c:6:5: note: ...this statement, but the latter is misleadingly indented as if it were g
uarded by the 'if'
    6 |     {
      |     ^
```

Program yine de derlenir ve çalışır:

```
ÇIKTI
Sayı 5'ten büyük
```

`sayi` 3 olduğu hâlde "5'ten büyük" yazıldı. Bir `if`'in beklenmedik biçimde her zaman çalıştığını gördüğümüzde ilk bakacağımız yer, koşulun kapanış parantezinden sonraki `;` işaretidir. Aynı tuzağın döngülerdeki karşılığını Bölüm 8'de göreceğiz.

7.5 İç İçe if ve Sarkan else

Bir `if`'in gövdesine başka bir `if` yazabiliriz. Buna **iç içe if** (nested if) denir. Birden fazla koşulun sırayla sağlanması gerektiğinde işe yarar: önce geçerlilik, sonra duruma göre karar gibi.

▸ **icIce.c**

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    int ehliyet = 0;
    printf("Yaş ve ehliyet (1: var, 0: yok): ");
    scanf("%d %d", &yas, &ehliyet);

    if (yas >= 18) {
        if (ehliyet) {
            printf("Araç kiralayabilirsiniz.\n");
        } else {
            printf("Önce ehliyet almalısınız.\n");
        }
    } else {
        printf("Yaş sınırı nedeniyle kiralama yapılamaz.\n");
    }
    return 0;
}
```

```
ÇIKTI
Yaş ve ehliyet (1: var, 0: yok): 20 0
Önce ehliyet almalısınız.
```

Süslü parantezler sayesinde her `else`'in hangi `if`'e ait olduğu açıktır. Parantezleri bıraktığımızda **sarkan else** (dangling else) sorunu ortaya çıkar. Kural şudur: `else`, kendisinden önceki ve henüz bir `else` almamış **en yakın** `if`'e bağlanır. Girinti bu kuralı değiştirmez.

▸ sarkanElse.c

```c
#include <stdio.h>

int main(void) {
    int a = 0;
    int b = 0;
    if (a == 1)
        if (b == 1)
            printf("A\n");
    else
        printf("B\n");
    printf("Bitti\n");
    return 0;
}
```

`else`'i dış `if`'in hizasına yazdık; "a 1 değilse B yaz" demek istedik. `gcc` şüphelenir (`-Wall` içindeki `-Wdangling-else`):

TERMİNAL

```
$ gcc -std=c17 -Wall sarkanElse.c -o sarkanElse
sarkanElse.c: In function 'main':
sarkanElse.c:6:8: warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
    6 |     if (a == 1)
      |        ^
```

Gerçekte `else` iç `if`'e (`b == 1`) bağlanır. `a` sıfır olduğu için iç `if`'e hiç girilmez ve `B` de yazılmaz:

ÇIKTI

```
Bitti
```

Niyetimiz `else`'in dış `if`'e ait olmasıysa iç `if`'i süslü parantezle kapatırız:

```c
if (a == 1) {
    if (b == 1) {
        printf("A\n");
    }
} else {
    printf("B\n");
}
```

Bu kez `else` dış `if`'e bağlanır ve aynı `a`, `b` değerleri için program `B` yazar. Kural basittir: önce girintiyi yok sayarız, `else`'i en yakın `if`'e bağlarız, sonra çalıştırırız. Süslü parantez varsa bağlama süslü parantezlere göre yapılır.

7.6 Mantıksal Operatörlerle Bileşik Koşullar

Gerçek koşulların çoğu tek bir karşılaştırmayla ifade edilemez. "Puan 0 ile 100 arasında olmalı", "kullanıcı yönetici ya da editör olmalı" gibi koşullar için Bölüm 5'te gördüğümüz `&&` (VE), `||` (VEYA) ve `!` (DEĞİL) operatörlerini kullanırız. Hatırlatalım: `&&` yalnızca iki tarafı da doğruysa, `||` en az biri doğruysa doğrudur; sonuç her zaman `int` olan 1 ya da 0'dır.

En çok kullanıldığı yer aralık denetimidir:

```
if (puan >= 0 && puan <= 100) {
    // puan geçerli ("içinde mi?")
}
if (puan < 0 || puan > 100) {
    // puan geçersiz ("dışında mı?")
}
```

Bu iki koşul birbirinin tersidir. Böyle bir koşulu tersine çevirmek için **De Morgan kuralları** kullanılır:

> **Tanım: De Morgan kuralları**
>
> `!(a && b)` ifadesi `!a || !b` ile, `!(a || b)` ifadesi `!a && !b` ile aynıdır. Olumsuzlama parantezin içine dağıtılırken `&&` ile `||` yer değiştirir, her karşılaştırma da tersine döner (`>=` yerine `<`, `<=` yerine `>`).

Yukarıdaki örnekte `!(puan >= 0 && puan <= 100)`, kurala göre `puan < 0 || puan > 100` olur. Karmaşık bir koşul bir türlü doğru çalışmıyorsa tersini yazıp De Morgan ile sadeleştirmek çoğu zaman işi kolaylaştırır.

Bölüm 5'te gördüğümüz kısa devre kuralı koşullarda güvenlik sağlar: `&&`'de sol taraf yanlışsa, `||`'de sol taraf doğruysa sağ taraf hiç hesaplanmaz. Sıfıra bölmeyi engelleyen klasik kalıbı bir `if` içinde yazalım:

▸ guvenliBolme.c

```
#include <stdio.h>

int main(void) {
    int x = 0;
    printf("x: ");
    scanf("%d", &x);

    if (x != 0 && 10 / x > 1) {
        printf("10 / x, 1'den büyük.\n");
    } else {
        printf("x sıfır ya da 10 / x, 1'den büyük değil.\n");
    }
    return 0;
}
```

```
ÇIKTI
x: 0
x sıfır ya da 10 / x, 1'den büyük değil.
```

`x` sıfırken `x != 0` yanlış olduğundan `10 / x` hiç hesaplanmadı. Sırayı ters yazsaydık (`10 / x > 1 && x != 0`) bölme önce yapılırdı; tamsayıyı sıfıra bölmek tanımsız davranıştır. Koruyucu koşul her zaman solda durur.

Sık yapılan koşul hataları

İki hata o kadar yaygındır ki ayrı ayrı bilmemiz gerekir. Her ikisi de matematikteki yazış biçimini C'ye olduğu gibi aktarmaktan doğar.

`0 < x < 10` yazımı. Matematikte "x, 0 ile 10 arasındadır" demektir. C'de derlenir ama anlamı farklıdır: ilişkisel operatörler soldan sağa işlenir, önce `0 < x` hesaplanır ve sonucu (0 ya da 1) `10` ile karşılaştırılır. 0 ya da 1 her zaman 10'dan küçük olduğundan ifade **her zaman doğrudur**:

zincir.c

```c
#include <stdio.h>

int main(void) {
    int x = 15;
    if (0 < x < 10) {
        printf("x 0 ile 10 arasında\n");
    } else {
        printf("x aralıkta değil\n");
    }
    return 0;
}
```

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra zincir.c -o zincir
zincir.c: In function 'main':
zincir.c:5:15: warning: comparison of constant '10' with boolean expression is always true [-
Wbool-compare]
    5 |     if (0 < x < 10) {
      |               ^
zincir.c:5:11: warning: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-
Wparentheses]
    5 |     if (0 < x < 10) {
      |         ~~^~~
```

ÇIKTI

```
x 0 ile 10 arasında
```

`x` 15 olduğu hâlde "arasında" yazıldı: `0 < 15` doğru, yani 1; `1 < 10` de doğru. Doğrusu iki koşulu `&&` ile bağlamaktır: `x > 0 && x < 10`. Aynı tuzak eşitlikte de vardır: `a == b == c` ifadesi "üçü de eşit" demek değildir, `(a == b) == c` demektir. Doğrusu `a == b && b == c`'dir.

`x == 1 || 2` yazımı. "x, 1 ya da 2'ye eşit mi?" sorusunu `if (x == 1 || 2)` diye yazmak çok yaygındır. İfade `(x == 1) || (2)` diye okunur ve `2` sıfırdan farklı olduğu için **her zaman doğrudur**. Doğrusu her karşılaştırmayı ayrı yazmaktır: `x == 1 || x == 2`.

> **▲ Sık Yapılan Hata · Tek karakterli & ve | mantıksal değildir**
>
> Mantıksal VE/VEYA için her zaman çift karakter (`&&`, `||`) yazılır. Tek karakterli `&` ve `|` bit düzeyinde işlem yapan başka operatörlerdir (Bölüm 16); koşullarda yanlışlıkla kullanıldıklarında kısa devre yapmazlar ve beklenmedik sonuç verebilirler.

7.7 = ile == Karışıklığı

`=` atama, `==` karşılaştırma operatörüdür. Koşula yanlışlıkla `=` yazarsak derleyici hata vermez; çünkü atama da bir ifadedir ve değeri atanan değerdir: `x = 5` ifadesinin değeri 5'tir, yani doğrudur.

atamaTuzagi.c

```c
#include <stdio.h>

int main(void) {
    int x = 3;
    if (x = 5) {
        printf("x 5'e eşit\n");
```

```
    }
    printf("x = %d\n", x);
    return 0;
}
```

`gcc` koşulun etrafına parantez önerir:

```
TERMİNAL
$ gcc -std=c17 -Wall atamaTuzagi.c -o atamaTuzagi
atamaTuzagi.c: In function 'main':
atamaTuzagi.c:5:9: warning: suggest parentheses around assignment used as truth value [-
Wparentheses]
    5 |     if (x = 5) {
      |         ^
```

Program yine de derlenir ve çalışır:

```
ÇIKTI
x 5'e eşit
x = 5
```

Kötü haber iki tanedir: `x` 3 iken "5'e eşit" yazıldı ve `x` kalıcı olarak 5 oldu. `if (x = 0)` yazsaydık değer 0 (yanlış) olacağından blok hiç çalışmaz, `x` de sıfırlanırdı. Bilerek atama yapıp sonucuna bakacaksak uyarıyı çift parantezle susturuz: `if ((x = 5))`. Bu kalıp, ilerleyen bölümlerde bir fonksiyonun döndürdüğü değeri hem değişkene atayıp hem denetlerken karşımıza çıkacak.

> **İpucu · Sabiti sola yazmak**
>
> Karşılaştırmada sabiti sola yazarsak (`if (5 == x)`) bu hatadan korunuruz: yanlışlıkla `5 = x` yazarsak 5'e değer atanamayacağı için derleyici hemen hata verir. Bu yazıma "Yoda koşulu" denir. Kitapta okunaklı olduğu için `x == 5` yazıyoruz, ama ikisini de tanımak gerekir. Asıl koruma, `-Wall` ile derlemek ve uyarıları ciddiye almaktır.

7.8 Ondalıklı Sayıları Koşulda Karşılaştırmak

Bölüm 5'te gördüğümüz gibi ondalıklı sayılar bellekte ikilik tabanda tutulur ve `0.1` gibi sayılar tam olarak yazılamaz. İki hesabın sonucu matematikte eşit olsa da bitleri aynı olmayabilir; `==` ise tam eşitlik arar. Bu yüzden `double` değerler `==` ya da `!=` ile karşılaştırılmaz. Bunun yerine farkın mutlak değerine, yani `<math.h>` içindeki `fabs` fonksiyonuna bakılır:

▸ ondalik.c

```
#include <stdio.h>
#include <math.h>

int main(void) {
    double a = 0.1 + 0.2;
    double b = 0.3;
    const double tolerans = 1e-9;

    if (a == b) {
        printf("a ve b eşit\n");
    } else {
        printf("a ve b == ile eşit değil\n");
    }
    if (fabs(a - b) < tolerans) {
```

```
        printf("a ve b yeterince yakın\n");
    }
    return 0;
}
```

```
ÇIKTI
a ve b == ile eşit değil
a ve b yeterince yakın
```

`1e-9`, "1 çarpı 10 üzeri eksi 9" yani 0,000000001 demektir. Uygun tolerans değeri probleme bağlıdır; çoğu hesapta `1e-9` ya da `1e-6` yeterlidir. `fabs` `<math.h>` içindedir ve fonksiyonları Bölüm 9'da ayrıntılı göreceğiz. Linux'ta derlerken bazen bağlayıcıya `-lm` seçeneğini eklemek gerekir.

Sorun asıl `==` ve `!=` ile çıkar. `<` ve `>` ise yalnızca iki değerin matematikte tam eşit olması gereken sınır durumunda aynı tuzağa düşürür (`0.1 + 0.2 > 0.3` doğru çıkar). `0.25 + 0.25 == 0.5` gibi ikinin kuvvetlerinden oluşan sayılar tam gösterilebildiği için doğru çıkabilir; bu bizi yanıltmamalıdır.

7.9 ! Operatörünün Önceliği

`!` tekli bir operatördür ve öncelik tablosunda karşılaştırmalardan **önce** gelir (Bölüm 5). Bu yüzden "x, 5'e eşit değil mi?" sorusunu `if (!x == 5)` diye yazmak klasik bir hatadır: ifade `(!x) == 5` diye okunur. `!x` yalnızca 0 ya da 1 verebildiği için bu koşul hiçbir `x` için doğru olamaz; `gcc` de `-Wlogical-not-parentheses` ile uyarır. Doğru yazımlar `x != 5` ya da `!(x == 5)`'tir. Aynı sebeple `!x + 1` ifadesi `!(x + 1)` değil, `(!x) + 1` demektir.

Karışık `&&` ve `||` içeren koşullarda da parantez koymak iyi bir alışkanlıktır. `&&`, `||`'dan önce işlenir; ancak `a > 3 || b > 5 && c > 5` yazdığımızda `gcc` yine de `-Wparentheses` ile uyarır. Niyeti açık yazmak hem uyarıyı susturur hem okuyanı hesap yapmaktan kurtarır:

▸ oncelikKosul.c

```c
#include <stdio.h>

int main(void) {
    int a = 5;
    int b = 3;
    int c = 2;
    printf("%d\n", a + b > 7 && c * 2 == 4);
    printf("%d\n", a > 3 || (b > 5 && c > 5));
    printf("%d\n", (a > 3 || b > 5) && c > 5);
    return 0;
}
```

```
ÇIKTI
1
1
0
```

İlk satırda önce `a + b` (8) ve `c * 2` (4) hesaplanır; `8 > 7` ve `4 == 4` doğru olduğundan sonuç 1'dir. İkinci satırda `b > 5 && c > 5` yanlıştır ama `a > 3` doğru olduğu için `||` 1 verir. Üçüncü satırda parantez önceliği değiştirir: `(a > 3 || b > 5)` doğru, ama `c > 5` yanlış olduğundan sonuç 0'dır.

7.10 Üçlü Operatör ?:

C'nin üç işlenenli tek operatörü **koşul operatörü** `?:` (üçlü operatör, ternary operator) iki değerden birini bir koşula göre seçer:

> **Tanım: Üçlü operatör**
>
> `koşul ? değerDoğruysa : değerYanlışsa` biçiminde yazılır. Koşul doğruysa ikinci işlenenin, yanlışsa üçüncü işlenenin değerini üretir; seçilmeyen işlenen hiç hesaplanmaz.

`if-else` bir **deyimdir** (statement) ve değer üretmez; `?:` ise bir **ifadedir** (expression) ve değer üretir. Bu yüzden sonucu doğrudan bir değişkene atayabilir ya da `printf`'in içine koyabiliriz:

▸ ucluOperator.c

```c
#include <stdio.h>

int main(void) {
    int a = 12;
    int b = 20;
    int sayi = 7;
    int puan = 85;

    int enBuyuk = a > b ? a : b;
    printf("En büyük: %d\n", enBuyuk);
    printf("%d sayısı %s\n", sayi, sayi % 2 == 0 ? "çift" : "tek");
    printf("Not: %s\n", puan >= 90 ? "A" : puan >= 80 ? "B" : "C");
    return 0;
}
```

```
ÇIKTI
En büyük: 20
7 sayısı tek
Not: B
```

`%s`, ekrana bir metin yazdıran yer tutucudur; metinleri Bölüm 11'de ayrıntılı işleyeceğiz. Son satırda `?:` iç içe yazıldı. `?:` sağdan sola birleştiği için `a ? b : c ? d : e`, `a ? b : (c ? d : e)` demektir. Çalışır, ama okunması hızla zorlaşır.

Kural şudur: `?:` iki değerden birini seçmek için güzeldir (en büyük, tek/çift); koşullar iç içe geçtiğinde ya da seçeneklerin içinde birden fazla işlem gerektiğinde `else if` merdivenini tercih ederiz. İki seçenek aynı tipte olmalıdır ya da birbirine dönüştürülebilmelidir; `?:` ifadesi bir değer olduğu için `if` deyimi gibi "bir şey yapma" amacıyla kullanılması okunaksız kod doğurur.

> **Üç Dilde · Koşullu yapılar üç dilde** *C · C++ · C#*
>
> C'de gördüğümüz `if`, `else if`, `else` ve `?:` aynı biçimde C++'ta ve C#'ta da vardır; yalnızca küçük farklar bulunur. C++17 ile `if` içinde değişken tanımlamak mümkün olacak (`if (int n = f(); n > 0)`), C# ise koşul olarak yalnızca `bool` kabul eder.
>
> ```c
> // C: sayı bir koşul olabilir
> int n = 5;
> if (n) { printf("sıfır değil\n"); }
> ```

```
// C++: C ile aynı; C++17'de başlatıcılı if
if (int n = f(); n > 0) { /* n yalnızca bu if içinde */ }
```

```
// C#: koşul bool olmalı; if (n) derlenmez
int n = 5;
if (n != 0) { Console.WriteLine("sıfır değil"); }
```

C#'ta `if (x = 5)` yazımı da derlenmez, çünkü atamanın sonucu `int`'tir, `bool` değildir. Bu, bu bölümde gördüğümüz `=` / `==` tuzağını dil düzeyinde ortadan kaldırır.

7.11 switch Deyimi

Bir değişkeni ya da ifadeyi birkaç sabit değerle karşılaştırıyorsak uzun bir `else if` merdiveni yazmak yerine `switch` deyimini kullanabiliriz. Menü seçimi, komut kodu, ay numarası gibi "şu değerse şunu yap" biçimindeki durumlar için okunaklı bir alternatiftir.

```
switch (ifade) {
    case sabit1:
        // ifade == sabit1 ise buradan başlar
        break;
    case sabit2:
        // ifade == sabit2 ise buradan başlar
        break;
    default:
        // hiçbir case uymazsa (isteğe bağlı)
}
```

`ifade` hesaplanır; değeri hangi `case` etiketine eşitse program o etiketin altından çalışmaya başlar. `break` görülünce `switch`'ten çıkılır. Hiçbir etiket uymazsa `default`'tan devam edilir. Bir örnekle görelim: girilen harfin sesli olup olmadığını bulalım.

▸ sesliHarf.c

```
#include <stdio.h>

int main(void) {
    char harf = ' ';
    printf("Bir harf girin (a-z): ");
    scanf(" %c", &harf);

    switch (harf) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            printf("'%c' bir sesli harftir.\n", harf);
            break;
        default:
            printf("'%c' sesli harf değil.\n", harf);
    }
    return 0;
}
```

ÇIKTI

```
Bir harf girin (a-z): e
'e' bir sesli harftir.
```

`scanf` içindeki `%c`'den önce yer alan boşluk, önceki girdiden kalan boşluk ve satır sonu karakterlerini atlar; bunu Bölüm 6'da ayrıntılı göreceğiz. İlk satırdaki beş etiket yan yana yazılmıştır ve aralarında deyim yoktur: hangisi eşleşirse aynı `printf`'e düşülür. Bu, aşağıda göreceğimiz kazara düşmeden farklı olarak bilerek yapılan bir gruplamadır. Örnek yalnızca İngiliz alfabesindeki harfleri ele alır; `ü`, `ö` gibi Türkçe karakterler UTF-8'de birden fazla bayt tuttuğundan tek bir `char`'a sığmaz ve metin işlemeyi anlattığımız Bölüm 11'de ele alınacaktır.

switch'in kuralları

- `switch (ifade)` içindeki ifade **tamsayı tipinde** olmalıdır: `int`, `char`, `short`, `long` ve ileride göreceğimiz `enum`. `double` ya da `float` olamaz.
- `case` etiketleri **derleme zamanında bilinen sabitler** olmalıdır: `3`, `'a'`, `2 + 3` ya da `#define` ile tanımlı bir ad. Değişken olmaz. Aynı değer iki `case`'de yazılamaz.
- `default` isteğe bağlıdır ve herhangi bir yerde olabilir; sona koymak iyi bir alışkanlıktır.
- `switch` yalnızca **eşitlik** denetler. "Puan 70'ten büyükse" gibi aralıklar için `else if` merdiveni gerekir.

break unutulursa: düşme

`break` yazmazsak program etiketi geçince durmaz; bir alttaki `case`'in kodunu da çalıştırmaya devam eder. Buna **düşme** (fall-through) denir ve `switch`'in en bilinen tuzağıdır.

▸ breakYok.c

```c
#include <stdio.h>

int main(void) {
    int secim = 1;
    switch (secim) {
        case 1: printf("Bir\n");
        case 2: printf("İki\n");
        default: printf("Başka\n");
    }
    return 0;
}
```

`gcc`, `-Wextra` içindeki `-Wimplicit-fallthrough` ile bunu yakalar:

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra breakYok.c -o breakYok
breakYok.c: In function 'main':
breakYok.c:6:17: warning: this statement may fall through [-Wimplicit-fallthrough=]
    6 |         case 1: printf("Bir\n");
      |                 ^~~~~~~~~~~~~~~
breakYok.c:7:9: note: here
    7 |         case 2: printf("İki\n");
      |         ^~~~
breakYok.c:7:17: warning: this statement may fall through [-Wimplicit-fallthrough=]
    7 |         case 2: printf("İki\n");
      |                 ^~~~~~~~~~~~~~~
breakYok.c:8:9: note: here
    8 |         default: printf("Başka\n");
      |         ^~~~~~~
```

```
ÇIKTI
Bir
İki
Başka
```

`secim` 1 olduğu için `case 1`'in altından başlandı ve `break` olmadığı için `case 2` ile `default`'taki satırlar da çalıştı. Etiket yalnızca **nereden başlanacağını** söyler; nerede duracağını `break` söyler. Son bloğa `break` gerekmez.

> ◆ **Kritik Nokta · switch'i adım adım izlemek**
>
> Bir `switch`'in çıktısını bulmak için eşleşen etiketi buluruz, oradan aşağı doğru ilk `break`'e kadar (ya da `switch`'in sonuna kadar) her satırı çalıştırırız. Aradaki etiketler yalnızca giriş noktasıdır, kendileri bir şey yapmaz. `default` en üstte olsa bile yalnızca hiçbir etiket uymadığında devreye girer.

Düşme bazen işe yarar. Birden fazla durumun ortak bir işi paylaştığı ama her birinin kendine ait bir adımı da olduğu durumlarda bilerek bırakılabilir. Böyle durumlarda niyeti belli etmek gerekir: C23, bilerek düşmeyi `[[fallthrough]];` ile işaretlemeye izin verir (`gcc`'ye özgü eski yazım `__attribute__((fallthrough));`'tur). Araya hiç deyim girmeyen boş etiketler (`case 1: case 2:`) uyarı üretmez.

▸ ayAdi.c

```c
#include <stdio.h>

int main(void) {
    int ay = 0;
    printf("Ay numarası (1-12): ");
    scanf("%d", &ay);

    switch (ay) {
        case 12: case 1: case 2:
            printf("Kış\n");
            break;
        case 3: case 4: case 5:
            printf("İlkbahar\n");
            break;
        case 6: case 7: case 8:
            printf("Yaz\n");
            break;
        case 9: case 10: case 11:
            printf("Sonbahar\n");
            break;
        default:
            printf("Geçersiz ay!\n");
    }
    return 0;
}
```

```
ÇIKTI
Ay numarası (1-12): 7
Yaz
```

case etiketinde neler yazılamaz

`case` etiketi derleme zamanında bilinen bir tamsayı sabiti olmalıdır. Üç yaygın hatayı birlikte görelim:

caseHatalari.c

```c
#include <stdio.h>

int main(void) {
    int sinir = 5;
    int n = 3;
    switch (n) {
        case sinir: printf("a\n"); break;
        case 2.5: printf("b\n"); break;
        case "evet": printf("c\n"); break;
    }
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
caseHatalari.c: In function 'main':
caseHatalari.c:7:9: error: case label does not reduce to an integer constant
    7 |         case sinir: printf("a\n"); break;
      |         ^~~~
caseHatalari.c:8:9: error: case label does not reduce to an integer constant
    8 |         case 2.5: printf("b\n"); break;
      |         ^~~~
caseHatalari.c:9:9: error: pointers are not permitted as case values
    9 |         case "evet": printf("c\n"); break;
      |         ^~~~
```

- `case sinir:` değişken olduğu için yazılamaz. `#define` ile tanımlı bir ad ya da `2 + 3` gibi sabit ifade yazabiliriz. C'de `const int sinir = 5;` bile `case` için sabit sayılmaz (C++'ta sayılır).
- `case 2.5:` ondalıklı sayı tamsayı sabiti değildir. `switch (d)` ile `double` bir ifadeyi seçmeye kalkarsak `gcc` `switch quantity not an integer` hatası verir.
- `case "evet":` metin yazılamaz; metinleri `==` ile de karşılaştıramayız. Doğru yol Bölüm 11'de göreceğimiz `strcmp` fonksiyonudur.

`gcc`'nin `case 1 ... 5:` biçiminde aralık yazan bir eklentisi vardır ama standart C değildir; taşınabilir kodda kullanmayız. `case 1, 2:` gibi virgüllü yazım da yoktur; bunun yerine `case 1: case 2:` yazılır.

if-else mi, switch mi?

İkisi de dallanma sağlar; seçimi koşulun biçimi belirler:

Durum	Uygun yapı
Aralık ya da karmaşık koşul (`puan >= 70`)	`else if` merdiveni
Tamsayı/karakter değişkenin belirli sabitlerle eşitliği	`switch`
Metin karşılaştırması	`if` + `strcmp` (Bölüm 11)
Ondalıklı sayı karşılaştırması	`if`
İki seçenekten birini değer olarak seçmek	`?:`

> **Üç Dilde · switch üç dilde** *C · C++ · C#*
>
> C ve C++'ta `switch`'in sözdizimi aynıdır ve her ikisinde de `break` unutulursa düşme olur. C++17 bilerek düşmeyi `[[fallthrough]];` ile işaretlemek için bir özellik getirmiştir. C#'ta ise düşme **yasaktır**: boş olmayan

her `case` `break`, `return` ya da benzeri bir çıkışla bitmek zorundadır; aksi halde program derlenmez. Ayrıca C#'ta `switch` `string` değerlerle de çalışır:

```
/* C: metin için switch olmaz; strcmp ile if-else gerekir */
if (strcmp(komut, "evet") == 0) { /* ... */ }
```

```
// C#: string ile switch, düşme yok
string komut = "evet";
switch (komut)
{
    case "evet":
    case "e":                // boş etiketler gruplanabilir
        Console.WriteLine("Onaylandı");
        break;               // break (ya da return) zorunlu
    default:
        Console.WriteLine("Reddedildi");
        break;
}
```

C#'ın daha yeni sürümlerinde `switch` bir değer üreten ifade olarak da yazılabilir; bunu C# kısmında göreceğiz.

7.12 Örnek Programlar

Bu bölümde öğrendiklerimizi birleştiren dört program yazalım. Her biri farklı bir karar kalıbını gösteriyor: birleşik koşul, `else if` ile sınıflandırma, `if` ile `switch`'in birlikte kullanımı ve `switch` tabanlı menü.

Artık yıl

Bir yıl 4'e bölünüyorsa artık yıldır; ama 100'e de bölünüyorsa artık değildir; 400'e de bölünüyorsa yine artıktır. Örneğin 2024 artık yıldır, 1900 değildir (100'e bölünür, 400'e bölünmez), 2000 artık yıldır (400'e bölünür). Kural tek bir koşula çevrilir:

artikYil.c

```c
#include <stdio.h>

int main(void) {
    int yil = 0;
    printf("Yıl: ");
    scanf("%d", &yil);

    if ((yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0) {
        printf("%d artık yıldır.\n", yil);
    } else {
        printf("%d artık yıl değildir.\n", yil);
    }
    return 0;
}
```

ÇIKTI

```
Yıl: 1900
1900 artık yıl değildir.
```

Parantezler `&&` ile `||`'nin nasıl gruplanacağını açıkça gösterir. Aynı kuralı iç içe `if` ile de yazabilirdik; tek koşul hem daha kısa hem de kuralı bire bir yansıttığı için daha okunaklıdır. Programı 2024, 2000 ve 2023 ile de deneyerek sonucu kendimiz doğrulayabiliriz.

Üçgenin geçerliliği ve türü

Üç kenar uzunluğu bir üçgen oluşturur ancak ve ancak her kenar diğer ikisinin toplamından küçükse (üçgen eşitsizliği). Geçerliyse üç kenar eşitse eşkenar, ikisi eşitse ikizkenar, hiçbiri eşit değilse çeşitkenar üçgen elde ederiz:

▸ ucgen.c

```c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0, c = 0;
    printf("Üç kenar uzunluğu: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a <= 0 || b <= 0 || c <= 0) {
        printf("Kenarlar pozitif olmalıdır.\n");
    } else if (a + b <= c || a + c <= b || b + c <= a) {
        printf("Bu kenarlarla üçgen olmaz.\n");
    } else if (a == b && b == c) {
        printf("Eşkenar üçgen\n");
    } else if (a == b || a == c || b == c) {
        printf("İkizkenar üçgen\n");
    } else {
        printf("Çeşitkenar üçgen\n");
    }
    return 0;
}
```

```
ÇIKTI
Üç kenar uzunluğu: 3 4 5
Çeşitkenar üçgen
```

Sıra burada da önemlidir. Önce geçersiz durumlar elenir, böylece sonraki dallar yalnızca gerçek üçgenler için çalışır. Eşkenar denetimi ikizkenardan önce gelir; çünkü eşkenar üçgenin de iki kenarı eşittir ve sıra ters olsaydı her zaman "ikizkenar" yazılırdı. `a == b && b == c` yazımına dikkat edelim: `a == b == c` olmazdı.

Ağırlıklı ortalama

İki notun ağırlıklı ortalamasını hesaplayıp geçme durumuna karar veren bir program yazalım. Notlardan biri 0–100 dışındaysa program uyarıp bitsin. Aksi halde ortalama `ödev * 0.4 + proje * 0.6` olarak hesaplanır; ortalama en az 50 ve proje notu en az 45 ise "Geçtiniz." yazılır.

▸ notHesabi.c

```c
#include <stdio.h>

int main(void) {
    int odev = 0, proje = 0;
    double ortalama = 0.0;
    printf("Ödev ve proje notu: ");
    scanf("%d %d", &odev, &proje);

    if (odev < 0 || odev > 100 || proje < 0 || proje > 100) {
```

```
        printf("Geçersiz not!\n");
        return 0;
    }

    ortalama = odev * 0.4 + proje * 0.6;
    printf("Ortalama: %.2f\n", ortalama);

    if (ortalama >= 50 && proje >= 45) {
        printf("Geçtiniz.\n");
    } else {
        printf("Kaldınız.\n");
    }
    return 0;
}
```

```
ÇIKTI
Ödev ve proje notu: 90 40
Ortalama: 60.00
Kaldınız.
```

Önce geçersizlik `||` ile "dışında mı?" sorusuyla denetlenir ve `return 0;` ile program erken bitirilir. Geçerli girdi için ortalama hesaplanır ve iki koşul `&&` ile bağlanır. Girdi `90 40` olduğunda ortalama 60,00'dır, ama proje notu 45'in altında olduğu için sonuç "Kaldınız." olur. Erken `return`, iç içe `if`'lerin derinleşmesini önleyen yaygın bir kalıptır: geçersiz durumu en başta eleyip asıl işi girintisiz yazarız.

Basit hesap makinesi ve menü

Son program iki sayıyı ve aralarındaki işlemi okur, işlemi `switch` ile seçer. `scanf` içindeki `%c`'den önceki boşluk, sayı ile işlem arasındaki boşlukları atlar (Bölüm 6). Bölme kolunda `switch` içinde bir `if` kullanılmıştır; ikisi birlikte kullanılabilir.

▸ hesapMakinesi.c

```c
#include <stdio.h>

int main(void) {
    double sayi1 = 0.0, sayi2 = 0.0;
    double sonuc = 0.0;
    char islem = ' ';
    printf("İşlem (ör. 12.5 * 4): ");
    scanf("%lf %c %lf", &sayi1, &islem, &sayi2);

    switch (islem) {
        case '+': sonuc = sayi1 + sayi2; break;
        case '-': sonuc = sayi1 - sayi2; break;
        case '*': sonuc = sayi1 * sayi2; break;
        case '/':
            if (sayi2 == 0) {
                printf("Sıfıra bölünemez!\n");
                return 0;
            }
            sonuc = sayi1 / sayi2;
            break;
        default:
            printf("Bilinmeyen işlem: %c\n", islem);
            return 0;
    }
```

```
    printf("%.2f %c %.2f = %.2f\n", sayi1, islem, sayi2, sonuc);
    return 0;
}
```

```
ÇIKTI
İşlem (ör. 12.5 * 4): 12.5 * 4
12.50 * 4.00 = 50.00
```

Bölmeden önce `sayi2 == 0` denetlenir. Ondalıklı sayı için `==` kullanmak normalde sakıncalıdır; burada kullanıcının girdiği `0` ifadesi tam sıfır olarak okunduğundan karşılaştırma güvenlidir. `default` kolu bilinmeyen işleme karşı korur. `char` ile `switch` kullanırken etiketlerin tek tırnaklı olduğuna dikkat edelim: `'+'`, `"+"` değildir.

Menü mantığı da aynı kalıba dayanır. Bir kafede seçilen içeceğin fiyatını ve adetini alıp toplam tutarı hesaplayalım:

▸ menu.c

```c
#include <stdio.h>

int main(void) {
    int secim = 0;
    int adet = 0;
    double fiyat = 0.0;
    const char *ad = "";

    printf("1) Çay    15 TL\n");
    printf("2) Kahve  40 TL\n");
    printf("3) Su     10 TL\n");
    printf("Seçiminiz ve adet: ");
    scanf("%d %d", &secim, &adet);

    switch (secim) {
        case 1: ad = "Çay";   fiyat = 15.0; break;
        case 2: ad = "Kahve"; fiyat = 40.0; break;
        case 3: ad = "Su";    fiyat = 10.0; break;
        default:
            printf("Geçersiz seçim!\n");
            return 0;
    }

    if (adet <= 0) {
        printf("Adet pozitif olmalıdır.\n");
    } else {
        printf("%d x %s = %.2f TL\n", adet, ad, adet * fiyat);
    }
    return 0;
}
```

```
ÇIKTI
1) Çay    15 TL
2) Kahve  40 TL
3) Su     10 TL
Seçiminiz ve adet: 2 3
3 x Kahve = 120.00 TL
```

`const char *ad` bir metne işaret eden değişkendir; bunu Bölüm 11 ile Bölüm 12'de ayrıntılı öğreneceğiz. Burada yalnızca metni tutan bir değişken olarak kullanıyoruz. Menüyü kullanıcı geçerli bir seçim yapana kadar yeniden göstermek için döngü gerekir; bu da Bölüm 8'in konusudur.

Bölüm Özeti

- `if (koşul) { ... }` koşul doğruysa bloğu çalıştırır; `else` yanlış durumunu, `else if` ek yolları ekler. Koşullar yukarıdan aşağıya denenir, ilk doğru olanın bloğu çalışır ve merdivenin kalanı atlanır.
- C'de koşul olarak sıfır yanlış, sıfırdan farklı her değer (`-1` ve `0.5` dahil) doğrudur. İlişkisel ve mantıksal operatörler `int` olarak 1 ya da 0 üretir.
- Ayrı `if`'ler bağımsız denenir ve birden fazlası çalışabilir; yalnızca biri çalışsın istiyorsak `else if` kullanırız.
- Gövde tek deyim bile olsa `{ }` yazarız. Süslü parantezsiz `if` yalnızca ilk deyimi koşula bağlar; girinti bu bağı değiştirmez.
- `if (koşul);` boş gövdedir: koşul ne olursa olsun alttaki blok çalışır.
- `else`, kendisinden önceki henüz `else` almamış en yakın `if`'e bağlanır (sarkan else). Niyeti süslü parantezle belirtiriz.
- Aralık denetimi `x > 0 && x < 10` biçiminde yazılır. `0 < x < 10` ve `x == 1 || 2` derlenir ama her zaman doğrudur.
- `=` atama, `==` karşılaştırmadır. `if (x = 5)` hata vermez; `x`'i değiştirir ve her zaman doğru olur. `-Wall` bunu yakalar.
- Kısa devre sayesinde `x != 0 && 10 / x > 1` güvenlidir; sağ taraftaki yan etkiler (`++b`) atlanabilir.
- Ondalıklı sayılar `==` ile değil, `fabs(a - b) < tolerans` ile karşılaştırılır.
- `!x == 5` ifadesi `(!x) == 5` demektir; karışık `&&`/`||` ifadelerinde parantez kullanırız.
- `koşul ? a : b` bir değer üretir; iki değerden birini seçmek için uygundur, iç içe yazılması okunurluğu bozar.
- `switch` tamsayı ya da `char` ifadeyi sabit `case` etiketleriyle karşılaştırır. `case`'de değişken, `double` ve metin olmaz.
- `break` yoksa sonraki `case`'ler de çalışır (düşme). Etiket yalnızca giriş noktasıdır; araya deyim girmeyen etiketler bilerek gruplama için kullanılır.
- C#'ta `switch`'te düşme yasaktır ve `string` ile `switch` yapılabilir; koşul her zaman `bool` olmalıdır.

Alıştırmalar

1. `x` değişkeninin 10 ile 20 arasında olduğunu (uç değerler dışarıda) doğru biçimde denetleyen ifade hangisidir?

   A) `10 < x < 20`
   B) `x > 10 || x < 20`
   C) `x > 10 && x < 20`
   D) `x >= 10 && x <= 20`
   E) `x > 10 && x <= 20`

2. Aşağıdaki ifadelerden hangisinde `x` değeri 0 iken `10 / x` işlemi hiçbir zaman çalıştırılmaz?

   A) `10 / x > 1 && x != 0`
   B) `x == 0 && 10 / x > 1`
   C) `x != 0 && 10 / x > 1`

D) `x != 0 || 10 / x > 1`
E) `10 / x > 1 || x == 0`

3. Programda `#define MAKS 10` ve `int sinir = 10;` tanımlıdır. Aşağıdakilerden hangisi bir `switch` içinde `case` etiketi olarak **kullanılamaz**?

A) `case 'x':`
B) `case MAKS + 1:`
C) `case 2 * 3:`
D) `case sinir + 1:`
E) `case -1:`

4. Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>
int main(void) {
    int n = 25;
    printf("%d\n", n > 10 ? n > 20 ? 3 : 2 : 1);
    return 0;
}
```

A) 1
B) 2
C) 3
D) 25
E) Derleme hatası verir

5. Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>
int main(void) {
    int x = 0;
    if (x = 7) { printf("A\n"); } else { printf("B\n"); }
    printf("%d\n", x);
    return 0;
}
```

A) `A` ve alt satırda `0`
B) `A` ve alt satırda `7`
C) `B` ve alt satırda `7`
D) `B` ve alt satırda `0`
E) Derleme hatası verir

6. Aşağıdaki programın çıktısı nedir? Girintiye değil, `else`'in gerçekte hangi `if`'e bağlandığına bakınız.

```c
#include <stdio.h>
int main(void) {
    int x = 2;
    int y = 9;
    if (x > 3)
        if (y > 5)
            printf("A\n");
    else
        printf("B\n");
```

```
    printf("C\n");
    return 0;
}
```

7. Aşağıdaki programın çıktısı nedir? Hangi sağ tarafların çalıştırılmadığını işaretleyerek ilerleyiniz.

```
#include <stdio.h>
int main(void) {
    int a = 0, b = 0, c = 0;
    if (a > 5 && ++b > 0) { c = 10; }
    if (a < 5 || ++b > 0) { c += 1; }
    if (a == 0 && ++b > 0) { c += 8; }
    if (!b) { c += 2; }
    if (a) { c += 4; }
    printf("%d %d %d\n", a, b, c);
    return 0;
}
```

8. Aşağıdaki programın çıktısı nedir?

```
#include <stdio.h>
int main(void) {
    int kod = 4;
    int toplam = 0;
    switch (kod) {
        case 1: toplam += 1;
        case 2: toplam += 10; break;
        case 4: toplam += 1000;
        case 3: toplam += 100;
        default: toplam += 5;
    }
    printf("%d\n", toplam);
    return 0;
}
```

9. Aşağıdaki kod yalnızca puan 50 ya da daha büyükse "Geçtiniz" yazdırmak için yazılmıştır. `puan` 30 olduğunda ne olur ve hata nerededir? Düzeltilmiş hâlini yazınız.

```
if (puan >= 50);
{
    printf("Geçtiniz\n");
}
```

10. Aşağıdaki kod `x` değeri 1 ya da 2 ise mesaj yazdırmak için yazılmıştır. `x` 7 iken ne olur ve neden? Düzeltiniz.

```
if (x == 1 || 2) {
    printf("x, 1 ya da 2\n");
}
```

11. Aşağıdaki program neden derlenmez? İki ayrı sorunu belirtiniz ve düzeltmenin yolunu söyleyiniz.

```
#include <stdio.h>
int main(void) {
    double sicaklik = 20.5;
    switch (sicaklik) {
        case 20.5: printf("ılık\n"); break;
```

```
        default: printf("başka\n");
    }
    return 0;
}
```

12. Aşağıdaki ifadelerden hangisi doğrudur?

    A) C#'ta `switch` `string` değerlerle kullanılamaz.

    B) C#'ta içinde deyim olan bir `case`, `break` (ya da benzeri bir çıkış) olmadan biterse program derlenmez.

    C) C'de `case` etiketi bir metin sabiti olabilir.

    D) C#'ta `int x` için `if (x = 5)` yazmak derlenir ve her zaman doğrudur.

    E) C'de `switch` yalnızca `int` ifadeyle çalışır, `char` ile çalışmaz.

13. Klavyeden bir tamsayı okuyup "pozitif", "negatif" ya da "sıfır" yazdıran `isaret` programını yazınız.

14. Ay numarasını (1-12) ve yılı okuyup o ayın kaç gün çektiğini yazdıran `gunSayisi` programını yazınız. `switch` kullanınız; aynı sonucu veren ayları `break` yazmadan gruplayarak kod tekrarını azaltınız. Şubat için bu bölümdeki artık yıl kuralını kullanınız. Ay geçersizse `Geçersiz ay!` yazdırınız.

```
ÇIKTI
Ay ve yıl: 2 2024
2/2024: 29 gün
```

15. Kilo (kg, `double`) ve boy (metre, `double`) değerlerini okuyup vücut kitle indeksini (kilo / boy²) hesaplayan `bki` programını yazınız. İndeksi iki ondalıkla yazdırıp şu sınıflandırmayı `else if` merdiveniyle yapınız: 18,5'ten küçükse "Zayıf", 25'ten küçükse "Normal", 30'dan küçükse "Fazla kilolu", değilse "Obez".

16. İkinci derece denklemin (`a x² + b x + c = 0`) katsayılarını okuyup köklerini bulan `denklem` programını yazınız. `a` sıfırsa "Bu bir ikinci derece denklem değil." yazdırıp bitiriniz. Diskriminant `b² - 4ac` negatifse "Gerçel kök yok", sıfırsa tek kökü, pozitifse iki kökü yazdırınız. Kök için `<math.h>` içindeki `sqrt` fonksiyonunu kullanabilirsiniz.

BÖLÜM 8

Döngüler

BU BÖLÜMDE

- `while`, `do-while` ve `for` döngülerinin işleyişini ve hangisinin ne zaman seçileceğini
- `for` başlığının üç kısmını, `for` ile `while` arasındaki dönüşümü ve döngü değişkeninin kapsamını
- Sayaç, toplayıcı ve bayrak (flag) değişkenlerinin rolünü ve ilk değerlerini
- `break`, `continue`, sonsuz döngüleri ve `for (...);` noktalı virgül tuzağını
- İç içe döngülerle yıldız ve sayı desenleri çizmeyi
- İzleme tablosu (trace table) ile bir döngüyü adım adım çözmeyi ve off-by-one hatalarını yakalamayı
- Klasik algoritmaları: faktöriyel, basamak işlemleri, asal sayı, EBOB-EKOK, Fibonacci, mükemmel sayı
- Girdi ile çalışan döngüleri: sentinel, girdi doğrulama, girdi bitene kadar okuma ve `goto`'nun yeri

Bilgisayarların insanlardan üstün olduğu bir iş vardır: aynı şeyi sıkılmadan, hata yapmadan, saniyede milyonlarca kez yapmak. 1'den 1000'e kadar sayıları ekrana yazdırmak için bin tane `printf` satırı yazmayız; aynı satırı bir kez yazar ve bilgisayara "bunu bin kez tekrarla" deriz. Bu işi yapan yapıya **döngü** (loop) denir.

Önceki bölümde koşullarla programın akışına karar verme yeteneği kazandık. Döngüler, koşulları tekrarla birleştirir ve programlamanın asıl gücünü açar: veri toplama, arama, sayma, desen üretme, hesaplamaları sonuca varana kadar sürdürme. Bu bölümde C'nin üç döngüsünü, onlarla yazılan klasik algoritmaları ve döngülerde en sık yapılan hataları göreceğiz.

8.1 while Döngüsü

En yalın döngü `while`'dır:

```
while (koşul) {
    // koşul doğru olduğu sürece tekrarlanan gövde
}
```

İşleyiş şöyledir: koşul denetlenir; doğruysa gövde bir kez çalışır; sonra başa dönülüp koşul yeniden denetlenir. Koşul yanlış olduğunda döngü biter ve akış döngüden sonraki satırdan sürer. Hatırlayacağınız gibi C'de sıfırdan farklı her değer doğru, sıfır yanlış sayılır (Bölüm 5).

> **Tanım: Döngü**
>
> Bir kod bloğunun (döngü gövdesinin) bir koşula bağlı olarak art arda birden çok kez çalıştırılmasını sağlayan denetim yapısıdır. Gövdenin her çalışmasına bir **yineleme** (iteration) ya da **tur** denir.

▸ while1.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 1;
    while (sayi <= 5) {
        printf("%d ", sayi);
        sayi++;
    }
    printf("\nDöngü bitti, sayi = %d\n", sayi);
    return 0;
}
```

ÇIKTI

```
1 2 3 4 5
Döngü bitti, sayi = 6
```

Her `while` döngüsünde üç parça aranır:

1. **İlk değer**: koşulda kullanılan değişkenin başlangıç değeri (`sayi = 1`).
2. **Koşul**: döngünün sürüp sürmeyeceğini belirleyen ifade (`sayi <= 5`).
3. **İlerleme**: koşulu er geç yanlış yapacak değişiklik (`sayi++`).

Döngü bittiğinde `sayi` 5 değil 6'dır: döngü, koşulu bozan ilk değerde durur. Üçüncü parçayı, yani ilerlemeyi unutursak koşul hiç bozulmaz ve program sonsuza kadar döner. Bir döngü yazarken kendimize hep şunu soralım: bu döngü hangi satırda biter?

Koşul en baştan yanlışsa gövde hiç çalışmaz:

```c
int n = 10;
while (n < 5) {
    printf("Bu satır hiç yazılmaz\n");
}
```

Bu özellik `while`'ı "sıfır kez dönebilen" döngü yapar. Aşağıdaki program bir sayının kaç kez ikiye bölünerek 1'e indirilebileceğini sayar; sayı 1 ise döngü hiç çalışmaz ve sonuç 0 çıkar.

▸ ikiyeBol.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 100;
    int adim = 0;

    while (sayi > 1) {
        sayi = sayi / 2;
        adim++;
    }
    printf("100 sayısı %d adımda 1'e indi\n", adim);
    return 0;
}
```

ÇIKTI

```
100 sayısı 6 adımda 1'e indi
```

Sayının sırasıyla 50, 25, 12, 6, 3, 1 değerlerini aldığına dikkat edelim: tamsayı bölmesi küsuratı atar (Bölüm 5). Adım sayısı aslında 100'ün ikilik tabandaki logaritmasının taban değeridir; bu tür "yarıya bölme" döngüleri, ilerideki arama algoritmalarında karşımıza çıkacak.

8.2 do-while Döngüsü

`while`'da koşul gövdeden önce denetlenir. `do-while`'da ise koşul gövdeden **sonra** denetlenir; dolayısıyla gövde koşul ne olursa olsun en az bir kez çalışır:

```
do {
    // en az bir kez çalışan gövde
} while (koşul);   // bu noktalı virgül ZORUNLUDUR
```

Koşul baştan yanlışsa `while` gövdeyi 0 kez, `do-while` 1 kez çalıştırır. Bu davranış, "önce yap, sonra sor" mantığındaki işler için uygundur. Klasik örnek menüdür: menü ekrana en az bir kez gelmeli, kullanıcının seçimine göre tekrar edilmelidir.

▸ menu.c

```c
#include <stdio.h>

int main(void) {
    int secim = 0;

    do {
        printf("1) Merhaba  0) Çıkış: ");
        scanf("%d", &secim);
        if (secim == 1) {
            printf("Merhaba!\n");
        }
    } while (secim != 0);

    printf("Program bitti.\n");
    return 0;
}
```

```
ÇIKTI
1) Merhaba  0) Çıkış: 1
Merhaba!
1) Merhaba  0) Çıkış: 1
Merhaba!
1) Merhaba  0) Çıkış: 0
Program bitti.
```

Seçim değeri gövdenin içinde okunduğu için `secim`'in koşulda kullanılabilmesi ancak gövde çalıştıktan sonra mümkündür. `secim`'e baştan 0 vermiş olmamız yalnızca ilk değer verme alışkanlığımızdandır; `do-while` olduğu için bu değer hiç denetlenmez.

◆ Kritik Nokta · Hangi döngüyü seçmeli?

- Kaç kez döneceği **önceden belliyse** (1'den 10'a, bir dizinin her elemanı için): `for`.
- Tur sayısı bir koşula bağlıysa ve **hiç dönmeyebilirse**: `while`.
- Gövde **en az bir kez** çalışmalıysa (menü, girdi doğrulama): `do-while`.

Üçü de birbirinin yerine yazılabilir; seçim, kodu okuyana niyetimizi anlatmak içindir.

▲ Sık Yapılan Hata · do-while sonundaki noktalı virgül

`do { ... } while (koşul)` yazıp sonuna `;` koymazsak derleyici hata verir. `while` ve `for` başlıklarından sonra ise noktalı virgül konmaz (aşağıda göreceğimiz tuzak). Üç döngüde noktalı virgülün yerini kesin öğrenelim: yalnızca `do-while`'ın en sonunda.

8.3 for Döngüsü

Sayaçla yürüyen işlerde ilk değer, koşul ve ilerleme `while`'da üç ayrı yere dağılır. `for` bunları tek satırda toplar:

```
for (başlangıç; koşul; adım) {
    // gövde
}
```

İşleyiş sırası şöyledir:

1. **Başlangıç** yalnızca bir kez, döngüye girerken çalışır.
2. **Koşul** denetlenir. Yanlışsa döngü biter.
3. **Gövde** çalışır.
4. **Adım** çalışır ve 2. maddeye dönülür.

▸ forTemel.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    printf("\n");

    for (int i = 10; i >= 0; i -= 5) {
        printf("%d ", i);
    }
    printf("\n");

    for (int i = 1; i <= 100; i *= 2) {
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

```
ÇIKTI
1 2 3 4 5
10 5 0
1 2 4 8 16 32 64
```

Adım kısmı `i++` ile sınırlı değildir: `i--` geriye sayar, `i += 5` beşer atlar, `i *= 2` her turda iki katına çıkar. Üç kısmın herhangi biri boş bırakılabilir (`for (;;)` gibi), ancak aradaki iki noktalı virgül her durumda yazılır.

Virgül operatörü sayesinde birden çok değişken aynı başlıkta yönetilebilir:

▸ virgulFor.c

```c
#include <stdio.h>

int main(void) {
    for (int bas = 0, son = 5; bas < son; bas++, son--) {
        printf("bas=%d son=%d\n", bas, son);
    }
    return 0;
}
```

```
ÇIKTI
bas=0 son=5
bas=1 son=4
bas=2 son=3
```

Başlangıçtaki `int bas = 0, son = 5` bir tanımlamadır (virgül burada değişkenleri ayırır); adımdaki `bas++, son--` ise iki ifadeyi soldan sağa çalıştıran virgül operatörüdür. İki uç birbirine yaklaşan bu kalıbı, dizileri ve metinleri ters çevirirken (Bölüm 10 ve 11) kullanacağız.

for ile while arasındaki dönüşüm

Her `for`, `while`'a çevrilebilir: başlangıç döngünün önüne, adım gövdenin sonuna gider.

```c
// for biçimi
for (int i = 1; i <= 3; i++) {
    printf("%d ", i);
}

// while biçimi
int i = 1;
while (i <= 3) {
    printf("%d ", i);
    i++;
}
```

İkisi aynı işi yapar. Tek fark, `for`'da `i`'nin döngü bitince ortadan kalkması, `while` biçiminde ise döngüden sonra da yaşamasıdır. Bunu sıradaki başlıkta inceleyelim.

Döngü değişkeninin kapsamı

C99'dan itibaren `for`'un başlangıç kısmında değişken tanımlayabiliriz. Böyle tanımlanan değişken yalnızca o `for` döngüsünün içinde geçerlidir; buna değişkenin **kapsamı** (scope) denir.

▸ kapsamHata.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 3; i++) {
        printf("%d ", i);
    }
    printf("\nSon değer: %d\n", i);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
kapsamHata.c: In function 'main':
kapsamHata.c:7:33: error: 'i' undeclared (first use in this function)
    7 |     printf("\nSon değer: %d\n", i);
      |                                 ^
```

Döngü bittikten sonra da sayacın değerine ihtiyaç varsa değişkeni döngüden önce tanımlarız:

```c
int i;
for (i = 0; i < 3; i++) {
    // ...
}
// burada i == 3
```

Ardışık iki `for` aynı adı kullanabilir; her biri kendi `i`'sini yaratır (yukarıdaki `forTemel.c`'de olduğu gibi).

> *Tarihçe · for içinde tanımlama*
>
> C89'da `for (int i = 0; ...)` yazılamazdı; sayaç döngüden önce tanımlanırdı. Döngü içi tanımlama C99 ile geldi; C++ dilinde bu yazım zaten vardı.

8.4 Sayaç, Toplayıcı ve Bayrak

Döngülerde üç tür yardımcı değişken sürekli karşımıza çıkar:

Rol	Görevi	İlk değer	Döngüde
Sayaç (counter)	Bir şeyin kaç kez olduğunu sayar	0	`sayac++;`
Toplayıcı (sum)	Değerleri biriktirir	0	`toplam += x;`
Çarpım (product)	Değerleri çarpar	1	`carpim *= x;`
Bayrak (flag)	Bir durumun olup olmadığını taşır	`true` ya da `false`	koşula göre değişir

İlk değerler rastgele seçilmez: 0 toplamanın, 1 çarpmanın etkisiz elemanıdır. Çarpımı 0 ile başlatırsak sonuç hep 0 olur. Bayrak ise genellikle `bool` tipindedir (`<stdbool.h>`, Bölüm 5) ve "aradığımı buldum mu?", "her şey geçerli mi?" gibi sorulara cevap taşır; asal sayı örneğinde göreceğiz.

Bir programda hepsini birlikte görelim: 1'den `n`'e kadar toplamı, çarpımı ve çift sayıların adedini hesaplayalım.

roller.c

```c
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("n = ");
    scanf("%d", &n);

    int toplam = 0;
    int carpim = 1;
    int sayac = 0;
    for (int i = 1; i <= n; i++) {
        toplam += i;
        carpim *= i;
        if (i % 2 == 0) {
            sayac++;
        }
    }
    printf("1'den %d'e toplam: %d\n", n, toplam);
    printf("Çarpım (%d!): %d\n", n, carpim);
    printf("Çift sayı adedi: %d\n", sayac);
    return 0;
}
```

ÇIKTI

```
n = 5
1'den 5'e toplam: 15
Çarpım (5!): 120
Çift sayı adedi: 2
```

Toplam için kapalı bir formül de vardır: `n * (n + 1) / 2`. Çarpım ise `n` büyüdükçe `int`'e sığmaz; bunu faktöriyel örneğinde göreceğiz.

> **▲ Sık Yapılan Hata · Toplayıcıya ilk değer vermemek**
>
> `int toplam;` yazıp ilk değer vermeden döngüde `toplam += i;` dersek toplam, bellekte kalmış bir çöp değere eklenir. Çöp bazen tesadüfen 0 çıkar; program bizim bilgisayarda doğru çalışır, başka bir yerde yanlış sonuç verir. Bunun tanımsız davranış olduğunu Bölüm 5'te görmüştük. Kural: sayaç, toplayıcı ve çarpım, tanımlandıkları satırda ilk değerlerini alır.

8.5 break, continue ve Sonsuz Döngüler

Bazen bir döngüyü koşulu bozulmadan önce kesmek ya da belirli bir turu atlamak isteriz. İki anahtar sözcük bunu sağlar:

- **`break`**: içinde bulunduğu döngüyü tamamen bitirir; akış döngüden sonraki satıra geçer.
- **`continue`**: gövdenin kalanını atlar ve bir sonraki tura geçer.

breakContinue.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 50; i <= 100; i++) {
        if (i % 7 == 0) {
            printf("50'den sonra 7'ye bölünen ilk sayı: %d\n", i);
```

```
            break;
        }
    }

    for (int i = 1; i <= 10; i++) {
        if (i % 3 == 0) {
            continue;
        }
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

```
ÇIKTI
50'den sonra 7'ye bölünen ilk sayı: 56
1 2 4 5 7 8 10
```

İlk döngü 56'da bitti; 57'den 100'e kadar olan sayılar hiç denenmedi. İkinci döngü 3, 6 ve 9'da `printf`'i atladı, ama döngü bitmedi.

İki kuralı aklımızda tutalım:

1. `break` yalnızca **en içteki** döngüyü bitirir. İç içe döngülerde dış döngü sürer.
2. `for`'da `continue` yine de adım kısmını çalıştırır. `while`'da ise adım gövdenin içinde olduğundan `continue` onu atlayabilir:

```
int i = 1;
while (i <= 5) {
    if (i == 3) {
        continue;      // i++ atlanır, i hep 3 kalır: sonsuz döngü
    }
    printf("%d ", i);
    i++;
}
```

Burada `i` 3 olduğunda `continue` adımı (`i++`) atlar, koşul yeniden denetlenir ve döngü sonsuza kadar sürer. `continue` kullanan `while` döngülerinde ilerleme satırının atlanmadığından emin olmak gerekir.

Sonsuz döngüler

Koşulu hiç yanlış olmayan döngüye **sonsuz döngü** denir. İki yaygın yazılışı vardır: `while (1)` ve `for (;;)`. Sonsuz döngü her zaman hata değildir; çıkışı `break` ile sağlanan, kaç tur döneceği önceden bilinmeyen işlerde bilerek kullanılır:

```
▸ sonsuzDongu.c
#include <stdio.h>

int main(void) {
    int kuvvet = 1;
    while (1) {
        kuvvet *= 2;
        if (kuvvet > 1000) {
            break;
        }
    }
```

```
    printf("1000'i ilk geçen 2 kuvveti: %d\n", kuvvet);
    return 0;
}
```

```
ÇIKTI
1000'i ilk geçen 2 kuvveti: 1024
```

`break` olmasaydı bu döngü sonsuza kadar dönerdi. Bilerek sonsuz döngü yazdığımızda `break` yolunun gerçekten çalışacağından emin olmalıyız. Bilmeden yazılan sonsuz döngüleri ise çalışırken terminalden `Ctrl+C` ile durdururuz.

Noktalı virgül tuzağı

`for (...)` ya da `while (...)` satırının sonuna yanlışlıkla `;` koyarsak C bunu **boş bir gövde** sayar. Altındaki satırlar döngüden bağımsız, sıradan satırlar olur:

noktaliVirgul.c

```c
#include <stdio.h>

int main(void) {
    int i = 0;
    for (i = 1; i <= 3; i++);
        printf("Merhaba i=%d\n", i);
    return 0;
}
```

Derleyici `-Wall -Wextra` ile uyarır, ama derlemeyi durdurmaz:

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra noktaliVirgul.c -o noktaliVirgul
noktaliVirgul.c: In function 'main':
noktaliVirgul.c:5:5: warning: this 'for' clause does not guard... [-Wmisleading-indentation]
    5 |     for (i = 1; i <= 3; i++);
      |     ^~~
noktaliVirgul.c:6:9: note: ...this statement, but the latter is misleadingly indented as if it w
ere guarded by the 'for'
    6 |         printf("Merhaba i=%d\n", i);
      |         ^~~~~~
```

```
ÇIKTI
Merhaba i=4
```

Girinti insanı kandırır, derleyiciyi kandırmaz. Döngü boş gövdeyle üç tur döndü (`i` 4 oldu), `printf` ise döngüden sonra yalnızca bir kez çalıştı. `while` karşılığı daha tehlikelidir:

```c
int i = 1;
while (i <= 3);        // boş gövde: i hiç değişmez, sonsuz döngü
    i++;               // buraya hiç gelinmez
```

> **◆ Kritik Nokta · Süslü parantezi her zaman yazın**
>
> Gövde tek satır olsa bile `{ }` kullanmak, bu tür hataları ve "sonradan satır eklenince gövdeye girmiyor" sorunlarını önler. Bu kitapta tek satırlık gövdelerde de süslü parantez kullanıyoruz.

Bilerek boş gövde yazmak gerektiğinde (ileride tampon temizlerken göreceğiz) boşluğu `{ }` ile belli ederiz; böylece okuyan kişi bunun kazara değil kasıtlı olduğunu anlar.

8.6 İç İçe Döngüler ve Desenler

Bir döngünün gövdesine başka bir döngü yazabiliriz. Kural basittir: **dış döngünün her turunda iç döngü baştan sona çalışır.** Dış döngü 3 kez, iç döngü 4 kez dönüyorsa iç gövde toplam 12 kez çalışır.

İç içe döngülerin en tanıdık kullanımı satır-sütun yapılarıdır: iç döngü bir satırı, dış döngü satırların sayısını üretir. Satırın sonundaki `printf("\n")` iç döngünün dışında, dış döngünün içinde durur.

▸ desenler.c

```c
#include <stdio.h>

int main(void) {
    int n = 4;

    printf("Dik üçgen:\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    printf("Ters üçgen:\n");
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    printf("Sayı üçgeni:\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d", j);
        }
        printf("\n");
    }
    return 0;
}
```

```
ÇIKTI
Dik üçgen:
*
**
***
****
Ters üçgen:
****
***
**
*
Sayı üçgeni:
1
12
123
1234
```

Dik üçgende `i` satır numarasıdır ve `i`. satırda `i` yıldız vardır; bu yüzden iç döngünün sınırı dış sayaca bağlıdır (`j <= i`). Ters üçgende yalnızca dış döngü geriye sayar. Sayı üçgeninde yıldız yerine iç sayaç `j` yazdırılır.

Boşluk da içeren desenlerde önce her satır için "kaç boşluk, kaç yıldız" tablosunu çıkarmak işi kolaylaştırır. Piramit için, `n = 4` iken:

Satır `i`	Boşluk	Yıldız
1	3	1
2	2	3
3	1	5
4	0	7

Tablodan formüller okunur: `i`. satırda `n - i` boşluk ve `2 * i - 1` yıldız vardır. Bir satırda iki ayrı iç döngü art arda çalışır:

▸ piramit.c

```c
#include <stdio.h>

int main(void) {
    int n = 4;
    for (int i = 1; i <= n; i++) {
        for (int bosluk = 1; bosluk <= n - i; bosluk++) {
            printf(" ");
        }
        for (int yildiz = 1; yildiz <= 2 * i - 1; yildiz++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

```
ÇIKTI
   *
  ***
 *****
*******
```

Sınırları kâğıtta çıkardıktan sonra döngüler kendiliğinden yazılır. Bir desenin kaç yıldız yazdırdığı sorulduğunda iç gövdenin toplam kaç kez çalıştığını sayarız. İç sınır dış sayaca bağlıysa (`j <= i`) tur başına değişen sayıları toplarız: `1 + 2 + 3 + 4 = 10`. İç sınır sabitse (`j <= 4`) çarparız: 3 dış tur için `3 × 4 = 12`.

İç içe döngülerin bir başka klasik uygulaması çarpım tablosudur:

▸ carpimTablosu.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }
    return 0;
}
```

ÇIKTI

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

`%4d` her sayıyı en az 4 karakter genişliğinde, sağa yaslı yazar ve sütunları hizalar (Bölüm 6).

İç içe döngülerde `break` yalnızca iç döngüyü bitirir. Her iki döngüden birden çıkmak gerekirse bir bayrak değişkeni kullanılır ya da (nadiren) `goto`'ya başvurulur; bunu bölümün sonunda göreceğiz.

8.7 İzleme Tablosu ve Off-by-One Hatası

Bir döngünün ne yazdıracağını kestirmenin en güvenilir yolu **izleme tablosudur** (trace table): her değişken için bir sütun açar, her turdan sonra değişkenlerin yeni değerlerini yazarız. Aşağıdaki programın çıktısını önce tabloyla bulalım.

▸ izleme.c

```c
#include <stdio.h>

int main(void) {
    int a = 1;
    int b = 10;
    int tur = 0;

    while (a < b) {
        a += 2;
        b -= 1;
        tur++;
    }
    printf("a=%d b=%d tur=%d\n", a, b, tur);
    return 0;
}
```

Aşama	Koşul `a < b`	`a`	`b`	`tur`
Başlangıç		1	10	0
1. tur	1 < 10 doğru	3	9	1
2. tur	3 < 9 doğru	5	8	2
3. tur	5 < 8 doğru	7	7	3
Bitiş	7 < 7 yanlış	7	7	3

```
ÇIKTI
a=7 b=7 tur=3
```

> **İpucu · İzleme tablosunun kuralları**
>
> 1. Değişkenlerin ilk değerlerini yaz.
> 2. Her tur başında koşulu yaz ve doğru mu yanlış mı olduğuna karar ver.
> 3. Doğruysa gövdeyi sırayla uygula, değişkenleri güncelle.
> 4. Son satır koşulun yanlış çıktığı satırdır. Döngüden sonraki `printf`'i o satırdaki değerlerle çöz.

Off-by-one hatası

Off-by-one ("bir eksik ya da bir fazla") hatası, bir döngünün istenenden tam bir tur eksik veya fazla dönmesidir. Neredeyse her zaman `<` ile `<=` arasındaki karışıklıktan ya da başlangıç değerinin 0 mı 1 mi olacağını yanlış seçmekten doğar.

Tur sayısını şu tabloyla hesaplayabiliriz:

Döngü başlığı	Tur sayısı
`for (i = a; i < b; i++)`	`b - a` (örnek: `i = 0; i < 5` ise 5 tur)
`for (i = a; i <= b; i++)`	`b - a + 1` (örnek: `i = 0; i <= 5` ise 6 tur)
`for (i = 0; i < 10; i += 3)`	`i` = 0, 3, 6, 9 için 4 tur (adım `k` ise tek tek say)

1'den `n`'e kadar toplarken `i <= n` yerine `i < n` yazarsak `n` toplama girmez: `n = 5` için 15 yerine 10 buluruz. Döngü sonrası sayaç değeri de buradan çıkar: `for (i = 0; i < 5; i++)` biterken `i` 5'tir; `break` ile biten bir döngüde ise `i`, `break`'in çalıştığı andaki değerde kalır.

> **▲ Sık Yapılan Hata · Sıfırdan mı, birden mi?**
>
> Dizilerle çalışırken (Bölüm 10) indeksler 0'dan başlar; `n` elemanlı bir dizide son indeks `n - 1`'dir ve doğru döngü `i < n` ile yazılır. Gündelik sayma işlerinde (1'den 10'a kadar yaz) ise `i <= 10` doğaldır. Hangi durumda olduğumuzu baştan belirlemek, off-by-one hatalarının çoğunu önler.

8.8 Klasik Algoritmalar

Döngülerin asıl gücü, hesaplama kalıplarında görülür. Bu başlıkta programlama derslerinin vazgeçilmez örneklerini işleyeceğiz. Hepsini şimdilik `main` içinde yazıyoruz; Bölüm 9'da fonksiyonlara dönüştüreceğiz.

Faktöriyel ve int taşması

`n! = 1 · 2 · … · n` ve `0! = 1` olduğundan çarpım 1 ile başlar. Faktöriyel çok hızlı büyür: `12! = 479001600` bir `int`'e (en fazla 2147483647) sığar, `13!` sığmaz. İşaretli tamsayı taşması tanımsız davranıştır (Bölüm 5); bu yüzden büyük faktöriyeller için `long long` kullanır ve `%lld` ile yazdırırız. `long long`'a `20!` sığar, `21!` sığmaz.

▸ faktoriyel.c

```c
#include <stdio.h>

int main(void) {
    long long f = 1;
    for (int n = 1; n <= 20; n++) {
        f *= n;
        if (n % 5 == 0 || n == 12 || n == 13) {
            printf("%2d! = %lld\n", n, f);
        }
    }
    return 0;
}
```

```
ÇIKTI
 5! = 120
10! = 3628800
12! = 479001600
13! = 6227020800
15! = 1307674368000
20! = 2432902008176640000
```

`13!` değerinin 4 bayt `int`'e sığmayacak kadar büyük olduğuna, ama `long long`'da doğru göründüğüne dikkat edelim. `%2d` belirteci, tek ve çift haneli sayıların alt alta hizalanmasını sağlar.

Basamak sayısı, basamak toplamı ve ters çevirme

Anahtar fikir iki işlemdir: `n % 10` son basamağı verir, `n / 10` son basamağı atar. Sayı 0 olana kadar bu iki işlemi tekrarlarsak sayının bütün basamaklarını sondan başa doğru dolaşmış oluruz.

▸ basamaklar.c

```c
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("Pozitif bir sayı: ");
    scanf("%d", &n);

    int kalan = n;
    int basamakSayisi = 0;
    int basamakToplami = 0;
    int ters = 0;

    while (kalan > 0) {
        int son = kalan % 10;
        basamakSayisi++;
        basamakToplami += son;
        ters = ters * 10 + son;
        kalan /= 10;
    }
    printf("Basamak sayısı: %d\n", basamakSayisi);
```

```
    printf("Basamak toplamı: %d\n", basamakToplami);
    printf("Ters: %d\n", ters);
    return 0;
}
```

```
ÇIKTI
Pozitif bir sayı: 4712
Basamak sayısı: 4
Basamak toplamı: 14
Ters: 2174
```

`ters` değişkeni tur tur şöyle büyür: 2, 21, 217, 2174. Her turda eski değer 10 ile çarpılıp yeni basamak eklenir. Bu programla ilgili üç nokta var:

1. `n` bozulmasın diye çalışma, kopyası olan `kalan` üzerinde yapıldı.
2. `n` sıfırsa `while` hiç dönmez ve basamak sayısı 0 çıkar; oysa 0 tek basamaklıdır. Bu özel durumu `do-while` çözer.
3. Sondaki sıfırlar tersi kısaltır (1200'ün tersi 21 olur); çok basamaklı sayıların tersi de `int`'e sığmayabilir.

Basamak işlemlerinin bir uygulaması palindrom (tersiyle aynı olan) sayı denetimidir: 1221'in tersi 1221 olduğundan palindromdur. Bu denetimi alıştırmalarda yapacağız.

Asal sayı testi

Asal sayı, 1 ve kendisinden başka böleni olmayan, 1'den büyük doğal sayıdır. Bir sayının asal olup olmadığını anlamak için 2'den başlayıp bölen aramak yeterlidir. Ama `n`'e kadar gitmeye gerek yoktur: `n = a · b` ise küçük çarpan `√n`'i geçemez. Yani `i * i <= n` koşuluyla aramak yeter.

Burada bayrak değişkeni devreye girer. "Bölen bulundu mu?" bilgisini `bool` tipinde bir değişkende taşırız: başta `true` kabul edilir, bir bölen bulunursa `false` yapılır. Program 2'den `n`'e kadar tüm asalları listeler:

▸ asalListe.c

```
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int n = 0;
    printf("Üst sınır: ");
    scanf("%d", &n);

    for (int sayi = 2; sayi <= n; sayi++) {
        bool asal = true;
        for (int i = 2; i * i <= sayi; i++) {
            if (sayi % i == 0) {
                asal = false;
                break;
            }
        }
        if (asal) {
            printf("%d ", sayi);
        }
    }
    printf("\n");
    return 0;
}
```

```
ÇIKTI
Üst sınır: 50
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

`break` iç döngüden çıkar: bölen bulununca o sayı için aramayı sürdürmenin anlamı yoktur. `bool asal = true;` satırı **dış döngünün içinde** olmalıdır; dışarıya alırsak bir kez `false` olan bayrak bir daha `true` olmaz ve sonraki sayıların hiçbiri asal sayılmaz. 2 ve 3 için iç döngü hiç dönmez (`2 * 2 <= 2` yanlıştır), bayrak `true` kalır ve doğru sonuç çıkar.

EBOB ve EKOK

İki sayının **EBOB**'u (en büyük ortak bölen), ikisini de tam bölen en büyük sayıdır. Öklid algoritması, yaklaşık 2300 yıllık olmasına rağmen hâlâ kullanılan en zarif döngülerden biridir: `a` ile `b`'nin EBOB'u, `b` ile `a % b`'nin EBOB'u ile aynıdır. `b` sıfır olunca cevap `a`'dır.

EKOK (en küçük ortak kat) ise EBOB'dan çıkar: `EKOK = a / EBOB * b`. Bölmeyi önce yapmak, `a * b` çarpımının taşma riskini azaltır.

▸ ebobEkok.c

```c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0;
    printf("İki sayı: ");
    scanf("%d %d", &a, &b);

    int x = a;
    int y = b;
    while (y != 0) {
        int kalan = x % y;
        x = y;
        y = kalan;
    }
    printf("EBOB = %d\n", x);
    printf("EKOK = %d\n", a / x * b);
    return 0;
}
```

```
ÇIKTI
İki sayı: 48 18
EBOB = 6
EKOK = 144
```

`(x, y)` çifti şöyle değişir: (48, 18) → (18, 12) → (12, 6) → (6, 0). `y` sıfır olunca döngü biter ve EBOB `x`'te kalır. Orijinal değerleri korumak için çalışma kopyalar (`x`, `y`) üzerinde yapıldı; EKOK hesabında `a` ve `b` hâlâ gerekiyor.

Fibonacci

Fibonacci dizisi 0, 1, 1, 2, 3, 5, 8, … biçiminde, her terimi önceki iki terimin toplamı olan dizidir. Döngüde iki değişken "kayar": `onceki` ve `simdiki`. Yeni terim bir geçici değişkende hesaplanır, ardından iki değişken bir adım ilerletilir. Atama sırası bozulursa dizi bozulur.

fibonacci.c

```c
#include <stdio.h>

int main(void) {
    long long onceki = 0;
    long long simdiki = 1;

    for (int i = 1; i <= 10; i++) {
        printf("%lld ", onceki);
        long long sonraki = onceki + simdiki;
        onceki = simdiki;
        simdiki = sonraki;
    }
    printf("\n");
    return 0;
}
```

ÇIKTI

```
0 1 1 2 3 5 8 13 21 34
```

Her turda önce `onceki` yazdırılır, sonra çift bir adım ilerler: (0, 1) → (1, 1) → (1, 2) → (2, 3) → … Terimler hızla büyüdüğünden `long long` kullandık; 90. terime kadar `long long`'a sığar.

Mükemmel sayı

Kendisi hariç bölenlerinin toplamı kendisine eşit olan sayıya **mükemmel sayı** denir: 6 = 1 + 2 + 3. Kendisi dışındaki en büyük bölen en fazla `n / 2` olduğundan taramayı orada bitirebiliriz. Bölen toplamı her sayı için sıfırdan başlamalıdır.

mukemmel.c

```c
#include <stdio.h>

int main(void) {
    for (int n = 2; n <= 10000; n++) {
        int bolenToplami = 0;
        for (int i = 1; i <= n / 2; i++) {
            if (n % i == 0) {
                bolenToplami += i;
            }
        }
        if (bolenToplami == n) {
            printf("%d ", n);
        }
    }
    printf("\n");
    return 0;
}
```

ÇIKTI

```
6 28 496 8128
```

Toplayıcıyı dış döngünün dışında tanımlarsak bölenler sayıdan sayıya birikir ve hiçbir sayı mükemmel çıkmaz. Genel kural: her turda yeniden başlaması gereken sayaç, toplayıcı ve bayraklar, o turun **içinde** ilk değerlerini alır.

Üç Dilde · Döngüde dolaşmanın üç dildeki yüzü *C · C++ · C#*

C'de bir kümenin elemanlarını dolaşmak için indeksli `for` kullandık. Dizileri Bölüm 10'da göreceğiz; şimdilik adı konmuş bir değer listesini düşünelim. İleride C++ ve C#'ta bunun için daha kısa bir yazım olduğunu göreceğiz.

```c
// C: indeksli for
int sayilar[] = {10, 20, 30};
for (int i = 0; i < 3; i++) {
    printf("%d ", sayilar[i]);
}
```

```cpp
// C++11'den itibaren: aralık tabanlı for (range-for)
int sayilar[] = {10, 20, 30};
for (int s : sayilar) {
    std::cout << s << ' ';
}
```

```csharp
// C#: foreach
int[] sayilar = { 10, 20, 30 };
foreach (int s in sayilar) {
    Console.Write(s + " ");
}
```

C++'taki `range-for` ve C#'taki `foreach`, indeks tutmadan her elemanı sırayla verir; off-by-one hatasına yer bırakmaz. `while`, `do-while` ve klasik `for` ise üç dilde de aynı söz dizimiyle çalışır. Farklar Bölüm 24 ve 45'te ele alınacak.

8.9 Kullanıcı Girdisiyle Çalışan Döngüler

Döngülerin en yararlı kullanımlarından biri, kaç veri geleceğini önceden bilmediğimiz durumlarda kullanıcıdan veri okumaktır. Bu başlıkta üç kalıp göreceğiz: sentinel ile bitirme, hatalı girdiyi tekrar sorma ve girdi bitene kadar okuma. Bu kalıplar Bölüm 6'daki `scanf` bilgisine dayanır: `scanf` başarıyla okuduğu değer sayısını döndürür.

Sentinel: -1 girilene kadar oku

Kaç değer girileceği baştan belli değilse girdinin bittiğini bildiren özel bir değer kararlaştırılır. Bu değere **sentinel** (nöbetçi değer) denir. Notlar 0-100 arasındaysa -1 hiçbir not olamaz; dolayısıyla bitiş işareti olabilir.

Kalıp şudur: değeri döngüden **önce** oku, döngü gövdesinin **sonunda** yeniden oku.

▸ ortalama.c

```c
#include <stdio.h>

int main(void) {
    int notu = 0;
    int toplam = 0;
    int adet = 0;

    printf("Notları girin (bitirmek için -1): ");
    scanf("%d", &notu);
    while (notu != -1) {
        toplam += notu;
        adet++;
```

```
        scanf("%d", &notu);
    }

    if (adet == 0) {
        printf("Hiç not girilmedi.\n");
    } else {
        printf("%d not, ortalama: %.2f\n", adet, toplam / (double)adet);
    }
    return 0;
}
```

```
ÇIKTI
Notları girin (bitirmek için -1): 70 85 90 -1
3 not, ortalama: 81.67
```

`(double)adet` ifadesi `adet`'i geçici olarak `double`'a çevirir; böylece bölme tamsayı bölmesi olmaz (245 / 3 = 81 değil, 81,67). -1 ortalamaya katılmaz, çünkü veri değil işarettir. Hiç not girilmezse `adet` sıfır kalır ve sıfıra bölme olurdu; bunu `if` ile önlüyoruz.

Girdi doğrulama: do-while ve tampon temizleme

Kullanıcı yanlış değer girerse programın tekrar sorması gerekir. "En az bir kez sor" olduğu için `do-while` burada doğal seçimdir.

Bir tuzağa dikkat edelim. Klavyeden yazılanlar önce bir **tampona** (buffer) gider, `scanf` oradan okur. `scanf("%d", ...)` bir harf görürse okuyamaz, 0 döndürür ve harfleri tamponda bırakır. Aynı `scanf` tekrar çağrılırsa yine aynı harfleri görür: sonsuz döngü. Çözüm, bozuk satırı tampondan satır sonuna kadar boşaltmaktır:

```
int c;
while ((c = getchar()) != '\n' && c != EOF) { }
```

`getchar()` tamponun başındaki bir karakteri okuyup tüketir; döngü `'\n'` görene kadar bunu tekrarlar. Gövde bilerek boştur ve bunu `{ }` ile belli ettik. `EOF` denetimi, girdi tamamen bittiğinde `getchar()`'ın `'\n'` yerine `EOF` döndürmesi içindir; yoksa döngü sonsuza kadar dönerdi. `c`'nin `char` değil `int` olması da bu yüzdendir: `EOF` negatif bir sayıdır ve `char`'a güvenle sığmayabilir.

▸ dogrulama.c

```
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int notu = 0;
    bool gecerli = false;

    do {
        printf("Notunuzu girin (0-100): ");
        if (scanf("%d", &notu) != 1) {
            printf("Sayı girmediniz!\n");
            int c = 0;
            while ((c = getchar()) != '\n' && c != EOF) { }
        } else if (notu < 0 || notu > 100) {
            printf("Not 0-100 arasında olmalı!\n");
        } else {
            gecerli = true;
        }
```

```
    } while (!gecerli);

    printf("Kabul edildi: %d\n", notu);
    return 0;
}
```

```
ÇIKTI
Notunuzu girin (0-100): abc
Sayı girmediniz!
Notunuzu girin (0-100): -5
Not 0-100 arasında olmalı!
Notunuzu girin (0-100): 150
Not 0-100 arasında olmalı!
Notunuzu girin (0-100): 85
Kabul edildi: 85
```

`abc` girilince `scanf` 0 döndürdü ve tampon temizlendi. -5 ve 150 sayıdır ama aralık dışıdır. 85 kabul edilince `gecerli` doğru oldu ve `do-while` bitti. Bu program girdi tümüyle biterse (`scanf` `EOF` döndürürse) sonsuza kadar sormaya devam eder; gerçek programlarda `EOF` durumu ayrıca ele alınır.

Girdi bitene kadar oku

Tek bir `%d` okuyorsak döngü koşulunu doğrudan `scanf`'e bağlayabiliriz:

▸ eofOku.c

```c
#include <stdio.h>

int main(void) {
    int x = 0;
    int adet = 0;
    int toplam = 0;

    while (scanf("%d", &x) == 1) {
        toplam += x;
        adet++;
    }
    printf("%d sayı okundu, toplam %d\n", adet, toplam);
    return 0;
}
```

```
ÇIKTI
4 8 15
16 23 42
6 sayı okundu, toplam 108
```

Döngü, `scanf` bir sayı okuyamayınca biter: girdi tükendiğinde (EOF, dosya sonu) ya da sayı yerine harf geldiğinde. Klavyeden çalıştırırken girdiyi bitirmek için Linux ve macOS'ta satır başında `Ctrl+D`, Windows'ta `Ctrl+Z` ve ardından `Enter` basılır. Girdiyi bir dosyadan yönlendirirsek (`./eofOku < sayilar.txt`) dosya sonu kendiliğinden gelir.

▲ Sık Yapılan Hata · while (scanf(...) != EOF)

`!= EOF` yazarsak, kullanıcı harf girdiğinde program sonsuz döngüye girer: `scanf` harfi okuyamayınca `EOF` (-1) değil 0 döndürür, 0 ise `EOF`'tan farklı olduğu için koşul doğru kalır. Harf tamponda durduğundan `scanf` her turda yine 0 döndürür. Doğru kalıp, okunmasını istediğimiz değer sayısıyla karşılaştırmaktır: `== 1` (iki sayı okuyorsak `== 2`).

8.10 Ondalık Sayaçla Döngü Kurmak

Kayan noktalı sayılar bellekte ikilik tabanda yaklaşık tutulur; 0,1 gibi basit görünen bir değer bile tam yazılamaz (Bölüm 5). Bu yüzden 0,1 adımla ilerleyen bir döngüde küçük hatalar birikir:

▸ ondalikDongu.c

```c
#include <stdio.h>

int main(void) {
    double toplam = 0.0;
    for (int i = 0; i < 10; i++) {
        toplam += 0.1;
    }
    printf("10 kez 0.1 eklendi: %.17f\n", toplam);
    printf("toplam == 1.0 mı? %d\n", toplam == 1.0);

    int adet = 0;
    for (double x = 0.0; x < 1.0; x += 0.1) {
        adet++;
    }
    printf("x < 1.0 döngüsü %d kez döndü\n", adet);
    return 0;
}
```

```
ÇIKTI
10 kez 0.1 eklendi: 0.99999999999999989
toplam == 1.0 mı? 0
x < 1.0 döngüsü 11 kez döndü
```

On kez 0,1 eklemek tam 1,0 etmedi; bu yüzden `x < 1.0` döngüsü beklenen 10 yerine 11 kez döndü. Daha kötüsü, `for (double x = 0.0; x != 1.0; x += 0.1)` yazsaydık `x` hiçbir zaman tam 1,0 olmaz ve döngü sonsuza kadar sürerdi. `float` da çare değildir; hatası `double`'dan daha büyüktür.

◆ Kritik Nokta · Döngü sayacı tamsayı olsun

Ondalık değerler gerekiyorsa sayacı tamsayı tutar, ondalık değeri sayaçtan hesaplarız: `for (int i = 0; i < 10; i++)` içinde `double x = i / 10.0;`. Böylece tur sayısı kesindir. Ondalık sayıları `==` ile karşılaştırmak da güvenilmezdir.

8.11 goto Üzerine

C'de bir de `goto` deyimi vardır. Bir **etiket** (label) tanımlarız ve `goto etiket;` ile akışı doğrudan oraya atlatırız:

gotoCikis.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 2; i <= 9; i++) {
        for (int j = 2; j <= 9; j++) {
            if (i * j == 42) {
                printf("%d x %d = 42\n", i, j);
                goto bitti;       // iki döngüden birden çık
            }
        }
    }
bitti:
    printf("Arama sona erdi.\n");
    return 0;
}
```

ÇIKTI

```
6 x 7 = 42
Arama sona erdi.
```

Bu örnekte `goto`, iç içe iki döngüden tek hamlede çıkmanın pratik yoludur; `break` tek başına yalnızca iç döngüyü bitirirdi. Aynı iş bir bayrak değişkeniyle ya da (Bölüm 9'dan sonra) aramayı bir fonksiyona taşıyıp `return` ile de yapılabilir.

`goto` günlük kodda kaçınılan bir araçtır, çünkü akışı rastgele noktalara atlatan kod izlemesi ve değiştirmesi zor, "spagetti" bir yapı doğurur. Döngüler ve koşullar, bu ihtiyacın büyük bölümünü düzenli bir biçimde karşılar. Pratikte `goto` iki yerde meşru görülür: iç içe döngülerden çıkmak ve hata durumunda bir fonksiyonun sonundaki temizlik koduna gitmek. Bu kitapta ikinci kullanımı ileride bellek ve dosya yönetiminde göreceğiz. Bunların dışında `goto` kullanmayacağız.

> *Tarihçe · goto tartışması*
>
> 1968'de Edsger Dijkstra, "Go To Statement Considered Harmful" başlıklı kısa yazısıyla `goto`'nun programların anlaşılırlığını bozduğunu savundu. Bu görüş yapısal programlamanın (`if`, `while`, `for` gibi düzenli yapılarla yazma anlayışının) yaygınlaşmasına katkıda bulundu. C yine de `goto`'yu dilde tuttu; C++ ve C# da onu barındırır.

Bölüm Özeti

- `while` koşulu gövdeden önce denetler (0 kez dönebilir); `do { } while (koşul);` sonra denetler (en az 1 kez) ve sonunda `;` ister.
- `for (başlangıç; koşul; adım)` sayaçlı işler içindir. Sıra: başlangıç (bir kez), koşul, gövde, adım; sonra yine koşul.
- `for`, başlangıcı döngünün önüne, adımı gövdenin sonuna alarak `while`'a çevrilir. `for (int i = ...)` ile tanımlanan `i` yalnızca döngünün içinde yaşar.
- Sayaç ve toplayıcı 0 ile, çarpım 1 ile başlar. İlk değersiz yerel değişkenin içinde çöp vardır.
- `break` en içteki döngüyü bitirir, `continue` turun kalanını atlar. `while`'da `continue` ilerleme satırını atlayıp sonsuz döngü yaratabilir.
- `while (1)` ve `for (;;)` sonsuz döngüdür; çıkış `break` iledir.

- `for (...);` ve `while (...);` boş gövde demektir. Her zaman `{ }` yazmak bu hatayı önler.
- İç içe döngüde dış döngünün her turunda iç döngü baştan çalışır. İç sınır dış sayaca bağlıysa tur sayıları toplanır, sabitse çarpılır.
- İzleme tablosu bir döngünün çıktısını bulmanın en güvenilir yoludur. `i < b` için `b - a` tur, `i <= b` için `b - a + 1` tur dönülür; döngü bitince sayaç, koşulu bozan ilk değerdedir.
- Algoritma kalıpları: basamaklar için `% 10` ve `/ 10`; asal testi için `i * i <= n` ve bayrak; EBOB için Öklid; EKOK için `a / ebob * b`.
- Büyük faktöriyel ve Fibonacci için `long long` ve `%lld` kullanılır (`13!` `int`'e sığmaz, `20!` `long long`'a sığar).
- Sentinel döngüsünde değer döngüden önce ve gövdenin sonunda okunur. Girdi bitene kadar okumak için `while (scanf("%d", &x) == 1)` kullanılır; `!= EOF` kullanılmaz.
- Hatalı girdiyi tekrar sormak için `do-while` ve `while ((c = getchar()) != '\n' && c != EOF) { }` ile tampon temizliği kullanılır.
- Döngü sayacı `double` olmamalıdır; 0,1 adımı birikerek şaşar. `goto` yalnızca iç içe döngülerden çıkmak gibi nadir durumlarda kullanılır.

Alıştırmalar

1. `for (int i = 1; i <= 10; i += 3)` döngüsünün gövdesi kaç kez çalışır?

   A) 3
   B) 10
   C) 4
   D) 5
   E) 9

2. Döngülerle ilgili aşağıdaki ifadelerden hangisi doğrudur?

   A) `while` döngüsünde koşul, gövde bir kez çalıştıktan sonra denetlenir.
   B) `break`, yalnızca döngünün o turunun kalanını atlar ve bir sonraki tura geçer.
   C) `continue`, içinde bulunduğu döngüyü tamamen bitirir.
   D) `for (int i = 0; i < 3; i++)` ile tanımlanan `i`, döngü bittikten sonra da kullanılabilir.
   E) `do-while` döngüsünün gövdesi, koşul baştan yanlış olsa bile en az bir kez çalışır.

3. Aşağıdaki iç içe döngüde `say++` satırı toplam kaç kez çalışır?

```c
int say = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j <= i; j++) {
        say++;
    }
}
```

   A) 3
   B) 6
   C) 9

D) 4
E) 12

4. `while (scanf("%d", &x) == 1) { ... }` döngüsü ne zaman biter?

A) `x` sıfır olduğunda
B) Kullanıcı `Enter` tuşuna bastığında
C) `scanf` bir tamsayı okuyamadığında (girdi bittiğinde ya da harf geldiğinde)
D) Yalnızca girdi dosyası bittiğinde; harf girilirse döngü sonsuza kadar sürer
E) Hiçbir zaman biter

5. Programın çıktısı nedir?

```
for (int i = 0; i < 6; i++) {
    if (i == 2) {
        continue;
    }
    if (i == 4) {
        break;
    }
    printf("%d ", i);
}
```

6. Programın çıktısı nedir?

```
int s = 0;
for (int i = 1; i <= 4; i++) {
    for (int j = i; j <= 4; j++) {
        s++;
    }
}
printf("%d\n", s);
```

7. Programın çıktısı nedir?

```
int n = 10;
do {
    printf("%d ", n);
    n -= 3;
} while (n > 5);
printf("| %d\n", n);
```

8. Programın çıktısı nedir?

```
int i;
for (i = 0; i < 3; i++);
printf("%d\n", i);
```

9. Aşağıdaki kod 1'den 9'a kadar tek sayıları yazdırmak için yazılmıştır, ancak program sonsuza kadar çalışmaktadır. Nedenini açıklayın ve düzeltin.

```
int i = 0;
while (i < 10) {
    if (i % 2 == 0) {
        continue;
    }
```

```
    printf("%d ", i);
    i++;
}
```

10. Aşağıdaki program 1'den 100'e kadar sayıların toplamı olan 5050'yi yazdırmak için yazılmıştır, ancak ekrana 4950 yazmaktadır. Hatayı bulun ve düzeltin.

    ```c
    int toplam = 0;
    for (int i = 1; i < 100; i++) {
        toplam += i;
    }
    printf("%d\n", toplam);
    ```

11. Klavyeden bir `n` okuyan, 1'den `n`'e kadar 3'e bölünebilen sayıların adedini ve toplamını yazdıran bir program yazın.
12. Klavyeden pozitif bir tamsayı okuyup palindrom olup olmadığını (tersiyle aynı olup olmadığını) yazdıran bir program yazın. Örneğin 1221 palindromdur, 1231 değildir.
13. Klavyeden bir `n` okuyup `n` yüksekliğinde, boşluklarla ortalanmış yıldızlardan oluşan aşağıdaki elmas desenini çizen bir program yazın (`n = 3` için):

    ```
      *
     ***
    *****
     ***
      *
    ```

14. Klavyeden 2'den büyük bir tamsayı okuyup asal çarpanlarına ayıran bir program yazın. Örneğin 60 için `2 2 3 5` yazmalıdır.

BÖLÜM 9

Fonksiyonlar

BU BÖLÜMDE

- Fonksiyonun neden gerekli olduğunu; dönüş tipi, ad, parametre listesi, gövde ve `return` parçalarını tanımayı
- `void` fonksiyonları, erken `return`'ü ve dönüş değerinin kullanılmasını
- Prototipin (bildirimin) neden gerektiğini ve bununla ilgili derleyici hatalarını okumayı
- Parametre ile argüman farkını, örtük tip dönüşümünü ve değerle geçirmenin sonuçlarını (`takas` neden çalışmaz)
- Çağrı yığınında her çağrının kendi çerçevesini nasıl açtığını
- Yerel, blok ve global kapsamı, gölgelemeyi ve `static` yerel değişkenleri
- `<math.h>`, `<stdlib.h>` ve `<ctype.h>` fonksiyonlarını kullanmayı; algoritmaları fonksiyonlara bölerek program tasarlamayı

Şimdiye kadar yazdığımız programların hemen hepsi `main` içindeydi. Küçük programlarda bu yeterlidir, ama program büyüdükçe aynı hesabı üç ayrı yerde yapmamız, yüz satırlık bir `main` içinde hata aramamız gerekir. Bu bölümün konusu olan **fonksiyon**, bir işi yapan ve adı olan kod parçasıdır: bir kez yazarız, istediğimiz kadar çağırırız.

Aslında fonksiyon kullanmaya çoktan başladık: `main`, `printf` ve `scanf` birer fonksiyondur. Bu bölümde kendi fonksiyonlarımızı yazmayı, onlara veri geçirmeyi, değişkenlerin nerede görünüp nerede yok olduğunu ve çağrıların bellekte nasıl yürüdüğünü öğreneceğiz. Önceki bölümlerde `main` içinde yazdığımız asal sayı, EBOB gibi algoritmaları da fonksiyonlara dönüştüreceğiz.

9.1 Neden Fonksiyon?

> **Tanım: Fonksiyon**
>
> Bir işi yapmak üzere adlandırılmış, gerektiğinde girdi (parametre) alıp sonuç (dönüş değeri) üretebilen, istenen yerden çağrılabilen kod parçasıdır.

Fonksiyon kullanmanın üç temel nedeni vardır:

- **Tekrarı önler.** Aynı kodu kopyalamak yerine fonksiyonu çağırırız; hata varsa tek yerde düzeltiriz.
- **Okunurluğu artırır.** `if (asalMi(n))` satırı, yirmi satırlık bir bölen arama döngüsünden çok daha anlaşılırdır.
- **Problemi böler.** Büyük işi küçük parçalara ayırır, her parçayı ayrı yazar ve ayrı deneriz.

Fonksiyonlar iki gruba ayrılır. **Kütüphane fonksiyonlarını** (`printf`, `sqrt`, `rand`) başkaları yazmıştır; bildirimleri `#include` ettiğimiz başlık dosyalarındadır, kodları bağlayıcı (linker) aşamasında programa eklenir. **Kendi fonksiyonlarımızı** (`asalMi`, `ebob`) ise biz yazarız.

9.2 Fonksiyonun Anatomisi

Bir fonksiyon tanımı şu biçimdedir:

```
dönüşTipi ad(parametreListesi) {
    gövde
}
```

Bunu küçük bir örnekle görelim: bir tamsayının karesini döndüren `kareAl`.

Parça	Anlamı	`kareAl` için
Dönüş tipi (return type)	Çağırana geri verilen değerin tipi	`int`
Ad	Çağırırken kullanılan ad	`kareAl`
Parametre listesi	Fonksiyonun dışarıdan aldığı değerler (tip + ad)	`int x`
Gövde (body)	`{ }` içindeki deyimler	`return x * x;`
`return`	Değeri çağırana verir, fonksiyonu bitirir	`return x * x;`

▸ kareAl.c

```c
#include <stdio.h>

int kareAl(int x) {
    return x * x;
}

int main(void) {
    int a = kareAl(5);
    int b = kareAl(a);
    printf("%d %d\n", a, b);
    printf("%d\n", kareAl(2) + kareAl(3));
    return 0;
}
```

ÇIKTI

```
25 625
13
```

Çağrıda şunlar olur: `main` durur, kontrol `kareAl`'a geçer, `x` parametresi 5 değerini alır, `return` değeri geri verir ve `main` kaldığı yerden devam eder. `kareAl(5)` gibi bir **çağrı ifadesi**, döndürdüğü değerin yerine geçer. Onu bir değişkene atayabilir, başka bir fonksiyona argüman yapabilir ya da `kareAl(2) + kareAl(3)` gibi işlemlerde kullanabiliriz.

Birkaç kural:

- Fonksiyon adı, değişken adı kurallarına uyar. Kitapta camelCase kullanıyoruz ve adı yaptığı işi anlatan bir fiil ya da soru seçiyoruz: `ortalamaHesapla`, `asalMi`, `menuyuGoster`.

- Aynı adlı iki fonksiyon tanımlanamaz. Standart C'de fonksiyonun içinde başka bir fonksiyon da tanımlanmaz; tanımlar yan yana yazılır.
- Dönüş tipi `int`, `double`, `char`, `long long`, `bool` gibi herhangi bir tip ya da `void` olabilir.

İyi bir fonksiyon tek iş yapar ve adı o işi anlatır. `asalMi` sayının asal olup olmadığına karar verir; sonucu ekrana yazmak ya da sayıyı klavyeden okumak `main`'in işidir. "Hesaplayan fonksiyon yazdırmaz" kuralı, fonksiyonu başka programlarda da kullanılabilir kılar.

9.3 void Fonksiyonlar ve Erken Çıkış

Bazı fonksiyonlar değer üretmez, yalnızca bir iş yapar: ekrana yazar, menü gösterir. Bunların dönüş tipi `void`'dir (boş). `void` bir fonksiyonda `return deger;` yazılmaz; istersek yalnızca `return;` yazarak fonksiyonu erken bitirebiliriz. Sona gelince fonksiyon zaten kendiliğinden biter.

`return` çalıştığı anda fonksiyon biter ve altındaki satırlar çalışmaz. Bu yüzden özel durumları fonksiyonun başında ele alıp hemen çıkmak yaygın bir kalıptır:

▸ erkenCikis.c

```c
#include <stdio.h>

void bol(int pay, int payda) {
    if (payda == 0) {
        printf("Sıfıra bölünemez!\n");
        return;
    }
    printf("%d / %d = %d\n", pay, payda, pay / payda);
}

int mutlak(int n) {
    if (n < 0) {
        return -n;
    }
    return n;
}

int main(void) {
    bol(17, 5);
    bol(8, 0);
    printf("%d %d\n", mutlak(-7), mutlak(4));
    return 0;
}
```

ÇIKTI

```
17 / 5 = 3
Sıfıra bölünemez!
7 4
```

`bol(8, 0)` çağrısında `return;` bölmeyi atlattı; Bölüm 5'te tamsayıyı sıfıra bölmenin tanımsız davranış olduğunu görmüştük. `mutlak` içinde `else` yok: `n < 0` ise `return` fonksiyonu bitirdiği için ikinci `return`'e hiç gelinmez.

Dönüş değerini kullanmak zorunda değiliz: `topla(4, 5);` tek başına yazılırsa fonksiyon çalışır, değer atılır. `printf` ve `scanf` de değer döndürür (yazılan karakter sayısı, okunan öğe sayısı); çoğu zaman bu değeri atarız, ama `scanf(...) == 1` kontrolünü Bölüm 6'da kullanmıştık. Bir fonksiyon en fazla **bir** değer döndürür; birden fazla sonuç için yollar vardır (adres geçirme Bölüm 12'de, yapılar Bölüm 14'te).

▲ Sık Yapılan Hata · Sonucu kullanmayı unutmak

`kareAl(x);` tek başına yazılırsa `x` kare olmaz; hesaplanan değer atılır. Sonuç isteniyorsa bir yere konmalıdır: `x = kareAl(x);`. Bu hata, fonksiyonun çağıranın değişkenini değiştirdiğini sanmaktan gelir; nedenini "Değerle Geçirme" başlığında göreceğiz. `void` bir fonksiyonun çağrısını da değişkene atayamayız: `int s = selamVer();` ifadesi için gcc `void value not ignored as it ought to be` hatası verir.

9.4 Prototip (Bildirim)

gcc dosyayı yukarıdan aşağıya okur. Bir fonksiyon çağrısına geldiğinde o fonksiyonu daha önce görmüş olması gerekir. Fonksiyonu `main`'den sonra yazarsak sorun çıkar:

```c
#include <stdio.h>

int main(void) {
    int sonuc = kareAl(5);
    printf("%d\n", sonuc);
    return 0;
}

int kareAl(int x) {
    return x * x;
}
```

```
TERMİNAL
$ gcc -std=c17 -Wall prototipsiz.c -o prototipsiz
prototipsiz.c: In function 'main':
prototipsiz.c:4:17: error: implicit declaration of function 'kareAl' [-Wimplicit-function-
declaration]
    4 |     int sonuc = kareAl(5);
      |                 ^~~~~~
```

Bu, Bölüm 4'te `printf` yerine yanlış bir ad yazınca gördüğümüz `implicit declaration of function` hatasının aynısıdır: fonksiyon kodda aşağıda var, ama gcc çağrı satırına geldiğinde onu henüz okumamıştır. (gcc 14'ten önceki sürümler bunu yalnızca uyarı sayardı; yeni sürümler hata verir.) İki çözüm vardır:

1. Fonksiyonu `main`'den önce tanımlamak. Küçük programlarda yeterlidir.
2. Fonksiyonu `main`'den önce **tanıtmak**: bir prototip yazmak.

Tanım: Prototip

Fonksiyon tanımının gövdesiz hâlidir ve `;` ile biter. Derleyiciye "adı şu, dönüş tipi şu, parametreleri şunlar; tanımı daha aşağıda" der. Fonksiyon bildirimi (function declaration) de denir.

▸ prototipli.c

```c
#include <stdio.h>

int kareAl(int x);
double ortalama(int a, int b);

int main(void) {
    printf("%d\n", kareAl(6));
    printf("%.1f\n", ortalama(3, 4));
```

```
    return 0;
}

int kareAl(int x) {
    return x * x;
}

double ortalama(int a, int b) {
    return (a + b) / 2.0;
}
```

```
ÇIKTI
36
3.5
```

Önce prototipler, sonra `main`, en altta tanımlar: bu düzen çok yaygındır, çünkü dosyayı açan kişi önce programın akışını okur, ayrıntılara sonra bakar. Prototip kuralları şunlardır:

- Dönüş tipi, ad ve parametre tipleri tanımla birebir aynı olmalıdır. Parametre adları serbesttir ve yazılmayabilir: `int kareAl(int);` da geçerlidir.
- `#include <stdio.h>` satırı, `printf`, `scanf` ve onlarca fonksiyonun prototipini programımıza ekler. Başlık dosyası, prototipler topluluğudur.

Prototiple tanım uyuşmazsa gcc şikâyet eder:

▸ prototipCakisma.c

```
#include <stdio.h>

int alan(int en, int boy);

int main(void) {
    printf("%d\n", alan(3, 4));
    return 0;
}

double alan(int en, int boy) {
    return en * boy;
}
```

```
DERLEYİCİ ÇIKTISI
prototipCakisma.c:10:8: error: conflicting types for 'alan'; have 'double(int, int)'
   10 | double alan(int en, int boy) {
      |        ^~~~
prototipCakisma.c:3:5: note: previous declaration of 'alan' with type 'int(int, int)'
    3 | int alan(int en, int boy);
      |     ^~~~
```

`conflicting types for 'alan'`: aynı fonksiyon için iki farklı tip görüldü (prototipte `int`, tanımda `double`). Parametre sayısı ya da tipi farklı olsa da aynı hatayı alırız.

> **▲ Sık Yapılan Hata · Tanımın başlığına noktalı virgül koymak**
>
> Prototip `;` ile biter, tanım bitmez. `int topla(int a, int b); { ... }` yazarsak gcc süslü parantezi gövde olarak değil, başıboş bir blok olarak görür ve `{` işaretini gösterip `expected identifier or '(' before '{' token` der. Bölüm 8'deki `for (...);` tuzağının akrabasıdır.

> ◆ **Kritik Nokta · Boş parantez, (void) değildir**
>
> `int menu();` ile `int menu(void);` aynı şey değildir. Boş parantez C'de "parametreler belirtilmedi" demektir ve hatalı çağrıları (`menu(5)` gibi) yakalatmaz. Parametresiz fonksiyonlarda her zaman `(void)` yazarız. (C23'te boş parantez de `(void)` anlamına gelir, ama alışkanlık olarak `(void)` yazmak her sürümde güvenlidir.)

9.5 Parametre ve Argüman

İki terim sık karıştırılır:

Terim	Nerede?	Örnek
Parametre (parameter)	Fonksiyonun **tanımında**, parantez içinde	`int topla(int a, int b)`: `a` ve `b`
Argüman (argument)	Fonksiyonun **çağrısında**	`topla(5, y + 1)`: `5` ve `y + 1`

Argümanlar parametrelere sırayla eşlenir. Parametreler fonksiyonun içinde birer yerel değişken gibi davranır ve çağrıda argümanların değeriyle başlatılır. Her parametrenin tipi ayrı ayrı yazılır: `int topla(int a, int b)` doğrudur; `int topla(int a, b)` yazarsak gcc `unknown type name 'b'` der. Argüman sayısı da parametre sayısına eşit olmalıdır:

▸ eksikArguman.c

```c
#include <stdio.h>

int topla(int a, int b) {
    return a + b;
}

int main(void) {
    printf("%d\n", topla(5));
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

eksikArguman.c: In function 'main':
eksikArguman.c:8:20: error: too few arguments to function 'topla'; expected 2, have 1
    8 |     printf("%d\n", topla(5));
      |                    ^~~~~
eksikArguman.c:3:5: note: declared here
    3 | int topla(int a, int b) {
      |     ^~~~~
```

gcc fonksiyonun tanımlandığı yeri de gösterdi (`note: declared here`). Fazla argümanda mesaj `too many arguments to function` olur. Tip yalnızca tanımda ve prototipte yazılır, çağrıda yazılmaz: `topla(int 5, int 3)` yanlıştır.

Tip dönüşümü

Prototip ya da tanım görünürken argüman, parametrenin tipine kendiliğinden çevrilir; `return` ile döndürülen değer de dönüş tipine çevrilir. Bölüm 5'teki kurallar burada da geçerlidir: `int` değerin `double` parametreye geçmesi kayıpsızdır, `double`'ın `int`'e çevrilmesi ise ondalık kısmı atar (sıfıra doğru).

▸ tipDonusumu.c

```c
#include <stdio.h>

int tamKisim(double x) {
    return x;
}

double ortalamaYanlis(int a, int b) {
    return (a + b) / 2;
}

int main(void) {
    printf("%d %d\n", tamKisim(7.9), tamKisim(-7.9));
    printf("%.2f\n", ortalamaYanlis(3, 4));
    return 0;
}
```

```
ÇIKTI
7 -7
3.00
```

`tamKisim(7.9)`: `return x;` satırı `double`'ı `int`'e çevirirken ondalık kısmı attı. Negatifte sonuç yuvarlama değil kesme olduğu için -7 çıkar.

`ortalamaYanlis(3, 4)` ise bir tuzaktır. `(a + b) / 2` iki `int`'in bölmesidir; sonuç `7 / 2 = 3` olur ve ancak sonra `double`'a çevrilip `3.00` yazılır. Dönüş tipinin `double` olması bölmeyi `double` yapmaz. Doğrusu `/ 2.0` yazmaktır; `prototipli.c` içindeki `ortalama` bu yüzden doğru çalışmıştı.

9.6 Değerle Geçirme

> **Tanım: Değerle geçirme**
>
> Fonksiyon çağrılırken argümanın değerinin parametreye kopyalanmasıdır (pass by value). Parametre, argümandan bağımsız yeni bir yerel değişkendir.

Bu bölümün en önemli kuralı şudur: C'de argümanlar fonksiyona **değerle** (pass by value) geçirilir. Fonksiyon argümanın bir **kopyasını** alır; parametreyi fonksiyon içinde ne kadar değiştirirsek değiştirelim, çağıran fonksiyondaki değişken aynı kalır. Bir belgenin fotokopisine not almanın asıl belgeyi değiştirmemesi gibi.

Bu kuralın en bilinen sonucu, iki değişkenin değerini değiştirmeyi (takas, swap) deneyen fonksiyondur:

▸ takasCalismaz.c

```c
#include <stdio.h>

void takas(int a, int b) {
    int gecici = a;
    a = b;
    b = gecici;
    printf("takas içinde: a = %d, b = %d\n", a, b);
}

int main(void) {
    int x = 3;
    int y = 8;
    takas(x, y);
```

```
    printf("main içinde : x = %d, y = %d\n", x, y);
    return 0;
}
```

```
ÇIKTI
takas içinde: a = 8, b = 3
main içinde : x = 3, y = 8
```

Takas, `a` ve `b` kopyaları üzerinde başarıyla yapıldı; ama `x` ve `y` yerinde durdu. Fonksiyon bitince kopyalar yok oldu ve işin hiçbir izi kalmadı. Parametre adlarını `x` ve `y` yapsak da değişmez: aynı adlı iki değişken yine iki ayrı değişkendir.

Çalışan bir `takas`, değişkenlerin **adresini** alan bir fonksiyondur; bunu Bölüm 12'de işaretçilerle yazacağız. Şimdilik kural şudur: fonksiyon sonucu çağırana yalnızca `return` ile gönderir (`x = ikiKati(x);` gibi).

> ◆ **Kritik Nokta · Değerle geçirme**
>
> `takas(x, y)` çağrısından sonra `x` ve `y` değişmez; çünkü fonksiyon kopyalar üzerinde çalışır. Dizilerde durum görünürde farklıdır; nedenini Bölüm 10'da ve işaretçileri öğrendikten sonra Bölüm 12'de açıklayacağız.

9.7 Çağrı Yığını

Değerle geçirmenin arkasında bir mekanizma vardır. Program çalışırken her fonksiyon çağrısı için bellekte bir **yığın çerçevesi** (stack frame) açılır. Çerçevede fonksiyonun parametreleri, yerel değişkenleri ve fonksiyon bitince nereye dönüleceği bilgisi bulunur. Çerçeveler **çağrı yığını** (call stack) denen bellek bölgesinde üst üste dizilir; fonksiyon `return` edince çerçevesi silinir.

> **Tanım: Çağrı yığını**
>
> Çalışan fonksiyonların çerçevelerinin üst üste tutulduğu bellek bölgesidir. En son çağrılan fonksiyonun çerçevesi en üsttedir; fonksiyon bitince çerçevesi yığından atılır ve bir alttaki fonksiyon kaldığı yerden devam eder.

Yerel değişkenler ve parametreler bu yüzden fonksiyon bitince yok olur. Aynı adlı değişkenler de birbirini etkilemez: her çağrının kendi çerçevesi, yani kendi kopyaları vardır. Aşağıdaki programda `main`, `karelerToplami`'nı, o da `kare`'yi çağırıyor:

▸ cagriYigini.c

```
#include <stdio.h>

int kare(int x) {
    return x * x;
}

int karelerToplami(int a, int b) {
    int ka = kare(a);
    int kb = kare(b);
    return ka + kb;
}

int main(void) {
    int sonuc = karelerToplami(3, 4);
```

```
    printf("Sonuç: %d\n", sonuc);
    return 0;
}
```

```
ÇIKTI
Sonuç: 25
```

Programın yığın üzerindeki hareketini adım adım yazalım. Her satırda en üstteki çerçeve o an çalışan fonksiyondur; `?` değişkenin tanımlandığını ama henüz değer almadığını gösterir.

Adım	Olay	Yığın (üstten alta)
1	`main` başlar	`main` (sonuc=?)
2	`karelerToplami(3, 4)` çağrılır	`karelerToplami` (a=3, b=4, ka=?, kb=?) / `main`
3	`kare(3)` çağrılır	`kare` (x=3) / `karelerToplami` / `main`
4	`kare` 9 döndürür, çerçevesi silinir; `ka = 9`	`karelerToplami` (a=3, b=4, ka=9, kb=?) / `main`
5	`kare(4)` çağrılır, 16 döner; `kb = 16`	`kare` (x=4) / `karelerToplami` / `main`; sonra yine `karelerToplami` / `main`
6	`karelerToplami` 25 döndürür, çerçevesi silinir	`main` (sonuc=25)

Üç noktaya dikkat edelim:

1. Aynı `kare` fonksiyonu 3. ve 5. adımlarda iki ayrı çerçeve açtı; `x` her seferinde sıfırdan oluştu.
2. Çerçevedeki `a` ve `b`, `main`'deki değerlerin kopyasıdır.
3. Çağıran fonksiyon, çağırdığı bitene kadar bekler.

İç içe çağrılarda içteki çağrı önce çalışır, çünkü dıştaki fonksiyonun argümanı içtekinin sonucudur. `birEkle(ikiKati(5))` ifadesinde önce `ikiKati(5) = 10`, sonra `birEkle(10) = 11` hesaplanır. Sıra değişirse sonuç da değişir: `ikiKati(birEkle(5))` 12 verir. Ancak `f(g(x), h(y))` çağrısında `g` ile `h`'den hangisinin önce çalışacağını C belirlemez (Bölüm 5'te operatörlerin işleniş sırası için söylediğimizin aynısı). Bu yüzden aynı çağrıda yan etkisi olan (ekrana yazan, global değişkeni değiştiren) iki çağrıyı argüman yapmamalıyız.

Bir fonksiyon kendi kendini de çağırabilir; buna **özyineleme** (recursion) denir ve her çağrı kendi çerçevesini açtığı için çalışır. Yığın sınırlı bir bölge olduğundan durma koşulu olmayan özyineleme yığını doldurur ve program çöker. Konuyu Bölüm 18'de ayrıntılı işleyeceğiz.

9.8 Kapsam

> **Tanım: Kapsam**
>
> Bir adın programın hangi bölgelerinde geçerli olduğunu, yani o ada kodun neresinden erişilebildiğini belirleyen kuraldır (scope). Ömür (lifetime) ise değişkenin bellekte ne kadar süre var olduğudur.

Bir değişkenin **kapsamı** (scope), programın hangi bölgelerinden o değişkene erişilebildiğini belirler. Fonksiyonun içinde tanımlanan değişken **yereldir**: yalnızca o fonksiyonda görünür ve fonksiyon bitince yok olur. Parametreler de yereldir. Bir fonksiyonda tanımlı değişkeni başka bir fonksiyonda kullanmaya çalışırsak

gcc `'sonucDegeri' undeclared (first use in this function)` hatası verir. Değeri çağırana `return` ile göndermemiz gerekir.

Yerel değişkene ilk değer vermezsek içinde çöp değer bulunur ve onu okumak tanımsız davranıştır (Bölüm 5). Her yerel değişkene değerini tanımlarken vermeliyiz.

Blok kapsamı ve gölgeleme

`{ }` ile oluşturulan her blok kendi kapsamını açar. İç blokta, dış bloktaki bir değişkenle aynı adı kullanırsak iç olan dış olanı **gölgeler** (shadowing): blok boyunca o adın altında iç değişken görünür, blok bitince dış olan yeniden görünür.

▸ kapsamBlok.c

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    printf("1) x = %d\n", x);
    {
        int x = 20;
        int y = 5;
        printf("2) x = %d, y = %d\n", x, y);
    }
    printf("3) x = %d\n", x);
    return 0;
}
```

ÇIKTI

```
1) x = 10
2) x = 20, y = 5
3) x = 10
```

İç bloktaki `x = 20` dıştaki `x`'i değiştirmedi; ayrı, yeni bir değişkendi. Blok bitince `y` ile içteki `x` yok oldu, dıştaki `x` hâlâ 10. Gölgeleme geçerlidir ama okunması zordur; kendi kodumuzda kaçınmalıyız. Tek sınır şudur: fonksiyonun en dış bloğunda, parametreyle aynı adlı bir değişken tanımlayamayız (`int ikiKati(int n) { int n = 2; ... }` için gcc `'n' redeclared as different kind of symbol` der).

Global değişkenler

Hiçbir fonksiyonun içinde olmayan, dosyanın üstünde tanımlanan değişkene **global değişken** denir. Tanımından sonraki bütün fonksiyonlar onu görür ve değiştirebilir; program boyunca yaşar. İlk değer verilmezse otomatik olarak 0 olur (Bölüm 5).

▸ globalYerel.c

```c
#include <stdio.h>

int sayi = 100;

void yerelGolgeler(void) {
    int sayi = 7;
    printf("yerelGolgeler: sayi = %d\n", sayi);
}

int main(void) {
    printf("main         : sayi = %d\n", sayi);
```

```
    yerelGolgeler();
    sayi = sayi + 1;
    printf("main         : sayi = %d\n", sayi);
    return 0;
}
```

```
ÇIKTI
main         : sayi = 100
yerelGolgeler: sayi = 7
main         : sayi = 101
```

`yerelGolgeler` içinde yeni bir yerel `sayi` tanımlandığı için o fonksiyonda global `sayi` görünmez. `main`'in kendi `sayi`'sı olmadığından global olanı değiştirdi (100 ⟶ 101).

Global değişken kolaydır: parametre geçirmeden her yerden erişiriz. Ama bu yüzden az kullanılır:

- Değeri hangi fonksiyonun değiştirdiğini bulmak zorlaşır.
- Global değişkene bağımlı fonksiyon başka bir programa taşınamaz ve tek başına denenemez; girdisi ve çıktısı artık parametre ve `return` değil, görünmeyen ortak bir yerdir.
- Yerel değişkenle aynı ad verilirse yanlışlıkla gölgeleme olur.

> ◆ **Kritik Nokta · Ad arama kuralı**
>
> Bir ad, kullanıldığı yerden başlayarak içten dışa doğru aranır: önce kendi bloğu, sonra çevreleyen bloklar, en son global kapsam. Bulunan ilk tanım geçerlidir. Gölgelenmiş bir değişken, gölgeleyen blok bitince eski hâliyle yeniden görünür.

Kural basittir: veriyi parametreyle veririz, sonucu `return` ile alırız. Global değişkeni yalnızca programın her yerinde gereken ve değişmeyen değerler (sabitler) için kullanırız.

9.9 static Yerel Değişken

Yerel değişkenler her çağrıda sıfırdan oluşur ve fonksiyon bitince silinir; bunun nedenini çağrı yığınında gördük. Bazen fonksiyonun çağrılar arasında bir değeri hatırlamasını isteriz: kaç kez çağrıldığını saymak gibi. Global değişken bir çözümdür ama onu herkes görür. Daha iyisi `static` yerel değişkendir: değişkenin başına `static` yazarız.

▸ staticSayac.c

```c
#include <stdio.h>

int normalSayac(void) {
    int sayac = 0;
    sayac++;
    return sayac;
}

int kaliciSayac(void) {
    static int sayac = 0;
    sayac++;
    return sayac;
}

int main(void) {
    for (int i = 1; i <= 4; i++) {
```

```
        int n = normalSayac();
        int k = kaliciSayac();
        printf("Çağrı %d: normal = %d, static = %d\n", i, n, k);
    }
    return 0;
}
```

```
ÇIKTI
Çağrı 1: normal = 1, static = 1
Çağrı 2: normal = 1, static = 2
Çağrı 3: normal = 1, static = 3
Çağrı 4: normal = 1, static = 4
```

`normalSayac` her çağrıda `sayac`'ı 0'dan başlatır, 1 yapar ve döndürür; hep 1 verir. `kaliciSayac`'taki `static int sayac = 0;` satırı ise yalnızca bir kez, program başlarken işlenir. Değişken fonksiyon bitince silinmez; sonraki çağrıda kaldığı değerle devam eder. `static` değişken çağrı yığınında değil, program boyunca yaşayan ayrı bir bölgede durur (Bölüm 13). İlk değeri sabit bir ifade olmak zorundadır (`static int n = 0;` olur, `static int n = x;` olmaz) ve verilmezse 0 olur.

Üç değişken türünü karşılaştıralım:

	Yerel (normal)	`static` yerel	Global
Ömrü	Fonksiyon çalışırken	Programın tamamı	Programın tamamı
Kim görür?	Tanımlandığı blok	Tanımlandığı blok	Tanımından sonraki bütün fonksiyonlar
İlk değer verilmezse	Çöp	0	0
İlk değer ne zaman verilir?	Her çağrıda	Bir kez	Bir kez

> **Not · Fonksiyon dışındaki static**
>
> `static` bir fonksiyonun ya da global değişkenin başına yazılırsa farklı bir anlam taşır: ad yalnızca o `.c` dosyasında görünür. Birden çok dosyalı programlarda yardımcı fonksiyonları gizlemek için kullanılır; Bölüm 15'te göreceğiz. Tek dosyayla çalıştığımız sürece pratik farkı yoktur.

9.10 return Olmadan Biten Fonksiyon

Dönüş tipi `void` olmayan bir fonksiyon her yolda bir değer döndürmelidir. Aşağıdaki `isaret` fonksiyonu, `n` sıfırsa hiçbir `return`'e uğramadan sona erer:

▸ returnYok.c

```
#include <stdio.h>

int isaret(int n) {
    if (n > 0) {
        return 1;
    } else if (n < 0) {
        return -1;
    }
}

int main(void) {
```

```
    printf("%d\n", isaret(7));
    return 0;
}
```

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra returnYok.c -o returnYok
returnYok.c: In function 'isaret':
returnYok.c:9:1: warning: control reaches end of non-void function [-Wreturn-type]
    9 | }
      | ^
```

Program derlenir, ama `isaret(0)` çağrılırsa fonksiyon değer döndürmeden biter ve çağıranın aldığı değer tanımsızdır. Bu yüzden `isaret(0)` için bir çıktı göstermiyoruz. Uyarıyı hata gibi ciddiye almalıyız. En sık neden, `return`'leri yalnızca `if` ve `else if` kollarına koyup "başka ihtimal olamaz" sanmaktır; derleyici ihtimalleri bizim kadar bilmez. Çözüm, fonksiyonun son satırını bir `return` yapmaktır:

```
int isaret(int n) {
    if (n > 0) {
        return 1;
    }
    if (n < 0) {
        return -1;
    }
    return 0;
}
```

`void` fonksiyonlarda bu sorun yoktur. `main`'in sonundaki `return 0;`'ın zorunlu olmaması da yalnızca `main`'e özgü bir kuraldır (Bölüm 4); başka hiçbir fonksiyon için geçerli değildir.

9.11 Hazır Fonksiyonlar

Standart kütüphane, sık gereken işler için hazır fonksiyonlar sunar. Her fonksiyonun prototipi bir başlık dosyasındadır ve kullanmak için o dosyayı `#include` etmemiz gerekir; başlığı unutursak `implicit declaration of function` hatasını alırız.

Başlık	İçerdiği fonksiyonlar
`<math.h>`	`sqrt`, `pow`, `fabs`, `ceil`, `floor`, `round`, `fmod`, `sin`, `cos`, `log`, `exp`, …
`<stdlib.h>`	`abs`, `rand`, `srand`, `exit`, … (dinamik bellek ve dönüştürme fonksiyonları sonraki bölümlerde)
`<ctype.h>`	`isdigit`, `isalpha`, `toupper`, `tolower`, …

math.h

`<math.h>` fonksiyonlarının çoğu `double` alır ve `double` döndürür. Bölüm 5'te `fabs`, `round`, `fmod`, `isnan` gibi fonksiyonlara söz vermiştik; hepsini bir arada görelim:

▸ matematik.c

```
#include <stdio.h>
#include <math.h>

int main(void) {
    printf("sqrt(144)    = %.1f\n", sqrt(144.0));
    printf("pow(2, 10)   = %.0f\n", pow(2.0, 10.0));
```

```
    printf("fabs(-3.7)   = %.1f\n", fabs(-3.7));
    printf("ceil(2.1)    = %.1f\n", ceil(2.1));
    printf("floor(-2.1)  = %.1f\n", floor(-2.1));
    printf("round(2.5)   = %.1f\n", round(2.5));
    printf("round(-2.5)  = %.1f\n", round(-2.5));
    printf("fmod(7.5, 2) = %.1f\n", fmod(7.5, 2.0));
    double pi = acos(-1.0);
    printf("sin(pi/2)    = %.1f\n", sin(pi / 2));
    printf("cos(pi)      = %.1f\n", cos(pi));
    printf("log(e)       = %.1f\n", log(exp(1.0)));
    printf("log10(1000)  = %.1f\n", log10(1000.0));
    printf("sqrt(-1) nan mi? %d\n", isnan(sqrt(-1.0)) != 0);
    return 0;
}
```

```
ÇIKTI
sqrt(144)    = 12.0
pow(2, 10)   = 1024
fabs(-3.7)   = 3.7
ceil(2.1)    = 3.0
floor(-2.1)  = -3.0
round(2.5)   = 3.0
round(-2.5)  = -3.0
fmod(7.5, 2) = 1.5
sin(pi/2)    = 1.0
cos(pi)      = -1.0
log(e)       = 1.0
log10(1000)  = 3.0
sqrt(-1) nan mi? 1
```

Bu çıktıdan çıkaracaklarımız şunlardır:

- `ceil` yukarı, `floor` aşağı yuvarlar; `round` en yakın tamsayıya yuvarlar ve yarımlarda sıfırdan uzağa gider (`2.5` ⟶ 3, `-2.5` ⟶ -3). Bunlar `int` değil `double` döndürür.
- `sin` ve `cos` açıyı **radyan** cinsinden ister. Dereceyi radyana çevirmek için `derece * pi / 180` kullanılır. C standardında hazır bir `pi` sabiti yoktur; `acos(-1.0)` π'yi verir.
- `log` doğal logaritmadır (e tabanı), `log10` onluk tabanlıdır.
- `sqrt` negatif sayıda `nan` döndürür; `isnan` bunu denetler (Bölüm 5).

Fonksiyonlar `double` ile çalıştığı için `pow(2, 10)` gibi bir çağrıda tamsayı argümanlar kendiliğinden `double`'a çevrilir. Yine de sonuç `double` olduğundan `printf` ile `%d` değil `%f` ailesi kullanılır. Büyük tamsayı kuvvetlerinde `pow`'un sonucu tam olmayabilir; tamsayı kuvveti için kendi fonksiyonumuzu aşağıda yazacağız.

Gelin bunları kendi fonksiyonumuzun içinde kullanalım: iki nokta arasındaki uzaklık ve dairenin alanı.

▸ mesafe.c

```
#include <stdio.h>
#include <math.h>

double mesafe(double x1, double y1, double x2, double y2) {
    double dx = x2 - x1;
    double dy = y2 - y1;
    return sqrt(dx * dx + dy * dy);
}

double daireAlani(double r) {
    const double pi = acos(-1.0);
    return pi * r * r;
```

```c
}

int main(void) {
    printf("Uzaklık: %.2f\n", mesafe(0, 0, 3, 4));
    printf("Uzaklık: %.2f\n", mesafe(1, 1, 4, 5));
    printf("Alan: %.2f\n", daireAlani(2.5));
    return 0;
}
```

```
ÇIKTI
Uzaklık: 5.00
Uzaklık: 5.00
Alan: 19.63
```

`mesafe(0, 0, 3, 4)` çağrısında `int` argümanlar parametrelerin tipine, yani `double`'a çevrilir; prototip görünür olduğu için bu dönüşüm güvenlidir.

> **▲ Sık Yapılan Hata · Linux'ta -lm unutulursa**
>
> `sqrt`, `pow` gibi fonksiyonların kodu ayrı bir matematik kütüphanesindedir. Bazı Linux derleyicilerinde `gcc program.c -o program` komutu `undefined reference to 'sqrt'` bağlama hatası verir. Çözüm komutun sonuna `-lm` eklemektir: `gcc -std=c17 -Wall -Wextra program.c -o program -lm`. Derleyici sabit argümanlı çağrıları kendisi hesaplayabildiği için hata her programda çıkmaz; ama çıkabileceğini bilmek gerekir.

stdlib.h: abs, rand, srand, exit

`abs` bir `int`'in mutlak değerini alır ve `<stdlib.h>` içindedir. `double` için `<math.h>` içindeki `fabs` kullanılır. `abs(-3.7)` yazarsak `-3.7` önce `int`'e çevrilir (`-3`) ve sonuç 3 olur; gcc bunu `-Wabsolute-value` uyarısıyla yakalar.

`rand()` her çağrıda 0 ile `RAND_MAX` arasında bir **sözde rastgele** (pseudo-random) tamsayı döndürür. `RAND_MAX` en az 32767'dir; Windows'ta (MinGW) 32767, Linux ve macOS'ta 2147483647'dir. Sayılar bir formülle üretildiği için aynı başlangıç değeri (**tohum**, seed) hep aynı diziyi verir. Tohumu `srand(tohum)` ile belirleriz. İstediğimiz aralığa sığdırmak için kalan operatörünü kullanırız:

İstenen	İfade	Neden?
0 ile 99 arası	`rand() % 100`	Kalan 0..99
Zar (1 ile 6)	`rand() % 6 + 1`	Kalan 0..5, `+ 1` ile 1..6
a ile b arası	`a + rand() % (b - a + 1)`	10..20 için `10 + rand() % 11`
0 ile 1 arası `double`	`(double)rand() / RAND_MAX`	Bölme `double` olur

`srand` hiç çağrılmazsa her çalıştırmada aynı dizi gelir. Sayıların her seferinde değişmesi için tohumu değişen bir değerden alırız; en yaygın kalıp şimdiki zamandır: `srand(time(NULL));`. `time` fonksiyonu `<time.h>` içindedir ve 1970'ten beri geçen saniye sayısını döndürür.

▸ zarAtma.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL));
```

```
    for (int i = 1; i <= 5; i++) {
        printf("Zar %d: %d\n", i, rand() % 6 + 1);
    }
    return 0;
}
```

```
ÇIKTI
Zar 1: 5
Zar 2: 6
Zar 3: 1
Zar 4: 2
Zar 5: 5
```

Bu çıktı örnektir; program her çalıştırıldığında farklı sayılar üretir. `rand()` işletim sistemlerinde farklı formüllerle gerçeklendiği için aynı tohumla bile farklı diziler çıkabilir. Değişmeyen şey, her sayının 1 ile 6 arasında olmasıdır. Bunu ve tohum davranışını çıktısı kesin olan bir programla da gösterebiliriz:

▸ tohum.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

bool zarlarAralikta(int adet) {
    for (int i = 0; i < adet; i++) {
        int zar = rand() % 6 + 1;
        if (zar < 1 || zar > 6) {
            return false;
        }
    }
    return true;
}

bool ayniDiziMi(unsigned tohum) {
    srand(tohum);
    int a1 = rand();
    int a2 = rand();
    srand(tohum);
    int b1 = rand();
    int b2 = rand();
    return a1 == b1 && a2 == b2;
}

int main(void) {
    printf("1000 zar da 1..6 arasında mı? %s\n",
           zarlarAralikta(1000) ? "evet" : "hayır");
    printf("Aynı tohum aynı diziyi verir mi? %s\n",
           ayniDiziMi(42) ? "evet" : "hayır");
    return 0;
}
```

```
ÇIKTI
1000 zar da 1..6 arasında mı? evet
Aynı tohum aynı diziyi verir mi? evet
```

> **▲ Sık Yapılan Hata · srand'ı döngüde çağırmak**
>
> `srand(time(NULL));` satırı programın başında bir kez çalışmalıdır. Döngünün içine koyarsak, `time` saniye çözünürlüklü olduğundan döngü bir saniyeden kısa sürdüğünde her tur aynı tohumla başlar ve `rand()` hep aynı sayıyı verir. Bir de `rand() % 6` yazıp `+ 1`'i unutursak zar 0 ile 5 arasında çıkar.

`<stdlib.h>` içindeki `exit(kod)` programı **herhangi bir fonksiyonun içinden** hemen bitirir. `main` içindeki `return kod;` ile aynı çıkış kodunu verir, ama `exit` her fonksiyondan çalışır. Başarı için `EXIT_SUCCESS`, başarısızlık için `EXIT_FAILURE` sabitleri kullanılabilir. Kurtarılamayan bir hata durumunda (ör. gerekli bir dosya yoksa) yaygın kalıp, mesajı yazıp `exit(EXIT_FAILURE);` çağırmaktır; Bölüm 13'te bellek ayrılamayınca bu kalıpla karşılaşacağız.

ctype.h: karakter sınıflandırma

`<ctype.h>` tek karakterle çalışan fonksiyonlar verir. `isdigit`, `isalpha`, `isupper`, `islower`, `isspace` karakterin türünü sorar; sıfırdan farklı (doğru) ya da 0 (yanlış) döndürür. `toupper` ve `tolower` büyük/küçük harf dönüşümü yapar. Bölüm 5'te `kucuk - 'a' + 'A'` ile elle yaptığımız işi bunlar daha okunaklı ve güvenli yapar. Girdi bitene kadar karakter okuyup sınıflandıran bir program yazalım:

▸ harfSay.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    int harf = 0, rakam = 0, bosluk = 0, diger = 0;
    int c = 0;
    printf("Bir satır yazın: ");
    while ((c = getchar()) != '\n' && c != EOF) {
        if (isalpha(c)) {
            harf++;
        } else if (isdigit(c)) {
            rakam++;
        } else if (isspace(c)) {
            bosluk++;
        } else {
            diger++;
        }
    }
    printf("harf=%d rakam=%d boşluk=%d diğer=%d\n",
           harf, rakam, bosluk, diger);
    printf("%c -> %c\n", 'g', toupper('g'));
    return 0;
}
```

```
ÇIKTI
Bir satır yazın: Merhaba Dunya 2024!
harf=12 rakam=4 boşluk=2 diğer=1
g -> G
```

`isalpha` yalnızca ASCII harflerini tanır; `ç`, `ğ` gibi Türkçe harfler çok baytlı karakterler olduğundan bu programda "diğer" sayılır (Bölüm 11'de ayrıntısına gireceğiz). `c` değişkeninin `char` değil `int` olması, `getchar`'ın `EOF` dönebilmesindendir (Bölüm 6).

9.12 Algoritmaları Fonksiyona Dönüştürmek

Bölüm 8'de klasik algoritmaları `main` içinde yazmıştık. Şimdi bunları fonksiyonlara çevirelim. Tarif basittir: algoritmanın girdileri parametre olur, sonucu `return` ile döner; `scanf` ve `printf` fonksiyonun dışında, `main`'de kalır.

▸ algoritmalar.c

```c
#include <stdio.h>
#include <stdbool.h>

bool asalMi(int n) {
    if (n < 2) {
        return false;
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}

int ebob(int a, int b) {
    while (b != 0) {
        int kalan = a % b;
        a = b;
        b = kalan;
    }
    return a;
}

int ekok(int a, int b) {
    return a / ebob(a, b) * b;
}

long long faktoriyel(int n) {
    long long sonuc = 1;
    for (int i = 2; i <= n; i++) {
        sonuc *= i;
    }
    return sonuc;
}

int basamakToplami(int n) {
    int toplam = 0;
    while (n > 0) {
        toplam += n % 10;
        n /= 10;
    }
    return toplam;
}

long long ustAl(int taban, int us) {
    long long sonuc = 1;
    for (int i = 0; i < us; i++) {
        sonuc *= taban;
    }
    return sonuc;
}

int main(void) {
    printf("2'den 30'a asallar:");
```

```
    for (int n = 2; n <= 30; n++) {
        if (asalMi(n)) {
            printf(" %d", n);
        }
    }
    printf("\n");
    printf("ebob(48, 18) = %d\n", ebob(48, 18));
    printf("ekok(4, 6) = %d\n", ekok(4, 6));
    printf("20! = %lld\n", faktoriyel(20));
    printf("basamakToplami(4712) = %d\n", basamakToplami(4712));
    printf("2^10 = %lld\n", ustAl(2, 10));
    return 0;
}
```

```
ÇIKTI
2'den 30'a asallar: 2 3 5 7 11 13 17 19 23 29
ebob(48, 18) = 6
ekok(4, 6) = 12
20! = 2432902008176640000
basamakToplami(4712) = 14
2^10 = 1024
```

Bu fonksiyonlardan öğrendiklerimiz:

- `asalMi` bir `bool` döndürür. Bölüm 8'de bir bayrağı `false` yapıp `break` ile çıkıyorduk; fonksiyonda bölen bulunca doğrudan `return false;` yazarız, bayrağa ve `break`'e gerek kalmaz. Döngü bölensiz biterse `return true;` çalışır. Kullanırken `if (asalMi(n))` yeterlidir, `== true` yazmayız.
- `ebob` parametreleri doğrudan değiştirir (`a = b; b = kalan;`). Parametreler zaten birer kopya olduğundan çağıranın değişkenleri korunur; değerle geçirmenin işe yaradığı yer burasıdır. `basamakToplami` de aynı şekilde `n`'i eritir ve `main`'deki sayı bozulmaz.
- `ekok`, başka bir fonksiyonu çağırır. Fonksiyonlar birbirini kullanarak büyür. İfadede önce bölmeyi yapmamız, ara sonucun taşma ihtimalini azaltır: `a * b / ebob(a, b)` yerine `a / ebob(a, b) * b`.
- `faktoriyel` `long long` döndürür; 20! sığar, 21! sığmaz (Bölüm 5). `faktoriyel(0)` döngüye girmez ve 1 verir.
- `ustAl` tamsayı kuvvetini döngüyle hesaplar ve negatif olmayan üs varsayar. `pow` `double` döndürür; kendi fonksiyonumuz tamsayılarda tam sonuç verir.

> **İpucu · Dönüş tipini fonksiyonun adından seçin**
>
> "...mı?" diye sorulan fonksiyonlar `bool` döndürür (`asalMi`, `ciftMi`, `mukemmelMi`). "Kaç tane?" ve "ne kadar?" soruları `int` ya da `long long`, ortalama gibi ondalıklı sonuçlar `double` döndürür. Yazdırma ve menü gibi iş yapanlar `void` olur. Fonksiyonun adını soru gibi okuyunca dönüş tipi çoğu zaman kendiliğinden ortaya çıkar.

9.13 Fonksiyonlarla Program Tasarımı: Hesap Makinesi

Fonksiyonların "böl ve yönet" gücü, menülü programlarda en iyi görünür. `main` yalnızca ana döngüyü yönetir; her seçenek kendi fonksiyonuna devredilir. Şimdi dört işlem yapan bir hesap makinesi yazalım. Tasarım şöyle:

- `menuyuGoster` menüyü yazar (`void`).
- `sayiOku` bir ondalıklı sayı okur ve hatalı girdiyi kendisi halleder.
- `islemYap` seçilen işlemi iki sayıyla uygular ve sonucu yazar.
- `main` yalnızca menü döngüsüdür.

▸ hesapMakinesi.c

```c
#include <stdio.h>

void menuyuGoster(void);
double sayiOku(const char *istek);
void islemYap(int secim);

int main(void) {
    int secim = -1;
    do {
        menuyuGoster();
        secim = (int)sayiOku("Seçim: ");
        if (secim >= 1 && secim <= 4) {
            islemYap(secim);
        } else if (secim != 0) {
            printf("Geçersiz seçim!\n");
        }
    } while (secim != 0);
    printf("Güle güle!\n");
    return 0;
}

void menuyuGoster(void) {
    printf("\n1) Topla  2) Çıkar  3) Çarp  4) Böl  0) Çıkış\n");
}

double sayiOku(const char *istek) {
    double sayi = 0.0;
    printf("%s", istek);
    while (scanf("%lf", &sayi) != 1) {
        int c = 0;
        while ((c = getchar()) != '\n' && c != EOF) {
            // hatalı girdiyi satır sonuna kadar at
        }
        if (c == EOF) {
            return 0.0;
        }
        printf("Sayı girin: ");
    }
    return sayi;
}

double topla(double a, double b) {
    return a + b;
}

double cikar(double a, double b) {
    return a - b;
}

double carp(double a, double b) {
    return a * b;
}

void islemYap(int secim) {
    double a = sayiOku("Birinci sayı: ");
    double b = sayiOku("İkinci sayı: ");
    switch (secim) {
        case 1:
            printf("Sonuç: %.2f\n", topla(a, b));
            break;
        case 2:
            printf("Sonuç: %.2f\n", cikar(a, b));
```

```
            break;
        case 3:
            printf("Sonuç: %.2f\n", carp(a, b));
            break;
        case 4:
            if (b == 0) {
                printf("Sıfıra bölünemez!\n");
            } else {
                printf("Sonuç: %.2f\n", a / b);
            }
            break;
    }
}
```

```
ÇIKTI
1) Topla  2) Çıkar  3) Çarp  4) Böl  0) Çıkış
Seçim: 1
Birinci sayı: 12.5
İkinci sayı: 7
Sonuç: 19.50

1) Topla  2) Çıkar  3) Çarp  4) Böl  0) Çıkış
Seçim: 4
Birinci sayı: 9
İkinci sayı: 0
Sıfıra bölünemez!

1) Topla  2) Çıkar  3) Çarp  4) Böl  0) Çıkış
Seçim: x
Sayı girin: 7
Geçersiz seçim!

1) Topla  2) Çıkar  3) Çarp  4) Böl  0) Çıkış
Seçim: 0
Güle güle!
```

Programdaki tasarım kararları şunlardır:

1. `main` kısa ve okunaklıdır; ne yaptığı bir bakışta anlaşılır.
2. `sayiOku`, `scanf` işini ve hata denetimini tek yerde toplar. `scanf` sayı okuyamazsa (`x` yazıldı) tamponu boşaltır ve yeniden sorar (Bölüm 6). Bu denetim olmasaydı `x` tamponda kalır, `sayi` ilk değeri olan 0'da kalır ve menüde 0 "Çıkış" olduğu için program habersizce kapanırdı. Girdi tamamen biterse (`EOF`) 0 döndürür ve program düzgünce çıkar.
3. Her işlem ayrı bir fonksiyondadır; yeni bir işlem eklemek için bir `case` ve bir fonksiyon yazarız, mevcut kod bozulmaz.
4. `topla`, `cikar` ve `carp` tanımları kullanıldıkları `islemYap`'tan önce yazıldığı için prototip gerektirmedi; `menuyuGoster`, `sayiOku` ve `islemYap` ise `main`'den sonra tanımlandıkları için dosyanın başında tanıtıldı.

`sayiOku` parametresindeki `const char *istek`, ekrana yazılacak metindir. İşaretçileri Bölüm 12'de öğrenene kadar bunu "bir metin alır" diye okuyabiliriz; çağrıda `"Seçim: "` gibi bir metin yazarız.

main de Bir Fonksiyondur

`main` ayrıcalıklı ama sıradan bir fonksiyondur: dönüş tipi `int`, parametre listesi `(void)`'tir. Onu işletim sistemi çağırır ve `return` ile verdiği değer programın çıkış kodu olur (Bölüm 4). `main` içindeki `return`

programı bitirir; bu yüzden hata durumunda `return 1;` ile erken ve hatalı çıkış yapabiliriz. Normal akışta `return 0;` yazarız. Komut satırından argüman alan `main` biçimini Bölüm 12'de göreceğiz.

Üç Dilde · Fonksiyon aşırı yükleme ve metotlar *C · C++ · C#*

C'de aynı adlı iki fonksiyon olamaz. Farklı tiplerle çalışan benzer işler için ad değiştirmek gerekir:

```c
int topla(int a, int b) { return a + b; }
double toplaD(double a, double b) { return a + b; }   // farklı ad şart
```

C++ aynı adlı birden çok fonksiyona izin verir (**aşırı yükleme**, overloading); derleyici argümanların tiplerine bakıp doğrusunu seçer. Ayrıca parametrelere varsayılan değer verilebilir. Ayrıntılarını Bölüm 25'te göreceğiz:

```cpp
int topla(int a, int b) { return a + b; }
double topla(double a, double b) { return a + b; }
int artir(int n, int adim = 1) { return n + adim; }   // varsayılan argüman
```

C#'ta fonksiyonlara **metot** denir ve her metot bir sınıfın içindedir (top-level statements kullanırken derleyici ana sınıfı bizim yerimize kurar). C# aşırı yüklemeyi, isteğe bağlı parametreleri ve adlandırılmış argümanları destekler. Ayrıca bir metodun tanımlandığı yerin çağrıdan önce ya da sonra olması fark etmez; prototip kavramı yoktur. Metotları Bölüm 46'da işleyeceğiz:

```csharp
class Hesap {
    public static int Topla(int a, int b) => a + b;
    public static double Topla(double a, double b) => a + b;
}

Console.WriteLine(Hesap.Topla(b: 2, a: 1));   // adlandırılmış argüman
```

Üç dilde de argümanlar varsayılan olarak değerle geçirilir; adresle (C'de işaretçiyle, C++'ta referansla, C#'ta `ref` / `out` ile) geçirme için ayrı araçlar vardır.

9.14 Sık Yapılan Hatalar

Bu bölümde karşılaştığımız tuzakları toplu olarak hatırlayalım:

Hata	Belirti	Çözüm
Fonksiyonu tanıtmadan çağırmak	`implicit declaration of function`	Tanımı önce yazmak ya da prototip eklemek
Prototip ile tanımın uyuşmaması	`conflicting types for ...`	Dönüş tipi ve parametreleri aynı yazmak
Argüman sayısı yanlış	`too few` / `too many arguments`	Tanımdaki parametre sayısına uymak
Dönüş değerini kullanmamak	Sonuç değişmiyor (`kareAl(x);`)	`x = kareAl(x);`
`takas(x, y)` ile değişken değiştirmeyi beklemek	Değişkenler aynı kalır	Değerle geçirme; sonucu `return` ile almak
`return` olmayan yol	`control reaches end of non-void function`	Fonksiyonun sonuna `return` koymak
`(a + b) / 2` ile `double` ortalama	Ondalık kısım kaybolur	`/ 2.0` yazmak

Hata	Belirti	Çözüm
Global değişkene aşırı bağımlılık	Hata aramak zorlaşır	Parametre ve `return` kullanmak
`srand`'ı döngüde çağırmak	Hep aynı sayı	Programın başında bir kez çağırmak
`int menu()` yazmak	Hatalı çağrılar yakalanmaz	`int menu(void)`

Bölüm Özeti

- Fonksiyon, `dönüşTipi ad(parametreler) { gövde }` biçiminde tanımlanır; tekrarı önler, okunurluğu artırır ve problemi parçalara böler.
- `void` fonksiyon değer döndürmez (`return;` ile erken çıkabilir) ve çağrısı bir değişkene atanamaz. Diğer fonksiyonlar `return` ile en fazla bir değer verir; `return` çalıştığı anda fonksiyon biter.
- Fonksiyon çağrılmadan önce tanıtılmış olmalıdır: tanımı `main`'den önce yazarız ya da prototip (`int f(int);`) ekleriz. Prototipsiz çağrı `implicit declaration of function`, uyuşmayan prototip `conflicting types` hatasıdır.
- Parametre tanımdaki, argüman çağrıdaki değerdir. Sayı ve sıra uyuşmalıdır; tip kendiliğinden çevrilir (`double` → `int` ondalığı atar). `(a + b) / 2` tamsayı bölmesidir.
- C'de argümanlar **değerle** geçirilir: fonksiyon kopya alır, `takas(x, y)` çağıranın değişkenlerini değiştirmez (çözümü Bölüm 12'de).
- Her çağrı çağrı yığınında kendi çerçevesini açar; fonksiyon bitince çerçeve silinir. İç içe çağrıda içteki önce çalışır.
- Yerel değişken yalnızca kendi bloğunda yaşar; iç blokta aynı ad dıştakini gölgeler. Global değişken her yerden görünür, ilk değeri 0'dır ve az kullanılmalıdır.
- `static` yerel değişken çağrılar arasında değerini korur; ilk değeri bir kez verilir, verilmezse 0'dır.
- Değer döndüren fonksiyonun her yolu `return` ile bitmelidir; yoksa `-Wreturn-type` uyarısı verilir ve dönüş değeri tanımsızdır.
- `<math.h>` fonksiyonları `double` ile çalışır (Linux'ta gerekirse `-lm`); açılar radyandır. `abs` `int`, `fabs` `double` içindir.
- `rand() % n` 0..n-1 verir (zar: `rand() % 6 + 1`); `srand(time(NULL))` programın başında bir kez çağrılır; `rand` çıktısı sistemlere göre değişir.
- Algoritmayı fonksiyona çevirirken girdiler parametre, sonuç `return` olur; `printf` ve `scanf` `main`'de kalır. "...mı?" fonksiyonları `bool` döndürür.
- `main` de bir fonksiyondur: `return 0;` başarıyı, sıfırdan farklı değer hatayı bildirir; `exit` programı her fonksiyondan bitirir.
- C'de aşırı yükleme yoktur; C++ ve C# aynı adlı birden çok fonksiyonu destekler.

Alıştırmalar

1. `double ortalama(int a, int b)` fonksiyonu `main`'den sonra tanımlanmıştır. `main` içinde sorunsuz çağrılabilmesi için `main`'den önce hangi satır yazılmalıdır?

A) `double ortalama;`
B) `ortalama(int a, int b);`
C) `double ortalama(int, int);`
D) `void ortalama(int a, int b);`
E) `double ortalama(int a, int b) { }`

2. C'de bir fonksiyona argüman geçirilmesiyle ilgili hangisi doğrudur?

A) Fonksiyon argümanın bir kopyasını alır; parametreyi değiştirmek çağıran fonksiyondaki değişkeni değiştirmez.
B) Parametre fonksiyon içinde değiştirilirse çağıran fonksiyondaki değişken de aynı değeri alır.
C) Yalnızca `void` fonksiyonlarda yapılan değişiklikler çağıranı etkiler.
D) Fonksiyonlara yalnızca sabit değerler argüman olarak geçirilebilir; değişken geçirilemez.
E) Parametre adı çağırandaki değişkenle aynıysa ikisi aynı bellek hücresini paylaşır.

3. Aşağıdaki `say` fonksiyonu `main` içinde art arda üç kez çağrılıp döndürdüğü değerler sırayla yazdırılırsa ekranda ne görülür?

```c
int say(void) {
    static int n = 10;
    n += 5;
    return n;
}
```

A) `15 15 15`
B) `10 15 20`
C) `15 25 35`
D) `10 10 10`
E) `15 20 25`

4. `int topla(int a, int b) { return a + b; }` tanımıyla birlikte, tanımdan önce yazılan aşağıdaki prototiplerden hangisi derleme hatasına yol açar?

A) `int topla(int a, int b);`
B) `int topla(int x, int y);`
C) `int topla(int, int);`
D) `int topla(int a, int b, int c);`
E) Hiçbiri; hepsi geçerlidir.

5. Dönüş tipi `void` olan bir fonksiyonla ilgili hangisi doğrudur?

A) `return 5;` yazarak değer döndürebilir.
B) `return;` yazılarak erken bitirilebilir.
C) Çağrısı bir `int` değişkene atanabilir.
D) `main` dışında `void` fonksiyon tanımlanamaz.
E) Parametre alamaz.

6. Aşağıdaki ifadelerden hangisi, bir `static` yerel değişkenin özelliklerini doğru anlatır?

A) Her çağrıda ilk değerinden yeniden başlar.
B) Fonksiyonun dışındaki bütün fonksiyonlar tarafından görülür.
C) Program boyunca yaşar, ilk değeri bir kez verilir ve yalnızca tanımlandığı bloktan erişilir.

D) Çağrı yığınındaki çerçevede saklanır ve fonksiyon bitince silinir.
E) İlk değer verilmezse çöp değer taşır.

7. Programın çıktısı nedir?

```
int biriktir(int n) {
    static int toplam = 0;
    int tur = 0;
    toplam += n;
    tur += n;
    printf("%d/%d ", toplam, tur);
    return toplam;
}

int main(void) {
    biriktir(2);
    biriktir(3);
    biriktir(5);
    printf("\n");
    return 0;
}
```

8. Programın çıktısı nedir?

```
int f(int x) {
    x = x * 2;
    return x + 1;
}

void g(int x) {
    x = x + 100;
}

int main(void) {
    int a = 4;
    int b = f(a);
    g(a);
    printf("%d %d\n", a, b);
    printf("%d\n", f(f(a)));
    return 0;
}
```

9. Programın çıktısı nedir? (Her sayıdan sonra bir boşluk olduğuna, son satırın ise `\n` ile bittiğine dikkat edin.)

```
int g = 2;

void artir(void) {
    g = g + 1;
}

void gizli(void) {
    int g = 50;
    g = g + 1;
    printf("%d ", g);
}

int main(void) {
    artir();
    gizli();
```

```
    artir();
    {
        int g = 7;
        artir();
        printf("%d ", g);
    }
    printf("%d\n", g);
    return 0;
}
```

10. Programın çıktısı nedir?

```c
int f(int n) {
    return n + 1;
}

int g(int n) {
    return f(n) * 2;
}

int main(void) {
    printf("%d\n", g(f(3)) + f(g(3)));
    return 0;
}
```

11. Aşağıdaki program derlenmiyor. Hataları bulun, nedenlerini açıklayın ve düzeltin.

```c
#include <stdio.h>

int ucKati(int n);

void yazdir(int n) {
    printf("%d\n", n);
}

int main(void) {
    int s = yazdir(ucKati(4));
    printf("%d\n", s);
    return 0;
}

int ucKati(int n, int m) {
    return n * 3;
}
```

12. Aşağıdaki fonksiyon, `kisim` değerinin `toplam` içindeki yüzdesini hesaplamak için yazılmıştır. `yuzde(1, 4)` için 25.0 beklenirken 0.0 döndürülüyor. Nedenini açıklayın ve iki ayrı yoldan düzeltin.

```c
double yuzde(int kisim, int toplam) {
    return kisim / toplam * 100;
}
```

13. Üç tamsayı okuyan, `int enBuyuk(int a, int b, int c)` fonksiyonuyla en büyüğünü bulan ve yazdıran bir program yazın. Fonksiyon yazdırma ya da okuma yapmasın.
14. Kenar uzunlukları `a`, `b`, `c` olan bir üçgenin alanını Heron formülüyle (`s = (a + b + c) / 2`, alan = `sqrt(s * (s - a) * (s - b) * (s - c))`) hesaplayan bir program yazın. `bool ucgenMi(double a, double b, double c)` fonksiyonu kenarların üçgen oluşturup oluşturmadığını (her kenar diğer ikisinin

toplamından küçük) denetlesin; `double ucgenAlani(...)` alanı hesaplasın. Üçgen oluşmuyorsa uygun bir mesaj yazdırın.

15. Her çağrıda sıradaki numarayı döndüren `int siradakiNumara(void)` fonksiyonunu `static` değişkenle yazın: ilk çağrı 1001, ikincisi 1002, … Fonksiyonu `main`'de üç kez çağırıp numaraları yazdırın.
16. Okunan bir `n` sayısına kadar olan asal sayıların kaç tane olduğunu ve toplamını bulan bir program yazın. `bool asalMi(int n)` ve `int asalSayisi(int ust)` fonksiyonlarını kullanın; asalların toplamı için üçüncü bir fonksiyon (`long long asalToplami(int ust)`) yazın. Örnek: `n = 30` için 10 asal vardır ve toplamları 129'dur.

BÖLÜM 10

Diziler

BU BÖLÜMDE

- Dizinin ne olduğunu, nasıl tanımlandığını ve elemanlarının bellekte ardışık durduğunu
- İlk değer listelerini: eksik elemanların 0 olmasını, `{0}` kalıbını, `int a[] = {...}` ve belirlenmiş başlatıcıları
- İndeksin 0'dan başladığını, son indeksin `N-1` olduğunu ve C'nin sınır dışı erişimi denetlemediğini
- `sizeof(a) / sizeof(a[0])` ile eleman sayısını bulmayı ve bunun fonksiyon içinde neden çalışmadığını
- Diziyi döngüyle doldurmayı ve dolaşmayı; toplam, ortalama, en büyük/en küçük ve sayma kalıplarını
- Diziyi fonksiyona geçirmeyi, değişikliğin çağırana yansımasını ve `const` parametreyi
- Kopyalama, karşılaştırma, ters çevirme, kaydırma, doğrusal ve ikili arama, kabarcık ve seçmeli sıralamayı
- İki boyutlu dizileri (matris), değişken uzunluklu dizileri ve üç dildeki dizi anlayışını

Elli öğrencinin notunu tutan bir program düşünelim. Şimdiye kadar öğrendiklerimizle `int not1, not2, ..., not50;` diye elli değişken tanımlamamız, ortalamayı bulmak için de elli terimli bir toplama yazmamız gerekirdi. Döngü (Bölüm 8) burada işe yaramaz, çünkü `not1`, `not2` adlarını bir döngüye sokamayız: "i'inci değişken" demenin bir yolu yok.

Bu bölümün konusu, bu sorunu çözen **dizi**dir. Önce dizinin tanımını, bellekteki yerleşimini ve C'nin sınır denetimi yapmamasının sonuçlarını göreceğiz. Sonra diziler üzerinde en sık kullanılan algoritmaları yazacağız: toplam, en büyük eleman, arama, ters çevirme, sıralama. Bölümün sonunda iki boyutlu dizilere (matrislere) geçeceğiz.

10.1 Neden Dizi? Tanımlama ve Bellek

> **Tanım: Dizi**
>
> Aynı tipten birçok değeri tek bir adla, bellekte art arda tutan veri yapısıdır. Her değere dizinin **elemanı** denir ve elemana sıra numarasıyla, yani **indeksiyle** (index) ulaşılır.

Dizi tanımının genel biçimi şöyledir:

```
tip diziAdi[boyut];
```

- **tip**: Bütün elemanların ortak tipi. `int notlar[50];`, `double fiyat[10];`, `char harf[26];` geçerli tanımlardır (karakter dizilerini Bölüm 11'de ayrıca ele alacağız).

- **boyut**: Eleman sayısı. Derleme sırasında bilinen, sıfırdan büyük bir tamsayı sabiti olmalıdır. `int a[2.5];` geçersizdir.

Boyutu doğrudan sayı olarak yazmak yerine `#define` ile adlandırmak iyi bir alışkanlıktır (Bölüm 5'te sabitleri konuşurken önizlemiştik). Böylece boyut tek yerde durur ve döngü sınırları da aynı adı kullanır. Dizi tanımlandıktan sonra büyüyemez ve küçülemez.

▸ ilkDizi.c

```c
#include <stdio.h>

#define N 4

int main(void) {
    int notlar[N];
    notlar[0] = 70;
    notlar[1] = 85;
    notlar[2] = 90;
    notlar[3] = 55;

    int i = 2;
    printf("İlk not: %d, son not: %d\n", notlar[0], notlar[N - 1]);
    printf("notlar[i - 1] + notlar[i + 1] = %d\n", notlar[i - 1] + notlar[i + 1]);
    printf("Dizi: %zu bayt\n", sizeof(notlar));
    return 0;
}
```

ÇIKTI

```
İlk not: 70, son not: 55
notlar[i - 1] + notlar[i + 1] = 140
Dizi: 16 bayt
```

`notlar[2]` sıradan bir `int` değişkeni gibi kullanılır: `notlar[2] = 90;`, `notlar[2]++` ve `scanf("%d", &notlar[2])` geçerlidir. İndeks sabit olmak zorunda değildir; `notlar[i + 1]` gibi her tamsayı ifadesi olabilir. Önce indeks ifadesi hesaplanır, sonra o eleman kullanılır.

`sizeof(notlar)` bütün dizinin bayt sayısını verir: 4 eleman, her biri 4 bayt, toplam 16 bayt. Elemanlar bellekte boşluksuz, art arda durur:

```
 notlar[0]   notlar[1]   notlar[2]   notlar[3]
+-----------+-----------+-----------+-----------+
|    70     |    85     |    90     |    55     |
+-----------+-----------+-----------+-----------+
 0. bayt      4. bayt     8. bayt     12. bayt
 (başlangıçtan uzaklık)
```

İndeksin neden 0'dan başladığı da bu çizimden anlaşılır: indeks, elemanın dizinin başlangıcından **kaç eleman uzakta** olduğunu söyler. İlk eleman başlangıçta durduğu için uzaklığı 0'dır. `notlar[i]` elemanının yeri "başlangıç + i × 4 bayt" işlemiyle bulunur; bu yüzden dizi erişimi çok hızlıdır. Aynı düzen, Bölüm 12'de işaretçileri anlamamızın anahtarı olacak.

Boyut sabit ifade olmalıdır

Boyutu `const` bir değişkenle vermek C'de sabit ifade sayılmaz. `const` bir değişkeni yalnızca salt okunur yapar; derleme sırasında değeri bilinen bir sabite dönüştürmez:

▸ sabitBoyut.c

```c
const int N = 5;
int a[N];

int main(void) {
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
sabitBoyut.c:2:5: error: variably modified 'a' at file scope
    2 | int a[N];
      |     ^
```

Dosya düzeyinde (global) böyle bir tanım geçersizdir. Fonksiyonun içinde aynı yazım derlenir, ama dizi bu kez "değişken uzunluklu dizi" olur; bunu bölümün sonunda göreceğiz. Güvenilir çözüm `#define N 5` ya da adsız bir `enum { N = 5 };` yazmaktır (`enum`'u Bölüm 14'te anlatacağız).

10.2 İlk Değer Verme

Dizinin elemanlarına tanım satırında, süslü parantez içinde bir **ilk değer listesi** verebiliriz. Bütün biçimleri tek programda görelim:

▸ ilkDegerler.c

```c
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int b[5] = {1, 2, 3};
    int c[5] = {0};
    int d[5] = {7};
    int e[] = {4, 8, 15, 16, 23, 42};

    printf("%s %4s %4s %4s %4s\n", "i", "a", "b", "c", "d");
    for (int i = 0; i < 5; i++) {
        printf("%d %4d %4d %4d %4d\n", i, a[i], b[i], c[i], d[i]);
    }
    printf("e dizisi %zu elemanlı, e[5] = %d\n",
           sizeof(e) / sizeof(e[0]), e[5]);
    return 0;
}
```

ÇIKTI

```
i    a    b    c    d
0   10    1    0    7
1   20    2    0    0
2   30    3    0    0
3   40    0    0    0
4   50    0    0    0
e dizisi 6 elemanlı, e[5] = 42
```

Çıktıdan şu kuralları çıkarıyoruz:

- Liste boyuttan kısaysa kalan elemanlar **0** olur (`b`). Çöp değer değil, sıfırdır.
- `{0}` bütün diziyi sıfırlar (`c`): ilk eleman 0'dır, kalanlar da eksik oldukları için 0 olur.

- `{7}` bütün elemanları 7 yapmaz (`d`): yalnızca ilk eleman 7'dir, kalanlar 0'dır.
- Boyut yazılmazsa derleyici liste uzunluğunu boyut yapar (`e`: 6 eleman, indeksler 0-5).
- Liste yalnızca tanım satırında yazılabilir. Tanımdan sonra `a = {1, 2, 3};` yazılamaz; bunun nedenini kopyalama başlığında göreceğiz.
- Listede boyuttan fazla eleman olması (`int x[3] = {1, 2, 3, 4};`) derleyici uyarısıdır (`excess elements in array initializer`).

> ◆ **Kritik Nokta · `{1}` ile `{0}`**
>
> `int a[5] = {1};` dizisi `1 0 0 0 0` olur, `int a[5] = {0};` dizisi `0 0 0 0 0`. "Liste kısa olunca kalan elemanlar çöp değer alır" cümlesi yanlıştır; listenin vermediği her eleman 0 olur.

İlk değer listesi hiç yazılmayan **yerel** bir dizi, Bölüm 5'teki ilk değersiz değişken gibi çöp değer taşır. `int x[5]; printf("%d", x[0]);` tanımsız davranıştır ve çıktısı çalıştırmadan çalıştırmaya değişebilir. Dizi, okunmadan önce mutlaka değer almalıdır (liste ya da döngüyle). Global ve `static` diziler bu kuralın dışındadır: program başlarken 0 ile başlarlar.

Belirlenmiş başlatıcılar

C99 ile gelen **belirlenmiş başlatıcılar** (designated initializers), yalnızca belirli indekslere değer vermemizi sağlar. Geri kalan elemanlar yine 0 olur:

▸ belirliBaslatici.c

```c
#include <stdio.h>

int main(void) {
    int haftaIci[7] = {[0] = 8, [4] = 6, [6] = 2};
    int aylar[] = {[11] = 31};

    for (int i = 0; i < 7; i++) {
        printf("%d ", haftaIci[i]);
    }
    printf("\n");
    printf("aylar dizisi %zu elemanlı\n", sizeof(aylar) / sizeof(aylar[0]));
    return 0;
}
```

```
ÇIKTI
8 0 0 0 6 0 2
aylar dizisi 12 elemanlı
```

Boyut yazılmadığında dizi, verilen en büyük indeksi kapsayacak kadar büyür: `[11]` verildiği için `aylar` 12 elemanlıdır. Bu söz dizimi seyrek dolu tablolarda ve ileride `enum` değerleriyle indekslenen tablolarda işimize yarayacak.

10.3 İndeks ve Sınır Dışı Erişim

N elemanlı bir dizide geçerli indeksler 0'dan `N-1`'e kadardır. `int a[5]` için `a[5]`, `a[-1]` ve `a[100]` dizinin dışındaki belleğe uzanır. C bunu **denetlemez**; aşağıdaki satırlar sorunsuz derlenir:

```c
int a[5] = {10, 20, 30, 40, 50};
a[5] = 99;                  // geçersiz: son indeks 4
a[-1] = 7;                  // geçersiz: indeks negatif
printf("%d\n", a[100]);     // geçersiz: okumak bile
```

Bu kodun davranışı **tanımsızdır** (Bölüm 5). Olabilecekler şunlardır:

- Okumada çöp bir değer ya da komşu bir değişkenin değeri gelir.
- Yazmada komşu bir değişken sessizce bozulur; hata, programın ilgisiz bir yerinde, çok sonra ortaya çıkar.
- Program çöker (Linux ve macOS'ta `Segmentation fault` mesajıyla; Windows'ta çoğu zaman uyarısız kapanır).
- Hiçbir şey olmamış gibi doğru çalışıyormuş izlenimi verir. En tehlikeli sonuç budur.

Aşağıdaki çizimde `a[5] = 99;` komutunun neden zararlı olduğunu görüyoruz. Diziden hemen sonra başka bir değişkenin durduğunu varsayalım (gerçek yerleşim derleyiciye bağlıdır):

```
 a[0]  a[1]  a[2]  a[3]  a[4]  |  toplam
+-----+-----+-----+-----+-----+-----------+
| 10  | 20  | 30  | 40  | 50  |   150     |
+-----+-----+-----+-----+-----+-----------+
                              ^
                              a[5] burayı gösterir:
                              yazarsak "toplam" bozulur
```

Birçok gerçek güvenlik açığı (arabellek taşması, buffer overflow) dizi sınırını aşan yazmalardan doğmuştur. Sınır denetimi bu dilde derleyicinin değil programcının işidir; Java ve Python gibi dillerde sınır dışı erişim programı hata ile durdurur, C'de durdurmaz. Yüksek eniyileme düzeylerinde `gcc` bazı sabit indeks hatalarını `-Warray-bounds` uyarısıyla yakalayabilir, ama buna güvenemeyiz.

> **▲ Sık Yapılan Hata · Döngüde `<=` yazmak**
>
> Sınır taşmasının en yaygın nedeni, Bölüm 8'de gördüğümüz bir eksik/fazla (off-by-one) hatasıdır. `for (int i = 0; i <= N; i++)` yazarsak son turda `i == N` olur ve `a[N]` dizinin dışına çıkar. Doğru kalıp `for (int i = 0; i < N; i++)` biçimidir. `int a[10]` dizisinin son elemanı `a[9]`'dur; `a[10]` yoktur.

Dizi elemanına `a[i, j]` biçiminde de erişilmez. C'de `i, j` ifadesi virgül operatörüdür: önce `i` hesaplanıp atılır, sonuç `j` olur; yani yazdığımız `a[j]` anlamına gelir. İki indeks için ayrı köşeli parantez kullanılır (`m[i][j]`), bunu iki boyutlu dizilerde göreceğiz.

> **Üç Dilde · Sınır denetimi üç dilde** *C · C++ · C#*
>
> C'de sınır dışı erişim tanımsız davranıştır. İleride göreceğimiz C++ ve C# bu konuda farklı davranır:
>
> ```c
> int a[5] = {0};
> a[7] = 1;             // C: derlenir, tanımsız davranış
> ```
>
> ```cpp
> std::array<int, 5> a{};
> a.at(7) = 1;          // C++: std::out_of_range istisnası fırlatır
> a[7] = 1;             // C++: [] hâlâ denetlemez (C ile aynı)
> ```

```
int[] a = new int[5];
a[7] = 1;              // C#: her erişim denetlenir,
                       // IndexOutOfRangeException fırlatılır
```

C++'ta `a.at(i)` denetli, `a[i]` denetsiz erişimdir; C#'ta denetim her zaman vardır ve atlanamaz. Bu fark nedeniyle C# programlarında arabellek taşması güvenlik açığı olarak neredeyse görülmez.

10.4 Eleman Sayısını Bulmak: sizeof

Eleman sayısını elle yazmak yerine, dizinin bayt sayısını bir elemanın bayt sayısına bölerek buluruz:

```
int n = sizeof(a) / sizeof(a[0]);
```

Böleni `sizeof(int)` yerine `sizeof(a[0])` olarak yazarız; böylece dizinin tipi sonradan değişse formül doğru kalır. `sizeof` sonucu `size_t` tipindedir ve `%zu` ile yazdırılır (Bölüm 5).

Bu formül **yalnızca dizinin tanımlandığı yerde** çalışır. Diziyi bir fonksiyona geçirdiğimizde durum değişir:

▸ sizeofTuzak.c

```c
#include <stdio.h>

void yanlis(int a[]) {
    int n = sizeof(a) / sizeof(a[0]);
    printf("Fonksiyon içinde n = %d\n", n);
}

int main(void) {
    int dizi[6] = {1, 2, 3, 4, 5, 6};
    int n = sizeof(dizi) / sizeof(dizi[0]);

    printf("Dizi %zu bayt, eleman %zu bayt\n", sizeof(dizi), sizeof(dizi[0]));
    printf("main içinde n = %d\n", n);
    yanlis(dizi);
    return 0;
}
```

ÇIKTI

```
Dizi 24 bayt, eleman 4 bayt
main içinde n = 6
Fonksiyon içinde n = 2
```

Derleyici de bu hatayı sezer:

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra sizeofTuzak.c -o sizeofTuzak
sizeofTuzak.c: In function 'yanlis':
sizeofTuzak.c:4:19: warning: 'sizeof' on array function parameter 'a'
will return size of 'int *' [-Wsizeof-array-argument]
```

Fonksiyon dizinin kendisini değil **adresini** alır; `int a[]` yazsak bile `a` bir adres tutar ve `sizeof(a)` adresin boyutunu verir. 64 bitlik bir bilgisayarda adres 8 bayttır, dolayısıyla 8 / 4 = 2 çıkar (32 bitlik sistemde 1 çıkardı). Sonuç ne olursa olsun yanlıştır. Çözüm, eleman sayısını dizinin tanımlandığı yerde hesaplayıp fonksiyona ayrı bir `int n` parametresiyle geçirmektir. Nedenini Bölüm 12'de işaretçileri öğrendiğimizde tam olarak göreceğiz.

10.5 Diziyi Doldurmak ve Dolaşmak

Dizinin omurgası `for` döngüsüdür; döngü değişkeni indeks olur:

- Baştan sona: `for (int i = 0; i < N; i++)`
- Sondan başa: `for (int i = N - 1; i >= 0; i--)`
- Birer atlayarak: `for (int i = 0; i < N; i += 2)`

Klavyeden doldururken `scanf`'e elemanın adresini veririz: `&notlar[i]`. `&` unutulursa `scanf` değeri adres sanır; `gcc` uyarır, program çoğu zaman çöker. Kullanıcıya `i + 1` gösteriyoruz çünkü insanlar 1'den sayar, dizide ise 0'dan başlayan `i` kullanıyoruz.

▸ notGirisi.c

```c
#include <stdio.h>

#define N 4

int main(void) {
    int notlar[N];

    for (int i = 0; i < N; i++) {
        printf("%d. notu girin: ", i + 1);
        scanf("%d", &notlar[i]);
    }

    printf("\nNo Not\n");
    for (int i = 0; i < N; i++) {
        printf("%2d %3d\n", i + 1, notlar[i]);
    }
    return 0;
}
```

```
ÇIKTI
1. notu girin: 70
2. notu girin: 85
3. notu girin: 90
4. notu girin: 100

No Not
 1  70
 2  85
 3  90
 4 100
```

10.6 Temel Dizi Algoritmaları

Dizilerle ilgili problemlerin çoğu birkaç kalıba iner: toplam, ortalama, en büyük/en küçük (ve yeri), sayma ve koşula uyanları bulma. Hepsini tek programda görelim:

▸ istatistik.c

```c
#include <stdio.h>

#define N 8

int main(void) {
    int notlar[N] = {72, 45, 88, 60, 95, 38, 81, 67};
```

```
    int toplam = 0;
    int enBuyuk = notlar[0];
    int enBuyukIndeks = 0;
    int enKucuk = notlar[0];
    int enKucukIndeks = 0;
    int gecen = 0;

    for (int i = 0; i < N; i++) {
        toplam += notlar[i];
        if (notlar[i] > enBuyuk) {
            enBuyuk = notlar[i];
            enBuyukIndeks = i;
        }
        if (notlar[i] < enKucuk) {
            enKucuk = notlar[i];
            enKucukIndeks = i;
        }
        if (notlar[i] >= 60) {
            gecen++;
        }
    }

    double ortalama = (double)toplam / N;
    printf("Toplam: %d, ortalama: %.2f\n", toplam, ortalama);
    printf("En büyük: %d (indeks %d)\n", enBuyuk, enBuyukIndeks);
    printf("En küçük: %d (indeks %d)\n", enKucuk, enKucukIndeks);
    printf("Geçen: %d kişi\n", gecen);

    printf("Ortalamanın üstündekiler:");
    for (int i = 0; i < N; i++) {
        if (notlar[i] > ortalama) {
            printf(" %d", notlar[i]);
        }
    }
    printf("\n");
    return 0;
}
```

```
ÇIKTI
Toplam: 546, ortalama: 68.25
En büyük: 95 (indeks 4)
En küçük: 38 (indeks 5)
Geçen: 6 kişi
Ortalamanın üstündekiler: 72 88 95 81
```

Programdaki kalıpları sırayla inceleyelim:

- **Toplam** 0 ile başlar ve her turda eleman eklenir.
- **Ortalama** için bölenlerden biri `double`'a çevrilmelidir. `toplam / N` yazarsak 546 / 8 tamsayı bölmesi olur ve 68.25 yerine 68 çıkar.
- **En büyük/en küçük** için aday, dizinin ilk elemanıyla başlatılır; 0 ile değil, çünkü bütün sayılar negatifse 0 hep kazanır. İndeksi de sakladığımızda "kim en yüksek?" sorusuna cevap verebiliriz. Koşulda `>` kullandığımız için en büyük değer birden çok yerde varsa ilk bulunanın indeksi kalır (`>=` olsaydı sonuncusu kalırdı).
- **Sayma**, Bölüm 8'deki sayaç kalıbıdır: koşul doğruysa `gecen++`.
- **Ortalamanın üstündekiler** iki geçiş ister: ortalama bilinmeden kimin üstte olduğu bilinemez. Önce ortalamayı hesaplarız, sonra diziyi yeniden geziriz.

Sıklık dizisi

Dizilerin güçlü bir kullanımı da değerin kendisini indeks olarak kullanmaktır. Bir zarın on atışını sayalım: her yüz değeri için bir sayaç gerekir. Altı ayrı değişken yerine altı elemanlı bir dizi kullanırız ve `atislar[i]` değerini `sayac[atislar[i] - 1]` elemanına yönlendiririz:

▸ zarSayac.c

```c
#include <stdio.h>

#define ATIS 10
#define YUZ 6

int main(void) {
    int atislar[ATIS] = {3, 6, 1, 3, 3, 5, 6, 2, 3, 1};
    int sayac[YUZ] = {0};

    for (int i = 0; i < ATIS; i++) {
        sayac[atislar[i] - 1]++;
    }

    for (int yuz = 1; yuz <= YUZ; yuz++) {
        printf("%d geldi: %d kez\n", yuz, sayac[yuz - 1]);
    }
    return 0;
}
```

ÇIKTI

```
1 geldi: 2 kez
2 geldi: 1 kez
3 geldi: 4 kez
4 geldi: 0 kez
5 geldi: 1 kez
6 geldi: 2 kez
```

`atislar[i] - 1` ifadesi, 1-6 aralığındaki zar değerini 0-5 indeksine çevirir. Bu "değeri indeks yapma" tekniği, harf sıklığı ya da not dağılımı gibi sayma problemlerinde sıkça karşımıza çıkacak. Atış değerinin 1-6 dışında olması hâlinde `sayac` sınırının dışına yazılacağını unutmamalıyız; gerçek bir programda önce değerin aralıkta olduğu denetlenir.

10.7 Diziler ve Fonksiyonlar

Dizi parametresi, köşeli parantezli ama boyutsuz yazılır. Eleman sayısını daha önce gördüğümüz nedenle ayrı bir `int n` parametresiyle geçiririz. Çağrıda ise dizi yalnızca adıyla verilir: `yazdir(notlar, N)`.

▸ diziFonksiyon.c

```c
#include <stdio.h>

#define N 5

void yazdir(const int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}
```

```c
void ikiyeKatla(int a[], int n) {
    for (int i = 0; i < n; i++) {
        a[i] *= 2;
    }
}

void artir(int x) {
    x++;
    printf("artir içinde x = %d\n", x);
}

int main(void) {
    int a[N] = {1, 2, 3, 4, 5};
    int sayi = 10;

    ikiyeKatla(a, N);
    yazdir(a, N);
    artir(sayi);
    printf("main içinde sayi = %d\n", sayi);
    return 0;
}
```

```
ÇIKTI
2 4 6 8 10
artir içinde x = 11
main içinde sayi = 10
```

`ikiyeKatla` diziyi değiştirdi ve değişiklik `main`'deki dizide göründü; `artir` ise kendi `x`'ini artırdı, `main`'deki `sayi` 10 kaldı. Bu durum, Bölüm 9'daki "C'de her şey değerle geçer" kuralıyla çelişmez. Dizi adı bir ifadede kullanıldığında çoğu zaman dizinin **ilk elemanının adresine** dönüşür; fonksiyona kopyalanan şey bu adrestir. Fonksiyon adres üzerinden gerçek diziye ulaştığı için değişiklikler çağırana yansır:

	`int x` parametresi	`int a[]` parametresi
Fonksiyona giden	Değerin kopyası	Dizinin başlangıç adresi
Fonksiyonda yapılan değişiklik	Çağırana yansımaz	Çağırana yansır

Büyük bir diziyi her çağrıda kopyalamak yavaş olurdu; C bu yüzden diziyi adres olarak geçirir. Ayrıntılarını Bölüm 12'de göreceğiz. Dizi fonksiyondan **döndürülemez**: sonuç, çağıranın verdiği diziye yazılır. Dizinin tek bir elemanını geçirmek (`artir(a[2])`) ise sıradan bir `int` geçirmektir ve kopya oluşturur.

> **▲ Sık Yapılan Hata · Çağrıda köşeli parantez yazmak**
>
> Diziyi çağırırken adın dışında bir şey yazmayız. `yazdir(a[], N)` ve `yazdir(int a[], N)` derleme hatasıdır; `yazdir(a[N], N)` ise `a[N]` dizinin dışındaki tek bir eleman olduğu için yanlıştır. Parametre listesine `int a[5]` yazsak bile derleyici 5'i yok sayar: `a` yine bir adrestir ve eleman sayısını `n` ile geçirmek zorundayız.

const dizi parametresi

`yazdir` diziyi yalnızca okur. Parametreyi `const int a[]` yazarak bunu derleyiciye de bildiririz. Fonksiyon bir elemanı değiştirmeye kalkarsa (`a[0] = 0;`) `gcc` bunu `assignment of read-only location` hatasıyla reddeder.

◆ Kritik Nokta · const kuralı

Diziyi yalnızca okuyan fonksiyonlar (yazdırma, toplama, arama) `const` parametre alır; diziyi değiştirenler (doldurma, sıralama, ters çevirme) almaz. Fonksiyon başlığına bakan biri, dizinin başına ne geleceğini hemen anlar.

10.8 Kopyalama ve Karşılaştırma

Dizi bütün olarak atanamaz. `b = a;` ve (tanımdan sonra) `a = {1, 2, 3};` derleme hatasıdır:

▸ kopyaHata.c

```c
#include <stdio.h>

int main(void) {
    int a[3] = {1, 2, 3};
    int b[3];

    b = a;
    printf("%d\n", b[0]);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
kopyaHata.c: In function 'main':
kopyaHata.c:7:7: error: assignment to expression with array type
    7 |     b = a;
      |       ^
```

Sebebi yukarıdaki dönüşümdür: `b = a` ifadesinde `a` bir adrese dönüşür ve adres, `b`'nin içine yazılabilecek bir değer değildir; `b` bir dizidir, değişken gibi yeniden atanamaz. Kopyalamanın iki yolu vardır: eleman eleman döngüyle ya da `<string.h>` içindeki `memcpy(hedef, kaynak, bayt)` ile. Sıra atamadaki gibidir: önce hedef.

Eşitliği de elle denetleriz. `ayniMi` fonksiyonu ilk farkta durur:

▸ kopyala.c

```c
#include <stdio.h>
#include <string.h>

#define N 5

int ayniMi(const int a[], const int b[], int n) {
    for (int i = 0; i < n; i++) {
        if (a[i] != b[i]) {
            return 0;
        }
    }
    return 1;
}

int main(void) {
    int a[N] = {1, 2, 3, 4, 5};
    int b[N];
    int c[N];

    for (int i = 0; i < N; i++) {
        b[i] = a[i];
```

```
    }
    memcpy(c, a, sizeof(a));

    printf("Kopyalar a ile aynı mı? %d %d\n", ayniMi(a, b, N), ayniMi(a, c, N));
    b[0] = 99;
    printf("b[0] = %d, a[0] = %d, aynı mı? %d\n", b[0], a[0], ayniMi(a, b, N));
    return 0;
}
```

```
ÇIKTI
Kopyalar a ile aynı mı? 1 1
b[0] = 99, a[0] = 1, aynı mı? 0
```

Kopyalar birbirinden bağımsızdır: `b[0]` değişince `a[0]` etkilenmedi.

> ▲ **Sık Yapılan Hata · memcpy'de eleman sayısı vermek**
>
> `memcpy`'nin üçüncü parametresi **bayt** sayısıdır. `memcpy(c, a, N)` yalnızca 5 bayt kopyalar; bu, ilk elemanın tamamı ile ikinci elemanın ilk baytı demektir, dizinin geri kalanı çöp kalır. Doğrusu `memcpy(c, a, sizeof(a))` ya da `memcpy(c, a, N * sizeof(a[0]))` yazmaktır.

Karşılaştırmada da bir tuzak vardır: `a == b` iki dizinin **içeriğini** değil **adreslerini** karşılaştırır. İki ayrı dizi bellekte farklı yerde durduğundan, içerikleri aynı olsa bile sonuç hep yanlıştır; `gcc` de `comparison between two arrays` uyarısı verir. İçeriği `ayniMi` gibi bir döngüyle ya da `memcmp(a, b, sizeof(a)) == 0` ile karşılaştırırız.

10.9 Ters Çevirme ve Kaydırma

Bu bölümdeki algoritmaların çoğu iki elemanın yer değiştirmesine dayanır. Bu işlem için **geçici bir değişken** şarttır. `a[i] = a[j]; a[j] = a[i];` yazarsak ilk satır `a[i]`'nin eski değerini yok eder ve ikisi de `a[j]`'nin değerini alır. İki `int` parametresini değiştirmeye çalışan bir `swap(int a, int b)` fonksiyonu, parametreler kopya olduğu için çağıranı etkilemez (Bölüm 9). Dizi ise fonksiyona adres olarak geçtiği için `degistir(a, i, j)` gerçek elemanları değiştirir.

Ters çevirme için biri baştan, biri sondan ilerleyen iki indeks (`bas`, `son`) elemanları değiştirir ve birbirine yaklaşır; birbirini geçince işlem biter. **Sola kaydırmada** ilk eleman kenara alınır, her eleman bir soldaki yere çekilir, kenardaki eleman sona konur. **Sağa kaydırma** bunun aynadaki hâlidir ve sondan başa doğru işler:

▸ donusum.c

```
#include <stdio.h>

void yazdir(const int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

void degistir(int a[], int i, int j) {
    int gecici = a[i];
    a[i] = a[j];
    a[j] = gecici;
}
```

```c
void tersCevir(int a[], int n) {
    int bas = 0;
    int son = n - 1;
    while (bas < son) {
        degistir(a, bas, son);
        bas++;
        son--;
    }
}

void solaKaydir(int a[], int n) {
    int ilk = a[0];
    for (int i = 0; i < n - 1; i++) {
        a[i] = a[i + 1];
    }
    a[n - 1] = ilk;
}

void sagaKaydir(int a[], int n) {
    int son = a[n - 1];
    for (int i = n - 1; i > 0; i--) {
        a[i] = a[i - 1];
    }
    a[0] = son;
}

int main(void) {
    int a[6] = {1, 2, 3, 4, 5, 6};
    int b[5] = {1, 2, 3, 4, 5};
    int c[5] = {1, 2, 3, 4, 5};

    tersCevir(a, 6);
    solaKaydir(b, 5);
    sagaKaydir(c, 5);
    yazdir(a, 6);
    yazdir(b, 5);
    yazdir(c, 5);
    return 0;
}
```

```
ÇIKTI
6 5 4 3 2 1
2 3 4 5 1
5 1 2 3 4
```

`tersCevir(a, 6)` çağrısında (`bas`, `son`) = (0, 5), (1, 4), (2, 3) çiftleri sırayla yer değiştirir:

```
1 2 3 4 5 6   (0,5) değişir ->  6 2 3 4 5 1
6 2 3 4 5 1   (1,4) değişir ->  6 5 3 4 2 1
6 5 3 4 2 1   (2,3) değişir ->  6 5 4 3 2 1
```

Bundan sonra `bas = 3`, `son = 2` olur ve `bas < son` koşulu yanlış çıkar. Eleman sayısı tekse ortadaki eleman yerinde kalır; toplam `n / 2` değiş tokuş yapılır.

> **◆ Kritik Nokta · Kaydırma yönü**
>
> Sola kaydırırken baştan sona, sağa kaydırırken sondan başa gideriz. Sağa kaydırmayı baştan sona yazarsak (`for (i = 1; i < n; i++) a[i] = a[i - 1];`) her adım az önce yazılan değeri okur ve ilk eleman bütün diziye yayılır: `{1, 2, 3, 4, 5}` için sonuç `1 1 1 1 1` olur. Kural: bir eleman, üzerine yazılmadan önce okunmalıdır.

10.10 Doğrusal ve İkili Arama

Doğrusal arama (linear search) diziyi baştan sona gezer. Aranan değeri bulursa indeksini döndürür; bulamazsa geçerli bir indeks olamayacak bir değer, gelenek olarak `-1` döndürür.

İkili arama (binary search) ise dizi **sıralıysa** çok daha hızlıdır. Ortadaki elemana bakılır; aranan ortadakinden büyükse yalnızca sağ yarıda, küçükse yalnızca sol yarıda aranır. Aday aralığı her adımda yarıya iner; bu, sözlükte bir sözcük ararken sayfayı ortadan açmaya benzer. Üç indeks gerekir: aralığın başı `alt`, sonu `ust`, ortası `orta`.

▸ arama.c

```c
#include <stdio.h>

int ara(const int a[], int n, int aranan) {
    for (int i = 0; i < n; i++) {
        if (a[i] == aranan) {
            return i;
        }
    }
    return -1;
}

int ikiliAra(const int a[], int n, int aranan) {
    int alt = 0;
    int ust = n - 1;
    int adim = 0;

    while (alt <= ust) {
        int orta = alt + (ust - alt) / 2;
        adim++;
        printf("  adım %d: alt=%d ust=%d orta=%d a[orta]=%d\n",
               adim, alt, ust, orta, a[orta]);
        if (a[orta] == aranan) {
            return orta;
        } else if (a[orta] < aranan) {
            alt = orta + 1;
        } else {
            ust = orta - 1;
        }
    }
    return -1;
}

int main(void) {
    int a[] = {3, 7, 12, 19, 25, 31, 42, 56, 68, 77};
    int n = sizeof(a) / sizeof(a[0]);

    printf("ara: 31 -> %d, 10 -> %d\n", ara(a, n, 31), ara(a, n, 10));
    printf("31 ikili aranıyor:\n");
    printf("  sonuç: %d\n", ikiliAra(a, n, 31));
    printf("10 ikili aranıyor:\n");
    printf("  sonuç: %d\n", ikiliAra(a, n, 10));
    return 0;
}
```

```
ÇIKTI
ara: 31 -> 5, 10 -> -1
31 ikili aranıyor:
  adım 1: alt=0 ust=9 orta=4 a[orta]=25
  adım 2: alt=5 ust=9 orta=7 a[orta]=56
  adım 3: alt=5 ust=6 orta=5 a[orta]=31
  sonuç: 5
10 ikili aranıyor:
  adım 1: alt=0 ust=9 orta=4 a[orta]=25
  adım 2: alt=0 ust=3 orta=1 a[orta]=7
  adım 3: alt=2 ust=3 orta=2 a[orta]=12
  sonuç: -1
```

`ara` fonksiyonunda `return`, ilk bulunan yerde fonksiyonu bitirir; aranan birden çok yerdeyse ilk indeks döner. `-1` ise döngüden **sonra** gelir. En kötü durumda `n` karşılaştırma yapılır, ama dizinin sıralı olması gerekmez.

> **▲ Sık Yapılan Hata · Döngünün içinde -1 döndürmek**
>
> `if (a[i] == aranan) { return i; } else { return -1; }` yazarsak fonksiyon yalnızca ilk elemana bakar: ilk eleman aranan değilse `else` hemen `return`'e girer ve diğer elemanlara sıra gelmez. "Bulunamadı" kararı, bütün elemanlar denendikten sonra döngünün dışında verilir.

`31` aramasının adımlarını bir tabloyla izleyelim (indeksler 0-9):

Adım	`alt`	`ust`	`orta`	`a[orta]`	Karar
1	0	9	4	25	25 < 31, `alt = orta + 1 = 5`
2	5	9	7	56	56 > 31, `ust = orta - 1 = 6`
3	5	6	5	31	bulundu: indeks 5

`10` dizide olmadığı için `alt = 2`, `ust = 1` olduğunda döngü biter ve `-1` döner. İkili aramada dikkat edeceğimiz noktalar şunlardır:

- Döngü koşulu `alt <= ust`'tir. `<` yazarsak tek eleman kalan aralığı atlarız.
- `alt = orta + 1` ve `ust = orta - 1` yazılır, çünkü `orta` zaten denenmiştir.
- Dizi sıralı değilse sonuç yanlıştır.
- `n` elemanlı sıralı dizide en çok $\lfloor \log_2 n \rfloor + 1$ adım gerekir: 10 elemanda en çok 4, 100 elemanda 7, bir milyon elemanda yalnızca 20 adım. Doğrusal aramada bu sayılar 10, 100 ve 1 000 000'dur.

> **▲ Sık Yapılan Hata · Ortayı bulurken taşma**
>
> Ortayı `(alt + ust) / 2` diye de yazabilirdik. Çok büyük dizilerde `alt + ust` toplamı `int` sınırını aşabilir (Bölüm 5'teki taşma). `alt + (ust - alt) / 2` aynı sonucu verir ve toplamı hiçbir zaman taşıramaz; bu yüzden onu tercih ettik.

10.11 Sıralama: Kabarcık ve Seçmeli

Bir diziyi küçükten büyüğe dizmek **sıralama** (sorting) problemidir. İki klasik yöntemi inceleyeceğiz; ikisi de yer değiştirme için `degistir` fonksiyonunu kullanır.

Kabarcık sıralaması (bubble sort): Komşu elemanlar karşılaştırılır, soldaki büyükse yer değiştirilir. Bir geçiş sonunda en büyük eleman kabarcık gibi yüzüp sona ulaşır. Hiç yer değiştirme olmayan bir geçiş dizinin sıralı olduğunu kanıtladığından, bir bayrak değişkeniyle bu durumda döngüden çıkarız.

Seçmeli sıralama (selection sort): Her turda henüz sıralanmamış kısmın en küçüğü bulunur ve o kısmın başındaki elemanla değiştirilir.

▸ siralama.c

```c
#include <stdio.h>

void yazdir(const int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

void degistir(int a[], int i, int j) {
    int gecici = a[i];
    a[i] = a[j];
    a[j] = gecici;
}

void kabarcikSirala(int a[], int n) {
    for (int gecis = 1; gecis < n; gecis++) {
        int degisti = 0;
        for (int i = 0; i < n - gecis; i++) {
            if (a[i] > a[i + 1]) {
                degistir(a, i, i + 1);
                degisti = 1;
            }
        }
        printf("%d. geçiş: ", gecis);
        yazdir(a, n);
        if (!degisti) {
            break;
        }
    }
}

void secmeliSirala(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int enKucuk = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[enKucuk]) {
                enKucuk = j;
            }
        }
        degistir(a, i, enKucuk);
        printf("i=%d (en küçük indeks %d): ", i, enKucuk);
        yazdir(a, n);
    }
}

int main(void) {
    int a[5] = {5, 1, 4, 2, 8};
    int b[5] = {64, 25, 12, 22, 11};

    printf("Kabarcık, başlangıç: ");
    yazdir(a, 5);
    kabarcikSirala(a, 5);
```

```
    printf("Seçmeli, başlangıç: ");
    yazdir(b, 5);
    secmeliSirala(b, 5);
    return 0;
}
```

```
ÇIKTI
Kabarcık, başlangıç: 5 1 4 2 8
1. geçiş: 1 4 2 5 8
2. geçiş: 1 2 4 5 8
3. geçiş: 1 2 4 5 8
Seçmeli, başlangıç: 64 25 12 22 11
i=0 (en küçük indeks 4): 11 25 12 22 64
i=1 (en küçük indeks 2): 11 12 25 22 64
i=2 (en küçük indeks 3): 11 12 22 25 64
i=3 (en küçük indeks 3): 11 12 22 25 64
```

Kabarcık sıralamasında ilk geçişin karşılaştırmalarını dizi `5 1 4 2 8` için adım adım yazalım:

Karşılaştırma	Karar	Dizi
`a[0]` ile `a[1]` : 5 > 1	değiştir	1 5 4 2 8
`a[1]` ile `a[2]` : 5 > 4	değiştir	1 4 5 2 8
`a[2]` ile `a[3]` : 5 > 2	değiştir	1 4 2 5 8
`a[3]` ile `a[4]` : 5 < 8	değiştirme	1 4 2 5 8

`gecis`'inci geçişte iç döngü `n - gecis` karşılaştırma yapar, çünkü sondaki `gecis - 1` eleman zaten yerine oturmuştur. Bu örnekte dizi 2. geçişte sıralanmış hâle geldi; 3. geçişte hiç değiş tokuş olmadığı için bayrak döngüyü bitirdi. Bayrak olmasaydı 4. geçiş de yapılırdı.

Seçmeli sıralamada `i` yalnızca `n - 2`'ye kadar gider, çünkü son eleman kendiliğinden yerine oturur. `i = 3` satırında 25, kendisiyle yer değiştirdi; bu zararsızdır.

İki yöntemin karşılaştırma sayısı `n(n-1)/2`'dir (bayraklı kabarcık daha azına inebilir). Değiş tokuş sayısı kabarcıkta çok olabilir, seçmelide en çok `n - 1`'dir. İkisinde de `n` iki katına çıkınca iş yaklaşık dört katına çıkar (bu davranışa "O(n²)" denir). Büyük veride çok daha hızlı yöntemler kullanılır; standart kütüphanedeki `qsort`'u Bölüm 12'de göreceğiz. Büyükten küçüğe sıralamak için karşılaştırmayı tersine çevirmek yeter (`a[i] < a[i + 1]`).

> **◆ Kritik Nokta · Geçişlerin anlamı**
>
> Kabarcık sıralamasında `k` geçiş sonunda **sondaki** `k` eleman kesinlikle yerindedir (en büyük `k` eleman); baştakiler hâlâ karışık olabilir. Seçmeli sıralamada `k` tur sonunda ise **baştaki** `k` eleman, yani en küçükler yerindedir.

> **İpucu · Sıralamayı kâğıtta izleyin**
>
> Her geçiş ya da tur sonunda dizinin tamamını alt alta yazın; hangi iki elemanı karşılaştırdığınızı ve yer değiştirip değiştirmediğinizi yanına not edin. İç döngü sınırlarını karıştırmamak için: kabarcıkta `i < n - gecis`, seçmelide `j = i + 1`'den `n`'e kadar.

10.12 İki Boyutlu Diziler

Tablo ya da matris gibi satır ve sütunlu verileri **iki boyutlu dizi** tutar:

```
tip diziAdi[SATIR][SUTUN];
```

Eleman `m[i][j]` ile gösterilir: `i` satır numarası (0'dan `SATIR-1`'e), `j` sütun numarasıdır (0'dan `SUTUN-1`'e). `int m[3][4];` tanımı 3 satır, 4 sütun, toplam 12 eleman demektir. Bellekte elemanlar yine art arda durur, ama **satır satır**; her satır kendi başına bir tek boyutlu dizidir:

```
m[0][0] m[0][1] m[0][2] m[0][3] | m[1][0] m[1][1] ... | m[2][0] ...
\_______ 0. satır ____________/   \______ 1. satır ___/   \_ 2. satır
```

İlk değerler satır başına bir küme parantezi ile verilir; eksik olanlar yine 0 olur:

```
int z[2][3] = { {1}, {4, 5} };          // 1 0 0 / 4 5 0
int y[][3] = { {1, 2, 3}, {4, 5, 6} };  // satır sayısı listeden çıkar
```

Yalnızca **ilk boyut** atlanabilir; `int y[][] = ...` geçersizdir. Tek parantezli düz liste de derlenir, ama `gcc -Wmissing-braces` ile uyarabilir; iç içe yazmak daha okunaklıdır. `int m[2][3]` için `m[2][0]` diye bir eleman yoktur: satır indeksleri 0 ve 1'dir.

İki boyutlu diziler iç içe iki `for` ile gezilir: dış döngü satırları, iç döngü sütunları dolaşır. Aşağıdaki programda 3 × 3 bir matris için satır toplamları, iki köşegen ve **transpoz** hesaplanıyor. Transpoz, satırları sütun yapmaktır: `t[j][i] = m[i][j]`.

▸ matris.c

```c
#include <stdio.h>

#define N 3

int main(void) {
    int m[N][N] = {
        {5, 1, 2},
        {0, 3, 4},
        {9, 8, 7}
    };
    int t[N][N];
    int ana = 0;
    int ikincil = 0;

    for (int i = 0; i < N; i++) {
        int satirToplam = 0;
        for (int j = 0; j < N; j++) {
            printf("%3d", m[i][j]);
            satirToplam += m[i][j];
            t[j][i] = m[i][j];
            if (i == j) {
                ana += m[i][j];
            }
            if (i + j == N - 1) {
                ikincil += m[i][j];
            }
        }
        printf(" | satır toplamı: %d\n", satirToplam);
    }
```

```
    printf("Ana köşegen: %d, ikincil köşegen: %d\n", ana, ikincil);

    printf("Transpoz:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%3d", t[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
ÇIKTI
  5  1  2 | satır toplamı: 8
  0  3  4 | satır toplamı: 7
  9  8  7 | satır toplamı: 24
Ana köşegen: 15, ikincil köşegen: 14
Transpoz:
  5  0  9
  1  3  8
  2  4  7
```

Satır toplamında iç döngü `j`'dir ve toplam her satırın başında 0'dan yeniden başlar. Sütun toplamı için döngüler yer değiştirir (dış döngü `j`, iç döngü `i`). Kare matriste bölgeler şu koşullarla seçilir:

Bölge	Koşul	Bu matriste
Ana köşegen	`i == j`	5 + 3 + 7 = 15
İkincil köşegen	`i + j == N - 1`	2 + 3 + 9 = 14
Köşegenin üstü	`j > i`	1 + 2 + 4 = 7
Köşegenin altı	`j < i`	0 + 9 + 8 = 17

İki boyutlu dizide de `sizeof` işe yarar: `sizeof(m)` bütün dizinin, `sizeof(m[0])` bir satırın bayt sayısıdır. Bu matriste 3 × 3 × 4 = 36 bayt ve bir satır 12 bayttır. Satır sayısı `sizeof(m) / sizeof(m[0])` (36 / 12 = 3), sütun sayısı `sizeof(m[0]) / sizeof(m[0][0])` (12 / 4 = 3) ile bulunur.

Fonksiyona iki boyutlu dizi geçirmek

İki boyutlu dizi parametresinde **sütun sayısı yazılmak zorundadır**, satır sayısı yazılmaz (ayrı bir parametre olur): `int m[][SUTUN]`. Çünkü elemanlar art arda durur ve derleyici `m[i][j]` elemanının yerini "başlangıç + (i × SUTUN + j) × 4 bayt" diye hesaplar. Formülde `SATIR` yoktur ama `SUTUN` vardır. `void yazdir(int m[][], int satir)` yazarsak `gcc` `array type has incomplete element type 'int[]'` hatası verir.

Aşağıdaki programda iki matrisi toplayan ve çarpan fonksiyonlar var. Matris çarpımında `A` matrisi `S × K`, `B` matrisi `K × M` boyutundadır ve sonuç `S × M` olur; `c[i][j]`, `A`'nın `i`'inci satırı ile `B`'nin `j`'inci sütununun elemanlarının çarpımlarının toplamıdır:

▸ matrisIslem.c

```
#include <stdio.h>

#define S 2
#define K 3
#define M 2

void yazdirMatris(const int m[][M], int satir) {
```

```
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < M; j++) {
            printf("%5d", m[i][j]);
        }
        printf("\n");
    }
}

void topla(const int a[][K], const int b[][K], int c[][K], int satir) {
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < K; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }
}

void carp(const int a[][K], const int b[][M], int c[][M]) {
    for (int i = 0; i < S; i++) {
        for (int j = 0; j < M; j++) {
            c[i][j] = 0;
            for (int k = 0; k < K; k++) {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}

int main(void) {
    int a[S][K] = {{1, 2, 3}, {4, 5, 6}};
    int b[S][K] = {{10, 20, 30}, {40, 50, 60}};
    int toplam[S][K];

    topla(a, b, toplam, S);
    printf("a + b:\n");
    for (int i = 0; i < S; i++) {
        for (int j = 0; j < K; j++) {
            printf("%5d", toplam[i][j]);
        }
        printf("\n");
    }

    int d[K][M] = {{7, 8}, {9, 10}, {11, 12}};
    int carpim[S][M];

    carp(a, d, carpim);
    printf("a x d:\n");
    yazdirMatris(carpim, S);
    return 0;
}
```

```
ÇIKTI
a + b:
   11   22   33
   44   55   66
a x d:
   58   64
  139  154
```

Çarpımın ilk elemanı `1·7 + 2·9 + 3·11 = 58`, ikincisi `1·8 + 2·10 + 3·12 = 64` olur. Üç iç içe döngünün en içteki `k`, toplamı biriktirir; `c[i][j]` her yeni `(i, j)` çifti için 0'dan başlatılır. Tek boyutlu dizilerde olduğu gibi `toplam` ve `carpim` fonksiyonlara adres olarak geçer; yazılanlar `main`'deki dizilerde görünür. Dizi döndürülemediği için sonucu her zaman çağıran verir.

◆ Kritik Nokta · İki boyutlu dizi parametresi

Üç kural: (1) `m[i][j]` bir elemandır: önce satır, sonra sütun. (2) Köşegen koşulları `i == j` ve `i + j == N - 1` 'dir. (3) Fonksiyon başlığında `int m[][SUTUN]` doğru, `int m[][]` yanlıştır.

Üç Dilde · Eleman sayısı ve çok boyutlu diziler *C · C++ · C#*

C'de dizi kendi boyutunu taşımaz; boyutu ayrıca tutar ve fonksiyonlara ayrı parametre olarak geçiririz. C++ ve C# dizileri ise boyutlarını bilir:

```c
int a[5] = {0};
int n = sizeof(a) / sizeof(a[0]);   // C: eleman sayısı elle hesaplanır
int m[3][4];                        // C: tek bellek bloğu, satır satır
```

```cpp
std::array<int, 5> a{};
std::cout << a.size();              // C++: 5
std::vector<std::vector<int>> m(3, std::vector<int>(4));
// m.size() == 3 satır, m[0].size() == 4 sütun
```

```csharp
int[] a = new int[5];
Console.WriteLine(a.Length);        // C#: 5
int[,] m = new int[3, 4];           // dikdörtgen çok boyutlu dizi
Console.WriteLine(m.GetLength(0)); // 3 (satır)
Console.WriteLine(m.Length);        // 12 (toplam eleman)
int[][] p = new int[3][];           // "dişli" (jagged) dizi: satırlar
                                    // farklı uzunlukta olabilir
```

C# dizileri ayrıca her zaman sınır denetimlidir ve `new` ile çalışma zamanında istenen boyutta oluşturulur. C++'ın `std::vector` ve C#'ın `List<T>` türleri ise büyüyüp küçülebilir; bunların karşılığını C'de Bölüm 13'te `realloc` ile kendimiz yazacağız.

10.13 Değişken Uzunluklu Diziler

C99'dan beri dizi boyutu, çalışma anında bilinen bir değişkenle de verilebilir. Buna **değişken uzunluklu dizi** (variable length array, VLA) denir:

▸ vla.c

```c
#include <stdio.h>

int main(void) {
    int n;
    printf("Kaç sayı? ");
    scanf("%d", &n);

    int sayilar[n];
    int toplam = 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &sayilar[i]);
        toplam += sayilar[i];
    }
    printf("%d eleman, toplam %d, dizi %zu bayt\n",
           n, toplam, sizeof(sayilar));
    return 0;
}
```

```
ÇIKTI
Kaç sayı? 4
10 20 30 40
4 eleman, toplam 100, dizi 16 bayt
```

Bu özellik kolay görünür, ama sınırları vardır:

- VLA'ya ilk değer listesi verilemez (`int a[n] = {0};` geçersizdir).
- Dizi yığında (stack) yaratılır. Çok büyük bir `n` yığını taşırır ve programı çökertir; `n`'in makul olduğunu denetlemek programcıya düşer.
- VLA global düzeyde tanımlanamaz.
- Dizi yaratıldıktan sonra boyutu yine değişmez.

> *Tarihçe · VLA'nın kaderi*
>
> VLA, C99 ile dile girdi. C11 standardı ise VLA desteğini **isteğe bağlı** yaptı: her derleyicinin sunması zorunlu değildir. C++ standardında VLA yoktur; `g++` bunu eklenti olarak kabul eder. C# dilinde de gerek yoktur, çünkü `new int[n]` zaten çalışma zamanında istenen boyutta dizi yaratır.

Bu kitapta ilk değer listesi gerektiren her yerde ve taşınabilir kodda sabit boyutlu dizi kullanacağız. Boyutu çalışma anında belirlenen büyük veriler için doğru çözüm, Bölüm 13'te anlatacağımız dinamik bellektir.

Bölüm Özeti

- Dizi, aynı tipten elemanları tek adla ve bellekte art arda tutar: `tip ad[N];`. Boyut, derleme sırasında bilinen bir sabittir (`#define N 5`); dizi tanımlandıktan sonra büyüyemez.
- `const int N = 5;` sabit ifade değildir: global `int a[N];` derleme hatasıdır, yerelde ise VLA olur. Boyut için `#define` ya da `enum` kullanırız.
- İndeks 0'dan başlar, son eleman `a[N-1]`'dir. C sınır denetimi yapmaz; sınır dışı erişim tanımsız davranıştır. Döngüde `i < N` yazarız.
- İlk değer listesinde eksik elemanlar 0 olur; `{0}` bütün diziyi sıfırlar, `{7}` yalnızca ilkini 7 yapar. `int a[] = {...}` boyutu listeden çıkarır. `[4] = 9` gibi belirlenmiş başlatıcılar C99'dan beri vardır. Listesiz yerel dizi çöp değer taşır.
- Eleman sayısı `sizeof(a) / sizeof(a[0])` ile bulunur, ama yalnızca dizinin tanımlandığı yerde; fonksiyonda adres boyutu bulunur. Bu yüzden fonksiyonlara `(int a[], int n)` biçiminde geçeriz.
- Temel kalıplar: toplam (0'dan başla), ortalama (`(double)toplam / N`), en büyük/en küçük (ilk elemandan başla, indeksi sakla), sayma, sıklık dizisi (değeri indeks yap).
- Dizi fonksiyona adres olarak geçer: fonksiyon diziyi değiştirirse çağırana yansır. Yalnızca okuyan fonksiyon `const` parametre alır. Çağrıda yalnızca dizi adı yazılır.
- `b = a` derlenmez; kopyalama döngü ya da `memcpy(hedef, kaynak, bayt)` ile yapılır (üçüncü parametre bayttır). `a == b` içeriği değil adresleri karşılaştırır; içerik döngü ya da `memcmp` ile karşılaştırılır.
- Ters çevirmede iki indeks ve geçici değişken kullanılır. Kaydırmada eleman, üzerine yazılmadan önce okunmalıdır: sola baştan sona, sağa sondan başa.
- Doğrusal arama bulunursa indeksi, bulunamazsa `-1` döndürür (döngüden sonra). İkili arama sıralı dizide `while (alt <= ust)` ile çalışır ve en çok $\lfloor \log_2 n \rfloor + 1$ adım ister.

- Kabarcık sıralamasında `k` geçiş sonunda en büyük `k` eleman, seçmeli sıralamada `k` tur sonunda en küçük `k` eleman yerindedir. İkisi de yaklaşık `n(n-1)/2` karşılaştırma yapar.
- İki boyutlu dizi `m[i][j]` (`i` satır, `j` sütun) ile kullanılır; iç içe iki `for` ile gezilir. Transpoz `t[j][i] = m[i][j]`, köşegenler `i == j` ve `i + j == N - 1` ile seçilir. Fonksiyon parametresinde sütun sayısı zorunludur: `int m[][SUTUN]`.
- Değişken uzunluklu diziler (C99, C11'de isteğe bağlı) çalışma anında boyut verir ama ilk değer alamaz ve büyük boyutta yığını taşırır. C++ ve C# dizileri/koleksiyonları boyutlarını bilir ve sınır denetimi yapar (C++'ta `at()`, C#'ta her erişimde).

Alıştırmalar

1. `int puan[8];` tanımı için aşağıdakilerden hangisi doğrudur?

   A) Geçerli indeksler 1'den 8'e kadardır.
   B) `puan[8]` dizinin son elemanıdır.
   C) Geçerli indeksler 0'dan 7'ye kadardır; `puan[8]` yazılırsa derleyici çoğunlukla hata vermez, ama program tanımsız davranır.
   D) `puan[8]` yazılırsa C çalışma anında hata fırlatıp programı güvenle durdurur.
   E) Dizi 8 bayt yer kaplar.

2. `int a[5] = {3};` tanımından hemen sonra `a[0] + a[1] + a[2] + a[3] + a[4]` ifadesinin değeri kaçtır?

   A) 3
   B) 15
   C) 0
   D) 8
   E) Tanımsızdır, çünkü kalan elemanlar çöp değer taşır.

3. Sıralı ve 100 elemanlı bir dizide ikili arama yapılıyor. Döngü en kötü durumda en çok kaç adımda sona erer?

   A) 1
   B) 5
   C) 6
   D) 7
   E) 100

4. Aşağıdaki cümlelerden hangisi doğrudur?

   A) `int a[3], b[3];` tanımından sonra `b = a;` yazmak `a`'nın elemanlarını `b`'ye kopyalar.
   B) Bir fonksiyona dizi geçirilirse fonksiyon içinde yapılan eleman değişiklikleri çağıran fonksiyondaki diziye de yansır.
   C) Dizi parametresi alan bir fonksiyonda `sizeof(a) / sizeof(a[0])` her zaman eleman sayısını verir.
   D) `a == b` ifadesi iki dizinin elemanlarının tek tek eşit olup olmadığını denetler.
   E) Parametresi `const int a[]` olan bir fonksiyon `a[0] = 1;` satırını sorunsuz derler.

5. `int m[3][4];` tanımına göre (`int` 4 bayttır) `sizeof(m) / sizeof(m[0])` ifadesi neyi verir?

   A) 4
   B) 12

C) 16
D) 48
E) 3

6. Programın çıktısı nedir?

```
int a[5] = {2, 4, 0, 3, 1};
printf("%d\n", a[a[0]]);
printf("%d\n", a[a[a[0]]]);
a[a[3]] = 8;
int toplam = 0;
for (int i = 0; i < 5; i++) {
    toplam += a[i];
}
printf("%d\n", toplam);
printf("%d %d\n", a[a[4]], a[3]);
```

7. Programın çıktısı nedir?

```
void f(int x) {
    x += 10;
}

void g(int a[]) {
    a[0] += 10;
    a[2] = a[1];
}

int main(void) {
    int s = 1;
    int d[3] = {1, 2, 3};
    f(s);
    g(d);
    f(d[1]);
    printf("%d %d %d %d\n", s, d[0], d[1], d[2]);
    return 0;
}
```

8. Aşağıdaki seçmeli sıralamanın yalnızca ilk iki turu çalıştırılıyor (`i = 0` ve `i = 1`). Dizinin son hâli nedir?

```
int a[5] = {6, 9, 8, 1, 4};
for (int i = 0; i < 2; i++) {
    int enKucuk = i;
    for (int j = i + 1; j < 5; j++) {
        if (a[j] < a[enKucuk]) {
            enKucuk = j;
        }
    }
    int t = a[i];
    a[i] = a[enKucuk];
    a[enKucuk] = t;
}
for (int i = 0; i < 5; i++) {
    printf("%d ", a[i]);
}
printf("\n");
```

9. Aşağıdaki kod parçasındaki hatayı bulun ve düzeltin.

```c
int kare[5];
for (int i = 1; i <= 5; i++) {
    kare[i] = i * i;
}
```

10. Aşağıdaki kod parçasındaki iki hatayı bulun ve düzeltin.

```c
int dizi[4] = {1, 2, 3, 4};
int kopya[4];
kopya = dizi;
if (dizi == kopya) {
    printf("Diziler aynı\n");
}
```

11. Klavyeden 5 tamsayı okuyup dizide saklayan ve bunları ters sırada, aralarında boşlukla yazdıran `tersYaz` programını yazın.

```
ÇIKTI
5 sayı girin: 4 8 15 16 23
Ters sıra: 23 16 15 8 4
```

12. Klavyeden 6 tamsayı (not) okuyup dizide saklayan `notAnalizi` programını yazın. Sırayla en büyük değeri ve indeksini, ortalamayı (iki ondalık basamak) ve ortalamanın üstündeki eleman sayısını yazdırsın. Boyut için `#define` kullanın.

```
ÇIKTI
6 not girin: 55 80 92 47 80 66
En büyük: 92 (indeks 2)
Ortalama: 70.00
Ortalamanın üstünde: 3 eleman
```

13. Klavyeden 3 × 3 bir matris okuyan `kosegenToplam` programını yazın. Önce her satırın toplamını, sonra ana ve ikincil köşegen toplamlarını yazdırsın.

```
ÇIKTI
Matris (3x3): 2 7 1
4 5 9
3 8 6
1. satır toplamı: 10
2. satır toplamı: 18
3. satır toplamı: 17
Ana köşegen toplamı: 13
İkincil köşegen toplamı: 9
```

14. Klavyeden birbirinden farklı 6 tamsayı okuyup dizide saklayan `ikinciBuyuk` programını yazın. Diziyi sıralamadan, tek bir döngüyle en büyük ve ikinci en büyük elemanı bulup yazdırsın.

```
ÇIKTI
6 sayı girin: 12 45 7 38 51 29
En büyük: 51
İkinci en büyük: 45
```

BÖLÜM 11

Karakter Dizileri (String)

BU BÖLÜMDE

- C'de ayrı bir string tipinin bulunmadığını; metnin `'\0'` ile biten bir `char` dizisi olduğunu ve `'A'` ile `"A"` farkını
- Karakter dizisini tanımlamanın yollarını, `sizeof` ile uzunluk arasındaki farkı
- `%s`, `puts` ve indeksle dolaşarak metni yazdırmayı; `scanf` ile `fgets` ile okumayı, satır sonunu temizlemeyi
- `<string.h>` fonksiyonlarını (`strlen`, `strcpy`, `strcat`, `strcmp`, `strchr`, `strstr`, `strtok`) ve `==` ile karşılaştırma tuzağını
- `<ctype.h>` ile karakter denetimini; metin ile sayı arasında dönüşümü (`atoi`, `strtol`, `snprintf`)
- Arabellek taşmasının (buffer overflow) nedenini ve güvenli yazım alışkanlıklarını
- Türkçe karakterlerin `strlen` sonucunu neden şaşırttığını; metin dizilerini ve klasik metin algoritmalarını

Programların büyük bölümü sayıyla olduğu kadar metinle de çalışır: bir kullanıcı adı, bir dosya yolu, bir ekran mesajı. Şimdiye kadar metni yalnızca `printf("...")` içinde sabit olarak gördük. Bu bölümde metni bir değişkende tutmayı, klavyeden okumayı, değiştirmeyi ve karşılaştırmayı öğreneceğiz.

C'de bunun için hazır bir `string` tipi yoktur. Metin, bir `char` dizisidir ve bu yüzden Bölüm 10'daki dizi bilgimiz doğrudan işimize yarar. Diğer yandan metin dizileri C'nin en tehlikeli köşelerinden biridir: sınır denetimi yapılmaz, taşan bir yazma sessizce başka verilerin üzerine biner. Bu nedenle bölüm boyunca "bu işlem güvenli mi?" sorusunu sorma alışkanlığı kazanacağız.

11.1 String Nedir?

Bellekteki bir `char` dizisinin metin sayılabilmesi için bir kural gerekir: metin nerede bitiyor? Dizinin boyutu bunun için yeterli değildir, çünkü 20 baytlık bir dizide yalnızca ilk üç bayt metin olabilir. C bu soruyu bir sonlandırıcıyla çözer: metnin bittiği yeri, dizinin içine konan özel bir karakter gösterir. Bu karaktere **null karakter** (null character) denir ve `'\0'` biçiminde yazılır.

> **Tanım: String**
>
> C'de string, sonunda kodu 0 olan `'\0'` karakteri bulunan ardışık `char` değerleridir. Metne ait karakterlerin sayısına string'in **uzunluğu** denir; `'\0'` uzunluğa dahil değildir ama bellekte yer kaplar.

`'\0'` ile rakam karakteri `'0'` birbirine karıştırılmamalıdır: `'\0'` değeri 0'dır, `'0'` ise ASCII'de 48'dir. `"Ali"` sözcüğü belleğe şöyle yerleşir:

```
indeks      0      1      2      3
          +------+------+------+------+
          | 'A'  | 'l'  | 'i'  | '\0' |
          +------+------+------+------+
kod          65    108    105      0
```

`"Ali"` üç karakterlidir ama dört bayt yer kaplar. `printf("%s")` gibi fonksiyonlar string'in nerede bittiğini dizinin boyutundan değil, ilk `'\0'` karakterine rastlayınca anlar. Bunu bir programla görelim:

▸ nullKarakter.c

```c
#include <stdio.h>

int main(void) {
    char ad[] = "Ali";

    for (int i = 0; i < 4; i++) {
        printf("ad[%d] kodu %d\n", i, ad[i]);
    }
    printf("sizeof(ad) = %zu\n", sizeof(ad));
    return 0;
}
```

ÇIKTI

```
ad[0] kodu 65
ad[1] kodu 108
ad[2] kodu 105
ad[3] kodu 0
sizeof(ad) = 4
```

Dizinin dördüncü elemanını biz yazmadık; derleyici string sabitinin sonuna kendisi ekledi. `sizeof(ad)` bu yüzden 3 değil 4 verir.

> ◆ **Kritik Nokta · Uzunluk ile bellekte kapladığı yer aynı değildir**
>
> `n` karakterlik bir metni tutmak için dizinin en az `n + 1` elemanlı olması gerekir. `char s[] = "Merhaba";` için `sizeof s` 8'dir (7 karakter ve `'\0'`). Boş string `""` 1 bayt, `"A"` 2 bayt kaplar. Bu fazladan bayt unutulursa sonlandırıcı sığmaz ve tanımsız davranış başlar.

Karakter sabiti ve string sabiti

Tek tırnak ve çift tırnak C'de farklı şeyleri anlatır. `'A'` tek bir karakterin kodudur (bir `int` değeri, 65). `"A"` ise iki baytlık bir dizidir: `'A'` ve `'\0'`.

▸ tirnakFarki.c

```c
#include <stdio.h>

int main(void) {
    char c = 'A';

    printf("sizeof c   : %zu\n", sizeof c);
    printf("sizeof 'A' : %zu\n", sizeof 'A');
    printf("sizeof \"A\" : %zu\n", sizeof "A");
    printf("sizeof \"\"  : %zu\n", sizeof "");
    return 0;
}
```

```
ÇIKTI
sizeof c   : 1
sizeof 'A' : 4
sizeof "A" : 2
sizeof ""  : 1
```

`sizeof 'A'` ifadesinin 4 çıkması C'ye özgü bir ayrıntıdır: C'de karakter sabitinin tipi `int`'tir (C++'ta `char`'dır ve 1 verir). Önemli olan sonuç şudur: `char c = "A";` yazmak hatadır, çünkü iki baytlık bir diziyi tek baytlık bir değişkene koymaya çalışırız. Derleyici bunu uyarı olarak bildirir.

11.2 Tanımlama ve İlk Değer

Bir karakter dizisi üç biçimde tanımlanabilir:

▸ tanimlama.c

```c
#include <stdio.h>

int main(void) {
    char a[20] = "Ali";                  // 20 bayt, kalanı sıfır
    char b[] = "Ali";                    // boyutu derleyici belirler: 4
    char c[] = {'A', 'l', 'i', '\0'};    // eleman eleman, sonlandırıcı bizden

    printf("%s %zu\n", a, sizeof a);
    printf("%s %zu\n", b, sizeof b);
    printf("%s %zu\n", c, sizeof c);
    return 0;
}
```

```
ÇIKTI
Ali 20
Ali 4
Ali 4
```

Üç dizi de aynı metni tutar; yalnızca büyüklükleri farklıdır. `a` dizisinde `"Ali"` yazıldıktan sonra kalan 16 bayt sıfırla doldurulur, yani metnin sonrasında bol bol `'\0'` vardır. `b` için boyutu yazmamız gerekmez: köşeli parantez boş bırakılınca derleyici string sabitinin uzunluğuna bir ekleyerek boyutu kendisi belirler. `c` dizisinde sonlandırıcıyı kendimiz yazdık; bu biçim sıkıcıdır ve nadiren kullanılır.

Üç noktaya dikkat etmek gerekir:

1. Boyut yetersiz olursa taşan karakterler dizide yer bulamaz. `char d[3] = "Ali";` yasaldır ama `'\0'` dizide yer bulamaz; ortaya sonlandırıcısız bir dizi çıkar ve onu `%s` ile yazdırmak tanımsız davranıştır. Metinden daha çok yer ayırmak (ya da boyutu boş bırakmak) bu hatayı önler. `char d[2] = "Ali";` ise fazla ilk değer verdiği için derleyici uyarısına yol açar.
2. `{'A', 'l', 'i'}` biçiminde `'\0'` yazılmazsa dizi string olmaz; `"Ali"` ile bu iki yazım aynı değildir.
3. Dizinin ilk değer verilmeden bırakılması çöp değer demektir (Bölüm 5). `char tampon[50];` tanımlandığında içinde ilk `'\0'` nerede, bilinmez. Bu yüzden okumadan önce `char tampon[50] = "";` yazmak iyi bir alışkanlıktır.

Elemanı değiştirmek, metni değiştirmek

Dizi elemanlarına köşeli parantezle erişebildiğimiz gibi, tek tek yazabiliriz de. Metnin sonu ilk `'\0'` ile belirlendiği için ortaya bir şaşırtıcı sonuç çıkar:

▸ tanimDegistir.c

```c
#include <stdio.h>

int main(void) {
    char s[] = "kar";

    s[0] = 'b';                  // "bar"
    printf("%s\n", s);

    s[1] = '\0';                 // metni erkenden bitir
    printf("%s\n", s);
    printf("dizi boyutu: %zu\n", sizeof s);
    return 0;
}
```

ÇIKTI

```
bar
b
dizi boyutu: 4
```

İkinci yazdırmada yalnızca `b` görünür; `'a'` ve `'r'` karakterleri bellekte durmaya devam eder ama `printf` ilk `'\0'`'da durur. Dizinin boyutu ise değişmemiştir. Yani metnin uzunluğu dizinin boyutundan bağımsızdır ve dizinin içeriğine bağlıdır.

> **▲ Sık Yapılan Hata · Bir diziye atama operatörüyle metin koyamayız**
>
> `=` yalnızca tanımlama sırasında metin verebilir. Sonradan `ad = "Mehmet";` yazmak derleme hatasıdır, çünkü dizi adı atama yapılabilen bir değişken gibi davranmaz. Metni sonradan değiştirmek için `strcpy` (birazdan göreceğiz) ya da eleman eleman atama kullanılır.

▸ atamaHata.c

```c
#include <stdio.h>

int main(void) {
    char ad[10];
    ad = "Mehmet";
    printf("%s\n", ad);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
atamaHata.c: In function 'main':
atamaHata.c:5:8: error: assignment to expression with array type
    5 |     ad = "Mehmet";
      |        ^
```

Bu kısıtın nedenini Bölüm 12'de işaretçileri anlatırken göreceğiz: dizi adı, atama yapılabilen bir değer değil, dizinin başlangıç adresini temsil eden sabit bir ifadedir.

11.3 Yazdırma ve Karakter Karakter Dolaşma

Bir string'i yazdırmanın üç yolu vardır:

- `printf("%s", s)`: biçim belirteciyle yazdırır; satır sonu eklemez.
- `puts(s)`: string'i yazdırır ve sonuna kendisi `'\n'` ekler.
- `putchar(s[i])` ile döngü: karakter karakter yazar.

`%s` belirteci, verilen adresten başlayarak ilk `'\0'` bulunana kadar karakterleri yazar. Bu yüzden sonlandırıcısı olmayan bir dizi, `printf` tarafından dizinin ötesindeki belleğe doğru okunmaya devam edilir. Buna dair bir örneği güvenlik başlığında göreceğiz.

String üzerinde döngü kurarken uzunluğu önceden bilmek gerekmez. Döngü, `'\0'` görene kadar sürer:

▸ dolasma.c

```c
#include <stdio.h>

int main(void) {
    char s[] = "C dili";
    int i = 0;

    while (s[i] != '\0') {
        printf("s[%d] = '%c'\n", i, s[i]);
        i++;
    }
    printf("uzunluk = %d\n", i);

    puts("puts satir sonu ekler");
    printf("printf eklemez");
    putchar('\n');
    return 0;
}
```

ÇIKTI

```
s[0] = 'C'
s[1] = ' '
s[2] = 'd'
s[3] = 'i'
s[4] = 'l'
s[5] = 'i'
uzunluk = 6
puts satir sonu ekler
printf eklemez
```

Döngü bittiğinde `i` değişkeni string'in uzunluğunu tutar. Bu, `strlen` fonksiyonunun yaptığı işin kendisidir. Aynı döngünün `for` ile yazılışı da yaygındır: `for (int i = 0; s[i] != '\0'; i++)`. Koşulu `s[i]` biçiminde kısaltmak da mümkündür, çünkü `'\0'` sıfırdır ve sıfır C'de "yanlış" demektir (Bölüm 5). Ama okunurluk için `!= '\0'` yazmak daha iyidir.

11.4 Klavyeden String Okumak

Girdi alma işini Bölüm 6'da `scanf` ve `fgets` ile görmüştük. Bu bölümde string'e özgü ayrıntılara bakalım.

scanf ile %s

okuScanf.c

```c
#include <stdio.h>

int main(void) {
    char ad[20];
    char soyad[20];

    printf("Ad ve soyad: ");
    scanf("%19s %19s", ad, soyad);
    printf("Ad: %s\n", ad);
    printf("Soyad: %s\n", soyad);
    return 0;
}
```

ÇIKTI

```
Ad ve soyad: Elif Kaya
Ad: Elif
Soyad: Kaya
```

Bu örnekte üç ayrıntı vardır:

- Dizi adının önünde `&` yoktur. Sayı okurken `&sayi` yazıyorduk; dizi adı zaten adres olarak kullanıldığı için `&` gerekmez (nedenini Bölüm 12'de göreceğiz).
- `%19s` içindeki `19`, **en çok 19 karakter oku** demektir. Dizi 20 baytlıktır; bir bayt `'\0'` için kalmalıdır. Genişlik her zaman dizi boyutundan bir eksik yazılır.
- `%s` boşluğa kadar okur. `Elif Kaya` yazıldığında ilk `%19s` yalnızca `Elif`'i alır; ikinci `%19s` kalan `Kaya`'yı alır.

> **▲ Sık Yapılan Hata · scanf("%s") genişliksiz kullanılmamalıdır**
>
> `scanf("%s", ad);` dizinin boyutunu bilmez. Kullanıcı 20 karakterden uzun bir şey yazarsa fazla karakterler dizinin ötesindeki belleğe yazılır. Buna arabellek taşması denir (aşağıda ayrıntısıyla ele alınacak). `scanf` ile string okurken genişlik sınırı yazmak zorunludur: `scanf("%19s", ad)`.

`scanf("%s")` ile boşluklu bir cümle okunamaz. Bunun için `fgets` kullanırız.

fgets ile satır okuma

`fgets(tampon, boyut, stdin)` fonksiyonu en çok `boyut - 1` karakter okur, sonuna `'\0'` ekler ve satır sonuna (`'\n'`) rastlarsa onu da tampona alır. Boyutu fonksiyona biz verdiğimiz için taşma olmaz. Satır `'\n'` ile bitiyorsa bu karakter metnin sonunda kalır; çoğu zaman onu atmak isteriz:

okuFgets.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char satir[50];

    printf("Ad soyad: ");
    fgets(satir, sizeof satir, stdin);
    printf("Ham uzunluk: %zu\n", strlen(satir));
```

```
    satir[strcspn(satir, "\n")] = '\0';     // satır sonunu sil
    printf("Temiz uzunluk: %zu\n", strlen(satir));
    printf("[%s]\n", satir);
    return 0;
}
```

```
ÇIKTI
Ad soyad: Elif Kaya
Ham uzunluk: 10
Temiz uzunluk: 9
[Elif Kaya]
```

`strlen`, string'in uzunluğunu verir. `strcspn(s, "\n")` ise `s` içinde `'\n'` karakterinin ilk bulunduğu indeksi döndürür; bulamazsa string'in uzunluğunu verir. Bu indeksteki elemana `'\0'` yazmak, satır sonu varsa onu siler, yoksa zaten bulunan sonlandırıcının üzerine yazar. İki durumda da doğru çalıştığı için bu deyim `fgets` sonrası temizliğin standart yoludur. `sizeof satir` ifadesi burada dizi boyutunu (50) verir; bu kullanım yalnızca `satir` gerçek bir dizi olduğu yerde doğrudur (fonksiyon parametresinde farklıdır, Bölüm 12).

`fgets` satır sonuna ulaşamadan tampon dolarsa satırın kalanı girdi akışında bekler ve bir sonraki okumada gelir. Çok uzun satırlar bu yüzden iki okumaya bölünebilir; basit programlarda tamponu gerekenden geniş seçmek bu sorunu çoğunlukla ortadan kaldırır.

scanf sonrası fgets tuzağı

Bölüm 6'da gördüğümüz bir tuzak string okumada da karşımıza çıkar. `scanf("%d")` sayıyı okur ama sayının ardından basılan Enter (`'\n'`) girdi akışında kalır. Hemen ardından çağrılan `fgets`, bu artık `'\n'` ile karşılaşıp satırın boş olduğunu sanır ve hiçbir şey beklemeden döner.

▸ okuTuzak.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    int yas;
    char isim[30];

    printf("Yaş: ");
    scanf("%d", &yas);
    printf("İsim: ");
    fgets(isim, sizeof isim, stdin);          // artık '\n'i okur
    printf("fgets %zu karakter okudu\n", strlen(isim));

    printf("Yaş: ");
    scanf("%d", &yas);
    int c;
    while ((c = getchar()) != '\n' && c != EOF) {
        // satırın kalanını at
    }
    printf("İsim: ");
    fgets(isim, sizeof isim, stdin);
    isim[strcspn(isim, "\n")] = '\0';
    printf("fgets %zu karakter okudu: %s\n", strlen(isim), isim);
    return 0;
}
```

```
ÇIKTI
Yaş: 25
İsim: fgets 1 karakter okudu
Yaş: 30
İsim: Mehmet Can
fgets 10 karakter okudu: Mehmet Can
```

İlk bölümde `fgets` kullanıcıdan bir şey beklemeden `"\n"` okudu (uzunluk 1). İkinci bölümde `scanf`'ten sonra `while` döngüsü satırın kalanını tüketti ve `fgets` doğru çalıştı. Bu temizleme döngüsü her yerde kullanılan bir kalıptır: `getchar` ile karakterler `'\n'` ya da dosya sonu (`EOF`) görülene kadar okunup atılır. Burada `c` değişkeninin `char` değil `int` olması gerekir; çünkü `EOF` değeri bir `char` içine sığmayabilir.

> **İpucu · Karışıklığı kökten çözmek**
>
> Bir programda hem sayı hem metin okuyorsak her girdiyi `fgets` ile satır olarak alıp sayıları sonradan `sscanf` ya da `strtol` ile ayrıştırmak (bu bölümün ilerisinde göreceğiz) tuzaktan en temiz kaçış yoludur. Böylece girdi akışında artık karakter kalmaz.

11.5 string.h Fonksiyonları

Metinle sık yapılan işler (uzunluk bulma, kopyalama, birleştirme, karşılaştırma, arama) için standart kütüphane `<string.h>` başlık dosyasında hazır fonksiyonlar sunar. Hepsi aynı ilkeye dayanır: string'in sonunu `'\0'` belirler.

Fonksiyon	Görevi
`strlen(s)`	`'\0'` hariç karakter sayısını verir
`strcpy(hedef, kaynak)`	`kaynak`'ı `'\0'` dahil `hedef`'e kopyalar
`strncpy(hedef, kaynak, n)`	En çok `n` karakter kopyalar
`strcat(hedef, kaynak)`	`kaynak`'ı `hedef`'in sonuna ekler
`strncat(hedef, kaynak, n)`	En çok `n` karakter ekler, sonuna her zaman `'\0'` koyar
`strcmp(a, b)`	İki string'i sözlük sırasıyla karşılaştırır
`strncmp(a, b, n)`	İlk `n` karakteri karşılaştırır
`strchr(s, c)`	`c` karakterinin ilk geçtiği yeri bulur
`strrchr(s, c)`	`c` karakterinin son geçtiği yeri bulur
`strstr(s, alt)`	`alt` metninin ilk geçtiği yeri bulur
`strtok(s, ayrac)`	Metni ayraç karakterlerinden parçalara böler

strlen, strcpy ve strcat

▸ stringFonk.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char ad[20] = "Elif";
    char kopya[20];
```

```c
    printf("strlen = %zu, sizeof = %zu\n", strlen(ad), sizeof ad);

    strcpy(kopya, ad);              // ad'ı kopya'ya kopyala
    strcat(kopya, " Kaya");         // sonuna ekle
    printf("%s\n", kopya);
    printf("uzunluk = %zu\n", strlen(kopya));
    return 0;
}
```

ÇIKTI

```
strlen = 4, sizeof = 20
Elif Kaya
uzunluk = 9
```

`strlen` ile `sizeof` farkı burada açıkça görünür: `strlen` metnin uzunluğunu, yani ilk `'\0'`'a kadar olan karakter sayısını verir (4); `sizeof` dizinin tüm büyüklüğünü verir (20). `strlen` her çağrıda string'i baştan sona gezer, yani maliyeti uzunlukla orantılıdır. Bu yüzden bir döngü koşuluna `i < strlen(s)` yazmak, büyük metinlerde gereksiz yavaşlık yaratır; uzunluğu bir kez bir değişkene almak daha iyidir.

`strcpy` ve `strcat`, hedef dizinin yeterince büyük olup olmadığını **kontrol etmez**. Hedefte yer yoksa fazla karakterler dizinin ötesine yazılır. Yukarıda `kopya` 20 baytlık ve 9 karakterlik metni alıyor; ama bu bir programcı sorumluluğudur.

> ◆ **Kritik Nokta · strcpy ve strcat sınır bilmez**
>
> `strcpy(hedef, kaynak)` yazarken `strlen(kaynak) + 1 <= sizeof hedef` olduğundan emin olunmalıdır. `strcat` için hedefte hem mevcut metin hem eklenecek metin hem `'\0'` sığmalıdır. Kaynağın uzunluğu kullanıcıya bağlıysa bu fonksiyonlar yerine `snprintf` kullanılır (aşağıda).

strncpy ve strncat

`strncpy(hedef, kaynak, n)` en çok `n` karakter kopyalar. Ne var ki ilk bakışta güvenli görünen bu fonksiyonun iki sürprizi vardır: kaynak `n` karakter ya da daha uzunsa hedefe `'\0'` **yazılmaz**; kaynak daha kısaysa hedefin kalanı `'\0'` ile doldurulur. Bu nedenle `strncpy` sonrasında sonlandırıcıyı elle yazmak gerekir:

▸ guvenliKopya.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char kaynak[] = "Merhaba dunya";
    char hedef[6];

    strncpy(hedef, kaynak, sizeof hedef - 1);
    hedef[sizeof hedef - 1] = '\0';          // sonlandırıcıyı biz koyarız
    printf("strncpy: %s\n", hedef);
    return 0;
}
```

ÇIKTI

```
strncpy: Merha
```

`snprintf(hedef, sizeof hedef, "%s", kaynak);` çağrısı aynı işi tek satırda ve her zaman `'\0'` koyarak yapar; en çok `boyut - 1` karakter yazar (gcc, bu örnekte metnin kırpılacağını önceden görüp uyarı verebilir).

Bu yüzden metin kopyalamak ve birleştirmek için çoğu zaman en güvenli ve en sade seçenektir. Tam kullanımını sayı dönüşümlerinde de göreceğiz.

`strncat(hedef, kaynak, n)` ise `strncpy`'nin tersine her zaman `'\0'` ekler; ama `n` hedefteki **boş yer** değil, eklenecek **karakter sayısı** sınırıdır. Yani `strncat(hedef, kaynak, sizeof hedef - strlen(hedef) - 1)` biçiminde yazılmalıdır. Bu karışık kullanım nedeniyle birleştirme işlerinde `snprintf` daha kolaydır.

strcmp ile karşılaştırma

`strcmp(a, b)` iki string'i karakter karakter karşılaştırır ve ilk farklı karakterin koduna bakar:

- `a`, sözlük sırasında `b`'den önceyse **negatif** bir değer,
- eşitse **0**,
- sonraysa **pozitif** bir değer döndürür.

Dönen sayının tam büyüklüğü standartta belirtilmemiştir; yalnızca işaretine güvenilir. Bu yüzden `strcmp(a, b) == -1` gibi koşullar yazılmaz, `< 0`, `== 0`, `> 0` kullanılır.

▸ strcmpOrnek.c

```c
#include <stdio.h>
#include <string.h>

const char *yon(int r) {
    if (r < 0) return "negatif";
    if (r > 0) return "pozitif";
    return "sifir";
}

int main(void) {
    printf("elma, armut : %s\n", yon(strcmp("elma", "armut")));
    printf("armut, elma : %s\n", yon(strcmp("armut", "elma")));
    printf("elma, elma  : %s\n", yon(strcmp("elma", "elma")));
    printf("elma, elmas : %s\n", yon(strcmp("elma", "elmas")));
    printf("Elma, elma  : %s\n", yon(strcmp("Elma", "elma")));
    printf("ilk 2 harf  : %s\n", yon(strncmp("kalem", "kalas", 2)));
    return 0;
}
```

ÇIKTI

```
elma, armut : pozitif
armut, elma : negatif
elma, elma  : sifir
elma, elmas : negatif
Elma, elma  : negatif
ilk 2 harf  : sifir
```

Beşinci satır önemlidir: `'E'` (65) küçük `'e'`'den (101) küçük olduğu için `"Elma"`, `"elma"`'dan önce gelir. `strcmp` büyük/küçük harf duyarlıdır ve ASCII sırasını kullanır; sıralama alfabetik sıra değil kod sırasıdır. Altıncı satırda `strncmp` yalnızca ilk iki karaktere (`ka`) baktığı için sonuç sıfırdır. Bir string başka bir string'in başı ise (`"elma"` ve `"elmas"`), kısa olan önce gelir, çünkü `'\0'` (0) herhangi bir harften küçüktür.

(Örnekteki `yon` fonksiyonunun dönüş tipi `const char *`, yani bir metin sabitine işaret eden bir işaretçidir. Bunu Bölüm 12'de ayrıntılı göreceğiz; şimdilik "bir metin döndürür" diye okumak yeter.)

strchr, strrchr ve strstr

Bu fonksiyonlar aranan karakterin ya da alt metnin **bulunduğu yeri** döndürür. Dönüş değeri, string içindeki o konumu gösteren bir adrestir (işaretçi); bulunamazsa `NULL` döner. Adres `%s` ile yazdırılırsa, o noktadan string'in sonuna kadar olan kısım yazılır. İki adresin farkı ise indeksi verir.

▸ aramaFonk.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char yol[] = "belgeler/rapor.txt";

    char *nokta = strchr(yol, '.');
    if (nokta != NULL) {
        printf("uzanti: %s\n", nokta + 1);
    }

    char *egik = strrchr(yol, '/');
    printf("dosya: %s\n", egik + 1);

    char *bulunan = strstr(yol, "rapor");
    if (bulunan != NULL) {
        printf("rapor indeksi: %td\n", bulunan - yol);
        printf("kalan: %s\n", bulunan);
    }

    if (strstr(yol, "xyz") == NULL) {
        printf("xyz bulunamadi\n");
    }
    return 0;
}
```

```
ÇIKTI
uzanti: txt
dosya: rapor.txt
rapor indeksi: 9
kalan: rapor.txt
xyz bulunamadi
```

`nokta + 1` ifadesi "bulunan yerden bir karakter sonrası" demektir; böylece noktanın kendisi atlanır. `%td`, iki adres farkı için kullanılan biçim belirtecidir. Burada işaretçileri yalnızca bir kullanıcı olarak ele aldık; işaretçi aritmetiğini Bölüm 12'de öğreneceğiz. Önemli kural şudur: bu fonksiyonların döndürdüğü değer **kullanılmadan önce `NULL` ile karşılaştırılmalıdır**. `NULL` olan bir adres üzerinden okuma yapmak programı çökertir.

strtok ile metni parçalamak

`strtok`, bir metni verilen ayraç karakterlerinden bölerek parçaları (token) tek tek verir. İlk çağrıda metin verilir; sonraki çağrılarda `NULL` yazılır ve fonksiyon kaldığı yerden devam eder.

▸ ayristir.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char veri[] = "elma,armut;muz,,kiraz";
```

```
    char *parca = strtok(veri, ",;");

    while (parca != NULL) {
        printf("[%s]\n", parca);
        parca = strtok(NULL, ",;");
    }
    return 0;
}
```

```
ÇIKTI
[elma]
[armut]
[muz]
[kiraz]
```

İkinci argümandaki `",;"` ayraç kümesidir: virgül ve noktalı virgülden herhangi biri parçalama noktası olur. Art arda gelen ayraçlar (`,,`) tek ayraç sayılır, bu yüzden boş parça üretilmez.

`strtok` iki önemli özelliğe sahiptir:

- Metnin **kendisini değiştirir**: ayraçların yerine `'\0'` yazar. Bu nedenle değiştirilemeyen string sabitleri üzerinde kullanılamaz; yukarıda olduğu gibi bir `char` dizisine kopyalanmış metin gerekir.
- Kaldığı yeri fonksiyon içinde saklar, yani aynı anda iki metni `strtok` ile gezemeyiz.

11.6 == ile Karşılaştırma ve = ile Atama Tuzağı

Sayılarda `a == b` ile eşitliğe bakarız. String'lerde bu alışkanlık sessizce yanlış sonuç verir:

```
char a[] = "elma";
char b[] = "elma";

if (a == b) {
    printf("esit\n");
} else {
    printf("farkli\n");        // ekrana bu yazılır
}
```

Dizi adları metnin kendisini değil, dizilerin bellekteki **başlangıç adresini** temsil eder. `a == b` bu iki adresi karşılaştırır. İki ayrı dizi bellekte farklı yerlerde durduğu için adresler hiçbir zaman eşit değildir; içerikleri aynı olsa bile sonuç yanlıştır. Doğru yol `strcmp` kullanmaktır:

```
if (strcmp(a, b) == 0) {
    printf("esit\n");
}
```

Aynı mantıkla `b = a;` ile bir diziyi bir diğerine atayamayız (yukarıda gördüğümüz `assignment to expression with array type` hatası); kopyalama için `strcpy` ya da `snprintf` gerekir. Bunlar, C'nin dizilere sayı gibi değer tipi muamelesi yapmamasının sonuçlarıdır. C++ ve C#'taki string tipleri bu yükü ortadan kaldırır; bölümün sonundaki karşılaştırmaya bakalım.

11.7 Kendi String Fonksiyonlarımızı Yazmak

Standart fonksiyonların nasıl çalıştığını anlamanın en iyi yolu bir tanesini kendimiz yazmaktır. Aşağıdaki programda dört küçük fonksiyon var: uzunluk bulma, kopyalama, ters çevirme ve palindrom denetimi. Fonksiyon parametresi olarak dizi aldıkları için Bölüm 10'daki dizi geçirme kuralları geçerlidir (dizi kopyalanmaz, fonksiyon özgün diziyi değiştirebilir).

kendiFonksiyon.c

```c
#include <stdio.h>

int uzunluk(const char s[]) {
    int n = 0;
    while (s[n] != '\0') {
        n++;
    }
    return n;
}

void kopyala(char hedef[], const char kaynak[]) {
    int i = 0;
    while (kaynak[i] != '\0') {
        hedef[i] = kaynak[i];
        i++;
    }
    hedef[i] = '\0';                  // sonlandırıcıyı unutmayalım
}

void tersCevir(char s[]) {
    int bas = 0;
    int son = uzunluk(s) - 1;

    while (bas < son) {
        char gecici = s[bas];
        s[bas] = s[son];
        s[son] = gecici;
        bas++;
        son--;
    }
}

int palindromMu(const char s[]) {
    int bas = 0;
    int son = uzunluk(s) - 1;

    while (bas < son) {
        if (s[bas] != s[son]) {
            return 0;
        }
        bas++;
        son--;
    }
    return 1;
}

int main(void) {
    char a[] = "programlama";
    char b[30];

    printf("uzunluk: %d\n", uzunluk(a));
    kopyala(b, a);
    tersCevir(b);
```

```
    printf("ters: %s\n", b);
    printf("orijinal: %s\n", a);

    printf("radar: %s\n", palindromMu("radar") ? "evet" : "hayir");
    printf("kayak: %s\n", palindromMu("kayak") ? "evet" : "hayir");
    printf("merhaba: %s\n", palindromMu("merhaba") ? "evet" : "hayir");
    return 0;
}
```

```
ÇIKTI
uzunluk: 11
ters: amalmargorp
orijinal: programlama
radar: evet
kayak: evet
merhaba: hayir
```

Programdaki fikirler şunlardır:

- `kopyala` fonksiyonunun sonunda `hedef[i] = '\0';` satırı olmazsa hedef string olmaktan çıkar. String yazan her fonksiyonun son işi sonlandırıcıyı koymaktır.
- `tersCevir` ve `palindromMu` iki indeksi iki uçtan ortaya doğru yürütür. `bas < son` koşulu, ortada kalan tek karakterin (tek uzunluklu metinlerde) kendisiyle karşılaştırılmasını önler.
- Fonksiyonların değişmeyen parametrelerinde `const` kullandık: `uzunluk` ve `palindromMu` metni yalnızca okur, değiştirmez. `const`, derleyiciye "bu fonksiyon bu diziyi değiştirmeyecek" garantisi verir ve yanlışlıkla yazmaya çalışırsak hata bildirir.

11.8 ctype.h: Karakter Denetimi ve Dönüşümü

`<ctype.h>` başlığı, tek bir karakter üzerinde sorgu ve dönüşüm yapan fonksiyonları içerir:

Fonksiyon	Doğru döndürdüğü durum
`isalpha(c)`	Harf (`a-z` , `A-Z`)
`isdigit(c)`	Rakam (`0-9`)
`isalnum(c)`	Harf ya da rakam
`isupper(c)` , `islower(c)`	Büyük harf, küçük harf
`isspace(c)`	Boşluk, sekme, satır sonu gibi boşluk karakterleri
`ispunct(c)`	Noktalama işareti
`toupper(c)` , `tolower(c)`	Büyük/küçük harf karşılığını döndürür

`is...` fonksiyonları doğruysa sıfırdan farklı, yanlışsa 0 döndürür; bu yüzden doğrudan `if` içinde kullanılırlar. Karakter dönüşümü yapan `toupper` ve `tolower` ise karakterin kendisini değiştirmez, yeni değeri **döndürür**. Dönüşüm uygulanamayan karakterler (rakam, noktalama) aynen geri gelir.

▸ ctypeOrnek.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char s[] = "Merhaba Dunya 2024!";
```

```
    int harf = 0, rakam = 0, bosluk = 0, noktalama = 0;

    for (int i = 0; s[i] != '\0'; i++) {
        unsigned char c = (unsigned char)s[i];
        if (isalpha(c)) {
            harf++;
        } else if (isdigit(c)) {
            rakam++;
        } else if (isspace(c)) {
            bosluk++;
        } else if (ispunct(c)) {
            noktalama++;
        }
    }
    printf("harf=%d rakam=%d bosluk=%d noktalama=%d\n",
           harf, rakam, bosluk, noktalama);

    for (int i = 0; s[i] != '\0'; i++) {
        s[i] = (char)toupper((unsigned char)s[i]);
    }
    printf("%s\n", s);
    return 0;
}
```

```
ÇIKTI
harf=12 rakam=4 bosluk=2 noktalama=1
MERHABA DUNYA 2024!
```

Gördüğümüz `(unsigned char)` dönüşümünün bir nedeni vardır: `<ctype.h>` fonksiyonları argüman olarak ya `unsigned char` değer aralığında (0–255) ya da `EOF` bekler. `char` işaretli olan sistemlerde 127'den büyük kodlu bir karakter (örneğin UTF-8 metnin bir baytı) negatif sayıya dönüşür ve tanımsız davranışa yol açar. Dönüşümü yazmak bu riski ortadan kaldırır.

> **▲ Sık Yapılan Hata · toupper(s) ile s büyümez**
>
> `toupper(s[i]);` satırını tek başına yazmak hiçbir şey yapmaz; çünkü fonksiyon karakteri değiştirmez, sonucu döndürür. Dönüş değerini bir yere atamak gerekir: `s[i] = toupper(s[i]);` .

11.9 Metin ile Sayı Arasında Dönüşüm

Rakam karakterinden sayıya

Tek bir rakam karakterini sayıya çevirmek için Bölüm 5'te gördüğümüz `'0'` çıkarma yöntemini kullanırız: `'7' - '0'` ifadesi 7 verir. Ters yönde, 0–9 arası bir sayıyı karaktere çevirmek için `'0'` eklenir.

atoi, strtol ve strtod

Çok basamaklı bir metni sayıya çevirmek için `<stdlib.h>` içindeki fonksiyonlar kullanılır:

- `atoi(s)` : metni `int` 'e çevirir. Basittir ama hata bildirme yolu yoktur: `atoi("abc")` ile `atoi("0")` ikisi de 0 döndürür.
- `strtol(s, &son, taban)` : metni `long` 'a çevirir; dönüşümün durduğu yeri `son` işaretçisine yazar. Böylece metnin ne kadarının sayı olduğu anlaşılır. `taban` 10 ise onluk, 16 ise onaltılık okur.
- `strtod(s, &son)` : metni `double` 'a çevirir; `strtol` gibi çalışır.

Ters yönde, sayıyı metne çevirmek için `snprintf` kullanırız. Çalışma biçimi `printf` ile aynıdır; tek fark sonucu ekrana değil bir dizinin içine yazmasıdır.

▸ sayiDonusum.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    char a[] = "123";
    printf("atoi: %d\n", atoi(a) + 1);

    char giris[] = "42abc";
    char *son;
    long v = strtol(giris, &son, 10);
    printf("strtol: %ld, kalan: %s\n", v, son);

    char hatali[] = "abc";
    strtol(hatali, &son, 10);
    if (son == hatali) {
        printf("\"%s\" icinde sayi yok\n", hatali);
    }

    double d = strtod("3.14xyz", &son);
    printf("strtod: %.2f, kalan: %s\n", d, son);

    char tampon[40];
    int n = snprintf(tampon, sizeof tampon, "%d x %d = %d", 6, 7, 6 * 7);
    printf("%s\n", tampon);
    printf("yazilan karakter sayisi: %d\n", n);
    return 0;
}
```

ÇIKTI

```
atoi: 124
strtol: 42, kalan: abc
"abc" icinde sayi yok
strtod: 3.14, kalan: xyz
6 x 7 = 42
yazilan karakter sayisi: 10
```

`strtol(giris, &son, 10)` çağrısında `son` değişkeninin adresini veriyoruz (`&son`); böylece fonksiyon bu değişkene, dönüşümün durduğu yeri yazabiliyor. Dönüşüm hiç yapılamazsa (`"abc"` örneği) `son` başlangıç adresinde kalır, yani `son == hatali` olur. Kullanıcıdan gelen sayıyı güvenle okumanın kalıbı budur: satırı `fgets` ile al, `strtol` ile çevir, `son` ile hatayı denetle. `snprintf` ise yazması gereken karakter sayısını döndürür; bu değer `boyut` ya da daha büyükse metin kırpılmıştır. `sprintf` adlı sınır bilmeyen kardeşi kullanılmamalıdır.

> **İpucu · sscanf ile ayrıştırma**
>
> `sscanf(metin, "%d %d", &a, &b)` fonksiyonu, `scanf`'in aynısıdır ama klavye yerine bir string'den okur. Dönüş değeri başarıyla okunan öğe sayısıdır. `fgets` ile aldığımız satırı `sscanf` ile ayrıştırmak, girdi akışında artık karakter bırakmadığı için güvenlidir.

11.10 Güvenlik: Arabellek Taşması

C'de dizilere sınır denetimi yoktur (Bölüm 10). String işlemlerinde bu durum, bilgisayar güvenliğinin en eski ve en yaygın zayıflığının kaynağıdır: **arabellek taşması** (buffer overflow). Bir dizi, boyutundan fazla veriyle doldurulduğunda fazla bayt komşu belleğe yazılır. Komşuda başka bir değişken, bir dönüş adresi ya da programın akışını belirleyen bir veri olabilir.

> **Tanım: Arabellek taşması**
>
> Bir dizi (arabellek) için ayrılan bellek alanının dışına veri yazılmasıdır. Sonuç tanımsız davranıştır: program çökebilir, başka değişkenlerin değeri bozulabilir ya da kötü niyetli bir kullanıcı programın akışını ele geçirebilir.

Aşağıdaki parça taşmanın nasıl oluştuğunu gösterir. Bu kodu **çalıştırmayacağız**; sonucu tanımsızdır.

```
char kisa[8];
strcpy(kisa, "Bu metin cok uzun");    // 18 bayt gerekir, yalnızca 8 var
```

Bellekte `kisa` dizisinin hemen yanında başka veriler durur. Taşan 10 bayt onların üzerine yazılır:

```
Taşmadan önce:
+----------------+----------------------+
| kisa[8]        | komşu veri           |
+----------------+----------------------+

Taşmadan sonra:
+----------------+----------------------+
| "Bu metin "    | "cok uzun\0" (bozuk) |
+----------------+----------------------+
         kisa dizisi sınırı aşıldı
```

Derleyici çoğu durumda hata vermez; program hatayı fark etmeden çalışmaya devam edebilir. En kötü ihtimal budur, çünkü hata başka bir makinede ya da başka bir girdiyle ortaya çıkar. Taşmayı yakalamak için derleme sırasında `-fsanitize=address` seçeneği kullanılabilir; bu seçenek çalışma anında sınır dışı yazmaları denetler (geliştirme sırasında kullanılır, hazır programda kapatılır).

gets neden kaldırıldı?

Eski programlarda satır okumak için `gets(tampon)` kullanılırdı. Bu fonksiyon tamponun boyutunu hiç öğrenemez: kullanıcı ne kadar uzun yazarsa o kadarını yazar. Taşmayı engellemenin hiçbir yolu olmadığı için `gets`, C99'un bir düzeltmesinde kullanımdan kaldırılmış (deprecated), **C11 standardında ise dilden tamamen çıkarılmıştır**. Bugün yerine her zaman `fgets` kullanırız: o, boyutu parametre olarak alır.

> **◆ Kritik Nokta · Güvenli string yazmanın kuralları**
>
> 1. Okumada: `scanf("%s")` yerine genişlikli `scanf("%19s")` ya da boyut alan `fgets`; asla `gets`.
> 2. Kopyalamada ve birleştirmede: hedefin yeterli olduğunu önce doğrula, ya da `snprintf` kullan; `sprintf` kullanma.
> 3. Her string yazan koddan sonra `'\0'` için yer bıraktığından emin ol: `n` karakter için `n + 1` bayt.
> 4. Boyutu sabit sayıyı kopyalayarak değil, `sizeof dizi` ile hesapla; böylece dizi büyütüldüğünde kod bozulmaz.

11.11 Türkçe Karakterler ve UTF-8

Şimdiye kadar yalnızca ASCII karakterleriyle çalıştık: her karakter bir baytlık bir kod. ASCII tablosunda `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` gibi Türkçe harfler yoktur. Bu harfleri taşıyabilmek için bugün çoğu sistem **UTF-8** kodlamasını kullanır. UTF-8'de ASCII karakterleri yine bir bayttır; ama Türkçe harfler iki, bazı diğer simgeler üç ya da dört bayt yer kaplar.

Bu durum C'nin string işlevleri için önemli bir sonuç doğurur: `char` bir bayttır, karakter değil. `strlen` bayt sayar. Kaynak dosya UTF-8 olarak kaydedildiğinde şu program çalışır:

▸ utf8Bayt.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "Ağaç";
    size_t n = strlen(s);

    printf("strlen = %zu\n", n);
    for (size_t i = 0; i < n; i++) {
        printf("%02X ", (unsigned char)s[i]);
    }
    printf("\n");

    int karakter = 0;
    for (size_t i = 0; i < n; i++) {
        if (((unsigned char)s[i] & 0xC0) != 0x80) {
            karakter++;
        }
    }
    printf("karakter sayisi = %d\n", karakter);
    return 0;
}
```

ÇIKTI

```
strlen = 6
41 C4 9F 61 C3 A7
karakter sayisi = 4
```

`"Ağaç"` dört harflidir ama 6 bayt yer kaplar: `A` (41), `ğ` (C4 9F), `a` (61), `ç` (C3 A7). `ğ` ve `ç` ikişer bayttır. Programın son bölümü UTF-8'in bir özelliğinden yararlanır: bir karakterin ilk baytı dışındaki devam baytları ikilik düzende `10` ile başlar (`0x80`–`0xBF`). Devam baytlarını saymayıp geri kalanları saymak karakter sayısını verir. Bu `&` işlemi bir bit işlemidir; ayrıntısını Bölüm 16'da göreceğiz.

Bu durumun sonuçlarını listeleyelim:

- `strlen("çay")` 3 değil 4 döner.
- Metnin ters çevrilmesi gibi bayt bayt çalışan algoritmalar, Türkçe harflerde karakteri ikiye bölerek bozuk çıktı üretir.
- `toupper('ç')` çalışmaz; `<ctype.h>` fonksiyonları tek baytlık karakterler içindir. Üstelik klasik Türkçe büyük/küçük harf kuralı (`i` ile `İ`) bu fonksiyonlarda zaten yoktur.
- Windows konsolu varsayılan olarak başka bir kodlamayla çalışabilir; ekranda bozuk karakter görünürse Bölüm 3'te anlattığımız UTF-8 ayarlarına bakılmalıdır.

Standart kütüphane geniş karakter tipi (`wchar_t`) ve çok baytlı karakterleri işleyen fonksiyonlar da sunar; ancak güvenilir çok dilli metin işleme için pratikte genellikle özel kütüphaneler (ICU gibi) kullanılır. Bu kitapta C'deki çoğu algoritmayı ASCII ile kuracağız; Türkçe karakterleri ise yalnızca ekrana yazılan sabit metinlerde (`printf("Merhaba, dünya\n")` gibi) kullanacağız.

> **Not · C++ ve C#'ta durum**
>
> C#'ta `string` içindeki her öğe UTF-16 kodlu bir `char`'dır ve `"Ağaç".Length` 4 verir. C++'ın `std::string`'i ise C gibi bayt tabanlıdır; `std::string("Ağaç").size()` 6 döner. C++'ta Unicode için ek kütüphaneler gerekir.

11.12 String Dizisi

Bir metin listesi (gün adları, öğrenci adları) tutmak istersek **iki boyutlu `char` dizisi** kullanırız. Her satır bir string'dir; sütun sayısı, en uzun metne ve `'\0'` için yer bırakacak kadar büyük seçilir:

▸ stringDizisi.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char gunler[3][10] = {"Pazartesi", "Sali", "Carsamba"};

    printf("toplam bayt: %zu\n", sizeof gunler);
    for (int i = 0; i < 3; i++) {
        printf("%d: %-10s uzunluk=%zu\n", i, gunler[i], strlen(gunler[i]));
    }
    return 0;
}
```

```
ÇIKTI
toplam bayt: 30
0: Pazartesi  uzunluk=9
1: Sali       uzunluk=4
2: Carsamba   uzunluk=8
```

`gunler[i]` bir satırı, yani bir string'i verir. `%-10s` belirteci metni 10 karakterlik alana sola yaslı yazar (Bölüm 6). Bu yapı basittir ama bellek israf eder: en uzun gün adı 9 karakter olduğu için her satıra 10 bayt ayırdık, `"Sali"` için 5 baytı kullanmadan bıraktık. Satırlar farklı uzunlukta olacaksa bunun yerine her biri bir string sabitine işaret eden **işaretçi dizisi** kullanılır:

```c
const char *aylar[] = {"Ocak", "Subat", "Mart"};
printf("%s\n", aylar[1]);       // Subat
```

Bu kullanım yalnızca okumaya uygundur (string sabitleri değiştirilemez). İşaretçi dizilerini Bölüm 12'de ayrıntılı ele alacağız.

String dizilerinde en sık yapılan iş sıralamadır. Bölüm 10'daki kabarcık sıralamasını yeniden kullanıyoruz; tek fark karşılaştırma için `strcmp`, yer değiştirme için `strcpy` kullanmamızdır:

isimSirala.c

```c
#include <stdio.h>
#include <string.h>

#define ADET 6
#define UZUNLUK 16

int main(void) {
    char isimler[ADET][UZUNLUK] = {
        "Mert", "Ayse", "Zeynep", "Can", "Elif", "Burak"
    };
    char gecici[UZUNLUK];

    for (int tur = 0; tur < ADET - 1; tur++) {
        for (int j = 0; j < ADET - 1 - tur; j++) {
            if (strcmp(isimler[j], isimler[j + 1]) > 0) {
                strcpy(gecici, isimler[j]);
                strcpy(isimler[j], isimler[j + 1]);
                strcpy(isimler[j + 1], gecici);
            }
        }
    }

    for (int i = 0; i < ADET; i++) {
        printf("%s\n", isimler[i]);
    }
    return 0;
}
```

ÇIKTI

```
Ayse
Burak
Can
Elif
Mert
Zeynep
```

`isimler[j] > isimler[j + 1]` yazamayız; string'ler `strcmp` ile karşılaştırılır ve yer değiştirme için üçüncü bir dizi (`gecici`) gerekir.

11.13 Klasik String Algoritmaları

Kelime sayma

Bir metinde kelime sayısını bulmak için "şu an bir kelimenin içinde miyiz?" bilgisini tutan bir bayrak (flag) kullanırız. Boşluktan boşluk olmayan karaktere geçiş yeni bir kelimenin başıdır:

kelimeSay.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char metin[] = "  bir iki   uc  dort ";
    int kelime = 0;
    int icinde = 0;

    for (int i = 0; metin[i] != '\0'; i++) {
```

```
        if (isspace((unsigned char)metin[i])) {
            icinde = 0;
        } else if (!icinde) {
            icinde = 1;
            kelime++;
        }
    }
    printf("kelime sayisi: %d\n", kelime);
    return 0;
}
```

```
ÇIKTI
kelime sayisi: 4
```

Harf frekansı

Her harfin kaç kez geçtiğini bulmak için 26 elemanlı bir sayaç dizisi kullanılır. `'a'`'dan başlayan harfler ardışık kodlara sahip olduğundan `c - 'a'` ifadesi harfin sayaç indeksini verir (`'a'` için 0, `'b'` için 1 ...):

▸ harfFrekans.c

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char metin[] = "Merhaba Dunya";
    int sayac[26] = {0};

    for (int i = 0; metin[i] != '\0'; i++) {
        unsigned char c = (unsigned char)metin[i];
        if (isalpha(c)) {
            sayac[tolower(c) - 'a']++;
        }
    }

    for (int k = 0; k < 26; k++) {
        if (sayac[k] > 0) {
            printf("%c: %d\n", 'a' + k, sayac[k]);
        }
    }
    return 0;
}
```

```
ÇIKTI
a: 3
b: 1
d: 1
e: 1
h: 1
m: 1
n: 1
r: 1
u: 1
y: 1
```

`sayac[tolower(c) - 'a']++` satırı küçük harfe çevirip indeksi hesaplar ve ilgili sayacı artırır. Bu yöntem yalnızca ASCII harfleri için çalışır; `ç` gibi harfler `isalpha` tarafından zaten harf sayılmaz (yukarıdaki UTF-8 uyarısı).

Sezar şifresi

Sezar şifresi, her harfi alfabede belirli sayıda ileri kaydırır. Alfabenin sonunu geçince başa döneriz, bu da `% 26` ile sağlanır. Negatif kaydırmalarda (çözme) `%` sonucu negatif olabileceğinden önce 26 ekleyip tekrar `% 26` alırız:

▸ sezar.c

```c
#include <stdio.h>
#include <ctype.h>

void sezar(char s[], int kaydir) {
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (isupper((unsigned char)c)) {
            s[i] = (char)('A' + ((c - 'A' + kaydir) % 26 + 26) % 26);
        } else if (islower((unsigned char)c)) {
            s[i] = (char)('a' + ((c - 'a' + kaydir) % 26 + 26) % 26);
        }
    }
}

int main(void) {
    char metin[] = "Merhaba Dunya!";

    sezar(metin, 3);
    printf("sifreli: %s\n", metin);
    sezar(metin, -3);
    printf("cozulmus: %s\n", metin);
    return 0;
}
```

ÇIKTI

```
sifreli: Phukded Gxqbd!
cozulmus: Merhaba Dunya!
```

Harf olmayan karakterler (boşluk, `!`) değişmeden kalır. Şifrelemeyi çözmek aynı fonksiyonu ters işaretli kaydırmayla çağırmaktır. Bu şifre günümüzde güvenli değildir (yalnızca 25 olası anahtar vardır); algoritma düşüncesini göstermek için iyi bir örnektir.

Üç Dilde · Üç dilde metin *C · C++ · C#*

C'de metin `'\0'` ile biten `char` dizisidir; uzunluk, kopyalama ve karşılaştırma için fonksiyon çağırırız. C++ ve C#'ta bu işlerin çoğu tipin kendisine yerleşmiştir.

İş	C	C++ (`std::string`)	C# (`string`)
Tanımlama	`char s[] = "Ali";`	`std::string s = "Ali";`	`string s = "Ali";`
Uzunluk	`strlen(s)`	`s.size()`	`s.Length`
Birleştirme	`strcat` (yer ayır!)	`s + " Kaya"`	`s + " Kaya"`
Karşılaştırma	`strcmp(a, b) == 0`	`a == b`	`a == b`
Atama	`strcpy`	`b = a;`	`b = a;`
Değiştirilebilir mi	Evet (dizi elemanları)	Evet	Hayır (değişmez, immutable)
Boyut taşması	Programcı sorumluluğu	Otomatik büyür	Otomatik yönetilir

> C++'ta `std::string` gerektiğinde bellek ayırır ve `==` operatörü içerikleri karşılaştırır; böylece yukarıdaki tuzağın büyük kısmı ortadan kalkar. C#'ta `string` **değişmez** bir tiptir: `s[0] = 'b';` hata verir; her değişiklik yeni bir string üretir. Bu konuları ilgili bölümlerde ayrıntılı göreceğiz.

Bölüm Özeti

- C'de ayrı bir string tipi yoktur; **string**, sonunda `'\0'` (kodu 0) bulunan bir `char` dizisidir. Uzunluk `'\0'` hariç karakter sayısıdır; bellekte kapladığı yer bir bayt fazladır (`char s[] = "Ali"` için `sizeof s == 4`).
- `'A'` bir karakter sabitidir (C'de `int` tipinde, 65); `"A"` iki baytlık bir string'dir (`'A'` ve `'\0'`).
- Tanımlama biçimleri: `char a[20] = "Ali";` , `char b[] = "Ali";` , `char c[] = {'A', 'l', 'i', '\0'};` . Boyut en az `uzunluk + 1` olmalıdır.
- Bir diziye `=` ile sonradan metin atanamaz; kopyalama için `strcpy` ya da `snprintf` kullanılır. Metnin sonu ilk `'\0'` ile belirlenir.
- Yazdırma: `printf("%s")` , `puts` (satır sonu ekler), karakter karakter `s[i] != '\0'` döngüsü.
- Okuma: `scanf("%19s", ad)` boşluğa kadar okur ve genişlik sınırı zorunludur (boyut - 1); boşluklu satırlar için `fgets(tampon, sizeof tampon, stdin)` kullanılır. Satır sonu `tampon[strcspn(tampon, "\n")] = '\0';` ile silinir.
- `scanf("%d")` sonrasında girdi akışında kalan `'\n'` , ardından gelen `fgets` 'i boş bırakır; satırın kalanı `getchar` döngüsüyle temizlenir ya da tüm girdi `fgets` ile alınır.
- `<string.h>` : `strlen` (uzunluk), `strcpy` / `strncpy` (kopyalama), `strcat` / `strncat` (ekleme), `strcmp` / `strncmp` (karşılaştırma; yalnızca işaretine güvenilir), `strchr` / `strrchr` / `strstr` (arama; bulunamazsa `NULL`), `strtok` (metni parçalama; metni değiştirir).
- String'ler `==` ile karşılaştırılamaz: `==` içeriği değil adresleri karşılaştırır. Eşitlik için `strcmp(a, b) == 0` yazılır.
- `strcpy` , `strcat` , `sprintf` ve `gets` hedef boyutunu bilmez; taşmaya yol açabilir. `gets` C11'de dilden çıkarılmıştır. Güvenli seçenekler `fgets` , genişlikli `scanf` ve `snprintf` 'tir.
- `<ctype.h>` : `isalpha` , `isdigit` , `isspace` , `isupper` , `ispunct` sorgulama yapar; `toupper` ve `tolower` karakteri değiştirmez, yeni değeri döndürür. Argüman `(unsigned char)` olarak verilmelidir.
- Rakam karakteri sayıya `c - '0'` ile çevrilir. Metinden sayıya `atoi` (basit, hata bildirmez), `strtol` / `strtod` (kalan metni ve hatayı bildirir); sayıdan metne `snprintf` kullanılır.
- `char` bir bayttır, karakter değil. UTF-8'de Türkçe harfler iki bayttır; bu nedenle `strlen("ğ")` 2 verir ve bayt bayt çalışan algoritmalar Türkçe harflerde bozulur.
- Metin listeleri iki boyutlu `char` dizisiyle (`char isimler[N][UZUNLUK]`) tutulur; sıralamada karşılaştırma için `strcmp` , yer değiştirme için `strcpy` kullanılır.
- C++'ta `std::string` ve C#'ta `string` , uzunluk yönetimini, birleştirmeyi ve `==` ile içerik karşılaştırmasını tipin kendisine yerleştirir.

Alıştırmalar

1. `char s[] = "Merhaba";` tanımından sonra `sizeof s` değeri kaçtır?

A) 6
B) 7
C) 8
D) 9
E) Derleyiciye bağlıdır

2. Aşağıdakilerden hangisi `"A"` ile `'A'` arasındaki farkı doğru anlatır?

A) İkisi de aynı şeydir; yalnızca yazım biçimi farklıdır.
B) `"A"` tek karakterli bir `char` değişkenidir, `'A'` iki karakterli bir dizidir.
C) `'A'` bir karakter sabitidir (kodu 65); `"A"` ise `'A'` ve `'\0'` karakterlerinden oluşan iki baytlık bir dizidir.
D) `"A"` sonlandırıcı içermez, `'A'` içerir.
E) `'A'` yalnızca `printf` içinde kullanılabilir.

3. Aşağıdaki programın çıktısı nedir?

```c
#include <stdio.h>

int main(void) {
    char s[] = "Merhaba";
    s[3] = '\0';
    printf("%s %zu\n", s, sizeof s);
    return 0;
}
```

4. `strcmp("Zeynep", "ali")` çağrısı hakkında hangisi doğrudur?

A) 0 döner, çünkü büyük/küçük harf yok sayılır.
B) Pozitif bir değer döner, çünkü `Z` alfabede `a`'dan sonra gelir.
C) Negatif bir değer döner, çünkü `'Z'` (90) kodu `'a'` (97) kodundan küçüktür.
D) Her zaman -1 döner.
E) Derleme hatası verir.

5. Aşağıdaki programdaki hatayı bulun ve düzeltin.

```c
#include <stdio.h>

int main(void) {
    char ad[10];
    ad = "Can";
    printf("%s\n", ad);
    return 0;
}
```

6. Aşağıdaki kod parçası iki string'in aynı olup olmadığını denetlemek istiyor. Neden güvenilir değildir ve nasıl düzeltilir?

```c
char a[20];
char b[20];
fgets(a, sizeof a, stdin);
fgets(b, sizeof b, stdin);
```

```
if (a == b) {
    printf("Ayni\n");
}
```

7. Kullanıcı `fgets(s, sizeof s, stdin)` çağrısına `Ali` yazıp Enter'a bastığında, `s` yeterince büyükse `strlen(s)` kaç döner?

   A) 2
   B) 3
   C) 4
   D) 5
   E) Tanımsızdır

8. Aşağıdaki programın çıktısı nedir?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "bir.iki.uc";
    char *p = strrchr(s, '.');
    printf("%s\n", p + 1);
    printf("%td\n", strchr(s, '.') - s);
    return 0;
}
```

9. Aşağıdakilerden hangisi arabellek taşmasına **yol açabilir**?

   A) `char t[20]; fgets(t, sizeof t, stdin);`
   B) `char t[20]; scanf("%19s", t);`
   C) `char t[20]; snprintf(t, sizeof t, "%s", girdi);`
   D) `char t[20]; scanf("%s", t);`
   E) `char t[20] = "";`

10. Aşağıdaki programın çıktısı nedir?

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char s[] = "a1b2";
    for (int i = 0; s[i] != '\0'; i++) {
        if (isdigit((unsigned char)s[i])) {
            s[i] = '*';
        } else {
            s[i] = (char)toupper((unsigned char)s[i]);
        }
    }
    printf("%s\n", s);
    return 0;
}
```

11. UTF-8 kodlamalı bir kaynak dosyada `strlen("şeker")` kaç döner? (`ş` iki bayttır.)

    A) 4
    B) 5
    C) 6

D) 7

E) Tanımsızdır

12. Aşağıdaki programın çıktısı nedir?

```c
#include <stdio.h>

int main(void) {
    char t[6];
    int n = snprintf(t, sizeof t, "%s", "Merhaba");
    printf("%s %d\n", t, n);
    return 0;
}
```

13. Aşağıdaki kodda `strcpy` çağrısının ne sakıncası vardır? Nasıl düzeltilebilir?

```c
char kod[8];
char girdi[100];
fgets(girdi, sizeof girdi, stdin);
strcpy(kod, girdi);
```

14. **Program yazın:** Bir `char` dizisinde tutulan metindeki sesli harflerin (`a, e, i, o, u`) sayısını bulan bir fonksiyon yazın. Büyük ve küçük harfleri aynı sayın. Fonksiyonu `"Merhaba Dunya"` metniyle deneyin.

15. **Program yazın:** Kullanıcıdan `fgets` ile bir satır okuyun. Satırın palindrom olup olmadığını, harf olmayan karakterleri atlayarak ve büyük/küçük harf farkını yok sayarak belirleyin (ör. `"Ey Edip Adanada pide ye"` ve `"A man a plan a canal Panama"` palindromdur).

16. **Program yazın:** Bir `int` değeri alıp onu `char` dizisine onluk sayı metni olarak yazan `sayiyiMetneCevir(int n, char hedef[])` fonksiyonunu `snprintf` kullanmadan yazın. Negatif sayıları ve sıfırı da doğru işleyin (`INT_MIN` değerini göz ardı edebilirsiniz). Rakamları sondan başa doğru üretip sonra diziyi ters çevirebilirsiniz.

BÖLÜM 12

İşaretçiler (Pointers)

BU BÖLÜMDE

- Her değişkenin bellekte bir adresi olduğunu; adresi `&` ile almayı ve `%p` ile yazdırmayı
- İşaretçi tanımlamayı, `*` ile gösterilen yere erişmeyi (dolaylama) ve `int *p, q;` tuzağını
- `NULL` işaretçiyi, başlatılmamış ve sarkan işaretçilerin tehlikesini
- Adresle geçirmeyi: çalışan `swap`, birden çok sonuç döndürme ve `scanf`'teki `&`'ın gerçek anlamını
- İşaretçi aritmetiğini, `a[i]` ile `*(a + i)` eşdeğerliğini ve dizilerin fonksiyona geçişini
- İşaretçi ve stringleri; `const` ile işaretçi birleşimlerini (dört kombinasyon)
- İşaretçi dizisini, `**` türünü, `argc` / `argv`'yi, `void *` işaretçiyi ve fonksiyon işaretçisini (`qsort`)

Bu bölüm kitabın en kritik bölümüdür, çünkü önceki bölümlerde üç soruyu "sonra anlatacağız" diye erteledik: `scanf`'te değişkenin önüne neden `&` konduğunu (Bölüm 3), Bölüm 9'daki `takas` fonksiyonunun nasıl çalışır hâle getirileceğini ve dizilerin fonksiyona neden kopyalanmadan geçtiğini (Bölüm 10). Üçünün cevabı aynıdır: işaretçi.

İşaretçiler C'nin gücünün ve tehlikesinin kaynağıdır. Belleğe doğrudan erişim sağlarlar; bu sayede hızlı ve esnek programlar yazılır, ama bir yanlışlık programı çökertebilir ya da sessizce bozabilir. Konunun zorluğu çoğu zaman kavramın kendisinden değil, belleği kafada canlandıramamaktan gelir. Bu yüzden her örnekte bir bellek diyagramı çizeceğiz. Siz de yeni bir işaretçi kodu okurken kâğıda kutuları ve okları çizin; satır satır izlemek en güvenilir yöntemdir.

12.1 Bellek ve Adres

Bilgisayarın belleği (RAM), sıra numaralı bayt hücrelerinden oluşan çok uzun bir sıradır. Bu numaraya **adres** (address) denir. Bölüm 2'de değişkeni "adı olan kutu" diye anlatmıştık; kutunun bir de numarası vardır. `int yas = 25;` yazdığımızda derleyici 4 baytlık yer ayırır, ona `yas` adını verir ve 25'i oraya yazar. Değişkenin adresi, kapladığı baytların ilkinin numarasıdır.

```
Bellek (adresler örnektir; gerçek adresler çok daha büyük sayılardır)

  adres      değer     ad
+--------+----------+--------+
|  1000  |    25    |  yas   |   int: 1000-1003 arası 4 bayt
+--------+----------+--------+
|  1004  |   'A'    |  harf  |   char: 1 bayt
+--------+----------+--------+
```

Bir değişkenin adresini **adres operatörü** (address-of operator) `&` verir. `scanf("%d", &yas)` yazarken kullandığımız işaret budur. Adresler `printf` ile `%p` belirteciyle, onaltılık (hexadecimal) sayı olarak yazdırılır. `%p` için argümanı `(void *)` türüne çevirmek gerekir; `void *`'ı bölümün sonunda göreceğiz.

▸ adresGoster.c

```c
#include <stdio.h>

int main(void) {
    int yas = 25;
    char harf = 'A';

    printf("yas  : deger = %d, adres = %p\n", yas, (void *)&yas);
    printf("harf : deger = %c, adres = %p\n", harf, (void *)&harf);
    return 0;
}
```

ÇIKTI

```
yas  : deger = 25, adres = 0x7ffd3c1a5b4c
harf : deger = A, adres = 0x7ffd3c1a5b4b
```

Adresler her bilgisayarda, hatta aynı programın farklı çalıştırmalarında bile değişebilir; yerleştirmeyi işletim sistemi yapar. Önemli olan sayının kendisi değil, her değişkenin bir adresi olduğudur. Bu yüzden bu kitapta adres yazdıran programların çıktısı yalnızca örnektir.

12.2 İşaretçi Tanımlama ve Dolaylama

> **Tanım: İşaretçi**
>
> Değeri başka bir nesnenin bellek adresi olan değişkendir. Tanımlarken tipin yanına `*` konur: `int *p;` ifadesi "`p`, bir `int`'in adresini tutabilir" demektir.

İşaretçi üç adımda kullanılır:

Adım	Kod	Anlamı
1. Tanımla	`int *p;`	`p`, bir `int`'in adresini tutabilir
2. Adres ver	`p = &x;`	`p`'nin içine `x`'in adresi yazılır
3. Kullan	`*p`	`p`'nin gösterdiği yerdeki değer

`*p` işlemine **dolaylama** (indirection, dereference) denir: "`p`'nin içindeki adrese git ve orada duran değişkenin kendisini kullan." `*p` hem okunabilir hem yazılabilir; `x` ile aynı kutudur.

▸ ilkIsaretci.c

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    int *p = &x;            // p, x'in adresini tutar

    printf("x = %d\n", x);
    printf("*p = %d\n", *p);    // p'nin gösterdiği yerdeki değer
    printf("p == &x : %d\n", p == &x);
```

```
    *p = 20;                // x'in kendisi değişir
    printf("x = %d\n", x);
    *p = *p + 5;
    printf("x = %d\n", x);
    return 0;
}
```

```
ÇIKTI
x = 10
*p = 10
p == &x : 1
x = 20
x = 25
```

İşaretçi de bir değişkendir; kendi kutusu vardır ve o kutunun içinde `x`'in adresi yazılıdır:

```
  int x = 10;            int *p = &x;

  x (adres 1000)         p (adres 1008)
+-----------------+   +-----------------+
|       10        |<--|      1000       |
+-----------------+   +-----------------+
                       p'nin içindeki sayı = x'in adresi
```

Satır satır izleyelim. `*p = 20;` ile `x` 20 olur, `p` değişmez. `*p = *p + 5;` satırında sağdaki `*p` okur (20), soldaki `*p` yazar; `x` 25 olur.

C'de `*` işaretinin üç ayrı görevi vardır; hangisi olduğunu bulunduğu yer belirler:

Nerede?	Anlamı	Örnek
Tanımda, tipin yanında	"bu bir işaretçi"	`int *p;`
İfadede, tek başına ve önde	dolaylama: "gösterilen yer"	`*p = 3;` , `y = *p;`
İki değerin arasında	çarpma	`a * b`

> **▲ Sık Yapılan Hata · p ile *p'yi karıştırmak**
>
> `p` işaretçinin kendisidir (adres), `*p` ise gösterdiği değerdir. `x`'i değiştirmek için `*p = 20;` yazılır. `p = 20;` ise işaretçiye adres yerine bir sayı atamaya çalışır ve derleyici uyarı verir (`-Wall` ile; gcc'nin yeni sürümlerinde derleme hatasıdır). Benzer biçimde `double d; int *p = &d;` tip uyuşmazlığıdır.

int *p, q; tuzağı

`*` tipe değil, o değişkene aittir. Aynı satırda iki değişken tanımlarsak ikincisi işaretçi olmaz:

▸ ikiTanim.c

```
#include <stdio.h>

int main(void) {
    int x = 7;
    int *p, q;              // p işaretçi, q sıradan int!

    p = &x;
    q = x;
    printf("sizeof(p) = %zu, sizeof(q) = %zu\n", sizeof(p), sizeof(q));
```

```
    printf("*p = %d, q = %d\n", *p, q);
    return 0;
}
```

```
ÇIKTI
sizeof(p) = 8, sizeof(q) = 4
*p = 7, q = 7
```

`q` işaretçi değil, sıradan bir `int`'tir (`sizeof` farkına bakalım). `int* p, q;` diye yazsak da değişmez; boşluğun yeri anlamı etkilemez. İki işaretçi için `int *p, *q;` yazılır. En temizi, her işaretçiyi ayrı satırda tanımlamaktır.

İşaretçinin tipi

İşaretçinin içinde yalnızca bir adres vardır. O hâlde neden `int *`, `double *`, `char *` diye ayrı tipler var? Çünkü adres bir yerin başlangıcını söyler ama orada ne olduğunu söylemez. İşaretçinin tipi iki şeyi belirler:

1. `*p` yazıldığında kaç baytın okunacağı ve bu baytların nasıl yorumlanacağı.
2. `p + 1` yazıldığında adresin kaç bayt ilerleyeceği.

Bugünkü masaüstü bilgisayarlarda bütün işaretçiler aynı boyuttadır, çünkü hepsi bir adres tutar: 64 bitlik sistemlerde 8 bayt, eski 32 bitlik sistemlerde 4 bayt. (C standardı bunu her sistem için garanti etmez, ama pratikte böyledir.) Gösterdiği değerin boyutu ise tipe göre değişir: `sizeof(*p)` bir `int` için 4'tür.

12.3 NULL ve Başlatılmamış İşaretçi

Gösterecek bir şeyi henüz olmayan işaretçiye, hiçbir yeri göstermeyen özel değeri veririz: `NULL` (boş işaretçi, null pointer). `<stdio.h>`, `<stdlib.h>` ve `<string.h>` içinde tanımlıdır. Bir işaretçiyi kullanmadan önce `if (p == NULL)` ile denetlemek standart güvenlik adımıdır.

`NULL` işaretçinin gösterdiği yere erişmek tanımsız davranıştır; masaüstü sistemlerde program neredeyse her zaman çöker. Aşağıdaki program bunu kasıtlı olarak gösterir.

▸ nullCokme.c

```c
#include <stdio.h>

int main(void) {
    int *p = NULL;          // henüz hiçbir yeri göstermiyor

    if (p == NULL) {
        printf("p henuz bir yeri gostermiyor.\n");
    }
    printf("Yazmadan once\n");
    fflush(stdout);
    *p = 5;                 // kontrol yok: NULL'ın gösterdiği yere yazma
    printf("Yazdiktan sonra\n");
    return 0;
}
```

```
ÇIKTI
p henuz bir yeri gostermiyor.
Yazmadan once
Segmentation fault
```

`Segmentation fault` yazısını Linux ve macOS terminali yazar; Windows'ta program çoğu zaman mesajsız kapanır. Son `printf` hiç çalışmaz. Bu çökme aslında iyi haberdir, çünkü hata hemen ortaya çıkar. Daha kötüsü başlatılmamış işaretçidir:

```
int *p;                  // ilk değer verilmedi: içinde rastgele bir adres var
*p = 5;                  // TANIMSIZ DAVRANIŞ: bilinmeyen bir yere yazar
```

Bölüm 2'deki çöp değerin işaretçideki karşılığıdır: `p`'nin içinde, o bellekte kalmış rastgele bir sayı vardır ve bu sayı adres sayılır. Program çökebilir, başka bir değişkeni sessizce bozabilir ya da hiçbir şey olmamış gibi çalışabilir. Üstelik içinde `NULL` değil çöp olduğu için `p != NULL` denetimi de bunu yakalayamaz.

İşaretçinin durumu	İçinde ne var?	`*p` kullanılırsa
Geçerli bir değişkeni gösteriyor	O değişkenin adresi	Sorun yok
`NULL`	Hiçbir yeri göstermeyen özel değer	Çökme; `if (p != NULL)` ile önlenir
Başlatılmamış	Rastgele bir sayı	Tanımsız davranış; denetlenemez

◆ Kritik Nokta · Her işaretçiye tanımlarken değer verin

İşaretçiyi tanımlarken ya geçerli bir adres ya da `NULL` verin (`int *p = NULL;`). Kullanmadan önce de `NULL` olup olmadığını denetleyin.

Tarihçe · NULL ve nullptr

`0` sabiti işaretçiye atanınca "boş işaretçi" anlamına gelir; yani `p = 0;` geçerlidir, ama okuyan kişi sayı mı işaretçi mi olduğunu anlayamaz. Bu yüzden her zaman `NULL` yazılır. C++11 boş işaretçi için ayrı bir anahtar sözcük getirdi: `nullptr`. C23 de aynı sözcüğü C'ye ekledi; ancak bu kitapta C17 kullandığımız için `NULL` ile devam edeceğiz.

12.4 Adresle Geçirme

Bölüm 9'dan hatırlayalım: C'de argümanlar değerle geçirilir. Fonksiyondaki parametre, çağıran taraftaki değişkenin kopyasıdır; bu yüzden `takas(x, y)` `main`'deki değişkenlere dokunamıyordu. Çözüm, değişkenin kendisini değil **adresini** geçirmektir. Fonksiyon adresin bir kopyasını alır, ama kopya da aynı yeri gösterdiği için `*a = ...` yazınca çağıranın değişkeni değişir.

▸ swapDuzgun.c

```
#include <stdio.h>

void takas(int *a, int *b) {
    int gecici = *a;
    *a = *b;
    *b = gecici;
}

int main(void) {
    int x = 3;
    int y = 8;
    printf("Once  : x = %d, y = %d\n", x, y);
    takas(&x, &y);          // değişkenlerin adreslerini ver
```

```
    printf("Sonra : x = %d, y = %d\n", x, y);
    return 0;
}
```

```
ÇIKTI
Once  : x = 3, y = 8
Sonra : x = 8, y = 3
```

`takas` çalışırken bellek şöyledir. `a` ve `b`, `takas`'ın yerel değişkenleridir (içlerinde adres vardır); `gecici` sıradan bir `int`'tir:

```
        main                          takas
 +-------+--------+           +--------+--------+
 |   x   |   3    |<----------|   a    |   &x   |
 +-------+--------+           +--------+--------+
 |   y   |   8    |<----------|   b    |   &y   |
 +-------+--------+           +--------+--------+
                              | gecici |   ?    |
                              +--------+--------+
```

Adım adım: çağrıda `gecici` çöptür; `gecici = *a;` ile 3 olur; `*a = *b;` ile `x` 8 olur; `*b = gecici;` ile `y` 3 olur. Dönüşte `a`, `b` ve `gecici` yok olur (Bölüm 9'daki çağrı yığını), ama `x` ile `y` `main`'e aittir ve değişmiş hâlde kalır.

> **◆ Kritik Nokta · C'de "adresle geçirme" ayrı bir mekanizma değildir**
>
> C'de her şey değerle geçirilir. Adresle geçirme dediğimiz şey, geçirilen değerin bir adres olmasıdır. Fonksiyonun çağıranın değişkenini değiştirebilmesi bu yüzdendir.

> **▲ Sık Yapılan Hata · Çağırırken & unutmak**
>
> `void takas(int *a, int *b)` fonksiyonunu `takas(x, y);` diye çağırırsak `int` değerini işaretçi parametreye vermiş oluruz. Derleyici uyarı verir (yeni gcc sürümlerinde hata); uyarı gözden kaçarsa program 3 sayısını adres sanıp çöker. Parametre `T *` ise çağırırken `&değişken` yazılır.

Birden çok sonuç döndürmek

`return` tek değer döndürür. Birden çok sonuç gerekiyorsa sonuçların yazılacağı yerlerin adreslerini parametre olarak veririz:

▸ bolVeKalan.c

```c
#include <stdio.h>

void bolVeKalan(int a, int b, int *bolum, int *kalan) {
    *bolum = a / b;
    *kalan = a % b;
}

int main(void) {
    int bolum = 0;
    int kalan = 0;
    bolVeKalan(17, 5, &bolum, &kalan);
```

```
    printf("17 / 5 = %d, kalan %d\n", bolum, kalan);
    return 0;
}
```

```
ÇIKTI
17 / 5 = 3, kalan 2
```

Girdi alan parametrelerle sonuç yazılan parametreler bu biçimde birlikte kullanılır: `a` ve `b` değerle, `bolum` ve `kalan` adresle geçer. Bölüm 14'te bunun bir alternatifini, yapıları (`struct`) göreceğiz.

scanf'teki & işareti

Artık `scanf("%d", &yas)` içindeki `&`'ın nedeni açık: `scanf` de bir fonksiyondur ve `yas` değişkenine değer yazması gerekir. Bunun için `yas`'ın değerini değil adresini almalıdır; tıpkı `takas`'taki `*a = ...` gibi. Elimizde zaten bir işaretçi varsa `&` koymayız:

```
int sayi = 0;
int *p = &sayi;
scanf("%d", p);            // scanf("%d", &sayi); ile aynı iş
```

> **▲ Sık Yapılan Hata · scanf'e işaretçinin adresini vermek**
>
> `p` bir işaretçiyken `scanf("%d", &p);` yazarsak `p`'nin kendi adresini (`int **`) vermiş oluruz; sayı `p`'nin kutusuna yazılır ve içindeki adres bozulur. `-Wall` bunu "format '%d' expects argument of type 'int *'" uyarısıyla bildirir. `scanf("%s", s)` yazarken `&` koymamamızın nedeni de aynıdır: dizi adı zaten adrestir (aşağıda göreceğiz).

12.5 İşaretçi Aritmetiği

İşaretçilerle toplama ve çıkarma yapılabilir, ama kural şudur: `p + 1` adresi 1 bayt değil, **1 eleman** ilerletir. Eleman boyutu işaretçinin tipinden gelir (`char *` için 1, `int *` için 4, `double *` için 8 bayt; yani `sizeof(*p)`). İşaretçi tipinin önemi burada ortaya çıkar.

```
  a[0]   a[1]   a[2]   a[3]   a[4]
+------+------+------+------+------+
|  10  |  20  |  30  |  40  |  50  |
+------+------+------+------+------+
 1000   1004   1008   1012   1016      (örnek adresler)
  ^
  p        p + 1 = 1004 (1001 değil!),   p + 3 = 1012
```

İşlem	Anlamı	Sonuç
`p + n`, `p - n`	`n` eleman ileri/geri	işaretçi
`p++`, `p--`	bir eleman ileri/geri (`p`'yi değiştirir)	işaretçi
`p - q`	aralarındaki eleman sayısı (aynı dizide)	tamsayı (`ptrdiff_t`)
`p < q`, `p == q`	adres karşılaştırması	0 ya da 1
`p + q`, `p * 2`, `p / 2`	anlamsız	derlenmez

aritmetik.c

```c
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = &a[0];

    printf("%d\n", *p);         // 10
    p++;                        // a[1]'e geç
    printf("%d\n", *p);         // 20
    p = p + 2;                  // a[3]'e geç
    printf("%d\n", *p);         // 40
    p--;                        // a[2]'ye dön
    printf("%d\n", *p);         // 30
    printf("%d\n", *(p + 1));   // 40, p değişmez

    int *q = &a[4];
    printf("q - p = %d\n", (int)(q - p));    // 4 - 2
    return 0;
}
```

ÇIKTI

```
10
20
40
30
40
q - p = 2
```

İki işaretçiyi çıkarmak bayt farkını değil eleman sayısını verir. Çıkarma ve karşılaştırma yalnızca aynı dizinin elemanlarını gösteren işaretçiler için anlamlıdır. Dizinin bir ötesini gösteren işaretçiyi (`a + 5`) oluşturmak ve karşılaştırmak serbesttir (döngülerde kullanılır), ama onu `*` ile okumak ya da dizinin daha ötesine gitmek tanımsızdır.

*p++ ile (*p)++ farkı

Sonek `++` operatörü `*` operatöründen daha yüksek önceliklidir (Bölüm 2). Bu yüzden `*p++`, "`p`'nin gösterdiğini kullan, sonra `p`'yi ilerlet" demektir; gösterilen değeri artırmaz.

İfade	Nasıl okunur	Ne yapar?
`*p++`	`*(p++)`	Önce `*p`'yi kullan, sonra `p`'yi bir eleman ilerlet
`(*p)++`		`p` sabit kalır, gösterilen değer 1 artar (kullanılan, artmadan önceki değerdir)
`*++p`	`*(++p)`	Önce `p`'yi ilerlet, sonra yeni yerdeki değeri kullan

artirmaOncelik.c

```c
#include <stdio.h>

int main(void) {
    int a[4] = {10, 20, 30, 40};
    int *p = a;

    int ilk = *p++;         // ilk = 10, sonra p -> a[1]
    printf("ilk = %d, *p = %d\n", ilk, *p);
    (*p)++;                 // a[1]: 20 -> 21, p yerinde
    printf("*p = %d, a[1] = %d\n", *p, a[1]);
```

```
    int ucuncu = *++p;      // önce p -> a[2], sonra oku
    printf("ucuncu = %d\n", ucuncu);
    printf("%d %d %d %d\n", a[0], a[1], a[2], a[3]);
    return 0;
}
```

```
ÇIKTI
ilk = 10, *p = 20
*p = 21, a[1] = 21
ucuncu = 30
10 21 30 40
```

> **İpucu · İşaretçi ifadelerini izlemek**
>
> İki şeyi ayrı ayrı yazalım: "`p` nereyi gösteriyor?" ve "dizide ne var?". `*p++`'da `p` değişir, dizi değişmez. `(*p)++`'da dizi değişir, `p` değişmez. Parantez yoksa `++` önce `p`'ye yapışır.

12.6 Diziler ve İşaretçiler

Bölüm 10'daki sorunun cevabı şudur: dizi adı, ifadelerde çoğu zaman ilk elemanın adresine dönüşür. `int a[5] = {10, 20, 30, 40, 50};` için:

İfade	Eşdeğeri	Anlamı
`a`	`&a[0]`	ilk elemanın adresi
`a + i`	`&a[i]`	`i`. elemanın adresi
`*(a + i)`	`a[i]`	`i`. eleman

`a[i]` yazımı derleyici için tam olarak `*(a + i)` demektir; indeks yazımı bu işlemin kısa yoludur. İşaretçilere de indeks uygulanabilir: `p[i]` ile `*(p + i)` aynıdır.

▸ diziIsaretci.c

```
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;             // &a[0] ile aynı

    printf("a[2]     = %d\n", a[2]);
    printf("*(a + 2) = %d\n", *(a + 2));
    printf("p[2]     = %d\n", p[2]);
    printf("*(p + 2) = %d\n", *(p + 2));
    return 0;
}
```

```
ÇIKTI
a[2]     = 30
*(a + 2) = 30
p[2]     = 30
*(p + 2) = 30
```

Diziyi dolaşmanın bir yolu "gezen işaretçi" kalıbıdır:

```
for (int *p = a; p < a + n; p++) {     // a: dizi, n: eleman sayısı
    toplam += *p;                      // a[i] yerine *p
}
```

`a + n` dizinin bir ötesini gösterir; `p < a + n` bitiş koşuludur ve `*p` o adrese hiç erişmez. Bu kalıbı C kodlarında sık göreceğiz.

Dizi adı ile işaretçi aynı şey değildir

Aralarındaki ilişki dönüşümdür, eşitlik değil. Dizi adı bir işaretçi değişkeni olmadığı için `sizeof(a)` bütün dizinin boyutunu verir (`sizeof(p)` ise işaretçinin boyutunu), ve `a`'ya atama yapılamaz; dizinin yeri sabittir. İşaretçi değişken olduğundan `p`'ye atama yapılabilir.

▸ diziAdiAtama.c

```
#include <stdio.h>

int main(void) {
    int a[3] = {1, 2, 3};
    int b[3] = {4, 5, 6};
    int *p = b;             // olur: p değişken
    a = p;                  // hata: dizi adına atama
    a++;                    // hata: dizi adı ilerletilemez
    printf("%d %d\n", a[0], p[0]);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
diziAdiAtama.c: In function 'main':
diziAdiAtama.c:8:7: error: assignment to expression with array type
    8 |     a = p;                 // hata: dizi adına atama
      |       ^
diziAdiAtama.c:9:6: error: lvalue required as increment operand
    9 |     a++;                   // hata: dizi adı ilerletilemez
      |      ^~
```

Fonksiyona dizi geçirmek

Dizi adı adrese dönüştüğü için fonksiyon dizinin kopyasını değil, ilk elemanın adresini alır. Parametrede `int a[]` yazmak `int *a` yazmakla aynıdır. Fonksiyon, ana programdaki dizinin kendisini kullanır:

▸ diziGecir.c

```
#include <stdio.h>

void ikiyeKatla(int *a, int n) {      // int a[] ile aynı
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * 2;
    }
}

int main(void) {
    int sayilar[4] = {1, 2, 3, 4};
    ikiyeKatla(sayilar, 4);
    printf("%d %d %d %d\n", sayilar[0], sayilar[1], sayilar[2], sayilar[3]);
    return 0;
}
```

```
ÇIKTI
2 4 6 8
```

Fonksiyondaki değişiklik `main`'deki diziyi değiştirdi, çünkü aynı belleği gösteriyordu. Fonksiyon yalnızca adresi bildiği için dizinin kaç elemanlı olduğunu bilemez; uzunluğu (`n`) ayrıca geçirmek zorundayız. Bölüm 10'da uyardığımız `sizeof` tuzağı buradan doğar:

▸ boyutFonk.c

```c
#include <stdio.h>

void boyutYaz(int a[]) {
    printf("fonksiyonda: sizeof(a)    = %zu\n", sizeof(a));
}

int main(void) {
    int dizi[5] = {1, 2, 3, 4, 5};
    printf("main'de    : sizeof(dizi) = %zu\n", sizeof(dizi));
    boyutYaz(dizi);
    return 0;
}
```

```
ÇIKTI
main'de    : sizeof(dizi) = 20
fonksiyonda: sizeof(a)    = 8
```

`gcc -Wall` burada "`sizeof` on array function parameter 'a' will return size of 'int *'" uyarısını verir. Fonksiyondaki `a` bir dizi değil `int *`'tır; `sizeof(a) / sizeof(a[0])` eleman sayısını vermez (8 / 4 = 2 çıkar). Eleman sayısını her zaman dizinin tanımlandığı yerde hesaplayıp fonksiyona ayrı parametre olarak veririz.

> ◆ **Kritik Nokta · Dizi kopyalanmaz, int kopyalanır**
>
> `f(dizi)` çağrısında fonksiyonun yaptığı `a[0] = 99;` değişikliği çağıranın dizisini etkiler; çünkü adres geçer. `f(dizi[0])` çağrısında ise elemanın kopyası geçer ve çağıran etkilenmez. Fonksiyon içinde diziyi değiştirmeyeceksek parametreyi `const int *a` yapmak (Bölüm 10'daki `const` dizi parametresi) bu sözü derleyiciye denetletir.

12.7 İşaretçiler ve Stringler

Bölüm 11'de string'in `'\0'` ile biten bir `char` dizisi olduğunu öğrendik. Dizi adı adrese dönüştüğü için string'ler çoğunlukla `char *` ile taşınır. Ama string tanımlamanın iki yolu aynı değildir:

▸ diziVsIsaretci.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char dizi[] = "Ali";        // 4 baytlık dizi: 'A' 'l' 'i' '\0'
    const char *p = "Ali";      // işaretçi + salt okunur "Ali"

    dizi[0] = 'E';              // olur: dizi bizim kopyamız
    printf("dizi = %s, p = %s\n", dizi, p);
    printf("sizeof(dizi) = %zu, sizeof(p) = %zu\n", sizeof(dizi), sizeof(p));
```

```
    printf("strlen(dizi) = %zu, strlen(p) = %zu\n", strlen(dizi), strlen(p));
    p = "Veli";                 // olur: p başka yeri gösterebilir
    printf("p = %s\n", p);
    return 0;
}
```

```
ÇIKTI
dizi = Eli, p = Ali
sizeof(dizi) = 4, sizeof(p) = 8
strlen(dizi) = 3, strlen(p) = 3
p = Veli
```

```
char dizi[] = "Ali";                const char *p = "Ali";

dizi (4 bayt)                       p (8 bayt)          salt okunur bölge
+-----+-----+-----+------+          +--------+          +-----+-----+-----+------+
| 'A' | 'l' | 'i' | '\0' |          | adres  |--------> | 'A' | 'l' | 'i' | '\0' |
+-----+-----+-----+------+          +--------+          +-----+-----+-----+------+
bizim kopyamız, değiştirilebilir                         programa gömülü, değişmez
```

	`char dizi[] = "Ali";`	`const char *p = "Ali";`
Bellekte	4 baytlık dizi (metnin kopyası)	8 baytlık işaretçi + salt okunur metin
`sizeof`	4	8
Harfi değiştirme	Olur	Yapılamaz (`const` ile derleme hatası)
Başka metne yöneltme	`dizi = "Veli";` derlenmez	`p = "Veli";` olur

Çift tırnak içindeki `"Ali"` bir **string sabitidir** (string literal) ve programın salt okunur bir bölgesinde durur. `char *p = "Ali"; p[0] = 'E';` ile değiştirmeye çalışmak tanımsız davranıştır (Linux ve macOS'ta çoğu zaman `Segmentation fault`, ama garanti yok). Bu yüzden string sabitini gösteren işaretçiyi `const char *` olarak tanımlarız: yanlışlıkla değiştirmeye kalkarsak hata derleyicide yakalanır. Değiştireceğimiz metin için `char dizi[] = "..."` kullanırız. Fonksiyona değiştirilmeyecek bir string veriyorsak parametreyi `const char *` yaparız (`strlen`, `puts` ve `printf` böyle tanımlanmıştır).

İşaretçilerle `strlen`'in kendi sürümünü yazalım:

▸ ozelStrlen.c

```
#include <stdio.h>

size_t uzunluk(const char *s) {
    const char *basla = s;
    while (*s != '\0') {
        s++;
    }
    return (size_t)(s - basla);     // iki işaretçinin farkı = harf sayısı
}

int main(void) {
    printf("%zu\n", uzunluk("Merhaba"));
    printf("%zu\n", uzunluk(""));
    return 0;
}
```

```
ÇIKTI
7
0
```

12.8 const ve İşaretçiler

`const`, işaretçide iki farklı şeyi sabitleyebilir: gösterilen değeri ya da işaretçinin kendisini. Ayrımı `const`'ın `*`'a göre yeri belirler:

Tanım	Gösterilen değer (`*p = ...`)	İşaretçi (`p = ...`)	Okunuşu
`int *p`	değişir	değişir	`int`'e işaretçi
`const int *p`	değişmez	değişir	sabit `int`'e işaretçi
`int *const p`	değişir	değişmez	`int`'e sabit işaretçi
`const int *const p`	değişmez	değişmez	sabit `int`'e sabit işaretçi

Ezber kuralı: `const` `*`'ın **solundaysa** gösterilen veri sabittir; `*`'ın **sağındaysa** işaretçi sabittir. Karışırsak tanımı sağdan sola okuruz: `int *const p` → "`p`, sabit bir işaretçidir ve `int`'e işaret eder".

▸ constHata.c

```c
#include <stdio.h>

int main(void) {
    int a = 1;
    int b = 2;
    const int *p1 = &a;         // gösterilen değer sabit
    int *const p2 = &a;         // işaretçi sabit
    const char *s = "Ali";

    p1 = &b;                    // olur: işaretçi değişebilir
    *p2 = 5;                    // olur: gösterilen değişebilir
    *p1 = 9;                    // HATA
    p2 = &b;                    // HATA
    s[0] = 'E';                 // HATA
    printf("%d %d %s\n", *p1, *p2, s);
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI
constHata.c: In function 'main':
constHata.c:13:9: error: assignment of read-only location '*p1'
   13 |     *p1 = 9;                    // HATA
      |         ^
constHata.c:14:8: error: assignment of read-only variable 'p2'
   14 |     p2 = &b;                    // HATA
      |        ^
constHata.c:15:10: error: assignment of read-only location '*s'
   15 |     s[0] = 'E';                 // HATA
      |          ^
```

Üç mesaj da "salt okunur" (read-only) diyor. En çok kullanacağımız biçim `const T *`'dır: fonksiyona "bu veriyi yalnızca okuyacağım" sözü vermek için (`uzunluk(const char *s)`). Derleyici bu sözü bizim yerimize denetler.

Not · restrict ve volatile

`const` gibi iki tip niteleyici (type qualifier) daha vardır. Kendi programlarımızda nadiren yazarız, ama kütüphane prototiplerinde ve gömülü sistem kodunda görürüz.

`restrict` (C99) yalnızca işaretçilere yazılır: "Bu işaretçinin gösterdiği belleğe, fonksiyon boyunca başka işaretçiyle erişilmeyecek" sözüdür; derleyici bu sözle daha hızlı kod üretebilir. Sözü tutmak bize düşer: iki `restrict` işaretçi aynı belleği gösterirse davranış tanımsızdır. `memcpy`'nin prototipi tipik örnektir (örtüşen bellekler için `memmove` kullanılır):

```
void *memcpy(void *restrict hedef, const void *restrict kaynak, size_t n);
```

`volatile`, "bu değişkenin değeri program dışından değişebilir" der (donanım yazmacı, sinyal işleyicisi). Derleyici onun okumalarını ve yazmalarını gereksiz sayıp silemez. Masaüstü programlarında nadirdir, mikrodenetleyici kodunda sık görülür.

12.9 Yerel Değişkenin Adresini Döndürmek

Yerel değişken fonksiyon bitince yok olur (Bölüm 9'daki çağrı yığını). Adresini döndürürsek çağıranın elinde artık var olmayan bir yeri gösteren işaretçi kalır. Buna **sarkan işaretçi** (dangling pointer) denir.

```
int *yanlis(void) {
    int sayi = 42;
    return &sayi;          // HATA: sayi, fonksiyon bitince yok olur
}
```

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra yerelAdres.c -o yerelAdres
yerelAdres.c: In function 'yanlis':
yerelAdres.c:3:12: warning: function returns address of local variable [-Wreturn-local-addr]
    3 |     return &sayi;
      |            ^~~~~
```

Bu fonksiyonun döndürdüğü işaretçiyle `*p` okumak tanımsız davranıştır: o bellek sonraki çağrılarda başka işler için kullanılır; `*p` eski değeri de verebilir, çöp de verebilir, program da çökebilir. Bu yüzden böyle bir programı çalıştırıp çıktısını göstermiyoruz. `-Wreturn-local-addr` uyarısı hatayı yakalar; uyarı da olsa asla yok sayılmaz. Aynı hata yerel dizide de olur: `char sonuc[50]; ...; return sonuc;` yazan bir fonksiyon çağırana sarkan işaretçi verir.

Üç çözüm vardır:

1. Sonucu çağıranın verdiği yere yazmak (`bolVeKalan` gibi, ya da `char *sonuc` parametresiyle).
2. `static` yerel değişkenin adresini döndürmek; `static` değişken program boyunca yaşar (ama her çağrı aynı yeri paylaşır).
3. Kalıcı yer ayırmak: `malloc`, Bölüm 13'te.

12.10 İşaretçinin İşaretçisi

İşaretçi de bir değişken olduğuna göre onun da adresi vardır. O adresi tutan değişkene **işaretçinin işaretçisi** denir: `int **pp`. Her `*` bir basamak gösterir.

ciftIsaretci.c

```c
#include <stdio.h>

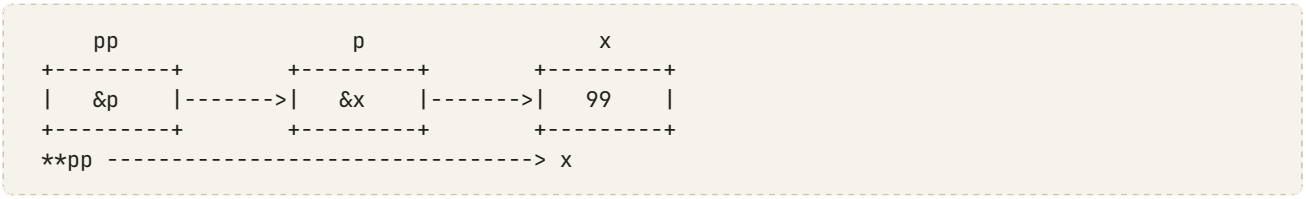
void yonelt(int **hedef, int *yeni) {
    *hedef = yeni;          // main'deki işaretçiyi değiştir
}

int main(void) {
    int x = 5;
    int y = 8;
    int *p = &x;            // p, x'i gösterir
    int **pp = &p;          // pp, p'yi gösterir

    printf("x = %d, *p = %d, **pp = %d\n", x, *p, **pp);
    **pp = 99;              // pp -> p -> x
    printf("x = %d\n", x);
    yonelt(&p, &y);         // p artık y'yi gösterir
    printf("*p = %d\n", *p);
    return 0;
}
```

ÇIKTI

```
x = 5, *p = 5, **pp = 5
x = 99
*p = 8
```



Asıl kullanım yeri, fonksiyonun bir işaretçiyi değiştirmesi gereken durumdur. `takas`'ta bir `int`'i değiştirmek için `int *` geçirmiştik; bir `int *`'ı değiştirmek için de `int **` geçirmek gerekir (`yonelt`). Kural aynıdır, bir seviye yukarıdadır. Bölüm 19'da bağlı listenin başını değiştiren fonksiyonlarda bunu kullanacağız.

12.11 İşaretçi Dizisi ve Diziye İşaretçi

İki tanım yalnızca bir çift parantezle ayrılır, ama anlamları çok farklıdır:

Tanım	Okunuşu	`sizeof` (64 bit)
`int *p[3]`	`p` bir **dizidir**: 3 elemanlı, elemanları `int *` (işaretçi dizisi)	24
`int (*q)[3]`	`q` bir **işaretçidir**: 3 elemanlı bir `int` dizisini gösterir (diziye işaretçi)	8

`[ ]` operatörü `*`'dan önceliklidir. `int *p[3]` içinde `p` önce `[3]` ile birleşir: dizidir, elemanları `int *`'tır. `int (*q)[3]` içinde parantez `*`'ı öne alır: `q` önce işaretçidir, gösterdiği tip `int[3]`'tür. Emin olamazsak tanımı adından başlayarak okuruz.

isaretciDizisi.c

```c
#include <stdio.h>

int main(void) {
    int a = 10, b = 20, c = 30;
```

```
    int *p[3] = {&a, &b, &c};       // 3 tane int işaretçisi
    int m[3] = {1, 2, 3};
    int (*q)[3] = &m;               // 3 elemanlı diziyi gösteren tek işaretçi

    printf("sizeof(p) = %zu, sizeof(q) = %zu\n", sizeof(p), sizeof(q));
    printf("sizeof(*p) = %zu, sizeof(*q) = %zu\n", sizeof(*p), sizeof(*q));
    printf("*p[1] = %d, (*q)[1] = %d\n", *p[1], (*q)[1]);
    return 0;
}
```

```
ÇIKTI
sizeof(p) = 24, sizeof(q) = 8
sizeof(*p) = 8, sizeof(*q) = 12
*p[1] = 20, (*q)[1] = 2
```

`p`'nin her elemanı ayrı bir `int`'i gösterir (`*p[1]` → `b`). `q` ise tek bir adres tutar, ama gösterdiği şey bütün dizidir: `sizeof(*q)` 12 çıkar. İşaretçi dizisi, uzunlukları farklı string listeleri için doğal yoldur (Bölüm 11'deki string dizisinin gerçek yüzü budur; bir sonraki başlıktaki `argv` de böyledir). Diziye işaretçi ise iki boyutlu diziyi satır satır gezmeye yarar:

▸ diziyeIsaretci.c

```
#include <stdio.h>

int satirToplami(int (*m)[3], int satir) {     // int m[][3] ile aynıdır
    return m[satir][0] + m[satir][1] + m[satir][2];
}

int main(void) {
    int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int (*satir)[3] = m;            // m[0] satırının tamamını gösterir

    printf("%d %d\n", satir[0][1], satir[1][2]);
    satir++;                        // 3 int (12 bayt) ileri: sonraki satır
    printf("%d\n", (*satir)[0]);
    printf("%d\n", satirToplami(m, 1));
    return 0;
}
```

```
ÇIKTI
2 6
4
15
```

`satir++` işaretçiyi 3 `int` (12 bayt) ilerletti, çünkü işaretçi aritmetiği gösterilen tipin boyutuyla çalışır. Bölüm 10'da iki boyutlu diziyi fonksiyona `int m[][SUTUN]` diye geçirirken sütun sayısının zorunlu olduğunu görmüştük: derleyici o parametreyi `int (*m)[SUTUN]` diye okur ve satır başına kaç bayt atlayacağını sütun sayısından bulur.

12.12 Komut Satırı Argümanları

Programı çalıştırırken adının yanına yazdığımız sözcükler programa **komut satırı argümanı** (command-line argument) olarak ulaşır. Bunun için `main`'in ikinci biçimi kullanılır: `int main(int argc, char *argv[])`.

Parametre	Anlamı
`argc`	Argüman sayısı (argument count). Program adı da sayıldığı için en az 1'dir
`argv`	Argüman dizisi (argument vector): her elemanı bir string'i gösteren `char *`
`argv[0]`	Programın adı
`argv[1]`, `argv[2]`, ...	Yazdıklarımız; hepsi string'tir (`"5"` bir sayı değil, metindir)
`argv[argc]`	Her zaman `NULL`

`char *argv[]` parametrede `char **argv` ile aynıdır (dizi parametresi işaretçiye dönüşür). Metni sayıya çevirmek için `<stdlib.h>`'deki `atoi` kullanılır (Bölüm 11).

▸ argumanlar.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int toplam = 0;
    printf("Arguman sayisi: %d\n", argc);
    for (int i = 1; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
        toplam += atoi(argv[i]);
    }
    printf("Toplam: %d\n", toplam);
    return 0;
}
```

ÇIKTI

```
Arguman sayisi: 1
Toplam: 0
```

Bu çıktı programı argümansız çalıştırdığımızda elde edilir: `argc` 1'dir ve döngü hiç dönmez. Argümanlı çalıştırmak için terminalde program adının yanına boşlukla ayırarak yazarız:

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra argumanlar.c -o argumanlar
$ ./argumanlar 12 30 elma
Arguman sayisi: 4
argv[1] = 12
argv[2] = 30
argv[3] = elma
Toplam: 42
```

`atoi("elma")` sayıya çevrilemediği için 0 döndürür: 12 + 30 + 0 = 42. Bellekte `argv`, elemanları `char *` olan (her biri bir string'in ilk harfini gösteren) bir dizidir:

```
argv (argc = 4)
+--------+
| adres -|--> "./argumanlar"      argv[0]
| adres -|--> "12"                argv[1]
| adres -|--> "30"                argv[2]
| adres -|--> "elma"              argv[3]
| NULL   |                        argv[4] = argv[argc]
+--------+
```

> **İpucu · Argüman sayısını denetleyin**
> Program belli sayıda argüman bekliyorsa en başta `if (argc != 3) { printf("Kullanim: ...\n"); return 1; }` ile denetleyelim. Kod düzenleyicilerdeki "çalıştır" düğmeleri genellikle argüman vermez; argümanlı denemeyi terminalden yapmak daha kolaydır. Windows `cmd`'de `./argumanlar` yerine `.\argumanlar` yazılır.

12.13 void İşaretçi ve Fonksiyon İşaretçisi

İşaretçilerin son iki türünü kısaca görelim; ikisini de `qsort`'ta birlikte kullanacağız.

void *

`void *` tipi, türü olmayan genel bir adrestir. Herhangi bir işaretçi `void *`'a, `void *` da herhangi bir işaretçiye dönüştürme (cast) yazmadan atanabilir. Ama tipi olmadığı için ne `*` ile okunabilir (kaç bayt okunacağını bilmez) ne de standart C'de `v + 1` ile ilerletilebilir. Önce gerçek tipe çevirmek gerekir: `*(int *)v`. `malloc` (Bölüm 13) `void *` döndürür; `qsort` da elemanların tipini bilmediği için `void *` ile çalışır.

Fonksiyon işaretçisi

Fonksiyonların da bellekte adresi vardır; fonksiyon adı, dizi adı gibi o adrese dönüşür. **Fonksiyon işaretçisi** (function pointer) tanımı fonksiyonun dönüş tipini ve parametre tiplerini içerir:

▸ fonksiyonIsaretcisi.c

```c
#include <stdio.h>

int topla(int a, int b) {
    return a + b;
}

int carp(int a, int b) {
    return a * b;
}

int uygula(int (*islem)(int, int), int x, int y) {
    return islem(x, y);             // gelen fonksiyonu çağır
}

int main(void) {
    int (*f)(int, int);             // (int, int) alıp int döndüren fonksiyon işaretçisi
    f = topla;                      // fonksiyon adı = fonksiyonun adresi
    printf("%d\n", f(3, 4));        // topla(3, 4)
    printf("%d\n", uygula(topla, 5, 6));
    printf("%d\n", uygula(carp, 5, 6));
    return 0;
}
```

ÇIKTI

```
7
11
30
```

Parantezler çok önemlidir: `int (*f)(int, int)` bir fonksiyon işaretçisidir; `int *f(int, int)` ise "`int *` döndüren bir fonksiyon"dur. `uygula` gibi bir fonksiyon, davranışının bir kısmını çağıranın vermesine olanak tanır. Aynı fikir C++'ta `std::function` ve lambda'lara, C#'ta delegelere dönüşecektir.

qsort

`<stdlib.h>`'deki `qsort` her tipten diziyi sıralar. Tipi bilmediği için dört şeye ihtiyaç duyar:

```
void qsort(void *dizi, size_t adet, size_t boyut,
           int (*karsilastir)(const void *, const void *));
```

Sırayla: dizinin adresi, eleman sayısı, bir elemanın bayt boyutu (`sizeof(dizi[0])`) ve iki elemanı kıyaslayan bizim yazdığımız fonksiyonun adresi. Karşılaştırma fonksiyonu iki elemanın **adresini** alır; birinci eleman önce gelecekse negatif, eşitse 0, ikinci önce gelecekse pozitif döndürür.

▸ qsortOrnek.c

```c
#include <stdio.h>
#include <stdlib.h>

int kucuktenBuyuge(const void *x, const void *y) {
    int a = *(const int *)x;        // void * -> int *, sonra oku
    int b = *(const int *)y;
    if (a < b) {
        return -1;
    }
    if (a > b) {
        return 1;
    }
    return 0;
}

int main(void) {
    int sayilar[6] = {42, 7, 19, 3, 88, 7};
    size_t n = sizeof(sayilar) / sizeof(sayilar[0]);

    qsort(sayilar, n, sizeof(sayilar[0]), kucuktenBuyuge);
    for (size_t i = 0; i < n; i++) {
        printf("%d ", sayilar[i]);
    }
    printf("\n");
    return 0;
}
```

```
ÇIKTI
3 7 7 19 42 88
```

`qsort(..., kucuktenBuyuge)` yazarken fonksiyon adının yanına parantez koymadık: `kucuktenBuyuge(...)` fonksiyonu çağırır, `kucuktenBuyuge` ise adresini verir. Büyükten küçüğe sıralamak için karşılaştırma sonuçlarını tersine çevirmek yeter.

> **▲ Sık Yapılan Hata · Karşılaştırmada a - b yazmak**
>
> `return a - b;` kısa ve çekici görünür, ama çıkarma `int` sınırlarını aşabilir (örneğin `INT_MIN` ile 1) ve işaretli taşma tanımsız davranıştır. Karşılaştırma fonksiyonunda `<` ve `>` kullanalım.

`qsort` string dizisini de sıralar. Bu kez dizinin elemanları `char *`'tır; karşılaştırma fonksiyonuna verilen adresler `char *`'ın adresidir, yani `char **` türündedir:

▸ qsortString.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int alfabetik(const void *x, const void *y) {
    const char *a = *(const char *const *)x;    // elemanın kendisi: bir char *
    const char *b = *(const char *const *)y;
    return strcmp(a, b);
}

int main(void) {
    const char *meyveler[] = {"muz", "elma", "kiraz", "armut"};
    size_t n = sizeof(meyveler) / sizeof(meyveler[0]);

    qsort(meyveler, n, sizeof(meyveler[0]), alfabetik);
    for (size_t i = 0; i < n; i++) {
        printf("%s\n", meyveler[i]);
    }
    return 0;
}
```

ÇIKTI

```
armut
elma
kiraz
muz
```

`strcmp` ASCII sıralamasına göre karşılaştırdığı için Türkçe harfler (`ç`, `ğ`, `ş` vb.) doğru sıralanmaz; bu örnekte yalnızca ASCII harfler kullandık (Bölüm 11'deki UTF-8 uyarısı burada da geçerlidir).

12.14 Sık Yapılan İşaretçi Hataları

Bölüm boyunca gördüğümüz hataları bir arada toplayalım. Hepsinin ortak noktası, derleyicinin çoğunu yakalamaması veya yalnızca uyarı vermesidir; bu yüzden `-Wall -Wextra` ile derlemek ve uyarıları ciddiye almak şarttır.

Hata	Sonuç	Çözüm
Başlatılmamış işaretçiyi kullanmak	Tanımsız davranış	`NULL` ya da geçerli adres ile başlatmak
`NULL` işaretçiyi `*` ile kullanmak	Çökme	Kullanmadan önce `p != NULL` denetlemek
Yerel değişkenin adresini döndürmek	Sarkan işaretçi	Çağıranın verdiği yere yazmak, `static` ya da `malloc`
Dizi sınırını aşan işaretçi ilerletmek	Tanımsız davranış	Bitiş adresini (`a + n`) aşmamak
String sabitini değiştirmek	Çökme/tanımsız davranış	`const char *` kullanmak, değişecek metin için `char s[]`
Fonksiyonda `sizeof(dizi)` ile eleman saymak	Yanlış sayı	Eleman sayısını ayrı parametre olarak geçirmek
`int *p, q;` ile iki işaretçi sanmak	`q` işaretçi değil	Her işaretçiyi ayrı satırda tanımlamak
`scanf`'e `&` unutmak	Çökme/tanımsız davranış	Değişken için `&x` , dizi ve işaretçi için adın kendisi

Üç Dilde · Üç Dilde: Adres ve Referans *C · C++ · C#*

C: Adresi açıkça `&` ile alır, `*` ile kullanırız; işaretçi aritmetiği, `NULL` ve `void *` elimizdedir. Bu bölümün konusu budur. İleride C++'ta aynı işi çoğu zaman daha güvenli yollarla yapacağız, C#'ta ise adres kavramı programcıdan büyük ölçüde gizlenecek.

C++: C işaretçilerinin hepsi C++'ta da vardır. Ek olarak **referans** (reference) gelir: `int &r = x;` ifadesi `r`'yi `x`'in bir takma adı yapar. `*` ve `&` yazmadan adresle geçirme sağlar:

```
void takas(int &a, int &b) {        // çağrı: takas(x, y);
    int gecici = a;
    a = b;
    b = gecici;
}
```

Referans bir kez bağlandıktan sonra başka bir nesneye yöneltilemez ve `NULL` olamaz; bu yüzden C'deki birçok işaretçi hatası ortadan kalkar. C++11 ile `NULL` yerine `nullptr` gelir ve ham işaretçilerin sahipliğini yöneten akıllı işaretçiler (`unique_ptr`, `shared_ptr`) Bölüm 29'da anlatılır.

C#: Sınıf nesnelerine erişim zaten referans üzerinden olur; programcı adres görmez, işaretçi aritmetiği yapmaz. Bir parametreyi çağıranın değişkenine bağlamak için `ref` ya da `out` kullanılır:

```
void Takas(ref int a, ref int b)    // çağrı: Takas(ref x, ref y);
{
    int gecici = a;
    a = b;
    b = gecici;
}
```

C# işaretçileri yalnızca `unsafe` bloklarında, derleyici ayarıyla izin verilirse kullanılabilir; olağan programlarda ihtiyaç duyulmaz. Dizi sınırlarını çalışma zamanı denetlediği için sarkan işaretçi ve sınır aşımı hataları da oluşmaz.

Bölüm Özeti

- Her değişkenin bir adresi vardır; `&x` adresi verir, `%p` ile yazdırılır. Adres değerleri bilgisayardan bilgisayara değişir.
- İşaretçi, adres tutan değişkendir: `int *p = &x;`. `*p` gösterilen yere erişir (dolaylama); `*p = 20;` `x`'i değiştirir. `p` adrestir, `*p` değerdir.
- `int *p, q;` yalnızca `p`'yi işaretçi yapar. Masaüstü sistemlerde işaretçilerin boyutu aynıdır (64 bitte 8 bayt); tip, `*p`'nin kaç bayt okuyacağını ve `p + 1`'in ne kadar ilerleyeceğini belirler.
- `NULL` hiçbir yeri göstermez; `*p` ile erişmek tanımsız davranıştır ve genellikle çökertir. Başlatılmamış işaretçi tanımsızdır. Tanımlarken adres ya da `NULL` verilir, kullanmadan önce `p != NULL` denetlenir.
- C'de her şey değerle geçer; "adresle geçirme" adresin değer olarak geçirilmesidir. `takas(&x, &y)`, `scanf("%d", &x)` ve birden çok sonuç döndürme aynı kalıptır.
- `p + 1` bir eleman ilerler (bayt değil); `p - q` eleman sayısını verir. `*p++` işaretçiyi ilerletir, `(*p)++` gösterilen değeri artırır.
- `a[i]` ile `*(a + i)` aynıdır. Fonksiyona dizi geçince adres geçer (`int a[]` ile `int *a` aynıdır); bu yüzden fonksiyon içinde `sizeof(a)` işaretçi boyutunu verir. Dizi adına atama yapılamaz.

- `char s[] = "Ali"` değiştirilebilir bir kopyadır; `const char *p = "Ali"` salt okunur metni gösterir. String sabitini değiştirmek tanımsız davranıştır.
- `const int *p`: değer sabit. `int *const p`: işaretçi sabit. `const` `*`'ın solundaysa değer, sağındaysa işaretçi sabittir.
- Yerel değişkenin (ya da yerel dizinin) adresini döndürmek sarkan işaretçi üretir.
- `int **pp`, işaretçiyi değiştiren fonksiyonlarda ve `argv`'de karşımıza çıkar. `main(int argc, char *argv[])` komut satırı argümanlarını alır; `argv[0]` program adıdır, `argv[argc]` `NULL`'dır.
- `int *p[3]` işaretçi dizisidir (24 bayt); `int (*p)[3]` 3 elemanlı diziyi gösteren tek işaretçidir (8 bayt). `int m[][3]` parametresi ikincisidir.
- `void *` türsüz genel adrestir; kullanmadan önce gerçek tipe çevrilir. Fonksiyon işaretçisi `int (*f)(int, int)` biçimindedir. `qsort` bunların ikisini birlikte kullanır; karşılaştırma fonksiyonunda `a - b` yerine `<` ve `>` kullanılır.
- `restrict`, "bu belleğe başka işaretçiyle erişilmez" sözüdür (`memcpy`); `volatile`, değerin dışarıdan değişebileceğini bildirir.

Alıştırmalar

1. `int *p, q;` tanımından sonra aşağıdakilerden hangisi doğrudur?

   A) `p` ve `q` ikisi de `int` işaretçisidir.
   B) `p` bir `int` işaretçisidir, `q` sıradan bir `int`'tir.
   C) `p` sıradan bir `int`'tir, `q` bir `int` işaretçisidir.
   D) İkisi de sıradan `int`'tir; `*` işareti burada anlam taşımaz.
   E) Tanım derlenmez.

2. Aşağıdaki kod parçasından sonra `p[2] + *(p + 1)` ifadesinin değeri kaçtır?

```
int a[] = {10, 20, 30, 40, 50};
int *p = a + 1;
```

   A) 50
   B) 60
   C) 70
   D) 80
   E) 90

3. `int a = 1, b = 2; const int *p = &a; int *const q = &a;` tanımlarından sonra aşağıdaki satırlardan hangisi derlenmez?

   A) `p = &b;`
   B) `*q = 5;`
   C) `a = 7;`
   D) `*p = 5;`
   E) `printf("%d", *p);`

4. `void f(int a[]) { ... }` fonksiyonu ana programdan `f(dizi);` diye çağrılıyor. Aşağıdakilerden hangisi doğrudur?

A) `a`, fonksiyon içinde bir işaretçidir; `sizeof(a)` dizinin boyutunu değil işaretçinin boyutunu verir.
B) `a`, `dizi`'nin kopyasıdır; fonksiyonda yapılan değişiklikler `dizi`'yi etkilemez.
C) `int a[]` ile `int *a` farklı anlama gelir; ikisi aynı parametre olarak kullanılamaz.
D) `sizeof(a) / sizeof(a[0])` her zaman dizinin eleman sayısını verir.
E) Fonksiyon, diziyi değiştiremez.

5. 64 bitlik bir sistemde `int *p[4];` tanımı için `sizeof(p)` kaçtır?

A) 4
B) 8
C) 16
D) 32
E) 40

6. Aşağıdakilerden hangisi, `int` alıp `int` döndüren bir fonksiyonu gösterebilen `f` işaretçisinin doğru tanımıdır?

A) `int *f(int);`
B) `int (*f)(int);`
C) `int *(f)(int);`
D) `int f(*int);`
E) `(int *)f(int);`

7. Programın çıktısı nedir?

```
int x = 4, y = 9;
int *p = &x, *q = &y;
*p = *q + 1;
q = p;
*q += 5;
printf("%d %d\n", x, y);
```

8. Programın çıktısı nedir?

```
int a[5] = {1, 2, 3, 4, 5};
int *p = a + 4;
int *q = a + 1;
printf("%d %d %d\n", (int)(p - q), *(p - 2), q[2]);
```

9. Programın çıktısı nedir?

```
int a[] = {5, 6, 7};
int *p = a;
*p++ += 10;
(*p)++;
printf("%d %d %d %d\n", a[0], a[1], a[2], *p);
```

10. Programın çıktısı nedir?

```
void f(int *a, int b) {
    *a = *a + b;
    b = 100;
    a = &b;
    *a = 7;
}
```

```c
int main(void) {
    int x = 1, y = 2;
    f(&x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

11. Aşağıdaki `takas` fonksiyonu çağırıldıktan sonra `main`'deki değişkenler değişmiyor. Nedenini açıklayın ve düzeltin.

```c
void takas(int *a, int *b) {
    int *gecici = a;
    a = b;
    b = gecici;
}
```

12. Aşağıdaki programda üç ayrı işaretçi hatası vardır. Bulun, her birinin nedenini açıklayın ve düzeltin.

```c
#include <stdio.h>

int *kare(int n) {
    int s = n * n;
    return &s;
}

int main(void) {
    int *p;
    *p = 5;
    char *m = "abc";
    m[0] = 'X';
    printf("%d %s\n", *kare(3), m);
    return 0;
}
```

13. Bir `int` dizisinin aritmetik ortalamasını ve en büyük elemanını tek fonksiyonda hesaplayan `void istatistik(const int *a, int n, double *ortalama, int *enBuyuk)` fonksiyonunu yazın. `main`'de `{4, 9, 2, 7, 5}` dizisiyle deneyin.
14. İki işaretçi kullanarak (biri başta, biri sonda) bir string'i yerinde ters çeviren `void ters(char *s)` fonksiyonunu yazın. `main`'de `char metin[] = "bilgisayar";` ile deneyin.
15. Komut satırından verilen tamsayıların toplamını ve en büyüğünü yazdıran bir program yazın. Hiç argüman verilmemişse "Kullanim: program sayi1 sayi2 ..." yazıp 1 ile çıksın.
16. `int uygula(int *a, int n, int (*f)(int))` fonksiyonunu yazın: `a` dizisinin her elemanını `f` fonksiyonundan geçirip yerine yazsın. `kare` ve `iki katı` fonksiyonlarıyla `{1, 2, 3, 4}` dizisinde deneyin. (Fonksiyon geri değer olarak işlenen eleman sayısını döndürsün.)

BÖLÜM 13

Dinamik Bellek Yönetimi

BU BÖLÜMDE

- Sabit boyutlu dizinin neden yetmediğini ve değişken uzunluklu dizinin (VLA) risklerini
- Bellek bölgelerini: kod, statik alan, yığın (stack) ve öbek (heap)
- `malloc`, `calloc`, `realloc` ve `free` işlevlerini; dönüş değerini denetlemeyi
- Çalışma zamanında boyutu belirlenen ve gerektiğinde büyüyen dizileri
- Fonksiyondan öbekte dizi döndürmeyi ve belleği kimin bırakacağını (sahiplik)
- Dinamik string ile dinamik iki boyutlu dizi oluşturmayı
- Bellek sızıntısı, sarkan işaretçi ve çifte `free` hatalarını tanımayı; Valgrind ve AddressSanitizer'ı kullanmayı

Şimdiye kadar yazdığımız programlarda her dizinin boyutu kaynak kodda sabitti: `int notlar[30];`. Oysa bir programın kaç veriyle çalışacağı çoğu zaman program çalışırken, kullanıcıdan ya da bir dosyadan öğrenilir. Boyutu büyük tutarsak bellek boşa gider, küçük tutarsak veriler sığmaz ve Bölüm 10'da gördüğümüz sınır taşması başlar.

Çözüm, gereken yeri çalışma anında, gereken miktarda istemek ve işimiz bitince geri vermektir. Buna **dinamik bellek ayırma** (dynamic memory allocation) denir. Bu bölümde yer isteyen ve geri veren dört standart fonksiyonu, bunların doğru kullanım kalıplarını ve yanlış kullanıldıklarında ortaya çıkan hataları göreceğiz. C'de belleği kendimiz yönettiğimiz için bu bölüm, dilin gücünün de tehlikesinin de en açık görüldüğü yerdir.

13.1 Neden Dinamik Bellek?

Bir sınıftaki öğrencilerin notlarını tutan bir program düşünelim. Öğrenci sayısını bilmiyorsak `int notlar[100];` yazmak bir tahmindir: 101. kayıt sığmaz, 10 kişilik bir sınıfta ise 90 yer boşa gider. C99 bunun için bir kestirme getirdi: **değişken uzunluklu dizi** (VLA, variable length array); boyutu çalışma anında belli olan dizi.

```
int n;
scanf("%d", &n);
int a[n];              // VLA: boyut çalışma anında belli
```

Bu yazım kısa ve cazip görünür, ama kitapta kullanmayacağız. Nedenleri şunlardır:

- VLA yığında (stack) yer alır ve yığın küçüktür (genellikle birkaç MB). `n` büyükse yığın taşar ve program çöker; hatayı yakalamanın yolu yoktur. `malloc` ise yer bulamazsa `NULL` döndürür ve biz kontrol ederiz.
- VLA, tanımlandığı blok bitince yok olur; fonksiyondan döndürülemez.
- Boyutu sonradan büyütülemez.

- C11'den beri VLA desteği derleyiciler için zorunlu değildir; bazıları (örneğin MSVC) hiç desteklemez.

> *Tarihçe · VLA'nın tutarsız geçmişi*
>
> VLA, C99 ile dile girdi. Ancak uygulaması zor bulunduğu için C11 standardı onu isteğe bağlı bir özellik yaptı: destekleyen derleyiciler `__STDC_NO_VLA__` makrosunu tanımlamaz, desteklemeyenler tanımlar. C++ hiçbir sürümünde VLA'yı standardına almadı.

13.2 Bellek Bölgeleri

Çalışan bir program belleği amaçları farklı birkaç bölgede kullanır:

Bölge	İçinde ne var?	Ömrü	Kim yönetir?	Boyutu
Kod	Derlenmiş makine komutları	Program boyunca	Sistem	Sabit
Statik alan	Global ve `static` değişkenler, string sabitleri	Program boyunca	Derleyici	Derleme anında belli
Yığın (stack)	Yerel değişkenler, parametreler	Fonksiyon bitene kadar	Otomatik	Küçük (genelde birkaç MB)
Öbek (heap)	`malloc`, `calloc`, `realloc` ile alınan bloklar	`free` edilene kadar	Programcı	Büyük (RAM'in elverdiği kadar)

Yerel değişkenlerin çağrı yığınında yaşadığını Bölüm 9'dan biliyoruz. Öbek ise elle yönetilen tek bölgedir: yeri biz isteriz, işimiz bitince biz geri veririz.

> **Not · Yığın ve öbek karışmasın**
>
> Türkçe kaynakların çoğu hem stack'e hem heap'e "yığın" der. Bu kitapta **stack = yığın**, **heap = öbek** olarak kullanılır. Karışma olasılığı olan yerde İngilizcesini parantez içinde yazacağız.

Bir işaretçinin (pointer) öbekteki bir bloğu göstermesi şöyle çizilebilir:

```
  YIĞIN (stack)                    ÖBEK (heap)
+----------------+              +----+----+----+----+----+
| n = 5          |              | 10 | 20 | 30 | 40 | 50 |
| p = (adres) ---+------------> +----+----+----+----+----+
+----------------+               malloc(5 * sizeof *p) ile
 main'in yerel                   alındı; free(p) yazılana
 değişkenleri                    kadar yaşar
```

`p` yığındadır ve fonksiyon bitince yok olur. Gösterdiği blok ise öbektedir ve `p` yok olsa bile durmaya devam eder. `free` yazmadan `p`'yi kaybedersek bloğa bir daha ulaşamayız; buna **bellek sızıntısı** (memory leak) denir. Sızıntıyı bölümün sonunda ayrıntılı göreceğiz.

13.3 malloc ve free

Dinamik bellek fonksiyonlarının hepsi `<stdlib.h>` başlık dosyasındadır:

```
void *malloc(size_t boyut);
void free(void *p);
```

`malloc` (memory allocation), öbekten `boyut` bayt ister ve ayrılan bloğun ilk baytının adresini döndürür. Yer bulamazsa `NULL` döndürür. Ayrılan bloğun içi ilk değersizdir; yerel değişkenlerdeki gibi çöp değer taşır (Bölüm 2). Dönüş tipi `void *` olduğundan, sonuç herhangi bir işaretçiye ek bir dönüşüm yazmadan atanabilir.

▸ mallocTemel.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *dizi = malloc(n * sizeof *dizi);   // 5 int'lik yer iste
    if (dizi == NULL) {                     // her malloc'tan hemen sonra kontrol
        printf("Bellek ayrılamadı.\n");
        return 1;
    }

    int toplam = 0;
    for (int i = 0; i < n; i++) {
        dizi[i] = (i + 1) * 10;             // sıradan dizi gibi kullan
        toplam += dizi[i];
    }
    printf("Ayrılan bayt: %zu\n", n * sizeof *dizi);
    printf("İlk: %d, son: %d, toplam: %d\n", dizi[0], dizi[n - 1], toplam);

    free(dizi);                             // işimiz bitti: yeri geri ver
    return 0;
}
```

```
ÇIKTI
Ayrılan bayt: 20
İlk: 10, son: 50, toplam: 150
```

Satır satır inceleyelim:

- `malloc(n * sizeof *dizi)`: bayt sayısı, eleman sayısı çarpı bir elemanın boyutudur. `sizeof *dizi`, "`dizi`'nin gösterdiği şeyin boyutu" demektir (`int`, yani 4 bayt); 5 × 4 = 20 bayt çıkar. `sizeof(int)` de yazılabilir, ama `sizeof *dizi` tercih edilir: ileride `dizi`'yi `double *` yaparsak bu satır kendiliğinden doğru kalır.
- Dönen adres bir diziyi gösterir. Bölüm 12'deki `a[i] == *(a + i)` eşitliği burada da geçerlidir; bu yüzden `dizi[i]` yazabiliriz.
- `free(dizi)`: bloğu öbeğe geri verir. Her `malloc` için tam bir `free` yazılır.

> **Not · C'de dönüşüm (cast) yazılmaz**
>
> Eski kitaplarda `int *p = (int *)malloc(...)` biçimi görülür. C'de bu dönüşüm gereksizdir, çünkü `void *` her işaretçi tipine kendiliğinden dönüşür. C++ ise bu dönüşümü ister; ama C++'ta `malloc` yerine `new` kullanılır (bölümün sonundaki karşılaştırmaya bakınız). Bu kitapta C'de dönüşüm yazmıyoruz.

> **▲ Sık Yapılan Hata · NULL denetimini atlamak**
>
> `malloc` yer bulamazsa `NULL` döndürür. Denetlemeden kullanırsak `NULL` işaretçi üzerinden yazmaya çalışırız ve program çöker (Linux'ta `Segmentation fault`). Küçük programlarda başarısızlık nadirdir, ama çok büyük bir blok istendiğinde ya da bellek kısıtlı sistemlerde olur. Alışkanlık: `malloc` satırının hemen altına `if (p == NULL)` yazmak. Denetimi `main`'de yapıyorsak `return 1;` ile çıkarız; bellek ayıran bir yardımcı fonksiyondaysak `NULL` döndürüp kararı çağırana bırakırız.

`free`'nin kuralları kısadır ama hepsi önemlidir:

- Yalnızca `malloc`, `calloc` veya `realloc`'tan gelen adres `free` edilir ve bloğun **başlangıç adresiyle** (ortasıyla değil) çağrılır.
- `free(p)` sonrasında `p`'nin içindeki adres değişmez, ama o bloğa artık dokunamayız.
- `free(NULL)` güvenlidir; hiçbir şey yapmaz.

13.4 calloc

```
void *calloc(size_t adet, size_t boyut);
```

`calloc` (clear allocation), `adet × boyut` bayt ayırır ve **hepsini sıfırlar**. Çarpmayı kendisi yaptığı için parametreleri ayrıdır:

▸ callocFark.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *a = calloc(n, sizeof *a);   // 5 int, hepsi 0
    if (a == NULL) {
        return 1;
    }
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    free(a);
    return 0;
}
```

```
ÇIKTI
0 0 0 0 0
```

`malloc` ise içeriğe dokunmaz. Onun verdiği bloğu doğrudan okumak, Bölüm 2'de gördüğümüz çöp değer sorunudur:

```c
int *p = malloc(5 * sizeof *p);
printf("%d\n", p[0]);        // TANIMSIZ: henüz değer verilmedi
```

	`malloc(n * sizeof *p)`	`calloc(n, sizeof *p)`
Parametre	Toplam bayt sayısı	Eleman sayısı ve eleman boyutu

	`malloc(n * sizeof *p)`	`calloc(n, sizeof *p)`
Başlangıç içeriği	Çöp (belirsiz)	Tüm baytlar 0
Ne zaman?	Hemen üstüne değer yazacaksak	Sayaç, toplam gibi 0'dan başlayacak diziler için

> **Not · Sıfır baytı her zaman 0.0 demek mi?**
> Tamsayılar için "tüm baytlar sıfır" değeri 0'dır. Kayan noktalı sayılar için de, kullandığımız yaygın platformlarda 0.0 olur; ama C standardı bunu işaretçiler ve kayan noktalı sayılar için garanti etmez. Bu kitapta `calloc`'u yalnızca tamsayı dizilerinin sıfırlanmasında kullanacağız.

13.5 Boyutu Kullanıcıdan Alıp Dizi Oluşturmak

Dinamik belleğin asıl işi budur: eleman sayısı ancak çalışma anında belli olur. Aşağıdaki program önce kaç not gireceğimizi sorar, sonra tam o kadar yer ayırır:

▸ dinamikOrtalama.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 0;
    printf("Kaç not girilecek? ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Geçersiz sayı.\n");
        return 1;
    }

    int *notlar = malloc(n * sizeof *notlar);
    if (notlar == NULL) {
        printf("Bellek ayrılamadı.\n");
        return 1;
    }

    printf("Notları girin: ");
    int toplam = 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &notlar[i]);
        toplam += notlar[i];
    }
    printf("Ortalama: %.2f\n", (double)toplam / n);

    free(notlar);
    return 0;
}
```

```
ÇIKTI
Kaç not girilecek? 5
Notları girin: 70 85 90 60 95
Ortalama: 80.00
```

Sıraya dikkat edelim: önce `n` okunur ve geçerliliği denetlenir (`n <= 0` ise `malloc(0)` ya da negatif bir sayı anlamsız olur), sonra `malloc` çağrılır, sonra da `NULL` denetimi yapılır. `scanf("%d", &notlar[i])` yerine `notlar + i` yazmak da mümkündü; bu eşdeğerliği Bölüm 12'de görmüştük.

> **◆ Kritik Nokta · Bayt hesabında çarpmayı unutmayalım**
>
> `malloc(n)` yazarsak `n` **bayt** ayrılır, `n` tane `int` değil. `int` dizisi için `malloc(n * sizeof *p)` yazılmalıdır. Hatalı yazım derleyiciyi geçer, ama dizi gerekenin dörtte biri kadar yer kaplar ve taşma başlar.

Boyut hesabında çarpma çok büyük değerlerde taşabilir. `calloc(adet, boyut)` bu çarpmayı kendisi denetler ve taşarsa `NULL` döndürür; `malloc(adet * boyut)` ise sessizce küçük bir sayı hesaplayabilir. Boyutu dışarıdan gelen programlarda bu fark güvenlik açısından önemlidir.

13.6 realloc ile Büyüyen Dizi

Eleman sayısını baştan hiç bilmiyorsak? "0 girilene kadar sayı oku" gibi durumlarda diziyi doldukça büyütmemiz gerekir. Bunun için `realloc` vardır:

```
void *realloc(void *p, size_t yeniBoyut);
```

`realloc` (re-allocation), `p`'nin gösterdiği bloğu `yeniBoyut` bayta büyütür ya da küçültür. Kuralları şöyledir:

- Eski içerik korunur (küçültürken sığan kısmı).
- Blok yerinde genişleyemiyorsa `realloc` yeni bir yer bulur, içeriği oraya kopyalar ve eski bloğu kendisi `free` eder. Dönen adres bu yüzden eskisinden **farklı olabilir**; eski adresi kullanmaya devam etmemeliyiz.
- Eklenen yeni kısım sıfırlanmaz; çöptür.
- Başarısız olursa `NULL` döndürür ve eski blok olduğu gibi durur.
- `realloc(NULL, n)` tam olarak `malloc(n)` gibi çalışır.
- `realloc(p, 0)` yazmayalım; davranışı taşınabilir değildir. Bloğu bırakmak için `free` vardır.

Üç elemanlı bir diziyi beş elemanlıya büyütmenin doğru kalıbı şudur:

```
int *yeni = realloc(p, 5 * sizeof *yeni);
if (yeni == NULL) {
    free(p);                // eski blok hâlâ bizim
    return 1;
}
p = yeni;                   // artık yeni adres geçerli
```

Büyütme başarılıysa ilk üç eleman olduğu gibi korunur; yalnızca yeni eklenen 3. ve 4. indeksler çöptür ve onlara değer vermemiz gerekir. Geçici `yeni` işaretçisi bir tesadüf değildir, nedeni aşağıdaki kutudadır.

> **▲ Sık Yapılan Hata · p = realloc(p, ...) yazmak**
>
> En çok görülen `realloc` hatası, sonucu doğrudan aynı işaretçiye atamaktır:
>
> ```
> p = realloc(p, yeniBoyut);   // YANLIŞ: başarısız olursa p = NULL olur
> ```
>
> `realloc` başarısız olursa `NULL` döner ve bu `p`'ye atanır. Eski bloğun adresini tutan tek işaretçi gitmiş olur: blok öbekte durur ama ona ulaşamayız (sızıntı), üstelik içindeki veri de kaybolur. Sonuç her zaman önce geçici bir işaretçiye alınır, `NULL` değilse asıl işaretçiye verilir.

Kapasiteyi ikiye katlamak

Her yeni elemanda diziyi yalnızca 1 büyütürsek, `n` eleman için `n` kez `realloc` çağrılır ve blok her seferinde kopyalanabilir. Bunun yerine **kapasite** (capacity) ile **adet** kavramlarını ayırırız: kapasite, ayrılmış yer sayısıdır; adet, o an gerçekten kullanılan eleman sayısıdır. Adet kapasiteye ulaştığında kapasiteyi iki katına çıkarırız. Böylece `n` eleman için `realloc` yaklaşık $\log_2(n)$ kez çağrılır.

▸ buyuyenDizi.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int kapasite = 2;                   // başlangıçta 2 elemanlık yer
    int adet = 0;                       // şu an kaç eleman var
    int *sayilar = malloc(kapasite * sizeof *sayilar);
    if (sayilar == NULL) {
        return 1;
    }

    printf("Sayıları girin (0 ile biter): ");
    int x = 0;
    while (scanf("%d", &x) == 1 && x != 0) {
        if (adet == kapasite) {         // dizi dolu: büyüt
            int yeniKapasite = kapasite * 2;
            int *yeni = realloc(sayilar, yeniKapasite * sizeof *yeni);
            if (yeni == NULL) {
                free(sayilar);
                return 1;
            }
            sayilar = yeni;
            printf("[kapasite %d -> %d]\n", kapasite, yeniKapasite);
            kapasite = yeniKapasite;
        }
        sayilar[adet] = x;
        adet++;
    }

    int toplam = 0;
    for (int i = 0; i < adet; i++) {
        toplam += sayilar[i];
    }
    printf("Adet: %d, kapasite: %d, toplam: %d\n", adet, kapasite, toplam);

    free(sayilar);
    return 0;
}
```

```
ÇIKTI
Sayıları girin (0 ile biter): 4 8 15 16 23 42 0
[kapasite 2 -> 4]
[kapasite 4 -> 8]
Adet: 6, kapasite: 8, toplam: 108
```

İzleyelim. Kapasite 2 iken 4 ve 8 sığar. 15 gelince `adet == kapasite` olur ve kapasite 4'e çıkar; 15 ve 16 yerleşir. 23 gelince kapasite 8'e çıkar; 23 ve 42 de sığar. 0 gelince okuma biter. Sonuçta 6 eleman ve 8 kapasite vardır. Bir eleman eklemeden önce "dolu mu?" diye bakmak, dinamik dizinin temel kalıbıdır.

Büyütme işini fonksiyona taşımak

Bu kalıbı her yerde elle yazmak yerine bir fonksiyona koyabiliriz. Fonksiyon, dizinin adresini, adedi ve kapasiteyi değiştirmek zorundadır; bu yüzden üçünün de adresini alır (işaretçi parametreleri Bölüm 12'de gördük). `dizi` bir `int *` olduğundan, adresi `int **` tipindedir:

▸ dinamikListe.c

```c
#include <stdio.h>
#include <stdlib.h>

// Diziye bir eleman ekler; dolduysa kapasiteyi iki katına çıkarır.
// Başarılıysa 0, bellek ayrılamazsa -1 döndürür (dizi bozulmaz).
int ekle(int **dizi, int *adet, int *kapasite, int deger) {
    if (*adet == *kapasite) {
        int yeniKapasite = *kapasite * 2;
        int *yeni = realloc(*dizi, yeniKapasite * sizeof **dizi);
        if (yeni == NULL) {
            return -1;
        }
        *dizi = yeni;
        *kapasite = yeniKapasite;
    }
    (*dizi)[*adet] = deger;
    (*adet)++;
    return 0;
}

int main(void) {
    int kapasite = 2;
    int adet = 0;
    int *dizi = malloc(kapasite * sizeof *dizi);
    if (dizi == NULL) {
        return 1;
    }

    for (int i = 1; i <= 10; i++) {
        if (ekle(&dizi, &adet, &kapasite, i * i) != 0) {
            free(dizi);
            return 1;
        }
    }

    printf("Adet: %d, kapasite: %d\n", adet, kapasite);
    for (int i = 0; i < adet; i++) {
        printf("%d ", dizi[i]);
    }
    printf("\n");

    free(dizi);
    return 0;
}
```

```
ÇIKTI
Adet: 10, kapasite: 16
1 4 9 16 25 36 49 64 81 100
```

`(*dizi)[*adet]` ifadesinde parantez gereklidir: `*dizi[*adet]` yazarsak `[]` operatörü `*`'dan önce çalışır ve yanlış şeyi çözmeye çalışırız (öncelik kuralları Bölüm 2'de). Bölüm 14'te `dizi`, `adet` ve `kapasite`'yi tek bir yapı (`struct`) içinde toplayarak bu üç parametreyi tek parametreye indireceğiz.

13.7 Fonksiyondan Dinamik Dizi Döndürmek

Bölüm 9'da yerel bir dizinin adresini döndürmenin sarkan işaretçi verdiğini görmüştük, çünkü dizi yığındadır ve fonksiyon bitince yok olur. Öbekteki blok ise `free` edilene kadar yaşar; fonksiyon onu rahatça döndürebilir:

▸ kareDizisi.c

```c
#include <stdio.h>
#include <stdlib.h>

// 1'den n'e kadar sayıların karelerini tutan yeni bir dizi döndürür.
// Hata olursa NULL. Dönen belleği ÇAĞIRAN free etmelidir.
int *kareleriOlustur(int n) {
    int *a = malloc(n * sizeof *a);
    if (a == NULL) {
        return NULL;
    }
    for (int i = 0; i < n; i++) {
        a[i] = (i + 1) * (i + 1);
    }
    return a;
}

int main(void) {
    int n = 5;
    int *kareler = kareleriOlustur(n);
    if (kareler == NULL) {
        return 1;
    }
    for (int i = 0; i < n; i++) {
        printf("%d ", kareler[i]);
    }
    printf("\n");
    free(kareler);
    return 0;
}
```

ÇIKTI

```
1 4 9 16 25
```

Burada yeni bir soru çıkar: belleği kim serbest bırakacak? Bloğu `kareleriOlustur` ayırdı, ama `main` kullandı; `free`'yi `main` yazar. Bu sorumluluğa **sahiplik** (ownership) denir: fonksiyon bloğu döndürdüğü anda sahipliği çağırana devreder. Bunu fonksiyonun üstüne yorum olarak yazmak iyi bir alışkanlıktır.

Kalıp	Kim ayırır?	Kim `free` eder?
`p = olustur(...)` (yukarıdaki)	Fonksiyon	Çağıran
`doldur(tampon, n)` (`fgets` gibi)	Çağıran	Çağıran
`char s[50]; ... return s;`	Yığın	Geçersiz! (Bölüm 9)

> ◆ **Kritik Nokta · Sahiplik kuralı**
>
> Bir fonksiyon "yeni blok döndürür" diyorsa, onu çağıran `free` eder. Fonksiyon içinde `free(a)` yapıp ardından `return a;` yazarsak sarkan işaretçi döndürmüş oluruz.

13.8 Dinamik String

String, sonu `'\0'` ile biten `char` dizisidir (Bölüm 11). Öbekte string tutmak için `strlen(s) + 1` bayt ayırırız: `+1`, sondaki `'\0'` içindir. `sizeof(char)` her zaman 1 olduğundan çarpmaya gerek yoktur.

▸ kopyaOlustur.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// kaynak'ın öbekte yeni bir kopyasını döndürür (strdup benzeri).
// Hata olursa NULL. Dönen belleği ÇAĞIRAN free etmelidir.
char *kopyaOlustur(const char *kaynak) {
    char *kopya = malloc(strlen(kaynak) + 1);   // +1: '\0' için
    if (kopya == NULL) {
        return NULL;
    }
    strcpy(kopya, kaynak);
    return kopya;
}

int main(void) {
    const char *sabit = "Merhaba";           // string sabiti: değiştirilemez
    char *kopya = kopyaOlustur(sabit);
    if (kopya == NULL) {
        return 1;
    }
    kopya[0] = 'm';                          // kopya öbekte: değiştirilebilir

    printf("sabit: %s\n", sabit);
    printf("kopya: %s\n", kopya);
    printf("uzunluk: %zu, ayrılan: %zu bayt\n", strlen(kopya), strlen(kopya) + 1);

    free(kopya);
    return 0;
}
```

```
ÇIKTI
sabit: Merhaba
kopya: merhaba
uzunluk: 7, ayrılan: 8 bayt
```

İki kazanç var: kopya, orijinal değişse bile ondan bağımsızdır ve string sabitinin aksine değiştirilebilir (Bölüm 12'de string sabitlerinin salt okunur olduğunu görmüştük). Fonksiyonun `kaynak` parametresini `const char *` yaptık, çünkü onu yalnızca okuyoruz.

▲ Sık Yapılan Hata · Dinamik stringde +1'i unutmak

```c
char *kopya = malloc(strlen(kaynak));    // '\0' için yer yok
strcpy(kopya, kaynak);                   // '\0' blok dışına yazılır
```

"Merhaba" için 7 bayt ayrılır, ama `strcpy` 8 bayt yazar. Bu tanımsız davranıştır: bazen hiçbir şey olmaz, bazen sonraki `free` ya da `malloc` çöker. Test sırasında fark edilmediği için özellikle tehlikelidir. Kural: `malloc(strlen(s) + 1)`. Türkçe karakter varsa `strlen` karakteri değil baytı saydığından (Bölüm 11), ayrılan miktar da bayt cinsindedir ve tutarlıdır.

> *Tarihçe · strdup standartlaşıyor*
>
> Bu `kopyaOlustur` fonksiyonunun işini kütüphanedeki `strdup` yapar. POSIX sistemlerinde uzun süredir vardır, ama C17'ye kadar C standardında yoktu. C23, `strdup`'ı `<string.h>` içine aldı. Taşınabilirlik için kalıbı bilmek, gerektiğinde kendimiz yazabilmek yararlıdır.

13.9 Dinamik İki Boyutlu Dizi

Matrisin boyutu da çalışma anında belli olabilir. İki yol vardır.

Yol 1: İşaretçi dizisi

Önce satır sayısı kadar `int *` tutan bir dizi ayırırız (`int **m`, işaretçinin işaretçisi; Bölüm 12), sonra her satır için ayrı bir `int` dizisi ayırırız. Bu bloklar ayrı `malloc`'lardır; bu yüzden her birine ayrı `free` gerekir ve **ters sırayla** bırakılır: önce satırlar, en son satır işaretçileri. Önce `m`'yi bırakırsak `m[i]` adreslerini okuyacak yer kalmaz.

```
  m               satırlar (her biri ayrı malloc)
+------+
| m[0] |--------> [  1 |  2 |  3 |  4 ]
+------+
| m[1] |--------> [  5 |  6 |  7 |  8 ]
+------+
| m[2] |--------> [  9 | 10 | 11 | 12 ]
+------+
```

▸ matris.c

```c
#include <stdio.h>
#include <stdlib.h>

int **matrisOlustur(int satir, int sutun) {
    int **m = malloc(satir * sizeof *m);           // satır işaretçileri
    if (m == NULL) {
        return NULL;
    }
    for (int i = 0; i < satir; i++) {
        m[i] = malloc(sutun * sizeof **m);         // i. satırın elemanları
        if (m[i] == NULL) {
            for (int k = 0; k < i; k++) {          // şimdiye kadarkileri geri ver
                free(m[k]);
            }
            free(m);
            return NULL;
        }
    }
    return m;
}

void matrisBirak(int **m, int satir) {
    for (int i = 0; i < satir; i++) {
        free(m[i]);                                // önce satırlar
    }
    free(m);                                       // en son satır işaretçileri
}

int main(void) {
```

```
    int satir = 3;
    int sutun = 4;
    int **m = matrisOlustur(satir, sutun);
    if (m == NULL) {
        return 1;
    }

    int toplam = 0;
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            m[i][j] = i * sutun + j + 1;
            toplam += m[i][j];
            printf("%3d", m[i][j]);
        }
        printf("\n");
    }
    printf("Toplam: %d\n", toplam);

    matrisBirak(m, satir);
    return 0;
}
```

```
ÇIKTI
  1  2  3  4
  5  6  7  8
  9 10 11 12
Toplam: 78
```

`m[i][j]` yazımı, sabit boyutlu `int m[3][4]` ile aynıdır. Fark, satırların bellekte peş peşe durmak zorunda olmamasıdır. Bunun bir yararı, satır uzunluklarının farklı olabilmesidir. Aşağıdaki program, `i`. satırında `i + 1` eleman bulunan bir üçgen dizi (Pascal üçgeni) kurar; sabit boyutlu bir dizide bu kadar esnek olamazdık:

▸ ucgenDizi.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int satir = 5;
    int **u = malloc(satir * sizeof *u);
    if (u == NULL) {
        return 1;
    }

    for (int i = 0; i < satir; i++) {
        u[i] = malloc((i + 1) * sizeof **u);        // i. satırda i+1 eleman
        if (u[i] == NULL) {
            for (int k = 0; k < i; k++) {
                free(u[k]);
            }
            free(u);
            return 1;
        }
        u[i][0] = 1;
        u[i][i] = 1;
        for (int j = 1; j < i; j++) {
            u[i][j] = u[i - 1][j - 1] + u[i - 1][j];
        }
    }

    for (int i = 0; i < satir; i++) {
        for (int j = 0; j <= i; j++) {
```

```
            printf("%d ", u[i][j]);
        }
        printf("\n");
    }

    for (int i = 0; i < satir; i++) {
        free(u[i]);
    }
    free(u);
    return 0;
}
```

```
ÇIKTI
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Yol 2: Tek blok

İkinci yol, `satir × sutun` elemanı tek seferde ayırıp `[i][j]` yerine indeksi kendimiz hesaplamaktır: `m[i * sutun + j]`.

```
int *m = malloc(satir * sutun * sizeof *m);    // tek malloc
m[i * sutun + j] = 5;                          // m[i][j] yerine
free(m);                                       // tek free
```

	İşaretçi dizisi (`int **m`)	Tek blok (`int *m`)
`malloc` / `free` sayısı	`satir + 1`	1
Eleman erişimi	`m[i][j]`	`m[i * sutun + j]`
Elemanlar bellekte bitişik mi?	Hayır (satırlar ayrı)	Evet
Satır uzunlukları farklı olabilir mi?	Evet	Hayır

Tek blok yolu daha az `malloc` gerektirir ve bellekte bitişik olduğu için işlemci önbelleğiyle daha iyi çalışır. İşaretçi dizisi ise okunaklı `m[i][j]` yazımını ve değişken satır uzunluğunu sağlar.

13.10 Bellek Hataları

Dinamik bellek hataları genellikle derleyiciyi geçer. Program çalışır, çoğu zaman doğru sonuç da verir; hata günler sonra, başka bir yerde çöküş olarak ortaya çıkar. Aşağıdaki kod parçaları bilerek hatalıdır. Sonuçları tanımsız olduğu için bunların çıktısını göstermeyeceğiz.

Bellek sızıntısı

```
void sizinti(void) {
    int *p = malloc(100 * sizeof *p);
    if (p == NULL) {
        return;
```

```
    }
    p[0] = 1;
}                                   // free(p) yok: 400 baytlık blok öbekte kalır
```

Ayrılan bloğun adresini tutan son işaretçi kaybolunca blok, program bitene kadar kullanılamaz hâle gelir. Program hemen bozulmaz; ama sunucu ya da oyun gibi uzun çalışan programlarda bellek yavaş yavaş tükenir. Program bitince işletim sistemi belleği geri alır, yine de sızıntı bir hatadır. Klasik sızıntı yolları şunlardır:

- `free` yazmayı unutmak (yukarıdaki örnek).
- Adresi kaybetmek: `p = malloc(...); p = malloc(...);`. İlk bloğun adresi `p`'den silindi.
- Erken çıkış: hata durumunda `return` ederken, daha önce ayrılan blokları `free` etmeyi unutmak.
- `p = realloc(p, ...)` yazıp `realloc`'un başarısız olması (yukarıda gördük).

free sonrası kullanım (sarkan işaretçi)

```
int *p = malloc(sizeof *p);
*p = 7;
free(p);
printf("%d\n", *p);         // TANIMSIZ: p artık bizim olmayan bir yeri gösteriyor
```

`free`'den sonra `p`'nin içindeki adres değişmez, ama blok artık bize ait değildir; başka bir `malloc` o yeri başkasına verebilir. `*p` eski değeri de verebilir, çöp de verebilir, program da çökebilir. Böyle bir işaretçiye **sarkan işaretçi** (dangling pointer) denir. Bölüm 12'de yerel değişkenin adresini döndürünce de aynı durumu görmüştük.

Çifte free ve yanlış adresi free etmek

```
free(p);
free(p);                    // TANIMSIZ: aynı bloğu ikinci kez bırakmak

int x = 5;
free(&x);                   // TANIMSIZ: x yığında, öbekte değil
free(p + 1);                // TANIMSIZ: bloğun başı değil, ortası
```

Üçü de öbeğin iç düzenini bozar. Çoğu sistemde program çöker, ama bunun garantisi yoktur.

Blok sınırını aşmak

```
int *p = malloc(5 * sizeof *p);
p[5] = 1;                   // TANIMSIZ: geçerli indeksler 0..4
```

Bu, Bölüm 10'daki dizi taşmasının öbekteki hâlidir. Komşu bloğun yönetim bilgilerini bozabildiği için hata çoğu zaman çok sonra, bir `free` çağrısında patlar.

> ◆ **Kritik Nokta · Sızıntıyı ve sarkan işaretçiyi nasıl okuruz?**
>
> Bellek hatalarını kod okurken bulmanın yöntemi tek sorudur: "Bu bloğun adresini şu an hangi işaretçiler tutuyor?" Blok bırakılmadan önce son işaretçi başka bir adrese atanırsa ya da yok olursa sızıntı vardır. Blok bırakıldıktan sonra ona erişen her ifade sarkan işaretçi kullanımıdır.

free(p); p = NULL;

Çifte `free` ve sarkan işaretçiden korunmanın en ucuz yolu, `free`'den hemen sonra işaretçiyi `NULL` yapmaktır. `free(NULL)` güvenli olduğundan ikinci bir `free` zarar vermez, `if (p != NULL)` ile de bloğun hâlâ geçerli olup olmadığına bakılabilir:

▸ freeNull.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(sizeof *p);
    if (p == NULL) {
        return 1;
    }
    *p = 7;
    printf("Önce : %d\n", *p);

    free(p);
    p = NULL;                   // sarkan işaretçiyi temizle
    if (p == NULL) {
        printf("Sonra: p boş, kullanılmaz\n");
    }

    free(p);                    // free(NULL): güvenli, bir şey yapmaz
    printf("free(NULL) sorun çıkarmadı\n");
    return 0;
}
```

ÇIKTI

```
Önce : 7
Sonra: p boş, kullanılmaz
free(NULL) sorun çıkarmadı
```

> **İpucu · Her malloc için bir free**
>
> `malloc` satırını yazdığımız anda `free` satırını da (ya da en az "burada free edilecek" yorumunu) yazalım. Fonksiyondan erken çıkan her `return` yolunun `free`'yi atlamadığını kontrol edelim. `p = NULL` alışkanlığı yalnızca o işaretçiyi korur: `int *q = p;` ile oluşan ikinci işaretçi, `free(p)` sonrasında hâlâ sarkan kalır.

13.11 Bellek Hatalarını Bulan Araçlar

Bellek hatalarını gözle bulmak zordur; bunun için araçlar vardır. Hepsinde programı hata ayıklama bilgisiyle (`-g`) derlemek, raporun satır numarasını da göstermesini sağlar.

Araç	Platform	Kullanım
Valgrind	Linux	`valgrind --leak-check=full ./program`
Dr. Memory	Windows	`drmemory -- program.exe`
AddressSanitizer	gcc ve clang	derlerken `-fsanitize=address`

Sızıntılı bir programı Valgrind ile çalıştıralım. Program, `malloc` ile 400 bayt ayırıp `free` etmeden bitiyor:

▸ **sizinti.c**

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(100 * sizeof *p);
    if (p == NULL) {
        return 1;
    }
    p[0] = 1;
    printf("p[0] = %d\n", p[0]);
    return 0;                   // free(p) yok
}
```

ÇIKTI

```
p[0] = 1
```

TERMİNAL

```
$ gcc -std=c17 -Wall -g sizinti.c -o sizinti
$ valgrind --leak-check=full ./sizinti
p[0] = 1
==18382== HEAP SUMMARY:
==18382==     in use at exit: 400 bytes in 1 blocks
==18382==   total heap usage: 2 allocs, 1 frees, 4,496 bytes allocated
==18382==
==18382== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==18382==    at 0x4846828: malloc (in .../vgpreload_memcheck-amd64-linux.so)
==18382==    by 0x10917E: main (sizinti.c:5)
==18382==
==18382== LEAK SUMMARY:
==18382==    definitely lost: 400 bytes in 1 blocks
```

Rapor (okunurluk için kısaltılmıştır) 400 baytlık bir bloğun "kesin kaybolmuş" (definitely lost; hiçbir işaretçinin göstermediği blok) olduğunu ve bu bloğun `sizinti.c` dosyasının 5. satırındaki `malloc` ile ayrıldığını söyler. Sayıların bir kısmı çalıştırmaya göre değişir. "2 allocs, 1 frees" ifadesindeki fazladan ayırma, büyük olasılıkla `printf`'in kullandığı iç arabellektir.

AddressSanitizer ise derleyicinin programa eklediği denetimlerle taşma, `free` sonrası kullanım ve çifte `free` hatalarını çalışma anında yakalar. Önceki bölümdeki `p[5] = 1;` hatasını içeren bir programı şöyle derleyip çalıştırırız:

TERMİNAL

```
$ gcc -std=c17 -g -fsanitize=address tasma.c -o tasma
$ ./tasma
==18388==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x503000000054
WRITE of size 4 at 0x503000000054 thread T0
    #0 0x5582a6cb42b2 in main tasma.c:9
0x503000000054 is located 0 bytes after 20-byte region [0x503000000040,0x503000000054)
allocated by thread T0 here:
    #0 0x7fc9fccfd9c7 in malloc
    #1 0x5582a6cb425e in main tasma.c:5
```

Mesaj, 20 baytlık (5 `int`) bir bloğun hemen ardına yazıldığını (`0 bytes after 20-byte region`), yazmanın 9. satırda, bloğun ayrıldığı yerin ise 5. satırda olduğunu gösterir.

Not · Platformlara göre araç seçimi

Windows'ta MinGW gcc çoğu kurulumda AddressSanitizer içermez; orada Dr. Memory kullanılır. macOS'ta (özellikle Apple işlemcilerde) Valgrind çalışmaz; `-fsanitize=address` çalışır, sızıntı için `leaks --atExit -- ./program` kullanılabilir. Linux'ta AddressSanitizer sızıntıları da raporlar. Bu araçlarla bir programı teslim etmeden önce bir kez taramak iyi bir alışkanlıktır.

13.12 sizeof ve Ayrılan Boyut

Bir dizinin `sizeof` değeri tüm dizinin bayt sayısıdır (Bölüm 10). `malloc` ile aldığımız bloğu gösteren işaretçi için ise `sizeof`, yalnızca **işaretçinin** boyutunu verir:

▸ sizeofIsaretci.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int sabit[10];
    int *dinamik = malloc(10 * sizeof *dinamik);
    if (dinamik == NULL) {
        return 1;
    }
    printf("sizeof(sabit)   = %zu\n", sizeof(sabit));
    printf("sizeof(dinamik) = %zu\n", sizeof(dinamik));
    free(dinamik);
    return 0;
}
```

ÇIKTI

```
sizeof(sabit)   = 40
sizeof(dinamik) = 8
```

64 bitlik sistemlerde bir işaretçi 8 bayttır (32 bitlikte 4); yani `sizeof(dinamik)` bloğun 40 baytını vermez. Bloğun büyüklüğünü işaretçiden sonradan öğrenemeyiz. Bu yüzden eleman sayısını ayrı bir değişkende tutarız (`n`, `adet`, `kapasite`) ve diziyi fonksiyona geçirirken boyutu da geçiririz (Bölüm 10). Bölüm 14'te diziyi ve eleman sayısını tek bir `struct` içinde birlikte taşımayı göreceğiz.

13.13 Üç Dilde Bellek Yönetimi

Üç Dilde · Belleği kim temizler? *C · C++ · C#*

C'de her `malloc` için bir `free` yazmak programcının işidir; unutulursa sızıntı, fazla yazılırsa çifte `free` oluşur. İleride göreceğimiz C++ ve C# bu yükü farklı yollarla hafifletir.

C++'ta `malloc` yerine `new` ve `delete` vardır; ayrıca nesne ömrüne bağlı otomatik temizlik (RAII) ve akıllı işaretçiler (smart pointer) kullanılır. Dizi ihtiyacı için de çoğu zaman `std::vector` yeterlidir:

```cpp
int *p = new int[5];            // C'deki malloc(5 * sizeof *p)
delete[] p;                     // C'deki free(p)
```

```
auto q = std::make_unique<int[]>(5);   // kapsam bitince kendiliğinden silinir
std::vector<int> v;                    // kendi kendini büyütür ve temizler
v.push_back(42);
```

`vector`, bu bölümde elle yazdığımız "kapasiteyi ikiye kat" mantığını kendisi yönetir.

C#'ta bellek, **çöp toplayıcı** (garbage collector, GC) tarafından otomatik yönetilir. `new` ile nesne ya da dizi oluştururuz; ona hiçbir referans kalmayınca GC bloğu uygun bir anda geri alır, `free` karşılığı bir komut yoktur:

```
int[] a = new int[5];           // öbekte (managed heap) ayrılır
var liste = new List<int>();    // büyüyen dizi
liste.Add(42);
// a = null; ya da kapsam bitişi: GC zamanı gelince temizler
```

Sızıntı mümkün olmaz demek yanlış olur (kullanılmayan nesnelere referans tutmak belleği şişirir), ama `free` unutmak ya da sarkan işaretçi gibi hatalar C#'ta yoktur. Karşılığında, belleğin tam ne zaman boşalacağı programcının elinde değildir.

	C	C++	C#
Yer ayırma	`malloc`, `calloc`	`new`, `make_unique`, `vector`	`new`
Büyütme	`realloc` (elle)	`vector::push_back`	`List<T>.Add`
Geri verme	`free` (elle)	`delete` / otomatik (RAII)	Çöp toplayıcı
Sarkan işaretçi riski	Var	Ham işaretçide var	Yok

Bölüm Özeti

- Dizi boyutu derleme anında belli olmak zorundaysa sabit dizi, çalışma anında belli oluyorsa dinamik bellek kullanılır. VLA yığında yer aldığı için riskli kabul edilir ve kitapta kullanılmaz.
- Bellek bölgeleri: kod, statik alan (global ve `static`), yığın (yerel değişkenler, otomatik) ve öbek (`malloc` ile alınan, elle yönetilen).
- `malloc(n * sizeof *p)`, `n` eleman için yer ayırır; içi çöptür. Dönen `void *` C'de dönüşüm istemez. Her `malloc`'tan hemen sonra `NULL` denetimi yapılır ve her `malloc` için bir `free` yazılır.
- `calloc(adet, boyut)`, yeri sıfırlayarak ayırır ve çarpma taşmasını denetler. `malloc(25 * sizeof(int))`, 4 baytlık `int` ile 100 bayt ister.
- `realloc`, bloğu büyütür ya da küçültür, eski içeriği korur; adres değişebilir, yeni kısım çöptür. `p = realloc(p, ...)` yazılmaz, geçici işaretçi kullanılır.
- Büyüyen dizide kapasiteyi ikiye katlamak, `realloc` çağrı sayısını yaklaşık $\log_2(n)$'e indirir. Adet ve kapasite ayrı tutulur.
- Fonksiyon öbekte blok döndürebilir (yerel diziden farklı olarak); bloğu çağıran `free` eder (sahiplik). Fonksiyon içinde `free` edilen adres döndürülmez.
- Dinamik string için `malloc(strlen(s) + 1)` kullanılır; `+1` unutulursa `'\0'` blok dışına yazılır.
- Dinamik iki boyutlu dizi, ya işaretçi dizisiyle (`int **m`, `satir + 1` `malloc`, ters sırada `free`) ya da tek blokla (`m[i * sutun + j]`) kurulur.
- Başlıca hatalar: sızıntı (adres kaybı, `free` unutmak), sarkan işaretçi (`free` sonrası kullanım), çifte `free`, öbekte olmayan ya da bloğun başı olmayan adresi `free` etmek ve blok sınırını aşmak.

- `free(NULL)` güvenlidir. `free(p); p = NULL;` alışkanlığı tek işaretçiyi korur, kopya işaretçileri korumaz.
- `sizeof(p)` bloğun boyutunu değil işaretçinin boyutunu (64 bitte 8 bayt) verir; eleman sayısı ayrıca tutulur.
- Araçlar: Valgrind (Linux), Dr. Memory (Windows), `-fsanitize=address` (gcc ve clang).
- C++ `new` / `delete`, akıllı işaretçiler ve `vector` ile; C# çöp toplayıcıyla bu işi otomatikleştirir.

Alıştırmalar

1. `int` 4 bayt olduğuna göre `int *p = malloc(25 * sizeof(int));` satırı öbekten kaç bayt ister?

   A) 25
   B) 29
   C) 100
   D) 104
   E) 8

2. Aşağıdakilerden hangisi `malloc` ile `calloc` arasındaki farkı doğru anlatır?

   A) `malloc` ayrılan baytları sıfırlar, `calloc` sıfırlamaz.
   B) `calloc(adet, boyut)` ayırdığı baytların hepsini 0 yapar; `malloc` içeriği olduğu gibi (çöp) bırakır.
   C) `malloc` öbekten, `calloc` yığından yer ayırır.
   D) `calloc` ile ayrılan bloğa `free` uygulanmaz.
   E) `malloc` yer bulamazsa `NULL` döndürür, `calloc` yer bulamazsa programı sonlandırır.

3. Aşağıdaki kod parçası çalışıp bittiğinde öbekte ne kalır?

```
int *a = malloc(10 * sizeof *a);
int *b = malloc(10 * sizeof *b);
a = b;
free(a);
```

   A) Hiçbir şey; iki blok da serbest bırakıldı.
   B) Bir blok sızar: ikinci `malloc`'ın bloğu, çünkü `b` doğrudan `free` edilmedi.
   C) İki blok sızar: `free(a)` yalnızca işaretçiyi siler, blokları değil.
   D) Bir blok sızar, ama hangisi olduğu belirsizdir.
   E) Bir blok sızar: ilk `malloc`'ın bloğu, çünkü `a = b` ile adresi kayboldu.

4. `p = realloc(p, yeniBoyut);` yazımının asıl sorunu nedir?

   A) `realloc` başarısız olup `NULL` döndürürse `p`'nin eski adresi kaybolur; eski blok sızar.
   B) `realloc` her zaman yeni bir yere taşır, bu yüzden eski veriler silinir.
   C) `realloc` yalnızca bloğu küçültmek için kullanılabilir.
   D) `realloc`'un döndürdüğü adres `p`'ye atanmamalıdır; `p` kendiliğinden güncellenir.
   E) `realloc` yeni eklenen elemanları sıfırladığı için programı gereksiz yavaşlatır.

5. Dinamik bellekle ilgili aşağıdakilerden hangisi doğrudur?

   A) `free(p)` çağrısından sonra `p` kendiliğinden `NULL` olur.
   B) `malloc`'ın döndürdüğü adres C'de mutlaka `(int *)` ile dönüştürülmek zorundadır.
   C) `malloc` ile ayrılan bloğun bayt sayısı `sizeof(p)` ile öğrenilir.

D) `free(NULL)` çağrısı güvenlidir; hiçbir şey yapmaz.

E) Aynı adrese ikinci kez `free` yazmak güvenlidir, çünkü blok zaten serbesttir.

6. Aşağıdaki programın çıktısı nedir?

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = calloc(5, sizeof *p);
    if (p == NULL) {
        return 1;
    }
    p[0] = 3;
    p[2] = p[0] * 4;
    p[4] = p[2] - p[0];

    int toplam = 0;
    for (int i = 0; i < 5; i++) {
        toplam += p[i];
    }

    int *yeni = realloc(p, 7 * sizeof *yeni);
    if (yeni == NULL) {
        free(p);
        return 1;
    }
    p = yeni;
    p[5] = toplam;
    p[6] = p[5] / p[0];

    printf("%d\n", toplam);
    for (int i = 0; i < 7; i++) {
        printf("%d ", p[i]);
    }
    printf("\n");
    free(p);
    return 0;
}
```

7. Aşağıdaki programın çıktısı nedir?

```
#include <stdio.h>
#include <stdlib.h>

int *kopyala(const int *a, int n) {
    int *k = malloc(n * sizeof *k);
    if (k == NULL) {
        return NULL;
    }
    for (int i = 0; i < n; i++) {
        k[i] = a[n - 1 - i] + 10;
    }
    return k;
}

int main(void) {
    int a[4] = {1, 2, 3, 4};
    int *k = kopyala(a, 4);
    if (k == NULL) {
        return 1;
    }
```

```
    a[0] = 100;
    for (int i = 0; i < 4; i++) {
        printf("%d ", k[i]);
    }
    printf("\n%d %d\n", a[0], k[3]);
    free(k);
    return 0;
}
```

8. Aşağıdaki programın çıktısı nedir? (Kısalık için `calloc` dönüş değerleri denetlenmemiştir.)

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int satir = 4;
    int adet = 0;
    int toplam = 0;
    int **m = malloc(satir * sizeof *m);
    if (m == NULL) {
        return 1;
    }
    for (int i = 0; i < satir; i++) {
        m[i] = calloc(i + 1, sizeof **m);
        for (int j = 0; j <= i; j++) {
            m[i][j] = i * j + 1;
            adet++;
            toplam += m[i][j];
        }
    }
    printf("%d %d\n", adet, toplam);
    printf("%d %d\n", m[3][2], m[2][1]);
    for (int i = 0; i < satir; i++) {
        free(m[i]);
    }
    free(m);
    return 0;
}
```

9. Aşağıdaki fonksiyonda `malloc` kullanımıyla ilgili iki hata vardır. Bulun ve düzeltin. (Gerekli `#include` satırlarının yazıldığını varsayın.)

```
char *buyukHarfKopya(const char *s) {
    char *k = malloc(strlen(s));
    strcpy(k, s);
    for (int i = 0; k[i] != '\0'; i++) {
        k[i] = toupper(k[i]);
    }
    return k;
}
```

10. Aşağıdaki fonksiyon neden hatalıdır? Doğru hâlini yazın.

```
int *ilkSayilar(int n) {
    int a[100];
    for (int i = 0; i < n; i++) {
        a[i] = i + 1;
```

```
    }
    return a;
}
```

11. Aşağıdaki fonksiyonun hangi çağrısında bellek sızıntısı oluşur?

```
int topla(int n) {
    int *a = malloc(n * sizeof *a);
    if (a == NULL) {
        return -1;
    }
    int toplam = 0;
    for (int i = 0; i < n; i++) {
        a[i] = i;
        toplam += a[i];
    }
    if (toplam > 100) {
        return 100;
    }
    free(a);
    return toplam;
}
```

    A) Hiçbir çağrıda sızıntı yoktur.
    B) `malloc` başarısız olursa.
    C) `toplam` 100'den büyükse.
    D) Her çağrıda.
    E) `n` sıfırsa.

12. Klavyeden 0 girilene kadar tamsayı okuyan `ortalamaUstu` programını yazın. Kaç sayı gireceği belli değildir; 0 diziye eklenmez. Sayıları `malloc` ile 2 elemanlık başlayan, dolunca `realloc` ile kapasitesi ikiye katlanan dinamik dizide saklayın. Okuma bitince sayıların ortalamasını ve ortalamadan büyük olanların adedini yazdırın. Hiç sayı girilmediyse `Sayı girilmedi.` yazıp çıkın. Bellek hatalarını denetleyin ve diziyi serbest bırakın.

```
Sayıları girin (0 ile biter): 10 20 30 40 50 60 0
Ortalama: 35.00
Ortalamadan büyük: 3
```

13. Ad ve soyadı okuyup yeni bir string olarak birleştiren `birlestir` programını yazın. `char *birlestir(const char *ad, const char *soyad)` fonksiyonu sonucu (ad, bir boşluk, soyad) `malloc` ile tam gereken kadar yer ayırarak oluştursun; yer yoksa `NULL` döndürsün. `main`, sonucu ve uzunluğunu yazdırsın, sonra belleği serbest bıraksın. Ad ve soyad en fazla 49 karakterdir ve boşluk içermez.

```
Ad Soyad: Elif Demir
Tam ad: Elif Demir (10 karakter)
```

14. Satır ve sütun sayısı klavyeden alınan bir tamsayı matrisini tek bir `malloc` bloğunda (`m[i * sutun + j]`) tutan, matrisin **devriğini** (transpozunu) ayrı bir bloğa hesaplayıp yazdıran `devrik` programını yazın. Bellek ayırma hatalarını denetleyin; her iki bloğu da serbest bırakın.

BÖLÜM 14

Yapılar: struct, union, enum ve typedef

BU BÖLÜMDE

- Paralel diziler yerine neden tek bir kayıt (`struct`) kullanıldığını
- `struct` tanımlamayı, `.` ile üyeye erişmeyi, ilk değer vermeyi (belirlenmiş başlatıcılar dahil) ve `typedef` ile kısa ad vermeyi
- Yapı atamasının tüm üyeleri kopyaladığını ve yapıların `==` ile karşılaştırılamadığını
- Yapı dizileriyle kayıt listesi okumayı, aramayı ve sıralamayı (`qsort` dahil)
- Fonksiyona yapı geçirmeyi (değerle ve işaretçiyle), `->` operatörünü ve `const` kullanımını
- İç içe yapıları, `malloc` ile dinamik yapı dizisini, `sizeof`, hizalamayı (padding) ve bit alanlarını
- `union` ile etiketli birleşimi; `enum` ile adlandırılmış sabitleri

Şimdiye kadar yazdığımız programlarda her değer ayrı bir değişkende ya da aynı tipten elemanlar tutan bir dizide durdu. Gerçek veri ise çoğunlukla karışık tiptedir: bir öğrencinin adı bir metindir, numarası bir tamsayı, not ortalaması ondalıklı bir sayı. Bu üçünü bir bütün olarak taşımak istediğimizde diziler yetmez.

Bu bölümde C'nin kendi tiplerimizi oluşturma araçlarını göreceğiz: farklı tipte verileri bir araya getiren **yapı** (`struct`), aynı belleği paylaşan alternatif verileri tutan **birleşim** (`union`), adlandırılmış sabit kümeleri kuran **sayım tipi** (`enum`) ve bunlara kısa ad veren `typedef`. Bölüm 12 ve 13'te öğrendiğimiz işaretçiler ve dinamik bellek, yapılarla birleşince veri yapılarının (Bölüm 19) ve dosya kayıtlarının (Bölüm 17) temelini oluşturur.

14.1 Neden Yapı?

Bir sınıfın öğrenci bilgilerini tutan bir program yazalım. Her öğrencinin adı, numarası ve ortalaması var. Şimdiye kadar öğrendiklerimizle üç ayrı dizi kullanırdık:

```
char ad[50][30];        // 50 öğrencinin adı
int no[50];             // numaraları
double ortalama[50];    // ortalamaları
```

Öğrenci `i`'nin bilgisi bu üç dizinin `i`. gözündedir. Böyle düzene **paralel diziler** (parallel arrays) denir ve iki sorunu vardır. Birincisi, öğrenciyle ilgili her işlem üç diziye birden dokunur: iki öğrencinin yerini değiştirmek üç ayrı takas ister (adlar için `strcpy` ve geçici bir metin, numaralar için geçici bir `int`, ortalamalar için geçici bir `double`). Birini unutursak bir öğrencinin adı başkasının numarasıyla kalır ve derleyici bunu fark etmez. İkincisi, bir öğrenciyi fonksiyona geçirmek üç parametre demektir.

> **Tanım: Yapı (struct)**
> Farklı tipten değişkenleri tek bir isim altında toplayan, programcının tanımladığı bileşik tiptir. Yapının içindeki değişkenlere **üye** (member) denir.

Yapıyla öğrenci tek bir değişken olur, takas tek satırdır, fonksiyona tek parametre yeter. Dizi aynı tipten elemanları art arda dizer; yapı ise farklı tipten üyeleri bir araya getirir. İkisini birleştirip yapı dizisi de kurabiliriz (birazdan göreceğiz).

14.2 Yapı Tanımı ve Üyelere Erişim

Yapıyı iki adımda kullanırız: önce yeni tipi tanıtırız, sonra o tipten değişken tanımlarız.

▸ ilkYapi.c

```c
#include <stdio.h>
#include <string.h>

struct Ogrenci {
    char ad[30];
    int no;
    double ortalama;
};

int main(void) {
    struct Ogrenci a;
    strcpy(a.ad, "Ayse Demir");
    a.no = 101;
    a.ortalama = 78.5;

    struct Ogrenci b = {"Mehmet Kaya", 102, 91.25};
    struct Ogrenci c = {.no = 103, .ad = "Elif Aydin"};

    printf("%s %d %.2f\n", a.ad, a.no, a.ortalama);
    printf("%s %d %.2f\n", b.ad, b.no, b.ortalama);
    printf("%s %d %.2f\n", c.ad, c.no, c.ortalama);
    return 0;
}
```

```
ÇIKTI
Ayse Demir 101 78.50
Mehmet Kaya 102 91.25
Elif Aydin 103 0.00
```

Programı satır satır okuyalım:

Kod	Anlamı
`struct Ogrenci { ... };`	Yeni bir tip tanıtır; tipin adı `struct Ogrenci` (iki sözcük). Bellek ayırmaz. İçindekiler yapının üyeleridir.
`struct Ogrenci a;`	Değişken tanımlar. Bellek şimdi ayrılır; üç üyenin hepsi `a` 'nın içindedir.
`a.no = 101;`	Nokta operatörüyle (`.`) üyeye erişir: "`a` 'nın `no` 'su".
`{"Mehmet Kaya", 102, 91.25}`	İlk değer listesi: değerler üyelerle tanım sırasına göre eşleşir.

Kod	Anlamı
`{.no = 103, .ad = "..."}`	Belirlenmiş başlatıcı (designated initializer, C99): üyeyi adıyla seçeriz, sıra önemli değildir. Değer verilmeyen üye sıfırlanır.

Bellekte `a` değişkeni, üyeleri tanım sırasıyla art arda içeren tek bir alandır:

```
 a (struct Ogrenci)
+--------------------------+
| ad       : char[30]      |
+--------------------------+
| no       : int           |
+--------------------------+
| ortalama : double        |
+--------------------------+
```

(Üyeler arasında boşluk baytları kalabilir; bunu "Bellek Yerleşimi ve Dolgu" başlığında göreceğiz.)

Üye, kendi tipinin sıradan bir değişkenidir: `a.no++` ve `scanf("%d", &a.no)` geçerlidir. Dizi üyesi için `&` gerekmez: `scanf("%29s", a.ad)`. Yapının tamamını sıfırlamak için `struct Ogrenci bos = {0};` yazılır; ilk üye ve ardından gelenlerin hepsi 0 olur.

İpucu · Belirlenmiş başlatıcı mı, konumlu liste mi?

Konumlu listeyi eksik bırakırsak (`{"Ayse"}`) `-Wextra` "missing initializer" uyarısı verir. Üyelerin yalnızca bir kısmına değer verecekseksek `.ad = "Ayse"` biçimini kullanmak hem uyarıyı önler hem de hangi üyenin ne aldığını gösterir.

▲ Sık Yapılan Hata · Tanımın sonundaki noktalı virgül

`struct` tanımı `;` ile biter. Unutursak derleyici hatayı `}` işaretinin yanında değil, sonraki satırda bildirir; çünkü `struct Nokta {...} int main(void)` ifadesi "bu yapı tipinden `main` adında bir şey tanımlıyorsun" diye okunur:

▸ structNokta.c

```c
#include <stdio.h>

struct Nokta {
    int x;
    int y;
}

int main(void) {
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
structNokta.c:8:1: error: expected ';', identifier or '(' before 'int'
    8 | int main(void) {
      | ^~~
```

Hata `int main` satırını gösteriyorsa bir üstteki yapının `}` işaretinden sonra `;` olup olmadığına bakmalıyız.

14.3 typedef ile Tip Takma Adı

Her yerde `struct Ogrenci` yazmak yorucudur. `typedef` mevcut bir tipe ikinci bir ad verir:

> **Tanım: typedef**
>
> Var olan bir tipe yeni bir ad (takma ad, alias) veren bildirimdir. Yeni bir tip yaratmaz; aynı tipin ikinci bir adını tanımlar.

Yapılarla üç kalıp görürüz:

Kalıp	Değişken tanımı
`struct Ogrenci { ... };`	`struct Ogrenci a;`
`typedef struct { ... } Ogrenci;`	`Ogrenci a;` (yapının etiketi yok, tek adı `Ogrenci`)
`typedef struct Ogrenci { ... } Ogrenci;`	`Ogrenci a;` ve `struct Ogrenci a;`

Kitapta çoğunlukla ikinci kalıbı kullanacağız. Üçüncüsü, yapı kendi türünden bir işaretçi taşıyacaksa şarttır; bağlı listeyi kurarken Bölüm 19'da göreceğiz. Tip adlarını büyük harfle başlatırız (`Ogrenci`), böylece `ogrenci` adlı bir değişkenle karışmazlar.

`typedef` yalnızca yapılara özgü değildir. Temel tiplere de anlamlı ad verebiliriz:

```
typedef unsigned char Bayt;
typedef double Para;

Para fiyat = 249.90;
```

▲ Sık Yapılan Hata · Etiketsiz typedef'ten sonra struct yazmak

`typedef struct { ... } Nokta;` biçiminde yapının etiketi yoktur, yalnızca `Nokta` adı vardır. Sonradan `struct Nokta c;` yazarsak C, `struct Nokta` adında başka, tanımsız bir tip arar:

▸ yanlisEtiket.c

```c
typedef struct {
    int x;
    int y;
} Nokta;

int main(void) {
    struct Nokta c;
    c.x = 1;
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
yanlisEtiket.c: In function 'main':
yanlisEtiket.c:7:18: error: storage size of 'c' isn't known
    7 |     struct Nokta c;
      |                  ^
yanlisEtiket.c:7:18: warning: unused variable 'c' [-Wunused-variable]
```

`unused variable` uyarısı asıl hatanın yan etkisidir. Çözüm: `Nokta c;` yazmak ya da yapıya etiket vermektir (`typedef struct Nokta {...} Nokta;`).

14.4 Atama ve Karşılaştırma

Aynı tipten iki yapı `=` ile atanabilir ve atama bütün üyeleri kopyalar. Bölüm 10'da dizilerin `=` ile kopyalanamadığını görmüştük; yapının *içindeki* dizi üyesi ise yapıyla birlikte kopyalanır:

▸ yapiAtama.c

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    char ad[30];
    int no;
} Ogrenci;

int main(void) {
    Ogrenci a = {"Ayse", 101};
    Ogrenci b = a;              // b, a'nın üye üye kopyası

    strcpy(b.ad, "Zeynep");
    b.no = 102;

    printf("a: %s %d\n", a.ad, a.no);
    printf("b: %s %d\n", b.ad, b.no);
    return 0;
}
```

```
ÇIKTI
a: Ayse 101
b: Zeynep 102
```

`b`, `a`'nın bağımsız bir kopyasıdır; `b.ad`'ı değiştirmek `a.ad`'ı etkilemez. İşaretçi üyeler ise yalnızca adresi kopyalar, yani iki yapı aynı belleği paylaşır. Bu ayrıntı Bölüm 19'da önem kazanacak.

İki yapıyı `==` ile karşılaştıramayız; C bu işlemi tanımlamamıştır. Üyeleri tek tek karşılaştırırız (metin üyeleri `strcmp` ile). Metin üyeye `=` ile metin de atanamaz; Bölüm 11'deki kural aynen geçerlidir:

▸ yapiHatalari.c

```c
typedef struct {
    char ad[30];
    int no;
} Ogrenci;

int main(void) {
    Ogrenci a = {"Ayse", 101};
    Ogrenci b = {"Ayse", 101};

    if (a == b) {
        a.no = 0;
    }
    a.ad = "Zeynep";
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI
yapiHatalari.c: In function 'main':
yapiHatalari.c:10:11: error: invalid operands to binary == (have 'Ogrenci' and 'Ogrenci')
   10 |     if (a == b) {
      |           ^~
yapiHatalari.c:13:10: error: assignment to expression with array type
   13 |     a.ad = "Zeynep";
      |          ^
```

Doğrusu `strcmp(a.ad, b.ad) == 0 && a.no == b.no` ile karşılaştırmak ve `strcpy(a.ad, "Zeynep")` ile kopyalamaktır.

> ◆ **Kritik Nokta · Yapı atamasında dizi üyeler de kopyalanır**
>
> `Ogrenci b = a;` ifadesinden sonra `b`'deki her değişiklik `a`'ya yansımaz; çünkü iki yapı ayrı bellek alanlarıdır. Dizilerin `=` ile kopyalanamaması kuralı, yapı üyesi olan dizileri bağlamaz.

14.5 Yapı Dizileri

Yapıların asıl gücü diziye dizildiklerinde ortaya çıkar. `Ogrenci liste[50];` elli kayıt tutar; `liste[i]` bir kayıt, `liste[i].no` o kaydın numarasıdır. Aşağıdaki program öğrencileri okur, tablo yazar, en yüksek ortalamayı bulur ve ada göre arar:

▸ ogrenciListe.c

```c
#include <stdio.h>
#include <string.h>
```

```c
#define MAX 50

typedef struct {
    char ad[30];
    int no;
    double ortalama;
} Ogrenci;

int enYuksek(const Ogrenci liste[], int n) {
    int enIyi = 0;
    for (int i = 1; i < n; i++) {
        if (liste[i].ortalama > liste[enIyi].ortalama) {
            enIyi = i;
        }
    }
    return enIyi;
}

int adaGoreBul(const Ogrenci liste[], int n, const char *aranan) {
    for (int i = 0; i < n; i++) {
        if (strcmp(liste[i].ad, aranan) == 0) {
            return i;
        }
    }
    return -1;
}

int main(void) {
    Ogrenci liste[MAX];
    int n = 0;
    char aranan[30];

    if (scanf("%d", &n) != 1 || n < 1 || n > MAX) {
        return 1;
    }
    for (int i = 0; i < n; i++) {
        scanf("%29s %d %lf", liste[i].ad, &liste[i].no,
              &liste[i].ortalama);
    }
    scanf("%29s", aranan);

    for (int i = 0; i < n; i++) {
        printf("%d %-6s %6.2f\n", liste[i].no, liste[i].ad,
               liste[i].ortalama);
    }
    int en = enYuksek(liste, n);
    printf("En yüksek: %s (%.2f)\n", liste[en].ad, liste[en].ortalama);

    int yer = adaGoreBul(liste, n, aranan);
    if (yer >= 0) {
        printf("%s bulundu: no %d\n", aranan, liste[yer].no);
    } else {
        printf("%s listede yok\n", aranan);
    }
    return 0;
}
```

```
ÇIKTI
4
Ayse 101 72.5
Mehmet 102 91.25
Can 103 64
Elif 104 85.75
Can
101 Ayse    72.50
102 Mehmet  91.25
103 Can     64.00
104 Elif    85.75
En yüksek: Mehmet (91.25)
Can bulundu: no 103
```

Dikkat edilecek birkaç nokta var:

- `liste[i].ad` için `&` yazmadık (dizi adı zaten adrestir), `liste[i].no` için yazdık.
- `liste[i]` bir yapıdır, işaretçi değildir; bu yüzden `.` kullanırız. `liste[i]->ad` derlenmez.
- En büyüğü bulma ve doğrusal arama algoritmaları Bölüm 10'dakilerle aynıdır; yalnızca karşılaştırılan şey `liste[i].ortalama` ya da `strcmp(liste[i].ad, ...)` olur.
- `%-6s` metni 6 genişlikte sola yaslar. Genişlik bayt sayar; bu yüzden tablolarda ASCII adlar kullanıyoruz (Bölüm 11'deki UTF-8 uyarısı).

Yapı dizisini sıralamak

Kabarcık sıralaması (Bölüm 10) yapılarla aynen çalışır. Fark, karşılaştırılan üyenin `.ortalama` olması ve takasın bütün kaydı tek seferde taşımasıdır. Paralel dizilerdeki üç takas, tek bir yapı takasına iner:

▸ kabarcikYapi.c

```c
#include <stdio.h>

typedef struct {
    char ad[20];
    int no;
    double ortalama;
} Ogrenci;

int main(void) {
    Ogrenci liste[5] = {
        {"Ayse", 101, 72.5}, {"Mehmet", 102, 91.25}, {"Can", 103, 64.0},
        {"Elif", 104, 85.75}, {"Mert", 105, 91.25}
    };
    int n = 5;

    for (int tur = 0; tur < n - 1; tur++) {
        for (int j = 0; j < n - 1 - tur; j++) {
            if (liste[j].ortalama < liste[j + 1].ortalama) {  // büyükten küçüğe
                Ogrenci gecici = liste[j];
                liste[j] = liste[j + 1];
                liste[j + 1] = gecici;
            }
        }
    }

    for (int i = 0; i < n; i++) {
        printf("%d. %-6s %.2f\n", i + 1, liste[i].ad, liste[i].ortalama);
```

```
    }
    return 0;
}
```

```
ÇIKTI
1. Mehmet 91.25
2. Mert   91.25
3. Elif   85.75
4. Ayse   72.50
5. Can    64.00
```

Mehmet ile Mert'in ortalaması eşittir. Eşitlikte takas yapmadığımız için (`<` kullandık, `<=` değil) aralarındaki ilk sıra korunur. Eşit elemanların sırasını koruyan sıralamaya **kararlı** (stable) denir.

> **İpucu · Yapıyı tek parça takas edin**
>
> Üyeleri tek tek takas etmeye kalkmak, biri eksik kaldığında kaydı bozar. `Ogrenci gecici = liste[j];` bütün kaydı taşır.

Yapı dizisini kütüphanenin `qsort` fonksiyonuyla da sıralayabiliriz. `qsort`'u ve fonksiyon işaretçilerini Bölüm 12'de görmüştük. Karşılaştırma fonksiyonu iki elemanın adresini `const void *` olarak alır; bunu `const Ogrenci *`'a atayıp `->` ile üyelere ulaşırız (`->` operatörünü hemen sonraki başlıkta açıklıyoruz). Ortalamaya göre büyükten küçüğe, eşitlikte ada göre sıralayalım:

```c
int ortalamayaGore(const void *x, const void *y) {
    const Ogrenci *a = x;
    const Ogrenci *b = y;
    if (a->ortalama > b->ortalama) return -1;   // a önce gelsin
    if (a->ortalama < b->ortalama) return 1;
    return strcmp(a->ad, b->ad);                // eşitlikte ada göre
}

// kullanım:
qsort(liste, n, sizeof liste[0], ortalamayaGore);
```

> **Not · qsort kararlı değildir**
>
> `qsort` eşit elemanların sırasını korumayı garanti etmez; sonuç kütüphaneye göre değişebilir. Çıktının her platformda aynı olması için eşitliği, yukarıdaki gibi ikinci bir anahtarla (burada `ad`) bozmak gerekir.

14.6 Yapılar ve Fonksiyonlar

C'de her argüman değerle geçer (Bölüm 9). Yapıyı doğrudan verirsek bütün yapı kopyalanır ve fonksiyon kopya üzerinde çalışır; orijinali değiştirmek için adresini geçirmemiz gerekir (Bölüm 12). Fonksiyon yapıyı `return` ile de döndürebilir. Yapı kopya olarak döner; bu yüzden yerel değişkenin adresini döndürmenin sorunu (Bölüm 12) burada yoktur.

▸ degerAdres.c

```c
#include <stdio.h>

typedef struct {
    char ad[20];
    int no;
```

```c
    double ortalama;
} Ogrenci;

Ogrenci yeniOgrenci(const char *ad, int no, double ortalama) {
    Ogrenci o;
    snprintf(o.ad, sizeof o.ad, "%s", ad);
    o.no = no;
    o.ortalama = ortalama;
    return o;                       // kopya olarak döner
}

void zamDeger(Ogrenci o) {
    o.ortalama += 10;
    printf("zamDeger içinde: %.1f\n", o.ortalama);
}

void zamAdres(Ogrenci *p) {
    p->ortalama += 10;
    printf("zamAdres içinde: %.1f\n", p->ortalama);
}

int main(void) {
    Ogrenci a = yeniOgrenci("Ayse", 101, 60.0);
    zamDeger(a);
    printf("dışarıda: %.1f\n", a.ortalama);
    zamAdres(&a);
    printf("dışarıda: %.1f\n", a.ortalama);
    return 0;
}
```

```
ÇIKTI
zamDeger içinde: 70.0
dışarıda: 60.0
zamAdres içinde: 70.0
dışarıda: 70.0
```

`zamDeger` kopyayı değiştirdi, dışarıdaki `a` aynı kaldı. `zamAdres` ise gerçek `a`'yı değiştirdi. `yeniOgrenci` içindeki `snprintf`, Bölüm 11'de gördüğümüz güvenli kopyalamadır: uzun adı keser, diziyi taşırmaz.

Ok operatörü

İşaretçi üzerinden üyeye `(*p).no` biçiminde erişiriz. Parantez şarttır, çünkü `.` operatörünün önceliği `*`'ınkinden yüksektir. Bu kalıp çok sık yazıldığı için C kısa bir yol sunar: **ok operatörü** (arrow operator) `->`.

Yazım	Anlamı
`p->no`	`p`'nin gösterdiği yapının `no` üyesi
`(*p).no`	Aynısı, parantezli uzun yazım
`*p.no`	`*(p.no)` demektir; `p` işaretçi olduğu için derlenmez

Hangisini kullanacağımızı soldaki ifade belirler: yapıysa `.`, yapıya işaretçiyse `->`. Örneğin `a.no` (yapı değişkeni), `liste[i].no` (dizi elemanı yapıdır), `p->no` (işaretçi), `isaretciler[i]->no` (işaretçi dizisinin elemanı işaretçidir).

const ile büyük yapıyı geçirmek

Yapı büyükse her çağrıda kopyalamak israftır; içinde büyük diziler olan bir yapı yüzlerce bayt tutabilir. Adres geçirmek hızlıdır. Fonksiyonun yapıyı değiştirmeyeceğini garanti etmek için `const` yazarız (Bölüm 12). Aşağıdaki programda iki hatayı birlikte görelim:

▸ okVeConst.c

```c
#include <stdio.h>

typedef struct {
    char ad[20];
    int no;
} Ogrenci;

void yazdir(const Ogrenci *o) {
    printf("%d %s\n", o->no, o->ad);
    o->no = 0;
}

int main(void) {
    Ogrenci a = {"Ayse", 101};
    Ogrenci *p = &a;
    printf("%c\n", *p.ad);
    yazdir(p);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
okVeConst.c: In function 'yazdir':
okVeConst.c:10:11: error: assignment of member 'no' in read-only object
   10 |     o->no = 0;
      |           ^
okVeConst.c: In function 'main':
okVeConst.c:16:22: error: 'p' is a pointer; did you mean to use '->'?
   16 |     printf("%c\n", *p.ad);
      |                     ^
      |                     ->
```

`*p.ad` ifadesinde `.` önce işlendiği için `p.ad` aranır; doğrusu `p->ad` ya da `(*p).ad`'dır. İkinci hata, salt okunur yapıya yazmaktır. Kural basittir: yalnızca okuyan fonksiyon `const Ogrenci *`, değiştiren fonksiyon `Ogrenci *` alır.

> ◆ **Kritik Nokta · Yapıyı değerle mi, işaretçiyle mi geçirmeli?**
>
> Küçük yapılar (iki üç sayı) değerle geçirilebilir. Büyük yapılar ya da fonksiyonun orijinali değiştirmesi gereken durumlar için işaretçi kullanılır; yalnızca okuma yapılacaksa `const` eklenir.

14.7 İç İçe Yapılar

Bir yapının üyesi başka bir yapı olabilir. İç yapıya erişirken noktalar art arda gelir: `o.dogum.yil`. İç yapının ilk değer listesi de iç içe süslü parantez alır.

icIceYapi.c

```c
#include <stdio.h>

typedef struct {
    int gun;
    int ay;
    int yil;
} Tarih;

typedef struct {
    char ad[20];
    int no;
    Tarih dogum;
} Ogrenci;

int main(void) {
    Ogrenci o = {"Ayse", 101, {7, 3, 2005}};
    Ogrenci *p = &o;

    o.dogum.yil = 2004;
    p->dogum.ay = 11;

    printf("%s: %02d.%02d.%d\n", o.ad, o.dogum.gun, o.dogum.ay,
           o.dogum.yil);
    return 0;
}
```

ÇIKTI

```
Ayse: 07.11.2004
```

İfadeyi soldan sağa okuyalım: `p->dogum.ay`, "`p`'nin gösterdiği yapının `dogum` üyesinin `ay`'ı" demektir. `->` yalnızca ilk adımdır; `dogum` bir yapı olduğu için ikinci adımı yine `.` ile atarız.

Bir yapı, kendi türünden bir üye taşıyamaz; boyutu sonsuz olurdu. Ama kendi türünden bir **işaretçi** taşıyabilir. Bağlı liste ve ağaç gibi yapılar böyle kurulur; Bölüm 19'da kullanacağız.

14.8 Yapılar ve Dinamik Bellek

`malloc` ile yapı için bellek ayırmak diğer tiplerden farksızdır (Bölüm 13). Boyutu `sizeof(Ogrenci)` ya da `sizeof *p` ile veririz; sonuç `Ogrenci *` tipindedir. Öğrenci sayısını çalışırken öğrenen bir program `n * sizeof *liste` ile yapı dizisi ayırır:

dinamikOgrenci.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char ad[20];
    int no;
    double ortalama;
} Ogrenci;

int main(void) {
    int n = 0;
    if (scanf("%d", &n) != 1 || n < 1) {
        return 1;
    }
```

```
    Ogrenci *liste = malloc(n * sizeof *liste);   // n kayıtlık dizi
    if (liste == NULL) {
        return 1;
    }

    double toplam = 0;
    for (int i = 0; i < n; i++) {
        scanf("%19s %d %lf", liste[i].ad, &liste[i].no,
              &liste[i].ortalama);
        toplam += liste[i].ortalama;
    }
    printf("Ortalama: %.2f\n", toplam / n);

    free(liste);
    return 0;
}
```

```
ÇIKTI
3
Ayse 101 72.5
Mehmet 102 91.25
Can 103 64
Ortalama: 75.92
```

Tek bir kayıt için `Ogrenci *tek = malloc(sizeof(Ogrenci));` yazar, üyeye `tek->no` ile erişiriz. `liste` de bir işaretçidir ama `liste[i]` yapının kendisidir (`liste[i]` ile `*(liste + i)` eşdeğerdir), bu yüzden `liste[i].ad` yazarız. `sizeof *liste` kullanmanın yararı, tip değişirse bu satıra dokunmak gerekmemesidir. `sizeof` ifadeyi değerlendirmez, yalnızca tipinin boyutunu alır. Her `malloc` için bir `free` yazmayı unutmayalım.

14.9 Bellek Yerleşimi ve Dolgu

Üyelerin boyutlarını toplayarak yapının boyutunu bulmaya çalışırsak yanılırız: derleyici üyeler arasına boşluk baytları (**padding**, dolgu) koyar. İşlemci, adresi 4'ün katı olan bir `int`'i ve 8'in katı olan bir `double`'ı tek hamlede okuyabilir. Bu yüzden her üye kendi **hizalama** (alignment) gereksinimine uyan bir adreste başlar ve yapının toplam boyutu en büyük hizalamanın katına yuvarlanır.

▸ hizalama.c

```
#include <stdio.h>
#include <stddef.h>

struct A { char c; int i; };
struct B { char c1; int i; char c2; };
struct C { int i; char c1; char c2; };

typedef struct {
    char ad[30];
    int no;
    double ortalama;
} Ogrenci;

int main(void) {
    printf("struct A: %zu bayt, i %zu. bayttan başlar\n",
           sizeof(struct A), offsetof(struct A, i));
    printf("struct B: %zu bayt\n", sizeof(struct B));
    printf("struct C: %zu bayt\n", sizeof(struct C));
```

```
    printf("Ogrenci : %zu bayt\n", sizeof(Ogrenci));
    printf("no %zu, ortalama %zu\n", offsetof(Ogrenci, no),
           offsetof(Ogrenci, ortalama));
    return 0;
}
```

```
ÇIKTI
struct A: 8 bayt, i 4. bayttan başlar
struct B: 12 bayt
struct C: 8 bayt
Ogrenci : 48 bayt
no 32, ortalama 40
```

`offsetof(tip, üye)` (`<stddef.h>`), üyenin yapının başından kaç bayt sonra başladığını verir. Sayıları elle şöyle buluruz:

- `A`: `c` 0. baytta; `i` 4'ün katında başlamalı, 1-3. baytlar boş kalır: 1 + 3 + 4 = 8.
- `B`: `c1` (1) + 3 boşluk + `i` (4) + `c2` (1) = 9; en büyük hizalama olan 4'ün katına yuvarlanır: 12.
- `C`: `i` (4) + `c1` (1) + `c2` (1) = 6; 4'ün katına yuvarlanır: 8. `B` ile aynı üyeler, ama büyükten küçüğe dizildi.
- `Ogrenci`: `ad` 0-29, 30-31 boşluk, `no` 32'de, 36-39 boşluk, `ortalama` 40'ta: toplam 48. Üyelerin boyutlarının toplamı ise 30 + 4 + 8 = 42'dir.

```
Ogrenci (48 bayt)
0                               30 32      36    40            48
+-------------------------------+--+-------+-----+-------------+
|            ad[30]             |//|  no   |/////|  ortalama   |
+-------------------------------+--+-------+-----+-------------+
                                 //  boşluk (padding)
```

Bu sayılar 64 bitlik gcc için geçerlidir. C standardı yapı boyutunu garanti etmez; `double`'ın 4 bayta hizalandığı 32 bitlik bir derleyicide `Ogrenci` 44 bayt olabilir. Boşluk baytlarının içeriği belirsizdir; bu yüzden iki yapıyı `memcmp` ile karşılaştırmayız, üyeleri tek tek karşılaştırırız.

> ◆ **Kritik Nokta · Dolgu hesabı**
>
> Her üyeyi sırayla yerleştirir, kendi hizalamasına uyan adrese getirir (gerekirse boşluk ekler) ve en sonda toplamı en büyük hizalamanın katına yuvarlarız. Örneğin `struct { char a; double b; char c; }` için 1 + 7 (boşluk) + 8 + 1 + 7 (kuyruk) = 24 bayttır; `struct { double b; char a; char c; }` ise 8 + 1 + 1 + 6 = 16 bayttır. Üyeleri büyükten küçüğe dizmek boşluğu azaltır.

Bit alanları

Bazen bir üyenin tam bir `int` kaplamasına gerek yoktur: ayın günü 1 ile 31 arasındadır, 5 bit yeter. Üye adından sonra `: bitSayisi` yazarsak üye yalnızca o kadar bit kaplar. Buna **bit alanı** (bit-field) denir. Tipi her zaman `unsigned int` yazarız.

▸ bitAlani.c

```
#include <stdio.h>

struct NormalTarih {
    int gun;
    int ay;
    int yil;
};
```

```c
struct SikTarih {
    unsigned int gun : 5;     // 0 ... 31
    unsigned int ay : 4;      // 0 ... 15
    unsigned int yil : 12;    // 0 ... 4095
};

int main(void) {
    struct SikTarih t = {9, 10, 2026};
    unsigned int fazla = 33;

    printf("%u/%u/%u\n", t.gun, t.ay, t.yil);
    printf("normal: %zu bayt, sık: %zu bayt\n",
           sizeof(struct NormalTarih), sizeof(struct SikTarih));
    t.gun = fazla;            // 5 bit'e sığmaz
    printf("gun = %u\n", t.gun);
    return 0;
}
```

```
ÇIKTI
9/10/2026
normal: 12 bayt, sık: 4 bayt
gun = 1
```

Bit alanı kullanırken şunları bilmeliyiz:

- 5 + 4 + 12 = 21 bit tek bir 4 baytlık birime sığar; boyut 12'den 4'e iner. Bitlerin hangi sırayla dizildiği derleyiciye bağlıdır, bu yüzden bit alanlı yapıyı olduğu gibi dosyaya ya da ağa yazmayız.
- Sığmayan değerin üst bitleri atılır: 5 bit en çok 31 tutar, 33 yazınca 33 % 32 = 1 kalır.
- `unsigned` yazmalıyız. Düz `int` bit alanının işaretli mi işaretsiz mi olduğunu standart derleyiciye bırakır.
- Bit alanının adresi alınamaz: bellekte baytlar adreslenir, bitler değil. `&t.gun` ve `scanf("%u", &t.gun)` derlenmez. Önce sıradan bir değişkene okuyup sonra atarız.

▸ bitAlaniAdres.c

```c
struct Bayrak {
    unsigned int acik : 1;
};

int main(void) {
    struct Bayrak b = {1};
    unsigned int *p = &b.acik;      // hata: bit alanının adresi alınamaz
    return (int)*p;
}
```

```
DERLEYİCİ ÇIKTISI

bitAlaniAdres.c: In function 'main':
bitAlaniAdres.c:7:23: error: cannot take address of bit-field 'acik'
    7 |     unsigned int *p = &b.acik;      // hata: bit alanının adresi alınamaz
      |                       ^
```

Bitleri maskeler ve kaydırma işleçleriyle elle yönetmeyi Bölüm 16'da göreceğiz; çoğu durumda bu yöntem bit alanlarından daha taşınabilirdir.

Üç Dilde · C++ ve C#'ta yapı *C · C++ · C#*

C'de yapı yalnızca veri taşır; işlevleri ayrı fonksiyonlara yazarız. İleride göreceğimiz C++ ve C#'ta yapı, davranış da taşıyabilir.

C++'ta `struct` ile `class` neredeyse aynıdır. Tek fark varsayılan erişimdir: `struct`'ta üyeler `public`, `class`'ta `private`'tır. `struct` üye fonksiyonlar taşıyabilir ve değişken tanımlarken `struct` sözcüğünü yazmak gerekmez:

```
struct Ogrenci {
    std::string ad;
    int no;
    double ortalama;
    void yazdir() const { std::cout << no << " " << ad << "\n"; }
};

Ogrenci a{"Ayse", 101, 78.5};   // struct sözcüğü gerekmez
a.yazdir();
```

C#'ta `struct` bir **değer tipidir**: atamada C'deki gibi kopyalanır. `class` ise **başvuru tipidir**: atamada yalnızca başvuru (adres) kopyalanır, iki değişken aynı nesneyi gösterir.

```
struct Nokta { public int X, Y; }
class Kisi { public string Ad = ""; }

var n1 = new Nokta { X = 1, Y = 2 };
var n2 = n1;  n2.X = 9;          // n1.X hâlâ 1 (kopya)

var k1 = new Kisi { Ad = "Ayse" };
var k2 = k1;  k2.Ad = "Zeynep";  // k1.Ad de "Zeynep" (aynı nesne)
```

14.10 union

`union` (birleşim) yapıya benzer, ama üyelerini art arda değil, **aynı bellek alanında üst üste** saklar. Boyutu en büyük üyenin boyutu kadardır (gerekirse hizalamaya göre yuvarlanır) ve aynı anda yalnızca bir üye anlamlıdır: en son yazdığımız.

Tanım: union

Bütün üyelerinin belleğin aynı yerinden başladığı bileşik tiptir. Bir `union` değişkeni, üyelerinden yalnızca birini tutabilir.

▸ birlesim.c

```
#include <stdio.h>

union Deger {
    int tam;
    double ondalik;
    char harf;
};

int main(void) {
    union Deger d;
    printf("boyut: %zu bayt\n", sizeof d);
```

```
    d.tam = 65;
    printf("tam = %d\n", d.tam);

    d.ondalik = 2.5;              // tam'ın yerini ezer
    printf("ondalik = %.1f\n", d.ondalik);

    d.harf = 'Z';                 // ondalik'in ilk baytını ezer
    printf("harf = %c\n", d.harf);
    return 0;
}
```

```
ÇIKTI
boyut: 8 bayt
tam = 65
ondalik = 2.5
harf = Z
```

```
 union Deger (8 bayt)
+---------------------------------+
| tam      : ilk 4 bayt           |
| ondalik  : 8 bayt (tamamı)      |
| harf     : ilk 1 bayt           |
+---------------------------------+
  Üçü de belleğin aynı başlangıcını kullanır.
```

Boyut 8'dir: `int` (4), `double` (8) ve `char` (1) üyelerinin en büyüğü; toplamları olan 13 değil. Her yazma, öncekini ezer.

> **▲ Sık Yapılan Hata · Yazılmayan üyeyi okumak**
>
> `d.ondalik = 2.5;` yazıp `d.tam` okursak aynı baytları `int` olarak yorumlarız. Çıkan sayı anlamlı olmaz ve değeri makineye göre değişir. Hangi üyenin geçerli olduğunu kendimiz bilmeli ve saklamalıyız; standart yol, aşağıdaki etiketli birleşimdir.

Bellek baytlarına bakmak için de `union` kullanılır. Aşağıda 32 bitlik bir sayıyı, aynı alanı paylaşan bir bayt dizisiyle birlikte tanımlıyoruz:

▸ baytGorunumu.c

```c
#include <stdio.h>
#include <stdint.h>

union Gorunum {
    uint32_t sayi;
    unsigned char bayt[4];
};

int main(void) {
    union Gorunum g;
    g.sayi = 0x01020304;
    for (int i = 0; i < 4; i++) {
        printf("bayt[%d] = %02x\n", i, g.bayt[i]);
    }
    return 0;
}
```

```
ÇIKTI
bayt[0] = 04
bayt[1] = 03
bayt[2] = 02
bayt[3] = 01
```

Düşük anlamlı bayt önce geliyor: bu makine küçük sonlu (little-endian) düzenindedir (Bölüm 5). Büyük sonlu bir makinede çıktı ters sırada olurdu. Yani bu program belleğin düzenini gösteren bir araçtır, ama taşınabilir sonuç beklememeliyiz.

Etiketli birleşim

Bir değişken bazen tamsayı, bazen ondalıklı sayı taşıyacaksa `union`'ı, hangi üyenin geçerli olduğunu söyleyen bir **etiketle** (tag) birlikte bir yapıya koyarız. Etiket olarak `enum` uygundur (sonraki başlıkta ayrıntılı göreceğiz; şimdilik "adlandırılmış tamsayı sabitleri" diyelim):

▸ etiketliBirlesim.c

```c
#include <stdio.h>

typedef enum { TUR_TAM, TUR_ONDALIK } Tur;

typedef struct {
    Tur tur;                    // hangi üye geçerli?
    union {
        int tam;
        double ondalik;
    } deger;
} Sayi;

int main(void) {
    Sayi liste[2] = {
        {TUR_TAM, {.tam = 42}},
        {TUR_ONDALIK, {.ondalik = 3.14159}}
    };

    for (int i = 0; i < 2; i++) {
        switch (liste[i].tur) {
            case TUR_TAM:
                printf("tam: %d\n", liste[i].deger.tam);
                break;
            case TUR_ONDALIK:
                printf("ondalık: %.2f\n", liste[i].deger.ondalik);
                break;
        }
    }
    return 0;
}
```

```
ÇIKTI
tam: 42
ondalık: 3.14
```

İki farklı tipte değer aynı dizide durdu; `switch` etiketi okuyup yalnızca geçerli üyeye baktı. Etiketli birleşim, başka dillerdeki "toplam tip" (sum type) fikrinin C'deki karşılığıdır. Örneğin bir yorumlayıcının değerleri, bir ağ mesajının çeşitli biçimleri böyle modellenir.

14.11 enum

Programlarda gün, renk, durum gibi sınırlı sayıda seçenek bulunur. Bunları `1`, `2`, `3` gibi sihirli sayılarla taşımak okunamaz; hepsi için ayrı `#define` yazmak ise sıkıcıdır. `enum` (enumeration, sayım tipi), bir sabit kümesine ad verir.

> **Tanım: enum**
>
> Adlandırılmış tamsayı sabitlerinden oluşan bir tiptir. Sabitler, değer verilmezse 0'dan başlayarak birer birer artar.

▸ gunler.c

```c
#include <stdio.h>

enum Gun { PAZARTESI, SALI, CARSAMBA, PERSEMBE, CUMA, CUMARTESI, PAZAR };

typedef enum {
    DUSUK = 1,
    ORTA,
    YUKSEK = 10,
    COK_YUKSEK
} Seviye;

int main(void) {
    const char *gunAdi[] = {"Pazartesi", "Salı", "Çarşamba", "Perşembe",
                            "Cuma", "Cumartesi", "Pazar"};

    enum Gun bugun = CUMARTESI;
    printf("SALI = %d, PAZAR = %d\n", SALI, PAZAR);
    printf("bugun = %d (%s)\n", bugun, gunAdi[bugun]);

    switch (bugun) {
        case CUMARTESI:
        case PAZAR:
            printf("Hafta sonu\n");
            break;
        default:
            printf("Hafta içi\n");
    }

    printf("%d %d %d %d\n", DUSUK, ORTA, YUKSEK, COK_YUKSEK);
    Seviye s = ORTA;
    s = s + 1;                  // enum bir int gibi toplanır
    printf("s = %d, s == YUKSEK? %d\n", s, s == YUKSEK);
    return 0;
}
```

```
ÇIKTI
SALI = 1, PAZAR = 6
bugun = 5 (Cumartesi)
Hafta sonu
1 2 10 11
s = 3, s == YUKSEK? 0
```

Kuralları sırayla çıkaralım:

- Sabitler 0'dan başlar: `PAZARTESI` 0, `SALI` 1, … `PAZAR` 6.
- Bir sabite açık değer verebiliriz; sonraki sabit, değer verilmediği sürece bir öncekinin bir fazlasıdır: `ORTA` = 2, `COK_YUKSEK` = 11. İki sabite aynı değer de verilebilir.

- Sabitler `int` sabitleridir: `printf("%d", SALI)` çalışır, dizi indeksi (`gunAdi[bugun]`) ve `case` etiketi olabilir, toplanabilir. `s + 1`'den sonra `s` 3'tür; hiçbir sabitle eşleşmez ama C itiraz etmez.
- Sabit adları büyük harfle yazılır ve değişken adlarıyla aynı ad alanını paylaşır. İki farklı `enum` içinde aynı sabit adı (`KIRMIZI`) kullanılamaz.
- Standart, `enum` değişkeninin aralığını denetlemez: `enum Gun g = 42;` geçerlidir.

> **İpucu · enum mu, #define mı?**
>
> Birbirine bağlı sabitler (günler, durumlar, hata kodları) için `enum` kullanırız: sabitleri derleyici tanır, hata ayıklayıcıda adlarıyla görünürler ve `-Wall` ile bir `switch`'te eksik kalan sabit için `enumeration value 'PAZAR' not handled in switch` uyarısı alırız. Tek tük sabit (`MAX` gibi) için `#define` (Bölüm 5, Bölüm 15) ya da `const` yeterlidir.

`enum` değerini ada çevirmek için sık kullanılan kalıp, bir dizi ya da `switch`'tir. Dizinin indeksi `enum` sabitiyle örtüşüyorsa dizi yeterlidir (yukarıdaki `gunAdi`). Sabitler düzensizse `switch` daha güvenlidir:

▸ durumAdi.c

```c
#include <stdio.h>

typedef enum { BEKLIYOR = 1, HAZIRLANIYOR = 5, TESLIM = 9 } Durum;

const char *durumAdi(Durum d) {
    switch (d) {
        case BEKLIYOR:     return "Bekliyor";
        case HAZIRLANIYOR: return "Hazırlanıyor";
        case TESLIM:       return "Teslim edildi";
    }
    return "Bilinmiyor";
}

int main(void) {
    Durum d = HAZIRLANIYOR;
    printf("%s\n", durumAdi(d));
    printf("%s\n", durumAdi(TESLIM));
    printf("%s\n", durumAdi((Durum)7));
    return 0;
}
```

ÇIKTI

```
Hazırlanıyor
Teslim edildi
Bilinmiyor
```

Aralığı denetlenmeyen `enum`'a dışarıdan gelen bir sayı atanabildiği için fonksiyonun sonunda varsayılan bir dönüş değeri bırakmak iyi bir alışkanlıktır.

> **Üç Dilde · C++ ve C#'ta enum** *C · C++ · C#*
>
> **C'de** `enum` sabitleri `int` gibi davranır ve adları kapsamı kirletir. İleride göreceğimiz C++ ve C#'ta bu sorunlar giderilmiştir.
>
> **C++'ta** `enum class` (C++11), sabitleri tipin kapsamına alır ve örtük `int` dönüşümünü kapatır:

```
enum class Renk { Kirmizi, Yesil, Mavi };

Renk r = Renk::Yesil;            // Renk:: ile erişilir
// int n = r;                    // hata: örtük dönüşüm yok
int n = static_cast<int>(r);     // açık dönüşüm: 1
```

C#'ta `enum` her zaman adıyla erişilen güçlü bir tiptir ve `ToString()` değeri ada çevirir; böylece `durumAdi` gibi fonksiyonlara gerek kalmaz:

```
enum Durum { Bekliyor = 1, Hazirlaniyor = 5, Teslim = 9 }

Durum d = Durum.Hazirlaniyor;
Console.WriteLine(d);            // Hazirlaniyor
Console.WriteLine((int)d);       // 5
```

14.12 Uygulama: Sınıf Kayıt Sistemi

Öğrendiklerimizi küçük bir kayıt sisteminde birleştirelim. Sınıf, en çok `MAX` öğrenci tutan bir dizi ile bir sayaçtan oluşur. İkisini tek yapıda toplarsak fonksiyonlara tek parametre (`Sinif *`) yeter. `ekle` fonksiyonu yer varsa 1, doluysa 0 döndürür.

▸ sinifKayit.c

```
#include <stdio.h>

#define MAX 3

typedef struct {
    char ad[20];
    int no;
    double ortalama;
} Ogrenci;

typedef struct {
    Ogrenci liste[MAX];
    int adet;
} Sinif;

int ekle(Sinif *s, const char *ad, int no, double ortalama) {
    if (s->adet == MAX) {
        return 0;
    }
    Ogrenci *o = &s->liste[s->adet];
    snprintf(o->ad, sizeof o->ad, "%s", ad);
    o->no = no;
    o->ortalama = ortalama;
    s->adet++;
    return 1;
}

void listele(const Sinif *s) {
    for (int i = 0; i < s->adet; i++) {
        const Ogrenci *o = &s->liste[i];
        printf("%d. %-6s %d %6.2f\n", i + 1, o->ad, o->no, o->ortalama);
    }
}

double sinifOrtalamasi(const Sinif *s) {
```

```
    double toplam = 0;
    for (int i = 0; i < s->adet; i++) {
        toplam += s->liste[i].ortalama;
    }
    return s->adet > 0 ? toplam / s->adet : 0.0;
}

int main(void) {
    Sinif sinif = {.adet = 0};

    ekle(&sinif, "Ayse", 101, 72.5);
    ekle(&sinif, "Mehmet", 102, 91.25);
    ekle(&sinif, "Can", 103, 64.0);
    if (!ekle(&sinif, "Elif", 104, 85.75)) {
        printf("Liste dolu: Elif eklenemedi\n");
    }

    listele(&sinif);
    printf("Sınıf ortalaması: %.2f\n", sinifOrtalamasi(&sinif));
    return 0;
}
```

```
ÇIKTI
Liste dolu: Elif eklenemedi
1. Ayse   101  72.50
2. Mehmet 102  91.25
3. Can    103  64.00
Sınıf ortalaması: 75.92
```

Dördüncü öğrenci sığmadığı için `ekle` 0 döndürdü ve liste bozulmadı. `&s->liste[s->adet]` ifadesini şöyle okuruz: `s->liste` bir dizidir, `[s->adet]` ilk boş gözdür, `&` o gözün adresini verir. Gerçek bir sistemde sabit boyutlu dizinin yerine Bölüm 13'teki dinamik dizi ya da Bölüm 19'daki bağlı liste kullanılır. Kayıtları dosyaya yazıp geri okumayı Bölüm 17'de, bütün parçaları birleştiren tam bir uygulamayı Bölüm 20'de göreceğiz.

Bölüm Özeti

- `struct`, farklı tipteki üyeleri tek tipte toplar. Tanım bellek ayırmaz, değişken ayırır; `;` ile biter. Üyeye `.` ile erişilir.
- İlk değer, konumla (`{"Ayse", 101, 72.5}`) ya da adla (`{.no = 101}`) verilir; verilmeyen üyeler sıfırlanır. `= {0}` yapının tamamını sıfırlar.
- `typedef struct { ... } Ogrenci;` ile `Ogrenci a;` yazılır. Etiketsiz `typedef`'ten sonra `struct Ogrenci` yazılamaz; kendine işaretçi taşıyan yapılarda etiket gerekir.
- Yapı ataması (`b = a`) dizi üyeler dahil bütün üyeleri kopyalar. `a == b` ve `o.ad = "Ayse"` derlenmez; `strcmp` ve `strcpy` kullanılır.
- Yapı dizisinde `liste[i].no` yazılır. Takas için `Ogrenci gecici = liste[j];` yeter. `qsort` karşılaştırma fonksiyonu `const void *` alır ve eşitlikte ikinci bir anahtar kullanmalıdır.
- Yapı fonksiyona değerle (kopya) ya da işaretçiyle geçer; `return` ile döndürülebilir. Yalnızca okuyan fonksiyon `const Ogrenci *` alır.
- Soldaki yapıysa `.`, yapıya işaretçiyse `->` kullanılır: `p->no`, `(*p).no` ile aynıdır; `*p.no` derlenmez.
- İç içe yapıda noktalar art arda gelir (`o.dogum.yil`). Yapı kendi türünden üye taşıyamaz, ama kendine işaretçi taşıyabilir.

- `malloc(n * sizeof *liste)` yapı dizisi ayırır; her `malloc` için bir `free` gerekir.
- Dolgu: üyeler kendi hizalamasına göre yerleşir, toplam boyut en büyük hizalamanın katı olur. `struct { char c; int i; }` 64 bitlik gcc'de 8 bayttır. `offsetof` üyenin konumunu verir.
- Bit alanı (`unsigned int gun : 5;`) üyeyi belirli sayıda bite sığdırır; adresi alınamaz, yerleşimi derleyiciye bağlıdır.
- `union` üyeleri aynı belleği paylaşır; boyut en büyük üyeye bağlıdır ve aynı anda tek üye geçerlidir. Hangisinin geçerli olduğu bir `enum` etiketiyle saklanır.
- `enum` sabitleri 0'dan başlar, açık değerden sonra birer artar, `int` gibi davranır ve aralığı denetlenmez.

Alıştırmalar

1. Aşağıdaki kod çalıştırıldıktan sonra hangisi doğrudur?

```c
typedef struct { char ad[10]; int puan; } Kayit;
Kayit a = {"Ayse", 50};
Kayit b = a;
strcpy(b.ad, "Can");
b.puan = 70;
```

   A) `a.ad` de `"Can"` olur, çünkü `b = a` yalnızca dizinin adresini kopyalar.
   B) `a.ad` `"Ayse"`, `a.puan` 50 kalır; `b` bağımsız bir kopyadır.
   C) `a.ad` `"Ayse"` kalır ama `a.puan` 70 olur.
   D) `Kayit b = a;` satırı derlenmez; yapılar `=` ile kopyalanamaz.
   E) `a.puan` 50 kalır ama `a.ad` `"Can"` olur.

2. `Ogrenci o = {"Ayse", 101}; Ogrenci *p = &o;` tanımlarından sonra aşağıdaki ifadelerden hangisi derlenmez?

   A) `o.no`
   B) `p->no`
   C) `(*p).no`
   D) `p.no`
   E) `(&o)->no`

3. Aşağıdaki tanımda `M + O` ifadesinin değeri kaçtır?

```c
enum E { K, L = 5, M, N = 2, O };
```

   A) 4
   B) 6
   C) 8
   D) 9
   E) 11

4. Bir gcc 64 bit derleyicisinde `struct { char a; int b; char c; }` yapısının `sizeof` değeri nedir?

   A) 6
   B) 8
   C) 9

D) 12
E) 16

5. `union { char s[5]; int i; }` tipinin boyutu 64 bitlik gcc'de kaç bayttır?

A) 4
B) 5
C) 8
D) 9
E) 10

6. Aşağıdaki tanımdan sonra hangi satır **derlenmez**?

```
typedef struct { int x; int y; } Nokta;
```

A) `Nokta a;`
B) `Nokta b = {1, 2};`
C) `struct Nokta c;`
D) `Nokta d = {.y = 5};`
E) `Nokta *e = &b;` (`b` önceki satırdaki değişken)

7. Bu programın çıktısı nedir?

```
#include <stdio.h>
typedef struct { int x; int y; } Nokta;

void kaydir(Nokta p) { p.x += 5; p.y += 5; }
Nokta kaydirilmis(Nokta p) { p.x += 5; p.y += 5; return p; }

int main(void) {
    Nokta a = {1, 2};
    kaydir(a);
    Nokta b = kaydirilmis(a);
    Nokta c = b;
    c.x = 0;
    printf("%d %d\n", a.x, a.y);
    printf("%d %d\n", b.x, b.y);
    printf("%d %d\n", c.x, c.y);
    return 0;
}
```

8. Bu programın çıktısı nedir?

```
#include <stdio.h>
typedef struct { char harf; int deger; } Cift;

int main(void) {
    Cift d[3] = {{'a', 10}, {'b', 20}, {'c', 30}};
    Cift *p = d;
    p++;
    p->deger += 5;
    printf("%c %d\n", p->harf, p->deger);
    printf("%c %d\n", (p + 1)->harf, (*(p + 1)).deger);
    printf("%d\n", (int)(p - d));
    int toplam = 0;
    for (Cift *q = d; q < d + 3; q++) {
        toplam += q->deger;
    }
```

```
    printf("%d\n", toplam);
    return 0;
}
```

9. Bu programın çıktısı nedir?

```
#include <stdio.h>
enum Renk { KIRMIZI, YESIL = 4, MAVI, SARI = 2, MOR };
union U { int i; double d; char c; };

int main(void) {
    printf("%d %d %d\n", MAVI, SARI, MOR);
    printf("%zu\n", sizeof(union U));
    enum Renk r = MOR;
    r = r + 1;
    printf("%d %d\n", r, r == MAVI);
    return 0;
}
```

10. Aşağıdaki programda derleme hatası veren üç satırı bulun ve düzeltin.

```
#include <stdio.h>
typedef struct { char ad[20]; int no; } Ogrenci;

int main(void) {
    Ogrenci a = {"Ayse", 101};
    Ogrenci b = a;
    b.ad = "Mehmet";
    if (a == b) {
        printf("ayni\n");
    }
    Ogrenci *p = &a;
    printf("%d\n", p.no);
    return 0;
}
```

11. Aşağıdaki kod derlenir ama yanlış çalışır. Üç sorunu bulun.

```
#include <stdlib.h>
typedef struct { char ad[20]; int no; } Ogrenci;

int main(void) {
    Ogrenci *liste = malloc(10);
    for (int i = 0; i < 10; i++) {
        liste[i].no = i;
    }
    return 0;
}
```

12. Aşağıdaki `takas` fonksiyonu neden iki noktayı değiştirmez? Nasıl düzeltilir?

```
typedef struct { int x; int y; } Nokta;

void takas(Nokta a, Nokta b) {
    Nokta t = a;
    a = b;
    b = t;
}
```

13. Kısa program yazma: `Nokta` yapısı (iki `double` koordinat) tanımlayın. İki noktayı parametre alıp aralarındaki uzaklığı döndüren `mesafe` fonksiyonunu yazın ve `main` içinde (0, 0) ile (3, 4) noktaları için çağırın.
14. Kısa program yazma: Sol alt ve sağ üst köşesi birer `Nokta` olan bir `Dikdortgen` yapısı tanımlayın. Dikdörtgene işaretçi alan `alan` ve `cevre` fonksiyonlarını yazın.
15. Program yazma: Klavyeden önce öğrenci sayısını, sonra her öğrencinin adını, numarasını ve ortalamasını okuyun (bellek `malloc` ile ayrılsın). Sınıf ortalamasını yazın, ardından ortalamanın üstünde kalan öğrencileri `qsort` ile büyükten küçüğe sıralayarak listeleyin.
16. Program yazma: Bir `Sekil` etiketli birleşimi tanımlayın: şekil daire (yarıçap) ya da dikdörtgen (en ve boy) olabilsin. Bir şekil dizisi oluşturun ve her şeklin alanını `switch` ile hesaplayıp yazdırın.

BÖLÜM 15

Önişlemci, Makrolar ve Çok Dosyalı Programlar

BU BÖLÜMDE

- Önişlemcinin derleyiciden önce çalışan, yalnızca metin değiştiren bir adım olduğunu; `#include <>` ile `#include ""` farkını
- `#define` ile nesne benzeri ve parametreli makroları yazmayı, parantez ve yan etki tuzaklarını, `#undef`'i
- `#` ve `##` işleçlerini, çok deyimli makrolarda `do { } while (0)` kalıbını
- `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, `#endif` ile koşullu derlemeyi; `-D` seçeneğini; hata ayıklama makrolarını
- `__FILE__`, `__LINE__`, `__func__`, `__DATE__` gibi önceden tanımlı adları ve `assert`'i
- Bir programı `.h` ve `.c` dosyalarına bölmeyi, başlık koruyucusunu (include guard) ve `#pragma once`'ı
- `gcc a.c b.c`, ayrı derleme (`-c`) ve bağlama aşamalarını; `extern` ile dosya kapsamlı `static`'i
- Küçük bir `Makefile` yazıp `make` ile derlemeyi otomatikleştirmeyi

Şimdiye kadarki programlarımız tek dosyalıydı ve tek dosyalı programlar kısa olduğu sürece sorun çıkarmaz. Program büyüdükçe iki ihtiyaç doğar: tekrar eden metinleri ve ayarları tek yerden yönetmek, kodu da anlamlı parçalara bölüp her parçayı ayrı bir dosyada tutmak. C'de ilk ihtiyacı **önişlemci** (preprocessor), ikincisini **başlık ve kaynak dosya ayrımı** karşılar.

Bu bölümde önce önişlemcinin komutlarını, sonra programı dosyalara bölmeyi öğreneceğiz. Bölümün sonunda birkaç dosyadan oluşan bir programı elle ve `make` ile derleyebilecek, "undefined reference" ve "multiple definition" gibi bağlama hatalarının nedenini bulabileceksiniz.

15.1 Önişlemci Nedir?

Bölüm 4'te derleme sürecinin dört aşamadan geçtiğini gördük: önişleme, derleme, çevirme, bağlama. İlk aşamayı yapan **önişlemci**, kodun anlamıyla ilgilenmez; `#` ile başlayan satırları bulur ve metni buna göre düzenler. Çıktısı yine C metnidir ve derleyiciye o gider.

> **Tanım: Önişlemci**
>
> Derleyiciden önce çalışan, `#` ile başlayan komutları (yönergeleri) işleyerek kaynak metni değiştiren programdır. Dosya ekler, makroları açar, koşullu olarak kod satırlarını ayıklar ve yorumları siler. Değişkenin, tipin ya da fonksiyonun ne olduğunu bilmez; yalnızca metin üzerinde çalışır.

Başlıca önişlemci komutları şunlardır:

Komut	Görevi
`#include`	Başka bir dosyanın içeriğini bulunduğu yere ekler
`#define`, `#undef`	Makro tanımlar, tanımı kaldırır
`#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, `#endif`	Kod parçalarını koşula göre derlemeye alır ya da atar
`#error`	Derlemeyi bir ileti vererek durdurur
`#pragma`	Derleyiciye özgü ayarlar verir

Önişlemci komutları satırın ilk boşluk olmayan karakteri `#` olan satırlardır. C deyimi olmadıkları için sonlarına `;` konmaz ve yalnızca tek satır sürerler; uzun bir satırı bölmek için satır sonuna `\` yazılır.

Önişlemcinin ne ürettiğini görmenin yolu `-E` seçeneğidir. Bu seçenek yalnızca önişlemeyi yapar ve sonucu ekrana yazar; `-P` ise çıktıyı sadeleştiren satır işaretlerini kaldırır. Aşağıdaki küçük dosyayı ele alalım:

```c
// alan.c
#define PI 3.14159
#define KARE(x) ((x) * (x))

double daireAlani(double r) {
    return PI * KARE(r);   // yarıçap
}
```

```
TERMİNAL
$ gcc -E -P alan.c
double daireAlani(double r) {
    return 3.14159 * ((r) * (r));
}
```

Derleyici `PI` ve `KARE` adlarını hiç görmez; ona yalnızca bu açılmış metin ulaşır. Yorum da silinmiştir. Makrolarla ilgili bir şey anlaşılmadığında yapılacak ilk iş `gcc -E` çıktısına bakmaktır.

#include: açılı parantez ve tırnak

`#include` komutunun iki yazılışı vardır:

```c
#include <stdio.h>      // sistem başlığı
#include "istatistik.h" // kendi başlığımız
```

Açılı parantez, dosyayı derleyicinin sistem başlık klasörlerinde arar (`stdio.h`, `math.h` gibi standart başlıkların bulunduğu yerler). Tırnak işareti önce **içeren dosyanın klasöründe** arar; bulamazsa sistem klasörlerine bakar. Kendi yazdığımız başlıklar için tırnak, standart ve sistem başlıkları için açılı parantez kullanırız. Başka klasörlerdeki başlıklar için `-I` seçeneği arama yoluna klasör ekler: `gcc -I include main.c`.

15.2 Nesne Benzeri Makrolar: #define

Daha önce `#define` ile sabit tanımladık (Bölüm 5). Genel biçim şöyledir:

```c
#define AD yerineGecenMetin
```

Önişlemci, `AD` adını dosyanın geri kalanında gördüğü her yerde `yerineGecenMetin` ile değiştirir. Değiştirme **sözcük** düzeyindedir: `AD` bir başka adın parçasıysa (`ADRES`) dokunulmaz, çift tırnak içindeki metne de dokunulmaz. Yer değiştirilen metin de yeniden taranır, yani bir makro başka makroları kullanabilir.

▸ makroSabit.c

```c
#include <stdio.h>

#define SANIYE_DAKIKA 60
#define SANIYE_SAAT (SANIYE_DAKIKA * 60)
#define SANIYE_GUN (SANIYE_SAAT * 24)
#define SELAM "Merhaba"
#define CIZGI_YAZ printf("----------\n")

int main(void) {
    printf("Bir günde %d saniye var\n", SANIYE_GUN);
    printf("SANIYE_GUN yazısı: SANIYE_GUN\n");
    CIZGI_YAZ;
    printf("%s\n", SELAM);

#undef SANIYE_DAKIKA
#define SANIYE_DAKIKA 100
    printf("%d\n", SANIYE_DAKIKA);
    return 0;
}
```

```
ÇIKTI
Bir günde 86400 saniye var
SANIYE_GUN yazısı: SANIYE_GUN
----------
Merhaba
100
```

Bu programdan çıkardığımız sonuçlar:

- `SANIYE_GUN` önce `(SANIYE_SAAT * 24)`, sonra `((SANIYE_DAKIKA * 60) * 24)`, sonunda `((60 * 60) * 24)` olarak açılır. Tanımları **parantezle sarmak** gerekir; nedenini az sonra göreceğiz.
- Çift tırnak içindeki `SANIYE_GUN` yazısı olduğu gibi kalır.
- Bir makro bir deyimi de karşılayabilir (`CIZGI_YAZ`). Kullanırken sonuna kendimiz `;` yazarız.
- `#undef` bir makronun tanımını kaldırır. Aynı adı farklı bir değerle yeniden tanımlamak istiyorsak önce `#undef` yazmalıyız; yoksa derleyici "redefined" uyarısı verir.

Makrolar bir değişken yaratmaz, bellekte yer kaplamaz ve tipleri yoktur. `const` ile `#define` arasındaki ayrımı Bölüm 5'te tablo hâlinde görmüştük. Tek tek sabitler için `const` ya da birbiriyle ilişkili tamsayı sabitleri için `enum` (Bölüm 14) çoğu zaman daha iyi bir seçimdir. Makroların vazgeçilmez olduğu yerler koşullu derleme ve çağrı yerinin bilgisine ihtiyaç duyan işlerdir.

15.3 Parametreli Makrolar

`#define` adının hemen ardına, **araya boşluk koymadan**, parantezli bir parametre listesi yazarsak makro fonksiyona benzer:

```c
#define KARE(x) ((x) * (x))
```

Bu bir fonksiyon çağrısı değil, bir metin kalıbıdır. Önişlemci `KARE(5)` gördüğünde `x` yerine `5` koyar ve `((5) * (5))` metnini yazar. Adla `(` arasına boşluk koyarsak (`#define KARE (x) ...`) parametresiz bir makro tanımlamış oluruz; bu klasik bir hatadır.

Parametreli makrolar tip denetimi yapmaz; aynı `KARE` hem `int` hem `double` ile çalışır. Bu, onları ilk bakışta cazip kılar. Ama metin değiştirme, iki ayrı tuzağı beraberinde getirir.

Birinci tuzak: parantez

Aşağıdaki programda iki makro var: biri parantezsiz, biri parantezli.

▸ makroParantez.c

```c
#include <stdio.h>

#define KARE_YANLIS(x) x * x
#define KARE(x) ((x) * (x))

int main(void) {
    printf("%d %d\n", KARE_YANLIS(5), KARE(5));
    printf("%d %d\n", KARE_YANLIS(2 + 3), KARE(2 + 3));
    printf("%d %d\n", 100 / KARE_YANLIS(5), 100 / KARE(5));
    return 0;
}
```

ÇIKTI

```
25 25
11 25
100 4
```

`KARE_YANLIS(2 + 3)` ifadesi `2 + 3 * 2 + 3` olarak açılır; çarpma önce yapıldığı için sonuç 11'dir. Üçüncü satırda `100 / 5 * 5` soldan sağa işlenir: önce 20, sonra 100. Parantezli makro ise her durumda beklediğimiz sonucu verir. Buradan şu kural çıkar:

> ◆ **Kritik Nokta · Makroda parantez kuralı**
>
> Makro gövdesinde parametrenin **her geçişini** ve gövdenin **tamamını** parantez içine alın. Makroyu hesap makinesi gibi değil, "bul ve değiştir" gibi okuyun: önce açılmış hâlini yazın, sonra normal öncelik kurallarıyla hesaplayın.

İkinci tuzak: argümanın birden fazla işlenmesi

Parametre makro gövdesinde birden çok kez geçiyorsa, argüman olarak yazdığımız ifade o kadar kez yeniden yazılır, dolayısıyla o kadar kez hesaplanır. Fonksiyon çağrısında ise argüman çağrıdan önce yalnızca bir kez hesaplanır.

İki sayının büyüğünü veren bir makro yazalım ve ona yan etkili ifadeler verelim:

▸ makroYanEtki.c

```c
#include <stdio.h>

#define MAKS(a, b) ((a) > (b) ? (a) : (b))

int maksFonksiyon(int a, int b) {
    return a > b ? a : b;
```

```c
}

int main(void) {
    int x = 5, y = 3;
    int m = MAKS(x++, y++);
    printf("makro   : m = %d, x = %d, y = %d\n", m, x, y);

    x = 5;
    y = 3;
    m = maksFonksiyon(x++, y++);
    printf("fonksiyon: m = %d, x = %d, y = %d\n", m, x, y);
    return 0;
}
```

```
ÇIKTI
makro   : m = 6, x = 7, y = 4
fonksiyon: m = 5, x = 6, y = 4
```

Makro çağrısı şu metne dönüşür: `((x++) > (y++) ? (x++) : (y++))`. Koşul bir kez hesaplanır (`5 > 3`, sonra `x` 6, `y` 4 olur), sonra seçilen kol bir kez daha: `x++` değeri 6 verir ve `x` 7 olur. Fonksiyonda ise `x++` ifadesi çağrıdan önce bir kez işlenir, fonksiyona 5 gider.

Bu örnekte koşul işleci (`?:`) iki işlenen arasına bir "sıralama noktası" koyduğu için sonuç tanımlıdır. `KARE(i++)` gibi bir makro ise `((i++) * (i++))` olur; bir değişkeni aynı ifadede iki kez değiştirmek tanımsız davranıştır ve `-Wall` bunu `operation on 'i' may be undefined` uyarısıyla bildirir. Bu yüzden böyle bir çağrının çıktısını göstermiyoruz.

> **▲ Sık Yapılan Hata · Makro argümanına yan etkili ifade vermeyin**
>
> `MAKS(x++, y++)`, `KARE(i++)`, `KARE(rand())` gibi çağrılar, argümanın birden çok kez işlenmesine yol açar. Makro adı büyük harfle yazılır; bunun nedeni, kodu okuyanın bunun bir fonksiyon olmadığını ve argümanlarına dikkat etmesi gerektiğini bilmesidir.

Makro mu, fonksiyon mu?

	Makro	Fonksiyon
Kim işler?	Önişlemci (metin değiştirme)	Derleyici
Tip denetimi	Yok	Var
Argüman	Gövdede geçtiği her yerde yeniden yazılır	Çağrıdan önce bir kez hesaplanır
Adresi alınabilir mi?	Hayır, kod değil metindir	Evet (Bölüm 12)
Hata ayıklama	Zor; hata açılmış metinde görünür	Kolay

Şüphede kalırsak fonksiyon yazarız. Küçük fonksiyonlarda çağrı maliyetinden kaçınmak istiyorsak C99'dan beri `static inline` fonksiyonlar vardır:

```c
static inline int kare(int x) {
    return x * x;
}
```

`inline`, derleyiciye çağrı yerine fonksiyon gövdesini doğrudan yerleştirmeyi önerir. Tip denetimi ve tek seferlik argüman hesabı korunur. Günümüz derleyicileri optimizasyonla bunu zaten çoğu kez kendiliğinden yapar.

Birden çok deyimli makrolar: do { } while (0)

Bir makro birden fazla deyimi karşılayacaksa süslü parantezle sarmak akla ilk gelen çözümdür. Ama bu, `if` ile birlikte kullanıldığında bozulur. İki değişkenin değerini takas eden bir makro yazalım:

```
#define TAKAS_YANLIS(a, b) { int gecici = (a); (a) = (b); (b) = gecici; }

if (x < y)
    TAKAS_YANLIS(x, y);   // açılır: { ... };
else                      // hata: ';' sonrası 'else' eşleşmez
    printf("sıralı\n");
```

Makro `{ ... }` olarak açılınca ardından gelen `;` ayrı bir boş deyim olur ve `if` burada biter; `else` ortada kalır, gcc `'else' without a previous 'if'` hatası verir. Çözüm, gövdeyi `do { ... } while (0)` içine almaktır. Bu döngü tam bir kez çalışır ve dışarıdan tek bir deyim gibi görünür; sonuna konan `;` ona aittir:

▸ makroBlok.c

```
#include <stdio.h>

#define TAKAS(tip, a, b) \
    do {                 \
        tip gecici = (a); \
        (a) = (b);       \
        (b) = gecici;    \
    } while (0)

int main(void) {
    int x = 8, y = 3;

    if (x > y)
        TAKAS(int, x, y);
    else
        printf("zaten sıralı\n");

    printf("x = %d, y = %d\n", x, y);

    double p = 1.5, q = 0.5;
    TAKAS(double, p, q);
    printf("p = %.1f, q = %.1f\n", p, q);
    return 0;
}
```

```
ÇIKTI
x = 3, y = 8
p = 0.5, q = 1.5
```

Satır sonlarındaki `\` karakteri, makro tanımının bir sonraki satırda sürdüğünü söyler; `\` sonrasında boşluk bırakmamak gerekir. Makroya tip de argüman olarak verilebildiği için aynı `TAKAS` hem `int` hem `double` ile çalıştı. Fonksiyonda bu mümkün değildir; C'de bu tür genel işler için ya makrolara ya `void *` işaretçilerine başvurulur (Bölüm 12). C++ şablonları (Bölüm 33) ve C# genelleri (Bölüm 54) bu ihtiyacı tip güvenli biçimde çözer.

15.4 Metne Çevirme (#) ve Birleştirme (##)

Parametreli makrolarda iki özel işleç vardır. `#` işleci, argümanı çift tırnaklı bir metne çevirir. `##` işleci ise iki parçayı yan yana yapıştırıp tek bir ad üretir.

▸ makroOzel.c

```c
#include <stdio.h>

#define YAZDIR_INT(ifade) printf(#ifade " = %d\n", ifade)
#define DEGER(n) deger##n

int main(void) {
    int deger1 = 10, deger2 = 20;
    int a = 4, b = 5;

    YAZDIR_INT(a * b);
    YAZDIR_INT(deger1 + deger2);
    printf("%d\n", DEGER(2));
    return 0;
}
```

ÇIKTI

```
a * b = 20
deger1 + deger2 = 30
20
```

`YAZDIR_INT(a * b)` makrosu `printf("a * b" " = %d\n", a * b)` olarak açılır; yan yana yazılan iki metin sabiti derleyici tarafından birleştirilir. Böylece ifadenin hem metnini hem değerini tek yazımla alırız; hata ayıklarken çok işe yarar. `DEGER(2)` ise `deger2` adını üretir. `##` günlük kodda seyrek kullanılır; büyük makro kütüphaneleri ve kod üreten kalıplar için vardır.

15.5 Koşullu Derleme

Önişlemci, bir koşula bağlı olarak kod satırlarını derleyiciye hiç göndermeyebilir. Üç yaygın kullanım alanı vardır: hata ayıklama çıktıları, platforma özgü kod ve ayarlanabilir sabitler.

Komut	Satırlar ne zaman derlenir?
`#ifdef AD`	`AD` makrosu tanımlıysa
`#ifndef AD`	`AD` makrosu tanımlı değilse
`#if ifade`	Sabit ifade sıfırdan farklıysa; değişken kullanılamaz, tanımsız ad 0 sayılır
`#elif ifade`	Önceki koşullar sağlanmadı ve bu ifade sıfırdan farklıysa
`#else`	Önceki koşulların hiçbiri sağlanmadıysa
`#endif`	Koşullu bloğu bitirir; her koşul için zorunludur

`#if` içinde `defined(AD)` işleci de kullanılabilir; `defined(A) && defined(B)` gibi birleşik koşullar böyle yazılır. Aşağıdaki program üç farklı mekanizmayı bir arada gösterir:

kosulluDerleme.c

```c
#include <stdio.h>

#ifndef ARABELLEK_BOYUT
#define ARABELLEK_BOYUT 64
#endif

#define VERSIYON 2

#if VERSIYON >= 2
#define SURUM_YAZISI "yeni sürüm"
#else
#define SURUM_YAZISI "eski sürüm"
#endif

int main(void) {
    int toplam = 0;
    for (int i = 1; i <= 3; i++) {
        toplam += i;
#ifdef HATA_AYIKLA
        printf("[ayıklama] i = %d, toplam = %d\n", i, toplam);
#endif
    }
    printf("Toplam: %d\n", toplam);
    printf("Arabellek: %d, %s\n", ARABELLEK_BOYUT, SURUM_YAZISI);
    return 0;
}
```

ÇIKTI

```
Toplam: 6
Arabellek: 64, yeni sürüm
```

`HATA_AYIKLA` tanımlı olmadığı için ara çıktı satırı derlemeye hiç girmedi. Kaynak dosyaya dokunmadan makro tanımlamak için derleyicinin `-D` seçeneği kullanılır. `-DAD` makroyu tanımlar, `-DAD=değer` ise bir değer verir:

TERMİNAL

```
$ gcc -std=c17 -Wall -DHATA_AYIKLA -DARABELLEK_BOYUT=128 kosulluDerleme.c -o kosullu
$ ./kosullu
[ayıklama] i = 1, toplam = 1
[ayıklama] i = 2, toplam = 3
[ayıklama] i = 3, toplam = 6
Toplam: 6
Arabellek: 128, yeni sürüm
```

Komut satırında verilen `ARABELLEK_BOYUT` kaynaktaki `#ifndef` korumasından geçti ve varsayılan değer yerine 128 kullanıldı. Ayarlanabilir sabitlerde "varsayılanı kaynakta ver, isteyen `-D` ile değiştirsin" kalıbı çok yaygındır.

Hata ayıklama makroları

Koşullu derleme ile parametreli makroyu birleştirirsek, derleme sırasında açılıp kapanabilen bir izleme aracı elde ederiz. Değişen sayıda argüman almak için C99, parametre listesinde `...` ve gövdede `__VA_ARGS__` kullanımını getirdi:

izleme.c

```c
#include <stdio.h>

#ifdef HATA_AYIKLA
#define IZLE(...)                                   \
    do {                                            \
        printf("[%s:%d] ", __func__, __LINE__);     \
        printf(__VA_ARGS__);                        \
        printf("\n");                               \
    } while (0)
#else
#define IZLE(...) ((void)0)
#endif

int bol(int a, int b) {
    IZLE("bol(%d, %d) çağrıldı", a, b);
    return a / b;
}

int main(void) {
    IZLE("program başladı");
    printf("%d\n", bol(20, 4));
    return 0;
}
```

ÇIKTI

```
5
```

`-DHATA_AYIKLA` ile derlendiğinde aynı program izleme satırlarını da yazar:

TERMİNAL

```
$ gcc -std=c17 -Wall -DHATA_AYIKLA izleme.c -o izleme
$ ./izleme
[main:20] program başladı
[bol:15] bol(20, 4) çağrıldı
5
```

`HATA_AYIKLA` tanımlı değilken `IZLE(...)` makrosu `((void)0)` olur, yani hiçbir şey yapmayan bir ifade. Böylece izleme çağrıları kodda kalır ama sürüm programına tek bayt eklemez.

Platform ve sürüm denetimi

Derleyiciler bulundukları ortamı belirten makrolar tanımlar: Windows için `_WIN32`, Linux için `__linux__`, macOS için `__APPLE__` ve gcc için `__GNUC__` gibi. Bu makrolar standart değil, derleyiciye özgüdür. Platforma özgü kodu `#if defined(...)` ile ayırırız:

platform.c

```c
#include <stdio.h>

int main(void) {
#if defined(_WIN32)
    printf("Windows\n");
#elif defined(__linux__)
    printf("Linux\n");
#elif defined(__APPLE__)
    printf("macOS\n");
#else
```

```
    printf("bilinmeyen sistem\n");
#endif

#if __STDC_VERSION__ < 201112L
#error "Bu program en az C11 gerektirir"
#endif
    printf("C sürüm numarası: %ld\n", __STDC_VERSION__);
    return 0;
}
```

```
ÇIKTI
Linux
C sürüm numarası: 201710
```

Çıktı çalıştığı sisteme göre değişir; yukarıdaki çıktı Linux'ta `-std=c17` ile alındı. `__STDC_VERSION__` standardın sürümünü verir: C99 için `199901L`, C11 için `201112L`, C17 için `201710L`. `#error` ise koşul sağlanırsa derlemeyi verilen iletiyle durdurur. Bir programın gerektirdiği özelliklerin yokluğunu derleme aşamasında yakalamak, çalışma zamanında bulmaktan çok daha iyidir.

15.6 Önceden Tanımlı Adlar ve assert

Önişlemci birkaç makroyu kendisi tanımlar. Bunlar yazıldıkları yerin bilgisini verir:

Ad	Değeri
`__FILE__`	O anki kaynak dosyanın adı (metin)
`__LINE__`	O anki satır numarası (`int`)
`__DATE__`	Derlemenin tarihi (`"Jan 5 2025"` biçiminde metin; ay adı İngilizcedir)
`__TIME__`	Derlemenin saati (`"14:32:07"` biçiminde metin)
`__STDC_VERSION__`	Standart sürümü (`201710L` gibi)

`__func__` bir makro değildir; C99'dan beri her fonksiyonun içinde derleyicinin tanımladığı, o fonksiyonun adını tutan bir metin değişkenidir. Yukarıdaki `izleme.c` programında bu sayede fonksiyon adını yazdırdık.

▸ hazirMakrolar.c

```
#include <stdio.h>

#define KONUM() printf("%s, satır %d, fonksiyon %s\n", \
                       __FILE__, __LINE__, __func__)

void deneme(void) {
    KONUM();
}

int main(void) {
    KONUM();
    deneme();
    printf("Derleme zamanı: %s %s\n", __DATE__, __TIME__);
    return 0;
}
```

```
ÇIKTI
hazirMakrolar.c, satır 11, fonksiyon main
hazirMakrolar.c, satır 7, fonksiyon deneme
Derleme zamanı: Oct 18 2026 13:10:02
```

`__FILE__`, dosyayı derleyiciye hangi adla verdiysek onu yazar. `__LINE__` ise `KONUM()` makrosunun **çağrıldığı** satırı verir, çünkü makro o noktada açılır; aynı bilgiyi fonksiyonla elde edemezdik. Tarih ve saat her derlemede değişir; çıktıdaki değerler örnektir.

assert: varsayımları denetlemek

`<assert.h>` başlığındaki `assert(koşul)` makrosu, koşul doğruysa hiçbir şey yapmaz; yanlışsa koşulun metnini, dosya adını ve satır numarasını yazıp programı `abort` ile durdurur. "Burada bu doğru olmak zorunda" dediğimiz varsayımları denetlemenin kısa yoludur.

▸ assertBol.c

```c
#include <assert.h>
#include <stdio.h>

int bol(int a, int b) {
    assert(b != 0);          // varsayım: bölen sıfır olamaz
    return a / b;
}

int main(void) {
    printf("%d\n", bol(10, 2));
    printf("%d\n", bol(5, 0));
    return 0;
}
```

```
TERMİNAL
$ gcc -std=c17 -Wall assertBol.c -o assertBol
$ ./assertBol
5
assertBol: assertBol.c:5: bol: Assertion `b != 0' failed.
Aborted (core dumped)
```

İleti biçimi sisteme göre değişir; koşul, dosya ve satır her zaman yer alır. `assert` programcı hatalarını yakalamak içindir. Kullanıcının yazdığı bir değeri denetlemek için `if` kullanırız. `<assert.h>` eklenmeden önce `NDEBUG` tanımlanırsa (`-DNDEBUG`) bütün `assert`'ler derlemeden silinir. Bu yüzden `assert` içine yan etkili ifade yazmayız: `assert(i++ > 0)` içindeki `i++` yalnızca `assert` açıkken çalışır.

> *Tarihçe · Önişlemci standartlaşıyor*
>
> Önişlemci C'nin ilk yıllarından beri derleme sürecinin parçasıdır, ancak davranışı uzun süre derleyiciden derleyiciye değişti. ANSI C (C89) önişlemciyi standartlaştırdı ve `#`, `##` işleçlerini ile `defined` ifadesini getirdi. C99, değişen sayıda argümanlı makroları (`__VA_ARGS__`), `inline` fonksiyonları ve `__func__`'i ekledi.

15.7 Programı Dosyalara Bölmek

Yüzlerce satırlık bir programı tek dosyada tutmak, aradığımız fonksiyonu bulmayı zorlaştırır. Birden çok kişi çalışıyorsa herkes aynı dosyayı değiştirmeye çalışır. Ayrıca küçük bir değişiklikte bütün programı yeniden

derlemek gerekir. C'nin çözümü, programı **dosyalara bölmektir**: her `.c` dosyası ayrı derlenir, bağlayıcı da parçaları birleştirir.

Bölme işinde görev paylaşımı sabittir:

> **Tanım: Başlık dosyası ve kaynak dosya**
>
> **Başlık dosyası** (header, `.h`) bir modülün dışarıya sunduğu arayüzü taşır: fonksiyon prototipleri, tip tanımları (`struct`, `enum`, `typedef`), makrolar ve `extern` bildirimleri. **Kaynak dosya** (source, `.c`) uygulamayı taşır: fonksiyon gövdeleri ve global değişkenlerin tanımları. Başlık "ne sunuyoruz" sorusunu, kaynak "nasıl yapıyoruz" sorusunu yanıtlar.

`stdio.h` ile `printf` arasındaki ilişki de budur. `stdio.h` bize `printf`'in prototipini verir; gövdesi standart kütüphanede, derlenmiş hâlde bulunur ve bağlayıcı onu programa ekler (Bölüm 4).

Kendi modülümüzü yazarak başlayalım. Bir dizi tamsayı üzerinde toplam, ortalama ve en büyük değeri bulan küçük bir "istatistik" modülü olsun. Önce başlık dosyası:

```
// istatistik.h
#ifndef ISTATISTIK_H
#define ISTATISTIK_H

#define EN_FAZLA 100

int toplam(const int dizi[], int n);
double ortalama(const int dizi[], int n);
int enBuyuk(const int dizi[], int n);

#endif
```

Başlıkta yalnızca bildirimler ve bir makro var; fonksiyonların gövdesi yok. `const int dizi[]` yazmamız, fonksiyonların diziyi değiştirmeyeceğini belirtir (Bölüm 10). Gövdeler kaynak dosyadadır:

```
// istatistik.c
#include "istatistik.h"

int toplam(const int dizi[], int n) {
    int t = 0;
    for (int i = 0; i < n; i++) {
        t += dizi[i];
    }
    return t;
}

double ortalama(const int dizi[], int n) {
    return (double)toplam(dizi, n) / n;
}

int enBuyuk(const int dizi[], int n) {
    int en = dizi[0];
    for (int i = 1; i < n; i++) {
        if (dizi[i] > en) {
            en = dizi[i];
        }
    }
    return en;
}
```

`istatistik.c` kendi başlığını da ekler. Bunun iki yararı vardır: prototiple gövde uyuşmazsa derleyici hemen `conflicting types` hatası verir ve başlıktaki makrolar, tipler kaynakta da kullanılabilir. Son olarak modülü kullanan ana dosya:

```c
// main.c
#include <stdio.h>
#include "istatistik.h"

int main(void) {
    int notlar[EN_FAZLA] = {70, 85, 90, 65, 100};
    int adet = 5;

    printf("Toplam   : %d\n", toplam(notlar, adet));
    printf("Ortalama : %.1f\n", ortalama(notlar, adet));
    printf("En buyuk : %d\n", enBuyuk(notlar, adet));
    return 0;
}
```

`main.c`, `toplam` fonksiyonunun gövdesini görmez; yalnızca başlıktaki prototipi görür. Derleyici bu bilgiyle çağrıyı denetleyebilir. Gövdeyi bulmak bağlayıcının işidir.

15.8 Derleme ve Bağlama

Üç dosyalı programı derlemenin en kısa yolu, bütün `.c` dosyalarını tek komutta vermektir. Başlık dosyaları komuta yazılmaz; `#include` ile zaten ilgili dosyalara girerler.

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra main.c istatistik.c -o notlar
$ ./notlar
Toplam   : 410
Ortalama : 82.0
En buyuk : 100
```

gcc bu komutta her `.c` dosyasını önce ayrı bir **nesne dosyasına** (object file, `.o`) çevirir, sonra nesne dosyalarını bağlayıp çalıştırılabilir dosyayı üretir. Bu iki adımı kendimiz ayırabiliriz. `-c` seçeneği bağlamayı yapmaz, yalnızca `.o` üretir:

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra -c main.c
$ gcc -std=c17 -Wall -Wextra -c istatistik.c
$ ls
istatistik.c  istatistik.h  istatistik.o  main.c  main.o
$ gcc main.o istatistik.o -o notlar
```

Ayrı derlemenin getirisi büyük projelerde ortaya çıkar. Yalnızca `istatistik.c` değiştiyse `istatistik.o` yeniden üretilir, diğer dosyalar yeniden derlenmez; yalnızca bağlama tekrarlanır. Bunu birazdan `make` ile otomatikleştireceğiz.

Nesne dosyasının içinde ne olduğuna `nm` komutuyla bakabiliriz. Bölüm 4'te gördüğümüz `T` (bu dosyada tanımlı) ve `U` (başka yerde aranacak) işaretleri burada işimize yarar:

```
TERMİNAL
$ nm istatistik.o
0000000000000084 T enBuyuk
0000000000000049 T ortalama
```

```
0000000000000000 T toplam
$ nm main.o
                 U enBuyuk
0000000000000000 T main
                 U ortalama
                 U printf
                 U toplam
```

`main.o`, `toplam`, `ortalama`, `enBuyuk` ve `printf` için `U` taşır; yani bu adların tanımını başka yerde arayacak. Bağlayıcının işi her `U` için bir `T` bulmaktır. Bulamazsa ya da birden çok bulursa hata verir. Bu iki bağlama hatasını şimdi tanıyalım.

undefined reference

`main.c`'yi `istatistik.c` olmadan derlemeye çalışalım. Derleme aşaması geçer, çünkü prototipler başlıkta vardır; bağlama aşaması ise tanımları bulamaz:

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra main.c -o notlar
/usr/bin/ld: /tmp/ccIP0hHB.o: in function `main':
main.c:(.text+0x84): undefined reference to `toplam'
/usr/bin/ld: main.c:(.text+0xb1): undefined reference to `ortalama'
/usr/bin/ld: main.c:(.text+0xe6): undefined reference to `enBuyuk'
collect2: error: ld returned 1 exit status
```

`undefined reference` mesajı, bir adın kullanıldığını ama hiçbir nesne dosyasında tanımlanmadığını söyler. Nedenleri genelde şunlardır: ilgili `.c` dosyasını komuta eklemeyi unutmak, prototipte yazılan adla tanımın adının farklı olması ve `-lm` gibi bir kütüphane seçeneğini unutmak (Bölüm 9).

multiple definition

Bu hata öncekinin tersidir: aynı ad iki nesne dosyasında tanımlanmıştır. En sık nedeni, **tanımı başlık dosyasına yazmaktır.** Bir `sayac.h` başlığı düşünelim:

```
// sayac.h: YANLIŞ, başlıkta tanım var
int sayac = 0;
```

Bu başlığı iki `.c` dosyası eklerse, her ikisinin nesne dosyasında `sayac` değişkeni ayrı ayrı tanımlanır:

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra main.c a.c -o prog
/usr/bin/ld: /tmp/cck5stCf.o:(.bss+0x0): multiple definition of `sayac'; /tmp/ccQfancl.o:(.bss+0
x0): first defined here
collect2: error: ld returned 1 exit status
```

Aynı sonuca bir `.c` dosyasını başka bir `.c` dosyasına `#include` ederek ve ikisini de derlemeye vererek de varırız: fonksiyonlar iki kez tanımlanmış olur.

> **▲ Sık Yapılan Hata · Başlığa tanım yazmayın, .c dosyası include etmeyin**
>
> Başlıklara bildirimler, `.c` dosyalarına tanımlar yazılır. Başlığa fonksiyon gövdesi ya da `int sayac = 0;` gibi bir tanım koymak, başlık birden çok dosyaya girdiğinde `multiple definition` hatası doğurur. Aynı nedenle `#include "istatistik.c"` yazılmaz; `.c` dosyaları derlemeye verilir, `.h` dosyaları eklenir. (İstisna: `static inline` fonksiyonlar başlıkta tanımlanabilir, çünkü her dosya kendi kopyasını alır.)

15.9 Başlık Koruyucusu

`#include`, başlığın metnini bulunduğu yere yapıştırmaktır. Bir başlık aynı kaynak dosyaya iki ayrı yoldan gelirse metin iki kez yapışır. Tip tanımları ikinci kez görülünce derleyici hata verir.

Bir nokta ve bir de iki noktadan oluşan çizgi tanımlayan iki başlığımız olsun. `cizgi.h`, `nokta.h`'yi kendi içinde ekler; `main.c` ise iki başlığı da ekler:

```c
// nokta.h (koruyucusuz)
struct Nokta {
    int x;
    int y;
};
```

```c
// cizgi.h (koruyucusuz)
#include "nokta.h"

struct Cizgi {
    struct Nokta bas;
    struct Nokta son;
};
```

```c
// main.c
#include "nokta.h"
#include "cizgi.h"      // içinden nokta.h bir kez daha gelir
```

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra main.c -o prog
In file included from cizgi.h:1,
                 from main.c:2:
nokta.h:1:8: error: redefinition of 'struct Nokta'
    1 | struct Nokta {
      |        ^~~~~
In file included from main.c:1:
nokta.h:1:8: note: originally defined here
```

Derleyicinin gördüğü metinde `struct Nokta` iki kez tanımlanmıştır. Çözüm, her başlığın içeriğini **başlık koruyucusu** ile sarmaktır.

> **Tanım: Başlık koruyucusu**
>
> Bir başlığın aynı derleme biriminde birden fazla kez işlenmesini önleyen `#ifndef` / `#define` / `#endif` kalıbıdır (include guard). İlk eklemede başlığa özgü makro tanımlı değildir; içerik alınır ve makro tanımlanır. Sonraki eklemelerde makro tanımlıdır, içerik atlanır.

```c
// nokta.h
#ifndef NOKTA_H
#define NOKTA_H

struct Nokta {
    int x;
    int y;
```

```
};

#endif
```

```
// cizgi.h
#ifndef CIZGI_H
#define CIZGI_H

#include "nokta.h"

struct Cizgi {
    struct Nokta bas;
    struct Nokta son;
};

#endif
```

```
// main.c
#include <stdio.h>
#include "nokta.h"
#include "cizgi.h"

int main(void) {
    struct Cizgi c = {{0, 0}, {3, 4}};
    printf("son nokta: (%d, %d)\n", c.son.x, c.son.y);
    return 0;
}
```

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra main.c -o prog
$ ./prog
son nokta: (3, 4)
```

`cizgi.h` ikinci kez `nokta.h`'yi ekleyince `NOKTA_H` zaten tanımlıdır ve içerik atlanır. Koruyucu makronun adı genellikle dosya adının büyük harfli hâlidir (`NOKTA_H`). Adın başka bir başlıkla çakışmaması önemlidir; büyük projelerde proje adı da öne eklenir (`PROJE_NOKTA_H`). `stdio.h` gibi sistem başlıkları da aynı kalıpla korunduğu için `#include <stdio.h>` satırını iki kez yazmak sorun çıkarmaz.

Daha kısa bir yol dosyanın başına tek satır yazılan `#pragma once`'tır:

```
#pragma once
// ... başlığın içeriği
```

`#pragma once` C standardında yoktur, ama gcc, clang ve MSVC destekler. Kısalığı ve ad çakışması riski olmaması avantajdır. Taşınabilirlik önemliyse klasik koruyucu tercih edilir. Bu kitapta klasik biçimi kullanacağız.

> **◆ Kritik Nokta · Her başlığa koruyucu koyun**
>
> Başlık yazarken ilk iş `#ifndef` / `#define` ve son iş `#endif` satırlarını yazmaktır. Koruyucuyu sonradan eklemeyi unutmak, proje büyüyüp başlıklar birbirini eklemeye başladığında anlaşılması zor "redefinition" hatalarına yol açar.

15.10 Başlıkta Neler Bulunur?

Hangi öğenin nereye yazılacağını kural hâline getirelim:

Öğe	Başlık (`.h`)	Kaynak (`.c`)
Fonksiyon prototipi	evet	
Fonksiyon gövdesi		evet
`struct` , `enum` , `typedef` tanımı	evet	
`#define` sabitleri ve makroları	evet	
`extern` bildirimi	evet	
Global değişken tanımı (`int x = 0;`)		evet
Yalnızca bir dosyada kullanılan yardımcı fonksiyon		evet, `static` ile
`static inline` küçük fonksiyon	evet	

Başlıkta yalnızca modülü **kullanmak** için gereken bilgiyi bırakırız. Bir yardımcı fonksiyon yalnızca kaynak dosyada kullanılıyorsa başlığa yazılmaz ve `static` yapılır; böylece modülün arayüzü küçük ve anlaşılır kalır.

15.11 extern ve Dosya Kapsamlı static

Fonksiyonlar dosyalar arasında prototiple paylaşılır. Global değişkenler için aynı işi `extern` yapar. İki kavramı ayıralım:

- **Bildirim** (declaration) bir adın var olduğunu ve tipini duyurur, yer ayırmaz. `extern int sayac;` bir bildirimdir ve istediğimiz kadar dosyada yazılabilir.
- **Tanım** (definition) değişken için bellekte yer ayırır. `int sayac = 0;` bir tanımdır ve bütün programda yalnızca bir `.c` dosyasında bulunmalıdır.

Fonksiyonlarda da aynı ayrım vardır: prototip bildirimdir, gövde tanımdır. Fonksiyon prototipinde `extern` yazmak isteğe bağlıdır; ad varsayılan olarak başka dosyalardan görülebilir.

İşte bir `sayac` modülü. Başlıkta `extern` bildirimi, kaynakta tanımı bulunur:

```c
// sayac.h
#ifndef SAYAC_H
#define SAYAC_H

extern int sayac;          // bildirim
void artir(void);
int simdiki(void);

#endif
```

```c
// sayac.c
#include "sayac.h"

int sayac = 0;             // tanım: yer burada ayrılır
static int adim = 1;       // yalnızca bu dosyada görünür

static void uyar(void) { } // yalnızca bu dosyada görünür
```

```c
void artir(void) {
    sayac += adim;
}

int simdiki(void) {
    uyar();
    return sayac;
}
```

```c
// main.c
#include <stdio.h>
#include "sayac.h"

int main(void) {
    artir();
    artir();
    sayac += 10;            // başka dosyadaki global değişken
    printf("sayac = %d, simdiki = %d\n", sayac, simdiki());
    return 0;
}
```

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra main.c sayac.c -o prog
$ ./prog
sayac = 12, simdiki = 12
```

`main.c`, `sayac`'a doğrudan erişebildi, çünkü başlıktaki `extern` bildirimi onu tanıttı. Tanım ise yalnızca `sayac.c` içinde bulunuyor.

Dikkat edersek `sayac.c` içinde iki şey `static` ile yazılmış. Bölüm 9'da yerel `static` değişkenin değeri çağrılar arasında koruduğunu gördük. **Fonksiyonların dışında** yazılan `static` başka bir anlam taşır: o ad yalnızca tanımlandığı `.c` dosyasında görünür. Buna **iç bağlantı** (internal linkage) denir; bağlayıcı o adı başka dosyalara göstermez. `static` olmayan global adlar **dış bağlantılıdır** (external linkage) ve bütün programdan erişilebilir.

Başka bir dosyadan `static` bir adı kullanmaya çalışırsak bağlayıcı onu bulamaz:

```c
// hata.c: başka dosyadan sayac.c'deki adim'a erişmeye çalışıyor
#include <stdio.h>
extern int adim;
int main(void) { printf("%d\n", adim); return 0; }
```

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra hata.c sayac.c -o prog
/usr/bin/ld: /tmp/ccFCy8re.o: in function `main':
hata.c:(.text+0xa): undefined reference to `adim'
collect2: error: ld returned 1 exit status
```

Aynı durum `uyar` fonksiyonu için de geçerlidir. Bu davranış, modülün iç ayrıntılarını gizlemenin C'deki yoludur: arayüzü başlıkta sunar, yardımcıları `static` yaparız.

> **İpucu · Global değişken paylaşmaktan kaçının**
>
> Bir global değişkeni `extern` ile paylaşmak, her dosyanın ona dokunabilmesi demektir; hata çıktığında suçlu aramak zorlaşır. Çoğu zaman değişkeni `static` yapıp ona `artir()` ve `simdiki()` gibi fonksiyonlarla erişmek daha sağlamdır. Sayaç örneğinde `sayac`'ı `static` yapıp `main.c`'nin doğrudan ona dokunmasını engelleyebilirdik.

15.12 make ile Derlemeyi Otomatikleştirmek

Dosya sayısı arttıkça komutları elle yazmak sıkıcı olur. Daha önemlisi, hangi dosyaların yeniden derlenmesi gerektiğini aklımızda tutmak zorunda kalırız. **make** aracı bu işi bir kural dosyasına (`Makefile`) devreder. Her kural şu biçimdedir:

```
hedef: bağımlılıklar
<TAB>komut
```

Komut satırı mutlaka bir **sekme** (TAB) karakteriyle başlamalıdır; boşluk kabul edilmez. `make`, hedef dosyanın bağımlılıklarından herhangi biri hedeften daha yeniyse komutu çalıştırır; değilse atlar. İstatistik programımız için bir `Makefile` şöyle olabilir:

```
CC = gcc
CFLAGS = -std=c17 -Wall -Wextra

notlar: main.o istatistik.o
        $(CC) main.o istatistik.o -o notlar

main.o: main.c istatistik.h
        $(CC) $(CFLAGS) -c main.c

istatistik.o: istatistik.c istatistik.h
        $(CC) $(CFLAGS) -c istatistik.c

clean:
        rm -f *.o notlar
```

`CC` ve `CFLAGS`, `make` değişkenleridir; `$(CC)` ile kullanılırlar. Kuralları şöyle okuyoruz: `notlar` programı, `main.o` ve `istatistik.o`'dan üretilir. `main.o`, `main.c` ya da `istatistik.h` değişirse yeniden derlenir. `clean` bir dosya adı değil, temizlik komutudur; `make clean` ile çalıştırılır.

```
TERMİNAL
$ make
gcc -std=c17 -Wall -Wextra -c main.c
gcc -std=c17 -Wall -Wextra -c istatistik.c
gcc main.o istatistik.o -o notlar
$ make
make: 'notlar' is up to date.
$ touch istatistik.c
$ make
gcc -std=c17 -Wall -Wextra -c istatistik.c
gcc main.o istatistik.o -o notlar
$ make clean
rm -f *.o notlar
```

İkinci `make` hiçbir şey yapmadı, çünkü hiçbir dosya değişmemişti. `istatistik.c`'nin zamanını güncellediğimizde (`touch`) yalnızca onun nesne dosyası yeniden üretildi ve bağlama tekrarlandı; `main.c` yeniden derlenmedi. Büyük projelerde bu, derleme süresini dakikalardan saniyelere indirir.

> **▲ Sık Yapılan Hata · Makefile'da bağımlılık listesini eksik bırakmak**
>
> `main.o` kuralına `istatistik.h`'yi yazmasaydık, başlıkta bir prototip değiştiğinde `main.c` yeniden derlenmez ve eski tanımla çalışan tutarsız bir program oluşurdu. Başlık dosyası, onu ekleyen her `.c` dosyasının bağımlılığıdır. Büyük projelerde bu listeyi elle yazmak yerine derleyicinin `-MMD` seçeneği ya da `CMake` gibi araçlar kullanılır.

15.13 Üç Dilde: Önişlemci ve Program Yapısı

Üç Dilde · Üç Dilde: Makro, başlık ve modül *C · C++ · C#*

C. Sabit ve küçük işler için makrolar, tip tanımları ve arayüzler için başlık dosyaları kullanılır. Önişlemci metin düzeyinde çalıştığı için tip güvenliği yoktur. İleride C++ ve C#'ta bu işlerin çoğunun dilin kendi özellikleriyle yapıldığını göreceğiz.

C++. Başlık/kaynak ayrımı ve önişlemci C'deki gibi sürer; fakat makro ihtiyacının çoğu ortadan kalkar:

```
constexpr int EN_FAZLA = 100;                 // #define yerine
constexpr int kare(int x) { return x * x; }   // makro yerine
inline int topla(int a, int b) { return a + b; }  // başlıkta tanımlanabilir

namespace istatistik {                        // ad çakışmasını önler
    double ortalama(const int* dizi, int n);
}
```

`constexpr` derleme zamanında hesaplanan, tipli sabitler ve fonksiyonlar sağlar (Bölüm 22). C++'ta `const int N = 10;` doğrudan bir sabit ifadedir. Ad alanları (namespace) iki modülün aynı adlı fonksiyonlarını ayrı tutar. Şablonlar (Bölüm 33) `TAKAS` gibi tipten bağımsız makroların yerini alır. C++20 ile gelen **modüller** (`import`), başlık dosyası modelinin uzun vadeli bir alternatifidir.

C#. Başlık dosyası ve `#include` yoktur. Parametreli makro da yoktur; bu işleri metotlar, genel (generic) türler ve `const` alanlar yapar. Önişlemci komutlarının küçük bir bölümü vardır ve yalnızca koşullu derleme içindir:

```
#define OZEL_SURUM

#if DEBUG
Console.WriteLine("Hata ayıklama derlemesi");
#endif

#if OZEL_SURUM
Console.WriteLine("Özel sürüm");
#endif
```

C#'ta `#define` yalnızca bir **sembol** tanımlar; ona değer atanamaz ve dosyadaki ilk koddan önce yazılmalıdır. `DEBUG` sembolünü Debug yapılandırması otomatik tanımlar. Bir metodu yalnızca belirli yapılandırmada çağırmak için `[Conditional("DEBUG")]` özniteliği de vardır. Program parçalarını ayırmanın yolu ise başlıklar değil, **ad alanları** (`namespace`) ve **derlemelerdir** (assembly): bir projedeki bütün `.cs` dosyaları birlikte derlenir, türler birbirini `using` ve ad alanı üzerinden görür (Bölüm 41).

15.14 Sık Yapılan Hatalar

Bölümdeki tuzakları toparlayalım:

- **Makro gövdesinde ve argümanlarında parantez unutmak.** `#define KARE(x) x * x` kullanıldığında `KARE(2 + 3)` 25 değil 11 verir.
- **Makroya yan etkili argüman vermek.** `MAKS(x++, y++)` argümanları birden çok kez işler. Şüphede kalırsak `static inline` fonksiyon yazarız.
- **Makro adı ile `(` arasına boşluk koymak.** `#define KARE (x) ((x)*(x))` parametresiz bir makrodur.
- **`#define` satırının sonuna `;` ya da `=` koymak.** Bunlar yer değiştirilen metne dahildir (Bölüm 5).
- **Başlığa tanım yazmak veya `.c` dosyası `#include` etmek.** Sonuç `multiple definition` hatasıdır.
- **Başlık koruyucusunu unutmak.** Sonuç `redefinition` hatasıdır.
- **`.c` dosyasını derleme komutuna eklemeyi unutmak.** Sonuç `undefined reference` hatasıdır.
- **Makefile komut satırını boşlukla girmek.** `make` yalnızca TAB kabul eder; `missing separator` hatası verir.

Bölüm Özeti

- Önişlemci, derleyiciden önce çalışan ve yalnızca metin düzeyinde işlem yapan adımdır; `gcc -E` ile çıktısı görülebilir.
- `#include <...>` sistem klasörlerinde, `#include "..."` önce içeren dosyanın klasöründe arar. Kendi başlıklarımız için tırnak kullanılır.
- `#define AD metin` nesne benzeri makrodur; yeniden tanımlamadan önce `#undef` gerekir. Satır sonundaki `;` ve `=` metne dahil olur.
- Parametreli makro bir fonksiyon değil, metin kalıbıdır. Gövdenin ve parametrelerin her geçişi parantez içine alınmalıdır.
- Makro argümanı gövdede kaç kez geçiyorsa o kadar kez işlenir; yan etkili ifadeler tehlikelidir. Küçük işler için `static inline` fonksiyon daha güvenlidir.
- Çok deyimli makrolar `do { ... } while (0)` içine yazılır; böylece `if` / `else` ile sorunsuz kullanılır.
- `#` argümanı metne çevirir, `##` iki parçayı birleştirip ad üretir; `__VA_ARGS__` değişen sayıda argüman almayı sağlar.
- Koşullu derleme `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, `#endif` ile yapılır; makrolar komut satırında `-DAD` ya da `-DAD=değer` ile tanımlanır.
- `__FILE__`, `__LINE__`, `__DATE__`, `__TIME__`, `__STDC_VERSION__` önceden tanımlı makrolardır; `__func__` ise fonksiyon adını veren bir tanımlayıcıdır.
- `assert(koşul)` programcı varsayımlarını denetler; `NDEBUG` tanımlıysa derlemeden silinir.
- Başlık dosyası (`.h`) bildirimleri, kaynak dosya (`.c`) tanımları taşır. Başlığa tanım yazmak ve `.c` dosyası eklemek `multiple definition` hatasına yol açar.
- Başlık koruyucusu (`#ifndef` / `#define` / `#endif`) ya da `#pragma once`, bir başlığın aynı dosyada iki kez işlenmesini önler.
- `gcc a.c b.c -o prog` hepsini tek adımda, `-c` ile `.o` üretip sonra bağlayarak ayrı ayrı derler. Eksik tanım `undefined reference` hatası verir.

- `extern` bir global değişkeni bildirir, tanım tek `.c` dosyasında bulunur. Fonksiyon dışındaki `static` adı o dosyaya özel (iç bağlantılı) yapar.
- `make` ve `Makefile`, yalnızca değişen dosyaların yeniden derlenmesini sağlar; komut satırları TAB ile başlar.

Alıştırmalar

1. `#define KARE(x) x * x` makrosu tanımlıdır. `KARE(3 + 1)` ifadesinin değeri nedir?

   A) 16
   B) 7
   C) 10
   D) 13
   E) 9

2. Önişlemci ile ilgili hangisi doğrudur?

   A) Önişlemci komutları `;` ile bitmek zorundadır.
   B) Önişlemci derleyicinin tip denetiminden sonra çalışır.
   C) Önişlemci kodun anlamını değil, yalnızca metnini işler.
   D) `#define` bellekte bir değişken için yer ayırır.
   E) Önişlemci yalnızca `#include` komutunu tanır.

3. Aşağıdaki makro `if` / `else` ile birlikte kullanıldığında derleme hatası verir. Neden?

```c
#define YAZ_IKI(a, b) { printf("%d\n", a); printf("%d\n", b); }

if (x > 0)
    YAZ_IKI(x, y);
else
    printf("negatif\n");
```

   A) `printf` için `stdio.h` eklenmemiştir.
   B) Makro içinde `;` kullanılamaz.
   C) Makro parametreleri parantez içine alınmamıştır.
   D) Makro `{ }` olarak açıldığı için ardındaki `;` `if` deyimini bitirir ve `else` eşleşmeden kalır.
   E) `if` içinde makro çağrılamaz.

4. `-DHATA_AYIKLA` seçeneğinin görevi hangisidir?

   A) Programı hata ayıklayıcıda başlatır.
   B) Kaynak dosyaya bir `#define HATA_AYIKLA` satırı eklemeden bu makroyu tanımlar.
   C) `HATA_AYIKLA` adlı bir kütüphaneyi bağlar.
   D) Tüm uyarıları kapatır.
   E) `HATA_AYIKLA` adlı bir dosyayı derlemeye ekler.

5. Bir başlık dosyasında aşağıdaki satırlardan hangisi `multiple definition` hatasına yol açar (başlık birden fazla `.c` dosyasından eklenmektedir)?

   A) `extern int sayac;`
   B) `int topla(int a, int b);`

C) `#define EN_FAZLA 100`
D) `int sayac = 0;`
E) `struct Nokta { int x; int y; };` (başlık koruyucusu var)

6. `main.c` ve `modul.c` dosyalarını derleyip programı çalıştırmaya çalışırken `undefined reference to 'hesapla'` hatası alınıyor; `main.c` içinde `hesapla` çağrılıyor ve prototipi `modul.h`'de duruyor. Olası neden hangisidir?

A) `modul.h` içinde başlık koruyucusu yoktur.
B) `gcc main.c -o prog` komutuyla `modul.c` derlemeye verilmemiştir.
C) `hesapla` fonksiyonunun dönüş tipi `int`'tir.
D) `main.c` içinde `#include <stdio.h>` yoktur.
E) `-Wall` seçeneği kullanılmamıştır.

7. Fonksiyonların dışında bir `.c` dosyasında yazılan `static int sayi = 5;` tanımı için hangisi doğrudur?

A) Değişken yalnızca tanımlandığı `.c` dosyasından erişilebilir.
B) Değişken bütün programdan `extern` ile erişilebilir.
C) Değişken her çağrıda yeniden oluşturulur.
D) Değişken yığında (stack) yaşar.
E) Değişken salt okunurdur.

8. Bir `Makefile`'da komut satırı için hangisi gereklidir?

A) Satır sonunda `;` bulunmalıdır.
B) Satır bir TAB karakteriyle başlamalıdır.
C) Komut büyük harfle yazılmalıdır.
D) Komut tırnak içine alınmalıdır.
E) Komut satırı 80 karakteri aşmamalıdır.

9. Bu programın çıktısı nedir?

```c
#include <stdio.h>
#define IKI_KAT(x) 2 * (x)
#define TOPLA(a, b) a + b

int main(void) {
    printf("%d\n", IKI_KAT(3 + 1));
    printf("%d\n", 10 * TOPLA(2, 3));
    return 0;
}
```

10. Bu programın çıktısı nedir?

```c
#include <stdio.h>
#define SINIR 5
#ifdef SINIR
#define MESAJ "tanimli"
#else
#define MESAJ "tanimsiz"
#endif

int main(void) {
    #if SINIR > 3
    printf("buyuk %s\n", MESAJ);
    #else
    printf("kucuk %s\n", MESAJ);
```

```
    #endif
    return 0;
}
```

11. Bu programın çıktısı nedir?

```c
#include <stdio.h>
#define YAZ(x) printf(#x " = %d\n", x)
#define BIRLESTIR(a, b) a##b

int main(void) {
    int xy = 7;
    YAZ(xy * 2);
    printf("%d\n", BIRLESTIR(x, y));
    return 0;
}
```

12. `olcu.h` başlığı aşağıdaki gibidir. `main.c` hem `olcu.h`'yi hem de onu kendi içinde ekleyen `kutu.h`'yi eklediğinde `redefinition of 'struct Olcu'` hatası alınıyor. Hatanın nedenini söyleyin ve başlığı düzeltin.

```c
// olcu.h
struct Olcu {
    int en;
    int boy;
};
```

13. Aşağıdaki makroda hata bulun ve düzeltin:

```c
#define DAIRE_ALANI(r) 3.14159 * r * r
// DAIRE_ALANI(2 + 1) çağrısı beklenen değeri vermiyor.
```

14. `a.c` ve `b.c` dosyalarını önce ayrı ayrı nesne dosyasına çeviren, sonra bağlayarak `uygulama` adlı programı üreten üç `gcc` komutunu yazın.
15. Bir `ucgenAlani.h` ve `ucgenAlani.c` çifti yazın: `double ucgenAlani(double taban, double yukseklik)` fonksiyonu tabanı ve yüksekliği alıp alanı döndürsün. Başlık koruyuculu olsun. Ardından `main.c` içinde `taban = 6`, `yukseklik = 4` için alanı yazdıran bir ana program ve bu üç dosyayı derleyen komutu verin.
16. İki sayının küçüğünü bulan `MIN(a, b)` makrosunu doğru biçimde (parantezli) yazın. Ardından `MIN(i++, 5)` çağrısında neden dikkatli olunması gerektiğini ve makro yerine ne kullanılabileceğini açıklayın.

BÖLÜM 16

Bit Düzeyinde İşlemler

BU BÖLÜMDE

- Bir tamsayının bitlerini numaralandırmayı ve ikilik, onaltılık gösterim arasında geçiş yapmayı
- `&`, `|`, `^`, `~` operatörlerinin her bit çifti üzerinde nasıl çalıştığını ve mantıksal `&&`, `||` ile farkını
- `<<` ve `>>` ile kaydırmanın ikinin kuvvetiyle çarpma ve bölmeyle ilişkisini
- Bit maskeleriyle belirli bir biti açmayı, kapatmayı, tersine çevirmeyi ve sınamayı
- Bir sayının bitlerini ekrana yazan fonksiyonu yazmayı
- Bayrakları (flags) tek bir tamsayıda toplayarak izin sistemi gibi yapıları kurmayı
- İkiye tümleyen gösterimi ve işaretli sayılarda kaydırmanın tuzaklarını

Bu kitapta şimdiye kadar sayıları hep bir bütün olarak ele aldık: toplayıp çıkardık, karşılaştırdık. Oysa bir `int`, bellekte 32 tane bitin yan yana dizilmesinden başka bir şey değildir (Bölüm 5). C, bu bitlere tek tek dokunmamıza izin verir. Buna **bit düzeyinde işlem** (bitwise operation) denir.

Neden gerekli? Çünkü bellek ve hız kısıtlı olduğunda, sekiz tane evet/hayır bilgisini sekiz ayrı değişkende değil tek bir baytın sekiz bitinde tutmak çok daha ekonomiktir. Dosya izinleri, ağ paketlerinin başlıkları, grafik programlarındaki renk değerleri, donanım kayıtları ve sıkıştırma algoritmaları bu yöntemle çalışır. Bu bölümde operatörleri, onlarla yapılan tipik işleri ve işaretli sayılardaki tuzakları göreceğiz.

16.1 İkilik Gösterim ve Bit Numaralama

İkilik ve onaltılık sistemleri Bölüm 1'de görmüştük. Burada yalnızca bu bölümde işimize yarayacak kısmı hatırlayalım. Bir tamsayının bitleri **sağdan sola**, 0'dan başlayarak numaralanır. En sağdaki bit 0. bittir ve **en düşük anlamlı bit** (least significant bit) adını alır; en soldaki bit **en yüksek anlamlı bit** (most significant bit) olur. n numaralı bitin değeri 2^n'dir.

```
bit no :  7   6   5   4   3   2   1   0
değeri :  128 64  32  16  8   4   2   1
sayı   :  0   0   0   0   1   1   0   0     = 8 + 4 = 12
```

Dört bit tam bir onaltılık rakama karşılık geldiği için bitleri dörderli gruplayıp onaltılıkla yazmak çok yaygındır. Aşağıdaki tablo bu bölümde sık göreceğimiz değerleri özetler:

Onluk	İkilik (8 bit)	Onaltılık
1	`00000001`	`0x01`
10	`00001010`	`0x0A`

Onluk	İkilik (8 bit)	Onaltılık
12	`00001100`	`0x0C`
15	`00001111`	`0x0F`
128	`10000000`	`0x80`
255	`11111111`	`0xFF`

C'de ikilik sabit yazmanın standart bir yolu C17'ye kadar yoktur (gcc bir eklenti olarak `0b1100` biçimini kabul eder; C23 bunu standartlaştırdı). Bu yüzden bit kalıplarını onaltılık sabitle (`0x0C`) ya da kaydırmayla (`1u << 3`) yazacağız.

16.2 Bit Operatörleri

C'de bitlere dokunan altı operatör vardır. Dördü iki sayının **aynı sıradaki bit çiftlerini** ayrı ayrı işler:

Operatör	Adı	Bit çifti için kural
`a & b`	VE (AND)	İkisi de 1 ise 1, değilse 0
`a | b`	VEYA (OR)	En az biri 1 ise 1, değilse 0
`a ^ b`	ÖZEL VEYA (XOR)	Bitler farklıysa 1, aynıysa 0
`~a`	DEĞİL (NOT)	Tekli: her biti ters çevirir

Aynı bilgiyi doğruluk tablosu olarak da yazabiliriz:

a	b	a & b	a | b	a ^ b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

12 (`1100`) ile 10 (`1010`) sayılarını alt alta yazıp işlemleri elle yapalım:

```
  1100  (12)        1100  (12)        1100  (12)
& 1010  (10)      | 1010  (10)      ^ 1010  (10)
------              ------            ------
  1000  ( 8)        1110  (14)        0110  ( 6)
```

Üç operatörün işleyişi şöyle özetlenebilir. `&` yalnızca iki tarafta da açık olan bitleri tutar; bir sayıdan istediğimiz bitleri **süzmek** için kullanılır. `|` iki taraftan birinde açık olan her biti açık bırakır; bit **açmak** için kullanılır. `^` ise bitleri **farka** göre işaretler; bir sayıyı aynı değerle iki kez XOR'lamak sayıyı eski haline getirir (`a ^ b ^ b == a`), bu özellik şifrelemede ve bit tersleme işlerinde işe yarar.

Şimdi aynı işlemleri bilgisayara yaptıralım:

▸ bitOperatorleri.c

```c
#include <stdio.h>

int main(void) {
```

```
    unsigned int a = 12;     // 1100
    unsigned int b = 10;     // 1010

    printf("a & b = %u\n", a & b);
    printf("a | b = %u\n", a | b);
    printf("a ^ b = %u\n", a ^ b);
    printf("~a    = %u\n", ~a);
    printf("~a    = %#x\n", ~a);
    return 0;
}
```

```
ÇIKTI
a & b = 8
a | b = 14
a ^ b = 6
~a    = 4294967283
~a    = 0xfffffff3
```

İlk üç sonuç elle bulduklarımızla aynı. `~a` ise 32 bitin hepsini ters çevirir: 12'nin başındaki yirmi sekiz sıfır birer olur, son dört bit `1100` yerine `0011` olur. Sonuç `0xfffffff3`, yani 4294967283'tür. Bu, `UINT_MAX - 12` demektir. Yalnızca düşük dört bite ihtiyacımız varsa sonucu bir maskeyle süzmemiz gerekir: `~a & 0xF` ifadesi 3 verir.

Bit operatörleri ile mantıksal operatörler

`&` ve `&&`, `|` ve `||` benzer görünür, ama tamamen farklı işler yapar. Mantıksal operatörler (Bölüm 5) iki ifadenin doğru olup olmadığına bakar ve yalnızca 1 ya da 0 üretir. Bit operatörleri ise bitlere bakıp yeni bir sayı üretir:

```
int a = 12, b = 10;
printf("%d %d\n", a & b, a && b);    // 8 1
printf("%d %d\n", a | b, a || b);    // 14 1
```

> **▲ Sık Yapılan Hata · Tek ve çift karakterli operatörleri karıştırmak**
>
> `if (x & 4)` ile `if (x && 4)` farklı koşullardır. İlki `x`'in 2. bitine bakar; ikincisi `x`'in sıfırdan farklı olup olmadığını sorar. Bit operatörleri `&&` ve `||` gibi **kısa devre** yapmaz: iki taraf da her zaman hesaplanır.

> **▲ Sık Yapılan Hata · ^ üs alma değildir**
>
> Bazı dillerde `^` üs alma işlecidir. C'de `2 ^ 3` ifadesi 8 değil 1 verir, çünkü `010` ile `011` yalnızca son bitte farklıdır. İkinin kuvvetini hesaplamak için kaydırma (`1 << 3`), genel üs için `<math.h>` içindeki `pow` kullanılır.

16.3 Kaydırma Operatörleri

Kalan iki operatör bitleri sola ya da sağa kaydırır:

Operatör	Adı	Ne yapar
`a << n`	Sola kaydırma	Bitleri n basamak sola iter; sağdan n tane 0 gelir
`a >> n`	Sağa kaydırma	Bitleri n basamak sağa iter; sağdaki n bit atılır

```
  5 = 00000101
  5 << 1 = 00001010  (10)     her bit bir basamak sola
  5 << 3 = 00101000  (40)
 40 >> 2 = 00001010  (10)     sağdan iki bit düştü
```

Onluk sistemde bir sayıyı sola kaydırmak 10 ile çarpmak olurdu (`25` ⟶ `250`). İkilik sistemde ise taban 2 olduğu için **bir basamak sola kaydırmak 2 ile çarpmaktır**; **bir basamak sağa kaydırmak** ise 2'ye bölüp kesiri atmaktır. Genel olarak `a << n`, `a * 2ⁿ`; `a >> n`, `a / 2ⁿ` anlamına gelir (negatif olmayan `a` için).

▸ kaydirma.c

```c
#include <stdio.h>

int main(void) {
    unsigned int a = 5;

    printf("%u\n", a << 1);       // 5 * 2
    printf("%u\n", a << 3);       // 5 * 8
    printf("%u\n", (a << 3) >> 2);
    printf("%u\n", 100u >> 3);    // 100 / 8, kesir atılır
    printf("%u\n", 1u << 10);     // 2 üzeri 10
    printf("%u\n", 1u << 31);     // en yüksek bit
    return 0;
}
```

ÇIKTI

```
10
40
10
12
1024
2147483648
```

`1u << n` kalıbı, yalnızca n. biti 1 olan sayıyı üretir; bu bölümün en çok kullanacağımız ifadesidir. Sonuna `u` koymamızın nedeni var: `1` bir `int`'tir ve `1 << 31` işaretli sayıda taşmaya yol açar (aşağıda göreceğiz). `1u` ile işaretsiz hesaplarız.

> ◆ **Kritik Nokta · Kaydırma çarpma ve bölmeden hızlı mıdır?**
>
> Eskiden derleyiciler çarpmayı kaydırmaya çevirmediği için `x << 1` yazmak hız kazandırırdı. Bugün derleyiciler `x * 2` ifadesini kendileri en uygun komuta çevirir. Bu yüzden çarpma için kaydırma kullanmayız; kaydırma, **bitlerle çalıştığımızı** ifade etmek istediğimizde tercih edilir.

16.4 Bitleri Ekrana Yazmak

`printf`'in ikilik tabanda yazdıran bir belirteci yoktur. Bit işlemlerini görerek öğrenmek için kendi fonksiyonumuzu yazalım. Fikir basit: en yüksek bitten başlayıp sıfırıncı bite kadar her biti sırayla sınayacağız. Bir sayıyı `i` basamak sağa kaydırıp `& 1` ile süzersek `i` numaralı biti elde ederiz:

```
  sayi = 12 = 00001100,   i = 2 için:
  sayi >> 2        = 00000011
  (sayi >> 2) & 1  = 00000001   ->  2. bit 1
```

Bunu bir döngüyle tüm bitlere uygulayalım:

bitYaz.c

```c
#include <stdio.h>

// sayinin en düşük 'genislik' bitini soldan sağa yazar
void bitleriYaz(unsigned int sayi, int genislik) {
    for (int i = genislik - 1; i >= 0; i--) {
        putchar(((sayi >> i) & 1) ? '1' : '0');
    }
}

void satirYaz(const char *ad, unsigned int deger) {
    printf("%-6s = ", ad);
    bitleriYaz(deger, 8);
    printf("  (%u)\n", deger);
}

int main(void) {
    unsigned int a = 12, b = 10;

    satirYaz("a", a);
    satirYaz("b", b);
    satirYaz("a & b", a & b);
    satirYaz("a | b", a | b);
    satirYaz("a ^ b", a ^ b);
    satirYaz("~a", ~a & 0xFF);     // yalnız düşük 8 bit
    satirYaz("a << 2", a << 2);
    satirYaz("a >> 2", a >> 2);
    return 0;
}
```

ÇIKTI

```
a      = 00001100  (12)
b      = 00001010  (10)
a & b  = 00001000  (8)
a | b  = 00001110  (14)
a ^ b  = 00000110  (6)
~a     = 11110011  (243)
a << 2 = 00110000  (48)
a >> 2 = 00000011  (3)
```

`%-6s`, metni 6 karakterlik alanda sola yaslar (Bölüm 6). `~a` satırında yalnızca düşük sekiz biti gösterebilmek için `& 0xFF` ile süzdük; bu da `&` operatörünün bir **maske** olarak kullanımının ilk örneğidir. `bitleriYaz` fonksiyonunu bölümün geri kalanında da kullanacağız; `genislik` parametresi 8, 16 ya da 32 verilerek istenen kadar bit gösterilebilir.

Bu fonksiyonla ilgili iki ayrıntı vardır. Birincisi, `putchar` tek bir karakteri ekrana yazan `<stdio.h>` fonksiyonudur. İkincisi, `?:` operatörü (Bölüm 7) her bit için `'1'` ya da `'0'` karakterini seçer.

16.5 Bit Maskeleri

Gerçek programlarda amaç tüm bitleri değil, **belirli bir biti** değiştirmek ya da okumaktır. Bunun için yalnızca ilgilendiğimiz bitin 1 olduğu bir sayı hazırlarız. Bu sayıya **bit maskesi** (bit mask) denir. n. bitin maskesi `1u << n`'dir.

> **Tanım: Bit maskesi**
> Bir sayıdaki belirli bitleri seçmek, açmak, kapatmak ya da tersine çevirmek için, yalnızca ilgili bitleri 1 olan yardımcı sayıdır.

Dört temel işlem ve her birinin operatörü şöyledir:

İşlem	İfade	Neden çalışır
Biti **aç** (1 yap)	`x | (1u << n)`	VEYA ile 1 olan bit hep 1 kalır, diğer bitler değişmez
Biti **kapat** (0 yap)	`x & ~(1u << n)`	Maskenin tersi n. bitte 0'dır; VE onu siler, diğer bitler 1'le korunur
Biti **ters çevir**	`x ^ (1u << n)`	XOR ile 1 olan bit tersine döner, 0 olanlar aynı kalır
Biti **sına**	`x & (1u << n)`	Sonuç sıfırdan farklıysa bit açıktır

Bu dört işlemi birer fonksiyon olarak yazıp denemek en anlaşılır yoldur:

▸ bitMaske.c

```c
#include <stdio.h>

void bitleriYaz(unsigned int sayi, int genislik) {
    for (int i = genislik - 1; i >= 0; i--) {
        putchar(((sayi >> i) & 1) ? '1' : '0');
    }
}

unsigned int bitAc(unsigned int x, int n)    { return x | (1u << n); }
unsigned int bitKapat(unsigned int x, int n) { return x & ~(1u << n); }
unsigned int bitTers(unsigned int x, int n)  { return x ^ (1u << n); }
int bitAcikMi(unsigned int x, int n)         { return (x & (1u << n)) != 0; }

int main(void) {
    unsigned int x = 10;     // 00001010

    printf("x          = "); bitleriYaz(x, 8); printf("\n");
    printf("bitAc(0)   = "); bitleriYaz(bitAc(x, 0), 8); printf("\n");
    printf("bitKapat(3)= "); bitleriYaz(bitKapat(x, 3), 8); printf("\n");
    printf("bitTers(2) = "); bitleriYaz(bitTers(x, 2), 8); printf("\n");
    printf("1. bit açık mı? %d\n", bitAcikMi(x, 1));
    printf("2. bit açık mı? %d\n", bitAcikMi(x, 2));
    return 0;
}
```

```
ÇIKTI
x          = 00001010
bitAc(0)   = 00001011
bitKapat(3)= 00000010
bitTers(2) = 00001110
1. bit açık mı? 1
2. bit açık mı? 0
```

`bitKapat` işlevinin içindeki `~(1u << n)` ifadesine dikkat edelim. n = 3 için `1u << 3` = `00001000` olur; `~` ile tersi `...11110111` çıkar. Bu sayıyla `&` yapılırsa yalnızca 3. bit silinir, diğer bitler olduğu gibi kalır. Sınama fonksiyonunda `!= 0` karşılaştırmasını yazmamızın nedeni, `x & (1u << n)` sonucunun 1 değil, o bitin değeri (2, 4, 8…) olmasıdır.

> **▲ Sık Yapılan Hata · Sınama sonucunu 1'le karşılaştırmayın**
>
> `(x & (1u << 3)) == 1` yanlış bir kontroldür: sonuç 3. bit açıksa 8 olur, 1 olmaz. Doğru kontrol `!= 0` ya da doğrudan `if (x & (1u << 3))` biçimindedir.

Birden çok bitle çalışmak: alan çıkarma ve birleştirme

Maske tek bit olmak zorunda değildir. Birkaç bitten oluşan bir **alan** (bit field) tek bir sayıda saklanabilir. Tipik örnek renklerdir: kırmızı, yeşil, mavi bileşenlerinin her biri 0–255 arasında bir değerdir ve her biri bir bayta sığar. Üçünü tek bir sayıya yerleştirebiliriz:

```
  bit:   23 ..... 16 | 15 ..... 8 | 7 ..... 0
         kırmızı (R)  |  yeşil (G)  |  mavi (B)
```

Bileşeni yerleştirmek için sola kaydırıp `|` ile birleştiririz; geri almak için sağa kaydırıp `& 0xFF` ile yalnızca alt sekiz biti tutarız:

▸ renkPaketle.c

```c
#include <stdio.h>

unsigned int renkOlustur(unsigned int r, unsigned int g, unsigned int b) {
    return (r << 16) | (g << 8) | b;
}

int main(void) {
    unsigned int renk = renkOlustur(255, 128, 64);
    printf("renk = 0x%06X (%u)\n", renk, renk);

    unsigned int kirmizi = (renk >> 16) & 0xFF;
    unsigned int yesil   = (renk >> 8) & 0xFF;
    unsigned int mavi    = renk & 0xFF;
    printf("R=%u G=%u B=%u\n", kirmizi, yesil, mavi);

    // yeşil bileşeni sıfırla
    unsigned int yesilsiz = renk & ~(0xFFu << 8);
    printf("yeşilsiz = 0x%06X\n", yesilsiz);
    return 0;
}
```

```
ÇIKTI
renk = 0xFF8040 (16744512)
R=255 G=128 B=64
yeşilsiz = 0xFF0040
```

Onaltılık gösterim burada bit gruplarını okumayı kolaylaştırır: `0xFF8040` içindeki her iki rakam bir bileşendir. Web sayfalarındaki `#FF8040` renk kodu da aynı düzendir. Bu "kaydır ve süz" kalıbı (`(x >> konum) & maske`), bir sayıdan alan okumanın standart yoludur ve ağ başlıklarında, dosya biçimlerinde ve donanım kayıtlarında sürekli karşımıza çıkar.

16.6 Bayraklar ve İzin Sistemi

Bir uygulamada kullanıcının "okuma", "yazma" ve "çalıştırma" izinleri olsun. Her izin ya vardır ya yoktur; yani birer evet/hayır bilgisidir. Üç ayrı `int` değişken tutmak yerine üç biti tek bir sayıda toplayabiliriz. Her izne ayrı

bir bit veririz ve bu bitlerin maskelerini adlandırırız. Bu tür tek bitlik evet/hayır göstergelerine **bayrak** (flag) denir.

```
  CALISTIR  YAZ  OKU
     bit 2  bit 1 bit 0
       4      2     1
```

Aşağıdaki program Unix dosya izinlerindeki `rwx` gösterimini taklit eder:

▸ **izinSistemi.c**

```c
#include <stdio.h>

#define OKU      1u        // 001
#define YAZ      2u        // 010
#define CALISTIR 4u        // 100

void izinYaz(unsigned int izin) {
    printf("%c%c%c (%u)\n",
           (izin & OKU) ? 'r' : '-',
           (izin & YAZ) ? 'w' : '-',
           (izin & CALISTIR) ? 'x' : '-',
           izin);
}

int main(void) {
    unsigned int izin = OKU | YAZ;      // iki bayrağı birleştir
    izinYaz(izin);

    izin |= CALISTIR;                    // bayrak ekle
    izinYaz(izin);

    izin &= ~YAZ;                        // bayrak sil
    izinYaz(izin);

    izin ^= OKU;                         // bayrağı tersle
    izinYaz(izin);

    unsigned int gerekli = OKU | CALISTIR;
    izin = OKU | YAZ | CALISTIR;
    if ((izin & gerekli) == gerekli) {
        printf("Gerekli izinlerin hepsi var.\n");
    }
    if (izin & (YAZ | CALISTIR)) {
        printf("Yazma ya da çalıştırma izni var.\n");
    }
    return 0;
}
```

```
ÇIKTI
rw- (3)
rwx (7)
r-x (5)
--x (4)
Gerekli izinlerin hepsi var.
Yazma ya da çalıştırma izni var.
```

Kalıpları sırayla inceleyelim:

- Bayrakları birleştirmek için `|` kullanırız: `OKU | YAZ` = `011` = 3.

- Bayrak eklemek `izin |= CALISTIR`, silmek `izin &= ~YAZ`, tersine çevirmek `izin ^= OKU` ifadeleridir. Bunlar bölümün başında gördüğümüz tek bit işlemlerinin bileşik atama biçimleridir.
- **Birden çok** bayrağın hepsinin açık olduğunu sınamak için sonucu maskenin kendisiyle karşılaştırırız: `(izin & gerekli) == gerekli`.
- **En az birinin** açık olduğunu sınamak için sonucun sıfırdan farklı olmasına bakarız: `izin & (YAZ | CALISTIR)`.

Bu yöntemin gücü, tek bir `int` ile 32 ayrı bayrağı, tek bir `unsigned long long` ile 64 bayrağı taşıyabilmesidir. İzinleri dosyaya yazmak ya da fonksiyona geçirmek de tek bir değerin aktarılmasına iner. Bölüm 14'te `enum` ile bayrak sabitlerini daha okunaklı adlandırmayı göreceğiz.

> **İpucu · Bayrak sabitlerini kaydırmayla yazın**
>
> `#define OKU 1u`, `#define YAZ 2u`, `#define CALISTIR 4u` yerine `(1u << 0)`, `(1u << 1)`, `(1u << 2)` yazmak, yeni bayrak eklerken 1, 2, 4, 8, 16, 32… sırasını yanlış saymaktan korur. Yeni bayrak bir sonraki bit numarasını alır.

16.7 Sık Kullanılan Küçük Bit Teknikleri

Bit operatörlerinin birkaç kullanımı o kadar yaygındır ki kalıp olarak bilmekte yarar vardır. Hepsi işaretsiz sayılarla çalışacak biçimde yazılmıştır.

Tek mi, çift mi? Çift bir sayının sıfırıncı biti 0, tek bir sayının 1'dir. Bu yüzden `x & 1` bize tekliği verir. (`x % 2` ile aynı sonucu verir; derleyici zaten bu dönüşümü kendisi yapar, ama kalıp bit işlerinde sık görülür.)

2'nin kuvveti mi? 2'nin kuvveti olan sayıların ikilik gösteriminde tek bir bit 1'dir (`8 = 1000`). Bundan 1 çıkarılınca o bit sıfırlanır, altındaki tüm bitler 1 olur (`7 = 0111`). İkisinin `&`'i bu yüzden sıfırdır:

```
   8 = 1000        6 = 0110
   7 = 0111        5 = 0101
 8 & 7 = 0000    6 & 5 = 0100   (sıfır değil, 6 ikinin kuvveti değil)
```

Açık bit sayısı. `x & (x - 1)` ifadesi `x`'in en sağdaki 1 bitini siler. Sıfır olana kadar kaç kez silebildiğimizi sayarsak açık bit sayısını buluruz. Bu yöntem 32 bitin hepsine bakmaktan hızlıdır, çünkü yalnızca açık bit sayısı kadar döner.

▸ bitHileleri.c

```c
#include <stdio.h>

int tekMi(unsigned int x) {
    return x & 1;
}

int ikininKuvvetiMi(unsigned int x) {
    return x != 0 && (x & (x - 1)) == 0;
}

int acikBitSayisi(unsigned int x) {
    int sayac = 0;
    while (x != 0) {
        x &= x - 1;          // en sağdaki 1 bitini sil
        sayac++;
    }
```

```
    return sayac;
}

int main(void) {
    unsigned int sayilar[] = {1, 6, 8, 12, 64, 255};

    printf("sayi tek  2^n  bit\n");
    for (int i = 0; i < 6; i++) {
        unsigned int s = sayilar[i];
        printf("%4u %3d %4d %4d\n", s, tekMi(s), ikininKuvvetiMi(s),
               acikBitSayisi(s));
    }

    // XOR ile ek değişkensiz takas
    int a = 5, b = 9;
    a ^= b;
    b ^= a;
    a ^= b;
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

```
ÇIKTI
sayi tek  2^n  bit
   1   1    1    1
   6   0    0    2
   8   0    1    1
  12   0    0    2
  64   0    1    1
 255   1    0    8
a = 9, b = 5
```

Son bölümdeki XOR takası şöyle çalışır: `a ^= b` ile `a`, iki değerin farkını taşır; `b ^= a` ile `b` eski `a`'ya dönüşür; `a ^= b` ile `a` eski `b`'ye dönüşür. Bunu bir merak ürünü olarak görmek yeterlidir. Bölüm 5'teki üçüncü değişkenli takas hem daha okunaklıdır hem de aynı hızdadır. XOR takası ayrıca `a` ve `b` aynı değişkense (Bölüm 12'de işaretçilerle olabilir) değeri sıfırlar.

> *Tarihçe · Bit düzeyinde işlemlerin kökeni*
>
> Bit operatörleri C'nin ilk sürümlerinden beri dilin parçasıdır; C, UNIX işletim sistemini yazmak için geliştirilmiştir. UNIX dosya sisteminde izinler de (`rwx`) bu bölümdeki gibi bitler olarak saklanır; yukarıdaki program o düzenin küçük bir benzetimidir.

16.8 İşaretli Sayılar ve İkiye Tümleyen

Şimdiye kadar bit düzeyinde işlemleri işaretsiz sayılarla yaptık. İşaretli sayıların bitleri nasıl yorumlanır? Bilgisayar negatif sayıları **ikiye tümleyen** (two's complement) yöntemiyle saklar. Kuralı iki cümleyle söyleyebiliriz:

1. Pozitif sayılar olduğu gibi yazılır. En yüksek bit (işaret biti) 0'dır.
2. Negatif sayıyı bulmak için pozitif karşılığının bitleri ters çevrilir ve sonuca 1 eklenir. Yani `-x` değeri `~x + 1`'dir.

8 bitlik bir örnekle 5 ve -5'e bakalım:

```
   5 = 00000101
  ~5 = 11111010
 ~5+1 = 11111011  = -5
```

Neden bu yöntem seçildi? Çünkü toplama devresi işaretli ve işaretsiz sayılar için aynı kalır. `5 + (-5)` işlemini bit bit toplarsak `00000101 + 11111011 = 1 00000000` çıkar; dokuzuncu bit taşıp atılır ve sonuç sıfırdır. Çıkarma için ayrı bir devreye gerek kalmaz. Ayrıca sıfırın tek bir gösterimi vardır.

Bu düzende işaret biti sayıya -2^{n-1} değeriyle katılır. 8 bitte en soldaki bit `-128` ağırlığındadır: `11111011` = -128 + 64 + 32 + 16 + 8 + 2 + 1 = -5. Bölüm 5'te gördüğümüz asimetrinin (-128 … 127) nedeni budur: pozitif tarafta sıfır da bir değer tuttuğu için en büyük pozitif sayı 127'de kalır.

▸ **ikiyeTumleyen.c**

```c
#include <stdio.h>

void bitleriYaz(unsigned int sayi, int genislik) {
    for (int i = genislik - 1; i >= 0; i--) {
        putchar(((sayi >> i) & 1) ? '1' : '0');
    }
}

int main(void) {
    signed char degerler[] = {5, -5, 1, -1, 127, -128};

    for (int i = 0; i < 6; i++) {
        printf("%4d = ", degerler[i]);
        bitleriYaz((unsigned char)degerler[i], 8);
        printf("\n");
    }

    int x = 5;
    printf("~x     = %d\n", ~x);
    printf("~x + 1 = %d\n", ~x + 1);

    unsigned char c = (unsigned char)-1;
    printf("(unsigned char)-1 = %d\n", c);
    return 0;
}
```

ÇIKTI

```
   5 = 00000101
  -5 = 11111011
   1 = 00000001
  -1 = 11111111
 127 = 01111111
-128 = 10000000
~x     = -6
~x + 1 = -5
(unsigned char)-1 = 255
```

`-1` her bitin 1 olduğu sayıdır; bu yüzden `-1`'i işaretsiz bir tipe çevirmek o tipin en büyük değerini verir (`255`, `UINT_MAX`). Bölüm 5'te gördüğümüz `-1 < 1u` karşılaştırmasının yanlış çıkması da bundandır: `-1` işaretsize çevrilince dev bir sayı olur.

İşaretli sayılarda kaydırma uyarıları

Kaydırma operatörleri işaretli sayılarda tehlikelidir. C standardı şunları söyler:

- **Negatif bir sayıyı sola kaydırmak** tanımsız davranıştır.
- Sola kaydırmanın sonucu işaretli tipe **sığmıyorsa** tanımsız davranıştır. Bu yüzden `1 << 31` (32 bitlik `int` için) güvenli değildir; `1u << 31` yazmalıyız.
- **Negatif bir sayıyı sağa kaydırmanın** sonucu **derleyiciye bağlıdır** (implementation-defined). gcc ve çoğu derleyici işaret bitini koruyarak kaydırır (aritmetik kaydırma); `-16 >> 2` ifadesi bu derleyicilerde -4 verir. Başka bir derleyici 0 bitleri getirebilir (mantıksal kaydırma).
- Kaydırma miktarı negatifse ya da tipin bit genişliğine eşit veya büyükse (`int` için 32 ya da fazlası) davranış tanımsızdır.

> ◆ **Kritik Nokta · Bit işlemlerini işaretsiz tiplerle yapın**
>
> Yukarıdaki kuralların hepsi tek bir ilkeye bağlanır: bit operatörleri `unsigned` tiplerde tamamen tanımlı ve taşınabilir, işaretli tiplerde ise tuzak doludur. Bit maskeleri, bayraklar ve kaydırmalar için `unsigned int` (ya da `<stdint.h>` içindeki `uint32_t`, `uint8_t`) kullanın.

16.9 Küçük Tiplerde Tip Yükseltmesi Tuzağı

Bölüm 5'te `char` ve `short` değerlerin aritmetikte önce `int`'e yükseltildiğini görmüştük. Bu kural bit operatörlerine de uygulanır ve sonuçlar şaşırtıcı olabilir:

▸ tipYukseltme.c

```c
#include <stdio.h>

int main(void) {
    unsigned char c = 0x0F;

    printf("~c = %d\n", ~c);
    printf("~c == 0xF0 : %d\n", ~c == 0xF0);
    printf("(unsigned char)~c == 0xF0 : %d\n", (unsigned char)~c == 0xF0);

    unsigned char d = 0x81;
    printf("d << 1 = %d\n", d << 1);
    printf("(unsigned char)(d << 1) = %d\n", (unsigned char)(d << 1));
    return 0;
}
```

ÇIKTI

```
~c = -16
~c == 0xF0 : 0
(unsigned char)~c == 0xF0 : 1
d << 1 = 258
(unsigned char)(d << 1) = 2
```

`~c` ifadesinde `c` önce 32 bitlik `int`'e yükselir (`0x0000000F`), sonra bitleri ters çevrilir (`0xFFFFFFF0`). Bu değer -16'dır ve 8 bitlik `0xF0`'a (240) eşit değildir. Benzer şekilde `d << 1`, `int` üzerinde yapıldığından 8 bitin dışına taşan değeri (258) korur. Sonucun yine 8 bitte kalmasını istiyorsak açık dönüşüm yazmalıyız.

▲ Sık Yapılan Hata · Bit işleminden sonra sonucu tipe geri çevirin

`unsigned char` ya da `uint8_t` ile çalışırken `~`, `<<` sonuçlarını bir değişkene atarken ya da karşılaştırırken `(unsigned char)` dönüşümü ya da `& 0xFF` maskesi kullanın. Aksi halde yükseltilmiş `int` değeri beklenmedik sonuç verir.

16.10 Önceliğe Dikkat

Bit operatörlerinin önceliği alışılmadık biçimdedir. Bölüm 5'teki öncelik tablosunu hatırlayalım: `~` tekli operatörlerle aynı yüksek seviyededir; `<<` ve `>>` toplama ve çıkarmanın hemen altındadır; `&`, `^`, `|` ise karşılaştırma operatörlerinin **bile altındadır** (`&` en yüksek, ardından `^`, sonra `|`; hepsinden sonra `&&`, `||` gelir).

Bu yüzden iki ifade göründüğünden farklı çalışır:

```
if (x & 1 == 0)        // x & (1 == 0) olarak okunur: yanlış
if ((x & 1) == 0)      // doğru: x çift mi?

int a = 1 << 2 + 1;    // 1 << (2 + 1) = 8, (1 << 2) + 1 = 5 değil
```

Pratik kural açıktır: bit operatörü içeren her karışık ifadede **parantez kullanın**. gcc `-Wall` ile bu tür ifadelerde uyarı verir; uyarıları ciddiye alın.

Üç Dilde · Bit işlemleri üç dilde *C · C++ · C#*

Operatörler üç dilde de aynıdır: `&`, `|`, `^`, `~`, `<<`, `>>`. C#'ta ek olarak işaretsiz sağa kaydırma yapan `>>>` (C# 11) vardır. Farklar, bitleri görme ve bayrak tutma konusundaki hazır araçlardadır:

Konu	C	C++	C#
İkilik sabit	`0b1100` (C23; gcc eklentisi)	`0b1100` (C++14), rakam ayırıcı `0b0000'1100`	`0b0000_1100`
Bitleri yazma	Kendi fonksiyonumuz	`std::bitset`	`Convert.ToString(x, 2)`
Açık bit sayısı	Kendi fonksiyonumuz	`bitset::count()`	`BitOperations.PopCount`
Bayrak kümesi	`#define` / `enum` ile maskeler	`enum` , `std::bitset`	`[Flags] enum`

C++'ın `std::bitset` sınıfı sabit uzunlukta bit kümesi tutar; operatörler üzerinde doğrudan tanımlıdır ve ekrana ikilik yazılır. Bölüm 6'da gördüğümüz `<<` akış operatörü ile `bitset`'in bit kaydırması aynı işareti kullanır ama farklı bağlamlardadır.

▸ bitsetOrnek.cpp

```
#include <bitset>
#include <iostream>

int main() {
    std::bitset<8> a(12), b(10);
    std::cout << (a & b) << ' ' << (a | b) << ' ' << (a ^ b) << '\n';
    std::cout << ~a << ' ' << (a << 2) << '\n';
    std::cout << a.count() << ' ' << a.test(2) << '\n';

    a.set(0);       // 0. biti aç
```

```
    a.reset(3);     // 3. biti kapat
    a.flip(7);      // 7. biti ters çevir
    std::cout << a << ' ' << a.to_ulong() << '\n';
    return 0;
}
```

```
ÇIKTI
00001000 00001110 00000110
11110011 00110000
2 1
10000101 133
```

C#'ta `[Flags]` ile işaretlenmiş bir `enum`, C'deki bayrak maskelerinin tür güvenli karşılığıdır. Üyelerin değerleri ikinin kuvvetleri olur ve `|`, `&`, `~` operatörleri doğrudan kullanılabilir. `ToString()` açık bayrakların adlarını virgülle yazar:

▸ bayrakIzin.cs

```
using System.Numerics;

Izin izin = Izin.Oku | Izin.Yaz;
izin |= Izin.Calistir;
izin &= ~Izin.Yaz;
Console.WriteLine(izin);
Console.WriteLine(izin.HasFlag(Izin.Calistir));
Console.WriteLine((int)izin);

Console.WriteLine(Convert.ToString(12, 2).PadLeft(8, '0'));
Console.WriteLine(BitOperations.PopCount(12u));

[Flags]
enum Izin { Yok = 0, Oku = 1, Yaz = 2, Calistir = 4 }
```

```
ÇIKTI
Oku, Calistir
True
5
00001100
2
```

C#'ta bu tür bayrakları `enum` ile yazmak, sayısal sabitlerden daha okunaklıdır ve yanlış tipte değer verilmesini derleme aşamasında yakalar. `[Flags]` özelliğini Bölüm 52'de ayrıntılı göreceğiz.

Bölüm Özeti

- Bit düzeyinde işlemler bir tamsayının bitlerine tek tek dokunur. Bitler sağdan sola 0'dan başlayarak numaralanır; n. bitin değeri 2^n'dir.
- `&` (VE), `|` (VEYA), `^` (XOR) iki sayının aynı sıradaki bitlerini işler; `~` tektir ve her biti ters çevirir. `&` süzmek, `|` açmak, `^` farkı bulmak ve tersine çevirmek içindir.
- Bit operatörleri `&&`, `||` ile karıştırılmamalıdır: `12 & 10` = 8 iken `12 && 10` = 1'dir. `^` üs alma değildir.
- `a << n`, `a * 2ⁿ`; `a >> n` ise (negatif olmayan `a` için) `a / 2ⁿ` demektir. `1u << n`, yalnız n. biti 1 olan maskeyi üretir.

- Tek bit işlemleri: aç `x | m`, kapat `x & ~m`, tersle `x ^ m`, sına `(x & m) != 0`. Sınama sonucunu 1 ile değil sıfırla karşılaştırın.
- Alan okumanın kalıbı `(x >> konum) & maske`, alan yerleştirmenin kalıbı `(değer << konum) | ...`'dir. Onaltılık gösterim bit gruplarını okumayı kolaylaştırır.
- Bayraklarla birçok evet/hayır bilgisi tek sayıda tutulur. Hepsinin açık olduğunu `(x & m) == m`, en az birinin açık olduğunu `x & m` ile sınarız.
- `x & 1` tek/çift sorar; `x != 0 && (x & (x - 1)) == 0` ikinin kuvveti sorar; `x &= x - 1` en sağdaki 1 bitini siler.
- Negatif sayılar ikiye tümleyen biçiminde saklanır: `-x == ~x + 1`. İşaret biti -2^{n-1} ağırlığındadır; `-1` tüm bitleri 1 olan sayıdır.
- İşaretli sayılarda negatif değeri sola kaydırmak ve sonucu tipe sığmayan kaydırma tanımsızdır; negatif değeri sağa kaydırmak derleyiciye bağlıdır. Bit işlemlerini `unsigned` tiplerle yapın.
- `char` ve `short` değerler bit işlemlerinde de önce `int`'e yükselir; `~c` ve `c << n` sonuçlarını `(unsigned char)` ya da `& 0xFF` ile tipe geri çevirin.
- `&`, `^`, `|` karşılaştırmadan düşük önceliklidir: `x & 1 == 0` yanlıştır, `(x & 1) == 0` doğrudur. Şüphede parantez kullanın.
- C++'ta `std::bitset`, C#'ta `[Flags] enum` ve `BitOperations` bu işleri hazır araçlarla yapar.

Alıştırmalar

1. `unsigned int a = 12, b = 10;` için `a ^ b` işleminin sonucu nedir?

   A) 2
   B) 6
   C) 8
   D) 14
   E) 22

2. Aşağıdakilerden hangisi `x` değişkeninin 3. bitini (0'dan sayarak) kapatır?

   A) `x = x | (1u << 3);`
   B) `x = x & (1u << 3);`
   C) `x = x & ~(1u << 3);`
   D) `x = x ^ (1u << 3);`
   E) `x = x >> 3;`

3. `printf("%u", 3u << 4);` satırı ekrana ne yazdırır?

   A) 7
   B) 12
   C) 34
   D) 48
   E) 81

4. Aşağıdakilerden hangisi `unsigned int x` değerinin tek olup olmadığını doğru biçimde sınar?

   A) `x & 1`
   B) `x && 1`

C) `x ^ 1`
D) `x | 1`
E) `x >> 1`

5. 8 bitlik ikiye tümleyen gösterimde `11111110` hangi sayıyı gösterir?

A) 254
B) -126
C) -2
D) -1
E) -254

6. Aşağıdaki kod parçası ne yazdırır?

```
unsigned int x = 40;
int sayac = 0;
while (x != 0) {
    x &= x - 1;
    sayac++;
}
printf("%d", sayac);
```

A) 1
B) 2
C) 3
D) 5
E) 6

7. Bu programın çıktısı nedir?

```
#include <stdio.h>
int main(void) {
    unsigned int a = 6, b = 3;
    printf("%u %u %d %d\n", a & b, a | b, a && b, a == (b << 1));
    return 0;
}
```

8. Bu programın çıktısı nedir?

```
#include <stdio.h>
int main(void) {
    unsigned int izin = 0;
    izin |= (1u << 0);
    izin |= (1u << 3);
    izin ^= (1u << 0);
    izin |= (1u << 2);
    printf("%u\n", izin);
    return 0;
}
```

9. Aşağıdaki iki satır da `x`'in çift olup olmadığını sormak ister. Hangisi yanlıştır ve neden?

```
if (x & 1 == 0) { printf("çift\n"); }
if ((x & 1) == 0) { printf("çift\n"); }
```

10. `int` tipinin 32 bit olduğu bir platformda aşağıdaki ifade neden sorunludur ve nasıl düzeltilir?

```
int bayrak = 1 << 31;
```

11. Aşağıdaki kodda `bit3Acik` değişkeni 3. bit açık olduğu halde neden 0 çıkar? Düzeltin.

```
unsigned int x = 8;
int bit3Acik = ((x & (1u << 3)) == 1);
printf("%d\n", bit3Acik);
```

12. Aşağıdaki programın çıktısı nedir? `~c` ifadesi neden doğrudan 240 değildir?

```
#include <stdio.h>
int main(void) {
    unsigned char c = 0x0F;
    printf("%d %d\n", ~c, (~c) & 0xFF);
    return 0;
}
```

13. Bir `unsigned int` sayının en düşük anlamlı 4 bitini döndüren `int dusukDortBit(unsigned int x)` fonksiyonunu yazın.
14. `unsigned int` bir sayının belirli bir bitinin açık olup olmadığını sınayan ve açıksa 1, değilse 0 döndüren `int bitAcikMi(unsigned int x, int n)` fonksiyonunu yazın. Ardından `main` içinde `x = 37` için 0'dan 5'e kadar tüm bitleri sınayıp sonucu tek satırda `1 0 1 0 0 1` biçiminde yazdırın.
15. 8 bitlik bir `unsigned char` değerin bitlerini **ters sırada**, yani en düşük bitten başlayarak yazan `void tersBitYaz(unsigned char x)` fonksiyonunu yazın. `tersBitYaz(12)` çağrısı `00110000` yazdırmalıdır.
16. Üç izin bayrağı (`OKU`, `YAZ`, `SIL`) tanımlayın. `izin` değişkenini `OKU | YAZ` olarak başlatıp `SIL` bayrağını ekleyen, sonra `YAZ` bayrağını silen ve her adımdan sonra üç bayrağın durumunu `OKU=1 YAZ=0 SIL=1` biçiminde yazdıran bir program yazın.

BÖLÜM 17

Dosya İşlemleri

BU BÖLÜMDE

- Verinin neden dosyada saklandığını; metin ve ikili (binary) dosya farkını
- `FILE *`, `fopen` kiplerini, `fclose`'u ve `NULL` kontrolünü
- `fprintf`, `fputs`, `fputc` ile yazmayı; `fscanf`, `fgets`, `fgetc` ile okumayı
- `while (!feof(f))` tuzağını ve doğru okuma döngüsünü
- Satır, sözcük ve karakter sayma gibi tipik metin dosyası programlarını
- `fwrite` ve `fread` ile yapıları ikili dosyaya yazıp geri okumayı
- `fseek`, `ftell`, `rewind` ile dosyada istenen konuma gitmeyi
- `errno` ve `perror` ile dosya hatalarını bildirmeyi

Şimdiye kadar yazdığımız programların bütün verisi değişkenlerdeydi, yani RAM'de. Program bittiğinde hepsi kayboldu. Bir not listesini, bir telefon rehberini ya da bir oyunun en yüksek puanını saklamak istiyorsak veriyi **dosyaya** (file) yazmamız ve program bir sonraki çalışışında oradan okumamız gerekir.

Bu bölüm, C'nin standart kütüphanesindeki dosya işlevlerini anlatır. Konuların çoğu, Bölüm 6'da öğrendiğimiz `printf`, `scanf` ve `fgets` işlevlerinin dosya karşılıklarıdır; bu nedenle yeni olan şey çoğunlukla "nereye ya da nereden" bilgisidir.

17.1 Dosya Nedir? Metin ve İkili Dosyalar

Dosya, diskte (ya da başka bir kalıcı ortamda) bir adla saklanan bayt dizisidir. C açısından bir dosya, baştan sona okunabilen ya da yazılabilen bir **akımdır** (stream). Bütün dosya işleri aynı üç adımdan geçer: dosyayı **aç**, okuma/yazma işlemlerini yap, dosyayı **kapat**.

Dosyalar içerik bakımından ikiye ayrılır:

	Metin dosyası (text file)	İkili dosya (binary file)
İçerik	Okunabilir karakterler (`.txt`, `.csv`, `.c`)	Bellekteki baytların kopyası (`.bin`, `.dat`)
Bir düzenleyiciyle açınca	Okunur	Anlamsız semboller görünür
Başlıca işlevler	`fprintf`, `fscanf`, `fgets`, `fputs`	`fwrite`, `fread`
`int` değer 12345	5 karakter (5 bayt)	Bellekteki 4 bayt

Metin dosyası insanların da okuyabildiği biçimdir ve hata ayıklaması kolaydır. İkili dosya daha az yer tutar, bellekteki veriyi hiçbir dönüşüm yapmadan kopyaladığı için hızlıdır, ama yalnızca onu yazan programın (ve aynı platformun) anlayacağı bir biçimdir. Önce metin dosyalarıyla başlayacağız.

17.2 FILE, fopen ve fclose

`FILE`, `<stdio.h>` içinde tanımlı bir yapı tipidir. İçinde dosyanın durumuna ilişkin bilgiler (konum, arabellek, hata bayrağı) bulunur; biz içine bakmayız, yalnızca ona bir **işaretçi** tutarız (Bölüm 12). Dosyayı açan işlev `fopen`'dır:

> **Tanım: fopen**
>
> `FILE *fopen(const char *ad, const char *kip)` işlevi, adı verilen dosyayı belirtilen kipte açar ve dosyayı temsil eden bir `FILE *` döndürür. Dosya açılamazsa `NULL` döndürür.

```
FILE *f = fopen("notlar.txt", "r");
```

İkinci parametre olan **kip** (mode), dosyayla ne yapacağımızı söyler:

Kip	Anlamı	Dosya yoksa	Dosya varsa
`"r"`	Yalnızca okuma	`NULL` döner	Baştan okur
`"w"`	Yalnızca yazma	Oluşturur	İçeriği siler (0 bayt olur)
`"a"`	Sona ekleme (append)	Oluşturur	İçeriği korur, hep sona yazar
`"r+"`	Okuma ve yazma	`NULL` döner	İçeriği korur
`"w+"`	Okuma ve yazma	Oluşturur	İçeriği siler
`"a+"`	Okuma ve sona ekleme	Oluşturur	İçeriği korur

Kipin sonuna `b` eklenirse (`"rb"`, `"wb"`, `"r+b"`) dosya ikili olarak açılır. Linux ve macOS'ta metin ve ikili kip arasında fark yoktur; Windows'ta vardır (bu bölümün sonunda göreceğiz). İkili dosyalarda her zaman `b` yazmak taşınabilir koddur.

> **▲ Sık Yapılan Hata · "w" kipi dosyayı hemen siler**
>
> `"w"` ile açılan var olan bir dosya, `fopen` döner dönmez sıfır bayta indirilir; henüz tek karakter yazmamış olsak bile. Eski içeriği korumak istiyorsak `"a"` ya da `"r+"` kullanmalıyız.

Dosyayla işimiz bitince `fclose(f)` ile kapatırız. Kapatma yalnızca kaynağı geri vermez; arabellekte bekleyen verinin diske yazılmasını da sağlar. Kapatılmadan program çökerse son yazılanlar kaybolabilir.

Her fopen'dan sonra NULL kontrolü

Dosya açma, programın kontrolü dışındaki nedenlerle başarısız olabilir: dosya yoktur, klasör yoktur, izin yoktur, disk doludur. `fopen` bu durumda `NULL` döndürür ve `NULL` işaretçiyle yapılan okuma/yazma programı çökertir. Bu yüzden her `fopen` çağrısının hemen ardından denetim yazılır:

▸ ilkDosya.c

```
#include <stdio.h>

int main(void) {
    FILE *f = fopen("selam.txt", "w");
    if (f == NULL) {
        printf("Dosya açılamadı.\n");
```

```
        return 1;
    }
    fprintf(f, "Merhaba dosya!\n");
    fprintf(f, "Sayı: %d\n", 42);
    fclose(f);

    f = fopen("selam.txt", "r");
    if (f == NULL) {
        printf("Dosya açılamadı.\n");
        return 1;
    }
    char satir[100];
    while (fgets(satir, sizeof satir, f) != NULL) {
        printf("Okunan: %s", satir);
    }
    fclose(f);
    return 0;
}
```

```
ÇIKTI
Okunan: Merhaba dosya!
Okunan: Sayı: 42
```

Program önce `selam.txt` dosyasını yazma kipinde açıp iki satır yazdı, kapattı, sonra aynı dosyayı okuma kipinde açıp satır satır ekrana döktü. `fgets` satır sonu karakterini (`\n`) de okuduğu için `printf` biçiminde ayrıca `\n` yazmadık.

Çalışma dizininde ortaya çıkan `selam.txt` dosyasını herhangi bir metin düzenleyiciyle açıp içine bakabiliriz. Dosya adı yalnızca ad olarak verildiğinde, program hangi klasörden çalıştırıldıysa o klasörde aranır; klasör yolları bu bölümün sonunda anlatılacak.

Açılamama nedeni: perror ve errno

`NULL` döndüğünü bilmek yetmez, nedenini de bilmek isteriz. Bir sistem çağrısı başarısız olduğunda C kütüphanesi hata kodunu `<errno.h>` içindeki `errno` değişkenine yazar. `<string.h>` içindeki `strerror(errno)` bu kodu okunabilir bir metne çevirir; `perror("başlık")` ise aynı metni "başlık: neden" biçiminde **hata akımına** (`stderr`) yazar.

▸ hataNedeni.c

```c
#include <stdio.h>
#include <string.h>
#include <errno.h>

int main(void) {
    FILE *f = fopen("olmayanDosya.txt", "r");
    if (f == NULL) {
        printf("Hata kodu: %d\n", errno);
        printf("Neden: %s\n", strerror(errno));
        perror("olmayanDosya.txt");
        return 1;
    }
    fclose(f);
    return 0;
}
```

```
ÇIKTI
Hata kodu: 2
Neden: No such file or directory
```

`perror` satırı `stderr`'e yazıldığı için çıktı bloğunda görünmüyor; terminalde `olmayanDosya.txt: No such file or directory` satırı ayrıca belirir. Hata kodunun sayısal değeri sistemden sisteme değişebilir, metni de dile ve sisteme bağlıdır; bu yüzden kodu sayıyla karşılaştırmak yerine `errno.h` içindeki adlarla (`ENOENT`: dosya yok, `EACCES`: izin yok) karşılaştırırız. `errno` yalnızca bir işlev **başarısız olduğunda** anlamlıdır; başarılı çağrılar onu sıfırlamaz.

Kitabın geri kalanında hata iletisi için `perror`'ı kullanacağız; kısa ve kullanıcıya doğru bilgiyi verir.

17.3 Metin Dosyasına Yazmak

Dosyaya yazan üç temel işlev vardır. Üçü de ekrana yazan arkadaşlarının başına bir `f` ve bir `FILE *` parametresi almış hâlidir:

Ekrana	Dosyaya	Ne yapar
`printf(...)`	`fprintf(f, ...)`	Biçimli yazar
`puts(s)`	`fputs(s, f)`	Metni yazar; **satır sonu eklemez**
`putchar(c)`	`fputc(c, f)`	Tek karakter yazar

`puts` ile `fputs` arasındaki fark dikkatimizi çekmeli: `puts` sonuna `\n` ekler, `fputs` eklemez. Satır sonunu kendimiz yazmalıyız.

▸ yazmaKipleri.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("liste.txt", "w");
    if (f == NULL) {
        perror("liste.txt");
        return 1;
    }
    fputs("Meyveler\n", f);
    fputc('-', f);
    fputc(' ', f);
    fputs("elma\n", f);
    fprintf(f, "- %s (%d adet)\n", "armut", 3);
    fclose(f);

    // "a" kipi: var olan dosyanın sonuna ekler
    f = fopen("liste.txt", "a");
    if (f == NULL) {
        perror("liste.txt");
        return 1;
    }
    fprintf(f, "- %s (%d adet)\n", "kiraz", 10);
    fclose(f);

    f = fopen("liste.txt", "r");
    if (f == NULL) {
        perror("liste.txt");
        return 1;
```

```
    }
    int c;
    while ((c = fgetc(f)) != EOF) {
        putchar(c);
    }
    fclose(f);
    return 0;
}
```

```
ÇIKTI
Meyveler
- elma
- armut (3 adet)
- kiraz (10 adet)
```

Son döngü, bir sonraki başlıkta ayrıntılı göreceğimiz `fgetc` ile dosyayı karakter karakter okuyup ekrana yazar. `"a"` kipi sayesinde `kiraz` satırı öncekilerin silinmesine yol açmadan dosyanın sonuna eklendi. Bu, günlük (log) dosyalarının tipik çalışma biçimidir.

> **İpucu · Çıktıyı hem dosyaya hem ekrana yazan kod**
>
> `fprintf` işlevinin ilk parametresi bir `FILE *` olduğu için aynı yazma kodu, `FILE *` yerine `stdout` verilerek ekrana da yazdırabilir: `fprintf(stdout, "...")` ile `printf("...")` aynı şeydir. Bu özelliği "Standart Akımlar" başlığında kullanacağız.

17.4 Metin Dosyasından Okumak

Okuma işlevleri de yazma işlevlerinin karşılığıdır:

Klavyeden	Dosyadan	Ne yapar
`scanf(...)`	`fscanf(f, ...)`	Biçime göre okur; boşlukta durur
`fgets(s, n, stdin)`	`fgets(s, n, f)`	En fazla `n - 1` karakterlik bir satır okur
`getchar()`	`fgetc(f)`	Tek karakter okur

Üçünün de dosyanın sonuna geldiğinde verdiği işaret farklıdır. `fgetc`, dosya bittiğinde `EOF` sabitini (genellikle -1) döndürür. `fgets`, okuyacak bir şey kalmadığında `NULL` döndürür. `fscanf` ise başarıyla doldurduğu öğe sayısını döndürür; dosya bittiyse `EOF` verir.

> **◆ Kritik Nokta · fgetc'nin dönüş tipi int olmalıdır**
>
> `fgetc`, 0-255 arasındaki karakter kodlarını ve bunlardan ayırt edilebilmesi için -1 olan `EOF`'u aynı dönüş değerinde taşır. Dönüş değerini `char` değişkene koyarsak `EOF` ile gerçek bir karakter karışabilir ve döngü ya hiç bitmez ya da erken biter. Bu nedenle okunan değeri her zaman `int` türünde saklarız.

fscanf ile biçimli okuma

`fscanf`, `scanf` gibi çalışır; beyaz boşluk (boşluk, sekme, satır sonu) görünce değeri bitirir ve sonrakine geçer. Önce örnek bir not dosyası yazıp sonra okuyalım:

notOkuma.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("notlar.txt", "w");
    if (f == NULL) {
        perror("notlar.txt");
        return 1;
    }
    fprintf(f, "Ayşe 85\nMehmet 70\nElif 92\n");
    fclose(f);

    f = fopen("notlar.txt", "r");
    if (f == NULL) {
        perror("notlar.txt");
        return 1;
    }
    char ad[30];
    int not1;
    int toplam = 0;
    int adet = 0;
    while (fscanf(f, "%29s %d", ad, &not1) == 2) {
        printf("%-8s %3d\n", ad, not1);
        toplam += not1;
        adet++;
    }
    fclose(f);
    printf("Ortalama: %.2f\n", (double)toplam / adet);
    return 0;
}
```

ÇIKTI

```
Ayşe     85
Mehmet    70
Elif      92
Ortalama: 82.33
```

Döngünün koşulu `fscanf(...) == 2` biçimindedir: işlev iki öğeyi (bir metin ve bir tamsayı) başarıyla okuduğu sürece sürer. Dosya bittiğinde ya da biçim uymadığında 2'den farklı bir değer döner ve döngü sona erer. `%29s`, 30 elemanlı `ad` dizisinin taşmasını önler (Bölüm 11); çıktıdaki hizalama ise `%-8s` ile `%3d` biçimlerinden gelir. ("Ayşe" satırı bir boşluk eksik hizalanmış görünür: `ş` UTF-8'de iki bayttır ve `printf` genişliği bayt cinsinden sayar; bu konuya bölümün sonunda döneceğiz.)

> ◆ **Kritik Nokta · fscanf'in dönüş değerini her zaman denetleyin**
>
> `fscanf` her zaman istediğimiz sayıda öğeyi okuyamayabilir. Dönüş değeri denetlenmeden kullanılan `fscanf`, dosya bozuk ya da bitmiş olsa bile eski değerlerle çalışmaya devam eder. Hem sonlandırma koşulu hem hata denetimi için dönüş değerini beklenen öğe sayısıyla karşılaştırırız.

fgets ile satır satır okuma

Boşluk içeren metinler (ad soyad, adres, cümle) için `fscanf("%s")` uygun değildir; ilk boşlukta durur. Bu durumlarda satırı bütün olarak `fgets` ile alırız. `fgets`, satır sonu karakterini de dizinin içine koyar; çoğu zaman bunu silmek isteriz.

satirOkuma.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *f = fopen("siir.txt", "w");
    if (f == NULL) {
        perror("siir.txt");
        return 1;
    }
    fputs("Bir varmış bir yokmuş\n", f);
    fputs("\n", f);
    fputs("Evvel zaman içinde", f);   // son satırda \n yok
    fclose(f);

    f = fopen("siir.txt", "r");
    if (f == NULL) {
        perror("siir.txt");
        return 1;
    }
    char satir[80];
    int no = 0;
    while (fgets(satir, sizeof satir, f) != NULL) {
        satir[strcspn(satir, "\n")] = '\0';   // satır sonunu sil
        no++;
        printf("%d: [%s]\n", no, satir);
    }
    fclose(f);
    return 0;
}
```

```
ÇIKTI
1: [Bir varmış bir yokmuş]
2: []
3: [Evvel zaman içinde]
```

`strcspn(satir, "\n")`, satırda ilk `\n` karakterinin konumunu verir; o konuma `'\0'` yazmak satır sonunu siler. Son satırda `\n` bulunmasa bile `strcspn` metnin sonunu döndürür, dolayısıyla kod her durumda güvenlidir. `fgets`'in ikinci parametresi `sizeof satir` olduğu için dizi boyutu değişirse kodu ayrıca düzeltmemiz gerekmez.

Bir satır dizinin alabileceğinden uzunsa `fgets` onu parçalar halinde döndürür; kalan kısım sonraki çağrıda gelir. Satır uzunluğunun sınırsız olabileceği programlarda bu durum ayrıca düşünülmelidir.

17.5 EOF ve feof Tuzağı

Yeni başlayanların neredeyse hepsi dosya sonunu şu kalıpla yakalamaya çalışır:

```c
while (!feof(f)) {          // YANLIŞ
    fscanf(f, "%d", &x);
    printf("%d\n", x);
}
```

Bu kalıp neden yanlıştır? `feof(f)`, dosya sonu **bir okuma girişimi başarısız olduktan sonra** doğru olur; son veriyi okuduktan hemen sonra henüz doğru değildir. Sonuç olarak döngü bir kez fazladan döner. Gözlemleyelim:

feofTuzagi.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("sayilar.txt", "w");
    if (f == NULL) {
        perror("sayilar.txt");
        return 1;
    }
    fprintf(f, "10\n20\n30\n");
    fclose(f);

    f = fopen("sayilar.txt", "r");
    if (f == NULL) {
        perror("sayilar.txt");
        return 1;
    }
    int x = 0;
    printf("Yanlış kalıp:\n");
    while (!feof(f)) {
        fscanf(f, "%d", &x);
        printf("%d\n", x);
    }
    fclose(f);

    f = fopen("sayilar.txt", "r");
    if (f == NULL) {
        perror("sayilar.txt");
        return 1;
    }
    printf("Doğru kalıp:\n");
    while (fscanf(f, "%d", &x) == 1) {
        printf("%d\n", x);
    }
    fclose(f);
    return 0;
}
```

ÇIKTI

```
Yanlış kalıp:
10
20
30
30
Doğru kalıp:
10
20
30
```

Yanlış kalıpta üçüncü okumadan sonra dosyada hâlâ son `\n` karakteri vardır, `feof` yanlış (0) döner ve döngü bir tur daha çalışır. Bu turda `fscanf` hiçbir şey okuyamaz, `x` eski değerini korur ve `30` iki kez yazdırılır. Doğru kalıpta döngünün koşulu zaten okuma işleminin kendisidir.

> ◆ **Kritik Nokta · Okuma işleminin dönüş değeri döngü koşulu olmalıdır**
>
> Dosya sonunu `feof` ile önceden tahmin etmeye çalışmayın. Okuma işlevinin kendisini koşula yazın: `while (fscanf(...) == beklenen)`, `while (fgets(...) != NULL)` ya da `while ((c = fgetc(f)) != EOF)`. `feof` ve `ferror` yalnızca döngüden **sonra**, döngünün neden bittiğini anlamak için kullanılır.

17.6 Metin Dosyası Programları

Şimdiye kadarki işlevlerle işe yarar araçlar yazabiliriz. İlk örnek, Unix'teki `wc` komutunun basit bir benzeridir: dosyadaki karakter, sözcük ve satır sayısını bulur. Sözcük, boşluk olmayan karakterlerden oluşan bir dizidir; bir sözcüğün içinde olup olmadığımızı bir bayrakla izleriz.

▸ sayac.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    FILE *f = fopen("metin.txt", "w");
    if (f == NULL) {
        perror("metin.txt");
        return 1;
    }
    fputs("C dili\n1972 yılında\ndoğdu.\n", f);
    fclose(f);

    f = fopen("metin.txt", "r");
    if (f == NULL) {
        perror("metin.txt");
        return 1;
    }
    int karakter = 0, sozcuk = 0, satir = 0;
    int icerde = 0;                       // bir sözcüğün içinde miyiz?
    int c;
    while ((c = fgetc(f)) != EOF) {
        karakter++;
        if (c == '\n') {
            satir++;
        }
        if (isspace(c)) {
            icerde = 0;
        } else if (!icerde) {
            icerde = 1;
            sozcuk++;
        }
    }
    fclose(f);
    printf("Karakter: %d\nSözcük: %d\nSatır: %d\n", karakter, sozcuk, satir);
    return 0;
}
```

```
ÇIKTI
Karakter: 30
Sözcük: 5
Satır: 3
```

`karakter` sayısı baytları sayar: `ı` ve `ğ` gibi Türkçe harfler UTF-8'de iki bayt tuttuğundan, 24 görünen karakter, 3 satır sonu ve 3 fazladan bayt (iki `ı`, bir `ğ`) toplamı 30 yaptı. `isspace` boşluk, sekme ve satır sonu için doğru döner (`<ctype.h>`, Bölüm 11).

İkinci örnek, bir dosyayı başka bir dosyaya kopyalayan ve her satırın başına numara ekleyen bir programdır. Bu tür "oku, dönüştür, yaz" döngüleri dosya işlemlerinin en yaygın biçimidir:

numaraliKopya.c

```c
#include <stdio.h>

int main(void) {
    FILE *kaynak = fopen("kaynak.txt", "w");
    if (kaynak == NULL) {
        perror("kaynak.txt");
        return 1;
    }
    fputs("alfa\nbeta\ngama\n", kaynak);
    fclose(kaynak);

    kaynak = fopen("kaynak.txt", "r");
    FILE *hedef = fopen("hedef.txt", "w");
    if (kaynak == NULL || hedef == NULL) {
        perror("dosya");
        if (kaynak != NULL) fclose(kaynak);
        if (hedef != NULL) fclose(hedef);
        return 1;
    }
    char satir[100];
    int no = 1;
    while (fgets(satir, sizeof satir, kaynak) != NULL) {
        fprintf(hedef, "%02d| %s", no, satir);
        no++;
    }
    fclose(kaynak);
    fclose(hedef);

    hedef = fopen("hedef.txt", "r");
    if (hedef == NULL) {
        perror("hedef.txt");
        return 1;
    }
    while (fgets(satir, sizeof satir, hedef) != NULL) {
        fputs(satir, stdout);
    }
    fclose(hedef);
    return 0;
}
```

ÇIKTI

```
01| alfa
02| beta
03| gama
```

İki dosya birlikte açıldığında biri açılıp diğeri açılamazsa, açılmış olanı kapatmayı unutmamalıyız; programda bunu iki `if` ile yaptık. Küçük programlarda işletim sistemi program biterken dosyaları zaten kapatır, ama uzun süre çalışan programlarda unutulan her `fclose` bir kaynak sızıntısıdır.

17.7 Standart Akımlar ve Yönlendirme

Program başlarken C, bizim için üç akımı hazır açar. Üçü de `FILE *` türündedir ve `<stdio.h>` içinde tanımlıdır:

Akım	Varsayılan bağlantı	Kullanım
`stdin`	Klavye	Girdi (`scanf`, `fgets(..., stdin)`)

Akım	Varsayılan bağlantı	Kullanım
`stdout`	Ekran	Normal çıktı (`printf` , `puts`)
`stderr`	Ekran	Hata iletileri (`perror` , `fprintf(stderr, ...)`)

Bu üçü sıradan dosyalar gibi davranır: `printf("x")` ile `fprintf(stdout, "x")` aynıdır, `scanf` ile `fscanf(stdin, ...)` aynıdır. Bu da bir işlevi hem ekrana hem dosyaya çalışacak biçimde yazabilmemizi sağlar; işleve hedef `FILE *` olarak verilir:

akimlar.c

```c
#include <stdio.h>

void rapor(FILE *hedef, int adet, double toplam) {
    fprintf(hedef, "Adet: %d\n", adet);
    fprintf(hedef, "Toplam: %.2f\n", toplam);
}

int main(void) {
    rapor(stdout, 3, 125.5);           // ekrana

    FILE *f = fopen("rapor.txt", "w");
    if (f == NULL) {
        perror("rapor.txt");
        return 1;
    }
    rapor(f, 3, 125.5);                // dosyaya
    fclose(f);

    fprintf(stderr, "Uyarı: bu satır hata akımına gider.\n");
    return 0;
}
```

ÇIKTI

```
Adet: 3
Toplam: 125.50
```

`stdout` ve `stderr` ikisi de varsayılan olarak ekrana bağlıdır, ama **ayrı akımlardır**. Neden iki ayrı akım var? Çünkü çıktıyı bir dosyaya yönlendirdiğimizde yalnızca normal sonuçların dosyaya gitmesini, hata iletilerinin ise ekranda görünmeye devam etmesini isteriz.

Terminalde yönlendirme

Komut satırı kabukları (Linux ve macOS'ta bash, Windows'ta Komut İstemi ve PowerShell) bir programın akımlarını programa dokunmadan dosyalara bağlayabilir:

TERMİNAL

```
$ ./akimlar > cikti.txt
Uyarı: bu satır hata akımına gider.
$ ./akimlar 2> hatalar.txt
Adet: 3
Toplam: 125.50
$ ./sayac < metin.txt
$ ./akimlar >> gunluk.txt
```

`>` standart çıktıyı dosyaya yönlendirir (var olan içeriği siler), `>>` sona ekler, `2>` standart hata akımını yönlendirir, `<` bir dosyayı standart girdi yapar. Programı `stdin`'den okuyacak biçimde yazarsak, yönlendirme

sayesinde aynı program hem klavyeyle hem dosyayla çalışır. Bu yüzden basit araçları dosya adı sormak yerine `stdin`'den okuyan biçimde yazmak yaygın bir Unix alışkanlığıdır.

17.8 İkili Dosyalar: fwrite ve fread

Metin dosyasında `12345` sayısı beş karakter olarak yazılır ve okunurken yeniden sayıya çevrilir. Bu hem yer kaplar hem de zaman alır. İkili dosyada bellekteki baytlar olduğu gibi diske kopyalanır. Bu iş için iki işlev vardır:

> **Tanım: fwrite ve fread**
>
> `size_t fwrite(const void *veri, size_t boyut, size_t adet, FILE *f)` bellekteki `veri` adresinden başlayarak, her biri `boyut` bayt olan `adet` öğeyi dosyaya yazar. `size_t fread(void *veri, size_t boyut, size_t adet, FILE *f)` aynı biçimde okur. İkisi de **başarıyla aktarılan öğe sayısını** döndürür.

Dönüş değerini beklenen `adet` ile karşılaştırmak, hem hata hem de dosya sonu denetiminin bir arada yapılmasını sağlar. Önce basit bir tamsayı dizisi yazıp okuyalım:

▸ ikiliSayi.c

```c
#include <stdio.h>

int main(void) {
    int sayilar[5] = {10, 20, 30, 40, 50};

    FILE *f = fopen("sayilar.bin", "wb");
    if (f == NULL) {
        perror("sayilar.bin");
        return 1;
    }
    size_t yazilan = fwrite(sayilar, sizeof(int), 5, f);
    fclose(f);
    printf("Yazılan öğe: %zu\n", yazilan);

    int okunan[5];
    f = fopen("sayilar.bin", "rb");
    if (f == NULL) {
        perror("sayilar.bin");
        return 1;
    }
    size_t adet = fread(okunan, sizeof(int), 5, f);
    fclose(f);
    printf("Okunan öğe: %zu\n", adet);
    for (size_t i = 0; i < adet; i++) {
        printf("%d ", okunan[i]);
    }
    printf("\n");
    return 0;
}
```

```
ÇIKTI
Yazılan öğe: 5
Okunan öğe: 5
10 20 30 40 50
```

Dizinin adı bir adres olduğu için `fwrite`'a `&` olmadan verildi (Bölüm 10). Tek bir değişken yazarken adresini alırız: `fwrite(&x, sizeof x, 1, f)`. Dosyanın içine bir hex görüntüleyici ile bakarsak 20 baytlık bir içerik

görürüz: her `int` için 4 bayt. Aynı sayıları metin olarak yazsaydık (`10 20 30 40 50\n`) 15 bayt tutardı; ikili biçim burada büyük görünür, ama büyük sayılarda ve ondalıklı değerlerde avantaj tersine döner.

Yapıları dosyaya yazmak

İkili dosyaların asıl gücü, bir yapıyı (Bölüm 14) tek işlemle yazıp okumaktır. Bir öğrenci kaydını ele alalım:

▸ ogrenciDosya.c

```c
#include <stdio.h>
#include <string.h>

struct Ogrenci {
    char ad[20];
    int no;
    double ortalama;
};

int main(void) {
    struct Ogrenci liste[3] = {
        {"Ayşe", 101, 85.5},
        {"Mehmet", 102, 72.25},
        {"Elif", 103, 91.0}
    };

    FILE *f = fopen("ogrenciler.dat", "wb");
    if (f == NULL) {
        perror("ogrenciler.dat");
        return 1;
    }
    fwrite(liste, sizeof(struct Ogrenci), 3, f);
    fclose(f);

    struct Ogrenci o;
    f = fopen("ogrenciler.dat", "rb");
    if (f == NULL) {
        perror("ogrenciler.dat");
        return 1;
    }
    printf("Kayıt boyutu: %zu bayt\n", sizeof(struct Ogrenci));
    while (fread(&o, sizeof o, 1, f) == 1) {
        printf("%-8s %d %6.2f\n", o.ad, o.no, o.ortalama);
    }
    fclose(f);
    return 0;
}
```

ÇIKTI

```
Kayıt boyutu: 32 bayt
Ayşe    101  85.50
Mehmet   102  72.25
Elif     103  91.00
```

`fread(&o, sizeof o, 1, f) == 1` okuma döngüsünün doğru kalıbıdır: kayıt tam okunabildiği sürece sürer, dosya bittiğinde 0 döner. Bu kalıp, `feof` tuzağından da korunmanın ikili dosya karşılığıdır.

> **▲ Sık Yapılan Hata · İkili dosyalar taşınabilir değildir**
>
> Bir yapıyı olduğu gibi diske yazdığımızda, derleyicinin yerleştirdiği **dolgu baytları** (padding, Bölüm 14), `int` ve `long` boyutları ve bayt sıralaması (little-endian / big-endian) da dosyaya girer. Aynı dosya farklı bir derleyici ya da işlemciyle derlenen programda farklı okunabilir. Ayrıca yapının içinde işaretçi varsa (`char *ad`), dosyaya yazılan şey metin değil bir adrestir; program bir sonraki çalışışında bu adres anlamsızdır. Bu yüzden yapıyı ikili yazarken içine yalnızca sabit boyutlu diziler ve sayılar koyarız. Dosya başka sistemlerle paylaşılacaksa metin biçimi (CSV, JSON) daha güvenlidir.

17.9 Dosyada Gezinmek: fseek, ftell, rewind

Dosyayı açtığımızda bir **dosya konumu göstergesi** (file position indicator) vardır. Her okuma ve yazma, bu göstergenin bulunduğu yerden başlar ve gösterge aktarılan bayt kadar ilerler. Bu göstergeyi biz de yönetebiliriz:

İşlev	Ne yapar
`ftell(f)`	Göstergenin başlangıçtan bayt cinsinden uzaklığını verir (`long`); hata olursa -1
`fseek(f, uzaklik, orijin)`	Göstergeyi `orijin` 'e göre `uzaklik` bayt kaydırır; başarıda 0 döner
`rewind(f)`	Göstergeyi dosyanın başına alır

`fseek` 'in üçüncü parametresi, kaydırmanın neye göre yapılacağını belirler:

Orijin	Anlamı
`SEEK_SET`	Dosyanın başı
`SEEK_CUR`	Göstergenin o anki konumu
`SEEK_END`	Dosyanın sonu

İki klasik kullanımı vardır. Birincisi, dosya boyutunu bulmaktır: sona git, `ftell` ile konumu oku. İkincisi, kayıtlı bir dosyada n'inci kayda doğrudan **rastgele erişim** yapmaktır: kayıt boyutu sabit olduğundan, n'inci kaydın dosyadaki başlangıcı `n * sizeof(kayıt)` baytır. Bunu, bir önceki programın yazdığı dosya üzerinde deneyelim; aynı program bir kaydı yerinde güncelleyecek:

▸ rastgeleErisim.c

```c
#include <stdio.h>

struct Ogrenci {
    char ad[20];
    int no;
    double ortalama;
};

int main(void) {
    struct Ogrenci liste[3] = {
        {"Ayşe", 101, 85.5},
        {"Mehmet", 102, 72.25},
        {"Elif", 103, 91.0}
    };
    FILE *f = fopen("kayit.dat", "wb");
    if (f == NULL) {
        perror("kayit.dat");
        return 1;
    }
```

```
    fwrite(liste, sizeof(struct Ogrenci), 3, f);
    fclose(f);

    f = fopen("kayit.dat", "r+b");          // okuma + yazma, ikili
    if (f == NULL) {
        perror("kayit.dat");
        return 1;
    }

    fseek(f, 0, SEEK_END);
    long boyut = ftell(f);
    printf("Dosya boyutu: %ld bayt\n", boyut);
    printf("Kayıt sayısı: %ld\n", boyut / (long)sizeof(struct Ogrenci));

    struct Ogrenci o;
    fseek(f, 1 * (long)sizeof o, SEEK_SET);    // 2. kayıt (indeks 1)
    fread(&o, sizeof o, 1, f);
    printf("2. kayıt: %s %.2f\n", o.ad, o.ortalama);

    o.ortalama = 80.0;                         // güncelle
    fseek(f, 1 * (long)sizeof o, SEEK_SET);    // aynı kayda geri dön
    fwrite(&o, sizeof o, 1, f);

    rewind(f);
    while (fread(&o, sizeof o, 1, f) == 1) {
        printf("%s %.2f\n", o.ad, o.ortalama);
    }
    fclose(f);
    return 0;
}
```

```
ÇIKTI
Dosya boyutu: 96 bayt
Kayıt sayısı: 3
2. kayıt: Mehmet 72.25
Ayşe 85.50
Mehmet 80.00
Elif 91.00
```

Güncelleme sırasında önemli bir ayrıntı vardı: `fread` göstergeyi kaydın sonuna taşıdığı için yazmadan önce `fseek` ile tekrar kaydın başına döndük. Unutursak yeni değer yanlış kayda, bir sonrakinin üstüne yazılırdı.

> **◆ Kritik Nokta · Okuma ve yazma arasında fseek gerekir**
>
> Bir dosyayı `+` kipiyle açıp okumadan yazmaya (ya da yazmadan okumaya) geçerken araya bir `fseek`, `rewind` ya da `fflush` çağrısı koymak zorundayız; standart bunu şart koşar. Yoksa davranış tanımsızdır.

`fseek` ve `ftell` ikili dosyalar içindir. Metin dosyalarında konum, platformun satır sonu dönüşümleri yüzünden bayt sayısı olmayabilir; bu yüzden metin dosyalarında yalnızca `ftell` ile aldığımız bir değeri yine `fseek(f, değer, SEEK_SET)` ile geri vermek ve `0` ile `SEEK_END` kullanmak güvenlidir. Çok büyük (2 GB üstü) dosyalarda `long` yetmeyebilir; bu durumlarda sisteme özgü `fseeko`/`ftello` ya da `_fseeki64` gibi işlevler kullanılır.

17.10 Dosya Hatalarını Yönetmek

Dosya işlemleri programın dışındaki dünyaya bağlıdır ve her an başarısız olabilir. Şimdiye kadar yalnızca `fopen`'ın `NULL` dönmesini denetledik. Tam bir hata yönetiminde şu noktalara bakılır:

- `fopen`: `NULL` dönerse neden `perror` ile bildirilir.
- `fread` ve `fwrite`: dönüş değeri beklenen öğe sayısından küçükse ya hata ya da dosya sonu vardır. Hangisi olduğunu `feof(f)` ve `ferror(f)` söyler.
- `fclose`: yazılan dosyalarda arabellekteki veri kapatma sırasında yazıldığı için disk dolması gibi hatalar `fclose`'un 0 dışında bir değer döndürmesine yol açabilir.

▸ hataYonetimi.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("kisa.bin", "wb");
    if (f == NULL) {
        perror("kisa.bin");
        return 1;
    }
    int veri[2] = {7, 8};
    if (fwrite(veri, sizeof(int), 2, f) != 2) {
        perror("yazma");
        fclose(f);
        return 1;
    }
    if (fclose(f) != 0) {
        perror("kapatma");
        return 1;
    }

    f = fopen("kisa.bin", "rb");
    if (f == NULL) {
        perror("kisa.bin");
        return 1;
    }
    int okunan[4];
    size_t n = fread(okunan, sizeof(int), 4, f);   // 4 istiyoruz, 2 var
    printf("İstenen: 4, okunan: %zu\n", n);
    if (feof(f)) {
        printf("Dosya sonuna gelindi.\n");
    } else if (ferror(f)) {
        printf("Okuma hatası.\n");
    }
    fclose(f);
    return 0;
}
```

```
ÇIKTI
İstenen: 4, okunan: 2
Dosya sonuna gelindi.
```

Burada `feof` doğru yerde, yani başarısız bir okumanın **ardından**, nedeni anlamak için kullanıldı. Ayrıca dosyayı silmek ve yeniden adlandırmak için `<stdio.h>` içinde `remove("ad")` ve `rename("eski", "yeni")` işlevleri vardır; ikisi de başarıda 0 döndürür.

17.11 Platform Farkları: Satır Sonu, Yollar, Kodlama

Satır sonları

Metin dosyalarında satır sonu platforma göre farklı baytlarla gösterilir: Linux ve macOS'ta tek bir `\n` (LF), Windows'ta `\r\n` (CRLF). C programı her platformda `\n` yazar. Windows'ta metin kipindeki akım, yazarken `\n`'yi `\r\n`'ye, okurken `\r\n`'yi `\n`'ye **kendiliğinden çevirir**. İkili kipte (`"wb"` , `"rb"`) bu çeviri yapılmaz.

Sonuç olarak şunları söyleyebiliriz:

- Metin dosyası `"r"` / `"w"` ile, ikili dosya `"rb"` / `"wb"` ile açılmalıdır. Bir ikili dosyayı Windows'ta `"w"` ile yazarsak, veri içindeki 10 değerli her bayt (`\n`) iki bayta dönüşür ve dosya bozulur.
- Windows'ta oluşturulmuş bir metin dosyasını Linux'ta okurken satır sonunda fazladan `\r` karakteri kalabilir. Programın bu karaktere dayanıklı olması gerekiyorsa `strcspn(satir, "\r\n")` ile ikisini birden silebiliriz.
- İkili dosyalarda `fseek` / `ftell` bayt konumunu tam olarak verir; metin dosyalarında bu garanti yoktur.

Dosya yolları

Dosya adı olarak yalnızca ad verirsek, program hangi klasörden başlatıldıysa orada arar (**çalışma dizini**). Başka bir klasördeki dosyayı yol (path) ile belirtiriz. Windows'ta ayırıcı ters eğik çizgidir, ve C metinlerinde ters eğik çizgi kaçış karakteri olduğu için iki kez yazılması gerekir:

```
FILE *a = fopen("C:\\Belgelerim\\notlar.txt", "r");   // Windows
FILE *b = fopen("C:/Belgelerim/notlar.txt", "r");     // Windows'ta da çalışır
FILE *c = fopen("/home/ayse/notlar.txt", "r");        // Linux, macOS
FILE *d = fopen("veri/notlar.txt", "r");              // göreli yol
```

Windows API'si de `/` ayırıcısını kabul ettiği için taşınabilir kodda `/` kullanmak çoğu zaman en kolay yoldur. `..\\notlar.txt` gibi yolları yazarken tek bir `\` karakteri yazmak ise `\n` ya da `\t` gibi istenmeyen bir kaçış dizisi oluşturabilir.

Türkçe karakterli dosyalar

Bölüm 11'de gördüğümüz gibi `char` , tek bir baytı tutar. Modern sistemlerde metin dosyalarının çoğu **UTF-8** ile kodlanır; bu kodlamada `ç` , `ğ` , `ı` , `ö` , `ş` , `ü` ve büyük karşılıkları iki bayt yer kaplar. Dosya işlemleri açısından bunun sonuçları şöyledir:

- `fputs` , `fprintf` , `fgets` bayt akışını olduğu gibi taşır; Türkçe metni UTF-8 olarak yazan bir program, UTF-8 okuyan bir düzenleyicide doğru görünür. Bu bölümdeki örnekler bu yüzden sorunsuz çalıştı.
- `fgetc` karakteri değil bayt okur; `ş` iki ayrı çağrıda iki bayt olarak gelir. Karakter saymak ya da büyük/küçük harf çevirmek (`toupper`) UTF-8'de bayt düzeyinde yapılamaz. Doğru çözüm için geniş karakter kütüphaneleri ya da UTF-8 destekleyen harici kütüphaneler gerekir.
- `printf("%-8s")` gibi genişlik belirtimleri **bayt** sayar; Türkçe harf içeren metinler ekranda eksik boşlukla hizalanır. Bu davranışı Bölüm 6'da da görmüştük.
- Eski Windows programlarının yazdığı dosyalar UTF-8 yerine `Windows-1254` gibi tek baytlı bir kodlamada olabilir. Bu dosyalar UTF-8 olarak açılınca Türkçe harfler bozuk görünür. Bazı Windows düzenleyicileri UTF-8 dosyanın başına ayrıca üç baytlık bir **BOM** (EF BB BF) ekler; C programı bunu sıradan üç bayt olarak okur ve ilk satırın başında görünmez karakterler çıkabilir.

> **İpucu · Kodlama sorunu şüphesi**
>
> Beklenmedik `?` ya da `Ã§` gibi diziler görüyorsak, dosyanın kodlamasıyla terminalin ya da düzenleyicinin kodlaması uyuşmuyor demektir. Önce dosyanın hangi kodlamada yazıldığını bulur, sonra okuyan tarafı ona göre ayarlarız; program kodunu değiştirmek çoğu zaman gerekmez.

17.12 Uygulama: Dosyada Telefon Rehberi

Öğrendiklerimizi birleştirelim. Kayıtları ikili bir dosyada tutan bir rehber yazacağız: her kayıt bir ad ve bir numaradan oluşan sabit boyutlu bir yapıdır. Program üç işlev sunar: kişi ekle (dosyanın sonuna), hepsini listele, ada göre ara. Ekleme `"ab"` kipiyle yapıldığı için eski kayıtlar korunur; böylece programı her çalıştırdığımızda rehber büyür.

Programın akışı tam olarak tekrarlanabilsin diye menü yerine sabit bir senaryo uyguluyoruz: üç kişi ekle, listele, bir ad ara, olmayan bir adı ara. Gerçek bir kullanıcı arayüzü ise Bölüm 20'deki proje programında kurulacak.

▸ rehber.c

```
#include <stdio.h>
#include <string.h>

#define DOSYA "rehber.dat"

struct Kisi {
    char ad[30];
    char tel[16];
};

int ekle(const char *ad, const char *tel) {
    struct Kisi k;
    memset(&k, 0, sizeof k);              // dolgu baytlarını sıfırla
    strncpy(k.ad, ad, sizeof k.ad - 1);
    strncpy(k.tel, tel, sizeof k.tel - 1);

    FILE *f = fopen(DOSYA, "ab");
    if (f == NULL) {
        perror(DOSYA);
        return 0;
    }
    int basari = fwrite(&k, sizeof k, 1, f) == 1;
    fclose(f);
    return basari;
}

void listele(void) {
    FILE *f = fopen(DOSYA, "rb");
    if (f == NULL) {
        printf("Rehber boş.\n");
        return;
    }
    struct Kisi k;
    int sira = 1;
    while (fread(&k, sizeof k, 1, f) == 1) {
        printf("%d. %-10s %s\n", sira, k.ad, k.tel);
        sira++;
    }
    fclose(f);
}
```

```c
void ara(const char *ad) {
    FILE *f = fopen(DOSYA, "rb");
    if (f == NULL) {
        printf("Rehber boş.\n");
        return;
    }
    struct Kisi k;
    int bulundu = 0;
    while (fread(&k, sizeof k, 1, f) == 1) {
        if (strcmp(k.ad, ad) == 0) {
            printf("Bulundu: %s -> %s\n", k.ad, k.tel);
            bulundu = 1;
        }
    }
    fclose(f);
    if (!bulundu) {
        printf("%s rehberde yok.\n", ad);
    }
}

int main(void) {
    remove(DOSYA);                          // temiz başlangıç
    ekle("Ayse", "0532 111 22 33");
    ekle("Mehmet", "0533 444 55 66");
    ekle("Elif", "0534 777 88 99");

    listele();
    ara("Mehmet");
    ara("Can");
    return 0;
}
```

```
ÇIKTI
1. Ayse       0532 111 22 33
2. Mehmet     0533 444 55 66
3. Elif       0534 777 88 99
Bulundu: Mehmet -> 0533 444 55 66
Can rehberde yok.
```

Kodda birkaç tasarım kararı var. `memset` ile kaydı önce sıfırlıyoruz; böylece adın kullanılmayan kısımları ve olası dolgu baytları dosyaya rastgele çöp olarak yazılmaz, dosya her çalıştırmada aynı baytlarla oluşur. `strncpy`'nin hedefi taşırmaması için `sizeof k.ad - 1` kadar kopyalıyoruz; geri kalan bayt zaten `'\0'` olarak kalır. Dosya adı bir `#define` sabitidir, tek yerde değiştirilir. Her işlev kendi dosyasını açıp kapatır: çağıranın dosya durumuyla uğraşmak zorunda kalmaması, küçük programlarda sade bir tasarımdır.

Bu programı geliştirmek için kayıt silmeyi (kalan kayıtları geçici bir dosyaya yazıp `remove` ve `rename` ile yer değiştirmek) ve kaydı yerinde güncellemeyi (`fseek` ile) bölüm sonu alıştırmalarında deneyeceğiz.

> **Üç Dilde · Dosya İşlemleri Üç Dilde** *C · C++ · C#*
>
> **C**: Yukarıda gördüğümüz `FILE *` ve `fopen` / `fprintf` / `fgets` / `fclose` işlevleri. Kaynağı kapatmak ve hataları denetlemek programcının sorumluluğundadır. C++'ta akım sınıfları (Bölüm 37) ve C#'ta `File` ile `StreamWriter` sınıfları (Bölüm 57) aynı işi daha güvenli biçimde yapacak.
>
> **C++**: `<fstream>` başlığındaki `ofstream` (yazma) ve `ifstream` (okuma) sınıfları, dosyayı yapıcıda açar ve nesne kapsamdan çıkınca **kendiliğinden kapatır**; `fclose` unutulamaz. Okuma `cin` / `cout` ile aynı `>>` ve `<<` işleçleriyle yapılır:

```cpp
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ofstream yaz("notlar.txt");
    yaz << "Ayşe 85\n";
    yaz.close();

    std::ifstream oku("notlar.txt");
    std::string satir;
    while (std::getline(oku, satir)) {      // C'deki fgets döngüsü
        std::cout << satir << "\n";
    }
}
```

C#: `System.IO` ad alanındaki `File` sınıfı en sık işleri tek çağrıyla yapar. Daha fazla denetim için `StreamWriter` / `StreamReader` ve `using` bildirimi (kapsam bitince dosyayı kapatır) kullanılır; hata durumunda `NULL` yerine `IOException` türü istisnalar fırlatılır:

```cs
File.WriteAllText("notlar.txt", "Ayşe 85\n");
string[] satirlar = File.ReadAllLines("notlar.txt");   // bütün satırlar
foreach (string satir in satirlar)
{
    Console.WriteLine(satir);
}

using (var yaz = new StreamWriter("liste.txt", append: true))
{
    yaz.WriteLine("yeni satır");            // using sonunda otomatik kapanır
}
```

Bölüm Özeti

- Dosyalar, programın bittikten sonra da yaşayan veriyi saklar; C onlara `FILE *` işaretçisiyle akım üzerinden erişir. Her dosya işi aç, kullan, kapat adımlarından oluşur.
- Metin dosyaları okunabilir karakterlerden oluşur ve `fprintf`, `fscanf`, `fgets`, `fputs` ile işlenir; ikili dosyalar bellek baytlarının kopyasıdır ve `fwrite`, `fread` ile işlenir.
- `fopen` kipleri: `"r"` oku, `"w"` yaz (dosyayı siler), `"a"` sona ekle, `+` okuma ve yazma, `b` ikili. `"w"` var olan içeriği hemen siler.
- Her `fopen` çağrısından sonra `NULL` denetimi yapılır; nedeni `perror` ya da `strerror(errno)` ile bildirilir. Her açılan dosya `fclose` ile kapatılır.
- `puts` satır sonu ekler, `fputs` eklemez. `fgets` satır sonunu diziye alır; `strcspn` ile silinebilir.
- `fgetc` dönüş değeri `int` olarak saklanmalıdır, yoksa `EOF` ile gerçek bir karakter karışabilir.
- `while (!feof(f))` kalıbı yanlıştır; son öğe bir kez fazla işlenir. Doğrusu okuma işlevinin dönüş değerini koşul yapmaktır: `fscanf(...) == n`, `fgets(...) != NULL`, `fgetc(f) != EOF`, `fread(...) == 1`.
- `stdin`, `stdout` ve `stderr` otomatik açılan üç akımdır; `>`, `>>`, `<` ve `2>` ile terminalde yönlendirilebilirler. Hata iletileri `stderr`'e gönderilir.

- `fwrite` / `fread` yapıları tek işlemle diske yazar ve okur; dönüş değeri aktarılan öğe sayısıdır. İşaretçi içeren yapılar bu yolla saklanamaz; ikili biçim derleyiciye ve platforma bağlıdır.
- `fseek`, `ftell` ve `rewind` dosya konumunu yönetir; sabit boyutlu kayıtlarda n'inci kayda doğrudan erişim `n * sizeof(kayıt)` ile yapılır. `+` kipinde okuma ile yazma arasında `fseek` gerekir.
- `remove` ve `rename` dosya silme ve yeniden adlandırma işlevleridir.
- Windows metin kipinde `\n` ile `\r\n` arasında dönüşüm yapar; ikili dosyalar bu yüzden `"rb"` / `"wb"` ile açılmalıdır. Yol ayırıcı olarak `/` her platformda çalışır; `\` yazılacaksa iki kez yazılır.
- UTF-8 dosyalarda Türkçe harfler iki bayttır; dosya işlevleri baytları olduğu gibi taşır, ama karakter sayma ve büyük/küçük harf çevirme bayt düzeyinde yapılamaz.

Alıştırmalar

1. `fopen("veri.txt", "w")` çağrısı için aşağıdakilerden hangisi doğrudur?

   A) Dosya yoksa `NULL` döner.
   B) Dosya varsa içeriği korunur ve sona yazılır.
   C) Dosya varsa içeriği hemen silinir.
   D) Dosya yalnızca okunabilir.
   E) Dosya ikili kipte açılır.

2. Sona ekleme yapmak ve var olan içeriği korumak için hangi kip kullanılmalıdır?

   A) `"r"`
   B) `"w"`
   C) `"a"`
   D) `"r+"`
   E) `"rb"`

3. Aşağıdakilerden hangisi dosya sonuna kadar okuyan **doğru** döngü kalıbıdır?

   A) `while (!feof(f)) { fscanf(f, "%d", &x); ... }`
   B) `while (feof(f) == 0) { x = fgetc(f); ... }`
   C) `while (fscanf(f, "%d", &x) == 1) { ... }`
   D) `do { fscanf(f, "%d", &x); } while (!feof(f));`
   E) `for (;;) { fscanf(f, "%d", &x); }`

4. `puts("merhaba")` ile `fputs("merhaba", stdout)` arasındaki fark nedir?

   A) Fark yoktur.
   B) `puts` sonuna satır sonu ekler, `fputs` eklemez.
   C) `fputs` sonuna satır sonu ekler, `puts` eklemez.
   D) `puts` yalnızca dosyaya yazar.
   E) `fputs` yalnızca tek karakter yazar.

5. `fgetc` işlevinin dönüş değeri neden `char` değil `int` değişkende saklanır?

   A) `char` tipi işlevlerden değer alamaz.
   B) `EOF`, geçerli bir karakter koduyla karışmamalıdır.
   C) `int` daha hızlıdır.

D) `fgetc` iki karakter okur.
E) Bu zorunlu değildir, yalnızca gelenektir.

6. `fread(&k, sizeof k, 1, f)` çağrısı dosyada tam bir kayıt kalmadığında ne döndürür?

A) `-1`
B) `NULL`
C) `EOF`
D) 1'den küçük bir değer (kayıt tam okunamadıysa 0)
E) Kalan bayt sayısı

7. `fseek(f, 0, SEEK_END)` ardından `ftell(f)` çağrısı ne verir?

A) Dosyadaki satır sayısı
B) Dosyanın bayt cinsinden boyutu
C) Her zaman 0
D) Son karakterin kodu
E) Dosya açık değilse `NULL`

8. Windows'ta bir ikili dosyayı `"w"` (metin) kipiyle yazmak neden sorun yaratır?

A) Dosya salt okunur olur.
B) 10 değerli baytlar `\r\n` olarak iki bayta çevrilir ve veri bozulur.
C) Dosya hiç oluşmaz.
D) Dosyanın sonuna fazladan `EOF` yazılır.
E) Bir sorun yaratmaz.

9. Aşağıdaki program `giris.txt` dosyasında "3 5 8" yazıyorsa (satır sonu ile) ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("giris.txt", "r");
    if (f == NULL) return 1;
    int x = 0, toplam = 0;
    while (!feof(f)) {
        fscanf(f, "%d", &x);
        toplam += x;
    }
    fclose(f);
    printf("%d\n", toplam);
    return 0;
}
```

10. Aşağıdaki program kaç satır yazdırır ve hangi değerleri?

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("t.bin", "wb");
    if (f == NULL) return 1;
    int a[4] = {1, 2, 3, 4};
    fwrite(a, sizeof(int), 4, f);
    fclose(f);

    f = fopen("t.bin", "rb");
    if (f == NULL) return 1;
    int x;
```

```
    fseek(f, 2 * (long)sizeof(int), SEEK_SET);
    fread(&x, sizeof x, 1, f);
    printf("%d\n", x);
    printf("%ld\n", ftell(f));
    fclose(f);
    return 0;
}
```

11. Aşağıdaki kodda dosya kapatılmadan işlev dönebiliyor. Hatayı bulun ve düzeltin.

```
int ilkSatir(const char *ad, char *tampon, int boyut) {
    FILE *f = fopen(ad, "r");
    if (f == NULL) {
        return 0;
    }
    if (fgets(tampon, boyut, f) == NULL) {
        return 0;
    }
    fclose(f);
    return 1;
}
```

12. Aşağıdaki kodda en az iki hata vardır. Bulun.

```
FILE *f = fopen("sonuc.txt", "w");
char c;
fprintf(f, "Merhaba\n");
fclose(f);
f = fopen("sonuc.txt", "r");
while ((c = fgetc(f)) != EOF) {
    putchar(c);
}
```

13. Bir `struct Urun { char *ad; double fiyat; }` yapısını `fwrite` ile doğrudan dosyaya yazmanın neden sorunlu olduğunu açıklayın. Sorunu nasıl giderirsiniz?
14. **Program yazma (kolay):** `satirlar.txt` adlı bir dosyaya klavyeden okunan üç satırı yazan, sonra dosyayı baştan okuyup satırları numaralayarak ekrana basan bir program yazın.
15. **Program yazma (orta):** Bir metin dosyasını açıp içindeki en uzun satırı ve uzunluğunu bulan, sonucu ekrana yazan bir program yazın. Programın kendisi önce örnek dosyayı oluştursun.
16. **Program yazma (zor):** Bölümdeki rehber programına `sil(const char *ad)` işlevi ekleyin: verilen adı taşıyan kayıt dışındaki kayıtları `gecici.dat` dosyasına yazsın, ardından `remove` ve `rename` ile yeni dosyayı asıl dosyanın yerine koysun. Silme işleminden sonra rehberi listeleyerek sonucu gösterin.

BÖLÜM 18

Özyineleme (Recursion)

BU BÖLÜMDE

- Özyinelemenin (kendini çağıran fonksiyon) ne olduğunu ve hangi problemlerde işe yaradığını
- Temel durum (base case) ile özyinelemeli durumu; her çağrının temel duruma yaklaşması gerektiğini
- `faktoriyel(4)` çağrısında çağrı yığınının adım adım nasıl dolup boşaldığını
- Faktöriyel, üs alma (hızlı üs dahil), basamak toplamı, EBOB ve ikilik tabana çevirmeyi özyinelemeyle yazmayı
- Fibonacci'de çağrı ağacını, tekrar eden hesapları ve bellekleme (memoization) ile hızlandırmayı
- Dizi ve string üzerinde özyinelemeyi; Hanoi kulelerini ve özyinelemeli ikili aramayı
- Yazdırmayı çağrıdan önce ya da sonra yapmanın çıktıyı nasıl tersine çevirdiğini
- Yığın taşmasını ve özyineleme ile döngü arasında nasıl seçim yapılacağını

Bazı problemler, kendi küçük bir kopyasını içerir. Bir klasörün içindeki dosyaları saymak, içindeki alt klasörlerin dosyalarını saymayı da gerektirir; alt klasörler de aynı biçimde alt klasör içerebilir. Böyle bir problemi çözmenin doğal yolu, aynı yöntemi daha küçük parçaya yeniden uygulamaktır. Programlamada bu yönteme **özyineleme** denir.

Bölüm 9'da bir fonksiyonun kendi kendini çağırabildiğini ve her çağrının yığında ayrı bir çerçeve açtığını görmüştük. Bu bölümde bu olanağı bilinçli kullanmayı öğreneceğiz: özyinelemeli bir fonksiyonun iki parçasını, çağrı yığınındaki işleyişini, klasik örneklerini ve tehlikelerini. Özyineleme, bağlı listeler ve ağaçlar gibi veri yapılarında (Bölüm 19) vazgeçilmez olduğu için burada kuracağımız alışkanlıklar sonraki bölümlerde doğrudan işimize yarayacak.

18.1 Özyineleme Nedir?

Bir matruşka bebeği açtığımızı düşünelim: içinden bir küçüğü çıkar, onu açınca bir küçüğü daha çıkar. Her bebeğin içinde kendisinin daha küçük bir hâli vardır; en küçük bebek açılmaz ve işlem orada biter. Özyinelemenin fikri budur: problemi aynı türden ama daha küçük bir probleme indirgemek, en küçük problemi ise doğrudan çözmek.

> **Tanım: Özyineleme**
>
> Bir fonksiyonun kendi kendini çağırması (recursion). Özyinelemeli bir tanım, bir problemi aynı türden daha küçük bir problem cinsinden ifade eder ve bu küçülme, doğrudan cevabı bilinen en küçük probleme ulaşana kadar sürer.

Örnek olarak 1'den `n`'e kadar sayıların toplamını alalım. `n` için toplam, `n` sayısına `n - 1` için toplamı eklemektir; 0 için toplam 0'dır.

▸ ilkOzyineleme.c

```c
#include <stdio.h>

int topla(int n) {
    if (n == 0) {
        return 0;
    }
    return n + topla(n - 1);
}

int main(void) {
    printf("topla(1) = %d\n", topla(1));
    printf("topla(5) = %d\n", topla(5));
    printf("topla(100) = %d\n", topla(100));
    return 0;
}
```

ÇIKTI

```
topla(1) = 1
topla(5) = 15
topla(100) = 5050
```

`topla(3)` hesaplanırken fonksiyon `3 + topla(2)` ifadesine gelir ve `topla(2)`'nin cevabını bekler. O da `2 + topla(1)` der, o da `1 + topla(0)` der. `topla(0)` kendini çağırmadan 0 döndürür ve zincir geriye doğru çözülür: `1 + 0 = 1`, `2 + 1 = 3`, `3 + 3 = 6`.

Bu işi Bölüm 8'deki gibi bir `for` döngüsüyle de yazabilirdik; toplam gibi doğrusal işlerde döngü daha uygundur. Özyinelemeyi öğrenmemizin nedeni, bazı problemlerin (Hanoi kuleleri, ikili arama, bağlı liste ve ağaçlar) özyinelemeyle döngüden çok daha kısa ve doğal yazılmasıdır.

> *Tarihçe · Özyineleme ve programlama dilleri*
>
> Özyinelemeli tanımlar matematikte eskidir; faktöriyel ve Fibonacci dizisi bunun bilinen örnekleridir. Programlama dillerinde özyinelemeli fonksiyonlar LISP ile ve ALGOL 60 ile yerleşti. İlk Fortran sürümlerinde ise bir alt programın kendini çağırması desteklenmiyordu. C, baştan beri özyinelemeye izin verir; çünkü her çağrı için yığında ayrı bir çerçeve açılır (Bölüm 9).

18.2 Temel Durum ve Özyinelemeli Durum

Her özyinelemeli fonksiyonun iki parçası vardır:

```c
int fonksiyon(int n) {
    if (/* en küçük problem */) {
        return cevap;                  // TEMEL DURUM: kendini çağırmadan cevap ver
    }
    /* ... fonksiyon(daha küçük problem) ... */   // ÖZYİNELEMELİ DURUM
}
```

Parça	Görevi	`topla` örneğinde
Temel durum (base case)	Kendini çağırmadan doğrudan cevap verir; zinciri durdurur	`n == 0` ise 0

Parça	Görevi	`topla` örneğinde
Özyinelemeli durum (recursive case)	Problemi küçültüp kendini çağırır	`n + topla(n - 1)`
Yaklaşma	Her çağrı temel duruma biraz daha yaklaşmalıdır	`n` her çağrıda 1 azalır, sonunda 0 olur

Üçüncü satır en çok unutulandır: temel durumun yazılması yetmez, çağrıların ona ulaşması da gerekir. `topla(n + 1)` yazarsak `n` temel durumdan uzaklaşır ve zincir hiç bitmez.

> ◆ **Kritik Nokta · Önce temel durumu yazın**
>
> Özyinelemeli bir fonksiyon yazarken önce temel durumu, sonra özyinelemeli durumu yazın. Hata ararken de sırayla şunlara bakın: temel durum var mı, her girdi için ona ulaşılıyor mu (örneğin `n == 0` yazıp negatif sayı göndermek yetmez), argüman doğru yönde mi küçülüyor?

Özyinelemeli fonksiyon yazmanın en rahat yolu **güven ilkesidir**: `topla(n - 1)` çağrısının doğru çalıştığını kabul ederiz ve yalnızca şunu sorarız: "`topla(n - 1)`'in cevabından `topla(n)`'in cevabını nasıl çıkarırım?" Çağrının içini kafamızda sonuna kadar açmaya çalışmak yorucudur ve gereksizdir; zinciri açıp izlemeyi yalnızca bir fonksiyonun ne yaptığını anlamak için yaparız.

18.3 Çağrı Yığını ve Faktöriyel

Bölüm 9'da her çağrının yığın çerçevesi açtığını, çerçevede parametrelerin ve yerel değişkenlerin durduğunu ve fonksiyon bitince çerçevenin silindiğini görmüştük. Özyinelemede aynı fonksiyonun birçok çerçevesi aynı anda yığındadır ve her birinin kendi `n`'i, kendi yerel değişkenleri vardır.

Bunu faktöriyel ile izleyelim: `n! = n · (n-1)!` ve `0! = 1! = 1`. Aşağıdaki program, derinliği girintiyle göstererek her aşamayı yazdırır. `derinlik` global değişkeni yalnızca girintiyi ayarlamak içindir.

▸ faktoriyelIz.c

```c
#include <stdio.h>

int derinlik = 0;    // çıktıdaki girintiyi ayarlamak için

void girinti(void) {
    for (int i = 0; i < derinlik; i++) {
        printf("  ");
    }
}

int faktoriyel(int n) {
    girinti();
    printf("faktoriyel(%d) başladı\n", n);
    if (n <= 1) {
        girinti();
        printf("faktoriyel(%d): temel durum, 1 döndürüyor\n", n);
        return 1;
    }
    derinlik++;
    int sonuc = n * faktoriyel(n - 1);
    derinlik--;
    girinti();
    printf("faktoriyel(%d): sonuç = %d\n", n, sonuc);
    return sonuc;
```

```c
}

int main(void) {
    printf("Sonuç: %d\n", faktoriyel(4));
    return 0;
}
```

```
ÇIKTI
faktoriyel(4) başladı
  faktoriyel(3) başladı
    faktoriyel(2) başladı
      faktoriyel(1) başladı
      faktoriyel(1): temel durum, 1 döndürüyor
    faktoriyel(2): sonuç = 2
  faktoriyel(3): sonuç = 6
faktoriyel(4): sonuç = 24
Sonuç: 24
```

Girinti, her çağrının bir kat içeride başladığını gösterir. Aynı olayın yığındaki karşılığı aşağıdaki tablodadır. `f(4)`, `faktoriyel`'in `n = 4` olan çerçevesini anlatır; her satırda soldaki çerçeve en üsttedir, yani o an çalışandır.

Adım	Olay	Yığın (üstten alta)
1	`main`, `faktoriyel(4)`'ü çağırır	`f(4)` / `main`
2	`f(4)`, `faktoriyel(3)`'ü çağırır ve bekler	`f(3)` / `f(4)` / `main`
3	`f(3)`, `faktoriyel(2)`'yi çağırır	`f(2)` / `f(3)` / `f(4)` / `main`
4	`f(2)`, `faktoriyel(1)`'i çağırır	`f(1)` / `f(2)` / `f(3)` / `f(4)` / `main`
5	`f(1)` temel durum: 1 döndürür, çerçevesi silinir	`f(2)` / `f(3)` / `f(4)` / `main`
6	`f(2)`: `2 * 1 = 2` döndürür, silinir	`f(3)` / `f(4)` / `main`
7	`f(3)`: `3 * 2 = 6` döndürür, silinir	`f(4)` / `main`
8	`f(4)`: `4 * 6 = 24` döndürür, silinir	`main`

Üç noktaya dikkat edelim:

- Gidişte (adım 1-4) hiçbir çarpma yapılmaz; çağrılar açılır ve cevap beklenir. Çarpmalar dönüşte (5-8), çerçeveler kapanırken yapılır.
- Dört çerçevede dört ayrı `n` ve dört ayrı `sonuc` vardır; hiçbiri ötekini bozmaz. `f(2)`'nin `n`'i, `f(4)`'ün `n`'ini değiştirmez.
- Yığın en derin hâlinde `faktoriyel`'in 4 çerçevesini taşır (`main` ile 5). Yığın belleği çağrı derinliği kadar büyür.

> **▲ Sık Yapılan Hata · Derinliği sayarken temel durumu unutmayın**
>
> Temel durumu `n <= 1` olan `faktoriyel(5)` için yığındaki en çok çerçeve sayısı 5'tir (`n = 5, 4, 3, 2, 1`); `main` de sayılırsa 6. Temel durumu `n == 0` olsaydı `n = 0` da bir çerçeve açacağı için derinlik 6 olurdu. Temel durumun çerçevesi de bir çerçevedir.

Bu programda `int` kullandık; `13!` bir `int`'e sığmaz (Bölüm 5). Büyük `n` için `long long` kullanılır, `20!`'e kadar sığar (Bölüm 8).

18.4 Sayılarla Özyineleme

Üs alma ve hızlı üs

$a^n = a \cdot a^{n-1}$ ve $a^0 = 1$. Bu doğrudan `us(a, n)` fonksiyonudur. **Hızlı üs** ise daha akıllıdır: `n` çiftse $a^n = (a^{n/2})^2$, tekse $a^n = a \cdot (a^{n/2})^2$. Yani `n` her çağrıda yarıya iner. İkisinin kaç çağrı yaptığını global bir sayaçla sayalım:

usAlma.c

```c
#include <stdio.h>

int cagri = 0;

long long us(long long a, int n) {
    cagri++;
    if (n == 0) {
        return 1;
    }
    return a * us(a, n - 1);
}

long long hizliUs(long long a, int n) {
    cagri++;
    if (n == 0) {
        return 1;
    }
    long long yarim = hizliUs(a, n / 2);    // a üzeri n/2 bir kez hesaplanır
    if (n % 2 == 0) {
        return yarim * yarim;
    }
    return yarim * yarim * a;
}

int main(void) {
    cagri = 0;
    long long sonuc = us(3, 20);
    printf("us(3, 20) = %lld, çağrı sayısı: %d\n", sonuc, cagri);
    cagri = 0;
    sonuc = hizliUs(3, 20);
    printf("hizliUs(3, 20) = %lld, çağrı sayısı: %d\n", sonuc, cagri);
    return 0;
}
```

ÇIKTI

```
us(3, 20) = 3486784401, çağrı sayısı: 21
hizliUs(3, 20) = 3486784401, çağrı sayısı: 6
```

`hizliUs`'ta `n` şöyle iner: 20 ⟶ 10 ⟶ 5 ⟶ 2 ⟶ 1 ⟶ 0. `n = 5` ve `n = 1` tek olduğundan bu iki çağrı sonuca fazladan bir `a` çarpar. `us`'ta `n + 1` çağrı gerekirken `hizliUs`'ta yaklaşık $\log_2 n$ çağrı yeter. `n = 1 000 000` için fark 1 000 001 çağrıya karşılık yaklaşık 21 çağrıdır.

> **▲ Sık Yapılan Hata · Aynı özyinelemeli çağrıyı iki kez yazmak**
>
> `return hizliUs(a, n / 2) * hizliUs(a, n / 2);` yazarsak sonuç doğru çıkar, ama hız kazancı yok olur: her çağrı iki çağrı açar ve toplam çağrı sayısı yine `n` ile orantılı olur. Yarım sonucu bir değişkende (`yarim`) tutup bir kez hesaplamak gerekir.

Basamak toplamı, EBOB ve ikilik tabana çevirme

Bölüm 8'de döngüyle yazdığımız üç algoritma, özyinelemeyle çok kısalır. Üçü de `n >= 0` için yazılmıştır.

▸ sayilarlaOzyineleme.c

```c
#include <stdio.h>

int basamakToplami(int n) {
    if (n < 10) {
        return n;
    }
    return n % 10 + basamakToplami(n / 10);
}

int ebob(int a, int b) {
    if (b == 0) {
        return a;
    }
    return ebob(b, a % b);
}

void ikiliYaz(int n) {
    if (n > 1) {
        ikiliYaz(n / 2);
    }
    printf("%d", n % 2);
}

int main(void) {
    printf("basamakToplami(2749) = %d\n", basamakToplami(2749));
    printf("ebob(48, 18) = %d\n", ebob(48, 18));
    printf("13 ikilik tabanda: ");
    ikiliYaz(13);
    printf("\n0 ikilik tabanda: ");
    ikiliYaz(0);
    printf("\n");
    return 0;
}
```

ÇIKTI

```
basamakToplami(2749) = 22
ebob(48, 18) = 6
13 ikilik tabanda: 1101
0 ikilik tabanda: 0
```

- `basamakToplami`: Son basamak `n % 10`, geri kalanı `n / 10`'dur. `basamakToplami(2749) = 9 + basamakToplami(274) = 9 + 4 + basamakToplami(27) = 9 + 4 + 7 + basamakToplami(2)`. Tek basamaklı sayı temel durumdur: `9 + 4 + 7 + 2 = 22`.
- `ebob`: Öklid algoritması, Bölüm 8'deki döngüyle aynıdır: `(48, 18) → (18, 12) → (12, 6) → (6, 0)`. Döngüde `x` ve `y` değişkenlerine atama yapıyorduk; burada yeni değerler bir sonraki çağrının argümanı olur. `b` sıfır olunca cevap `a`'dır.
- `ikiliYaz`: Bir sayıyı 2'ye bölüp kalanları sırayla not alırsak bunlar sondan başa doğru okunur (13 ⟶ 6 ⟶ 3 ⟶ 1 için kalanlar 1, 0, 1, 1; ters okuyunca `1101`). Özyineleme bu "ters okumayı" kendiliğinden yapar: `printf` özyinelemeli çağrıdan sonra olduğundan önce en yüksek bit yazılır. `printf`'i çağrıdan önceye alsaydık `1011` çıkardı. Bu ayrıntıya "Yazdırmayı Çağrıdan Önce mi Sonra mı Yapmalı?" başlığında yeniden döneceğiz.

C'de negatif sayıların kalanı negatif çıkabildiğinden (Bölüm 5) bu fonksiyonlar negatif girdi için yazılmamıştır. Negatifleri desteklemek istersek önce işareti ayrı ele alıp sayının mutlak değeriyle çağırmak gerekir.

18.5 Fibonacci ve Bellekleme

Fibonacci dizisi 0, 1, 1, 2, 3, 5, 8, … biçimindedir: `fib(0) = 0`, `fib(1) = 1`, diğer her terim iki öncekinin toplamıdır. Tanım zaten özyinelemelidir ve koda çevirmesi kolaydır; ama bu kez fonksiyon **iki** kez kendini çağırır.

▸ fibonacci.c

```c
#include <stdio.h>

int cagri = 0;

int fib(int n) {
    cagri++;
    if (n < 2) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

int main(void) {
    int degerler[] = {5, 10, 20};
    for (int i = 0; i < 3; i++) {
        cagri = 0;
        int sonuc = fib(degerler[i]);
        printf("fib(%d) = %d, çağrı sayısı: %d\n", degerler[i], sonuc, cagri);
    }
    return 0;
}
```

ÇIKTI

```
fib(5) = 5, çağrı sayısı: 15
fib(10) = 55, çağrı sayısı: 177
fib(20) = 6765, çağrı sayısı: 21891
```

`fib(20)` için 21 891 çağrı yapıldı; oysa Bölüm 8'deki döngü cevabı 20 adımda buluyordu. Nedenini `fib(4)`'ün **çağrı ağacında** (call tree) görebiliriz:

```
                fib(4)
              /        \
          fib(3)        fib(2)
          /    \        /    \
      fib(2)  fib(1) fib(1) fib(0)
      /    \
  fib(1)  fib(0)
```

`fib(2)` iki kez, `fib(1)` üç kez hesaplanıyor. Aynı işler defalarca tekrarlanıyor. Çağrı sayısı `C(n)` için `C(0) = C(1) = 1` ve `C(n) = 1 + C(n-1) + C(n-2)` kuralı geçerlidir: 1, 1, 3, 5, 9, 15 (`n = 0...5`). Bu değerler, programın `fib(5)` için bulduğu 15 ile uyuşur. Kapalı formül `C(n) = 2 · fib(n+1) - 1`'dir.

`n` bir arttığında çağrı sayısı yaklaşık 1,6 katına çıkar; buna **üstel büyüme** denir. `fib(30)` yaklaşık 2,7 milyon, `fib(40)` yaklaşık 331 milyon çağrı ister. Bu yüzden bu yöntemle büyük Fibonacci sayıları hesaplamıyoruz.

Bellekleme (memoization)

Çözüm basittir: bir kez hesapladığımız değeri bir dizide saklarız, gerekince yeniden hesaplamak yerine diziden okuruz. Bu tekniğe **bellekleme** (memoization) denir.

> **Tanım: Bellekleme**
>
> Bir fonksiyonun daha önce hesapladığı sonuçları saklayıp aynı girdi tekrar geldiğinde yeniden hesaplamak yerine saklanan değeri döndürmesidir (memoization).

`hafiza[n] == 0`, "henüz hesaplanmadı" demektir (`n >= 2` için `fib(n)` en az 1'dir). Global diziler 0 ile başladığı için ayrıca doldurmaya gerek yoktur (Bölüm 5):

▸ fibHafiza.c

```c
#include <stdio.h>

long long hafiza[91];    // global: başlangıçta hepsi 0
int cagri = 0;

long long fibHafiza(int n) {
    cagri++;
    if (n < 2) {
        return n;
    }
    if (hafiza[n] != 0) {    // daha önce hesaplandı
        return hafiza[n];
    }
    hafiza[n] = fibHafiza(n - 1) + fibHafiza(n - 2);
    return hafiza[n];
}

int main(void) {
    long long sonuc = fibHafiza(50);
    printf("fib(50) = %lld, çağrı sayısı: %d\n", sonuc, cagri);
    return 0;
}
```

```
ÇIKTI
fib(50) = 12586269025, çağrı sayısı: 99
```

Düz özyinelemeyle `fib(50)` onlarca milyar çağrı gerektirirdi; burada 99 çağrı yetti. `fibHafiza(n-1)` zinciri en alta kadar iner ve dönüşte diziyi doldurur; ardından `fibHafiza(n-2)` hazır cevabı okur. Genel olarak `fibHafiza(n)` için `2n - 1` çağrı yapılır. Dizinin 91 elemanlı olmasının nedeni, `fib(90)`'ın `long long`'a sığan son terim olmasıdır.

Bellekleme, özyinelemenin okunaklı yapısını korurken üstel maliyeti ortadan kaldırır; ama bir dizi ve derinliği `n` olan bir yığın kullanır. Bölüm 8'deki döngülü sürüm ise bu belleğe ihtiyaç duymaz. "Aynı alt problem tekrar tekrar çözülüyor" gördüğümüz her yerde bellekleme akla gelmelidir.

> **İpucu · Çağrı ağacını kâğıda çizin**
>
> İki çağrılı bir fonksiyonun çıktısını ya da çağrı sayısını bulmanın en güvenli yolu ağacı çizmektir. Her düğüme argümanı yazın; yapraklar temel durumlardır. Çıktı yazdırılıyorsa ağacı soldan sağa, derinlemesine gezin.

18.6 Dizi ve String Üzerinde Özyineleme

Bir dizi üzerinde özyinelemede "daha küçük problem", dizinin daha kısa hâlidir. Diziyi kopyalamayız (Bölüm 10); yalnızca "ilk `n` eleman" demek için `n`'i azaltırız. Aşağıdaki üç fonksiyon dizinin son elemanını ayırır, kalanı için kendini çağırır:

▸ diziOzyineleme.c

```c
#include <stdio.h>

int toplam(const int a[], int n) {     // ilk n elemanın toplamı
    if (n == 0) {
        return 0;
    }
    return a[n - 1] + toplam(a, n - 1);
}

int enBuyuk(const int a[], int n) {    // n >= 1 olmalı
    if (n == 1) {
        return a[0];
    }
    int diger = enBuyuk(a, n - 1);
    return (a[n - 1] > diger) ? a[n - 1] : diger;
}

void tersYazdir(const int a[], int n) {
    if (n == 0) {
        return;
    }
    printf("%d ", a[n - 1]);
    tersYazdir(a, n - 1);
}

int main(void) {
    int a[] = {4, 9, 2, 7, 5};
    printf("toplam = %d\n", toplam(a, 5));
    printf("enBuyuk = %d\n", enBuyuk(a, 5));
    printf("ters: ");
    tersYazdir(a, 5);
    printf("\n");
    return 0;
}
```

```
ÇIKTI
toplam = 27
enBuyuk = 9
ters: 5 7 2 9 4
```

- `toplam`: Temel durum boş dizidir (`n == 0`) ve toplamı 0'dır.
- Aynı işi başlangıcı ilerleterek de yazabiliriz: `return a[0] + toplam(a + 1, n - 1);`. İşaretçi aritmetiğinde (Bölüm 12) `a + 1`, ikinci elemandan başlayan "kısalmış dizi"dir.
- `enBuyuk`: Boş dizinin en büyüğü olmadığından temel durum tek elemandır (`n == 1`). `n >= 1` olmayan bir çağrı temel duruma ulaşamaz.
- `tersYazdir`: Önce son elemanı yazar, sonra kalan kısmı çağırır. Elemanları baştan sona yazmak için `printf` çağrıdan sonraya konurdu.

String'lerde "küçük problem", ilk karakteri atılmış string'dir; `s + 1` bunu verir. Bitiş `'\0'` karakteridir (Bölüm 11):

stringOzyineleme.c

```c
#include <stdio.h>
#include <string.h>

int uzunluk(const char *s) {
    if (*s == '\0') {
        return 0;
    }
    return 1 + uzunluk(s + 1);
}

int palindromMu(const char *s, int bas, int son) {
    if (bas >= son) {
        return 1;
    }
    if (s[bas] != s[son]) {
        return 0;
    }
    return palindromMu(s, bas + 1, son - 1);
}

void tersYaz(const char *s) {
    if (*s == '\0') {
        return;
    }
    tersYaz(s + 1);
    putchar(*s);
}

int main(void) {
    const char *kelimeler[] = {"kayak", "araba", "radar"};
    printf("uzunluk(\"merhaba\") = %d\n", uzunluk("merhaba"));
    for (int i = 0; i < 3; i++) {
        int n = (int)strlen(kelimeler[i]);
        printf("%s: %s\n", kelimeler[i],
               palindromMu(kelimeler[i], 0, n - 1) ? "palindrom" : "değil");
    }
    printf("merhaba tersi: ");
    tersYaz("merhaba");
    printf("\n");
    return 0;
}
```

ÇIKTI

```
uzunluk("merhaba") = 7
kayak: palindrom
araba: değil
radar: palindrom
merhaba tersi: abahrem
```

- `uzunluk("abc") = 1 + uzunluk("bc") = 1 + 1 + uzunluk("c") = 1 + 1 + 1 + uzunluk("") = 3`. `strlen` de aynı işi yapar (Bölüm 11).
- `palindromMu`: İki ucu karşılaştırır ve içeri doğru daralır. İki temel durum vardır: uçlar karşılaşırsa (`bas >= son`) tüm çiftler eşleşmiştir, yani metin palindromdur; uçlardaki karakterler farklıysa palindrom değildir. `bas >= son` koşulu uzunluk tek de olsa çift de olsa doğru çalışır.
- `tersYaz`: Yazdırma özyinelemeli çağrıdan sonradır; önce string'in sonuna inilir, harfler dönüşte sondan başa yazılır. `putchar(*s)` çağrıdan önce olsaydı string düz yazılırdı.

18.7 Hanoi Kuleleri

Üç çubuk (A, B, C) ve A'da üst üste dizilmiş, küçükten büyüğe `n` disk var. Amaç tüm diskleri C'ye taşımaktır. Kurallar: bir seferde yalnızca bir disk taşınır ve büyük disk küçüğün üstüne konamaz.

> *Tarihçe · Hanoi Kulesi*
>
> Bulmaca, Fransız matematikçi Édouard Lucas tarafından 1883'te tanıtılmıştır.

Döngüyle yazması çok zordur; özyinelemeyle birkaç satırdır. Fikir şu: `n` diski A'dan C'ye taşımak için (B yedek)

1. üstteki `n - 1` diski A'dan B'ye taşırız (C yedek),
2. en büyük diski A'dan C'ye taşırız,
3. `n - 1` diski B'den C'ye taşırız (A yedek).

Temel durum, hiç disk kalmamasıdır (`n == 0`): yapılacak iş yoktur. Güven ilkesi burada çok işe yarar: `n - 1` diski taşımayı bilen bir fonksiyonumuz olduğunu varsayarız, ona iki kez görev veririz.

▸ hanoi.c

```c
#include <stdio.h>

void hanoi(int n, char kaynak, char hedef, char yedek) {
    if (n == 0) {
        return;
    }
    hanoi(n - 1, kaynak, yedek, hedef);
    printf("Disk %d: %c -> %c\n", n, kaynak, hedef);
    hanoi(n - 1, yedek, hedef, kaynak);
}

long long hamleSayisi(int n) {
    if (n == 0) {
        return 0;
    }
    return 2 * hamleSayisi(n - 1) + 1;
}

int main(void) {
    hanoi(3, 'A', 'C', 'B');
    int olculer[] = {1, 2, 3, 4, 10, 20, 30};
    for (int i = 0; i < 7; i++) {
        printf("n = %2d: %lld hamle\n", olculer[i], hamleSayisi(olculer[i]));
    }
    return 0;
}
```

```
ÇIKTI
Disk 1: A -> C
Disk 2: A -> B
Disk 1: C -> B
Disk 3: A -> C
Disk 1: B -> A
Disk 2: B -> C
Disk 1: A -> C
n =  1: 1 hamle
n =  2: 3 hamle
n =  3: 7 hamle
n =  4: 15 hamle
n = 10: 1023 hamle
n = 20: 1048575 hamle
n = 30: 1073741823 hamle
```

`hanoi(3, 'A', 'C', 'B')` çağrısı iki alt çağrıya ayrılır; ilk yedi hamle bu yapıdan okunur:

```
hanoi(3: A -> C)
 +-- hanoi(2: A -> B)    1. Disk 1: A -> C
 |                       2. Disk 2: A -> B
 |                       3. Disk 1: C -> B
 +-- Disk 3: A -> C      4. hamle
 +-- hanoi(2: B -> C)    5. Disk 1: B -> A
                         6. Disk 2: B -> C
                         7. Disk 1: A -> C
```

Hamle sayısı için `H(0) = 0` ve `H(n) = 2 · H(n-1) + 1` yazılır: `n - 1` diski iki kez taşıyoruz, arada bir hamle var. Çözüm `H(n) = 2ⁿ - 1`'dir: 1, 3, 7, 15, 31, ... Programdaki tablo da bunu doğrular (`2¹⁰ - 1 = 1023`, `2²⁰ - 1 = 1 048 575`).

Her ek disk hamle sayısını iki katına çıkarır; 64 diskin hamle sayısı `2⁶⁴ - 1`, yani yaklaşık 1,8 · 10^{19}'dur. Saniyede bir hamle yapsak yaklaşık 585 milyar yıl sürer. Özyinelemenin kısa yazılması, işin hızlı bitmesi anlamına gelmez.

18.8 Özyinelemeli İkili Arama

Bölüm 10'da ikili aramayı `while (alt <= ust)` döngüsüyle yazmıştık. Fikir özyinelemeye çok uygundur: ortaya bakarız; bulduysak iş biter, bulamadıysak aynı aramayı yarım dizide yineleriz. Aynı dizi ve aynı iz çıktısıyla:

▸ **ikiliAraOzyineleme.c**

```c
#include <stdio.h>

int ikiliAra(const int a[], int alt, int ust, int aranan) {
    if (alt > ust) {
        return -1;
    }
    int orta = alt + (ust - alt) / 2;
    printf("  alt=%d ust=%d orta=%d a[orta]=%d\n", alt, ust, orta, a[orta]);
    if (a[orta] == aranan) {
        return orta;
    }
    if (a[orta] < aranan) {
        return ikiliAra(a, orta + 1, ust, aranan);
    }
```

```
    return ikiliAra(a, alt, orta - 1, aranan);
}

int main(void) {
    int a[] = {3, 7, 12, 19, 25, 31, 42, 56, 68, 77};
    printf("31 aranıyor:\n");
    printf("  sonuç: %d\n", ikiliAra(a, 0, 9, 31));
    printf("10 aranıyor:\n");
    printf("  sonuç: %d\n", ikiliAra(a, 0, 9, 10));
    return 0;
}
```

```
ÇIKTI
31 aranıyor:
  alt=0 ust=9 orta=4 a[orta]=25
  alt=5 ust=9 orta=7 a[orta]=56
  alt=5 ust=6 orta=5 a[orta]=31
  sonuç: 5
10 aranıyor:
  alt=0 ust=9 orta=4 a[orta]=25
  alt=0 ust=3 orta=1 a[orta]=7
  alt=2 ust=3 orta=2 a[orta]=12
  sonuç: -1
```

`31` için üç satır, Bölüm 10'daki adım tablosuyla aynıdır. Döngü ile özyinelemenin eşleşmesi şöyledir:

	Döngü (Bölüm 10)	Özyineleme
Aranacak aralık	`alt` ve `ust` değişkenleri güncellenir	Yeni çağrının argümanları olur: `orta + 1, ust` ya da `alt, orta - 1`
Bulunamadı	`while (alt <= ust)` bozulunca döngüden çıkılır, `-1` döner	`alt > ust` temel durumdur: `-1`
Bulundu	`return orta;`	Temel durum: `return orta;`
Bellek	Sabit	Her adımda bir çerçeve: en çok yaklaşık $\log_2 n$ çerçeve

> **▲ Sık Yapılan Hata · Özyinelemeli çağrının sonucunu return etmeyi unutmak**
>
> `ikiliAra(a, orta + 1, ust, aranan);` satırını `return` olmadan yazarsak çağrı yapılır ama sonuç atılır; akış fonksiyonun altındaki satıra düşer ve yanlış sonuç çıkar. Fonksiyonun son satırındaki `return` silinirse fonksiyon değer döndürmeden biter: sonuç tanımsızdır (çöp değer) ve `gcc -Wall` `control reaches end of non-void function` uyarısını verir. Kural: Sonuç döndüren fonksiyonda özyinelemeli çağrı ya `return ikiliAra(...)` biçiminde yazılır ya da değeri bir ifadede kullanılır.

18.9 Yazdırmayı Çağrıdan Önce mi Sonra mı Yapmalı?

Özyinelemeli bir fonksiyonda `printf`'i çağrıdan önce ya da sonra yazmak çıktıyı tamamen değiştirir. Çağrıdan önceki satırlar **gidişte** (yığın dolarken), sonraki satırlar **dönüşte** (yığın boşalırken, ters sırayla) çalışır:

▸ onceSonra.c

```
#include <stdio.h>

void once(int n) {          // önce yaz, sonra çağır
    if (n == 0) {
```

```
        return;
    }
    printf("%d ", n);
    once(n - 1);
}

void sonra(int n) {          // önce çağır, sonra yaz
    if (n == 0) {
        return;
    }
    sonra(n - 1);
    printf("%d ", n);
}

void cetvel(int n) {         // iki özyinelemeli çağrı
    if (n == 0) {
        return;
    }
    cetvel(n - 1);
    printf("%d ", n);
    cetvel(n - 1);
}

int main(void) {
    printf("once(3):   ");
    once(3);
    printf("\nsonra(3):  ");
    sonra(3);
    printf("\ncetvel(3): ");
    cetvel(3);
    printf("\n");
    return 0;
}
```

```
ÇIKTI
once(3):   3 2 1
sonra(3):  1 2 3
cetvel(3): 1 2 1 3 1 2 1
```

Yazdırma yeri	Ne zaman çalışır?	`n = 3` için çıktı
Çağrıdan önce	Giderken: `n` büyükten küçüğe	`3 2 1`
Çağrıdan sonra	Dönerken: en son açılan çerçeve ilk kapanır	`1 2 3`

`sonra(3)`'ü yığında izleyelim: `sonra(3) → sonra(2) → sonra(1) → sonra(0)`; `sonra(0)` temel durumdur ve döner. Şimdi çerçeveler kapanır: `sonra(1)` 1'i yazar, `sonra(2)` 2'yi, `sonra(3)` 3'ü. Yığın "son giren ilk çıkar" çalıştığı için özyineleme bir şeyi tersine çevirmenin doğal yoludur; `ikiliYaz` ve `tersYaz` bu yüzden kendiliğinden ters sırada çalışıyordu.

İki özyinelemeli çağrılı `cetvel` için tanımı açık yazalım: `cetvel(3) = cetvel(2), 3, cetvel(2)`. `cetvel(2) = cetvel(1), 2, cetvel(1)` ve `cetvel(1) = 1` olduğundan (`cetvel(0)` hiçbir şey yazmaz) `cetvel(2)` çıktısı `1 2 1`, `cetvel(3)` çıktısı `1 2 1 3 1 2 1` olur.

> ◆ **Kritik Nokta · Çıktıyı izlerken**
>
> Her çağrıyı kendi satırına açın. Çağrıdan önceki `printf`'ler aşağı inerken, çağrıdan sonrakiler yukarı çıkarken çalışır. Fonksiyon kendini iki kez çağırıyorsa iki çağrıyı ayrı ayrı açıp soldan sağa okuyun.

18.10 Temel Durumu Unutmak: Yığın Taşması

Temel durum yoksa zincir bitmez, ama bellek sonsuz değildir. Her çağrı bir çerçeve açar ve çağrı yığını sınırlıdır (genellikle birkaç MB; Linux'ta varsayılan 8 MB, Windows'ta 1 MB civarındadır). Çerçeveler yığını doldurunca program **yığın taşması** (stack overflow) ile çöker.

Aşağıdaki program kasıtlı olarak hatalıdır. Çıktı karışmasın diye yalnızca ilk üç çağrıda yazdırır:

▸ sonsuz.c

```c
#include <stdio.h>

int sayac = 0;

int sonsuzToplam(int n) {
    sayac++;
    if (sayac <= 3) {
        printf("çağrı %d: n = %d\n", sayac, n);
    }
    return n + sonsuzToplam(n - 1);    // temel durum yok!
}

int main(void) {
    printf("Başlıyor\n");
    printf("%d\n", sonsuzToplam(5));
    return 0;
}
```

TERMİNAL

```
$ gcc -std=c17 -Wall -Wextra sonsuz.c -o sonsuz
sonsuz.c: In function 'sonsuzToplam':
sonsuz.c:5:5: warning: infinite recursion detected [-Winfinite-recursion]
sonsuz.c:10:16: note: recursive call
$ ./sonsuz
Başlıyor
çağrı 1: n = 5
çağrı 2: n = 4
çağrı 3: n = 3
Segmentation fault (core dumped)
```

Yeni gcc sürümleri bu hatayı `-Wall` ile yakalayıp uyarı verir (`-Winfinite-recursion`); ama her durumda yakalayamaz, uyarıya güvenmemek gerekir. Program ilk üç çağrıdan sonra sessizce çerçeve açmaya devam eder, yığın dolar ve çöker. `Segmentation fault` iletisini Linux ve macOS terminali yazar; Windows'ta program çoğu zaman bir şey yazmadan sonlanır. Yukarıdaki çıktı terminalde alınmıştır; çıktı bir dosyaya ya da boru hattına yönlendirilirse tamponlanan satırlar çökme sırasında kaybolabilir.

İkinci hata türü, temel durumun var olması ama **ulaşılamamasıdır**:

```c
int topla(int n) {
    if (n == 0) {          // negatif n bu koşula hiç ulaşamaz
        return 0;
    }
    return n + topla(n - 1);
}
// topla(-1): -1, -2, -3, ... sonsuza gider
```

> **▲ Sık Yapılan Hata · Temel durumu == ile yazmak**
>
> `topla(-1)` çağrısı `n == 0` koşuluna hiç ulaşamaz: `-1`, `-2`, `-3` diye gider ve yığın taşar. Girdinin negatif olabileceği her yerde temel durumu `n <= 0` yazmalı (ya da girdiyi önceden denetlemeli). Aynı hata `n == 1` ile biten bir faktöriyel fonksiyonuna 0 gönderince de ortaya çıkar.

Doğru yazılmış bir fonksiyon bile yeterince derin çağrılırsa yığını doldurabilir: `topla(1000000)` gibi bir çağrı çoğu sistemde çöker (sınır sisteme ve çerçeve boyutuna bağlıdır). Bu yüzden milyonlarca adım süren işleri döngüyle yazarız.

18.11 Özyineleme mi, Döngü mü?

Her özyinelemeli çözüm döngüye (gerekirse kendi yığınımızla birlikte) çevrilebilir. Seçim hız, bellek ve okunabilirlik açısından yapılır:

Ölçüt	Döngü	Özyineleme
Bellek	Sabit (birkaç değişken)	Derinlik kadar çerçeve; çok derinse yığın taşar
Hız	Genellikle biraz daha hızlı	Her çağrının çerçeve açma/kapama maliyeti vardır; `fib` gibi yerlerde tekrar eden hesap üstel yavaşlık yaratır
Okunabilirlik	Doğrusal işlerde (toplam, faktöriyel, dizi gezme) daha doğal	Problem kendine benzeyen parçalara bölünüyorsa (Hanoi, ikili arama, liste ve ağaç gezme) çok daha kısa ve net
Tipik hata	Sonsuz döngü	Temel durum eksikse yığın taşması

Pratik kural: İş doğrusalsa (1'den `n`'e topla, dizi gez) döngü; problem aynı türden alt problemlere bölünüyorsa (Hanoi, ağaç) ya da çözüm doğal olarak özyinelemeli tanımlanmışsa (Fibonacci, EBOB) özyineleme.

Kuyruk özyinelemesi

Özyinelemeli çağrı fonksiyonun **son işi** ise (çağrıdan sonra çarpma, toplama gibi bir şey yapılmıyorsa) buna **kuyruk özyinelemesi** (tail recursion) denir. `ebob` ve `ikiliAra` böyledir: `return ebob(b, a % b);`. `faktoriyel` böyle değildir: `n * faktoriyel(n - 1)` ifadesinde çağrıdan sonra çarpma yapılır.

Sonucu fazladan bir parametrede biriktirirsek faktöriyel de kuyruk özyinelemeli olur:

```
long long faktoriyelKuyruk(int n, long long biriken) {
    if (n <= 1) {
        return biriken;
    }
    return faktoriyelKuyruk(n - 1, n * biriken);    // çağrı son iş
}
// ilk çağrı: faktoriyelKuyruk(5, 1)
```

Kuyruk çağrısında çağıran çerçevenin işi bittiği için derleyici, `-O2` gibi optimizasyonlarla çerçeveyi yeniden kullanıp bunu döngüye çevirebilir. Ancak C standardı bunu garanti etmez; optimizasyonsuz derlemede çerçeve yine açılır. Dolayısıyla çok derin bir özyinelemeyi "kuyruk özyinelemesi olduğu için güvenli" saymamalıyız.

> **Üç Dilde · Özyineleme üç dilde** *C · C++ · C#*
>
> Özyineleme üç dilde neredeyse aynıdır; bu bölümdeki fikirler olduğu gibi geçerlidir. C++ ve C# kısımlarında konuyu kısaca tekrar edeceğiz (Bölüm 38 ve 58).

```c
// C
long long faktoriyel(int n) {
    if (n <= 1) return 1;
    return n * faktoriyel(n - 1);
}
```

```cpp
// C++ (Bölüm 38)
long long faktoriyel(int n) {
    if (n <= 1) return 1;
    return n * faktoriyel(n - 1);
}
```

```csharp
// C# (Bölüm 58)
static long Faktoriyel(int n)
{
    if (n <= 1) return 1;
    return n * Faktoriyel(n - 1);
}
```

Farklar küçüktür. C#'ta `long` her platformda 64 bittir; C ve C++'taki `long long` karşılığıdır. C# metot adları geleneksel olarak büyük harfle başlar. C#'ta da yığın taşması (`StackOverflowException`) yakalanamaz ve süreç sonlanır. Üç dilin hiçbiri kuyruk çağrısı optimizasyonunu dil düzeyinde garanti etmez.

Bölüm Özeti

- **Özyineleme**, fonksiyonun kendi kendini çağırmasıdır. Her özyinelemeli fonksiyonda bir **temel durum** (kendini çağırmadan cevap) ve bir **özyinelemeli durum** (daha küçük problemle kendini çağırma) bulunur; her çağrı temel duruma yaklaşmalıdır.
- Her çağrı kendi yığın çerçevesini açar; parametreler ve yerel değişkenler çağrıya özeldir. Yığındaki çerçeve sayısı özyineleme derinliğidir. Hesaplamalar çoğu zaman dönüşte yapılır (`n * faktoriyel(n - 1)`).
- Yazarken **güven ilkesi** işe yarar: küçük problemin doğru çözüldüğünü varsayıp büyük problemin cevabını ondan çıkarırız.
- Kalıplar: `topla(n) = n + topla(n - 1)`, `basamakToplami(n) = n % 10 + basamakToplami(n / 10)`, `ebob(a, b) = ebob(b, a % b)` (`b == 0` iken `a`), `us(a, n) = a * us(a, n - 1)`.
- Hızlı üs `n`'i her adımda yarıya indirir ve yaklaşık $\log_2 n$ çağrı yapar; yarım sonuç bir değişkende tutulmalı, iki kez hesaplanmamalıdır.
- Fibonacci'nin düz özyinelemeli yazımı aynı alt problemleri tekrar tekrar çözer; çağrı sayısı `C(n) = 1 + C(n-1) + C(n-2)` kuralıyla üstel büyür (1, 1, 3, 5, 9, 15, …).
- **Bellekleme** (memoization), hesaplanan sonuçları bir dizide saklayarak Fibonacci'yi `2n - 1` çağrıya indirir.
- Dizi ve string üzerinde problem, "ilk `n` eleman" ya da `s + 1` ile küçülür. Boş dizi (`n == 0`) ve boş string (`*s == '\0'`) tipik temel durumlardır; en büyük eleman gibi işlerde temel durum tek elemandır.
- Hanoi kuleleri: `n - 1` diski yedeğe, en büyüğü hedefe, `n - 1` diski hedefe taşı. Hamle sayısı $2^n - 1$'dir.
- Özyinelemeli ikili aramada aralık argümanlarla daralır; temel durumlar `alt > ust` (`-1`) ve `a[orta] == aranan`'dır. Özyinelemeli çağrının sonucu `return` edilmelidir.
- Çağrıdan **önce** yazdırma gidişte (büyükten küçüğe), **sonra** yazdırma dönüşte (ters sırada) çalışır. İki çağrılı fonksiyonlarda çağrılar soldan sağa açılır.

- Temel durum yoksa ya da ulaşılamıyorsa yığın taşar (Linux ve macOS'ta `Segmentation fault`). Negatif girdi ihtimalinde `n == 0` yerine `n <= 0` yazılır.
- Döngü sabit bellekle çalışır ve genelde biraz daha hızlıdır; özyineleme yığın kullanır ama kendine benzeyen problemlerde çok kısa ve nettir. Doğrusal işlerde döngü, bölünebilen problemlerde özyineleme seçilir.
- **Kuyruk özyinelemesi**, özyinelemeli çağrının fonksiyonun son işi olmasıdır (`ebob`). Derleyici bunu döngüye çevirebilir, ama C standardı bunu garanti etmez.
- C++ ve C#'ta özyineleme aynı biçimde yazılır (Bölüm 38 ve 58). Bağlı listelerde özyineleme çok doğaldır; Bölüm 19'da göreceğiz.

Alıştırmalar

1. `n <= 0` için 1 döndüren, `n > 0` için `n!` hesaplayan ve her `n` değeri için sonlanan fonksiyon hangisidir?
   A) `int f(int n) { if (n <= 1) return 1; return n * f(n - 1); }`
   B) `int f(int n) { return n * f(n - 1); }`
   C) `int f(int n) { if (n == 1) return 1; return n * f(n - 1); }`
   D) `int f(int n) { if (n == 0) return 1; return n * f(n + 1); }`
   E) `int f(int n) { if (n > 1) return 1; return n * f(n - 1); }`
2. `int fib(int n) { if (n < 2) return n; return fib(n - 1) + fib(n - 2); }` fonksiyonu `fib(5)` ile çağrılırsa `fib` ilk çağrı dahil toplam kaç kez çağrılır?
   A) 8
   B) 11
   C) 13
   D) 15
   E) 25
3. Temel durumu `n <= 1` olan bir `faktoriyel` fonksiyonuna `faktoriyel(6)` ile başlanıyor. Çağrı yığınında aynı anda en çok kaç `faktoriyel` çerçevesi bulunur? (`main`'in çerçevesi sayılmaz.)
   A) 5
   B) 6
   C) 7
   D) 720
   E) 1
4. Üç çubuklu Hanoi kulelerinde 5 diski kurallara uyarak bir çubuktan diğerine taşımak için en az kaç hamle gerekir?
   A) 15
   B) 25
   C) 31
   D) 32
   E) 63
5. Aşağıdakilerden hangisi doğrudur?
   A) Bellekleme, Fibonacci'nin çağrı sayısını üstelden doğrusala indirir.
   B) Özyinelemeli bir fonksiyonun yerel değişkenleri çağrılar arasında paylaşılır.
   C) Temel durumu olmayan özyinelemeli fonksiyon derleme hatası verir.

D) Bazı problemler yalnızca özyinelemeyle çözülebilir; döngüyle yazılamaz.

E) Özyinelemeli çözüm, aynı işi yapan döngüden her zaman daha hızlıdır.

6. Aşağıdaki fonksiyonlardan hangisi kuyruk özyinelemelidir (özyinelemeli çağrı fonksiyonun son işidir)?

A) `int f(int n) { if (n == 0) return 0; return n + f(n - 1); }`

B) `int f(int n, int b) { if (n == 0) return b; return f(n - 1, b + n); }`

C) `int f(int n) { if (n < 2) return n; return f(n - 1) + f(n - 2); }`

D) `int f(int n) { if (n == 0) return 1; return n * f(n - 1); }`

E) `int f(int n) { if (n == 0) return 0; return 1 + f(n / 10); }`

7. Aşağıdaki programın çıktısı nedir?

```c
#include <stdio.h>

void f(int n) {
    if (n == 0) {
        return;
    }
    printf("%d ", n);
    f(n - 1);
    printf("%d ", n);
}

int main(void) {
    f(3);
    printf("\n");
    return 0;
}
```

8. Aşağıdaki programın çıktısı nedir?

```c
#include <stdio.h>

int g(int a, int b) {
    if (b == 0) {
        return a;
    }
    return g(b, a % b);
}

int main(void) {
    printf("%d\n", g(84, 36));
    printf("%d\n", g(17, 5));
    return 0;
}
```

9. Aşağıdaki programın çıktısı nedir? (Çıktıdaki yıldızları sayın.)

```c
#include <stdio.h>

void f(int n) {
    if (n <= 1) {
        printf("* ");
        return;
    }
    f(n - 1);
    f(n - 2);
}
```

```c
int main(void) {
    f(5);
    printf("\n");
    return 0;
}
```

10. Aşağıdaki fonksiyonun hatası nedir? Derleyici ne bildirir ve nasıl düzeltilir?

```c
int topla(int n) {
    if (n == 0) {
        return 0;
    }
    n + topla(n - 1);
}
```

11. Aşağıdaki fonksiyon `n` sayısının ikilik tabanda kaç kez 2'ye bölünerek 1'e indirilebileceğini hesaplamak içindir (`say(16)` 4 vermelidir). `say(0)` çağrılırsa ne olur? Nasıl düzeltilir?

```c
int say(int n) {
    if (n == 1) {
        return 0;
    }
    return 1 + say(n / 2);
}
```

12. `fibHafiza` fonksiyonunda `hafiza` dizisi global yerine fonksiyonun içinde `long long hafiza[91];` olarak tanımlanırsa ne olur? Neden?
13. **Program yazın:** Klavyeden pozitif bir `n` okuyup `1² + 2² + … + n²` toplamını özyinelemeli `kareToplami(int n)` fonksiyonuyla hesaplayan bir program yazın (döngü yok).
14. **Program yazın:** `char` dizisini yerinde tersine çeviren özyinelemeli `tersCevir(char s[], int bas, int son)` fonksiyonunu yazın. Uçlardaki karakterleri değiştirip içeri doğru daralsın. `"ozyineleme"` metniyle deneyin.
15. **Program yazın:** Collatz kuralını uygulayan özyinelemeli `long long adim(long long n)` fonksiyonunu yazın: `n == 1` ise 0; `n` çiftse `1 + adim(n / 2)`; tekse `1 + adim(3 * n + 1)`. `adim(6)` ve `adim(27)` değerlerini yazdırın.
16. **Program yazın:** Pascal üçgeninin `n`. satırındaki `k`. değeri `c(n, k)` özyinelemeli tanımlanır: `k == 0` ya da `k == n` ise 1, aksi hâlde `c(n - 1, k - 1) + c(n - 1, k)`. Bu fonksiyonu yazıp üçgenin ilk beş satırını (`n = 0...4`) yazdıran bir program hazırlayın.

BÖLÜM 19

Veri Yapılarına Giriş: Bağlı Liste, Yığın ve Kuyruk

BU BÖLÜMDE

- Veri yapısı kavramını ve dizi ile bağlı listenin güçlü ve zayıf yanlarını
- Öz-referanslı yapıyla düğüm tanımlamayı; tek yönlü bağlı listede ekleme, arama, silme ve serbest bırakmayı
- Liste başını değiştiren fonksiyonlarda neden işaretçiye işaretçi (`Dugum **`) kullanıldığını
- Yığını (LIFO) dizi ve bağlı listeyle gerçekleştirmeyi; parantez dengesi denetimini
- Kuyruğu (FIFO) dairesel dizi ve bağlı listeyle gerçekleştirmeyi; bir sıra simülasyonunu
- Çift yönlü bağlı listenin neyi değiştirdiğini
- Big-O gösteriminin sezgisel anlamını ve işlemlerin maliyetini karşılaştırmayı

Şimdiye kadar veriyi hep dizilerde tuttuk. Dizi basit ve hızlıdır, ama her iş için uygun değildir: boyutu önceden belli olmalıdır, araya eleman sokmak için diğerlerini kaydırmak gerekir. Bu bölümde, verinin bellekte nasıl düzenleneceğine dair üç klasik çözümü tanıyacağız: **bağlı liste**, **yığın** ve **kuyruk**.

Bu bölüm, işaretçilerin (Bölüm 12), dinamik belleğin (Bölüm 13) ve yapıların (Bölüm 14) birlikte kullanıldığı ilk yerdir. Bu üç konuda kendimizi rahat hissetmiyorsak önce onlara dönmek iyi olur. Buradaki fikirler yalnızca C'ye ait değildir; C++'ta STL (Bölüm 35) ve C#'ta koleksiyonlar (Bölüm 54) aynı yapıları hazır sunar. Onları kullanmadan önce içlerinde ne olduğunu bir kez kendimiz kurmak, kitaplıkları gerçekten anlamamızı sağlar.

19.1 Veri Yapısı Nedir?

Tanım: Veri yapısı

Verinin bellekte, belirli işlemlerin (ekleme, silme, arama, dolaşma) verimli yapılabileceği biçimde düzenlenmesidir. Her veri yapısı, verinin nasıl saklanacağını ve hangi işlemlerin nasıl çalışacağını birlikte tanımlar.

Dizi de bir veri yapısıdır: elemanlar bellekte yan yana durur ve `dizi[i]` ile hemen erişilir. Bu güç, aynı zamanda sınırlarının kaynağıdır:

Özellik	Dizi	Bağlı liste
Bellek düzeni	Ardışık, tek blok	Dağınık, her eleman ayrı blok
Boyut	Sabit (ya da `realloc` ile kopyalayarak büyür)	Elemanla birlikte büyür ve küçülür
`i`. elemana erişim	Anında	`i` adım gezmek gerekir

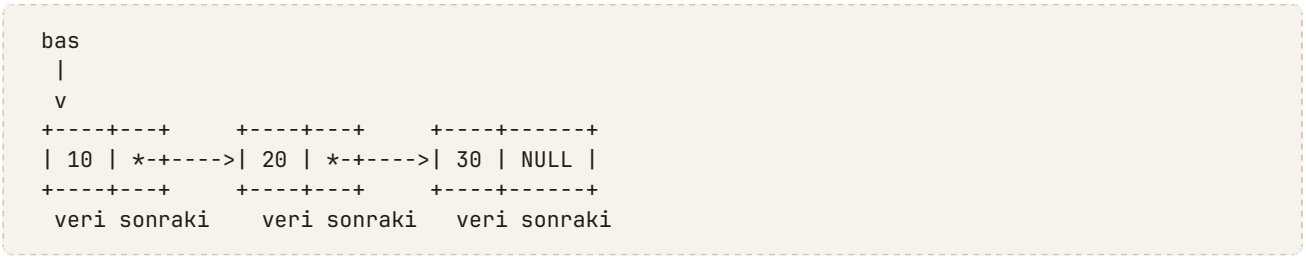
Özellik	Dizi	Bağlı liste
Başa ekleme	Hepsi kaydırılır	Birkaç işaretçi güncellenir
Araya ekleme/silme	Kaydırma gerekir	Konum biliniyorsa işaretçi güncellemesi yeter
Eleman başına ek bellek	Yok	Bir işaretçi kadar

Hiçbiri her durumda üstün değildir. Elemana sıra numarasıyla çok erişiyorsak dizi, sürekli ekleyip çıkarıyorsak bağlı liste daha uygun olabilir. Bu bölümün sonunda bu seçimi sayılarla tartışacağız.

19.2 Düğüm ve Tek Yönlü Bağlı Liste

Bağlı listenin fikri şudur: her eleman, kendi verisinin yanında **bir sonraki elemanın adresini** de taşır. Bu parçaya **düğüm** (node) denir. Elemanlar bellekte yan yana durmak zorunda değildir; birbirlerine işaretçilerle bağlanırlar.

> **Tanım: Bağlı liste**
>
> Her düğümün bir veri alanı ve bir sonraki düğümü gösteren bir işaretçisi olduğu, son düğümün işaretçisinin `NULL` olduğu veri yapısıdır. Listeye, ilk düğümü gösteren **baş** (head) işaretçisi aracılığıyla ulaşılır.



Düğümü bir yapıyla (Bölüm 14) tanımlarız. Yapının içinde kendi türünden bir işaretçi bulunur; bu duruma **öz-referanslı yapı** denir:

```c
typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;   // bir sonraki düğümün adresi
} Dugum;
```

Burada küçük bir ayrıntı var: yapının içinde `Dugum *sonraki` yazamayız, çünkü `typedef` adı henüz tamamlanmamıştır. Yapı etiketini (`struct Dugum`) kullanmak zorundayız. Dolayısıyla etiketi hem `struct Dugum` hem `typedef` satırında tutuyoruz.

> **◆ Kritik Nokta · Liste = baş işaretçisi**
>
> Listenin kendisi diye ayrı bir nesne yoktur; elimizde yalnızca ilk düğümü gösteren bir işaretçi vardır. Bu işaretçiyi kaybedersek bütün listeye erişimi kaybederiz. Boş liste, baş işaretçisinin `NULL` olmasıdır.

Düğüm oluşturma ve elle bağlama

Her düğümü `malloc` ile öbekte (heap) yaratırız; böylece düğüm, onu yaratan fonksiyon bitse de yaşamaya devam eder. Bir düğüm yaratıp ilk değerlerini veren küçük bir fonksiyon her yerde işimize yarayacak:

```
Dugum *dugumOlustur(int veri) {
    Dugum *d = malloc(sizeof(Dugum));
    if (d == NULL) {
        return NULL;           // bellek yetersiz
    }
    d->veri = veri;
    d->sonraki = NULL;
    return d;
}
```

Yapı işaretçisinden üyeye `->` ile ulaşıldığını Bölüm 14'te gördük: `d->veri`, `(*d).veri` ile aynıdır. Şimdi üç düğümü elle bağlayıp listeyi dolaşalım:

▸ elleBaglama.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

Dugum *dugumOlustur(int veri) {
    Dugum *d = malloc(sizeof(Dugum));
    if (d == NULL) {
        return NULL;
    }
    d->veri = veri;
    d->sonraki = NULL;
    return d;
}

int main(void) {
    Dugum *bas = dugumOlustur(10);
    bas->sonraki = dugumOlustur(20);
    bas->sonraki->sonraki = dugumOlustur(30);

    Dugum *p = bas;                // dolaşma işaretçisi
    while (p != NULL) {
        printf("%d\n", p->veri);
        p = p->sonraki;            // bir sonraki düğüme ilerle
    }

    free(bas->sonraki->sonraki);   // sondan başa serbest bırak
    free(bas->sonraki);
    free(bas);
    return 0;
}
```

```
ÇIKTI
10
20
30
```

Bu kısa programda dört şey öğreniyoruz:

1. `bas->sonraki->sonraki` zinciri, "başın sonrakisinin sonrakisi" demektir; üçüncü düğüme gider.
2. Dolaşmak için baş işaretçisini bozmayız; kopyası olan `p`'yi ilerletiriz. `bas`'ı ilerletseydik listenin başını kaybederdik.

3. Dolaşma koşulu `p != NULL`'dır; yani son düğümde de içeri girer, `NULL`'a ulaşınca durur. `p->sonraki != NULL` yazarsak son düğümü işlemeden çıkarız.
4. Her `malloc`'a bir `free` düşer. Burada `free`'yi sondan başa yaptık, çünkü bir düğümü serbest bırakınca ona ait `sonraki` alanı da gider. Bunu genel biçimde birazdan yazacağız.

Elle bağlamak üç düğüm için tamam, ama yüz düğüm için olmaz. Ekleme ve silmeyi fonksiyonlara bölelim.

19.3 Liste Başını Değiştiren Fonksiyonlar

Başa eleman eklemek, baş işaretçisinin değerini değiştirir: artık liste yeni düğümden başlar. Fonksiyonlara değerle geçtiğimizi hatırlayalım (Bölüm 9): `void basaEkle(Dugum *bas, int veri)` yazarsak fonksiyon yalnızca işaretçinin kopyasını değiştirir, çağıran taraftaki `bas` aynen kalır. Bir değişkeni fonksiyonda değiştirmek için adresini geçmek gerektiğini Bölüm 12'de `swap` ile görmüştük. Burada değiştirilecek değişken bir işaretçidir; dolayısıyla adresi **işaretçiye işaretçi** olur:

```
void basaEkle(Dugum **bas, int veri);   // bas: baş işaretçisinin adresi
```

Çağrı `basaEkle(&liste, 5);` biçiminde yapılır; fonksiyon içinde `*bas`, çağıran taraftaki `liste`'nin kendisidir. İlk bakışta zor görünen bu çift yıldız, `swap` mantığının aynısıdır.

> **▲ Sık Yapılan Hata · Baş değişiyorsa Dugum ** gerekir**
>
> Başa ekleme, baştan silme ve ters çevirme gibi baş işaretçisini değiştirebilen işlemlerde parametre `Dugum **` olmalıdır. Yalnızca dolaşan işlemler (yazdırma, arama, uzunluk) için `Dugum *` yeter. Alternatif, yeni baş işaretçisini fonksiyondan döndürmektir (`bas = basaEkle(bas, 5);`); bu da geçerlidir ama dönüş değerini atamayı unutmak kolaydır.

Başa ve sona ekleme

Başa eklemek üç adımdır ve **sıra önemlidir**:

```
Önce:     bas --> [20] --> [30] --> NULL

Adım 1:   yeni = [10]                      (yeni düğüm yarat)
Adım 2:   yeni->sonraki = bas              [10] --> [20] --> [30]
Adım 3:   bas = yeni                       bas --> [10] --> [20] --> [30]
```

Üçüncü adımı ikinciden önce yaparsak eski listenin başı kaybolur ve bellek sızar. Sona eklemek ise önce son düğümü bulmayı gerektirir; boş listede ise ekleme başa eklemeyle aynıdır.

Aşağıdaki program, listeyi yazdırma, uzunluk bulma, arama ve serbest bırakma fonksiyonlarıyla birlikte tüm temel işlemleri içerir:

▸ baglaListe.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;
```

```
Dugum *dugumOlustur(int veri) {
    Dugum *d = malloc(sizeof(Dugum));
    if (d == NULL) {
        return NULL;
    }
    d->veri = veri;
    d->sonraki = NULL;
    return d;
}

int basaEkle(Dugum **bas, int veri) {
    Dugum *yeni = dugumOlustur(veri);
    if (yeni == NULL) {
        return 0;
    }
    yeni->sonraki = *bas;       // önce eski listeye bağla
    *bas = yeni;                // sonra başı güncelle
    return 1;
}

int sonaEkle(Dugum **bas, int veri) {
    Dugum *yeni = dugumOlustur(veri);
    if (yeni == NULL) {
        return 0;
    }
    if (*bas == NULL) {         // boş liste: yeni düğüm baş olur
        *bas = yeni;
        return 1;
    }
    Dugum *p = *bas;
    while (p->sonraki != NULL) {    // son düğümü bul
        p = p->sonraki;
    }
    p->sonraki = yeni;
    return 1;
}

void yazdir(const Dugum *bas) {
    for (const Dugum *p = bas; p != NULL; p = p->sonraki) {
        printf("%d -> ", p->veri);
    }
    printf("NULL\n");
}

int uzunluk(const Dugum *bas) {
    int sayac = 0;
    for (const Dugum *p = bas; p != NULL; p = p->sonraki) {
        sayac++;
    }
    return sayac;
}

Dugum *ara(Dugum *bas, int aranan) {
    for (Dugum *p = bas; p != NULL; p = p->sonraki) {
        if (p->veri == aranan) {
            return p;           // bulunan düğümün adresi
        }
    }
    return NULL;
}

void serbestBirak(Dugum **bas) {
    Dugum *p = *bas;
```

```
    while (p != NULL) {
        Dugum *sonraki = p->sonraki;    // free'den önce sakla
        free(p);
        p = sonraki;
    }
    *bas = NULL;
}

int main(void) {
    Dugum *liste = NULL;

    basaEkle(&liste, 20);
    basaEkle(&liste, 10);
    sonaEkle(&liste, 30);
    sonaEkle(&liste, 40);
    yazdir(liste);
    printf("Eleman sayısı: %d\n", uzunluk(liste));

    Dugum *bulunan = ara(liste, 30);
    if (bulunan != NULL && bulunan->sonraki != NULL) {
        printf("30 bulundu, ardından %d geliyor\n", bulunan->sonraki->veri);
    }
    if (ara(liste, 99) == NULL) {
        printf("99 listede yok\n");
    }

    serbestBirak(&liste);
    yazdir(liste);
    return 0;
}
```

```
ÇIKTI
10 -> 20 -> 30 -> 40 -> NULL
Eleman sayısı: 4
30 bulundu, ardından 40 geliyor
99 listede yok
NULL
```

Programın dikkat çeken yerleri şunlardır:

- `basaEkle(&liste, 20)` ve ardından `basaEkle(&liste, 10)` listeyi `10 -> 20` yapar; başa eklenen eleman öne geçer, yani ekleme sırasının tersi oluşur.
- Dolaşan fonksiyonlar `const Dugum *` alır: listeyi değiştirmeyeceklerini imzalarında söylerler (Bölüm 12).
- `ara`, değer yerine **düğümün adresini** döndürür. Böylece çağıran taraf yalnızca var mı yok mu öğrenmez, düğümün kendisine de ulaşır. Bulunamazsa `NULL` döner; bu yüzden dönüşü kullanmadan önce denetlemek gerekir.
- `serbestBirak` içinde `sonraki` önceden saklanır. Bunu yapmasaydık `free(p)` sonrasında `p->sonraki` okumak, serbest bırakılmış belleği kullanmak olurdu.

> **▲ Sık Yapılan Hata · free(p); p = p->sonraki; hatası**
>
> Bu iki satırı bu sırayla yazmak, listeyi boşaltırken yapılan en yaygın hatadır. `free(p)` sonrasında `p` artık geçersiz bir adres gösterir ve `p->sonraki` tanımsız davranıştır. Bazen çalışır gibi görünür, bazen çöker. Doğrusu, bir sonraki adresi önce bir değişkene almak ve ancak sonra `free` çağırmaktır.

Sona eklemenin maliyeti

`sonaEkle` her çağrıda listenin tamamını gezer; 1000 elemanlı bir listeye eklemek 1000 adım sürer. Sürekli sona ekleyen programlar için çözümlerden biri, baş işaretçisinin yanında **son işaretçisini** de tutmaktır. Kuyruğu gerçekleştirirken bu fikri kullanacağız.

19.4 Silme ve Sıralı Ekleme

Bir düğümü silmek için ondan **önceki** düğümün `sonraki` alanını, silinecek düğümün sonrakine bağlamak gerekir:

```
Önce:   bas --> [10] --> [20] --> [30] --> NULL
                 onceki    p

20 silinince:
        bas --> [10] ---------> [30] --> NULL      ([20] free ile bırakılır)
                onceki->sonraki = p->sonraki
```

Bu yüzden arama sırasında hem `p`'yi hem de bir geriden gelen `onceki` işaretçisini taşırız. İki özel durum vardır: silinecek düğüm listenin başındaysa `onceki` yoktur ve baş işaretçisi değişir (bu yüzden `Dugum **`); değer listede yoksa hiçbir şey yapılmaz.

Sıralı ekleme de aynı iki işaretçi mantığına dayanır: yeni değerden küçük olan son düğümde durup yeni düğümü onun arkasına bağlarız. Ekleme küçükten büyüğe sıralı yaparsak liste her zaman sıralı kalır. Aşağıdaki programda ayrıca listeyi ters çeviren bir fonksiyon da var. Ters çevirme, her düğümün `sonraki` alanını bir öncekine döndürmekten ibarettir:

▸ listeSilme.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

Dugum *dugumOlustur(int veri) {
    Dugum *d = malloc(sizeof(Dugum));
    if (d == NULL) {
        return NULL;
    }
    d->veri = veri;
    d->sonraki = NULL;
    return d;
}

void siraliEkle(Dugum **bas, int veri) {
    Dugum *yeni = dugumOlustur(veri);
    if (yeni == NULL) {
        return;
    }
    if (*bas == NULL || veri < (*bas)->veri) {   // başa eklenecek
        yeni->sonraki = *bas;
        *bas = yeni;
        return;
    }
    Dugum *p = *bas;
```

```
    while (p->sonraki != NULL && p->sonraki->veri < veri) {
        p = p->sonraki;
    }
    yeni->sonraki = p->sonraki;     // önce yeni düğümü sonrasına bağla
    p->sonraki = yeni;              // sonra öncesinden yeni düğüme bağla
}

int sil(Dugum **bas, int aranan) {
    Dugum *p = *bas;
    Dugum *onceki = NULL;
    while (p != NULL && p->veri != aranan) {
        onceki = p;
        p = p->sonraki;
    }
    if (p == NULL) {
        return 0;                   // bulunamadı
    }
    if (onceki == NULL) {
        *bas = p->sonraki;          // baştaki düğüm siliniyor
    } else {
        onceki->sonraki = p->sonraki;
    }
    free(p);
    return 1;
}

void tersCevir(Dugum **bas) {
    Dugum *onceki = NULL;
    Dugum *p = *bas;
    while (p != NULL) {
        Dugum *sonraki = p->sonraki;
        p->sonraki = onceki;        // oku geri çevir
        onceki = p;
        p = sonraki;
    }
    *bas = onceki;
}

void yazdir(const Dugum *bas) {
    for (const Dugum *p = bas; p != NULL; p = p->sonraki) {
        printf("%d -> ", p->veri);
    }
    printf("NULL\n");
}

void serbestBirak(Dugum **bas) {
    while (*bas != NULL) {
        Dugum *sonraki = (*bas)->sonraki;
        free(*bas);
        *bas = sonraki;
    }
}

int main(void) {
    Dugum *liste = NULL;
    int degerler[] = {30, 10, 20, 40, 25};
    for (int i = 0; i < 5; i++) {
        siraliEkle(&liste, degerler[i]);
    }
    yazdir(liste);

    printf("10 silindi mi: %d\n", sil(&liste, 10));    // baştaki
    printf("40 silindi mi: %d\n", sil(&liste, 40));    // sondaki
    printf("99 silindi mi: %d\n", sil(&liste, 99));    // yok
```

```
    yazdir(liste);

    sil(&liste, 25);
    tersCevir(&liste);
    yazdir(liste);

    serbestBirak(&liste);
    return 0;
}
```

```
ÇIKTI
10 -> 20 -> 25 -> 30 -> 40 -> NULL
10 silindi mi: 1
40 silindi mi: 1
99 silindi mi: 0
20 -> 25 -> 30 -> NULL
30 -> 20 -> NULL
```

`serbestBirak` bu kez baş işaretçisinin kendisini ilerleterek çalışır; döngü bittiğinde `*bas` zaten `NULL` olur. Her iki biçim de doğrudur.

> **İpucu · Önce kâğıda çizin**
>
> Bağlı liste hatalarının çoğu, işaretçi atamalarının sırasından çıkar. Kod yazmadan önce düğümleri ve okları kâğıda çizmek, hangi atamanın hangisinden önce gelmesi gerektiğini hemen gösterir. Şüphede kaldığımızda iki kural yardımcı olur: bir işaretçiyi değiştirmeden önce eski değerine hâlâ ulaşabildiğimizden emin olmak ve her `malloc`'a karşılık tek bir `free` yazmak.

Programlarımızın sızıntı yapıp yapmadığını `valgrind --leak-check=full ./liste` ile ya da derlerken `-fsanitize=address` seçeneğiyle denetleyebiliriz (Bölüm 13). Yukarıdaki programların hepsinde her düğüm bırakılır.

> **Not · Özyinelemeyle liste**
>
> Bağlı liste özyinelemeli bir yapıdır: bir liste, bir düğüm ile "geri kalan listeden" oluşur. Bu yüzden yazdırma, uzunluk ve ters yazdırma gibi işlemler Bölüm 18'deki yöntemle kısacık yazılabilir. Ancak çok uzun listelerde her düğüm bir çağrı çerçevesi demektir ve yığın taşabilir; bu nedenle burada döngüleri tercih ettik.

19.5 Yığın (Stack)

Bir tabak yığınını düşünelim: yeni tabağı en üste koyarız, tabak almak istediğimizde de en üsttekini alırız. En son konulan ilk çıkar. Yığın bu davranışın veri yapısıdır.

> **Tanım: Yığın (stack)**
>
> Ekleme ve çıkarmanın yalnızca bir uçtan, **tepe** (top) denilen uçtan yapıldığı veri yapısıdır. Son giren ilk çıkar; bu ilkeye **LIFO** (Last In, First Out) denir.

Yığının üç temel işlemi vardır:

İşlem	Anlamı
`push`	Tepeye bir eleman koyar

İşlem	Anlamı
`pop`	Tepedeki elemanı çıkarır ve değerini verir
`peek`	Tepedeki elemana bakar, çıkarmaz

Bunlara boş olup olmadığını (`bosMu`) ve dolu olup olmadığını (`doluMu`) soran işlemler de eşlik eder. Bölüm 9'da gördüğümüz çağrı yığını (call stack) aynı mantıkla çalışır: son çağrılan fonksiyon önce biter. Bir editördeki "geri al" (undo) de, tarayıcının "geri" düğmesi de yığınla çalışır.

Dizi ile yığın

En basit gerçekleştirme, bir dizi ve tepenin indeksini tutan bir sayaçtır. Boş yığında tepe `-1`'dir; `push` önce tepeyi bir artırır, sonra elemanı yazar. Dizi ve tepe sayacı birlikte anlamlı olduğu için ikisini bir yapıda toplarız:

▸ dizilYigin.c

```c
#include <stdio.h>

#define KAPASITE 5

typedef struct {
    int veri[KAPASITE];
    int tepe;                   // en üstteki elemanın indeksi
} Yigin;

void yiginBaslat(Yigin *y) {
    y->tepe = -1;               // boş yığın
}

int bosMu(const Yigin *y) {
    return y->tepe == -1;
}

int doluMu(const Yigin *y) {
    return y->tepe == KAPASITE - 1;
}

int push(Yigin *y, int deger) {
    if (doluMu(y)) {
        return 0;               // taşma (overflow)
    }
    y->tepe++;
    y->veri[y->tepe] = deger;
    return 1;
}

int pop(Yigin *y, int *deger) {
    if (bosMu(y)) {
        return 0;               // boş yığından çıkarılamaz (underflow)
    }
    *deger = y->veri[y->tepe];
    y->tepe--;
    return 1;
}

int peek(const Yigin *y, int *deger) {
    if (bosMu(y)) {
        return 0;
    }
    *deger = y->veri[y->tepe];
```

```
    return 1;
}

int main(void) {
    Yigin y;
    yiginBaslat(&y);
    int d;

    push(&y, 10);
    push(&y, 20);
    push(&y, 30);
    if (peek(&y, &d)) {
        printf("Tepe: %d\n", d);
    }

    pop(&y, &d);
    printf("Çıkan: %d\n", d);
    pop(&y, &d);
    printf("Çıkan: %d\n", d);

    for (int i = 40; i <= 80; i += 10) {
        if (!push(&y, i)) {
            printf("%d eklenemedi, yığın dolu\n", i);
        }
    }

    printf("Boşaltılıyor:");
    while (pop(&y, &d)) {
        printf(" %d", d);
    }
    printf("\n");
    printf("Boş mu: %d\n", bosMu(&y));
    return 0;
}
```

```
ÇIKTI
Tepe: 30
Çıkan: 30
Çıkan: 20
80 eklenemedi, yığın dolu
Boşaltılıyor: 70 60 50 40 10
Boş mu: 1
```

Eklenme sırası 10, 40, 50, 60, 70'tir; çıkış sırası bunun tersidir. `pop` ve `peek` iki değer taşımak zorundadır: işlemin başarılı olup olmadığı ve çıkan değer. Boş yığından gelen "0" geçerli bir veri de olabileceği için bu ikisini ayırdık: başarı durumu dönüş değeriyle, değer ise işaretçi parametreyle iletilir (Bölüm 12).

`Yigin y;` satırından sonra `yiginBaslat(&y)` çağırmak zorundayız; yoksa `tepe` çöp değer taşır. İlk değerleri tek satırda vermek isteseydik `Yigin y = { {0}, -1 };` yazabilirdik (Bölüm 14).

Bağlı liste ile yığın

Kapasiteyi önceden sınırlamak istemiyorsak yığını bağlı listeyle kurabiliriz. Burada liste başı yığının tepesidir: `push` başa ekleme, `pop` baştan silme demektir ve ikisi de tek adımdır. Bir önceki bölümde yazdığımız `basaEkle` ile baştan silmenin yeniden kullanıldığını görmek önemlidir; yeni bir fikir yok, yalnızca adlar değişiyor.

Uygulama olarak, bir sayıyı ikilik tabana çeviren programı yazalım. Bir sayıyı 2'ye bölüp kalanları topladığımızda basamakları sondan başa elde ederiz; yığın tam olarak bu sırayı tersine çevirir:

ikilikYigin.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

int push(Dugum **tepe, int deger) {
    Dugum *yeni = malloc(sizeof(Dugum));
    if (yeni == NULL) {
        return 0;
    }
    yeni->veri = deger;
    yeni->sonraki = *tepe;
    *tepe = yeni;
    return 1;
}

int pop(Dugum **tepe, int *deger) {
    if (*tepe == NULL) {
        return 0;
    }
    Dugum *silinecek = *tepe;
    *deger = silinecek->veri;
    *tepe = silinecek->sonraki;
    free(silinecek);
    return 1;
}

void ikilikYazdir(unsigned int sayi) {
    Dugum *yigin = NULL;
    int basamak;

    if (sayi == 0) {
        printf("0");
    }
    while (sayi > 0) {
        push(&yigin, (int)(sayi % 2));   // kalanlar sondan başa gelir
        sayi /= 2;
    }
    while (pop(&yigin, &basamak)) {      // yığın sırayı tersine çevirir
        printf("%d", basamak);
    }
}

int main(void) {
    unsigned int sayilar[] = {13, 37, 255, 0};
    for (int i = 0; i < 4; i++) {
        printf("%u = ", sayilar[i]);
        ikilikYazdir(sayilar[i]);
        printf("\n");
    }
    return 0;
}
```

```
ÇIKTI
13 = 1101
37 = 100101
255 = 11111111
0 = 0
```

`ikilikYazdir` bittiğinde yığın zaten boştur, çünkü ikinci döngü `pop` başarısız olana kadar sürmüştür; ayrıca bir `free` döngüsüne gerek kalmaz.

Uygulama: parantez dengesi

Yığının en klasik uygulaması, bir ifadedeki parantezlerin doğru eşleşip eşleşmediğini denetlemektir. Kural basittir:

- Açan parantez (`(`, `[`, `{`) görünce yığına koyarız.
- Kapatan parantez görünce yığının tepesindeki açan parantezle eşleşmesi gerekir; eşleşmiyorsa ya da yığın boşsa ifade dengesizdir.
- İfade bitince yığın boş olmalıdır; kapanmamış parantez kalmamalıdır.

Son açılan parantezin ilk kapanması gerektiği için, bu kural kendiliğinden LIFO'dur.

▸ parantezDenge.c

```c
#include <stdio.h>

#define AZAMI 100

int eslesiyor(char acan, char kapatan) {
    return (acan == '(' && kapatan == ')') ||
           (acan == '[' && kapatan == ']') ||
           (acan == '{' && kapatan == '}');
}

int dengeliMi(const char *metin) {
    char yigin[AZAMI];
    int tepe = -1;

    for (int i = 0; metin[i] != '\0'; i++) {
        char c = metin[i];
        if (c == '(' || c == '[' || c == '{') {
            if (tepe == AZAMI - 1) {
                return 0;               // yığın kapasitesi aşıldı
            }
            yigin[++tepe] = c;          // push
        } else if (c == ')' || c == ']' || c == '}') {
            if (tepe == -1) {
                return 0;               // eşi olmayan kapatan parantez
            }
            char acan = yigin[tepe--];  // pop
            if (!eslesiyor(acan, c)) {
                return 0;               // yanlış türde eşleşme
            }
        }
    }
    return tepe == -1;                  // kapanmamış parantez kaldı mı?
}

int main(void) {
    const char *denemeler[] = {
        "(a + b) * [c - d]",
```

```
        "{[()]}",
        "([)]",
        "((x)",
        "x + y)",
        ""
    };
    for (int i = 0; i < 6; i++) {
        printf("\"%s\" -> %s\n", denemeler[i],
               dengeliMi(denemeler[i]) ? "dengeli" : "dengesiz");
    }
    return 0;
}
```

```
ÇIKTI
"(a + b) * [c - d]" -> dengeli
"{[()]}" -> dengeli
"([)]" -> dengesiz
"((x)" -> dengesiz
"x + y)" -> dengesiz
"" -> dengeli
```

Bu kez yığın yalnızca bir fonksiyonun yerel dizisidir; ayrı bir yapı tanımlamaya gerek duymadık. Dizi ve tepe indeksiyle `yigin[++tepe] = c` ve `yigin[tepe--]` yazmak, `push` ve `pop`'un en kısa halidir. `[` ile `)` gibi farklı türleri eşleştirmek için yığına yalnızca açan parantezin kendisini koymak yeterli oldu. `"([)]"` örneğinde parantez sayıları eşit olduğu hâlde ifade dengesizdir; yalnızca sayaç tutan bir çözüm bunu yakalayamazdı.

Üç Dilde · Yığın: Üç dilde *C · C++ · C#*

C'de yığını kendimiz yazdık. C++ ve C# aynısını hazır sunar; ayrıntıları Bölüm 35 ve 54'te işleyeceğiz.

```cpp
#include <stack>
std::stack<int> y;
y.push(10);
y.push(20);
int t = y.top();   // 20, çıkarmaz
y.pop();           // değer döndürmez, yalnızca çıkarır
bool bos = y.empty();
```

```csharp
var y = new Stack<int>();
y.Push(10);
y.Push(20);
int t = y.Peek();  // 20, çıkarmaz
int c = y.Pop();   // 20'yi çıkarır ve döndürür
bool bos = y.Count == 0;
```

C++'ta `pop()` değer döndürmez; değeri önce `top()` ile okumak gerekir. C#'ta `Pop()` hem çıkarır hem döndürür. Her ikisinde de bellek yönetimi ve taşma denetimi kitaplığın işidir.

19.6 Kuyruk (Queue)

Bir bilet gişesindeki sırada ilk gelen ilk hizmet alır. Kuyruk bu davranışın veri yapısıdır.

> **Tanım: Kuyruk (queue)**
> Eklemenin bir uçtan (**son**, rear), çıkarmanın diğer uçtan (**ön**, front) yapıldığı veri yapısıdır. İlk giren ilk çıkar; bu ilkeye **FIFO** (First In, First Out) denir.

Kuyruğun işlemleri `enqueue` (sona ekle) ve `dequeue` (önden çıkar) adlarıyla anılır. Yazıcı kuyruğu, işletim sisteminin süreç sıraları ve ağ paketlerinin ara belleği kuyruk mantığıyla çalışır.

Dizi ile kuyruk: dairesel dizi

Kuyruğu düz bir dizide kurmaya kalkarsak bir sorunla karşılaşırız. `dequeue` ön tarafı boşaltır, `enqueue` arka tarafa ekler; bir süre sonra dizinin sonu dolar, başı ise boş kalır. Elemanları sola kaydırabiliriz, ama bu her çıkarmada yük bindirir. Daha iyi çözüm, diziyi **dairesel** (circular) düşünmektir: dizinin sonuna gelen indeks, `%` ile başa döner.

```
KAPASITE = 4, indeksler 0..3

  [ ][ ][ ][ ]   boş kuyruk,   bas = 0, son = 0, adet = 0
  [A][B][C][D]   dolu,         bas = 0, son = 0, adet = 4
  [ ][ ][C][D]   A ve B çıktı, bas = 2, son = 0, adet = 2
  [E][F][C][D]   E ve F baştan eklendi, bas = 2, son = 2, adet = 4
```

İki indeks tutarız: `bas`, çıkarılacak elemanın yerini; `son`, bir sonraki elemanın yazılacağı yeri gösterir. Her ikisi de ilerlerken `(indeks + 1) % KAPASITE` ile dizinin başına döner. Üçüncü bir sayaç olan `adet`, boş ile dolu durumlarını ayırmaya yarar: son çizimde `bas` ve `son` ikisi de 2'dir, ama kuyruk doludur; ilk çizimde ise ikisi de 0'dır ve kuyruk boştur. Yalnızca indekslere bakarak bu ikisini ayıramayız.

▸ dairesel.c

```c
#include <stdio.h>

#define KAPASITE 4

typedef struct {
    int veri[KAPASITE];
    int bas;                    // çıkarılacak elemanın indeksi
    int son;                    // yeni elemanın yazılacağı indeks
    int adet;
} Kuyruk;

int enqueue(Kuyruk *k, int deger) {
    if (k->adet == KAPASITE) {
        return 0;               // kuyruk dolu
    }
    k->veri[k->son] = deger;
    k->son = (k->son + 1) % KAPASITE;
    k->adet++;
    return 1;
}

int dequeue(Kuyruk *k, int *deger) {
    if (k->adet == 0) {
        return 0;               // kuyruk boş
    }
    *deger = k->veri[k->bas];
    k->bas = (k->bas + 1) % KAPASITE;
    k->adet--;
    return 1;
```

```
}

int main(void) {
    Kuyruk k = { {0}, 0, 0, 0 };
    int d;

    for (int i = 10; i <= 50; i += 10) {
        if (!enqueue(&k, i)) {
            printf("%d eklenemedi, kuyruk dolu\n", i);
        }
    }

    dequeue(&k, &d);
    printf("Çıkan: %d\n", d);
    dequeue(&k, &d);
    printf("Çıkan: %d\n", d);

    enqueue(&k, 50);            // dizinin başındaki boş yere yazılır
    enqueue(&k, 60);
    printf("bas = %d, son = %d, adet = %d\n", k.bas, k.son, k.adet);

    printf("Çıkanlar:");
    while (dequeue(&k, &d)) {
        printf(" %d", d);
    }
    printf("\n");
    return 0;
}
```

```
ÇIKTI
50 eklenemedi, kuyruk dolu
Çıkan: 10
Çıkan: 20
bas = 2, son = 2, adet = 4
Çıkanlar: 30 40 50 60
```

`50` ilk denemede dolu kuyruğa sığmadı; iki eleman çıktıktan sonra aynı değer, `veri[0]`'a yazıldı ve `son` 1'e sarıldı. Elemanlar hiç kaydırılmadı; yalnızca iki indeks döndü.

> **▲ Sık Yapılan Hata · Dairesel kuyrukta % unutmak**
>
> `son++` yazıp `% KAPASITE` ile sarmayı unutmak, dizinin dışına yazmaya ve tanımsız davranışa yol açar. Aynı biçimde `bas` için de sarma gerekir. İlk dört elemanla her şey doğru çalışıp beşinci elemanda bozulması bu hatanın belirtisidir.

Bağlı liste ile kuyruk

Kapasite sınırı istemiyorsak kuyruğu bağlı listeyle kurarız. Burada az önce andığımız fikir devreye girer: hem **baş** hem **son** düğümü gösteren iki işaretçi tutarız. Ekleme son düğümün arkasına, çıkarma baştan yapılır; ikisi de tek adımdır, liste gezmek gerekmez.

Uygulama olarak, bir gişe simülasyonu yazalım. Müşteriler geldikçe sıraya numara alır (`G` olayı), gişe boşaldıkça sıradaki müşteri çağrılır (`C` olayı). Olayları bir metin olarak veriyoruz ve soldan sağa işliyoruz:

▸ gise.c

```
#include <stdio.h>
#include <stdlib.h>
```

```c
typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

typedef struct {
    Dugum *bas;                 // önde bekleyen
    Dugum *son;                 // en son gelen
    int adet;
} Kuyruk;

int enqueue(Kuyruk *k, int deger) {
    Dugum *yeni = malloc(sizeof(Dugum));
    if (yeni == NULL) {
        return 0;
    }
    yeni->veri = deger;
    yeni->sonraki = NULL;
    if (k->son == NULL) {       // kuyruk boştu
        k->bas = yeni;
    } else {
        k->son->sonraki = yeni;
    }
    k->son = yeni;
    k->adet++;
    return 1;
}

int dequeue(Kuyruk *k, int *deger) {
    if (k->bas == NULL) {
        return 0;
    }
    Dugum *silinecek = k->bas;
    *deger = silinecek->veri;
    k->bas = silinecek->sonraki;
    if (k->bas == NULL) {       // kuyruk boşaldı: son da sıfırlanmalı
        k->son = NULL;
    }
    free(silinecek);
    k->adet--;
    return 1;
}

int main(void) {
    Kuyruk gise = {NULL, NULL, 0};
    const char *olaylar = "GGGCGCCCC";
    int sonrakiNumara = 1;
    int numara;

    for (int i = 0; olaylar[i] != '\0'; i++) {
        if (olaylar[i] == 'G') {
            enqueue(&gise, sonrakiNumara);
            printf("Bilet alındı: %d (sırada %d kişi)\n",
                   sonrakiNumara, gise.adet);
            sonrakiNumara++;
        } else if (dequeue(&gise, &numara)) {
            printf("Gişeye çağrıldı: %d\n", numara);
        } else {
            printf("Sırada kimse yok\n");
        }
    }
    return 0;
}
```

```
ÇIKTI
Bilet alındı: 1 (sırada 1 kişi)
Bilet alındı: 2 (sırada 2 kişi)
Bilet alındı: 3 (sırada 3 kişi)
Gişeye çağrıldı: 1
Bilet alındı: 4 (sırada 3 kişi)
Gişeye çağrıldı: 2
Gişeye çağrıldı: 3
Gişeye çağrıldı: 4
Sırada kimse yok
```

Çıkış sırası, gelişle birebir aynıdır: kuyruk adildir. `dequeue` içindeki `if (k->bas == NULL) k->son = NULL;` satırı, kolay unutulan bir inceliktir. Son düğüm de çıkınca `son` işaretçisi serbest bırakılmış belleği göstermeye devam ederdi; sonraki `enqueue` o adrese yazardı. Program sonunda kuyruk boş olduğu için ayrıca bir `free` döngüsü gerekmedi; boş olmayan bir kuyrukta her düğümün `dequeue` ile ya da bir döngüyle bırakılması gerekir.

Üç Dilde · Kuyruk: Üç dilde *C · C++ · C#*

```c
#include <queue>
std::queue<int> k;
k.push(10);
k.push(20);
int on = k.front();   // 10, çıkarmaz
k.pop();              // 10'u çıkarır, değer döndürmez
```

```c
var k = new Queue<int>();
k.Enqueue(10);
k.Enqueue(20);
int on = k.Peek();    // 10, çıkarmaz
int c = k.Dequeue();  // 10'u çıkarır ve döndürür
```

C++ kitaplığı kuyruğu, kaplayıcının (varsayılan olarak `std::deque`) üzerine kurulu bir "adaptör" olarak sunar. C#'taki `Queue<T>` ise içeride bizim dairesel dizimize benzer bir yapı kullanır ve gerektiğinde kapasiteyi kendisi büyütür.

19.7 Çift Yönlü Bağlı Liste

Tek yönlü listede yalnızca ileri gidebiliriz; bir düğümü silmek için bile önceki düğümü aramamız gerekir. **Çift yönlü bağlı listede** (doubly linked list) her düğüm hem bir sonrakini hem bir öncekini gösterir:

```
bas                                          son
 |                                            |
 v                                            v
NULL <-- [10] <--> [20] <--> [30] --> NULL
```

Bunun bedeli, düğüm başına bir işaretçi daha ve her ekleme/silmede güncellenecek iki kat bağlantıdır. Karşılığında şunları kazanırız:

- Her iki yönde dolaşabiliriz.
- Adresini bildiğimiz bir düğümü, önceki düğümü aramadan silebiliriz.
- Son işaretçisi sayesinde sona ekleme de tek adımdır.

ciftYonlu.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct CiftDugum {
    int veri;
    struct CiftDugum *onceki;
    struct CiftDugum *sonraki;
} CiftDugum;

typedef struct {
    CiftDugum *bas;
    CiftDugum *son;
} Liste;

CiftDugum *yeniDugum(int veri) {
    CiftDugum *d = malloc(sizeof(CiftDugum));
    if (d != NULL) {
        d->veri = veri;
        d->onceki = NULL;
        d->sonraki = NULL;
    }
    return d;
}

void basaEkle(Liste *l, int veri) {
    CiftDugum *yeni = yeniDugum(veri);
    if (yeni == NULL) {
        return;
    }
    yeni->sonraki = l->bas;
    if (l->bas != NULL) {
        l->bas->onceki = yeni;
    } else {
        l->son = yeni;          // liste boştu
    }
    l->bas = yeni;
}

void sonaEkle(Liste *l, int veri) {
    CiftDugum *yeni = yeniDugum(veri);
    if (yeni == NULL) {
        return;
    }
    yeni->onceki = l->son;
    if (l->son != NULL) {
        l->son->sonraki = yeni;
    } else {
        l->bas = yeni;          // liste boştu
    }
    l->son = yeni;
}

CiftDugum *ara(Liste *l, int aranan) {
    for (CiftDugum *p = l->bas; p != NULL; p = p->sonraki) {
        if (p->veri == aranan) {
            return p;
        }
    }
    return NULL;
}

void dugumSil(Liste *l, CiftDugum *d) {
```

```
    if (d->onceki != NULL) {
        d->onceki->sonraki = d->sonraki;
    } else {
        l->bas = d->sonraki;    // baştaki düğüm siliniyor
    }
    if (d->sonraki != NULL) {
        d->sonraki->onceki = d->onceki;
    } else {
        l->son = d->onceki;     // sondaki düğüm siliniyor
    }
    free(d);
}

void ileriYazdir(const Liste *l) {
    for (const CiftDugum *p = l->bas; p != NULL; p = p->sonraki) {
        printf("%d ", p->veri);
    }
    printf("\n");
}

void geriYazdir(const Liste *l) {
    for (const CiftDugum *p = l->son; p != NULL; p = p->onceki) {
        printf("%d ", p->veri);
    }
    printf("\n");
}

int main(void) {
    Liste l = {NULL, NULL};

    sonaEkle(&l, 10);
    sonaEkle(&l, 20);
    sonaEkle(&l, 30);
    sonaEkle(&l, 40);
    basaEkle(&l, 5);
    printf("İleri: ");
    ileriYazdir(&l);
    printf("Geri : ");
    geriYazdir(&l);

    dugumSil(&l, ara(&l, 20));  // ortadan
    dugumSil(&l, l.bas);        // baştan
    dugumSil(&l, l.son);        // sondan
    printf("Silmeden sonra: ");
    ileriYazdir(&l);
    printf("Geri          : ");
    geriYazdir(&l);

    while (l.bas != NULL) {
        dugumSil(&l, l.bas);
    }
    return 0;
}
```

```
ÇIKTI
İleri: 5 10 20 30 40
Geri : 40 30 20 10 5
Silmeden sonra: 10 30
Geri          : 30 10
```

`dugumSil` içindeki dört dal, baş ve son düğümlerin özel durumlarını karşılar. Bu özel durumlar çift yönlü listenin en hata yapılan yeridir. `ara(&l, 20)` bir `NULL` döndürseydi `dugumSil` onu kullanıp çökerdi; gerçek kodda önce sonucu denetleriz. Bu programda 20 listede olduğu için güvenlidir.

> **Not · Özel durumları azaltan hile: nöbetçi düğüm**
>
> Baş ve son için ayrı dallar yazmak yerine, listenin iki ucuna veri taşımayan **nöbetçi** (sentinel) düğümler koyan çözümler de vardır. Böylece her gerçek düğümün önünde ve arkasında mutlaka bir düğüm bulunur ve `NULL` denetimleri azalır. Burada konuyu basit tutmak için uygulamadık; C++'taki `std::list` gibi kütüphaneler bunu sık kullanır.

19.8 Karmaşıklık: Büyük O Gösterimine Giriş

Dizi ile bağlı liste arasında "hızlı" ve "yavaş" dedik. Bu hükmü sayıya dökmenin yolu, bir işlemin **eleman sayısı büyüdükçe** kaç adım gerektirdiğine bakmaktır. Milisaniye cinsinden süre makineye bağlıdır; adım sayısının eleman sayısıyla nasıl büyüdüğü ise algoritmanın kendi özelliğidir.

> **Tanım: Büyük O gösterimi**
>
> Bir algoritmanın adım sayısının, `n` girdi büyüklüğü arttıkça hangi hızla büyüdüğünü, sabit çarpanları ve küçük terimleri atarak gösterir. `O(n)` "adım sayısı yaklaşık `n` ile orantılı büyür" demektir.

Sık karşılaşılan sınıflar, küçükten büyüğe şöyledir:

Gösterim	Ad	Örnek
`O(1)`	Sabit	`dizi[i]`, yığına `push`, bağlı listenin başına ekleme
`O(log n)`	Logaritmik	Sıralı dizide ikili arama
`O(n)`	Doğrusal	Listede arama, listeyi yazdırma, diziyi tarama
`O(n log n)`	Doğrusal-logaritmik	İyi sıralama algoritmaları
`O(n²)`	Karesel	İç içe iki döngü, kabarcık sıralama

Atılan sabit çarpanlar yüzünden `O(n)` ile `O(2n)` aynı sınıftadır; `n` büyüdükçe belirleyici olan büyüme biçimidir, sabit değil. Aşağıdaki program bu büyümeyi gözle görülür kılar. Gerçekten sayaçlarla adım sayar:

▸ buyukO.c

```c
#include <stdio.h>

int main(void) {
    int boyutlar[] = {8, 64, 1024};

    printf("%6s %8s %10s %12s\n", "n", "log n", "n", "n*n");
    for (int k = 0; k < 3; k++) {
        int n = boyutlar[k];

        int logAdim = 0;
        for (int m = n; m > 1; m /= 2) {    // her adımda yarıya inen
            logAdim++;
        }

        long dogrusalAdim = 0;
        for (int i = 0; i < n; i++) {
```

```
            dogrusalAdim++;
        }

        long keseselAdim = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                keseselAdim++;
            }
        }

        printf("%6d %8d %10ld %12ld\n", n, logAdim, dogrusalAdim, keseselAdim);
    }
    return 0;
}
```

```
ÇIKTI
     n    log n          n          n*n
     8        3          8           64
    64        6         64         4096
  1024       10       1024      1048576
```

`n` 128 kat arttığında (8'den 1024'e) logaritmik işlem yalnızca 3'ten 10'a çıkarken doğrusal işlem 128 kat, karesel işlem 16.384 kat büyüdü. Yüz bin elemanda `O(n²)` bir algoritma on milyar adım demektir; `O(log n)` olan ise 17 adımla biter.

Bu tabloyu bildiğimizde dizi, bağlı liste, yığın ve kuyruğu aynı ölçüyle karşılaştırabiliriz:

İşlem	Dizi	Tek yönlü bağlı liste
`i`. elemana erişim	`O(1)`	`O(n)`
Başa ekleme	`O(n)` (kaydırma)	`O(1)`
Sona ekleme	`O(1)`, yer varsa	`O(n)`; son işaretçisi varsa `O(1)`
Değere göre arama	`O(n)`; sıralıysa `O(log n)`	`O(n)`
Konumu belli düğümden sonra ekleme	`O(n)`	`O(1)`
Konumu belli düğümü silme	`O(n)`	`O(n)`; çift yönlüde `O(1)`
Kapasite	Önceden belli, büyütmek kopyalama	Sınırsız (bellek yettiği sürece)

Bu tabloda bir uyarı gerekir: bağlı listede "ekleme `O(1)`" demek, ekleme noktasının zaten elimizde olduğunu varsayar. O noktaya ulaşmak için listeyi gezmek `O(n)` sürer. Ayrıca düğümler bellekte dağınık olduğu için, gerçek bilgisayarlarda dizilerin ardışık düzeni önbelleğe (cache) çok daha uygundur. Bu yüzden pratikte dizi, kâğıt üzerindeki karşılaştırmadan sıklıkla daha hızlı çıkar. Bu, "bağlı liste kötüdür" demek değildir; hangi işlemin sık yapıldığına bakıp seçim yapmak gerektiğini gösterir.

Yığın ve kuyruk için işlemlerin tamamı (`push`, `pop`, `peek`, `enqueue`, `dequeue`) bu bölümdeki her iki gerçekleştirmede de `O(1)`'dir. Diziyle gerçekleştirmede bunun bedeli kapasitenin sabit olması, bağlı listeyle gerçekleştirmede ise düğüm başına işaretçi ve her işlemde `malloc`/`free` çağrısıdır. Bölüm 13'te gördüğümüz kapasiteyi ikiye katlayan `realloc` yöntemi ise bu iki ucun arasında kalır: arada bir pahalı kopyalama yapar, ama ortalama olarak ekleme `O(1)` kalır.

◆ Kritik Nokta · Veri yapısını işlemlere göre seçin

Doğru veri yapısı, programın en sık yaptığı işleme göre belirlenir. Sıra numarasıyla erişim ağırlıklıysa dizi; baştan ekleyip çıkarma ağırlıklıysa bağlı liste, yığın ya da kuyruk uygundur. Seçimi ezbere değil, işlem sayısını ve büyüklüğünü düşünerek yapmak gerekir.

Üç Dilde · Bağlı liste: Üç dilde *C · C++ · C#*

C'de düğümleri, işaretçileri ve `free`'yi kendimiz yönettik. C++ ve C# aynı yapıyı kitaplık olarak sunar:

```cpp
#include <list>
std::list<int> l;        // çift yönlü bağlı liste
l.push_back(20);
l.push_front(10);
l.remove(20);            // değeri 20 olan elemanları siler
for (int x : l) { /* dolaş */ }
// düğümleri serbest bırakmak list'in yıkıcısının (destructor) işidir
```

```csharp
var l = new LinkedList<int>();   // çift yönlü bağlı liste
l.AddLast(20);
l.AddFirst(10);
l.Remove(20);
foreach (int x in l) { /* dolaş */ }
// düğümleri çöp toplayıcı (GC) temizler
```

C++'taki `std::list` ve C#'taki `LinkedList<T>` çift yönlüdür. Her iki dilde de gündelik işlerde çoğu zaman dizi tabanlı `std::vector` ve `List<T>` tercih edilir; bağlı liste, ortadan sık ekleme-silme gerektiğinde ve eleman adresleri sabit kalmak zorundayken öne çıkar. İşaretçi hatalarının ve bellek yönetiminin kitaplık tarafından üstlenilmesi, bu bölümde yaşadığımız en büyük zorlukları ortadan kaldırır.

Tarihçe · Bağlı listenin kökenleri

Bağlı liste, 1950'lerin ortasında Allen Newell, Cliff Shaw ve Herbert Simon'ın yapay zekâ programlama çalışmalarında (IPL dili) geliştirdiği bir fikirdir. Listeleri temel veri yapısı olarak benimseyen Lisp dili de 1958'de ortaya çıktı. C'de işaretçiler sayesinde bu yapı doğrudan kurulabildiği için, bağlı liste uzun yıllar C öğretiminin vazgeçilmez konularından biri olmuştur.

Bölüm Özeti

- Veri yapısı, verinin belirli işlemleri verimli yapabilecek biçimde düzenlenmesidir; dizi, bağlı liste, yığın ve kuyruk bunların temel örnekleridir.
- Bağlı listede her düğüm, veriyi ve bir sonraki düğümün adresini taşır; son düğümün işaretçisi `NULL`'dır. Liste, ilk düğümü gösteren baş işaretçisinden ibarettir.
- Düğüm yapısı, kendi türünden işaretçi içerdiği için `struct` etiketiyle tanımlanır: `struct Dugum *sonraki;`.
- Liste boyunca ilerlerken baş işaretçisini bozmayız; kopyasını (`p`) ilerletiriz. Dolaşma koşulu `p != NULL`'dır.
- Baş işaretçisini değiştiren işlemler (başa ekleme, baştan silme, ters çevirme) `Dugum **` alır.
- Başa ekleme `O(1)`'dir; sona ekleme son işaretçisi yoksa `O(n)`'dir. Eklerken önce yeni düğümü listeye bağlar, sonra baş ya da önceki bağlantıyı güncelleriz.

- Silerken önceki düğümü de taşırız; baştaki düğümün silinmesi ayrı bir durumdur. Silinen düğüm `free` ile bırakılır.
- Listeyi bırakırken `free(p)` çağrılmadan önce `p->sonraki` bir değişkene alınmalıdır.
- Yığın LIFO ilkesiyle çalışır: `push`, `pop`, `peek`. Dizi ile kurulduğunda taşma ve boşluk denetimi gerekir; bağlı listeyle kurulduğunda liste başı tepe olur.
- Parantez dengesi denetimi ve sayı tabanı çevirme, yığının tipik uygulamalarıdır.
- Kuyruk FIFO ilkesiyle çalışır: `enqueue` sondan ekler, `dequeue` önden çıkarır. Dairesel dizide indeksler `% KAPASITE` ile başa sarar; boş ile dolu durumunu ayırmak için sayaç tutulur.
- Bağlı listeyle kurulan kuyrukta hem baş hem son işaretçisi tutulur; kuyruk boşalınca `son` da `NULL` yapılmalıdır.
- Çift yönlü listede her düğüm öncesini de gösterir; her iki yönde dolaşmak ve bir düğümü doğrudan silmek mümkün olur, ama bağlantı güncellemeleri artar.
- Büyük O gösterimi, adım sayısının girdi büyüklüğüyle nasıl büyüdüğünü ifade eder: `O(1)`, `O(log n)`, `O(n)`, `O(n²)`. Veri yapısı, en sık yapılan işleme göre seçilir.
- C++ (`std::list`, `std::stack`, `std::queue`) ve C# (`LinkedList<T>`, `Stack<T>`, `Queue<T>`) bu yapıları hazır sunar.

Alıştırmalar

1. Tek yönlü bağlı listenin başına eleman eklemenin zaman karmaşıklığı nedir?

   A) `O(1)`
   B) `O(log n)`
   C) `O(n)`
   D) `O(n²)`
   E) Listenin uzunluğuna ve verinin değerine göre değişir

2. Aşağıdakilerden hangisi tipik olarak yığın (stack) mantığıyla çalışır?

   A) Yazıcıya gönderilen belgelerin sırası
   B) Bir editörde "geri al" (undo) komutu
   C) Gişede bekleyen müşteriler
   D) Bir ağ kartına gelen paketlerin ara belleği
   E) Telefon rehberindeki isimlerin alfabetik sırası

3. Bu programın çıktısı nedir?

```c
#include <stdio.h>

int main(void) {
    int y[10];
    int tepe = -1;
    for (int i = 1; i <= 4; i++) {
        y[++tepe] = i * i;
    }
    tepe--;
    y[++tepe] = 100;
    while (tepe >= 0) {
        printf("%d ", y[tepe--]);
    }
```

```
    printf("\n");
    return 0;
}
```

4. Bu programın çıktısı nedir?

```c
#include <stdio.h>

int main(void) {
    int son = 0;
    for (int i = 0; i < 7; i++) {
        son = (son + 1) % 3;
    }
    printf("%d\n", son);
    return 0;
}
```

5. Aşağıdaki fonksiyon listeyi boşaltmak için yazılmıştır. Hatayı bulun ve düzeltin.

```c
void bosalt(Dugum *bas) {
    Dugum *p = bas;
    while (p != NULL) {
        free(p);
        p = p->sonraki;
    }
}
```

6. Aşağıdaki fonksiyon listenin başına eleman eklemek için yazılmıştır, ancak çağrıdan sonra çağıran taraftaki liste değişmemektedir. Nedenini açıklayın ve düzeltin.

```c
void basaEkle(Dugum *bas, int veri) {
    Dugum *yeni = malloc(sizeof(Dugum));
    yeni->veri = veri;
    yeni->sonraki = bas;
    bas = yeni;
}
```

7. Aşağıdaki dizgilerden hangisinde parantezler dengelidir?

   A) `([)]`
   B) `(()`
   C) `{[}]`
   D) `([]{})`
   E) `())(`

8. Bu programın çıktısı nedir?

```c
#include <stdio.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

int main(void) {
    Dugum c = {3, NULL};
    Dugum b = {2, &c};
```

```
    Dugum a = {1, &b};
    Dugum *p = &a;

    p = p->sonraki;
    printf("%d %d\n", p->veri, p->sonraki->veri);
    b.sonraki = NULL;
    printf("%d\n", a.sonraki->sonraki == NULL);
    return 0;
}
```

9. Dairesel dizi ile kurulan kuyrukta, `bas` ve `son` indekslerinin yanında neden bir `adet` sayacı tutulur?
10. Sıralı bir dizide 1000 elemanlı bir aramayı ikili aramayla yaparsak en çok yaklaşık kaç karşılaştırma gerekir?
    A) 1
    B) 10
    C) 100
    D) 500
    E) 1000
11. Bağlı listeyle kurulan bir kuyrukta `dequeue` son elemanı çıkardığında `son` işaretçisini `NULL` yapmazsak ne olur?
12. Aşağıdakilerden hangisi tek yönlü listeye göre çift yönlü listenin avantajıdır?
    A) Düğüm başına daha az bellek kullanır
    B) Adresi bilinen bir düğüm, öncekini aramadan silinebilir
    C) Elemanlara indeksle `O(1)` erişim sağlar
    D) Hiç `free` gerektirmez
    E) Baş işaretçisine ihtiyaç duymaz
13. Tek yönlü bağlı listedeki en büyük değeri döndüren `int enBuyuk(const Dugum *bas, int *sonuc)` fonksiyonunu yazın. Liste boşsa 0, aksi hâlde 1 döndürsün ve en büyük değeri `*sonuc`'a yazsın. Sonra `3 -> 9 -> 4` listesiyle deneyin.
14. Bir listede belirli bir değere sahip **tüm** düğümleri silen `int silHepsi(Dugum **bas, int deger)` fonksiyonunu yazın. Silinen düğüm sayısını döndürsün.
15. Dizi ile gerçekleştirilmiş bir yığın kullanarak klavyeden okunan bir kelimeyi ters çeviren program yazın.
16. İki sıralı (küçükten büyüğe) tek yönlü bağlı listeyi, yeni düğüm yaratmadan, tek bir sıralı listede birleştiren `Dugum *birlestir(Dugum *a, Dugum *b)` fonksiyonunu yazın.

BÖLÜM 20

C Uygulama Projesi: Öğrenci Kayıt Sistemi

BU BÖLÜMDE

- Küçük bir uygulamanın gereksinimlerini yazıp bunlardan bir tasarım çıkarmayı
- Yapı, dinamik dizi, işaretçi, string ve dosya bilgilerini tek bir programda birleştirmeyi
- Programı, her biri tek iş yapan fonksiyonlara bölmeyi
- Güvenli satır tabanlı girdi okuma kalıbını kullanmayı
- Kayıtları ikili dosyaya yazıp geri yüklemeyi
- Çalışan bir programı kendi gereksinimlerinize göre genişletmeyi

Kitabın birinci kısmında tek tek öğrendiğimiz araçları bu bölümde birlikte kullanacağız: değişkenler ve operatörler (Bölüm 5), girdi ve çıktı (Bölüm 6), koşullar ve döngüler (Bölüm 7 ve 8), fonksiyonlar (Bölüm 9), diziler ve stringler (Bölüm 10 ve 11), işaretçiler ve dinamik bellek (Bölüm 12 ve 13), yapılar (Bölüm 14) ve dosyalar (Bölüm 17). Konu bir **öğrenci kayıt sistemi**dir: menüden seçim yapılan, kayıt ekleyen, listeleyen, arayan, silen ve kayıtları dosyada saklayan bir konsol uygulaması.

Bu bölümde yeni bir C özelliği öğrenmeyeceğiz. Amaç, bilinen parçaların bir araya gelirken nasıl düzenlendiğini görmektir. Gerçek programlar bir konuyu anlatmak için değil, bir işi yapmak için yazılır; o işi küçük adımlara bölmek ve her adımı doğru yere koymak, tek tek komutları bilmek kadar önemlidir. Bölümün sonunda tam program tek parça hâlinde verilecek; önce onu nasıl kurduğumuzu adım adım izleyelim.

20.1 Gereksinimler

Programa başlamadan önce ne yapmasını istediğimizi yazılı hâle getirmek iyi bir alışkanlıktır. **Gereksinim** (requirement), programın yapabilmesi gereken bir işin kısa tanımıdır. Bizim programımız şunları yapacak:

1. Her öğrenci için numara, ad, soyad ve not ortalaması tutulacak.
2. Kayıt sayısı önceden bilinmeyecek; program gerektiği kadar büyüyecek.
3. Kullanıcı bir menüden işlem seçecek: ekleme, listeleme, numaraya göre arama, numaraya göre silme, istatistik, dosyaya kaydetme, dosyadan yükleme, çıkış.
4. Aynı numaralı iki öğrenci kaydedilemeyecek; ortalama 0 ile 100 arasında olmalı.
5. Hatalı girdi (harf yazılan sayı alanı, boş satır) programı çökertmeyecek.
6. Kayıtlar bir dosyaya yazılacak ve daha sonra aynı dosyadan geri yüklenebilecek.

Bunlardan ikinci madde bellek yönetimini (Bölüm 13), beşinci madde girdi okuma biçimimizi, altıncı madde dosya biçimini belirler. Tasarımı bu üç karara göre yapacağız.

20.2 Tasarım

Veri: Ogrenci yapısı

Bir öğrencinin bilgileri farklı tiptedir: numara `int`, ad ve soyad string, ortalama `double`. Bunları tek bir birimde toplamanın yolu yapıdır (Bölüm 14):

```
#define AD_UZUNLUK 30

typedef struct {
    int no;
    char ad[AD_UZUNLUK];
    char soyad[AD_UZUNLUK];
    double ortalama;
} Ogrenci;
```

Ad ve soyad için sabit uzunlukta `char` dizisi seçtik. Böylece bir `Ogrenci` değeri kendi içinde eksiksizdir: içinde başka bir bellek bölgesine işaret eden adres yoktur. Bu tercih, kaydı dosyaya olduğu gibi yazmayı (`fwrite`) mümkün kılar. Ad için `char *` kullansaydık dosyaya yalnızca adres yazılırdı ve bu adres bir sonraki çalıştırmada anlamsız olurdu.

Bellek: büyüyen dizi

Öğrenci sayısını bilmediğimiz için kayıtları `malloc` ile ayrılan bir dizide tutacağız ve dizi dolduğunda `realloc` ile büyüteceğiz (Bölüm 13). Dizi işaretçisi, kaç elemanın dolu olduğu ve dizinin kapasitesi birbirine bağlı üç bilgidir; bunları da bir yapıda toplayalım:

```
typedef struct {
    Ogrenci *dizi;     // öbekteki dizi
    int adet;          // dolu eleman sayısı
    int kapasite;      // ayrılmış eleman sayısı
} Liste;
```

Fonksiyonlara tek tek `dizi`, `adet` ve `kapasite` geçirmek yerine `Liste` işaretçisi geçireceğiz. Hem parametre listeleri kısalır hem de üç bilginin birbiriyle tutarsız hâle gelme ihtimali ortadan kalkar. `main` içinde yalnızca bir `Liste` değişkeni bulunur; başka hiçbir global değişken kullanılmaz.

Program yapısı

Programı, her biri tek bir iş yapan fonksiyonlara böleceğiz. Fonksiyonlar üç katmanda toplanır:

```
        main  (menü döngüsü)
          |
   +------+-----------+------------------+
   |                  |                  |
komutEkle        komutAra          kaydet / yukle
komutSil         listele             (dosya)
                 istatistik
   |                  |                  |
   +------------------+------------------+
```

```
          |
   listeBaslat, ogrenciEkle, noBul, ogrenciSil, satirOku
          (liste ve girdi yardımcıları)
```

En altta, ekrandan ve menüden habersiz çalışan liste işlemleri vardır: ekle, bul, sil. Ortada, kullanıcıyla konuşan `komut...` fonksiyonları yer alır; bunlar girdiyi okur, denetler ve alttaki işlevleri çağırır. En üstte yalnızca menüyü gösterip seçime göre doğru fonksiyonu çağıran `main` bulunur. Bu ayrım programı değiştirmeyi kolaylaştırır: örneğin menüyü bir pencere arayüzüyle değiştirirsek alt katmanlara dokunmamız gerekmez.

> **Üç Dilde · Aynı tasarım üç dilde** *C · C++ · C#*
>
> C'de büyüyen diziyi ve `adet` / `kapasite` bilgisini kendimiz yönettik. C++'ta bu işi `std::vector<Ogrenci>` üstlenir; `push_back` gerektiğinde kapasiteyi kendisi artırır ve bellek otomatik bırakılır. C#'ta karşılığı `List<Ogrenci>`'dir; bellek yönetimini çöp toplayıcı yapar. C++ ve C# kısımlarında, buradaki `Liste` yapısı ve ona bağlı fonksiyonların büyük bölümünün tek bir hazır tiple yer değiştirdiğini göreceğiz.

20.3 Girdi Okuma Yardımcısı

Menü tabanlı programlarda en sık yapılan hata `scanf` ile sayı okuyup ardından satır sonu karakterini arabellekte bırakmaktır (Bölüm 6). Bir sonraki okuma bu artığa takılır ve program beklenmedik davranır. Ayrıca `scanf("%d")` harf yazılırsa girdiyi tüketmeden başarısız olur ve döngüde sonsuza dek aynı hatayı üretebilir.

Bu sorunların ikisini birden çözen kalıp şudur: **her girdiyi önce satır olarak `fgets` ile oku, sonra `sscanf` ile ayrıştır.** Satırı okumayı tek bir yardımcıda toplayalım:

```c
int satirOku(const char *istek, char *tampon, int boyut) {
    printf("%s", istek);
    if (fgets(tampon, boyut, stdin) == NULL) {
        return 0;                       // girdi bitti
    }
    tampon[strcspn(tampon, "\n")] = '\0';
    return 1;
}
```

Fonksiyon önce istemi (`"Numara: "` gibi) yazar, sonra bir satır okur. `fgets` satırın sonundaki `\n` karakterini de tamponda bırakır; `strcspn` bu karakterin yerini bulur ve o noktaya `'\0'` yazarak onu siler (Bölüm 11). Dönüş değeri, okumanın başarılı olup olmadığını bildirir. `fgets`'in `NULL` döndürmesi girdinin bittiği anlamına gelir (dosya sonu ya da kapanmış girdi); çağıran bu durumu anlayıp işlemi bırakabilir.

Sayı gereken yerlerde satırı `sscanf` ile ayrıştırır ve dönüş değerine bakarız:

```c
char t[100];
int no;
if (!satirOku("Numara: ", t, sizeof t) || sscanf(t, "%d", &no) != 1) {
    printf("Geçersiz numara.\n");
    return;
}
```

`sscanf`, başarıyla doldurduğu değişken sayısını döndürür. Kullanıcı "abc" yazdıysa dönüş 0 olur ve `!= 1` koşulu hatayı yakalar. Satır baştan sona okunduğu için artık arabellekte kalan bir şey yoktur; hatalı girdi yalnızca o işlemi iptal eder.

> ◆ **Kritik Nokta · Kullanıcı girdisine asla güvenmeyin**
>
> Kullanıcıdan gelen her veri yanlış, eksik ya da beklenmedik uzunlukta olabilir. Okuma işlevinin dönüş değeri denetlenmeli, sayı beklenen yerde sayı gelip gelmediği kontrol edilmeli, string okurken tamponun boyutu aşılmamalıdır. `sscanf(t, "%29s", ad)` içindeki `29`, 30 baytlık `ad` dizisine en fazla 29 karakter ve sonuna `'\0'` yazılmasını sağlar.

20.4 Liste İşlemleri

Başlatma ve ekleme

Liste önce küçük bir kapasiteyle (burada 2) başlar. Her ekleme öncesinde dizinin dolup dolmadığına bakılır; doluysa kapasite ikiye katlanır:

```
int listeBaslat(Liste *l) {
    l->adet = 0;
    l->kapasite = 2;
    l->dizi = malloc(l->kapasite * sizeof(Ogrenci));
    return l->dizi != NULL;
}

int ogrenciEkle(Liste *l, Ogrenci o) {
    if (l->adet == l->kapasite) {
        int yeni = l->kapasite * 2;
        Ogrenci *p = realloc(l->dizi, yeni * sizeof(Ogrenci));
        if (p == NULL) {
            return 0;                   // eski dizi hâlâ geçerli
        }
        l->dizi = p;
        l->kapasite = yeni;
    }
    l->dizi[l->adet++] = o;
    return 1;
}
```

İki ayrıntıya dikkat edelim. Birincisi, `realloc` sonucunu doğrudan `l->dizi`'ye atamadık; önce geçici `p` değişkenine aldık. `realloc` başarısız olursa `NULL` döndürür ve eski bloğu serbest bırakmaz; sonucu doğrudan `l->dizi`'ye yazsaydık eski adresi kaybeder ve bellek sızdırırdık (Bölüm 13). İkincisi, fonksiyon başarıyı `1`, başarısızlığı `0` olarak bildirir; ekrana bir şey yazmaz. Kullanıcıya mesaj vermek, katman ayrımı gereği `komutEkle`'nin işidir.

Kapasiteyi her seferinde bir artırmak yerine ikiye katlamak, ardışık eklemelerde `realloc` çağrısı sayısını azaltır: 1000 kayıt için yaklaşık 10 büyütme yeter, 1000 değil.

`ogrenciEkle` yapıyı **değerle** alır (`Ogrenci o`); yani çağıranın değişkeninin bir kopyasıyla çalışır ve kopyayı dizinin sonuna yazar. Yapı küçük olduğu için bu kabul edilebilir; büyük yapılarda işaretçiyle geçirmek daha verimlidir (Bölüm 14).

Arama ve silme

Numaraya göre arama, dizinin başından sonuna doğrusal bir taramadır. Fonksiyon bulduğu kaydın **sırasını** döndürür; bulamazsa geçerli hiçbir sıra olamayacak olan `-1`'i döner:

```c
int noBul(const Liste *l, int no) {
    for (int i = 0; i < l->adet; i++) {
        if (l->dizi[i].no == no) {
            return i;
        }
    }
    return -1;
}
```

Sıra döndürmek, kaydın kendisini ya da adresini döndürmekten daha esnektir: aynı sıra hem gösterme hem silme için kullanılabilir. Parametrenin `const Liste *` olması, fonksiyonun listeyi değiştirmeyeceğinin sözüdür (Bölüm 12).

Silmek için araya açılan boşluğu kapatmak gerekir. Silinen elemandan sonraki bütün elemanlar bir sıra öne kaydırılır:

```c
void ogrenciSil(Liste *l, int sira) {
    memmove(&l->dizi[sira], &l->dizi[sira + 1],
            (l->adet - sira - 1) * sizeof(Ogrenci));
    l->adet--;
}
```

```
Silmeden önce (sira = 1):   [ A ][ B ][ C ][ D ]   adet = 4
Kaydırma:                         B <- C <- D
Silmeden sonra:             [ A ][ C ][ D ][ D ]   adet = 3
                                              ^ artık dizinin dışında
```

Kaynak ve hedef bölgeler üst üste bindiği için `memcpy` yerine `memmove` kullanılır; `memmove` çakışmalı kopyalamada doğru çalışmayı garanti eder. Son elemandan sonra kalan eski kopya `adet` azaltıldığı için artık dizinin parçası sayılmaz; üzerine sonraki ekleme yazacaktır.

> **▲ Sık Yapılan Hata · Sırayı değil numarayı silmeyin**
>
> `ogrenciSil` bir **dizi sırası** bekler, öğrenci numarası değil. Kullanıcıdan gelen numarayı önce `noBul` ile sıraya çevirmek ve `-1` dönüp dönmediğine bakmak gerekir. `-1`'i doğrudan `ogrenciSil`'e vermek dizi sınırının dışına yazmak demektir.

20.5 Ekrana Yazan İşlevler

Bir öğrenci satırını hizalı yazmak için `printf`'in alan genişliği belirteçlerini kullanırız (Bölüm 6): `%-10s` metni 10 karakterlik alanda sola yaslar, `%6.2f` sayıyı 6 karakterlik alanda virgülden sonra iki basamakla yazar. Başlık satırı ve veri satırları aynı genişlikleri kullandığı için sütunlar alt alta gelir.

```c
void satirYaz(const Ogrenci *o) {
    printf("%-6d %-10s %-10s %6.2f\n", o->no, o->ad, o->soyad, o->ortalama);
}

void listele(const Liste *l) {
    if (l->adet == 0) {
        printf("Kayıt yok.\n");
        return;
    }
    printf("%-6s %-10s %-10s %6s\n", "No", "Ad", "Soyad", "Ort.");
```

```
    for (int i = 0; i < l->adet; i++) {
        satirYaz(&l->dizi[i]);
    }
}
```

`satirYaz` yapıyı işaretçiyle alır ve üyelere `->` ile erişir. `listele` ile `komutAra` aynı satır biçimini kullandığı için biçimi tek yerde, `satirYaz`'da tutmak, ileride sütun eklemek istediğimizde yalnızca bir fonksiyonu değiştirmemizi sağlar.

`istatistik` fonksiyonu genel ortalamayı, en yüksek ve en düşük ortalamalı öğrenciyi bulur. En yüksek ve en düşük değerin **kendisini** değil, onun dizideki **sırasını** (`enIyi`, `enKotu`) saklıyoruz; böylece ad ve soyad da aynı sıradan okunabiliyor:

```
double toplam = 0;
int enIyi = 0, enKotu = 0;
for (int i = 0; i < l->adet; i++) {
    toplam += l->dizi[i].ortalama;
    if (l->dizi[i].ortalama > l->dizi[enIyi].ortalama) enIyi = i;
    if (l->dizi[i].ortalama < l->dizi[enKotu].ortalama) enKotu = i;
}
```

Bu parça, fonksiyonun başında `adet == 0` kontrolü yapıldıktan sonra çalışır; boş listede `toplam / l->adet` sıfıra bölme olurdu. Tam hâli bölüm sonundaki programdadır.

20.6 Menü Komutları

`komutEkle`, `komutAra` ve `komutSil` aynı kalıba uyar: girdiyi oku, denetle, alttaki liste işlevini çağır, sonucu bildir. En uzunu `komutEkle`'dir; çünkü dört ayrı denetim yapar:

```
void komutEkle(Liste *l) {
    char t[100];
    Ogrenci o;

    if (!satirOku("Numara: ", t, sizeof t) || sscanf(t, "%d", &o.no) != 1) {
        printf("Geçersiz numara.\n");
        return;
    }
    if (noBul(l, o.no) != -1) {
        printf("Bu numara zaten kayıtlı.\n");
        return;
    }
    if (!satirOku("Ad Soyad: ", t, sizeof t) ||
        sscanf(t, "%29s %29s", o.ad, o.soyad) != 2) {
        printf("Ad ve soyad gerekli.\n");
        return;
    }
    if (!satirOku("Ortalama: ", t, sizeof t) ||
        sscanf(t, "%lf", &o.ortalama) != 1 ||
        o.ortalama < 0 || o.ortalama > 100) {
        printf("Ortalama 0-100 arasında olmalı.\n");
        return;
    }
    if (ogrenciEkle(l, o)) {
        printf("Kayıt eklendi.\n");
    } else {
```

```
        printf("Bellek yetersiz.\n");
    }
}
```

Fonksiyon, bir denetim başarısız olduğu anda mesajı yazıp `return` ile çıkar. Buna **erken dönüş** (early return) denir; iç içe `if` basamaklarının yerine düz, yukarıdan aşağı okunan bir akış sağlar. Ad ve soyad tek satırda, boşlukla ayrılmış olarak alınır: `sscanf` iki `%29s` ile iki sözcüğü ayırır. Bu nedenle bileşik adlar ("Ayşe Nur") desteklenmez; bu kısıtı geliştirme önerilerinde ele alacağız. Geçerli girdide `Ogrenci o` tamamen doldurulmuş olur ve `ogrenciEkle`'ye geçirilir.

`komutAra` ve `komutSil`, aynı biçimde numara okur, `noBul` ile sırayı bulur ve `-1` dönerse "Öğrenci bulunamadı." yazar. Birincisi bulunan satırı `satirYaz` ile gösterir, ikincisi `ogrenciSil`'i çağırır. Tam kodları programın içindedir.

20.7 Dosyaya Kaydetme ve Yükleme

Kayıtları dosyada saklamak için ikili (binary) biçimi seçtik (Bölüm 17). Dosyanın yapısı çok basittir: önce kayıt sayısını tutan bir `int`, ardından o sayıda `Ogrenci` yapısı:

```
 kayitlar.bin
+-----------+------------+------------+-----
| adet (int)| Ogrenci #0 | Ogrenci #1 | ...
+-----------+------------+------------+-----
```

Dizinin elemanları bellekte art arda durduğu için, bütün dizi tek bir `fwrite` çağrısıyla yazılır:

```
int kaydet(const Liste *l) {
    FILE *f = fopen(DOSYA_ADI, "wb");
    if (f == NULL) {
        return 0;
    }
    int ok = fwrite(&l->adet, sizeof(int), 1, f) == 1 &&
             fwrite(l->dizi, sizeof(Ogrenci), l->adet, f) == (size_t)l->adet;
    fclose(f);
    return ok;
}
```

`fwrite` yazdığı eleman sayısını döndürür; her çağrının dönüşünü istenen sayıyla karşılaştırarak yazmanın başarısını denetleriz. İki yazma `&&` ile bağlandığı için ilki başarısızsa ikincisi denenmez. `fclose` her durumda çağrılır.

Yükleme daha dikkatli olmalıdır, çünkü dosyanın içeriği bize ait olmayabilir: bozulmuş, kısa kesilmiş ya da başka bir programdan gelmiş olabilir. Bu yüzden kayıtları önce **geçici bir diziye** okuruz ve her şey yolunda giderse mevcut listenin yerine koyarız:

```
int yukle(Liste *l) {
    FILE *f = fopen(DOSYA_ADI, "rb");
    if (f == NULL) {
        return 0;
    }
    int adet;
    if (fread(&adet, sizeof(int), 1, f) != 1 || adet < 0) {
        fclose(f);
        return 0;
```

```
    }
    int kapasite = adet < 2 ? 2 : adet;
    Ogrenci *yeni = malloc(kapasite * sizeof(Ogrenci));
    if (yeni == NULL ||
        fread(yeni, sizeof(Ogrenci), adet, f) != (size_t)adet) {
        free(yeni);
        fclose(f);
        return 0;
    }
    fclose(f);
    free(l->dizi);                      // eski diziyi bırak, yenisini al
    l->dizi = yeni;
    l->adet = adet;
    l->kapasite = kapasite;
    return 1;
}
```

Bu tasarımın değeri, hata durumunda mevcut listenin bozulmamasıdır. Dosya okunamazsa `l` değişmeden kalır; kullanıcı hâlâ elindeki kayıtlarla çalışabilir. Başarılı olunca eski dizi `free` ile bırakılır ve yeni dizi ona bağlanır; bellek sızıntısı olmaz. `free(NULL)` güvenli olduğu için `yeni == NULL` iken de aynı çıkış yolunu kullanabildik.

> **▲ Sık Yapılan Hata · İkili dosya taşınabilir değildir**
>
> Yapıyı olduğu gibi yazmak kolaydır ama dosya, yazıldığı makineye ve derleyiciye bağlıdır. Yapı üyeleri arasındaki dolgu (padding), `int`'in boyutu ve bayt sıralaması farklı bir sistemde değişebilir (Bölüm 14). Dosyayı aynı programın aynı platformda yazıp okuması bu proje için yeterlidir; farklı sistemler arasında veri taşınacaksa metin biçimi (örneğin satır başına bir kayıt, alanlar `;` ile ayrılmış) daha güvenlidir.

20.8 Ana Program

`main` yalnızca üç iş yapar: listeyi başlatmak, menü döngüsünü yürütmek ve çıkarken belleği geri vermek. Menü, `switch` ile seçilen komutları ilgili fonksiyonlara dağıtır (Bölüm 7):

```
int main(void) {
    Liste liste;
    char t[100];

    if (!listeBaslat(&liste)) {
        printf("Bellek ayrılamadı.\n");
        return 1;
    }

    menuYaz();
    for (;;) {
        int secim;
        if (!satirOku("> ", t, sizeof t)) {
            break;                      // girdi sona erdi
        }
        if (sscanf(t, "%d", &secim) != 1) {
            secim = -1;
        }
        if (secim == 0) {
            break;
        }
        switch (secim) {
            case 1: komutEkle(&liste); break;
            // ... diğer komutlar
            default: printf("Geçersiz seçim. Menü için 9 yazın.\n");
```

```
        }
    }

    free(liste.dizi);
    printf("Güle güle.\n");
    return 0;
}
```

Sonsuz döngü (`for (;;)`) içinde menüden bir seçim okunur. `0` ya da girdinin bitmesi döngüyü `break` ile sonlandırır. Seçim sayıya çevrilemezse `-1` yapılır; böylece `switch`'in `default` kolu geçersiz girdiyi karşılar. Döngü bittiğinde `free(liste.dizi)` ile öbekteki dizi bırakılır; `listeBaslat` ile başlayan `malloc`'un karşılığı budur.

20.9 Tam Program

Parçaları birleştirince programın tamamı aşağıdaki gibi olur. Dosyanın tek parça olması, `gcc -std=c17 -Wall ogrenciKayit.c -o ogrenciKayit` komutuyla derlenmesi için yeterlidir. Büyük projelerde bu fonksiyonlar başlık ve kaynak dosyalarına bölünür (Bölüm 15); biz okumayı kolaylaştırmak için tek dosyada bıraktık.

▸ ogrenciKayit.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define AD_UZUNLUK 30
#define DOSYA_ADI "kayitlar.bin"

typedef struct {
    int no;
    char ad[AD_UZUNLUK];
    char soyad[AD_UZUNLUK];
    double ortalama;
} Ogrenci;

typedef struct {
    Ogrenci *dizi;
    int adet;
    int kapasite;
} Liste;

// ---------- girdi yardımcıları ----------

int satirOku(const char *istek, char *tampon, int boyut) {
    printf("%s", istek);
    if (fgets(tampon, boyut, stdin) == NULL) {
        return 0;                       // girdi bitti
    }
    tampon[strcspn(tampon, "\n")] = '\0';
    return 1;
}

// ---------- liste işlemleri ----------

int listeBaslat(Liste *l) {
    l->adet = 0;
    l->kapasite = 2;
    l->dizi = malloc(l->kapasite * sizeof(Ogrenci));
    return l->dizi != NULL;
```

```
}

int ogrenciEkle(Liste *l, Ogrenci o) {
    if (l->adet == l->kapasite) {
        int yeni = l->kapasite * 2;
        Ogrenci *p = realloc(l->dizi, yeni * sizeof(Ogrenci));
        if (p == NULL) {
            return 0;                   // eski dizi hâlâ geçerli
        }
        l->dizi = p;
        l->kapasite = yeni;
    }
    l->dizi[l->adet++] = o;
    return 1;
}

int noBul(const Liste *l, int no) {
    for (int i = 0; i < l->adet; i++) {
        if (l->dizi[i].no == no) {
            return i;
        }
    }
    return -1;
}

void ogrenciSil(Liste *l, int sira) {
    memmove(&l->dizi[sira], &l->dizi[sira + 1],
            (l->adet - sira - 1) * sizeof(Ogrenci));
    l->adet--;
}

// ---------- ekran işlemleri ----------

void satirYaz(const Ogrenci *o) {
    printf("%-6d %-10s %-10s %6.2f\n", o->no, o->ad, o->soyad, o->ortalama);
}

void listele(const Liste *l) {
    if (l->adet == 0) {
        printf("Kayıt yok.\n");
        return;
    }
    printf("%-6s %-10s %-10s %6s\n", "No", "Ad", "Soyad", "Ort.");
    for (int i = 0; i < l->adet; i++) {
        satirYaz(&l->dizi[i]);
    }
}

void istatistik(const Liste *l) {
    if (l->adet == 0) {
        printf("Kayıt yok.\n");
        return;
    }
    double toplam = 0;
    int enIyi = 0, enKotu = 0;
    for (int i = 0; i < l->adet; i++) {
        toplam += l->dizi[i].ortalama;
        if (l->dizi[i].ortalama > l->dizi[enIyi].ortalama) enIyi = i;
        if (l->dizi[i].ortalama < l->dizi[enKotu].ortalama) enKotu = i;
    }
    printf("Öğrenci sayısı : %d\n", l->adet);
    printf("Genel ortalama : %.2f\n", toplam / l->adet);
    printf("En yüksek      : %s %s (%.2f)\n", l->dizi[enIyi].ad,
           l->dizi[enIyi].soyad, l->dizi[enIyi].ortalama);
```

```
    printf("En düşük       : %s %s (%.2f)\n", l->dizi[enKotu].ad,
           l->dizi[enKotu].soyad, l->dizi[enKotu].ortalama);
}

// ---------- menü komutları ----------

void komutEkle(Liste *l) {
    char t[100];
    Ogrenci o;

    if (!satirOku("Numara: ", t, sizeof t) || sscanf(t, "%d", &o.no) != 1) {
        printf("Geçersiz numara.\n");
        return;
    }
    if (noBul(l, o.no) != -1) {
        printf("Bu numara zaten kayıtlı.\n");
        return;
    }
    if (!satirOku("Ad Soyad: ", t, sizeof t) ||
        sscanf(t, "%29s %29s", o.ad, o.soyad) != 2) {
        printf("Ad ve soyad gerekli.\n");
        return;
    }
    if (!satirOku("Ortalama: ", t, sizeof t) ||
        sscanf(t, "%lf", &o.ortalama) != 1 ||
        o.ortalama < 0 || o.ortalama > 100) {
        printf("Ortalama 0-100 arasında olmalı.\n");
        return;
    }
    if (ogrenciEkle(l, o)) {
        printf("Kayıt eklendi.\n");
    } else {
        printf("Bellek yetersiz.\n");
    }
}

void komutAra(const Liste *l) {
    char t[100];
    int no;
    if (!satirOku("Aranan numara: ", t, sizeof t) || sscanf(t, "%d", &no) != 1) {
        printf("Geçersiz numara.\n");
        return;
    }
    int sira = noBul(l, no);
    if (sira == -1) {
        printf("Öğrenci bulunamadı.\n");
    } else {
        satirYaz(&l->dizi[sira]);
    }
}

void komutSil(Liste *l) {
    char t[100];
    int no;
    if (!satirOku("Silinecek numara: ", t, sizeof t) || sscanf(t, "%d", &no) != 1) {
        printf("Geçersiz numara.\n");
        return;
    }
    int sira = noBul(l, no);
    if (sira == -1) {
        printf("Öğrenci bulunamadı.\n");
    } else {
        ogrenciSil(l, sira);
        printf("Kayıt silindi.\n");
```

```
    }
}

// ---------- dosya işlemleri ----------

int kaydet(const Liste *l) {
    FILE *f = fopen(DOSYA_ADI, "wb");
    if (f == NULL) {
        return 0;
    }
    int ok = fwrite(&l->adet, sizeof(int), 1, f) == 1 &&
             fwrite(l->dizi, sizeof(Ogrenci), l->adet, f) == (size_t)l->adet;
    fclose(f);
    return ok;
}

int yukle(Liste *l) {
    FILE *f = fopen(DOSYA_ADI, "rb");
    if (f == NULL) {
        return 0;
    }
    int adet;
    if (fread(&adet, sizeof(int), 1, f) != 1 || adet < 0) {
        fclose(f);
        return 0;
    }
    int kapasite = adet < 2 ? 2 : adet;
    Ogrenci *yeni = malloc(kapasite * sizeof(Ogrenci));
    if (yeni == NULL ||
        fread(yeni, sizeof(Ogrenci), adet, f) != (size_t)adet) {
        free(yeni);
        fclose(f);
        return 0;
    }
    fclose(f);
    free(l->dizi);                      // eski diziyi bırak, yenisini al
    l->dizi = yeni;
    l->adet = adet;
    l->kapasite = kapasite;
    return 1;
}

// ---------- ana program ----------

void menuYaz(void) {
    printf("1 Ekle 2 Listele 3 Ara 4 Sil 5 İstatistik "
           "6 Kaydet 7 Yükle 9 Menü 0 Çıkış\n");
}

int main(void) {
    Liste liste;
    char t[100];

    if (!listeBaslat(&liste)) {
        printf("Bellek ayrılamadı.\n");
        return 1;
    }

    menuYaz();
    for (;;) {
        int secim;
        if (!satirOku("> ", t, sizeof t)) {
            break;                      // girdi sona erdi
        }
```

```
        if (sscanf(t, "%d", &secim) != 1) {
            secim = -1;
        }
        if (secim == 0) {
            break;
        }
        switch (secim) {
            case 1: komutEkle(&liste); break;
            case 2: listele(&liste); break;
            case 3: komutAra(&liste); break;
            case 4: komutSil(&liste); break;
            case 5: istatistik(&liste); break;
            case 6:
                if (kaydet(&liste)) {
                    printf("%d kayıt dosyaya yazıldı.\n", liste.adet);
                } else {
                    printf("Dosya yazılamadı.\n");
                }
                break;
            case 7:
                if (yukle(&liste)) {
                    printf("%d kayıt yüklendi.\n", liste.adet);
                } else {
                    printf("Dosya okunamadı.\n");
                }
                break;
            case 9: menuYaz(); break;
            default: printf("Geçersiz seçim. Menü için 9 yazın.\n");
        }
    }

    free(liste.dizi);
    printf("Güle güle.\n");
    return 0;
}
```

20.10 Örnek Çalışma

Programı derleyip aşağıdaki sırayla çalıştıralım: üç öğrenci ekleyelim, birini arayalım, birini silelim, kayıtları dosyaya yazalım, sonra bir öğrenciyi daha silip dosyadan geri yükleyelim ve son durumu görelim. Satırların başındaki `>` işaretinden sonra yazılan değerler kullanıcının girdisidir.

```
ÇIKTI
1 Ekle 2 Listele 3 Ara 4 Sil 5 İstatistik 6 Kaydet 7 Yükle 9 Menü 0 Çıkış
> 1
Numara: 101
Ad Soyad: Ayse Yilmaz
Ortalama: 84.5
Kayıt eklendi.
> 1
Numara: 102
Ad Soyad: Mehmet Demir
Ortalama: 71
Kayıt eklendi.
> 1
Numara: 103
Ad Soyad: Elif Kaya
Ortalama: 92.25
Kayıt eklendi.
> 3
Aranan numara: 103
103    Elif       Kaya        92.25
> 4
Silinecek numara: 102
Kayıt silindi.
> 6
2 kayıt dosyaya yazıldı.
> 4
Silinecek numara: 101
Kayıt silindi.
> 7
2 kayıt yüklendi.
> 2
No     Ad         Soyad        Ort.
101    Ayse       Yilmaz      84.50
103    Elif       Kaya        92.25
> 5
Öğrenci sayısı : 2
Genel ortalama : 88.38
En yüksek      : Elif Kaya (92.25)
En düşük       : Ayse Yilmaz (84.50)
> 0
Güle güle.
```

Çıktıdan birkaç gözlem yapalım:

- Üç ekleme, başlangıç kapasitesi 2 olan listeyi bir kez büyüttü (2'den 4'e). Bu, kullanıcıya görünmez ama `realloc` yolunu da çalıştırmış olduk.
- Genel ortalama (84,50 + 92,25) / 2 = 88,375'tir; `%.2f` bunu 88.38 olarak yuvarlar.
- Altıncı seçimle 102 numaralı öğrenci silindikten sonraki hâl dosyaya yazıldı (2 kayıt). Ardından 101 numaralı öğrenci de silindi, ama yükleme komutu dosyadaki iki kaydı geri getirdi; son listede 101 ve 103 yer alıyor.
- `main` çıkışta `free` çağırdığı için `valgrind` ya da AddressSanitizer (Bölüm 13) bu çalışmada sızıntı bildirmez. Denemek için `gcc -std=c17 -g -fsanitize=address ogrenciKayit.c` ile derleyebilirsiniz.

> **İpucu · Büyük programı küçük adımlarla kurun**
>
> Bu program ilk günden bu hâlde yazılmadı. Önce yalnızca `Ogrenci` yapısı ve `listele` yazılır, sabit birkaç kayıtla denenir. Sonra `ogrenciEkle`, sonra arama ve silme gelir; dosya işlemleri en sona kalır. Her adımda program derlenip çalıştırılır. Bir hata çıktığında yalnızca son eklenen fonksiyon şüphelidir.

Geliştirme Önerileri

Program çalışıyor ve gereksinimleri karşılıyor, ama bir uygulamayı gerçekten öğrenmenin yolu onu kendi isteklerinize göre değiştirmektir. Aşağıdaki öneriler, yaklaşık artan zorluk sırasıyla verilmiştir. Her birini bağımsız yapabilirsiniz.

1. **Ad ve soyada boşluk desteği.** `Ogrenci` yapısındaki `ad` ve `soyad` alanlarını, satırın tamamını okuyan tek bir `adSoyad` alanıyla değiştirin. Bunun için `sscanf`'te `%[^\n]` kullanılabilir. Tablo sütunlarının hizasını yeniden düşünün.
2. **Ortalamaya göre sıralama.** Kayıtları ortalamaya göre büyükten küçüğe sıralayan bir `komutSirala` yazın. Bölüm 10'daki sıralama algoritmalarından birini ya da standart kütüphanedeki `qsort`'u kullanabilirsiniz. `qsort` için karşılaştırma fonksiyonu yazmanız gerekir.
3. **Güncelleme komutu.** Numarası verilen öğrencinin ortalamasını değiştiren bir komut ekleyin. Girdi denetimlerini `komutEkle` ile paylaşmak için ortalamayı okuyan ayrı bir yardımcı fonksiyon çıkarın.
4. **Ders notları.** Her öğrenci için ortalama yerine, en fazla beş dersin notunu tutan bir `double notlar[5]` dizisi tanımlayın; ortalamayı hesaplayarak bulun. Listeleme tablosuna yeni sütunlar ekleyin.
5. **Metin dosyası biçimi.** İkili kaydetme yerine, her satırda bir öğrencinin `no;ad;soyad;ortalama` biçiminde yazıldığı bir metin dosyası kullanın. Yüklemede `fgets` ve `sscanf` (ya da `strtok`) ile satırları ayrıştırın. Bozuk bir satırla karşılaşınca ne yapılacağına karar verin.
6. **Başlangıçta otomatik yükleme.** Program açılırken `kayitlar.bin` varsa kayıtları otomatik yükleyin, çıkarken kullanıcıya kaydetmek isteyip istemediğini sorun.
7. **Bağlı liste ile yeniden yazım.** Dizi yerine Bölüm 19'daki tek yönlü bağlı listeyi kullanın. Silme işleminin `memmove` yerine tek bir işaretçi ataması olduğuna dikkat edin; buna karşılık numaraya göre aramanın hangi maliyette kaldığını düşünün.
8. **Çok dosyalı yapı.** Programı `ogrenci.h`, `liste.c`, `dosya.c` ve `main.c` olarak bölün (Bölüm 15). Hangi fonksiyonların başlık dosyasında bildirilmesi, hangilerinin `static` olması gerektiğine karar verin ve bir `Makefile` yazın.

Bölüm Özeti

- Bir uygulama önce gereksinimlerin yazılmasıyla başlar; gereksinimler veri yapısını, bellek yönetimini ve dosya biçimini belirler.
- İlgili verileri tek bir `struct` altında toplamak, parametre listelerini kısaltır ve tutarsızlığı önler; `typedef` ile tipe kısa bir ad verilir.
- `dizi`, `adet` ve `kapasite` üçlüsünü bir yapıda tutmak, büyüyen dizilerin standart kalıbıdır.
- `realloc` sonucu önce geçici bir işaretçiye alınmalıdır; aksi hâlde başarısızlıkta eski blok sızar.
- Kapasiteyi ikiye katlamak, ardışık eklemelerde büyütme sayısını az tutar.
- Her girdiyi `fgets` ile satır olarak okuyup `sscanf` ile ayrıştırmak, arabellekte artık bırakmayan ve dönüş değeri denetlenebilen güvenli bir kalıptır.
- Programı katmanlara bölmek (liste işlemleri, komutlar, `main`) değişikliği kolaylaştırır; alt katman işlevleri ekrana yazmaz.
- Silmede `memmove` ile elemanlar bir öne kaydırılır; `-1` dönüş değerleri sıraya çevrilmeden kullanılmamalıdır.

- Aşırı iç içe `if`'ler yerine denetim başarısız olunca erken dönüş yapmak akışı düzleştirir.
- Sabit boyutlu `char` dizileri içeren yapılar, `fwrite` / `fread` ile tek çağrıda dosyaya yazılıp okunabilir; ancak bu biçim platforma bağlıdır.
- Dosyadan yüklerken verileri önce geçici belleğe okuyup başarıdan sonra mevcut listeyle değiştirmek, hata durumunda eldeki veriyi korur.
- Ayrılan her bellek bloğu bir yerde `free` edilmelidir; `malloc` ile `free` çiftlerinin sayısı tutmalıdır.

KISIM II

C++

1985 · Bell Laboratuvarları

Bjarne Stroustrup 1979'da Bell Laboratuvarları'nda C'ye sınıflar eklemeye başladığında amacı, C'nin hızını korurken büyük programları düzenlemeyi kolaylaştırmaktı. Ortaya çıkan C++, 1985'te dünyaya açıldı ve zamanla oyun motorlarından tarayıcılara, finans sistemlerinden bilimsel hesaplamaya kadar performansın önemli olduğu her alana yayıldı.

Bu kısımda C bilgimizin üzerine inşa edeceğiz. Önce C++'ın C'ye getirdiği küçük ama önemli yenilikleri, ardından nesne yönelimli programlamayı, şablonları, standart şablon kütüphanesini (STL) ve modern C++'ı göreceğiz.

BÖLÜM 21

C'den C++'a: Tarih, Felsefe ve İlk Program

BU BÖLÜMDE

- C'nin büyük programlarda zorlandığı noktaları ve C++'ın bunlara getirdiği yanıtları
- C++'ın tasarım felsefesini: çok paradigmalılık, sıfır maliyetli soyutlama, "kullanmadığın şeyin bedelini ödemezsin" ilkesi
- C ile C++ arasındaki uyumu ve C kodunun C++ derleyicisinde neden bazen derlenmediğini
- İlk C++ programını satır satır: `<iostream>`, `std::cout`, `<<`, `std::endl` ile `'\n'` farkı
- Ad alanlarını (`namespace`) ve `using namespace std;` satırının yararını ve bedelini
- `g++` ile derlemeyi, dosya uzantılarını ve kitabın kullandığı C++17 standardını

C'yi şimdiye kadar öğrendik: değişkenler, kontrol yapıları, fonksiyonlar, işaretçiler, dinamik bellek, yapılar, dosyalar ve veri yapıları. Bu bilgilerle çalışan, yüzlerce satırlık programlar yazabiliyoruz. Bu bölümden itibaren kitabın ikinci kısmına, C++'a geçiyoruz.

C++'ı sıfırdan bir dil gibi değil, C'nin üzerine kurulmuş bir dil olarak öğreneceğiz. Bu yüzden her yerde aynı soruyu soracağız: C'de nasıl yapıyorduk, C++ bunu neden ve nasıl değiştirdi? Bölümün ilk yarısı bu sorunun genel yanıtıdır; ikinci yarısında ilk C++ programımızı yazıp derleyeceğiz. C++'ın tarihçesini Bölüm 2'de kısaca anlatmıştık, burada tarihi tekrar etmeyecek, tarihin ortaya çıkardığı **tasarım kararlarına** odaklanacağız. Derleyici kurulumu için Bölüm 3'e başvurulabilir.

21.1 C++ Neden Doğdu?

Bölüm 2'de gördüğümüz gibi Bjarne Stroustrup, Simula'nın sınıf fikrini C'nin hızıyla birleştirmek istedi. Bu istek soyut bir merak değildi; büyük programlarda C ile çalışırken somut sorunlar yaşanıyordu. Bu sorunları, şimdiye kadar yazdığımız programların büyümüş hâlini düşünerek görelim.

Sorun 1: Her ad herkesin ortak malı

C'de fonksiyon ve global değişken adları tüm programda tek bir ortak havuzdadır. İki ayrı kütüphane aynı adı kullanırsa bağlayıcı (linker) bunu çözemez. Aşağıdaki iki dosya ayrı ayrı sorunsuzdur, birlikte derlenemez:

```c
// grafik.c
void baslat(void) { /* ekranı hazırla */ }

// ses.c
void baslat(void) { /* ses kartını hazırla */ }
```

```
TERMİNAL
$ gcc grafik.c ses.c main.c -o program
/usr/bin/ld: ses.o: in function `baslat':
ses.c:(.text+0x0): multiple definition of `baslat'; grafik.o:grafik.c:(.text+0x0): first
defined here
```

C'de bunun çaresi adlara önek koymaktır (`grafik_baslat`, `ses_baslat`). Çalışır, ama adlar uzar ve önekleri unutmamak programcıya kalır. C++ bunun yerine dilin kendisine **ad alanı** (namespace) kavramını koydu; bu bölümde standart kütüphane üzerinden tanışacağız.

Sorun 2: Veri ile onu işleyen kod birbirinden kopuk

Bir yığın (stack) yapısını Bölüm 19'da `struct` ve fonksiyonlarla yazmıştık. Veri ve işlemler ayrı yerlerde durur, aralarındaki bağ yalnızca adlandırma geleneğidir:

```c
struct Yigin {
    int eleman[100];
    int ust;              // en üstteki elemanın indeksi
};

void yiginaEkle(struct Yigin *y, int deger);
int  yigindanAl(struct Yigin *y);
```

Bu tasarımın zayıf yanı, `ust` alanına herkesin doğrudan ulaşabilmesidir. Kodun herhangi bir yerinde `y.ust = -7;` yazılırsa yığın bozulur ve derleyici buna ses çıkarmaz. Yapının iç düzenini dışarıdan korumanın C'de dil düzeyinde bir yolu yoktur. C++'ın **sınıfı** (class) veriyi ve ona uygulanabilecek işlemleri tek bir birimde toplar, iç ayrıntıları da dışarıdan gizleyebilir (Bölüm 30).

Sorun 3: Kaynakları elle geri vermek

C'de aldığımız her kaynağı kendimiz geri vermek zorundayız: `malloc` ile `free`, `fopen` ile `fclose`. Fonksiyonun ortasında erken `return` yazdığımız her yerde bu çiftlerden birini unutmak mümkündür. Bellek sızıntılarının çoğu böyle doğar. C++ nesnelerin ömrünü dile bağlar: bir nesne kapsamdan çıkarken kendi temizlik kodunu otomatik çalıştırır. Bu fikre **RAII** denir ve Bölüm 29'un konusudur.

Sorun 4: Tipten bağımsız kod yazmanın zorluğu

İki sayının büyüğünü veren fonksiyonu `int` için yazarsak `double` için ikincisini yazmak gerekir. C'de iki çıkış yolu vardır: makro kullanmak ya da `void *` ile tip bilgisini bırakmak.

```c
#define BUYUK(a, b) ((a) > (b) ? (a) : (b))   // tip denetimi yok

void *aramaYap(void *dizi, int adet, int elemanBoyu);  // tip bilgisi kayıp
```

İkisi de güvenli değildir: makro argümanı iki kez değerlendirir (Bölüm 15), `void *` ise derleyicinin tip hatalarını yakalamasını engeller. C++ bu ihtiyaca **şablonlarla** (templates) yanıt verir; derleyici tipi bilir, kodu her tip için ayrı üretir (Bölüm 33).

Sorunlar ve yanıtlar bir arada

Bu sorunların ve birkaç benzerinin C++ karşılığı şu tabloda toplanabilir:

C'de sorun	C++'ın yanıtı	Nerede işlenecek
Ortak ad havuzu, ad çakışması	Ad alanları (`namespace`)	Bölüm 21, 22
Veri ile işlemler kopuk, veri korunmuyor	Sınıflar, `private` üyeler	Bölüm 30
Kaynağı elle geri verme	Yıkıcılar, RAII, akıllı işaretçiler	Bölüm 29
Tipten bağımsız kodda makro ve `void *`	Şablonlar	Bölüm 33
Hata bildirimi için dönüş kodu ve `errno`	İstisnalar (`exception`)	Bölüm 34
Tekrar eden benzer yapılar	Kalıtım ve çok biçimlilik	Bölüm 31

Bunların hepsini bu bölümde öğrenmeyeceğiz. Amaç, ileride karşılaşacağımız her özelliğin bir sorunu çözmek için eklendiğini baştan görmektir. Sebebini bilerek öğrenilen özellik hem daha kolay akılda kalır hem de ne zaman kullanılmayacağı daha rahat anlaşılır.

> *Tarihçe · C++'ın dayandığı fikir*
>
> C++'ın tasarımında iki kaynak öne çıkar: Simula'dan gelen sınıf ve nesne kavramı, C'den gelen donanıma yakınlık ve hız. Stroustrup'un amacı Simula'nın düzenini C'nin verimliliğinden ödün vermeden sağlamaktı. Dilin bugünkü kimliği bu iki ucun arasındaki gerilimden doğar: üst düzey soyutlamalar sunar, ama alt düzeyde bellek ve donanım denetimini de elden bırakmaz.

21.2 C++'ın Tasarım Felsefesi

Çok paradigmalı bir dil

Programlama paradigmalarını Bölüm 1'de tanımıştık. C yordamsal (procedural) bir dildir: program, birbirini çağıran fonksiyonlardan oluşur. C++ ise tek bir yaklaşımı dayatmaz; aynı programda birlikte kullanılabilen birkaç yaklaşımı destekler:

- **Yordamsal**: C'deki gibi fonksiyonlar ve veri yapıları. C++'ta da aynen yazılabilir.
- **Nesne yönelimli**: Sınıflar, kalıtım, çok biçimlilik.
- **Genel (generic) programlama**: Şablonlar ve STL.
- **Fonksiyonel öğeler**: Lambda ifadeleri, algoritmaların fonksiyon alması (C++11'den itibaren).

Bu çok yönlülük bir güç olduğu kadar bir sorumluluktur. C++'ta aynı işi yapmanın çok yolu vardır ve hangisini seçeceğimiz bizim kararımızdır. Kitap boyunca "bu işi C'de şöyle yapardık, C++'ta şu daha güvenli ve kısadır" yönünde seçim yapmayı öğreneceğiz.

Sıfır maliyetli soyutlama

C++'ın temel ilkelerinden biri **sıfır maliyetli soyutlama** ilkesidir. İlke iki cümleyle söylenebilir:

> **Tanım: Sıfır maliyetli soyutlama**
>
> (zero-overhead principle) Kullanmadığımız bir özelliğin bedelini ödemeyiz; kullandığımız bir özelliği ise elle yazsaydık yazacağımızdan daha verimli yazamayız. Başka bir deyişle, üst düzey bir yapı (sınıf, şablon, kap) kullanmak, aynı işi elle düşük düzeyde yazmaktan daha yavaş olmamalıdır.

Bu ilke iki yönlüdür. Birincisi, dilin yeni özellikleri, o özellikleri kullanmayan kodu yavaşlatmaz. İkincisi, soyutlamanın bir bedeli olacaksa bunu yalnızca o soyutlamayı kullanan taraf öder.

Bunu somut bir deneyle görelim. Aşağıdaki programda dört farklı yapının bellekte kapladığı yeri ölçüyoruz. Sınıf söz dizimine henüz yabancıyız; üyeleri okumaya çalışmayalım, yalnızca `sizeof` sonuçlarına bakalım:

▸ sifirMaliyet.cpp

```cpp
#include <iostream>
#include <array>
using namespace std;

struct Nokta {
    int x;
    int y;
};

class Sayac {
    int deger = 0;              // dışarıdan erişilemez
public:
    void artir() { ++deger; }
    int al() const { return deger; }
};

class Sesli {
    int deger = 0;
public:
    virtual void konus() {}     // çok biçimlilik için
    virtual ~Sesli() {}
};

int main() {
    cout << "Nokta (struct)    : " << sizeof(Nokta) << '\n';
    cout << "Sayac (class)     : " << sizeof(Sayac) << '\n';
    cout << "array<int,5>      : " << sizeof(array<int, 5>) << '\n';
    cout << "int[5]            : " << sizeof(int[5]) << '\n';
    cout << "Sesli (virtual)   : " << sizeof(Sesli) << '\n';
    return 0;
}
```

```
ÇIKTI
Nokta (struct)    : 8
Sayac (class)     : 4
array<int,5>      : 20
int[5]            : 20
Sesli (virtual)   : 16
```

Çıktı 64 bitlik bir Linux sisteminde alındı. Üç gözlem yapabiliriz:

1. `Sayac` sınıfı tek bir `int` tutar ve boyutu 4 bayttır. Veri gizleme ve üye fonksiyonlar bellekte hiçbir ek yer kaplamaz; bu özellikler derleme zamanında çözülür.
2. `array<int, 5>` ile C dizisi `int[5]` aynı büyüklüktedir. Standart kütüphanenin güvenli sarmalayıcısı, ham dizinin üzerine eklenmiş gizli bir yük taşımaz.

3. `Sesli` sınıfı `virtual` fonksiyon içerdiği için `int` verisinin yanında bir de işaretçi (sanal tablo işaretçisi) taşır; 64 bitlik sistemde 4 + 8 bayt, hizalamayla 16 bayt eder. Bedel, yalnızca çok biçimlilik **kullanıldığı** için ödenir.

İkinci ilke, "kullanmadığın şeyin bedelini ödemezsin"in diğer yüzüdür: çok biçimlilik istemeyen sınıfta sanal tablo işaretçisi yoktur. Aynı ilkenin başka örneklerini şablonlarda ve standart kütüphanede göreceğiz.

> ◆ **Kritik Nokta · C++ güvenlik ağı kurmaz, seçenek sunar**
>
> Sıfır maliyet ilkesinin getirdiği sonuç şudur: C++ varsayılan olarak C kadar hızlı ve C kadar tehlikelidir. `vector` elemanına `[]` ile erişirken sınır denetimi yapılmaz; denetimli erişim (`.at()`) ayrı bir seçenektir. Güvenlik özelliklerinin çoğu isteğe bağlıdır ve bunları kullanmak programcının sorumluluğundadır.

C++ ile C arasındaki iş bölümü

C++'ın modern üslubu C üslubundan belirgin biçimde farklıdır. Aşağıdaki tablo, kitabın C++ kısmında hangi C aracının yerini hangisinin alacağını önceden gösterir:

C'de	Modern C++'ta	Bölüm
`printf` , `scanf`	`cout` , `cin`	21, 23
`char[]` ve `<string.h>`	`std::string`	27
Dizi ve eleman sayısını ayrıca taşımak	`std::vector` , `std::array`	26
`malloc` / `free`	`new` / `delete` , ya da daha iyisi akıllı işaretçiler	29
İşaretçi ile "adres geçirme"	Referans (`&`)	25, 28
`#define` sabiti	`const` / `constexpr`	22
Makro ile genel kod	Şablon	33
Dönüş kodu ile hata bildirimi	İstisna (`exception`)	34
`FILE *`	`ifstream` / `ofstream`	37

Bu liste, "C++ kitabında C'nin hangi araçlarını kullanmayacağız?" sorusunun da cevabıdır. C araçları C++'ta çalışmaya devam eder ve gerektiğinde onları kullanabiliriz; ama alışkanlık olarak modern karşılıklarını seçeceğiz.

21.3 C ile Uyumluluk

C++ "C'nin üst kümesi" olarak anılır, ama bu ifade ancak yaklaşık olarak doğrudur. Dilin tasarımında C ile uyumluluğa büyük önem verildi: C kütüphaneleri C++ programlarından çağrılabilir, çoğu C kodu değişmeden derlenir ve C programcısı C++'a kendi bildiğinin üzerine ekleyerek geçebilir. Fakat iki dil aynı standart komitelerinde değil, ayrı ayrı gelişti ve zamanla yollar ayrıldı.

C'den gelen her şey çalışır

C++'ta, aşağıdakilerin hepsi C'dekiyle aynı anlama gelir ve aynı biçimde yazılır:

- Temel tipler, operatörler, işleç öncelikleri (Bölüm 5).
- `if` , `switch` , `for` , `while` , `do-while` , `break` , `continue` , `goto` (Bölüm 7, 8).

- Fonksiyonlar, özyineleme (Bölüm 9, 18).
- Diziler, işaretçiler, işaretçi aritmetiği (Bölüm 10, 12).
- `struct`, `union`, `enum`, `typedef` (Bölüm 14).
- Önişlemci komutları (Bölüm 15), bit işlemleri (Bölüm 16).

Standart C kütüphanesi de C++'ta kullanılabilir. Başlık adlarının başına `c` konur, `.h` uzantısı düşer: `<stdio.h>` yerine `<cstdio>`, `<string.h>` yerine `<cstring>`, `<math.h>` yerine `<cmath>`. Bu başlıkların sunduğu adlar `std` ad alanında bulunur (`std::printf`, `std::sqrt`); uygulamada çoğu derleyici onları genel alana da koyar, ama buna güvenmemek iyidir. Eski `.h` biçimli başlıklar da çalışır. Aşağıdaki program C'nin `printf` ve `sqrt` fonksiyonlarını C++'ta kullanır:

▸ cKutuphane.cpp

```cpp
#include <cstdio>
#include <cmath>

int main() {
    double kenar = 3.0;
    std::printf("Karenin köşegeni: %.4f\n", kenar * std::sqrt(2.0));
    return 0;
}
```

ÇIKTI

```
Karenin köşegeni: 4.2426
```

C kodu her zaman C++ kodu değildir

Şimdi "neredeyse" sözcüğünün neyi gizlediğine bakalım. Aşağıdaki farkların her biri, geçerli bir C programının C++ derleyicisinde hata verdiği ya da farklı davrandığı durumlardır.

Fark 1: `void *` örtük olarak dönüştürülmez

C'de `malloc`'un döndürdüğü `void *`, herhangi bir işaretçiye atanabilir (Bölüm 13). C++ bu dönüşümü açıkça yazmamızı ister:

▸ ortukVoid.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(4 * sizeof(int));
    p[0] = 7;
    printf("%d\n", p[0]);
    free(p);
    return 0;
}
```

ÇIKTI

```
7
```

Bu kod C'de sorunsuzdur. Aynı dosyayı C++ olarak derlemesini `g++`'a `-x c++` seçeneğiyle söyleyelim:

TERMİNAL

```
$ g++ -std=c++17 -Wall -x c++ ortukVoid.c -o ortukVoid
```

DERLEYİCİ ÇIKTISI

```
ortukVoid.c: In function 'int main()':
ortukVoid.c:5:20: error: invalid conversion from 'void*' to 'int*' [-fpermissive]
    5 |     int *p = malloc(4 * sizeof(int));
      |              ~~~~~~^~~~~~~~~~~~~~~~~
      |                    |
      |                    void*
```

C++'ta çözüm `malloc`'a başvurmak değil, `new` kullanmaktır (Bölüm 29). Yine de C kodunu C++'a taşırken `(int *)malloc(...)` biçimindeki dönüşüm işe yarar.

Fark 2: Boş parantezin anlamı

C'de `void yaz();` bildirimi "parametreleri belirtilmemiş" demektir ve `yaz(1, 2)` çağrısı derlenir. C++'ta `void yaz();` ile `void yaz(void);` aynıdır: fonksiyon hiç parametre almaz.

bosParantez.c

```c
#include <stdio.h>

void yaz();

int main(void) {
    yaz(1, 2);
    return 0;
}

void yaz() {
    printf("yaz\n");
}
```

ÇIKTI

```
yaz
```

C'de derlenir ve çalışır (fazladan verilen argümanlar sessizce yok sayılır). C++ derleyicisinde ise çağrı reddedilir:

TERMİNAL

```
$ g++ -std=c++17 -Wall -x c++ bosParantez.c -o bosParantez
```

DERLEYİCİ ÇIKTISI

```
bosParantez.c: In function 'int main()':
bosParantez.c:6:8: error: too many arguments to function 'void yaz()'
    6 |     yaz(1, 2);
      |     ~~~^~~~~~
bosParantez.c:3:6: note: declared here
    3 | void yaz();
      |      ^~~
```

Fark 3: C++ anahtar sözcükleri C'de serbest adlardır

C++'ta `class`, `new`, `delete`, `this`, `template`, `namespace`, `try`, `catch`, `throw`, `public`, `private`, `virtual` ve daha pek çok sözcük anahtar sözcüktür. C'de bunlar sıradan adlardır ve değişken ismi olabilir:

anahtarSozcuk.c

```c
#include <stdio.h>

int main(void) {
    int new = 5;
    int class = 2;
    printf("%d\n", new * class);
    return 0;
}
```

```
ÇIKTI
10
```

Aynı kod C++ olarak derlenirse ilk değişken satırında `expected unqualified-id before 'new'` hatası alırız. C kütüphanesi yazan biri, başlık dosyasında `class` ya da `new` adlı bir parametre kullanmışsa, o başlık C++'tan kullanılamaz hâle gelir.

Fark 4: `struct` adı doğrudan tip adıdır

C'de `struct Nokta {...};` tanımından sonra tipi kullanırken `struct` sözcüğünü yazmak (ya da `typedef` ile ad vermek) gerekir. C++'ta etiket doğrudan tip adı olur:

```c
struct Nokta { int x; int y; };

struct Nokta a = {1, 2};   // C'de ve C++'ta geçerli
Nokta b = {3, 4};          // yalnızca C++'ta geçerli
```

Bu yüzden C++ kodunda `typedef struct {...} Nokta;` kalıbına gerek kalmaz. C derleyicisi ikinci satır için `unknown type name 'Nokta'; use 'struct' keyword to refer to the type` hatası verir.

Fark 5: Karakter sabitinin tipi

C'de `'a'` gibi bir karakter sabitinin tipi `int`'tir; C++'ta `char`'dır. Fark, `sizeof` ile ya da fonksiyon aşırı yükleme ile ortaya çıkar. Aynı mantıkla iki programı karşılaştıralım:

harfBoyutuC.c

```c
#include <stdio.h>

int main(void) {
    printf("%zu\n", sizeof('a'));
    return 0;
}
```

```
ÇIKTI
4
```

harfBoyutuCpp.cpp

```cpp
#include <iostream>

int main() {
```

```
    std::cout << sizeof('a') << '\n';
    return 0;
}
```

```
ÇIKTI
1
```

Aynı ifade iki dilde farklı sonuç verdi. Bu, derlenip de sessizce farklı davranan nadir örneklerden biridir.

Diğer farklar

Bunlar dışında daha az sık karşılaşılan başka farklar da vardır. Kitabın ilerleyen bölümlerinde ilgili yerde yeniden değineceğiz; şimdilik tablo olarak görelim:

Konu	C	C++
Mantıksal tip	`_Bool` (C99), `<stdbool.h>` ile `bool`	`bool`, `true`, `false` yerleşik
`main` sonunda `return 0`	C99'dan beri isteğe bağlı	İsteğe bağlı
Değişken uzunluklu dizi (VLA)	C99'da standart	Standart değil (`g++` uzantı olarak kabul eder)
Atanmış ilk değer (`.x = 1`)	C99	C++20'de sınırlı biçimde geldi
Bileşik sabit (`(struct P){1,2}`)	C99	Yok
Hiçbir argüman almayan `main()`	`main(void)` önerilir	`main()` yeterli
`const` global değişkenin bağlantısı	Dış bağlantı (extern)	İç bağlantı (dosyaya özel)
`enum` değerinden tamsayıya atama	Serbest	`int`'ten `enum`'a örtük dönüşüm yok
Aynı ada sahip fonksiyonlar	Yok	Aşırı yükleme (Bölüm 25)

Bu listeyi ezberlemek gerekmez. Kuralı şöyle koyabiliriz: C++ derleyicisi C'den daha katı bir tip denetimi yapar. C kodu C++'a taşındığında karşılaşılan hataların neredeyse tamamı bu sıkılıktan gelir.

> **▲ Sık Yapılan Hata · C kodunu g++ ile derlemek, C++ yazmak demek değildir**
>
> C dosyasının uzantısını `.cpp` yapıp derleyebilmek, o kodun iyi bir C++ kodu olduğu anlamına gelmez. `malloc`, `printf`, `char[]` ve makrolarla dolu bir programın `g++` ile derlenmesi, yalnızca uyumluluğun bir sonucudur. Bu kitabın C++ kısmında amacımız C++'ın kendi araçlarıyla yazmayı öğrenmektir; C'den yadigâr yöntemler yalnızca eski kodu anlamak için gerekir.

C kodunu C++ içinden çağırmak

C kütüphanelerini C++ programlarında kullanmak yaygın bir ihtiyaçtır. Burada karşılaşılan tek zorluk fonksiyon adlarıdır: C++ aşırı yüklemeyi desteklediği için derleyici fonksiyon adlarını imzalarına göre değiştirir (name mangling, Bölüm 25). C derleyicisi ise adı olduğu gibi bırakır. Bağlayıcı eşleştirme yapamazsa hata verir. Çözüm, bildirimleri `extern "C"` ile işaretlemektir:

```
extern "C" {
    #include "ckutuphane.h"      // C ile yazılmış başlık
}
```

`extern "C"` bloğu, "içindeki adları C kurallarıyla bağla" demektir. Sistemin C başlıkları genellikle bunu kendi içinde yapar; çoğu zaman yalnızca kendi ya da üçüncü taraf C kütüphanelerimiz için elle yazmamız gerekir. Çok dosyalı programlarda başlık dosyalarının hem C hem C++ tarafından kullanılabilmesi için `__cplusplus` önişlemci adının tanımlı olup olmadığına bakan koşullu derleme kullanılır; bunu "Standartlar" başlığında göreceğiz.

> **Üç Dilde · Uyumluluk üç dilde** *C · C++ · C#*
>
> C++, C ile kaynak düzeyinde büyük ölçüde uyumludur (yukarıdaki farklarla). C# ise C ile kaynak uyumlu değildir; sözdizimi benzer olsa da `#include`, işaretçi ve `malloc` ilk bakışta C#'ta yoktur. C# koduyla C kütüphanesi konuşturmak için **P/Invoke** adlı ayrı bir mekanizma vardır. Bu konuyu ileride, C# kısmında yeri geldiğinde anacağız.

21.4 İlk C++ Programı

Tasarım felsefesini gördük; şimdi kod yazalım. Bölüm 4'te C ile yazdığımız "Merhaba Dünya" programının C++ karşılığı şudur:

▸ merhabaCpp.cpp

```
#include <iostream>

int main() {
    std::cout << "Merhaba Dünya!\n";
    return 0;
}
```

ÇIKTI

```
Merhaba Dünya!
```

Programı satır satır inceleyelim ve C'deki karşılığıyla ilişkilendirelim.

Satır	Anlamı	C'deki karşılığı
`#include <iostream>`	Giriş-çıkış akışlarının bildirimlerini ekler	`#include <stdio.h>`
`int main()`	Programın başlangıç noktası	`int main(void)`
`std::cout`	Standart çıkış akışı (ekran)	`stdout`
`<<`	Sağdaki veriyi soldaki akışa gönderir	`printf(...)` çağrısı
`"Merhaba Dünya!\n"`	Metin sabiti; `\n` satır sonudur	Aynı
`return 0;`	Başarıyla bitiş	Aynı

`#include <iostream>`

`iostream` adı "input/output stream" ifadesinden gelir. C++ standart başlıklarının adında `.h` yoktur. Bu başlık `cout` (ekrana yazma), `cin` (klavyeden okuma) ve `cerr` (hata çıktısı) nesnelerini tanıtır. C'deki `#include` ile

aynı işi yapar: derleyiciye, kullanacağımız adların bildirimlerini verir (Bölüm 4). Kodlarını eklemez; onlar bağlama aşamasında standart kütüphaneden gelir.

`int main()`

C'de parametre almayan `main`'i `main(void)` yazmayı önermiştik. C++'ta boş parantez zaten "parametre yok" demektir (yukarıdaki Fark 2), bu yüzden `main()` yazılır. Dönüş tipi yine `int`'tir. C++'ın bir kolaylığı da, `main` fonksiyonunun sonuna `return 0;` yazmasak bile derleyicinin onu ekleyecek olmasıdır. Bu kuralı yalnızca `main` için tanır. Biz yine de bu bölümde açıkça yazacağız.

`std::cout` ve `<<` operatörü

`cout`, "character output" kısaltmasıdır ve okunuşu "si-aut"tur. Bir **akış** (stream) nesnesidir: veri soldan sağa doğru bir boru gibi aktarılır. `<<` ise **ekleme operatörüdür** (insertion operator): sağındaki veriyi soldaki akışa ekler. C'de bit kaydırma için tanıdığımız `<<` (Bölüm 16), C++'ta akışlarla kullanıldığında bambaşka bir iş yapar. Bu, **operatör aşırı yükleme** denen özelliktir ve Bölüm 32'de ayrıntısıyla görülecektir. Şimdilik `<<` işaretinin ağzını verinin gittiği yönü gösteren bir ok gibi düşünebiliriz.

Zincirleme yazdırma

`<<` işleci, çalıştıktan sonra soldaki akışın kendisini geri döndürür. Bu sayede birkaç `<<` arka arkaya eklenebilir ve her biri bir öncekinin bıraktığı yerden devam eder:

zincir.cpp

```cpp
#include <iostream>

int main() {
    int yas = 19;
    double boy = 1.75;
    char harf = 'A';

    std::cout << "Yaş: " << yas << '\n';
    std::cout << "Boy: " << boy << " m, harf: " << harf << '\n';
    std::cout << "Toplam: " << 3 + 4 << '\n';
    std::cout << "Karşılaştırma: " << (5 > 3) << '\n';
    return 0;
}
```

ÇIKTI

```
Yaş: 19
Boy: 1.75 m, harf: A
Toplam: 7
Karşılaştırma: 1
```

Bu programda C'den ayrıldığımız önemli bir nokta var: **biçim belirteci yazmadık**. C'de `printf("%d", yas)` ile `%lf` gibi belirteçleri tipe göre seçmek zorundaydık; yanlış belirteç tanımsız davranıştı (Bölüm 6). C++'ta `cout`, değerin tipine bakarak ne yapacağını kendisi seçer: `int`'i tamsayı, `double`'ı ondalıklı sayı, `char`'ı karakter olarak yazar. Böylece belirteç yanlışı diye bir hata sınıfı kalmaz. Biçimlendirme ayrıntıları (ondalık basamak sayısı, hizalama, genişlik) Bölüm 23'te gelecek.

Dikkat edilecek iki ayrıntı vardır:

- Satır sonu için çift tırnakla `"\n"` da yazılabilir, tek tırnakla `'\n'` da. Tek karakter için tek tırnak biraz daha verimlidir.

- `(5 > 3)` ifadesini parantez içine aldık. `<<` işlecinin önceliği karşılaştırma işleçlerinden yüksektir; parantezsiz yazılsaydı `std::cout << "..." << 5 > 3` ifadesi, önce `<<` işlemlerini yapıp sonucu `3` ile karşılaştırmaya çalışırdı ve derlenmezdi. Aritmetik işlemler (`3 + 4`) ise `<<`'dan önce yapıldığı için parantez gerektirmez.

`std::endl` ile `'\n'`

C++ programlarında satır sonu için iki yaygın yazım görürüz:

```
std::cout << "Merhaba" << std::endl;
std::cout << "Merhaba" << '\n';
```

İkisi de ekranda alt satıra geçer. Fark, perde arkasındadır. `std::endl` önce satır sonu karakterini yazar, ardından akışın tamponunu **boşaltır** (flush). Tampon, çıkış akışlarının verimlilik için önde bellekte topladığı yazıdır; boşaltma işlemi bu yazıyı hemen hedefe (ekrana ya da dosyaya) gönderir. `'\n'` ise yalnızca satır sonunu yazar ve tamponun ne zaman boşalacağına sistem karar verir.

Çok sayıda satır yazan bir döngüde her satırdan sonra `std::endl` kullanmak, her seferinde boşaltma yaptığı için yavaşlatır. Kural basittir: sıradan satır sonlarında `'\n'` kullanırız. Yazının hemen görünmesi gereken nadir durumlarda (örneğin programın çökebileceği bir noktadan önce) `std::endl` ya da `std::flush` tercih edilir. C'de aynı işi `fflush(stdout)` yapar.

> **Not · Kitaptaki tercih**
>
> Bölüm 4'teki ucdil kutusunda `std::endl` kullanmıştık; o noktada amaç yalnızca ilk görünümdü. Bu bölümden sonra satır sonu için kural olarak `'\n'` kullanacağız.

Sık hatalar

Yeni başlayanların ilk günlerde en çok yaptığı üç hata ve derleyicinin yanıtları şunlardır. İlki, `#include <iostream>` satırını yazmamaktır:

`baslikYok.cpp`

```
int main() {
    std::cout << "Merhaba\n";
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

baslikYok.cpp: In function 'int main()':
baslikYok.cpp:2:10: error: 'cout' is not a member of 'std'
    2 |     std::cout << "Merhaba\n";
      |          ^~~~
baslikYok.cpp:1:1: note: 'std::cout' is defined in header '<iostream>'; did you forget to
'#include <iostream>'?
  +++ |+#include <iostream>
    1 | int main() {
```

g++ burada çözümü de söylüyor: eksik başlığı ekleyeceğiz. İkincisi, `std::` önekini unutmaktır:

oneksiz.cpp

```cpp
#include <iostream>

int main() {
    cout << "Merhaba\n";
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
oneksiz.cpp: In function 'int main()':
oneksiz.cpp:4:5: error: 'cout' was not declared in this scope; did you mean 'std::cout'?
    4 |     cout << "Merhaba\n";
      |     ^~~~
      |     std::cout
In file included from oneksiz.cpp:1:
/usr/include/c++/13/iostream:63:18: note: 'std::cout' declared here
   63 |   extern ostream cout;          ///< Linked to standard output
      |                  ^~~~
```

Üçüncüsü, C'den gelen alışkanlıkla `;` unutmaktır:

noktaliVirgul.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Merhaba\n"
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
noktaliVirgul.cpp: In function 'int main()':
noktaliVirgul.cpp:4:29: error: expected ';' before 'return'
    4 |     std::cout << "Merhaba\n"
      |                             ^
      |                             ;
    5 |     return 0;
      |     ~~~~~~
```

Bu üçüncü hata Bölüm 4'te gördüğümüz C hatasının aynısıdır: derleyici eksikliği, bir sonraki satıra geldiğinde fark eder.

21.5 Ad Alanları ve `using namespace std;`

Yukarıdaki programlarda `cout` yerine her seferinde `std::cout` yazdık. Bu `std::` öneki neden gerekli, onu kaldırmanın yolu nedir?

> **Tanım: Ad alanı**
>
> (namespace) Bir grup adı (değişken, fonksiyon, sınıf vb.) ortak bir kapsamın altında toplayan, böylece farklı yerlerde aynı adın çakışmadan kullanılabilmesini sağlayan dil yapısıdır. Bir ad alanındaki adlara `ad_alani::ad` biçiminde, **kapsam çözümleme operatörü** `::` ile ulaşılır.

C++ standart kütüphanesindeki bütün adlar (`cout`, `cin`, `string`, `vector`, `sort`...) `std` adlı bir ad alanının içindedir. Bunu bir soyadı gibi düşünebiliriz: `std::cout`, "standart kütüphanenin `cout`'u" demektir. Yukarıda Sorun 1'de gördüğümüz `baslat` çakışması, iki kütüphane farklı ad alanlarında olsaydı (`grafik::baslat`, `ses::baslat`) ortaya çıkmazdı. Kendi ad alanımızı nasıl tanımlayacağımızı Bölüm 22'de göreceğiz.

Her seferinde `std::` yazmak uzundur. Bunu kısaltmanın üç yolu vardır:

```
// 1. Her kullanımda önek (en açık yöntem)
std::cout << "Merhaba\n";

// 2. Yalnızca belirli adları getirmek
using std::cout;
cout << "Merhaba\n";

// 3. Ad alanının tamamını getirmek
using namespace std;
cout << "Merhaba\n";
```

Üçüncü satır en kısa yoldur: `using namespace std;` "bundan sonra `std::` yazmadan da `std` içindeki adları kullanabilirsin" der. C++ öğretim kitaplarının çoğu bunu küçük programlarda kullanır; bu kitapta da bölümlerin ilerleyen kısımlarında örnekleri kısa tutmak için çoğu zaman kullanacağız.

Peki bu satırı gerçek projelerde neden önermiyorlar? Çünkü getirdiği şey yalnızca `cout` değil, `std`'nin **tüm** adlarıdır; bunların arasında kendi değişken ya da fonksiyon adlarımızla aynı olanlar bulunabilir. Aşağıdaki program, bunun masum görünen bir örneğini gösterir:

▸ adCakismasi.cpp

```
#include <algorithm>
#include <iostream>
using namespace std;

int count = 0;

int main() {
    count = count + 1;
    cout << count << '\n';
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

adCakismasi.cpp: In function 'int main()':
adCakismasi.cpp:8:5: error: reference to 'count' is ambiguous
    8 |     count = count + 1;
      |     ^~~~~
```

Standart kütüphanede `std::count` adlı bir algoritma vardır (Bölüm 36). `using namespace std;` ile bu ad görünür hâle geldi ve bizim `count` değişkenimizle karıştı; derleyici hangisini kastettiğimizi bilemiyor. Hata mesajının tamamı daha uzundur, burada kısalttık. Kütüphaneye yeni bir sürümde eklenen bir ad, daha önce derlenen bir programı bozabilir. Bu risk, program büyüdükçe artar.

Bu nedenle pratikte şu kurallara uyarız:

- **Başlık dosyalarında** (`.hpp`, `.h`) `using namespace std;` asla yazılmaz; başlığı ekleyen her dosyayı etkiler.
- Gerçek projelerde `std::` önekini ya da tek tek `using std::cout;` bildirimlerini tercih ederiz.

- Kısa örnek programlarda ve alıştırmalarda `using namespace std;` kullanmak kabul edilebilir. Kitaptaki yoğun örnekleri kısa tutmak için biz de çoğu yerde bunu yapacağız; bu satırın bir kolaylık olduğunu, bir zorunluluk olmadığını unutmayacağız.

> **▲ Sık Yapılan Hata · using namespace satırını başlıkta kullanmayın**
>
> `using namespace std;` bir `.hpp` dosyasının içinde yazılırsa, o başlığı ekleyen her dosya ad çakışması riskini istemeden devralır. Bu hatanın kaynağını bulmak çok güçtür, çünkü hata başlıkla ilgisiz görünen bir dosyada ortaya çıkar.

Aynı program `using namespace std;` ile şöyle görünür:

merhabaUsing.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Merhaba Dünya!\n";
    return 0;
}
```

ÇIKTI

```
Merhaba Dünya!
```

Çıktı değişmedi; yalnızca yazım kısaldı. `using namespace std;` bir **deyimdir**, bu yüzden sonunda `;` bulunur. `#include` ise önişlemci komutudur ve `;` ile bitmez.

21.6 Derleme, Dosya Uzantıları ve Başlıklar

C++ programını derlemek, C'dekiyle aynı dört aşamadan geçer: önişleme, derleme, çevirme, bağlama (Bölüm 4). Yalnızca komut değişir; `gcc` yerine `g++` yazarız:

TERMİNAL

```
$ g++ -std=c++17 -Wall -Wextra merhabaCpp.cpp -o merhabaCpp
$ ./merhabaCpp
Merhaba Dünya!
```

Seçeneklerin anlamları C'dekiyle aynıdır: `-std=c++17` standardı belirler, `-Wall -Wextra` uyarıları açar, `-o` çıktı dosyasının adını verir (Bölüm 4). Windows'ta çalıştırma komutu `.\merhabaCpp` ya da `merhabaCpp.exe` olur.

> **Not · gcc ve g++ farkı**
>
> `gcc` komutu dosyanın uzantısına bakarak dili seçer: `.c` ise C, `.cpp` ise C++ olarak derler. Ancak bağlama aşamasında C++ standart kütüphanesini kendiliğinden eklemez; bu yüzden `.cpp` dosyası için `gcc` kullanılırsa `undefined reference` hataları alınabilir. `g++` ise C++ kütüphanesini bağlar ve `.c` uzantılı dosyayı bile C++ olarak derler. C++ için her zaman `g++` kullanacağız.

Dosya uzantıları

C++ için uzantı konusunda tek bir standart yoktur; yaygın olanlar şunlardır:

Dosya türü	Yaygın uzantılar	Bu kitapta
Kaynak dosya	`.cpp`, `.cc`, `.cxx`, `.C`	`.cpp`
Başlık dosyası	`.hpp`, `.hh`, `.hxx`, `.h`	`.hpp`

Derleyici uzantıya bakarak dili belirler. `.c` uzantısı C, `.cpp` uzantısı C++ demektir. Başlık uzantısı olarak `.h` de C++'ta kullanılır, ama hem C hem C++ tarafından okunabilen başlıklarda tercih edilir; yalnızca C++'a özgü başlıklar için `.hpp` bunu açıkça belirtir. Çok dosyalı C++ projelerini Bölüm 30 sonrasında, sınıflar bağlamında yazacağız.

Standart kütüphane başlıkları

Standart kütüphane başlıkları `.h` almaz: `<iostream>`, `<vector>`, `<string>`, `<algorithm>`. C'den miras kalanlar `c` önekiyle yazılır: `<cstdio>`, `<cstdlib>`, `<cmath>`, `<cstring>`. Önişlemcinin `#include <iostream>` satırıyla ne kadar kod getirdiğini görmek için `-E` seçeneğini kullanalım:

```
TERMİNAL
$ g++ -std=c++17 -E -P baslik.cpp | wc -l
24409
```

Yalnızca `#include <iostream>` ve boş bir `main` içeren dosya, önişleme sonunda yaklaşık yirmi dört bin satıra dönüşür (sayı g++ sürümüne göre değişir). C'deki `<stdio.h>` ile kıyaslandığında bu büyük bir farktır ve C++ derlemelerinin neden daha yavaş olduğunun nedenlerinden biridir. Bu ağırlığın çoğu şablon kodudur.

21.7 Standartlar ve Kitabın Kullandığı Sürüm

C++ standartlarının tarihi ve getirdikleri yenilikler Bölüm 2'deki tabloda verildi. Burada yalnızca uygulamayla ilgili kısmı hatırlayalım: bir derleyici bu standartların hepsini aynı anda bilir ve hangisine göre derleyeceğini `-std=` seçeneğinden öğrenir.

Seçenek	Anlamı
`-std=c++11`	C++11 kuralları
`-std=c++14`	C++14 kuralları
`-std=c++17`	C++17 kuralları (kitabın standardı)
`-std=c++20`	C++20 kuralları
`-std=c++23`	C++23 kuralları (yeni derleyicilerde)

`-std` verilmezse derleyicinin varsayılan standardı kullanılır. Varsayılan, derleyici sürümüne göre değişir; bu yüzden kitapta her zaman açıkça yazıyoruz. Programın hangi standartla derlendiğini önişlemcinin tanımladığı `__cplusplus` adıyla öğrenebiliriz:

`surum.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "Standart: " << __cplusplus << '\n';
}
```

```
ÇIKTI
Standart: 201703
```

Bu değer yıl ve aydır: `201703`, 2017 yılı anlamına gelir. C++11 için `201103`, C++20 için `202002` değeri görürüz. Aynı programı `-std=c++11` ile derlersek `201103` yazar. Bu adı hem C hem C++'a ortak yazılan başlıklarda koşullu derleme için de kullanırız (Bölüm 15):

```
#ifdef __cplusplus
extern "C" {
#endif

void baslat(void);        // C ile yazılmış fonksiyonun bildirimi

#ifdef __cplusplus
}
#endif
```

Bu kalıp C++'tan görüldüğünde `extern "C"` bloğunu açar, C'den görüldüğünde ise onu yok sayar.

Neden C++17?

Kitabın C++ kısmı C++17 standardına göre yazılmıştır. Bunun üç nedeni var:

1. C++17 hâlâ en yaygın desteklenen modern sürümdür. Güncel `g++`, `clang++` ve Visual Studio derleyicilerinin hepsi onu tam destekler.
2. C++11'in getirdiği "modern C++" üslubunun (`auto`, aralık tabanlı `for`, akıllı işaretçiler, lambda) tamamı C++17'de bulunur; C++17 bunlara `if` içinde ilk değer, `optional`, `string_view` gibi eklemeler yapar.
3. C++20 ve C++23 özellikleri (kavramlar, aralıklar, `std::format`, `std::print`) her derleyicide eşit olgunlukta değildir.

Daha yeni standartların öne çıkan özelliklerinden ilgili yerlerde kısaca söz edeceğiz; bunları göstereceğimiz kodların üstünde hangi standardı gerektirdiğini belirteceğiz.

> **İpucu · Derleyici hangi standardı varsayıyor?**
>
> Güncel `g++` sürümleri varsayılan olarak GNU uzantılarıyla birlikte C++17'yi (`-std=gnu++17`) kullanır. `-std=c++17` yazmak GNU uzantılarını kapatır ve programı standarda daha sıkı bağlar. Hangi derleyici sürümünü kullandığımızı `g++ --version` ile öğrenebiliriz.

21.8 Üç Dilde: Aynı Program

Bu bölümde gördüğümüz farkları toplamak için aynı küçük programı üç dilde yazalım: iki sayı tutsun, toplamlarını ve ortalamalarını yazdırsın.

Üç Dilde · Toplam ve ortalama *C · C++ · C#*

```
#include <stdio.h>

int main(void) {
    int a = 7, b = 4;
    printf("Toplam: %d\n", a + b);
```

```
    printf("Ortalama: %.1f\n", (a + b) / 2.0);
    return 0;
}
```

```cpp
#include <iostream>

int main() {
    int a = 7, b = 4;
    std::cout << "Toplam: " << a + b << '\n';
    std::cout << "Ortalama: " << (a + b) / 2.0 << '\n';
    return 0;
}
```

```
int a = 7, b = 4;
Console.WriteLine("Toplam: " + (a + b));
Console.WriteLine("Ortalama: " + (a + b) / 2.0);
```

C'de tip ile biçim belirteci elle eşleştirilir (`%d`, `%.1f`). C++'ta `cout` tipe göre kendisi karar verir, ama ondalık basamak sayısını ayarlamak için `<iomanip>` gerekir (Bölüm 23). C#'ta `+` ile metin birleştirilir ve sayılar otomatik olarak metne çevrilir; bu satırlar için `using` gerekmez, çünkü `System` ad alanı kendiliğinden eklenir (Bölüm 41).

Üç programın çıktısı da `Toplam: 11` ve `Ortalama: 5.5` satırlarıdır; fark hesapta değil, biçimlendirme araçlarındadır.

Bölümün ana farklarını bir tabloda toplayabiliriz:

Üç Dilde · C, C++ ve C# karşılaştırması *C · C++ · C#*

Özellik	C	C++	C#
İlk sürüm	1972	1985	2002
Çalışma biçimi	Makine koduna derlenir	Makine koduna derlenir	Ara dile (IL) derlenir, .NET çalıştırır
Dosya uzantısı	`.c`, `.h`	`.cpp`, `.hpp`	`.cs`
Ekrana yazma	`printf`	`std::cout <<`	`Console.WriteLine`
Ad çakışmasına çare	Ad önekleri	`namespace`	`namespace`
Veri gizleme	Yok	`private`	`private`
Bellek yönetimi	Elle	Elle, RAII, akıllı işaretçi	Çöp toplayıcı
Şablon / genel kod	Makro, `void *`	`template`	`Generics`
Hata bildirimi	Dönüş kodu	İstisna	İstisna

Tablodan çıkan sonuç şudur: C++, C'nin sorunlarına **derleme zamanı** araçlarıyla yanıt verir (ad alanı, sınıf, şablon) ve bellek yönetimini programcıdan almaz, ona daha iyi araçlar verir. C# ise bellek yönetimini çalışma zamanına devreder. Bu tercihin sonuçlarını C# kısmında göreceğiz.

Bölüm Özeti

- C++, C'nin büyük programlarda yaşadığı sorunlara (ad çakışması, veri korumasızlığı, elle kaynak yönetimi, tipten bağımsız kodun zorluğu) yanıt vermek için doğdu.
- C++ çok paradigmalı bir dildir: yordamsal, nesne yönelimli, genel ve fonksiyonel yaklaşımlar aynı programda birlikte kullanılabilir.
- Sıfır maliyetli soyutlama ilkesine göre kullanmadığımız bir özelliğin bedelini ödemeyiz ve kullandığımız soyutlama, elle yazılan kodu geçmeyecek kadar verimlidir.
- C++ güvenlik ağını varsayılan olarak kurmaz; güvenli seçenekler (`.at()` , akıllı işaretçi) ayrıca kullanılır.
- C++ büyük ölçüde C ile uyumludur, ama bir üst küme değildir: `void *` örtük dönüştürülmez, boş parantez "parametre yok" demektir, C++ anahtar sözcükleri C'de serbest adlardır ve karakter sabiti `char` tipindedir.
- C kütüphanelerini C++ içinden kullanırken `extern "C"` ile adların C kurallarıyla bağlanması sağlanır; C standart başlıkları `<cstdio>` gibi `c` önekiyle çağrılır.
- İlk C++ programı `#include <iostream>` , `int main()` ve `std::cout << ... << '\n';` ile yazılır. `<<` ekleme operatörüdür ve zincirlenebilir.
- `cout` tipe göre biçimlendirme yaptığı için `printf` 'teki biçim belirteci hatası oluşmaz.
- `std::endl` satır sonu yazar ve tamponu boşaltır; sıradan satır sonlarında `'\n'` tercih edilir.
- Standart kütüphane `std` ad alanındadır. `using namespace std;` yazımı kısaltır, ama ad çakışması riski taşır; başlık dosyalarında kullanılmaz.
- C++ programı `g++ -std=c++17 -Wall -Wextra dosya.cpp -o dosya` ile derlenir; kaynak dosya uzantısı `.cpp` , başlık uzantısı `.hpp` olarak seçilir.
- Kitabın standardı C++17'dir; `__cplusplus` değeri hangi standartla derlendiğini gösterir.

Alıştırmalar

1. Aşağıdakilerden hangisi C++'ın doğuş nedenlerinden biri **değildir**?

   A) Ad çakışmasını önlemek
   B) Veriyi ve işlemleri bir arada tutup korumak
   C) Programları makine koduna derlemek yerine yorumlamak
   D) Kaynakların otomatik geri verilmesini sağlamak
   E) Tipten bağımsız kod yazmayı kolaylaştırmak

2. "Kullanmadığın şeyin bedelini ödemezsin" ilkesine en iyi uyan durum hangisidir?

   A) Her programın çöp toplayıcıyla çalışması
   B) `virtual` fonksiyon içermeyen bir sınıfta sanal tablo işaretçisinin bulunmaması
   C) Her dizi erişiminde sınır denetimi yapılması
   D) Bütün sınıfların ortak bir ana sınıftan türemesi
   E) Her programın ek bir çalışma ortamı gerektirmesi

3. Aşağıdaki C kodu C++ derleyicisinde neden hata verir? Nasıl düzeltilir?

```
double *dizi = malloc(10 * sizeof(double));
```

4. Aşağıdaki iki programdan ilkini C, ikincisini C++ olarak derleyip çalıştırırsak çıktıları ne olur? Neden farklıdır?

```c
#include <stdio.h>
int main(void) {
    printf("%zu\n", sizeof('x'));
    return 0;
}
```

```cpp
#include <iostream>
int main() {
    std::cout << sizeof('x') << '\n';
}
```

5. C'de geçerli olan `void islem();` bildirimi ve `islem(3);` çağrısı C++'ta ne olur?

   A) Aynen derlenir
   B) Bildirim hata verir
   C) Çağrı hata verir, çünkü boş parantez parametre yok demektir
   D) Yalnızca uyarı verir
   E) Çalışma zamanında hata verir

6. Aşağıdaki program neden derlenmez? Bulup düzeltin.

```cpp
#include <iostream>

int main() {
    cout << "Merhaba" << '\n';
    return 0;
}
```

7. Bu programın çıktısı nedir?

```cpp
#include <iostream>

int main() {
    int x = 6, y = 4;
    std::cout << x + y << ' ' << x * y << ' ' << (x > y) << '\n';
    return 0;
}
```

8. `std::endl` ile `'\n'` arasındaki fark nedir? Çok sayıda satır yazan bir döngüde hangisi tercih edilir?
9. Aşağıdakilerden hangisi doğrudur?

   A) `using namespace std;` bir önişlemci komutudur ve `;` ile bitmez
   B) `std::` öneki yalnızca `cout` için gerekir
   C) `using namespace std;` başlık dosyalarında rahatça kullanılabilir
   D) `std` standart kütüphanenin ad alanıdır
   E) `std::cout` yerine `cout` yazmak her zaman hata verir

10. Aşağıdaki programdaki hatayı bulun:

```cpp
#include <algorithm>
#include <iostream>
using namespace std;

int count = 0;

int main() {
    count = 5;
    cout << count << '\n';
    return 0;
}
```

Bu hatayı üç farklı biçimde giderin.

11. `g++ -std=c++17 -Wall -Wextra program.cpp -o program` komutundaki her seçeneğin görevini yazın.
12. `__cplusplus` değeri `201703` olan bir programın hangi standartla derlendiğini söyleyin. Bu değer `201103` olsaydı ne anlama gelirdi?
13. Aşağıdakilerden hangisi C++ kaynak dosyası için yaygın uzantı **değildir**?

    A) `.cpp`
    B) `.cc`
    C) `.cxx`
    D) `.cs`
    E) `.C`

14. Ekrana şu üç satırı yazan bir C++ programı yazın: bir kişinin adını, yaşını ve boyunu (`Elif`, `20`, `1.68`). Her değer ayrı değişkende tutulsun.
15. İki tamsayıyı değişkenlerde tutup toplam, fark, çarpım ve bölümlerini (bölüm gerçek sayı olarak) yazdıran bir C++ programı yazın.
16. `<cmath>` başlığını kullanarak yarıçapı 2.5 olan dairenin alanını (`3.14159` ya da `std::acos(-1.0)` ile) hesaplayıp yazdıran bir C++ programı yazın.

BÖLÜM 22

Tipler, Değişkenler ve Operatörler: C++'ın Getirdikleri

BU BÖLÜMDE

- C'den C++'a geçen tip, değişken ve operatör bilgisinin neyin aynı, neyin farklı olduğunu ayırt etmeyi
- `bool` tipini, `auto` ile tip çıkarımını ve süslü parantezli ilk değer vermenin daraltma (narrowing) korumasını kullanmayı
- `const` ile `constexpr` arasındaki farkı ve `static_cast`, `const_cast`, `reinterpret_cast` dönüşümlerinin işini açıklamayı
- `nullptr` ile `NULL` arasındaki farkı ve `std::numeric_limits` ile sayısal sınırları sorgulamayı
- Referansın (`int& r = x;`) ne olduğunu ve işaretçiden nasıl ayrıldığını görmeyi
- Kapsamı, `namespace` tanımlamayı ve `::` operatörünü kullanmayı

Bölüm 5'te C'nin tiplerini, değişkenlerini ve operatörlerini ayrıntılı gördük. C++ bu temeli olduğu gibi devralır: `int`, `double`, `char`, aritmetik operatörler, tamsayı bölmesi, öncelik kuralları, taşma ve kayan nokta hassasiyeti C++'ta da aynı çalışır. Bu bölüm o konuları yeniden anlatmayacak; yalnızca C++'ın eklediği ya da değiştirdiği noktalara odaklanacak.

Bu eklemelerin ortak bir amacı var: C'de derleyicinin sessizce kabul ettiği hataları derleme zamanında yakalamak. `bool` ayrı bir tip olur, `{}` veri kaybını yasaklar, `static_cast` dönüşümün niyetini görünür kılar, `nullptr` işaretçi ile sayıyı karıştırmaz. Bölümün sonunda küçük ama modern bir C++ programı yazacak, C# ile ortak noktaları da göreceğiz. Örneklerde `std::` önekini açıkça yazıyoruz; `using namespace std;` tartışmasına Bölüm 21'de değinmiştik, bu bölümün sonunda ad alanlarını ayrıntılı ele alacağız.

22.1 C'den Gelenler: Hızlı Bakış

Aşağıdaki tablo, Bölüm 5'teki konuların C++'taki durumunu özetler:

Konu	C++'ta durum
`char`, `short`, `int`, `long`, `long long`, `float`, `double`	Aynı tipler; boyutlar yine platforma bağlı, `sizeof` aynı
`signed` / `unsigned`	Aynı; işaretli-işaretsiz karşılaştırma tuzağı da aynı
Aritmetik, atama, `++` / `--`, bileşik atama	Aynı; tamsayı bölmesi sıfıra doğru keser (C++11'den beri standartta garanti)
Taşma	İşaretli taşma tanımsız, işaretsiz taşma tanımlı; aynı

Konu	C++'ta durum
Öncelik ve birleşme yönü	Aynı tablo (Ek B)
Başlık dosyaları	`<stdio.h>` yerine `<cstdio>`, `<math.h>` yerine `<cmath>`, `<limits.h>` yerine `<climits>`, `<stdint.h>` yerine `<cstdint>` yazılır
`printf` biçim belirteçleri	Çalışır, ama C++'ın kendi akış sistemi (`std::cout`) tip güvenlidir (Bölüm 23)

C başlık dosyalarının C++ karşılıkları adın başına `c` konup `.h` atılarak yazılır. Bu başlıklardaki adlar `std` ad alanında tanımlanır: `std::sqrt`, `std::int32_t`, `std::size_t`. Bölüm 5'te gördüğümüz `int32_t`, `uint8_t` gibi sabit genişlikli tipler de böyledir.

C ile C++ arasındaki küçük farklar

Aynı görünen bazı ifadeler iki dilde farklı davranır. Karakter sabiti bunların en bilinenidir. Bölüm 5'te C'de `'A'` ifadesinin `int` olduğunu, `sizeof('A')` değerinin 4 çıktığını görmüştük. C++'ta `'A'` gerçekten `char` tipindedir:

karakterSabiti.cpp

```cpp
#include <cstdint>
#include <iostream>

int main() {
    std::cout << sizeof('A') << ' ' << sizeof(char) << '\n';

    char harf = 'A';
    std::cout << harf << ' ' << harf + 1 << ' '
              << static_cast<char>(harf + 1) << '\n';

    std::uint8_t paket = 65;
    std::cout << paket << ' ' << +paket << '\n';
    return 0;
}
```

```
ÇIKTI
1 1
A 66 B
A 65
```

Üç satırdan üç ders çıkıyor:

- C++'ta `sizeof('A')` 1'dir; C'de 4'tü.
- `harf + 1` ifadesi yine bir `int`'tir (tamsayı yükseltmesi, Bölüm 5), ama `std::cout` tipe bakarak yazdığı için 66 görürüz. `%c` ya da `%d` seçmek zorunda değiliz; tipi dönüştürerek (`static_cast<char>`) karakter yazdırırız.
- `std::uint8_t` çoğu platformda `unsigned char`'ın takma adıdır. `std::cout` onu bir karakter sayar ve 65 yerine `A` yazar. Sayı olarak görmek için `+paket` (tekli artı, değeri `int`'e yükseltir) ya da `static_cast<int>(paket)` yazılır. Bu, C'den gelenlerin en sık karşılaştığı tuzaklardan biridir.

C'den C++'a taşınan kodda iki fark daha dikkat ister. C'de `void*` değeri herhangi bir işaretçiye örtük olarak atanabilirdi (`int *p = malloc(...)`); C++ bunu kabul etmez, açık dönüşüm ister. Bunun neden C++'ta nadiren gerektiğini Bölüm 29'da `new` ile göreceğiz. İkincisi, C'de bir `int` değeri bir `enum` değişkenine atanabilirdi; C++'ta bu da hatadır (`enum` konusunu Bölüm 14'te C için görmüştük).

22.2 bool: Gerçek Bir Tip

C'nin ilk standardında doğru/yanlış için ayrı bir tip yoktu; C99 `_Bool` ve `<stdbool.h>` ekledi (Bölüm 5). C++'ta ise `bool` baştan beri dilin kendi tipidir ve `true` ile `false` anahtar sözcüktür. Hiçbir başlık dosyası eklemek gerekmez.

boolTip.cpp

```cpp
#include <iostream>

int main() {
    bool yagmurVar{true};
    bool sicak = false;
    bool kopya = 42;
    std::cout << yagmurVar << ' ' << sicak << ' ' << kopya << '\n';

    std::cout << std::boolalpha;
    std::cout << yagmurVar << ' ' << sicak << '\n';
    std::cout << (3 > 2) << ' ' << (3 > 2 && 5 < 1) << '\n';
    std::cout << sizeof(bool) << ' ' << sizeof(3 > 2) << '\n';

    int stok{0};
    if (stok) {
        std::cout << "stok var\n";
    } else {
        std::cout << "stok yok\n";
    }
    return 0;
}
```

```
ÇIKTI
1 0 1
true false
true false
1 1
stok yok
```

Programdan çıkarılacak noktalar şunlar:

- `std::cout` varsayılan olarak `bool` değerini `1` ya da `0` yazar. `std::boolalpha` akış düzenleyicisi (manipulator) bunu `true` / `false` yazıma çevirir; ayrıntısı Bölüm 23'te.
- Sıfırdan farklı her sayı `bool`'a atanırken `true` (1) olur; sıfır `false` olur. C'deki koşul kuralı değişmedi: `if (stok)` sıfırı yanlış, diğer her şeyi doğru sayar.
- Karşılaştırma ve mantıksal operatörlerin sonucu C++'ta `bool` tipindedir. `sizeof(3 > 2)` burada 1 yazdı; C'de aynı ifade `int` olduğu için 4 verirdi. Gündelik kodda farkı hissetmeyiz, çünkü `bool` gerektiğinde `int`'e yükselir.

> **▲ Sık Yapılan Hata · bool'a artırma operatörü uygulanamaz**
>
> Eski C++ kodlarında `bool` bir değişkene `++` uygulandığını görebilirsiniz (değişken `true` olurdu). C++98'den beri bu kullanım kullanımdan kaldırılmıştı ve C++17'de tamamen kaldırıldı. `bool` için `++` ve `--` artık derleme hatasıdır. Doğru/yanlış değerini değiştirmek için doğrudan atama ya da `!` kullanılır: `acik = !acik;`.

22.3 İlk Değer Verme Biçimleri ve Daraltma Koruması

C'de bir değişkene ilk değer vermenin tek yolu `=` idi. C++ üç biçim sunar:

```
int a = 5;      // kopyalayarak ilk değer verme (C'deki gibi)
int b(5);       // parantezli ilk değer verme
int c{5};       // süslü parantezli ilk değer verme (C++11)
```

Üçü basit durumda aynı sonucu verir. Aralarındaki farkı gösteren özellik, süslü parantezin **daraltıcı dönüşümü** (narrowing conversion) yasaklamasıdır.

> **Tanım: Daraltıcı dönüşüm**
>
> Bir değerin, onu tam olarak saklayamayacak bir tipe çevrilmesidir. Örneğin `double`'dan `int`'e, geniş bir tamsayı tipinden daha dar bir tipe ya da işaretliden işaretsize dönüşüm, değer değişebileceği için daraltıcıdır. C++ bu dönüşümleri `=` ile sessizce kabul eder, ama `{}` ile ilk değer verirken hata sayar.

▸ daralma.cpp

```
#include <iostream>

int main() {
    int a = 5.9;
    int b{5.9};
    std::cout << a << ' ' << b << '\n';
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

daralma.cpp: In function 'int main()':
daralma.cpp:5:11: error: narrowing conversion of '5.9000000000000004e+0' from 'double' to
'int' [-Wnarrowing]
    5 |     int b{5.9};
      |           ^~~
```

`int a = 5.9;` satırı derlenir ve `a` 5 olur; ondalık kısmın sessizce kaybolması, Bölüm 5'te örtük dönüşümde gördüğümüz davranıştır. `int b{5.9};` ise derleyici tarafından reddedilir. Kayıp olmadığını derleyicinin görebildiği durumlar serbesttir:

İfade	Sonuç	Neden
`char c{65};`	Geçerli	65, `char`'a sığar (sabit)
`char c{300};`	Hata	300 `char`'a sığmaz
`double d{3};`	Geçerli	`int`'ten `double`'a kayıpsız
`unsigned u{-1};`	Hata	Negatif değer işaretsize sığmaz
`int n{x};` (`x` bir `double` değişken)	Hata	Değer çalışma zamanında bilinir, kayıp olabilir
`long long b{n};` (`n` bir `int` değişken)	Geçerli	Geniş tipe çevirme kayıpsızdır

> **Not · Derleyiciler ve daraltma**
>
> C++ standardına göre daraltıcı dönüşüm `{}` içinde hatalı programdır. gcc, kaynak sabit bir değerse (`5.9`, `300`) hata verir; değer bir değişkense (`int b{fiyat};`) varsayılan olarak yalnızca `-Wnarrowing` uyarısı yazar. clang

ikisinde de hata verir. gcc'de uyarıyı hataya çevirmek için `-pedantic-errors` ya da `-Werror=narrowing` seçeneği kullanılır. Hangi derleyiciyle çalışırsak çalışalım, kuralı "hata" olarak kabul etmeliyiz.

Boş süslü parantez: sıfır değer

Süslü parantezin ikinci yararı, boş bırakıldığında değişkeni **sıfır değerle** başlatmasıdır:

▸ baslatma.cpp

```cpp
#include <iostream>

int main() {
    int b = 5;
    int c(5);
    int d{5};
    int e{};
    double f{};
    bool g{};

    std::cout << std::boolalpha;
    std::cout << b << ' ' << c << ' ' << d << '\n';
    std::cout << e << ' ' << f << ' ' << g << '\n';
    return 0;
}
```

ÇIKTI

```
5 5 5
0 0 false
```

Bölüm 5'te `int sayi;` biçiminin çöp değer taşıdığını ve tanımsız davranışa yol açtığını görmüştük. C++'ta `int sayi{};` yazmak aynı kolaylıkta güvenli bir alternatiftir: sayısal tipler 0, `bool` `false`, işaretçiler `nullptr` olur. Bu kitapta değişkenlere bundan sonra mümkün olduğunca `{}` ile ilk değer vereceğiz.

◆ **Kritik Nokta · Parantezli biçime dikkat**

`int x();` satırı bir değişken tanımlamaz; "parametresiz, `int` döndüren `x` adlı bir fonksiyon bildirimi" olarak okunur (derleyicinin en çok şaşırttığı kalıplardandır). Boş parantez yerine boş süslü parantez kullanmak bu belirsizliği ortadan kaldırır: `int x{};`.

22.4 auto ile Tip Çıkarımı

Değişkenin ilk değeri tipini zaten söylüyorsa, tipi bir daha yazmak gereksizdir. C++11 ile gelen `auto`, derleyiciden tipi ilk değerden **çıkarmasını** ister.

Tanım: Tip çıkarımı (type deduction)

Derleyicinin bir değişkenin tipini, ona verilen ilk değerin tipine bakarak kendisinin belirlemesidir. `auto` ile yapılır. Tip yine derleme zamanında kesinleşir ve sonradan değişmez; `auto`, "tipsiz" ya da "her şeyi tutan" bir değişken demek değildir.

Tipin gerçekten beklediğimiz gibi çıkıp çıkmadığını sınamak için `<type_traits>` başlığındaki `std::is_same_v` yardımcısını ve bir ifadenin tipini veren `decltype`'ı kullanacağız. `std::is_same_v<A, B>`, iki tipin aynı olup olmadığını `bool` olarak verir:

autoOrnek.cpp

```cpp
#include <iostream>
#include <type_traits>

int main() {
    auto a{5};
    auto b{5.5};
    auto c = 'X';
    auto d = true;
    auto e = 5LL;
    auto f = 5 / 2.0;
    auto g = 'a' + 1;
    auto h = "metin";

    std::cout << sizeof(a) << ' ' << sizeof(b) << ' ' << sizeof(c) << ' '
              << sizeof(d) << ' ' << sizeof(e) << '\n';
    std::cout << std::boolalpha;
    std::cout << std::is_same_v<decltype(f), double> << ' '
              << std::is_same_v<decltype(g), int> << ' '
              << std::is_same_v<decltype(h), const char*> << '\n';

    const int sabit{3};
    auto kopya = sabit;
    kopya = 4;
    std::cout << kopya << ' '
              << std::is_same_v<decltype(kopya), int> << '\n';
    return 0;
}
```

ÇIKTI

```
4 8 1 1 8
true true true
4 true
```

`auto`, değeri alırken en üstteki `const` niteleyicisini (ve referansı) atar. Bu yüzden `sabit` bir `const int` olduğu hâlde `kopya` düz bir `int`'tir ve değeri değiştirilebilir. `"metin"` gibi bir metin sabiti ise `const char*` çıkar; `std::string` elde etmek için bunu açıkça yazmak gerekir (Bölüm 27). Ayrıca `auto` için ilk değer zorunludur: `auto x;` yazarsak derleyici tipi çıkaramaz.

autoHata.cpp

```cpp
int main() {
    auto x;
    x = 5;
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
autoHata.cpp: In function 'int main()':
autoHata.cpp:2:5: error: declaration of 'auto x' has no initializer
    2 |     auto x;
      |     ^~~~
```

▲ Sık Yapılan Hata · auto ile = {} birleşimi

C++17'den itibaren `auto x{5};` bir `int` verir. Ama `auto y = {5};` yazılırsa `y`, `int` değil `std::initializer_list<int>` tipinde olur. Bu kalıpla yanlışlıkla karşılaşmamak için tek değerli ilk değer verirken ya `auto x{5};` ya da `auto x = 5;` yazın.

`auto` ne zaman iyidir? `int sayac{0};` gibi kısa ve belli tiplerde tipi yazmak okunurluğu bozmaz, hatta `auto`'dan daha açıktır. `auto`'nun asıl yararı, tipi uzun ya da karmaşık olan değişkenlerde ortaya çıkar; Bölüm 35'te kapsayıcıların yineleyicileriyle `std::vector<int>::iterator` gibi adları `auto` ile nasıl kısalttığımızı göreceğiz. Genel ilke şudur: tip sağ taraftan hemen anlaşılıyorsa `auto` kullanın, tipin kendisi bilgi taşıyorsa (örneğin `double` ve `float` farkının önemli olduğu bir hesapta) tipi açıkça yazın.

Tarihçe · auto'nun iki hayatı

`auto` sözcüğü C'de de vardır, ama anlamı bambaşkadır: yerel değişkenin "otomatik depolama süresine" sahip olduğunu belirtir ve hiç kimse yazmaz, çünkü zaten varsayılandır (Bölüm 5'teki anahtar sözcük tablosunda gördük). C++11, bu işe yaramaz sözcüğü tip çıkarımı için yeniden kullandı. Eski anlam C++'ta kalmadı. C23 ise, C++'tan esinlenerek, `auto` ile tip çıkarımını C'ye de getirdi.

22.5 Sabitler: const ve constexpr

Bölüm 5'te sabitleri üç yolla tanımlamıştık: literal, `const` değişken ve `#define`. C++'ta `const` aynı işi görür ve `#define`'a tercih edilir; çünkü tipi, kapsamı ve adresi vardır, hata mesajlarında adıyla görünür. C++11 buna `constexpr` anahtar sözcüğünü ekledi.

Tanım: Sabit ifade (constant expression)

Değeri derleme sırasında hesaplanabilen ifadedir. Dizi boyutu, `static_assert` koşulu ve şablon argümanı gibi yerlerde derleme zamanında bilinen değer gerekir; bu yerlere yalnızca sabit ifadeler yazılabilir.

İki anahtar sözcüğün farkı şudur: `const` "bu değişkeni sonradan değiştiremem" der; ilk değeri çalışma zamanında hesaplanıyor olabilir. `constexpr` ise "bu değer derleme zamanında bilinmeli" der ve bunu derleyiciye denetletir.

constexprOrnek.cpp

```cpp
#include <iostream>

constexpr double PI{3.14159};

constexpr int kare(int n) {
    return n * n;
}

int main() {
    constexpr int KENAR{4};
    constexpr int ALAN{kare(KENAR)};
    int dizi[ALAN]{};
    static_assert(ALAN == 16, "alan 16 olmali");

    int girdi{5};
    const int sonuc{kare(girdi)};

    std::cout << "Çevre: " << 2 * PI * 2.0 << '\n';
```

```
    std::cout << "ALAN = " << ALAN << '\n';
    std::cout << "Dizi: " << sizeof(dizi) / sizeof(dizi[0]) << " eleman\n";
    std::cout << "sonuc = " << sonuc << '\n';
    return 0;
}
```

```
ÇIKTI
Çevre: 12.5664
ALAN = 16
Dizi: 16 eleman
sonuc = 25
```

Bu programda dört şey öğreniyoruz:

- `constexpr` bir fonksiyon (`kare`) sabit ifadelerle çağrılırsa sonuç derleme sırasında hesaplanır; `ALAN` bu yüzden dizi boyutu olabilir. Aynı fonksiyon `girdi` gibi değişken bir değerle çağrılırsa sıradan bir fonksiyon gibi çalışma zamanında çalışır.
- `static_assert(koşul, "mesaj")` koşulu derleme sırasında denetler; sağlanmıyorsa program derlenmez ve mesaj gösterilir. Çalışma zamanına hiçbir maliyeti yoktur.
- `sonuc` bir `const`'tır ama sabit ifade **değildir**: değeri `girdi`'ye bağlıdır. Onu dizi boyutu yapamayız. `constexpr` olsaydı derleyici hata verirdi.
- Başlangıçta `{}` kullandığımıza dikkat edelim; sabitlerde de aynı daraltma koruması geçerlidir.

Sabite değer atamaya kalkarsak Bölüm 5'teki hata burada da aynıdır:

▸ constHata.cpp

```
int main() {
    const int HAFTA_GUN{7};
    HAFTA_GUN = 8;
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

constHata.cpp: In function 'int main()':
constHata.cpp:3:15: error: assignment of read-only variable 'HAFTA_GUN'
    3 |     HAFTA_GUN = 8;
      |     ~~~~~~~~~~^~~
```

Pratik kural: değeri derleme zamanında bilinen sabitlere `constexpr`, yalnızca değiştirilmemesini istediğimiz çalışma zamanı değerlerine `const` yazılır.

> **Not · Global const ve dosya bağlantısı**
>
> C++'ta fonksiyon dışında tanımlanan bir `const` değişken varsayılan olarak yalnızca tanımlandığı dosyada görünür (iç bağlantı, internal linkage). C'de aynı değişken diğer dosyalardan erişilebilirdi. Çok dosyalı programlarda (Bölüm 15) bir sabiti başlık dosyasına koyarken bu fark işimizi kolaylaştırır: her dosya kendi kopyasını alır ve bağlayıcı (linker) çoklu tanım hatası vermez.

> *Tarihçe · constexpr ve C*
>
> `constexpr` C++11'de gelmiştir (C++14 ve sonrasında fonksiyon gövdelerindeki kısıtlamalar gevşetildi). C'de `const` değişken sabit ifade sayılmadığından dizi boyutu için `#define` kullanılırdı. C23 nesneler için `constexpr` belirtecini C'ye de ekledi; C++ kısmında gördüğümüz bu fikir, geç de olsa C'ye geri döndü.

Literal yazımları

Literal tipleri Bölüm 5'teki gibidir (`5`, `5L`, `5LL`, `5U`, `5.0f`). C++14 iki kolaylık ekledi: ikilik literal ve basamak ayracı.

literaller.cpp

```cpp
#include <iostream>

int main() {
    int ikilik{0b1010};
    int onaltilik{0x1F};
    int milyon{1'000'000};
    long long buyuk{9'000'000'000LL};
    double hiz{299'792.458};

    std::cout << ikilik << ' ' << onaltilik << '\n';
    std::cout << milyon << ' ' << buyuk << ' ' << hiz << '\n';
    return 0;
}
```

ÇIKTI

```
10 31
1000000 9000000000 299792
```

Tek tırnak (`'`) basamak ayracı yalnızca okunurluk içindir, değere etkisi yoktur. Son sütunda `299792` yazıldığına dikkat edelim: `std::cout` varsayılan olarak 6 anlamlı basamak yazar. Bunu denetlemeyi Bölüm 23'te öğreneceğiz.

22.6 C++ Tarzı Tip Dönüştürmeleri

C'deki dönüştürme `(tip)ifade` biçimindeydi (Bölüm 5). Bu biçim tek bir sözdizimiyle birbirinden çok farklı işler yapar: sayı çevirir, `const`'u kaldırır, işaretçiyi başka bir tipe çevirir. Hangisinin yapıldığı kodda görünmez ve arama motoruyla bulmak da zordur. C++ bu işleri ayırdı; her dönüşümün kendi adı vardır ve niyeti açıkça gösterir.

Dönüşüm	Ne yapar?	Tipik kullanım
`static_cast<T>(x)`	Derleyicinin bildiği, makul dönüşümler	`int` / `double` arası, `void*`'ten işaretçiye, kalıtımda yukarı-aşağı (Bölüm 31)
`const_cast<T>(x)`	`const` (ve `volatile`) niteleyicisini ekler ya da kaldırır	`const` doğru olmayan eski arayüzlerle çalışmak
`reinterpret_cast<T>(x)`	Bit desenini başka tipe yeniden yorumlar	İşaretçi-tamsayı çevirmesi, bayt düzeyinde inceleme
`dynamic_cast<T>(x)`	Çalışma zamanında denetimli dönüşüm	Çok biçimli (polymorphic) sınıflar (Bölüm 31)
`(T)x`	Yukarıdakilerden uygun olanını, yetmezse birkaçını birden dener	Eski kod; yeni kodda kullanmayın

Aşağıda ilk üçünü görüyoruz:

castTurleri.cpp

```cpp
#include <cstddef>
#include <iostream>

void eskiYazdir(char* metin) {
    std::cout << metin << '\n';
}

int main() {
    int toplam{145};
    int adet{2};
    double ortalama = static_cast<double>(toplam) / adet;
    int tam = static_cast<int>(3.99);
    char harf = static_cast<char>(65);
    std::cout << ortalama << ' ' << tam << ' ' << harf << '\n';

    const char* mesaj = "merhaba";
    eskiYazdir(const_cast<char*>(mesaj));

    int sayi{0x01020304};
    const unsigned char* bayt = reinterpret_cast<const unsigned char*>(&sayi);
    for (std::size_t i{0}; i < sizeof(sayi); ++i) {
        std::cout << static_cast<int>(bayt[i]) << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

ÇIKTI

```
72.5 3 A
merhaba
4 3 2 1
```

`static_cast` en sık kullanılan dönüşümdür; Bölüm 5'teki `(double)toplam / adet` yazımının karşılığıdır. Aynı kural geçerlidir: dönüşüm bölmeden önce, işlenenlerden birine uygulanmalıdır.

`const_cast` burada `eskiYazdir`'ın `char*` istemesine rağmen metni değiştirmediğini bildiğimiz için kullanıldı. Ama dikkat: aslında `const` olarak yaratılmış bir nesneyi bu yolla değiştirmek tanımsız davranıştır. `const_cast` yalnızca yazılımın, kodu elimizde olmayan yanlış işaretlenmiş bir arayüzle konuşması için vardır.

`reinterpret_cast` ile `int`'in bellekteki baytlarına baktık. Çıktı `4 3 2 1`: en düşük bayt önce geliyor. Bu, Bölüm 5'te anlattığımız küçük sonlu (little-endian) düzendir; büyük sonlu bir makinede `1 2 3 4` görürdük. Bu çıktı donanıma bağlı olduğundan taşınabilir kodda ona güvenilmez. `reinterpret_cast` yalnızca bu tür düşük düzey işlerde kullanılır; `unsigned char`, `char` ve `std::byte` dışındaki tipler üzerinden aynı belleği okumak çoğunlukla tanımsız davranıştır.

`static_cast` anlamsız dönüşümleri reddeder; bu, onu `(tip)` yazımından güvenli kılar:

castHata.cpp

```cpp
int main() {
    double d{2.5};
    int* p = static_cast<int*>(&d);
    return p != nullptr;
}
```

```
DERLEYİCİ ÇIKTISI

castHata.cpp: In function 'int main()':
castHata.cpp:3:14: error: invalid 'static_cast' from type 'double*' to type 'int*'
    3 |     int* p = static_cast<int*>(&d);
      |              ^~~~~~~~~~~~~~~~~~~~~
```

C tarzı `(int*)&d` aynı satırı sessizce kabul eder (içeride `reinterpret_cast` gibi davranır) ve `d`'nin baytlarını `int` sanarak okumaya izin verir.

> **◆ Kritik Nokta · Cast ihtiyacı bir uyarıdır**
>
> Bir cast yazmak, derleyiciye "ne yaptığımı biliyorum" demektir. Yeni kodda her `static_cast` düşünülmüş olmalı, `const_cast` ve `reinterpret_cast` ise istisna olmalıdır. Kodumuzda bunlardan çok varsa asıl sorun genellikle tasarımdadır. Aranması da kolaydır: `grep _cast` bütün dönüşümleri listeler.

İşaretli-işaretsiz karşılaştırma tuzağı (`-1 < 1u` yanlıştır) C++'ta da aynen geçerlidir. Burada `g++ -Wall`, `-Wsign-compare` uyarısını kendiliğinden açar; C'deki gcc'den farklı olarak ek bir seçenek gerekmez. Bu uyarının en sık tetiklendiği yer, Bölüm 26'da göreceğimiz `std::vector::size()` ile `int` sayaçların karşılaştırılmasıdır.

22.7 nullptr ve NULL

C'de boş işaretçi için `NULL` makrosunu kullanırdık (Bölüm 12). C++'ta `NULL` yalnızca 0 ya da `((void*)0)` gibi bir şeye açılır; yani aslında bir tamsayıdır. Bu, **fonksiyon aşırı yükleme** (overloading) ile birlikte sorun çıkarır. Aşırı yükleme, aynı adlı birden çok fonksiyonun parametre tipine göre ayrışmasıdır; Bölüm 25'te ayrıntılı göreceğiz. Şimdilik yalnızca derleyicinin çağrıyı hangi fonksiyona yönlendireceğini tipe bakarak seçtiğini bilmemiz yeter.

▸ nullHata.cpp

```cpp
#include <iostream>

void yaz(int n) {
    std::cout << "int: " << n << '\n';
}

void yaz(const char* s) {
    std::cout << "metin: " << (s ? s : "yok") << '\n';
}

int main() {
    yaz(NULL);
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI
nullHata.cpp: In function 'int main()':
nullHata.cpp:12:8: error: call of overloaded 'yaz(NULL)' is ambiguous
   12 |     yaz(NULL);
      |     ~~~^~~~~~
nullHata.cpp:3:6: note: candidate: 'void yaz(int)'
    3 | void yaz(int n) {
      |      ^~~
nullHata.cpp:7:6: note: candidate: 'void yaz(const char*)'
    7 | void yaz(const char* s) {
      |      ^~~
```

`NULL` hem bir tamsayı hem boş işaretçi gibi davrandığı için derleyici kararsız kalır. (Başka bir derleyici sessizce `yaz(int)`'i seçebilir; bu daha da kötüdür.) C++11 çözümü, yalnızca işaretçi olabilen bir sabit getirdi: **`nullptr`**.

▸ nullptrOrnek.cpp

```cpp
#include <iostream>

void yaz(int n) {
    std::cout << "int: " << n << '\n';
}

void yaz(const char* s) {
    std::cout << "metin: " << (s ? s : "yok") << '\n';
}

int main() {
    yaz(0);
    yaz(nullptr);

    int* p{nullptr};
    if (p == nullptr) {
        std::cout << "p boş\n";
    }
    return 0;
}
```

```
ÇIKTI
int: 0
metin: yok
p boş
```

`nullptr` kendine özgü bir tipe (`std::nullptr_t`) sahiptir; her işaretçi tipine dönüşebilir, ama tamsayıya dönüşmez. `int n = nullptr;` derleme hatasıdır. Kural basit: C++'ta boş işaretçi için daima `nullptr` yazılır, `NULL` ve `0` kullanılmaz.

> *Tarihçe · nullptr C'ye de geldi*
>
> `nullptr`, C++11'de (2011) eklendi. C23 de `nullptr` sabitini ve `nullptr_t` tipini C'ye ekledi.

22.8 Sayısal Sınırlar: numeric_limits

Bölüm 5'te tip sınırlarını `<limits.h>` ve `<float.h>` makrolarıyla (`INT_MAX`, `DBL_EPSILON` ...) öğrenmiştik. C++ aynı bilgiyi tek bir şablon sınıfıyla sunar: `<limits>` başlığındaki `std::numeric_limits<T>`. Köşeli

parantez içine tip yazılır (şablonları Bölüm 33'te göreceğiz; şimdilik "T tipi için" diye okumak yeterli) ve `::` ile sorulan özellik eklenir.

▸ sinirlar.cpp

```cpp
#include <cmath>
#include <iostream>
#include <limits>

int main() {
    std::cout << "int en küçük     : " << std::numeric_limits<int>::min() << '\n';
    std::cout << "int en büyük     : " << std::numeric_limits<int>::max() << '\n';
    std::cout << "unsigned en büyük: "
              << std::numeric_limits<unsigned>::max() << '\n';
    std::cout << "long long en büyük: "
              << std::numeric_limits<long long>::max() << '\n';
    std::cout << "int bit sayısı   : " << std::numeric_limits<int>::digits + 1
              << '\n';
    std::cout << "double en büyük  : " << std::numeric_limits<double>::max() << '\n';
    std::cout << "double en küçük  : " << std::numeric_limits<double>::lowest() << '\n';
    std::cout << "double min       : " << std::numeric_limits<double>::min() << '\n';
    std::cout << "double epsilon   : " << std::numeric_limits<double>::epsilon()
              << '\n';
    std::cout << "float epsilon    : " << std::numeric_limits<float>::epsilon()
              << '\n';

    double t{0.1 + 0.2};
    std::cout << std::boolalpha << (t == 0.3) << ' '
              << (std::abs(t - 0.3) < std::numeric_limits<double>::epsilon())
              << '\n';
    return 0;
}
```

ÇIKTI

```
int en küçük     : -2147483648
int en büyük     : 2147483647
unsigned en büyük: 4294967295
long long en büyük: 9223372036854775807
int bit sayısı   : 32
double en büyük  : 1.79769e+308
double en küçük  : -1.79769e+308
double min       : 2.22507e-308
double epsilon   : 2.22045e-16
float epsilon    : 1.19209e-07
false true
```

Dikkat edilecek bir nokta var: tamsayı tiplerinde `min()` en küçük değeri verir, ama kayan noktalı tiplerde **en küçük pozitif** normalleşmiş değeri verir. En negatif `double` değerini istiyorsak `lowest()` yazmalıyız. Bu tuzak, C'de `DBL_MIN`'in aynı biçimde pozitif olmasından gelir.

Son iki satır Bölüm 5'teki hassasiyet konusunun C++'a çevrilmiş hâlidir: `0.1 + 0.2 == 0.3` yanlıştır, ama iki sayının farkı `epsilon`'dan küçükse onları "yeterince yakın" sayabiliriz. `epsilon`, 1.0 ile ondan büyük en küçük `double` değeri arasındaki farktır. Değerler 1'den çok büyük ya da küçükse mutlak fark yerine göreceli fark kullanmak gerekir; bu bölümün kapsamı dışında.

Kütüphanede başka sorgular da vardır: `is_signed` tipin işaretli olup olmadığını, `digits10` kesin saklanan onluk basamak sayısını, `infinity()` sonsuz değerini verir.

22.9 Referanslara İlk Bakış

C'de bir fonksiyonun çağıran tarafın değişkenini değiştirmesi için adres geçirirdik: `takas(&a, &b)` (Bölüm 12). C++ aynı iş için daha sade bir araç sunar: **referans**.

> **Tanım: Referans (reference)**
> Var olan bir nesneye verilen ikinci bir addır. Referans tanımlandıktan sonra, onun üzerinden yapılan her işlem asıl nesneye yapılmış olur. Tip adından sonra `&` yazılarak tanımlanır: `int& r = x;` .

`referans.cpp`

```cpp
#include <iostream>

void artir(int& n) {
    ++n;
}

void takas(int& a, int& b) {
    int gecici{a};
    a = b;
    b = gecici;
}

int main() {
    int x{10};
    int& r{x};
    r = 20;
    std::cout << x << ' ' << r << '\n';

    int y{99};
    r = y;
    std::cout << x << ' ' << r << ' ' << y << '\n';
    std::cout << std::boolalpha << (&r == &x) << '\n';

    artir(x);
    std::cout << x << '\n';

    int a{3};
    int b{8};
    takas(a, b);
    std::cout << a << ' ' << b << '\n';
    return 0;
}
```

```
ÇIKTI
20 20
99 99 99
true
100
8 3
```

Çıktının üçüncü satırı referansın doğasını gösteriyor: `r = y;` ifadesi `r`'yi `y`'ye **bağlamaz**; `y`'nin değerini `r`'nin işaret ettiği nesneye (`x`) yazar. `&r == &x` karşılaştırması da `r` ve `x`'in aynı bellek adresinde olduğunu doğrular. `artir` ve `takas` fonksiyonlarında `&` işaretini yalnızca parametre tanımında yazdık; çağırırken `takas(a, b)` demek yetti, adres almak gerekmedi.

Referansın işaretçiden ayrıldığı kurallar şunlardır:

- Tanımlanırken ilk değer **zorunludur**.
- Bir kez bağlandıktan sonra başka nesneye bağlanamaz.
- "Boş referans" yoktur; her zaman geçerli bir nesneye bağlıdır.
- Kullanırken `*` ile içerik almak gerekmez; referans nesnenin kendisi gibi kullanılır.

referansHata.cpp

```cpp
int main() {
    int x{1};
    int& r;
    return x;
}
```

DERLEYİCİ ÇIKTISI

```
referansHata.cpp: In function 'int main()':
referansHata.cpp:3:10: error: 'r' declared as reference but not initialized
    3 |     int& r;
      |          ^
```

`const int& cr{x};` biçimindeki **sabit referans** (const reference), nesneyi okuma amaçlı gösterir, ama üzerinden değiştirmeye izin vermez. Sabit referanslar geçici değerlere de bağlanabilir (`const double& oran{0.18};`); bu, büyük nesneleri kopyalamadan fonksiyona geçirmenin temelidir. Parametre geçirme biçimlerinin karşılaştırmasını Bölüm 25'te, referans-işaretçi ayrımını ise Bölüm 28'de ele alacağız.

> **▲ Sık Yapılan Hata · Aynı simge, farklı anlam**
>
> `&` simgesi C++'ta bağlama göre iki işi yapar. Tipin yanında (`int& r`) referans tanımlar; bir ifadenin başında (`&x`) adres alır. `int& r{x};` ile `int* p{&x};` yan yana okunmalıdır: ilkinde `&` tipin parçası, ikincisinde adres operatörüdür.

22.10 Kapsam, Ad Alanları ve :: Operatörü

Kapsam kuralları C ile aynıdır (Bölüm 9): bir değişken, tanımlandığı bloğun sonuna kadar yaşar; iç bloktaki aynı adlı değişken dıştakini gölgeler. C++ iki şey ekler: gölgelenen global değişkene ulaşmak için `::` operatörü ve adların çakışmasını önleyen **ad alanları**.

> **Tanım: Ad alanı (namespace)**
>
> İçine tanım (değişken, fonksiyon, tip) koyabildiğimiz, adlandırılmış bir kapsamdır. Farklı ad alanlarındaki aynı adlar çakışmaz. İçindeki bir ada dışarıdan `AdAlani::ad` yazılarak ulaşılır.

> **Tanım: Kapsam çözümleme operatörü (scope resolution)**
>
> `::` operatörüdür. Solunda bir ad alanı (ya da sınıf) adı, sağında o kapsamdaki ad yazılır. Solu boşsa (`::ad`) global kapsama başvurur.

Büyük programlarda iki ayrı kütüphane aynı adlı fonksiyon tanımlayabilir. C'de bunun çözümü `kutuphane_islev` gibi önekli adlar uydurmaktı. C++'ta ad alanı bu işi dil düzeyinde yapar. Standart kütüphanenin tamamı da `std` adlı bir ad alanındadır; bu yüzden `std::cout`, `std::string` yazıyoruz.

adAlani.cpp

```cpp
#include <iostream>

int sayac{100};

namespace Metrik {
    constexpr double MIL_KM{1.609344};
    double cevir(double mil) {
        return mil * MIL_KM;
    }
}

namespace Imperyal {
    double cevir(double km) {
        return km / 1.609344;
    }
}

namespace Okul::Puan {
    constexpr int GECME{50};
}

namespace M = Metrik;

int main() {
    int sayac{1};
    std::cout << sayac << ' ' << ::sayac << '\n';

    std::cout << Metrik::cevir(10) << ' ' << Imperyal::cevir(10) << '\n';
    std::cout << M::MIL_KM << ' ' << Okul::Puan::GECME << '\n';

    using Metrik::cevir;
    std::cout << cevir(5) << '\n';

    {
        int sayac{7};
        std::cout << sayac << ' ' << ::sayac << '\n';
    }
    return 0;
}
```

ÇIKTI

```
1 100
16.0934 6.21371
1.60934 50
8.04672
7 100
```

Programda şu yapıları gördük:

- İki `cevir` fonksiyonu, farklı ad alanlarında olduğu için çakışmaz: `Metrik::cevir` mili kilometreye, `Imperyal::cevir` kilometreyi mile çevirir.
- `::sayac` yerel `sayac` tarafından gölgelenen global değişkene ulaşır. Aynı ad iç blokta bir kez daha gölgelenebilir.
- `namespace Okul::Puan { }` iç içe ad alanını tek satırda tanımlar (C++17). C++11'de bunu iki ayrı `namespace` bloğuyla yazardık.
- `namespace M = Metrik;` uzun bir ad alanına kısa takma ad verir.

- `using Metrik::cevir;` (**using bildirimi**) yalnızca belirtilen adı içeri alır; `cevir(5)` artık ön ek olmadan `Metrik::cevir`'i çağırır.

Aynı ad alanı programın başka yerinde yeniden açılıp içine yeni tanımlar eklenebilir; standart kütüphane başlık dosyaları `std`'yi bu yolla kendi içine yayar. Hiçbir ada sahip olmayan **isimsiz ad alanı** (`namespace { ... }`) ise içindeki tanımları yalnızca o dosyaya özel yapar; C'de bu iş `static` ile görülürdü (Bölüm 9, Bölüm 15).

using namespace tartışması

`using namespace std;` satırı `std`'nin tüm adlarını bir anda kapsama alır. Küçük programlarda yazımı kısaltır, ama büyük projede iki sakınca doğurur: hangi adın nereden geldiği görünmez olur ve ileride `std`'ye eklenen bir ad, bizim kendi adımızla çakışabilir. Bu yüzden bu kitapta `std::` önekini yazıyoruz.

> **▲ Sık Yapılan Hata · Başlık dosyasında using namespace yazmayın**
>
> Bir başlık dosyasındaki `using namespace` satırı, o başlığı içeren **her** dosyayı etkiler ve geri alınamaz. Başlık dosyalarında (Bölüm 15) ad alanı önekini daima açıkça yazın. `using` kullanacaksak bile en dar kapsamda, tercihen bir fonksiyonun içinde ve tek tek adlar için (`using std::cout;`) kullanalım.

22.11 Operatörler: C++'ta Değişenler

Bölüm 5'teki operatörlerin hepsi C++'ta aynıdır. Eklenen ya da değişen noktalar şunlardır:

- **Sonuç tipi.** `<`, `==`, `&&`, `!` gibi operatörler C++'ta `bool` üretir (C'de `int`). Koşullarda fark görmeyiz.
- **Alternatif adlar.** C++'ta `&&`, `||`, `!` yerine `and`, `or`, `not` yazılabilir (C'de bunlar için `<iso646.h>` gerekir). Ayrıca `bitand`, `xor` gibi karşılıkları vardır. Kitapta simgeleri kullanacağız.
- **Akış operatörleri.** `<<` ve `>>` operatörleri C'deki bit kaydırma işini sürdürür, ama `std::cout << x` ifadesinde akışa yazma anlamı da taşır. Bir operatörün farklı tiplerde farklı iş yapması **operatör aşırı yüklemesi** (operator overloading) sayesindedir; Bölüm 32'de kendi tiplerimiz için bunu yazacağız. Öncelik tablosu değişmediğinden `std::cout << a & b` ifadesi `(std::cout << a) & b` diye okunur ve derlenmez; bit ya da karşılaştırma işlemlerini akışa yazarken parantez içine alırız: `std::cout << (a & b)`.
- **`++` / `--` ve `bool`.** Daha önce gördük: `bool` üzerinde kullanılamaz.
- **Sıfıra bölme ve taşma.** Tamsayıyı sıfıra bölmek ve işaretli taşma C'deki gibi tanımsızdır.

`<cmath>` kütüphanesi C'nin `<math.h>` fonksiyonlarını `std` ad alanında sunar ve bir kolaylık ekler: fonksiyonlar aşırı yüklüdür. C'de `abs` yalnızca `int`, `fabs` `double` içindi; C++'ta `std::abs` tipe uyar. `^` yine üs alma değil, bit düzeyinde XOR'dur (Bölüm 16); üs için `std::pow` kullanılır.

▸ cmathOrnek.cpp

```cpp
#include <cmath>
#include <iostream>

int main() {
    std::cout << std::abs(-7) << ' ' << std::abs(-3.5) << '\n';
    std::cout << std::pow(2, 10) << ' ' << std::sqrt(2.0f) << '\n';
    std::cout << std::round(2.5) << ' ' << std::round(-2.5) << '\n';
    std::cout << std::floor(-3.2) << ' ' << std::ceil(-3.2) << '\n';
    int taban{2};
    int us{3};
```

```
    std::cout << std::fmod(7.5, 2.0) << ' ' << (taban ^ us) << '\n';
    return 0;
}
```

```
ÇIKTI
7 3.5
1024 1.41421
3 -3
-4 -3
1.5 1
```

`std::round` yarım değerleri sıfırdan uzağa yuvarlar (2.5 ⟶ 3, -2.5 ⟶ -3). `std::fmod` ondalıklı sayılar için `%` işlevini görür. Son satırda `2 ^ 3` işleminin (değişkenlerle yazdık, çünkü gcc sabitlerle yazılışı uyarır) 8 değil 1 verdiğini görüyoruz; bu, C'den gelenlerin sık yaptığı bir hatadır.

22.12 Hepsini Bir Araya Getirelim

Aşağıdaki program bu bölümde gördüğümüz araçları küçük bir fiyat hesabında birleştirir: ad alanı, `constexpr`, `auto`, `{}` ve `static_cast`. Çıktıyı biçimlendirmek için kullandığımız `std::fixed` ve `std::setprecision`, Bölüm 23'te ayrıntılı işlenecek.

kdvHesap.cpp

```
#include <iomanip>
#include <iostream>

namespace Fiyat {
    constexpr double KDV_ORANI{0.20};

    double kdvli(double net) {
        return net * (1.0 + KDV_ORANI);
    }
}

int main() {
    constexpr int ADET{3};
    const auto birimFiyat{249.90};

    auto net = birimFiyat * ADET;
    auto brut = Fiyat::kdvli(net);
    int tamLira = static_cast<int>(brut);
    bool yuksek{brut > 500};

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Net      : " << net << '\n';
    std::cout << "KDV'li   : " << brut << '\n';
    std::cout << "Tam lira : " << tamLira << '\n';
    std::cout << std::boolalpha << "Yüksek mi: " << yuksek << '\n';
    return 0;
}
```

```
ÇIKTI
Net      : 749.70
KDV'li   : 899.64
Tam lira : 899
Yüksek mi: true
```

`brut > 500` ifadesinin sonucu `bool` olduğu için `bool yuksek{...}` daraltma uyarısı vermeden çalışır.

Üç Dilde · Tip çıkarımı, sabit ve boş değer üç dilde *C · C++ · C#*

C'de tip çıkarımı C23'e kadar yoktu; C++ ve C# benzer araçlar sunar. Sabit ve boş değer kavramları üç dilde şöyle eşleşir:

Kavram	C	C++	C#
Doğru/yanlış tipi	`_Bool` / `bool` (`<stdbool.h>`)	`bool` (yerleşik)	`bool` (sayıya dönüşmez)
Tip çıkarımı	yok (C23: `auto`)	`auto`	`var`
Derleme zamanı sabiti	`#define` , C23 `constexpr`	`constexpr` , `const`	`const`
Çalışma zamanında bir kez atanan	`const`	`const`	`readonly` (alan)
Boş işaretçi/başvuru	`NULL`	`nullptr`	`null`
Daraltma	sessiz	`{}` ile hata	açık cast gerekir

```
auto n = 5;                    // C++: int
const auto oran{0.2};          // C++: double, değiştirilemez
constexpr int GUN{7};          // derleme zamanı sabiti
```

```
var n = 5;                     // C#: int, derleme zamanında çıkarılır
const int GUN = 7;             // derleme zamanı sabiti
// readonly yalnızca sınıf alanlarında: class A { readonly int x = 3; }
```

C#'ın `var` 'ı da `auto` gibi tipi derleme zamanında belirler (dinamik tip değildir) ve ilk değer ister. C#'ta `constexpr` yoktur: `const` her zaman derleme zamanı sabitidir, `readonly` ise çalışma zamanında (örneğin yapıcıda) bir kez atanan alandır. `if (x = 5)` gibi bir ifade C#'ta bool olmadığı için derlenmez; C ve C++'ta derlenir. Ayrıntılar Bölüm 42'de.

Üç Dilde · Dönüştürme üç dilde *C · C++ · C#*

C'nin `(tip)x` yazımı C#'ta da vardır; C++ ise bu işi dört ada böler.

```
double d{3.99};
int i = static_cast<int>(d);   // C++: i = 3
```

```
double d = 3.99;
int i = (int)d;                // C#: i = 3, açık cast zorunlu
int j = Convert.ToInt32(d);    // C#: 4, yuvarlar
```

C# `const_cast` ve `reinterpret_cast` gibi araçlara sahip değildir; güvenli kodda bellek yeniden yorumlanamaz. `Convert` sınıfı ve `Parse` / `TryParse` yöntemleri Bölüm 42'de ele alınacak.

Bölüm Özeti

- Temel tipler, aritmetik, tamsayı bölmesi, öncelik, taşma ve kayan nokta hassasiyeti C'dekiyle aynıdır (Bölüm 5). C başlıkları `<cstdio>` , `<cmath>` , `<cstdint>` gibi `c` önekiyle yazılır ve adlar `std::` altındadır.
- C++'ta `'A'` bir `char` 'tır (`sizeof` 1); C'de `int` 'ti. `std::uint8_t` değerini `cout` karakter olarak yazar; sayı için `+x` ya da `static_cast<int>` gerekir.

- `bool` yerleşik bir tiptir; `true` ve `false` anahtar sözcüktür. Karşılaştırmalar `bool` üretir. `cout` varsayılanda 1/0 yazar, `std::boolalpha` ile `true`/`false` yazar.
- Üç ilk değer verme biçimi vardır: `=`, `()` ve `{}`. Süslü parantez daraltıcı dönüşümü (`int b{5.9};`) engeller; boş `{}` değişkeni sıfır değerle başlatır.
- `auto` tipi ilk değerden çıkarır; ilk değer zorunludur. `auto` üstteki `const` ve referansı atar. `auto x{5}` `int`, `auto y = {5}` `initializer_list<int>` verir.
- `const` değişmezliği, `constexpr` derleme zamanında bilinmeyi garanti eder. Yalnızca sabit ifadeler dizi boyutu ya da `static_assert` koşulu olabilir.
- C++14 ikilik literal (`0b1010`) ve basamak ayracı (`1'000'000`) ekledi.
- `static_cast` makul dönüşümler, `const_cast` `const` kaldırma, `reinterpret_cast` bit deseni yeniden yorumlama içindir; `dynamic_cast` Bölüm 31'dedir. Yeni kodda `(tip)x` yazılmaz.
- `nullptr` boş işaretçi için tek doğru yazımdır; `NULL` aşırı yüklemede belirsizlik yaratır. `nullptr` tamsayıya dönüşmez.
- `std::numeric_limits<T>` tip sınırlarını verir: `min()`, `max()`, `lowest()`, `epsilon()`. Kayan noktalıda `min()` en küçük pozitif değerdir.
- Referans (`int& r = x;`) var olan nesnenin ikinci adıdır: ilk değer zorunlu, yeniden bağlanamaz, boş olamaz. `r = y;` yeniden bağlama değil, değer atamadır.
- `::` kapsam çözümleme operatörüdür; `::ad` global adı, `Ad::ad` ad alanındaki adı gösterir. `namespace` çakışmayı önler; `namespace A::B` (C++17) iç içe tanımlar.
- Başlık dosyalarında `using namespace` yazılmaz.
- `<cmath>` fonksiyonları aşırı yüklüdür (`std::abs`); `^` üs değil XOR'dur, üs için `std::pow` kullanılır.

Alıştırmalar

1. `std::cout << sizeof('A');` ifadesi C++'ta ne yazar?

   A) 1
   B) 2
   C) 4
   D) 8
   E) Derlenmez

2. Aşağıdaki satırlardan hangisi derleme hatası verir (sabit değerler için)?

   A) `int a = 7.5;`
   B) `int b{7};`
   C) `int c{7.5};`
   D) `double d{7};`
   E) `char e{65};`

3. `auto y = {5};` ifadesinde `y`'nin tipi nedir (C++17)?

   A) `int`
   B) `unsigned`
   C) `bool`

D) `std::initializer_list<int>`
E) Derleme hatası

4. `const` ile `constexpr` arasındaki temel fark hangisidir?

A) `constexpr` değer değiştirmeye izin verir
B) `constexpr` değerin derleme zamanında bilinmesini zorunlu kılar
C) `const` yalnızca fonksiyonlarda kullanılır
D) İkisi tamamen aynıdır
E) `const` global olamaz

5. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>
int main() {
    bool a{true}, b{};
    std::cout << a << b << (a && b) << std::boolalpha << (a || b);
    return 0;
}
```

6. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>
int main() {
    int x{10};
    int& r{x};
    int y{4};
    r = y;
    y = 6;
    std::cout << x << ' ' << r << ' ' << y;
    return 0;
}
```

7. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>
int n{1};
int main() {
    int n{2};
    {
        int n{3};
        std::cout << n << ::n;
    }
    std::cout << n;
    return 0;
}
```

8. Aşağıdaki kodda hata nerededir ve nasıl düzeltilir?

```
const int N{5};
int dizi[N];
int* p = NULL;
auto q;
q = 3;
```

9. Hangisi `static_cast` ile **yapılamaz**?

A) `int`'ten `double`'a dönüşüm
B) `double*`'dan `int*`'a dönüşüm
C) `double`'dan `int`'e dönüşüm
D) `int`'ten `char`'a dönüşüm
E) `void*`'dan `int*`'a dönüşüm

10. `std::numeric_limits<double>::min()` neyi verir?

    A) En negatif `double` değerini
    B) En büyük `double` değerini
    C) Sıfırı
    D) En küçük pozitif normal `double` değerini
    E) Derleme hatası verir

11. Aşağıdaki kodun çıktısı nedir?

```
#include <cstdint>
#include <iostream>
int main() {
    std::uint8_t v = 66;
    std::cout << v << ' ' << +v << ' ' << static_cast<int>(v);
    return 0;
}
```

12. Aşağıdaki kodun çıktısı nedir?

```
#include <iostream>
namespace A { int f() { return 1; } }
namespace B { int f() { return 2; } }
int main() {
    using A::f;
    std::cout << f() + B::f();
    return 0;
}
```

13. Kısa program: Yarıçapı `double` bir değişkende tutulan dairenin alanını hesaplayan bir program yazın. `PI` değerini `constexpr` olarak tanımlayın, sonucu `auto` ile alın ve tamsayıya `static_cast` ile çevrilmiş değerini de yazdırın.
14. Kısa program: `int` türünün en küçük ve en büyük değerlerini `std::numeric_limits` ile yazdıran, ardından en büyük değere 1 eklemeyi `long long` üzerinden yaparak taşmayı önleyen bir program yazın.
15. Kısa program: `Geometri` adlı bir ad alanı içinde `alan(double kenar)` (kare alanı) ve `cevre(double kenar)` fonksiyonlarını tanımlayın. `main`'de bunları `using Geometri::alan;` ve ad alanı öneki ile çağırarak 3.5 kenarlı kare için sonuçları yazdırın.
16. Kısa program: İki `double` değişkeni referans parametreli bir `takas` fonksiyonuyla yer değiştiren program yazın. Ayrıca `0.1 + 0.2` ile `0.3` değerlerinin `epsilon` ile karşılaştırmasını yapıp sonucu `true` / `false` olarak yazdırın.

BÖLÜM 23

iostream ile Girdi ve Çıktı

BU BÖLÜMDE

- Akış (stream) kavramını ve `cout`, `cin`, `cerr`, `clog` nesnelerinin işini
- `<<` ve `>>` işleçlerini zincirlemeyi ve `cout` içinde işleç önceliği tuzağından kaçınmayı
- `cin >>` ile boşlukla ayrılmış değerler okumayı; `getline` ve `cin.ignore` ile satır okumayı
- `<iomanip>` ile genişlik, hizalama, dolgu ve ondalık basamak sayısını ayarlamayı
- Hatalı girdide akışın nasıl kilitlendiğini ve `fail`, `clear`, `ignore` ile nasıl onarıldığını
- Normal çıktıyı hata iletilerinden `cerr` ile ayırmayı; `printf` ile akışları karşılaştırmayı

Bölüm 6'da C'nin `printf` ve `scanf` işlevleriyle klavyeden okumayı ve biçimli yazmayı gördük. Bölüm 21'de ise C++'ın `std::cout` ile yazdığını, `'\n'` ile `std::endl` arasındaki farkı ve `cout`'un değerin tipine bakarak ne yapacağını kendisinin seçtiğini öğrendik. Bu bölüm o ilk görünümü tamamlar: okuma, ayrıntılı biçimlendirme ve hatalı girdiyle başa çıkma.

C'deki duruma göre iki önemli fark var. Birincisi, biçim belirteci (`%d`, `%lf`) ve `&` işleci yoktur; tipi derleyici bilir. İkincisi, hatalı girdi durumunda `scanf`'in dönüş değerini denetlemek yerine akışın kendi durum bilgisini sorgularız. Bölümün sonunda tablolu bir fatura yazdıran, hatalı girdiyi onaran küçük bir program kuracağız. Örneklerde Bölüm 21'de tartıştığımız gibi `using namespace std;` kullanıyoruz.

23.1 Akış Kavramı

C++'ta ekrana yazmak, klavyeden okumak, ileride (Bölüm 37) dosyaya yazmak ya da dosyadan okumak aynı fikre dayanır: **akış** (stream).

> **Tanım: Akış (stream)**
> Baytların bir yönde aktığı soyut bir borudur. Programın bir ucunda durur, çıkış akışına yazarız; veri öbür uçtan hedefe (ekran, dosya) gider. Giriş akışından okuruz; veri kaynaktan (klavye, dosya) gelir.

```
            cout   (karakter çıkış akışı)
program  ----------------------------->  ekran
            cerr, clog   (hata akışları)
program  ----------------------------->  ekran (ayrı akış)

            cin    (karakter giriş akışı)
program  <-----------------------------  klavye
```

Dört hazır nesne `<iostream>` başlığıyla gelir:

Nesne	Yönü	C'deki karşılığı	Özelliği
`cin`	Giriş	`stdin`	Klavyeden okur
`cout`	Çıkış	`stdout`	Normal çıktı; tamponludur
`cerr`	Çıkış	`stderr`	Hata iletileri; tamponsuzdur, her yazımda hemen boşalır
`clog`	Çıkış	`stderr`	Günlük (log) iletileri; tamponludur

`cout << değer` ifadesi değeri boruya iter; `cin >> değişken` ifadesi borudan alıp değişkene koyar. İşleçlerin okları da akışın yönünü gösterir. C'de `printf` bir işlevdi ve ekran kavramı işlevin içinde gizliydi; C++'ta ekran bir **nesnedir** ve aynı işleçlerle dosyalara, hatta bellekteki metinlere (`stringstream`) yazabiliriz. Bu birliktelik, bölümün asıl kazancıdır.

> *Tarihçe · İşleç yeniden kullanımı*
>
> `<<` ve `>>`, C'de bit kaydırma işleçleridir (Bölüm 16). Akış kütüphanesi, işleçlere yeni anlam vermeyi sağlayan işleç aşırı yüklemeden (Bölüm 32) yararlanır: solundaki işlenen bir akışsa "akışa yaz/oku", tamsayıysa "bit kaydır" anlamına gelir.

23.2 cout ile Yazdırma

Zincirleme

`cout << x` ifadesinin değeri yine `cout`'un kendisidir. Bu yüzden art arda yazılan `<<` işleçleri soldan sağa zincirlenir: `cout << a << b` önce `a`'yı yazar, elde kalan `cout` ile `b`'yi yazar.

zincir.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string ad = "Elif";
    int yas = 19;
    double boy = 1.68;
    bool kayitli = true;

    cout << "Merhaba " << ad << "!\n";
    cout << "Yaş: " << yas << ", boy: " << boy << " m\n";
    cout << "Gelecek yıl " << yas + 1 << " yaşında olacak.\n";
    cout << "Kayıtlı: " << kayitli << '\n';
    cout << 'A' << ' ' << 'A' + 1 << ' ' << static_cast<char>('A' + 1) << '\n';
    return 0;
}
```

ÇIKTI

```
Merhaba Elif!
Yaş: 19, boy: 1.68 m
Gelecek yıl 20 yaşında olacak.
Kayıtlı: 1
A 66 B
```

Bu çıktıdan üç şey öğreniyoruz:

- `yas + 1` gibi ifadeler doğrudan yazdırılabilir; işlem önce yapılır.
- `bool` varsayılan olarak `true` / `false` değil `1` / `0` olarak yazılır. Sözcük görmek için `boolalpha` manipülatörü gerekir (aşağıda).
- `'A' + 1` bir `int` ifadesidir (Bölüm 5'teki tip yükseltmesi), bu yüzden `66` yazar. Karakter görmek istiyorsak `static_cast<char>` ile dönüştürmeliyiz.

Kaçış dizileri (`\n`, `\t`, `\\`, `\"`) C'deki gibi çalışır (Bölüm 4 ve 6). Satır sonu için `'\n'`, nadiren gereken zorla boşaltma için `endl` ya da `flush` kullanılır (Bölüm 21).

> **Not · Neden soru, cin'den önce görünür?**
>
> `cout << "Yaş: ";` satırında satır sonu yoktur; tampon boşalmak için bir neden yok gibidir. Ama `cin` ile `cout` birbirine **bağlanmıştır** (tied): `cin`'den okumaya başlamadan hemen önce `cout` otomatik olarak boşaltılır. Bu yüzden soru her zaman cevap beklenmeden önce ekranda görünür. C'de benzer garantiyi çoğu zaman `fflush(stdout)` ile vermemiz gerekirdi.

cout içinde işleç önceliği

`<<` işleci, çarpma ve toplamadan **sonra**, ama karşılaştırma (`>`, `==`) ve mantıksal (`&&`, `||`) işleçlerden **önce** işlenir (öncelik tablosu Ek B'dedir). Toplama için parantez gerekmez, karşılaştırma için gerekir:

oncelik.cpp

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 3;
    cout << "Büyük mü: " << a > b << '\n';
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
oncelik.cpp: In function 'int main()':
oncelik.cpp:6:31: error: no match for 'operator>' (operand types are
'std::basic_ostream<char>' and 'int')
    6 |     cout << "Büyük mü: " << a > b << '\n';
      |     ~~~~~~~~~~~~~~~~~~~~~~~~~ ^ ~~~~~~~~~
```

Derleyici ifadeyi `(cout << "Büyük mü: " << a) > (b << '\n')` olarak okudu; soldaki bir akış, sağdaki ise `int` kaydırması. Hata mesajı bu yüzden `>` işlecinin bir akışla (`basic_ostream`) bir `int` arasında tanımlı olmadığını söylüyor. Çözüm parantezdir:

```
cout << "Büyük mü: " << (a > b) << '\n';   // 1
cout << "Toplam: " << a + b << '\n';        // parantez gerekmez
cout << "Kaydırma: " << (1 << 4) << '\n';  // bit kaydırma da parantezli
```

◆ Kritik Nokta · Karşılaştırma, mantık ve bit işlemleri parantez ister

`cout` içinde `==`, `<`, `&&`, `?:`, `&` ve `<<` (bit kaydırma anlamında) kullanıyorsak ifadeyi daima parantez içine alalım. `>` gibi işleçlerde derleyici hata verir; ama `cout << "x: " << a && b;` ifadesi hatasız derlenir ve `(cout << "x: " << a) && b` anlamına gelir: `b` hiç yazdırılmaz.

23.3 cin ile Okuma

Klavyeden okumak için `cin` ve **çıkarma işleci** (extraction operator) `>>` kullanılır:

```
cin >> değişken;
```

yasOku.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int yas = 0;
    cout << "Yaşınızı girin: ";
    cin >> yas;
    cout << "Gelecek yıl " << yas + 1 << " yaşında olacaksınız.\n";
    return 0;
}
```

ÇIKTI

```
Yaşınızı girin: 19
Gelecek yıl 20 yaşında olacaksınız.
```

C'deki `scanf("%d", &yas)` ile karşılaştıralım:

	C	C++
Okuma	`scanf("%d", &yas);`	`cin >> yas;`
Tür bilgisi	Biçim belirteci (`%d` , `%lf`)	Değişkenin tipi
Adres	`&` zorunlu	Gerekmez
Yanlış belirteç	Tanımsız davranış	Mümkün değil

`&` neden gerekmiyor? `cin >> yas` ifadesinde `>>` işlecinin `yas`'ı **referans** olarak aldığını ve doğrudan değiştirebildiğini Bölüm 25 ve 28'de göreceğiz. Şimdilik kural şudur: `cin >> değişken`. `cin` ayrıca `string` (bir sözcük), `char` (bir karakter), `double` ve `bool` gibi tipleri de aynı kalıpla okur.

Birden çok değer

`>>` da zincirlenir. Aşağıdaki program dört değeri, dört ayrı tipte okur:

uclu.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
```

```
    string ad;
    int yas = 0;
    double boy = 0.0;
    char grup = ' ';

    cout << "Ad, yaş, boy (m), kan grubu: ";
    cin >> ad >> yas >> boy >> grup;
    cout << ad << " " << yas << " " << boy << " m " << grup << '\n';
    return 0;
}
```

```
ÇIKTI
Ad, yaş, boy (m), kan grubu: Elif 19 1.78 A
Elif 19 1.78 m A
```

Kullanıcı dört değeri tek satıra yazsa da, dört ayrı satıra yazsa da (her birinin ardından Enter) sonuç aynıdır. Çünkü `>>` işleci üç adım izler:

1. Baştaki **boşluk karakterlerini** (boşluk, sekme, Enter) atlar.
2. Değişkenin tipine uyan karakterleri okur.
3. Uymayan ilk karakterde durur ve o karakteri akışta bırakır.

`char` okurken de baştaki boşluklar atlanır. C'de `scanf(" %c")` yazmayı gerektiren tuzak (Bölüm 6) burada yoktur: `cin >> harf` Enter'ı hiçbir zaman karakter diye okumaz. Bunun yan etkisi, boşluk karakterinin kendisini `>>` ile okuyamamamızdır; bunun için `cin.get()` kullanılır.

Ayraçlı girdi

`15/03/2005` gibi girdilerde ayracı C'de biçim metnine yazardık; C++'ta bir `char` değişkenine okuruz: `cin >> gun >> ayrac >> ay >> ayrac >> yil;` ifadesi `15/03/2005` girdisini üç sayıya ve iki `/` karakterine ayırır. `>>` boşluk atladığı için `15 / 03 / 2005` de çalışır. Baştaki sıfırı yeniden yazdırmak için `setfill` ve `setw` gerekir; birazdan göreceğiz.

Girdi tamponu

Klavyede yazdıklarımız programa tek tek gitmez.

> **Tanım: Tampon (buffer)**
>
> Giriş ve çıkış sırasında verinin geçici olarak bekletildiği bellek alanıdır. Veri tek tek değil, topluca aktarılır; bu hem hızlıdır hem de veriyi okuyanın ihtiyacından fazlasının bir yerde beklemesine yol açar.

İşletim sistemi Enter'a basılana kadar karakterleri bir **tamponda** biriktirir; Enter'a basınca satırın sonuna `'\n'` ekleyerek tamponu programa verir. `cin >>` bu tampondan ihtiyacı kadarını alır, geri kalan tamponda bekler:

```
Klavyede yazılan:       2 0 [Enter]
Tamponun içi:           '2'  '0'  '\n'

cin >> yas  (yas = 20)
Tamponda kalan:                   '\n'
```

`>>` sayıyı okuyup durur; Enter'ın bıraktığı `'\n'` tamponda kalır. Sonraki bir `>>` için sorun yoktur, çünkü `>>` baştaki boşlukları atlar. Ama sonraki okuma satır okuyan `getline` ise sorun çıkar. Bunu hemen göreceğiz (bu, C'deki `scanf` sonrası `fgets` tuzağının aynısıdır; Bölüm 6).

23.4 Satır Okumak: getline

`cin >> ad` ile bir `string` okurken `>>` ilk boşlukta durur. Kullanıcı `Ayşe Nur` yazarsa `ad` yalnızca `Ayşe` olur; `Nur` tamponda bekler ve sonraki bir `cin >> soyad` kullanıcıya sormadan onu alır.

Boşluk içeren metni bütün olarak almak için `getline(cin, değişken)` kullanılır. `getline`, Enter'a kadar olan her şeyi (boşluklar dahil) `string`'e koyar ve Enter karakterini tampondan alıp atar. `<string>` başlığındadır.

▸ tamAd.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string tamAd;
    cout << "Ad soyad: ";
    getline(cin, tamAd);
    cout << "Merhaba, " << tamAd << "!\n";
    return 0;
}
```

```
ÇIKTI
Ad soyad: Ayşe Nur Kaya
Merhaba, Ayşe Nur Kaya!
```

C'de aynı işi `fgets` yapardı (Bölüm 11); farkı, `getline`'ın tampon boyutu sormaması ve sondaki `'\n'`'i `string`'e koymamasıdır. `string` kendi uzunluğunu yönetir, taşma riski yoktur (Bölüm 27).

cin >> sonrası getline tuzağı

Şimdi iki okumayı birleştirelim: önce sayı, sonra satır.

▸ getlineTuzak.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int yas = 0;
    string ad = "(okunmadı)";

    cout << "Yaş: ";
    cin >> yas;
    cout << "Ad soyad: ";
    getline(cin, ad);
    cout << "yas = " << yas << ", ad = [" << ad << "]\n";
    return 0;
}
```

```
ÇIKTI
Yaş: 20
Ad soyad: yas = 20, ad = []
```

Kullanıcı `20` yazıp Enter'a bastı; program `Ad soyad:` sorusunu yazdı ve bir şey beklemeden bitti. İzleyelim: Enter'dan sonra tamponda `20` ve `'\n'` vardı. `cin >> yas` yalnızca `20`'yi aldı, `'\n'` kaldı. `getline` tampondan bir satır ister; ilk karakter zaten `'\n'` olduğundan boş bir satır okur ve hemen döner.

Çözüm, `getline`'dan önce tamponda kalan satır sonunu atmaktır. Bunu `cin.ignore` yapar:

▸ getlineCozum.cpp

```cpp
#include <iostream>
#include <limits>
#include <string>
using namespace std;

int main() {
    int yas = 0;
    string ad;

    cout << "Yaş: ";
    cin >> yas;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Ad soyad: ";
    getline(cin, ad);
    cout << "yas = " << yas << ", ad = [" << ad << "]\n";
    return 0;
}
```

```
ÇIKTI
Yaş: 20
Ad soyad: Ayşe Nur
yas = 20, ad = [Ayşe Nur]
```

`cin.ignore(n, ayrac)`, tampondan en çok `n` karakter atar; `ayrac` karakterini görürse onu da atıp durur. Yani satırın kalanını, `'\n'` dahil siler. `numeric_limits<streamsize>::max()` "mümkün olan en büyük sayı" demektir (`<limits>` başlığı gerekir; Bölüm 22'de `numeric_limits`'i görmüştük); pratikte "ne kadar olursa olsun satır sonuna kadar" anlamına gelir. Bu satırı bir kalıp olarak benimseyebiliriz.

> **▲ Sık Yapılan Hata · Parametresiz cin.ignore() yetmez**
>
> `cin.ignore();` yalnızca **bir** karakter atar. Kullanıcı sayıdan sonra yanlışlıkla bir boşluk bırakırsa (`20`) atılan boşluk olur, `'\n'` tamponda kalır ve `getline` yine boş satır okur. Uzun biçim her durumda çalışır.

Hangi okuma kalıbının ardından araya `ignore` koymamız gerektiğini şu tablo özetler:

Önce	Sonra	Tuzak var mı?
`cin >>`	`cin >>`	Hayır; `>>` baştaki boşlukları atlar
`getline`	`getline`	Hayır; `getline` satır sonunu kendisi tüketir
`getline`	`cin >>`	Hayır
`cin >>`	`getline`	**Evet**; araya `cin.ignore(...)` koyulur

◆ Kritik Nokta · cin >> ile getline'ı art arda kullanırken

Sayıdan hemen sonra satır okuyacaksak `cin >> sayi;` ile `getline(...)` arasına `cin.ignore(numeric_limits<streamsize>::max(), '\n');` koymayı alışkanlık edinelim. Bu satır, C'de `scanf` sonrasında `fgets` kullanırken yaşadığımız aynı tuzağın C++ karşılığıdır.

23.5 Biçimli Çıktı: iomanip

C'de `printf("%8.2f")` genişliği ve hassasiyeti biçim metninin içinde taşırdı (Bölüm 6). `cout` biçim metni kullanmaz; aynı ayarları akışa yazılan özel değerlerle yaparız. Bunlara **manipülatör** (manipulator) denir.

Tanım: Manipülatör (manipulator)

Akışa `<<` ile yazılan, ekrana bir şey yazmayan ama akışın biçim ayarını değiştiren özel değerdir. `endl` de bir manipülatördür. Parametre alanlar (`setw`, `setprecision`, `setfill`) `<iomanip>` başlığında, parametresizler (`fixed`, `left`, `right`, `boolalpha`) `<iostream>` içindedir.

Varsayılan hâl: 6 anlamlı basamak

Hiçbir şey ayarlamazsak `cout` bir ondalıklı sayıyı en çok **6 anlamlı basamakla** yazar. Anlamlı basamak, sayının soldan ilk sıfırdan farklı rakamından başlayarak sayılan rakamlardır; noktadan sonraki hane sayısı değildir. Gereksiz sıfırlar yazılmaz, çok büyük ya da çok küçük sayılar bilimsel gösterime geçer:

varsayilan.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << 3.14159265 << '\n';
    cout << 100.0 << '\n';
    cout << 2.50 << '\n';
    cout << 1.0 / 3 << '\n';
    cout << 123456.789 << '\n';
    cout << 1234567.0 << '\n';
    cout << 0.00001 << '\n';
    cout << 0.1 + 0.2 << '\n';
    return 0;
}
```

ÇIKTI

```
3.14159
100
2.5
0.333333
123457
1.23457e+06
1e-05
0.3
```

Çıktıdan çıkan sonuçlar şunlardır:

- `3.14159265` yedi basamaklı olduğu için 6 basamağa yuvarlanarak yazıldı; bellekteki değer değişmedi.

- `100.0` ve `2.50` için sondaki sıfırlar atıldı: `100` ve `2.5`. C'nin `%f` belirteci hep noktadan sonra altı hane yazardı; `cout` farklı çalışır.
- `123456.789` altı basamağa sığması için `123457` oldu, ondalık kısım kayboldu. `1234567.0` sığmadığı için `1.23457e+06` ($1{,}23457 \times 10^6$) biçimine geçti.
- `0.1 + 0.2` ikilik tabanda tam 0,3 etmez (Bölüm 5), ama 6 basamakla yazılınca `0.3` görünür. Gerçek değeri görmek için daha çok basamak istemeliyiz.

fixed ve setprecision

Basamak sayısını `setprecision(n)` ayarlar. Ne anlama geldiği `fixed` manipülatörünün açık olup olmamasına bağlıdır:

Ayar	`setprecision(n)` ne demek?	`3.14159265` için `n = 3`
`fixed` yok (varsayılan)	Toplam anlamlı basamak sayısı	`3.14`
`fixed` var	Noktadan sonraki hane sayısı	`3.142`

Para, ortalama, VKİ gibi "iki ondalık" isteyen yerlerde ikisini birlikte kullanırız:

▸ fixedOrnek.cpp

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double pi = 3.14159265;

    cout << setprecision(3) << pi << '\n';
    cout << fixed << setprecision(3) << pi << '\n';
    cout << pi * 1000 << '\n';
    cout << 100.0 << '\n';
    cout << defaultfloat << setprecision(6) << pi << '\n';
    cout << setprecision(17) << 0.1 + 0.2 << '\n';
    return 0;
}
```

```
ÇIKTI
3.14
3.142
3141.593
100.000
3.14159
0.30000000000000004
```

Satır satır inceleyelim:

1. `fixed` olmadan `setprecision(3)`: 3 anlamlı basamak, `3.14`.
2. `fixed` ile `setprecision(3)`: noktadan sonra 3 hane, `3.142`.
3. `fixed` hâlâ açık; `3141.59265` üç ondalıkla `3141.593` yazıldı.
4. `100.0` bile `100.000` yazıldı: `fixed` sayıyı kısaltmaz, gereken sıfırları ekler.
5. `defaultfloat` varsayılan biçime döner; `setprecision(6)` basamak sayısını eski hâline getirir.
6. 17 anlamlı basamak, `0.1 + 0.2` toplamının gerçek değerini gösterir: `0.30000000000000004`.

> ◆ **Kritik Nokta · Çoğu manipülatör kalıcıdır**
>
> `fixed`, `setprecision`, `left`, `right`, `setfill`, `boolalpha`, `hex` gibi manipülatörler bir kez yazıldığında biz değiştirene kadar sonraki bütün çıktıları etkiler. Tek istisna `setw`'dir: yalnızca bir sonraki değer için geçerlidir. `cout << fixed << setprecision(2);` satırını programın başında bir kez yazmak yeterlidir.

Kalıcı ayarları geri almak için ilgili manipülatörün karşıtını (`dec`, `defaultfloat`, `noboolalpha`) yazarız.

Yuvarlama ile kesme

`fixed` ve `setprecision` sayıyı gösterirken **yuvarlar**; `static_cast<int>` ise ondalık kısmı **keser** (Bölüm 22):

yuvarlama.cpp

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << fixed << setprecision(2);
    cout << 3.999 << '\n';
    cout << 2.675 << '\n';
    cout << 1.005 << '\n';
    cout << setprecision(0) << 2.7 << '\n';
    cout << static_cast<int>(2.7) << '\n';
    return 0;
}
```

```
ÇIKTI
4.00
2.67
1.00
3
2
```

`2.675` ve `1.005` için elle yuvarlama `2.68` ve `1.01` verirdi; ekranda `2.67` ve `1.00` görüyoruz. Sebep, bu sayıların ikilik tabanda tam saklanamamasıdır: `2.675` bellekte `2.67499999...` olarak durur ve `cout` gerçek değeri yuvarlar. Bu bir hata değil, kayan noktalı sayıların doğal sonucudur (Bölüm 5). C'nin `printf("%.2f")` ifadesi de aynı değerleri aynı biçimde yuvarlar (Bölüm 6).

setw, left, right

`setw(n)` (set width), bir sonraki değerin **en az** `n` karakterlik bir alana yazılmasını sağlar. Değer kısaysa kalan yer dolgu karakteriyle doldurulur. `left` ve `right` değerin alanın hangi yanına yaslanacağını belirler; varsayılan sağa yaslıdır. Köşeli parantezler boşlukları görmemizi sağlar:

setwOrnek.cpp

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << "[" << setw(6) << 42 << "]\n";
    cout << "[" << left << setw(6) << 42 << "]\n";
    cout << "[" << right << setw(6) << "Ali" << "]\n";
```

```
    cout << "[" << setw(2) << 12345 << "]\n";
    cout << "[" << setw(6) << 7 << 8 << "]\n";
    return 0;
}
```

```
ÇIKTI
[    42]
[42    ]
[   Ali]
[12345]
[     78]
```

1. `setw(6)` ile `42`: iki karakter, önüne dört boşluk konur (sağa yaslı).
2. `left`: boşluklar sağa gider. `left` kalıcıdır.
3. `right`: yine sağa yaslı; metinler de aynı kuralla çalışır.
4. `setw(2)` ile `12345`: genişlik "en az" demektir, sığmayan değer kesilmez.
5. `setw(6)` yalnızca `7` için geçerli oldu; `8` genişliksiz yazıldı.

> **▲ Sık Yapılan Hata · setw'in kalıcı olduğunu sanmak**
>
> `cout << setw(8) << ad << puan << notu;` satırında yalnızca `ad` 8 karakterlik alana yazılır; `puan` ile `notu` bitişik gelir. Her sütundan önce `setw` yazmak zorundayız: `cout << setw(8) << ad << setw(6) << puan << setw(6) << notu`.

setfill

`setfill(c)`, boşluk yerine hangi karakterle doldurulacağını belirler. En sık kullanımı sayıları başına sıfır koyarak sabit genişliğe tamamlamaktır (`09:05`). Menülerdeki "ürün ..... fiyat" görünümü de bununla yapılır:

```
▸ dolgu.cpp
```

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << setfill('0') << setw(2) << 9 << ':' << setw(2) << 5 << '\n';
    cout << setfill('.') << left << setw(12) << "Cay"
         << right << setw(6) << 15 << '\n';
    cout << setw(4) << 7 << '\n';
    cout << setfill(' ') << setw(4) << 7 << '\n';
    return 0;
}
```

```
ÇIKTI
09:05
Cay.....15
...7
   7
```

Dördüncü satır `setfill`'in kalıcılığının sonucudur: nokta dolgu hâlâ açık olduğundan `7` sayısı `...7` yazıldı. Son satırda `setfill(' ')` ile varsayılan boşluğa döndük. C'deki `%02d` ve `%-12s` biçimlerinin bu karşılıkları, Bölüm 6'daki tablolu çıktı mantığıyla aynıdır.

Diğer manipülatörler

Aşağıdaki manipülatörler de kalıcıdır:

Manipülatör	Etkisi
`boolalpha` / `noboolalpha`	`bool` değerleri `1` / `0` yerine `true` / `false` yazar
`showpos` / `noshowpos`	Pozitif sayılara da `+` koyar
`dec`, `hex`, `oct`	Tamsayıları onluk, onaltılık, sekizlik tabanda yazar
`showbase` / `noshowbase`	`hex` için `0x`, `oct` için `0` ön ekini ekler
`uppercase` / `nouppercase`	Onaltılık harfleri ve bilimsel gösterimdeki `e`'yi büyük yazar
`scientific`	Her zaman bilimsel gösterim

```
cout << (5 > 3) << ' ' << boolalpha << (5 > 3) << '\n';   // 1 true
cout << hex << showbase << 255 << dec << '\n';            // 0xff
cout << showpos << 42 << ' ' << scientific << 1234.5;      // +42 +1.234500e+03
```

`255` onaltılıkta `ff`'dir (15 × 16 + 15). Ayarları işimiz bittiğinde geri almak (`dec`, `noboolalpha`, `defaultfloat`) iyi bir alışkanlıktır. Parametre alan manipülatörleri kullanıp `<iomanip>` eklemeyi unutursak derleyici `'setw' was not declared in this scope` der ve düzeltmeyi (`std::setw is defined in header <iomanip>`) kendisi önerir.

23.6 Hatalı Girdi ve Akışın Kilitlenmesi

Kullanıcı her zaman beklediğimiz şeyi yazmaz. Sayı beklerken harf basabilir. C'de bunu `scanf`'in dönüş değeriyle anlıyorduk (Bölüm 6); `cin` ise hatayı kendi içinde bir durum bayrağı olarak tutar. Başlangıç değeri `-1` olan iki değişkene `abc 5` girdisini okutalım:

▸ harfGirdim.cpp

```
#include <iostream>
using namespace std;

int main() {
    int a = -1;
    int b = -1;
    cout << "İki sayı girin: ";
    cin >> a;
    cin >> b;
    cout << "a = " << a << ", b = " << b << ", fail = " << cin.fail() << '\n';
    return 0;
}
```

```
ÇIKTI
İki sayı girin: abc 5
a = 0, b = -1, fail = 1
```

`cin >> a` bir rakam bekliyordu, `abc` içindeki `a` rakam değil; okuma başarısız oldu. Program çökmedi, uyarı vermedi. `cin` kendi içinde **failbit** bayrağını açtı ve `cin.fail()` bu bayrağı okuyup `true` döndürdü (`cout` onu `1` olarak yazdı). Üç noktaya dikkat edelim:

- `a` değişti ve `0` oldu. C++11'den beri standart, geçersiz girdide değişkene `0` yazılmasını ister (daha eski kaynaklarda "değişken olduğu gibi kalır" yazar). `0` zaten geçerli bir girdi olabileceği için `a == 0` denetimiyle hatayı anlayamayız; doğru denetim `cin.fail()`'dir.
- `abc` tamponda kaldı. `cin` hiçbir şeyi tüketmedi.
- Akış hata durumuna düştüğü için ikinci `cin >> b` **hiç çalışmadı**: kullanıcı geçerli bir `5` de yazdı, ama `b` hâlâ `-1`. Bu **kilitlenme** durumudur: `failbit` açık olduğu sürece sonraki bütün okumalar sessizce atlanır. "Program soruları peş peşe atlıyor" şikâyetinin nedeni budur.

Akış durumu

> **Tanım: Akış durumu (stream state)**
>
> Bir akışın başına gelenleri tutan bayraklar topluluğudur (`goodbit`, `failbit`, `eofbit`, `badbit`). İşlevlerle sorgulanır; hata bayrakları açıkken akış okuma isteklerini yerine getirmez.

İşlev	`true` ise anlamı
`cin.good()`	Her şey yolunda; okuma yapılabilir
`cin.fail()`	Okuma başarısız oldu (örneğin sayı yerine harf geldi)
`cin.eof()`	Girdi bitti (Linux ve macOS'ta Ctrl+D, Windows'ta Ctrl+Z ve Enter)
`cin.bad()`	Akışın kendisi bozuldu (donanım sorunu gibi, nadir)

Girdi bittiğinde okunacak değer bulunamazsa okuma yine başarısız sayılır; yani `eof` olduğunda `fail` de `true` olur. `cin >> x` ifadesinin değeri akışın kendisi olduğundan başarıyı doğrudan sorabiliriz: `if (cin >> yas) { ... }` "okuma başarılıysa" demektir. `scanf(...) == 1` denetiminin C++ karşılığı budur.

Onarım: clear ve ignore

Akışı eski hâline getirmek iki adım ister ve sıra önemlidir:

1. `cin.clear()` hata bayraklarını temizler; akış yine okumaya hazırdır.
2. `cin.ignore(numeric_limits<streamsize>::max(), '\n')` tamponda kalan bozuk girdiyi satır sonuna kadar siler.

onarim.cpp

```cpp
#include <iostream>
#include <limits>
using namespace std;

int main() {
    int yas = 0;
    cout << "Yaş: ";
    cin >> yas;
    if (cin.fail()) {
        cout << "Geçersiz girdi, satır atlandı.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
        cout << "Yaş: ";
        cin >> yas;
    }
    cout << "Okunan yaş: " << yas << '\n';
    return 0;
}
```

```
ÇIKTI
Yaş: abc
Geçersiz girdi, satır atlandı.
Yaş: 25
Okunan yaş: 25
```

Sırayı bozarsak ne olur?

- Yalnızca `clear()` yazarsak akış okumaya hazır olur ama `abc` tamponda durur; sonraki `>>` aynı yerde yine takılır ve akış yeniden kilitlenir.
- `ignore()`'u `clear()`'dan önce yazarsak `ignore` bir şey yapmaz, çünkü hata durumundaki akışta işlemler çalışmaz.

> **◆ Kritik Nokta · Önce clear, sonra ignore**
>
> Hata durumundaki akışı onarırken sıra değişmez: önce bayrakları temizleriz (`clear`), sonra tamponu boşaltırız (`ignore`). Birini unutmak ya da sırayı tersine çevirmek programı ya sonsuz bir döngüye ya da "sorular atlanıyor" davranışına götürür.

Geçerli değer gelene kadar sormak

Bölüm 8'deki döngülerle bu kalıbı yeniden kullanılabilir bir işleve dönüştürebiliriz. Aşağıdaki işlev 1 ile 10 arasında bir not alana kadar sorar; girdi biterse sonsuza kadar dönmemek için `eof` durumunu da denetler:

▸ sayiIste.cpp

```
#include <iostream>
#include <limits>
using namespace std;

// 1 ile 10 arasında geçerli bir tamsayı alana kadar sorar
int notIste() {
    int deger = 0;
    while (true) {
        cout << "Not (1-10): ";
        if (cin >> deger && deger >= 1 && deger <= 10) {
            return deger;
        }
        if (cin.eof()) {
            return -1;  // girdi bitti
        }
        cout << "Geçersiz not.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int main() {
    int not1 = notIste();
```

```
    cout << "Okunan not: " << not1 << '\n';
    return 0;
}
```

```
ÇIKTI
Not (1-10): abc
Geçersiz not.
Not (1-10): 15
Geçersiz not.
Not (1-10): 0
Geçersiz not.
Not (1-10): 7
Okunan not: 7
```

`cin >> deger && deger >= 1 && ...` koşulunda önce okuma yapılır; başarısızsa `&&` kısa devre yaptığı için aralık denetimine geçilmez (Bölüm 7). `15` ve `0` okuma açısından başarılıdır ama aralığın dışındadır; yine de `clear` ve `ignore` çalışır, çünkü satırda başka artık karakter olabilir.

Girdi bitene kadar okumak

`cin >> x` ifadesi koşul olarak kullanılabildiği için, kaç değer geleceğini bilmediğimiz girdileri `while (cin >> x)` ile okuruz. Döngü, okuma başarısız olduğunda (girdi bittiğinde ya da bozuk değer geldiğinde) sona erer:

girdiSonu.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    double sayi = 0.0;
    double toplam = 0.0;
    int adet = 0;

    while (cin >> sayi) {
        toplam += sayi;
        ++adet;
    }
    cout << adet << " sayı, toplam " << toplam << '\n';
    if (adet > 0) {
        cout << "Ortalama: " << toplam / adet << '\n';
    }
    return 0;
}
```

```
ÇIKTI
4 5.5
10
3 sayı, toplam 19.5
Ortalama: 6.5
```

Girdiyi klavyeden veriyorsak bitirmek için Linux ve macOS'ta Ctrl+D, Windows'ta Ctrl+Z ve Enter'a basarız. Girdi bir dosyadan yönlendirilmişse (`./girdiSonu < sayilar.txt`) dosyanın sonu aynı işi yapar. Aynı kalıbı dosya akışlarıyla Bölüm 37'de yeniden göreceğiz.

Yarım okunan girdi

Her hata `fail()` ile görünmez. `cin >> int` türe uyan kısmı okur ve uymayan ilk karakterde durur; kalan tamponda bekler. Kullanıcı `int` beklenirken `3.9` yazarsa:

ondalikGirdi.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int tam = 0;
    char sonraki = ' ';
    cin >> tam >> sonraki;
    cout << "tam = " << tam << ", sonraki = " << sonraki << '\n';
    cout << "fail = " << cin.fail() << '\n';
    return 0;
}
```

ÇIKTI

```
3.9
tam = 3, sonraki = .
fail = 0
```

`tam` değişkenine `3` okundu ve okuma hatasız bitti; `.` ile durdu. `.9` tamponda kaldı ve sıradaki okuma (burada `char`) `.` karakterini aldı. `fail()` `false` bile döndü; ama veri kullanıcının kastettiği gibi değil. `12abc` yazılırsa da `12` okunur ve `abc` tamponda kalır. Bu yüzden satırın tamamının geçerli olduğundan emin olmak gereken yerlerde satırı `getline` ile bir `string` olarak alıp ayrıştırmak daha sağlamdır (Bölüm 27).

Taşma da sessizdir: `int`'e sığmayan bir sayı (`99999999999`) yazılırsa değişkene `2147483647` atanır ve `failbit` açılır.

> **▲ Sık Yapılan Hata · Ondalık sayıda virgül**
>
> Türkçe klavyede sayı tuş takımı ondalık ayırıcı olarak virgül yazabilir; oysa `cin` ondalık ayırıcı olarak noktayı bekler. Kullanıcı `1,78` yazarsa `double` değişkene `1` okunur, `,78` tamponda kalır: yine hata vermeden yanlış veri. Çıktıda da `cout` her zaman nokta yazar (`1.78`).

23.7 Hata İletileri İçin cerr

Bir program iki tür şey yazar: işin sonucu ve bir sorun olduğunu bildiren iletiler. Bunları ayrı akışlara yazmak işimize yarar. `cerr` standart hata akışıdır (C'deki `stderr`, Bölüm 17):

- `cout` normal çıktı içindir ve tamponludur.
- `cerr` hata iletileri içindir ve **tamponsuzdur**: her yazımda hemen boşalır; program hemen ardından çökse bile ileti kaybolmaz.

Ekranda ikisi ayırt edilemez. Fark, çıktıyı yönlendirdiğimizde ortaya çıkar:

hataAkisi.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
```

```
    cout << "Sonuç: 42\n";
    cerr << "Uyarı: girdi dosyası bulunamadı\n";
    return 0;
}
```

```
ÇIKTI
Sonuç: 42
```

Bu çıktı bloğunda yalnızca `cout` akışı görünür. Terminalde çalıştırırken iki satırı da görürüz. `>` normal çıktıyı, `2>` hata akışını bir dosyaya yönlendirir:

```
TERMİNAL
$ ./hataAkisi
Sonuç: 42
Uyarı: girdi dosyası bulunamadı
$ ./hataAkisi > sonuc.txt
Uyarı: girdi dosyası bulunamadı
$ ./hataAkisi 2> hatalar.txt
Sonuç: 42
```

İkinci komutta ekrana yalnızca uyarı gelir ve `sonuc.txt` içinde `Sonuç: 42` yazar; üçüncüde tersi olur. Programın sonuçlarını dosyaya alırken hata iletilerinin dosyaya karışmaması bu yüzden değerlidir. Windows'ta (cmd ve PowerShell) yönlendirme yazımı aynıdır; yalnızca çalıştırma biçimi değişir (`.\hataAkisi.exe`). Kural: hata iletilerini `cerr`'e, geri kalanını `cout`'a yazalım. `clog` de aynı hedefe yazar ama tamponludur; günlük iletileri için kullanılır.

23.8 printf ile Karşılaştırma

C'nin `printf`'i C++'ta da kullanılabilir (`<cstdio>`, `std::printf`). Aynı satırı iki biçimde yazalım:

`printfKarsi.cpp`

```
#include <cstdio>
#include <iomanip>
#include <iostream>
#include <string>

int main() {
    double fiyat = 12.5;
    int adet = 3;
    std::string ad = "Elma";

    std::printf("printf: %-6s %3d x %8.2f\n", ad.c_str(), adet, fiyat);
    std::cout << "cout  : " << std::left << std::setw(6) << ad
              << ' ' << std::right << std::setw(3) << adet << " x "
              << std::fixed << std::setprecision(2) << std::setw(8) << fiyat
              << '\n';
    return 0;
}
```

```
ÇIKTI
printf: Elma     3 x    12.50
cout  : Elma     3 x    12.50
```

İki yaklaşımın dengesi şöyledir:

	`printf`	`cout`
Tip güvenliği	Yok; yanlış belirteç tanımsız davranıştır	Var; tip derleyiciden bilinir
`std::string` yazdırma	`ad.c_str()` gerekir	Doğrudan yazılır
Biçim	Tek metinde, kısa	Manipülatörlerle, uzun
Kalıcılık	Yok; her belirteç bağımsızdır	Çoğu ayar kalıcıdır
Kendi tipimiz	Yazdırılamaz	`<<` aşırı yüklenerek yazdırılır (Bölüm 32)

`printf`'in kısalığı özellikle tablolarda çekici gelebilir; `cout`'un üstünlüğü ise tip güvenliği ve kendi sınıflarımızı yazdırabilmemizdir. Bu kitapta C++ programlarında `cout` kullanıyoruz. İkisini aynı programda karıştırmak güvenlidir; çünkü C++ varsayılan olarak `cout` ile `stdout`'u eşgüdümlü tutar. Bu eşgüdüm küçük bir hız bedeli taşır. Çok büyük girdi/çıktılarda başına `ios::sync_with_stdio(false);` yazılarak kapatılabilir, ancak o zaman `printf` ile `cout` karıştırılmaz.

C++20 ile gelen `std::format`, `printf`'in kısalığını tip güvenliğiyle birleştirmeyi amaçlar. Bu kitabın C++17 programlarında kullanılamaz; yalnızca görmek için:

```
// C++20 (<format>): {} yer tutucu, :<8 sola yasla, :>8.2f sağa yasla, 2 ondalık
std::cout << std::format("{:<8}{:>3} x {:>8.2f} TL\n", "Cay", 3, 12.5);
// Cay       3 x    12.50 TL
```

Üç Dilde · Biçimli çıktı üç dilde *C · C++ · C#*

Aynı satırı (`Toplam: 12.50 TL`) üç dilde yazalım:

```
/* C: biçim metni */
printf("Toplam: %8.2f TL\n", 12.5);
```

```
// C++: akış ve manipülatörler
cout << "Toplam: " << fixed << setprecision(2) << setw(8) << 12.5 << " TL\n";
```

```
// C#: dize enterpolasyonu; {değer,genişlik:biçim}
Console.WriteLine($"Toplam: {12.5,8:F2} TL");
// aynı işin bileşik biçimlendirme yazımı
Console.WriteLine("Toplam: {0,8:F2} TL", 12.5);
```

C'de her şey tek bir biçim metnindedir ve tip uyuşmazlığını yalnızca derleyici uyarısı yakalar. C++'ta ayarlar akışa yazılan manipülatörlerdir; bunlar kalıcıdır (`setw` hariç) ve bu yüzden hem esnek hem de dikkat ister. C#'ta `{0,8:F2}` yazımı, "0 numaralı değeri 8 karakter genişliğinde, 2 ondalıklı sabit noktalı sayı olarak yaz" demektir; genişlik ve biçim her değerin kendi yer tutucusundadır, kalıcı durum yoktur. Ayrıntıları Bölüm 43'te göreceğiz.

23.9 Örnek Uygulama: Fatura

Şimdiye kadar öğrendiklerimizi birleştirelim. Program önce kalem sayısını, sonra her kalem için `ad adet birimFiyat` satırını okuyacak; hizalı bir fatura ve KDV'li genel toplam yazdıracak. Hata iletileri `cerr`'e gidecek. Ürün adlarını tek sözcük ve ASCII harflerle yazıyoruz; nedenini sonraki başlıkta göreceğiz.

fatura.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Kalem {
    string ad;
    int adet;
    double fiyat;
};

// "etiket ..... değer" biçiminde bir toplam satırı yazar
void toplamSatiri(string etiket, double deger, string birim) {
    cout << left << setw(28) << etiket << right << setw(11) << deger
         << birim << '\n';
}

int main() {
    const double KDV_ORANI = 0.20;   // örnek oran
    Kalem kalemler[10];
    int n = 0;

    cout << "Kalem sayısı (en çok 10): ";
    cin >> n;
    if (n < 1 || n > 10) {
        cerr << "Geçersiz kalem sayısı.\n";
        return 1;
    }
    cout << "Her satıra: ad adet birimFiyat\n";
    for (int i = 0; i < n; ++i) {
        cin >> kalemler[i].ad >> kalemler[i].adet >> kalemler[i].fiyat;
    }

    cout << fixed << setprecision(2);
    cout << '\n' << left << setw(12) << "Kalem" << right << setw(6) << "Adet"
         << setw(10) << "Fiyat" << setw(11) << "Tutar" << '\n';
    cout << string(39, '-') << '\n';

    double araToplam = 0.0;
    for (int i = 0; i < n; ++i) {
        double tutar = kalemler[i].adet * kalemler[i].fiyat;
        araToplam += tutar;
        cout << left << setw(12) << kalemler[i].ad << right
             << setw(6) << kalemler[i].adet
             << setw(10) << kalemler[i].fiyat << setw(11) << tutar << '\n';
    }

    double kdv = araToplam * KDV_ORANI;
    cout << string(39, '-') << '\n';
    toplamSatiri("Ara toplam", araToplam, "");
    toplamSatiri("KDV (%20)", kdv, "");
    toplamSatiri("GENEL TOPLAM", araToplam + kdv, " TL");
    return 0;
}
```

```
ÇIKTI
Kalem sayısı (en çok 10): 3
Her satıra: ad adet birimFiyat
Ekmek 4 12.5
Sut 2 38.75
Peynir 1 215

Kalem         Adet     Fiyat      Tutar
--------------------------------------
Ekmek            4     12.50      50.00
Sut              2     38.75      77.50
Peynir           1    215.00     215.00
--------------------------------------
Ara toplam                       342.50
KDV (%20)                         68.50
GENEL TOPLAM                     411.00 TL
```

Programdan şu noktaları çıkaralım:

- `cin >> ... >> ...` zinciri üç farklı tipte (`string`, `int`, `double`) tek ifadede okur; `struct` üyeleri `kalemler[i].ad` biçiminde okunur (Bölüm 14).
- `fixed << setprecision(2)` bir kez yazıldı; `adet` tamsayı olduğu için etkilenmedi, `fiyat` ve `tutar` iki ondalıkla çıktı.
- `left` ve `right` kalıcı olduğundan ad sütunundan sonra `right`'a geri döndük.
- Başlık ile veri satırları aynı genişlikleri (12, 6, 10, 11) kullandığı için sütunlar hizalıdır.
- Geçersiz kalem sayısında hata iletisi `cerr`'e yazılır ve program `1` ile biter. Okuma hataları (`abc` gibi) için `fail()` denetimi eklemek bir geliştirme olarak okura bırakılmıştır.

23.10 Türkçe Karakterler ve Hizalama

Bölüm 3 ve 6'dan bildiğimiz iki konu burada da geçerlidir. Birincisi, Türkçe karakterlerin Windows konsolunda bozuk görünmesi bir kodlama sorunudur: kaynak dosya UTF-8 olmalı, konsol `chcp 65001` ile UTF-8'e alınmalıdır. İkincisi, UTF-8'de `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` gibi harfler 2 bayt yer kaplar ve `setw` **bayt** sayar, harf saymaz. Fatura örneğinde ASCII ad kullanmamızın nedeni budur; `Çorba` yazsaydık satır bir karakter kısa kalırdı.

Çözüm, genişliği kendimiz harf sayısına göre hesaplamaktır. UTF-8'de çok baytlı bir harfin ilk baytından sonraki baytların üst iki biti her zaman `10`'dur (`10xxxxxx`). Bu "devam baytlarını" saymazsak harf sayısını buluruz. Bit maskeleme işlemini Bölüm 16'da görmüştük:

▸ turkceHiza.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

// UTF-8'de devam baytları 10xxxxxx biçimindedir; harf sayarken onları atlarız
int harfSayisi(string s) {
    int say = 0;
    for (size_t i = 0; i < s.size(); ++i) {
        unsigned char b = static_cast<unsigned char>(s[i]);
        if ((b & 0xC0) != 0x80) {
            ++say;
        }
```

```
    }
    return say;
}

int main() {
    string adlar[3] = {"Ekmek", "Çorba", "Süt"};

    for (int i = 0; i < 3; ++i) {
        int bosluk = 8 - harfSayisi(adlar[i]);   // harf sayısına göre dolgu
        cout << "[" << left << setw(8) << adlar[i] << "]  ";
        cout << "[" << adlar[i] << string(bosluk, ' ') << "]  "
             << adlar[i].size() << " bayt, "
             << harfSayisi(adlar[i]) << " harf\n";
    }
    return 0;
}
```

```
ÇIKTI
[Ekmek   ]  [Ekmek   ]  5 bayt, 5 harf
[Çorba   ]  [Çorba   ]  6 bayt, 5 harf
[Süt     ]  [Süt     ]  4 bayt, 3 harf
```

Her satırdaki ilk köşeli parantez `setw` ile, ikincisi harf sayısından hesapladığımız boşlukla yazıldı. `setw` ile yazılan `Çorba` ve `Süt` satırları, ikişer baytlı bir harf içerdikleri için ekranda bir karakter kısa kalır; ikinci sütun ise düzgün hizalıdır. `size()` bayt sayısını verir (Bölüm 27'de ayrıntısını göreceğiz); harf sayısını bulmak için kendi işlevimize ihtiyaç duyduk. Büyük/küçük harf dönüşümü ve sıralama gibi işlerde bu yaklaşım yetmez; Unicode'u tam destekleyen kütüphaneler (örneğin ICU) gerekir.

Bölüm Özeti

- Akış (stream), baytların aktığı soyut bir borudur: `cin` giriş, `cout` ve `cerr` çıkış akışıdır. Hepsi `<iostream>` içindedir.
- `cout << a << b` zincirlenir, çünkü `<<` ifadesinin değeri akışın kendisidir. Tipi derleyici bilir; biçim belirteci gerekmez.
- `<<`, `+` ve `*`'dan sonra, ama `>`, `==`, `&&`'den önce işlenir. Karşılaştırma ve mantık ifadelerini `cout` içinde parantezle yazmalıyız.
- `cin >> x` biçim belirteci ve `&` istemez. `>>` baştaki boşlukları (Enter dahil) atlar, tipe uymayan ilk karakterde durur ve o karakteri tamponda bırakır.
- `cin >> ad` yalnızca ilk sözcüğü alır. Boşluklu satır için `getline(cin, ad)` kullanılır.
- `cin >>`'dan sonra `getline` gelecekse araya `cin.ignore(numeric_limits<streamsize>::max(), '\n');` konur (`<limits>` gerekir).
- Varsayılan ondalık çıktı 6 anlamlı basamaktır (`100.0` → `100`). `fixed` olmadan `setprecision(n)` toplam basamak sayısını, `fixed` ile noktadan sonraki hane sayısını belirler.
- `fixed`, `setprecision`, `left`, `right`, `setfill`, `hex` gibi manipülatörler kalıcıdır; `setw` yalnızca bir sonraki değeri etkiler ve sığmayan değeri kesmez.
- `setw`, `setprecision`, `setfill` için `<iomanip>` eklenmelidir.
- Sayı beklerken harf gelirse `cin.fail()` `true` olur, değişken `0` olur, bozuk girdi tamponda kalır ve sonraki okumalar atlanır. Onarım sırası: önce `cin.clear()`, sonra `cin.ignore(...)`.
- `while (cin >> x)` girdi bitene kadar okur. `3.9` gibi girdiler `int`'e hatasız ama kısmen okunur.

- `cerr` tamponsuz hata akışıdır; `2>` ile ayrı dosyaya yönlendirilebilir. Sonuçlar `cout`'a, hata iletileri `cerr`'e yazılır.
- `printf` C++'ta da kullanılabilir, ama tip güvenli değildir. C++20'nin `std::format`'ı bu kitabın C++17 programlarında kullanılmaz.
- UTF-8'de Türkçe harfler 2 bayttır ve `setw` bayt sayar; hizalamada harf sayısı gerekirse kendimiz hesaplarız.

Alıştırmalar

1. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    cout << setprecision(2) << 3.14159 << '\n';
    return 0;
}
```

   A) `3.14`
   B) `3`
   C) `3.1`
   D) `3.14159`
   E) `3.10`

2. Bir program önce `int yas` değişkenini `cin >> yas;` ile, hemen ardından `string ad` değişkenini `getline(cin, ad);` ile okuyor. Kullanıcı ilk satıra `20`, ikinci satıra `Ayşe Nur` yazıp her birinin ardından Enter'a basıyor. `ad` hakkında hangisi doğrudur?

   A) Program çöker.
   B) `ad` `Ayşe Nur` olur.
   C) Program derlenmez; `cin` ile `getline` aynı programda kullanılamaz.
   D) `ad` `20` olur.
   E) `ad` boş kalır; `getline`, sayıdan sonra tamponda kalan satır sonunu okuyup hemen döner.

3. Aşağıdaki programın çıktısı nedir? (Köşeli parantezler çıktının parçasıdır.)

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    cout << "[" << setw(5) << 1 << 2 << 3 << "]\n";
    return 0;
}
```

4. `cin >> sayi;` satırında `abc` girildi ve akış hata durumuna düştü. Akışı yeniden kullanılabilir hâle getirmek için hangisi doğrudur?

   A) Önce `cin.ignore(numeric_limits<streamsize>::max(), '\n');`, sonra `cin.clear();`
   B) Önce `cin.clear();`, sonra `cin.ignore(numeric_limits<streamsize>::max(), '\n');`
   C) Yalnızca `cin.clear();` yeterlidir.

D) Yalnızca `cin.ignore();` yeterlidir.
E) Hiçbir şey gerekmez; akış kendini onarır.

5. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    cout << fixed << setprecision(1);
    cout << 7.96 << ' ' << 7.04 << ' ' << 12 << '\n';
    cout << setprecision(0) << 7.96 << '\n';
    cout << scientific << setprecision(2) << 12345.678 << '\n';
    return 0;
}
```

6. Aşağıdaki program derlenmiyor. Hatanın nedenini açıklayın ve düzeltin.

```
#include <iostream>
using namespace std;
int main() {
    int x = 5;
    cout << "x beş mi: " << x == 5 << '\n';
    return 0;
}
```

7. Aşağıdaki program çalıştırılıyor ve klavyeden `3.9` yazılıp Enter'a basılıyor. Çıktı nedir?

```
#include <iostream>
using namespace std;
int main() {
    int a = 0;
    double b = 0.0;
    cin >> a >> b;
    cout << "a=" << a << " b=" << b << '\n';
    return 0;
}
```

8. Çoktan seçmeli: UTF-8 kaynak dosyada `cout << left << setw(6) << "Süt" << "|";` satırı kaç boşluk yazdırır?

A) 0
B) 1
C) 2
D) 3
E) 6

9. Kullanıcıdan iki ondalıklı sayı okuyup ortalamasını iki ondalık basamakla yazdıran bir program yazın.
10. Kullanıcıdan önce yaşını (`int`), ardından ad ve soyadını (boşluklu) okuyan; sonra adı 15 karakterlik alana sola yaslı, yaşı parantez içinde yazdıran bir program yazın. Örnek çıktı: `Ece Kaya (30)`.
11. Girdi bitene kadar tamsayı okuyan; kaç sayı geldiğini, toplamlarını ve en büyüğünü yazdıran bir program yazın.
12. Çarpım tablosunu (1'den 5'e kadar) `setw(4)` ile hizalı yazdıran bir program yazın. Ardından, kullanıcıdan pozitif bir tamsayı alana kadar soran bir `while` döngüsü yazın: geçersiz girdide `cin.clear()` ve `cin.ignore(...)` çağrılmalı, girdi biterse `cerr`'e ileti yazılıp program sonlanmalıdır.

BÖLÜM 24

Kontrol Yapıları: Koşullar ve Döngüler

BU BÖLÜMDE

- C'den gelen `if`, `switch`, `while`, `do-while` ve `for` yapılarının C++'ta aynı çalıştığını, farkın nerelerde başladığını ayırt etmeyi
- Karşılaştırma sonuçlarının `bool` olduğunu ve `std::cout << a > b` tuzağını açıklamayı
- C++17'nin `if` ve `switch` başlatıcılarını (`if (int x = f(); x > 0)`) ve değişkenin kapsamını kullanmayı
- `[[fallthrough]]` niteleyicisini ve `case` etiketinde ilk değer verme kuralını bilmeyi
- Aralık tabanlı `for` döngüsünü (`for (auto& x : kap)`) değer, referans ve `const` referans biçimleriyle yazmayı
- `std::gcd`, `std::lcm` ve `std::vector` gibi araçlarla klasik döngü algoritmalarını C++ biçiminde yazmayı
- C# `foreach` ve `switch` ifadelerinin C++ karşılıklarıyla ilişkisini görmeyi

Bölüm 7'de koşulları, Bölüm 8'de döngüleri C ile ayrıntılı gördük. C++ bu yapıları olduğu gibi devralır: `if (n % 2 == 0)` iki dilde de aynı yazılır, aynı çalışır ve aynı tuzaklara sahiptir. Bu bölüm o konuları yeniden anlatmayacak; C++'ın eklediği ve değiştirdiği noktalara odaklanacak.

Eklemelerin çoğu iki amaca hizmet eder: değişkenlerin ömrünü gerçekten gerektiği kadar kısa tutmak (başlatıcılı `if` ve `switch`) ve indeks hatalarını ortadan kaldırmak (aralık tabanlı `for`). Bölümün sonunda bu araçlarla küçük bir not istatistiği programı yazacağız. Örneklerde `std::vector` ve `std::string` kısa kısa görünecek; ikisini de ayrıntılı olarak Bölüm 26 ve 27'de işleyeceğiz, burada yalnızca "elemanlarını sırayla dolaşabildiğimiz kutular" olarak kullanacağız.

24.1 C'den Gelenler: Hızlı Bakış

Aşağıdaki tablo, Bölüm 7 ve 8'deki konuların C++'taki durumunu özetler:

Konu	C++'ta durum
`if`, `if-else`, `else if` merdiveni	Aynı; koşul parantez içinde, tek deyimli gövdede bile `{}` önerilir
Sarkan `else`	Aynı kural: `else` kendisine en yakın `if`'e bağlanır
`&&`, `||`, `!` ve kısa devre	Aynı; sonuç tipi C'de `int`, C++'ta `bool`
`and`, `or`, `not`	C++'ta anahtar sözcük olarak yazılabilir (C'de `<iso646.h>` gerekir)
`?:`	Aynı; C++'ta sonuç, koşullara göre atama yapılabilen bir değer de olabilir
`switch`	Aynı kurallar; ek olarak C++17'de başlatıcı ve `[[fallthrough]]`
`while`, `do-while`, klasik `for`	Aynı; döngü değişkeni `for` içinde tanımlanabilir (C99'dan beri C'de de)

Konu	C++'ta durum
`break`, `continue`, `goto`	Aynı; etiketli `break` iki dilde de yoktur
Aralık tabanlı `for`	Yalnız C++ (C++11)
Doğru/yanlış kuralı	Aynı: 0 yanlış, sıfırdan farklı her değer doğru

C'deki `<stdbool.h>` ile gelen `bool`, `true` ve `false` C++'ta gerçek anahtar sözcüklerdir (Bölüm 22); başlık eklemek gerekmez. C++'ta `and`, `or` ve `not` yazılabilmesi bir sözdizimi kolaylığıdır, `&&`, `||` ve `!` ile birebir aynı anlama gelir:

```
if (yas >= 18 and not misafir) { /* ... */ }   // yas >= 18 && !misafir
```

Kitapta C ile uyumlu olması için simgesel biçimleri (`&&`, `||`, `!`) kullanacağız; ama başka kodlarda bu sözcükleri görürseniz şaşırmayın.

24.2 Karşılaştırma Sonuçları bool'dur

C'de `3 > 2` ifadesinin tipi `int`'ti ve sonuç 1 ya da 0 olurdu (Bölüm 7). C++'ta karşılaştırma ve mantıksal operatörlerin sonucu `bool` tipindedir. Gündelik kodda farkı hissetmeyiz, çünkü `bool` gerektiğinde `int`'e dönüşür. Fark en çok yazdırırken ortaya çıkar: `std::cout` bir `bool`'u varsayılan olarak `1` ya da `0` yazar (Bölüm 22'de gördük; `std::boolalpha` ile `true`/`false` yazdırılır, ayrıntısı Bölüm 23'te).

karsilastirma.cpp

```
#include <iostream>

int main() {
    int a{7};
    int b{4};

    std::cout << (a > b) << ' ' << (a == b) << '\n';
    std::cout << std::boolalpha;
    std::cout << (a > b) << ' ' << (a + b == 11) << '\n';

    bool buyukMu = a > b;
    if (buyukMu) {
        std::cout << "a, b'den büyük\n";
    }
    return 0;
}
```

```
ÇIKTI
1 0
true true
a, b'den büyük
```

Karşılaştırmayı parantez içine almamız tesadüf değil. `<<` ve `>` operatörleri aynı ifadede karşılaşırsa `<<` önce işlenir (`<<` kaydırma operatörüdür ve karşılaştırmalardan öncelikli; Bölüm 5 ve Ek B). Yani `std::cout << a > b` ifadesi `(std::cout << a) > b` olarak okunur:

coutTuzagi.cpp

```cpp
#include <iostream>

int main() {
    int a{7};
    int b{4};
    std::cout << a > b;
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
coutTuzagi.cpp: In function 'int main()':
coutTuzagi.cpp:6:20: error: no match for 'operator>' (operand types are
'std::basic_ostream<char>' and 'int')
    6 |     std::cout << a > b;
      |     ~~~~~~~~~~~~~~ ^ ~
      |              |      |
      |              |      int
      |              std::basic_ostream<char>
```

`std::cout << a` ifadesi bir akış nesnesi verir ve akışı bir sayıyla karşılaştıran bir `>` operatörü yoktur. g++ bu mesajın ardından denenebilecek onlarca aday işleci uzun uzun listeler; biz yalnızca ilk satırı ve işaretli kısmı gösterdik. Hata mesajlarında asıl bilgi her zaman ilk `error:` satırındadır (Ek D). Çözüm, karşılaştırmayı parantez içine almaktır: `std::cout << (a > b)`.

> **▲ Sık Yapılan Hata · Akış çıktısında koşulu parantezle**
>
> `std::cout << a > b`, `std::cout << x == 0` ve `std::cout << a && b` gibi ifadeler beklenen anlamda çalışmaz. Çıktıya karşılaştırma ya da mantıksal ifade yazacaksak ifadenin tamamını parantez içine alırız. C'de `printf("%d", a > b)` bu sorunu yaşatmazdı, çünkü karşılaştırma bir fonksiyon argümanıdır ve `<<` operatörüne bağlı değildir.

Koşul olarak akış nesneleri

`if` ve `while` koşulu `bool`'a dönüştürülebilen her ifade olabilir. Akış nesneleri (`std::cin` gibi) de bunlara dahildir: bir okuma işlemi başarılıysa koşul doğru, başarısızsa (biçim hatası ya da girdi sonu) yanlış olur. Bu, C'deki `scanf("%d", &x) == 1` denetiminin C++ karşılığıdır ve `while` döngüsünde çok kullanışlıdır:

girdiDongusu.cpp

```cpp
#include <iostream>

int main() {
    int sayi{};
    int toplam{};
    int adet{};

    std::cout << "Sayıları girin (sonlandırmak için harf yazın): ";
    while (std::cin >> sayi) {
        toplam += sayi;
        ++adet;
    }

    std::cout << adet << " sayı okundu, toplam " << toplam << '\n';
    return 0;
}
```

```
ÇIKTI
Sayıları girin (sonlandırmak için harf yazın): 4 10 -2 7 q
4 sayı okundu, toplam 19
```

`std::cin >> sayi` ifadesi hem okumayı yapar hem de değer olarak akışın kendisini verir; koşul bu akışın "hâlâ sağlam mı" sorusunu yanıtlar. `q` girildiğinde sayı okunamaz, akış hata durumuna geçer ve döngü biter. Hata durumundaki akışın nasıl onarılacağını (`cin.clear()`, `cin.ignore()`) Bölüm 23'te gördük. Bölüm 8'deki `scanf` döndürme değeri denetimiyle karşılaştırdığımızda aynı fikri daha kısa yazdığımızı fark ederiz.

24.3 Başlatıcılı if ve switch (C++17)

Bir değişken yalnızca bir koşulu denetlemek için gerekiyorsa, onu kapsayan bloğun tamamına yaymak gereksizdir. C'de bunun yolu fazladan bir süslü parantez bloğu açmaktı:

```
{
    int kalan = n % 3;
    if (kalan == 0) { /* ... */ }
    else if (kalan == 1) { /* ... */ }
}   // kalan burada biter
```

C++17 aynı işi `if` deyiminin kendi içinde yapmamıza izin verir: koşulun önüne, noktalı virgülle ayrılmış bir başlatıcı (initializer) yazarız.

> **Tanım: Başlatıcılı if**
>
> `if (başlatıcı; koşul)` biçimidir. Başlatıcıda tanımlanan değişken `if`'in koşulunda, gövdesinde ve **bütün `else` kollarında** kullanılabilir; `if` deyimi bittiğinde yok olur. `switch` için de aynı biçim geçerlidir: `switch (başlatıcı; ifade)`.

▸ **baslaticiIf.cpp**

```cpp
#include <iostream>

int main() {
    for (int n : {9, 10, 11}) {
        if (int kalan = n % 3; kalan == 0) {
            std::cout << n << ": 3'e tam bölünür\n";
        } else if (kalan == 1) {
            std::cout << n << ": kalan 1\n";
        } else {
            std::cout << n << ": kalan " << kalan << '\n';
        }
        // burada kalan artık yok
    }
    return 0;
}
```

```
ÇIKTI
9: 3'e tam bölünür
10: kalan 1
11: kalan 2
```

`kalan` yalnızca `if` yapısının içinde yaşar; `else if` ve `else` kolları da onu görür. Döngünün `{9, 10, 11}` biçiminde yazılan listesini birazdan aralık tabanlı `for` başlığında ele alacağız.

Başlatıcının en yararlı olduğu yer, girdi okuma ve denetim kalıbıdır. Değişkeni tanımlar, okur ve aynı satırda denetleriz:

girdiDenetim.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Yaşınız: ";
    if (int yas{}; std::cin >> yas && yas >= 0) {
        std::cout << yas + 1 << " yaşına gireceksiniz\n";
    } else {
        std::cout << "Geçerli bir yaş girilmedi\n";
    }
    return 0;
}
```

ÇIKTI

```
Yaşınız: 25
26 yaşına gireceksiniz
```

Burada `yas` başlatıcıda sıfır değerle tanımlanıyor, koşulda okunuyor ve `&&` kısa devre yaptığı için okuma başarısızsa `yas >= 0` hiç denenmiyor. Okuma başarısız olsa bile `yas` çöp değer taşımaz, çünkü `{}` onu sıfırlamıştı.

C++ başlatıcısız bir biçimi de eskiden beri destekler: koşulun içinde tanımlama. `if (int sonuc = hesapla())` biçimi değişkeni tanımlar ve **değeri koşul olarak kullanır** (sıfır yanlış, diğerleri doğru). Başlatıcılı biçim bundan daha açıktır, çünkü koşulu ayrıca yazarız; yeni kodda onu tercih edeceğiz.

switch başlatıcısı

`switch` için de aynı kalıp geçerlidir. Değişken yalnızca `switch` boyunca yaşar:

baslaticiSwitch.cpp

```cpp
#include <iostream>

int main() {
    for (int saat : {8, 13, 21}) {
        switch (int dilim = saat / 6; dilim) {
            case 1:
                std::cout << saat << ": sabah\n";
                break;
            case 2:
                std::cout << saat << ": öğleden sonra\n";
                break;
            default:
                std::cout << saat << ": akşam\n";
                break;
        }
    }
    return 0;
}
```

ÇIKTI

```
8: sabah
13: öğleden sonra
21: akşam
```

`saat / 6` tamsayı bölmesi olduğundan 8 için 1, 13 için 2, 21 için 3 çıkar; üçüncü değer `default`'a düşer.

24.4 switch: Düşme ve case Etiketinde Tanımlama

Bölüm 7'de `switch`'in `break` yazılmazsa bir sonraki `case`'e düştüğünü (fall-through) görmüştük. Bu davranış bazen istenir; çoğu zaman da unutulmuş bir `break`'in sonucudur. Derleyici bu iki durumu ayıramaz. C++17 bu niyeti açıkça belirtmek için `[[fallthrough]];` niteleyicisini getirdi.

▸ dusme.cpp

```cpp
#include <iostream>

int main() {
    for (int ay : {1, 4, 7, 12}) {
        int ceyrek{};
        switch (ay) {
            case 1:
            case 2:
            case 3:
                ceyrek = 1;
                break;
            case 4:
            case 5:
            case 6:
                ceyrek = 2;
                break;
            case 7:
                std::cout << "temmuz tatil dönemi, ";
                [[fallthrough]];
            case 8:
            case 9:
                ceyrek = 3;
                break;
            default:
                ceyrek = 4;
                break;
        }
        std::cout << ay << ". ay: " << ceyrek << ". çeyrek\n";
    }
    return 0;
}
```

```
ÇIKTI
1. ay: 1. çeyrek
4. ay: 2. çeyrek
temmuz tatil dönemi, 7. ay: 3. çeyrek
12. ay: 4. çeyrek
```

`case 1:`, `case 2:`, `case 3:` gibi içi boş etiketlerin yan yana yazılması düşme sayılmaz; birden çok değeri aynı koda bağlamanın olağan yoludur. `[[fallthrough]];` ise **içinde deyim bulunan** bir `case`'in sonunda, bilerek yazılmış düşmeyi işaretler. `case 7`'de yazdıktan sonra `case 8`'in koduna geçilir.

Niteleyici derleyiciye, okuyucuya ve araçlara aynı bilgiyi verir: "Bu `break` eksik değil." `g++ -Wextra` seçeneği `[[fallthrough]]` olmadan yazılmış düşmeleri bildirir; böylece unutulan `break`'ler yakalanır.

> **◆ Kritik Nokta · Düşmeyi her zaman işaretleyin**
>
> İçinde deyim bulunan bir `case`, `break`, `return` ya da `[[fallthrough]];` ile bitmiyorsa bunu bir hata adayı olarak görürüz. Bilerek yaptıysak niteleyiciyi yazarız; yazmadıysak kodu düzeltiriz.

case etiketini atlayarak tanımlama

C++'ın `switch` konusunda C'den sert olduğu bir nokta vardır: bir `case` içinde ilk değer verilen bir değişken tanımlarsak ve program sonraki bir `case`'e atlayarak bu tanımı atlarsa, derleyici bunu hata sayar.

caseTanim.cpp

```cpp
#include <iostream>

int main() {
    int secim{2};
    switch (secim) {
        case 1:
            int ikiKat = secim * 2;
            std::cout << ikiKat << '\n';
            break;
        case 2:
            std::cout << "iki\n";
            break;
    }
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
caseTanim.cpp: In function 'int main()':
caseTanim.cpp:10:14: error: jump to case label
   10 |         case 2:
      |              ^
caseTanim.cpp:7:17: note:   crosses initialization of 'int ikiKat'
    7 |             int ikiKat = secim * 2;
      |                 ^~~~~~
```

`ikiKat` bütün `switch` bloğunun kapsamındadır; `case 2:` etiketine atlandığında tanım atlanmış olur ve değişken ilk değersiz kalırdı. C++ bunu yasaklar; C ise bu atlamaya izin verir. Çözüm, o `case`'in kodunu kendi süslü parantezine almaktır:

```cpp
case 1: {
    int ikiKat = secim * 2;
    std::cout << ikiKat << '\n';
    break;
}
```

Süslü parantez değişkenin kapsamını `case 1`'in içine hapseder, böylece atlanabilecek bir tanım kalmaz. Bölüm 7'deki "`case` etiketinde neler yazılamaz" kurallarına bu da eklenir: `switch` yalnızca tamsayı türünde (karakter ve sabit sayılar dahil) değerlerle çalışır ve `case` değerleri derleme zamanında bilinen sabitler olmalıdır. `std::string` ile `switch` yapılamaz (Bölüm 27'de metinleri `==` ile karşılaştırmayı göreceğiz).

24.5 Küçük Farklar: ?: ve Ondalık Karşılaştırma

Üçlü operatör (`koşul ? a : b`) C++'ta da aynı çalışır ve Bölüm 7'deki öneriler geçerlidir. Küçük bir ek: iki kol da aynı tipte bir değişkense sonuç bir **değer olarak değil, bir nesne olarak** da kullanılabilir. Yani atamanın sol tarafına geçebilir:

ucluOperator.cpp

```cpp
#include <iostream>

int main() {
    int a{3};
    int b{8};

    (a > b ? a : b) = 0;    // büyük olanı sıfırla
    std::cout << a << ' ' << b << '\n';
    return 0;
}
```

```
ÇIKTI
3 0
```

C'de bu satır derlenmez. Bu kullanım ilk bakışta okunması zor olduğundan sık rastlanmaz; yine de C++ kodunda görürsek hata sanmamalıyız.

Ondalık sayıları `==` ile karşılaştırmanın güvenilmezliği (`0.1 + 0.2 != 0.3`) Bölüm 5 ve 7'de anlatıldı. Fark denetimini C++'ta `std::abs` ve `<limits>` başlığından gelen makine epsilonuyla yazarız. Bölüm 22'de gördüğümüz `std::numeric_limits<double>::epsilon()` , 1.0 ile ondan büyük en küçük `double` arasındaki farktır:

ondalikKarsilastir.cpp

```cpp
#include <cmath>
#include <iostream>
#include <limits>

bool yakinMi(double x, double y) {
    double tolerans = 1e-9;
    return std::abs(x - y) < tolerans;
}

int main() {
    double toplam = 0.1 + 0.2;
    std::cout << std::boolalpha;
    std::cout << (toplam == 0.3) << '\n';
    std::cout << yakinMi(toplam, 0.3) << '\n';
    std::cout << (std::numeric_limits<double>::epsilon() < 1e-9) << '\n';
    return 0;
}
```

```
ÇIKTI
false
true
true
```

Bölüm 7'deki yöntem aynıdır: iki sayının farkının mutlak değerini küçük bir toleransla karşılaştırırız. Tolerans değeri probleme göre seçilir; `epsilon()` yalnızca 1 civarındaki sayılar için anlamlı bir alt sınırdır, büyük ya da

küçük büyüklüklerde doğrudan tolerans olarak kullanılmaz. C'deki `fabs` yerine C++'ta `std::abs` yazılır; aynı ad hem tamsayılar hem ondalık sayılar için çalışır (aşırı yükleme, Bölüm 25).

24.6 while, do-while ve Klasik for

Bölüm 8'deki üç döngü C++'ta değişmeden çalışır: `while` koşulu önce denetler, `do-while` gövdeyi en az bir kez çalıştırır, `for` sayaç, koşul ve adımı tek satırda toplar. Noktalı virgül tuzağı (`for (...);`), `break` ile `continue`'nun işi, iç içe döngülerin çalışma düzeni ve tek tek kayma (off-by-one) hatası da aynıdır. Bu başlıkta yeni bir şey öğrenmiyoruz; yalnızca C++ biçimiyle bir örnek yazıyoruz.

C++'ın çıktı yazma biçimi desen programlarını kısaltır. `std::string(n, c)`, `c` karakterinden `n` tane içeren bir metin üretir (Bölüm 27). C'de iç döngü gerektiren yıldız satırı burada tek ifadedir:

ucgenDesen.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    for (int satir = 1; satir <= 4; ++satir) {
        std::cout << std::string(4 - satir, ' ')
                  << std::string(2 * satir - 1, '*') << '\n';
    }
    return 0;
}
```

```
ÇIKTI
   *
  ***
 *****
*******
```

Boşluk sayısı `4 - satir`, yıldız sayısı `2 * satir - 1`'dir. Bölüm 8'de iç içe iki `for` ile yazdığımız desen burada tek döngüyle çıktı.

> **▲ Sık Yapılan Hata · Döngü sayacında ++i ve i++**
>
> C++'ta `for` başlığında `++i` yazmak yaygın bir alışkanlıktır. Tamsayılarda `i++` ile sonuç aynıdır; fark, ileride yineleyiciler (iterator) ve sınıf nesneleri için artırma operatörlerini yazmaya başladığımızda ortaya çıkar: ön ek biçimi eski değerin kopyasını tutmaz. Bu kitapta C++ döngülerinde `++i` kullanıyoruz.

İç içe döngüden çıkmak

`break` yalnızca içinde bulunduğu en yakın döngüyü sonlandırır; C'de olduğu gibi C++'ta da etiketli `break` yoktur. Dıştaki döngüden de çıkmak istiyorsak Bölüm 8'deki seçenekler geçerlidir: bayrak değişkeni, `goto` ya da döngüyü bir fonksiyona koyup `return` ile çıkmak. Üçüncüsü en okunaklı olandır:

ciftBul.cpp

```cpp
#include <iostream>

// toplamı hedef olan ilk (a, b) çiftini yazar; bulursa true döndürür
bool ciftBul(int hedef) {
    for (int a = 1; a <= 9; ++a) {
        for (int b = a; b <= 9; ++b) {
```

```
            if (a * b == hedef) {
                std::cout << a << " x " << b << " = " << hedef << '\n';
                return true;
            }
        }
    }
    return false;
}

int main() {
    std::cout << std::boolalpha << ciftBul(42) << '\n';
    std::cout << ciftBul(11) << '\n';
    return 0;
}
```

```
ÇIKTI
6 x 7 = 42
true
false
```

`return`, iki döngüyü birden sonlandırır. `ciftBul(11)` için 11 asal olduğundan 1 ile 9 arasında çarpım bulunamaz ve fonksiyon `false` döndürür. Fonksiyonların C++'taki ayrıntılarını Bölüm 25'te göreceğiz.

24.7 Aralık Tabanlı for (C++11)

Bir dizinin ya da bir kapsayıcının bütün elemanlarını dolaşmak programlamada en sık yapılan işlerden biridir. C'de bunu indeksli `for` ile yaparız:

```
for (int i = 0; i < 5; i++) {
    toplam += dizi[i];
}
```

Bu yazımda iki hata türü mümkündür: sınırı yanlış yazmak (`i <= 5`) ve indeksi kaçırmak ya da karıştırmak. Çoğu zaman indeksin kendisini hiç kullanmayız; yalnızca eleman lazımdır. C++11, bu durum için **aralık tabanlı for** (range-based for) getirdi.

> **Tanım: Aralık tabanlı for**
>
> `for (tip değişken : kapsayıcı) gövde` biçiminde yazılan ve kapsayıcının her elemanını sırayla `değişken`'e alarak gövdeyi çalıştıran döngüdür. Kapsayıcı olarak C dizileri, `std::array`, `std::vector`, `std::string`, `{1, 2, 3}` gibi değer listeleri ve `begin()` / `end()` sunan her tip kullanılabilir.

`aralikFor.cpp`

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    int notlar[] = {70, 85, 90, 55};
    int toplam{};
    for (int notu : notlar) {
        toplam += notu;
    }
    std::cout << "Toplam: " << toplam << '\n';
```

```
    std::vector<std::string> sehirler{"Ankara", "İzmir", "Bursa"};
    for (const std::string& sehir : sehirler) {
        std::cout << sehir << ' ';
    }
    std::cout << '\n';

    for (char c : std::string("abc")) {
        std::cout << static_cast<int>(c) << ' ';
    }
    std::cout << '\n';

    for (int kuvvet : {1, 2, 4, 8}) {
        std::cout << kuvvet << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
Toplam: 300
Ankara İzmir Bursa
97 98 99
1 2 4 8
```

Dört döngü dört farklı kaynak üzerinde aynı sözdizimini kullandı: C dizisi, `std::vector`, `std::string` ve `{...}` değer listesi. Sınır yazmadık, indeks kullanmadık; bu yüzden tek tek kayma hatası yapma olasılığı yoktur. Kapsayıcıların kendisini Bölüm 26 ve 27'de ayrıntılı işleyeceğiz; burada yalnızca dolaşma biçimine bakıyoruz.

Değer, referans ve const referans

Döngü değişkeninin tipi, elemana nasıl eriştiğimizi belirler. Üç biçim vardır ve aralarındaki fark önemlidir:

Yazım	Anlamı	Ne zaman
`for (int x : kap)`	Her eleman `x`'e **kopyalanır**	Küçük tiplerde (`int`, `char`, `double`) ve değiştirmeyeceksek
`for (int& x : kap)`	`x`, elemanın **kendisidir**	Elemanları değiştirmek istiyorsak
`for (const auto& x : kap)`	Kopyasız, yalnızca okuma	Büyük tiplerde (`std::string` vb.) ve değiştirmeyeceksek

Kopyalama biçiminde döngü değişkenini değiştirmek kapsayıcıyı etkilemez; bu başlangıçta en çok karıştırılan noktadır:

▸ referansFor.cpp

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> sayilar{1, 2, 3, 4};

    for (int x : sayilar) {
        x *= 10;            // kopyayı değiştirir
    }
    for (int x : sayilar) {
        std::cout << x << ' ';
    }
```

```
    std::cout << '\n';

    for (int& x : sayilar) {
        x *= 10;            // elemanın kendisini değiştirir
    }
    for (const auto& x : sayilar) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
1 2 3 4
10 20 30 40
```

İlk döngüde `x` her turda yeni bir kopyadır; 10 ile çarpılan şey kopyadır ve tur bitince yok olur. İkinci döngüde `int& x` elemanın takma adıdır, değişiklik kalıcıdır. Üçüncü döngüde `const auto&` yalnızca okumak için kullanılmıştır: derleyici `x`'i değiştirmeye çalışan bir satırı reddeder.

Tipi `auto` ile de yazabiliriz: `for (auto x : kap)` kopya, `for (auto& x : kap)` referans, `for (const auto& x : kap)` const referanstır (`auto` kuralları Bölüm 22'de anlatıldı). Elemanların tipi uzunsa `auto`, döngüyü okunaklı tutar. Küçük tiplerde `const` referansa gerek yoktur; `int`'in kopyalanması bir adres taşımaktan daha pahalı değildir. Bunu aşağıdaki kuralla özetleyebiliriz.

> ◆ **Kritik Nokta · Hangi biçimi seçmeli?**
>
> Değiştireceksek `auto&`. Değiştirmeyeceksek ve tip büyükse (`std::string`, başka kapsayıcılar, sınıf nesneleri) `const auto&`. Değiştirmeyeceksek ve tip `int`, `double`, `char` gibi küçükse düz `auto`. Büyük bir tipi düz `auto` ile dolaşmak her turda gereksiz bir kopya üretir; kod doğru çalışır ama yavaşlar.

Dizi parametresinde çalışmaz

Aralık tabanlı `for`, dizinin boyutunu bilmesi gerekir. Bir C dizisi fonksiyona parametre olarak geçtiğinde ise boyut bilgisi kaybolur ve dizi, ilk elemanına işaretçiye dönüşür (Bölüm 10 ve 12). İşaretçinin "sonu" olmadığından döngü derlenmez:

▸ diziParametre.cpp

```
#include <iostream>

int topla(int dizi[], int n) {
    int toplam{};
    for (int x : dizi) {
        toplam += x;
    }
    return toplam;
}

int main() {
    int sayilar[] = {1, 2, 3};
    std::cout << topla(sayilar, 3) << '\n';
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

diziParametre.cpp: In function 'int topla(int*, int)':
diziParametre.cpp:5:18: error: 'begin' was not declared in this scope; did you mean
'std::begin'?
    5 |     for (int x : dizi) {
      |                  ^~~~
      |                  std::begin
```

Hata mesajı `begin` adlı bir şey arar: aralık tabanlı `for` aslında kapsayıcının başlangıç ve bitiş yineleyicilerini (`begin()` ve `end()`) kullanır; bir `int*` için bunlar tanımlı değildir. `main` içinde, dizinin tanımlandığı yerde aynı döngü sorunsuz çalışır, çünkü orada derleyici boyutu bilir. Fonksiyonlara dizi geçirmenin C++'a uygun yollarını (`std::array`, `std::vector`, referans) Bölüm 25 ve 26'da göreceğiz.

İndeks gerektiğinde

Aralık tabanlı `for` indeks vermez. İki dizinin karşılıklı elemanlarını işlemek, her ikinci elemanı atlamak ya da elemanın sırasını yazdırmak gibi durumlarda klasik `for` hâlâ doğru araçtır. Ancak kapsayıcı boyutu `size()` işlevinden gelir ve bu işlevin sonucu işaretsizdir (`std::size_t`). İşaretli `int` sayaçla karşılaştırınca derleyici bir uyarı verir:

```
std::vector<int> v{5, 6, 7};
for (int i = 0; i < v.size(); ++i) { }     // uyarı
```

```
TERMİNAL
$ g++ -std=c++17 -Wall indeks.cpp -o indeks
indeks.cpp:5:23: warning: comparison of integer expressions of different signedness: 'int' and '
std::vector<int>::size_type' {aka 'long unsigned int'} [-Wsign-compare]
```

Bölüm 5'te işaretli-işaretsiz karşılaştırma tuzağını görmüştük; bu, onun en sık karşılaşılan hâlidir. Çözüm sayacı `std::size_t` yapmaktır (`for (std::size_t i = 0; i < v.size(); ++i)`). Geriye doğru sayarken ise `std::size_t` tuzağa dönüşebilir (`i >= 0` hiçbir zaman yanlış olmaz); bu durumda `int` sayaç ve `static_cast<int>(v.size())` kullanılır.

> **Not · Döngü sırasında kapsayıcıyı büyütmeyin**
>
> Aralık tabanlı `for` çalışırken aynı `std::vector`'e `push_back` yapmak ya da eleman silmek, döngünün kullandığı yineleyicileri geçersiz kılar ve tanımsız davranışa yol açar. Kapsayıcının boyutunu değiştirmek gerekiyorsa klasik `for` ya da başka bir yöntem kullanılır (Bölüm 26'da yineleyici geçersizleşmesi anlatılıyor).

C++17'de yapısal bağlama (structured binding) denen bir özellik, `for (const auto& [anahtar, deger] : harita)` biçiminde iki parçalı elemanların alanlarını doğrudan adlandırmamızı sağlar; bunu `std::map` ile birlikte Bölüm 35'te göreceğiz. C++20 ise aralık tabanlı `for` başlığına da bir başlatıcı eklenmesine izin verir; kitabın C++17 örneklerinde bunu kullanmıyoruz.

24.8 Döngü Algoritmalarının C++ Sürümleri

Bölüm 8'de faktöriyel, asal sayı testi, EBOB ve Fibonacci gibi klasik algoritmaları yazmıştık. Bunların mantığı değişmez; C++ iki konuda işi kolaylaştırır: ortak işler için hazır fonksiyonlar (`std::gcd`, `std::lcm`) ve sonucu biriktirecek bir `std::vector`. Aşağıdaki program bu araçlarla üç iş yapar:

algoritmalar.cpp

```cpp
#include <iostream>
#include <numeric>
#include <vector>

bool asalMi(int n) {
    if (n < 2) {
        return false;
    }
    for (int bolen = 2; bolen * bolen <= n; ++bolen) {
        if (n % bolen == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    std::vector<int> asallar;
    for (int n = 1; n <= 30; ++n) {
        if (asalMi(n)) {
            asallar.push_back(n);
        }
    }
    std::cout << asallar.size() << " asal: ";
    for (int p : asallar) {
        std::cout << p << ' ';
    }
    std::cout << '\n';

    std::cout << "EBOB(84, 36) = " << std::gcd(84, 36) << '\n';
    std::cout << "EKOK(4, 6)   = " << std::lcm(4, 6) << '\n';

    unsigned long long faktoriyel{1};
    int n{1};
    while (faktoriyel * (n + 1) / (n + 1) == faktoriyel) {
        ++n;
        faktoriyel *= n;
    }
    std::cout << "Taşmadan hesaplanan son faktöriyel: " << n << "! = "
              << faktoriyel << '\n';
    return 0;
}
```

ÇIKTI

```
10 asal: 2 3 5 7 11 13 17 19 23 29
EBOB(84, 36) = 12
EKOK(4, 6)   = 12
Taşmadan hesaplanan son faktöriyel: 20! = 2432902008176640000
```

Program üç şey gösteriyor:

- `asalMi` Bölüm 8'deki sürümle aynıdır; `bolen * bolen <= n` koşulu karekökten sonrasını denememizi sağlar. Bulunan asallar `push_back` ile `asallar` kapsayıcısına eklenir ve aralık tabanlı `for` ile yazdırılır; boyutu önceden bilmemiz gerekmedi.
- `std::gcd` ve `std::lcm`, C++17 ile `<numeric>` başlığına eklenen EBOB ve EKOK fonksiyonlarıdır. Öklid algoritmasını kendimiz yazmak yerine bunları kullanabiliriz; algoritmayı öğrenmek için yazmak ise yine değerlidir (Bölüm 18'de özyinelemeli sürümünü yazdık).

- Faktöriyel döngüsünde taşmayı, bir sonraki çarpımı yapıp geri bölerek denetliyoruz: `faktoriyel * (n + 1)` taşarsa bölme eski değeri vermez. `unsigned long long` taşması tanımlı davranıştır (Bölüm 5), bu yüzden bu denetim güvenlidir. 21! sayısı 64 bite sığmadığından döngü 20!'de durur.

Okuma ve sentinel kalıpları (belirli bir değer girilene ya da girdi bitene kadar okumak) Bölüm 8'deki gibi çalışır, yalnızca `scanf` yerine `std::cin >> x` kullanırız. Şimdi tüm araçları bir araya getiren bir program yazalım.

24.9 Örnek: Not İstatistikleri

Program, kullanıcıdan notları -1 girilene kadar okuyup bir `std::vector`'e yazar; sonra aralık tabanlı `for` ile toplam, en yüksek ve geçen sayısını hesaplar ve `switch` ile ortalamanın harf karşılığını yazdırır. Geçersiz not (0–100 dışı) uyarısıyla atlanır.

▸ notIstatistik.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> notlar;

    std::cout << "Notları girin (bitirmek için -1): ";
    for (int notu{}; std::cin >> notu && notu != -1;) {
        if (notu < 0 || notu > 100) {
            std::cout << notu << " geçersiz, atlandı\n";
            continue;
        }
        notlar.push_back(notu);
    }

    if (notlar.empty()) {
        std::cout << "Hiç not girilmedi\n";
        return 0;
    }

    int toplam{};
    int enYuksek{notlar.front()};
    int gecen{};
    for (int n : notlar) {
        toplam += n;
        if (n > enYuksek) {
            enYuksek = n;
        }
        if (n >= 50) {
            ++gecen;
        }
    }

    double ortalama = static_cast<double>(toplam) / notlar.size();
    std::cout << "Not sayısı: " << notlar.size() << '\n';
    std::cout << "Ortalama: " << ortalama << '\n';
    std::cout << "En yüksek: " << enYuksek << '\n';
    std::cout << "Geçen: " << gecen << '\n';

    switch (int onluk = static_cast<int>(ortalama) / 10; onluk) {
        case 10:
        case 9:
            std::cout << "Harf: A\n";
            break;
        case 8:
```

```
            std::cout << "Harf: B\n";
            break;
        case 7:
            std::cout << "Harf: C\n";
            break;
        default:
            std::cout << "Harf: D\n";
            break;
    }
    return 0;
}
```

```
ÇIKTI
Notları girin (bitirmek için -1): 80 105 90 45 70 -1
105 geçersiz, atlandı
Not sayısı: 4
Ortalama: 71.25
En yüksek: 90
Geçen: 3
Harf: C
```

Programın kilit noktaları şunlardır:

- Okuma döngüsü `for` başlığında yazıldı: başlatıcı `notu` değişkenini tanımlar, koşul hem okur hem sentinel değeri denetler, adım kısmı boştur. Aynı iş `while` ile de yazılabilirdi; `for` burada `notu` değişkeninin kapsamını döngüye hapsettiği için tercih edildi.
- `continue`, geçersiz notu `push_back` satırına varmadan atlar.
- `notlar.size()` işaretsiz bir tamsayıdır; `static_cast<double>(toplam) / notlar.size()` ifadesinde pay `double` olduğu için bölme ondalıklı yapılır (Bölüm 5'teki tamsayı bölmesi tuzağı).
- Ortalama 71,25 olduğundan `static_cast<int>(ortalama) / 10` değeri 7'dir ve `case 7` çalışır. `case 10` ile `case 9` boş etiketlerle aynı koda bağlanmıştır; bu bir düşme sayılmaz.
- Çıktıda ondalık ayırıcı noktadır; `std::cout`, varsayılan olarak 6 anlamlı basamak yazar, bu yüzden `71.25` olduğu gibi görünür.

Üç Dilde · Aralık for, foreach ve switch ifadesi *C · C++ · C#*

C++ (bu bölüm) aralık tabanlı `for` ve klasik `switch` sunar. **C** (Bölüm 8) aralık döngüsüne sahip değildir. **C#** ise `foreach` döngüsüne ve değer üreten `switch` ifadesine (switch expression, C# 8) sahiptir; ayrıntıları Bölüm 44 ve 45'te göreceğiz.

```
/* C: indeksli dolaşma */
for (int i = 0; i < 4; i++) { toplam += dizi[i]; }
```

```
// C++: aralık tabanlı for
for (int x : dizi) { toplam += x; }
```

```
// C#: foreach, aynı fikir
foreach (int x in dizi) { toplam += x; }

// C#: switch ifadesi, değer döndürür
string harf = onluk switch { 10 or 9 => "A", 8 => "B", 7 => "C", _ => "D" };
```

C++ ve C# döngüleri neredeyse aynı kuruluşa sahiptir; fark, C#'ta elemanın koleksiyonun içinde değiştirilemeyen salt okunur bir kopyası olmasıdır (`foreach` değişkenine atama derleme hatasıdır). C++'ta `switch` yalnızca bir

deyimdir ve değer üretmez; aynı işi `?:` zinciri, `if-else` merdiveni ya da bir fonksiyonun `return`'leriyle görürüz. C++'ın başlatıcılı `if`/`switch` özelliğinin C#'ta doğrudan karşılığı yoktur; C#'ta aynı etkiyi desen eşleştirmesindeki `is` ifadesi (`if (x is int n && n > 0)`) verir.

Tarihçe · Aralık tabanlı for nereden geldi?

Aralık tabanlı `for` C++11 ile (2011) dile girdi. O güne kadar bir kapsayıcıyı dolaşmak için yineleyicilerle uzun bir başlık yazılırdı: `for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it)`. C++11'in `auto`'su ve aralık `for`'u bu yazımı bir satıra indirdi. Başlatıcılı `if` ve `switch`, `[[fallthrough]]` niteleyicisi ise C++17'nin eklemeleridir.

Bölüm Özeti

- Koşul ve döngü yapıları (`if`, `else`, `switch`, `while`, `do-while`, `for`, `break`, `continue`, `goto`) C ile aynı çalışır; Bölüm 7 ve 8'deki kurallar C++'ta da geçerlidir.
- Karşılaştırma ve mantıksal operatörlerin sonucu `bool`'dur; `std::cout << a > b` yerine `std::cout << (a > b)` yazılır.
- Koşul olarak akış nesneleri kullanılabilir: `while (std::cin >> x)` okuma başarılı olduğu sürece döner.
- C++17 başlatıcılı `if (başlatıcı; koşul)` ve `switch (başlatıcı; ifade)` biçimleri değişkenin ömrünü ilgili yapıyla sınırlar; değişken `else` kollarında da görünür.
- İçinde deyim bulunan bir `case` düşecekse bu `[[fallthrough]];` ile işaretlenir; boş `case` yığınları düşme sayılmaz.
- C++'ta ilk değer verilen bir değişkenin tanımını atlayan `case` hatadır; çözüm, o `case`'i süslü parantez içine almaktır.
- `switch` tamsayı türü değerlerle çalışır; `case` değerleri derleme zamanında bilinen sabitlerdir.
- Aralık tabanlı `for (tip x : kap)` C dizilerini, `std::vector`, `std::string` ve `{...}` listelerini indekssiz dolaşır.
- Döngü değişkeni `int x` ise kopya, `auto& x` ise elemanın kendisi, `const auto& x` ise kopyasız okuma erişimidir; elemanı değiştirmek için referans, büyük tipleri okumak için `const` referans kullanılır.
- Fonksiyona parametre olarak geçen C dizisi işaretçiye dönüştüğü için aralık tabanlı `for` ile dolaşılamaz.
- İndeks gerekiyorsa klasik `for` kullanılır; `size()` işaretsiz olduğundan sayaç `std::size_t` yapılır.
- Döngü sırasında dolaşılan `std::vector`'ün boyutunu değiştirmek tanımsız davranıştır.
- `<numeric>` içindeki `std::gcd` ve `std::lcm` EBOB ve EKOK hesabını hazır sunar (C++17).
- Ondalık sayıların eşitliği toleransla (`std::abs(x - y) < tolerans`) denetlenir.

Alıştırmalar

1. Aşağıdakilerden hangisi `std::cout` ile `a` ile `b` karşılaştırmasının sonucunu doğru yazdırır?

   A) `std::cout << a > b;`
   B) `std::cout << (a > b);`
   C) `std::cout << a > (b);`

D) `std::cout >> (a > b);`
E) `std::cout(a > b);`

2. Bu programın çıktısı nedir?

```
#include <iostream>
int main() {
    for (int n : {5, 8, 12}) {
        if (int kalan = n % 4; kalan == 0) {
            std::cout << "T ";
        } else if (kalan == 1) {
            std::cout << "B ";
        } else {
            std::cout << kalan << ' ';
        }
    }
    std::cout << '\n';
}
```

3. Başlatıcılı `if (int x = f(); x > 0)` biçiminde tanımlanan `x` değişkeni nerede kullanılabilir?

A) Yalnızca `if` gövdesinde
B) `if` gövdesinde ve `if` koşulunda, `else` kollarında değil
C) Koşulda, gövdede ve bütün `else` kollarında; `if` deyimi bittikten sonra değil
D) `if` deyiminden sonra, fonksiyonun sonuna kadar
E) Yalnızca koşulda

4. `[[fallthrough]];` ne için kullanılır?

A) `switch` içindeki `break`'in yerine geçmek için
B) Bir `case`'in bilerek bir sonraki `case`'e düştüğünü belirtmek için
C) `default` etiketini zorunlu kılmak için
D) Döngüden çıkmak için
E) Sabit olmayan `case` değerlerine izin vermek için

5. Bu programın çıktısı nedir?

```
#include <iostream>
int main() {
    int x{2};
    switch (x) {
        case 1: std::cout << "bir ";
        case 2: std::cout << "iki ";
        case 3: std::cout << "üç "; break;
        default: std::cout << "başka ";
    }
    std::cout << '\n';
}
```

6. Aşağıdaki program neden derlenmez? Düzeltin.

```
#include <iostream>
int main() {
    int k{1};
    switch (k) {
        case 1:
            int yarim = k / 2;
            std::cout << yarim << '\n';
```

```
            break;
        case 2:
            std::cout << "iki\n";
            break;
    }
}
```

7. Bu programın çıktısı nedir?

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v{1, 2, 3};
    for (int x : v) {
        x += 100;
    }
    for (auto& x : v) {
        x *= 2;
    }
    for (int x : v) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
}
```

8. Aşağıdaki durumların hangisinde `for (const auto& s : liste)` biçimi en uygun seçimdir?

   A) `liste` elemanlarını değiştireceğimizde
   B) `liste` bir `std::vector<std::string>` ve yalnızca okuyacağımızda
   C) Elemanın sıra numarasına ihtiyaç duyduğumuzda
   D) Döngü sırasında `liste`'ye eleman ekleyeceğimizde
   E) `liste` fonksiyona `int*` olarak geçirilmiş olduğunda

9. Bu kodda hata var; nedir ve nasıl düzeltilir?

```
int topla(int dizi[], int n) {
    int t = 0;
    for (int x : dizi) {
        t += x;
    }
    return t;
}
```

10. `std::vector<int> v{4, 5, 6};` için `for (int i = 0; i < v.size(); ++i)` döngüsü derleyiciden hangi uyarıyı alır ve neden?

    A) Kullanılmayan değişken uyarısı
    B) İşaretli ve işaretsiz tamsayıların karşılaştırılması (`-Wsign-compare`)
    C) Eksik `return`
    D) Dizi sınırı aşımı
    E) Hiçbir uyarı

11. Bu programın çıktısı nedir?

```
#include <iostream>
#include <string>
int main() {
    for (int i = 1; i <= 3; ++i) {
```

```
        std::cout << std::string(i, '#') << std::string(4 - i, '.') << '\n';
    }
}
```

12. `std::cout << a == b;` ifadesi neden beklenen sonucu vermez? Doğru yazımı nedir?
13. Bu programın çıktısı nedir?

```
#include <iostream>
#include <numeric>
int main() {
    std::cout << std::gcd(18, 24) << ' ' << std::lcm(3, 5) << '\n';
}
```

14. Kısa program: Kullanıcıdan 0 girilene kadar tamsayılar okuyun; `std::vector<int>`'te saklayın. Sonra aralık tabanlı `for` ile pozitif sayıların toplamını ve negatif sayıların adedini yazdırın. (0 vektöre eklenmesin.)
15. Kısa program: Başlatıcılı `if` kullanarak kullanıcıdan bir tamsayı okuyun. Okuma başarılıysa sayının çift mi tek mi olduğunu, başarısızsa "Geçersiz girdi" yazdırın.
16. Kısa program: `std::vector<std::string>` içindeki dört şehir adını `const auto&` ile yazdırın; ardından uzunluğu 5'ten büyük olanların sayısını bulun. (İpucu: `s.size()` bir metnin uzunluğunu verir; `std::string` Bölüm 27'de ayrıntılıdır.)

BÖLÜM 25

Fonksiyonlar: C++'ın Getirdikleri

BU BÖLÜMDE

- C'de öğrendiğimiz fonksiyon bilgisinin C++'ta nelerinin aynı kaldığını, nelerinin değiştiğini (`f()`, prototip zorunluluğu) söylemeyi
- Referans parametreyle çağıranın değişkenini değiştirmeyi ve bunu Bölüm 12'deki işaretçili çözümle karşılaştırmayı
- Birden çok değeri çıkış parametresiyle, `struct` ya da `std::pair` ile (yapısal bağlama dahil) döndürmeyi
- `const &` ile büyük nesneleri kopyalamadan ve değiştirmeden geçirmeyi; parametre tipini dört soruyla seçmeyi
- Varsayılan parametreleri ve fonksiyon aşırı yüklemeyi; derleyicinin aday seçme sırasını ve belirsiz çağrıyı açıklamayı
- `inline`, `constexpr`, dönüş tipi `auto` ve `[[nodiscard]]` ile fonksiyon şablonu, fonksiyon işaretçisi, `std::function` ve lambda önizlemesini tanımayı

Bölüm 9'da fonksiyonun anatomisini, prototipi, değerle geçirmeyi, çağrı yığınını, kapsamı ve `static` yerel değişkeni ayrıntılı gördük. Bu bilgilerin hepsi C++'ta aynen geçerlidir; bu bölüm onları yeniden anlatmayacak. Bunun yerine C++'ın fonksiyonlara kattığı araçlara odaklanacağız.

Bu araçların çoğu, C'de ya işaretçiyle ve makroyla dolambaçlı yoldan yaptığımız ya da hiç yapamadığımız işleri doğrudan dile yerleştirir. Referans parametre, işaretçi yıldızlarını ortadan kaldırır. Aşırı yükleme, `topla` ile `toplaD` gibi ad uydurma zorunluluğunu bitirir. Varsayılan parametre, aynı fonksiyonun kısa ve uzun hâllerini tek tanımda toplar. Bölümün sonunda bu araçları birlikte kullanan küçük bir program yazacak, C#'ın `ref`, `out` ve `in` anahtar sözcükleriyle de karşılaştıracağız. Örneklerde, Bölüm 22'deki gibi `std::` önekini açıkça yazıyoruz.

25.1 C'den Gelenler: Hızlı Bakış

Aşağıdaki tablo, Bölüm 9'daki başlıkların C++'taki durumunu özetler:

Konu	C++'ta durum
Tanım, `return`, `void` fonksiyon, erken çıkış	Aynı
Parametre ve argüman, örtük tip dönüşümü	Aynı; `(a + b) / 2` yine tamsayı bölmesidir
Değerle geçirme, çağrı yığını	Aynı

Konu	C++'ta durum
Yerel/global kapsam, gölgeleme, `static` yerel	Aynı; gölgelenen globale `::ad` ile ulaşılır (Bölüm 22)
Prototipsiz çağrı	C++'ta her zaman **hata**; C'de C99'dan beri yalnızca uyarı ya da hata
`int f();`	C++'ta "parametresiz" demektir, `int f(void);` ile aynıdır. C'de (C17'ye kadar) "parametreleri belirtilmemiş" anlamına gelir
`main` sonunda `return 0;`	İsteğe bağlı; yalnızca `main` için geçerli bir özel kural (C99'dan beri C'de de)
`main`'i kendi koddan çağırmak	C++'ta yasak; C'de izinli
Aynı adlı birden çok fonksiyon	C++'ta serbest (aşırı yükleme); C'de yok
Varsayılan parametre	C++'ta var; C'de yok
Referansla geçirme	C++'ta var; C'de yalnızca işaretçiyle

Aşırı yükleme, varsayılan parametre ve referans farklarını sırayla işleyeceğiz. Önce `int f();` farkını görelim. C'de `int f();` bildirimi çağrıda herhangi bir argüman sayısını kabul ederdi; C++'ta derleyici parametre listesini kesin sayar:

▸ bosParametre.cpp

```cpp
int f();

int main() {
    return f(5);
}
```

DERLEYİCİ ÇIKTISI

```
bosParametre.cpp: In function 'int main()':
bosParametre.cpp:4:13: error: too many arguments to function 'int f()'
    4 |     return f(5);
      |            ~^~~
bosParametre.cpp:1:5: note: declared here
    1 | int f();
      |     ^
```

Fonksiyonların kapsam ve ömür kuralları değişmediği için çağrı yığınını ve `static` yerel değişkenleri yeniden işlemeyeceğiz. Hazır fonksiyonlar da aynıdır; yalnızca başlık adları değişir: `<math.h>` yerine `<cmath>`, `<stdlib.h>` yerine `<cstdlib>` yazılır ve adlar `std` ad alanından gelir (`std::sqrt`, `std::rand`).

25.2 Referansla Geçirme

C'de çağıranın değişkenini değiştirmek için fonksiyona değişkenin adresini verirdik (Bölüm 12). Fonksiyon içinde her kullanımda `*` ile dolaylama yapılır, çağrıda `&` yazılırdı. C++ aynı işi referansla çok daha sade yapar.

Bölüm 22'de gördüğümüz gibi referans, var olan bir değişkene verilen ikinci bir addır. Parametreyi tipinin yanına `&` koyarak tanımladığımızda, parametre çağıran fonksiyondaki değişkenin kopyası değil **kendisi** olur:

Tanım: Referans parametre

Tipinin yanında `&` bulunan parametredir (`int& a`). Fonksiyon çağrılırken argümanın bir kopyası alınmaz; parametre, argüman olarak verilen değişkene bağlanan bir takma ad olur. Parametreye yapılan her atama çağıranın değişkenini değiştirir. Buna **referansla geçirme** (pass by reference) denir.

Takas işlevini önce işaretçiyle, sonra referansla yazıp karşılaştıralım:

takasKarsilastir.cpp

```cpp
#include <iostream>

void takasIsaretci(int* a, int* b) {
    int gecici = *a;
    *a = *b;
    *b = gecici;
}

void takasReferans(int& a, int& b) {
    int gecici = a;
    a = b;
    b = gecici;
}

int main() {
    int x{3};
    int y{8};

    takasIsaretci(&x, &y);
    std::cout << "isaretci : x = " << x << ", y = " << y << '\n';

    takasReferans(x, y);
    std::cout << "referans : x = " << x << ", y = " << y << '\n';
    return 0;
}
```

```
ÇIKTI
isaretci : x = 8, y = 3
referans : x = 3, y = 8
```

İki fonksiyon aynı işi yapar. Referanslı sürümde gövdede yıldız, çağrıda `&` yoktur; fonksiyonun gövdesi, değerle geçirilen takasın (Bölüm 9'da çalışmadığını gördüğümüz sürümün) birebir aynısıdır. Tek fark parametrelerdeki `&` işaretleridir.

Bu sadelik bir bedelle gelir: `takasReferans(x, y)` çağrısına bakarak `x` ve `y`'nin değişip değişmeyeceğini anlayamayız. Bunu görmek için fonksiyonun bildirimine bakmak gerekir. İşaretçili sürümde `&x` ifadesi, "bu değişken değişebilir" diye çağrıda görünürdü. Bu yüzden bazı kod rehberleri çıkış parametreleri için işaretçiyi tercih eder; bu kitapta referansı kullanacağız ve parametre adlarını rolünü belli edecek biçimde seçeceğiz.

Referans parametreye bir **değişken** vermemiz gerekir. Değiştirilecek bir kutu bulunmayan bir sabit ya da hesap sonucu verirsek derleyici durdurur:

takasHata.cpp

```cpp
#include <iostream>

void takasReferans(int& a, int& b) {
    int gecici = a;
    a = b;
```

```
    b = gecici;
}

int main() {
    int x{3};
    takasReferans(x, 5);
    std::cout << x << '\n';
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

takasHata.cpp: In function 'int main()':
takasHata.cpp:11:22: error: cannot bind non-const lvalue reference of type 'int&' to an rvalue
of type 'int'
   11 |     takasReferans(x, 5);
      |                   ^
takasHata.cpp:3:33: note:   initializing argument 2 of 'void takasReferans(int&, int&)'
    3 | void takasReferans(int& a, int& b) {
      |                            ~~~~~^
```

Hata iletisindeki "lvalue" bellekte adı ya da adresi olan, atamanın sol tarafına gelebilen ifadedir (değişkenler gibi); "rvalue" ise `5` ya da `a + b` gibi geçici değerdir. Bu terimleri Bölüm 28 ve Bölüm 39'da ayrıntılı işleyeceğiz. Şimdilik şunu bilelim: `int&` parametresi yalnızca adı olan bir değişkene bağlanır.

Çıkış parametreleri

`return` en fazla bir değer taşır. Bir bölme işleminin hem bölümünü hem kalanını döndürmek istiyorsak, sonuçları koyacağımız kutuları referans parametre olarak verebiliriz. Bu parametrelere **çıkış parametresi** (output parameter) denir:

bolVeKalan.cpp

```
#include <iostream>

void bolVeKalan(int bolunen, int bolen, int& bolum, int& kalan) {
    bolum = bolunen / bolen;
    kalan = bolunen % bolen;
}

int main() {
    int b{0};
    int k{0};
    bolVeKalan(47, 5, b, k);
    std::cout << "47 / 5 = " << b << ", kalan " << k << '\n';
    return 0;
}
```

```
ÇIKTI
47 / 5 = 9, kalan 2
```

İlk iki parametre girdidir ve değerle geçer; son ikisi çıktıdır ve referansla geçer. Bu yöntem çalışır, ama çağrı satırı ağırdır: sonuçları almak için önce boş değişkenler tanımlamamız gerekir. C++'ta çoğu zaman daha iyi bir yol vardır.

Birden çok değeri döndürmek

Sonuçları tek pakette döndürmenin iki yaygın yolu vardır. Biri ihtiyaca özel bir `struct` (Bölüm 14) tanımlamak, diğeri standart kütüphanedeki `std::pair`'dir (`<utility>`). `std::pair<A, B>` iki değeri `first` ve `second` üyeleriyle taşır; ayrıntısını Bölüm 35'te göreceğiz. C++17 ile gelen **yapısal bağlama** (structured binding) ise dönen paketi tek satırda adlandırılmış değişkenlere açar:

birdenCokDeger.cpp

```cpp
#include <iostream>
#include <utility>

struct Ozet {
    int enKucuk;
    int enBuyuk;
    double ortalama;
};

Ozet ozetle(int a, int b, int c) {
    Ozet o{a, a, 0.0};
    if (b < o.enKucuk) o.enKucuk = b;
    if (c < o.enKucuk) o.enKucuk = c;
    if (b > o.enBuyuk) o.enBuyuk = b;
    if (c > o.enBuyuk) o.enBuyuk = c;
    o.ortalama = (a + b + c) / 3.0;
    return o;
}

std::pair<int, int> bolme(int bolunen, int bolen) {
    return {bolunen / bolen, bolunen % bolen};
}

int main() {
    auto [kucuk, buyuk, ort] = ozetle(7, 2, 10);
    std::cout << kucuk << " " << buyuk << " " << ort << '\n';

    auto [bolum, kalan] = bolme(47, 5);
    std::cout << bolum << " " << kalan << '\n';

    std::pair<int, int> p = bolme(20, 6);
    std::cout << p.first << " " << p.second << '\n';
    return 0;
}
```

```
ÇIKTI
2 10 6.33333
9 2
3 2
```

`auto [kucuk, buyuk, ort] = ...` satırı üç değişkeni bir adımda tanımlar ve dönen yapının üyeleriyle sırayla başlatır. Çağıran taraf boş değişken hazırlamak zorunda kalmaz. Kural olarak şunu izleriz: fonksiyon bir iş sonucunu hesaplıyorsa onu `return` ile döndürürüz; çıkış parametresini yalnızca büyük bir nesneyi yerinde doldurmak gibi özel durumlarda kullanırız.

Bu yaklaşım C'de de `struct` döndürerek yapılabilirdi, ama `auto [...]` yazımı ve `std::pair` C++'a özgüdür. C#'ın karşılığı demetlerdir (`(int, int)`); bölümün sonundaki karşılaştırmada göreceğiz.

Referans döndürmek

Fonksiyon referans da döndürebilir. Bu durumda çağrı ifadesi, dönen değişkenin kendisinin yerine geçer ve atamanın sol tarafına bile yazılabilir:

▸ referansDonus.cpp

```cpp
#include <iostream>

int& buyukOlan(int& a, int& b) {
    return (a > b) ? a : b;
}

int main() {
    int x{4};
    int y{9};
    buyukOlan(x, y) = 0;
    std::cout << "x = " << x << ", y = " << y << '\n';
    return 0;
}
```

ÇIKTI

```
x = 4, y = 0
```

`buyukOlan(x, y)` büyük olan değişkene, yani `y`'ye bağlandı ve ona 0 atandı. Bu yetenek, ileride sınıfların `operator[]` gibi üye fonksiyonlarında (Bölüm 32) işimize yarayacak. Bir koşulu vardır: dönen referansın gösterdiği değişken fonksiyon bittikten sonra da yaşamalıdır. Burada `x` ve `y` çağıranın değişkenleri olduğu için sorun yoktur.

▲ Sık Yapılan Hata · Yerel değişkene referans döndürmek

Fonksiyon bitince yerel değişkenlerin çerçevesi silinir (Bölüm 9). Bir yerel değişkenin referansını döndürmek, ortadan kalkmış bir kutuyu göstermektir:

```cpp
int& hatali() {
    int x{5};
    return x;          // x fonksiyonla birlikte yok olur
}
```

Bu kod tanımsız davranıştır (Bölüm 5); çökebilir, çöp değer verebilir ya da tesadüfen çalışıyormuş gibi görünebilir. Bu yüzden çıktısını göstermiyoruz. `g++ -Wall` bunu çoğu zaman yakalar:

TERMİNAL

```
$ g++ -std=c++17 -Wall hatali.cpp -c
hatali.cpp: In function 'int& hatali()':
hatali.cpp:3:12: warning: reference to local variable 'x' returned [-Wreturn-local-addr]
    3 |     return x;          // x fonksiyonla birlikte yok olur
      |            ^
hatali.cpp:2:9: note: declared here
    2 |     int x{5};
      |         ^
```

Referansı yalnızca fonksiyon bittikten sonra da yaşayan bir şeyi (parametre olarak aldığımız bir değişkeni, bir dizi elemanını, bir üye değişkeni) geri vermek için kullanırız. Şüphedeyken değerle döndürürüz.

25.3 const & ile Geçirme

Değerle geçirme küçük tiplerde (`int`, `double`, `char`, `bool`, işaretçiler) hızlı ve zararsızdır. Büyük bir nesnede ise her çağrıda koca bir kopya çıkar. Aşağıdaki programda 500 `double` ve 64 karakterlik bir başlık tutan `Rapor` yapısını iki biçimde geçiriyoruz:

kopyaMaliyeti.cpp

```
#include <iostream>

struct Rapor {
    char baslik[64];
    double olcum[500];
};

double toplamDeger(Rapor r) {              // her çağrıda tüm yapı kopyalanır
    double t = 0.0;
    for (double d : r.olcum) t += d;
    return t;
}

double toplamReferans(const Rapor& r) {    // kopya yok, değiştirme de yok
    double t = 0.0;
    for (double d : r.olcum) t += d;
    return t;
}

int main() {
    Rapor rapor{};
    for (int i = 0; i < 500; i++) rapor.olcum[i] = i;

    std::cout << "Rapor boyutu: " << sizeof(Rapor) << " bayt\n";
    std::cout << toplamDeger(rapor) << '\n';
    std::cout << toplamReferans(rapor) << '\n';
    return 0;
}
```

```
ÇIKTI
Rapor boyutu: 4064 bayt
124750
124750
```

İki fonksiyon aynı sonucu verir; fark, `toplamDeger` çağrısında 4064 baytın kopyalanmasıdır. `const Rapor& r` parametresi ise yalnızca bir bağlantı taşır. Döngüdeki `for (double d : r.olcum)` yazımı aralık tabanlı `for`'dur; Bölüm 24'te işledik.

Referansla geçirmenin kopyayı ortadan kaldırdığını gördük; ama bu kez fonksiyon asıl nesneyi yanlışlıkla değiştirebilir. Çözüm referansı `const` yapmaktır:

> **Tanım: Sabit referans (const reference)**
>
> `const T&` tipindeki parametredir. Argümanın kopyası alınmaz, ama fonksiyon argümanı değiştiremez. Değiştirmeye çalışan her kod derleme hatası verir. Üstelik sabit referansa geçici değerler (sabitler, hesap sonuçları) de verilebilir.

Bu son özellik, `int&`'dan önemli bir farktır. `std::string` üzerinde görelim (`std::string`'i Bölüm 27'de ayrıntılı işleyeceğiz; şimdilik "metin tutan tip" olarak alalım):

sabitReferans.cpp

```cpp
#include <iostream>
#include <string>

void selamla(const std::string& ad) {
    std::cout << "Merhaba " << ad << "!\n";
}

int main() {
    std::string isim{"Ayşe"};
    selamla(isim);               // değişken
    selamla("Mehmet");           // sabit metin: geçici bir std::string oluşur
    selamla(isim + " Hanım");    // hesap sonucu da olur
    return 0;
}
```

ÇIKTI

```
Merhaba Ayşe!
Merhaba Mehmet!
Merhaba Ayşe Hanım!
```

`selamla("Mehmet")` çağrısında derleyici metin sabitinden geçici bir `std::string` yaratır ve parametreyi ona bağlar; geçici nesne çağrı bitene kadar yaşar. Fonksiyon bu nesneyi değiştiremeyeceği için işlem güvenlidir. Değiştirmeye kalkarsak derleyici bizi durdurur:

constHata.cpp

```cpp
#include <iostream>

void artir(const int& n) {
    n = n + 1;
}

int main() {
    int x{1};
    artir(x);
    std::cout << x << '\n';
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
constHata.cpp: In function 'void artir(const int&)':
constHata.cpp:4:7: error: assignment of read-only reference 'n'
    4 |     n = n + 1;
      |     ~~^~~~~~
```

Parametre tipini seçmek

Bir parametre için hangi biçimi seçeceğimize dört soruyla karar verebiliriz:

İpucu · Dört soruyla parametre tipi

1. Fonksiyon bu değeri değiştirip çağırana yansıtacak mı? Evet ise `T&`.
2. Hayır, ama tip büyük mü (`std::string`, `std::vector`, kendi `struct`'larımız)? Evet ise `const T&`.
3. Tip küçük mü (`int`, `double`, `char`, `bool`, işaretçi)? Düz değer: `T`.
4. Fonksiyon kendi içinde bir kopyayı değiştirip kullanacak mı (Bölüm 9'daki `ebob` örneği gibi)? Düz değer: `T`.

Dört yolu yan yana koyalım:

Yöntem	Parametre	Çağrı	Kopya var mı?	Çağıranı değiştirir mi?	Geçici değer verilebilir mi?	Boş (null) olabilir mi?
Değerle	`int n`	`f(x)`	Evet	Hayır	Evet	Hayır
İşaretçiyle	`int* p`	`f(&x)`	Yalnızca adres	Evet	Hayır	Evet
Referansla	`int& r`	`f(x)`	Hayır	Evet	Hayır	Hayır
Sabit referansla	`const int& r`	`f(x)`	Hayır	Hayır	Evet	Hayır

İşaretçi, yalnızca "değer yok" durumunun anlamlı olduğu yerde gerekir; örneğin isteğe bağlı bir çıkış parametresi `nullptr` ile "istemiyorum" diyebilir. Referans hiçbir zaman boş olamaz ve bu, fonksiyonun içinde `nullptr` kontrolü yapma ihtiyacını ortadan kaldırır. İşaretçi ile referansı Bölüm 28'de ayrıntılı karşılaştıracağız.

Dizileri ve kapsayıcıları geçirmek

C dizileri C++'ta da fonksiyona geçirilirken ilk elemanın adresine dönüşür (Bölüm 10 ve 12) ve eleman sayısını ayrıca vermemiz gerekir. Standart kütüphanenin `std::array` ve `std::vector` tipleri (Bölüm 26) ise tek bir nesnedir; boyutlarını kendileri bilir. Büyük nesneler oldukları için `const` referansla geçirilirler:

```
#include <array>
#include <vector>

double ortalama(const int* dizi, std::size_t n);            // C tarzı: adres + boyut
double ortalama(const std::vector<int>& sayilar);           // kopyalama yok, boyut içinde
double ortalama(const std::array<int, 5>& sayilar);         // boyut tipin parçası
```

Üç bildirim aynı adı taşır; bunun nasıl mümkün olduğunu aşırı yükleme başlığında göreceğiz. C++20'de eklenen `std::span`, bu üç biçimi tek bir parametre tipinde toplar; ancak kitabın C++17 kodlarında kullanmayacağız.

25.4 Varsayılan Parametreler

Bazı parametreler çağrıların çoğunda aynı değeri alır. Her seferinde o değeri yazmak yerine parametreye bir **varsayılan değer** verebiliriz; çağrıda o argüman atlanırsa varsayılan kullanılır:

Tanım: Varsayılan parametre (default argument)

Fonksiyon bildiriminde `parametre = değer` biçiminde yazılan ve çağrıda karşılığı verilmezse kullanılan değerdir. Çağıran taraf yalnızca sondaki argümanları atlayabilir.

varsayilan.cpp

```cpp
#include <iostream>

double kdvliFiyat(double fiyat, double oran = 0.20) {
    return fiyat * (1 + oran);
}

void cizgiCiz(int uzunluk = 10, char karakter = '-') {
    for (int i = 0; i < uzunluk; i++) {
        std::cout << karakter;
    }
    std::cout << '\n';
}

int main() {
    std::cout << kdvliFiyat(100) << '\n';
    std::cout << kdvliFiyat(100, 0.10) << '\n';
    cizgiCiz();
    cizgiCiz(5);
    cizgiCiz(8, '*');
    return 0;
}
```

```
ÇIKTI
120
110
----------
-----
********
```

`kdvliFiyat(100)` çağrısında `oran` için 0,20 kullanıldı. `cizgiCiz()` iki varsayılanı birden kullandı. Bu çağrılar derleyici tarafından tam argüman listesine genişletilir: `cizgiCiz(5)` aslında `cizgiCiz(5, '-')` olarak çalışır. Çağrı sırasında hiçbir karar verilmez, yani varsayılan parametre çalışma zamanında bir maliyet getirmez.

Varsayılan parametrenin kuralları şunlardır:

- Varsayılan değerler **sağdan sola** verilir: bir parametrenin varsayılanı varsa, onun sağındaki bütün parametrelerin de olmalıdır.
- Argümanlar soldan sağa eşlenir. Ortadaki bir argümanı atlayıp sondakini verme yolu yoktur: `cizgiCiz('*')` yazarsak `'*'` karakteri `uzunluk` parametresine (`int`) gider ve 42 tire çizilir. Yalnızca karakteri değiştirmek istiyorsak uzunluğu da yazmalıyız. (C#'ta adlandırılmış argümanlar bu sorunu çözer; bölüm sonunda göreceğiz.)
- Varsayılan değer sabit olmak zorunda değildir; çağrı sırasında her seferinde hesaplanan herhangi bir ifade olabilir (başka bir fonksiyonun çağrısı gibi).
- Varsayılan değer yalnızca **bir** yerde, derleyicinin ilk gördüğü bildirimde yazılır. Yukarıdaki programda prototip olmadığı için tanımın kendisidir. Prototip varsa varsayılanı prototipe yazarız, tanımda tekrarlamayız:

```cpp
double kdvliFiyat(double fiyat, double oran = 0.20);    // prototip: varsayılan burada

double kdvliFiyat(double fiyat, double oran) {          // tanım: tekrar yok
    return fiyat * (1 + oran);
}
```

Çok dosyalı programlarda (Bölüm 15) prototip başlık dosyasında durur; bu yüzden varsayılanlar da başlık dosyasında yazılır.

Sağdan sola kuralını bozarsak derleyici durdurur:

varsayilanHata.cpp

```cpp
int topla(int a = 1, int b) {
    return a + b;
}

int main() {
    return topla(2, 3);
}
```

```
DERLEYİCİ ÇIKTISI

varsayilanHata.cpp:1:26: error: default argument missing for parameter 2 of 'int topla(int,
int)'
    1 | int topla(int a = 1, int b) {
      |                      ~~~~^
varsayilanHata.cpp:1:15: note: ...following parameter 1 which has a default argument
    1 | int topla(int a = 1, int b) {
      |           ~~~~^~~~~
```

C'de bu özellik yoktur; aynı etkiyi iki ayrı fonksiyonla ya da "varsayılan için `0` ya da `NULL` ver" sözleşmesiyle sağlardık. C++'ta varsayılan parametre, aynı işin kısa ve uzun hâllerini tek tanımda toplar.

25.5 Fonksiyon Aşırı Yükleme

İki sayının büyüğünü bulan bir fonksiyonu `int` ve `double` için ayrı ayrı yazmamız gerekirse C'de iki ayrı ad uydururduk (`enBuyuk` ve `enBuyukD`; Bölüm 9'daki üç dil kutusunu hatırlayalım). C++'ta aynı adı taşıyan, ama parametre listeleri farklı olan birden çok fonksiyon tanımlayabiliriz:

> **Tanım: Aşırı yükleme (overloading)**
>
> Aynı kapsamda aynı adı taşıyan, parametre listeleri (sayı, tip ya da sıra bakımından) birbirinden farklı birden çok fonksiyon tanımlamaktır. Derleyici, çağrıdaki argümanlara bakarak hangisinin çağrılacağını **derleme zamanında** seçer. Adla birlikte parametre listesi, fonksiyonun **imzasını** (signature) oluşturur; dönüş tipi imzaya girmez.

asiriYukleme.cpp

```cpp
#include <iostream>

int enBuyuk(int a, int b) {
    return (a > b) ? a : b;
}

double enBuyuk(double a, double b) {
    return (a > b) ? a : b;
}

int enBuyuk(int a, int b, int c) {
    return enBuyuk(enBuyuk(a, b), c);     // iki parametreli sürümü çağırır
}

int main() {
```

```
    std::cout << enBuyuk(3, 8) << '\n';
    std::cout << enBuyuk(2.5, 1.5) << '\n';
    std::cout << enBuyuk(4, 9, 6) << '\n';
    return 0;
}
```

```
ÇIKTI
8
2.5
9
```

Üç `enBuyuk` ayrı fonksiyondur. `enBuyuk(3, 8)` iki `int` argümanı olduğu için ilk sürüme, `enBuyuk(2.5, 1.5)` ikinciye, üç argümanlı çağrı üçüncüye gider. Aşırı yükleme, ilgili işleri aynı adla çağırmamıza izin verdiği için arayüzü sadeleştirir: `std::abs` hem `int` hem `double` için, `std::cout <<` her tip için çalışır. Operatörlerin aşırı yüklenmesini Bölüm 32'de, sınıfların kurucularının aşırı yüklenmesini Bölüm 30'da göreceğiz.

Aday seçimi

Çağrıda tam uyan bir fonksiyon yoksa derleyici "en iyi" adayı şu sırayla arar:

1. **Tam eşleşme**: tipler aynıdır (değerden `const` referansa bağlama gibi küçük ayarlamalar da buraya girer).
2. **Terfi** (promotion): `char`, `short` ve `bool` değerlerinin `int`'e, `float`'ın `double`'a yükseltilmesi (Bölüm 5'teki tamsayı yükseltmesi).
3. **Standart dönüşüm**: `int` → `double`, `double` → `int`, `long` → `int` gibi diğer dönüşümler.
4. **Kullanıcı tanımlı dönüşüm**: sınıflardaki dönüşümler (Bölüm 30 ve sonrası).

Derleyici en iyi düzeydeki adayı alır. Aynı düzeyde iki aday varsa çağrı **belirsizdir** ve program derlenmez. Önce terfinin etkisini görelim:

yuklemeSecimi.cpp

```
#include <iostream>

void yaz(int x) {
    std::cout << "int: " << x << '\n';
}

void yaz(double x) {
    std::cout << "double: " << x << '\n';
}

int main() {
    yaz(5);        // tam eşleşme: int
    yaz(5.5);      // tam eşleşme: double
    yaz('A');      // char -> int terfisi, double'a dönüşümden iyidir
    yaz(2.5f);     // float -> double terfisi
    yaz(true);     // bool -> int terfisi
    return 0;
}
```

```
ÇIKTI
int: 5
double: 5.5
int: 65
double: 2.5
int: 1
```

`yaz('A')` karakteri değil 65 sayısını yazdırır: `char` → `int` terfi sayılır ve `char` → `double` dönüşümünden daha iyidir. Şimdi iki adayın da dönüşüm gerektirdiği bir çağrıya bakalım. `long` tipindeki `5L`, ne `int`'e ne `double`'a tam uyar; ikisine de standart dönüşüm gerekir ve ikisi eşit derecede iyidir:

yuklemeBelirsiz.cpp

```cpp
#include <iostream>

void yaz(int x) {
    std::cout << "int: " << x << '\n';
}

void yaz(double x) {
    std::cout << "double: " << x << '\n';
}

int main() {
    yaz(5L);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
yuklemeBelirsiz.cpp: In function 'int main()':
yuklemeBelirsiz.cpp:12:8: error: call of overloaded 'yaz(long int)' is ambiguous
   12 |     yaz(5L);
      |     ~~~^~~~
yuklemeBelirsiz.cpp:3:6: note: candidate: 'void yaz(int)'
    3 | void yaz(int x) {
      |      ^~~
yuklemeBelirsiz.cpp:7:6: note: candidate: 'void yaz(double)'
    7 | void yaz(double x) {
      |      ^~~
```

Çözüm, çağrıda dönüşümü açıkça yazmaktır: `yaz(static_cast<int>(5L))`. Derleyicinin iletisi adayları sıraladığı için hangi fonksiyonlar arasında kaldığını da söyler.

▲ Sık Yapılan Hata · Metin sabiti bool'a gider

Aşırı yükleme kurallarının en şaşırtıcı sonucu şudur: bir `bool` ve bir `std::string` sürümü varsa, `"merhaba"` gibi bir metin sabiti `std::string`'e değil `bool`'a gider. Metin sabiti önce işaretçiye dönüşür, işaretçiden `bool`'a dönüşüm standart dönüşümdür; `std::string`'e dönüşüm ise kullanıcı tanımlı olduğu için daha kötü sayılır.

yuklemeTuzak.cpp

```cpp
#include <iostream>
#include <string>

void goster(bool deger) {
    std::cout << "bool\n";
}

void goster(const std::string& metin) {
    std::cout << "string: " << metin << '\n';
}

int main() {
    goster("merhaba");
    goster(std::string{"merhaba"});
    return 0;
}
```

ÇIKTI

```
bool
string: merhaba
```

Bu, derleyicinin hatası değil, kuralların sonucudur. Böyle bir arayüz tasarlarken `bool` parametreli sürümden kaçınmak ya da bir `const char*` sürümü eklemek gerekir.

Aşırı yüklemenin sınırları

Aşağıdakiler birbirinden **ayrı** sürüm sayılmaz; aynı adla birlikte tanımlanırsa derleme hatası verir:

- Yalnızca **dönüş tipi** farklı olan fonksiyonlar (`int yarisi(int)` ile `double yarisi(int)`): `yarisi(7)` çağrısında hangisinin istendiği belli olmaz.
- Yalnızca değerle geçirilen parametrede `const` farkı olanlar (`f(int)` ile `f(const int)`): fonksiyonun içindeki `const` çağıranı ilgilendirmez, imza aynıdır.
- Aynı tipin başka adı (`typedef` / `using` takma adı) kullanılan sürümler.

Buna karşılık referans ve işaretçi parametrelerinde `const` imzayı değiştirir: `f(int&)` ile `f(const int&)` iki ayrı fonksiyondur. `f(x)` (değişken) için birincisi, `f(5)` (geçici değer) için ikincisi seçilir.

Varsayılan parametreler de belirsizlik doğurabilir. `f(int a)` ile `f(int a, int b = 0)` birlikte tanımlanırsa `f(1)` çağrısı iki fonksiyona da uyar ve derlenmez. Aşırı yükleme ile varsayılan parametreyi aynı fonksiyon ailesinde birlikte kullanırken bu çakışmaya dikkat etmek gerekir.

İsim bozma (name mangling)

C'de bir fonksiyonun bağlayıcıya (linker) görünen adı, kaynak koddaki adıyla aynıdır. Aynı adlı iki fonksiyon olamayacağı için bu yeterliydi. C++'ta ise aynı ada sahip birden çok fonksiyon bulunur; bağlayıcı bunları ayırt edebilmelidir. Derleyici bunu fonksiyonun adını parametre tiplerini de içeren bir ada çevirerek yapar. Bu işleme **isim bozma** (name mangling) denir.

Üç `topla` sürümü içeren bir dosyayı derleyip nesne dosyasındaki simgeleri `nm` aracıyla listeleyelim:

```
TERMİNAL
$ g++ -std=c++17 -c topla.cpp
$ nm topla.o
0000000000000018 T _Z5topladd
0000000000000000 T _Z5toplaii
0000000000000036 T _Z5toplaiii
$ nm -C topla.o
0000000000000018 T topla(double, double)
0000000000000000 T topla(int, int)
0000000000000036 T topla(int, int, int)
```

`_Z5toplaii` adında `_Z` C++ simgesini, `5topla` uzunluğu 5 olan "topla" adını, sonraki harfler parametre tiplerini (`i` int, `d` double) gösterir. `-C` seçeneği bu adları insan okuyabilir hâle geri çevirir. Bu düzen Linux ve macOS'taki derleyicilerin (GCC ve Clang) kullandığı kuraldır; Visual C++ başka bir bozma düzeni kullanır. Ayrıntıyı ezberlemek gerekmez, ama iki sonucu bilmeliyiz:

- Bir hata iletisinde ya da araç çıktısında `_Z...` ile başlayan bir ad görürsek, bu bir C++ fonksiyonunun bozulmuş adıdır. Bağlayıcı hatalarında (`undefined reference to ...`) sık karşılaşılır (Ek D).
- C++ kodundan C ile yazılmış bir kütüphaneyi çağırırsak, derleyicinin C kütüphanesindeki `topla` adını `_Z5toplaii` diye aramasını engellememiz gerekir. Bunun için bildirimleri `extern "C"` ile işaretleriz:

```
extern "C" int topla(int a, int b);     // C bağlantısı: isim bozulmaz
```

C başlık dosyaları hem C hem C++ derleyicisiyle kullanılacaksa bu işaret bir koşulla sarılır:

```
#ifdef __cplusplus
extern "C" {
#endif

int topla(int a, int b);

#ifdef __cplusplus
}
#endif
```

`__cplusplus` yalnızca C++ derleyicilerinin tanımladığı bir makrodur (Bölüm 15). Standart kütüphanenin C başlıkları bu kalıbı kendileri içerir; bu yüzden `<cmath>` ya da `<cstdio>` kullanırken bunu düşünmemiz gerekmez.

25.6 inline ve constexpr Fonksiyonlar

Çok kısa bir fonksiyonun her çağrısı, çağrı yığınında çerçeve açma maliyeti taşır (Bölüm 9). `inline` anahtar sözcüğü, derleyiciye çağrı yerine fonksiyonun gövdesini çağrı noktasına yerleştirmesini önerir:

```
inline int kareAl(int x) {
    return x * x;
}
```

Bu bir istek değil öneridir: derleyici dilerse uymaz; yazmasak bile optimizasyon açıkken kısa fonksiyonlarda satır içi yerleştirmeye kendisi karar verir. Bu yüzden sıradan programlarda performans için `inline` yazmamız gerekmez. C++'ta anahtar sözcüğün bugünkü asıl işi başkadır: **aynı tanımın birden çok kaynak dosyada**

bulunmasına izin vermek. Normalde bir fonksiyon programda yalnızca bir kez tanımlanabilir; bir başlık dosyasına koyduğumuz sıradan tanım, o başlığı iki `.cpp` dosyası dahil ettiğinde bağlayıcıda "çoklu tanım" hatası verir. `inline` işaretli tanım ise her dosyada aynı olmak koşuluyla serbesttir. Bu yüzden başlık dosyalarındaki küçük fonksiyonlar `inline` yazılır (Bölüm 15). C99'un `inline` kuralları C++'tan farklıdır; C'den alışkanlık taşımak yanıltıcı olabilir.

`constexpr` daha güçlü bir sözdür. Bölüm 22'de `constexpr` değişkenleri gördük; fonksiyon da `constexpr` olabilir:

> **Tanım: constexpr fonksiyon**
>
> Argümanları derleme zamanında bilinen bir çağrıda, sonucu **derleme zamanında** hesaplanabilen fonksiyondur. Argümanlar çalışma zamanında belli oluyorsa aynı fonksiyon sıradan bir fonksiyon gibi çalışır. Gövdesinde yalnızca derleme zamanında yapılabilecek işlemler bulunabilir.

`constexprFonk.cpp`

```cpp
#include <iostream>

constexpr int faktoriyel(int n) {
    int sonuc = 1;
    for (int i = 2; i <= n; i++) {
        sonuc *= i;
    }
    return sonuc;
}

static_assert(faktoriyel(5) == 120, "faktoriyel yanlis");

int main() {
    constexpr int sabit = faktoriyel(6);      // derleme zamanında hesaplanır
    int dizi[faktoriyel(4)] = {};             // dizi boyutu sabit ifade olmalı
    int girdi = 5;
    int calisma = faktoriyel(girdi);          // çalışma zamanında da çağrılabilir

    std::cout << sabit << ' ' << sizeof(dizi) / sizeof(dizi[0]) << ' '
              << calisma << '\n';
    return 0;
}
```

```
ÇIKTI
720 24 120
```

`static_assert`, koşulu derleme zamanında sınar; yanlışsa program derlenmez. `faktoriyel(4)` ifadesi bir dizi boyutu olabildi, çünkü derleyici onu 24 olarak önceden hesapladı. Aynı fonksiyon `girdi` değişkeniyle çalışma zamanında da çağrıldı.

> *Tarihçe · constexpr'ın gelişimi*
>
> `constexpr` C++11 ile geldi; o sürümde bir `constexpr` fonksiyonun gövdesi tek bir `return` ifadesinden ibaret olmak zorundaydı. C++14, döngülere, yerel değişkenlere ve birden çok deyime izin verdi; yukarıdaki `faktoriyel` bu yüzden yazılabildi. C++20 `consteval` anahtar sözcüğünü ekledi: `consteval` işaretli fonksiyon her çağrıda derleme zamanında çalışmak zorundadır. Bu kitapta C++17 kullandığımız için `consteval`'i kullanmayacağız.

25.7 Dönüş Tipi auto ve [[nodiscard]]

Bölüm 22'de `auto`'yu değişkenler için gördük. C++14'ten itibaren fonksiyonun dönüş tipi de `auto` yazılabilir; derleyici tipi `return` ifadesinden çıkarır:

▸ otomatikDonus.cpp

```cpp
#include <iostream>

auto kare(int x) {
    return x * x;              // int
}

auto yarisi(double x) {
    return x / 2;              // double
}

int main() {
    std::cout << kare(7) << ' ' << yarisi(5) << '\n';
    std::cout << sizeof(kare(2)) << ' ' << sizeof(yarisi(1)) << '\n';
    return 0;
}
```

ÇIKTI

```
49 2.5
4 8
```

Birden fazla `return` varsa hepsi aynı tipi çıkarmalıdır; farklı tipler çıkarsa derleyici durdurur:

▸ otomatikHata.cpp

```cpp
auto secim(int a) {
    if (a > 0) {
        return 1;
    }
    return 2.5;
}

int main() {
    return secim(1) > 0;
}
```

DERLEYİCİ ÇIKTISI

```
otomatikHata.cpp: In function 'auto secim(int)':
otomatikHata.cpp:5:12: error: inconsistent deduction for auto return type: 'int' and then
'double'
    5 |     return 2.5;
      |            ^~~
```

`auto` dönüş tipinde iki kısıt vardır: fonksiyonun tanımı çağrıdan önce görünmelidir (yalnızca prototip yetmez, çünkü tip tanımın içinden çıkarılır) ve özyinelemeli fonksiyonda tip, özyinelemeli olmayan bir `return`'den belirlenmelidir. Bu nedenle sıradan fonksiyonlarda dönüş tipini açık yazmak çoğu zaman daha okunaklıdır; `auto` dönüş tipi asıl olarak şablonlarda ve lambda'larda işe yarar (sonraki başlıklarda göreceğiz). C++11'de `auto kare(int x) -> int` biçimi de vardır (sona yazılan dönüş tipi); şablon kodunda dönüş tipi parametrelere bağlı olduğunda kullanılır.

[[nodiscard]]

Bir fonksiyonun sonucunu atmak çoğu zaman hatadır. Bölüm 9'da `kareAl(x);` yazıp sonucu kullanmamanın sessizce hiçbir şey yapmadığını görmüştük. C++17 ile gelen **nitelik** (attribute) `[[nodiscard]]`, sonucu atılırsa derleyicinin uyarı vermesini sağlar:

```
[[nodiscard]] int dogrula(int kod) {
    return kod == 0 ? 1 : 0;
}

int main() {
    dogrula(5);              // sonuç atıldı
    return 0;
}
```

```
TERMİNAL
$ g++ -std=c++17 -Wall kontrol.cpp -o kontrol
kontrol.cpp: In function 'int main()':
kontrol.cpp:6:12: warning: ignoring return value of 'int dogrula(int)', declared with attribute
'nodiscard' [-Wunused-result]
    6 |     dogrula(5);
      |     ~~~~~~~^~~
kontrol.cpp:1:19: note: declared here
    1 | [[nodiscard]] int dogrula(int kod) {
      |                   ^~~~~~~
```

Hata kodu döndüren, kaynak ayıran ya da yalnızca değer hesaplayan fonksiyonlar için `[[nodiscard]]` yazmak, yanlış kullanımı derleme aşamasında yakalar. Standart kütüphanenin birçok fonksiyonu da bu niteliği taşır. C++20, niteliğe açıklama metni eklemeyi de mümkün kılar (`[[nodiscard("sebep")]]`). Nitelik C23'e de eklenmiştir; C'de `[[nodiscard]]` yazabiliriz, ancak yalnızca C23'ü destekleyen derleyicilerle.

25.8 Fonksiyon Şablonuna İlk Bakış

Aşırı yükleme, `enBuyuk` fonksiyonunu `int` ve `double` için ayrı ayrı yazmamızı gerektirir; üçüncü bir tip eklendiğinde dördüncü bir kopya daha yazarız. Üç sürümün gövdesi birbirinin aynıdır; yalnızca tip değişir. C'de bu işi `#define MAX(a, b) ((a) > (b) ? (a) : (b))` gibi bir makroyla çözerdik ve makronun argümanları iki kez değerlendirmesi gibi tuzaklarla uğraşırdık (Bölüm 15). C++ aynı işi tip güvenli biçimde yapan **şablonlar** (templates) sunar:

▸ sablonOnizleme.cpp

```
#include <iostream>
#include <string>

template <typename T>
T enBuyuk(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    std::cout << enBuyuk(3, 8) << '\n';
    std::cout << enBuyuk(2.5, 1.5) << '\n';
    std::cout << enBuyuk(std::string{"elma"}, std::string{"armut"}) << '\n';
```

```
    std::cout << enBuyuk<double>(3, 2.5) << '\n';
    return 0;
}
```

```
ÇIKTI
8
2.5
elma
3
```

`template <typename T>` satırı, `T`'nin bir tip yer tutucusu olduğunu söyler. Derleyici her çağrıda argümanların tipinden `T`'yi çıkarır ve o tip için fonksiyonun ayrı bir kopyasını **derleme zamanında** üretir. Yani `enBuyuk(3, 8)` için `int`, `enBuyuk(2.5, 1.5)` için `double` sürümü yazılmış gibi olur; çalışma zamanı maliyeti sıradan bir fonksiyonla aynıdır. Son çağrıda `<double>` ile tipi kendimiz verdik; bu, `3` değerinin `double`'a dönüştürülmesini sağladı.

Aynı tipte olmayan iki argüman verirsek derleyici `T`'yi çıkaramaz:

sablonHata.cpp

```
template <typename T>
T enBuyuk(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    return enBuyuk(3, 2.5);
}
```

```
DERLEYİCİ ÇIKTISI
sablonHata.cpp: In function 'int main()':
sablonHata.cpp:7:19: error: no matching function for call to 'enBuyuk(int, double)'
    7 |     return enBuyuk(3, 2.5);
      |            ~~~~~~~^~~~~~~~
sablonHata.cpp:2:3: note: candidate: 'template<class T> T enBuyuk(T, T)'
    2 | T enBuyuk(T a, T b) {
      |   ^~~~~~~
sablonHata.cpp:2:3: note:   template argument deduction/substitution failed:
sablonHata.cpp:7:19: note:   deduced conflicting types for parameter 'T' ('int' and 'double')
    7 |     return enBuyuk(3, 2.5);
      |            ~~~~~~~^~~~~~~~
```

İleti, `T` için iki çelişen tip (`int` ve `double`) çıkarıldığını söylüyor. Çözüm, `enBuyuk<double>(3, 2.5)` yazmak ya da argümanlardan birini dönüştürmektir. Standart kütüphanede bu iş hazırdır: `std::max` ve `std::min` (`<algorithm>`) bu biçimde şablonlardır. Şablonları, sınıf şablonlarını ve kısıtlarını Bölüm 33'te ayrıntılı işleyeceğiz.

Bir şablon ile aynı ada sahip sıradan bir fonksiyon birlikte bulunabilir. Çağrı ikisine de eşit derecede uyuyorsa derleyici sıradan fonksiyonu seçer; böylece belirli bir tip için özel bir sürüm yazmamız mümkün olur.

25.9 Fonksiyon Gibi Davranan Değerler

Bölüm 12'de fonksiyon işaretçilerini gördük: bir fonksiyonun adresini değişkende tutmak ve başka fonksiyona parametre olarak geçirmek (`qsort`'un karşılaştırma fonksiyonu gibi). C++ bu fikri genişletir. Üç araç yan yana

durur: fonksiyon işaretçisi, `std::function` ve lambda ifadesi. Burada yalnızca ilk tanışmayı yapıyoruz; ayrıntı Bölüm 36'dadır.

Lambda ifadesi, bir fonksiyonu adı olmadan, tam kullanılacağı yerde yazmamızı sağlar:

```
[yakalama](parametreler) { gövde }
```

Köşeli parantez içindeki **yakalama listesi**, lambda'nın çevresindeki hangi yerel değişkenleri kullanabileceğini söyler: `[katsayi]` değişkenin değerini kopyalar, `[&sayac]` değişkene referansla bağlanır, boş `[]` ise hiçbir şey yakalamaz. Lambda'nın tipini yazmak güçtür (derleyici ona adı olmayan benzersiz bir tip verir); bu yüzden onu çoğunlukla `auto` ile bir değişkende tutarız.

`std::function` (`<functional>` başlığı), "şu imzaya sahip, çağrılabilen herhangi bir şey" tutabilen genel bir kaptır. İmza `std::function<dönüşTipi(parametre tipleri)>` biçiminde yazılır: `std::function<int(int, int)>` iki `int` alıp `int` döndüren bir çağrılabiliri gösterir. İçine sıradan fonksiyon, yakalamalı lambda ya da ileride göreceğimiz fonksiyon nesneleri girebilir.

Aşağıdaki program aynı işi üç yolla yapar:

▸ fonksiyonDegerleri.cpp

```
#include <functional>
#include <iostream>

int topla(int a, int b) {
    return a + b;
}

int carp(int a, int b) {
    return a * b;
}

int uygulaIsaretci(int a, int b, int (*islem)(int, int)) {
    return islem(a, b);
}

int uygulaFonksiyon(int a, int b, const std::function<int(int, int)>& islem) {
    return islem(a, b);
}

template <typename F>
int uygulaSablon(int a, int b, F islem) {
    return islem(a, b);
}

int main() {
    std::cout << uygulaIsaretci(3, 4, topla) << ' '
              << uygulaIsaretci(3, 4, carp) << '\n';

    int katsayi{10};
    auto olcekle = [katsayi](int a, int b) { return (a + b) * katsayi; };
    std::cout << uygulaFonksiyon(3, 4, olcekle) << '\n';
    std::cout << uygulaFonksiyon(3, 4, topla) << '\n';
    std::cout << uygulaSablon(3, 4, [](int a, int b) { return a - b; }) << '\n';

    int sayac{0};
    auto say = [&sayac]() { sayac++; };
    say();
    say();
    say();
```

```
    std::cout << "sayac = " << sayac << '\n';
    return 0;
}
```

```
ÇIKTI
7 12
70
7
-1
sayac = 3
```

Programdan şunları çıkarıyoruz:

- `int (*islem)(int, int)` C'deki fonksiyon işaretçisi yazımıdır ve C++'ta aynen çalışır. Yalnızca düz fonksiyonları ve **hiçbir şey yakalamayan** lambda'ları alabilir; `olcekle`'yi `uygulaIsaretci`'ye vermeye kalkarsak derleme hatası alırız, çünkü `katsayi`'yı yakalar.
- `std::function<int(int, int)>` parametresi, düz fonksiyonu da yakalamalı lambda'yı da kabul eder. Karşılığında küçük bir çalışma zamanı maliyeti vardır: çağrı dolaylı yapılır ve yakalamalı bir lambda'nın kopyası kapta saklanır.
- Şablon parametresi (`F islem`) her çağrılabilir tip için ayrı bir sürüm üretir ve çağrıyı derleyici satır içine alabilir. Hız önemliyse bu yol tercih edilir; standart algoritmalar da böyle yazılmıştır (Bölüm 36).
- `[&sayac]` yakalaması lambda'nın dışındaki `sayac` değişkenini değiştirebilmesini sağladı. Referansla yakalanan değişken, lambda çağrılırken hâlâ yaşıyor olmalıdır; fonksiyondan döndürülen bir lambda yerel değişkeni referansla yakalarsa, "yerel değişkene referans döndürmek" hatasının bir başka biçimi ortaya çıkar.

> *Tarihçe · Lambda ve std::function*
>
> Lambda ifadeleri ve `std::function` C++11 ile geldi. Öncesinde aynı işi görmek için ayrı bir fonksiyon ya da `operator()` tanımlayan bir sınıf yazmak gerekiyordu. C++14 lambda parametrelerinde `auto` kullanmaya izin verdi; C++17 lambda'ların `constexpr` olmasını sağladı.

25.10 Üç Dilde Parametre Geçirme

Üç Dilde · Referans, çıkış ve varsayılan parametreler *C · C++ · C#*

C++'taki `&` işaretinin C#'taki karşılığı `ref` anahtar sözcüğüdür; ama C# bu işi daha ayrıntılı üç anahtar sözcüğe böler. C'de bunların hepsini işaretçiyle yapardık.

Amaç	C	C++	C#
Çağıranın değişkenini değiştirmek	`int *p`, çağrıda `&x`	`int& a` (çağrı `f(x)`)	`ref int a` (çağrı `f(ref x)`)
Yalnızca çıktı döndürmek	`int *sonuc`	`int& sonuc`	`out int sonuc` (atama zorunlu)
Büyük nesneyi kopyalamadan, salt okunur geçirmek	`const T *p`	`const T& r`	`in T r`
Birden çok değer döndürmek	`struct` ya da çıkış işaretçileri	`struct`, `std::pair`, `auto [a, b]`	demet `(int, int)`, `var (a, b)`
Aşırı yükleme	Yok	Var	Var

Amaç	C	C++	C#
Varsayılan parametre	Yok	Var (sağdan sola)	Var (sağdan sola, sabit ifade)
Adlandırılmış argüman	Yok	Yok	Var

C# kodu şöyle yazılır (üst düzey deyimlerle, Bölüm 41):

```
void Takas(ref int a, ref int b)
{
    (a, b) = (b, a);                        // demetle takas
}

void BolVeKalan(int bolunen, int bolen, out int bolum, out int kalan)
{
    bolum = bolunen / bolen;
    kalan = bolunen % bolen;
}

(int Bolum, int Kalan) Bol(int a, int b) => (a / b, a % b);

string Selamla(string ad, string unvan = "Sayın") => $"{unvan} {ad}";

int x = 3, y = 8;
Takas(ref x, ref y);                        // ref çağrıda da yazılır
BolVeKalan(47, 5, out int b1, out int k1);   // değişken çağrıda tanımlanabilir
var (b2, k2) = Bol(47, 5);                  // demeti açmak
Console.WriteLine($"{x} {y} {b1} {k1} {b2} {k2}");
Console.WriteLine(Selamla("Elif", unvan: "Dr."));
Console.WriteLine(Selamla(unvan: "Dr.", ad: "Mehmet"));
```

İki fark dikkat çeker. Birincisi, C#'ta `ref` ve `out` çağrıda da yazılır; böylece C++'ta yaşadığımız "bu çağrı argümanı değiştirir mi?" belirsizliği ortadan kalkar. İkincisi, adlandırılmış argümanlar (`unvan: "Dr."`), C++'taki "ortadaki varsayılanı atlayamama" sorununu çözer. C#'ta `class` tipleri zaten referans tipidir: bir nesneyi parametre olarak geçirmek nesnenin kendisini değil ona bağlantıyı kopyalar. Bu yüzden `ref`, esas olarak değer tipleri (`int`, `struct`) ve parametrenin kendisine yeni nesne atamak için gerekir. Ayrıntıları Bölüm 46'da göreceğiz.

Üç Dilde · Fonksiyon değerleri *C · C++ · C#*

Fonksiyonu bir değer gibi taşıma fikri üç dilde de vardır, ama ağırlığı farklıdır:

	C	C++	C#
Fonksiyon işaretçisi	`int (*f)(int, int)`	Aynı	Delegate (`Func`, `Action`)
Genel çağrılabilir kap	Yok	`std::function<int(int, int)>`	`Func<int, int, int>`
Satır içi isimsiz fonksiyon	Yok	`[](int a, int b) { return a + b; }`	`(a, b) => a + b`
Dışarıdan değişken yakalama	Yok	`[x]`, `[&x]` (açık seçim)	Otomatik, her zaman referansla

C#'ın delegate ve lambda'ları Bölüm 55'te işlenir. C++'taki yakalama listesinin açıklığı, bellek ömrünü programcının sorumluluğuna bırakır; C#'ta çöp toplayıcı yakalanan değişkeni yaşatır.

25.11 Örnek: Fonksiyonlarla Puan Özeti

Şimdiye kadar gördüğümüz araçları küçük bir programda birleştirelim. Program bir öğrencinin puan listesini özetler (en düşük, en yüksek, ortalama), puanlara ek puan ekler ve her aşamada özeti yazdırır. Kullanılan araçlar şunlardır: çok değerli sonuç için `struct` dönüşü, büyük nesneler için `const &`, değiştirilecek liste için `&`, varsayılan parametre, aşırı yüklenmiş `yaz` fonksiyonu ve `[[nodiscard]]`. `std::vector` ve `std::min`/`std::max` Bölüm 26'da ayrıntılı işlenecek; şimdilik "değişken uzunlukta liste" ve "iki değerden küçüğü/büyüğü" olarak okuyalım.

▸ puanOzeti.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct Ozet {
    int enDusuk;
    int enYuksek;
    double ortalama;
};

[[nodiscard]] Ozet ozetle(const std::vector<int>& puanlar) {
    Ozet o{puanlar[0], puanlar[0], 0.0};
    int toplam{0};
    for (int p : puanlar) {
        o.enDusuk = std::min(o.enDusuk, p);
        o.enYuksek = std::max(o.enYuksek, p);
        toplam += p;
    }
    o.ortalama = static_cast<double>(toplam) / puanlar.size();
    return o;
}

void puanEkle(std::vector<int>& puanlar, int ek = 5) {
    for (int& p : puanlar) {
        p = std::min(100, p + ek);
    }
}

char harfNotu(double ortalama) {
    if (ortalama >= 90) return 'A';
    if (ortalama >= 75) return 'B';
    if (ortalama >= 60) return 'C';
    return 'F';
}

void yaz(const std::string& etiket, const Ozet& o) {
    std::cout << etiket << ": ort = " << o.ortalama
              << ", en düşük = " << o.enDusuk
              << ", en yüksek = " << o.enYuksek
              << ", harf = " << harfNotu(o.ortalama) << '\n';
}

void yaz(const std::string& etiket, int deger) {
    std::cout << etiket << ": " << deger << '\n';
}

int main() {
    std::vector<int> puanlar{72, 85, 58, 91, 64};

    yaz("Başlangıç", ozetle(puanlar));
```

```
    puanEkle(puanlar);
    yaz("5 puan ekli", ozetle(puanlar));
    puanEkle(puanlar, 20);
    yaz("20 puan ekli", ozetle(puanlar));
    yaz("Son puan", puanlar.back());
    return 0;
}
```

```
ÇIKTI
Başlangıç: ort = 74, en düşük = 58, en yüksek = 91, harf = C
5 puan ekli: ort = 79, en düşük = 63, en yüksek = 96, harf = B
20 puan ekli: ort = 93.8, en düşük = 83, en yüksek = 100, harf = A
Son puan: 89
```

Fonksiyonların parametre tipleri, "dört soru" kuralıyla seçilmiştir. `ozetle` listeyi yalnızca okur ve liste büyük olabilir: `const std::vector<int>&`. `puanEkle` listeyi değiştirir: `std::vector<int>&`; değiştirilecek ek puan küçük bir sayıdır ve değerle geçer, üstelik varsayılanı vardır. `harfNotu` küçük bir değer alır: düz `double`. `yaz` iki farklı sürümüyle aynı ada sahip olduğu için çağrılar `yaz("...", ozetle(puanlar))` ve `yaz("...", puanlar.back())` olarak aynı biçimde yazılabildi; derleyici ikinci argümanın tipine bakarak doğrusunu seçti. `ozetle(puanlar)` bir geçici `Ozet` döndürür; bu geçici değer `const Ozet&` parametresine bağlanabildi. Döngüdeki `int& p`, listenin elemanını yerinde değiştirmemizi sağlar (Bölüm 24).

25.12 Sık Yapılan Hatalar

- **Çıkış parametresi sanıp değerle geçirmek.** `void artir(int n) { n++; }` çağıranın değişkenini değiştirmez; `int& n` gerekir.
- **Yerel değişkene referans ya da işaretçi döndürmek.** Fonksiyon bitince değişken yok olur; değerle döndürmek en güvenli yoldur.
- **Büyük nesneyi değerle geçirmek.** `std::string` ve `std::vector` gibi tipler için `const T&` kullanılır.
- **Varsayılan değeri hem prototipte hem tanımda yazmak** ya da varsayılan parametreleri sağdan sola sıralamamak.
- **Aşırı yüklemede belirsiz çağrı.** Eşit derecede iyi iki aday varsa çağrıdaki argümanlardan birini `static_cast` ile dönüştürürüz.
- **Metin sabiti ile `bool` aşırı yüklemesini karıştırmak.** `"metin"` argümanı `std::string` yerine `bool` sürümüne gider.
- **`[[nodiscard]]` uyarısını susturmak için sonucu atmak.** Uyarı varsa sonucu kullanmalı ya da fonksiyonun gerçekten gerektiğini yeniden düşünmeliyiz.
- **Referansla yakalanan değişkenin ömründen uzun yaşayan lambda yazmak.** Lambda'yı fonksiyondan döndürüyorsak yerel değişkenleri değerle yakalarız.

Bölüm Özeti

- Fonksiyonun anatomisi, prototip, değerle geçirme, çağrı yığını, kapsam ve `static` yerel değişken C++'ta C ile aynıdır (Bölüm 9). Farklar: `int f();` parametresiz demektir, prototipsiz çağrı her zaman hatadır, `main` çağrılamaz.

- Referans parametre (`T& r`), argümanın kopyasını değil kendisini alır; atamalar çağıranın değişkenine yansır. Çağrıda `&` ya da `*` gerekmez ama çağrıya bakarak referans olup olmadığı anlaşılmaz. Referans parametreye geçici değer verilemez.
- Birden çok sonuç gerekiyorsa `struct` ya da `std::pair` döndürmek, çıkış parametresinden genellikle daha okunaklıdır; C++17'nin `auto [a, b] = f();` yapısal bağlaması sonucu tek adımda açar.
- Yerel değişkene referans döndürmek tanımsız davranıştır; referans yalnızca fonksiyon bittikten sonra yaşayan nesneler için döndürülür.
- `const T&` kopyayı ve değiştirmeyi engeller, geçici değerleri de kabul eder. Büyük tipler için `const T&`, küçük tipler için değer, değiştirilecek nesneler için `T&` kullanılır; boş olabilen durumlarda işaretçi tercih edilir.
- Varsayılan parametreler sağdan sola verilir, ortadaki argüman atlanamaz ve değer yalnızca bir bildirimde (genellikle prototipte) yazılır.
- Aşırı yüklemede imzayı ad ve parametre listesi belirler; dönüş tipi belirlemez. Seçim sırası tam eşleşme, terfi, standart dönüşüm, kullanıcı tanımlı dönüşümdür; eşit iki aday belirsiz çağrı hatası verir.
- Derleyici aşırı yüklenmiş fonksiyonların adlarını parametre tiplerini de içerecek biçimde bozar (name mangling). C kütüphanelerine bağlanırken bildirimler `extern "C"` ile işaretlenir.
- `inline` önerisi performanstan çok başlık dosyalarında çoklu tanıma izin verir. `constexpr` fonksiyon, argümanları sabitse derleme zamanında çalışır, aksi hâlde sıradan fonksiyon gibi davranır.
- C++14'ten itibaren dönüş tipi `auto` olabilir; bütün `return`'ler aynı tipi çıkarmalıdır. `[[nodiscard]]` (C++17) sonucu atılan çağrıları uyarıyla yakalar.
- `template <typename T>` ile tek tanımdan her tip için fonksiyon üretilir; tipler çelişirse tip çıkarımı başarısız olur.
- Fonksiyon işaretçisi, `std::function` ve lambda ifadesi (`[yakalama](parametreler) { gövde }`) fonksiyonları değer gibi taşımamızı sağlar; yakalamalı lambda düz fonksiyon işaretçisine dönüştürülemez.
- C#'ta karşılıkları `ref`, `out`, `in`, varsayılan ve adlandırılmış parametreler, demetler ve `Func<...>` delegate'leridir.

Alıştırmalar

1. Aşağıdaki fonksiyon için hangi çağrı derlenir?

```
void sifirla(int& n) {
    n = 0;
}
```

   A) `sifirla(5);`
   B) `sifirla(3 + 4);`
   C) `int x{9}; sifirla(x);`
   D) `sifirla(&x);` (x bir `int` değişken)
   E) `sifirla(x + 1);` (x bir `int` değişken)

2. `const std::string&` parametresinin `std::string` (değerle) parametresine göre en önemli üstünlüğü hangisidir?

   A) Parametre fonksiyon içinde değiştirilebilir.
   B) Çağıranın değişkeni her zaman güncellenir.

C) Metnin kopyası çıkarılmaz ve fonksiyon metni yanlışlıkla değiştiremez.
D) Fonksiyon `nullptr` alabilir.
E) Yalnızca değişkenlerle çağrılabilir, sabitlerle çağrılamaz.

3. Aşağıdaki fonksiyon çiftlerinden hangisi tanımlandıktan sonra `f(1)` çağrısını sorunsuz derler?

A) `int f(int a);` ve `double f(int a);`
B) `int f(int a);` ve `int f(const int a);`
C) `void f(int a);` ve `void f(int a, int b = 0);`
D) `void f(int a);` ve `void f(double a);`
E) `void f(long a);` ve `void f(short a);`

4. Aşağıdaki bildirimlerden hangisi geçerlidir?

A) `int topla(int a = 1, int b);`
B) `int topla(int a, int b = 2, int c);`
C) `int topla(int a, int b = 2, int c = 3);`
D) `int topla(int a = 1, int b, int c = 3);`
E) `int topla(int = 1, int, int);`

5. Aşağıdakilerden hangisi doğrudur?

A) `constexpr` fonksiyon yalnızca derleme zamanında çağrılabilir.
B) `auto` dönüş tipli bir fonksiyonda `return 1;` ve `return 2.5;` birlikte bulunabilir.
C) `[[nodiscard]]` işaretli fonksiyonun sonucunu atmak derleme hatası verir.
D) `inline` yazmak, fonksiyonun satır içi yerleştirileceğinin garantisidir.
E) Yakalama yapmayan bir lambda, düz fonksiyon işaretçisine dönüştürülebilir.

6. Programın çıktısı nedir?

```
#include <iostream>

void degistir(int a, int& b) {
    a = a + 10;
    b = b + 10;
}

int main() {
    int x{1};
    int y{1};
    degistir(x, y);
    std::cout << x << ' ' << y << '\n';
    degistir(y, x);
    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

7. Programın çıktısı nedir?

```
#include <iostream>

void f(int) { std::cout << "A"; }
void f(double) { std::cout << "B"; }
void f(char) { std::cout << "C"; }

int main() {
    f('a');
    f(97);
    f(97.0);
    f(2.5f);
```

```
    f(true);
    std::cout << '\n';
    return 0;
}
```

8. Programın çıktısı nedir?

```
#include <iostream>

int topla(int a, int b = 10, int c = 100);

int main() {
    std::cout << topla(1) << ' ' << topla(1, 2) << ' '
              << topla(1, 2, 3) << '\n';
    return 0;
}

int topla(int a, int b, int c) {
    return a + b + c;
}
```

9. Programın çıktısı nedir?

```
#include <iostream>

int main() {
    int n{5};
    auto deger = [n]() { return n * 2; };
    auto bag = [&n]() { return n * 2; };
    n = 20;
    std::cout << deger() << ' ' << bag() << '\n';
    return 0;
}
```

10. Aşağıdaki fonksiyonda ne gibi bir sorun vardır? Nasıl düzeltilir?

```
int& enBuyuk(int a, int b) {
    return (a > b) ? a : b;
}
```

11. Aşağıdaki program neden derlenmez? İki farklı düzeltme önerin.

```
#include <iostream>

void yaz(int a) {
    std::cout << "tek: " << a << '\n';
}

void yaz(int a, int b = 0) {
    std::cout << "cift: " << a << ' ' << b << '\n';
}

int main() {
    yaz(1);
    return 0;
}
```

12. Üç `int` değişkenin değerlerini döndüren `void dondur(int& a, int& b, int& c)` fonksiyonunu yazın: `a` eski `b`'yi, `b` eski `c`'yi, `c` eski `a`'yı alsın. `main`'de `1 2 3` değerleriyle deneyin.
13. `ortalama` adlı üç aşırı yüklenmiş fonksiyon yazın: ikisi iki ve üç `int` alsın, üçüncüsü `const std::vector<int>&` alsın. Hepsi `double` döndürsün. `main`'de her birini çağırıp sonuçları yazdırın.
14. `double kuvvet(double taban, int us = 2)` fonksiyonunu yazın (döngüyle; `us` negatif olmayacak). `kuvvet(3)`, `kuvvet(2, 10)` ve `kuvvet(1.5, 3)` sonuçlarını yazdırın. Fonksiyonu `constexpr` yapıp `static_assert(kuvvet(2, 10) == 1024);` ile derleme zamanında sınayın.
15. `int ikiKez(int deger, const std::function<int(int)>& islem)` fonksiyonunu yazın: `islem`'i önce `deger`'e, sonra çıkan sonuca uygulasın. `main`'de bir yakalamasız lambda (örneğin 3 ekleyen) ve `int carpan{4}` değerini yakalayan bir lambda ile deneyin.

BÖLÜM 26

Diziler, std::array ve std::vector

BU BÖLÜMDE

- C dizilerinin C++'ta nasıl davrandığını; süslü parantezli ilk değer verme, daralma denetimi ve `std::size` gibi farkları
- `std::array` ile boyutunu bilen, kopyalanabilen sabit boyutlu dizi kurmayı; `[]` ile `at()` arasındaki farkı
- `std::vector`'ü oluşturmayı, `push_back`, `pop_back`, `insert`, `erase`, `resize`, `clear` ile değiştirmeyi ve çeşitli yollarla dolaşmayı
- `size()` ile `capacity()` arasındaki farkı, vector'ün bellekte nasıl büyüdüğünü ve yineleyicilerin ne zaman geçersiz kaldığını
- `vector<vector<int>>` ile matris kurmayı ve vector'ü fonksiyona değer, referans ve `const` referansla geçirmeyi
- `<algorithm>` başlığındaki `sort`, `find`, `count`, `max_element` ve `reverse` işlevlerine ilk bakışı

Bölüm 10'da C dizilerini ayrıntılı gördük: aynı tipten elemanları art arda tutarlar, boyutları derleme zamanında sabittir, sınır denetimi yoktur ve fonksiyona geçerken işaretçiye dönüşüp boyut bilgisini kaybederler. Bölüm 13'te bu sınırlamayı `malloc` ve `realloc` ile elle aşmayı denedik; çalıştı, ama bellek yönetimi ve boyut takibi tamamen bize kalmıştı.

C++ bu iki sorunu da standart kütüphaneyle çözer. Boyutu derleme zamanında sabit olan veri için `std::array`, boyutu çalışma anında değişen veri için `std::vector` vardır. İkisi de kendi boyutunu bilir, kopyalanabilir, yineleyicilerle ve algoritmalarla uyumludur; `vector` ayrıca belleği kendisi yönetir. Bu bölümde C dizisinin C++'taki yerini kısaca gözden geçirecek, sonra bu iki kapsayıcıyı (container) ayrıntılı işleyeceğiz. Bölüm 10 ve 13'ü okuduğunuzu varsayıyoruz; orada anlatılanları tekrar etmeden yalnızca farkları vurgulayacağız.

26.1 C Dizileri C++'ta

C++ C dizilerini olduğu gibi devralır. `int notlar[4];` tanımı, `notlar[0]` ile erişim, bellekte art arda durma, `sizeof(notlar)` ile toplam bayt sayısını bulma, dizinin fonksiyona adres olarak geçmesi ve sınır dışı erişimin tanımsız davranış olması C'deki gibidir. Bölüm 10'daki her dizi örneği, `printf` yerine `cout` yazılarak C++'ta da çalışır. Farklar küçük ama önemlidir.

Süslü parantezle ilk değer verme ve daralma

C++11 ile gelen tekdüze ilk değer verme (uniform initialization) dizilerde de kullanılabilir. `=` işareti isteğe bağlıdır, boş liste diziyi sıfırlar:

```
int a[5] = {10, 20, 30};   // kalanlar 0: 10 20 30 0 0
int b[5]{};                // hepsi 0
int c[]{4, 8, 15, 16};     // boyut listeden çıkar: 4 eleman
```

C ile temel fark, süslü parantezin **daralma dönüşümünü** (narrowing conversion) yasaklamasıdır. Veri kaybı olabilecek bir dönüşüm C'de sessizce yapılırdı; C++'ta liste içinde derleme hatasıdır (Bölüm 22'de ayrıntısıyla göreceğiz):

daralma.cpp

```
#include <iostream>

int main() {
    int a[3] = {1, 2, 3};
    int b[3] = {1, 2.5, 3};
    std::cout << a[0] << b[1] << '\n';
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
daralma.cpp: In function 'int main()':
daralma.cpp:5:24: error: narrowing conversion of '2.5e+0' from 'double' to 'int' [-Wnarrowing]
    5 |     int b[3] = {1, 2.5, 3};
      |                        ^
```

Listede boyuttan fazla eleman olması da C++'ta hatadır (`too many initializers`); C'de yalnızca uyarıdır.

Değişken uzunluklu diziler standart dışıdır

Bölüm 10'da C99'un değişken uzunluklu dizilerini (VLA) kısa ve uyarılı olarak anlatmıştık. C++ standardında VLA **yoktur**. `int n = 5; int a[n];` yazımını g++ bir eklenti olarak kabul eder, ama `-Wpedantic` ile uyarır ve Visual C++ derleyicisi reddeder. Boyutu çalışma anında belli olan veri için doğru araç `vector`'dür.

```
int n = 5;
int a[n];                // g++ eklenti olarak derler, standart C++ değil
const int N = 5;
int b[N];                // geçerli: N derleme zamanında bilinen sabit
```

sizeof tuzağı ve std::size

Bölüm 10'daki tuzak burada da geçerlidir: dizi fonksiyona parametre olarak geçince `sizeof(a) / sizeof(a[0])` işaretçinin boyutunu kullanır ve yanlış sonuç verir. C++17, `<iterator>` başlığıyla (çoğu başlık onu zaten içerir) `std::size` işlevini getirdi. Dizi tipinin boyutunu **derleme zamanında tipten** okur; dizi yerine işaretçi verilirse yanlış sonuç üretmek yerine derlenmez:

eleman.cpp

```
#include <iostream>
#include <iterator>

void yanlis(int a[]) {
    // std::size(a) burada derlenmez: a bir işaretçidir
    std::cout << "işaretçi boyutu: " << sizeof(a) << '\n';
}
```

```
int main() {
    int dizi[6] = {1, 2, 3, 4, 5, 6};
    std::cout << "eleman sayısı: " << std::size(dizi) << '\n';
    std::cout << "eski yöntem : " << sizeof(dizi) / sizeof(dizi[0]) << '\n';
    yanlis(dizi);
    return 0;
}
```

```
ÇIKTI
eleman sayısı: 6
eski yöntem : 6
işaretçi boyutu: 8
```

`g++ -Wall` bu `sizeof(a)` satırı için `-Wsizeof-array-argument` uyarısı verir; uyarı tam olarak bu tuzağı işaret eder. Son satırdaki 8, 64 bitlik sistemde işaretçinin bayt sayısıdır; başka sistemde farklı olabilir. `std::size` kullanmak, tuzağı sessiz bir hata olmaktan çıkarıp derleme hatasına çevirir.

Aralık tabanlı for

C'de dizi dolaşmak için indeksli `for` kullanmıştık. C++11, indeks gerektirmeyen **aralık tabanlı for** (range-based for) getirdi: `for (tip ad : dizi)`. Bölüm 24'te bu döngüyü tüm yönleriyle işleyeceğiz; burada dizi ve vector ile çalışmak için gerekli kalıbı özetleyelim.

▸ araliklifor.cpp

```
#include <iostream>

int main() {
    int a[4] = {1, 2, 3, 4};

    for (int x : a) {           // x elemanın kopyasıdır
        x *= 10;
    }
    std::cout << a[0] << ' ' << a[3] << '\n';

    for (int& x : a) {          // x elemanın kendisidir
        x *= 10;
    }
    std::cout << a[0] << ' ' << a[3] << '\n';

    int toplam = 0;
    for (const int& x : a) {    // yalnız okuyoruz
        toplam += x;
    }
    std::cout << toplam << '\n';
    return 0;
}
```

```
ÇIKTI
1 4
10 40
100
```

Kopya alan sürümde dizi değişmez; `&` ile referans alan sürümde elemanın kendisi değişir; yalnız okuyorsak `const &` yazarız. Bu döngü yalnızca dizinin tanımlandığı yerde çalışır: fonksiyona işaretçiye dönüşmüş olarak geçen bir dizide derleyici bitiş noktasını bilemez. `array` ve `vector`'de bu sorun yoktur.

> ◆ **Kritik Nokta · C dizisi kopyalanamaz ve boyutunu bilmez**
>
> `int b[3]; b = a;` C++'ta da derlenmez; `a == b` karşılaştırması elemanları değil adresleri karşılaştırır. Bunları düzeltmek için C++'ın yeni kapsayıcılarına ihtiyaç var. Bu bölümün geri kalanı onlarla ilgilidir.

26.2 std::array: Boyutu Bilen Sabit Dizi

`std::array`, `<array>` başlığında tanımlı, C dizisinin üzerine ince bir sarmalayıcıdır. Boyutu C dizisi gibi **derleme zamanında sabittir** ve tipin bir parçasıdır: `std::array<int, 5>` "5 tane `int`" demektir. Bellek düzeni C dizisiyle aynıdır (elemanlar art arda, ek yük yok); ama bir sınıf olduğu için kendi boyutunu bilir ve kopyalanabilir.

Açısal parantez `<int, 5>` bir **şablon** (template) argümanıdır; şablonları Bölüm 33'te göreceğiz. Şimdilik kalıp olarak alabiliriz: ilk bölüm eleman tipi, ikinci bölüm eleman sayısıdır.

▸ arrayTemel.cpp

```cpp
#include <array>
#include <iostream>

int main() {
    std::array<int, 5> a{10, 20, 30, 40, 50};
    std::array<int, 5> b = a;        // bütün elemanlar kopyalanır
    b[0] = 1;

    std::cout << "boyut: " << a.size() << '\n';
    std::cout << "a[0]=" << a[0] << " b[0]=" << b[0] << '\n';
    std::cout << "a.at(2)=" << a.at(2) << '\n';
    std::cout << "ilk=" << a.front() << " son=" << a.back() << '\n';
    std::cout << (a == b ? "eşit" : "farklı") << '\n';

    b.fill(7);                       // hepsini 7 yap
    for (int x : b) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

ÇIKTI

```
boyut: 5
a[0]=10 b[0]=1
a.at(2)=30
ilk=10 son=50
farklı
7 7 7 7 7
```

C dizisine göre kazanımları şöyle sıralayabiliriz:

- `size()` her zaman doğru boyutu verir; fonksiyona geçerken de kaybolmaz.
- `=` ile kopyalanır, `==`, `!=`, `<` gibi işleçlerle elemanları sırayla karşılaştırılır.
- `front()`, `back()`, `fill()` gibi yardımcı işlevleri vardır.
- Aralık tabanlı `for` ve algoritmalarla doğrudan çalışır.
- Fonksiyona `const std::array<int, 5>&` olarak geçirilir; kopyalamak istemiyorsak referans kullanırız.

Boyut tipin parçası olduğu için `std::array<int, 5>` ile `std::array<int, 6>` birbirinden **farklı tiplerdir** ve birbirine atanamaz. Bu yüzden boyutu değişebilen veri için `array` uygun değildir.

[] ve at()

`a[i]` C dizisinde olduğu gibi denetimsizdir; indeks sınır dışıysa tanımsız davranış oluşur. `a.at(i)` ise önce indeksi denetler ve sınır dışındaysa `std::out_of_range` **istisnası** (exception) fırlatır. İstisnayı Bölüm 34'te ayrıntılı işleyeceğiz; şimdilik "programın normal akışını kesen bir hata sinyali" diye bilelim. Yakalanmazsa program sonlanır, ama sessizce yanlış sonuç üretmez.

▸ atHata.cpp

```cpp
#include <array>
#include <iostream>
#include <stdexcept>

int main() {
    std::array<int, 3> a{10, 20, 30};

    try {
        std::cout << a.at(1) << '\n';
        std::cout << a.at(5) << '\n';     // istisna fırlatır
        std::cout << "buraya gelinmez\n";
    } catch (const std::out_of_range& e) {
        std::cout << "5. indeks yok\n";
    }
    return 0;
}
```

ÇIKTI

```
20
5. indeks yok
```

Kural basittir: indeksin geçerli olduğundan eminsek (`i < a.size()` olan bir döngüde) `[]`, indeks kullanıcıdan ya da bir hesaptan geliyorsa `at()` kullanırız. `at()` her erişimde bir denetim yaptığı için `[]`'den çok az yavaştır; çoğu programda bu fark önemsizdir.

> ▲ **Sık Yapılan Hata · Sıfırlanmamış std::array çöp taşır**
>
> `std::array<int, 5> x;` ilk değer vermez; yerel bir değişken olarak tanımlandığında elemanlar C dizisindeki gibi çöp değerli kalır (Bölüm 5). Sıfırlamak için `std::array<int, 5> x{};` yazarız.

std::array ne zaman?

`std::array`, boyutu program yazılırken belli olan küçük veri kümeleri için uygundur: haftanın gün adları, bir satranç tahtasının kenar uzunluğu, bir 3×3 dönüşüm matrisi. Elemanlar `vector`'ün aksine ayrı bir bellek bölgesinde (heap) değil, nesnenin kendi içinde durur; bu da küçük diziler için hem hızlı hem de bellek yönetiminden bağımsız demektir.

İki boyutlu sabit tablo da iç içe `array` ile kurulur:

```cpp
std::array<std::array<int, 3>, 2> m{};   // 2 satır, 3 sütun
m[1][2] = 5;
```

C dizisindeki `int m[2][3]`'ün aksine bu tablo kopyalanabilir, boyutları `size()` ile sorulabilir ve fonksiyona sütun sayısı elle yazılmadan geçirilebilir (tip zaten sütun sayısını içerir).

26.3 std::vector: Büyüyen Dizi

Veri miktarı çoğu zaman program yazılırken bilinmez: kullanıcı kaç not girecek, bir dosyada kaç satır var, bir ağdan kaç paket gelecek? Bölüm 13'te bu durumda `malloc` ile bir blok ayırıp dolduğunda `realloc` ile büyütüyor, boyutu ayrı bir değişkende tutuyor ve sonunda `free` çağırıyorduk. `std::vector` tüm bunları bir pakete koyar: elemanları bellekte art arda tutar (bu yüzden C dizisi kadar hızlı erişir), eleman sayısı çalışma anında artıp azalabilir, boyutunu kendisi bilir ve nesne yok edilirken belleği kendiliğinden geri verir.

> **Tanım: vector**
>
> `<vector>` başlığında tanımlı, elemanları bellekte art arda tutan ve eleman sayısı çalışma anında değişebilen dizi benzeri bir kapsayıcıdır. Belleği kendisi yönetir; kopyalanabilir, karşılaştırılabilir ve kendi boyutunu bilir.

Oluşturma

`vector<int>` "int tutan vector" demektir; açısal parantezin içine herhangi bir tip yazılabilir (`vector<double>`, `vector<string>`). Birkaç oluşturma biçimi vardır ve aralarındaki fark önemlidir:

vectorOlustur.cpp

```
#include <iostream>
#include <vector>

void yazdir(const std::vector<int>& v) {
    std::cout << "boyut " << v.size() << ":";
    for (int x : v) {
        std::cout << ' ' << x;
    }
    std::cout << '\n';
}

int main() {
    std::vector<int> a;              // boş
    std::vector<int> b{10, 20, 30};  // ilk değer listesi
    std::vector<int> c(4, 7);        // 4 eleman, hepsi 7
    std::vector<int> d{4, 7};        // 2 eleman: 4 ve 7
    std::vector<int> e(3);           // 3 eleman, hepsi 0
    std::vector<int> f = b;          // b'nin kopyası

    yazdir(a);
    yazdir(b);
    yazdir(c);
    yazdir(d);
    yazdir(e);
    yazdir(f);
    return 0;
}
```

```
ÇIKTI
boyut 0:
boyut 3: 10 20 30
boyut 4: 7 7 7 7
boyut 2: 4 7
boyut 3: 0 0 0
boyut 3: 10 20 30
```

> **▲ Sık Yapılan Hata · Parantez ve süslü parantez farklı iş yapar**
>
> `vector<int> c(4, 7)` "4 tane 7" demektir: parantez yapıcıya **boyut ve değer** verir. `vector<int> d{4, 7}` ise "elemanları 4 ve 7 olan vector"dür: süslü parantez bir **eleman listesidir**. Tek argümanlı `vector<int> e(3)` üç elemanlı bir vector kurar; `vector<int> g{3}` ise tek elemanı 3 olan vector'dür. Bu fark Bölüm 30'da yapıcıları anlatırken kaynağıyla birlikte açıklanacak.

C dizisinin aksine `vector`'ün elemanlarına **her zaman ilk değer verilir**: boyut belirtip değer belirtmezsek elemanlar 0 olur (sınıf tiplerinde varsayılan yapıcı çağrılır). Çöp değer sorunu yoktur.

Temel işlemler

Elemanlara dizideki gibi `v[i]` ile ulaşılır. Sona eleman eklemek için `push_back`, sondan silmek için `pop_back` kullanılır. Bu işlevler `v.size()` gibi bir nesnenin **üye işlevleridir** (member function); nokta ile çağrılırlar. Üye işlevleri Bölüm 30'da ayrıntısıyla öğreneceğiz.

İşlev	Ne yapar
`v.size()`	Eleman sayısı
`v.empty()`	Boşsa `true`
`v.push_back(x)`	`x`'i sona ekler; boyut bir artar
`v.pop_back()`	Son elemanı siler (değer döndürmez)
`v.front()`, `v.back()`	İlk ve son eleman
`v[i]`	`i` numaralı eleman, denetimsiz
`v.at(i)`	`i` numaralı eleman, denetimli
`v.clear()`	Bütün elemanları siler; boyut 0 olur
`v.resize(n)`	Boyutu `n` yapar

▸ vectorTemel.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    for (int i = 1; i <= 4; i++) {
        v.push_back(i * i);          // 1 4 9 16
    }
    v[0] = 40;

    std::cout << "ilk: " << v.front() << ", son: " << v.back() << '\n';
    std::cout << "v[2] = " << v[2] << ", v.at(3) = " << v.at(3) << '\n';

    v.pop_back();
```

```
    std::cout << "pop_back sonrası boyut: " << v.size() << '\n';

    v.clear();
    std::cout << "clear sonrası boş mu? " << v.empty() << '\n';
    std::cout << std::boolalpha << v.empty() << '\n';
    return 0;
}
```

```
ÇIKTI
ilk: 40, son: 16
v[2] = 9, v.at(3) = 16
pop_back sonrası boyut: 3
clear sonrası boş mu? 1
true
```

`empty()` bir `bool` döndürür; `cout` varsayılan olarak `true`'yu 1 yazar, `std::boolalpha` düzenleyicisi (Bölüm 23) ise sözcük olarak yazdırır.

`[]` ve `at()` ayrımı `array`'dekiyle aynıdır: `v[i]` sınır dışında tanımsız davranıştır, `v.at(i)` ise `out_of_range` fırlatır. Boş bir vector'ün `v[0]`'ına yazmak da sınır dışıdır, çünkü henüz hiç eleman yoktur; eleman eklemenin yolu `push_back`'tir.

```
std::vector<int> v;
v[0] = 5;                    // tanımsız davranış: eleman yok
v.push_back(5);              // doğrusu bu
std::vector<int> w(3);
std::cout << w[3];           // tanımsız davranış: son indeks 2
```

size() işaretsizdir

`size()` bir `int` değil, işaretsiz bir tamsayı (`std::size_t`) döndürür. Bölüm 5'te gördüğümüz işaretli/işaretsiz karşılaştırma tuzağı burada sık karşımıza çıkar. `for (int i = 0; i < v.size(); i++)` derlenir, ama `-Wall -Wextra` ile "comparison of integer expressions of different signedness" uyarısı verir.

Asıl tehlike boş bir vector'de ortaya çıkar. `v.size() - 1`, `size()` sıfır olduğunda -1 değil, `size_t`'nin en büyük değeri olur:

```
// v boşsa v.size() - 1 çok büyük bir pozitif sayıdır
for (size_t i = 0; i <= v.size() - 1; i++) { /* ... */ }   // hiç bitmez

// i işaretsiz olduğundan i >= 0 her zaman doğrudur
for (size_t i = v.size() - 1; i >= 0; i--) { /* ... */ }   // hiç bitmez
```

Sondan başa giderken güvenli kalıp `for (size_t i = v.size(); i > 0; i--)` yazıp elemana `v[i - 1]` ile ulaşmaktır. İndeks gerekmiyorsa aralık tabanlı `for` bu tuzakların hepsinden korur.

26.4 Eleman Ekleme, Silme ve Yeniden Boyutlandırma

`push_back` ve `pop_back` yalnızca vector'ün sonunda çalışır. Ortaya eleman eklemek ya da ortadan eleman silmek için **yineleyici** (iterator) alan `insert` ve `erase` kullanılır. Yineleyiciyi Bölüm 36'da ayrıntılı işleyeceğiz; şimdilik "bir elemanın konumunu gösteren nesne" diye düşünelim. `v.begin()` ilk elemanı, `v.end()` ise son elemanın **bir ötesini** gösterir; yineleyiciye sayı ekleyerek ilerletebiliriz: `v.begin() + 2` üçüncü elemandır.

vectorDegistir.cpp

```cpp
#include <iostream>
#include <vector>

void yazdir(const std::vector<int>& v) {
    std::cout << "[" << v.size() << "]";
    for (int x : v) {
        std::cout << ' ' << x;
    }
    std::cout << '\n';
}

int main() {
    std::vector<int> v{10, 20, 30, 40, 50};

    v.insert(v.begin() + 1, 15);          // 1 numaralı konuma 15
    yazdir(v);
    v.insert(v.end(), {60, 70});          // sona iki eleman
    yazdir(v);

    v.erase(v.begin());                   // ilk elemanı sil
    yazdir(v);
    v.erase(v.begin() + 1, v.begin() + 3); // [1, 3) aralığını sil
    yazdir(v);

    v.resize(3);                          // küçült: fazlalar atılır
    yazdir(v);
    v.resize(5);                          // büyüt: yeniler 0
    yazdir(v);
    v.resize(6, 9);                       // büyüt: yeniler 9
    yazdir(v);
    return 0;
}
```

ÇIKTI

```
[6] 10 15 20 30 40 50
[8] 10 15 20 30 40 50 60 70
[7] 15 20 30 40 50 60 70
[5] 15 40 50 60 70
[3] 15 40 50
[5] 15 40 50 0 0
[6] 15 40 50 0 0 9
```

Bu işlevlerde iki ayrıntı dikkat ister:

- Aralık verilirken **ilk sınır dahil, ikinci sınır hariçtir**: `erase(v.begin() + 1, v.begin() + 3)` 1 ve 2 numaralı elemanları siler, 3 numaralıya dokunmaz. C++ kütüphanesinin her yerinde aynı yarı açık aralık kuralı geçerlidir.
- `insert` ve `erase` ortadaki elemanları kaydırmak zorundadır. Vector'ün başına ya da ortasına ekleme/silme, eleman sayısı arttıkça yavaşlar; sonuna ekleme/silme ise hızlıdır. Başa sık ekleme gerekiyorsa başka bir kapsayıcı (Bölüm 35'teki `deque` ya da `list`) daha uygundur.

`push_back` yerine `emplace_back` de vardır: nesneyi vector'ün içinde doğrudan kurar, sınıf tipli elemanlarda gereksiz kopyadan kaçınır. `int` gibi temel tiplerde fark yoktur; sınıfları işledikten sonra Bölüm 39'da geri döneceğiz.

Dolaşırken silmek

Bir koşula uyan elemanları silmek sık yapılan bir işlemdir. Yanlış kurulmuş bir döngü sessiz hata üretir; doğru kalıp, `erase`'in **sildiği elemandan sonrakini gösteren yineleyici döndürmesine** dayanır:

`silmeDongusu.cpp`

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3, 4, 5, 6, 7, 8};

    for (auto it = v.begin(); it != v.end();) {
        if (*it % 2 == 0) {
            it = v.erase(it);       // erase sonraki elemanı gösterir
        } else {
            ++it;                   // yalnız silmediysek ilerle
        }
    }

    for (int x : v) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
1 3 5 7
```

`auto` Bölüm 22'de işlenen tip çıkarımıdır; burada yineleyicinin uzun tip adını yazmaktan kurtarır. `*it` yineleyicinin gösterdiği elemanı verir (işaretçideki `*p` gibi). Döngü başlığında ilerletme ifadesi yoktur, çünkü `it` yalnızca silmediğimizde ilerler.

> **▲ Sık Yapılan Hata · Aralık tabanlı for içinde eleman silmeyin**
>
> `for (int x : v) { if (x % 2 == 0) v.erase(...); }` biçimi tanımsız davranıştır: döngü arka planda `begin` / `end` yineleyicileriyle çalışır ve `erase` onları geçersiz kılar. Dolaşırken vector'ün boyutunu değiştiren işlem (`push_back`, `insert`, `erase`) yapmayın; yukarıdaki yineleyici kalıbını ya da Bölüm 36'daki `remove_if` ile `erase` birleşimini kullanın.

26.5 Boyut, Kapasite ve Bellekte Büyüme

Bölüm 13'te `realloc` ile büyüyen bir dizi yazarken her büyütmede bir miktar yer fazladan ayırmıştık; aksi hâlde her eklemede tüm diziyi taşımak gerekirdi. `vector` aynı stratejiyi kendisi uygular. İki ayrı kavram vardır:

> **Tanım: Boyut ve kapasite**
>
> **Boyut** (`size()`), vector'ün şu an tuttuğu eleman sayısıdır. **Kapasite** (`capacity()`), vector'ün yeni bir bellek bloğu ayırmadan tutabileceği eleman sayısıdır. Her zaman `size() <= capacity()` geçerlidir.

`push_back` çağrıldığında yer varsa (`size() < capacity()`) eleman yeni yere yazılır. Yer yoksa vector daha büyük bir blok ayırır, mevcut elemanları oraya taşır, eskisini serbest bırakır ve yeni elemanı ekler. Bloğun ne

kadar büyüyeceğini standart belirlemez; yaygın uygulamalar kapasiteyi her seferinde yaklaşık iki katına çıkarır. Gözlemleyelim:

▸ kapasite.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    std::size_t onceki = v.capacity();
    std::cout << "başlangıç: boyut 0, kapasite " << onceki << '\n';

    for (int i = 1; i <= 20; i++) {
        v.push_back(i);
        if (v.capacity() != onceki) {
            onceki = v.capacity();
            std::cout << "boyut " << v.size() << " -> kapasite "
                      << onceki << '\n';
        }
    }

    v.reserve(100);
    std::cout << "reserve(100): boyut " << v.size() << ", kapasite "
              << v.capacity() << '\n';

    v.shrink_to_fit();
    std::cout << "shrink_to_fit: boyut " << v.size() << ", kapasite "
              << v.capacity() << '\n';
    return 0;
}
```

ÇIKTI

```
başlangıç: boyut 0, kapasite 0
boyut 1 -> kapasite 1
boyut 2 -> kapasite 2
boyut 3 -> kapasite 4
boyut 5 -> kapasite 8
boyut 9 -> kapasite 16
boyut 17 -> kapasite 32
reserve(100): boyut 20, kapasite 100
shrink_to_fit: boyut 20, kapasite 20
```

Bu çıktı g++ (libstdc++) ile alındı. Visual C++ kapasiteyi yaklaşık 1,5 katına çıkarır ve ara değerler farklı olur; standart yalnızca sonuçları (`size() <= capacity()`, ekleme işleminin ortalama sabit zamanlı olması) garanti eder. Yirmi eleman için yalnızca altı kez bellek ayrıldı, yirmi kez değil: bu yüzden `push_back` ortalamada çok hızlıdır.

İki işlev kapasiteyi doğrudan etkiler:

- `reserve(n)` kapasiteyi en az `n` yapar, boyutu **değiştirmez**. Kaç eleman ekleyeceğimizi önceden biliyorsak tekrarlı yeniden ayırmayı önler.
- `shrink_to_fit()` kullanılmayan kapasiteyi geri vermeyi **ister** (bağlayıcı bir emir değildir, ama yaygın uygulamalarda yerine getirilir).

`resize(n)` ile `reserve(n)` karıştırılmamalıdır: `resize` eleman sayısını değiştirir ve yeni elemanlar oluşturur; `reserve` yalnızca yer ayırır, `v[0]`'a yazmak için yine de eleman eklemek gerekir.

Yineleyiciler ve başvurular ne zaman geçersiz kalır?

Yeniden ayırma sırasında elemanlar **başka bir bellek adresine taşınır**. Eski adresleri tutan her şey (işaretçiler, referanslar, yineleyiciler) artık serbest bırakılmış belleği gösterir. Buna **geçersizleşme** (invalidation) denir ve kullanılması tanımsız davranıştır:

```
std::vector<int> v{1, 2, 3};
auto ilk = v.begin();
int& r = v[0];

v.push_back(4);           // kapasite aşılırsa elemanlar taşınır
std::cout << *ilk;        // tanımsız davranış olabilir
std::cout << r;           // tanımsız davranış olabilir
```

Burada kapasitenin tam olarak ne zaman aşıldığını bilmediğimiz için hatanın ortaya çıkıp çıkmayacağı da belirsizdir; kod bazen doğru çalışıyormuş gibi görünür. Genel kurallar şunlardır:

İşlem	Geçersiz kılar
`push_back`, `insert`, `emplace_back`	Yeniden ayırma olduysa tüm yineleyici ve referanslar; olmadıysa ekleme noktasından sonrakiler (`end()` her durumda)
`erase`	Silinen konumdan itibaren tüm yineleyici ve referanslar
`reserve`, `shrink_to_fit`	Kapasite değiştiyse hepsi
`clear`	Hepsi

Pratik çözüm basittir: bir vector'e eleman eklerken önceden alınmış yineleyici ya da referansı tutmayın; gerektiğinde `v[i]` ile **indeks** kullanın, çünkü indeks taşınmadan etkilenmez. `g++ -fsanitize=address` seçeneği (Bölüm 13'te tanıtılan araç) bu tür hataları çalışma anında yakalar.

> ◆ **Kritik Nokta · Vector'ün elemanları art arda durur**
>
> `&v[0]`, `v.data()` ile aynı adrestir ve elemanlar C dizisi gibi bitişiktir. Bu sayede C'den kalma işlevlere vector verebiliriz: `v.data()` ilk elemanın işaretçisini, `v.size()` eleman sayısını verir. Boş vector'de `v.data()`'nın göstereceği yer belirsizdir; boyut 0 ise bu işaretçiyi kullanmayız.

▸ cYoluyla.cpp

```
#include <iostream>
#include <vector>

// C tarzı işlev: işaretçi ve eleman sayısı alır (Bölüm 10)
double ortalama(const int* d, int n) {
    double toplam = 0;
    for (int i = 0; i < n; i++) {
        toplam += d[i];
    }
    return toplam / n;
}

int main() {
    std::vector<int> notlar{70, 85, 90, 55};
    std::cout << ortalama(notlar.data(), static_cast<int>(notlar.size()))
              << '\n';
    return 0;
}
```

```
ÇIKTI
75
```

> **▲ Sık Yapılan Hata · bool tutan vector özeldir**
>
> `std::vector<bool>`, bellek tasarrufu için her elemanı tek bir bitte saklayan özel bir uzmanlaşmadır. `&v[0]` ile bool işaretçisi alınamaz, `v.data()` yoktur ve `v[i]` gerçek bir `bool&` döndürmez. Bayrak dizisine gerçekten ihtiyacımız varsa `vector<char>` ya da `vector<int>` daha az sürprizlidir.

26.6 Vector'ü Fonksiyona Geçirmek

C dizisi fonksiyona hep adres olarak geçerdi; boyutu ayrıca vermemiz gerekirdi (Bölüm 10). `vector` ise sıradan bir nesne gibi davranır ve Bölüm 25'te gördüğümüz üç geçirme yolunun hepsi geçerlidir. Boyutu ayrı parametre olarak vermek gerekmez, çünkü `v.size()` her zaman doğrudur.

▸ vectorGecir.cpp

```cpp
#include <iostream>
#include <vector>

void kopyayla(std::vector<int> v) {          // tamamı kopyalanır
    v.push_back(99);
    v[0] = 100;
    std::cout << "kopyada: " << v.size() << ' ' << v[0] << '\n';
}

void referansla(std::vector<int>& v) {       // çağıranın vector'ü
    v.push_back(99);
    v[0] = 100;
}

void yazdir(const std::vector<int>& v) {     // yalnız okunur, kopya yok
    std::cout << v.size() << ':';
    for (int x : v) {
        std::cout << ' ' << x;
    }
    std::cout << '\n';
}

std::vector<int> kareler(int n) {            // vector döndürmek
    std::vector<int> sonuc;
    for (int i = 1; i <= n; i++) {
        sonuc.push_back(i * i);
    }
    return sonuc;
}

int main() {
    std::vector<int> sayilar{1, 2, 3};
    kopyayla(sayilar);
    yazdir(sayilar);
    referansla(sayilar);
    yazdir(sayilar);
    yazdir(kareler(5));
    return 0;
}
```

```
ÇIKTI
kopyada: 4 100
3: 1 2 3
4: 100 2 3 99
5: 1 4 9 16 25
```

Parametre	Kopyalanır mı?	Fonksiyon değiştirebilir mi?	Ne zaman
`vector<int> v`	Evet, tamamı	Yalnız kendi kopyasını	Kopyayla çalışmak gerekiyorsa
`vector<int>& v`	Hayır	Evet, çağırana yansır	Fonksiyon vector'ü değiştirecekse
`const vector<int>& v`	Hayır	Hayır	Fonksiyon vector'ü yalnız okuyacaksa

Okuyan fonksiyonlarda `const vector<T>&` yazmak alışkanlık olmalıdır: büyük bir vector'ün her çağrıda kopyalanması hem yavaştır hem de gereksizdir. Dizilerde bir fonksiyondan dizi döndüremezdik; `vector` döndürmek ise C++'ta sorunsuzdur. `return sonuc;` satırı vector'ü kopyalamaz, içindeki belleği çağırana **devreder**; bunun nasıl çalıştığını Bölüm 39'daki taşıma semantiğinde göreceğiz.

Kopyalama ve karşılaştırma

C dizisinde `b = a` derlenmezdi ve `a == b` adresleri karşılaştırırdı. `vector` ve `array` için ikisi de beklediğimiz gibi çalışır:

▸ vectorKopya.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> a{1, 2, 3};
    std::vector<int> b = a;              // bağımsız kopya
    std::cout << (a == b) << '\n';

    b[0] = 99;                           // yalnız b değişir
    std::cout << a[0] << ' ' << b[0] << ' ' << (a != b) << '\n';

    b = {7, 8};                          // yeni liste atanabilir
    std::cout << b.size() << '\n';
    std::cout << (a < b) << '\n';        // sözlük sırası: 1 < 7
    return 0;
}
```

```
ÇIKTI
1
1 99 1
2
1
```

`==` boyutların ve elemanların tek tek eşit olmasına bakar. `<`, `>` gibi işleçler **sözlük sırasıyla** (lexicographic) karşılaştırır: ilk farklı elemana göre karar verilir. Kopyalama elemanların kendisini çoğaltır; elemanlar işaretçiyse işaretçiler kopyalanır, gösterdikleri yerler değil.

26.7 `vector<vector<int>>` ile Matris

Bölüm 10'daki `int m[3][4]` biçimi, fonksiyona geçerken sütun sayısını sabit olarak yazmayı zorunlu kılıyordu. Her elemanı kendisi bir `vector<int>` olan bir vector bu kısıtlamaları kaldırır: `m[i][j]` yine "i. satırın j. elemanı"dır, ama boyutlar çalışma anında belirlenir. Tipin sonundaki iki `>` C++11'den beri bitişik yazılabilir.

vectorMatris.cpp

```
#include <iostream>
#include <vector>

using Matris = std::vector<std::vector<int>>;

Matris transpoz(const Matris& m) {
    std::size_t satir = m.size();
    std::size_t sutun = m[0].size();
    Matris t(sutun, std::vector<int>(satir));
    for (std::size_t i = 0; i < satir; i++) {
        for (std::size_t j = 0; j < sutun; j++) {
            t[j][i] = m[i][j];
        }
    }
    return t;
}

void yazdir(const Matris& m) {
    for (const std::vector<int>& satir : m) {
        for (int x : satir) {
            std::cout << x << ' ';
        }
        std::cout << '\n';
    }
}

int main() {
    Matris m(2, std::vector<int>(3, 0));   // 2 satır, 3 sütun, hepsi 0
    m[0] = {1, 2, 3};
    m[1][0] = 4;
    m[1][1] = 5;
    m[1][2] = 6;

    yazdir(m);
    std::cout << "--\n";
    yazdir(transpoz(m));

    m.push_back({7, 8});                   // uzunluğu farklı yeni satır
    std::cout << "satır: " << m.size() << ", son satır: "
              << m[2].size() << " eleman\n";
    return 0;
}
```

```
ÇIKTI
1 2 3
4 5 6
--
1 4
2 5
3 6
satır: 3, son satır: 2 eleman
```

`using Matris = ...` bir **tip takma adı**dır (C'deki `typedef`'in C++ karşılığı, Bölüm 14); uzun tip adını tekrar yazmayı önler. `Matris t(sutun, std::vector<int>(satir))` ifadesi "`sutun` tane satır, her biri `satir` elemanlı" demektir.

C tarzı iki boyutlu diziden iki fark vardır. Birincisi, satırların uzunlukları farklı olabilir (yukarıdaki son satır iki elemanlıdır); düzgün bir matris için bu sorumluluk bizdedir. İkincisi, satırlar bellekte art arda değildir: her satır ayrı bir bellek bloğudur. Çok büyük sayısal tablolarda bu hız farkı yaratabilir; böyle durumlarda tek bir `vector<int>` içinde `m[i * sutun + j]` gibi hesaplanan indeks kullanılır.

26.8 Algoritmalara İlk Bakış

Bölüm 10'da toplam, arama, ters çevirme ve sıralamayı elle yazmıştık; mantığı anlamak için gerekliydi. C++ standart kütüphanesi bu işlerin çoğunu `<algorithm>` başlığında hazır sunar. Algoritmalar bir **aralık** üzerinde çalışır ve aralığı yineleyicilerle alır: `v.begin()` başlangıç, `v.end()` bitiş (hariç). Bölüm 36'da bunların hepsini işleyeceğiz; burada en sık kullanılan beşini görelim.

▸ algoritmaOzet.cpp

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <iterator>
#include <vector>

int main() {
    std::vector<int> v{42, 7, 19, 7, 88, 3};

    std::sort(v.begin(), v.end());                // küçükten büyüğe
    for (int x : v) {
        std::cout << x << ' ';
    }
    std::cout << '\n';

    auto it = std::find(v.begin(), v.end(), 19);   // ilk 19'u bul
    if (it != v.end()) {
        std::cout << "19 bulundu, indeks " << (it - v.begin()) << '\n';
    }
    if (std::find(v.begin(), v.end(), 5) == v.end()) {
        std::cout << "5 yok\n";
    }

    std::cout << "7 sayısı: " << std::count(v.begin(), v.end(), 7)
              << " kez\n";

    auto en = std::max_element(v.begin(), v.end());
    std::cout << "en büyük " << *en << ", indeks " << (en - v.begin())
              << '\n';

    std::reverse(v.begin(), v.end());             // yerinde ters çevir
    std::cout << "ters: ";
    for (int x : v) {
        std::cout << x << ' ';
    }
    std::cout << '\n';

    std::sort(v.begin(), v.end(), std::greater<int>());  // büyükten küçüğe
    std::cout << "azalan: ";
    for (int x : v) {
        std::cout << x << ' ';
```

```
    }
    std::cout << '\n';

    int c[] = {5, 2, 9, 1};                        // C dizisiyle de çalışır
    std::sort(std::begin(c), std::end(c));
    for (int x : c) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
3 7 7 19 42 88
19 bulundu, indeks 3
5 yok
7 sayısı: 2 kez
en büyük 88, indeks 5
ters: 88 42 19 7 7 3
azalan: 88 42 19 7 7 3
1 2 5 9
```

Algoritmaların dönüş değerlerine dikkat edelim:

- `find` bulduğu elemanın **yineleyicisini**, bulamazsa `end()` döndürür; indeks değil. İndeksi bulmak için `it - v.begin()` hesaplanır.
- `max_element` (ve `min_element`) de yineleyici döndürür; değeri için `*` ile açarız. Boş aralıkta `end()` döner ve bu yineleyiciyi açmak tanımsız davranıştır.
- `count` eşleşen eleman sayısını verir.
- `sort` varsayılan olarak `<` ile küçükten büyüğe sıralar; üçüncü argüman olarak bir karşılaştırma nesnesi verilebilir (`std::greater<int>()` büyükten küçüğe). Kendi ölçütümüzü Bölüm 36'daki lambda ifadeleriyle yazacağız.
- Aynı işlevler `std::array`, C dizisi (`std::begin`/`std::end` ile) ve Bölüm 35'teki diğer kapsayıcılarla da çalışır. Bu birliktelik, elle yazılmış döngülere göre en büyük kazançtır.

> **İpucu · Elle yazmak mı, hazır algoritma mı?**
>
> Öğrenme amacıyla kabarcık sıralaması ya da doğrusal arama yazmak değerlidir, çünkü algoritmanın nasıl çalıştığını gösterir. Gerçek programlarda ise `std::sort` gibi hazır işlevler hem daha kısa hem daha hızlı hem de denenmiştir; kendi sıralamamızı yazmak için iyi bir nedenimiz olmalıdır.

26.9 Hangisini Seçmeli?

Üç kapsayıcı arasındaki seçim kısa bir kuralla özetlenebilir:

Durum	Seçim
Eleman sayısı çalışma anında değişiyor ya da belli değil	`std::vector`
Eleman sayısı derleme zamanında sabit ve küçük	`std::array`
Eski C kodu ya da C arayüzü ile uğraşıyoruz	C dizisi (gerekirse `vector::data()`)
Varsayılan tercih, emin değilsek	`std::vector`

Kitabın geri kalanında eleman topluluklarını tutmak için çoğunlukla `vector` kullanacağız.

Üç Dilde · Dinamik dizi: C, C++ ve C# *C · C++ · C#*

C'de büyüyen dizi `malloc` / `realloc` / `free` ile elle kurulur, boyut ayrı değişkende tutulur (Bölüm 13). C++'ta `vector` hepsini yapar. C#'ta karşılığı `List<T>` sınıfıdır (Bölüm 54); çöp toplayıcı belleği yönettiği için `free` karşılığı bir çağrı yoktur.

Özellik	C	C++ `vector<int>`	C# `List<int>`
Ekleme	`realloc` + elle sayaç	`v.push_back(x)`	`l.Add(x)`
Eleman sayısı	Ayrı değişken	`v.size()`	`l.Count`
Kapasite	Elle yönetilir	`v.capacity()`	`l.Capacity`
Sınır denetimi	Yok	`[]` yok, `at()` var	Her erişimde, `ArgumentOutOfRangeException`
Bellek serbest bırakma	`free(p)`	Otomatik (yıkıcı)	Çöp toplayıcı
Kopyalama	`memcpy` /elle döngü	`b = a` (derin kopya)	Referans kopyalanır; yeni liste için `new List<int>(l)`

```cpp
std::vector<int> v;      // C++
v.push_back(10);
v.push_back(20);
```

```csharp
var l = new List<int>(); // C#
l.Add(10);
l.Add(20);
```

C#'ta `List<T>` bir **referans tipidir**: `b = a` iki değişkenin aynı listeyi göstermesini sağlar. C++'ta `vector` bir **değer tipi** gibi davranır: `b = a` bağımsız bir kopya üretir. Bu fark, C++'tan C#'a geçerken en sık yanlış anlaşılan noktalardandır.

Üç Dilde · Sabit boyutlu diziler ve sınır denetimi *C · C++ · C#*

C dizisinde sınır denetimi yoktur; C++'ta `std::array` ve `vector` iki seçenek sunar: hızlı ama denetimsiz `[]` ve denetimli `at()`. C#'ta diziler sabit boyutlu referans tipleridir ve **her** erişim denetlenir; sınır dışı indeks `IndexOutOfRangeException` fırlatır, tanımsız davranış yoktur. Eleman sayısı `.Length` özelliğiyle alınır (Bölüm 47).

```csharp
int[] a = new int[5];    // C#: boyut çalışma anında verilebilir
a[5] = 1;                // IndexOutOfRangeException
```

Boyutu çalışma anında verilen dizi C#'ta geçerlidir, standart C++'ta değildir (orada `vector` kullanılır).

Bölüm Özeti

- C dizileri C++'ta aynen çalışır; farklar: süslü parantezle ilk değer verme, daralma dönüşümünün hata olması, listede fazla elemanın hata olması, standart C++'ta VLA'nın bulunmaması.

- `std::size(dizi)` (C++17) eleman sayısını tipten okur; dizi bir işaretçiye dönüşmüşse derlenmez. Bu, `sizeof(a) / sizeof(a[0])` tuzağından korur.
- Aralık tabanlı `for` (`int x`, `int& x`, `const int& x`) indeks gerektirmez; kopya, değiştirme ve salt okuma ayrımı `&` ve `const` ile yapılır.
- `std::array<T, N>` boyutu tipin parçası olan, `size()` bilen, `=` ile kopyalanabilen sabit dizidir; `[]` denetimsiz, `at()` denetimlidir (`out_of_range`).
- `std::vector<T>` (`<vector>`) çalışma anında büyür: `push_back`, `pop_back`, `insert`, `erase`, `resize`, `clear`, `front`, `back`, `empty`, `size`. Elemanlar art arda durur ve her zaman ilk değer alır.
- `vector<int> c(4, 7)` dört tane 7; `vector<int> d{4, 7}` iki elemanlı listedir.
- `size()` işaretsizdir (`size_t`). Boş vector'de `size() - 1` çok büyük bir sayıdır; sondan başa `for (size_t i = v.size(); i > 0; i--)` ve `v[i - 1]` ile gidilir.
- Aralıklar yarı açıktır: ilk sınır dahil, ikinci sınır hariç. `end()` son elemandan bir ötesini gösterir.
- `capacity()` yeniden ayırma yapmadan tutulabilecek eleman sayısıdır; büyüme genellikle katlamalıdır. `reserve(n)` yer ayırır, `resize(n)` eleman sayısını değiştirir.
- Yeniden ayırma ve `erase` yineleyicileri, işaretçileri ve referansları geçersiz kılabilir; dolaşırken boyutu değiştirmeyin, silerken `it = v.erase(it)` kalıbını kullanın.
- `v.data()` ve `v.size()` ile vector, işaretçi ve eleman sayısı bekleyen C tarzı işlevlere verilebilir.
- Okuyan fonksiyon `const vector<int>&`, değiştiren `vector<int>&` alır; değerle geçirmek tamamını kopyalar. `vector` fonksiyondan döndürülebilir.
- `=` vector'ü kopyalar, `==` ve `<` elemanları sözlük sırasıyla karşılaştırır. `vector<vector<int>>` boyutları çalışma anında belirlenen matristir.
- `<algorithm>`: `sort`, `find`, `count`, `max_element`, `min_element`, `reverse`. `find` ve `max_element` yineleyici döndürür; indeks için `it - v.begin()` kullanılır.
- C#'ta karşılığı `List<T>` ve sabit diziler `T[]`'dir; C# her erişimi denetler, C++'ta `vector` değer gibi kopyalanır, C#'ta `List<T>` referans tipidir.

Alıştırmalar

1. `std::vector<int> v{3, 3};` ve `std::vector<int> w(3, 3);` tanımlarından sonra `v.size()` ile `w.size()` sırasıyla kaçtır?

   A) 2 ve 2
   B) 3 ve 3
   C) 2 ve 3
   D) 3 ve 2
   E) Birinci tanım derlenmez.

2. Aşağıdaki kodla ilgili hangisi doğrudur?

```
void f(int a[]) {
    std::cout << std::size(a);
}
```

   A) Dizinin eleman sayısını yazar.
   B) İşaretçinin bayt sayısını yazar.

C) Her zaman 1 yazar.
D) Derleme hatası verir, çünkü `a` bir dizi değil, işaretçidir.
E) Çalışma anında istisna fırlatır.

3. `std::vector<int> v{1, 2, 3}; auto it = v.begin();` tanımlarından sonra hangi işlem `it`'i **kesin olarak** geçersiz kılar?

A) `v[0] = 9;`
B) `v.at(1);`
C) `v.clear();`
D) `v.size();`
E) `v.front();`

4. `std::array<int, 3> a{1, 2, 3}; std::array<int, 4> b{1, 2, 3, 4};` tanımlarından sonra `a = b;` satırı için hangisi doğrudur?

A) İlk üç eleman kopyalanır.
B) `a` dört elemanlı olur.
C) Derleme hatası verir; iki dizi farklı tiplerdir.
D) Çalışma anında `out_of_range` fırlatır.
E) Tanımsız davranıştır.

5. Boş bir `std::vector<int> bos;` için `bos.size() - 1` ifadesi nedir?

A) -1
B) 0
C) `size_t` tipinin en büyük değeri
D) Derleme hatası
E) `out_of_range` istisnası

6. Aşağıdaki programın çıktısı nedir?

```
std::vector<int> v{5, 1, 4};
v.push_back(9);
v.insert(v.begin() + 1, 7);
v.erase(v.begin() + 3);
v.resize(6, 2);
std::cout << v.size() << ' ' << v.front() << ' ' << v.back() << '\n';
int t = 0;
for (int x : v) { t += x; }
std::cout << t << '\n';
std::cout << v[v[2]] << '\n';
```

7. Aşağıdaki programın çıktısı nedir?

```
void a(std::vector<int> v) { v.push_back(1); }
void b(std::vector<int>& v) { v.push_back(2); v[0] = 5; }
void c(const std::vector<int>& v) {
    std::cout << v.size() << ' ' << v.back() << '\n';
}
int main() {
    std::vector<int> v{0};
    a(v);
    c(v);
    b(v);
```

```
    c(v);
    std::cout << v[0] << '\n';
}
```

8. Aşağıdaki programın çıktısı nedir?

```
std::vector<std::vector<int>> t(3, std::vector<int>(2, 1));
t[1][0] = 4;
t[2].push_back(9);
int s = 0;
for (const std::vector<int>& satir : t) {
    for (int x : satir) { s += x; }
}
std::cout << t.size() << ' ' << t[2].size() << '\n';
std::cout << s << '\n';
std::cout << t.at(1).at(1) << '\n';
```

9. Aşağıdaki programın çıktısı nedir?

```
std::vector<int> v{4, 8, 1, 8, 3};
std::sort(v.begin(), v.end());
std::cout << std::count(v.begin(), v.end(), 8) << '\n';
auto it = std::find(v.begin(), v.end(), 4);
std::cout << (it - v.begin()) << '\n';
std::reverse(v.begin(), v.end());
std::cout << v.front() << ' ' << *std::min_element(v.begin(), v.end())
          << '\n';
std::cout << (std::find(v.begin(), v.end(), 5) == v.end() ? "yok" : "var")
          << '\n';
```

10. Aşağıdaki kod çift sayıları silmek istiyor. Neden yanlıştır ve nasıl düzeltilir?

```
std::vector<int> v{1, 2, 3, 4};
for (int x : v) {
    if (x % 2 == 0) {
        v.erase(std::find(v.begin(), v.end(), x));
    }
}
```

11. Aşağıdaki döngü bir vector'ü sondan başa yazdırmak istiyor. Hatayı bulun ve düzeltin.

```
for (std::size_t i = v.size() - 1; i >= 0; i--) {
    std::cout << v[i] << ' ';
}
```

12. Aşağıdaki kodda derleme hatası veren ya da standart dışı olan satırları bulun.

```
int n = 3;
int a[n];
std::array<int, 3> b = {1, 2, 3, 4};
std::array<int, 3> c{1, 2, 3};
std::array<int, 3> d = c;
int e[2] = {1, 2.5};
```

13. Önce kaç sayı gireceğini, sonra o kadar tamsayıyı klavyeden okuyan `sayiAnalizi` programını yazın. Sayıları ters sırada, çift sayıların adedini ve en büyük sayıyı yazdırsın (sayı adedi en az 1).

```
ÇIKTI
Kaç sayı? 5
Sayılar: 4 9 2 7 6
Ters: 6 7 2 9 4
Çift sayı: 3
En büyük: 9
```

14. Satır ve sütun sayısını, ardından matrisin elemanlarını okuyan `transpozAl` programını yazın. Matrisi `vector<vector<int>>` içinde tutsun ve transpozunu yazdırsın.

```
ÇIKTI
Satır ve sütun: 2 3
1 2 3
4 5 6
Transpoz:
1 4
2 5
3 6
```

15. Klavyeden 0 girilene kadar tamsayı okuyup her sayının yalnızca **ilk** geçtiği yeri koruyan, yani tekrar edenleri atan `tekrarsiz` programını yazın. Sonunda kalan sayıları ilk geliş sırasıyla yazdırsın (`std::find` kullanın).

```
ÇIKTI
Sayılar (0 ile biter): 3 5 3 8 5 5 1 0
Tekrarsız: 3 5 8 1
```

BÖLÜM 27

std::string ve Karakter İşlemleri

BU BÖLÜMDE

- C karakter dizilerinin sorunlarını ve `std::string`'in bunları nasıl çözdüğünü açıklamayı
- `std::string` oluşturmayı, birleştirmeyi (`+`), karşılaştırmayı (`==`, `<`) ve `size`, `[]`, `at` ile erişmeyi
- `find`, `rfind`, `substr`, `insert`, `erase`, `replace` ve `npos` ile aramayı ve değiştirmeyi
- `std::to_string`, `std::stoi`, `std::stod` ve `std::stringstream` ile sayı ile metin arasında dönüşüm yapmayı
- `<cctype>` ile karakter sınıflandırmayı, büyük/küçük harf dönüştürmeyi; palindrom ve kelime sayma gibi klasik işleri yazmayı
- `c_str()` ile C kütüphaneleriyle çalışmayı; C, C++ ve C#'ta metin tiplerini karşılaştırmayı

Bölüm 11'de C'nin metin anlayışını ayrıntılı gördük: `'\0'` ile biten bir `char` dizisi, `<string.h>` fonksiyonları, `fgets`, arabellek taşması. Hepsi C++'ta da geçerlidir ve eski kodlarda karşımıza çıkar. Ancak C++ bu işi bizim yerimize yapan hazır bir tip sunar: `std::string`. Bu bölümde C'de elle yaptığımız her işin `std::string` ile nasıl kısaldığını göreceğiz.

Bölüm 11'i okuduğumuzu varsayıyoruz; ASCII tablosunu, null karakteri ve UTF-8 bayt sorununu yeniden anlatmayacağız. Yalnızca C++'ın getirdiği farklara ve yeni tuzaklara odaklanacağız. Bu bölümdeki tüm örnekler `std::` önekini açıkça yazar (Bölüm 22'deki gibi).

27.1 C Stringlerinden std::string'e

Bölüm 11'de C stringlerinin başımıza açtığı sorunları görmüştük. Bunları kısaca hatırlayalım:

- Boyut sabittir; uzun bir metin dizinin dışına taşar ve tanımsız davranış doğurur.
- Kopyalamak için `strcpy`, birleştirmek için `strcat`, karşılaştırmak için `strcmp` gerekir; `=`, `+`, `==` çalışmaz ya da yanlış şeyi yapar.
- Uzunluğu bulmak için `strlen` her seferinde diziyi baştan sona tarar.
- `'\0'` yönetimi programcının sorumluluğundadır.

`std::string` bu işleri üstlenir. `<string>` başlık dosyasıyla gelir ve aslında bir sınıftır (class); sınıfları Bölüm 30'da kendimiz yazmayı öğreneceğiz. Şimdilik onu hazır bir araç gibi kullanıyoruz.

Tanım: std::string

Karakter dizisini içeride kendisi saklayan, uzunluğunu kendisi bilen ve metin uzadıkça belleği kendisi büyüten standart kütüphane tipidir. Atama, birleştirme ve karşılaştırma işleçleri ile birçok yardımcı fonksiyon (üye fonksiyon) sunar.

Karşılaştırmalı bir tablo bu farkı özetler:

İş	C karakter dizisi	`std::string`
Boyut	Sabit, aşılırsa taşma	Kendiliğinden büyür
Kopyalama	`strcpy(hedef, kaynak)`	`hedef = kaynak;`
Birleştirme	`strcat(hedef, kaynak)` (yer yeterli olmalı)	`a + b`, `a += b`
Eşitlik	`strcmp(a, b) == 0` (`==` adresleri karşılaştırır)	`a == b`
Uzunluk	`strlen(a)`	`a.size()`
`'\0'`	Programcı yönetir	Gizli

Fark bellekte de görülür. Bölüm 11'de `"Ali"` dört baytlık bir dizi olarak tutuluyordu. `std::string` ise ayrı bir nesnedir; metni, uzunluğunu ve ayrılan yeri birlikte yönetir. Ayrıntıyı bilmemiz gerekmez, ama uzunluğun nesnenin içinde hazır durduğunu bilmek yeterlidir; bu yüzden `size()` her çağrıldığında metni taramaz.

27.2 Oluşturma ve İlk Değer

`std::string` birkaç biçimde oluşturulabilir:

stringTanim.cpp

```
#include <iostream>
#include <string>

int main() {
    std::string ad{"Ayşe"};
    std::string soyad = "Demir";
    std::string bos;
    std::string yildiz(5, '*');
    std::string kopya{ad};
    kopya += " Hanım";

    std::cout << ad << ' ' << soyad << ' ' << kopya << '\n';
    std::cout << '[' << bos << "] " << yildiz << '\n';
    std::cout << bos.empty() << ' ' << ad.empty() << '\n';
    return 0;
}
```

ÇIKTI

```
Ayşe Demir Ayşe Hanım
[] *****
1 0
```

Satırları sırayla açıklayalım:

- `std::string ad{"Ayşe"};` ve `std::string soyad = "Demir";` ilk değer vermedir. Süslü parantezli biçimi Bölüm 22'de görmüştük.

- `std::string bos;` ilk değer verilmese de **boş metinle** (`""`) başlar. C'deki çöp değer sorunu burada yoktur; bu, `int`'ten önemli bir farktır.
- `std::string yildiz(5, '*');` "şu karakterden şu kadar" anlamına gelir.
- `std::string kopya{ad};` bir **kopya** oluşturur. `kopya`'yı değiştirmek `ad`'ı etkilemez. C'de dizileri kopyalamak için `strcpy` gerekirdi.
- `empty()` metin boşsa `true` döndürür. `std::cout` bir `bool`'u varsayılan olarak `1` ya da `0` yazar.

> ▲ **Sık Yapılan Hata · Süslü parantez ile yuvarlak parantez**
>
> `std::string yildiz{5, '*'};` yazarsak beklediğimiz `*****` oluşmaz. Süslü parantez değerleri tek tek karakter listesi sayar ve kodu 5 olan görünmez bir karakter ile `'*'` içeren iki karakterlik bir metin üretir. "Şu karakterden şu kadar" kalıbı için yuvarlak parantez kullanılır: `std::string yildiz(5, '*');`.

27.3 Birleştirme ve Karşılaştırma

Birleştirme

`+` iki metni uç uca ekler, `+=` soldakinin sonuna ekler. Eklenen şey bir `std::string`, bir string sabiti ya da tek bir `char` olabilir:

▸ birlestirme.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string ad{"Elif"};
    std::string soyad{"Kaya"};
    std::string tam = ad + " " + soyad;
    std::string selam = "Merhaba " + tam;
    tam += '!';
    tam += " :)";

    std::cout << selam << '\n';
    std::cout << tam << '\n';
    return 0;
}
```

```
ÇIKTI
Merhaba Elif Kaya
Elif Kaya! :)
```

İfade soldan sağa işlenir: `ad + " "` bir `std::string` üretir, sonra bu `soyad` ile birleşir. Her adımda en az bir taraf `std::string` olduğu için sorun çıkmaz. İki string sabitini doğrudan toplamak ise derlenmez:

▸ sabitToplama.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string s = "Merhaba" + " dünya";
```

```
    std::cout << s << '\n';
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

sabitToplama.cpp: In function 'int main()':
sabitToplama.cpp:5:32: error: invalid operands of types 'const char [8]' and 'const char [8]' to
binary 'operator+'
    5 |     std::string s = "Merhaba" + " dünya";
      |                     ~~~~~~~~~ ^ ~~~~~~~~
      |                     |         |
      |                     |         const char [8]
      |                     const char [8]
```

String sabiti `std::string` değil, C'deki gibi ham bir karakter dizisidir (`const char` dizisi) ve iki ham diziyi toplamak anlamsızdır. Çözüm, zincirin başına bir `std::string` koymak ya da sabitleri yan yana yazmaktır: `"Merhaba" " dünya"` biçimindeki bitişik sabitleri derleyici zaten birleştirir.

> **▲ Sık Yapılan Hata · Sabit + sabit + string**
>
> `"Merhaba " + ad + "!"` derlenir, çünkü ilk toplama (`"Merhaba " + ad`) bir `std::string` verir. Ama `"Merhaba " + "dünya" + ad` derlenmez; ilk toplama iki sabit arasındadır. Sayıyı metne eklemek için `"Not: " + 85` yazılamaz; bunun için `std::to_string` kullanılır (bu bölümün ilerisinde).

Karşılaştırma

`==`, `!=`, `<`, `>`, `<=`, `>=` işleçleri `std::string`'lerde **içeriği** karşılaştırır. Bölüm 11'de C'de `==` yalnızca adresleri karşılaştırdığı için `strcmp` kullanmak zorundaydık; C++'ta bu tuzak ortadan kalkar. Sıralama karakter kodlarına göre soldan sağa yapılır (sözlük sırası, leksikografik karşılaştırma): ilk farklı karakter sonucu belirler, biri diğerinin başlangıcıysa kısa olan küçüktür.

▸ karsilastirma.cpp

```cpp
#include <iostream>
#include <string>

void karsilastir(const std::string& a, const std::string& b) {
    std::cout << a << " ve " << b << ": ";
    if (a == b) {
        std::cout << "eşit\n";
    } else if (a < b) {
        std::cout << "ilki küçük\n";
    } else {
        std::cout << "ilki büyük\n";
    }
}

int main() {
    karsilastir("Ali", "Ali");
    karsilastir("Ali", "Veli");
    karsilastir("Ali", "Alice");
    karsilastir("ali", "Ali");
    karsilastir("kedi", "Zebra");
    return 0;
}
```

```
ÇIKTI
Ali ve Ali: eşit
Ali ve Veli: ilki küçük
Ali ve Alice: ilki küçük
ali ve Ali: ilki büyük
kedi ve Zebra: ilki büyük
```

`Ali` ile `Veli` ilk harfte ayrılır (`'A'` 65, `'V'` 86). `Ali` ile `Alice`'te üç harf aynıdır ve `Ali` orada biter; kısa olan küçüktür. `"ali"` ile `"Ali"` karşılaştırmasında küçük `'a'` (97) büyük `'A'`'dan (65) büyüktür. Tüm büyük harfler tüm küçük harflerden önce geldiği için `"kedi"`, `"Zebra"`'dan büyük çıkar. Büyük/küçük harfi yok sayan karşılaştırma istiyorsak iki tarafı da önce küçük harfe çevirmemiz gerekir (aşağıda `kucukYap` fonksiyonunda yazacağız).

Fonksiyonun parametrelerinin `const std::string&` olduğunu fark edelim. Bu, metnin kopyalanmadan salt okunur geçirilmesidir; Bölüm 25'te ayrıntısını gördük. `std::string` büyük olabileceği için fonksiyonlara böyle vermek iyi bir alışkanlıktır. `"Ali"` gibi bir sabit bu parametreye verildiğinde derleyici onu geçici bir `std::string`'e çevirir.

27.4 Uzunluk ve Karaktere Erişim

`size()` ve `length()` aynı işi yapar ve karakter sayısını (aslında bayt sayısını, bu bölümün sonundaki UTF-8 başlığına bakın) döndürür. `s[i]` indeksi 0'dan başlayan karaktere erişir; `front()` ilk, `back()` son karakterdir. `s[i]` bir değer atanabilen ifadedir: karakteri yerinde değiştirir.

▸ erisim.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string s{"Merhaba"};
    std::cout << s.size() << ' ' << s.length() << '\n';
    std::cout << s[0] << s[3] << ' ' << s.at(6) << '\n';
    std::cout << s.front() << s.back() << '\n';

    s[0] = 'm';
    s.at(6) = 'A';
    std::cout << s << '\n';
    return 0;
}
```

```
ÇIKTI
7 7
Mh a
Ma
merhabA
```

`size()` işaretli bir `int` değil, işaretsiz bir tamsayı olan `std::size_t` döndürür. Bölüm 5'teki işaretli-işaretsiz karşılaştırma tuzağı burada da geçerlidir; bunu döngülerde göreceğiz.

[] ile at() farkı

`s[i]` ile `s.at(i)` aynı karaktere ulaşır. Fark, geçersiz indekste ortaya çıkar. `s[i]` indeksi **denetlemez**; geçersizse tanımsız davranış oluşur. `s.at(i)` indeksi denetler ve geçersizse `std::out_of_range` türünde bir

istisna (exception) fırlatır. İstisna mekanizmasını Bölüm 34'te ayrıntılı göreceğiz; şimdilik "hata, programın akışını kesen bir olay olarak bildirilir ve `catch` ile yakalanabilir" diye okuyabiliriz:

▸ atSinir.cpp

```cpp
#include <iostream>
#include <stdexcept>
#include <string>

int main() {
    std::string s{"abc"};
    try {
        std::cout << s.at(10) << '\n';
    } catch (const std::out_of_range&) {
        std::cout << "indeks sınırın dışında\n";
    }
    std::cout << "program sürüyor\n";
    return 0;
}
```

```
ÇIKTI
indeks sınırın dışında
program sürüyor
```

Yakalanmayan bir istisna programı sonlandırır. İndeksin geçerli olduğundan kendi döngümüzle emin olduğumuz yerde `[]` yeterlidir ve daha hızlıdır; emin olmadığımız yerde `at()` hatayı sessizce geçirmez.

27.5 Dolaşma

Bir metnin her karakterine tek tek bakmak çok yaygındır. İki yolumuz var. **Aralık tabanlı** `for` (Bölüm 24) karakterleri sırayla bir değişkene verir; yalnızca okuyacaksak en temiz yazımdır. **İndeksli** `for` ise `s[i]` ile hem okumaya hem değiştirmeye izin verir.

▸ stringDongu.cpp

```cpp
#include <cctype>
#include <iostream>
#include <string>

int main() {
    std::string s{"Merhaba Dunya 2025"};
    int harf = 0;
    int rakam = 0;

    for (char c : s) {
        if (std::isalpha(static_cast<unsigned char>(c))) {
            harf++;
        } else if (std::isdigit(static_cast<unsigned char>(c))) {
            rakam++;
        }
    }
    std::cout << "harf: " << harf << ", rakam: " << rakam << '\n';

    for (std::size_t i = 0; i < s.size(); i++) {
        s[i] = static_cast<char>(std::toupper(static_cast<unsigned char>(s[i])));
    }
    std::cout << s << '\n';

    for (char& c : s) {
```

```
        c = static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
    }
    std::cout << s << '\n';
    return 0;
}
```

```
ÇIKTI
harf: 12, rakam: 4
MERHABA DUNYA 2025
merhaba dunya 2025
```

İlk döngü yalnızca okuduğu için aralık tabanlıdır. İkinci döngü indeksle her harfi büyütür. Üçüncü döngü `char&` kullanır: `&` olmasaydı `c` karakterin yalnızca bir kopyası olurdu ve değişiklik metne yansımazdı (Bölüm 24'teki `auto&` ile aynı fikir). `static_cast<unsigned char>` dönüşümünün nedenini `<cctype>` başlığında açıklayacağız.

size_t ve geriye doğru döngü

İndeks değişkeni için `std::size_t` kullanırız; çünkü `size()`, `find()` ve `substr()` bu tipi kullanır. `int i` ile `i < s.size()` karşılaştırması derleyicinin işaretli-işaretsiz uyarısını (`-Wsign-compare`) tetikler.

İşaretsiz olmanın bir tuzağı da geriye doğru döngüdür:

```
for (std::size_t i = s.size() - 1; i >= 0; i--) {   // hatalı
    std::cout << s[i];
}
```

`std::size_t` hiçbir zaman negatif olmaz; `i` 0 iken `i--` onu çok büyük bir sayıya taşır ve `i >= 0` her zaman doğru kalır. Döngü bitmez ve `s[i]` sınırın çok dışına çıkar. Güvenli kalıp, `i`'yi "kaç karakter kaldı" diye sayıp içeride `i - 1` ile erişmektir:

```
for (std::size_t i = s.size(); i > 0; i--) {
    std::cout << s[i - 1];
}
```

> **▲ Sık Yapılan Hata · Boş string ve size() - 1**
>
> `s.size() - 1` işaretsiz bir çıkarmadır. `s` boşsa sonuç -1 değil, çok büyük bir sayıdır. `s.size() - 3 > 0` gibi karşılaştırmalar da kısa metinlerde yanlış sonuç verir; çıkarma yerine `s.size() > 3` yazmalıyız. Son karaktere erişmeden önce `s.empty()` denetlemek ya da `s.back()` kullanmak (o da boş metinde geçersizdir) gerekir.

27.6 Arama: find ve npos

Bir metnin içinde başka bir metin ya da karakter aramak çok yapılan bir iştir (C'de `strchr` ve `strstr` bunun karşılığıydı). `find(aranan)` aranan şeyin ilk bulunduğu indeksi döndürür; aranan bir karakter ya da metin olabilir. İkinci parametre aramanın başlayacağı indeksi belirler. `rfind` sondan başa doğru arar.

Bulunamazsa ne olur? `-1` döndürmez. `std::string::npos` adlı özel bir değer döndürür. Bu, `std::size_t` tipinin alabileceği en büyük sayıdır (yani `-1`'in işaretsiz hâli). Bulunup bulunmadığını her zaman `npos` ile karşılaştırarak denetleriz:

aramaFind.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string s{"Merhaba dünya, merhaba"};
    std::cout << s.find("haba") << '\n';
    std::cout << s.find('a') << ' ' << s.find('a', 5) << '\n';
    std::cout << s.rfind("haba") << '\n';

    std::size_t konum = s.find("xyz");
    if (konum == std::string::npos) {
        std::cout << "xyz bulunamadı\n";
    } else {
        std::cout << "xyz " << konum << ". indekste\n";
    }
    return 0;
}
```

ÇIKTI

```
3
4 6
19
xyz bulunamadı
```

`"Merhaba"` içinde `"haba"` 3. indekste başlar. `find('a')` ilk `a`'yı (4), `find('a', 5)` 5. indeksten sonrakini (6) bulur. `rfind("haba")` sondaki `merhaba`'nın içindeki `haba`'yı yakalar. `ü` karakteri UTF-8'de iki bayt olduğu için 19, karakter sayarak bulacağımız 18'den bir fazladır; `find` bayt indeksi verir (bu bölümün sonundaki UTF-8 başlığına bakın). Küçük harfle başlayan ikinci `merhaba`, büyük `M` ile yazılandan farklıdır: arama büyük/küçük harfe duyarlıdır.

Bir şeyin kaç kez geçtiğini bulmak için aramayı her bulunan konumdan bir sonrasından yeniden başlatırız:

```cpp
std::size_t konum = metin.find("ana");
while (konum != std::string::npos) {
    say++;
    konum = metin.find("ana", konum + 1);
}
```

▲ Sık Yapılan Hata · find sonucunu doğrudan if'e vermek

`if (s.find("ab"))` yaygın bir hatadır. Bulunamazsa `npos` döner ve bu sıfırdan farklı olduğu için koşul doğru çıkar; `"ab"` metnin en başındaysa 0 döner ve koşul yanlış çıkar. Yani tam tersi çalışır. Doğrusu `if (s.find("ab") != std::string::npos)` yazmaktır. Sonucu `int` değişkene atamak da kötü alışkanlıktır; `std::size_t` kullanılmalıdır.

◆ Kritik Nokta · npos ekrana -1 yazmaz

`find` bulamadığında `npos` döner. `npos`'u yazdırırsak `-1` değil, `std::size_t`'in en büyük değeri çıkar (64 bitlik sistemlerde 18446744073709551615). Bulundu-bulunmadı denetimi için her zaman `npos` ile `==` ya da `!=` karşılaştırması kullanılır.

27.7 Parçalama ve Değiştirme

`substr(başlangıç, uzunluk)` metnin bir parçasını **yeni bir** `std::string` olarak döndürür; asıl metin değişmez. Uzunluk verilmezse başlangıçtan sona kadar alınır. Başlangıç metnin uzunluğundan büyükse `std::out_of_range` istisnası fırlatılır.

Metnin kendisini değiştiren üye fonksiyonlar şunlardır:

Fonksiyon	Ne yapar?
`insert(i, metin)`	`i`. indeksin önüne `metin` ekler
`erase(i, n)`	`i`. indeksten başlayarak `n` karakter siler; `erase(i)` sona kadar siler
`replace(i, n, metin)`	`i`'den başlayan `n` karakteri `metin` ile değiştirir
`push_back(c)` / `pop_back()`	Sona bir karakter ekler / son karakteri siler
`append(metin)`	Sona ekler (`+=` ile aynı)
`clear()`	Metni boşaltır

▸ degistirme.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    std::string s{"Merhaba Dunya"};
    std::cout << s.substr(8, 3) << ' ' << s.substr(8) << '\n';

    s.insert(7, " guzel");
    std::cout << s << '\n';

    s.erase(0, 8);
    std::cout << s << '\n';

    s.replace(0, 5, "iyi");
    std::cout << s << '\n';

    s.push_back('!');
    std::cout << s << ' ' << s.size() << '\n';

    s.pop_back();
    s.erase(3);
    std::cout << '[' << s << "]\n";

    s.clear();
    std::cout << s.size() << '\n';
    return 0;
}
```

ÇIKTI

```
Dun Dunya
Merhaba guzel Dunya
guzel Dunya
iyi Dunya
iyi Dunya! 10
[iyi]
0
```

Adım adım izleyelim. `insert(7, " guzel")`, `"Merhaba"` yedi karakter olduğu için 7. indeksin önüne ekleme yapar. `erase(0, 8)` baştaki `"Merhaba "` kısmını (boşluk dahil sekiz karakter) siler. `replace(0, 5, "iyi")` ilk beş karakteri (`guzel`) üç karakterlik `iyi` ile değiştirir; uzunluk değişse de sorun çıkmaz. Sondaki `erase(3)` tek parametreyle çağrıldığı için 3. indeksten sona kadar siler ve geriye `iyi` kalır.

> **▲ Sık Yapılan Hata · erase(3) tek karakter silmez**
>
> `erase(3)` "3. karakteri sil" demek değildir; "3. indeksten sona kadar sil" demektir. Tek bir karakteri silmek için `erase(3, 1)` yazılır. Bu fonksiyonlar iç içe kullanıldığında her adımda metnin güncel hâlini alt alta yazıp indeksleri tek tek saymak, zihinden hesaplamaktan çok daha güvenlidir.

> **İpucu · += tercih edin**
>
> Uzun bir metni adım adım kurarken `s += parca;` yazın. `s = s + parca;` aynı sonucu verir ama her seferinde geçici bir `std::string` oluşturur. Küçük programlarda fark görünmez; yine de doğru alışkanlıktır.

27.8 Sayı ile Metin Arasında Dönüşüm

Sayıyı metne, metni sayıya çevirmek sık gereken bir iştir: bir notu mesajın içine koymak, kullanıcıdan gelen `"42"` ile hesap yapmak. `"42"` metni otomatik olarak 42 sayısına dönüşmez. C'de bunun için `atoi`, `strtol`, `snprintf` kullanıyorduk (Bölüm 11); C++ aynı işi tek satırda yapan fonksiyonlar sunar:

Yön	Fonksiyon	Örnek
Sayı ⟶ string	`std::to_string(sayi)`	`std::to_string(42)` ⟶ `"42"`
string ⟶ `int`	`std::stoi(s)`	`std::stoi("42")` ⟶ 42
string ⟶ `long long`	`std::stoll(s)`	`std::stoll("5000000000")`
string ⟶ `double`	`std::stod(s)`	`std::stod("2.5")` ⟶ 2.5

▸ donusumTemel.cpp

```
#include <iostream>
#include <string>

int main() {
    int puan{85};
    std::string mesaj = "Puan: " + std::to_string(puan);
    std::cout << mesaj << '\n';
    std::cout << std::to_string(3.5) << '\n';

    std::cout << std::stoi("123") + 1 << '\n';
    std::cout << std::stoi("  42") << ' ' << std::stoi("-7") << '\n';
    std::cout << std::stod("2.5") * 2 << '\n';

    std::cout << std::stoi("12abc") << '\n';
    std::cout << std::stod("3,14") << '\n';
    return 0;
}
```

```
ÇIKTI
Puan: 85
3.500000
124
42 -7
5
12
3
```

Çıktıdan çıkan sonuçlar şunlardır:

- `std::to_string(3.5)` `3.5` değil `3.500000` verir; ondalıklı sayılarda her zaman altı basamak yazar. Biçimi ayarlamak istiyorsak akış düzenleyicilerini (`std::fixed`, `std::setprecision`; Bölüm 23) ya da `stringstream`'i kullanırız.
- `std::stoi` baştaki boşlukları atlar ve işaretleri anlar.
- `std::stoi("12abc")` hata vermez: baştaki sayıyı alır (12) ve geri kalanı yok sayar.
- `std::stod("3,14")`, Türkçe alışkanlığıyla yazılan virgülü anlamaz ve 3'te durur. Ondalık ayırıcı noktadır.

Peki metin sayıyla hiç başlamıyorsa? `std::stoi("abc")` bir değer üretemez ve 0 döndürmez; `std::invalid_argument` istisnası fırlatır. `int`'e sığmayacak kadar büyük bir sayı için `std::out_of_range` fırlatılır. C'deki `atoi("abc")` ise sessizce 0 dönerdi. Hatalı girdiyi kontrollü karşılamak için dönüşümü `try` / `catch` içine alırız:

guvenliDonusum.cpp

```cpp
#include <iostream>
#include <stdexcept>
#include <string>

int sayiyaCevir(const std::string& metin, int varsayilan) {
    try {
        return std::stoi(metin);
    } catch (const std::invalid_argument&) {
        return varsayilan;
    } catch (const std::out_of_range&) {
        return varsayilan;
    }
}

int main() {
    std::cout << sayiyaCevir("250", -1) << '\n';
    std::cout << sayiyaCevir("abc", -1) << '\n';
    std::cout << sayiyaCevir("99999999999", -1) << '\n';
    return 0;
}
```

```
ÇIKTI
250
-1
-1
```

Burada yalnızca kalıbı kullandık; `try`, `catch` ve istisna türleri Bölüm 34'te anlatılacak. `"7x"` gibi yarım sayıları da reddetmek istiyorsak `stoi`'nin ikinci parametresini kullanırız: bu parametre, sayının nerede bittiğini bildiren bir indeks değişkeninin adresidir. Ancak bu kitapta ayrıntıya girmeyeceğiz; gerekirse metni önce `std::isdigit` ile denetlemek yeterlidir.

stringstream ile dönüşüm ve ayrıştırma

`<sstream>` başlığındaki `std::stringstream`, `std::cin` ve `std::cout` gibi bir **akıştır** (stream; Bölüm 23); ama kaynağı klavye, hedefi ekran değil, bir `std::string`'dir. Aynı `<<` ve `>>` işleçleriyle metin kurabilir ya da metin ayrıştırabiliriz. Bu, `std::to_string`'in yetmediği yerlerde (biçimli yazım) ve bir metni parçalarına ayırırken işe yarar:

▸ stringstreamOrnek.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

int main() {
    std::ostringstream yaz;
    yaz << "Ortalama: " << std::fixed << std::setprecision(2) << 78.456;
    std::string sonuc = yaz.str();
    std::cout << sonuc << '\n';

    std::string cumle{"Bugun  hava cok guzel"};
    std::istringstream oku{cumle};
    std::vector<std::string> kelimeler;
    std::string kelime;
    while (oku >> kelime) {
        kelimeler.push_back(kelime);
    }
    std::cout << kelimeler.size() << " kelime:";
    for (const std::string& k : kelimeler) {
        std::cout << " [" << k << ']';
    }
    std::cout << '\n';

    std::istringstream csv{"Mert,20,Ankara"};
    std::string parca;
    while (std::getline(csv, parca, ',')) {
        std::cout << parca << '|';
    }
    std::cout << '\n';

    std::istringstream sayilar{"10 20 30"};
    int n = 0;
    int toplam = 0;
    while (sayilar >> n) {
        toplam += n;
    }
    std::cout << "toplam: " << toplam << '\n';
    return 0;
}
```

ÇIKTI

```
Ortalama: 78.46
4 kelime: [Bugun] [hava] [cok] [guzel]
Mert|20|Ankara|
toplam: 60
```

Örnekte iki sınıf kullandık: `std::ostringstream` yalnızca yazmak, `std::istringstream` yalnızca okumak içindir; `std::stringstream` ikisini birden yapar. `yaz.str()` biriken metni verir. `oku >> kelime` boşlukları atlayarak sıradaki kelimeyi alır, okunacak bir şey kalmayınca koşulu yanlış yapar ve döngü biter; çift boşluk

sorun çıkarmadı. `std::getline(csv, parca, ',')` virgüle kadar okur; son alan virgülle bitmediği hâlde yine okunur. Aynı `>>` döngüsü `int` ile de çalışır, böylece `"10 20 30"` gibi metinden sayıları ayıklayabiliriz.

27.9 Satır Okumak: getline

Klavyeden bir kelime okumak için `std::cin >> ad` yeterlidir ama okuma boşlukta durur. Boşluklu bir satırı okumak için `std::getline(std::cin, degisken)` kullanılır (ayrıntılar Bölüm 23'te). C'deki `fgets`'ten üç farkı vardır: hedef `std::string` olduğu için boyut sorunu yoktur, satır sonu `'\n'` okunur ama `std::string`'e **konmaz**, bu yüzden sonradan silmek gerekmez.

Bölüm 6'daki `scanf` tuzağının bir benzeri burada da var: `std::cin >> sayi` Enter'ın `'\n'` karakterini tamponda bırakır, hemen ardından gelen `getline` bunu görür ve kullanıcıya hiç sormadan boş bir metinle döner. Çare, ikisinin arasındaki satır sonunu atmaktır:

girdiSirasi.cpp

```cpp
#include <iostream>
#include <limits>
#include <string>

int main() {
    int yas = 0;
    std::string ad;

    std::cout << "Yaş: ";
    std::cin >> yas;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    std::cout << "Ad soyad: ";
    std::getline(std::cin, ad);

    std::cout << ad << ", " << yas << " yaşında\n";
    std::cout << "ad uzunluğu: " << ad.size() << '\n';
    return 0;
}
```

```
ÇIKTI
Yaş: 21
Ad soyad: Zeynep Aksoy
Zeynep Aksoy, 21 yaşında
ad uzunluğu: 12
```

`std::cin.ignore(sınır, '\n')` en fazla `sınır` karakteri, `'\n'` görene kadar atar. Sınır olarak `std::numeric_limits<std::streamsize>::max()` (Bölüm 22) yazmak, satır ne kadar uzun olursa olsun tamamını atmayı garanti eder. `ignore` satırı kaldırılırsa `getline` boş döner ve ikinci sorunun cevabı hiç beklenmez. Tersi sorun yoktur: `getline`'dan sonra `std::cin >>` kullanmak sorunsuzdur.

Dosya sonuna kadar satır okumak için `while (std::getline(std::cin, satir)) { ... }` yazılır; okuma başarısız olunca `getline` koşulu yanlış yapar. Üçüncü bir parametre verilirse satır sonu yerine o karakterde durur; yukarıda virgüle kadar okurken bunu kullandık.

27.10 cctype: Karakter Sınıflandırma ve Dönüştürme

Bölüm 11'de `<ctype.h>` fonksiyonlarını görmüştük. C++'ta aynıları `<cctype>` başlığından gelir ve `std::` ile yazılır. Bir karakterin harf mi rakam mı olduğunu `c >= '0' && c <= '9'` gibi koşullarla da yazabiliriz, ama hazır fonksiyonlar daha okunaklıdır.

Fonksiyon	Doğru döndürür, eğer karakter...
`std::isalpha(c)`	harfse
`std::isdigit(c)`	rakamsa (`'0'` ... `'9'`)
`std::isalnum(c)`	harf ya da rakamsa
`std::isupper(c)` / `std::islower(c)`	büyük / küçük harfse
`std::isspace(c)`	boşluk, sekme, `'\n'` gibi boşluk karakteriyse
`std::ispunct(c)`	noktalama işaretiyse
`std::toupper(c)` / `std::tolower(c)`	büyük / küçük harfe çevrilmiş hâlini döndürür; harf değilse aynısını

`is...` fonksiyonları `bool` değil `int` döndürür ve "doğru" için sıfırdan farklı bir değer verir; kesin değerin 1 olduğu garanti edilmez. Bu yüzden `if (std::isdigit(c))` yazılır, `== 1` ile karşılaştırılmaz.

Asıl tuzak `toupper` ve `tolower`'dadır: ikisi de `char` değil **`int`** döndürür. `std::cout` bir `int` gördüğü için karakteri değil kodunu yazar:

▸ donusumTuzagi.cpp

```cpp
#include <cctype>
#include <iostream>
#include <string>

int main() {
    char k{'q'};
    std::cout << std::toupper(k) << '\n';
    std::cout << static_cast<char>(std::toupper(k)) << '\n';

    char buyuk = static_cast<char>(std::toupper(k));
    std::cout << buyuk << '\n';

    (void)std::toupper(k);   // sonuç kullanılmıyor: k değişmez
    std::cout << k << '\n';

    std::string s{"abc"};
    s += std::toupper('d');
    std::cout << s << '\n';
    return 0;
}
```

ÇIKTI

```
81
Q
Q
q
abcD
```

İlk satır `'Q'`'nun kodu olan 81'i yazdırdı. Karakter görmek için sonucu `static_cast<char>` ile dönüştürürüz ya da bir `char` değişkene atarız. `std::toupper(k);` gibi bir satır sonucu hiçbir yere koymadığı için işe

yaramaz (örnekte uyarı çıkmasın diye `(void)` yazdık): `k` hâlâ `q`'dur. Sonuncu satırda `s += std::toupper('d')` çalıştı, çünkü `+=` operatörü gelen `int`'i `char` olarak ekler. Ama `s = s + std::toupper('d')` derlenmez: `std::string` ile `int`'i toplayan bir `+` yoktur. O durumda `s = s + static_cast<char>(...)` yazılır.

Parametre neden unsigned char?

`<cctype>` fonksiyonlarının parametresi `int`'tir ve yalnızca `unsigned char` aralığındaki (0-255) değerler ile `EOF` için tanımlıdır. `char` işaretli olabildiğinden ASCII dışı bir karakter negatif görünür ve doğrudan vermek tanımsız davranıştır. Bu yüzden Türkçe harf içerebilecek metinde güvenli yazım `std::isalpha(static_cast<unsigned char>(c))` biçimindedir. Yukarıdaki örneklerde bunu bilerek kullandık. Ayrıca bu fonksiyonlar yalnızca ASCII içindir; `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` harflerini harf saymaz (Bölüm 11'deki UTF-8 uyarısı geçerlidir).

> **Not · Büyük/küçük harf dönüştürmeyi tekrar kullanmak**
>
> Metnin tamamını dönüştürmek için yukarıdaki döngüleri her seferinde yazmayız. Fonksiyon olarak ayırmak ve Bölüm 36'daki `std::transform` algoritması ile çözmek de mümkündür. Biz bu bölümde döngü yazarak ilerliyoruz; çünkü hem mantığı görmek, hem de Bölüm 36 öncesinde yineleyici bilgisi gerektirmemek istiyoruz.

27.11 Klasik Metin İşleri

Bölüm 11'de ters çevirme ve palindrom kontrolünü `char` dizileriyle ve indeks hesaplarıyla yazmıştık. `std::string` ile aynı algoritmalar çok daha kısa olur; çünkü uzunluk hazırdır, `'\0'` düşünmeyiz ve karşılaştırma tek bir `==`'dir.

Ters çevirme ve palindrom

Ters çevirmek için `s[i]` ile karşısındaki `s[n - 1 - i]` yer değiştirir; ortaya kadar (`n / 2`) gitmek yeter. **Palindrom**, tersiyle aynı okunan sözcük ya da cümledir (`kayak`). Büyük/küçük harf ve noktalama göz ardı edilecekse önce yalnızca harf ve rakamları küçük harfe çevirerek temiz bir kopya çıkarılır, sonra kopya tersiyle karşılaştırılır.

▸ tersPalindrom.cpp

```
#include <cctype>
#include <iostream>
#include <string>

std::string tersCevir(std::string s) {
    std::size_t n = s.size();
    for (std::size_t i = 0; i < n / 2; i++) {
        char gecici = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = gecici;
    }
    return s;
}

bool palindromMu(const std::string& s) {
    std::string temiz;
    for (char c : s) {
        unsigned char b = static_cast<unsigned char>(c);
        if (std::isalnum(b)) {
```

```
            temiz += static_cast<char>(std::tolower(b));
        }
    }
    return temiz == tersCevir(temiz);
}

int main() {
    std::string s{"Merhaba"};
    std::cout << tersCevir(s) << ' ' << s << '\n';
    std::cout << (palindromMu("Radar") ? "evet" : "hayır") << '\n';
    std::cout << (palindromMu("Ali") ? "evet" : "hayır") << '\n';
    std::cout << (palindromMu("A man, a plan, a canal: Panama")
                      ? "evet" : "hayır") << '\n';
    return 0;
}
```

```
ÇIKTI
abahreM Merhaba
evet
hayır
evet
```

`tersCevir` parametreyi **değer** olarak aldığı için kendi kopyasını çevirir ve sonucu döndürür; `main`'deki `s` değişmedi. Tek uzunlukta ortadaki karakter kendi karşısındadır ve yerinde kalır; boş metinde döngü hiç dönmez. `palindromMu` ise parametreyi salt okunur referansla alır, çünkü yalnızca bakar. C'de aynı iş için `strcpy` ile kopya, elle uzunluk hesabı ve `'\0'` yerleştirme gerekirdi.

Kütüphanede hazır bir `std::reverse` de vardır (`<algorithm>`, yineleyicilerle çalışır; Bölüm 36). Burada döngüyü yazdık, çünkü indeks mantığı string işlemlerinin temelidir.

Büyük/küçük harf ve başlık biçimi

Küçük/büyük harfe duyarsız karşılaştırma, başlık biçimine çevirme gibi işler küçük yardımcı fonksiyonlarla yazılır:

▸ metinAraclari.cpp

```
#include <cctype>
#include <iostream>
#include <string>

std::string kucukYap(std::string s) {
    for (char& c : s) {
        c = static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
    }
    return s;
}

std::string baslikYap(std::string s) {
    bool yeniKelime = true;
    for (char& c : s) {
        unsigned char b = static_cast<unsigned char>(c);
        if (std::isspace(b)) {
            yeniKelime = true;
        } else if (yeniKelime) {
            c = static_cast<char>(std::toupper(b));
            yeniKelime = false;
        } else {
            c = static_cast<char>(std::tolower(b));
        }
```

```
    }
    return s;
}

int kelimeSay(const std::string& s) {
    int say = 0;
    bool icerde = false;
    for (char c : s) {
        if (std::isspace(static_cast<unsigned char>(c))) {
            icerde = false;
        } else if (!icerde) {
            icerde = true;
            say++;
        }
    }
    return say;
}

int main() {
    std::string a{"Elma"};
    std::string b{"eLMA"};
    std::cout << (a == b) << ' ' << (kucukYap(a) == kucukYap(b)) << '\n';

    std::cout << baslikYap("merHABA  dunyA nasilsiN") << '\n';

    std::cout << kelimeSay("  bir iki   üç ") << '\n';
    std::cout << kelimeSay("") << '\n';
    return 0;
}
```

```
ÇIKTI
0 1
Merhaba  Dunya Nasilsin
3
0
```

`kelimeSay`, bir durum değişkeni (`icerde`) tutar: kelimenin dışından içine geçtiğimiz anda sayacı artırır. Böylece ardışık boşluklar kelimeyi birden çok kez saymaz. `baslikYap` aynı fikri kullanır: boşluktan sonraki ilk harf büyür, geri kalanlar küçülür. Her iki fonksiyon da boşluk dışındaki her şeyi kelimenin parçası sayar; noktalama işaretlerini ayırmak istersek `std::ispunct` ile ek denetim ekleriz.

27.12 C ile Birlikte Çalışma: c_str ve Ham Diziler

Bir `std::string` ile C kütüphanesi arasında köprü gerekebilir: eski bir fonksiyon `const char*` bekler. `c_str()` üye fonksiyonu, sonunda `'\0'` bulunan C tarzı bir metin verir. Dönen işaretçi yalnızca string değişmediği ve yaşadığı sürece geçerlidir. İşaretçilerin kendisini Bölüm 28'de ayrıntılı ele alacağız; burada onu "C fonksiyonuna verilen salt okunur metin" olarak kullanıyoruz.

▸ cStrKopru.cpp

```
#include <cstdio>
#include <cstring>
#include <iostream>
#include <string>

int main() {
    char a[] = "Ali";
    std::cout << sizeof a << ' ' << std::strlen(a) << '\n';
```

```
    std::cout << sizeof('A') << ' ' << sizeof("A") << '\n';

    std::string s = a;
    s += " Veli";
    std::cout << s << ' ' << s.size() << '\n';

    std::printf("%s\n", s.c_str());
    std::cout << std::strlen(s.c_str()) << '\n';

    std::string t{"ab"};
    t.push_back('\0');
    t += "cd";
    std::cout << t.size() << ' ' << std::strlen(t.c_str()) << '\n';
    return 0;
}
```

```
ÇIKTI
4 3
1 2
Ali Veli 8
Ali Veli
8
5 2
```

İlk iki satır Bölüm 11'in tanıdık sonuçlarıdır: `"Ali"` dört bayttır, `sizeof('A')` ise C++'ta 1'dir (C'de 4'tü; Bölüm 22). `"A"` iki baytlık dizi olduğu için `char c = "A";` derlenmez. Son satır ince bir farkı gösterir: `std::string` içinde `'\0'` da bir karakter olabilir ve `size()` onu sayar; oysa C fonksiyonu (`strlen`) ilk `'\0'`'da durur. Bu yüzden `printf("%s", s.c_str())` gibi çağrılarda metin `'\0'` içeriyorsa kesilir.

Tersi yönde dönüşüm de kolaydır: `std::string s = cMetin;` bir C metnini kopyalayarak `std::string` yapar. `std::strcpy` gibi taşmaya açık C fonksiyonlarına yeni codda gerek kalmaz.

27.13 Türkçe Karakterler ve Bayt Uzunluğu

Bölüm 11'de gördüğümüz UTF-8 sorunu `std::string` için de aynıdır: C++ standart kütüphanesindeki `std::string` karakter değil **bayt** dizisidir. Türkçe harfler iki bayt tuttuğundan `size()` karakter sayısını değil bayt sayısını verir.

▸ turkceBayt.cpp

```
#include <cctype>
#include <iostream>
#include <string>

int main() {
    std::string s{"ağaç"};
    std::cout << s.size() << '\n';

    int karakter = 0;
    int harf = 0;
    for (char c : s) {
        unsigned char b = static_cast<unsigned char>(c);
        if (b < 0x80 || b >= 0xC0) {
            karakter++;
        }
        if (std::isalpha(b)) {
            harf++;
        }
```

```
    }
    std::cout << "karakter: " << karakter << ", isalpha: " << harf << '\n';
    return 0;
}
```

```
ÇIKTI
6
karakter: 4, isalpha: 2
```

`"ağaç"` dört karakterdir ama `ğ` ve `ç` ikişer bayt olduğu için `size()` 6 verir. UTF-8'de çok baytlı bir karakterin ilk baytı `0xC0` ve üstündedir, devam baytları `0x80`-`0xBF` arasındadır; döngü yalnızca ilk baytları ve ASCII karakterleri sayarak gerçek karakter sayısını bulur. `std::isalpha` ise yalnızca iki `a`'yı harf sayar. (Bu `0xC0` denetimi, Bölüm 11'deki `& 0xC0` denetimiyle aynı şeyi söyler.)

Pratik sonuç: bu bölümdeki araçlar ASCII metin içindir. `size()` Türkçe harf içeren metinde bayt sayısı verir; `s[i]` çok baytlı bir karakterin yarısına ulaşabilir; `std::toupper` `ç`'yi büyütemez ve `i`'yi `İ` değil `I` yapar; `<` ve `>` Türkçe alfabe sırasını vermez; indeksle ters çevirirsek çok baytlı karakterlerin baytları ters sıraya girer ve çıktı bozulur. Türkçe harfleri yalnızca ekrana yazılan sabit metinlerde kullanmak güvenlidir, nitekim bu bölümdeki örnek çıktılarda öyle yaptık.

27.14 Üç Dilde Metin

Üç Dilde · Üç Dilde Metin *C · C++ · C#*

C'de metin `char` dizisidir ve her işi `<string.h>` fonksiyonları ile elle yaparız. C++'ta `std::string` işleçlerle ve üye fonksiyonlarla bu işi üstlenir. C#'ta `string` dilin temel bir tipidir ve **değişmezdir** (immutable): bir `string` nesnesi oluşturulduktan sonra içindeki karakterler değiştirilemez; "değiştiren" her işlem yeni bir `string` döndürür. Çok sayıda değişiklik yapılacaksa `StringBuilder` kullanılır.

İş	C	C++	C#
Tanım	`char s[] = "Ali";`	`std::string s{"Ali"};`	`string s = "Ali";`
Uzunluk	`strlen(s)`	`s.size()`	`s.Length`
Birleştirme	`strcat(h, k)`	`a + b` , `a += b`	`a + b` , `$"{a}{b}"`
Eşitlik	`strcmp(a, b) == 0`	`a == b`	`a == b`
Karakter değiştirme	`s[0] = 'a';`	`s[0] = 'a';`	Mümkün değil (değişmez)
Arama	`strstr(s, "an")`	`s.find("an")` (`npos`)	`s.IndexOf("an")` (-1)
Alt metin	elle kopya	`s.substr(i, n)`	`s.Substring(i, n)`
Değiştirme	elle	`s.replace(i, n, m)`	`s.Replace("a", "b")`
Sayıya çevirme	`atoi` , `strtol`	`std::stoi`	`int.Parse` , `int.TryParse`
Sayıdan metne	`snprintf`	`std::to_string`	`n.ToString()`
Çok değişiklik	dizi boyutunu elle yönetmek	`s += parca;`	`StringBuilder`

Bir C# örneği farkı gösterir:

```
string s = "merhaba";
// s[0] = 'M';              // derleme hatası: string değişmez
string t = s.Replace("m", "M");   // yeni string döndürür
Console.WriteLine(s + " " + t);   // merhaba Merhaba
```

`std::string` ise değiştirilebilir bir tiptir (`s[0] = 'M';` geçerlidir); bu yüzden fonksiyona referansla geçirildiğinde asıl metin değişebilir, `const` ile korumak bizim işimizdir. C#'ta ise metinler UTF-16 kodlu `char` değerlerinden oluşur ve `"Ağaç".Length` 4 verir; C++'ın `std::string`'i bayt sayar ve 6 verir. C#'ın dönüşüm tarafı da C++'tan ayrılır: `int.Parse("abc")` bir `FormatException` fırlatır; hatasız deneme için `int.TryParse` bulunur (C# bölümlerinde ayrıntılı göreceğiz).

Üç dilde de ortak ders aynıdır: metin işlemede sınırları ve kodlamayı bilmek gerekir. C bunu tamamen programcıya bırakır, C++ boyut ve bellek yönetimini üstlenir ama bayt tabanlı kalır, C# ise değişmezlik ve UTF-16 ile hem güvenlik hem kolaylık sağlar.

Bölüm Özeti

- `std::string` (`<string>`), C karakter dizilerinin sorunlarını çözen standart kütüphane tipidir: boyutunu kendisi büyütür, uzunluğunu bilir, `'\0'` yönetimini gizler. İlk değer verilmezse boş metinle başlar.
- Atama `=`, birleştirme `+` / `+=`, karşılaştırma `==`, `<` işleçleriyle yapılır. `==` içeriği karşılaştırır (C'deki adres tuzağı yoktur). Sıralama karakter koduna göredir: büyük harfler küçüklerden önce gelir.
- İki string sabiti toplanamaz: `"a" + "b"` derlenmez. Zincirin başında bir `std::string` olmalıdır; sayıyı metne eklemek için `std::to_string` kullanılır.
- `std::string yildiz(5, '*')` beş yıldız üretir; süslü parantezli `{5, '*'}` ise karakter listesi sayılır. Yuvarlak ve süslü parantez farklı anlamdadır.
- `size()` ve `length()` bayt sayısını işaretsiz `std::size_t` olarak verir. `s[i]` sınırı denetlemez (tanımsız davranış), `s.at(i)` denetler ve `std::out_of_range` fırlatır.
- Döngüde indeks için `std::size_t` kullanılır; `i >= 0` ile geriye doğru döngü ve boş metinde `s.size() - 1` işaretsiz taşma yüzünden hatalıdır.
- `find`, `rfind` ilk/son konumu döndürür; bulamazsa `std::string::npos` döner (`-1` değil, `size_t`'in en büyük değeri). `if (s.find(x))` yazılmaz, `!= std::string::npos` ile karşılaştırılır.
- `substr(i, n)` kopya döndürür; `insert`, `erase`, `replace`, `push_back`, `pop_back`, `append`, `clear` metni yerinde değiştirir. `erase(i)` i'den sona kadar siler.
- Sayıdan metne `std::to_string` (ondalıkta altı basamak), metinden sayıya `std::stoi`, `std::stoll`, `std::stod` kullanılır. `stoi("12abc")` 12 verir; `stoi("abc")` `std::invalid_argument` fırlatır (C'deki `atoi` 0 dönerdi).
- `std::stringstream` ailesi (`<sstream>`) bir metni akış gibi okur/yazar: `while (ak >> kelime)` kelimelere, `std::getline(ak, parca, ',')` ayraca göre böler, `std::ostringstream` biçimli metin kurar.
- `std::getline(std::cin, s)` boşluklu satırı okur ve `'\n'`'i metne koymaz; `std::cin >>` sonrasında önce `std::cin.ignore(...)` gerekir.
- `<cctype>` fonksiyonları (`std::isalpha`, `std::isdigit`, `std::toupper`…) `int` alır ve döndürür; `toupper`/`tolower` sonucu `cout`'a doğrudan verilirse sayı yazılır. Argüman `unsigned char`'a dönüştürülerek verilir.

- `c_str()` C fonksiyonları için `const char*` verir. `std::string` içinde `'\0'` bulunabilir; `size()` onu sayar, `strlen` onda durur.
- `std::string` UTF-8'de bayt sayar: `"ağaç".size()` 6'dır; `cctype` ve karşılaştırmalar yalnızca ASCII içindir. C#'ın `string`'i değişmezdir ve UTF-16 kodludur (`"ağaç".Length` 4).

Alıştırmalar

1. Aşağıdaki kod parçasının çıktısı nedir?

```
std::cout << std::toupper('b') << ' '
          << static_cast<char>(std::toupper('b')) << '\n';
```

   A) `B B`
   B) `b B`
   C) `66 B`
   D) `98 B`
   E) `66 66`

2. `std::string s{"merhaba"};` tanımından sonra `s.find("z")` ifadesi ne döndürür?

   A) 0
   B) -1 (ekrana `-1` yazdırılır)
   C) `s.size()`
   D) `std::out_of_range` istisnası fırlatır.
   E) `std::string::npos`

3. Aşağıdaki kod parçasının çıktısı nedir?

```
std::string s{"programlama"};
s.erase(0, 3);
s.replace(0, 4, "XY");
std::cout << s << ' ' << s.size();
```

   A) `XYlama 6`
   B) `XYlama 8`
   C) `XYmlama 7`
   D) `XYama 5`
   E) `XYlama 11`

4. UTF-8 kodlamasıyla çalışan bir g++ ile aşağıdaki kod parçası ne yazdırır?

```
std::string s{"çay"};
std::cout << s.size();
```

   A) 3
   B) 2
   C) 5

D) 4
E) Derleme hatası verir.

5. Aşağıdaki kod parçasının çıktısı nedir?

```
std::cout << std::stoi("42abc") + 1 << '\n';
```

A) 42
B) 43
C) `42abc1`
D) Program derlenmez.
E) `stoi` istisna fırlatır ve program sonlanır.

6. Aşağıdakilerden hangisi derlenir?

A) `std::string s = "Merhaba" + " dünya";`
B) `std::string s = "a" + "b" + std::string("c");`
C) `std::string s = std::string("Merhaba") + " dünya";`
D) `std::string s = 'a' + 'b';`
E) `std::string s = "abc" - "b";`

7. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <string>

int main() {
    std::string s{"banana"};
    std::cout << s.find("an") << ' ' << s.rfind("an") << ' ';
    std::cout << s.find('n', 3) << '\n';
    s.erase(1, 2);
    s.replace(s.find('n'), 1, "ll");
    std::cout << s << ' ' << s.size() << '\n';
    std::cout << s.substr(1, 3) << '\n';
    return 0;
}
```

8. Aşağıdaki program ekrana ne yazdırır?

```
#include <cctype>
#include <iostream>
#include <string>

int main() {
    std::string s{"a1b2c3"};
    int toplam = 0;
    std::string harfler;
    for (std::size_t i = 0; i < s.size(); i++) {
        if (std::isdigit(static_cast<unsigned char>(s[i]))) {
            toplam += s[i] - '0';
        } else {
            harfler += static_cast<char>(
                std::toupper(static_cast<unsigned char>(s[i])));
        }
    }
    std::cout << toplam << ' ' << harfler << '\n';
```

```
    std::cout << s[0] + 1 << ' ' << static_cast<char>(s[0] + 1) << '\n';
    return 0;
}
```

9. Aşağıdaki program çalışırken kullanıcı önce `5` yazıp Enter'a basıyor, sonra ikinci satıra `Ali` yazıyor. Program ekrana ne yazdırır? Köşeli parantezlerin içine ve sayılara dikkat edin.

```
#include <iostream>
#include <string>

int main() {
    int n{0};
    std::string ad;
    std::cin >> n;
    std::getline(std::cin, ad);
    std::cout << n << '[' << ad << "] " << ad.size() << '\n';
    std::getline(std::cin, ad);
    std::cout << ad << '\n';
    return 0;
}
```

10. Aşağıdaki kod parçası metni sondan başa yazdırmak istiyor ama istenen sonucu vermiyor. Hatayı bulun ve düzeltin.

```
std::string s{"abc"};
for (std::size_t i = s.size() - 1; i >= 0; i--) {
    std::cout << s[i];
}
```

11. Aşağıdaki fonksiyon metinde `"ab"` bulunup bulunmadığını söylemek istiyor. Hangi durumlarda yanlış sonuç verir? Düzeltin.

```
bool abVarMi(const std::string& s) {
    if (s.find("ab")) {
        return true;
    }
    return false;
}
```

12. Aşağıdaki program ekrana ne yazdırır? Her adımda metnin güncel hâlini yazarak ilerleyin.

```
#include <iostream>
#include <string>

int main() {
    std::string s{"C++ ogrenmek"};
    s.insert(3, " dilini");
    std::cout << s << '\n';
    s.erase(s.size() - 3);
    std::cout << s << '\n';
    s.replace(s.find("ogren"), 5, "bil");
    std::cout << s << ' ' << s.size() << '\n';
    std::cout << s.substr(s.find(' ') + 1, 5) << '\n';
    return 0;
}
```

13. Kullanıcıdan `std::getline` ile bir satır okuyun. Satırdaki harfleri, rakamları ve boşlukları sayıp aşağıdaki biçimde yazdıran `karakterSay` programını yazın. Harf, rakam ve boşluk dışında kalan işaretler (`!`, `,` gibi) sayılmasın.

```
ÇIKTI
Bir satır yazın: Merhaba Dunya 2025!
Harf: 12
Rakam: 4
Boşluk: 2
```

14. Kullanıcıdan bir cümle okuyun ve kelimelerin sırasını tersine çevirerek yazdırın (kelimelerin kendisi değişmez). Boşluk sayısı birden fazla olabilir; çıktıda kelimeler tek boşlukla ayrılsın.

```
ÇIKTI
Cümle: bir iki uc dort
dort uc iki bir
```

15. `sezar(metin, kaydirma)` fonksiyonunu yazın: yalnızca İngilizce harfleri alfabede `kaydirma` kadar ileri kaydırsın (`z` sonrası `a`'ya dönsün), büyük/küçük harf korunsun, diğer karakterler değişmesin. `sezar("Hello, World xyz", 3)` çağrısının sonucu `Khoor, Zruog abc` olmalı; sonucu 23 kaydırmayla tekrar işleme sokarsanız özgün metin gelmeli.
16. `hepsiniDegistir(metin, eski, yeni)` fonksiyonunu yazın: `metin` içindeki `eski` metninin tüm geçişlerini `yeni` ile değiştirsin ve sonucu döndürsün. `eski` boşsa metin değişmesin. Değiştirilen kısmın içinde yeniden arama yapmayın (`"aaaa"` metninde `"a"` yerine `"aa"` konunca sonsuz döngüye girilmemeli; sonuç `"aaaaaaaa"` olmalı).

BÖLÜM 28

İşaretçiler ve Referanslar

BU BÖLÜMDE

- C işaretçilerinin C++'ta nelerde aynı, nelerde daha katı olduğunu ayırt etmeyi
- Referansın kurallarını (ilk değer zorunlu, yeniden bağlanmaz, boş olamaz) ve işaretçiden farkını açıklamayı
- Referans ile işaretçiyi gereksinime göre seçmeyi; işaretçiye referans (`int*&`) kullanmayı
- `nullptr` ile "değer yok" durumunu ifade etmeyi
- `const` doğruluğunu (const correctness) uygulamayı: `const T*` , `T* const` , `const T&`
- Referans döndürmeyi ve sarkan referans tuzağından kaçınmayı
- `this` işaretçisini, lvalue ve rvalue ayrımını sezgisel düzeyde tanımayı

Bölüm 12'de işaretçileri, Bölüm 22'de referansın ilk tanımını gördük. Bu bölüm ikisini yan yana koyar. C++'ta bir nesneye "kendisi olarak" ulaşmanın iki yolu vardır ve hangisini ne zaman seçeceğimiz, C++ kodunun okunurluğunu doğrudan etkiler. C'de yalnızca işaretçi vardı; bu yüzden `scanf("%d", &x)` ve `takas(&a, &b)` gibi çağrılar bize olağan göründü. C++'ta aynı işlerin çoğu referansla yapılır ve çağrı yerinde hiçbir `&` ya da `*` gerekmez.

Bölümün ikinci yarısı referansın yan etkilerini ele alır: referans döndürmek, sarkan referans ve lvalue/rvalue ayrımı. Son ayrım, Bölüm 39'daki taşıma semantiğinin (move semantics) zeminidir; burada yalnızca sezgisini kuracağız.

28.1 C İşaretçileri C++'ta

İşaretçinin tanımı, `&` ile adres alma, `*` ile dolaylama (dereference), işaretçi aritmetiği, dizi bozunması ve fonksiyona adres geçirme C++'ta Bölüm 12'dekiyle aynıdır. Bu konuları yeniden anlatmayacağız. Yalnızca C++'ın farklılaştığı noktaları sıralayalım:

Konu	C	C++
Boş işaretçi	`NULL`	`nullptr` (Bölüm 22)
İlk değer verme	`int *p = &x;`	`int* p{&x};` (daraltma korumalı)
`void*`'ten dönüşüm	Örtük, tür dökümü gerekmez	Örtük değil; `static_cast` gerekir
Metin sabiti	`char *s = "Ali";` uyarısız	`const char*` zorunlu; `char*` hataya yakın
`cout` ile adres yazdırma	`printf("%p", (void *)p)`	`cout << p` (`char*` hariç, aşağıya bakın)
Metin için	`char[]` ve `char*`	`std::string` (Bölüm 27)

`void*` farkı pratikte en çok `malloc` çağrısında görünür. C'de `int *p = malloc(sizeof(int));` yazılabilir; C++ ise `void*`'ten `int*`'e örtük dönüşüme izin vermez:

▸ voidHata.cpp

```cpp
#include <cstdlib>

int main() {
    int* p = malloc(sizeof(int));
    free(p);
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
voidHata.cpp: In function 'int main()':
voidHata.cpp:4:20: error: invalid conversion from 'void*' to 'int*' [-fpermissive]
    4 |     int* p = malloc(sizeof(int));
      |              ~~~~~~^~~~~~~~~~~~~
      |                    |
      |                    void*
```

C++'ta dinamik bellek için `malloc` yerine `new` ve akıllı işaretçiler kullanırız; bunlar Bölüm 29'un konusudur. Bu hata mesajını yalnızca C kodunu C++ derleyicisine verdiğimizde karşılaşacağımız bir fark olarak not edelim.

İkinci fark `cout` ile ilgilidir. `cout << p` çoğu işaretçi için adresi onaltılık yazar (`0x7ffd...`). Ancak `char*` türünde bir işaretçi verirsek `cout` onu bir C metni sayar ve `'\0'` bulana kadar karakterleri yazar. `const char*` bu yüzden metin gibi, başka tür işaretçiler adres gibi basılır:

▸ cKalinti.cpp

```cpp
#include <iostream>

int main() {
    const char* metin{"Merhaba"};
    std::cout << metin << '\n';
    std::cout << *metin << '\n';
    std::cout << metin + 3 << '\n';

    char harf{'A'};
    char* ph{&harf};
    std::cout << (static_cast<void*>(ph) != nullptr) << '\n';
    return 0;
}
```

ÇIKTI

```
Merhaba
M
haba
1
```

`metin + 3` ifadesi dördüncü karakteri gösteren yeni bir `const char*`'tır ve `cout` oradan itibaren yazar. `ph`'yi doğrudan `cout`'a verseydik, `harf`'ten sonra `'\0'` bulunmadığı için bellekte rastgele ilerleyen tanımsız davranış doğardı. Bir `char` nesnesinin adresini görmek istiyorsak `static_cast<void*>(ph)` ile türü `void*`'e çevirirız. Adresin sayısal değeri her çalıştırmada değiştiğinden burada yazdırmak yerine yalnızca boş olup olmadığını sınadık.

28.2 Referans: Kurallar ve Davranış

Bölüm 22'de referansı "var olan bir nesneye verilen ikinci ad" diye tanımlamıştık. Şimdi kurallarını, işaretçiyle yan yana koymadan önce tek tek örnekleyelim.

> **Tanım: Referans (reference)**
> Tanımlandığı anda bir nesneye bağlanan ve ömrü boyunca o nesnenin takma adı olarak kalan addır. Referans üzerinden yapılan her okuma ve yazma, bağlandığı nesne üzerinde yapılır.

refKurallar.cpp

```cpp
#include <iostream>
#include <vector>

int main() {
    int x{10};
    int& r{x};
    int y{50};

    r = y;                      // x = y anlamına gelir
    y = 99;
    std::cout << "x = " << x << ", y = " << y << ", r = " << r << '\n';
    std::cout << std::boolalpha << (&r == &x) << '\n';
    std::cout << sizeof(r) << ' ' << sizeof(x) << '\n';

    int a[3] = {1, 2, 3};
    int& ilk{a[0]};
    ilk = 100;
    std::cout << a[0] << '\n';

    std::vector<int> v{1, 2, 3};
    for (int& e : v) {
        e *= 10;
    }
    for (int e : v) {
        std::cout << e << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
x = 50, y = 99, r = 50
true
4 4
100
10 20 30
```

Çıktıdan çıkan sonuçlar şunlardır:

- `r = y;` satırı `r`'yi `y`'ye bağlamadı; `y`'nin değerini `r`'nin bağlı olduğu `x`'e yazdı. Sonraki `y = 99;` `x`'i etkilemediği için `r` hâlâ 50 gösterir. **Referans bir kez bağlanır, yeniden bağlanamaz.**
- `&r == &x` doğrudur: referans ayrı bir nesne değil, aynı nesnenin ikinci adıdır. Referansa uygulanan `&` operatörü, bağlandığı nesnenin adresini verir; referansın kendi adresi diye bir şey yoktur.
- `sizeof(r)` referansın değil, bağlandığı `int`'in boyutunu verir (4).
- Referans yalnızca değişkene değil, dizi elemanına da bağlanabilir (`a[0]`).

- Aralık tabanlı `for` döngüsünde (Bölüm 24) `int& e` yazmak, elemanın kopyası yerine kendisini verir; bu yüzden `e *= 10` vektörün içeriğini değiştirir. İkinci döngüde `int e` kopya aldığı için yalnızca okuyoruz.

Referansın kurallarını üç madde hâlinde toplayalım:

1. Tanımlanırken ilk değer zorunludur (`int& r;` derlenmez; Bölüm 22).
2. Bağlandığı nesneyi sonradan değiştiremez.
3. Const olmayan referans geçici değere (rvalue) bağlanamaz.

Üçüncü kuralın hata mesajına bakalım:

refHata.cpp

```
int main() {
    int& s{5};
    return s;
}
```

DERLEYİCİ ÇIKTISI

```
refHata.cpp: In function 'int main()':
refHata.cpp:2:13: error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of
type 'int'
    2 |     int& s{5};
      |             ^
```

`5` bir değişken değil, adı ve kalıcı adresi olmayan bir değerdir. Ona bağlanan bir `int&` üzerinden yazmak anlamsız olurdu. Mesajdaki "non-const lvalue reference" ve "rvalue" terimlerini bölümün sonunda açacağız.

Sabit referans geçiciye bağlanabilir

`const` referans bu kuralın istisnasıdır: yalnızca okuma sözü verdiği için geçici değere bağlanabilir. Üstelik derleyici geçici nesnenin ömrünü referansın ömrüne kadar uzatır:

gecici.cpp

```
#include <iostream>
#include <string>

std::string adSoyad() {
    return "Elif Demir";
}

int main() {
    const std::string& a{adSoyad()};
    std::cout << a << ' ' << a.size() << '\n';
    const double& oran{0.18};
    std::cout << oran * 100 << '\n';
    return 0;
}
```

ÇIKTI

```
Elif Demir 10
18
```

`adSoyad()` bir `std::string` geçici nesnesi döndürür. Normalde bu nesne ifadenin sonunda yok olurdu; `const&`'a bağlandığı için `a` kapsamdan çıkana kadar yaşar. Bu özellik, fonksiyon parametresi olarak `const`

`std::string&` yazdığımızda `yazdir("metin")` gibi çağrıların da çalışmasını sağlar: derleyici `"metin"` için geçici bir `std::string` yaratır ve parametreye bağlar.

> ◆ **Kritik Nokta · Referansın bellekteki gerçeği**
>
> C++ standardı referansın nasıl gerçekleştirileceğini söylemez; yalnızca takma ad gibi davranmasını ister. Pratikte derleyiciler referansı çoğu zaman gizli bir işaretçi olarak uygular, ya da en iyi durumda hiç bellek ayırmadan doğrudan asıl nesneyi kullanır. Bu yüzden "referansın adresi" ya da "referans dizisi" diye bir kavram yoktur. Referansı bir işaretçi gibi düşünmemiz gerekmez; kodun anlamı, referansın aynı nesnenin ikinci adı olduğudur.

28.3 Referans mı, İşaretçi mi?

Her iki araç da bir nesneye dolaylı ulaşmamızı sağlar. Farkları aşağıdaki tabloda toplanmıştır:

Özellik	İşaretçi (`T*`)	Referans (`T&`)
Boş olabilir mi?	Evet (`nullptr`)	Hayır
İlk değer zorunlu mu?	Hayır (verilmezse çöp değer)	Evet
Başka nesneye yönelebilir mi?	Evet	Hayır
Kullanım	`*p` , `p->üye`	Doğrudan `r` , `r.üye`
Çağrıda	`f(&x)`	`f(x)`
Aritmetik	Var (`p + 1`)	Yok
Dizisi olur mu?	Evet (`int* d[3]`)	Hayır
Kendi adresi var mı?	Evet (`&p` , `int**`)	Hayır (`&r` asıl nesnenin adresidir)
`sizeof`	İşaretçinin boyutu (8)	Asıl nesnenin boyutu

Bu farklar bir karar kuralı verir:

- Parametre her zaman var olacaksa ve çağrı onu değiştirecekse ya da büyük bir nesneyi kopyalamadan okuyacaksa **referans** (`T&` , `const T&`).
- "Yok" olabilen bir nesne için ya da çalışma sırasında başka nesneye yönelmesi gereken durumlarda **işaretçi** (`T*`).
- C tarzı dizi ve eleman sayısı çifti, C kütüphanesi arayüzleri ve bir sonraki bölümün dinamik belleği için **işaretçi**.

Özetle: önce referansı düşünürüz, referans yetmiyorsa işaretçiye geçeriz. Parametre geçirme biçimlerinin ayrıntılı karşılaştırması (değer, işaretçi, referans, `const` referans) Bölüm 25'te yer alır; burada yalnızca bir noktaya bakalım.

İşaretçiye referans

Bölüm 12'de bir fonksiyonun çağıranın işaretçisini değiştirebilmesi için `int**` geçirmemiz gerektiğini görmüştük. C++'ta aynı iş `int*&` ile, yani işaretçiye referansla yapılır:

refIsaretci.cpp

```cpp
#include <iostream>

void yonelt(int*& p, int& yeni) {
    p = &yeni;
}

void yoneltC(int** p, int* yeni) {
    *p = yeni;
}

int main() {
    int x{5};
    int y{8};
    int z{13};
    int* p{&x};

    yonelt(p, y);
    std::cout << *p << '\n';
    yoneltC(&p, &z);
    std::cout << *p << '\n';

    int* q{nullptr};
    std::cout << std::boolalpha << (q == nullptr) << ' '
              << static_cast<bool>(p) << '\n';
    return 0;
}
```

```
ÇIKTI
8
13
true true
```

`int*&` türünü sağdan sola okuruz: "`p`, bir `int` işaretçisine referanstır". Fonksiyon gövdesinde `p = &yeni;` dememiz, çağıranın `p`'sini doğrudan değiştirir; `**` ve `&` kalabalığı kalkar. İki biçim aynı sonucu verir. Son satırda bir işaretçinin koşul olarak kullanılışı görülür: boş değilse `true`, boşsa `false` sayılır.

28.4 nullptr ile "Değer Yok"

Referans boş olamadığı için "bu nesne olmayabilir" bilgisini yalnızca işaretçi taşıyabilir. Bölüm 22'de gördüğümüz `nullptr` bu işin standart yoludur. Tipik kullanım, bulamama ihtimali olan bir arama fonksiyonudur:

ilkNegatif.cpp

```cpp
#include <iostream>

const int* ilkNegatifiBul(const int* dizi, int adet) {
    for (int i = 0; i < adet; i++) {
        if (dizi[i] < 0) {
            return &dizi[i];
        }
    }
    return nullptr;
}

int main() {
    int a[4] = {5, 8, -2, 7};
```

```
    int b[2] = {1, 2};

    const int* p{ilkNegatifiBul(a, 4)};
    if (p != nullptr) {
        std::cout << "bulundu: " << *p << '\n';
    }

    const int* q{ilkNegatifiBul(b, 2)};
    if (q == nullptr) {
        std::cout << "negatif yok\n";
    }
    return 0;
}
```

```
ÇIKTI
bulundu: -2
negatif yok
```

Fonksiyon bir referans döndürseydi "bulunamadı" diyemezdi; geçerli bir nesne döndürmek zorunda kalırdı. Bu nedenle `nullptr`, işaretçinin referansa göre gerçek üstünlüklerinden biridir. Kurallar Bölüm 12'dekilerle aynıdır: `nullptr`'ı dolaylayarak (`*q`) okumak tanımsız davranıştır, işaretçiyi kullanmadan önce `if (p)` ya da `p != nullptr` ile sınamalıyız. Başlatılmamış işaretçi ise `nullptr` bile değildir; tanımlarken ya geçerli bir adres ya da `nullptr` vermek en güvenli alışkanlıktır.

> **Not · std::optional**
>
> Modern C++ "değer olabilir ya da olmayabilir" durumu için `std::optional<T>` sınıfını da sunar (C++17). İşaretçi, nesnenin kendisini taşımadan yalnızca onu gösterir; `optional` ise değerin kendisini taşır. Bu kitapta işaretçi yaklaşımıyla yetineceğiz.

28.5 const Doğruluğu

Bölüm 12'de `const int*` ile `int* const` arasındaki farkı gördük: `const` yıldızın solundaysa gösterilen değer, sağındaysa işaretçinin kendisi sabittir. Aynı kurallar C++'ta geçerlidir ve referansa da uzanır. C++ programcılarının "const doğruluğu" (const correctness) dediği ilke şudur: **bir şeyi değiştirmeyeceksek bunu tipte belirtiriz ve derleyiciye denetletiriz.**

Referansın kendisini sabitlemek anlamsızdır, çünkü referans zaten yeniden bağlanamaz. Bu yüzden C++'ta `int& const r` diye bir biçim yoktur; yalnızca `const int& r` yazarız. Bunun anlamı, "bu referans üzerinden değer değiştirilemez" şeklindedir.

▸ constDogru.cpp

```
#include <iostream>
#include <string>

int topla(const int* dizi, int adet) {
    int t{0};
    for (int i = 0; i < adet; i++) {
        t += dizi[i];
    }
    return t;
}

void buyukHarfYap(std::string& s) {
    for (char& c : s) {
```

```
        if (c >= 'a' && c <= 'z') {
            c = static_cast<char>(c - 'a' + 'A');
        }
    }
}

void yazdir(const std::string& s) {
    std::cout << '[' << s << "]\n";
}

int main() {
    int a[3] = {4, 5, 6};
    int* p{a};
    const int* cp{p};
    std::cout << topla(p, 3) << ' ' << topla(cp, 3) << '\n';

    std::string ad{"selin"};
    buyukHarfYap(ad);
    yazdir(ad);
    yazdir("geçici metin");
    return 0;
}
```

```
ÇIKTI
15 15
[SELIN]
[geçici metin]
```

Bu programdan çıkan kurallar:

- `topla`, diziyi yalnızca okuyacağı için `const int*` alır. Hem `int*` hem `const int*` argüman olarak verilebilir: **sabit olmayandan sabite dönüşüm güvenlidir ve örtüktür.**
- `buyukHarfYap` metni değiştireceği için `std::string&` alır. Çağrıda bunun bir değişken olması gerekir.
- `yazdir`, `const std::string&` aldığı için hem değişkenle hem de geçici bir metinle çağrılabilir.

Ters yön ise yasaktır: sabit bir şeyi sabit olmayan bir işaretçi ya da referansa vermek, "değiştirmeyeceğim" sözünü bozmak olurdu. İki örneğe bakalım:

▸ constIsaretciHata.cpp

```
int main() {
    const int sabit{5};
    int* p = &sabit;
    return *p;
}
```

```
DERLEYİCİ ÇIKTISI

constIsaretciHata.cpp: In function 'int main()':
constIsaretciHata.cpp:3:14: error: invalid conversion from 'const int*' to 'int*' [-fpermissive]
    3 |     int* p = &sabit;
      |              ^~~~~~
      |              |
      |              const int*
```

▸ constRefHata.cpp

```
#include <string>

void buyut(std::string& s) {
```

```
    s += "!";
}

int main() {
    std::string ad{"Can"};
    const std::string& r{ad};
    buyut(r);
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

constRefHata.cpp: In function 'int main()':
constRefHata.cpp:10:11: error: binding reference of type 'std::string&' {aka
'std::__cxx11::basic_string<char>&'} to 'const std::string' {aka 'const
std::__cxx11::basic_string<char>'} discards qualifiers
   10 |     buyut(r);
      |           ^
constRefHata.cpp:3:25: note:   initializing argument 1 of 'void buyut(std::string&)'
    3 | void buyut(std::string& s) {
      |            ~~~~~~~~~~~~~^
```

Her iki mesaj da aynı şeyi söyler: bir `const` bilgisi atılmak (discards qualifiers) isteniyor. `const std::string&` ile alınan bir metni değiştiren başka bir fonksiyona vermek, zincirleme biçimde bütün çağrıların da `const` olmasını gerektirir. Bu yayılma "const bulaşıcıdır" diye bilinir ve istenen bir özelliktir: bir yerde `const` söz verirsek bütün yol boyunca derleyici sözün tutulduğunu denetler.

> ◆ **Kritik Nokta · const doğruluğunun üç kuralı**
>
> 1. Fonksiyon parametresini yalnızca okuyacaksak `const` yaparız: `const T&` (büyük nesne) ya da `const T*` (dizi, "yok" olabilen nesne).
> 2. Değiştirmeyeceğimiz değişkeni `const` tanımlarız; derleyici kazara yapılan değişikliği yakalar.
> 3. `const_cast` ile `const`'ı kaldırmak son çaredir (Bölüm 22). Gerçekten sabit tanımlanmış bir nesneyi bu yolla değiştirmek tanımsız davranıştır.

Sınıflarda aynı ilke üye fonksiyonlara uzanır: nesneyi değiştirmeyen üye fonksiyonlar `const` işaretlenir ve `const` nesne üzerinde çağrılabilir. Bunu Bölüm 30'da göreceğiz.

28.6 Referans Döndürmek

Bir fonksiyon değer yerine referans da döndürebilir. O zaman çağrı ifadesi bir kopya değil, bir nesnenin ta kendisidir ve atamanın sol tarafına bile yazılabilir:

▸ refDondur.cpp

```
#include <iostream>
#include <vector>

int& buyugu(int& a, int& b) {
    return (a > b) ? a : b;
}

int main() {
    int x{3};
    int y{8};
    buyugu(x, y) = 0;
```

```
    buyugu(x, y) += 10;
    std::cout << "x = " << x << ", y = " << y << '\n';

    std::vector<int> v{5, 6, 7};
    v[1] = 60;
    v.back() += 1;
    std::cout << v[0] << ' ' << v[1] << ' ' << v[2] << '\n';
    return 0;
}
```

```
ÇIKTI
x = 13, y = 0
5 60 8
```

İlk çağrıda `y` büyüktür; `buyugu(x, y)` ifadesi `y`'nin kendisidir ve `= 0` onu sıfırlar. İkinci çağrıda `x` (3) artık `y`'den (0) büyüktür, dolayısıyla `+= 10` `x`'e uygulanır. Programın ikinci yarısı bu özelliğin standart kütüphanede nasıl kullanıldığını gösterir: `v[1]` ve `v.back()` birer referans döndürür, bu yüzden `v[1] = 60;` yazabiliriz. Bölüm 32'de operatör aşırı yüklemeyi işlerken `[]` operatörünü de kendimiz referans döndürecek biçimde yazacağız.

Bir koşul vardır: döndürülen referans, fonksiyon bittikten sonra da yaşayan bir nesneye bağlı olmalıdır. Parametre olarak gelen nesneler, `static` değişkenler ve sınıfın üyeleri bu koşulu sağlar. Yerel değişken sağlamaz.

Sarkan referans

Yerel değişken fonksiyon bitince yok olur (Bölüm 9). Onun referansını döndürürsek çağıranın elinde var olmayan bir nesneye bağlı bir referans kalır. Buna **sarkan referans** (dangling reference) denir; Bölüm 12'deki sarkan işaretçinin eşdeğeridir.

```
int& yanlis() {
    int sayi{42};
    return sayi;           // HATA: sayi fonksiyonla birlikte yok olur
}

int main() {
    int& r{yanlis()};      // r yok olmuş bir nesneyi gösterir
    std::cout << r << '\n';    // tanımsız davranış
}
```

```
TERMİNAL
$ g++ -std=c++17 -Wall -Wextra sarkanRef.cpp -o sarkanRef
sarkanRef.cpp: In function 'int& yanlis()':
sarkanRef.cpp:5:12: warning: reference to local variable 'sayi' returned [-Wreturn-local-addr]
    5 |     return sayi;
      |            ^~~~
sarkanRef.cpp:4:9: note: declared here
    4 |     int sayi{42};
      |         ^~~~
```

Bu program derlenir (yalnızca uyarı verir) ve çalıştırıldığında denediğimiz sistemde bellek erişim hatasıyla çöktü. Başka bir derleyicide ya da başka bir ayarda 42 de yazabilir, çöp de. Bu yüzden çıktısını göstermiyoruz. Uyarıyı hiçbir zaman yok saymayalım.

Aynı tuzağın daha sinsi bir biçimi, geçici değeri `const&` ile döndürmektir:

```
const std::string& ad() {
    return std::string("Elif");    // HATA: geçici nesne ifade sonunda yok olur
}
```

Ömür uzatma kuralı yalnızca **yerel** bir `const&` değişkene bağlanan geçici için geçerlidir (`gecici.cpp` programındaki gibi). Bir `return` ifadesinde bu kural işlemez: geçici nesne fonksiyonla birlikte yok olur ve dönen referans sarkar.

> **▲ Sık Yapılan Hata · Referans döndürmeden önce üç soru**
>
> 1. Döndürdüğümüz nesne fonksiyon bittikten sonra yaşıyor mu? (Parametre, üye veya `static` olabilir; yerel olamaz.)
> 2. Çağıranın bu nesneyi değiştirmesini istiyor muyuz? İstemiyorsak `const T&` döndürürüz.
> 3. Hesapladığımız yeni bir değeri mi döndürüyoruz? O zaman referans değil, değer döndürürüz (`int topla(int, int)`).

28.7 this İşaretçisi: Küçük Bir Önizleme

Sınıfları Bölüm 30'da ayrıntılı işleyeceğiz; ancak referans döndürmenin en yaygın kullanımı sınıfların içinde ortaya çıkar, bu yüzden küçük bir önizleme yapalım. Sınıfın üye fonksiyonu bir nesne üzerinde çağrıldığında, derleyici fonksiyona gizli bir parametre geçirir: çağrıldığı nesnenin adresi. Bu işaretçinin adı `this`'tir ve fonksiyon içinde doğrudan kullanılabilir.

▸ thisOn.cpp

```
#include <iostream>

struct Sayac {
    int deger{0};

    Sayac& artir() {
        ++deger;
        return *this;
    }
    Sayac& ekle(int n) {
        deger += n;
        return *this;
    }
    bool benMi(const Sayac* baska) const {
        return this == baska;
    }
};

int main() {
    Sayac s;
    s.artir().artir().ekle(10).artir();
    std::cout << s.deger << '\n';

    Sayac t;
    std::cout << std::boolalpha << s.benMi(&s) << ' ' << s.benMi(&t) << '\n';
    return 0;
}
```

```
ÇIKTI
13
true false
```

`struct Sayac { ... }` içinde tanımlanan `artir` ve `ekle`, üye fonksiyonlardır. `this` burada `Sayac*` türünde bir işaretçidir; `*this` çağrıldığı nesnenin kendisidir. Fonksiyonlar `*this`'i referans olarak döndürdüğü için çağrıları zincirleyebildik: `s.artir()` yine `s`'nin kendisini verir, ona `.artir()` ve `.ekle(10)` uygulanabilir. Bu desen, akış operatörlerinin zincirleme kullanımının (`cout << a << b`) temelindedir: `operator<<` `cout`'a bir referans döndürür.

`benMi` fonksiyonunun sonundaki `const`, o fonksiyonun nesneyi değiştirmeyeceğini belirtir; `this` bu durumda `const Sayac*` olur. Üye fonksiyonların ayrıntıları ve `this`'in derleyici tarafından nasıl kullanıldığı Bölüm 30'a kalıyor.

28.8 lvalue ve rvalue: Sezgisel Giriş

Referansın hata mesajlarında "lvalue" ve "rvalue" sözcüklerini gördük. Bu ayrımın tam tanımı karmaşıktır; burada işimize yarayan sezgiyi kuralım. Ayrıntılı kullanımı Bölüm 39'daki taşıma semantiğinde işleyeceğiz.

> **Tanım: lvalue ve rvalue**
>
> **lvalue**, adı ve bellekte kalıcı bir yeri olan, adresi alınabilen ifadedir (değişkenler, dizi elemanları, `*p`, referans döndüren fonksiyon çağrıları). **rvalue**, adı olmayan, ifadenin sonunda yok olan geçici bir değerdir (`5`, `x + 1`, değer döndüren fonksiyon çağrıları).

Pratik bir sınama: bir ifadenin başına `&` koyabiliyorsak lvalue'dur. `&x` geçerlidir, `&(x + 1)` ve `&5` geçerli değildir. Atamanın sol tarafında da genellikle lvalue bulunur; "l" ve "r" harfleri "left" (sol) ve "right" (sağ) sözcüklerinden gelir.

Daha önce gördüğümüz `T&` yalnızca lvalue'ya bağlanır. C++11 bir de **rvalue referansı** getirdi: `T&&`. Bu tür yalnızca rvalue'ya bağlanır. İkisini aşırı yüklemeyle ayırt eden küçük bir program yazalım (aşırı yükleme Bölüm 25'in konusudur; şimdilik derleyicinin argümanın türüne göre doğru fonksiyonu seçtiğini bilmemiz yeter):

▸ lvalue.cpp

```
#include <iostream>

void bak(int&) {
    std::cout << "lvalue referansı\n";
}

void bak(int&&) {
    std::cout << "rvalue referansı\n";
}

int iki() {
    return 2;
}

int main() {
    int x{10};
    bak(x);
    bak(x + 1);
    bak(iki());
    bak(5);
```

```
    int&& gecici{x * 2};
    gecici += 1;
    std::cout << gecici << '\n';
    bak(gecici);
    return 0;
}
```

```
ÇIKTI
lvalue referansı
rvalue referansı
rvalue referansı
rvalue referansı
21
lvalue referansı
```

Çıktının önemli noktaları şunlardır:

- `x` bir değişken olduğu için lvalue'dur; `x + 1`, `iki()` ve `5` geçici değerlerdir, rvalue'dur.
- `int&& gecici{x * 2};` geçici sonuca bir ad verir ve ömrünü uzatır. `gecici += 1;` ile değiştirebildiğimize dikkat edelim.
- Son satır şaşırtıcıdır: `gecici` türü `int&&` olduğu hâlde `bak(gecici)` lvalue sürümünü çağırdı. Çünkü `gecici` artık **adı olan** bir ifadedir ve adresi alınabilir. Tür ile ifade kategorisi ayrı kavramlardır: bir değişkenin türü rvalue referansı olabilir, ama değişkenin adı bir ifade olarak her zaman lvalue'dur.

İki yanlış bağlama denemesinin hata mesajlarına da bakalım:

▸ lvalueHata.cpp

```
int main() {
    int x{10};
    int& a{x + 1};
    int&& b{x};
    return a + b;
}
```

```
DERLEYİCİ ÇIKTISI

lvalueHata.cpp: In function 'int main()':
lvalueHata.cpp:3:17: error: cannot bind non-const lvalue reference of type 'int&' to an rvalue
of type 'int'
    3 |     int& a{x + 1};
      |                 ^
lvalueHata.cpp:4:14: error: cannot bind rvalue reference of type 'int&&' to lvalue of type 'int'
    4 |     int&& b{x};
      |              ^
```

Bağlanacak ifade	`T&`	`const T&`	`T&&`
lvalue (değişken)	Evet	Evet	Hayır
`const` lvalue	Hayır	Evet	Hayır
rvalue (geçici)	Hayır	Evet	Evet

Bu tablo neden önemlidir? Bir fonksiyon `T&&` ile aldığı geçici nesnenin artık kimse tarafından kullanılmayacağını bilir; içindeki kaynakları (örneğin bir vektörün dinamik belleğini) kopyalamak yerine

"çalabilir". Bu fikre **taşıma** (move) denir ve Bölüm 39'da işlenecektir. Şu aşamada `T&&` yazmamız gerekmez; yalnızca hata mesajlarında rvalue ve lvalue sözcüklerini gördüğümüzde ne dediklerini anlamamız yeterlidir.

28.9 Üç Dilde Referanslar

Üç Dilde · C, C++ ve C#'ta "kendisine ulaşmak" *C · C++ · C#*

C: Yalnızca işaretçi vardır. Fonksiyonun çağıranın değişkenini değiştirmesi için adres geçirilir; çağrı yerinde `&` yazılır.

```
void artir(int *n) { (*n)++; }
// artir(&x);
```

C++: Hem işaretçi hem referans vardır. Çağrı yerinde hiçbir işaret gerekmez; sabit olmayan referans parametre değişkeni değiştirir.

```
void artir(int& n) { n++; }
// artir(x);
```

C#: Referans tipleri (`class`) değişken atamasında zaten nesneye başvuruyu kopyalar. Değer tiplerini (`int` , `struct`) referansla geçirmek için `ref` , `out` , `in` anahtar sözcükleri kullanılır; hem tanımda hem çağrıda yazılır.

```
static void Artir(ref int n) { n++; }
// Artir(ref x);
```

C#'ta işaretçi yoktur (yalnızca `unsafe` kod bloklarında bulunur). `class` ile tanımlanan her tür **referans tipidir**: bir değişken nesnenin kendisini değil, ona bir başvuruyu tutar ve atama başvuruyu kopyalar. `struct` ise **değer tipidir**: atama nesnenin kopyasını alır. C++'ta bu ayrım türde değil, kullanıma bağlıdır: aynı `struct` hem değer olarak hem referans olarak taşınabilir. C#'ın `ref` anahtar sözcüğü C++ referansına yakın çalışır. Bir yerel değişken (`ref int`) ve bir dönüş değeri (`ref int`) da referans olabilir. Aşağıdaki program bu davranışları yan yana gösterir:

▸ ReferansTipleri.cs

```
var a = new Nokta { X = 1 };
var b = a;              // aynı nesneye ikinci başvuru
b.X = 99;
Console.WriteLine(a.X);

var s = new Sayi { Deger = 1 };
var t = s;              // kopya
t.Deger = 99;
Console.WriteLine(s.Deger);

int x = 5;
Araclar.Artir(ref x);
Console.WriteLine(x);

int[] d = { 4, 9, 2 };
ref int enBuyuk = ref Araclar.Buyuk(d);
enBuyuk = 0;
Console.WriteLine(string.Join(" ", d));

string? bos = null;
Console.WriteLine(bos is null);
```

```
class Nokta
{
    public int X;
}

struct Sayi
{
    public int Deger;
}

static class Araclar
{
    public static ref int Buyuk(int[] dizi)
    {
        int enb = 0;
        for (int i = 1; i < dizi.Length; i++)
        {
            if (dizi[i] > dizi[enb]) enb = i;
        }
        return ref dizi[enb];
    }

    public static void Artir(ref int n) => n++;
}
```

```
ÇIKTI
99
1
6
4 0 2
True
```

İlk iki çıktı sınıf ile yapı arasındaki farkı gösterir: `class` nesnesinde `b.X = 99` `a`'yı da değiştirir, `struct`'ta `t.Deger = 99` `s`'ye dokunmaz. `Buyuk` metodu C++'taki `buyugu` gibi bir referans döndürür; `ref int enBuyuk = ref ...` ile diziye bağlanan yerel bir referans tanımlayıp elemanı doğrudan sıfırladık. Son iki satır C++'taki `nullptr`'ın karşılığını gösterir: referans tipindeki bir değişken `null` olabilir ve `is null` ile sınanır. C#'ta çöp toplayıcı (GC) bellek yönetimini üstlendiği için sarkan referans sorunu yaşanmaz; `ref` yerel değişkenler ve dönüşler bile derleyici tarafından, yerel bir değişkenin referansı kaçırılamayacak biçimde denetlenir. `ref`, `out` ve `in` parametrelerini Bölüm 46'da, sınıfları ve referans tiplerini Bölüm 49'da ayrıntılı işleyeceğiz.

28.10 Sık Yapılan Hatalar

Bölümü kapatmadan önce başlıca tuzakları tek yerde toplayalım:

Hata	Sonuç	Çözüm
`int& r;`	Derleme hatası	Tanımda nesneye bağla
`r = y;` ile referansı yeniden bağladığını sanmak	`y`'nin değeri bağlı nesneye kopyalanır	Başka nesneye yönelmesi gerekiyorsa işaretçi kullan
Yerel değişkenin referansını/adresini döndürmek	Sarkan referans/işaretçi, tanımsız davranış	Değer, parametre, `static` veya üye döndür
`int* p, q;`	Yalnızca `p` işaretçidir	Her işaretçiyi ayrı satırda tanımla

Hata	Sonuç	Çözüm
`nullptr` işaretçiyi dolaylamak	Tanımsız davranış	Kullanmadan önce `if (p)` ile sına
`const` olmayan parametreye geçici değer vermek	Derleme hatası	Parametreyi `const T&` yap
`cout << charIsaretcisi` ile adres yazmak	Metin olarak yorumlanır	`static_cast<void*>` kullan
Sabit tanımlı nesneden `const_cast` ile yazmak	Tanımsız davranış	`const`'ı kaldırma

Bölüm Özeti

- İşaretçinin sözdizimi ve aritmetiği C'dekiyle aynıdır (Bölüm 12). C++ farklı olarak `nullptr` kullanır, `void*`'ten örtük dönüşüme izin vermez ve `const char*` olmayan metin işaretçisini hataya yakın sayar.
- `cout`, `char*` türündeki işaretçiyi metin olarak yazar; adres görmek için `static_cast<void*>` gerekir.
- Referans, bir nesnenin ikinci adıdır. Tanımda ilk değer zorunludur, sonradan başka nesneye bağlanamaz ve boş olamaz.
- Referansa atama (`r = y;`) bağlama değil, bağlı nesneye değer kopyalamadır.
- `const` referans geçici değere bağlanabilir ve geçicinin ömrünü referansın ömrüne kadar uzatır; bu yüzden büyük nesneleri okumak için `const T&` kullanırız.
- Karar kuralı: nesne her zaman varsa referans, "yok" olabiliyorsa ya da yeniden yönelmesi gerekiyorsa işaretçi. İşaretçiyi değiştirecek fonksiyon `T**` yerine `T*&` alabilir.
- `nullptr`, referansın veremediği "değer yok" bilgisini taşır; kullanmadan önce sınanmalıdır.
- `const` doğruluğu: değiştirmeyeceğimizi `const T&`, `const T*` ile belirtiriz. Sabit olmayandan sabite dönüşüm örtüktür, tersi yasaktır.
- Referans döndüren fonksiyon, fonksiyon bittikten sonra da yaşayan bir nesneyi göstermelidir. Yerel değişkenin referansı sarkar.
- `this`, üye fonksiyona gizlice geçen nesne adresidir; `return *this;` zincirleme çağrıya olanak tanır.
- lvalue adı ve adresi olan ifadedir, rvalue geçici değerdir. `T&` lvalue'ya, `T&&` rvalue'ya, `const T&` ikisine de bağlanır. Taşıma semantiği Bölüm 39'da işlenecektir.
- C#'ta işaretçi yerine referans tipleri ve `ref` / `out` / `in` vardır; sarkan referans, çöp toplayıcı sayesinde oluşmaz.

Alıştırmalar

1. `int x{5}, y{9}; int& r{x}; r = y;` satırlarından sonra aşağıdakilerden hangisi doğrudur?

   A) `r`, artık `y`'nin ikinci adıdır; `x` 5 kalır.
   B) `r`, hâlâ `x`'in ikinci adıdır ve `x` artık 9'dur.
   C) `r = y;` satırı derleme hatasıdır.

D) `r` kullanılmadan önce `nullptr` kontrolü gerekir.
E) `sizeof(r)` her zaman 8'dir.

2. Aşağıdakilerden hangisi referansın işaretçiden bir üstünlüğü değil, bir **kısıtıdır**?

A) Çağrı yerinde `&` yazmak gerekmez.
B) Tanımlanırken ilk değer vermek zorunludur.
C) Fonksiyon içinde `*` kullanmadan kullanılır.
D) Dolaylama sırasında `nullptr` kontrolü gerekmez.
E) Büyük nesneyi kopyalamadan geçirmeye olanak verir.

3. `int a{1}, b{2}; const int* p{&a}; int* const q{&a}; const int& r{a};` tanımlarından sonra aşağıdaki satırlardan hangisi derlenmez?

A) `p = &b;`
B) `*q = 5;`
C) `a = 7;`
D) `r = 3;`
E) `std::cout << r;`

4. Aşağıdaki fonksiyonlardan hangisi sarkan referans üretir?

A) `int& f(int& a) { return a; }`
B) `int& f() { static int s{0}; return s; }`
C) `int& f() { int s{0}; return s; }`
D) `int f() { int s{0}; return s; }`
E) `const int& f(const int& a) { return a; }` (çağıranın değişkeniyle çağrılırsa)

5. Bir `std::vector<int>` yalnızca okunacak ve her çağrıda kopyalanması istenmiyorsa parametre nasıl yazılmalıdır?

A) `void f(std::vector<int> v)`
B) `void f(std::vector<int>& v)`
C) `void f(const std::vector<int>& v)`
D) `void f(std::vector<int>* v)`
E) `void f(std::vector<int>&& v)`

6. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>

int main() {
    int a{1}, b{2};
    int& r{a};
    int* p{&b};
    r = *p + 3;
    p = &a;
    *p += 10;
    r = b;
    std::cout << a << " " << b << " " << r << " " << *p << '\n';
    return 0;
}
```

7. Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

int& ikinci(int* d) {
    return d[1];
}

int main() {
    int a[3] = {10, 20, 30};
    ikinci(a) = 99;
    ikinci(a) += 1;
    int& r{ikinci(a)};
    r = r / 2;
    std::cout << a[0] << " " << a[1] << " " << a[2] << '\n';
    return 0;
}
```

8. Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

void yaz(int&) { std::cout << "L "; }
void yaz(int&&) { std::cout << "R "; }

int main() {
    int x{4};
    int& r{x};
    yaz(x);
    yaz(r);
    yaz(x * 2);
    yaz(3);
    yaz(static_cast<int&&>(x));
    std::cout << '\n';
    return 0;
}
```

9. Aşağıdaki kodda hata bulun ve düzeltin.

```cpp
#include <iostream>
#include <string>

std::string& selamla(const std::string& ad) {
    std::string mesaj{"Merhaba " + ad};
    return mesaj;
}

int main() {
    std::cout << selamla("Mert") << '\n';
    return 0;
}
```

10. Aşağıdaki kodda hata bulun ve düzeltin.

```cpp
#include <string>

void ekle(std::string& s) {
    s += "!";
}

int main() {
```

```
    const std::string ad{"Ece"};
    ekle(ad);
    ekle("Deniz");
    return 0;
}
```

11. Aşağıdaki ifadelerin doğru mu yanlış mı olduğunu belirtin ve gerekçelendirin: (a) `int& r;` tanımı derlenir. (b) `const int& r{5};` tanımı derlenir. (c) `int* p{nullptr}; if (p) { ... }` bloğu çalışmaz. (d) `int* p, q;` ile `q` da işaretçidir.
12. "Bir fonksiyon neden `const std::string&` alır, `std::string` almaz?" sorusunu iki gerekçeyle yanıtlayın. Ardından `int` için aynı gerekçelerin neden geçerli olmadığını açıklayın.
13. Verilen `int` dizisinde değeri sıfır olan ilk elemanın adresini döndüren `int* ilkSifir(int* d, int adet)` fonksiyonunu yazın; bulunamazsa `nullptr` döndürsün. `main`'de dizinin bu elemanını 1 yapın ve dizi içeriğini yazdırın. Eleman bulunamazsa "sıfır yok" yazdırın.
14. İki `double`'ın büyüğünü referansla döndüren `double& enBuyuk(double& a, double& b)` fonksiyonunu yazın. `main`'de iki değişken tanımlayıp büyük olanı 0 yapın, ardından iki değişkeni yazdırın.
15. Bir `int` dizisini ve eleman sayısını alan, tüm elemanları yerinde ikiyle çarpan `void ikiyleCarp(int* d, int adet)` fonksiyonunu yazın. Sonra aynı işi yapan, parametresi `std::vector<int>&` olan bir sürüm yazın ve aralık tabanlı `for` ile `int&` kullanın. İki sürümü de `main`'de deneyin.
16. `Sayac` sınıfına (bölümdeki `thisOn.cpp` programı) çıkarma yapan `Sayac& azalt()` ve sıfırlayan `Sayac& sifirla()` üye fonksiyonlarını ekleyin. `s.artir().artir().azalt().ekle(5)` zinciriyle sonucu yazdırın.

BÖLÜM 29

Dinamik Bellek, RAII ve Akıllı İşaretçiler

BU BÖLÜMDE

- `new` ve `delete` ile `new[]` ve `delete[]` çiftlerini; `malloc` / `free`'den farkını ve `new`'ün yer bulamadığında ne yaptığını
- `new`'ün nesneyi kurarken yapıcıyı, `delete`'in yok ederken yıkıcıyı çağırdığını
- Elle bellek yönetiminin neden zor olduğunu; istisna fırlatıldığında bile sızıntının nasıl oluştuğunu
- RAII ilkesini: kaynağın ömrünü bir nesnenin ömrüne bağlamayı
- `std::unique_ptr` ile tek sahiplikli yönetimi, `std::make_unique`'i ve sahipliğin `std::move` ile devrini
- `std::shared_ptr` ile referans sayımını, `std::weak_ptr` ile döngüsel referansın nasıl kırıldığını
- Hangi durumda hangi aracı seçeceğimizi ve modern C++'ta neden "çıplak `new`" yazmadığımızı

Bölüm 13'te C'nin dinamik bellek araçlarını (`malloc`, `calloc`, `realloc`, `free`) öğrenmiş ve bunlarla ne kadar kolay hata yapılabildiğini görmüştük: unutulan `free`, iki kez yapılan `free`, `free`'den sonra kullanılan işaretçi. C++ bu araçları ortadan kaldırmaz; ama bellek yönetiminin büyük bölümünü dilin kendisine, yani nesnelerin yaşam döngüsüne devretmeyi mümkün kılar.

Bu bölümün ana fikri şudur: belleği elle geri vermek zorunda kalmamak. Önce C++'ın `new` ve `delete` işleçlerini ve C'den farklarını göreceğiz. Sonra elle yönetimin neden kırılgan olduğunu bir istisna örneğiyle göstereceğiz ve çözümü, yani RAII ilkesini ve onu hazır sunan akıllı işaretçileri (smart pointer) işleyeceğiz. Bölüm 10 ve 13'teki bellek bölgeleri anlatımını (yığın, öbek, statik alan) tekrarlamayacağız; yalnızca farkları vurgulayacağız.

29.1 new ve delete

C'de öbekten (heap) bellek isteyen araç bir işlevdir: `malloc(bayt)`. C++'ta bunun yerine bir **işleç** (operator) olan `new` vardır. Tipi bilir, bayt hesabı yapmaz, dönüş değerini dönüştürmeyi gerektirmez.

```
int* p = new int;          // öbekte bir int; içinde çöp değer var
int* q = new int(7);       // öbekte bir int, ilk değeri 7
int* r = new int{7};       // aynısı, süslü parantezle
delete p;                  // bloğu öbeğe geri ver
```

`new`, öbekte bir nesne için yer ayırır, nesneyi kurar ve **adresini** döndürür; adresi bir işaretçide saklarız. İşimiz bitince `delete` ile bloğu geri veririz. Birlikte çalışan bir program yazalım:

newTemel.cpp

```cpp
#include <iostream>

int main() {
    int yigindaki = 10;                // yığında
    int* obekteki = new int{20};       // öbekte; işaretçinin kendisi yığında

    std::cout << "yigindaki: " << yigindaki << '\n';
    std::cout << "obekteki : " << *obekteki << '\n';
    *obekteki += 5;                    // dolaylama ile değeri değiştir
    std::cout << "obekteki : " << *obekteki << '\n';

    delete obekteki;                   // bloğu geri ver
    obekteki = nullptr;                // sarkan işaretçiyi temizle
    if (obekteki == nullptr) {
        std::cout << "obekteki artik bos\n";
    }
    return 0;
}
```

ÇIKTI

```
yigindaki: 10
obekteki : 20
obekteki : 25
obekteki artik bos
```

`delete`, işaretçinin içindeki adresi değiştirmez; yalnızca bloğu geri verir. Bu yüzden `delete`'ten sonra işaretçiyi `nullptr` yapmak iyi bir alışkanlıktır (Bölüm 28'de `nullptr`'ı ayrıntısıyla gördük). `nullptr` üzerinde `delete` çağırmak güvenlidir ve hiçbir şey yapmaz.

Başlatma biçimleri

`new` ile alınan nesnenin ilk değeri, yazılış biçimine göre belirlenir:

ilkDeger.cpp

```cpp
#include <iostream>
#include <string>

int main() {
    int* a = new int{};            // boş süslü parantez: 0
    int* b = new int(7);           // 7
    int* c = new int{9};           // 9
    std::string* s = new std::string("merhaba");

    std::cout << *a << ' ' << *b << ' ' << *c << '\n';
    std::cout << *s << " (" << s->size() << " karakter)\n";

    delete a;
    delete b;
    delete c;
    delete s;
    return 0;
}
```

ÇIKTI

```
0 7 9
merhaba (7 karakter)
```

Burada `s->size()`, `(*s).size()` ile aynıdır: işaretçiden üyeye ulaşmanın kısa yazılışıdır. `new int;` biçimi ise (parantezsiz) ilk değer vermez; temel tiplerde çöp değer kalır ve Bölüm 5'te gördüğümüz gibi okumak tanımsız davranıştır. `new int{}` yazmak bu riski kaldırır.

new ve malloc arasındaki farklar

	`malloc` / `free` (C)	`new` / `delete` (C++)
Ne olduğu	Kütüphane işlevi	Dil işleci
Boyut	Bayt cinsinden elle hesaplanır	Tipten otomatik
Dönüş tipi	`void*` (C++'ta dönüştürme gerekir)	Doğru tipte işaretçi
Yapıcı/yıkıcı	Çağırmaz; ham bayt verir	Çağırır
Yer bulamazsa	`NULL` döndürür	`std::bad_alloc` istisnası fırlatır
Büyütme	`realloc`	Yok

> ◆ **Kritik Nokta · Eşleşme kuralı**
>
> `new` ile alınan bloğu `delete` ile, `new[]` ile alınanı `delete[]` ile, `malloc` ile alınanı `free` ile bırakırız. Aileleri karıştırmak (örneğin `new` ile alıp `free` ile bırakmak) tanımsız davranıştır. C++ kodunda nesne için `malloc` kullanmayız: nesne kurulmamış ham baytlar üzerinde işlem yapmak yapıcıyı atlamak demektir.

Yer bulunamazsa

C'de `malloc` sonucu her seferinde `NULL` ile karşılaştırmak zorundaydık. C++'ta `new` yer bulamazsa `nullptr` döndürmez; `std::bad_alloc` türünde bir **istisna** (exception) fırlatır. İstisnaları Bölüm 34'te ayrıntılı işleyeceğiz; şimdilik `try` / `catch` kalıbını kalıp olarak alalım:

▸ yerBulunamadi.cpp

```cpp
#include <iostream>
#include <new>

int main() {
    try {
        // 2^50 tane int: hiçbir bilgisayarda yok
        int* dev = new int[1ULL << 50];
        std::cout << "yer ayrildi\n";
        delete[] dev;
    } catch (const std::bad_alloc& e) {
        std::cout << "bellek ayrilamadi: " << e.what() << '\n';
    }
    return 0;
}
```

```
ÇIKTI
bellek ayrilamadi: std::bad_alloc
```

Bu yüzden `new`'dan sonra `if (p == nullptr)` denetimi gereksizdir; istisna yakalanmazsa program kapanır. İstisna istemeyen kodda `new (std::nothrow) int[n]` biçimi vardır ve yer bulunamazsa `nullptr` döndürür, ama modern kodda nadiren kullanılır. (Yazdığımız `what()` metni derleyiciye göre değişebilir; 32 bitlik sistemlerde bu örnek başka bir `bad_alloc` türevi fırlatabilir.)

29.2 new[] ve delete[]: Dinamik Dizi

Birden çok elemanı art arda ayırmak için `new Tip[adet]` yazılır. Eleman sayısı bir değişken olabilir ve asıl işe yaradığı yer burasıdır:

```
Tip* p = new Tip[adet];    // adet elemanlık dizi
delete[] p;                // DİZİ için delete[] (köşeli parantez şart)
```

Aşağıdaki program önce kaç not gireceğini sorar, sonra tam o kadar yer ayırır:

▸ dinamikOrtalama.cpp

```
#include <iostream>

int main() {
    int n = 0;
    std::cout << "Kac not girilecek? ";
    if (!(std::cin >> n) || n <= 0) {
        std::cout << "Gecersiz sayi.\n";
        return 1;
    }

    int* notlar = new int[n]{};        // n elemanlık dizi, hepsi 0
    std::cout << "Notlari gir: ";
    int toplam = 0;
    for (int i = 0; i < n; i++) {
        std::cin >> notlar[i];         // sıradan dizi gibi kullan
        toplam += notlar[i];
    }
    std::cout << "Ortalama: " << static_cast<double>(toplam) / n << '\n';

    delete[] notlar;                   // dizi: köşeli parantezli delete
    return 0;
}
```

ÇIKTI

```
Kac not girilecek? 5
Notlari gir: 72 85 90 60 95
Ortalama: 80.4
```

Dikkat edilecek noktalar:

- `n` önce denetlenir, sonra `new` çağrılır.
- `new int[n]{}` sondaki boş süslü parantezle bütün elemanları 0 yapar. Yazmazsak elemanlar çöp değerli kalır. `new int[4]{3, 1}` yazarsak ilk iki eleman 3 ve 1, kalanlar 0 olur.
- `notlar` bir diziyi değil bir işaretçiyi tutar; `sizeof(notlar)` eleman sayısını değil işaretçinin boyutunu verir. Eleman sayısını ayrıca (`n`) saklamak bize düşer.
- Dizi boyutu değişken olan `int a[n];` (VLA) C++ standardında yoktur. g++ bunu bir uzantı olarak kabul eder, ama taşınabilir değildir. Boyutu çalışma anında belli olan veri için `new[]` ya da, çok daha iyisi, `std::vector` kullanırız (Bölüm 26).

▲ Sık Yapılan Hata · new[] ile alıp delete ile bırakmak

```
int* p = new int[10];
delete p;              // YANLIŞ: dizi delete[] ile bırakılır
```

Köşeli parantezsiz `delete`, tek nesne bırakır. Diziye uygulandığında davranış tanımsızdır: program çökebilir, öbek bozulabilir ya da hiçbir şey olmamış gibi görünebilir. Tersi de hatadır: tek nesneyi `delete[]` ile bırakmak. Kural eşleşmedir: `new` ile `delete`, `new[]` ile `delete[]`.

29.3 Yapıcı ve Yıkıcı: new Ne Yapar?

`malloc` ile `new` arasındaki en önemli fark, `new`'ün **nesne kurmasıdır**. Sınıf tipinde bir nesne için `new` önce yer ayırır, sonra yapıcıyı (constructor) çağırır. `delete` ise önce yıkıcıyı (destructor) çağırır, sonra yeri geri verir. Yapıcı ve yıkıcı Bölüm 30'un konusudur; burada yalnızca davranışlarını gözlemleyeceğiz.

Tanım: Yapıcı ve yıkıcı

Yapıcı (constructor), sınıfla aynı adı taşıyan ve nesne oluşturulurken otomatik çalışan işlevdir. **Yıkıcı** (destructor), adı `~` ile başlayan ve nesne yok edilirken otomatik çalışan işlevdir. Yıkıcı, nesnenin arkasını toplamak (dosya kapatmak, bellek geri vermek, bağlantı kesmek) için kullanılır.

Kitabın geri kalan bölümlerinde sıkça kullanacağımız küçük bir sınıf yazalım. `Kaynak`, doğarken ve ölürken ekrana yazı yazar; böylece ne zaman temizlendiğini gözle görürüz. Şimdilik yapıyı bir kalıp olarak alalım: `Kaynak(int n)` nesne doğarken, `~Kaynak()` yok olurken çalışan bloktur.

kaynakSizinti.cpp

```
#include <iostream>

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; std::cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { std::cout << "Kaynak " << no << " yok edildi\n"; }
};

int main() {
    Kaynak* k1 = new Kaynak(1);
    Kaynak* k2 = new Kaynak(2);
    std::cout << "ikisi de kullanildi, k2->no = " << k2->no << '\n';
    delete k1;                    // k1 geri verildi
    std::cout << "main bitiyor\n";
    return 0;                     // k2 için delete YOK
}
```

ÇIKTI

```
Kaynak 1 olustu
Kaynak 2 olustu
ikisi de kullanildi, k2->no = 2
Kaynak 1 yok edildi
main bitiyor
```

`k1` için `delete` yazdık: "yok edildi" satırı çıktı. `k2` için yazmadık: satır hiç çıkmadı, yani yıkıcı hiç çalışmadı. Program bitince işletim sistemi öbeği topladığı için ekranda bozuk bir şey görmeyiz. Ancak `Kaynak` bir dosya

ya da ağ bağlantısı tutsaydı, yıkıcının çalışmaması bekleyen verilerin yazılmaması anlamına gelirdi. Günlerce çalışan bir sunucuda unutulan her `delete` ise belleği biraz biraz tüketir. Bu duruma **bellek sızıntısı** (memory leak) diyoruz; Bölüm 13'te C'deki karşılığını görmüştük.

> **Not · Dizi için de aynısı**
>
> `new Kaynak[3]` üç nesne için üç yapıcı çağrısı, `delete[]` ise üç yıkıcı çağrısı yapar. `malloc(3 * sizeof(Kaynak))` ise hiçbir yapıcı çalıştırmaz; `free` de hiçbir yıkıcı çalıştırmaz. Sınıf tipleriyle `malloc` kullanmamamızın asıl nedeni budur.

29.4 Elle Yönetimin Zorluğu

Bölüm 13'te C için sıraladığımız bellek hataları C++'ta da aynen geçerlidir. Kısaca hatırlayalım ve C++'a özgü eklerini görelim:

Hata	Örnek	C++'a özgü ek
Sızıntı	`delete` unutmak; adresi kaybetmek (`p = new int{2};`)	Yıkıcı da çalışmaz
Sarkan işaretçi	`delete p; *p = 1;`	Yıkılmış nesnenin üyelerine erişim
Çifte silme	İki işaretçi, aynı blok, iki `delete`	Yıkıcı iki kez çalışır
Yanlış eşleşme	`new[]` ile alıp `delete` ile bırakmak	Yıkıcılar eksik çalışır
Sınır aşımı	`new int[5]` için `p[5] = 1;`	Komşu bloğu bozar

Bunların hepsinin ortak noktası, geri verme sorumluluğunun programcıda olmasıdır. C'de bu sorumluluğu disiplinle taşırdık. C++'ta ise disiplin yetmez, çünkü başka bir sızıntı yolu daha vardır: **istisna**. Aşağıdaki kod görünürde doğru: `new` var, `delete` var.

istisnaSizinti.cpp

```cpp
#include <iostream>
#include <stdexcept>

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; std::cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { std::cout << "Kaynak " << no << " yok edildi\n"; }
};

void islem(int deger) {
    Kaynak* k = new Kaynak(1);
    if (deger < 0) {
        throw std::runtime_error("negatif deger");   // delete'e ulasilmaz
    }
    delete k;
}

int main() {
    try {
        islem(-5);
    } catch (const std::exception& e) {
        std::cout << "yakalandi: " << e.what() << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Kaynak 1 olustu
yakalandi: negatif deger
```

`throw` satırı çalışınca işlev oradan terk edilir; `delete k;` hiç yürütülmez ve "yok edildi" satırı görünmez. Aynı sorun, `delete`'ten önce yazılmış herhangi bir erken `return` için de geçerlidir. Her çıkış yoluna `delete` eklemek hem sıkıcı hem de hataya açıktır; kodu değiştirdikçe yeni çıkış yolları eklenir ve biri unutulur.

> **İpucu · Bellek hatalarını makineye buldurmak**
>
> Bu hatalar gözle bulunması zor hatalardır. g++ ile `g++ -std=c++17 -g -fsanitize=address program.cpp -o program` derlersek AddressSanitizer, `delete` sonrası kullanımı, sınır aşımını ve çifte silmeyi çalışma anında satır numarasıyla bildirir; Linux'ta sızıntıları da raporlar. Valgrind ve AddressSanitizer'ın ayrıntıları Bölüm 13'te anlatıldı.

Çare, geri vermeyi hatırlamamıza bırakmamaktır. Bunu yapan fikre geçelim.

29.5 RAII: Kaynağı Bir Nesneye Emanet Etmek

Bir `std::vector` ya da `std::string` kullanırken hiç `delete` yazmadık, ama bellek sızmadı. Bunun nedeni, bu nesnelerin yok edilirken kendi belleklerini kendilerinin geri vermesidir. C++'ın en önemli fikirlerinden biri buna dayanır.

Önce dilin temel garantisine bakalım: yerel bir nesne, içinde bulunduğu `{ }` bloğunun sonunda otomatik olarak yok edilir ve yıkıcısı çalışır. Üstelik yok edilme sırası oluşturulma sırasının tersidir. Bunu `new` kullanmadan, sıradan yerel değişkenlerle görelim:

kapsamSonu.cpp

```cpp
#include <iostream>

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; std::cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { std::cout << "Kaynak " << no << " yok edildi\n"; }
};

int main() {
    Kaynak a(1);
    {
        Kaynak b(2);
        Kaynak c(3);
        std::cout << "ic blok bitiyor\n";
    }
    std::cout << "dis blok bitiyor\n";
    return 0;
}
```

```
ÇIKTI
Kaynak 1 olustu
Kaynak 2 olustu
Kaynak 3 olustu
ic blok bitiyor
Kaynak 3 yok edildi
Kaynak 2 yok edildi
dis blok bitiyor
Kaynak 1 yok edildi
```

Hiç `delete` yazmadık, ama temizlik kendiliğinden ve ters sırada yapıldı: en son oluşan `c` ilk yok edildi. Bu sıra bilinçlidir; sonradan kurulan nesne genellikle öncekine dayanır, bu yüzden önce o temizlenmelidir.

> **Tanım: RAII**
>
> **RAII** (Resource Acquisition Is Initialization, "kaynak edinimi ilk değer vermedir"), bir kaynağın (bellek, dosya, bağlantı, kilit) ömrünü bir nesnenin ömrüne bağlayan ilkedir. Kaynak, nesnenin yapıcısında edinilir ve yıkıcısında bırakılır. Nesne ister normal akışla, ister `return` ile, ister istisna yüzünden kapsamdan çıksın, yıkıcı çalışır ve kaynak bırakılır.

Bu ilke, C'de ve çoğu dilde bulunmayan bir imkâna dayanır: yıkıcının **ne olursa olsun** çağrılması garantisi. Önceki bölümdeki sızıntılı örneği bu fikirle düzeltelim. `Kaynak*` işaretçisini, yıkıcısında `delete` çağıran küçük bir sınıfa sarıyoruz:

▸ elleRaii.cpp

```cpp
#include <iostream>
#include <stdexcept>

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; std::cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { std::cout << "Kaynak " << no << " yok edildi\n"; }
};

// Kaynak'ın sahibi: yapıcıda edinir, yıkıcıda bırakır
class KaynakSahibi {
    Kaynak* k;
public:
    KaynakSahibi(int no) { k = new Kaynak(no); }
    ~KaynakSahibi() { delete k; }
    KaynakSahibi(const KaynakSahibi&) = delete;             // kopya yasak
    KaynakSahibi& operator=(const KaynakSahibi&) = delete;
};

void islem(int deger) {
    KaynakSahibi s(1);
    if (deger < 0) {
        throw std::runtime_error("negatif deger");
    }
    std::cout << "islem tamam\n";
}

int main() {
    islem(5);
    try {
        islem(-5);
    } catch (const std::exception& e) {
        std::cout << "yakalandi: " << e.what() << '\n';
```

```
    }
    return 0;
}
```

```
ÇIKTI
Kaynak 1 olustu
islem tamam
Kaynak 1 yok edildi
Kaynak 1 olustu
Kaynak 1 yok edildi
yakalandi: negatif deger
```

İkinci çağrıda istisna fırlatıldığı hâlde "yok edildi" satırı, `catch` bloğundan **önce** göründü. İstisna işlevden çıkarken yerel nesneler yok edilir (buna yığın çözülmesi, stack unwinding denir; Bölüm 34'te işleyeceğiz). `s` yerel bir nesne olduğu için yıkıcısı çalıştı ve bellek geri verildi. Yazmamız gereken hiçbir `delete`, hiçbir ek denetim yok.

`KaynakSahibi` sınıfındaki iki `= delete` satırı kopyalamayı yasaklar. Yasak olmasaydı sahip kopyalanınca iki nesne aynı `Kaynak`'ı gösterir ve ikisi de `delete` ederdi (çifte silme). Kopya yapıcı ve kopya atamayı Bölüm 30'da işleyeceğiz.

Her `new` için böyle bir sınıf yazmak zorunda değiliz. Standart kütüphane bu işi bizim yerimize yapan hazır sınıflar sunar: **akıllı işaretçiler**.

> **Tanım: Akıllı işaretçi**
>
> İçinde bir adres tutan ve yok edilirken o adresteki nesneyi kendisi geri veren sınıftır (smart pointer). Normal işaretçi gibi `*` ve `->` ile kullanılır, ama `delete` yazmayı gerektirmez. `<memory>` başlığında üç çeşit bulunur: `unique_ptr`, `shared_ptr` ve `weak_ptr`.

> *Tarihçe · Akıllı işaretçilerin kısa tarihi*
>
> C++98 standardında tek bir akıllı işaretçi vardı: `std::auto_ptr`. Kopyalandığında sahipliği sessizce devrettiği için kapsayıcılarla birlikte hatalara yol açıyordu. C++11 (2011) `unique_ptr`, `shared_ptr` ve `weak_ptr`'yi getirdi ve `auto_ptr`'yi eskimiş ilan etti; C++17'de dilden tamamen kaldırıldı. `std::make_shared` C++11'de, `std::make_unique` C++14'te eklendi. RAII terimi ise Bjarne Stroustrup'a atfedilir.

29.6 std::unique_ptr: Tek Sahipli İşaretçi

`std::unique_ptr` bir nesnenin **tek** sahibidir. Kapsam bittiğinde ya da yeniden atandığında gösterdiği nesneyi `delete` eder. Oluşturmanın modern yolu `std::make_unique`'dir; `new` yazmadan nesneyi açar ve sahibini kurar:

```
std::unique_ptr<Tip> p = std::make_unique<Tip>(yapıcı_argümanları);
auto q = std::make_unique<Tip>(...);       // aynısı, kısa yazım
```

Kullanımı sıradan işaretçiye benzer: `*p` ile dolaylama, `p->üye` ile üye erişimi.

▸ uniqueTemel.cpp

```
#include <iostream>
#include <memory>
#include <string>
```

```
int main() {
    std::unique_ptr<int> p = std::make_unique<int>(42);   // öbekte bir int
    std::cout << *p << '\n';
    *p += 1;
    std::cout << *p << '\n';

    auto ad = std::make_unique<std::string>("Merhaba");   // öbekte bir string
    std::cout << *ad << " (" << ad->size() << " karakter)\n";

    p.reset();                        // bloğu şimdi geri ver
    if (p == nullptr) {
        std::cout << "p artik bos\n";
    }
    return 0;                         // ad burada yok edilir: delete yok!
}
```

```
ÇIKTI
42
43
Merhaba (7 karakter)
p artik bos
```

`p.reset()` bloğu hemen geri verir ve `p`'yi boşaltır. Bir `unique_ptr`'nin dolu ya da boş olduğunu `if (p)` ile de sorabiliriz. Bellek yalnızca `unique_ptr`'nin kendisi kadar yer kaplar; sıradan işaretçiye göre ek maliyeti yoktur.

Kopyalanamaz, taşınır

"Tek sahipli" demek, bir nesnenin aynı anda yalnız bir `unique_ptr` tarafından tutulması demektir. Bu yüzden `unique_ptr` kopyalanamaz:

uniqueKopya.cpp

```
#include <memory>

int main() {
    std::unique_ptr<int> p = std::make_unique<int>(5);
    std::unique_ptr<int> q = p;      // kopya: iki sahip olurdu
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

uniqueKopya.cpp: In function 'int main()':
uniqueKopya.cpp:5:30: error: use of deleted function 'std::unique_ptr<_Tp,
_Dp>::unique_ptr(const std::unique_ptr<_Tp, _Dp>&) [with _Tp = int; _Dp =
std::default_delete<int>]'
    5 |     std::unique_ptr<int> q = p;      // kopya: iki sahip olurdu
      |                              ^
```

Mesajın özü "silinmiş işlev kullanıldı"dır: kopyalama bilerek yasaklanmıştır. Bu iyi bir şeydir; kopya mümkün olsaydı iki sahip aynı nesneyi ikişer kez silerdi. Sahipliği başka birine **devretmek** istiyorsak `std::move` kullanırız. `std::move`, "bu değeri artık kullanmayacağız, sahipliği alabilirsin" demektir; nasıl çalıştığını Bölüm 39'da ayrıntılı göreceğiz. Devirden sonra eski işaretçi boş (`nullptr`) kalır. Sahiplik fonksiyona da devredilebilir, fonksiyondan da dönebilir:

uniqueTasima.cpp

```cpp
#include <iostream>
#include <memory>
#include <utility>

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; std::cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { std::cout << "Kaynak " << no << " yok edildi\n"; }
};

void goster(const Kaynak& k) {          // sadece KULLANIR, sahip olmaz
    std::cout << "goster: Kaynak " << k.no << '\n';
}

void devral(std::unique_ptr<Kaynak> k) {   // sahipliği ALIR
    std::cout << "devral basladi: " << k->no << '\n';
    std::cout << "devral bitiyor\n";
}                                          // k burada yok olur

std::unique_ptr<Kaynak> uret(int no) {
    return std::make_unique<Kaynak>(no);    // sahiplik çağırana geçer
}

int main() {
    auto a = std::make_unique<Kaynak>(1);
    std::unique_ptr<Kaynak> b = std::move(a);   // sahiplik a'dan b'ye
    if (a == nullptr) {
        std::cout << "a artik bos\n";
    }
    goster(*b);
    devral(std::move(b));                       // sahiplik fonksiyona
    if (b == nullptr) {
        std::cout << "devraldan sonra b bos\n";
    }
    auto c = uret(2);
    std::cout << "main bitiyor, c->no = " << c->no << '\n';
    return 0;
}
```

ÇIKTI

```
Kaynak 1 olustu
a artik bos
goster: Kaynak 1
devral basladi: 1
devral bitiyor
Kaynak 1 yok edildi
devraldan sonra b bos
Kaynak 2 olustu
main bitiyor, c->no = 2
Kaynak 2 yok edildi
```

Çıktıyı izleyelim:

- `std::move(a)` ile `a` boşaldı, `b` nesnenin sahibi oldu. Nesne bir kez oluştu; kopyalanmadı.
- `goster(*b)` yalnızca bakar: nesneyi referansla alır ve sahipliğe karışmaz.
- `devral(std::move(b))` çağrısında sahiplik fonksiyona geçti. Fonksiyon bitince `Kaynak 1` orada yok edildi, `main`'e dönmeden. `b` artık boştur.

- `uret(2)`: fonksiyonun yerel `unique_ptr`'si `return` ile çağırana taşınır; burada açıkça `std::move` yazmak gerekmez.
- `main` bitince `c` yok edilir ve `Kaynak 2` geri verilir.

C'deki "fonksiyon yeni blok döndürür, çağıran `free` eder" kuralı (Bölüm 13) burada güvenli hâline kavuştu: sahiplik yorum satırında değil, tipin içinde yazılıdır.

Erken çıkış ve dinamik dizi

`unique_ptr`'nin asıl kazancı, fonksiyondan hangi yoldan çıkılırsa çıkılsın nesnenin bırakılmasıdır:

```
void islemHam(int deger) {
    Kaynak* k = new Kaynak(deger);
    if (deger < 0) {
        return;                    // SIZINTI: delete'e ulaşılmadı
    }
    delete k;
}

void islem(int deger) {
    auto k = std::make_unique<Kaynak>(deger);
    if (deger < 0) {
        return;                    // k burada da yok edilir: sızıntı yok
    }
}                                  // k burada yok edilir
```

`unique_ptr` diziyi de taşıyabilir: tip `unique_ptr<int[]>` olur ve `std::make_unique<int[]>(n)` n elemanı sıfırlanmış olarak açar. Bırakırken doğru olanı, yani `delete[]`'i kendisi çağırır; eşleşme hatası yapamayız:

```
auto a = std::make_unique<int[]>(5);     // 5 eleman, hepsi 0
a[2] = 7;                                // sıradan dizi gibi kullan
std::unique_ptr<int[]> b = std::move(a); // sahiplik taşınır; delete[] yazılmaz
```

▲ Sık Yapılan Hata · unique_ptr<int[]> bir vector değildir

Bu tip de yalnızca bir işaretçidir: eleman sayısını bilmez (`n`'i bizim tutmamız gerekir), `size()` ve `push_back` işlevleri yoktur. Boyutu değişebilecek bir dizi için `vector` kullanmaya devam ederiz (Bölüm 26). `make_unique<int[]>`, boyutu hiç değişmeyen ve öbekte durması gereken bir blok ya da C'den kalma bir arayüz gerektiğinde işe yarar.

▲ Sık Yapılan Hata · Aynı ham adresi iki akıllı işaretçiye vermek

```
int* ham = new int{5};
std::unique_ptr<int> a(ham);
std::unique_ptr<int> b(ham);     // YANLIŞ: iki "tek" sahip, aynı blok
```

Derleyici bunu kabul eder, ama `a` ve `b` yok olurken ikisi de `delete` eder: çifte silme. Aynı hata `delete p.get();` yazmaktır. `get()` sahiplik vermeden ham adresi verir (gözlemlemek içindir); onu silmeyiz ve başka bir akıllı işaretçiye de vermeyiz. Kural: ham adres alınırsa hemen bir akıllı işaretçiye veririz; en iyisi `make_unique` kullanmaktır, çünkü o zaman ham adres hiç görünmez.

İpucu · unique_ptr ve C kütüphaneleri

`unique_ptr`, ikinci bir şablon argümanıyla özel bir bırakma işlevi alabilir. Böylece `fopen` ile açılan bir dosyayı da RAII'ye bağlayabiliriz:

```
std::unique_ptr<FILE, decltype(&fclose)> dosya(fopen("not.txt", "r"), &fclose);
```

Bu kalıbı Bölüm 37'de dosya işlemlerinde `fstream` ile karşılaştıracağız; `fstream` aynı işi çok daha rahat yapar.

29.7 std::shared_ptr: Paylaşımlı Sahiplik

Bazen bir nesneyi gerçekten birden çok yerin paylaşması gerekir ve "son kullanan temizlesin" isteriz. `std::shared_ptr` bunu yapar: içeride bir **referans sayacı** (reference count) tutar. İşaretçi kopyalandıkça sayaç artar, bir kopya yok oldukça azalır; sayaç 0'a inince nesne geri verilir. Oluşturmanın yolu `std::make_shared`, sayacı görmenin yolu `use_count()`'tur.

Tanım: Referans sayımı

Bir nesneyi kaç `shared_ptr`'nin gösterdiğini sayan düzenektir. Sayaç sıfıra indiği anda nesne yok edilir. Sayaç `shared_ptr`'nin paylaştığı ayrı bir denetim bloğunda durur ve sayaç işlemleri iş parçacığı bakımından güvenlidir; gösterilen nesne ise güvenli hâle gelmez.

paylasimliSayac.cpp

```
#include <iostream>
#include <memory>

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; std::cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { std::cout << "Kaynak " << no << " yok edildi\n"; }
};

int main() {
    auto a = std::make_shared<Kaynak>(5);
    std::cout << "sayac: " << a.use_count() << '\n';
    {
        std::shared_ptr<Kaynak> b = a;        // kopya: sayaç artar
        auto c = a;                           // bir kopya daha
        std::cout << "ic blokta sayac: " << a.use_count() << '\n';
    }                                         // b ve c yok olur: sayaç azalır
    std::cout << "ic bloktan sonra sayac: " << a.use_count() << '\n';
    a.reset();                                // son sahip bıraktı
    std::cout << "reset sonrasi\n";
    return 0;
}
```

ÇIKTI

```
Kaynak 5 olustu
sayac: 1
ic blokta sayac: 3
ic bloktan sonra sayac: 1
Kaynak 5 yok edildi
reset sonrasi
```

`b` ve `c` yok olunca nesne silinmedi, çünkü `a` hâlâ sahipti (sayaç 1). `a.reset()` ile sayaç 0'a indi ve `Kaynak 5` yok edildi. Aynı fikri daha gerçekçi bir durumda görelim: üç kullanıcı aynı bağlantıyı paylaşıyor ve bağlantıyı son ayrılan kapatıyor.

paylasilanKaynak.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; std::cout << "Baglanti " << no << " acildi\n"; }
    ~Kaynak() { std::cout << "Baglanti " << no << " kapandi\n"; }
};

struct Kullanici {
    std::string ad;
    std::shared_ptr<Kaynak> baglanti;
};

int main() {
    auto ortak = std::make_shared<Kaynak>(100);
    std::vector<Kullanici> liste;
    liste.push_back({"Ayse", ortak});
    liste.push_back({"Mehmet", ortak});
    liste.push_back({"Elif", ortak});
    ortak.reset();                  // main'in kendi payını bıraktı

    while (!liste.empty()) {
        std::cout << liste.back().ad << " ayriliyor, sayac "
                  << liste.back().baglanti.use_count() << '\n';
        liste.pop_back();
    }
    std::cout << "liste bos\n";
    return 0;
}
```

ÇIKTI

```
Baglanti 100 acildi
Elif ayriliyor, sayac 3
Mehmet ayriliyor, sayac 2
Ayse ayriliyor, sayac 1
Baglanti 100 kapandi
liste bos
```

Hiçbir kullanıcı bağlantıyı kapatmaktan sorumlu değil; en son ayrılan Ayşe'nin ardından, sayaç sıfırlanınca bağlantı kapandı. `unique_ptr`'den farkı: `shared_ptr` kopyalanabilir; karşılığında sayaç tutmanın küçük bir maliyeti vardır.

> **▲ Sık Yapılan Hata · shared_ptr her derde deva değildir**
>
> Sahiplik belirsizleştiğinde `shared_ptr` akla gelir, ama çoğu zaman tasarım sorununu gizler. Sayaç işlemleri `unique_ptr`'den yavaştır; nesnenin ne zaman yok edileceğini kestirmek de zorlaşır. Varsayılan tercihimiz `unique_ptr` olmalı, `shared_ptr`'ye yalnızca birden çok sahibin gerçekten gerekli olduğu yerde geçmeliyiz. İyi bir ölçü: sahiplik tek bir yerde belirlenebiliyorsa `unique_ptr`.

Fonksiyona `shared_ptr` değerle geçirmek kopya demektir ve çağrı süresince sayacı bir artırır. Nesneyi yalnızca kullanacak bir fonksiyon `const Kaynak&` almalıdır; sahipliği paylaşacak bir fonksiyon ise `shared_ptr`'yi değerle alır.

29.8 std::weak_ptr ve Döngüsel Referans

`shared_ptr`'nin bir zayıf noktası vardır: iki nesne birbirini `shared_ptr` ile tutarsa sayaçlar hiçbir zaman sıfıra inmez.

▸ dongu.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>

struct Dugum {
    std::string ad;
    std::shared_ptr<Dugum> sonraki;
    Dugum(std::string a) : ad(a) { std::cout << ad << " olustu\n"; }
    ~Dugum() { std::cout << ad << " yok edildi\n"; }
};

int main() {
    auto a = std::make_shared<Dugum>("A");
    auto b = std::make_shared<Dugum>("B");
    a->sonraki = b;                      // A, B'yi tutar
    b->sonraki = a;                      // B, A'yı tutar: döngü
    std::cout << "sayaclar: " << a.use_count() << ' ' << b.use_count() << '\n';
    std::cout << "main bitiyor\n";
    return 0;
}
```

```
ÇIKTI
A olustu
B olustu
sayaclar: 2 2
main bitiyor
```

`main` bitince `a` ve `b` yok olur, ama her nesneyi diğerinin `sonraki` üyesi hâlâ tutuyor; sayaçlar 2'den 1'e iner ve 0'a hiç ulaşmaz. Hiçbir "yok edildi" satırı görünmedi: iki nesne de sızdı. Elle erişilemeyen bir döngü olduğu için sızıntıyı programcı da kapatamaz.

Çözüm, döngünün bir yönünü sahiplik **taşımayan** bir bağla kurmaktır. `std::weak_ptr`, bir `shared_ptr`'nin yönettiği nesneyi gösterir ama sayaca katılmaz.

> **Tanım: weak_ptr**
>
> Bir `shared_ptr`'nin yönettiği nesneyi gözlemleyen, ama nesnenin ömrünü uzatmayan akıllı işaretçidir. Nesneye doğrudan erişemeyiz; `lock()` ile geçici bir `shared_ptr` alırız. Nesne yok edilmişse `lock()` boş bir `shared_ptr` döndürür ve `expired()` `true` olur.

Döngüyü kıralım: bir düğüm sonrakini sahiplikle (`shared_ptr`), öncekini gözlemle (`weak_ptr`) tutsun.

zayifBag.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>

struct Dugum {
    std::string ad;
    std::shared_ptr<Dugum> sonraki;      // sahiplik
    std::weak_ptr<Dugum> onceki;         // gözlem: sayaca katılmaz
    Dugum(std::string a) : ad(a) { std::cout << ad << " olustu\n"; }
    ~Dugum() { std::cout << ad << " yok edildi\n"; }
};

int main() {
    auto a = std::make_shared<Dugum>("A");
    auto b = std::make_shared<Dugum>("B");
    a->sonraki = b;
    b->onceki = a;
    std::cout << "sayaclar: " << a.use_count() << ' ' << b.use_count() << '\n';

    if (auto o = b->onceki.lock()) {     // geçici shared_ptr
        std::cout << "B'nin oncesi: " << o->ad << '\n';
    }
    std::cout << "main bitiyor\n";
    return 0;
}
```

ÇIKTI

```
A olustu
B olustu
sayaclar: 1 2
B'nin oncesi: A
main bitiyor
A yok edildi
B yok edildi
```

Artık `a`'nın sayacı 1'dir (yalnızca `main`'deki `a` onu tutuyor), `b`'ninki 2'dir (`main`'deki `b` ve `a->sonraki`). `main` biterken yerel değişkenler ters sırada yok olur: önce `b` bırakılır (sayaç 2'den 1'e), sonra `a` bırakılır (1'den 0'a); `A` yok edilirken `sonraki` üyesi de bırakılır ve `B`'nin sayacı da 0'a iner. İki düğüm de temizlendi.

`lock()` çağrısı iki işi birden yapar: nesne hâlâ yaşıyorsa onu çağrı süresince canlı tutan bir `shared_ptr` verir, yoksa boş döner. Yaşam süresi belirsiz bir nesneye erişmenin güvenli yolu budur:

weakKilit.cpp

```cpp
#include <iostream>
#include <memory>

int main() {
    std::weak_ptr<int> zayif;
    {
        auto guclu = std::make_shared<int>(99);
        zayif = guclu;
        std::cout << "sayac: " << zayif.use_count() << '\n';
        if (auto g = zayif.lock()) {
            std::cout << "erisilebilir: " << *g << '\n';
        }
    }
    std::cout << "suresi doldu mu? " << std::boolalpha << zayif.expired() << '\n';
    if (zayif.lock() == nullptr) {
```

```
        std::cout << "nesne artik yok\n";
    }
    return 0;
}
```

```
ÇIKTI
sayac: 1
erisilebilir: 99
suresi doldu mu? true
nesne artik yok
```

Zayıf işaretçiler önbellek (cache), gözlemci (observer) ve ebeveyn bağlantılarında yaygındır: ebeveyn çocuğu `shared_ptr` ile sahiplenir, çocuk ebeveyni `weak_ptr` ile gösterir.

29.9 Ne Zaman Hangisi?

Bellek gerektiğinde şu sırayla düşünürüz; ilk uyan çözümde dururuz:

İhtiyaç	Seçim	Neden
Ömrü kapsamla sınırlı bir değer	Sıradan yerel değişken (yığın)	En hızlı, temizlik ücretsiz
Metin	`std::string`	RAII, kendi büyür
Çok sayıda eleman, boyut değişebilir	`std::vector`	RAII, varsayılan tercih
Kapsamı aşan tek bir nesne, tek sahip	`unique_ptr` + `make_unique`	RAII, kopya yok, ek maliyet yok
Gerçekten birden çok sahip	`shared_ptr` + `make_shared`	Son sahip temizler
Paylaşılan nesneyi gözlemek, döngüyü kırmak	`weak_ptr`	Sayaca katılmaz
Sadece kullanmak (sahip olmadan)	Referans ya da ham işaretçi	Sahipliğe karışmaz
Doğrudan `new` / `delete`	Neredeyse hiç	Eski kodu okurken, kendi RAII sınıfımızı yazarken

Fonksiyon parametrelerinde sahiplik niyeti imzadan okunabilir:

Fonksiyonun niyeti	Parametre
Nesneyi yalnızca okur/değiştirir	`const T&` ya da `T&`
Nesne olmayabilir (isteğe bağlı)	`T*` (sahip değil, `nullptr` olabilir)
Nesnenin sahipliğini alır	`std::unique_ptr<T>` (değerle; çağıran `std::move` yazar)
Sahipliği paylaşır	`std::shared_ptr<T>` (değerle)
Yeni nesne üretip sahipliği verir	Dönüş tipi `std::unique_ptr<T>`

◆ Kritik Nokta · Modern C++ kuralı: çıplak new ve delete yazma

Öbeğe elle ulaşmamız gerekmiyorsa `vector`, `string` ya da yığındaki değişkeni kullanırız; öbekte tek bir nesne gerekiyorsa `make_unique`; paylaşılıyorsa `make_shared`. Böylece sızıntı, çifte silme ve `delete[]` ile `delete` karışması gündemimizden çıkar. Koda `new` yazmak üzereyken şu iki soruyu soralım: "Bunun `delete`'i nerede?" ve "Bunun yerine `vector` ya da `make_unique` olamaz mı?" Cevap neredeyse hep ikincisidir.

Peki `new` ve `delete`'i neden öğrendik? Dört nedenle: `vector` ve akıllı işaretçilerin içte ne yaptığını anlamak için, mevcut eski kodu okuyabilmek için, Bölüm 30'da kendi sınıfımızın içinde bellek yönetirken ve Bölüm 39'da taşıma semantiğini kavrayabilmek için. Ham işaretçi yeni yazdığımız kodda yalnızca bir şeye **bakmak** için kalır; sahiplik taşımaz.

Üç Dilde · Belleği kim temizler: C, C++ ve C# *C · C++ · C#*

C'de her `malloc` için bir `free` yazarız ve sahiplik kuralı yalnızca yorum satırındadır (Bölüm 13). **C++**'ta aynı iş bir nesneye bırakılır:

```
auto p = std::make_unique<int>(42);      // C: malloc + free
auto v = std::vector<int>(100);          // C: malloc(100 * sizeof(int)) + free
```

C#'ta bellek, **çöp toplayıcı** (garbage collector, GC) tarafından yönetilir: nesneye hiçbir referans kalmayınca GC onu uygun bir anda toplar. `delete` ya da `free` karşılığı bir komut yoktur; sarkan işaretçi ve çifte silme hataları da yoktur. Ancak GC yalnızca **belleği** yönetir. Dosya, ağ bağlantısı gibi kaynaklar için C#, `IDisposable` arayüzünü ve `using` deyimini sunar; `using` bloğu bitince `Dispose()` çağrılır. Bu, C++'ın RAII'sine karşılık gelir, ama tek farkla: C++'ta yıkıcı otomatiktir, C#'ta `using` yazmayı hatırlamak programcıya düşer.

```
using (var yazar = new StreamWriter("not.txt"))
{
    yazar.WriteLine("merhaba");
}                                        // blok bitince Dispose() çağrılır
```

	C	C++	C#
Bellek isteme	`malloc`	`new`, `make_unique`	`new`
Geri verme	`free` (elle)	Yıkıcı / akıllı işaretçi	GC (kesin zamanı bilinmez)
Bellek dışı kaynaklar	Elle `fclose` vb.	RAII (yıkıcı)	`IDisposable` + `using`
Döngüsel referans	Yok (sahiplik elle)	`shared_ptr` sızdırır, `weak_ptr` çözer	GC döngüyü kendisi çözer

Çöp toplayıcıya Bölüm 41'de kısa bir giriş yapacak, `using` ve `IDisposable`'ı Bölüm 53'te işleyeceğiz.

Bölüm Özeti

- `new Tip(değer)` öbekte nesne kurar ve adresini döndürür; `delete p` yıkıcıyı çağırıp bloğu geri verir. `new[]` ile `delete[]`, `new` ile `delete` eşleşmelidir; karışırsa davranış tanımsızdır.
- `new` ham bayt vermez, nesne kurar: sınıf tiplerinde yapıcıyı, `delete` de yıkıcıyı çağırır. Bu yüzden C++ nesneleri için `malloc`/`free` kullanılmaz.

- `new` yer bulamazsa `nullptr` döndürmez, `std::bad_alloc` istisnası fırlatır; sonrasında `nullptr` denetimi gereksizdir.
- `delete`'ten sonra işaretçi aynı adresi tutar. `p = nullptr;` yalnızca o işaretçiyi korur; aynı bloğu gösteren diğer işaretçiler sarkan kalır.
- Elle yönetimde sızıntı, sarkan işaretçi, çifte silme ve yanlış eşleşme hataları vardır. C++'ta ek yol istisnadır: `throw` ile çıkılan işlevde `delete` satırına ulaşılmaz.
- RAII: kaynağı yapıcıda edin, yıkıcıda bırak. Yerel nesneler kapsam sonunda, oluşturulmanın tersi sırada ve her çıkış yolunda (normal, `return`, istisna) yok edilir.
- `std::unique_ptr` (`make_unique`) tek sahiplidir; kopyalanamaz, `std::move` ile taşınır ve taşınan boş kalır. Kapsam bitince nesneyi kendisi siler.
- `unique_ptr<T[]>` dizi taşır ve `delete[]`'i kendisi çağırır; eleman sayısını bilmez, büyümez. Büyüyecek veri için `vector` kullanılır.
- `std::shared_ptr` (`make_shared`) referans sayar. Kopya sayacı artırır, yok olan kopya azaltır, sayaç 0 olunca nesne silinir. Değerle geçirmek kopya, yani sayaç artışıdır.
- İki nesne birbirini `shared_ptr` ile tutarsa sayaçlar sıfıra inmez ve ikisi de sızar. `std::weak_ptr` sayaca katılmayan gözlemcidir; `lock()` ile geçici `shared_ptr` alınır, `expired()` ile nesnenin ölüp ölmediği sorulur.
- Seçim sırası: yerel değişken, `string`/`vector`, `unique_ptr`, `shared_ptr`, en son doğrudan `new`/`delete`. Çıplak `new` yazmayız.
- Parametrelerde niyet imzada yazılır: okuma için `const T&`, sahipliği almak için `unique_ptr<T>` (çağıran `std::move` yazar), paylaşmak için `shared_ptr<T>`.
- C#'ta çöp toplayıcı belleği yönetir; bellek dışı kaynaklar için `IDisposable` ve `using` kullanılır. C++'ta ikisini de RAII karşılar.

Alıştırmalar

1. `int* d = new int[10];` satırıyla alınan bloğu doğru biçimde geri vermenin yolu hangisidir?

   A) `delete d;`
   B) `delete[] d;`
   C) `free(d);`
   D) `delete d[10];`
   E) `d = nullptr;`

2. Aşağıdaki satırlardan sonra program bitiyor (başka satır yok). Kaç öbek bloğu sızmıştır?

```
int* a = new int(1);
int* b = new int(2);
a = b;
delete b;
int* c = new int[3]{};
```

   A) 0
   B) 1
   C) 2

D) 3
E) 4

3. Aşağıdaki iki satırdan sonra hangisi doğrudur?

```
auto p = std::make_unique<int>(10);
auto q = std::move(p);
```

A) `q` nesnenin sahibidir, `p` boştur.
B) `p` ve `q` ikisi de sahiptir; nesne iki kez silinir.
C) Satır derlenmez, çünkü `unique_ptr` taşınamaz.
D) `p` hâlâ 10'u gösterir, `q` boştur.
E) 10 değeri iki ayrı bloğa kopyalanır.

4. Aşağıdaki kod ekrana ne yazdırır?

```
auto a = std::make_shared<int>(5);
auto b = a;
{
    auto c = a;
    auto d = b;
}
auto e = a;
std::shared_ptr<int> f;
f = b;
std::cout << a.use_count();
```

A) 1
B) 2
C) 3
D) 4
E) 5

5. Eleman sayısı çalışma anında belli olan ve sonradan büyüyebilecek bir `int` listesini saklamak için modern C++'ta en uygun yol hangisidir?

A) `int* d = (int*)malloc(n * sizeof(int));`
B) `int* d = new int[n];` ve sonunda `delete[] d;`
C) `int d[n];`
D) `std::shared_ptr<int> d(new int[n]);`
E) `std::vector<int> d(n);`

6. `std::weak_ptr` ile ilgili hangisi doğrudur?

A) Referans sayacını artırır.
B) Gösterdiği nesneyi `*` ile doğrudan kullanabiliriz.
C) Nesne yok edilmişse `lock()` boş bir `shared_ptr` döndürür.
D) `unique_ptr`'nin kopyalanabilir biçimidir.
E) Nesnenin ömrünü uzatır.

7. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <memory>
```

```
struct Iz {
    char ad;
    Iz(char a) : ad(a) { std::cout << '+' << ad; }
    ~Iz() { std::cout << '-' << ad; }
};

int main() {
    auto p = std::make_unique<Iz>('A');
    {
        auto q = std::make_unique<Iz>('B');
        auto r = std::move(p);
        std::cout << '|';
    }
    std::cout << '|' << (p == nullptr) << '\n';
    auto s = std::make_unique<Iz>('C');
    return 0;
}
```

8. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <memory>
#include <vector>

void say(std::shared_ptr<int> p) {
    std::cout << p.use_count() << ' ';
}

int main() {
    auto x = std::make_shared<int>(8);
    std::vector<std::shared_ptr<int>> v;
    v.push_back(x);
    v.push_back(x);
    say(x);
    v.pop_back();
    say(x);
    auto y = x;
    std::cout << x.use_count() << ' ';
    x.reset();
    std::cout << y.use_count() << ' ' << (x == nullptr) << ' ' << *v[0] << '\n';
    return 0;
}
```

9. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>

int main() {
    int* d = new int[5]{2, 4};
    d[2] = d[0] * d[1];
    int* e = d + 1;
    e[2] = e[1] + 1;
    int toplam = 0;
    for (int i = 0; i < 5; i++) {
        toplam += d[i];
    }
    std::cout << toplam << '\n';
    for (int i = 0; i < 5; i++) {
        std::cout << d[i] << ' ';
    }
    std::cout << '\n';
```

```
    delete[] d;
    return 0;
}
```

10. Aşağıdaki kod parçasındaki bellek hatalarını bulun ve her birini nasıl düzelteceğinizi söyleyin.

```
int* p = new int[3]{1, 2, 3};
int* q = p;
delete p;
std::cout << q[0] << '\n';
delete[] q;
```

11. Aşağıdaki kod derlenmez. Hatanın nedenini açıklayın ve iki farklı biçimde düzeltin.

```
void isle(std::unique_ptr<int> p) {
    std::cout << *p << '\n';
}

int main() {
    auto a = std::make_unique<int>(3);
    isle(a);
    return 0;
}
```

12. Aşağıdaki program derlenir ve çalışır, ama "yok edildi" satırları hiç görünmez. Nedenini açıklayın ve düzeltin.

```
struct Cocuk;
struct Ebeveyn {
    std::shared_ptr<Cocuk> cocuk;
    ~Ebeveyn() { std::cout << "Ebeveyn yok edildi\n"; }
};
struct Cocuk {
    std::shared_ptr<Ebeveyn> ebeveyn;
    ~Cocuk() { std::cout << "Cocuk yok edildi\n"; }
};

int main() {
    auto e = std::make_shared<Ebeveyn>();
    auto c = std::make_shared<Cocuk>();
    e->cocuk = c;
    c->ebeveyn = e;
    return 0;
}
```

13. Klavyeden önce bir `n`, sonra `n` tane tamsayı okuyan `tersSirada` programını yazın. Sayıları `std::make_unique<int[]>` ile açtığınız dizide tutun; sayıları ters sırada ve en küçük sayıyı yazdırın. `n` geçerli değilse (`n <= 0` ya da okuma başarısız) `Gecersiz sayi.` yazıp çıkın. Programda `new` ve `delete` kullanmayın.

```
ÇIKTI
Kac sayi? 5
Sayilari gir: 4 9 2 7 3
Ters: 3 7 2 9 4
En kucuk: 2
```

14. `std::unique_ptr<int[]> kareler(int n)` işlevini yazın: ilk `n` doğal sayının karelerini (1, 4, 9, …) yeni açtığı dizide tutup sahipliği çağırana döndürsün. `main` kullanıcıdan `n`'i okusun, kareleri ve toplamlarını yazdırsın.

```
ÇIKTI
Kac eleman? 6
1 4 9 16 25 36
Toplam: 91
```

15. `Ogrenci` adında, yapıcısında `... kaydedildi`, yıkıcısında `... silindi` yazan bir yapı ve içinde `std::vector<std::shared_ptr<Ogrenci>>` tutan `Kurs` yapısı yazın. `main`'de bir öğrenciyi iki farklı kursa ekleyin, kendi işaretçinizi bırakın, sayacı yazdırın; sonra öğrenciyi önce bir kurstan, sonra diğerinden çıkarın ve öğrencinin ne zaman silindiğini gözleyin.

BÖLÜM 30

Sınıflar ve Nesneler

BU BÖLÜMDE

- Veri ile onu işleyen fonksiyonları tek pakette toplama fikrini; C'deki `struct` ile C++'taki `class` arasındaki geçişi
- Sınıf ile nesne farkını, üye değişkenleri ve üye fonksiyonları
- `public`, `private`, `protected` erişim belirteçlerini ve kapsüllemeyi; getter ve setter'ın asıl işlevini
- Yapıcıları (varsayılan, parametreli, aşırı yüklenmiş, devreden), üye ilk değer listesini ve yıkıcıyı
- `this`, `const` üye fonksiyonlar ve `static` üyeler
- Nesneleri fonksiyonlara geçirmeyi, `vector` içinde tutmayı
- Kopya yapıcı, kopya atama, sığ ve derin kopya farkını, üçler kuralını
- Bir sınıfı `.hpp` ve `.cpp` dosyalarına ayırmayı; `struct` ile `class` farkını ve `friend`'i

Bölüm 14'te `struct` ile birbirine ait verileri tek tipte topladık, Bölüm 21'de de C'nin bu yaklaşımındaki zayıflıkları gördük: veri ile işlemler birbirinden kopuktur ve veriyi kimin değiştirdiği denetlenemez. Bu bölümde C++'ın ana yapı taşına, **sınıfa** (class) geçiyoruz. Sınıf, verileri ve onlar üzerinde çalışan fonksiyonları tek bir tipte birleştirir; hangi parçaların dışarıdan görüleceğine de sınıfın kendisi karar verir.

Bu bölüm nesne yönelimli programlamanın temelidir. Sonraki bölümlerdeki kalıtım (Bölüm 31), operatör aşırı yükleme (Bölüm 32), şablonlar (Bölüm 33) ve istisnalar (Bölüm 34) sınıfların üzerine kurulur. Ayrıca Bölüm 29'da anlattığımız RAII'nin "kaynağı nesnenin ömrüne bağlamak" fikri, yapıcı ve yıkıcı sayesinde burada somutlaşır. Bölüm 14, 25, 28 ve 29'u okuduğunuzu varsayıyoruz; oradaki konuları tekrar anlatmayacak, yalnızca sınıflar açısından ne değiştiğini göstereceğiz.

30.1 Neden Sınıf?

Bölüm 14'teki yaklaşımı C++'ta bir kez daha yazalım. Bir banka hesabı için `struct` ve ona etki eden serbest bir fonksiyon tanımlıyoruz:

▸ hesapStruct.cpp

```
#include <iostream>
#include <string>
using namespace std;

struct Hesap {
    string sahip;
    double bakiye;
};

void yatir(Hesap& h, double miktar) {
```

```
    if (miktar > 0) {
        h.bakiye += miktar;       // yalnızca pozitif tutar kabul edilir
    }
}

int main() {
    Hesap h{"Ayşe", 500};
    yatir(h, 250);
    h.bakiye = -5000;             // derleyici sesini çıkarmaz
    cout << h.sahip << ": " << h.bakiye << " TL\n";
    return 0;
}
```

```
ÇIKTI
Ayşe: -5000 TL
```

`yatir` fonksiyonu tutarı denetliyor, ama bu denetim yalnızca o fonksiyonu **çağırırsak** işe yarıyor. Programın herhangi bir yerinden `h.bakiye` doğrudan değiştirilebilir ve hesap geçersiz bir duruma düşer. Veri ile ona uygulanan kurallar arasındaki bağ yalnızca bir adlandırma geleneğidir.

Nesne yönelimli yaklaşımın cevabı üç adımdır:

1. Veriyi ve onu işleyen fonksiyonları aynı tipin içine koymak.
2. Veriyi dışarıdan erişime kapatıp yalnızca belirli fonksiyonlara izin vermek (**kapsülleme**).
3. Nesne doğarken geçerli bir durumla başlamasını ve yok olurken kaynaklarını geri vermesini tipin kendisine bırakmak (**yapıcı** ve **yıkıcı**).

Bu bölümün geri kalanı bu üç adımın ayrıntısıdır.

30.2 İlk Sınıf: Üyeler ve Üye Fonksiyonlar

> **Tanım: Sınıf ve nesne**
>
> **Sınıf** (class), bir tipin hangi verileri tutacağını ve hangi işlemleri yapabileceğini tarif eden şablondur. **Nesne** (object), bu şablondan üretilmiş somut bir örnektir. Kurabiye kalıbı sınıf, kalıptan çıkan kurabiyeler nesnedir: hepsinin yapısı aynıdır, ama her birinin değerleri ayrıdır.

Sınıfın içindeki değişkenlere **üye değişken** (data member), içindeki fonksiyonlara **üye fonksiyon** (member function) denir. Başka dillerde üye fonksiyonlara "metot" da denir; C++ metinlerinde "üye fonksiyon" deyimi yaygındır ve bu kitapta onu kullanacağız.

▸ ilkSinif.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Ogrenci {
public:
    string ad;
    int no;
    double ortalama;

    void bilgiYazdir() {              // üye fonksiyon
        cout << no << " - " << ad << " - " << ortalama << '\n';
    }
```

```
};

int main() {
    Ogrenci a{"Mehmet", 101, 72.5};
    Ogrenci b{"Elif", 102, 88.0};
    a.bilgiYazdir();
    b.bilgiYazdir();

    Ogrenci* p = &b;
    p->ortalama = 90.5;              // (*p).ortalama ile aynı
    p->bilgiYazdir();
    return 0;
}
```

```
ÇIKTI
101 - Mehmet - 72.5
102 - Elif - 88
102 - Elif - 90.5
```

Bu örnekten çıkan noktalar şunlardır:

- Üyelere, `struct`'ta olduğu gibi nokta operatörüyle (`a.ad`), nesneye işaretçi varsa ok operatörüyle (`p->ortalama`) ulaşılır (Bölüm 14).
- Üye fonksiyon hangi nesne üzerinden çağrıldıysa **onun** üyelerini kullanır. `a.bilgiYazdir()` içindeki `no`, `a`'nın `no`'sudur. Çağrı noktasındaki nesne "kimin?" sorusunu cevaplar.
- Sınıf içinde üye fonksiyonlar, kendilerinden sonra tanımlanan üyeleri de görür; üyelerin yazılış sırası önemsizdir.
- `Ogrenci a{...}` biçimindeki ilk değer verme, üyeleri tanım sırasıyla doldurur. Bu biçim yalnızca tüm üyeler `public` ve sınıfta yapıcı yokken çalışır; yapıcıları az sonra göreceğiz.

Sınıf yazmanın ilk kazancı budur: `bilgiYazdir` artık `Ogrenci` tipinin bir parçasıdır, ayrı bir serbest fonksiyon olarak aranmaz. Ama `ortalama` hâlâ herkese açıktır; sıradaki konu bunu çözüyor.

30.3 Erişim Belirteçleri ve Kapsülleme

Üyeler herkese açıksa biri `ogr.ortalama = 250;` yazıp nesneyi geçersiz duruma düşürebilir. **Kapsülleme** (encapsulation), veriyi dış dünyadan gizleyip ona yalnızca sınıfın izin verdiği yoldan erişilmesini sağlamaktır. Bankamatikte kasayı açıp parayı kendimiz almayız; düğmelerle kurallara uygun işlem yaparız. Gizlemeyi **erişim belirteçleri** (access specifiers) yapar:

Belirteç	Sınıfın kendi üyeleri	Sınıfın dışı (`main`, serbest fonksiyonlar)
`public`	Erişir	Erişir
`private`	Erişir	Erişemez
`protected`	Erişir	Erişemez (türemiş sınıflar erişir, Bölüm 31)

Bir belirteç, bir sonrakini görene ya da sınıf bitene kadar geçerlidir. Belirteç yazmazsak `class` içindeki üyeler `private` olur. Bunu unutmak, ilk sınıfını yazan herkesin yaşadığı hatadır:

privateErisim.cpp

```cpp
class Hesap {
    double bakiye{100};          // belirteç yok: class'ta private
};

int main() {
    Hesap h;
    h.bakiye = -5000;
    return 0;
}
```

DERLEYİCİ ÇIKTISI

```
privateErisim.cpp: In function 'int main()':
privateErisim.cpp:7:7: error: 'double Hesap::bakiye' is private within this context
    7 |     h.bakiye = -5000;
      |       ^~~~~~
privateErisim.cpp:2:12: note: declared private here
    2 |     double bakiye{100};          // belirteç yok: class'ta private
      |            ^~~~~~
```

Kural şudur: **üye değişkenler `private`, dışarıya sunulan işlemler `public` olur.** Dışarıdan okunması gereken üyeler için **getter**, değiştirilmesi gereken üyeler için **setter** yazarız. Setter'ın asıl değeri, değeri yazmadan önce doğrulama yapabilmesidir:

kapsulleme.cpp

```cpp
#include <iostream>
using namespace std;

class Ogrenci {
private:
    double ortalama{0};              // sınıf içi ilk değer

public:
    double ortalamaAl() const { return ortalama; }       // getter

    void ortalamaAyarla(double yeni) {                   // setter
        if (yeni < 0 || yeni > 100) {
            cout << "Geçersiz ortalama: " << yeni << '\n';
            return;                  // alanı değiştirmeden çık
        }
        ortalama = yeni;
    }

    double ortalamaFarki(const Ogrenci& diger) const {
        return ortalama - diger.ortalama;   // başka nesnenin private üyesi
    }
};

int main() {
    Ogrenci o;
    Ogrenci p;
    o.ortalamaAyarla(78.5);
    o.ortalamaAyarla(250);
    p.ortalamaAyarla(30);
    cout << o.ortalamaAl() << " " << p.ortalamaAl() << '\n';
    cout << "Fark: " << o.ortalamaFarki(p) << '\n';
    return 0;
}
```

```
ÇIKTI
Geçersiz ortalama: 250
78.5 30
Fark: 48.5
```

Doğrulama sayesinde geçersiz 250 değeri alana hiç yazılmadı. Parantezden sonraki `const`'u "`const` üye fonksiyonlar" başlığında açıklayacağız; şimdilik kalıp olarak alabiliriz.

◆ Kritik Nokta · Erişimi sınıf belirler, nesne değil

`ortalamaFarki` içinde `diger.ortalama` yazılabildiğine dikkat edelim: `private`, **sınıfa** özeldir, nesneye değil. Bir sınıfın üye fonksiyonu, aynı sınıftan başka bir nesnenin `private` üyesini de okuyabilir. Sınıfın dışındaki `main` ya da serbest fonksiyonlar ise okuyamaz.

İpucu · Her üyeye get/set yazmayalım

Her üyeye mekanik bir getter ve setter yazmak kapsülleme değildir; veriyi yalnızca daha uzun bir yoldan açık bırakmaktır. Okunması gereken üyeye getter, değişmesi ve doğrulanabilmesi gereken üyeye setter yazalım. Hesap sınıfında `bakiyeAyarla(5000)` yerine `yatir` ve `cek` gibi anlamlı işlemler sunmak çok daha iyidir. Başka kitaplarda `getAd()` ve `setAd()` adlarını görebilirsiniz; bu kitapta `adAl()` ve `adAyarla()` biçimini kullanıyoruz.

30.4 Yapıcılar

`private` üyeleri dışarıdan değiştiremiyorsak nesneye ilk değerini kim verecek? Hiçbir şey yapılmazsa üyeler, Bölüm 5'te gördüğümüz çöp değerle doğar. Çözüm **yapıcıdır** (constructor): nesne oluşturulurken otomatik çalışan özel bir üye fonksiyon.

Tanım: Yapıcı

Sınıfla aynı adı taşıyan, dönüş tipi olmayan (`void` bile yazılmaz) ve nesne yaratıldığı anda otomatik çağrılan üye fonksiyondur. Amacı nesneyi geçerli bir durumla başlatmaktır.

▸ hesapYapici.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankaHesabi {
private:
    string sahip;
    double bakiye;

public:
    BankaHesabi(string yeniSahip, double ilkBakiye) {    // yapıcı
        sahip = yeniSahip;
        bakiye = ilkBakiye >= 0 ? ilkBakiye : 0;         // geçersizse 0
        cout << sahip << " için hesap açıldı\n";
    }

    void bilgiYazdir() const {
        cout << sahip << ": " << bakiye << " TL\n";
    }
```

```
};

int main() {
    BankaHesabi a{"Mehmet", 500};     // yapıcı bu satırda çalışır
    BankaHesabi b("Zeynep", -20);     // parantezle de yazılır
    a.bilgiYazdir();
    b.bilgiYazdir();
    return 0;
}
```

```
ÇIKTI
Mehmet için hesap açıldı
Zeynep için hesap açıldı
Mehmet: 500 TL
Zeynep: 0 TL
```

Yapıcıyı biz çağırmıyoruz; nesne tanımlanırken `{...}` (ya da `(...)`) içine yazılan değerler yapıcıya argüman olarak gider. Kitap boyunca Bölüm 22'de gördüğümüz süslü parantezi tercih edeceğiz: `int` parametreye `2.5` gibi veri kaybettiren bir değer verilirse derleyici itiraz eder. Zeynep'in hesabının `-20` ile açılmaya çalışıldığına, ama yapıcının bakiyeyi 0'a çektiğine dikkat edelim: asıl kazanç budur, hiçbir nesne geçersiz doğmaz.

Varsayılan yapıcı ve kaybolması

Hiç yapıcı yazmazsak derleyici **varsayılan yapıcıyı** (default constructor, parametresiz yapıcı) kendisi üretir. Bu yüzden `kapsulleme.cpp` içinde `Ogrenci o;` yazabildik. Tuzak şudur: kendimiz herhangi bir yapıcı yazdığımız anda derleyici bunu üretmeyi bırakır.

▸ varsayilanYok.cpp

```
class Kutu {
public:
    int en;
    Kutu(int e) { en = e; }       // kendi yapıcımızı yazdık
};

int main() {
    Kutu k;                       // parametresiz yapıcı artık yok
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI
varsayilanYok.cpp: In function 'int main()':
varsayilanYok.cpp:8:10: error: no matching function for call to 'Kutu::Kutu()'
    8 |     Kutu k;                       // parametresiz yapıcı artık yok
      |          ^
varsayilanYok.cpp:4:5: note: candidate: 'Kutu::Kutu(int)'
    4 |     Kutu(int e) { en = e; }       // kendi yapıcımızı yazdık
      |     ^~~~
varsayilanYok.cpp:4:5: note:   candidate expects 1 argument, 0 provided
```

Parametresiz nesne de gerekiyorsa ya ikinci bir yapıcı yazarız (yapıcılar da aşırı yüklenebilir, Bölüm 25'teki kurallar aynen geçerlidir) ya da `= default` ile "derleyicinin üreteceği hâli istiyorum" deriz:

▸ yapiciYukleme.cpp

```
#include <iostream>
using namespace std;
```

```
class Kutu {
private:
    int en;
    int boy;

public:
    Kutu(int e, int b) : en{e}, boy{b} {}      // asıl yapıcı
    Kutu(int kenar) : Kutu{kenar, kenar} {}    // kare kutu
    Kutu() : Kutu{1, 1} {}                     // varsayılan: 1x1

    int alan() const { return en * boy; }
};

int main() {
    cout << Kutu{}.alan() << " " << Kutu{5}.alan() << " ";
    cout << Kutu{3, 4}.alan() << '\n';
    return 0;
}
```

```
ÇIKTI
1 25 12
```

Bu örnekte iki yeni öğe var. `: en{e}, boy{b}` kısmını az sonra açıklayacağız. `Kutu(int kenar) : Kutu{kenar, kenar}` ise bir yapıcının işi başka bir yapıcıya devretmesidir (**devreden yapıcı**, delegating constructor): ortak iş tek yerde kalır, her yapıcıda tekrar yazılmaz. Gövdesi boş bir `Kutu() = default;` da yazabilirdik, ama o durumda `en` ve `boy` çöp değerle başlardı; bu yüzden 1x1 değerini veren devreden yapıcıyı seçtik.

> **▲ Sık Yapılan Hata · Parantezli boş nesne tanımı**
>
> `BankaHesabi h();` yazıp parametresiz bir nesne oluşturduğumuzu sanmayalım. C++ bunu "`h` adlı, parametresiz, `BankaHesabi` döndüren bir **fonksiyon bildirimi**" diye okur ve `h.bilgiYazdir()` anlamsız bir hata verir. Parantezsiz (`BankaHesabi h;`) ya da süslü parantezli (`BankaHesabi h{};`) yazmalıyız. Bu tuzağa "en can sıkıcı ayrıştırma" (most vexing parse) denir.

Üye ilk değer listesi

Yapıcıların `:` ile başlayan kısmı **üye ilk değer listesidir** (member initializer list). Yapıcı gövdesinde `sahip = yeniSahip;` yazmak bir **atamadır**: üye önce boş olarak doğar, sonra değer atanır. Listede yazmak ise üyeyi doğduğu anda istenen değerle oluşturur. `int` için fark ihmal edilebilir, ama bazı üyeler için liste **zorunludur**:

- `const` üyeler ve referans üyeler: doğduktan sonra atanamazlar.
- Varsayılan yapıcısı olmayan sınıf tipli üyeler: yapıcıya argüman vermek gerekir.

`const int no;` üyesi olan bir sınıfta yapıcı gövdesine `no = n;` yazarsak derleme hatası alırız, çünkü atama değiştirmeye çalışmaktır. Doğrusu `Ogrenci(int n) : no{n} {}` yazmaktır. Genel kural olarak tüm üyeleri listede başlatalım; gövdeyi doğrulama gibi ek işlere ayıralım. `BankaHesabi` yapıcısını bu biçimde yeniden yazarsak şöyle olur:

```
BankaHesabi(string yeniSahip, double ilkBakiye)
    : sahip{yeniSahip}, bakiye{ilkBakiye >= 0 ? ilkBakiye : 0} {}
```

> **▲ Sık Yapılan Hata · Başlatma sırası, listedeki sıra değildir**
>
> Üyeler listede yazdığımız sıraya göre değil, **sınıfta tanımlandıkları sıraya** göre başlatılır. Sınıfta `alt`, `ust`'tan önce tanımlıysa `Aralik(int a) : ust{a}, alt{ust - 5} {}` yanıltıcıdır: `alt` önce başlatıldığı için henüz değeri olmayan `ust`'u okur ve çöp değer alır. `g++` bu durumda `-Wreorder` uyarısı verir. Listeyi sınıftaki sırayla yazalım ve uyarıyı ciddiye alalım.

Üyeye sınıfın içinde de ilk değer verebiliriz: `kapsulleme.cpp` içindeki `double ortalama{0};` satırı böyleydi. Buna **sınıf içi üye ilk değeri** (default member initializer) denir. Yapıcının listesi o üyeye değer vermezse bu değer kullanılır; listede değer verilirse listedeki kazanır. Böylece hangi yapıcı çalışırsa çalışsın üye çöp değerle kalmaz. `Kutu` sınıfında `int en{1};` ve `int boy{1};` yazsaydık `Kutu() = default;` da güvenli olurdu.

30.5 Yıkıcı

Yapıcı nesne doğarken çalışır; **yıkıcı** (destructor) nesne yok edilirken otomatik çalışır.

> **Tanım: Yıkıcı**
>
> Adı `~` işaretiyle başlayan sınıf adı olan (`~BankaHesabi`), parametre almayan, dönüş tipi bulunmayan ve sınıfta en fazla bir tane olabilen üye fonksiyondur. Nesnenin ömrü bittiğinde otomatik çalışır.

Yıkıcının asıl görevi, nesnenin sahip olduğu kaynağı (bellek, dosya, bağlantı) geri vermektir. Bölüm 29'da RAII'yi anlatırken bu fikri görmüştük; yıkıcı o fikrin uygulandığı yerdir. Ne zaman çalıştığını görmek için içine mesaj yazan bir sınıf yeter. Aşağıdaki örnek aynı zamanda çağrı sırasının kurallarını gösteriyor:

▸ yikiciSirasi.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Iz {
private:
    string ad;

public:
    Iz(string yeniAd) : ad{yeniAd} { cout << "+" << ad << '\n'; }
    ~Iz() { cout << "-" << ad << '\n'; }
};

Iz genel{"genel"};                    // main'den önce doğar

int main() {
    cout << "main basladi\n";
    Iz a{"a"};
    {
        Iz b{"b"};
        Iz c{"c"};
        cout << "blok sonu\n";
    }
    Iz* p{new Iz{"p"}};               // öbekte nesne
    cout << "delete oncesi\n";
    delete p;                         // yıkıcıyı çalıştırır
    Iz d{"d"};
```

```
    cout << "main bitiyor\n";
    return 0;
}
```

```
ÇIKTI
+genel
main basladi
+a
+b
+c
blok sonu
-c
-b
+p
delete oncesi
-p
+d
main bitiyor
-d
-a
-genel
```

Kuralları çıktıdan okuyabiliriz:

- **Yerel nesneler** tanımlandıkları sırayla doğar, kapsam bittiğinde **ters sırayla** yok edilir (`c` , sonra `b` ; `main` bitince `d` , sonra `a`). Bölüm 29'daki RAII sırası budur.
- `new` ile oluşturulan nesne kapsam bitince yok olmaz; yıkıcı yalnızca `delete` yazdığımız anda çalışır. `delete` unutulursa yıkıcı hiç çalışmaz ve bellek sızar. Akıllı işaretçilerin değeri buradadır: yıkıcıları `delete` işini bizim yerimize yapar.
- **Global nesne** `main` 'den önce doğar, `main` bittikten sonra en son yok edilir.

Üye nesnelerde sıra bir bina gibidir: temelden kurulur, çatıdan yıkılır.

uyeSirasi.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Iz {
    string ad;
    Iz(string a) : ad{a} { cout << "+" << ad << '\n'; }
    ~Iz() { cout << "-" << ad << '\n'; }
};

struct Kutu {
    Iz ic;                              // üye nesne
    Kutu() : ic{"ic"} { cout << "+kutu\n"; }
    ~Kutu() { cout << "-kutu\n"; }
};

int main() {
    Kutu k;
    return 0;
}
```

```
ÇIKTI
+ic
+kutu
-kutu
-ic
```

Yapıcıda önce üyeler oluşur, sonra dıştaki sınıfın gövdesi çalışır (içten dışa). Yıkımda tersi olur: önce dıştaki sınıfın yıkıcı gövdesi, sonra üyelerin yıkıcıları çalışır (dıştan içe).

Yıkıcıyı, temizlik gerektiren sınıflarda yazarız. `new[]` ile bellek alan bir sınıfı "Kopyalama" başlığında yazacağız; yapıcı belleği alacak, yıkıcı geri verecek, sınıfı kullanan kod hiç `delete[]` yazmayacak. Üyeleri `vector`, `string` ve `unique_ptr` gibi RAII sınıflarıyla tutan bir sınıfta ise yıkıcıyı hiç yazmamız gerekmez; üyelerin yıkıcıları işi kendiliğinden yapar.

30.6 this: Nesnenin Kendisi

`a.bilgiYazdir()` diye çağrıldığında, fonksiyonun içindeki `no` nasıl `a`'nın `no`'su oluyor? Derleyici her `static` olmayan üye fonksiyona gizli bir parametre ekler: fonksiyonun çağrıldığı nesneyi gösteren işaretçi, yani **`this`**. Tipi `Sinif*`'dır (`const` üye fonksiyonda `const Sinif*`). `no` yazmak aslında `this->no` yazmaktır. Çoğu zaman `this`'i yazmamız gerekmez; gerektiği iki yer vardır.

1. Parametre adı üye adıyla aynıysa. İç kapsamdaki ad dıştakini gölgeler (Bölüm 22). Parametresi `x` olan bir `xAyarla(int x)` fonksiyonunda üyeye `this->x = x;` ile yazarız (sol taraf üye, sağ taraf parametre). Yalnızca `x = x;` yazsaydık parametreyi kendisine atardık ve üye değişmezdi. Bu yüzden kitapta çoğunlukla farklı adlar seçiyoruz (`yeniSahip`, `ilkBakiye`).

2. Nesnenin kendisini döndürmek. `*this`, çağrıldığı nesnenin kendisidir. Üye fonksiyon `Sinif&` döndürüp `return *this;` derse çağrılar zincirlenir. `cout << a << b` ifadesinin arkasındaki fikir de budur (Bölüm 32).

▸ thisZincir.cpp

```cpp
#include <iostream>
using namespace std;

class Sayac {
public:
    int sayi{0};

    Sayac& artirRef() { sayi++; return *this; }     // nesnenin kendisi
    Sayac artirDeger() { sayi++; return *this; }    // nesnenin kopyası
    Sayac& ekle(int n) { sayi += n; return *this; }
};

int main() {
    Sayac s;
    s.artirRef().artirRef().ekle(10).artirRef();    // soldan sağa
    cout << s.sayi << '\n';

    Sayac t;
    t.artirDeger().artirDeger().artirDeger();
    cout << t.sayi << '\n';
    return 0;
}
```

```
ÇIKTI
13
1
```

`s` üzerindeki zincirde her çağrı `s`'nin kendisini geri verir, bir sonraki çağrı yine `s` üzerinde çalışır (1 + 1 + 10 + 1 = 13). `artirDeger` ise `Sayac` döndürdüğü için `return *this;` nesnenin **kopyasını** üretir; ikinci ve üçüncü çağrılar kopyaya uygulanır ve `t` yalnızca bir kez artar.

> **▲ Sık Yapılan Hata · Zincirde & unutmak**
>
> Dönüş tipini `Sayac&` yerine `Sayac` yazarsak derleyici ses çıkarmaz, ama zincir sessizce yanlış sonuç verir. Zincirleme için dönüş tipi daima `Sinif&` olmalıdır.

30.7 const Üye Fonksiyonlar

Daha önce gördüğümüz `double ortalamaAl() const` ve `void bilgiYazdir() const` satırlarındaki parantezden sonraki `const`, derleyiciye verilen bir sözdür: "bu fonksiyon nesnenin hiçbir üyesini değiştirmez." Söz tutulmazsa derleyici hata verir. Asıl faydası şuradadır: `const` bir nesne ya da `const &` parametre üzerinden (Bölüm 25 ve 28) yalnızca `const` üye fonksiyonlar çağrılabilir.

▸ constHata.cpp

```cpp
#include <iostream>
using namespace std;

class Dikdortgen {
private:
    int en{3};
    int boy{4};

public:
    int alan() const { return en * boy; }
    void buyut(int kat) { en *= kat; boy *= kat; }   // const değil
};

void incele(const Dikdortgen& d) {
    cout << d.alan() << '\n';         // tamam: alan() const
    d.buyut(2);                       // hata: buyut() const değil
}

int main() {
    Dikdortgen d;
    incele(d);
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI
constHata.cpp: In function 'void incele(const Dikdortgen&)':
constHata.cpp:16:12: error: passing 'const Dikdortgen' as 'this' argument discards qualifiers [-
fpermissive]
   16 |     d.buyut(2);                       // hata: buyut() const değil
      |     ~~~~~~~~^~
constHata.cpp:11:10: note:   in call to 'void Dikdortgen::buyut(int)'
   11 |     void buyut(int kat) { en *= kat; boy *= kat; }   // const değil
      |          ^~~~
```

Mesajın özü şudur: `const` bir nesne üzerinden `const` olmayan bir fonksiyon çağırdık (`discards qualifiers`: "nitelikleri atıyor"). Kural iki yönlüdür:

	`const` üye fonksiyon	`const` olmayan üye fonksiyon
`const` nesne ya da `const &` üzerinden çağrılır mı?	Evet	Hayır
Üyeleri değiştirebilir mi?	Hayır	Evet
Normal nesne üzerinden çağrılır mı?	Evet	Evet

> **◆ Kritik Nokta · const doğruluğu**
>
> Nesneyi değiştirmeyen her üye fonksiyonu `const` yazalım (getter'lar, `alan()`, `bilgiYazdir()`). Aksi hâlde sonradan `const &` parametre alan bir fonksiyondan o üye fonksiyonu çağırmak gerektiğinde `const` eksikliği bizi zorlar. Buna **const doğruluğu** (const correctness) denir. `const` yazılmış bir fonksiyonun içinde üyeye atama yapmak da derleme hatasıdır.

Aynı ada sahip `const` ve `const` olmayan iki sürüm yazmak da mümkündür (aşırı yükleme); `const` nesne için `const` olan sürüm seçilir. `std::vector`'ün `operator[]` işlevi bunun bilinen bir örneğidir.

30.8 static Üyeler

Oluşturulan her `Ogrenci` nesnesine sıra numarası vermek için nesneleri sayan bir sayaç istediğimizi düşünelim. Sayaç her öğrencinin değil, **sınıfın** malı olmalıdır: tek bir tane olmalı ve tüm nesneler onu paylaşmalıdır. Üye değişkenin başına `static` yazarsak nesne başına bir kopya yerine tüm sınıf için tek kopya oluşur. `static` üye fonksiyonun `this`'i yoktur; nesne olmadan `Sinif::fonksiyon()` biçiminde çağrılır ve yalnızca `static` üyelere erişebilir.

staticUye.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Ogrenci {
private:
    string ad;
    int no;
    static int sonNo;                 // sınıf içinde yalnızca BİLDİRİM

public:
    Ogrenci(string yeniAd) : ad{yeniAd}, no{++sonNo} {}
    void bilgiYazdir() const { cout << no << " - " << ad << '\n'; }
    static int toplamAl() { return sonNo; }          // this yok
};

int Ogrenci::sonNo{0};                // TANIM: sınıf dışında, bir kez

int main() {
    cout << "Baslangic: " << Ogrenci::toplamAl() << '\n';
    Ogrenci a{"Mehmet"};
    Ogrenci b{"Elif"};
    Ogrenci c{"Can"};
    a.bilgiYazdir();
    c.bilgiYazdir();
```

```
    cout << "Toplam: " << Ogrenci::toplamAl() << " " << b.toplamAl() << '\n';
    return 0;
}
```

```
ÇIKTI
Baslangic: 0
1 - Mehmet
3 - Can
Toplam: 3 3
```

- `sonNo` üç nesne için tek değişkendir; her yapıcı onu bir artırıp sonucu `no`'ya yazar. Numaralar 1, 2, 3 olur.
- `static` veri üyesinin sınıf içindeki satırı yalnızca bildirimdir. Belleği ayıran **tanım**, sınıfın dışındaki `int Ogrenci::sonNo{0};` satırıdır ve programda bir kez yazılır. `::` kapsam çözümleme operatörüdür (Bölüm 22); burada "`Ogrenci` sınıfının `sonNo`'su" anlamına gelir.
- Çağrı `Ogrenci::toplamAl()` biçiminde hiç nesne olmadan yapılabilir (ilk satır). `b.toplamAl()` de çalışır, ama sınıf adıyla yazmak okuyana fonksiyonun `static` olduğunu söyler.

C++17'den beri `inline static int sonNo{0};` yazarsak sınıf içinde hem bildirim hem tanım olur ve dıştaki satır gerekmez. Burada `inline`, bu tanımı bir başlık dosyasına yazdığımızda birden çok `.cpp` içinde tekrar tanım hatası oluşmasını önler; şimdilik kalıp olarak alabiliriz.

> **▲ Sık Yapılan Hata · static üyenin tanımını unutmak**
>
> Sınıf içinde `static int sonNo;` yazıp dıştaki `int Ogrenci::sonNo{0};` satırını unutursak derleme geçer, **bağlama** aşamasında hata alırız (Bölüm 4). Mesaj `undefined reference to 'Ogrenci::sonNo'` biçimindedir ve satır numarası vermez. Tanımı başlık dosyasına değil `.cpp` dosyasına yazmalıyız; başlıkta yazarsak o başlığı çağıran her dosyada tanım tekrarlanır (`multiple definition`).

30.9 Nesneleri vector'de Tutmak ve Fonksiyonlara Geçirmek

Sınıf yazmanın pratik bir kazancı da Bölüm 26'daki paralel `vector`'lerden kurtulmaktır: ürünün adı, fiyatı ve adedi için üç ayrı `vector` yerine tek bir `vector<Urun>` yeter. Nesne `vector`'ünü `int` `vector`'ü gibi kullanırız. Eleman eklemek için `push_back` kullanabiliriz; yapıcı argümanlarını doğrudan veren `emplace_back` ise nesneyi `vector`'ün içinde, yerinde oluşturur.

▸ nesneVector.cpp

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;

class Urun {
private:
    string ad;
    double fiyat;
    int adet;

public:
    Urun(string yeniAd, double yeniFiyat, int yeniAdet)
        : ad{yeniAd}, fiyat{yeniFiyat}, adet{yeniAdet} {}

    double deger() const { return fiyat * adet; }
    void zamYap(double yuzde) { fiyat += fiyat * yuzde / 100; }
```

```
};

double toplamDeger(const vector<Urun>& liste) {
    double toplam{0};
    for (const Urun& u : liste) {
        toplam += u.deger();
    }
    return toplam;
}

int main() {
    vector<Urun> liste;
    liste.push_back(Urun{"Defter", 25.0, 10});
    liste.emplace_back("Kalem", 7.5, 40);
    liste.emplace_back("Silgi", 4.0, 25);

    cout << fixed << setprecision(2);
    cout << "Toplam: " << toplamDeger(liste) << '\n';

    for (auto u : liste) {            // u, elemanın KOPYASI
        u.zamYap(10);
    }
    cout << "Kopyaya zam: " << toplamDeger(liste) << '\n';

    for (auto& u : liste) {           // u, elemanın kendisi
        u.zamYap(10);
    }
    cout << "Referansa zam: " << toplamDeger(liste) << '\n';
    return 0;
}
```

```
ÇIKTI
Toplam: 650.00
Kopyaya zam: 650.00
Referansa zam: 715.00
```

İlk zam döngüsü sonucu hiç değiştirmedi: `for (auto u : liste)` her elemanın bir **kopyasını** `u` yapar, `zamYap` kopyaya uygulanır ve döngü bitince kopya atılır. Elemanı değiştireceksek `auto&`, yalnızca okuyacaksak `const auto&` yazmalıyız. Bölüm 26'daki aralık tabanlı `for` kuralı nesnelerde de aynen geçerlidir.

Ham dizi de olur (`Urun u[3];`), ama dizideki her eleman için varsayılan yapıcı çağrılır; sınıfta yoksa derlenmez. `vector<Urun> v(3);` da aynı sebepten varsayılan yapıcı ister. `Urun` gibi varsayılan yapıcısı olmayan sınıflarda elemanları tek tek `emplace_back` ile ekleriz.

Nesneleri fonksiyona geçirirken Bölüm 25'teki kuralı sınıflara uygularız:

Parametre	Anlamı	Ne zaman
`const Hesap& h`	Kopya yok, değiştirme yasak	Nesne yalnızca okunacaksa
`Hesap& h`	Kopya yok, değiştirme serbest	Fonksiyon nesneyi değiştirecekse
`Hesap h`	Nesnenin kopyası	Fonksiyon kopya üzerinde çalışacaksa

Değerle geçirme ağır olabilir (içinde `string` ya da `vector` bulunabilir) ve fonksiyonun yaptığı değişiklik çağıranın nesnesine ulaşmaz:

nesneGecirme.cpp

```cpp
#include <iostream>
using namespace std;

class Hesap {
private:
    double bakiye;

public:
    Hesap(double ilk) : bakiye{ilk} {}
    void yatir(double miktar) { bakiye += miktar; }
    double bakiyeAl() const { return bakiye; }
};

void kopyayaYatir(Hesap h) { h.yatir(100); }        // h bir kopya
void gercegeYatir(Hesap& h) { h.yatir(100); }       // h asıl nesne
void goster(const Hesap& h) { cout << h.bakiyeAl() << '\n'; }

int main() {
    Hesap a{500};
    kopyayaYatir(a);
    goster(a);
    gercegeYatir(a);
    goster(a);
    return 0;
}
```

ÇIKTI

```
500
600
```

Sınıf nesnelerini `==` ile karşılaştıramayız; bunun için operatör aşırı yükleme gerekir (Bölüm 32).

30.10 Kopyalama: Kopya Yapıcı ve Kopya Atama

Bir nesne `Hesap b{a};` ya da `Hesap b = a;` biçiminde başka bir nesneden oluşturulursa, ya da `b = a;` ile mevcut bir nesneye atanırsa, nesnenin **kopyası** alınır. Hiçbir şey yazmazsak derleyici iki özel üye fonksiyon üretir ve bunlar üyeleri tek tek kopyalar:

Özel üye fonksiyon	Ne zaman çalışır	Örnek
Kopya yapıcı (copy constructor)	Nesne, mevcut bir nesneden yaratılırken; değerle geçirme ve değerle döndürmede	`Hesap b{a};` `Hesap b = a;` `f(a)`
Kopya atama (copy assignment)	Zaten var olan nesneye atama yapılırken	`b = a;`

`BankaHesabi`, `Urun` gibi sınıflarda bu davranış tam istediğimiz gibidir: `string` ve `double` üyeler doğru biçimde kopyalanır. Sorun, sınıfın **işaretçi** üye taşıdığı ve o işaretçinin gösterdiği belleğe sahip olduğu durumda ortaya çıkar. Derleyicinin ürettiği kopya işaretçinin **değerini**, yani adresi kopyalar; gösterdiği veriyi değil.

```
Sığ kopya (derleyicinin ürettiği):

   a.veri ---+
             +--->  [10][ 0][ 0]       tek blok, iki sahip
   b.veri ---+
```

```
Derin kopya (bizim yazacağımız):

   a.veri --------->  [10][ 0][ 0]
   b.veri --------->  [10][ 0][ 0]     iki ayrı blok
```

Sığ kopyada `a` ve `b` aynı bloğu paylaşır. Birini değiştirmek ötekini de değiştirir, daha kötüsü ikisinin yıkıcısı da aynı bloğa `delete[]` uygular. Bu bir **çifte silmedir** (double free) ve tanımsız davranıştır (Bölüm 13). Bu yüzden ilk bakışta "çalışıyormuş gibi" görünen böyle bir programın çıktısını kitapta göstermeyeceğiz.

> **Tanım: Sığ kopya ve derin kopya**
>
> **Sığ kopya** (shallow copy), üyelerin değerlerini olduğu gibi kopyalar; işaretçi üyelerde yalnızca adres çoğalır. **Derin kopya** (deep copy), işaretçinin gösterdiği veriyi de yeni bir bellek bloğuna kopyalar; her nesne kendi kaynağının tek sahibi olur.

Çözüm, kopya yapıcıyı ve kopya atamayı kendimiz yazmaktır. Aşağıdaki sınıf `new[]` ile bir `int` dizisi tutuyor:

derinKopya.cpp

```cpp
#include <iostream>
using namespace std;

class Dizi {
private:
    int* veri;
    int n;

public:
    Dizi(int boyut) : veri{new int[boyut]{}}, n{boyut} {}

    Dizi(const Dizi& diger)                     // kopya yapıcı
        : veri{new int[diger.n]}, n{diger.n} {
        for (int i{0}; i < n; i++) {
            veri[i] = diger.veri[i];            // elemanları kopyala
        }
        cout << "kopya yapici\n";
    }

    Dizi& operator=(const Dizi& diger) {        // kopya atama
        cout << "kopya atama\n";
        if (this != &diger) {                   // öz-atamayı ele
            int* yeni{new int[diger.n]};
            for (int i{0}; i < diger.n; i++) {
                yeni[i] = diger.veri[i];
            }
            delete[] veri;                      // eski bloğu geri ver
            veri = yeni;
            n = diger.n;
        }
        return *this;
    }

    ~Dizi() {
        delete[] veri;
        cout << "yikici\n";
    }

    void ayarla(int i, int deger) { veri[i] = deger; }
    int al(int i) const { return veri[i]; }
    int boyut() const { return n; }
};
```

```
int main() {
    Dizi a{3};
    a.ayarla(0, 10);

    Dizi b{a};                      // kopya yapıcı
    b.ayarla(0, 99);                // a etkilenmez
    cout << a.al(0) << " " << b.al(0) << '\n';

    Dizi c{1};
    c = a;                          // kopya atama
    c = c;                          // öz-atama güvenli
    cout << c.boyut() << " " << c.al(0) << '\n';
    return 0;
}
```

```
ÇIKTI
kopya yapici
10 99
kopya atama
kopya atama
3 10
yikici
yikici
yikici
```

Kopya yapıcının parametresi `const Dizi&` olmak zorundadır; değerle alsaydı bu parametreyi oluşturmak için yine kopya yapıcı çağrılır ve sonsuz özyineleme başlardı. Kopya atamada `operator=` yazımı bir **operatör aşırı yüklemedir**; ayrıntısını Bölüm 32'de göreceğiz, şimdilik kalıp olarak alabiliriz. Üç noktaya dikkat edelim:

- **Kopya yapıcı** yeni, henüz bir kaynağı olmayan nesneyi kurar; eski bir şeyi temizlemesi gerekmez.
- **Kopya atama** nesnenin önceden bir kaynağı olduğunu varsayar. Önce yeni bloğu alıp kopyalıyor, sonra eskisini siliyoruz; böylece `new` bellek yetersizliği nedeniyle başarısız olursa nesne bozulmadan kalır.
- `this != &diger` denetimi **öz-atamayı** (`c = c;`) korur. Denetim olmasaydı ve eski blok önce silinseydi, kopyalanacak veri kaybolurdu.

Üçler kuralı

> **◆ Kritik Nokta · Üçler kuralı**
>
> Bir sınıf için **yıkıcı**, **kopya yapıcı** ve **kopya atama** işlevlerinden birini kendimiz yazmak gerekiyorsa, büyük olasılıkla üçünü de yazmamız gerekir (rule of three). Yıkıcı yazmamızın nedeni sınıfın bir kaynağa sahip olmasıdır; kaynağa sahip bir sınıfın derleyicinin ürettiği sığ kopyası ise neredeyse her zaman yanlıştır.

Üçler kuralı C++11 ile gelen taşıma işlemlerini kapsamaz; onlarla birlikte **beşler kuralı** (rule of five) ortaya çıkar ve Bölüm 39'da anlatılacaktır. Daha iyi bir yol ise **sıfır kuralıdır**: kaynağı elle yönetmek yerine `vector`, `string` ve `unique_ptr` gibi RAII sınıflarıyla tutarsak özel üye fonksiyonların hiçbirini yazmamız gerekmez, çünkü üyeler kendilerini doğru kopyalar. `Dizi` sınıfını `vector<int> veri;` ile yazsaydık yukarıdaki üç işlevin tamamı gereksiz olurdu. Bu bölümde elle yazdık, çünkü bir `vector`'ün içinde olup bitenleri görmek istedik.

Bazı nesnelerin kopyalanmasını hiç istemeyebiliriz (dosya tutamacı, ağ bağlantısı). Bu durumda ilgili işlevi `= delete` ile silmek, yanlış kullanımı derleme aşamasında yakalar:

```
class Baglanti {
public:
    Baglanti() = default;
    Baglanti(const Baglanti&) = delete;            // kopyalama yasak
    Baglanti& operator=(const Baglanti&) = delete;
};
```

`Baglanti b2{b1};` yazıldığında derleyici "silinmiş işlevi kullanma" hatası verir. `unique_ptr`'in kopyalanamaması da aynı mekanizmayla sağlanır (Bölüm 29).

30.11 Sınıfı .hpp ve .cpp Dosyalarına Bölmek

Gerçek projelerde sınıf `main`'in üstüne yazılmaz. Bölüm 15'teki düzenin aynısı geçerlidir: sınıfın **bildirimi** (hangi üyeler var) bir başlık dosyasına (`.hpp` ya da `.h`), üye fonksiyonların **tanımları** bir kaynak dosyasına (`.cpp`) yazılır. `main` yalnızca başlığı `#include` eder. Sınıfı kullananlar gövdeleri görmek zorunda kalmaz ve bir gövde değişince yalnızca o `.cpp` yeniden derlenir.

Üç dosyalı bir örnek; her bir bloğun ilk satırı dosya adıdır:

```
// hesap.hpp
#ifndef HESAP_HPP
#define HESAP_HPP

#include <string>

class Hesap {
private:
    std::string sahip;
    double bakiye;
    static int adet;

public:
    Hesap(const std::string& yeniSahip, double ilkBakiye);
    void yatir(double miktar);
    double bakiyeAl() const;
    static int adetAl();
};

#endif
```

```
// hesap.cpp
#include "hesap.hpp"

int Hesap::adet{0};                    // static üyenin tanımı

Hesap::Hesap(const std::string& yeniSahip, double ilkBakiye)
    : sahip{yeniSahip}, bakiye{ilkBakiye} {
    adet++;
}

void Hesap::yatir(double miktar) {
    if (miktar > 0) {
        bakiye += miktar;
    }
}

double Hesap::bakiyeAl() const {       // const burada da yazılır
```

```
    return bakiye;
}

int Hesap::adetAl() {                 // static yazılmaz
    return adet;
}
```

```
// main.cpp
#include <iostream>
#include "hesap.hpp"

int main() {
    Hesap a{"Mehmet", 100};
    Hesap b{"Elif", 40};
    a.yatir(50);
    std::cout << a.bakiyeAl() << " " << Hesap::adetAl() << '\n';
    return 0;
}
```

Üç dosyayı birlikte derleriz. Derleyici her `.cpp` dosyasını ayrı bir nesne dosyasına çevirir, bağlayıcı (linker) bunları birleştirir (Bölüm 4 ve 15):

```
TERMİNAL
$ g++ -std=c++17 -Wall -Wextra main.cpp hesap.cpp -o hesapProgrami
$ ./hesapProgrami
150 2
```

Çıktı, `100 + 50 = 150` bakiye ve iki açılmış hesabı gösterir. Bu düzende dikkat edilecekler şunlardır:

- **Koruma satırları** (`#ifndef` / `#define` / `#endif`) başlığın bir derleme biriminde yalnızca bir kez işlenmesini sağlar. Aksi hâlde aynı başlık iki yoldan gelirse sınıf iki kez tanımlanır ve `redefinition` hatası çıkar. `#pragma once` da aynı işi görür (Bölüm 15).
- Başlık dosyasına `using namespace std;` **yazılmaz**; başlığı çağıran herkesin dosyasına bu satırı sokmuş oluruz (Bölüm 22). Bu yüzden başlıkta `std::string` yazdık. `.cpp` dosyasında istersek kullanabiliriz.
- Tanımda `const` bildirimdeki gibi **tekrarlanır** (`double Hesap::bakiyeAl() const`), `static` ise **tekrarlanmaz**. Aksi hâlde `no declaration matches` türünden hatalar alırız.
- Kendi başlığımızı tırnakla (`"hesap.hpp"`), standart başlıkları açılı parantezle (`<string>`) çağırırız.
- Tanımda `Hesap::` önekini unutursak yazdığımız şey sınıfın üyesi değil, sıradan bir fonksiyon olur. Tanımı hiç verilmemiş bir üye fonksiyon ya da derleme komutuna eklenmeyen bir `.cpp` ise bağlama hatasıdır: `undefined reference to 'Hesap::yatir(double)'`.

Kısa üye fonksiyonları sınıfın içinde tanımlamak (`double alan() const { return en * boy; }`) örtük olarak `inline` yapar; bu yüzden başlıkta gövdesiyle yazılabilirler. Uzun gövdeleri `.cpp` dosyasına koymak daha okunaklıdır.

30.12 struct ile class ve friend

C++'ta `struct` ile `class` arasındaki **tek dil farkı varsayılan erişimdir**: `struct` üyeleri varsayılan olarak `public`, `class` üyeleri `private` olur. Yapıcı, yıkıcı, üye fonksiyon, kalıtım ikisinde de vardır. Alışkanlık olarak yalnızca veri taşıyan basit paketler için `struct`, davranışı ve korunan verisi olan tipler için `class` kullanılır. Bu kitapta da öyle yapıyoruz (bkz. `uyeSirasi.cpp` içindeki küçük `struct`'lar).

Bu fark C#'taki `struct` ile `class` farkıyla (değer tipi ve referans tipi) karıştırılmamalıdır. C++'ta ikisi de değer gibi davranır.

Bazen sınıfın dışındaki bir fonksiyonun `private` üyelere erişmesi gerekir. **`friend`** (arkadaş) bildirimi bunu sınıfın kendi kararıyla mümkün kılar:

▸ friendOrnek.cpp

```cpp
#include <cmath>
#include <iostream>
using namespace std;

class Nokta {
private:
    double x;
    double y;

public:
    Nokta(double yeniX, double yeniY) : x{yeniX}, y{yeniY} {}
    friend double uzaklik(const Nokta& a, const Nokta& b);   // arkadaş
};

double uzaklik(const Nokta& a, const Nokta& b) {
    double dx{a.x - b.x};             // private üyelere erişir
    double dy{a.y - b.y};
    return sqrt(dx * dx + dy * dy);
}

int main() {
    Nokta p{0, 0};
    Nokta q{3, 4};
    cout << uzaklik(p, q) << '\n';
    return 0;
}
```

```
ÇIKTI
5
```

`uzaklik` bir üye fonksiyon değildir; `p.uzaklik(q)` diye çağrılmaz. `friend` yalnızca erişim izni verir. Bu izin kapsüllemeyi delmek yerine, sınıfla sıkı çalışan işlevleri sınıfın **arayüzünün parçası** saymak içindir. Bölüm 32'de akış operatörlerini `friend` ile yazacağız. Gereksiz yere kullanılan `friend`, kapsüllemeyi zayıflatır; önce `public` bir üye fonksiyonla çözülüp çözülemediğine bakalım.

30.13 Hepsini Bir Araya Getirelim: Kesir Sınıfı

Bölümün özetini bir uygulamada toplayalım. `Kesir` sınıfı payı ve paydayı tutar; yapıcı kesri sadeleştirir, paydayı pozitif yapar ve sıfır paydaya izin vermez. Dört işlemden üçünü üye fonksiyon olarak yazıyoruz (`kesir1 + kesir2` biçiminde yazım Bölüm 32'nin konusudur).

▸ kesir.cpp

```cpp
#include <iostream>
#include <numeric>
using namespace std;

class Kesir {
private:
    int pay;
```

```
    int payda;

    void sadelestir() {
        if (payda < 0) {              // işaret her zaman payda
            pay = -pay;
            payda = -payda;
        }
        int ebob{gcd(pay, payda)};    // <numeric>, C++17
        if (ebob > 1) {
            pay /= ebob;
            payda /= ebob;
        }
    }

public:
    Kesir(int p = 0, int q = 1) : pay{p}, payda{q != 0 ? q : 1} {
        sadelestir();                 // payda 0 ise 1 yapılır
    }

    int payAl() const { return pay; }
    int paydaAl() const { return payda; }

    Kesir topla(const Kesir& d) const {
        return Kesir{pay * d.payda + d.pay * payda, payda * d.payda};
    }
    Kesir cikar(const Kesir& d) const {
        return Kesir{pay * d.payda - d.pay * payda, payda * d.payda};
    }
    Kesir carp(const Kesir& d) const {
        return Kesir{pay * d.pay, payda * d.payda};
    }

    double ondalik() const { return static_cast<double>(pay) / payda; }
    void yazdir() const {
        cout << pay;
        if (payda != 1) {
            cout << '/' << payda;
        }
    }
};

int main() {
    Kesir a{1, 2};
    Kesir b{3, 4};
    Kesir c{6, -8};                   // -3/4 olarak sadeleşir
    Kesir d{5};                       // 5/1

    cout << "a + b = "; a.topla(b).yazdir(); cout << '\n';
    cout << "a - b = "; a.cikar(b).yazdir(); cout << '\n';
    cout << "a * b = "; a.carp(b).yazdir(); cout << '\n';
    cout << "c = ";     c.yazdir();         cout << '\n';
    cout << "d = ";     d.yazdir();         cout << '\n';
    cout << "b ondalik = " << b.ondalik() << '\n';
    return 0;
}
```

```
ÇIKTI
a + b = 5/4
a - b = -1/4
a * b = 3/8
c = -3/4
d = 5
b ondalik = 0.75
```

Bu küçük sınıfta bölümün birçok fikri bir arada:

- Üye değişkenler `private`; dışarıya yalnızca `payAl()` gibi getter'lar ve işlemler açık.
- `sadelestir()` yalnızca sınıfın içinden çağrılan **özel yardımcı** bir üye fonksiyondur. Her yapıcı sonunda bunu çağırdığı için hiçbir `Kesir` nesnesi sadeleşmemiş ya da paydası sıfır biçimde doğmaz. Bu, sınıfın **değişmezi** (class invariant) olarak adlandırılan "geçerli durum" kuralıdır.
- Varsayılan argümanlı tek yapıcı (`p = 0, q = 1`) hem varsayılan yapıcının (`Kesir{}` → 0), hem `Kesir{5}` biçiminin görevini görür. Tek argümanla çağrılabilen yapıcılar `Kesir k = 5;` gibi örtük dönüşüme de izin verir; bunu istemiyorsak yapıcıyı `explicit` ile işaretleriz.
- Sıfır paydanın ne yapılacağı burada basitçe 1'e çekilerek geçiştirildi. Daha doğru çözüm hata bildiren bir **istisna** fırlatmaktır (Bölüm 34).
- İşlemler `const` ve parametreler `const Kesir&`; hiçbir kopya ve hiçbir yan etki yok.

Üç Dilde · Sınıflar üç dilde *C · C++ · C#*

C'de `struct` ve serbest fonksiyonlar vardır; kapsülleme, yapıcı ve yıkıcı yoktur, doğrulamayı programcı unutmamalıdır. **C++**'ta `class` bunların hepsini sağlar ve yıkıcı sayesinde kaynak yönetimi nesneye bağlanır. **C#**'ta da `class` vardır, ama iki önemli farkla: nesneler öbekte yaşar ve değişkenler yalnızca referans tutar; bellek çöp toplayıcıyla (GC) otomatik temizlenir.

▸ Hesap.cs

```
var h = new Hesap("Mehmet", 500);
h.Yatir(250);
h.Ortalama = 250;                    // set bloğundaki denetim bunu reddeder
Console.WriteLine($"{h.Sahip}: {h.Bakiye} TL, ortalama {h.Ortalama}");

class Hesap
{
    private double bakiye;
    private double ortalama = 50;

    public Hesap(string sahip, double ilk)
    {
        Sahip = sahip;
        bakiye = ilk >= 0 ? ilk : 0;
    }

    public string Sahip { get; }              // yalnızca okunur property
    public double Bakiye => bakiye;           // hesaplanan property

    public double Ortalama                    // doğrulamalı property
    {
        get => ortalama;
        set { if (value >= 0 && value <= 100) ortalama = value; }
    }

    public void Yatir(double miktar)
    {
```

```
        if (miktar > 0) bakiye += miktar;
    }
}
```

```
ÇIKTI
Mehmet: 750 TL, ortalama 50
```

Konu	C++	C#
Varsayılan erişim	`class` : `private` , `struct` : `public`	Üyeler `private` (sınıfta), erişim her üyeye tek tek yazılır
Getter/setter	`adAl()` / `adAyarla()` üye fonksiyonları	`property` : `get` / `set` blokları, kullanımı alan gibi
Yıkıcı	`~Sinif()` : ömür bitince **kesinlikle** çalışır	Sonlandırıcı `~Sinif()` : GC ne zaman isterse çalışır; kaynak için `IDisposable` + `using` kullanılır
Kopyalama	`b = a` nesnenin kopyasını alır (kopya yapıcı)	`b = a` aynı nesneyi gösteren ikinci bir referans üretir
Üçler/beşler kuralı	Var	Yok; GC bellek sahipliğini çözer
`const` üye fonksiyon	`foo() const`	Yok; `readonly` alanlar vardır
Bölme	`.hpp` + `.cpp`	Tek `.cs` dosyası; bildirim ile tanım ayrılmaz

C#'taki bu farkları Bölüm 49 ve 52'de ayrıntısıyla işleyeceğiz.

Bölüm Özeti

- Sınıf; veri (üye değişken) ile davranışı (üye fonksiyon) tek tipte toplar. Sınıf şablon, nesne ondan üretilen örnektir. Üyelere `nesne.uye` ya da `isaretci->uye` ile ulaşılır.
- Kapsülleme: üye değişkenler `private`, işlemler `public` olmalıdır. Erişimi sınıf belirler, nesne değil. Setter'ın değeri doğrulama yapabilmesidir; her üyeye mekanik get/set yazılmaz.
- Yapıcının adı sınıfla aynıdır, dönüş tipi yoktur ve nesne doğarken çalışır. Kendi yapıcımızı yazarsak varsayılan yapıcı kaybolur; `= default` onu geri getirir. Yapıcılar aşırı yüklenebilir ve birbirine iş devredebilir.
- `BankaHesabi h();` nesne değil, fonksiyon bildirimidir; `h;` ya da `h{};` yazılmalıdır.
- Üye ilk değer listesi (`: ad{a}, no{n}`) `const` ve referans üyeler için zorunludur. Üyeler listedeki sıraya değil, sınıftaki tanım sırasına göre başlatılır.
- Yıkıcı (`~Sinif()`) nesnenin ömrü bitince çalışır. Yerel nesneler ters sırayla yok edilir; `new` ile açılan nesnede yıkıcı `delete` anında çalışır; global nesne `main`'den sonra yok edilir. Üye nesneler içten dışa kurulur, dıştan içe yıkılır.
- `this` çağrılan nesneyi gösteren işaretçidir. `return *this;` ile `Sinif&` döndürmek çağrı zinciri kurar; `Sinif` döndürmek kopya demektir.
- `const` üye fonksiyon üyeleri değiştirmez ve `const` nesne ile `const &` üzerinden çağrılabilir. Nesneyi değiştirmeyen her üye fonksiyonu `const` yazmalıyız.

- `static` üye, tüm nesneler için tek kopyadır. Veri üyesinin tanımı sınıf dışında bir kez yazılır (ya da C++17 `inline static`). `static` üye fonksiyonun `this`'i yoktur.
- Nesne parametresi: yalnızca okunacaksa `const Sinif&`, değiştirilecekse `Sinif&`; değerle geçirme kopya alır. `vector<Sinif>` içindeki elemanı değiştirmek için `auto&` kullanılır, `auto` kopya üzerinde çalışır.
- Derleyicinin ürettiği kopya sığdır. İşaretçi üyesi olan sınıfta çifte silme oluşur; kopya yapıcı ve kopya atama ile derin kopya yazmalıyız. Yıkıcı, kopya yapıcı ve kopya atama birlikte düşünülür (üçler kuralı); mümkünse `vector` ve `unique_ptr` kullanarak hiçbirini yazmamalıyız (sıfır kuralı).
- Kopyalanması istenmeyen sınıflarda ilgili işlev `= delete` ile silinir.
- Bildirim `.hpp`'ye (koruma satırları, `std::` önekli, `using namespace` yok), tanımlar `.cpp`'ye yazılır; tanımda `Sinif::` öneki ve `const` tekrarlanır, `static` tekrarlanmaz. Derleme: `g++ main.cpp hesap.cpp`.
- C++'ta `struct` ile `class` arasındaki tek dil farkı varsayılan erişimdir. `friend`, sınıfın dışındaki bir işleve `private` üyelere erişim izni verir.

Alıştırmalar

1. Aşağıdaki sınıf için hangi satır **derlenir**?

```
class Kutu {
public:
    int en;
    Kutu(int e) : en{e} {}
};
```

   A) `Kutu k;`
   B) `Kutu k{7};`
   C) `Kutu k{};`
   D) `Kutu dizi[3];`
   E) `Kutu k = Kutu();`

2. C++'ta `struct` ile `class` arasındaki dil farkı hangisidir?

   A) `struct` yapıcı içeremez
   B) `struct` nesneleri yığında, `class` nesneleri öbekte oluşur
   C) Üyelerin varsayılan erişimi: `struct` için `public`, `class` için `private`
   D) `class` kalıtıma izin verir, `struct` vermez
   E) `struct` değer tipi, `class` referans tipidir

3. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
using namespace std;
class Sayac {
public:
    static int adet;
    Sayac() { adet++; }
    ~Sayac() { adet--; }
};
int Sayac::adet{0};
int main() {
```

```
    Sayac a;
    {
        Sayac b;
        Sayac c;
    }
    Sayac d;
    Sayac e;
    cout << Sayac::adet;
    return 0;
}
```

A) 1
B) 2
C) 3
D) 4
E) 5

4. Aşağıdaki sınıf için hangi satır **derleme hatası** verir?

```
class Sicaklik {
private:
    double derece{0};
public:
    void ayarla(double d) { derece = d; }
    double al() const { return derece; }
};
void f(const Sicaklik& s, Sicaklik& t) {
    // aşağıdaki şıklardan biri buraya yazılıyor
}
```

A) `s.ayarla(5);`
B) `t.ayarla(5);`
C) `cout << s.al();`
D) `cout << t.al();`
E) `const Sicaklik& r{t}; cout << r.al();`

5. İşaretçi üyesini `new[]` ile dolduran ama kopya yapıcısı yazılmamış bir sınıftan `b{a}` ile kopya alınırsa ne olur?

A) Derleme hatası oluşur
B) Derin kopya yapılır
C) `a` ve `b` aynı bellek bloğunu gösterir; iki yıkıcı da bloğu silmeye çalışır
D) Kopya yapılmaz, `b` boş kalır
E) Yıkıcı hiç çalışmaz

6. Aşağıdaki sınıfta `xAyarla(10)` çağrıldığında ne olur?

```
class Nokta {
private:
    int x{0};
public:
    void xAyarla(int x) { x = x; }
    int xAl() const { return x; }
};
```

A) `x` 10 olur
B) Parametre kendisine atanır, üye `x` değişmez
C) Derleme hatası oluşur
D) Çalışma zamanı hatası oluşur
E) Üye `x` 0 yerine çöp değer alır

7. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>
#include <string>
using namespace std;
struct Parca {
    string ad;
    Parca(string a) : ad{a} { cout << "P+" << ad << '\n'; }
    ~Parca() { cout << "P-" << ad << '\n'; }
};
struct Makine {
    Parca m1;
    Parca m2;
    Makine() : m2{"ikinci"}, m1{"birinci"} { cout << "M+\n"; }
    ~Makine() { cout << "M-\n"; }
};
int main() {
    cout << "basla\n";
    {
        Makine k;
    }
    cout << "bitis\n";
    return 0;
}
```

8. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>
using namespace std;
class Sayac {
private:
    int sayi{0};
    static int toplam;
public:
    void artir() { sayi++; toplam++; }
    int sayiAl() const { return sayi; }
    static int toplamAl() { return toplam; }
};
int Sayac::toplam{0};
void kopyaArtir(Sayac s) { s.artir(); }
void gercekArtir(Sayac& s) { s.artir(); }
int main() {
    Sayac x;
    kopyaArtir(x);
    cout << x.sayiAl() << ' ';
    gercekArtir(x);
    gercekArtir(x);
    cout << x.sayiAl() << ' ';
    cout << Sayac::toplamAl() << '\n';
    return 0;
}
```

9. Aşağıdaki programın çıktısı nedir?

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;
class Oyuncu {
private:
    string ad;
    int puan;
public:
    Oyuncu(string a, int p) : ad{a}, puan{p} {}
    void puanEkle(int n) { puan += n; }
    int puanAl() const { return puan; }
};
int main() {
    vector<Oyuncu> takim;
    takim.emplace_back("Mehmet", 10);
    takim.emplace_back("Elif", 25);
    takim.emplace_back("Can", 15);
    for (auto& o : takim) {
        o.puanEkle(1);
    }
    for (auto o : takim) {
        o.puanEkle(100);
    }
    for (const Oyuncu& o : takim) {
        cout << o.puanAl() << ' ';
    }
    cout << '\n';
    return 0;
}
```

10. Aşağıdaki sınıfın yapıcısı neden derlenmez? Nasıl düzeltilir?

```cpp
class Ogrenci {
private:
    const int no;
    int sinif;
public:
    Ogrenci(int n, int s) {
        no = n;
        sinif = s;
    }
};
```

11. Aşağıdaki programda `h.bilgiYazdir()` satırı neden hata verir?

```cpp
#include <iostream>
using namespace std;
class Hesap {
public:
    void bilgiYazdir() const { cout << "hesap\n"; }
};
int main() {
    Hesap h();
    h.bilgiYazdir();
    return 0;
}
```

12. Bir sınıfın üye fonksiyonunu `.cpp` dosyasında tanımlarken şu hatalardan hangisi **bağlama** (linker) hatasına yol açar?

A) Tanımda parametre listesinin hatalı yazılması
B) `static` üye değişkenin sınıf dışındaki tanımının hiç yazılmaması
C) Üye değişkenin `private` olması
D) `const` üye fonksiyonun gövdesinde `return` bulunması
E) Başlıkta `using namespace std;` yazılması

13. **Program yazma.** `BankaHesabi` adlı bir sınıf yazın. `private` üyeleri `sahip` (`string`) ve `bakiye` (`double`) olsun. Yapıcı sahibi ve ilk bakiyeyi üye ilk değer listesiyle alsın; ilk bakiye negatifse 0 yapsın. `yatir(double)` yalnızca pozitif tutarı kabul etsin. `bool cek(double)`, tutar pozitif ve bakiyeyi aşmıyorsa parayı düşüp `true`, aksi hâlde `false` döndürsün. `sahipAl()` ve `bakiyeAl()` `const` olsun. `main` klavyeden sahip adını (tek sözcük), ilk bakiyeyi, yatırılacak ve çekilecek tutarı okusun; çekme başarısızsa `Yetersiz bakiye.` yazsın, sonunda hesabı `Ad: bakiye TL` biçiminde (iki ondalıkla) yazdırsın.

```
Sahip, ilk bakiye, yatirilacak, cekilecek: Ayse 500 250 900
Yetersiz bakiye.
Ayse: 750.00 TL
```

14. **Program yazma.** `Dikdortgen` sınıfı yazın. Yapıcı `en` ve `boy` alsın; sıfır ya da negatif kenarı 1 yapsın. `alan()`, `cevre()` ve `kareMi()` `const` üye fonksiyon olsun. `main` önce `n` sayısını, sonra `n` tane `en boy` çiftini okuyup `vector<Dikdortgen>` içinde saklasın; her dikdörtgenin alanını ve çevresini, sonda en büyük alanı ve kare sayısını yazdırsın.

15. **Program yazma.** `Ogrenci` sınıfı yazın: `private` üyeleri ad, öğrenci numarası ve `vector<double>` tipinde notlar olsun. Numara, `static` bir sayaçtan otomatik atansın (ilk öğrenci 1). `notEkle(double)` yalnızca 0–100 aralığındaki notu kabul etsin (kabul ettiyse `true` döndürsün). `ortalama()` notlar boşsa 0 döndürsün. `static int sayiAl()` toplam öğrenci sayısını versin. `main` üç öğrenci oluşturup notlar eklesin ve `numara ad ortalama` biçiminde (iki ondalık) yazdırsın.

16. **Program yazma (zor).** `OlcumDizisi` adlı bir sınıf yazın: `new double[]` ile `n` elemanlık dizi tutsun. Yıkıcıyı, kopya yapıcıyı ve kopya atamayı derin kopya yapacak biçimde yazın. `main` içinde bir nesne oluşturup kopyalayın, kopyayı değiştirin ve asıl nesnenin değişmediğini gösterin.

BÖLÜM 31

Kalıtım ve Polimorfizm

BU BÖLÜMDE

- Kalıtımın (inheritance) ne işe yaradığını, "bir mi, sahip mi?" sorusuyla ne zaman kullanılacağını
- `public`, `protected` ve `private` erişimin türetilmiş sınıfta nasıl çalıştığını
- Yapıcıların ve yıkıcıların çağrı sırasını; taban sınıf yapıcısına argüman geçirmeyi
- Fonksiyon gizleme ile ezme (override) arasındaki farkı; `virtual`, `override` ve `final` anahtar sözcüklerini
- Taban işaretçi ya da referans üzerinden çalışan çalışma zamanı polimorfizmini ve sanal tabloyu
- Saf sanal fonksiyonu, soyut sınıfı ve arayüz benzeri sınıfları
- Sanal yıkıcının zorunluluğunu, nesne dilimleme tuzağını, çoklu kalıtımı ve elmas problemini

Bölüm 30'da veriyi ve o veriyle çalışan fonksiyonları tek pakette toplayan sınıfları yazdık. Gerçek programlarda ise birbirine çok benzeyen sınıflar çıkar: bir `Daire` de bir `Dikdortgen` de bir şekildir, bir `Mudur` de bir `Muhendis` de bir çalışandır. Ortak kısmı her sınıfa kopyalamak hem tekrar demektir hem de bir hatayı birden fazla yerde düzeltmeyi gerektirir.

Bu bölümde iki fikir öğreneceğiz. **Kalıtım**, ortak kısmı bir kez yazıp başka sınıfların onu devralmasını sağlar. **Polimorfizm** (çok biçimlilik) ise farklı türden nesneleri tek bir tür gibi ele alıp her birinin kendi davranışını göstermesini sağlar. İkisi birlikte nesne yönelimli programlamanın asıl gücünü oluşturur ve C'de karşılığı olmayan, C++'ın C'ye en belirgin eklerinden biridir.

31.1 Kalıtım Nedir?

Bir hayvan barınağı programı düşünelim. `Kedi` sınıfında ad, `uyu()` ve `miyavla()` var; `Kopek` sınıfında ad, `uyu()` ve `havla()` var. Ad ve `uyu()` iki sınıfta aynıdır. Üçüncü bir hayvan eklediğimizde aynı kodu üçüncü kez kopyalamamız gerekir.

> **Tanım: Kalıtım (inheritance)**
>
> Bir sınıfın başka bir sınıfın üyelerini devralmasıdır. Devralınan sınıfa **taban sınıf** (base class), devralan sınıfa **türetilmiş sınıf** (derived class) denir. Türetilmiş sınıf, taban sınıfın bütün üyelerine sahiptir ve üzerine kendi üyelerini ekler.

Kalıtımı ne zaman kullanacağımızı tek bir test belirler: "X bir Y'dir" cümlesi doğru mu? Kedi bir hayvandır, öyleyse `Kedi`, `Hayvan`'dan türetilebilir. Araba bir motor değildir, motora sahiptir; orada kalıtım değil, bir **üye değişken** kullanırız.

```
class Kedi : public Hayvan { /* ... */ };   // Kedi bir Hayvan'dır (is-a)

class Araba {                               // Araba bir Motor'a sahiptir (has-a)
    Motor motor;
};
```

> **İpucu · Kalıtımdan önce "bir mi, sahip mi?" diye sorun**
>
> Yalnızca birkaç satır kod tekrar etmemek için kalıtım kurmayın. Kalıtım iki sınıfı sıkı bağlar: taban sınıfı değiştirdiğimizde bütün türetilmiş sınıflar etkilenir. İlişki "sahip" ise üye değişken, "bir" ise kalıtım kullanılır.

C'de de buna benzer bir şey yapılabilir: bir `struct`'ı ötekinin ilk üyesi olarak yerleştirir (Bölüm 14), `k.temel.ad` biçiminde erişiriz. Ancak bu yalnızca verinin yeniden kullanımıdır; davranışın devralınması, erişim denetimi ve aşağıda göreceğimiz çok biçimlilik dilin parçası değildir. C++'ta bunların hepsini derleyici yönetir.

İlk örneğimizde taban sınıf `Hayvan`, türetilmiş sınıflar `Kedi` ve `Kopek` olsun:

▸ kalitimIlk.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Hayvan {
public:
    Hayvan(string adi) : ad{adi} {}
    void tanit() const { cout << ad << " bir hayvan.\n"; }
    void uyu() const { cout << ad << " uyuyor.\n"; }
protected:
    string ad;
};

class Kedi : public Hayvan {
public:
    Kedi(string adi) : Hayvan{adi} {}
    void miyavla() const { cout << ad << ": Miyav!\n"; }
};

class Kopek : public Hayvan {
public:
    Kopek(string adi) : Hayvan{adi} {}
    void havla() const { cout << ad << ": Hav hav!\n"; }
};

int main() {
    Kedi kedi{"Pamuk"};
    Kopek kopek{"Boncuk"};
    kedi.tanit();
    kedi.miyavla();
    kopek.uyu();
    kopek.havla();
    return 0;
}
```

```
ÇIKTI
Pamuk bir hayvan.
Pamuk: Miyav!
Boncuk uyuyor.
Boncuk: Hav hav!
```

`class Kedi : public Hayvan` ifadesi "`Kedi`, `Hayvan`'dan türetilmiştir" demektir. `Kedi` içinde `tanit()` ve `uyu()` yazmadık, ama kedi nesnesi bunları çağırabilir. `miyavla()` ise `Hayvan`'dan devralınan `ad` üyesini kullanıyor. `Hayvan{adi}` ifadesi taban sınıfın yapıcısının çağrılmasıdır; bunu birazdan ayrıntılı göreceğiz. Yeni bir `Kus` sınıfı eklemek için `class Kus : public Hayvan` yazıp yalnızca kuşa özgü üyeleri tanımlamak yeter.

31.2 Erişim Denetimi: public, protected, private

Bölüm 30'da `public` ve `private` erişim belirteçlerini (access specifier) gördük. Kalıtımla birlikte üçüncüsü devreye girer: `protected` (korumalı). Korumalı bir üyeye sınıfın dışından erişilemez, ama türetilmiş sınıflar erişebilir.

Üyenin erişimi	Kendi sınıfı	Türetilmiş sınıf	Sınıfın dışı (`main` vb.)
`public`	evet	evet	evet
`protected`	evet	evet	hayır
`private`	evet	hayır	hayır

Dikkat edelim: türetilmiş sınıf, taban sınıfın `private` üyelerini de devralır, yani nesnenin içinde dururlar; ama bu üyelere doğrudan erişemez. Taban sınıfın `public` ya da `protected` fonksiyonları üzerinden dolaylı yoldan ulaşabilir.

Üç erişim düzeyini tek programda deneyelim. Program bilerek hatalıdır:

▸ erisimHata.cpp

```cpp
#include <iostream>

class Hayvan {
public:
    Hayvan() : kilo{4}, yas{3} {}
protected:
    int kilo;
private:
    int yas;
};

class Kedi : public Hayvan {
public:
    int yasVer() const { return yas; }
    int kiloVer() const { return kilo; }
};

int main() {
    Kedi kedi;
    std::cout << kedi.kiloVer() << '\n';
    std::cout << kedi.kilo << '\n';
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

erisimHata.cpp: In member function 'int Kedi::yasVer() const':
erisimHata.cpp:14:33: error: 'int Hayvan::yas' is private within this context
   14 |     int yasVer() const { return yas; }
      |                                 ^~~
erisimHata.cpp:9:9: note: declared private here
    9 |     int yas;
      |         ^~~
erisimHata.cpp: In function 'int main()':
erisimHata.cpp:21:23: error: 'int Hayvan::kilo' is protected within this context
   21 |     std::cout << kedi.kilo << '\n';
      |                       ^~~~
erisimHata.cpp:7:9: note: declared protected here
    7 |     int kilo;
      |         ^~~~
```

İki hata var. `yas`, `Hayvan`'ın `private` üyesi olduğu için `Kedi` içinden okunamıyor. `kilo` ise `protected` olduğundan `Kedi`'nin içinde serbesttir (`kiloVer` derleniyor), ama `main` içinde `kedi.kilo` yazılamaz.

> **◆ Kritik Nokta · Veri üyelerini mümkünse private tutun**
>
> Kapsülleme ilkesi (Bölüm 30) kalıtımda da geçerlidir. `protected` veri üyesi, türetilmiş sınıfı taban sınıfın iç yapısına bağlar; taban sınıfın veri düzenini değiştirmek bütün alt sınıfları kırar. İyi tasarımda veri `private` kalır, türetilmiş sınıfların ihtiyacı `protected` fonksiyonlarla karşılanır. Bu bölümde örnekleri kısa tutmak için yer yer `protected` veri üyesi kullanacağız.

Kalıtımın türü: neden public?

`class Kedi : public Hayvan` satırındaki `public`, **kalıtımın türünü** belirler. C++'ta `public`, `protected` ve `private` kalıtım vardır. Türe göre taban sınıfın üyeleri türetilmiş sınıfta şöyle görünür:

Kalıtım türü	Taban `public` üye	Taban `protected` üye
`public`	`public` kalır	`protected` kalır
`protected`	`protected` olur	`protected` olur
`private`	`private` olur	`private` olur

Taban sınıfın `private` üyeleri her durumda türetilmiş sınıfa erişilemez kalır. Kitapta yalnızca `public` kalıtımı kullanacağız, çünkü "Kedi bir hayvandır" ilişkisini ancak o kurar. `protected` ve `private` kalıtım "bir"in değil, "içinde kullanır"ın ifadesidir ve pratikte nadirdir; bu ilişki için üye değişken çoğu zaman daha açıktır.

> **▲ Sık Yapılan Hata · public yazmayı unutmak**
>
> `class` ile tanımlanan sınıfta kalıtım türünü yazmazsak varsayılan `private`'tır: `class Kedi : Hayvan` yazarsak `kedi.tanit()` bile dışarıdan çağrılamaz ve bir `Kedi`'yi `Hayvan` işaretçisine atayamayız. `struct`'ta varsayılan `public`'tir. Çok biçimliliği kullanacağımız her yerde `public` kalıtım yazmayı alışkanlık edinelim.

31.3 Yapıcı ve Yıkıcı Sırası

Türetilmiş bir nesnenin içinde bir de taban sınıf parçası vardır. Bu parçanın da yapıcısı çalışmalıdır; çünkü `ad` gibi üyelerin ilk değerini o verir. Kural şudur:

1. Önce taban sınıfın yapıcısı çalışır (çok basamaklı kalıtımda en üstteki sınıftan başlayarak).
2. Sonra türetilmiş sınıfın üye nesnelerinin yapıcıları, **sınıf içinde tanımlandıkları sırayla** çalışır.
3. En son türetilmiş sınıfın yapıcı gövdesi çalışır.

Yıkıcılar tam ters sırada çalışır: önce türetilmiş sınıfın yıkıcı gövdesi, sonra üye nesneler, en son taban sınıf. Bir evi inşa ederken önce temeli atarız; yıkarken temeli en son bırakırız.

▸ yapiciSirasi.cpp

```cpp
#include <iostream>
using namespace std;

class Motor {
public:
    Motor()  { cout << "  Motor yapıcı\n"; }
    ~Motor() { cout << "  Motor yıkıcı\n"; }
};

class Arac {
public:
    Arac()  { cout << "  Arac yapıcı\n"; }
    ~Arac() { cout << "  Arac yıkıcı\n"; }
};

class Araba : public Arac {
public:
    Araba()  { cout << "  Araba yapıcı\n"; }
    ~Araba() { cout << "  Araba yıkıcı\n"; }
private:
    Motor motor;
};

int main() {
    cout << "bloğa giriliyor\n";
    {
        Araba araba;
        cout << "  blok içinde\n";
    }
    cout << "blok bitti\n";
    return 0;
}
```

```
ÇIKTI
bloğa giriliyor
  Arac yapıcı
  Motor yapıcı
  Araba yapıcı
  blok içinde
  Araba yıkıcı
  Motor yıkıcı
  Arac yıkıcı
blok bitti
```

`Araba` yapıcısına hiçbir şey yazmadığımız hâlde önce `Arac` yapıcısı çalıştı: derleyici taban sınıfın varsayılan yapıcısını kendiliğinden çağırır. Sonra üye nesne `motor` yaratıldı, en son `Araba`'nın kendi gövdesi çalıştı. Blok bitince sıra tersine döndü; süslü parantez, `araba` nesnesinin ömrünü sınırlıyor (Bölüm 30'daki otomatik ömür).

> ◆ **Kritik Nokta · Yapıcılar tabandan türetilmişe, yıkıcılar türetilmişten tabana**
>
> Üye nesneler, taban sınıf yapıcısından sonra ve kendi gövdeden önce yaratılır; ters sırada yok edilir. Üyelerin sırasını başlatma listesindeki yazım sırası değil, **sınıf içindeki tanım sırası** belirler.

Taban sınıf yapıcısına argüman geçirmek

Taban sınıfın yapıcısı parametre alıyorsa türetilmiş sınıf, bu argümanları başlatma listesinde (member initializer list; Bölüm 30) kendisi vermelidir. Daha önce yazdığımız `Hayvan{adi}` tam olarak buydu.

▸ tabanYapici.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Tasit {
public:
    Tasit(string plakaNo) : plaka{plakaNo} {
        cout << "Tasit: " << plaka << '\n';
    }
    void yazdir() const { cout << "Plaka: " << plaka << '\n'; }
private:
    string plaka;
};

class Otobus : public Tasit {
public:
    Otobus(string plakaNo, int koltuk)
        : Tasit{plakaNo}, koltukSayisi{koltuk} {
        cout << "Otobus: " << koltukSayisi << " koltuk\n";
    }
    void koltukYazdir() const { cout << koltukSayisi << " koltuk\n"; }
private:
    int koltukSayisi;
};

int main() {
    Otobus otobus{"34 ABC 123", 45};
    otobus.yazdir();
    otobus.koltukYazdir();
    return 0;
}
```

ÇIKTI

```
Tasit: 34 ABC 123
Otobus: 45 koltuk
Plaka: 34 ABC 123
45 koltuk
```

`Otobus` yapıcısı plakayı `Tasit{plakaNo}` ile yukarı iletti. Çıktıda `Tasit` satırının önce geldiğine dikkat edelim. `plaka`, `Tasit`'in `private` üyesidir; `Otobus` ona doğrudan dokunamaz, ama `yazdir()` aracılığıyla ulaşılabilir.

> **▲ Sık Yapılan Hata · Taban sınıfın yapıcısını çağırmayı unutmak**
>
> Taban sınıfta varsayılan yapıcı yoksa (her yapıcı parametre istiyorsa) türetilmiş sınıf, başlatma listesinde taban yapıcıyı açıkça çağırmak zorundadır. `Otobus(int koltuk) : koltukSayisi{koltuk} {}` yazsaydık derleyici `Tasit` için varsayılan yapıcıyı arar, bulamaz ve "no matching function for call to 'Tasit::Tasit()'" benzeri bir hata verirdi. Çözüm: `: Tasit{"?"}, koltukSayisi{koltuk}` gibi uygun bir çağrı yazmak.

Yapıcılar ve atama işleçleri kalıtımla **devralınmaz**; her sınıf kendininkini yazar. Taban sınıfın yapıcılarını olduğu gibi kullanmak istersek C++11'den beri `using Tasit::Tasit;` yazabiliriz, ama başlangıçta açık yapıcılar yazmak yeterlidir.

31.4 Fonksiyon Gizleme

Türetilmiş sınıf, taban sınıftan gelen bir fonksiyonu kendi sürümüyle değiştirmek isteyebilir. Kedinin `sesCikar()` fonksiyonu "..." değil "Miyav" yazsın. Bunun için türetilmiş sınıfta aynı adı taşıyan bir fonksiyon yazarız. Taban sınıftaki fonksiyon `virtual` değilse bu, **gizleme** (hiding) olur: türetilmiş sürüm, taban sürümünün önünü kapatır.

▸ gizleme.cpp

```cpp
#include <iostream>
using namespace std;

class Hayvan {
public:
    void sesCikar() const { cout << "...\n"; }
};

class Kedi : public Hayvan {
public:
    void sesCikar() const { cout << "Miyav\n"; }
};

int main() {
    Kedi kedi;
    kedi.sesCikar();
    kedi.Hayvan::sesCikar();

    Hayvan* hayvan = &kedi;
    hayvan->sesCikar();
    return 0;
}
```

```
ÇIKTI
Miyav
...
...
```

Üç çağrı, iki ayrı sonuç verdi:

- `kedi.sesCikar()`: `kedi` değişkeninin tipi `Kedi` olduğundan `Kedi`'nin sürümü çalıştı.
- `kedi.Hayvan::sesCikar()`: Başa `Hayvan::` yazarak taban sürümünü açıkça çağırdık. Türetilmiş sürüm taban davranışını genişletmek isterse de aynı kalıbı kullanır: `Hayvan::sesCikar();`.

- `hayvan->sesCikar()`: `hayvan` bir `Hayvan*` işaretçisidir ve gösterdiği nesne bir `Kedi`'dir. Buna rağmen `Hayvan`'ın sürümü çalıştı; çünkü fonksiyon `virtual` değil ve derleyici sürümü işaretçinin tipine bakarak, program çalışmadan önce seçti.

Üçüncü satır önemlidir. Taban sınıf işaretçisi türetilmiş nesneyi gösterebilir (bir kedi bir hayvandır); buna **yukarı dönüşüm** (upcasting) denir ve açık dönüştürme gerektirmez. Ama `virtual` olmadan çağrı, işaretçinin tipine göre bağlanır. Biz tersini istiyoruz: nesne kediyse, işaretçi `Hayvan*` olsa bile kedi miyavlasın.

Gizleme bütün aşırı yüklemeleri kapatır

Gizleme ada göre çalışır, imzaya göre değil. Türetilmiş sınıfta `f` adlı **herhangi** bir fonksiyon tanımlanırsa, taban sınıftaki bütün `f` aşırı yüklemeleri (Bölüm 25) türetilmiş sınıfın kapsamında görünmez olur. Taban sürümlerini geri getirmek için `using` bildirimi kullanırız:

▸ gizlemeUsing.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Taban {
public:
    void yaz(int sayi) const { cout << "int: " << sayi << '\n'; }
};

class Turemis : public Taban {
public:
    using Taban::yaz;              // taban aşırı yüklemelerini görünür kıl
    void yaz(const string& s) const { cout << "string: " << s << '\n'; }
};

int main() {
    Turemis t;
    t.yaz(5);
    t.yaz("merhaba");
    return 0;
}
```

ÇIKTI

```
int: 5
string: merhaba
```

`using Taban::yaz;` satırını kaldırırsak `t.yaz(5)` derlenmez: `Turemis` içinde yalnızca `string` alan sürüm görünür.

31.5 virtual, override ve Dinamik Bağlama

Gizleme çoğu zaman istediğimiz şey değildir. Taban sınıftaki fonksiyonun başına `virtual` yazarsak o fonksiyon **sanal fonksiyon** olur. Türetilmiş sınıf onu aynı imzayla yeniden yazdığında (**ezme**, override) çağrıyı hangi sürümün karşılayacağına derleyici değil, program çalışırken nesnenin gerçek tipi karar verir.

Tanım: Dinamik ve statik bağlama

Dinamik bağlama (dynamic binding, geç bağlama), bir çağrının hangi fonksiyona gideceğinin program çalışırken nesnenin gerçek tipine bakılarak seçilmesidir. **Statik bağlama** (static binding), bu seçimin derleme zamanında ifadenin tipine göre yapılmasıdır. Sanal fonksiyonlar dinamik, diğerleri statik bağlanır.

Tanım: Çok biçimlilik (polymorphism)

Aynı kodun (aynı çağrı satırının) nesnenin gerçek tipine göre farklı davranmasıdır. C++'ta çalışma zamanı çok biçimliliği, sanal fonksiyonlar ve taban sınıf işaretçisi ya da referansıyla sağlanır.

dinamikBaglama.cpp

```cpp
#include <iostream>
using namespace std;

class Hayvan {
public:
    virtual void sesCikar() const { cout << "...\n"; }
};

class Kedi : public Hayvan {
public:
    void sesCikar() const override { cout << "Miyav\n"; }
};

class Kopek : public Hayvan {
public:
    void sesCikar() const override { cout << "Hav hav\n"; }
};

void konustur(const Hayvan& hayvan) {
    hayvan.sesCikar();
}

int main() {
    Hayvan hayvan;
    Kedi kedi;
    Kopek kopek;
    konustur(hayvan);
    konustur(kedi);
    konustur(kopek);

    Hayvan* isaretci = &kopek;
    isaretci->sesCikar();
    isaretci = &kedi;
    isaretci->sesCikar();
    return 0;
}
```

ÇIKTI

```
...
Miyav
Hav hav
Hav hav
Miyav
```

`konustur` fonksiyonunun parametresi `const Hayvan&` (Bölüm 25). Ona üç ayrı nesne verdik ve her seferinde o nesnenin kendi `sesCikar()` sürümü çalıştı. Benzer biçimde `isaretci->sesCikar()` satırı hiç değişmediği

hâlde, işaretçi önce köpeği sonra kediyi gösterdiği için iki farklı sonuç verdi. `konustur`, yarın yazacağımız `Kus` sınıfıyla da tek satır değişmeden çalışır.

İki ayrıntı daha var:

- Parantezden sonraki `const`, fonksiyonun nesneyi değiştirmeyeceğini söyler (Bölüm 30) ve **imzanın parçasıdır**; ezerken aynen yazılmalıdır.
- `override`, derleyiciye "bu fonksiyon taban sınıftaki bir sanal fonksiyonu eziyor, kontrol et" der. Neden vazgeçilmez olduğunu aşağıda göreceğiz.

Statik tip ve dinamik tip

`Hayvan* isaretci = &kopek;` satırında aynı nesne için iki ayrı tip vardır:

Kavram	Örnekte	Neyi belirler?
Statik tip (ifadenin tanımlı tipi)	`Hayvan*`	Hangi üyelerin çağrılabileceğini, derleme zamanında
Dinamik tip (nesnenin gerçek tipi)	`Kopek`	Sanal bir fonksiyonun hangi sürümünün çalışacağını, çalışma zamanında

Derleyici yalnızca statik tipe bakar. `Kedi`'ye `tirmala()` eklesek bile `Hayvan*` üzerinden `hayvan->tirmala()` yazamayız; "`Hayvan` sınıfında `tirmala` adlı üye yok" hatası alırız. Çözüm, fonksiyonu `Hayvan`'a da (sanal olarak) eklemek ya da nesneyi `Kedi` tipiyle kullanmaktır.

> ◆ **Kritik Nokta · Dinamik bağlamanın üç şartı**
>
> Bir çağrının dinamik bağlanması için üçü birlikte gerekir. Biri eksikse çağrı statik bağlanır, yani taban sınıfın sürümü çalışır:
>
> 1. Fonksiyon taban sınıfta `virtual` olarak tanımlanmış olmalı.
> 2. Türetilmiş sınıf onu aynı imzayla (aynı ad, aynı parametreler, aynı `const`) ezmiş olmalı.
> 3. Çağrı bir **işaretçi ya da referans** üzerinden yapılmalı. Nesnenin kendisi (değer) üzerinden yapılan çağrı hep statik bağlanır.

override neden yazılmalı?

`override` zorunlu değildir; aynı imzayı yazmak ezmek için yeter. Ama imzayı yanlış yazarsak (örneğin `const`'u unutursak) derleyici bunu yeni ve ayrı bir fonksiyon sayar. Ezme gerçekleşmez, taban sürümü çalışmaya devam eder ve hata mesajı gelmez. `override` yazarsak derleyici "ezdiğin bir şey yok" diye hata verir:

▸ overrideHata.cpp

```cpp
#include <iostream>
using namespace std;

class Hayvan {
public:
    virtual void sesCikar() const { cout << "...\n"; }
};

class Kedi : public Hayvan {
public:
    void sesCikar() override { cout << "Miyav\n"; }
};

int main() {
```

```
    Kedi kedi;
    kedi.sesCikar();
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

overrideHata.cpp:11:10: error: 'void Kedi::sesCikar()' marked 'override', but does not override
   11 |     void sesCikar() override { cout << "Miyav\n"; }
      |          ^~~~~~~~
```

`Hayvan`'ın fonksiyonu `const`, `Kedi`'ninki değildi; iki ayrı imza. `override` sayesinde hata derleme sırasında yakalandı. Kural basittir: bir sanal fonksiyonu ezerken her zaman `override` yazarız. Türetilmiş sınıfta ayrıca `virtual` yazmak gerekmez, fonksiyon zaten sanaldır.

> **▲ Sık Yapılan Hata · virtual yazmayı unutmak**
>
> Taban sınıfta `virtual` yazmazsak, türetilmiş sınıftaki aynı imzalı fonksiyon taban fonksiyonu yalnızca gizler. `Hayvan*` ya da `Hayvan&` üzerinden çağrıda hep `Hayvan`'ın sürümü çalışır. Program derlenir, hata vermez, yanlış çıktı üretir. Ezilmesini istediğimiz her fonksiyonun taban sınıfta `virtual` olduğundan emin olalım.

final

Bir sanal fonksiyonu ya da sınıfı "burada dur" diye kilitlemek için `final` yazılır. Bir sınıfın `final` olması ondan türetmeyi, bir fonksiyonun `final` olması daha fazla ezilmesini yasaklar:

```
class Kare final : public Sekil { /* ... */ };
class BuyukKare : public Kare { };            // HATA: Kare final

class Daire : public Sekil {
public:
    double alan() const final { return 3.14 * r * r; }   // artık ezilemez
private:
    double r{1};
};
```

31.6 Perde Arkası: Sanal Tablo

Dinamik bağlama sihir değildir; derleyici bunu küçük bir düzenekle gerçekleştirir. Sanal fonksiyonu olan her sınıf için bir **sanal tablo** (virtual table, vtable) hazırlanır: sınıfın sanal fonksiyonlarının adreslerini tutan bir dizi. Bu sınıftan yaratılan her nesneye de gizli bir **sanal tablo işaretçisi** (vptr) eklenir ve işaretçi, nesnenin gerçek sınıfının tablosunu gösterir.

```
  Kedi nesnesi             Kedi'nin vtable'ı
+--------------+          +----------------------+
| vptr  -------+--------> | sesCikar -> Kedi::sesCikar  |
| ad           |          +----------------------+
+--------------+

  Kopek nesnesi            Kopek'in vtable'ı
+--------------+          +----------------------+
| vptr  -------+--------> | sesCikar -> Kopek::sesCikar |
| ad           |          +----------------------+
```

`hayvan->sesCikar()` çağrısı için derleyici şu kodu üretir: nesnenin `vptr`'sini oku, tabloda `sesCikar`'ın sırasındaki adresi al, o adrese git. Taban sınıf işaretçisi hangi tipte olursa olsun nesnenin kendi `vptr`'si doğru tabloyu gösterdiği için doğru sürüm çalışır.

Bunun iki küçük bedeli vardır: her nesne bir işaretçi kadar büyür ve sanal çağrı bir adım dolaylıdır. Sanal fonksiyonu olmayan sınıf bu bedeli ödemez; Bölüm 21'deki "kullanmadığın şeyin bedelini ödemezsin" ilkesi budur. Boyuttaki farkı ölçelim:

▸ sanalBoyut.cpp

```cpp
#include <iostream>
using namespace std;

class Duz {
    int x;
public:
    void yaz() const {}
};

class Sanal {
    int x;
public:
    virtual void yaz() const {}
};

int main() {
    cout << "sanalsiz: " << sizeof(Duz) << '\n';
    cout << "sanalli : " << sizeof(Sanal) << '\n';
    return 0;
}
```

ÇIKTI

```
sanalsiz: 4
sanalli : 16
```

Bu çıktı 64 bitlik bir sistemde alındı: gizli işaretçi 8 bayt yer kaplar ve `int` ile hizalanması için araya dolgu girer (4 + 8 yerine 16). Kesin sayı derleyiciye ve platforma göre değişebilir; önemli olan, sanal fonksiyonun nesneye bir işaretçi eklediğidir.

C'de aynı düzeneği elle kurmak mümkündü: yapının içine fonksiyon işaretçileri (Bölüm 12) koyup her "nesne" için doldururduk.

```cpp
struct Hayvan {
    const char *ad;
    void (*sesCikar)(const struct Hayvan *);   // elle kurulmuş "vptr"
};
```

C++ bu işi derleyiciye yaptırır: işaretçileri doldurmayı, tabloyu paylaşmayı ve türetilmiş sınıfta değiştirmeyi dil güvence altına alır.

31.7 Saf Sanal Fonksiyon ve Soyut Sınıf

Bir `Sekil` sınıfı düşünelim. Şeklin alanı "sıfır" değil, **tanımsızdır**: yalnızca daire ya da dikdörtgen gibi somut bir şeklin alanı vardır. `Sekil sekil;` yazıp anlamsız bir nesne yaratmak hata olmalıdır. Bunu dile anlatmanın yolu soyut sınıflardır.

Tanım: Saf sanal fonksiyon ve soyut sınıf

Gövdesi olmayan, imzasının sonuna `= 0` yazılan sanal fonksiyona **saf sanal fonksiyon** (pure virtual function) denir. En az bir saf sanal fonksiyonu olan sınıf **soyut sınıf** (abstract class) olur: ondan nesne yaratılamaz, yalnızca türetilmek için vardır. Saf sanal fonksiyon "her somut türetilmiş sınıf bunu yazmak zorunda" demektir.

soyutSinif.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const double PI{3.14159265358979};

class Sekil {
public:
    virtual ~Sekil() = default;
    virtual double alan() const = 0;
    virtual double cevre() const = 0;
    void bilgi() const {
        cout << ad << ": alan=" << alan() << ", çevre=" << cevre() << '\n';
    }
protected:
    Sekil(string adi) : ad{adi} {}
private:
    string ad;
};

class Daire : public Sekil {
public:
    Daire(double yaricap) : Sekil{"Daire"}, r{yaricap} {}
    double alan() const override { return PI * r * r; }
    double cevre() const override { return 2 * PI * r; }
private:
    double r;
};

class Dikdortgen : public Sekil {
public:
    Dikdortgen(double en, double boy) : Sekil{"Dikdörtgen"}, e{en}, b{boy} {}
    double alan() const override { return e * b; }
    double cevre() const override { return 2 * (e + b); }
private:
    double e, b;
};

int main() {
    Daire daire{1.5};
    Dikdortgen dikdortgen{2, 5};
    daire.bilgi();
    dikdortgen.bilgi();

    const Sekil& s = dikdortgen;     // soyut sınıf referansı geçerlidir
    cout << "referanstan alan: " << s.alan() << '\n';
    return 0;
}
```

```
ÇIKTI
Daire: alan=7.06858, çevre=9.42478
Dikdörtgen: alan=10, çevre=14
referanstan alan: 10
```

`Sekil::bilgi()` sanal olmayan, gövdeli bir fonksiyondur. İçindeki `alan()` ve `cevre()` çağrıları aslında `this->alan()` demektir; `this` bir `Daire`'yi ya da `Dikdortgen`'i gösterdiği için doğru sürüm seçilir. Taban sınıfın kendi fonksiyonundan yapılan sanal çağrılar da dinamik bağlanır. Bu, "şablon yöntem" (template method) adıyla bilinen yaygın bir tasarım kalıbının temelidir: değişmeyen akışı tabanda yazarız, değişen adımları türetilmiş sınıflara bırakırız.

Soyut sınıfın kuralları şunlardır:

- Soyut sınıftan nesne yaratılamaz, ama soyut sınıf tipinde **işaretçi ve referans** tanımlanabilir (`Sekil*`, `const Sekil&`, `unique_ptr<Sekil>`). Çok biçimlilik zaten bunlar üzerinden çalışır.
- Soyut sınıfın üye değişkenleri, yapıcısı ve gövdeli fonksiyonları olabilir. Yapıcı türetilmiş sınıfın başlatma listesinden çağrılır; nesne yaratılamayacağı için çoğunlukla `protected` yazılır.
- Somut bir türetilmiş sınıf, tabandaki **bütün** saf sanal fonksiyonları ezmek zorundadır. Birini bile ezmeyen sınıf da soyut kalır.
- Saf sanal fonksiyonun gövdesi de olabilir (`Sekil::alan` için ayrıca tanım yazılabilir ve türetilmiş sınıftan `Sekil::alan()` ile çağrılır); fonksiyon yine de ezilmek zorundadır. Bu kullanım nadirdir.

Soyut sınıftan nesne yaratmaya çalışırsak derleyici şöyle yanıt verir:

▸ soyutHata.cpp

```cpp
#include <iostream>

class Sekil {
public:
    virtual double alan() const = 0;
};

class Kare : public Sekil {
public:
    Kare(double k) : kenar{k} {}
private:
    double kenar;
};

int main() {
    Kare kare{3};
    std::cout << kare.alan() << '\n';
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

soyutHata.cpp: In function 'int main()':
soyutHata.cpp:16:10: error: cannot declare variable 'kare' to be of abstract type 'Kare'
   16 |     Kare kare{3};
      |          ^~~~
soyutHata.cpp:8:7: note:   because the following virtual functions are pure within 'Kare':
    8 | class Kare : public Sekil {
      |       ^~~~
soyutHata.cpp:5:20: note:     'virtual double Sekil::alan() const'
    5 |     virtual double alan() const = 0;
      |                    ^~~~
```

`Kare`, `alan()`'ı ezmediği için soyut kalmıştır. Hata, sınıfın tanımlandığı satırda değil, onun bir nesnesinin yaratılmaya çalışıldığı satırda bildirilir. Ya eksik fonksiyonu yazarız ya da `Kare`'yi bilerek soyut bırakırız. İmza birebir tutmalıdır: `const`'u unuttuğumuz bir `alan()` saf sanalı ezmez.

Arayüz benzeri sınıflar

Hiç üye değişkeni olmayan ve yalnızca saf sanal fonksiyonlardan (bir de sanal yıkıcıdan) oluşan soyut sınıflar, başka dillerdeki **arayüz** (interface) kavramının C++'taki karşılığıdır. C++'ta ayrı bir `interface` anahtar sözcüğü yoktur; kural olarak böyle sınıflar bir davranış sözleşmesini tanımlar.

```
class Yazdirilabilir {
public:
    virtual ~Yazdirilabilir() = default;
    virtual void yazdir() const = 0;       // sözleşme: her uygulayıcı yazdırabilir
};
```

Bir sınıf birden fazla böyle arayüzü uygulayabilir; bunu çoklu kalıtım başlığında göreceğiz.

31.8 Sanal Yıkıcı Neden Gerekli?

Soyut `Sekil` sınıfında `virtual ~Sekil() = default;` satırı vardı. Nedenine gelelim. Çok biçimliliğin tipik kullanımı, türetilmiş nesneyi taban sınıf işaretçisiyle tutmaktır. Nesneyi silme zamanı geldiğinde de silme bu taban işaretçi üzerinden yapılır (`delete` ya da `unique_ptr`'nin yıkıcısı). Taban sınıfın yıkıcısı `virtual` değilse silme işlemi de statik bağlanır: yalnızca taban sınıfın yıkıcısı çalışır, türetilmiş sınıfın yıkıcısı hiç çağrılmaz. Standart bu durumu **tanımsız davranış** sayar.

```
class Kaynak {
public:
    ~Kaynak() { cout << "Kaynak temizlendi\n"; }       // virtual DEĞİL
};

class TamponluKaynak : public Kaynak {
    int* tampon = new int[1000];
public:
    ~TamponluKaynak() { delete[] tampon; }              // çağrılmayabilir
};

Kaynak* kaynak = new TamponluKaynak();
delete kaynak;      // tanımsız davranış; tampon sızabilir
```

Pratikte `TamponluKaynak` yıkıcısı çalışmaz ve `tampon` için ayrılan bellek geri verilmez: bellek sızıntısı (Bölüm 29). Üstelik program tanımsız davranışa girdiği için başka sonuçlar da olabilir; bu yüzden böyle bir kodun

çıktısını göstermiyoruz. Çözüm tek satırdır: taban sınıfın yıkıcısını `virtual` yapmak. Aynı programı düzeltip akıllı işaretçiyle çalıştıralım:

sanalYikici.cpp

```cpp
#include <iostream>
#include <memory>
using namespace std;

class Kaynak {
public:
    virtual ~Kaynak() { cout << "Kaynak temizlendi\n"; }
};

class TamponluKaynak : public Kaynak {
public:
    TamponluKaynak() : tampon{new int[1000]} {
        cout << "Tampon ayrıldı\n";
    }
    ~TamponluKaynak() override {
        delete[] tampon;
        cout << "Tampon serbest bırakıldı\n";
    }
private:
    int* tampon;
};

int main() {
    {
        unique_ptr<Kaynak> kaynak = make_unique<TamponluKaynak>();
        cout << "kaynak kullanılıyor\n";
    }
    cout << "blok bitti\n";
    return 0;
}
```

ÇIKTI

```
Tampon ayrıldı
kaynak kullanılıyor
Tampon serbest bırakıldı
Kaynak temizlendi
blok bitti
```

`unique_ptr<Kaynak>` yok edilirken içindeki `Kaynak*` üzerinden silme yapıldı. Yıkıcı sanal olduğu için önce `TamponluKaynak` yıkıcısı (tamponu geri verdi), sonra `Kaynak` yıkıcısı çalıştı; sıra, "yıkıcılar tersine" kuralıyla aynıdır.

> ◆ **Kritik Nokta · Sanal fonksiyonu olan sınıfın yıkıcısı da sanal olmalı**
>
> Bir sınıfta en az bir `virtual` fonksiyon varsa yıkıcısını da `virtual` yapalım. Böyle bir sınıf taban olmak için tasarlanmıştır ve nesneleri büyük olasılıkla taban işaretçisiyle silinecektir. Yıkıcı gövdesi boşsa `virtual ~Sinif() = default;` yazmak yeter. Taban yıkıcı sanalsa türetilmiş sınıfların yıkıcıları da otomatik sanaldır; `override` yazmak yine de serbesttir.

31.9 Çok Biçimli Koleksiyon

Çok biçimliliğin asıl işe yaradığı yer, farklı türden nesneleri tek bir koleksiyonda tutmaktır: içinde daire, dikdörtgen, üçgen olan bir `Sekil` listesi ve hangisinin ne olduğunu bilmeden hepsinin alanını toplayan bir döngü.

Burada küçük bir engel var. `vector<Sekil>` yazarsak her eleman gerçekten bir `Sekil` nesnesi olur; bir `Daire` koyduğumuzda `Daire`'ye ait kısım kesilip atılır (bir sonraki başlıkta görülecek). Üstelik `Sekil` soyutsa `vector<Sekil>` derlenmez. Çözüm, nesnelerin kendisini değil **onlara işaretçileri** tutmaktır. Nesneler öbekte (heap) yaratılır; ham işaretçi yerine Bölüm 29'daki `unique_ptr` kullanırsak `delete` yazmamız gerekmez, `vector` yok edilirken nesneler de silinir.

▸ sekilListesi.cpp

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

const double PI{3.14159265358979};

class Sekil {
public:
    virtual ~Sekil() = default;
    virtual double alan() const = 0;
    virtual string ad() const = 0;
};

class Daire : public Sekil {
public:
    Daire(double yaricap) : r{yaricap} {}
    double alan() const override { return PI * r * r; }
    string ad() const override { return "Daire"; }
private:
    double r;
};

class Dikdortgen : public Sekil {
public:
    Dikdortgen(double en, double boy) : e{en}, b{boy} {}
    double alan() const override { return e * b; }
    string ad() const override { return "Dikdörtgen"; }
private:
    double e, b;
};

class Ucgen : public Sekil {
public:
    Ucgen(double taban, double yukseklik) : t{taban}, y{yukseklik} {}
    double alan() const override { return t * y / 2; }
    string ad() const override { return "Üçgen"; }
private:
    double t, y;
};

int main() {
    vector<unique_ptr<Sekil>> sekiller;
    sekiller.push_back(make_unique<Daire>(1.5));
    sekiller.push_back(make_unique<Dikdortgen>(2, 5));
    sekiller.push_back(make_unique<Ucgen>(4, 3));
```

```
    double toplam{0};
    for (const auto& sekil : sekiller) {
        cout << sekil->ad() << ": " << sekil->alan() << '\n';
        toplam += sekil->alan();
    }
    cout << "Toplam alan: " << toplam << '\n';
    return 0;
}
```

```
ÇIKTI
Daire: 7.06858
Dikdörtgen: 10
Üçgen: 6
Toplam alan: 23.0686
```

Üç kalıp dikkat çeker:

- `make_unique<Daire>(1.5)` bir `unique_ptr<Daire>` üretir; `push_back` bunu `unique_ptr<Sekil>`'e kendiliğinden dönüştürür (yukarı dönüşümün akıllı işaretçi sürümü).
- Döngüde `for (const auto& sekil : sekiller)` yazarız. `unique_ptr` kopyalanamaz (nesnenin tek sahibi olmalıdır); `for (auto sekil : sekiller)` yazarsak derleyici hata verir.
- `sekil->alan()` çağrısı her elemanın gerçek tipine göre ilerler. Listeye yarın `Kare` eklesek döngüde tek karakter değiştirmemiz gerekmez.

31.10 Nesne Dilimleme Tuzağı

Taban sınıf tipinde bir **nesneye** (işaretçiye ya da referansa değil) türetilmiş bir nesne atarsak, türetilmiş sınıfa ait her şey dilimlenip atılır; geriye yalnızca taban parçası kalır. Buna **nesne dilimleme** (object slicing) denir. Aynı şey bir fonksiyona taban tipinde **değerle** geçirmede ve `vector<Hayvan>` içine türetilmiş nesne eklemede de olur.

▸ dilimleme.cpp

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Hayvan {
public:
    Hayvan(string adi) : ad{adi} {}
    virtual ~Hayvan() = default;
    virtual string ses() const { return "..."; }
    string adi() const { return ad; }
private:
    string ad;
};

class Kopek : public Hayvan {
public:
    Kopek(string adi) : Hayvan{adi} {}
    string ses() const override { return "Hav hav"; }
};

void degerIle(Hayvan hayvan) {
    cout << "değer ile    : " << hayvan.adi() << " " << hayvan.ses() << '\n';
```

```
}

void referansIle(const Hayvan& hayvan) {
    cout << "referans ile : " << hayvan.adi() << " " << hayvan.ses() << '\n';
}

int main() {
    Kopek kopek{"Boncuk"};
    degerIle(kopek);
    referansIle(kopek);

    vector<Hayvan> dilimli;
    dilimli.push_back(kopek);
    cout << "vector<Hayvan>: " << dilimli[0].ses() << '\n';
    return 0;
}
```

```
ÇIKTI
değer ile    : Boncuk ...
referans ile : Boncuk Hav hav
vector<Hayvan>: ...
```

Dilimleme sessizce olur: derleyici uyarı vermez, program çökmez, köpek yalnızca "..." der. Çünkü `Hayvan` tipindeki bir değişken gerçekten bir `Hayvan` nesnesidir ve yalnızca `Hayvan` kadar yer ayrılır. `Kopek`'e ait kısım kopyalanmaz, nesnenin sanal tablo işaretçisi de artık `Hayvan`'ın tablosunu gösterir. Yeni nesne, yazdığımız yapıcılardan biriyle değil, `Hayvan`'ın kopya yapıcısıyla (Bölüm 30) oluşur.

Korunmanın yolları belli:

- Çok biçimli sınıfları fonksiyona `const Sinif&` ile ya da işaretçiyle verelim (Bölüm 25).
- Koleksiyonda `vector<Hayvan>` değil `vector<unique_ptr<Hayvan>>` kullanalım.
- Taban sınıfı soyut yaparsak dilimleme zaten derlenmez: `Hayvan` tipinde nesne yaratılamaz.

> **▲ Sık Yapılan Hata · Çok biçimli sınıfı değerle geçirmek**
>
> `void goster(Hayvan h)` ve `vector<Hayvan>`, çok biçimliliği kapatmanın en kolay yoludur. Program derlenir ve çalışır, ama türetilmiş sınıfların davranışı kaybolur. Taban tipini parametrede `const Hayvan&`, koleksiyonda akıllı işaretçi olarak yazalım.

Yapıcıda ve yıkıcıda sanal fonksiyon çağırmak

Taban sınıfın yapıcısı çalışırken türetilmiş kısım henüz yaratılmamıştır. Bu yüzden yapıcıda (ve yıkıcıda) çağrılan sanal fonksiyon dinamik bağlanmaz: o anda hangi sınıfın yapıcısı çalışıyorsa onun sürümü seçilir.

▸ yapicidaSanal.cpp

```
#include <iostream>
using namespace std;

class Taban {
public:
    Taban() { selamla(); }
    virtual void selamla() const { cout << "Taban selamlıyor\n"; }
    virtual ~Taban() = default;
};

class Turemis : public Taban {
```

```
public:
    Turemis() { selamla(); }
    void selamla() const override { cout << "Turemis selamlıyor\n"; }
};

int main() {
    Turemis t;
    t.selamla();
    return 0;
}
```

```
ÇIKTI
Taban selamlıyor
Turemis selamlıyor
Turemis selamlıyor
```

İlk satır `Taban` yapıcısından geldi ve `Turemis` ezmesi çalışmadı; `Turemis` bölümü henüz kurulmamıştı ve tanımsız üyelere dokunulmasını önlemek için C++ böyle davranır. Saf sanal bir fonksiyon yapıcıdan çağrılırsa durum daha da kötüdür (çağrılacak gövde yoktur). Yapıcılarda ve yıkıcılarda sanal fonksiyon çağırmamak iyi bir alışkanlıktır.

31.11 Aşağı Dönüşüm: dynamic_cast

Yukarı dönüşüm (türetilmişten tabana) güvenli ve örtüktür. Tersi, yani taban işaretçisinden türetilmiş işaretçiye **aşağı dönüşüm** (downcasting), nesnenin gerçekten o türden olup olmadığına bağlıdır. Bunu çalışma zamanında denetleyen işleç `dynamic_cast`'tir (C++ tarzı dönüşümleri Bölüm 22'de görmüştük): dönüşüm mümkünse türetilmiş işaretçiyi, değilse `nullptr` döndürür. İşleç yalnızca **sanal fonksiyonu olan** (polimorfik) sınıflarla çalışır.

```
const Mudur* m = dynamic_cast<const Mudur*>(calisan.get());
if (m != nullptr) {
    // calisan gerçekten bir Mudur; Mudur'e özgü üyeler kullanılabilir
}
```

Bir sonraki örnekte kullanacağız. Ancak sık sık `dynamic_cast` gerektiğini görüyorsak tasarım yeniden düşünülmelidir: tür kontrolü yerine ortak bir sanal fonksiyon çoğu zaman daha iyi çözümdür.

31.12 Çalışan Hiyerarşisi

Şimdiye kadarki parçaları bir araya getirelim. Bir şirkette üç tür çalışan var ve maaş hesabı her tür için farklıdır: müdür sabit maaşa ek olarak ekibinin büyüklüğüne göre prim alır, mühendis saat ücretiyle çalışır, stajyer sabit bir ödenek alır.

calisanlar.cpp

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Calisan {
public:
    Calisan(string adi) : ad{adi} {}
```

```
    virtual ~Calisan() = default;
    virtual double maas() const = 0;
    virtual string unvan() const = 0;
    void yazdir() const {
        cout << unvan() << " " << ad << ": " << maas() << " TL\n";
    }
private:
    string ad;
};

class Mudur : public Calisan {
public:
    Mudur(string adi, int ekip) : Calisan{adi}, ekipBuyuklugu{ekip} {}
    double maas() const override { return 40000 + 2500.0 * ekipBuyuklugu; }
    string unvan() const override { return "Müdür"; }
    int ekip() const { return ekipBuyuklugu; }
private:
    int ekipBuyuklugu;
};

class Muhendis : public Calisan {
public:
    Muhendis(string adi, double saatUcreti, int saat)
        : Calisan{adi}, ucret{saatUcreti}, calisilanSaat{saat} {}
    double maas() const override { return ucret * calisilanSaat; }
    string unvan() const override { return "Mühendis"; }
private:
    double ucret;
    int calisilanSaat;
};

class Stajyer : public Calisan {
public:
    using Calisan::Calisan;                 // taban yapıcısını devral
    double maas() const override { return 17000; }
    string unvan() const override { return "Stajyer"; }
};

int main() {
    vector<unique_ptr<Calisan>> kadro;
    kadro.push_back(make_unique<Mudur>("Ayşe", 8));
    kadro.push_back(make_unique<Muhendis>("Mehmet", 350, 160));
    kadro.push_back(make_unique<Stajyer>("Elif"));

    double toplam{0};
    for (const auto& c : kadro) {
        c->yazdir();
        toplam += c->maas();

        const Mudur* m = dynamic_cast<const Mudur*>(c.get());
        if (m != nullptr) {
            cout << "  (ekip: " << m->ekip() << " kişi)\n";
        }
    }
    cout << "Toplam maaş gideri: " << toplam << " TL\n";
    return 0;
}
```

```
ÇIKTI
Müdür Ayşe: 60000 TL
  (ekip: 8 kişi)
Mühendis Mehmet: 56000 TL
Stajyer Elif: 17000 TL
Toplam maaş gideri: 133000 TL
```

Programın her parçası bölümde gördüğümüz bir kurala dayanıyor:

- `Calisan` soyuttur ve `maas()` ile `unvan()` saf sanaldır; her tür kendi hesabını yazmak zorundadır.
- `yazdir()` taban sınıfta tek kez yazıldı ve sanal çağrılar sayesinde her türde doğru unvanı ve maaşı gösterir.
- `Stajyer`, `using Calisan::Calisan;` ile tabanın yapıcısını devraldı; kendi yapıcısını yazmadı.
- Kadro `vector<unique_ptr<Calisan>>` olduğu için dilimleme olmaz, bellek de kendiliğinden temizlenir.
- `dynamic_cast` yalnızca müdüre özgü `ekip()` fonksiyonuna ulaşmak için kullanıldı.

Yeni bir `Yonetici` türü eklemek yalnızca yeni bir sınıf yazmak demektir; `main` içindeki döngü değişmez. Çok biçimliliğin sağladığı bu esnekliğe "açık-kapalı ilkesi" denir: yeni davranışa açık, var olan kodun değişmesine kapalı.

31.13 Çoklu Kalıtım

C++'ta bir sınıf birden fazla taban sınıftan türetilebilir. Bunun güvenli ve yaygın kullanımı, birden fazla **arayüz** uygulamaktır:

coklu.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Yazdirilabilir {
public:
    virtual ~Yazdirilabilir() = default;
    virtual void yazdir() const = 0;
};

class Kaydedilebilir {
public:
    virtual ~Kaydedilebilir() = default;
    virtual void kaydet() const = 0;
};

class Rapor : public Yazdirilabilir, public Kaydedilebilir {
public:
    Rapor(string b) : baslik{b} {}
    void yazdir() const override { cout << "Yazdırılıyor: " << baslik << '\n'; }
    void kaydet() const override { cout << "Kaydediliyor: " << baslik << '\n'; }
private:
    string baslik;
};

void yazdirVeGoster(const Yazdirilabilir& y) { y.yazdir(); }
void kaydetVeGoster(const Kaydedilebilir& k) { k.kaydet(); }

int main() {
    Rapor rapor{"Aylık Satışlar"};
    yazdirVeGoster(rapor);
```

```
    kaydetVeGoster(rapor);
    return 0;
}
```

```
ÇIKTI
Yazdırılıyor: Aylık Satışlar
Kaydediliyor: Aylık Satışlar
```

`Rapor` hem `Yazdirilabilir` hem `Kaydedilebilir` olarak kullanılabiliyor. Sorun, iki taban sınıf da aynı bir üst sınıftan türediğinde çıkar.

Elmas problemi

`Kara` ve `Su` sınıflarının ikisi de `Canli`'dan türesin, `Amfibi` de ikisinden türesin. Sınıf çizelgesi bir elmasa benzer:

```
        Canli
        /   \
     Kara   Su
        \   /
       Amfibi
```

Varsayılan olarak `Amfibi` nesnesi, `Canli` parçasının **iki ayrı kopyasını** taşır; biri `Kara` yoluyla, biri `Su` yoluyla. Üyeye erişim belirsiz olur:

▸ elmasHata.cpp

```
class Canli { public: int yas{1}; };
class Kara : public Canli {};
class Su : public Canli {};
class Amfibi : public Kara, public Su {};

int main() {
    Amfibi a;
    a.yas = 3;
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

elmasHata.cpp: In function 'int main()':
elmasHata.cpp:8:7: error: request for member 'yas' is ambiguous
    8 |     a.yas = 3;
      |       ^~~
elmasHata.cpp:1:27: note: candidates are: 'int Canli::yas'
    1 | class Canli { public: int yas{1}; };
      |                           ^~~
elmasHata.cpp:1:27: note:                 'int Canli::yas'
```

Çözüm **sanal kalıtımdır** (virtual inheritance): aradaki sınıflar tabandan `virtual` türetilir ve en alttaki nesne ortak taban parçasından yalnızca bir tane taşır.

▸ elmas.cpp

```
#include <iostream>
using namespace std;

class Canli {
```

```
public:
    Canli() { cout << "Canli\n"; }
    int yas{1};
};

class Kara : public virtual Canli {
public:
    Kara() { cout << "Kara\n"; }
};

class Su : public virtual Canli {
public:
    Su() { cout << "Su\n"; }
};

class Amfibi : public Kara, public Su {
public:
    Amfibi() { cout << "Amfibi\n"; }
};

int main() {
    Amfibi a;
    a.yas = 3;
    cout << "yas = " << a.yas << '\n';
    return 0;
}
```

```
ÇIKTI
Canli
Kara
Su
Amfibi
yas = 3
```

`Canli` yalnızca bir kez yaratıldı ve `a.yas` artık belirsiz değil. Sanal taban sınıfı, en alttaki sınıf (burada `Amfibi`) kurar. Sanal kalıtım ek bir dolaylılık ve karmaşıklık getirir; bu yüzden başlangıçta tek kalıtım ve saf sanal fonksiyonlardan oluşan arayüz sınıflarıyla ilerlemek yeterlidir. Çoklu kalıtımda ayrıca iki tabanın aynı adlı fonksiyonları çakışırsa çağrıyı `Taban::fonksiyon` biçiminde nitelemek gerekir.

Üç Dilde · Kalıtım ve çok biçimlilik: C, C++, C# *C · C++ · C#*

	C	C++	C#
Kalıtım	Yok; `struct` içine `struct` konur	`class B : public A`; çoklu kalıtım var	`class B : A`; yalnızca tek sınıftan kalıtım
Sanal metot	Elle, fonksiyon işaretçileriyle	Taban sınıfta `virtual`, türetilmişte `override`	Taban sınıfta `virtual`, türetilmişte `override`
Varsayılan	Yok	Fonksiyonlar sanal **değildir**	Metotlar sanal **değildir**
Soyut sınıf	Yok	En az bir `= 0` fonksiyon	`abstract class`, `abstract` metot
Arayüz	Yok	Yalnızca saf sanal sınıf	`interface` anahtar sözcüğü; birden fazla uygulanabilir
Ezmeyi kilitleme	Yok	`final`	`sealed`
Gizleme	Yok	Aynı ad türetilmişi önceler	`new` anahtar sözcüğü ile belirtilir

	C	C++	C#
Kök sınıf	Yok	Yok	Her şey `object`'ten türer

C'de çok biçimlilik, C++'ın derleyiciye yaptırdığı sanal tablonun elle kurulmasıdır. C#'ın yaklaşımı C++'a çok yakındır: metotlar varsayılan olarak sanal değildir, `virtual` ve `override` anahtar sözcükleri aynıdır ve `override` yazmak burada **zorunludur**. Farklı olan, çoklu kalıtımın yasak olması (elmas problemi doğmaz) ve birden fazla `interface` uygulayabilmenin onun yerini almasıdır:

```
abstract class Sekil
{
    public abstract double Alan();                  // C++: virtual ... = 0
}

class Daire : Sekil                                 // C++: : public Sekil
{
    private readonly double r;
    public Daire(double r) { this.r = r; }
    public override double Alan() => Math.PI * r * r;
}
```

Ayrıntılar Bölüm 50 ve 51'de.

Bölüm Özeti

- Kalıtım, `class Turetilmis : public Taban` ile yazılır ve "X bir Y'dir" ilişkisi için kullanılır; "sahip" ilişkisi için üye değişken tercih edilir.
- `private` üyeyi türetilmiş sınıf devralır ama erişemez; `protected` üyeye türetilmiş sınıf erişir, dışarısı erişemez. `class`'ta varsayılan kalıtım `private`'tır, bu yüzden `public` yazmayı unutmayız.
- Yapıcılar tabandan türetilmişe, yıkıcılar tersine çalışır. Taban yapıcısına argüman, türetilmiş sınıfın başlatma listesinde verilir.
- `virtual` olmayan bir fonksiyonu türetilmiş sınıfta yeniden yazmak onu yalnızca gizler; çağrı işaretçinin ya da referansın tipine göre statik bağlanır.
- Sanal fonksiyonlarda çağrı, nesnenin gerçek tipine göre dinamik bağlanır; bunun için fonksiyonun `virtual` olması, aynı imzayla ezilmesi ve işaretçi/referans üzerinden çağrılması gerekir.
- Sanal fonksiyonları ezerken her zaman `override` yazarız; imza hatalarını derleyiciye yakalatır. `final` ezmeyi ya da türetmeyi yasaklar.
- Dinamik bağlama sanal tablo (vtable) ve nesnedeki gizli işaretçiyle gerçekleşir; bunun bedeli bir işaretçi boyutunda bellek ve bir dolaylı çağrıdır.
- Saf sanal fonksiyon (`= 0`) sınıfı soyut yapar; soyut sınıftan nesne yaratılamaz ama işaretçi ve referansı tanımlanabilir. Yalnızca saf sanal fonksiyonlardan oluşan sınıf arayüz görevi görür.
- Taban sınıf işaretçisiyle silinecek her sınıfın yıkıcısı `virtual` olmalıdır; aksi hâlde davranış tanımsızdır.
- Çok biçimli nesneleri değerle geçirmek ya da `vector<Taban>` içinde tutmak nesneyi dilimler; referans, işaretçi ve `vector<unique_ptr<Taban>>` kullanılır.
- Yapıcı ve yıkıcı içinde çağrılan sanal fonksiyon dinamik bağlanmaz; bu çağrılardan kaçınırız.
- `dynamic_cast`, taban işaretçisini çalışma zamanında türetilmiş işaretçiye dönüştürür; başarısızsa `nullptr` verir. Sık gerekiyorsa tasarım gözden geçirilmelidir.

- Çoklu kalıtım arayüz uygulamak için güvenlidir; ortak bir tabandan iki yoldan türeyince elmas problemi çıkar ve `virtual` kalıtımla çözülür.

Alıştırmalar

1. Aşağıdaki ilişkilerden hangisi için kalıtım uygun **değildir**?

   A) `Kare` ve `Sekil`
   B) `Ogrenci` ve `Kisi`
   C) `Araba` ve `Motor`
   D) `Muhendis` ve `Calisan`
   E) `Kedi` ve `Hayvan`

2. Aşağıdaki sınıflarda `main` içinden hangi üyeye erişilebilir?

```
class T { public: int a; protected: int b; private: int c; };
class U : public T { };
// main: U u;
```

   A) Yalnızca `u.a`
   B) `u.a` ve `u.b`
   C) `u.a`, `u.b` ve `u.c`
   D) Yalnızca `u.b`
   E) Hiçbiri

3. Bu programın çıktısı nedir?

```
#include <iostream>
using namespace std;
struct A { A() { cout << "A "; } ~A() { cout << "~A "; } };
struct B : A { B() { cout << "B "; } ~B() { cout << "~B "; } };
struct C : B {
    A uye;
    C() { cout << "C "; }
    ~C() { cout << "~C "; }
};
int main() { C c; cout << "| "; }
```

4. `Otobus` sınıfı `Tasit`'ten türetilmiş ve `Tasit`'in yalnızca `Tasit(string plaka)` yapıcısı var. Aşağıdaki yapıcı neden derlenmez, nasıl düzeltilir?

```
class Otobus : public Tasit {
public:
    Otobus(int koltuk) : koltukSayisi{koltuk} {}
private:
    int koltukSayisi;
};
```

5. Bu programın çıktısı nedir?

```cpp
#include <iostream>
using namespace std;
struct T {
    void a() const { cout << "T::a "; }
    virtual void b() const { cout << "T::b "; }
};
struct U : T {
    void a() const { cout << "U::a "; }
    void b() const override { cout << "U::b "; }
};
int main() {
    U u;
    T* p = &u;
    T& r = u;
    T t = u;
    p->a(); p->b(); r.b(); t.b(); u.a();
}
```

6. Aşağıdaki kodda `Kopek` nesnesi `Hayvan*` üzerinden çağrıldığında "Hav hav" yerine "..." çıkıyor. Hata nerede, nasıl düzeltilir?

```cpp
class Hayvan {
public:
    virtual string ses() const { return "..."; }
};
class Kopek : public Hayvan {
public:
    string ses() { return "Hav hav"; }
};
```

7. Aşağıdakilerden hangisi bir çağrının dinamik bağlanması için gerekli koşullardan **biri değildir**?

   A) Fonksiyonun taban sınıfta `virtual` olması
   B) Türetilmiş sınıfın fonksiyonu aynı imzayla (`const` dahil) ezmesi
   C) Çağrının işaretçi ya da referans üzerinden yapılması
   D) Nesnenin öbekte (`new` ile) yaratılmış olması
   E) Fonksiyonun taban ve türetilmiş sınıfta aynı parametre listesine sahip olması

8. Aşağıdakilerden hangisi doğrudur?

   A) Soyut sınıftan nesne yaratılabilir ama işaretçi tanımlanamaz.
   B) Soyut sınıfın yapıcısı olamaz.
   C) Taban sınıfın tüm saf sanal fonksiyonlarını ezmeyen türetilmiş sınıf da soyuttur.
   D) Saf sanal fonksiyon her zaman `static` olmak zorundadır.
   E) Bir sınıf yalnızca bir saf sanal fonksiyona sahip olabilir.

9. Bu programın çıktısı nedir?

```cpp
#include <iostream>
#include <string>
using namespace std;
struct S { virtual string ad() const { return "S"; } };
struct D : S { string ad() const override { return "D"; } };
void f(S s) { cout << s.ad(); }
void g(const S& s) { cout << s.ad(); }
int main() {
    D d;
```

```
    S* p = &d;
    f(d); g(d); cout << p->ad() << '\n';
}
```

10. Aşağıdaki kodda ne yanlıştır ve sonucu ne olur? Düzeltin.

```
class Taban {
public:
    virtual void islem() {}
    ~Taban() {}
};
class Alt : public Taban {
    int* veri = new int[100];
public:
    ~Alt() { delete[] veri; }
};
Taban* p = new Alt;
delete p;
```

11. Bu programın çıktısı nedir?

```
#include <iostream>
using namespace std;
struct T {
    T() { goster(); }
    virtual void goster() const { cout << "T\n"; }
};
struct U : T {
    U() { goster(); }
    void goster() const override { cout << "U\n"; }
};
int main() { U u; u.goster(); }
```

12. `vector<Sekil> sekiller; sekiller.push_back(Daire{2});` satırının sorunu nedir? Doğru yazımı gösterin.
13. Aşağıdaki hiyerarşide `Amfibi` içindeki `yas` üyesine neden erişilemez ve sorun nasıl çözülür?

```
class Canli { public: int yas{1}; };
class Kara : public Canli {};
class Su : public Canli {};
class Amfibi : public Kara, public Su {};
```

14. **Program yazma (kolay).** Soyut bir `Hayvan` sınıfı yazın (`ad` alanı, saf sanal `ses()` ve sanal yıkıcı). `Kedi`, `Kopek` ve `Kus` sınıflarını türetip her birinin sesini döndürün. `main` içinde `vector<unique_ptr<Hayvan>>` ile üçünü tutup "Pamuk: Miyav" biçiminde yazdırın.
15. **Program yazma (orta).** `Sekil` soyut sınıfından (`alan()` ve `ad()` saf sanal) `Kare` ve `Ucgen` sınıflarını türetin. Şekillerden oluşan bir listeden **en büyük alanlı** şekli bulup adıyla birlikte yazdıran bir fonksiyon yazın.
16. **Program yazma (zor).** `Hesap` adlı soyut bir sınıf yazın: `bakiye` alanı, `para yatir(double)`, `virtual bool cek(double)` ve saf sanal `aySonu()` (faiz ya da kesinti uygular) olsun. `VadesizHesap` (aylık 5 TL işlem ücreti keser) ve `VadeliHesap` (bakiyeye %2 faiz ekler, yalnızca bakiyenin yarısına kadar çekime izin verir) sınıflarını türetin. Farklı hesapları tek bir listede tutup `aySonu()` çağırarak bakiyeleri yazdırın.

BÖLÜM 32

Operatör Aşırı Yükleme

BU BÖLÜMDE

- Operatör aşırı yüklemenin ne işe yaradığını ve `a + b` ifadesinin derleyici gözünde sıradan bir fonksiyon çağrısı olduğunu
- Aşırı yüklenebilen ve yüklenemeyen operatörleri, dilin getirdiği kural ve sınırları
- Aritmetik, bileşik atama ve karşılaştırma operatörlerini üye ve üye olmayan fonksiyon olarak yazmayı; hangisinin ne zaman seçileceğini
- `<<` ve `>>` akış operatörlerini ve `friend` bildirimini
- `[]`, `()`, önek ve sonek `++` / `--` operatörlerini
- Aşırı yüklenmemesi gereken operatörleri ve iyi uygulama kurallarını
- Aynı özelliğin C'de bulunmayışını ve C#'taki farklı biçimini

Bölüm 21'de `std::cout << "Merhaba"` yazarken `<<` işaretinin C'deki bit kaydırmadan farklı bir iş yaptığını söylemiş, nedenini bu bölüme bırakmıştık. Bölüm 25'te ise aynı adlı fonksiyonları farklı parametre listeleriyle tanımlamayı, yani fonksiyon aşırı yüklemeyi (function overloading) görmüştük. **Operatör aşırı yükleme** (operator overloading) bu fikrin operatörlere uygulanmış hâlidir: `+`, `==`, `<<`, `[]` gibi operatörlerin kendi sınıflarımız için ne yapacağını biz belirleriz.

Bu özellik C'de yoktur. C'de iki `struct`'ı toplamak için `topla(a, b)` gibi bir fonksiyon yazarız; C++'ta ise sınıfımıza `a + b` yazma hakkını da kazandırabiliriz. Standart kütüphanenin rahat kullanımı bu özelliğe dayanır: `std::string` için `+` ve `[]` (Bölüm 27), `vector` için `[]` (Bölüm 26), akıllı işaretçiler için `*` ve `->` (Bölüm 29) hep aşırı yüklenmiş operatörlerdir. Bu bölümde iki sınıfı baştan sona yazacağız: Bölüm 30'da `topla`, `cikar`, `carp` üye fonksiyonlarıyla yazdığımız `Kesir` sınıfını operatörlerle yeniden kuracağız ve düzlemdeki vektörler için `Vektor2D` sınıfını ekleyeceğiz. Bölüm 28'deki referans döndürme ile Bölüm 30'daki `const` üye fonksiyonlar, yapıcılar ve `friend` bilgisi bu bölümde sürekli kullanılacaktır.

32.1 Neden Operatör Aşırı Yükleme?

`int` ve `double` ile `a + b`, `a == b`, `a < b` yazmak doğaldır. Bir kesir sınıfı yazdığımızda aynı doğallığı beklemek de makuldür. Ama derleyici kendi sınıfımızın nesnelerini nasıl toplayacağını bilmez:

kesirYok.cpp

```
#include <iostream>

class Kesir {
public:
    Kesir(int pay, int payda) : pay_{pay}, payda_{payda} {}
private:
```

```
    int pay_;
    int payda_;
};

int main() {
    Kesir a{1, 2};
    Kesir b{1, 3};
    Kesir c = a + b;
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

kesirYok.cpp: In function 'int main()':
kesirYok.cpp:14:17: error: no match for 'operator+' (operand types are 'Kesir' and 'Kesir')
   14 |     Kesir c = a + b;
      |               ~ ^ ~
      |               |   |
      |               |   Kesir
      |               Kesir
```

Mesajın dili öğreticidir: derleyici `operator+` adında, işlenenleri iki `Kesir` olan bir fonksiyon aramış ve bulamamıştır. Demek ki `a + b` yazıldığında derleyicinin aradığı şey, adı `operator+` olan sıradan bir fonksiyondur. Bu fonksiyonu biz yazarsak ifade derlenir.

> **Tanım: Operatör aşırı yükleme**
>
> Bir operatörün, en az bir işleneni kullanıcı tanımlı tip (sınıf ya da `enum`) olduğunda ne yapacağını, adı `operator` sözcüğü ile operatörün simgesinden oluşan özel bir fonksiyonla belirleme olanağıdır. Örneğin `operator+`, `operator==`, `operator<<`.

Operatör aşırı yükleme dile yeni bir yetenek katmaz. `a + b` ifadesi `a.operator+(b)` ya da `operator+(a, b)` çağrısının yazım kolaylığıdır. Kazancımız okunabilirliktir: `a.topla(b).topla(c)` yerine `a + b + c` yazarız. Bu yüzden asıl ölçüt şudur: operatörü yalnızca anlamı herkesçe bilinen yerde tanımlamalıyız. İki kesiri `+` ile toplamak doğaldır; `+` operatörünün çıkarma yapması ya da `<<` operatörünün dosya silmesi ise kodu okuyanı yanıltır.

32.2 Kurallar ve Sınırlar

Aşırı yükleme serbest bir oyun alanı değildir. Dil şu sınırları koyar:

- **Yeni operatör icat edilemez.** `**` ile üs almak ya da `<>` yazmak mümkün değildir; yalnızca var olan operatörler yüklenir.
- **İşlenen sayısı değişmez.** İki işlenenli `+` iki işlenenli kalır; tek işlenenli `+x` ayrı bir operatördür ve ayrıca tanımlanır.
- **Öncelik ve birleşme yönü değişmez.** `a + b * c` ifadesinde `*` her zaman önce çalışır (Bölüm 5), bizim sınıfımız için de.
- **En az bir işlenen kullanıcı tanımlı tip olmalıdır.** `int + int` işleminin anlamı değiştirilemez.
- **Bazı operatörler hiç yüklenemez.** Bunlar kapsam çözümleme `::`, üye erişim `.`, üye işaretçisi `.*`, koşul `?:` ve `sizeof` operatörleridir.
- **Dört operatör yalnızca üye fonksiyon olabilir:** atama `=`, indeks `[]`, çağrı `()` ve ok `->`.

Her operatör için varsayılan bir anlam da yoktur; tanımlamadığımız operatör kullanılamaz. İki istisna vardır: atama `=` ve adres alma `&` operatörleri sınıflar için derleyici tarafından kendiliğinden sağlanır. Atamayı ve derleyicinin yazdığı sürümünü "Atama Operatörleri" başlığında ele alacağız.

32.3 Üye Fonksiyon Olarak Operatörler

Bir operatörü sınıfın üye fonksiyonu olarak yazarsak **sol işlenen çağrının yapıldığı nesne** (`*this`), sağ işlenen ise parametre olur. `a + b` ifadesi `a.operator+(b)` demektir. Kesir sınıfımızın ilk sürümünde `+=`, `+`, `==` ve `<` operatörlerini üye olarak yazalım:

kesirUye.cpp

```
#include <iostream>
#include <numeric>
#include <stdexcept>

class Kesir {
public:
    Kesir(int pay = 0, int payda = 1) : pay_{pay}, payda_{payda} {
        if (payda_ == 0) {
            throw std::invalid_argument{"payda sıfır olamaz"};
        }
        sadelestir();
    }
    int pay() const { return pay_; }
    int payda() const { return payda_; }

    Kesir& operator+=(const Kesir& d) {
        pay_ = pay_ * d.payda_ + d.pay_ * payda_;
        payda_ = payda_ * d.payda_;
        sadelestir();
        return *this;
    }
    Kesir operator+(const Kesir& d) const {
        Kesir sonuc{*this};
        sonuc += d;
        return sonuc;
    }
    bool operator==(const Kesir& d) const {
        return pay_ == d.pay_ && payda_ == d.payda_;
    }
    bool operator<(const Kesir& d) const {
        return pay_ * d.payda_ < d.pay_ * payda_;
    }

private:
    void sadelestir() {
        if (payda_ < 0) {
            pay_ = -pay_;
            payda_ = -payda_;
        }
        int g = std::gcd(pay_, payda_);
        pay_ /= g;
        payda_ /= g;
    }
    int pay_;
    int payda_;
};

void yazdir(const Kesir& k) {
```

```
    std::cout << k.pay() << '/' << k.payda();
}

int main() {
    std::cout << std::boolalpha;
    Kesir a{1, 2};
    Kesir b{1, 3};

    Kesir c = a + b;
    std::cout << "a + b = ";
    yazdir(c);
    std::cout << '\n';

    Kesir d = a.operator+(b);
    std::cout << "a.operator+(b) = ";
    yazdir(d);
    std::cout << '\n';

    Kesir e = a + 2;
    std::cout << "a + 2 = ";
    yazdir(e);
    std::cout << '\n';

    std::cout << "Kesir{2, 4} == a: " << (Kesir{2, 4} == a) << '\n';
    std::cout << "b < a: " << (b < a) << '\n';

    a += b;
    std::cout << "a += b -> ";
    yazdir(a);
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
a + b = 5/6
a.operator+(b) = 5/6
a + 2 = 5/2
Kesir{2, 4} == a: true
b < a: true
a += b -> 5/6
```

Bölüm 30'daki gibi yapıcı her kesiri sadeleştirir; `std::gcd` C++17'de `<numeric>` başlığına eklenmiş en büyük ortak bölen fonksiyonudur. Payda her zaman pozitif tutulur; bu yüzden `==` yalnızca pay ve paydaları karşılaştırarak doğru çalışır (`Kesir{2, 4} == a` ifadesi doğrudur). Bölüm 30'da sıfır paydayı sessizce 1'e çekmiştik; burada daha doğru olanı yapıp bir istisna fırlatıyoruz. İstisnaları Bölüm 34'te göreceğiz, şimdilik "yanlış kullanımı durdurur" demek yeterlidir.

Operatör imzalarındaki üç ayrıntı kuraldır:

1. **Parametre `const Kesir&`.** Sağ işlenen kopyalanmaz (referans) ve fonksiyon onu değiştiremez (`const`). Bölüm 28'deki `const` doğruluğu kuralının aynısıdır.
2. **Parantezden sonraki `const`.** `operator+`, `==` ve `<` sol işlenenlerini de değiştirmez; bu yüzden `const` üye fonksiyon olarak yazılırlar. Aksi hâlde `const` bir nesne ya da `const Kesir&` parametresi üzerinde çağrılamazlar.
3. **Dönüş tipleri farklıdır.** `+` yeni bir nesneyi **değer olarak** döndürür, çünkü `a + b` ne `a`'yı ne `b`'yi değiştirmelidir. `+=` ise soldaki nesneyi değiştirir ve `*this`'i **referansla** (`Kesir&`) döndürür; böylece `(a += b) += c` gibi zincirler çalışır. Bölüm 28'de `this` ile gördüğümüz zincirleme bu desenin aynısıdır.

İpucu · + operatörünü += üzerinden yazın

`operator+` kendi hesabını yapmadı; nesnenin kopyasını alıp `+=`'i çağırdı. Hesap mantığı tek yerde (`+=`) durur, `+` ona dayanır. Bu kalıp `-` / `-=`, `*` / `*=`, `/` / `/=` çiftlerinde de geçerlidir. Mantığı iki yerde tutmak, birini düzeltip diğerini unutmak demektir.

▲ Sık Yapılan Hata · const unutulan operatör

`Kesir operator+(const Kesir& d)` biçiminde, parantezden sonra `const` yazmazsak, `const Kesir` nesnesi ya da `const Kesir&` parametresiyle `a + b` yazan kod derlenmez. g++ `passing 'const Kesir' as 'this' argument discards qualifiers` iletisini verir. Nesneyi değiştirmeyen her üye operatörün sonuna `const` ekleyin.

`a + 2` ifadesinin de çalıştığına dikkat edelim. `Kesir(int pay = 0, int payda = 1)` yapıcısı tek `int` argümanla çağrılabildiği için derleyici `2`'yi örtük olarak `Kesir{2, 1}`'e dönüştürdü. Aynı dönüşüm *sol* işlenende yapılamaz: üye `operator+` için `2 + a` yazılamaz, çünkü `2` bir `int`'tir ve `int`'in üyesi olan bir `operator+(Kesir)` yoktur. Bu sorunun çözümü bir sonraki başlıktadır.

32.4 Üye Olmayan Operatörler

Üye operatörde sol işlenen her zaman kendi sınıfımızın nesnesidir. Peki `2.0 * v` gibi sol işlenenin `double` olduğu bir ifade istersek? `Vektor2D` sınıfına yalnızca üye `Vektor2D operator*(double k) const` yazarsak `v * 2.0` çalışır, ama `2.0 * v` satırı `no match for 'operator*' (operand types are 'double' and 'Vektor2D')` hatasıyla derlenmez; `double` bir sınıf olmadığı için ona üye fonksiyon ekleyemeyiz. Çözüm, operatörü **sınıfın dışında**, iki parametreli sıradan bir fonksiyon olarak yazmaktır: `operator*(double k, const Vektor2D& v)`. Bu durumda `a * b` ifadesi `operator*(a, b)` çağrısına dönüşür. Aşağıda tam bir `Vektor2D` sınıfı var; değişen nesneler üye, yeni değer üretenler üye olmayan operatörlerdir:

vektor2D.cpp

```
#include <cmath>
#include <iostream>

class Vektor2D {
public:
    Vektor2D(double x = 0, double y = 0) : x_{x}, y_{y} {}
    double x() const { return x_; }
    double y() const { return y_; }
    double uzunluk() const { return std::sqrt(x_ * x_ + y_ * y_); }

    Vektor2D& operator+=(const Vektor2D& d) {
        x_ += d.x_;
        y_ += d.y_;
        return *this;
    }
    Vektor2D& operator*=(double k) {
        x_ *= k;
        y_ *= k;
        return *this;
    }
    Vektor2D operator-() const { return Vektor2D{-x_, -y_}; }

private:
    double x_;
    double y_;
```

```
};

Vektor2D operator+(Vektor2D a, const Vektor2D& b) {
    a += b;
    return a;
}

Vektor2D operator-(const Vektor2D& a, const Vektor2D& b) {
    return a + (-b);
}

Vektor2D operator*(Vektor2D v, double k) {
    v *= k;
    return v;
}

Vektor2D operator*(double k, const Vektor2D& v) {
    return v * k;
}

bool operator==(const Vektor2D& a, const Vektor2D& b) {
    return a.x() == b.x() && a.y() == b.y();
}

void yazdir(const Vektor2D& v) {
    std::cout << '(' << v.x() << ", " << v.y() << ')';
}

int main() {
    Vektor2D a{3, 4};
    Vektor2D b{1, -2};

    std::cout << "a + b = ";
    yazdir(a + b);
    std::cout << "\na - b = ";
    yazdir(a - b);
    std::cout << "\n2 * a = ";
    yazdir(2 * a);
    std::cout << "\na * 0.5 = ";
    yazdir(a * 0.5);
    std::cout << "\n-a = ";
    yazdir(-a);
    std::cout << "\n|a| = " << a.uzunluk() << '\n';
    std::cout << std::boolalpha << "a - a == Vektor2D{}: "
              << (a - a == Vektor2D{}) << '\n';
    return 0;
}
```

```
ÇIKTI
a + b = (4, 2)
a - b = (2, 6)
2 * a = (6, 8)
a * 0.5 = (1.5, 2)
-a = (-3, -4)
|a| = 5
a - a == Vektor2D{}: true
```

Bu programda dikkat edilecek noktalar şunlardır:

- **Tek işlenenli** `-` ayrı bir fonksiyondur: parametresiz üye `operator-() const`. İki işlenenli `-` ise üye olmayan `operator-(a, b)` olarak yazıldı ve tek işlenenli olanı kullandı. Derleyici işlenen sayısına bakarak doğru sürümü seçer.

- `operator+` **birinci parametresini değer olarak alır.** Parametre zaten bir kopyadır; `+=` ile ona ekleme yapıp döndürmek, ayrıca yerel nesne açmaktan kısadır. `operator*` de aynı kalıbı izler.
- `2 * a` için `int` olan `2`, `double`'a dönüşerek `operator*(double, const Vektor2D&)` sürümüne gider. İki `operator*` aşırı yüklemesi birbirinin yerine geçmez; ikisini de yazmamız gerekir. İkincisi birinciyi çağırarak işi tek yerde tutar.
- `==` **üye olmayan fonksiyon olarak** yalnızca açık (public) erişimcileri kullandı; özel üyelere dokunması gerekmedi.
- `a - a == Vektor2D{}` ifadesinde `-` önceliği `==`'den yüksek olduğu için önce çıkarma yapılır. Öncelikler, operatörler yüklenince de aynıdır.

Üye mi, üye olmayan mı?

Karar verirken şu tablo işimizi görür:

Operatör	Nasıl yazılır	Neden
`=`, `[]`, `()`, `->`	Üye (zorunlu)	Dil böyle şart koşar
`+=`, `-=`, `*=`, `++`, `--`	Üye	Soldaki nesneyi değiştirirler
Tek işlenenli `-`, `!`, `~`	Üye	Tek işlenen nesnenin kendisidir
`+`, `-`, `*`, `/`, `==`, `<` …	Üye olmayan	Sol işlenen de örtük dönüşümden yararlansın (`2 * v`, `2 + a`)
`<<`, `>>` (akış)	Üye olmayan (zorunlu)	Sol işlenen `ostream` / `istream`, bizim sınıfımız değil

> ◆ **Kritik Nokta · İkili operatörleri üye olmayan yazın**
>
> `Kesir` için `operator+`'ı üye yazarsak `a + 2` çalışır, `2 + a` çalışmaz. Üye olmayan sürümde iki işlenen de aynı muameleyi görür; ikisinde de örtük dönüşüm yapılabilir. Bu nedenle değeri değiştirmeyen ikili operatörleri (`+ - * / == != < >`) üye olmayan fonksiyon olarak, değiştirenleri (`+=` gibi) üye olarak yazmak yaygın kuraldır.

Üye olmayan operatör sınıfın özel üyelerine ihtiyaç duyarsa iki yol vardır: erişimcileri kullanmak (yukarıda yaptığımız gibi) ya da fonksiyonu `friend` yapmak. İkincisini şimdi göreceğiz.

32.5 Akış Operatörleri: << ve >>

`std::cout << k` yazabilmek için `operator<<` gerekir. İşleç `<<` iki işlenenlidir ve sol işleneni `std::cout`, yani `std::ostream` tipinde bir nesnedir. Üye fonksiyon olsaydı `ostream` sınıfının içinde yazılması gerekirdi; o sınıf bizim değil. Dolayısıyla `operator<<` **üye olmayan** bir fonksiyon olmak zorundadır. Kalıbı sabittir:

```
std::ostream& operator<<(std::ostream& os, const Kesir& k);
```

- İlk parametre `std::ostream&`'dir: akış nesneleri kopyalanamaz, bu yüzden referansla geçerler.
- Fonksiyon `os`'a yazar ve `os`**'u referansla geri döndürür**. `cout << a << b` ifadesi `(cout << a) << b` diye okunur; ilk `<<`, `cout`'u geri verdiği için ikincisi çalışır. Bölüm 28'de `*this` döndürerek gördüğümüz zincirleme aynı fikirdir.
- `>>` için kalıp `std::istream& operator>>(std::istream& is, Kesir& k)` biçimindedir; `k` burada `const` olmayan referanstır, çünkü okunan değer içine yazılacaktır.

Sınıfın özel üyelerine akış operatöründen erişmek için `friend` kullanabiliriz:

Tanım: friend (arkadaş) bildirimi

Bölüm 30'da tanıştığımız `friend`, sınıfın içinde bildirilen bir fonksiyonun (ya da başka bir sınıfın) sınıfın `private` ve `protected` üyelerine erişmesine izin verir. Arkadaş fonksiyon sınıfın üyesi değildir; yalnızca erişim izni almıştır. Arkadaşlık sınıfın kendi verdiği bir izindir, tek yönlüdür ve kalıtımla geçmez.

Aşağıdaki program `Kesir` için hem `<<` hem `>>` yazar. Okuma operatörü `pay/payda` biçimindeki girdiyi ayrıştırır ve biçim bozuksa akışı hatalı duruma getirir:

kesirAkis.cpp

```
#include <iostream>
#include <numeric>

class Kesir {
public:
    Kesir(int pay = 0, int payda = 1) : pay_{pay}, payda_{payda} {
        sadelestir();
    }
    friend std::ostream& operator<<(std::ostream& os, const Kesir& k);
    friend std::istream& operator>>(std::istream& is, Kesir& k);

private:
    void sadelestir() {
        if (payda_ < 0) {
            pay_ = -pay_;
            payda_ = -payda_;
        }
        int g = std::gcd(pay_, payda_);
        pay_ /= g;
        payda_ /= g;
    }
    int pay_;
    int payda_;
};

std::ostream& operator<<(std::ostream& os, const Kesir& k) {
    os << k.pay_;
    if (k.payda_ != 1) {
        os << '/' << k.payda_;
    }
    return os;
}

std::istream& operator>>(std::istream& is, Kesir& k) {
    int pay{};
    int payda{};
    char ayrac{};
    is >> pay >> ayrac >> payda;
    if (is && ayrac == '/' && payda != 0) {
        k = Kesir{pay, payda};
    } else {
        is.setstate(std::ios::failbit);
    }
    return is;
}

int main() {
    Kesir a{6, 8};
    Kesir b{4, 2};
    std::cout << "a = " << a << ", b = " << b << '\n';
```

```
    Kesir x;
    std::cout << "Bir kesir girin: ";
    std::cin >> x;
    std::cout << "Okunan: " << x << '\n';

    Kesir y;
    std::cout << "Bir kesir daha: ";
    std::cin >> y;
    std::cout << "Okunan: " << y << '\n';

    Kesir z;
    std::cout << "Son kesir: ";
    if (!(std::cin >> z)) {
        std::cout << "geçersiz giriş\n";
    }
    return 0;
}
```

```
ÇIKTI
a = 3/4, b = 2
Bir kesir girin: 9/12
Okunan: 3/4
Bir kesir daha: 5/10
Okunan: 1/2
Son kesir: 3/0
geçersiz giriş
```

`operator<<` içinde `k.pay_` ve `k.payda_`'ya doğrudan erişmemizi `friend` bildirimi sağladı. Paydası 1 olan kesir tam sayı gibi yazılır (`b = 2`). `operator>>` içinde `is` akışı, Bölüm 23'te gördüğümüz gibi, hata bayrağı tutan bir nesnedir; biçim bozuk ya da payda sıfır ise `failbit` ayarlanır ve çağıran `if (!(std::cin >> z))` ile bunu yakalar. Akışı geri döndürmek burada da zincirleme (`std::cin >> a >> b`) için gereklidir.

> **▲ Sık Yapılan Hata · void döndüren operator<<**
>
> `operator<<` içinde `os`'u döndürmeyi unutursak (`void` yazarsak) tek başına `cout << a;` çalışır, ama `cout << a << b;` çalışmaz: ilk `<<`'un sonucu `void` olur ve `void << b` diye bir işlem yoktur. Dönüş tipini `std::ostream` (referanssız) yazmak da derlenmez, çünkü akışı kopyalamaya çalışır. Hata mesajı `no match for 'operator<<'` ile başlar ve kütüphanedeki onlarca adayı sıralayarak yüzlerce satıra uzar (Ek D); ilk `error:` satırı yeterlidir.

`friend` yerine erişimci fonksiyonlar da yeterli olabilir. Önce erişimcilerle çözülüp çözülemediğine bakmak iyi bir alışkanlıktır; `friend` kapsüllemeyi delmez, ama sınıfın dışına açılan kapı sayısını artırır.

32.6 Karşılaştırma Operatörleri

Karşılaştırma operatörleri altı tanedir: `==`, `!=`, `<`, `>`, `<=`, `>=`. Hepsini ayrı ayrı hesaplamak yerine ikisini temel alıp ötekileri onlardan türetiriz:

```
bool operator==(const Kesir& a, const Kesir& b) {
    return a.pay() == b.pay() && a.payda() == b.payda();
}
bool operator!=(const Kesir& a, const Kesir& b) { return !(a == b); }
bool operator<(const Kesir& a, const Kesir& b) {
    return a.pay() * b.payda() < b.pay() * a.payda();
}
```

```
bool operator>(const Kesir& a, const Kesir& b)  { return b < a; }
bool operator<=(const Kesir& a, const Kesir& b) { return !(b < a); }
bool operator>=(const Kesir& a, const Kesir& b) { return !(a < b); }
```

Altısını da yazmalıyız, çünkü `a > b` için derleyici `operator>` arar; `operator<`'dan kendiliğinden üretmez. Standart algoritmalar (`std::sort`) ve sıralı kapsayıcılar (`std::set`, `std::map`; Bölüm 35) yalnızca `<` kullanır, ama sınıfın kullanıcısı öteki operatörleri de bekler.

> **◆ Kritik Nokta · Tutarlı sıralama**
>
> `<` tanımı tutarlı bir sıralama vermelidir: `a < b` ve `b < a` aynı anda doğru olamaz; `a < b` ve `b < c` ise `a < c` demektir. Bu şartları bozan bir `<`, `std::sort`'un yanlış sonuç vermesine ya da çökmesine yol açabilir.

> *Tarihçe · C++20 ve üç yönlü karşılaştırma*
>
> C++20 `<=>` operatörünü (üç yönlü karşılaştırma, halk arasında "uzay gemisi" operatörü) getirdi. Sınıfa `auto operator<=>(const T&) const = default;` yazmak, üyeleri sırayla karşılaştıran `<`, `>`, `<=`, `>=` operatörlerini ve `==`'i derleyiciye ürettirir. Kesir gibi karşılaştırma kuralı alanların sırasından farklı olan sınıflarda `<=>`'u elle yazmak gerekir. Bu kitabın örnekleri C++17 ile derlendiği için altı operatörü elle yazıyoruz.

32.7 Kesir Sınıfı Bir Arada

Şimdiye kadar öğrendiklerimizi bir araya getirelim. Aşağıdaki `Kesir`, dört aritmetik işlemi, tek işlenenli eksiyi, altı karşılaştırmayı ve `<<` operatörünü içerir. Bileşik atama operatörleri üyedir, ikili operatörler onların üzerine kurulur. Bu kez erişimcileri kullandığımız için `friend` gerekmedi. Sonda yeni sınıfımızı standart kütüphanenin algoritmalarıyla (Bölüm 36) kullanıyoruz: `std::sort` `<` operatörüne, `std::accumulate` ise `+` operatörüne ihtiyaç duyar.

▸ kesirTam.cpp

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <stdexcept>
#include <vector>

class Kesir {
public:
    Kesir(int pay = 0, int payda = 1) : pay_{pay}, payda_{payda} {
        if (payda_ == 0) {
            throw std::invalid_argument{"payda sıfır olamaz"};
        }
        sadelestir();
    }
    int pay() const { return pay_; }
    int payda() const { return payda_; }

    Kesir operator-() const { return Kesir{-pay_, payda_}; }
    Kesir& operator+=(const Kesir& d) {
        pay_ = pay_ * d.payda_ + d.pay_ * payda_;
        payda_ *= d.payda_;
        sadelestir();
        return *this;
    }
    Kesir& operator-=(const Kesir& d) { return *this += -d; }
    Kesir& operator*=(const Kesir& d) {
```

```
        pay_ *= d.pay_;
        payda_ *= d.payda_;
        sadelestir();
        return *this;
    }
    Kesir& operator/=(const Kesir& d) {
        if (d.pay_ == 0) {
            throw std::invalid_argument{"sıfıra bölme"};
        }
        pay_ *= d.payda_;
        payda_ *= d.pay_;
        sadelestir();
        return *this;
    }

private:
    void sadelestir() {
        if (payda_ < 0) {
            pay_ = -pay_;
            payda_ = -payda_;
        }
        int g = std::gcd(pay_, payda_);
        pay_ /= g;
        payda_ /= g;
    }
    int pay_;
    int payda_;
};

Kesir operator+(Kesir a, const Kesir& b) { return a += b; }
Kesir operator-(Kesir a, const Kesir& b) { return a -= b; }
Kesir operator*(Kesir a, const Kesir& b) { return a *= b; }
Kesir operator/(Kesir a, const Kesir& b) { return a /= b; }

bool operator==(const Kesir& a, const Kesir& b) {
    return a.pay() == b.pay() && a.payda() == b.payda();
}
bool operator!=(const Kesir& a, const Kesir& b) { return !(a == b); }
bool operator<(const Kesir& a, const Kesir& b) {
    return a.pay() * b.payda() < b.pay() * a.payda();
}
bool operator>(const Kesir& a, const Kesir& b) { return b < a; }
bool operator<=(const Kesir& a, const Kesir& b) { return !(b < a); }
bool operator>=(const Kesir& a, const Kesir& b) { return !(a < b); }

std::ostream& operator<<(std::ostream& os, const Kesir& k) {
    os << k.pay();
    if (k.payda() != 1) {
        os << '/' << k.payda();
    }
    return os;
}

int main() {
    Kesir a{3, 4};
    Kesir b{5, 6};
    std::cout << a << " + " << b << " = " << a + b << '\n';
    std::cout << a << " - " << b << " = " << a - b << '\n';
    std::cout << a << " * " << b << " = " << a * b << '\n';
    std::cout << a << " / " << b << " = " << a / b << '\n';
    std::cout << "2 + a = " << 2 + a << ", -a = " << -a << '\n';

    std::cout << std::boolalpha;
    std::cout << "a > b: " << (a > b) << ", a <= b: " << (a <= b)
```

```
              << ", a != b: " << (a != b) << '\n';

    std::vector<Kesir> liste{{1, 2}, {1, 3}, {3, 4}, {-1, 6}, {2, 3}};
    std::sort(liste.begin(), liste.end());
    std::cout << "Sıralı:";
    for (const Kesir& k : liste) {
        std::cout << ' ' << k;
    }
    std::cout << '\n';
    Kesir toplam = std::accumulate(liste.begin(), liste.end(), Kesir{});
    std::cout << "Toplam: " << toplam << '\n';
    std::cout << "En büyük: " << *std::max_element(liste.begin(), liste.end())
              << '\n';
    return 0;
}
```

```
ÇIKTI
3/4 + 5/6 = 19/12
3/4 - 5/6 = -1/12
3/4 * 5/6 = 5/8
3/4 / 5/6 = 9/10
2 + a = 11/4, -a = -3/4
a > b: false, a <= b: true, a != b: true
Sıralı: -1/6 1/3 1/2 2/3 3/4
Toplam: 25/12
En büyük: 3/4
```

Sonuçları kontrol edelim: `3/4 + 5/6` ortak paydada `9/12 + 10/12 = 19/12`'dir; bölme `3/4 * 6/5 = 9/10` verir. `2 + a` üye olmayan `operator+` sayesinde çalıştı: `2` önce `Kesir{2}`'ye dönüştü. Sıralı listenin toplamı `(-2 + 4 + 6 + 8 + 9) / 12 = 25/12`'dir. Sadeleştirme ve sıfıra bölme denetimi gibi ayrıntılar sınıfın içinde kaldı; kullanıcı `Kesir`'i `int` kadar rahat kullanıyor.

32.8 İndeks ve Fonksiyon Çağrı Operatörleri

Köşeli parantez `[]` ve çağrı parantezi `()` operatörleri yalnızca üye fonksiyon olabilir. Bunlar bir sınıfın nesnelerini dizi ya da fonksiyon gibi kullanabilmemizi sağlar.

operator[]

`vector` ve `std::string` için `v[i]` yazabilmemizin nedeni budur (Bölüm 26, 27). Önemli olan, `operator[]`'in **referans döndürmesidir**: `v[i] = 5;` ifadesi, dönen referansın gösterdiği elemana yazar. Bölüm 28'de gördüğümüz "referans döndürme" tekniğinin en yaygın kullanımı budur. Değer döndürseydik atamanın sol tarafına yazabileceğimiz bir şey (lvalue) elde edemezdik.

`const` nesnelerde de kullanılabilsin diye genellikle iki sürüm yazılır: `const` olmayan sürüm değiştirilebilir referans, `const` sürüm `const` referans (ya da değer) döndürür. Derleyici, nesnenin `const` olup olmamasına bakarak doğrusunu seçer.

operator()

`()` operatörü herhangi sayıda parametre alabilir. İki yaygın kullanımı vardır. Birincisi, `[]` yalnızca tek indeks alabildiği için çok boyutlu erişimi `t(satir, sutun)` biçiminde yazmaktır. İkincisi, nesneyi fonksiyon gibi çağırmaktır; bu tür nesnelere **fonksiyon nesnesi** (function object, functor) denir. İkisini de gösterelim:

tabloIndeks.cpp

```cpp
#include <iostream>
#include <vector>

class Tablo {
public:
    Tablo(std::size_t satir, std::size_t sutun)
        : sutun_{sutun}, veri_(satir * sutun) {}

    int& operator()(std::size_t s, std::size_t t) {
        return veri_[s * sutun_ + t];
    }
    const int& operator()(std::size_t s, std::size_t t) const {
        return veri_[s * sutun_ + t];
    }
    int& operator[](std::size_t i) { return veri_.at(i); }
    const int& operator[](std::size_t i) const { return veri_.at(i); }
    std::size_t boyut() const { return veri_.size(); }

private:
    std::size_t sutun_;
    std::vector<int> veri_;
};

int toplam(const Tablo& t) {
    int s = 0;
    for (std::size_t i = 0; i < t.boyut(); ++i) {
        s += t[i];
    }
    return s;
}

int main() {
    Tablo t(2, 3);
    for (std::size_t i = 0; i < 2; ++i) {
        for (std::size_t j = 0; j < 3; ++j) {
            t(i, j) = static_cast<int>(i * 10 + j);
        }
    }
    std::cout << "t(1, 2) = " << t(1, 2) << '\n';
    std::cout << "t[4] = " << t[4] << '\n';
    t[0] = 100;
    std::cout << "t(0, 0) = " << t(0, 0) << '\n';
    std::cout << "toplam = " << toplam(t) << '\n';
    return 0;
}
```

ÇIKTI

```
t(1, 2) = 12
t[4] = 11
t(0, 0) = 100
toplam = 136
```

`Tablo` iki satır, üç sütunluluk veriyi tek bir `vector`'de düz biçimde tutar. `t(i, j)` iki indeksi birleştirip tek indekse çevirir; `t[4]` ise düz sırada beşinci elemandır (`1 * 3 + 1` konumu, değeri 11). `veri_(satir * sutun)` ifadesinde **parantez** kullandığımıza dikkat edelim: `vector<int>` için süslü parantez eleman listesi olarak yorumlanırdı (`vector<int>{6}` tek elemanlı vektördür); boyut belirtmek için yuvarlak parantez gerekir. `toplam` fonksiyonu `const Tablo&` aldığı için `t[i]` ifadesi `const` sürümü çağırdı. `[]` içinde `at`

kullandığımız için sınır dışı bir indeks `std::out_of_range` istisnası fırlatır (Bölüm 34); doğrudan `veri_[i]` yazsaydık sınır denetimi olmazdı.

Fonksiyon nesneleri

Durum taşıyan bir fonksiyon yazmak istediğimizde, `operator()`'li bir sınıf çözüm sunar. Aşağıdaki `Esik` nesnesi içinde bir eşik değeri saklar ve çağrıldığında sayının eşikten büyük olup olmadığını söyler:

esikNesnesi.cpp

```
#include <algorithm>
#include <iostream>
#include <vector>

class Esik {
public:
    explicit Esik(int deger) : deger_{deger} {}
    bool operator()(int sayi) const { return sayi > deger_; }

private:
    int deger_;
};

int main() {
    Esik onaBuyuk{10};
    std::cout << std::boolalpha;
    std::cout << onaBuyuk(5) << ' ' << onaBuyuk(12) << '\n';

    std::vector<int> v{4, 15, 8, 23, 42, 7};
    long adet = std::count_if(v.begin(), v.end(), Esik{10});
    std::cout << "10'dan büyük: " << adet << '\n';
    return 0;
}
```

```
ÇIKTI
false true
10'dan büyük: 3
```

`onaBuyuk(12)` ifadesi `onaBuyuk.operator()(12)` çağrısıdır. `std::count_if` algoritması bir koşul fonksiyonu ister; ona `Esik{10}` verdik ve algoritma her eleman için bu nesneyi fonksiyon gibi çağırdı. C'de aynı iş, eşik değeri taşıyamayan bir fonksiyon işaretçisiyle (Bölüm 12) ya da global değişkenle yapılırdı. Fonksiyon nesneleri bu sınırı kaldırır. C++11'den beri aynı işi çoğu zaman **lambda ifadeleri** kısaca yazar; onları Bölüm 36'da göreceğiz ve derleyicinin aslında her lambda için bu türden isimsiz bir sınıf ürettiğini anlatacağız.

32.9 Artırma ve Azaltma Operatörleri

`++` ve `--` operatörlerinin her biri iki biçimde vardır: **önek** (`++x`) ve **sonek** (`x++`). Davranışları farklıdır (Bölüm 5): önek önce artırır ve yeni değeri verir, sonek önce eski değeri saklar, sonra artırır ve eski değeri verir. İki ayrı fonksiyon yazmamız gerekir, ama adları da aynıdır: `operator++`. Derleyici ikisini nasıl ayırt eder? Sonek sürüme **kullanılmayan bir `int` parametre** ekleriz; bu parametre yalnızca ayrım içindir ve çağrıda değer taşımaz.

artirma.cpp

```cpp
#include <iostream>

class Sayac {
public:
    explicit Sayac(int deger = 0) : deger_{deger} {}
    int deger() const { return deger_; }

    Sayac& operator++() {
        ++deger_;
        return *this;
    }
    Sayac operator++(int) {
        Sayac eski{*this};
        ++deger_;
        return eski;
    }

private:
    int deger_;
};

int main() {
    Sayac s{5};
    Sayac onek = ++s;
    std::cout << "önek : dönen " << onek.deger() << ", s " << s.deger() << '\n';
    Sayac sonek = s++;
    std::cout << "sonek: dönen " << sonek.deger() << ", s " << s.deger()
              << '\n';
    ++++s;
    std::cout << "++++s: " << s.deger() << '\n';
    return 0;
}
```

ÇIKTI

```
önek : dönen 6, s 6
sonek: dönen 6, s 7
++++s: 9
```

Önek sürüm nesneyi artırır ve `*this`'i **referansla** döndürür; bu yüzden `++++s` (iki kez artırma) çalışır ve `s` 7'den 9'a çıkar. Sonek sürüm ise artırmadan önce nesnenin kopyasını alır, nesneyi artırır ve kopyayı **değerle** döndürür. Sonek sürümün bir kopya üretmesi gerekir; önek bundan kaçınır. Bölüm 24'te `for` döngülerinde `++i` yazmamızın nedeni budur: sayı tiplerinde fark yoktur, ama yineleyiciler ve büyük nesneler için önek daha ucuzdur. Tipik kural: sonucu kullanmayacaksak her zaman önek yazarız.

`--` operatörü aynı kalıpla yazılır. Genellikle `++` tanımlandığında `--` da tanımlanır, çünkü kullanıcı ikisinin birlikte olmasını bekler.

32.10 Atama Operatörleri

Bölüm 30'da yazdığımız kopya atama `operator=`, bir operatör aşırı yüklemesidir ve yalnızca üye fonksiyon olabilir. Biz yazmazsak derleyici üyeleri tek tek kopyalayan bir sürümünü üretir. Üyeler `int`, `string`, `vector` gibi kendi kopyasını doğru yapan tiplerse bu yeterlidir; `Kesir` ve `Vektor2D` bu yüzden ayrıca `operator=` yazmadı. Sınıf `new` ile bellek yönetiyorsa derin kopyayı kendimiz yazarız (üçler kuralı, Bölüm 30); taşıma atamasını Bölüm 39'da göreceğiz.

Bileşik atama operatörleri (`+=`, `-=` ...) ise bu bölümde yazdığımız sıradan operatörlerdir; derleyici onları `+`'tan üretmez, biz `+`'ı onlardan türetiriz. İkisinde de `*this`'i referansla (`T&`) döndürürüz; böylece `a = b = c` ve `(a += b) += c` zincirleri çalışır.

32.11 Aşırı Yüklenmemesi Gerekenler ve İyi Uygulamalar

Dil, birçok operatörün aşırı yüklenmesine izin verir; ama izin verilen her şeyi yapmak akıllıca değildir. Pratik kurallar şunlardır.

Anlam tanıdık olsun. İki vektörü `+` ile toplamak, iki kesiri `<` ile karşılaştırmak herkesin beklediği şeydir. `Kitap + Kitap` ifadesinin ne yapacağı ise belirsizdir; böyle yerde `birlestir` gibi adı olan bir fonksiyon yazalım. Operatörlerin standart anlamını koruyalım: `==` simetrik olmalı, `+` operandlarını değiştirmemeli, `<<` yazı yazmalı.

İlişkili operatörleri birlikte tanımlayalım ve tutarlı kalalım. `==` ile `!=`, `<` ile diğer karşılaştırmalar, `+` ile `+=`, önek ile sonek `++` birlikte beklenir. `a += b` ile `a = a + b` aynı sonucu vermelidir; `+`'ı `+=` üzerinden yazmak bunu kendiliğinden sağlar.

Doğru dönüş tipini seçelim. Yeni değer üretenler (`+ - * /`) değer döndürür; yerel nesneye referans döndürmek sarkan referanstır (Bölüm 28). Nesneyi değiştirenler (`+=`, önek `++`, `=`) `*this`'e referans döndürür. Karşılaştırmalar `bool` döndürür.

`explicit` ile örtük dönüşümü denetleyelim. `Kesir(int pay = 0, int payda = 1)` sayesinde `2 + a` yazabildik. Bu rahatlık istenmiyorsa yapıcıyı `explicit` ile işaretleriz; `Esik` ve `Sayac` sınıflarında bunu yaptık, çünkü `int`'ten bu tiplere sessiz dönüşüm anlamsız olurdu.

Yüklenmemesi gereken operatörler

İki grup operatör özel dikkat ister:

- **`&&` ve `||`:** Yerleşik hâlleri **kısa devre** (short-circuit) yapar; soldaki işlenen sonucu belirliyorsa sağdakini hiç hesaplamaz (Bölüm 7). Aşırı yüklenmiş `&&` ve `||` ise sıradan fonksiyondur; iki argüman da hesaplanır. Bunu görelim:

kisaDevre.cpp

```
#include <iostream>

struct Deger {
    bool v;
};

bool operator&&(Deger a, Deger b) { return a.v && b.v; }

bool yanlis() {
    std::cout << "yanlis() çağrıldı\n";
    return false;
}

bool dogru() {
    std::cout << "dogru() çağrıldı\n";
    return true;
}

int main() {
    std::cout << "Yerleşik &&:\n";
```

```
    bool a = yanlis() && dogru();
    std::cout << "Aşırı yüklenmiş &&:\n";
    bool b = Deger{yanlis()} && Deger{dogru()};
    std::cout << a << ' ' << b << '\n';
    return 0;
}
```

```
ÇIKTI
Yerleşik &&:
yanlis() çağrıldı
Aşırı yüklenmiş &&:
yanlis() çağrıldı
dogru() çağrıldı
0 0
```

Yerleşik `&&` yalnızca `yanlis()`'i çağırdı; `dogru()` hiç çalışmadı. Aşırı yüklenmiş sürümde ise ikisi de çağrıldı, çünkü fonksiyon çağrılmadan önce iki argümanın da hesaplanması gerekir. Koda bakan biri kısa devre bekleyeceği için bu ayrım ince hatalara yol açar. `&&` ve `||`'u aşırı yüklemeyelim.

- **Virgül `,` ve adres alma `&`:** Aşırı yüklemek mümkündür, ama beklenmedik davranışlar doğurur; kod `&x`'in nesnenin gerçek adresini verdiğini varsayar.

> **◆ Kritik Nokta · Operatörler fonksiyondur**
>
> Aşırı yüklenmiş operatör sıradan bir fonksiyondur ve yerleşik sürümün özel garantilerinden (kısa devre gibi) yararlanamaz. Anlamını yerleşik operatörlere benzetmek bizim sorumluluğumuzdur.

32.12 Üç Dilde Operatör Aşırı Yükleme

Üç Dilde · Operatör aşırı yükleme *C · C++ · C#*

C: Operatör aşırı yükleme yoktur. Operatörler yalnızca yerleşik tipler için çalışır; kendi `struct`'ımız için `topla(a, b)` gibi adlı fonksiyonlar yazarız:

```
Kesir c = kesir_topla(a, b);
```

C++: Operatör `operator+` adlı bir fonksiyon olarak tanımlanır; üye ya da üye olmayan olabilir:

```
Kesir operator+(Kesir a, const Kesir& b) { return a += b; }
Kesir c = a + b;
```

C#: Operatörler her zaman `public static` yazılır ve en az bir parametresi kendi tipi olmalıdır; üye olma seçeneği yoktur:

```
public static Kesir operator +(Kesir a, Kesir b) =>
    new Kesir(a.Pay * b.Payda + b.Pay * a.Payda, a.Payda * b.Payda);
```

C# operatör aşırı yüklemeyi C++'tan almış, ama kuralları sadeleştirmiştir. C# sınıflarını Bölüm 49'da ayrıntılı işleyeceğiz; şimdilik farkları şöyle özetleyebiliriz:

Konu	C++	C#
Tanım biçimi	Üye ya da üye olmayan `operator+`	Yalnızca `public static ... operator +`
`+=`	Ayrıca tanımlanır	`+` tanımlanınca `a += b`, `a = a + b` olarak otomatik çalışır
`++` / `--`	Önek ve sonek ayrı yazılır	Tek tanım, hem önek hem sonek için kullanılır
`==` ve `!=`	Birbirinden bağımsız	Çift hâlinde tanımlanmalıdır (biri varsa öteki zorunlu); `Equals` ve `GetHashCode` da önerilir
`[]`	`operator[]`	Operatör değil, **indeksleyici** (`this[int i]`)
`()`	`operator()` ile fonksiyon nesnesi	Yüklenemez; yerine temsilciler (delegate) kullanılır
`&&`, `||`	Yüklenebilir (kısa devre kaybolur)	Doğrudan yüklenmez; `&`, `|` ve `true` / `false` operatörlerinden türetilir
Çıktı	`operator<<` ile akışa yazma	Akış operatörü kullanılmaz; `ToString()` metodu ezilir

Özellikle son satıra dikkat edelim: C#'ta `Console.WriteLine(k)` yazdığımızda nesnenin `ToString()` metodu çağrılır, bu yüzden `<<` yazmaya gerek yoktur. C'de ise bu özelliğin hiçbiri yoktur; kütüphane tasarımcıları `string_concat`, `vec2_add` gibi ön ekli adlarla bu eksiği kapatır.

Bölüm Özeti

- Operatör aşırı yükleme, `operator+` gibi adlı bir fonksiyonla bir operatörün kendi sınıfımız için ne yapacağını belirlemektir. `a + b` ifadesi `a.operator+(b)` ya da `operator+(a, b)` çağrısına dönüşür.
- Yeni operatör icat edilemez; işlenen sayısı, öncelik ve birleşme yönü değişmez. En az bir işlenen kullanıcı tanımlı tip olmalıdır.
- `::`, `.`, `.*`, `?:` ve `sizeof` aşırı yüklenemez. `=`, `[]`, `()` ve `->` yalnızca üye fonksiyon olarak yazılır.
- Üye operatörde sol işlenen `*this`'tir. Nesneyi değiştirmeyen üye operatörler `const` olmalı, sağ işlenen `const T&` alınmalıdır.
- `+`, `-`, `*`, `/`, `==`, `<` gibi ikili operatörleri üye olmayan fonksiyon olarak yazarız ki sol işlenen de örtük dönüşümden yararlansın (`2 * v`, `2 + a`). `+=` gibi nesneyi değiştirenler üyedir.
- `+` operatörünü `+=` üzerinden, `!=`, `>`, `<=`, `>=` operatörlerini `==` ve `<` üzerinden yazmak mantığı tek yerde tutar.
- `operator<<` ve `operator>>` üye olmayan fonksiyonlardır; akışı referansla alır ve aynı akışı referansla döndürür, böylece zincirleme çalışır. Özel üyelere erişmek için `friend` ya da erişimci fonksiyonlar kullanılır.
- `operator[]` referans döndürür, `const` ve `const` olmayan iki sürüm yazılır. `operator()` çok boyutlu erişim ve fonksiyon nesneleri için kullanılır.
- Sonek `++` / `--` kullanılmayan bir `int` parametresiyle ayırt edilir ve kopya döndürür; önek sürüm daha ucuzdur ve sonuç kullanılmıyorsa tercih edilir.
- Kopya atama `operator=` üyedir; derleyici yazmazsa üyeleri kopyalar. `+` değer, `+=` ve önek `++` `*this` referansı döndürür. `&&` ve `||` aşırı yüklenirse kısa devre kaybolur.
- C'de operatör aşırı yükleme yoktur. C#'ta `public static` operatör yazılır; `+=` otomatiktir, `[]` indeksleyiciyle, çıktı ise `ToString()` ile sağlanır.

Alıştırmalar

1. Aşağıdakilerden hangisi aşırı yüklenebilir bir operatör **değildir**?

   A) `[]`
   B) `()`
   C) `::`
   D) `->`
   E) `,`

2. `std::cout << nesne` ifadesini mümkün kılan `operator<<` neden üye olmayan bir fonksiyon olarak yazılır?

   A) Üye operatörler `friend` olamaz.
   B) Sol işlenen `std::ostream`'dir, bizim sınıfımız değildir; ona üye ekleyemeyiz.
   C) `<<` operatörü yalnızca üye olmayan olarak yüklenebilen bir operatördür.
   D) Akış nesneleri `const` olduğundan.
   E) Üye fonksiyonlar zincirleme yapamaz.

3. Sonek `operator++` önek olandan hangi yolla ayırt edilir?

   A) Dönüş tipinin `void` olmasıyla
   B) `const` olarak işaretlenmesiyle
   C) Kullanılmayan fazladan bir `int` parametresiyle
   D) Adının `operator_++` olmasıyla
   E) Üye olmayan fonksiyon olarak yazılmasıyla

4. `Kesir operator+(const Kesir&) const` üye fonksiyonunun dönüş tipi neden `Kesir&` değil `Kesir`'dir?

   A) Referans döndürmek yasaktır.
   B) `a + b` yeni bir nesne üretir; yerel nesneye referans döndürmek sarkan referans olurdu.
   C) `Kesir&` döndüren fonksiyon `const` olamaz.
   D) Zincirleme yazım yalnızca değer döndürenlerde çalışır.
   E) Derleyici referans döndüren `operator+`'ı kabul etmez.

5. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>

struct T {
    int v;
    T operator+(const T& d) const {
        std::cout << "+(" << v << "," << d.v << ") ";
        return T{v + d.v};
    }
};

int main() {
    T a{1}, b{2}, c{3};
    T s = a + b + c;
    std::cout << s.v << '\n';
    return 0;
}
```

6. Aşağıdaki programın çıktısı nedir?

```cpp
#include <iostream>

class Sayac {
public:
    explicit Sayac(int d) : d_{d} {}
    Sayac& operator++() { ++d_; return *this; }
    Sayac operator++(int) { Sayac e{*this}; ++d_; return e; }
    int deger() const { return d_; }
private:
    int d_;
};

int main() {
    Sayac s{1};
    int a = (s++).deger();
    int b = (++s).deger();
    std::cout << a << ' ' << b << ' ' << s.deger() << '\n';
    return 0;
}
```

7. Aşağıdaki kodda derleme hatası vardır. Hatayı bulun ve düzeltin.

```cpp
class Kesir {
public:
    Kesir(int p, int q) : pay_{p}, payda_{q} {}
    bool operator<(const Kesir& d) { return pay_ * d.payda_ < d.pay_ * payda_; }
private:
    int pay_;
    int payda_;
};

bool kucukMu(const Kesir& a, const Kesir& b) { return a < b; }
```

8. Aşağıdaki kodun son satırı neden derlenmez? Düzeltin.

```cpp
#include <iostream>

struct Nokta {
    int x, y;
};

void operator<<(std::ostream& os, const Nokta& p) {
    os << '(' << p.x << ", " << p.y << ')';
}

int main() {
    Nokta p{1, 2};
    std::cout << "Nokta: " << p << '\n';
    return 0;
}
```

9. Aşağıdaki ifadelerin doğru ya da yanlış olduğunu belirtin ve gerekçelendirin. (a) `operator[]` yalnızca üye fonksiyon olarak yazılabilir. (b) `**` ile üs alma operatörü tanımlanabilir. (c) `int + int` işleminin anlamı `operator+` tanımlanarak değiştirilebilir. (d) Aşırı yüklenmiş `&&` her zaman kısa devre yapar.

10. `operator+`'ın `operator+=` üzerinden yazılmasının iki yararını ve ikili operatörlerin üye olmayan fonksiyon olarak yazılmasının bir nedenini açıklayın.

11. İki sayı alan `Sure(int saat, int dakika)` sınıfını yazın. Sınıf, süreyi dakika cinsinden saklasın. `+`, `==`, `<` ve `<<` operatörlerini tanımlayın; `<<` süreyi `2 sa 15 dk` biçiminde yazsın. `main`'de 1 sa 30 dk ile 0 sa 45 dk'yı toplayıp yazdırın ve `mola < ders` ile `ders + mola == Sure{2, 15}` ifadelerinin sonuçlarını `true` / `false` olarak yazdırın.
12. Gerçek ve sanal kısmı olan `Karmasik` sınıfını yazın. `+`, `*` ve `==` operatörlerini, ayrıca `1+2i` ya da `3-1i` biçiminde yazan `<<` operatörünü tanımlayın. `(1+2i) + (3-1i)` ve `(1+2i) * (3-1i)` sonuçlarını yazdırın. Çarpma kuralı `(a+bi)(c+di) = (ac-bd) + (ad+bc)i` biçimindedir.
13. 2x2 `double` matrisini temsil eden `Matris2` sınıfını yazın. Elemanlara `m(i, j)` ile erişilsin (hem `const` hem `const` olmayan sürüm), `*` matris çarpımını, `==` eşitliği yapsın, `<<` matrisi `[7 10; 15 22]` biçiminde yazsın. `{{1, 2}, {3, 4}}` matrisinin karesini ve birim matrisle çarpımının kendisine eşit olduğunu gösterin.
14. `Sayac` sınıfına hem önek hem sonek `++` ve `--` ile `+=(int)` operatörlerini ekleyin. `main`'de sayacı 10 ile başlatıp 5 ekleyin, sonek `--` ile eski değeri `eski` değişkenine alıp hem `eski`'yi hem sayacın yeni değerini yazdırın. Ardından `--s; ++s;` uygulayıp sayacı yazdırın.

BÖLÜM 33

Şablonlar (Templates)

BU BÖLÜMDE

- Genel programlama (generic programming) fikrini ve şablonun makro ile `void *` çözümlerinden farkını
- Fonksiyon şablonu yazmayı, tip çıkarımının (type deduction) nasıl çalıştığını ve çelişen çıkarımın nasıl çözüleceğini
- Sınıf şablonu yazmayı (`Kutu<T>`, `Yigin<T>`), üyeleri sınıf dışında tanımlamayı ve C++17 sınıf şablonu argüman çıkarımını
- Birden fazla şablon parametresini, tip olmayan şablon parametrelerini (`template<int N>`) ve varsayılan şablon argümanlarını
- Tam ve kısmi özelleştirmeyi (specialization)
- Şablonun derleme zamanında nasıl örneklendiğini (instantiation), tanımların neden başlık dosyasına yazıldığını ve uzun hata mesajlarını nasıl okuyacağımızı
- C# generics ile aralarındaki temel farkları

Bölüm 25'te aynı adlı fonksiyonları farklı parametre tipleriyle yazmayı (aşırı yükleme) öğrendik. Ancak iki sayıdan büyüğünü bulan fonksiyonu `int`, `double` ve `string` için yazdığımızda üç gövdenin birebir aynı olduğunu görürüz; değişen tek şey tiptir. Her yeni tip için kodu kopyalamak sıkıcıdır ve hataya açıktır: bir kopyayı düzeltip ötekini unutmak işten bile değildir.

Şablon (template), tipi bir yer tutucu yaparak kodu bir kez yazmamızı sağlar; derleyici gerektiğinde gerçek tiple kodu kendisi üretir. Bölüm 32'de operatör aşırı yüklemeyi öğrendik; şablonlarla birleşince `<` ya da `+` tanımlı her tipte çalışan kod yazabileceğiz. Bölüm 35'te göreceğimiz `vector`, `map`, `unique_ptr` gibi standart kütüphane araçlarının hepsi şablondur. Bu bölümde onların nasıl yazıldığını anlayacağız.

33.1 Neden Şablon? C'deki Çözümler

C'de tipten bağımsız kod yazmanın iki yolu vardı ve ikisinin de bedeli ağırdı. Birincisi **makro** idi. Bölüm 15'te `TAKAS` makrosunu yazmış, makronun argümanı birden fazla kez değerlendirdiğini görmüştük:

```
#define ENBUYUK(a, b) ((a) < (b) ? (b) : (a))

int x = ENBUYUK(i++, j++);   // i++ ya da j++ iki kez çalışabilir
```

İkincisi `void *` kullanmaktı (Bölüm 12). Tipi unutturup bayt sayısıyla çalışırız:

```
void takas(void *a, void *b, size_t n);   // her tip için çalışır, ama...
```

Bu yöntemde derleyici hiçbir tip denetimi yapamaz: `takas(&sayi, &metin, sizeof(int))` sessizce derlenir ve belleği bozar. Üçüncü yol, her tip için ayrı fonksiyon yazmaktır (`enBuyukInt`, `enBuyukDouble`...), ki bu da kod tekrarından başka bir şey değildir.

C++ şablonları iki sorunu birden çözer: kod tek yerde yazılır, ama derleyici her kullanımda tipleri bilir ve denetler. Üstelik şablonun ürettiği kod, elle yazılmış tipe özel koda eşdeğerdir; çalışma zamanında ek bedel yoktur.

Tanım: Şablon (template)

Tipleri (ve bazı değerleri) parametre olarak alan, derleyicinin kullanıldığı her parametre kümesi için gerçek bir fonksiyon ya da sınıf üretmesini sağlayan kod kalıbıdır. Tipi parametre olarak alan programlamaya genel programlama (generic programming) denir.

Tarihçe · Şablonların doğuşu

Şablonlar 1980'lerin sonu ve 1990'ların başında Bjarne Stroustrup'un tasarımıyla C++'a eklendi ve ilk ISO standardında (C++98) yerini aldı. Alexander Stepanov ve arkadaşlarının geliştirdiği Standart Şablon Kütüphanesi (STL; Bölüm 2'de değinmiştik) şablonların gücünü gösteren ilk büyük örnekti. Bugün standart kütüphanenin büyük bölümü şablonlardan oluşur.

33.2 Fonksiyon Şablonu

Fonksiyon şablonu, normal bir fonksiyonun başına bir **şablon başlığı** (`template <typename T>`) eklenerek yazılır. Başlık, "aşağıdaki fonksiyon `T` adlı bir tip parametresi alıyor" der. Aşağıdaki program, üç farklı kullanımı gösteriyor:

sablonFonk.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Sure {
public:
    Sure(int dakika) : dakika_{dakika} {}
    bool operator<(const Sure& d) const { return dakika_ < d.dakika_; }
    int dakika() const { return dakika_; }
private:
    int dakika_;
};

template <typename T>
T enBuyuk(T a, T b) {
    return (a < b) ? b : a;
}

template <typename T, typename U>
void ciftYaz(const T& ilk, const U& ikinci) {
    cout << ilk << " - " << ikinci << '\n';
}

int main() {
    cout << enBuyuk(3, 8) << '\n';
    cout << enBuyuk(2.5, 1.5) << '\n';
    cout << enBuyuk(string("elma"), string("armut")) << '\n';
```

```
    cout << enBuyuk<double>(3, 4.5) << '\n';
    cout << enBuyuk(Sure{45}, Sure{90}).dakika() << '\n';
    ciftYaz(42, "elma");
    ciftYaz(3.5, 'A');
    return 0;
}
```

```
ÇIKTI
8
2.5
elma
4.5
90
42 - elma
3.5 - A
```

Şablonun parçalarını sırayla inceleyelim:

- `template <typename T>` satırında `T` bir **tip adıdır**. İstediğimiz adı verebiliriz, ama gelenek `T`'dir. `typename` yerine `class` da yazılabilir; ikisi aynı işi görür.
- Fonksiyonu çağırdığımızda derleyici `T`'yi argümanlardan kendisi bulur. Buna **tip çıkarımı** (type deduction) denir: `enBuyuk(3, 8)` için `T = int`, `enBuyuk(2.5, 1.5)` için `T = double`.
- Tipi kendimiz de söyleyebiliriz: `enBuyuk<double>(3, 4.5)` çağrısında `T = double` olur ve `3`, `double`'a dönüştürülür.
- Birden fazla şablon parametresi olabilir (`ciftYaz`); her biri ayrı çıkarılır.
- Kendi sınıfımız da tip olabilir. `Sure` sınıfına `operator<` yazdığımız (Bölüm 32) için `enBuyuk(Sure{45}, Sure{90})` çalıştı.

> **Tanım: Örnekleme (instantiation)**
>
> Derleyicinin bir şablonu belirli tip argümanlarıyla gerçek bir fonksiyona ya da sınıfa dönüştürmesidir. `enBuyuk(3, 8)` çağrısı `enBuyuk<int>`'i, `enBuyuk(2.5, 1.5)` çağrısı `enBuyuk<double>`'ı üretir. Kullanılmayan şablon hiç kod üretmez.

Şablon, tipin gövdede kullanılan her işlemi desteklemesini bekler. `enBuyuk` yalnızca `<` kullandığı için `<` operatörü olan her tiple çalışır: `int` ve `double` için bu yerleşiktir, `string` için kütüphane tanımlamıştır, `Sure` için biz yazdık. Operatör aşırı yükleme ile şablonlar tam burada buluşur.

Tip çıkarımı ve çelişen çıkarım

`enBuyuk(3, 4.5)` yazarsak ne olur? İlk parametreden `T = int`, ikinciden `T = double` çıkar. Derleyici ikisini uzlaştırmaz; kendiliğinden `int`'i `double`'a çevirmez:

▸ cikarimCeliski.cpp

```
template <typename T>
T enBuyuk(T a, T b) {
    return (a < b) ? b : a;
}

int main() {
    auto sonuc = enBuyuk(3, 4.5);
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

cikarimCeliski.cpp: In function 'int main()':
cikarimCeliski.cpp:7:25: error: no matching function for call to 'enBuyuk(int, double)'
    7 |     auto sonuc = enBuyuk(3, 4.5);
      |                  ~~~~~~~^~~~~~~~
cikarimCeliski.cpp:2:3: note: candidate: 'template<class T> T enBuyuk(T, T)'
cikarimCeliski.cpp:2:3: note:   template argument deduction/substitution failed:
cikarimCeliski.cpp:7:25: note:   deduced conflicting types for parameter 'T' ('int' and
'double')
```

Son satır asıl ipucudur: `T` için çelişen iki tip çıkarıldı. İki çözüm vardır: tipi açıkça yazmak (`enBuyuk<double>(3, 4.5)`) ya da argümanları aynı tipe getirmek (`enBuyuk(3.0, 4.5)`).

▲ Sık Yapılan Hata · enBuyuk("elma", "armut") yazmayın

Sabit metinleri doğrudan verirsek `T`, `const char *` olur ve `<` iki **adresi** karşılaştırır, metinlerin içeriğini değil. Sonuç anlamsızdır. Metin karşılaştırmak için argümanları `string(...)` ile sarmalayalım ya da `enBuyuk<string>("elma", "armut")` yazalım. Aşağıda özelleştirme başlığında bu sorunun bir başka çözümünü göreceğiz.

◆ Kritik Nokta · Şablon bir fonksiyon değil, fonksiyon üretme tarifidir

`enBuyuk<int>` ile `enBuyuk<double>` birbirinden bağımsız iki ayrı fonksiyondur. Aynı `T` iki parametrede geçiyorsa argümanların tipi tam aynı olmalıdır. Şablon tanımı, onu kullanan koddan önce görülebilir olmalıdır; bu yüzden şablonlar pratikte başlık dosyalarına yazılır (bu konuya "Perde Arkası" başlığında döneceğiz).

33.3 Sınıf Şablonu

Fonksiyonlar gibi sınıflar da şablon olabilir. İçinde bir `int` tutan basit bir `Kutu` sınıfı düşünelim. `double` ya da `string` tutan kutu için sınıfı kopyalamak yerine, tutulan tipi şablon parametresi yaparız. Kullanırken tipi açıkça yazarız: `Kutu<int>`, `Kutu<string>`.

▸ kutu.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

template <typename T>
class Kutu {
public:
    Kutu(const T& deger) : deger_{deger} {}
    const T& al() const { return deger_; }
    void koy(const T& yeni);
    friend ostream& operator<<(ostream& os, const Kutu& k) {
        return os << "[" << k.deger_ << "]";
    }
private:
    T deger_;
};

template <typename T>
void Kutu<T>::koy(const T& yeni) {
    deger_ = yeni;
}
```

```
int main() {
    Kutu<int> sayi{42};
    Kutu<string> metin{"merhaba"};
    Kutu<double> oran{0.75};
    cout << sayi << " " << metin << " " << oran << '\n';

    sayi.koy(sayi.al() + 1);
    metin.koy("dunya");
    cout << sayi << " " << metin << '\n';

    Kutu otomatik{2.5};              // C++17: T = double çıkarılır
    cout << otomatik << '\n';
    return 0;
}
```

```
ÇIKTI
[42] [merhaba] [0.75]
[43] [dunya]
[2.5]
```

Dikkat edilecek noktalar:

- Sınıfın başına `template <typename T>` yazarız; sınıf içinde `T` herhangi bir tip gibi kullanılır.
- Üye fonksiyonu sınıfın dışında tanımlarsak şablon başlığını her seferinde tekrar yazar ve sınıf adını `Kutu<T>::` biçiminde belirtiriz. Sınıfın içinde `Kutu` yazmak zaten `Kutu<T>` anlamına gelir; orada `<T>` yazmaya gerek yoktur.
- `operator<<` Bölüm 32'deki kalıpla, sınıf içinde `friend` olarak tanımlandı. Her `Kutu<T>` örneği kendi `operator<<`'unu üretir.
- `Kutu otomatik{2.5};` satırında tip yazılmadı. C++17'den itibaren derleyici yapıcı argümanından `T = double` çıkarır. Buna **sınıf şablonu argüman çıkarımı** (CTAD, class template argument deduction) denir. Daha önce `Kutu<double>` yazmak zorundaydık.
- `Kutu<int>` ve `Kutu<string>` birbirinden bağımsız, birbirine atanamayan iki farklı tiptir.

> **İpucu · Şablonu önce somut tiple yazalım**
>
> Şablon yazarken önce `int` ile çalışan sıradan bir sınıf yazıp çalıştığından emin olmak, sonra `int`'leri `T` ile değiştirmek en kolay yoldur. Bir hata çıkarsa mesajı somut sınıfta okumak çok daha kolaydır.

Yığın şablonu: `Yigin<T>`

Sınıf şablonunun klasik örneği **yığın** (stack) veri yapısıdır: son giren ilk çıkar. Bunun, programın yerel değişkenlerini tutan bellek bölgesiyle yalnızca adı benzer. Tabaklar üst üste dizilir; en son konan tabak ilk alınır. Yığının işlemleri şunlardır: `ekle` (en üste koy), `cikar` (en üsttekini al), `ust` (en üsttekine bak), `bos` ve `boyut`.

C'de aynı yapıyı Bölüm 19'da bağlı liste ve dizi ile elle yazmıştık; her eleman tipi için ayrı kod gerekiyordu. Burada elemanları saklamak için `vector<T>` kullanacağız (Bölüm 26). Bellek yönetimi `vector`'e kalır, Bölüm 30'daki üçler kuralını düşünmemiz gerekmez.

```
▸ yiginSablon.cpp
#include <iostream>
#include <string>
#include <vector>
```

```
using namespace std;

template <typename T>
class Yigin {
public:
    void ekle(const T& eleman) { elemanlar_.push_back(eleman); }
    bool cikar(T& hedef) {
        if (elemanlar_.empty()) {
            return false;
        }
        hedef = elemanlar_.back();
        elemanlar_.pop_back();
        return true;
    }
    const T& ust() const { return elemanlar_.back(); }  // bos() değilken çağır
    bool bos() const { return elemanlar_.empty(); }
    int boyut() const { return static_cast<int>(elemanlar_.size()); }
private:
    vector<T> elemanlar_;
};

bool parantezDengeli(const string& metin) {
    Yigin<char> acik;
    for (char c : metin) {
        if (c == '(' || c == '[') {
            acik.ekle(c);
        } else if (c == ')' || c == ']') {
            char son;
            if (!acik.cikar(son)) {
                return false;
            }
            if ((c == ')' && son != '(') || (c == ']' && son != '[')) {
                return false;
            }
        }
    }
    return acik.bos();
}

int main() {
    Yigin<int> sayilar;
    sayilar.ekle(10);
    sayilar.ekle(20);
    sayilar.ekle(30);
    cout << "ust: " << sayilar.ust() << ", boyut: " << sayilar.boyut() << '\n';

    int deger;
    while (sayilar.cikar(deger)) {
        cout << deger << " ";
    }
    cout << "| bos mu: " << sayilar.bos() << '\n';

    cout << parantezDengeli("(a[1] + b) * (c)") << " ";
    cout << parantezDengeli("(a[1)]") << " ";
    cout << parantezDengeli("((a)") << '\n';
    return 0;
}
```

```
ÇIKTI
ust: 30, boyut: 3
30 20 10 | bos mu: 1
1 0 0
```

Aynı `Yigin` şablonu hem `int` hem `char` için kullanıldı; elle yazılsaydı iki sınıf gerekecekti. `Yigin<char>` ile parantez dengesi denetimi yığının en bilinen kullanımıdır: her açan paranteze `ekle`, her kapayana `cikar` uygularız. Çıkan açılış kapanışla eşleşmiyorsa, yığın boşken kapanış geldiyse ya da sonda yığın boş değilse ifade dengesizdir.

Boş yığından eleman almayı burada `bool` döndürerek çözdük. C++'ın bunun için daha doğal bir yolu vardır: istisna (exception). Bölüm 34'te göreceğiz. Standart kütüphanedeki hazır `std::stack`, `std::queue` ve `std::vector` şablonlarını Bölüm 35'te inceleyeceğiz; burada onların nasıl yazıldığını anlamak için kendimiz yazdık.

33.4 Birden Fazla Parametre, Tip Olmayan Parametreler

Bir şablon istediği kadar parametre alabilir. Parametreler tip olabileceği gibi, derleme zamanında bilinen bir **değer** de olabilir (`int`, `bool`, `size_t` gibi tamsayı tipli sabitler). Değer alan parametrelere **tip olmayan şablon parametresi** (non-type template parameter) denir. Bölüm 26'da gördüğümüz `std::array<int, 5>` tam olarak budur: ilk argüman tip, ikincisi eleman sayısıdır.

Ayrıca şablon parametrelerine **varsayılan argüman** verilebilir. Aşağıdaki program dört fikri birlikte gösteriyor:

▸ sabitDizi.cpp

```
#include <cstddef>
#include <iostream>
#include <string>
using namespace std;

template <typename A, typename B>
struct Cift {
    A birinci;
    B ikinci;
};

template <typename T, int N>
class SabitDizi {
public:
    SabitDizi() : veri_{} {}                 // elemanlar sıfırlanır
    T& operator[](int i) { return veri_[i]; }
    const T& operator[](int i) const { return veri_[i]; }
    int boyut() const { return N; }
    T toplam() const {
        T t{};
        for (int i = 0; i < N; ++i) {
            t += veri_[i];
        }
        return t;
    }
private:
    T veri_[N];
};

template <typename T, size_t N>
size_t elemanSayisi(const T (&)[N]) {
    return N;
}

template <typename T = int>
T yarisi(T x) {
    return x / 2;
}
```

```
int main() {
    Cift<string, int> kayit{"elma", 12};
    Cift<int, double> olcum{1, 36.6};
    cout << kayit.birinci << " " << kayit.ikinci << " | "
         << olcum.birinci << " " << olcum.ikinci << '\n';

    SabitDizi<int, 4> a;
    for (int i = 0; i < a.boyut(); ++i) {
        a[i] = (i + 1) * 10;
    }
    SabitDizi<double, 3> b;
    b[0] = 0.5;
    b[2] = 1.25;
    cout << a.toplam() << " " << b.toplam() << '\n';

    int notlar[] = {70, 85, 90, 55, 100, 40};
    char harfler[] = "merhaba";
    cout << elemanSayisi(notlar) << " " << elemanSayisi(harfler) << '\n';

    cout << yarisi(9) << " " << yarisi(9.0) << " " << yarisi<>(7) << '\n';
    return 0;
}
```

```
ÇIKTI
elma 12 | 1 36.6
100 1.75
6 8
4 4.5 3
```

Adım adım:

- `Cift<A, B>` iki ayrı tip alır; `Cift<string, int>` ile `Cift<int, double>` ayrı sınıflardır. Standart kütüphanedeki `std::pair` bu fikrin olgun hâlidir (Bölüm 35).
- `SabitDizi<T, N>` içinde `N` bir tip değil, sayıdır ve derleme zamanında bilinir. Bu yüzden `T veri_[N]` gibi sabit boyutlu bir dizi tanımlayabiliriz; dinamik bellek gerekmez.
- `elemanSayisi`, dizinin **referansını** alır: `const T (&)[N]`. Dizi referansla geçtiği için işaretçiye dönüşmez ve derleyici `T` ile `N`'yi dizinin tipinden çıkarır. Bölüm 10'daki `sizeof(dizi) / sizeof(dizi[0])` hilesi işaretçiye dönüşen parametrede çöküyordu; bu yöntem çökmez. İkinci çağrıda 8 çıktığına dikkat edelim: `"merhaba"` yedi harf artı sonlandırıcı `'\0'` olmak üzere sekiz elemanlı bir `char` dizisidir (Bölüm 11). C++17'de bu iş için kütüphane `std::size(dizi)` sunar (Bölüm 26).
- `template <typename T = int>` ile `T`'ye varsayılan tip verdik. Argümandan çıkarım yapılabildiği sürece varsayılan kullanılmaz: `yarisi(9.0)` için `T = double` olur. Varsayılan, yalnızca `T`'nin çıkarılamadığı ya da sınıf şablonunda hiç belirtilmediği durumlarda devreye girer. Boş `<>` "şablon argümanlarını kendin çıkar" demenin yoludur.

`SabitDizi<int, 3>` ile `SabitDizi<int, 4>` farklı tiplerdir; birini diğerine atamak derleme hatasıdır. Boyut tipin parçası olduğundan hata çalışma zamanında değil derleme sırasında yakalanır. Tip olmayan parametrenin değeri de sabit bir ifade olmalıdır:

```
int n = 5;
SabitDizi<int, n> d;          // hata: n sabit ifade değil
constexpr int m = 5;
SabitDizi<int, m> e;          // geçerli
```

▲ Sık Yapılan Hata · Şablon argümanı derleme zamanında bilinmelidir

`int n = 5;` ile tanımlanan değişken çalışma zamanında değişebileceği için şablon argümanı olamaz; gcc "the value of 'n' is not usable in a constant expression" der. Değişkeni `constexpr` (Bölüm 22) ya da `const` ile sabit bir değerle tanımlamak yeterlidir. Boyutu çalışma zamanında belirlenen yapılar için `vector` kullanırız.

33.5 Şablon Özelleştirme

Şablon her tip için aynı kodu üretir. Bazen belirli bir tip için farklı davranış isteriz. Önceki bölümdeki `enBuyuk("elma", "armut")` sorunu buna iyi bir örnektir: `const char *` için adres değil içerik karşılaştırılmalıdır. **Özelleştirme** (specialization), genel şablonun yanına belirli tip için ayrı bir tanım yazmaktır.

Tanım: Şablon özelleştirme (specialization)

Genel bir şablonun belirli tip argümanları için ayrıca yazılmış özel sürümüdür. Tüm parametreler sabitlenirse **tam özelleştirme** (`template <>`), parametrelerin bir kısmı genel bırakılıp bir desen verilirse **kısmi özelleştirme** (partial specialization) olur.

ozellestirme.cpp

```cpp
#include <cstring>
#include <iostream>
#include <string>
using namespace std;

template <typename T>
T enBuyuk(T a, T b) {
    return (a < b) ? b : a;
}

// tam özelleştirme: C metinleri adres yerine içerikle karşılaştırılır
template <>
const char* enBuyuk<const char*>(const char* a, const char* b) {
    return (strcmp(a, b) < 0) ? b : a;
}

template <typename T>
class Yazici {
public:
    void yaz(const T& x) const { cout << "deger: " << x << '\n'; }
};

// tam özelleştirme: bool için evet/hayır
template <>
class Yazici<bool> {
public:
    void yaz(const bool& x) const {
        cout << "deger: " << (x ? "evet" : "hayir") << '\n';
    }
};

// kısmi özelleştirme: her işaretçi tipi için
template <typename T>
class Yazici<T*> {
public:
    void yaz(T* p) const {
        if (p == nullptr) {
            cout << "bos isaretci\n";
```

```
        } else {
            cout << "isaretlenen " << *p << '\n';
        }
    }
};

int main() {
    cout << enBuyuk(3, 7) << '\n';
    cout << enBuyuk("elma", "armut") << '\n';

    Yazici<int>().yaz(5);
    Yazici<bool>().yaz(true);
    int x = 42;
    Yazici<int*>().yaz(&x);
    Yazici<double*>().yaz(nullptr);
    return 0;
}
```

```
ÇIKTI
7
elma
deger: 5
deger: evet
isaretlenen 42
bos isaretci
```

Derleyici, argümanlara en uygun tanımı seçer: `Yazici<bool>` tam özelleştirmeyi, `Yazici<int*>` ve `Yazici<double*>` kısmi özelleştirmeyi, `Yazici<int>` genel şablonu kullanır. `enBuyuk("elma", "armut")` çağrısında dizi argümanlar değer parametresine geçtiği için işaretçiye dönüşür ve `T = const char *` çıkar; böylece özelleştirilmiş sürüm çalışır.

Üç noktayı not edelim:

- Özelleştirme, genel şablondan sonra ve ilk kullanımdan önce yazılmalıdır.
- Fonksiyon şablonlarında kısmi özelleştirme yoktur. Fonksiyonlar için çoğu zaman özelleştirme yerine normal bir **aşırı yükleme** (Bölüm 25) yazmak daha öngörülebilirdir: `const char* enBuyuk(const char*, const char*)` gibi.
- Standart kütüphane de özelleştirme kullanır. `vector<bool>` özelleştirmesi her `bool`'u bir bit olarak saklar ve bu yüzden diğer `vector` türlerinden farklı davranan, sık eleştirilen bir istisnadır.

33.6 Perde Arkası: Örnekleme ve Başlık Dosyaları

Şablon bir fonksiyon değil, fonksiyon üretme tarifidir. Derleyici şablonu gördüğünde yalnızca söz dizimini denetler; gerçek kod, şablon belirli tip argümanlarıyla kullanıldığında üretilir. Her farklı tip kümesi ayrı bir fonksiyona ya da sınıfa karşılık gelir. Bunu sayaçlarla görebiliriz:

ornekleme.cpp

```
#include <iostream>
using namespace std;

template <typename T>
int cagriSayisi() {
    static int sayac = 0;
    return ++sayac;
}
```

```
template <typename T>
struct Kimlik {
    static int sonraki;
    int no;
    Kimlik() : no{sonraki++} {}
};

template <typename T>
int Kimlik<T>::sonraki = 1;

int main() {
    cout << cagriSayisi<int>() << " " << cagriSayisi<int>() << " "
         << cagriSayisi<double>() << '\n';

    Kimlik<int> a, b;
    Kimlik<char> c;
    cout << a.no << " " << b.no << " " << c.no << '\n';
    return 0;
}
```

```
ÇIKTI
1 2 1
1 2 1
```

`cagriSayisi<int>` iki kez çağrıldığı için sayaç 2'ye çıktı; `cagriSayisi<double>` ayrı bir fonksiyon olduğundan kendi `static` değişkeniyle 1'den başladı. Sınıfta da aynı: `Kimlik<int>` ve `Kimlik<char>` ayrı sınıflardır ve her birinin kendi `sonraki` üyesi vardır.

Bu çalışma biçiminin üç sonucu vardır.

Çalışma zamanı bedeli yoktur. Üretilen kod, elle yazılmış tipe özel koda eşdeğerdir; sanal fonksiyon çağrısındaki (Bölüm 31) dolaylılık burada yoktur. Şablon, derleme zamanında çözülen bir çok biçimlilik biçimidir; sanal fonksiyonlar ise çalışma zamanında çözülür. Karşılığında, çok sayıda farklı tip için örneklenen bir şablon çalıştırılabilir dosyayı büyütebilir ve derleme süresini uzatır.

Tanım, kullanıldığı yerde görünmelidir. Derleyici `kare<int>`'i üretebilmek için şablonun gövdesini görmek zorundadır. Sıradan fonksiyonlarda yaptığımız gibi bildirimi başlık dosyasına, tanımı `.cpp` dosyasına koyarsak (Bölüm 30) bağlayıcı hatası alırız:

```
// kare.hpp
#pragma once

template <typename T>
T kare(T x);
```

```
// kare.cpp
#include "kare.hpp"

template <typename T>
T kare(T x) {
    return x * x;
}
```

```
// main.cpp
#include <iostream>
#include "kare.hpp"
```

```
int main() {
    std::cout << kare(7) << '\n';
    return 0;
}
```

```
TERMİNAL
$ g++ -std=c++17 -Wall main.cpp kare.cpp -o p
/usr/bin/ld: /tmp/ccDqglwz.o: in function `main':
main.cpp:(.text+0xe): undefined reference to `int kare<int>(int)'
collect2: error: ld returned 1 exit status
```

`kare.cpp` derlenirken `kare<int>` hiç kullanılmadığı için derleyici onu üretmedi; `main.cpp` derlenirken ise gövde görünmediği için üretemedi. Bağlayıcı `kare<int>`'i hiçbir yerde bulamadı. Çözüm, şablonun tanımını da başlık dosyasına yazmaktır. Bu yüzden şablon kitaplıkları çoğu zaman yalnızca başlık dosyalarından oluşur. Sınıf şablonlarının üye fonksiyonları için de aynı kural geçerlidir.

> ◆ **Kritik Nokta · Şablonların tanımı başlık dosyasındadır**
>
> Fonksiyon şablonunun ve sınıf şablonunun üyelerinin gövdesi, onları kullanan her çeviri birimince görülebilmelidir. Pratik kural: şablonları `.hpp` dosyasına, tanımıyla birlikte yazın. Aynı şablon birden çok `.cpp` dosyasında örneklense bile bağlayıcı kopyaları birleştirir; bu yüzden tek tanım kuralı (ODR) çiğnenmiş olmaz.

Şablon, kullanılmadığı sürece tam denetlenmez. Derleyici gövdedeki `T`'ye bağlı ifadeleri örnekleme sırasında denetler. Bu yüzden şablon, hatalı bir tiple kullanılıncaya kadar derlenmiş görünebilir. Hatayı bir sonraki başlıkta inceleyelim.

33.7 Şablon Hata Mesajları

Şablonlarda en çok yorulduğumuz yer hata mesajlarıdır. `enBuyuk` şablonunu `<` operatörü olmayan bir sınıfla çağıralım:

▸ sablonHata.cpp

```
template <typename T>
T enBuyuk(T a, T b) {
    return (a < b) ? b : a;
}

class Nokta {
public:
    int x;
    int y;
};

int main() {
    Nokta p{1, 2};
    Nokta q{3, 4};
    enBuyuk(p, q);
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

sablonHata.cpp: In instantiation of 'T enBuyuk(T, T) [with T = Nokta]':
sablonHata.cpp:15:12:   required from here
sablonHata.cpp:3:15: error: no match for 'operator<' (operand types are 'Nokta' and 'Nokta')
    3 |     return (a < b) ? b : a;
      |            ~~~^~~~
```

Hata, `enBuyuk` şablonunun içinde (`a < b` satırında) çıkıyor; oysa yanlışı yapan `main`'deki çağrıdır. Derleyici bu yüzden iki bilgi verir: hatanın şablonun hangi satırında olduğunu ve hangi tiple örneklendiğinde olduğunu (`In instantiation of ... [with T = Nokta]`). Mesajı okurken önce ilk `error:` satırına, sonra `with T = ...` kısmına bakalım. Sorun çoğu zaman "bu tip bu operatörü desteklemiyor"dur. Çözüm, `Nokta` için `operator<` yazmaktır (Bölüm 32).

Şablon hataları üç nedenle uzar. Hata şablonun derinliklerinde çıktığı için derleyici oraya götüren çağrı zincirinin tamamını yazar. `<` gibi bir operatör için kütüphanedeki yüzlerce aşırı yüklenmiş aday sıralanır. Tipler kısaltılmadan yazılır: `Kutu<vector<string>>` gibi adlar bir satıra sığmaz. Burada küçük bir örnekle kaldık; `<vector>` ve `<map>` gibi başlıkları kullanan bir şablon çağrısında mesaj yüzlerce satıra çıkabilir. Alışkanlık: ilk `error:` satırını okuyup gerisine yalnızca gerektiğinde bakmak. Ek D'de sık görülen mesajları topladık.

Not · C++20: kavramlar (concepts)

C++20, şablon parametresine koşul koymayı sağlar. `template <std::totally_ordered T>` yazarsak `enBuyuk` yalnızca `<` , `>` , `==` gibi karşılaştırmaları destekleyen tiplerle çağrılabilir. Koşul sağlanmazsa derleyici, hatayı şablonun içinde değil çağrı satırında ve "constraints not satisfied" (koşullar sağlanmadı) gibi okunaklı bir ifadeyle bildirir. Bu kitap C++17 kullandığı için kavramları ayrıntılı işlemiyoruz:

```
#include <concepts>

template <std::totally_ordered T>
T enBuyuk(T a, T b) { return (a < b) ? b : a; }
```

▲ Sık Yapılan Hata · Şablon kodunda sık yapılan hatalar

- Üyeyi sınıf dışında tanımlarken `template <typename T>` başlığını ve `Kutu<T>::` önekini unutmak.
- Çelişen argüman tiplerini (`enBuyuk(3, 4.5)`) tek `T` parametresine vermek.
- Şablon tanımını `.cpp` dosyasına koyup bağlayıcı hatası almak.
- Gövdede kullanılan işlemi (`<` , `+` , `<<`) desteklemeyen bir tipi şablona vermek.

Üç Dilde · Şablon, makro ve generics *C · C++ · C#*

C'de tipten bağımsız kod yazmanın yolları makro ve `void *` idi (Bölüm 12 ve 15). Makro tip denetimi yapmaz ve argümanı birden fazla kez değerlendirebilir; `void *` ise tipi unutturur. İkisinde de hata derleme zamanında yakalanmaz.

```
#define ENBUYUK(a, b) ((a) < (b) ? (b) : (a))
void takas(void *a, void *b, size_t n);
```

C++ şablonları tip denetimli, çalışma zamanı bedelsiz ve her tip için ayrı kod üreten bir çözümdür. **C#** aynı amaçla **generics** (genel tipler) sunar; sözdizimi benzer (`List<int>` , `Kutu<T>`), ama çalışma biçimi farklıdır:

```
T EnBuyuk<T>(T a, T b) where T : IComparable<T>
{
    return a.CompareTo(b) < 0 ? b : a;
}
```

	C++ şablonları	C# generics
Ne zaman çözülür	Derleme zamanında, her tip için ayrı kod üretilir	Derleme zamanında denetlenir; çalışma zamanında JIT derleyicisi tip için kodu oluşturur (başvuru tipleri kod paylaşır)
Tip üzerindeki işlemler	Gövdede ne kullanılırsa tip onu desteklemelidir; kısıt örneklemede denetlenir	Kısıt (`where T : ...`) bildirilmeden `T` üzerinde yalnızca `object` işlemleri yapılabilir
Hata zamanı	Örnekleme sırasında, uzun mesajlarla	Genel kod tanımlanırken, kısa mesajlarla
Tip olmayan parametre	Var (`template <int N>`)	Yok
Özelleştirme	Tam ve kısmi	Yok
Operatör kullanımı	`a + b` doğrudan yazılır	Kısıt olmadan yazılamaz (`INumber<T>` gibi arayüzler gerekir)

C++'ta `T` 'nin ne yapabildiğini şablon gövdesi dolaylı olarak dayatır; C#'ta ise kısıt (`where`) bunu açıkça bildirir. C++20 kavramları, bu fikri C++'a getirir. C# generics'i, koleksiyonlarla birlikte Bölüm 54'te ayrıntılı işleyeceğiz.

Bölüm Özeti

- Şablon (template), tipi parametre yapan bir kalıptır; derleyici kullanılan her tip kümesi için gerçek fonksiyon ya da sınıf üretir (örnekleme).
- Fonksiyon şablonu `template <typename T>` başlığıyla yazılır; `typename` yerine `class` da kullanılabilir.
- Tip çıkarımı argümanlardan yapılır. Aynı `T` için çelişen tipler çıkarsa derleme hatası olur; çözüm tipi açıkça yazmak (`enBuyuk<double>(3, 4.5)`) ya da argümanları aynı tipe getirmektir.
- Şablon gövdesinde kullanılan her işlem (`<` , `+` , `<<`) tip tarafından desteklenmelidir; operatör aşırı yükleme şablonlarla birlikte çalışır.
- Sınıf şablonlarında `Kutu<int>` ve `Kutu<string>` birbirinden bağımsız tiplerdir. Üyeler sınıf dışında `template <typename T>` başlığı ve `Kutu<T>::` ile tanımlanır.
- C++17 ile yapıcı argümanından tip çıkarılabilir: `Kutu k{2.5};` (CTAD).
- `Yigin<T>` gibi veri yapıları, tipten bağımsız bir kez yazılır; eleman saklamak için `vector<T>` kullanmak bellek yönetimini kütüphaneye bırakır.
- Şablon birden fazla tip parametresi, tip olmayan parametre (`template <typename T, int N>`) ve varsayılan argüman (`typename T = int`) alabilir. Tip olmayan argüman derleme zamanında bilinen sabit olmalıdır.
- Dizi referansı alan şablon (`const T (&)[N]`), dizinin eleman sayısını derleme zamanında verir.
- Özelleştirme belirli tip için ayrı tanım yazar: tam (`template <>`) ve kısmi (`Yazici<T*>`). Fonksiyonlar için kısmi özelleştirme yoktur; çoğu zaman aşırı yükleme tercih edilir.

- Her örnekleme ayrı bir fonksiyon ya da sınıf üretir; `static` üyeler de ayrı tutulur. Şablonun çalışma zamanı bedeli yoktur, ama kod şişmesi ve derleme süresi artışı olabilir.
- Şablonun tanımı kullanıldığı yerde görünmelidir; bu yüzden gövdeler başlık dosyasına yazılır. Aksi hâlde bağlayıcı `undefined reference` hatası verir.
- Şablon hata mesajlarında önce ilk `error:` satırı ve `[with T = ...]` kısmı okunur. C++20 kavramları bu mesajları kısaltır.
- C# generics'te kısıtlar (`where`) açıkça bildirilir, tip olmayan parametre ve özelleştirme yoktur; C'de makro ve `void *` aynı ihtiyacı tip denetimi olmadan karşılar.

Alıştırmalar

1. Aşağıdakilerden hangisi şablon için doğrudur?

   A) Şablon, derleme sırasında hiçbir koda dönüşmeyen yorum satırıdır.
   B) Şablon, derleyicinin her tip için kod üretmesini sağlayan bir tariftir.
   C) Şablon, çalışma zamanında tipi seçen sanal bir fonksiyondur.
   D) Şablon yalnızca sınıflar için yazılabilir.
   E) Şablonlar yalnızca `int` ve `double` ile çalışır.

2. Aşağıdaki kod için ne olur?

```
template <typename T>
T enBuyuk(T a, T b) { return (a < b) ? b : a; }

auto s = enBuyuk(3, 4.5);
```

   A) `s` değişkeni 4.5 olur.
   B) `s` değişkeni 4 olur.
   C) Derleme hatası: `T` için çelişen tipler çıkar.
   D) Çalışma zamanı hatası oluşur.
   E) Uyarı verir ve 3'ü döndürür.

3. Bu programın çıktısı nedir?

```
#include <iostream>
#include <string>
using namespace std;

template <typename T>
T ikiKat(T x) { return x + x; }

int main() {
    cout << ikiKat(4) << " " << ikiKat(1.5) << " "
         << ikiKat(string("ab")) << '\n';
    return 0;
}
```

4. Aşağıdaki üye fonksiyon tanımında hata nedir?

```
template <typename T>
class Kutu {
public:
    T al() const;
private:
    T deger_;
};

T Kutu<T>::al() const { return deger_; }
```

5. Şablon fonksiyonların ve sınıfların tanımı neden başlık dosyasına yazılır?

A) `.cpp` dosyaları şablon içeremez.
B) Başlık dosyaları daha hızlı derlenir.
C) Derleyici örnekleme sırasında gövdeyi görmek zorundadır; aksi hâlde bağlayıcı `undefined reference` verir.
D) Şablonlar yalnızca `#pragma once` ile çalışır.
E) Standart bunu yalnızca `class` şablonları için ister.

6. Bu programın çıktısı nedir?

```
#include <iostream>
using namespace std;

template <typename T>
void say() {
    static int n = 0;
    cout << ++n << ",";
}

int main() {
    say<int>(); say<char>(); say<int>(); say<int>();
    cout << '\n';
    return 0;
}
```

7. `template <typename T, int N> class SabitDizi` şablonunda `N` hakkında hangisi doğrudur?

A) `N` çalışma zamanında klavyeden okunabilir.
B) `N` bir tip adıdır.
C) `N`, derleme zamanında bilinen bir sabit olmalıdır.
D) `N` yalnızca `size_t` olabilir.
E) `N` yazılmazsa her zaman 0 kabul edilir.

8. Aşağıdaki kod neden derlenmez? Nasıl düzeltilir?

```
template <typename T, int N>
class SabitDizi { public: T veri[N]; };

int main() {
    int n = 5;
    SabitDizi<int, n> d;
    return 0;
}
```

9. Şablon özelleştirmenin amacı aşağıdakilerden hangisidir?

A) Şablonun daha hızlı derlenmesini sağlamak.
B) Belirli bir tip için genel şablondan farklı davranış tanımlamak.
C) Şablonu bir sanal fonksiyona dönüştürmek.
D) Bir şablonu başka bir dosyaya taşımak.
E) Şablonun tip çıkarımını kapatmak.

10. Bu programın çıktısı nedir?

```cpp
#include <iostream>
using namespace std;

template <typename T>
struct Ad { static const char* al() { return "genel"; } };

template <>
struct Ad<int> { static const char* al() { return "int"; } };

int main() {
    cout << Ad<int>::al() << " " << Ad<double>::al() << " "
         << Ad<char>::al() << '\n';
    return 0;
}
```

11. C++ şablonları ile C# generics arasındaki farklarla ilgili hangisi doğrudur?

A) C#'ta tip olmayan şablon parametresi vardır, C++'ta yoktur.
B) C# generics'te kısıtlar `where` ile bildirilir; C++ şablonlarında kısıt yoktur (C++17'de) ve hata örneklemede ortaya çıkar.
C) C++ şablonları çalışma zamanında, C# generics derleme zamanında çözülür.
D) İkisi de aynı biçimde tam ve kısmi özelleştirme destekler.
E) C#'ta generics yalnızca `struct` ile kullanılabilir.

12. Aşağıdaki şablon hatası mesajında önce hangi kısma bakmak en yararlıdır?

```
In instantiation of 'T enBuyuk(T, T) [with T = Nokta]':
error: no match for 'operator<' (operand types are 'Nokta' and 'Nokta')
```

A) Dosyanın en sonundaki satıra.
B) İlk `error:` satırına ve `[with T = ...]` bilgisine.
C) Yalnızca satır numaralarına.
D) Derleyici sürümüne.
E) Hiçbir kısım yararlı değildir.

13. İki değerden küçüğünü döndüren `enKucuk` fonksiyon şablonunu yazın. `int`, `double` ve `string` ile çağırın.

14. İki tipli `Cift<A, B>` yapısını ve elemanların yerlerini değiştirip `Cift<B, A>` döndüren `takas` fonksiyon şablonunu yazın. `Cift<string, int>` ile deneyin.

15. En çok `N` eleman alabilen `SabitYigin<T, N>` sınıf şablonunu yazın. `ekle` doluyken `false` döndürsün, `cikar` boşken `false` döndürsün. `SabitYigin<int, 2>` ile üç eleman eklemeyi deneyin.

BÖLÜM 34

İstisna Yönetimi

BU BÖLÜMDE

- Dönüş koduyla hata bildiriminin neden yetersiz kaldığını ve istisnanın (exception) buna nasıl çözüm getirdiğini
- `throw`, `try` ve `catch` ile bir hatayı fırlatmayı, yakalamayı ve ele almayı; yakalanmayan istisnada ne olduğunu
- `<stdexcept>` içindeki standart istisna hiyerarşisini ve `what()` ile hata iletisini okumayı
- Birden çok `catch` bloğunda sıranın önemini, `catch (...)` ve `throw;` ile yeniden fırlatmayı
- `std::runtime_error`'dan türeterek kendi istisna sınıfımızı yazmayı
- Yığın çözülmesini (stack unwinding), yapıcıda fırlatılan istisnayı ve RAII ile ilişkisini
- `noexcept` sözünü ve istisnanın ne zaman kullanılıp ne zaman kullanılmayacağını

Bölüm 26, 27 ve 29'da `at()`, `stoi` ve `new` ile ilgili örneklerde "istisna" sözcüğüyle karşılaştık ve `try`/`catch` kalıbını şimdilik kalıp olarak aldık. Bu bölümde mekanizmanın kendisini işleyeceğiz: istisna nasıl fırlatılır, nasıl yakalanır, yolda neler olur ve ne zaman kullanılmalıdır.

C'den gelen okur için asıl yenilik şudur: C'de bir hatayı bildirmenin standart yolu yoktu, her kütüphane kendi yöntemini seçerdi. C++ hatayı **normal dönüş yolundan ayrı bir kanaldan** göndermeyi dilin parçası yapar. Bölüm 29'da gördüğümüz RAII ve Bölüm 31'deki kalıtım, bu mekanizmanın iki dayanağıdır; ikisini yeniden anlatmayacak, istisnalarla nasıl birleştiklerini göstereceğiz.

34.1 Dönüş Koduyla Hata Bildiriminin Sorunları

C'de hata bildirmenin iki yaygın yolunu bilirsiniz: fonksiyon özel bir değer döndürür (`fopen` başarısızlıkta `NULL`, Bölüm 17) ya da genel bir değişkene neden yazılır (`errno`). Bölme işlemini bu yöntemle yazalım: bölen sıfırsa -1 döndürelim.

hataKodu.cpp

```
#include <iostream>

int bol(int a, int b) {
    if (b == 0) {
        return -1;                  // hata kodu
    }
    return a / b;
}

int main() {
    std::cout << "10 / 2 = " << bol(10, 2) << '\n';
```

```
    std::cout << "10 / 0 = " << bol(10, 0) << '\n';
    std::cout << "-10 / 10 = " << bol(-10, 10) << '\n';
    return 0;
}
```

```
ÇIKTI
10 / 2 = 5
10 / 0 = -1
-10 / 10 = -1
```

Son iki satır aynı sayıyı yazdı; biri hata, diğeri gerçek sonuç. Bu yaklaşımın dört sorunu vardır:

1. **Sonuçla karışır.** `-1` hem "hata" hem de geçerli bir sonuç olabilir. Her değer geçerliyse ayrılacak "hata değeri" kalmaz.
2. **Denetimi unutmak kolaydır.** Çağıran dönüş değerine bakmazsa derleyici ses çıkarmaz; program yanlış değerle devam eder.
3. **Hata elle taşınır.** `a()` fonksiyonu `b()`'yi, `b()` de `c()`'yi çağırıyorsa ve hata `c()`'de olduysa, `b()` kodu `a()`'ya iletmek zorundadır. Asıl iş, `if` yığınında kaybolur.
4. **Yapıcı değer döndüremez.** Bölüm 30'daki yapıcının dönüş tipi yoktur; nesne kurulurken bir şey ters giderse bildirmenin yolu kalmaz.

> **Tanım: İstisna (exception)**
>
> Bir hatayı, fonksiyonun normal dönüş değerinden bağımsız olarak, hatayı ele alabilecek koda doğrudan ileten dil mekanizmasıdır. Hata oluştuğunda fonksiyon normal yoldan dönmez; çağrı zincirinde yukarı doğru, uygun bir yakalayıcı bulunana kadar ilerlenir.

> *Tarihçe · İstisnalar C++'a nasıl girdi?*
>
> İstisna yönetimi C++'a 1980'lerin sonu ile 1990'ların başında eklendi ve dilin standart parçası olarak ilk kez 1998 tarihli C++98 standardında yer aldı. Tasarımda belirleyici fikir, istisna sırasında yerel nesnelerin yıkıcılar aracılığıyla otomatik temizlenmesiydi; bu fikir daha sonra RAII adıyla anıldı. C dilinde böyle bir mekanizma yoktur (en yakın karşılığı `setjmp` / `longjmp` çiftidir, ama yerel nesnelerin temizliğini bilmez).

34.2 throw, try ve catch

İstisna mekanizması üç anahtar sözcüğe dayanır:

- `throw` hatayı **fırlatır** (bildirir).
- `try` bloğu, hata fırlatabilecek kodu sarar.
- `catch` bloğu, belirli türdeki hatayı **yakalar** ve ele alır.

ilkIstisna.cpp

```
#include <iostream>
#include <stdexcept>

int bol(int a, int b) {
    if (b == 0) {
        throw std::invalid_argument("bölen sıfır olamaz");
    }
    return a / b;
}
```

```
int main() {
    try {
        std::cout << "10 / 2 = " << bol(10, 2) << '\n';
        std::cout << "10 / 0 = " << bol(10, 0) << '\n';
        std::cout << "Bu satır hiç çalışmaz\n";
    } catch (const std::invalid_argument& hata) {
        std::cout << "Hata yakalandı: " << hata.what() << '\n';
    }
    std::cout << "Program devam ediyor\n";
    return 0;
}
```

```
ÇIKTI
10 / 2 = 5
10 / 0 = Hata yakalandı: bölen sıfır olamaz
Program devam ediyor
```

Adım adım ne olduğuna bakalım:

1. `throw` ifadesi bir istisna nesnesi oluşturur ve fırlatır. `std::invalid_argument`, `<stdexcept>` başlığındaki hazır bir sınıftır; yapıcısı bir ileti metni alır.
2. `bol(10, 0)` içinde `throw` çalışınca fonksiyon `return` etmeden terk edilir; `return a / b;` hiç çalışmaz.
3. Denetim, türü uyan `catch` bloğuna geçer. `try` bloğunun kalanı atlanır.
4. `hata.what()` nesnenin taşıdığı iletiyi verir.
5. `catch` bitince program `try` / `catch` yapısının altından sürer. Fırlatıldığı noktaya geri dönülmez.

İkinci satırın yarım kalmasına dikkat edelim: `"10 / 0 = "` metni, `bol(10, 0)` çağrılmadan önce akışa yazılmıştı. `<<` zinciri soldan sağa işlenir (C++17'den beri bu sıra garantilidir); hata satırın ortasında kesmiştir.

Birkaç kısa kural:

- `try` hatasız biterse hiçbir `catch` çalışmaz.
- `catch`, `try` bloğunun hemen ardından gelir ve tek satır da olsa süslü parantez ister (`if`'ten farkı).
- İstisna nesnesine ihtiyacımız yoksa parametre adını yazmayabiliriz: `catch (const std::invalid_argument&)`.
- `try` bir bloktur ve kendi kapsamını oluşturur. İçinde tanımlanan değişken `catch`'te ve sonrasında görünmez; dışarıda kullanılacaksa `try`'dan önce tanımlanmalıdır.

Yakalanmayan istisna

Hiçbir `catch` bir istisnayı yakalamazsa istisna fonksiyonları teker teker terk ederek yukarı çıkar, `main`'den de geçerse çalışma zamanı `std::terminate` fonksiyonunu çağırır ve program anında sonlanır.

```
▸ yakalanmayan.cpp
#include <iostream>
#include <stdexcept>

int main() {
    std::cout << "basla\n";
    throw std::runtime_error("beklenmedik");
}
```

```
TERMİNAL
```

```
$ g++ -std=c++17 yakalanmayan.cpp -o yakalanmayan
$ ./yakalanmayan
basla
terminate called after throwing an instance of 'std::runtime_error'
  what():  beklenmedik
Aborted
```

Bu iletinin biçimi g++'a özgüdür; başka derleyiciler ve işletim sistemleri farklı metin verir, ama davranış aynıdır: program anormal biçimde sonlanır. Yakalanmayan istisna, sessizce yanlış sonuç üretmekten iyidir; ama kullanıcıya düzgün bir ileti göstermek istiyorsak beklediğimiz istisnaları kendimiz yakalamalıyız.

34.3 Standart İstisna Hiyerarşisi

Standart kütüphanenin istisna sınıfları bir kalıtım ağacında (Bölüm 31) toplanır. Kök `std::exception` sınıfıdır ve her birinde `what()` adlı sanal bir fonksiyon bulunur:

```
std::exception
├── std::logic_error          programcının hatası; kod doğru olsa olmazdı
│   ├── std::invalid_argument     geçersiz argüman
│   ├── std::out_of_range         geçerli aralığın dışına erişim
│   ├── std::length_error         izin verilen uzunluk aşıldı
│   └── std::domain_error         değer, işlevin tanım kümesi dışında
├── std::runtime_error        çalışırken ortaya çıkan, önceden bilinemeyen hata
│   ├── std::range_error
│   ├── std::overflow_error
│   └── std::underflow_error
├── std::bad_alloc            new ile bellek ayrılamadı  (<new>)
└── std::bad_cast             referansla dynamic_cast başarısız  (<typeinfo>)
```

`logic_error` ve `runtime_error` ailesi `<stdexcept>` başlığındadır. Ayrımı kabaca şöyle yapabiliriz: `logic_error` "bu çağrıyı yapmamalıydın" der (negatif yaş, olmayan eleman numarası); `runtime_error` "kodun doğruydu ama dışarıdaki bir şey ters gitti" der (dosya yok, bakiye yetmiyor). Kendi kodumuzda en çok `invalid_argument`, `out_of_range` ve `runtime_error` işimize yarar; `bad_alloc` ve `out_of_range`'i çoğunlukla kütüphane fırlatır (Bölüm 29 ve 26).

`what()`, hatanın açıklamasını `const char*` olarak verir. Kendi fırlattığımız istisnanın iletisi yapıcıya verdiğimiz metindir. Kütüphanenin fırlattıklarında ileti derleyiciye göre değişir (`stoi` için g++ yalnızca `stoi` yazar); bu yüzden iletinin metnine bakarak program akışına karar vermeyiz, yalnızca kullanıcıya ya da kayda (log) yazarız.

const referansla yakalamak

`what()` sanal olduğu için Bölüm 31'deki çok biçimlilik burada da işler: `const std::exception&` parametreli bir fonksiyona `invalid_argument` da gönderebiliriz, `runtime_error` da; `what()` her seferinde gerçek türün iletisini verir. Bu yalnızca **referans** ya da işaretçi üzerinden geçerlidir. İstisnayı değerle yakalarsak Bölüm 31'de gördüğümüz **nesne dilimleme** (object slicing) devreye girer:

▸ dilimleme.cpp

```cpp
#include <iostream>
#include <stdexcept>

int main() {
    try {
```

```
        throw std::runtime_error("dosya bozuk");
    } catch (std::exception e) {          // değerle yakalama: dilimlenir
        std::cout << "ileti: " << e.what() << '\n';
    }
    return 0;
}
```

```
TERMİNAL
$ g++ -std=c++17 -Wall -Wextra dilimleme.cpp -o dilimleme
dilimleme.cpp:7:29: warning: catching polymorphic type 'class std::exception' by value [-Wcatch-
value=]
    7 |     } catch (std::exception e) {          // değerle yakalama: dilimlenir
      |                              ^
```

```
ÇIKTI
ileti: std::exception
```

`runtime_error`'a ait kısım kopyalanırken kesildi; "dosya bozuk" iletisi kayboldu. (Çıktıdaki `std::exception` metni g++'ın kütüphanesine özgüdür; iletinin kaybolması ise her derleyicide geçerlidir.)

> ◆ **Kritik Nokta · İstisnayı const referansla yakalayın**
>
> Kalıp her zaman `catch (const TurAdi& ad)` olmalıdır. Değerle yakalamak gereksiz kopya ve dilimleme demektir; işaretçiyle yakalamak yalnızca `throw new ...` ile çalışır ve sızıntı riski getirir. `const` ise yakalayıcının istisnayı değiştirmeyeceğini belirtir.

Kütüphanenin fırlattıkları

Daha önce gördüğümüz iki işlev bu hiyerarşiyi kullanır. `std::stoi` sayı bulamazsa `invalid_argument`, sayı `int`'e sığmazsa `out_of_range` fırlatır (Bölüm 27); `vector::at()` ve `string::at()` sınır dışında `out_of_range` fırlatır, `[]` ise denetim yapmaz. Dikkat edilecek bir ayrıntı: `stoi("12abc")` hata vermez, baştaki sayıyı okuyup 12 döndürür.

Tamsayı sıfıra bölme ise bir istisna **değildir**; C'deki gibi tanımsız davranıştır. C# bu noktada farklıdır ve `DivideByZeroException` fırlatır. C++'ta bölmeden önce `bolen != 0` denetimi bizim işimizdir; yukarıdaki `bol` fonksiyonunda yaptığımız gibi.

34.4 Birden Çok catch ve Sıra

Bir `try` bloğundan farklı türde istisnalar çıkabilir; bu durumda `try`'ın ardına birden çok `catch` yazarız. Yukarıdan aşağıya bakılır ve **türü uyan ilk** blok çalışır, kalanlar atlanır.

▸ coklu.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

void dene(const std::string& metin) {
    try {
        int sayi = std::stoi(metin);
        std::cout << "\"" << metin << "\" -> " << sayi << '\n';
    } catch (const std::invalid_argument&) {
```

```
        std::cout << "\"" << metin << "\" -> sayı değil\n";
    } catch (const std::out_of_range&) {
        std::cout << "\"" << metin << "\" -> çok büyük\n";
    }
}

int main() {
    dene("42");
    dene("abc");
    dene("99999999999");

    std::vector<int> v{10, 20, 30};
    try {
        std::cout << v.at(1) << '\n';
        std::cout << v.at(5) << '\n';
    } catch (const std::exception&) {
        std::cout << "at() hata verdi\n";
    }
    return 0;
}
```

```
ÇIKTI
"42" -> 42
"abc" -> sayı değil
"99999999999" -> çok büyük
20
at() hata verdi
```

Son `catch`'te `std::exception` kökünü kullandık: `out_of_range`, `exception`'ın torunu olduğu için yakalanır. Bir üst sınıf türündeki `catch` alt sınıfları da yakalar; bu kolaylık aynı zamanda bir tuzağın kaynağıdır.

> **▲ Sık Yapılan Hata · Önce özel, sonra genel**
>
> Genel bir tür özelinden **önce** yazılırsa özel blok hiçbir zaman çalışmaz. C++ bunu hata saymaz, yalnızca uyarı verir.

▸ catchSirasi.cpp

```
#include <iostream>
#include <stdexcept>

int main() {
    try {
        throw std::out_of_range("indeks 9, boyut 3");
    } catch (const std::logic_error&) {
        std::cout << "logic_error bloğu\n";
    } catch (const std::out_of_range&) {
        std::cout << "out_of_range bloğu\n";
    }
    return 0;
}
```

```
TERMİNAL
$ g++ -std=c++17 -Wall -Wextra catchSirasi.cpp -o catchSirasi
catchSirasi.cpp:9:7: warning: exception of type 'std::out_of_range' will be caught by earlier h
andler [-Wexceptions]
    9 |     } catch (const std::out_of_range&) {
      |       ^~~~~
```

```
catchSirasi.cpp:7:7: note: for type 'std::logic_error'
    7 |     } catch (const std::logic_error&) {
      |       ^~~~~
```

```
ÇIKTI
logic_error bloğu
```

`out_of_range` fırlatıldı ama `logic_error` bloğu çalıştı; ikinci blok ölü koddur. Doğrusu iki bloğun yerini değiştirmektir. Kural şudur: ağaçta aşağıdakileri yukarıdakilerden önce yazarız. Kardeş sınıflar (`invalid_argument` ile `out_of_range`) birbirini yakalamaz, aralarındaki sıra önemsizdir.

34.5 catch (...) ve Yeniden Fırlatma

C++'ta yalnızca `std::exception` türevlerini değil, **her tipten** değeri fırlatabiliriz; `throw 42;` geçerlidir. Üç nokta ile yazılan `catch (...)` her türlü istisnayı yakalar, ama içeride istisna nesnesine ulaşamayız; yani `what()` çağıramayız. Bu yüzden son çare bloğudur ve her zaman en sona yazılır:

ucNokta.cpp

```cpp
#include <iostream>
#include <stdexcept>

int main() {
    for (int secim = 0; secim <= 2; secim++) {
        try {
            if (secim == 1) {
                throw std::runtime_error("çalışma zamanı hatası");
            }
            if (secim == 2) {
                throw 42;                 // kötü stil, ama geçerli
            }
            std::cout << secim << ": hata yok\n";
        } catch (const std::exception& e) {
            std::cout << secim << ": exception -> " << e.what() << '\n';
        } catch (...) {
            std::cout << secim << ": bilinmeyen tür\n";
        }
    }
    return 0;
}
```

```
ÇIKTI
0: hata yok
1: exception -> çalışma zamanı hatası
2: bilinmeyen tür
```

`int` fırlatırsak kimse hatanın ne olduğunu öğrenemez. Kural: yalnızca `std::exception`'dan türeyen sınıflar fırlatırız.

throw; ile yeniden fırlatma

Bazen hatayı bir yerde yakalayıp bir şey yapmak (örneğin kayda yazmak), asıl ele alma işini ise daha üstteki koda bırakmak isteriz. `catch` bloğunda argümansız `throw;` yazmak, yakalanan istisnayı **olduğu gibi** yukarı gönderir. Benzer görünen `throw e;` ise farklıdır:

yenidenFirlat.cpp

```cpp
#include <iostream>
#include <stdexcept>

void alt() {
    throw std::out_of_range("indeks sınır dışı");
}

void dogru() {
    try {
        alt();
    } catch (const std::exception& e) {
        std::cout << "kayıt: " << e.what() << '\n';
        throw;                          // aynı nesne yukarı gider
    }
}

void yanlis() {
    try {
        alt();
    } catch (const std::exception& e) {
        std::cout << "kayıt: " << e.what() << '\n';
        throw e;                        // kopya: tür exception'a dilimlenir
    }
}

void dene(void (*islev)()) {
    try {
        islev();
    } catch (const std::out_of_range&) {
        std::cout << "out_of_range olarak yakalandı\n";
    } catch (const std::exception& e) {
        std::cout << "exception olarak yakalandı: " << e.what() << '\n';
    }
}

int main() {
    dene(dogru);
    dene(yanlis);
    return 0;
}
```

ÇIKTI

```
kayıt: indeks sınır dışı
out_of_range olarak yakalandı
kayıt: indeks sınır dışı
exception olarak yakalandı: std::exception
```

`throw e;` yeni bir istisna fırlatır; nesne `exception` türünde kopyalandığı için asıl tür ve ileti kaybolur. Yeniden fırlatırken her zaman `throw;` yazarız.

34.6 Kendi İstisna Sınıfımız

Hazır sınıflar her durumu anlatmaya yetmez. "Stok yetersiz" gibi programa özgü bir hata için kendi sınıfımızı yazmanın iki nedeni vardır: `catch` hatayı **türüne** bakarak ayırt edebilir ve istisna, iletinin yanında hatayla ilgili **veri** de taşıyabilir.

Kalıp basittir: `std::runtime_error`'dan (duruma uygunsa başka bir standart sınıftan) türetiriz ve iletiyi taban sınıfın yapıcısına iletiriz (Bölüm 31'deki taban sınıf yapıcısı kuralı). `what()`'u yeniden yazmak gerekmez; `runtime_error` iletiyi zaten saklar.

depo.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

class StokHatasi : public std::runtime_error {
public:
    StokHatasi(const std::string& urun, int istenen, int mevcut)
        : std::runtime_error(urun + " için stok yetersiz"),
          eksik_(istenen - mevcut) {}

    int eksik() const { return eksik_; }

private:
    int eksik_;
};

class Depo {
public:
    Depo(const std::string& urun, int stok) : urun_(urun), stok_(stok) {
        if (stok < 0) {
            throw std::invalid_argument("başlangıç stoku negatif olamaz");
        }
    }

    void cikar(int adet) {
        if (adet <= 0) {
            throw std::invalid_argument("adet pozitif olmalı");
        }
        if (adet > stok_) {
            throw StokHatasi(urun_, adet, stok_);
        }
        stok_ -= adet;                    // değişiklik, denetimlerden sonra
    }

    int stok() const { return stok_; }

private:
    std::string urun_;
    int stok_;
};

int main() {
    try {
        Depo hatali("kalem", -5);
        std::cout << "Bu satıra gelinmez\n";
    } catch (const std::invalid_argument& e) {
        std::cout << "Depo açılamadı: " << e.what() << '\n';
    }

    Depo depo("defter", 10);
    int istekler[] = {4, -1, 8, 6, 1};
    for (int adet : istekler) {
        try {
            depo.cikar(adet);
            std::cout << adet << " çıkarıldı, kalan " << depo.stok() << '\n';
        } catch (const StokHatasi& e) {
            std::cout << "Reddedildi: " << e.what()
```

```
                      << ", " << e.eksik() << " eksik\n";
        } catch (const std::invalid_argument& e) {
            std::cout << "Geçersiz istek: " << e.what() << '\n';
        }
    }
    return 0;
}
```

```
ÇIKTI
Depo açılamadı: başlangıç stoku negatif olamaz
4 çıkarıldı, kalan 6
Geçersiz istek: adet pozitif olmalı
Reddedildi: defter için stok yetersiz, 2 eksik
6 çıkarıldı, kalan 0
Reddedildi: defter için stok yetersiz, 1 eksik
```

Çıktıdan üç tasarım kararını okuyabiliriz:

- `StokHatasi` bir `runtime_error` olduğu için `catch (const std::runtime_error&)` ya da `catch (const std::exception&)` de onu yakalar. Sıra kuralı burada da geçerlidir: özel sınıf önce yazılır.
- `Depo` hatayı ekrana yazmaz, yalnızca fırlatır. Hatayı kullanıcıya nasıl göstereceğine çağıran kod karar verir.
- `cikar` içinde denetim ve fırlatma, `stok_ -= adet;` satırından **önce** yapılır. Hata olduğunda nesne önceki tutarlı durumunda kalır. Buna **güçlü istisna güvencesi** (strong exception safety) denir ve iyi bir alışkanlıktır.

`exception`'dan doğrudan türetirsek iletiyi `what()`'u kendimiz yazarak vermeliyiz:

```
class BasitHata : public std::exception {
public:
    const char* what() const noexcept override { return "basit hata"; }
};
```

İmzadaki `const noexcept` aynen yazılmalıdır; aksi hâlde taban sınıfın işlevini ezmiş (override) olmayız. Çoğu durumda `runtime_error`'dan türetmek daha kısadır.

34.7 Yığın Çözülmesi

Bir istisna fırlatıldığında C++ `throw`'un bulunduğu fonksiyondan başlayarak çağrı yığınında geriye doğru ilerler ve uyan bir `catch` arar. Her fonksiyonu terk ederken, o fonksiyondaki yerel nesnelerin yıkıcılarını çağırır. Buna **yığın çözülmesi** (stack unwinding) denir.

▸ yiginCozulmesi.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

class Izci {
public:
    Izci(const std::string& ad) : ad_(ad) {
        std::cout << "  + " << ad_ << " oluştu\n";
    }
    ~Izci() { std::cout << "  - " << ad_ << " yok edildi\n"; }

private:
    std::string ad_;
```

```
};

void c() {
    Izci iz("c");
    throw std::runtime_error("c'de hata");
    std::cout << "  c bitti\n";             // hiç çalışmaz
}

void b() {
    Izci iz("b");
    c();
    std::cout << "  b bitti\n";             // hiç çalışmaz
}

int main() {
    Izci iz("main");
    try {
        b();
    } catch (const std::exception& e) {
        std::cout << "yakalandı: " << e.what() << '\n';
    }
    std::cout << "main bitiyor\n";
    return 0;
}
```

```
ÇIKTI
  + main oluştu
  + b oluştu
  + c oluştu
  - c yok edildi
  - b yok edildi
yakalandı: c'de hata
main bitiyor
  - main yok edildi
```

`c bitti` ve `b bitti` satırları atlandı, ama `c` ve `b`'nin yerel nesneleri yok edildi; üstelik oluşma sırasının tersine ve `catch` bloğu çalışmadan **önce**. `main`'deki nesne `try` dışında olduğundan program sonunda yok edildi.

Bölüm 29'da gördüğümüz RAII'nin neden istisnayla birlikte vazgeçilmez olduğu burada görünür. Yıkıcı, istisna yüzünden çıkışta bile çalıştığı için kaynağı yapıcıda alıp yıkıcıda bırakan sınıf, elle yazdığımız `delete` ve `close` çağrılarının atlanması sorununu ortadan kaldırır. `new` ile alınmış **ham işaretçinin** kendisi ise bir `int` gibi sıradan bir değerdir; yok edilmesi gösterdiği nesneyi yok etmez. Bu yüzden `unique_ptr`, `vector` ve `string` gibi kaynağı kendisi bırakan türleri kullanırız.

> **◆ Kritik Nokta · Yıkıcıdan istisna kaçırmayın**
>
> Yıkıcılar varsayılan olarak `noexcept` sayılır. Yığın çözülürken bir yıkıcı istisna fırlatırsa aynı anda iki istisna oluşur ve C++ doğrudan `std::terminate` çağırır. Yıkıcının içinde hata çıkabilecek bir iş varsa (dosya kapatma gibi), hatayı yıkıcının içinde `try` / `catch` ile yakalayıp ele alırız; dışarı bırakmayız.

Yapıcıda fırlatılan istisna

C'de yarım kalmış bir başlatmayı bildirmek zordu. C++'ta yapıcı `throw` edebilir ve nesne **hiç oluşmamış** sayılır: yapıcısı tamamlanmayan nesnenin yıkıcısı çağrılmaz. Ancak o ana kadar tamamen kurulmuş **üyeler** yok edilir:

yapiciIstisna.cpp

```cpp
#include <iostream>
#include <stdexcept>

struct Parca {
    int no;
    Parca(int n) : no(n) { std::cout << "Parça " << no << " kuruldu\n"; }
    ~Parca() { std::cout << "Parça " << no << " söküldü\n"; }
};

class Makine {
public:
    Makine(bool bozuk) : a_(1), b_(2) {
        std::cout << "Makine kuruluyor\n";
        if (bozuk) {
            throw std::runtime_error("makine kurulamadı");
        }
    }
    ~Makine() { std::cout << "Makine yok edildi\n"; }

private:
    Parca a_;
    Parca b_;
};

int main() {
    try {
        Makine m(true);
    } catch (const std::exception& e) {
        std::cout << "yakalandı: " << e.what() << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Parça 1 kuruldu
Parça 2 kuruldu
Makine kuruluyor
Parça 2 söküldü
Parça 1 söküldü
yakalandı: makine kurulamadı
```

`Makine`'nin yıkıcısı çalışmadı, ama iki `Parca` ters sırada söküldü. Bu yüzden üyeleri ham `new` ile değil, `vector`, `string` ve `unique_ptr` gibi RAII türleriyle tutarız; yapıcı ortasında fırlatılan bir istisna sızıntı bırakmaz.

34.8 noexcept

Bir fonksiyonun istisna fırlatmayacağını biliyorsak imzasına `noexcept` yazabiliriz. Bu hem okuyana hem derleyiciye verilen bir sözdür:

```cpp
int topla(int a, int b) noexcept {
    return a + b;
}
```

Sözü tutmazsak ceza ağırdır: `noexcept` bir fonksiyondan dışarı istisna çıkmaya çalışırsa program doğrudan `std::terminate` ile sonlanır ve çağıranın `catch` bloğu devreye girmez.

noexceptCokme.cpp

```
#include <iostream>
#include <stdexcept>

void riskliIs() {
    throw std::runtime_error("beklenmedik hata");
}

void sozVerdim() noexcept {
    std::cout << "sozVerdim başladı\n";
    riskliIs();
}

int main() {
    try {
        sozVerdim();
    } catch (const std::exception&) {
        std::cout << "yakalandı\n";          // çalışmaz
    }
    return 0;
}
```

TERMİNAL

```
$ ./noexceptCokme
sozVerdim başladı
terminate called after throwing an instance of 'std::runtime_error'
  what():  beklenmedik hata
Aborted
```

Bu yüzden `noexcept`'i yalnızca gerçekten istisna çıkmayacak işlevlere yazarız. Yanlış söz vermek, hiç söz vermemekten kötüdür. Taşıma işlemlerinin (Bölüm 39) `noexcept` olması ise kapsayıcıların performansını doğrudan etkiler; bunu orada göreceğiz.

> *Tarihçe · Eski istisna belirtimleri*
>
> C++98'de bir fonksiyonun hangi türleri fırlatabileceğini imzada `void f() throw(std::out_of_range);` biçiminde yazmak mümkündü. Bu "dinamik istisna belirtimleri" pratikte işe yaramadığı için C++11'de eskimiş ilan edildi ve C++17'de dilden kaldırıldı. Yerini yalnızca `noexcept` aldı. Eski kodlarda `throw(...)` görürsek bunu okuyabilmemiz yeterlidir.

34.9 İstisna mı, Dönüş Değeri mi?

İstisna her derde deva değildir. Fırlatıp yakalamak, aynı denetimi `if` ile yapmaktan genellikle çok daha yavaştır; hata hiç çıkmayan yolda ise maliyet küçüktür. Ölçüt şudur:

Durum	Tercih
Hata sık ve beklenen (kullanıcı yanlış tuşa bastı, aranan kayıt yok)	Dönüş değeri, `bool`, `if`
Gerçekten istisnai durum (dosya açılamadı, bellek bitti, bakiye yetmiyor)	İstisna
Hata, birkaç fonksiyon yukarıda ele alınacak	İstisna

Durum	Tercih
Hata yapıcıda	İstisna
Programcı hatası (yanlış çağrı)	Düzeltilir; kullanıcıya gösterilmez

`cin` ile sayı okurken yanlış girdi varsayılan olarak istisna fırlatmaz; akış hata durumuna geçer ve `cin.fail()` ile denetlenir (Bölüm 23). Kullanıcı girdisi "beklenen hata" sınıfındadır. `getline` ile satırı alıp `stoi` ile çevirmek ise hataları `try` / `catch` ile ele almamızı sağlar; iki yolu da bilmeliyiz.

▲ Sık Yapılan Hata · Boş catch hatayı gizler

```
std::vector<std::string> veriler{"10", "yirmi", "30"};
int toplam = 0;
for (const std::string& v : veriler) {
    try {
        toplam += std::stoi(v);
    } catch (...) {
        // hata sessizce yutuldu
    }
}
std::cout << toplam << '\n';      // 40; "yirmi" sayılmadı, kimse uyarılmadı
```

Hatayı yakalayıp hiçbir şey yapmamak, program çökmediği için daha tehlikelidir. En azından ileti yazdırmalı, kayda geçirmeli ya da yeniden fırlatmalıyız. Ayrıca `catch (...)` yerine beklediğimiz özel türü yakalarız.

34.10 Üç Dilde Hata Yönetimi

Üç Dilde · C dönüş kodları, C++ istisnaları, C# try/catch/finally *C · C++ · C#*

C'de hata bildirimi dönüş değeri ve `errno` ile yapılır; denetim tamamen çağıranın sorumluluğundadır (Bölüm 17):

```
FILE* f = fopen("veri.txt", "r");
if (f == NULL) {
    perror("veri.txt");           // errno'daki nedeni yazar
    return 1;
}
```

C++'ta hata fırlatılır, uygun yerde yakalanır; temizlik yıkıcılara bırakılır. C++'ta `finally` yoktur, çünkü RAII aynı işi yapar:

```
try {
    std::ifstream dosya("veri.txt");      // yıkıcı dosyayı kapatır
    // ...
} catch (const std::exception& e) {
    std::cerr << e.what() << '\n';
}
```

C#'ta mekanizma benzerdir, ama bellek çöp toplayıcıya ait olduğu için kaynak temizliği için `finally` bloğu (ya da `using`) vardır:

```
try
{
    var okuyucu = new StreamReader("veri.txt");
    // ...
```

```
}
catch (FileNotFoundException e)
{
    Console.WriteLine(e.Message);
}
finally
{
    // hata olsa da olmasa da çalışır
}
```

	C	C++	C#
Hata bildirimi	Dönüş değeri, `errno`	`throw`	`throw`
Fırlatılabilen	-	Her tip (öneri: `std::exception`)	Yalnızca `Exception` türevleri
Temizlik	Elle	Yıkıcı (RAII)	`finally`, `using`
Yeniden fırlatma	-	`throw;`	`throw;`
Tamsayı sıfıra bölme	Tanımsız davranış	Tanımsız davranış	`DivideByZeroException`

C#'taki karşılığı Bölüm 53'te ayrıntılı işleyeceğiz; orada `finally`, `using`, `when` filtreleri ve `throw ex;` ile `throw;` farkı yer alacak.

Bölüm Özeti

- Dönüş koduyla hata bildiriminde hata değeri sonuçla karışabilir, denetim unutulabilir, hata elle taşınır ve yapıcı değer döndüremez. İstisna hatayı ayrı bir kanaldan, ele alabilecek koda iletir.
- `throw` nesne fırlatır; `try { ... } catch (const Tur& ad) { ... }` yakalar. İstisna olunca `try`'ın kalanı atlanır, türü uyan ilk `catch` çalışır ve program `try` / `catch`'in altından sürer.
- İstisnayı her zaman `const` referansla yakalarız; değerle yakalamak nesne dilimlemeye yol açar ve `what()` gerçek iletiyi vermez.
- Kök sınıf `std::exception`'dır; `what()` iletiyi verir. `logic_error` ailesi (`invalid_argument`, `out_of_range`, `length_error`) ve `runtime_error` ailesi (`overflow_error` vb.) `<stdexcept>` başlığındadır. Kütüphane istisnalarının iletisi derleyiciye göre değişir.
- `stoi` sayı yoksa `invalid_argument`, `int`'e sığmıyorsa `out_of_range` fırlatır; `at()` sınır dışında `out_of_range` fırlatır. Tamsayı sıfıra bölme C++'ta istisna değil, tanımsız davranıştır.
- Birden çok `catch`'te türü uyan ilk blok çalışır; sıra özelden genele olmalıdır. C++ yanlış sırayı hata saymaz, yalnızca uyarır.
- `catch (...)` her şeyi yakalar ama nesneye ulaşamaz; en sona yazılır. Yeniden fırlatmak için `throw;` yazarız; `throw e;` kopyalar ve türü dilimler.
- Yakalanmayan istisna `std::terminate` ile programı sonlandırır.
- Kendi istisna sınıfımızı `runtime_error`'dan türetir, iletiyi taban sınıfa iletir, gerekirse hatayla ilgili veri üyeleri ekleriz.
- Yığın çözülmesinde `throw` ile yakalayan `catch` arasındaki yerel nesneler ters sırada yok edilir. Yapıcı fırlatırsa nesnenin yıkıcısı çalışmaz, ama kurulmuş üyeler yok edilir. RAII ham `new` / `delete`'ten üstündür.
- Yıkıcıdan istisna kaçırmayız. `noexcept` sözü tutulmazsa program doğrudan `terminate` olur.

- Sık ve beklenen hata için dönüş değeri, gerçekten istisnai durum için istisna kullanırız. Boş `catch` yazmayız.

Alıştırmalar

1. Aşağıdakilerden hangisi `std::runtime_error`'ın alt sınıfıdır?

   A) `invalid_argument`
   B) `out_of_range`
   C) `overflow_error`
   D) `length_error`
   E) `domain_error`

2. Bir `try` bloğunda `throw std::out_of_range("x");` çalışıyor. `catch` blokları sırasıyla `runtime_error`, `invalid_argument`, `logic_error` ve `exception` türünde ve hepsi `const` referansla yazılmış. Hangisi çalışır?

   A) `runtime_error` bloğu
   B) `invalid_argument` bloğu
   C) `logic_error` bloğu
   D) `exception` bloğu
   E) Hiçbiri; program `terminate` ile sonlanır.

3. Fırlatılan bir istisnayı hiçbir `catch` yakalamazsa ne olur?

   A) Program hatayı yok sayıp sonraki satırdan sürer.
   B) `std::terminate` çağrılır ve program sonlanır.
   C) `main` otomatik olarak 0 döndürür.
   D) Derleyici bunu derleme hatası olarak bildirir.
   E) İstisna programdaki ilk `catch (...)` bloğunca yakalanır.

4. Aşağıdakilerden hangisi doğrudur?

   A) Fonksiyon imzasına fırlatacağı istisnaları yazmak zorunludur.
   B) İstisnayı en güvenli biçimde değerle yakalarız.
   C) `std::stoi("12abc")` çağrısı `invalid_argument` fırlatır.
   D) `noexcept` bir fonksiyondan çıkan istisna, çağıranın `catch` bloğunca yakalanır.
   E) Bir `catch` bloğunda `throw;` yakalanan istisnayı türünü koruyarak yeniden fırlatır.

5. Bir yapıcı, iki üyesi kurulduktan sonra `throw` ediyor ve istisna yakalanıyor. Hangisi doğrudur?

   A) Sınıfın yıkıcısı ve iki üyenin yıkıcısı çalışır.
   B) Hiçbir yıkıcı çalışmaz.
   C) Yalnızca sınıfın yıkıcısı çalışır.
   D) Sınıfın yıkıcısı çalışmaz; iki üyenin yıkıcısı ters sırada çalışır.
   E) Üyelerin yıkıcıları kuruluş sırasıyla çalışır.

6. Bu programın çıktısı nedir?

```
#include <iostream>
#include <stdexcept>
#include <string>
```

```
class Kutu {
public:
    Kutu(const std::string& ad) : ad_(ad) { std::cout << ad_ << "+ "; }
    ~Kutu() { std::cout << ad_ << "- "; }
private:
    std::string ad_;
};

void g() {
    Kutu k1("A");
    Kutu k2("B");
    throw std::runtime_error("hata");
}

int main() {
    try {
        Kutu t("T");
        g();
        std::cout << "sonra ";
    } catch (const std::exception&) {
        std::cout << "yakalandi ";
    }
    std::cout << "bitti\n";
    return 0;
}
```

7. Bu programın çıktısı nedir?

```
#include <iostream>
#include <stdexcept>

void f(int n) {
    try {
        if (n == 1) throw std::invalid_argument("bir");
        if (n == 2) throw std::runtime_error("iki");
        std::cout << "tamam ";
    } catch (const std::logic_error&) {
        std::cout << "L ";
        throw;
    }
}

int main() {
    for (int i = 0; i <= 2; i++) {
        try {
            f(i);
        } catch (const std::invalid_argument&) {
            std::cout << "I ";
        } catch (const std::exception&) {
            std::cout << "E ";
        }
    }
    std::cout << '\n';
    return 0;
}
```

8. Bu programın çıktısı nedir?

```
#include <iostream>

int main() {
```

```
    for (int i = 0; i < 3; i++) {
        try {
            if (i == 0) throw 7;
            if (i == 1) throw 'x';
            std::cout << "hic ";
        } catch (int k) {
            std::cout << "int" << k << " ";
        } catch (...) {
            std::cout << "diger ";
        }
    }
    std::cout << '\n';
    return 0;
}
```

9. Aşağıdaki kodda iki sorun vardır. Bulun ve düzeltin.

```
try {
    throw std::out_of_range("sınır");
} catch (std::exception e) {
    std::cout << e.what() << '\n';
} catch (const std::out_of_range& e) {
    std::cout << "özel: " << e.what() << '\n';
}
```

10. Bu kod neden derlenmez? Nasıl düzeltilir?

```
try {
    int sayi = std::stoi(metin);
} catch (const std::invalid_argument&) {
    sayi = 0;
}
std::cout << sayi << '\n';
```

11. Aşağıdaki program `0` yazdıktan sonra çöker. Nedenini açıklayın ve düzeltin.

```
try {
    std::cout << "0\n";
    throw 42;
} catch (const std::exception& e) {
    std::cout << e.what() << '\n';
}
```

12. `double bol(double a, double b)` fonksiyonunu yazın: `b` sıfırsa `std::invalid_argument` fırlatsın. `main` içinde `10 / 4` ve `1 / 0` işlemlerini deneyip iletiyi yazdırsın.
13. `PuanHatasi` adlı, `std::runtime_error`'dan türeyen bir sınıf yazın; geçersiz puan değerini taşısın ve `deger()` ile döndürsün. `puanKontrol(int)` fonksiyonu 0-100 dışındaki puan için bu hatayı fırlatsın. `main`'de 85, 120 ve -5 değerlerini deneyin.
14. `std::vector<int>` tutan basit bir `Yigin` sınıfı yazın. `it(int)`, `cek()` ve `bos()` üyeleri olsun; boş yığından `cek()` çağrılırsa `std::underflow_error` fırlatılsın. 1 ve 2'yi ekleyip üç kez `cek()` çağırın, hatayı yakalayın.
15. Metin listesindeki sayıları toplayan bir program yazın. Sayıya çevrilemeyen ya da `int`'e sığmayan metinleri atlasın, ama sessizce yutmasın: her biri için ileti yazdırsın ve sonunda toplamla geçersiz sayısını bildirsin. Liste: `{"10", "yirmi", "30", "99999999999", "5"}`.

BÖLÜM 35

STL Kapsayıcıları

BU BÖLÜMDE

- STL'nin üç ayağını (kapsayıcı, yineleyici, algoritma) ve aralarındaki işbölümünü açıklamayı
- Sıralı kapsayıcıları (`vector`, `deque`, `list`, `array`) bellek düzenlerine göre ayırt etmeyi
- `pair`, `tuple` ve C++17 yapısal bağlamayı (structured binding) kullanmayı
- `set`, `multiset`, `map` ve `multimap` ile sıralı ilişkisel veri tutmayı; `operator[]` tuzağından kaçınmayı
- `unordered_map` ve `unordered_set` ile karma tablo (hash table) kullanmayı
- `stack`, `queue` ve `priority_queue` uyarlayıcılarını (adapter) uygulamayı
- Yineleyici geçersizleşmesini ve karmaşıklık tablosunu okuyarak iş için doğru kapsayıcıyı seçmeyi

Bölüm 26'da `vector`'ü ve `std::array`'i, Bölüm 25'te `pair`'i tanıdık. Gerçek programlar ise yalnızca ardışık listelerle yetinmez: "ada göre telefon bul", "tekrarlayanları ele", "sıradaki işi al" gibi sorular başka veri düzenleri ister. C'de bunların her biri için bağlı liste, ikili ağaç ya da karma tabloyu elle yazardık (Bölüm 19). C++ bunları hazır, tipten bağımsız (şablon, Bölüm 33) ve sınanmış biçimde standart kütüphanede sunar.

Bu bölüm kapsayıcıları (container) tanıtır. Algoritmaları, yineleyici türlerini ve lambda ifadelerini Bölüm 36'ya bıraktık; burada yineleyiciyi yalnızca kapsayıcılarda gezmek için gereken ölçüde kullanacağız. Bölüm 26'daki `vector` ayrıntılarını tekrar etmiyoruz.

35.1 STL'nin Üç Ayağı

STL (Standard Template Library, Standart Şablon Kütüphanesi) üç parçadan oluşur:

Parça	Görevi	Örnekler
Kapsayıcı (container)	Veriyi saklar	`vector`, `list`, `map`, `set`, `stack`
Yineleyici (iterator)	Kapsayıcının elemanları arasında gezer; işaretçiye benzer	`begin()`, `end()`
Algoritma (algorithm)	Yineleyici aralığı üzerinde iş yapar	`sort`, `find`, `accumulate`

Tasarımın anahtarı ayrışmadır: algoritma kapsayıcının içini bilmez, yalnızca yineleyici ister. Böylece tek bir `accumulate` hem `vector`'de hem `list`'te çalışır. Aşağıdaki program aynı toplama algoritmasını iki farklı kapsayıcıya uygular:

ucAyak.cpp

```
#include <algorithm>
#include <iostream>
#include <list>
#include <numeric>
```

```cpp
#include <vector>

int main() {
    std::vector<int> v{4, 1, 3};
    std::list<int> l{9, 5, 7};

    std::sort(v.begin(), v.end());   // vector: genel sort
    l.sort();                        // list: kendi üye fonksiyonu

    for (auto it = v.begin(); it != v.end(); ++it) {
        std::cout << *it << ' ';
    }
    std::cout << '\n';
    for (int x : l) {
        std::cout << x << ' ';
    }
    std::cout << '\n';

    std::cout << std::accumulate(v.begin(), v.end(), 0) << ' '
              << std::accumulate(l.begin(), l.end(), 0) << '\n';
    return 0;
}
```

```
ÇIKTI
1 3 4
5 7 9
8 21
```

`list` için `l.sort()` yazmamızın nedeni var: genel `std::sort`, elemanlara indeksle sıçrayabilen (rastgele erişimli) yineleyici ister, `list` yineleyicisi ise yalnızca adım adım ilerler. Denersek derleyici şöyle yanıt verir:

▸ listSort.cpp

```cpp
#include <algorithm>
#include <list>

int main() {
    std::list<int> l{9, 5, 7};
    std::sort(l.begin(), l.end());
    return 0;
}
```

```
DERLEYİCİ ÇIKTISI

/usr/include/c++/13/bits/stl_algo.h:1948:50: error: no match for 'operator-'
 (operand types are 'std::_List_iterator<int>' and 'std::_List_iterator<int>')
```

Şablon hata mesajları uzundur (Bölüm 33); gerçek sebep genellikle ilk `error:` satırındadır. Yineleyici türlerini Bölüm 36'da işleyeceğiz.

Kapsayıcıları iki ana gruba ayırabiliriz:

- **Sıralı kapsayıcılar** (sequence containers): elemanlar eklenme sırasıyla durur. `vector`, `deque`, `list`, `array`, `forward_list`.
- **İlişkisel kapsayıcılar** (associative containers): eleman konumu değil değerin kendisi (anahtar) belirler. Sıralı olanlar (`set`, `map`, `multiset`, `multimap`) dengeli bir ağaçla, sırasız olanlar (`unordered_*`) karma tabloyla çalışır.

Bunlara ek olarak `stack`, `queue` ve `priority_queue`, başka bir kapsayıcıyı kısıtlı bir arayüzle saran **uyarlayıcılardır** (container adapter).

> *Tarihçe · STL'nin kökeni*
>
> STL'yi 1990'larda Alexander Stepanov ve arkadaşları tasarladı; fikir, algoritmaları veri yapılarından ayırıp şablonlarla genel yazmaktı. Kütüphane C++98 standardına girdi (Bölüm 2). Karma tablolu `unordered_*` kapsayıcıları, `array` ve `forward_list` ise C++11 ile standarda eklendi.

35.2 Sıralı Kapsayıcılar

`vector` (Bölüm 26) elemanları tek bir ardışık bellek bloğunda tutar. Sonuna ekleme hızlıdır, indeksle erişim hızlıdır, başa ve ortaya ekleme pahalıdır. Çoğu iş için doğru varsayılan seçimdir. Diğer üç sıralı kapsayıcı farklı ödünleşimler sunar.

deque: iki uçtan büyüyen dizi

`deque` (double-ended queue, "dek" diye okunur) hem başa hem sona hızlı ekleme/silme yapar ve `[]` ile indeksli erişim sağlar. İçeride tek blok yerine birkaç sabit boyutlu bloktan oluşur; bu yüzden elemanları tek bir ardışık bellek bölgesinde değildir. Başlığı `<deque>`'dir.

```
blok haritası      bloklar
+-------+       +----+----+----+----+
|   *---+-----> | 0  | 1  | 2  | 3  |
+-------+       +----+----+----+----+
|   *---+-----> | 4  | 5  | 6  | 7  |
+-------+       +----+----+----+----+
|  ...  |       baştaki/sondaki bloklar yeni blokla büyür
+-------+
```

▸ dequeOrnek.cpp

```cpp
#include <deque>
#include <iostream>

void yazdir(const std::deque<int>& d) {
    for (int x : d) {
        std::cout << x << ' ';
    }
    std::cout << "(boyut " << d.size() << ")\n";
}

int main() {
    std::deque<int> d;
    d.push_back(2);
    d.push_back(3);
    d.push_front(1);
    d.push_front(0);
    yazdir(d);

    std::cout << "d[2] = " << d[2] << '\n';
    d.pop_front();
    d.pop_back();
    yazdir(d);

    d.insert(d.begin() + 1, 99);     // ortaya ekleme: kaydırma gerekir
```

```
    yazdir(d);
    return 0;
}
```

```
ÇIKTI
0 1 2 3 (boyut 4)
d[2] = 2
1 2 (boyut 2)
1 99 2 (boyut 3)
```

`vector`'de `push_front` yoktur; başa ekleme için `insert(begin(), x)` gerekir ve her eleman kayar. `deque`'de `push_front` sabit sürede çalışır. Bedeli: elemanlar tek blokta olmadığından `&d[0] + 1` ile bir sonraki elemana gidilemez ve `data()` üyesi yoktur; ayrıca ardışık bellek, işlemci önbelleğine daha elverişli olduğundan `vector` çoğu durumda biraz daha hızlıdır. Kuyruk benzeri işlerde (iki uçtan alıp verme) `deque` iyi bir seçimdir.

list: çift bağlı liste

`list`, çift bağlı listedir (doubly linked list). Her eleman ayrı bir düğümdür ve önceki ile sonraki düğümü gösterir. Bölüm 19'da C'de elle yazdığımız yapının hazır hâlidir. Başlığı `<list>`'tir.

```
  head <-> [ 5 ] <-> [ 3 ] <-> [ 8 ] <-> tail
```

Güçlü yanı: bir yineleyici elimizdeyse o konuma ekleme ve silme, eleman sayısından bağımsız olarak sabit sürede yapılır; hiçbir eleman kaymaz. Ayrıca ekleme ve silme **diğer yineleyicileri geçersiz kılmaz**. Zayıf yanı: `[]` yoktur, n'inci elemana gitmek için n adım yürümek gerekir; düğümler bellekte dağınık olduğundan gezinmek `vector`'den yavaştır ve her düğüm için ayrıca bellek harcanır.

▸ listOrnek.cpp

```
#include <iostream>
#include <iterator>
#include <list>

void yazdir(const std::list<int>& l) {
    for (int x : l) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
}

int main() {
    std::list<int> l{5, 3, 8, 3, 1};

    auto it = std::next(l.begin(), 2);   // 8'i gösterir
    l.insert(it, 42);                    // 8'in önüne
    l.push_front(0);
    l.insert(l.begin(), -1);             // it hâlâ geçerli
    std::cout << "*it = " << *it << '\n';
    yazdir(l);

    l.remove(3);                         // değeri 3 olanların hepsini sil
    yazdir(l);

    l.sort();
    yazdir(l);

    l.push_back(42);
```

```
    l.push_back(42);
    l.unique();                          // ardışık tekrarları tek yap
    yazdir(l);

    std::list<int> b{100, 200};
    l.splice(l.end(), b);                // b'nin düğümlerini taşı, kopyalama yok
    yazdir(l);
    std::cout << "b boş mu? " << (b.empty() ? "evet" : "hayır") << '\n';
    return 0;
}
```

```
ÇIKTI
*it = 8
-1 0 5 3 42 8 3 1
-1 0 5 42 8 1
-1 0 1 5 8 42
-1 0 1 5 8 42
-1 0 1 5 8 42 100 200
b boş mu? evet
```

Çıktıyı adım adım izleyelim:

- `insert(it, 42)` ve iki ekleme sonrasında liste `-1 0 5 3 42 8 3 1` olur. Başa iki eleman eklememize rağmen `it` hâlâ `8`'i gösterir: `list`'te ekleme yineleyicileri geçersiz kılmaz.
- `remove(3)` değeri 3 olan her elemanı siler.
- `sort()` sıralar. Sona eklediğimiz iki `42` ile birlikte `42` üç kez geçer; `unique()` ardışık tekrarları tek elemana indirir ve liste yeniden `-1 0 1 5 8 42` olur. `unique` yalnızca **yan yana** tekrarları siler; bu yüzden genellikle sıralamadan sonra kullanılır.
- `splice` ile `b`'nin iki düğümü `l`'nin sonuna taşınır. `b` boşalır, hiçbir değer kopyalanmaz.

`list`'in üye fonksiyonları şunlardır: `sort()`, `unique()`, `remove(değer)`, `reverse()`, `merge()` ve `splice()`. `splice`, başka bir listenin düğümlerini kopyalamadan, yalnızca bağlantıları değiştirerek bu listeye taşır; bu, `list`'e özgü bir üstünlüktür.

array ve forward_list

`std::array<T, N>` (Bölüm 26) boyutu derleme zamanında sabit, bellek düzeni C dizisiyle aynı olan kapsayıcıdır; yığın (stack) üzerinde durur ve dinamik bellek kullanmaz. `forward_list` (C++11) yalnızca ileri yönlü tek bağlı listedir: `list`'ten daha az bellek harcar, ama `size()` ve geriye gitme yoktur. Gündelik işlerde nadiren gerekir.

Sıralı kapsayıcıları tek tabloda toplayalım:

Özellik	`array`	`vector`	`deque`	`list`
Boyut	Sabit	Değişken	Değişken	Değişken
Bellek	Ardışık	Ardışık	Bloklu	Düğüm düğüm
`[]` ile erişim	O(1)	O(1)	O(1)	Yok
Sona ekleme	Yok	O(1) amortize	O(1)	O(1)
Başa ekleme	Yok	O(n)	O(1)	O(1)
Ortaya ekleme	Yok	O(n)	O(n)	O(1) (konum biliniyorsa)

Tablodaki O(...) gösterimi, veri büyüdükçe adım sayısının nasıl arttığını söyler: O(1) sabit, O(log n) logaritmik, O(n) doğrusal anlamındadır. Bölüm sonundaki karmaşıklık tablosunda bunu ilişkisel kapsayıcılara da genişleteceğiz.

35.3 pair, tuple ve Yapısal Bağlama

İlişkisel kapsayıcılar elemanlarını çiftler hâlinde tutar; bu yüzden önce çift ve demet türlerine bakalım. `std::pair<A, B>` (`<utility>`), farklı tipte iki değeri `first` ve `second` üyeleriyle taşır. Bölüm 25'te onu bir fonksiyondan iki değer döndürmek için kullanmıştık. `std::tuple` (`<tuple>`) aynı fikrin istenen sayıda değere genellemesidir; elemanlara `std::get<indeks>` ile erişilir ve indeks derleme zamanında bilinmelidir.

pairTuple.cpp

```cpp
#include <iostream>
#include <string>
#include <tuple>
#include <utility>

std::tuple<std::string, int, double> ozetGetir() {
    return {"Elif", 3, 4.5};
}

int main() {
    std::pair<std::string, int> p{"Ayşe", 85};
    p.second += 5;
    auto q = std::make_pair(std::string("Mert"), 70);
    std::cout << p.first << " " << p.second << '\n';
    std::cout << "p < q ? " << (p < q) << '\n';    // önce first, eşitse second

    auto t = ozetGetir();
    std::cout << std::get<0>(t) << " " << std::get<1>(t) << " "
              << std::get<2>(t) << '\n';

    auto [ad, sinif, ort] = ozetGetir();             // yapısal bağlama
    std::cout << ad << " " << sinif << " " << ort << '\n';

    auto& [isim, puan] = p;                          // referansla bağla
    puan = 100;
    std::cout << p.first << " " << p.second << '\n';

    std::string kisi;
    double ortalama;
    std::tie(kisi, std::ignore, ortalama) = ozetGetir();
    std::cout << kisi << " " << ortalama << '\n';
    return 0;
}
```

ÇIKTI

```
Ayşe 90
p < q ? 1
Elif 3 4.5
Elif 3 4.5
Ayşe 100
Elif 4.5
```

Bu programdan şunları çıkarıyoruz:

- `pair` ve `tuple` karşılaştırılabilir: elemanlar soldan sağa sözlük sırasıyla karşılaştırılır. `"Ayşe" < "Mert"` olduğundan `p < q` doğrudur. Bu özellik, çiftleri `set` anahtarı yapmayı ve `sort` ile sıralamayı kolaylaştırır.
- `make_pair` tipleri yazmadan çift üretir. C++17'den itibaren `std::pair p{"Ayşe", 85}` gibi sınıf şablonu tip çıkarımı da çalışır.
- **Yapısal bağlama** (structured binding, C++17) `auto [a, b, c] = ifade;` biçimindedir; `pair`, `tuple`, diziler ve üyeleri açık (`public`) `struct`'lar için çalışır. `auto` kopya alır, `auto&` referans alır; ikincisinde `puan = 100` doğrudan `p.second`'u değiştirdi.
- `std::tie`, var olan değişkenlere açma işini yapar; ilgilenmediğimiz elemanı `std::ignore` ile atlarız.

Yapısal bağlamanın asıl yeri, bundan sonra göreceğimiz `map` dolaşmalarıdır.

35.4 set ve multiset: Sıralı Kümeler

Yoklama listesinde bir kişinin adı birden çok yazılmış olabilir ve biz kaç **farklı** kişi geldiğini bilmek istiyoruz. `std::set<T>` (`<set>`) elemanlarını tekrarsız ve sıralı tutar. Eleman değerinin kendisi aynı zamanda anahtardır; `set[2]` gibi indeksle erişim yoktur. İçeride dengeli bir ikili arama ağacı (genellikle kırmızı-siyah ağaç) bulunur; ekleme, silme ve arama O(log n) sürer.

kumeOrnek.cpp

```cpp
#include <iostream>
#include <set>

int main() {
    std::set<int> s{50, 20, 40, 20, 10};      // 20 iki kez verildi
    for (int x : s) {
        std::cout << x << ' ';
    }
    std::cout << "(boyut " << s.size() << ")\n";

    auto [it, eklendi] = s.insert(30);
    std::cout << "30 eklendi mi? " << eklendi << ", *it = " << *it << '\n';
    auto [it2, eklendi2] = s.insert(30);
    std::cout << "30 tekrar eklendi mi? " << eklendi2 << '\n';

    std::cout << "40 var mı? " << (s.find(40) != s.end()) << '\n';
    std::cout << "25 sayısı: " << s.count(25) << '\n';
    std::cout << "silinen: " << s.erase(20) << '\n';

    std::cout << "en küçük " << *s.begin() << ", en büyük " << *s.rbegin()
              << '\n';
    std::cout << "lower_bound(25) = " << *s.lower_bound(25) << '\n';
    std::cout << "upper_bound(30) = " << *s.upper_bound(30) << '\n';

    if (auto bulunan = s.find(50); bulunan != s.end()) {   // C++17 başlatıcı
        std::cout << "50 bulundu\n";
    }
    return 0;
}
```

```
ÇIKTI
10 20 40 50 (boyut 4)
30 eklendi mi? 1, *it = 30
30 tekrar eklendi mi? 0
40 var mı? 1
25 sayısı: 0
silinen: 1
en küçük 10, en büyük 50
lower_bound(25) = 30
upper_bound(30) = 40
50 bulundu
```

- `insert` bir `pair<yineleyici, bool>` döndürür: `second` eklemenin yapılıp yapılmadığını, `first` ilgili elemanı gösterir. Zaten var olan elemanı eklemeye çalışmak hata değil, etkisiz bir işlemdir.
- `find` bulamazsa `end()` döndürür. `count` küme için yalnızca 0 ya da 1 verir. (C++20'de `contains` eklendi; C++17 kodunda `find` ya da `count` kullanırız.)
- Eleman sıralı olduğundan `lower_bound(x)` "x'e eşit ya da büyük ilk eleman", `upper_bound(x)` "x'ten büyük ilk eleman" yineleyicisini verir. Bu, aralık sorguları için ağacı kullanır: O(log n).
- Küme elemanları `const`'tır. Değiştirmek sıralamayı bozacağından izin verilmez; değiştirmek istiyorsak eskisini silip yenisini ekleriz.

Sıralamanın yönünü ikinci şablon argümanı belirler: `std::set<int, std::greater<int>>` büyükten küçüğe sıralar (`<functional>`). Kendi sınıfımızı kümeye koymak için ya `operator<` tanımlarız (Bölüm 32) ya da bir karşılaştırma nesnesi veririz:

▸ ogrenciKumesi.cpp

```cpp
#include <iostream>
#include <set>
#include <string>

struct Ogrenci {
    std::string ad;
    int no;
    bool operator<(const Ogrenci& d) const { return no < d.no; }
};

int main() {
    std::set<Ogrenci> sinif;
    sinif.insert({"Zeynep", 12});
    sinif.insert({"Can", 5});
    sinif.insert({"Deniz", 12});       // no aynı: "eşit" sayılır, eklenmez
    for (const Ogrenci& o : sinif) {
        std::cout << o.no << " " << o.ad << '\n';
    }
    return 0;
}
```

```
ÇIKTI
5 Can
12 Zeynep
```

> **◆ Kritik Nokta · Küme ve harita "eşitliği" == ile değil < ile belirler**
>
> İki eleman, `!(a < b) && !(b < a)` ise denktir ve ikincisi eklenmez. Yukarıda `Deniz`, adı farklı olsa da numarası `Zeynep` ile aynı olduğu için kümeye girmedi. Karşılaştırma kuralı sıkı zayıf sıralama olmalıdır: `<` doğru, `<=` yanlıştır.

`std::multiset` aynı şeydir, ancak **tekrara izin verir**. Elemanlar yine sıralı tutulur.

▸ **cokluKume.cpp**

```cpp
#include <iostream>
#include <set>

int main() {
    std::multiset<int> m{3, 1, 3, 2, 3};
    std::cout << "3 kaç kez? " << m.count(3) << ", boyut " << m.size() << '\n';

    m.erase(m.find(3));                  // yalnızca bir tane sil
    for (int x : m) {
        std::cout << x << ' ';
    }
    std::cout << '\n';

    std::cout << "erase(3) silinen: " << m.erase(3) << '\n';   // hepsini sil
    for (int x : m) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
3 kaç kez? 3, boyut 5
1 2 3 3
erase(3) silinen: 2
1 2
```

Değer ile `erase(3)` o değere eşit **tüm** elemanları siler; yalnızca birini silmek için yineleyici ile `erase(m.find(3))` yazılır.

Tekrarları ayıklamak

Bir `vector`'deki tekrarları atmanın iki yaygın yolu vardır. Sıra önemli değilse doğrudan kümeye aktarırız. Sıra önemliyse küme yalnızca "daha önce gördük mü?" sorusunu yanıtlar:

▸ **benzersiz.cpp**

```cpp
#include <iostream>
#include <set>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> yoklama{"Can", "Ece", "Ali", "Can", "Ece", "Can"};

    std::set<std::string> tekil(yoklama.begin(), yoklama.end());
    std::cout << yoklama.size() << " kayıt, " << tekil.size() << " kişi:";
    for (const std::string& ad : tekil) {
        std::cout << ' ' << ad;
```

```
    }
    std::cout << '\n';

    std::set<std::string> gorulen;
    std::vector<std::string> sirali;
    for (const std::string& ad : yoklama) {
        if (gorulen.insert(ad).second) {         // ilk kez eklendiyse
            sirali.push_back(ad);
        }
    }
    std::cout << "ilk geliş sırasıyla:";
    for (const std::string& ad : sirali) {
        std::cout << ' ' << ad;
    }
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
6 kayıt, 3 kişi: Ali Can Ece
ilk geliş sırasıyla: Can Ece Ali
```

35.5 map ve multimap: Anahtar–Değer Çiftleri

Telefon rehberinde ada bakıp numarayı buluruz. `std::map<K, V>` (`<map>`) her **anahtarı** (key) bir **değere** (value) eşler. Anahtarlar benzersizdir ve anahtara göre sıralı tutulur; değerler tekrar edebilir. Elemanların tipi `std::pair<const K, V>`'dir, yani `first` anahtar, `second` değerdir. Anahtar `const`'tır, çünkü değişirse ağaçtaki yeri bozulur.

▸ haritaTemel.cpp

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> stok;
    stok["kalem"] = 40;                      // yoksa ekler
    stok.insert({"defter", 25});
    stok.emplace("silgi", 60);

    auto [it, yeni] = stok.insert({"kalem", 99});   // var: dokunmaz
    std::cout << "eklendi mi? " << yeni << ", kalem " << it->second << '\n';

    stok.insert_or_assign("kalem", 45);    // varsa değeri değiştirir (C++17)
    stok.try_emplace("defter", 1000);      // varsa hiçbir şey yapmaz (C++17)

    for (const auto& [ad, adet] : stok) {  // anahtara göre sıralı
        std::cout << ad << " -> " << adet << '\n';
    }

    auto bulunan = stok.find("silgi");
    if (bulunan != stok.end()) {
        bulunan->second -= 10;             // değer değiştirilebilir
        std::cout << "silgi: " << bulunan->second << '\n';
    }

    try {
        std::cout << stok.at("cetvel") << '\n';
    } catch (const std::out_of_range&) {
```

```
        std::cout << "cetvel kayıtlı değil\n";
    }
    std::cout << "silinen: " << stok.erase("defter") << ", boyut "
              << stok.size() << '\n';
    return 0;
}
```

```
ÇIKTI
eklendi mi? 0, kalem 40
defter -> 25
kalem -> 45
silgi -> 60
silgi: 50
cetvel kayıtlı değil
silinen: 1, boyut 2
```

Ekleme ve erişim yöntemleri arasındaki farkları tabloda toplayalım:

İşlem	Anahtar yoksa	Anahtar varsa
`m[k] = v`	Ekler	Değeri değiştirir
`m.insert({k, v})`	Ekler	Dokunmaz
`m.insert_or_assign(k, v)`	Ekler	Değeri değiştirir
`m.try_emplace(k, args...)`	Değeri kurarak ekler	Hiçbir şey yapmaz (değer nesnesi kurulmaz)
`m.at(k)`	`out_of_range` istisnası	Değere referans
`m[k]` (okuma)	Varsayılan değerle **ekler**	Değere referans
`m.find(k)`	`end()`	Elemanı gösteren yineleyici
`m.count(k)`	0	1
`m.erase(k)`	0 döndürür	1 döndürür ve siler

▲ Sık Yapılan Hata · operator[] okurken bile eleman ekler

`m[k]`, anahtar yoksa onu varsayılan değerle (sayılarda 0) ekler. "Var mı?" diye bakmak için köşeli parantez kullanmak haritayı kirletir:

```
std::map<std::string, int> oy;
oy["Ayşe"] = 3;
if (oy["Mert"] > 0) { /* ... */ }   // oy artık 2 elemanlı!
```

Var olup olmadığını öğrenmek için `count` ya da `find`, değeri okumak için bulunmazsa istisna veren `at` kullanılır. Ayrıca `operator[]`, `const` haritada çalışmaz.

Dolaşırken silmek

`map` ve `set`'te bir eleman silmek yalnızca o elemanı gösteren yineleyiciyi geçersiz kılar. `erase(yineleyici)` sonraki elemanın yineleyicisini döndürür; bu, `vector`'deki kalıbın aynısıdır (Bölüm 26):

haritaSilme.cpp

```
#include <iostream>
#include <map>
#include <string>
```

```
int main() {
    std::map<std::string, int> notlar{
        {"Ayşe", 85}, {"Burak", 40}, {"Can", 55}, {"Deniz", 92}};

    for (auto it = notlar.begin(); it != notlar.end();) {
        if (it->second < 60) {
            it = notlar.erase(it);       // sonraki elemanı döndürür
        } else {
            ++it;
        }
    }
    for (const auto& [ad, puan] : notlar) {
        std::cout << ad << " " << puan << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Ayşe 85
Deniz 92
```

Değer olarak kapsayıcı: gruplama

Değer herhangi bir tip olabilir; `map<string, vector<int>>` bir anahtarın birden çok değerini toplamak için çok kullanışlıdır. `operator[]`'in "yoksa ekle" davranışı burada işimize yarar:

gruplama.cpp

```
#include <iostream>
#include <map>
#include <string>
#include <vector>

int main() {
    std::map<std::string, std::vector<int>> dersNotlari;
    dersNotlari["Matematik"].push_back(70);    // boş vector oluşur, sonra eklenir
    dersNotlari["Fizik"].push_back(85);
    dersNotlari["Matematik"].push_back(90);
    dersNotlari["Fizik"].push_back(60);
    dersNotlari["Fizik"].push_back(75);

    for (const auto& [ders, liste] : dersNotlari) {
        int toplam = 0;
        for (int n : liste) {
            toplam += n;
        }
        std::cout << ders << ": " << liste.size() << " not, ortalama "
                  << static_cast<double>(toplam) / liste.size() << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Fizik: 3 not, ortalama 73.3333
Matematik: 2 not, ortalama 80
```

Sıralama ölçütü ve multimap

Tıpkı `set` gibi, `map`'in üçüncü şablon argümanı karşılaştırma ölçütüdür: `std::map<std::string, int, std::greater<std::string>>` anahtarları ters sırada tutar.

`std::multimap` aynı anahtara birden çok değer koymaya izin verir. Anahtar benzersiz olmadığından **`operator[]` ve `at` yoktur**; bir anahtarın değerlerine `equal_range` ile ulaşılır. Eşit anahtarlı elemanlar eklenme sırasını korur.

▸ cokluHarita.cpp

```cpp
#include <iostream>
#include <map>
#include <string>

int main() {
    std::multimap<std::string, std::string> sozluk;
    sozluk.insert({"yaz", "mevsim"});
    sozluk.insert({"yaz", "kaleme al"});
    sozluk.insert({"al", "renk"});
    sozluk.insert({"yaz", "not et"});

    std::cout << "yaz için " << sozluk.count("yaz") << " anlam:\n";
    auto aralik = sozluk.equal_range("yaz");      // [ilk, son) çifti
    for (auto it = aralik.first; it != aralik.second; ++it) {
        std::cout << "  " << it->second << '\n';
    }
    std::cout << "toplam kayıt: " << sozluk.size() << '\n';
    return 0;
}
```

```
ÇIKTI
yaz için 3 anlam:
  mevsim
  kaleme al
  not et
toplam kayıt: 4
```

Kelime sıklığı

`map`'in klasik görevi saymaktır: kelime anahtar, geçiş sayısı değerdir. `sayac[kelime]++` yeni kelimeyi sıfırdan başlatır ve bir artırır. Sonuçları sıklığa göre görmek için ikinci bir harita kullanırız: `multimap<int, string, greater<int>>` sayıyı anahtar yapar ve büyükten küçüğe sıralar. İlk haritayı alfabetik gezdiğimiz için eşit sayılı kelimeler multimap'te de alfabetik sırada kalır.

▸ kelimeSay.cpp

```cpp
#include <functional>
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> sayac;
    std::string kelime;
    int toplam = 0;
    while (std::cin >> kelime) {            // girdi bitene kadar
        sayac[kelime]++;
        toplam++;
    }
```

```
    std::multimap<int, std::string, std::greater<int>> sikliga;
    for (const auto& [k, n] : sayac) {
        sikliga.insert({n, k});
    }
    for (const auto& [n, k] : sikliga) {
        std::cout << k << ": " << n << '\n';
    }
    std::cout << toplam << " kelime, " << sayac.size() << " farklı\n";
    return 0;
}
```

```
ÇIKTI
bir elma bir armut iki elma bir de kiraz
bir: 3
elma: 2
armut: 1
de: 1
iki: 1
kiraz: 1
9 kelime, 6 farklı
```

Haritadaki sıralama karakterlerin sayısal kodlarına göredir: büyük harfler küçüklerden önce gelir ve `ç`, `ş` gibi Türkçe harfler UTF-8'de `z`'den sonra yer alır. Doğru Türkçe sıralama için yerel ayar bilgisi gerekir ve bu kitabın kapsamı dışındadır.

35.6 Sırasız Kapsayıcılar

`map` ve `set` anahtarları sıralı tutmak için ağaç kullanır ve her işlem O(log n) sürer. Sıra bizim için önemli değilse **karma tablo** (hash table) daha hızlıdır. `std::unordered_map`, `std::unordered_set` (ve çoklu karşılıkları `unordered_multimap`, `unordered_multiset`) C++11 ile geldi. Arayüzleri `map` ve `set` ile neredeyse aynıdır: `[]`, `at`, `find`, `count`, `insert`, `erase`, `size`.

Karma tablo, anahtarı bir **karma işlevi** (hash function) ile bir sayıya çevirir ve bu sayıya göre elemanı bir "kovaya" (bucket) yerleştirir. Aynı kovaya birden çok eleman düşerse (çakışma) kova içinde aranır. Kova sayısı eleman sayısına göre otomatik büyütülür (yeniden karmalama, rehash). Ortalama arama, ekleme ve silme O(1)'dir; kötü durumda (çok çakışma) O(n)'e çıkabilir.

▸ sirasizOrnek.cpp

```
#include <iostream>
#include <map>
#include <string>
#include <unordered_map>
#include <unordered_set>

int main() {
    std::unordered_map<std::string, int> yas;
    yas["Ayşe"] = 20;
    yas["Mert"] = 25;
    yas.insert({"Elif", 22});
    yas["Mert"] += 1;

    std::cout << "Mert: " << yas.at("Mert") << '\n';
    std::cout << "Selin var mı? " << yas.count("Selin") << '\n';
    std::cout << "boyut: " << yas.size() << '\n';

    // dolaşma sırası belirsiz: düzenli çıktı için sıralı haritaya aktar
```

```
    std::map<std::string, int> sirali(yas.begin(), yas.end());
    for (const auto& [ad, y] : sirali) {
        std::cout << ad << " " << y << '\n';
    }

    std::unordered_set<int> gorulen;
    int dizi[] = {4, 8, 4, 15, 8, 16};
    for (int x : dizi) {
        if (!gorulen.insert(x).second) {
            std::cout << x << " tekrar etti\n";
        }
    }
    return 0;
}
```

```
ÇIKTI
Mert: 26
Selin var mı? 0
boyut: 3
Ayşe 20
Elif 22
Mert 26
4 tekrar etti
8 tekrar etti
```

Dolaşma sırası standartta belirtilmemiştir ve kütüphaneden kütüphaneye değişir (GNU libstdc++ ile LLVM libc++ farklı sıralar verebilir). Bu yüzden çıktısı sıraya bağlı kod yazmayız; sıralı sonuç gerekiyorsa `map` kullanırız ya da yukarıdaki gibi aktarırız.

`unordered_*` kapsayıcıların anahtar tipi için `std::hash` özelleştirmesi ve `==` gerekir. `int`, `string` ve diğer temel tipler için hazırdır. `pair` ya da kendi `struct`'ımız için karma işlevini kendimiz veririz:

ozelKarma.cpp

```
#include <functional>
#include <iostream>
#include <unordered_set>

struct Nokta {
    int x, y;
    bool operator==(const Nokta& d) const { return x == d.x && y == d.y; }
};

struct NoktaKarma {
    std::size_t operator()(const Nokta& n) const {
        return std::hash<int>()(n.x) * 31 + std::hash<int>()(n.y);
    }
};

int main() {
    std::unordered_set<Nokta, NoktaKarma> ziyaret;
    ziyaret.insert({1, 2});
    ziyaret.insert({3, 4});
    ziyaret.insert({1, 2});              // tekrar
    std::cout << "farklı nokta: " << ziyaret.size() << '\n';
    std::cout << "(3,4) ziyaret edildi mi? "
              << ziyaret.count({3, 4}) << '\n';
    return 0;
}
```

```
ÇIKTI
farklı nokta: 2
(3,4) ziyaret edildi mi? 1
```

`NoktaKarma`, `()` işlecini aşırı yükleyen bir sınıftır (işlev nesnesi, Bölüm 32); karma değeri `x` ve `y`'den birleştirir. Büyük veri eklemeden önce `reserve(n)` çağrısı yeniden karmalamaları azaltır.

35.7 Uyarlayıcılar: stack, queue, priority_queue

Bazen elemanlara istediğimiz yerden değil, yalnızca belirli bir uçtan ulaşmak isteriz. Bu kısıtlama bir sorun değil, bilinçli bir tasarımdır: arayüz dar olduğunda yanlış kullanım zorlaşır ve kodun niyeti okunur. Üç uyarlayıcı vardır; üçü de başka bir kapsayıcıyı sarar ve **yineleyicileri yoktur**, yani aralık tabanlı `for` ile dolaşılamaz. Elemanı görmenin tek yolu onu çıkarmaktır.

Uyarlayıcı	Düzen	Ekleme	Okuma	Çıkarma	Varsayılan altta yatan kapsayıcı
`stack` (`<stack>`)	Son giren ilk çıkar (LIFO)	`push`	`top`	`pop`	`deque`
`queue` (`<queue>`)	İlk giren ilk çıkar (FIFO)	`push`	`front`, `back`	`pop`	`deque`
`priority_queue` (`<queue>`)	En büyük önce	`push`	`top`	`pop`	`vector`

Bölüm 19'da yığını `struct` ve fonksiyonlarla kendimiz yazmıştık. Burada hazır sürümü kullanıyoruz. Çağrı bilgilerinin tutulduğu "yığın" bellek bölgesi ile bu veri yapısı aynı fikirden (LIFO) türeyen ama farklı şeylerdir.

> ◆ **Kritik Nokta · pop() değer döndürmez**
>
> `int x = s.pop();` derlenmez: `pop()` `void` döndürür. Önce `top()` ile okuyup sonra `pop()` ile atarız. Boş bir `stack`'te `top()` ya da boş `queue`'da `front()` çağırmak **tanımsız davranıştır**; istisna fırlatmaz. Bu yüzden döngü kalıbı `while (!s.empty())` biçimindedir.

stack: ters Polonya notasyonu

Yığının klasik uygulaması ifade hesaplamaktır. Ters Polonya (postfix) notasyonunda işleç, işlenenlerden sonra yazılır: `3 4 + 2 *` ifadesi `(3 + 4) * 2` demektir. Sayıları yığına atarız; işleç görünce iki sayıyı çıkarır, işlemi yapar ve sonucu geri koyarız.

▸ tersPolonya.cpp

```
#include <iostream>
#include <sstream>
#include <stack>
#include <string>

int main() {
    std::string satir;
    std::getline(std::cin, satir);
    std::istringstream akis(satir);

    std::stack<double> yigin;
    std::string simge;
```

```
    while (akis >> simge) {
        if (simge == "+" || simge == "-" || simge == "*" || simge == "/") {
            if (yigin.size() < 2) {
                std::cout << "hatalı ifade\n";
                return 1;
            }
            double b = yigin.top();
            yigin.pop();
            double a = yigin.top();
            yigin.pop();
            if (simge == "+") yigin.push(a + b);
            else if (simge == "-") yigin.push(a - b);
            else if (simge == "*") yigin.push(a * b);
            else yigin.push(a / b);
        } else {
            yigin.push(std::stod(simge));
        }
    }
    std::cout << "sonuç: " << yigin.top() << '\n';
    return 0;
}
```

```
ÇIKTI
5 1 2 + 4 * + 3 -
sonuç: 14
```

`5 1 2 + 4 * + 3 -` ifadesi `5 + (1 + 2) * 4 - 3` demektir: 5 + 12 - 3 = 14. Çıkarmada ve bölmede işlenen sırasına dikkat ettik: önce çıkan `b`, sonra çıkan `a` olur ve işlem `a - b` biçiminde yapılır.

queue: iş kuyruğu

isKuyrugu.cpp

```
#include <iostream>
#include <queue>
#include <string>

int main() {
    std::queue<std::string> isler;
    isler.push("rapor.pdf");
    isler.push("fatura.pdf");
    isler.push("sunum.pdf");
    std::cout << "ön: " << isler.front() << ", arka: " << isler.back() << '\n';

    while (!isler.empty()) {
        std::cout << "yazdırılıyor: " << isler.front() << '\n';
        isler.pop();
    }
    std::cout << "kuyruk boş, boyut " << isler.size() << '\n';
    return 0;
}
```

```
ÇIKTI
ön: rapor.pdf, arka: sunum.pdf
yazdırılıyor: rapor.pdf
yazdırılıyor: fatura.pdf
yazdırılıyor: sunum.pdf
kuyruk boş, boyut 0
```

priority_queue: öncelikli kuyruk

`priority_queue` içeride bir yığın (heap) tutar ve `top()` her zaman **en büyük** elemanı verir. `push` ve `pop` O(log n), `top` O(1)'dir. En küçüğü önce istiyorsak karşılaştırmayı `std::greater` ile tersine çevirirız. Bu kapsayıcının "yığın" adı bellek bölgesi ya da `stack` ile ilgili değildir; heap, ağaç biçimli bir veri düzenidir.

oncelikli.cpp

```
#include <functional>
#include <iostream>
#include <queue>
#include <string>
#include <utility>
#include <vector>

int main() {
    std::priority_queue<int> enBuyukOnce;
    for (int x : {5, 1, 8, 3}) {
        enBuyukOnce.push(x);
    }
    while (!enBuyukOnce.empty()) {
        std::cout << enBuyukOnce.top() << ' ';
        enBuyukOnce.pop();
    }
    std::cout << '\n';

    std::priority_queue<int, std::vector<int>, std::greater<int>> enKucukOnce;
    for (int x : {5, 1, 8, 3}) {
        enKucukOnce.push(x);
    }
    while (!enKucukOnce.empty()) {
        std::cout << enKucukOnce.top() << ' ';
        enKucukOnce.pop();
    }
    std::cout << '\n';

    std::priority_queue<std::pair<int, std::string>> isler;
    isler.push({2, "rapor"});
    isler.push({5, "yedekle"});
    isler.push({1, "temizle"});
    isler.push({5, "arşivle"});
    while (!isler.empty()) {
        std::cout << isler.top().first << " " << isler.top().second << '\n';
        isler.pop();
    }
    return 0;
}
```

```
ÇIKTI
8 5 3 1
1 3 5 8
5 yedekle
5 arşivle
2 rapor
1 temizle
```

Çiftler `pair` karşılaştırmasıyla sıralanır: önce öncelik, eşitse metin. Bu yüzden öncelik 5 olanlardan `"yedekle"`, `"arşivle"`'den büyük sayılır ve önce çıkar. Üç şablon argümanlı biçim `priority_queue<Tip, AltKapsayıcı, Karşılaştırma>` biçimindedir; `greater` kullanırken ikinci argümanı da yazmak gerekir.

Uyarlayıcının altta yatan kapsayıcısı seçilebilir: `std::stack<int, std::vector<int>>` yığını `vector` üzerinde kurar. `queue` için altta yatan kapsayıcı `push_back` ve `pop_front` sunmalıdır; bu yüzden `deque` ya da `list` olur, `vector` olamaz.

35.8 Yineleyici Geçersizleşmesi

Ekleme ve silme işlemleri, daha önce aldığımız yineleyicileri, işaretçileri ve başvuruları geçersiz kılabilir. Geçersiz yineleyiciyi kullanmak tanımsız davranıştır. Bölüm 26'da `vector` için gördük; aşağıdaki tablo bütün kapsayıcıları özetler:

Kapsayıcı	Ekleme	Silme
`vector`	Yeniden ayırma olduysa hepsi; olmadıysa ekleme noktasından sonrakiler	Silinen noktadan sonrakiler
`deque`	Ortaya eklemede hepsi; uçlara eklemede yineleyiciler geçersiz, elemanlara başvurular geçerli	Ortadan silmede hepsi; uçlardan silmede yalnızca silinenler
`list`, `forward_list`	Hiçbiri	Yalnızca silinen eleman
`set`, `map` (ve çoklu)	Hiçbiri	Yalnızca silinen eleman
`unordered_*`	Yeniden karmalama olduysa tüm yineleyiciler; elemanlara başvurular geçerli	Yalnızca silinen eleman

Pratik kural: ağaç ve bağlı liste tabanlı kapsayıcılarda bir elemanı silerken yalnızca o yineleyiciyi kaybederiz. Dolaşırken silme kalıbımız hepsinde aynıdır: `it = k.erase(it);` ve silmediysek `++it`.

35.9 Hangi Kapsayıcı, Ne Zaman?

Kapsayıcıları karşılaştırmak için işlem sürelerine bakarız. Aşağıda n eleman sayısıdır; O(1) sabit, O(log n) logaritmik, O(n) doğrusal süre demektir. Veri iki katına çıktığında O(log n) yalnızca bir adım artar, O(n) ise iki katına çıkar. Bir milyon elemanlı bir ağaçta arama yaklaşık yirmi adımda biter; bir milyon elemanlı `vector`'de sırasız aramak en kötü durumda bir milyon adım gerektirir.

Kapsayıcı	Erişim (`[]`)	Arama	Ekleme / silme	Not
`array`	O(1)	O(n)	Yok	Sabit boyut
`vector`	O(1)	O(n)	Sonda O(1)*; ortada O(n)	*Amortize
`deque`	O(1)	O(n)	İki uçta O(1); ortada O(n)	
`list`	Yok	O(n)	Konum biliniyorsa O(1)	Düğüm düğüm
`set`, `map`	Yok (`map::[]` O(log n))	O(log n)	O(log n)	Sıralı
`unordered_set/map`	Yok (`map::[]` ortalama O(1))	Ortalama O(1)	Ortalama O(1)	Sırasız
`stack`, `queue`	Yok	Yok	O(1)	Tek uç / iki uç
`priority_queue`	`top` O(1)	Yok	O(log n)	Yığın (heap)

Seçim için kısa bir karar sırası izleyebiliriz:

1. Başka bir gereksinim yoksa **`vector`**. Ardışık bellek önbelleğe uygundur; ortaya ekleme O(n) olsa bile küçük ve orta boyutlu veride çoğu zaman `list`'ten hızlıdır.
2. İki uçtan ekleyip silmek gerekiyorsa **`deque`**.
3. Bir yineleyici verilmiş konuma çok sık ekleme/silme ve diğer yineleyicilerin geçerli kalması gerekiyorsa **`list`**.
4. Anahtarla bulmak (sözlük, sayaç) gerekiyorsa **`map`**; sıra gerekmiyorsa **`unordered_map`**.
5. Tekrarsız küme ya da "var mı?" sorusu için **`set`** ya da **`unordered_set`**.
6. Yalnızca bir uçtan çalışan düzen için **`stack`** / **`queue`**; "en büyüğü ver" için **`priority_queue`**.

Emin olmadığımızda `vector` ile başlarız; ölçüm gerektirirse değiştiririz. Kapsayıcıları değiştirmek çoğu zaman bir `using` satırını ya da tip adını değiştirmek kadar kolaydır, çünkü ortak arayüzleri (`begin`, `end`, `size`, `insert`, `erase`) benzerdir.

Üç Dilde · STL ve C# koleksiyonları *C · C++ · C#*

C'de standart kapsayıcı yoktur; bağlı liste, yığın ve karma tabloyu Bölüm 19'daki gibi elle yazarız. C#'ta karşılıkları `System.Collections.Generic` ad alanındadır (Bölüm 54).

C++	C#	Not
`vector<T>`	`List<T>`	Büyüyen dizi
`array<T, N>`	`T[]`	C# dizisi çalışma anında boyutlanır
`deque<T>`	Doğrudan karşılığı yok	
`list<T>`	`LinkedList<T>`	Çift bağlı liste
`set<T>`	`SortedSet<T>`	Sıralı küme
`unordered_set<T>`	`HashSet<T>`	Karma tablo
`map<K, V>`	`SortedDictionary<K, V>`	Sıralı sözlük
`unordered_map<K, V>`	`Dictionary<K, V>`	En çok kullanılan sözlük
`multimap<K, V>`	`Dictionary<K, List<V>>`	Elle kurulur
`stack<T>`, `queue<T>`	`Stack<T>`, `Queue<T>`	`Pop` değer **döndürür**
`priority_queue<T>`	`PriorityQueue<TElement, TPriority>`	.NET 6+; en küçük öncelik önce çıkar
`pair<A, B>`, `tuple<...>`	`ValueTuple` `(A, B)`	`KeyValuePair<K, V>` sözlük elemanıdır

Üç Dilde · Anahtarla erişim: eksik anahtar davranışı *C · C++ · C#*

C++'ta `m[k]` eksik anahtarı sessizce ekler. C#'ta `Dictionary`'nin indeksleyicisi okurken eksik anahtar için `KeyNotFoundException` fırlatır; güvenli okuma için `TryGetValue` kullanılır. Ekleme ve değiştirme ise `d[k] = v` ile yapılır ve her iki durumda da çalışır.

```
std::map<std::string, int> m;
auto it = m.find("a");
if (it != m.end()) { /* it->second */ }
```

```
var d = new Dictionary<string, int>();
if (d.TryGetValue("a", out int deger)) { /* deger */ }
d["b"] = 5;                 // ekler ya da değiştirir
```

C#'ta `Dictionary` dolaşma sırası uygulamada genellikle ekleme sırasını andırır ama garanti edilmez; sıra gerekiyorsa `SortedDictionary` kullanılır. `foreach (var (anahtar, deger) in d)` biçimi, C++17'deki yapısal bağlamanın karşılığıdır.

Bölüm Özeti

- STL üç parçadır: kapsayıcı (veriyi saklar), yineleyici (elemanlar arasında gezer), algoritma (yineleyici aralığı üzerinde iş yapar). Algoritmalar kapsayıcının içini bilmez.
- Sıralı kapsayıcılar: `vector` (ardışık, varsayılan seçim), `deque` (iki uçtan hızlı, bloklu), `list` (çift bağlı, yineleyiciler geçersizleşmez, `splice` / `sort` üyeleri), `array` (sabit boyut). `list` rastgele erişimli olmadığından genel `std::sort` ile kullanılamaz.
- `pair` ve `tuple` karşılaştırılabilir; `first` / `second` ve `std::get<i>` ile erişilir. Yapısal bağlama `auto [a, b] = ...;` kopya, `auto& [a, b]` referans bağlar.
- `set` tekrarsız ve sıralıdır; `insert` `pair<yineleyici, bool>` döndürür; `lower_bound` / `upper_bound` aralık sorguları yapar. `multiset` tekrara izin verir; değerle `erase` hepsini siler.
- Eşitlik `<` ile belirlenir: `!(a < b) && !(b < a)` ise iki eleman aynıdır. Karşılaştırma `<=` olamaz.
- `map` anahtar–değer çiftlerini anahtara göre sıralı tutur. `m[k]` eksik anahtarı ekler; "var mı?" için `count` / `find`, istisna isteniyorsa `at` kullanılır. `insert` var olana dokunmaz, `insert_or_assign` değiştirir, `try_emplace` yoksa kurar.
- `multimap` aynı anahtara birden çok değer koyar; `[]` yoktur, `equal_range` kullanılır.
- `unordered_map` ve `unordered_set` karma tabloyla ortalama O(1) çalışır ama dolaşma sırası belirsizdir. Kendi tipimiz anahtar olacaksa `==` ve bir karma işlevi gerekir.
- `stack` (LIFO), `queue` (FIFO), `priority_queue` (en büyük önce) uyarlayıcılardır; yineleyicisi yoktur. `pop()` değer döndürmez; boşken `top()` / `front()` tanımsız davranıştır.
- `priority_queue` varsayılan olarak en büyüğü verir; en küçüğü için `priority_queue<T, vector<T>, greater<T>>`.
- Ekleme/silme yineleyicileri geçersiz kılabilir: `vector` ve `deque`'te geniş, `list`, `map`, `set`'te yalnızca silinen elemanı. Dolaşırken silmede `it = k.erase(it)` kalıbı kullanılır.
- Karmaşıklık: `vector` araması O(n); `map` / `set` O(log n); `unordered_*` ortalama O(1). Emin değilsek `vector` ile başlarız.
- C#'ta karşılıkları `List`, `LinkedList`, `SortedSet`, `HashSet`, `SortedDictionary`, `Dictionary`, `Stack`, `Queue`, `PriorityQueue`'dur.

Alıştırmalar

1. Aşağıdakilerden hangisi `deque` için sabit sürede çalışır, ama `vector` için çalışmaz?

A) `push_back`
B) `operator[]`
C) `push_front`
D) `size()`
E) `clear()`

2. Aşağıdaki kodun çıktısı nedir?

```
std::map<std::string, int> m;
m["a"];
m["b"] = 2;
m.insert({"b", 9});
m.insert_or_assign("a", 7);
std::cout << m["a"] << " " << m["b"] << " " << m.size();
```

A) `0 2 2`
B) `7 9 2`
C) `7 2 2`
D) `7 2 3`
E) `out_of_range` istisnası

3. Bir `unordered_set`'te `find` işleminin ortalama karmaşıklığı nedir?

A) O(1)
B) O(log n)
C) O(n)
D) O(n log n)
E) $O(n^2)$

4. `std::multiset<int> m{3, 3, 1};` için `m.erase(3);` çağrısından sonra `m.size()` kaçtır?

A) 0
B) 1
C) 2
D) 3
E) Tanımsız davranış

5. "Son giren ilk çıkar" düzeniyle çalışan, geri alma (undo) gibi işler için en uygun uyarlayıcı hangisidir?

A) `queue`
B) `priority_queue`
C) `set`
D) `stack`
E) `multimap`

6. Bu programın çıktısı nedir?

```
#include <iostream>
#include <set>

int main() {
    std::set<int> s{8, 3, 5, 3, 9};
    s.insert(5);
    s.insert(4);
    s.erase(8);
```

```
    auto it = s.lower_bound(5);
    std::cout << s.size() << " " << *it << " " << *s.rbegin() << '\n';
    for (int x : s) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

7. Bu programın çıktısı nedir?

```
#include <iostream>
#include <queue>
#include <stack>

int main() {
    std::stack<int> s;
    std::queue<int> q;
    std::priority_queue<int> p;
    for (int i = 1; i <= 4; i++) {
        s.push(i);
        q.push(i);
        p.push(i % 2 == 0 ? i * 10 : i);
    }
    s.pop();
    q.pop();
    p.pop();
    std::cout << s.top() << " " << q.front() << " " << p.top() << '\n';
    std::cout << s.size() << " " << q.size() << " " << p.size() << '\n';
    return 0;
}
```

8. Bu programın çıktısı nedir?

```
#include <iostream>
#include <set>
#include <string>
#include <utility>

int main() {
    std::set<std::pair<int, std::string>> s;
    s.insert({2, "b"});
    s.insert({1, "z"});
    s.insert({2, "a"});
    s.insert({1, "z"});
    for (const auto& [n, t] : s) {
        std::cout << n << t << ' ';
    }
    std::cout << s.size() << '\n';
    return 0;
}
```

9. Aşağıdaki kod belirli girdilerde tanımsız davranışa yol açar. Hatayı bulun ve düzeltin.

```
std::map<std::string, int> m{{"a", 0}, {"b", 3}, {"c", 0}};
for (auto it = m.begin(); it != m.end(); ++it) {
    if (it->second == 0) {
        m.erase(it);
    }
}
```

10. Aşağıdaki kod neden derlenmez?

```
std::set<int> s{1, 2, 3};
*s.begin() = 10;
```

11. Aşağıdaki kodda iki hata vardır. Bulun.

```
std::stack<int> s;
int x = s.pop();
```

12. `unordered_map<string, int>` elemanlarını dolaşan bir programdan hangi çıkarım doğrudur?

    A) Elemanlar ekleme sırasıyla gelir.
    B) Elemanlar anahtara göre alfabetik gelir.
    C) Sıra belirsizdir; farklı kütüphanelerde farklı olabilir.
    D) Elemanlar her zaman değere göre sıralıdır.
    E) Dolaşma yapılamaz, yalnızca `find` kullanılabilir.

13. Satır satır okuduğu her ifadedeki `()`, `[]` ve `{}` parantezlerinin dengeli olup olmadığını `stack<char>` ile denetleyen bir program yazın. Örnek: `{[()]}` dengeli, `((a+b]` dengesiz.

14. Klavyeden bir satır okuyup her harfin kaç kez geçtiğini `map<char, int>` ile sayan ve harfleri alfabetik sırada `harf: sayı` biçiminde yazdıran bir program yazın (boşlukları atlayın).

15. İki `vector<int>`'in ortak (kesişim) elemanlarını `set` kullanarak, tekrarsız ve sıralı olarak yazdıran bir program yazın.

16. Klavyeden okunan sayılar arasından en büyük üç tanesini `priority_queue` ile bulan ve büyükten küçüğe yazdıran bir program yazın. İpucu: boyutu 3'ü aşmayacak bir en küçük-önce yığın tutun.

BÖLÜM 36

STL Algoritmaları, Yineleyiciler ve Lambda İfadeleri

BU BÖLÜMDE

- Yineleyicinin (iterator) ne olduğunu, `begin()` / `end()` ve yarı açık aralık kuralını, yineleyici türlerini
- `<algorithm>` ve `<numeric>` içindeki sıralama, arama, sayma, toplama ve dönüştürme algoritmalarını kullanmayı
- Silme-silme deyimini (`remove_if` + `erase`) ve neden gerektiğini
- Lambda ifadesinin sözdizimini, yakalama listesini (`[=]`, `[&]`, `[x]`) ve `mutable`'ı
- `std::function` ile fonksiyon, işaretçi ve lambda'ları ortak bir tipte tutmayı
- Bir kayıt listesini farklı ölçütlere göre sıralayıp süzmeyi
- C `qsort`, C++ lambda ve C# LINQ yaklaşımlarını karşılaştırmayı

Bölüm 35'te veriyi nasıl saklayacağımızı gördük: `vector`, `list`, `set`, `map` ve diğerleri. Bu bölümde STL'nin geri kalan iki ayağını tamamlıyoruz: kapsayıcının içinde gezinmeyi sağlayan **yineleyiciler** ve kapsayıcılar üzerinde hazır iş yapan **algoritmalar**. Üçüncü araç olan **lambda ifadesi**, algoritmalara kendi ölçütümüzü satır içinde vermemizi sağlar.

C'de bir diziyi sıralamak için `qsort`'a bir karşılaştırma fonksiyonunun adresini veriyorduk (Bölüm 12). C++ aynı fikri üç yönden geliştirir: algoritmalar tipe duyarlıdır (`void *` yok), kapsayıcıdan bağımsızdır ve karşılaştırma ölçütü, çağrının tam ortasına yazılan küçük bir isimsiz fonksiyon olabilir. Bu bölümün sonunda "öğrencileri nota göre sırala, geçenleri süz, ortalamayı hesapla" gibi işleri döngü yazmadan, birkaç satırda yapabileceğiz.

36.1 Yineleyici Nedir?

Bölüm 26'da `insert` ve `erase` çağırırken yineleyiciyi "bir elemanın konumunu gösteren nesne" diye tanıtmıştık. Şimdi tanımını tamamlayalım.

> **Tanım: Yineleyici (iterator)**
>
> Bir kapsayıcının elemanlarından birini gösteren, bir sonrakine geçebilen ve gösterdiği elemana erişim sağlayan nesnedir. İşaretçinin genelleştirilmiş hâlidir: aynı işaretler (`*`, `->`, `++`) kullanılır, ama arkasındaki kapsayıcı dizi, bağlı liste ya da ağaç olabilir.

Neden gerekli? `sort`'un yalnızca `vector<int>` için yazılmış olduğunu düşünelim. `vector<string>`, `array<double, 5>` ve C dizisi için ayrı ayrı yazmamız gerekirdi. STL'nin çözümü şudur: algoritma

kapsayıcının içini bilmez, yalnızca iki yineleyici alır ve aradaki elemanlarla çalışır. Kapsayıcı kendi yineleyicisini sağlar; algoritma yalnızca `*it`, `++it` ve `it != son` gibi ortak işlemleri kullanır.

```
  Kapsayıcılar          Yineleyiciler           Algoritmalar
 +-------------+                              +--------------+
 | vector      |--+                        +---| sort         |
 | list        |--+                        +---| find_if      |
 | deque       |--+-->  [ begin, end )  --+---| accumulate   |
 | set / map   |--+                        +---| transform    |
 | C dizisi    |--+                        +---| ...          |
 +-------------+                              +--------------+
```

begin(), end() ve yarı açık aralık

Her kapsayıcının iki temel yineleyicisi vardır: `begin()` ilk elemanı, `end()` ise **son elemandan bir sonrasını** gösterir.

```
  indeks:   0      1      2      3
         +------+------+------+------+
         | Pzt  | Sal  | Car  | Per  |   (bu konumda eleman yok)
         +------+------+------+------+
          ^                           ^
       begin()                     end()
```

Bu seçimin iki nedeni vardır. Dolaşma döngüsü `it != end()` ile sade biçimde yazılır ve kapsayıcı boşsa `begin() == end()` olduğundan döngü hiç dönmez. Ayrıca `[begin, end)` biçimindeki **yarı açık aralık**, uzunluğun `end - begin` ile hesaplanmasını ve iki aralığın aralarında boşluk ya da çakışma olmadan birleştirilmesini kolaylaştırır. Bölüm 26'da değindiğimiz bu kural bu bölümdeki her algoritmada geçerlidir: ilk sınır dahil, ikinci sınır hariç.

Temel işlemler:

İşlem	Anlamı
`*it`	Yineleyicinin gösterdiği elemana eriş
`it->uye`	Gösterilen elemanın üyesine eriş (`(*it).uye` ile aynı)
`++it`	Sonraki elemana geç
`--it`	Önceki elemana geç (çift yönlü yineleyicilerde)
`it + n`, `it - n`	`n` eleman ileri/geri (yalnızca rastgele erişimli yineleyicilerde)
`b - a`	İki yineleyici arasındaki eleman sayısı
`it1 == it2`, `it1 != it2`	Aynı konumu mu gösteriyorlar?

Bir örnekte hepsini görelim:

▸ yineleyiciGiris.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> gunler{"Pzt", "Sal", "Car", "Per"};
```

```
    std::vector<std::string>::iterator it = gunler.begin();
    std::cout << *it << '\n';
    ++it;
    std::cout << *it << " (" << it->size() << " harf)\n";
    it += 2;
    std::cout << *it << '\n';
    std::cout << "end - begin = " << gunler.end() - gunler.begin() << '\n';

    for (auto i = gunler.begin(); i != gunler.end(); ++i) {
        std::cout << *i << ' ';
    }
    std::cout << '\n';

    *gunler.begin() = "Pazartesi";              // yineleyiciyle değiştir
    for (auto r = gunler.rbegin(); r != gunler.rend(); ++r) {
        std::cout << *r << ' ';                 // sondan başa
    }
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
Pzt
Sal (3 harf)
Per
end - begin = 4
Pzt Sal Car Per
Per Car Sal Pazartesi
```

Üç nokta dikkat çeker:

1. `std::vector<std::string>::iterator` uzun bir tip adıdır; pratikte `auto` yazarız (Bölüm 22).
2. `it->size()` ile `(*it).size()` aynıdır; `->` işaretçilerdeki gibi üyeye erişir.
3. `rbegin()` ve `rend()` **ters yineleyici** (reverse iterator) verir. Bunlarda `++` geriye gider; sondan başa dolaşmanın en temiz yolu budur.

Aralık tabanlı `for` (Bölüm 24), perde arkasında tam olarak bu döngüyü kurar: `begin()` ile başlar, `end()`'e kadar `++` ile ilerler ve her adımda `*it`'i değişkene bağlar. Yineleyiciyi doğrudan kullanmamız gereken durumlar, yalnızca bir kısım aralığı dolaşmak, dolaşırken silmek ya da bir algoritmanın döndürdüğü konumu işlemektir.

> **▲ Sık Yapılan Hata · end() yineleyicisini açmak**
>
> `end()` hiçbir elemanı göstermez; ona `*` uygulamak tanımsız davranıştır. Boş bir kapsayıcının `begin()`'i de `end()`'e eşittir ve açılamaz. Her `*it` öncesinde `it != v.end()` olduğundan emin olmak gerekir. Bu kuralı aşağıda `find` ile arama yaparken sürekli uygulayacağız.

const_iterator ve cbegin()

Kapsayıcıyı `const` olarak aldığımızda (örneğin `const std::vector<int>&` parametresi) `begin()` salt okunur bir yineleyici döndürür; elemanı değiştirmeye çalışırsak derleme hatası alırız. Kapsayıcı `const` olmasa bile salt okunur yineleyici istiyorsak `cbegin()` ve `cend()` kullanırız:

```
std::vector<int> v{1, 2, 3};
auto it = v.cbegin();
// *it = 10;      // derleme hatası: const_iterator ile yazılamaz
```

Okuma amaçlı dolaşmalarda `cbegin()` / `cend()` yazmak, niyeti açıkça gösterir. Tür adı `const_iterator`'dır; `auto` ile bunu çoğu zaman yazmayız.

Yineleyici türleri

Her yineleyici her işlemi desteklemez; bunu kapsayıcının iç yapısı belirler. Tekli bağlı listede geriye gidemeyiz, bağlı listede `it + 5` yazmak ise 5 adım ilerlemeyi gerektirir ve bu yüzden desteklenmez. STL yineleyicileri, desteklediği işlemlere göre beş kategoriye ayırır:

Tür	Ek yetenek	Örnek kapsayıcılar
Giriş (input)	Bir kez ileri okuma	`cin` akışı (`istream_iterator`)
Çıkış (output)	Bir kez ileri yazma	`cout` akışı (`ostream_iterator`), `back_inserter`
İleri (forward)	Tekrar tekrar ileri gitme	`forward_list`, `unordered_set`
Çift yönlü (bidirectional)	`--` ile geri gitme	`list`, `set`, `map`
Rastgele erişimli (random access)	`it + n`, `it[n]`, `<` ile karşılaştırma	`vector`, `array`, `deque`, string, C dizisi

Kategoriler iç içedir: rastgele erişimli yineleyici, çift yönlü olanın her işlemini de yapar. Bu ayrımın pratik sonucu şudur: bir algoritma hangi kategoriyi gerektiriyorsa kapsayıcının yineleyicisi en az o düzeyde olmalıdır. Örneğin `std::sort` rastgele erişim ister; `std::list` bunu sağlamadığı için `std::sort(l.begin(), l.end())` derlenmez. `list` bu yüzden kendi `l.sort()` üye fonksiyonunu sunar.

```
std::list<int> l{3, 1, 2};
// std::sort(l.begin(), l.end());   // derleme hatası: rastgele erişim gerekir
l.sort();                           // list'in kendi sıralaması
```

Yineleyiciyi ilerletmek: next, prev, advance, distance

Rastgele erişimli olmayan bir yineleyicide `it + 3` yazamayız; ama `<iterator>` başlığındaki yardımcılar her kategoride çalışır (rastgele erişimde tek adımda, diğerlerinde adım adım ilerleyerek):

İşlev	Anlamı
`std::next(it, n)`	`it`'ten `n` adım ilerideki yineleyiciyi döndürür (`n` varsayılan 1)
`std::prev(it, n)`	`n` adım gerideki yineleyiciyi döndürür
`std::advance(it, n)`	`it`'in kendisini `n` adım ilerletir
`std::distance(a, b)`	`a`'dan `b`'ye kaç adım olduğunu verir

yineleyiciYardimci.cpp

```
#include <iostream>
#include <iterator>
#include <list>

int main() {
    std::list<int> l{10, 20, 30, 40, 50};

    auto ikinci = std::next(l.begin());          // 20
    auto sonEleman = std::prev(l.end());         // 50
```

```
    std::cout << *ikinci << ' ' << *sonEleman << '\n';

    auto it = l.begin();
    std::advance(it, 3);                        // 3 adım ilerle
    std::cout << *it << '\n';
    std::cout << std::distance(l.begin(), it) << '\n';
    return 0;
}
```

```
ÇIKTI
20 50
40
3
```

`std::prev(l.end())` bize son elemanı verir; `end()`'in kendisi açılamadığından bu, son elemana ulaşmanın yoludur. Ayrıca `std::begin(c)` ve `std::end(c)` serbest fonksiyonları C dizilerinde de çalışır (Bölüm 26'da `std::sort(std::begin(c), std::end(c))` yazmıştık), bu yüzden dizi ve kapsayıcı için aynı kodu yazabiliriz.

Dolaşırken silmek ve geçersizleşme

Bölüm 26'da `erase` sonrası eldeki yineleyicinin geçersiz olduğunu ve `it = v.erase(it)` kalıbının gerektiğini görmüştük. Aynı kural bu bölümde de geçerlidir ve bir sonraki alt başlıktaki `remove_if` deyimi, bu döngünün çok daha kısa ve verimli biçimidir. Genel ilke: kapsayıcının yapısını değiştiren her işlem (`push_back` yeniden ayırma yaparsa, `insert`, `erase`, `clear`) o kapsayıcıya ait yineleyicileri geçersiz kılabilir; geçersiz yineleyiciyi kullanmak tanımsız davranıştır. `list` ve `map` gibi düğüm tabanlı kapsayıcılarda yalnızca **silinen** elemanın yineleyicisi geçersiz olur; diğerleri yerinde kalır.

36.2 Hazır Algoritmalar

`<algorithm>` başlığı yüzü aşkın algoritma içerir; toplama gibi sayısal olanlar `<numeric>`'tedir. Hepsi aynı kalıbı izler: `(baslangic, bitis, ...)` biçiminde bir yineleyici çifti alırlar, çoğu da bir koşul ya da işlem alabilir. Bölüm 26'da `sort`, `find`, `count`, `max_element` ve `reverse` ile tanışmıştık. Bu bölümde önce koşul gerektirmeyen algoritmaları tamamlayacağız; ardından lambda ifadelerini öğrenip koşul ve ölçüt alan sürümlere (`find_if`, `count_if`, `transform`, `remove_if`, özel karşılaştırmalı `sort`) geçeceğiz.

Bir algoritma çağırırken akılda tutulacak üç kural:

- Aralık geçerli olmalıdır: `baslangic`'tan `++` ile ilerleyince `bitis`'e ulaşılmalıdır. İki ayrı kapsayıcının yineleyicilerini karıştırmak tanımsız davranıştır.
- Algoritmalar **kapsayıcıyı değil, elemanlarını** değiştirir. Hiçbir algoritma kapsayıcının boyutunu doğrudan değiştiremez; çünkü kapsayıcıyı tanımaz, yalnızca yineleyici görür. Bu, `remove_if` kısmında önemli sonuçlar doğuracak.
- Hedefe yazan algoritmalar (`copy`, `transform`) hedefte yeterli yer olmasını bekler.

36.3 Temel Algoritmalar

Toplama, en küçük ve en büyük

`std::accumulate` (`<numeric>`) bir aralıktaki elemanları ilk değerin üzerine ekler. En küçük ve en büyük elemanı `min_element` ile `max_element` bulur; `minmax_element` ikisini tek geçişte bulur ve bir çift döndürür. Üçü de değer değil **yineleyici** döndürür (Bölüm 26).

▸ **temelAlgoritma.cpp**

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> notlar{75, 45, 88, 91, 60, 45, 100};

    int toplam = std::accumulate(notlar.begin(), notlar.end(), 0);
    double ort = static_cast<double>(toplam) / notlar.size();
    std::cout << "toplam " << toplam << ", ortalama " << ort << '\n';

    auto sonuc = std::minmax_element(notlar.begin(), notlar.end());
    std::cout << "en küçük " << *sonuc.first
              << ", en büyük " << *sonuc.second << '\n';

    std::sort(notlar.begin(), notlar.end());
    for (int n : notlar) {
        std::cout << n << ' ';
    }
    std::cout << '\n';

    std::cout << std::boolalpha;
    std::cout << "88 var mı? " << std::binary_search(notlar.begin(),
                                                     notlar.end(), 88) << '\n';
    std::cout << "89 var mı? " << std::binary_search(notlar.begin(),
                                                     notlar.end(), 89) << '\n';

    auto alt = std::lower_bound(notlar.begin(), notlar.end(), 80);
    std::cout << "80'den küçük olmayan ilk: " << *alt
              << ", indeks " << (alt - notlar.begin()) << '\n';

    auto sonEleman = std::unique(notlar.begin(), notlar.end());
    notlar.erase(sonEleman, notlar.end());
    std::cout << "tekrarsız " << notlar.size() << " eleman\n";
    return 0;
}
```

```
ÇIKTI
toplam 504, ortalama 72
en küçük 45, en büyük 100
45 45 60 75 88 91 100
88 var mı? true
89 var mı? false
80'den küçük olmayan ilk: 88, indeks 4
tekrarsız 6 eleman
```

Programdaki yeni algoritmalar şunlardır:

- `binary_search(b, e, x)` sıralı bir aralıkta `x`'in var olup olmadığını ikili aramayla söyler (`bool` döndürür). Adım sayısı eleman sayısının logaritması kadardır: bir milyon elemanda yaklaşık 20 karşılaştırma. **Aralığın sıralı olması şarttır**; sıralı değilse sonuç anlamsızdır.
- `lower_bound(b, e, x)` sıralı aralıkta `x`'ten küçük olmayan ilk elemanın yineleyicisini verir. `x` aralıkta varsa onu, yoksa sonrasındaki ilk elemanı gösterir; sıralı bir diziye eleman eklerken eklenecek yeri bulmak için de kullanılır. `upper_bound` ise `x`'ten **büyük** ilk elemanı bulur.
- `unique(b, e)` art arda gelen eşit elemanları tek eleman bırakacak biçimde başa toplar ve yeni mantıksal sonun yineleyicisini döndürür. Kapsayıcının boyutunu değiştirmez; bu yüzden ardından `erase(sonEleman, v.end())` çağırmak gerekir. Tekrarlıları ayıklamak için önce `sort`, sonra `unique` + `erase` uygulanır. Bu iki adımlı kalıbın nedenini `remove_if` bölümünde ayrıntılı göreceğiz.

`std::minmax_element` çiftin `first` üyesinde en küçüğü, `second` üyesinde en büyüğü tutar. `std::min` ve `std::max` ise iki değeri (ya da süslü parantezle bir listeyi) karşılaştırır ve yineleyici değil değer döndürür: `std::max(3, 9)`, `std::min({4, 8, 1})`.

> **▲ Sık Yapılan Hata · accumulate'in ilk değeri sonucun tipini belirler**
>
> `accumulate`'in üçüncü parametresi hem başlangıç değeridir hem de toplamın tipidir. `0` yazarsak toplam `int` olarak tutulur ve ondalıklı elemanlar her adımda kırpılır.

`accumulateTuzak.cpp`

```cpp
#include <functional>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<double> fiyatlar{1.5, 2.5, 3.25};
    std::cout << std::accumulate(fiyatlar.begin(), fiyatlar.end(), 0) << '\n';
    std::cout << std::accumulate(fiyatlar.begin(), fiyatlar.end(), 0.0) << '\n';

    std::vector<int> v{1, 2, 3, 4, 5};
    std::cout << std::accumulate(v.begin(), v.end(), 1,
                                 std::multiplies<int>()) << '\n';
    return 0;
}
```

```
ÇIKTI
6
7.25
120
```

İlk satırda toplam her adımda `int`'e çevrildi: 0 + 1.5 -> 1, 1 + 2.5 -> 3, 3 + 3.25 -> 6. Doğrusu `0.0` yazmaktır. Dördüncü parametre, toplama yerine başka bir işlem vermemizi sağlar: `std::multiplies<int>()` (`<functional>`) çarpmayı temsil eder ve ilk değer bu yüzden 1'dir; sonuç 5! = 120 olur. Kendi işlemimizi lambda ile de verebiliriz (bir sonraki ana başlıkta göreceğiz).

Kopyalamak ve doldurmak

`copy` bir aralığı başka bir yere kopyalar, `fill` aralığı tek değerle doldurur, `iota` (`<numeric>`) artan değerlerle doldurur. `copy` hedefte yer olmasını bekler: hedef önceden yeterince büyük değilse tanımsız davranış

oluşur. Boş bir `vector`'e eklemek için `std::back_inserter` (`<iterator>`) kullanılır; bu yardımcı her yazmada hedefte `push_back` çağırır. Akışa yazmak için `std::ostream_iterator` kullanılır.

kopyaDoldur.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <iterator>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> a(5);
    std::iota(a.begin(), a.end(), 10);          // 10 11 12 13 14

    std::vector<int> b;
    std::copy(a.begin(), a.end(), std::back_inserter(b));

    std::fill(a.begin(), a.begin() + 2, 0);     // ilk iki eleman 0
    std::copy(a.begin(), a.end(),
              std::ostream_iterator<int>(std::cout, " "));
    std::cout << '\n';
    std::copy(b.begin(), b.end(),
              std::ostream_iterator<int>(std::cout, ", "));
    std::cout << '\n';
    return 0;
}
```

```
ÇIKTI
0 0 12 13 14
10, 11, 12, 13, 14,
```

`ostream_iterator<int>(std::cout, " ")` her yazılan eleman için akışa elemanı ve ayırıcıyı gönderen bir çıkış yineleyicisidir; bu sayede bir kapsayıcıyı tek satırda yazdırabiliriz. Son elemandan sonra da ayırıcı yazıldığına dikkat edelim.

36.4 Lambda İfadeleri

Algoritmaların büyük kısmı bir **koşul** ya da **ölçüt** bekler: "hangi elemanlar", "hangi sıraya göre". Bölüm 12'de bunu fonksiyon işaretçisiyle, ayrı bir fonksiyon yazarak yapıyorduk. Koşul çoğu zaman tek satırlıktır ve yalnızca bir çağrıda kullanılır; ona ayrı bir ad vermek ve kodu iki yere bölmek yorucudur. Lambda ifadesi, bu işlevi çağrının tam içine yazmamızı sağlar. Bölüm 25'te kısaca tanıdığımız bu aracı şimdi tüm ayrıntılarıyla görelim.

> **Tanım: Lambda ifadesi (lambda expression)**
>
> Adı olmayan, bulunduğu yerde tanımlanan ve çevresindeki yerel değişkenleri yakalayabilen fonksiyon nesnesidir. Genel biçimi `[yakalama](parametreler) -> dönüşTipi { gövde }` şeklindedir; dönüş tipi çoğu zaman yazılmaz.

En basit biçimiyle bir lambda, bir değişkende saklanıp fonksiyon gibi çağrılabilir:

lambdaTemel.cpp

```cpp
#include <iostream>

int main() {
```

```
    auto kare = [](int x) { return x * x; };
    auto topla = [](int a, int b) -> int { return a + b; };

    std::cout << kare(7) << '\n';
    std::cout << topla(3, 4) << '\n';
    std::cout << [](int x) { return x + 1; }(4) << '\n';   // tanımla ve çağır
    return 0;
}
```

```
ÇIKTI
49
7
5
```

Parçaları sırayla inceleyelim:

- `[]` **yakalama listesidir**; boş olması lambda'nın dış değişkenlere erişmediğini söyler.
- `(int x)` sıradan bir parametre listesidir.
- `-> int` dönüş tipini açıkça yazar. Gövdede tek bir `return` varsa ya da tüm `return`'ler aynı tipteyse derleyici tipi kendisi çıkarır; bu yüzden `kare` için yazmadık. Farklı tiplerde `return` varsa (`return 1;` ve `return 2.5;`) dönüş tipini yazmamız gerekir.
- Lambda'nın tipi derleyicinin ürettiği, adı olmayan benzersiz bir tiptir. Bu yüzden değişkenini `auto` ile tanımlarız. Son satırdaki gibi, tanımın hemen ardına `(4)` yazarak lambda'yı anında çağırabiliriz; ama bu biçim pratikte nadiren işe yarar.

Yakalama listesi

Lambda gövdesi, çevredeki yerel değişkenleri yalnızca yakalama listesinde belirtilmişse görebilir. Yakalama iki yöntemle yapılır:

Yazım	Anlamı
`[]`	Hiçbir şey yakalanmaz
`[x]`	`x` **değerle** yakalanır: lambda kendi kopyasını tutar
`[&x]`	`x` **referansla** yakalanır: lambda asıl değişkeni görür ve değiştirebilir
`[=]`	Gövdede kullanılan her değişken değerle yakalanır
`[&]`	Gövdede kullanılan her değişken referansla yakalanır
`[=, &toplam]`	Varsayılan değerle, ama `toplam` referansla
`[&, x]`	Varsayılan referansla, ama `x` değerle

▸ yakalama.cpp

```
#include <iostream>

int main() {
    std::cout << std::boolalpha;

    int esik = 70;
    auto degerle = [esik](int n) { return n >= esik; };
    auto referansla = [&esik](int n) { return n >= esik; };

    esik = 90;                                  // lambda'lar tanımlandıktan sonra
    std::cout << degerle(75) << '\n';           // kopya hâlâ 70
```

```
    std::cout << referansla(75) << '\n';        // asıl değişken artık 90

    int sayac = 0;
    auto artir = [&sayac]() { ++sayac; };
    artir();
    artir();
    std::cout << "sayac = " << sayac << '\n';

    int c = 10;
    auto m = [c]() mutable {
        c += 5;
        return c;
    };
    std::cout << m() << ' ' << m() << ' ' << c << '\n';
    return 0;
}
```

```
ÇIKTI
true
false
sayac = 2
15 20 10
```

Değerle yakalama, lambda'nın **tanımlandığı andaki** değeri saklar: `esik` sonradan 90 olsa da `degerle` 70'i görmeye devam eder. Referansla yakalanan değişken ise canlıdır; değişen değeri hemen görürüz ve lambda içinden değiştirebiliriz (`artir` bu yüzden `sayac`'ı artırabildi).

Değerle yakalanan değişkenler lambda içinde varsayılan olarak **salt okunurdur**; `c += 5` yazmak derleme hatasıdır. Kopyayı lambda içinde değiştirmek istiyorsak parametre listesinden sonra `mutable` yazarız. Değişen şey lambda'nın kendi kopyasıdır: `m()` ikinci çağrıda 20 döndürür çünkü kopya çağrılar arasında korunur, ama dışarıdaki `c` hep 10 kalır.

> **▲ Sık Yapılan Hata · Referansla yakalanan değişken ölmüş olabilir**
>
> `[&]` ile yakalanan yerel değişkenler, lambda onlardan uzun yaşarsa **sarkan referansa** dönüşür. Bir fonksiyondan lambda döndürüyorsak ya da onu daha sonra çalıştırmak üzere saklıyorsak yerel değişkenleri değerle yakalamalıyız. Bir algoritmaya hemen çalışıp biten bir lambda verirken `[&]` güvenlidir. Genel öneri: ne yakalandığını açıkça yazmak (`[esik, &toplam]`) okunurluğu artırır; `[=]` ve `[&]` kısa lambda'larda kabul edilebilir.

Sınıf üye fonksiyonunun içinde yazılan lambda, nesnenin üyelerine `this` yakalayarak erişir (`[this]`); sınıfları Bölüm 30'da işlediğimizden ayrıntıya girmiyoruz.

Genel lambda

C++14'ten itibaren lambda parametresinde `auto` yazılabilir. Derleyici, çağrıldığı her tip için lambda'nın gövdesini ayrı üretir; bu, Bölüm 33'te göreceğimiz şablonların kısa bir biçimidir:

```
▸ genelLambda.cpp
```

```
#include <iostream>
#include <string>

int main() {
    auto topla = [](auto a, auto b) { return a + b; };

    std::cout << topla(3, 4) << '\n';
```

```
    std::cout << topla(1.5, 2.25) << '\n';
    std::cout << topla(std::string("C"), std::string("++")) << '\n';
    return 0;
}
```

```
ÇIKTI
7
3.75
C++
```

Aynı lambda `int`, `double` ve `string` ile çalıştı. Algoritmalarla birlikte bu, "elemanın tipi ne olursa olsun çalışan" ölçüt yazmamıza olanak verir.

36.5 Koşul Alan Algoritmalar

Lambda'yı tanıdığımıza göre, koşul ya da işlem bekleyen algoritmaları kullanabiliriz. Bu algoritmaların çoğu, adı `_if` ile biten bir sürüme sahiptir: `find` bir **değer** arar, `find_if` bir **koşulu** sağlayan ilk elemanı arar; `count` ile `count_if` de aynı ilişkidedir.

Algoritma	Ne yapar?	Döndürdüğü
`find_if(b, e, kosul)`	Koşulu sağlayan ilk eleman	Yineleyici (yoksa `e`)
`count_if(b, e, kosul)`	Koşulu sağlayan eleman sayısı	Sayı
`any_of`, `all_of`, `none_of`	Bazısı / hepsi / hiçbiri koşulu sağlıyor mu?	`bool`
`for_each(b, e, islem)`	Her elemana işlemi uygular	İşlem nesnesi
`transform(b, e, hedef, islem)`	Her elemana işlem uygular, sonucu hedefe yazar	Hedefin sonu
`remove_if(b, e, kosul)`	Koşulu sağlayanları dışarıda bırakacak biçimde kaydırır	Yeni mantıksal son
`copy_if(b, e, hedef, kosul)`	Koşulu sağlayanları hedefe kopyalar	Hedefin sonu
`sort(b, e, karsilastir)`	Verilen ölçüte göre sıralar	—

Koşul, elemanı alıp `bool` döndüren herhangi bir çağrılabilir olabilir; çoğu zaman bir lambda'dır.

▸ kosulluAlgoritma.cpp

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> notlar{75, 45, 88, 91, 60, 45, 100};
    std::cout << std::boolalpha;

    auto it = std::find_if(notlar.begin(), notlar.end(),
                           [](int n) { return n > 90; });
    if (it != notlar.end()) {
        std::cout << "90'dan büyük ilk not: " << *it
                  << " (indeks " << (it - notlar.begin()) << ")\n";
    }

    int dusuk = std::count_if(notlar.begin(), notlar.end(),
                              [](int n) { return n < 60; });
    std::cout << "60'ın altında: " << dusuk << " not\n";
```

```
    std::cout << "100 var mı? "
              << std::any_of(notlar.begin(), notlar.end(),
                             [](int n) { return n == 100; }) << '\n';
    std::cout << "hepsi 45 ve üstü mü? "
              << std::all_of(notlar.begin(), notlar.end(),
                             [](int n) { return n >= 45; }) << '\n';
    std::cout << "negatif yok mu? "
              << std::none_of(notlar.begin(), notlar.end(),
                              [](int n) { return n < 0; }) << '\n';

    int toplam = 0;
    int adet = 0;
    std::for_each(notlar.begin(), notlar.end(), [&toplam, &adet](int n) {
        if (n >= 75) {
            toplam += n;
            adet++;
        }
    });
    std::cout << adet << " notun toplamı " << toplam << '\n';
    return 0;
}
```

```
ÇIKTI
90'dan büyük ilk not: 91 (indeks 3)
60'ın altında: 2 not
100 var mı? true
hepsi 45 ve üstü mü? true
negatif yok mu? true
4 notun toplamı 354
```

Üç ayrıntı dikkat çekiyor. `find_if` her zaman yineleyici döndürür; bulunamadıysa `end()` gelir ve açmadan önce denetlemek zorundayız. `any_of`, `all_of` ve `none_of` boş aralıkta sırasıyla `false`, `true` ve `true` döndürür. `for_each` içindeki lambda `[&toplam, &adet]` ile iki dış değişkeni referansla yakaladığı için onları değiştirebildi.

transform

`transform`, her elemanı bir işlemden geçirip sonucu başka bir yere (ya da aynı yere) yazar. Üçüncü parametre sonucun yazılacağı yerin başlangıcıdır:

```
std::transform(kaynakBas, kaynakSon, hedefBas, islem);
```

Hedef kaynakla aynı olabilir; bu durumda elemanlar yerinde dönüşür. Farklı bir kapsayıcıya yazıyorsak orada yeterli yer olmalıdır (boş bir `vector` için `back_inserter` kullanılır).

Silme-silme deyimi: remove_if ve erase

Algoritmalar yalnızca yineleyici gördüğü için kapsayıcının boyutunu değiştiremez. Bu yüzden `remove_if` elemanları gerçekte silmez: koşulu **sağlamayan** elemanları aralığın başına kaydırır ve bunların bittiği yeri, yani yeni mantıksal sonu döndürür. Bu noktadan sonraki elemanların değerleri belirsizdir. Gerçek silme, kapsayıcının kendi `erase` üyesiyle yapılır:

```
v.erase(std::remove_if(v.begin(), v.end(), kosul), v.end());
```

Bu kalıba **silme-silme deyimi** (erase-remove idiom) denir. Adım adım görelim:

transformSilme.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <iterator>
#include <vector>

void yazdir(const std::vector<int>& v) {
    for (int x : v) {
        std::cout << x << ' ';
    }
    std::cout << "(boyut " << v.size() << ")\n";
}

int main() {
    std::vector<int> v{3, 8, 12, 5, 6, 7};

    std::vector<int> kareler;
    std::transform(v.begin(), v.end(), std::back_inserter(kareler),
                   [](int x) { return x * x; });
    yazdir(kareler);

    std::transform(v.begin(), v.end(), v.begin(),
                   [](int x) { return x + 1; });       // yerinde dönüşüm
    yazdir(v);

    auto yeniSon = std::remove_if(v.begin(), v.end(),
                                  [](int x) { return x % 2 == 0; });
    std::cout << "kalan eleman: " << (yeniSon - v.begin())
              << ", boyut hâlâ " << v.size() << '\n';

    v.erase(yeniSon, v.end());
    yazdir(v);
    return 0;
}
```

ÇIKTI

```
9 64 144 25 36 49 (boyut 6)
4 9 13 6 7 8 (boyut 6)
kalan eleman: 3, boyut hâlâ 6
9 13 7 (boyut 3)
```

Çift sayıları silmek istedik: `remove_if` üç tek sayıyı (9, 13, 7) başa topladı ve `yeniSon` onların bittiği yeri gösterdi; boyut hâlâ 6'ydı. `erase` çağrısıyla artık kullanılmayan kuyruk gerçekten silindi. Bölüm 26'da dolaşırken `it = v.erase(it)` ile yaptığımız işin bu hâli hem daha kısadır hem de her elemanı bir kez kaydırdığı için verimlidir: tek tek `erase`, her silmede sağdaki elemanları kaydırır.

▲ Sık Yapılan Hata · remove_if'i erase olmadan kullanmak

`v.erase` çağrılmazsa kapsayıcının boyutu aynı kalır ve sonda eskiden kalan, anlamsız (ama geçerli) değerler durur. Yalnızca `std::remove_if(...)` yazıp sonucu yok saymak, en sık yapılan STL hatalarındandır. Bu aynı mantık `std::remove` (değere göre) ve `std::unique` için de geçerlidir.

Tarihçe · C++20: erase ve erase_if

C++20, bu iki adımı tek çağrıya indiren `std::erase(v, deger)` ve `std::erase_if(v, kosul)` işlevlerini ekledi. Kitabın C++17 örneklerinde bunları kullanmıyoruz; silme-silme deyimi hem eski kodda hem de C++17'de geçerli olan biçimdir ve eski projelerde sık görülür.

36.6 Sıralama Ölçütleri

`std::sort` varsayılan olarak `<` ile küçükten büyüğe sıralar. Başka bir sıra istediğimizde üçüncü parametre olarak bir **karşılaştırma** veririz: iki eleman `a` ve `b` alan ve `a`'nın `b`'den **önce gelmesi gerekiyorsa** `true` döndüren bir çağrılabilir. Hazır karşılaştırıcılar da vardır (`std::greater<int>()` büyükten küçüğe), ama kayıtlar gibi karmaşık tiplerde lambda yazarız.

Üç ölçütle çalışalım: nota göre azalan (eşitlikte ada göre), ada göre ve yaşa göre artan. Üçüncüsünde `std::stable_sort` kullanacağız; bu algoritma eşit elemanların **eski göreli sırasını korur**, `sort` ise korumayı garanti etmez.

siralamaOlcut.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct Ogrenci {
    std::string ad;
    int notu;
    int yas;
};

void yazdir(const std::string& baslik, const std::vector<Ogrenci>& liste) {
    std::cout << baslik << ":";
    for (const Ogrenci& o : liste) {
        std::cout << ' ' << o.ad << '(' << o.notu << ',' << o.yas << ')';
    }
    std::cout << '\n';
}

int main() {
    std::vector<Ogrenci> ogrenciler{
        {"Ayşe", 85, 20}, {"Mehmet", 70, 22}, {"Elif", 85, 19},
        {"Can", 55, 21},  {"Zeynep", 92, 20}, {"Mert", 70, 18}};

    auto notaGore = ogrenciler;
    std::sort(notaGore.begin(), notaGore.end(),
              [](const Ogrenci& a, const Ogrenci& b) {
                  if (a.notu != b.notu) {
                      return a.notu > b.notu;       // not büyük olan önce
                  }
                  return a.ad < b.ad;               // eşitse ad sırası
              });
    yazdir("nota göre", notaGore);

    auto adaGore = ogrenciler;
    std::sort(adaGore.begin(), adaGore.end(),
              [](const Ogrenci& a, const Ogrenci& b) { return a.ad < b.ad; });
    yazdir("ada göre ", adaGore);

    auto yasaGore = ogrenciler;
    std::stable_sort(yasaGore.begin(), yasaGore.end(),
                     [](const Ogrenci& a, const Ogrenci& b) {
                         return a.yas < b.yas;
                     });
    yazdir("yaşa göre", yasaGore);
    return 0;
}
```

```
ÇIKTI
nota göre: Zeynep(92,20) Ayşe(85,20) Elif(85,19) Mehmet(70,22) Mert(70,18) Can(55,21)
ada göre : Ayşe(85,20) Can(55,21) Elif(85,19) Mehmet(70,22) Mert(70,18) Zeynep(92,20)
yaşa göre: Mert(70,18) Elif(85,19) Ayşe(85,20) Zeynep(92,20) Can(55,21) Mehmet(70,22)
```

Karşılaştırma lambda'sı parametreleri `const Ogrenci&` olarak alır; kayıtları kopyalamak hem yavaştır hem de gereksizdir (Bölüm 25). Birinci ölçütteki iki aşamalı karşılaştırma ("önce nota, eşitse ada bak") çok anahtarlı sıralamanın standart kalıbıdır. Sonuncu ölçütte `Ayşe` ile `Zeynep` ikisi de 20 yaşında; `stable_sort` onları özgün listedeki sırasıyla bıraktı.

> **◆ Kritik Nokta · Karşılaştırma katı zayıf sıralama olmalıdır**
>
> `sort`'a verilen karşılaştırmanın "önce gelir" anlamında tutarlı olması gerekir: `a` ile `a` karşılaştırıldığında `false` dönmelidir, `a`, `b`'den önce geliyorsa `b`, `a`'dan önce gelmemelidir. Bu koşullara **katı zayıf sıralama** (strict weak ordering) denir. En sık yapılan hata `<` yerine `<=` yazmaktır: eşit elemanlar için `true` döndüğünden kural bozulur ve `sort` tanımsız davranışla çökebilir ya da sınır dışına yazabilir. Eşitlik durumunda her zaman `false` döndürmeliyiz.

36.7 std::function ve Çağrılabilir Nesneler

Bölüm 25'te gördüğümüz `std::function<dönüşTipi(parametreler)>`, fonksiyon, lambda ve fonksiyon nesnelerini ortak bir tipte tutar. Algoritmalara lambda'yı doğrudan vermek en hızlı yoldur (derleyici çağrıyı satır içine açabilir); `std::function` ise çağrılabilir şeyi **saklamamız**, bir kapsayıcıya koymamız ya da imzası sabit bir fonksiyona parametre olarak vermemiz gerektiğinde işe yarar.

▸ fonksiyonKap.cpp

```cpp
#include <functional>
#include <iostream>
#include <map>
#include <string>

double bol(double a, double b) {
    return a / b;
}

std::function<int(int)> carpanUret(int k) {
    return [k](int x) { return x * k; };       // k değerle yakalandı
}

int main() {
    std::map<std::string, std::function<double(double, double)>> islemler;
    islemler["topla"] = [](double a, double b) { return a + b; };
    islemler["cikar"] = [](double a, double b) { return a - b; };
    islemler["carp"] = [](double a, double b) { return a * b; };
    islemler["bol"] = bol;                      // sıradan fonksiyon

    for (const auto& cift : islemler) {
        std::cout << cift.first << ": " << cift.second(12, 4) << '\n';
    }

    auto ucle = carpanUret(3);
    std::cout << "ucle(14) = " << ucle(14) << '\n';
    return 0;
}
```

```
ÇIKTI
bol: 3
carp: 48
cikar: 8
topla: 16
ucle(14) = 42
```

`map` elemanları anahtara göre sıralı tuttuğu için çıktı alfabetik gelir (Bölüm 35). `carpanUret` içindeki lambda `k`'yı **değerle** yakaladığı için fonksiyon bittikten sonra da çalışmaya devam eder; `[&k]` yazsaydık sarkan referans olurdu. Üç aracın karşılaştırması:

Özellik	Fonksiyon işaretçisi	Lambda	`std::function`
Durum (yakalama) taşır mı?	Hayır	Evet	Evet
Tipi yazılabilir mi?	`int (*)(int)`	Hayır (`auto`)	`std::function<int(int)>`
Farklı çağrılabilirleri tek tipte toplar mı?	Yalnızca düz fonksiyonları	Hayır	Evet
Çağrı maliyeti	Dolaylı çağrı	Çoğu zaman satır içi	Dolaylı çağrı, küçük ek yük

36.8 Örnek: Öğrenci Listesini Süzmek ve Özetlemek

Şimdiye kadarki parçaları tek bir programda birleştirelim. Kayıt listesinden geçenleri süzen, ortalamayı hesaplayan, en yüksek notu bulan ve geçemeyenleri listeden silen bir program yazalım. Süzme ölçütünü fonksiyona parametre olarak `std::function` ile geçireceğiz; böylece aynı `suz` fonksiyonu her koşulla çalışır.

▸ ogrenciSuz.cpp

```
#include <algorithm>
#include <functional>
#include <iomanip>
#include <iostream>
#include <iterator>
#include <numeric>
#include <string>
#include <vector>

struct Ogrenci {
    std::string ad;
    int notu;
    int yas;
};

using Kosul = std::function<bool(const Ogrenci&)>;

std::vector<Ogrenci> suz(const std::vector<Ogrenci>& liste, const Kosul& kosul) {
    std::vector<Ogrenci> sonuc;
    std::copy_if(liste.begin(), liste.end(), std::back_inserter(sonuc), kosul);
    return sonuc;
}

void adlariYaz(const std::string& baslik, const std::vector<Ogrenci>& liste) {
    std::cout << baslik << ":";
    for (const Ogrenci& o : liste) {
        std::cout << ' ' << o.ad;
    }
    std::cout << '\n';
}
```

```
int main() {
    std::vector<Ogrenci> ogrenciler{
        {"Ayşe", 85, 20}, {"Mehmet", 70, 22}, {"Elif", 85, 19},
        {"Can", 55, 21},  {"Zeynep", 92, 20}, {"Mert", 70, 18}};

    int toplam = std::accumulate(
        ogrenciler.begin(), ogrenciler.end(), 0,
        [](int t, const Ogrenci& o) { return t + o.notu; });
    double ortalama = static_cast<double>(toplam) / ogrenciler.size();
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "ortalama: " << ortalama << '\n';

    auto enIyi = std::max_element(
        ogrenciler.begin(), ogrenciler.end(),
        [](const Ogrenci& a, const Ogrenci& b) { return a.notu < b.notu; });
    std::cout << "en yüksek: " << enIyi->ad << " (" << enIyi->notu << ")\n";

    int esik = 80;
    adlariYaz("ortalama üstü",
              suz(ogrenciler, [ortalama](const Ogrenci& o) {
                  return o.notu > ortalama;
              }));
    adlariYaz("80 ve üstü",
              suz(ogrenciler, [esik](const Ogrenci& o) {
                  return o.notu >= esik;
              }));
    adlariYaz("20 yaş altı",
              suz(ogrenciler, [](const Ogrenci& o) { return o.yas < 20; }));

    ogrenciler.erase(
        std::remove_if(ogrenciler.begin(), ogrenciler.end(),
                       [](const Ogrenci& o) { return o.notu < 60; }),
        ogrenciler.end());
    adlariYaz("kalanlar silindi", ogrenciler);
    std::cout << "geçen sayısı: " << ogrenciler.size() << '\n';
    return 0;
}
```

```
ÇIKTI
ortalama: 76.17
en yüksek: Zeynep (92)
ortalama üstü: Ayşe Elif Zeynep
80 ve üstü: Ayşe Elif Zeynep
20 yaş altı: Elif Mert
kalanlar silindi: Ayşe Mehmet Elif Zeynep Mert
geçen sayısı: 5
```

Programda öğrendiklerimizin hemen hepsini kullandık: `accumulate`'e dördüncü parametre olarak iki parametreli lambda vermek, `max_element`'e karşılaştırma vermek, `copy_if` + `back_inserter` ile süzmek, `[ortalama]` ve `[esik]` gibi değerle yakalama ve silme-silme deyimi. Ortalama ile eşik dışarıdaki değişkenler olduğu hâlde ölçüt lambda'sının içine yakalama listesi aracılığıyla taşındı; ayrı bir fonksiyon yazsaydık bu değerleri parametre ya da global değişken olarak geçirmek zorunda kalacaktık.

> **Üç Dilde · Sıralama ve süzme: C, C++ ve C#** *C · C++ · C#*
>
> **C'de** sıralama için `qsort` ve ayrı bir karşılaştırma fonksiyonu gerekir; elemanlar `void *` ile gelir, tip denetimi yoktur (Bölüm 12). Süzme için döngü yazarız. **C++'ta** algoritma tipe duyarlıdır ve ölçüt çağrının içine lambda olarak yazılır. **C#'ta** koleksiyonlar üzerinde sorgu yazmak için LINQ kullanılır (Bölüm 56); sorgu zincirleme çağrılarla kurulur ve lambda ifadeleri `=>` ile yazılır (Bölüm 55).

```c
/* C */
int karsilastir(const void *x, const void *y) {
    const struct Ogrenci *a = x, *b = y;
    return (a->notu < b->notu) - (a->notu > b->notu);   /* azalan */
}
qsort(liste, adet, sizeof(liste[0]), karsilastir);
```

```cpp
// C++
std::sort(l.begin(), l.end(),
          [](const Ogrenci& a, const Ogrenci& b) { return a.notu > b.notu; });
```

```csharp
// C#: LINQ
var sirali = liste.OrderByDescending(o => o.Notu).ThenBy(o => o.Ad);
var gecenler = liste.Where(o => o.Notu >= 60).ToList();
double ort = liste.Average(o => o.Notu);
```

İş	C	C++ (STL)	C# (LINQ)
Sıralama	`qsort` + karşılaştırma fonksiyonu	`std::sort(b, e, lambda)`	`OrderBy(...)`, `list.Sort(...)`
Süzme	Elle döngü	`copy_if`, `remove_if` + `erase`	`Where(...)`
Dönüştürme	Elle döngü	`std::transform`	`Select(...)`
Toplama	Elle döngü	`std::accumulate`	`Sum(...)`, `Average(...)`
Koşul sorgusu	Elle döngü	`any_of`, `all_of`, `find_if`	`Any(...)`, `All(...)`, `First(...)`

Önemli bir fark: STL algoritmaları kapsayıcıyı **yerinde** değiştirir (`sort` listenin kendisini sıralar) ve işlemler hemen yapılır. LINQ sorguları ise özgün koleksiyonu değiştirmez; yeni bir sıra döndürür ve çoğu zaman sonuç istenene kadar çalışmaz (ertelenmiş yürütme).

36.9 Hangi Algoritma, Hangi Kapsayıcı?

Algoritmaları kapsayıcılarla birlikte seçerken şu noktalara bakalım:

- Sıralı bir aralık varsa `binary_search`, `lower_bound`, `upper_bound` kullanırız; sıralı değilse `find` ya da `find_if` doğrusal tarar. `set` ve `map` zaten sıralı ve arama için kendi üye `find` fonksiyonlarını sunar; onlarda genel `std::find` yerine üye sürümü kullanırız, çünkü ağaç yapısından yararlanır (Bölüm 35).
- `std::sort` rastgele erişimli yineleyici ister: `vector`, `array`, `deque` ve C dizisiyle çalışır; `list`'te `l.sort()` kullanılır. `set` ve `map` sıralı tutulduğundan sıralanmaz.
- Silmek için `vector`, `deque` ve `string`'de silme-silme deyimi uygulanır. `list`'te `l.remove_if(kosul)` üye fonksiyonu doğrudan siler.
- Algoritma çağrısı ve `for` döngüsü arasında seçim yaparken: amacı adlandıran algoritma (`count_if`, `any_of`) çoğu zaman döngüden daha okunaklıdır. Karmaşık, birden çok değişkeni birlikte güncelleyen bir işlem lambda içinde karışıksa, düz bir aralık tabanlı `for` yazmak daha iyi olabilir.

Bölüm Özeti

- STL üç parçadan oluşur: kapsayıcılar (veriyi saklar), yineleyiciler (elemanlar arasında gezinir) ve algoritmalar (iş yapar). Algoritma kapsayıcıyı tanımaz, yalnızca yineleyici çifti görür.
- Yineleyici işaretçinin genelleştirilmiş hâlidir: `*it` elemanı verir, `it->uye` üyeye erişir, `++it` ilerletir.
- `begin()` ilk elemanı, `end()` son elemandan bir ötesini gösterir; aralıklar yarı açıktır. `end()` açılamaz.
- `rbegin()` / `rend()` ters, `cbegin()` / `cend()` salt okunur dolaşma sağlar; `std::next`, `std::prev`, `std::advance` ve `std::distance` her tür yineleyicide çalışır.
- Yineleyici kategorileri giriş, çıkış, ileri, çift yönlü ve rastgele erişimlidir; `sort` rastgele erişim ister, bu yüzden `list` kendi `sort`'unu sunar.
- `accumulate`'in ilk değeri sonucun tipini belirler (`0` ile `0.0` farkı); `binary_search` ve `lower_bound` yalnızca sıralı aralıkta anlamlıdır.
- Lambda `[yakalama](parametreler) -> dönüşTipi { gövde }` biçimindedir; dönüş tipi çoğunlukla çıkarılır, `auto` parametre genel lambda yapar (C++14).
- `[x]` değerle, `[&x]` referansla yakalar; `[=]` ve `[&]` varsayılan yakalamadır. Değerle yakalanan kopya `mutable` olmadan değiştirilemez. Fonksiyondan çıkan lambda yerel değişkenleri değerle yakalamalıdır.
- `find_if`, `count_if`, `any_of`, `all_of`, `none_of`, `for_each`, `copy_if`, `transform` koşul ya da işlem alır.
- `remove_if` boyutu değiştirmez; silme-silme deyimi `v.erase(remove_if(...), v.end())` ile tamamlanır. `unique` de aynı biçimde `erase` ister.
- `sort` karşılaştırması katı zayıf sıralama olmalıdır (`<=` yazılmaz); çok anahtarlı sıralamada önce ana ölçüt, eşitse ikinci ölçüt karşılaştırılır; `stable_sort` eşit elemanların sırasını korur.
- `std::function` fonksiyon, lambda ve fonksiyon nesnelerini ortak bir tipte tutar; fonksiyon işaretçisinden farkı durum taşıyabilmesi, doğrudan lambda'dan farkı küçük bir çağrı maliyeti getirmesidir.
- C'de `qsort` + fonksiyon işaretçisi, C++'ta algoritma + lambda, C#'ta LINQ + lambda aynı ihtiyaca yanıt verir.

Alıştırmalar

1. `std::vector<int> v{1, 2, 3};` için `v.end()` ifadesi neyi gösterir?
   - **A)** Son elemanı (3)
   - **B)** İlk elemanı (1)
   - **C)** Son elemandan bir sonraki konumu
   - **D)** `nullptr`'ı
   - **E)** Hiçbir şeyi; derleme hatasıdır
2. Aşağıdakilerden hangisi `std::list<int>` için derlenmez?
   - **A)** `std::next(l.begin())`
   - **B)** `std::distance(l.begin(), l.end())`
   - **C)** `std::find(l.begin(), l.end(), 3)`
   - **D)** `std::sort(l.begin(), l.end())`
   - **E)** `l.sort()`
3. `std::accumulate(v.begin(), v.end(), 0)` çağrısını `vector<double>` üzerinde yazarsak ne olur?

A) Derleme hatası
B) Sonuç `double` olur ve ondalıklar korunur
C) Toplam `int` olarak tutulur, ondalıklar her adımda kırpılır
D) Çalışma zamanı istisnası
E) Sonuç her zaman 0 olur

4. Aşağıdaki yakalama listelerinden hangisi `toplam` değişkenini lambda içinde değiştirmeye izin verir?

A) `[]`
B) `[toplam]`
C) `[=]`
D) `[&toplam]`
E) `[toplam]` ve parantezlerden önce `const`

5. `v.erase(std::remove_if(v.begin(), v.end(), kosul), v.end());` ifadesinde `erase` çağrısı neden gereklidir?

A) `remove_if` hiçbir şey yapmaz
B) `remove_if` yalnızca elemanları kaydırır, kapsayıcının boyutunu değiştiremez
C) `erase` yineleyiciyi geçersiz kılar
D) `remove_if` sıralı aralık ister
E) `erase` olmadan program derlenmez

6. Aşağıdaki karşılaştırmayla `sort` neden tehlikelidir?

```
std::sort(v.begin(), v.end(), [](int a, int b) { return a <= b; });
```

A) Küçükten büyüğe sıralamaz
B) Katı zayıf sıralama kuralını bozar; eşit elemanlarda tanımsız davranışa yol açabilir
C) Lambda `int` alamaz
D) `sort` karşılaştırma kabul etmez
E) Tehlikeli değildir

7. `std::binary_search` hangi koşulda doğru çalışır?

A) Aralık sıralı olmalıdır
B) Aralık tekrarsız elemanlardan oluşmalıdır
C) Yalnızca `int` ile çalışır
D) Yalnızca `vector` ile çalışır
E) Aralık en az 100 eleman içermelidir

8. Aşağıdaki lambda neden derlenmez?

```
int sayac = 0;
auto f = [sayac]() { sayac++; return sayac; };
```

A) Dönüş tipi yazılmadı
B) `sayac` değerle yakalandı ve `mutable` olmadığı için lambda içinde değiştirilemez
C) Lambda `int` döndüremez
D) Yakalama listesi boş olmalıdır
E) Lambda en az bir parametre almalıdır

9. Bu programın çıktısı nedir?

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{5, 2, 8, 1, 9, 3};
    auto it = std::find_if(v.begin(), v.end(), [](int x) { return x > 6; });
    std::cout << *it << ' ' << (it - v.begin()) << '\n';
    std::cout << std::count_if(v.begin(), v.end(),
                               [](int x) { return x % 2 == 1; }) << '\n';
    std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
    for (int x : v) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
}
```

10. Bu programın çıktısı nedir?

```
#include <iostream>

int main() {
    int x = 1;
    auto a = [x]() mutable { x += 10; return x; };
    auto b = [&x]() { x += 100; return x; };
    std::cout << a() << ' ' << a() << ' ' << x << '\n';
    std::cout << b() << ' ' << x << '\n';
}
```

11. Bu programın çıktısı nedir?

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{4, 7, 4, 9, 4, 1};
    auto son = std::remove(v.begin(), v.end(), 4);
    std::cout << (son - v.begin()) << ' ' << v.size() << '\n';
    v.erase(son, v.end());
    for (int x : v) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
}
```

12. Aşağıdaki kodda iki hata vardır; bulup düzeltin.

```
std::vector<int> a{1, 2, 3};
std::vector<int> b;
std::copy(a.begin(), a.end(), b.begin());

std::list<int> l{3, 1, 2};
std::sort(l.begin(), l.end());
```

13. Aşağıdaki fonksiyonun hatasını bulun.

```
std::function<int(int)> uret() {
    int k = 5;
    return [&k](int x) { return x + k; };
}
```

14. Klavyeden 0 girilene kadar tamsayı okuyup sayıları `vector`'e ekleyen `tekCiftAyir` programını yazın. Çift sayıların sayısını `count_if` ile bulsun, tek sayıları `remove_if` + `erase` ile silip kalanları (çiftleri) yazdırsın.

```
ÇIKTI
Sayılar (0 ile biter): 4 7 10 3 8 0
Çift sayı: 3
Kalanlar: 4 10 8
```

15. Kelimeleri (`std::string`) önce uzunluğa göre, uzunluklar eşitse alfabetik sıraya göre sıralayan `kelimeSirala` programını yazın. Girdi tek satırda, boşlukla ayrılmış beş kelimedir; `cin >>` ile okuyun.

```
ÇIKTI
Beş kelime: elma kiraz fil armut ayva
fil ayva elma armut kiraz
```

16. `Urun` yapısını (`ad`, `fiyat`) tanımlayın. Dört ürünü `vector`'e koyup fiyatı `esik` değişkeninden büyük olanları `copy_if` ile başka bir vector'e süzün (`esik` lambda'ya yakalama ile geçsin). Süzülenleri fiyata göre azalan sıralayıp adlarıyla yazdırın; ardından `accumulate` ile süzülenlerin toplam fiyatını bulun.

BÖLÜM 37

Dosya İşlemleri ve Akışlar

BU BÖLÜMDE

- `<fstream>` başlığındaki `ofstream`, `ifstream` ve `fstream` sınıflarını, açma kiplerini ve `is_open()` denetimini
- Dosyadan `>>`, `getline` ve `get` ile okumayı; döngü koşulunu okumanın kendisiyle kurmayı ve `eof()` tuzağını
- Akış durum bayraklarını (`fail`, `eof`, `clear`) ve dosyaya ekleme kipini (`ios::app`)
- `stringstream` yardımıyla CSV satırlarını ayrıştırıp bir `vector`'e doldurmayı
- Dosya akışının RAII sayesinde kendiliğinden kapanmasını, istisna durumunda bile
- `write`/`read`, `seekg`/`seekp` ile ikili dosyada kayıt okuyup güncellemeyi
- `<filesystem>` (C++17) ile dosya var mı, boyut, klasör listeleme işlerini

Bölüm 23'te `cin` ve `cout`'u birer **akış** (stream) olarak tanıdık: veri bir borudan akar, biz de `>>` ve `<<` ile boruya bağlanırız. C++'ta dosya da aynı fikirdir. Dosyaya bağlanan bir akış nesnesi açarız ve ona `cout` ile `cin`'de kullandığımız işleçlerin aynısını uygularız. Yeni bir sözdizimi öğrenmiyoruz; ekranın yerine dosyayı koyuyoruz.

C'de aynı işi `FILE *`, `fopen` ve `fprintf` ile yapmıştık (Bölüm 17). Bu bölümde o bölümün bildiğimiz sorularını yeniden soracağız: dosya açılamazsa ne olur, dosya sonu nasıl anlaşılır, ikili kayıt nasıl yazılır? Farkın büyük kısmı, akış nesnelerinin bir **sınıf** olmasından gelir: açma bir yapıcıda, kapama bir yıkıcıda gerçekleşir ve kapatmayı unutmak çoğu durumda mümkün olmaz.

37.1 fstream Sınıfları ve İlk Program

Dosya akışları `<fstream>` başlığındadır ve üç sınıf olarak gelir:

Sınıf	Yön	Benzediği nesne	Ne için
`std::ofstream` (output file stream)	Programdan dosyaya	`cout`	Dosyaya yazmak
`std::ifstream` (input file stream)	Dosyadan programa	`cin`	Dosyadan okumak
`std::fstream`	İki yönlü	–	Aynı dosyada hem okumak hem yazmak

Nesneyi tanımlarken dosya adını veririz ve dosya o anda açılır:

```
std::ofstream yaz("dosya.txt");    // yazmak için açar
std::ifstream oku("dosya.txt");    // okumak için açar
```

Her dosya işlemi aynı üç adımdan oluşur: aç, kullan, kapat. İlk programımız bir dosya oluşturur, iki satır yazar, sonra aynı dosyayı açıp satır satır okuyarak ekrana basar:

ilkDosya.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::ofstream yaz("selam.txt");
    if (!yaz.is_open()) {
        std::cerr << "selam.txt açılamadı\n";
        return 1;
    }
    yaz << "Merhaba dosya!\n";
    yaz << "Sayı: " << 42 << '\n';
    yaz.close();                        // yazmayı bitir, tamponu boşalt

    std::ifstream oku("selam.txt");
    std::string satir;
    while (std::getline(oku, satir)) {
        std::cout << "Okunan: " << satir << '\n';
    }
    return 0;
}
```

ÇIKTI

```
Okunan: Merhaba dosya!
Okunan: Sayı: 42
```

`yaz << ...` satırları `cout << ...` ile aynıdır; yalnızca nesnenin adı farklıdır. `42` sayısı dosyaya `4` ve `2` karakterleri olarak gitti: metin dosyasına yazarken sayılar yine metne çevrilir (Bölüm 17'deki `fprintf` gibi). Okuma tarafındaki `std::getline(oku, satir)` ise Bölüm 23'te `cin` ile gördüğümüz çağrının aynısıdır; `cin` yerine dosya akışını vermemiz yeterlidir.

Açılıp açılmadığını denetlemek

Dosya açılamayabilir: ad yanlış yazılmıştır, dosya yoktur, klasör yanlıştır ya da izin yoktur. `is_open()` bunu söyler. Olmayan bir dosyayı açmayı deneyelim:

olmayanDosya.cpp

```cpp
#include <iostream>
#include <fstream>

int main() {
    std::ifstream oku("yok.txt");
    if (!oku.is_open()) {
        std::cerr << "yok.txt açılamadı\n";
        return 1;
    }
    std::cout << "Dosya açıldı\n";
    return 0;
}
```

Bu programın ekrana `cout` ile yazdığı bir şey yoktur; `cerr` mesajı (Bölüm 23) ve `1` çıkış kodu görünür:

TERMİNAL

```
$ ./olmayanDosya
yok.txt açılamadı
$ echo $?
1
```

> **▲ Sık Yapılan Hata · Açılmayan akışı denetlemeden kullanmak**
>
> C'de `fopen` başarısız olunca `NULL` döner ve denetlemezsek program çöker. C++'ta durum daha sessizdir: açılamayan bir `ifstream`'den okumaya çalışırsak akış hata durumuna geçer, hiçbir şey okunmaz ve program ilerler. Sonuç "sıfır öğrenci, ortalama 0" türünden sessiz yanlışlardır. Açılamayan bir `ofstream`'e yazarsak da hiçbir şey yazılmaz ve hata mesajı çıkmaz. Dosya adındaki tek bir yazım hatası bu yüzden saatlerce aranabilir. Her açıştan hemen sonra `is_open()` denetleyelim.

Akış nesnesi `if` içinde doğrudan mantıksal değer gibi de kullanılabilir (`if (!oku)`); bu "akış hata durumunda mı?" diye sorar. Açtıktan hemen sonra iki denetim aynı sonucu verir. Aradaki fark sonradan ortaya çıkar: okuma bir kez başarısız olunca `!oku` doğru olur, ama dosya hâlâ açık olduğundan `is_open()` `true` döner. Açma denetiminde okunaklı olduğu için `is_open()` yazacağız.

37.2 Açma Kipleri ve Kapatma

Akış nesnesinin ikinci parametresi **kip**tir (mode) ve dosyayla ne yapılacağını belirler. Kipler `std::ios::` öneğiyle yazılır ve `|` ile birleştirilir:

Kip	Anlamı
`ios::in`	Okumak için aç (`ifstream`'in varsayılanı)
`ios::out`	Yazmak için aç; dosya yoksa oluşturur, **varsa içeriğini siler** (`ofstream`'in varsayılanı)
`ios::app`	Yazılanı hep dosyanın sonuna ekle (append); dosya yoksa oluşturur, içeriği korur
`ios::trunc`	Açılırken içeriği sil
`ios::ate`	Açılınca konumu dosyanın sonuna al (at end)
`ios::binary`	İkili kip; satır sonu çevirisi yapılmaz

`std::ofstream yaz("a.txt");` ile `std::ofstream yaz("a.txt", std::ios::out);` aynıdır. Okuyup yazan `fstream` için kipi kendimiz veririz: `ios::in | ios::out` dosya yoksa açılamaz, varsa içeriği korur; `ios::in | ios::out | ios::trunc` yoksa oluşturur, varsa siler.

C ile eşleştirirsek `ios::out` `"w"`, `ios::app` `"a"`, `ios::in` `"r"` kipine karşılık gelir (Bölüm 17). `ios::in | ios::out` ise `"r+"` gibidir.

> **◆ Kritik Nokta · ofstream var olan dosyanın içeriğini siler**
>
> Var olan bir dosya için `std::ofstream yaz("liste.txt");` yazmak, tek bir karakter yazmadan bile dosyayı boşaltır. Eski içeriği korumak istiyorsak `ios::app` kullanırız. Bu, C'deki `fopen(..., "w")` tuzağının aynısıdır.

close() ve tampon

`close()` dosyayı kapatır. Yazdığımız veri hemen diske gitmez; akış onu bellekte bir **tamponda** (buffer) biriktirir ve toplu olarak yazar, kapatırken tampon boşaltılır. Bu yüzden bir dosyaya yazıp hemen aynı

programda okuyacaksak önce `close()` çağırırız (ya da `flush()` ile tamponu boşaltırız); yoksa okuyan akış henüz diske inmemiş veriyi göremez. Bölüm 23'te `std::endl`'in "satır sonu ve `flush`" yaptığını görmüştük. Dosyaya yazarken her satırda tamponu boşaltmak gereksiz yavaşlık olduğundan `'\n'` yazarız.

37.3 Dosyadan Okumak

Okumanın üç yolu vardır; hepsini `cin` ile zaten tanıyoruz:

Yöntem	Ne okur	Nerede durur
`dosya >> degisken`	Bir değer (sayı, kelime)	Boşlukta (boşluk, sekme, satır sonu)
`std::getline(dosya, satir)`	Bir satırın tamamı	Satır sonunda; satır sonunu okur ama `satir`'a koymaz
`dosya.get(c)`	Tek karakter (boşluk dahil)	Hemen

Operatörle okuma: >>

`>>` baştaki boşlukları atlar, bir değer okur ve onu değişkenin tipine çevirir. Aşağıdaki program önce `notlar.txt` dosyasını kendisi oluşturur (dosyayı elle hazırlamak gerekmez), sonra okuyup ortalamayı bulur:

▸ notOrtalamasi.cpp

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <string>

int main() {
    {
        std::ofstream yaz("notlar.txt");
        yaz << "Ayşe 85\nMehmet 92\nElif 70\nCan 100\n";
    }                                   // blok bitti: dosya kapandı

    std::ifstream oku("notlar.txt");
    if (!oku.is_open()) {
        std::cerr << "notlar.txt açılamadı\n";
        return 1;
    }
    std::string ad;
    int puan{0};
    int toplam{0};
    int adet{0};
    while (oku >> ad >> puan) {
        std::cout << std::left << std::setw(8) << ad << " -> " << puan << '\n';
        toplam += puan;
        adet++;
    }
    if (adet > 0) {
        std::cout << adet << " öğrenci, ortalama " << std::fixed
                  << std::setprecision(2)
                  << static_cast<double>(toplam) / adet << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Ayşe    -> 85
Mehmet   -> 92
Elif     -> 70
Can      -> 100
4 öğrenci, ortalama 86.75
```

Programın özü `while (oku >> ad >> puan)` satırıdır. `>>` işleci okuduktan sonra akışın kendisini geri verir ve akış koşulda mantıksal değer olarak değerlendirilir: son okuma başarılıysa doğru, başarısızsa yanlıştır. Döngü, iki değeri de okuyabildiğimiz sürece döner; dosya bitince ya da veri bozuk olunca okuma başarısız olur ve döngü sona erer. `adet > 0` denetimi boş dosyada sıfıra bölmeyi önler.

`setw(8)` karakter değil bayt saydığı için `Ayşe` satırı bir sütun kısa hizalanır: `ş` UTF-8'de iki bayttır (Bölüm 23). Çıktıda `Ayşe` satırındaki `->` işareti diğerlerinden bir boşluk önce başlar.

Satır satır okuma: getline

`std::getline(dosya, satir)` bir satırı, satır sonuna kadar, boşluklar dahil `satir` içine koyar. Satır sonu karakterini dosyadan alır ama `string`'e eklemez. Aşağıda üçüncü satırı bilerek boş bıraktık:

satirOku.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

int main() {
    {
        std::ofstream yaz("metin.txt");
        yaz << "Dosyalar verileri kalıcı tutar.\n"
            << "Program kapanır, dosya kalır.\n"
            << "\n"
            << "Son satır.\n";
    }

    std::ifstream oku("metin.txt");
    if (!oku.is_open()) {
        std::cerr << "metin.txt açılamadı\n";
        return 1;
    }
    std::string satir;
    int n{0};
    while (std::getline(oku, satir)) {
        n++;
        std::cout << n << ": [" << satir << "]\n";
    }
    return 0;
}
```

```
ÇIKTI
1: [Dosyalar verileri kalıcı tutar.]
2: [Program kapanır, dosya kalır.]
3: []
4: [Son satır.]
```

Boş satır, boş bir `string` olarak okunur (`[]`); atlanmaz. C'deki `fgets` satır uzunluğu için bir tampon boyutu ister ve satır sonunu tampona katar (Bölüm 17). `getline` ise boyut sormaz, çünkü `string` kendiliğinden büyür (Bölüm 27), ve satır sonunu atar.

Kelime kelime ve karakter karakter okuma

`string`'e `>>` ile okumak boşlukta durur; yani dosyayı kelime kelime okur. Satır sonu da boşluk sayıldığından satırların nerede bittiği önemsizdir. Kalıp `getline` döngüsüyle aynıdır, okuma işlemi yine koşulun içindedir:

```
std::string kelime;
int adet{0};
while (oku >> kelime) {
    adet++;                  // her tur bir kelime
}
```

Noktalama kelimeye yapışıksa (`tutar.`) kelimenin parçası sayılır. Tek karakter için `dosya.get(c)` vardır: boşluk ve satır sonu dahil bir `char` okur ve başarılıysa akışı "doğru" bırakır. `char c; while (oku.get(c)) { ... }` dosyayı karakter karakter işler (sayma, kopyalama, dönüştürme).

37.4 Döngü Koşulunda eof Tuzağı

Akışların `eof()` adında bir üyesi vardır ve adı yüzünden yanıltıcıdır. Gerçek kural şudur: `eof()`, dosya sonunu **aşan** bir okuma girişimi başarısız olduktan sonra doğru olur. Son satırı başarıyla okumak onu doğru yapmaz. C'de aynı tuzağı `feof` ile görmüştük (Bölüm 17). Üç satırlık bir dosyada deneyelim:

▸ eofTuzak.cpp

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
    {
        std::ofstream yaz("meyve.txt");
        yaz << "elma\nkiraz\nüzüm\n";
    }

    std::ifstream oku("meyve.txt");
    if (!oku.is_open()) {
        std::cerr << "meyve.txt açılamadı\n";
        return 1;
    }
    std::string satir;
    int n{0};
    while (!oku.eof()) {              // YANLIŞ KALIP
        std::getline(oku, satir);
        n++;
        std::cout << n << ": [" << satir << "]\n";
    }
    return 0;
}
```

```
ÇIKTI
1: [elma]
2: [kiraz]
3: [üzüm]
4: []
```

Üç satırlık dosya dört satır yazdırdı. Üçüncü turda `üzüm` okundu, ama dosya sonu henüz "görülmedi": `eof()` hâlâ yanlış. Döngü bir tur daha döndü, `getline` okuyacak bir şey bulamadı ve `satir`'ı boşalttı. (C'deki `feof`

tuzağında son satır **tekrar** yazılırdı; burada `getline` `string`'i temizlediği için boş bir satır görürüz. Sayılar `>>` ile okunuyorsa değişken eski değerini korur ve son sayı iki kez işlenir.)

> ◆ **Kritik Nokta · Döngü koşulu okumanın kendisi olsun**
>
> `while (!dosya.eof())` ile döngü kurmayalım. Doğru kalıp okuma işlemini koşula yazmaktır: `while (std::getline(dosya, satir))`, `while (dosya >> x)`, `while (dosya.get(c))`. Okuma başarısızsa gövdeye girilmez; "okumadan işlem yapma" olasılığı kalmaz.

37.5 Akış Durumu: fail ve clear

Akış hata aldığında **durum bayrakları** tutar. `eof()` okunacak bir şey kalmadan dosya sonuna gelindiğini, `fail()` okumanın biçime uymadığı için (ya da dosya bittiği için) başarısız olduğunu, `good()` her şeyin yolunda olduğunu, `bad()` ciddi bir giriş/çıkış hatasını bildirir (çok nadirdir). Bölüm 23'te klavye girişinde gördüğümüz kilitlenme burada da aynıdır: akış bozulunca `clear()` ile bayrakları temizleyene kadar sonraki okumalar da başarısız olur.

Aşağıdaki dosyada `3 9 x 4` yazıyor. Döngü sayıları toplarken `x`'te durur:

▸ akisDurumu.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

int main() {
    {
        std::ofstream yaz("veri.txt");
        yaz << "3 9 x 4\n";
    }

    std::ifstream oku("veri.txt");
    if (!oku.is_open()) {
        std::cerr << "veri.txt açılamadı\n";
        return 1;
    }
    int sayi{0};
    int toplam{0};
    while (oku >> sayi) {
        toplam += sayi;
    }
    std::cout << "Toplam: " << toplam << '\n';
    std::cout << "fail: " << oku.fail() << ", eof: " << oku.eof() << '\n';

    oku.clear();                        // bayrakları temizle
    std::string kelime;
    oku >> kelime;                      // x hâlâ dosyada bekliyor
    std::cout << "Temizledikten sonra: " << kelime << '\n';
    oku >> sayi;
    std::cout << "Sonraki sayı: " << sayi << '\n';
    return 0;
}
```

```
ÇIKTI
Toplam: 12
fail: 1, eof: 0
Temizledikten sonra: x
Sonraki sayı: 4
```

`x` sayıya çevrilemediği için döngü durdu: `fail()` doğru (1), `eof()` yanlış (0), çünkü dosyanın sonuna gelinmedi. `clear()` bayrakları sildi; `x` okunamamıştı, dosyada yerinde bekliyordu, bu yüzden `string` olarak okunabildi ve ardından `4` de okundu.

Bir dosyayı iki kez okumak gerekirse (önce satırları saymak, sonra işlemek gibi) ilk okuma dosya sonuyla bittiği için başa sarmadan önce bayrakları da temizlemeliyiz: `oku.clear(); oku.seekg(0);` (sıra önemlidir; `seekg`'yi ikili dosyalarda göreceğiz). Çoğu zaman daha kolay yol, dosyayı bir kez okuyup bir `vector`'e koymak ve bellekte işlemektir.

> **▲ Sık Yapılan Hata · >> sonrası getline**
>
> Bölüm 23'te `cin` için gördüğümüz tuzak dosyada da aynen geçerlidir: `>>` değeri okur ama satır sonu karakterini bırakır. Hemen ardından `getline` çağırırsak kalan satır sonunu okur ve boş bir satır verir. Çare aynıdır: arada `dosya.ignore(std::numeric_limits<std::streamsize>::max(), '\n');` (`<limits>`) ile satırın kalanını atarız.

37.6 Dosyaya Ekleme: ios::app

`ios::app` kipi yazılanları dosyanın sonuna ekler, içeriği silmez, dosya yoksa oluşturur. Günlük (log) dosyaları böyle tutulur. Programımız önce `gunluk.txt`'ye tek satır yazar (varsayılan kip içeriği siler), sonra dosyayı `ios::app` ile açıp iki satır ekler ve tamamını gösterir. Program kaç kez çalışırsa çalışsın aynı sonucu verir:

▸ eklemeKipi.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::ofstream ilk("gunluk.txt");   // varsayılan kip: sıfırdan yazar
    if (!ilk.is_open()) {
        std::cerr << "gunluk.txt açılamadı\n";
        return 1;
    }
    ilk << "09:00 Sistem başladı\n";
    ilk.close();

    std::ofstream ekle("gunluk.txt", std::ios::app);   // sona ekle
    if (!ekle.is_open()) {
        std::cerr << "gunluk.txt açılamadı\n";
        return 1;
    }
    ekle << "10:00 Yedekleme yapıldı\n";
    ekle << "10:15 Kullanıcı girişi\n";
    ekle.close();

    std::ifstream oku("gunluk.txt");
    std::string satir;
    while (std::getline(oku, satir)) {
        std::cout << satir << '\n';
```

```
    }
    return 0;
}
```

```
ÇIKTI
09:00 Sistem başladı
10:00 Yedekleme yapıldı
10:15 Kullanıcı girişi
```

Düz `ofstream` kullansaydık ikinci açış ilk satırı silerdi. `ios::app` ile açılmış dosyada `seekp` ile başa gitsek bile her yazma yine sona yapılır; standart böyle der.

37.7 CSV Dosyaları

CSV (comma-separated values), her satırı bir kayıt, alanları virgülle ayrılmış bir metin dosyasıdır; elektronik tabloların "CSV olarak kaydet" çıktısı budur. Yöntem iki adımdır: satırı `getline` ile alırız, sonra satırı alanlara ayırırız. Ayırmak için satırı bir `stringstream`'e (Bölüm 27) koyup ondan bir kez daha `getline` yaparız. `std::getline(ss, alan, ',')` "bir sonraki virgüle kadar oku" demektir; üçüncü parametre, satır sonu yerine kullanılacak **ayraçtır**. Alanlar metin olarak gelir; sayıya çevirmek için `stoi` ve `stod` kullanılır. Yazı sayıya benzemiyorsa bu işlevler `invalid_argument` istisnası fırlatır (Bölüm 34); bozuk satırı yakalamak için tam bunu kullanırız.

Aşağıdaki dosyada ilk satır başlıktır, dördüncü veri satırı da bilerek bozuktur. Program dosyayı okur, kayıtları bir `struct` (Bölüm 14) topluluğundan oluşan `vector`'e (Bölüm 26) doldurur, bozuk satırı bildirir, `rapor.txt` dosyasını yazar ve onu da ekrana gösterir:

▸ csvOku.cpp

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <iomanip>
#include <stdexcept>

struct Ogrenci {
    std::string ad;
    int no;
    double puan;
};

int main() {
    {
        std::ofstream yaz("ogrenciler.csv");
        yaz << "ad,no,puan\n"
            << "Ayşe Demir,101,85.5\n"
            << "Mehmet Kaya,102,92\n"
            << "Elif Şen,103,70.25\n"
            << "Can Er,abc,60\n"
            << "Zeynep Ak,104,100\n";
    }

    std::ifstream oku("ogrenciler.csv");
    if (!oku.is_open()) {
        std::cerr << "ogrenciler.csv açılamadı\n";
        return 1;
```

```
    }
    std::vector<Ogrenci> liste;
    std::string satir;
    std::getline(oku, satir);          // başlık satırını atla
    while (std::getline(oku, satir)) {
        std::stringstream ss{satir};
        std::string ad, noYazi, puanYazi;
        std::getline(ss, ad, ',');
        std::getline(ss, noYazi, ',');
        std::getline(ss, puanYazi, ',');
        try {
            liste.push_back({ad, std::stoi(noYazi), std::stod(puanYazi)});
        } catch (const std::invalid_argument&) {
            std::cout << "Hatalı satır atlandı: " << satir << '\n';
        }
    }

    std::ofstream yaz("rapor.txt");
    if (!yaz.is_open()) {
        std::cerr << "rapor.txt açılamadı\n";
        return 1;
    }
    double toplam{0};
    yaz << std::fixed << std::setprecision(2);
    for (const Ogrenci& o : liste) {
        yaz << o.no << ' ' << o.ad << ' ' << o.puan << '\n';
        toplam += o.puan;
    }
    if (!liste.empty()) {
        yaz << "Ortalama: " << toplam / liste.size() << '\n';
    }
    yaz.close();

    std::ifstream rapor("rapor.txt");  // raporu göster
    while (std::getline(rapor, satir)) {
        std::cout << satir << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Hatalı satır atlandı: Can Er,abc,60
101 Ayşe Demir 85.50
102 Mehmet Kaya 92.00
103 Elif Şen 70.25
104 Zeynep Ak 100.00
Ortalama: 86.94
```

`liste.push_back({ad, std::stoi(noYazi), std::stod(puanYazi)})` içindeki dönüşümlerden biri istisna fırlatırsa `push_back` hiç çağrılmaz; yarım kayıt oluşmaz. Programda üç noktaya dikkat edelim:

- `stoi("12abc")` hata vermez, baştaki sayı kısmını alıp `12` döndürür. Daha katı bir denetim gerekiyorsa dönüşümü `stringstream` ile `>>` kullanarak yapıp akışın sonuna gelinip gelinmediğine bakarız.
- Dosya Windows'ta `\r\n` satır sonuyla kaydedilip Linux ya da macOS'ta okunursa her satırın sonunda bir `\r` kalır ve son alana karışır. Önlem: `getline`'dan sonra `if (!satir.empty() && satir.back() == '\r') satir.pop_back();`.
- Alanın içinde virgül varsa ve tırnakla korunmuşsa (`"Demir, Ayşe",101`) bu basit yöntem şaşar. Gerçek CSV için ayrı bir ayrıştırıcı gerekir.

İpucu · Türkçe ayarlı elektronik tablolar noktalı virgül kullanır

Ondalık ayırıcının virgül olduğu yerel ayarlarda tablo programları alanları `;` ile ayırır (`Ayşe;101;85,5`). Böyle bir dosyada `std::getline(ss, alan, ';')` yazarız ve `stod`'dan önce ondalık virgülü noktaya çevririz: `std::replace(alan.begin(), alan.end(), ',', '.');` (`<algorithm>`).

37.8 RAII: Dosya Kendiliğinden Kapanır

Bölüm 29'da bir nesnenin kapsamından (`{ }` bloğundan) çıkarken yıkıcısının otomatik çalıştığını gördük. Dosya akışı sınıflarının yıkıcısı da tamponu boşaltır ve dosyayı kapatır. Bu yüzden çoğu zaman `close()` yazmamız gerekmez: akış nesnesini dosyaya ihtiyaç duyulan en küçük kapsamda tanımlarız, kapsam bitince dosya kapanır. `notOrtalamasi.cpp` programındaki ilk blok bu yüzden `close()` içermiyordu:

```
{
    std::ofstream yaz("kayit.txt");
    yaz << "veri\n";
}                                   // kapsam bitti: yaz yok olur, dosya kapanır
```

Tanım: RAII

Resource Acquisition Is Initialization ("kaynağı edinmek, nesneyi oluşturmaktır"): kaynağı (dosya, bellek, kilit) bir nesnenin yapıcısında ediniriz, yıkıcısında bırakırız. Bırakmayı unutma olasılığı kalmaz.

`unique_ptr` (Bölüm 29) aynı fikrin bellek için olanıdır; `fstream` ise dosya içindir. C'deki `fopen`/`fclose` çiftinde kapatmayı biz hatırlamak zorundaydık. Bölüm 29'da `unique_ptr<FILE, decltype(&fclose)>` ile `FILE *`'ı RAII'ye bağlamayı ima etmiştik; `fstream` aynı işi hiçbir ek yapı kurmadan yapar.

RAII'nin asıl değeri hata durumunda ortaya çıkar. Bir işlev istisna fırlattığında yığın çözülürken (stack unwinding; Bölüm 34) kapsamdaki nesnelerin yıkıcıları çalışır. Dosya akışı da kapanır ve tamponundaki veri diske yazılır; `close()` satırına hiç ulaşılamamış olsa bile:

`istisnaKapanma.cpp`

```
#include <iostream>
#include <fstream>
#include <string>
#include <stdexcept>

void yazVeFirlat() {
    std::ofstream yaz("istisna.txt");
    yaz << "Bu satır diske ulaşır\n";
    throw std::runtime_error("yazarken sorun çıktı");
}                                   // yığın çözülürken yaz kapanır

int main() {
    try {
        yazVeFirlat();
    } catch (const std::runtime_error& e) {
        std::cout << "Yakalandı: " << e.what() << '\n';
    }
    std::ifstream oku("istisna.txt");
    std::string satir;
    std::getline(oku, satir);
```

```
    std::cout << "Dosyada: " << satir << '\n';
    return 0;
}
```

```
ÇIKTI
Yakalandı: yazarken sorun çıktı
Dosyada: Bu satır diske ulaşır
```

C'de benzer bir durumda `fclose` çağrısı atlanır, dosya açık kalır ve tampondaki veri kaybolabilirdi. C++'ta bunun için `finally` benzeri bir yapıya gerek yoktur.

Akışlar işlevlere referansla geçirilir

Akış nesneleri kopyalanamaz; bir dosyanın iki "sahibi" olmaz. Bir işleve akış vermek istersek **referansla** veririz (Bölüm 28). Parametre tipi olarak somut `ofstream` yerine `std::ostream&` yazarsak işlev ekrana da, dosyaya da, `stringstream`'e de yazabilir. Bu, Bölüm 32'de `operator<<` aşırı yüklerken kullandığımız fikirdir:

ortakYazim.cpp

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>

void listeyiYaz(std::ostream& cikis, const std::vector<int>& v) {
    for (int x : v) {
        cikis << x << ' ';
    }
    cikis << '\n';
}

int main() {
    std::vector<int> sayilar{4, 8, 15};

    listeyiYaz(std::cout, sayilar);              // ekrana

    std::ofstream dosya("liste.txt");
    listeyiYaz(dosya, sayilar);                  // dosyaya
    dosya.close();

    std::ostringstream metin;
    listeyiYaz(metin, sayilar);                  // bellekteki metne
    std::cout << "Metin uzunluğu: " << metin.str().size() << '\n';
    return 0;
}
```

```
ÇIKTI
4 8 15
Metin uzunluğu: 8
```

Okuma tarafında karşılığı `std::istream&` parametresidir. Aynı işlev, `std::cin`'den de dosyadan da okuyabilir.

37.9 Konum ve Atlama: seekg, seekp, tellg

Metin dosyasını başından sonuna doğru sırayla okumak çoğu zaman yeter. Bazen ortadaki bir yere doğrudan gitmek isteriz. Her akışın bir **konum işaretçisi** vardır: `tellg()` okuma konumunu, `tellp()` yazma konumunu bayt cinsinden verir; `seekg(konum)` ve `seekp(konum)` onları taşır. İkinci bir parametre başlangıç noktasını seçer: `ios::beg` (baş, varsayılan), `ios::cur` (şimdiki yer), `ios::end` (son). Bu işlevler C'deki `ftell` ve `fseek` karşılığıdır (Bölüm 17); farkları, okuma ve yazma konumunun ayrı adlandırılmış olmasıdır.

```
oku.seekg(0, std::ios::end);        // sona git
std::streamoff boyut = oku.tellg(); // son konum = dosya boyutu (bayt)
oku.seekg(0);                       // başa dön
```

Bir dosyanın boyutunu böyle bulabiliriz; `<filesystem>` ile daha kolay bir yolu aşağıda göreceğiz.

37.10 İkili Dosyalar: write ve read

Metin dosyasında `12345` sayısı beş karakter olarak yazılır ve okunurken yeniden sayıya çevrilir. **İkili** (binary) dosyada ise bellekteki baytlar olduğu gibi diske kopyalanır. Akış üyeleri `write` ve `read`'dir; C'deki `fwrite` ve `fread`'in karşılığıdır (Bölüm 17). İki fark vardır: dosyayı `ios::binary` kipiyle açarız ve bellek adresi `char *` tipinde verilmelidir, bunun için `reinterpret_cast` kullanırız.

```
yaz.write(reinterpret_cast<const char*>(&nesne), sizeof nesne);
oku.read(reinterpret_cast<char*>(&nesne), sizeof nesne);
```

Sabit boyutlu bir kayıt yapısı tanımlayıp üç kaydı yazalım ve okuyalım. Yapıda `std::string` değil sabit boyutlu `char` dizisi kullanıyoruz, nedeni aşağıdaki kutuda:

▸ ikiliKayit.cpp

```
#include <iostream>
#include <fstream>
#include <iomanip>

struct Kayit {
    char ad[15];
    int no;
    double puan;
};

int main() {
    Kayit liste[3] = {{"Ayse", 101, 85.5}, {"Mehmet", 102, 92.0},
                      {"Elif", 103, 70.25}};

    std::ofstream yaz("kayitlar.bin", std::ios::binary);
    if (!yaz.is_open()) {
        std::cerr << "kayitlar.bin açılamadı\n";
        return 1;
    }
    yaz.write(reinterpret_cast<const char*>(liste), sizeof liste);
    yaz.close();

    std::ifstream oku("kayitlar.bin", std::ios::binary);
    if (!oku.is_open()) {
        std::cerr << "kayitlar.bin açılamadı\n";
        return 1;
    }
```

```
    oku.seekg(0, std::ios::end);
    std::cout << "Dosya boyutu: " << oku.tellg() << " bayt\n";
    oku.seekg(0);

    Kayit k{};
    while (oku.read(reinterpret_cast<char*>(&k), sizeof k)) {
        std::cout << k.no << ' ' << std::left << std::setw(7) << k.ad
                  << k.puan << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Dosya boyutu: 96 bayt
101 Ayse   85.5
102 Mehmet 92
103 Elif   70.25
```

`while (oku.read(...))` kalıbı da okumanın kendisini koşul yapar: `read` istenen bayt sayısını okuyamazsa akış hata durumuna geçer. Dosya boyutu, kayıt sayısı ile `sizeof(Kayit)` çarpımıdır. `Kayit` için alanların toplamı 15 + 4 + 8 = 27 bayttır, ama `sizeof` 32 çıkar (3 × 32 = 96): derleyici alanları hizalamak için yapının içine **dolgu baytları** (padding) koyar (Bölüm 14). Bu değerler 64 bitlik bir g++ içindir; başka bir platformda farklı çıkabilir. Dolgu baytları da dosyaya yazılır ve içerikleri belirsizdir.

> **Not · İkili dosya taşınabilir değildir**
>
> İkili dosya baytları aynen yazdığı için bir bilgisayarda yazılıp başka birinde okunabileceğinin garantisi yoktur: `long` Windows'ta 4, Linux ve macOS'ta 8 bayttır; yapı hizalaması derleyiciye göre değişir; bazı işlemciler çok baytlı sayıları ters sırada saklar (Bölüm 5). Dosyayı yalnızca aynı programla okuyacaksak ikili biçim uygundur; başkalarıyla paylaşacaksak metin (CSV) güvenlidir. Ayrıca yapının içinde `std::string` ya da `vector` bulunuyorsa `write` metni değil, nesnenin içindeki işaretçiyi yazar; bu adres dosya okunduğunda geçersizdir (Bölüm 29). Bu yüzden kaydedilecek metinler `char ad[15]` gibi sabit boyutlu dizilerde tutulur.

Kayıt numarasıyla okuma ve güncelleme

Kayıtlar sabit boyutlu olduğundan `n` numaralı kaydın konumu `n * sizeof(Kayit)`'tir. Bu, ikili dosyanın asıl kazancıdır: dosyayı baştan sona taramadan istediğimiz kayda atlarız. Aşağıda 1 numaralı (0'dan sayılır) kaydı okuyup puanını 10 artırır ve yerine geri yazarız. Dosyayı `fstream` ile `ios::in | ios::out | ios::binary` kipinde açıyoruz: hem okur hem yazar, ama içeriği silmez (`ios::trunc` silerdi).

▸ kayitGuncelle.cpp

```
#include <iostream>
#include <fstream>
#include <iomanip>

struct Kayit {
    char ad[15];
    int no;
    double puan;
};

int main() {
    Kayit liste[3] = {{"Ayse", 101, 85.5}, {"Mehmet", 102, 92.0},
                      {"Elif", 103, 70.25}};
    {
        std::ofstream yaz("guncel.bin", std::ios::binary);
```

```
        yaz.write(reinterpret_cast<const char*>(liste), sizeof liste);
    }

    std::fstream dosya("guncel.bin",
                       std::ios::in | std::ios::out | std::ios::binary);
    if (!dosya.is_open()) {
        std::cerr << "guncel.bin açılamadı\n";
        return 1;
    }
    Kayit k{};
    const std::streamoff konum = static_cast<std::streamoff>(1 * sizeof(Kayit));
    dosya.seekg(konum);                          // 1 numaralı kayda git
    if (dosya.read(reinterpret_cast<char*>(&k), sizeof k)) {
        std::cout << "Okunan: " << k.ad << ' ' << k.puan << '\n';
        k.puan += 10;
        dosya.seekp(konum);                      // aynı kaydın başına dön
        dosya.write(reinterpret_cast<const char*>(&k), sizeof k);
    }

    dosya.seekg(0);
    while (dosya.read(reinterpret_cast<char*>(&k), sizeof k)) {
        std::cout << k.no << ' ' << std::left << std::setw(7) << k.ad
                  << k.puan << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Okunan: Mehmet 92
101 Ayse   85.5
102 Mehmet 102
103 Elif   70.25
```

`read` konumu bir kayıt ilerlettiği için yazmadan önce `seekp(konum)` ile geri dönmek şarttır; yoksa güncellenmiş kayıt bir sonrakinin üstüne yazılırdı. Bir kural daha: aynı `fstream` ile okumadan yazmaya (ya da tersine) geçerken araya bir `seekg` / `seekp` (ya da `flush`) koymalıyız; yukarıdaki iki konumlandırma bunu da karşılar.

Metin kipi ve ikili kip

Windows satır sonunu iki baytla (`\r\n`), Linux ve macOS tek baytla (`\n`) yazar. Metin kipinde (`ios::binary` yok) C++ bu farkı gizler: Windows'ta `'\n'` yazarsak diske `\r\n` gider, okurken `\r\n` yine `'\n'` olarak gelir. İkili kipte çeviri yapılmaz. Kural: metin dosyasına metin kipi; ikili veri, bayt sayma ve konumla gezinme için ikili kip. Resim, PDF gibi dosyaları `ios::binary` olmadan kopyalarsak Windows'ta bozarız (Bölüm 17'deki `"rb"` / `"wb"` kuralının aynısı).

Türkçe karakterler (`ş`, `ç`, `ğ`, `ü`, `ö`, `ı`) UTF-8'de iki bayttır ve dosyada da iki bayt yer kaplar. `<<`, `>>` ve `getline` baytları yorumlamadan taşıdığı için Türkçe metni sorunsuz kopyalar; sorun sayarken (`size()` bayt verir) ve `toupper` gibi ASCII'ye özgü işlevlerde çıkar (Bölüm 27). Bazı Windows editörleri dosyayı "UTF-8 with BOM" kaydeder ve başa üç görünmez bayt (`EF BB BF`) ekler; okunan ilk kelime bu baytlarla başlar. Dosyaları BOM'suz UTF-8 kaydetmek en güvenlisidir.

37.11 Dosya Yolu ve Çalışma Klasörü

`std::ifstream oku("notlar.txt");` gibi yalnızca ad yazarsak bu bir **göreli yol**dur (relative path): dosya, programın o anda çalıştığı klasörde aranır. Bu klasör `.cpp` ya da çalıştırılabilir dosyanın bulunduğu yer değil, **terminalin bulunduğu klasördür**. Program `ders/` klasörü içinde duruyor ama terminal bir üst klasörde açıksa dosya bir üst klasörde aranır:

```
TERMİNAL
proje$ g++ -std=c++17 ders/dosya.cpp -o dosya
proje$ ./dosya          # notlar.txt, proje/ içinde aranır; ders/ içinde değil
```

Çözüm, terminalde `ls` (Windows `cmd`'de `dir`) ile dosyanın yerini görüp ya terminali o klasöre taşımak (`cd ders`) ya da dosyayı programın çalıştığı klasöre koymaktır. Yazan programlar için de aynıdır: `ofstream` dosyayı çalışma klasöründe oluşturur; "dosyam nereye gitti?" sorusunun yanıtı çoğu zaman terminalin klasörüdür. Çalışma klasörünü `std::filesystem::current_path()` ile yazdırabiliriz (sonraki başlıkta).

Tam yol (absolute path) de yazabiliriz. Windows'ta ters bölü kaçış dizisinin başlangıcı olduğundan çift yazılır (Bölüm 6); eğik çizgi her sistemde kabul edilir. C++'a özgü üçüncü yol **ham string** (raw string) sabitidir: `R"(...)"` içinde ters bölüler olduğu gibi kalır. Üç yazılış da aynı yolu verir:

```
std::ifstream a("C:\\ders\\notlar.txt");     // her ters bölü çift
std::ifstream b("C:/ders/notlar.txt");       // eğik çizgi
std::ifstream c(R"(C:\ders\notlar.txt)");    // ham string
```

> **▲ Sık Yapılan Hata · Yolda tek ters bölü**
>
> `std::ifstream oku("C:\notlar.txt");` yazarsak `\n` yeni satır karakteri olur ve aranan ad bozulur; `"C:\test"` içindeki `\t` de sekmedir. Dosya bulunamaz. Her ters bölüyü `\\` yazmalı ya da ham string kullanmalıyız. Ayrıca Windows dosya uzantılarını gizliyorsa `notlar.txt` aslında `notlar.txt.txt` olabilir.

Standart akışların yönlendirilmesi (`>`, `<`, `2>`) C'deki gibi çalışır; ayrıntıları Bölüm 17'de verdik. `cerr`'in `cout`'tan ayrı bir kanal olması burada işe yarar: `./program > cikti.txt` yalnızca `cout`'u dosyaya yollar, hata iletileri ekranda kalır.

37.12 filesystem Kitaplığı (C++17)

Dosya içeriğini akışlar yönetir; dosyanın kendisiyle ilgili sorular (var mı, boyutu ne, klasörde neler var, sil, yeniden adlandır) için C++17 `<filesystem>` başlığını getirdi. C'de bunlar standart değildi: `stat`, `opendir` gibi işletim sistemine özgü işlevler gerekirdi. Ad alanı uzundur, bu yüzden bir takma ad (alias) tanımlarız: `namespace fs = std::filesystem;`.

İşlev	Ne yapar
`fs::exists(yol)`	Dosya ya da klasör var mı
`fs::is_directory(yol)`, `fs::is_regular_file(yol)`	Klasör mü, düz dosya mı
`fs::file_size(yol)`	Dosya boyutu (bayt)
`fs::create_directories(yol)`	Klasörü (ve eksik üst klasörleri) oluşturur
`fs::remove(yol)`, `fs::remove_all(yol)`	Siler; ikincisi klasörü içindekilerle birlikte siler, silinen sayısını döndürür

İşlev	Ne yapar
`fs::rename(eski, yeni)`, `fs::copy_file(kaynak, hedef)`	Yeniden adlandırır, kopyalar
`fs::current_path()`	Çalışma klasörü
`fs::directory_iterator(klasor)`	Klasördeki girdileri gezer

`fs::path` sınıfı yolları temsil eder; `/` işleci yol parçalarını işletim sisteminin ayırıcısıyla birleştirir ve `filename()`, `stem()`, `extension()` üyeleri parçalarına ayırır. Bir deneme klasörü kurup inceleyelim:

▸ klasorGezme.cpp

```cpp
#include <algorithm>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

namespace fs = std::filesystem;

int main() {
    const fs::path klasor = "deneme_klasor";
    fs::remove_all(klasor);                      // önceki çalışmadan kalanı sil
    fs::create_directories(klasor / "alt");
    {
        std::ofstream a(klasor / "a.txt");
        a << "merhaba\n";
        std::ofstream b(klasor / "b.csv");
        b << "dosya icerigi\n";
    }

    std::cout << std::boolalpha;
    std::cout << "Var mı: " << fs::exists(klasor / "a.txt") << '\n';
    std::cout << "Klasör mü: " << fs::is_directory(klasor / "alt") << '\n';
    std::cout << "a.txt boyutu: " << fs::file_size(klasor / "a.txt") << '\n';
    std::cout << "b.csv boyutu: " << fs::file_size(klasor / "b.csv") << '\n';

    std::vector<std::string> adlar;
    for (const auto& girdi : fs::directory_iterator(klasor)) {
        std::string ad = girdi.path().filename().string();
        if (girdi.is_directory()) {
            ad += '/';
        }
        adlar.push_back(ad);
    }
    std::sort(adlar.begin(), adlar.end());       // gezme sırası belirsizdir
    for (const std::string& ad : adlar) {
        std::cout << "  " << ad << '\n';
    }

    const fs::path p = klasor / "b.csv";
    std::cout << "Ad: " << p.stem().string() << ", uzantı: "
              << p.extension().string() << '\n';
    std::cout << "Silinen: " << fs::remove_all(klasor) << '\n';
    return 0;
}
```

```
ÇIKTI
Var mı: true
Klasör mü: true
a.txt boyutu: 8
b.csv boyutu: 14
  a.txt
  alt/
  b.csv
Ad: b, uzantı: .csv
Silinen: 4
```

`directory_iterator` girdileri hangi sırayla verdiğini garanti etmez; çıktıyı kararlı kılmak için adları bir `vector`'e alıp `sort` ile sıraladık (Bölüm 36). `remove_all` dört şeyi sildi: klasör, `alt` klasörü ve iki dosya. Dosya sistemi işlevleri başarısız olduğunda varsayılan olarak `fs::filesystem_error` istisnası fırlatır (Bölüm 34); istisna istemiyorsak her işlevin `std::error_code` alan bir aşırı yüklenmiş sürümü vardır.

Üç Dilde · Üç Dilde: dosyadan satır satır okumak *C · C++ · C#*

Aynı işi, dosyayı açıp satırları ekrana yazmayı üç dilde yan yana koyalım.

C (Bölüm 17): açma, kapama ve hata denetimi elle yapılır.

```c
FILE *f = fopen("notlar.txt", "r");
if (f == NULL) { perror("notlar.txt"); return 1; }
char satir[256];
while (fgets(satir, sizeof satir, f) != NULL) {
    printf("%s", satir);          // satır sonu satir[] içinde kalır
}
fclose(f);                        // unutulursa dosya açık kalır
```

C++: akış bir nesnedir; kapanış yıkıcıdadır, satır uzunluğu sınırı yoktur.

```cpp
std::ifstream f("notlar.txt");
if (!f.is_open()) { std::cerr << "açılamadı\n"; return 1; }
std::string satir;
while (std::getline(f, satir)) {
    std::cout << satir << '\n';
}                                 // f kapsam sonunda kapanır
```

C# (ileride Bölüm 41'den itibaren göreceğiz): `System.IO` ad alanındaki `File` sınıfı en sık işleri tek çağrıyla yapar; denetim gerektiğinde `StreamReader` ve `using` kullanılır. Hata durumunda `NULL` ya da bayrak yerine `IOException` fırlatılır.

```csharp
foreach (string satir in File.ReadLines("notlar.txt"))
    Console.WriteLine(satir);

using (var okuyucu = new StreamReader("notlar.txt"))   // using: otomatik kapanır
{
    string? satir;
    while ((satir = okuyucu.ReadLine()) != null)
        Console.WriteLine(satir);
}
```

	C	C++	C#
Dosya tutamacı	`FILE *`	`ifstream` / `ofstream` nesnesi	`StreamReader` / `StreamWriter`, `File`

	C	C++	C#
Kapatma	`fclose` (elle)	yıkıcı (RAII)	`using` / `Dispose`
Açılamama	`NULL` döner	akış hata durumuna geçer	`FileNotFoundException` vb.
Tüm dosyayı tek çağrıyla okuma	yok	yok (döngü gerekir)	`File.ReadAllText`, `ReadAllLines`

Bölüm Özeti

- Dosya kalıcıdır; metin dosyası okunabilir karakterlerden, ikili dosya bellek baytlarından oluşur.
- `<fstream>`: `ofstream` yazar, `ifstream` okur, `fstream` ikisini yapar. Dosya akışları `cin` ve `cout` ile aynı `<<` ve `>>` işleçlerini kullanır.
- Her açıştan sonra `is_open()` denetlenir; açılmayan akış sessizce başarısız olur.
- Kipler: `ofstream`'in varsayılanı var olan dosyayı siler, `ios::app` sona ekler, `ios::in | ios::out` hem okur hem yazar, `ios::binary` ikili kiptir.
- Okuma biçimleri: `dosya >> x` boşlukta durur, `getline(dosya, satir)` bir satır alır ve satır sonunu `string`'e koymaz, `dosya.get(c)` tek karakter okur.
- Döngü koşulu okumanın kendisidir: `while (getline(dosya, s))`, `while (dosya >> x)`. `while (!dosya.eof())` yanlıştır; son kayıt iki kez işlenir ya da sonda boş kayıt görülür.
- Akış hata alınca `fail()` doğru olur ve `clear()` çağrılana kadar okumalar çalışmaz. Dosyayı yeniden okumadan önce `clear()`, sonra `seekg(0)` çağrılır.
- `>>`'dan sonra `getline` boş satır okuyabilir; arada `ignore` kullanılır.
- CSV için satır `getline` ile alınır, `stringstream` ve `getline(ss, alan, ',')` ile alanlara bölünür, `stoi` / `stod` ile çevrilir, bozuk satır `invalid_argument` ile yakalanır.
- Dosya akışı kapsamdan çıkınca, istisna olsa bile, yıkıcısıyla kapanır (RAII). Bir dosyaya yazıp aynı programda okuyacaksak önce `close()` çağırırız.
- Akışlar kopyalanamaz; işlevlere `std::istream&` ve `std::ostream&` ile verilir, böylece işlev ekranla dosyayı ayırt etmez.
- Göreli yol terminalin bulunduğu klasörden aranır; Windows yollarında `\\` ya da ham string kullanılır.
- `seekg` / `tellg` okuma, `seekp` / `tellp` yazma konumudur. İkili dosyada `write` / `read` ve `reinterpret_cast` kullanılır; `n` numaralı kaydın konumu `n * sizeof(Kayit)`'tir; ikili dosya taşınabilir değildir.
- `<filesystem>` (C++17) dosyanın kendisiyle ilgili işleri yapar: `exists`, `file_size`, `create_directories`, `remove_all`, `directory_iterator`.

Alıştırmalar

1. `defter.txt` dosyası var ve 5 satır içeriyor. Programda yalnızca `std::ofstream f("defter.txt");` satırı çalışıyor, hiçbir şey yazılmıyor ve program bitiyor. Dosyanın durumu nedir?

A) 5 satırın tamamı korunur.
B) Dosya var olduğu için açılamaz, `is_open()` yanlış döner.
C) Dosya boşalır (0 bayt olur).
D) Dosya diskten silinir.
E) Yalnızca ilk satır korunur.

2. `sayilar.txt` dosyasında `1 2 3` yazıyor ve dosya satır sonuyla bitiyor. `int x{0}; int toplam{0};` tanımlıyken aşağıdaki döngülerden hangisi `toplam`'ı doğru (6) bulur?

A) `while (!dosya.eof()) { dosya >> x; toplam += x; }`
B) `while (dosya.good()) { dosya >> x; toplam += x; }`
C) `do { dosya >> x; toplam += x; } while (dosya);`
D) `for (int i = 0; i < 100; i++) { dosya >> x; toplam += x; }`
E) `while (dosya >> x) { toplam += x; }`

3. `gunluk.txt` içinde yalnızca `A` satırı var. Program dosyayı `std::ofstream f("gunluk.txt", std::ios::app);` ile açıyor, sırayla `f.seekp(0);` ve `f << "B\n";` çalıştırıp kapatıyor. Dosyanın içeriği (satır satır) ne olur?

A) `B`, `A`
B) `A`, `B`
C) Yalnızca `B`
D) Dosya boş kalır.
E) `seekp` yüzünden kod derlenmez.

4. Her kaydı 20 bayt olan ikili bir dosya `std::ifstream oku("k.bin", std::ios::binary);` ile açılmış. 0'dan başlayarak numaralanan 3 numaralı kaydın başına gitmek için hangi çağrı yazılır?

A) `oku.seekg(3, std::ios::beg);`
B) `oku.seekg(3, std::ios::cur);`
C) `oku.seekg(20, std::ios::beg);`
D) `oku.seekg(3 * 20, std::ios::beg);`
E) `oku.seekg(-3 * 20, std::ios::end);`

5. `kisi.txt` dosyasının ilk satırı `7`, ikinci satırı `Ali Veli`. Program `int n{0}; std::string ad;` tanımlayıp sırayla `dosya >> n;` ve `std::getline(dosya, ad);` çalıştırıyor. `ad` içinde ne vardır?

A) Boş string (`""`)
B) `Ali Veli`
C) `7`
D) `Ali`
E) Okuma başarısız olur ve program çöker.

6. Aşağıdaki program için `soru6.txt` dosyasında tek satır `5 10 20` vardır ve satır sonuyla biter. Program ekrana ne yazdırır?

```
#include <iostream>
#include <fstream>

int main() {
    std::ifstream f("soru6.txt");
    int x{0};
    int toplam{0};
    while (!f.eof()) {
```

```
        f >> x;
        toplam += x;
        std::cout << x << ' ';
    }
    std::cout << "\ntoplam=" << toplam << '\n';
    return 0;
}
```

7. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <fstream>

int main() {
    std::fstream f("harf.txt",
                   std::ios::in | std::ios::out | std::ios::trunc);
    f << "ABCDEFGHIJ";
    std::cout << f.tellp() << '\n';
    f.seekg(0);
    std::cout << static_cast<char>(f.get()) << '\n';
    f.seekg(3, std::ios::cur);
    std::cout << static_cast<char>(f.get()) << '\n';
    f.seekg(-2, std::ios::end);
    char c = static_cast<char>(f.get());
    std::cout << c << ' ' << f.tellg() << '\n';
    return 0;
}
```

8. `soru8.txt` dosyasının içeriği `10 20 abc 30` biçimindedir. Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::ifstream f("soru8.txt");
    int x{0};
    int n{0};
    int toplam{0};
    while (f >> x) {
        toplam += x;
        n++;
    }
    std::cout << n << ' ' << toplam << ' ' << f.fail() << ' ' << f.eof() << '\n';
    f.clear();
    std::string s;
    f >> s;
    std::cout << s << '\n';
    f >> x;
    std::cout << x << '\n';
    return 0;
}
```

9. Aşağıdaki program parçasında iki hata vardır. Hataları bulun ve düzeltin.

```
std::ifstream oku("C:\notlar.txt");
std::string satir;
while (!oku.eof()) {


```

```
    std::getline(oku, satir);
    std::cout << satir << '\n';
}
```

10. Aşağıdaki program, ikili dosyaya yazdığı `std::string`'i neden doğru okuyamaz? Çözüm olarak ne yapılabilir?

```
struct Kisi {
    std::string ad;
    int yas;
};
Kisi k{"Ayşe", 20};
std::ofstream yaz("kisi.bin", std::ios::binary);
yaz.write(reinterpret_cast<const char*>(&k), sizeof k);
```

11. Bir dosyayı `ios::app` ile açmakla `ios::out` ile açmak arasındaki fark nedir? Hangi durumda hangisi seçilir? Kısaca yazın.
12. `sayilar.txt` dosyasını (her satırda bir tamsayı) okuyup sayıları bir `vector<int>`'e dolduran, ardından toplamı ve en büyük sayıyı ekrana yazan `sayiToplam` programını yazın. Dosya açılamazsa `cerr`'e iletiyi yazıp 1 ile çıkın.
13. Bir metin dosyasının satır sayısını, kelime sayısını ve karakter (bayt) sayısını bulan `sayac` programını yazın. Dosya adı programın içinde sabit olsun; dosya yoksa program onu birkaç satırla kendisi oluştursun.
14. `kaynak.txt` dosyasını `hedef.txt` dosyasına, `ios::binary` kipiyle blok blok (4096 baytlık bir `char` dizisi ile `read` / `gcount` / `write` kullanarak) kopyalayan `kopyala` programını yazın. Sonunda iki dosyanın boyutunu `<filesystem>` ile karşılaştırıp aynı olduklarını ekrana yazdırın.
15. Çalışma klasöründeki `.txt` uzantılı dosyaları, adlarına göre sıralı biçimde `ad: boyut bayt` biçiminde listeleyen `txtListele` programını `<filesystem>` ile yazın.

BÖLÜM 38

Özyineleme C++'ta

BU BÖLÜMDE

- Bölüm 18'deki özyineleme kalıplarını `std::vector`, `std::string` ve `std::map` ile nasıl yazacağımızı
- Büyük nesneleri `const` referansla geçirerek her çağrıda kopya oluşmasını nasıl önlediğimizi
- Referansla geçirilen bir kapsayıcıyı geri izleme (backtracking) yöntemiyle kullanarak permütasyonları ve alt kümeleri üretmeyi
- `std::pair` anahtarlı bir bellekleme tablosunu `std::map` ile kurmayı
- Birleştirmeli sıralama (merge sort) ile hızlı sıralamayı (quicksort) özyinelemeyle yazmayı
- Kötü pivot seçiminin çağrı derinliğini nasıl artırdığını ve yığın taşmasına nasıl yol açtığını

Bölüm 18'de özyinelemenin temel durum ile özyinelemeli durum ayrımını, çağrı yığınını, Fibonacci'nin üstel maliyetini ve bellekleme fikrini C ile gördük. Bu bölümde aynı kavramları baştan anlatmayacağız. Odağımız, C++'ın özyinelemeye kazandırdığı araçlar: referanslar, standart kapsayıcılar ve yığın üzerinde geri alınabilen değişiklikler. Kodda `std::` önekini kullanacağız; Bölüm 36'daki gibi standart kütüphane adlarının nereden geldiği böylece açık kalır.

38.1 Bölüm 18'den Taşıdıklarımız

Üç kural C++'ta da aynen geçerlidir:

- Her özyinelemeli fonksiyon önce temel durumu yazar; her çağrı temel duruma biraz daha yaklaşır.
- Her çağrı kendi çerçevesini açar; aynı anda yığında duran çerçeve sayısı özyineleme derinliğidir.
- Aynı alt problem defalarca çözülüyorsa bellekleme düşünülür.

Hanoi kuleleri, ikili arama, yazdırma sırasının çıktıyı tersine çevirmesi ve yığın taşması Bölüm 18'de işlendi; burada tekrarlamıyoruz. Aşağıdaki dört konu, C++'ın standart kapsayıcılarıyla daha doğal yazılan kalıpları ele alıyor.

38.2 vector ile Özyineleme

C'de bir diziyi fonksiyona verirken dizinin adresini ve uzunluğunu ayrı ayrı geçirirdik. `std::vector` ise kendi uzunluğunu bilir; ama değer ile geçirilirse her çağrıda bütün elemanlarıyla kopyalanır. `n` elemanlı bir vector için `n` çağrı yapılırsa toplamda yaklaşık n^2 eleman kopyalanmış olur.

◆ Kritik Nokta · Büyük nesneyi referansla geçirin

Vector'ü `const std::vector<int>&` biçiminde geçirin. Referans bir takma addır, kopya oluşmaz; `const` ise fonksiyonun vector'ü değiştirmediğini derleyiciye bildirir.

Örnekte iki soru soruyoruz: `i`. elemandan sona kadar olan çift sayıların toplamı ve en küçük eleman. İki fonksiyonda da `i` her çağrıda bir artar.

vektorOzyineleme.cpp

```cpp
#include <cstddef>
#include <iostream>
#include <vector>

// i. elemandan sona kadar çift sayıların toplamı
long long ciftToplam(const std::vector<int>& v, std::size_t i) {
    if (i == v.size()) {
        return 0;               // boş aralık
    }
    long long kalan{ciftToplam(v, i + 1)};
    return (v[i] % 2 == 0) ? v[i] + kalan : kalan;
}

// i. elemandan sona kadar en küçük eleman; v boş olmamalı
int enKucuk(const std::vector<int>& v, std::size_t i) {
    if (i + 1 == v.size()) {
        return v[i];           // son eleman: temel durum
    }
    int diger{enKucuk(v, i + 1)};
    return (v[i] < diger) ? v[i] : diger;
}

int main() {
    std::vector<int> v{4, 9, 2, 7, 5, 6};
    std::cout << "cift toplam = " << ciftToplam(v, 0) << '\n';
    std::cout << "en kucuk = " << enKucuk(v, 0) << '\n';
    return 0;
}
```

ÇIKTI

```
cift toplam = 12
en kucuk = 2
```

`ciftToplam` için temel durum, kalan aralığın boş olduğu `i == v.size()` anıdır ve 0 döner. Çift eleman ise değerini, değilse yalnızca kalanın toplamını döndürür.

`enKucuk` boş vector'ü kabul etmez: temel durum son eleman olduğu için `v[0]` gibi bir elemanın var olması gerekir. Bu yüzden çağırmadan önce vector'ün boş olmadığından emin olmalıyız.

▲ Sık Yapılan Hata · İşaretsiz tür ile dizin karışmasın

`v.size()` işaretsiz bir tür olan `std::size_t` döndürür. Dizin parametresini `int` yazıp `i < v.size()` karşılaştırması yaparsak g++ `-Wall` ile `-Wsign-compare` uyarısı verir. Daha kötüsü, işaretsiz bir değerden 1 çıkarmak 0'da sarmalanır ve çok büyük bir sayı olur. Bu yüzden dizin parametrelerini `std::size_t` yazmayı alışkanlık edinin.

38.3 Bellekleme: std::map ile Tablo

Bölüm 18'de tabloyu bir dizi ile tuttuk. Dizinin indeksi `n` olduğu için anahtarlar `0`'dan `n`'e sıkışık olmalıydı. Anahtar bazen bir çift olur. Örneğin `r` satır ve `c` sütunluk bir kutu ızgarasının sol üst köşesinden sağ alt köşesine, yalnızca sağa ya da aşağı giderek kaç farklı yolla gidilebilir?

Bu sorunun özyinelemeli bir tanımı vardır: `r == 0` ya da `c == 0` ise tek bir yön kalmıştır ve yol sayısı 1'dir. Aksi hâlde yol sayısı, bir adım yukarıdan ya da soldan gelen yolların toplamıdır: `yol(r, c) = yol(r - 1, c) + yol(r, c - 1)`.

Tablonun anahtarı `(r, c)` çifti olduğu için `std::map<std::pair<int, int>, long long>` kullanıyoruz.

izgaraYol.cpp

```
#include <iostream>
#include <map>
#include <utility>

// r satır, c sütunluk ızgarada sol üstten sağ alta yol sayısı
long long izgaraYol(int r, int c,
                    std::map<std::pair<int, int>, long long>& tablo) {
    if (r == 0 || c == 0) {
        return 1;               // tek yön kaldı: yalnızca bir yol var
    }
    std::pair<int, int> anahtar{r, c};
    auto it = tablo.find(anahtar);
    if (it != tablo.end()) {
        return it->second;      // daha önce hesaplandı
    }
    long long sonuc{izgaraYol(r - 1, c, tablo) + izgaraYol(r, c - 1, tablo)};
    tablo[anahtar] = sonuc;
    return sonuc;
}

int main() {
    std::map<std::pair<int, int>, long long> tablo;
    std::cout << "3x3 izgara: " << izgaraYol(3, 3, tablo) << " yol\n";
    std::cout << "16x16 izgara: " << izgaraYol(16, 16, tablo) << " yol\n";
    std::cout << "tabloda " << tablo.size() << " değer var\n";
    return 0;
}
```

```
ÇIKTI
3x3 izgara: 20 yol
16x16 izgara: 601080390 yol
tabloda 256 değer var
```

Sonuç `C(r + c, r)` binom katsayısına eşittir: 3x3 için `C(6, 3) = 20`, 16x16 için `C(32, 16) = 601 080 390`. Tablo, her `(r, c)` çiftini en fazla bir kez hesaplar; 16x16 için 256 çift vardır.

Tabloyu okurken `find` kullandık. `tablo[anahtar]` ifadesi anahtar yoksa boş bir kayıt ekler, bu yüzden okuma için `find` daha temizdir. `tablo[anahtar] = sonuc;` ise yalnızca hesaplanan değeri kaydetmek için kullanılır.

> **İpucu · Dizi mi, map mi?**
>
> Izgara boyutu baştan belliyse `std::vector<std::vector<long long>>` de kullanılabilir (Bölüm 26). Anahtarlar sıkışık değilse ya da boyut baştan bilinmiyorsa `std::map` daha esnektir; yalnızca gerçekten hesaplanan çiftler bellekte yer tutar.

38.4 Geri İzleme: Permütasyonlar

> **Tanım: Geri izleme (backtracking)**
> Bir seçim yapıp özyinelemeyle devam etme, dönüldüğünde seçimi geri alıp bir sonraki seçeneği deneme yöntemi. Çözümü adım adım kurar; yanlış bir yola girildiğinde geri döner.

Bir kelimenin harflerini yeniden dizmenin bütün yollarına **permütasyon** denir. `abc` için `3! = 6` dizilim vardır: `abc`, `acb`, `bac`, `bca`, `cba`, `cab`. Fikir şudur: `k`. konuma sırayla `k`'dan sonraki her harfi getiririz, özyinelemeyle `k + 1`. konumu doldururuz ve dönünce yerleri eski hâline getiririz.

> **Tanım: Permütasyon**
> Bir dizinin elemanlarının bütün sıralanış biçimleri. `n` farklı eleman için `n!` tane permütasyon vardır.

▸ permutasyon.cpp

```cpp
#include <cstddef>
#include <iostream>
#include <string>
#include <utility>

// s[0..k-1] sabit; s[k..] içindeki harflerden sırayla k. konum denenir
void permutasyon(std::string& s, std::size_t k) {
    if (k == s.size()) {
        std::cout << s << '\n';     // tam bir dizilim bulundu
        return;
    }
    for (std::size_t i{k}; i < s.size(); ++i) {
        std::swap(s[k], s[i]);      // i. harfi k. konuma getir
        permutasyon(s, k + 1);
        std::swap(s[k], s[i]);      // geri al
    }
}

int main() {
    std::string kelime{"abc"};
    permutasyon(kelime, 0);
    std::cout << "kelime hâlâ: " << kelime << '\n';
    return 0;
}
```

```
ÇIKTI
abc
acb
bac
bca
cba
cab
kelime hâlâ: abc
```

`std::swap` iki harfin yerini değiştirir. Dönüşte aynı değiş tokuşu bir kez daha yaparak harfleri eski yerlerine koyuyoruz. Son satır bunun işe yaradığını gösterir: fonksiyon bittiğinde kelime değişmemiştir.

Dizilim sayısı hızla büyür. On harfli bir kelimenin `10! = 3 628 800` dizilimi vardır. Standart kütüphanede bu iş için `<algorithm>` başlığındaki `std::next_permutation` bulunur (Bölüm 36); burada mekanizmayı görmek için kendi sürümümüzü yazdık.

38.5 Alt Kümeler ve Geri Alma

`n` elemanlı bir kümenin `2^n` tane alt kümesi vardır. Her eleman için iki seçenek vardır: alınır ya da alınmaz. Seçilen elemanları tek bir `secilen` vector'ünde tutuyoruz ve bunu referansla geçiriyoruz. Bir eleman alındığında `push_back` ile eklenir; alt dal bittiğinde `pop_back` ile çıkarılır.

> **Tanım: Alt küme**
>
> Bir kümenin elemanlarından hiçbiri, bazısı ya da hepsi alınarak oluşturulan küme. Boş küme de bir alt kümedir.

▸ altKume.cpp

```cpp
#include <cstddef>
#include <iostream>
#include <vector>

// seçilen vector'ü yazdırır: {1, 3} gibi
void yazdir(const std::vector<int>& s) {
    std::cout << "{";
    for (std::size_t i{0}; i < s.size(); ++i) {
        if (i > 0) {
            std::cout << ", ";
        }
        std::cout << s[i];
    }
    std::cout << "}\n";
}

// v[i..] için her eleman alınır ya da alınmaz
void altKumeler(const std::vector<int>& v, std::size_t i,
                std::vector<int>& secilen) {
    if (i == v.size()) {
        yazdir(secilen);
        return;
    }
    altKumeler(v, i + 1, secilen);   // v[i] alınmadan
    secilen.push_back(v[i]);         // v[i] alınarak
    altKumeler(v, i + 1, secilen);
    secilen.pop_back();              // geri al
}

int main() {
    std::vector<int> v{1, 2, 3};
    std::vector<int> secilen;        // başta boş
    altKumeler(v, 0, secilen);
    return 0;
}
```

ÇIKTI

```
{}
{3}
{2}
{2, 3}
{1}
{1, 3}
{1, 2}
{1, 2, 3}
```

Çıktının sırası, her elemanda önce "alınmadı" dalının tamamen bitmesinden gelir. Bölüm 18'deki çağrı ağacı burada da işe yarar: üç eleman için sekiz yaprak vardır ve her yaprak bir satır yazdırır.

> **▲ Sık Yapılan Hata · Geri almayı unutmak**
>
> `secilen` referansla geçirildiği için bütün çağrılar tek bir vector'ü paylaşır. `pop_back` çağrısını unutursanız eklenen eleman bir sonraki dalda da kalır ve sonraki alt kümeler yanlış çıkar. Değer ile geçirseydik her çağrı kendi kopyasında çalışacağı için geri almaya gerek kalmazdı; ama her çağrıda vector kopyalanırdı. Referans, kopyadan kaçınmayı sağlar; karşılığında geri alma sorumluluğunu da getirir.

> **◆ Kritik Nokta · Referans parametre ve geri alma eşleşmeli**
>
> Bir özyinelemeli fonksiyon paylaşılan bir kapsayıcıya eleman eklediyse, çağrı dönmeden önce o eklemeyi geri almalıdır. Eklemeyle geri alma aynı sayıda ve sırayla eşleşmelidir.

38.6 Birleştirmeli Sıralama (Merge Sort)

Birleştirmeli sıralamanın fikri şudur: aralığı ortadan ikiye böl, iki yarıyı özyinelemeyle sırala, sonra iki sıralı yarıyı tek bir sıralı dizide birleştir. Temel durum 0 ya da 1 elemanlı aralıktır; bu aralıklar zaten sıralıdır. Aralık her adımda yarıya indiği için derinlik yaklaşık $\log_2 n$ olur ve her düzeyde toplam `n` eleman birleştirilir.

Aralığı yarı açık tutuyoruz: `[bas, son)` biçiminde, `son` dahil değildir. Bu, Bölüm 36'daki yineleyici aralıklarıyla aynı kuraldır. Orta noktayı `bas + (son - bas) / 2` diye hesaplamak, büyük indekslerde toplamın taşmasını önler.

▸ birlesmeliSirala.cpp

```
#include <cstddef>
#include <iostream>
#include <vector>

// v[bas..son-1] aralığını sıralar
void birlesmeliSirala(std::vector<int>& v, std::size_t bas, std::size_t son) {
    if (son - bas <= 1) {
        return;     // 0 ya da 1 elemanlı aralık zaten sıralıdır
    }
    std::size_t orta{bas + (son - bas) / 2};
    birlesmeliSirala(v, bas, orta);
    birlesmeliSirala(v, orta, son);

    // iki sıralı yarıyı geçici bir vector'de birleştir
    std::vector<int> gecici;
    std::size_t i{bas};
    std::size_t j{orta};
    while (i < orta && j < son) {
        if (v[i] <= v[j]) {
            gecici.push_back(v[i++]);
        } else {
            gecici.push_back(v[j++]);
        }
    }
    while (i < orta) {
        gecici.push_back(v[i++]);
    }
    while (j < son) {
        gecici.push_back(v[j++]);
    }
```

```
    for (std::size_t k{0}; k < gecici.size(); ++k) {
        v[bas + k] = gecici[k];     // sonucu diziye geri yaz
    }
}

// vector'ün elemanlarını boşlukla yazdırır
void yazdir(const std::vector<int>& v) {
    for (std::size_t i{0}; i < v.size(); ++i) {
        if (i > 0) {
            std::cout << ' ';
        }
        std::cout << v[i];
    }
    std::cout << '\n';
}

int main() {
    std::vector<int> v{38, 27, 43, 3, 9, 82, 10};
    std::cout << "Önce: ";
    yazdir(v);
    birlesmeliSirala(v, 0, v.size());
    std::cout << "Sonra: ";
    yazdir(v);
    return 0;
}
```

```
ÇIKTI
Önce: 38 27 43 3 9 82 10
Sonra: 3 9 10 27 38 43 82
```

Birleştirme döngüsünde `<=` kullanmak, eşit elemanların göreli sırasını korur; bu özelliğe kararlılık (stability) denir. Birleştirme için geçici bir vector kullandık; bu, sıralamanın ek bellek harcamasına neden olur.

> **Üç Dilde · Diziyi özyinelemeye geçirmek: C, C++ ve C#** *C · C++ · C#*
>
> **C'de** dizi ve uzunluk ayrı parametrelerle geçirilir: `void sirala(int dizi[], int bas, int son)`. Dizi adres olarak geçtiği için kopyalanmaz; ama uzunluk bilgisi fonksiyona kendiliğinden gelmez, ayrıca geçirilir.
>
> **C++'ta** `std::vector<int>&` hem uzunluğu taşır hem de referans olduğu için kopyalanmaz. Aynı işi `std::size_t` ile yazılmış `bas` ve `son` indeksleriyle sürdürürüz.
>
> **C#'ta** `int[]` bir referans tipidir; parametre olarak geçtiğinde dizi kopyalanmaz. Aynı sıralama kalıbı C# dizisi üzerinde de aynen kurulur.

38.7 Hızlı Sıralama (Quicksort)

Hızlı sıralamada iş bölmede yapılır. Bir **pivot** seçilir; pivottan küçük elemanlar sola, büyük ya da eşit olanlar sağa taşınır. Bölme bittiğinde pivot doğru yerindedir. Sol ve sağ parçalar aynı yöntemle özyinelemeyle sıralanır. Burada pivot olarak aralığın son elemanını seçtik (Lomuto şeması).

> **Tanım: Pivot**
>
> Hızlı sıralamada, bölme işlemini yönlendiren eleman. Pivottan küçükler sola, büyükler sağa alınır; pivot bu iki parçanın arasında kesin yerine oturur.

> *Tarihçe · Hızlı sıralama*
>
> C. A. R. Hoare hızlı sıralamayı 1959'da geliştirdi ve 1961'de yayımladı. Birleştirmeli sıralama ise 1945'te John von Neumann tarafından önerildi.

hizliSirala.cpp

```cpp
#include <cstddef>
#include <iostream>
#include <utility>
#include <vector>

int enDerin{0};     // en derin özyineleme düzeyi

// v[bas..son-1] içinde son elemanı pivot alır, bölmeyi yapar,
// pivotun son konumunu döndürür
std::size_t bol(std::vector<int>& v, std::size_t bas, std::size_t son) {
    int pivot{v[son - 1]};
    std::size_t i{bas};     // pivottan küçüklerin biteceği konum
    for (std::size_t j{bas}; j + 1 < son; ++j) {
        if (v[j] < pivot) {
            std::swap(v[i], v[j]);
            ++i;
        }
    }
    std::swap(v[i], v[son - 1]);    // pivotu yerine koy
    return i;
}

void hizliSirala(std::vector<int>& v, std::size_t bas, std::size_t son,
                 int derinlik) {
    if (son - bas <= 1) {
        return;
    }
    if (derinlik > enDerin) {
        enDerin = derinlik;
    }
    std::size_t p{bol(v, bas, son)};
    hizliSirala(v, bas, p, derinlik + 1);       // pivotun solu
    hizliSirala(v, p + 1, son, derinlik + 1);   // pivotun sağı
}

void yazdir(const std::vector<int>& v) {
    for (std::size_t i{0}; i < v.size(); ++i) {
        if (i > 0) {
            std::cout << ' ';
        }
        std::cout << v[i];
    }
    std::cout << '\n';
}

int main() {
    std::vector<int> v{38, 27, 43, 3, 9, 82, 10};
    hizliSirala(v, 0, v.size(), 1);
    std::cout << "Sıralı: ";
    yazdir(v);

    std::vector<int> sirali{1, 2, 3, 4, 5, 6, 7};
    enDerin = 0;
    hizliSirala(sirali, 0, sirali.size(), 1);
```

```
    std::cout << "Sıralı girdide en derin düzey: " << enDerin << '\n';
    return 0;
}
```

```
ÇIKTI
Sıralı: 3 9 10 27 38 43 82
Sıralı girdide en derin düzey: 6
```

`bol` fonksiyonunda `i`, pivottan küçük elemanların biteceği konumu gösterir. `j` döngüde ilerler; `v[j] < pivot` olduğunda `v[j]` ile `v[i]` yer değiştirir ve `i` bir artar. Döngü bitince pivot `v[son - 1]`'den `v[i]`'ye taşınır ve `i` döndürülür. Pivot artık yerindedir, bu yüzden özyineleme `[bas, p)` ve `[p + 1, son)` aralıklarına iner; `p` hiçbir çağrıya girmez.

Ortalama durumda her bölme aralığı kabaca yarıya indirir ve toplam iş `n log n` olur.

> **▲ Sık Yapılan Hata · Sıralı girdi ve kötü pivot**
>
> Son elemanı pivot aldığımızda, dizi zaten sıralıysa her bölme pivotu tek başına bir uca koyar: bir parça `n - 1` elemanlı, öbürü boş olur. Derinlik `n`'e çıkar ve yığın taşabilir. Çıktıdaki `6`, 7 elemanlı dizide derinliğin `n - 1` olduğunu gösterir. Gerçek kütüphaneler pivot seçimine özen gösterir ve derinlik çok arttığında başka bir yönteme geçer; `std::sort` böyle çalışır (Bölüm 36).

> **◆ Kritik Nokta · Her özyinelemeli çağrı aralığı küçültmeli**
>
> Hem birleştirmeli hem hızlı sıralamada temel durum, kalan aralığın 0 ya da 1 elemana inmesidir. Bölme ise aralığı gerçekten küçültmelidir. Hızlı sıralamada `p` aralığın dışına kadar giden bir değer seçilirse aynı aralık tekrar tekrar açılır.

Bölüm Özeti

- Bölüm 18'deki özyineleme kalıpları C++'ta da geçerlidir: temel durum, küçülen argüman ve çağrı yığını.
- Büyük nesneleri `const` referansla geçirin; `const std::vector<T>&` kopya oluşturmaz.
- Dizin parametrelerinde `std::size_t` kullanın. `int` ile karşılaştırma `-Wsign-compare` uyarısı verir; işaretsizde 0'dan 1 çıkarmak sarmalanır.
- Geri izleme: seçimi yap, özyinele, dönünce geri al. Referansla paylaşılan bir vector için `push_back` ve `pop_back` çiftini eşleşmiş kurun.
- Permütasyonlar için `std::swap` ile yer değiştirip dönüşte aynı değişimi geri yapın. `n` harf için `n!` dizilim vardır.
- Alt kümelerde her eleman için iki seçenek vardır; `n` eleman için `2^n` alt küme oluşur.
- Bellekleme anahtarı bir çift olduğunda `std::map<std::pair<int, int>, long long>` kullanın; okumayı `find` ile yapın.
- Bellekleme tablosunu değer ile geçirmeyin: tablo her çağrıda kopyalanır ve yazılan değerler çağırana ulaşmaz.
- Birleştirmeli sıralama aralığı ortadan böler, iki yarıyı sıralar ve birleştirir. Derinlik `log n`, toplam iş `n log n`'dir.

- Hızlı sıralama pivota göre böler. Ortalama `n log n`'dir; sıralı girdide son eleman pivotla en kötü durum (`n²`) oluşur.
- Derinlik sınırsızca artarsa yığın taşar. Pivot seçimi bu yüzden önemlidir.

Alıştırmalar

1. Aşağıdakilerden hangisi, `int toplam(std::vector<int> v, std::size_t n)` gibi değer ile geçirilen bir vector için doğrudur?

   A) Vector her çağrıda kopyalanır; `n` elemanlı bir vector için yaklaşık `n²` eleman kopyalanır.
   B) Vector yalnızca ilk çağrıda kopyalanır, sonraki çağrılar referans kullanır.
   C) Değer ile geçirme, `const` ile birlikte kopyayı önler.
   D) Vector'ün kopyası yalnızca eleman sayısı 10'dan büyükse oluşur.
   E) Değer ile geçirilen vector fonksiyonun içinde değiştirilemez.

2. `std::size_t n{0}` iken `n - 1` ifadesinin değeri nedir?

   A) -1
   B) 0
   C) `std::size_t` türünün en büyük değeri (sarmalanma)
   D) Derleme hatası
   E) Program çöker

3. `izgaraYol` fonksiyonunda tablo `std::map<std::pair<int, int>, long long>& tablo` yerine değer ile (`&` olmadan) geçirilirse ne olur?

   A) Program derlenmez.
   B) Her çağrı tablonun kendi kopyasıyla çalışır; çağıranın tablosu boş kalır, bellekleme etkisiz olur. Sonuç doğru ama yavaş çıkar.
   C) Sonuç yanlış çıkar, çünkü yol sayısı negatif olur.
   D) Derleyici referansa kendiliğinden çevirir.
   E) Çağrı derinliği artar ve program çöker.

4. Dört harfli bir kelimenin bütün permütasyonlarını `permutasyon` fonksiyonuyla yazdırdığımızda kaç satır çıktı oluşur?

   A) 4
   B) 8
   C) 16
   D) 24
   E) 256

5. `altKumeler` fonksiyonunu dört elemanlı bir vector için çalıştırırsak kaç satır çıktı oluşur?

   A) 4
   B) 8
   C) 15
   D) 16
   E) 24

6. Birleştirmeli sıralamanın `n` eleman için zaman karmaşıklığı (ortalama ve en kötü durum) hangisidir?

   A) `n`
   B) `n log n`

C) `n²`
D) `log n`
E) `2^n`

7. Son elemanı pivot alan bir hızlı sıralama uygulamasında hangi girdi en kötü durumu yaratır?

A) Rastgele karıştırılmış dizi
B) Küçükten büyüğe sıralı dizi
C) İki elemanlı dizi
D) Tek elemanlı dizi
E) Yalnızca negatif sayılardan oluşan dizi

8. Aşağıdaki programın çıktısı nedir?

```
#include <cstddef>
#include <iostream>
#include <string>
#include <utility>

void f(std::string& s, std::size_t k) {
    if (k == s.size()) {
        std::cout << s << ' ';
        return;
    }
    for (std::size_t i{k}; i < s.size(); ++i) {
        std::swap(s[k], s[i]);
        f(s, k + 1);
        std::swap(s[k], s[i]);
    }
}

int main() {
    std::string s{"ab"};
    f(s, 0);
    std::cout << s << '\n';
    return 0;
}
```

9. Aşağıdaki programın çıktısı nedir? (Her yaprakta `secilen.size()` yazdırılıyor.)

```
#include <cstddef>
#include <iostream>
#include <vector>

void altKumeler(const std::vector<int>& v, std::size_t i,
                std::vector<int>& secilen) {
    if (i == v.size()) {
        std::cout << secilen.size() << ' ';
        return;
    }
    altKumeler(v, i + 1, secilen);
    secilen.push_back(v[i]);
    altKumeler(v, i + 1, secilen);
    secilen.pop_back();
}

int main() {
    std::vector<int> v{5, 6};
    std::vector<int> secilen;
    altKumeler(v, 0, secilen);
```

```
    std::cout << '\n';
    return 0;
}
```

10. Aşağıdaki kontrol, `son` ve `bas` değerleri `std::size_t` olduğunda ne yapar?

```
bool gecerliMi(std::size_t bas, std::size_t son) {
    if (son - bas < 0) {
        return false;
    }
    return true;
}
```

    A) Her zaman `false` döner, çünkü koşul hiçbir zaman doğru olmaz.
    B) `son < bas` olsa bile fark sarmalanarak büyük bir pozitif değere döner; koşul hiçbir zaman doğru olmaz.
    C) Derleme hatası verir.
    D) Yalnızca `bas` sıfırken doğru olur.
    E) Yalnızca `son` ile `bas` eşitken doğru olur.

11. Aşağıdaki kodu `g++ -std=c++17 -Wall` ile derlediğimizde ne uyarısı alırız? Uyarının adını ve nedenini yazın.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3};
    for (int i = 0; i < v.size(); ++i) {
        std::cout << v[i] << '\n';
    }
    return 0;
}
```

12. `izgaraYol` fonksiyonunda temel durumu yalnızca `r == 0` olarak yazarsak ne olur? Nedenini açıklayın.

13. **Program yazın:** Klavyeden 1 ile 4 arasında bir `n` okuyup 1'den `n`'e kadar sayıların bütün permütasyonlarını yazdıran programı yazın. Permütasyon fonksiyonu `std::vector<int>` ve `std::swap` kullansın. `n = 3` için çıktı şöyle olmalı:

```
Sayı girin: {{3}}
1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2
```

14. **Program yazın:** Birleştirmeli sıralamayı, sıralama düzenini seçen bir `bool azalan` parametresiyle yeniden yazın. `{38, 27, 43, 3, 9, 82, 10}` dizisi `azalan = true` ile `82 43 38 27 10 9 3` olarak sıralanmalı.

15. **Program yazın:** Bir vector ile bir hedef toplam `S` alın ve toplamı `S` olan bütün alt kümeleri yazdırın. Alt kümeleri Bölüm 38'deki geri izleme yöntemiyle bulun. `{1, 2, 3, 4}` ve `S = 5` için çıktı şöyle olmalı:

```
{2, 3}
{1, 4}
```

BÖLÜM 39

Modern C++: Taşıma Semantiği ve Ötesi

BU BÖLÜMDE

- Kopyalamanın maliyetini ve taşıma (move) fikrinin neden gerektiğini açıklamayı
- lvalue, rvalue ve rvalue referansı kullanarak `std::move`'un gerçekte ne yaptığını söylemeyi
- Bir sınıf için taşıma yapıcısını ve taşıma atamasını yazmayı; beşler kuralı ile sıfır kuralını ayırt etmeyi
- `auto`, `decltype`, `constexpr`, `enum class`, `std::optional`, `std::variant` ve `std::string_view` özelliklerini yerinde kullanmayı
- C++20'nin kavramlar (concepts), aralıklar (ranges), `std::format` ve modüller özelliklerini tanımayı
- Modern C++ yazım önerilerini kendi kodunda uygulamayı

C++ kısmı boyunca Bölüm 21'de çizdiğimiz yolda ilerledik: C'nin üzerine sınıflar, şablonlar, kapsayıcılar ve RAII geldi. Bu bölümde dilin 2011'den sonra edindiği en önemli yeniliği, **taşıma semantiğini** ele alıyoruz. Bölüm 28'de `T&&` yazımını ve lvalue/rvalue ayrımını sezgisel olarak tanımıştık; Bölüm 29'da `std::unique_ptr`'nin `std::move` ile devredildiğini görmüştük. Şimdi bu parçaları birleştirip kendi sınıflarımıza taşıma yeteneği kazandıracağız.

Ardından modern C++ olarak anılan üslubun geri kalan araçlarını topluca gözden geçireceğiz. Bölümün son kısmı C++20'ye kısa bir bakıştır; orada kullanılan özellikler kitabın varsayılan C++17 ayarında derlenmez ve ilgili örneklerin başında `` notu bulunur.

39.1 Neden Taşıma? Kopyalamanın Maliyeti

Bölüm 30'da bir sınıf dinamik bellek tutuyorsa kopyalamanın **derin kopya** gerektirdiğini gördük: yeni bellek ayrılır, içerik eleman eleman aktarılır. Bir milyon elemanlı bir `vector<int>`'i kopyalamak 4 MB'lık yeni bellek ayırmak ve onu doldurmak demektir.

Her zaman böyle bir bedel ödememiz gerekmez. Şu satıra bakalım:

```
std::vector<int> v = veriOku();   // fonksiyon yerel bir vector döndürüyor
```

`veriOku` içindeki yerel vektör, fonksiyon biter bitmez yok olacaktır. Elemanlarını kopyalayıp sonra orijinali silmek, bir evi tuğla tuğla yeni bir arsaya taşıyıp eskisini yıkmak gibidir. Akıllıca olan, **bellek bloğunun sahipliğini** yeni nesneye devretmektir: yalnızca birkaç işaretçi kopyalanır, tuğlalara dokunulmaz.

```
Kopya sonrası                       Taşıma sonrası
a: [boyut|veri] --> [1 2 3 ...]     a: [0|nullptr]
b: [boyut|veri] --> [1 2 3 ...]     b: [boyut|veri] --> [1 2 3 ...]
(iki ayrı blok)                     (tek blok, sahibi değişti)
```

Taşıma, kaynağı elindeki nesneden bir başkasına **çalmaktır**; ama bunu yalnızca kaynak nesne artık kullanılmayacaksa yapabiliriz. Geçici nesneler tam olarak bu durumdadır. Derleyicinin bir değerin "artık kullanılmayacak" olduğunu anlaması için de C++11, rvalue referansını getirdi.

> **Tanım: Taşıma semantiği**
>
> Bir nesnenin sahip olduğu kaynağı (dinamik bellek, dosya tutamacı vb.) kopyalamak yerine başka bir nesneye devretme anlayışıdır (move semantics). Devir sonrasında kaynak nesne **geçerli ama belirsiz** (valid but unspecified) bir durumda kalır; yok edilebilir ve yeniden atanabilir.

Taşımanın görünür etkisi

Standart kapsayıcılar taşımayı zaten bilir. Bir `vector`'ü kopyalayıp taşıyarak farkı gözle görelim. Tamponun adresini `data()` ile alıp karşılaştırıyoruz:

▸ tasimaTemel.cpp

```cpp
#include <iostream>
#include <vector>
#include <utility>

int main() {
    std::vector<int> a(1000000, 7);
    const int* adres = a.data();

    std::vector<int> kopya = a;              // kopya: yeni bellek, 1 milyon eleman kopyalanır
    std::vector<int> tasinan = std::move(a); // taşıma: bellek el değiştirir

    std::cout << std::boolalpha;
    std::cout << "kopya ayni bellek mi : " << (kopya.data() == adres) << '\n';
    std::cout << "tasinan ayni bellek mi: " << (tasinan.data() == adres) << '\n';
    std::cout << "a.size() = " << a.size() << '\n';
    std::cout << "tasinan.size() = " << tasinan.size() << '\n';

    a.push_back(1);                          // taşınmış nesne yine kullanılabilir
    std::cout << "a.size() = " << a.size() << '\n';
    return 0;
}
```

```
ÇIKTI
kopya ayni bellek mi : false
tasinan ayni bellek mi: true
a.size() = 0
tasinan.size() = 1000000
a.size() = 1
```

Kopya yeni bir bellek bloğuna sahip olur. Taşınan vektör ise `a`'nın bloğunu **aynen** devralmıştır; milyon eleman kopyalanmamış, yalnızca işaretçi aktarılmıştır. `a` boşalmıştır, ama hâlâ geçerli bir vektördür: `push_back` ile yeniden doldurabildik.

Not · Taşınmış nesne ne kadar boştur?

Standart, taşınmış `vector`'ün boş olacağını garanti eder. Ama genel kural daha zayıftır: standart kütüphane türleri için "geçerli ama belirsiz". Örneğin taşınmış bir `std::string`'in içeriği hakkında bir şey varsaymamalıyız; yalnızca onu yok edebilir ya da yeniden değer atayabiliriz. Kendi sınıflarımızda ise taşıma sonrası durumu biz belirleriz ve en güvenlisi nesneyi tanımlı bir boş duruma getirmektir.

39.2 lvalue, rvalue ve std::move

Bölüm 28'de ayrımı şöyle özetlemiştik: adı olan ve adresi alınabilen ifade lvalue, ifadenin sonunda yok olan geçici değer rvalue'dur. `T&` lvalue'ya, `T&&` rvalue'ya bağlanır.

Bu bilgiyle bir işlevi iki biçimde aşırı yükleyebiliriz: lvalue geldiğinde kopyala, rvalue geldiğinde taşı. İşte taşıma yapıcısının dayandığı fikir budur. Sorun, elimizde adı olan bir nesne varken ve onun artık gerekmediğini bildiğimizde ortaya çıkar: ad taşıyan her ifade lvalue'dur ve derleyici onu kendiliğinden taşımaz. Bunu bildirmek için `std::move` kullanırız.

Tanım: std::move

`<utility>` başlığındaki `std::move(x)`, hiçbir şeyi taşımaz; yalnızca `x` ifadesini bir rvalue'ya (daha doğrusu xvalue'ya) **dönüştürür**. Taşımayı, rvalue kabul eden taşıma yapıcısı ya da taşıma ataması gerçekleştirir.

Yani `std::move`, bir tür `static_cast<T&&>(x)` dönüşümüdür ve "bu nesneyi artık kullanmayacağım, kaynaklarını alabilirsin" diye derleyiciye verilen **iznin adıdır**. İzni verdikten sonra o nesneye dokunmamak programcıya düşer.

▲ Sık Yapılan Hata · Taşınmış nesneyi kullanmak

```
std::string a = "merhaba";
std::string b = std::move(a);
std::cout << a;   // derlenir, ama a'nın içeriği belirsizdir
```

`std::move(a)`'dan sonra `a`'ya yalnızca yeniden atama yapılmalı ya da `a` yok edilmelidir. Değerini okumak hata değildir ama sonucu varsaymak hatadır.

`std::move`'un iki sık yanlış kullanımı daha vardır:

- **`const` nesneyi taşımak**: `const T` bir rvalue olarak dönüştürüldüğünde `const T&&` olur; taşıma yapıcısı `T&&` bekler, bu yüzden kopya yapıcısı seçilir. Taşıma sessizce kopyaya döner.
- **`return` ile gereksiz `std::move`**: Yerel bir nesneyi `return std::move(yerel);` ile döndürmek derleyicinin yaptığı iyileştirmeyi (aşağıda göreceğiz) engeller. Yerel nesneyi doğrudan `return yerel;` yazarak döndürürüz.

39.3 Taşıma Yapıcısı ve Taşıma Ataması

Kaynağa sahip bir sınıf için taşıma işlemlerini yazalım. Bölüm 30'daki `Dizi` sınıfına benzer, ama konuyu odaklayan küçük bir `Tampon` sınıfı kullanacağız. Her özel üye fonksiyon, hangisinin çalıştığını görebilmemiz için bir satır yazdırıyor.

Kalıp şudur:

1. Taşıma yapıcısı `Tampon(Tampon&& o) noexcept` kaynağın işaretçilerini **kopyalar**, ardından kaynağı boş duruma getirir (`nullptr`, boyut 0).
2. Taşıma ataması önce kendi kaynağını serbest bırakır, sonra aynı devri yapar. Kendine atamayı (`this != &o`) denetleriz.
3. İkisi de `noexcept` olarak işaretlenir (nedenini birazdan göreceğiz).

tampon.cpp

```cpp
#include <algorithm>
#include <cstddef>
#include <iostream>
#include <utility>

class Tampon {
    std::size_t boyut{0};
    int* veri{nullptr};

public:
    explicit Tampon(std::size_t n) : boyut{n}, veri{new int[n]{}} {
        std::cout << "yapici(" << boyut << ")\n";
    }

    ~Tampon() {
        std::cout << "yikici(" << boyut << ")\n";
        delete[] veri;
    }

    Tampon(const Tampon& o) : boyut{o.boyut}, veri{new int[o.boyut]} {
        std::copy(o.veri, o.veri + boyut, veri);
        std::cout << "kopya yapici\n";
    }

    Tampon& operator=(const Tampon& o) {
        std::cout << "kopya atama\n";
        if (this != &o) {
            int* yeni = new int[o.boyut];
            std::copy(o.veri, o.veri + o.boyut, yeni);
            delete[] veri;
            veri = yeni;
            boyut = o.boyut;
        }
        return *this;
    }

    Tampon(Tampon&& o) noexcept : boyut{o.boyut}, veri{o.veri} {
        o.boyut = 0;
        o.veri = nullptr;
        std::cout << "tasima yapici\n";
    }

    Tampon& operator=(Tampon&& o) noexcept {
        std::cout << "tasima atama\n";
        if (this != &o) {
            delete[] veri;
            boyut = o.boyut;
            veri = o.veri;
            o.boyut = 0;
            o.veri = nullptr;
        }
        return *this;
```

```
    }

    std::size_t boyutu() const { return boyut; }
};

Tampon uret() {
    Tampon t{8};
    return t;                    // taşıma da kopya da gerekmez
}

int main() {
    Tampon a{4};
    Tampon b = a;                // kopya yapıcı
    Tampon c = std::move(a);     // taşıma yapıcı
    std::cout << "a: " << a.boyutu() << ", c: " << c.boyutu() << '\n';

    b = c;                       // kopya atama
    b = std::move(c);            // taşıma ataması
    std::cout << "b: " << b.boyutu() << ", c: " << c.boyutu() << '\n';

    Tampon d = uret();
    std::cout << "d: " << d.boyutu() << '\n';
    return 0;
}
```

```
ÇIKTI
yapici(4)
kopya yapici
tasima yapici
a: 0, c: 4
kopya atama
tasima atama
b: 4, c: 0
yapici(8)
d: 8
yikici(8)
yikici(0)
yikici(4)
yikici(0)
```

Çıktıyı satır satır okuyalım:

- `Tampon b = a;` adı olan bir lvalue'dan yapıldığı için kopya yapıcısı çalıştı. `Tampon c = std::move(a);` ise `a`'yı rvalue'ya çevirdiği için taşıma yapıcısı seçildi. Sonrasında `a.boyutu()` sıfırdır: `a` boşaltılmıştır.
- `b = c;` kopya atamasını, `b = std::move(c);` taşıma atamasını çağırdı. Bu satırlarda `b`'nin eski belleği serbest bırakıldı, `c`'nin bloğu `b`'ye geçti.
- `uret()` için **hiçbir** özel üye fonksiyon satırı yazılmadı; yalnızca `yapici(8)` görünüyor. Derleyici yerel `t` nesnesini doğrudan `d`'nin bulunacağı yerde oluşturdu. Bu iyileştirmeye **kopya eleme** (copy elision) denir. C++17'den itibaren isimsiz bir geçici döndürmek (`return Tampon{8};`) için eleme zorunludur. `return t;` gibi adlandırılmış yerel nesnede ise (NRVO) zorunlu değildir; derleyiciler uygular, uygulamazsa taşıma yapıcısı devreye girer. Her iki durumda da `return std::move(t);` yazmak yararsızdır.
- Yok edilme sırası tanımlama sırasının tersidir: `d` (8), `c` (0), `b` (4), `a` (0). Taşınmış nesneler de yok edilir; boş durumda olmaları `delete[] nullptr` çağrısını zararsız kıldığı için çifte silme olmaz. Bölüm 29'da gördüğümüz `delete` güvencesi burada işe yarar.

noexcept neden önemli?

Taşıma işlemlerini `noexcept` işaretlemek bir süs değildir. `vector` büyürken (yeniden bellek ayırırken) elemanları yeni bloğa aktarır. Bunu taşıyarak yapmak ister; ama taşıma sırasında bir istisna (Bölüm 34) atılırsa eski bloktaki elemanlar yarı taşınmış, yeni bloktakiler eksik kalır ve vektörü eski sağlam hâline döndürmek imkânsızlaşır. Bu nedenle `vector`, yalnızca taşıma yapıcısı `noexcept` ise (ya da kopyalama mümkün değilse) taşır; aksi hâlde güvenli olan kopyalamaya döner.

noexceptTasima.cpp

```cpp
#include <iostream>
#include <utility>
#include <vector>

struct Hizli {
    Hizli() = default;
    Hizli(const Hizli&) { std::cout << "  Hizli kopya\n"; }
    Hizli(Hizli&&) noexcept { std::cout << "  Hizli tasima\n"; }
};

struct Yavas {
    Yavas() = default;
    Yavas(const Yavas&) { std::cout << "  Yavas kopya\n"; }
    Yavas(Yavas&&) { std::cout << "  Yavas tasima\n"; }
};

int main() {
    std::cout << "Hizli:\n";
    std::vector<Hizli> h;
    h.emplace_back();
    h.emplace_back();           // bellek büyür, elemanlar yeni yere aktarılır

    std::cout << "Yavas:\n";
    std::vector<Yavas> y;
    y.emplace_back();
    y.emplace_back();
    return 0;
}
```

```
ÇIKTI
Hizli:
  Hizli tasima
Yavas:
  Yavas kopya
```

İki sınıfın tek farkı `noexcept`'tir. İkinci `emplace_back` bellek büyümesini tetikler; birinci eleman `Hizli` için taşındı, `Yavas` için kopyalandı. Kopyalanan her eleman, gerçek bir sınıfta tüm kaynakların yeniden ayrılması demektir.

> **◆ Kritik Nokta · Taşıma işlemlerini noexcept yazın**
>
> Kaynağı işaretçi devriyle aktaran bir taşıma işlemi zaten istisna atmaz. `noexcept` yazmayı unutursak sınıf doğru çalışmaya devam eder ama `vector` gibi kapsayıcılar sessizce kopyaya döner ve taşımanın kazancı kaybolur.

39.4 Beşler Kuralı ve Sıfır Kuralı

Bölüm 30'da üçler kuralını öğrenmiştik: yıkıcı, kopya yapıcı ve kopya atamasından birini yazıyorsak üçünü de yazmalıyız. Taşıma işlemleri eklendiğinde özel üye fonksiyonların sayısı beşe çıkar.

> **Tanım: Beşler kuralı**
>
> Bir sınıf için yıkıcıyı, kopya yapıcıyı, kopya atamasını, taşıma yapıcısını ve taşıma atamasını (rule of five) birlikte düşünmek gerekir. Bunlardan birini kendimiz yazıyorsak, çoğunlukla beşini de bilerek yazmalı, silmeli ya da `= default` ile açıkça isteyerek bildirmeliyiz.

Derleyici bu işlevleri bizim için üretirken şu kurallara uyar. Tabloyu ezberlemek gerekmez; önemli sonuç sondaki satırdadır.

Biz ne yazdık?	Kopya işlemleri	Taşıma işlemleri
Hiçbiri	Üretilir	Üretilir
Yıkıcı ya da kopya işlemi	Üretilir (eski kod için, önerilmez)	**Üretilmez**; taşıma istekleri kopyaya düşer
Taşıma işlemlerinden biri	**Silinir**	Yazdığımız kadarı vardır

Yıkıcıyı yazıp taşımayı unutursak sınıf hâlâ derlenir ve çalışır; yalnızca her taşıma isteği sessizce kopyalamaya dönüşür. Taşıma işlemlerini yazıp kopyayı yazmazsak ise sınıf yalnızca taşınabilir olur (`unique_ptr` gibi).

İstediğimiz davranışı açıkça yazmanın yolu `= default` ve `= delete`'tir:

```
class Kaynak {
public:
    Kaynak() = default;
    ~Kaynak() = default;
    Kaynak(const Kaynak&) = delete;              // kopya yok
    Kaynak& operator=(const Kaynak&) = delete;
    Kaynak(Kaynak&&) noexcept = default;         // taşıma var
    Kaynak& operator=(Kaynak&&) noexcept = default;
};
```

Sıfır kuralı

Beş fonksiyonu elle yazmak uzun, hataya açık ve çoğu zaman gereksizdir. **Sıfır kuralı** der ki: kaynak yönetimini RAII sınıflarına (`vector`, `string`, `unique_ptr`) bırakırsak özel üye fonksiyonların hiçbirini yazmamız gerekmez. Derleyicinin ürettikleri üyeleri tek tek kopyalar ya da taşır.

▸ sifirKurali.cpp

```
#include <iostream>
#include <string>
#include <type_traits>
#include <utility>
#include <vector>

struct Kayit {
    std::string ad;
    std::vector<int> notlar;
};

static_assert(std::is_nothrow_move_constructible_v<Kayit>);
```

```cpp
static_assert(std::is_copy_constructible_v<Kayit>);

int main() {
    Kayit a{"Zeynep", {80, 90, 100}};
    Kayit b = a;                          // üyeler tek tek kopyalanır
    Kayit c = std::move(a);               // üyeler tek tek taşınır

    std::cout << b.ad << ' ' << b.notlar.size() << '\n';
    std::cout << c.ad << ' ' << c.notlar.size() << '\n';
    std::cout << "a.notlar.size() = " << a.notlar.size() << '\n';
    return 0;
}
```

```
ÇIKTI
Zeynep 3
Zeynep 3
a.notlar.size() = 0
```

`Kayit` içinde tek bir özel üye fonksiyon yok; buna rağmen kopyalanabilir ve `noexcept` taşınabilir (`static_assert`'ler bunu derleme zamanında sınadı). Bu, Bölüm 29'un "çıplak `new` yazmayın" önerisinin doğal sonucudur. `Tampon` örneği öğretici bir alıştırmaydı; gerçek kodda bu sınıfı `std::vector<int>` ile yazar ve beş işlevi derleyiciye bırakırdık.

> **◆ Kritik Nokta · Hangi kural?**
>
> Sahip olduğumuz kaynak varsa ve bunu standart bir RAII sınıfı karşılıyorsa **sıfır kuralı**. Standart sınıfın karşılamadığı bir kaynağı (özel bir tutamaç, bir C kütüphanesi nesnesi) sarmalıyorsak **beşler kuralı** ve çoğunlukla yalnızca taşınabilir bir sınıf.

Taşımayı doğru kullanmak

Taşımanın en yaygın kullanım yerleri şunlardır:

- **Fonksiyondan nesne döndürmek**: `return yerel;` yeter. Değerle döndürmek artık ucuzdur; Bölüm 25'te çıkış parametreleri yerine `struct` ya da `pair` döndürmeyi önermemizin bir gerekçesi de budur.
- **Sahipliği devretmek**: `std::move` ile `unique_ptr` (Bölüm 29), `thread` ve akış nesneleri taşınır.
- **Kapsayıcıya eklemek**: `liste.push_back(std::move(nesne));` nesnenin kopyalanmasını önler. Geçici bir nesneyi doğrudan eklerken `push_back(Nesne{...})` zaten taşır.
- **Parametreyi değerle alıp taşıyarak saklamak**: Bir sınıf bir `string` saklayacaksa yapıcısı `Kisi(std::string ad) : ad_{std::move(ad)} {}` biçiminde yazılabilir. Lvalue verilirse bir kopya, rvalue verilirse bir taşıma yapılır; tek fonksiyon iki durumu da karşılar.

Aynı değişkeni bir döngü içinde tekrar tekrar `std::move`'lamak ise hatadır; ilk taşımadan sonra değişken boştur.

39.5 auto ve decltype

`auto`'yu Bölüm 22'den beri kullanıyoruz: derleyici tipi ilk değerden çıkarır. Burada kuralını netleştirelim, çünkü `auto` bazı durumlarda beklenmedik bir **kopya** alır.

Tanım: auto ve decltype

`auto`, değişkenin tipini ilk değerinden çıkarır (type deduction); çıkarım sırasında üst düzey `const` ve referans **atılır**. `decltype(ifade)`, ifadenin tipini olduğu gibi, referans ve `const` dahil verir. `decltype(auto)` ise `auto` kadar kısa yazılır ama çıkarımı `decltype` kurallarıyla yapar.

Aşağıdaki program üç davranışı gösteriyor: `auto`'nun referansı atması, `auto&` ile referansın korunması ve `decltype(auto)` ile bir fonksiyonun döndürdüğü referansın aynen alınması.

autoDecltype.cpp

```cpp
#include <iostream>
#include <map>
#include <string>
#include <vector>

// dönüş tipi parametrelerden çıkarılır
template <typename A, typename B>
auto topla(const A& a, const B& b) -> decltype(a + b) {
    return a + b;
}

std::vector<std::string> isimler{"Ayşe", "Mehmet", "Elif"};

std::string& ilkIsim() { return isimler[0]; }

int main() {
    auto n = 42;                 // int
    auto d = 3.5;                // double
    auto s = std::string{"merhaba"};

    auto kopya = ilkIsim();      // auto referansı atar: kopya alınır
    kopya += "!";
    auto& ref = ilkIsim();       // referans kalır
    ref += "?";
    std::cout << isimler[0] << ' ' << kopya << '\n';

    decltype(ilkIsim()) r2 = isimler[1];   // std::string&
    r2 = "Can";
    decltype(auto) r3 = ilkIsim();         // referans korunur
    r3 = "Selin";
    std::cout << isimler[0] << ' ' << isimler[1] << '\n';

    std::map<std::string, int> yas{{"Ece", 21}, {"Burak", 25}};
    for (const auto& [ad, deger] : yas) {
        std::cout << ad << '=' << deger << ' ';
    }
    std::cout << '\n';

    std::cout << topla(n, d) << ' ' << topla(s, std::string{"!"}) << '\n';
    return 0;
}
```

```
ÇIKTI
Ayşe? Ayşe!
Selin Can
Burak=25 Ece=21
45.5 merhaba!
```

- `kopya`, `ilkIsim()` bir referans döndürmesine rağmen `std::string` tipindedir; `auto` referansı attı ve `"Ayşe"` kopyalandı. `kopya += "!"` bu yüzden `isimler`'e dokunmadı. `ref` ise gerçekten `isimler[0]`'ı gösterdiği için `"Ayşe?"` oldu.
- `r2` ve `r3` referans olduklarından atamalar `isimler`'i değiştirdi: önce `isimler[1]`, sonra `isimler[0]` yeni değerini aldı.
- `topla`'daki sondan dönüş tipi (`-> decltype(a + b)`) fonksiyonun dönüş tipinin parametrelere bağlı olduğu şablonlarda kullanılır. C++14'ten beri yalnızca `auto` yazmak da yeterlidir; `decltype` yazımını okuyabilmek için bilmemiz gerekir.
- Son döngüde `map`'in elemanlarını `const auto& [ad, deger]` ile açtık. Bu **yapısal bağlama**dır (structured binding, C++17); Bölüm 35'te `pair`, `tuple` ve `map` ile görmüştük. Aynı sözdizimi basit bir `struct` ile de çalışır: `auto [x, y] = nokta;`. Metinde `map`'in anahtar sırasıyla yazdığına, yani `Burak`'ın `Ece`'den önce geldiğine dikkat edelim.

> **▲ Sık Yapılan Hata · auto ile istenmeyen kopya**
>
> `for (auto s : metinler)` her turda bir `string` kopyalar. Okumak yeterliyse `const auto&`, değiştirmek gerekiyorsa `auto&` yazın. Bu, modern C++ kodunda en sık görülen gereksiz performans kaybıdır.

`auto`'yu ne zaman kullanmalı? Tip sağ taraftan zaten belliyse (`auto it = v.begin();`, `auto p = std::make_unique<Sinif>();`) ya da tip uzun ve gürültülüyse. Tamsayı ya da `double` gibi basit tiplerde ise tipi açıkça yazmak okunurluğu artırır; `auto n = 42;` yerine `int n{42};` demek çoğu zaman daha anlaşılırdır.

39.6 std::optional

Bölüm 28'de "değer olabilir ya da olmayabilir" durumunu `nullptr` döndüren işaretçilerle ifade etmiştik ve `std::optional`'ı bir nota bırakmıştık. `std::optional<T>` (C++17, `<optional>`) bu işi işaretçi olmadan, bellek ayırmadan yapar: ya bir `T` değeri taşır ya da boştur.

optionalVariant.cpp

```cpp
#include <iostream>
#include <optional>
#include <string>
#include <variant>
#include <vector>

std::optional<int> indeksBul(const std::vector<std::string>& v,
                             const std::string& aranan) {
    for (std::size_t i = 0; i < v.size(); ++i) {
        if (v[i] == aranan) {
            return static_cast<int>(i);
        }
    }
    return std::nullopt;
}

int main() {
```

```
    std::vector<std::string> gunler{"Pzt", "Sal", "Car"};

    auto a = indeksBul(gunler, "Sal");
    auto b = indeksBul(gunler, "Cum");

    if (a) {
        std::cout << "Sal bulundu: " << *a << '\n';
    }
    std::cout << "Cum: " << b.value_or(-1) << '\n';
    std::cout << std::boolalpha << b.has_value() << '\n';

    std::variant<int, double, std::string> v = 7;
    auto yaz = [](const auto& x) { std::cout << "deger: " << x << '\n'; };
    std::visit(yaz, v);
    v = 2.5;
    std::visit(yaz, v);
    v = std::string{"metin"};
    std::visit(yaz, v);
    std::cout << "indeks: " << v.index() << '\n';
    if (std::holds_alternative<std::string>(v)) {
        std::cout << std::get<std::string>(v).size() << '\n';
    }
    return 0;
}
```

```
ÇIKTI
Sal bulundu: 1
Cum: -1
false
deger: 7
deger: 2.5
deger: metin
indeks: 2
5
```

`indeksBul` bulunamadığında `-1` gibi özel bir değer ("sihirli sayı") uydurmak zorunda kalmıyor; `std::nullopt` döndürüyor. Çağıran taraf iki yoldan değeri alır:

- `if (a)` ya da `a.has_value()` ile doluluğu denetleyip `*a` ile değere ulaşmak,
- `b.value_or(-1)` ile boşsa yedek değer vermek.

Boş bir `optional`'ın değerini `*b` ile okumak tanımsız davranıştır (`nullptr`'ı dolaylamak gibi). `b.value()` ise boşsa `std::bad_optional_access` istisnası atar.

39.7 std::variant

`std::variant<int, double, std::string>`, bu üç tipten **yalnızca birini** tutan, tip güvenli bir birleşimdir. C'nin `union`'ını (Bölüm 14) anımsayalım: orada hangi alanın dolu olduğunu ayrı bir değişkende kendimiz tutardık. `variant` bunu kendi içinde tutar: `index()` etkin alternatifin sırasını (burada `string` için 2) verir, `holds_alternative<T>(v)` ile sorabilir, `get<T>(v)` ile okuyabiliriz. Yanlış tipi `get` ile istersek `std::bad_variant_access` fırlatılır.

`std::visit`, etkin alternatifin tipine göre bir çağrılabilir nesneyi çalıştırır. Yukarıda `const auto&` parametreli genel bir lambda kullandık; derleyici her alternatif için ayrı bir sürüm üretir (Bölüm 33'teki şablon mantığı). Alternatiflere göre farklı iş yapmak gerekirse lambda içinde `if constexpr` kullanılır; bunu `constexpr` kısmında göreceğiz.

> **Not · variant bu kitapta kısa tutuldu**
>
> `variant`, kendi başına bir bölümü doldurabilecek bir konudur. Burada yalnızca neyin mümkün olduğunu görmemiz yeterli: bir değer birkaç tipten biri olabiliyorsa `void*` ya da el yapımı `union` yerine `variant` kullanırız.

39.8 std::string_view

Bir fonksiyon metni yalnızca **okuyacaksa** parametresi ne olmalıdır? `const std::string&` yazarsak `const char*` ya da metin sabiti verildiğinde geçici bir `string` oluşur (bellek ayrılabilir, kopya yapılır). `const char*` yazarsak `string`'in uzunluk bilgisini kaybederiz. `std::string_view` (C++17, `<string_view>`) bu ikilemi çözer.

> **Tanım: string_view**
>
> Bir karakter dizisinin başlangıç adresini ve uzunluğunu tutan, metne **sahip olmayan** hafif bir görünümdür (non-owning view). Kopyalaması çok ucuzdur (iki sözcük) ve `std::string`, `const char*` ve metin sabitlerinden doğrudan oluşturulabilir.

stringView.cpp

```cpp
#include <iostream>
#include <string>
#include <string_view>

int boslukSay(std::string_view metin) {
    int n = 0;
    for (char c : metin) {
        if (c == ' ') {
            ++n;
        }
    }
    return n;
}

int main() {
    std::string s{"bir iki uc dort"};
    const char* c = "alfa beta";

    std::cout << boslukSay(s) << ' ' << boslukSay(c) << ' '
              << boslukSay("a b c") << '\n';

    std::string_view sv{s};
    std::string_view ilk = sv.substr(0, 3);     // kopya yok, aynı bellek
    sv.remove_prefix(4);
    std::cout << ilk << '|' << sv << '|' << sv.size() << '\n';

    s[0] = 'B';
    std::cout << ilk << '\n';                   // görünüm değişikliği yansıtır
    return 0;
}
```

```
ÇIKTI
3 1 2
bir|iki uc dort|11
Bir
```

`boslukSay` üç farklı kaynakla çağrıldı; hiçbirinde kopya yapılmadı. `substr` ve `remove_prefix` görünümün pencere sınırlarını oynatır, metni kopyalamaz. Son satır bunun kanıtıdır: `s[0]` değişince `ilk` görünümü de değişikliği yansıttı.

Bu özellik aynı zamanda tehlikelidir. `string_view` metne sahip olmadığı için metin yok edildiğinde görünüm **asılı kalır** (dangling):

```
std::string_view tehlikeli() {
    std::string yerel = "gecici metin";
    return yerel;              // yerel yok edilir, görünüm boşa gider
}
```

> ◆ **Kritik Nokta · string_view yalnızca parametre için**
>
> `string_view`'i genellikle fonksiyon **parametresi** olarak kullanırız; sınıf üyesi olarak saklamak ya da yerel metinden bir görünümü döndürmek, kaynağın ömrü konusunda dikkat gerektirir. Ayrıca `string_view` sonlandırıcı `'\0'` garantisi vermez; C fonksiyonlarına (`printf`, `fopen`) doğrudan `.data()` geçirmek güvenli değildir.

39.9 constexpr Fonksiyonlar

Bölüm 22'de `constexpr` sabitleri ve basit bir `constexpr` fonksiyonu görmüştük. Modern C++ bu özelliği her sürümde genişletti: C++11'de bir `constexpr` fonksiyon tek bir `return` ifadesiyle sınırlıydı; C++14'ten itibaren yerel değişkenler, döngüler ve `if` kullanabilir; C++17'de lambda'lar da, C++20'de ise `std::vector` ve `std::string` bile derleme zamanı hesaplarında kullanılabilir.

Bir `constexpr` fonksiyon, argümanları derleme zamanında bilinirse derleme sırasında, bilinmezse çalışma zamanında hesaplanır. İkisinde de aynı kod çalışır.

▸ constexprEnum.cpp

```
#include <array>
#include <iostream>
#include <type_traits>

constexpr int faktoriyel(int n) {
    int sonuc = 1;
    for (int i = 2; i <= n; ++i) {
        sonuc *= i;
    }
    return sonuc;
}

static_assert(faktoriyel(5) == 120);

constexpr std::array<int, 5> kareler() {
    std::array<int, 5> a{};
    for (int i = 0; i < 5; ++i) {
        a[i] = i * i;
    }
```

```
    return a;
}

enum class Renk { Kirmizi, Yesil, Mavi };
enum class Seviye : unsigned char { Dusuk = 1, Orta = 5, Yuksek = 10 };

const char* ad(Renk r) {
    switch (r) {
        case Renk::Kirmizi: return "kirmizi";
        case Renk::Yesil:   return "yesil";
        case Renk::Mavi:    return "mavi";
    }
    return "?";
}

template <typename T>
auto metne(T x) {
    if constexpr (std::is_integral_v<T>) {
        return x * 2;
    } else {
        return x;
    }
}

int main() {
    constexpr int f = faktoriyel(6);
    constexpr auto k = kareler();
    int dizi[faktoriyel(3)] = {};          // sabit ifade olduğu için geçerli

    std::cout << f << ' ' << k[4] << ' ' << sizeof(dizi) / sizeof(int) << '\n';
    std::cout << ad(Renk::Yesil) << '\n';
    std::cout << static_cast<int>(Seviye::Orta) << ' '
              << sizeof(Seviye) << '\n';
    std::cout << metne(21) << ' ' << metne(1.5) << '\n';
    return 0;
}
```

```
ÇIKTI
720 16 6
yesil
5 1
42 1.5
```

Program `enum class` örneklerini de içeriyor; onları sonraki başlıkta ele alacağız.

- `faktoriyel(5) == 120` denetimi `static_assert` ile **derleme sırasında** yapıldı; yanlış olsaydı program derlenmezdi. `constexpr int f` ve `constexpr auto k` değerleri de derleme sırasında hesaplandı. `faktoriyel(3)` bir sabit ifade olduğundan dizi boyutu olarak kullanılabildi (Bölüm 22'deki `ALAN` örneğinin aynısı).
- `kareler()` bütün bir `std::array`'i derleme zamanında dolduruyor. Çalışma zamanı yalnızca hazır tabloyu okur: bu, arama tabloları ve sabit veriler için güçlü bir yöntemdir.
- `metne` fonksiyonundaki `if constexpr` (C++17) koşulu derleme zamanında değerlendirir ve seçilmeyen dalı o şablon sürümü için hiç derlemez. Düz bir `if` seçilmeyen dalı da derlemek zorunda kalırdı. Bu örnekte iki dal da her tip için geçerli olduğundan fark görünmez; ama dalın birinde `x.size()` gibi yalnızca bazı tiplerde geçerli bir ifade olsaydı, yalnızca `if constexpr` şablonun derlenmesini sağlardı.

> **İpucu · const mı, constexpr mı?**
> Değer derleme zamanında biliniyorsa ve sabit ifade olarak gerekiyorsa `constexpr`; yalnızca "sonradan değişmesin" demek istiyorsak `const`. Yapabildiğimiz her yerde `constexpr` kullanmak hem derleme zamanı denetimi hem de hız kazandırır.

39.10 enum class

C'den devralınan düz `enum`'un (Bölüm 14) iki sorunu vardır. Birincisi, sabitler çevreleyen kapsama sızar: `Kirmizi` doğrudan yazılabildiği için iki farklı `enum` aynı adlı sabiti taşıyamaz. İkincisi, değerler sessizce `int`'e dönüşür; bu yüzden bir rengi bir trafik ışığı değeriyle karşılaştırmak hatasız derlenir.

`enum class` (kapsamlı numaralandırma, scoped enum, C++11) bu sorunları çözer:

- Sabitler enum'un kendi kapsamındadır: her zaman `Renk::Yesil` yazılır.
- Başka bir tipe örtük dönüşüm yoktur; sayıya çevirmek için `static_cast<int>(...)` yazılır.
- Temel tip belirtilebilir: `enum class Seviye : unsigned char` yukarıda 1 bayt yer kapladı (`sizeof(Seviye)` çıktısı `1`).

Yasak dönüşümleri derleyicinin nasıl yakaladığına bakalım:

enumHata.cpp

```
enum class Renk { Kirmizi, Yesil };

int main() {
    int x = Renk::Yesil;
    Renk r = 1;
    return x + static_cast<int>(r);
}
```

DERLEYİCİ ÇIKTISI

```
enumHata.cpp: In function 'int main()':
enumHata.cpp:4:19: error: cannot convert 'Renk' to 'int' in initialization
    4 |     int x = Renk::Yesil;
      |             ~~~~~~^~~~
enumHata.cpp:5:14: error: cannot convert 'int' to 'Renk' in initialization
    5 |     Renk r = 1;
      |              ^
```

Her iki yönde de dönüşüm açıkça istenmek zorundadır. Bu, yanlış değer atamalarını derleme aşamasında yakalar. `switch` ile kullanırken `case` etiketlerinin tam adla (`case Renk::Yesil:`) yazıldığına dikkat edelim. Yeni kodda varsayılan tercihimiz `enum class` olmalıdır.

39.11 C++20'ye Bakış

C++20, C++11'den bu yana dile eklenen en büyük sürümdür. Bu kitabın örnekleri C++17 ile derlendiği için burada yalnızca dört büyük yeniliği tanıtıyoruz ve iki kısa program gösteriyoruz. Bu iki programın başında `` `notu bulunur; derlemek için g++ -std=c++20` `` gerekir. Bölüm 21'de belirttiğimiz gibi C++20 özelliklerinin desteği derleyiciden derleyiciye değişir; örnekler GCC 13 ile denenmiştir.

Kavramlar (concepts)

Bölüm 33'te şablonların bir eksiğini görmüştük: şablon parametresine yanlış tip verilirse hata, şablonun derinliklerinde, uzun ve anlaşılmaz bir mesajla ortaya çıkıyordu. **Kavramlar**, şablon parametresinin sağlaması gereken koşulu şablonun başında bildirmemizi sağlar.

> **Tanım: Kavram (concept)**
>
> Bir tipin sağlaması gereken koşulları adlandıran, derleme zamanında `true` ya da `false` veren bir şablon yüklemidir (C++20). `concept` anahtar sözcüğüyle tanımlanır; `requires` ifadesiyle bir tipte hangi işlemlerin geçerli olması gerektiği yazılır.

kavramlar.cpp

```cpp
#include <concepts>
#include <iostream>
#include <string>

template <typename T>
concept Sayisal = std::integral<T> || std::floating_point<T>;

template <Sayisal T>
T ikiKati(T x) {
    return x * 2;
}

template <typename T>
concept Yazilabilir = requires(T x) {
    { x.yaz() } -> std::same_as<void>;
};

struct Rapor {
    void yaz() const {}
};

template <Yazilabilir T>
void yazdir(const T& t) {
    t.yaz();
    std::cout << "yazildi\n";
}

int main() {
    std::cout << ikiKati(21) << ' ' << ikiKati(1.25) << '\n';
    yazdir(Rapor{});
    return 0;
}
```

```
ÇIKTI
42 2.5
yazildi
```

`Sayisal` kavramı `<concepts>` başlığındaki hazır kavramlardan (`std::integral`, `std::floating_point`) kuruldu. `template <Sayisal T>` yazımı "T, Sayisal olmalıdır" demektir. `Yazilabilir` ise `requires` ifadesiyle "tipin değeri üzerinde `x.yaz()` çağrılabilmeli ve `void` dönmeli" koşulunu ifade eder.

Sayısal olmayan bir tiple çağırırsak koşul sağlanmadığı için derleyici sorunu çağrı noktasında bildirir:

kavramHata.cpp

```cpp
#include <concepts>
#include <string>

template <typename T>
concept Sayisal = std::integral<T> || std::floating_point<T>;

template <Sayisal T>
T ikiKati(T x) { return x * 2; }

int main() {
    ikiKati(std::string{"a"});
}
```

GCC'nin mesajından (kısaltılmış) en önemli satırlar şunlardır:

DERLEYİCİ ÇIKTISI

```
kavramHata.cpp:11:12: error: no matching function for call to 'ikiKati(std::string)'
   11 |     ikiKati(std::string{"a"});
      |     ~~~~~~~^~~~~~~~~~~~~~~~~~
kavramHata.cpp:8:3: note: candidate: 'template<class T>  requires  Sayisal<T> T ikiKati(T)'
kavramHata.cpp:8:3: note: constraints not satisfied
kavramHata.cpp:5:36: note: no operand of the disjunction is satisfied
```

Mesaj artık "şablon içinde `*` bulunamadı" demiyor; doğrudan "`Sayisal` koşulu karşılanmadı" diyor. Bu, kavramların asıl kazancıdır.

Aralıklar (ranges) ve std::format

Aralıklar kitaplığı (`<ranges>`), Bölüm 36'daki algoritmaları kapsayıcının tamamına uygulanabilir ve borular gibi birbirine bağlanabilir hâle getirir. `begin()` ve `end()` yazmak gerekmez; ara sonuçlar için geçici kapsayıcı da oluşmaz, çünkü görünümler (`views`) elemanları **tembel** (lazy) olarak, döngüde istenince hesaplar.

C++20 ayrıca `std::format` ile `printf`'in kısalığını tip güvenliğiyle birleştirir. Bölüm 23'te biçimlendirme alternatifi olarak görmüştük; burada aynı örnekte kullanalım:

aralikFormat.cpp

```cpp
#include <format>
#include <iostream>
#include <ranges>
#include <string>
#include <vector>

int main() {
    std::vector<int> sayilar{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    auto sonuc = sayilar
        | std::views::filter([](int x) { return x % 2 == 0; })
        | std::views::transform([](int x) { return x * x; });

    for (int x : sonuc) {
        std::cout << x << ' ';
    }
    std::cout << '\n';

    std::string satir = std::format("{:<8}{:>3} x {:>7.2f}", "Cay", 3, 12.5);
    std::cout << satir << '|' << '\n';
```

```
    std::cout << std::format("{} / {} = {:.3f}\n", 10, 3, 10.0 / 3);
    return 0;
}
```

```
ÇIKTI
4 16 36 64 100
Cay       3 x   12.50|
10 / 3 = 3.333
```

`|` işleci görünümleri soldan sağa zincirler: önce çift sayılar süzülür, sonra kareleri alınır. Hiçbir ara `vector` oluşmadı; `for` her turda bir eleman çeker. `std::format` ise `{}` yer tutucularını kullanır; `:<8` sola yaslı 8 genişlik, `:>7.2f` sağa yaslı 7 genişlikte iki ondalık anlamına gelir. Tipler derleme zamanında denetlendiği için `printf`'teki `%d` / `%f` uyuşmazlığı hataları ortadan kalkar.

Modüller

Bölüm 21'de C++'ın `#include` ile çalışan başlık dosyası modelini anlatmıştık: önişlemci başlığı metin olarak kopyalar ve aynı başlık her `.cpp` dosyasında yeniden derlenir. **Modüller** (modules, C++20) bu modelin yerine geçmeyi hedefler. Bir modül bir kez derlenir ve derleyicinin ikili bir gösterimi olarak saklanır; kullanan dosya onu `import` ile alır. Başlıklardaki makro sızıntıları ve derleme süresi sorunları böylece azalır.

```
// matematik.cppm
export module matematik;

export int kare(int x) { return x * x; }

// anaProgram.cpp
import matematik;
import <iostream>;

int main() { std::cout << kare(5) << '\n'; }
```

Bu parça tek başına derlenmez: modül dosyaları derleyiciye ve yapı sistemine göre farklı bayraklar, uzantılar ve derleme sırası gerektirir. Bu yüzden modülleri kitabın örneklerinde kullanmıyoruz; kavramı tanımak yeterlidir. Ekosistem hâlâ olgunlaşmaktadır ve büyük projelerde kullanmadan önce derleyici ve yapı sistemi desteğini denetlemek gerekir.

> **Not · C++20'den sonra**
>
> C++23 bu çizgiyi sürdürdü (örneğin `std::print`, `std::expected`, `std::flat_map`). Bu kitap C++17 tabanlı kaldığı için yalnızca yönü gösteriyoruz: dil her sürümde daha güvenli ve daha anlatımlı kod yazmaya doğru ilerliyor.

39.12 Modern C++ Yazım Önerileri

Bu kısmı önceki bölümlerden de topladığımız bir başvuru listesi olarak kullanabiliriz. Hepsinin gerekçesi kitapta anlatıldı; burada tek yerde topluyoruz.

Alan	Eski alışkanlık	Modern öneri
Bellek	`new` / `delete`, `malloc`	`std::make_unique`, `std::make_shared`; kapsayıcılar
Diziler	`T dizi[N]`, `T*` + boyut	`std::array`, `std::vector`

Alan	Eski alışkanlık	Modern öneri
Metin	`char*`, `strcpy`	`std::string`; salt okuma için `std::string_view`
Boş gösterici	`NULL`, `0`	`nullptr`; "değer olmayabilir" için `std::optional`
Başlatma	`int x = 5;`, belirsiz değişken	`int x{5};` ve her zaman ilk değerle
Tipler	Yinelenen uzun tip adları	`auto`, ama basit tiplerde açık tip
Sabitler	`#define`	`constexpr`, `const`
Sabit gruplar	düz `enum`	`enum class`
Döngüler	indeksli `for`	aralık tabanlı `for`, algoritmalar
Kopya	Değer parametresi her yerde	okuma için `const T&`, sahiplik için değer + `std::move`
Özel üyeler	Beşini elle yazmak	Sıfır kuralı; gerekirse `= default`, `= delete`
Çok biçimlilik	Çıplak işaretçi dizileri	`vector<unique_ptr<Taban>>`
Sanal işlevler	`virtual`	`override`, `final` yazmak
Hata yönetimi	Dönüş kodu, `errno`	İstisna ve RAII; hata beklenen yerde `optional`

Birkaç ek öneri:

- **Derleyici uyarılarını açık tutun**: `-Wall -Wextra` ile derleyip uyarı bırakmayın.
- **`const` doğruluğu**: değişmeyen her şeyi `const` yapın (Bölüm 22).
- **Gereksiz kopyayı yakalayın**: `for (auto s : v)` yerine `for (const auto& s : v)`.
- **Ham kaynak sahipliğinden kaçının**: kaynağı bir nesneye sarın (RAII).
- **Taşıma ile kopya arasında bilinçli seçim**: `return yerel;` yazın, `std::move`'u yalnızca sahiplik gerçekten devrediliyorsa yazın.
- **Yeni özelliklere hız kesmeden koşmayın**: Takımın ve derleyicinin desteklediği standart neyse onu kullanın; yeni bir özellik ancak okunurluğu artırıyorsa değerlidir.

39.13 Üç Dilde: Aynı Sorunlar, Farklı Çözümler

Taşıma semantiğinin var olmasının sebebi, C++'ın nesneleri **değer olarak** tutması ve bellek sahipliğini programcıya bırakmasıdır. C#'ın tasarımı bu sorunların çoğunu baştan başka yoldan çözer.

> **Üç Dilde · Kopya, sahiplik ve "değer yok" durumu** *C · C++ · C#*
>
> **C**: Taşıma kavramı yoktur. Büyük bir `struct`'ı değerle geçirirsek tamamı kopyalanır; bu yüzden işaretçi geçirir ve sahipliği yorum satırlarıyla, bir sözleşme olarak belgeleriz (kimin `free` edeceği). "Değer yok" için `NULL` ya da `-1` gibi sihirli değerler kullanılır.
>
> **C++**: Nesne bir değerdir; kopyalama ve taşıma açıkça ayrı işlemlerdir. Kaynağın sahipliği tip sisteminde ifade edilir (`unique_ptr`, taşınabilir ama kopyalanamaz sınıflar). "Değer yok" için `std::optional`, "birkaç tipten biri" için `std::variant`.
>
> **C#**: Sınıflar **referans tipidir**; `b = a` yalnızca referansı kopyalar, nesne kopyalanmaz. Bellek çöp toplayıcıdadır (GC), bu yüzden "kim silecek" sorusu yoktur ve taşıma yapıcısına ihtiyaç doğmaz. `struct` ve `enum` gibi değer tipleri kopyalanır ama çoğunlukla küçüktür. Kaynak (dosya, bağlantı) yönetimi için `IDisposable` ve `using` kullanılır; RAII'nin karşılığı budur. "Değer yok" için `null` ve `Nullable<T>` (`int?`), referans tiplerde derleyici denetimli `string?`; birleşim için sınıf kalıtımı ve desen eşleme.

Konu	C	C++	C#
Nesneyi başka yere vermek	İşaretçi ya da `struct` kopyası	Kopya, taşıma ya da referans	Referans kopyası (sınıf)
Sahiplik	Programcı disiplini	`unique_ptr`, `shared_ptr`, RAII	Çöp toplayıcı; kaynaklar için `using`
`var` / `auto`	Yok (C23'te sınırlı `auto`)	`auto`, `decltype`	`var`
Derleme zamanı hesap	Makro, `const` sınırlı	`constexpr`	`const` (yalnızca sabitler)
Güvenli enum	Yok	`enum class`	`enum` (sayıya örtük dönüşmez)
Değer olmayabilir	`NULL` /sihirli değer	`std::optional`	`T?` / `null`
Metin görünümü	`const char*` + uzunluk	`std::string_view`	`ReadOnlySpan<char>`
Tip kısıtı	Yok	`concept` / `requires`	`where T : ...`

C# bu bölümdeki kavramların çoğunu farklı mekanizmalarla ama benzer amaçla sunar. Örneğin `Span<T>` ve `ReadOnlySpan<T>`, `string_view`'in kardeşidir ve aynı "sahip olmayan görünüm" mantığını taşır. C# kısmında bu konulara değineceğiz.

Bölüm Özeti

- Kopya derin ve pahalı olabilir; taşıma, kaynağın sahipliğini işaretçi devriyle aktarır ve kaynak nesneyi geçerli ama belirsiz bir durumda bırakır.
- lvalue adı olan, rvalue geçici ifadedir. `T&&` rvalue'ya bağlanır; taşıma yapıcısı ve taşıma ataması bu referansı alır.
- `std::move` hiçbir şeyi taşımaz; ifadeyi rvalue'ya dönüştürür. Taşımayı taşıma işlemi gerçekleştirir.
- `const` bir nesneyi `std::move` etmek kopyaya döner. Taşınmış nesneye yalnızca yeniden atama yapılmalı ya da nesne yok edilmelidir.
- Taşıma yapıcısı kaynağı devralır, kaynak nesneyi boş duruma getirir; taşıma işlemleri `noexcept` olmalıdır, aksi hâlde `vector` büyürken kopyalamaya döner.
- `return yerel;` yazılır; `return std::move(yerel);` kopya elemeyi engelleyebilir. Geçici döndürmede C++17'den itibaren eleme zorunludur.
- Beşler kuralı: yıkıcı, kopya yapıcı, kopya atama, taşıma yapıcı, taşıma atama birlikte düşünülür. Sıfır kuralı: kaynakları `vector`, `string`, `unique_ptr` gibi RAII sınıflarına bırakıp hiçbirini yazmamak.
- `auto` referansı ve üst düzey `const`'u atar; `auto&`, `const auto&` ve `decltype(auto)` ile korunur. `decltype(ifade)` tipi aynen verir.
- `std::optional` "değer olabilir ya da olmayabilir" durumunu, `std::variant` "birkaç tipten biri" durumunu tip güvenli ifade eder; boş `optional`'ı dolaylamak tanımsız davranıştır.
- `std::string_view` metne sahip olmayan hafif bir görünümdür; parametre olarak idealdir, ama kaynağın ömrü bitince asılı kalır.
- `constexpr` fonksiyonlar derleme zamanında çalışabilir; `static_assert` ile denetim, `if constexpr` ile derleme zamanı dallanma yapılır.
- `enum class` sabitleri kendi kapsamına alır ve örtük `int` dönüşümünü yasaklar.

- C++20: kavramlar şablon koşullarını açıkça yazdırır ve hata mesajlarını netleştirir; aralıklar algoritmaları borulayarak tembel zincirler; `std::format` tip güvenli biçimlendirme sunar; modüller `#include` modelinin yerini almaya aday ama ekosistem olgunlaşmaktadır.
- C# aynı sorunları referans tipleri, çöp toplayıcı, `using` / `IDisposable` ve `Span<T>` ile çözer.

Alıştırmalar

1. `std::move(x)` ifadesi tam olarak ne yapar?
   - **A)** `x`'in bellek bloğunu yeni bir yere kopyalar
   - **B)** `x`'i, taşımaya izin veren bir rvalue ifadesine dönüştürür
   - **C)** `x`'in yıkıcısını çağırır
   - **D)** `x`'i `nullptr` yapar
   - **E)** `x`'i `const` yapar
2. `const std::string s = "metin"; std::string t = std::move(s);` satırı için hangisi doğrudur?
   - **A)** Derleme hatası verir
   - **B)** `s` taşınır, `t` metni alır
   - **C)** Taşıma yapıcısı seçilir ve `s` boşalır
   - **D)** Kopya yapıcısı çalışır, `s` değişmez
   - **E)** Tanımsız davranıştır
3. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>
#include <utility>

struct S {
    S() { std::cout << "v "; }
    S(const S&) { std::cout << "k "; }
    S(S&&) noexcept { std::cout << "t "; }
};

int main() {
    S a;
    S b = a;
    S c = std::move(a);
    S d = S{};
    std::cout << '\n';
}
```

4. Beşler kuralındaki beş özel üye fonksiyon hangi seçenekte doğru verilmiştir?
   - **A)** Yapıcı, yıkıcı, kopya yapıcı, kopya atama, `swap`
   - **B)** Yıkıcı, kopya yapıcı, kopya atama, taşıma yapıcı, taşıma atama
   - **C)** Yapıcı, yıkıcı, `new`, `delete`, `operator=`
   - **D)** Kopya yapıcı, taşıma yapıcı, `get`, `set`, yıkıcı
   - **E)** Varsayılan yapıcı, parametreli yapıcı, kopya, taşıma, yıkıcı
5. Aşağıdaki fonksiyonda performans açısından ne yanlıştır ve nasıl düzeltilir?

```
std::vector<int> hazirla() {
    std::vector<int> v(1000, 1);
    return std::move(v);
}
```

6. Bir sınıfın taşıma yapıcısında `noexcept` yazmayı unutursak hangi sonuç ortaya çıkabilir?

   A) Sınıf derlenmez
   B) Taşıma yapıcısı hiç çağrılmaz, program çöker
   C) `vector` büyürken elemanları taşımak yerine kopyalayabilir
   D) İstisnalar yutulur
   E) Hiçbir fark yoktur

7. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>

int main() {
    int x = 5;
    auto y = x;
    auto& z = x;
    y++;
    z++;
    std::cout << x << ' ' << y << ' ' << z << '\n';
}
```

8. `std::optional<int> o;` tanımlandıktan sonra hangisi tanımsız davranıştır?

   A) `o.has_value()`
   B) `o.value_or(0)`
   C) `*o`
   D) `o = 3;`
   E) `if (o) {}`

9. Aşağıdaki kodda hata nedir?

```
std::string_view sv = std::string("gecici metin");
std::cout << sv << '\n';
```

10. Aşağıdaki programın çıktısı nedir?

```
#include <iostream>
#include <string>
#include <variant>

int main() {
    std::variant<int, std::string> v = 3;
    v = std::string("ab");
    std::cout << v.index() << std::get<1>(v).size() << '\n';
}
```

11. `enum class Renk { Kirmizi, Yesil };` verildiğine göre hangi satır derlenir?

    A) `int a = Renk::Yesil;`
    B) `Renk r = 1;`
    C) `Renk r = Yesil;`

D) `int a = static_cast<int>(Renk::Yesil);`

E) `if (Renk::Yesil == 1) {}`

12. Aşağıdaki programın çıktısı nedir?

```cpp
#include <iostream>

constexpr int f(int n) { return n <= 1 ? 1 : n * f(n - 1); }

int main() {
    constexpr int a = f(4);
    std::cout << a + f(3) << '\n';
}
```

13. Aşağıdakilerden hangisi C++20 ile gelen bir özellik **değildir**?

    A) Kavramlar (`concept`)

    B) Aralıklar (`<ranges>`)

    C) `std::format`

    D) `std::optional`

    E) Modüller

14. Bir `int` dizisini `new int[n]` ile tutan `Kutu` sınıfı yazın. Kopya yapıcısı, kopya ataması, taşıma yapıcısı, taşıma ataması ve yıkıcısı bulunsun. `main` içinde bir `Kutu`'yu taşıyarak ikinci bir `Kutu` oluşturun ve ilkinin boyutunun 0 olduğunu gösterin.

15. `std::optional<double> bol(double a, double b)` fonksiyonunu yazın: `b` sıfırsa `std::nullopt`, değilse bölümü döndürsün. `main` içinde `10 / 4` ve `1 / 0` için sonucu yazdırın (boşsa "tanimsiz" yazsın).

16. `enum class Gun { Pzt, Sal, Car }` tanımlayın. Bir `Gun` alıp `const char*` ya da `std::string` döndüren `adi` fonksiyonunu `switch` ile yazın. Üç günü de yazdıran bir program oluşturun. (İsteğe bağlı, C++20): `Sayisal` adlı bir kavram tanımlayıp en büyük iki sayıyı döndüren şablon yazın.

BÖLÜM 40

C++ Uygulama Projesi: Kütüphane Yönetim Sistemi

BU BÖLÜMDE

- Bir uygulamanın gereksinimlerinden sınıf tasarımına geçmeyi
- Ortak özellikleri taban sınıfta, farklı davranışı türemiş sınıflarda toplayan bir sınıf hiyerarşisi kurmayı
- `std::map` ve `std::unique_ptr` ile çok biçimli nesneleri güvenle saklamayı
- Hata durumlarını kendi istisna sınıfımızla bildirmeyi ve tek yerde yakalamayı
- Nesneleri metin dosyasına yazıp doğrulayarak geri yüklemeyi
- Çalışan bir programı kendi gereksinimlerinize göre genişletmeyi

Bölüm 20'de C ile bir öğrenci kayıt sistemi yazmıştık: yapılar, elle yönetilen dinamik dizi ve `fwrite` kullanmıştık. Bu bölümde C++ kısmında öğrendiklerimizi aynı türden bir uygulamada birleştireceğiz: sınıflar (Bölüm 30), kalıtım ve sanal fonksiyonlar (Bölüm 31), akıllı işaretçiler (Bölüm 29), `map` (Bölüm 35), istisnalar (Bölüm 34), dosya akışları (Bölüm 37) ve akışla biçimli çıktı (Bölüm 23).

Konu bir **kütüphane yönetim sistemidir**: kitap ve dergi kaydı tutan, üye ekleyen, yayınları ödünç veren ve iade alan, her şeyi dosyada saklayan bir konsol uygulaması. Yeni bir C++ özelliği öğrenmeyeceğiz. Amaç, bilinen parçaların bir arada nasıl düzenlendiğini görmektir. Bölüm 20'yi hatırlayan okur, C sürümünde elle yazdığımız işlerin (büyüyen dizi, bellek bırakma, hata kodları) burada dilin kendi araçlarına devredildiğini fark edecektir.

40.1 Gereksinimler

Önce programın yapacaklarını yazalım:

1. Kütüphanede iki tür yayın bulunur: kitap (başlık, yazar, sayfa sayısı) ve dergi (başlık, sayı numarası). Her yayının sistemin verdiği bir numarası vardır.
2. Üyelerin numarası ve adı tutulur.
3. Bir yayın aynı anda yalnızca bir üyede olabilir; bir üye en fazla üç yayın ödünç alabilir.
4. Menüden şu işlemler yapılır: kitap ekle, dergi ekle, üye ekle, yayınları listele, ödünç ver, iade al, üye durumu, dosyaya kaydet, dosyadan yükle, çıkış.
5. Hatalı girdi ve kural ihlalleri (olmayan numara, ödünçteki yayını tekrar ödünç vermek) programı çökertmeyecek; kullanıcıya açıklayıcı bir mesaj gösterilecek.
6. Veriler okunabilir bir metin dosyasında saklanacak; bozuk dosya, eldeki verileri bozmayacak.

40.2 Tasarım

Sınıflar

Gereksinimlerdeki isimlerden sınıf adaylarını çıkarırız: yayın, kitap, dergi, üye, kütüphane. Kitap ile dergi ortak özellikler taşır (numara, başlık, kimde olduğu) ama kendilerine özgü alanları ve ekranda gösterilme biçimleri vardır. Bu, "bir kitap bir yayındır" ilişkisini, yani kalıtımı işaret eder:

```
            Yayin  (soyut: tur, ayrinti, kayitSatiri)
           /     \
       Kitap     Dergi

   Uye  (düz veri: no, ad)

   Kutuphane  ---- map<int, unique_ptr<Yayin>>
              ---- map<int, Uye>
```

`Yayin` soyut bir sınıftır; çünkü "genel bir yayın" nesnesi oluşturmanın anlamı yoktur. Saf sanal fonksiyonlar (Bölüm 31) her türemiş sınıfı, kendi türünü, ayrıntı metnini ve dosya satırını üretmeye zorlar. `Uye` yalnızca iki alanı olan, davranışı bulunmayan bir veri taşıyıcısı olduğundan basit bir `struct` olarak kalır (Bölüm 30).

`Kutuphane` sınıfı bütün koleksiyonların sahibidir ve kuralları denetler. Menü ve ekran işleri ise sınıfın dışındaki küçük fonksiyonlarda kalır. Bu ayrım, Bölüm 20'deki katmanlı yapının aynısıdır: iş kuralları aşağıda, kullanıcı ile konuşan kod yukarıdadır.

Saklama kapları

Yayınlar kitap ya da dergi olabileceğinden kapta taban sınıf türünde işaretçi tutarız. Çıplak `Yayin*` yerine `std::unique_ptr<Yayin>` kullanırız: kap yok olurken nesneler kendiliğinden silinir, `delete` yazmamız gerekmez (Bölüm 29). Anahtarı yayın numarası olan bir `std::map`, hem numaraya göre hızlı arama sağlar hem de listelemede yayınları numara sırasıyla verir (Bölüm 35).

> **▲ Sık Yapılan Hata · Çok biçimli nesneleri değerle saklamayın**
>
> `vector<Yayin>` ya da `map<int, Yayin>` gibi bir kap, türemiş nesneyi taban sınıfın boyutuna kırpar (nesne dilimleme, Bölüm 31) ve soyut sınıf zaten değer olarak yaratılamaz. Taban türünde akıllı işaretçi tutmak, sanal çağrıların doğru türemiş sınıfa ulaşmasını sağlar.

> **Üç Dilde · Aynı tasarım üç dilde** *C · C++ · C#*
>
> C'de (Bölüm 20) tek bir `struct` ve elle büyüyen dizi yeterliydi; iki yayın türü için `union` ya da tür alanı gerekirdi. C++'ta kalıtım ve `unique_ptr` bunu doğal hâle getirir. C#'ta aynı tasarım `abstract class Yayin` ile `Dictionary<int, Yayin>` olarak yazılır; referans tipleri olduğu için dilimleme sorunu yoktur ve bellek çöp toplayıcıya aittir.

40.3 Hata Sınıfı ve Yardımcılar

Kural ihlallerini dönüş koduyla değil istisnayla bildireceğiz (Bölüm 34). Bölüm 20'de her fonksiyon başarıyı `1` / `0` ile bildirir, mesajı ise çağıran yazardı. Burada iş kuralını bozan kod, nedeni mesaj olarak taşıyan bir istisna

fırlatır; hiçbir ara katman kodu bunu taşımak zorunda kalmaz ve mesajı yalnızca `main` içindeki tek bir `catch` ekrana yazar:

```
class KutuphaneHatasi : public runtime_error {
public:
    explicit KutuphaneHatasi(const string& mesaj) : runtime_error(mesaj) {}
};
```

İki küçük yardımcı daha gerekir. İlki, satırı bir ayraç karakterinden parçalara ayırır (Bölüm 37'deki CSV ayrıştırmasının aynısı):

```
vector<string> bol(const string& s, char ayrac) {
    vector<string> parcalar;
    stringstream ss(s);
    string parca;
    while (getline(ss, parca, ayrac)) {
        parcalar.push_back(parca);
    }
    return parcalar;
}
```

İkincisi, metni tamsayıya çevirir. `stoi` harfle başlayan metinde `invalid_argument`, taşan sayıda `out_of_range` fırlatır; ikisi de `logic_error`'dan türer. `"12abc"` gibi yarım sayıları da reddetmek için `stoi`'nin ikinci parametresiyle işlenen karakter sayısına bakarız. Her durumda kendi hata sınıfımızı fırlatırız; böylece programın başka yerlerinde yalnızca tek tür hata kalır:

```
int sayiyaCevir(const string& s) {
    try {
        size_t konum = 0;
        int deger = stoi(s, &konum);
        if (konum != s.size()) {
            throw invalid_argument(s);
        }
        return deger;
    } catch (const logic_error&) {
        throw KutuphaneHatasi("Sayı bekleniyordu: '" + s + "'");
    }
}
```

40.4 Yayın Hiyerarşisi

`Yayin` sınıfı ortak veriyi (numara, başlık, ödünç alan üye) ve bu veriye erişimi barındırır. Ödünç durumunu ayrı bir bayrakla değil, tek bir sayıyla gösteririz: `uye_` değeri `0` ise yayın rafta, değilse o numaralı üyededir. İki ayrı alan birbiriyle çelişebilirdi; tek alan bu olasılığı ortadan kaldırır.

```
class Yayin {
public:
    Yayin(int id, const string& baslik) : id_(id), baslik_(baslik) {}
    virtual ~Yayin() = default;

    int id() const { return id_; }
    const string& baslik() const { return baslik_; }
    int oduncUye() const { return uye_; }
    bool oduncte() const { return uye_ != 0; }
    void uyeAyarla(int uyeNo) { uye_ = uyeNo; }
```

```
    virtual string tur() const = 0;
    virtual string ayrinti() const = 0;
    virtual string kayitSatiri() const = 0;

protected:
    string ortakAlanlar() const {
        return to_string(id_) + ";" + baslik_;
    }

private:
    int id_;
    string baslik_;
    int uye_ = 0;               // 0: rafta, değilse ödünç alan üyenin no'su
};
```

Sanal yıkıcı şarttır: nesneleri `unique_ptr<Yayin>` üzerinden sileceğimiz için yıkıcı sanal olmazsa türemiş sınıfın yıkıcısı çağrılmaz (Bölüm 31). `ortakAlanlar` yalnızca türemiş sınıfların kullanacağı yardımcı olduğundan `protected` bölümdedir.

Türemiş sınıflar yalnızca kendilerine özgü alanı ve üç sanal fonksiyonu tanımlar:

```
class Kitap : public Yayin {
public:
    Kitap(int id, const string& baslik, const string& yazar, int sayfa)
        : Yayin(id, baslik), yazar_(yazar), sayfa_(sayfa) {}

    string tur() const override { return "Kitap"; }
    string ayrinti() const override {
        return yazar_ + ", " + to_string(sayfa_) + " sayfa";
    }
    string kayitSatiri() const override {
        return "K;" + ortakAlanlar() + ";" + yazar_ + ";" +
               to_string(sayfa_) + ";" + to_string(oduncUye());
    }

private:
    string yazar_;
    int sayfa_;
};
```

`Dergi` aynı kalıbı izler; `yazar_` ve `sayfa_` yerine tek bir `sayi_` alanı taşır, kayıt satırı `D;` ile başlar. Tam kodu bölüm sonundaki programdadır. `override` anahtar sözcüğünü yazdığımız için imzada bir yazım hatası yaparsak derleyici bunu hemen bildirir.

Dosya satırının ilk harfi (`K`, `D`) yükleme sırasında hangi sınıfın yaratılacağını belirler. Böylece her yayın, kendi biçimini kendisi üretir ve `Kutuphane` sınıfı `Kitap` ile `Dergi`'nin alanlarını bilmek zorunda kalmaz. Üçüncü bir tür (örneğin DVD) eklemek yeni bir sınıf yazmaktan ve yükleme kodunda bir dal eklemekten ibarettir.

40.5 Kütüphane Sınıfı

Ekleme

`Kutuphane`, iki kabı ve sıradaki numaraları özel alan olarak tutar:

```
map<int, unique_ptr<Yayin>> yayinlar_;
map<int, Uye> uyeler_;
int sonrakiYayin_ = 1;
int sonrakiUye_ = 1;
```

Ekleme fonksiyonları önce girdiyi denetler, sonra nesneyi yaratıp kaba yerleştirir. Kuralları ekranı bilen kodda değil burada denetlemek, sınıfı başka bir arayüzle (örneğin bir pencere uygulamasıyla) kullanırsak da kuralların korunmasını sağlar:

```
int kitapEkle(const string& baslik, const string& yazar, int sayfa) {
    if (baslik.empty() || yazar.empty() || sayfa <= 0) {
        throw KutuphaneHatasi("Kitap bilgileri geçersiz.");
    }
    int id = sonrakiYayin_++;
    yayinlar_[id] = make_unique<Kitap>(id, baslik, yazar, sayfa);
    return id;
}
```

`make_unique<Kitap>` bir `unique_ptr<Kitap>` üretir; atama sırasında örtük olarak `unique_ptr<Yayin>`'a dönüşür. `dergiEkle` ve `uyeEkle` aynı biçimdedir. Kitap ve dergi aynı numara dizisinden numara alır; bu yüzden bir numara yalnızca bir yayına aittir ve ödünç işleminde "kitap mı dergi mi" sormamıza gerek kalmaz.

Arama ve ödünç verme

Numaradan nesneye ulaşan iki özel yardımcı vardır. Bulunamazsa `nullptr` döndürmek yerine istisna fırlatırlar; çağıran her seferinde boş değer denetlemek zorunda kalmaz:

```
Yayin& yayinBul(int id) {
    auto it = yayinlar_.find(id);
    if (it == yayinlar_.end()) {
        throw KutuphaneHatasi("Yayın bulunamadı: " + to_string(id));
    }
    return *it->second;
}
```

`it->second` bir `unique_ptr<Yayin>`'dır; `*` onu açarak `Yayin&` verir. Referans döndürdüğümüz için sahiplik kapta kalır ve kopyalama yoktur (Bölüm 28). `uyeBul` aynı işi `const Uye&` döndürerek yapar.

Ödünç verme, iş kurallarının toplandığı yerdir. Kuralları sırayla denetler ve hepsi geçilirse tek bir atama yapar:

```
void oduncVer(int uyeNo, int yayinNo) {
    uyeBul(uyeNo);
    Yayin& y = yayinBul(yayinNo);
    if (y.oduncte()) {
        throw KutuphaneHatasi("Yayın zaten ödünçte.");
    }
    if (oduncSayisi(uyeNo) >= MAKS_ODUNC) {
        throw KutuphaneHatasi("Üye en fazla " +
                              to_string(MAKS_ODUNC) + " yayın alabilir.");
    }
    y.uyeAyarla(uyeNo);
}
```

`oduncSayisi`, üye başına sayaç tutmak yerine kabı tarayıp o üyede olan yayınları sayar. Sayaç ek bir veri olurdu ve `iadeAl` ile `oduncVer` onu doğru güncellemeyi unutursa tutarsızlık doğardı. Tek kaynaktan türetmek (burada yayının `uye_` alanı) daha güvenlidir; bu kütüphane boyutunda tarama maliyeti de önemsizdir:

```
int oduncSayisi(int uyeNo) const {
    int adet = 0;
    for (const auto& [id, y] : yayinlar_) {
        if (y->oduncUye() == uyeNo) {
            ++adet;
        }
    }
    return adet;
}
```

Döngüde C++17 yapısal bağlaması (Bölüm 35) kullandık; `y` her turda `const unique_ptr<Yayin>&` olur. Fonksiyon `const` olduğundan kabı değiştiremez.

Listeleme ve çok biçimlilik

Listeleme kodunda `Kitap` ya da `Dergi` adı hiç geçmez. Her yayına yalnızca `Yayin` arayüzüyle konuşulur ve sanal çağrılar doğru sürümü seçer:

```
void listele(ostream& os) const {
    if (yayinlar_.empty()) {
        os << "Kayıt yok.\n";
        return;
    }
    for (const auto& [id, y] : yayinlar_) {
        os << "[" << id << "] " << y->tur() << "  " << y->baslik()
           << " (" << y->ayrinti() << ")";
        if (y->oduncte()) {
            os << " -> " << uyeler_.at(y->oduncUye()).ad;
        }
        os << '\n';
    }
}
```

Fonksiyon `cout` yerine `ostream&` parametresi alır. Bu sayede aynı kod ekrana da, bir `ostringstream`'e de, dosyaya da yazabilir ve test etmek kolaylaşır. `uyeler_.at(...)`, anahtar yoksa istisna fırlattığı için `operator[]`'dan daha güvenlidir; `[]` olmayan anahtarı sessizce yaratırdı, ayrıca `const` bir kapta kullanılamazdı.

İade ve üye durumu

İade, ödünç vermenin tersidir. Yayın zaten rafta ise bunu hata sayarız; çünkü böyle bir çağrı büyük olasılıkla yanlış numara girildiğini gösterir:

```
void iadeAl(int yayinNo) {
    Yayin& y = yayinBul(yayinNo);
    if (!y.oduncte()) {
        throw KutuphaneHatasi("Yayın zaten rafta.");
    }
    y.uyeAyarla(0);
}
```

`uyeDurumu` da `listele` gibi bir `ostream&` alır; üyenin adını, ödünçteki yayın sayısını ve başlıklarını yazar. Kodu tam programdadır.

40.6 Dosyaya Kaydetme ve Yükleme

Bölüm 20'de kayıtları ikili biçimde yazmıştık; bu biçim yapının bellek düzenine bağlıydı. Burada her satırı bir kayıt olan, alanları `;` ile ayrılmış bir **metin dosyası** kullanacağız. Metin biçimi, `Yayin` nesnelerinin içinde işaretçi ya da sanal tablo bulunduğu için de gereklidir: nesneyi baytlarıyla yazmak anlamsız olurdu. Bunun yerine her nesne kendi alanlarını yazar:

```
U;1;Ayşe Çelik
K;1;Algoritmalar;Selin Arslan;320;1
K;2;Bellek ve İşaretçiler;Mert Aksoy;210;0
D;3;Bilgisayar Dünyası;14;0
```

İlk harf kaydın türünü söyler. Kitap satırının son alanı ödünç alan üyenin numarasıdır (`0`: rafta). Alan ayracı `;` olduğundan, kullanıcının girdisinde `;` bulunamaz: girdiyi zaten `;` ile bölüyoruz, dolayısıyla hiçbir alan bu karakteri taşıyamaz. Bu küçük değişmezlik (invariant) dosya biçiminin tutarlı kalmasını sağlar.

Kaydetme basittir. Her yayın kendi satırını üretir, `Kutuphane` yalnızca sırayla yazar:

```
void kaydet(const string& dosya) const {
    ofstream f(dosya);
    if (!f) {
        throw KutuphaneHatasi("Dosya yazılamadı: " + dosya);
    }
    for (const auto& [no, u] : uyeler_) {
        f << "U;" << no << ';' << u.ad << '\n';
    }
    for (const auto& [id, y] : yayinlar_) {
        f << y->kayitSatiri() << '\n';
    }
    f.close();
    if (f.fail()) {
        throw KutuphaneHatasi("Yazma sırasında hata oluştu.");
    }
}
```

`ofstream` bir RAII nesnesidir (Bölüm 37); bir istisna fırlatılsa bile yıkıcısı dosyayı kapatır. Yine de `close()` çağırıp durumuna bakıyoruz: tamponun diske yazılması kapanışta gerçekleştiği için disk dolu gibi hatalar ancak o noktada ortaya çıkar.

Yükleme daha dikkatli olmalıdır; Bölüm 20'deki ilkeyi uygularız: **dosyayı önce geçici kaplara oku, her şey doğruysa eldekilerle değiştir.** Satırın ilk alanına bakılarak doğru sınıf yaratılır:

```
vector<string> a = bol(satir, ';');
if (a[0] == "U" && a.size() == 3) {
    int no = sayiyaCevir(a[1]);
    uyeler.emplace(no, Uye{no, a[2]});
} else if (a[0] == "K" && a.size() == 6) {
    int id = sayiyaCevir(a[1]);
    auto k = make_unique<Kitap>(id, a[2], a[3], sayiyaCevir(a[4]));
    k->uyeAyarla(sayiyaCevir(a[5]));
    yayinlar[id] = move(k);
} else if (a[0] == "D" && a.size() == 5) {
```

```
    // ... dergi için aynısı
} else {
    throw KutuphaneHatasi("Bozuk kayıt satırı: " + satir);
}
```

Alan sayısı beklenenden farklıysa ya da sayı alanı bozuksa istisna fırlar ve döngüden çıkılır. Geçici kaplar (`yayinlar`, `uyeler`) yerel değişkenlerdir; istisna onları yok eder, `unique_ptr`'ler nesneleri siler. Sınıfın kendi kapları hiç değişmediği için eldeki veri korunur. Bölüm 20'de bu işi elle `free` çağrılarıyla yapmak gerekmişti.

Döngü bittikten sonra bir de tutarlılık denetimi yapılır: ödünçte görünen her yayının sahibi olan üye dosyada bulunmalıdır. Ardından değiştirme tek hamlede yapılır:

```
yayinlar_.swap(yayinlar);       // her şey doğruysa değiştir
uyeler_.swap(uyeler);
sonrakiYayin_ = yayinlar_.empty() ? 1 : yayinlar_.rbegin()->first + 1;
sonrakiUye_ = uyeler_.empty() ? 1 : uyeler_.rbegin()->first + 1;
```

`swap` kapları işaretçi düzeyinde takas ettiği için istisna fırlatmaz ve hızlıdır. Sıradaki numaraları, `map` anahtara göre sıralı olduğundan `rbegin()` ile en büyük anahtarın bir fazlası olarak yeniden hesaplarız; böylece yükleme sonrası eklenen yayın, var olan bir numarayı ezmez.

> **◆ Kritik Nokta · Hata durumunda eldeki veriyi koruyun**
>
> Yükleme yarıda kesilirse program, yarısı eski yarısı yeni bir duruma düşmemelidir. Önce geçici nesnelere yaz, doğrula, sonra değiştir kalıbı hem dosya okumada hem ağ ya da veritabanı işlemlerinde kullanılır. İstisna güvenliği (exception safety) denen bu ilke, RAII ile birlikte C++'ın hata yönetimindeki temel taşıdır.

40.7 Menü ve Ana Program

Menü komutları, kullanıcıyla konuşan ince fonksiyonlardır: girdiyi oku, ayrıştır, `Kutuphane`'nin ilgili işlevini çağır, sonucu yaz. Hiçbirinin hata denetimi kendine ait değildir; sorun çıkarsa istisna yukarı doğru ilerler. İstemi yazıp bir satır okuyan yardımcı, girdi bittiğinde de istisna fırlatır:

```
string girdiAl(const string& istek) {
    cout << istek;
    string satir;
    if (!getline(cin, satir)) {
        throw KutuphaneHatasi("Girdi sona erdi.");
    }
    return satir;
}
```

Bölüm 23'te gördüğümüz gibi `>>` ile sayı okumak satır sonunu akışta bırakır ve hatalı girdide akışı bozar. Bunun yerine her girdiyi `getline` ile satır olarak alıp kendi ayrıştırıcılarımıza (`bol`, `sayiyaCevir`) veririz; Bölüm 20'deki `fgets` ve `sscanf` kalıbının C++ karşılığıdır. Örneğin ödünç komutu iki sayıyı bir `istringstream` ile ayırır:

```
void komutOdunc(Kutuphane& k) {
    istringstream ss(girdiAl("Üye no ve yayın no: "));
    string a, b;
    if (!(ss >> a >> b)) {
        throw KutuphaneHatasi("İki numara gerekli.");
    }
```

```
    k.oduncVer(sayiyaCevir(a), sayiyaCevir(b));
    cout << "Ödünç verildi.\n";
}
```

`main` yalnızca menü döngüsünü yürütür. Bütün komutlar tek bir `try` bloğunun içinde çağrıldığı için hata yönetimi tek yerde toplanır:

```
try {
    switch (sayiyaCevir(t)) {
        case 1: komutKitap(kutuphane); break;
        // ... diğer komutlar
        default: cout << "Geçersiz seçim.\n";
    }
} catch (const exception& e) {
    cout << "Hata: " << e.what() << '\n';
}
```

`catch`, `KutuphaneHatasi` yerine taban sınıf `exception`'ı yakalar; böylece bellek yetersizliği gibi standart kütüphane hataları da program çökmeden bildirilir. Menü girişi sayı değilse `sayiyaCevir` aynı hata mesajını üretir, ayrı bir denetim yazmayız. `Kutuphane` nesnesi `main` içinde yerel değişken olduğu için döngü bittiğinde yıkılır ve tüm yayın nesneleri otomatik silinir; Bölüm 20'deki `free(liste.dizi)` satırının karşılığı yoktur.

40.8 Tam Program

Parçaları birleştirince tam program aşağıdaki gibi olur. Tek dosyadır ve `g++ -std=c++17 -Wall kutuphane.cpp -o kutuphane` ile derlenir. Büyük projelerde sınıflar `.hpp` ve `.cpp` dosyalarına bölünür (Bölüm 30); okumayı kolaylaştırmak için tek dosyada bıraktık. Kısa bir program olduğundan ve başlık dosyası içermediğinden `using namespace std;` kullandık.

kutuphane.cpp

```
#include <fstream>
#include <iostream>
#include <map>
#include <memory>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

using namespace std;

const string DOSYA_ADI = "kutuphane.txt";
const int MAKS_ODUNC = 3;

// ---------- hata sınıfı ve yardımcılar ----------

class KutuphaneHatasi : public runtime_error {
public:
    explicit KutuphaneHatasi(const string& mesaj) : runtime_error(mesaj) {}
};

vector<string> bol(const string& s, char ayrac) {
    vector<string> parcalar;
    stringstream ss(s);
    string parca;
```

```
    while (getline(ss, parca, ayrac)) {
        parcalar.push_back(parca);
    }
    return parcalar;
}

int sayiyaCevir(const string& s) {
    try {
        size_t konum = 0;
        int deger = stoi(s, &konum);
        if (konum != s.size()) {
            throw invalid_argument(s);
        }
        return deger;
    } catch (const logic_error&) {
        throw KutuphaneHatasi("Sayı bekleniyordu: '" + s + "'");
    }
}

// ---------- yayın hiyerarşisi ----------

class Yayin {
public:
    Yayin(int id, const string& baslik) : id_(id), baslik_(baslik) {}
    virtual ~Yayin() = default;

    int id() const { return id_; }
    const string& baslik() const { return baslik_; }
    int oduncUye() const { return uye_; }
    bool oduncte() const { return uye_ != 0; }
    void uyeAyarla(int uyeNo) { uye_ = uyeNo; }

    virtual string tur() const = 0;
    virtual string ayrinti() const = 0;
    virtual string kayitSatiri() const = 0;

protected:
    string ortakAlanlar() const {
        return to_string(id_) + ";" + baslik_;
    }

private:
    int id_;
    string baslik_;
    int uye_ = 0;               // 0: rafta, değilse ödünç alan üyenin no'su
};

class Kitap : public Yayin {
public:
    Kitap(int id, const string& baslik, const string& yazar, int sayfa)
        : Yayin(id, baslik), yazar_(yazar), sayfa_(sayfa) {}

    string tur() const override { return "Kitap"; }
    string ayrinti() const override {
        return yazar_ + ", " + to_string(sayfa_) + " sayfa";
    }
    string kayitSatiri() const override {
        return "K;" + ortakAlanlar() + ";" + yazar_ + ";" +
               to_string(sayfa_) + ";" + to_string(oduncUye());
    }

private:
    string yazar_;
    int sayfa_;
```

```
};

class Dergi : public Yayin {
public:
    Dergi(int id, const string& baslik, int sayi)
        : Yayin(id, baslik), sayi_(sayi) {}

    string tur() const override { return "Dergi"; }
    string ayrinti() const override { return "sayı " + to_string(sayi_); }
    string kayitSatiri() const override {
        return "D;" + ortakAlanlar() + ";" + to_string(sayi_) + ";" +
               to_string(oduncUye());
    }

private:
    int sayi_;
};

struct Uye {
    int no;
    string ad;
};

// ---------- kütüphane ----------

class Kutuphane {
public:
    int kitapEkle(const string& baslik, const string& yazar, int sayfa) {
        if (baslik.empty() || yazar.empty() || sayfa <= 0) {
            throw KutuphaneHatasi("Kitap bilgileri geçersiz.");
        }
        int id = sonrakiYayin_++;
        yayinlar_[id] = make_unique<Kitap>(id, baslik, yazar, sayfa);
        return id;
    }

    int dergiEkle(const string& baslik, int sayi) {
        if (baslik.empty() || sayi <= 0) {
            throw KutuphaneHatasi("Dergi bilgileri geçersiz.");
        }
        int id = sonrakiYayin_++;
        yayinlar_[id] = make_unique<Dergi>(id, baslik, sayi);
        return id;
    }

    int uyeEkle(const string& ad) {
        if (ad.empty()) {
            throw KutuphaneHatasi("Üye adı boş olamaz.");
        }
        int no = sonrakiUye_++;
        uyeler_.emplace(no, Uye{no, ad});
        return no;
    }

    void oduncVer(int uyeNo, int yayinNo) {
        uyeBul(uyeNo);
        Yayin& y = yayinBul(yayinNo);
        if (y.oduncte()) {
            throw KutuphaneHatasi("Yayın zaten ödünçte.");
        }
        if (oduncSayisi(uyeNo) >= MAKS_ODUNC) {
            throw KutuphaneHatasi("Üye en fazla " +
                                  to_string(MAKS_ODUNC) + " yayın alabilir.");
        }
```

```
        y.uyeAyarla(uyeNo);
    }

    void iadeAl(int yayinNo) {
        Yayin& y = yayinBul(yayinNo);
        if (!y.oduncte()) {
            throw KutuphaneHatasi("Yayın zaten rafta.");
        }
        y.uyeAyarla(0);
    }

    int oduncSayisi(int uyeNo) const {
        int adet = 0;
        for (const auto& [id, y] : yayinlar_) {
            if (y->oduncUye() == uyeNo) {
                ++adet;
            }
        }
        return adet;
    }

    void listele(ostream& os) const {
        if (yayinlar_.empty()) {
            os << "Kayıt yok.\n";
            return;
        }
        for (const auto& [id, y] : yayinlar_) {
            os << "[" << id << "] " << y->tur() << "  " << y->baslik()
               << " (" << y->ayrinti() << ")";
            if (y->oduncte()) {
                os << " -> " << uyeler_.at(y->oduncUye()).ad;
            }
            os << '\n';
        }
    }

    void uyeDurumu(int uyeNo, ostream& os) const {
        const Uye& u = uyeBul(uyeNo);
        os << u.ad << ": " << oduncSayisi(uyeNo) << " yayın ödünçte\n";
        for (const auto& [id, y] : yayinlar_) {
            if (y->oduncUye() == uyeNo) {
                os << "  [" << id << "] " << y->baslik() << '\n';
            }
        }
    }

    int yayinSayisi() const { return static_cast<int>(yayinlar_.size()); }
    int uyeSayisi() const { return static_cast<int>(uyeler_.size()); }

    void kaydet(const string& dosya) const {
        ofstream f(dosya);
        if (!f) {
            throw KutuphaneHatasi("Dosya yazılamadı: " + dosya);
        }
        for (const auto& [no, u] : uyeler_) {
            f << "U;" << no << ';' << u.ad << '\n';
        }
        for (const auto& [id, y] : yayinlar_) {
            f << y->kayitSatiri() << '\n';
        }
        f.close();
        if (f.fail()) {
            throw KutuphaneHatasi("Yazma sırasında hata oluştu.");
        }
```

```
    }

    void yukle(const string& dosya) {
        ifstream f(dosya);
        if (!f) {
            throw KutuphaneHatasi("Dosya açılamadı: " + dosya);
        }
        map<int, unique_ptr<Yayin>> yayinlar;
        map<int, Uye> uyeler;
        string satir;
        while (getline(f, satir)) {
            if (satir.empty()) {
                continue;
            }
            vector<string> a = bol(satir, ';');
            if (a[0] == "U" && a.size() == 3) {
                int no = sayiyaCevir(a[1]);
                uyeler.emplace(no, Uye{no, a[2]});
            } else if (a[0] == "K" && a.size() == 6) {
                int id = sayiyaCevir(a[1]);
                auto k = make_unique<Kitap>(id, a[2], a[3],
                                            sayiyaCevir(a[4]));
                k->uyeAyarla(sayiyaCevir(a[5]));
                yayinlar[id] = move(k);
            } else if (a[0] == "D" && a.size() == 5) {
                int id = sayiyaCevir(a[1]);
                auto d = make_unique<Dergi>(id, a[2], sayiyaCevir(a[3]));
                d->uyeAyarla(sayiyaCevir(a[4]));
                yayinlar[id] = move(d);
            } else {
                throw KutuphaneHatasi("Bozuk kayıt satırı: " + satir);
            }
        }
        for (const auto& [id, y] : yayinlar) {
            if (y->oduncte() && uyeler.count(y->oduncUye()) == 0) {
                throw KutuphaneHatasi("Kayıtlı olmayan üye: " +
                                      to_string(y->oduncUye()));
            }
        }
        yayinlar_.swap(yayinlar);       // her şey doğruysa değiştir
        uyeler_.swap(uyeler);
        sonrakiYayin_ = yayinlar_.empty() ? 1 : yayinlar_.rbegin()->first + 1;
        sonrakiUye_ = uyeler_.empty() ? 1 : uyeler_.rbegin()->first + 1;
    }

private:
    Yayin& yayinBul(int id) {
        auto it = yayinlar_.find(id);
        if (it == yayinlar_.end()) {
            throw KutuphaneHatasi("Yayın bulunamadı: " + to_string(id));
        }
        return *it->second;
    }

    const Uye& uyeBul(int no) const {
        auto it = uyeler_.find(no);
        if (it == uyeler_.end()) {
            throw KutuphaneHatasi("Üye bulunamadı: " + to_string(no));
        }
        return it->second;
    }

    map<int, unique_ptr<Yayin>> yayinlar_;
    map<int, Uye> uyeler_;
```

```
    int sonrakiYayin_ = 1;
    int sonrakiUye_ = 1;
};

// ---------- menü komutları ----------

string girdiAl(const string& istek) {
    cout << istek;
    string satir;
    if (!getline(cin, satir)) {
        throw KutuphaneHatasi("Girdi sona erdi.");
    }
    return satir;
}

void komutKitap(Kutuphane& k) {
    vector<string> p = bol(girdiAl("Kitap (başlık;yazar;sayfa): "), ';');
    if (p.size() != 3) {
        throw KutuphaneHatasi("Üç alan gerekli: başlık;yazar;sayfa");
    }
    int id = k.kitapEkle(p[0], p[1], sayiyaCevir(p[2]));
    cout << "Kitap eklendi. No: " << id << '\n';
}

void komutDergi(Kutuphane& k) {
    vector<string> p = bol(girdiAl("Dergi (başlık;sayı): "), ';');
    if (p.size() != 2) {
        throw KutuphaneHatasi("İki alan gerekli: başlık;sayı");
    }
    int id = k.dergiEkle(p[0], sayiyaCevir(p[1]));
    cout << "Dergi eklendi. No: " << id << '\n';
}

void komutUye(Kutuphane& k) {
    int no = k.uyeEkle(girdiAl("Üye adı: "));
    cout << "Üye eklendi. No: " << no << '\n';
}

void komutOdunc(Kutuphane& k) {
    istringstream ss(girdiAl("Üye no ve yayın no: "));
    string a, b;
    if (!(ss >> a >> b)) {
        throw KutuphaneHatasi("İki numara gerekli.");
    }
    k.oduncVer(sayiyaCevir(a), sayiyaCevir(b));
    cout << "Ödünç verildi.\n";
}

void komutIade(Kutuphane& k) {
    k.iadeAl(sayiyaCevir(girdiAl("Yayın no: ")));
    cout << "İade alındı.\n";
}

void komutUyeDurumu(const Kutuphane& k) {
    k.uyeDurumu(sayiyaCevir(girdiAl("Üye no: ")), cout);
}

void menuYaz() {
    cout << "1 Kitap 2 Dergi 3 Üye 4 Liste 5 Ödünç 6 İade "
            "7 Üye durumu 8 Kaydet 9 Yükle 0 Çıkış\n";
}

int main() {
    Kutuphane kutuphane;
```

```
    menuYaz();
    string t;
    while (true) {
        cout << "> ";
        if (!getline(cin, t) || t == "0") {
            break;
        }
        try {
            switch (sayiyaCevir(t)) {
                case 1: komutKitap(kutuphane); break;
                case 2: komutDergi(kutuphane); break;
                case 3: komutUye(kutuphane); break;
                case 4: kutuphane.listele(cout); break;
                case 5: komutOdunc(kutuphane); break;
                case 6: komutIade(kutuphane); break;
                case 7: komutUyeDurumu(kutuphane); break;
                case 8:
                    kutuphane.kaydet(DOSYA_ADI);
                    cout << kutuphane.yayinSayisi() << " yayın, "
                         << kutuphane.uyeSayisi() << " üye kaydedildi.\n";
                    break;
                case 9:
                    kutuphane.yukle(DOSYA_ADI);
                    cout << kutuphane.yayinSayisi() << " yayın, "
                         << kutuphane.uyeSayisi() << " üye yüklendi.\n";
                    break;
                default: cout << "Geçersiz seçim.\n";
            }
        } catch (const exception& e) {
            cout << "Hata: " << e.what() << '\n';
        }
    }
    cout << "Güle güle.\n";
    return 0;
}
```

40.9 Örnek Çalışma

Programı derleyip şu sırayla çalıştıralım: iki kitap, bir dergi ve bir üye ekleyelim, yayınları listeleyelim, bir kitabı ödünç verelim, aynı kitabı yeniden ödünç vermeyi deneyelim, durumu dosyaya kaydedelim, kitabı iade edelim, dosyayı geri yükleyelim ve üyenin durumuna bakalım. `>` işaretinden sonra yazılan değerler kullanıcının girdisidir.

```
ÇIKTI
1 Kitap 2 Dergi 3 Üye 4 Liste 5 Ödünç 6 İade 7 Üye durumu 8 Kaydet 9 Yükle 0 Çıkış
> 1
Kitap (başlık;yazar;sayfa): Algoritmalar;Selin Arslan;320
Kitap eklendi. No: 1
> 1
Kitap (başlık;yazar;sayfa): Bellek ve İşaretçiler;Mert Aksoy;210
Kitap eklendi. No: 2
> 2
Dergi (başlık;sayı): Bilgisayar Dünyası;14
Dergi eklendi. No: 3
> 3
Üye adı: Ayşe Çelik
Üye eklendi. No: 1
> 4
[1] Kitap  Algoritmalar (Selin Arslan, 320 sayfa)
[2] Kitap  Bellek ve İşaretçiler (Mert Aksoy, 210 sayfa)
[3] Dergi  Bilgisayar Dünyası (sayı 14)
> 5
Üye no ve yayın no: 1 1
Ödünç verildi.
> 5
Üye no ve yayın no: 1 1
Hata: Yayın zaten ödünçte.
> 8
3 yayın, 1 üye kaydedildi.
> 6
Yayın no: 1
İade alındı.
> 9
3 yayın, 1 üye yüklendi.
> 7
Üye no: 1
Ayşe Çelik: 1 yayın ödünçte
  [1] Algoritmalar
> 0
Güle güle.
```

Çıktıdan birkaç gözlem yapalım:

- Kitap ve dergi aynı numara dizisinden numara aldı (1, 2, 3). Listeleme kodu iki türü ayırt etmeden dolaştı; "Kitap" ve "Dergi" adlarını, parantez içindeki ayrıntıyı sanal fonksiyonlar üretti.
- İkinci ödünç denemesi `oduncVer` içinde `KutuphaneHatasi` fırlattı; `main`'deki `catch` mesajı yazdı ve program menüye döndü.
- Kayıttan sonra kitap iade edildi, ama yükleme dosyadaki eski durumu geri getirdi. Bu yüzden üye durumunda 1 numaralı kitap yeniden ödünçte görünüyor.
- Program çıkarken hiçbir `delete` yazmadık; `unique_ptr`'ler nesneleri sildi. Sızıntı denetimi için `g++ -std=c++17 -g -fsanitize=address kutuphane.cpp` komutuyla derleyebilirsiniz (Bölüm 29).

> **İpucu · Bir sınıfı ekrandan bağımsız tutun**
>
> `Kutuphane` sınıfının hiçbir yerinde `cin` yoktur; `cout` yalnızca `ostream&` parametresi olarak geçer. Bu yüzden sınıfı `main`'den bağımsız denemek kolaydır: birkaç satırlık bir test programı yayın ekler, ödünç verir ve beklenen istisnanın fırlayıp fırlamadığına bakar. Girdi-çıktı ile iş kuralını ayırmak, programı hem sınamayı hem de başka bir arayüze taşımayı kolaylaştırır.

Geliştirme Önerileri

Program çalışıyor ve gereksinimleri karşılıyor. Ama bir uygulamayı öğrenmenin yolu onu kendi isteklerinize göre değiştirmektir. Öneriler yaklaşık artan zorluk sırasıyla verilmiştir; her birini bağımsız yapabilirsiniz.

1. **Başlıkta arama.** Başlığında verilen sözcük geçen yayınları listeleyen bir komut ekleyin. Büyük/küçük harf farkını yok saymak için Bölüm 27'deki karakter işlemlerini kullanın.
2. **Sıralı listeleme.** Yayınları başlığa göre alfabetik sıralayan bir listeleme seçeneği yazın. Kap `map<int, ...>` olduğundan önce işaretçileri bir `vector<const Yayin*>`'a kopyalayıp `sort` ve bir lambda ile sıralayın (Bölüm 36).
3. **Üye silme.** Üye silme komutu ekleyin. Üzerinde ödünç yayın olan üyenin silinmesini bir istisnayla reddedin; dosya yüklemedeki tutarlılık denetimiyle ilişkisini düşünün.
4. **Yeni yayın türü.** `Yayin`'den türeyen bir `Dvd` sınıfı (başlık, süre) ekleyin. `Kutuphane`'nin hangi kısımlarına dokunmanız gerektiğine, hangilerinin değişmeden çalıştığına dikkat edin.
5. **Teslim tarihi ve gecikme.** Ödünç verirken `<chrono>` ile bir tarih saklayıp iade tarihi geçen yayınları listeleyen bir komut yazın. Tarihi dosyaya nasıl yazacağınıza karar verin.
6. **Başlıkta noktalı virgül.** Ayraç `;` olduğu için başlıkta bu karakter kullanılamıyor. Alanları tırnak içine alan ya da kaçış karakteri kullanan bir yazma ve okuma kuralı tasarlayın.
7. **Çok dosyalı yapı.** Programı `yayin.hpp`, `kutuphane.hpp`, `kutuphane.cpp` ve `main.cpp` olarak bölün; gerekirse bir `Makefile` ya da `CMakeLists.txt` yazın. Sanal yıkıcı ve `unique_ptr` kullanan sınıflarda başlık dosyasına neyin konması gerektiğini düşünün.
8. **Kopyalama ve taşıma.** `Kutuphane` nesnesini kopyalamaya çalışın ve derleyicinin neden hata verdiğini açıklayın. Sonra, derin kopya gerektiren bir `kopyala` fonksiyonu için `Yayin`'e sanal bir `klonla()` metodu eklemeyi deneyin (Bölüm 39'daki taşıma kavramları işinize yarar).

Bölüm Özeti

- Gereksinimlerdeki isimler sınıf adaylarını, fiiller üye fonksiyonları gösterir; "bir X bir Y'dir" ilişkisi kalıtıma, "bir X bir Y'ye sahiptir" ilişkisi üyelik ilişkisine işaret eder.
- Ortak veriyi taban sınıfta, türe özgü davranışı sanal fonksiyonlarda toplamak, yeni tür eklemeyi mevcut kodu değiştirmeden mümkün kılar.
- Soyut sınıf, saf sanal fonksiyonlarla türemiş sınıfları gerekli işlevleri yazmaya zorlar; çok biçimli nesneler taban sınıfın sanal yıkıcısını gerektirir.
- Çok biçimli nesneler kapta değerle değil, `std::unique_ptr<Taban>` ile saklanır; böylece dilimleme olmaz ve bellek otomatik bırakılır.
- `std::map` hem hızlı arama hem sıralı dolaşma sağlar; `at` ve `find` olmayan anahtarı yaratmadan sorgular.
- Aynı bilgiyi iki yerde tutmak tutarsızlık doğurur; ödünç sayısı gibi değerleri tek kaynaktan türetmek daha güvenlidir.
- Kural ihlalleri kendi istisna sınıfımızla bildirilir; tek bir `catch` ekrana mesaj yazar ve ara katmanlar hata kodlarını taşımak zorunda kalmaz.
- Girdiyi `getline` ile satır olarak okuyup `stringstream` ve `stoi` ile ayrıştırmak, `>>` kullanımının doğurduğu akış bozulmalarından korur.

- Her nesnenin kendi dosya satırını üretmesi, saklama biçimini sınıfın sorumluluğunda tutar; yüklemede ilk alan hangi sınıfın yaratılacağını belirler.
- Yüklemede önce geçici kaplara okuyup doğrulamak, sonra `swap` ile değiştirmek hata durumunda eldeki veriyi korur.
- Nesne ömrü RAII ile yönetildiği için programda hiç `new` ya da `delete` yazmak gerekmez; elle yönetilen Bölüm 20 sürümünün karşılığı sınıfların yıkıcılarındadır.
- İş kuralını barındıran sınıfı ekran ve klavyeden ayırmak, onu sınamayı ve başka bir arayüze taşımayı kolaylaştırır.

KISIM III

C#

2000 · Microsoft

C#, 2000 yılında Microsoft tarafından Anders Hejlsberg liderliğindeki bir ekip tarafından duyuruldu ve 2002'de .NET Framework ile birlikte yayımlandı. C ailesinin tanıdık sözdizimini korurken bellek yönetimini çöp toplayıcıya bıraktı, güvenli bir tip sistemi ve geniş bir standart kütüphane sundu. Bugün açık kaynaklı ve çapraz platform .NET ile web servislerinden Unity oyunlarına, masaüstü uygulamalardan bulut sistemlerine kadar geniş bir alanda kullanılır.

Bu kısımda C#'ı öğrenirken sık sık C ve C++'a geri döneceğiz. Böylece aynı fikrin üç dilde nasıl farklı biçimler aldığını göreceğiz.

BÖLÜM 41

.NET Dünyası ve C#'a Giriş

BU BÖLÜMDE

- C#'ın tasarım hedeflerini (güvenli, yönetilen, nesne yönelimli, üretken) ve C#'ın bir dil, .NET'in bir platform olduğunu
- Kaynak kodun IL'e, oradan CLR ve JIT aracılığıyla makine koduna nasıl dönüştüğünü; assembly kavramını
- SDK, çalışma zamanı (runtime), temel sınıf kütüphanesi (BCL) ve NuGet'in görevlerini
- Yönetilen belleği ve çöp toplayıcının (GC) C'deki `malloc` / `free` düzeninden farkını
- `class Program { static void Main() }` biçimini ve top-level statements biçimini; `namespace` ve `using` yapısını
- `dotnet new`, `dotnet build`, `dotnet run` komutlarını ve `.csproj` dosyasını
- Derleyici mesajlarını okumayı; C#'ın kullanım alanlarını ve üç dilin derleme modeli farkını

C ve C++ bölümlerinde programlama dilinin makineye çok yakın bir katmanında çalıştık: bellek bizim elimizdeydi, derleyici kodu doğrudan işlemcinin dilinden bir dosyaya çeviriyordu ve hataların bedelini çoğu zaman çalışma anında ödüyorduk. Bu bölümle C# kısmına geçiyoruz. C# aynı sözdizimi ailesinden gelir; ama kodun çalıştığı ortam farklıdır. Programlarımız bir **çalışma zamanı** (runtime) tarafından yönetilir ve bu durum dilin birçok kuralını belirler.

Bu bölümde dilin ayrıntılarına girmeyeceğiz. Önce C#'ın çalıştığı platformu, yani .NET'i tanıyacağız: kaynak kodun hangi aşamalardan geçtiğini, belleğin kim tarafından yönetildiğini ve bir C# projesinin nasıl göründüğünü. Ardından ilk programlarımızı yazacağız. Bölüm 2'de C#'ın tarihini ve sürümlerini anlattık; kurulum Bölüm 3'te anlatıldı. Burada o bölümlerde anlatılanları tekrarlamayıp üzerine ekleyeceğiz.

41.1 C#'ın Tasarım Hedefleri

C#, 2000 yılında Microsoft'ta Anders Hejlsberg liderliğinde, .NET platformunun ana dili olarak tasarlandı (Bölüm 2). Tasarımcıların elinde iki örnek vardı: C++'ın gücü ve Java'nın yönetilen ortam modeli. Hedeflerini dört başlıkta özetleyebiliriz.

Güvenli. Dil, programcıyı sık yapılan bellek hatalarından korur. Yerel bir değişken değer atanmadan okunamaz, dizi sınırları çalışma anında denetlenir, geçersiz bir tip dönüşümü sessizce yapılmaz, işaretçi aritmetiği varsayılan olarak kapalıdır. C'de tanımsız davranışa yol açan birçok durum burada ya derleme hatasıdır ya da düzgün tanımlanmış bir **istisnadır** (exception).

Yönetilen. Programın bellek ve yürütme işlerini bir çalışma zamanı üstlenir. Bellek, çöp toplayıcı tarafından geri alınır; `free` ya da `delete` yazmayız. Bu çalışma zamanının denetiminde çalışan koda **yönetilen kod** (managed code) denir.

Nesne yönelimli. C# başından beri nesne yönelimli bir dildir. Sınıflar, arayüzler, kalıtım ve çok biçimlilik dilin merkezindedir. Değer tipleri dahil her tip, en sonunda `System.Object` sınıfından türer. C++'tan farklı olarak sınıflar yalnızca tek bir sınıftan türeyebilir; çoklu davranış arayüzlerle sağlanır (Bölüm 50 ve 51). Başlık dosyası ve önişlemci makroları yoktur.

Üretken. Dil, sık yapılan işleri kısaltacak özellikleri (özellikler, LINQ, `async`/`await`, desen eşleme) ve zengin bir araç desteğini yıl yıl ekledi. Aynı işi C'de elle yazdığımız onlarca satır, çoğu zaman tek bir kütüphane çağrısıyla yapılır.

> ◆ **Kritik Nokta**
>
> **C# bir dildir, .NET bir platformdur.** C# yalnızca söz dizimini ve anlamı tanımlar. Programı derleyen araçlar, çalıştıran çalışma zamanı ve hazır kütüphaneler .NET'e aittir. Aynı platformda F# ve Visual Basic gibi başka diller de çalışır; hepsi aynı kütüphaneleri paylaşır. Bu bölüm iki kavramı birlikte ele alır, çünkü C#'ın kurallarının çoğu .NET'in çalışma biçiminden gelir.

41.2 .NET Platformu: Kaynak Koddan Makine Koduna

C ve C++'ta derleme modeli şöyleydi: derleyici kaynak dosyayı doğrudan hedef işlemcinin makine koduna çevirir, bağlayıcı (linker) parçaları birleştirir ve ortaya o işlemciye ve işletim sistemine özgü bir çalıştırılabilir dosya çıkar (Bölüm 4). C# ise bu işi iki aşamaya böler.

1. **Derleme.** C# derleyicisi (adı **Roslyn**) kaynak kodu **IL**'e (Intermediate Language, ara dil) çevirir. IL, ne insanın okuyacağı bir dildir ne de bir işlemcinin komut kümesidir; .NET'e özgü, işlemciden bağımsız bir komut kümesidir. Çıktı bir `.dll` ya da `.exe` dosyasıdır.
2. **Çalıştırma.** Program başlatılınca **CLR** (Common Language Runtime, ortak dil çalışma zamanı) devreye girer. CLR içindeki **JIT** (Just-In-Time) derleyicisi, bir metot ilk kez çağrılacağı zaman onun IL kodunu o bilgisayarın işlemcisine uygun makine koduna çevirir. İşlemci yalnızca bu makine kodunu çalıştırır.

```
C / C++:
  merhaba.c --[derleyici]--> merhaba.o --[bağlayıcı]--> merhaba (makine kodu)

C#:
  Program.cs --[Roslyn]--> Merhaba.dll (IL) --[CLR + JIT]--> makine kodu
   derleme zamanı                              çalışma zamanı
```

JIT, çevirdiği makine kodunu bellekte saklar; aynı metot ikinci kez çağrılınca yeniden çevrilmez. Modern .NET ayrıca sık çalışan metotları daha sonra yeniden, daha çok eniyilemeyle derleyebilir. Bu ayrıntılar programımızı etkilemez; bilmemiz gereken, **makine kodunun çalışma anında üretildiğidir**.

IL'e bakış

Basit bir programın IL'ini görmek, ara dilin nasıl bir şey olduğunu anlamak için yeterlidir. `Console.WriteLine("Merhaba, Dünya!");` tek satırlık programından üretilen giriş metodu, sadeleştirilmiş olarak şuna benzer:

```
.method private hidebysig static void '<Main>$'(string[] args) cil managed
{
    .entrypoint
    ldstr "Merhaba, Dünya!"
```

```
    call void [System.Console]System.Console::WriteLine(string)
    ret
}
```

`ldstr` komutu bir metni **değerlendirme yığınına** (evaluation stack) koyar, `call` o yığındaki değeri alıp `WriteLine` metodunu çağırır, `ret` metottan döner. IL, yığın tabanlı bir makine gibi çalışır; işlemcinin yazmaçlarından söz etmez. Yazmaç ayırma gibi işlemciye özgü kararları JIT verir. Bu listeyi `ildasm` gibi araçlarla ya da IL gösteren çevrim içi araçlarla kendimiz görebiliriz; ama C# yazmak için IL bilmek gerekmez.

Assembly

Derleme sonunda üretilen `.dll` ya da `.exe` dosyasına **assembly** (derleme birimi) denir. Bir assembly yalnızca IL kodunu taşımaz; içinde ayrıca **üst veri** (metadata) bulunur: hangi tiplerin tanımlandığı, bunların üyeleri, assembly'nin adı ve sürümü, bağımlı olduğu diğer assembly'ler. C'de başlık dosyaları bu bilgiyi derleme sırasında dışarıdan sağlardı. .NET'te bilgi dosyanın kendisinde durur; bu yüzden C#'ta başlık dosyası yoktur ve derleyici, bir kütüphanenin tiplerini doğrudan o kütüphanenin `.dll` dosyasından öğrenir.

Assembly, dağıtım ve sürümleme birimidir. Bir proje derlenince bir assembly üretir; başka projeler onu referans olarak ekleyebilir.

Çalışan bir program kendi assembly bilgisini de sorgulayabilir. Aşağıdaki program, `int` ve `Console` tiplerinin hangi assembly'lerde tanımlı olduğunu yazdırır, ayrıca derleyicinin bizim için üretip giriş noktası yaptığı metodu gösterir. Programdaki `GetType()` ve `Assembly` ayrıntılarını şimdilik bir kenara bırakıp çıktıya bakalım:

▸ assemblyBilgi.cs

```
Console.WriteLine(typeof(int).Assembly.GetName().Name);
Console.WriteLine(typeof(Console).Assembly.GetName().Name);
Console.WriteLine(typeof(List<int>).Namespace);
var giris = System.Reflection.Assembly.GetEntryAssembly()!.EntryPoint!;
Console.WriteLine(giris.Name);
```

ÇIKTI

```
System.Private.CoreLib
System.Console
System.Collections.Generic
<Main>$
```

`int` temel kütüphane assembly'sindedir, `Console` ise `System.Console` adlı ayrı bir assembly'de. `List<int>` ise `System.Collections.Generic` **ad alanındadır**; ad alanı ile assembly aynı şey değildir (aşağıda göreceğiz). Son satır, kodumuzda hiç yazmadığımız bir `Main` metodunun var olduğunu gösterir; adı `<Main>$`'dır ve bunu derleyici üretmiştir. Bu konuya biraz sonra döneceğiz. Sondaki `!` işaretlerini Bölüm 59'da açıklayacağız; şimdilik "bu değer boş olmayacak" demek için kullanıldığını bilmemiz yeterli.

Ortak tip sistemi ve diller arası çalışma

IL bir ara dil olduğu için C#, F# ve Visual Basic aynı IL'e derlenir. Bunun çalışması için tüm dillerin tipleri aynı biçimde tanımlaması gerekir. Bu kuralları **CTS** (Common Type System, ortak tip sistemi) belirler: `int`, `string` ya da bir sınıf, hangi dilde yazılırsa yazılsın aynı .NET tipidir. Bölüm 42'de `int` anahtar sözcüğünün `System.Int32` tipinin kısa adı olduğunu görürken bu sistemin sonucuyla karşılaşacağız. Böylece bir F# kütüphanesini C# programından, kopyalama ya da dönüştürme yapmadan kullanabiliriz.

Önceden derleme seçenekleri

JIT'in başlangıç maliyeti vardır: program ilk çalıştığında metotların çevrilmesi zaman alır. Bu maliyeti azaltmak için .NET iki ek yol sunar. **ReadyToRun**, IL ile birlikte önceden üretilmiş makine kodunu da assembly'ye koyar. **Native AOT** ise programı JIT ve çalışma zamanı gerektirmeyen, doğrudan makine kodu içeren tek bir dosyaya çevirir; ancak bazı dil ve kütüphane özelliklerini kısıtlar. Bu kitabın örnekleri her zamanki IL + JIT modeliyle çalışır.

41.3 SDK, Çalışma Zamanı ve Sınıf Kütüphanesi

Bölüm 3'te .NET SDK'yı kurduk. Şimdi "kurduğumuz şey neydi?" sorusunu cevaplayalım. .NET'in parçaları iç içe bir yapı oluşturur.

Parça	Açılımı	İçeriği ve görevi
CLR	Common Language Runtime	IL'i çalıştıran motor: JIT, çöp toplayıcı, tip yükleme, güvenlik denetimleri
BCL	Base Class Library	Hazır tip ve metotlar: `Console`, `List<T>`, `File`, `Math` ...
Runtime	.NET Runtime	CLR ile BCL'in birlikte sunulmuş hâli; hazır bir .NET programını çalıştırmaya yeter
SDK	Software Development Kit	Runtime + C# derleyicisi (Roslyn) + `dotnet` komutu ve proje araçları

Program yazmak için SDK, yalnızca hazır bir programı çalıştırmak için Runtime yeterlidir. C ve C++ ile karşılaştırırsak: gcc kurmak SDK kurmaya benzer; derlenmiş bir C programı ise çalışma için ayrıca bir şey gerektirmez (en fazla paylaşılan kütüphaneler). C# programı ise çalışacağı bilgisayarda bir .NET Runtime ister. İstersek programı çalışma zamanıyla birlikte yayımlayabiliriz (self-contained); bu, `dotnet publish` konusunda biraz sonra görülecek.

Temel sınıf kütüphanesi

BCL, C'nin standart kütüphanesinin ve C++'ın STL'inin çok daha genişidir: metin işleme, koleksiyonlar, dosya ve ağ işlemleri, tarih ve saat, eşzamanlılık, şifreleme, JSON ve XML işleme gibi pek çok iş hazır gelir. BCL tipleri **ad alanları** (namespace) altında düzenlenir:

Ad alanı	Tipik içerik
`System`	`Console`, `Math`, `String`, `Int32`, `DateTime`, `Exception`
`System.Collections.Generic`	`List<T>`, `Dictionary<TKey,TValue>`
`System.IO`	`File`, `Directory`, `StreamReader`
`System.Linq`	Koleksiyon sorguları (LINQ)
`System.Text`	`StringBuilder`, kodlamalar
`System.Threading.Tasks`	`Task`, `async` / `await` desteği

NuGet

BCL'de bulunmayan işler için hazır paketler kullanırız. **NuGet**, .NET paket yöneticisidir; paketler `nuget.org` adresindeki depoda yayımlanır. Bir paketi projeye eklemek tek komuttur:

```
TERMİNAL
```

```
$ dotnet add package PaketAdi
```

Komut, paketi indirir ve `.csproj` dosyasına bir `PackageReference` satırı ekler. Böylece bağımlılıklar, kaynak kodla birlikte taşınan küçük bir metin dosyasında listelenir. C'de bir kütüphaneyi kullanmak için başlık dosyalarını bulup bağlayıcıya `-l` ile haber vermemiz, C++'ta ise çoğu zaman CMake gibi ek araçlar kullanmamız gerekirdi. .NET'te bu iş SDK'nın içindedir.

> **Not · .NET sürümleri ve bu kitap**
>
> .NET'in her sürümünün bir destek süresi vardır. Çift numaralı sürümler **LTS** (Long-Term Support, uzun süreli destek) alır. .NET 8 (Kasım 2023) bir LTS sürümüdür; desteği **Kasım 2026'da sona eriyor**. Güncel LTS sürümü **.NET 10**'dur (Kasım 2025, C# 14). Yeni bir kurulum yapacaksak .NET 10 SDK'yı seçmek uygundur; kurulum adımları Bölüm 3'te anlatılan yolla aynıdır, yalnızca sürüm numarası değişir.
>
> Bu kitaptaki C# örnekleri **C# 12** ile yazılmış ve .NET 8 üzerinde doğrulanmıştır. C# yeni sürümlerde eski kodu bozmayacak biçimde ilerlediği için aynı örnekler .NET 8 ve sonrasındaki bütün sürümlerde (.NET 9, .NET 10) çalışır. Yeni sürümlerde gelen özelliklere, gerektiği yerde ayrıca değineceğiz.

41.4 Yönetilen Bellek ve Çöp Toplayıcı

C'de bir `malloc` çağrısı yaptıysak `free` çağırmak bizim işimizdi; unutursak bellek sızardı, iki kez çağırırsak program çökebilirdi (Bölüm 13). C++'ta aynı işi RAII ve akıllı işaretçilerle otomatikleştirdik (Bölüm 29). C# bu sorunu başka bir yöntemle çözer: bellek yönetimini çalışma zamanına bırakır.

> **Tanım: Çöp toplayıcı**
>
> Çöp toplayıcı (garbage collector, GC), programın artık ulaşamayacağı nesneleri bulup kapladıkları belleği geri alan çalışma zamanı bileşenidir. Programcı bellek ayırır (`new`), ama bırakmaz.

Bir nesneyi `new` ile oluşturduğumuzda CLR onu **yönetilen öbekte** (managed heap) ayırır. Hiçbir değişken ya da başka bir nesne artık o nesneyi göstermiyorsa nesneye ulaşılamaz demektir; GC bir sonraki çalışmasında bu nesneyi toplar. Sarkan işaretçi (dangling pointer), çifte `free` ya da `free` unutmaktan doğan sızıntı bu modelde, normal kodda ortaya çıkmaz.

GC'nin bazı özellikleri şunlardır:

- **Zamanı belirsizdir.** Nesne ulaşılamaz hâle gelir gelmez toplanmaz; GC uygun gördüğü bir anda çalışır. C++'taki yıkıcının "kapsam bitince çalışır" garantisi burada yoktur.
- **Nesneler kuşaklara ayrılır.** Yeni nesneler 0. kuşakta başlar ve toplamadan sağ çıkanlar sonraki kuşaklara taşınır. Çoğu nesne kısa ömürlü olduğundan, GC çoğu zaman yalnızca küçük 0. kuşağı tarar, bu da hızlıdır.
- **Bellek sıkıştırılabilir.** GC canlı nesneleri yan yana taşıyarak öbekteki boşlukları kapatır. Bu yüzden öbekteki bir nesnenin adresi sabit değildir; referanslar GC tarafından güncellenir.

Davranışı küçük bir programla gözleyelim. `WeakReference`, nesneyi canlı tutmayan zayıf bir referanstır; nesne toplandığında `IsAlive` özelliği `false` olur. Metin içine `{...}` ile değer yerleştiren `$"..."` yazımını Bölüm 43'te işleyeceğiz; şimdilik çıktıdaki sonuca bakmamız yeterli. Nesneyi ayrı bir metotta oluşturuyoruz ki ona giden güçlü referans, metot bitince ortadan kalksın:

gcGozlem.cs

```
using System.Runtime.CompilerServices;

WeakReference zayif = Olustur();
Console.WriteLine($"GC'den önce: {zayif.IsAlive}");

GC.Collect();
GC.WaitForPendingFinalizers();
Console.WriteLine($"GC'den sonra: {zayif.IsAlive}");

[MethodImpl(MethodImplOptions.NoInlining)]
static WeakReference Olustur()
{
    var veri = new byte[1_000_000];
    veri[0] = 1;
    return new WeakReference(veri);
}
```

ÇIKTI

```
GC'den önce: True
GC'den sonra: False
```

Nesne, `Olustur` metodu döndükten sonra hiçbir güçlü referansla tutulmadığı için `GC.Collect()` çağrısında toplandı. Gerçek programlarda `GC.Collect()` çağırmayız; GC'nin kendi zamanlamasına güveniriz. Burada yalnızca gözlem amacıyla kullandık. `[MethodImpl(...)]` satırı, derleyicinin küçük metodu çağıran yere gömmesini engeller; ayrıntıya girmeyelim.

> **▲ Sık Yapılan Hata**
>
> GC yalnızca **belleği** yönetir. Dosya tanıtıcısı, ağ bağlantısı, veritabanı bağlantısı gibi işletim sistemi kaynakları GC'nin zamanlamasına bırakılamaz. Bunlar için `IDisposable` arayüzü ve `using` deyimi vardır (Bölüm 53). C++'ın RAII'si kaynağı kapsam bitince otomatik bırakırdı; C#'ta `using` yazmak bizim sorumluluğumuzdadır.

Değer tipleri ile referans tipleri arasındaki ayrımı ve bunun bellekteki sonuçlarını Bölüm 42'de ayrıntılı göreceğiz. Bu noktada bilmemiz gereken, `new` ile öbekte oluşturulan nesnelerin çöp toplayıcı tarafından yönetildiği ve elle serbest bırakılmadığıdır.

41.5 İlk Program: Main ve Top-Level Statements

Kurulumu Bölüm 3'te yaptık ve orada üç dilde "Merhaba, Dünya!" programını çalıştırdık. Şimdi C# programının yapısına yakından bakalım. İlk olarak programı bütün parçalarıyla yazalım:

merhabaKlasik.cs

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Merhaba, Dünya!");
    }
}
```

```
ÇIKTI
Merhaba, Dünya!
```

Bu, C# programının klasik biçimidir. Parçalarını sırayla inceleyelim:

Parça	Anlamı
`using System;`	`System` ad alanındaki tipleri (`Console` gibi) kısa adlarıyla kullanmamızı sağlar. C'deki `#include <stdio.h>` satırına benzer görünür; ama dosya eklemez, yalnızca adları kısaltır
`class Program`	Program adlı bir sınıf tanımlar. C#'ta bütün kod bir tipin içinde bulunur; tek başına duran fonksiyon yoktur
`static void Main(string[] args)`	Programın giriş noktasıdır. CLR programı başlatırken ilk bu metodu çağırır. `args`, komut satırı argümanlarını taşır
`Console.WriteLine(...)`	`Console` sınıfının `WriteLine` metodunu çağırır; metni yazar ve satır sonuna geçer
`{ ... }`	Bir bloğu sınırlar. Kitapta, C# geleneğine uygun olarak açılış parantezi ayrı satırda yazılır

C'deki `int main(void)` fonksiyonuna benzer biçimde buradaki `Main` de giriş noktasıdır. Farklar şunlardır: `Main` büyük `M` ile yazılır (C# büyük/küçük harf ayrımı yapar), bir sınıfın üyesi olmak zorundadır ve `static` olmalıdır. `static` ve `class` sözcüklerinin tam anlamını Bölüm 46 ve Bölüm 49'da öğreneceğiz; şimdilik "program buradan başlar" bilgisi yeterlidir.

`Main` farklı biçimlerde yazılabilir: `static void Main()`, `static int Main(string[] args)` (C'deki gibi bir çıkış kodu döndürür) ya da eşzamansız programlar için `static async Task Main()`. Hepsi geçerlidir; ama bir projede yalnızca bir giriş noktası bulunmalıdır.

Top-level statements

Bir ilk programın bu kadar iskelet gerektirmesi C# 9'a kadar sık eleştirilirdi. C# 9 ile **üst düzey deyimler** (top-level statements) geldi: dosyaya doğrudan deyimleri yazarız, derleyici bunları bizim yerimize bir `Main` metodunun içine koyar.

```
▸ merhaba.cs
Console.WriteLine("Merhaba, Dünya!");
```

```
ÇIKTI
Merhaba, Dünya!
```

Bu tek satır, yukarıdaki sekiz satırlık programla aynı işi yapar ve aynı IL'e derlenir. `using System;` satırı da gerekmez; nedenini biraz sonra göreceğiz. Daha önce çalıştırdığımız `assemblyBilgi.cs` programında `<Main>$` adlı bir metot görmüştük: derleyici, üst düzey deyimlerimizi `<Program>$` adlı gizli bir sınıfın içinde `<Main>$` adıyla bir metoda koyar. Adların `<` ve `$` içermesi, kendi kodumuzdaki adlarla çakışmamaları içindir.

Top-level statements biçiminde de komut satırı argümanlarına `args` adıyla ulaşırız ve `return` ile çıkış kodu verebiliriz:

```
▸ argumanlar.cs
Console.WriteLine($"Argüman sayısı: {args.Length}");
Console.WriteLine($"Argümanlar: {string.Join(", ", args)}");
return 0;
```

```
ÇIKTI
Argüman sayısı: 0
Argümanlar:
```

Metnin başındaki `$` işareti, süslü parantez içindeki ifadelerin değerini metne yerleştirir (string interpolation); Bölüm 43'te ayrıntılı işleyeceğiz. Programa argüman vermek için `dotnet run` komutundan sonra `--` yazıp argümanları sıralarız:

```
TERMİNAL
$ dotnet run -- Ayşe Can
Argüman sayısı: 2
Argümanlar: Ayşe, Can
```

Top-level statements için birkaç kural vardır:

- Bir projede yalnızca **bir** dosya üst düzey deyim içerebilir.
- `using` satırları kodun en başında, deyimlerden önce yazılır.
- Sınıf, yapı ve metot tanımları deyimlerden **sonra** gelir. Önce bir sınıf tanımlayıp ardından deyim yazarsak derleyici durur:

sonradanKod.cs

```
class Selam { }
Console.WriteLine("Merhaba");
```

```
DERLEYİCİ ÇIKTISI

sonradanKod.cs(2,1): error CS8803: Top-level statements must precede namespace and type
declarations.
```

Kitabın bundan sonraki örnekleri genellikle top-level statements biçimindedir; yardımcı sınıflar ve metotlar ana kodun altında yer alır. Klasik biçim de aynı derecede geçerlidir ve gerçek projelerde, özellikle büyük uygulamalarda çok görülür; ikisini de tanımalıyız.

▲ Sık Yapılan Hata

Klasik biçimde `Main` yerine `main` yazarsak program derlenmez: derleyici giriş noktası bulamaz. C'de benzer hata bağlama aşamasında `undefined reference to 'main'` olarak görünürdü (Bölüm 4); C#'ta derleme aşamasında çıkar.

kucukMain.cs

```
class Program
{
    static void main()
    {
        Console.WriteLine("Merhaba");
    }
}
```

```
DERLEYİCİ ÇIKTISI

error CS5001: Program does not contain a static 'Main' method suitable for an entry point
```

41.6 Ad Alanları ve using

Büyük programlar yüzlerce tip içerir ve farklı kütüphanelerdeki tiplerin adları çakışabilir. C'de bu sorun ad önekleriyle (`stdio_`, `lib_`) çözülürdü; C++'ta `namespace` geldi (Bölüm 21). C# de aynı kavramı kullanır: **ad alanı** (namespace), ilgili tipleri bir ad altında toplar.

> **Tanım: Ad alanı**
> Tiplerin adlarını gruplamaya ve çakışmaları önlemeye yarayan mantıksal bir kapsamdır. Bir tipin tam adı, ad alanı ile tip adının noktayla birleşiminden oluşur: `System.Console`.

`Console` tipi `System` ad alanındadır; tam adı `System.Console`'dur. İki yolla kullanabiliriz: tam adıyla (`System.Console.WriteLine("...")`) ya da dosyanın başına `using System;` yazarak kısa adıyla (`Console.WriteLine("...")`). C++'taki `using namespace std;` satırının karşılığı budur. C++ ile bir fark: C#'taki `using` yalnızca ad çözümler; kod ekleyen bir `#include` karşılığı değildir. Tiplerin bulunduğu kütüphaneyi projeye eklemek ayrı bir iştir (`.csproj` içindeki referanslar).

Aşağıdaki program, kendi ad alanımızı tanımlamayı ve `using` direktifinin üç biçimini bir arada gösterir. Sınıf ve metot sözdizimini Bölüm 46 ve Bölüm 49'da öğreneceğiz; şimdilik yalnızca ad alanı kısmına bakalım:

▸ adAlani.cs

```
using System.Text;                  // StringBuilder için
using static System.Math;           // Math.Max yerine doğrudan Max
using Yol = System.IO.Path;         // uzun adlara kısa takma ad

Console.WriteLine(Max(3, 8));
Console.WriteLine(Yol.GetExtension("rapor.txt"));
var sb = new StringBuilder("Merhaba");
sb.Append(", ").Append(Geometri.Daire.Alan(2));
Console.WriteLine(sb);

namespace Geometri
{
    static class Daire
    {
        public static double Alan(double r)
        {
            return Math.Round(Math.PI * r * r, 2);
        }
    }
}
```

ÇIKTI

```
8
.txt
Merhaba, 12.57
```

Üç `using` biçimi şöyledir:

- `using System.Text;` ad alanındaki bütün tipleri kısa adlarıyla kullanılabilir yapar.
- `using static System.Math;` bir tipin `static` üyelerini sınıf adı olmadan çağırmamızı sağlar.
- `using Yol = System.IO.Path;` bir ada takma ad (alias) verir.

Kendi tiplerimizi bir ad alanına koymak için `namespace Ad { ... }` yazarız. C# 10 ile gelen **dosya kapsamlı ad alanı** biçimi ise `namespace Geometri;` satırını dosyanın başına koyar ve dosyadaki bütün tipleri o ad alanına katar; ek süslü parantez ve girinti gerektirmez. Yeni projelerde bu biçim yaygındır.

> ◆ **Kritik Nokta**
>
> **Ad alanı ile assembly aynı şey değildir.** Ad alanı, adlandırma için mantıksal bir gruptur; assembly ise tiplerin fiziksel olarak bulunduğu dosyadır (`.dll`). Bir ad alanının tipleri birden çok assembly'ye dağılabilir ve bir assembly birden çok ad alanı içerebilir. Önceki örnekte `List<int>` tipinin `System.Collections.Generic` ad alanında olduğunu gördük; tipin hangi assembly'de olduğu ayrı bir bilgidir.

Örtük using'ler

`merhaba.cs` programında `using System;` yazmamıştık, ama `Console` yine de bulundu. Bunun nedeni proje dosyasındaki `ImplicitUsings` ayarıdır (Bölüm 3). Bu ayar açıkken SDK, bir konsol projesindeki her dosyaya şu ad alanlarını kendiliğinden ekler:

`System`, `System.Collections.Generic`, `System.IO`, `System.Linq`, `System.Net.Http`, `System.Threading`, `System.Threading.Tasks`

Bu yüzden `List<int>` yazarken ya da `File.ReadAllText(...)` çağırırken ek bir `using` gerekmez. `using System;` satırı yazmak zararsızdır ve program yine derlenir. Başka bir ad alanı gerektiğinde (örneğin `System.Text`) satırı kendimiz yazarız. Kitabın örneklerinde, örtük olarak gelmeyen ad alanları için `using` satırı gösterilir.

41.7 Konsola Yazmak, Yorumlar ve Derleyici Mesajları

C bölümünde `printf`, C++'ta `std::cout` ile yazmayı öğrenmiştik. C#'ta temel iki metot `Console.Write` ve `Console.WriteLine`'dır. `WriteLine` metni yazar ve satırın sonuna geçer, `Write` ise aynı satırda kalır. Parametresiz `Console.WriteLine()` yalnızca boş bir satır bırakır. Girdi almayı ve biçimlendirmeyi Bölüm 43'te ayrıntılı işleyeceğiz.

Metinler çift tırnakla yazılır ve C'dekine çok benzeyen **kaçış dizileri** (escape sequence) kullanır:

Kaçış dizisi	Anlamı
`\n`	Satır sonu
`\t`	Sekme (tab)
`\"`	Çift tırnak karakteri
`\'`	Tek tırnak karakteri
`\\`	Ters bölü karakterinin kendisi
`\0`	Boş (null) karakter
`\u00E7`	Onaltılık kodu verilen Unicode karakteri (`ç`)

C'den bir fark: C#'ta `\101` gibi sekizlik kaçış dizisi yoktur; Unicode karakterler için `\u` ya da `\U` kullanılır. Tek tırnak yalnızca tek bir karakter (`'a'`) içindir; `'Selam'` yazarsak derleyici hata verir.

yazdirma.cs

```
Console.Write("Bir ");
Console.Write("İki ");
Console.WriteLine("Üç");
Console.WriteLine("Ad\tSoyad");
Console.WriteLine("\"Merhaba\" dedi.");
Console.WriteLine("Yol: C:\\ders\\cs");
Console.WriteLine(@"Yol: C:\ders\cs");
Console.WriteLine("\u00E7ay");
Console.WriteLine("Satır 1" + Environment.NewLine + "Satır 2");
```

ÇIKTI

```
Bir İki Üç
Ad      Soyad
"Merhaba" dedi.
Yol: C:\ders\cs
Yol: C:\ders\cs
çay
Satır 1
Satır 2
```

Metnin önüne `@` koyarsak (**verbatim string**) içindeki ters bölüler olduğu gibi yazılır; dosya yollarında işe yarar. `Environment.NewLine` ise çalışılan sistemin satır sonu dizisidir (Windows'ta `\r\n`, Linux ve macOS'ta `\n`). Çıktıda Türkçe karakterler bozuk görünürse Bölüm 3'te anlatılan UTF-8 ayarına bakmalıyız.

Metin ile sayıyı `+` ile birleştirirken işlem soldan sağa yapılır; bir metinle karşılaşıldığı andan sonra `+` birleştirme olur:

birlestirme.cs

```
Console.WriteLine("S" + 4 + 5);
Console.WriteLine(4 + 5 + "S");
Console.WriteLine("S" + (4 + 5));
```

ÇIKTI

```
S45
9S
S9
```

Yorum satırları

Yorumlar derleyici tarafından yok sayılır. C'deki iki biçim aynen vardır, bir de üçüncüsü eklenmiştir:

```
// Tek satırlık yorum
int a = 5;   // kodun yanına da yazılabilir

/* Çok satırlı yorum.
   İç içe yazılamaz: ilk */ yorumu kapatır. */

/// <summary>Belge yorumudur; metotların üstüne yazılır.</summary>
```

`///` ile başlayan **XML belge yorumları**, editörün bir metodun üzerine gelindiğinde açıklama göstermesini sağlar; metotları öğrendiğimizde kullanacağız.

Derleyici mesajlarını okumak

Bir programda yazım hatası yaparsak derleyici derlemeyi durdurur ve her sorun için bir satır yazar. Mesajın biçimi şöyledir:

```
dosyaAdı(satır,sütun): error CSxxxx: açıklama
```

`CS` ile başlayan sayı hata kodudur. Mesajın dili sistemin diline göre değişebilir; ama kod her dilde aynıdır. Bir hatayı araştırırken açıklamayı değil kodu aratmak daha verimlidir. Bölüm 42'den itibaren bu biçimde mesajlar göreceğiz. En sık karşılaşılan iki örneğe bakalım. Birincisi, noktalı virgülü unutmaktır:

`eksikNoktali.cs`

```
Console.WriteLine("Birinci satır");
Console.WriteLine("İkinci satır")
Console.WriteLine("Üçüncü satır");
```

DERLEYİCİ ÇIKTISI

```
eksikNoktali.cs(2,34): error CS1002: ; expected
```

Derleyici hatanın yerini, `;` beklenen noktayı, yani ikinci satırın sonunu gösterir. İkincisi büyük/küçük harf karışıklığıdır:

`kucukHarf.cs`

```
console.WriteLine("Merhaba");
```

DERLEYİCİ ÇIKTISI

```
kucukHarf.cs(1,1): error CS0103: The name 'console' does not exist in the current context
```

C# büyük ve küçük harfi ayırır; `Console` ile `console` farklı adlardır. .NET'in hazır adları her sözcüğün ilk harfi büyük yazılarak (`WriteLine`, `ReadLine`) kurulur. Birden çok hata varsa en üsttekinden başlamak gerekir; çoğu zaman diğerleri ona bağlıdır.

Derleme sırasında hata değil de uyarı çıkabilir: `warning CSxxxx`. Uyarı, kodun derlendiğini ama bir sorun olabileceğini söyler (örneğin hiç kullanılmayan bir değişken). Programı durdurmaz; ama C bölümünde `-Wall` ile öğrendiğimiz gibi uyarıları da gidermek iyi alışkanlıktır. Çalışırken ortaya çıkan hatalar ise **istisna** (exception) olarak görünür; bunları Bölüm 53'te işleyeceğiz.

41.8 Proje Yapısı ve dotnet Araçları

Bölüm 3'te `dotnet new console`, `dotnet build` ve `dotnet run` komutlarını kullanmış, `.csproj` dosyasının `TargetFramework`, `ImplicitUsings` ve `Nullable` ayarlarını görmüştük. Burada o bilgiyi proje bakış açısıyla genişletelim.

C'de derlenecek dosyaları komut satırında tek tek yazardık (`gcc a.c b.c -o program`). .NET'te bunu **proje dosyası** (`.csproj`) üstlenir. Proje klasöründeki bütün `.cs` dosyaları kendiliğinden derlemeye katılır; yeni dosya eklemek için hiçbir listeyi güncellemeyiz. Proje dosyası ayrıca hedef sürümü, bağımlı paketleri ve derleme ayarlarını tutar:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <LangVersion>12</LangVersion>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="PaketAdi" Version="1.2.3" />
  </ItemGroup>

</Project>
```

`OutputType` projenin çalıştırılabilir (`Exe`) mi yoksa başka projelerin kullanacağı bir kütüphane (`Library`) mi olduğunu belirler. `LangVersion` kullanılacak C# sürümünü sabitler; yazılmazsa hedef çerçeveye uygun varsayılan sürüm kullanılır. `PackageReference` satırları NuGet paketlerini listeler; `dotnet add package` komutu bu satırı bizim için ekler.

Günlük işlerde sık kullandığımız komutları bir araya toplayalım:

Komut	Ne yapar?
`dotnet new list`	Kullanılabilir proje şablonlarını listeler
`dotnet new console -o Ad`	`Ad` klasöründe konsol projesi oluşturur
`dotnet new classlib -o Ad`	Kütüphane projesi (`.dll`) oluşturur
`dotnet build`	Projeyi derler; çıktı `bin/Debug/net8.0/` altına yazılır
`dotnet run`	Gerekirse derler, programı çalıştırır
`dotnet run -- a b`	Programa `a` ve `b` argümanlarını geçirir
`dotnet add package Ad`	NuGet paketi ekler
`dotnet build -c Release`	Eniyilemeli (Release) yapılandırmayla derler
`dotnet publish -c Release`	Dağıtıma hazır dosyaları `publish` klasörüne koyar
`dotnet clean`	Derleme çıktılarını siler

Bölüm 3'te `bin` ve `obj` klasörlerini görmüştük. İçeride `Debug` ve `Release` olmak üzere iki **yapılandırma** (configuration) bulunur. Debug, hata ayıklamaya uygun, az eniyilenmiş kod üretir; Release ise eniyilemeleri açar ve yayın için kullanılır. `dotnet publish` komutuyla programı başka bilgisayara götürmek için gereken dosyalar bir klasörde toplanır. Hedef bilgisayarda .NET Runtime kurulu olmalıdır; kurulu olmasını istemiyorsak `--self-contained` seçeneğiyle çalışma zamanını da pakete katabiliriz. Bu durumda paket büyür ama kurulum gerektirmez.

Birden çok projeyi (örneğin bir uygulama ve onun kütüphanesi) birlikte yönetmek için **çözüm dosyası** (`.sln`) kullanılır; `dotnet new sln` ile oluşturulur. Bu kitapta örnekler tek projede olacağından çözüm dosyasına ihtiyaç duymayacağız.

İpucu · Visual Studio ile çalışmak

Visual Studio'da aynı işleri menülerle yaparız: **Create a new project** ile **Console App** şablonu seçilir (adında ".NET Framework" geçen şablon eski ve yalnızca Windows'a özgüdür; seçmeyelim), framework olarak yüklü .NET sürümü işaretlenir ve **Ctrl+F5** programı çalıştırır. Yeni proje penceresindeki **Do not use top-level statements** kutusu boş bırakılırsa dosya top-level biçiminde, işaretlenirse klasik `Main` biçiminde oluşur. Arka planda Visual Studio de aynı `.csproj` dosyasını ve aynı derleyiciyi kullanır; dolayısıyla bir proje iki ortamda da açılabilir. Ayrıntılar Bölüm 3'tedir.

Not · Tek dosyalık çalıştırma

.NET 10 SDK ile proje dosyası oluşturmadan tek bir dosyayı doğrudan çalıştırabiliriz: `dotnet run Program.cs`. Küçük denemeler için pratiktir. .NET 8 SDK'da bu biçim yoktur; bu kitapta örnekleri, Bölüm 3'te anlatıldığı gibi bir proje içinde çalıştırmayı varsayıyoruz.

41.9 C# Nerede Kullanılır?

.NET'in çapraz platform ve açık kaynaklı olması, C#'ı çok geniş bir alanda kullanılabilir yaptı. Başlıca alanlar şunlardır:

Alan	Teknoloji	Not
Web siteleri ve API'ler	ASP.NET Core	Sunucu tarafı; Blazor ile C# kodu tarayıcı arayüzünde de kullanılabilir
Oyun	Unity, Godot	Unity'nin betikleri C# ile yazılır. Unity kendi .NET çalışma ortamını kullandığından desteklediği C# sürümü bu kitabınkinden farklı olabilir
Masaüstü	WPF, Windows Forms	Yalnızca Windows'ta çalışır
Çok platformlu arayüz	.NET MAUI	Tek kod tabanıyla Android, iOS, macOS ve Windows uygulamaları
Bulut ve servisler	Azure Functions, konteynerler, arka plan servisleri	Bulut hizmetlerinin yanı sıra herhangi bir bulutta çalışır
Konsol ve araçlar	Konsol uygulamaları, otomasyon betikleri	Bu kitabın konusu
Veri ve yapay zekâ	ML.NET, veritabanı erişim kütüphaneleri	Büyük veri işleme genellikle başka dillerle birlikte kullanılır

Bu alanların her biri BCL'in üzerine ek kütüphaneler koyar; ama dilin kendisi aynıdır. Bu kitapta konsol uygulamalarıyla ilerleyeceğiz: dilin temelini bu zeminde öğrenmek, sonra herhangi bir alana geçmek en kolay yoldur.

Üç Dilde · Üç dilde merhaba ve derleme modeli *C · C++ · C#*

Aynı programın üç dildeki biçimi şöyledir:

```c
#include <stdio.h>

int main(void) {
```

```
    printf("Merhaba, Dünya!\n");
    return 0;
}
```

```cpp
#include <iostream>

int main() {
    std::cout << "Merhaba, Dünya!\n";
}
```

```
Console.WriteLine("Merhaba, Dünya!");
```

C ve C++'ta metin bir fonksiyona ya da akışa gönderilir; başlık dosyası eklemek gerekir. C#'ta ise ad alanı `System` kendiliğinden eklenir ve `Main` iskeleti derleyici tarafından üretilir. Derleme ve çalıştırma modelleri de şöyle ayrılır:

Özellik	C	C++	C#
Derleyicinin çıktısı	makine kodu	makine kodu	IL içeren assembly (`.dll`)
Makine koduna çevirme	derleme zamanında	derleme zamanında	çalışırken (JIT) ya da önceden (ReadyToRun, Native AOT)
Tip bilgisi	başlık dosyası	başlık dosyası (C++20'de modül)	assembly'nin üst verisi
Parçaların birleştirilmesi	bağlayıcı	bağlayıcı	çalışma zamanı assembly'leri yükler
Bellek yönetimi	elle `malloc` / `free`	RAII, akıllı işaretçiler	çöp toplayıcı
Taşınabilirlik	her platform için yeniden derleme	her platform için yeniden derleme	aynı `.dll` platformlar arasında çalışır (platforma özgü API kullanılmadıkça)
Çalıştırmak için	işletim sistemi yeter	işletim sistemi yeter	.NET Runtime gerekir (ya da paketin içindedir)
Derleme komutu	`gcc ... -o ad`	`g++ ... -o ad`	`dotnet build`
Çalıştırma komutu	`./ad`	`./ad`	`dotnet run`

Bölüm Özeti

- C#, güvenli, yönetilen, nesne yönelimli ve üretken olacak biçimde tasarlanmıştır. C# bir dildir; .NET ise onu derleyen ve çalıştıran platformdur.
- C# kaynak kodu Roslyn derleyicisiyle IL'e çevrilir ve bir assembly'ye (`.dll` / `.exe`) yazılır. Program çalışırken CLR içindeki JIT, IL'i makine koduna çevirir. C ve C++ ise doğrudan makine koduna derlenir.
- Assembly, IL ile birlikte üst veri taşır; bu yüzden C#'ta başlık dosyası yoktur. Ortak tip sistemi (CTS) sayesinde farklı .NET dilleri aynı tipleri paylaşır.
- SDK, Runtime'ı ve derleyiciyi içerir; program yazmak için SDK, yalnızca çalıştırmak için Runtime gerekir. BCL hazır tipleri sağlar, NuGet dış paketleri getirir.
- .NET 8 desteği Kasım 2026'da biter; güncel LTS sürümü .NET 10'dur. Kitaptaki örnekler C# 12 ile yazılmıştır ve .NET 8 ile sonrasında çalışır.

- Bellek çöp toplayıcı (GC) tarafından yönetilir; `free` ya da `delete` karşılığı yoktur. GC'nin zamanı belirsizdir ve yalnızca belleği yönetir; dosya, bağlantı gibi kaynaklar için `IDisposable` ve `using` gerekir.
- Klasik program `class Program { static void Main() }` biçimindedir. Top-level statements ile deyimler doğrudan yazılır; derleyici `<Main>$` metodunu üretir. Üst düzey deyimler tip tanımlarından önce gelmelidir.
- `using` ad alanındaki tipleri kısa adlarıyla kullanmayı sağlar; kod eklemez. `using static` ve takma ad biçimleri de vardır. Ad alanı mantıksal bir gruptur, assembly fiziksel bir dosyadır.
- `ImplicitUsings` açıkken `System`, `System.Linq`, `System.IO` gibi yaygın ad alanları her dosyaya kendiliğinden eklenir.
- `Console.Write` aynı satırda kalır, `Console.WriteLine` satırı bitirir. Kaçış dizileri C'dekine benzer (`\n`, `\t`, `\"`, `\\`); sekizlik kaçış yoktur, `\u` vardır. `@"..."` kaçış dizilerini kapatır.
- Derleyici mesajları `dosya(satır,sütun): error CSxxxx: açıklama` biçimindedir. C# büyük/küçük harf ayırt eder (`Console` ve `console` farklıdır).
- `dotnet new`, `dotnet build`, `dotnet run` ve `dotnet publish` günlük komutlardır. `.csproj` hedef sürümü, ayarları ve paketleri tutar; klasördeki bütün `.cs` dosyaları kendiliğinden derlenir.
- C# web (ASP.NET Core), oyun (Unity), masaüstü (WPF, Windows Forms, MAUI), bulut ve konsol uygulamalarında kullanılır.

Alıştırmalar

1. C# derleyicisi (Roslyn) bir `.cs` dosyasını derlediğinde asıl çıktı olarak ne üretir?
   A) Doğrudan işlemcinin makine kodunu içeren çalıştırılabilir dosya
   B) IL kodu ve üst veri içeren bir assembly
   C) Yalnızca bir başlık dosyası
   D) Kaynak kodun şifrelenmiş bir kopyasını
   E) Çöp toplayıcının ayar dosyasını
2. IL kodunu program çalışırken makine koduna çeviren bileşen hangisidir?
   A) Roslyn
   B) NuGet
   C) JIT derleyicisi
   D) `dotnet new`
   E) `.csproj` dosyası
3. .NET SDK ile .NET Runtime arasındaki fark hangisidir?
   A) İkisi aynı pakettir
   B) Runtime derleyici içerir, SDK içermez
   C) SDK, Runtime'a ek olarak C# derleyicisini ve `dotnet` araçlarını içerir
   D) SDK yalnızca Windows'ta bulunur
   E) Runtime yalnızca Linux içindir
4. Çöp toplayıcı (GC) için hangisi doğrudur?
   A) Nesnenin son referansı kaybolur kaybolmaz o nesneyi toplar
   B) Dosya ve ağ bağlantıları gibi kaynakları da otomatik kapatır
   C) Programcıdan `free` çağrısı bekler
   D) Yönetilen öbekteki ulaşılamayan nesnelerin belleğini, kendi belirlediği bir anda geri alır
   E) Yalnızca Windows'ta çalışır

5. Top-level statements ile ilgili hangisi doğrudur?
   A) Birden çok dosya aynı projede üst düzey deyim içerebilir
   B) Üst düzey deyimler, sınıf tanımlarından önce yazılmalıdır
   C) Derleyici `Main` metodu üretmez; program hiçbir yerden başlamaz
   D) `args` değişkenine erişilemez
   E) Yalnızca .NET Framework'te çalışır
6. Aşağıdakilerden hangisi doğrudur?
   A) Ad alanı ile assembly aynı şeydir
   B) `using System;` satırı `System` kütüphanesini projeye ekler
   C) Bir ad alanındaki tipler birden çok assembly'de bulunabilir
   D) `using` satırı yalnızca dosyanın sonuna yazılır
   E) `using static`, bir sınıfın örneğini oluşturur
7. Projenin hedef .NET sürümü ve ayarları hangi dosyada tutulur?
   A) `Program.cs`
   B) `bin/Debug`
   C) `obj` klasöründe
   D) `.csproj` dosyasında
   E) `README.md` dosyasında
8. Aşağıdaki program ekrana ne yazdırır?

```
Console.Write("A");
Console.WriteLine("B");
Console.Write("C\n");
Console.WriteLine("D\tE");
Console.WriteLine("\"F\"");
```

9. Aşağıdaki program ekrana ne yazdırır?

```
Console.WriteLine("x" + 1 + 2);
Console.WriteLine(1 + 2 + "x");
Console.WriteLine("x" + (1 + 2));
Console.WriteLine(1 + 2 * 3 + "x");
```

10. Aşağıdaki program ekrana ne yazdırır?

```
using static System.Math;
using Dizin = System.IO.Path;

Console.WriteLine(Max(4, 9) - Min(4, 9));
Console.WriteLine(Dizin.GetExtension("not.txt"));
Console.WriteLine(Abs(-2.5));
```

11. Aşağıdaki programdaki hataları bulun ve düzeltin.

```
console.WriteLine("Merhaba")
Console.WriteLine("Dünya");
```

12. Aşağıdaki program neden derlenmez? Nasıl düzeltilir?

```
class Kutu
{
    public int Deger = 5;
}

Console.WriteLine("Başladı");
```

13. Aşağıdaki klasik biçimli programı top-level statements biçimine çevirin.

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Toplam: " + (7 + 8));
        Console.WriteLine("Argüman sayısı: " + args.Length);
    }
}
```

14. Tek bir `Console.WriteLine` çağrısı ve kaçış dizileri kullanarak aşağıdaki çıktıyı üreten programı yazın (ilk satırdaki sütunlar sekme ile ayrılmıştır).

```
Kod     Ad
"cs"    C#
C:\proje\cs
```

15. Komut satırından aldığı ilk argümanı selamlayan bir program yazın: argüman verilmişse `Merhaba, <argüman>!`, verilmemişse `Merhaba, Dünya!` yazsın. Programı klasik `Main` biçiminde yazın.

BÖLÜM 42

Değişkenler, Veri Tipleri ve Operatörler

BU BÖLÜMDE

- Değişken tanımlamayı, ilk değer vermeyi ve isimlendirme kurallarını
- Değer tipleri ile referans tiplerinin farkını
- C#'ın temel veri tiplerini ve sabit boyutlarını (`int` her zaman 32 bit)
- Literal yazımlarını (`1_000_000`, `0xFF`, `0b1010`, `m`, `f`, `L` ekleri), `var` ve `const` kullanımını
- Aritmetik, karşılaştırma, mantıksal ve bit operatörlerini; C ile farklarını
- Örtük ve açık dönüşümleri, `Convert`, `Parse`, `TryParse`, `checked` ve `unchecked` denetimini
- Nullable değer tiplerini (`int?`, `??`, `??=`)

Bölüm 41'de ilk C# programımızı yazdık ve bir C# kaynak dosyasının IL koduna, oradan da CLR aracılığıyla makine koduna nasıl dönüştüğünü gördük. Şimdi dilin yapı taşlarına geçiyoruz: verileri bellekte saklayan değişkenlere, bu verilerin tiplerine ve onlar üzerinde işlem yapan operatörlere.

C ve C++ bölümlerinden gelen bir okur için bu konuların büyük kısmı tanıdık olacak. Yine de C# aynı görünen yerlerde daha sıkı kurallar koyar. Tip boyutları dilin kendisi tarafından sabitlenmiştir, `bool` bir sayı değildir, değeri verilmemiş bir değişken okunamaz, ifadelerin değerlendirme sırası tanımlıdır. C ve C++'ta "tanımsız davranış" (undefined behavior) bırakılan birçok durum burada ya derleme hatasıdır ya da açıkça belirtilmiş bir davranıştır. Bu bölümde bu farkları sırayla göreceğiz.

Bu bölümdeki programlar, Bölüm 41'de tanıştığımız **top-level statements** biçimindedir: `Main` metodu ve `Program` sınıfı yazmadan doğrudan deyimleri yazarız. `Console` ve `Math` gibi tipler için ayrıca `using System;` satırı gerekmez; .NET proje şablonu bu ad alanlarını örtük olarak ekler.

> **Not · Çıktılardaki ondalık ayracı**
>
> Bu kitaptaki bütün çıktılar kültür bağımsız (invariant) ayarlarla alınmıştır: ondalık ayraç **nokta**dır (`1.78`) ve `bool` değerleri `True` / `False` diye yazılır. Türkçe bölge ayarlı bir bilgisayarda aynı program `1,78` yazabilir. Bu bir hata değildir; ondalık sayıların ekrana nasıl yazıldığı bilgisayarın kültür ayarına bağlıdır. Konuyu Bölüm 43'te ayrıntılı işleyeceğiz. Kod içinde yazdığımız sayı sabitleri ise her zaman noktalıdır (`3.5`).

42.1 Değişkenler ve İsimlendirme

Bir program, üzerinde çalıştığı verileri bellekte bir yerde tutmak zorundadır: kullanıcının yaşı, bir ürünün fiyatı, bir döngüde kaç kez döndüğümüz. Bu verilere kodun içinden ulaşabilmek için onlara ad veririz.

> **Tanım: Değişken**
>
> Bellekte bir değeri saklayan, adı ve tipi olan alana değişken (variable) denir. Tip, alana hangi türden değerlerin konabileceğini ve alanın ne kadar yer kaplayacağını belirler; ad, o alana kodun içinden erişmemizi sağlar.

C#'ta bir değişken kullanılmadan önce tanımlanmalıdır (declaration). Tanımlama, önce tipi sonra adı yazarak yapılır. İlk değer tanımlama sırasında verilebilir ya da sonradan atanabilir (assignment):

```
int yas = 19;      // tanımlama ve ilk değer
double boy;        // yalnızca tanımlama
boy = 1.78;        // sonradan atama
```

Aşağıdaki programda üç değişken tanımlayıp değerlerini ekrana yazdıralım. Metnin başındaki `$` işareti, süslü parantez içine yazılan değişkenlerin değerini metne yerleştirir. Buna dize içine değer yerleştirme (string interpolation) denir; ayrıntısı Bölüm 43'te.

▸ **degiskenOrnek.cs**

```
int yas = 19;
double boy;
boy = 1.78;
string ad = "Can";

Console.WriteLine($"{ad} {yas} yaşında, boyu {boy}");
yas = 20;               // eski değer kaybolur
yas = yas + 1;          // sağ taraf önce hesaplanır: 20 + 1
Console.WriteLine($"Yeni yaş: {yas}");
```

```
ÇIKTI
Can 19 yaşında, boyu 1.78
Yeni yaş: 21
```

`yas = yas + 1;` satırı matematikte anlamsız görünür; ama programlamada `=` işareti eşitlik değil **atama** anlamına gelir: "Sağdaki ifadeyi hesapla, sonucu soldaki değişkene yaz." Önce `yas + 1` yani 21 hesaplanır, sonra sonuç `yas` değişkenine yazılır.

Aynı tipten birden çok değişkeni tek satırda, virgülle ayırarak tanımlayabiliriz:

```
int en = 4, boy = 6;     // ikisi de int
int alan = en * boy;     // sonradan tanımlanan değişken öncekileri kullanabilir
```

Kullanılmadan önce değer verilmeli

C'de yerel bir değişkene ilk değer vermezsek içinde "çöp" bir değer bulunur ve programımız bu değeri sessizce kullanır. C# bu durumu kabul etmez. Derleyici, bir yerel değişkenin okunduğu her noktada ona kesin olarak bir değer atandığını (definite assignment) kanıtlayamazsa programı derlemez:

▸ **ilkDegerYok.cs**

```
int sayi;
Console.WriteLine(sayi);
```

```
DERLEYİCİ ÇIKTISI

ilkDegerYok.cs(2,19): error CS0165: Use of unassigned local variable 'sayi'
```

Hata mesajındaki `(2,19)`, hatanın ikinci satırın on dokuzuncu karakterinde olduğunu söyler. Bu denetim yalnızca derleyicinin akış analizine dayanır; yani değişkene bir `if` içinde değer atayıp başka bir yolda atamazsak da aynı hatayı alırız.

Kapsam

Bir değişken, tanımlandığı bloğun (süslü parantez çifti) içinde yaşar. Blok bitince değişkenin adı da kullanılamaz hale gelir. İç içe bir blokta, dıştaki bloğun değişkeniyle aynı adı taşıyan yeni bir değişken tanımlayamayız. C++'ta izin verilen bu "gölgeleme" C#'ta hatadır:

▸ kapsamHata.cs

```
int x = 5;
{
    int y = 10;
    int x = 6;
}
Console.WriteLine(y);
```

```
DERLEYİCİ ÇIKTISI

kapsamHata.cs(4,9): error CS0136: A local or parameter named 'x' cannot be declared in this
scope because that name is used in an enclosing local scope to define a local or parameter
kapsamHata.cs(6,19): error CS0103: The name 'y' does not exist in the current context
```

Aynı bloğa ikinci kez aynı ad ile değişken tanımlamak da hatadır (`CS0128`). Var olan bir değişkene ise istediğimiz kadar yeni değer atayabiliriz.

İsimlendirme kuralları

Değişken, metot, sınıf gibi program öğelerine verdiğimiz adlara tanımlayıcı (identifier) denir. C# tanımlayıcıları için kurallar şunlardır:

1. Harf, rakam ve alt çizgi `_` içerebilir.
2. Rakamla başlayamaz: `2sayi` geçersiz, `sayi2` geçerlidir.
3. Boşluk ve `-`, `+`, `.`, `$` gibi özel karakterler içeremez.
4. Anahtar sözcükler (`int`, `class`, `public`, `if`, `for`, `new`, `string` …) ad olarak kullanılamaz.
5. Büyük ve küçük harf farklıdır: `sayi`, `Sayi` ve `SAYI` üç ayrı addır.

Bir anahtar sözcüğün önüne `@` koyarsak derleyici onu sıradan bir ad olarak kabul eder (`@class`). Bu yazım, başka dillerden gelen adlarla çakışmayı çözmek için vardır; kendi kodumuzda kullanmayız.

C# tanımlayıcılarda Unicode harflere izin verir; `öğrenciSayısı` geçerli bir addır. Yine de bu kitapta ve pratikte değişken adlarını Türkçe karakter kullanmadan (`ogrenciSayisi`) yazmayı tercih ediyoruz. Farklı klavyelerde, farklı dosya kodlamalarında ve farklı araçlarda sorun çıkmasını önlemenin en kolay yolu budur. Türkçe karakterleri yalnızca tırnak içindeki metinlerde kullanırız.

Kurallara ek olarak, C# topluluğunun benimsediği adlandırma gelenekleri (naming conventions) vardır. Uymasak da program çalışır; ama uyduğumuzda başkalarının yazdığı kodu okumak ve başkalarının bizim kodumuzu okuması kolaylaşır:

Ne için?	Gelenek	Örnek
Yerel değişken, parametre	camelCase: ilk sözcük küçük, sonrakiler büyük harfle başlar	`ogrenciSayisi`, `notOrtalamasi`
Metot, sınıf, özellik	PascalCase: her sözcük büyük harfle başlar	`OrtalamaHesapla`, `Ogrenci`, `Ad`
Sabit (`const`)	PascalCase	`Pi`, `MaksOgrenci`

C'de `ogrenci_sayisi` gibi alt çizgili (snake_case) adlar, standart kütüphanenin de etkisiyle yaygındır. C++ topluluğunda ise tek bir gelenek yoktur; standart kütüphane `snake_case` kullanır, birçok proje `camelCase` ya da `PascalCase` tercih eder. C#'ta .NET sınıf kütüphanesi PascalCase'i benimsemiştir ve geliştiriciler de buna uyar. Metot, sınıf ve özellik kavramlarını ilerleyen bölümlerde öğreneceğiz; şimdilik yerel değişken adlarının camelCase yazıldığını bilmemiz yeterli.

42.2 Değer Tipleri ve Referans Tipleri

C# tiplerini iki büyük aileye ayırır. Bu ayrım, dilin geri kalanını anlamak için en önemli kavramlardan biridir; çünkü atama, parametre geçirme ve karşılaştırma işlemleri iki ailede farklı çalışır.

> **Tanım: Değer tipi (value type)**
>
> Değişkeni, verinin kendisini doğrudan içinde tutan tiptir. Bir değer tipli değişken başka bir değişkene atandığında değer kopyalanır; iki değişken birbirinden bağımsız olur. `int`, `double`, `char`, `bool`, `decimal`, `struct` ve `enum` değer tipidir.

> **Tanım: Referans tipi (reference type)**
>
> Değişkeni, verinin kendisini değil, verinin bellekte bulunduğu yere giden bir bağlantıyı (referans) tutan tiptir. Atamada bağlantı kopyalanır; iki değişken aynı nesneyi gösterir. `string`, `class`, dizi (array), `delegate`, `interface` ve `object` referans tipidir.

Sezgisel bir resim şöyle çizilebilir. Değer tipli bir yerel değişken, değerini yığında (stack) kendi alanında taşır. Referans tipli bir değişken ise yığında yalnızca bir adres taşır; asıl nesne öbekte (heap) durur ve çöp toplayıcı (garbage collector) tarafından yönetilir. Bu resim işimizi görür; ama kesin bir kural değildir. Örneğin bir sınıfın içindeki `int` alanı, nesneyle birlikte öbekte yaşar. Doğru ölçüt yer değil, **davranıştır**: değer tipinde atama kopyalar, referans tipinde atama aynı nesneyi paylaştırır.

Farkı bir programda görelim. Programın altındaki `Nokta` yapısı (struct) bir değer tipi, `Kutu` sınıfı (class) bir referans tipidir. Yapı ve sınıfları Bölüm 49 ve Bölüm 52'de ayrıntılı işleyeceğiz; burada yalnızca davranışlarını karşılaştırıyoruz.

▸ degerReferans.cs

```
int x = 10;
int y = x;              // değer kopyalanır
y = 20;
Console.WriteLine($"x={x} y={y}");

Nokta n1 = new Nokta();
n1.X = 1;
Nokta n2 = n1;          // yapı: bütün alanlar kopyalanır
n2.X = 5;
```

```
Console.WriteLine($"n1.X={n1.X} n2.X={n2.X}");

Kutu k1 = new Kutu();
k1.Icerik = 10;
Kutu k2 = k1;           // sınıf: yalnızca referans kopyalanır
k2.Icerik = 20;
Console.WriteLine($"k1={k1.Icerik} k2={k2.Icerik}");
Console.WriteLine(ReferenceEquals(k1, k2));

struct Nokta { public int X; }
class Kutu { public int Icerik; }
```

```
ÇIKTI
x=10 y=20
n1.X=1 n2.X=5
k1=20 k2=20
True
```

`y = 20` ataması `x`'i etkilemedi; `n2.X = 5` de `n1`'i etkilemedi. Oysa `k2.Icerik = 20` ataması `k1.Icerik` değerini de değiştirdi, çünkü `k1` ve `k2` aynı nesneyi gösteriyor. `ReferenceEquals` iki değişkenin gerçekten aynı nesneye bağlı olup olmadığını sorar.

C ile bir benzetme kurabiliriz. C'de atama her zaman kopyalar; iki değişkenin aynı veriyi paylaşması için açıkça işaretçi (`Kutu *k2 = k1;`) kullanmamız gerekir. C#'ta sınıf tipli bir değişken zaten "işaretçi gibi" davranır; ama adres aritmetiği yoktur, bellek serbest bırakmak bizim işimiz değildir ve referans geçersiz bir adrese işaret edemez. Ya bir nesneyi gösterir ya da `null`'dır.

null

Referans tipli bir değişken hiçbir nesneyi göstermiyorsa değeri `null`'dır. Değer tipli bir değişken ise `null` olamaz; her zaman bir değeri vardır. Bu kısıtın nasıl aşıldığını bu bölümün sonunda nullable değer tipleriyle göreceğiz.

Varsayılan değerler

Her tipin bir varsayılan değeri (default value) vardır: sayısal tiplerde sıfır, `bool`'da `false`, `char`'da `'\0'`, referans tiplerinde `null`. Bir sınıfın alanlarına ve bir dizinin elemanlarına değer vermezsek bu değerler kendiliğinden atanır. Yerel değişkenler ise yukarıda gördüğümüz gibi bu güvenceden yararlanmaz; onlara değer vermek zorundayız. Bir tipin varsayılan değerini `default(tip)` ifadesiyle ya da hedef tipi belli olan yerlerde yalın `default` ile elde ederiz:

▸ varsayilanDegerler.cs

```
Console.WriteLine(default(int));
Console.WriteLine(default(double));
Console.WriteLine(default(bool));
Console.WriteLine(default(char) == '\0');
string? ad = default;
Console.WriteLine(ad == null);
int[] dizi = new int[3];
Console.WriteLine($"{dizi[0]} {dizi[1]} {dizi[2]}");
```

```
ÇIKTI
0
0
False
True
True
0 0 0
```

`string?` yazımındaki soru işareti, değişkenin `null` alabileceğini derleyiciye bildirir; nullable referans tiplerini Bölüm 59'da işleyeceğiz. Şimdilik `string?` ile `string` arasındaki farkı önemsemeyebiliriz.

> ◆ **Kritik Nokta**
>
> C# tiplerini değer tipi ya da referans tipi olarak sınıflayabilmek, ileride karşılaşacağımız birçok davranışı açıklar: metoda parametre geçirme (Bölüm 46), dizi kopyalama (Bölüm 47), `==` karşılaştırmasının anlamı ve `struct` ile `class` arasındaki seçim (Bölüm 52). Yeni bir tip gördüğümüzde ilk sorumuz "değer tipi mi, referans tipi mi?" olmalıdır.

42.3 Sayısal Tipler ve Sabit Boyutları

C'de `int` tipinin kaç bit olduğu derleyiciye ve platforma bağlıdır; standart yalnızca alt sınırlar verir. `long` tipi bir platformda 4, başka bir platformda 8 bayt olabilir. C++ bu mirası aynen devralır. C# ise bu belirsizliği ortadan kaldırmıştır: her yerleşik tipin boyutu dil belirtiminde sabittir. `int` her platformda, her işletim sisteminde 32 bittir.

Tam sayı tipleri işaretli ve işaretsiz olarak çiftler halinde bulunur:

Tip	.NET adı	Boyut	Değer aralığı
`sbyte`	`System.SByte`	1 bayt	−128 … 127
`byte`	`System.Byte`	1 bayt	0 … 255
`short`	`System.Int16`	2 bayt	−32 768 … 32 767
`ushort`	`System.UInt16`	2 bayt	0 … 65 535
`int`	`System.Int32`	4 bayt	−2 147 483 648 … 2 147 483 647
`uint`	`System.UInt32`	4 bayt	0 … 4 294 967 295
`long`	`System.Int64`	8 bayt	yaklaşık ±9,2 × 10^{18}
`ulong`	`System.UInt64`	8 bayt	0 … yaklaşık 1,8 × 10^{19}

İşaretsiz (unsigned) tipler eksi sayı tutamaz; karşılığında aynı bayt sayısıyla pozitif tarafta iki kat büyük sayılara ulaşır. Adında `u` olan tip işaretsizdir. Tek istisna `byte`'tır: Adında `u` yoktur ama işaretsizdir; işaretli karşılığına `sbyte` denir.

Gerçek sayılar için üç tip vardır:

Tip	.NET adı	Boyut	Duyarlık	Kullanım
`float`	`System.Single`	4 bayt	yaklaşık 7 basamak	grafik, büyük veri dizileri
`double`	`System.Double`	8 bayt	yaklaşık 15-17 basamak	varsayılan gerçek sayı tipi
`decimal`	`System.Decimal`	16 bayt	28-29 basamak	para ve ondalık hesaplar

Geriye üç tip kalır: `char` (2 bayt, tek bir UTF-16 kod birimi), `bool` (1 bayt, yalnızca `true` ya da `false`) ve metin için `string`. Bunları sonraki başlıklarda ayrıca işleyeceğiz. Günlük işlerin çoğunu `int`, `long`, `double`, `decimal`, `bool`, `char` ve `string` ile yaparız; `byte` ham veri ve 0-255 arası değerler için, `float` ise bellek ya da grafik kaynaklı sınırlar olduğunda tercih edilir.

Boyutlar ve sınırlar

Bir tipin boyutunu `sizeof` ile, en küçük ve en büyük değerini `MinValue` ve `MaxValue` alanlarıyla öğrenebiliriz. `sizeof` yerleşik tipler için derleme zamanında hesaplanan bir sabittir; C'deki `sizeof` operatörünün karşılığıdır.

▸ **tipBilgisi.cs**

```
Console.WriteLine($"sbyte : {sizeof(sbyte)} bayt, " +
    $"{sbyte.MinValue} ... {sbyte.MaxValue}");
Console.WriteLine($"byte  : {sizeof(byte)} bayt, " +
    $"{byte.MinValue} ... {byte.MaxValue}");
Console.WriteLine($"short : {sizeof(short)} bayt, " +
    $"{short.MinValue} ... {short.MaxValue}");
Console.WriteLine($"ushort: {sizeof(ushort)} bayt, " +
    $"{ushort.MinValue} ... {ushort.MaxValue}");
Console.WriteLine($"int   : {sizeof(int)} bayt, " +
    $"{int.MinValue} ... {int.MaxValue}");
Console.WriteLine($"uint  : {sizeof(uint)} bayt, " +
    $"{uint.MinValue} ... {uint.MaxValue}");
Console.WriteLine($"long  : {sizeof(long)} bayt, " +
    $"{long.MinValue} ... {long.MaxValue}");
Console.WriteLine($"ulong : {sizeof(ulong)} bayt, " +
    $"{ulong.MinValue} ... {ulong.MaxValue}");
Console.WriteLine($"char  : {sizeof(char)} bayt");
Console.WriteLine($"bool  : {sizeof(bool)} bayt");
Console.WriteLine($"float : {sizeof(float)} bayt");
Console.WriteLine($"double: {sizeof(double)} bayt");
Console.WriteLine($"decimal: {sizeof(decimal)} bayt");
```

```
ÇIKTI
sbyte : 1 bayt, -128 ... 127
byte  : 1 bayt, 0 ... 255
short : 2 bayt, -32768 ... 32767
ushort: 2 bayt, 0 ... 65535
int   : 4 bayt, -2147483648 ... 2147483647
uint  : 4 bayt, 0 ... 4294967295
long  : 8 bayt, -9223372036854775808 ... 9223372036854775807
ulong : 8 bayt, 0 ... 18446744073709551615
char  : 2 bayt
bool  : 1 bayt
float : 4 bayt
double: 8 bayt
decimal: 16 bayt
```

Bu programın çıktısı her bilgisayarda aynıdır. C'de aynı çıktıyı garanti edemezdik.

.NET tip adları

C#'taki `int`, `long`, `string` gibi anahtar sözcükler aslında .NET sınıf kütüphanesindeki tiplerin kısa adlarıdır (alias). `int` ile `System.Int32` aynı tiptir; hangisini yazdığımız derlenen koda hiçbir fark yaratmaz. Başka .NET dilleri (F#, Visual Basic) kendi anahtar sözcüklerini kullansa da bu tiplerin tam adı hepsinde aynıdır. Bölüm 41'de gördüğümüz ortak tip sisteminin (CTS) sonucu budur.

netAdlari.cs

```
int a = 42;
Int32 b = a;             // int ile aynı tip
System.Int64 c = 7;      // long ile aynı tip
Console.WriteLine(a.GetType());
Console.WriteLine(c.GetType().Name);
Console.WriteLine(typeof(string).FullName);
Console.WriteLine(typeof(int) == typeof(Int32));
```

ÇIKTI

```
System.Int32
Int64
System.String
True
```

Kodda tip yazarken `int`, `string`, `double` gibi kısa adları kullanmak yaygın bir gelenektir. Sınıf adı gibi çağrılan statik üyelerde (`Int32.Parse`, `String.Join`) ise uzun ad da sık görülür; ikisi de geçerlidir.

İşaretsiz tipler ve sessiz sarma

İşaretsiz tipler eksi bir değeri tutamaz. Eksi bir **sabit** atamaya çalışırsak derleyici bunu hemen yakalar:

isaretsizHata.cs

```
byte b = -1;
uint u = -5;
```

DERLEYİCİ ÇIKTISI

```
isaretsizHata.cs(1,10): error CS0031: Constant value '-1' cannot be converted to a 'byte'
isaretsizHata.cs(2,10): error CS0031: Constant value '-5' cannot be converted to a 'uint'
```

Sabit olmayan bir işlem sonucu tipin dışına çıkarsa ne olur? Aşağıdaki programda `uint` tipli iki değişkenin farkını alıyoruz:

isaretsiz.cs

```
uint a = 3;
uint b = 5;
Console.WriteLine(a - b);     // eksiye inmez, başa döner
uint sayac = 0;
sayac--;
Console.WriteLine(sayac);
```

ÇIKTI

```
4294967294
4294967295
```

`3 - 5` işlemi `uint` ile yapıldığında −2 çıkmaz; sayı en küçük değerin altına inip en büyük uca döner. Bu davranış, bölümün sonunda göreceğimiz taşma (overflow) kavramının bir örneğidir.

▲ Sık Yapılan Hata

İşaretsiz tipleri yalnızca bit işlemleri, ham veri ya da gerçekten eksi olmayacak değerler için kullanalım. Bir döngü sayacını ya da bir farkı `uint` yaparsak, sayacın sıfırın altına inmesi hata vermez; sessizce çok büyük bir sayıya döner. C'den gelen alışkanlıkla `unsigned` tipleri sık kullanmak C#'ta pek önerilmez. .NET kütüphanesi bile uzunluk ve sayaçlar için işaretli `int` kullanır (`Length` , `Count`).

Üç Dilde · Tam sayı tiplerinin boyutları *C · C++ · C#*

Aynı kavramın üç dildeki durumunu yan yana koyalım.

Kavram	C	C++	C#
`int`	en az 16 bit; pratikte 32 bit	C ile aynı	her zaman 32 bit
`long`	en az 32 bit; Linux 64 bit'te 64, Windows 64 bit'te 32	C ile aynı	her zaman 64 bit
`long long`	en az 64 bit (C99)	en az 64 bit (C++11)	yok; `long` zaten 64 bit
tek karakter	`char` : 1 bayt	`char` : 1 bayt, ayrıca `wchar_t` , `char16_t` , `char32_t`	`char` : 2 bayt (UTF-16)
mantıksal	`_Bool` / `bool` (`<stdbool.h>` , C23'te anahtar sözcük)	`bool` yerleşik	`bool` yerleşik, 1 bayt
ondalık para tipi	yok	yok	`decimal`
gerçek sayı	`float` , `double` , `long double`	aynı	`float` , `double` , `decimal`
işaretsiz	`unsigned int`	`unsigned int`	`uint`

C'de `long` tipinin boyutu platforma göre değişir:

```
printf("%zu\n", sizeof(long));   // Linux (64 bit): 8, Windows (64 bit): 4
```

C#'ta aynı sorunun cevabı hep aynıdır:

```
Console.WriteLine(sizeof(long));   // her platformda 8
```

Bu yüzden C#'ta "tipin kaç bit olduğu" sorusunu sormadan program yazabiliriz. Eski kodla veri alışverişi yapılırken (ikili dosya, ağ protokolü) bu sabit boyutlar büyük kolaylık sağlar.

42.4 Gerçek Sayılar: float, double ve decimal

Bilgisayar sayıları ikilik (binary) düzende saklar. 0.5 gibi ikinin kuvvetlerinin toplamı olan kesirler tam saklanabilir; ama 0.1 ya da 0.2 gibi onluk düzende basit görünen sayılar ikilik düzende sonsuz bir kesirdir. `float` ve `double` bu sayıları yaklaşık olarak tutar. C ve C++'ta öğrendiğimiz bu gerçek C#'ta da aynen geçerlidir (Bölüm 5 ve Bölüm 22).

`hassasiyet.cs`

```
double d1 = 0.1, d2 = 0.2;
Console.WriteLine(d1 + d2);
Console.WriteLine(d1 + d2 == 0.3);
Console.WriteLine(1.0 - 0.9);

decimal m1 = 0.1m, m2 = 0.2m;
Console.WriteLine(m1 + m2);
Console.WriteLine(m1 + m2 == 0.3m);

float f = 1.0f / 3;
double d = 1.0 / 3;
decimal m = 1.0m / 3;
Console.WriteLine($"{f} {d} {m}");
```

```
ÇIKTI
0.30000000000000004
False
0.09999999999999998
0.3
True
0.33333334 0.3333333333333333 0.3333333333333333333333333333
```

`double` ile `0.1 + 0.2` tam olarak `0.3` etmez; bu yüzden eşitlik karşılaştırması `False` döner. `decimal` ise değerleri ikilik düzende değil onluk düzende tuttuğu için `0.1m + 0.2m` tam `0.3m`'dir. Son satır üç tipin duyarlığını gösterir: `float` yaklaşık 7, `double` yaklaşık 15-17, `decimal` 28-29 anlamlı basamak tutar.

`decimal`'in bir bedeli vardır: işlemleri donanım yerine yazılım düzeyinde yapıldığı için `double`'dan yavaştır ve ondalık sayıların çok büyük ya da çok küçük üslerini tutamaz (yaklaşık ±7,9 × 10^{28} aralığına sahiptir; `double`'ın aralığı yaklaşık ±1,8 × 10^{308}'dir). Bilimsel hesaplar, grafik ve fizik benzetimleri için `double` doğru seçimdir; para ve muhasebe gibi **tam ondalık sonuç** beklediğimiz yerlerde `decimal` kullanırız.

`paraHesabi.cs`

```
decimal fiyat = 19.99m;
int adet = 3;
decimal toplam = fiyat * adet;
decimal vergili = Math.Round(toplam * 1.20m, 2);
Console.WriteLine($"Toplam: {toplam}");
Console.WriteLine($"Vergili: {vergili}");
```

```
ÇIKTI
Toplam: 59.97
Vergili: 71.96
```

`fiyat * adet` işleminde `int` olan `adet`, kendiliğinden `decimal`'e dönüşür. `toplam * 1.20m` sonucu 71.964'tür; `Math.Round(x, 2)` onu iki basamağa yuvarlar. `decimal` ile `double`'ı birlikte kullanmaya çalışırsak derleyici durdurur:

`decimalKarisim.cs`

```
decimal tutar = 10m * 1.20;
```

```
DERLEYİCİ ÇIKTISI

decimalKarisim.cs(1,17): error CS0019: Operator '*' cannot be applied to operands of type
'decimal' and 'double'
```

Doğrusu `1.20m` yazmaktır. C#, para hesabında yanlışlıkla `double` karıştırılmasını bu şekilde engeller.

Özel değerler ve karşılaştırma

Gerçek sayılar tamsayılardan farklı olarak sıfıra bölünce hata vermez. Sonuç, IEEE 754 standardının tanımladığı özel değerlerden biridir: sonsuz (`Infinity`) ya da sayı değil (`NaN`, Not a Number). `decimal` ise sıfıra bölünürse istisna fırlatır.

▸ ozelDegerler.cs

```
double sifir = 0.0;
Console.WriteLine(1.0 / sifir);
Console.WriteLine(-1.0 / sifir);
Console.WriteLine(sifir / sifir);
double nan = sifir / sifir;
double baska = nan;
Console.WriteLine(nan == baska);
Console.WriteLine(double.IsNaN(nan));
Console.WriteLine(double.MaxValue);
Console.WriteLine(float.MaxValue);
Console.WriteLine(decimal.MaxValue);
```

```
ÇIKTI
Infinity
-Infinity
NaN
False
True
1.7976931348623157E+308
3.4028235E+38
79228162514264337593543950335
```

> **▲ Sık Yapılan Hata**
>
> İki `double` değerini `==` ile karşılaştırmayalım. Hesaplama hataları birikebileceği için "yaklaşık eşit" mi diye bakmak gerekir: `Math.Abs(a - b) < 1e-9` . Eşik değeri, çalıştığımız büyüklüğe göre seçilir. `NaN` ise kendisine bile eşit değildir; `NaN` olup olmadığını `double.IsNaN` ile sorarız. Kesin sonuç gereken yerlerde baştan `decimal` kullanmak en güvenli yoldur.

42.5 char, bool ve string

Sayısal tiplerin dışında kalan üç temel tipi bu başlıkta toplayalım: tek karakter için `char`, doğru/yanlış bilgisi için `bool`, metin için `string`.

char

`char`, tek bir karakteri tutar ve tek tırnakla yazılır: `'A'`, `'7'`, `'ş'`. C'deki `char` 1 bayttır ve çoğunlukla ASCII karakterleri taşır. C#'ta `char` 2 bayttır ve bir **UTF-16 kod birimi** (code unit) saklar. Bu sayede Türkçe harflerin tamamı, Yunan ve Kiril alfabeleri gibi geniş bir karakter kümesi tek bir `char` içinde tutulabilir.

Bir `char` aslında 0 ile 65 535 arasında bir sayıdır: her karakterin bir Unicode numarası (kod noktası) vardır. İlk 128 karakter ASCII ile aynıdır: `'A'` 65, `'a'` 97, `'0'` 48. Bu yüzden karakterlerle aritmetik yapabiliriz. Bu davranış C'dekine benzer; fark, C#'ın `char` ile `int` arasındaki dönüşümü açıkça sorgulamasıdır.

charIslemleri.cs

```
char c = 'A';
Console.WriteLine(c + 1);               // char + int -> int
Console.WriteLine((char)(c + 1));       // tekrar karaktere çevir
Console.WriteLine((int)'a');
c++;                                    // c artık 'B'
Console.WriteLine(c);
Console.WriteLine('7' - '0');           // rakam karakterinden sayıya
char harf = 'ş';
Console.WriteLine($"{harf} {(int)harf}");
Console.WriteLine('A');            // Unicode numarasıyla 'A'
```

ÇIKTI

```
66
B
97
B
7
ş 351
A
```

`c + 1` ifadesinde `char` önce `int`'e dönüştürülür; sonuç `int` olduğu için ekrana `66` yazılır. Karakter olarak görmek istiyorsak `(char)` ile açıkça dönüştürmemiz gerekir. `'7' - '0'` ise rakam karakterini sayıya çevirmenin klasik yoludur: `'7'` 55, `'0'` 48 olduğu için fark 7 çıkar.

> **İpucu**
>
> Akılda tutulacak üç değer: `'A'` = 65, `'a'` = 97, `'0'` = 48. Gerisini bunlardan türetiriz: `'C'` = 67, `'z'` = 122. Aynı harfin büyük ve küçük biçimi arasındaki fark her zaman 32'dir.

Özel karakterleri, ters bölü ile başlayan kaçış dizileriyle (escape sequence) yazarız:

Dizi	Anlamı
`\n`	yeni satır
`\t`	yatay sekme (tab)
`\r`	satır başı
`\\`	ters bölü karakterinin kendisi
`\'`	tek tırnak
`\"`	çift tırnak
`\0`	sıfır numaralı (null) karakter
`A`	dört onaltılık basamakla Unicode karakteri (A)

> **Not · UTF-16 ve yedek çiftler**
>
> UTF-16'da bazı karakterlerin kod noktası 65 535'ten büyüktür (örneğin U+1F600). Bu karakterler iki `char` ile, yani bir yedek çifti (surrogate pair) ile saklanır. Sonuç olarak bir `string`'in `Length` değeri, ekranda görünen

> karakter sayısından büyük çıkabilir. Türkçe metinlerde bu durumla karşılaşmayız; ayrıntıyı Bölüm 48'de ele alacağız.

bool

`bool`, yalnızca `true` ya da `false` değerini alır. C'de mantıksal ifadeler `int` ile yürütülür: sıfır yanlış, sıfırdan farklı her şey doğrudur. C99 ile gelen `_Bool` ve C++'ın `bool` tipi bu duruma bir tip ekler; ama her ikisinde de `bool` ile `int` arasında örtük dönüşüm sürer. C# bu köprüyü tamamen kaldırmıştır: `bool`, sayıya dönüşmez; sayı da `bool`'a dönüşmez.

▸ boolSayi.cs

```
bool durum = 1;
```

```
DERLEYİCİ ÇIKTISI
boolSayi.cs(1,14): error CS0029: Cannot implicitly convert type 'int' to 'bool'
```

Bu sıkılığın pratik sonuçlarını karşılaştırma ve mantıksal operatörleri işlerken göreceğiz. `bool` değerini kodda `true` yazarız; ama ekrana yazdırılınca `True` görünür.

string

`string`, bir metni (karakter dizisini) tutar ve çift tırnakla yazılır: `"Merhaba"`. Referans tipidir; ama atama ve `==` karşılaştırması bakımından değer tipi gibi davranmasını sağlayan iki özelliği vardır. Birincisi, `string` **değiştirilemezdir** (immutable): bir `string` oluşturulduktan sonra içindeki karakterler değişmez; "değiştiren" her işlem yeni bir `string` üretir. İkincisi, `==` operatörü referansları değil **içerikleri** karşılaştırır.

▸ stringTemel.cs

```
string ad = "Elif";
string soyad = "Aksoy";
string tam = ad + " " + soyad;       // birleştirme
Console.WriteLine(tam);
Console.WriteLine(tam.Length);

string ilk = tam;
tam = tam + "!";                     // yeni bir string üretilir
Console.WriteLine(ilk);
Console.WriteLine(tam);

Console.WriteLine("5" + 3);
Console.WriteLine(5 + 3 + "5");
Console.WriteLine("5" + 3 + 5);
Console.WriteLine(ad == "Elif");
```

```
ÇIKTI
Elif Aksoy
10
Elif Aksoy
Elif Aksoy!
53
85
535
True
```

`tam = tam + "!"` satırı eski metni değiştirmedi; yeni bir metin üretti ve `tam` onu göstermeye başladı. `ilk` hâlâ eski metni gösteriyor. Birleştirmede bir tarafın `string` olması yeterlidir: diğer taraf metne çevrilir. Ama işlemler soldan sağa yapıldığı için `5 + 3 + "5"` önce `8`'i hesaplar, sonra `"85"` olur; `"5" + 3 + 5` ise baştan metin birleştirmesi yaptığı için `"535"` verir.

Bir `string`'in karakterlerine köşeli parantezle ulaşabiliriz (`ad[0]`); ama bu erişim salt okunurdur:

▸ stringDegismez.cs

```
string s = "Merhaba";
s[0] = 'm';
```

DERLEYİCİ ÇIKTISI

```
stringDegismez.cs(2,1): error CS0200: Property or indexer 'string.this[int]' cannot be assigned
to -- it is read only
```

C'de karakter dizilerinin değiştirilebilir olması ve `==` ile iki dizinin adreslerinin karşılaştırılması, sık hataların kaynağıdır (Bölüm 11). C#'ta içerik karşılaştırması `==` ile yapılır, `strcmp` benzeri bir işleve gerek kalmaz. Metot ve yöntemleri, `StringBuilder`'ı ve Türkçeye özgü büyük/küçük harf sorunlarını Bölüm 48'de işleyeceğiz.

> **▲ Sık Yapılan Hata**
>
> Karakter ve metin sabitleri farklı tırnak ister. Tek karakter için tek tırnak, metin için çift tırnak kullanılır. `char c = "A";` derlenmez, çünkü `"A"` bir `string`'dir. Tersine, birden çok karakteri tek tırnağa sığdırmak (`'Merhaba'`) da hatadır.
>
> ▸ charString.cs
>
> ```
> char c = "A";
> ```
>
> DERLEYİCİ ÇIKTISI
>
> ```
> charString.cs(1,10): error CS0029: Cannot implicitly convert type 'string' to 'char'
> ```

42.6 Sabit Değerler (Literal)

Kodun içine doğrudan yazdığımız `42`, `3.14`, `'A'`, `"Merhaba"` gibi değerlere sabit değer ya da **literal** denir. Her literalin bir tipi vardır ve bu tip, yazılışından anlaşılır. Aynı sayıyı farklı tiplerde yazabilmek için sonuna ek (suffix) getiririz.

Yazılış	Tip	Örnek
ondalık noktası olmayan sayı	`int`, sığmazsa `uint`, `long`, `ulong`	`42`, `3_000_000_000`
`L` eki	`long`	`42L`
`u` eki	`uint` (sığmazsa `ulong`)	`42u`
`UL` eki	`ulong`	`42UL`
ondalık noktalı ya da üslü sayı	`double`	`3.14`, `1.5e3`
`d` eki	`double`	`2.5d`
`f` eki	`float`	`3.14f`

Yazılış	Tip	Örnek
`m` eki	`decimal`	`3.14m`
tek tırnak	`char`	`'A'`
çift tırnak	`string`	`"A"`
`true`, `false`	`bool`	`true`

Eklerin büyük ya da küçük harfle yazılması fark etmez; ama küçük `l` harfi `1` rakamına benzediği için `L` yazmayı tercih ederiz.

Bir tam sayı literalinin tipi, ek yoksa, değerin sığdığı ilk tip olarak belirlenir: `int`, sonra `uint`, sonra `long`, sonra `ulong`. Bu yüzden `3_000_000_000` bir `uint`, `5_000_000_000` ise bir `long` olur. Aşağıdaki program, literal tiplerini `GetType` ile sorguluyor ve sayıları farklı tabanlarda yazma biçimlerini gösteriyor:

literaller.cs

```
Console.WriteLine((42).GetType().Name);
Console.WriteLine((3_000_000_000).GetType().Name);
Console.WriteLine((5_000_000_000).GetType().Name);
Console.WriteLine((42L).GetType().Name);
Console.WriteLine((3.14).GetType().Name);
Console.WriteLine((3.14f).GetType().Name);
Console.WriteLine((3.14m).GetType().Name);
Console.WriteLine('x'.GetType().Name);

int milyon = 1_000_000;           // alt çizgi yalnızca okunabilirlik içindir
int onaltilik = 0xFF;             // 0x ile onaltılık
int ikilik = 0b1010_1010;         // 0b ile ikilik
double bilimsel = 1.5e3;          // 1.5 x 10^3
Console.WriteLine($"{milyon} {onaltilik} {ikilik} {bilimsel}");
```

```
ÇIKTI
Int32
UInt32
Int64
Int64
Double
Single
Decimal
Char
1000000 255 170 1500
```

Sayı içindeki alt çizgi (`_`) basamak ayırıcıdır ve yalnızca okumayı kolaylaştırır; `1_000_000` ile `1000000` aynı sayıdır. Alt çizgi sayının başında, sonunda ya da ondalık noktasının yanında olamaz.

> *Tarihçe · Basamak ayırıcı ve ikilik sabit*
>
> `0b` ikilik sabitler ile `_` basamak ayırıcısı C# 7.0'da (2017) eklendi. C++ aynı özellikleri C++14'te getirdi; ancak C++'ın basamak ayırıcısı alt çizgi değil tek tırnaktır (`1'000'000`). C'de ikisi de ancak C23 ile standarda girdi.

Ek yazmayı unutmak, özellikle `float` ve `decimal`'de derleme hatasına yol açar. `0.75` bir `double` sabitidir ve `float` değişkene örtük olarak konamaz:

floatHata.cs

```
float oran = 0.75;
```

DERLEYİCİ ÇIKTISI

```
floatHata.cs(1,14): error CS0664: Literal of type double cannot be implicitly converted to type
'float'; use an 'F' suffix to create a literal of this type
```

Doğrusu `0.75f` yazmaktır. Aynı nedenle `decimal fiyat = 19.99;` da derlenmez, `19.99m` yazılmalıdır. C'de `float x = 0.75;` sessizce derlenir ve değeri `float`'a daraltır; C#'ın bu kadar katı olması veri kaybını görünür kılmak içindir.

42.7 var, const ve readonly

var ile tip çıkarımı

Tipi açıkça yazmak yerine `var` anahtar sözcüğünü kullanabiliriz. Derleyici değişkenin tipini, sağ taraftaki ifadeden **derleme zamanında** çıkarır (type inference).

varOrnek.cs

```
var sayi = 10;                // int
var oran = 2.5;               // double
var mesaj = "Selam";          // string
var fiyat = 9.99m;            // decimal

Console.WriteLine($"{sayi * oran} {mesaj}");
Console.WriteLine(sayi.GetType());
Console.WriteLine(oran.GetType());
Console.WriteLine(mesaj.GetType());
Console.WriteLine(fiyat.GetType());
```

ÇIKTI

```
25 Selam
System.Int32
System.Double
System.String
System.Decimal
```

`var` "tipsiz" anlamına gelmez. C# yine statik tipli bir dildir: tip derleme sırasında kesinleşir ve sonradan değişmez. `var sayi = 10;` dedikten sonra `sayi = "on";` yazamayız. Çıkarım yapılacak bir değer olmadığı için `var x;` biçiminde değersiz tanımlama da geçersizdir:

varHata.cs

```
var x;
var y = 10;
y = "on";
```

DERLEYİCİ ÇIKTISI

```
varHata.cs(1,5): error CS0818: Implicitly-typed variables must be initialized
varHata.cs(3,5): error CS0029: Cannot implicitly convert type 'string' to 'int'
```

> *Tarihçe · var ve auto*
>
> `var`, C# 3.0 ile (2007) geldi. Asıl gerekçesi, adı uzun ya da yazması zor tiplerin (ve ileride göreceğimiz isimsiz tiplerin) kullanılabilmesiydi. C++ aynı fikri C++11'de `auto` ile getirdi. İkisi de aynı şekilde çalışır: tip derleme zamanında sağ taraftan belirlenir.

`var`'ı ne zaman kullanmalıyız? Tip sağ taraftan zaten açıkça okunuyorsa (`var liste = new List<int>();`) tekrar yazmak gereksizdir. Tip belli değilse ya da çıkarılan tip bir sayı sabitine bağlıysa tipi açıkça yazmak okunurluğu artırır. Öğrenme aşamasında tipleri açıkça yazmak, tiplerle düşünme alışkanlığını da pekiştirir; bu kitapta çoğu zaman bunu yapacağız.

const ile sabitler

Program boyunca değişmeyecek bir değer için `const` kullanırız. Sabit, tanımlandığı satırda ilk değerini almalıdır ve bu değer **derleme sırasında** hesaplanabilen bir ifade olmalıdır. Klavyeden okunan ya da başka bir değişkene bağlı bir değer sabit olamaz. Sabite sonradan atama yapmak da hatadır.

sabitOrnek.cs

```
const double Pi = 3.14159;
const int GundeSaat = 24;
const int SaattaDakika = 60;
const string Birim = "dakika";

double yaricap = 2.0;
Console.WriteLine($"Alan: {Pi * yaricap * yaricap}");
Console.WriteLine($"Bir günde {GundeSaat * SaattaDakika} {Birim} var");
```

ÇIKTI

```
Alan: 12.56636
Bir günde 1440 dakika var
```

constHata.cs

```
const int Max = 100;
Max = 200;
int sayi = 10;
const int Yuz = sayi * 10;
```

DERLEYİCİ ÇIKTISI

```
constHata.cs(2,1): error CS0131: The left-hand side of an assignment must be a variable,
property or indexer
constHata.cs(4,17): error CS0133: The expression being assigned to 'Yuz' must be constant
```

`const` yalnızca yerleşik sayısal tipler, `bool`, `char`, `string`, `enum` ve `null` ile kullanılabilir. Sabitlerin adı, gelenek olarak PascalCase yazılır; C'deki gibi BÜYÜK_HARF kullanılmaz.

C'de sabit için iki seçenek vardır: önişlemci makrosu (`#define PI 3.14159`) ya da `const` niteleyicisi. C++ bunlara `constexpr` ekler (Bölüm 39). C#'ta `#define` ile bir değer tanımlanamaz; önişlemci yalnızca koşullu derleme içindir. C#'ta sabit için `const` yeterlidir; tipli, kapsamlı ve derleyici tarafından denetlenen bir yoldur.

readonly (önizleme)

`readonly`, başka bir tür salt-okunur değer için vardır. Yerel değişkenlerde kullanılamaz; sınıf ve yapıların **alanlarında** (field) kullanılır. `const`'tan farkı, değerinin derleme zamanında bilinmesi gerekmemesidir: bir kez, ya tanımlandığı yerde ya da nesne oluşturulurken (yapıcıda) atanabilir. Sınıfları Bölüm 49'da öğreneceğiz; şimdilik farkı bir kesitle görelim.

```
class Hesap
{
    public const int AzamiLimit = 1000;     // derleme zamanı sabiti
    public readonly string Numara;           // nesne oluşurken bir kez atanır

    public Hesap(string numara)
    {
        Numara = numara;                     // yapıcıda atamaya izin var
    }
}
```

> ◆ **Kritik Nokta**
>
> `const` değeri derleme sırasında kodun içine gömülür; `readonly` ise çalışma sırasında bir kez atanır. Değeri başka bir yerden (kullanıcı girdisi, dosya, hesaplama) geliyorsa `const` kullanamayız. Ayrıntılı karşılaştırmayı Bölüm 49'da, `static` üyelerle birlikte yapacağız.

42.8 Aritmetik Operatörler

Beş temel aritmetik operatör C ve C++'takilerle aynıdır:

Operatör	İşlem	Örnek	Sonuç
`+`	toplama	`7 + 2`	`9`
`-`	çıkarma	`7 - 2`	`5`
`*`	çarpma	`7 * 2`	`14`
`/`	bölme	`7 / 2`	`3`
`%`	kalan (mod)	`7 % 2`	`1`

Bu başlığın en önemli kuralı tam sayı bölmesidir. İki tam sayı birbirine bölünürse sonuç da tam sayıdır; ondalık kısım atılır. Yuvarlama yapılmaz, **kesme** yapılır. Sonucun ondalıklı olması için işlenenlerden en az biri ondalıklı olmalıdır:

bolme.cs

```
Console.WriteLine(7 / 2);
Console.WriteLine(7.0 / 2);
Console.WriteLine(7 / 2.0);
Console.WriteLine(9 / 10);
Console.WriteLine(-7 / 2);
Console.WriteLine(7 % 2);
Console.WriteLine(10 % 3);
Console.WriteLine(3 % 10);
Console.WriteLine(-7 % 3);
Console.WriteLine(7 % -3);
Console.WriteLine(7.5 % 2);
```

```
ÇIKTI
3
3.5
3.5
0
-3
1
1
3
-1
1
1.5
```

Çıktıdaki satırları sırayla yorumlayalım:

- `9 / 10` işleminin sonucu `0.9` değil `0`'dır; ondalık kısım kesilir.
- `-7 / 2` işleminde `-3.5` değeri sıfıra doğru kesilir ve `-3` elde edilir (`-4` değil).
- `%` bölmeden kalanı verir. `3 % 10` işleminde 3, 10'a sıfır kez bölünür ve 3 kalır.
- Kalanın işareti soldaki sayının işaretiyle aynıdır: `-7 % 3` sonucu `-1`, `7 % -3` sonucu `1` olur. Bu kural C99'dan beri C'de, C++11'den beri C++'ta da sabittir.
- `%` ondalıklı sayılarla da çalışır: `7.5 % 2` sonucu `1.5` olur. Bu, C'den farklıdır; C'de `%` yalnızca tam sayılarla kullanılabilir (ondalıklılar için `fmod` gerekir).

> ◆ **Kritik Nokta**
>
> `%` operatörünün klasik kullanımları:
>
> - Tek/çift denetimi: `sayi % 2 == 0` ise sayı çifttir.
> - Bölünebilme: `sayi % 5 == 0` ise sayı 5'e tam bölünür.
> - Son basamağı almak: `1234 % 10` sonucu `4`.
> - Son basamağı atmak: `1234 / 10` sonucu `123`.
>
> Son iki ifade, bir sayının basamaklarını toplamak ya da ters çevirmek gibi birçok algoritmanın temelini oluşturur.

Tam sayıyı sıfıra bölmenin sonucu, işlenenlerin sabit mi değişken mi olduğuna bağlıdır. Sıfıra bölme derleyicinin görebildiği bir sabitse program derlenmez:

▸ sifirHata.cs

```
Console.WriteLine(5 / 0);
```

```
DERLEYİCİ ÇIKTISI
sifirHata.cs(1,19): error CS0020: Division by constant zero
```

Payda çalışma sırasında sıfır çıkıyorsa derleyici bunu göremez; program çalışırken `DivideByZeroException` istisnası fırlatılır. İstisna yakalamayı Bölüm 53'te öğreneceğiz; şimdilik `try` ve `catch` bloklarını yalnızca sonucu izlemek için kullanalım:

▸ sifiraBolme.cs

```
int pay = 10;
int payda = 0;
try
{
```

```
    Console.WriteLine(pay / payda);
}
catch (DivideByZeroException)
{
    Console.WriteLine("Tam sayı sıfıra bölünemez.");
}
Console.WriteLine(pay / (double)payda);
```

```
ÇIKTI
Tam sayı sıfıra bölünemez.
Infinity
```

C'de tam sayıyı sıfıra bölmek tanımsız davranıştır; program çökebilir ya da sessizce yanlış çalışabilir. C#'ta davranış her zaman tanımlıdır ve bir istisnaya dönüşür.

İşlem önceliği

Matematikteki gibi `*`, `/` ve `%` işlemleri `+` ve `-`'den önce yapılır. Aynı öncelikteki işlemler soldan sağa yürütülür. Parantez her şeyden önce gelir.

oncelik.cs

```
Console.WriteLine(2 + 3 * 4);
Console.WriteLine((2 + 3) * 4);
Console.WriteLine(10 - 4 - 3);
Console.WriteLine(20 / 4 * 2);
Console.WriteLine(2 + 10 % 4);
Console.WriteLine(17 % 5 * 2 + 1);
```

```
ÇIKTI
14
20
3
10
4
5
```

`20 / 4 * 2` ifadesinde `/` ve `*` aynı öncelikte olduğundan soldan sağa gidilir: `(20 / 4) * 2 = 10`. Sonuç `20 / 8` değildir. Tüm operatörlerin öncelik sırasını, bit operatörleri de anlatıldıktan sonra tek bir tabloda toplayacağız.

42.9 Atama, Artırma ve Azaltma

Bileşik atama

Bir değişkenin değerini kendisine bağlı olarak değiştirmek çok yaygın olduğu için kısa yazımlar vardır:

Kısa yazım	Uzun hali
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 5`	`x = x * 5`
`x /= 5`	`x = x / 5`

Kısa yazım	Uzun hali
`x %= 5`	`x = x % 5`

bilesikAtama.cs

```
int x = 10;
x += 5;          // 15
x *= 2;          // 30
x -= 4;          // 26
x /= 4;          // 6 (tam sayı bölmesi)
x %= 4;          // 2
Console.WriteLine(x);

double d = 10;
d /= 4;
Console.WriteLine(d);
```

ÇIKTI

```
2
2.5
```

Bileşik atamanın ince bir özelliği vardır: işlem sonucunu sol taraftaki değişkenin tipine **kendisi dönüştürür**. `b += 5` ifadesi aslında `b = (byte)(b + 5)` anlamına gelir. Bu yüzden `byte` tipli bir değişkende `b += 5` derlenir, ama `b = b + 5` derlenmez. Neden olduğunu, dönüşümleri anlattığımız bölümde göreceğiz.

gizliCast.cs

```
byte b = 10;
b += 5;
Console.WriteLine(b);
```

ÇIKTI

```
15
```

bilesikHata.cs

```
byte b = 10;
b = b + 5;
```

DERLEYİCİ ÇIKTISI

```
bilesikHata.cs(2,5): error CS0266: Cannot implicitly convert type 'int' to 'byte'. An explicit
conversion exists (are you missing a cast?)
```

x++ ve ++x

Bir değişkeni 1 artırmak için `x++` ya da `++x`, 1 azaltmak için `x--` ya da `--x` yazılır. Tek başına bir satırda kullanıldıklarında aralarında fark yoktur. Fark, ifadenin içinde kullanıldıklarında ortaya çıkar:

- Son ek (postfix) `x++`: önce `x`'in mevcut değerini kullan, sonra artır.
- Ön ek (prefix) `++x`: önce artır, sonra yeni değeri kullan.

artirma.cs

```
int a = 5;
int b = a++;                // b = 5 (eski değer), sonra a = 6
Console.WriteLine($"a={a} b={b}");

int c = 5;
int d = ++c;                // önce c = 6, sonra d = 6
Console.WriteLine($"c={c} d={d}");

int x = 3;
Console.WriteLine(x++);
Console.WriteLine(x);
Console.WriteLine(++x);

int y = 10;
int z = y-- - --y;          // 10 - 8
Console.WriteLine($"y={y} z={z}");

int w = 5;
w = w++;                    // w artmaz!
Console.WriteLine(w);
```

ÇIKTI

```
a=6 b=5
c=6 d=6
3
4
5
y=8 z=2
5
```

`y-- - --y` ifadesini adım adım çözelim. Önce `y--` değer olarak 10 verir ve `y` 9 olur. Sonra `--y`, `y`'yi 8'e indirir ve değer olarak 8 verir. Sonuç `z = 10 - 8 = 2`, `y`'nin son değeri 8'dir.

`w = w++` satırı da dikkat çekicidir: Önce `w`'nun eski değeri (5) bir kenara alınır, `w` 6 olur, sonra eski değer (5) `w`'ya geri atanır. Sonuç olarak `w` hâlâ 5'tir.

Değerlendirme sırası tanımlıdır

C ve C++'ta `i = i++ + ++i` gibi bir ifade tanımsız davranıştır: standart, aynı değişkenin iki kez değiştirildiği ifadelerde işlemlerin hangi sırayla yapılacağını belirtmez. Derleyiciye göre farklı sonuç çıkabilir. C# bu belirsizliği bırakmaz: operandlar **her zaman soldan sağa** değerlendirilir. Öncelik ve birleşme kuralları yalnızca ifadenin nasıl gruplandığını belirler.

siraTanimli.cs

```
int i = 1;
int r = i++ + i++ * 2;
Console.WriteLine($"r={r} i={i}");
```

ÇIKTI

```
r=5 i=3
```

Soldaki `i++` değer olarak 1 verir ve `i` 2 olur. Sağdaki `i++ * 2` ifadesinde `i++` değer olarak 2 verir (`i` 3 olur) ve çarpım 4 çıkar. Toplam `1 + 4 = 5`. Sonuç her derleyicide ve her çalıştırmada aynıdır.

> **▲ Sık Yapılan Hata**
>
> Sırası tanımlı olsa bile, aynı ifadede bir değişkeni birden çok kez artırıp kullanmak okunmasını çok zorlaştırır. `w = w++` gibi satırları gerçek kodda yazmayız. Artırma ve azaltmayı tek başına bir deyim olarak (`sayac++;`) ya da bir döngü başlığında kullanmak en temiz yoldur.

42.10 Karşılaştırma ve Mantıksal Operatörler

Karşılaştırma operatörleri iki değeri kıyaslar ve sonuç olarak her zaman bir `bool` üretir:

Operatör	Anlamı	Örnek	Sonuç
`==`	eşit mi?	`3 == 3`	`True`
`!=`	eşit değil mi?	`3 != 3`	`False`
`<`	küçük mü?	`2 < 5`	`True`
`>`	büyük mü?	`2 > 5`	`False`
`<=`	küçük ya da eşit mi?	`5 <= 5`	`True`
`>=`	büyük ya da eşit mi?	`4 >= 5`	`False`

Mantıksal operatörler `bool` değerleri birleştirir:

Operatör	Adı	Açıklama
`!`	DEĞİL (not)	`true` ise `false`, `false` ise `true` verir
`&&`	VE (and)	iki taraf da `true` ise `true`; kısa devre yapar
`||`	VEYA (or)	en az biri `true` ise `true`; kısa devre yapar
`^`	özel VEYA (xor)	iki taraf farklıysa `true`
`&`, `|`	VE, VEYA (kısa devre yapmaz)	iki tarafı da her zaman değerlendirir

`&&` ve `||` operatörlerinde **kısa devre** (short-circuit) değerlendirme vardır: sonuç sol tarafla belirlenebiliyorsa sağ taraf hiç hesaplanmaz. `false && ...` her zaman `false`, `true || ...` her zaman `true` verdiğinden sağ taraf atlanır. Bu, sağ tarafın ancak sol taraf uygunsa güvenle çalıştırılabildiği denetimlerde çok işimize yarar. `&` ve `|` ise `bool` değerlerine uygulandığında iki tarafı da değerlendirir.

▸ mantiksal.cs

```
int sayac = 0;
bool kapali = false;
bool sonuc = kapali && (++sayac > 0);     // sağ taraf çalışmaz
Console.WriteLine($"{sonuc} {sayac}");
sonuc = kapali & (++sayac > 0);           // sağ taraf çalışır
Console.WriteLine($"{sonuc} {sayac}");

Console.WriteLine(true ^ true);
Console.WriteLine(true ^ false);

int payda = 0;
Console.WriteLine(payda != 0 && 10 / payda > 2);   // sıfıra bölme olmaz
```

```
int yas = 20;
string durum = yas >= 18 ? "yetişkin" : "çocuk";    // üçlü operatör
Console.WriteLine(durum);
```

```
ÇIKTI
False 0
False 1
False
True
False
yetişkin
```

İlk iki satır farkı gösterir: `&&` kullanıldığında `sayac` artmaz, `&` kullanıldığında artar. `payda != 0 && 10 / payda > 2` ifadesi de kısa devre sayesinde sıfıra bölmeyi hiç denemez.

Son örnekteki `? :` operatörü **üçlü (ternary) operatör**dür: `koşul ? degerEger : degerDegilse` biçimindedir. Koşul `true` ise ilk değeri, `false` ise ikinci değeri verir. C ve C++'taki ile aynı çalışır; ayrıntılarını ve desen eşleme ile birlikte kullanımını Bölüm 44'te göreceğiz.

bool sıkılığı: if (x = 5) hatası

C'nin klasik tuzaklarından biri, koşulda `==` yerine yanlışlıkla `=` yazmaktır. C ve C++'ta bu kod derlenir, `x` değişkenine 5 atar ve atamanın sonucu (5) sıfırdan farklı olduğu için koşul her zaman doğru çıkar. C# koşulun `bool` olmasını zorunlu kıldığından aynı yanlış burada derleme hatasıdır:

▸ atamaKosul.cs

```
int x = 5;
if (x = 5)
{
    Console.WriteLine("beş");
}
```

```
DERLEYİCİ ÇIKTISI

atamaKosul.cs(2,5): error CS0029: Cannot implicitly convert type 'int' to 'bool'
```

Aynı sıkılık sayıları koşul olarak kullanmayı da yasaklar. `if (sayac)` ya da `while (n)` gibi C alışkanlıkları C#'ta derlenmez; açıkça `if (sayac != 0)` yazmak gerekir.

> **Not · Bool değişkenle atama**
>
> Tek bir durumda bu koruma işlemez: değişkenin kendisi `bool` ise `if (bitti = true)` ifadesi geçerli bir koşuldur. Derleyici yalnızca `CS0665` uyarısı verir ("Assignment in conditional expression is always constant") ve kod derlenir. Bu tür uyarıları görmezden gelmeyelim. Bir `bool` değişkeni sınarken `if (bitti)` yazmak hem daha kısa hem de bu tuzağı ortadan kaldırır.

Üç Dilde · Koşulda yanlışlıkla atama *C · C++ · C#*

```
int x = 5;
if (x = 5) { /* C: derlenir (en fazla uyarı), koşul hep doğru */ }
```

```
int x = 5;
if (x = 5) { /* C++: derlenir, koşul hep doğru */ }
```

```
int x = 5;
if (x = 5) { /* C#: derleme hatası CS0029 */ }
```

C ve C++'ta bu sorun için "Yoda koşulu" (`5 == x`) gibi savunma alışkanlıkları geliştirilmiştir. C#'ta buna gerek yoktur; derleyici koşulun tipini denetler.

42.11 Bit Operatörleri ve Operatör Önceliği

Bit operatörleri, tam sayıların bitleri üzerinde tek tek işlem yapar. Mantıksal operatörlerle aynı işaretleri taşıdıkları için `bool` ile tam sayı arasındaki ayrımı bilmek önemlidir: `bool` operandlarla `&` ve `|` mantıksal işlem, tam sayı operandlarla bit işlemi yapar.

Operatör	Adı	Açıklama
`a & b`	VE	iki bit de 1 ise 1
`a | b`	VEYA	en az biri 1 ise 1
`a ^ b`	XOR	bitler farklıysa 1
`~a`	DEĞİL	bütün bitleri ters çevirir
`a << n`	sola kaydırma	bitleri `n` basamak sola iter; 2^n ile çarpmak gibidir
`a >> n`	sağa kaydırma	bitleri `n` basamak sağa iter; işaret biti korunur
`a >>> n`	işaretsiz sağa kaydırma (C# 11)	soldan sıfır doldurur

Örnek olarak `a = 1100` (12) ve `b = 1010` (10) sayılarını ele alalım:

	İkilik	Onluk
`a`	`1100`	12
`b`	`1010`	10
`a & b`	`1000`	8
`a | b`	`1110`	14
`a ^ b`	`0110`	6

bitOperatorleri.cs

```
int a = 0b1100;                         // 12
int b = 0b1010;                         // 10
Console.WriteLine($"a & b = {a & b}");
Console.WriteLine($"a | b = {a | b}");
Console.WriteLine($"a ^ b = {a ^ b}");
Console.WriteLine($"~a = {~a}");
Console.WriteLine($"a << 2 = {a << 2}");
Console.WriteLine($"a >> 1 = {a >> 1}");
Console.WriteLine($"-16 >> 2 = {-16 >> 2}");
Console.WriteLine($"-16 >>> 2 = {-16 >>> 2}");
Console.WriteLine($"2 ^ 3 = {2 ^ 3}");
```

```
Console.WriteLine($"13 & 1 = {13 & 1}");
int kay = 33;
Console.WriteLine(1 << kay);
```

```
ÇIKTI
a & b = 8
a | b = 14
a ^ b = 6
~a = -13
a << 2 = 48
a >> 1 = 6
-16 >> 2 = -4
-16 >>> 2 = 1073741820
2 ^ 3 = 1
13 & 1 = 1
2
```

Çıktıdan birkaç sonuç çıkaralım:

- `~a` her zaman `-a - 1` verir: `~12` sonucu `-13`.
- `>>` operatörü negatif sayılarda işaret bitini korur (aritmetik kaydırma): `-16 >> 2` sonucu `-4`. `>>>` ise işaret bitini korumaz, soldan sıfır doldurur; bu yüzden negatif bir sayıda çok büyük bir pozitif değer verir.
- Kaydırma miktarı, işlenenin bit sayısına göre kırpılır: `int` için yalnızca alttaki 5 bit kullanılır. Bu yüzden `1 << 33` işlemi `1 << 1` gibi çalışır ve `2` verir. C ve C++'ta bit sayısından büyük kaydırma tanımsız davranıştır; C# davranışı tanımlamıştır.
- `13 & 1` işlemi bir sayının en sağdaki bitini verir: bit 1 ise sayı tektir. Negatif olmayan sayılarda `x & 1` ile `x % 2` aynı sonucu verir.

> **◆ Kritik Nokta**
>
> C#'ta üs alma operatörü yoktur. `2 ^ 3` ifadesi 8 değil, bitleri karşılaştıran XOR işlemi olduğu için 1 verir. Üs almak için `Math.Pow(2, 3)` kullanırız (Bölüm 46).

Bit işlemlerinin en yaygın kullanımı, tek bir sayı içinde birden çok açık/kapalı bilgiyi (bayrak) saklamaktır. Her bayrak sayının farklı bir bitidir:

▸ bayrakOrnek.cs

```
const int Oku = 1;             // 0b001
const int Yaz = 2;             // 0b010
const int Calistir = 4;        // 0b100

int izin = Oku | Yaz;          // iki bayrağı birlikte aç
Console.WriteLine((izin & Yaz) != 0);        // Yaz var mı?
Console.WriteLine((izin & Calistir) != 0);   // Çalıştır var mı?
izin &= ~Yaz;                  // Yaz bayrağını kapat
Console.WriteLine(izin);
```

```
ÇIKTI
True
False
1
```

Bit düzeyindeki işlemlerin daha geniş bir incelemesini C bölümünde yaptık (Bölüm 16); C#'ta aynı fikirler `enum` ve `[Flags]` özniteliğiyle daha okunaklı hale gelir (Bölüm 52).

Öncelik tuzağı

`&` operatörünün önceliği `==` operatörünün altındadır. Bu yüzden `x & 1 == 0` ifadesi `(x & 1) == 0` olarak değil, `x & (1 == 0)` olarak gruplanır. C'de bu kod derlenir ve sessizce yanlış çalışır (`x & 0`, yani her zaman 0). C# `int` ile `bool`'u `&` ile işleme sokamadığı için hatayı derleme sırasında yakalar:

bitHata.cs

```
int x = 6;
Console.WriteLine(x & 1 == 0);
```

DERLEYİCİ ÇIKTISI

```
bitHata.cs(2,19): error CS0019: Operator '&' cannot be applied to operands of type 'int' and
'bool'
```

Doğrusu parantezle yazmaktır: `(x & 1) == 0`.

Operatörlerin öncelik tablosu

Şimdiye kadar gördüğümüz operatörleri yüksekten düşüğe doğru sıralayalım. Aynı satırdaki operatörler eşit önceliklidir ve (atama, `?:` ve `??` dışında) soldan sağa birleşir.

Sıra	Operatörler
1	Birincil: `x.y`, `f(x)`, `a[i]`, `x++`, `x--`, `new`, `typeof`, `sizeof`, `checked`, `unchecked`
2	Tekli: `+x`, `-x`, `!x`, `~x`, `++x`, `--x`, `(T)x`
3	Çarpımsal: `*`, `/`, `%`
4	Toplamsal: `+`, `-`
5	Kaydırma: `<<`, `>>`, `>>>`
6	İlişkisel: `<`, `>`, `<=`, `>=`, `is`, `as`
7	Eşitlik: `==`, `!=`
8	Bit VE: `&`
9	Bit XOR: `^`
10	Bit VEYA: `|`
11	Koşullu VE: `&&`
12	Koşullu VEYA: `||`
13	Null birleştirme: `??`
14	Üçlü koşul: `? :` (sağdan sola)
15	Atama: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`, `??=` (sağdan sola)

Tablonun pratik bir özeti vardır: aritmetik, karşılaştırma ve mantıksal operatörleri doğal sırasıyla karıştırabiliriz (`a + b > c && d`). Bit operatörleri ise bu sıranın ortasına yerleşir ve şaşırtıcı sonuçlar verebilir. Şüphede kaldığımızda parantez kullanırız; parantez hem okunaklıdır hem de güvenlidir.

42.12 Tip Dönüşümleri ve Taşma

Programlar sık sık bir tipteki değeri başka bir tipe çevirmek zorunda kalır: tam sayıyı gerçek sayıyla toplamak, klavyeden gelen metni sayıya çevirmek, bir hesabın sonucunu daha küçük bir değişkene sığdırmak gibi. C# bu dönüşümleri üç gruba ayırır.

Örtük (genişleyen) dönüşüm

Veri kaybı olmayacağı garanti ise derleyici dönüşümü kendisi yapar. Buna örtük dönüşüm (implicit conversion) denir. Sayısal tiplerde genel yön küçükten büyüğe doğrudur:

```
sbyte -> short -> int -> long -> float -> double
byte  -> short -> int -> long
byte  -> ushort -> uint -> ulong
char  -> ushort -> int
int   -> decimal, long -> decimal
```

Bu şemanın okunması şöyledir: okun başındaki tip, sonundaki tipe (ve onun dönüşebildiği tüm tiplere) kendiliğinden dönüşebilir. Birkaç kural akılda tutulmalıdır:

- `char`, `int` ve ondan geniş sayısal tiplere örtük dönüşür; ama `short`'a dönüşmez.
- `int` ve `long`, `float`'a örtük dönüşür. Bu dönüşümde bir miktar duyarlık kaybolabilir; yine de C# buna izin verir.
- `double` ile `decimal` arasındaki dönüşüm **her iki yönde de açıktır**; `float` ile `decimal` arasında da örtük dönüşüm yoktur. `decimal` ile `double`'ı aynı işlemde kullanamayız.

Bir ifadede farklı tipler bir araya gelirse küçük olan büyüğe yükseltilir (binary numeric promotion). Kural kabaca şöyledir: işlenenlerden biri `decimal` ise sonuç `decimal`; değilse biri `double` ise `double`; değilse biri `float` ise `float`; değilse biri `ulong` ise `ulong`; değilse biri `long` ise `long`; değilse biri `uint` ise ve diğeri işaretli ise `long`, değilse `uint`; hepsi bunun altındaysa `int`. Küçük tipler (`byte`, `short`, `char`) işleme girmeden önce `int`'e yükseltilir. Bu yüzden iki `byte`'ı toplamanın sonucu `byte` değil `int`'tir.

▸ tipYukselme.cs

```
byte b1 = 100, b2 = 100;
var s1 = b1 + b2;           // byte + byte -> int
var s2 = 5 + 3L;            // int + long -> long
var s3 = 5 + 2.5f;          // int + float -> float
var s4 = 5 + 2.5;           // int + double -> double
var s5 = 5 + 2.5m;          // int + decimal -> decimal
uint u = 3;
int i = -5;
var s6 = u + i;             // uint + int -> long
char ch = 'a';
var s7 = ch + 1;            // char + int -> int

Console.WriteLine($"{s1} {s1.GetType().Name}");
Console.WriteLine($"{s2} {s2.GetType().Name}");
Console.WriteLine($"{s3} {s3.GetType().Name}");
Console.WriteLine($"{s4} {s4.GetType().Name}");
Console.WriteLine($"{s5} {s5.GetType().Name}");
Console.WriteLine($"{s6} {s6.GetType().Name}");
Console.WriteLine($"{s7} {s7.GetType().Name}");
```

```
ÇIKTI
200 Int32
8 Int64
7.5 Single
7.5 Double
7.5 Decimal
-2 Int64
98 Int32
```

`byte + byte` toplamının `int` olması, daha önce gördüğümüz `b = b + 5` hatasının nedenidir: sağ taraf `int` olur ve `int`'ten `byte`'a örtük dönüşüm yoktur. `uint + int` işleminde sonucun `long` olması da dikkate değerdir; çünkü iki tipin de bütün değerlerini tutabilen tek tip `long`'dur. Tam sayı bölmesi kuralının nedeni de aynıdır: `7 / 2.0` işleminde `7`, önce `7.0`'a dönüştürülür.

Açık (daralan) dönüşüm: cast

Büyük bir tipi daha küçük bir tipe koymak veri kaybına yol açabilir. C# bunu kendiliğinden yapmaz; veri kaybını göze aldığımızı açıkça belirtmemizi bekler. Bunun için parantez içinde hedef tipi yazarız. Bu işleme **cast** (tip dönüştürme) denir.

▸ daralmaHata.cs

```
int x = 3.5;
Console.WriteLine(x);
```

```
DERLEYİCİ ÇIKTISI

daralmaHata.cs(1,9): error CS0266: Cannot implicitly convert type 'double' to 'int'. An explicit
conversion exists (are you missing a cast?)
```

Hata mesajının sonundaki "açık bir dönüşüm var" ipucu `(int)` cast'ine işaret eder. Gerçek sayıyı tam sayıya çevirmenin iki yolu vardır: cast ve `Convert.ToInt32`. İkisi aynı davranmaz:

▸ donusum.cs

```
double d = 3.99;
int i = (int)d;                         // ondalık kısım KESİLİR
Console.WriteLine(i);
Console.WriteLine((int)-3.99);
Console.WriteLine(Convert.ToInt32(d));  // YUVARLAR
Console.WriteLine(Convert.ToInt32(3.5));
Console.WriteLine(Convert.ToInt32(2.5));

int toplam = 7, adet = 2;
Console.WriteLine(toplam / adet);
Console.WriteLine((double)toplam / adet);
Console.WriteLine((double)(toplam / adet));
```

```
ÇIKTI
3
-3
4
4
2
3
3.5
3
```

Satırları sırayla okuyalım:

- `(int)3.99` sonucu `3`: cast ondalık kısmı keser, yuvarlamaz. `(int)-3.99` da `-3` olur (sıfıra doğru kesme).
- `Convert.ToInt32(3.99)` sonucu `4`: `Convert` yuvarlar. Tam ortadaki değerde (`.5`) ise çift sayıya yuvarlar: `3.5` sonucu `4`, `2.5` sonucu `2`. Bu davranışa **banker yuvarlaması** (banker's rounding) denir.
- `(double)toplam / adet` ifadesinde cast, bölmeden önce yapılır: `7.0 / 2 = 3.5`.
- `(double)(toplam / adet)` ifadesinde önce parantez içi hesaplanır: `7 / 2 = 3` (tam sayı bölmesi), sonra `3.0` olur. Ondalık kısım çoktan kaybolmuştur ve ekrana `3` yazılır.

Banker yuvarlaması, çok sayıda yuvarlama yapıldığında hatanın hep aynı yönde birikmemesini sağlar. `Math.Round` de aynı kuralı izler; istenirse `MidpointRounding.AwayFromZero` ile "okuldaki" yuvarlama seçilebilir. `Math.Round` metodunu Bölüm 46'da ayrıntılı işleyeceğiz; burada davranışını yan yana görelim:

yuvarlama.cs

```
Console.WriteLine(Math.Round(2.5));
Console.WriteLine(Math.Round(3.5));
Console.WriteLine(Math.Round(-2.5));
Console.WriteLine(Math.Round(2.5, MidpointRounding.AwayFromZero));
Console.WriteLine(Math.Round(3.14159, 2));
Console.WriteLine(Math.Round(0.125m, 2));
Console.WriteLine(Math.Round(0.125m, 2, MidpointRounding.AwayFromZero));
```

```
ÇIKTI
2
4
-2
3
3.14
0.12
0.13
```

◆ Kritik Nokta

`Math.Round(2.5)` sonucu `3` değil `2`'dir. `Math.Round(x)` ve `Convert.ToInt32(x)` tam ortadaki değerde çift sayıya gider: `0.5` sonucu `0`, `1.5` sonucu `2`, `2.5` sonucu `2`, `3.5` sonucu `4`. Okuldaki gibi yukarı yuvarlamak için `MidpointRounding.AwayFromZero` yazmak zorundayız. Para hesaplarında hangi yuvarlama kuralının istendiğini önceden belirlemek gerekir.

Tam sayılar arasındaki daraltmada, değer hedef tipe sığmıyorsa üst bitler atılır. Bu davranışı taşma başlığında inceleyeceğiz.

▲ Sık Yapılan Hata

Ortalama hesabında tam sayı bölmesi sık yapılan bir hatadır. Üç notu `int` olarak toplayıp `double ort = toplam / 3;` yazarsak, bölme `int / int` olarak yapılır ve ondalık kısım atılır; `ort` değişkeni `double` olsa bile sonuç `82.33` yerine `82` çıkar. Doğrusu `toplam / 3.0` ya da `(double)toplam / 3` yazmaktır.

Üç Dilde · Tip dönüştürme yazımı *C · C++ · C#*

C'de cast `(tip)deger` biçimindedir ve her tür dönüşüm için aynı yazım kullanılır. C++ bu dönüşümleri `static_cast`, `reinterpret_cast`, `const_cast` gibi niyeti belli eden operatörlere ayırdı (Bölüm 22). C# ise

`(tip)deger` yazımını korur; ama dönüşümün güvenliği tipe bağlıdır: yanlış bir nesne dönüşümü çalışma zamanında `InvalidCastException` fırlatır; güvenli kodda bellek üzerinde işaretçi hileleri yapılamaz.

```
double d = 3.99;
int i = (int)d;                 // C
```

```
double d = 3.99;
int i = static_cast<int>(d);    // C++
```

```
double d = 3.99;
int i = (int)d;                 // C#
int j = Convert.ToInt32(d);     // C#: yuvarlayarak dönüştürme
```

Metinden sayıya: Parse, TryParse ve Convert

Klavyeden, dosyadan ya da ağdan gelen veriler çoğunlukla metindir. Metni sayıya çevirmenin üç yolu vardır:

Yöntem	Geçersiz girdide ne olur?	Ne zaman kullanılır?
`int.Parse(s)`	`FormatException` fırlatır	girdinin geçerli olduğundan eminsek
`Convert.ToInt32(s)`	`FormatException` fırlatır; `null` için 0 verir	farklı tiplerden dönüşümde
`int.TryParse(s, out int sonuc)`	`false` döner, program devam eder	kullanıcı girdisi gibi güvenilmeyen veride

`TryParse`, sonucu `out` anahtar sözcüğüyle işaretlenmiş bir parametreyle geri verir; `out` parametreleri Bölüm 46'da ayrıntılı işlenecek. Şimdilik `out int sonuc` yazımının "sonucu bu yeni değişkene yaz" anlamına geldiğini bilmemiz yeterlidir.

parseOrnek.cs

```
string metin = "123";
int a = int.Parse(metin);
Console.WriteLine(a + 1);
int b = Convert.ToInt32("456");
Console.WriteLine(b + 1);

bool tamam = int.TryParse("12x", out int c);
Console.WriteLine($"{tamam} {c}");
if (int.TryParse("789", out int d))
{
    Console.WriteLine(d * 2);
}

try
{
    int e = int.Parse("abc");
}
catch (FormatException)
{
    Console.WriteLine("Geçersiz sayı biçimi");
}

Console.WriteLine(Convert.ToString(255, 2));
```

```
Console.WriteLine(Convert.ToString(255, 16));
Console.WriteLine(Convert.ToInt32("FF", 16));
Console.WriteLine(Convert.ToInt32("1010", 2));
```

```
ÇIKTI
124
457
False 0
1578
Geçersiz sayı biçimi
11111111
ff
255
10
```

`TryParse` başarısız olursa `out` parametresine tipin varsayılan değerini (`int` için 0) yazar; bu yüzden `c` değeri 0'dır. `Convert.ToString(sayi, taban)` ve `Convert.ToInt32(metin, taban)`, 2, 8, 10 ve 16 tabanları arasında dönüşüm yapar.

Tersi, yani sayıyı metne çevirmek, her tipin `ToString()` metoduyla yapılır. Birleştirme ve interpolasyon (`$"{sayi}"`) bu çağrıyı bizim yerimize yapar.

> **Not · Gerçek sayıları okumak**
>
> `double.Parse("3.75")` ve `double.TryParse` gibi gerçek sayı dönüşümleri **kültür ayarına** bağlıdır. Türkçe bölge ayarlı bir bilgisayarda ondalık ayraç virgüldür ve `"3.75"` geçerli bir sayı olarak okunmayabilir; `"3,75"` okunur. Girdi güvenilmez ya da farklı bilgisayarlarda aynı biçimde okunmalıysa kültür açıkça belirtilir (`CultureInfo.InvariantCulture`). Bu konuyu Bölüm 43'te ele alacağız.

Taşma: checked ve unchecked

Her tipin bir sınırı vardır. Bir hesabın sonucu bu sınırın dışına çıkarsa **taşma** (overflow) oluşur. C# varsayılan olarak bu durumda hata vermez; sayı, bir saatin 12'den 1'e dönmesi gibi diğer uca sarar ve sonuç sessizce yanlış çıkar:

▸ tasma.cs

```
int enBuyuk = int.MaxValue;
Console.WriteLine(enBuyuk);
Console.WriteLine(enBuyuk + 1);

int bin = 100_000;
int saniye = bin * bin;          // taşar!
long dogru = 100_000L * bin;     // çarpma long olarak yapılır
Console.WriteLine($"{saniye} / {dogru}");

byte kucuk = 250;
kucuk += 10;                     // 260, byte'a sığmaz
Console.WriteLine(kucuk);

int buyuk = 300;
Console.WriteLine((byte)buyuk);  // üst bitler atılır
```

```
ÇIKTI
2147483647
-2147483648
1410065408 / 10000000000
4
44
```

`bin * bin` işleminde iki `int` çarpılır; sonuç `int` sınırını aşar ve anlamsız bir sayı çıkar. Sonucu `long` bir değişkene koymak yetmez, çünkü çarpma işlemi `int` olarak yapılıp sonra genişletilir. Çarpmanın kendisinin `long` olması için bir işlenene `L` eki ekleyerek işleme `long` olarak başlamamız gerekir. `byte` örneğinde `260`, 0-255 aralığına sığmadığı için başa dönüp `4` olur. `(byte)300` işleminde ise sayının alt 8 biti korunur: `300 - 256 = 44`.

Bu sessiz davranış C'dekinden iki bakımdan farklıdır. C ve C++'ta **işaretli** tam sayı taşması tanımsız davranıştır; C#'ta davranış tanımlıdır ve sayı sarar. İşaretsiz tiplerin sarması ise C'de de tanımlıdır.

> **İpucu**
>
> Derleyici, değerleri derleme sırasında bilinen bir ifade taşıyorsa (`int.MaxValue + 1` gibi) bunu `CS0220` hatasıyla yakalar. Bu nedenle yukarıdaki örneklerde sabit yerine değişken kullandık. Taşma yalnızca çalışma sırasında, değişkenlerle ortaya çıkıyorsa C# sessiz kalır; asıl tehlike budur.

Sessiz taşma istemiyorsak işlemi `checked` ile yaparız. Taşma olursa program `OverflowException` (taşma istisnası) fırlatır. Bir ifade için `checked(ifade)`, bir deyim grubu için `checked { ... }` yazılır. Tersine, `unchecked` taşmaya kasıtlı olarak izin verir; sabit ifadelerde derleyicinin `CS0221` ya da `CS0220` hatasını vermesini engellemek için gerekir.

▸ kontrollu.cs

```
int enBuyuk = int.MaxValue;

try
{
    int sonuc = checked(enBuyuk + 1);
    Console.WriteLine(sonuc);
}
catch (OverflowException)
{
    Console.WriteLine("Taşma yakalandı");
}

checked
{
    try
    {
        byte k = 250;
        k += 10;
        Console.WriteLine(k);
    }
    catch (OverflowException)
    {
        Console.WriteLine("Blokta taşma");
    }
}

int sabit = unchecked((int)3_000_000_000u);   // bilerek sar
Console.WriteLine(sabit);
```

```
ÇIKTI
Taşma yakalandı
Blokta taşma
-1294967296
```

`try` ve `catch` yapısını Bölüm 53'te ayrıntılı işleyeceğiz. Burada yalnızca taşmanın bir istisna olarak ortaya çıktığını görmemiz yeterlidir. Yakalanmayan bir `OverflowException`, programı hata mesajıyla sonlandırır.

Bütün bir projede taşma denetimini açmak istersek `.csproj` dosyasına `<CheckForOverflowUnderflow>true</CheckForOverflowUnderflow>` satırını ekleriz. O zaman varsayılan davranış `checked` olur ve `unchecked` ile açıkça sarmaya izin vermedikçe taşma istisna doğurur. Denetim biraz işlem yükü getirdiği için varsayılan olarak kapalıdır.

> **◆ Kritik Nokta**
>
> Tam sayı taşması sessizdir. Para, süre ya da sayaç gibi sonucu önemli değerlerde `checked` kullanmak ya da hesabı `long` ya da `decimal` gibi daha geniş bir tipte yapmak güvenli yoldur. Hızı kritik olan ve sarmanın istenen davranış olduğu yerlerde (örneğin sağlama toplamı, karma hesapları) ise `unchecked` açıkça yazılarak niyet belirtilir.

42.13 Nullable Değer Tipleri

Değer tipli bir değişken her zaman bir değer taşır; `int` değişkeni `null` olamaz. Oysa gerçek verilerde "değer yok" durumu sık görülür: formda boş bırakılmış bir yaş alanı, veritabanında bilgisi girilmemiş bir sütun, henüz ölçülmemiş bir sıcaklık. C'de bu durum genellikle özel bir değerle (`-1`) ya da ayrı bir bayrak değişkeniyle ifade edilir; C++17 bunun için `std::optional<T>` getirmiştir (Bölüm 39). C#'ta çözüm, tipin sonuna soru işareti koymaktır.

> **Tanım: Nullable değer tipi**
>
> `T?` yazımı, `T` değer tipinin değerlerine ek olarak `null` değerini de taşıyabilen bir tip tanımlar (örneğin `int?`, `double?`, `bool?`). Arka planda `System.Nullable<T>` yapısı kullanılır. Bir nullable değişkenin değeri olup olmadığını `HasValue` özelliği, değerini ise `Value` özelliği verir.

▸ nullableOrnek.cs

```
int? yas = null;
Console.WriteLine(yas.HasValue);
Console.WriteLine(yas ?? -1);          // null ise -1 kullan
yas = 25;
Console.WriteLine(yas.HasValue);
Console.WriteLine(yas.Value + 1);

int? bos = null;
int? toplam = yas + bos;               // null ile işlem null verir
Console.WriteLine(toplam.HasValue);

int? a = null;
int? b = null;
Console.WriteLine(a == b);             // iki null eşittir
Console.WriteLine(a < 5);              // null ile karşılaştırma false

int? sayi = null;
sayi ??= 10;                           // null ise ata
```

```
Console.WriteLine(sayi);
sayi ??= 20;                            // dolu: değişmez
Console.WriteLine(sayi);

double? ort = null;
Console.WriteLine(ort.GetValueOrDefault());
```

```
ÇIKTI
False
-1
True
26
False
True
False
10
10
0
```

Programdan çıkan kuralları sıralayalım:

- `??` (null birleştirme, null-coalescing) operatörü soldaki değer `null` değilse onu, `null` ise sağdaki değeri verir: `yas ?? -1`.
- `??=` operatörü, soldaki değişken `null` ise sağdaki değeri atar; doluysa dokunmaz. Bu operatör C# 8.0 ile geldi.
- Nullable değerlerle aritmetik yapıldığında işlenenlerden biri `null` ise sonuç da `null` olur. Karşılaştırmada ise `null` herhangi bir sayıdan küçük ya da büyük sayılmaz: `a < 5` ifadesi `false` verir. İki `null` birbirine `==` ile eşit çıkar.
- `GetValueOrDefault()` değer varsa onu, yoksa tipin varsayılan değerini (`double` için 0) verir.

Nullable değer tipinin değerini, `null` olabileceğini unutarak normal bir değişkene atayamayız:

▸ nullableHata.cs

```
int? x = null;
int y = x;
```

```
DERLEYİCİ ÇIKTISI

nullableHata.cs(2,9): error CS0266: Cannot implicitly convert type 'int?' to 'int'. An explicit
conversion exists (are you missing a cast?)
```

Çözüm, `null` durumunu bilinçli olarak ele almaktır: `int y = x ?? 0;` ya da `if (x.HasValue) { ... }`. `Value` özelliğini ya da `(int)x` dönüşümünü değer yokken kullanırsak çalışma sırasında `InvalidOperationException` oluşur.

> *Tarihçe · Nullable değer tipleri*
>
> Nullable değer tipleri ve `??` operatörü C# 2.0 ile (2005) generic tiplerle birlikte geldi. `??=` operatörü C# 8.0'da eklendi. Aynı sürümde, referans tipleri için ayrı bir "nullable referans tipi" denetimi de getirildi; `string?` yazımı onunla ilgilidir. Bu özelliği Bölüm 59'da işleyeceğiz.

Bölüm Özeti

- Değişken; ad, tip ve değerden oluşur. C#'ta yerel değişken, değer atanmadan okunamaz (`CS0165`); aynı kapsamda aynı ad iki kez tanımlanamaz.
- Adlar harf ya da `_` ile başlar, rakam içerebilir, anahtar sözcük olamaz ve büyük/küçük harfe duyarlıdır. Yerel değişkenler `camelCase`, metot, sınıf ve sabitler `PascalCase` yazılır.
- Değer tiplerinde (`int`, `double`, `bool`, `char`, `struct`) atama değeri kopyalar; referans tiplerinde (`string`, `class`, dizi) atama referansı kopyalar ve iki değişken aynı nesneyi gösterir.
- Tip boyutları sabittir: `int` 32 bit, `long` 64 bit, `char` 16 bit (UTF-16), `bool` 1 bayt, `decimal` 16 bayt. `int.MaxValue`, `int.MinValue` ve `sizeof` ile sınırlar öğrenilir.
- `int`, `long` ve `double` günlük işlerin varsayılan tipleridir. Para gibi tam ondalık sonuç gereken hesaplarda `decimal` kullanılır; `double` değerler `==` ile karşılaştırılmaz.
- Literal tipleri yazılışından anlaşılır: `42` `int`, `42L` `long`, `3.14` `double`, `3.14f` `float`, `3.14m` `decimal`. `1_000_000`, `0xFF`, `0b1010` yazımları okunurluğu artırır.
- `var` tipi derleme zamanında çıkarır ama C# yine statik tiplidir. `const` derleme zamanında bilinen sabitler içindir; `readonly` ise alanlara bir kez atanabilen değerler içindir.
- Tam sayı bölmesi tam sayı verir (`7 / 2 = 3`); `%` kalan verir ve işareti soldaki sayınınkiyle aynıdır. Tam sayıyı sıfıra bölmek `DivideByZeroException` doğurur, gerçek sayıda sonuç `Infinity` ya da `NaN` olur.
- `x++` değeri kullanıp artırır, `++x` artırıp kullanır. C# ifadeleri soldan sağa değerlendirir; C'deki tanımsız davranış burada yoktur.
- `bool` sayıya, sayı `bool`'a dönüşmez; `if (x = 5)` derleme hatasıdır. `&&` ve `||` kısa devre yapar, `&` ve `|` yapmaz.
- Bit operatörleri `&`, `|`, `^`, `~`, `<<`, `>>`, `>>>` tam sayıların bitleri üzerinde çalışır. `^` üs alma değil XOR'dur; `x & 1 == 0` yazımı `(x & 1) == 0` anlamına gelmez.
- Genişleyen dönüşümler otomatiktir; daralan dönüşümler cast ister. `(int)3.99` keser (3), `Convert.ToInt32(3.99)` yuvarlar (4) ve tam ortada çift sayıya gider (`2.5` sonucu 2). `byte + byte` sonucu `int`'tir; `b += 5` gizli bir cast içerir.
- Metni sayıya çevirmek için güvenilmeyen girdilerde `TryParse` kullanılır; `Parse` geçersiz girdide `FormatException` fırlatır.
- Tam sayı taşması varsayılan olarak sessizdir ve sayı diğer uca sarar. `checked` taşmada `OverflowException` fırlatır, `unchecked` sarmayı açıkça belirtir.
- `T?` ile değer tipleri `null` taşıyabilir. `??` yedek değer verir, `??=` yalnızca `null` ise atar, `HasValue` değer olup olmadığını söyler.

Alıştırmalar

1. Aşağıdakilerden hangisi geçerli bir C# değişken adıdır?
   A) `3adim`
   B) `toplam-tutar`
   C) `_sayac1`
   D) `new`
   E) `fiyat$`

2. `double sonuc = 17 / 5; Console.WriteLine(sonuc);` komutları ekrana ne yazdırır?
   A) `3.4`
   B) `3`
   C) `3.0`
   D) `4`
   E) Derleme hatası verir

3. Aşağıdaki satırlardan hangisi derlenmez? (Her satır tek başına bir programdadır.)
   A) `long l = 100;`
   B) `float f = 2.5;`
   C) `double d = 10;`
   D) `decimal m = 5;`
   E) `int i = 'A';`

4. `int a = 3; int b = ++a * a--;` satırları çalıştıktan sonra `a` ve `b` değişkenlerinin değerleri nedir?
   A) `a = 3`, `b = 16`
   B) `a = 4`, `b = 16`
   C) `a = 3`, `b = 12`
   D) `a = 2`, `b = 15`
   E) `a = 4`, `b = 12`

5. `Console.WriteLine((int)9.8 + 23 % 6 * 2);` komutu ne yazdırır?
   A) `19`
   B) `20`
   C) `4`
   D) `14`
   E) `19.8`

6. Aşağıdaki program ekrana ne yazdırır?

```
int a = 17, b = 5;
double c = a / b;
double d = (double)a / b;
Console.WriteLine($"{a / b} {a % b}");
Console.WriteLine($"{c} {d}");
Console.WriteLine(b / a + b % a);
Console.WriteLine(a / (double)b * 0);
```

7. Aşağıdaki program ekrana ne yazdırır?

```
int n = 3;
int p = n++ + n++;
int q = ++n * 2;
int r = n-- - --n;
Console.WriteLine($"{n} {p} {q} {r}");
```

8. Aşağıdaki program ekrana ne yazdırır?

```
char h = 'a';
h = (char)(h + 2);
Console.WriteLine(h);
Console.WriteLine(h + 1);
Console.WriteLine((char)(h - 32));
```

```
Console.WriteLine("" + h + 1);
Console.WriteLine(Math.Round(6.5) + Math.Abs(-2));
Console.WriteLine(Math.Round(7.5) + " " + Math.Round(8.5));
```

9. Aşağıdaki program ekrana ne yazdırır?

```
int? a = null;
Console.WriteLine(a ?? 7);
a ??= 3;
a ??= 9;
int? b = a * 2;
Console.WriteLine($"{a} {b}");
int? c = null;
Console.WriteLine((c + 1).HasValue);
Console.WriteLine(c == null);
```

10. Aşağıdaki program ekrana ne yazdırır?

```
Sayac s1 = new Sayac();
Sayac s2 = s1;
s2.Deger = 5;
Cift c1 = new Cift();
Cift c2 = c1;
c2.Deger = 5;
string m1 = "kar";
string m2 = m1;
m2 += "ne";
Console.WriteLine($"{s1.Deger} {s2.Deger}");
Console.WriteLine($"{c1.Deger} {c2.Deger}");
Console.WriteLine($"{m1} {m2}");

class Sayac { public int Deger; }
struct Cift { public int Deger; }
```

11. Aşağıdaki kodda derleme hatası veren dört satır vardır. Bu satırları bulun ve her birini düzeltin.

```
byte kucuk = 200;
kucuk = kucuk + 100;
float oran = 0.5;
decimal tutar = 12.5m * 1.18;
int sayac = 0;
if (sayac = 0)
{
    Console.WriteLine("sıfır");
}
```

12. `byte b = 10;` tanımından sonra `b += 5;` satırı derlenirken `b = b + 5;` satırı neden derlenmez? Aradaki farkı açıklayın.

13. 4 basamaklı bir sayı `int sayi = 4821;` olarak verilmiştir. Basamakları toplayan ve sonucu ekrana yazdıran bir program yazın (çıktı `15` olmalıdır). Yalnızca `/` ve `%` operatörlerini kullanın; döngü gerekmez.

14. Santigrat derece cinsinden `double c = 37.5;` değişkeni verilmiştir. Fahrenayt karşılığını `F = C × 9 / 5 + 32` formülüyle hesaplayıp `37.5 C = 99.5 F` biçiminde yazdıran bir program yazın. Tam sayı bölmesi tuzağına dikkat edin.

15. Saniye cinsinden verilen `int toplamSaniye = 5025;` süresini saat, dakika ve saniyeye ayırıp `1 saat 23 dakika 45 saniye` biçiminde yazdıran bir program yazın. Yalnızca `/` ve `%` operatörlerini kullanın.

16. İki `int` değişkeni `a = 100_000` ve `b = 100_000` verilmiştir. Çarpımlarını önce `long` kullanarak doğru biçimde hesaplayıp yazdıran, ardından aynı çarpımı `checked` ile yapıp taşma olursa `int sınırı aşıldı` mesajı veren bir program yazın.

BÖLÜM 43

Konsol Girdi/Çıktı ve Biçimlendirme

BU BÖLÜMDE

- `Console.Write`, `Console.WriteLine`, `Console.ReadLine` ve `Console.ReadKey` ile konsoldan okuma ve yazmayı
- `string?` tipini ve `ReadLine()` sonucunu `??` ile güvenli hale getirmeyi
- Birleşik biçimlendirmeyi (`{0}`), dize içine değer yerleştirmeyi (`$"..."`) ve biçim belirteçlerini (`F2`, `N0`, `C`, `P`, `D5`, `X`)
- Genişlik ve hizalamayla (`{x,10}`, `{x,-10}`) düzgün tablolar yazdırmayı
- Kültür ayarının sayıların yazılışını ve okunuşunu nasıl değiştirdiğini; Türkçe ondalık virgülü ve `CultureInfo.InvariantCulture`'ı
- `int.Parse` ile `int.TryParse` arasındaki farkı ve güvenli bir girdi döngüsünü
- Türkçe karakterler için `Console.OutputEncoding` ayarını

Bölüm 42'de değişkenleri ve tipleri tanıdık; programlarımızın çıktıları ise şimdiye kadar yalnızca `Console.WriteLine` ile ekrana gitti. Bu bölümde konsolla iki yönlü konuşmayı öğreniyoruz: ekrana düzgün biçimlenmiş çıktı yazmayı ve klavyeden gelen veriyi güvenle okumayı.

C bilen okur için bu konunun karşılıkları `printf` ile `scanf` (Bölüm 6), C++ bilen okur için `cout` ile `cin`'dir (Bölüm 23). C# bu ikisinden farklı bir yol seçer: biçimlendirme bilgisi çıktı akışının içinde değil, **metnin içinde** durur. `Console.ReadLine()` ise her zaman bir satırı metin olarak getirir; sayıya çevirmeyi biz açıkça yaparız. Bu tasarım, `scanf`'in arabellekte kalan satır sonu ya da `cin`'in kilitlenen akış durumu gibi tuzaklarının çoğunu daha baştan ortadan kaldırır.

Bölüm 42'deki not burada da geçerlidir: kitaptaki C# çıktıları kültür bağımsız ayarla alınmıştır, ondalık ayraç noktadır. Kültür ayarlarının tam olarak ne yaptığını bu bölümün ilgili kısmında göreceğiz.

43.1 Console Sınıfı ve Yazdırma

`Console`, `System` ad alanındaki statik bir sınıftır; nesne oluşturmadan `Console.WriteLine(...)` biçiminde kullanılır. Standart girdi, standart çıktı ve standart hata akışlarına erişim bu sınıf üzerinden yapılır.

> **Tanım: Console.Write ve Console.WriteLine**
>
> `Console.Write(değer)` değeri metne çevirip standart çıktıya yazar ve satırın sonunda kalır. `Console.WriteLine(değer)` aynı işi yapar, ardından alt satıra geçer. Parametresiz `Console.WriteLine()` yalnızca boş bir satır yazar.

Her iki metot da hemen her tipten değeri kabul eder: sayı, `char`, `bool`, metin, hatta herhangi bir nesne. Değer önce `ToString()` ile metne çevrilir. C'de her tip için ayrı bir `printf` belirteci (`%d`, `%f`, `%c`) seçmek gerekirdi; C#'ta tip bilgisini derleyici bilir.

▸ yazdirma.cs

```
Console.Write("Merhaba");
Console.Write(", ");
Console.WriteLine("dünya");
Console.WriteLine();
Console.WriteLine(42);
Console.WriteLine(3.5);
Console.WriteLine(true);
Console.WriteLine('Z');
Console.WriteLine("Toplam: " + 5 + 3);
Console.WriteLine("Toplam: " + (5 + 3));
Console.WriteLine(5 + 3 + " toplam");
```

```
ÇIKTI
Merhaba, dünya

42
3.5
True
Z
Toplam: 53
Toplam: 8
8 toplam
```

Son üç satır `+` operatörünün iki anlamını gösterir. Operandlardan biri metinse `+` birleştirme yapar ve soldan sağa işlenir: `"Toplam: " + 5` önce `"Toplam: 5"` olur, sonra `+ 3` ile `"Toplam: 53"` çıkar. Toplamayı istiyorsak parantez kullanırız. `5 + 3 + " toplam"` ifadesinde ise sıra gereği önce iki sayı toplanır (8), sonra metinle birleşir. Bu tuzaktan kurtulmanın en temiz yolu, birazdan göreceğimiz `$"..."` yazımıdır.

> **Not · bool değerleri**
>
> `Console.WriteLine(true)` ekrana `True` yazar: C#'ta `bool` değerlerinin metin hali baş harfi büyük `True` ve `False`'tur. C'de `bool` aslında bir tam sayı olduğu için `printf("%d")` ile 1 ve 0 görürdük; C++'ta `cout << boolalpha` gerekirdi (Bölüm 23).

Kaçış dizileri ve düz metinler

C#'ta normal metinler C'dekiyle aynı kaçış dizilerini (escape sequence) kullanır: `\n` satır sonu, `\t` sekme, `\\` ters eğik çizgi, `\"` çift tırnak. Dosya yolları gibi çok sayıda `\` içeren metinler için metnin önüne `@` koyarak **düz metin** (verbatim string) yazabiliriz. Düz metinde `\` özel anlam taşımaz, çift tırnak için ise tırnak iki kez yazılır.

C# 11 ile gelen **ham metin** (raw string literal) üç ya da daha çok çift tırnakla başlar ve biter. İçinde kaçış gerekmez; tırnaklar, ters eğik çizgiler ve satır sonları olduğu gibi yazılır. Kapanış tırnaklarının girintisi, bütün satırlardan çıkarılır.

▸ metinBicimleri.cs

```
Console.WriteLine("Birinci satır\nİkinci satır");
Console.WriteLine("Yol: C:\\temp\\yeni");
Console.WriteLine(@"Yol: C:\temp\yeni");
```

```
Console.WriteLine("Dedi ki: \"merhaba\"");
Console.WriteLine(@"Dedi ki: ""merhaba""");

string json = """
    {"ad": "Elif", "yas": 20}
    """;
Console.WriteLine(json);
```

```
ÇIKTI
Birinci satır
İkinci satır
Yol: C:\temp\yeni
Yol: C:\temp\yeni
Dedi ki: "merhaba"
Dedi ki: "merhaba"
{"ad": "Elif", "yas": 20}
```

> **▲ Sık Yapılan Hata · Geçersiz kaçış dizisi**
>
> `"C:\temp\yeni"` yazarsak `\t` bir sekmeye dönüşür, `\y` ise tanımlı olmayan bir kaçış dizisi olduğu için program **derlenmez** (`CS1009`). C'de bu sessizce bir uyarıya dönüşebilirdi. Yol yazarken `@"C:\temp\yeni"` ya da `"C:\\temp\\yeni"` kullanmak gerekir.

Hata akışı ve yönlendirme

Programlar iki ayrı akışa yazabilir: normal çıktı (`Console.Out` , `Console.WriteLine` bunu kullanır) ve hata iletileri için ayrılmış `Console.Error` . C'deki `stderr` ve C++'taki `cerr` ile aynı işi görür (Bölüm 23). İki akış ekranda aynı yerde görünür; ama komut satırında ayrı ayrı yönlendirilebilir.

```
▸ hataAkisi.cs
Console.WriteLine("Normal çıktı");
Console.Error.WriteLine("Hata iletisi");
```

```
TERMİNAL
$ dotnet run > cikti.txt
Hata iletisi
$ cat cikti.txt
Normal çıktı
```

Burada `>` yalnızca standart çıktıyı dosyaya yönlendirdi; hata iletisi ekrana düştü. Aynı mekanizma girdi için de çalışır: `dotnet run < girdi.txt` komutu, `ReadLine()` çağrılarını klavye yerine dosyadan besler. Bu kitaptaki örneklerin çıktılarını doğrularken de bu yöntemi kullanıyoruz.

Konsolun diğer olanakları

`Console` sınıfında ekranı yönetmek için birkaç üye daha vardır. Bunlar öğrenme için zorunlu değildir; ama küçük konsol uygulamalarında işe yarar:

Üye	Ne yapar?
`Console.Clear()`	Ekranı temizler
`Console.ForegroundColor` , `Console.BackgroundColor`	Yazı ve zemin rengini ayarlar; `ConsoleColor.Red` gibi değerler alır

Üye	Ne yapar?
`Console.ResetColor()`	Renkleri başlangıç değerine döndürür
`Console.Title`	Pencere başlığını değiştirir
`Console.Beep()`	Uyarı sesi çıkarır (Windows'ta)

```
Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("İşlem başarılı");
Console.ResetColor();
```

Yönlendirilmiş çıktıda renk kodları dosyaya yazılmaz; renk yalnızca gerçek bir terminalde görünür.

43.2 Klavyeden Okuma

Girdi tarafının temel metodu `Console.ReadLine()`'dır. Kullanıcı Enter'a basana kadar bekler, yazılan satırı (satır sonu karakteri olmadan) **metin olarak** döndürür. Kullanıcı sayı yazsa bile dönen şey metindir.

Okumadan önce kullanıcıya ne beklediğimizi söylemek için `Console.Write` (satır sonu olmayan sürüm) kullanırız; böylece imleç soru ile aynı satırda kalır:

adOku.cs

```
Console.Write("Adınız: ");
string ad = Console.ReadLine() ?? "";
Console.Write("Şehriniz: ");
string sehir = Console.ReadLine() ?? "";
Console.WriteLine($"{ad} ({ad.Length} karakter), {sehir}");
```

```
ÇIKTI
Adınız: Ayşe Nur
Şehriniz: Eskişehir
Ayşe Nur (8 karakter), Eskişehir
```

`ReadLine()` satırın tamamını, boşluklar dahil tek parça alır. C'de `scanf("%s")` ilk boşlukta durur, `cin >> ad` de öyledir; boşluklu metin için `fgets` ya da `getline` gerekirdi (Bölüm 6 ve 23). C#'ta bu ayrım yoktur: `ReadLine()` her zaman satırın tamamını okur. Bir satırdaki birden çok değeri ayırmak için metni bölmek gerekir; `string` metotlarını Bölüm 48'de işleyeceğiz.

string? ve ?? ile null denetimi

Kodda `?? ""` yazdığımıza dikkat edelim. `ReadLine()`'ın dönüş tipi `string` değil, `string?`'tir. Soru işareti, dönen değerin `null` olabileceğini söyler. Bölüm 42'de `int?` ile gördüğümüz nullable kavramının aynısı, bu kez referans tipleri için: `null`, "hiçbir nesneyi göstermiyor" demektir (C'deki `NULL` işaretçisine benzer).

> **Tanım: Girdi sonu (EOF)**
>
> Okunacak başka veri kalmadığında girdi akışı kapanır; buna dosya sonu (end of file, EOF) denir. Klavyede bu durum Linux ve macOS'ta `Ctrl+D`, Windows'ta `Ctrl+Z` ve Enter ile oluşturulur. Girdi bir dosyadan yönlendirilmişse dosyanın bitmesi de EOF'tur. `ReadLine()` bu durumda `null` döndürür.

Derleyici, `string?` değerini `string` değişkenine koymak istediğimizde uyarı verir. Uyarıyı susturmanın yolu null durumunu ele almaktır. `a ?? b` ifadesi, `a` `null` değilse `a`'yı, `null` ise `b`'yi verir. Dolayısıyla `Console.ReadLine() ?? ""` "satır geldiyse onu, gelmediyse boş metni al" demektir ve sonuç `string` tipindedir.

> **▲ Sık Yapılan Hata · Boş satır ile null aynı şey değildir**
>
> Kullanıcı hiçbir şey yazmadan Enter'a basarsa `ReadLine()` `null` değil, boş metin (`""`) döndürür. `??` yalnızca `null` için devreye girer, boş metni değiştirmez. Boş girdiyi yakalamak için `string.IsNullOrWhiteSpace(metin)` kullanılır. Bu metot hem `null`, hem `""`, hem de yalnızca boşluklardan oluşan metinler için `true` verir.

Tek tuş okumak: ReadKey

Enter beklemeden tek bir tuşa tepki vermek istiyorsak `Console.ReadKey()` kullanırız. Metot, basılan tuşu bir `ConsoleKeyInfo` değeri olarak döndürür; `intercept: true` verirsek tuş ekrana yazılmaz.

```
Console.Write("Devam için bir tuşa basın...");
ConsoleKeyInfo tus = Console.ReadKey(intercept: true);
Console.WriteLine($"\nBasılan tuş: {tus.Key}, karakter: {tus.KeyChar}");
```

`tus.Key` tuşun kendisini (`ConsoleKey.A`, `ConsoleKey.Enter`), `tus.KeyChar` ürettiği karakteri verir. Menülerde "e/h" gibi tek harflik seçimler için uygundur. `Console.Read()` ise C'deki `getchar` gibi bir karakterin kodunu `int` olarak döndürür (EOF'ta `-1`), ama satır tamponuyla çalıştığı için günlük kullanımda `ReadLine()` tercih edilir.

> **◆ Kritik Nokta**
>
> `ReadKey()` gerçek bir klavye gerektirir. Girdi dosyadan ya da başka bir programdan yönlendirilmişse `InvalidOperationException` fırlatır. Yönlendirilebilecek programlarda satır tabanlı `ReadLine()` kullanmak daha güvenlidir.

43.3 Birleşik Biçimlendirme

Birden çok değeri tek bir metinde birleştirmenin ilk yolu, C# 1.0'dan beri var olan **birleşik biçimlendirme**dir (composite formatting). Biçim metninde süslü parantez içine değerin sırası yazılır; değerler metinden sonra, virgülle ayrılarak verilir. Numaralar 0'dan başlar.

▸ birlesikBicim.cs

```
string urun = "Defter";
int adet = 3;
double fiyat = 45.5;

Console.WriteLine("{0} x {1} = {2} TL", urun, adet, adet * fiyat);
Console.WriteLine("{0} + {0} = {1}", 5, 10);

string satir = string.Format("{0}-{1}", urun, adet);
Console.WriteLine(satir);
Console.WriteLine(satir.Length);
```

```
ÇIKTI
Defter x 3 = 136.5 TL
5 + 5 = 10
Defter-3
8
```

`Console.WriteLine` ilk argümanı biçim metni sayar; kalan argümanlar `{0}`, `{1}`, ... yerlerine konur. Aynı numara birden çok kez kullanılabilir. `string.Format` aynı kuralları uygular ama sonucu ekrana yazmak yerine bir `string` olarak döndürür; biçimlenmiş metni saklamak ya da başka yere göndermek istediğimizde kullanılır.

Bu yazım C'nin `printf("%s x %d", ...)` yazımına benzer görünür; ama iki önemli fark vardır. Birincisi, belirteç **tipi** söylemez, yalnızca değerin **sırasını** söyler: tip uyuşmazlığı diye bir sorun yoktur. İkincisi, numara ile değer sayısı uyuşmazsa hata derleme sırasında değil, çalışırken ortaya çıkar:

> **▲ Sık Yapılan Hata · Eksik argüman**
>
> `Console.WriteLine("{0} {1}", 5);` derlenir, ama çalışırken `FormatException` fırlatır: metin iki değer ister, biz bir tane verdik. Fazla argüman ise sessizce yok sayılır. `printf`'te bu hata tanımsız davranışa yol açardı; burada en azından açık bir istisna alırız.

43.4 Dize İçine Değer Yerleştirme

Birleşik biçimlendirmenin zayıf yanı, metin ile değerlerin birbirinden uzak durmasıdır: `{3}`'ün hangi değer olduğunu görmek için gözümüzü sona götürmemiz gerekir. C# 6.0 bu sorunu **dize içine değer yerleştirme** (string interpolation) ile çözdü: metnin önüne `$` konur ve değerler süslü parantez içine doğrudan yazılır.

> *Tarihçe · Dize içine değer yerleştirme*
>
> `$"..."` yazımı C# 6.0 ile (2015, Visual Studio 2015) geldi. Derleyici bu yazımı arka planda yine bir biçimlendirme çağrısına çevirir; yani `$"{ad} {yas}"` ile `string.Format("{0} {1}", ad, yas)` aynı işi yapar. C++'ta benzer bir yazım C++20'de `std::format` ile eklendi, ama değerler yine metnin dışında verilir. C'de karşılığı yoktur.

Süslü parantezin içine yalnızca değişken değil, herhangi bir ifade yazılabilir:

`interpolasyon.cs`

```
string ad = "Deniz";
int yas = 20;
double ort = 3.456789;

Console.WriteLine($"{ad} {yas} yaşında.");
Console.WriteLine($"Gelecek yıl {yas + 1} yaşında olacak.");
Console.WriteLine($"Ad uzunluğu: {ad.Length}, büyük harf: {ad.ToUpper()}");
Console.WriteLine($"Reşit mi? {(yas >= 18 ? "evet" : "hayır")}");
Console.WriteLine($"Ortalama: {ort:F2}");
Console.WriteLine($"Süslü parantez: {{ad}} = {ad}");
Console.WriteLine("Ad: {ad}");
```

```
ÇIKTI
Deniz 20 yaşında.
Gelecek yıl 21 yaşında olacak.
Ad uzunluğu: 5, büyük harf: DENIZ
Reşit mi? evet
Ortalama: 3.46
Süslü parantez: {ad} = Deniz
Ad: {ad}
```

Programdaki küçük ayrıntılar şunlardır:

- Metot çağrıları ve hesaplamalar süslü parantezin içinde yapılabilir (`yas + 1`, `ad.ToUpper()`).
- Üçlü `?:` operatörünü kullanırken ifadeyi **parantez içine almalıyız**; çünkü iki nokta üst üste biçim belirtecini başlatır. Yukarıdaki `(yas >= 18 ? "evet" : "hayır")` bu yüzden parantezlidir.
- `{ort:F2}` yazımında iki noktadan sonra biçim belirteci gelir; bunu bir sonraki kısımda işleyeceğiz.
- Süslü parantezin kendisini yazdırmak için iki kez yazarız: `{{` ve `}}`.
- Sondaki satırda `$` yoktur; bu yüzden `{ad}` olduğu gibi yazılır.

> **▲ Sık Yapılan Hata · $ işaretini unutmak**
>
> `"Ad: {ad}"` yazarsak derleyici hata vermez ve uyarmaz; ekranda kelimesi kelimesine `Ad: {ad}` görünür. Süslü parantezli bir metin yazdırırken başındaki `$` işaretini kontrol etmek gerekir.

Düz metin ile interpolasyonu birleştirmek mümkündür: `$@"..."` ya da `@$"..."` yazımı hem `\` işaretini özel saymaz hem de süslü parantezleri değer yerleştirme için kullanır. Ham metinlerde ise `$` işaretinin sayısı, değer yerleştirmek için kaç süslü parantez gerektiğini belirler (`$$"""..."""` için iki parantez); bu, JSON gibi süslü parantezli metinlerde işe yarar.

Bu kitapta bundan sonra çıktı yazarken ağırlıklı olarak interpolasyonu kullanacağız. Birleşik biçimlendirmeyi tanımak da gereklidir; çünkü eski kodlarda, `string.Format` çağrılarında ve günlük (log) kütüphanelerinde sık görülür.

43.5 Biçim Belirteçleri

Süslü parantezin içinde değerden sonra iki nokta üst üste ve bir **biçim belirteci** (format specifier) yazarak değerin nasıl metne çevrileceğini seçeriz: `{deger:F2}`. Aynı belirteçler birleşik biçimlendirmede de (`{0:F2}`) ve doğrudan `ToString("F2")` çağrısında da geçerlidir.

> **Tanım: Biçim belirteci**
>
> Bir değerin metne çevrilirken nasıl yazılacağını belirleyen kısa kod. Bir harf (standart biçim) ve isteğe bağlı bir sayıdan oluşur: harf biçimin türünü, sayı ise genellikle basamak sayısını verir. Harf yerine `0`, `#`, `,` gibi simgelerle kendi kalıbımızı da (özel biçim) yazabiliriz.

En sık kullanılan standart sayısal belirteçler şunlardır:

Belirteç	Anlamı	Örnek (değer → sonuç)
`F2`	Sabit noktalı, 2 ondalık basamak	`3.14159` → `3.14`
`N2`	Binlik ayraçlı, 2 ondalık basamak	`1234567` → `1,234,567.00`

Belirteç	Anlamı	Örnek (değer → sonuç)
`C`	Para birimi (kültüre bağlı simge)	`1234.5` → `₺1.234,50` (Türkçe kültürde)
`P1`	Yüzde: 100 ile çarpar, simge ekler	`0.256` → `25.6 %` ya da `%25,6` (kültüre bağlı)
`D5`	Tam sayı, en az 5 basamak, başa sıfır	`42` → `00042`
`X` , `x4`	Onaltılık (büyük/küçük harf, en az 4 basamak)	`255` → `FF` , `00ff`
`E2`	Bilimsel gösterim	`12345.678` → `1.23E+004`
`G`	Genel: en kısa uygun biçim	`3.50` → `3.5`

Hepsini tek programda görelim:

▸ bicimler.cs

```
double pi = 3.14159;
int buyuk = 1234567;
int sayi = 255;

Console.WriteLine($"F2    -> {pi:F2}");
Console.WriteLine($"F0    -> {pi:F0}");
Console.WriteLine($"N0    -> {buyuk:N0}");
Console.WriteLine($"N2    -> {buyuk:N2}");
Console.WriteLine($"D8    -> {42:D8}");
Console.WriteLine($"X     -> {sayi:X}");
Console.WriteLine($"x4    -> {sayi:x4}");
Console.WriteLine($"E2    -> {12345.678:E2}");
Console.WriteLine($"0.00  -> {pi:0.00}");
Console.WriteLine($"#,##0 -> {buyuk:#,##0}");
Console.WriteLine($"000   -> {7:000}");
```

ÇIKTI

```
F2    -> 3.14
F0    -> 3
N0    -> 1,234,567
N2    -> 1,234,567.00
D8    -> 00000042
X     -> FF
x4    -> 00ff
E2    -> 1.23E+004
0.00  -> 3.14
#,##0 -> 1,234,567
000   -> 007
```

Son üç satır **özel biçim** örnekleridir. Özel biçimde `0` "burada mutlaka bir rakam olsun, yoksa sıfır yaz", `#` "rakam varsa yaz, yoksa boş bırak", `,` binlik ayraç, `.` ondalık ayraç anlamına gelir. Standart belirteçler çoğu iş için yeterlidir; özel biçimi yalnızca standartların vermediği bir kalıp gerektiğinde kullanırız.

C'nin `printf` belirteçleriyle kıyaslarsak: `%.2f` ↔ `F2` , `%x` ↔ `x` , `%05d` ↔ `D5` , `%e` ↔ `E` , `%g` ↔ `G` . Farklı olarak, `N` (binlik ayraç) ve `P` (yüzde) C'de hazır karşılığı olmayan biçimlerdir.

Yuvarlama ve kesme

Biçimlendirme bir sayıyı **yuvarlayarak** yazar; sayının kendisi değişmez. Tip dönüşümü ise **keser**. İkisi sık karıştırılır:

yuvarlama.cs

```
double x = 2.678;
Console.WriteLine($"{x:F2}");
Console.WriteLine((int)x);
Console.WriteLine(Math.Round(x));

Console.WriteLine($"{Math.Round(2.5)} {Math.Round(3.5)}");
Console.WriteLine($"{Math.Round(2.5, MidpointRounding.AwayFromZero)}");
Console.WriteLine($"{Math.Round(2.678, 1)}");
Console.WriteLine($"{2.675:F2} {2.675m:F2}");
```

```
ÇIKTI
2.68
2
3
2 4
3
2.7
2.67 2.68
```

`Math.Round` varsayılan olarak tam yarım değerleri **en yakın çift sayıya** yuvarlar (banker's rounding): 2,5 → 2, 3,5 → 4. Okulda öğrendiğimiz "yarımı yukarı" kuralı için `MidpointRounding.AwayFromZero` verilir. Son satır başka bir tuzağı gösterir: `2.675` değeri `double` olarak ikili tabanda tam gösterilemez ve gerçekte `2.67499999...` biçimindedir; bu yüzden `F2` onu `2.67` yapar. `decimal` değer ise ondalık tabanda tam saklanır ve `2.68` çıkar. Para ile çalışırken `decimal` kullanmamızın nedenlerinden biri budur (Bölüm 42).

> **◆ Kritik Nokta**
>
> Biçimlendirme yalnızca **gösterimi** değiştirir. `$"{x:F2}"` yazmak `x` değişkeninin değerini yuvarlamaz; sonraki hesaplar yine tam değerle yapılır. Değeri gerçekten yuvarlamak istiyorsak `Math.Round` kullanırız.

> **İpucu**
>
> `C` belirteci para simgesini bilgisayarın kültüründen alır; kullanıcının makinesine göre `₺`, `$` ya da `€` görünür. Simgenin sabit olmasını istiyorsak `{tutar:N2} TL` yazmak daha öngörülebilirdir.

43.6 Genişlik ve Hizalama

Tablo biçiminde çıktı için her değerin sabit genişlikte bir alana yazılması gerekir. Süslü parantez içinde değerden sonra virgül ve bir sayı yazarsak değer en az o kadar karakterlik alana yerleştirilir. Sayı pozitifse değer sağa, negatifse sola yaslanır. Biçim belirteci varsa genişlikten sonra gelir: `{değer,genişlik:biçim}`.

Yazım	Anlamı	Değer 42 için
`{x,6}`	6 karakterlik alan, sağa yaslı	`42`
`{x,-6}`	6 karakterlik alan, sola yaslı	`42`
`{x:D6}`	sıfırla doldur (hizalama değil, biçim)	`000042`
`{y,8:F2}`	8 karakterlik alan ve 2 ondalık	`y = 3.14159` için `3.14`

urunTablosu.cs

```
Console.WriteLine($"{"Ürün",-10}{"Adet",6}{"Fiyat",10}");
Console.WriteLine(new string('-', 26));
Console.WriteLine($"{"Kalem",-10}{3,6}{12.5,10:F2}");
Console.WriteLine($"{"Defter",-10}{12,6}{45.0,10:F2}");
Console.WriteLine($"{"Silgi",-10}{125,6}{7.25,10:F2}");
Console.WriteLine(new string('-', 26));
Console.WriteLine($"[{42,5}] [{42,-5}] [{42:D5}]");
Console.WriteLine($"[{1234567,12:N0}] [{Math.PI,8:F3}]");
```

```
ÇIKTI
Ürün        Adet     Fiyat
--------------------------
Kalem          3     12.50
Defter        12     45.00
Silgi        125      7.25
--------------------------
[   42] [42   ] [00042]
[   1,234,567] [   3.142]
```

Metinleri de aynı biçimde hizalayabiliriz; yalnızca metnin kendisi süslü parantez içinde olduğu için tırnaklarıyla birlikte yazılır (`{"Ürün",-10}`). `new string('-', 26)` ifadesi 26 tire içeren bir metin üretir; çizgi çekmek için pratik bir yoldur.

Genişlik bir **alt sınırdır**. Değer alana sığmazsa kesilmez, alan taşar ve sonraki sütunlar kayar. Sütun genişliklerini en uzun değere göre seçmeliyiz.

Hizalama için `string` tipinin `PadLeft` ve `PadRight` metotları da vardır: `"42".PadLeft(6)` sağa yaslı, `"42".PadRight(6)` sola yaslı, `"42".PadLeft(6, '0')` sıfırla dolu `000042` verir. Interpolasyondaki `,genişlik` yazımı aynı işi daha kısa yapar.

Aynı tabloyu C'nin `printf("%-10s%6d%10.2f")` ve C++'ın `setw`/`left`/`right` manipülatörleriyle de yazabilirdik. Karşılaştırmayı bölüm sonundaki **Üç Dilde** kutusunda yapacağız.

43.7 Kültür Ayarları

Şimdiye kadar yazdığımız çıktılarda ondalık ayraç hep noktaydı. Gerçek bir Türkçe bilgisayarda ise `Console.WriteLine(3.75)` ekrana `3,75` yazar. Bunun nedeni, C#'ın sayıları metne çevirirken (ve metinden okurken) işletim sisteminin bölge ayarlarını kullanmasıdır.

> **Tanım: Kültür (culture)**
>
> Bir dilin ve bölgenin biçim kurallarının bütünü: ondalık ve binlik ayraç, para birimi simgesi, yüzde yazımı, tarih ve saat düzeni, büyük-küçük harf dönüşümü gibi. .NET'te bir kültür `CultureInfo` nesnesiyle temsil edilir ve `tr-TR` (Türkçe, Türkiye) ya da `en-US` (İngilizce, ABD) gibi adlarla anılır.

Üç kültürde aynı değerin nasıl yazıldığına bakalım. Kültürü her seferinde açıkça vermek için `ToString` metodunun kültür alan sürümünü kullanıyoruz. `CultureInfo` sınıfı `System.Globalization` ad alanında olduğundan dosyanın başına `using System.Globalization;` yazarız.

kulturKarsilastirma.cs

```
using System.Globalization;

CultureInfo tr = new CultureInfo("tr-TR");
CultureInfo us = new CultureInfo("en-US");
CultureInfo degismez = CultureInfo.InvariantCulture;

double x = 1234.5;
foreach (CultureInfo k in new[] { tr, us, degismez })
{
    string ad = k.Name == "" ? "Invariant" : k.Name;
    Console.WriteLine($"{ad,-9} {x.ToString(k)} | {x.ToString("N2", k)} | {x.ToString("C", k)}")
;
}
Console.WriteLine("title".ToUpper(tr) + " " + "title".ToUpper(us));
```

```
ÇIKTI
tr-TR     1234,5 | 1.234,50 | ₺1.234,50
en-US     1234.5 | 1,234.50 | $1,234.50
Invariant 1234.5 | 1,234.50 | ¤1,234.50
TİTLE TITLE
```

Türkçe kültürde ondalık ayraç virgül, binlik ayraç noktadır; İngilizce kültürde tersi. `CultureInfo.InvariantCulture` (değişmez kültür) belirli bir ülkeye bağlı olmayan, İngilizceye yakın sabit bir kuraldır: ondalık nokta, binlik virgül. Son satır kültürün yalnızca sayıları değil, harf dönüşümünü de etkilediğini gösterir: Türkçede `i` harfinin büyüğü `İ`'dir.

Programımız hangi kültürde çalışıyorsa, ayrı bir şey söylemediğimiz sürece o kültür kullanılır; buna **geçerli kültür** (current culture) denir. İşletim sistemi Türkçe ise geçerli kültür `tr-TR` olur. Bu kitaptaki örneklerin çıktılarını aynı şekilde yeniden üretebilmek için kültür bağımsız ayarla aldığımızı Bölüm 42'de söylemiştik. Türkçe bir bilgisayarda gerçekte ne göreceğimizi, geçerli kültürü kod içinde değiştirerek taklit edebiliriz:

turkceMakine.cs

```
using System.Globalization;

CultureInfo.CurrentCulture = new CultureInfo("tr-TR");

double fiyat = 3.75;
Console.WriteLine(fiyat);
Console.WriteLine($"{1234567.891:N2}");
Console.WriteLine($"Yüzde: {0.256:P1}");
Console.WriteLine(true);
```

```
ÇIKTI
3,75
1.234.567,89
Yüzde: %25,6
True
```

Dikkat edelim: kodun içine yazdığımız sayı sabiti (`3.75`) kültürden bağımsızdır ve her zaman noktalıdır; derleyici kaynak kodu kültüre göre okumaz. Kültür yalnızca sayı **metne çevrilirken** ve **metinden okunurken** devreye girer. `bool` değerleri (`True` / `False`) ise kültürden etkilenmez.

> *Tarihçe · Başka dillerde varsayılan*
>
> C'de `printf` programın başlangıcında "C" yerel ayarını kullanır: `setlocale` çağrılmadıkça işletim sistemi Türkçe olsa bile ondalık ayraç nokta kalır (Bölüm 6). C++'ta `cout` da varsayılan olarak "klasik" yerel ayarı kullanır (Bölüm 23). C#'ın farkı, işletim sisteminin kültürünü **varsayılan** almasıdır. Bu yaklaşım kullanıcıya yakın çıktı verir; ama aynı program iki bilgisayarda farklı metin üretebilir.

Okurken kültür tuzağı

Asıl tehlike çıktıda değil, girdidedir. `double.Parse` ve `Convert.ToDouble`, metni geçerli kültürün kurallarıyla çözer. Türkçe kültürde nokta ondalık ayraç değil, binlik ayraçtır; üstelik ayraç yeri denetlenmeden sessizce yok sayılır:

kulturTuzagi.cs

```
using System.Globalization;

CultureInfo tr = new CultureInfo("tr-TR");

double a = double.Parse("3,5", tr);
double b = double.Parse("3.5", tr);
double c = double.Parse("1.234", tr);
double d = double.Parse("3.5", CultureInfo.InvariantCulture);

Console.WriteLine($"{a} {b} {c} {d}");
```

```
ÇIKTI
3.5 35 1234 3.5
```

Türkçe kültürde `"3.5"` hata vermez, `35` olur; `"1.234"` ise `1,234` değil `1234` okunur. Program çökmez, yalnızca sessizce yanlış hesap yapar. Bu yüzden kültür davranışını bilerek tasarlamak gerekir.

> **◆ Kritik Nokta**
>
> Kullanıcıdan gelen ondalık sayı için iki seçenek vardır. Kullanıcı kendi bölge alışkanlığıyla yazacaksa geçerli kültürü bırakırız (Türkçe makinede virgül). Program her makinede aynı davranmalıysa ya da veri bir dosya, ağ mesajı gibi makineler arası bir kaynaktan geliyorsa kültürü **açıkça** `CultureInfo.InvariantCulture` olarak veririz. Kültürü hiç düşünmeden `Parse` yazmak, programı yalnızca geliştiricinin bilgisayarında doğru çalışan hale getirir.

Kültürü açıkça vermek

Kültürü seçmenin birkaç yolu vardır:

Yöntem	Etki alanı
`x.ToString("F2", CultureInfo.InvariantCulture)`	yalnızca bu çağrı
`double.Parse(metin, CultureInfo.InvariantCulture)`	yalnızca bu çağrı
`string.Format(CultureInfo.InvariantCulture, "{0:N2}", x)`	yalnızca bu çağrı
`FormattableString.Invariant($"...")`	bu interpolasyon
`CultureInfo.CurrentCulture = ...`	bu iş parçacığı (thread)

Yöntem	Etki alanı
`CultureInfo.DefaultThreadCurrentCulture = ...`	sonradan oluşacak bütün iş parçacıkları

Küçük konsol programlarında `CurrentCulture` atamasını `Main`'in başında yapmak yeterlidir. Dosyaya yazarken, ağa gönderirken ya da günlük tutarken ise o noktadaki biçimlendirme çağrısına `InvariantCulture` vermek daha doğrudur; çünkü başka bir bilgisayarda okunacak dosyanın o bilgisayarın kültürüne göre değişmesini istemeyiz.

```
double fiyat = 3.75;
string satir = FormattableString.Invariant($"fiyat={fiyat}");   // fiyat=3.75
string serbest = $"fiyat={fiyat}";                              // kültüre bağlı
```

`FormattableString.Invariant` interpolasyonlu metnin hemen önüne yazılır ve içindeki bütün değerleri değişmez kültürle biçimlendirir.

43.8 Metni Sayıya Çevirmek ve Güvenli Girdi

`ReadLine()` her zaman metin getirir. Sayı gerekiyorsa metni biz çeviririz. `Parse`, `Convert` ve `TryParse` arasındaki temel farkı Bölüm 42'de tabloyla gördük; burada hatırlayıp kullanıcı girdisine uygulayacağız.

- `int.Parse(metin)` ve `Convert.ToInt32(metin)` geçersiz metinde `FormatException`, tipin sınırına sığmayan sayıda `OverflowException` fırlatır. Yakalanmazsa program çöker.
- `int.TryParse(metin, out int sonuc)` hiçbir zaman istisna fırlatmaz; çeviri başarılıysa `true`, değilse `false` döndürür. Başarısızlıkta `sonuc` 0 olur.

C ile kıyas: `scanf("%d", &x)` hem okuma hem çeviriyi yapıp başarıyı dönüş değeriyle bildirirdi (Bölüm 6); C++'ta `cin >> x` akışı kilitleyerek bildirirdi (Bölüm 23). C#'ta okuma (`ReadLine`) ve çeviri (`TryParse`) iki ayrı adımdır. Bu ayrım, hatalı girdinin arabellekte kalıp sonraki okumayı bozması sorununu ortadan kaldırır: `ReadLine()` her çağrıda satırın tamamını tüketir.

Kullanıcı geçerli bir değer girene kadar soran klasik kalıp şudur. Döngüleri Bölüm 45'te ayrıntılı işleyeceğiz; `while` C'dekiyle aynıdır.

guvenliGirdi.cs

```
int yas;
while (true)
{
    Console.Write("Yaşınız (0-120): ");
    string? metin = Console.ReadLine();
    if (metin is null)
    {
        Console.WriteLine("Girdi sona erdi.");
        return;
    }
    if (int.TryParse(metin, out yas) && yas >= 0 && yas <= 120)
    {
        break;
    }
    Console.WriteLine("Geçersiz giriş, tekrar deneyin.");
}
Console.WriteLine($"Kaydedilen yaş: {yas}");
```

```
ÇIKTI
Yaşınız (0-120): abc
Geçersiz giriş, tekrar deneyin.
Yaşınız (0-120): 150
Geçersiz giriş, tekrar deneyin.
Yaşınız (0-120): 25
Kaydedilen yaş: 25
```

Programdaki `metin is null` denetimi önemlidir. `?? ""` ile boş metne çevirseydik, girdi bittiğinde (EOF) `TryParse` hep başarısız olur ve döngü **sonsuza dek** dönerdi. `null` gelirse döngüden çıkmak gerekir. `&&` operatörü kısa devre yaptığı için `TryParse` başarısızsa aralık denetimi hiç yapılmaz; `yas`'a atama ise `TryParse` çağrısında zaten olmuştur.

> **▲ Sık Yapılan Hata · Girdi bittiğinde sonsuz döngü**
>
> Girdi dosyadan yönlendirildiğinde ya da kullanıcı `Ctrl+D` / `Ctrl+Z` bastığında `ReadLine()` sürekli `null` döndürür. "Geçerli olana kadar sor" döngüsü bu durumda çıkış yolu içermiyorsa sonsuz döner. Okuma döngülerinde `null` durumu için ayrı bir çıkış bulundurmalıyız.

Ondalık sayı okumak

Ondalık sayıda iki sorunu birlikte çözmemiz gerekir: kullanıcı `3,5` da yazabilir, `3.5` da; ve programın sonucu hangi bilgisayarda çalışırsa çalışsın aynı olmalıdır. Pratik çözüm, virgülü noktaya çevirip değişmez kültürle çözmektir:

▸ ondalikOku.cs

```
using System.Globalization;

double? OndalikCevir(string metin)
{
    metin = metin.Trim().Replace(',', '.');
    if (double.TryParse(metin, NumberStyles.Float,
                        CultureInfo.InvariantCulture, out double d))
    {
        return d;
    }
    return null;
}

string[] denemeler = { "3.5", "3,5", " 12 ", "1e3", "abc", "1.234,5" };
foreach (string s in denemeler)
{
    double? sonuc = OndalikCevir(s);
    Console.WriteLine($"[{s}] -> {(sonuc is null ? "geçersiz" : sonuc.ToString())}");
}
```

```
ÇIKTI
[3.5] -> 3.5
[3,5] -> 3.5
[ 12 ] -> 12
[1e3] -> 1000
[abc] -> geçersiz
[1.234,5] -> geçersiz
```

Bu kodda birkaç yeni öğe var. `OndalikCevir` bir **metottur**: adı olan, parametre alan ve değer döndüren kod bloğu, yani C'deki fonksiyonun karşılığı. Metotları Bölüm 46'da işleyeceğiz; şimdilik kalıbı olduğu gibi

kullanabiliriz. `double?` dönüş tipi, sonucun `null` (çeviri başarısız) olabileceğini söyler (Bölüm 42). `foreach` döngüsü diziyi eleman eleman dolaşır (Bölüm 45). `NumberStyles.Float` hangi biçimlere izin verildiğini belirler: baştaki ve sondaki boşluk, işaret, ondalık nokta ve üstel gösterim. Binlik ayracına izin vermediği için `"1.234,5"` gibi karışık yazımlar bilerek reddedilir.

> **İpucu**
>
> `decimal` için de aynı kalıp geçerlidir: `decimal.TryParse(metin, NumberStyles.AllowDecimalPoint, CultureInfo.InvariantCulture, out decimal d)`. Para tutarlarında `double` yerine `decimal` okumalıyız.

43.9 Türkçe Karakterler

C# kaynak dosyaları UTF-8 saklanır ve metin içinde `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` serbestçe kullanılabilir. Çıktı tarafında sorun, programın değil konsolun kodlamasından gelir: konsol Türkçe karakterleri göstermeyen bir kod sayfasıyla çalışıyorsa `ş`, `ğ` yerine `?` ya da anlamsız simgeler görürüz. Bu durum özellikle eski Windows komut isteminde görülür; Windows Terminal, Visual Studio konsolu ve Linux/macOS terminalleri genellikle UTF-8 kullanır.

Çözüm, programın başında çıktı (ve gerekirse girdi) kodlamasını UTF-8 yapmaktır:

`turkceKarakter.cs`

```
using System.Text;

Console.OutputEncoding = Encoding.UTF8;
Console.InputEncoding = Encoding.UTF8;

Console.WriteLine("ç ğ ı i ö ş ü");
Console.WriteLine("Ç Ğ I İ Ö Ş Ü");
Console.WriteLine("Şişli'de çığ düştü, İğdır'a gidelim.");
```

```
ÇIKTI
ç ğ ı i ö ş ü
Ç Ğ I İ Ö Ş Ü
Şişli'de çığ düştü, İğdır'a gidelim.
```

Modern ortamlarda bu satırlar gerekmez ve kitaptaki diğer örneklerde yazmıyoruz. Konsolunuzda Türkçe karakterler bozuk görünürse ilk çare terminal uygulamasını değiştirmektir; olmazsa bu iki satırı programın başına ekleriz. Karakter kodlamalarının ayrıntısı için Bölüm 48'e ve Ek A'ya bakılabilir.

43.10 Örnek Programlar

Öğrendiklerimizi iki programda birleştirelim. İkisinde de ondalık girdi, kültürden bağımsız olarak hem virgül hem nokta ile verilebilir; çıktı ise geçerli kültüre göre biçimlenir (kitapta kültür bağımsız, yani noktalı ve virgülle binlik ayraçlı gösteriyoruz).

Fatura hesabı

Program kalem sayısını sorar, her kalem için ad, adet ve birim fiyat okur, ardından hizalı bir fatura tablosu ve KDV'li toplamı yazar. Para tutarları `decimal` tipindedir. Geçersiz girdi, yardımcı metotlar içindeki döngülerde ayıklanır.

faturaHesap.cs

```
using System.Globalization;

const decimal KdvOrani = 0.20m;

decimal OkuOndalik(string soru)
{
    while (true)
    {
        Console.Write(soru);
        string? satir = Console.ReadLine();
        if (satir is null)
        {
            return 0m;   // girdi bitti
        }
        satir = satir.Trim().Replace(',', '.');
        if (decimal.TryParse(satir, NumberStyles.AllowDecimalPoint,
                             CultureInfo.InvariantCulture, out decimal d) && d >= 0)
        {
            return d;
        }
        Console.WriteLine("Geçerli bir sayı girin.");
    }
}

Console.Write("Kalem sayısı: ");
if (!int.TryParse(Console.ReadLine(), out int n) || n < 1)
{
    Console.WriteLine("Geçersiz kalem sayısı.");
    return;
}

string[] adlar = new string[n];
decimal[] adetler = new decimal[n];
decimal[] fiyatlar = new decimal[n];
for (int i = 0; i < n; i++)
{
    Console.Write($"{i + 1}. Ürün adı: ");
    adlar[i] = Console.ReadLine() ?? "";
    adetler[i] = OkuOndalik("   Adet: ");
    fiyatlar[i] = OkuOndalik("   Birim fiyat: ");
}

Console.WriteLine();
Console.WriteLine($"{"Ürün",-12}{"Adet",5}{"Birim",10}{"Tutar",12}");
Console.WriteLine(new string('=', 39));
decimal araToplam = 0m;
for (int i = 0; i < n; i++)
{
    decimal tutar = adetler[i] * fiyatlar[i];
    araToplam += tutar;
    Console.WriteLine($"{adlar[i],-12}{adetler[i],5:0.##}{fiyatlar[i],10:N2}{tutar,12:N2}");
}
decimal kdv = araToplam * KdvOrani;
string kdvEtiket = $"KDV (%{KdvOrani * 100:0})";
Console.WriteLine(new string('-', 39));
Console.WriteLine($"{"Ara toplam",-27}{araToplam,12:N2}");
Console.WriteLine($"{kdvEtiket,-27}{kdv,12:N2}");
Console.WriteLine($"{"GENEL TOPLAM",-27}{araToplam + kdv,12:N2}");
```

```
ÇIKTI
Kalem sayısı: 3
1. ürün adı: Kalem
   Adet: 10
   Birim fiyat: 2,50
2. ürün adı: Defter
   Adet: 4
   Birim fiyat: abc
Geçerli bir sayı girin.
   Birim fiyat: 18.75
3. ürün adı: Çanta
   Adet: 2
   Birim fiyat: 1250,5

Ürün         Adet     Birim       Tutar
=====================================
Kalem          10      2.50       25.00
Defter          4     18.75       75.00
Çanta           2  1,250.50    2,501.00
-------------------------------------
Ara toplam                    2,601.00
KDV (%20)                       520.20
GENEL TOPLAM                  3,121.20
```

Programdaki `{adetler[i],5:0.##}` özel biçimi, adet tam sayıysa ondalık basamak yazmaz (`10`), kesirliyse en fazla iki basamak yazar (`2.5`). Tutarlar `decimal` olduğu için çarpma ve toplama sırasında ikili tabandaki yuvarlama sapmaları oluşmaz. Birim fiyatta ilk `abc` girişi reddedilir, ardından `18.75` ile `1250,5` aynı kodla okunur.

Not ortalaması

İkinci programda üç notun ağırlıklı ortalaması hesaplanır. Notlar 0 ile 100 arasında olmalıdır; aralık dışı girdi yeniden sorulur.

notOrtalamasi.cs

```
int OkuNot(string ad)
{
    while (true)
    {
        Console.Write($"{ad} notu (0-100): ");
        if (int.TryParse(Console.ReadLine(), out int deger) && deger >= 0 && deger <= 100)
        {
            return deger;
        }
        Console.WriteLine("0 ile 100 arasında bir tam sayı girin.");
    }
}

Console.Write("Ad soyad: ");
string ogrenci = Console.ReadLine() ?? "";

int proje = OkuNot("Proje");
int odev = OkuNot("Ödev");
int uygulama = OkuNot("Uygulama");

double ort = proje * 0.5 + odev * 0.2 + uygulama * 0.3;

string harf;
if (ort >= 85)
{
```

```
    harf = "A";
}
else if (ort >= 70)
{
    harf = "B";
}
else if (ort >= 55)
{
    harf = "C";
}
else
{
    harf = "D";
}

Console.WriteLine();
Console.WriteLine($"{ogrenci}");
Console.WriteLine($"  Proje    (%50): {proje,3}");
Console.WriteLine($"  Ödev     (%20): {odev,3}");
Console.WriteLine($"  Uygulama (%30): {uygulama,3}");
Console.WriteLine($"  Ortalama      : {ort,6:F2}  Harf: {harf}");
```

```
ÇIKTI
Ad soyad: Ayşe Nur Kaya
Proje notu (0-100): 90
Ödev notu (0-100): 250
0 ile 100 arasında bir tam sayı girin.
Ödev notu (0-100): 70
Uygulama notu (0-100): 80

Ayşe Nur Kaya
  Proje    (%50):  90
  Ödev     (%20):  70
  Uygulama (%30):  80
  Ortalama      :  83.00  Harf: B
```

Koşulların ve `if` / `else if` zincirinin ayrıntısı Bölüm 44'tedir; burada C'den tanıdık yapıyı kullandık.

Üç Dilde · Aynı tablo satırı üç dilde *C · C++ · C#*

Bu bölümde gördüğümüz hizalı çıktıyı üç dilde yan yana yazalım. Üçü de aynı satırı üretir: `Kalem` sola yaslı 10 karakterde, `3` sağa yaslı 6 karakterde, `12.5` sağa yaslı 10 karakterde ve 2 ondalıkla.

```
printf("%-10s%6d%10.2f\n", "Kalem", 3, 12.5);
```

```
cout << left << setw(10) << "Kalem" << right << setw(6) << 3
     << setw(10) << fixed << setprecision(2) << 12.5 << '\n';
```

```
Console.WriteLine($"{"Kalem",-10}{3,6}{12.5,10:F2}");
```

```
ÇIKTI
Kalem          3     12.50
```

	C `printf`	C++ `cout`	C#
Biçim nerede?	biçim metninde (`%`)	akıma verilen manipülatörlerde	metnin içinde, değerin yanında

	C `printf`	C++ `cout`	C#
Tip uyuşmazlığı	tanımsız davranış (derleyici uyarabilir)	yok (tip otomatik)	yok (tip otomatik)
Biçim kalıcı mı?	hayır, her `%` ayrı	çoğu manipülatör akışta **kalır** (`fixed` , `left`); `setw` yalnızca bir sonraki değeri etkiler	hayır, her `{...}` ayrı
Yeni satır	`\n` elle	`'\n'` elle	`WriteLine` otomatik
Varsayılan kültür	"C" (nokta)	klasik yerel ayar (nokta)	işletim sistemi kültürü

C++'ın akışa yapışan biçimleri (`fixed` , `left`) sonraki yazdırmaları da etkiler (Bölüm 23); C#'ta her yer tutucunun biçimi kendisine aittir, başka satırlara sızmaz.

Üç Dilde · Girdi okuma ve hatalı girdi *C · C++ · C#*

Bir tam sayıyı klavyeden okuyup başarısızlığı kontrol etmenin üç dildeki karşılığı:

```c
int x;
if (scanf("%d", &x) != 1) { /* hatalı girdi */ }
```

```cpp
int x;
if (!(cin >> x)) { cin.clear(); cin.ignore(10000, '\n'); /* hatalı girdi */ }
```

```csharp
if (!int.TryParse(Console.ReadLine(), out int x)) { /* hatalı girdi */ }
```

	C	C++	C#
Bir satır (boşluklu)	`fgets(tampon, boyut, stdin)`	`getline(cin, s)`	`Console.ReadLine()`
Sayı okuma	`scanf("%d", &x)`	`cin >> x`	`int.Parse` / `int.TryParse`
Başarısızlık bildirimi	dönüş değeri	akış durumu (`fail()`)	`false` ya da istisna
Satır sonu arabellekte kalır mı?	evet	evet (`>>` sonrası)	hayır
EOF	`EOF` / `NULL`	`cin.eof()`	`null`

C#'ta okuma ve çevirme ayrı adımlar olduğundan akışın "bozulması" diye bir durum yoktur; hatalı bir satır okunur, çevrilemez ve atılır.

Bölüm Özeti

- `Console.Write` satır sonu eklemeden, `Console.WriteLine` satır sonuyla yazar. İkisi de her tipten değeri kabul eder; `bool` değerler `True` / `False` biçiminde görünür. Hata iletileri için `Console.Error` kullanılır.
- `+` operatörü operandlardan biri metinse birleştirir ve soldan sağa işler: `"T: " + 2 + 3` sonucu `T: 23` olur. Toplama için parantez gerekir.

- `@"..."` düz metin, `"""..."""` ham metindir (C# 11). Geçersiz kaçış dizisi (`"C:\temp\yeni"`) derleme hatasıdır.
- `Console.ReadLine()` satırın tamamını `string?` olarak döndürür; girdi bitince (EOF) `null` verir. Uyarısız kalıp `Console.ReadLine() ?? ""` biçimindedir. Boş satır `null` değil, `""` döndürür.
- `Console.ReadKey()` Enter beklemeden tek tuş okur; girdi yönlendirilmişse istisna fırlatır.
- Birleşik biçimlendirme (`"{0} {1}"`, `string.Format`) değerleri sırayla yerleştirir; argüman eksikse `FormatException` oluşur. Dize içine değer yerleştirme (`$"{x}"`) değerleri metnin içine yazar, her ifade kullanılabilir; süslü parantez için `{{` ve `}}` yazılır, `?:` parantez içine alınır.
- Standart biçim belirteçleri: `F2` (sabit nokta), `N2` (binlik ayraçlı), `C` (para), `P` (yüzde), `D5` (başa sıfır), `X` (onaltılık), `E2` (bilimsel), `G` (genel). `0`, `#` ile özel kalıp yazılabilir.
- Biçimlendirme yalnızca gösterimi yuvarlar; `(int)` keser; `Math.Round` varsayılan olarak yarımları çifte yuvarlar. Para için `decimal` kullanılır.
- `{x,10}` sağa, `{x,-10}` sola yaslı 10 karakterlik alan açar; `{x,10:F2}` hizalama ile biçimi birleştirir. Genişlik alt sınırdır, taşan değer kesilmez.
- Kültür; ondalık ve binlik ayraç, para simgesi, yüzde yazımı ve harf dönüşümünü belirler. C#'ta varsayılan, işletim sisteminin kültürüdür; Türkçede ondalık ayraç virgüldür. C ve C++ varsayılan olarak "C" yerel ayarını kullanır.
- Kod içindeki sayı sabitleri kültürden bağımsızdır (`3.5`). Türkçe kültürde `double.Parse("3.5")` hata vermeden `35` üretir. Makineler arası veri için `CultureInfo.InvariantCulture` açıkça verilir.
- `Parse` ve `Convert` geçersiz metinde istisna fırlatır; `TryParse` fırlatmaz, `bool` döndürür. Kullanıcı girdisinde `TryParse` ve doğrulama döngüsü kullanılır; döngüde `null` (EOF) için çıkış yolu bulunmalıdır.
- Ondalık girdide virgülü noktaya çevirip `InvariantCulture` ile çözmek, her iki yazımı da kabul eden pratik bir kalıptır.
- Türkçe karakterler bozuk görünürse `Console.OutputEncoding = Encoding.UTF8;` eklenir.

Alıştırmalar

1. `Console.ReadLine()` metodunun dönüş tipi nedir?
   A) `int`
   B) `string`
   C) `string?`
   D) `char`
   E) `object`
2. `Console.WriteLine("Toplam: " + 2 + 3);` komutu ekrana ne yazdırır?
   A) `Toplam: 5`
   B) `Toplam: 23`
   C) `Toplam: 2 3`
   D) Derleme hatası verir.
   E) `5`
3. Aşağıdakilerden hangisi `7` değerini `007` olarak yazdırır?
   A) `$"{7,3}"`
   B) `$"{7:D3}"`
   C) `$"{7:F3}"`

D) `$"{7,-3}"`
E) `$"{7:N3}"`

4. Türkçe (`tr-TR`) kültürlü bir bilgisayarda `Convert.ToDouble("1.250")` ifadesinin sonucu nedir?
   A) `1.25`
   B) `1250`
   C) `FormatException` fırlatır.
   D) `1,25`
   E) `0`
5. `string ad = Console.ReadLine() ?? "";` satırı için aşağıdakilerden hangisi doğrudur?
   A) Kullanıcı yalnızca Enter'a basarsa `ad` değişkeni `null` olur.
   B) `??` operatörü sağ tarafı, yalnızca soldaki değer `null` ise kullanır.
   C) Satır `ReadLine()` her çağrıldığında `""` değerini döndürür.
   D) Satır, `ReadLine()` çağrısını iki kez yapar.
   E) Girdi bittiğinde (EOF) program `FormatException` ile sona erer.
6. Aşağıdaki program ekrana ne yazdırır? (Kültür bağımsız ayarla çalıştığını varsayın.)

```
int a = 7;
double b = 2.0 / 3;
Console.WriteLine($"[{a,4}][{a,-4}][{a:D4}]");
Console.WriteLine($"{b:F3} {b:0.0}");
Console.WriteLine("{0}-{1}-{0}", a, "x");
Console.WriteLine($"{{{a}}}");
Console.WriteLine($"{1234.5:N1} {255:X2}");
```

7. Aşağıdaki program ekrana ne yazdırır?

```
using System.Globalization;

CultureInfo tr = new CultureInfo("tr-TR");
double x = double.Parse("2.5", tr);
Console.WriteLine(x.ToString(tr));
Console.WriteLine((x / 4).ToString("F2", tr));
Console.WriteLine((x / 4).ToString("F2", CultureInfo.InvariantCulture));
```

8. Aşağıdaki kodda iki sorun vardır. Hangileridir ve nasıl düzeltilir?

```
Console.Write("Yaş: ");
int yas = Console.ReadLine();
Console.WriteLine("Gelecek yıl: " + yas + 1);
```

9. `Console.WriteLine("{0} {1} {2}", a, b);` satırı derlenir mi? Çalıştırıldığında ne olur?
10. Aşağıdaki satır ekrana ne yazar? Amaç değişkenin değerini yazdırmaksa nasıl düzeltilir?

```
string ad = "Elif";
Console.WriteLine("Merhaba {ad}!");
```

11. Aşağıdaki döngü, girdi bir dosyadan yönlendirilmiş ve dosya sayılardan önce bitmişse ne yapar? Neden? Nasıl düzeltilir?

```
int sayi;
while (true)
{
```

```
    string metin = Console.ReadLine() ?? "";
    if (int.TryParse(metin, out sayi))
    {
        break;
    }
}
```

12. Kullanıcıdan iki ondalık sayı okuyan, sayıları hem `3,5` hem `3.5` biçiminde kabul eden ve toplamlarıyla ortalamalarını iki ondalık basamakla yazdıran bir program yazın. Geçersiz girdide sayıyı yeniden sorun.
13. 1'den 5'e kadar sayıların, kareleri ve küpleriyle birlikte, başlık satırı olan ve üç sütunu sağa yaslı (sütun genişliği 3, 6 ve 6) bir tablo olarak yazdırılmasını sağlayan bir program yazın.
14. Kullanıcıdan 1 ile 10 arasında bir tam sayı isteyen, geçerli bir sayı girilene kadar soruyu tekrarlayan ve girdi biterse (EOF) programı sonlandıran bir program yazın.
15. Kullanıcıdan ad soyad ve brüt maaşı (ondalık, virgüllü ya da noktalı) okuyan bir program yazın. Brüt maaşın yüzde 15'i kesinti olsun. Brüt, kesinti ve net tutarı `decimal` ile hesaplayıp, etiketleri sola, tutarları sağa yaslı ve iki ondalıklı yazdırın.
16. Aşağıdaki C satırının eşdeğerini C#'ta dize içine değer yerleştirme ile yazın:

```
printf("%-8s|%5d|%8.3f|\n", ad, n, x);
```

BÖLÜM 44

Koşullu İfadeler ve Desen Eşleme

BU BÖLÜMDE

- `if`, `else if` ve `else` yapılarını C# kurallarıyla (koşulun `bool` olması, kapsam, `out` değişkenleri) kullanmayı
- `switch` deyiminin C'den farklarını: düşmenin yasak olması, `goto case`, `string` ve `enum` ile kullanım
- `switch` ifadesini (C# 8) ve `_` ile her durumu kapsama kuralını
- Tür, sabit, ilişkisel (`>= 90`), mantıksal (`and`, `or`, `not`), özellik ve çoklu (tuple) desenlerini
- `is` ile desen sorgulamayı ve `when` ile ek koşul yazmayı
- Üçlü operatörü, `??` ve `?.` operatörlerini
- Not harfi, menü ve kargo ücreti gibi gerçekçi karar programlarını

Programlar, verilere bakıp farklı yollardan birini seçebildiği ölçüde yararlıdır. C'de bu işi `if` ve `switch` görür (Bölüm 7); C++ bunlara başlatıcı ve `[[fallthrough]]` gibi küçük eklemeler yaptı (Bölüm 24). C# aynı iki yapıyı devralır ve üzerine büyük bir katman ekler: **desen eşleme** (pattern matching). Bir değerin tipine, aralığına, içeriğine bakıp doğrudan bir sonuca varmak, C# kodunun en belirgin özelliklerinden biri hâline gelmiştir.

Bu bölümde önce tanıdık yapıları, C ve C++'tan farklarını vurgulayarak hızla gözden geçireceğiz. `if (x = 5)` hatasını, `&&` ile `||` operatörlerinin kısa devre davranışını ve operatör önceliğini Bölüm 42'de işledik; burada yinelemeyeceğiz. Sonra `switch` deyimini, ardından C# 8 ile gelen `switch` ifadesini ve desenleri ele alacağız. Bölümün sonunda üç küçük program yazacağız.

Programlar, Bölüm 41'den beri kullandığımız top-level statements biçimindedir. Klavyeden girdi okuyan programlarda `Console.ReadLine` kullanıyoruz; ayrıntısı Bölüm 43'tedir.

44.1 if, else if ve else

`if` söz dizimi C ile aynıdır: koşul parantez içine yazılır, doğruysa gövde çalışır, `else` yanlış durumu karşılar, `else if` zincirleri sırayla denenir. C ve C++ okuru için iki fark önemlidir; geri kalanı olduğu gibi geçerlidir.

Birinci fark: koşul `bool` olmak zorundadır. Sayı, işaretçi ya da nesne koşul olarak yazılamaz. Bu kuralın `if (x = 5)` ve `if (x)` hatalarını nasıl önlediğini Bölüm 42'de gördük. Dolayısıyla C'deki `if (kalan)` yerine `if (kalan != 0)` yazarız.

`bool?` tipi de bu kurala tabidir. Üç durumlu (`true`, `false`, `null`) bir değer doğrudan koşul olamaz:

nullableKosul.cs

```
bool? onay = true;
if (onay)
{
    Console.WriteLine("onaylandı");
}
```

```
DERLEYİCİ ÇIKTISI

nullableKosul.cs(2,5): error CS0266: Cannot implicitly convert type 'bool?' to 'bool'. An
explicit conversion exists (are you missing a cast?)
```

Çözüm, `null` durumunu bilinçli ele almaktır: `if (onay == true)` ya da `if (onay ?? false)`. Sıradan `bool` için `== true` yazmak gereksizdir; `bool?` için ise tam olarak bu sorunun cevabıdır.

İkinci fark: değişkenler `if` içinde de tanımlanabilir. C++17'nin `if (int x = f(); x > 0)` başlatıcısına benzer bir iş, C#'ta `out` parametreleri ve `is` deseni üzerinden görülür. `int.TryParse(girdi, out int derece)` yazdığımızda `derece` değişkeni `if` koşulunda tanımlanır; ama C++'taki başlatıcının tersine, kapsamı `if` ile bitmez, `if`'i çevreleyen bloğun sonuna kadar sürer. Bu, "önce geçersiz durumu ele al ve çık" kalıbını mümkün kılar:

tryParseKosul.cs

```
Console.Write("Sıcaklık: ");
string? girdi = Console.ReadLine();

if (!int.TryParse(girdi, out int derece))
{
    Console.WriteLine("Geçerli bir tam sayı girilmedi.");
    return;                      // top-level kodda programı bitirir
}

// derece burada kullanılabilir ve kesin olarak atanmıştır
if (derece > 30)
{
    Console.WriteLine($"{derece} derece: hava sıcak.");
}
else if (derece >= 15)
{
    Console.WriteLine($"{derece} derece: ılıman.");
}
else
{
    Console.WriteLine($"{derece} derece: soğuk.");
}
```

```
ÇIKTI
Sıcaklık: 34
34 derece: hava sıcak.
```

`return` ile koşulun olumsuz hâlini baştan elemeye **koruma koşulu** (guard clause) denir. Yararı, asıl işi yapan kodun iç içe `if` bloklarına gömülmemesidir. `TryParse`'ı Bölüm 42'de görmüştük; `string?` yazımının ne anlama geldiğini Bölüm 59'da açıklayacağız.

Ortak noktalar. Sarkan `else` ve boş deyim C'dekiyle aynıdır. `else` en yakın `if`'e bağlanır, girinti hiçbir anlam taşımaz, `if (...);` boş bir deyim üretir (`CS0642` uyarısı). Bölüm 7'deki ilkeler aynen geçerlidir: tek satırlık gövdeler için bile süslü parantez yazmak güvenli yoldur.

> **◆ Kritik Nokta**
>
> C#'ta koşul her zaman `bool` türünde olmalıdır. Sıfır "yanlış", sıfırdan farklı "doğru" kabulü yoktur. Bu kural, C ve C++'ta öğrendiğimiz `if (p)`, `while (n)` gibi kısaltmaları C#'ta yazamayacağımız anlamına gelir; karşılığında atama yüzünden oluşan bir hata sınıfı baştan ortadan kalkar.

44.2 switch Deyimi

Bir değişkenin birkaç belirli değerine göre farklı işler yapılacaksa `else if` zinciri yerine `switch` daha okunaklıdır. Yapısı C'dekine benzer, kuralları ise daha katıdır. Önce basit bir örnek yazalım:

gunAdi.cs

```
int gun = 3;

switch (gun)
{
    case 1:
        Console.WriteLine("Pazartesi");
        break;
    case 2:
        Console.WriteLine("Salı");
        break;
    case 3:
        Console.WriteLine("Çarşamba");
        break;
    default:
        Console.WriteLine("Geçersiz gün");
        break;
}
```

```
ÇIKTI
Çarşamba
```

Parantez içindeki değer hesaplanır, değeri eşleşen `case` etiketinden devam edilir, `break` görülünce `switch` biter. Hiçbir etiket eşleşmezse `default` çalışır; `default` zorunlu değildir.

Düşme yasaktır

C'de `break` unutulursa program bir sonraki `case`'e düşer (fall-through). Bu davranışı Bölüm 7'de bir hata kaynağı olarak, Bölüm 24'te de `[[fallthrough]]` ile "bilerek düşme"nin işaretlenmesi olarak görmüştük. C# bu sorunu kökten çözer: **içinde en az bir deyim bulunan her `case` bölümü, kontrolü başka bir bölüme bırakmadan bitmek zorundadır.** Aksi hâlde program derlenmez:

dusmeHata.cs

```
int gun = 1;
switch (gun)
{
    case 1:
        Console.WriteLine("Pazartesi");
    case 2:
        Console.WriteLine("Salı");
        break;
}
```

```
DERLEYİCİ ÇIKTISI

dusmeHata.cs(4,5): error CS0163: Control cannot fall through from one case label ('case 1:') to
another
```

Bir bölümü bitirmenin yolları `break`, `return`, `throw`, `continue` (bir döngünün içindeysek) ve `goto`'dur. Bunların hiçbiri yazılmamışsa derleme hatası alırız. Kural `default` bölümü için de geçerlidir.

Yan yana yazılan **boş** etiketler ise düşme sayılmaz; birden çok değeri aynı koda bağlamanın yoludur (C ve C++'ta olduğu gibi):

haftaSonu.cs

```
string gun = "Pazar";

switch (gun)
{
    case "Cumartesi":
    case "Pazar":
        Console.WriteLine("Hafta sonu");
        break;
    case "Cuma":
        Console.WriteLine("Bir gün kaldı");
        break;
    default:
        Console.WriteLine("Hafta içi");
        break;
}
```

```
ÇIKTI
Hafta sonu
```

Bu örnekte iki önemli fark daha görülüyor. C'de `switch` yalnızca tam sayı ve karakter türlerinde çalışırdı; C#'ta `string` ile de kullanılır. Karşılaştırma `==` gibi içeriğe bakar ve büyük/küçük harfe duyarlıdır: `"pazar"` yazsaydık `default` çalışırdı.

goto case ve goto default

Bazen bilerek başka bir bölümün koduna geçmek isteriz. C#'ta bunu açıkça `goto case` ile belirtiriz. Böylece düşme, istenmeden değil bilinçli olarak yapılır; niyet satırda görünür:

gotoCase.cs

```
int paket = 3;
int ucret = 0;

switch (paket)
{
    case 3:
        ucret += 50;        // premium paketin ek ücreti
        goto case 2;        // orta paketin ücretlerini de ekle
    case 2:
        ucret += 30;
        goto case 1;
    case 1:
        ucret += 20;        // temel ücret
        break;
    default:
        Console.WriteLine("Bilinmeyen paket");
```

```
        break;
}

Console.WriteLine($"Toplam ücret: {ucret}");
```

```
ÇIKTI
Toplam ücret: 100
```

`goto default;` da aynı biçimde `default` bölümüne atlar. C#'ta `goto`'nun bu iki kullanımı dışında etiketli kullanımı vardır; ama günlük kodda neredeyse hiç gerekmez. `goto case` de seyrek görülür; sık ihtiyaç duyuyorsak bölümleri metotlara ayırmak daha temiz bir çözümdür.

C++'taki `[[fallthrough]];` ile `goto case`'in amacı aynıdır: düşmenin bilerek yapıldığını okuyucuya ve derleyiciye bildirmek. Fark, C#'ta bildirimin zorunlu olmasıdır.

case etiketleri ve bölüm kapsamı

Klasik `case` etiketlerinde yazılan değerler **derleme zamanında bilinen sabitler** olmalıdır: sayı ve karakter sabitleri, `string` sabitleri, `const` değişkenler, `enum` üyeleri. Sıradan bir değişken etiket olamaz ve aynı değer iki kez yazılamaz (`CS0152`).

`switch` bloğunun tamamı **tek bir kapsamdır**. Bir `case` bölümünde tanımlanan değişken sonraki bölümlerde de görünür (ama değer atanmadıysa okunamaz). Bunun, C++'taki "`case` etiketini atlayarak tanımlama" sorununa (Bölüm 24) karşılık gelen güvenli bir tarafı vardır: C# kesin atama denetimi yaptığı için atlanmış bir tanımın değeri okunamaz. Yine de bölümlerde ayrı değişkenler gerekiyorsa her bölümü süslü parantezle sarmak, adların çakışmasını önler:

bolumKapsami.cs

```
int islem = 2;

switch (islem)
{
    case 1:
    {
        int sonuc = 10 + 5;
        Console.WriteLine($"Toplama: {sonuc}");
        break;
    }
    case 2:
    {
        int sonuc = 10 * 5;      // yukarıdaki sonuc ile çakışmaz
        Console.WriteLine($"Çarpma: {sonuc}");
        break;
    }
}
```

```
ÇIKTI
Çarpma: 50
```

enum ile switch

`switch`, sabit kümeli değerlerle çok iyi çalışır. Haftanın günleri, durum kodları, menü seçenekleri gibi sınırlı bir kümeyi `enum` ile adlandırırız. `enum` tipini Bölüm 52'de ayrıntılı işleyeceğiz; burada, bir tamsayı grubuna ad vermenin yolu olduğunu bilmemiz yeterlidir. Tip tanımları top-level kodun altına yazılır:

enumSwitch.cs

```
Mevsim simdi = Mevsim.Yaz;

switch (simdi)
{
    case Mevsim.Ilkbahar:
    case Mevsim.Sonbahar:
        Console.WriteLine("Mevsim geçişi, ince giyin.");
        break;
    case Mevsim.Yaz:
        Console.WriteLine("Sıcak, güneş kremi sürün.");
        break;
    case Mevsim.Kis:
        Console.WriteLine("Soğuk, kalın giyin.");
        break;
}

enum Mevsim { Ilkbahar, Yaz, Sonbahar, Kis }
```

ÇIKTI

```
Sıcak, güneş kremi sürün.
```

Etiketlerin başına `Mevsim.` yazmak zorunludur; C'deki gibi çıplak `Yaz` yazılamaz. Böylece iki farklı `enum`'un aynı adlı üyeleri birbirine karışmaz.

break ve döngü

`switch` bir döngünün içindeyse `break` yalnızca `switch`'ten çıkar, döngüden değil; bu C'deki gibidir. Döngüyü bitirmek için bir bayrak değişkeni (bu bölümün menü programında göreceğiz) ya da `return` kullanılır. `continue` ise döngünün sonraki turuna geçer ve `case` bölümünü bitiren geçerli bir yoldur.

Üç Dilde · switch'te düşme ve tür kısıtı *C · C++ · C#*

C'de `switch` yalnızca tamsayı ve karakter türlerinde çalışır ve `break` yazılmazsa sonraki `case`'e düşer. İleride C++ ve C#'ta bunun nasıl değiştiğini göreceğiz.

```
switch (n) {
    case 1: puts("bir");          /* düşer! */
    case 2: puts("iki"); break;
}
```

C++ aynı kuralları korur; yalnızca bilinçli düşmeyi `[[fallthrough]];` ile işaretlemeye izin verir (Bölüm 24):

```
switch (n) {
    case 1: std::puts("bir"); [[fallthrough]];
    case 2: std::puts("iki"); break;
}
```

C# düşmeyi yasaklar; bilinçli geçiş `goto case` ile yazılır ve `string` de kullanılabilir:

```
switch (ad)
{
    case "bir": Console.WriteLine("bir"); goto case "iki";
    case "iki": Console.WriteLine("iki"); break;
}
```

44.3 switch İfadesi

`switch` deyimi bir **iş** yapar: bölümlerden birini çalıştırır. Çoğu zaman ise yaptığımız şey, bir değere göre başka bir **değer** seçmektir: puandan harf notu, kodden ad, gün numarasından gün adı. C# 8 bu iş için `switch` ifadesini (switch expression) getirdi. İfade bir değer üretir, bu yüzden atamanın sağına ya da bir metodun argümanı olarak yazılabilir.

Biçimi şöyledir: değerden sonra `switch` sözcüğü gelir, süslü parantez içinde her kol `desen => sonuç` biçiminde yazılır, kollar virgülle ayrılır. `case`, `break` ve `default` yoktur. Hiçbir kolun eşleşmediği durumu `_` (atma deseni, discard) yakalar.

▸ gunAdiIfade.cs

```
int gun = 3;

string ad = gun switch
{
    1 => "Pazartesi",
    2 => "Salı",
    3 => "Çarşamba",
    4 => "Perşembe",
    5 => "Cuma",
    6 => "Cumartesi",
    7 => "Pazar",
    _ => "Geçersiz gün"
};

Console.WriteLine(ad);
```

ÇIKTI

```
Çarşamba
```

Aynı iş `switch` deyimiyle on dört satır ve yedi `break` gerektirirdi. İfadenin kuralları şunlardır:

- Kollar yukarıdan aşağı denenir ve **ilk eşleşen** kol sonucu verir; düşme kavramı yoktur.
- Son kolun sonunda virgül bırakmak serbesttir, ama zorunlu değildir.
- İfade bir atamanın parçası olduğu için kapanan `}` işaretinden sonra `;` gelir.
- Bütün kolların sonuçları ortak bir tipe dönüşebilmelidir.
- Bir kolda birden fazla deyim çalıştırmak gerekiyorsa `switch` ifadesi uygun araç değildir; o durumda `switch` deyimi ya da `if` kullanılır.

Her durumu kapsama zorunluluğu

`switch` ifadesi, giriş değerinin olası **her** durumu için bir sonuç vermek zorundadır ve derleyici bunu denetler. `_` kolunu unutursak derleyici hata değil uyarı verir (`CS8509`). Bu uyarı ciddiye alınmalıdır; çünkü çalışma sırasında hiçbir kol eşleşmezse program `SwitchExpressionException` ile durur:

▸ kapsamaUyari.cs

```
using System.Runtime.CompilerServices;

int kod = 9;

try
{
    string ad = kod switch
```

```
    {
        1 => "bir",
        2 => "iki"
    };
    Console.WriteLine(ad);
}
catch (SwitchExpressionException)
{
    Console.WriteLine("Hiçbir kol eşleşmedi.");
}
```

```
ÇIKTI
Hiçbir kol eşleşmedi.
```

Derleyici bu programı `CS8509` uyarısıyla derler: "The switch expression does not handle all possible values of its input type (it is not exhaustive)". `SwitchExpressionException` tipi `System.Runtime.CompilerServices` ad alanındadır; bu yüzden en üste `using` satırı ekledik. `try`/`catch` yapısını Bölüm 53'te işleyeceğiz; burada yalnızca istisnanın yakalandığını gösteriyoruz. `enum` ile kullanımda derleyici bir adım ileri gider: `enum`'un tüm üyeleri kollarda varsa `_` gerekmez, ama bir üye eksikse uyarı vererek hangisinin eksik olduğunu söyler. Bu, `enum`'a yeni bir üye eklendiğinde `switch` ifadelerini güncellemeyi unutmamızı sağlar.

44.4 Desen Eşleme

Desen (pattern), bir değerin belirli bir **biçime** uyup uymadığını sorgulayan küçük bir ifadedir. Şimdiye kadar yalnızca en basit desen olan sabit desenini kullandık (`1`, `"Pazar"`). C# yedi yıl içinde birçok desen ekledi; hepsi `switch` ifadelerinde, `switch` deyimlerinde (`case` etiketinden sonra) ve `is` operatöründe aynı biçimde yazılır.

> **Tanım: Desen eşleme**
>
> Bir değeri bir desene karşı sınayan ve istenirse eşleşen kısmı yeni bir değişkene ayıran dil özelliğine desen eşleme (pattern matching) denir. Desen; bir sabit, bir tür, bir aralık, bir nesnenin özelliklerine ilişkin koşullar ya da bunların `and`, `or`, `not` ile birleşimi olabilir.

İlişkisel ve mantıksal desenler

Aralık denetimi `else if` zincirlerinin en sık işidir. İlişkisel desenler (`<`, `<=`, `>`, `>=`) bu işi doğrudan kolda yazmamızı sağlar. `and`, `or` ve `not` sözcükleri desenleri birleştirir (C# 9). Puandan harf notu hesaplayalım:

```
▸ harfNotu.cs
```

```
Console.Write("Puan (0-100): ");
int puan = int.Parse(Console.ReadLine()!);

string harf = puan switch
{
    < 0 or > 100 => "Geçersiz",
    >= 90 => "AA",
    >= 80 => "BB",
    >= 70 => "CC",
    >= 60 => "DD",
    _ => "FF"
};

string durum = puan switch
{
```

```
    >= 0 and < 50 => "Kaldı",
    >= 50 and <= 100 => "Geçti",
    _ => "Geçersiz"
};

Console.WriteLine($"Harf: {harf}, durum: {durum}");
```

```
ÇIKTI
Puan (0-100): 85
Harf: BB, durum: Geçti
```

İlk ifadede sıra önemlidir. Kollar yukarıdan aşağı denendiği için `< 0 or > 100` kolunu en üste koyduk; yoksa 105 puan `>= 90` kolunda "AA" alırdı. Her kol önceki kolların bıraktığı değerleri yakaladığından `>= 80` demek, "90'ın altında ve 80 ve üstü" demektir. `else if` merdivenlerindeki mantığın aynısıdır (Bölüm 7).

İkinci ifadede kollar `and` ile birbirinden kesin ayrıldığı için sıranın önemi yoktur. Bu biçim daha uzundur ama kolların sıradan bağımsız okunmasını sağlar.

Derleyici sıra hatalarını yakalamamıza yardım eder. Daha önceki kollar bir kolu tümüyle kapsıyorsa o kol hiçbir zaman çalışamaz ve program derlenmez:

▸ siraHata.cs

```
int puan = 85;
string harf = puan switch
{
    >= 60 => "DD",
    >= 90 => "AA",
    _ => "FF"
};
Console.WriteLine(harf);
```

```
DERLEYİCİ ÇIKTISI
siraHata.cs(5,5): error CS8510: The pattern is unreachable. It has already been handled by a
previous arm of the switch expression or it is impossible to match.
```

Aynı sıra hatası `else if` merdiveninde sessiz bir mantık hatası olurdu. Kollar yalnızca **kısmen** örtüşüyorsa derleme hatası alınmaz ve yine ilk eşleşen kol kazanır.

`and`, `or`, `not` yalnızca desenlerde kullanılır; `&&`, `||` ve `!` ise `bool` ifadelerde. İkisini karıştırmamak gerekir: `puan >= 0 and < 50` yazımı bir `if` koşulunda tek başına geçerli değildir, ama `puan is >= 0 and < 50` geçerlidir.

is operatörü

`is`, bir değeri desene karşı sınayıp `bool` döndürür. Böylece `if` koşullarında okunaklı aralık denetimleri yazarız. C'de `0 <= x <= 100` yazamıyorduk (Bölüm 7'de gördüğümüz hata); C#'ta `x is >= 0 and <= 100` yazabiliriz:

▸ isDeseni.cs

```
int sayi = 72;

Console.WriteLine(sayi is >= 0 and <= 100);   // aralık
Console.WriteLine(sayi is 0 or 100);          // iki değerden biri mi?
Console.WriteLine(sayi is not 50);            // 50'den farklı mı?
```

```
char harf = 'k';
Console.WriteLine(harf is >= 'a' and <= 'z'); // küçük harf mi?
```

```
ÇIKTI
True
False
True
True
```

`sayi is not 50` yazımı `sayi != 50` ile aynı sonucu verir. `not` deseninin asıl değeri `null` ile birlikte ortaya çıkar; birazdan göreceğiz.

Tür deseni

Bir `object` değişkeni her tipten değeri tutabilir (Bölüm 42). Elimizdeki değerin gerçekte hangi tipte olduğunu desenle sorgulayabiliriz. Tür deseni, tipin yanına bir değişken adı yazarsak eşleşen değeri o tipte kullanılabilir hâle de getirir:

turDeseni.cs

```
object[] degerler = { 42, "merhaba", 3.5, 'x', true, null };

foreach (object? d in degerler)
{
    string aciklama = d switch
    {
        int n => $"tam sayı, karesi {n * n}",
        string s => $"metin, uzunluğu {s.Length}",
        double x => $"ondalıklı sayı, yarısı {x / 2}",
        bool => "mantıksal değer",
        null => "boş (null)",
        _ => $"başka bir tip: {d.GetType().Name}"
    };
    Console.WriteLine(aciklama);
}
```

```
ÇIKTI
tam sayı, karesi 1764
metin, uzunluğu 7
ondalıklı sayı, yarısı 1.75
başka bir tip: Char
mantıksal değer
boş (null)
```

`foreach` ile dizinin her elemanını sırayla ele alırız; ayrıntısı Bölüm 45'te. Burada önemli olan kollardır. `int n` kolu, değer `int` ise onu `n` adıyla verir. Değişken yalnızca o kolda vardır ve oradaki tipe sahiptir. Hiç değişken adı gerekmiyorsa (`bool` kolundaki gibi) yalnızca tipi yazarız. `null` kolu, `null` değerini ayrıca yakalar; tür desenleri `null` ile eşleşmez.

Bu davranış C ve C++ için yeni bir fikirdir. C'de bir `void*` işaretçisinin gösterdiği verinin tipini çalışma sırasında öğrenmenin bir yolu yoktur; C++'ta bunun için sanal işlevler (Bölüm 33) ya da `dynamic_cast` kullanılır. C#'ta her nesne kendi tipini taşır ve desen eşleme bu bilgiyi güvenli biçimde kullanır. Sınıf hiyerarşileriyle kullanımını Bölüm 51'de göreceğiz.

`is` ile tür deseni, "dönüştür ve kullan" işini tek adımda yapar:

isTur.cs

```
object o = "Merhaba dünya";

if (o is string metin && metin.Length > 5)
{
    Console.WriteLine($"Uzun metin: {metin.ToUpper()}");
}

if (o is not int)
{
    Console.WriteLine("Bu bir int değil.");
}
```

ÇIKTI

```
Uzun metin: MERHABA DÜNYA
Bu bir int değil.
```

`metin` değişkeni yalnızca desen eşleştiğinde atanmış sayılır; derleyici bu yüzden `&&`'in sağ tarafında ve `if` gövdesinde onu serbestçe kullanmamıza izin verir. Eşleşmeyen yolda kullanmaya kalkarsak `CS0165` alırız.

Üç Dilde · Tür sorgulama *C · C++ · C#*

C'de çalışma sırasında tür sorgusu yoktur. C++ `dynamic_cast` ile çok biçimli (polymorphic) sınıf hiyerarşilerinde sorgulama yapar; C# ise her değerde `is` ile sorgular ve dönüşümü aynı adımda yapar.

```
/* C: void* işaretçisinin gerçek tipini programcı bilmek zorundadır */
void *p = &deger;
int *ip = (int *)p;          /* doğru mu yanlış mı, derleyici bilmez */
```

```
Hayvan* h = ornek();
if (Kedi* k = dynamic_cast<Kedi*>(h)) {   // C++17 öncesinde de çalışır
    k->miyavla();
}
```

```
if (hayvan is Kedi kedi)
{
    kedi.Miyavla();
}
```

when ile ek koşul

Bir kolda tür ya da değer eşleşmesi yetmiyorsa `when` anahtar sözcüğüyle ek bir `bool` koşul ekleriz. `when` koşulu, desenden elde edilen değişkeni kullanabilir:

whenOrnek.cs

```
object[] girdiler = { 7, -3, 12, 0, "on", "" };

foreach (object g in girdiler)
{
    string sonuc = g switch
    {
        int n when n < 0 => "negatif",
        int n when n % 2 == 0 && n != 0 => "pozitif çift",
        int n when n > 0 => "pozitif tek",
```

```
        int => "sıfır",
        string { Length: 0 } => "boş metin",
        string s => $"metin: {s}",
        _ => "bilinmiyor"
    };
    Console.WriteLine(sonuc);
}
```

```
ÇIKTI
pozitif tek
negatif
pozitif çift
sıfır
metin: on
boş metin
```

Kolların sırası yine belirleyicidir. `int => "sıfır"` kolu en sona yakın olduğu için yalnızca önceki `int` kollarının yakalamadığı değeri, yani sıfırı alır. Derleyici `when` içindeki koşulu çözümleyemez; bu yüzden `when` içeren kollarda sıra hatalarını yakalayamaz. Aralık ve değer desenleriyle yazılabilen kararlarda önce onları, yetmediğinde `when`'i kullanmak daha güvenlidir.

Özellik deseni

Özellik deseni (property pattern), bir nesnenin özelliklerini (property) süslü parantez içinde sınar. Yukarıdaki `string { Length: 0 }` kolu, uzunluğu sıfır olan `string`'i yakalar. Özellikler ilişkisel ve mantıksal desenlerle birleşebilir:

▸ **ozellikDeseni.cs**

```
string[] adlar = { "Ece", "Can", "Zeynep", "" };

foreach (string ad in adlar)
{
    string tur = ad switch
    {
        { Length: 0 } => "boş",
        { Length: <= 3 } => "kısa ad",
        { Length: > 3 and < 7 } => "orta uzunlukta ad",
        _ => "uzun ad"
    };
    Console.WriteLine($"'{ad}' -> {tur}");
}
```

```
ÇIKTI
'Ece' -> kısa ad
'Can' -> kısa ad
'Zeynep' -> orta uzunlukta ad
'' -> boş
```

Özellik deseninin gerçek gücü, sınıf ve `record` tipleriyle ortaya çıkar (Bölüm 49 ve 52). Elimizdeki `Length` özelliği her `string` için hazırdır; bu yüzden örnek şimdiden çalışır. Bu desen iç içe de yazılabilir (`{ Adres: { Sehir: "Ankara" } }`).

Çoklu (tuple) deseni

Birden fazla değere aynı anda bakmak gerekirse onları parantez içinde bir demet (tuple) hâline getirip desenlerle eşleştirebiliriz. Demetleri Bölüm 46'da ayrıntılı işleyeceğiz; burada yalnızca `(a, b)` yazımının iki değeri birlikte ele aldığını bilmemiz yeterlidir. Taş-kağıt-makas oyununda kazananı seçelim:

▸ tasKagitMakas.cs

```
string[] oyuncu1 = { "taş", "kağıt", "makas", "taş" };
string[] oyuncu2 = { "makas", "kağıt", "taş", "kağıt" };

for (int i = 0; i < oyuncu1.Length; i++)
{
    string sonuc = (oyuncu1[i], oyuncu2[i]) switch
    {
        ("taş", "makas") or ("kağıt", "taş") or ("makas", "kağıt")
            => "1. oyuncu kazandı",
        ("makas", "taş") or ("taş", "kağıt") or ("kağıt", "makas")
            => "2. oyuncu kazandı",
        _ => "berabere"
    };
    Console.WriteLine($"{oyuncu1[i]} - {oyuncu2[i]}: {sonuc}");
}
```

```
ÇIKTI
taş - makas: 1. oyuncu kazandı
kağıt - kağıt: berabere
makas - taş: 2. oyuncu kazandı
taş - kağıt: 2. oyuncu kazandı
```

Aynı kararı iç içe `if` ve `switch` ile yazsaydık satır sayısı en az üç kat olurdu. Tuple deseninde `_` herhangi bir değeri yakalar: `(0, _)` "ilk değer sıfır, ikinci ne olursa olsun" demektir.

> **Not · Liste desenleri**
>
> C# 11 (2022), dizilerin ve listelerin biçimini sorgulayan **liste desenlerini** ekledi: `dizi is [1, 2, 3]` tam eşleşme, `dizi is [var ilk, .., var son]` ise ilk ve son elemanı ayırır. Dizileri Bölüm 47'de öğrendikten sonra bu desene bir kez daha bakmak yararlıdır.

> *Tarihçe · Desen eşleme*
>
> Desen eşleme C#'a küçük adımlarla girdi. C# 7.0 (2017) `is` ile tür desenini, sabit desenini ve `switch` deyiminde tür desenleri ile `when` koşulunu getirdi. C# 8.0 (2019) `switch` ifadesini, özellik desenini ve tuple desenini ekledi. C# 9.0 (2020) ilişkisel desenleri ve `and`, `or`, `not` birleştiricilerini, C# 11 (2022) liste desenlerini getirdi. Eski kodlarda `case int n when n >= 90:` gibi daha uzun yazımlar görülür; yeni kodda ilişkisel desenler tercih edilir. Desen eşleme fikri işlevsel dillerden (örneğin ML ve Haskell ailesi) gelir.

44.5 Üçlü Operatör ve null Operatörleri

Üçlü operatör `?:`, koşula göre iki değerden birini seçer. C ve C++'tan farksızdır ve temel kullanımını Bölüm 42'de gördük. Burada C#'a özgü ayrıntılarına bakalım.

ucluOrnek.cs

```
int a = 8, b = 13;

int buyuk = a > b ? a : b;
string durum = buyuk >= 10 ? "iki haneli" : "tek haneli";
Console.WriteLine($"{buyuk} {durum}");

int? sonuc = a > b ? a : null;      // C# 9: hedef tipe göre çözülür
Console.WriteLine(sonuc.HasValue);

Console.WriteLine($"Sonuç: {(a > b ? "a büyük" : "b büyük")}");
Console.WriteLine((a < b ? 1 : 2.5) / 2);
```

ÇIKTI

```
13 iki haneli
False
Sonuç: b büyük
0.5
```

Son satırda `1` (`int`) ve `2.5` (`double`) aynı ifadede karşılaşınca sonuç `double` olur ve bölme ondalıklı yapılır. Dize içine yazılan üçlü ifade ise parantez gerektirir; çünkü `{ }` içindeki `:` karakteri biçim belirtecini (`{x:F2}`) başlatır (Bölüm 43).

`int? sonuc = a > b ? a : null;` satırı C# 9'dan önce derlenmezdi: `int` ile `null` arasında ortak tip yoktur. C# 9'dan itibaren derleyici, atamanın hedef tipine (`int?`) bakarak ifadeyi çözümler.

İç içe üçlü operatörler kısa yazılır ama okunması zordur. İkiden fazla seçenek varsa `switch` ifadesi tercih edilmelidir.

?? ve ?.

`??` (null birleştirme) ve `??=` operatörlerini Bölüm 42'de nullable değer tipleri için gördük; aynı operatörler referans tiplerinde de çalışır. `?.` (null koşullu, null-conditional) operatörü ise nesne `null` ise üyeye erişmeden `null` verir; böylece `NullReferenceException` oluşmaz:

nullOperatorleri.cs

```
string? ad = null;

int uzunluk = ad?.Length ?? 0;        // ad null ise Length'e erişilmez
Console.WriteLine(uzunluk);

ad = "Deniz";
Console.WriteLine(ad?.Length ?? 0);
Console.WriteLine(ad?.ToLower() ?? "(yok)");
```

ÇIKTI

```
0
5
deniz
```

`ad` `null` iken `ad?.Length` ifadesi `null` verir ve `?? 0` ona yedek değer atar. Dolu iken ise normal erişim yapılır. Referans tiplerinde `null` yönetimini (`string?`) Bölüm 59'da işleyeceğiz.

> **▲ Sık Yapılan Hata**
>
> `?.` ile üretilen değerin tipi, üyenin tipinin `null` alabilen hâlidir. `ad?.StartsWith("D")` ifadesi `bool` değil `bool?` verir ve bu yüzden `if (ad?.StartsWith("D"))` derlenmez (`CS0266`). Doğrusu `if (ad?.StartsWith("D") == true)` yazmaktır. Sayısal karşılaştırmalarda ise (`ad?.Length > 3`) sonuç `bool`'dur ve `ad` `null` iken `false` olur.

44.6 Örnek Uygulama: Menülü Banka Programı

`switch` deyiminin en doğal kullanım yeri menülerdir. Aşağıdaki program, bir hesap için bakiye görüntüleme, para yatırma ve para çekme işlemlerini sunar. Üç noktaya dikkat edelim:

- Menü bir `while` döngüsü içindedir. `switch` içindeki `break` yalnızca `switch`'ten çıkar; döngüyü bitirmek için `devam` bayrağı `false` yapılır.
- `"2"` ve `"3"` etiketleri tutarın alınması işini paylaşır; yatırma ya da çekme ayrımı sonradan yapılır.
- Para hesabında `decimal` kullanırız (Bölüm 42).

bankaMenu.cs

```
decimal bakiye = 1000m;
bool devam = true;

while (devam)
{
    Console.WriteLine("1) Bakiye  2) Yatır  3) Çek  0) Çıkış");
    Console.Write("Seçim: ");
    string secim = Console.ReadLine() ?? "0";

    switch (secim)
    {
        case "1":
            Console.WriteLine($"Bakiye: {bakiye} TL");
            break;
        case "2":
        case "3":
            Console.Write("Tutar: ");
            if (!decimal.TryParse(Console.ReadLine(), out decimal tutar)
                || tutar <= 0)
            {
                Console.WriteLine("Geçersiz tutar.");
                break;              // switch'ten çıkar
            }
            if (secim == "2")
            {
                bakiye += tutar;
                Console.WriteLine("Yatırıldı.");
            }
            else if (tutar > bakiye)
            {
                Console.WriteLine("Yetersiz bakiye.");
            }
            else
            {
                bakiye -= tutar;
                Console.WriteLine("Çekildi.");
            }
            break;
        case "0":
```

```
            devam = false;
            break;
        default:
            Console.WriteLine("Geçersiz seçim.");
            break;
    }
}

Console.WriteLine($"Son bakiye: {bakiye} TL");
```

```
ÇIKTI
1) Bakiye  2) Yatır  3) Çek  0) Çıkış
Seçim: 1
Bakiye: 1000 TL
1) Bakiye  2) Yatır  3) Çek  0) Çıkış
Seçim: 2
Tutar: 250
Yatırıldı.
1) Bakiye  2) Yatır  3) Çek  0) Çıkış
Seçim: 3
Tutar: 5000
Yetersiz bakiye.
1) Bakiye  2) Yatır  3) Çek  0) Çıkış
Seçim: 3
Tutar: -20
Geçersiz tutar.
1) Bakiye  2) Yatır  3) Çek  0) Çıkış
Seçim: 7
Geçersiz seçim.
1) Bakiye  2) Yatır  3) Çek  0) Çıkış
Seçim: 0
Son bakiye: 1250 TL
```

`tutar` değişkeni `if` koşulunda tanımlandığı için `switch` bölümünün geri kalanında kullanılabilir. Koşuldaki `||` kısa devre yaptığından `TryParse` başarısız olduğunda `tutar <= 0` hiç değerlendirilmez; böylece atanmamış değişken okunmaz. Bölüm 46'da menü gibi tekrarlayan işleri metotlara ayırmayı öğrenecek ve bu programı daha düzenli yazabileceğiz.

44.7 Örnek Uygulama: Kargo Ücreti

İkinci örnekte iki değere, bölgeye ve ağırlığa birlikte bakarak ücret hesaplıyoruz. Kural şöyle olsun:

Bölge	2 kg'a kadar	10 kg'a kadar
yerel	40 TL	65 TL
sehirlerarasi	60 TL	95 TL
yurtdisi	250 TL	400 TL

10 kg'ın üzerindeki paketlerde, 10 kg'ı aşan her başlayan kilogram için 8 TL eklenir. Tuple deseni bölge ve ağırlık koşullarını tek ifadede toplar. Fazla ağırlığı sonradan ayrı hesaplayacağımız için desene en çok 10 kg'lık değeri veriyoruz:

▸ kargoUcreti.cs

```
Console.Write("Ağırlık (kg): ");
double kg = double.Parse(Console.ReadLine()!);
Console.Write("Bölge (yerel/sehirlerarasi/yurtdisi): ");
```

```
string bolge = Console.ReadLine() ?? "";

double sinirli = Math.Min(kg, 10);       // tablo en çok 10 kg'ı kapsar

decimal temel = (bolge, sinirli) switch
{
    (_, <= 0) => -1m,                    // geçersiz ağırlık
    ("yerel", <= 2) => 40m,
    ("yerel", _) => 65m,
    ("sehirlerarasi", <= 2) => 60m,
    ("sehirlerarasi", _) => 95m,
    ("yurtdisi", <= 2) => 250m,
    ("yurtdisi", _) => 400m,
    _ => -1m                             // bilinmeyen bölge
};

if (temel < 0)
{
    Console.WriteLine("Geçersiz bölge ya da ağırlık.");
    return;
}

decimal ek = kg > 10 ? (decimal)Math.Ceiling(kg - 10) * 8m : 0m;
Console.WriteLine($"Ücret: {temel + ek:F2} TL");
```

```
ÇIKTI
Ağırlık (kg): 12.5
Bölge (yerel/sehirlerarasi/yurtdisi): yurtdisi
Ücret: 424.00 TL
```

400 TL temel ücrete 2,5 kg'lık fazlalık eklendi. Başlayan kilogram olduğu için 2,5 yukarı yuvarlanıp 3 olur; 3 × 8 = 24 TL eklenir. Sırada dikkat edilecek iki nokta var. Birincisi, `(_, <= 0)` kolu en üstte olduğu için geçersiz ağırlık her bölge için önce yakalanır. İkincisi, `("yerel", _)` gibi kollar, daha önceki `<= 2` kolundan sonra geldiği için "2 ile 10 kg arası" anlamını taşır; sıra, aralığın alt sınırını örtük olarak belirler. `F2` biçimi iki ondalık basamak yazar (Bölüm 43).

> **▲ Sık Yapılan Hata**
>
> Kaynağı klavye olan metinler `"Yerel"` ya da `"yerel "` gibi gelebilir ve `switch` bunlarla eşleşmez. Gerçek programlarda girdiyi `Trim()` ve `ToLowerInvariant()` ile normalleştirmek iyi bir alışkanlıktır (Bölüm 48).

Üç Dilde · Başlatıcılı koşul ve değişken kapsamı *C · C++ · C#*

C++17, `if` içinde bir değişkeni başlatıp kapsamını `if` ile sınırlamaya izin verir (Bölüm 24). C#'ta benzer iş `out` değişkeni ve `is` deseniyle yapılır; ama değişkenin kapsamı `if`'i çevreleyen bloktur.

```
if (int n = std::stoi(s); n > 0) {      // n yalnızca if'in içinde yaşar
    std::cout << n;
}
```

```
if (int.TryParse(s, out int n) && n > 0)
{
    Console.WriteLine(n);
}
Console.WriteLine(n);       // geçerli: n bu bloğun sonuna kadar yaşar
```

Bölüm Özeti

- C#'ta `if` koşulu `bool` olmak zorundadır. `bool?` bile doğrudan koşul olamaz; `== true` ya da `?? false` gerekir.
- `out int x` ve `is string s` ile koşulda tanımlanan değişkenler, `if`'i çevreleyen bloğun sonuna kadar yaşar. Bu, koruma koşulu (`if (!TryParse(...)) return;`) kalıbını mümkün kılar.
- `switch` deyiminde içinde deyim bulunan her `case` bölümü `break`, `return`, `throw`, `continue` ya da `goto` ile bitmelidir; aksi hâlde `CS0163` oluşur. Boş etiketler birleşir.
- Bilinçli geçiş `goto case değer;` ya da `goto default;` ile yazılır. Klasik `case` etiketleri derleme zamanı sabiti olmalıdır.
- `switch` tam sayı ve karakterle sınırlı değildir; `string`, `enum` ve desenlerle de çalışır. Bütün `switch` bloğu tek bir kapsamdır.
- `switch` ifadesi (C# 8) `değer switch { desen => sonuç, ... }` biçimindedir; ilk eşleşen kol kazanır, `_` her şeyi yakalar, son `}` sonrasında `;` gelir.
- Her durumu kapsamayan `switch` ifadesi `CS8509` uyarısı verir; hiçbir kol eşleşmezse çalışma sırasında `SwitchExpressionException` oluşur. Daha önceki kolların tamamen kapsadığı bir kol `CS8510` hatasıdır.
- İlişkisel (`< 5`, `>= 90`), mantıksal (`and`, `or`, `not`), sabit, tür (`int n`), özellik (`{ Length: 0 }`) ve tuple (`(0, _)`) desenleri bir arada kullanılabilir.
- `x is >= 0 and <= 100` ve `x is not null` gibi `is` desenleri, `if` içinde aralık ve tür denetimini okunaklı yazar.
- `when`, kola `bool` bir ek koşul ekler. Derleyici `when` içeren kollarda sıra hatalarını yakalayamaz.
- Üçlü operatör farklı sayısal tipleri büyük tipe yükseltir; C# 9'dan beri `int? x = koşul ? 1 : null;` yazılabilir.
- `??` boşsa yedek değeri verir, `?.` nesne `null` ise üyeye erişmez. `?.` bir `bool?` üretebilir ve bu doğrudan koşul olamaz.
- Desen eşleme C# 7'den C# 11'e kadar adım adım gelişti: tür ve sabit desenleri (7), `switch` ifadesi, özellik ve tuple desenleri (8), ilişkisel ve mantıksal desenler (9), liste desenleri (11).

Alıştırmalar

1. Aşağıdaki satırlardan hangisi (tek başına bir programdadır; `int x = 5; bool? b = true;` tanımlıdır) derleme hatası verir?
   A) `if (x > 3 && x < 9) { }`
   B) `if (b == true) { }`
   C) `if (b) { }`
   D) `if (x is > 3) { }`
   E) `if (!(x == 5)) { }`
2. Aşağıdaki `switch` deyiminde `case 1:` bölümünden sonra `break` yazılmazsa ne olur?

```
switch (n)
{
    case 1:
        Console.WriteLine("bir");
    case 2:
```

```
        Console.WriteLine("iki");
        break;
}
```

A) Program çalışır ve `n` 1 ise "bir" ve "iki" yazar
B) Program çalışır ve `n` 1 ise yalnızca "bir" yazar
C) Derleme hatası oluşur (`CS0163`)
D) Çalışma zamanında istisna oluşur
E) Yalnızca uyarı verilir, program derlenir

3. `int x = 10;` iken `string s = x switch { < 10 => "a", < 20 => "b", _ => "c" };` ifadesinin sonucu nedir?
   A) `"a"`
   B) `"b"`
   C) `"c"`
   D) Derleme hatası
   E) Çalışma zamanı hatası
4. Aşağıdakilerden hangisi `switch` ifadeleri için doğrudur?
   A) Her kolun sonunda `break` yazılmalıdır.
   B) Kollar düşebilir.
   C) Kollar yukarıdan aşağı denenir ve ilk eşleşen kol sonucu verir.
   D) `_` kolu `default` gibi zorunludur ve her zaman en üstte olmalıdır.
   E) Yalnızca `int` değerler üzerinde kullanılabilir.
5. `int n = 4; Console.WriteLine(n is not 3);` komutu ne yazdırır?
   A) `True`
   B) `False`
   C) `4`
   D) `not`
   E) Derleme hatası
6. Aşağıdaki program ekrana ne yazdırır?

```
int n = 2;
int toplam = 0;
switch (n)
{
    case 1:
        toplam += 1;
        break;
    case 2:
        toplam += 10;
        goto case 3;
    case 3:
        toplam += 100;
        break;
    default:
        toplam += 1000;
        break;
}
Console.WriteLine(toplam);
```

7. Aşağıdaki program ekrana ne yazdırır?

```
int x = 0, y = 5;

string a = (x, y) switch
{
    (0, 0) => "orijin",
    (0, _) => "y ekseni",
    (_, 0) => "x ekseni",
    _ => "düzlem"
};

string b = (y - 5, x) switch
{
    (0, 0) => "orijin",
    (0, _) => "y ekseni",
    _ => "diğer"
};
Console.WriteLine(a + " " + b);
```

8. Aşağıdaki program ekrana ne yazdırır?

```
object[] liste = { 15, 4.0, "kaf", -2, 'z', 8 };
int say = 0;

foreach (object o in liste)
{
    string s = o switch
    {
        int n when n > 10 => "A",
        int n when n > 0 => "B",
        int => "C",
        double => "D",
        string t => t.Length.ToString(),
        _ => "?"
    };
    Console.Write(s);
    say++;
}
Console.WriteLine($" {say}");
```

9. Aşağıdaki program ekrana ne yazdırır?

```
string? ad = null;
int? yas = null;

Console.WriteLine(ad?.Length ?? -1);
Console.WriteLine((yas ?? 18) >= 18 ? "yetişkin" : "çocuk");
yas ??= 12;
Console.WriteLine(yas >= 18 ? "yetişkin" : "çocuk");
Console.WriteLine(yas is > 10 and < 20);
Console.WriteLine(ad is null);
```

10. Aşağıdaki program ekrana ne yazdırır? (`continue` ve `break` davranışına dikkat edin.)

```
for (int i = 0; i < 5; i++)
{
    switch (i)
    {
        case 1:
            continue;
        case 3:
```

```
            break;
        default:
            Console.Write(i);
            break;
    }
    Console.Write("-");
}
Console.WriteLine();
```

11. Aşağıdaki program ekrana ne yazdırır?

```
int x = 7;
string s = x switch
{
    < 5 => "küçük",
    < 10 => "orta",
    _ => "büyük"
};
string t = x switch
{
    > 5 and < 8 => "yedi civarı",
    _ => "diğer"
};
bool b = x is not (> 3 and < 6);
Console.WriteLine(s + " " + t + " " + b);
```

12. Aşağıdaki kodda dört ayrı derleme sorunu vardır. Bulun ve her birini düzeltin.

```
int ay = 2;
int sinir = 5;
switch (ay)
{
    case 1:
        Console.WriteLine("Ocak");
    case sinir:
        Console.WriteLine("Beşinci ay");
        break;
}
bool? izin = null;
if (izin)
{
    Console.WriteLine("izinli");
}
string durum = ay switch
{
    >= 1 => "geçerli",
    > 6 => "yaz",
    _ => "diğer"
};
```

13. `string? ad = "Mert"; if (ad?.StartsWith("M")) { ... }` kodu neden derlenmez? Nasıl düzeltilir?
14. Ay numarasını (1–12) ve yılı okuyan, o ayın kaç gün çektiğini `switch` ifadesiyle hesaplayıp yazdıran bir program yazın. Şubat için artık yıl kuralını kullanın (4'e bölünür ve 100'e bölünmez ya da 400'e bölünür). Geçersiz ayda `Geçersiz ay` yazdırın.
15. Üç kenar uzunluğunu (`double a, b, c`) alıp üçgenin türünü yazdıran bir program yazın: "geçersiz" (en küçük iki kenarın toplamı üçüncüden büyük değil ya da bir kenar sıfır veya negatif), "eşkenar", "ikizkenar" ya da "çeşitkenar". `switch` ifadesi ve `when` kullanın.

16. İki sayı ve bir işlem karakteri (`+`, `-`, `*`, `/`) okuyup sonucu yazdıran bir hesap makinesi yazın. Bölmede ikinci sayı sıfırsa `Sıfıra bölünemez` yazdırın; tanımsız işlem karakterinde `Geçersiz işlem` yazdırın. `switch` ifadesi kullanın.

BÖLÜM 45

Döngüler

BU BÖLÜMDE

- `while`, `do-while` ve `for` döngülerinin C# sözdizimini ve C'den farklarını
- Döngü koşulunun neden `bool` olmak zorunda olduğunu ve `for` sayacının kapsamını
- `break`, `continue` ve sonsuz döngüleri; C#'ta etiketli `break` bulunmadığını
- `foreach` döngüsünü ve iterasyon değişkeninin neden salt okunur olduğunu
- İç içe döngülerle desen yazmayı; `new string(...)` ve `{x,4}` yazımlarını
- Klasik algoritmaların C# sürümlerini ve `Math.Abs`, `checked` gibi C#'a özgü tuzakları

Bölüm 8'de C'nin üç döngüsünü, Bölüm 24'te de C++'ın aralık tabanlı `for` döngüsünü gördük. C#'ın döngüleri C'dekilere büyük ölçüde benzer: `while`, `do-while` ve `for` söz dizimi neredeyse birebir aynıdır, `break` ve `continue` aynı işi yapar. Farklar çoğunlukla derleyicinin bu farkları hata olarak bildirmesinde ortaya çıkar.

Bu bölümde önce üç döngüyü C# kurallarıyla gözden geçireceğiz, sonra C#'a özgü `foreach` döngüsüne geçeceğiz. Klasik algoritmaların mantığını Bölüm 8'den biliyoruz; burada yalnızca C#'a özgü ayrıntılara bakacağız.

Programlar Bölüm 41'de tanıdığımız üst düzey deyim (top-level statements) biçiminde yazılacak. `List<int>` gibi tipleri ayrıntılı işlemek için Bölüm 54'ü bekleyeceğiz; şimdilik bunları bir koleksiyon olarak okumak yeterli.

45.1 while Döngüsü

`while` döngüsünde koşul her turdan önce denetlenir; koşul doğruysa gövde çalışır. C#'ta da üç parça aynıdır: ilk değer, koşul ve ilerleme.

whileOrnek.cs

```
int sayi = 1;
while (sayi <= 5)
{
    Console.Write($"{sayi} ");
    sayi++;
}
Console.WriteLine();
Console.WriteLine($"Döngü bitti, sayi = {sayi}");
```

ÇIKTI

```
1 2 3 4 5
Döngü bitti, sayi = 6
```

Döngü bittiğinde `sayi` 5 değil 6'dır. Koşulu bozan ilk değerde döngü durur. Bir döngü yazarken bunu önceden kestirebilmek en önemli becerilerden biridir.

Koşul bool olmalı

C'de sıfırdan farklı her değer doğru sayılır. C#'ta ise `bool` bir sayı değildir (Bölüm 42) ve tam sayı bir koşul yerine kullanılamaz:

▸ whileSayi.cs

```
int kalan = 3;
while (kalan)
{
    kalan--;
}
```

DERLEYİCİ ÇIKTISI

```
whileSayi.cs(2,8): error CS0029: Cannot implicitly convert type 'int' to 'bool'
```

> **▲ Sık Yapılan Hata · Koşulu açıkça yazın**
>
> `while (kalan)` yazmak C'den gelen bir alışkanlıktır. C#'ta karşılaştırmayı açıkça yazarız: `while (kalan > 0)`. Sonsuz döngü de `while (1)` ile değil, `while (true)` ile yazılır.

45.2 do-while Döngüsü

`do-while`'da koşul gövdeden sonra denetlenir; gövde en az bir kez çalışır. Kapanış parantezinden sonraki noktalı virgül zorunludur. Söz dizimi C'dekiyle aynıdır. Girdinin belirli bir aralıkta olmasını isteyen doğrulama, bu yapının en tipik kullanımıdır:

▸ aralikDogrula.cs

```
int sayi;
do
{
    Console.Write("1 ile 10 arasında bir sayı girin: ");
    sayi = Convert.ToInt32(Console.ReadLine());
} while (sayi < 1 || sayi > 10);
Console.WriteLine($"Girdiğiniz sayı: {sayi}");
```

ÇIKTI

```
1 ile 10 arasında bir sayı girin: 15
1 ile 10 arasında bir sayı girin: 0
1 ile 10 arasında bir sayı girin: 7
Girdiğiniz sayı: 7
```

`sayi` değişkenini `do` bloğunun dışında tanımladık. Koşul da bloğun dışında olduğu için değişken koşulda görünür. Değişkeni bloğun içinde tanımlasaydık koşul onu göremezdi ve derleme `CS0103` hatasıyla durardı.

45.3 for Döngüsü

`for` döngüsü tekrar sayısı bilindiğinde en okunaklı döngüdür. Üç parça tek satırda yazılır: `for (başlangıç; koşul; ilerleme)`. Çalışma sırası C'dekiyle aynıdır: başlangıç yalnızca bir kez yapılır, koşul her turdan önce, ilerleme her turdan sonra denetlenir.

forOrnek.cs

```
for (int i = 10; i > 0; i -= 3)
{
    Console.Write($"{i} ");
}
Console.WriteLine();
```

ÇIKTI

```
10 7 4 1
```

Parantez içindeki üç parça da isteğe bağlıdır. `for (;;)` hiçbir parçası olmayan, `break` ile çıkılana kadar dönen bir döngüdür.

Sayaç kapsamı

`for` parantezinde tanımlanan sayaç (`int i`) yalnızca döngünün içinde yaşar. Döngüden sonra ona erişmek derleme hatasıdır:

forKapsam.cs

```
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(i);
}
Console.WriteLine(i);
```

DERLEYİCİ ÇIKTISI

```
forKapsam.cs(5,19): error CS0103: The name 'i' does not exist in the current context
```

Sayacı döngüden sonra da kullanacaksak onu döngüden önce tanımlarız: `int i; for (i = 0; i < 3; i++) { ... }`.

Noktalı virgül tuzağı

`for (...)` parantezinden hemen sonraki `;` gövdeyi boşaltır. Alttaki blok döngüye ait değildir ve yalnızca bir kez çalışır. C# bu durumu derleme hatası yapmaz; ama uyarı verir:

```
int sayac = 0;
for (int i = 0; i < 5; i++);
{
    sayac++;
}
Console.WriteLine($"sayac = {sayac}");
```

DERLEYİCİ ÇIKTISI

```
noktaliVirgul.cs(2,28): warning CS0642: Possible mistaken empty statement
```

Uyarı derlemeyi durdurmaz; program `sayac = 1` yazdırır. Uyarıyı görmezden gelmek yerine nedenini bulup düzeltmek gerekir. Her döngü gövdesini `{ }` içine almak bu hatayı önler.

45.4 break, continue ve Sonsuz Döngü

`break` en içteki döngüyü bitirir. `continue` ise turun kalanını atlayıp sonraki tura geçer. `for`'da `continue` sonrasında ilerleme yine çalışır. `while`'da ilerleme gövdenin sonundaysa `continue` onu da atlar ve döngü sonsuz olabilir; Bölüm 8'de gördüğümüz bu tuzak C#'ta da aynıdır.

▸ cikisVeAtla.cs

```
for (int i = 1; i <= 10; i++)
{
    if (i % 2 == 0)
    {
        continue;
    }
    if (i > 7)
    {
        break;
    }
    Console.Write($"{i} ");
}
Console.WriteLine();
```

```
ÇIKTI
1 3 5 7
```

Çıkış koşulunu döngünün ortasında denetlemek istediğimizde `while (true)` ile başlayıp `break` ile çıkarız. Aşağıdaki program, 0 girilene kadar sayıları toplar:

▸ toplamGirisi.cs

```
int toplam = 0;
while (true)
{
    Console.Write("Sayı (0 bitirir): ");
    int sayi = Convert.ToInt32(Console.ReadLine());
    if (sayi == 0)
    {
        break;
    }
    toplam += sayi;
}
Console.WriteLine($"Toplam: {toplam}");
```

```
ÇIKTI
Sayı (0 bitirir): 5
Sayı (0 bitirir): 12
Sayı (0 bitirir): -3
Sayı (0 bitirir): 0
Toplam: 14
```

`sayi` değişkeni döngünün gövdesinde tanımlandığı için her turda yeniden doğar. Toplamı ise döngüden önce tanımladık; çünkü döngü bittikten sonra da onu yazdırıyoruz.

C#'ta etiketli break yoktur

C++'ta iç içe iki döngüden birden çıkmak için etiketli `break` kullanılabilir. C#'ta böyle bir yazım yoktur. Derleyici `break dis;` satırını bir söz dizimi hatası olarak bildirir:

etiketliBreak.cs

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (i == 2 && j == 2)
        {
            break dis;
        }
    }
}
```

```
DERLEYİCİ ÇIKTISI

etiketliBreak.cs(7,19): error CS1002: ; expected
```

Dış döngüden de çıkmak istiyorsak bir bayrak (flag) değişkeni kullanırız; bu yöntemi Bölüm 8'de gördük. C#'ta `goto` ile etiket yazmak da mümkündür, ama akışı izlemeyi zorlaştırdığı için bayrak çoğu durumda daha okunaklıdır.

45.5 foreach Döngüsü

`foreach`, bir koleksiyonun elemanlarını baştan sona, indeks tutmadan dolaşır. Söz dizimi `foreach (tip ad in koleksiyon)` biçimindedir. Koleksiyon bir dizi, bir `string` ya da dolaşılabilir başka bir tip olabilir. Dizileri Bölüm 47'de, koleksiyonları Bölüm 54'te ayrıntılı işleyeceğiz; burada dizi ve `List<T>` ile yetiniyoruz.

foreachOrnek.cs

```
int[] puanlar = { 72, 85, 90, 64, 88 };
int toplam = 0;
int enYuksek = int.MinValue;
foreach (int puan in puanlar)
{
    toplam += puan;
    if (puan > enYuksek)
    {
        enYuksek = puan;
    }
}
Console.WriteLine($"Toplam: {toplam}, en yüksek: {enYuksek}");
Console.WriteLine($"Ortalama: {(double)toplam / puanlar.Length:F2}");

string metin = "Merhaba 2026";
int rakam = 0;
foreach (char c in metin)
{
    if (char.IsDigit(c))
    {
        rakam++;
```

```
    }
}
Console.WriteLine($"Rakam sayısı: {rakam}");
```

```
ÇIKTI
Toplam: 399, en yüksek: 90
Ortalama: 79.80
Rakam sayısı: 4
```

En büyük değeri ararken başlangıç değerini `0` yerine `int.MinValue` yaptık. Tüm notlar negatif olsaydı `0` ile başlanan arama yanlış sonuç verirdi. Dizinin uzunluğunu `Length` özelliğiyle alıyoruz; dizinin sınırını elle yazmak zorunda değiliz.

`foreach` değişkeninin tipini `var` ile de yazabiliriz: `foreach (var puan in puanlar)`. Derleyici tipi koleksiyondan çıkarır. `string` üzerinde dolaşıldığında her eleman bir `char` olduğu için `c` değişkeni `char` tipindedir.

foreach değişkeni salt okunurdur

Iterasyon değişkeni (`puan`, `c`) yalnızca okunabilir. Ona atama yapmaya kalkarsak derleyici hata verir:

▸ elemanDegistir.cs

```
int[] sayilar = { 1, 2, 3 };
foreach (int s in sayilar)
{
    s = s * 2;
}
```

```
DERLEYİCİ ÇIKTISI
elemanDegistir.cs(4,5): error CS1656: Cannot assign to 's' because it is a 'foreach iteration
variable'
```

Dizinin elemanlarını değiştirmek istiyorsak indeksli bir `for` kullanırız:

▸ elemanDegistirFor.cs

```
int[] sayilar = { 1, 2, 3 };
for (int i = 0; i < sayilar.Length; i++)
{
    sayilar[i] *= 2;
}
foreach (int s in sayilar)
{
    Console.Write($"{s} ");
}
Console.WriteLine();
```

```
ÇIKTI
2 4 6
```

Dolaşırken koleksiyonu değiştirmek

`foreach` çalışırken koleksiyona eleman eklemek ya da koleksiyondan eleman silmek yasaktır. Bu durum derleme sırasında yakalanmaz; çalışma zamanında `InvalidOperationException` fırlatılır ve mesaj şudur: `Collection was modified; enumeration operation may not execute.`

```
var liste = new List<int> { 1, 2, 3 };
foreach (int x in liste)
{
    if (x == 2)
    {
        liste.Add(9);
    }
}
```

▲ Sık Yapılan Hata · Dolaşırken boyu değiştirmeyin

Silme ya da ekleme gerekiyorsa iki yoldan birini seçin. Birincisi, değişiklikleri ayrı bir listede toplayıp dolaşma bittikten sonra uygulamak. İkincisi, silme yapacaksak `for` döngüsünü sondan başa doğru çalıştırmak; böylece silinen elemandan sonraki indeksler kaymaz.

Üç Dilde · Üç dilde dolaşma *C · C++ · C#*

C'de (Bölüm 8) ve C++'ta (Bölüm 24) bir dizinin elemanlarını dolaşmanın yolları, C#'ın `foreach` döngüsüyle şöyle karşılaştırılır:

Konu	C	C++	C#
Yazım	`for (int i = 0; i < n; i++)`	`for (int s : dizi)`	`foreach (int s in dizi)`
Eleman değiştirme	`dizi[i] = ...`	`for (int& s : dizi)` ile mümkün	iterasyon değişkeni salt okunur; indeksli `for` gerekir
Uzunluğu bilme	`n` değerini elle verirsiniz	`dizi` tipinden ya da kapsayıcının `size()` 'ından	`Length` ya da `Count` özelliği
Dolaşılabilen tipler	yalnızca indeksle erişilen diziler	dizi ve standart kapsayıcılar	dolaşılabilir her koleksiyon
Dolaşırken boyu değiştirme	derleyici denetlemez	yineleyiciler geçersizleşebilir (tanımsız davranış)	`InvalidOperationException` fırlatılır

C#'ta `foreach`, C++'taki aralık tabanlı `for` ile aynı fikri taşır; ama iterasyon değişkenine yazmaya izin vermez. Bu kısıt döngü değişkenini garanti altına alır: değişken dizideki yerini değiştiremez. Referans tipli elemanların (örneğin sınıf nesnelerinin) içeriği yine değiştirilebilir; yalnızca değişkenin kendisi salt okunurdur.

45.6 İç İçe Döngüler ve Desenler

İç içe döngülerde dış döngü satırları, iç döngü her satırdaki sütunları sayar. Bu düzen Bölüm 8'de yıldız ve sayı desenlerinde kullanıldı. C#'ta iki kolaylık vardır: `new string('*', n)` kurucusu bir karakteri `n` kez üreten bir metin verir; ara değerleri ise `{x,4}` yazımıyla sağa hizalarız. C'de bu iş `printf("%4d")` ile yapılırdı.

▸ desenPiramit.cs

```
int n = 4;
for (int i = 1; i <= n; i++)
{
    string bosluk = new string(' ', n - i);
    string yildiz = new string('*', 2 * i - 1);
    Console.WriteLine(bosluk + yildiz);
}
Console.WriteLine();
```

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= 4; j++)
    {
        Console.Write($"{i * j,4}");
    }
    Console.WriteLine();
}
```

```
ÇIKTI
   *
  ***
 *****
*******

   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
```

Piramitte her satırda önce `n - i` boşluk, sonra `2 * i - 1` yıldız basılır. Bu formül Bölüm 8'deki ile aynıdır; burada yalnızca satırı bir metin olarak kurup bir kerede yazdırdık.

45.7 Klasik Algoritmaların C# Sürümleri

Bölüm 8'deki algoritmaların mantığı C#'ta değişmez. Aşağıda yalnızca C#'ın getirdiği kolaylıkları ve tuzakları gösteriyoruz.

Basamaklar ve Math.Abs tuzağı

Bir sayının basamaklarını `% 10` ve `/ 10` ile ayırmak Bölüm 8'de gördüğümüz gibidir. Negatif girdiyi `Math.Abs` ile pozitife çeviriyoruz:

basamaklar.cs

```
Console.Write("Bir sayı gir: ");
int sayi = Convert.ToInt32(Console.ReadLine());
int n = Math.Abs(sayi);
int adet = 0, toplam = 0, ters = 0;
do
{
    int basamak = n % 10;
    adet++;
    toplam += basamak;
    ters = ters * 10 + basamak;
    n /= 10;
} while (n > 0);
Console.WriteLine($"Basamak sayısı: {adet}, toplam: {toplam}");
Console.WriteLine($"Tersi: {ters}");
```

```
ÇIKTI
Bir sayı gir: -472
Basamak sayısı: 3, toplam: 13
Tersi: 274
```

> **▲ Sık Yapılan Hata · Math.Abs(int.MinValue) bir istisna fırlatır**
>
> `int` tipinin en küçük değeri `int.MinValue` (-2147483648) olduğu için bunun pozitif karşılığı `int` sınırının dışında kalır. `Math.Abs` bu durumda sessizce sarmaz; `OverflowException` fırlatır:
>
> ```
> Console.WriteLine(Math.Abs(int.MinValue));
> ```
>
> Bu, Bölüm 42'de gördüğümüz sessiz taşmanın tersi bir durumdur: burada sonuç yanlış olmak yerine bir istisnayla kendini belli eder.

Asal sayı: karşılaştırmayı long ile yapmak

Asal kontrolünde bölenleri `i * i <= n` koşuluna kadar ararız. `i` yaklaşık 46341 olduğunda `i * i` `int` sınırını aşar ve sessizce sarar. Bu yüzden karşılaştırmayı `long` ile yapıyoruz:

▸ asalMi.cs

```
Console.Write("Sayı: ");
int n = Convert.ToInt32(Console.ReadLine());
bool asal = n >= 2;
for (int i = 2; (long)i * i <= n; i++)
{
    if (n % i == 0)
    {
        Console.WriteLine($"{i} bir böleni.");
        asal = false;
        break;
    }
}
string sonuc = asal ? "asaldır." : "asal değildir.";
Console.WriteLine($"{n} {sonuc}");
```

```
ÇIKTI
Sayı: 91
7 bir böleni.
91 asal değildir.
```

`asal` bir bayraktır: önce `true` kabul edilir, bir bölen bulununca `false` yapılır ve döngü `break` ile biter.

Faktöriyel: int, long ve checked

Faktöriyel çok hızlı büyür. `int` 12!'ye kadar, `long` 20!'ye kadar doğru tutabilir. C#'ta taşma hata vermeden sarar; yani `int` ile 20! hesaplamak yanlış bir sayı üretir:

▸ faktoriyel.cs

```
int n = 20;
long f = 1;
int fInt = 1;
for (int i = 1; i <= n; i++)
{
    f *= i;
    fInt *= i;
}
Console.WriteLine($"{n}! (long) = {f}");
Console.WriteLine($"{n}! (int) = {fInt}");
```

```
ÇIKTI
20! (long) = 2432902008176640000
20! (int) = -2102132736
```

Yanlış sonucu fark etmek yerine taşmayı yakalamak istiyorsak `checked` kullanırız (Bölüm 42). Taşma olduğunda program `OverflowException` fırlatır; bu istisnayı Bölüm 53'te ayrıntılı işleyeceğiz, şimdilik `catch` bloğunu yalnızca mesaj yazmak için kullanıyoruz:

faktoriyelChecked.cs

```
int sonuc = 1;
try
{
    for (int i = 1; i <= 13; i++)
    {
        sonuc = checked(sonuc * i);
    }
    Console.WriteLine(sonuc);
}
catch (OverflowException)
{
    Console.WriteLine("13! int'e sığmadı.");
}
```

```
ÇIKTI
13! int'e sığmadı.
```

13! sayısı 6227020800 eder ve `int` sınırını aşar. `checked` taşmayı `OverflowException` ile bildirdiği için `Console.WriteLine(sonuc)` satırına hiç ulaşılmaz; yalnızca `catch` bloğundaki mesaj yazılır.

Fibonacci ve EBOB: demet atama

Fibonacci'de iki değişkenin değerini birbirine geçirmek için geçici bir değişken yazardık (Bölüm 8). C#'ta demet (tuple) ile tek satırda yazabiliriz: sağ taraftaki demet önce tamamen hesaplanır, sonra soldakilere atanır.

fibonacciEbob.cs

```
int a = 0, b = 1;
for (int i = 1; i <= 12; i++)
{
    Console.Write($"{a} ");
    (a, b) = (b, a + b);
}
Console.WriteLine();

int p = 48, q = 18;
while (q != 0)
{
    (p, q) = (q, p % q);
}
Console.WriteLine($"EBOB(48, 18) = {p}");
```

```
ÇIKTI
0 1 1 2 3 5 8 13 21 34 55 89
EBOB(48, 18) = 6
```

Aynı kalıp Öklid algoritmasında da işe yarar: `(p, q) = (q, p % q);` satırı, kalanı bir sonraki adımın bölenine taşır.

> *Tarihçe · Demetler ve demet atama*
>
> Demetler ve demet atama C# 7.0 ile (2017) dile girdi. `foreach` ise C# 1.0'dan (2002) beri vardır. Bu bölümdeki kısa programlar üst düzey deyim biçimini kullanır; bu özellik C# 9.0 (2020) ile geldi. Örtük `using` satırları da .NET 6 proje şablonlarında varsayılan oldu.

En büyük ve en küçük değer

Bir dizinin en büyüğünü ve en küçüğünü `Math.Max` ve `Math.Min` ile bulmak, `if` yazmaktan daha kısadır. Başlangıç değerleri olarak `int.MaxValue` ve `int.MinValue` kullanırız; böylece her sayı en az bir kez karşılaştırılmış olur.

enBuyukKucuk.cs

```
int[] sicakliklar = { -4, 12, 7, -15, 3 };
int enBuyuk = int.MinValue;
int enKucuk = int.MaxValue;
foreach (int derece in sicakliklar)
{
    enBuyuk = Math.Max(enBuyuk, derece);
    enKucuk = Math.Min(enKucuk, derece);
}
Console.WriteLine($"En büyük: {enBuyuk}, en küçük: {enKucuk}");
```

```
ÇIKTI
En büyük: 12, en küçük: -15
```

Bu döngü, dizinin her elemanına bir kez bakar; `foreach` burada indeks gerektirmediği için en doğal seçimdir.

Bölüm Özeti

- `while` koşulu gövdeden önce, `do-while` koşulu gövdeden sonra denetler; `do-while` gövdeyi en az bir kez çalıştırır ve sonunda `;` ister.
- C#'ta döngü koşulu `bool` olmak zorundadır. `while (sayi)` gibi tam sayı koşulları `CS0029` hatasıyla reddedilir.
- `for (başlangıç; koşul; ilerleme)` tekrar sayısı belli döngülerde en okunaklı yapıdır. Parantezdeki sayaç yalnızca döngü içinde yaşar; dışarıda kullanmak `CS0103` hatasıdır.
- `for (...);` ve `while (...);` gövdeyi boşaltır. C# bunu hata değil `CS0642` uyarısıyla bildirir; her gövdeyi `{ }` içine almak bu tuzağı önler.
- `break` en içteki döngüyü bitirir, `continue` turun kalanını atlar. `while`'da ilerlemeyi gövdenin sonuna koymuşsak `continue` sonsuz döngü yaratabilir.
- `while (true)` ve `for (;;)` sonsuz döngüdür; çıkış `break` ile yapılır.
- C#'ta etiketli `break` yoktur; `break dis;` `CS1002` hatası verir. Dış döngüden çıkmak için bayrak ya da `goto` kullanılır.

- `foreach (tip ad in koleksiyon)` bir koleksiyonun her elemanını indeks tutmadan dolaşır. Dizi, `string` ve `List<T>` ile çalışır.
- `foreach` iterasyon değişkeni salt okunurdur (`CS1656`); elemanı değiştirmek için indeksli `for` kullanılır.
- Dolaşma sırasında koleksiyonun boyunu değiştirmek `InvalidOperationException` fırlatır. Silme gerekiyorsa `for` döngüsünü sondan başa doğru çalıştırırız.
- İç içe döngülerde dış döngü satırı, iç döngü sütunu belirler. `new string('*', n)` bir karakteri `n` kez üretir; `{x,4}` yazımı sayıyı sağa hizalar.
- `Math.Abs(int.MinValue)` `OverflowException` fırlatır. Asal kontrolünde `(long)i * i` ile karşılaştırma taşmadan korunur.
- `checked` taşmayı istisnayla bildirir. `int` 12!'ye, `long` 20!'ye kadar güvenilir; daha büyük faktöriyeller için başka bir veri tipi gerekir.
- `(a, b) = (b, a + b);` demet atamasıyla geçici değişkene gerek kalmadan Fibonacci ve Öklid adımları yazılır.
- En büyük ve en küçük değer aramada başlangıç değeri olarak `int.MaxValue` ve `int.MinValue` kullanılır; `Math.Max` ve `Math.Min` kodu kısaltır.

Alıştırmalar

1. `int x = 3; while (x) { x--; }` satırları için hangisi doğrudur?
   A) Üç kez döner ve biter.
   B) Sonsuz döngüye girer.
   C) Derleme hatası verir (CS0029).
   D) Hiç çalışmaz, yalnızca uyarı verir.
   E) Çalışma zamanında `InvalidCastException` fırlatır.

2. Aşağıdaki program ekrana ne yazdırır?

```
int n = 10;
do
{
    Console.Write(n);
} while (n < 5);
```

   A) Hiçbir şey yazdırmaz.
   B) `10`
   C) `5`
   D) Sonsuz döngüye girer.
   E) Derleme hatası verir.

3. Aşağıdakilerden hangisi derlenir? (`dizi` ifadesi `int[] dizi = { 1, 2 };` ile tanımlıdır.)
   A) `while (1) { break; }`
   B) `for (int i = 0; i < 3; i++) { } Console.WriteLine(i);`
   C) `for (;;) { break; }`
   D) `foreach (int s in dizi) { s++; }`
   E) `int k = 0; while (k) { k++; }`

4. Aşağıdaki program ekrana ne yazdırır?

```
int sayac = 0;
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++)
    {
        if (j == 2)
        {
            break;
        }
        sayac++;
    }
}
Console.WriteLine(sayac);
```

A) 4
B) 10
C) 7
D) 3
E) 6

5. Aşağıdaki program ekrana ne yazdırır?

```
int toplam = 0;
for (int i = 1; i <= 10; i++)
{
    if (i % 3 == 0)
    {
        continue;
    }
    if (i == 8)
    {
        break;
    }
    toplam += i;
}
Console.WriteLine(toplam);
```

6. Aşağıdaki kodda iki derleme hatası vardır. Hataları bulun, derleyicinin verdiği kodu yazın ve düzeltin.

```
int[] puanlar = { 10, 20, 30 };
foreach (int p in puanlar)
{
    p += 5;
}
int say = 0;
while (say)
{
    say++;
}
```

7. Aşağıdaki program ekrana ne yazdırır?

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (j > i)
        {
```

```
            break;
        }
        Console.Write($"{i}{j} ");
    }
}
Console.WriteLine();
```

8. `foreach (var c in "abc")` döngüsünde `c` değişkeninin tipi nedir?
   A) `string`
   B) `char`
   C) `int`
   D) `object`
   E) Derleme hatası verir.

9. Aşağıdaki program ekrana ne yazdırır?

```
int i = 0;
while (i < 5)
{
    i++;
    if (i % 2 == 0)
    {
        continue;
    }
    Console.Write(i);
}
```

10. `Console.WriteLine(Math.Abs(int.MinValue));` satırı ne yapar?
    A) `-2147483648` yazdırır.
    B) `2147483648` yazdırır.
    C) `0` yazdırır.
    D) `OverflowException` fırlatır.
    E) `int.MaxValue` yazdırır.

11. C#'ta `break dis;` yazılan bir döngü için ne olur?
    A) Yalnızca iç döngüden çıkılır.
    B) Dış döngüden çıkılır.
    C) Sonsuz döngüye girilir.
    D) Derlenir, yalnızca uyarı verilir.
    E) Derleme hatası verilir (CS1002).

12. Kullanıcıdan sayılar okuyun. 0 girilene kadar pozitif sayıların adedini ve toplamını yazdıran bir program yazın. Döngü için `while (true)` ve `break` kullanın. Negatif sayılar toplama katılmasın.

13. Kullanıcıdan bir `n` sayısı okuyup `n` satırlık ters üçgen yazdıran bir program yazın. `n = 4` için çıktı şöyle olmalıdır:

```
****
***
**
*
```

    Her satırı `new string('*', i)` ile üretin.

14. Bir metindeki sesli harfleri sayan bir program yazın. Sesli harfler `a`, `e`, `ı`, `i`, `o`, `ö`, `u`, `ü` olsun; büyük ve küçük harf farkı gözetilmesin. Metin `Merhaba Dünya` için çıktı `5` olmalıdır. Metni `foreach` ile dolaşın.

BÖLÜM 46

Metotlar

BU BÖLÜMDE

- Metot tanımının beş parçasını ve metotların hem ana kodun altında hem de sınıf içinde nasıl yazıldığını
- `void` ve değer döndüren metotların farkını, `return` kullanımını ve "her yol değer döndürmeli" kuralını
- Parametre ile argüman farkını, değerle geçirmeyi ve `ref`, `out`, `in` ile değişkenin kendisini geçirmeyi
- `params`, varsayılan değerli ve adlandırılmış parametreleri
- Aşırı yüklemeyi (overloading), ifade gövdeli metotları, yerel fonksiyonları ve demet (tuple) döndürmeyi
- `Math` sınıfını ve `Random` ile rastgele sayı üretmeyi

Şimdiye kadar bütün kodu dosyanın en üstüne, tek bir akış olarak yazdık. Program büyüdükçe iki sorun çıkar: aynı kod parçasını tekrar tekrar yazarız ve ana kod okunamayacak kadar uzar. Çözüm, bir işi yapan kod parçasına ad verip istediğimiz yerden çağırmaktır. Buna C#'ta **metot** (method) denir.

C bölümünde aynı kavramı **fonksiyon** olarak görmüştük (Bölüm 9), C++'ta aşırı yükleme, varsayılan parametre ve referanslarla genişlemişti (Bölüm 25 ve 28). C#'ın metotları bu iki mirasın birleşimi gibidir: C'nin sade söz dizimi, C++'ın aşırı yüklemesi ve varsayılan parametreleri, ayrıca `ref`, `out` ve adlandırılmış argüman gibi C#'a özgü eklentiler. Bu bölümde tanıdık kısımları hızlı geçip farkları vurgulayacağız.

C#'ta bütün kod bir tipin içinde yaşadığı için fonksiyonlara metot diyoruz. Ana kodun altına yazdıklarımız teknik olarak "yerel fonksiyondur"; farkı birazdan göreceğiz.

46.1 Neden Metot?

Üç neden öne çıkar. Birincisi **tekrarın önlenmesi**: bir kez yazarız, her yerden çağırırız; bir değişiklik gerekirse tek yeri düzeltiriz. İkincisi **okunabilirlik**: `Cizgi();` satırı ne yaptığını adıyla söyler, içindeki döngüyü okumamız gerekmez. Üçüncüsü **sınanabilirlik**: küçük bir metodu tek başına deneyebilir, hatayı nerede arayacağımızı bilebiliriz.

Aslında metotları baştan beri kullanıyoruz: `Console.WriteLine`, `int.Parse` ve `Math.Round` hazır metotlardır. Şimdi kendi metotlarımızı yazacağız.

▸ cizgiMetodu.cs

```
Cizgi();
Console.WriteLine("Aylık Rapor");
Cizgi();
Console.WriteLine("Satış: 1250");
Cizgi();

static void Cizgi()
```

```
{
    Console.WriteLine(new string('=', 20));
}
```

```
ÇIKTI
====================
Aylık Rapor
====================
Satış: 1250
====================
```

`Cizgi` metodunu bir kez tanımladık, üç kez çağırdık. Çizginin uzunluğunu ya da karakterini değiştirmek için tek bir satırı düzeltmemiz yeter.

46.2 Metodun Anatomisi

> **Tanım: Metot**
>
> Bir ada sahip, parametre alabilen ve değer döndürebilen kod bloğuna metot (method) denir. Metot çağrıldığında gövdesindeki deyimler çalışır; iş bitince akış çağrının yapıldığı yere döner.

Bir metot tanımı beş parçadan oluşur:

```
static int Topla(int a, int b)
{
    return a + b;
}
```

Parça	Örnekte	Anlamı
Değiştirici	`static`	Metodun nasıl bir üye olduğunu belirtir. Şimdilik her metodun başına yazacağız (nedeni sonraki başlıkta)
Dönüş tipi (return type)	`int`	Metodun geri verdiği değerin tipi. Değer vermiyorsa `void`
Ad	`Topla`	PascalCase ve genellikle fiil: `OrtalamaHesapla`, `Yazdir`, `AsalMi`
Parametre listesi	`(int a, int b)`	Metodun dışarıdan aldığı değerler; her biri tip + ad. Boş olabilir: `()`
Gövde (body)	`{ return a + b; }`	Metot çağrıldığında çalışan deyimler

C'deki tanımla karşılaştırdığımızda tek görünür fark `static` sözcüğüdür; C++'ta dönüş tipi ve parametre listesi de aynı yerdedir. Metot adları PascalCase, parametre ve yerel değişken adları camelCase yazılır (Bölüm 42).

Metodu çağırırken verdiğimiz değerlere **argüman** (argument), metodun tanımındaki değişkenlere **parametre** (parameter) denir. Çağrıda argümanlar parametrelere soldan sağa, sırayla bağlanır.

ilkMetot.cs

```
int a1 = DikdortgenAlani(3, 4);
Console.WriteLine(a1);
Console.WriteLine(DikdortgenAlani(10, 20) * 2);
Console.WriteLine(DikdortgenAlani(DikdortgenAlani(1, 2), 3));

static int DikdortgenAlani(int en, int boy)
```

```
{
    int alan = en * boy;
    return alan;
}
```

```
ÇIKTI
12
400
6
```

`DikdortgenAlani(3, 4)` çağrısında `en` 3, `boy` 4 olur; gövde çalışır ve `return` ile 12 çağrının yapıldığı yere döner. Yani `int a1 = DikdortgenAlani(3, 4);` satırı, metot bittikten sonra `int a1 = 12;` gibi davranır. Değer döndüren bir çağrıyı, o tipte bir değerin yazılabileceği her yerde kullanabiliriz: bir ifadenin içinde, `WriteLine`'ın argümanı olarak, hatta başka bir çağrının argümanı olarak. Son satırda önce içteki çağrı 2 döndürür, sonra `DikdortgenAlani(2, 3)` çalışır.

Ana kodun altında ve sınıfın içinde

Bölüm 41'de gördüğümüz iki program biçimi metotlar için de geçerlidir. Top-level statements kullandığımızda metotları ana kodun altına yazarız. Klasik biçimde ise metotlar `class Program` içinde, `Main`'in dışında yer alır:

klasikBicim.cs

```
class Program
{
    static int Topla(int a, int b)
    {
        return a + b;
    }

    static double Ortalama(int a, int b)
    {
        return (a + b) / 2.0;
    }

    static void Main()
    {
        Console.WriteLine(Topla(3, 4));
        Console.WriteLine(Ortalama(7, 8));
    }
}
```

```
ÇIKTI
7
7.5
```

Metotların `Main`'den önce ya da sonra yazılması fark etmez; derleyici dosyanın tamamını okuduktan sonra çağrıları çözer. C'deki gibi önceden bildirim (prototip) gerekmez (Bölüm 9, Bölüm 15). Bu, C ve C++'a göre rahatlatıcı bir farktır.

İki biçim arasında bilmemiz gereken ince farklar vardır. Ana kodun altındaki metotları derleyici, gizli `Main`'in içine **yerel fonksiyon** (local function) olarak yerleştirir. Sınıf içindeki metotlar ise gerçek sınıf üyeleridir:

	Ana kodun altı (yerel fonksiyon)	Sınıf içi metot
Yeri	Top-level deyimlerin altında	`class` içinde, `Main`'in dışında

	Ana kodun altı (yerel fonksiyon)	Sınıf içi metot
`public` , `private`	Yazılamaz (`CS0106`)	Yazılabilir; yazılmazsa `private`
`static`	Zorunlu değil; yazarsak ana koddaki değişkenleri göremez	`Main` 'den doğrudan çağıracaksak şart
Aynı ada iki metot (aşırı yükleme)	Olmaz (`CS0128`)	Olur
Ana koddaki değişkenlere erişim	Var (`static` değilse)	Yok

Sınıf içinde `static` yazmayı unutursak program derlenmez:

staticHata.cs

```
class Program
{
    int Kare(int n)
    {
        return n * n;
    }

    static void Main()
    {
        Console.WriteLine(Kare(6));
    }
}
```

DERLEYİCİ ÇIKTISI

```
staticHata.cs(10,27): error CS0120: An object reference is required for the non-static field,
method, or property 'Program.Kare(int)'
```

`static` olmayan metotlar bir nesneye aittir ve o nesne üzerinden çağrılır. Nesneleri ve `static` 'in tam anlamını Bölüm 49'da öğreneceğiz. O zamana kadar kural basittir: yazdığımız her metodun başına `static` koyalım.

> ◆ **Kritik Nokta**
>
> Ana kodun altındaki metotlar `Main` 'in yerel fonksiyonlarıdır; bu yüzden `public` yazılamaz ve aşırı yüklenemez. Aşırı yükleme gibi sınıf düzeyi özellikler gerektiğinde metotları bir `class` içine alırız. Kitabın örneklerinde yardımcı metotları çoğunlukla ana kodun altına yazacağız; aşırı yüklemede sınıf kullanacağız.

Çağrı yığını

Bir metot başka bir metodu çağırabilir. Çalışma zamanı, o anda hangi metotların tamamlanmayı beklediğini **çağrı yığınında** (call stack) tutar. Yeni çağrı yığının tepesine bir çerçeve (frame) ekler; metot bitince çerçeve atılır ve akış bir önceki çerçeveye döner. Bu, C'de gördüğümüz yığın mantığının aynısıdır (Bölüm 9 ve Bölüm 18).

cagriYigini.cs

```
Console.WriteLine("Ana kod başladı");
Birinci();
Console.WriteLine("Ana kod bitti");
```

```
static void Birinci()
{
    Console.WriteLine("  Birinci başladı");
    Ikinci();
    Console.WriteLine("  Birinci bitti");
}

static void Ikinci()
{
    Console.WriteLine("    İkinci çalışıyor");
}
```

```
ÇIKTI
Ana kod başladı
  Birinci başladı
    İkinci çalışıyor
  Birinci bitti
Ana kod bitti
```

`Ikinci` en son çağrıldığı için ilk biten odur: son giren ilk çıkar (LIFO). Bir metodun çağrısı bitmeden onu tekrar çağıran özyinelemeli (recursive) metotlarda bu yığın büyür; konuyu Bölüm 58'de işleyeceğiz.

46.3 void ve Değer Döndüren Metotlar

Metotlar iki gruba ayrılır. `void` metotlar bir iş yapar (ekrana yazmak gibi) ama değer vermez; çağrıları tek başına bir deyimdir. Değer döndüren metotlar bir hesap yapar ve sonucu `return` ile çağırana verir. Dönüş tipi, döndürülen değerin tipidir.

donusTipleri.cs

```
Selamla("Ece");
int k = Kare(5);
Kare(9);                            // dönen değer kullanılmadı
Console.WriteLine($"{k} {Kare(k)}");
Console.WriteLine(Ortalama(7, 8));

static void Selamla(string ad)
{
    Console.WriteLine($"Merhaba {ad}!");
}

static int Kare(int n)
{
    return n * n;
}

static double Ortalama(int a, int b)
{
    return (a + b) / 2.0;
}
```

```
ÇIKTI
Merhaba Ece!
25 625
7.5
```

`Kare(9);` satırı derlenir ve çalışır; ama dönen 81 hiçbir yerde kullanılmadığı için kaybolur. C# bunu hata saymaz, uyarı da vermez. `Ortalama` içinde `2.0` yerine `2` yazsaydık tam sayı bölmesi yapılırdı; dönüş tipinin `double` olması bölmeyi ondalıklı yapmaz.

`return` ifadesinin tipi, dönüş tipine uymalıdır. Genişleyen dönüşümler otomatiktir (`double` dönen metotta `return 5;` yazılabilir); daralan dönüşümler yapılmaz (`int` dönen metotta `return 2.5;` derleme hatasıdır). Bu kurallar Bölüm 42'deki atama kurallarının aynısıdır.

> **▲ Sık Yapılan Hata**
>
> `void` bir metodun "değerini" kullanamayız. `int x = Selamla("Can");` yazarsak derleyici `CS0029` hatası verir (`Cannot implicitly convert type 'void' to 'int'`). Değer üretmeyen çağrıyı ifade içine koymaya çalıştığımızda bu hatayı görürüz.

Bir hesap yapan metodun sonucu ekrana yazması yerine `return` ile döndürmesi çoğu zaman daha iyidir: döndürülen değeri yazdırabilir, saklayabilir ya da başka bir hesapta kullanabiliriz.

return ile erken çıkış

`return` metodu o anda bitirir. `void` metotlarda dönüş değeri yazılmaz (`return;`), akışı erken kesmek için kullanılır. Birden çok `return`, `else` zincirlerini gereksiz kılar:

▸ erkenReturn.cs

```
Console.WriteLine(Degerlendir(92));
Console.WriteLine(Degerlendir(55));
Console.WriteLine(Degerlendir(-4));
BolumuYaz(17, 5);
BolumuYaz(17, 0);

static string Degerlendir(int puan)
{
    if (puan < 0 || puan > 100)
    {
        return "geçersiz puan";
    }
    if (puan >= 60)
    {
        return "geçti";
    }
    return "kaldı";
}

static void BolumuYaz(int bolunen, int bolen)
{
    if (bolen == 0)
    {
        Console.WriteLine("Sıfıra bölünemez.");
        return;                         // bölme satırına hiç gelinmez
    }
    Console.WriteLine($"{bolunen} / {bolen} = {bolunen / bolen}");
}
```

```
ÇIKTI
geçti
kaldı
geçersiz puan
17 / 5 = 3
Sıfıra bölünemez.
```

Önce geçersiz durumları eleyip erken dönmek, asıl işi girintisiz bırakır. Bu desene "koruma deyimi" (guard clause) denir ve okunabilirliği artırır. `return`'den hemen sonra, aynı blokta bir deyim yazarsak derleyici ona asla ulaşılamayacağını söyleyen `CS0162` uyarısını verir.

Her yol değer döndürmeli

Dönüş tipi `void` olmayan bir metotta, derleyici metodun her olası yoldan bir değer döndürdüğünden emin olmak ister. Bir yol bile `return`'süz bitiyorsa program derlenmez. Derleyici koşulların değerini hesaplamaz, yalnızca akışın yapısına bakar:

▸ eksikReturn.cs

```
Console.WriteLine(Isaret(-3));

static string Isaret(int n)
{
    if (n >= 0)
    {
        return "pozitif veya sıfır";
    }
    if (n < 0)
    {
        return "negatif";
    }
}
```

```
DERLEYİCİ ÇIKTISI
eksikReturn.cs(3,15): error CS0161: 'Isaret(int)': not all code paths return a value
```

İki koşuldan birinin mutlaka doğru olduğunu biz biliriz; derleyici ise iki ayrı `if` görür ve "ikisi de yanlışsa ne dönecek?" diye sorar. Çözüm, ikinci `if`'i kaldırıp metodu düz bir `return "negatif";` ile bitirmek ya da `if-else` kullanmaktır. C'de aynı durum yalnızca derleyici uyarısıdır ve değersiz yoldan dönülen sonucu kullanmak tanımsız davranıştır; C#'ta bu durum derleme hatasıdır.

> **▲ Sık Yapılan Hata**
>
> Döngünün içindeki `return` tek başına yetmez; döngü hiç dönmeyebilir. `for` ya da `while` bittikten sonra da bir `return` yazmak gerekir. `while (true)` gibi sonsuz bir döngüden çıkış yalnızca `return` iledir; derleyici bunu bildiği için döngüden sonra `return` istemez.

46.4 Parametre Geçirme: Varsayılan Davranış

C'de her argüman kopyalanır; bir değişkeni değiştirmek istiyorsak işaretçi geçiririz (Bölüm 9 ve Bölüm 12). C++ buna referansı ekledi (Bölüm 25). C# de varsayılan olarak **değerle geçirir**: metot, argümanın bir kopyasıyla çalışır. Ancak "kopyalanan şey" tipe göre değişir, ve bu Bölüm 42'de öğrendiğimiz değer tipi/referans tipi ayrımının doğrudan sonucudur.

Tanım: Değerle geçirme (pass by value)

Parametreye argümanın değerinin bir kopyası verilir. Değer tipinde kopyalanan, verinin kendisidir; referans tipinde kopyalanan, nesneye giden bağlantıdır. Metot içinde parametreye yeni bir değer atamak çağıran koddaki değişkeni etkilemez.

degerleGecirme.cs

```
int sayi = 10;
Arttir(sayi);
Console.WriteLine($"Çağrıdan sonra: {sayi}");

int[] dizi = { 1, 2, 3 };
DiziyiDegistir(dizi);
Console.WriteLine($"{dizi[0]} {dizi[1]} {dizi[2]} (uzunluk {dizi.Length})");

static void Arttir(int s)
{
    s++;
    Console.WriteLine($"Metotta: {s}");
}

static void DiziyiDegistir(int[] d)
{
    d[0] = 99;                  // nesnenin içeriği değişir
    d = new int[] { 7, 7, 7, 7 };  // yalnızca yerel bağlantı değişir
    Console.WriteLine($"Metotta yeni dizi: {d.Length} eleman");
}
```

```
ÇIKTI
Metotta: 11
Çağrıdan sonra: 10
Metotta yeni dizi: 4 eleman
99 2 3 (uzunluk 3)
```

`Arttir` içinde `s` artırıldı ama `sayi` değişmedi: `int` kopyalandı. `DiziyiDegistir` içinde ise iki ayrı olay var. `d[0] = 99` ataması, `dizi` ile `d` aynı nesneyi gösterdiği için dışarıdan da görünür. Ama `d = new int[...]` ataması yalnızca metodun yerel bağlantısını yeni bir nesneye çevirir; `dizi` hâlâ eski nesneyi gösterir.

◆ Kritik Nokta

Referans tipinde "değerle geçirme", nesnenin kopyalanması demek değildir; yalnızca bağlantı kopyalanır. Bu yüzden metot, aldığı nesnenin **içeriğini** değiştirebilir ama çağıranın değişkenini **başka bir nesneye bağlayamaz**. Bunu yapmak istiyorsak `ref` gerekir. Bu ayrım C'deki `int *p` ile `int **p` farkına benzer: `p` kopyalanır, ama `*p` ortaktır.

46.5 ref: Değişkenin Kendisini Geçirmek

`ref` parametresi, argümanın bir kopyasını değil değişkenin **kendisini** (adresini) geçirir. C++'taki `int&` ile aynı işi görür; C'de bunu `int *` ve çağrıda `&x` ile yapardık. C# farkı, `ref` sözcüğünün çağrıda da yazılmasını zorunlu kılmasıdır. Böylece çağrı satırına bakarak hangi değişkenlerin değişebileceğini görürüz:

refTakas.cs

```
int x = 3, y = 8;
Takas(ref x, ref y);
Console.WriteLine($"x={x} y={y}");

int[] dizi = { 1, 2, 3 };
Sifirla(ref dizi);
Console.WriteLine($"Dizi uzunluğu: {dizi.Length}");

static void Takas(ref int a, ref int b)
{
    int gecici = a;
    a = b;
    b = gecici;
}

static void Sifirla(ref int[] d)
{
    d = new int[0];               // çağıranın değişkeni değişir
}
```

ÇIKTI

```
x=8 y=3
Dizi uzunluğu: 0
```

`Takas` iki değişkenin değerini çağıranın tarafında değiştirdi. `Sifirla` ise önceki programda yapamadığımız şeyi yaptı: `dizi` değişkenini başka bir nesneye bağladı. Parametre `ref` olduğu için `d` ile `dizi` aynı değişkenin iki adıdır.

`ref` kullanırken iki kural vardır. Birincisi, argüman **değeri atanmış bir değişken** olmalıdır; sabit ya da ifade geçirilemez. İkincisi, `ref` çağrıda da yazılmalıdır:

refHata.cs

```
int x = 3, y = 8;
Takas(x, y);
Takas(ref x, ref 5);

static void Takas(ref int a, ref int b)
{
    (a, b) = (b, a);
}
```

DERLEYİCİ ÇIKTISI

```
refHata.cs(2,7): error CS1620: Argument 1 must be passed with the 'ref' keyword
refHata.cs(2,10): error CS1620: Argument 2 must be passed with the 'ref' keyword
refHata.cs(3,18): error CS1510: A ref or out value must be an assignable variable
```

`Takas` içindeki `(a, b) = (b, a);` satırı, demetle yer değiştirme yapar ve geçici değişkeni ortadan kaldırır. Demetleri bu bölümün ilerisinde göreceğiz.

46.6 out: Değer Döndürmenin İkinci Yolu

Bir metot `return` ile yalnızca bir değer döndürebilir. Birden fazla sonuç gerekiyorsa C'de çıkış işaretçileri, C++'ta referans parametreler kullanırdık. C#'ın karşılığı `out` parametresidir. `ref` gibi değişkenin kendisini geçirir;

farkı, **değerinin çağrıdan önce atanmış olması gerekmez**, ama **metodun dönmeden önce ona mutlaka değer atamasıdır**.

> **Tanım: out parametresi**
>
> Metodun çağırana ek bir sonuç taşımasını sağlayan parametredir. Çağrıda `out` sözcüğü yazılır; metot dönmeden önce parametreye değer atamak zorundadır. Metot içinde, atama yapılana kadar parametre okunamaz.

outBolum.cs

```
BolVeKalan(47, 5, out int bolum, out int kalan);
Console.WriteLine($"47 / 5 = {bolum} kalan {kalan}");

BolVeKalan(20, 6, out int b2, out _);      // kalanı istemiyoruz
Console.WriteLine($"Bölüm: {b2}");

static void BolVeKalan(int bolunen, int bolen, out int bolum, out int kalan)
{
    bolum = bolunen / bolen;
    kalan = bolunen % bolen;
}
```

ÇIKTI

```
47 / 5 = 9 kalan 2
Bölüm: 3
```

İlk çağrıda `out int bolum` yazımıyla değişken tam çağrının içinde tanımlanıyor; önceden `int bolum;` yazmamız gerekmiyor. İkinci çağrıdaki `out _` bir **atma** (discard) değişkenidir: sonucu istemediğimizi belirtir ve bir ad oluşturmaz.

Metot, `out` parametresine değer atamadan dönmeye çalışırsa program derlenmez:

outHata.cs

```
static void Doldur(out int sonuc, bool kosul)
{
    if (kosul)
    {
        sonuc = 1;
    }
}
```

DERLEYİCİ ÇIKTISI

```
outHata.cs(1,13): error CS0177: The out parameter 'sonuc' must be assigned to before control
leaves the current method
```

Derleyici hatayı metodun başlığında bildirir: `kosul` yanlışsa `sonuc`'a atama yapılmadan metodun bittiği bir yol vardır. Bu kural `out`'u güvenli kılar: çağıran, `out` ile geçirdiği değişkenin dönüşte dolu olduğundan emin olabilir.

TryParse deseni

`out`'un en yaygın kullanımı, başarısız olabilecek işlemlerde "sonuç + başarı bilgisi" döndürmektir. Bölüm 42'de gördüğümüz `int.TryParse` tam olarak budur: dönüş değeri `bool` olarak başarıyı bildirir, çevrilen sayı `out` parametresiyle gelir. Kendi `Try...` metodumuzu aynı kalıpla yazabiliriz:

tryDeseni.cs

```
foreach (double d in new double[] { 16, -4, 2.25 })
{
    if (KareKokBul(d, out double kok))
    {
        Console.WriteLine($"{d} sayısının karekökü {kok}");
    }
    else
    {
        Console.WriteLine($"{d} için karekök yok");
    }
}

static bool KareKokBul(double sayi, out double kok)
{
    if (sayi < 0)
    {
        kok = 0;                    // başarısızlıkta da atama zorunlu
        return false;
    }
    kok = Math.Sqrt(sayi);
    return true;
}
```

```
ÇIKTI
16 sayısının karekökü 4
-4 için karekök yok
2.25 sayısının karekökü 1.5
```

Başarısız yolda bile `kok`'a bir değer verdik; kural böyle gerektiriyor. Bu desenin avantajı, başarısızlığı istisnayla (Bölüm 53) değil dönüş değeriyle bildirmesidir; sık başarısız olabilecek işlerde çok daha ucuzdur.

46.7 in: Salt Okunur Referans

`in` parametresi de `ref` gibi argümanı kopyalamadan geçirir, ama metodun parametreyi **değiştirmesini yasaklar**. C++'taki `const T&` ile aynı amaca hizmet eder: büyük bir değer tipini (örneğin birçok alanı olan bir `struct`) kopyalamanın maliyetinden kaçınmak. `int` gibi küçük tiplerde `in` hiçbir kazanç sağlamaz, hatta yavaşlatabilir.

inParametre.cs

```
Vektor v = new Vektor { X = 3, Y = 4, Z = 12 };
Console.WriteLine(Uzunluk(in v));
Console.WriteLine(Uzunluk(v));          // çağrıda in yazmak isteğe bağlı

static double Uzunluk(in Vektor v)
{
    return Math.Sqrt(v.X * v.X + v.Y * v.Y + v.Z * v.Z);
}

struct Vektor
{
    public double X, Y, Z;
}
```

```
ÇIKTI
13
13
```

`in` parametresine atama yapmaya çalışırsak derleme hatası alırız:

inHata.cs

```
static void Olcekle(in Vektor v)
{
    v.X = 0;
}

struct Vektor
{
    public double X, Y, Z;
}
```

```
DERLEYİCİ ÇIKTISI
inHata.cs(3,5): error CS8332: Cannot assign to a member of variable 'v' or use it as the right
hand side of a ref assignment because it is a readonly variable
```

Üç anahtar sözcüğü karşılaştıralım:

	`ref`	`out`	`in`
Çağrıdan önce değer atanmış olmalı mı?	Evet	Hayır	Evet
Metot içinde okunabilir mi?	Evet	Atamadan sonra	Evet
Metot içinde değiştirilebilir mi?	Evet	Evet (atama zorunlu)	Hayır
Çağrıda sözcük yazılır mı?	Zorunlu	Zorunlu	İsteğe bağlı
Tipik kullanım	Çağıranın değişkenini güncelleme	Ek sonuç döndürme	Büyük `struct`'ı kopyasız, salt okunur geçirme

> **İpucu**
>
> Birden fazla sonuç döndürmek gerektiğinde `out` yerine demet (tuple) döndürmek çoğu zaman daha okunaklıdır; bu bölümün ilerisinde göreceğiz. `out` en çok `TryParse` gibi "başarı + sonuç" desenlerinde ve yerleşik kütüphane metotlarında karşımıza çıkar. `ref` ise değer tipli bir değişkeni yerinde güncellemek gerektiğinde kullanılır.

46.8 params: Değişken Sayıda Argüman

`Console.WriteLine("a", "b", "c")` gibi bazı metotlar istenen sayıda argüman alır. Kendi metodumuzu bu biçimde yazmak için son parametreyi `params` ile dizi olarak tanımlarız. C'de benzer iş `...` ve `stdarg.h` ile yapılır ve tip denetimi yoktur; C++'ta `std::initializer_list` ya da değişken sayılı şablonlar kullanılır. C#'ta `params` tip güvenlidir ve çok basittir.

paramsToplam.cs

```
Console.WriteLine(Topla());
Console.WriteLine(Topla(5));
Console.WriteLine(Topla(1, 2, 3, 4));
```

```
int[] notlar = { 70, 80, 90 };
Console.WriteLine(Topla(notlar));            // hazır dizi de verilebilir
Yazdir("Sonuçlar", 10, 20, 30);

static int Topla(params int[] sayilar)
{
    int toplam = 0;
    foreach (int s in sayilar)
    {
        toplam += s;
    }
    return toplam;
}

static void Yazdir(string baslik, params int[] degerler)
{
    Console.WriteLine($"{baslik}: {string.Join(", ", degerler)}");
}
```

```
ÇIKTI
0
5
10
240
Sonuçlar: 10, 20, 30
```

Çağrıda verdiğimiz argümanlar derleyici tarafından bir diziye toplanır; hiç argüman verilmezse boş dizi gelir. İstersek hazır bir diziyi de doğrudan geçirebiliriz. Kurallar: `params` parametresi listenin **son** parametresi olmalıdır ve bir metotta yalnızca bir tane olabilir. Tipi tek boyutlu dizi olmalıdır (daha yeni sürümlerde bazı koleksiyon tipleri de desteklenir, ama bu kitapta dizi kullanacağız).

46.9 Varsayılan Değerli ve Adlandırılmış Parametreler

C'de varsayılan parametre yoktur; C++'ta vardır (Bölüm 25). C# de destekler ve ek olarak argümanı **adıyla** vermeye izin verir.

Parametreye `= değer` ekleyerek onu isteğe bağlı yaparız. Varsayılan değer, derleme zamanında bilinen bir sabit olmalıdır. İsteğe bağlı parametreler, zorunluların **sağında** yer alır.

▸ varsayilanParametre.cs

```
Console.WriteLine(Fiyatla(100));
Console.WriteLine(Fiyatla(100, 10));
Console.WriteLine(Fiyatla(100, 10, "USD"));
Console.WriteLine(Fiyatla(100, para: "EUR"));        // ortadakini atladık
Console.WriteLine(Fiyatla(para: "TL", tutar: 250, indirim: 20));

static string Fiyatla(decimal tutar, int indirim = 0, string para = "TL")
{
    decimal net = tutar - tutar * indirim / 100;
    return $"{net} {para}";
}
```

```
ÇIKTI
100 TL
90 TL
90 USD
100 EUR
200 TL
```

`para: "EUR"` biçimi **adlandırılmış argümandır** (named argument). Adlandırılmış argümanlarla sırayı değiştirebilir ve ortadaki isteğe bağlı parametreyi atlayabiliriz. C++'ta bu mümkün değildir: orta parametreyi atlamak için ondan önceki bütün parametreleri yazmak gerekirdi (Bölüm 25).

Çağrı bir bakışta anlaşılır olsun diye adlandırılmış argümanları, özellikle `true`/`false` gibi anlamı belirsiz değerlerde tercih ederiz: `Kaydet(dosya, true, false)` yerine `Kaydet(dosya, ustuneYaz: true, yedekle: false)`.

Zorunlu bir parametrenin isteğe bağlıdan sonra gelmesi hatadır:

varsayilanHata.cs

```
static void Yaz(int a = 1, int b)
{
}
```

```
DERLEYİCİ ÇIKTISI

varsayilanHata.cs(1,33): error CS1737: Optional parameters must appear after all required
parameters
```

> **▲ Sık Yapılan Hata**
>
> Varsayılan değerler **çağıran kodun içine** derlenir. Başka bir kütüphanedeki metodun varsayılan değeri sonradan değişirse, çağıran kod yeniden derlenmedikçe eski değeri kullanmaya devam eder. Kütüphane yazarken bu yüzden varsayılan değerlere dikkat etmek, çoğu zaman aşırı yüklemeyi (aşağıda) tercih etmek gerekir.

46.10 Aşırı Yükleme

Bazen aynı işi farklı tiplerle ya da farklı sayıda argümanla yapmak isteriz: iki `int`'in toplamı, iki `double`'ın toplamı, üç sayının toplamı. C'de her biri için ayrı ad bulmak gerekirdi (`topla_int`, `topla_double`). C++ ve C# aynı ada sahip birden çok metoda izin verir.

> **Tanım: Aşırı yükleme (overloading)**
>
> Aynı sınıfta, aynı adı taşıyan ama parametre listeleri (sayı, tip ya da sıra bakımından) farklı birden çok metot tanımlamaya aşırı yükleme denir. Hangi metodun çalışacağını derleyici, çağrıdaki argümanlara bakarak derleme zamanında seçer.

Ana kodun altındaki yerel fonksiyonlar aşırı yüklenemediği için bu örnekte metotları bir sınıfın içine alıyoruz. Sınıfları Bölüm 49'da ayrıntılı işleyeceğiz; şimdilik `static class`'ı, ilgili metotları bir ad altında toplayan bir kap olarak düşünelim. `Math` ve `Console` da böyle sınıflardır ve metotlarını `Sinif.Metot()` biçiminde çağırırız.

asiriYukleme.cs

```
Console.WriteLine(Hesap.Topla(2, 3));
Console.WriteLine(Hesap.Topla(2.5, 3.5));
Console.WriteLine(Hesap.Topla(1, 2, 3));
Console.WriteLine(Hesap.Topla("Merhaba, ", "dünya"));
Console.WriteLine(Hesap.Topla(2, 3.5));      // int -> double dönüşümü

static class Hesap
{
    public static int Topla(int a, int b)
    {
        Console.Write("[int,int] ");
        return a + b;
    }

    public static double Topla(double a, double b)
    {
        Console.Write("[double,double] ");
        return a + b;
    }

    public static int Topla(int a, int b, int c)
    {
        Console.Write("[int,int,int] ");
        return a + b + c;
    }

    public static string Topla(string a, string b)
    {
        Console.Write("[string,string] ");
        return a + b;
    }
}
```

ÇIKTI

```
[int,int] 5
[double,double] 6
[int,int,int] 6
[string,string] Merhaba, dünya
[double,double] 5.5
```

Son çağrıda `(int, double)` biçiminde bir tanım yoktur. Derleyici, argümanlardan hiçbirini kaybetmeden uygulanabilen adaylara bakar: `Topla(double, double)` uygundur, çünkü `int` örtük olarak `double`'a dönüşür. `Topla(int, int)` uygun değildir, çünkü `double` örtük olarak `int`'e dönüşmez. Birden çok aday uygunsa, argüman tiplerine **en yakın** olan seçilir. Bu seçim kuralları C++'takine benzer ama daha basittir (Bölüm 25).

Aşırı yüklemeyi ayıran şey metodun **imzasıdır** (signature): ad, parametre sayısı, tipleri ve `ref` / `out` / `in` değiştiricileri. **Dönüş tipi imzaya dahil değildir.** Yalnızca dönüş tipi farklı iki metot tanımlayamayız:

ayniImza.cs

```
static class Hesap
{
    public static int Kare(int a) => a * a;
    public static double Kare(int a) => a * a;
}
```

```
DERLEYİCİ ÇIKTISI

ayniImza.cs(4,26): error CS0111: Type 'Hesap' already defines a member called 'Kare' with the
same parameter types
```

Neden? Çağrıda `Hesap.Kare(3);` yazdığımızda derleyici, dönüş değerine bakmadan hangisini seçeceğini bilemez. Bir başka sorun, iki aday da eşit derecede uygun olduğunda çıkar:

▸ **belirsizCagri.cs**

```
Console.WriteLine(Hesap.Yaz(1, 2));

static class Hesap
{
    public static string Yaz(int a, double b) => "int,double";
    public static string Yaz(double a, int b) => "double,int";
}
```

```
DERLEYİCİ ÇIKTISI

belirsizCagri.cs(1,25): error CS0121: The call is ambiguous between the following methods or
properties: 'Hesap.Yaz(int, double)' and 'Hesap.Yaz(double, int)'
```

`(1, 2)` her iki adaya da dönüşümle uyar ve hiçbiri diğerinden daha iyi değildir. Çağrıyı `Hesap.Yaz(1, 2.0)` biçiminde yazarak ya da bir argümanı `(double)` ile dönüştürerek belirsizliği gideririz.

> **▲ Sık Yapılan Hata**
>
> Aşırı yükleme ile varsayılan parametre birlikte kullanıldığında beklenmeyen seçimler olabilir: `F(int a)` ve `F(int a, int b = 0)` varken `F(1)` çağrısı, varsayılan değer kullanılmayan ilk metodu seçer. Aynı işi yapan metotlar için ya aşırı yüklemeyi ya da varsayılan parametreyi seçelim, ikisini birden değil.

46.11 İfade Gövdeli Metotlar

Gövdesi tek bir ifadeden oluşan metodu `=>` ile kısa yazabiliriz. `return` ve süslü parantezler kalkar; ifade doğrudan okun sağına yazılır:

▸ **ifadeGovdeli.cs**

```
Console.WriteLine(Carp(6, 7));
Console.WriteLine(CiftMi(10));
Console.WriteLine(Mutlak(-8));
Selamla("Ece");

static int Carp(int a, int b) => a * b;
static bool CiftMi(int n) => n % 2 == 0;
static int Mutlak(int n) => n < 0 ? -n : n;
static void Selamla(string ad) => Console.WriteLine($"Merhaba {ad}!");
```

```
ÇIKTI
42
True
8
Merhaba Ece!
```

`=>` yalnızca yazım kısaltmasıdır; derlenen kod aynıdır. `void` metotlarda da kullanılabilir (son satır). Ok, tek ifadeyle sınırlıdır; birden çok deyim gerekiyorsa normal gövdeyi kullanırız. Bu `=>` simgesinin lambda ifadelerindekiyle aynı olması rastlantı değildir; lambdaları Bölüm 55'te göreceğiz.

46.12 Yerel Fonksiyonlar

Bir metodun içinde, yalnızca o metodun kullandığı küçük bir yardımcıya ihtiyaç duyabiliriz. C ve C++'ta fonksiyon içinde fonksiyon tanımlanamaz; C#'ta tanımlanabilir.

> **Tanım: Yerel fonksiyon (local function)**
>
> Başka bir metodun gövdesi içinde tanımlanan ve yalnızca o metodun içinden çağrılabilen metottur. Dışarıdaki metodun parametrelerini ve yerel değişkenlerini kullanabilir.

yerelFonksiyon.cs

```
Console.WriteLine(OrtalamaFarki(new int[] { 4, 8, 6 }));

static double OrtalamaFarki(int[] sayilar)
{
    double ort = 0;
    foreach (int s in sayilar)
    {
        ort += s;
    }
    ort /= sayilar.Length;

    double toplamFark = 0;
    foreach (int s in sayilar)
    {
        toplamFark += Uzaklik(s);
    }
    return toplamFark / sayilar.Length;

    double Uzaklik(int deger)          // yerel fonksiyon, ort'u kullanır
    {
        return Math.Abs(deger - ort);
    }
}
```

```
ÇIKTI
1.3333333333333333
```

`Uzaklik` yalnızca `OrtalamaFarki` içinde görünür ve dıştaki `ort` değişkenine doğrudan erişir. Yerel fonksiyonu, kullanıldığı yerden önce ya da sonra tanımlayabiliriz. Ana kodun altına yazdığımız metotlar da aynı mekanizmayla çalışır; bu yüzden onlar da ana koddaki yerel değişkenleri görebilir.

Yakalama istemiyorsak yerel fonksiyonu `static` yaparız. Bu durumda dış değişkene erişim derleme hatasıdır. Böylece fonksiyonun yalnızca parametreleriyle çalıştığını derleyiciye garanti ettirmiş oluruz:

staticYerel.cs

```
int oran = 3;
Console.WriteLine(Kat(5));

static int Kat(int x) => x * oran;
```

```
DERLEYİCİ ÇIKTISI

staticYerel.cs(4,30): error CS8421: A static local function cannot contain a reference to
'oran'.
```

46.13 Birden Çok Değer Döndürmek: Demetler

`out` parametreleri birden çok sonuç döndürmenin bir yoludur ama kullanımı hantaldır: önce değişkenleri tanımlarız, sonra çağırırız, sonra okuruz. C'de bu iş için bir `struct` tanımlamak, C++'ta `std::pair` ya da `std::tuple` kullanmak gerekirdi (Bölüm 25). C# bunun için **demet** (tuple) söz dizimini dile yerleştirmiştir.

> **Tanım: Demet (tuple)**
>
> Birden çok değeri tek bir değer olarak taşıyan hafif bir yapıdır. Parantez içinde, virgülle ayrılmış tiplerle yazılır: `(int, double)`. Elemanlara isim verilebilir: `(int Enk, int Enb)`.

demetDondur.cs

```
int[] notlar = { 72, 45, 88, 91, 60 };

var sonuc = EnkEnb(notlar);
Console.WriteLine($"En küçük {sonuc.Enk}, en büyük {sonuc.Enb}");

var (enk, enb) = EnkEnb(notlar);          // demeti açmak (deconstruction)
Console.WriteLine($"Aralık: {enb - enk}");

(int bolum, int kalan) = BolVeKalan(47, 5);
Console.WriteLine($"{bolum} {kalan}");

var (_, sadeceEnb) = EnkEnb(new int[] { 3, 9, 4 });
Console.WriteLine(sadeceEnb);

static (int Enk, int Enb) EnkEnb(int[] dizi)
{
    int enk = dizi[0], enb = dizi[0];
    foreach (int d in dizi)
    {
        if (d < enk) enk = d;
        if (d > enb) enb = d;
    }
    return (enk, enb);
}

static (int, int) BolVeKalan(int a, int b) => (a / b, a % b);
```

```
ÇIKTI
En küçük 45, en büyük 91
Aralık: 46
9 2
9
```

Dönüş tipindeki `(int Enk, int Enb)` yazımı, elemanlara `sonuc.Enk` ve `sonuc.Enb` olarak erişmemizi sağlar. İsim verilmezse elemanlara `Item1`, `Item2` diye ulaşılır; bu okunaksızdır, bu yüzden dönüş tipinde adları vermeyi tercih ederiz. `var (a, b) = ...` yazımı demeti tek satırda değişkenlere açar; ilgilenmediğimiz elemanı `_` ile atlayabiliriz. Başta kullandığımız `(a, b) = (b, a);` biçimi de aynı yapıdan gelir: sağ taraf demet olarak kurulur, sonra sol tarafa açılır.

Demetler bir değer tipidir (`System.ValueTuple`) ve öbekte yer ayırmaz, bu yüzden hızlıdır. Dönen değerin birkaç parçası geçici olarak bir arada taşınacaksa demet idealdir. Sonuç birçok yerde kullanılacaksa, adı ve belgelenmiş alanları olan bir `struct`, `record` ya da `class` daha uygundur (Bölüm 49 ve Bölüm 52).

46.14 Math Sınıfı

`Math`, `System` ad alanında bulunan ve matematik işlevlerini toplayan statik bir sınıftır. C'deki `<math.h>` ve C++'taki `<cmath>` karşılığıdır; farkı, ek bir başlık dosyası ya da bağlama seçeneği (`-lm`) gerektirmemesidir.

Metot / sabit	Anlamı	Örnek
`Math.Abs(x)`	Mutlak değer	`Math.Abs(-7.5)` → 7.5
`Math.Max(a, b)`, `Math.Min(a, b)`	Büyük / küçük olan	`Math.Max(3, 9)` → 9
`Math.Pow(x, y)`	x üzeri y (`double`)	`Math.Pow(2, 10)` → 1024
`Math.Sqrt(x)`	Karekök	`Math.Sqrt(81)` → 9
`Math.Floor(x)`, `Math.Ceiling(x)`	Aşağı / yukarı tam sayıya yuvarlama	`Math.Floor(-2.5)` → -3
`Math.Round(x)`, `Math.Round(x, n)`	En yakına yuvarlama, `n` basamak	`Math.Round(3.14159, 2)` → 3.14
`Math.Truncate(x)`	Ondalık kısmı atma	`Math.Truncate(-2.7)` → -2
`Math.Sign(x)`	İşaret: -1, 0 ya da 1	`Math.Sign(-9)` → -1
`Math.Clamp(x, alt, ust)`	Değeri aralıkta tutma	`Math.Clamp(15, 0, 10)` → 10
`Math.Sin`, `Cos`, `Tan`	Trigonometri (açı **radyan**)	`Math.Sin(Math.PI / 2)` → 1
`Math.Log(x)`, `Math.Log10(x)`	Doğal / 10 tabanlı logaritma	`Math.Log10(1000)` → 3
`Math.PI`, `Math.E`	π ve e sabitleri	3.141592653589793

Tabloyu bir programla deneyelim. `Floor`, `Ceiling`, `Round` ve `Truncate`, negatif sayılarda farklı sonuçlar verir; yuvarlama davranışını dikkatle gözleyelim:

▸ mathOrnek.cs

```
double x = -2.5, y = 2.5, z = 3.7;
Console.WriteLine($"Floor    : {Math.Floor(x)} {Math.Floor(z)}");
Console.WriteLine($"Ceiling  : {Math.Ceiling(x)} {Math.Ceiling(z)}");
Console.WriteLine($"Truncate : {Math.Truncate(x)} {Math.Truncate(z)}");
Console.WriteLine($"Round    : {Math.Round(x)} {Math.Round(y)} {Math.Round(z)}");
Console.WriteLine($"AwayFrom : {Math.Round(y, MidpointRounding.AwayFromZero)}");
Console.WriteLine($"Round(2) : {Math.Round(3.14159, 2)}");

Console.WriteLine(Math.Pow(2, 10));
Console.WriteLine(Math.Sqrt(3 * 3 + 4 * 4));
Console.WriteLine(Math.Sqrt(-1));
Console.WriteLine(Math.Max(3, 9) + Math.Min(-1, 4));
Console.WriteLine(Math.Clamp(15, 0, 10));
Console.WriteLine(Math.Round(Math.Sin(Math.PI / 6), 4));
Console.WriteLine(Math.Log10(1000));

double yaricap = 5;
double alan = Math.PI * Math.Pow(yaricap, 2);
Console.WriteLine($"Daire alanı: {alan:F2}");
```

```
ÇIKTI
Floor    : -3 3
Ceiling  : -2 4
Truncate : -2 3
Round    : -2 2 4
AwayFrom : 3
Round(2) : 3.14
1024
5
NaN
8
10
0.5
3
Daire alanı: 78.54
```

`Math.Round(-2.5)` ve `Math.Round(2.5)` ikisi de çifte yuvarlanır: `-2` ve `2`. Bölüm 42'de gördüğümüz bu "banker yuvarlaması" davranışını `MidpointRounding.AwayFromZero` ile okuldaki yuvarlamaya çevirebiliriz.

> **▲ Sık Yapılan Hata**
>
> `Math.Pow` ve `Math.Sqrt` gibi metotlar `double` döndürür. `int kare = Math.Pow(5, 2);` derlenmez; açık dönüşüm gerekir: `(int)Math.Pow(5, 2)`. Tam sayıların karesi için `5 * 5` yazmak hem daha hızlı hem de kesindir. Sinüs, kosinüs gibi trigonometrik metotların açıyı **radyan** cinsinden beklediğini de unutmamalıyız: derece cinsinden açıyı `derece * Math.PI / 180` ile çevirirız.

46.15 Random Sınıfı ve Rastgele Sayılar

C'de rastgele sayı için `rand()` ve `srand(time(NULL))`, C++'ta `<random>` kütüphanesi kullanılır. C#'ta `System.Random` sınıfı bu işi tek başına görür.

Rastgele sayı üreteçleri gerçek anlamda rastgele değil, **sözde rastgeledir** (pseudo-random): bir başlangıç değerinden (tohum, seed) deterministik bir diziyi hesaplarlar. Tohum verilmezse sınıf kendisi, her çalıştırmada farklı bir tohum seçer.

En kolay kullanım, .NET 6'dan beri bulunan ve hazır bir örnek sunan `Random.Shared`'dır:

▸ zarAtma.cs

```
Console.WriteLine(Random.Shared.Next());            // 0 ile int.MaxValue arası
Console.WriteLine(Random.Shared.Next(10));          // 0 ... 9
Console.WriteLine(Random.Shared.Next(1, 7));        // 1 ... 6 (zar)
Console.WriteLine(Random.Shared.NextDouble());      // 0.0 <= x < 1.0

int toplam = 0;
for (int i = 0; i < 3; i++)
{
    int zar = Random.Shared.Next(1, 7);
    Console.WriteLine($"{i + 1}. zar: {zar}");
    toplam += zar;
}
Console.WriteLine($"Toplam: {toplam}");
```

Çıktısı her çalıştırmada değişeceği için örnek çıktı vermiyoruz; `Next` metotlarının aralıklarını bir tabloda özetleyelim:

Çağrı	Aralık
`Next()`	0 … `int.MaxValue` - 1
`Next(max)`	0 … `max` - 1 (üst sınır **dahil değil**)
`Next(min, max)`	`min` … `max` - 1
`NextDouble()`	0.0 ≤ x < 1.0

> **◆ Kritik Nokta**
>
> `Next(1, 7)` 1 ile 6 arasında sayı verir; üst sınır dahil değildir. Zar atmak için `Next(1, 6)` yazarsak 6 hiç gelmez. C'de aynı sonucu `rand() % 6 + 1` ile elde ederdik; C#'ta aralığı doğrudan belirtiriz ve `%` işlemine gerek kalmaz.

Tohumu biz verirsek aynı diziyi her seferinde yeniden üretebiliriz. Bu özellik, rastgele içeren bir programı sınarken ve tekrarlanabilir deney yaparken çok değerlidir:

▸ tohumluRandom.cs

```
Random r1 = new Random(2024);
Random r2 = new Random(2024);

bool ayni = true;
for (int i = 0; i < 1000; i++)
{
    if (r1.Next(100) != r2.Next(100))
    {
        ayni = false;
    }
}
Console.WriteLine($"Aynı tohum, aynı dizi mi: {ayni}");

Random r3 = new Random(2025);
Random r4 = new Random(2024);
int fark = 0;
for (int i = 0; i < 1000; i++)
{
    if (r3.Next(100) != r4.Next(100))
    {
        fark++;
    }
}
Console.WriteLine($"Farklı tohumda fark var mı: {fark > 0}");
```

```
ÇIKTI
Aynı tohum, aynı dizi mi: True
Farklı tohumda fark var mı: True
```

Tohumlu üretecin ürettiği sayı dizisinin, farklı .NET sürümlerinde bile aynı kaldığına güvenmemeliyiz; belgeler bunu yalnızca aynı sürüm içinde garanti eder. Bu yüzden programlarda kesin sayılara bağlanmayız, yukarıdaki gibi yalnızca iki üretecin birbirine uyduğunu sınarız.

Her işlem için yeni bir `Random` oluşturmak yerine tek bir örneği (`Random.Shared`) yeniden kullanmak doğru alışkanlıktır.

> **▲ Sık Yapılan Hata**
>
> `Random` güvenlik gerektiren işler için uygun değildir: parola, oturum anahtarı ya da şifreleme anahtarı üretmek için `System.Security.Cryptography.RandomNumberGenerator` kullanılmalıdır. `Random`'ın çıktısı, birkaç değer görüldükten sonra tahmin edilebilir.

46.16 Örnek: Metotlarla Küçük Bir Hesap Kütüphanesi

Öğrendiklerimizi birleştirelim. Aşağıdaki program, her biri tek iş yapan küçük metotlardan oluşur ve metotlar birbirini çağırır: `Ekok`, `Ebob`'u kullanır; `AsalSay`, `AsalMi`'yi kullanır ve iki sonucu demetle döndürür.

▸ hesapKutuphanesi.cs

```
int a = 84, b = 36;
Console.WriteLine($"EBOB({a}, {b}) = {Ebob(a, b)}");
Console.WriteLine($"EKOK({a}, {b}) = {Ekok(a, b)}");

Console.Write("30'a kadar asal sayılar:");
for (int n = 2; n <= 30; n++)
{
    if (AsalMi(n))
    {
        Console.Write($" {n}");
    }
}
Console.WriteLine();

var (adet, toplam) = AsalSay(100);
Console.WriteLine($"100'e kadar {adet} asal var, toplamları {toplam}");

/// <summary>Öklid algoritmasıyla en büyük ortak bölen.</summary>
static int Ebob(int x, int y)
{
    while (y != 0)
    {
        (x, y) = (y, x % y);
    }
    return x;
}

static long Ekok(int x, int y) => (long)x / Ebob(x, y) * y;

static bool AsalMi(int sayi)
{
    if (sayi < 2)
    {
        return false;
    }
    for (long i = 2; i * i <= sayi; i++)
    {
        if (sayi % i == 0)
        {
            return false;
        }
    }
    return true;
}

static (int Adet, int Toplam) AsalSay(int ust)
{
```

```
    int adet = 0, toplam = 0;
    for (int n = 2; n <= ust; n++)
    {
        if (AsalMi(n))
        {
            adet++;
            toplam += n;
        }
    }
    return (adet, toplam);
}
```

```
ÇIKTI
EBOB(84, 36) = 12
EKOK(84, 36) = 252
30'a kadar asal sayılar: 2 3 5 7 11 13 17 19 23 29
100'e kadar 25 asal var, toplamları 1060
```

Programın ana kısmı artık bir iş akışı gibi okunuyor: EBOB'u yaz, EKOK'u yaz, asalları dök. Ayrıntılar metotların içinde saklı. `Ekok` hesabında önce bölüp sonra çarpmamızın nedeni, `x * y` çarpımının `int` sınırını aşabilmesidir; `(long)` dönüşümü de bu yüzdendir (Bölüm 42). `///` ile başlayan satır, Bölüm 41'de tanıştığımız XML belge yorumudur: editörler metodun üzerine gelindiğinde bu açıklamayı gösterir.

> **İpucu**
>
> İyi bir metot tek bir iş yapar ve adı o işi söyler. Gövdesi bir ekrana sığmıyorsa onu daha küçük metotlara bölmek çoğu zaman doğru harekettir.

46.17 Üç Dilde Metotlar

Üç Dilde · Aynı metot, üç dil *C · C++ · C#*

Bir değişkenin kendisini değiştiren `Takas` metodunu üç dilde yazalım. Tanıdık C'den başlayıp farkları sırayla görelim.

C'de her argüman kopyalanır; değişkenin kendisine ulaşmak için adresini işaretçi olarak geçiririz. C++ ve C# bu işi dilin içine almıştır:

```
void takas(int *a, int *b) { int t = *a; *a = *b; *b = t; }
takas(&x, &y);
```

C++'ta aynı iş referansla yapılır; çağrıda hiçbir işaret görünmez (Bölüm 25 ve Bölüm 28):

```
void takas(int& a, int& b) { int t = a; a = b; b = t; }
takas(x, y);          // çağrıdan değişkenin değişeceği anlaşılmaz
```

C#'ta `ref` hem tanımda hem çağrıda yazılır:

```
void Takas(ref int a, ref int b) { (a, b) = (b, a); }
Takas(ref x, ref y);  // çağrıdan değişkenin değişeceği anlaşılır
```

Başka farkları tabloda toplayalım:

Özellik	C	C++	C#
Önceden bildirim (prototip)	Gerekir	Gerekir	Gerekmez
Aşırı yükleme	Yok	Var	Var (sınıf içinde)
Varsayılan parametre	Yok	Var	Var
Adlandırılmış argüman	Yok	Yok	Var
Referansla geçirme	İşaretçi (`*` , `&`)	`&` (çağrıda belli olmaz)	`ref` , `out` , `in` (çağrıda yazılır)
Salt okunur, kopyasız geçirme	`const T *`	`const T&`	`in T`
Değişken sayıda argüman	`...` ve `stdarg.h`	`initializer_list` , şablonlar	`params`
Birden çok değer döndürme	`struct` , çıkış işaretçileri	`pair` , `tuple` , `struct`	demet, `out`
Fonksiyon içinde fonksiyon	Yok	Yok (lambda var)	Yerel fonksiyon
Fonksiyon dışında kod	Yalnızca bildirim ve tanım	Aynı	Top-level statements (tek dosyada)

Tarihçe · Metot özelliklerinin tarihi

Aşırı yükleme ve sınıf içi metotlar C# 1.0'dan (2002) beri vardır. Varsayılan ve adlandırılmış parametreler C# 4.0 ile (2010) geldi; o zamana kadar aynı etki aşırı yüklemeyle sağlanırdı. `out` değişkenlerin çağrıda tanımlanması, demetler, yerel fonksiyonlar ve atma değişkenleri `_` C# 7.0'da (2017) eklendi. `in` parametreleri C# 7.2'de, `static` yerel fonksiyonlar C# 8.0'da geldi. `Random.Shared` ise .NET 6 ile birlikte sunuldu.

Bölüm Özeti

- Metot, ada sahip, parametre alabilen ve değer döndürebilen kod bloğudur. Tanımı: değiştirici (`static`), dönüş tipi, ad, parametre listesi ve gövdeden oluşur.
- Ana kodun altına yazılan metotlar `Main` 'in yerel fonksiyonlarıdır: `public` yazılamaz, aşırı yüklenemez. Sınıf içindeki `static` metotlar bu kısıtlamalara tabi değildir.
- C#'ta önceden bildirim (prototip) gerekmez; metot, tanımlandığı yerden bağımsız olarak çağrılabilir.
- Parametre, metodun tanımındaki değişkendir; argüman, çağrıda verilen değerdir. Çağrılar çağrı yığınında tutulur ve son çağrılan metot ilk biter.
- `void` metot değer üretmez; diğerleri `return` ile değer döndürür. Dönüş tipi `void` olmayan metodun her yolu bir değerle bitmelidir (`CS0161`).
- Varsayılan geçirme biçimi değerle geçirmedir. Referans tiplerinde kopyalanan nesne değil bağlantıdır: içerik değişebilir, ama çağıranın değişkeni başka bir nesneye bağlanamaz.
- `ref` değişkenin kendisini geçirir (önceden değer atanmış olmalı); `out` ek sonuç döndürür (metot atama yapmak zorundadır); `in` kopyasız ama salt okunur geçirir. `ref` ve `out` çağrıda da yazılır.
- `TryParse` deseni, başarıyı `bool` dönüş değeriyle, sonucu `out` parametresiyle bildirir. İstisnaya göre daha ucuz ve daha açıktır.

- `params` son parametreyi, değişken sayıda argüman alan bir diziye dönüştürür. Bir metotta en fazla bir tane olabilir.
- Varsayılan değerli parametreler zorunlulardan sonra gelir; adlandırılmış argümanlarla sırayı değiştirebilir ya da ortadakini atlayabiliriz.
- Aşırı yüklemede imzayı ad ve parametreler belirler, dönüş tipi belirlemez. Derleyici en uygun adayı derleme zamanında seçer; eşit uygun iki aday `CS0121` belirsizlik hatasıdır.
- İfade gövdeli metotlar (`=> ifade`) tek ifadelik metotlar içindir. Yerel fonksiyonlar bir metodun içinde tanımlanır; `static` yaparsak dış değişkenleri yakalayamaz.
- Demetler `(int Enk, int Enb)` birden çok değeri tek değer olarak döndürür; `var (a, b) = ...` ile açılırlar. Okunabilirlik için dönüş tipinde eleman adı vermek iyi bir alışkanlıktır.
- `Math` sınıfı matematik işlevlerini (`Abs` , `Pow` , `Sqrt` , `Round` , `Floor` , `Clamp` …) sunar; trigonometri radyanla çalışır, `Round` varsayılan olarak çifte yuvarlar.
- `Random.Shared.Next(min, max)` üst sınırı **dahil etmez**. Tohumlu `Random` , aynı sürüm içinde aynı diziyi üretir. Güvenlik gerektiren işlerde `Random` kullanılmaz.

Alıştırmalar

1. Aynı sınıfta `static int Topla(int a, int b)` metodu varken aşağıdakilerden hangisi onun geçerli bir **aşırı yüklemesi** olur?
   A) `static double Topla(int a, int b)`
   B) `static int Topla(int x, int y)`
   C) `static int Topla(int a, int b, int c)`
   D) `static void Topla(int a, int b)`
   E) `public static int Topla(int a, int b)`
2. `out` parametreleriyle ilgili aşağıdakilerden hangisi doğrudur?
   A) Çağrıdan önce değişkene değer atanmış olmalıdır.
   B) Çağrıda `out` sözcüğü yazılmaz.
   C) Metot, dönmeden önce `out` parametresine mutlaka değer atamalıdır.
   D) Bir metotta en fazla bir `out` parametresi olabilir.
   E) `out` parametresi yalnızca referans tiplerle kullanılabilir.
3. `Console.WriteLine(Math.Round(2.5) + Math.Floor(-1.5));` komutu ne yazdırır?
   A) `0`
   B) `1`
   C) `-1`
   D) `2`
   E) `1.5`
4. `Random.Shared.Next(2, 5)` çağrısı aşağıdaki değerlerden hangilerini üretebilir?
   A) Yalnızca 2, 3, 4, 5
   B) Yalnızca 2, 3, 4
   C) Yalnızca 3, 4
   D) 0 ile 5 arasındaki herhangi bir tam sayı
   E) 2 ile 5 arasındaki herhangi bir gerçek sayı
5. Aşağıdaki program ekrana ne yazdırır?

```
int a = 5;
int b = 5;
Isle(a, ref b);
Console.WriteLine($"{a} {b}");

static void Isle(int x, ref int y)
{
    x *= 2;
    y *= 2;
}
```

6. Aşağıdaki program ekrana ne yazdırır?

```
int[] d = { 1, 2, 3 };
Degistir(d);
Console.WriteLine(string.Join(",", d));
DegistirRef(ref d);
Console.WriteLine(string.Join(",", d));

static void Degistir(int[] x)
{
    x[0] = 10;
    x = new int[] { 7, 7 };
}

static void DegistirRef(ref int[] x)
{
    x[0] = 20;
    x = new int[] { 7, 7 };
}
```

7. Aşağıdaki program ekrana ne yazdırır?

```
Console.WriteLine(Yaz("Can"));
Console.WriteLine(Yaz("Ece", "Bayan"));
Console.WriteLine(Yaz("Mert", sayi: 3));
Console.WriteLine(Yaz(unvan: "Dr.", ad: "Selin"));

static string Yaz(string ad, string unvan = "Bay", int sayi = 1)
{
    return $"{unvan} {ad} x{sayi}";
}
```

8. Aşağıdaki program ekrana ne yazdırır?

```
Console.WriteLine(K.F(5));
Console.WriteLine(K.F(5.0));
Console.WriteLine(K.F(5L));
Console.WriteLine(K.F('a'));
Console.WriteLine(K.F(2.5f));
Console.WriteLine(K.F("5"));

static class K
{
    public static string F(int x) => "int";
    public static string F(long x) => "long";
    public static string F(double x) => "double";
    public static string F(string x) => "string";
}
```

9. Aşağıdaki program ekrana ne yazdırır?

```
var (b, k) = Bol(47, 5);
Console.WriteLine($"{b} {k}");
Console.WriteLine(Dene("42", out int s) + " " + s);
Console.WriteLine(Dene("x2", out int t) + " " + t);

static (int, int) Bol(int a, int c) => (a / c, a % c);

static bool Dene(string metin, out int sonuc)
{
    return int.TryParse(metin, out sonuc);
}
```

10. Aşağıdaki program ekrana ne yazdırır?

```
int sayac = 0;
Console.WriteLine(Say() + Say() * 10);
Console.WriteLine(sayac);

int Say()
{
    sayac++;
    return sayac;
}
```

11. Aşağıdaki kodda dört satırda derleme hatası vardır. Satırları bulup her birini düzeltin.

```
int x = 1, y = 2;
Takas(x, y);

static void Takas(ref int a, ref int b) { (a, b) = (b, a); }
static int Yarim(int n) { if (n > 0) { return n / 2; } }
static void Oku(out int s) { Console.WriteLine("okunuyor"); }
static void Ekle(int a = 1, int b) { }
```

12. `ref` ve `out` parametreleri arasındaki iki önemli farkı yazın. Hangisini ne zaman tercih ederiz?
13. Bir `int` değerin faktöriyelini (`n!`) döngüyle hesaplayıp `long` olarak döndüren `Faktoriyel` metodunu yazın. Ana kodda `5!` ve `15!` değerlerini yazdırın.
14. İstenen sayıda `int` argüman alan (`params`) ve en küçük ile en büyük değeri demet olarak döndüren `Aralik` metodunu yazın. `Aralik(7, 2, 9, 4)` çağrısının sonucunu `En küçük: 2, en büyük: 9` biçiminde yazdırın. En az bir argüman verildiğini varsayabilirsiniz.
15. `SicaklikCevir(double deger, string hedef = "F")` metodunu yazın. Değer Celsius cinsindendir; `hedef` `"F"` ise Fahrenayt (`C * 9 / 5 + 32`), `"K"` ise Kelvin (`C + 273.15`) döndürsün. Başka bir hedef verilirse `ArgumentException` fırlatsın. `SicaklikCevir(100)` ve `SicaklikCevir(0, hedef: "K")` çağrılarının sonuçlarını yazdırın.
16. `TryBol(int a, int b, out double sonuc)` metodunu yazın: `b` sıfırsa `false` döndürüp `sonuc`'a 0 atasın, değilse `a / (double)b` değerini atayıp `true` döndürsün. Ana kodda `(7, 2)` ve `(5, 0)` çiftlerini deneyin ve her biri için ya sonucu ya da `Bölme yapılamadı` yazdırın.

BÖLÜM 47

Diziler

BU BÖLÜMDE

- Dizi tanımlamayı, `new` ile oluşturmayı, ilk değer vermeyi ve varsayılan eleman değerlerini
- `Length`, indeks mantığını ve her erişimde yapılan sınır denetimini (`IndexOutOfRangeException`)
- Dizileri `for` ve `foreach` ile dolaşmayı; `^` ile sondan indeksi ve `..` ile aralığı
- Dizilerin referans tipi olduğunu: atama, `==` karşılaştırması ve kopyalama yolları (`Clone`, `Array.Copy`)
- Diziyi metoda geçirmeyi ve `Array` sınıfının hazır metotlarını (`Sort`, `Reverse`, `IndexOf`, `BinarySearch`, `Fill`)
- Arama ve sıralama algoritmalarının C# sürümlerini
- Çok boyutlu (`int[,]`) ve düzensiz (`int[][]`) dizileri

Aynı türden çok sayıda veriyi saklamak gerektiğinde tek tek değişken tanımlamak çözüm olmaz: kırk öğrencinin notu için `not1`, `not2`, ..., `not40` yazmak hem zahmetlidir hem de bu değişkenleri bir döngüyle dolaşamayız. Çözüm, bu değerleri tek bir ad altında, bellekte art arda tutan **dizidir (array)**.

Dizileri C'de Bölüm 10'da, C++'ta `std::array` ve `std::vector` ile Bölüm 26'da görmüştük. Kavramlar aynıdır: sabit sayıda, aynı tipten eleman ve sıfırdan başlayan indeksler. C# bu temele iki önemli şey ekler. Birincisi, dizi bir **nesnedir** ve her zaman referans tipidir; boyutunu kendisi bilir (`Length`) ve her erişimde sınırları denetlenir. İkincisi, `Array` sınıfı sıralama, arama ve kopyalama gibi işleri hazır metotlarla sunar. Bu bölümde tanıdık kısımları hızlı geçip farkları ayrıntılı işleyeceğiz.

47.1 Dizi Tanımlama ve Oluşturma

> **Tanım: Dizi**
>
> Aynı tipten sabit sayıda elemanı ardışık olarak tutan ve her elemana sıfırdan başlayan bir indeksle erişilen veri yapısıdır. C#'ta bütün diziler `System.Array` sınıfından türeyen referans tipli nesnelerdir.

C#'ta dizi tipi, eleman tipinin yanına köşeli parantez yazılarak belirtilir: `int[]` "int dizisi" demektir. Dizi değişkeni tanımlamak ile diziyi oluşturmak ayrı adımlardır. `new int[5]` bellekte beş `int`'lik yer ayırır ve bu diziye bir referans döndürür:

```
int[] notlar;              // yalnızca değişken: henüz dizi yok
notlar = new int[5];       // beş elemanlı dizi oluşturuldu
int[] puanlar = new int[5]; // ikisi bir arada
```

C'deki `int a[5];` yazımı C#'ta geçersizdir. Köşeli parantez değişkenin değil **tipin** parçasıdır:

cStiliHata.cs

```
int a[5];
```

DERLEYİCİ ÇIKTISI

```
cStiliHata.cs(1,6): error CS0650: Bad array declarator: To declare a managed array the rank
specifier precedes the variable's identifier. To declare a fixed size buffer field, use the
fixed keyword before the field type.
cStiliHata.cs(1,7): error CS0270: Array size cannot be specified in a variable declaration (try
initializing with a 'new' expression)
```

C'den bir fark daha var: boyut olarak herhangi bir `int` ifadesi yazılabilir (`new int[n]`); C'deki "sabit ifade olmalı" kısıtı ya da değişken uzunluklu dizi (VLA) tartışması burada yoktur. Dizi her zaman çalışma zamanında öbekte (heap) oluşturulur. Boyut negatifse `OverflowException` fırlatılır. Oluşturulduktan sonra boyut değişmez.

Elemanların değerleri baştan biliniyorsa süslü parantezle ilk değer verilebilir. Boyutu derleyici sayar:

diziTanimlama.cs

```
int[] notlar = new int[5];
notlar[0] = 85;
notlar[1] = 70;
notlar[4] = notlar[0] + 5;
Console.WriteLine($"{notlar[0]} {notlar[1]} {notlar[2]} {notlar[4]}");

string[] gunler = { "Pzt", "Sal", "Çar", "Per", "Cum" };
double[] fiyatlar = new double[] { 12.5, 8.0, 99.9 };
var sayilar = new[] { 3, 1, 4 };    // eleman tipi değerlerden çıkarılır
Console.WriteLine($"{gunler[2]} {fiyatlar[1]} {sayilar[0]}");
Console.WriteLine($"{notlar.Length} {gunler.Length} {fiyatlar.Length}");
```

ÇIKTI

```
85 70 0 90
Çar 8 3
5 5 3
```

Programdaki beş yazım şöyle özetlenebilir:

Yazım	Anlamı
`new int[5]`	Beş eleman, hepsi varsayılan değerde
`new int[] { 1, 2, 3 }`	Boyutu derleyici sayar
`new[] { 1, 2, 3 }`	Eleman tipi de değerlerden çıkarılır
`int[] a = { 1, 2, 3 };`	Yalnızca **tanımlama satırında** geçerli kısa yazım
`int[] a = [1, 2, 3];`	Koleksiyon ifadesi (C# 12)

`{ 1, 2, 3 }` kısa yazımı yalnızca değişkenin tanımlandığı satırda kullanılabilir. Var olan bir diziye `a = { 4, 5 };` yazılamaz; bunun için `a = new int[] { 4, 5 };` gerekir.

Koleksiyon ifadeleri

C# 12 ile gelen koleksiyon ifadesi (collection expression), diziyi köşeli parantezle yazmamızı sağlar. Tip sol taraftan anlaşıldığı için `new` yazılmaz. İki nokta `..` (yayma, spread) başka bir dizinin elemanlarını araya yerleştirir:

`koleksiyonIfadesi.cs`

```
int[] c = [1, 2, 3];
int[] d = [.. c, 4, 5];       // c'nin elemanları, sonra 4 ve 5
int[] bos = [];
Console.WriteLine(string.Join(", ", d));
Console.WriteLine(bos.Length);
```

ÇIKTI

```
1, 2, 3, 4, 5
0
```

`string.Join(ayirac, dizi)` dizinin elemanlarını araya ayraç koyarak tek bir metinde birleştirir. Dizileri yazdırmanın en kısa yolu budur; ayrıntısını Bölüm 48'de göreceğiz.

Varsayılan değerler

`new` ile oluşturulan dizide elemanlara değer vermesek bile hepsi **varsayılan değeri** alır. Bu, Bölüm 42'de gördüğümüz "yerel değişken kullanılmadan önce değer almalı" kuralından farklıdır: yerel bir `int` değişkenini atamadan okuyamayız, ama yeni bir dizinin elemanını okuyabiliriz. C'deki gibi çöp değer yoktur; yerel `int a[3];` dizisinin içinde belirsiz değerler bulunurdu.

Eleman tipi	Varsayılan değer
`int`, `long`, `byte` vb.	`0`
`double`, `float`	`0.0`
`decimal`	`0`
`bool`	`false`
`char`	`'\0'` (kodu 0 olan karakter)
`string` ve diğer bütün sınıflar	`null`

`varsayilanDegerler.cs`

```
int[] a = new int[3];
double[] b = new double[2];
bool[] c = new bool[2];
char[] d = new char[2];
string[] e = new string[2];
decimal[] m = new decimal[1];
Console.WriteLine($"{a[0]} {b[1]} {c[0]} {(int)d[0]} {e[0] == null} {m[0]}");
```

ÇIKTI

```
0 0 False 0 True 0
```

`string` dizisinin elemanları boş metin (`""`) değil `null` olur: "burada henüz bir nesne yok" demektir. `e[0].Length` yazarsak program `NullReferenceException` ile durur. Aynısı sınıf tipli her dizi için geçerlidir; dizi oluşturmak, elemanların nesnelerini oluşturmak anlamına gelmez.

47.2 Length, İndeks ve Sınır Denetimi

Her dizi kendi eleman sayısını `Length` özelliğinde (property) taşır. C'de bunun için `sizeof(a) / sizeof(a[0])` hesabı yapar ve dizi fonksiyona geçince bu hesabın bozulmasını yaşardık (Bölüm 10). C#'ta `Length` her yerde doğru çalışır, çünkü uzunluk dizi nesnesinin içinde saklanır. Geçerli indeksler `0` ile `Length - 1` arasındadır.

> **▲ Sık Yapılan Hata · Length bir özelliktir, metot değildir**
>
> Dizide `a.Length` yazılır, parantez konmaz. `a.Length()` derleme hatasıdır:
>
> ▸ lengthHata.cs
>
> ```
> int[] a = { 1, 2, 3 };
> Console.WriteLine(a.Length());
> ```
>
> DERLEYİCİ ÇIKTISI
>
> ```
> lengthHata.cs(2,21): error CS1955: Non-invocable member 'Array.Length' cannot be used like
> a method.
> ```
>
> `string` için de aynı kural geçerlidir (`s.Length`). Koleksiyonlarda ise eleman sayısı `Count` adını taşır (Bölüm 54).

Her erişim denetlenir

C'de `a[5] = 99;` komutu 5 elemanlı bir dizinin dışına yazar ve davranış tanımsızdır (Bölüm 10). C#'ta bu program **derlenir**, ama çalışma zamanında her indeks erişimi denetlenir ve geçersiz bir indeks `IndexOutOfRangeException` fırlatır. Derleyici indeksin değerini genelde bilemez; denetim çalışırken yapılır. Aşağıdaki programda hatayı `try` / `catch` ile yakalıyoruz; bu yapıyı Bölüm 53'te ayrıntılı işleyeceğiz, şimdilik "tehlikeli kodu `try` içine al, hata olursa `catch` çalışır" demek yeterli.

▸ sinirDisi.cs

```
int[] a = { 10, 20, 30 };
Console.WriteLine($"{a[0]} {a[a.Length - 1]}");
try
{
    for (int i = 0; i <= a.Length; i++)    // <= : hata
    {
        Console.WriteLine($"{i}: {a[i]}");
    }
}
catch (IndexOutOfRangeException hata)
{
    Console.WriteLine($"Hata: {hata.Message}");
}
Console.WriteLine("Program devam ediyor");
```

```
ÇIKTI
10 30
0: 10
1: 20
2: 30
Hata: Index was outside the bounds of the array.
Program devam ediyor
```

Yakalanmayan bir istisna programı sonlandırır ve konsola `Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.` mesajı ile çağrı izini yazar. Sessizce bellek bozan bir hata yerine, nedenini ve yerini söyleyen bir hata almış oluruz.

> **▲ Sık Yapılan Hata · Bir fazla dönmek (off-by-one)**
>
> Üç elemanlı dizinin indeksleri 0, 1 ve 2'dir. Döngü koşulu `i < a.Length` olmalıdır; `i <= a.Length` son turda `a[3]` okur. Bu hata C'de sessizce yanlış sonuç üretirdi, C#'ta çalışma anında istisnaya dönüşür. Dizi dolaşılırken indeks gerekmiyorsa `foreach` bu hatayı baştan ortadan kaldırır.

Denetimin bir maliyeti vardır, ama küçüktür. JIT derleyicisi `i < a.Length` gibi kalıplarda indeksin sınır içinde kaldığını kanıtlayabildiği durumlarda denetimi koddan çıkarır. Yine de denetimi kapatmanın bir yolu yoktur; güvenli kodda arabellek taşması (buffer overflow) açığı C# diziler için fiilen mümkün değildir.

> **Üç Dilde · Sınır denetimi üç dilde** *C · C++ · C#*
>
> ```c
> int a[5] = {0};
> a[7] = 1;              // C: derlenir; tanımsız davranış
> ```
>
> ```cpp
> std::array<int, 5> a{};
> a[7] = 1;              // C++: [] denetlemez; tanımsız davranış
> a.at(7) = 1;           // C++: std::out_of_range fırlatır
> ```
>
> ```csharp
> int[] a = new int[5];
> a[7] = 1;              // C#: IndexOutOfRangeException
> ```
>
> C'de denetim programcıya aittir. C++'ta iki yol vardır: hızlı ama denetimsiz `[]` ve denetimli `at()`. C#'ta tek yol vardır ve her zaman denetimlidir.

47.3 Diziyi Dolaşmak

Diziyi dolaşmanın iki yolu vardır: indeksli `for` ve `foreach`. Bölüm 45'te gördüğümüz `foreach`, "dizideki her eleman için" diye okunur ve indeksle uğraştırmaz. C++11'in aralık tabanlı `for` döngüsünün karşılığıdır (Bölüm 26).

▸ dolasma.cs

```csharp
int[] notlar = { 85, 70, 92, 64, 78 };

for (int i = 0; i < notlar.Length; i++)
{
    Console.Write($"[{i}]={notlar[i]} ");
}
Console.WriteLine();
```

```
for (int i = notlar.Length - 1; i >= 0; i--)    // tersten
{
    Console.Write($"{notlar[i]} ");
}
Console.WriteLine();

foreach (int puan in notlar)
{
    Console.Write($"{puan} ");
}
Console.WriteLine();

Console.WriteLine(string.Join(", ", notlar));
Console.WriteLine(notlar);                      // elemanları değil, tipi
```

```
ÇIKTI
[0]=85 [1]=70 [2]=92 [3]=64 [4]=78
78 64 92 70 85
85 70 92 64 78
85, 70, 92, 64, 78
System.Int32[]
```

Son satırda `Console.WriteLine(notlar)` elemanları değil dizinin tipini yazdı: `System.Int32[]`, "`Int32` dizisi" demektir. C#'ta dizinin metin gösterimi elemanlarını içermez. Elemanları görmek için döngü ya da `string.Join` kullanırız.

foreach değişkeni salt okunurdur

`foreach` döngüsündeki değişken yalnızca okunabilir. Ona değer atamak, dizinin kendisini değiştirmez; üstelik derleyici buna izin vermez:

▸ foreachHata.cs

```
int[] a = { 1, 2, 3 };
foreach (int x in a)
{
    x = x * 10;
}
```

```
DERLEYİCİ ÇIKTISI

foreachHata.cs(4,9): error CS1656: Cannot assign to 'x' because it is a 'foreach iteration
variable'
```

Elemanları değiştirmek istiyorsak indeksli `for` kullanırız: `a[i] = a[i] * 10;`. Eleman bir sınıf nesnesiyse `foreach` içinde nesnenin alanlarını değiştirebiliriz; yasak olan, döngü değişkenine yeni bir değer atamaktır.

	`for`	`foreach`
İndekse erişim	Var (`i`)	Yok
Elemanı değiştirme	`a[i] = ...` yapılabilir	Yapılamaz (`CS1656`)
Tersten ya da atlayarak dolaşma	Yapılabilir	Yapılamaz; hep baştan sona
Ne zaman?	İndeks gerekiyorsa ya da dizi değişecekse	Yalnızca okunacaksa

47.4 İndeksler ve Aralıklar

Son elemana `a[a.Length - 1]` ile ulaşabiliriz. C# 8, bu işi kısaltan iki yazım getirdi. `^` işareti **sondan indeks** (index from end) verir: `a[^1]` son eleman, `a[^2]` sondan ikinci elemandır. `..` işareti ise **aralık** (range) tanımlar: `a[b..e]`, `b` indeksinden başlayıp `e` indeksine kadar gider; `e` dahil değildir. Başlangıç ya da bitiş yazılmazsa baştan ya da sona kadar alınır.

Yazım	Anlamı
`a[^1]`	Son eleman (`a[a.Length - 1]` ile aynı)
`a[1..4]`	1., 2. ve 3. indeksler; 4 hariç. Üç elemanlı yeni dizi
`a[..2]`	Baştan 2. indekse kadar: ilk iki eleman
`a[2..]`	2. indeksten sona kadar
`a[^3..]`	Son üç eleman
`a[..]`	Bütün dizinin kopyası

`▸ araliklar.cs`

```
int[] a = { 10, 20, 30, 40, 50 };
Console.WriteLine($"{a[a.Length - 1]} {a[^1]} {a[^2]}");

int[] orta = a[1..4];
Console.WriteLine(string.Join(", ", orta));
Console.WriteLine(string.Join(", ", a[..2]));
Console.WriteLine(string.Join(", ", a[2..]));
Console.WriteLine(string.Join(", ", a[^3..]));

orta[0] = 99;                 // orta ayrı bir kopya: a değişmez
Console.WriteLine($"{a[1]} {orta[0]}");

Index son = ^1;               // indeks ve aralık da birer değerdir
Range r = 1..3;
Console.WriteLine($"{son} {r} {a[son]} {string.Join(", ", a[r])}");
```

```
ÇIKTI
50 50 40
20, 30, 40
10, 20
30, 40, 50
30, 40, 50
20 99
^1 1..3 50 20, 30
```

Dizi üzerinde aralık almak **yeni bir dizi oluşturur** ve elemanları ona kopyalar; kopyayı değiştirmek asıl diziyi etkilemez. Aralığın eleman sayısı `e - b`'dir. Son satırda görüldüğü gibi `^1` bir `Index`, `1..3` bir `Range` değeridir; bunları değişkende saklayabiliriz. Aynı yazım `string` üzerinde de çalışır (Bölüm 48).

> ◆ **Kritik Nokta**
>
> `^1` son elemandır; `^0` ise `a[a.Length]` demektir ve geçersizdir. Aralıklarda ise `^0` meşrudur: `a[2..^0]` "2'den sona kadar" demektir. Sınırın dışına taşan bir aralık `ArgumentOutOfRangeException` fırlatır. Ayrıca köşeli parantez içinde ve sayının önünde duran `^`, Bölüm 42'deki bit operatörü `a ^ b` (XOR) değildir.

▸ araliklarSinir.cs

```
int[] a = { 10, 20, 30, 40, 50 };
Console.WriteLine(a[2..^0].Length);
try { Console.WriteLine(a[^0]); }
catch (IndexOutOfRangeException h) { Console.WriteLine(h.GetType().Name); }
try { int[] x = a[3..10]; }
catch (ArgumentOutOfRangeException h) { Console.WriteLine(h.GetType().Name); }
```

ÇIKTI

```
3
IndexOutOfRangeException
ArgumentOutOfRangeException
```

47.5 Diziler Referans Tipidir

Bölüm 42'de değer tipleri ile referans tiplerini ayırmıştık. Dizi, elemanları `int` gibi değer tipli olsa bile **referans tipidir**: dizi değişkeni dizinin kendisini değil, öbekteki dizi nesnesine giden bir referansı tutar. Bu durumun sonuçları C'deki dizi davranışından farklıdır. C'de `int b[3] = a;` yazılamaz, dizi atanamaz; C++'ta `std::array` atandığında bütün elemanlar kopyalanır. C#'ta atama yalnızca referansı kopyalar:

▸ referansAtama.cs

```
int x = 5;
int y = x;                  // değer kopyalandı
y = 99;
Console.WriteLine($"x = {x}");

int[] a = { 1, 2, 3 };
int[] b = a;                // referans kopyalandı: tek dizi, iki ad
b[0] = 99;
Console.WriteLine($"a = {string.Join(", ", a)}");

int[] c = { 99, 2, 3 };     // içeriği aynı, ama ayrı bir dizi
Console.WriteLine(a == b);
Console.WriteLine(a == c);
Console.WriteLine(a.SequenceEqual(c));
```

ÇIKTI

```
x = 5
a = 99, 2, 3
True
False
True
```

`int[] b = a;` satırı yeni dizi oluşturmaz, aynı diziyi gösteren ikinci bir ad üretir. `b` üzerinden yapılan değişiklik `a`'dan da görünür:

```
a ---+
     +--->  [ 99 | 2 | 3 ]     tek dizi, iki referans
b ---+

c ------->  [ 99 | 2 | 3 ]     ayrı dizi, içeriği aynı
```

Dizilerde `==` operatörü **referansları** karşılaştırır: "bu iki değişken aynı diziyi mi gösteriyor?" sorusunu sorar. `a == c` içerikler aynı olduğu halde `False`'tur. İçerik karşılaştırması için kendi döngümüzü yazabilir ya da Bölüm

56'da göreceğimiz LINQ'un `SequenceEqual` metodunu kullanabiliriz. C'de dizi adlarını `==` ile karşılaştırmak adresleri karşılaştırdığı için bu mantık size tanıdık gelecektir; farkı, karşılaştırılan şeyin adres değil referans olmasıdır.

> **▲ Sık Yapılan Hata · string ile karıştırmayalım**
>
> `string` de referans tipidir, ama `==` operatörü onun için içeriği karşılaştıracak biçimde yeniden tanımlanmıştır (`"ali" == "ali"` doğrudur). Dizi tipinde böyle bir tanım yoktur. Bu yüzden `==` iki tipte farklı davranır; Bölüm 48'de `string` tarafını ayrıntılı göreceğiz.

Referans tipli bir değişken hiçbir nesneyi göstermeyebilir: `int[]? d = null;` yazabiliriz. Böyle bir değişkenin `Length`'ine ya da elemanına ulaşmak `NullReferenceException` doğurur (Bölüm 49).

> **Üç Dilde · Dizinin bellekteki yeri** *C · C++ · C#*
>
	C	C++	C#
Yerel dizi	`int a[5];` yığında	`std::array<int, 5> a;` yığında	`new int[5]` her zaman öbekte
Dinamik dizi	`malloc` (elle `free`)	`std::vector` (kendiliğinden serbest)	Aynı `new`; çöp toplayıcı serbest bırakır
Boyut bilgisi	Dizide yok; ayrıca taşınır	`array` / `vector` bilir (`size()`)	Dizi bilir (`Length`)
Atama `b = a`	Dizi için geçersiz	Elemanlar kopyalanır	Referans kopyalanır
Sınır denetimi	Yok	`[]` yok, `at()` var	Her erişimde var
>
> C#'ta yığında dizi oluşturmak mümkündür ama özel bir yazım ister (`stackalloc`, güvenli kodda `Span<T>` ile); gündelik kodda diziler öbekte yaşar ve değişkenler yalnızca referans taşır.

47.6 Diziyi Kopyalamak

`b = a` kopya oluşturmadığına göre gerçek bir kopyayı nasıl elde ederiz? C#'ta birkaç yol vardır:

- `Clone()` metodu yeni bir dizi döndürür. Dönüş tipi `object` olduğundan `(int[])` dönüşümü gerekir.
- `Array.Copy(kaynak, hedef, uzunluk)` var olan bir hedef diziye kopyalar. Hedef önceden oluşturulmuş olmalıdır. Başlangıç indekslerini veren bir sürümü de vardır.
- `[.. a]` koleksiyon ifadesi ya da `a[..]` aralığı yeni bir kopya üretir.
- Elle yazılmış bir döngü de elbette olur.

`kopyalama.cs`

```
int[] a = { 5, 10, 15 };

int[] k1 = (int[])a.Clone();
int[] k2 = new int[a.Length];
Array.Copy(a, k2, a.Length);
int[] k3 = [.. a];
int[] k4 = a[..];

k1[0] = -1;
k2[0] = -2;
```

```
k3[0] = -3;
k4[0] = -4;
Console.WriteLine(string.Join(", ", a));          // a etkilenmedi
Console.WriteLine($"{k1[0]} {k2[0]} {k3[0]} {k4[0]}");

int[] buyuk = new int[5];                         // fazlası 0 kalır
Array.Copy(a, buyuk, a.Length);
Console.WriteLine(string.Join(", ", buyuk));

Array.Copy(a, 1, buyuk, 3, 2);   // a[1]'den başla, buyuk[3]'e, 2 eleman
Console.WriteLine(string.Join(", ", buyuk));

Array.Resize(ref a, 5);          // yeni 5 elemanlı dizi oluşturur, a ona bağlanır
Console.WriteLine(string.Join(", ", a));
```

```
ÇIKTI
5, 10, 15
-1 -2 -3 -4
5, 10, 15, 0, 0
5, 10, 15, 10, 15
5, 10, 15, 0, 0
```

Dizinin boyutu değişmediğinden "büyütme" gerçekte yeni, daha büyük bir dizi oluşturup elemanları ona kopyalamaktır. `Array.Resize` bunu yapar ve yeni diziyi değişkene atar; bu yüzden dizi `ref` ile geçirilir. Sık sık ekleme yapılacaksa baştan `List<T>` kullanmak gerekir; bu, C++'taki `vector`'ün karşılığıdır (Bölüm 54).

Kopyalama yöntemlerinin hepsi **yüzeysel** (shallow) kopyadır: elemanlar değer tipliyse değerleri kopyalanır, referans tipliyse yalnızca referansları kopyalanır. Bu ayrımı düzensiz dizilerde somut olarak göreceğiz.

47.7 Diziyi Metoda Geçirmek

Parametre geçirme kuralı dizilerde de aynıdır: C# parametreleri **değerle** geçirir. Fakat dizi değişkeninin değeri bir referans olduğundan, metodun aldığı kopya da aynı diziyi gösterir. Sonuçta metot dizinin elemanlarını değiştirebilir, ama parametreye yeni bir dizi atarsa bu çağıran taraftan görünmez. C'de diziyi geçirmek işaretçi geçirmek demekti ve boyut ayrıca taşınırdı (Bölüm 10); C#'ta `Length` dizinin içinde olduğundan ek parametre gerekmez.

▸ diziMetot.cs

```
int[] a = { 1, 2, 3 };

IkiyleCarp(a);
Console.WriteLine(string.Join(", ", a));

YeniDiziAta(a);
Console.WriteLine(string.Join(", ", a));

YeniDiziAtaRef(ref a);
Console.WriteLine(string.Join(", ", a));

int[] k = Kareler(5);
Console.WriteLine(string.Join(", ", k));

static void IkiyleCarp(int[] d)
{
    for (int i = 0; i < d.Length; i++)
    {
        d[i] *= 2;              // aynı diziyi değiştirir
```

```
    }
}

static void YeniDiziAta(int[] d)
{
    d = new int[] { 0, 0, 0 };  // yalnızca yerel kopya yeni diziyi gösterir
    d[0] = 100;
}

static void YeniDiziAtaRef(ref int[] d)
{
    d = new int[] { 100, 0, 0 };    // çağıranın değişkeni de değişir
}

static int[] Kareler(int n)
{
    int[] sonuc = new int[n];
    for (int i = 0; i < n; i++)
    {
        sonuc[i] = (i + 1) * (i + 1);
    }
    return sonuc;               // dizi metot bitince de yaşar
}
```

```
ÇIKTI
2, 4, 6
2, 4, 6
100, 0, 0
1, 4, 9, 16, 25
```

Satırları sırayla yorumlayalım:

- `IkiyleCarp`: `d` ile `a` aynı diziyi gösterir. Elemanlardaki değişiklik çağıranda görünür.
- `YeniDiziAta`: `d = new ...` yalnızca metodun kendi `d` değişkenini yeni bir diziye yöneltir. `a` hâlâ eski diziyi gösterir; yeni dizi metot bitince çöp olur.
- `YeniDiziAtaRef`: parametre `ref` olduğu için `d`, çağıranın `a` değişkeninin kendisidir (Bölüm 46). Atama çağıranda da görünür.
- `Kareler`: metodun yerel değişkeni `sonuc` metot bitince yok olur, ama dizi nesnesi öbekte yaşamaya devam eder; çünkü artık çağıran taraftaki `k` onu gösterir. C'deki "yerel dizinin adresini döndürme" hatası (Bölüm 13) burada oluşmaz.

> **◆ Kritik Nokta**
>
> "C#'ta diziler referansla geçirilir" demek yanlıştır. Doğrusu: parametre değerle geçirilir, ama değerin kendisi bir referans olduğu için metot **aynı diziye** erişir. Elemanı değiştirmek dışarıdan görünür; parametreyi başka bir diziye yöneltmek görünmez. Görünmesini istiyorsak `ref` gerekir.

Bir metoda değişken sayıda değer geçirmek için `params` ile işaretlenmiş dizi parametresi kullanılır. Bölüm 46'da gördüğümüz bu yapıda çağıran, değerleri tek tek yazar ya da hazır bir dizi verir; metot ikisini de aynı biçimde bir dizi olarak alır.

▸ paramsDizi.cs

```
Console.WriteLine(Topla(1, 2, 3));
Console.WriteLine(Topla());
int[] dizi = { 4, 5 };
Console.WriteLine(Topla(dizi));         // hazır dizi doğrudan geçirilir
```

```
static int Topla(params int[] sayilar)
{
    int toplam = 0;
    foreach (int sayi in sayilar)
    {
        toplam += sayi;
    }
    return toplam;
}
```

```
ÇIKTI
6
0
9
```

47.8 Array Sınıfı

Bütün diziler `System.Array` sınıfından türediği için bu sınıfın `static` metotlarını kullanabiliriz. C'de `qsort` ve `bsearch` (Bölüm 10), C++'ta `<algorithm>` başlığındaki `std::sort` ve `std::find` (Bölüm 36) neyse, `Array` sınıfı da C#'ta odur. `using` gerekmez; `System` ad alanı örtük olarak açıktır.

Metot	Ne yapar?
`Array.Sort(a)`	Diziyi küçükten büyüğe, yerinde sıralar; `void` döner
`Array.Reverse(a)`	Diziyi yerinde ters çevirir; `void` döner
`Array.IndexOf(a, x)`	`x`'in ilk indeksini verir; yoksa `-1`
`Array.BinarySearch(a, x)`	**Sıralı** dizide ikili arama yapar; bulunamazsa negatif değer
`Array.Fill(a, x)`	Bütün elemanlara `x` değerini atar
`Array.Clear(a, b, n)`	`b` indeksinden başlayarak `n` elemanı varsayılan değere döndürür
`Array.Copy(k, h, n)`	`k`'nin ilk `n` elemanını `h`'ye kopyalar
`Array.Resize(ref a, n)`	Diziyi `n` elemanlı yeni bir dizi ile değiştirir

▸ arrayMetotlari.cs

```
int[] a = { 42, 7, 19, 3, 25 };
Array.Sort(a);
Console.WriteLine(string.Join(", ", a));
Console.WriteLine($"25 indeksi: {Array.IndexOf(a, 25)}");
Console.WriteLine($"100 indeksi: {Array.IndexOf(a, 100)}");

Array.Reverse(a);                       // büyükten küçüğe sıralama
Console.WriteLine(string.Join(", ", a));

string[] isimler = { "Zeynep", "ali", "Can", "Ayşe" };
Array.Sort(isimler, StringComparer.Ordinal);    // karakter koduna göre
Console.WriteLine(string.Join(", ", isimler));

int[] b = new int[6];
Array.Fill(b, 7);
Array.Clear(b, 0, 2);                   // ilk iki eleman 0 olur
Console.WriteLine(string.Join(", ", b));
```

```
ÇIKTI
3, 7, 19, 25, 42
25 indeksi: 3
100 indeksi: -1
42, 25, 19, 7, 3
Ayşe, Can, Zeynep, ali
0, 0, 7, 7, 7, 7
```

`Array.Sort` ve `Array.Reverse` diziyi **yerinde** değiştirir ve bir şey döndürmez; `int[] s = Array.Sort(a);` yazımı derlenmez. Büyükten küçüğe sıralama için önce `Sort`, sonra `Reverse` çağrılır. Asıl dizinin sırası korunacaksa önce kopyası alınır: `int[] s = (int[])a.Clone(); Array.Sort(s);`.

`string` dizilerinde `Array.Sort` varsayılan olarak bilgisayarın kültür ayarına göre karşılaştırır; harf büyüklüğü ve Türkçe karakterler bu ayara bağlı olarak farklı sıralanabilir. Sonucun her bilgisayarda aynı olmasını istediğimizde yukarıdaki gibi `StringComparer.Ordinal` veririz: `A`'nın kodu `a`'dan küçük olduğundan büyük harfle başlayanlar öne gelir. Kültür ve karşılaştırma ayrıntıları Bölüm 48'dedir.

İkili arama

`Array.BinarySearch` yalnızca **sıralı** dizide doğru çalışır. Eleman bulunursa indeksini verir. Bulunamazsa, elemanın olması gereken yerin indeksinin bit düzeyindeki tersini (`~`) negatif sayı olarak döndürür. Böylece hem "yok" bilgisi hem de araya eklenecek yer tek değerle iletilir:

▸ **ikiliAramaHazir.cs**

```
int[] a = { 3, 7, 19, 25, 42 };           // sıralı
Console.WriteLine(Array.BinarySearch(a, 25));

int sonuc = Array.BinarySearch(a, 10);    // yok: 7 ile 19 arasına girerdi
Console.WriteLine($"{sonuc} {~sonuc}");

if (sonuc < 0)
{
    Console.WriteLine($"10, {~sonuc}. indekse eklenmeli");
}
```

```
ÇIKTI
3
-3 2
10, 2. indekse eklenmeli
```

`-3` değeri `~2` demektir. Sıralı olmayan bir dizide `BinarySearch` hata vermez, ama sonuç güvenilmezdir.

`Array` sınıfında koşula göre arayan `Exists`, `Find`, `FindAll` gibi metotlar da vardır; bunlar bir metodu ya da lambda ifadesini parametre alır ve Bölüm 55'ten sonra rahatça kullanılabilir. Dizilerle çalışırken en güçlü araç ise Bölüm 56'daki LINQ'tur.

47.9 Temel Dizi Algoritmaları

Dizi algoritmalarının mantığı C'de Bölüm 10'da anlatıldı; burada onları C#'ın araçlarıyla yeniden yazıyoruz. İlk örnek toplam, ortalama, en büyük ve en küçük değeri bulur. En büyük ve en küçük için başlangıç değeri olarak `0` değil **dizinin ilk elemanını** alırız; yoksa yalnızca negatif sayılardan oluşan bir dizide sonuç yanlış çıkar.

istatistik.cs

```
int[] d = { 12, -5, 47, 8, 47, -20, 3 };

int toplam = 0;
foreach (int x in d)
{
    toplam += x;
}
double ortalama = (double)toplam / d.Length;   // double'a çevirmezsek 13 çıkar

int enBuyuk = d[0], enKucuk = d[0], enBuyukIndeks = 0;
for (int i = 1; i < d.Length; i++)
{
    if (d[i] > enBuyuk)
    {
        enBuyuk = d[i];
        enBuyukIndeks = i;
    }
    if (d[i] < enKucuk)
    {
        enKucuk = d[i];
    }
}

Console.WriteLine($"Toplam: {toplam}");
Console.WriteLine($"Ortalama: {ortalama:F2}");
Console.WriteLine($"En büyük: {enBuyuk} (indeks {enBuyukIndeks})");
Console.WriteLine($"En küçük: {enKucuk}");
```

ÇIKTI

```
Toplam: 92
Ortalama: 13.14
En büyük: 47 (indeks 2)
En küçük: -20
```

47 dizide iki kez geçtiği halde `>` kullandığımız için ilk bulunanın indeksi (2) kaldı; `>=` yazsaydık sonuncunun indeksi (4) kalırdı. Bu işleri LINQ tek satırda yapar: `d.Sum()`, `d.Average()`, `d.Max()`, `d.Min()` (Bölüm 56). Algoritmayı elle yazmayı bilmek yine de gereklidir; çünkü ölçüt değiştiğinde (örneğin "en büyüğün indeksi") hazır metot yetmez.

Doğrusal arama ve sayma

Bir değeri baştan sona sırayla bakarak aramaya **doğrusal arama (linear search)** denir. Bulunamazsa geçersiz bir indeks olan `-1` döndürmek gelenektir; `Array.IndexOf` da aynısını yapar. İlk bulunca `break` ile çıkarız. Sayma ise bütün diziyi tarar:

aramaSayma.cs

```
int[] d = { 4, 9, 2, 9, 7, 9, 1 };
int aranan = 9;

int konum = -1;                     // -1: bulunamadı
for (int i = 0; i < d.Length; i++)
{
    if (d[i] == aranan)
    {
        konum = i;
        break;                      // ilkini bulunca dur
    }
```

```
}
Console.WriteLine($"{aranan} ilk olarak {konum}. indekste");

int adet = 0;
foreach (int x in d)
{
    if (x == aranan)
    {
        adet++;
    }
}
Console.WriteLine($"{aranan} dizide {adet} kez geçiyor");
Console.WriteLine(Array.IndexOf(d, aranan) == konum);
```

```
ÇIKTI
9 ilk olarak 1. indekste
9 dizide 3 kez geçiyor
True
```

Klavyeden dizi doldurmak

Bazı işler dizi olmadan yapılamaz. Ortalamanın üzerindeki notları saymak için notları iki kez dolaşmak gerekir: ilk turda ortalama henüz bilinmez. Aşağıdaki programda dizinin boyutu da çalışma zamanında belli oluyor. Girdi alma ve dönüştürme ayrıntıları Bölüm 43'tedir.

ortalamaUstu.cs

```
Console.Write("Kaç öğrenci? ");
int n = int.Parse(Console.ReadLine()!);
int[] notlar = new int[n];              // boyut çalışırken belli oldu

for (int i = 0; i < n; i++)
{
    Console.Write($"{i + 1}. öğrencinin notu: ");
    notlar[i] = int.Parse(Console.ReadLine()!);
}

int toplam = 0;
foreach (int nt in notlar)
{
    toplam += nt;
}
double ort = (double)toplam / n;

int ustunde = 0;
foreach (int nt in notlar)
{
    if (nt > ort)
    {
        ustunde++;
    }
}
Console.WriteLine($"Ortalama: {ort:F1}");
Console.WriteLine($"Ortalamanın üstünde: {ustunde} öğrenci");
```

```
ÇIKTI
Kaç öğrenci? 5
1. öğrencinin notu: 85
2. öğrencinin notu: 70
3. öğrencinin notu: 92
4. öğrencinin notu: 64
5. öğrencinin notu: 78
Ortalama: 77.8
Ortalamanın üstünde: 3 öğrenci
```

Kullanıcıya gösterilen sıra numarası `i + 1`, dizideki indeks ise `i`'dir: insanlar saymaya 1'den, diziler 0'dan başlar.

Yerinde ters çevirme, ikili arama ve sıralama

Bu üç algoritma da C'deki karşılıklarıyla aynıdır. Fark yalnızca dizinin uzunluğunu `Length` ile alıp ayrı parametre taşımamamızdır. Ters çevirmede baştaki ve sondaki elemanlar yer değiştirerek ortaya doğru ilerlenir; döngü yalnızca `Length / 2` kez döner, yoksa her çift iki kez değişir ve dizi eski haline döner. İkili aramada aralık her adımda yarıya iner. Kabarcık sıralamada komşu iki eleman yanlış sıradaysa yer değiştirir; hiç yer değiştirme olmayan bir geçiş dizinin sıralı olduğunu gösterir.

siralamaAlgo.cs

```
int[] a = { 5, 1, 4, 2, 8 };

KabarcikSirala(a);
Console.WriteLine(string.Join(", ", a));
Console.WriteLine($"5 aranıyor: {IkiliAra(a, 5)}, 6 aranıyor: {IkiliAra(a, 6)}");

TersCevir(a);
Console.WriteLine(string.Join(", ", a));

static void KabarcikSirala(int[] d)
{
    for (int tur = 0; tur < d.Length - 1; tur++)
    {
        bool degisti = false;
        for (int j = 0; j < d.Length - 1 - tur; j++)
        {
            if (d[j] > d[j + 1])
            {
                (d[j], d[j + 1]) = (d[j + 1], d[j]);    // yer değiştirme
                degisti = true;
            }
        }
        if (!degisti)
        {
            break;                      // sıralı: erken çık
        }
    }
}

static int IkiliAra(int[] d, int aranan)    // d sıralı olmalı
{
    int alt = 0, ust = d.Length - 1;
    while (alt <= ust)
    {
        int orta = alt + (ust - alt) / 2;
        if (d[orta] == aranan)
        {
```

```
            return orta;
        }
        if (d[orta] < aranan)
        {
            alt = orta + 1;
        }
        else
        {
            ust = orta - 1;
        }
    }
    return -1;
}

static void TersCevir(int[] d)
{
    for (int i = 0; i < d.Length / 2; i++)
    {
        (d[i], d[d.Length - 1 - i]) = (d[d.Length - 1 - i], d[i]);
    }
}
```

```
ÇIKTI
1, 2, 4, 5, 8
5 aranıyor: 3, 6 aranıyor: -1
8, 5, 4, 2, 1
```

Yer değiştirme için C'de geçici değişken kullanmıştık. C# demet ataması yla `(x, y) = (y, x);` yazmamıza izin verir: sağ taraf önce hesaplanır, sonra sola atanır. Bu kitapta ileride yaygın kullanacağız. `orta` hesabında `(alt + ust) / 2` yerine `alt + (ust - alt) / 2` yazdık; dizi çok büyük olduğunda toplamın `int` sınırını aşmasını önlemenin klasik yoludur. Pratikte sıralama için elle yazılmış kabarcık yerine `Array.Sort` kullanılır; kabarcık algoritmayı anlamak içindir.

47.10 Çok Boyutlu Diziler

Satır ve sütunlardan oluşan tablo ya da matris verileri için çok boyutlu dizi kullanılır. C ve C++'ta `int m[3][4]` biçiminde yazılan yapı, bellekte satır satır art arda dizilen tek bir bloktur. C#'ta iki ayrı yapı vardır ve ikisi de C'dekinden farklı yazılır. Bu bölümde **dikdörtgen dizi** `int[,]` ile başlıyoruz: tek bir nesnedir, bütün satırlar aynı uzunluktadır ve indeksler virgülle ayrılır.

```
int[,] m = new int[3, 4];     // 3 satır, 4 sütun, hepsi 0
m[1, 2] = 7;                  // 1. satır, 2. sütun
```

- `m[i, j]`: `i`. satırın `j`. sütunundaki eleman. C'deki `m[i][j]` yazımı burada geçersizdir.
- `m.GetLength(0)` satır sayısını, `m.GetLength(1)` sütun sayısını verir.
- `m.Length` **toplam eleman sayısıdır** (3 × 4 = 12); satır sayısı değildir.
- `m.Rank` boyut sayısını verir (burada 2).
- `foreach` bütün elemanları satır satır, tek bir akış gibi dolaşır.

```
▸ matris.cs
int[,] m =
{
    { 3, 8, 1, 6 },
```

```
    { 4, 0, 7, 2 },
    { 9, 5, 2, 4 }
};
int satir = m.GetLength(0), sutun = m.GetLength(1);
Console.WriteLine($"Satır: {satir}, sütun: {sutun}, toplam: {m.Length}");

for (int i = 0; i < satir; i++)
{
    int satirToplami = 0;
    for (int j = 0; j < sutun; j++)
    {
        Console.Write($"{m[i, j],4}");        // 4 genişliğinde, sağa hizalı
        satirToplami += m[i, j];
    }
    Console.WriteLine($" | {satirToplami}");
}

for (int j = 0; j < sutun; j++)               // sütun toplamları
{
    int sutunToplami = 0;
    for (int i = 0; i < satir; i++)
    {
        sutunToplami += m[i, j];
    }
    Console.Write($"{sutunToplami,4}");
}
Console.WriteLine();

int enBuyuk = m[0, 0];
foreach (int x in m)                          // tüm elemanları dolaşır
{
    if (x > enBuyuk)
    {
        enBuyuk = x;
    }
}
Console.WriteLine($"En büyük: {enBuyuk}");
```

```
ÇIKTI
Satır: 3, sütun: 4, toplam: 12
   3   8   1   6 | 18
   4   0   7   2 | 13
   9   5   2   4 | 20
  16  13  10  12
En büyük: 9
```

`{m[i, j],4}` biçimindeki virgül, hizalama belirtir (Bölüm 43). Satır toplamında dış döngü satırı sabit tutar, iç döngü sütunları gezer. Sütun toplamında roller tersine döner.

> **▲ Sık Yapılan Hata · Length ile satır sayısını karıştırmak**
>
> İç döngüyü `j < m.Length` diye yazarsak sınır sütun sayısı değil toplam eleman sayısı olur (3 × 4 matriste 12) ve program `IndexOutOfRangeException` ile durur. Kural: dış döngü `GetLength(0)`, iç döngü `GetLength(1)`.

C'de iki boyutlu diziyi fonksiyona geçirirken sütun sayısını tipe yazmak zorundaydık (`int m[][4]`, Bölüm 10). C#'ta parametre `int[,]` yazılır ve boyutlar `GetLength` ile öğrenilir; her boyuttan matris için aynı metot çalışır:

devrik.cs

```
int[,] m = { { 1, 2, 3 }, { 4, 5, 6 } };
int[,] t = Devrik(m);
Yazdir(m);
Console.WriteLine("---");
Yazdir(t);

static int[,] Devrik(int[,] m)
{
    int[,] sonuc = new int[m.GetLength(1), m.GetLength(0)];
    for (int i = 0; i < m.GetLength(0); i++)
    {
        for (int j = 0; j < m.GetLength(1); j++)
        {
            sonuc[j, i] = m[i, j];
        }
    }
    return sonuc;
}

static void Yazdir(int[,] m)
{
    for (int i = 0; i < m.GetLength(0); i++)
    {
        for (int j = 0; j < m.GetLength(1); j++)
        {
            Console.Write($"{m[i, j]} ");
        }
        Console.WriteLine();
    }
}
```

ÇIKTI

```
1 2 3
4 5 6
---
1 4
2 5
3 6
```

Üç ve daha fazla boyut aynı biçimde yazılır (`int[,,]`); ama gündelik programlarda iki boyuttan fazlasına seyrek ihtiyaç duyulur.

47.11 Düzensiz Diziler

İkinci yapı **düzensiz dizi (jagged array)** `int[][]`'dir: "dizilerin dizisi". Dış dizinin her elemanı ayrı bir `int[]` nesnesine referanstır ve satırların uzunlukları farklı olabilir. Önce yalnızca satır sayısı verilir (`new int[5][]`), satırlar sonra tek tek oluşturulur. Oluşturulmamış satırlar `null`'dır. Elemana `d[i][j]` ile ulaşılır: önce `i`. satır dizisi alınır, sonra onun `j`. elemanı. Pascal üçgeni bunun için uygun bir örnektir:

duzensiz.cs

```
int[][] pascal = new int[5][];              // yalnızca satır sayısı belli
for (int i = 0; i < pascal.Length; i++)
{
    pascal[i] = new int[i + 1];             // her satır farklı uzunlukta
    pascal[i][0] = 1;
    pascal[i][i] = 1;
```

```
    for (int j = 1; j < i; j++)
    {
        pascal[i][j] = pascal[i - 1][j - 1] + pascal[i - 1][j];
    }
}

foreach (int[] satir in pascal)
{
    Console.WriteLine($"{string.Join(" ", satir)}   ({satir.Length} eleman)");
}
```

```
ÇIKTI
1   (1 eleman)
1 1   (2 eleman)
1 2 1   (3 eleman)
1 3 3 1   (4 eleman)
1 4 6 4 1   (5 eleman)
```

Her satırın uzunluğu farklı olduğundan iç döngülerde `pascal[i].Length` kullanmak gerekir; tek bir sütun sayısı yoktur. Hazır değerle tanımlarken her satır ayrıca `new[]` ile verilir: `int[][] j = { new[] { 1, 2 }, new[] { 3, 4, 5 } };`.

Düzensiz dizilerde yüzeysel kopyanın etkisi görünür hale gelir. `Clone()` ya da `Array.Copy` dış diziyi kopyalar, ama dış dizinin elemanları satır **referansları** olduğundan satırlar paylaşılır. Gerçekten bağımsız bir kopya için her satırı ayrıca kopyalamak gerekir:

derinKopya.cs

```
int[][] a = { new[] { 1, 2 }, new[] { 3, 4, 5 } };

int[][] yuzeysel = (int[][])a.Clone();      // satırlar paylaşılır
int[][] derin = new int[a.Length][];
for (int i = 0; i < a.Length; i++)
{
    derin[i] = (int[])a[i].Clone();         // her satır ayrıca kopyalanır
}

a[0][0] = 99;
Console.WriteLine($"{yuzeysel[0][0]} {derin[0][0]}");
```

```
ÇIKTI
99 1
```

Yüzeysel kopyada `a[0][0]` değişince kopya da değişti, çünkü iki dış dizi aynı satır nesnesini gösteriyor. Derin kopyada satırlar bağımsızdır.

Hangisini seçmeli?

	Dikdörtgen `int[,]`	Düzensiz `int[][]`
Elemana erişim	`m[i, j]`	`d[i][j]`
Satır sayısı	`m.GetLength(0)`	`d.Length`
Satırın sütun sayısı	`m.GetLength(1)` (hepsi aynı)	`d[i].Length` (değişebilir)
Toplam eleman	`m.Length`	Satır satır toplanır
Bellekte	Tek nesne, satırlar art arda	Dış dizi + her satır için ayrı nesne

	Dikdörtgen `int[,]`	Düzensiz `int[][]`
Satır değiştirme	Satır tek başına alınamaz	`d[i] = new int[...]` ile satır değişir

Tablo gerçekten dikdörtgen ise (görüntü, oyun tahtası, matris) `int[,]` daha okunaklıdır. Satır uzunlukları değişiyorsa (üçgen, grafikte komşuluk listesi, ders başına farklı sayıda not) `int[][]` doğal seçimdir. Düzensiz diziler tek boyutlu dizi erişimine dayandığından çoğu zaman biraz daha hızlıdır.

Üç Dilde · İki boyutlu dizi bellekte *C · C++ · C#*

```c
int m[3][4];       // C: 12 int'lik tek blok, satır satır; m[i][j]
```

```cpp
int m[3][4];                                    // C++: C ile aynı
std::vector<std::vector<int>> v(3, std::vector<int>(4));   // satırlar ayrı bloklar
```

```csharp
int[,] m = new int[3, 4];      // C#: tek nesne, m[i, j]
int[][] d = new int[3][];      // C#: dizi referansları dizisi, d[i][j]
```

C'nin `int m[3][4]` yapısı, bellek düzeni bakımından C#'ın `int[,]` dizisine; C++'ın `vector<vector<int>>` yapısı ise C#'ın `int[][]` dizisine daha yakındır. Yazım ise üç dilde de farklıdır: C ve C++'ta `m[i][j]`, C#'ta dikdörtgen dizi için `m[i, j]`.

Bölüm Özeti

- Dizi, aynı tipten sabit sayıda elemanı tek ad altında tutar: `int[] a = new int[5];`, `int[] a = { 1, 2, 3 };`, `int[] a = [1, 2, 3];` (C# 12). Boyut sonradan değişmez; `int a[5]` yazımı C#'ta geçersizdir.
- `new` ile oluşan dizide elemanlar varsayılan değer alır: sayılar `0`, `bool` `false`, `char` `'\0'`, `string` ve sınıflar `null`. Çöp değer yoktur.
- Geçerli indeksler `0` ile `Length - 1` arasındadır. Her erişim denetlenir; geçersiz indeks çalışma zamanında `IndexOutOfRangeException` fırlatır. `Length` bir özelliktir, parantezsizdir.
- `foreach` yalnızca okumak içindir; döngü değişkenine atama derleme hatasıdır (`CS1656`). Elemanları değiştirmek için indeksli `for` kullanılır.
- `Console.WriteLine(dizi)` elemanları değil `System.Int32[]` gibi tip adını yazar; elemanları görmek için `string.Join` kullanılır.
- `^` sondan indeks verir (`a[^1]` son eleman, `^0` geçersiz); `a[b..e]` `e` hariç aralığı yeni bir dizi olarak kopyalar (C# 8).
- Dizi referans tipidir: `b = a` kopya değil takma addır. `==` içeriği değil referansı karşılaştırır; içerik için `SequenceEqual` ya da elle yazılmış döngü gerekir.
- Gerçek kopya: `Clone()`, `Array.Copy`, `[.. a]`, `a[..]`. Hepsi yüzeysel kopyadır; referans tipli elemanlar paylaşılır.
- Metoda dizi geçirmek referansın kopyasını geçirir. Elemanlardaki değişiklik çağıranda görünür; parametreye yeni dizi atamak görünmez, `ref` gerekir.

- `Array` sınıfı `Sort`, `Reverse`, `IndexOf`, `BinarySearch`, `Fill`, `Clear`, `Copy`, `Resize` metotlarını sunar. `Sort` ve `Reverse` diziyi yerinde değiştirir. `BinarySearch` sıralı dizi ister; bulamazsa `~` ile eklenecek yeri veren negatif sayı döndürür.
- En büyük/en küçük aramasına dizinin ilk elemanı ile başlanır; doğrusal arama bulamazsa `-1` döndürür. Ters çevirme döngüsü `Length / 2` kez döner. Demet ataması `(x, y) = (y, x);` yer değiştirmenin kısa yoludur.
- `int[,]` dikdörtgen dizidir: `m[i, j]`, `GetLength(0)` satır, `GetLength(1)` sütun, `Length` toplam eleman sayısıdır.
- `int[][]` düzensiz dizidir: her satır ayrı bir dizidir, uzunlukları farklı olabilir, `d[i][j]` ile erişilir. Satırlar tek tek `new` ile oluşturulmalıdır.
- C'de sınır denetimi yoktur, C++'ta `at()` ile isteğe bağlıdır, C#'ta her zaman vardır.

Alıştırmalar

1. `double[] d = new double[4];` satırından sonra `d[2]` ve `d.Length` değerleri nedir?
   A) `0.0` ve 4
   B) `null` ve 4
   C) `0` ve 3
   D) Değer verilmediği için derleme hatası
   E) `0.0` ve 3
2. `int[] a` bir dizi, `string s` bir metin olsun. Uzunluklarını veren doğru ifade çifti hangisidir?
   A) `a.Length()` ve `s.Length()`
   B) `a.Count` ve `s.Length()`
   C) `a.Size` ve `s.Count`
   D) `a.Length()` ve `s.Length`
   E) `a.Length` ve `s.Length`
3. Aşağıdaki kod ekrana ne yazdırır?

```
int[] a = { 1, 2, 3 };
int[] b = a;
b[1] = 20;
a = new int[] { 7, 8, 9 };
Console.WriteLine(b[1] + a[1]);
```

   A) 10
   B) 28
   C) 16
   D) 22
   E) 40

4. Aşağıdaki metot ve çağrı için ekrana ne yazılır?

```
static void Degistir(int[] d, int n)
{
    d[0] = 50;
    n = 50;
}
// çağıran kod:
```

```
int[] a = { 1, 2 };
int x = 1;
Degistir(a, x);
Console.WriteLine($"{a[0]} {x}");
```

A) `1 1`
B) `50 50`
C) `1 50`
D) `50 1`
E) Derleme hatası

5. `int[] a = { 10, 20, 30, 40, 50 };` için `a[^2]`, `a[1..3].Length` ve `a[3..].Length` değerleri sırasıyla nedir?
A) 40, 2, 2
B) 40, 3, 2
C) 30, 2, 2
D) 40, 2, 3
E) 20, 2, 2

6. Aşağıdaki tanıma göre `m.Length + m[1].Length + m[2][0]` ifadesinin değeri nedir?

```
int[][] m = { new[] { 1, 2 }, new[] { 3, 4, 5 }, new[] { 6 } };
```

A) 9
B) 11
C) 12
D) 6
E) `IndexOutOfRangeException`

7. Aşağıdaki program ekrana ne yazdırır?

```
int[] a = { 3, 8, 1, 6, 4 };
int t = 0;
for (int i = 1; i < a.Length; i += 2)
{
    t += a[i];
}
Console.WriteLine(t);
for (int i = a.Length - 1; i > 0; i--)
{
    a[i] = a[i - 1];
}
Console.WriteLine(string.Join(", ", a));
int k = 0;
foreach (int x in a)
{
    if (x > a[1])
    {
        k++;
    }
}
Console.WriteLine(k);
```

8. Aşağıdaki program ekrana ne yazdırır?

```
static void Degistir(int[] d)
{
    d[0] = d[d.Length - 1];
    d = new int[] { 9, 9, 9 };
    d[1] = 0;
}

static int[] IkiKat(int[] d)
{
    int[] s = (int[])d.Clone();
    for (int i = 0; i < s.Length; i++)
    {
        s[i] *= 2;
    }
    return s;
}

int[] a = { 1, 2, 3 };
Degistir(a);
int[] b = IkiKat(a);
int[] c = a;
c[1] = 7;
Console.WriteLine(string.Join(", ", a));
Console.WriteLine(string.Join(", ", b));
Console.WriteLine($"{a == c} {a == b}");
```

9. Aşağıdaki program ekrana ne yazdırır?

```
int[][] m = { new[] { 2, 0, 1 }, new[] { 4, 5, 3 }, new[] { 1, 7, 6 } };
int kosegen = 0, ters = 0;
for (int i = 0; i < m.Length; i++)
{
    kosegen += m[i][i];
    ters += m[i][m.Length - 1 - i];
}
Console.WriteLine($"{kosegen} {ters}");
foreach (int[] satir in m)
{
    Console.Write($"{satir[satir.Length - 1]} ");
}
Console.WriteLine();
int[] ikinci = m[1];
ikinci[0] = 0;
Console.WriteLine(m[1][0] + m.Length * m[2][0]);
```

10. Aşağıdaki program ekrana ne yazdırır?

```
int[] a = { 40, 10, 30, 20 };
int[] b = a[1..3];
Array.Sort(a);
int konum = Array.BinarySearch(a, 25);
Console.WriteLine(string.Join(", ", a));
Console.WriteLine(string.Join(", ", b));
Console.WriteLine($"{konum} {~konum}");
```

11. Aşağıdaki kodda derleme hatası veren üç satır vardır. Bu satırları bulun ve düzeltin.

```
int sayilar[3] = { 1, 2, 3 };
int[] b = new int[2];
int uzunluk = b.Length();
```

```
foreach (int x in b)
{
    x = x + 1;
}
```

12. Aşağıdaki döngü neden çalışma zamanında hata verir? Hatanın adını söyleyin ve düzeltin.

```
int[] a = { 5, 6, 7 };
for (int i = 0; i <= a.Length; i++)
{
    Console.WriteLine(a[i]);
}
```

13. `int[,] m = new int[3, 5];` dizisi için `m.Length`, `m.GetLength(0)` ve `m.GetLength(1)` değerlerini yazın. Bu dizinin iç içe iki döngüyle dolaşılması için doğru döngü sınırları nelerdir?
14. Bir `int` dizisindeki en büyük ikinci **farklı** değeri döndüren `IkinciEnBuyuk(int[] d)` metodunu yazın. Diziyi sıralamadan, tek bir döngüyle çözün; dizide en az iki farklı değer olduğunu varsayın. Program metodu iki diziyle denesin: `{ 12, 45, 7, 45, 30 }` için 30, `{ -3, -8, -1 }` için -3 yazmalıdır.
15. Bir `int` dizisini ve bir kaydırma miktarı `k` değerini alan, diziyi `k` kadar sola döndüren `SolaDondur(int[] d, int k)` metodunu yazın. Döndürmede başa dönülür: `{ 1, 2, 3, 4, 5 }` dizisi `k = 2` için `{ 3, 4, 5, 1, 2 }` olur. Yeni bir dizi döndürün, asıl diziyi değiştirmeyin.
16. Klavyeden önce `n` değerini, sonra her satırda boşlukla ayrılmış `n` sayı olan `n × n` bir matrisi okuyan program yazın. Matrisin simetrik olup olmadığını (her `i`, `j` için `m[i, j] == m[j, i]`) `static bool SimetrikMi(int[,] m)` metoduyla denetleyip sonucu yazdırın. (İpucu: bir satırı parçalamak için `satir.Split(' ')` kullanılır; ayrıntısı Bölüm 48'dedir.)

BÖLÜM 48

String, StringBuilder ve Karakterler

BU BÖLÜMDE

- `string` tipinin yapısını, değiştirilemezliğini ve `==` karşılaştırmasının neden içeriğe baktığını açıklamayı
- `Length`, `IndexOf`, `Substring`, `Replace`, `Trim`, `Split`, `Join`, `PadLeft` gibi temel üyeleri doğru kullanmayı
- Karşılaştırmada `StringComparison` seçeneklerini ve Türkçe `i`/`İ`, `ı`/`I` sorununu ele almayı
- Ham metin sabitlerinin girinti kuralını ve `StringBuilder` ile verimli metin oluşturmayı
- `char` metotlarını, `char` aritmetiğini ve UTF-16 ile bayt sayısı arasındaki farkı kavramayı
- Palindrom, kelime sayma ve Sezar şifresi gibi klasik metin algoritmalarını C# ile yazmayı

Bölüm 42'de `string` tipini tanıdık, Bölüm 43'te metni biçimlendirmeyi ve klavyeden okumayı öğrendik. Bu bölümde metnin kendisiyle çalışacağız: parçalarını alacak, içinde arayacak, ayıracak, birleştirecek ve dönüştüreceğiz.

C'de metin, `'\0'` ile biten bir `char` dizisiydi ve her işlem `<string.h>` işlevleriyle, bellek taşmasını kendimiz denetleyerek yapılırdı (Bölüm 11). C++'ta `std::string` bu yükü aldı ve değiştirilebilir bir nesne oldu (Bölüm 27). C# de nesne tabanlı bir `string` sunar; ama C++'tan önemli bir farkla: C#'ın `string`'i **değiştirilemezdir**. Bu bölümün büyük kısmı, bu tek tasarım kararının sonuçlarını anlamakla ilgilidir.

48.1 string Tipinin Yapısı

`string`, .NET sınıf kütüphanesindeki `System.String` sınıfının kısa adıdır (Bölüm 42). Bir `string` değişkeni metnin kendisini değil, bellekte duran bir `string` nesnesine giden **referansı** tutar. Nesnenin içinde karakterler (her biri bir UTF-16 kod birimi) ardışık olarak durur ve 0'dan başlayarak numaralanır. Bu numaraya indeks (index) denir.

Karakter	C	#	boşluk	1	2
İndeks	0	1	2	3	4

Yukarıdaki `"C# 12"` metni için `Length` değeri 5, son indeks ise 4'tür: son indeks her zaman `Length - 1`'dir. C'deki gibi sondaki `'\0'` karakteri yoktur; uzunluk nesnenin içinde ayrıca saklanır, bu yüzden `Length` metni baştan sona taramaz (C'deki `strlen` ise tarar).

stringYapisi.cs

```
string ad = "C# 12";
Console.WriteLine(ad.Length);
Console.WriteLine(ad[0]);
```

```
Console.WriteLine(ad[ad.Length - 1]);
Console.WriteLine(ad[^1]);

string bos = "";
Console.WriteLine(bos.Length);
Console.WriteLine(new string('-', 10));
Console.WriteLine(string.IsNullOrEmpty(bos));
```

```
ÇIKTI
5
C
2
2
0
----------
True
```

`ad[i]` ifadesi `i`. indeksteki karakteri bir `char` olarak verir. `ad[^1]` yazımı sondan birinci karakteri ifade eder (C# 8'in indeks operatörü; Bölüm 47'de ayrıntılı işlenir). Boş metin `""` ya da `string.Empty` ile yazılır. `new string('-', 10)` ise aynı karakteri verilen sayıda tekrarlayan bir metin üretir.

> **▲ Sık Yapılan Hata · Length bir özelliktir**
>
> Dizilerde olduğu gibi metinde de uzunluk bir **özelliktir** (property), metot değildir. `s.Length()` yazarsak program derlenmez.
>
> ▸ lengthHata.cs
>
> ```
> string s = "C#";
> Console.WriteLine(s.Length());
> ```
>
> ```
> DERLEYİCİ ÇIKTISI
> lengthHata.cs(2,21): error CS1955: Non-invocable member 'string.Length' cannot be used like
> a method.
> ```

Sınırların dışına çıkmak C'deki gibi sessiz bir bellek bozulması yapmaz: `s[10]` ifadesi çalışma zamanında `IndexOutOfRangeException` fırlatır. Aynı hata, benzer biçimde `Substring` için `ArgumentOutOfRangeException` olarak görünür. İstisnaları Bölüm 53'te ayrıntılı ele alacağız; şimdilik bunların program çöküşü demek olduğunu bilmemiz yeter.

null, boş ve boşluk

Bir `string` değişkeninin üç farklı "boş" hali olabilir: hiçbir nesneyi göstermeyen `null`, sıfır karakterli `""` ve yalnızca boşluklardan oluşan `" "`. Bunları ayırt etmek için iki yardımcı metot vardır:

▸ bosMetinler.cs

```
string?[] ornekler = { null, "", "   ", "abc" };

foreach (string? s in ornekler)
{
    string gorunum = s is null ? "null" : "\"" + s + "\"";
    Console.WriteLine($"{gorunum,-7} {string.IsNullOrEmpty(s),-6} " +
                      $"{string.IsNullOrWhiteSpace(s)}");
}
```

```
ÇIKTI
null    True   True
""      True   True
"   "   False  True
"abc"   False  False
```

`string.IsNullOrEmpty(s)` metin `null` ya da `""` ise, `string.IsNullOrWhiteSpace(s)` bunlara ek olarak yalnızca boşluk, sekme ve satır sonundan oluşan metinler için de `true` verir. Kullanıcıdan gelen girdiyi denetlerken genellikle ikincisi doğru seçimdir. `null` bir metin üzerinde `Length` ya da başka bir üye çağırırsak `NullReferenceException` oluşur; nullable referans tiplerinin bunu nasıl derleme zamanına çektiğini Bölüm 59'da göreceğiz.

48.2 Değiştirilemezlik

Bölüm 42'de gördüğümüz gibi, bir `string` nesnesi oluşturulduktan sonra içeriği hiçbir zaman değişmez. `ToUpper`, `Replace`, `Trim` gibi metotlar mevcut metni değiştirmez; **yeni** bir `string` üretip döndürür. Sonucu bir değişkene atamazsak üretilen metin kaybolur.

▸ **degismezlik.cs**

```
string s = "merhaba";
s.ToUpper();                    // sonuç atılmadı, hiçbir etkisi yok
Console.WriteLine(s);

string buyuk = s.ToUpper();     // yeni metin saklandı
Console.WriteLine($"{buyuk} {s}");

string t = s;                   // t de aynı nesneyi gösteriyor
s = s + "!";                    // s artık yeni bir nesneyi gösteriyor
Console.WriteLine($"{s} {t}");
```

```
ÇIKTI
merhaba
MERHABA merhaba
merhaba! merhaba
```

`s = s + "!"` satırında nesne değil, `s` değişkeninin gösterdiği yer değişti. `t` hâlâ eski nesneyi gösterdiği için etkilenmedi. Böylece `string`, referans tipi olduğu halde atama bakımından değer tipi gibi davranır: iki değişkeni birbirine bağlamak, birinin sonradan diğerini bozması riskini taşımaz.

> **▲ Sık Yapılan Hata · Sonucu atanmayan metot**
>
> `s.ToUpper();`, `s.Trim();` ve `s.Replace("a", "b");` satırları tek başına hiçbir şey yapmaz. Derleyici bu konuda uyarı da vermez. Doğrusu `s = s.ToUpper();` biçimindedir. C ve C++'ta metni yerinde değiştirmeye alışkın olanlar için bu, en sık düşülen tuzaklardan biridir.

Değiştirilemezliğin dört yararı vardır:

- **Güvenli paylaşım.** Aynı nesneyi birçok değişken, hatta birçok iş parçacığı (thread) aynı anda kullanabilir; kimse içeriği değiştiremeyeceği için kilitlemeye gerek kalmaz.
- **Anahtar olarak kullanım.** Bir metin sözlük gibi yapılarda anahtar yapıldığında içeriği değişmez; böylece özet değeri (hash) sabit kalır (Bölüm 54).

- **Metin havuzu.** Derleyici, programdaki aynı metin sabitlerini tek bir nesnede toplar. Buna dizi havuzlama (string interning) denir.
- **Ucuz alt işlemler.** Kopyalama gerekmediğinden metin geçirmek ucuzdur.

Bedeli ise şudur: metni parça parça büyüten döngüler her adımda yeni bir nesne üretir. Bu sorunu `StringBuilder` bölümünde ele alacağız.

İçerik eşitliği ve referans eşitliği

C'de iki metni `==` ile karşılaştırmak, adresleri karşılaştırmak demekti ve içerik için `strcmp` kullanırdık (Bölüm 11). C++'ta `std::string` bu işleci içerik için aşırı yükler (Bölüm 27). C# de aynısını yapar: `string` tipinde `==` ve `!=` **içeriği** karşılaştırır.

esitlik.cs

```
string a = "elma";
string b = "el" + "ma";                    // derleme anında birleşir
string yarim = "el";
string c = yarim + "ma";                   // çalışırken yeni nesne

Console.WriteLine(a == c);
Console.WriteLine(a.Equals(c));
Console.WriteLine(object.ReferenceEquals(a, b));
Console.WriteLine(object.ReferenceEquals(a, c));
```

ÇIKTI

```
True
True
True
False
```

`a` ve `c` ayrı nesnelerdir (son satır `False`), ama içerikleri aynı olduğu için `==` ve `Equals` `True` verir. `a` ile `b` ise tek nesnedir: `"el" + "ma"` derleme sırasında hesaplanıp aynı havuz girdisine bağlanır. İçerik eşitliği için `==` yeterlidir; `ReferenceEquals` yalnızca iki değişkenin gerçekten aynı nesneyi gösterip göstermediğini merak ettiğimizde gerekir.

Üç Dilde · Metin tipi ve eşitlik *C · C++ · C#*

	C	C++	C#
Metnin tipi	`char` dizisi (`char *`)	`std::string` (sınıf)	`string` (sınıf)
Değiştirilebilir mi?	Evet	Evet	Hayır
`a == b`	Adresleri karşılaştırır	İçeriği karşılaştırır	İçeriği karşılaştırır
İçerik karşılaştırma	`strcmp(a, b) == 0`	`a == b`	`a == b`
Uzunluk	`strlen(s)` (taranır)	`s.size()`	`s.Length` (özellik)
Karakter birimi	1 bayt	1 bayt (`char`)	2 bayt (UTF-16)
Sınır dışı erişim	Tanımsız davranış	`[]` 'de tanımsız, `at` 'te istisna	Her zaman istisna

48.3 Temel Üyeler

Aşağıdaki tablo bu bölümün başvuru listesidir. Örneklerde `s = "Merhaba"` kabul edilmiştir (`M0 e1 r2 h3 a4 b5 a6`). Hepsi yeni bir değer döndürür; `s` değişmez.

Üye	Ne yapar?	Örnek	Sonuç
`Length`	Karakter sayısı	`s.Length`	`7`
`IndexOf(x)`	`x`'in ilk konumu, yoksa `-1`	`s.IndexOf('a')`	`4`
`LastIndexOf(x)`	`x`'in son konumu	`s.LastIndexOf('a')`	`6`
`Substring(b)`	`b`'den sona kadar	`s.Substring(3)`	`"haba"`
`Substring(b, n)`	`b`'den başlayıp `n` karakter	`s.Substring(1, 3)`	`"erh"`
`Contains(x)`	`x` içinde geçiyor mu?	`s.Contains("rha")`	`true`
`StartsWith(x)`	`x` ile mi başlıyor?	`s.StartsWith("Me")`	`true`
`EndsWith(x)`	`x` ile mi bitiyor?	`s.EndsWith("ba")`	`true`
`Replace(a, b)`	Bütün `a`'ları `b` yapar	`s.Replace('a', 'e')`	`"Merhebe"`
`ToUpper()` / `ToLower()`	Büyük / küçük harf	`s.ToUpper()`	`"MERHABA"`
`Trim()`	Baş ve sondaki boşlukları atar	`" a b ".Trim()`	`"a b"`
`Insert(i, x)`	`i`'ye `x` ekler	`s.Insert(0, ">")`	`">Merhaba"`
`Remove(i, n)`	`i`'den `n` karakter siler	`s.Remove(3, 2)`	`"Merba"`
`PadLeft(n)` / `PadRight(n)`	Boşlukla `n` uzunluğa tamamlar	`"7".PadLeft(3)`	`" 7"`

Arama

`IndexOf` bir karakteri ya da bir alt metni arar ve bulduğu ilk indeksi döndürür; bulamazsa `-1` verir. İkinci bir sayı vererek aramanın başlayacağı yeri seçebiliriz. `Contains`, `StartsWith` ve `EndsWith` aynı aramanın `bool` döndüren kısa yollarıdır.

aramaMetotlari.cs

```
string s = "Merhaba dünya, merhaba herkes";

Console.WriteLine(s.IndexOf('a'));
Console.WriteLine(s.IndexOf('a', 5));
Console.WriteLine(s.LastIndexOf('a'));
Console.WriteLine(s.IndexOf("merhaba"));
Console.WriteLine(s.IndexOf("merhaba", StringComparison.OrdinalIgnoreCase));
Console.WriteLine(s.IndexOf('z'));
Console.WriteLine(s.Contains("dünya"));
Console.WriteLine(s.StartsWith("Mer") && s.EndsWith("kes"));
```

```
ÇIKTI
4
6
21
15
0
-1
True
True
```

Büyük ve küçük harf ayrımı vardır: `"merhaba"` ilk kez 15. indekste bulunur, çünkü başta büyük `M` ile yazılmış `"Merhaba"` eşleşmez. Aramayı büyük/küçük harfe duyarsız yapmak için `StringComparison.OrdinalIgnoreCase` verilir. Bu seçeneklerin ne anlama geldiğini karşılaştırma başlığında ayrıntılı göreceğiz.

> **◆ Kritik Nokta**
>
> Alt metin arayan `IndexOf("...")`, `StartsWith("...")` ve `EndsWith("...")` metotlarının `string` parametreli sürümleri varsayılan olarak **geçerli kültüre göre** karşılaştırır; `char` parametreli sürümler ise karakter koduna bakar. Programın iç mantığında (dosya adı, komut, anahtar sözcük arama) kültürün devreye girmesini istemeyiz. Bu durumda karşılaştırma türünü açıkça yazarız: `s.StartsWith("Mer", StringComparison.Ordinal)`.

Substring: ikinci parametre uzunluktur

`Substring(başlangıç, uzunluk)` metodunun ikinci parametresi bitiş indeksi **değil**, alınacak karakter sayısıdır. Bu yazım Java ve JavaScript'in `substring(başlangıç, bitiş)` biçiminden farklıdır ve sık karıştırılır. C++'taki `substr(poz, uzunluk)` ile aynı mantıktadır (Bölüm 27).

`altMetin.cs`

```
string s = "Merhaba";

Console.WriteLine(s.Substring(3));
Console.WriteLine(s.Substring(1, 3));
Console.WriteLine($"[{s.Substring(7)}]");
Console.WriteLine(s[1..4]);
Console.WriteLine(s[^3..]);
Console.WriteLine(s[..3]);
```

```
ÇIKTI
haba
erh
[]
erh
aba
Mer
```

`Substring(1, 3)` birinci indeksten başlayıp **üç karakter** alır (`erh`). `Substring(7)` metnin uzunluğuna eşit bir başlangıçla çağrıldığı için hata vermez, boş metin döndürür. Başlangıç uzunluktan büyükse ya da başlangıç ile uzunluğun toplamı metnin sonunu aşıyorsa `ArgumentOutOfRangeException` fırlatılır.

C# 8'in aralık söz dizimi (range) aynı işi daha okunur yapar: `s[1..4]` birinci indeksten **başlar**, dördüncü indeksten **önce** biter. Bitiş indeksi dahil değildir. `s[^3..]` sondan üç karakteri, `s[..3]` baştan üç karakteri alır. Aralıkları Bölüm 47'de dizilerle birlikte ayrıntılı işlemiştik; metinlerde de aynı şekilde çalışırlar.

> **▲ Sık Yapılan Hata · Substring bitiş indeksi sanılmasın**
>
> `s.Substring(1, 4)` ifadesini "1'den 4'e kadar" okursak `"erh"` bekleriz; oysa sonuç `"erha"` olur, çünkü dört **karakter** alınır. Aralık söz diziminde ise `s[1..4]` gerçekten `"erh"` verir. İki yazımın bitiş sayısı farklı anlam taşır; hangisini yazdığımızı bilmek gerekir.

Temizleme, düzenleme ve hizalama

`duzenleme.cs`

```
string ham = "   ali veli  ";
Console.WriteLine($"[{ham.Trim()}] [{ham.TrimStart()}] [{ham.TrimEnd()}]");
Console.WriteLine("007".TrimStart('0'));
Console.WriteLine("kitap".Replace('k', 'K'));
Console.WriteLine("a-b-c".Replace("-", "--"));
Console.WriteLine("abc".Insert(1, "XY"));
Console.WriteLine("abcdef".Remove(2, 3));
Console.WriteLine("abcdef".Remove(3));
Console.WriteLine("7".PadLeft(3, '0'));
Console.WriteLine("ad".PadRight(6, '.') + "|");
```

```
ÇIKTI
[ali veli] [ali veli  ] [   ali veli]
7
Kitap
a--b--c
aXYbc
abf
abc
007
ad....|
```

Bu metotların hepsi yeni bir metin döndürür; hepsi zincirlenebilir (`ham.Trim().ToUpper()`). `Trim` varsayılan olarak boşluk, sekme ve satır sonu karakterlerini atar; parametre olarak karakterler verirsek yalnızca onları siler (`TrimStart('0')`). `Replace` aranan metni bulamazsa özgün metni aynen döndürür. `PadLeft` ve `PadRight`, Bölüm 43'te gördüğümüz `{x,10}` hizalamasının metot karşılığıdır; ikisi aynı sonucu verir, hizalama biçim metninin içinde, `PadLeft` ise bağımsız bir `string` üzerinde çalışır.

48.4 Bölme ve Birleştirme

Bölüm 43'te `ReadLine()` satırın tamamını tek metin olarak getirdiğini ve bir satırdaki birden çok değeri ayırmak için metni bölmemiz gerektiğini söylemiştik. Bunun aracı `Split` metodudur: verilen ayraçlara göre metni parçalar ve bir `string[]` döndürür. `string.Join` ise tersini yapar, bir koleksiyonun elemanlarını araya ayraç koyarak tek metinde birleştirir.

`bolBirlestir.cs`

```
Console.Write("Sayıları yazın: ");
string satir = Console.ReadLine() ?? "";
string[] parcalar = satir.Split(' ', StringSplitOptions.RemoveEmptyEntries);

int toplam = 0;
int gecerli = 0;
foreach (string parca in parcalar)
{
```

```
    if (int.TryParse(parca, out int sayi))
    {
        toplam += sayi;
        gecerli++;
    }
    else
    {
        Console.WriteLine($"Atlandı: {parca}");
    }
}
Console.WriteLine($"{parcalar.Length} parça, {gecerli} geçerli, " +
                  $"toplam {toplam}");
Console.WriteLine(string.Join(" + ", parcalar));
```

```
ÇIKTI
Sayıları yazın: 12 7  30 x5 5
Atlandı: x5
5 parça, 4 geçerli, toplam 54
12 + 7 + 30 + x5 + 5
```

Girdide iki sayı arasında çift boşluk vardı. `StringSplitOptions.RemoveEmptyEntries` seçeneği bu yüzden oluşacak boş parçayı atar; seçeneği vermeseydik dizide bir de `""` elemanı bulunurdu ve `TryParse` onu da geçersiz sayıp "Atlandı: " yazdırırdı. `TryParse` ile `out` değişken kalıbını Bölüm 42 ve 46'dan hatırlıyoruz; `x5` gibi bozuk girdi programı çökertmez, atlanır.

`Split` birden çok ayraç kabul eder ve parçaları temizleyebilir:

splitSecenekleri.cs

```
string liste = "elma, armut ;kiraz,,muz ";

string[] a = liste.Split(',');
string[] b = liste.Split(new[] { ',', ';' });
string[] c = liste.Split(new[] { ',', ';' },
    StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries);

Console.WriteLine($"{a.Length} {b.Length} {c.Length}");
Console.WriteLine(string.Join("|", c));

Console.WriteLine("".Split(' ').Length);
Console.WriteLine("".Split(' ', StringSplitOptions.RemoveEmptyEntries).Length);
Console.WriteLine("a:b:c:d".Split(':', 2)[1]);
```

```
ÇIKTI
4 5 4
elma|armut|kiraz|muz
1
0
b:c:d
```

Üç çağrıyı sırayla inceleyelim:

- `Split(',')` yalnızca virgüle göre böler; `;` ayraç sayılmadığı için `" armut ;kiraz"` tek parça kalır. Ardışık iki virgül (`",,"`) arasında boş bir parça üretir; toplam dört parça çıkar.
- `Split(new[] { ',', ';' })` her iki karakteri ayraç sayar ve beş parça verir (bir tanesi boştur).
- `RemoveEmptyEntries` boş parçaları, `TrimEntries` (.NET 5 ile geldi) ise her parçanın baş ve sonundaki boşlukları atar. İkisi `|` ile birleştirilir.

Boş metni bölmek şaşırtıcı bir sonuç verir: `"".Split(' ')` sıfır elemanlı değil, **tek elemanlı** bir dizi (`{ "" }`) döndürür. Kelime sayarken bu yüzden `RemoveEmptyEntries` kullanmak gerekir. Son satırdaki `Split(':', 2)` en çok iki parça üretir; kalan metin ikinci parçada bütün olarak kalır. `anahtar:değer` gibi satırları ayırmak için kullanışlıdır.

`string.Join` yalnızca `string` dizilerine değil, her koleksiyona uygulanır; elemanlar metne çevrilir:

```
int[] sayilar = { 3, 5, 8 };
Console.WriteLine(string.Join(", ", sayilar));     // 3, 5, 8
Console.WriteLine(string.Join("-", "a", "b", "c")); // a-b-c
Console.WriteLine(string.Concat("ab", "cd", "ef")); // abcdef
```

`string.Concat` ayraç koymadan birleştirir. Birkaç parçayı birleştirmek için `+`, değer yerleştirmek için `$"..."`, bir koleksiyonu ayraçla birleştirmek için `string.Join` kullanırız. Döngü içinde sürekli büyüyen metinlerde ise `StringBuilder` gerekir; ona birazdan geleceğiz.

48.5 Karşılaştırma ve Kültür

İki metni karşılaştırmanın iki ayrı sorusu vardır: **eşit mi** ve **hangisi önce gelir**. Eşitlik için `==`, `Equals` ve `string.Equals` kullanılır. Sıralama için `CompareTo` ve `string.Compare` kullanılır; bunlar negatif bir sayı, sıfır ya da pozitif bir sayı döndürür. Yalnızca işaretine bakmak gerekir: dönen değerin `-1` ya da `1` olacağı garanti değildir.

> **Tanım: Sıra karşılaştırması**
>
> Ordinal karşılaştırma, metinleri karakter karakter, karakterlerin sayısal kodlarına göre karşılaştırır; dil kurallarına bakmaz. Kültüre duyarlı karşılaştırma ise seçilen dilin alfabe sırasını, büyük/küçük harf eşleşmelerini ve özel harf kurallarını uygular.

Karşılaştırma türünü `StringComparison` sabitleriyle seçeriz:

Sabit	Anlamı
`Ordinal`	Karakter koduna göre; büyük/küçük harf ayrımı var
`OrdinalIgnoreCase`	Karakter koduna göre; büyük/küçük harf ayrımı yok
`CurrentCulture` / `CurrentCultureIgnoreCase`	Geçerli kültürün kurallarıyla
`InvariantCulture` / `InvariantCultureIgnoreCase`	Değişmez kültürün kurallarıyla

▸ karsilastirma.cs

```
string a = "elma";
string b = "Elma";

Console.WriteLine(a == b);
Console.WriteLine(a.Equals(b, StringComparison.OrdinalIgnoreCase));
Console.WriteLine(string.Compare(a, b, StringComparison.Ordinal) > 0);
Console.WriteLine(
    string.Compare("elma", "ekmek", StringComparison.Ordinal) > 0);
Console.WriteLine(Math.Sign("armut".CompareTo("elma")));
```

```
ÇIKTI
False
True
True
True
-1
```

`a == b` büyük/küçük harfe duyarlıdır (`False`). `OrdinalIgnoreCase` ile ikisi eşit sayılır. Ordinal sıralamada küçük `e` (kod 101), büyük `E`'den (kod 69) sonra gelir; bu yüzden üçüncü satır `True` verir. `"elma"` ile `"ekmek"` ilk kez ikinci karakterde ayrılır; `l` (108), `k`'dan (107) büyüktür. Son satırda kültüre duyarlı `CompareTo` kullandık: `"armut"` alfabede `"elma"`dan önce geldiği için sonucun işareti eksidir.

Türkçe harfler ve kültür

Büyük/küçük harf dönüşümü ve alfabe sırası dile bağlıdır. Türkçede noktalı ve noktasız iki ayrı `i` vardır: `i` harfinin büyüğü `İ`, `ı` harfinin büyüğü `I`'dır. İngilizce kurallarda ise `i` harfinin büyüğü `I` olur. Parametresiz `ToUpper()` ve `ToLower()` **geçerli kültüre** göre çalışır; bu da aynı kodun iki bilgisayarda farklı sonuç vermesine yol açabilir.

turkceI.cs

```
using System.Globalization;

CultureInfo tr = new CultureInfo("tr-TR");

Console.WriteLine("istanbul".ToUpper(tr));
Console.WriteLine("istanbul".ToUpperInvariant());
Console.WriteLine("ISPARTA".ToLower(tr));
Console.WriteLine("ISPARTA".ToLowerInvariant());
Console.WriteLine(tr.TextInfo.ToTitleCase("ankara istanbul"));

string komut = "FILE";
Console.WriteLine(komut.ToLower(tr) == "file");
Console.WriteLine(komut.ToLowerInvariant() == "file");
Console.WriteLine(komut.Equals("file", StringComparison.OrdinalIgnoreCase));
```

```
ÇIKTI
İSTANBUL
ISTANBUL
ısparta
isparta
Ankara İstanbul
False
True
True
```

Türkçe kültürle `"FILE".ToLower()` sonucu `"fıle"` olur. Bu yüzden "komutu küçük harfe çevirip `"file"` ile karşılaştır" mantığı, Türkçe ayarlı bir bilgisayarda sessizce bozulur. Bu, yazılım dünyasında "Türkiye sorunu" (Turkish-I problem) diye bilinen yaygın bir hatadır. `ToUpperInvariant` ve `ToLowerInvariant` kültürden bağımsız çalışır; `OrdinalIgnoreCase` ise dönüşüm yapmadan, doğrudan büyük/küçük harf farkını yok sayarak karşılaştırır.

> ◆ **Kritik Nokta**
>
> Metinle ne yaptığımıza göre iki kural uygularız:
>
> - **Programın iç mantığı** için (komut, anahtar sözcük, dosya uzantısı, protokol alanı karşılaştırması) `StringComparison.Ordinal` ya da `OrdinalIgnoreCase` kullanırız. `ToUpperInvariant()` / `ToLowerInvariant()` de bu gruptadır.
> - **Kullanıcıya gösterilen** metinde (ad yazımı, başlık, sıralı liste) kültüre duyarlı yöntemleri tercih ederiz.

Alfabe sırası için de aynı ayrım geçerlidir. Türkçe sözlük sırası `a b c ç d e f g ğ h ı i j k l m n o ö p r s ş t u ü v y z` biçimindedir; ordinal sıralama ise karakter kodlarına bakar ve `ç`, `ı`, `ş` gibi harfleri bütün ASCII harflerin ardına atar.

▸ siralamaTr.cs

```
using System.Globalization;

string[] kelimeler =
    { "şeker", "iyi", "çay", "ılık", "zeytin", "ağaç", "sıcak", "cam" };

string[] sozluk = (string[])kelimeler.Clone();
Array.Sort(sozluk, StringComparer.Create(new CultureInfo("tr-TR"), false));

string[] kod = (string[])kelimeler.Clone();
Array.Sort(kod, StringComparer.Ordinal);

Console.WriteLine(string.Join(" ", sozluk));
Console.WriteLine(string.Join(" ", kod));
```

ÇIKTI

```
ağaç cam çay ılık iyi sıcak şeker zeytin
ağaç cam iyi sıcak zeytin çay ılık şeker
```

`StringComparer.Create` kültür ve büyük/küçük harf duyarlılığı verilerek bir karşılaştırıcı üretir; `Array.Sort`, bu karşılaştırıcıyı ikinci parametre olarak alır (Bölüm 47). C'nin `strcmp` işlevi ve C++'ın `std::string::operator<` her ikisi de bayt değerlerine, yani ordinal karşılaştırmaya bakar (Bölüm 11 ve 27). C#'ta kültüre duyarlı sıralamayı seçmek ise tek satırdır.

48.6 Çok Satırlı ve Ham Metin Sabitleri

Bölüm 43'te düz metni (`@"..."`) ve ham metni (`"""..."""`) tanıdık. Burada ham metnin satır ve girinti kurallarını tamamlayalım; çok satırlı SQL, HTML ve JSON metinlerinde bu kurallar gerekir.

Çok satırlı bir ham metinde açan `"""` kendi satırında durur, metin sonraki satırdan başlar ve kapatan `"""` yine kendi satırında yer alır. İlk ve son satır sonu metne girmez. Kapatan `"""` işaretinin soluna kaç boşluk koyduysak, o kadar girinti **her satırdan silinir**. Böylece metni kodun girintisine uydurarak yazabiliriz; çıktıya bu girinti geçmez.

▸ hamMetinGirinti.cs

```
string ad = "Elif";
int puan = 85;

string rapor = $"""
    Öğrenci: {ad}
```

```
    Puan: "{puan}"
      (girintili satır)
    """;
Console.WriteLine(rapor);

string json = $$"""
    {
      "ad": "{{ad}}",
      "puan": {{puan}}
    }
    """;
Console.WriteLine(json);
```

```
ÇIKTI
Öğrenci: Elif
Puan: "85"
  (girintili satır)
{
  "ad": "Elif",
  "puan": 85
}
```

Kapatan `"""` dört boşluk girintili olduğu için her satırdan dört boşluk silindi; "girintili satır" ise altı boşlukla başladığından iki boşluğu kaldı. İlk metinde `$` tek olduğu için değerler tek süslü parantezle yerleştirildi. JSON örneğinde `{` ve `}` zaten metnin parçası olduğundan `$$` yazdık; bu durumda değer yerleştirmek için **çift** süslü parantez gerekir (`{{ad}}`), tek süslü parantezler düz karakterdir.

> **▲ Sık Yapılan Hata · Kapatan işaretten az girintili satır**
>
> Çok satırlı ham metinde herhangi bir satırın girintisi kapatan `"""` satırından azsa program derlenmez:

hamGirinti.cs

```
string s = """
    Birinci
  İkinci
    """;
Console.WriteLine(s);
```

```
DERLEYİCİ ÇIKTISI

hamGirinti.cs(3,1): error CS8999: Line does not start with the same whitespace as the
closing line of the raw string literal.
```

Ham metinde, içeride geçen en uzun art arda tırnak dizisinden **daha uzun** bir tırnak dizisiyle açıp kapatma yapılır. Metnin içinde `"""` geçmesi gerekirse metni `""""` ile başlatıp bitiririz. Bu kural, metin içeriğini hiçbir kaçış dizisine ihtiyaç duymadan yazmamızı sağlar.

48.7 StringBuilder

Değiştirilemezliğin bedelini görelim. Bir döngüde metne her seferinde bir karakter ekleyen program, her adımda yeni bir `string` nesnesi üretir: önce 1 karakterlik, sonra 2, 3, 4 karakterlik metinler kopyalanır. Toplam kopyalanan karakter sayısı, eleman sayısının karesiyle orantılı büyür.

> **Tanım: StringBuilder**
>
> `System.Text` ad alanındaki `StringBuilder`, içeriği yerinde değiştirilebilen ve gerektikçe büyüyen bir karakter tamponudur. Ekleme, silme ve değiştirme işlemleri her seferinde yeni nesne üretmez. İşimiz bitince `ToString()` ile sıradan bir `string` elde ederiz.

Aynı işi iki yöntemle yapıp süresini ölçelim. Sonuçlar bilgisayardan bilgisayara değişir; bu yüzden tam sayıları değil, oranı önemseriz.

▸ birlestirmeSure.cs

```
using System.Diagnostics;
using System.Text;

const int Adet = 100_000;

Stopwatch sw = Stopwatch.StartNew();
string s = "";
for (int i = 0; i < Adet; i++)
{
    s += "x";
}
sw.Stop();
Console.WriteLine($"string +=     : {sw.ElapsedMilliseconds} ms, " +
                  $"uzunluk {s.Length}");

sw.Restart();
StringBuilder sb = new StringBuilder();
for (int i = 0; i < Adet; i++)
{
    sb.Append('x');
}
string sonuc = sb.ToString();
sw.Stop();
Console.WriteLine($"StringBuilder : {sw.ElapsedMilliseconds} ms, " +
                  $"uzunluk {sonuc.Length}");
```

```
ÇIKTI
string +=     : 1413 ms, uzunluk 100000
StringBuilder : 0 ms, uzunluk 100000
```

Bu çalıştırmada `+=` ile birleştirme yaklaşık 1,4 saniye sürdü; `StringBuilder` ise milisaniyenin altında bitti. Fark, eleman sayısı arttıkça daha da açılır. `StringBuilder` ilk tamponunu yeterli büyüklükte tutar, dolduğunda tamponu büyütür; her `Append` çoğunlukla yalnızca yeni karakterleri yazmaktan ibarettir.

Üyeler

`StringBuilder` nesnesi `new` ile oluşturulur; başlangıç metni ya da başlangıç kapasitesi verilebilir. `Append` metot zincirine olanak tanır, çünkü kendisini döndürür.

▸ stringBuilderUyeleri.cs

```
using System.Text;

StringBuilder sb = new StringBuilder("Merhaba");
sb.Append(' ').Append("dünya").Append('!');
Console.WriteLine(sb);
Console.WriteLine(sb.Length);
```

```
sb.Insert(0, ">> ");
sb.Replace("dünya", "C#");
sb.Remove(sb.Length - 1, 1);
Console.WriteLine(sb.ToString());

sb[3] = 'm';
Console.WriteLine(sb);

sb.Clear();
sb.AppendLine("Birinci satır");
sb.AppendLine($"Sayı: {42}");
sb.Append("Son");
Console.WriteLine(sb);

StringBuilder ayni = sb;
ayni.Append("!!");
Console.WriteLine(sb.ToString().EndsWith("Son!!"));
```

```
ÇIKTI
Merhaba dünya!
14
>> Merhaba C#
>> merhaba C#
Birinci satır
Sayı: 42
Son
True
```

Metotları sırayla şöyle özetleyebiliriz:

Üye	Ne yapar?
`Append(x)`	Sona ekler (`char` , `string` , sayı, `bool` ...)
`AppendLine(x)`	Ekler ve satır sonu koyar
`Insert(i, x)`	`i` konumuna ekler
`Remove(i, n)`	`i` 'den `n` karakter siler
`Replace(a, b)`	Bütün `a` 'ları `b` yapar (yerinde)
`Clear()`	İçeriği boşaltır
`Length`	Karakter sayısı; atanarak kısaltılabilir
`sb[i]`	`i` . karakteri okur **ya da değiştirir**
`ToString()`	Sıradan `string` üretir

`sb[3] = 'm'` satırına dikkat edelim: `string` 'te derlenmeyen bu işlem `StringBuilder` 'da serbesttir; çünkü tampon değiştirilebilir. `AppendLine` satır sonunu `Environment.NewLine` ile koyar (Windows'ta `\r\n` , Linux ve macOS'ta `\n`).

Son iki satır, `StringBuilder` 'ın `string` 'ten farklı olarak **gerçekten** bir referans gibi davrandığını gösterir. `ayni = sb` ataması yeni bir tampon üretmez, aynı tamponu gösteren ikinci bir değişken yaratır; birinden yapılan değişiklik diğerinde de görünür. `string` 'te ise iki değişkeni bağlamak güvenliydi, çünkü kimse nesneyi değiştiremezdi.

Kapasiteyi önceden bildiğimizde `new StringBuilder(1000)` yazarak tamponu baştan o büyüklükte ayırabilir, büyütme maliyetinden kaçınabiliriz. `Capacity` özelliği mevcut kapasiteyi gösterir; bu değerin tam büyüme düzeni .NET'in iç ayrıntısıdır ve ona güvenmemeliyiz.

◆ Kritik Nokta

`StringBuilder` her yerde `string`'in yerine geçmez. Birkaç parçayı birleştirmek için `+` ya da `$"..."` hem daha okunur hem yeterince hızlıdır: derleyici `a + b + c` ifadesini tek bir `string.Concat(a, b, c)` çağrısına çevirir ve tek nesne üretir. `StringBuilder` şu durumlarda gerekir:

- **Döngü içinde** büyüyen metinler (yüzlerce, binlerce ekleme).
- Parça sayısı çalışma zamanında belli olan ve **yinelenen** birleştirmeler.
- Metnin ortasına ekleme, silme ya da karakter değiştirme gibi yerinde işlemler.

Elimizde hazır bir koleksiyon varsa `string.Join` zaten tek seferde, verimli birleştirir.

Üç Dilde · Metne ekleme yapmak *C · C++ · C#*

	C	C++	C#
Ekleme	`strcat(hedef, kaynak)`	`s += "..."`	`s += "..."` (yeni nesne) ya da `sb.Append(...)`
Bellek sorumluluğu	Programcıda; tampon yetmezse taşma	`std::string` büyür	Çöp toplayıcı; `StringBuilder` büyür
Döngüde maliyet	Her `strcat` metnin sonunu baştan tarar	Amortize sabit (yerinde büyür)	`+=` : karesel; `StringBuilder` : amortize sabit
Kopya	Elle (`strcpy`)	Değer semantiği (kopya)	`string` paylaşılır; `StringBuilder` referanstır

```c
char s[32] = "Merhaba";
strcat(s, ", dunya");        // tamponun yeteceğini biz garanti ederiz
```

```cpp
std::string s = "Merhaba";
s += ", dunya";              // yerinde büyür
```

```csharp
StringBuilder sb = new StringBuilder("Merhaba");
sb.Append(", dünya");        // yerinde büyür
string s = sb.ToString();    // son hali sıradan string yap
```

48.8 char: Karakterlerle Çalışmak

Bir `string`'i `foreach` ile dolaşırsak elemanlar `char` olarak gelir. `char`, Bölüm 42'de gördüğümüz gibi 2 baytlık bir UTF-16 kod birimidir ve sayı gibi davranabilir. `System.Char` sınıfı, bir karakterin türünü sorgulayan statik metotlar sunar; C'deki `<ctype.h>` (Bölüm 11) ve C++'taki `<cctype>` (Bölüm 27) işlevlerinin karşılığıdır.

Metot	`true` verir, eğer karakter...
`char.IsDigit(c)`	bir onluk rakamsa
`char.IsLetter(c)`	bir harfse (`ş` , `ğ` , `ç` dahil)

Metot	`true` verir, eğer karakter...
`char.IsLetterOrDigit(c)`	harf ya da rakamsa
`char.IsUpper(c)` / `char.IsLower(c)`	büyük / küçük harfse
`char.IsWhiteSpace(c)`	boşluk, sekme, satır sonu gibi bir boşluk karakteriyse
`char.IsPunctuation(c)`	bir noktalama işaretiyse
`char.ToUpper(c)` / `char.ToLower(c)`	(dönüşüm) büyük / küçük harf karşılığını verir
`char.GetNumericValue(c)`	(dönüşüm) rakamın sayı değerini `double` verir, rakam değilse `-1`

karakterSiniflari.cs

```
string metin = "Merhaba Dünya 2025! Çok güzel.";
int harf = 0, rakam = 0, bosluk = 0, noktalama = 0, buyuk = 0;

foreach (char c in metin)
{
    if (char.IsLetter(c))
    {
        harf++;
    }
    if (char.IsDigit(c))
    {
        rakam++;
    }
    if (char.IsWhiteSpace(c))
    {
        bosluk++;
    }
    if (char.IsPunctuation(c))
    {
        noktalama++;
    }
    if (char.IsUpper(c))
    {
        buyuk++;
    }
}

Console.WriteLine($"Harf: {harf}, rakam: {rakam}, boşluk: {bosluk}");
Console.WriteLine($"Noktalama: {noktalama}, büyük harf: {buyuk}");
```

```
ÇIKTI
Harf: 20, rakam: 4, boşluk: 4
Noktalama: 2, büyük harf: 3
```

Bu metotlar Unicode'u tanır: `Ç` büyük harf, `ü` harf sayılır. C'nin `isalpha` işlevi ise (varsayılan "C" yerel ayarında) yalnızca ASCII harfleri tanırdı; Türkçe harfler için çok baytlı kodlamayla uğraşmak gerekirdi (Bölüm 11).

`char.IsDigit` de Unicode rakamlarını (örneğin Arapça-Hint rakamlarını) kabul eder. Yalnızca `'0'` ile `'9'` arasındaki ASCII rakamları istiyorsak `c >= '0' && c <= '9'` yazarız; .NET 7'den itibaren `char.IsAsciiDigit(c)` de vardır.

Karakter aritmetiği

`char` aslında bir sayıdır ve aritmetik işlemlere girebilir; ama C# tiplere C'den daha duyarlıdır. Bölüm 42'deki `char + int` kuralını bir araya getirelim:

karakterAritmetigi.cs

```
char c = 'a';
Console.WriteLine(c + 1);
Console.WriteLine((char)(c + 1));
c++;
Console.WriteLine(c);

Console.WriteLine('7' - '0');
Console.WriteLine(char.GetNumericValue('7'));
Console.WriteLine((int)'A');
Console.WriteLine((char)97);

Console.WriteLine("" + 'a' + 'b');
Console.WriteLine('a' + 'b' + "");
Console.WriteLine(char.ToUpper('ş'));
Console.WriteLine(char.IsDigit('٢'));
```

ÇIKTI

```
98
b
b
7
7
65
a
ab
195
Ş
True
```

Satırları yorumlayalım:

- `c + 1` ifadesi `int` verir (98). Karakter olarak görmek için `(char)` dönüşümü şarttır.
- `c++` ve `c += 1` ise `char` değişkende kalır; derleyici bu işleçlerde örtük dönüşümü kendisi yapar.
- `'7' - '0'` rakam karakterini sayıya çevirmenin klasik yoludur.
- `"" + 'a' + 'b'` metin birleştirmesidir (`ab`). `'a' + 'b' + ""` ise önce iki `char`'ı **toplar** (97 + 98 = 195), sonra metne çevirir. Soldan sağa işlemin getirdiği tuzak burada da geçerlidir.
- `٢` bir Arapça-Hint rakamıdır; `IsDigit` onu da rakam sayar.

▲ Sık Yapılan Hata · char + int işleminin sonucu int'tir

`char d = c + 1;` satırı derlenmez; sonuç `int` 'tir ve dar bir tipe örtük dönüşüm yoktur:

charHata.cs

```
char c = 'a';
char d = c + 1;
```

DERLEYİCİ ÇIKTISI

```
charHata.cs(2,10): error CS0266: Cannot implicitly convert type 'int' to 'char'. An
explicit conversion exists (are you missing a cast?)
```

Doğrusu `char d = (char)(c + 1);` biçimindedir. Sabit ifadelerde (`char d = 'a' + 1;`) derleyici değerin sığdığını bildiği için dönüşümü kendisi yapar.

string ile char dizisi arasında dönüşüm

`string` değiştirilemez, ama `char[]` değiştirilebilir. Bir metinde karakterleri yerinde oynatmamız gerekiyorsa metni `ToCharArray()` ile diziye çevirir, diziyi düzenler ve `new string(dizi)` ile yeniden metne dönüştürürüz. C'de metin zaten bir `char` dizisiydi; burada o davranışı açıkça seçmiş oluruz.

karakterDizisi.cs

```
string s = "merhaba";

char[] harfler = s.ToCharArray();
harfler[0] = 'M';
string yeni = new string(harfler);

Array.Reverse(harfler);
string ters = new string(harfler);

Console.WriteLine($"{s} {yeni} {ters}");
Console.WriteLine(string.Concat(harfler).Length);
```

ÇIKTI

```
merhaba Merhaba abahreM
7
```

Dizi kopyası alındığı için `s` değişmeden kalır. `Array.Reverse` dizinin yerinde ters çevrilmesini sağlar; `StringBuilder`'da bir `Reverse` metodu yoktur, ters çevirme için bu kalıbı kullanırız. Bölüm 56'da göreceğimiz LINQ ile `new string(s.Reverse().ToArray())` de aynı işi yapar.

UTF-16, bayt ve görünen karakter

C'de `strlen("şeker")` UTF-8 kaynak dosyada 6 döndürür; çünkü `ş` iki bayttır (Bölüm 11). C#'ın `Length` değeri bayt değil, **UTF-16 kod birimi** sayar. Türkçe harflerin hepsi tek kod birimine sığar, dolayısıyla `"şeker".Length` 5'tir. Ama Unicode'un 65 535'ten büyük kod noktaları (örneğin bazı simgeler ve tarihsel yazılar) iki kod birimiyle, bir yedek çiftiyle (surrogate pair) saklanır. Bölüm 42'de verdiğimiz sözü yerine getirelim.

unicodeSayim.cs

```
using System.Globalization;
using System.Text;

string a = "şeker";
Console.WriteLine(a.Length);
Console.WriteLine(Encoding.UTF8.GetByteCount(a));

string e = "a\U0001F600b";
Console.WriteLine(e.Length);
Console.WriteLine(char.IsSurrogate(e[1]));
Console.WriteLine(e.EnumerateRunes().Count());

string bilesik = "é";
Console.WriteLine(bilesik.Length);
Console.WriteLine(new StringInfo(bilesik).LengthInTextElements);
```

```
ÇIKTI
5
6
4
True
3
2
1
```

Üç farklı sayım ortaya çıktı:

- `Length`, UTF-16 kod birimlerini sayar. `"a\U0001F600b"` üç karakter gibi görünür; ama aradaki simge iki kod birimi tuttuğundan `Length` 4'tür ve `e[1]` tek başına anlamsız bir yedek yarısıdır.
- `EnumerateRunes()` yedek çiftlerini bir araya getirir ve Unicode kod noktalarını (`Rune`) sayar: 3.
- `StringInfo.LengthInTextElements` kullanıcının gördüğü karakter kümelerini sayar. `e` harfi ile ayrı yazılmış vurgu işareti (◌́) iki kod noktası ama ekranda tek harftir.

Türkçe metinlerde (hazır bileşik harflerle yazıldığı sürece) bu üç sayı genellikle aynıdır. Emoji, simge ve bazı yabancı yazı sistemlerini işleyen programlarda ise `Length` ile "görünen karakter sayısı" aynı olmayabilir.

> *Tarihçe · Neden 16 bit?*
>
> C#'ın (ve Java'nın) `char` tipi 16 bitliktir; çünkü bu dillerin tasarlandığı dönemde Unicode'un 65 536 kod noktasıyla yetineceği düşünülüyordu. Standart sonradan bu sınırı aştı ve çözüm, 16 bitlik kod birimlerini koruyup fazla kod noktaları için yedek çiftleri tanımlayan UTF-16 oldu. Bugün C# dosyaları çoğunlukla UTF-8 kaydedilir; ama bellekteki `string` her zaman UTF-16'dır.

48.9 Örnek Programlar

Şimdiye kadarki üyeleri üç klasik metin algoritmasında birleştirelim. Üçünde de aynı kalıp var: karakter karakter dolaş, karar ver, sonucu biriktir.

Palindrom ve ters çevirme

Palindrom, tersten okunuşu da aynı olan metindir: `kayak`, `Anastas mum satsana`. Cümle düzeyinde kontrol için önce metni temizleriz (yalnızca harf ve rakamları, küçük harfe çevirerek alırız); sonra iki uçtan ortaya doğru karşılaştırırız. Bu yöntem ters çevrilmiş bir kopya oluşturmaz ve ilk farklı çiftte durur.

palindrom.cs

```
using System.Text;

Console.WriteLine(PalindromMu("kayak"));
Console.WriteLine(PalindromMu("Kitap"));
Console.WriteLine(PalindromMu("Ey Edip Adana'da pide ye"));
Console.WriteLine(PalindromMu("Anastas mum satsana"));
Console.WriteLine(TersCevir("Bilgisayar"));

static bool PalindromMu(string metin)
{
    StringBuilder temiz = new StringBuilder();
    foreach (char c in metin)
    {
        if (char.IsLetterOrDigit(c))
        {
            temiz.Append(char.ToLowerInvariant(c));
        }
    }

    int sol = 0;
    int sag = temiz.Length - 1;
    while (sol < sag)
    {
        if (temiz[sol] != temiz[sag])
        {
            return false;
        }
        sol++;
        sag--;
    }
    return true;
}

static string TersCevir(string s)
{
    char[] harfler = s.ToCharArray();
    Array.Reverse(harfler);
    return new string(harfler);
}
```

```
ÇIKTI
True
False
True
True
rayasigliB
```

`PalindromMu` içinde `StringBuilder`, `char.IsLetterOrDigit` ve `char.ToLowerInvariant` birlikte çalışıyor. Karşılaştırma programın iç işi olduğu için kültürden bağımsız `ToLowerInvariant` seçtik. `TersCevir` ise `ToCharArray` ve `Array.Reverse` kalıbını kullanır.

Kelime sayma ve harf türleri

Kelime saymak için metni boşluklara göre böleriz. `RemoveEmptyEntries` seçeneği, kullanıcının fazladan boşluk bırakması ya da boş satır girmesi hallerinde yanlış sonuç çıkmasını önler. Sesli ve sessiz harfleri ayırmak için Türkçenin sekiz sesli harfini bir `string` içinde tutar, `Contains` ile aranırız. Bu kez kullanıcının yazdığı metni **kültüre duyarlı** küçültüyoruz; çünkü `İ` harfinin `i`, `I` harfinin ise `ı` olması gerekir ve ikisi de sesli harftir.

metinAnalizi.cs

```
using System.Globalization;

CultureInfo tr = new CultureInfo("tr-TR");
const string Sesliler = "aeıioöuü";

Console.Write("Bir cümle yazın: ");
string cumle = Console.ReadLine() ?? "";

string[] kelimeler = cumle.Split(' ', StringSplitOptions.RemoveEmptyEntries);

int sesli = 0;
int sessiz = 0;
foreach (char c in cumle.ToLower(tr))
{
    if (Sesliler.Contains(c))
    {
        sesli++;
    }
    else if (char.IsLetter(c))
    {
        sessiz++;
    }
}

string enUzun = "";
foreach (string kelime in kelimeler)
{
    string temiz = kelime.Trim('.', ',', '!', '?', ';', ':');
    if (temiz.Length > enUzun.Length)
    {
        enUzun = temiz;
    }
}

Console.WriteLine($"Kelime: {kelimeler.Length}");
Console.WriteLine($"Sesli: {sesli}, sessiz: {sessiz}");
Console.WriteLine($"En uzun kelime: {enUzun} ({enUzun.Length} harf)");
```

```
ÇIKTI
Bir cümle yazın:   Bilgisayar   Mühendisliği, İzmir'de okunur.
Kelime: 4
Sesli: 15, sessiz: 20
En uzun kelime: Mühendisliği (12 harf)
```

Girdideki baştaki, sondaki ve kelimeler arasındaki fazla boşluklar kelime sayısını bozmadı. Kesme işareti ve noktalama harf olmadığı için hiçbir sayaca girmedi. `Trim('.', ',', ...)` kelimenin sonundaki noktalama işaretlerini atar; böylece `Mühendisliği,` kelimesi 13 değil, 12 harf olarak ölçülür.

Türkçe alfabeyle Sezar şifresi

Sezar şifresi, her harfi alfabede sabit bir miktar (anahtar) kaydırır. Alfabenin sonunda başa dönmek için kalan işleci (`%`) kullanılır. Harfin alfabedeki yerini bulmanın en doğrudan yolu, alfabeyi bir `string` olarak tutup `IndexOf` ile aramaktır. Böylece `c - 'a'` hesabının yalnızca İngilizce alfabede çalışması sorunu da ortadan kalkar: Türkçenin 29 harfini ve `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` gibi harfleri doğrudan kaydırabiliriz.

sezarSifresi.cs

```
using System.Text;

const string Kucuk = "abcçdefgğhıijklmnoöprsştuüvyz";
const string Buyuk = "ABCÇDEFGĞHIİJKLMNOÖPRSŞTUÜVYZ";

string mesaj = "Şifreli Mesaj: Çocuklar, yarın saat 10.30'da buluşalım!";
string gizli = Sifrele(mesaj, 3);
Console.WriteLine(gizli);
Console.WriteLine(Sifrele(gizli, -3));
Console.WriteLine(Sifrele("zeytin", 1));
Console.WriteLine(Sifrele("wxq", 5));

static string Sifrele(string metin, int anahtar)
{
    int n = Kucuk.Length;
    int k = ((anahtar % n) + n) % n;

    StringBuilder sb = new StringBuilder();
    foreach (char c in metin)
    {
        int kucukSira = Kucuk.IndexOf(c);
        int buyukSira = Buyuk.IndexOf(c);

        if (kucukSira >= 0)
        {
            sb.Append(Kucuk[(kucukSira + k) % n]);
        }
        else if (buyukSira >= 0)
        {
            sb.Append(Buyuk[(buyukSira + k) % n]);
        }
        else
        {
            sb.Append(c);
        }
    }
    return sb.ToString();
}
```

```
ÇIKTI
Ülhtğol Öğuçm: Freynoçt, bçtkp uççv 10.30'gç dyoyüçokö!
Şifreli Mesaj: Çocuklar, yarın saat 10.30'da buluşalım!
afzujo
wxq
```

İlk satır şifreli metni, ikinci satır çözülmüş (özgün) metni gösterir. Rakamlar, noktalama ve alfabede bulunmayan harfler (`w`, `x`, `q`) olduğu gibi kalır. Şifreyi çözmek için ayrı bir metoda gerek yoktur; `-3` anahtarı 3 geriye kaydırır.

`int k = ((anahtar % n) + n) % n;` satırı eksi anahtarları doğru işler. C#'ta kalan işleci bölünenin işaretini korur (`-3 % 29` sonucu `-3`'tür); `+ n` ve yeniden `% n` değeri her zaman 0 ile `n - 1` arasına getirir. Bu satır olmasaydı eksi anahtarda indeks negatif olur ve program `IndexOutOfRangeException` ile çökerdi.

Bölüm Özeti

- `string` bir referans tipidir ama **değiştirilemezdir**. İndeksler 0'dan başlar, son indeks `Length - 1`'dir; `Length` bir özelliktir, parantez alınmaz. Metinde `null`, `""` ve boşluk farklı durumlardır; `string.IsNullOrWhiteSpace` üçünü birden denetler.
- `ToUpper`, `Replace`, `Trim` gibi metotlar yeni metin döndürür; sonuç bir değişkene atanmazsa kaybolur. `s[0] = 'M'` derlenmez.
- `string` tipinde `==` ve `!=` **içeriği** karşılaştırır (C'deki gibi adresi değil). Referans eşitliği için `object.ReferenceEquals` kullanılır.
- `IndexOf` bulamazsa `-1` verir. `Substring(b, n)` ikinci parametre olarak **uzunluk** alır; `s[b..e]` aralığında ise `e` bitiş indeksidir ve dahil değildir. Sınır dışı kullanım `ArgumentOutOfRangeException` fırlatır.
- `Split(ayraç, RemoveEmptyEntries)` boş parçaları atar, `TrimEntries` parçaları kırpar; seçeneksiz `"".Split(' ')` tek elemanlı bir dizi verir. `string.Join` bir koleksiyonu ayraçla birleştirir.
- `Trim`, `TrimStart`, `TrimEnd`, `PadLeft`, `PadRight`, `Insert`, `Remove`, `Replace` metni düzenleyen başlıca metotlardır; hepsi zincirlenebilir.
- Program iç mantığında karşılaştırma için `StringComparison.Ordinal` ya da `OrdinalIgnoreCase`; kullanıcıya gösterilen metinde kültüre duyarlı yöntemler seçilir. `string` parametreli `IndexOf` ve `StartsWith` varsayılan olarak kültüre duyarlıdır.
- Türkçe kültürde `"i".ToUpper()` sonucu `"İ"`, `"I".ToLower()` sonucu `"ı"` olur; `"FILE".ToLower()` artık `"file"` değildir. Kültürden bağımsız dönüşüm için `ToUpperInvariant` ve `ToLowerInvariant` kullanılır.
- Çok satırlı ham metinde (`"""`) kapatan `"""` satırının girintisi her satırdan silinir; daha az girintili satır derlenmez (`CS8999`). `$$"""` ile JSON gibi süslü parantezli metinlere değer yerleştirilir.
- Döngü içinde büyüyen metin `+=` ile karesel maliyetle, `StringBuilder` ile yaklaşık doğrusal maliyetle üretilir. `StringBuilder` değiştirilebilir bir referans tipidir (`Append`, `Insert`, `Remove`, `Replace`, `Clear`, `ToString`); birkaç parçalık birleştirmede `+` ve `$"..."` yeterlidir.
- `char.IsDigit`, `IsLetter`, `IsLetterOrDigit`, `IsUpper`, `IsLower`, `IsWhiteSpace`, `IsPunctuation` Unicode'u tanır. `char + int` işleminin sonucu `int`'tir; karakter için `(char)('a' + 1)` yazılır. Rakam karakterinden sayıya `c - '0'` ile geçilir.
- `ToCharArray` ve `new string(dizi)` metni düzenlenebilir hale getirir; ters çevirme için `Array.Reverse` kullanılır.
- `Length` UTF-16 kod birimlerini sayar; bayt sayısından (`Encoding.UTF8.GetByteCount`) ve ekranda görünen karakter sayısından (`StringInfo`) farklı olabilir.

Alıştırmalar

1. `"Programlama".Substring(2, 5)` ifadesinin sonucu nedir?
   A) `"ogr"`
   B) `"gram"`
   C) `"ogram"`
   D) `"ogramla"`
   E) `"gramlam"`

2. Aşağıdaki program ekrana ne yazdırır?

```
string s = "merhaba";
s.ToUpper();
s.Replace('a', 'e');
string t = s;
t = t + "!";
Console.WriteLine(s + t.Length);
```

3. Aşağıdakilerden hangisi için `"ISPARTA".ToLower()` ifadesi, geçerli kültür Türkçe iken `"ısparta"` değerini üretir?
   A) Her zaman; kültürden bağımsızdır.
   B) Yalnızca `tr-TR` gibi Türkçe kuralları kullanan kültürde.
   C) Yalnızca `ToLowerInvariant()` çağrılırsa.
   D) Yalnızca `StringComparison.Ordinal` verilirse.
   E) Hiçbir zaman; sonuç her zaman `"isparta"` olur.

4. `string s = "Veri";` tanımından sonra aşağıdaki ifadelerden hangisi çalışırken `ArgumentOutOfRangeException` fırlatır?
   A) `s[3]`
   B) `s.Substring(4)`
   C) `s.Substring(1, 3)`
   D) `s.Substring(2, 3)`
   E) `s.IndexOf('z')`

5. İki `string` değişkeni `a` ve `b` için `a == b` ifadesi hakkında hangisi doğrudur?
   A) İki değişkenin aynı nesneyi göstermesini denetler.
   B) Yalnızca ikisi de metin sabitiyse `true` verebilir.
   C) Metinlerin içeriğini karşılaştırır.
   D) Büyük/küçük harf farkını yok sayar.
   E) `null` değerlerle her zaman hata verir.

6. Aşağıdaki program ekrana ne yazdırır?

```
char c = 'a';
int x = "merhaba".IndexOf('z');
Console.WriteLine(x + " " + (c + 1) + " " + (char)(c + 1));
Console.WriteLine('a' + 'b' + "|" + 'a' + 'b');
```

7. Aşağıdaki program ekrana ne yazdırır?

```
string veri = " elma,, armut ;kiraz ";
string[] a = veri.Split(',');
string[] b = veri.Split(new[] { ',', ';' },
```

```
    StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries);
Console.WriteLine(a.Length + " " + b.Length);
Console.WriteLine(string.Join("/", b));
```

8. Aşağıdaki program ekrana ne yazdırır?

```
using System.Text;

StringBuilder sb = new StringBuilder("abc");
StringBuilder sb2 = sb;
sb.Append("d").Insert(0, 'x');
Console.WriteLine(sb + " " + sb2);
sb2.Remove(1, 2);
Console.WriteLine(sb);

string s = "abc";
string t = s;
s = s + "d";
Console.WriteLine(s + " " + t);
```

9. Aşağıdaki kodda üç sorun vardır. Hangileridir ve nasıl düzeltilir?

```
string ad = "elif";
ad[0] = 'E';
ad.Trim();
Console.WriteLine(ad.Length());
```

10. Aşağıdaki kod derlenmez. Neden? Nasıl düzeltilir?

```
char harf = 'c';
char sonraki = harf + 1;
Console.WriteLine(sonraki);
```

11. Aşağıdaki program ekrana ne yazdırır? (Satır başlarındaki boşlukları dikkate alın.)

```
string ad = "Ece";
string s = $"""
      Merhaba {ad}
    Hoş geldin
    """;
Console.WriteLine(s);
Console.WriteLine(s.Split('\n').Length);
```

12. `string t = "a\U0001F600b";` tanımından sonra `t.Length` değeri nedir ve neden bu değerdir? `t.EnumerateRunes().Count()` kaçtır?
13. Kullanıcıdan bir kurum adı okuyan bir program yazın. Program her kelimenin ilk harfini büyük, kalanını küçük yazmalı ve kelimelerin baş harflerinden bir kısaltma üretmelidir. Kelimeler arasında birden fazla boşluk olabilir; Türkçe harfler doğru çevrilmelidir (`i` harfi `İ` olmalı).
14. Bir kelimedeki art arda tekrar eden harfleri "harf + tekrar sayısı" biçiminde sıkıştıran `static string Sikistir(string s)` metodunu ve bu metodu kullanıcıdan okunan bir kelimeyle deneyen programı yazın. Örnekler: `aaabccdddd` için `a3b1c2d4`, `xyz` için `x1y1z1`.
15. İki kelimenin anagram olup olmadığını (aynı harflerin farklı sırayla yazılmış hali olup olmadığını) büyük/küçük harf farkını yok sayarak denetleyen bir metot ve onu deneyen bir program yazın. Örnek: `elma` ve `alem` anagramdır.

16. Kullanıcıdan bir parola okuyan bir program yazın. Parola en az 8 karakter olmalı ve en az bir büyük harf, bir küçük harf, bir rakam ve bir noktalama işareti içermelidir. Eksik olan koşulları tek tek yazdırın; hepsi sağlanıyorsa "Parola geçerli" yazdırın.

BÖLÜM 49

Sınıflar ve Nesneler

BU BÖLÜMDE

- Sınıf ile nesne farkını, `new` ile nesne oluşturmayı ve alanların varsayılan değerlerini
- `public`, `private`, `protected` ve `internal` erişim belirleyicilerini; kapsüllemeyi
- Özellikleri (property): tam özellik, otomatik özellik, `init`, salt-okunur ve hesaplanan özellik
- Yapıcıları, `this`, yapıcı zincirlemeyi, nesne başlatıcıları ve birincil yapıcıları (C# 12)
- `static` üyeleri, `static` sınıfları; `const` ile `readonly` farkını
- `ToString` ve `Equals` metotlarını ezmeyi
- Referans semantiğini: iki değişkenin aynı nesneyi göstermesi, `null` ve `NullReferenceException`
- C++ sınıfı (yıkıcı, kopya yapıcı) ile C# sınıfı arasındaki farkları

Bölüm 42'de değer tipleri ile referans tiplerini ayırmış, `class` ile yapılan tanımların referans tipi olduğunu söylemiştik. Bölüm 47'de dizilerin de birer nesne olduğunu gördük. Bu bölümde kendi nesne tiplerimizi tanımlamayı öğreniyoruz. C# bir nesne yönelimli dildir: programın hemen her parçası bir sınıfın içinde yaşar, `Console` ve `Math` de birer sınıftır.

C++ kısmından gelen okur için konu tanıdık olacak. Sınıf, kapsülleme, yapıcı ve `static` üyeler Bölüm 30'da ayrıntılı işlendi; burada yapıları yeniden anlatmayacağız. Onun yerine C#'ın aynı fikri hangi sözdizimiyle ve hangi farklarla sunduğuna odaklanacağız. En büyük üç fark şunlardır: C#'ta sınıf nesneleri her zaman referansla taşınır, bellek çöp toplayıcıya aittir (yıkıcı ve kopya yapıcı yoktur) ve veriye kontrollü erişim için dile yerleşik **özellik** (property) yapısı vardır.

Kalıtım, polimorfizm ve arayüzler sonraki bölümlerin (50 ve 51) konusudur; `struct`, `enum` ve `record` Bölüm 52'de işlenecek.

49.1 Neden Sınıf?

Öğrenci bilgilerini tutan bir program düşünelim. Bölüm 47'deki yaklaşımla ad, numara ve not için üç ayrı dizi kullanabiliriz:

```
string[] adlar = { "Ayşe", "Mehmet", "Elif" };
int[] numaralar = { 101, 102, 103 };
double[] ortalamalar = { 72.5, 88.0, 64.0 };
```

Bu yapı kırılgandır. Adları sıraladığımızda diğer iki diziyi de aynı biçimde sıralamayı unutursak Ayşe'nin ortalaması Mehmet'e geçer. Her yeni bilgi (bölüm, e-posta…) yeni bir dizi demektir ve hiçbir şey bu dizilerin birbirine ait olduğunu derleyiciye söylemez.

Nesne yönelimli programlama bu sorunu, birbirine ait verileri ve onlar üzerinde çalışan işlemleri **tek bir tipte** toplayarak çözer. Üç dizi yerine tek bir `Ogrenci[]` dizisi olur ve her öğrenci kendi adını, numarasını ve ortalamasını taşır. Bölüm 14'te C'deki `struct` ile verileri toplamış, Bölüm 30'da bunun üzerine işlemleri ve erişim denetimini eklemiştik. C# sınıfı aynı fikrin üçüncü sürümüdür.

> **Tanım: Sınıf ve nesne**
>
> Sınıf (class), nesnelerin hangi verileri tutacağını ve hangi işlemleri yapabileceğini tarif eden şablondur. Nesne (object), bu şablondan `new` ile üretilmiş somut bir örnektir (instance). Bir sınıftan istediğimiz kadar nesne üretebiliriz; her nesnenin yapısı aynı, verileri kendine aittir.

Her nesnenin iki yönü vardır: **durum** (state: ad, numara, ortalama) ve **davranış** (behavior: bilgilerini yazdır, ortalamasını güncelle). C#'ta durum **alanlarla** (field) ve **özelliklerle** (property), davranış **metotlarla** (method) ifade edilir.

49.2 İlk Sınıfımız: Alanlar ve Metotlar

Bölüm 41'de top-level statements kullanmıştık. Sınıf tanımları bu kodun **altına**, aynı dosyaya yazılır; sınıfı üst düzey deyimlerin üstüne yazarsak derleyici `CS8803` hatası verir. Gerçek projelerde her sınıf genellikle kendi dosyasında durur; aynı projedeki sınıflar birbirini ek bir `using` olmadan görür.

`ogrenciIlk.cs`

```
Ogrenci a = new Ogrenci();
a.Ad = "Ayşe";
a.No = 101;
a.Ortalama = 72.5;

Ogrenci b = new Ogrenci();
b.Ad = "Mehmet";
b.No = 102;
b.Ortalama = 88.0;

a.BilgiYazdir();
b.BilgiYazdir();

class Ogrenci
{
    public string Ad = "";      // alanlar: her nesnenin kendi kopyası var
    public int No;
    public double Ortalama;

    public void BilgiYazdir()   // metot: nesnenin davranışı
    {
        Console.WriteLine($"{No} - {Ad} - {Ortalama}");
    }
}
```

```
ÇIKTI
101 - Ayşe - 72.5
102 - Mehmet - 88
```

Programı adım adım okuyalım:

- `class Ogrenci { ... }` yeni bir tip tanımlar. Artık `int` gibi `Ogrenci` tipinde de değişken tanımlayabiliriz.
- `new Ogrenci()` bellekte (yönetilen öbekte, Bölüm 42) yeni bir nesne oluşturur ve ona bir **referans** döndürür. Her `new` yepyeni, bağımsız bir nesnedir. `a` ile `b` iki ayrı öğrenciyi gösterir.
- Nokta operatörü (`.`) ile nesnenin alanına (`a.Ad`) ve metoduna (`a.BilgiYazdir()`) ulaşırız. Metot, hangi nesne üzerinden çağrılmışsa onun alanlarını kullanır.
- Bölüm 46'daki metotların başında `static` vardı, `BilgiYazdir`'de yok. `static` olmayan metot, bir nesne üzerinden çağrılır.
- `88.0` çıktıda `88` olarak yazıldı; C# ondalık kısmı sıfır olan `double` değerlerini böyle gösterir.

C++ ile ilk fark: C++'ta `Ogrenci a;` yazmak nesneyi doğrudan yığında oluştururdu. C#'ta `Ogrenci a;` yalnızca bir referans değişkeni tanımlar ve nesne yoktur; nesne ancak `new` ile doğar. Bu farka "Referans Semantiği" kısmında geri döneceğiz.

Alanlar ve varsayılan değerler

Sınıfın içinde, metotların dışında tanımlanan değişkenlere alan (field) denir. Alan, nesne yaşadığı sürece yaşar ve sınıfın bütün metotları tarafından okunup değiştirilebilir. Metodun içindeki yerel değişken ise yalnızca o blokta geçerlidir.

Bölüm 42'de yerel değişkenin değer atanmadan okunamayacağını görmüştük. Alanlar için durum farklıdır: `new` ile nesne oluşturulurken C# bütün alanları otomatik olarak sıfırlar. Bu, C'deki yerel değişkenlerin "çöp" değer taşıması ile Bölüm 47'de diziler için gördüğümüz kuralın aynısıdır.

Alanın tipi	Varsayılan değer
`int`, `long`, `double`, `decimal`	`0`
`bool`	`false`
`char`	kodu 0 olan karakter (`'\0'`)
`string`, dizi, sınıf (referans tipleri)	`null`

`null`, "bu değişken henüz hiçbir nesneyi göstermiyor" demektir. Referans tipli bir alanın `null` olarak başlaması, ilerleyen kısımda göreceğimiz `NullReferenceException` hatasının kaynağıdır. Örnekteki `public string Ad = "";` satırı, alana tanımlandığı yerde bir **başlangıç değeri** verir; bu atama nesne oluşturulurken, yapıcıdan önce çalışır.

> **Not · Nullable referans tipleri**
>
> Yeni proje şablonlarında "nullable context" açıktır. Bu durumda `string` yazılan bir alanın `null` olmaması beklenir ve derleyici bunu garanti edemediği yerlerde uyarı verir (`CS8618`). `null` olabilecek alanı `string?` ile işaretleriz. Bu kitapta alanları `""` ile başlatarak uyarılardan kaçınıyoruz; konunun tamamı Bölüm 59'da işlenecek.

49.3 Erişim Belirleyiciler ve Kapsülleme

`ogrenciIlk.cs` programında `a.Ortalama = 250;` yazmamızı hiçbir şey engellemez. Alanlara doğrudan erişim, nesnenin geçersiz bir duruma düşmesine izin verir. Bölüm 30'da C++ için gördüğümüz çözüm burada da geçerlidir: alanları dışarıdan gizlemek ve değişikliklere yalnızca sınıfın sunduğu yoldan izin vermek.

> **Tanım: Kapsülleme**
>
> Bir nesnenin iç verilerini dış dünyadan gizleyip yalnızca sınıfın tanımladığı üyeler üzerinden erişime açma ilkesine kapsülleme (encapsulation) denir. Böylece nesnenin kuralları (ortalama 0 ile 100 arasındadır, bakiye eksiye düşmez) tek bir yerde, sınıfın içinde uygulanır.

Görünürlüğü **erişim belirleyiciler** (access modifiers) belirler:

Belirleyici	Nereden erişilir?
`public`	Her yerden
`private`	Yalnızca tanımlandığı sınıfın içinden
`protected`	Sınıfın içinden ve türeyen sınıflardan (Bölüm 50)
`internal`	Aynı proje (assembly) içinden
`protected internal`	`protected` ya da `internal` olan her yerden
`private protected`	Aynı projedeki türeyen sınıflardan

Varsayılanlar şöyledir: bir sınıfın **üyesi** için belirleyici yazılmazsa `private`, sınıfın **kendisi** için yazılmazsa `internal` kabul edilir. C++'ta `class` için de varsayılanın `private` olduğunu hatırlayalım (Bölüm 30); C#'ta bu kural hem `class` hem `struct` içindeki üyeler için geçerlidir. C++'taki `public:` bölümleri C#'ta yoktur: belirleyiciyi her üyenin başına ayrı ayrı yazarız.

Belirleyici yazmayı unutmak, ilk sınıfını yazan herkesin yaptığı hatadır:

▸ privateErisim.cs

```
Hesap h = new Hesap();
h.Bakiye = -5000;

class Hesap
{
    double Bakiye = 100;      // belirleyici yok: private sayılır
}
```

```
DERLEYİCİ ÇIKTISI

privateErisim.cs(2,3): error CS0122: 'Hesap.Bakiye' is inaccessible due to its protection level
```

`Hesap` ve ana kod aynı dosyada olmasına rağmen erişim engellendi: `private`, dosyaya değil **sınıfa** özeldir. Üstelik sınıfa özeldir, nesneye değil: `Ogrenci` sınıfının bir metodu, parametre olarak gelen başka bir `Ogrenci` nesnesinin `private` alanını da okuyabilir.

Genel kural şudur: alanlar `private` olur, dışarıya sunulan üyeler `public` olur, yalnızca sınıfın kendi işinde kullanılan yardımcı metotlar `private` kalır. Alanlara dışarıdan erişim gerekiyorsa bunu özelliklerle sağlarız.

49.4 Özellikler (Property)

Alanı `private` yapınca, değerini dışarıdan okumak ve yazmak için bir kapı gerekir. C++'ta bu kapı `getOrtalama()` ve `setOrtalama()` biçiminde iki ayrı fonksiyondu. C#'ta bunun için dile yerleşik bir yapı vardır.

> **Tanım: Özellik (property)**
> Dışarıdan alan gibi okunup yazılan (`o.Ortalama = 78.5;`), ama içinde kod çalıştıran üyedir. `get` erişimcisi değeri döndürür, `set` erişimcisi atanan değeri `value` adıyla alır. Doğrulama, hesaplama ve erişim denetimi `get` / `set` içinde yapılır.

Değeri tutan gizli alana **destek alanı** (backing field) denir. Bu kitapta `private` alan adlarını `_` ile başlatacağız (`_ortalama`); böylece alan (`_ortalama`), özellik (`Ortalama`) ve parametre (`ortalama`) birbirine karışmaz.

ozellikDogrulama.cs

```
Ogrenci o = new Ogrenci();
o.Ortalama = 78.5;
Console.WriteLine($"{o.Ortalama} {o.Gecti}");
o.Ortalama = 250;                    // geçersiz: reddedilecek
o.Ortalama = 30;
Console.WriteLine($"{o.Ortalama} {o.Gecti}");

class Ogrenci
{
    private double _ortalama;        // destek alanı

    public double Ortalama
    {
        get { return _ortalama; }
        set
        {
            if (value < 0 || value > 100)
            {
                Console.WriteLine($"Geçersiz ortalama: {value}");
                return;              // alanı değiştirmeden çık
            }
            _ortalama = value;
        }
    }

    public bool Gecti => _ortalama >= 50;   // hesaplanan özellik
}
```

ÇIKTI

```
78.5 True
Geçersiz ortalama: 250
30 False
```

`set` içindeki doğrulama, geçersiz `250` değerinin alana yazılmasını engelledi. Dışarıdan bakıldığında `o.Ortalama = 250;` sıradan bir atamaya benzer; ama arkasında kod çalışır.

`Gecti` özelliğinin `set`'i ve destek alanı yoktur. `=>` ile yazılan bu biçime ifade gövdeli üye (expression-bodied member) demiştik (Bölüm 46); değer her istendiğinde hesaplanır. Böyle özelliğe **hesaplanan özellik** diyoruz. Metodun çağrılırken parantezi vardır (`BilgiYazdir()`), özelliğin yoktur (`o.Gecti`).

Hata durumunda yalnızca mesaj yazıp devam etmek, bu küçük örnek için yeterli. Gerçek kodda geçersiz değer genellikle bir istisna (exception) ile bildirilir; Bölüm 53'te `ArgumentOutOfRangeException` ile bu işi daha doğru yapacağız.

Otomatik özellik

Çoğu özellikte `get` ve `set` alanı olduğu gibi okuyup yazar. Bu durumda gövdeyi derleyiciye bırakırız: `{ get; set; }`. Destek alanını derleyici üretir ve gizler. Erişimi daraltarak dört temel biçim elde ederiz:

Yazım	Dışarıdan okuma	Dışarıdan yazma	Değer nerede atanır?
`{ get; set; }`	Evet	Evet	Her yerde
`{ get; private set; }`	Evet	Hayır	Sınıfın içinde
`{ get; }`	Evet	Hayır	Yapıcıda ve başlangıç değerinde
`{ get; init; }`	Evet	Hayır	Yapıcıda ve nesne başlatıcıda

`init` erişimcisi C# 9'da geldi. Nesne oluşturulurken (yapıcı ya da nesne başlatıcı içinde) değer atanmasına izin verir, sonrasında özelliği salt-okunur yapar. Birazdan nesne başlatıcıları görürken kullanacağız.

`set` erişimcisi kapalı bir özelliğe sınıfın dışından atama yapamayız:

▸ ozellikTurleri.cs

```
Hesap h = new Hesap("Ayşe", 1001);
h.Sahip = "Elif";        // { get; set; }: olur
h.No = 2002;             // { get; }: set hiç yok
h.Bakiye = 5;            // { get; private set; }: set dışarıya kapalı

class Hesap
{
    public string Sahip { get; set; } = "";
    public int No { get; }
    public decimal Bakiye { get; private set; }

    public Hesap(string sahip, int no)
    {
        Sahip = sahip;
        No = no;         // { get; }: yapıcıda atanabilir
    }
}
```

DERLEYİCİ ÇIKTISI

```
ozellikTurleri.cs(3,3): error CS0200: Property or indexer 'Hesap.No' cannot be assigned to -- it
is read only
ozellikTurleri.cs(4,3): error CS0272: The property or indexer 'Hesap.Bakiye' cannot be used in
this context because the set accessor is inaccessible
```

`set` hiç yoksa `CS0200`, `set` var ama `private` ise `CS0272` hatası alırız. `Hesap` sınıfının kendi metotları `Bakiye`'yi (`private set`) serbestçe değiştirebilir; bu örüntüyü BankaHesabi örneğinde kullanacağız.

Özelliklerde başlangıç değeri de verilebilir: `public string Sahip { get; set; } = "";`. Bu satırın sonundaki atama, destek alanına nesne oluşturulurken yazılır.

İpucu · Özelliği elle yazmayın

Visual Studio'da sınıf içinde `prop` yazıp iki kez Tab'a basmak otomatik özellik iskeletini, `ctor` yazmak yapıcı iskeletini üretir. Visual Studio Code'daki C# Dev Kit de benzer kısayollar sunar. Bu, özellikle çok sayıda otomatik özellik yazarken zaman kazandırır.

◆ Kritik Nokta · Özellik alan değildir

Özellik dışarıdan alan gibi görünür; ama çağrıldığında kod çalışır. Doğrulama yapabilir, hesaplama yapabilir, hatta yavaş çalışabilir. Ayrıca özelliği `ref` ya da `out` argümanı olarak geçiremeyiz (Bölüm 46); çünkü özellik bir bellek konumu değil, bir çift metottur. Alanı ise geçirebiliriz.

Tarihçe · C# 14: field anahtar sözcüğü

Doğrulamalı bir özellik için ayrı bir `_ortalama` alanı yazmak zorunlu değil. C# 14 ile gelen `field` anahtar sözcüğü, derleyicinin ürettiği gizli destek alanını `get` ya da `set` içinde adıyla kullanmamıza izin verir; böylece yalnızca bir erişimcinin gövdesini yazıp diğerini otomatik bırakabiliriz. Bu kitabın örnekleri C# 12 ile uyumlu kalsın diye açık destek alanı yazıyoruz.

Salt-okunur durumu için hangi biçim?

Seçim basittir. Nesnenin ömrü boyunca değişmeyen veri (hesap numarası) için `{ get; }` ya da `{ get; init; }`, yalnızca sınıfın kendi metotlarının değiştireceği veri (bakiye) için `{ get; private set; }`, doğrulama gereken veri (ortalama) için tam özellik, herkesin değiştirebileceği ve kuralı olmayan veri için `{ get; set; }` kullanırız. Bu seçim, kapsüllemenin pratikteki karşılığıdır: her veriye gerektiği kadar yetki veririz.

49.5 Yapıcılar

Nesne `new` ile oluşturulurken çalışan özel metoda **yapıcı** (constructor) denir. Görevi nesneyi geçerli bir ilk durumla başlatmaktır. Yapıcının adı sınıfın adıyla aynıdır ve dönüş tipi yoktur (`void` bile yazılmaz). Bölüm 30'daki C++ yapıcılarıyla aynı mantıktır; farklar aşağıda belirtilmiştir.

yapiciZincir.cs

```
Urun u1 = new Urun("Kalem", 12.5m, 40);
Console.WriteLine("---");
Urun u2 = new Urun("Defter", 30m);
Console.WriteLine("---");
Urun u3 = new Urun("Silgi");
Console.WriteLine($"{u1.Ad} {u2.Stok} {u3.Fiyat}");

class Urun
{
    public string Ad { get; }
    public decimal Fiyat { get; }
    public int Stok { get; private set; }

    public Urun(string ad, decimal fiyat, int stok)
    {
        Ad = ad;
        Fiyat = fiyat;
        Stok = stok;
        Console.WriteLine($"3 parametreli: {ad}");
```

```
    }

    public Urun(string ad, decimal fiyat) : this(ad, fiyat, 0)
    {
        Console.WriteLine($"2 parametreli: {ad}");
    }

    public Urun(string ad) : this(ad, 0m)
    {
        Console.WriteLine($"1 parametreli: {ad}");
    }
}
```

```
ÇIKTI
3 parametreli: Kalem
---
3 parametreli: Defter
2 parametreli: Defter
---
3 parametreli: Silgi
2 parametreli: Silgi
1 parametreli: Silgi
Kalem 0 0
```

Bu programda üç kavram var.

Aşırı yükleme. Bir sınıfın birden çok yapıcısı olabilir; parametre listeleri farklı olmalıdır (Bölüm 46'daki aşırı yükleme kuralları).

Yapıcı zincirleme. `: this(...)` ifadesi, aynı sınıfın başka bir yapıcısını gövdeden **önce** çağırır. Böylece ilk değer verme mantığını tek bir "asıl" yapıcıda toplarız. Çıktıdaki sıraya dikkat edelim: önce en çok parametreli yapıcının gövdesi, sonra onu çağıranlar çalışır. C++ kısmında buna yapıcı devretme (delegating constructor) demiştik (Bölüm 30); mantık aynı, sözdizimi `: this(...)` biçiminde.

Salt-okunur özelliklerin atanması. `Ad` ve `Fiyat` yalnızca `get` taşır; değerleri yapıcıda atanır. `Stok` ise sınıfın içinden değişebilir.

C++'taki üye ilk değer listesi (`: ad(a), fiyat(f)`) C#'ta yoktur ve gerekmez; alanlar yapıcı gövdesinde atanır ya da tanımlandıkları yerde başlangıç değeri alır.

Varsayılan yapıcı kuralı

Sınıfa hiç yapıcı yazmazsak derleyici parametresiz bir **varsayılan yapıcı** ekler (`new Ogrenci()` bu yüzden çalışmıştı). Bir tane bile yapıcı yazdığımız anda bu otomatik yapıcı kaybolur. C++'ta da aynı kural geçerliydi:

▸ varsayilanYok.cs

```
Urun u = new Urun();

class Urun
{
    public string Ad { get; }
    public Urun(string ad) { Ad = ad; }
}
```

```
DERLEYİCİ ÇIKTISI

varsayilanYok.cs(1,11): error CS7036: There is no argument given that corresponds to the
required parameter 'ad' of 'Urun.Urun(string)'
```

Parametresiz yapıcıya da ihtiyacımız varsa onu kendimiz yazarız. Bazı kütüphaneler (serileştirme, veritabanı eşleme araçları) parametresiz yapıcı bekler; bu durumda `public Urun() { }` eklemek gerekir.

Nesne başlatıcılar

Yapıcıya bütün değerleri vermek yerine nesneyi oluşturup özelliklerini süslü parantez içinde atayabiliriz. Buna nesne başlatıcı (object initializer) denir. Önce yapıcı, sonra belirtilen atamalar çalışır:

▸ nesneBaslatici.cs

```
Ogrenci o = new Ogrenci { Ad = "Elif", No = 201 };
o.Ortalama = 91.5;
Console.WriteLine($"{o.No} {o.Ad} {o.Ortalama}");

class Ogrenci
{
    public required string Ad { get; init; }
    public int No { get; init; }
    public double Ortalama { get; set; }
}
```

```
ÇIKTI
201 Elif 91.5
```

`init` özelliklerine yalnızca nesne başlatıcıda ve yapıcıda değer atanabilir; `o.No = 5;` yazarsak `CS8852` hatası alırız. `required` değiştiricisi (C# 11) ise o özelliğin nesne başlatıcıda **mutlaka** verilmesini şart koşar. `new Ogrenci { No = 1 }` yazarsak `Ad` verilmediği için derleme hatası oluşur. Bu ikili, "yapıcıda uzun parametre listesi" sorununa ve değişmez nesne ihtiyacına hafif bir çözüm sunar.

Birincil yapıcılar (C# 12)

Yapıcıyı yalnızca parametreleri alana aktarmak için yazıyorsak sınıf adının hemen yanına parametre listesi koyabiliriz. Buna **birincil yapıcı** (primary constructor) denir:

▸ birincilYapici.cs

```
Vektor v = new Vektor(3, 4);
Console.WriteLine(v.Uzunluk());
Console.WriteLine(v);

class Vektor(double x, double y)
{
    public double X { get; } = x;      // parametre özelliğe aktarılır
    public double Y { get; } = y;

    public double Uzunluk() => Math.Sqrt(X * X + Y * Y);
    public override string ToString() => $"({X}, {Y})";
}
```

```
ÇIKTI
5
(3, 4)
```

Birincil yapıcı parametreleri sınıfın gövdesinde görünür, ancak **otomatik olarak özellik olmaz**; yukarıda `X` ve `Y` özelliklerini kendimiz tanımlayıp parametrelerle başlattık. Parametreleri doğrudan metotlarda kullanırsak derleyici bunları gizli alanlar olarak yakalar. Bu davranış `record` tiplerinden farklıdır; `record` konumsal parametrelerden özellik üretir (Bölüm 52). Birincil yapıcı yeni bir özellik olduğu için bu kitapta çoğunlukla klasik yapıcıları kullanıyoruz.

> **▲ Sık Yapılan Hata · Yapıcıda özelliği değil alanı atamak**
>
> Doğrulamalı bir özelliğin destek alanını yapıcıda doğrudan (`_ortalama = ortalama;`) atarsak `set` içindeki kontrol atlanır. Yapıcıda özelliğin kendisine atamak (`Ortalama = ortalama;`) doğrulamayı da çalıştırır. Hangisini istediğimizi bilerek seçmeliyiz.

Yıkıcı yok, kopya yapıcı yok

C++ sınıfında yıkıcı ve kopya yapıcıyı Bölüm 30'da yazmıştık. C#'ta bu iki yapı **gündelik kodda yoktur**:

- Nesnenin belleğini çöp toplayıcı (garbage collector) geri alır; hangi anda alacağı belli değildir. Bu yüzden C++'taki gibi "kapsamdan çıkınca yıkıcı çalışır" garantisi yoktur. Dosya, ağ bağlantısı gibi kaynakları zamanında bırakmak için `IDisposable` ve `using` kullanılır (Bölüm 53). C#'ın `~Sinif()` biçimindeki **sonlandırıcısı** (finalizer) yalnızca yönetilmeyen kaynaklara sahip nadir durumlarda yazılır.
- Kopya yapıcıya gerek yoktur; çünkü `b = a` zaten nesneyi kopyalamaz, referansı kopyalar (aşağıda göreceğiz). Gerçekten bağımsız bir kopya gerekirse bunun için kendi `Kopyala()` metodumuzu yazarız ya da `record` tiplerinin `with` ifadesini kullanırız (Bölüm 52).

Sonuç olarak C++'taki "üçler kuralı" (Bölüm 30) C#'ta konu dışıdır.

49.6 this Anahtar Sözcüğü

Metotlar nesneden bağımsız yazılır, ama her çağrıda belirli bir nesne üzerinde çalışır. `this`, o anda çalışan metodun üzerinde çağrıldığı nesneyi gösteren referanstır. C++'ta `this` bir işaretçiydi (`this->x`); C#'ta bir referanstır ve nokta ile kullanılır (`this.x`).

En yaygın kullanımı, parametre adı alan adıyla çakıştığında ayrım yapmaktır. İkinci kullanımı nesneyi geri döndürerek çağrıları art arda zincirlemektir:

▸ thisOrnek.cs

```
Kutu k = new Kutu(2, 3);
k.Genislet().Genislet().Uzat();
Console.WriteLine(k.Bilgi());

class Kutu
{
    private int en;
    private int boy;

    public Kutu(int en, int boy)
    {
        this.en = en;        // soldaki alan, sağdaki parametre
```

```
        this.boy = boy;
    }

    public Kutu Genislet()
    {
        en++;
        return this;          // aynı nesneyi döndür
    }

    public Kutu Uzat()
    {
        boy += 2;
        return this;
    }

    public string Bilgi() => $"{en}x{boy}";
}
```

```
ÇIKTI
4x5
```

Metot adları arasında geçen `.` zinciri (`Genislet().Genislet().Uzat()`) akıcı arayüz (fluent interface) olarak bilinir. `this` kullanmadan da alanlara adlarıyla erişebiliriz; `this.` yalnızca çakışmayı çözmek ya da nesnenin kendisini başka bir yere argüman olarak vermek için gereklidir. Yapıcı zincirlemedeki `: this(...)` ise farklı bir kullanımdır: aynı sınıfın başka bir yapıcısını çağırır.

49.7 static Üyeler ve static Sınıflar

Alan ve metotların büyük kısmı nesneye aittir. `static` üyeler ise **sınıfa** aittir ve bir nesne olmadan, sınıf adıyla kullanılır. Bölüm 46'daki `static` metotlar bu yüzden `Program` içinde doğrudan çağrılabilmişti. Bölüm 30'da C++ için gördüğümüz fikir aynıdır: `static` alanın tek bir kopyası vardır ve tüm nesneler onu paylaşır. C++'taki `Sinif::uye` yazımı C#'ta `Sinif.Uye` olur.

staticOrnek.cs

```
Hesap a = new Hesap();
Hesap b = new Hesap();
Hesap c = new Hesap();

Console.WriteLine(Hesap.ToplamHesap);
Console.WriteLine($"{a.No} {b.No} {c.No}");
Console.WriteLine($"{Hesap.BankaAdi} {Hesap.FaizOrani}");

class Hesap
{
    private static int _sayac = 0;                    // tüm nesneler paylaşır
    public const string BankaAdi = "Merkez";
    public static readonly double FaizOrani = 0.05;

    public int No { get; }

    public Hesap()
    {
        _sayac++;
        No = 1000 + _sayac;                           // her nesne kendi numarasını alır
    }
```

```
    public static int ToplamHesap => _sayac;
}
```

```
ÇIKTI
3
1001 1002 1003
Merkez 0.05
```

`static` bir metot nesneye ait olmadığından `this` yoktur ve nesne üyelerine (`No` gibi) doğrudan erişemez; yalnızca `static` üyeleri kullanabilir. Tersine, nesne metotları `static` üyelere serbestçe erişebilir.

`static` üyelere **nesne üzerinden** erişmeye çalışmak (`a.ToplamHesap`) C#'ta hatadır; C++'ta buna izin verilirdi. Bu, `static` üyenin sınıfa ait olduğunu okuyan için daha açık kılar.

static sınıflar

Hiç nesnesi üretilmeyecek, yalnızca yardımcı metotlar toplayan sınıflar `static class` olarak tanımlanır. `static` sınıfın bütün üyeleri `static` olmak zorundadır; derleyici `new` ile nesne üretmeyi engeller. `Math` ve `Console` birer `static` sınıftır:

staticSinif.cs

```
Console.WriteLine(Donusum.CdenF(100));
Console.WriteLine(Donusum.FdenC(32));

static class Donusum
{
    public static double CdenF(double c) => c * 9 / 5 + 32;
    public static double FdenC(double f) => (f - 32) * 5 / 9;
}
```

```
ÇIKTI
212
0
```

C ve C++'ta bu işi serbest fonksiyonlar görürdü. C#'ta serbest fonksiyon yoktur; her metot bir tipin içinde olmak zorundadır, `static` sınıflar bu ihtiyacı karşılar.

const ve readonly

Değişmeyen değerler için C#'ta iki mekanizma vardır ve farkları önemlidir:

	`const`	`readonly`
Değer ne zaman belirlenir?	Derleme zamanı	Çalışma zamanı (tanımlanırken ya da yapıcıda)
İzin verilen tipler	Sayılar, `bool`, `char`, `string`, `null`	Her tip
Örnek başına mı?	Hayır, örtük olarak `static`	Evet (örnek alanı) ya da `static readonly`
Atama yeri	Yalnızca tanımlanırken	Tanımlanırken ya da yapıcıda

sabitler.cs

```
Cember c = new Cember(2);
Console.WriteLine($"{Cember.Pi} {c.Alan()}");
Console.WriteLine(Cember.Birim.Alan());
```

```
class Cember
{
    public const double Pi = 3.14159;                    // derleme zamanı sabiti
    public readonly double Yaricap;                      // nesne başına, yapıcıda atanır
    public static readonly Cember Birim = new Cember(1); // tek, paylaşılan nesne

    public Cember(double yaricap)
    {
        Yaricap = yaricap;
    }

    public double Alan() => Pi * Yaricap * Yaricap;
}
```

```
ÇIKTI
3.14159 12.56636
3.14159
```

`Birim` gibi bir nesneyi `const` yapamayız, çünkü `new` derleme zamanında değerlendirilemez; bu durumda `static readonly` kullanırız. `Yaricap`'a yapıcı dışında (örneğin `Alan` içinde) atama yapmak derleme hatasıdır (`CS0191`).

> **▲ Sık Yapılan Hata · const değerleri çağıran koda gömülür**
>
> Derleyici bir `const` değerini, onu kullanan kodun içine doğrudan yazar. Sabit başka bir kütüphanede (`.dll`) tanımlıysa ve sonradan değiştirilirse, kullanan program yeniden derlenmedikçe eski değeri kullanmaya devam eder. Bir gün değişebilecek değerler için `static readonly` daha güvenlidir. `Pi` gibi gerçekten hiç değişmeyecek değerler için `const` yeterlidir.

49.8 ToString ve Equals

Her C# sınıfı, doğrudan ya da dolaylı olarak `object` sınıfından türer; bu konuyu Bölüm 50'de işleyeceğiz. Şimdilik şunu bilmemiz yeterli: `object` sınıfında `ToString`, `Equals` ve `GetHashCode` metotları hazır bulunur ve her sınıf bunları **ezebilir** (override).

`Console.WriteLine(nesne)` ve string interpolasyonu, bir nesneyi metne çevirmek için `ToString()` metodunu çağırır. Ezmezsek varsayılan davranış nesnenin tip adını yazmaktır:

▸ toStringOrnek.cs

```
Etiket e = new Etiket();
Satis s = new Satis("Kalem", 12.5m);

Console.WriteLine(e);
Console.WriteLine(s);
Console.WriteLine($"Seçilen: {s}");
string metin = "Fiş -> " + s;
Console.WriteLine(metin);

class Etiket { }                    // ToString ezilmemiş

class Satis
{
    public string Ad { get; }
    public decimal Fiyat { get; }
```

```
    public Satis(string ad, decimal fiyat)
    {
        Ad = ad;
        Fiyat = fiyat;
    }

    public override string ToString() => $"{Ad}: {Fiyat:F2} TL";
}
```

```
ÇIKTI
Etiket
Kalem: 12.50 TL
Seçilen: Kalem: 12.50 TL
Fiş -> Kalem: 12.50 TL
```

`override` anahtar sözcüğü, taban sınıftaki bir metodun yerine geçtiğimizi bildirir; ayrıntısı Bölüm 50'de. C++'ta bu iş için `operator<<` yazmıştık (Bölüm 32); C# bunu tek bir metotla ve her yerde otomatik çalışan biçimde çözer.

Eşitlik

`==` operatörü sınıf nesnelerinde, aksi belirtilmedikçe **referansları** karşılaştırır: iki değişken aynı nesneyi gösteriyorsa `true` olur. İçeriklerin eşitliğini sormak istiyorsak `Equals` metodunu ezeriz. `Equals` ezildiğinde `GetHashCode` da ezilmelidir; iki eşit nesne aynı karma kodunu (hash code) vermek zorundadır, aksi hâlde nesneler sözlük ve küme gibi koleksiyonlarda (Bölüm 54) yanlış davranır. Derleyici sadece birini ezdiğimizde uyarı verir.

esitlik.cs

```
Koordinat p = new Koordinat(1, 2);
Koordinat q = new Koordinat(1, 2);
Koordinat r = p;

Console.WriteLine(p == q);          // farklı nesneler
Console.WriteLine(p.Equals(q));     // içerik eşit
Console.WriteLine(p == r);          // aynı nesne
Console.WriteLine(p);

class Koordinat
{
    public int X { get; }
    public int Y { get; }

    public Koordinat(int x, int y)
    {
        X = x;
        Y = y;
    }

    public override string ToString() => $"({X}, {Y})";

    public override bool Equals(object? obj) =>
        obj is Koordinat k && k.X == X && k.Y == Y;

    public override int GetHashCode() => HashCode.Combine(X, Y);
}
```

```
ÇIKTI
False
True
True
(1, 2)
```

`obj is Koordinat k` ifadesi bir tür denetimi ile dönüşümü birleştirir (desen eşleme, Bölüm 44); ayrıntıları Bölüm 50'de işleyeceğiz. Bu kadar kod yazmadan değer eşitliği istiyorsak `record` tipi bunu otomatik üretir (Bölüm 52). `string` sınıfı kendi `==` operatörünü içerik karşılaştırması yapacak biçimde tanımlamıştır (Bölüm 48); bu yüzden metinlerde `==` içeriğe bakar.

49.9 Referans Semantiği

Bölüm 42'de referans tiplerinde atamanın nesneyi değil referansı kopyaladığını söylemiştik. Sınıf yazmaya başladığımıza göre bu davranış artık gündelik kodumuzun parçası. C++ okuru için burası en dikkat edilmesi gereken yerdir: C++'ta `Kutu b = a;` nesneyi kopyalar (kopya yapıcı çalışır, Bölüm 30); C#'ta aynı satır, aynı nesneyi gösteren ikinci bir ad üretir.

▸ referansOrnek.cs

```
Sayac a = new Sayac();
a.Deger = 5;

Sayac b = a;              // yeni nesne yok: b de aynı nesneyi gösterir
b.Deger = 10;
Console.WriteLine(a.Deger);
Console.WriteLine(a == b);

Sayac c = new Sayac();
c.Deger = 10;
Console.WriteLine(a == c);   // içerik aynı, nesne farklı

b = new Sayac();             // b artık başka bir nesneyi gösteriyor
b.Deger = 99;
Console.WriteLine($"{a.Deger} {b.Deger}");

class Sayac
{
    public int Deger;
}
```

```
ÇIKTI
10
True
False
10 99
```

`b.Deger = 10;` satırı `a` üzerinden de görünür, çünkü ikisi tek nesnenin iki adıdır. `a == c` sorusunun cevabı `False`'tur: `Sayac` sınıfı `==` operatörünü tanımlamadığı için karşılaştırma referanslara bakar, `Deger` alanları eşit olsa da nesneler farklıdır. Son iki satırda `b = new Sayac();` ataması `a`'yı etkilemez; yalnızca `b`'nin gösterdiği yeri değiştirir.

Bu durum, nesneleri metotlara geçirirken de geçerlidir. Bölüm 46'da varsayılan parametre geçirmenin değerle geçirme olduğunu görmüştük. Referans tipli parametrede kopyalanan şey **referanstır**: metot, nesnenin içeriğini değiştirebilir; ancak parametreye yeni bir nesne atamak çağıranı etkilemez (bunun için `ref` gerekir).

nesneGecirme.cs

```
BankaHesabi h = new BankaHesabi { Bakiye = 100 };

Zamla(h);
Console.WriteLine(h.Bakiye);

Yenile(h);
Console.WriteLine(h.Bakiye);

static void Zamla(BankaHesabi x)
{
    x.Bakiye += 50;             // aynı nesne değişir
}

static void Yenile(BankaHesabi x)
{
    x = new BankaHesabi();      // yerel kopya başka nesneye yönlendi
    x.Bakiye = 9999;
}

class BankaHesabi
{
    public decimal Bakiye;
}
```

ÇIKTI

```
150
150
```

◆ **Kritik Nokta · Sınıf nesnesi metoda "referansın kopyası" olarak gider**

Metot içinde `x.Alan = ...` çağıranın nesnesini değiştirir. Metot içinde `x = new ...` yalnızca yerel `x`'i başka yere yönlendirir. Çağıranın değişkenini de yeniden yönlendirmek istiyorsak parametreyi `ref` ile geçiririz (Bölüm 46). Gerçekten bağımsız bir kopya gerekiyorsa kopyayı kendimiz üretmeliyiz.

Bağımsız kopya üretmek

C#'ta kopya yapıcı yoktur; ama istersek aynı işi yapan bir yapıcı ya da metot yazabiliriz. Alanlar yalnızca değer tipli ya da `string` (değişmez) ise basit bir kopya yeterlidir:

kopyalama.cs

```
Hesap a = new Hesap("Ayşe", 500m);
Hesap b = a.Kopyala();
b.Bakiye = 1;
Console.WriteLine($"{a.Bakiye} {b.Bakiye} {a == b}");

class Hesap
{
    public string Sahip { get; }
    public decimal Bakiye { get; set; }

    public Hesap(string sahip, decimal bakiye)
    {
        Sahip = sahip;
        Bakiye = bakiye;
    }
```

```
    public Hesap Kopyala() => new Hesap(Sahip, Bakiye);
}
```

```
ÇIKTI
500 1 False
```

Sınıfın içinde başka nesnelere ya da dizilere referans varsa, kopyada onların da yeniden üretilmesi gerekir; aksi hâlde C++'taki sığ kopya (shallow copy) sorunu bu kez referanslar arasında yinelenir. Bölüm 47'deki `Array.Clone` ve `Array.Copy` da sığ kopya üretiyordu. C++'ta kopya yapıcı derleyici tarafından otomatik üretilirdi; C#'ta kopyalama her zaman **açık bir tasarım kararıdır**.

49.10 null ve NullReferenceException

Bir referans değişkeni hiçbir nesneyi göstermiyorsa değeri `null`'dır. Hem `new` ile bağlanmamış alanlar hem de `new Ogrenci[5]` ile açılmış dizinin elemanları `null` ile başlar. C'deki `NULL` işaretçisi ve C++'taki `nullptr` ile aynı fikirdir; fark, C#'ta `null` üzerinden erişimin bellek bozulmasına değil, tanımlı bir **istisnaya** yol açmasıdır.

`null` olan bir referansın üyesine erişmeye çalışırsak çalışma zamanı `NullReferenceException` fırlatır. Bu istisna, C# programlarında en sık görülen hatalardan biridir. İstisnaları Bölüm 53'te ayrıntılı işleyeceğiz; burada yalnızca yakalayıp mesajını okumak için basit bir `try` / `catch` kullanıyoruz.

▸ nullHata.cs

```
Kisi? k = null;

string ad = k?.Ad ?? "bilinmiyor";   // ?. ve ?? birlikte
Console.WriteLine(ad);

int? uzunluk = k?.Ad.Length;         // zincirin tamamı null döner
Console.WriteLine(uzunluk.HasValue);

if (k is null)
{
    Console.WriteLine("k henüz bir nesne göstermiyor");
}

try
{
    Console.WriteLine(k.Ad.Length);  // k null: istisna fırlar
}
catch (NullReferenceException e)
{
    Console.WriteLine($"Hata: {e.Message}");
}

class Kisi
{
    public string Ad { get; set; } = "";
}
```

```
ÇIKTI
bilinmiyor
False
k henüz bir nesne göstermiyor
Hata: Object reference not set to an instance of an object.
```

Burada üç araç görüyoruz:

- `?.` **(null-koşullu erişim).** Soldaki ifade `null` ise erişimi yapmaz ve sonuç `null` olur; zincirin geri kalanı hiç değerlendirilmez. Böylece `k?.Ad.Length` tek bir `?.` ile korunur.
- `??` **(null birleştirme).** Soldaki `null` ise sağdakini verir. `??=` ise yalnızca değişken `null` ise atama yapar. İkisini de Bölüm 42'de nullable değer tipleri için görmüştük; referans tiplerinde de aynı çalışırlar.
- `is null` / `is not null`. `null` denetiminin okunur yazımıdır. `==` operatörü bir sınıfta ezilmişse sonucu değiştirebilir; `is null` hiçbir zaman ezilemez, bu yüzden tercih edilir.

`Kisi?` yazımındaki `?`, bu değişkenin bilerek `null` olabileceğini derleyiciye bildirir. Nullable referans tipleri ve derleyicinin `null` akış analizi Bölüm 59'da ele alınacak.

> **▲ Sık Yapılan Hata · NullReferenceException'ın tipik kaynakları**
>
> Çoğunlukla üç yerden gelir: (1) `new` ile oluşturulmayan referans alan, (2) elemanları henüz atanmamış nesne dizisi, (3) bir aramanın (`Find` , sözlükte anahtar bulma) bulamayınca `null` döndürmesi. Çözüm her yerde `null` denetimi serpiştirmek değil, nesnenin geçerli durumla doğmasını sağlamak (yapıcıda başlangıç değerleri) ve `null` dönebilecek çağrıların sonucunu kontrol etmektir.

49.11 Nesne Dizileri

Bölüm 47'de dizilerin referans tipi olduğunu görmüştük. Elemanları sınıf tipinde olan bir dizide iki kademe vardır: dizinin kendisi `new` ile açılır, her eleman ise ayrı bir `new` ile doldurulur. Dizi açıldığında elemanlar `null`'dır.

▸ nesneDizisi.cs

```
Ogrenci[] sinif = new Ogrenci[3];
Console.WriteLine(sinif[0] == null);       // henüz nesne yok

sinif[0] = new Ogrenci("Ayşe", 80);
sinif[1] = new Ogrenci("Mehmet", 65);
sinif[2] = new Ogrenci("Elif", 92);

double toplam = 0;
Ogrenci enIyi = sinif[0];
foreach (Ogrenci o in sinif)
{
    toplam += o.Ortalama;
    if (o.Ortalama > enIyi.Ortalama)
    {
        enIyi = o;                          // yalnızca referans atanır
    }
}

Console.WriteLine($"Sınıf ortalaması: {toplam / sinif.Length:F2}");
Console.WriteLine($"En yüksek: {enIyi}");

class Ogrenci
```

```
{
    public string Ad { get; }
    public double Ortalama { get; }

    public Ogrenci(string ad, double ortalama)
    {
        Ad = ad;
        Ortalama = ortalama;
    }

    public override string ToString() => $"{Ad} ({Ortalama})";
}
```

```
ÇIKTI
True
Sınıf ortalaması: 79.00
En yüksek: Elif (92)
```

`foreach (Ogrenci o in sinif)` döngüsünde `o`, her turda dizideki nesneye giden bir referanstır. Nesnenin özelliğini döngüde değiştirmek mümkündür; ancak `o` döngü değişkenine yeni nesne atayamayız (Bölüm 45). C++ okuru için bir fark daha: `Ogrenci sinif[3];` C++'ta üç nesneyi birden oluştururdu. C#'ta `new Ogrenci[3]` yalnızca üç boş referans yerleştirir.

Boyutu değişen nesne listelerini `List<Ogrenci>` ile tutacağız (Bölüm 54).

49.12 Örnek: BankaHesabi

Bölümün ana kavramlarını tek bir sınıfta toplayalım. `BankaHesabi` şunları kullanıyor: `private set` ile korunan bakiye, yapıcı zincirleme, `static` sayaç ve hesap numarası üretimi, doğrulama içeren metotlar, nesneyi parametre olarak alan `Aktar` ve `ToString`.

▸ bankaHesabi.cs

```
BankaHesabi h1 = new BankaHesabi("Ayşe", 1000m);
BankaHesabi h2 = new BankaHesabi("Mehmet");

h1.Yatir(500m);
Console.WriteLine(h1.Cek(2000m));       // bakiye yetersiz
Console.WriteLine(h1.Cek(300m));
Console.WriteLine(h1.Aktar(h2, 400m));

Console.WriteLine(h1);
Console.WriteLine(h2);
Console.WriteLine($"Toplam hesap: {BankaHesabi.HesapSayisi}");

class BankaHesabi
{
    private static int _sonNo = 1000;

    public static int HesapSayisi { get; private set; }

    public int No { get; }
    public string Sahip { get; }
    public decimal Bakiye { get; private set; }

    public BankaHesabi(string sahip, decimal acilis)
    {
        Sahip = sahip;
        Bakiye = acilis;
```

```
        No = ++_sonNo;
        HesapSayisi++;
    }

    public BankaHesabi(string sahip) : this(sahip, 0m) { }

    public bool Yatir(decimal tutar)
    {
        if (tutar <= 0)
        {
            return false;
        }
        Bakiye += tutar;
        return true;
    }

    public bool Cek(decimal tutar)
    {
        if (tutar <= 0 || tutar > Bakiye)
        {
            return false;
        }
        Bakiye -= tutar;
        return true;
    }

    public bool Aktar(BankaHesabi hedef, decimal tutar)
    {
        if (!Cek(tutar))
        {
            return false;
        }
        hedef.Yatir(tutar);
        return true;
    }

    public override string ToString() => $"#{No} {Sahip}: {Bakiye:F2} TL";
}
```

```
ÇIKTI
False
True
True
#1001 Ayşe: 800.00 TL
#1002 Mehmet: 400.00 TL
Toplam hesap: 2
```

Tasarım kararlarına dikkat edelim. `Bakiye` dışarıdan okunabilir, ama yalnızca `Yatir` ve `Cek` değiştirebilir; negatif tutar ya da yetersiz bakiye kuralı tek bir yerde durur. `No` ve `Sahip` oluşturulduktan sonra değişemez. `HesapSayisi` `static` olduğu için tek bir sayaç tüm hesaplar arasında paylaşılır. Bu, Bölüm 30'daki `Kesir` örneğinde olduğu gibi, küçük bir sınıfın tek başına iyi korunan bir "sözleşme" sunmasının örneğidir.

49.13 Üç Dilde: Sınıf

> **Üç Dilde · Sınıf yazmak: C, C++ ve C#** *C · C++ · C#*
>
> **C:** `struct` yalnızca veri taşır; işlemler serbest fonksiyonlardır ve her alan herkese açıktır (Bölüm 14).

```c
struct Hesap { char sahip[30]; double bakiye; };
void yatir(struct Hesap *h, double tutar);
```

C++: `class` veriyi ve fonksiyonları birleştirir. Nesne varsayılan olarak yığında yaşar, değer semantiğiyle kopyalanır; kaynak yönetimi yıkıcı ve kopya yapıcıyla yapılır (Bölüm 30).

```cpp
class Hesap {
    string sahip; double bakiye;
public:
    Hesap(string s, double b) : sahip(s), bakiye(b) {}
    ~Hesap() {}
    Hesap(const Hesap&) = default;
    void yatir(double tutar);
};
Hesap a("Ayşe", 500);
Hesap b = a;               // kopya
```

C#: `class` bir referans tipidir; nesne öbekte yaşar ve çöp toplayıcıya bırakılır. Kopya yapıcı ve yıkıcı gündelik kodda yoktur; veriye erişim özelliklerle yapılır.

```csharp
class Hesap {
    public string Sahip { get; }
    public double Bakiye { get; private set; }
    public Hesap(string sahip, double bakiye) { Sahip = sahip; Bakiye = bakiye; }
    public void Yatir(double tutar) { /* ... */ }
}
Hesap a = new Hesap("Ayşe", 500);
Hesap b = a;               // aynı nesne
```

Ayrıntıları bir tabloda toplayalım:

Konu	C++	C#
Nesnenin yeri	Yığın ya da `new` ile öbek	Sınıf nesnesi her zaman öbek
`b = a`	Kopya (kopya yapıcı / atama)	Referans kopyası, aynı nesne
Bellek yönetimi	Yıkıcı, RAII, akıllı işaretçiler	Çöp toplayıcı; kaynak için `IDisposable`
Varsayılan erişim	`class` : private, `struct` : public	Üyeler: `private` (her ikisinde)
Erişim bölümü	`public:` bölümleri	Her üyenin başına belirleyici
Yapıcı ilk değerleri	Üye ilk değer listesi	Gövdede atama, alan başlangıç değeri, `: this(...)`
Veriye erişim	`get` / `set` fonksiyonları	`property` (`get` / `set` / `init`)
Sabit	`const` , `constexpr` , `static const`	`const` (derleme), `readonly` (çalışma)
`static` erişimi	`Sinif::uye` ve `nesne.uye`	Yalnızca `Sinif.Uye`
Metni dönüştürme	`operator<<`	`ToString()` ezme
Boş değer	`nullptr`	`null` ; `?.` , `??` , `is null`
Kopya	Otomatik kopya yapıcı, üçler/beşler kuralı	Elle `Kopyala()` ya da `record` + `with`

Bölüm Özeti

- Sınıf, veri ve işlemleri tek tipte toplayan şablondur; nesne `new` ile üretilen örneğidir. Sınıf tanımları top-level deyimlerin altına yazılır.
- Alanlar `new` sırasında varsayılan değerle (`0`, `false`, `'\0'`, `null`) başlar; yerel değişkenlerin aksine değer atamadan okunabilir.
- Üyelerde varsayılan erişim `private`, sınıfın kendisinde `internal`'dir. `private` sınıfa özeldir, nesneye değil. Alanlar `private`, sunulan üyeler `public` olur.
- Özellik (`get` / `set`), alan gibi kullanılan ama kod çalıştıran üyedir. `{ get; set; }`, `{ get; private set; }`, `{ get; }`, `{ get; init; }` ve `=>` ile hesaplanan özellik, veriye gereken kadar yetki vermenin yollarıdır.
- Özellik `ref` / `out` argümanı yapılamaz; doğrulama `set` içinde yapılır.
- Yapıcının adı sınıfla aynıdır, dönüş tipi yoktur. Bir yapıcı yazılınca otomatik varsayılan yapıcı kaybolur. `: this(...)` ile yapıcılar zincirlenir; zincirdeki hedef yapıcının gövdesi önce çalışır.
- Nesne başlatıcı (`new T { Ad = "Elif" }`) yapıcıdan sonra özellik atar; `init` yalnızca yapıcıda ve başlatıcıda atamaya, `required` başlatıcıda zorunlu verilmeye izin verir. Birincil yapıcı (C# 12) parametreleri sınıf gövdesinde görünür kılar ama özellik üretmez.
- C#'ta yıkıcı ve kopya yapıcı gündelik kodda yoktur: bellek çöp toplayıcıdadır, kaynaklar `IDisposable` / `using` ile bırakılır (Bölüm 53), kopya elle ya da `record` ile alınır (Bölüm 52).
- `this`, metodun çağrıldığı nesnedir; ad çakışmasını çözmek ve `return this` ile zincirleme yapmak için kullanılır.
- `static` üyeler sınıfa aittir, `Sinif.Uye` ile kullanılır; `static` metot nesne üyelerine erişemez. `static class` yalnızca `static` üye içerir ve nesnesi üretilemez.
- `const` derleme zamanında bilinen ilkel değerler içindir ve kullanan koda gömülür; `readonly` çalışma zamanında, tanımda ya da yapıcıda atanır. Nesne türünden paylaşılan sabit için `static readonly` kullanılır.
- `ToString` ezilmezse tip adı yazılır. `==` sınıf nesnelerinde referansları karşılaştırır; içerik eşitliği için `Equals` ve `GetHashCode` birlikte ezilir.
- Sınıf tipli atamada (`b = a`) referans kopyalanır, iki değişken aynı nesneyi gösterir. Metoda geçen referansın kopyasıdır: içerik değişir, yeniden atama çağıranı etkilemez.
- `null` hiçbir nesneyi göstermeyen referanstır; üyesine erişim `NullReferenceException` doğurur. `?.`, `??`, `??=` ve `is null` bu durumu yönetmenin araçlarıdır.
- Nesne dizisinde `new T[n]` yalnızca `n` adet `null` referans açar; her eleman ayrıca `new` ile doldurulur.

Alıştırmalar

1. Bir sınıfın içinde erişim belirleyicisi yazılmamış bir alanın erişim düzeyi nedir?
   A) `public`
   B) `protected`
   C) `private`
   D) `internal`
   E) Derleme hatası

2. Dışarıdan okunabilen, ama değeri yalnızca sınıfın kendi metotlarında değiştirilebilen bir özellik hangi yazımla tanımlanır?
   A) `public int Sayi { get; set; }`
   B) `public int Sayi { get; private set; }`
   C) `public int Sayi { get; init; }`
   D) `private int Sayi { get; set; }`
   E) `public readonly int Sayi { get; }`

3. Aşağıdaki programdan hangi sonuç alınır?

```
Urun u = new Urun();
Console.WriteLine(u.Ad);

class Urun
{
    public string Ad { get; }
    public Urun(string ad) { Ad = ad; }
}
```

   A) Boş bir satır yazılır
   B) `null` yazılır
   C) `0` yazılır
   D) Derleme hatası oluşur
   E) `NullReferenceException` fırlatılır

4. `const` ile `readonly` arasındaki fark aşağıdakilerden hangisidir?
   A) `const` her tipte kullanılabilir, `readonly` yalnızca sayılarda
   B) `const` değeri derleme zamanında belirlenir, `readonly` çalışma zamanında yapıcıda da atanabilir
   C) `readonly` her zaman `static`'tir
   D) `const` değeri yapıcıda değiştirilebilir
   E) Aralarında fark yoktur

5. Bir sınıfta `Equals` ve `==` ezilmemişse aşağıdaki ifadenin değeri nedir?

```
Nokta p = new Nokta(1, 2);
Nokta q = new Nokta(1, 2);
Console.WriteLine(p == q);
```

   A) `True`
   B) `False`
   C) Derleme hatası
   D) `1, 2`
   E) `NullReferenceException`

6. Aşağıdaki program ne yazdırır?

```
Kutu a = new Kutu { Deger = 1 };
Kutu b = a;
b.Deger += 5;
Kutu c = new Kutu { Deger = a.Deger };
c.Deger *= 2;
Console.WriteLine($"{a.Deger} {b.Deger} {c.Deger}");
Console.WriteLine(a == b);
Console.WriteLine(a == c);
```

```
class Kutu
{
    public int Deger;
}
```

7. Aşağıdaki program ne yazdırır?

```
Sepet s = new Sepet();
Ekle(s);
Sifirla(s);
Console.WriteLine(s.Adet);

static void Ekle(Sepet x)
{
    x.Adet += 3;
}

static void Sifirla(Sepet x)
{
    x = new Sepet();
    x.Adet = 100;
}

class Sepet
{
    public int Adet;
}
```

8. Aşağıdaki program ne yazdırır?

```
Nesne n1 = new Nesne();
Nesne n2 = new Nesne();
Nesne n3 = new Nesne();
Console.WriteLine($"{n1.Id} {n3.Id} {Nesne.Adet}");

class Nesne
{
    public static int Adet;
    public int Id;

    public Nesne()
    {
        Adet++;
        Id = Adet * 10;
    }
}
```

9. Aşağıdaki program ne yazdırır?

```
new Test();
Console.WriteLine();
new Test(1);
Console.WriteLine();

class Test
{
    public Test(int a, int b) { Console.Write("C "); }
    public Test(int a) : this(a, 0) { Console.Write("B "); }
    public Test() : this(7) { Console.Write("A "); }
}
```

10. Aşağıdaki program ne yazdırır?

```
Kisi? k = null;
string ad = k?.Ad ?? "bilinmiyor";
Console.WriteLine(ad);
k = new Kisi();
Console.WriteLine(k?.Ad is null);
k.Ad = "Deniz";
Console.WriteLine(k.Ad.Length);

class Kisi
{
    public string? Ad { get; set; }
}
```

11. Aşağıdaki kodda derleme hatasının nedenini bulun ve düzeltin.

```
class Hesap
{
    public readonly decimal Bakiye = 100;

    public void Yatir(decimal miktar)
    {
        Bakiye += miktar;
    }
}
```

12. Aşağıdaki kodda derleme hatasının nedenini bulun ve düzeltin.

```
class Sayac
{
    private int _n;

    public static void Artir()
    {
        _n++;
    }
}
```

13. Aşağıdaki kodda `Ogrenci[] sinif = new Ogrenci[3];` satırından sonra `Console.WriteLine(sinif[1].Ad);` yazılırsa ne olur, nedenini açıklayın; sonra hatayı gidermek için gereken değişikliği yazın.
14. `Dikdortgen` sınıfını yazın: `En` ve `Boy` özellikleri yalnızca pozitif değer kabul etsin (değilse eski değer kalsın), yapıcı iki değer alsın, `Alan()` ve `Cevre()` metotları ile `ToString()` ezilmiş olsun (`3x4` biçiminde). Örneği `new Dikdortgen(3, 4)` ile deneyin.
15. `Urun` sınıfını yazın: `Ad` (salt okunur), `Fiyat` (`decimal`, dışarıdan yalnızca okunur) ve `Stok` özellikleri olsun. `bool Sat(int adet)` metodu stok yeterliyse stoktan düşsün ve `true` döndürsün, yetersizse `false` döndürsün. Üç ürünü bir diziye koyun ve toplam stok değerini (`Fiyat * Stok`) hesaplayın.
16. `Ogrenci` nesnelerinden oluşan bir dizi tanımlayın (`Ad`, `Ortalama`). Ortalaması en yüksek öğrenciyi bulan ve `Ogrenci?` döndüren `EnIyi(Ogrenci[] liste)` isimli `static` bir metot yazın; dizi boşsa `null` döndürsün. Sonucu `?.` ve `??` kullanarak güvenli biçimde yazdırın.

BÖLÜM 50

Kalıtım

BU BÖLÜMDE

- Kalıtımın neden gerekli olduğunu, `: TabanSinif` söz dizimini ve "bir" ilişkisini
- Türetilmiş sınıfa nelerin geçtiğini; `private`, `protected` ve `internal` üyelerin davranışını
- Tekli kalıtım kuralını ve çoklu kalıtımın yerini arayüzlerin nasıl aldığını
- `base(...)` ile taban yapıcıyı çağırmayı ve yapıcıların çalışma sırasını
- `virtual` / `override` ile metot ezmeyi, `new` ile gizlemeyi ve `sealed` ile kilitlemeyi
- Yukarı ve aşağı dönüşümü; `is`, `as` ve desen eşlemeyle tür denetimini
- `object` sınıfını (`ToString`, `Equals`, `GetHashCode`, `GetType`) ve kalıtım ile kompozisyon farkını

Bölüm 49'da tek bir sınıfın nasıl yazıldığını gördük: alanlar, özellikler, yapıcılar, `static` üyeler. Gerçek programlarda ise birbirine çok benzeyen sınıflar ortaya çıkar. Bir `Ogrenci` de bir `Ogretmen` de bir kişidir; bir `Muhendis` de bir `Mudur` de bir çalışandır. Ortak kısmı her sınıfa kopyalamak hem tekrar demektir hem de bir hatayı birden çok yerde düzeltmek zorunda kalmak demektir.

Kalıtım (inheritance) bu sorunu, ortak kısmı bir sınıfta toplayıp diğer sınıfların onu devralmasıyla çözer. Kavramı Bölüm 31'de C++ ile ayrıntılı işledik: erişim denetimi, yapıcı sırası, `virtual` ve `override`, sanal tablo, saf sanal fonksiyonlar ve sanal yıkıcılar orada anlatıldı. Bu bölümde bu fikirleri yeniden öğretmeyeceğiz. C#'ın aynı mekanizmayı hangi söz dizimiyle sunduğuna ve nerede farklı davrandığına odaklanacağız. En büyük farklar şunlardır: C#'ta bir sınıfın yalnızca tek taban sınıfı olur, her sınıf kökte `object`'ten türer, `override` yazmak zorunludur ve nesne dilimleme (object slicing) diye bir sorun yoktur.

Soyut sınıflar, arayüzler ve çok biçimlilik Bölüm 51'in; `struct`, `enum` ve `record` Bölüm 52'nin konusudur.

50.1 Neden Kalıtım?

Bir okul uygulaması için iki sınıf yazdığımızı düşünelim:

```
class Ogrenci
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
    public int No { get; set; }
    public void Tanit() => Console.WriteLine($"Ben {Ad}, {Yas} yaşındayım.");
}

class Ogretmen
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
```

```
    public string Brans { get; set; } = "";
    public void Tanit() => Console.WriteLine($"Ben {Ad}, {Yas} yaşındayım.");
}
```

`Ad`, `Yas` ve `Tanit` iki sınıfta birebir aynıdır. Yarın bir e-posta alanı eklemek istersek iki yeri değiştirmemiz gerekir; memur, veli, yönetici sınıfları geldikçe iş içinden çıkılmaz hâle gelir. Çözüm, ortak kısmı tek bir `Kisi` sınıfında toplamak ve diğer sınıfları ondan türetmektir.

> **Tanım: Kalıtım**
>
> Kalıtım (inheritance), bir sınıfın (türetilmiş sınıf, derived class) başka bir sınıfın (taban sınıf, base class) üyelerini devralmasıdır. Türetilmiş sınıf taban sınıfın bir özelleşmiş hâlidir: taban sınıfın sahip olduğu her şeye sahiptir ve üstüne kendi üyelerini ekler.

Türetme, sınıf adından sonra gelen iki nokta ile yazılır. `class Ogrenci : Kisi` ifadesini "Ogrenci, Kisi'den türer" diye okuruz.

kalitimGiris.cs

```
Ogrenci o = new Ogrenci { Ad = "Elif", Yas = 19, No = 101 };
o.Tanit();                  // Kisi'den gelen metot
o.DersCalis();

Ogretmen t = new Ogretmen { Ad = "Mehmet", Yas = 41, Brans = "C#" };
t.Tanit();
t.DersAnlat();

class Kisi
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
    public void Tanit() => Console.WriteLine($"Ben {Ad}, {Yas} yaşındayım.");
}

class Ogrenci : Kisi
{
    public int No { get; set; }       // yalnız öğrencide olan özellik
    public void DersCalis() => Console.WriteLine($"{Ad} ders çalışıyor.");
}

class Ogretmen : Kisi
{
    public string Brans { get; set; } = "";
    public void DersAnlat() => Console.WriteLine($"{Ad} {Brans} anlatıyor.");
}
```

```
ÇIKTI
Ben Elif, 19 yaşındayım.
Elif ders çalışıyor.
Ben Mehmet, 41 yaşındayım.
Mehmet C# anlatıyor.
```

`Ogrenci` içinde `Ad`, `Yas` ve `Tanit` yazmadığımız hâlde `o.Ad` ve `o.Tanit()` çalıştı; bunlar `Kisi`'den devralındı. `DersCalis` metodu da devralınan `Ad`'ı kendi üyesi gibi kullanıyor. Bu kitapta `Kisi` için "taban sınıf", `Ogrenci` için "türetilmiş sınıf" diyeceğiz; "üst sınıf/alt sınıf" ve "ebeveyn/çocuk" aynı anlama gelen yaygın söyleyişlerdir.

Kalıtım bir "bir" (is-a) ilişkisidir: "Öğrenci bir kişidir" doğrudur, tersi doğru değildir. Kalıtım kurmadan önce bu cümleyi söyleyip anlamlı olup olmadığına bakmak iyi bir sınamadır. "Araba bir motordur" cümlesi anlamsızsa kalıtım da yanlıştır; bu durumda sahip olma ilişkisi vardır ve kompozisyon kullanılır (bölümün sonunda döneceğiz).

Üç Dilde · Kalıtımın yazılışı *C · C++ · C#*

	C	C++	C#
Türetme	Yok; `struct` içine `struct` konur	`class B : public A`	`class B : A`
Kalıtım türleri	Yok	`public`, `protected`, `private`	Yalnızca "public" gibi davranan tek tür
Varsayılan	Yok	`class` için `private`, `struct` için `public`	Tek tür olduğu için yazılmaz
Tabana erişim	Yok	`A::f()`	`base.f()`
Taban yapıcı	Yok	Başlatma listesi: `B() : A(5)`	`: base(5)`
Kök sınıf	Yok	Yok	Her sınıf `object` 'ten türer

C++ kısmında `public` yazmayı unutmak sık yapılan bir hataydı (Bölüm 31); C#'ta bu seçenek hiç yoktur, kalıtım her zaman genel arayüzü de devralan türdendir. `private` ve `protected` kalıtım C#'ta bulunmaz.

50.2 Neler Devralınır?

Türetilmiş sınıf, taban sınıfın `public`, `protected` ve `internal` üyelerini adıyla kullanabilir. `private` üyeler için durum biraz incedir. Türetilmiş sınıf nesnesinin içinde taban sınıfın `private` alanları da vardır ve bellekte yer kaplar; ama türetilmiş sınıfın kodu onlara adıyla erişemez. Taban sınıfın `public` ya da `protected` üyeleri üzerinden dolaylı yoldan ulaşılır. Bu, C++'taki davranışın aynısıdır.

▸ privateKalitim.cs

```
new Ogrenci().Yaz();

class Kisi
{
    private string ad = "Elif";
    public string Ad => ad;
}

class Ogrenci : Kisi
{
    public void Yaz()
    {
        Console.WriteLine(Ad);     // tamam: public özellik
        Console.WriteLine(ad);     // HATA: private alan
    }
}
```

DERLEYİCİ ÇIKTISI

```
privateKalitim.cs(14,27): error CS0122: 'Kisi.ad' is inaccessible due to its protection level
```

Taban sınıfın bir üyesini türetilmiş sınıflara açmak, ama dışarıya kapalı tutmak istersek `protected` kullanırız. `protected` üyeye yalnızca kendi sınıfı ve türetilmiş sınıflar erişir. C#'ta bir ayrıntı daha vardır: türetilmiş sınıf, `protected` üyeye yalnızca **kendi türünden** (ya da kendisinden türeyen) bir nesne üzerinden erişebilir; taban türünden bir değişken üzerinden erişemez.

▸ protectedHata.cs

```
VadeliHesap h = new VadeliHesap { FaizOrani = 0.05 };
h.Yatir(1000);
Console.WriteLine(h.bakiye);              // HATA 1: sınıfın dışı

class Hesap
{
    protected double bakiye;
    private int islemSayisi;

    public void Yatir(double miktar)
    {
        bakiye += miktar;
        islemSayisi++;
    }
}

class VadeliHesap : Hesap
{
    public double FaizOrani { get; set; }

    public void Karsilastir(VadeliHesap benzer, Hesap diger)
    {
        bakiye += bakiye * FaizOrani;              // tamam: kendi nesnemiz
        Console.WriteLine(benzer.bakiye);          // tamam: VadeliHesap tipi
        Console.WriteLine(diger.bakiye);           // HATA 2: Hesap tipi
        Console.WriteLine(islemSayisi);            // HATA 3: private
    }
}
```

DERLEYİCİ ÇIKTISI

```
protectedHata.cs(3,21): error CS0122: 'Hesap.bakiye' is inaccessible due to its protection level
protectedHata.cs(25,33): error CS1540: Cannot access protected member 'Hesap.bakiye' via a
qualifier of type 'Hesap'; the qualifier must be of type 'VadeliHesap' (or derived from it)
protectedHata.cs(26,27): error CS0122: 'Hesap.islemSayisi' is inaccessible due to its protection
level
```

İkinci hatanın gerekçesi şudur: `diger` aslında başka bir türetilmiş sınıfın (örneğin `CariHesap`) nesnesi olabilir ve bir kardeş sınıfın korunan verisine dokunmak kapsüllemeyi bozar. C++'ta aynı kural bulunur.

Erişim belirleyicilerin kalıtım açısından özeti şöyledir. "Aynı proje" ifadesi, aynı derleme birimini (assembly) anlatır.

Belirleyici	Kendi sınıfı	Türetilmiş sınıf	Aynı proje	Başka proje
`private`	evet	hayır	hayır	hayır
`protected`	evet	evet	hayır	hayır
`internal`	evet	yalnız aynı projede	evet	hayır
`protected internal`	evet	evet	evet	hayır (türetilmişse evet)
`private protected`	evet	yalnız aynı projede	hayır	hayır

Belirleyici	Kendi sınıfı	Türetilmiş sınıf	Aynı proje	Başka proje
`public`	evet	evet	evet	evet

◆ Kritik Nokta

Yapıcılar devralınmaz. Türetilmiş sınıf, taban sınıfın yapıcılarını kendi yapıcısı gibi sunmaz; yalnızca kendi yapıcısının içinden `: base(...)` ile çağırabilir. Aşağıda bunu ayrıntılı göreceğiz.

İpucu

Veriyi `protected` alan yapmak yerine genel okuma, korunan yazma izni veren bir özellik tanımlamak çoğu zaman daha iyidir: `public double Bakiye { get; protected set; }`. Böylece dışarıdaki herkes değeri okur, ama yalnızca sınıfın kendisi ve türetilmiş sınıflar değiştirir (özellikler Bölüm 49'da).

50.3 Tekli Kalıtım Kuralı

C#'ta bir sınıfın `:` işaretinden sonra yalnızca **bir** taban sınıfı olabilir. İki sınıftan birden türetmeye çalışırsak derleyici durdurur:

▸ cokluKalitim.cs

```
Console.WriteLine("Hiyerarşi kurulamadı");

class Telefon { }
class Kamera { }
class AkilliTelefon : Telefon, Kamera { }     // iki taban sınıf: yasak
```

```
DERLEYİCİ ÇIKTISI

cokluKalitim.cs(5,32): error CS1721: Class 'AkilliTelefon' cannot have multiple base classes:
'Telefon' and 'Kamera'
```

Bölüm 31'de C++'ın çoklu kalıtımını ve onun getirdiği elmas problemini görmüştük: iki taban sınıf ortak bir üst sınıftan türüyorsa nesnede o üst sınıfın iki kopyası oluşur ve `virtual` kalıtımla bu çözülürdü. C# tasarımcıları sorunu kökten kesmeyi seçti: sınıflar için çoklu kalıtım yoktur. Birden fazla "tür" olmak gerektiğinde **arayüzler** (interface) kullanılır. Bir sınıf tek taban sınıftan türer, ama istediği kadar arayüz uygular: `class AkilliTelefon : Cihaz, ITelefon, IKamera`. Arayüzleri Bölüm 51'de işleyeceğiz.

Serbest olan biçimler şunlardır: zincir biçimindeki çok seviyeli kalıtım (`B : A`, `C : B`) ve bir tabandan birden çok sınıf türeten hiyerarşik kalıtım (`B : A`, `C : A`). Yasak olan yalnızca `C : A, B` biçimindeki iki sınıflı türetmedir.

Üç Dilde · Çoklu kalıtım ve arayüz *C · C++ · C#*

```
// C++: iki sınıftan kalıtım (Bölüm 31)
class AkilliTelefon : public Telefon, public Kamera { };
```

```
// C#: tek sınıf + arayüzler (Bölüm 51)
class AkilliTelefon : Cihaz, ITelefon, IKamera { }
```

> C++'ta her iki taban sınıf da gerçek kod ve veri taşıyabilir; bu güç elmas problemi ve belirsiz çağrılar pahasına gelir. C#'ta arayüz yalnızca "ne yapılabileceğini" söyler, kodu ve veriyi tek taban sınıf taşır. Güç biraz azalır, ama dil çok daha öngörülebilir olur.

50.4 Yapıcılar ve base(...)

Yapıcılar devralınmadığı hâlde, bir `Ogrenci` nesnesinin içinde bir de `Kisi` kısmı vardır ve o kısmın başlangıç değerlerini `Kisi`'nin yapıcısı vermelidir. Türetilmiş sınıfın yapıcısı, taban yapıcıyı yapıcı başlığında `: base(...)` ile çağırır. C++'taki başlatma listesinin (`Ogrenci(...) : Kisi(ad, yas)`) karşılığı budur.

baseYapici.cs

```
Ogrenci o = new Ogrenci("Elif", 19, 101);
o.Yaz();

class Kisi
{
    public string Ad { get; }
    public int Yas { get; }

    public Kisi(string ad, int yas)
    {
        Ad = ad;
        Yas = yas;
        Console.WriteLine("Kisi yapıcısı çalıştı");
    }
}

class Ogrenci : Kisi
{
    public int No { get; }

    public Ogrenci(string ad, int yas, int no) : base(ad, yas)
    {
        No = no;
        Console.WriteLine("Ogrenci yapıcısı çalıştı");
    }

    public void Yaz() => Console.WriteLine($"{No} {Ad} ({Yas})");
}
```

```
ÇIKTI
Kisi yapıcısı çalıştı
Ogrenci yapıcısı çalıştı
101 Elif (19)
```

`: base(ad, yas)` parametre listesinden sonra, gövdeden önce yazılır ve gövdeden önce çalışır. Önce taban kısım kurulur, sonra türetilmiş sınıfın kendi satırları gelir. Kurallar şunlardır:

1. `: base(...)` yazmazsak derleyici gövdeden önce örtük olarak `: base()` ekler, yani taban sınıfın parametresiz yapıcısını çağırır.
2. Yapıcı başlığında `:` işaretinden sonra tek bir çağrı bulunabilir: ya `: this(...)` (Bölüm 49'daki yapıcı zinciri) ya da `: base(...)`. İkisi birlikte yazılamaz; zincirin son halkası mutlaka bir `base` çağrısına ulaşır.

İlk kuralın sonucu, en sık karşılaşılan derleme hatalarından birini doğurur. Taban sınıfta parametresiz yapıcı yoksa ve türetilmiş sınıf `: base(...)` yazmıyorsa, derleyicinin eklediği `: base()` çağrısının karşılığı bulunamaz:

baseEksik.cs

```
Ogrenci o = new Ogrenci(101);

class Kisi
{
    public string Ad { get; }
    public Kisi(string ad) => Ad = ad;
}

class Ogrenci : Kisi
{
    public int No { get; }
    public Ogrenci(int no) => No = no;     // derleyici buraya : base() ekler
}
```

DERLEYİCİ ÇIKTISI

```
baseEksik.cs(12,12): error CS7036: There is no argument given that corresponds to the required
parameter 'ad' of 'Kisi.Kisi(string)'
```

▲ Sık Yapılan Hata

`CS7036` hatasını alırsak çözüm iki yönlüdür: türetilmiş yapıcıya uygun bir `: base("...")` çağrısı eklemek ya da taban sınıfa parametresiz bir yapıcı yazmak. Türetilmiş sınıfta hiç yapıcı yazmasak da aynı hatayı alırız, çünkü derleyicinin eklediği varsayılan yapıcı `Ogrenci() : base()` biçimindedir.

Tarihçe · Birincil yapıcılar

C# 12 (2023), sınıf adının yanına parametre yazmaya izin verir (birincil yapıcı, primary constructor; Bölüm 49). Kalıtımda taban yapıcı çağrısı da aynı satıra gelir. Klasik `: base(...)` yazımı geçerliliğini korur; birincil yapıcı bir kısayoldur.

birincilKalitim.cs

```
Ogrenci o = new("Elif", 101);
Console.WriteLine(o.Bilgi());

class Kisi(string ad)
{
    public string Ad { get; } = ad;
}

class Ogrenci(string ad, int no) : Kisi(ad)
{
    public int No { get; } = no;
    public string Bilgi() => $"{No} {Ad}";
}
```

ÇIKTI

```
101 Elif
```

Yapıcıların çalışma sırası

Kalıtım zincirinde her yapıcının ilk işi (açıkça ya da örtük olarak) taban yapıcıyı çağırmaktır. Bu yüzden `new Kopek()` dediğimizde yapıcı gövdeleri en üstten aşağıya doğru çalışır. C++'ta da sıra böyleydi (Bölüm 31). Yıkıcıların ters sırayla çalışması konusu ise C#'ta yoktur, çünkü bellek temizliği çöp toplayıcıya aittir (Bölüm 49).

kurucuSirasi.cs

```
new Kopek();
Console.WriteLine("---");
new Hayvan();

class Canli
{
    public Canli() => Console.WriteLine("1. Canli yapıcısı");
}

class Hayvan : Canli
{
    public Hayvan() => Console.WriteLine("2. Hayvan yapıcısı");
}

class Kopek : Hayvan
{
    public Kopek() => Console.WriteLine("3. Kopek yapıcısı");
}
```

ÇIKTI

```
1. Canli yapıcısı
2. Hayvan yapıcısı
3. Kopek yapıcısı
---
1. Canli yapıcısı
2. Hayvan yapıcısı
```

Hiçbir yapıcıda `: base()` yazmadık; derleyici hepsine ekledi. Zincirin en tepesinde de `object` sınıfının yapıcısı vardır; bunu çıktıda görmeyiz, ama bütün zincir ona dayanır.

C# ile C++ arasında burada önemli bir **fark** vardır: alan başlatıcılarının (`private string ad = "hazır";`) çalışma yeri. C++'ta taban sınıf kısmı tamamen kurulur, sonra türetilmiş sınıfın üyeleri başlatılır. C#'ta ise türetilmiş sınıfın alan başlatıcıları, taban yapıcı çağrılmadan **önce** çalışır; sonra taban yapıcı, sonra türetilmiş yapıcının gövdesi gelir. Sonuç, yapıcıdan sanal metot çağırdığımızda ortaya çıkar:

yapiciSanal.cs

```
new Turetilmis();

class Taban
{
    public Taban()
    {
        Console.WriteLine("Taban yapıcısı");
        Bildir();                       // sanal çağrı
    }

    public virtual void Bildir() => Console.WriteLine("Taban.Bildir");
}

class Turetilmis : Taban
{
```

```
    private string ad = "hazır";        // alan başlatıcı
    private int sayi;

    public Turetilmis()
    {
        sayi = 5;
        Console.WriteLine("Turetilmis yapıcısı");
    }

    public override void Bildir() =>
        Console.WriteLine($"Turetilmis.Bildir: ad={ad}, sayi={sayi}");
}
```

```
ÇIKTI
Taban yapıcısı
Turetilmis.Bildir: ad=hazır, sayi=0
Turetilmis yapıcısı
```

Bölüm 31'de C++'ın yapıcı içindeki sanal çağrıyı nesnenin türetilmiş kısmı henüz yokken **taban sürümüne** bağladığını görmüştük. C# böyle korumaz: çağrı, nesnenin gerçek türü olan `Turetilmis`'in `Bildir` metoduna gider. Bu metot, yapıcı gövdesi henüz çalışmadığı için `sayi` alanını `0` görür; ama alan başlatıcısı çoktan çalıştığı için `ad` doludur. Yani tutarsız bir yarı kurulu nesneyle karşılaşabiliriz.

> **▲ Sık Yapılan Hata**
>
> Yapıcıdan sanal metot çağırmaktan kaçınmalıyız. C++'ta çağrı sessizce taban sürümüne gider; C#'ta ise türetilmiş sınıfın henüz tam kurulmamış nesnesi üzerinde çalışır. İki dilde de kaynak aynıdır: nesne kurulurken türü kesin değildir.

50.5 Metot Ezme: virtual ve override

Türetilmiş sınıf, taban sınıftan gelen bir metodu aynı imzayla yeniden yazarak kendi davranışını tanımlayabilir. Buna metot **ezme** (overriding) denir. C++'ta olduğu gibi iki anahtar sözcük birlikte çalışır: taban sınıftaki metot `virtual` (ezilebilir) olmalı, türetilmiş sınıftaki metot `override` ile yazılmalıdır. C#'ta da metotlar varsayılan olarak sanal değildir.

▸ ezme.cs

```
Kedi k = new Kedi { Ad = "Pamuk" };
Kopek d = new Kopek { Ad = "Boncuk" };
Balik b = new Balik { Ad = "Nemo" };

k.SesCikar();
d.SesCikar();
b.SesCikar();
Console.WriteLine("---");
k.Tanit();
d.Tanit();
b.Tanit();

class Hayvan
{
    public string Ad { get; set; } = "";

    public virtual void SesCikar() => Console.WriteLine("...");   // ezilebilir

    public void Tanit()
```

```
    {
        Console.Write($"Ben {Ad}. Sesim: ");
        SesCikar();                      // hangi SesCikar çalışacak?
    }
}

class Kedi : Hayvan
{
    public override void SesCikar() => Console.WriteLine("Miyav!");
}

class Kopek : Hayvan
{
    public override void SesCikar() => Console.WriteLine("Hav hav!");
}

class Balik : Hayvan { }                 // SesCikar ezilmedi
```

```
ÇIKTI
Miyav!
Hav hav!
...
---
Ben Pamuk. Sesim: Miyav!
Ben Boncuk. Sesim: Hav hav!
Ben Nemo. Sesim: ...
```

İkinci kısma dikkat edelim. `Tanit` metodu `Hayvan` sınıfının kodudur ve içinde `SesCikar()` çağrısı vardır. Yine de `k.Tanit()` "Miyav!" yazdırdı. Ezilmiş (`virtual`) bir metot çağrıldığında çalışma zamanı değişkenin tipine değil **nesnenin gerçek sınıfına** bakar; çağrı taban sınıfın içinden yapılmış olsa bile. Buna dinamik bağlama (dynamic binding) denir ve Bölüm 31'deki sanal tablo mekanizmasının aynısıdır. Bu davranışın farklı türleri tek bir listede toplayarak kullanılması, yani çok biçimlilik, Bölüm 51'in konusudur.

Ezmenin kuralları şunlardır:

Kural	Açıklama
Taban metot `virtual` (ya da `abstract` / `override`) olmalı	`virtual` olmayan metot ezilemez (`CS0506`)
Türetilmiş metot `override` yazmalı	Yazmazsak ezme olmaz, gizleme olur (aşağıda)
İmza aynı olmalı	Ad ve parametre listesi birebir aynı; liste farklıysa aşırı yüklemedir, ezilecek metot yoksa `CS0115`
Erişim belirleyici aynı olmalı	`public` bir metot `protected override` yapılamaz (`CS0507`)
`static` ve `private` metotlar	Ezilemez, `virtual` olamaz

C++'tan **fark** şudur: C++'ta `override` isteğe bağlıdır (yazılması önerilir, Bölüm 31), C#'ta ise ezme için zorunludur. Yanlış yazılmış bir ezme, örneğin `SesCikart`, sessizce yeni bir metoda dönüşmez; ya derleyici hata verir (`CS0115`) ya da, `override` hiç yazılmamışsa, aşağıda göreceğimiz gizleme uyarısını verir. Bir diğer fark, dönüş tipiyle ilgilidir: C# 9'dan beri ezen metodun dönüş tipi, taban metodunkinin türetilmiş bir sınıfı olabilir (kovaryant dönüş, covariant return). C++'ta bu yalnızca işaretçi ve referanslar için vardı.

> **Not · Aşırı yükleme ile ezme**
>
> İki kavram sık karıştırılır. Aşırı yükleme (overloading), aynı adlı ama farklı parametre listeli metotları anlatır ve hangisinin çağrılacağı **derleme zamanında**, argüman tiplerine göre seçilir (Bölüm 46). Ezme ise türetilmiş sınıfta

> aynı imzayla yeniden yazmadır ve hangisinin çalışacağı **çalışma zamanında**, nesnenin gerçek sınıfına göre seçilir. İkisi aynı sınıfta bir arada da bulunabilir.

50.6 base ile Taban Metodunu Çağırmak

Ezme, taban sınıfın metodunu tamamen değiştirir. Çoğu zaman ise taban sınıfın yaptığını yeniden yazmadan genişletmek isteriz. Türetilmiş sınıfın içinde `base.MetotAdi(...)` yazarak taban sınıfın sürümünü çağırabiliriz. Bu, C++'taki `Taban::MetotAdi()` yazımının karşılığıdır.

▸ baseMetot.cs

```
Kisi k = new Kisi { Ad = "Mehmet", Yas = 30 };
Ogrenci o = new Ogrenci { Ad = "Ayşe", Yas = 20, No = 102 };
k.Tanit();
Console.WriteLine("---");
o.Tanit();

class Kisi
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }

    public virtual void Tanit() =>
        Console.WriteLine($"Ad: {Ad}, yaş: {Yas}");
}

class Ogrenci : Kisi
{
    public int No { get; set; }

    public override void Tanit()
    {
        base.Tanit();                          // önce ortak bilgiler
        Console.WriteLine($"No: {No}");
    }
}
```

```
ÇIKTI
Ad: Mehmet, yaş: 30
---
Ad: Ayşe, yaş: 20
No: 102
```

> **▲ Sık Yapılan Hata**
>
> `Ogrenci.Tanit()` içinde `base.Tanit();` yerine `Tanit();` yazarsak metot kendini çağırır, o da yine kendini çağırır ve sonu gelmeyen çağrı zinciri çağrı yığınını doldurur. Program `StackOverflowException` ile çöker ve C#'ta bu istisna yakalanamaz (özyineleme Bölüm 58'de). Taban sürümünü istiyorsak `base.` şarttır. `base.base.Metot()` biçiminde bir yazım yoktur.

50.7 Gizleme ve new

Taban sınıfta `virtual` bir metot varken türetilmiş sınıfta aynı imzalı bir metodu `override` yazmadan yazarsak ne olur? C++'ta bu sessiz bir gizlemeydi (Bölüm 31). C# hata vermez, ama bir uyarı verir ve metodu ezme olarak değil, taban metodu **gizleyen** ayrı bir metot olarak alır.

gizlemeUyari.cs

```
Kedi kedi = new Kedi();
Hayvan h = kedi;
kedi.SesCikar();
h.SesCikar();

class Hayvan
{
    public virtual void SesCikar() => Console.WriteLine("...");
}

class Kedi : Hayvan
{
    public void SesCikar() => Console.WriteLine("Miyav!");    // override yok
}
```

ÇIKTI

```
Miyav!
...
```

Derleyici şu uyarıyı verir: `warning CS0114: 'Kedi.SesCikar()' hides inherited member 'Hayvan.SesCikar()'. To make the current member override that implementation, add the override keyword. Otherwise add the new keyword.` Uyarı iki çözüm sunar: niyetimiz ezmekse `override`, gizlemekse `new` yazmak. Taban metot `virtual` değilse aynı durumun uyarı kodu `CS0108`'dir.

> **▲ Sık Yapılan Hata**
>
> Yukarıdaki program derlenir ve çalışır; tehlikesi de buradadır. `h.SesCikar()` nesne `Kedi` olduğu hâlde taban sınıfın metodunu çalıştırdı ve çok biçimlilik sessizce bozuldu. Uyarıyı görmezden gelmemeliyiz: metodu `virtual` yaptıysak türetilmiş sınıfta `override` yazarız. `new` anahtar sözcüğünü uyarıyı susturmak için değil, "bu gizlemeyi bilerek yapıyorum" demek için kullanırız.

Gizlemeyi bilerek yapmak istersek metoda `new` ekleriz. Aynı nesneyi iki farklı tipte değişkenle çağırıp `override` ile `new` arasındaki farkı görelim:

newKarsilastirma.cs

```
Kedi kedi = new Kedi();
Hayvan hayvan = kedi;                  // aynı nesne, taban tipli değişken

Console.WriteLine("Kedi değişkeni:");
kedi.SesCikar();
kedi.Hareket();
Console.WriteLine("Hayvan değişkeni:");
hayvan.SesCikar();
hayvan.Hareket();

class Hayvan
{
    public virtual void SesCikar() => Console.WriteLine("Hayvan sesi: ...");
```

```
    public void Hareket() => Console.WriteLine("Hayvan yürüyor");
}

class Kedi : Hayvan
{
    public override void SesCikar() => Console.WriteLine("Kedi sesi: Miyav!");
    public new void Hareket() => Console.WriteLine("Kedi sinsice yürüyor");
}
```

```
ÇIKTI
Kedi değişkeni:
Kedi sesi: Miyav!
Kedi sinsice yürüyor
Hayvan değişkeni:
Kedi sesi: Miyav!
Hayvan yürüyor
```

Aynı nesne, aynı çağrı biçimi, farklı sonuç:

Çağrı	Değişkenin tipi	Nesne	Çalışan metot
`kedi.SesCikar()`	`Kedi`	`Kedi`	`Kedi` (`override`)
`hayvan.SesCikar()`	`Hayvan`	`Kedi`	`Kedi` (`override`, nesneye bakılır)
`kedi.Hareket()`	`Kedi`	`Kedi`	`Kedi` (`new`)
`hayvan.Hareket()`	`Hayvan`	`Kedi`	`Hayvan` (`new`, değişkenin tipine bakılır)

> ◆ **Kritik Nokta**
>
> Taban tipli bir değişken üzerinden yapılan çağrıda şunu sorarız: metot `virtual` / `override` çifti mi? Öyleyse **nesne** kazanır ve çok biçimlilik vardır. `new` ile gizlenmiş ya da sıradan (sanal olmayan) bir metot mu? Öyleyse **değişkenin tipi** kazanır. Bu kural taban sınıfın kendi içinden yapılan çağrılar için de geçerlidir: taban sınıfın kodundaki `Hareket()` çağrısı her zaman taban sınıfın `Hareket`'ine gider.

Gizleme çok biçimliliği kıran bir araçtır; gerçek projelerde nadiren gerekir. Başlıca meşru kullanım yeri, kendi yazmadığımız bir taban sınıfa sonradan aynı adlı bir üye eklendiğinde derleme uyarısını bilinçli olarak karşılamaktır.

50.8 sealed: Ezilemez, Türetilemez

`sealed` (mühürlü) iki yerde kullanılır. Bir sınıfa yazılırsa o sınıftan türetilemez; `string` sınıfı `sealed`'dır. Bir `override` metoda yazılırsa (`sealed override`) daha aşağıdaki türetilmiş sınıflar o metodu yeniden ezemez. `sealed` tek başına, ezme olmayan bir metoda yazılamaz. Bu anahtar sözcük C++'taki `final` karşılığıdır (Bölüm 31).

```
▸ sealedHatalari.cs
Console.WriteLine("...");

sealed class Muhur { }
class Deneme : Muhur { }                    // HATA 1: sealed sınıftan türetme

class Ust
{
    public virtual void Kritik() => Console.WriteLine("Ust");
```

```
    public sealed void Tek() => Console.WriteLine("Tek");   // HATA 2
}

class Orta : Ust
{
    public sealed override void Kritik() => Console.WriteLine("Orta");
}

class Alt : Orta
{
    public override void Kritik() => Console.WriteLine("Alt");   // HATA 3
}
```

```
DERLEYİCİ ÇIKTISI

sealedHatalari.cs(4,16): error CS0509: 'Deneme': cannot derive from sealed type 'Muhur'
sealedHatalari.cs(9,24): error CS0238: 'Ust.Tek()' cannot be sealed because it is not an
override
sealedHatalari.cs(19,26): error CS0239: 'Alt.Kritik()': cannot override inherited member
'Orta.Kritik()' because it is sealed
```

Bir sınıfın ya da metodun davranışının türetilmiş sınıflarca bozulmasını istemediğimizde (güvenlik, doğruluk, performans) `sealed` kullanırız. `struct` türleri de başka bir sınıftan türetilemez; onları Bölüm 52'de göreceğiz.

50.9 Yukarı ve Aşağı Dönüşüm

"Öğrenci bir kişidir" olduğuna göre `Ogrenci` nesnesini `Kisi` tipli bir değişkende tutabiliriz. Buna **yukarı dönüşüm** (upcasting) denir; her zaman güvenlidir ve cast yazmak gerekmez:

```
Ogrenci o = new Ogrenci { Ad = "Elif", No = 101 };
Kisi k = o;                 // yukarı dönüşüm: Ogrenci -> Kisi
```

`o` ve `k` **aynı nesneyi** gösterir (sınıf değişkenleri referanstır, Bölüm 49). Değişen tek şey değişkenin tipidir; tip bir gözlük gibidir: `k` üzerinden yalnızca `Kisi`'de tanımlı üyeleri görürüz, `k.No` derlenmez. Ama `virtual`/`override` çifti olan bir metodu `k` üzerinden çağırırsak yine `Ogrenci`'nin ezilmiş sürümü çalışır. Bölüm 31'deki işaretçi ve referanslardan farklı olarak, C#'ta sınıf nesneleri hiçbir zaman değerle kopyalanmaz; dolayısıyla taban tipli bir değişkene atama nesneyi **dilimlemez**. Dilimleme tuzağı C#'ta sınıflar için yoktur.

Tersi yön, yani `Kisi` tipli değişkenden tekrar `Ogrenci`'ye dönüş, **aşağı dönüşüm** (downcasting) olarak adlandırılır. Her `Kisi` bir `Ogrenci` olmadığı için önce sormamız gerekir. Üç yol vardır:

- `is` deseni: nesne uyuyorsa dönüştürüp yeni bir değişkene atar.
- `as` işleci: uyuyorsa dönüştürür, uymuyorsa `null` verir.
- Cast `(Ogrenci)k`: uymuyorsa `InvalidCastException` fırlatır.

```
▸ donusumler.cs

Kisi k1 = new Ogrenci { Ad = "Elif", No = 101 };
Kisi k2 = new Kisi { Ad = "Mert" };

if (k1 is Ogrenci o1)                  // is deseni: uyuyorsa o1'e ata
{
    Console.WriteLine($"k1 öğrenci, no: {o1.No}");
}

Ogrenci? s1 = k1 as Ogrenci;           // as: uymuyorsa null
```

```
Ogrenci? s2 = k2 as Ogrenci;
Console.WriteLine($"{s1 != null} {s2 == null}");
Console.WriteLine($"{k1 is Ogrenci} {k2 is Ogrenci}");

try
{
    Ogrenci o2 = (Ogrenci)k2;          // cast: uymuyorsa istisna
    Console.WriteLine(o2.No);
}
catch (InvalidCastException e)
{
    Console.WriteLine("Dönüşüm başarısız: " + e.Message);
}

class Kisi
{
    public string Ad { get; set; } = "";
}

class Ogrenci : Kisi
{
    public int No { get; set; }
}
```

```
ÇIKTI
k1 öğrenci, no: 101
True True
True False
Dönüşüm başarısız: Unable to cast object of type 'Kisi' to type 'Ogrenci'.
```

`k1 is Ogrenci o1` tek ifadede hem soru hem dönüşümdür: nesne gerçekten `Ogrenci` ise `true` olur ve `o1` hazırdır; değilse `false` olur. Desen değişkeni `o1` yalnızca koşulun doğru olduğu yerde kesin olarak atanmış sayılır. `as` uymadığında `null` verdiği için sonuç tipini `Ogrenci?` yazdık (Bölüm 59'da nullable referans tiplerini ayrıntılı göreceğiz). Cast ise `k2` gerçekte bir `Kisi` olduğu için istisna fırlattı; yakalanmayan istisnaları Bölüm 53'te işleyeceğiz.

Yazım	Nesne uyuyorsa	Nesne uymuyorsa
`(Ogrenci)k`	`Ogrenci` döner	`InvalidCastException`
`k as Ogrenci`	`Ogrenci` döner	`null` döner
`k is Ogrenci o`	`true`, `o` dolu	`false`

C++ karşılaştırması: `dynamic_cast<Ogrenci*>(k)` işaretçide `nullptr` veren (yani `as`'e benzeyen), referansta `std::bad_cast` fırlatan (yani cast'e benzeyen) bir araçtı. `static_cast` ile taban işaretçisini türetilmişe çevirmek ise çalışma zamanında denetlenmezdi. C#'ta bu denetimsiz yol yoktur: her referans dönüşümü çalışma zamanında denetlenir ve bellek güvenliği bozulmaz.

> ◆ **Kritik Nokta**
>
> `null is Ogrenci` her zaman `false`'tur; `null as Ogrenci` ise `null`'dır. `as` yalnızca referans tiplerinde (ve nullable değer tiplerinde) kullanılır. Aşağı dönüşüm nesneyi değiştirmez, yalnızca onu görme biçimimizi, yani değişkenin tipini değiştirir.

Tür denetiminde desen eşleme

`is` ifadesi, C# 7'den itibaren gelişen **desen eşleme** (pattern matching) özelliklerinin başlangıcıdır. Değeri bir türe karşı sınamak için `switch` ifadesinde de tür desenleri kullanılabilir. Tür sınamasını tersine çevirmek için C# 9 ile gelen `is not` yazımı da vardır.

desenEsleme.cs

```
Kisi[] liste =
{
    new Ogrenci { Ad = "Elif", No = 101 },
    new Ogretmen { Ad = "Mehmet", Brans = "Matematik" },
    new Kisi { Ad = "Can" }
};

foreach (Kisi kisi in liste)
{
    Console.WriteLine(Aciklama(kisi));
}

Kisi? yok = null;
Console.WriteLine(yok is not Ogrenci);     // null için de true

static string Aciklama(Kisi? k) => k switch
{
    Ogrenci o   => $"Öğrenci {o.Ad}, no {o.No}",
    Ogretmen t  => $"Öğretmen {t.Ad}, branş {t.Brans}",
    null        => "kimse yok",
    _           => $"Kişi {k.Ad}"
};

class Kisi
{
    public string Ad { get; set; } = "";
}

class Ogrenci : Kisi
{
    public int No { get; set; }
}

class Ogretmen : Kisi
{
    public string Brans { get; set; } = "";
}
```

ÇIKTI

```
Öğrenci Elif, no 101
Öğretmen Mehmet, branş Matematik
Kişi Can
True
```

`switch` kollarının sırası önemlidir: derleyici, bir kolu kendinden önceki kollar zaten kapsıyorsa hata verir. Bu yüzden önce özel (türetilmiş) türler, en sonda genel (taban) tür ya da `_` yazılır. Gerçekte `kisi` bir `Ogrenci` iken ilk kol çalışır, çünkü sınama yukarıdan aşağıya yapılır.

> **▲ Sık Yapılan Hata**
>
> Uzun bir `if (x is A) ... else if (x is B) ...` zinciri ya da tür sınayan `switch`, çoğu zaman çok biçimliliğin yerine kullanılmış demektir. Her türe özgü davranış, o türün `virtual` metoduna taşınabiliyorsa taşınmalıdır; tür sınama, davranışı sınıfa koyamadığımız durumlarda (örneğin başkasının yazdığı sınıflar) başvurulacak araçtır.

50.10 object: Her Sınıfın Atası

Bir sınıfa `: Taban` yazmadığımızda sınıf, örtük olarak C#'ın kök sınıfı olan `object`'ten türer: `class Kisi { }` ile `class Kisi : object { }` aynı şeydir. Yapıcı zincirinin tepesinde `object()` bulunmasının nedeni budur. C ve C++'ta böyle bir ortak kök yoktur (`void*` yalnızca tipsiz bir adrestir); C#'ta ise sınıflar, diziler, `string` ve hatta `int` gibi değer tipleri bile (kutulama yoluyla, Bölüm 52) bir `object` olarak ele alınabilir.

> **Tanım: object**
>
> `object` (`System.Object`), C# tip sisteminin kök sınıfıdır. Her tip doğrudan ya da dolaylı olarak ondan türer; bu yüzden her nesnede `ToString`, `Equals`, `GetHashCode` ve `GetType` metotları hazır bulunur.

ortakKok.cs

```
object[] kutu = { 42, "metin", 3.5, 'c', new Urun() };

foreach (object x in kutu)
{
    Console.WriteLine($"{x} -> {x.GetType().Name}");
}

class Urun { }
```

```
ÇIKTI
42 -> Int32
metin -> String
3.5 -> Double
c -> Char
Urun -> Urun
```

Tek bir `object[]` dizisine tamamen farklı tipleri koyabildik. Son satırda `Urun` sınıfı `ToString`'i ezmediği için `object`'in varsayılan sürümü çalıştı ve yalnızca tür adını yazdı. `object`'ten gelen dört üye şunlardır:

Metot	Varsayılan davranış	Genellikle ezilir mi?
`ToString()`	Sınıfın tür adını döndürür	Evet
`Equals(object? obj)`	Sınıflarda referans eşitliği: aynı nesne mi?	Evet, içerik karşılaştırması için
`GetHashCode()`	Nesneye özgü bir tam sayı	`Equals` ezildiyse mutlaka
`GetType()`	Nesnenin gerçek türü (`.Name` ile adı)	Hayır

Bölüm 49'da `ToString` ve `Equals`'ı kendi sınıflarımız için ezmiştik; o zaman bunun aslında `object`'in sanal metotlarını ezmek olduğunu söylememiştik. Bu yüzden metodun `public override string ToString()` biçiminde yazılması gerekir: ezerken erişim belirleyici ve imza değişmez. `Console.WriteLine(nesne)` ve

`$"{nesne}"` bu metodu otomatik çağırır. Türetilmiş sınıf, taban sınıfın metnini `base.ToString()` ile genişletebilir (bölüm sonundaki `Mudur` sınıfına bakalım).

Equals ve GetHashCode

`Equals` sınıflarda varsayılan olarak "aynı nesne mi?" sorusunu sorar, `==` ile aynı sonucu verir. İçerik karşılaştırması istiyorsak ezmeliyiz. İki ürünü, kodları aynıysa eşit sayalım:

equalsOrnek.cs

```
Urun u1 = new Urun { Kod = "K-101", Ad = "Kalem" };
Urun u2 = new Urun { Kod = "K-101", Ad = "Tükenmez Kalem" };
Urun u3 = new Urun { Kod = "D-202", Ad = "Defter" };

Console.WriteLine(u1);                          // ToString ezilmedi
Console.WriteLine(u1 == u2);                    // referans karşılaştırması
Console.WriteLine(u1.Equals(u2));
Console.WriteLine(u1.Equals(u3));
Console.WriteLine(u1.GetHashCode() == u2.GetHashCode());
Console.WriteLine(u1.Equals(null));
Console.WriteLine(object.ReferenceEquals(u1, u2));

class Urun
{
    public string Kod { get; set; } = "";
    public string Ad { get; set; } = "";

    public override bool Equals(object? obj)
        => obj is Urun diger && Kod == diger.Kod;

    public override int GetHashCode() => Kod.GetHashCode();
}
```

```
ÇIKTI
Urun
False
True
False
True
False
False
```

`u1 == u2` sınıflarda referansları karşılaştırır: iki ayrı nesne, dolayısıyla `False`. `Equals` ise yazdığımız kuralı çalıştırır. `obj is Urun diger` hem tür sorusu hem dönüşümdür; `null` hiçbir türden olmadığı için `Equals(null)` `False` döner. `object.ReferenceEquals` ise `Equals` ezilmiş olsa bile her zaman referansları karşılaştırır.

Kuralları şöyle özetleyebiliriz:

- Doğru imza `public override bool Equals(object? obj)` biçimindedir. `Equals(Urun u)` yazarsak ezme değil aşırı yükleme olur ve koleksiyonlar onu çağırmaz.
- İki nesne `Equals`'a göre eşitse `GetHashCode` değerleri de aynı olmalıdır (tersi gerekmez). `HashSet` ve `Dictionary` gibi yapılar (Bölüm 54) nesneleri önce `GetHashCode`'a göre gruplar; bu yüzden `Equals`'ta kullandığımız alanı `GetHashCode`'ta da kullanırız. İkisinden yalnız birini ezersek derleyici uyarı verir (`CS0659`).
- Karma değerin kendisi çalıştırmadan çalıştırmaya değişebilir; bu yüzden örnekte yalnızca iki değerin eşit olup olmadığını yazdırdık.

C++'ta karşılığı `operator==` aşırı yüklemesiydi (Bölüm 32). C#'ta `==` işleci de aşırı yüklenebilir, ama sınıflar için gelenek `Equals` ve `GetHashCode`'u ezmektir. `record` türleri (Bölüm 52) bu ikisini içeriğe göre derleyiciye yazdırır.

GetType ve is

`GetType()` nesnenin **gerçek** çalışma zamanı türünü verir. `is` ise türetilmiş türleri de kapsar. Bu fark, Bölüm 31'deki `typeid` ile `dynamic_cast` farkına benzer.

▸ tipBilgisi.cs

```
Hayvan h = new Kedi();

Console.WriteLine(h.GetType().Name);
Console.WriteLine(h.GetType() == typeof(Kedi));
Console.WriteLine(h.GetType() == typeof(Hayvan));
Console.WriteLine(h is Hayvan);
Console.WriteLine(h.GetType().BaseType?.Name);

class Hayvan { }
class Kedi : Hayvan { }
```

```
ÇIKTI
Kedi
True
False
True
Hayvan
```

`h` değişkeninin tipi `Hayvan`, nesnenin gerçek tipi `Kedi`'dir. `GetType()` tam tür eşleşmesini verir (`Hayvan` ile karşılaştırınca `False`); `is Hayvan` ise `Kedi` bir `Hayvan` olduğu için `True` döner. Tür adını yazdırırken `GetType().Name` kullanmak, bir sonraki bölümün örneğinde göreceğimiz gibi, türetilmiş sınıflarda ezme yapmadan doğru adı yazdırmanın pratik yoludur.

50.11 Kalıtım mı, Kompozisyon mu?

İki sınıf arasındaki her ilişki kalıtım değildir. "Araba bir taşıttır" ilişkisi **is-a**'dır ve kalıtımla kurulur: `class Araba : Tasit`. "Arabanın bir motoru vardır" ilişkisi ise **has-a**'dır ve **kompozisyonla** (composition) kurulur: `Araba` sınıfında `Motor` tipinde bir alan bulunur. `class Araba : Motor` yazmak derlenir, ama tasarım olarak yanlıştır: araba bir motor değildir ve motorun bütün üyelerini dışarıya açmak istemeyiz.

▸ kompozisyon.cs

```
Araba a = new Araba { Marka = "Delta" };
a.YolaCik();

class Motor
{
    public int Guc { get; set; }
    public void Calistir() => Console.WriteLine($"Motor çalıştı: {Guc} BG");
}

class Tasit
{
    public string Marka { get; set; } = "";
}
```

```
class Araba : Tasit                          // is-a: Araba bir Taşıt'tır
{
    private Motor motor = new Motor { Guc = 218 };    // has-a: bir Motoru var

    public void YolaCik()
    {
        motor.Calistir();                    // işi motora devreder
        Console.WriteLine($"{Marka} yola çıktı.");
    }
}
```

```
ÇIKTI
Motor çalıştı: 218 BG
Delta yola çıktı.
```

	is-a (kalıtım)	has-a (kompozisyon)
C#'ta	`class B : A`	Sınıf tipinde alan
Cümle testi	"X bir Y'dir"	"X'in bir Y'si vardır"
Örnek	Kedi - Hayvan, Daire - Şekil	Araba - Motor, Okul - Öğrenci
Bağ	Sıkı: taban sınıftaki değişiklik türetilmiş sınıfları etkiler	Gevşek: parça kolayca değiştirilebilir

◆ Kritik Nokta

Kalıtım cazip görünür, ama yalnızca gerçek bir "bir" ilişkisi varsa kullanılmalıdır. Yaygın tasarım ilkesi "kalıtım yerine kompozisyonu tercih et" biçiminde söylenir: taban sınıf ile türetilmiş sınıf birbirine sıkı bağlanır, kompozisyon ise parçaların değiştirilmesine izin verir. Bu fikir C++ için de aynen geçerlidir (Bölüm 31).

50.12 Örnek: Çalışan Hiyerarşisi

Bölümdeki araçları bir arada kullanalım. `Calisan` taban sınıfı ile iki türetilmiş sınıf yazacağız: `Muhendis` ve `Mudur`. Her türetilmiş sınıf `: base(...)` ile taban yapıcıya değer gönderiyor, `MaasHesapla()` metodunu eziyor ve taban sürümünü `base.` ile kullanıyor. `ToString` taban sınıfta `GetType().Name` sayesinde her çalışan için kendi sınıf adını yazıyor; `Mudur` ise bunu `base.ToString()` ile genişletiyor.

▸ calisanHiyerarsisi.cs

```
Calisan[] kadro =
{
    new Calisan("Ayşe", 40000m),
    new Muhendis("Mehmet", 50000m, 3),
    new Mudur("Burak", 60000m, 5),
    new Muhendis("Deniz", 55000m, 1)
};

decimal toplam = 0m;
Calisan enYuksek = kadro[0];

foreach (Calisan c in kadro)
{
    Console.WriteLine(c);
    toplam += c.MaasHesapla();
    if (c.MaasHesapla() > enYuksek.MaasHesapla())
    {
```

```
        enYuksek = c;
    }
}

Console.WriteLine($"Toplam: {toplam:F2} TL");
Console.WriteLine($"En yüksek: {enYuksek.Ad}");

int yoneticiSayisi = 0;
foreach (Calisan c in kadro)
{
    if (c is Mudur)
    {
        yoneticiSayisi++;
    }
}
Console.WriteLine($"Müdür sayısı: {yoneticiSayisi}");

class Calisan
{
    private decimal temelMaas;
    public string Ad { get; }

    public Calisan(string ad, decimal temelMaas)
    {
        Ad = ad;
        this.temelMaas = temelMaas;
    }

    public virtual decimal MaasHesapla() => temelMaas;

    public override string ToString()
        => $"{GetType().Name} {Ad}: {MaasHesapla():F2} TL";
}

class Muhendis : Calisan
{
    private int projeSayisi;

    public Muhendis(string ad, decimal maas, int projeSayisi) : base(ad, maas)
    {
        this.projeSayisi = projeSayisi;
    }

    public override decimal MaasHesapla()
        => base.MaasHesapla() + projeSayisi * 2500m;      // proje başına
}

class Mudur : Calisan
{
    private int ekipSayisi;

    public Mudur(string ad, decimal maas, int ekipSayisi) : base(ad, maas)
    {
        this.ekipSayisi = ekipSayisi;
    }

    public override decimal MaasHesapla()
        => base.MaasHesapla() + ekipSayisi * 1000m;       // kişi başına

    public override string ToString()
        => base.ToString() + $" (ekip: {ekipSayisi} kişi)";
}
```

```
ÇIKTI
Calisan Ayşe: 40000.00 TL
Muhendis Mehmet: 57500.00 TL
Mudur Burak: 65000.00 TL (ekip: 5 kişi)
Muhendis Deniz: 57500.00 TL
Toplam: 220000.00 TL
En yüksek: Burak
Müdür sayısı: 1
```

Programın ilginç yanları şunlardır:

- `Calisan[] kadro` dizisi dört farklı türden nesne taşıyor; yukarı dönüşüm otomatik yapıldı. `foreach` içindeki `c.MaasHesapla()` çağrısı her nesne için kendi sınıfının sürümüne gitti.
- `Console.WriteLine(c)` yalnızca `Calisan.ToString`'i kullandı; `Muhendis` onu ezmedi. İçindeki `MaasHesapla()` çağrısı yine de nesne `Muhendis` olduğu için `Muhendis`'inkidir.
- `temelMaas` `private` kaldı. Türetilmiş sınıflar ona hiç dokunmadı; taban sınıfın `MaasHesapla()` metodunu `base.` ile kullandılar. Kapsülleme ile kalıtım böyle birlikte çalışır.
- Çıktının `Müdür sayısı` satırı için tür sınamasını `is` ile yaptık. Her türü ayrı ayrı sınamak yerine ortak davranışı sanal metoda koymak, yeni bir tür eklendiğinde mevcut döngüyü değiştirmemizi gerektirmez. Farklı türleri tek bir tip altında ele almanın bu gücünü, soyut sınıflar ve arayüzlerle birlikte Bölüm 51'de ayrıntılı işleyeceğiz.

> *Tarihçe · C#'ta tür sınama*
>
> Bu bölümdeki `is` ile desen eşleme C# 7'de (2017) gelen bir özelliktir; `is not` yazımı ve `switch` ifadesindeki tür desenleri C# 8 ve 9 ile genişletildi. Öncesinde aşağı dönüşüm için `as` ile `null` denetimi ya da `is` sonrası ayrı bir cast gerekiyordu. Kovaryant dönüş tipleri C# 9'da, sınıflar için birincil yapıcılar C# 12'de eklendi.

50.13 C++'tan Gelenler İçin Farklar

Kalıtım konusunda iki dil birbirine çok yakın olsa da C#'ın bilinçli tercihleri öğrenilirken şaşırtabilir. Aşağıdaki tablo Bölüm 31 ile bu bölüm arasındaki başlıca farkları toplar:

Üç Dilde · Bölüm 31 ile farklar *C · C++ · C#*

Konu	C++	C#
Çoklu kalıtım	Var; elmas için `virtual` kalıtım	Sınıflarda yok; arayüzlerle sağlanır
Kalıtım türü	`public` / `protected` / `private`	Tek tür
`override`	İsteğe bağlı (önerilir)	Zorunlu; unutmak `CS0114` / `CS0108` uyarısı
Ezmeyi kilitleme	`final`	`sealed`
Gizleme	Sessiz	Uyarı verir, bilinçli olarak `new` yazılır
Nesne dilimleme	Değerle atama dilimler	Sınıflar referans olduğu için yoktur
Sanal yıkıcı	Taban işaretçisiyle silmek için şart	Yıkıcı sırası sorunu yok; çöp toplayıcı temizler
Yapıcıda sanal çağrı	Taban sürümüne gider	Türetilmiş sürüme gider (alanlar henüz hazır olmayabilir)
Alan başlatıcı sırası	Önce taban kısım, sonra türetilmiş üyeler	Önce türetilmiş alan başlatıcıları, sonra taban yapıcı
Güvenli dönüşüm	`dynamic_cast`	`is` , `as` , cast

Konu	C++	C#
Kök sınıf	Yok	`object`

Bölüm Özeti

- Kalıtım `class Turetilmis : Taban` ile yazılır ve "bir" (is-a) ilişkisini ifade eder. Türetilmiş sınıf, taban sınıfın üyelerini devralır ve kendi üyelerini ekler.
- C#'ta kalıtım türü tek biçimdedir (`private` / `protected` kalıtım yoktur) ve her sınıf doğrudan ya da dolaylı olarak `object`'ten türer.
- Bir sınıfın yalnızca bir taban sınıfı olabilir (`class C : A, B` bir `CS1721` hatasıdır). Birden çok tür olmak için arayüz uygulanır (Bölüm 51); elmas problemi sınıflarda doğmaz.
- `private` üyeler türetilmiş sınıf nesnesinde bulunur ama türetilmiş sınıfın kodu onlara adıyla erişemez. `protected` üyeye türetilmiş sınıf yalnızca kendi türünden bir nesne üzerinden erişir (aksi `CS1540`).
- Yapıcılar devralınmaz. Türetilmiş yapıcı taban yapıcıyı `: base(...)` ile çağırır; yazılmazsa örtük `: base()` eklenir ve taban sınıfta parametresiz yapıcı yoksa `CS7036` oluşur.
- Yapıcı gövdeleri tabandan türetilmişe doğru çalışır. C#'ta türetilmiş sınıfın alan başlatıcıları taban yapıcıdan önce çalışır ve yapıcıdan çağrılan sanal metot türetilmiş sürüme gider.
- Ezme için taban metot `virtual`, türetilmiş metot `override` olmalıdır; imza ve erişim aynı kalır. C#'ta `override` zorunludur ve çağrıda nesnenin gerçek sınıfı seçilir.
- `override` yazmayı unutmak gizleme olur. Gizlemeyi bilerek yapmak için `new` yazılır; gizlenmiş üyede değişkenin tipi, ezilmişte nesnenin gerçek sınıfı kazanır.
- `base.Metot()` taban sürümünü çağırır; `base.` unutulursa metot kendini çağırıp `StackOverflowException` ile çöker.
- `sealed` sınıf türetilemez, `sealed override` metot yeniden ezilemez.
- Yukarı dönüşüm otomatik ve güvenlidir; aşağı dönüşümde önce sorulur: `k is Ogrenci o`, `k as Ogrenci` (`null` döner) ya da cast (uymazsa `InvalidCastException`). `switch` ifadesinde tür desenleri kullanılabilir.
- Sınıf değişkenleri referans olduğu için nesne dilimleme sorunu yoktur.
- `object`, `ToString`, `Equals`, `GetHashCode` ve `GetType` üyelerini her tipe verir. `Equals` ezilirse `GetHashCode` da ezilmelidir. `GetType()` tam türü, `is` türetilmiş türleri de kapsar.
- "Sahip" ilişkisi kalıtımla değil kompozisyonla (alan olarak) kurulur: Araba - Motor.

Alıştırmalar

1. C#'ta kalıtım ile ilgili aşağıdakilerden hangisi doğrudur?

   A) Bir sınıf `:` ile birden fazla sınıftan türeyebilir.
   B) Türetilmiş sınıf, taban sınıfın `private` alanlarına adıyla doğrudan erişebilir.
   C) Taban sınıfın yapıcıları türetilmiş sınıfa devralınır.

D) `sealed` olarak tanımlanmış bir sınıftan başka bir sınıf türetilemez.

E) Ezilen bir metodun erişimi `public`'ten `protected`'a daraltılabilir.

2. Aşağıdaki `B` sınıfındaki işaretli üç metot için hangisi doğrudur?

```
class A
{
    public virtual void Yaz(int x) => Console.WriteLine("A: " + x);
}

class B : A
{
    public override void Yaz(int y) => Console.WriteLine("B: " + y);     // (I)
    public void Yaz(string s) => Console.WriteLine("B: " + s);           // (II)
    public int Yaz(int x, int y) => x + y;                               // (III)
}
```

A) (I) ezmedir; (II) ve (III) aşırı yüklemedir.

B) Üçü de ezmedir.

C) (I) aşırı yüklemedir; (II) ve (III) ezmedir.

D) (III) derleme hatası verir, çünkü dönüş tipi `void` değildir.

E) (I) derleme hatası verir, çünkü parametre adı taban sınıftakinden farklıdır.

3. Aşağıdaki program için hangisi doğrudur?

```
Otobus o = new Otobus(45);
Console.WriteLine(o.Koltuk);

class Arac
{
    public string Plaka { get; }
    public Arac(string plaka) => Plaka = plaka;
}

class Otobus : Arac
{
    public int Koltuk { get; }
    public Otobus(int koltuk) => Koltuk = koltuk;
}
```

A) Derlenir ve `45` yazdırır; `Plaka` özelliği `null` kalır.

B) Derlenmez; `Otobus` yapıcısına eklenen örtük `: base()` çağrısı için `Arac`'ta parametresiz yapıcı yoktur.

C) Derlenir, ama çalışırken `NullReferenceException` oluşur.

D) Derlenmez; çünkü `Otobus` sınıfında `Plaka` özelliği tanımlanmamıştır.

E) Derlenmez; çünkü türetilmiş sınıfın yapıcısında özellik atanamaz.

4. Aşağıdaki programda numaralı satırlardan hangileri derleme hatası verir?

```
Vadeli v = new Vadeli();
Console.WriteLine(v.bakiye);                 // (4)

class Hesap
{
    protected decimal bakiye = 100m;
}
```

```
class Vadeli : Hesap
{
    public void Kontrol(Vadeli v, Hesap h)
    {
        Console.WriteLine(bakiye);          // (1)
        Console.WriteLine(v.bakiye);        // (2)
        Console.WriteLine(h.bakiye);        // (3)
    }
}
```

A) Hiçbiri; hepsi derlenir.
B) Yalnız (4).
C) Yalnız (3).
D) (2), (3) ve (4).
E) (3) ve (4).

5. `Ogrenci`, `Kisi`'den türeyen bir sınıftır. Aşağıdaki program için hangisi doğrudur?

```
Kisi k = new Kisi { Ad = "Can" };
Ogrenci? a = k as Ogrenci;                  // (1)
bool b = k is Ogrenci;                      // (2)
Console.WriteLine($"{a == null} {b}");
Ogrenci c = (Ogrenci)k;                     // (3)
Console.WriteLine(c.No);
```

A) (1) `InvalidCastException` fırlatır; (2) `true` verir.
B) (1) `null` verir; (2) `true` verir; (3) başarılı olur.
C) (1) `null` verir; (2) `false` verir; (3) `InvalidCastException` fırlatır.
D) (1) `null` verir; (2) `false` verir; (3) `null` verir.
E) Üç satır da derleme hatası verir.

6. Aşağıdaki program ekrana ne yazdırır?

```
new Taksi();
Console.WriteLine("---");
new Otomobil();

class Arac
{
    public Arac() => Console.WriteLine("Arac()");
    public Arac(string tur) : this() => Console.WriteLine($"Arac({tur})");
}

class Otomobil : Arac
{
    public Otomobil() : base("oto") => Console.WriteLine("Otomobil()");
    public Otomobil(int kapi) => Console.WriteLine($"Otomobil({kapi})");
}

class Taksi : Otomobil
{
    public Taksi() : this(4) => Console.WriteLine("Taksi()");
    public Taksi(int kapi) : base(kapi) => Console.WriteLine($"Taksi({kapi})");
}
```

7. Aşağıdaki program ekrana ne yazdırır?

```
Tasit t = new Tasit();
Bisiklet b = new Bisiklet();
Tasit tb = b;
Console.WriteLine(t.Bilgi());
Console.WriteLine(b.Bilgi());
Console.WriteLine(tb.Tur());
Console.WriteLine(tb.Tekerlek());
Console.WriteLine(b.Tekerlek() + tb.Tekerlek());

class Tasit
{
    public virtual string Tur() => "Taşıt";
    public int Tekerlek() => 4;
    public string Bilgi() => Tur() + "/" + Tekerlek();
}

class Bisiklet : Tasit
{
    public override string Tur() => "Bisiklet";
    public new int Tekerlek() => 2;
}
```

8. Aşağıdaki program ekrana ne yazdırır?

```
Hayvan h = new Kedi { Ad = "Pamuk" };
Console.WriteLine(h);
Console.WriteLine(h.GetType().Name);
Console.WriteLine(h is Kedi);
Console.WriteLine(h is Kopek);
Kopek? d = h as Kopek;
Console.WriteLine(d == null);
if (h is Kedi k && k.Ad.Length > 4)
{
    Console.WriteLine(k.Mirla());
}
Hayvan g = new Hayvan { Ad = "Boncuk" };
Console.WriteLine(g);
Console.WriteLine(g is Kedi);

class Hayvan
{
    public string Ad { get; set; } = "";
    public override string ToString() => $"Hayvan[{Ad}]";
}

class Kedi : Hayvan
{
    public string Mirla() => Ad + " mırlıyor";
    public override string ToString() => "Kedi:" + base.ToString();
}

class Kopek : Hayvan { }
```

9. Aşağıdaki kodda derleme hatası veren dört nokta vardır. Bunları bulun ve nedenlerini söyleyin.

```
sealed class Sabit { }
class Turev : Sabit { }

class A
{
    public void Yaz() { }
```

```
}

class B : A
{
    public override void Yaz() { }
}

class C
{
    public virtual void Calis() { }
}

class D : C
{
    protected override void Calis() { }
}

class E : A, C { }
```

10. Aşağıdaki program neden istenmeyen bir çıktı üretir? Sorunu açıklayın ve iki farklı düzeltme önerin.

```
Sekil s = new Daire();
Console.WriteLine(s.Ad());

class Sekil
{
    public virtual string Ad() => "Şekil";
}

class Daire : Sekil
{
    public string Ad() => "Daire";
}
```

11. Aşağıdaki ilişkilerden hangisi için kalıtım uygun **değildir**?

    A) `Kare` ve `Sekil`
    B) `Ogrenci` ve `Kisi`
    C) `Kutuphane` ve `Kitap`
    D) `Muhendis` ve `Calisan`
    E) `Kedi` ve `Hayvan`

12. Bir mağaza için ürün sınıflarını yazın:

    - `Urun`: `private` alanlar `ad` (`string`) ve `fiyat` (`decimal`); yapıcı; `Ad` salt okunur özelliği; KDV'li fiyatı döndüren `virtual decimal KdvliFiyat()` (KDV yüzde 20); `ToString` biçimi `Kalem - 24.00 TL` (KDV'li fiyat, iki basamak).
    - `Kitap`: `Urun`'dan türer, ek `Yazar` özelliği vardır. Kitaplarda KDV yüzde 10 olduğu için `KdvliFiyat()` metodunu ezer. `ToString`, taban sınıfın metnine `(yazar: ...)` ekler.
    - `Elektronik`: `Urun`'dan türer, ek `GarantiYil` (`int`) özelliği vardır. `ToString`, taban sınıfın metnine `(garanti: ... yıl)` ekler.

    `Kitap` ve `Elektronik` için yalnızca `(ad, fiyat)` alan bir yapıcı yazın, ek özellikleri nesne başlatıcıyla verin. Ana kodda 20 TL'lik `Kalem`, 150 TL'lik `Sefiller` (Victor Hugo) ve 1200 TL'lik, 2 yıl garantili `Kulaklik` oluşturun; üçünü ve toplam KDV'li tutarı yazdırın. Beklenen çıktı:

```
ÇIKTI
Kalem - 24.00 TL
Sefiller - 165.00 TL (yazar: Victor Hugo)
Kulaklik - 1440.00 TL (garanti: 2 yıl)
Toplam: 1629.00 TL
```

13. `Hesap` sınıfında `protected decimal bakiye` alanı ve `Bakiye` salt okunur özelliği bulunsun. `Yatir(decimal)` ve `virtual bool Cek(decimal)` metotlarını yazın (`Cek`, yetersiz bakiyede `false` döndürür ve bakiyeyi değiştirmez). `VadeliHesap : Hesap` sınıfı `Cek`'i ezsin: hesap `vadesiDoldu` alanı `false` ise her çekim reddedilir, `true` ise taban sınıfın kuralı işler. `Hesap` tipli bir dizide bir normal hesap ile vadesi dolmamış bir vadeli hesabı tutun. Her ikisine 500 TL yatırıp 100 TL çekmeyi deneyin ve sonuçları `Cek 100: True` / `Cek 100: False` biçiminde yazdırın.
14. `Bildirim` taban sınıfı (`Mesaj` özelliği, `virtual string Ozet()`), ondan türeyen `EPosta` (`Konu` özelliği) ve `Sms` (`Numara` özelliği) sınıflarını yazın. `Ozet()` her sınıfta farklı bir metin döndürsün. Üç bildirimi (biri düz `Bildirim`) `Bildirim[]` dizisinde tutun. Dizi üzerinde dolaşıp her özeti yazdırın; `is` deseniyle e-postaların sayısını, `as` ile ilk `Sms` nesnesinin numarasını bulun.

BÖLÜM 51

Polimorfizm, Soyut Sınıflar ve Arayüzler

BU BÖLÜMDE

- Çok biçimliliğin (polymorphism) ne olduğunu; referans tipi ile nesne tipinin hangi kararları belirlediğini
- Sanal metot çağrısının nerede dinamik, nerede statik bağlandığını; `override` ile aşırı yükleme (overload) farkını
- Yapıcı içinden çağrılan sanal metodun C#'ta C++'tan farklı davrandığını
- Soyut sınıf (`abstract`) ve soyut üye yazmayı, kurallarını ve sık hatalarını
- Arayüz (`interface`) tanımlamayı, birden çok arayüz uygulamayı, açık uygulamayı ve varsayılan arayüz metotlarını
- Soyut sınıf ile arayüz arasında tasarım kararı vermeyi
- `IComparable<T>`, `IComparer<T>` ve `IEnumerable<T>` arayüzleriyle kendi tiplerimizi .NET'in sıralama ve `foreach` mekanizmasına bağlamayı
- Bağımlılığı arayüzle tersine çevirmenin (SOLID'deki "D") kodu nasıl esnek yaptığını

Bölüm 50'de kalıtımı gördük: bir sınıf başka bir sınıfı genişletiyor, `virtual` ile işaretlenen metotları `override` ile ezebiliyor. Bu bölüm, bu mekanizmanın asıl getirisini anlatıyor: kodu "genel" bir tipe göre yazmak, ama çalışırken her nesnenin kendi davranışını sergilemesini sağlamak.

C++ kısmında aynı konuyu Bölüm 31'de ayrıntılı işledik: sanal tablo, saf sanal fonksiyon, sanal yıkıcı, nesne dilimleme. Burada o anlatımları tekrarlamayacağız. C#'ın söz dizimini ve C++'tan ayrıldığı yerleri göstereceğiz. En önemli farklar şunlardır: C#'ta saf sanal fonksiyon yerine `abstract` vardır, "yalnızca saf sanal fonksiyonlardan oluşan sınıf" kalıbının yerini dilin kendisinin tanıdığı `interface` almıştır ve nesne dilimleme sorunu yoktur. Bölümün ikinci yarısı, .NET kütüphanesinin arayüzlerle nasıl konuştuğunu ve büyük programlarda arayüzlerin neden bu kadar yaygın kullanıldığını gösterecek.

51.1 Çok Biçimlilik: Aynı Çağrı, Farklı Davranış

Tanım: Çok biçimlilik (polymorphism)

Aynı ifadenin (örneğin `s.Alan()` çağrısının), üzerinde çağrıldığı nesnenin gerçek tipine göre farklı kod çalıştırması. Çağrıyı yazan kod tek bir "genel" tipi bilir; hangi sürümün çalışacağı çalışma zamanında nesneye bakılarak seçilir.

Bir şekil hiyerarşisiyle başlayalım. `Sekil` tabanında `virtual` iki metot var; türetilmiş sınıflar bunları `override` ile ezer. Program, her elemanın gerçekte ne olduğunu bilmeden hepsiyle çalışır:

polimorfikSekil.cs

```
Sekil[] sekiller =
{
    new Daire(1), new Dikdortgen(3, 4), new Kare(2), new Daire(2.5)
};

double toplam = 0;
foreach (Sekil s in sekiller)
{
    Console.WriteLine($"{s.Ad(),-11} {s.Alan(),7:F2}");
    toplam += s.Alan();
}
Console.WriteLine($"Toplam alan: {toplam:F2}");

class Sekil
{
    public virtual double Alan() => 0;
    public virtual string Ad() => "Şekil";
}

class Daire : Sekil
{
    private readonly double yaricap;
    public Daire(double yaricap) { this.yaricap = yaricap; }
    public override double Alan() => Math.PI * yaricap * yaricap;
    public override string Ad() => "Daire";
}

class Dikdortgen : Sekil
{
    private readonly double en, boy;
    public Dikdortgen(double en, double boy) { this.en = en; this.boy = boy; }
    public override double Alan() => en * boy;
    public override string Ad() => "Dikdörtgen";
}

class Kare : Dikdortgen
{
    public Kare(double kenar) : base(kenar, kenar) { }
    public override string Ad() => "Kare";
}
```

ÇIKTI

```
Daire          3.14
Dikdörtgen    12.00
Kare           4.00
Daire         19.63
Toplam alan: 38.78
```

Dizinin tipi `Sekil[]`, döngü değişkeninin tipi `Sekil`; yine de her satırda farklı bir `Alan()` sürümü çalıştı. `Kare`, alan hesabını `Dikdortgen`'den aldı, yalnızca adını değiştirdi. Programa yarın bir `Ucgen` sınıfı eklesek, `foreach` döngüsünde tek karakter değişmesi gerekmez. Çok biçimliliğin kazancı budur: yeni davranış, eski kodu değiştirmeden yeni bir sınıf yazarak eklenir.

Bu örnekte gizli bir tasarım kusuru da var: `Sekil.Alan()` sıfır döndürüyor. "Şekil" soyut bir kavramdır; alanı sıfır değil, belirsizdir. Üstelik kimsenin `new Sekil()` yazıp anlamsız bir nesne üretmemesi gerekir. Bu ihtiyaç, bölümün ikinci kısmındaki soyut sınıflara götürecek.

İki tip: referans tipi ve nesne tipi

`Sekil s = new Daire(1);` satırında iki ayrı tip vardır. Bunların hangi kararı verdiğini Bölüm 31'de statik ve dinamik tip olarak görmüştük; C#'ta durum aynıdır:

Kavram	Nerede yazılır	Neyi belirler
Referans tipi	Değişkenin bildiriminde (`Sekil`)	Hangi üyelerin çağrılabileceğini (derleme zamanı)
Nesne tipi	`new` 'den sonra (`Daire`)	Sanal bir metodun hangi sürümünün çalışacağını (çalışma zamanı)

Derleyici yalnızca referans tipine bakar. Nesne gerçekte `Kedi` olsa bile, değişkenin tipi `Hayvan` ise yalnızca `Hayvan` 'da bulunan üyeleri çağırabiliriz:

▸ eksikUye.cs

```
Hayvan h = new Kedi();
h.SesCikar();
h.Tirmala();

class Hayvan { public virtual void SesCikar() { } }
class Kedi : Hayvan { public void Tirmala() { } }
```

DERLEYİCİ ÇIKTISI

```
eksikUye.cs(3,3): error CS1061: 'Hayvan' does not contain a definition for 'Tirmala' and no
accessible extension method 'Tirmala' accepting a first argument of type 'Hayvan' could be found
(are you missing a using directive or an assembly reference?)
```

Derleyici `h` 'nin içinde gerçekte ne olduğunu bilemez; nesne bir metoda parametre olarak gelmiş olabilir. Çözüm, Bölüm 50'de gördüğümüz aşağı dönüşümdür: `if (h is Kedi k) { k.Tirmala(); }` .

C++'ta aynı sınırlama, taban sınıf işaretçisinden türetilmiş sınıfın üyesine erişememek biçiminde karşımıza çıkıyordu (Bölüm 31). C#'ta bir fark vardır: `virtual` olmayan bir metodu türetilmiş sınıfta aynı adla yazmak, `new` yazılmazsa hata değil, uyarı üretir (Bölüm 50). `override` ise taban sınıfta karşılığı olan bir `virtual` ya da `abstract` üyeyi şart koşar; yoksa derlenmez. C++'taki "yanlışlıkla gizleme" tuzağı burada büyük ölçüde kapatılmıştır.

Üst sınıfın içinden yapılan sanal çağrı

Dinamik bağlama, sanal metot taban sınıfın kendi içinden çağrıldığında da geçerlidir. Taban sınıftaki bir çağrı aslında `this.Metot()` demektir ve `this` o anda türetilmiş sınıfın nesnesidir. Bu, çok kullanılan bir tasarım kalıbının temelidir: taban sınıf algoritmanın iskeletini yazar, değişen adımları sanal metotlara bırakır.

▸ bordro.cs

```
Personel[] ekip = { new Personel("Ali"), new Mudur("Ayşe"), new Stajyer("Can") };
foreach (Personel p in ekip)
{
    p.Bordro();
}

class Personel
{
    protected string ad;
    protected double taban = 30000;

    public Personel(string ad) { this.ad = ad; }
```

```
    public virtual double Maas() => taban;
    public virtual string Unvan() => "Çalışan";

    // Yalnızca burada yazılıdır; alt sınıflar ezmez.
    public void Bordro()
    {
        Console.WriteLine($"{ad,-5} {Unvan(),-8} {Maas(),9:F2}");
    }
}

class Mudur : Personel
{
    public Mudur(string ad) : base(ad) { }
    public override double Maas() => base.Maas() * 2;
    public override string Unvan() => "Müdür";
}

class Stajyer : Personel
{
    public Stajyer(string ad) : base(ad) { }
    public override double Maas() => taban / 3;
}
```

```
ÇIKTI
Ali   Çalışan   30000.00
Ayşe  Müdür     60000.00
Can   Çalışan   10000.00
```

`Bordro()` metodu yalnızca `Personel` sınıfında yazılmıştır; yine de `Ayşe` için çağrıldığında `Mudur`'un `Unvan` ve `Maas` sürümleri çalışır. `Stajyer` yalnızca `Maas`'ı ezdiği için unvanını `Personel`'den alır. `base.Maas()` ise dinamik bağlamayı bilerek atlar ve doğrudan taban sınıfın sürümünü çağırır; bu yüzden `Mudur.Maas()` kendini sonsuza kadar çağırmaz.

> ◆ **Kritik Nokta**
>
> Bir çağrı hakkında iki ayrı soru sorarız. "Derlenir mi?" sorusunun cevabı referans tipine bakılarak verilir. "Hangi sürüm çalışır?" sorusunun cevabı, metot `virtual`, `abstract` ya da `override` ise nesne tipine bakılarak verilir. Metot sanal değilse bu soruyu da referans tipi cevaplar.

51.2 Sanal Çağrıların Tuzakları

Yapıcıdan sanal metot çağırmak: C#, C++'tan farklıdır

Bölüm 31'de, C++'ta bir yapıcının içinden çağrılan sanal fonksiyonun dinamik bağlanmadığını görmüştük: taban sınıfın yapıcısı çalışırken nesne henüz yalnızca "taban sınıf" sayıldığı için, çağrı taban sınıfın sürümüne gider. C# bu konuda farklı davranır: nesnenin tipi, ilk bellek ayrıldığı andan itibaren gerçek tipidir. Taban yapıcıdan çağrılan sanal metot, türetilmiş sınıfın `override` sürümüne ulaşır. Üstelik türetilmiş sınıfın yapıcı gövdesi henüz çalışmamıştır:

```
▸ yapicidaSanal.cs
```

```
new Turetilmis();

class Taban
{
```

```
    public Taban()
    {
        Console.WriteLine("Taban yapıcısı");
        Yaz();
    }

    public virtual void Yaz() => Console.WriteLine("Taban.Yaz");
}

class Turetilmis : Taban
{
    private string ad = "Elif";
    private int katsayi;

    public Turetilmis()
    {
        katsayi = 10;
        Console.WriteLine("Turetilmis yapıcısı");
    }

    public override void Yaz() =>
        Console.WriteLine($"Turetilmis.Yaz: ad={ad} katsayi={katsayi}");
}
```

```
ÇIKTI
Taban yapıcısı
Turetilmis.Yaz: ad=Elif katsayi=0
Turetilmis yapıcısı
```

Sıraya dikkat edelim. Önce türetilmiş sınıfın alan başlatıcıları (`ad = "Elif"`) çalıştı, sonra taban yapıcı, en sonda da türetilmiş yapıcının gövdesi. Taban yapıcıdan çağrılan `Yaz()` bu yüzden `ad`'ı görür, ama yapıcı gövdesinde atanan `katsayi`'yi henüz `0` olarak görür.

> **▲ Sık Yapılan Hata**
>
> Yapıcı içinde sanal metot çağırmaktan kaçınalım. C#'ta metot türetilmiş sınıfa ulaşır, ama o sınıfın nesnesi henüz tam kurulmamıştır; yapıcı gövdesinde atanan alanlar varsayılan değerindedir. Hata derleyici tarafından da yakalanmaz. C++'ta ise aynı çağrı sessizce taban sürümünü çalıştırır; iki dilde de tavsiye aynıdır.

Aşırı yükleme (overload) derleme zamanında seçilir

`override` ile aşırı yükleme (overload: aynı adlı, farklı parametreli metotlar) sık karıştırılır. İlki çalışma zamanında nesne tipine bakar; ikincisi derleme zamanında, **argümanın referans tipine** bakarak seçilir:

▸ overloadSecimi.cs

```
Hayvan h = new Kedi();
Kedi k = new Kedi();

Tanitici.Tanit(h);
Tanitici.Tanit(k);
Tanitici.Tanit((Kedi)h);

class Hayvan { }
class Kedi : Hayvan { }

class Tanitici
{
```

```
    public static void Tanit(Hayvan h) => Console.WriteLine("Bir hayvan");
    public static void Tanit(Kedi k) => Console.WriteLine("Bir kedi");
}
```

```
ÇIKTI
Bir hayvan
Bir kedi
Bir kedi
```

`h` bir kedi nesnesini tutsa da ilk çağrıda `Tanit(Hayvan)` seçildi; çünkü derleyici `h`'nin tipinin `Hayvan` olduğunu görür. Üçüncü satırda `(Kedi)h` cast'i nesneyi değiştirmez, yalnızca ifadenin tipini değiştirir. Aşırı yükleme kuralları Bölüm 46'da, C++'taki karşılığı Bölüm 25'te işlenmişti.

Aşağıdaki tablo, hangi kararın ne zaman verildiğini özetler:

Karar	Ne zaman verilir	Neye bakılır
`virtual` / `abstract` metot, `override`	Çalışma zamanı	Nesne tipi
Aşırı yüklenmiş metotlardan birini seçmek	Derleme zamanı	Argümanın referans tipi
`new` ile gizlenmiş metot	Derleme zamanı	Referans tipi
`virtual` olmayan metot	Derleme zamanı	Referans tipi
Alanlar ve `static` üyeler	Derleme zamanı	Referans tipi

Nesne dilimleme yoktur

Bölüm 31'de C++'ın en sinsi tuzaklarından birini görmüştük: türetilmiş bir nesneyi taban tipli bir **değişkene değerle** atadığımızda türetilmiş kısım kesilip atılıyordu. C#'ta sınıf tipli değişkenler nesnenin kendisini değil, ona giden bir referansı tutar (Bölüm 49). `Sekil s = new Daire(1);` satırı bir kopya üretmez; aynı nesneye giden bir referansı daha geniş bir tiple saklar. Sınıflar arasında değer ataması olmadığı için dilimleme de olamaz. Değer tipleri (`struct`) ise zaten kalıtım desteklemez; onları Bölüm 52'de göreceğiz.

> **Not · Tür denetimi zincirleri**
>
> Kodumuzda `if (x is A) ... else if (x is B) ... else if (x is C) ...` biçiminde uzayan bir zincir görüyorsak, çok biçimliliği kullanmıyoruz demektir. Ortak davranışı taban sınıfa (ya da arayüze) bir metot olarak koyup her alt sınıfa kendi sürümünü yazdırırsak zincir, tek bir `x.Metot()` çağrısına dönüşür ve yeni tür eklemek zincire dokunmayı gerektirmez. `is` ve `switch` desenleri (Bölüm 44), kaynak kodunu değiştiremediğimiz tipler ya da yalnızca bir alt sınıfa özgü işlemler için uygundur.

51.3 Soyut Sınıflar

Bu bölümün başındaki `Sekil` sınıfı iki sorun taşıyordu: `Alan()` metodu anlamsız bir `0` döndürüyordu ve `new Sekil()` yazan biri anlamsız bir nesne elde ediyordu. İkisini birden çözen araç, soyut sınıftır.

> **Tanım: Soyut sınıf (abstract class)**
>
> Kendisinden doğrudan nesne üretilemeyen, yalnızca türetilmek için var olan sınıf. İçinde gövdesi olmayan **soyut üyeler** (abstract member) bulunabilir; her somut alt sınıf bu üyeleri yazmak zorundadır.

C++'taki karşılığı, en az bir saf sanal fonksiyon (`= 0`) içeren sınıftı (Bölüm 31). C#'ta bu niyet doğrudan sözcüklerle yazılır: sınıfa `abstract`, gövdesiz üyelere de `abstract` deriz.

soyutSekil.cs

```
Sekil[] sekiller = { new Daire(1.5), new Dikdortgen(2, 5), new Kare(3) };
foreach (Sekil s in sekiller)
{
    s.Bilgi();
}

abstract class Sekil
{
    public string Ad { get; }

    protected Sekil(string ad)
    {
        Ad = ad;
    }

    public abstract double Alan();          // gövde yok, sonunda ;
    public abstract double Cevre();

    public void Bilgi()                     // somut (gövdeli) metot
    {
        Console.WriteLine($"{Ad,-10} alan={Alan(),6:F2} çevre={Cevre(),6:F2}");
    }
}

class Daire : Sekil
{
    private readonly double yaricap;
    public Daire(double yaricap) : base("Daire") { this.yaricap = yaricap; }
    public override double Alan() => Math.PI * yaricap * yaricap;
    public override double Cevre() => 2 * Math.PI * yaricap;
}

class Dikdortgen : Sekil
{
    private readonly double en, boy;

    public Dikdortgen(double en, double boy) : this("Dikdörtgen", en, boy) { }

    protected Dikdortgen(string ad, double en, double boy) : base(ad)
    {
        this.en = en;
        this.boy = boy;
    }

    public override double Alan() => en * boy;
    public override double Cevre() => 2 * (en + boy);
}

class Kare : Dikdortgen
{
    public Kare(double kenar) : base("Kare", kenar, kenar) { }
}
```

ÇIKTI

```
Daire      alan=  7.07 çevre=  9.42
Dikdörtgen alan= 10.00 çevre= 14.00
Kare       alan=  9.00 çevre= 12.00
```

`Bilgi()` somut bir metottur: gövdesi vardır ve soyut `Alan()` ile `Cevre()`'yi çağırır. Hangi sürümlerin çalışacağı nesneye göre belirlenir. Taban sınıf algoritmanın iskeletini yazar, alt sınıflar boşlukları doldurur. Bu kalıbın adı **şablon yöntem**dir (template method).

Soyut sınıfın kuralları

1. Soyut üyenin gövdesi olmaz: `public abstract double Alan();`. Süslü parantez yerine noktalı virgül yazılır. C++'ta saf sanal bir fonksiyona yine de gövde yazılabilirdi; C#'ta bu mümkün değildir.
2. Soyut üye örtük olarak sanaldır; ayrıca `virtual` yazılmaz. Alt sınıf `override` ile yazar.
3. En az bir soyut üyesi olan sınıf `abstract` olmak zorundadır. Tersi gerekmez: hiç soyut üyesi olmayan bir sınıf da `abstract` yapılabilir; bu, "bu sınıf yalnızca türetilmek için" demektir.
4. Soyut sınıfın alanları, özellikleri, yapıcıları ve gövdeli metotları olabilir. Yapıcıyı, yalnızca alt sınıflardan çağrılacağı için genelde `protected` yazarız.
5. Somut (soyut olmayan) bir alt sınıf, taban sınıftaki **bütün** soyut üyeleri `override` ile yazmak zorundadır. Yazmayacaksa kendisi de `abstract` kalır.
6. Özellikler de soyut olabilir: `public abstract string Tur { get; }`. Alt sınıf `public override string Tur => "Daire";` diye yazar.
7. Soyut sınıf tipinde değişken ve dizi tanımlayabiliriz (`Sekil s`, `new Sekil[3]`); ama nesne üretemeyiz.

Son kuralın ihlali şöyle görünür:

▸ soyutNesne.cs

```
Sekil s = new Sekil();

abstract class Sekil
{
    public abstract double Alan();
}
```

```
DERLEYİCİ ÇIKTISI

soyutNesne.cs(1,11): error CS0144: Cannot create an instance of the abstract type or interface
'Sekil'
```

Beşinci kuralın ihlali de derleme hatasıdır. Alt sınıf soyut üyelerden birini bile yazmayı unutursa derleyici bunu hemen bildirir:

▸ eksikEzme.cs

```
Sekil s = new Kare();

abstract class Sekil
{
    public abstract double Alan();
    public abstract double Cevre();
}

class Kare : Sekil
{
    private readonly double kenar = 2;
    public override double Alan() => kenar * kenar;
}
```

```
DERLEYİCİ ÇIKTISI

eksikEzme.cs(9,7): error CS0534: 'Kare' does not implement inherited abstract member
'Sekil.Cevre()'
```

▲ Sık Yapılan Hata

Soyut üye, alt sınıfta ezilmek için vardır. Bu yüzden ezilmeyi engelleyen belirteçlerle bir arada yazılamaz: `abstract` üye `private` veya `static` olamaz, `abstract sealed` sınıf da yazılamaz. Mühürlü (`sealed`) sınıf türetilemez, soyut sınıf ise türetilmek zorundadır; iki istek birbirini yok eder.

İpucu

Taban sınıfta "makul bir varsayılan davranış" varsa `virtual`, "bu davranış tipe bağlı ve taban sınıfta anlamlı bir karşılığı yok" ise `abstract` tercih edilir. `Sekil.Alan()` için makul bir varsayılan yoktur; `Personel.Unvan()` için "Çalışan" makuldür.

51.4 Arayüzler (interface)

Soyut sınıf "ne olduğumuzu" anlatır: `Daire` bir şekildir. Bazen ise ortak olan şey kimlik değil, yetenektir. Bir ördek uçabilir, bir uçak da uçabilir; ama uçak bir hayvan değildir ve ikisini ortak bir taban sınıfa sokmak yapay olur. Ayrıca bir sınıfın yalnızca tek taban sınıfı olabilir (Bölüm 50); "uçabilir", "yüzebilir", "sıralanabilir" gibi birçok yeteneği aynı anda taşıması gerekebilir.

Tanım: Arayüz (interface)

Bir tipin hangi üyeleri (metot, özellik, olay, indeksleyici) sağlayacağını söyleyen sözleşme. Arayüz kendisi bir uygulama içermez (varsayılan arayüz metotları dışında; aşağıda göreceğiz). Bir sınıf, arayüzü uyguladığını bildirir ve bütün üyelerini yazmak zorundadır.

C++'ta arayüz için dil düzeyinde bir anahtar sözcük yoktu; yalnızca saf sanal fonksiyonlardan oluşan bir sınıf yazar ve bunu arayüz olarak kullanırdık (Bölüm 31). C#'ta `interface` ayrı bir kavramdır. Adı geleneksel olarak `I` harfiyle başlar (`IUcabilir`). Bir sınıf, arayüzü sınıf adından sonra `:` ile yazar. Önce (varsa) taban sınıf, sonra arayüzler gelir:

▸ arayuzOrnek.cs

```
IUcabilir[] ucanlar = { new Ordek("Donald"), new Ucak("TC-JAN") };
foreach (IUcabilir u in ucanlar)
{
    u.Uc();                          // nesnenin kendi sürümü çalışır
    if (u is IYuzebilir yz)          // arayüze de is ile sorulur
    {
        yz.Yuz();
    }
}

IYuzebilir y = new Ordek("Vakvak");  // ördek aynı zamanda IYuzebilir
y.Yuz();
Console.WriteLine(ucanlar[1] is IYuzebilir);
Console.WriteLine(ucanlar[0] is Hayvan);

interface IUcabilir { void Uc(); }   // üyeler otomatik olarak public
interface IYuzebilir { void Yuz(); }
```

```
class Hayvan
{
    protected string ad;
    public Hayvan(string ad) { this.ad = ad; }
}

class Ordek : Hayvan, IUcabilir, IYuzebilir      // önce sınıf, sonra arayüzler
{
    public Ordek(string ad) : base(ad) { }
    public void Uc() => Console.WriteLine(ad + " kanat çırparak uçuyor");
    public void Yuz() => Console.WriteLine(ad + " gölde yüzüyor");
}

class Ucak : IUcabilir                           // hayvan değil, ama uçabilir
{
    private readonly string kod;
    public Ucak(string kod) { this.kod = kod; }
    public void Uc() => Console.WriteLine(kod + " 10000 metrede uçuyor");
}
```

```
ÇIKTI
Donald kanat çırparak uçuyor
Donald gölde yüzüyor
TC-JAN 10000 metrede uçuyor
Vakvak gölde yüzüyor
False
True
```

Arayüz de bir tiptir. `IUcabilir` tipinde değişken ve dizi tanımlayabilir, içine onu uygulayan herhangi bir sınıfın nesnesini koyabiliriz. Çok biçimlilik burada da geçerlidir: `u.Uc()` çağrısında çalışan kod, nesnenin sınıfındaki uygulamadır. Ama arayüzden nesne üretemeyiz; `new IUcabilir()` soyut sınıfla aynı hatayı verir (`CS0144`).

Kuralları sıralayalım:

1. Arayüz üyeleri örtük olarak `public`'tir; arayüz içinde erişim belirleyici yazmayız.
2. Sınıfta uygularken `override` yazılmaz; metodu `public` olarak doğrudan yazarız.
3. Arayüzde nesne alanı (`private int x;` gibi) ve yapıcı bulunamaz. Veri taşımak tek taban sınıfın işidir.
4. Arayüz tipli bir değişkenden yalnızca arayüzde tanımlı üyelere erişiriz. `u.Yuz()` yazamayız; önce `is` ile sormamız gerekir.
5. Bir sınıf sınırsız sayıda arayüz uygulayabilir, ama yalnızca tek bir sınıftan türeyebilir.

İkinci kuralın en sık ihlali, uygulayan metodu `public` yazmayı unutmaktır. Sınıf üyelerinin varsayılan erişimi `private` olduğu için metot arayüzü karşılamaz:

▸ zayifErisim.cs

```
IUcabilir k = new Kus();
k.Uc();

interface IUcabilir { void Uc(); }

class Kus : IUcabilir
{
    void Uc() => Console.WriteLine("Kuş uçuyor");
}
```

```
DERLEYİCİ ÇIKTISI

zayifErisim.cs(6,13): error CS0737: 'Kus' does not implement interface member 'IUcabilir.Uc()'.
'Kus.Uc()' cannot implement an interface member because it is not public.
```

Beşinci kuralın ikinci yarısı da bir hata kaynağıdır. Bölüm 50'de gördüğümüz gibi sınıflar için çoklu kalıtım yoktur:

```
▸ cokluSinif.cs

class A { }
class B { }
class C : A, B { }
```

```
DERLEYİCİ ÇIKTISI

cokluSinif.cs(3,14): error CS1721: Class 'C' cannot have multiple base classes: 'A' and 'B'
```

Çözüm `class C : A, IX` biçimidir: önce tek sınıf, sonra istediğimiz kadar arayüz. Arayüzü sınıftan önce yazarsak (`class D : IX, A`) `CS1722` hatası alırız.

Özellik içeren arayüzler

Arayüz yalnızca metot değil, özellik de isteyebilir. Arayüzdeki `string Baslik { get; }` bir alan değildir, bir sözleşmedir: "bu sınıf okunabilir bir `Baslik` özelliği sağlayacak". Özelliğin nasıl saklanacağı uygulayan sınıfa kalır:

```
▸ arayuzOzellik.cs

IYayin[] raf = { new Kitap("Algoritmalar", 320), new Dergi(48) };
int toplamDakika = 0;
foreach (IYayin y in raf)
{
    Console.WriteLine($"{y.Baslik}: {y.OkumaSuresi()} dakika");
    toplamDakika += y.OkumaSuresi();
}
Console.WriteLine($"Toplam: {toplamDakika} dakika");

interface IYayin
{
    string Baslik { get; }              // özellik sözleşmesi
    int OkumaSuresi();
}

class Kitap : IYayin
{
    private readonly int sayfa;
    public string Baslik { get; }       // otomatik özellik
    public Kitap(string baslik, int sayfa) { Baslik = baslik; this.sayfa = sayfa; }
    public int OkumaSuresi() => sayfa * 2;
}

class Dergi : IYayin
{
    private readonly int sayfa;
    public Dergi(int sayfa) { this.sayfa = sayfa; }
    public string Baslik => "Aylık dergi"; // ifade gövdeli özellik
    public int OkumaSuresi() => sayfa;
}
```

```
ÇIKTI
Algoritmalar: 640 dakika
Aylık dergi: 48 dakika
Toplam: 688 dakika
```

`Kitap` özelliği otomatik özellikle, `Dergi` ifade gövdeli özellikle sağladı (Bölüm 49). Arayüz ikisini de kabul eder; çünkü sözleşme yalnızca "okunabilir bir `Baslik` var" der.

Arayüzlerin arayüzlerden türemesi

Bir arayüz, bir ya da birden çok arayüzden türeyebilir. Bu, sınıflardaki tekli kalıtımın aksine arayüzlerde serbesttir, çünkü arayüzler durum taşımaz ve elmas problemi doğmaz. Türeyen arayüzü uygulayan sınıf, bütün üst arayüzlerin üyelerini de yazmak zorundadır:

▸ arayuzKalitim.cs

```
IDosya d = new BellekDosyasi();
d.Yaz("merhaba");
Console.WriteLine(d.Oku());

IOkur okur = d;                          // her IDosya aynı zamanda IOkur
Console.WriteLine(okur is IYazar);

interface IOkur { string Oku(); }
interface IYazar { void Yaz(string metin); }
interface IDosya : IOkur, IYazar { }     // iki arayüzü birleştirir

class BellekDosyasi : IDosya
{
    private string icerik = "";
    public string Oku() => icerik;
    public void Yaz(string metin) => icerik += metin;
}
```

```
ÇIKTI
merhaba
True
```

Açık arayüz uygulaması

İki farklı arayüz aynı imzalı bir üye isterse ne olur? Tek bir `Hareket()` metodu ikisini birden karşılar; ama iki arayüz için farklı davranış istiyorsak bunu ayrı ayrı yazmamız gerekir. Çözüm, üye adının önüne arayüz adını ekleyen **açık arayüz uygulaması** (explicit interface implementation) yazımıdır:

▸ acikUygulama.cs

```
Ordek o = new Ordek();
IKus k = o;
IBalik b = o;
k.Hareket();
b.Hareket();
((IKus)o).Hareket();

interface IKus { void Hareket(); }
interface IBalik { void Hareket(); }

class Ordek : IKus, IBalik
{
```

```
    void IKus.Hareket() => Console.WriteLine("uçuyor");     // erişim belirleyici yok
    void IBalik.Hareket() => Console.WriteLine("yüzüyor");
}
```

```
ÇIKTI
uçuyor
yüzüyor
uçuyor
```

Açık uygulanan üyeler `Ordek` tipinin kendi arayüzünde görünmez; yalnızca arayüz tipli bir ifade üzerinden çağrılabilir. `o.Hareket()` yazarsak derleyici `Ordek`'te böyle bir üye bulamaz (`CS1061`). Bu teknik ayrıca, bir üyeyi sınıfın normal kullanımından gizleyip yalnızca arayüz üzerinden sunmak istediğimizde de kullanılır.

Varsayılan arayüz metotları (C# 8)

> *Tarihçe · Arayüzlerde gövde*
>
> C# 8 (2019) ile arayüz üyelerine gövde yazmak mümkün oldu. Amaç, yayımlanmış bir arayüze yeni bir üye eklemek istediğimizde, onu uygulayan bütün mevcut sınıfların bozulmasını önlemekti. C++'ta saf sanal fonksiyonlara gövde yazılabiliyordu, ama bu başka bir amaca hizmet ediyordu; C# eşdeğeri `abstract` üyelerde yoktur.

Uygulayan sınıf metodu yazmazsa arayüzdeki gövde kullanılır:

varsayilanMetot.cs

```
ISarjli t = new Telefon(15);
Console.WriteLine(t.Durum());
t = new Telefon(80);
Console.WriteLine(t.Durum());

interface ISarjli
{
    int Seviye { get; }
    string Durum() => Seviye < 20 ? "şarj et" : "iyi";   // varsayılan gövde
}

class Telefon : ISarjli
{
    public int Seviye { get; }
    public Telefon(int seviye) { Seviye = seviye; }
}
```

```
ÇIKTI
şarj et
iyi
```

Varsayılan metot sınıfın üyesi hâline gelmez; yalnızca arayüz tipli bir ifadeyle çağrılabilir. `Telefon t = new Telefon(15); t.Durum();` yazarsak derleme hatası (`CS1061`) alırız. Bu özelliği arayüz evrimi için bir kolaylık olarak görmek, soyut sınıfın yerine geçecek bir mekanizma saymamak gerekir: durum (alan) taşıyamaz.

Arayüz çağrısı ve `new` tuzağı

Arayüz üyesinin hangi metoda bağlanacağını, arayüzü uygulayan sınıf belirler. Alt sınıf aynı adlı bir metodu `new` ile yazarsa, arayüz bundan haberdar olmaz:

arayuzGizleme.cs

```
IMesaj m = new Alt();
Ust u = new Alt();
Alt a = new Alt();
Console.WriteLine(m.Yaz() + " " + u.Yaz() + " " + a.Yaz());

interface IMesaj { string Yaz(); }
class Ust : IMesaj { public string Yaz() => "Ust"; }
class Alt : Ust { public new string Yaz() => "Alt"; }
```

ÇIKTI

```
Ust Ust Alt
```

`IMesaj`'ı `Ust` uyguladı; `Alt` arayüzü yeniden listelemedi ve `new` ile yazdığı `Yaz()` ayrı bir üyedir. Bu yüzden arayüz referansı da `Ust` referansı da `Ust`'ın metoduna gider; yalnızca `Alt` referansı `Alt`'ınkini görür. Üçünde de `Alt` yazmasını istiyorsak `Ust.Yaz()`'ı `virtual`, `Alt.Yaz()`'ı `override` yapmalıyız. Bir başka çözüm, arayüzü alt sınıfın başlığında yeniden listelemektir (`class Alt : Ust, IMesaj`); bu durumda arayüz eşlemesi yenilenir ve `m.Yaz()` artık `Alt`'ın metodunu bulur. Ama `u.Yaz()` hâlâ `Ust`'ın metodunda kalır, çünkü `new` ile gizleme referans tipine göre çalışmaya devam eder.

51.5 Soyut Sınıf mı, Arayüz mü?

İki kavram da nesnesi üretilemeyen bir tip tanımlar ve çok biçimliliği mümkün kılar. Aralarındaki farkları tablo hâlinde görelim:

Özellik	Soyut sınıf	Arayüz
Tanım ve kullanım	`abstract class A` / `class B : A`	`interface IA` / `class B : IA`
Kaç tane uygulanır?	Bir sınıf yalnızca bir sınıftan türer	Bir sınıf istediği kadar arayüz uygular
Nesne üretilebilir mi?	Hayır	Hayır
Yapıcı	Olabilir	Olamaz
Nesne alanı (durum)	Olabilir	Olamaz; özellik sözleşmesi olabilir
Üyeler	Soyut ve gövdeli, her erişim düzeyi	Soyut; gövdeli üye ikincil bir özelliktir (C# 8)
Üyelerin erişimi	Biz belirleriz	Varsayılan `public`
Alt sınıfta yazım	Soyut üye `override` ile yazılır	`override` yazılmaz
İlişki	"bir" (is-a): `Daire` bir şekildir	"yapabilir" (can-do): `Ordek` uçabilir
Ne zaman?	Ortak durum ve kod paylaşan yakın akraba sınıflar	İlgisiz sınıflara ortak bir yetenek kazandırmak

Pratik ölçü şudur: ortak **durum ve kod** paylaşılacaksa soyut sınıf, yalnızca ortak **sözleşme** gerekiyorsa arayüz kullanırız. İkisini birleştirmek de yaygındır: bir arayüz sözleşmeyi tanımlar, bir soyut sınıf bu arayüzün yaygın kısmını uygular, somut sınıflar yalnızca farklı kısımları yazar. .NET kütüphanesi bu kalıpla doludur; Bölüm 51'in sonundaki ödeme sistemi örneğinde de aynısını yapacağız.

> **Üç Dilde · Arayüz: C, C++ ve C#** *C · C++ · C#*
>
> **C**'de arayüz diye bir kavram yoktur. Aynı fikir, fonksiyon işaretçilerinden oluşan bir `struct` ile elle kurulur; Bölüm 31'de gördüğümüz sanal tabloyu programcı kendisi yazar:

```
struct Ucabilir {
    void (*uc)(void *kendisi);
};
```

C++'ta arayüz, yalnızca saf sanal fonksiyonlardan oluşan soyut sınıftır. Bir sınıf çoklu kalıtımla birden çok arayüzü uygular; tüm tabanlar `public` yazılır, sanal yıkıcı unutulmaz:

```
class IUcabilir {
public:
    virtual ~IUcabilir() = default;
    virtual void uc() = 0;
};
class Ordek : public Hayvan, public IUcabilir {
public:
    void uc() override { /* ... */ }
};
```

C#'ta ise `interface` dilin kendi kavramıdır. Sanal yıkıcı sorunu yoktur (çöp toplayıcı vardır), üyeler örtük `public`'tir, `override` yazılmaz ve çoklu kalıtım yalnızca arayüzler için serbesttir:

```
interface IUcabilir { void Uc(); }
class Ordek : Hayvan, IUcabilir
{
    public void Uc() { /* ... */ }
}
```

C++'ta "birden çok sınıftan türemek" ile "birden çok arayüz uygulamak" aynı sözdizimiyle yazılır; C# bu iki kavramı ayırarak yalnızca güvenli olanı serbest bırakmıştır.

51.6 .NET'in Hazır Arayüzleri

.NET kütüphanesi arayüzlerle doludur. Bir sınıfımız doğru arayüzü uygularsa, hazır kütüphane kodları (sıralama, `foreach`, koleksiyonlar, LINQ) bizim tipimizle de çalışır. Bu bölümde üç tanesini tanıyacağız. Hepsi, ayrıntısını Bölüm 54'te işleyeceğimiz **genel tip** (generic) söz dizimini kullanır: `IComparable<T>` yazımındaki `<T>`, arayüzün hangi tiple çalıştığını belirtir. Şimdilik `IComparable<Urun>`'ü "Urun nesneleriyle karşılaştırılabilir" diye okumak yeterlidir.

Arayüz	Sözleşme	Kullanıldığı yer
`IComparable<T>`	"Ben, bir `T` ile karşılaştırılabilirim" (`CompareTo`)	`Array.Sort`, sıralı koleksiyonlar
`IComparer<T>`	"İki `T` nesnesini karşılaştırabilirim" (`Compare`)	Farklı ölçütlerle sıralama
`IEnumerable<T>`	"Elemanlarımı sırayla verebilirim" (`GetEnumerator`)	`foreach`, LINQ
`IEquatable<T>`	"Bir `T` ile eşitlik karşılaştırması yapabilirim"	Sözlükler, `Contains`
`IDisposable`	"Serbest bırakılacak kaynak tutuyorum" (`Dispose`)	`using` (Bölüm 53)

`IComparable<T>`: kendi nesnelerimizi sıralamak

Bölüm 47'de `Array.Sort` ile sayı ve metin dizilerini sıralamıştık. Kendi sınıfımızın nesnelerini sıralamak için `Array.Sort`'un "hangisi önce gelir?" sorusunu nasıl cevaplayacağını öğretmemiz gerekir. Bunun yolu `IComparable<T>` arayüzünü uygulamaktır. Tek üyesi `CompareTo`'dur.

Tanım: CompareTo sözleşmesi

`a.CompareTo(b)` çağrısı, `a` sıralamada `b`'den önce gelmeliyse **negatif**, eşitse **sıfır**, sonra gelmeliyse **pozitif** bir tamsayı döndürür. Değerin büyüklüğünün önemi yoktur, yalnızca işareti anlamlıdır.

comparableOrnek.cs

```
Urun[] urunler =
{
    new Urun("Kalem", 12), new Urun("Çanta", 310),
    new Urun("Silgi", 8.25), new Urun("Defter", 45.5)
};

Array.Sort(urunler);                      // CompareTo kullanılır
Console.WriteLine(string.Join<Urun>(", ", urunler));

Console.WriteLine(urunler[0].CompareTo(urunler[3]));
Console.WriteLine(urunler[3].CompareTo(urunler[0]));
Console.WriteLine(urunler[1].CompareTo(urunler[1]));

class Urun : IComparable<Urun>
{
    public string Ad { get; }
    public double Fiyat { get; }

    public Urun(string ad, double fiyat)
    {
        Ad = ad;
        Fiyat = fiyat;
    }

    public int CompareTo(Urun? diger)
    {
        if (diger is null)
        {
            return 1;                     // null her nesneden "küçük" sayılır
        }
        return Fiyat.CompareTo(diger.Fiyat);   // double'ın CompareTo'suna havale
    }

    public override string ToString() => $"{Ad}({Fiyat})";
}
```

```
ÇIKTI
Silgi(8.25), Kalem(12), Defter(45.5), Çanta(310)
-1
1
0
```

İşi `double`'ın kendi `CompareTo` metoduna havale ettik: fiyatı küçük olan önce gelir. Büyükten küçüğe sıralamak için `diger.Fiyat.CompareTo(Fiyat)` yazmak yeterlidir. `string.Join<Urun>` yazımı, `Urun` dizisi için genel `Join` sürümünü açıkça seçer; yazmasak derleyici iki aday arasında kalır (`CS0121`). `Urun?` yazımı, parametrenin `null` olabileceğini söyleyen işarettir (Bölüm 59); arayüzün imzası böyle olduğu için uyarı almamak adına aynen yazarız. `string` ve `int` gibi tipler de bu arayüzü uyguladığı için `Array.Sort` onlarla hazır çalışır.

> **▲ Sık Yapılan Hata**
>
> `IComparable<T>` uygulamayan bir sınıfın dizisini `Array.Sort` ile sıralamaya çalışırsak program derlenir, ama çalışırken `InvalidOperationException` fırlatır (istisnalar Bölüm 53'te). Derleyici hatayı yakalamaz; çünkü `Array.Sort` genel `Array` parametresi alır.

`IComparer<T>`: farklı ölçütlerle sıralamak

`CompareTo` tek bir "doğal" sıralama tanımlar. Aynı sınıfı bir an fiyata, bir an ada göre sıralamak istersek ya da sınıfın kaynak koduna dokunamıyorsak, karşılaştırmayı **ayrı bir sınıfa** koyarız. Bu sınıf `IComparer<T>` arayüzünü uygular:

`comparerOrnek.cs`

```
Urun[] urunler =
{
    new Urun("Kalem", 12), new Urun("Çanta", 310),
    new Urun("Silgi", 8.25), new Urun("Defter", 45.5)
};

Array.Sort(urunler, new AdaGore());
Console.WriteLine(string.Join<Urun>(", ", urunler));

Array.Sort(urunler, new FiyataGoreAzalan());
Console.WriteLine(string.Join<Urun>(", ", urunler));

class Urun
{
    public string Ad { get; }
    public double Fiyat { get; }
    public Urun(string ad, double fiyat) { Ad = ad; Fiyat = fiyat; }
    public override string ToString() => $"{Ad}({Fiyat})";
}

class AdaGore : IComparer<Urun>
{
    public int Compare(Urun? x, Urun? y)
        => string.Compare(x?.Ad, y?.Ad, StringComparison.Ordinal);
}

class FiyataGoreAzalan : IComparer<Urun>
{
    public int Compare(Urun? x, Urun? y)
        => (y?.Fiyat ?? 0).CompareTo(x?.Fiyat ?? 0);
}
```

```
ÇIKTI
Defter(45.5), Kalem(12), Silgi(8.25), Çanta(310)
Çanta(310), Defter(45.5), Kalem(12), Silgi(8.25)
```

`StringComparison.Ordinal`, harfleri Unicode numaralarına göre karşılaştırır; `Ç` harfinin numarası `Z`'den büyük olduğu için `Çanta` ilk sıralamada sona düşer. Türkçe alfabe sırası gerekiyorsa kültüre duyarlı karşılaştırma kullanılır (Bölüm 48).

> **Üç Dilde · Karşılaştırma ölçütünü vermek** *C · C++ · C#*
>
> Üç dilde de sıralama algoritması "iki elemanı nasıl karşılaştıracağımı sen söyle" der; yalnızca bu bilginin verilme biçimi değişir.

	Karşılaştırmayı veren	Örnek
C	Fonksiyon işaretçisi	`qsort(dizi, n, sizeof(Urun), karsilastir);`
C++	`operator<` , lambda ya da karşılaştırma nesnesi	`std::sort(v.begin(), v.end(), [](auto& a, auto& b) { return a.fiyat < b.fiyat; });`
C#	`IComparable<T>` , `IComparer<T>` ya da temsilci (delegate)	`Array.Sort(dizi, new FiyataGore());`

C'nin `qsort` işlevi `void *` alır, bu yüzden tip güvenliği yoktur (Bölüm 12). C++'ın `std::sort` algoritması (Bölüm 36) şablondur ve lambda ya da `operator<` kabul eder. C# arayüz yolunu tercih eder; lambda ile aynı işi Bölüm 55'te `Comparison<T>` temsilcisi ve `Array.Sort(dizi, (a, b) => ...)` ile yapacağız.

`IEnumerable<T>`: foreach ile gezilebilen tipler

`foreach` her tip üzerinde çalışmaz; yalnızca "gezilebilir" olan tiplerde çalışır. Dizi, `string` ve koleksiyonlar bu şartı sağlar. C++'ta bu şart, tipin `begin()` ve `end()` üyelerinin olmasıydı (Bölüm 36). C#'ta şart bir arayüzdür: `IEnumerable<T>`. Tek üyesi, bir **sayıcı** (enumerator) döndüren `GetEnumerator()` metodudur. `foreach` satırı, derleyici tarafından sayıcının `MoveNext()` ve `Current` üyelerini kullanan bir döngüye çevrilir.

Sayıcıyı elle yazmak uzundur. Bu yüzden C#, `yield return` anahtar sözcüğüyle bir kısayol sunar: metot, elemanları teker teker "geri verir" ve derleyici sayıcıyı kendisi üretir. Ayrıntıyı başka bir bölüme bırakıp yalnızca kalıbı kullanalım:

gezilebilirTip.cs

```
using System.Collections;

HaftaGunleri hafta = new HaftaGunleri();

foreach (string gun in hafta)
{
    Console.Write(gun + " ");
}
Console.WriteLine();

Console.WriteLine(string.Join("-", hafta));   // IEnumerable<string> bekler
Console.WriteLine(hafta.Count());             // LINQ (Bölüm 56) bizim tipimizle çalışır
Console.WriteLine(hafta.Contains("Cum"));

class HaftaGunleri : IEnumerable<string>
{
    private readonly string[] gunler =
        { "Pzt", "Sal", "Çar", "Per", "Cum", "Cmt", "Paz" };

    public IEnumerator<string> GetEnumerator()
    {
        foreach (string g in gunler)
        {
            yield return g;
        }
    }

    // Eski, genel olmayan arayüz de istenir; açık uygulamayla köprü kurarız.
    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}
```

```
ÇIKTI
Pzt Sal Çar Per Cum Cmt Paz
Pzt-Sal-Çar-Per-Cum-Cmt-Paz
7
True
```

Üç şeye dikkat edelim. Birincisi, `IEnumerable<T>` arayüzü, genel olmayan `IEnumerable` arayüzünden türediği için ikisini de uygulamak gerekir; eski olanı açık arayüz uygulamasıyla yazıp genel olana yönlendirdik. İkincisi, tek bir arayüzü uygulamak, `foreach`, `string.Join` ve LINQ'nun yüzlerce işlemini bedavaya getirdi: `Count()` ve `Contains` bizim yazmadığımız metotlardır. Üçüncüsü, bu bir çok biçimlilik örneğidir: `string.Join` metodu `HaftaGunleri` sınıfını hiç tanımaz, yalnızca `IEnumerable<string>` sözleşmesine güvenir.

Bölüm 54'te `List<T>` ve `Dictionary<TKey,TValue>` gibi koleksiyonların da bu arayüzü uyguladığını göreceğiz.

51.7 Bağımlılığı Arayüzle Tersine Çevirmek

Arayüzlerin en önemli kullanımı, büyük programların parçalarını birbirinden ayırmaktır. Bu fikir, nesneye yönelik tasarımın beş ilkesinin baş harflerinden oluşan **SOLID** kısaltmasının son harfinde saklıdır. Beş ilkeyi ayrıntılı işlemeyeceğiz; yalnızca ne dediklerine bakalım:

Harf	İlke	Özü
S	Tek sorumluluk (Single responsibility)	Bir sınıfın değişmesi için tek bir nedeni olmalı
O	Açık/kapalı (Open/closed)	Yeni davranış, mevcut kodu değiştirmeden yeni sınıf yazarak eklenmeli
L	Liskov yerine geçme (Liskov substitution)	Türetilmiş sınıf, taban sınıfın beklendiği her yerde onun yerine geçebilmeli
I	Arayüz ayırma (Interface segregation)	Küçük, amaca yönelik arayüzler büyük, her işi yapan arayüzden iyidir
D	Bağımlılığı tersine çevirme (Dependency inversion)	Üst düzey kod, somut sınıflara değil arayüzlere bağımlı olmalı

Bu bölümde gördüğümüz `foreach` döngüsü zaten "O" ilkesinin küçük bir örneğiydi: `Ucgen` eklemek için döngüyü değiştirmemiz gerekmedi. Son harfi bir örnekle inceleyelim.

Bir sipariş sınıfı, sipariş alındığında müşteriye bildirim göndersin. İlk akla gelen çözüm, sınıfın içinde doğrudan e-posta gönderen sınıfı oluşturmaktır:

```
class SiparisServisi
{
    private readonly EpostaBildirimi bildirim = new EpostaBildirimi();   // somut sınıfa sıkı
bağlı

    public void SiparisVer(string musteri, int adet)
    {
        // ...
        bildirim.Gonder(musteri, "Siparişiniz alındı");
    }
}
```

Bu tasarımda `SiparisServisi`, e-postaya sonsuza dek bağımlıdır. SMS göndermek istesek sınıfın kendisini değiştirmemiz gerekir. Üstelik sınıfı denemek (test etmek) için gerçek e-posta göndermek zorunda kalırız. Çözüm, bağımlılığı bir arayüzle ifade etmek ve somut nesneyi dışarıdan, yapıcıyla vermektir:

bildirimArayuzu.cs

```
// Gerçek bildirimle çalışan servis
SiparisServisi servis = new SiparisServisi(new EpostaBildirimi());
servis.SiparisVer("Elif", 3);

// Aynı servis, farklı bir bildirimle: kod değişmedi
SiparisServisi sms = new SiparisServisi(new SmsBildirimi());
sms.SiparisVer("Mert", 1);

// Deneme için sahte bildirim: hiçbir şey gönderilmez, yalnızca kaydedilir
SahteBildirim sahte = new SahteBildirim();
new SiparisServisi(sahte).SiparisVer("Selin", 2);
Console.WriteLine($"Sahte kutuda {sahte.Sayac} mesaj var: {sahte.SonMesaj}");

interface IBildirim
{
    void Gonder(string alici, string mesaj);
}

class EpostaBildirimi : IBildirim
{
    public void Gonder(string alici, string mesaj)
        => Console.WriteLine($"E-posta -> {alici}: {mesaj}");
}

class SmsBildirimi : IBildirim
{
    public void Gonder(string alici, string mesaj)
        => Console.WriteLine($"SMS -> {alici}: {mesaj}");
}

class SahteBildirim : IBildirim
{
    public int Sayac { get; private set; }
    public string SonMesaj { get; private set; } = "";

    public void Gonder(string alici, string mesaj)
    {
        Sayac++;
        SonMesaj = $"{alici}: {mesaj}";
    }
}

class SiparisServisi
{
    private readonly IBildirim bildirim;       // arayüze bağımlı

    public SiparisServisi(IBildirim bildirim)  // bağımlılık dışarıdan verilir
    {
        this.bildirim = bildirim;
    }

    public void SiparisVer(string musteri, int adet)
    {
        Console.WriteLine($"{musteri} için {adet} ürün hazırlanıyor");
```

```
        bildirim.Gonder(musteri, "Siparişiniz alındı");
    }
}
```

```
ÇIKTI
Elif için 3 ürün hazırlanıyor
E-posta -> Elif: Siparişiniz alındı
Mert için 1 ürün hazırlanıyor
SMS -> Mert: Siparişiniz alındı
Selin için 2 ürün hazırlanıyor
Sahte kutuda 1 mesaj var: Selin: Siparişiniz alındı
```

Okların yönüne bakalım. İlk tasarımda üst düzey sınıf (`SiparisServisi`), alt düzey sınıfa (`EpostaBildirimi`) bağımlıydı. Şimdi ikisi de arayüze (`IBildirim`) bağımlıdır; "bağımlılık tersine çevrildi". `SiparisServisi`'nin hangi bildirim sınıfıyla çalışacağına, onu oluşturan kod karar verir. Bu tekniğe, somut nesneyi yapıcıdan geçirmeye **bağımlılık enjeksiyonu** (dependency injection) denir. Büyük .NET uygulamalarında bu iş için hazır çerçeveler kullanılır; ama temeli bu örnekteki kadar basittir.

> **◆ Kritik Nokta**
>
> Arayüz, çok biçimliliğin "tek kodla birçok davranış" gücünü sınıf hiyerarşisinden bağımsız kılar. `EpostaBildirimi` ile `SmsBildirimi` ortak bir taban sınıftan türemek zorunda değildir; yalnızca aynı sözleşmeyi taşırlar. Bu gevşek bağ, yeni bir tür eklemeyi (açık/kapalı ilkesi) ve gerçek nesnenin yerine sahtesini koyup denemeyi mümkün kılar.

51.8 Tam Örnek: Ödeme Sistemi

Şimdiye kadar öğrendiklerimizi tek bir tasarımda toplayalım. Bir mağaza, farklı ödeme yöntemlerini kabul ediyor. Tasarım şöyle:

- `IOdeme` arayüzü, kasanın bildiği tek sözleşmedir: yöntemin adı, komisyonu ve ödemeyi yapma işlemi.
- `OdemeYontemi` soyut sınıfı, ortak adımları (tutar kontrolü, sonucu yazdırma) yazar; yöntemlere özgü adımları soyut bırakır. Bu bir şablon yöntemdir.
- `KrediKarti`, `Havale` ve `Nakit` somut sınıflardır. Yalnızca ilk ikisi iade edilebilir; bu yetenek `IIadeEdilebilir` arayüzüyle ayrıca verilir.
- `Kasa` sınıfı yalnızca `IOdeme` bilir; hangi yöntemle çalıştığını bilmez.

▸ odemeSistemi.cs

```
IOdeme[] yontemler =
{
    new KrediKarti(3), new Havale(), new Nakit(), new KrediKarti(1)
};
decimal[] tutarlar = { 1200m, 800m, 7500m, 450m };

Kasa kasa = new Kasa();
for (int i = 0; i < yontemler.Length; i++)
{
    kasa.Tahsil(yontemler[i], tutarlar[i]);
}
Console.WriteLine($"Toplam komisyon: {kasa.ToplamKomisyon:F2} TL");

Console.WriteLine("--- İadeler ---");
foreach (IOdeme y in yontemler)
```

```
{
    if (y is IIadeEdilebilir iade)           // yalnızca yeteneği olanlar
    {
        iade.IadeEt(100m);
    }
}

interface IOdeme
{
    string Ad { get; }
    decimal Komisyon(decimal tutar);
    bool Ode(decimal tutar);
}

interface IIadeEdilebilir
{
    void IadeEt(decimal tutar);
}

abstract class OdemeYontemi : IOdeme
{
    public abstract string Ad { get; }
    public abstract decimal Komisyon(decimal tutar);
    protected abstract bool LimitUygun(decimal tutar);

    // Şablon yöntem: iskelet burada, ayrıntılar alt sınıflarda
    public bool Ode(decimal tutar)
    {
        if (tutar <= 0 || !LimitUygun(tutar))
        {
            Console.WriteLine($"{Ad,-11} {tutar,8:F2} TL reddedildi");
            return false;
        }
        Console.WriteLine(
            $"{Ad,-11} {tutar,8:F2} TL ödendi, komisyon {Komisyon(tutar):F2}");
        return true;
    }
}

class KrediKarti : OdemeYontemi, IIadeEdilebilir
{
    private readonly int taksit;
    public KrediKarti(int taksit) { this.taksit = taksit; }

    public override string Ad => "Kredi kartı";
    public override decimal Komisyon(decimal tutar)
        => tutar * (0.015m + 0.005m * taksit);
    protected override bool LimitUygun(decimal tutar) => tutar <= 10000m;
    public void IadeEt(decimal tutar)
        => Console.WriteLine($"{Ad}: {tutar:F2} TL karta iade");
}

class Havale : OdemeYontemi, IIadeEdilebilir
{
    public override string Ad => "Havale";
    public override decimal Komisyon(decimal tutar) => 5m;
    protected override bool LimitUygun(decimal tutar) => true;
    public void IadeEt(decimal tutar)
        => Console.WriteLine($"{Ad}: {tutar:F2} TL hesaba iade");
}

class Nakit : OdemeYontemi
{
    public override string Ad => "Nakit";
```

```
    public override decimal Komisyon(decimal tutar) => 0m;
    protected override bool LimitUygun(decimal tutar) => tutar <= 5000m;
}

class Kasa
{
    public decimal ToplamKomisyon { get; private set; }

    public void Tahsil(IOdeme yontem, decimal tutar)   // somut sınıfı bilmez
    {
        if (yontem.Ode(tutar))
        {
            ToplamKomisyon += yontem.Komisyon(tutar);
        }
    }
}
```

```
ÇIKTI
Kredi kartı  1200.00 TL ödendi, komisyon 36.00
Havale        800.00 TL ödendi, komisyon 5.00
Nakit        7500.00 TL reddedildi
Kredi kartı   450.00 TL ödendi, komisyon 9.00
Toplam komisyon: 50.00 TL
--- İadeler ---
Kredi kartı: 100.00 TL karta iade
Havale: 100.00 TL hesaba iade
Kredi kartı: 100.00 TL karta iade
```

Programı baştan sonra izleyelim. `Kasa.Tahsil` metodu yalnızca `IOdeme` tipini bilir; `Ode` çağrısı, nesnenin gerçek tipine göre bir `OdemeYontemi` iskeletine ulaşır ve iskelet de soyut `LimitUygun` ile `Komisyon`'u çağırır. Burada iki mekanizma iç içedir: kasa ile yöntemler arasında **arayüz**, yöntemlerin ortak kodu arasında **soyut sınıf**. `Nakit` iade edilemediği için `IIadeEdilebilir` uygulamaz; `is` sorusu o nesne için `false` verir. Yarın bir `Cek` ödeme türü eklemek, yalnızca yeni bir sınıf yazmak demektir: `Kasa` ve ana döngü değişmez.

> **Not · Çıktıdaki ondalık ayracı**
>
> Çıktılarda ondalık ayracı nokta görünür. Türkçe kültür ayarıyla çalışan bir sistemde `F2` biçimi virgül üretir (`36,00`); bu konuyu Bölüm 42 ve 43'te ele almıştık.

Bölüm Özeti

- Çok biçimlilikte aynı çağrı, nesnenin gerçek tipine göre farklı kod çalıştırır. Derleyici referans tipine bakar (üye var mı?); çalışma zamanı nesne tipine bakar (hangi sürüm?).
- `override` ile seçilen sürüm çalışma zamanında, aşırı yüklenmiş metot ve `new` ile gizlenmiş üye derleme zamanında, referans tipine göre belirlenir. Alanlar ve `static` üyeler de referans tipine göredir.
- Taban sınıfın içinden çağrılan sanal metot da nesne tipine göre seçilir. C#'ta bu, taban yapıcının içinde bile geçerlidir; C++'tan farklı olarak türetilmiş sürüm çalışır, ama türetilmiş yapıcı gövdesi henüz çalışmamıştır.
- C#'ta sınıf tipli değişkenler referans tuttuğu için nesne dilimleme yoktur.
- `abstract` sınıftan nesne üretilemez (`CS0144`); soyut üyenin gövdesi yoktur, somut alt sınıf hepsini `override` ile yazmak zorundadır (`CS0534`). Soyut sınıfın alanı, yapıcısı ve gövdeli metotları olabilir.
- Taban sınıfın iskelet algoritmayı yazıp değişen adımları soyut üyelere bırakması şablon yöntem kalıbıdır.

- `interface` bir sözleşmedir: üyeleri örtük `public`'tir, sınıfta `override` yazılmaz, nesne alanı ve yapıcı bulunmaz. Arayüz tipinde değişken tanımlanır, nesne üretilmez.
- Bir sınıf tek sınıftan türer ama istediği kadar arayüz uygular: `class C : Sinif, IA, IB`. Önce sınıf, sonra arayüzler yazılır.
- Arayüzler birbirinden türeyebilir. İki arayüzün aynı imzalı üyeleri açık uygulamayla (`void IA.Uye()`) ayrı ayrı yazılır; bu üyelere yalnızca arayüz tipli ifadeyle erişilir.
- Varsayılan arayüz metotları (C# 8) arayüz evrimi içindir; sınıfın üyesi olmazlar, yalnızca arayüz tipli ifadeyle çağrılırlar.
- Soyut sınıf ortak durum ve kod paylaşan yakın akrabalar ("bir" ilişkisi) için, arayüz ilgisiz tiplere ortak yetenek ("yapabilir") kazandırmak için uygundur; ikisi birlikte de kullanılır.
- `IComparable<T>` doğal sıralamayı (`CompareTo`), `IComparer<T>` ayrı ölçütleri (`Compare`), `IEnumerable<T>` `foreach` ile gezilebilmeyi sağlar. Doğru arayüz, hazır kütüphane kodunu kendi tipimize bağlar.
- Bağımlılığı arayüzle tersine çevirmek, üst düzey kodun somut sınıflara değil sözleşmelere bağlanmasıdır. Somut nesne yapıcıdan verilirse (bağımlılık enjeksiyonu) davranış değiştirmek ve kodu denemek kolaylaşır.

Alıştırmalar

1. `abstract class Sekil { public abstract double Alan(); }` tanımlı olsun ve `Sekil`'i doğru biçimde uygulayan, yapıcısı `Daire(double r)` olan bir `class Daire : Sekil` bulunsun. Aşağıdaki satırlardan hangisi derlenmez?

   A) `Sekil s = new Daire(2);`
   B) `Sekil[] dizi = new Sekil[3];`
   C) `Sekil s = new Sekil();`
   D) `object o = new Daire(2);`
   E) `Sekil? s = null;`

2. `A` ve `B` birer sınıf, `IX` ve `IY` birer arayüz olsun. Aşağıdaki sınıf başlıklarından hangisi derlenir?

   A) `class C : A, B`
   B) `class C : IX, A`
   C) `class C : IX, IY, A`
   D) `class C : A, IX, IY`
   E) `class C : A, B, IX`

3. Aşağıdaki kod parçası ne yazdırır?

```
interface IKonus { string Konus(); }
class Ana : IKonus { public string Konus() => "Ana"; }
class Cocuk : Ana, IKonus { public new string Konus() => "Cocuk"; }

// ana kodda:
IKonus k = new Cocuk();
Ana a = new Cocuk();
Cocuk c = new Cocuk();
Console.WriteLine(k.Konus() + a.Konus() + c.Konus());
```

A) `AnaAnaCocuk`
B) `CocukCocukCocuk`
C) `AnaCocukCocuk`
D) `CocukAnaCocuk`
E) Derleme hatası verir.

4. Arayüzlerle ilgili aşağıdakilerden hangisi **yanlıştır**?

A) Arayüz tipinde değişken ya da dizi tanımlanabilir.
B) Arayüzden `new` ile doğrudan nesne üretilemez.
C) Bir sınıf birden çok arayüz uygulayabilir.
D) Arayüz üyeleri örtük olarak `public` olduğundan, sınıftaki uygulamaları da `public` yazılır.
E) Arayüzde yapıcı (constructor) tanımlanabilir.

5. Aşağıdaki sınıf ve kod parçası verilmiştir. Çıktı ne olur?

```
class Paket : IComparable<Paket>
{
    public int Agirlik { get; }
    public Paket(int agirlik) { Agirlik = agirlik; }
    public int CompareTo(Paket? diger)
    {
        return diger is null ? 1 : diger.Agirlik.CompareTo(Agirlik);
    }
}

// ana kodda:
Paket[] d = { new Paket(70), new Paket(95), new Paket(40) };
Array.Sort(d);
Console.WriteLine(d[0].Agirlik + " " + d[2].Agirlik);
```

A) `40 95`
B) `95 40`
C) `70 40`
D) `70 95`
E) Derleme hatası verir, çünkü sınıf `Equals`'ı ezmemiştir.

6. Aşağıdaki program ekrana ne yazdırır?

```
Tasit[] filo = { new Tasit(), new Bisiklet(), new Tir() };
foreach (Tasit t in filo)
{
    t.Yaz();
}
Tasit x = filo[2];
Console.WriteLine(x is Bisiklet);
Console.WriteLine(x.Ad().Length + x.Teker());

class Tasit
{
    public virtual string Ad() => "Taşıt";
    public virtual int Teker() => 4;
    public virtual void Yaz() => Console.WriteLine(Ad() + " " + Teker());
}
class Bisiklet : Tasit
{
    public override string Ad() => "Bisiklet";
    public override int Teker() => 2;
```

```
}
class Tir : Tasit
{
    public override int Teker() => base.Teker() + 2;
    public override void Yaz()
    {
        Console.Write("[Yük] ");
        base.Yaz();
    }
}
```

7. Aşağıdaki program ekrana ne yazdırır? `override`, `new` ve aşırı yüklemeye dikkat edelim.

```
Hayvan a = new Kedi();
Kedi b = new Kedi();
Hayvan c = new Hayvan();
Console.WriteLine(Doktor.Sor(a));
Console.WriteLine(Doktor.Sor(b));
Console.WriteLine(Doktor.Sor(c));
Console.WriteLine(Doktor.Sor((Kedi)a));
Console.WriteLine(Doktor.Sor((Hayvan)b));

class Hayvan
{
    public virtual string Ses() => "...";
    public string Tur() => "Hayvan";
}
class Kedi : Hayvan
{
    public override string Ses() => "Miyav";
    public new string Tur() => "Kedi";
}
class Doktor
{
    public static string Sor(Hayvan h) => "H:" + h.Ses() + "/" + h.Tur();
    public static string Sor(Kedi k) => "K:" + k.Ses() + "/" + k.Tur();
}
```

8. Aşağıdaki program ekrana ne yazdırır?

```
ISelam[] liste = { new Ogrenci("Ece"), new Ogretmen("Kaya"), new Ogrenci("Mert") };
foreach (ISelam s in liste)
{
    Console.WriteLine(s.Selam());
}
Console.WriteLine(Kisi.Sayac);

interface ISelam { string Selam(); }
abstract class Kisi : ISelam
{
    public static int Sayac = 0;
    protected string isim;
    protected Kisi(string isim) { this.isim = isim; Sayac++; }
    public abstract string Ad();
    public virtual string Selam() => "Merhaba " + Ad();
}
class Ogrenci : Kisi
{
    public Ogrenci(string isim) : base(isim) { }
    public override string Ad() => isim;
    public override string Selam() => "Selam, ben " + Ad();
}
```

```
class Ogretmen : Kisi
{
    public Ogretmen(string isim) : base(isim) { }
    public override string Ad() => "Dr. " + isim;
}
```

9. Aşağıdaki program ekrana ne yazdırır? Aynı programın C++ karşılığı (Bölüm 31) farklı bir çıktı verirdi; farkı da açıklayalım.

```
new Kedi("Pamuk");
Console.WriteLine("---");
new Kedi("Tekir");

class Hayvan
{
    public Hayvan()
    {
        Console.WriteLine("Hayvan: " + Tanit());
    }
    public virtual string Tanit() => "hayvan";
}
class Kedi : Hayvan
{
    private string ad = "isimsiz";
    public Kedi(string ad)
    {
        this.ad = ad;
        Console.WriteLine("Kedi: " + Tanit());
    }
    public override string Tanit() => "kedi " + ad;
}
```

10. Aşağıdaki program ekrana ne yazdırır?

```
Kasa k = new Kasa();
Console.WriteLine(k.Ac());
ISifreli s = k;
Console.WriteLine(s.Ac());
IAnahtarli a = k;
Console.WriteLine(a.Ac());
Console.WriteLine(((ISifreli)k).Ac().Length);

interface ISifreli { string Ac(); }
interface IAnahtarli { string Ac(); }
class Kasa : ISifreli, IAnahtarli
{
    public string Ac() => "genel";
    string ISifreli.Ac() => "şifre";
}
```

11. Aşağıdaki tanımlar verilmiştir. Hangi satır derlenmez?

```
interface IYazici
{
    string Ad { get; }
    string Baslik() => "Yazıcı: " + Ad;
}
class Lazer : IYazici
```

```
{
    public string Ad => "Lazer";
}
```

A) `IYazici y = new Lazer();`
B) `Console.WriteLine(((IYazici)new Lazer()).Baslik());`
C) `Lazer l = new Lazer(); Console.WriteLine(l.Baslik());`
D) `IYazici y = new Lazer(); Console.WriteLine(y.Ad);`
E) `Lazer l = new Lazer(); IYazici y = l;`

12. Aşağıdaki programda iki ayrı derleme hatası vardır. Hatalı satırları bulalım, nedenlerini açıklayalım ve düzeltelim.

```
interface ITasinabilir
{
    void Tasi();
    int Agirlik { get; }
}
abstract class Esya
{
    public abstract string Ad();
}
class Koltuk : Esya, ITasinabilir
{
    public override string Ad() => "Koltuk";
    void Tasi() => Console.WriteLine("taşınıyor");
    public int Agirlik => 30;
}
class Masa : Esya
{
    public int Agirlik => 20;
}
```

13. Bir şirketin bordrosunu hesaplayan programı yazalım.
    - Soyut `Calisan` sınıfı: `Ad` özelliği, yapıcı, soyut `double Maas()` ve soyut `string Tur()` metotları ile bu ikisini kullanan somut bir `Satir()` metodu.
    - `TamZamanli`: aylık sabit maaş alır.
    - `SaatUcretli`: maaş, çalışılan saat çarpı saatlik ücrettir. Ancak 160 saatin üzerindeki her saat için ücretin 1,5 katı ödenir.
    - Ana kod: Elif (tam zamanlı, 52000), Kerem (140 saat, saati 250 TL), Ece (170 saat, saati 250 TL), Selin (tam zamanlı, 41000) çalışanlarını bir `Calisan[]` dizisine koysun. Her çalışanı `Satir()` ile (ad 6, tür 14, maaş 9 karakter genişliğinde, iki ondalıklı) ve en sonda toplam bordroyu yazdırsın.

14. Oyuncuları skora göre büyükten küçüğe sıralayan bir program yazalım. `Oyuncu` sınıfı `Ad` ve `Skor` özelliklerine sahip olsun ve `IComparable<Oyuncu>` uygulasın. Skorlar eşitse ada göre (alfabetik) sıralansın. `ToString()` ezilsin (`Ad: Skor`). Ana kod `Can 70`, `Zeynep 95`, `Mert 40`, `Deniz 95` oyuncularını `Array.Sort` ile sıralayıp yazdırsın ve en sonda lider ile son oyuncunun adını versin.

15. `IEnumerable<int>` uygulayan `Kareler` sınıfını yazalım. Yapıcısı bir `adet` alsın; `foreach` ile gezildiğinde 1'den `adet`'e kadar sayıların karelerini versin. Ana kodda `Kareler(5)` nesnesini `foreach` ile yazdıralım, ardından LINQ'nun `Sum()` ve `Max()` metotlarıyla toplamı ve en büyük değeri bulalım.

16. İndirim kurallarını arayüzle soyutlayalım. `IIndirim` arayüzü `string Ad { get; }` özelliği ve `double Uygula(double tutar)` metodunu taşısın. Üç uygulama yazalım: `YuzdeIndirim` (yüzde oranı alır), `SabitIndirim` (tutardan sabit miktar düşer, sonuç sıfırın altına inemez) ve `IndirimYok`. `Sepet` sınıfı

yalnızca `IIndirim` bilsin; `Toplam(IIndirim)` metodu sepet tutarına indirimi uygulayıp sonucu versin. Ana kodda 200 TL'lik sepeti üç kuralla da hesaplayalım.

BÖLÜM 52

Değer Tipleri: struct, enum ve record

BU BÖLÜMDE

- `struct` tipinin değer semantiğini ve `class` ile arasındaki farkları
- `struct` ile `class` arasında seçim yaparken kullanılan ölçütleri; `readonly struct` kullanımını
- `enum` tanımlamayı, temel tipini, `Enum.Parse` / `TryParse` ile metinden enum üretmeyi
- `[Flags]` ile bit bayrağı olarak kullanılan enum'ları
- `record` ve `record struct` ile değer eşitliğini, `with` ifadesini ve konumsal kaydı
- Kutulama (boxing) ve kutudan çıkarma (unboxing) işlemlerini ve gizli maliyetlerini
- C `struct` / `enum` ve C++ `struct` / `enum class` ile C# karşılıklarının farklarını

Bölüm 42'de tipleri değer tipi ve referans tipi olarak ikiye ayırmış, `struct`'ı bir değer tipi örneği olarak görmüştük. Bölüm 49'da ise `class`'ı ayrıntılı işledik. Bu bölümde değer tiplerinin kendi dünyasına giriyoruz: `struct`, `enum` ve C# 9 ile gelen `record`.

C ve C++ kısımlarındaki `struct` ve `enum` bilgimiz (Bölüm 14, Bölüm 39) bu bölümün temelidir; onları yeniden anlatmayacağız. Odak noktamız farklardır. C#'ta `struct` metot ve arayüz taşıyabilir, `enum` ad ile yazdırılıp metinden üretilebilir, `record` ise bir veri tipinin eşitlik, kopyalama ve yazdırma kodlarını derleyiciye yazdırır.

52.1 struct: Değer Semantiği

Bir `struct` tanımı, sınıf tanımına çok benzer: alanları, özellikleri, yapıcıları ve metotları olabilir. Aradaki asıl fark bellekte ve atamadadır.

> **Tanım: struct**
>
> Değer tipi olan, kullanıcı tanımlı bir tiptir. Bir `struct` değişkeni verisini doğrudan kendi içinde taşır; atama, metoda geçirme ve dönüş değeri olarak kullanma sırasında bütün alanlar kopyalanır. Sınıfın aksine `null` olamaz ve başka bir tipten türetilemez.

Bir 2B nokta için `struct` yazıp kopyalama davranışını izleyelim:

nokta.cs

```
Nokta a = new Nokta(3, 4);
Nokta b = a;                        // bütün alanlar kopyalanır
b.X = 10;
Console.WriteLine($"a = {a}, b = {b}");

KaydirKopya(a);                     // metot kendi kopyasını değiştirir
Console.WriteLine($"KaydirKopya sonrası: {a}");
```

```
KaydirRef(ref a);                  // ref ile gerçek değişkene ulaşır
Console.WriteLine($"KaydirRef sonrası: {a}");

Nokta o = default;                 // bütün alanlar sıfır
Nokta[] dizi = new Nokta[2];
dizi[0].X = 7;                     // dizi elemanı bir değişkendir
Console.WriteLine($"{o} {dizi[0]} {dizi[1]}");

void KaydirKopya(Nokta n) { n.Kaydir(1, 1); }
void KaydirRef(ref Nokta n) { n.Kaydir(1, 1); }

struct Nokta
{
    public int X;
    public int Y;

    public Nokta(int x, int y)
    {
        X = x;
        Y = y;
    }

    public void Kaydir(int dx, int dy)
    {
        X += dx;
        Y += dy;
    }

    public override string ToString() => $"({X}, {Y})";
}
```

```
ÇIKTI
a = (3, 4), b = (10, 4)
KaydirKopya sonrası: (3, 4)
KaydirRef sonrası: (4, 5)
(0, 0) (7, 0) (0, 0)
```

`Nokta b = a;` satırı `a`'nın bir kopyasını oluşturduğu için `b.X = 10` atamasından `a` etkilenmedi. `KaydirKopya` metodu da çağıran koddaki `a`'yı değil, parametre olarak gelen kopyayı değiştirdi. `ref` anahtar sözcüğü (Bölüm 46) kopya yerine değişkenin kendisini geçirir.

Bu davranış C'deki yapı atamasının aynısıdır (Bölüm 14): `b = a` bütün üyeleri kopyalar, iki yapı birbirinden bağımsızdır. Fark, C#'ta aynı sözdizimiyle bir de `class` bulunmasıdır. C'de paylaşım için açıkça işaretçi yazarız; C#'ta paylaşım isteyip istemediğimize tipi tanımlarken, `struct` ya da `class` seçerek karar veririz.

Dizi örneğindeki `dizi[0].X = 7` satırına da bakalım. `Nokta[]` dizisinin elemanları nesnelere giden referanslar değil, `Nokta` değerlerinin kendisidir ve dizinin belleğinde art arda durur. Bu yüzden her eleman bir **değişken**dir ve alanı doğrudan değiştirilebilir. `new Nokta[2]` ifadesi iki eleman için de alanları sıfırlar; `Nokta` sınıf olsaydı elemanlar `null` olurdu ve her birini ayrıca `new` ile oluşturmamız gerekirdi.

struct yapıcıları ve varsayılan değer

`struct` için her zaman, bizim yazmadığımız bir "parametresiz yapıcı" vardır: `new Nokta()` ya da `default`, bütün alanları sıfırlanmış bir değer üretir. Bu davranışı kapatamayız. Bu nedenle `struct`'ın "sıfır halini" geçerli bir durum olarak tasarlamak gerekir. Kendi yapıcımızda ise bütün alanlara değer atamamız beklenir.

Bir `struct` başka bir tipten türetilemez ve başka bir tip ondan türetilemez. Ancak arayüz uygulayabilir (Bölüm 51); örneğin `int`, `double`, `bool`, `DateTime` gibi tiplerin hepsi birer `struct`'tır ve çeşitli arayüzleri uygular. Bölüm 42'deki `int?` de aslında `Nullable<int>` adlı bir `struct`'tır.

▲ Sık Yapılan Hata · Değiştirilebilir struct'lar sürpriz üretir

Bir `struct`'ın kopyalanarak taşındığını unutursak, değişikliklerin "kaybolduğu" hatalar yazarız. Aşağıdaki iki satır derlenmez:

yapiHatalari.cs

```
var liste = new List<Nokta>();
liste.Add(new Nokta(1, 2));
liste[0].X = 5;
foreach (var n in liste) { n.X = 1; }

struct Nokta
{
    public int X;
    public int Y;
    public Nokta(int x, int y) { X = x; Y = y; }
}
```

DERLEYİCİ ÇIKTISI

```
yapiHatalari.cs(3,1): error CS1612: Cannot modify the return value of
'List<Nokta>.this[int]' because it is not a variable
yapiHatalari.cs(4,28): error CS1654: Cannot modify members of 'n' because it is a 'foreach
iteration variable'
```

`liste[0]` bir kopya döndürür; kopyaya yazılan değer kaybolacağı için derleyici bunu engeller. Koleksiyonları Bölüm 54'te işleyeceğiz; şimdilik, `struct` değerini değiştirmenin yolunun "kopyayı al, değiştir, geri yaz" ya da yapıyı baştan değişmez tasarlamak olduğunu bilmemiz yeterli.

Karşılaştırma

`struct` için `==` operatörü kendiliğinden tanımlı değildir:

yapiEsitlik.cs

```
Nokta a = new Nokta(1, 2);
Nokta b = new Nokta(1, 2);
Console.WriteLine(a == b);

struct Nokta { public int X, Y; public Nokta(int x, int y) { X = x; Y = y; } }
```

DERLEYİCİ ÇIKTISI

```
yapiEsitlik.cs(3,19): error CS0019: Operator '==' cannot be applied to operands of type 'Nokta'
and 'Nokta'
```

`a.Equals(b)` çağrısı çalışır ve alanları tek tek karşılaştırır; ancak bu varsayılan sürüm genellikle yavaştır. Kendi `Equals`, `GetHashCode` ve `==` metotlarımızı yazabiliriz (Bölüm 49'daki `Equals` ile aynı kalıp). Çoğu durumda daha kısa yol, bu bölümün sonunda göreceğimiz `record struct` tipidir.

52.2 struct mı, class mı?

Bu seçim, C# tasarımının en sık sorulan sorularından biridir. Yönerge niteliğindeki ölçütler şunlardır. Bir tip, aşağıdakilerin **hepsini** sağlıyorsa `struct` iyi bir adaydır:

- Mantıksal olarak tek bir değeri temsil eder (nokta, renk, para tutarı, tarih).
- Küçüktür; yaklaşık 16 baytı geçmez.
- Değişmezdir (immutable): oluşturulduktan sonra alanları değişmez.
- Çok sayıda kısa ömürlü kopyası oluşur ve sık sık `object`'e dönüştürülmez.

Bunların biri bile sağlanmıyorsa `class` seçmek daha güvenlidir. Aşağıdaki tablo iki tipin davranış farklarını özetler:

Özellik	`class`	`struct`
Tür	Referans tipi	Değer tipi
Atama	Referans kopyalanır, nesne paylaşılır	Bütün alanlar kopyalanır
`null` olabilir mi?	Evet	Hayır (`int?` gibi `Nullable` ile evet)
Kalıtım	Bir sınıftan türetilir, türetilebilir	Türetilemez; arayüz uygulayabilir
Varsayılan değer	`null`	Bütün alanlar sıfır
Çöp toplayıcı yükü	Her nesne öbekte yer ayırır	Yerel değişken olarak yığında, dizide dizinin içinde
`==`	Referans eşitliği	Tanımlı değil (kendimiz yazmalıyız)

> **◆ Kritik Nokta**
>
> `struct`'ı "daha hızlı sınıf" diye seçmek yaygın bir yanılgıdır. Büyük bir `struct` her atamada ve her metot çağrısında kopyalanır; bu, bir referansı kopyalamaktan çok daha pahalı olabilir. `struct` seçimini küçük ve değişmez değer benzeri tipler için yapalım; emin olmadığımızda `class` ile başlayalım.

52.3 readonly struct

Değiştirilebilir `struct`'ların sorunlarını kökten çözmenin yolu, yapıyı baştan değişmez tanımlamaktır. `struct` tanımının önüne `readonly` yazarsak derleyici bütün alanların salt okunur olmasını şart koşar ve yapının kendisini değiştiren hiçbir kod yazılamaz:

paraOrnek.cs

```
Para a = new Para(100m, "TL");
Para b = a.Ekle(50m);              // a değişmez, yeni değer döner
Console.WriteLine($"{a} {b}");
Console.WriteLine(a.Tutar);

readonly struct Para
{
    public decimal Tutar { get; }
    public string Birim { get; }

    public Para(decimal tutar, string birim)
    {
        Tutar = tutar;
        Birim = birim;
```

```
    }

    public Para Ekle(decimal miktar) => new Para(Tutar + miktar, Birim);

    public override string ToString() => $"{Tutar} {Birim}";
}
```

```
ÇIKTI
100 TL 150 TL
100
```

`Ekle` metodu mevcut değeri değiştirmez, yeni bir `Para` döndürür. `string` sınıfı da aynı mantıkla çalışır (Bölüm 48). Değişmez tasarım hem kopyalama sürprizlerini önler hem de değerin istediğimiz kadar paylaşılmasını güvenli kılar.

Yanlışlıkla değiştirilebilir bir üye bıraktığımızda derleyici hemen uyarır:

▸ readonlyHata.cs

```
Console.WriteLine(new Konum().Y);

readonly struct Konum
{
    public int X;
    public int Y { get; set; }
}
```

```
DERLEYİCİ ÇIKTISI
readonlyHata.cs(5,16): error CS8340: Instance fields of readonly structs must be readonly.
readonlyHata.cs(6,16): error CS8341: Auto-implemented instance properties in readonly structs
must be readonly.
```

`readonly struct`, derleyiciye başka bir kolaylık da sağlar: böyle bir değere erişirken gizli savunma kopyaları oluşturmaz. Bu sayede büyükçe bir `readonly struct` değerini metoda `in` parametresiyle (salt okunur referans) geçirmek, kopyalama maliyetini ortadan kaldırır. Yalnızca belirli metotları değişmez işaretlemek istersek üyenin önüne `readonly` da yazabiliriz (`public readonly double Uzunluk() => ...`). Ayrıntılı performans ayarları bu kitabın kapsamı dışındadır.

> *Tarihçe · struct'ın sürümleri*
>
> `struct`, C# 1.0'dan beri vardır. `readonly struct` C# 7.2'de (2017), `with` ifadesinin `struct`'larda kullanılabilmesi ile `record struct` ise C# 10'da (2021) eklenmiştir.

Üç Dilde · struct: C, C++ ve C# *C · C++ · C#*

C'de `struct` yalnızca veri toplar; metot taşımaz, atama alanları bayt bayt kopyalar (Bölüm 14). Paylaşım için işaretçi kullanılır.

C++'ta `struct` ile `class` neredeyse aynıdır; tek fark varsayılan erişimdir (Bölüm 30). İkisi de değer semantiğine sahiptir ve kopya yapıcıyla özelleştirilebilir. Paylaşım için işaretçi ya da referans yazılır.

C#'ta değer ya da referans olma kararı tipin tanımına bağlıdır. Aynı kodla `Nokta b = a;` yazdığımızda `Nokta` bir `struct` ise kopya, `class` ise paylaşım olur.

	C	C++	C#
Atama	Bayt bayt kopya	Kopya yapıcı / kopya ataması	Bayt bayt kopya (özelleştirilemez)

	C	C++	C#
Metot	Yok	Var	Var
Kalıtım	Yok	Var	Sınıftan yok; arayüz var
Paylaşım için	İşaretçi	İşaretçi / referans	`class` seçmek (ya da `ref`)
Başlangıç değeri	Belirsiz (yerel değişkende)	Tanıma bağlı	Her zaman sıfır (`default`)
`==`	Yok	Kendimiz tanımlarız (C++20: `= default` ile otomatik)	Kendimiz tanımlarız ya da `record struct`

52.4 enum: Adlandırılmış Sabitler

Haftanın günleri, sipariş durumları, öncelik seviyeleri gibi sınırlı bir seçenek kümesini sayılarla taşımak okunaksızdır: `durum = 2` ifadesinin neyi anlattığını kodun başka bir yerine bakmadan bilemeyiz. `enum` bu kümeye ad verir. C'deki `enum` ve C++'taki `enum class` fikrini (Bölüm 14, Bölüm 39) biliyoruz; C#'ın sürümü bu ikisinin güçlü yanlarını birleştirir.

Tanım: enum

Adlandırılmış tamsayı sabitlerinden oluşan bir değer tipidir (`enum` anahtar sözcüğüyle tanımlanır). Sabitlere her zaman `TipAdı.Sabit` biçiminde erişilir. Altta yatan sayısal tip varsayılan olarak `int`'tir.

enumTemel.cs

```
Gun bugun = Gun.Cuma;
Console.WriteLine(bugun);                       // ad yazılır
Console.WriteLine((int)bugun);                  // sayısal değer
Console.WriteLine(Tur(bugun));

Gun yarin = (Gun)((int)bugun + 1);              // sayıdan enum'a dönüşüm
Console.WriteLine($"{yarin}: {Tur(yarin)}");
Console.WriteLine(bugun < Gun.Cumartesi);       // sıralama karşılaştırılır

Console.WriteLine(Enum.GetUnderlyingType(typeof(Gun)));
Console.WriteLine(Enum.GetUnderlyingType(typeof(Oncelik)));

foreach (Gun g in Enum.GetValues<Gun>())
{
    Console.Write($"{(int)g}={g} ");
}
Console.WriteLine();

string Tur(Gun g) => g switch
{
    Gun.Cumartesi or Gun.Pazar => "hafta sonu",
    _ => "hafta içi"
};

enum Gun { Pazartesi = 1, Sali, Carsamba, Persembe, Cuma, Cumartesi, Pazar }

enum Oncelik : byte { Dusuk = 1, Orta = 5, Yuksek = 10 }
```

```
ÇIKTI
Cuma
5
hafta içi
Cumartesi: hafta sonu
True
System.Int32
System.Byte
1=Pazartesi 2=Sali 3=Carsamba 4=Persembe 5=Cuma 6=Cumartesi 7=Pazar
```

Programdan çıkardığımız kurallar şunlardır:

- Değer verilmeyen ilk sabit 0'dır; sonrakiler birer artar. `Pazartesi = 1` yazdığımız için `Cuma` 5 oldu.
- `Console.WriteLine(bugun)` sayıyı değil **adı** yazar. C'de bunun için `gunAdi[bugun]` gibi bir dizi ya da `switch` yazmak gerekiyordu (Bölüm 14); C#'ta `ToString()` bunu kendiliğinden yapar.
- Enum ile `int` arasında dönüşüm **açıktır**: `(int)bugun` ve `(Gun)5`. Bu yönüyle C++'ın `enum class`'ına yakındır; C'nin düz `enum`'undan ise ayrılır.
- Aynı tipten iki enum değeri `<`, `>`, `==` ile karşılaştırılabilir ve `switch` ifadesinde (Bölüm 44) kullanılabilir.
- Temel tipi `: byte` gibi bir ekle değiştirebiliriz. İzin verilenler `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long` ve `ulong`'dur. Dosyaya yazılacak ya da çok sayıda saklanacak değerlerde küçük bir tip yer kazandırır.
- `Enum.GetValues<Gun>()` bütün sabitleri, `Enum.GetNames<Gun>()` bütün adları döndürür. Genel (generic) sürümler .NET 5'ten beri vardır.

Enum'a metot ekleyemeyiz; bunun yerine yukarıdaki `Tur` gibi yardımcı metotlar ya da `switch` ifadeleri yazarız.

Aralık denetimi yoktur

C gibi C# da enum değişkenine tanımlı olmayan bir sayı atanmasına engel olmaz:

enumAralik.cs

```
Gun g = (Gun)42;
Console.WriteLine(g);
Console.WriteLine(Enum.IsDefined(g));
Console.WriteLine(Enum.IsDefined(Gun.Sali));

Gun varsayilan = default;
Console.WriteLine(varsayilan);
Console.WriteLine((int)varsayilan);

enum Gun { Pazartesi = 1, Sali, Carsamba }
```

```
ÇIKTI
42
False
True
0
0
```

Adı olmayan bir değer, ad yerine sayı olarak yazdırılır. Dışarıdan gelen bir sayıyı enum'a çevirmeden önce `Enum.IsDefined` ile denetlemek güvenli bir alışkanlıktır. Ayrıca `default(Gun)` her zaman 0'dır; yukarıdaki enum'da 0 değerli bir sabit olmadığı için varsayılan değerin adı yoktur. Bu yüzden iyi bir uygulama olarak 0

değerine anlamlı bir ad veririz (`Yok`, `Belirsiz`, `Bilinmiyor`). Böylece sıfırlanmış bir alan "geçersiz" olarak okunabilir.

> **▲ Sık Yapılan Hata · 0 değerine ayrıcalık tanınır**
>
> Sabit `0`, hiçbir dönüşüm yazmadan her enum tipine atanabilir (`Gun g = 0;` derlenir). Başka hiçbir sayı bu ayrıcalığa sahip değildir; `Gun g = 1;` hata verir. Bu istisna, `[Flags]` enum'larında `(izin & Izin.Yazma) != 0` gibi karşılaştırmaları mümkün kılmak için vardır.

52.5 Metinden Enum Üretmek

Dosyadan okunan ya da kullanıcının yazdığı bir metni enum değerine çevirmek için `Enum.Parse` ve `Enum.TryParse` kullanırız. Mantık, Bölüm 42'de gördüğümüz `int.Parse` ve `int.TryParse` ikilisiyle aynıdır: `Parse` geçersiz girdide istisna fırlatır (Bölüm 53), `TryParse` ise başarı durumunu `bool` olarak döndürür.

enumParse.cs

```
Gun g1 = Enum.Parse<Gun>("Sali");
Gun g2 = Enum.Parse<Gun>("persembe", ignoreCase: true);
Console.WriteLine($"{g1} {g2}");

Console.WriteLine(Enum.TryParse<Gun>("Cuma", out Gun g3));
Console.WriteLine(g3);

Console.WriteLine(Enum.TryParse<Gun>("Bayram", out Gun g4));
Console.WriteLine((int)g4);

// sayı içeren metin de başarılı sayılır
bool ok = Enum.TryParse<Gun>("42", out Gun g5);
Console.WriteLine($"{ok} {g5} {Enum.IsDefined(g5)}");

Gun? sonuc = Enum.TryParse<Gun>("Pazar", out var g6) && Enum.IsDefined(g6)
    ? g6 : null;
Console.WriteLine(sonuc);

enum Gun { Pazartesi = 1, Sali, Carsamba, Persembe, Cuma, Cumartesi, Pazar }
```

```
ÇIKTI
Sali Persembe
True
Cuma
False
0
True 42 False
Pazar
```

Üç ayrıntıya dikkat edelim:

- Karşılaştırma varsayılan olarak büyük/küçük harfe duyarlıdır; `ignoreCase: true` ile bu kapatılır.
- Başarısız `TryParse`, `out` değişkenine `default` değeri yazar. Burada `0` olan değer tanımlı bir sabit değildir.
- `"42"` gibi sayı içeren bir metin, sayı enum'da tanımlı olmasa bile **başarılı** sayılır. Dışarıdan gelen veride `TryParse` ile birlikte `IsDefined` denetimi gerekir; son satırdaki ifade bu ikilinin birleşimidir.

> **İpucu · Enum değerini metne çevirmek**
>
> `ToString()` adı verir; `ToString("D")` onluk sayıyı, `ToString("X")` onaltılık sayıyı yazar. Kullanıcıya gösterilecek metin Türkçe olacaksa (`Çarşamba`), enum adlarını doğrudan göstermek yerine bir `switch` ifadesiyle ayrı metinlere eşleriz; sabit adlarını Türkçe karakter içermeyecek biçimde tutarız.

52.6 [Flags]: Bit Bayrağı Olarak Enum

Bölüm 16'da `OKU`, `YAZ`, `CALISTIR` gibi bayrakları `#define` ile tanımlamış ve `|`, `&`, `~` operatörleriyle birleştirmiştik. C# bu kalıbı enum ile dile yerleştirir. Sabitleri ikinin kuvvetleri olarak seçeriz; enum'un üzerine de `[Flags]` yazarız. Köşeli parantez içindeki bu işarete öznitelik (attribute) denir; derleyiciye ve çalışma zamanına tip hakkında ek bilgi verir.

izinBayraklari.cs

```
Izin i = Izin.Okuma | Izin.Yazma;
Console.WriteLine($"{i} ({(int)i})");
Console.WriteLine(i.HasFlag(Izin.Yazma));
Console.WriteLine((i & Izin.Calistirma) != 0);

i |= Izin.Calistirma;                // bayrak ekle
Console.WriteLine(i);

i &= ~Izin.Yazma;                    // bayrak kaldır
Console.WriteLine(i);

i ^= Izin.Okuma;                     // bayrağı tersle
Console.WriteLine(i);

Console.WriteLine(Izin.Yok);
Console.WriteLine((Renk)3);          // [Flags] olmayan enum

[Flags]
enum Izin
{
    Yok = 0,
    Okuma = 1,
    Yazma = 2,
    Calistirma = 4,
    Tam = Okuma | Yazma | Calistirma
}

enum Renk { Kirmizi = 1, Yesil = 2 }
```

```
ÇIKTI
Okuma, Yazma (3)
True
False
Tam
Okuma, Calistirma
Calistirma
Yok
3
```

Bu programdan şunları öğreniyoruz:

- `|`, `&`, `^` ve `~` operatörleri doğrudan enum değerleri üzerinde çalışır; C++'ın `enum class`'ında olduğu gibi bunları kendimiz tanımlamak zorunda kalmayız.

- `[Flags]` bulunan bir enum'un değeri yazdırılırken birleşik bayrakların adları virgülle ayrılarak listelenir (`Okuma, Yazma`). Özniteliği olmayan `Renk` enum'unda `3` değeri adsız olduğu için sayı olarak yazılır.
- Üç izin birleştiğinde değer 7 olur; bu değerin tam adı `Tam` olarak tanımlı olduğu için ad olarak o yazılır.
- `HasFlag` tek bir bayrağı sorgular. Birden çok bayrağı verirsek hepsinin birden açık olup olmadığına bakar. `(i & Izin.Calistirma) != 0` biçimi aynı işi bit operatörleriyle yapar; sıfırla karşılaştırma yukarıdaki 0 ayrıcalığı sayesinde derlenir.
- Bir bayrağı kaldırmak için `&= ~bayrak`, tersleme için `^=` kullanılır; bunlar Bölüm 16'daki kalıpların aynısıdır.

> **◆ Kritik Nokta**
>
> `[Flags]` özniteliği değerleri **kendiliğinden** ikinin kuvveti yapmaz. Sabitleri `1, 2, 4, 8, ...` olarak biz yazarız; öznitelik yalnızca `ToString` ve `Parse` davranışını bayraklara göre ayarlar. Değerler çakışırsa (örneğin `Okuma = 1`, `Yazma = 3`) birleşimler ayırt edilemez. Ayrıca bir `[Flags]` enum'una daima `Yok = 0` sabiti ekleriz.

Üç Dilde · enum: C, C++ ve C# *C · C++ · C#*

	C	C++	C#
Tür	Tamsayı sabitleri, `int` gibi davranır	`enum class` ayrı bir tiptir (düz `enum` C gibi)	Ayrı bir değer tipidir
Sabitlere erişim	`PAZAR` (kapsama sızar)	`Gun::Pazar` (`enum class`)	`Gun.Pazar`
`int` dönüşümü	Örtük	`enum class`'ta açık (`static_cast`)	Açık (`(int)g`), yalnızca `0` örtük
Temel tip	Derleyici seçer	`enum class Gun : unsigned char`	`enum Gun : byte`
Ad yazdırma	Dizi ya da `switch`	Dizi ya da `switch`	`ToString()`
Metinden üretme	Elle yazılır	Elle yazılır	`Enum.Parse`, `Enum.TryParse`
Bütün sabitleri dolaşma	Elle yazılır	Elle yazılır	`Enum.GetValues<T>()`
Bayrak operatörleri	Yerleşik (`int`)	`enum class`'ta kendimiz tanımlarız	Yerleşik (`[Flags]` ile ad listesi)
Aralık denetimi	Yok	Yok	Yok (`Enum.IsDefined` ile elle)

52.7 record: Veri Taşıyan Tipler

Çoğu uygulamada, yalnızca veri taşıyan küçük tipler yazarız: bir kişi, bir ürün, bir koordinat. Bunlar için sınıf yazdığımızda hep aynı şablonu tekrarlarız: özellikler, yapıcı, içerik karşılaştıran `Equals`, uyumlu bir `GetHashCode`, okunaklı bir `ToString`. Bölüm 49'da bu işin sınıf için ne kadar uzun sürdüğünü görmüştük. `record` bu şablonu derleyiciye yazdırır.

> **Tanım: record**
>
> Veri taşımak amacıyla tasarlanmış tiptir. Derleyici bir `record` için değer eşitliği (`Equals`, `==`, `!=`, `GetHashCode`), okunaklı bir `ToString`, `with` ifadesiyle kopyalama ve konumsal tanımda yapıcı ile ayrıştırma (`Deconstruct`) kodunu kendisi üretir. Yalnızca `record` yazılırsa tip bir **sınıftır** (referans tipi); `record struct` ise bir değer tipidir.

En kısa tanım, konumsal kayıt (positional record) biçimidir. Parantez içine yazılan her parametre, hem yapıcı parametresi hem de bir özellik olur:

recordTemel.cs

```
var k1 = new Kisi("Elif", 20);
var k2 = new Kisi("Elif", 20);
var k3 = k1 with { Yas = 21 };         // kopya al, bir alanı değiştir

Console.WriteLine(k1);
Console.WriteLine(k3);
Console.WriteLine(k1 == k2);                      // içerik eşitliği
Console.WriteLine(ReferenceEquals(k1, k2));       // yine iki ayrı nesne
Console.WriteLine(k1.GetHashCode() == k2.GetHashCode());

var (ad, yas) = k3;                    // ayrıştırma
Console.WriteLine($"{ad} / {yas}");

var s1 = new KisiSinif { Ad = "Elif", Yas = 20 };
var s2 = new KisiSinif { Ad = "Elif", Yas = 20 };
Console.WriteLine(s1 == s2);                      // sınıfta referans eşitliği

record Kisi(string Ad, int Yas);

class KisiSinif
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
}
```

```
ÇIKTI
Kisi { Ad = Elif, Yas = 20 }
Kisi { Ad = Elif, Yas = 21 }
True
False
True
Elif / 21
False
```

Tek satırlık `record Kisi(string Ad, int Yas);` tanımı bize şunları verdi:

- `Ad` ve `Yas` özellikleri. Bunlar `init` erişimlidir (Bölüm 49): yalnızca nesne oluşturulurken atanabilir, sonra değiştirilemez.
- `new Kisi("Elif", 20)` yapıcısı.
- Alanlara göre `Equals`, `==`, `!=` ve uyumlu `GetHashCode`. İki ayrı nesne aynı içeriği taşıyorsa eşittir. Aynı içerikli sıradan bir `class` ise `==` ile eşit çıkmaz; sınıfta `==` varsayılan olarak referansı karşılaştırır (Bölüm 49).
- `Kisi { Ad = Elif, Yas = 20 }` biçiminde bir `ToString`.

- `with` ifadesi: nesnenin bir kopyasını oluşturur ve süslü parantez içinde belirtilen özellikleri değiştirir. Özgün nesne değişmez.
- `var (ad, yas) = k3;` ayrıştırması. Ayrıştırmayı Bölüm 46'daki demetlerden tanıyoruz.

Bir `record` tanımına istediğimiz üyeyi de ekleyebiliriz. Aşağıda kayıt, hesaplanan bir özellik ve bir metotla zenginleşiyor. Aynı örnekte `with`'in sığ (shallow) kopya yaptığını da görelim:

recordGovde.cs

```
var u = new Urun("Kitap", 250m);
Console.WriteLine($"{u.Ad}: {u.Fiyat} / KDV'li {u.KdvliFiyat}");

var zamli = u with { Fiyat = u.Fiyat * 2 };
Console.WriteLine($"{zamli.Fiyat} {zamli.Pahali()}");

var s1 = new Sepet("Mert", new[] { "kalem", "defter" });
var s2 = s1 with { Sahip = "Deniz" };
s2.Urunler[0] = "silgi";                  // dizi iki kayıt arasında paylaşılıyor
Console.WriteLine($"{s1.Sahip}: {s1.Urunler[0]}");
Console.WriteLine($"{s2.Sahip}: {s2.Urunler[0]}");
Console.WriteLine(s1 == s2);

record Urun(string Ad, decimal Fiyat)
{
    public decimal KdvliFiyat => Fiyat * 1.20m;
    public bool Pahali() => Fiyat > 400m;
}

record Sepet(string Sahip, string[] Urunler);
```

ÇIKTI

```
Kitap: 250 / KDV'li 300.00
500 True
Mert: silgi
Deniz: silgi
False
```

`with` yalnızca kayıt nesnesinin kendisini kopyalar; içindeki referans tipli özellikler (burada dizi) paylaşılmaya devam eder. Bu yüzden `s2.Urunler[0]` değişikliği `s1`'de de görünür. Son satırdaki `False` da ilginçtir: `Sahip` farklı olduğu için `s1 != s2`; ama sahipler aynı olsaydı bile diziler farklı nesneler olduğundan eşitlik yine `False` çıkardı. Kayıt eşitliği, alanları **kendi** eşitlikleriyle karşılaştırır; dizi içeriğini karşılaştırmaz.

▲ Sık Yapılan Hata · Kayıt değişmez değildir, yalnızca `init` ile korunur

Konumsal kayıtların özellikleri `init` olduğu için nesne oluşturtan sonra atanamaz:

recordInit.cs

```
var k = new Kisi("Elif", 20);
k.Yas = 30;

record Kisi(string Ad, int Yas);
```

DERLEYİCİ ÇIKTISI

```
recordInit.cs(2,1): error CS8852: Init-only property or indexer 'Kisi.Yas' can only be
assigned in an object initializer, or on 'this' or 'base' in an instance constructor or an
'init' accessor.
```

Ancak içindeki bir dizi ya da nesne yine değiştirilebilir (yukarıdaki `Sepet`). Gerçek değişmezlik, içerdiğimiz tiplerin de değişmez olmasına bağlıdır.

Kayıtlar kalıtıma da katılabilir: bir `record` yalnızca başka bir `record`'dan türetilebilir ve türeyen kayıt, `Equals` karşılaştırmasında tipin de aynı olmasını arar. Bu kitapta kayıtları kalıtımsız, küçük veri taşıyıcıları olarak kullanacağız.

record struct

Aynı fikri değer tipi olarak istersek `record struct` yazarız. Önemli bir fark vardır: konumsal `record struct`'ın özellikleri varsayılan olarak **değiştirilebilir** (`get; set;`). Değişmez istiyorsak `readonly record struct` yazarız:

recordStruct.cs

```
Konum a = new Konum(1, 2);
Konum b = a with { X = 9 };
Console.WriteLine($"{a} {b}");
Console.WriteLine(a == new Konum(1, 2));

a.X = 5;                              // record struct'ta izin var
Console.WriteLine(a);

Sabit s = new Sabit(3, 4);
Console.WriteLine(s with { Y = 0 });

record struct Konum(int X, int Y);

readonly record struct Sabit(int X, int Y);
```

ÇIKTI

```
Konum { X = 1, Y = 2 } Konum { X = 9, Y = 2 }
True
Konum { X = 5, Y = 2 }
Sabit { X = 3, Y = 0 }
```

`record struct` hem `struct`'ın (kopyalanır, `null` olmaz, küçük tiplerde ucuzdur) hem de `record`'un (değer eşitliği, `ToString`, `with`) avantajlarını birleştirir. Yeni bir küçük değer tipi yazarken ilk tercihimiz `readonly record struct` olmalıdır; `struct` için gereken `Equals` ve `==` kodunu bizim yerimize üretir.

Hangi tipi ne zaman seçeceğimizi aşağıdaki tablo özetler:

Tip	Tür	Eşitlik	Değişebilir mi?	Ne zaman?
`class`	Referans	Referans	Evet	Davranışı olan, kimliği önemli nesneler
`record`	Referans	İçerik	`init` ile hayır	Veri taşıyan, paylaşılan değişmez kayıtlar
`struct`	Değer	Kendimiz yazarız	Evet (önerilmez)	Çok özel durumlar; çoğu zaman `readonly struct`
`readonly struct`	Değer	Kendimiz yazarız	Hayır	Küçük değerler, özel eşitlik gerekirse
`record struct`	Değer	İçerik	Evet	Küçük, değişebilir değerler
`readonly record struct`	Değer	İçerik	Hayır	Küçük değerler için ilk tercih

Tarihçe · record'un tarihi

`record` (sınıf olarak) ve `init` erişimcisi C# 9 ile, .NET 5 sürümüyle birlikte 2020'de geldi. `record struct`, `readonly record struct` ve `struct`'larda `with` ifadesi C# 10 ile (2021, .NET 6) eklendi. C++'ta karşılığı olmayan bu özelliğe en yakın yaklaşım, `= default` ile üretilen karşılaştırma operatörleridir (C++20).

52.8 Kutulama ve Kutudan Çıkarma

Değer tipleri normalde yığında ya da bir nesnenin/dizinin içinde yaşar. Ama bazen bir değer tipli değişkeni `object` ya da arayüz tipli bir değişkene atamamız gerekir; bu tiplerin hepsi referans tipidir. Böyle bir atamada C# değeri öbekte (heap) yeni bir nesnenin içine sarar. Bu işleme kutulama (boxing) denir.

Tanım: Kutulama ve kutudan çıkarma

Bir değer tipi değerinin `object` ya da arayüz tipine dönüştürülmesine **kutulama** (boxing) denir: öbekte yeni bir nesne oluşturulur ve değer oraya kopyalanır. Kutudaki değeri yeniden değer tipine çevirmeye **kutudan çıkarma** (unboxing) denir; bu işlem açık dönüşüm gerektirir ve kopya oluşturur.

kutulama.cs

```
int x = 10;
object kutu = x;                    // kutulama: x'in kopyası öbeğe gider
x = 20;
Console.WriteLine($"{x} {kutu}");

int y = (int)kutu;                  // kutudan çıkarma
Console.WriteLine(y);
Console.WriteLine(kutu is int);
Console.WriteLine(kutu is long);    // kutudaki tip tam olarak int
Console.WriteLine(kutu.GetType());

Sayac s = new Sayac();
IArtir i = s;                       // kutulama: s kopyalanır
i.Artir();
i.Artir();
Console.WriteLine($"{s.Deger()} {i.Deger()}");

Sayac s2 = (Sayac)i;                // kutudan çıkarma
```

```
Console.WriteLine(s2.Deger());

interface IArtir
{
    void Artir();
    int Deger();
}

struct Sayac : IArtir
{
    private int sayi;
    public void Artir() => sayi++;
    public int Deger() => sayi;
}
```

```
ÇIKTI
20 10
10
True
False
System.Int32
0 2
2
```

İlk bölümde `kutu`, `x`'in kopyasını tutar; `x`'in sonradan değişmesi kutuyu etkilemez. `kutu is long` ifadesinin `False` çıkması, kutudan çıkarmanın **tam** tipi gerektirdiğini gösterir: kutunun içinde bir `int` varsa `(long)kutu` yazarak çıkaramayız; yazarsak çalışma zamanında `InvalidCastException` fırlatılır (Bölüm 53). Doğrusu `(long)(int)kutu` biçimindedir.

İkinci bölüm daha öğreticidir. `IArtir i = s;` satırı `s`'yi kutuladı; iki `Artir()` çağrısı kutudaki kopyayı değiştirdi. Orijinal `s` hâlâ 0'dadır. Kutudan çıkardığımızda `s2`, kutudaki değerin yeni bir kopyasıdır ve 2 değerini taşır.

Kutulamanın maliyeti

Kutulama her seferinde öbekte nesne oluşturur ve bu nesneyi sonradan çöp toplayıcı temizler. Tek tek önemsiz görünse de bir döngüde milyonlarca kez gerçekleşirse bellek baskısı ve yavaşlama yaratır. Kutulamanın gizlice gerçekleştiği başlıca yerler şunlardır:

- Değer tipini `object` parametreli bir metoda geçirmek.
- Değer tipini, uyguladığı bir arayüz tipli değişkene atamak.
- Eski, jenerik olmayan koleksiyonlara (`ArrayList` gibi) değer tipi eklemek. `List<int>` gibi jenerik koleksiyonlar tam da bu maliyeti ortadan kaldırmak için geliştirilmiştir (Bölüm 54).
- `struct`'ın `ToString`, `Equals` ya da `GetHashCode` gibi `object`'ten gelen ve kendisi ezmediği (override) metotlarını çağırmak; bu bazı durumlarda kutulamaya yol açabilir. Bu yüzden kendi `struct`'larımızda bu üç metodu ezmek (ya da `record struct` kullanmak) hem davranışı hem hızı iyileştirir.

> **◆ Kritik Nokta**
>
> Kutulama, "değer tipini referans tipi gibi kullanma" isteğinin bedelidir. Hızlı çalışması gereken kodda `object` ve jenerik olmayan koleksiyonlardan kaçınır, bunların yerine jenerik tipler kullanırız. Bir değerin kutulanıp kutulanmadığını, değer tipli bir değişkenin `object` ya da arayüz tipli bir değişkene atandığı yerlere bakarak anlarız.

Bölüm Özeti

- `struct` bir değer tipidir; atama, parametre ve dönüş değeri sırasında bütün alanlar kopyalanır. `null` olamaz, kalıtımla türetilemez, arayüz uygulayabilir.
- `struct` her zaman sıfırlanmış bir varsayılan değere sahiptir (`default`). Tasarımda bu "sıfır hali" geçerli bir durum olmalıdır.
- Küçük, değişmez ve tek bir değeri temsil eden tipler için `struct`; kimliği olan ya da büyük tipler için `class` seçeriz.
- Değiştirilebilir `struct`'lar, kopyaların değiştirilmesi yüzünden hata üretir (`CS1612`, `CS1654`). `readonly struct` bu sorunu kökten çözer.
- `enum`, adlandırılmış tamsayı sabitleri kümesidir; varsayılan temel tip `int`'tir ve `: byte` gibi bir ekle değiştirilebilir.
- Enum ile sayı arasındaki dönüşüm açıktır; enum değişkeni tanımlı olmayan değerler alabilir. `Enum.IsDefined` ile denetleriz, 0 değerine anlamlı bir ad veririz.
- `ToString()` enum değerini ada çevirir; `Enum.Parse` ve `Enum.TryParse` adı değere çevirir. `TryParse` sayı içeren metinleri tanımsız değer olsa da kabul eder.
- `[Flags]` enum'ları ikinin kuvveti değerlerle bit bayrağı olarak kullanılır; `|`, `&`, `^`, `~` doğrudan çalışır ve `ToString` birleşik adları listeler.
- `record`, değer eşitliği, `ToString`, `with` ve ayrıştırma kodunu otomatik üretir; tek başına yazıldığında bir sınıftır. Konumsal parametreler `init` özellikleri olur.
- `with` sığ kopya yapar; kayıt içindeki referans tipli özellikler paylaşılmaya devam eder.
- `record struct` değer tipi bir kayıttır; özellikleri varsayılan olarak değiştirilebilirdir. Küçük değerler için `readonly record struct` ilk tercihimiz olmalıdır.
- Değer tipi `object` ya da arayüze atandığında kutulama gerçekleşir: öbekte yeni bir nesne ve bir kopya oluşur. Kutudan çıkarma tam tipi ister.
- C ve C++ ile fark: C#'ta değer/referans kararı tip tanımında verilir; `enum` ad, ayrıştırma ve sabit listesi sunar; `record` ise karşılaştırma ve kopyalama kodunu derleyiciye bırakır.

Alıştırmalar

1. Aşağıdaki program ekrana ne yazdırır?

```
Nokta a = new Nokta { X = 1 };
Nokta b = a;
b.X = 2;
Nokta c = Artir(b);
Console.WriteLine($"{a.X} {b.X} {c.X}");

Nokta Artir(Nokta n)
{
    n.X += 10;
    return n;
}

struct Nokta { public int X; }
```

A) `1 2 12`
B) `2 2 12`
C) `1 12 12`
D) `2 12 12`
E) Derlenmez

2. `struct` ile ilgili aşağıdakilerden hangisi **yanlıştır**?

A) Bir `struct` değişkeni `null` olamaz.
B) Bir `struct` bir arayüzü uygulayabilir.
C) Bir `struct` başka bir `struct`'tan türetilebilir.
D) `new Nokta()` ifadesi alanları sıfırlanmış bir değer üretir.
E) `Nokta[]` dizisinin elemanları `new` kullanılmadan da kullanılabilir.

3. Aşağıdaki enum için `Enum.TryParse<Seviye>("7", out var s)` ifadesi ne döndürür ve `s` ne olur?

```
enum Seviye { Dusuk = 1, Orta = 2, Yuksek = 3 }
```

A) `false`, `s` = `Dusuk`
B) `false`, `s` = `0`
C) `true`, `s` = `7` (tanımsız değer)
D) `true`, `s` = `Yuksek`
E) İstisna fırlatır

4. İki `record` nesnesi `a` ve `b` için hangisi doğrudur?

A) `a == b` her zaman referansları karşılaştırır.
B) Alanları aynıysa `a == b` doğrudur, `ReferenceEquals(a, b)` ise yanlıştır.
C) `a with { }` aynı nesneyi döndürür.
D) `record` kullanmak için `Equals` metodunu elle yazmak gerekir.
E) `record` yalnızca değer tipi olabilir.

5. Aşağıdaki program ekrana ne yazdırır?

```
Izin i = Izin.Okuma;
i |= Izin.Yazma;
i |= Izin.Calistirma;
i &= ~Izin.Okuma;
Console.WriteLine(i);
Console.WriteLine((int)i);
Console.WriteLine(i.HasFlag(Izin.Okuma));

[Flags]
enum Izin { Yok = 0, Okuma = 1, Yazma = 2, Calistirma = 4 }
```

6. Aşağıdaki program ekrana ne yazdırır?

```
var a = new Ders("Fizik", 3);
var b = a with { Kredi = 4 };
var c = a with { };
Console.WriteLine(a);
Console.WriteLine(b);
Console.WriteLine(a == c);
Console.WriteLine(ReferenceEquals(a, c));

record Ders(string Ad, int Kredi);
```

7. Aşağıdaki program ekrana ne yazdırır?

```
object o = 5;
int n = (int)o;
n++;
Console.WriteLine($"{o} {n}");

object p = 5;
object q = p;
Console.WriteLine(p == q);
Console.WriteLine(p.Equals(5));
Console.WriteLine(ReferenceEquals(o, p));
```

8. Aşağıdaki kodda derleme hatası veren iki satır vardır. Bu satırları bulun ve nedenini açıklayın.

```
var liste = new List<Renk>();
liste.Add(new Renk(255, 0, 0));
liste[0].R = 100;
Renk r = new Renk(1, 2, 3);
Console.WriteLine(r == new Renk(1, 2, 3));

struct Renk
{
    public int R, G, B;
    public Renk(int r, int g, int b) { R = r; G = g; B = b; }
}
```

9. Aşağıdaki işlemlerden hangisinde kutulama **gerçekleşmez**?

   A) `object o = 3.14;`
   B) `IComparable<int> c = 5;`
   C) `int[] dizi = new int[10]; dizi[0] = 4;`
   D) `object[] liste = { 1, 2, 3 };`
   E) `ArrayList l = new ArrayList(); l.Add(7);`

10. `Gun g = (Gun)42;` satırı neden derlenir ve çalışır? Böyle bir değerle karşılaşmamak için hangi önlemleri alabiliriz?

11. Aşağıdaki durumların her biri için `class`, `struct` ya da `readonly record struct` seçeneklerinden birini önerin ve gerekçesini bir cümleyle yazın: (a) 3 boyutlu bir vektör (X, Y, Z), (b) veritabanı bağlantısını yöneten nesne, (c) bir müşterinin ad, soyad ve e-posta bilgisi, (d) iki tarih arasındaki aralık (başlangıç ve bitiş).

12. C'deki `enum Durum { ACIK, KAPALI }` ile C#'taki `enum Durum { Acik, Kapali }` arasında, kullanım açısından en az üç fark yazın.

13. `Dikdortgen` adında bir `readonly struct` yazın: `En` ve `Boy` özellikleri (`double`), `Alan()` ve `Cevre()` metotları ve `En` değerini iki katına çıkarıp yeni bir `Dikdortgen` döndüren `GenislikIkiKat()` metodu bulunsun. `new Dikdortgen(3, 4)` için alanı, çevreyi ve yeni dikdörtgenin alanını yazdırın.

14. `Mevsim` adında bir `enum` tanımlayın (`Ilkbahar`, `Yaz`, `Sonbahar`, `Kis`). Kullanıcıdan bir mevsim adını okuyup `Enum.TryParse` ile (büyük/küçük harf fark etmeksizin) çevirin; tanımlı bir mevsimse `switch` ifadesiyle o mevsimin ilk ayını yazdırın (`Ilkbahar` için `Mart`, `Yaz` için `Haziran`, `Sonbahar` için `Eylül`, `Kis` için `Aralik`); geçersizse `Geçersiz mevsim` yazdırın.

15. `[Flags]` ile `Gunler` adında bir enum tanımlayın (`Pzt`, `Sal`, `Car`, `Per`, `Cum`, `Cmt`, `Paz`; `Yok = 0` ve `HaftaIci`, `HaftaSonu` birleşik sabitleri de olsun). Bir toplantı günleri kümesi oluşturun, `Cum` gününü

ekleyin, `Sal` gününü çıkarın, sonucu yazdırın ve kümenin hafta sonuna denk gelip gelmediğini `&` ile sorgulayın.

16. `Urun` adında bir `record` tanımlayın (`Ad`, `Fiyat`). `Fiyat` değeri 100 olan bir ürün oluşturun, `with` ifadesiyle fiyatı yüzde 10 artırılmış yeni bir ürün üretin, iki ürünü yazdırın ve iki ürünün eşit olup olmadığını yazdırın.

BÖLÜM 53

İstisna Yönetimi

BU BÖLÜMDE

- İstisnanın (exception) ne olduğunu ve yakalanmayan bir istisnanın programı nasıl sonlandırdığını
- `try`, `catch` ve `finally` bloklarını yazmayı; birden çok `catch`'i doğru sırayla dizmeyi
- `Exception` hiyerarşisini ve en sık karşılaşılan istisna türlerini
- `throw` ile istisna fırlatmayı, `throw;` ile `throw ex;` arasındaki farkı, `when` süzgecini
- Kendi istisna sınıfımızı yazmayı ve iç istisna (`InnerException`) zincirini
- `using` deyimi ve `IDisposable` ile kaynakları otomatik bırakmayı
- İstisna ile `TryParse` deseni arasında doğru seçim yapmayı

Bölüm 34'te C++ istisnalarını, Bölüm 42 ve 43'te de `DivideByZeroException`, `FormatException` ve `OverflowException` gibi C# istisnalarının ilk örneklerini gördük. Orada `try` ve `catch` bloklarını yalnızca bir olayı izlemek için kullanmıştık. Şimdi mekanizmanın tamamını işleyeceğiz.

C++ istisnalarını bilen bir okur için temel fikir tanıdıktır: hata `throw` ile fırlatılır, çağrı zincirinde yukarı doğru tırmanır ve uyan bir `catch` tarafından yakalanır. Farklar ayrıntıdadır. C#'ta yalnızca `Exception` sınıfından türeyen nesneler fırlatılabilir, her istisna çağrı zincirinin bir kaydını taşır, kaynak temizliği için yıkıcı yerine `finally` ve `using` vardır, `noexcept` ya da `throw(...)` belirtimi yoktur. C++ kısmında anlattıklarımızı (yığın çözülmesi, `catch` sırası, özel istisna sınıfı) tekrar etmeyecek, yalnızca C#'a özgü olanlara odaklanacağız.

53.1 İstisna Nedir?

> **Tanım: İstisna (exception)**
>
> Program çalışırken oluşan ve normal akışı bozan olaya istisna denir. C#'ta her istisna bir nesnedir; hatanın türünü, açıklamasını ve oluştuğu andaki çağrı zincirini (stack trace) taşır.

Olmayan bir dizi elemanına erişmek, sayı beklenirken harf okumak, değeri `null` olan bir değişkenin üyesini kullanmak birer istisna doğurur. Bir istisna hiçbir yerde yakalanmazsa program durur ve çalışma zamanı bir hata raporu yazar. Aşağıdaki program not listesindeki bir metni sayıya çevirmeye çalışıyor:

cokme.cs

```
string[] notlar = { "85", "70", "doksan", "60" };
Console.WriteLine($"Ortalama: {Hesap.Ortalama(notlar)}");

class Hesap
{
    public static int Ortalama(string[] notlar)
```

```
    {
        int toplam = 0;
        foreach (string s in notlar)
        {
            toplam += int.Parse(s);
        }
        return toplam / notlar.Length;
    }
}
```

```
TERMİNAL
$ dotnet run
Unhandled exception. System.FormatException: The input string 'doksan' was not in a correct for
mat.
   at Hesap.Ortalama(String[] notlar) in cokme.cs:line 11
   at Program.<Main>$(String[] args) in cokme.cs:line 2
```

Raporu yukarıdan aşağıya okuyalım. İlk satır istisnanın türünü (`System.FormatException`) ve mesajını verir. Sonraki `at` satırları, hatanın oluştuğu andaki çağrı zinciridir: en üstteki satır hatanın fırlatıldığı yer, her alt satır bir üsttekini çağıran metottur. `Program.<Main>$`, top-level statements ile yazılan kodun çalıştığı gizli metodun adıdır. Gerçek raporda `System.` ile başlayan .NET iç satırları ve dosya adının önünde tam klasör yolu da görünebilir; burada yer kazanmak için kısalttık.

Tek bir bozuk veri yüzünden ortalama hiç hesaplanmadı. Hatayı yakalarsak programı sürdürebiliriz.

53.2 try ve catch

İstisna çıkabilecek kod `try` bloğuna, hata olursa çalışacak kod `catch` bloğuna yazılır:

```
try
{
    // istisna fırlatabilecek kod
}
catch (IstisnaTuru ex)
{
    // o türde istisna olursa çalışacak kod
}
```

Aynı not listesini bu kez hatalı girdiyi atlayarak işleyelim:

▸ yakalama.cs

```
string[] notlar = { "85", "70", "doksan", "60" };
int toplam = 0;
int adet = 0;

foreach (string s in notlar)
{
    try
    {
        toplam += int.Parse(s);
        adet++;
    }
    catch (FormatException)
    {
        Console.WriteLine($"Geçersiz not atlandı: {s}");
```

```
    }
}
Console.WriteLine($"Ortalama: {toplam / adet}");
```

```
ÇIKTI
Geçersiz not atlandı: doksan
Ortalama: 71
```

`"doksan"` için `int.Parse` istisna fırlatınca `try` bloğunun geri kalanı (`adet++`) atlandı, `catch` çalıştı, döngü sonraki nota geçti. Ortalama `(85 + 70 + 60) / 3 = 71` oldu.

◆ Kritik Nokta

`try` - `catch` akışı şu kurallara uyar:

1. İstisna oluştuğu anda `try` bloğunun geri kalanı atlanır.
2. Çalışma zamanı, istisnanın türüne uyan ilk `catch` bloğunu bulup çalıştırır.
3. `catch` bitince program `try` - `catch` yapısının altından devam eder; istisnanın oluştuğu noktaya geri dönülmez.
4. Uyan `catch` yoksa istisna metodu terk edip çağıran metoda geçer. Hiçbir yerde yakalanmazsa program sonlanır.

Parantez içindeki `ex`, yakalanan istisna nesnesidir. Nesneye ihtiyacımız yoksa adı yazmayız: `catch (FormatException)`. Nesnenin en çok kullanılan üyeleri şunlardır:

Üye	Anlamı
`ex.Message`	Hatanın açıklaması (çoğu zaman İngilizce)
`ex.GetType().Name`	İstisnanın gerçek türünün adı
`ex.StackTrace`	Çağrı zinciri metni
`ex.InnerException`	Bu istisnaya yol açan asıl istisna (yoksa `null`)
`ex.ToString()`	Tür, mesaj ve çağrı zincirini birlikte veren metin

▸ **istisnaUyeleri.cs**

```
try
{
    int[] dizi = new int[3];
    dizi[3] = 1;
}
catch (Exception ex)
{
    Console.WriteLine(ex.GetType().Name);
    Console.WriteLine(ex.Message);
    Console.WriteLine(ex.InnerException == null);
}
```

```
ÇIKTI
IndexOutOfRangeException
Index was outside the bounds of the array.
True
```

Bu program Bölüm 47'de anlatılan dizi sınır denetiminin sonucunu gösteriyor. C'de aynı hata bellek bozulmasına yol açardı; burada ise tanımlı bir istisna oluşur.

> **▲ Sık Yapılan Hata · try içinde tanımlanan değişken dışarıda yaşamaz**
>
> `try { int x = int.Parse(s); } catch (FormatException) { }` yazdıktan sonra `x`'i bu yapının dışında kullanamayız; her blok gibi `try` da kendi kapsamını oluşturur. Değişkeni dışarıda kullanacaksak `try`'dan önce tanımlayıp ilk değerini veririz: `int x = 0; try { x = int.Parse(s); } ...`

53.3 İstisna Hiyerarşisi

İstisnalar da sınıftır ve kalıtım ağacına (Bölüm 50) göre düzenlenmiştir. Hepsinin atası `System.Exception`'dır. Çalışma zamanının ve temel kütüphanenin fırlattığı istisnaların çoğu onun alt sınıfı `SystemException`'dan türer. En sık karşılaştıklarımız şunlardır:

İstisna	Üst sınıfı	Ne zaman oluşur?
`FormatException`	`SystemException`	Metin sayıya çevrilemez (`int.Parse("12a")`)
`DivideByZeroException`	`ArithmeticException`	Tam sayı sıfıra bölünür
`OverflowException`	`ArithmeticException`	`checked` taşma, sığmayan sayı metni
`NullReferenceException`	`SystemException`	`null` değişkenin üyesine erişilir
`IndexOutOfRangeException`	`SystemException`	Dizi sınırı dışına erişilir
`InvalidCastException`	`SystemException`	Geçersiz tür dönüşümü
`ArgumentException`	`SystemException`	Metoda geçersiz argüman verilir
`ArgumentNullException`	`ArgumentException`	Argüman `null` olmamalıydı
`ArgumentOutOfRangeException`	`ArgumentException`	Argüman geçerli aralığın dışında
`InvalidOperationException`	`SystemException`	Nesne bu işlem için uygun durumda değil
`KeyNotFoundException`	`SystemException`	Sözlükte olmayan anahtar (Bölüm 54)
`FileNotFoundException`	`IOException`	Olmayan dosya açılır (Bölüm 57)

Zincirleri sondan başa okuyabiliriz. Bir türün üst sınıflarını `Type.BaseType` özelliğiyle dolaşıp yazdıralım:

▸ hiyerarsi.cs

```
Zincir(typeof(DivideByZeroException));
Zincir(typeof(FileNotFoundException));
Zincir(typeof(ArgumentNullException));

static void Zincir(Type tur)
{
    string yol = tur.Name;
    for (Type? ust = tur.BaseType; ust != null; ust = ust.BaseType)
    {
        yol += " -> " + ust.Name;
    }
    Console.WriteLine(yol);
}
```

```
ÇIKTI
DivideByZeroException -> ArithmeticException -> SystemException -> Exception -> Object
FileNotFoundException -> IOException -> SystemException -> Exception -> Object
ArgumentNullException -> ArgumentException -> SystemException -> Exception -> Object
```

Bir `catch (X)` bloğu `X` türündeki istisnaları ve `X`'ten türeyen bütün alt sınıfları yakalar. `catch (ArithmeticException)` hem `DivideByZeroException`'ı hem de `OverflowException`'ı yakalar. Sıradaki başlıkta bu kuralın sonucunu göreceğiz.

> **Not · FormatException ve ArgumentException**
>
> `FormatException`, `ArgumentException`'ın alt sınıfı değildir; ikisi de doğrudan `SystemException`'dan türer. Yani `catch (ArgumentException)` bir `Parse` hatasını yakalamaz. Bu tür ilişkilerden emin olmadığımızda belgeye bakmak ya da yukarıdaki gibi `BaseType` zincirini yazdırmak en güvenli yoldur.

53.4 Birden Çok catch ve Sıra

Bir `try` bloğundan farklı türde istisnalar çıkabiliyorsa her tür için ayrı `catch` yazabiliriz. Çalışma zamanı `catch` bloklarını yukarıdan aşağıya dener ve türe uyan **ilkini** çalıştırır; diğerleri atlanır.

▸ cokluCatch.cs

```
string[] girdiler = { "0", "abc", "1", "7", "2" };
foreach (string g in girdiler)
{
    Dene(g);
}

static void Dene(string girdi)
{
    int[] bolenler = { 4, 0, 25 };
    try
    {
        int i = int.Parse(girdi);
        Console.WriteLine($"{girdi} -> {100 / bolenler[i]}");
    }
    catch (FormatException)
    {
        Console.WriteLine($"{girdi} -> sayı değil");
    }
    catch (DivideByZeroException)
    {
        Console.WriteLine($"{girdi} -> sıfıra bölme");
    }
    catch (IndexOutOfRangeException)
    {
        Console.WriteLine($"{girdi} -> geçersiz indeks");
    }
}
```

```
ÇIKTI
0 -> 25
abc -> sayı değil
1 -> sıfıra bölme
7 -> geçersiz indeks
2 -> 4
```

Üst sınıf türündeki `catch` alt sınıfları da yakaladığı için sıra önemlidir: önce özel (alt sınıf), sonra genel (üst sınıf) yazılır. Bölüm 34'te C++ için de aynı sıra kuralını görmüştük; C++ derleyicileri bu durumu en çok uyarıyla bildirir. C# derleyicisi ise ulaşılamayan `catch`'i **hata** sayar:

catchSirasi.cs

```
try
{
    int b = 0;
    Console.WriteLine(10 / b);
}
catch (Exception)
{
    Console.WriteLine("Genel hata");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Sıfıra bölme");
}
```

DERLEYİCİ ÇIKTISI

```
catchSirasi.cs(10,8): error CS0160: A previous catch clause already catches all exceptions of
this or of a super type ('Exception')
```

Kural kısadır: bir `catch`'in kendisinden önce, kendi üst sınıfını yakalayan bir `catch` varsa derleme hatası oluşur. `catch (Exception)` her zaman en sona yazılır. Kardeş türler (`FormatException` ile `NullReferenceException` gibi) hangi sırada yazılırsa yazılsın sorun çıkmaz.

C++'taki `catch (...)` karşılığı C#'ta türsüz `catch` bloğudur. Pratikte `catch (Exception)` ile aynı işi görür:

```
try { RiskliIs(); }
catch { Console.WriteLine("Bir şeyler ters gitti"); }
```

> **▲ Sık Yapılan Hata · Boş catch ve fazla geniş catch**
>
> `catch (Exception) { }` hatayı sessizce yutar; program yanlış durumda çalışmayı sürdürür ve hata aylar sonra başka bir yerde ortaya çıkar. Yalnızca yönetebileceğimiz türleri yakalamalıyız. Beklemediğimiz bir istisnayı yakalayıp devam etmek yerine, yakalanmadan yukarıya çıkmasına izin vermek çoğu zaman daha doğrudur.

53.5 finally: Her Durumda Çalışan Blok

`try-catch`'ten sonra isteğe bağlı bir `finally` bloğu yazılabilir. `finally`, istisna olsa da olmasa da, yakalansa da yakalanmasa da çalışır; `try` ya da `catch` içinde `return` bulunsa bile çalışır. Açılan bir dosyayı kapatmak gibi "ne olursa olsun yapılmalı" işler için kullanılır. C++'ta bu işi yıkıcılar (RAII) görürdü; C#'ta kaynak temizliğini `finally` ve ilerideki bölümde göreceğimiz `using` üstlenir.

finallyOrnek.cs

```
Console.WriteLine($"Sonuç: {Bol(10, 2)}");
Console.WriteLine($"Sonuç: {Bol(10, 0)}");

static int Bol(int a, int b)
{
    try
    {
```

```
        Console.WriteLine($"  try: {a}/{b}");
        return a / b;
    }
    catch (DivideByZeroException)
    {
        Console.WriteLine("  catch: sıfıra bölme");
        return -1;
    }
    finally
    {
        Console.WriteLine("  finally çalıştı");
    }
}
```

```
ÇIKTI
  try: 10/2
  finally çalıştı
Sonuç: 5
  try: 10/0
  catch: sıfıra bölme
  finally çalıştı
Sonuç: -1
```

İlk çağrıda `return a / b;` çalıştı ama metot hemen dönmedi. Dönüş değeri (5) bir kenara alındı, önce `finally` çalıştı, sonra metot 5'i döndürdü. `Sonuç:` satırının `finally`'den sonra çıkmasının nedeni budur.

◆ Kritik Nokta

`finally` ile ilgili bilinmesi gereken kurallar:

- `try` içinde `return x;` varsa `x`'in değeri `finally`'den **önce** alınır. `finally` içinde yerel değişkeni değiştirmek dönen değeri değiştirmez.
- `finally` içinden `return`, `break` ya da `continue` ile çıkılamaz; bu derleme hatasıdır (`CS0157`).
- `catch` olmadan `try` - `finally` yazılabilir; tek başına `try` yazılamaz.
- `Environment.Exit` ile program kapatılırsa ya da süreç `Environment.FailFast` gibi yollarla ani biçimde sonlanırsa `finally` çalışmaz.

İlk maddeyi küçük bir programla görelim:

▸ **finallyDeger.cs**

```
Console.WriteLine(Say());

static int Say()
{
    int x = 1;
    try
    {
        return x;
    }
    finally
    {
        x = 99;
    }
}
```

```
ÇIKTI
1
```

`return x;` çalıştığında dönecek değer olan 1 saklanmıştı; `finally` içindeki `x = 99;` yalnızca yerel değişkeni değiştirdi.

53.6 throw ile İstisna Fırlatmak

Şimdiye kadar istisnaları .NET çalışma zamanı fırlattı. `throw` ile biz de fırlatabiliriz. Genellikle bir metoda ya da özelliğe geçersiz değer geldiğinde, yanlış veriyle devam etmek yerine işi hemen durdurmak için kullanılır. Aşağıdaki sınıfın `Puan` özelliği geçersiz değeri reddediyor:

throwOrnek.cs

```
Ogrenci o = new Ogrenci("Elif");
foreach (int p in new int[] { 85, 120, -5, 70 })
{
    try
    {
        o.Puan = p;
        Console.WriteLine($"{o.Ad} puanı {o.Puan} olarak kaydedildi.");
    }
    catch (ArgumentOutOfRangeException ex)
    {
        Console.WriteLine($"Reddedildi: {ex.Message}");
    }
}

class Ogrenci
{
    private int _puan;
    public string Ad { get; }

    public Ogrenci(string ad)
    {
        Ad = ad;
    }

    public int Puan
    {
        get { return _puan; }
        set
        {
            if (value < 0 || value > 100)
            {
                throw new ArgumentOutOfRangeException(nameof(value),
                    "Puan 0-100 arası olmalı");
            }
            _puan = value;
        }
    }
}
```

```
ÇIKTI
Elif puanı 85 olarak kaydedildi.
Reddedildi: Puan 0-100 arası olmalı (Parameter 'value')
Reddedildi: Puan 0-100 arası olmalı (Parameter 'value')
Elif puanı 70 olarak kaydedildi.
```

`throw new ArgumentOutOfRangeException(...)` satırında önce `new` ile bir istisna nesnesi oluşturulur, sonra `throw` onu fırlatır. `throw`'dan sonraki satırlar çalışmaz; blok tıpkı `return`'de olduğu gibi o anda terk edilir. Böylece `_puan` alanına geçersiz değer hiçbir zaman yazılmaz. `nameof(value)` hatalı parametrenin adını derleme zamanında metne çevirir; ad değişirse metin de değişir. Mesajın sonundaki `(Parameter 'value')` kısmını `ArgumentException` ailesi kendisi ekler.

Geçersiz argüman için `ArgumentException` ailesini, nesnenin durumu o işleme uygun değilse (kapalı hesaptan para çekmek gibi) `InvalidOperationException`'ı seçeriz. Yalnızca `Exception`'dan türeyen nesneler fırlatılabilir. C++'ta `throw 42;` yazabiliyorduk; C#'ta bu derlenmez.

> **◆ Kritik Nokta**
>
> C#'ta `throws` yoktur ve "denetlenen istisna" (checked exception) kavramı da yoktur. Bir metodun hangi istisnaları fırlatabileceğini imzada bildirmeyiz, derleyici bizi bir istisnayı yakalamaya zorlamaz. C++'ın eski `throw(...)` belirtimi ve `noexcept` karşılığı da yoktur. Fırlatılabilecek istisnaları yalnızca belgelendiririz; XML belge yorumundaki `<exception>` etiketi bunun içindir ve derleyici onu denetlemez.

```
/// <summary>Puanı kaydeder.</summary>
/// <exception cref="ArgumentOutOfRangeException">Puan 0-100 dışındaysa.</exception>
public void PuanKaydet(int deger) { /* ... */ }
```

throw ifadesi ve ThrowIfNull

C# 7'den beri `throw`, bazı yerlerde ifade olarak da yazılabilir. En yaygın kullanımı `??` ve `?:` ile birlikte bir değerin zorunlu olduğunu belirtmektir. .NET 6'dan beri argüman denetimi için hazır yardımcılar da vardır:

▸ throwIfade.cs

```
Console.WriteLine(Selamla("Deniz"));
Dene(null);
Dene("   ");

static string Selamla(string ad)
{
    ArgumentNullException.ThrowIfNull(ad);
    return $"Merhaba {ad}";
}

static void Dene(string? ad)
{
    try
    {
        string temiz = ad?.Trim() ?? throw new ArgumentNullException(nameof(ad));
        if (temiz.Length == 0)
        {
            throw new ArgumentException("Ad boş olamaz", nameof(ad));
        }
        Console.WriteLine(temiz);
    }
    catch (ArgumentNullException ex)
    {
        Console.WriteLine($"null geldi: {ex.ParamName}");
    }
    catch (ArgumentException ex)
    {
```

```
        Console.WriteLine($"geçersiz: {ex.ParamName}");
    }
}
```

```
ÇIKTI
Merhaba Deniz
null geldi: ad
geçersiz: ad
```

`catch` sırasına dikkat edelim: `ArgumentNullException`, `ArgumentException`'ın alt sınıfı olduğu için önce yazıldı.

53.7 throw; ile throw ex; Farkı

Bir `catch` içinde istisnayı kısmen işleyip (günlüğe yazmak gibi) yukarıya yeniden fırlatmak isteyebiliriz. İki yazım vardır ve aynı değildir:

- `throw;` istisnayı olduğu gibi, çağrı zinciri korunarak yeniden fırlatır.
- `throw ex;` yakalanan nesneyi yeni bir istisna gibi fırlatır ve çağrı zincirini bu satırdan başlatır; hatanın gerçek yeri kaybolur.

C++'ta da `throw;` aynı nesneyi yeniden fırlatır; `throw e;` ise nesnenin bir kopyasını (üstelik `catch` parametresi taban sınıf türündeyse dilimlenmiş halini) fırlatırdı. C#'ta nesne kopyalanmaz, çünkü istisnalar referans tipidir; kaybolan şey nesnenin içindeki çağrı zinciri kaydıdır.

▸ yenidenFirlat.cs

```
foreach (bool koru in new bool[] { true, false })
{
    try
    {
        Aktar(koru);
    }
    catch (DivideByZeroException ex)
    {
        bool izde = (ex.StackTrace ?? "").Contains("Bol");
        Console.WriteLine($"koru={koru} izde Bol var mı: {izde}");
    }
}

static int Bol(int a, int b)
{
    return a / b;
}

static int Aktar(bool koru)
{
    try
    {
        return Bol(1, 0);
    }
    catch (DivideByZeroException ex)
    {
        Console.WriteLine("  günlüğe yazıldı");
        if (koru)
        {
            throw;
        }
```

```
        throw ex;
    }
}
```

```
ÇIKTI
  günlüğe yazıldı
koru=True izde Bol var mı: True
  günlüğe yazıldı
koru=False izde Bol var mı: False
```

Hata aslında `Bol` metodunda oluştu. `throw;` ile iz `Bol`'u içerir; `throw ex;` ile iz `Aktar`'da başlar ve hata ayıklayan kişi sorunun gerçek yerini göremez.

> **▲ Sık Yapılan Hata · throw ex; yazmak**
>
> Yeniden fırlatırken `throw ex;` değil `throw;` yazmalıyız. Derleyici ikisini de kabul eder; fark yalnızca hata ayıklarken ortaya çıkar.

İç istisna: başka türe çevirmek

Bazen alttaki katmanın istisnasını kendi katmanımızın anlamlı bir istisnasına çevirmek isteriz. Yeni istisnayı fırlatırken eskisini ikinci argüman olarak verirsek eski istisna yenisinin `InnerException` özelliğinde saklanır ve hiçbir bilgi kaybolmaz:

▸ icIstisna.cs

```
try
{
    Console.WriteLine(PortOku("8080"));
    Console.WriteLine(PortOku("80a"));
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(ex.Message);
    Console.WriteLine($"Neden: {ex.InnerException?.GetType().Name}");
    Console.WriteLine($"Ayrıntı: {ex.InnerException?.Message}");
}

static int PortOku(string metin)
{
    try
    {
        return int.Parse(metin);
    }
    catch (FormatException ex)
    {
        throw new InvalidOperationException("Ayar dosyası okunamadı", ex);
    }
}
```

```
ÇIKTI
8080
Ayar dosyası okunamadı
Neden: FormatException
Ayrıntı: The input string '80a' was not in a correct format.
```

C++11'in `std::nested_exception` ve `std::throw_with_nested` yapıları aynı amaca hizmet eder; C#'ta bu özellik her istisna nesnesinde hazır bulunur.

53.8 Kendi İstisna Sınıfımız ve when Süzgeci

.NET'in hazır istisnaları her durumu anlatmaya yetmez. "Bakiye yetersiz", "stok bitti" gibi programa özgü hatalar için kendi istisna sınıfımızı yazarız. Kural basittir: `Exception`'dan türetiriz, mesajı `: base(mesaj)` ile üst sınıfa iletiriz ve sınıf adını `Exception` ile bitiririz. Kalıtım ve `base` çağrısı Bölüm 50'deki yapıcı zincirinin aynısıdır.

İkinci `catch` bloğundaki `when (koşul)` ifadesine de dikkat edelim. `when` ile eklenen koşul doğru değilse blok, sanki türe hiç uymamış gibi atlanır. Buna **istisna süzgeci** (exception filter) denir.

ozelIstisna.cs

```
decimal kasa = 500;
foreach (decimal tutar in new decimal[] { 200, 400, 900, 300 })
{
    try
    {
        kasa = Cek(kasa, tutar);
        Console.WriteLine($"{tutar} TL çekildi, kalan: {kasa} TL");
    }
    catch (YetersizBakiyeException ex) when (ex.Eksik <= 100)
    {
        Console.WriteLine($"Az eksik ({ex.Eksik} TL): {ex.Message}");
    }
    catch (YetersizBakiyeException ex)
    {
        Console.WriteLine($"Çok eksik: {ex.Message}");
    }
}

static decimal Cek(decimal bakiye, decimal tutar)
{
    if (tutar > bakiye)
    {
        throw new YetersizBakiyeException(bakiye, tutar);
    }
    return bakiye - tutar;
}

class YetersizBakiyeException : Exception
{
    public decimal Eksik { get; }

    public YetersizBakiyeException(decimal bakiye, decimal istenen)
        : base($"Yetersiz bakiye: {bakiye} TL var, {istenen} TL istendi")
    {
        Eksik = istenen - bakiye;
    }
}
```

ÇIKTI

```
200 TL çekildi, kalan: 300 TL
Az eksik (100 TL): Yetersiz bakiye: 300 TL var, 400 TL istendi
Çok eksik: Yetersiz bakiye: 300 TL var, 900 TL istendi
300 TL çekildi, kalan: 0 TL
```

400 TL çekiminde eksik 100 olduğu için `when` koşulu doğru çıktı ve ilk `catch` çalıştı. 900 TL çekiminde eksik 600 olduğundan koşul yanlıştı; ilk `catch` atlandı ve ikincisi çalıştı. Özel istisna sıradan bir sınıf olduğu için özellikler taşıyabilir. Böylece `catch` yalnızca mesajı değil, hatayla ilgili veriyi de kullanabilir.

Özel istisna sınıflarında, .NET'in kendi sınıflarını örnek alarak üç yapıcı sağlamak gelenektir:

```
class VeriHatasiException : Exception
{
    public VeriHatasiException() { }
    public VeriHatasiException(string mesaj) : base(mesaj) { }
    public VeriHatasiException(string mesaj, Exception ic) : base(mesaj, ic) { }
}
```

İpucu · Ne zaman kendi istisnamızı yazmalı?

Hazır tür durumu zaten anlatıyorsa onu kullanırız: geçersiz argüman için `ArgumentException`, uygunsuz durum için `InvalidOperationException`. Çağıranın türe bakarak farklı davranması gerekiyorsa (yetersiz bakiyede tutar sor, kilitli hesapta uyar) ya da hatayla birlikte veri taşımamız gerekiyorsa kendi sınıfımızı yazarız. Her küçük hata için yeni sınıf üretmek gereksizdir.

Tarihçe · Süzgeç ve throw ifadesi

İstisna süzgeçleri (`when`) C# 6 ile (2015), `throw` ifadesi C# 7 ile geldi. Süzgeç yalnızca bir kısayol değildir: koşul yanlışsa istisna yakalanmamış sayılır ve çağrı zinciri henüz çözülmediği için hata ayıklayıcı hatanın gerçek yerini gösterebilir. `catch` içinde `if` yazıp `throw;` ile yeniden fırlatmak aynı sonucu vermez.

53.9 using ve IDisposable

Dosya, ağ bağlantısı, veritabanı bağlantısı gibi bellek dışı kaynaklar işin sonunda mutlaka serbest bırakılmalıdır. Çöp toplayıcı (Bölüm 41) yalnızca belleği yönetir ve ne zaman çalışacağı belli değildir. Bu işi `finally` ile yapmak uzun ve hataya açıktır:

```
var r = new Kaynak("Rapor");
try
{
    r.Kullan();
}
finally
{
    r.Dispose();      // hata olsa da olmasa da kaynak bırakılır
}
```

`using` deyimi aynı kalıbı derleyiciye yazdırır. Kaynak `using` ile açılır, blok biterken (istisna olsa bile) `Dispose()` metodu çağrılır. Bunun için sınıfın `IDisposable` arayüzünü uygulaması gerekir. Arayüzleri Bölüm 51'de ayrıntılı işleyeceğiz; şimdilik `IDisposable`'ın tek bir metodu olduğunu bilmemiz yeter: `void Dispose()`.

Tanım: IDisposable

Bellek dışı bir kaynağı elinde tutan sınıfların uyguladığı arayüzdür. Tek üyesi `Dispose()` metodudur; kaynağı serbest bırakma işini yapar. `using` deyimi, blok bittiğinde bu metodu otomatik çağırır.

`Dispose`'un ne zaman çağrıldığını görmek için kendi sınıfımızı yazalım:

kaynakOrnek.cs

```
try
{
    using (Kaynak a = new Kaynak("Dosya"))
    using (Kaynak b = new Kaynak("Bağlantı"))
    {
        a.Kullan();
        b.Kullan();
        throw new InvalidOperationException("işlem yarıda kaldı");
    }
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"catch: {ex.Message}");
}
finally
{
    Console.WriteLine("finally");
}

Rapor();

static void Rapor()
{
    using var r = new Kaynak("Rapor");
    r.Kullan();
    Console.WriteLine("Rapor metodu bitiyor");
}

class Kaynak : IDisposable
{
    private readonly string _ad;

    public Kaynak(string ad)
    {
        _ad = ad;
        Console.WriteLine($"{ad} açıldı");
    }

    public void Kullan() { Console.WriteLine($"{_ad} kullanılıyor"); }

    public void Dispose() { Console.WriteLine($"{_ad} kapandı"); }
}
```

ÇIKTI

```
Dosya açıldı
Bağlantı açıldı
Dosya kullanılıyor
Bağlantı kullanılıyor
Bağlantı kapandı
Dosya kapandı
catch: işlem yarıda kaldı
finally
Rapor açıldı
Rapor kullanılıyor
Rapor metodu bitiyor
Rapor kapandı
```

Çıktıdan üç kural çıkarırız:

1. Kaynaklar açıldıkları sıranın tersine bırakılır: önce `Bağlantı`, sonra `Dosya`. C++'ta yıkıcıların çalışma sırası da böyleydi.
2. İstisna olsa bile `Dispose` çalışır ve ilgili `catch` bloğundan **önce** çalışır; bu, C++'taki yığın çözülmesinin karşılığıdır.
3. `using var r = ...;` biçimi (C# 8) süslü parantez gerektirmez; kaynak, içinde bulunduğu bloğun sonunda bırakılır.

> **▲ Sık Yapılan Hata · using'i unutmak**
>
> C++'ta yıkıcı kapsam bitince kendiliğinden çalışır; kaynağı bırakmak için bir şey yazmamız gerekmez. C#'ta `using` yazmayı hatırlamak bizim sorumluluğumuzdadır. Derleyici unuttuğumuzda hata vermez, dosya ya da bağlantı açık kalır. `StreamReader` gibi dosya sınıfları da `IDisposable`'dır ve Bölüm 57'de bu kalıbı sürekli kullanacağız.

53.10 İstisna mı, Dönüş Değeri mi?

İstisna fırlatmak ve yakalamak, sıradan bir dönüş değerine göre çok daha yavaştır. Üstelik istisnaları akış kontrolü gibi kullanmak kodu okumayı güçleştirir. Bölüm 34'te C++ için vardığımız sonuç C#'ta da geçerlidir: **beklenen** ve sık görülen durumlar için dönüş değeri, **beklenmeyen** durumlar için istisna kullanırız.

Kullanıcı girdisi bunun tipik örneğidir. Yanlış yazılmış bir sayı sıradan bir olaydır, istisnai bir durum değildir. Bölüm 42'de tanıdığımız `TryParse` deseni (`bool` döndürür, sonucu `out` parametresiyle verir) bu yüzden vardır. İki yaklaşımı yan yana görelim:

girdiDogrulama.cs

```
string[] girdiler = { "25", "yirmi", "130", "-4", "99999999999" };

foreach (string g in girdiler)
{
    // 1. yol: istisna ile
    string sonuc1;
    try
    {
        int yas = int.Parse(g);
        sonuc1 = (yas >= 0 && yas <= 120) ? "geçerli" : "aralık dışı";
    }
    catch (FormatException)
    {
        sonuc1 = "sayı değil";
    }
    catch (OverflowException)
    {
        sonuc1 = "çok büyük";
    }

    // 2. yol: TryParse ile
    string sonuc2;
    if (!int.TryParse(g, out int deger))
    {
        sonuc2 = "geçersiz metin";
    }
    else
    {
```

```
        sonuc2 = (deger >= 0 && deger <= 120) ? "geçerli" : "aralık dışı";
    }
    Console.WriteLine($"{g,-12} {sonuc1,-12} {sonuc2}");
}
```

```
ÇIKTI
25           geçerli      geçerli
yirmi        sayı değil   geçersiz metin
130          aralık dışı  aralık dışı
-4           aralık dışı  aralık dışı
99999999999  çok büyük    geçersiz metin
```

`TryParse` biçim hatasıyla taşmayı ayırt etmez, hepsi için `false` döner. Ayrım gerekiyorsa `Parse` ve `catch` kullanılabilir; çoğu uygulamada "geçersiz girdi" demek yeter.

Aynı mantık başka yerlerde de geçerlidir. Sözlükte anahtar aramak için `TryGetValue` (Bölüm 54), `null` olabilecek bir değer için önce `null` denetimi, bir dosyanın varlığı için `File.Exists` (Bölüm 57) tercih edilir. Ancak yalnızca önceden denetim yapmak her zaman yeterli olmaz; dosya, kontrol edildiği an ile açıldığı an arasında silinmiş olabilir. Çevreye bağlı hatalarda istisna yakalamak yine de gerekir.

◆ Kritik Nokta

İstisna kullanımının iyi uygulamaları:

- Sık görülen, beklenen durumları (kullanıcı hatası, aramada bulunamama) istisnayla değil `TryParse`, `bool` ya da `null` dönüşüyle bildirelim.
- Yalnızca anlamlı bir şey yapabileceğimiz istisnaları yakalayalım. `catch (Exception)` yalnızca programın en dış katmanında, hatayı günlüğe yazıp düzgünce kapanmak için uygundur.
- Yeniden fırlatırken `throw;` yazalım, `throw ex;` yazmayalım. Türü değiştirirken asıl istisnayı `InnerException` olarak ekleyelim.
- Kaynakları `using` ile bırakalım; `finally` yalnızca `IDisposable` olmayan temizlik işleri için kalsın.
- Genel hata için `Exception`'ın kendisini fırlatmayalım; anlamlı bir alt tür seçelim.

53.11 Üç Dilde Hata Yönetimi

Üç Dilde · C dönüş kodları, C++ RAII ve istisna, C# finally ve using *C · C++ · C#*

C'de hata, dönüş değeriyle bildirilir ve her çağrıdan sonra denetlenmesi gerekir (Bölüm 17, Bölüm 34). Hiçbir mekanizma denetimi zorlamaz:

```c
FILE* f = fopen("veri.txt", "r");
if (f == NULL) {
    perror("veri.txt");
    return 1;
}
/* ... */
fclose(f);                  /* her çıkış yolunda elle */
```

C++'ta hata `throw` ile fırlatılır, temizliği yıkıcı üstlenir. Bu yüzden dilde `finally` yoktur:

```cpp
try {
    std::ifstream dosya("veri.txt");     // yıkıcı kapatır
    // ...
```

```
} catch (const std::exception& e) {
    std::cerr << e.what() << '\n';
}
```

C#'ta hata yine fırlatılır, ama temizlik için yıkıcı yerine `using` ve `finally` vardır; çağıran, istisna nesnesinden çağrı zincirini de okuyabilir:

```
try
{
    using var okuyucu = new StreamReader("veri.txt");   // Dispose çağrılır
    // ...
}
catch (FileNotFoundException ex)
{
    Console.WriteLine(ex.Message);
}
```

	C	C++	C#
Hata bildirimi	Dönüş değeri, `errno`	`throw`	`throw`
Fırlatılabilen	-	Her tip (öneri: `std::exception`)	Yalnızca `Exception` türevleri
Her şeyi yakalama	-	`catch (...)`	`catch` ya da `catch (Exception)`
Temizlik	Elle	Yıkıcı (RAII), otomatik	`using` / `finally`, programcı yazar
Yeniden fırlatma	-	`throw;`	`throw;` (`throw ex;` izi bozar)
Koşullu yakalama	-	Yok	`catch ... when (koşul)`
Çağrı zinciri	Yok	Standartta yok	Her istisnada (`StackTrace`)
İç içe hata	-	`nested_exception`	`InnerException`
Belirtim	-	`noexcept`	Yok (`throws` da yok)
Tam sayı sıfıra bölme	Tanımsız davranış	Tanımsız davranış	`DivideByZeroException`

Bölüm Özeti

- İstisna, normal akışı bozan ve nesne olarak temsil edilen bir olaydır. Hiçbir yerde yakalanmayan istisna programı sonlandırır ve tür, mesaj ve çağrı zincirinden oluşan bir rapor yazdırır.
- `try` istisna çıkabilecek kodu, `catch` istisnayı ele alan kodu içerir. İstisna oluşunca `try`'ın geri kalanı atlanır ve program `try`-`catch` yapısının altından devam eder.
- Bütün istisnalar `Exception`'dan türer. `catch (X)` hem `X`'i hem alt sınıflarını yakalar. `FormatException`, `DivideByZeroException`, `NullReferenceException`, `IndexOutOfRangeException`, `ArgumentException` ailesi ve `InvalidOperationException` en sık karşılaşılanlardır.
- Birden çok `catch` yukarıdan aşağıya denenir ve uyan ilki çalışır. Üst sınıfı önce yakalayan bir `catch` alt sınıfı gölgelerse derleme hatası oluşur (`CS0160`). `catch (Exception)` en sona yazılır.
- `finally` istisna olsa da olmasa da, `return` bulunsa bile çalışır. `return` değeri `finally`'den önce saklanır. `finally` içinden `return`, `break` ya da `continue` ile çıkılamaz.
- `throw` ile `Exception` türevi bir nesne fırlatılır; C++'taki gibi `int` ya da `string` fırlatılamaz. C#'ta `throws` belirtimi ve denetlenen istisna yoktur.

- `throw;` istisnayı çağrı zinciriyle birlikte yeniden fırlatır; `throw ex;` zinciri o satırdan başlatır ve hatanın gerçek yerini siler.
- İstisnayı başka türe çevirirken asıl istisna, yenisinin ikinci argümanı olarak verilir ve `InnerException` özelliğinde saklanır.
- Kendi istisna sınıfımız `Exception`'dan türer, adı `Exception` ile biter ve gerekirse hatayla ilgili veriyi özelliklerinde taşır.
- `catch ... when (koşul)` istisna süzgecidir: koşul yanlışsa blok atlanır.
- `ArgumentNullException.ThrowIfNull` ve `x ?? throw new ...` argüman denetimini kısaltır.
- `using` deyimi, `IDisposable` uygulayan nesnenin `Dispose()` metodunu blok bitince, istisna olsa bile çağırır. Kaynaklar açılış sırasının tersine bırakılır. `using var` biçimi bloğun sonunda bırakır.
- C++'ta yıkıcı kaynağı kendiliğinden bırakır; C#'ta `using` yazmak programcının sorumluluğundadır.
- İstisnalar beklenmeyen durumlar içindir. Sık görülen durumlar için `TryParse` gibi dönüş değeri kullanan yöntemler hem daha okunur hem daha hızlıdır.
- Yalnızca anlamlı bir şey yapabileceğimiz istisnaları yakalarız; hatayı sessizce yutan boş `catch` bloğundan kaçınırız.

Alıştırmalar

1. Aşağıdakilerden hangisi C#'ta geçerli bir `throw` deyimidir?
   A) `throw "hata";`
   B) `throw 42;`
   C) `throw new ArgumentException("geçersiz");`
   D) `throw true;`
   E) `throw 'x';`
2. Aşağıdaki `catch` dizilişlerinden hangisi derlenir? (Her biri aynı `try` bloğunun ardına yazılmıştır.)
   A) `catch (Exception)` ardından `catch (FormatException)`
   B) `catch (ArgumentException)` ardından `catch (ArgumentNullException)`
   C) `catch (FormatException)` ardından `catch (ArgumentException)`
   D) `catch (SystemException)` ardından `catch (DivideByZeroException)`
   E) `catch (IOException)` ardından `catch (FileNotFoundException)`
3. Aşağıdaki program ekrana ne yazdırır?

```
Test(0);
Test(1);
Test(2);

static void Test(int n)
{
    try
    {
        Console.Write("A ");
        if (n == 1) throw new InvalidOperationException();
        if (n == 2) throw new FormatException();
        Console.Write("B ");
    }
    catch (FormatException)
    {
        Console.Write("C ");
```

```
    }
    catch (Exception)
    {
        Console.Write("D ");
    }
    finally
    {
        Console.Write("E ");
    }
    Console.WriteLine("F");
}
```

4. `finally` bloğu hakkında hangisi doğrudur?
   A) Yalnızca istisna oluşursa çalışır.
   B) `try` içinde `return` varsa çalışmaz.
   C) İçinden `return` ile çıkılabilir.
   D) `catch` bloğu olmadan `try` ile birlikte yazılabilir.
   E) Bir `try` yapısında birden çok `finally` bulunabilir.

5. `throw;` ve `throw ex;` hakkında hangisi doğrudur?
   A) `throw ex;` çağrı zincirini korur, `throw;` sıfırlar.
   B) İkisi arasında hiçbir fark yoktur.
   C) `throw;` yalnızca `catch` bloğu içinde yazılabilir ve çağrı zincirini korur.
   D) `throw;` `try` bloğunda da kullanılabilir.
   E) `throw ex;` derleme hatasıdır.

6. Aşağıdaki program ekrana ne yazdırır?

```
Console.WriteLine(F());

static int F()
{
    int n = 0;
    try
    {
        n = 1;
        return n;
    }
    catch
    {
        n = 2;
        return n;
    }
    finally
    {
        n = 3;
        Console.WriteLine("n=" + n);
    }
}
```

7. Aşağıdaki kod parçası çalıştırılınca ne olur?

```
double pay = 1.0;
int payda = 0;
double d = pay / payda;
Console.WriteLine(d);
```

A) `DivideByZeroException` fırlatılır.
B) `0` yazılır.
C) `Infinity` yazılır.
D) `NaN` yazılır.
E) Derleme hatası oluşur.

8. Aşağıdaki program ekrana ne yazdırır?

```
using (var a = new Bel("X"))
{
    using var b = new Bel("Y");
    Console.WriteLine("içeride");
}
Console.WriteLine("dışarıda");

class Bel : IDisposable
{
    private readonly string _ad;
    public Bel(string ad) { _ad = ad; Console.WriteLine(ad + " açıldı"); }
    public void Dispose() { Console.WriteLine(_ad + " kapandı"); }
}
```

9. Aşağıdaki kodun derlenmemesinin nedeni nedir? Nasıl düzeltilir?

```
try
{
    int[] dizi = new int[2];
    dizi[5] = 1;
}
catch (Exception)
{
    Console.WriteLine("Genel hata");
}
catch (IndexOutOfRangeException)
{
    Console.WriteLine("İndeks hatası");
}
```

10. Aşağıdaki metot neden derlenmez? Düzeltmenin iki yolunu belirtin.

```
static int Oku(string s)
{
    try
    {
        return int.Parse(s);
    }
    catch (FormatException)
    {
        Console.WriteLine("hata");
    }
}
```

11. Kullanıcıdan yaşını okuyup geçersizse yeniden soran bir program için en uygun yaklaşım hangisidir?
A) `int.Parse` kullanıp `catch (Exception) { }` ile hatayı yutmak
B) `int.Parse` kullanıp hata olursa programı sonlandırmak
C) `int.TryParse` kullanıp dönen `bool` değerine göre yeniden sormak

D) Her karakteri tek tek `try` bloğuna almak
E) Girdiyi hiç doğrulamamak

12. Aşağıdaki program ekrana ne yazdırır?

```
foreach (int k in new[] { 1, 5, 10 })
{
    try
    {
        if (k > 3) throw new ArgumentException("büyük: " + k);
        Console.WriteLine("ok " + k);
    }
    catch (ArgumentException ex) when (k > 7)
    {
        Console.WriteLine("A " + ex.Message);
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine("B " + ex.Message);
    }
}
```

13. İki tam sayıyı bölen `static int? GuvenliBol(int a, int b)` metodunu yazın. Bölen sıfırsa `null` döndürsün; bunu `DivideByZeroException` yakalayarak yapsın. `GuvenliBol(10, 2)` ve `GuvenliBol(10, 0)` sonuçlarını yazdıran bir çağrı ekleyin.

14. `Urun` sınıfı yazın: `Ad`, `Stok` özellikleri ve `Cikar(int adet)` metodu olsun. Stoktan fazlası istenirse eksik miktarı `Eksik` özelliğinde taşıyan `StokYetersizException` fırlatılsın. Stoğu 5 olan bir ürün için sırayla 2, 4 ve 3 adet çıkarmayı deneyin; hataları yakalayıp mesajı ve eksik miktarı yazdırın.

15. `IDisposable` uygulayan `Gunluk` sınıfı yazın: yapıcısı "Günlük açıldı", `Dispose` metodu "Günlük kapandı" yazsın. Bir metot içinde `using var` ile bu sınıfı açın, bir satır kaydedin ve ardından `InvalidOperationException` fırlatın. Çağıran kodda istisnayı yakalayıp mesajı yazdırın ve çıktıdaki sırayı açıklayın.

16. `"10 / 4"`, `"8 / 0"`, `"7 x 2"`, `"on + 5"` gibi metin işlemlerini `+` ve `/` için hesaplayan bir `Hesapla` metodu yazın. Sayı okunamazsa ya da sıfıra bölme olursa `FormatException` ve `DivideByZeroException`'ı yakalayıp asıl istisnayı iç istisna olarak ekleyerek `InvalidOperationException` fırlatın. Bilinmeyen işleci de aynı türle bildirin. Çağıran kod her işlem için sonucu ya da hata mesajı ve iç istisna türünü yazdırsın.

BÖLÜM 54

Koleksiyonlar ve Generics

BU BÖLÜMDE

- Dizilerin sınırını ve koleksiyon (collection) kavramını; tipsiz `ArrayList` ile tip güvenli `List<T>` arasındaki farkı
- `List<T>` ile ekleme, silme, arama ve sıralamayı; `Count` ile `Capacity` ilişkisini
- `Dictionary<TKey,TValue>`, `HashSet<T>`, `Queue<T>`, `Stack<T>`, `LinkedList<T>` ve sıralı koleksiyonları
- Koleksiyonu dolaşırken değiştirmenin neden istisna verdiğini ve doğru yöntemleri
- Kendi generic sınıf ve metotlarımızı yazmayı; `where` kısıtlarını (`class`, `struct`, `new()`, arayüz)
- Doğru koleksiyonu işlem maliyetlerine bakarak seçmeyi
- STL kapsayıcıları ile C# koleksiyonlarının eşleşmesini

Bölüm 47'de gördüğümüz diziler hızlıdır ve basittir, ama boyutları oluşturulurken belirlenir ve değişmez. Gerçek programlarda eleman sayısını önceden bilmeyiz: kuyruğa yeni müşteriler gelir, sepete ürün eklenir, sözlüğe yeni kelimeler girer. Bu işleri kendi başımıza yapmak için C'de Bölüm 19'da bağlı liste ve yığın yazmıştık; C++'ta ise Bölüm 35'te hazır kapsayıcıları (container) kullandık.

C#'ta bu hazır yapılara **koleksiyon** (collection) denir ve `System.Collections.Generic` ad alanında bulunur. Proje şablonunun örtük `using` satırları sayesinde bu ad alanını ayrıca yazmamız gerekmez. Bölümde C++ kapsayıcılarını yeniden anlatmayacağız; Bölüm 35'e atıf yapıp farkları vurgulayacağız. Bölümün ikinci yarısında koleksiyonların dayandığı **generics** özelliğini ele alıp kendi generic tiplerimizi yazacağız.

54.1 Dizilerden Koleksiyonlara

Bir dizinin boyutunu büyütmek için yeni, daha büyük bir dizi oluşturup elemanları kopyalamamız gerekir. Bölüm 47'de `Array.Resize` bunu bizim yerimize yapıyordu. Koleksiyonlar bu işi içlerinde yapar:

diziyeKarsiListe.cs

```
string[] dizi = { "Ayşe", "Can", "Elif" };
Array.Resize(ref dizi, dizi.Length + 1);   // yeni dizi + kopyalama
dizi[3] = "Mert";
Console.WriteLine(string.Join(", ", dizi));

List<string> liste = new();                // boş liste
liste.Add("Ayşe");
liste.Add("Can");
liste.Add("Elif");
```

```
liste.Add("Mert");                          // büyümeyi liste kendisi yönetir
Console.WriteLine(string.Join(", ", liste));
Console.WriteLine($"Eleman sayısı: {liste.Count}");
```

```
ÇIKTI
Ayşe, Can, Elif, Mert
Ayşe, Can, Elif, Mert
Eleman sayısı: 4
```

`new()` yazımı, tipi sol taraftan çıkarır (hedefe göre tip belirlemeli `new`, C# 9). `List<string> liste = new();` ile `new List<string>()` aynı şeyi yapar. `List<T>`, C++'taki `vector<T>`'nin karşılığıdır: arkada bir dizi tutar, dolunca daha büyük bir dizi ayırıp elemanları taşır.

Tipsiz koleksiyonların sorunu

> *Tarihçe · ArrayList ve generics*
>
> C# 1.0'da (2002) generics yoktu. `System.Collections` ad alanındaki `ArrayList` ve `Hashtable` her şeyin atası olan `object` türünde eleman tutardı. Generics, C# 2.0 ve .NET Framework 2.0 ile 2005'te geldi ve `System.Collections.Generic` koleksiyonları bu eski sınıfların yerini aldı. Eski sınıflar geriye uyumluluk için hâlâ vardır; yeni kodda kullanılmaz.

Eski yaklaşımın neden bırakıldığını görelim. `ArrayList` her şeyi `object` olarak saklar, bu yüzden her okumada dönüşüm (cast) gerekir ve yanlış tip ancak çalışırken fark edilir:

▸ arrayListEski.cs

```
using System.Collections;

ArrayList karisik = new ArrayList();
karisik.Add("elma");
karisik.Add(42);                        // derleyici itiraz etmez

string ilk = (string)karisik[0]!;       // her okumada dönüşüm gerekir
Console.WriteLine(ilk);
try
{
    string ikinci = (string)karisik[1]!;
    Console.WriteLine(ikinci);
}
catch (InvalidCastException)
{
    Console.WriteLine("Hata: 42 bir string değil");
}
```

```
ÇIKTI
elma
Hata: 42 bir string değil
```

Eleman tipini belirten `List<string>` ile aynı hata derleme aşamasına gelir. Program hiç çalışmadan sorunu görürüz:

listeTipHatasi.cs

```
List<string> liste = new();
liste.Add("elma");
liste.Add(42);
string ilk = liste[0];                // dönüşüme gerek yok
```

```
DERLEYİCİ ÇIKTISI
listeTipHatasi.cs(3,11): error CS1503: Argument 1: cannot convert from 'int' to 'string'
```

Generics iki kazanç sağlar: **tip güvenliği** (yanlış tipte eleman derlenmez) ve **dönüşümsüz okuma**. Üçüncü bir kazanç değer tiplerinde ortaya çıkar. `ArrayList`'e eklenen her `int`, Bölüm 52'de gördüğümüz kutulama (boxing) ile öbekte bir nesneye sarılır; `List<int>` ise sayıları doğrudan kendi dizisinde tutar. Yani hem daha hızlı hem de daha az bellek kullanan bir yapıdır. Generics'in bu yönünü bölümün ikinci yarısında kendi tipimizi yazarak da göreceğiz.

54.2 List: Büyüyen Dizi

`List<T>`'deki `T`, "eleman tipi kullanılırken söylenecek" demektir: `List<int>`, `List<string>`, `List<Urun>`. Elemanlara dizi gibi indeksle ulaşırız. Eleman sayısı için dizilerdeki `Length` yerine `Count` özelliği kullanılır. C++'taki `size()` bir üye fonksiyondu; C#'ta `Count` ve `Length` özelliktir (property), parantez almaz.

listeIslemleri.cs

```
List<string> dersler = new();
dersler.Add("Fizik");                 // sona ekle
dersler.Add("Kimya");
dersler.Add("Programlama");
dersler.Insert(0, "Matematik");       // 0. konuma ekle, diğerleri sağa kayar
Console.WriteLine(string.Join(", ", dersler));

dersler[3] = "Biyoloji";              // indeksleyiciyle değiştir
Console.WriteLine(dersler.Contains("Kimya"));
Console.WriteLine(dersler.IndexOf("Kimya"));
Console.WriteLine(dersler.IndexOf("Tarih"));    // yoksa -1

dersler.Remove("Fizik");              // değere göre sil
dersler.RemoveAt(0);                  // indekse göre sil
Console.WriteLine(string.Join(", ", dersler));

dersler.AddRange(new[] { "Tarih", "Coğrafya" });
List<string> parca = dersler.GetRange(1, 2);    // 1. indeksten 2 eleman
Console.WriteLine(string.Join(", ", parca));
dersler.Clear();
Console.WriteLine(dersler.Count);
```

```
ÇIKTI
Matematik, Fizik, Kimya, Programlama
True
2
-1
Kimya, Biyoloji
Biyoloji, Tarih
0
```

Sık kullanılan üyeleri bir arada görelim:

Üye	Ne yapar?	Maliyet
`Add(e)`	Sona ekler	Amortize O(1)
`Insert(i, e)`	`i` . konuma ekler, sonrakileri kaydırır	O(n)
`liste[i]`	Okur veya değiştirir	O(1)
`Remove(e)`	Değeri `e` olan ilk elemanı siler, `bool` döndürür	O(n)
`RemoveAt(i)`	`i` . elemanı siler, sonrakileri kaydırır	O(n)
`RemoveAll(koşul)`	Koşulu sağlayan hepsini siler, sayısını döndürür	O(n)
`Contains(e)` , `IndexOf(e)`	Var mı? / İlk konumu (yoksa `-1`)	O(n)
`Sort()` , `Reverse()`	Yerinde sıralar / tersler	O(n log n) / O(n)
`ToArray()`	Elemanları yeni bir diziye kopyalar	O(n)

Maliyetler `vector` ile aynıdır (Bölüm 35'teki karşılaştırma tablosuna bakın). Sona eklemek ucuz, başa ya da ortaya eklemek ve silmek pahalıdır, çünkü sonraki elemanların hepsi kaydırılır.

> **▲ Sık Yapılan Hata · Remove ile RemoveAt karıştırılır**
>
> `Remove(değer)` değeri arar ve yalnızca **ilk** eşleşmeyi siler; bulamazsa istisna atmaz, `false` döndürür. `RemoveAt(indeks)` ise indeksteki elemanı siler; sonraki elemanların indeksleri bir azalır. `List<int>` üzerinde `liste.Remove(2)` "değeri 2 olanı", `liste.RemoveAt(2)` "2. indeksteki elemanı" siler. Tip `int` olduğunda ikisi yanlışlıkla birbirinin yerine yazılır.

Count, Capacity ve büyüme

`List<T>` iki sayı tutar: `Count` içindeki eleman sayısı, `Capacity` ise arkadaki dizinin uzunluğudur. `vector`'ün `size()` ve `capacity()` çiftiyle aynı fikirdir. Kapasite dolunca liste daha büyük bir dizi ayırır ve elemanları kopyalar. Bu ayırmanın ne sıklıkla olduğunu görelim:

▸ kapasiteBuyume.cs

```
List<int> liste = new();
int son = liste.Capacity;
Console.WriteLine($"Başlangıç: Count={liste.Count} Capacity={liste.Capacity}");
for (int i = 1; i <= 20; i++)
{
    liste.Add(i);
    if (liste.Capacity != son)
    {
        son = liste.Capacity;
        Console.WriteLine($"Count={liste.Count} Capacity={liste.Capacity}");
    }
}
```

ÇIKTI

```
Başlangıç: Count=0 Capacity=0
Count=1 Capacity=4
Count=5 Capacity=8
Count=9 Capacity=16
Count=17 Capacity=32
```

Kapasite her dolduğunda iki katına çıkar; bu yüzden sona ekleme "amortize" O(1) sayılır (Bölüm 35'te `vector` için aynı sonucu görmüştük). Başlangıç kapasitesi 4 ve iki katına çıkma, mevcut .NET uygulamasının ayrıntısıdır; dil bunu garanti etmez. Eleman sayısını önceden biliyorsak `new List<int>(1000)` ile kapasiteyi baştan verip gereksiz kopyalamaları önleriz. Bu, `vector::reserve`'ün karşılığıdır.

> **▲ Sık Yapılan Hata · Count ile Length karışıklığı**
>
> Dizi ve `string` için `Length`, `List<T>` ve diğer koleksiyonlar için `Count` kullanılır. `liste.Length` yazarsak `CS1061` derleme hatası alırız; `dizi.Count` da derlenmez (yalnızca LINQ'nun parantezli `Count()` metodu dizide çalışır, Bölüm 56). Son geçerli indeks `Count - 1`'dir. Geçersiz indekse erişim `ArgumentOutOfRangeException` fırlatır; `vector`'ün denetimsiz `[]` işlecinin aksine C#'ta sınır denetimi her zaman yapılır.

Koleksiyon ifadeleri

Listeyi baştan elemanlarla oluşturmanın üç yolu vardır. İlk ikisi eskidir; üçüncüsü C# 12 ile gelen **koleksiyon ifadesi**dir (collection expression). Köşeli parantezler diziler, `List<T>` ve başka koleksiyonlar için ortak yazımdır; `..` (yayma, spread) başka bir koleksiyonun elemanlarını yerine açar:

▸ koleksiyonIfadesi.cs

```
List<int> a = new List<int>();
a.Add(3); a.Add(8);                       // elle ekleme
List<int> b = new List<int> { 3, 8, 5 }; // koleksiyon başlatıcı
List<int> c = [3, 8, 5];                  // koleksiyon ifadesi (C# 12)
List<int> d = [..b, 9, ..c];              // iki listeyi ve bir değeri birleştir
List<int> bos = [];
Console.WriteLine($"{string.Join(" ", d)} | {bos.Count}");
```

```
ÇIKTI
3 8 5 9 3 8 5 | 0
```

54.3 Dolaşırken Değiştirmek

"Listedeki çift sayıları sil" görevinde akla gelen ilk çözüm `foreach` ile dolaşıp `Remove` çağırmaktır. Deneyelim:

▸ foreachSilme.cs

```
List<int> sayilar = new() { 3, 8, 5, 12, 7 };
try
{
    foreach (int s in sayilar)
    {
        if (s % 2 == 0)
        {
            sayilar.Remove(s);          // dolaşırken listeyi değiştiriyoruz
        }
    }
}
catch (InvalidOperationException e)
{
    Console.WriteLine(e.Message);
}
Console.WriteLine(string.Join(" ", sayilar));
```

```
ÇIKTI
Collection was modified; enumeration operation may not execute.
3 5 12 7
```

`foreach` arka planda koleksiyonun bir **sayıcısını** (enumerator) kullanır (Bölüm 51'deki `IEnumerable<T>`). Koleksiyon dolaşma sürerken değişirse sayıcı bunu fark eder ve `InvalidOperationException` fırlatır. Bu, C++'taki yineleyici geçersizleşmesinin (Bölüm 35) güvenli karşılığıdır: C++'ta aynı hata sessizce tanımsız davranışa yol açardı, C#'ta açık bir istisna alırız. `Dictionary` ve `HashSet` dolaşılırken yeni eleman eklemek de aynı istisnayı verir.

Doğru yollar iki tanedir. Birincisi sondan başa doğru indeksli döngüdür; silme henüz bakılmamış elemanları kaydırmaz. İkincisi, koşulu bir metot olarak alan `RemoveAll`'dır:

▸ dogruSilme.cs

```
List<int> a = new() { 3, 8, 5, 12, 7 };
for (int i = a.Count - 1; i >= 0; i--)    // sondan başa
{
    if (a[i] % 2 == 0)
    {
        a.RemoveAt(i);
    }
}
Console.WriteLine(string.Join(" ", a));

List<int> b = new() { 3, 8, 5, 12, 7 };
int silinen = b.RemoveAll(x => x % 2 == 0);
Console.WriteLine($"{string.Join(" ", b)} (silinen: {silinen})");
```

```
ÇIKTI
3 5 7
3 5 7 (silinen: 2)
```

`x => x % 2 == 0` bir **lambda ifadesidir**: "her `x` için `x` çift mi?" diye okuyabiliriz. Lambdaları Bölüm 55'te ayrıntılı işleyeceğiz; şimdilik "koşulu kısaca yazma biçimi" olarak kullanıyoruz.

> **▲ Sık Yapılan Hata · Baştan sona indeksle silmek eleman atlar**
>
> `for (int i = 0; i < liste.Count; i++)` içinde `RemoveAt(i)` çağırırsak istisna oluşmaz, ama eleman atlanır: silinen elemanın yerine sağdaki gelir, `i++` ise onun üzerinden geçer. Yan yana iki çift sayının ikincisi silinmeden kalır. Hatasız çalışıp yanlış sonuç veren bu durum, istisna veren `foreach` hatasından daha tehlikelidir.

54.4 Arama ve Sıralama

`Contains` ve `IndexOf` eşitlik denemesini `Equals` ile yapar (Bölüm 50). Kendi sınıflarımızda `Equals` ezilmemişse iki nesne yalnızca aynı nesneyse eşit sayılır. Koşula göre arama için `Exists`, `Find`, `FindIndex` ve `FindAll` vardır; bunlar bir koşul alır:

▸ aramaSiralama.cs

```
List<int> sayilar = [42, 7, 19, 3, 25];

Console.WriteLine(sayilar.Exists(x => x > 40));      // var mı?
Console.WriteLine(sayilar.Find(x => x > 20));        // ilk uyan
Console.WriteLine(sayilar.FindIndex(x => x > 20));   // ilk uyanın indeksi
```

```
Console.WriteLine(string.Join(" ", sayilar.FindAll(x => x < 20)));

sayilar.Sort();                                    // yerinde, artan
Console.WriteLine(string.Join(" ", sayilar));
Console.WriteLine(sayilar.BinarySearch(19));       // sıralı listede ikili arama
Console.WriteLine(sayilar.BinarySearch(20) < 0);   // bulunamazsa negatif

sayilar.Sort((x, y) => y.CompareTo(x));            // azalan
Console.WriteLine(string.Join(" ", sayilar));
sayilar.Reverse();
Console.WriteLine(string.Join(" ", sayilar));
```

```
ÇIKTI
True
42
0
7 19 3
3 7 19 25 42
2
True
42 25 19 7 3
3 7 19 25 42
```

`BinarySearch` yalnızca **sıralı** listede doğru çalışır; bulunca indeksi, bulamayınca negatif bir sayı döndürür. `Sort` ise listeyi yerinde değiştirir ve bir şey döndürmez; `liste = liste.Sort();` yazmak derlenmez.

`Sort((x, y) => y.CompareTo(x))` satırındaki lambda, iki elemanın hangisinin önce geleceğini söyler: negatif sayı "ilk önce", sıfır "eşit", pozitif sayı "ikinci önce" demektir. Bu, C'deki `qsort`'un karşılaştırma fonksiyonuyla (Bölüm 12) ve C++'taki `std::sort`'un karşılaştırıcısıyla (Bölüm 36) aynı sözleşmedir. Kendi sınıfımızın nesnelerini sıralamak için Bölüm 51'de gördüğümüz iki yol da geçerlidir: sınıf `IComparable<T>` uygularsa parametresiz `Sort()` onu kullanır; ayrı bir ölçüt gerekiyorsa `IComparer<T>` nesnesi ya da lambda verilir.

> **◆ Kritik Nokta**
>
> `List<T>.Sort` kararlı (stable) değildir: eşit elemanların göreli sırası korunmayabilir. Kararlı sıralama gerekiyorsa Bölüm 56'daki LINQ `OrderBy` kullanılır. C++'ta bu fark `std::sort` ile `std::stable_sort` arasındaydı.

54.5 Liste Bir Referans Tipidir

`List<T>` bir sınıftır, yani referans tipidir (Bölüm 42). `b = a` ataması listeyi kopyalamaz, ikinci bir değişkeni aynı listeye bağlar. C++'ta `vector` değer semantiğine sahipti ve atama kopyalardı; bu önemli bir farktır.

▸ listeKopya.cs

```
List<int> a = new() { 3, 1, 2 };
List<int> b = a;                   // aynı liste, ikinci ad
List<int> c = new(a);              // bağımsız kopya
b.Add(9);
c.Sort();
Console.WriteLine(string.Join(" ", a));
Console.WriteLine(string.Join(" ", c));
Console.WriteLine(ReferenceEquals(a, b));
```

```
ÇIKTI
3 1 2 9
1 2 3
True
```

`new List<int>(a)` yeni bir liste kurar ve elemanları kopyalar. Eleman tipi referans tipiyse kopyalanan yalnızca referanslardır, nesnelerin kendisi paylaşılır (sığ kopya, shallow copy). Bir metoda liste geçirdiğimizde de metot aynı listeyi görür ve değiştirebilir; Bölüm 46'daki parametre geçirme kuralları geçerlidir.

54.6 Dictionary: Anahtar–Değer Çiftleri

Bir telefon rehberinde ada bakıp numarayı buluruz: ad **anahtar** (key), numara **değer** (value) olur. `Dictionary<TKey,TValue>` her anahtarı bir değere eşleyen karma tablodur (hash table). Anahtarlar benzersizdir, değerler tekrar edebilir. Bölüm 35'teki `unordered_map<K,V>`'nin karşılığıdır; `map<K,V>`'nin karşılığı ise bölümün ilerisindeki `SortedDictionary`'dir. Anahtara göre arama, ekleme ve silme ortalama O(1) sürer.

`sozlukTemel.cs`

```
Dictionary<string, int> stok = new()
{
    ["kalem"] = 40,                         // başlatıcıda indeksleyici yazımı
    ["defter"] = 25
};
stok.Add("silgi", 60);                      // yeni çift ekle
stok["defter"] = 30;                        // anahtar varsa değeri değiştirir
stok["cetvel"] = 8;                         // yoksa ekler

Console.WriteLine($"Defter: {stok["defter"]}, Count: {stok.Count}");
Console.WriteLine($"{stok.ContainsKey("silgi")} {stok.ContainsKey("makas")}");

if (stok.TryGetValue("kalem", out int adet))
{
    Console.WriteLine($"Kalem: {adet}");
}
bool bulundu = stok.TryGetValue("makas", out int yok);
Console.WriteLine($"{bulundu}, değer: {yok}");

Console.WriteLine(stok.GetValueOrDefault("makas", -1));
Console.WriteLine(stok.TryAdd("kalem", 99));       // var: eklenmez, false
Console.WriteLine(stok.Remove("silgi"));           // silindi: true
Console.WriteLine(stok.Count);
```

```
ÇIKTI
Defter: 30, Count: 4
True False
Kalem: 40
False, değer: 0
-1
False
True
3
```

`TryGetValue` anahtar varsa `true` döndürür ve değeri `out` değişkenine yazar; yoksa `false` döndürür ve değişken tipin varsayılan değerini (`int` için `0`) alır. Bu, Bölüm 46'da gördüğümüz `TryParse` kalıbıdır ve

sözlükten güvenle okumanın yoludur. İki aramayı (`ContainsKey` sonra `[]`) arka arkaya yapmaktan daha hızlıdır, çünkü anahtar yalnızca bir kez aranır.

Üyelerin özeti:

Üye	Anahtar varsa	Anahtar yoksa
`d[k]` (okuma)	Değeri verir	`KeyNotFoundException`
`d[k] = v`	Değeri değiştirir	Çifti ekler
`d.Add(k, v)`	`ArgumentException`	Çifti ekler
`d.TryAdd(k, v)`	`false`, değişmez	Ekler, `true`
`d.TryGetValue(k, out v)`	`true`, değer `v`'de	`false`, `v` varsayılan
`d.Remove(k)`	Siler, `true`	`false`

> **Üç Dilde · Eksik anahtar: C++ ve C#** *C · C++ · C#*
>
> C++'ta `m[k]` okuma amacıyla yazılsa bile anahtar yoksa varsayılan değerle **ekler** (Bölüm 35). C#'ta `d[k]` okuması anahtar yoksa istisna fırlatır, yani sessizce çift eklemez:
>
> ```
> Dictionary<string, int> d = new() { ["a"] = 1 };
> try { Console.WriteLine(d["b"]); }
> catch (KeyNotFoundException) { Console.WriteLine("yok"); }   // yok
> ```
>
> C++'taki `find` + yineleyici denetimi, C#'ta `TryGetValue`'ya karşılık gelir. "Yoksa ekle, varsa güncelle" için C#'ta `d[k] = v` yeterlidir.

Dolaşma ve kelime sayma

`foreach` bir sözlükte her adımda bir `KeyValuePair<TKey,TValue>` verir; `Key` ve `Value` özellikleriyle okunur. C++'taki `pair` (`it->first`, `it->second`) ve yapısal bağlama (Bölüm 35) burada da vardır: C# ayrıştırma ile (deconstruction) iki değişkene açabilir. Klasik uygulama kelime saymaktır: her kelime bir anahtar, geçme sayısı değerdir.

▸ kelimeSayaci.cs

```
string metin = "bir elma bir armut iki elma bir de kiraz";
Dictionary<string, int> sayac = new();
foreach (string kelime in metin.Split(' '))
{
    sayac[kelime] = sayac.GetValueOrDefault(kelime) + 1;   // yoksa 0 + 1
}

List<string> anahtarlar = new(sayac.Keys);
anahtarlar.Sort();                          // sıra garanti değil, biz sıralıyoruz
foreach (string k in anahtarlar)
{
    Console.WriteLine($"{k,-6} {sayac[k]}");
}

string enSik = "";
int enCok = 0;
foreach (var (kelime, adet) in sayac)       // KeyValuePair ayrıştırma
{
    if (adet > enCok)
    {
```

```
        enCok = adet;
        enSik = kelime;
    }
}
Console.WriteLine($"En sık: {enSik} ({enCok}), farklı kelime: {sayac.Count}");
```

```
ÇIKTI
armut  1
bir    3
de     1
elma   2
iki    1
kiraz  1
En sık: bir (3), farklı kelime: 6
```

> **◆ Kritik Nokta**
>
> `Dictionary` ve `HashSet` dolaşma sırası konusunda söz vermez. Pratikte çoğu zaman ekleme sırası görünür, ama silme sonrası eklenen eleman boşalan yere girerek sırayı bozabilir. Çıktısı sıraya bağlı kod yazmayız: anahtarları listeye alıp sıralarız ya da sıradan bağımsız sonuçlar (toplam, en büyük) üretiriz. Anahtarları sıralı tutan koleksiyon gerekiyorsa `SortedDictionary` kullanırız.

Kendi tipimizi anahtar yapmak

Sözlük anahtarı önce `GetHashCode` ile bir kovaya (bucket) yönlendirir, sonra kovanın içinde `Equals` ile karşılaştırır (Bölüm 50). Kendi sınıfımızı anahtar yapacaksak ikisini de tutarlı biçimde ezmeliyiz. Ezilmemişse iki nesne yalnızca aynı nesneyse eşit sayılır:

▸ ozelAnahtar.cs

```
Dictionary<Konum, string> a = new();
a[new Konum(1, 2)] = "depo";
Console.WriteLine(a.ContainsKey(new Konum(1, 2)));   // farklı nesne

Dictionary<Nokta, string> b = new();
b[new Nokta(1, 2)] = "depo";
Console.WriteLine(b.ContainsKey(new Nokta(1, 2)));   // değer eşitliği

class Konum
{
    public int X, Y;
    public Konum(int x, int y) { X = x; Y = y; }
}

record Nokta(int X, int Y);
```

```
ÇIKTI
False
True
```

`record` tipleri `Equals` ve `GetHashCode`'u alanlara göre kendiliğinden üretir (Bölüm 52). Anahtar olarak kullandığımız nesnenin eşitlikte yer alan alanlarını sözlüğe koyduktan sonra değiştirmemeliyiz; hash değeri değişir ve nesne tabloda "kaybolur". Bu yüzden anahtarlar çoğunlukla `string`, sayı ya da değişmez (immutable) tiplerdir.

54.7 HashSet ve SortedSet: Kümeler

`HashSet<T>` tekrarsız eleman tutan karma kümedir; C++'taki `unordered_set<T>`'nin karşılığıdır. İndeksi yoktur, `kume[0]` yazılamaz. İki işi çok iyi yapar: "var mı?" sorusunu ortalama O(1)'de yanıtlar ve kopyaları ayıklar. `Add` yeni eleman eklenirse `true`, zaten varsa `false` döndürür; hata vermez.

Küme işlemleri için ayrı metotlar vardır ve hepsi kümeyi yerinde değiştirir. Sıralı yazdırmak için `SortedSet<T>` kullanalım; bu, elemanları sürekli sıralı tutan ağaç tabanlı kümedir ve C++'taki `set<T>`'in karşılığıdır (işlemler O(log n)):

▸ kumeler.cs

```
HashSet<string> dersler = new();
Console.WriteLine($"{dersler.Add("Fizik")} {dersler.Add("Kimya")}");
Console.WriteLine(dersler.Add("Fizik"));            // zaten var
Console.WriteLine($"{dersler.Count} {dersler.Contains("Kimya")}");

List<string> yoklama = new() { "Can", "Elif", "Ayşe", "Can", "Elif", "Can" };
HashSet<string> tekil = new(yoklama);               // kopyalar ayıklanır
Console.WriteLine($"{yoklama.Count} kayıt, {tekil.Count} kişi");

SortedSet<int> a = new() { 1, 2, 3, 4, 5 };
SortedSet<int> b = new() { 4, 5, 6, 7 };
SortedSet<int> birlesim = new(a);
birlesim.UnionWith(b);
SortedSet<int> kesisim = new(a);
kesisim.IntersectWith(b);
SortedSet<int> fark = new(a);
fark.ExceptWith(b);
Console.WriteLine(string.Join(" ", birlesim));
Console.WriteLine(string.Join(" ", kesisim));
Console.WriteLine(string.Join(" ", fark));
Console.WriteLine(kesisim.IsSubsetOf(a));
```

```
ÇIKTI
True True
False
2 True
6 kayıt, 3 kişi
1 2 3 4 5 6 7
4 5
1 2 3
True
```

C++'ta aynı işler `<algorithm>` başlığındaki `set_union`, `set_intersection` ve `set_difference` ile yapılır ve sonuç ayrı bir kapsayıcıya yazılır (Bölüm 36). C#'ta kümenin kendi metotları vardır.

54.8 Queue ve Stack

Kuyruk ve yığının mantığını C'de Bölüm 19'da elle yazmıştık; C++'ta `queue` ve `stack` uyarlayıcılarıydı (Bölüm 35). C#'ta `Queue<T>` ilk giren ilk çıkar (FIFO), `Stack<T>` son giren ilk çıkar (LIFO) çalışır. Önemli bir fark vardır: C++'ta `pop()` değer döndürmezdi; `front()` / `top()` ile bakıp sonra `pop()` çağırırdık. C#'ta `Dequeue()` ve `Pop()` hem çıkarır hem de değeri **döndürür**. Çıkarmadan bakmak için `Peek()` vardır.

	`Queue<T>`	`Stack<T>`
Ekle	`Enqueue(e)`	`Push(e)`

	`Queue<T>`	`Stack<T>`
Çıkar ve al	`Dequeue()`	`Pop()`
Çıkarmadan bak	`Peek()`	`Peek()`
Güvenli sürümler	`TryDequeue` , `TryPeek`	`TryPop` , `TryPeek`

kuyrukYigin.cs

```
Queue<string> kuyruk = new();
kuyruk.Enqueue("Ayşe");
kuyruk.Enqueue("Mert");
kuyruk.Enqueue("Elif");
Console.WriteLine($"Sıradaki: {kuyruk.Peek()}");     // çıkarmaz
Console.WriteLine($"Çağrılan: {kuyruk.Dequeue()}");  // çıkarır ve verir
Console.WriteLine($"{string.Join(", ", kuyruk)} | Count: {kuyruk.Count}");

Stack<string> yigin = new();
yigin.Push("a");
yigin.Push("b");
yigin.Push("c");
Console.WriteLine($"Tepe: {yigin.Peek()}");
Console.WriteLine($"Çıkan: {yigin.Pop()}");
Console.WriteLine($"{string.Join(", ", yigin)} | Count: {yigin.Count}");

Stack<int> bos = new();
Console.WriteLine(bos.TryPop(out int deger));        // boş: false, istisna yok
```

ÇIKTI

```
Sıradaki: Ayşe
Çağrılan: Ayşe
Mert, Elif | Count: 2
Tepe: c
Çıkan: c
b, a | Count: 2
False
```

`Queue` dolaşılırken baştan sona, `Stack` üstten alta doğru elemanları verir. Boş bir koleksiyonda `Dequeue`, `Pop` ve `Peek` çağrılırsa `InvalidOperationException` fırlar; boş olması olası durumlarda `Try...` sürümlerini ya da `Count` denetimini kullanırız.

Yığının klasik bir uygulaması parantez denetimidir: açan parantezi yığına atar, kapananla karşılaşınca yığının tepesiyle eşleştiririz. Sonunda yığın boş ve hiç eşleşmezlik olmamışsa ifade dengelidir:

parantezDenetimi.cs

```
foreach (string ifade in new[] { "(a[b]{c})", "(a[b)]", "((a)", "" })
{
    Console.WriteLine($"\"{ifade}\" -> {Dengeli(ifade)}");
}

static bool Dengeli(string metin)
{
    Stack<char> yigin = new();
    foreach (char c in metin)
    {
        if (c == '(' || c == '[' || c == '{')
        {
            yigin.Push(c);
        }
```

```
        else if (c == ')' || c == ']' || c == '}')
        {
            if (!yigin.TryPop(out char acan))
            {
                return false;                          // kapanan var, açan yok
            }
            bool eslesir = (acan == '(' && c == ')')
                        || (acan == '[' && c == ']')
                        || (acan == '{' && c == '}');
            if (!eslesir)
            {
                return false;
            }
        }
    }
    return yigin.Count == 0;                           // açıkta kalan parantez var mı?
}
```

```
ÇIKTI
"(a[b]{c})" -> True
"(a[b)]" -> False
"((a)" -> False
"" -> True
```

PriorityQueue

Önceliği en küçük olanın önce çıktığı kuyruk .NET 6'dan beri hazırdır; C++'taki `priority_queue`'nun karşılığıdır, ama iki farkı vardır. Eleman ile öncelik ayrı verilir ve en **küçük** öncelik önce çıkar (C++'ta varsayılan olarak en büyük çıkardı). Aynı önceliğe sahip elemanların hangisinin önce çıkacağı belirsizdir.

```
▸ oncelikKuyrugu.cs
```

```
PriorityQueue<string, int> isler = new();
isler.Enqueue("rapor yaz", 3);
isler.Enqueue("sunucuyu yeniden başlat", 1);
isler.Enqueue("e-postaları oku", 5);
isler.Enqueue("hatayı düzelt", 2);

while (isler.TryDequeue(out string? gorev, out int oncelik))
{
    Console.WriteLine($"{oncelik}: {gorev}");
}
```

```
ÇIKTI
1: sunucuyu yeniden başlat
2: hatayı düzelt
3: rapor yaz
5: e-postaları oku
```

54.9 LinkedList

`LinkedList<T>`, Bölüm 19'da elle yazdığımız çift yönlü bağlı listenin hazır hâlidir (C++'ta `list<T>`). Her eleman bir `LinkedListNode<T>` düğümüdür; `Value`, `Next` ve `Previous` özellikleri vardır. İndeksleyicisi yoktur, çünkü `i`. elemana gitmek O(n) sürerdi. Güçlü yanı, elimizde bir düğüm varsa o noktaya ekleme ve silmenin O(1) olmasıdır:

bagliListe.cs

```
LinkedList<string> liste = new();
liste.AddLast("b");
liste.AddLast("d");
liste.AddFirst("a");
LinkedListNode<string> dugum = liste.Find("b")!;     // O(n) arama
liste.AddAfter(dugum, "c");                          // O(1) ekleme
Console.WriteLine(string.Join(" ", liste));

liste.Remove("d");
Console.WriteLine($"{liste.First!.Value} {liste.Last!.Value} {liste.Count}");

List<string> ters = new();
for (var d = liste.Last; d != null; d = d.Previous)   // sondan başa
{
    ters.Add(d.Value);
}
Console.WriteLine(string.Join(" ", ters));
```

```
ÇIKTI
a b c d
a c 3
c b a
```

Pratikte `List<T>` çoğu zaman `LinkedList<T>`'den hızlıdır; düğümler bellekte dağınık durur ve önbellekten yararlanamaz. Bölüm 35'te `vector` için söylediğimiz ilke burada da geçerlidir: ölçüm başka bir şey söylemedikçe `List<T>` ile başlarız.

54.10 Sıralı Sözlük: SortedDictionary ve SortedList

`SortedDictionary<TKey,TValue>` anahtarları sürekli sıralı tutar; C++'taki `map<K,V>` gibi dengeli bir ağaç kullanır, işlemleri O(log n) sürer. Dolaşma anahtar sırasıyla yapılır. `SortedList<TKey,TValue>` aynı arayüzü iki sıralı dizi üstünde sunar: bellek açısından daha tutumludur ve indeksle erişim verir, ama ortaya ekleme O(n)'dir.

siraliSozluk.cs

```
SortedDictionary<string, int> sonuclar = new()
{
    ["mert"] = 70,
    ["ayse"] = 85,
    ["zeynep"] = 90,
    ["can"] = 60
};
foreach (KeyValuePair<string, int> cift in sonuclar)
{
    Console.WriteLine($"{cift.Key,-7}{cift.Value}");
}
sonuclar["bora"] = 75;                               // araya girer
Console.WriteLine(string.Join(", ", sonuclar.Keys));
```

```
ÇIKTI
ayse   85
can    60
mert   70
zeynep 90
ayse, bora, can, mert, zeynep
```

Dizgi anahtarlarında varsayılan karşılaştırma kültüre duyarlıdır; Türkçe harfler içeren anahtarlarda sıra beklenenden farklı çıkabilir. Kültür ayarlarının metin işlemlerine etkisini Bölüm 48'de görmüştük. Burada sıralamayı yalnızca ASCII harflerle gösterdik.

54.11 Kendi Generic Sınıfımız

Koleksiyonların hepsi generics üzerine kuruludur. Bu özelliği kendi tiplerimizde de kullanabiliriz. Sınıf adından sonra `<T>` yazarız; `T`, sınıf kullanılırken verilecek gerçek tipin yerini tutar. Bölüm 33'te C++ şablonu olarak yazdığımız `Yigin<T>`'nin C# karşılığını yazalım. Elemanlar bir `T[]` dizisinde durur ve dizi dolunca iki katına büyür:

genericYigin.cs

```
Yigin<int> sayilar = new();
for (int i = 1; i <= 6; i++)
{
    sayilar.Push(i * i);
}
Console.WriteLine($"Count={sayilar.Count} Kapasite={sayilar.Kapasite}");
Console.WriteLine($"{sayilar.Pop()} {sayilar.Pop()} {sayilar.Peek()}");

Yigin<string> adlar = new();
adlar.Push("Ayşe");
adlar.Push("Mert");
string son = adlar.Pop();                    // dönüşüm yok: tip string
Console.WriteLine(son.ToUpper());

try
{
    new Yigin<double>().Pop();
}
catch (InvalidOperationException e)
{
    Console.WriteLine(e.Message);
}

class Yigin<T>
{
    private T[] _ogeler = new T[4];
    private int _sayi;

    public int Count => _sayi;
    public int Kapasite => _ogeler.Length;

    public void Push(T oge)
    {
        if (_sayi == _ogeler.Length)
        {
            Array.Resize(ref _ogeler, _ogeler.Length * 2);
        }
        _ogeler[_sayi++] = oge;
    }
```

```
    public T Pop()
    {
        if (_sayi == 0)
        {
            throw new InvalidOperationException("Yığın boş");
        }
        T oge = _ogeler[--_sayi];
        _ogeler[_sayi] = default!;           // eski referansı serbest bırak
        return oge;
    }

    public T Peek()
    {
        if (_sayi == 0)
        {
            throw new InvalidOperationException("Yığın boş");
        }
        return _ogeler[_sayi - 1];
    }
}
```

```
ÇIKTI
Count=6 Kapasite=8
36 25 16
MERT
Yığın boş
```

`Yigin<int>` yazdığımızda derleyici sınıftaki her `T`'yi `int` gibi ele alır; `Yigin<string>` için ise `string` gibi. Bu kapalı tiplerin her biri ayrı bir tiptir. `Yigin<int>`'e `string` eklemek derlenmez, `Pop()` sonucu dönüşümsüz kullanılır. `default!` ifadesi, `T` ne olursa olsun onun varsayılan değerini verir: sayılar için `0`, `bool` için `false`, referans tipleri için `null`. C++'taki `T()` ya da `T{}` ile aynı işi görür. `!` işareti, Bölüm 59'da inceleyeceğimiz nullable uyarısını susturur.

Birden fazla tip parametresi virgülle ayrılır. .NET geleneğinde tek parametre `T`, birden fazlaysa anlamlı adlar verilir: `Dictionary<TKey,TValue>`. Parametreler sınıfın alanlarında, özelliklerinde ve metot imzalarında kullanılabilir:

```
class Cift<TIlk, TIkinci>
{
    public TIlk Ilk { get; }
    public TIkinci Ikinci { get; }
    public Cift(TIlk ilk, TIkinci ikinci) { Ilk = ilk; Ikinci = ikinci; }
}
```

Her kapalı tip kendi statik alanlarını taşır. `Sayac<int>.Adet` ile `Sayac<string>.Adet` ayrı değişkenlerdir. Bu davranış C++ şablonlarındakiyle aynıdır:

▸ genericStatik.cs

```
_ = new Sayac<int>();
_ = new Sayac<int>();
_ = new Sayac<string>();
Console.WriteLine($"{Sayac<int>.Adet} {Sayac<string>.Adet}");

class Sayac<T>
{
```

```
    public static int Adet;
    public Sayac() { Adet++; }
}
```

```
ÇIKTI
2 1
```

> **Not · Çalışma zamanında generics**
>
> C++ her şablon örneği için derleme zamanında ayrı kod üretir (Bölüm 33). C#'ta derleyici yalnızca genel tanımı denetler ve IL koduna `Yigin<T>` olarak yazar. Tip bilgisi çalışma zamanında vardır: JIT derleyicisi her **değer tipi** (`int`, `double`, `struct`) için özel makine kodu üretir, tüm **referans tipleri** için ise tek bir ortak kod kullanır. Sonuç olarak `List<int>` kutulama yapmaz, `List<string>` ile `List<Urun>` ise aynı kodu paylaşır. Çalışma zamanında `typeof(T)` ile gerçek tipe ulaşılabilir. Buna karşılık `T` üzerinde `T + T` gibi işlemler kısıt bildirilmeden yazılamaz (aşağıdaki kısıtlara bakın).

54.12 Generic Metotlar

Tek bir metodu generic yapmak için tip parametresini metot adından sonra yazarız. Çağırırken tip çoğunlukla argümanlardan çıkarılır (C++ şablonlarındaki tip çıkarımı gibi), istersek açıkça da belirtebiliriz. C'de `void *` ve `memcpy` ile yazdığımız, tip bilgisini unutan `takas` fonksiyonu burada tip güvenli olur:

▸ genericMetot.cs

```
int a = 3, b = 8;
Takas(ref a, ref b);                          // T = int, çıkarım
Console.WriteLine($"{a} {b}");

string x = "elma", y = "armut";
Takas<string>(ref x, ref y);                  // tipi açıkça belirttik
Console.WriteLine($"{x} {y}");

int[] ilkler = { 5, 9, 2 };
string[] meyveler = { "muz", "elma", "kiraz" };
Console.WriteLine($"{Ilk(ilkler)} {Ilk(meyveler)}");
Console.WriteLine(Ilk(new List<double> { 2.5, 1.5 }));

static void Takas<T>(ref T p, ref T q)
{
    T gecici = p;
    p = q;
    q = gecici;
}

static T Ilk<T>(IEnumerable<T> kaynak)
{
    foreach (T oge in kaynak)
    {
        return oge;
    }
    throw new InvalidOperationException("Kaynak boş");
}
```

```
ÇIKTI
8 3
armut elma
5 muz
2.5
```

`Takas(ref a, ref b)` çağrısında `ref` argümanları (Bölüm 46) bulunduğu için `T`, `int` olarak çıkarılır. `Takas(ref a, ref x)` gibi birbirine uymayan tiplerde ise çıkarım başarısız olur ve derleme hatası alırız.

54.13 Kısıtlar: where

Bir generic gövdesinde `T` hakkında bildiğimiz tek şey, onun bir `object` olduğudur. Bu yüzden iki `T` değerini `>` ile karşılaştıramayız:

kisitsizMetot.cs

```
Console.WriteLine(EnBuyuk(3, 5));

static T EnBuyuk<T>(T a, T b)
{
    return a > b ? a : b;
}
```

```
DERLEYİCİ ÇIKTISI

kisitsizMetot.cs(5,12): error CS0019: Operator '>' cannot be applied to operands of type 'T' and
'T'
```

C++'ta bu kod bir şablon olarak geçerliydi; `T = int` için örneklendiğinde derleniyor, `T`'nin `>` desteklemediği bir tiple örneklendiğinde ise uzun bir hata mesajı veriyordu (Bölüm 33). C#'ta hata, genel kod tanımlanırken verilir. `T`'nin ne yapabildiğini **kısıtlarla** (constraints) derleyiciye bildiririz. Kısıtlar `where` ile yazılır:

Kısıt	Anlamı	`T` ile neler yapılır
`where T : struct`	`T` bir değer tipi (null olamaz)	`T?` yazılabilir (`Nullable<T>`)
`where T : class`	`T` bir referans tipi	`null` ile karşılaştırılabilir
`where T : new()`	Parametresiz herkese açık yapıcısı var	`new T()` yazılabilir
`where T : Taban`	`T`, `Taban` sınıfından türer	`Taban`'ın üyeleri kullanılır
`where T : IArayuz`	`T`, arayüzü uygular	Arayüzün üyeleri kullanılır
`where T : notnull`	`T` null olamaz	Sözlük anahtarı gibi yerlerde

Birden fazla kısıt virgülle yazılır; `class`/`struct` başa, `new()` sona gelir: `where T : class, IComparable<T>, new()`. Birden fazla tip parametresi varsa her biri için ayrı `where` yazılır. Karşılaştırma ihtiyacı için en yaygın kısıt `IComparable<T>`'dir (Bölüm 51):

kisitlar.cs

```
Console.WriteLine(Sinirla(15, 0, 10));
Console.WriteLine(Sinirla(2.5, 3.0, 4.0));
Console.WriteLine(Sinirla("m", "a", "f"));

Ogrenci o = Yeni<Ogrenci>();
Console.WriteLine(o.Ad);
```

```
Console.WriteLine(IlkEleman(new List<int>()) is null);
Console.WriteLine(IlkEleman(new List<int> { 7, 8 }));

static T Sinirla<T>(T deger, T alt, T ust) where T : IComparable<T>
{
    if (deger.CompareTo(alt) < 0) return alt;
    if (deger.CompareTo(ust) > 0) return ust;
    return deger;
}

static T Yeni<T>() where T : class, new()
{
    return new T();                          // new() kısıtı sayesinde
}

static T? IlkEleman<T>(List<T> liste) where T : struct
{
    return liste.Count > 0 ? liste[0] : null;   // struct kısıtı: T? = Nullable<T>
}

class Ogrenci
{
    public string Ad { get; set; } = "adsız";
}
```

```
ÇIKTI
10
3
f
adsız
True
7
```

`Sinirla` hem `int`, hem `double`, hem de `string` ile çalışır; çünkü üçü de `IComparable<T>` uygular. `IlkEleman` boş listede `null` verebilir, çünkü dönüş tipi `int?` olur (Bölüm 42'deki nullable değer tipleri).

Genel matematik: INumber

Kısıtlar sayesinde `+` gibi işleçler de generic koda girebilir. C# 11 ve .NET 7 ile gelen **statik soyut arayüz üyeleri**, sayısal tiplerin ortak arayüzlerini mümkün kıldı. `INumber<T>` kısıtı, `T` için `+`, `*`, karşılaştırma ve `T.Zero` gibi üyeleri açar:

genelMatematik.cs

```
using System.Numerics;

Console.WriteLine(Toplam(new[] { 1, 2, 3 }));
Console.WriteLine(Toplam(new[] { 1.5, 2.25 }));
Console.WriteLine(Toplam(new[] { 0.1m, 0.2m }));

static T Toplam<T>(IEnumerable<T> kaynak) where T : INumber<T>
{
    T toplam = T.Zero;
    foreach (T oge in kaynak)
    {
        toplam += oge;
    }
    return toplam;
}
```

```
ÇIKTI
6
3.75
0.3
```

C++ şablonlarında `a + b` yazmak yeterliydi ve `+` desteklemeyen tipte hata ancak örneklemede çıkardı. C++20'nin kavramları (concepts) bu eksiği C++'a kısıt olarak getirir (Bölüm 39'daki C++20 özetine bakın). C#'ta kısıtlar baştan beri dilin parçasıdır.

54.14 Koleksiyon Arayüzleri

Bu bölümde tanıdığımız koleksiyonların hepsi ortak arayüzleri uygular. Bölüm 51'de `IEnumerable<T>`'nin `foreach` ve LINQ için gerekli olduğunu görmüştük. Aşağıdaki hiyerarşi, hangi koleksiyonun ne yapabildiğini özetler:

Arayüz	Ek olarak sunduğu	Uygulayanlar
`IEnumerable<T>`	Dolaşma (`foreach`)	Hepsi, diziler dahil
`IReadOnlyCollection<T>`	`Count`	Hepsi
`ICollection<T>`	`Count`, `Add`, `Remove`, `Contains`, `Clear`	`List`, `HashSet`, `LinkedList`, sözlükler…
`IList<T>`	İndeksleyici, `Insert`, `RemoveAt`	`List`, diziler
`IDictionary<TKey,TValue>`	Anahtarla erişim	`Dictionary`, `SortedDictionary`

Bir metodun parametresini **işini görecek en dar arayüzle** tanımlamak iyi bir alışkanlıktır. `Yazdir(List<int> l)` yalnızca listeleri kabul eder; `Yazdir<T>(IEnumerable<T> k)` ise dizi, liste, küme, kuyruk ve daha fazlasıyla çalışır:

▸ arayuzleCalisma.cs

```
List<int> liste = new() { 3, 1, 2 };
int[] dizi = { 4, 5 };
HashSet<int> kume = new() { 7, 7, 8 };
Queue<int> kuyruk = new(new[] { 9, 6 });
SortedDictionary<string, int> sozluk = new() { ["b"] = 2, ["a"] = 1 };

Yazdir(liste);
Yazdir(dizi);
Yazdir(kume);
Yazdir(kuyruk);
Yazdir(sozluk.Keys);                          // anahtar koleksiyonu da gezilebilir
Console.WriteLine(Say(dizi) + " " + Say(kume));

static void Yazdir<T>(IEnumerable<T> kaynak)
{
    Console.WriteLine(string.Join(" ", kaynak));
}

static int Say<T>(IReadOnlyCollection<T> kaynak)
{
    return kaynak.Count;
}
```

```
ÇIKTI
3 1 2
4 5
7 8
9 6
a b
2 2
```

C++'ta aynı esnekliği yineleyici çifti (`begin`, `end`) ya da şablon parametresi veriyordu (Bölüm 36). C#'ta arayüz verir. Bir koleksiyonu dışarıya salt okunur göstermek istersek metodun dönüş tipini `IReadOnlyList<T>` yaparız; çağıran tarafın `Add` çağırmasını derleyici engeller.

> **Not · Eşzamanlı koleksiyonlar**
>
> Bu bölümdeki koleksiyonlar birden çok iş parçacığından (thread) eşzamanlı yazma için güvenli değildir. Çok iş parçacıklı programlarda `System.Collections.Concurrent` ad alanındaki `ConcurrentDictionary<TKey,TValue>` ve `ConcurrentQueue<T>` gibi tipler kullanılır; bu kitabın kapsamı dışındadır.

54.15 Doğru Koleksiyonu Seçmek

Koleksiyon seçimi, programda en sık yaptığımız işlemlere bağlıdır. Aşağıdaki tabloda n eleman sayısıdır; O(1) sabit, O(log n) logaritmik, O(n) doğrusal süre demektir. Bölüm 35'teki tabloyla aynı gösterimi kullanıyoruz.

Koleksiyon	C++ karşılığı	Arama / `Contains`	Ekleme	Sıra
`List<T>`	`vector`	O(n); indeksle erişim O(1)	Sonda amortize O(1); ortada O(n)	Ekleme sırası
`Dictionary<K,V>`	`unordered_map`	Ortalama O(1)	Ortalama O(1)	Garanti yok
`HashSet<T>`	`unordered_set`	Ortalama O(1)	Ortalama O(1)	Garanti yok
`SortedDictionary<K,V>`	`map`	O(log n)	O(log n)	Anahtar sırası
`SortedSet<T>`	`set`	O(log n)	O(log n)	Sıralı
`Queue<T>`	`queue`	O(n)	O(1)	İlk giren ilk çıkar
`Stack<T>`	`stack`	O(n)	O(1)	Son giren ilk çıkar
`LinkedList<T>`	`list`	O(n)	Düğüm biliniyorsa O(1)	Ekleme sırası
`PriorityQueue<E,P>`	`priority_queue`	Yok	O(log n)	En küçük öncelik önce

Karar sırası şöyle olabilir:

1. Sıralı bir koleksiyon ve indeksle erişim gerekiyorsa, ya da emin değilsek **`List<T>`**.
2. Anahtarla değer bulacaksak **`Dictionary<K,V>`**; anahtarlar sıralı dolaşılacaksa `SortedDictionary`.
3. Tekrarsızlık ya da "var mı?" sorusu varsa **`HashSet<T>`**.
4. İşlem sırası belirleyici ise **`Queue<T>`** (sırayla), **`Stack<T>`** (tersine), **`PriorityQueue`** (önceliğe göre).
5. Ortaya çok sık düğüm ekleyip silmek gerekiyorsa ve bu düğümlere zaten ulaşılıyorsa `LinkedList<T>`.

Sabit boyutlu ve elemanları sık sık okunan veride dizi (`T[]`) hâlâ en hızlı ve en az bellek harcayan seçimdir. Örnek: bir sınıftaki öğrencileri numarasıyla bulmak için `Dictionary<int, Ogrenci>`, aynı öğrencileri puana göre listelemek için `List<Ogrenci>`, bir derse kayıtlı farklı kişileri saymak için `HashSet<string>`.

Üç Dilde · Kapsayıcı işlemleri: C, C++ ve C# *C · C++ · C#*

C'de standart bir koleksiyon yoktur: `malloc` / `realloc` ile dizi büyütür, bağlı liste ve yığını Bölüm 19'daki gibi elle yazarız. **C++**'ta STL kapsayıcıları (Bölüm 35), **C#**'ta `System.Collections.Generic` bu işi hazır sunar. Tip eşleşmelerini Bölüm 35'teki "STL ve C# koleksiyonları" tablosunda vermiştik; aşağıda gündelik işlemlerin ve davranışların karşılığı var:

İş	C++	C#
Sona ekleme	`v.push_back(x)`	`l.Add(x)`
Eleman sayısı	`v.size()`	`l.Count`
İndeksle erişim	`v[i]` denetimsiz, `v.at(i)` denetimli	`l[i]` her zaman denetimli (`ArgumentOutOfRangeException`)
Konuma göre silme	`v.erase(v.begin() + i)`	`l.RemoveAt(i)`
Değere göre silme	`erase(remove(...))`	`l.Remove(x)` / `l.RemoveAll(koşul)`
Arama	`find(v.begin(), v.end(), x)`	`l.IndexOf(x)` / `l.Contains(x)`
Sıralama	`sort(v.begin(), v.end())`	`l.Sort()`
Kapasite	`reserve`, `capacity()`	`new List<T>(n)`, `Capacity`
Sözlükte var mı?	`m.count(k)` / `m.find(k)`	`d.ContainsKey(k)` / `d.TryGetValue`
Kümeye ekleme	`s.insert(x).second`	`h.Add(x)` (`bool` döner)
Kuyruktan çıkarma	`q.front(); q.pop();` (`pop` değer vermez)	`q.Dequeue()` (değeri verir)
Atama `b = a`	Kopyalar (değer semantiği)	Aynı nesneyi gösterir (referans)
Dolaşırken değiştirme	Yineleyici geçersizleşir, tanımsız davranış	`InvalidOperationException`

Kısacası C# koleksiyonları sınıf (referans) olarak yaşar, sınır denetimi yapar ve hataları istisnayla bildirir; C++ kapsayıcıları değer olarak yaşar ve performans için denetimi programcıya bırakır.

Bölüm Özeti

- Diziler sabit boyutludur; koleksiyonlar (`System.Collections.Generic`) ihtiyaca göre büyür. `ArrayList` gibi tipsiz koleksiyonlar yerini generic olanlara bıraktı.
- Generics tip güvenliği, dönüşümsüz okuma ve değer tiplerinde kutulamasız saklama sağlar (C# 2.0, 2005).
- `List<T>`, `vector`'ün karşılığıdır: `Add`, `Insert`, `Remove` (değer, ilk eşleşme), `RemoveAt` (indeks), `RemoveAll`, `IndexOf`, `Sort`, `BinarySearch`. Eleman sayısı `Count`, arka dizi uzunluğu `Capacity`'dir.
- Koleksiyonlar referans tipidir: `b = a` kopyalamaz; kopya için `new List<T>(a)` yazılır (sığ kopya).
- Dolaşırken koleksiyonu değiştirmek `InvalidOperationException` verir. Çözüm sondan başa `for` ya da `RemoveAll`'dır.

- `Dictionary<K,V>` karma tablodur: `d[k] = v` ekler ya da değiştirir, `Add` var olan anahtarda `ArgumentException`, `d[k]` olmayan anahtarda `KeyNotFoundException` verir. Güvenli okuma `TryGetValue`'dur. Dolaşma sırasına güvenilmez.
- Kendi tipimizi anahtar yapmak için `Equals` ve `GetHashCode` tutarlı ezilmelidir; `record` bunu kendiliğinden yapar.
- `HashSet<T>` tekrarsız kümedir (`Add` `bool` döner, `UnionWith`, `IntersectWith`, `ExceptWith`); `SortedSet<T>` ve `SortedDictionary<K,V>` sıralı tutar.
- `Queue<T>` (FIFO) ve `Stack<T>` (LIFO) için `Dequeue` ve `Pop` değer döndürür; boşken istisna atar, güvenli sürümleri `TryDequeue` ve `TryPop`'tur. `PriorityQueue<E,P>` en küçük önceliği önce çıkarır.
- `LinkedList<T>` düğüm tabanlıdır, indeksleyicisi yoktur; çoğu işte `List<T>` daha hızlıdır.
- Generic sınıf `class Yigin<T>`, generic metot `static void Takas<T>(...)` biçiminde yazılır. Çağrıda tip çoğunlukla çıkarılır; her kapalı tipin statik alanları ayrıdır.
- Kısıtlar (`where T : struct`, `class`, `new()`, taban sınıf, arayüz, `notnull`) `T` ile yapılabilecekleri belirler. Kısıtsız `T` üzerinde yalnızca `object` işlemleri geçerlidir. `INumber<T>` ile sayısal işleçler kullanılabilir.
- Metot parametrelerinde en dar arayüz (`IEnumerable<T>`, `IReadOnlyCollection<T>`) tercih edilir.
- Koleksiyon seçimi işlem maliyetlerine bağlıdır: emin değilsek `List<T>`, anahtarla arama için `Dictionary`, "var mı?" için `HashSet`.

Alıştırmalar

1. `int[] dizi = new int[3];`, `List<int> liste = new() { 1, 2, 3 };` ve `string metin = "abc";` tanımları verilmiştir. Aşağıdaki ifadelerden hangisi derlenir ve `3` değerini verir?

   A) `liste.Length`
   B) `dizi.Count`
   C) `metin.Count`
   D) `liste.Count`
   E) `dizi.Length()`

2. Aşağıdaki kod parçası ne yazdırır?

```
List<int> l = new() { 4, 2, 0, 2 };
l.Remove(2);
l.RemoveAt(2);
Console.WriteLine(string.Join(" ", l));
```

   A) `4 0`
   B) `4 2`
   C) `4 0 2`
   D) `4 2 0`
   E) `ArgumentOutOfRangeException` fırlatır

3. Aşağıdaki kod parçasında ne olur?

```
Dictionary<string, int> d = new();
d["a"] = 1;
d["a"] = 2;
d.Add("b", 3);
d.Add("b", 4);
Console.WriteLine(d.Count);
```

A) `d["a"] = 2;` satırında `ArgumentException` fırlatılır.
B) `d["a"] = 2;` satırında `KeyNotFoundException` fırlatılır.
C) `d.Add("b", 3);` satırında `ArgumentException` fırlatılır.
D) Hata olmaz, ekrana `2` yazılır.
E) `d.Add("b", 4);` satırında `ArgumentException` fırlatılır ve program çöker.

4. Bir tarayıcıdaki "Geri" düğmesi en son ziyaret edilen sayfaya döner. Ziyaret edilen sayfaları tutmak için en uygun koleksiyon hangisidir?

A) `Queue<string>`
B) `Stack<string>`
C) `HashSet<string>`
D) `Dictionary<string, int>`
E) `string[]`

5. Aşağıdaki generic metot hangi kısıtı eklersek derlenir?

```
static T Olustur<T>() where T : class
{
    return new T();
}
```

A) `struct` kısıtını `class` yerine yazmak
B) `class, new()` yazmak
C) `IComparable<T>` eklemek
D) `notnull` eklemek
E) Kısıt gerekmez; `new T()` her zaman derlenir

6. Aşağıdaki program ekrana ne yazdırır?

```
List<int> l = new() { 2, 4, 4, 5, 6, 8 };
for (int i = 0; i < l.Count; i++)
{
    if (l[i] % 2 == 0)
    {
        l.RemoveAt(i);
    }
}
Console.WriteLine(string.Join(" ", l) + " | " + l.Count);
l.Insert(1, 9);
Console.WriteLine(l.IndexOf(8) + " " + l.Contains(4) + " " + string.Join(" ", l));
```

7. Aşağıdaki program ekrana ne yazdırır?

```
string[] kazananlar = { "Elif", "Can", "Elif", "Bora", "Elif", "Can" };
Dictionary<string, int> puan = new();
foreach (string k in kazananlar)
{
```

```
    puan[k] = puan.GetValueOrDefault(k) + 10;
}
puan["Can"] -= 5;
puan.Remove("Bora");
Console.WriteLine(puan.Count + " " + puan.ContainsKey("Bora"));
int toplam = 0;
foreach (int p in puan.Values)
{
    toplam += p;
}
Console.WriteLine(toplam);
Console.WriteLine(puan["Elif"] + " " + puan["Can"]);
```

8. Aşağıdaki program ekrana ne yazdırır?

```
Queue<int> k = new();
Stack<int> y = new();
foreach (int x in new[] { 1, 2, 3 })
{
    k.Enqueue(x);
    y.Push(x);
}
k.Enqueue(k.Dequeue());
y.Push(y.Pop() * 10);
Console.WriteLine(string.Join(" ", k));
Console.WriteLine(string.Join(" ", y));
HashSet<int> h = new(k);
Console.WriteLine(h.Add(2) + " " + h.Add(5));
```

9. Aşağıdaki generic metot neden derlenmez? Metodu `int`, `double` ve `string` listeleriyle çalışacak şekilde düzeltin.

```
static T EnKucuk<T>(List<T> liste)
{
    T enKucuk = liste[0];
    foreach (T oge in liste)
    {
        if (oge < enKucuk)
        {
            enKucuk = oge;
        }
    }
    return enKucuk;
}
```

10. Aşağıdaki kod listedeki çift sayıları silmek istiyor, ama çalışırken istisna verir. Nedenini açıklayın ve iki farklı yolla düzeltin.

```
List<int> sayilar = new() { 3, 8, 5, 12, 7 };
foreach (int s in sayilar)
{
    if (s % 2 == 0)
    {
        sayilar.Remove(s);
    }
}
```

11. Elemanları `int` olan bir listeyi `ArrayList` yerine `List<int>` ile tutmanın üç avantajını yazın.

12. `{ 5, 3, 9, 3, 5, 1, 9, 9 }` dizisindeki tekrarlı değerleri ayıklayıp küçükten büyüğe tek satırda yazdıran bir program yazın (çıktı `1 3 5 9`). `HashSet` ve `SortedSet` kullanın.
13. Verilen kelime dizisini ilk harflerine göre gruplayan bir program yazın: `Dictionary<char, List<string>>` kullanın, anahtarları sıralı yazdırın. Örnek dizi: `{ "armut", "elma", "ayva", "erik", "muz", "kiraz", "kavun" }`. Çıktı: `a: armut, ayva` biçiminde, her harf için bir satır.
14. Verilen bir `IEnumerable<T>` içinde en çok tekrar eden elemanı döndüren `EnCokTekrar<T>` metodunu yazın (`T`, sözlük anahtarı olabilmelidir). `{ "a", "b", "a", "c", "a" }` ve `{ 4, 4, 7, 7, 7 }` ile deneyin.
15. En çok `kapasite` kadar eleman alan generic bir `SinirliYigin<T>` sınıfı yazın: `Push` yer yoksa `false` döndürsün, `TryPop(out T oge)` boşsa `false` döndürsün. Sınıfı üç eleman kapasiteli bir `int` yığınıyla deneyin.

BÖLÜM 55

Delegate, Event ve Lambda İfadeleri

BU BÖLÜMDE

- Delegate'in bir metodu değişken gibi tutan tip olduğunu ve C fonksiyon işaretçisinden farkını
- `Func<>`, `Action<>` ve `Predicate<>` hazır delegate tiplerini
- Lambda ifadelerini ve dış değişken yakalamayı (closure)
- Çok noktaya yayın (multicast) delegate'leri ve `+=`, `-=` operatörlerini
- `event` anahtar sözcüğünü, yayıncı/abone modelini, `EventHandler` ve `EventArgs` kullanımını
- Olay tabanlı bir sıcaklık sensörü alarmı yazmayı
- Fonksiyon işaretçisi, `std::function` ve delegate'in karşılaştırmasını

Bölüm 12'de fonksiyon işaretçileriyle bir fonksiyonu başka bir fonksiyona parametre olarak geçirmeyi, Bölüm 36'da ise C++ lambda ifadelerini ve `std::function`'ı gördük. Bu bölümde aynı fikrin C#'taki karşılığını inceliyoruz: **metodları veri gibi taşımak**. Bir sıralama ölçütünü, bir düğmeye tıklanınca yapılacak işi ya da bir sensör alarm verince çağrılacak kodu başka koda parametre olarak vermek bu fikrin tipik kullanımlarıdır.

C# bu fikri üç katmanda sunar. **Delegate** metodu tutan tiptir. **Lambda ifadesi** delegate'e atanacak işlevi kısa yazmanın yoludur. **Event** ise delegate'in yayıncı/abone kurallarına bağlanmış güvenli halidir. İlk ikisi C ve C++ bilgimizin doğrudan devamıdır; asıl yenilik olaylardadır.

Bölümdeki programlar top-level statements biçimindedir ve Bölüm 49 ile 51'deki sınıf, özellik ve arayüz bilgisini kullanır.

55.1 Delegate: Metodu Değişkende Tutmak

Bölüm 12'de `int (*islem)(int, int)` yazımıyla bir fonksiyonun adresini tutan değişken tanımlamıştık. C# aynı işi bir **tip tanımı** ile yapar: önce "hangi imzalı metotlar tutulabilir" diye bir delegate tipi bildirilir, sonra o tipten değişkenler tanımlanır.

> **Tanım: Delegate**
>
> Belirli bir dönüş tipi ve parametre listesi (imza) taşıyan metotlara referans tutabilen tiptir. Bir delegate değişkenine imzası uyan herhangi bir metot atanabilir ve değişken bir metot gibi çağrılabilir. Delegate'ler referans tipidir.

Delegate tipi `delegate` anahtar sözcüğüyle, metot bildirimine benzer biçimde yazılır:

delegateTemel.cs

```
Islem islem = Topla;            // metodun adı atanır, parantez yok
Console.WriteLine(islem(3, 4));

islem = Carp;                   // aynı değişken başka metodu tutabilir
Console.WriteLine(islem(3, 4));

Console.WriteLine(islem.Invoke(5, 6));   // islem(5, 6) ile aynı

int Topla(int a, int b) => a + b;
int Carp(int a, int b) => a * b;

delegate int Islem(int a, int b);
```

```
ÇIKTI
7
12
30
```

`delegate int Islem(int a, int b);` satırı bir metot değil, **tip** bildirir; top-level kodun altına, sınıf tanımları gibi yazarız. `Islem`, iki `int` alıp `int` döndüren her metodu tutabilir. `Topla` yazıp parantez koymadığımızda metot çağrılmaz, metodun kendisi (adresi) atanır. Bu, C'deki `islem = topla;` ile aynıdır.

Delegate'i parametre olarak geçirmek

Delegate'in asıl değeri, davranışı parametre yapmasıdır. Aşağıdaki metot bir dizinin her elemanına, kendisine verilen işlemi uygular. İşlemin ne olduğunu çağıran belirler.

delegateParametre.cs

```
int[] sayilar = { 1, 2, 3, 4 };

int[] kareler = Donustur(sayilar, Kare);
Console.WriteLine(string.Join(", ", kareler));

int[] ikiKat = Donustur(sayilar, Ikiyle);
Console.WriteLine(string.Join(", ", ikiKat));

int[] Donustur(int[] kaynak, Donusum f)
{
    int[] sonuc = new int[kaynak.Length];
    for (int i = 0; i < kaynak.Length; i++)
        sonuc[i] = f(kaynak[i]);
    return sonuc;
}

int Kare(int x) => x * x;
int Ikiyle(int x) => x * 2;

delegate int Donusum(int x);
```

```
ÇIKTI
1, 4, 9, 16
2, 4, 6, 8
```

Bu, Bölüm 12'deki `qsort` ölçüt fonksiyonuyla aynı düzendir. Farklar küçük ama önemlidir:

- C'de fonksiyon işaretçisinin tipi imzadan gelir ve imzayı denetleyen tek şey derleyicidir; ama `void *` parametreleriyle tip güvenliği kolayca kaybedilir. C# delegate'i her zaman tam tiplidir; imzası uymayan metot atanamaz.
- C işaretçisi yalnızca bir adres tutar. Delegate ise metotla birlikte, metodun **hangi nesne üzerinde** çalışacağını da tutar (aşağıda göreceğiz).
- Delegate çöp toplayıcının yönettiği bir nesnedir; "boşta kalan işaretçi" sorunu yoktur.

Örnek metotlar ve statik metotlar

Bir delegate'e örnek metodu (nesneye ait metot) atarsak, delegate o nesneye de referans tutar. Çağrı yapıldığında metot o nesnenin alanlarıyla çalışır:

▸ hedefNesne.cs

```
var yuzde = new Indirim(oran: 10);
Fiyat hesapla = yuzde.Uygula;      // örnek metot + nesne birlikte saklanır
Console.WriteLine(hesapla(200m));

yuzde.Oran = 25;                   // nesne değişirse delegate de bunu görür
Console.WriteLine(hesapla(200m));
Console.WriteLine(hesapla.Target == yuzde);

class Indirim
{
    public int Oran;
    public Indirim(int oran) { Oran = oran; }
    public decimal Uygula(decimal fiyat) => fiyat * (100 - Oran) / 100;
}

delegate decimal Fiyat(decimal x);
```

```
ÇIKTI
180
150
True
```

C'de bunu yapmak için fonksiyon işaretçisinin yanında ayrıca bir `void *veri` parametresi taşırız. C++'ta aynı işi `std::function` ile yakalanan nesne ya da lambda görür. C#'ta delegate bu "metot + nesne" çiftini yerleşik olarak taşır.

> **▲ Sık Yapılan Hata**
>
> Delegate değişkeni `null` olabilir. Hiçbir metot atanmamış bir delegate'i çağırırsak `NullReferenceException` alırız. Belirsiz olan yerlerde `islem?.Invoke(...)` yazımını kullanırız; `?.` yalnızca değişken `null` değilse çağrıyı yapar.

55.2 Func, Action ve Predicate

Her iş için ayrı delegate tipi bildirmek hızla angarya olur. `Donusum`, `Islem`, `Fiyat` tiplerinin hepsi "şu parametreleri alıp şunu döndür" demektir. .NET bu yüzden sık gereken imzalar için **hazır genel (generic) delegate'ler** sunar. Genel tipleri Bölüm 54'te gördük; burada `<>` içindeki tipler parametre ve dönüş tipleridir.

Tip	İmza	Anlamı
`Action`	`void ()`	parametresiz, değer döndürmeyen
`Action<T1, T2>`	`void (T1, T2)`	parametreleri var, değer döndürmez
`Func<TSonuc>`	`TSonuc ()`	parametresiz, değer döndürür
`Func<T1, T2, TSonuc>`	`TSonuc (T1, T2)`	**son tip** dönüş tipidir
`Predicate<T>`	`bool (T)`	bir koşulu sınar

`Action` ve `Func` 16 parametreye kadar sürümlere sahiptir. `Func`'ta son tip parametresinin daima dönüş tipi olduğunu aklımızda tutalım: `Func<int, int, string>`, iki `int` alıp `string` döndürür.

▸ hazirDelegate.cs

```
Func<int, int, int> topla = (a, b) => a + b;
Func<double> pi = () => 3.14;
Action<string> selam = ad => Console.WriteLine($"Merhaba {ad}");
Action ayrac = () => Console.WriteLine("---");
Predicate<int> ciftMi = n => n % 2 == 0;

Console.WriteLine(topla(2, 3));
Console.WriteLine(pi());
selam("Ayşe");
ayrac();
Console.WriteLine(ciftMi(10));
Console.WriteLine(ciftMi(7));

List<int> liste = new() { 1, 2, 3, 4, 5, 6 };
Console.WriteLine(liste.FindAll(ciftMi).Count);   // koşulu sağlayanlar
liste.RemoveAll(n => n > 4);                      // koşulu sağlayanları sil
liste.ForEach(n => Console.Write($"[{n}]"));      // her elemana işlem
Console.WriteLine();
```

```
ÇIKTI
5
3.14
Merhaba Ayşe
---
True
False
3
[1][2][3][4]
```

`List<T>` sınıfının `FindAll`, `RemoveAll` ve `Exists` metotları `Predicate<T>`, `ForEach` metodu `Action<T>` ister. Bunlar C++ `<algorithm>` başlığındaki `remove_if`, `find_if` ve `for_each` işlevlerinin C#'taki karşılıklarıdır (Bölüm 36). `Predicate<T>` aslında `Func<T, bool>` ile aynı imzaya sahiptir; ama bu iki tip birbirine doğrudan atanamaz. Yeni kod yazarken genellikle `Func<T, bool>` tercih edilir; `Predicate<T>`'ye ise `List<T>` gibi eski API'lerde rastlarız.

Sıralama ölçütü için `Comparison<T>` delegate'i vardır; imzası `int (T, T)`'dir ve `Sort` metoduna verilir. Bölüm 36'daki karşılaştırma lambdasının karşılığıdır:

▸ siralamaOlcutu.cs

```
List<string> adlar = new() { "Zeynep", "Can", "Elif", "Deniz" };

adlar.Sort((a, b) => a.Length.CompareTo(b.Length));   // uzunluğa göre
Console.WriteLine(string.Join(" ", adlar));
```

```
adlar.Sort((a, b) => string.Compare(b, a));           // ters alfabetik
Console.WriteLine(string.Join(" ", adlar));
```

```
ÇIKTI
Can Elif Deniz Zeynep
Zeynep Elif Deniz Can
```

> **Not**
>
> `List<T>.Sort` kararlı (stable) bir sıralama yapmaz; uzunluğu aynı olan iki eleman birbirine göre yer değiştirebilir. Yukarıdaki örnekte uzunluğu eşit olan ad yoktur. Kararlılık gerektiğinde Bölüm 56'daki LINQ `OrderBy` kararlıdır.

55.3 Lambda İfadeleri

Her kısa işlem için ayrı bir adlı metot yazmak okumayı zorlaştırır. Lambda ifadesi, adı olmayan metodu doğrudan kullanıldığı yerde yazmamızı sağlar. Bölüm 36'daki `[](int x) { return x * 2; }` yazımının C# karşılığı `x => x * 2`'dir.

> **Tanım: Lambda ifadesi**
>
> `=>` (lambda operatörü) ile yazılan adsız işlevdir. Soldaki parametre listesi, sağdaki gövde ya da ifadedir. Bir delegate tipine ya da ifade ağacına dönüştürülür.

Lambda iki biçimde yazılır:

```
Func<int, int> kare = x => x * x;              // ifade gövdeli
Func<int, int, int> enBuyuk = (a, b) =>        // deyim gövdeli
{
    if (a > b) return a;
    return b;
};
Func<int, int> tipli = (int x) => x + 1;       // tipler açıkça yazılabilir
Action bos = () => { };                        // parametresiz
```

- Tek parametre ve tip yazılmadıysa parantez isteğe bağlıdır (`x => ...`); diğer durumlarda parantez gerekir.
- Parametre tipleri çoğunlukla delegate tipinden çıkarılır, bu yüzden yazılmaz.
- Gövde tek bir ifadeyse süslü parantez ve `return` gerekmez.

C# 10'dan beri bir lambda kendi başına da bir "doğal tipe" sahip olabilir. Parametre tiplerini açıkça yazarsak `var` ile saklayabiliriz:

```
var yarisi = (double x) => x / 2;    // Func<double, double> olarak çıkarılır
```

C# 9'dan beri lambdaya `static` ekleyerek dış değişken kullanmasını engelleyebiliriz (`static x => x * x`). C# 12 ise lambda parametrelerine varsayılan değer verilebilmesini getirdi. İkisi de günlük kullanımda seyrek gereken ayrıntılardır.

> *Tarihçe · Delegate'ten lambdaya*
>
> Delegate, C# 1.0'dan (2002) beri dilin parçasıdır. Satır içi adsız metotlar (`delegate (int x) { return x * 2; }`) C# 2.0'da geldi. Lambda ifadeleri ve `Func` / `Action` ailesi C# 3.0 ile .NET Framework 3.5'te (2007) eklendi; LINQ bu eklemeler üzerine kuruludur (Bölüm 56). C++'ta lambdaların C++11'de gelmesinden birkaç yıl önce C#'ta yerleşmişlerdi.

Dış değişkenleri yakalamak (closure)

Lambda gövdesi, tanımlandığı yerdeki yerel değişkenleri kullanabilir. Buna **dış değişken yakalama** denir ve yakalanan değişkenleri taşıyan lambdaya closure adı verilir. C++'ta yakalama listesini `[=]` ya da `[&]` ile biz belirtirdik (Bölüm 36). C#'ta böyle bir liste yoktur: lambda dış değişkeni **her zaman değişkenin kendisine bağlanarak** yakalar, değerini kopyalamaz.

▸ yakalama.cs

```
int carpan = 3;
Func<int, int> kat = x => x * carpan;
Console.WriteLine(kat(5));
carpan = 10;                    // lambda bu değişikliği görür
Console.WriteLine(kat(5));

Func<int> sayac = SayacUret();
Console.WriteLine(sayac());
Console.WriteLine(sayac());
Console.WriteLine(sayac());

Func<int> SayacUret()
{
    int n = 0;                  // metot bitse bile yaşamaya devam eder
    return () => ++n;
}
```

ÇIKTI

```
15
50
1
2
3
```

`SayacUret` döndükten sonra `n` yerel değişkeni normalde yok olurdu. Ama lambda onu yakaladığı için derleyici `n`'yi gizli bir sınıfın alanına taşır ve bu nesne, delegate yaşadığı sürece canlı kalır. Bölüm 12'de "yerel değişkenin adresini döndürmenin" neden ölümcül olduğunu görmüştük; C#'ta yerel değişkenin ömrünü çöp toplayıcı uzattığı için aynı kod güvenlidir. C++'ta `[&]` ile yakalanan yerel değişken, işlev dönünce sarkan referansa dönüşürdü.

Yakalamanın değişkene bağlı olması, döngülerde klasik bir tuzağa yol açar:

▸ donguYakalama.cs

```
var fonksiyonlar = new List<Func<int>>();
for (int i = 0; i < 3; i++)
    fonksiyonlar.Add(() => i);          // her lambda AYNI i'yi yakalar

var dogru = new List<Func<int>>();
for (int i = 0; i < 3; i++)
{
    int kopya = i;                      // her turda yeni değişken
```

```
    dogru.Add(() => kopya);
}

var a = new List<int>();
foreach (var f in fonksiyonlar) a.Add(f());
Console.WriteLine(string.Join(" ", a));

var b = new List<int>();
foreach (var f in dogru) b.Add(f());
Console.WriteLine(string.Join(" ", b));
```

```
ÇIKTI
3 3 3
0 1 2
```

`for` döngüsünde `i` tek bir değişkendir ve döngü bitince 3 olur; üç lambda da bu aynı değişkeni okur. Döngü gövdesinde tanımlanan `kopya` ise her turda yeni bir değişkendir. `foreach` döngü değişkeni C# 5'ten beri her turda yeni bir değişken sayılır; bu yüzden `foreach` içinde yakalama beklenen sonucu verir.

> **◆ Kritik Nokta**
>
> C#'ta lambda, yakaladığı değişkeni kopyalamaz; değişkenin kendisini paylaşır. Değerin o anki halini istiyorsak bloğun içinde yerel bir kopya alırız. C++'ta bu ayrım `[=]` ve `[&]` ile açıkça yazılırdı; C#'ta varsayılan davranış her zaman `[&]` gibidir, ama yaşam süresi sorunu olmaz.

55.4 Çok Noktaya Yayın (Multicast)

Bir delegate yalnızca tek bir metodu değil, bir metot **listesini** tutabilir. `+=` listeye metot ekler, `-=` listeden çıkarır. Delegate çağrıldığında listedeki metotlar eklenme sırasıyla çalışır.

`cokluYayin.cs`

```
Action<string> yayin = Ekrana;
yayin += Gunluge;
yayin += m => Console.WriteLine($"Lambda: {m}");

yayin("ilk");
Console.WriteLine("-");

yayin -= Ekrana;
yayin("ikinci");

void Ekrana(string m) => Console.WriteLine($"Ekran: {m}");
void Gunluge(string m) => Console.WriteLine($"Günlük: {m}");
```

```
ÇIKTI
Ekran: ilk
Günlük: ilk
Lambda: ilk
-
Günlük: ikinci
Lambda: ikinci
```

`yayin += Gunluge;` aslında `yayin = yayin + Gunluge;` demektir. Delegate'ler **değiştirilemez** (immutable) nesnelerdir; `+=` mevcut delegate'i değiştirmez, iki listenin birleşiminden yeni bir delegate üretir. Bu yüzden delegate'e `null` iken de `+=` yapılabilir; sonuç tek metotluk bir delegate olur.

Metot listesindeki metotlar değer döndürüyorsa **yalnızca sonuncunun** değeri elde edilir. Hepsinin sonucuna ihtiyaç varsa `GetInvocationList` ile listeyi alıp tek tek çağırırız:

donusDegeri.cs

```
Func<int> f = () => 1;
f += () => 2;
f += () => 3;

Console.WriteLine(f());                       // yalnızca sonuncunun değeri

int toplam = 0;
foreach (Func<int> g in f.GetInvocationList())
    toplam += g();
Console.WriteLine(toplam);
Console.WriteLine(f.GetInvocationList().Length);
```

```
ÇIKTI
3
6
3
```

> **▲ Sık Yapılan Hata**
>
> Çok noktaya yayın zincirindeki bir metot istisna fırlatırsa zincir orada kesilir ve sonraki metotlar çalışmaz. Abonelerin birbirinden bağımsız çalışması gereken durumlarda `GetInvocationList` ile her birini kendi `try` / `catch` bloğunda çağırmak gerekir (Bölüm 53). Ayrıca `Action<string> a = null; a += f;` yazılabilir; ama `a("x")` çağrısını `null` denetimi olmadan yapmak hâlâ `NullReferenceException` verir.

55.5 event: Yayıncı ve Abone

Çok noktaya yayın bir sorunu çözer, yenisini doğurur. Bir sınıf, durumu değiştiğinde başkalarını haberdar etmek için `public Action<string> Degisti;` gibi bir delegate alanı sunarsa, o sınıfın dışındaki herkes bu alana `=` ile yeni değer atayıp **bütün aboneleri silebilir** ya da delegate'i doğrudan çağırıp yayıncı adına sahte bildirim gönderebilir.

`event` anahtar sözcüğü bu açığı kapatır. Bir delegate alanının başına `event` yazdığımızda sınıfın dışından yalnızca `+=` ve `-=` kullanılabilir; çağırmak ve `=` ile atamak sadece sınıfın kendi içinde mümkündür.

> **Tanım: Olay (event)**
>
> Bir sınıfın (yayıncı, publisher) ilgilenen diğer nesnelere (aboneler, subscribers) "bir şey oldu" diye haber vermesini sağlayan, delegate tabanlı bir üyedir. Aboneler olaya `+=` ile kaydolur, `-=` ile kaydını siler; olayı yalnızca tanımlandığı sınıf tetikleyebilir.

Farkı derleyicinin tepkisinde görelim:

olaySinir.cs

```
var s = new Sensor();
s.Olay += () => Console.WriteLine("abone");   // abone olmak serbest
s.Olay();                                      // dışarıdan tetiklemek yasak
s.Olay = null;                                 // aboneleri silmek yasak

class Sensor
```

```
{
    public event Action? Olay;
    public void Tetikle() => Olay?.Invoke();   // sınıf içinde serbest
}
```

```
DERLEYİCİ ÇIKTISI

olaySinir.cs(3,1): error CS0070: The event 'Sensor.Olay' can only appear on the left hand side
of += or -= (except when used from within the type 'Sensor')
olaySinir.cs(4,1): error CS0070: The event 'Sensor.Olay' can only appear on the left hand side
of += or -= (except when used from within the type 'Sensor')
```

Yayıncı, abonelerin kim olduğunu bilmez ve bilmesi gerekmez; yalnızca "olay oldu" der. Abone de yayıncının iç işleyişini bilmeden yalnızca ilgilendiği olaya kaydolur. Bu gevşek bağlılık (loose coupling), grafik arayüzlerden sunucu uygulamalarına kadar C#'ın yaygın olay tabanlı tasarımının temelidir. C'de bunu geri çağırma (callback) fonksiyonlarını bir diziye kaydeden elle yazılmış kodla yapardık; C++'ta `std::vector<std::function<...>>` tutardık (Bölüm 36). C#'ta dil bu kalıbı hazır sunar.

EventHandler ve EventArgs

.NET'te olaylar için bir ortak desen vardır. Olayın tipi `EventHandler` ya da `EventHandler<TEventArgs>` delegate'idir; imzası şudur:

```
void Isleyici(object? gonderen, TEventArgs e)
```

- `gonderen` (sender): olayı tetikleyen nesnedir. Aynı işleyiciyi birden çok yayıncıya bağlayınca hangisinden geldiğini buradan anlarız.
- `e`: olayın ayrıntılarını taşıyan nesnedir. `EventArgs` sınıfından türeyen kendi sınıfımızı yazarak istediğimiz veriyi ekleriz. Taşınacak veri yoksa `EventArgs.Empty` kullanılır.

Bu desene uyarsak yazdığımız olaylar .NET kütüphanesindeki olaylarla aynı biçimde kullanılır. Genel kural olarak olayı tetikleyen kod ayrı bir `protected virtual void OnXxx` metoduna konur; böylece türeyen sınıflar olay tetiklenmeden önce araya girebilir (Bölüm 50).

Bir olayı tetiklerken `Olay?.Invoke(this, e)` yazarız. `?.`, hiç abone yoksa (delegate `null` ise) çağrıyı atlar. Bu yazım aynı zamanda iş parçacığı açısından güvenlidir: `?.` delegate'i bir kez okur, bu yüzden `if (Olay != null) Olay(...)` ikilisindeki gibi iki okuma arasında abonenin ayrılması sorunu yoktur.

Örnek: Sıcaklık sensörü alarmı

Şimdi hepsini bir araya getirelim. `SicaklikSensoru` her ölçümde `SicaklikDegisti` olayını, ölçüm sınırı aşarsa ek olarak `AlarmVerildi` olayını tetikler. Sensör, kendisine kimin abone olduğunu bilmez.

▸ sicaklikAlarmi.cs

```
var sensor = new SicaklikSensoru(limit: 30, baslangic: 20);
var kayit = new AlarmKaydi();

sensor.SicaklikDegisti += (gonderen, e) =>
    Console.WriteLine($"Ölçüm: {e.Eski} -> {e.Yeni}");
sensor.AlarmVerildi += EkranaAlarm;
sensor.AlarmVerildi += kayit.Ekle;

double[] olcumler = { 22.5, 25.0, 31.2, 29.8, 33.0 };
foreach (double o in olcumler)
```

```
    sensor.Olc(o);

Console.WriteLine($"Kayıtlı alarm sayısı: {kayit.Sayi}");

sensor.AlarmVerildi -= kayit.Ekle;      // kayıt artık dinlemiyor
sensor.Olc(35.0);
Console.WriteLine($"Kayıtlı alarm sayısı: {kayit.Sayi}");

void EkranaAlarm(object? gonderen, SicaklikEventArgs e)
{
    var s = (SicaklikSensoru)gonderen!;
    Console.WriteLine($"ALARM: {e.Yeni} derece, limit {s.Limit}");
}

class SicaklikEventArgs : EventArgs
{
    public double Eski { get; }
    public double Yeni { get; }

    public SicaklikEventArgs(double eski, double yeni)
    {
        Eski = eski;
        Yeni = yeni;
    }
}

class SicaklikSensoru
{
    private double son;
    public double Limit { get; }

    public event EventHandler<SicaklikEventArgs>? SicaklikDegisti;
    public event EventHandler<SicaklikEventArgs>? AlarmVerildi;

    public SicaklikSensoru(double limit, double baslangic)
    {
        Limit = limit;
        son = baslangic;
    }

    public void Olc(double yeni)
    {
        var e = new SicaklikEventArgs(son, yeni);
        son = yeni;
        OnSicaklikDegisti(e);
        if (yeni > Limit)
            OnAlarmVerildi(e);
    }

    protected virtual void OnSicaklikDegisti(SicaklikEventArgs e)
        => SicaklikDegisti?.Invoke(this, e);

    protected virtual void OnAlarmVerildi(SicaklikEventArgs e)
        => AlarmVerildi?.Invoke(this, e);
}

class AlarmKaydi
{
    public int Sayi { get; private set; }

    public void Ekle(object? gonderen, SicaklikEventArgs e) => Sayi++;
}
```

```
ÇIKTI
Ölçüm: 20 -> 22.5
Ölçüm: 22.5 -> 25
Ölçüm: 25 -> 31.2
ALARM: 31.2 derece, limit 30
Ölçüm: 31.2 -> 29.8
Ölçüm: 29.8 -> 33
ALARM: 33 derece, limit 30
Kayıtlı alarm sayısı: 2
Ölçüm: 33 -> 35
ALARM: 35 derece, limit 30
Kayıtlı alarm sayısı: 2
```

Programın akışına bakalım. Üstteki kodda `sensor` abonelikleri yapar: bir lambda, bir yerel metot ve başka bir nesnenin (`kayit`) örnek metodu aynı yayıncıya bağlanır. `Olc` her çağrıldığında sensör önce `SicaklikDegisti`, sınır aşıldıysa ardından `AlarmVerildi` olayını tetikler. `AlarmVerildi` iki aboneye sahip olduğu için sınır aşıldığında ikisi de, eklenme sırasıyla çalışır. `-=` ile `kayit` çıkarıldıktan sonra 35 derecelik ölçümde ekran alarmı yine yazılır; ama kayıt sayısı artmaz.

Sensörün kodunda ne `EkranaAlarm` ne de `AlarmKaydi` geçer. Yeni bir abone eklemek için sensör sınıfına dokunmamız gerekmez.

> **◆ Kritik Nokta**
>
> Olayın tetiklenmesi, abonelerin metotlarını **aynı iş parçacığında, tetikleyen kod beklerken** çağırmaktır. `Olc` metodu, bütün aboneler dönene kadar bitmez. Bir abone yavaşsa yayıncı da yavaşlar; bir abone istisna fırlatırsa kalan aboneler çalışmaz (Bölüm 53).

Abonelikten çıkmak

Bir olaya abone olmak, yayıncının abone nesneye bir referans tutması demektir. Yayıncı uzun yaşıyor, abone ise kısa ömürlü olması gerekiyorsa ve abone `-=` yapmazsa, çöp toplayıcı aboneyi yok edemez. Bellek yönetimi otomatik olsa da bu "unutulmuş abonelik" kaynaklı sızıntı pratikte sık görülür. İlke basittir: bir nesne için `+=` yazdıysak, nesnenin ömrü bittiğinde (çoğu zaman `Dispose` içinde) karşılığı olan `-=` de yazılmalıdır.

Lambdaya abone olursak, `-=` için aynı lambda örneğini değil aynı delegate'i vermemiz gerekir. Bu yüzden sonradan ayrılacağımız bir aboneliği adlı metotla ya da bir değişkende saklanan delegate ile yaparız:

```
EventHandler<SicaklikEventArgs> h = (g, e) => Console.WriteLine(e.Yeni);
sensor.SicaklikDegisti += h;
// ...
sensor.SicaklikDegisti -= h;     // aynı delegate nesnesi
```

> **▲ Sık Yapılan Hata**
>
> `sensor.SicaklikDegisti += (g, e) => ...;` satırını daha sonra aynı yazımla `-=` yapmaya çalışmak işe yaramaz: iki ayrı lambda yazımı iki ayrı delegate üretir ve `-=` hiçbir şeyi silmez (hata da vermez). Ayrılmayı planladığımız aboneliği adlandırılmış metotla ya da değişkende saklanan delegate'le yapalım.

Basit olay biçimleri

Her olay için `EventArgs` sınıfı yazmak zorunda değiliz. Yalnızca bildirim yeterliyse `event Action? Bitti;` ya da `event Action<double>? Degisti;` yazılabilir. Dahili ve küçük kodlarda bu yaygındır. Kütüphane

olarak dışarıya sunulan olaylarda ise `EventHandler<T>` deseni, genişlemeye açık olduğu için tercih edilir: sonradan `EventArgs`'a yeni bir özellik eklemek abonelerin imzasını bozmaz, oysa `Action<double>`'a parametre eklemek bütün abonelerin kodunu kırar.

55.6 Üç Dilde: Geri Çağırma

Üç Dilde · Metodu başka koda vermek *C · C++ · C#*

Üç dilde de "çağrılacak kodu sonradan bildirme" ihtiyacı vardır; yöntemler farklıdır.

	C	C++	C#
Araç	fonksiyon işaretçisi	`std::function`, lambda, işlev nesnesi	delegate, lambda
Tip denetimi	imza denetlenir, `void *` ile aşılabilir	şablon/imza ile denetlenir	tam tipli
Veri taşıma	ayrı `void *veri` parametresi	lambda yakalaması	closure ve hedef nesne
Çok alıcı	elle dizi tutulur	`vector<function<...>>` elle	`+=`, `-=` yerleşik
Olay modeli	yok (elle yazılır)	yok (elle yazılır)	`event` anahtar sözcüğü

C'de bir ölçüt fonksiyonunu parametre olarak geçirelim (Bölüm 12):

```c
int buyukten(const void *a, const void *b) {
    return *(const int *)b - *(const int *)a;
}
qsort(dizi, n, sizeof(int), buyukten);
```

C++'ta aynı işi lambda ve `std::function` ile yaparız (Bölüm 36):

```cpp
std::function<bool(int)> ciftMi = [](int n) { return n % 2 == 0; };
int say = std::count_if(v.begin(), v.end(), ciftMi);
```

C#'ta ise delegate ya da lambdayı doğrudan metoda veririz:

```csharp
Predicate<int> ciftMi = n => n % 2 == 0;
int say = liste.FindAll(ciftMi).Count;
```

`std::function` ile C# delegate'i en yakın karşılıklardır: ikisi de imzası uyan herhangi bir çağrılabilir şeyi tutar. C++ `std::function` ile lambda ayrı tiplerdir ve `std::function` ek bir nesne sarmalama maliyeti taşır; C# lambdası ise doğrudan bir delegate'e dönüşür. En büyük fark `event` ve çok noktaya yayındadır: C ve C++'ta abone listesini, ekleme ve silme kurallarını biz yazarız; C#'ta bunlar dilin içindedir.

55.7 Sık Hatalar ve İyi Uygulamalar

Delegate ve olaylarla çalışırken şu noktalara dikkat edelim:

1. **Olayı `null` denetimi olmadan tetiklemek.** Abonesi olmayan bir olayı doğrudan `Olay(this, e)` diye çağırırsak `NullReferenceException` alırız. Daima `Olay?.Invoke(this, e)` yazarız.
2. **Döngü değişkenini yakalamak.** `for` içinde lambda yazarken bölümün başında gördüğümüz `3 3 3` sorununa karşı yerel kopya alırız.
3. **Abonelikten çıkmayı unutmak.** Yayıncı abonenin ömründen uzun yaşıyorsa `-=` yazılmalıdır.

4. **Olay işleyicilerinde ağır iş yapmak.** İşleyici, yayıncının akışını bloke eder. Uzun işler ayrı iş parçacığına ya da `async` metoda devredilmelidir (Bölüm 59).
5. **Delegate yerine arayüz gerekebilir.** Bir nesneden birden fazla ilişkili geri çağırma bekliyorsak (başla, ilerle, bitir) bunları tek bir arayüze koymak (Bölüm 51) çoğu zaman üç ayrı delegate'ten daha düzenlidir.
6. **Lambdayı gereksiz uzatmak.** Gövdesi birkaç satırı aşan bir lambda adlı bir metoda taşınırsa hem okunur hem hata ayıklaması kolay olur.

Bölüm Özeti

- Delegate, imzası belirli metotları tutabilen referans tipidir; C fonksiyon işaretçisinin tip güvenli ve nesneye bağlanabilen karşılığıdır.
- Delegate tipi `delegate dönüşTipi Ad(parametreler);` ile bildirilir; değişkene metot adı atanır ve değişken metot gibi çağrılır.
- Delegate bir örnek metoda bağlanırsa metodun çalışacağı nesneyi de saklar (`Target`).
- `Action` dönüş değeri olmayan, `Func` dönüş değeri olan (son tip parametresi dönüş tipidir), `Predicate<T>` ise `bool` döndüren hazır genel delegate'lerdir.
- Lambda ifadesi `(parametreler) => ifade` ya da `=> { deyimler }` biçiminde yazılan adsız metottur; tipler genelde delegate tipinden çıkarılır.
- Lambda, dış değişkenleri kopyalamaz, değişkenin kendisine bağlanır (closure). Yakalanan değişkenin ömrünü çöp toplayıcı uzatır.
- `for` döngü değişkenini yakalayan lambdalar aynı değişkeni paylaşır; her turda yerel kopya alınmalıdır. `foreach` değişkeni her turda yenidir.
- Delegate'ler değiştirilemezdir ve `+=`, `-=` ile çok noktaya yayın yapılabilir; metotlar eklenme sırasıyla çalışır.
- Çok noktaya yayında dönüş değeri olarak yalnızca son metodun sonucu alınır; tümü için `GetInvocationList` kullanılır.
- `event`, delegate'in dışarıdan yalnızca `+=` ve `-=` ile kullanılmasına izin veren halidir; olayı yalnızca tanımlayan sınıf tetikleyebilir.
- .NET deseni `EventHandler<TEventArgs>` ile `(object? gonderen, TEventArgs e)` imzasını kullanır; veri `EventArgs`'tan türeyen sınıfla taşınır.
- Olay `Olay?.Invoke(this, e)` ile tetiklenir; tetikleme çoğunlukla `protected virtual OnXxx` metoduna konur.
- Olay işleyicileri yayıncının iş parçacığında sırayla çalışır; uzun süren ya da istisna fırlatan işleyici yayıncıyı etkiler. Ömrü bitmiş aboneler `-=` ile ayrılmalıdır.

Alıştırmalar

1. Aşağıdakilerden hangisi iki `int` alıp `string` döndüren bir delegate değişkenidir?
   - **A)** `Action<int, int> f;`
   - **B)** `Func<int, int, string> f;`
   - **C)** `Func<string, int, int> f;`

D) `Predicate<int, int> f;`
E) `Func<int, string> f;`

2. `Predicate<T>` delegate'i hangi imzaya sahiptir?

A) `void (T)`
B) `T ()`
C) `bool (T)`
D) `int (T, T)`
E) `T (T)`

3. Bir `event` tanımlandığında sınıfın dışından aşağıdakilerden hangisi yapılabilir?

A) Olayı doğrudan çağırmak
B) `=` ile `null` atamak
C) `+=` ile abone olmak
D) Abone listesini okumak
E) Hiçbiri

4. Bu programın çıktısı nedir?

```
int x = 2;
Func<int> f = () => x * 10;
x = 5;
Console.WriteLine(f());
```

5. Bu programın çıktısı nedir?

```
Action<string> a = s => Console.Write(s + "1 ");
a += s => Console.Write(s + "2 ");
a("k");
a -= s => Console.Write(s + "2 ");
a("m");
Console.WriteLine();
```

6. Bu programın çıktısı nedir?

```
var liste = new List<Action>();
for (int i = 0; i < 3; i++)
    liste.Add(() => Console.Write(i));
foreach (var a in liste) a();
Console.WriteLine();
```

7. Bu programın çıktısı nedir?

```
Func<int> f = () => 1;
f += () => 7;
Console.WriteLine(f());
Console.WriteLine(f.GetInvocationList().Length);
```

8. Aşağıdaki kodun hatasını bulun ve düzeltin.

```
delegate int Islem(int a, int b);
int Topla(int a) => a + 1;
Islem i = Topla;
```

9. Aşağıdaki kodun derlenmemesinin nedeni nedir?

```
class Dugme
{
    public event Action? Tiklandi;
}
var d = new Dugme();
d.Tiklandi();
```

10. `Func<int, int, double>` tipinin parametrelerini ve dönüş tipini yazın.
11. Bir olayı `Olay(this, e)` yerine `Olay?.Invoke(this, e)` ile tetiklemek neden önerilir?
12. `x => x * x` lambdasını iki farklı delegate tipine atayarak yazın (biri `int`, biri `double` için).
13. Yalnızca bildirim yapan bir olay için `Action` yerine neden çoğunlukla `EventHandler<T>` deseni önerilir? Kısaca açıklayın.
14. Kısa program: `Uygula(int[] dizi, Func<int, bool> kosul)` adında bir metot yazın; koşulu sağlayan elemanların toplamını döndürsün. Çift sayıların ve 10'dan büyüklerin toplamını lambda ile hesaplayın.
15. Kısa program: İçine eklenen her metni saklayan `MetinKutusu` sınıfı yazın; `Eklendi` adlı bir olay `EventHandler<string>` türünde olsun ve her ekleme sonrası eklenen metinle tetiklensin. İki abone ekleyip birini çıkarın.
16. Program yazma: Parametre olarak bir `Action` alıp onu üç kez çalıştıran `Tekrarla(Action eylem)` metodunu yazın. Bir sayacı yakalayan lambda ile çağırıp sayacın 3 olduğunu gösterin.

BÖLÜM 56

LINQ

BU BÖLÜMDE

- LINQ'nun ne olduğunu ve neden döngülerin yerine geçtiğini
- Yöntem sözdizimi ile sorgu sözdizimi arasındaki ilişkiyi
- `Where`, `Select`, `OrderBy`, `ThenBy` ile süzme, dönüştürme ve sıralamayı
- `GroupBy` ile gruplamayı ve gruplar üzerinde özet hesaplamayı
- `First`, `FirstOrDefault`, `Any`, `All`, `Count`, `Sum`, `Average`, `Min`, `Max`, `Distinct`, `Take`, `Skip`, `ToList` metotlarını
- Ertelenmiş yürütmeyi (deferred execution) ve ne zaman sorgunun gerçekten çalıştığını
- LINQ'nun C döngüleri ve C++ `<algorithm>` ile karşılaştırmasını

Bölüm 54'te verileri koleksiyonlara koymayı öğrendik. Bölüm 55'te davranışı bir değer gibi taşıyan delegate ve lambda ifadelerini gördük. Şimdi bu ikisini birleştireceğiz: koleksiyonun içindeki veriyi "kaç tane", "hangisi", "ortalaması ne", "gruplara ayırınca nasıl görünür" gibi sorularla sorgulayacağız.

Bunu döngü, `if` ve geçici değişkenlerle yapabiliriz; C'de hep böyle yaptık. C++'ta Bölüm 36'da `<algorithm>` ile aynı işleri daha kısa yazmayı öğrendik. C#'ın cevabı **LINQ**'dur (Language Integrated Query, dile gömülü sorgu). Amaç aynıdır: "nasıl yapılacağını" adım adım yazmak yerine "ne istediğimizi" söylemek.

56.1 LINQ Nedir?

Tanım: LINQ

Bellekteki koleksiyonlar (ve veritabanı tabloları, XML gibi başka veri kaynakları) üzerinde süzme, dönüştürme, sıralama, gruplama ve özetleme işlemlerini ortak bir metot ve söz dizimi kümesiyle yapmamızı sağlayan C# özelliğidir. Bellek içi koleksiyonlar için olan metotlar `System.Linq` ad alanındadır; .NET proje şablonu bu ad alanını örtük olarak ekler.

LINQ metotları `IEnumerable<T>` arayüzünün (Bölüm 51) üzerine kuruludur. `List<T>`, dizi, `HashSet<T>`, `Dictionary` ve `IEnumerable<T>` uygulayan kendi sınıfımız dahil `foreach` ile gezilebilen her şeyde LINQ çalışır. Bu metotlar, koleksiyon sınıflarının içinde tanımlı değildir; **genişletme metotlarıdır** (extension methods). Yani sınıfın kendisine dokunmadan, sanki sınıfa aitmiş gibi `liste.Where(...)` biçiminde çağırdığımız statik metotlardır.

İlk örneğimizde bir `int` dizisinden çift sayıları süzelim ve 8'den büyük sayıların karelerini alalım:

ilkLinq.cs

```
int[] sayilar = { 5, 12, 8, 21, 3, 16, 9 };

var ciftler = sayilar.Where(s => s % 2 == 0);
var kareler = sayilar.Where(s => s > 8).Select(s => s * s);

Console.WriteLine(string.Join(", ", ciftler));
Console.WriteLine(string.Join(", ", kareler));
```

```
ÇIKTI
12, 8, 16
144, 441, 256, 81
```

`Where` her eleman için lambdayı çağırır ve `true` dönenleri geçirir. `Select` her elemanı lambdanın döndürdüğü değere çevirir. İkisi de **yeni bir dizi döndürür**; `sayilar` dizisi değişmez. İki çağrıyı zincirleyebilmemizin nedeni budur: bir metodun çıktısı, sonrakinin girdisidir.

Yöntem sözdizimi ve sorgu sözdizimi

LINQ iki yazım sunar. Şimdiye kadar kullandığımız, nokta ile metot çağıran biçime **yöntem sözdizimi** (method syntax) denir. SQL'e benzeyen ikinci biçim **sorgu sözdizimi**dir (query syntax):

```
var kareler = from s in sayilar
              where s > 8
              select s * s;
```

İkisi aynı sonucu verir. Derleyici sorgu sözdizimini derleme sırasında yöntem sözdizimine çevirir; yani ikisi arasında başarım farkı yoktur. Bu kitapta çoğunlukla yöntem sözdizimini kullanacağız, çünkü LINQ'nun her işlemi bu biçimde ifade edilebilir. Sorgu sözdiziminde karşılığı olmayan metotlar (`Take`, `Skip`, `Any`, `Distinct` gibi) için zaten metot çağırmak gerekir ve çoğu zaman iki yazım karışık kullanılır.

> **Not · Hangisini seçmeli?**
>
> Çok sayıda ara değişkenin ve birleştirmenin (`join`) bulunduğu uzun sorgularda sorgu sözdizimi okunaklı olabilir. Kısa zincirlerde yöntem sözdizimi daha doğaldır. Takımın tercihi ne olursa olsun, ikisini de okuyabilmek gerekir.

56.2 Örnek Veri: Öğrenci Listesi

Bu bölümün büyük kısmında aynı veriyle çalışacağız. Bir `record` (Bölüm 52; kısa, değişmez veri sınıfı) ile öğrenci tanımlayalım:

```
record Ogrenci(string Ad, string Bolum, int Puan);
```

Aşağıdaki program listeyi kurar, puanı 60 ve üzerinde olanların adlarını yazar, sonra 85 ve üzerindekilerin adını ve bölümünü ekrana basar:

ogrenciSuz.cs

```
var liste = new List<Ogrenci>
{
    new("Ayşe", "Bilgisayar", 85),
```

```
    new("Mehmet", "Elektrik", 62),
    new("Elif", "Bilgisayar", 91),
    new("Can", "Makine", 48),
    new("Zeynep", "Elektrik", 77),
    new("Mert", "Makine", 55),
    new("Deniz", "Bilgisayar", 70),
    new("Selin", "Elektrik", 85),
};

var gecenler = liste.Where(o => o.Puan >= 60).Select(o => o.Ad);
Console.WriteLine("Geçenler: " + string.Join(", ", gecenler));

var iyiler = liste
    .Where(o => o.Puan >= 85)
    .Select(o => new { o.Ad, o.Bolum });
foreach (var i in iyiler)
{
    Console.WriteLine($"{i.Ad} ({i.Bolum})");
}

record Ogrenci(string Ad, string Bolum, int Puan);
```

```
ÇIKTI
Geçenler: Ayşe, Mehmet, Elif, Zeynep, Deniz, Selin
Ayşe (Bilgisayar)
Elif (Bilgisayar)
Selin (Elektrik)
```

İlk sorguda `Select` bir `Ogrenci` nesnesini yalnızca `Ad` alanına indirger; sonuç `IEnumerable<string>` olur. İkinci sorguda `new { o.Ad, o.Bolum }` bir **anonim tip** (anonymous type) üretir: adı olmayan, yalnızca okunabilir alanlardan oluşan, derleyicinin o noktada yarattığı küçük bir sınıf. Bu yüzden değişkeni `var` ile tanımlamak zorundayız; tipin adını yazamayız.

> **İpucu · Önce süz, sonra dönüştür**
>
> `Where` ile eleman sayısını azaltıp sonra `Select` ya da `OrderBy` uygulamak genellikle daha az iş yapar. Süzme sorgunun mümkün olduğunca başında olmalıdır.

56.3 Sıralama

`OrderBy` bir **anahtar seçici** (key selector) lambda alır ve elemanları o anahtara göre artan sırada dizer. Azalan sıra için `OrderByDescending` kullanılır. Anahtarlar eşit olduğunda ikinci ölçüt `ThenBy` ya da `ThenByDescending` ile eklenir.

▸ siralama.cs

```
var liste = new List<Ogrenci>
{
    new("Ayşe", "Bilgisayar", 85),
    new("Mehmet", "Elektrik", 62),
    new("Elif", "Bilgisayar", 91),
    new("Can", "Makine", 48),
    new("Zeynep", "Elektrik", 77),
    new("Mert", "Makine", 55),
    new("Deniz", "Bilgisayar", 70),
    new("Selin", "Elektrik", 85),
};
```

```
Console.WriteLine("Puana göre azalan, eşitlikte ada göre:");
foreach (var o in liste.OrderByDescending(o => o.Puan).ThenBy(o => o.Ad))
{
    Console.WriteLine($"  {o.Ad,-8}{o.Puan}");
}

Console.WriteLine("Bölüme göre, bölüm içinde puana göre azalan:");
var sirali = liste.OrderBy(o => o.Bolum).ThenByDescending(o => o.Puan);
foreach (var o in sirali)
{
    Console.WriteLine($"  {o.Bolum,-11}{o.Ad,-8}{o.Puan}");
}

record Ogrenci(string Ad, string Bolum, int Puan);
```

```
ÇIKTI
Puana göre azalan, eşitlikte ada göre:
  Elif    91
  Ayşe    85
  Selin   85
  Zeynep  77
  Deniz   70
  Mehmet  62
  Mert    55
  Can     48
Bölüme göre, bölüm içinde puana göre azalan:
  Bilgisayar Elif    91
  Bilgisayar Ayşe    85
  Bilgisayar Deniz   70
  Elektrik   Selin   85
  Elektrik   Zeynep  77
  Elektrik   Mehmet  62
  Makine     Mert    55
  Makine     Can     48
```

`OrderBy` **kararlıdır** (stable): anahtarları eşit olan elemanlar, kaynaktaki göreli sıralarını korur. Bölüm 54'te gördüğümüz `List<T>.Sort` ve C++'taki `std::sort` bu garantiyi vermez; kararlı sıralama için C++'ta `std::stable_sort` gerekiyordu (Bölüm 36). Ayrıca `OrderBy` kaynağı yerinde sıralamaz, **yeni bir sıralı dizi** döndürür. `liste` değişmeden kalır.

> **▲ Sık Yapılan Hata · ThenBy tek başına kullanılamaz**
>
> `ThenBy` yalnızca `OrderBy` ya da `OrderByDescending`'in sonucu üzerinde çağrılabilir. `liste.OrderBy(...).Where(...).ThenBy(...)` yazarsak `Where` araya girdiği için derleme hatası alırız. Birden fazla ölçüt varsa hepsini art arda, sıralama zincirinin içinde yazalım. Her ölçüt için yeniden `OrderBy` çağırmak ise önceki sıralamayı bozar ve yalnızca sonuncuyu geçerli kılar.

56.4 Sorgu Sözdizimiyle Aynı Sorgu

Sıralama sorgusunu sorgu sözdizimiyle de yazabiliriz. `orderby` ifadesinde virgülle ayrılan her ölçüt bir `ThenBy` olur; `descending` anahtar sözcüğü `...Descending` metoduna karşılık gelir. `select` sorgunun bittiği yerdir.

sorguSozdizimi.cs

```
var liste = new List<Ogrenci>
{
    new("Ayşe", "Bilgisayar", 85),
    new("Mehmet", "Elektrik", 62),
    new("Elif", "Bilgisayar", 91),
    new("Can", "Makine", 48),
    new("Zeynep", "Elektrik", 77),
    new("Mert", "Makine", 55),
    new("Deniz", "Bilgisayar", 70),
    new("Selin", "Elektrik", 85),
};

var sorgu = from o in liste
            where o.Puan >= 60
            orderby o.Puan descending, o.Ad
            select $"{o.Ad} {o.Puan}";

foreach (var satir in sorgu)
{
    Console.WriteLine(satir);
}

record Ogrenci(string Ad, string Bolum, int Puan);
```

```
ÇIKTI
Elif 91
Ayşe 85
Selin 85
Zeynep 77
Deniz 70
Mehmet 62
```

Aynı sorgu yöntem sözdizimiyle `liste.Where(o => o.Puan >= 60).OrderByDescending(o => o.Puan).ThenBy(o => o.Ad).Select(o => $"{o.Ad} {o.Puan}")` olurdu. Okuduğumuz her `from ... select` bloğunu zihnimizde bu zincire çevirebiliriz.

56.5 Gruplama: GroupBy

Elimizdeki öğrencileri bölümlerine göre ayırıp her bölümün ortalamasını bulmak istersek, C'de bir sözlük benzeri yapıyı elle kurmamız gerekirdi. `GroupBy` bunu tek çağrıyla yapar: anahtar seçici verilir, sonuçta **grup** dizisi döner.

> **Tanım: Grup (IGrouping)**
>
> `GroupBy`'ın döndürdüğü dizinin her elemanı bir `IGrouping<TKey, TElement>` nesnesidir. Grup, kendi anahtarını `Key` özelliğinde taşır ve aynı zamanda o anahtara sahip elemanların kendisi bir `IEnumerable<TElement>`'dir; yani içinde `Count()`, `Average()` gibi LINQ metotlarını kullanabiliriz.

gruplama.cs

```
var liste = new List<Ogrenci>
{
    new("Ayşe", "Bilgisayar", 85),
    new("Mehmet", "Elektrik", 62),
    new("Elif", "Bilgisayar", 91),
```

```
    new("Can", "Makine", 48),
    new("Zeynep", "Elektrik", 77),
    new("Mert", "Makine", 55),
    new("Deniz", "Bilgisayar", 70),
    new("Selin", "Elektrik", 85),
};

foreach (var g in liste.GroupBy(o => o.Bolum))
{
    string adlar = string.Join(", ", g.Select(o => o.Ad));
    Console.WriteLine($"{g.Key}: {g.Count()} kişi, ort. {g.Average(o => o.Puan):F1}");
    Console.WriteLine($"  {adlar}");
}

var ozet = liste
    .GroupBy(o => o.Bolum)
    .Select(g => new { Bolum = g.Key, En = g.Max(o => o.Puan) })
    .OrderByDescending(x => x.En);
foreach (var x in ozet)
{
    Console.WriteLine($"{x.Bolum} en yüksek: {x.En}");
}

record Ogrenci(string Ad, string Bolum, int Puan);
```

```
ÇIKTI
Bilgisayar: 3 kişi, ort. 82.0
  Ayşe, Elif, Deniz
Elektrik: 3 kişi, ort. 74.7
  Mehmet, Zeynep, Selin
Makine: 2 kişi, ort. 51.5
  Can, Mert
Bilgisayar en yüksek: 91
Elektrik en yüksek: 85
Makine en yüksek: 55
```

Gruplar, anahtarın kaynakta ilk göründüğü sırayla gelir; her grubun içindeki elemanlar da kaynaktaki sırayı korur. İkinci sorgu `GroupBy`'ı `Select` ile birleştirir: her grubu "bölüm adı ve en yüksek puan" çiftine indirger, sonra sıralar. Gruplayıp özetlemek, LINQ'nun en sık kullanılan kalıplarından biridir.

C'de ya da C++'ta aynı işi yapmak için önce bölümleri toplayan bir `map` ya da elle kurulmuş bir yapı gerekirdi. Burada ise "bölüme göre grupla, her grubu özetle" doğrudan kodun kendisi olur.

56.6 Tek Eleman Seçmek: First, FirstOrDefault, Single

Bazen dizinin tamamını değil, bir elemanını isteriz.

Metot	Ne yapar	Eleman yoksa
`First()` / `First(koşul)`	İlk (koşulu sağlayan ilk) elemanı verir	`InvalidOperationException`
`FirstOrDefault()` / `FirstOrDefault(koşul)`	Aynısı	Tipin varsayılan değerini verir (`null` , `0` , `false`)
`Single(koşul)`	Tek eleman beklenir ve onu verir	Hiç ya da birden çok eleman varsa istisna
`Last()` , `LastOrDefault()`	Sondaki eleman	`First` ile aynı kural

Metot	Ne yapar	Eleman yoksa
`ElementAt(i)`	`i`. eleman	`ArgumentOutOfRangeException`

tekEleman.cs

```
var liste = new List<Ogrenci>
{
    new("Ayşe", "Bilgisayar", 85),
    new("Can", "Makine", 48),
    new("Mert", "Makine", 55),
};

var ilkMakine = liste.First(o => o.Bolum == "Makine");
Console.WriteLine(ilkMakine.Ad);

var cokYuksek = liste.FirstOrDefault(o => o.Puan > 95);
Console.WriteLine(cokYuksek is null ? "Bulunamadı" : cokYuksek.Ad);

var enBasarili = liste.MaxBy(o => o.Puan);
Console.WriteLine(enBasarili?.Ad);

try
{
    liste.First(o => o.Puan > 95);
}
catch (InvalidOperationException e)
{
    Console.WriteLine("Hata: " + e.Message);
}

record Ogrenci(string Ad, string Bolum, int Puan);
```

```
ÇIKTI
Can
Bulunamadı
Ayşe
Hata: Sequence contains no matching element
```

`FirstOrDefault` sınıf tiplerinde `null` verir; sonucu kullanmadan önce `null` denetimi şarttır (`is null` ve `?.` kalıpları burada işe yarar). `int` gibi değer tiplerinde ise `0` döner ve bu değer, dizide gerçekten `0` olan bir elemandan ayırt edilemez. Böyle durumlarda `Any` ile önce varlığı sormak ya da `First` kullanıp istisnayı yönetmek daha güvenlidir.

`MaxBy` ve `MinBy` (.NET 6'dan itibaren), en büyük **değeri** değil, en büyük değere sahip **elemanı** verir. `Max(o => o.Puan)` yalnızca `91` döndürür; `MaxBy(o => o.Puan)` ise 91 alan öğrenci nesnesini döndürür.

56.7 Soru Soran ve Sayan Metotlar

Aşağıdaki metotlar diziyi tek bir değere indirger. Bir kısmı `bool`, bir kısmı sayı döndürür:

Metot	Anlamı
`Any()` / `Any(koşul)`	En az bir eleman (koşulu sağlayan) var mı?
`All(koşul)`	Bütün elemanlar koşulu sağlıyor mu?
`Contains(değer)`	Değer dizide var mı?

Metot	Anlamı
`Count()` / `Count(koşul)`	Eleman sayısı
`Sum` , `Average`	Toplam, aritmetik ortalama
`Min` , `Max`	En küçük, en büyük değer
`Aggregate`	Kendi biriktirme işlevimizle indirgeme

ozetMetotlar.cs

```
var puanlar = new List<int> { 85, 62, 91, 48, 77, 55, 70, 85 };

Console.WriteLine($"Adet: {puanlar.Count()}, toplam: {puanlar.Sum()}");
Console.WriteLine($"Ortalama: {puanlar.Average()}");
Console.WriteLine($"En düşük: {puanlar.Min()}, en yüksek: {puanlar.Max()}");
Console.WriteLine($"60 ve üzeri: {puanlar.Count(p => p >= 60)} kişi");
Console.WriteLine($"50 altı var mı: {puanlar.Any(p => p < 50)}");
Console.WriteLine($"Hepsi 40 üstü mü: {puanlar.All(p => p > 40)}");
Console.WriteLine($"Hepsi 60 üstü mü: {puanlar.All(p => p > 60)}");

int carpim = puanlar.Take(3).Aggregate(1, (c, p) => c * p);
Console.WriteLine($"İlk üçün çarpımı: {carpim}");
```

```
ÇIKTI
Adet: 8, toplam: 573
Ortalama: 71.625
En düşük: 48, en yüksek: 91
60 ve üzeri: 6 kişi
50 altı var mı: True
Hepsi 40 üstü mü: True
Hepsi 60 üstü mü: False
İlk üçün çarpımı: 479570
```

`Average` her zaman ondalıklı sonuç verir (`int` dizide `double`); tamsayı bölmesi tuzağına düşmeyiz. `Aggregate` ise C++'taki `std::accumulate`'in karşılığıdır: başlangıç değeri (`1`) ve her adımda birikimi güncelleyen bir lambda alır.

İpucu · Count() > 0 yerine Any()

"Listede eleman var mı?" sorusu için `Any()` kullanalım. `Count() > 0` , kaynak bir `List<T>` ise hızlıdır; ama bazı kaynaklarda bütün elemanları sayar. `Any()` ise ilk elemanı görünce durur. Listelerde özellik olarak doğrudan `liste.Count` da kullanılabilir (parantezsiz).

▲ Sık Yapılan Hata · Boş dizide Average, Min ve Max

`Average()` , `Min()` ve `Max()` boş bir `int` dizisinde `InvalidOperationException` fırlatır; ortalaması olmayan bir şeyin ortalamasını döndürmeye çalışmak anlamsızdır. Veri boş olabiliyorsa önce `Any()` ile denetleyelim ya da `nullable` kullanıp `puanlar.Select(p => (int?)p).Average()` yazalım; bu durumda sonuç `null` olur. `Sum()` ise boş dizide `0` verir.

56.8 Tekrarsız Yapmak, Dilimlemek ve Sonucu Saklamak

`Distinct()` tekrar eden değerleri atar. `Take(n)` ilk `n` elemanı alır, `Skip(n)` ilk `n` elemanı atlar. İkisini birleştirmek **sayfalama** (pagination) yapar. Sorgunun sonucunu kalıcı bir koleksiyona çevirmek için `ToList()`, `ToArray()` ya da `ToDictionary()` kullanılır.

▸ dilimleme.cs

```
var liste = new List<Ogrenci>
{
    new("Ayşe", "Bilgisayar", 85),
    new("Mehmet", "Elektrik", 62),
    new("Elif", "Bilgisayar", 91),
    new("Can", "Makine", 48),
    new("Zeynep", "Elektrik", 77),
    new("Mert", "Makine", 55),
    new("Deniz", "Bilgisayar", 70),
    new("Selin", "Elektrik", 85),
};

var bolumler = liste.Select(o => o.Bolum).Distinct();
Console.WriteLine("Bölümler: " + string.Join(", ", bolumler));

var sirali = liste.OrderByDescending(o => o.Puan).ThenBy(o => o.Ad);
const int sayfaBoyu = 3;
for (int sayfa = 0; sayfa < 3; sayfa++)
{
    var dilim = sirali.Skip(sayfa * sayfaBoyu).Take(sayfaBoyu);
    Console.WriteLine($"Sayfa {sayfa + 1}: " + string.Join(", ", dilim.Select(o => o.Ad)));
}

List<string> adlar = liste.Select(o => o.Ad).ToList();
Dictionary<string, int> puanSozlugu = liste.ToDictionary(o => o.Ad, o => o.Puan);
Console.WriteLine($"{adlar.Count} ad, Can'ın puanı {puanSozlugu["Can"]}");

record Ogrenci(string Ad, string Bolum, int Puan);
```

```
ÇIKTI
Bölümler: Bilgisayar, Elektrik, Makine
Sayfa 1: Elif, Ayşe, Selin
Sayfa 2: Zeynep, Deniz, Mehmet
Sayfa 3: Mert, Can
8 ad, Can'ın puanı 48
```

`Skip` ve `Take` sınır dışına taşmaktan korkmaz: son sayfada eleman azsa kalanı verir, hiç yoksa boş dizi döner. `ToDictionary` anahtarların benzersiz olmasını bekler; aynı anahtardan iki eleman varsa `ArgumentException` fırlatır. Anahtar tekrar edebiliyorsa `GroupBy` ya da `ToLookup` kullanılır.

56.9 Ertelenmiş Yürütme

LINQ'nun en önemli, en çok yanlış anlaşılan özelliği şudur: `Where`, `Select`, `OrderBy` gibi metotlar sorguyu **hemen çalıştırmaz**. Yalnızca "ne yapılacağını" tarif eden bir nesne döndürürler. İşlem, sonuç gerçekten **istendiği anda** (`foreach` ile gezerken, `ToList()`, `Count()`, `Sum()`, `First()` gibi metotlar çağrıldığında) yapılır.

> **Tanım: Ertelenmiş yürütme (deferred execution)**
>
> Sorgu tanımlandığı satırda değil, sonucu ilk kez tüketildiği anda çalışır. Sorgu her tüketildiğinde kaynak veriden baştan yeniden hesaplanır. Sonucu hemen üretip saklayan metotlara (`ToList`, `ToArray`, `ToDictionary`, `Count`, `Sum`, `First` ...) **anında yürütme** (immediate execution) metotları denir.

Davranışı izlemek için lambdanın içine bir yazdırma koyalım:

ertelenmis.cs

```
var sayilar = new List<int> { 1, 2, 3 };

var buyukler = sayilar.Where(s =>
{
    Console.WriteLine($"  denetim: {s}");
    return s > 1;
});

Console.WriteLine("Sorgu tanımlandı");
sayilar.Add(4);

foreach (var s in buyukler)
{
    Console.WriteLine($"sonuç: {s}");
}
```

ÇIKTI

```
Sorgu tanımlandı
  denetim: 1
  denetim: 2
sonuç: 2
  denetim: 3
sonuç: 3
  denetim: 4
sonuç: 4
```

Üç gözlem çıkarabiliriz. Birincisi, "Sorgu tanımlandı" satırından önce hiçbir denetim yapılmadı. İkincisi, listeye sorgudan **sonra** eklediğimiz `4` sonuca girdi; çünkü sorgu o ana kadar kaynağın bir kopyasını almadı, kaynağa bakmaya hazır bekledi. Üçüncüsü, denetim ve sonuç satırları iç içe geçti: eleman eleman akış vardır, önce hepsi süzülüp sonra yazdırılmaz.

Bu davranışın faydası büyüktür. Zincirdeki her metot ara bir koleksiyon üretmez; elemanlar zincirin içinden tek tek geçer. `Take(3)` gibi bir metot yeterli sayıda eleman alınca kaynağı okumayı bırakır. Aşağıdaki sorgu, sonsuza yakın büyüklükteki bir aralıktan yalnızca üç eleman için çalışır:

sonsuz.cs

```
var ilkUc = Enumerable.Range(1, int.MaxValue)
    .Where(x => x % 7 == 0)
    .Take(3);

Console.WriteLine(string.Join(", ", ilkUc));
```

ÇIKTI

```
7, 14, 21
```

`Enumerable.Range(başlangıç, adet)` bir sayı dizisi üretir, ama bellekte iki milyar sayılık bir dizi yaratmaz; elemanlar istendikçe üretilir.

Anında yürütme ve anlık görüntü

Sorgunun sonucunu belli bir anda dondurmak istiyorsak `ToList()` ya da `ToArray()` çağırırız. Bu andan sonra kaynaktaki değişiklikler sonucu etkilemez:

anlikGoruntu.cs

```
var sayilar = new List<int> { 1, 2, 3 };

var canli = sayilar.Select(s => s * 10);
var kopya = sayilar.Select(s => s * 10).ToList();

sayilar.Add(4);

Console.WriteLine("canlı : " + string.Join(", ", canli));
Console.WriteLine("kopya : " + string.Join(", ", kopya));
```

ÇIKTI

```
canlı : 10, 20, 30, 40
kopya : 10, 20, 30
```

◆ Kritik Nokta · Her tüketim sorguyu yeniden çalıştırır

Ertelenmiş bir sorguyu iki kez gezersek (örneğin önce `Count()`, sonra `foreach`), iki seferde de kaynak baştan okunur ve süzme, dönüştürme lambdaları yeniden çalışır. Lambdanın yan etkisi (yazdırma, sayaç artırma, veritabanı çağrısı) varsa her seferinde tekrarlanır. Sonuca birden çok kez ihtiyaç varsa `ToList()` ile bir kez hesaplayıp listeyi kullanalım.

▲ Sık Yapılan Hata · Gezerken kaynağı değiştirmek

Bir sorgunun sonucunu `foreach` ile gezerken kaynak listeye eleman ekler ya da silersek `InvalidOperationException` ("Collection was modified") alırız; Bölüm 54'te `List<T>` için gördüğümüz hatanın aynısıdır. Güvenli yol, önce `ToList()` ile sonucun bir kopyasını almak, sonra kopyayı gezerek asıl listeyi değiştirmektir.

◆ Kritik Nokta · Lambdanın yakaladığı değişken sorguyla birlikte yaşar

Sorgunun lambdası dışarıdaki bir değişkeni yakalarsa (Bölüm 55'teki kapanış), değişkenin değeri sorgu **çalıştığı anda** okunur, tanımlandığı anda değil. `int esik = 60; var q = liste.Where(o => o.Puan >= esik); esik = 80;` yazdıktan sonra `q` gezildiğinde süzme `80` ile yapılır.

56.10 Üç Dilde: Aynı İş, Üç Yol

Aynı işi üç dilde yazalım: bir `int` dizisinde çift sayıların karelerinin toplamını bulmak. Dizi `{ 5, 12, 8, 21, 3, 16, 9 }`; çift sayılar 12, 8, 16 olduğundan sonuç `144 + 64 + 256 = 464`.

Üç Dilde · Çift sayıların kareler toplamı *C · C++ · C#*

C (döngü): Her adımı kendimiz yazarız; süzme, dönüştürme ve toplama bir döngünün içinde iç içe geçer. İleride C++'ta bu işin algoritmalara, C#'ta zincirlenmiş metotlara dönüştüğünü göreceğiz.

```
int toplam = 0;
for (int i = 0; i < n; i++) {
    if (sayilar[i] % 2 == 0) {
        toplam += sayilar[i] * sayilar[i];
    }
}
```

C++ (Bölüm 36): `<numeric>` içindeki `std::accumulate` ve bir lambda ile döngü kalktı; ama süzme ile toplama aynı lambdanın içinde. Yineleyici çifti (`begin()` , `end()`) her çağrıda yazılır.

```
int toplam = std::accumulate(v.begin(), v.end(), 0,
    [](int t, int s) { return s % 2 == 0 ? t + s * s : t; });
```

C#: Her adım ayrı bir metot; okuma sırası işin sırasıdır (süz, dönüştür, topla).

```
int toplam = sayilar.Where(s => s % 2 == 0).Select(s => s * s).Sum();
```

Fark yalnızca kısalık değildir. C döngüsü ve `std::accumulate` kaynağı geçerken bir sonuç biriktirir; LINQ ise her adımı bağımsız ve adı olan bir işlem olarak tarif eder, sorgunun parçaları başka sorgularda yeniden kullanılabilir. Karşılığı, bu soyutlamanın küçük bir çalışma zamanı maliyeti olmasıdır: her metot bir yineleyici nesnesi ve her lambda bir delegate çağrısı demektir. Çok sıcak bir döngüde bu fark ölçülebilir; olağan uygulama kodunda ise okunabilirlik çoğunlukla ağır basar.

Bölüm 36'daki algoritmaların LINQ karşılıkları şöyledir:

C++ `<algorithm>` , `<numeric>`	LINQ	Not
`std::copy_if`	`Where`	LINQ yeni dizi verir, kaynağı değiştirmez
`std::transform`	`Select`	Çıktı tipi farklı olabilir
`std::sort` , `std::stable_sort`	`OrderBy` , `ThenBy`	`OrderBy` kararlıdır, kaynağı yerinde sıralamaz
`std::find_if`	`FirstOrDefault`	Yineleyici yerine eleman ya da `null` döner
`std::any_of` , `all_of`	`Any` , `All`	Aynı anlam
`std::count_if`	`Count(koşul)`	
`std::accumulate`	`Aggregate` , `Sum`	`Sum` , `Average` hazır özel durumlar
`std::min_element`	`MinBy` , `Min`	`MinBy` eleman verir
`std::unique` (sıralı dizide)	`Distinct`	`Distinct` sıralı olmayan dizide de çalışır

İki temel fark vardır. İlki, C++ algoritmaları yineleyici çiftiyle çalışır ve çoğu kapsayıcıyı **yerinde** değiştirir (`sort` , `erase` / `remove_if`). LINQ ise kaynağa dokunmaz; sonuç hep yeni bir dizidir. İkincisi, C++17 algoritmaları anında çalışır; ertelenmiş çalışma C++20 `std::ranges` görünümleriyle (`views::filter` , `views::transform`) gelmiştir ve LINQ'ya çok benzer bir biçimde zincirlenir.

56.11 Gerçekçi Bir Örnek: Ürün Stok Raporu

Bu bölümde gördüklerimizi bir mağaza stoğu üzerinde birleştirelim. `Urun` kaydında ad, kategori, birim fiyat ve stok adedi var. Para değerleri için `decimal` kullanıyoruz (Bölüm 42). Rapor üç soruya cevap verecek: hangi ürünler tükenmiş, kategorilerin stok değeri nedir, her kategorinin en pahalı ürünü hangisi?

▸ stokRaporu.cs

```
var urunler = new List<Urun>
{
    new("Kalem", "Kırtasiye", 5.50m, 120),
    new("Defter", "Kırtasiye", 24.90m, 40),
    new("Silgi", "Kırtasiye", 3.25m, 0),
    new("Klavye", "Elektronik", 450m, 12),
    new("Fare", "Elektronik", 199.90m, 0),
    new("Kulaklık", "Elektronik", 320m, 7),
    new("Lamba", "Ev", 89.50m, 15),
    new("Termos", "Ev", 140m, 0),
};

var tukenenler = urunler.Where(u => u.Stok == 0).Select(u => u.Ad);
Console.WriteLine("Tükenenler: " + string.Join(", ", tukenenler));

Console.WriteLine("Stok değeri:");
var degerler = urunler
    .GroupBy(u => u.Kategori)
    .Select(g => new { Kategori = g.Key, Deger = g.Sum(u => u.Fiyat * u.Stok) })
    .OrderByDescending(x => x.Deger);
foreach (var x in degerler)
{
    Console.WriteLine($"  {x.Kategori,-11}{x.Deger,10:F2}");
}

Console.WriteLine("En pahalı ürünler:");
foreach (var g in urunler.GroupBy(u => u.Kategori))
{
    var pahali = g.MaxBy(u => u.Fiyat)!;
    Console.WriteLine($"  {g.Key}: {pahali.Ad} ({pahali.Fiyat} TL)");
}

bool hepsiFiyatli = urunler.All(u => u.Fiyat > 0);
int kritik = urunler.Count(u => u.Stok > 0 && u.Stok < 10);
Console.WriteLine($"Hepsi fiyatlı: {hepsiFiyatli}, kritik stok: {kritik}");

record Urun(string Ad, string Kategori, decimal Fiyat, int Stok);
```

```
ÇIKTI
Tükenenler: Silgi, Fare, Termos
Stok değeri:
  Elektronik    7640.00
  Kırtasiye     1656.00
  Ev            1342.50
En pahalı ürünler:
  Kırtasiye: Defter (24.90 TL)
  Elektronik: Klavye (450 TL)
  Ev: Termos (140 TL)
Hepsi fiyatlı: True, kritik stok: 1
```

`GroupBy` her kullanımda kaynağı yeniden gruplar (ertelenmiş yürütme); raporun çok büyük veriyle çalışması gerekseydi gruplamayı bir kez yapıp `ToList()` ile saklardık. `MaxBy(...)!` sonundaki `!` işareti, derleyiciye

"bu sonuç `null` olmayacak" demenin yoludur: grup en az bir eleman içerdiği için `MaxBy` burada `null` dönmez.

Bölüm Özeti

- LINQ, koleksiyonlar üzerinde süzme, dönüştürme, sıralama, gruplama ve özetleme yapan genişletme metotları kümesidir; `IEnumerable<T>` uygulayan her şeyle çalışır.
- İki yazım vardır: yöntem sözdizimi (`liste.Where(...).Select(...)`) ve sorgu sözdizimi (`from ... where ... select`). Derleyici ikincisini birinciye çevirir.
- `Where` süzer, `Select` dönüştürür; ikisi de yeni dizi döndürür ve kaynağı değiştirmez. Süzmeyi mümkün olduğunca başa almak iyi uygulamadır.
- `OrderBy` / `OrderByDescending` ile `ThenBy` / `ThenByDescending` çok ölçütlü sıralama yapar; `OrderBy` kararlıdır.
- `GroupBy`, `Key` özelliği olan gruplar üretir; her grup kendi içinde `Count`, `Sum`, `Average` gibi metotlarla özetlenebilir.
- `First` eleman bulunamazsa istisna fırlatır; `FirstOrDefault` varsayılan değer verir. `MaxBy` / `MinBy` en büyük/küçük değeri değil elemanı döndürür.
- `Any`, `All`, `Contains` `bool`; `Count`, `Sum`, `Average`, `Min`, `Max` tek değer verir. Boş dizide `Average`, `Min`, `Max` istisna fırlatır.
- `Distinct` tekrarları atar; `Skip` ve `Take` dilimler ve sayfalama yapar; `ToList`, `ToArray`, `ToDictionary` sonucu saklar.
- Sorgular ertelenmiş yürütülür: sonuç ilk tüketildiği anda hesaplanır ve her tüketimde yeniden çalışır. Kaynakta sorgudan sonra yapılan değişiklikler sonuca yansır.
- `ToList()` gibi anında yürütme metotları sonucu dondurur; birden çok kez gezilecek sonuçlar için tercih edilir.
- LINQ, C++ `<algorithm>`'in karşılığıdır (`copy_if` / `Where`, `transform` / `Select`, `accumulate` / `Aggregate`) ama kaynağı yerinde değiştirmez ve ertelenmiş çalışır.
- Lambdanın yakaladığı değişkenler sorgu çalıştığı anda okunur.

Alıştırmalar

1. `int[] a = { 4, 9, 2, 7 };` için `a.Where(x => x > 3).Select(x => x * 2)` sorgusunun sonucu hangisidir?

   A) 8, 18, 14
   B) 8, 4, 18, 14
   C) 4, 9, 7
   D) 8, 18, 4, 14
   E) 9, 7

2. Aşağıdakilerden hangisi **anında yürütme** yapar?

   A) `Where`
   B) `Select`

C) `OrderBy`
D) `Count`
E) `Skip`

3. `liste.Where(...)` yazıp sonucu hiçbir yerde kullanmazsak ne olur?

A) Süzme yapılır, liste değişir
B) Süzme yapılır, sonuç atılır
C) Hiçbir şey çalışmaz
D) Derleme hatası oluşur
E) Çalışma zamanı istisnası oluşur

4. Boş bir `List<int>` için hangi çağrı istisna **fırlatmaz**?

A) `Average()`
B) `Max()`
C) `First()`
D) `Sum()`
E) `Single()`

5. Aşağıdaki program ekrana ne yazdırır?

```
var l = new List<int> { 1, 2, 3 };
var q = l.Select(x => x * x);
l.Add(4);
Console.WriteLine(q.Sum());
```

6. Aşağıdaki program ekrana ne yazdırır?

```
string[] ad = { "Ece", "Can", "Ali", "Zeynep", "Mert" };
var s = ad.Where(a => a.Length == 3).OrderBy(a => a);
Console.WriteLine(string.Join(",", s));
Console.WriteLine(ad.Skip(1).Take(2).Count());
```

7. Aşağıdaki program ekrana ne yazdırır?

```
int esik = 3;
var q = new[] { 1, 2, 3, 4, 5 }.Where(x => x > esik);
esik = 1;
Console.WriteLine(q.Count());
```

8. Aşağıdaki program ekrana ne yazdırır?

```
var k = new[] { "elma", "armut", "erik", "ayva", "kiraz" };
foreach (var g in k.GroupBy(x => x[0]))
{
    Console.WriteLine($"{g.Key}:{g.Count()}");
}
```

9. Aşağıdaki kodda derleme hatası olan satırı bulun ve düzeltin.

```
var sayilar = new List<int> { 3, 1, 2 };
var s = sayilar.OrderBy(x => x).Where(x => x > 1).ThenBy(x => -x);
```

10. Aşağıdaki kod çalışma zamanında hata verir. Nedenini açıklayın ve düzeltin.

```
var l = new List<int> { 1, 2, 3, 4 };
foreach (var x in l.Where(x => x % 2 == 0))
{
    l.Remove(x);
}
```

11. `FirstOrDefault` ile `First` arasındaki fark nedir? `int` dizilerinde `FirstOrDefault` kullanmak neden sorun yaratabilir?
12. `x.Where(...)` sorgusu iki kez `foreach` ile gezilirse lambda kaç kez çalışır? Bunu önlemek için ne yapılır?
13. `int[] a = { 3, -1, 4, -1, 5, 9, -2 };` dizisinde negatif sayıları atıp kalanların toplamını ve ortalamasını yazdıran bir program yazın.
14. Bir `string` listesindeki kelimeleri büyük/küçük harf farkı gözetmeden tekrarsız hâle getirip uzunluğa göre, eşitlikte alfabetik olarak sıralayan ve virgülle ayırarak yazdıran bir sorgu yazın. Veri: `{ "elma", "Armut", "ELMA", "kiraz", "armut", "Fig" }`.
15. `record Calisan(string Ad, string Departman, decimal Maas)` kaydı ve en az altı elemanlı bir liste için; her departmanın ortalama maaşını, ortalamaya göre azalan sırada, `Departman: ortalama` biçiminde yazdıran program yazın.
16. Aynı `Calisan` listesinde en yüksek maaşlı üç çalışanı ve her departmanın en düşük maaşlı çalışanını yazdıran program yazın. Ardından `foreach` yerine aynı işi C++ `<algorithm>` ile nasıl yapacağınızı iki cümleyle anlatın.

BÖLÜM 57

Dosya İşlemleri

BU BÖLÜMDE

- `File` sınıfının tek çağrılık metotlarıyla (`WriteAllText`, `ReadAllLines`, `AppendAllText`, `Exists`) dosya yazıp okumayı
- Dosya işlemlerinde oluşan hataları (`FileNotFoundException`, `IOException`) yakalamayı
- `StreamWriter` ve `StreamReader` ile satır satır çalışmayı, `using` ile dosyayı güvenle kapatmayı
- `Path`, `Directory` ve `FileInfo` ile yolları, klasörleri ve dosya bilgilerini yönetmeyi
- CSV biçimindeki kayıtları nesnelere dönüştürüp geri yazmayı
- `System.Text.Json` ile nesneleri JSON olarak kaydetmeyi ve geri okumayı
- C'deki `FILE *` ve C++'taki `fstream` ile C# yaklaşımını karşılaştırmayı

Programların değişkenlerde tuttuğu veri, program kapanınca yok olur. Bir not listesinin, bir ayar dosyasının ya da bir envanterin bir sonraki çalıştırmada da var olması için veriyi kalıcı ortama, yani dosyaya yazmamız gerekir.

Dosya kavramını, metin ve ikili dosya ayrımını Bölüm 17'de C ile, `fstream` akışlarını Bölüm 37'de C++ ile gördük; burada tekrar anlatmayacağız. C# bu işi iki katmanda sunar: tek çağrıyla bütün dosyayı okuyan ya da yazan `File` metotları ve bayt dizisini akış (stream) olarak işleyen `StreamReader` / `StreamWriter` sınıfları. Her şey `System.IO` ad alanındadır. Proje şablonu bu ad alanını örtük olarak eklediği için kitaptaki örneklerde ayrıca `using System.IO;` yazmayacağız.

Bu bölümdeki programlar çalışma klasöründe dosya oluşturur. Çalışma klasörü kavramını az sonra inceleyeceğiz.

57.1 File Sınıfı ile Yazma ve Okuma

`File`, statik metotlardan oluşan bir sınıftır; nesne oluşturmadan kullanılır. Dosyayı açma, kodlama ve kapatma işlerini bizim yerimize yapar. Küçük ve orta boyutlu metin dosyaları için çoğu zaman yeterlidir.

> **Tanım: File sınıfı**
>
> `System.IO.File`, bir dosyayı tek çağrıda oluşturan, okuyan, yazan, kopyalayan ya da silen statik metotlar sunar. Metin metotları varsayılan olarak UTF-8 kodlamasını kullanır.

En basit örnek, bir dize yazıp geri okumaktır:

yazOku.cs

```
File.WriteAllText("selam.txt", "Merhaba dosya!");
string icerik = File.ReadAllText("selam.txt");
Console.WriteLine(icerik);
Console.WriteLine(icerik.Length);
```

ÇIKTI

```
Merhaba dosya!
14
```

`WriteAllText` dosya yoksa oluşturur, varsa **içeriğini silip üzerine yazar**. `ReadAllText` bütün dosyayı tek bir `string` olarak döndürür. Dosyayı kapatmakla uğraşmadık; metot bunu kendisi yaptı. C'de `fopen` / `fclose` çiftini hatırlayanlar için büyük bir kolaylıktır.

> **▲ Sık Yapılan Hata · Yazma metodu mevcut dosyayı ezer**
>
> `WriteAllText` ve `WriteAllLines` varsa eski içeriği uyarmadan siler. Var olan dosyaya ekleme yapmak istiyorsak `AppendAllText` ya da `AppendAllLines` kullanmalıyız. C'deki `"w"` ve `"a"` kipleri arasındaki fark burada metot adına taşınmıştır.

Satır satır çalışma

Metin dosyaları genellikle satırlardan oluşur. `WriteAllLines` bir dize koleksiyonunu satır satır yazar, `ReadAllLines` dosyayı `string[]` olarak döndürür:

satirlar.cs

```
string[] sehirler = { "Ankara", "İzmir", "Bursa" };
File.WriteAllLines("sehirler.txt", sehirler);

File.AppendAllText("sehirler.txt", "Adana" + Environment.NewLine);
File.AppendAllLines("sehirler.txt", new[] { "Konya", "Mersin" });

string[] okunan = File.ReadAllLines("sehirler.txt");
Console.WriteLine($"Satır sayısı: {okunan.Length}");
for (int i = 0; i < okunan.Length; i++)
{
    Console.WriteLine($"{i + 1}. {okunan[i]}");
}
```

ÇIKTI

```
Satır sayısı: 6
1. Ankara
2. İzmir
3. Bursa
4. Adana
5. Konya
6. Mersin
```

`Environment.NewLine` işletim sistemine uygun satır sonunu verir (Windows'ta `\r\n`, Linux ve macOS'ta `\n`). `ReadAllLines` iki biçimi de doğru ayırır. `AppendAllText` satır sonu eklemez; onu biz yazmazsak sonraki metin aynı satıra yapışır.

Çok büyük dosyalarda `ReadAllLines` bütün satırları belleğe alır. Bunun yerine `File.ReadLines` satırları tembel (lazy) biçimde, istendikçe üretir ve LINQ ile birlikte kullanılabilir (Bölüm 56):

tembelOku.cs

```
File.WriteAllLines("sayilar.txt", new[] { "5", "12", "7", "30", "2" });

int buyukSayisi = File.ReadLines("sayilar.txt")
    .Select(int.Parse)
    .Count(n => n > 6);
Console.WriteLine($"6'dan büyük: {buyukSayisi}");
```

```
ÇIKTI
6'dan büyük: 3
```

Dosya var mı?

`File.Exists(yol)`, yolun bir **dosyayı** gösterip göstermediğini `bool` olarak döndürür. Yol bir klasörü gösteriyorsa ya da hiç yoksa `false` verir. Hata fırlatmaz. `File.Delete` dosyayı siler; dosya yoksa sessizce geçer.

varMi.cs

```
string ad = "gecici.txt";
Console.WriteLine(File.Exists(ad));
File.WriteAllText(ad, "geçici");
Console.WriteLine(File.Exists(ad));
File.Delete(ad);
Console.WriteLine(File.Exists(ad));
```

```
ÇIKTI
False
True
False
```

`File.Copy(kaynak, hedef)` ve `File.Move(kaynak, hedef)` dosyayı kopyalar ya da taşır. Hedef varsa hata oluşur; `File.Copy(kaynak, hedef, true)` ile üzerine yazılmasına izin verebiliriz.

> **▲ Sık Yapılan Hata · Exists ile sonra açmak arasında yarış vardır**
>
> `if (File.Exists(ad)) { ... File.ReadAllText(ad) ... }` kalıbı hatayı tamamen önlemez: kontrolle okuma arasında dosya başka bir program tarafından silinebilir ya da kilitlenebilir. Güvenilir yol, işlemi `try` / `catch` içinde yapmaktır. `Exists` kullanıcıya düzgün bir mesaj vermek için yine de yararlıdır.

57.2 Dosya Yolları, Çalışma Klasörü ve Path

Şimdiye kadar yalnızca dosya adı verdik: `"selam.txt"`. Böyle **göreli** (relative) bir yol, programın **geçerli çalışma klasörüne** göre yorumlanır. Çalışma klasörü, programı hangi klasörden başlattığımıza bağlıdır; çoğu zaman kaynak kodun ya da çalıştırılabilir dosyanın bulunduğu yer değildir. Örnekler çalıştırdığımız klasörde dosya oluşturur.

Bu klasörü `Directory.GetCurrentDirectory()` ile öğrenebiliriz. Yolun tamamını isteyen **mutlak** (absolute) yol ise sürücü ya da kökle başlar (`C:\veri\not.txt`, `/home/ayse/not.txt`).

Yolları elle birleştirmeyin

Yol ayracı Windows'ta `\`, Linux ve macOS'ta `/` karakteridir. Yolları `+` ile birleştirirsek program başka sistemde bozulabilir. `Path` sınıfı bu işi doğru yapar:

yolIslemleri.cs

```
string tam = Path.Combine("veri", "raporlar", "ocak.csv");
Console.WriteLine(Path.GetFileName(tam));
Console.WriteLine(Path.GetFileNameWithoutExtension(tam));
Console.WriteLine(Path.GetExtension(tam));
Console.WriteLine(Path.GetDirectoryName(tam));
Console.WriteLine(Path.ChangeExtension(tam, ".json"));
Console.WriteLine(Path.IsPathRooted(tam));
Console.WriteLine(Path.GetFullPath(tam) == Path.Combine(
    Directory.GetCurrentDirectory(), tam));
```

Çıktı, ayracı sistemden sisteme değiştiği için aşağıda Linux/macOS biçimiyle verilmiştir; Windows'ta `veri/raporlar` yerine `veri\raporlar` yazılır.

```
ÇIKTI
ocak.csv
ocak
.csv
veri/raporlar
veri/raporlar/ocak.json
False
True
```

`Path` metotları dize işler; dosyanın var olup olmadığına bakmaz. Yani `Path.GetExtension("yok.txt")` dosya olmasa da `.txt` döndürür.

> **İpucu · Geçici dosyalar**
>
> Deneme için geçici bir dosya gerekirse `Path.GetTempFileName()` ya da `Path.GetTempPath()` işletim sisteminin geçici klasörünü kullanır. Böylece çalışma klasörünü kirletmeyiz.

57.3 Hataları Yakalamak

Dosya işlemleri, programın kontrolü dışındaki dünyayla konuşur: dosya olmayabilir, izin verilmeyebilir, disk dolu olabilir. Bu yüzden dosya kodunda istisna yönetimi (Bölüm 53) neredeyse zorunludur. C'de `fopen` `NULL` döndürür ve `errno` bakılır; C#'ta ise istisna fırlatılır.

Sık karşılaşılan istisnalar şunlardır:

İstisna	Ne zaman oluşur
`FileNotFoundException`	Okunacak dosya yok
`DirectoryNotFoundException`	Yolun klasör kısmı yok
`UnauthorizedAccessException`	İzin yok ya da yol bir klasör
`IOException`	Genel G/Ç hatası: dosya başka süreçte kilitli, disk dolu vb.
`PathTooLongException`	Yol sistem sınırından uzun

`FileNotFoundException`, `DirectoryNotFoundException` ve `PathTooLongException`, `IOException`'dan türer. `UnauthorizedAccessException` ise `IOException`'dan **türemez**; ayrı yakalanmalıdır. Özelden genele doğru sıralanmış `catch` blokları yazarız:

hataYakala.cs

```
static string[] Oku(string ad)
{
    try
    {
        return File.ReadAllLines(ad);
    }
    catch (FileNotFoundException e)
    {
        Console.WriteLine($"Dosya yok: {Path.GetFileName(e.FileName)}");
    }
    catch (DirectoryNotFoundException)
    {
        Console.WriteLine("Klasör yok.");
    }
    catch (IOException e)
    {
        Console.WriteLine($"G/Ç hatası: {e.Message}");
    }
    return Array.Empty<string>();
}

Console.WriteLine(Oku("olmayan.txt").Length);
Console.WriteLine(Oku("olmayan_klasor/x.txt").Length);
File.WriteAllText("var.txt", "bir\niki");
Console.WriteLine(Oku("var.txt").Length);
```

ÇIKTI

```
Dosya yok: olmayan.txt
0
Klasör yok.
0
2
```

`e.FileName` mutlak yolu içerir; kullanıcıya göstermeden önce `Path.GetFileName` ile yalnızca ad kısmını almak çoğu zaman daha okunaklıdır.

Burada metot hata durumunda boş dizi döndürüyor. Hatayı kendi başımıza çözemiyorsak yakalamayıp üst katmana bırakmak daha doğru olur; her `catch` bir karar gerektirir.

57.4 StreamWriter, StreamReader ve using

`File` metotları dosyayı tek seferde işler. Dosya büyükse, veriyi parça parça üretiyorsak ya da yazma sırasında kararlar veriyorsak akışlarla (stream) çalışırız. `StreamWriter` metin yazar, `StreamReader` metin okur. Kullanımları `Console.WriteLine` ve `Console.ReadLine`'a çok benzer.

Akış sınıfları işletim sisteminden bir **kaynak** (dosya tutamacı) ister. Kaynak işimiz bitince bırakılmalıdır; bırakılmazsa dosya kilitli kalır ve yazılan veri diske hiç inmeyebilir. C'de bu iş `fclose` ile, C++'ta nesnenin yıkıcısı ile (RAII, Bölüm 37) yapılıyordu. C#'ta çöp toplayıcı bellek yönetir ama dosya tutamacı gibi kaynakların zamanında kapanmasını garanti etmez. Bunun için `IDisposable` arayüzü ve `using` deyimi vardır (Bölüm 53).

Tanım: using deyimi

`using` deyimi, `IDisposable` uygulayan bir nesneyi bloğun sonunda, hata olsa bile, otomatik olarak `Dispose` ile temizler. Akış sınıflarında `Dispose` akışı boşaltır ve dosyayı kapatır.

akisYaz.cs

```
using (StreamWriter yazici = new StreamWriter("gunluk.txt"))
{
    yazici.WriteLine("Birinci satır");
    yazici.WriteLine("Sayı: {0}", 42);
    yazici.Write("Yan ");
    yazici.Write("yana");
    yazici.WriteLine();
}   // blok biter, dosya kapanır

using (StreamReader okuyucu = new StreamReader("gunluk.txt"))
{
    string? satir;
    while ((satir = okuyucu.ReadLine()) != null)
    {
        Console.WriteLine($"> {satir}");
    }
}
```

ÇIKTI

```
> Birinci satır
> Sayı: 42
> Yan yana
```

`ReadLine` dosya sonuna gelince `null` döndürür. Bu yüzden döngü koşulunda atama ile `null` karşılaştırmasını birlikte yazdık. C'deki `feof` tuzağı (okumadan önce dosya sonunu denetlemek) burada yoktur: önce okur, sonra sonucu denetleriz. `string?` gösterimi Bölüm 59'da işlenecek nullable referans tipleridir; şimdilik "bu değişken `null` olabilir" diye okuyabiliriz.

Kısa using bildirimi

C# 8'den itibaren `using` blok olmadan, bildirim biçiminde de yazılabilir. Nesne, içinde tanımlandığı bloğun (burada metodun ya da dosyanın) sonunda atılır:

usingBildirimi.cs

```
using StreamWriter yazici = new StreamWriter("kisa.txt");
yazici.WriteLine("ilk");
yazici.WriteLine("ikinci");
yazici.Flush();

Console.WriteLine(File.ReadAllText("kisa.txt").Replace("\n", "|"));
```

ÇIKTI

```
ilk|ikinci|
```

Burada `Flush` çağırmamızın nedeni, dosyayı programın sonunda değil hemen okumamızdır; `StreamWriter` veriyi önce bir tamponda biriktirir. Bildirim biçiminde nesne programın sonunda atıldığı için, yazdıklarımızı o noktadan önce okumak istiyorsak `Flush` gerekir. Blok biçimli `using`, kapanışı daha görünür kıldığı için bu tür durumlarda daha anlaşılırdır.

> ◆ **Kritik Nokta**
>
> `StreamWriter`'ı `using` olmadan kullanırsak ve `Close` / `Dispose` / `Flush` çağırmazsak, tamponda kalan son satırlar dosyaya hiç yazılmayabilir. Akış nesnesi gördüğümüzde ilk refleksimiz `using` olmalıdır.

Ekleme kipi ve kodlama

`new StreamWriter(yol, true)` dosyayı **ekleme kipinde** açar; ikinci parametre `append` bayrağıdır (C'deki `"a"`). `File.AppendText(yol)` aynı işi yapar.

Kodlama (encoding) varsayılan olarak UTF-8'dir ve .NET'te dosyaya BOM (baş işareti) yazılmaz. Başka bir kodlamayı üçüncü parametre olarak verebiliriz:

▸ kodlama.cs

```
using System.Text;

string metin = "çiçek";    // 5 karakter, 2 tanesi Türkçe harf
File.WriteAllText("utf8.txt", metin);
File.WriteAllText("utf16.txt", metin, Encoding.Unicode);

Console.WriteLine(new FileInfo("utf8.txt").Length);
Console.WriteLine(new FileInfo("utf16.txt").Length);
Console.WriteLine(File.ReadAllText("utf8.txt") == metin);
Console.WriteLine(File.ReadAllText("utf16.txt", Encoding.Unicode) == metin);
```

```
ÇIKTI
7
12
True
True
```

UTF-8'de `ç` iki bayt, diğer harfler birer bayttır: 2 + 1 + 2 + 1 + 1 = 7. `Encoding.Unicode` (UTF-16) her karakter için iki bayt kullanır ve dosyanın başına iki baytlık BOM ekler: 5 x 2 + 2 = 12. Dosyayı yazdığımız kodlamayla okumazsak harfler bozulur; bu yüzden uygulamalar arası değişimde UTF-8 ve açıkça belirtilmiş kodlama en güvenli seçimdir. `FileInfo` sınıfını bir sonraki kısımda tanıyacağız.

57.5 Klasörler ve FileInfo

`Directory` sınıfı da `File` gibi statik metotlar sunar; `FileInfo` ve `DirectoryInfo` ise aynı bilgileri bir nesne olarak taşır.

▸ klasorler.cs

```
Directory.CreateDirectory("proje/notlar/eski");   // ara klasörlerle birlikte
File.WriteAllText(Path.Combine("proje", "oku.txt"), "oku beni");
File.WriteAllText(Path.Combine("proje", "notlar", "a.txt"), "a");
File.WriteAllText(Path.Combine("proje", "notlar", "b.log"), "bb");

Console.WriteLine(Directory.Exists("proje"));

foreach (string d in Directory.GetDirectories("proje"))
    Console.WriteLine($"klasör: {d}");

string[] txtler = Directory.GetFiles("proje", "*.txt",
                                     SearchOption.AllDirectories);
```

```
foreach (string f in txtler)
    Console.WriteLine($"txt: {f}");

Directory.Delete("proje", true);     // true: içindekilerle birlikte sil
Console.WriteLine(Directory.Exists("proje"));
```

```
ÇIKTI
True
klasör: proje/notlar
txt: proje/oku.txt
txt: proje/notlar/a.txt
False
```

`Directory.CreateDirectory` klasör zaten varsa hata vermez. `GetFiles(yol, desen, SearchOption.AllDirectories)` alt klasörleri de dolaşır; sırasının sistemler arasında aynı olacağı garanti edilmez, düzenli bir çıktı istiyorsak sonucu sıralamalıyız. `Directory.Delete(yol)` boş olmayan klasörde hata verir; ikinci parametre `true` ise içeriğiyle birlikte siler ve geri dönüşü yoktur.

Büyük klasörlerde `GetFiles` bütün sonucu önce dizi olarak üretir. `EnumerateFiles` aynı işi tembel yapar ve `foreach` ile ya da LINQ ile kullanıldığında ilk sonuçlar hemen gelir. Klasör ağacını kendi özyinelemeli metodumuzla dolaşmayı Bölüm 58'de göreceğiz.

FileInfo ile dosya bilgisi

▸ dosyaBilgisi.cs

```
File.WriteAllText("rapor.txt", "ilk satır\nikinci satır\n");

FileInfo bilgi = new FileInfo("rapor.txt");
Console.WriteLine($"Ad: {bilgi.Name}");
Console.WriteLine($"Uzantı: {bilgi.Extension}");
Console.WriteLine($"Boyut: {bilgi.Length} bayt");
Console.WriteLine($"Var mı: {bilgi.Exists}");
Console.WriteLine($"Yeni mi: {DateTime.Now - bilgi.LastWriteTime < TimeSpan.FromMinutes(1)}");

bilgi.CopyTo("rapor_yedek.txt", true);
bilgi.Delete();
Console.WriteLine(File.Exists("rapor.txt") + " " + File.Exists("rapor_yedek.txt"));
```

```
ÇIKTI
Ad: rapor.txt
Uzantı: .txt
Boyut: 25 bayt
Var mı: True
Yeni mi: True
False True
```

`FileInfo`, bir dosya hakkında birden çok bilgi isteyeceğimiz durumlarda kullanışlıdır: `Length`, `CreationTime`, `LastWriteTime`, `IsReadOnly`, `Directory`. Dikkat edilmesi gereken nokta, bazı özelliklerin ilk erişimde okunup önbelleğe alınmasıdır; dosya sonradan değişmişse güncel bilgi için `bilgi.Refresh()` çağırılır. Tek bir bilgiyi bir kez almak için ise `new FileInfo(...)` yerine `File.GetLastWriteTime` gibi statik metotlar daha kısadır.

57.6 CSV Dosyalarını Okumak ve Yazmak

CSV (comma-separated values), her satırı bir kayıt, her alanı virgül (ya da noktalı virgül) ile ayrılmış bir sütun olan basit metin biçimidir. Elektronik tablolar ve birçok araç bu biçimi içe ve dışa aktarabilir. C'de `fgets` ve `strtok` ile, C++'ta `stringstream` ile ayrıştırmıştık (Bölüm 37); C#'ta `string.Split` ve `int.Parse` / `TryParse` işi çok kolaylaştırır.

Önce kayıt tipimizi tanımlayalım ve listeyi dosyaya yazalım. Sayıları yazarken kültürden etkilenmemek için `CultureInfo.InvariantCulture` kullanıyoruz; aksi hâlde Türkçe bölge ayarlı bir bilgisayarda `85.5` değeri `85,5` olarak yazılır ve virgül ayracıyla karışır (Bölüm 43).

▸ ogrenciCsv.cs

```
using System.Globalization;

var liste = new List<Ogrenci>
{
    new Ogrenci("Ayşe", 20, 85.5),
    new Ogrenci("Mehmet", 22, 70),
    new Ogrenci("Elif", 21, 92.25),
};

// yaz
using (var yazici = new StreamWriter("ogrenciler.csv"))
{
    yazici.WriteLine("Ad,Yas,Not");   // başlık satırı
    foreach (Ogrenci o in liste)
        yazici.WriteLine(o.CsvSatiri());
}

// oku
var okunan = new List<Ogrenci>();
string[] satirlar = File.ReadAllLines("ogrenciler.csv");
for (int i = 1; i < satirlar.Length; i++)   // 0: başlık
{
    Ogrenci? o = Ogrenci.Ayristir(satirlar[i]);
    if (o != null) okunan.Add(o);
    else Console.WriteLine($"Satır {i + 1} atlandı: {satirlar[i]}");
}

foreach (Ogrenci o in okunan) Console.WriteLine(o);
Console.WriteLine($"Ortalama: {okunan.Average(o => o.Not):F2}");

class Ogrenci
{
    public string Ad { get; }
    public int Yas { get; }
    public double Not { get; }

    public Ogrenci(string ad, int yas, double notu)
    {
        Ad = ad; Yas = yas; Not = notu;
    }

    public string CsvSatiri() =>
        string.Join(",", Ad, Yas, Not.ToString(CultureInfo.InvariantCulture));

    public static Ogrenci? Ayristir(string satir)
    {
        string[] p = satir.Split(',');
        if (p.Length != 3) return null;
        if (!int.TryParse(p[1], out int yas)) return null;
```

```
        if (!double.TryParse(p[2], NumberStyles.Float,
                             CultureInfo.InvariantCulture, out double notu))
            return null;
        return new Ogrenci(p[0].Trim(), yas, notu);
    }

    public override string ToString() => $"{Ad} ({Yas}): {Not}";
}
```

```
ÇIKTI
Ayşe (20): 85.5
Mehmet (22): 70
Elif (21): 92.25
Ortalama: 82.58
```

Burada üç noktaya dikkat edelim. Birincisi, `Ayristir` bozuk satır için istisna fırlatmak yerine `null` döndürür; bozuk veri bu tür dosyalarda beklenen bir durumdur ve `TryParse` deseni (Bölüm 53) tam burada yerindedir. İkincisi, başlık satırını atlıyoruz. Üçüncüsü, parametre adını `not` yerine `notu` seçtik; `not` C# 9'dan beri desen eşlemede kullanılan bir anahtar sözcüktür (Bölüm 44) ve okuru yanıltır.

> **▲ Sık Yapılan Hata · Basit Split'in sınırı**
>
> `Split(',')`, alan içinde virgül bulunan kayıtları (`"Yılmaz, Can",20`) ayıramaz; gerçek CSV'de tırnaklı alanlar ve tırnak kaçışları bulunur. Alanların virgül içermediğinden emin olduğumuz küçük dosyalarda `Split` yeterlidir. Genel amaçlı CSV için olgun bir kütüphane (ör. CsvHelper paketi) kullanılmalıdır.

Tutarsız satırlarla da sınayalım. Dosyaya bilerek bozuk satırlar yazıp programın çökmeden devam ettiğini görelim:

```
▸ bozukCsv.cs
```

```
File.WriteAllLines("bozuk.csv", new[]
{
    "Ad,Yas,Not",
    "Can,19,60",
    "Zeynep,yirmi,88",
    "Mert,23",
    "Deniz,24,75"
});

int gecerli = 0, hatali = 0;
foreach (string satir in File.ReadLines("bozuk.csv").Skip(1))
{
    string[] p = satir.Split(',');
    if (p.Length == 3 && int.TryParse(p[1], out _) && int.TryParse(p[2], out _))
        gecerli++;
    else
        hatali++;
}
Console.WriteLine($"geçerli: {gecerli}, hatalı: {hatali}");
```

```
ÇIKTI
geçerli: 2, hatalı: 2
```

`out _` atılan (discard) bir çıkış değişkenidir: değeri istemiyoruz, yalnızca ayrıştırmanın başarılı olup olmadığına bakıyoruz.

57.7 JSON ile Kaydetme

CSV düz tablolar için uygundur. İç içe nesneler, listeler ve farklı tipte alanlar içeren veriler için JSON (JavaScript Object Notation) daha uygundur ve web dünyasının ortak biçimidir. .NET'te `System.Text.Json` ad alanı yerleşik olarak gelir.

> **Tanım: Serileştirme**
>
> Bellekteki bir nesneyi dosyaya ya da ağa gönderilebilecek bir metne (veya bayt dizisine) çevirmeye serileştirme (serialization), tersine işleme seri durumdan çıkarma (deserialization) denir.

`JsonSerializer.Serialize` nesneyi JSON metnine, `JsonSerializer.Deserialize<T>` JSON metnini nesneye çevirir. Serileştirilecek tipin genel (public) özellikleri kullanılır:

jsonKaydet.cs

```
using System.Text.Json;

var urunler = new List<Urun>
{
    new Urun { Ad = "Kalem", Fiyat = 12.5m, Stok = 100,
               Etiketler = new() { "kirtasiye" } },
    new Urun { Ad = "Defter", Fiyat = 30m, Stok = 40 },
};

var secenek = new JsonSerializerOptions { WriteIndented = true };
string json = JsonSerializer.Serialize(urunler, secenek);
File.WriteAllText("urunler.json", json);
Console.WriteLine(json);

string okunanJson = File.ReadAllText("urunler.json");
List<Urun>? geri = JsonSerializer.Deserialize<List<Urun>>(okunanJson);

Console.WriteLine(geri!.Count);
Console.WriteLine($"{geri[0].Ad}: {geri[0].Fiyat} x {geri[0].Stok}");

class Urun
{
    public string Ad { get; set; } = "";
    public decimal Fiyat { get; set; }
    public int Stok { get; set; }
    public List<string> Etiketler { get; set; } = new();
}
```

```
ÇIKTI
[
  {
    "Ad": "Kalem",
    "Fiyat": 12.5,
    "Stok": 100,
    "Etiketler": [
      "kirtasiye"
    ]
  },
  {
    "Ad": "Defter",
    "Fiyat": 30,
    "Stok": 40,
    "Etiketler": []
  }
]
2
Kalem: 12.5 x 100
```

Özellikler JSON'da kendi adlarıyla yazılır. Web dünyasında alan adları sıklıkla küçük harfle başlar (`"ad"`); bunun için `PropertyNamingPolicy = JsonNamingPolicy.CamelCase` seçeneği kullanılır. `JsonSerializerOptions` nesnesini her çağrıda yeniden oluşturmak yerine bir kez oluşturup paylaşmak daha verimlidir.

`Deserialize` sonucu `null` olabileceği için dönüş tipi `List<Urun>?` olur. `geri!` ifadesindeki `!` işareti "bu değer null değildir, biliyorum" demektir (Bölüm 59). Üretimde `null` denetimi yapmak daha doğrudur.

Bozuk bir JSON için `Deserialize` bir `JsonException` fırlatır. Dosya işlemlerinde olduğu gibi dış kaynaktan gelen JSON'u `try` / `catch` içinde okumalıyız:

jsonHata.cs

```
using System.Text.Json;

try
{
    var x = JsonSerializer.Deserialize<List<int>>("[1, 2, ");
    Console.WriteLine(x?.Count);
}
catch (JsonException)
{
    Console.WriteLine("Geçersiz JSON");
}
```

```
ÇIKTI
Geçersiz JSON
```

57.8 Asenkron Dosya İşlemleri

Disk ve ağ erişimi yavaştır. Masaüstü ya da sunucu uygulamalarında bu beklemeyi iş parçacığını bloke etmeden yapmak için `File` sınıfı `async` karşılıklar sunar: `ReadAllTextAsync`, `WriteAllTextAsync`, `ReadAllLinesAsync`. `async` / `await` mekanizmasını Bölüm 59'da ayrıntılı işleyeceğiz; burada kullanım biçimini görmek yeterli:

asenkronDosya.cs

```
await File.WriteAllTextAsync("asenk.txt", "asenkron yazıldı");
string s = await File.ReadAllTextAsync("asenk.txt");
Console.WriteLine(s);
```

```
ÇIKTI
asenkron yazıldı
```

Konsol programlarında çoğu zaman eşzamanlı sürüm yeterlidir; asenkron sürümler arayüzü donduran uygulamalarda yarar sağlar.

Üç Dilde · Üç dilde dosyaya satır yazma ve okuma *C · C++ · C#*

Aynı iş, yani üç satırı dosyaya yazıp geri okuma, üç dilde şöyle görünür.

C (Bölüm 17): açma, denetleme ve kapatma programcıdadır.

```
FILE *f = fopen("a.txt", "w");
if (f == NULL) { perror("a.txt"); return 1; }
fprintf(f, "bir\niki\nüç\n");
fclose(f);

f = fopen("a.txt", "r");
char satir[100];
while (fgets(satir, sizeof satir, f) != NULL)
    printf("%s", satir);
fclose(f);
```

C++ (Bölüm 37): akış nesnesi kapsamdan çıkınca dosyayı kendisi kapatır (RAII).

```
{
    std::ofstream yaz("a.txt");
    yaz << "bir\niki\nüç\n";
}
std::ifstream oku("a.txt");
std::string satir;
while (std::getline(oku, satir))
    std::cout << satir << '\n';
```

C#: tek çağrı ya da `using` ile akış.

```
File.WriteAllLines("a.txt", new[] { "bir", "iki", "üç" });
foreach (string satir in File.ReadLines("a.txt"))
    Console.WriteLine(satir);
```

Konu	C	C++	C#
Dosya nesnesi	`FILE *`	`ifstream` / `ofstream`	`StreamReader` / `StreamWriter`, `File`
Başarısızlık	`NULL` dönüşü, `errno`	`is_open()`, durum bayrakları	İstisna
Kapatma	`fclose` elle	Yıkıcı (RAII)	`using` / `Dispose`
Tek çağrıda tüm dosya	Yok	Yok (döngü gerekir)	`File.ReadAllText` vb.

Konu	C	C++	C#
Yol işlemleri	Elle	`<filesystem>`	`Path` , `Directory`

57.9 Sık Yapılan Hatalar

- **Göreli yolun nereye bağlı olduğunu bilmemek.** Dosya "yok" diyorsa önce `Directory.GetCurrentDirectory()` ile çalışma klasörüne bakalım.
- **`WriteAllText` ile istemeden üzerine yazmak.** Eklemek için `AppendAllText` kullanılır.
- **Akışı kapatmamak.** `using` kullanın; kapatılmamış `StreamWriter` verinin bir kısmını kaybettirir ve dosyayı kilitler.
- **Aynı dosyayı aynı anda açmak.** Bir `StreamWriter` açıkken aynı dosyayı `File.ReadAllText` ile okumaya çalışırsak `IOException` alırız.
- **Yolları `+` ile birleştirmek.** `Path.Combine` kullanılır.
- **Ondalıkları kültüre bağlı yazmak.** CSV ve JSON gibi makine okuyacak çıktılarda `InvariantCulture` seçilir.
- **Kodlamayı yazılan ve okunan tarafta farklı varsaymak.** Türkçe karakterler bozulur.

Bölüm Özeti

- `System.IO` ad alanı dosya ve klasör işlemlerinin evidir.
- `File.WriteAllText` / `WriteAllLines` içeriği sıfırdan yazar, `AppendAllText` / `AppendAllLines` sona ekler.
- `ReadAllText` dosyayı tek dize, `ReadAllLines` dize dizisi, `ReadLines` ise tembel dizilim olarak verir.
- `File.Exists` dosya varlığını denetler; ancak denetimden sonra durum değişebileceği için `try` / `catch` yine gerekir.
- Göreli yollar geçerli çalışma klasörüne göre çözülür; yolları `Path.Combine` ile birleştirmek taşınabilirlik sağlar.
- `FileNotFoundException` , `DirectoryNotFoundException` ve `IOException` sık hatalardır; `UnauthorizedAccessException` `IOException` 'dan türemez.
- `StreamWriter` ve `StreamReader` metni akış olarak işler; `ReadLine` dosya sonunda `null` döndürür.
- `using` deyimi, hata olsa bile akışı kapatıp tamponu boşaltmayı garanti eder.
- Varsayılan kodlama UTF-8'dir; farklı kodlama açıkça belirtilmelidir.
- `Directory` ve `FileInfo` klasör oluşturma, listeleme, silme ve dosya bilgisi için kullanılır.
- Basit CSV `Split` ile ayrıştırılır; bozuk satırlar için `TryParse` kullanılır, sayıların biçimi `InvariantCulture` ile sabitlenir.
- `System.Text.Json` ile `Serialize` ve `Deserialize<T>` nesneleri JSON'a çevirip geri okur.
- C'de kapatma elle, C++'ta yıkıcıyla, C#'ta `using` ile yapılır.

Alıştırmalar

1. `File.WriteAllText("a.txt", "x")` iki kez art arda çağrılırsa dosyada ne kalır?
   - **A)** `x`
   - **B)** `xx`
   - **C)** `x` ve alt satırda `x`
   - **D)** İkinci çağrı `IOException` fırlatır
   - **E)** Dosya boş olur
2. Dosyanın sonuna satır eklemek için hangisi uygundur?
   - **A)** `File.WriteAllText`
   - **B)** `File.AppendAllText`
   - **C)** `File.ReadAllText`
   - **D)** `File.Create`
   - **E)** `Directory.CreateDirectory`
3. Hangisi `IOException`'dan türemez?
   - **A)** `FileNotFoundException`
   - **B)** `DirectoryNotFoundException`
   - **C)** `PathTooLongException`
   - **D)** `UnauthorizedAccessException`
   - **E)** `EndOfStreamException`
4. `StreamReader.ReadLine()` dosya sonuna gelince ne döndürür?
   - **A)** Boş dize
   - **B)** `-1`
   - **C)** `null`
   - **D)** `EOF` sabiti
   - **E)** `EndOfStreamException` fırlatır
5. Aşağıdakilerden hangisi yolları taşınabilir biçimde birleştirir?
   - **A)** `"veri" + "/" + "a.txt"`
   - **B)** `"veri" + "\\" + "a.txt"`
   - **C)** `Path.Combine("veri", "a.txt")`
   - **D)** `string.Concat("veri", "a.txt")`
   - **E)** `Path.GetFullPath("veri a.txt")`
6. `using (var w = new StreamWriter("x.txt")) { ... }` bloğunun sonunda ne olur?
   - **A)** Hiçbir şey; çöp toplayıcı daha sonra kapatır
   - **B)** Tampon boşaltılır ve dosya kapatılır
   - **C)** Dosya silinir
   - **D)** Dosya salt okunur yapılır
   - **E)** Yalnızca tampon boşaltılır, dosya açık kalır
7. Aşağıdaki programın çıktısı nedir?

```
File.WriteAllLines("t.txt", new[] { "a", "b" });
File.AppendAllText("t.txt", "c");
File.AppendAllLines("t.txt", new[] { "d" });
string[] s = File.ReadAllLines("t.txt");
Console.WriteLine(s.Length);
Console.WriteLine(s[1] + s[2]);
```

8. Aşağıdaki programın çıktısı nedir?

```
string y = Path.Combine("a", "b", "rapor.final.txt");
Console.WriteLine(Path.GetFileNameWithoutExtension(y));
Console.WriteLine(Path.GetExtension(y));
Console.WriteLine(Path.ChangeExtension(y, "csv").EndsWith("rapor.final.csv"));
```

9. Aşağıdaki kodun yaptığı mantık hatasını bulun.

```
var w = new StreamWriter("log.txt");
w.WriteLine("başladı");
w.WriteLine("bitti");
```

10. Aşağıdaki program neden her çalıştırmada dosyada yalnızca tek satır bırakır? Düzeltin.

```
for (int i = 1; i <= 3; i++)
    File.WriteAllText("sayac.txt", $"Deneme {i}\n");
```

11. Aşağıdaki kodun hatası nedir? Düzeltin.

```
try
{
    string s = File.ReadAllText("yok.txt");
}
catch (IOException) { Console.WriteLine("G/Ç hatası"); }
catch (FileNotFoundException) { Console.WriteLine("Dosya yok"); }
```

12. `"Ad,Yas"` başlıklı ve `"Can,19"` gibi satırlardan oluşan bir CSV dosyasını okuyup ortalama yaşı yazdıran program yazın. Bozuk satırları atlayın.
13. Verilen bir klasördeki (alt klasörler dahil) `.txt` dosyalarının sayısını ve toplam boyutunu yazdıran bir program yazın. Klasör yoksa anlamlı bir mesaj verin.
14. Bir metin dosyasındaki satır, kelime ve karakter sayısını bulan program yazın. Dosyayı program önce kendisi oluştursun.
15. Bir `Kisi` sınıfı (`Ad`, `Yas`) tanımlayın; üç kişilik bir listeyi `kisiler.json` dosyasına yazın, sonra geri okuyup yaşa göre sıralı yazdırın.

BÖLÜM 58

Özyineleme C#'ta

BU BÖLÜMDE

- Bölüm 18 ve 38'deki özyineleme kalıplarının C#'taki karşılığını ve yazılış biçimini
- `Directory`, `Path` ve `FileInfo` ile klasör ağacını özyinelemeyle nasıl dolaşacağımızı
- `Dictionary<(int, int), long>` gibi değer demetli anahtarlarla hafızalamayı
- Sınıf tabanlı bir ikili ağacı (`Dugum`) özyinelemeyle nasıl dolaşacağımızı
- `StackOverflowException` istisnasının neden yakalanamadığını ve derin dolaşmayı `Stack<T>` ile nasıl çözdüğümüzü
- Kuyruk özyinelemesinin C#'ta garanti olmadığını ve döngüye nasıl çevrileceğini

Bölüm 18'de özyinelemenin temel durum, küçülen problem ve çağrı yığını kavramlarını C ile gördük. Bölüm 38'de aynı kalıpları `std::vector`, `std::map` ve geri izleme ile C++'ta yazdık. C#'ta kalıp değişmez; değişen, dilin bize verdiği araçlardır. Bölüm 18'in sonundaki üç dil karşılaştırmasında `Faktoriyel` metodunun C#'taki hâlini de görmüştük. Burada aynı klasik örnekleri tekrarlamayacağız. Odağımız C#'a özgü kullanım alanları olacak: dosya sistemi, sınıflarla kurulan ağaçlar ve .NET'in yığın davranışı.

58.1 ve 38'den Taşıdıklarımız

Üç kural C#'ta da aynen geçerlidir: temel durum önce yazılır, her çağrı temel duruma biraz daha yaklaşır ve aynı alt problem defalarca çözülüyorsa bellekleme düşünülür. Metot adları büyük harfle başlar (`Faktoriyel`, `KlasorDolas`); yerel değişkenler ve parametreler küçük harfle başlar.

Parametre geçirme konusunda C#'ın bir ayrımı vardır. Sayılar gibi değer tipleri (`int`, `long`, `struct`) değer ile geçirilir ve her çağrıda kopyalanır. Diziler, `List<T>`, `Dictionary` gibi referans tipleri ise yalnızca bir referansla geçirilir. Yani büyük bir koleksiyonu parametre olarak verirken C++'taki `const std::vector<int>&` yazımına gerek yoktur; referans zaten nesnenin kendisini gösterir.

> **Üç Dilde · Parametre geçirme: C, C++ ve C#** *C · C++ · C#*
>
> **C'de** büyük bir veriyi kopyalamamak için adresini (işaretçi) geçiririz; fonksiyon bu adres üzerinden değiştirir.
>
> **C++'ta** `const std::vector<T>&` yazılarak kopya engellenir ve `const` ile değiştirilmeyeceği garanti edilir.
>
> **C#'ta** referans tipleri zaten referansla geçer: `List<T>` ya da bir sınıf nesnesi kopyalanmaz. Değiştirilmeyeceğini bildirmek istersek `IReadOnlyList<T>` gibi salt okunur bir arayüz kullanırız. Değer tipleri için `in` anahtar sözcüğü (C# 7.2) bir referans geçirip değiştirilmesini yasaklar.

58.2 Klasör Ağacını Dolaşmak

Bir klasörün içinde dosyalar ve alt klasörler bulunur; alt klasörlerin içinde yine dosyalar ve alt klasörler vardır. Bu yapı bir ağaçtır ve ağacı gezmenin en doğal yolu özyinelemedir: klasörün içindeki her alt klasör için aynı işlemi bir kez daha yaparız. Derinlik sorunu yoktur; her düzey bir çağrıya karşılık gelir.

`Directory.GetDirectories` bir klasörün alt klasörlerini, `Directory.GetFiles` dosyalarını verir. Dönen yollar tam yoldur; yalnızca adını istiyorsak `Path.GetFileName` kullanırız. Yol birleştirmek için elle `"/"` ya da `"\\"` yazmayız, `Path.Combine` her işletim sisteminde doğru ayracı koyar.

Örnekte program önce geçici bir klasör ağacını kendisi kuruyor. `Directory.CreateDirectory` ara klasörleri de oluşturduğu için tek satır yeterlidir. Ardından ağacı gezip her girdiyi yazdırıyor ve dosya sayısı, klasör sayısı, toplam bayt değerlerini özyinelemeyle topluyoruz.

▸ klasorDolas.cs

```
// geçici bir klasör ağacını program kendisi kuruyor
string kok = Path.Combine(Path.GetTempPath(), "kitapDemo_" + Guid.NewGuid().ToString("N"));
Directory.CreateDirectory(Path.Combine(kok, "belgeler", "raporlar", "2026"));
Directory.CreateDirectory(Path.Combine(kok, "belgeler", "notlar"));
Directory.CreateDirectory(Path.Combine(kok, "gorseller"));
File.WriteAllText(Path.Combine(kok, "okuma.txt"), "merhaba dunya");
File.WriteAllText(Path.Combine(kok, "belgeler", "ozet.txt"), "ozet");
File.WriteAllText(Path.Combine(kok, "belgeler", "raporlar", "2026", "ocak.txt"), "ocak raporu");
File.WriteAllText(Path.Combine(kok, "belgeler", "notlar", "ders.txt"), "ders notu");

var sonuc = KlasorDolas(kok, 0);
Console.WriteLine($"Toplam: {sonuc.dosya} dosya, {sonuc.klasor} klasör, {sonuc.bayt} bayt");

Directory.Delete(kok, true);            // alt klasörleriyle birlikte sil
Console.WriteLine($"Silindi mi: {!Directory.Exists(kok)}");

// yol klasöründeki her girdiyi yazar; alt klasör için kendini çağırır
static (int dosya, int klasor, long bayt) KlasorDolas(string yol, int derinlik)
{
    int dosyaSayisi = 0;
    int klasorSayisi = 0;
    long toplamBayt = 0;
    string girinti = new string(' ', 2 * derinlik);

    foreach (string altYol in Directory.GetDirectories(yol).OrderBy(x => x, StringComparer.Ordi
nal))
    {
        Console.WriteLine($"{girinti}{Path.GetFileName(altYol)}/");
        var alt = KlasorDolas(altYol, derinlik + 1);    // özyineleme
        dosyaSayisi += alt.dosya;
        klasorSayisi += alt.klasor + 1;                  // alt klasörün kendisi de sayılır
        toplamBayt += alt.bayt;
    }

    foreach (string dosyaYolu in Directory.GetFiles(yol).OrderBy(x => x, StringComparer.Ordinal)
)
    {
        long boyut = new FileInfo(dosyaYolu).Length;
        Console.WriteLine($"{girinti}{Path.GetFileName(dosyaYolu)} ({boyut} bayt)");
        dosyaSayisi++;
        toplamBayt += boyut;
    }
    return (dosyaSayisi, klasorSayisi, toplamBayt);
}
```

```
ÇIKTI
belgeler/
  notlar/
    ders.txt (9 bayt)
  raporlar/
    2026/
      ocak.txt (11 bayt)
  ozet.txt (4 bayt)
gorseller/
okuma.txt (13 bayt)
Toplam: 4 dosya, 5 klasör, 37 bayt
Silindi mi: True
```

Çıktıya bakarsak ağacın şekli görünüyor. Alt klasörün içeriği, adından hemen sonra ve bir girinti derinliğinde yazılıyor; bu yüzden girinti her düzeyde iki boşluk artıyor. Metot bir alt klasöre girdiğinde kendi döndürdüğü toplamları üst düzeye taşır. `belgeler` için dönen değer 3 dosya, 3 klasör ve 24 bayttır; kök klasör bu değerlere `notlar`, `raporlar` ve `gorseller`'ı ve kendi dosyasını ekleyerek 4 dosya ve 5 klasöre ulaşır.

Burada toplamı bir `ref` parametresiyle biriktirmedik. Metot değer olarak bir tuple döndürüyor ve çağıran taraf bu değeri kendi toplamına ekliyor. Bu yaklaşım her çağrının sonucunun açıkça görünmesini sağlar; Toplamı bir `ref` parametresiyle biriktirmek de mümkündür, ama hangi çağrının neyi değiştirdiğini izlemek zorlaşır.

> **▲ Sık Yapılan Hata · Directory.Delete geri alınamaz**
>
> `Directory.Delete(yol, true)` klasörü ve içindeki bütün dosya ile alt klasörleri siler; çöp kutusuna göndermez. Örnekte yolu kendimiz oluşturduğumuz geçici klasör olduğu için güvenli. Kullanıcı tarafından seçilen bir yolu bu şekilde silmeden önce yolun doğru olduğunu mutlaka kontrol edin.

> **İpucu · Sıralamayı kendimiz yapalım**
>
> `Directory.GetFiles` sonuçlarının sırası garanti değildir; işletim sistemine göre değişebilir. Yukarıdaki kod `OrderBy` ile sıralıyor, böylece çıktı her çalıştırmada aynı kalır. Yalnızca tüm dosyaların düz listesi gerekiyorsa `Directory.EnumerateFiles(kok, "*", SearchOption.AllDirectories)` gibi çerçeve metotları da ağacı kendisi dolaşır. Elle özyineleme yazmak, her düzeyde ne yapıldığını kontrol etmek istediğimizde ya da klasöre özel bir kural uygulayacağımızda anlamlıdır.

Sembolik bağlar, bir klasörün kendi atasını göstermesine yol açabilir. Bu durumda elle yazdığımız özyineleme sonsuza kadar sürer. Gerçek programlarda bağları ayırt eden ek bir kontrol yazmayı düşünmeliyiz.

58.3 Dictionary ile Hafızalama: Para Bozma

Bölüm 18'de Fibonacci ile hafızalamayı, Bölüm 38'de `std::map` ile ızgara yollarını saydık. İki örnekte de anahtar ya tek bir sayıydı ya da C++'taki `std::pair`'di. C#'ta aynı işi `Dictionary<TKey, TValue>` ve değer demeti (value tuple, C# 7) ile yaparız. `(int, int)` yazımı iki tam sayıdan oluşan bir anahtar tanımlar.

Problem şöyle: elimizde 1, 2 ve 5 birimlik madeniler var. Bir tutarı kaç farklı yolla ödeyebiliriz? Sıra önemli değil; `1 + 2` ile `2 + 1` aynı yol sayılır. Çözüm için `Yol(tutar, k)` fonksiyonunu yazarız: `tutar` kalan tutarı, `k` ise kullanabileceğimiz ilk madeni indeksini gösterir. Bu iki değer birlikte bir durumu tanımlar ve yanıt yalnızca bu ikisine bağlıdır. Bu yüzden anahtar da iki değerden oluşur.

Özyinelemeli durum iki seçenek arasında karar verir: `k`. madeniden bir tane alıp aynı `k` ile devam etmek ya da `k`. madeniyi tamamen bırakıp `k + 1` ile devam etmek. Temel durumlar ise tutar tam sıfır olduğunda 1 yol (hiçbir şey yapmadan tamamlandı) ve tutar negatif olduğunda ya da madeni kalmadığında 0 yol.

paraBozmaHafiza.cs

```
int[] madeniler = { 1, 2, 5 };
var hafiza = new Dictionary<(int, int), long>();

Console.WriteLine($"10 birim için yol sayısı: {Yol(10, 0, madeniler, hafiza)}");
Console.WriteLine($"100 birim için yol sayısı: {Yol(100, 0, madeniler, hafiza)}");
Console.WriteLine($"hafızadaki kayıt sayısı: {hafiza.Count}");

// tutar: kalan tutar; k: kullanılabilecek ilk madeni indeksi
static long Yol(int tutar, int k, int[] para, Dictionary<(int, int), long> tablo)
{
    if (tutar == 0) return 1;                          // tam ödendi: bir yol
    if (tutar < 0 || k == para.Length) return 0;       // ödenemez ya da madeni kalmadı

    var anahtar = (tutar, k);
    if (tablo.TryGetValue(anahtar, out long onceden))
    {
        return onceden;                                // daha önce hesaplandı
    }

    long sonuc = Yol(tutar - para[k], k, para, tablo)    // k. madeniden bir tane al
               + Yol(tutar, k + 1, para, tablo);         // k. madeniyi bırak
    tablo[anahtar] = sonuc;
    return sonuc;
}
```

```
ÇIKTI
10 birim için yol sayısı: 10
100 birim için yol sayısı: 541
hafızadaki kayıt sayısı: 300
```

Program hafızada 300 kayıt tuttu. Her `(tutar, k)` çifti en fazla bir kez hesaplandığı için 100 birimlik çözüm de hızlıca bitti; hafızalama olmasaydı aynı alt problemler defalarca yeniden çözülürdü.

> **▲ Sık Yapılan Hata · Anahtarı eksik kurmak**
>
> Anahtara yalnızca `tutar` yazıp `k`'yı unutursak, aynı tutar farklı madeni sınırlarıyla geldiğinde önbellekteki yanlış değer döner. Bir durumu tanımlayan bütün parametreler anahtarda yer almalıdır. Bu kural Bölüm 38'deki `std::pair` anahtarında da geçerliydi.

> **İpucu · TryGetValue neden iyi?**
>
> `if (tablo.ContainsKey(anahtar)) return tablo[anahtar];` yazımı aynı işi yapar, ama sözlükte iki arama gerektirir. `TryGetValue` hem var olup olmadığını hem de değerini tek aramada döndürür ve değer `out` ile dışarı çıkar. Bu yüzden yukarıdaki kodda `out long onceden` kullandık.

58.4 İkili Ağaç: Dugum Sınıfı

Ağaç, her düğümün birden fazla çocuğa sahip olabildiği bir veri yapısıdır. Bölüm 19'da bağlı listede düğümleri işaretçilerle birbirine bağlamayı C ile gördük; ağaç da aynı fikrin dallanan hâlidir. Burada en çok iki çocuğu olan

ikili ağacı sınıf (`class`) ile kuruyoruz. Her düğüm bir değer, bir sol ve bir sağ çocuk tutar. Çocuk yoksa değeri `null` olur.

> **Üç Dilde · Ağaç düğümü: C, C++ ve C#** *C · C++ · C#*
>
> **C'de** düğüm bir `struct`'tır; çocuklar işaretçi olarak tutulur ve düğümler `malloc` ile ayrılıp `free` ile geri verilir.
>
> **C++'ta** çocuklar `Dugum*` ya da `std::unique_ptr<Dugum>` olarak tutulur; ayırma ve geri verme sorumluluğu programcıdadır, akıllı işaretçi bu işi kısmen üstlenir.
>
> **C#'ta** `Dugum` bir sınıftır, nesneler öbekte (heap) yaşar ve çöp toplayıcı kullanılmayan nesneleri temizler. `Dugum?` yazımı çocuğun `null` olabileceğini açıkça belirtir; bu, C# 8 ile gelen null atanabilir referans tiplerinin bir parçasıdır.

Örnekte kurduğumuz ağaç şöyle görünür: kökte 8 var, solunda 3, sağında 10 bulunur. Bu bir ikili arama ağacıdır: her düğümde sol alt ağaçtaki değerler düğümden küçük, sağ alt ağaçtaki değerler büyüktür. Bu özellik, aranan değeri tek bir yolda bulmamızı sağlar.

▸ ikiliAgac.cs

```
Dugum kok = new(8)
{
    Sol = new Dugum(3) { Sol = new Dugum(1), Sag = new Dugum(6) },
    Sag = new Dugum(10) { Sag = new Dugum(14) { Sol = new Dugum(13) } }
};

var liste = new List<int>();
Sirali(kok, liste);
Console.WriteLine($"Sıralı: {string.Join(" ", liste)}");
Console.WriteLine($"Toplam: {Toplam(kok)}");
Console.WriteLine($"Yükseklik: {Yukseklik(kok)}");
Console.WriteLine($"6 var mı: {Icerir(kok, 6)}");
Console.WriteLine($"7 var mı: {Icerir(kok, 7)}");

// önce sol alt ağaç, sonra düğüm, sonra sağ alt ağaç
static void Sirali(Dugum? d, List<int> sonuc)
{
    if (d is null) return;              // temel durum: boş ağaç
    Sirali(d.Sol, sonuc);
    sonuc.Add(d.Veri);
    Sirali(d.Sag, sonuc);
}

// her düğüm için kendi değeri ile iki alt ağacın toplamı
static int Toplam(Dugum? d)
{
    if (d is null) return 0;
    return d.Veri + Toplam(d.Sol) + Toplam(d.Sag);
}

// köke giden en uzun yoldaki düğüm sayısı
static int Yukseklik(Dugum? d)
{
    if (d is null) return 0;
    return 1 + Math.Max(Yukseklik(d.Sol), Yukseklik(d.Sag));
}

static bool Icerir(Dugum? d, int aranan)
{
    if (d is null) return false;
    if (d.Veri == aranan) return true;
```

```
    return aranan < d.Veri ? Icerir(d.Sol, aranan) : Icerir(d.Sag, aranan);
}

class Dugum
{
    public int Veri { get; }
    public Dugum? Sol { get; set; }
    public Dugum? Sag { get; set; }

    public Dugum(int veri) => Veri = veri;
}
```

```
ÇIKTI
Sıralı: 1 3 6 8 10 13 14
Toplam: 55
Yükseklik: 4
6 var mı: True
7 var mı: False
```

`Sirali`, `Toplam` ve `Yukseklik` iki özyinelemeli çağrı yapar; çünkü ağacın her düğümünün iki dalı vardır ve ikisi de işlenmelidir. Bölüm 18'deki Hanoi çağrı ağacı gibi bu da bir dallanma ağacıdır. Çıktının sırası, `Sirali` içindeki çağrılar yüzünden sol alt ağaç bitmeden düğümü eklememizden gelir. Bu dolaşmaya sıralı dolaşma (in-order) denir.

`Icerir` ise farklıdır: her düğümde yalnızca bir çağrı yapar. Aranan değer düğümden küçükse sola, büyükse sağa iner. Bu, Bölüm 18'de gördüğümüz özyinelemeli ikili aramayla aynı mantıktır; yalnızca dizi yerine ağaç üzerinde çalışır.

Temel durum her üç metotta da `null` kontrolüdür. Düğümün çocuğu yoksa çağrı `null` ile gelir ve hemen döner. Bu durum, dal sonundaki boş alt ağacı temsil eder. `d is null` yazımı `==` operatörünün üzerine yazılabilmesinden etkilenmediği için referans tipleri için güvenli bir `null` denetimidir ve C#'ta yaygın olarak tercih edilir.

58.5 Yığın Taşması ve StackOverflowException

Bölüm 18'deki üç dil karşılaştırmasında gördüğümüz gibi, C#'ta da temel durumu olmayan bir özyineleme yığını doldurur ve süreç çöker. Buradaki asıl fark, istisna mekanizmasının bu hatayı yakalamaya izin vermemesidir. `StackOverflowException` `try-catch` ile yakalanamaz: çalışma zamanı yığın dolduğunda programı sonlandırır ve `catch` bloğu hiç çalışmaz. Çözüm yakalamak değil, çağrı derinliğini kontrol altında tutmaktır.

Ana iş parçacığının yığın boyutu sınırlıdır ve işletim sistemine ve yapılandırmaya göre değişir. Bu yüzden binlerce düğümlük bir ağacı özyinelemeyle dolaşmak bir noktada sorun çıkarabilir. Elimizdeki bir doğrusal zinciri düşünelim: her düğümün `Sag` alanı bir sonrakini gösterir ve zincir 100 bin düğüm uzunluğundadır. Bu zinciri `Toplam` ile özyinelemeli olarak toplamaya kalkarsak derinlik 100 bin olur; yığın bunu taşıyamayabilir.

Çözüm, yığını kendimiz kurmaktır. `Stack<T>` nesnesi öbekte tutulduğu için sınırı yığın belleği değil, kullanılabilir bellek belirler. Aşağıdaki program aynı toplamı bir döngü ve bir `Stack<Dugum>` ile hesaplar.

▸ yiginToplam.cs

```
// doğrusal zincir: her düğümün Sag'i bir sonraki düğüm
Dugum zincir = new(0);
Dugum son = zincir;
for (int i = 1; i < 100000; i++)
{
```

```
    son.Sag = new Dugum(i);
    son = son.Sag;
}
Console.WriteLine($"Zincir toplamı: {ToplamYigin(zincir)}");

Dugum kok = new(8)
{
    Sol = new Dugum(3) { Sol = new Dugum(1), Sag = new Dugum(6) },
    Sag = new Dugum(10) { Sag = new Dugum(14) { Sol = new Dugum(13) } }
};
Console.WriteLine($"Ağaç toplamı: {ToplamYigin(kok)}");

// özyineleme yerine kendi yığınımız; yığın öbekte, derinlik yalnızca bellekle sınırlı
static long ToplamYigin(Dugum? kok)
{
    if (kok is null) return 0;

    var yigin = new Stack<Dugum>();
    yigin.Push(kok);
    long toplam = 0;
    while (yigin.Count > 0)
    {
        Dugum d = yigin.Pop();
        toplam += d.Veri;
        if (d.Sol is not null) yigin.Push(d.Sol);
        if (d.Sag is not null) yigin.Push(d.Sag);
    }
    return toplam;
}

class Dugum
{
    public int Veri { get; }
    public Dugum? Sol { get; set; }
    public Dugum? Sag { get; set; }

    public Dugum(int veri) => Veri = veri;
}
```

```
ÇIKTI
Zincir toplamı: 4999950000
Ağaç toplamı: 55
```

Zincirdeki değerler 0'dan 99 999'a kadardır ve toplamları `(0 + 99999) * 100000 / 2` ile 4 999 950 000 eder. Bu değer `int` sınırını aştığı için `long` kullandık; sınırı aşan bir `int` toplam sessizce taşar ve yanlış sonuç verir. Yukarıdaki `ToplamYigin` metodunda ayrıca bir `Stack<Dugum>` kullanılıyor; elemanlar eklenip çıkarılıyor. Her adımda çocukları yığına koymak, özyinelemedeki bekleyen çağrıların yerini alır.

Aynı toplamı özyinelemeyle yazsaydık 100 bin çerçeve birikirdi. Yığın sınırı ortama bağlı olduğu için bu derinliğin çökmeye yol açıp açmayacağını önceden kesin söyleyemeyiz; bu yüzden bu programda özyinelemeli sürümü bilerek yazmadık.

> **◆ Kritik Nokta · Derin özyinelemeyi döngü + yığınla değiştirin**
>
> Çağrı derinliği bilinmiyorsa (ağaçta uzun dal, klasörde çok düzey, bağlı listede çok düğüm) özyineleme yerine `Stack<T>` (ya da `Queue<T>`) ile döngü yazmak güvenlidir. Derinlik kendi yığınımızda olduğu için bellek sınırı sistemin yığın sınırı değil, kullanılabilir bellektir. Özyinelemenin okunaklılığını korumak istersek kodun derin kısmını yığınla yazıp küçük alt ağaçları özyinelemeyle bırakabiliriz.

58.6 Kuyruk Özyinelemesi

Bölüm 18'de kuyruk özyinelemesini (tail recursion) tanımladık: özyinelemeli çağrı metodun son işiyse, dönüşte yapılacak başka bir işlem yoktur. Böyle bir çağrı teoride döngüye çevrilebilir, çünkü çağıran çerçevenin tutması gereken bir şey kalmamıştır. C#'ta bu dönüşüm derleyicinin garantisi değildir. Derleyici kuyruk çağrılarını döngüye çevirmez; çalışma zamanı bazı durumlarda kuyruk çağrısını kendisi uygulayabilir, ama bu bir dil sözleşmesi değildir ve Debug yapılandırmasında beklenmemelidir.

Ebob hesabını iki biçimde yazıp farkı görelim:

kuyrukOzyineleme.cs

```
Console.WriteLine($"Özyinelemeli: {Ebob(48, 18)}");
Console.WriteLine($"Döngülü: {EbobDongu(48, 18)}");

// özyineleme çağrısı metodun son işi: kuyruk özyinelemesi
static int Ebob(int a, int b) => b == 0 ? a : Ebob(b, a % b);

static int EbobDongu(int a, int b)
{
    while (b != 0)
    {
        (a, b) = (b, a % b);      // değer demetiyle aynı anda atama
    }
    return a;
}
```

```
ÇIKTI
Özyinelemeli: 6
Döngülü: 6
```

İki metot aynı değeri üretir. Döngülü sürüm, yığında birikecek çerçeve bırakmaz ve derinliğinden bağımsız olarak sabit bellek kullanır. Ebob'un derinliği en fazla birkaç düzeydir, bu yüzden özyinelemeli sürüm burada sorunsuzdur. Derinliğin büyüyebileceği yerlerde döngülü biçimi seçmek daha güvenlidir.

> **Not · Yerel fonksiyonlar**
>
> Özyinelemeli bir yardımcı metodu başka bir metodun içine yerel fonksiyon olarak yazabiliriz. Bu biçimde yardımcı yalnızca dış metot içinde görünür ve dış metodun yerel değişkenlerine erişebilir. Sayaç gibi bir durumu tutmak için `ref` parametresi geçirmek yerine bu yolu seçebiliriz.

Bölüm Özeti

- Bölüm 18 ve 38'deki üç kural C#'ta da geçerlidir: temel durum önce, her çağrı temel duruma yaklaşır, tekrar eden alt problemler için bellekleme.
- Sınıf, dizi ve `List<T>` gibi referans tipleri referansla geçer ve kopyalanmaz; değer tipleri her çağrıda kopyalanır.
- `Directory.GetDirectories` ve `Directory.GetFiles` ile klasör ağacını özyinelemeyle dolaşırız; `Path.Combine` yolları doğru biçimde birleştirir.
- `Directory.Delete(yol, true)` alt klasörlerle birlikte her şeyi siler ve geri alınamaz.

- Hafızalamada anahtar, durumu tanımlayan bütün parametreleri içermelidir; `Dictionary<(int, int), long>` iki değerli anahtarı değer demetiyle tanımlar.
- `TryGetValue`, anahtarın varlığını ve değerini tek aramada verir.
- İkili ağaç `Dugum?` ile çocuğun yokluğunu `null` olarak gösterir; `Sirali`, `Toplam` ve `Yukseklik` iki çağrı yapar, `Icerir` yalnızca bir.
- `StackOverflowException` `try`-`catch` ile yakalanamaz; süreç sonlanır.
- Derin dolaşmalarda özyineleme yerine `Stack<T>` ile döngü yazmak, derinliği yığın belleğinden bağımsız kılar.
- C#'ta kuyruk özyinelemesi bir garanti değildir; derinliği bilinmeyen durumlarda döngüye çevirmek güvenlidir.
- Yerel fonksiyonlar, yardımcı özyinelemeyi dış metodun içinde tutar.

Alıştırmalar

1. `Directory.Delete(yol, true)` çağrısı hangi davranışı gösterir?
   - **A)** Yalnızca boş klasörü siler; klasörde dosya varsa hata verir.
   - **B)** Klasörü ve içindeki bütün dosya ile alt klasörleri siler.
   - **C)** Klasörü çöp kutusuna taşır.
   - **D)** Yalnızca alt klasörleri siler, kök klasör kalır.
   - **E)** Yalnızca dosyaları siler, alt klasörler kalır.
2. Aşağıdaki ifadelerden hangisi `StackOverflowException` için doğrudur?
   - **A)** `try`-`catch` ile yakalanabilir.
   - **B)** `catch (Exception)` ile yakalanabilir, ama `finally` çalışmaz.
   - **C)** Yakalanamaz; süreç sonlanır.
   - **D)** Yalnızca `Debug` yapılandırmasında oluşur.
   - **E)** `Stack<T>` kullanıldığında da her zaman oluşur.
3. `List<int>` türünde bir parametre ile `int` türünde bir parametre aynı metoda geçirildiğinde ne olur?
   - **A)** İkisi de kopyalanır.
   - **B)** İkisi de referansla geçer.
   - **C)** `List<int>` referansla geçer, `int` değer ile kopyalanır.
   - **D)** `int` referansla geçer, `List<int>` kopyalanır.
   - **E)** Derleme hatası oluşur.
4. Aşağıdaki programın çıktısı nedir?

```
static int F(int n)
{
    if (n < 3) return n;
    return F(n - 1) + F(n - 2);
}

Console.WriteLine(F(5));
```

5. `Yol(tutar, k)` fonksiyonunda hafızalama anahtarı yalnızca `tutar` olarak yazılırsa ne olur?
   - **A)** Program derlenmez.
   - **B)** Anahtar tekrar etmediği için bellekleme hiç çalışmaz, sonuç yine doğru olur.

C) Aynı tutar farklı `k` değerleriyle geldiğinde yanlış bir önbellek değeri dönebilir.
D) Program sonsuz döngüye girer.
E) Sözlük her çağrıda sıfırlanır, yalnızca yavaşlar.

6. Aşağıdaki kod ne yazdırır?

```
var d = new Dictionary<(int, int), string>();
d[(1, 2)] = "ilk";
d[(2, 1)] = "ikinci";
Console.WriteLine(d.Count);
Console.WriteLine(d[(1, 2)]);
```

A) `1` ve `ilk`
B) `2` ve `ilk`
C) `2` ve `ikinci`
D) `1` ve `ikinci`
E) Çalışma zamanı hatası oluşur.

7. `Sirali` metodunda `sonuc.Add(d.Veri);` satırı, ilk özyinelemeli çağrıdan (`Sirali(d.Sol, sonuc);`) önceye taşınırsa çıktı ne olur?

A) Hiç değişmez.
B) Çıktı `8 3 1 6 10 14 13` olur; düğüm önce yazılır, sıralı dolaşma bozulur.
C) Yalnızca sayıların sayısı değişir.
D) Derleme hatası oluşur.
E) Program sonsuz döngüye girer.

8. Aşağıdaki metot, `d` boş olmadığında ne döndürür ve neden?

```
static int Garip(Dugum? d)
{
    if (d is null) return 0;
    Garip(d.Sol);
    return d.Veri;
}
```

A) Ağacın toplamını döndürür; çünkü her düğüm bir kez toplanır.
B) Yalnızca kökün değerini döndürür; çünkü sol alt ağacın sonucu kullanılmaz ve sağ alt ağaç hiç gezilmez.
C) Ağacın en büyük değerini döndürür.
D) Her zaman 0 döndürür.
E) Derleme hatası verir.

9. Aşağıdaki kodda hatayı bulun. Metodun amacı dizinin toplamını döndürmektir.

```
static int Topla(int[] d, int i)
{
    if (i == d.Length) return 0;
    Topla(d, i + 1);
    return d[i];
}
```

10. **Program yazın:** Bir `Dugum` ağacındaki yaprak sayısını (iki çocuğu da `null` olan düğümleri) döndüren `YaprakSay` metodunu yazın. Bu bölümdeki ikili ağaç için sonuç 3 olmalıdır; yaprak düğümler 1, 6 ve 13'tür.

11. **Program yazın:** Bir merdivenin 1, 2 ya da 3 basamakla çıkılarak kaç farklı yolla tamamlanabileceğini hesaplayan `Yol` metodunu `Dictionary<int, long>` ile hafızalayarak yazın. Program `n = 10` için sonucu yazdırsın. Temel durumları doğru yazın: `n == 0` için bir yol vardır, negatif `n` için yol yoktur.
12. **Program yazın:** Bir klasör yolunu alıp içindeki bütün `.txt` dosyalarının toplam sayısını döndüren `TxtSay(string yol)` metodunu özyinelemeyle yazın. Programı, geçici bir klasör kurup çağırdıktan sonra klasörü silecek biçimde tamamlayın. Çıktı, kurduğunuz ağaçtaki `.txt` dosyası sayısını göstermelidir.
13. **Program yazın:** Bölüm 58'deki `Dugum` ağacında belirli bir değerin kökten olan derinliğini (kök için 0) döndüren `Derinlik(Dugum? d, int aranan)` metodunu yazın. Değer yoksa `-1` döndürsün. `Derinlik(kok, 13)` ve `Derinlik(kok, 99)` sonuçlarını yazdırın. Bu metodu yazarken `Icerir` gibi tek çağrılı bir yapı mı, yoksa iki çağrılı bir yapı mı gerektiğini düşünün.

Cevaplar

Cevap anahtarı `cevaplar/58.md` dosyasında yer alır.

BÖLÜM 59

Modern C#: Nullable Referans Tipleri, async/await ve Diğerleri

BU BÖLÜMDE

- Nullable referans tiplerinin (`string?`) `null` hatalarını derleme zamanına nasıl çektiğini
- `?.`, `??`, `??=` ve `!` operatörlerini ve derleyici uyarılarını okumayı
- Tuple tiplerini ve ayrıştırmayı (deconstruction)
- Liste desenleri dahil, desen eşlemenin ileri biçimlerini
- Hedef tipli `new()`, global using, dosya kapsamlı namespace ve koleksiyon ifadeleri gibi kısaltmaları
- `async` / `await`, `Task`, `Task.Delay` ve `Task.WhenAll` ile eşzamansız kod yazmayı
- Sürüm sürüm C# yeniliklerini ve eşzamanlılığın üç dildeki karşılığını

Bu bölümde C# kısmının son konularına, yani dilin son yirmi yılda eklediği özelliklerin en çok işe yarayanlarına geliyoruz. Bölüm 42'den beri `string?` yazımını gördük, `!` işaretini kullandık ve her seferinde konuyu bu bölüme bıraktık. Şimdi borcumuzu ödüyoruz.

Bölümün ikinci yarısı eşzamansız programlamayı (asynchronous programming) tanıtır. Bu, yazılım dünyasında en çok karıştırılan konulardan biridir; bu yüzden örnekleri bilerek küçük ve çıktısı öngörülebilir seçtik. Amacımız her ayrıntıyı işlemek değil, `async` ve `await` sözcüklerini gördüğümüzde ne olduğunu anlamamızı sağlamaktır.

59.1 Nullable Referans Tipleri

C. A. R. Hoare'un `null` başvurusunu icat etmesine "milyar dolarlık hata" dediği sık anılır. Nedeni açık: referans tipli bir değişken `null` olabildiği için her üye erişimi (`ad.Length`) potansiyel bir çökme noktasıdır. C'de bunun karşılığı geçersiz işaretçiyi göstermekti; C#'ta ise `NullReferenceException` fırlar.

Bölüm 42'de değer tipleri `null` tutamadığı için `int?` yazımını görmüştük. C# 8 (2019), aynı fikri tersinden uyguladı: referans tipleri varsayılan olarak "`null` olamaz" kabul edilir, `null` olabilecek olanlar ise açıkça `?` ile işaretlenir. Derleyici bu işaretlere bakarak `null` akışını izler ve tehlikeli yerlerde uyarı verir.

> **Tanım: Nullable referans tipleri (nullable reference types)**
>
> Derleyicinin referans tipli değişkenleri iki sınıfa ayırdığı özelliktir: `string` "asla `null` olmamalı", `string?` "`null` olabilir" demektir. Denetim yalnızca derleme zamanında yapılır ve ihlaller **uyarı** olarak bildirilir; çalışma zamanında iki tip aynıdır.

Özellik proje dosyasındaki `<Nullable>enable</Nullable>` ayarıyla açılır. .NET 6'dan bu yana yeni proje şablonları bu ayarı kendiliğinden ekler. Tek bir dosyada açıp kapatmak için `#nullable enable` ve `#nullable disable` yönergeleri de vardır. Bu kitaptaki örnekler ayarın açık olduğunu varsayar.

string ile string? farkı

nullableTemel.cs

```
string ad = "Elif";
string? takmaAd = null;       // null olabileceğini açıkça söyledik

Console.WriteLine(ad.Length);
Console.WriteLine(takmaAd == null);
Console.WriteLine(takmaAd?.Length);   // null güvenli erişim, aşağıda
```

ÇIKTI

```
4
True

```

Son satır boş bir satır yazar: `takmaAd` `null` olduğu için `?.` ifadesinin sonucu `null`'dır ve `WriteLine` `null` değeri için boş metin basar. Aşağıdaki kesitler ise derleyicinin ne dediğini gösterir (bunlar uyarıdır, program yine derlenir):

```
string a = null;              // uyarı CS8600: null değer, null olamayan tipe
string? b = null;
Console.WriteLine(b.Length);  // uyarı CS8602: olası null başvurusu
```

İlk satırda "olmamalı" dediğimiz `string`'e `null` atadık. İkinci satırda `b` `null` olabilir, ama `Length`'e doğrudan erişiyoruz. Her ikisi de çalışma zamanında çökmeye yol açabilir; uyarılar bunu derleme sırasında haber verir.

Derleyici akış analizi

Derleyici yalnızca tipe değil, koda da bakar. Bir `null` denetiminden sonra değişkenin artık `null` olmadığını bilir:

akisAnalizi.cs

```
string? Mesaj(string? giris)
{
    if (giris == null)
        return "boş";
    return $"{giris.ToUpper()} ({giris.Length})";   // burada uyarı yok
}

Console.WriteLine(Mesaj("selam"));
Console.WriteLine(Mesaj(null));
```

ÇIKTI

```
SELAM (5)
boş
```

`return` ile çıkan `if` bloğundan sonra `giris`'in `null` olmadığı kesindir; derleyici bunu fark eder ve `giris.ToUpper()` için uyarı vermez. `if (giris != null)`, `giris is not null`,

`string.IsNullOrEmpty(giris)` denetimleri de aynı etkiye sahiptir. Metot dönüş tipi `string?` yazılmıştır; çünkü ileride `null` döndürmek isteyebiliriz. Şimdilik bu gerekli değildir ama zararsızdır.

> **◆ Kritik Nokta**
>
> Nullable referans tipleri yeni bir tip getirmez, yalnızca **niyetimizi belgeler** ve derleyiciye denetleme olanağı verir. `string?` ile `string` çalışma zamanında aynı tiptir; uyarıyı susturmak hatayı ortadan kaldırmaz. Tip imzalarına (parametre, dönüş değeri, alan) doğru işaret koymak bu yüzden önemlidir: `null` olabilecek her yer `?` taşımalı, taşımayanlar `null` almamalıdır.

Alanlar ve yapıcılar

Sınıf alanları için de aynı kural geçerlidir. `null` olamayan bir alan, yapıcı bittiğinde değer almış olmalıdır. Aksi hâlde derleyici uyarır:

`kisiSinifi.cs`

```
var k = new Kisi("Mert");
Console.WriteLine($"{k.Ad} / {k.Soyad ?? "(yok)"}");

class Kisi
{
    public string Ad { get; }           // null olamaz
    public string? Soyad { get; set; }  // null olabilir

    public Kisi(string ad)
    {
        Ad = ad;        // yapıcı bitmeden Ad atanmış olmalı
    }
}
```

```
ÇIKTI
Mert / (yok)
```

Yapıcıda `Ad` atanmasaydı `CS8618` uyarısı alırdık (null olamayan özellik, yapıcıdan çıkarken `null` olamaz). Bu kitabın önceki bölümlerinde alanları `""` ile başlatmamızın nedeni budur. Ayrıca C# 11 ile gelen `required` değiştiricisi, nesne başlatıcıda belirli özelliklerin atanmasını zorunlu kılar:

```
class Urun
{
    public required string Ad { get; init; }
    public decimal Fiyat { get; init; }
}
// var u = new Urun { Fiyat = 5m };   // hata: required üye 'Ad' atanmadı
```

59.2 Null Operatörleri: ?., ??, ??= ve !

C#, `null` ile çalışmayı kısaltan bir operatör ailesi sunar. `??` ve `??=` ikilisini Bölüm 42'de nullable değer tipleri için gördük; ikisi de referans tiplerinde aynı şekilde çalışır.

Operatör	Ad	Anlamı
`a?.b`	null koşullu erişim	`a` `null` ise ifadenin tamamı `null`, değilse `a.b`

Operatör	Ad	Anlamı
`a?[i]`	null koşullu indeksleme	`a` `null` ise `null`, değilse `a[i]`
`a ?? b`	null birleştirme	`a` `null` değilse `a`, değilse `b`
`a ??= b`	null birleştirme ataması	`a` `null` ise `a = b` yapar
`a!`	null bağışıklığı (null-forgiving)	derleyiciye "bu `null` değil" der

?. ve ??

`?.` operatörü, zincirleme erişimlerde `null` denetimi yazma zahmetini ortadan kaldırır. Zincirin herhangi bir halkası `null` ise geri kalanı hiç çalışmaz ve sonuç `null` olur. `??` ile birleştirildiğinde bu, "yoksa varsayılan değer" kalıbını tek satıra indirir:

nullOperatorleri.cs

```
Musteri? m1 = new Musteri { Ad = "Ayşe", Adres = new Adres { Sehir = "İzmir" } };
Musteri? m2 = new Musteri { Ad = "Can" };      // adres yok
Musteri? m3 = null;

foreach (var m in new[] { m1, m2, m3 })
{
    string sehir = m?.Adres?.Sehir ?? "bilinmiyor";
    Console.WriteLine($"{m?.Ad ?? "-"}: {sehir}");
}

string[]? liste = null;
Console.WriteLine(liste?[0] ?? "liste yok");

string? onbellek = null;
onbellek ??= "ilk değer";       // null olduğu için atanır
onbellek ??= "ikinci değer";    // null değil, atama yapılmaz
Console.WriteLine(onbellek);

class Adres { public string? Sehir { get; set; } }
class Musteri
{
    public string Ad { get; set; } = "";
    public Adres? Adres { get; set; }
}
```

```
ÇIKTI
Ayşe: İzmir
Can: bilinmiyor
-: bilinmiyor
liste yok
ilk değer
```

`m2` için `m2?.Adres` `null` döndü, zincir orada kesildi ve `??` devreye girdi. `m3` için ise `m3?.Ad` bile `null` olduğundan `-` yazıldı. C++'ta bu zincirin karşılığı iç içe `if` blokları ya da `std::optional` zincirleridir; C'de ise her işaretçinin tek tek denetlenmesi gerekir.

> **▲ Sık Yapılan Hata**
>
> `?.` yalnızca **o noktada** `null` denetimi yapar. `a?.b.c` yazarsak `a` `null` ise tüm ifade atlanır; ama `a` dolu, `b` `null` ise `.c` yine çöker. Zincirin `null` olabilecek her halkasına `?.` koymak gerekir: `a?.b?.c`.

Null bağışıklığı operatörü: !

Bazen derleyiciden daha çok şey biliriz. Bir değerin `null` olmadığından eminizdir ama derleyici bunu kanıtlayamaz. Bu durumda değişkenin sonuna `!` koyarak uyarıyı susturabiliriz:

bagisiklik.cs

```
string? girdi = Environment.GetEnvironmentVariable("YOK_BOYLE_BIR_DEGISKEN");
string guvenli = girdi ?? "varsayılan";     // doğru yol
Console.WriteLine(guvenli);

string? s = "merhaba";
Console.WriteLine(s!.Length);               // biz biliyoruz, uyarı yok
```

ÇIKTI

```
varsayılan
7
```

Burada `s!` yalnızca derleyiciyi susturur; çalışma zamanında hiçbir denetim eklemez. Yanılırsak `NullReferenceException` yine fırlar. Operatörün adı da bu yüzdendir: "bağışıklık verir" ama koruma sağlamaz.

> **İpucu**
>
> `!` operatörünü son çare olarak kullanalım. Çoğu zaman daha iyi bir çözüm vardır: `if` ile denetlemek, `??` ile varsayılan vermek ya da `ArgumentNullException.ThrowIfNull(parametre)` ile metodun başında girişi doğrulamak. Bu son yöntem hem uyarıyı giderir hem de hatalı çağrıyı hemen yakalar.

```
static void Yaz(string? metin)
{
    ArgumentNullException.ThrowIfNull(metin);   // null ise ArgumentNullException
    Console.WriteLine(metin.Length);            // buradan sonra metin null değil
}
```

59.3 Tuple ve Ayrıştırma

Bir metottan birden fazla değer döndürmek istediğimizde C'de çıktı parametresi (işaretçi) ya da bir `struct`, C++'ta `std::pair` / `std::tuple` kullanırdık. C#'ın `out` parametrelerini Bölüm 46'da gördük. Modern C# üçüncü bir yol sunar: değer tuple'ları (value tuples).

> **Tanım: Tuple**
>
> Birden çok değeri, adlandırılmış ya da adsız öğeler olarak tek bir değerde toplayan hafif bir yapıdır. `(int, string)` biçiminde yazılır ve bir değer tipidir (`System.ValueTuple`). Ayrı bir sınıf tanımlamadan geçici veri grupları taşımak için uygundur.

tupleOrnek.cs

```
var (min, maks) = MinMaks(new[] { 7, 2, 9, 4 });
Console.WriteLine($"min={min} maks={maks}");

(string Ad, int Yas) kisi = ("Selin", 21);      // öğelere ad verdik
Console.WriteLine($"{kisi.Ad} {kisi.Yas}");
```

```
Console.WriteLine(kisi.Item1);                    // adsız erişim de var

var sonuc = Bol(17, 5);
Console.WriteLine($"{sonuc.Bolum} kalan {sonuc.Kalan}");

(int a, int b) = (3, 8);
(a, b) = (b, a);                                  // geçici değişkensiz takas
Console.WriteLine($"{a} {b}");

static (int Min, int Maks) MinMaks(int[] dizi) => (dizi.Min(), dizi.Max());
static (int Bolum, int Kalan) Bol(int x, int y) => (x / y, x % y);
```

```
ÇIKTI
min=2 maks=9
Selin 21
Selin
3 kalan 2
8 3
```

`var (min, maks) = ...` satırı bir **ayrıştırmadır** (deconstruction): tuple'ın öğeleri tek hamlede ayrı değişkenlere dağıtılır. Aynı sözdizimiyle iki değişkenin değerini yer değiştirebiliriz (`(a, b) = (b, a)`); C'deki üç adımlı takas kalıbına gerek kalmaz. Gerekmeyen bir öğeyi `_` ile atlayabiliriz: `var (bolum, _) = Bol(17, 5);`.

Ayrıştırma yalnızca tuple'larla sınırlı değildir. Bölüm 52'de gördüğümüz `record` tipleri konumsal parametreleri sayesinde otomatik ayrıştırılabilir. Kendi sınıfımıza da bir `Deconstruct` metodu yazarak aynı yeteneği verebiliriz:

▸ ayristirma.cs

```
var n = new Nokta(3, 4);
var (x, y) = n;
Console.WriteLine($"x={x} y={y}");

var p = new Personel("Zeynep", "Muhasebe");
var (ad, birim) = p;
Console.WriteLine($"{ad} - {birim}");

record Personel(string Ad, string Birim);   // Deconstruct otomatik gelir

class Nokta
{
    public int X { get; }
    public int Y { get; }
    public Nokta(int x, int y) { X = x; Y = y; }

    public void Deconstruct(out int x, out int y)
    {
        x = X;
        y = Y;
    }
}
```

```
ÇIKTI
x=3 y=4
Zeynep - Muhasebe
```

> **Not · Tuple ne zaman, sınıf ne zaman?**
> Tuple'lar kısa ömürlü, yerel sonuçlar için iyidir. Bir veri grubu metot sınırlarını aşıp programın birçok yerinde dolaşıyorsa, ona ad veren bir `record` ya da sınıf tanımlamak kodu daha okunur kılar.

59.4 Desen Eşlemenin İleri Biçimleri

Bölüm 44'te `is`, `switch` ifadesi, ilişkisel, mantıksal, tür, özellik ve tuple desenlerini gördük. Burada iki ek biçime bakacağız: iç içe özellik desenleri ve C# 11 ile gelen liste desenleri (list patterns).

İç içe özellik ve konumsal desenler

Özellik desenleri iç içe yazılabilir; böylece nesnenin içindeki nesnenin özelliğini tek bir koşulda sınayabiliriz. `record` tiplerinde ise parametre sırasına göre konumsal desen kullanılır:

ileriDesen.cs

```
var siparisler = new Siparis[]
{
    new("S1", new Musteri("Ece", "İstanbul"), 1200m),
    new("S2", new Musteri("Kerem", "Ankara"), 80m),
    new("S3", new Musteri("Deniz", "İstanbul"), 40m),
};

foreach (var s in siparisler)
{
    string etiket = s switch
    {
        { Musteri.Sehir: "İstanbul", Tutar: >= 1000 } => "büyük, İstanbul",
        { Musteri.Sehir: "İstanbul" }                 => "İstanbul",
        (_, _, < 50m)                                 => "küçük sipariş",
        _                                             => "diğer"
    };
    Console.WriteLine($"{s.No}: {etiket}");
}

record Musteri(string Ad, string Sehir);
record Siparis(string No, Musteri Musteri, decimal Tutar);
```

```
ÇIKTI
S1: büyük, İstanbul
S2: diğer
S3: İstanbul
```

`{ Musteri.Sehir: "İstanbul" }` iç içe özelliğe uzantılı yazımdır; `{ Musteri: { Sehir: "İstanbul" } }` ile aynı anlama gelir. Kollar yukarıdan aşağıya denenir: `S3` ikinci kolda yakalandığı için `küçük sipariş` koluna hiç ulaşmaz. Konumsal desen `(_, _, < 50m)` ise `Siparis` record'unun üç parametresine sırasıyla bakar.

Liste desenleri

Liste desenleri, bir dizi ya da listenin elemanlarını konumlarına göre eşler. `[a, b, c]` tam eleman sayısını, `..` ise "sıfır veya daha fazla eleman" anlamına gelen dilimleme desenini (slice pattern) belirtir:

listeDeseni.cs

```
foreach (var dizi in new[]
{
    new int[] { },
    new int[] { 5 },
    new int[] { 1, 2 },
    new int[] { 1, 2, 3 },
    new int[] { 9, 8, 7, 6 },
})
{
    string aciklama = dizi switch
    {
        []                    => "boş",
        [var tek]             => $"tek eleman: {tek}",
        [1, 2]                => "tam olarak 1, 2",
        [var ilk, .., var son] => $"ilk={ilk}, son={son}",
    };
    Console.WriteLine(aciklama);
}
```

ÇIKTI

```
boş
tek eleman: 5
tam olarak 1, 2
ilk=1, son=3
ilk=9, son=6
```

`[var ilk, .., var son]` kalıbı, en az iki elemanlı her diziyle eşleşir; aradaki elemanların sayısı önemsizdir. `[1, 2, 3]` dizisi `[1, 2]` kalıbıyla eşleşmez, çünkü eleman sayısı farklıdır ve üçüncü koldaki sabit eşleşme tam uzunluk ister. Liste desenleri dizilerde, `List<T>`'de ve `Length` / `Count` ile indeksleyicisi olan her tipte çalışır. `string` üzerinde de kullanılabilir: `"abc" is ['a', .., 'c']` doğrudur.

59.5 Kodu Kısaltan Özellikler

Son birkaç C# sürümü, her gün yazdığımız kalıpları kısaltan yazım kolaylıkları getirdi. Bunların çoğunu bu kitapta zaten kullandık; burada bir araya toplayalım.

var ve hedef tipli new

`var`, Bölüm 42'de gördüğümüz gibi derleyicinin tipi sağ taraftan çıkarmasını sağlar. C# 9 ile gelen **hedef tipli `new()`** ise tersini yapar: tipi sol taraftan alır, sağdaki `new` ifadesinde yazmayı gereksiz kılar. Alanlar, parametreler ve koleksiyon elemanları için özellikle işe yarar:

```
var liste = new List<string>();                    // tip sağda yazılı
List<string> liste2 = new();                       // tip solda yazılı
Dictionary<string, List<int>> notlar = new();
notlar["Can"] = new();                             // hedef tip: List<int>
```

> **İpucu**
>
> İkisinden birini seçelim: tipin sağda kolayca okunabildiği yerlerde `var`, tipi açıkça sola yazmak istediğimiz ya da alan tanımlarında hedef tipli `new()`. Tipin hiçbir yerde görünmediği `var x = Hesapla();` biçiminden kaçınmak okunabilirliği artırır.

Koleksiyon ifadeleri

C# 12 ile gelen koleksiyon ifadeleri (collection expressions), dizi, `List<T>` ve benzeri koleksiyonları köşeli parantezle oluşturur. `..` yayma (spread) operatörü başka bir koleksiyonun elemanlarını içeri ekler:

koleksiyonIfadesi.cs

```
int[] a = [1, 2, 3];
List<int> b = [10, 20];
List<int> hepsi = [0, .. a, .. b, 99];
Console.WriteLine(string.Join(",", hepsi));
```

```
ÇIKTI
0,1,2,3,10,20,99
```

Global using ve dosya kapsamlı namespace

Bölüm 41'de `System` ad alanının neden `using` olmadan göründüğünü görmüştük: proje, örtük `global using` yönergeleri ekliyordu. Bu yönergeyi kendimiz de yazabiliriz (C# 10). Bir dosyada `global using System.Text;` yazarsak, aynı projedeki **bütün** dosyalarda `System.Text` kullanılabilir. Genellikle projede tek bir `GlobalUsings.cs` dosyasında toplanır:

```
global using System.Text;
global using System.Collections.Generic;
```

Dosya kapsamlı namespace (C# 10) ise tüm dosyanın tek bir ad alanına ait olduğunu söyleyen satırdır. Küme parantezi ve bir girinti seviyesi kazandırır:

```
namespace Envanter.Modeller;     // dosyanın geri kalanı bu ad alanında

public class Urun
{
    public string Ad { get; set; } = "";
}
```

Eski biçimde (`namespace Envanter.Modeller { ... }`) bütün sınıf içeriği bir seviye içeri kayardı. Yeni kodların çoğu dosya kapsamlı biçimi kullanır. Bir dosyada yalnızca bir ad alanı varsa tercih edilir; aynı dosyada birden çok ad alanı gerekirse eski biçim kalır.

Birincil yapıcılar

C# 12 ile gelen birincil yapıcılar (primary constructors), sınıfın yapıcı parametrelerini sınıf adının yanına yazmamıza izin verir. Parametreler sınıfın her yerinde kullanılabilir. `record`'larda bu yazımı zaten gördük (Bölüm 52); artık sıradan sınıflarda da var:

birincilYapici.cs

```
var s = new Sayac("kapı", 3);
s.Artir();
s.Artir();
Console.WriteLine(s);

class Sayac(string ad, int baslangic)
{
    private int deger = baslangic;
```

```
    public void Artir() => deger++;
    public override string ToString() => $"{ad}: {deger}";
}
```

```
ÇIKTI
kapı: 5
```

Birincil yapıcı parametreleri otomatik olarak özellik olmaz (`record`'lardan farkı budur); yalnızca sınıf içinde kullanılabilen değerlerdir. Genel erişim gerekiyorsa bir özelliğe atarız.

59.6 Eşzamansız Programlama: Neden Gerekli?

Şimdiye kadar yazdığımız programlar sırayla çalıştı: bir satır bitmeden sonrakine geçilmedi. Bu çoğu iş için yeterlidir. Ancak bir dosyanın okunmasını, bir ağ isteğinin cevaplanmasını ya da bir veritabanı sorgusunun bitmesini beklerken işlemci boşta kalır. Bekleyen kod bir iş parçacığını (thread) tutmaya devam eder ve o iş parçacığı başka işe yaramaz.

Bu bekleyişin iki sakıncası vardır. Bir masaüstü programında arayüzü yöneten iş parçacığı beklerse pencere donar. Bir sunucuda ise her istek için bir iş parçacığı bekletmek, aynı anda hizmet verilebilecek istek sayısını düşürür.

> **Tanım: Eşzamansız (asynchronous) programlama**
>
> Uzun süren bir işin sonucunu beklerken çağıran iş parçacığını bloke etmeden başka işlere izin veren programlama biçimidir. İş bittiğinde kod kaldığı yerden devam eder.

Eşzamansızlık ile paralellik (parallelism) aynı şey değildir. Paralellikte birden çok işlemci çekirdeği aynı anda hesap yapar. Eşzamansızlıkta ise çoğu zaman tek bir iş parçacığı vardır; yalnızca beklemeler başka işlere yol açar. Bu bölümde yalnızca eşzamansızlığı işleyeceğiz.

59.7 async, await ve Task

C# 5 (2012) bu işi kolaylaştıran iki anahtar sözcük getirdi: `async` ve `await`. Bunlarla eşzamansız kodu, sanki sırayla çalışıyormuş gibi yazarız; derleyici gerisini bizim yerimize halleder.

> **Tanım: Task**
>
> Henüz bitmemiş olabilecek bir işi temsil eden nesnedir. Sonuç döndürmeyen işler için `Task`, `T` tipinde bir sonuç üretenler için `Task<T>` kullanılır. İş bittiğinde `Task` tamamlanmış sayılır; sonuç ya da istisna ona bağlıdır.

Üç kural yeterlidir:

1. Eşzamansız bir metot `async` işaretlenir ve dönüş tipi `Task` ya da `Task<T>` olur.
2. Bir `Task` üzerinde `await` yazdığımızda, görev bitene kadar metot **duraklar**, ama iş parçacığı bloke olmaz. Görev bitince metot kaldığı yerden sürer.
3. `await` yalnızca `async` metotlarda kullanılabilir. Top-level statements dosyalarında ise doğrudan kullanılabilir.

asyncTemel.cs

```
Console.WriteLine("Başlıyoruz");
int toplam = await ToplaGeciktir(20, 22);
Console.WriteLine($"Sonuç: {toplam}");
await Selamla("Selin");
Console.WriteLine("Bitti");

static async Task<int> ToplaGeciktir(int a, int b)
{
    await Task.Delay(100);          // 100 ms bekle, iş parçacığını tutma
    return a + b;
}

static async Task Selamla(string ad)
{
    await Task.Delay(50);
    Console.WriteLine($"Merhaba {ad}");
}
```

ÇIKTI

```
Başlıyoruz
Sonuç: 42
Merhaba Selin
Bitti
```

`Task.Delay(100)`, `Thread.Sleep(100)` işleminin eşzamansız karşılığıdır: 100 milisaniye sonra tamamlanan bir `Task` döndürür. `Thread.Sleep` iş parçacığını tutarak bekler; `await Task.Delay` ise iş parçacığını serbest bırakır. `return a + b;` satırında metodun dönüş tipi `Task<int>` olmasına rağmen yalnızca `int` döndürdüğümüze dikkat edelim; derleyici sonucu `Task` içine sarar. `await`, o `Task`'ı açıp içindeki `int` değerini bize verir.

> ◆ **Kritik Nokta**
>
> `async` bir metodu başka bir iş parçacığında çalıştırmaz. Metot, ilk `await`'e kadar çağıran iş parçacığında, olağan bir metot gibi çalışır. `await` bitmemiş bir görevle karşılaştığında metot duraklar ve çağıran kod devam eder. Eşzamansızlığı sağlayan, beklemenin işletim sistemi düzeyinde (zamanlayıcı, ağ, disk) yapılması ve iş parçacığının o sırada serbest kalmasıdır.

await'in arkasında ne olur?

Derleyici, `async` bir metodu bir durum makinesine (state machine) çevirir. Her `await` noktası bir durum olur; görev bitince durum makinesi bir sonraki durumdan devam eder. `yield return` için derleyicinin ürettiği sayıcıyı Bölüm 51'de gördük; mantık benzerdir. Bu ayrıntıyı bilmek zorunda değiliz, ama şunu bilmeliyiz: `await`'ten sonraki satırlar, öncekinden **farklı bir iş parçacığında** çalışabilir. Arayüz uygulamalarında devamlılık arayüz iş parçacığına döndürülür; konsol programlarında ise genelde havuzdaki herhangi bir iş parçacığında sürer.

Task döndüren ama await'siz metotlar

`async` işaretlenmiş bir metotta hiç `await` yoksa derleyici uyarı verir (`CS1998`), çünkü metot zaten eşzamanlı çalışacaktır. Sonucu hazır olan bir görevi `Task.FromResult` ile hemen döndürebiliriz:

```
static Task<int> Hazir() => Task.FromResult(7);
```

59.8 Birden Çok Görev: WhenAll

Birbirinden bağımsız üç işi sırayla beklersek toplam süre üçünün toplamı olur. Üçünü önce **başlatıp** sonra birlikte beklersek toplam süre yalnızca en uzun işin süresi kadardır. Bunu `Task.WhenAll` sağlar. Tüm görevler tamamlandığında tamamlanan tek bir `Task` döndürür; görevler `Task<T>` ise sonuçların dizisini verir ve **sonuçlar görevlerin verildiği sırayla** gelir.

whenAll.cs

```
Task<int> t1 = KareHesapla(3, 300);
Task<int> t2 = KareHesapla(4, 100);
Task<int> t3 = KareHesapla(5, 200);

int[] sonuclar = await Task.WhenAll(t1, t2, t3);
Console.WriteLine(string.Join(", ", sonuclar));
Console.WriteLine($"Toplam: {sonuclar.Sum()}");

static async Task<int> KareHesapla(int sayi, int beklemeMs)
{
    await Task.Delay(beklemeMs);
    return sayi * sayi;
}
```

ÇIKTI

```
9, 16, 25
Toplam: 50
```

`t2` en önce, `t1` en son tamamlanır; ama `WhenAll` sonuçları tamamlanma sırasıyla değil, **verilen sırayla** döndürür. Bu nedenle çıktı her çalıştırmada aynıdır. Toplam süre yaklaşık 300 ms'dir; sırayla beklesek 600 ms olurdu.

Görevlerin kaç tane olduğunu önceden bilmiyorsak LINQ ile bir görev listesi oluşturabiliriz (Bölüm 56):

whenAllListe.cs

```
string[] adlar = ["Ayşe", "Mehmet", "Elif"];
Task<string>[] gorevler = adlar.Select(HazirlaAsync).ToArray();
string[] mesajlar = await Task.WhenAll(gorevler);

foreach (string m in mesajlar)
    Console.WriteLine(m);

static async Task<string> HazirlaAsync(string ad)
{
    await Task.Delay(10);
    return $"Hoş geldin {ad}";
}
```

ÇIKTI

```
Hoş geldin Ayşe
Hoş geldin Mehmet
Hoş geldin Elif
```

Burada `Select` her ad için bir görev **başlatır** (ertelenmiş yürütme yüzünden `ToArray()` ile hemen başlatılmaları önemlidir), `WhenAll` hepsini bekler. Eşzamansız metotların adını `Async` ile bitirmek bir gelenektir (`HazirlaAsync`); okuyan kişi metodun `await` edilmesi gerektiğini adından anlar.

Tamamlanma sırası

Görevler bağımsız çalıştığı için bitiş sırası süreye bağlıdır. Aşağıdaki programda her görev bitince bir mesaj yazar. Hangi mesajın önce çıkacağı sürelerin farkına bağlıdır ve çok yüklü bir bilgisayarda değişebilir; bu yüzden çıktı yalnızca örnek niteliğindedir:

bitisSirasi.cs

```
var gorevler = new List<Task>
{
    Isle("A", 300),
    Isle("B", 100),
    Isle("C", 200),
};
await Task.WhenAll(gorevler);
Console.WriteLine("Hepsi bitti");

static async Task Isle(string ad, int ms)
{
    Console.WriteLine($"{ad} başladı");
    await Task.Delay(ms);
    Console.WriteLine($"{ad} bitti");
}
```

ÇIKTI

```
A başladı
B başladı
C başladı
B bitti
C bitti
A bitti
Hepsi bitti
```

"Başladı" satırları her zaman sırayla gelir; çünkü her `Isle` çağrısı kendi `await`'ine ulaşana kadar çağıran iş parçacığında senkron çalışır. "Bitti" satırlarının sırası ise zamanlayıcıya bağlıdır; bu ayrım yukarıdaki `kritik` kutusunun bir sonucudur.

> **▲ Sık Yapılan Hata**
>
> Görevleri **sırayla** `await` etmek istemeden sıralı hâle getirmeyelim. `await A(); await B(); await C();` yazarsak her görev öncekinin bitmesini bekler. Bağımsız işler için önce üçünü de başlatıp (`var a = A(); var b = B(); var c = C();`), sonra `await Task.WhenAll(a, b, c)` yazmak gerekir.

59.9 İstisnalar ve await

Eşzamansız kodda hata yönetimi, olağan koddaki gibi çalışır. Bir görev içinde fırlayan istisna, görev `await` edildiği noktada yeniden fırlatılır; böylece sıradan bir `try` / `catch` yeterlidir (Bölüm 53):

asyncIstisna.cs

```
try
{
    int sonuc = await Bol(10, 0);
    Console.WriteLine(sonuc);
}
catch (DivideByZeroException)
{
```

```
    Console.WriteLine("Sıfıra bölme yakalandı");
}
finally
{
    Console.WriteLine("Temizlik tamam");
}

static async Task<int> Bol(int x, int y)
{
    await Task.Delay(10);
    return x / y;           // y == 0 ise istisna, görevde saklanır
}
```

```
ÇIKTI
Sıfıra bölme yakalandı
Temizlik tamam
```

`Task.WhenAll` kullanırken birden çok görev hata verebilir. `await` bunlardan yalnızca **ilkini** fırlatır; hepsine `Task` nesnesinin `Exception` özelliğinden (`AggregateException`) ulaşılır. Çoğu program için ilk hatayı yakalamak yeterlidir.

Sık yapılan hatalar

> **▲ Sık Yapılan Hata**
>
> Eşzamansız kodda üç hata sık görülür. Birincisi, `async void` metot yazmaktır: dönüş tipi `void` olan bir `async` metodun bitişi izlenemez ve içinden çıkan istisna yakalanamaz, program çökebilir. `async void` yalnızca olay işleyicilerinde (event handler, Bölüm 55) kabul edilir. İkincisi, `Task`'ı `await` etmeden bırakmaktır: hata fırlasa bile kimse fark etmez. Üçüncüsü, `görev.Result` ya da `görev.Wait()` ile **bloke edici** bekleme yapmaktır; arayüz uygulamalarında bu, kilitlenmeye (deadlock) yol açabilir. Çözüm her zaman `await`'tir.

İptal

Uzun süren bir görevi iptal etmek için `CancellationToken` kullanılır. Görevi başlatan kod bir `CancellationTokenSource` oluşturur, görev ise aldığı belirteci (token) düzenli aralıklarla denetler. `Task.Delay` gibi kütüphane metotları belirteci doğrudan kabul eder ve iptal edildiğinde `OperationCanceledException` fırlatır:

▸ iptal.cs

```
using var kaynak = new CancellationTokenSource(TimeSpan.FromMilliseconds(100));
try
{
    await Task.Delay(5000, kaynak.Token);   // 100 ms sonra iptal edilir
    Console.WriteLine("Beklenen süre doldu");
}
catch (OperationCanceledException)
{
    Console.WriteLine("İptal edildi");
}
```

```
ÇIKTI
İptal edildi
```

Zaman aşımı 100 ms olduğundan bekleme 5 saniye sürmeden kesilir. Programın bu davranışı sistem yükünden bağımsız olarak aynıdır; yine de zamanlayıcıya bağlı bir örnek olduğu için doğrulama dışı bıraktık.

59.10 Eşzamanlılık: Üç Dile Genel Bakış

Eşzamanlılığın ayrıntıları kendi başına bir kitap konusudur. Burada yalnızca her dilin yaklaşımını yan yana koymakla yetineceğiz.

Üç Dilde · Eşzamanlılık *C · C++ · C#*

C: C diline iş parçacığı desteği C11 ile standarda girdi (`<threads.h>`); ancak pratikte işletim sistemi kütüphaneleri, özellikle POSIX iş parçacıkları (`pthread`), yaygındır. İş parçacığını açıkça biz yaratır, bekler ve paylaşılan veriyi kilitleriz (mutex).

```
pthread_t t;
pthread_create(&t, NULL, calis, &veri);   // iş parçacığını başlat
pthread_join(t, NULL);                    // bitmesini bekle
```

İleride C++ ve C#'ta bu işin daha yüksek düzeyli araçlarla yapıldığını göreceğiz.

C++: C++11 ile `std::thread`, `std::mutex` ve `std::async` standarda girdi. `std::async` bir işi başlatır ve sonucu `std::future` ile verir. `get()` çağrısı, sonuç hazır olana kadar **bloke eder**:

```
auto f = std::async(std::launch::async, [] { return 6 * 7; });
std::cout << f.get() << '\n';           // 42
```

C++20 ile gelen eş yordamlar (coroutines) `co_await` ile C# `await` fikrine yaklaşır; ancak çalışma kütüphanesini programcı ya da üçüncü taraf bir kütüphane sağlar.

C#: `Task` ve `async` / `await` dil düzeyindedir. Kodu sırayla okunan biçimde yazarız; bekleme sırasında iş parçacığı serbest kalır:

```
int sonuc = await Task.Run(() => 6 * 7);
Console.WriteLine(sonuc);               // 42
```

`Task.Run`, verilen işi havuzdaki bir iş parçacığına verir. İşlemciyi yoran hesaplar için kullanılır; ağ ve disk beklemelerinde zaten eşzamansız metotlar vardır.

Özellik	C (pthread)	C++ (`std::async`)	C# (`Task`)
İş parçacığı yaratma	`pthread_create`	`std::thread`, `std::async`	`Task.Run` (havuz)
Sonuç alma	ortak bellek, `pthread_join`	`std::future::get()` (bloke)	`await` (bloke etmez)
Beklerken iş parçacığı	tutulur	tutulur	serbest kalır
Kilit	`pthread_mutex_t`	`std::mutex`	`lock`, `Interlocked`
Dil desteği	yok (kütüphane)	`co_await` (C++20)	`async` / `await` (C# 5)

Eşzamanlılığın en tehlikeli tarafı, birden çok iş parçacığının aynı veriye aynı anda yazmasıdır (veri yarışı, data race). Bu üç dilde de aynıdır. C#'ta paylaşılan veriyi `lock` ile korumak ya da `Interlocked` işlemlerini kullanmak gerekir; ayrıntısı bu kitabın kapsamını aşar.

59.11 C# Sürüm Sürüm: Yeniliklerin Özeti

C# ilk sürümünden bu yana her büyük sürümde bir ya da iki ana özellik ekleyerek büyüdü. Aşağıdaki tablo, kitapta karşılaştığımız özellikleri sürümlerine göre özetler:

Sürüm	Yıl	Öne çıkanlar
C# 1.0	2002	Dilin ilk sürümü: sınıflar, arayüzler, delegate, olaylar
C# 2.0	2005	Generics, `yield`, nullable değer tipleri, anonim metotlar
C# 3.0	2007	`var`, lambda, LINQ, otomatik özellikler, nesne başlatıcılar
C# 4.0	2010	Adlandırılmış ve isteğe bağlı parametreler, `dynamic`
C# 5.0	2012	`async` / `await`
C# 6.0	2015	`?.`, `nameof`, dize yerleştirme (`$""`), ifade gövdeli üyeler
C# 7.0	2017	Tuple, `out var`, desen eşleme (başlangıç), yerel fonksiyonlar
C# 8.0	2019	Nullable referans tipleri, `switch` ifadesi, aralıklar (`..`)
C# 9.0	2020	`record`, `init`, top-level statements, hedef tipli `new()`
C# 10	2021	`global using`, dosya kapsamlı namespace, `record struct`
C# 11	2022	Liste desenleri, `required`, ham dize sabitleri (`"""`)
C# 12	2023	Birincil yapıcılar, koleksiyon ifadeleri
C# 13	2024	`params` ile koleksiyonlar, yeni `Lock` tipi
C# 14	2025	`field` anahtar sözcüğü, genişletme blokları (extension members)

> *Tarihçe · C# nasıl evrildi?*
>
> C#, Anders Hejlsberg liderliğindeki bir ekip tarafından Microsoft'ta geliştirildi; 2000'de duyuruldu, 2002'de .NET Framework 1.0 ile yayımlandı. İlk yıllarda Java'ya yakın bir nesne yönelimli dil olarak görüldü. 3.0 ile gelen LINQ ve lambdalar dile işlevsel programlamadan öğeler kattı. 5.0'daki `async` / `await` ise eşzamansız programlamayı sıradan bir sözdizimine çevirerek başka dillere de örnek oldu. 2016'dan sonra .NET Core ile dil platformlar arası hâle geldi; sürümler artık her yıl .NET ile birlikte yayımlanıyor.

Bu özelliklerin hepsini bilmek gerekmez. Günlük C# yazmak için çoğunlukla sınıflar, koleksiyonlar, LINQ, `async` / `await`, desen eşleme ve nullable referans tipleri yeterlidir. Yeni bir sürümün özelliğini öğrenmek için dilin belgelerine bakmak ve proje dosyasındaki `LangVersion` /hedef çerçeve ayarının o sürümü desteklediğinden emin olmak yeter.

Bölüm Özeti

- Nullable referans tipleri (C# 8) `null` hatalarını derleme zamanına çeker: `string` "`null` olmamalı", `string?` "`null` olabilir" demektir.
- Denetim yalnızca uyarıdır; çalışma zamanında `string` ile `string?` aynı tiptir.
- Derleyici akış analizi yapar: `null` denetiminden sonra değişkenin `null` olmadığını bilir.
- `?.` zincirde `null` görürse ifadenin tamamını `null` yapar; `??` varsayılan değer verir; `??=` yalnızca `null` ise atar.

- `!` derleyici uyarısını susturur ama çalışma zamanında koruma sağlamaz; son çare olmalıdır. `ArgumentNullException.ThrowIfNull` daha güvenlidir.
- Tuple'lar `(int Bolum, int Kalan)` gibi adlandırılmış öğelerle birden çok değer taşır; ayrıştırma `var (a, b) = ...` ile yapılır.
- Liste desenleri (`[var ilk, .., var son]`) dizi ve listeleri konumlarına göre eşler; özellik desenleri iç içe yazılabilir.
- Hedef tipli `new()`, koleksiyon ifadeleri (`[1, 2, 3]`), `global using`, dosya kapsamlı namespace ve birincil yapıcılar kodu kısaltır.
- `async` metotlar `Task` ya da `Task<T>` döndürür; `await` görev bitene kadar metodu duraklatır ama iş parçacığını bloke etmez.
- `async` kendiliğinden yeni iş parçacığı açmaz; eşzamansızlık beklemelerin iş parçacığı tutmadan yapılmasıdır.
- `Task.WhenAll` bağımsız görevleri birlikte bekler ve sonuçları verildiği sırayla döndürür.
- `await` ile `try` / `catch` birlikte çalışır; `async void` ve `.Result` ile bloke etmekten kaçınılır.
- C'de eşzamanlılık kütüphane (`pthread`) düzeyindedir, C++'ta `std::thread` / `std::async` bloke eder, C#'ta `await` bloke etmez.

Alıştırmalar

1. Aşağıdakilerden hangisi nullable referans tipleri açıkken uyarı **vermez**?
   - **A)** `string a = null;`
   - **B)** `string? b = null; int n = b.Length;`
   - **C)** `string? c = null; int n = c?.Length ?? 0;`
   - **D)** `string d = "x"; int n = d.Length;`
   - **E)** C ve D
2. `a?.b?.c` ifadesi hakkında hangisi doğrudur?
   - **A)** `a` `null` ise `NullReferenceException` fırlar.
   - **B)** `a` ya da `b` `null` ise ifadenin sonucu `null` olur.
   - **C)** Sonuç tipi her zaman `string`'dir.
   - **D)** `c` `null` ise hata verir.
   - **E)** Yalnızca `a` denetlenir.
3. Bu programın çıktısı nedir?

```
string? x = null;
x ??= "a";
x ??= "b";
Console.WriteLine(x + (x?.Length ?? 0));
```

4. Bu programın çıktısı nedir?

```
var (q, r) = (17 / 5, 17 % 5);
(q, r) = (r, q);
Console.WriteLine($"{q} {r}");
```

5. `null` bağışıklığı operatörü `!` için hangisi doğrudur?

A) Çalışma zamanında `null` denetimi ekler.
B) `null` değeri başka bir değerle değiştirir.
C) Derleyici uyarısını susturur ama çalışma zamanı davranışını değiştirmez.
D) Yalnızca değer tiplerinde kullanılabilir.
E) `?.` ile aynı işi yapar.

6. Bu programın çıktısı nedir?

```
int[] d = [4, 7, 1, 9];
string s = d switch
{
    [] => "boş",
    [var a] => "tek",
    [4, ..] => "dört ile başlıyor",
    [.., var son] => $"son {son}",
};
Console.WriteLine(s);
```

7. Aşağıdaki kodda neden `async void` yerine `async Task` tercih edilir?

```
static async void Kaydet() { await Task.Delay(10); throw new Exception("hata"); }
```

8. `async` anahtar sözcüğü hakkında hangisi doğrudur?

A) Metodu her zaman yeni bir iş parçacığında çalıştırır.
B) İçinde `await` kullanılabilmesini sağlar ve derleyicinin metodu durum makinesine çevirmesine yol açar.
C) Metodu otomatik olarak daha hızlı yapar.
D) Yalnızca `void` döndüren metotlarda kullanılabilir.
E) `Task` döndüren metotlarda yasaktır.

9. Bu programın çıktısı nedir?

```
var t = Cevir(3);
Console.WriteLine("A");
Console.WriteLine(await t);
Console.WriteLine("B");

static async Task<int> Cevir(int x)
{
    Console.WriteLine("içeride");
    await Task.Delay(10);
    return x * 2;
}
```

10. Aşağıdaki kod her bir bekleme 200 ms sürdüğüne göre yaklaşık kaç ms çalışır? Nasıl hızlandırılır?

```
await Task.Delay(200);
await Task.Delay(200);
await Task.Delay(200);
```

11. `Task.WhenAll(t1, t2, t3)` dönüşündeki `int[]` dizisinde sonuçlar hangi sırayla gelir?

A) Tamamlanma sırasıyla
B) Küçükten büyüğe sıralı
C) Verilen sırayla (`t1`, `t2`, `t3`)

D) Rastgele
E) Ters sırayla

12. Aşağıdaki kodun derleme uyarısını bulun ve düzeltin.

```
class Hesap
{
    public string Sahip { get; set; }
    public decimal Bakiye { get; set; }
}
```

13. **Program yazın.** Bir `string?` alan ve boş, `null` ya da yalnızca boşluklardan oluşuyorsa `"(isimsiz)"`, değilse metni baş ve sondaki boşluklar kırpılmış hâliyle döndüren `Temizle` metodunu yazın. Üç çağrıyla deneyin.
14. **Program yazın.** Bir `int` dizisinin ortalamasını ve en büyük elemanını aynı anda `(double Ortalama, int Maks)` tuple'ı olarak döndüren `Ozet` metodunu yazın; sonucu ayrıştırarak yazdırın.
15. **Program yazın.** `Karesi(int)` adlı bir eşzamansız metot yazın (10 ms bekleyip sayının karesini döndürsün). 1'den 5'e kadar olan sayılar için görevleri başlatıp `Task.WhenAll` ile bekleyin ve karelerin toplamını yazdırın.

BÖLÜM 60

C# Uygulama Projesi: Envanter Takip Sistemi

BU BÖLÜMDE

- Gereksinimlerden sınıf, `record` ve arayüz tasarımına geçmeyi
- Değişmez `record` nesnelerini `with` ifadesiyle güncelleyerek `Dictionary` içinde saklamayı
- LINQ ile gruplama, toplama ve süzme işlemlerinden rapor üretmeyi
- Kural ihlallerini kendi istisna sınıfımızla bildirip tek yerde yakalamayı
- Veriyi bir arayüzün arkasında CSV dosyasına kaydedip doğrulayarak geri yüklemeyi
- Çalışan bir programı kendi gereksinimlerinize göre genişletmeyi

Bölüm 20'de C ile bir öğrenci kayıt sistemi, Bölüm 40'ta C++ ile bir kütüphane yönetim sistemi yazmıştık. Bu bölümde C# kısmında öğrendiklerimizi aynı türden bir uygulamada birleştireceğiz: sınıflar (Bölüm 49), arayüzler (Bölüm 51), `record` (Bölüm 52), istisnalar (Bölüm 53), `Dictionary` (Bölüm 54), LINQ (Bölüm 56) ve dosya işlemleri (Bölüm 57).

Konu bir **envanter takip sistemidir**: ürün kaydı tutan, stok girişi ve çıkışı işleyen, kategori bazında rapor üreten ve her şeyi dosyada saklayan bir konsol uygulaması. Yeni bir özellik öğrenmeyeceğiz; amaç bilinen parçaların bir arada nasıl düzenlendiğini görmektir. Bölüm 40 ile karşılaştırınca çarpıcı olan, `unique_ptr`, sanal yıkıcı ve elle yazılan ayrıştırma kodunun yerini `record`, çöp toplayıcı ve LINQ'nun almasıdır.

60.1 Gereksinimler

1. Her ürünün sistemin verdiği bir numarası, adı, kategorisi, stok adedi ve birim fiyatı vardır.
2. Menüden şu işlemler yapılır: ürün ekle, stok girişi, stok çıkışı, listele, rapor, dosyaya kaydet, dosyadan yükle, çıkış.
3. Stok eksiye düşemez; olmayan numara, sayı olmayan girdi ve eksik alan programı çökertmez, açıklayıcı bir mesaj gösterilir.
4. Rapor, kategori başına ürün çeşidini, toplam adedi ve stok değerini (adet çarpı fiyat) ile eşik değerin altına düşmüş ürünleri gösterir.
5. Veriler okunabilir bir CSV dosyasında saklanır; bozuk dosya, eldeki verileri bozmaz.

60.2 Tasarım

Gereksinimlerdeki isimlerden sınıf adaylarını çıkaralım: ürün, envanter, depo. Ürün yalnızca veri taşır ve kendi içinde tutarlı bir bütündür; bu yüzden `record` için uygun bir adaydır (Bölüm 52). Envanter iş kurallarını denetler. Dosya işini ise bir arayüzün arkasına koyarız; böylece envanter verinin CSV'de mi, JSON'da mı, veritabanında mı durduğunu bilmez.

```
Urun (record)  <----  Envanter  ----> IDepo
No, Ad, Kategori,     Dictionary        |
Adet, Fiyat           <int, Urun>     CsvDepo

EnvanterHatasi : Exception     Menü ve ekran: top-level kod
```

İş kuralları `Envanter` içinde, kullanıcıyla konuşan kod ise dosyanın üstündeki menü döngüsündedir. Bölüm 40'taki katmanlı yapının aynısıdır.

> **Üç Dilde · Aynı tasarım üç dilde** *C · C++ · C#*
>
> C'de (Bölüm 20) ürün bir `struct`, kap elle büyüyen bir diziydi. C++'ta (Bölüm 40) kap `map` idi ve bellek akıllı işaretçilerle yönetiliyordu. C#'ta `record` hem veri taşır hem değer eşitliğini ve `with` ile kopyalayarak güncellemeyi kendiliğinden sağlar; bellek çöp toplayıcıya aittir.

60.3 Ürün ve Hata Sınıfı

`Urun` kaydı beş alanlıdır. Stok değeri alan olarak saklanmaz, özellik olarak hesaplanır: aynı bilgiyi iki yerde tutmak, biri değişip diğeri değişmeyince tutarsızlık doğurur.

```
record Urun(int No, string Ad, string Kategori, int Adet, decimal Fiyat)
{
    public decimal Deger => Adet * Fiyat;
}
```

Para gibi tam yuvarlanması gereken değerlerde `double` yerine `decimal` kullanırız (Bölüm 42). `record` değişmezdir; stoğu değiştirmek için `u with { Adet = yeni }` yazar, eski nesneyi atar ve sözlükteki girdiyi yenisiyle değiştiririz.

Kural ihlallerini istisnayla bildirelim (Bölüm 53). Ara katmanlar hata kodu taşımaz; mesajı yalnızca menü döngüsündeki tek bir `catch` yazar:

```
class EnvanterHatasi : Exception
{
    public EnvanterHatasi(string mesaj) : base(mesaj) { }
}
```

60.4 Envanter Sınıfı

Sınıf, ürünleri numaraya göre bir `Dictionary<int, Urun>` içinde tutar ve bir sonraki numarayı kendisi üretir. `Bul`, olmayan numara için sözlüğün `KeyNotFoundException`'ı yerine kendi hatamızı fırlatır; böylece program yalnızca tek tür hata bekler:

```
public Urun Bul(int no) =>
    urunler.TryGetValue(no, out var u)
        ? u
        : throw new EnvanterHatasi($"{no} numaralı ürün yok.");
```

`Ekle` girdiyi denetler. Alan ayracı `;` olduğundan ad ve kategoride bu karakteri kabul etmeyiz; aksi hâlde dosya yazılabilir ama geri okunamazdı:

```
public Urun Ekle(string ad, string kategori, int adet, decimal fiyat)
{
    Dogrula(ad, kategori, adet, fiyat);
    var u = new Urun(++sonNo, ad.Trim(), kategori.Trim(), adet, fiyat);
    urunler[u.No] = u;
    return u;
}
```

Stok girişi ve çıkışı tek metotta toplanır; çıkış, eksi fark olarak gelir. Yeterli stok yoksa nesne değiştirilmeden istisna fırlar:

```
public Urun StokDegistir(int no, int fark)
{
    var u = Bul(no);
    if (u.Adet + fark < 0)
        throw new EnvanterHatasi(
            $"Yetersiz stok: {u.Ad} için eldeki adet {u.Adet}.");
    var yeni = u with { Adet = u.Adet + fark };
    urunler[no] = yeni;
    return yeni;
}
```

> **▲ Sık Yapılan Hata · Dönüş değeri tuple olan LINQ sorguları**
>
> `Select` içinde adsız tuple üretirseniz alan adları kaybolur ve çağıran `Item1`, `Item2` yazmak zorunda kalır. Metodun dönüş tipinde alan adlarını bildirmek (`IEnumerable<(string Kategori, int Cesit)>`) çağıran tarafta okunabilir adlar sağlar.

60.5 LINQ ile Raporlar

Rapor, ürünleri kategoriye göre gruplayıp her gruptan üç özet çıkarır (Bölüm 56). Sonuç, adlandırılmış alanlı tuple'lardır:

```
public IEnumerable<(string Kategori, int Cesit, int Adet, decimal Deger)>
    KategoriRaporu() =>
    urunler.Values
        .GroupBy(u => u.Kategori)
        .OrderBy(g => g.Key, StringComparer.Ordinal)
        .Select(g => (g.Key, g.Count(),
                      g.Sum(u => u.Adet), g.Sum(u => u.Deger)));

public IEnumerable<Urun> DusukStok(int esik) =>
    Hepsi.Where(u => u.Adet <= esik);
```

`Hepsi`, ürünleri numara sırasıyla veren bir özelliktir. Sözlükte sıra güvencesi olmadığından listelemeden önce her seferinde `OrderBy(u => u.No)` uygularız. Bölüm 40'ta `map` bu sırayı kendiliğinden vermişti; `Dictionary` vermez.

60.6 Depo Arayüzü ve CSV

Arayüz yalnızca iki iş tanımlar. Envanter sınıfı bu arayüzü bilir, CSV'yi bilmez (Bölüm 51):

```
interface IDepo
{
    void Kaydet(IEnumerable<Urun> urunler);
    List<Urun> Yukle();
}
```

`CsvDepo` her ürünü `no;ad;kategori;adet;fiyat` biçiminde bir satıra yazar. Ondalık sayı yazarken ve okurken kültürü açıkça `InvariantCulture` verdik; böylece dosya, programı hangi dilde çalıştıran makinede olursa olsun aynı biçimde okunur (Bölüm 43). Yükleme sırasında her satır doğrulanır, bozuk satırda satır numarasıyla istisna fırlatılır:

```
Urun Ayristir(string satir, int sira)
{
    string[] p = satir.Split(';');
    if (p.Length != 5
        || !int.TryParse(p[0], out int no)
        || !int.TryParse(p[3], out int adet)
        || !decimal.TryParse(p[4], NumberStyles.Number,
               CultureInfo.InvariantCulture, out decimal fiyat))
        throw new EnvanterHatasi($"Dosyada {sira}. satır bozuk.");
    return new Urun(no, p[1], p[2], adet, fiyat);
}
```

`Envanter.Yukle` bu listeyi hemen kullanmaz: önce yeni bir sözlük kurar, aynı numaranın iki kez geçmesi ya da eksi adet gibi tutarsızlıkları yakalar, hepsi sağlam ise eldeki sözlüğün yerine koyar. Hata ortasında çıkarsa eldeki veri aynen kalır. Bu, Bölüm 40'taki "önce geçici kaba oku, sonra `swap` et" fikrinin C# karşılığıdır.

60.7 Menü ve Ana Program

Menü döngüsü bir `while` ve bir `switch` ifadesinden oluşur. Her komut kendi metoduna yönlenir, hata ise döngünün içindeki tek `try`/`catch` ile yakalanır. Girdi okuyan iki küçük yerel işlev (`Sayi`, `Ondalik`) metni çevirir, çeviremezse kendi hatamızı fırlatır; böylece `int.Parse`'ın `FormatException`'ı ekrana ham hâliyle hiç ulaşmaz.

Program başında `CultureInfo.CurrentCulture`'u `InvariantCulture` yaparız: bölümdeki çıktılar ondalık ayırıcı olarak nokta kullansın. Türkçe kültürde `F2` biçimi virgül üretirdi (Bölüm 42 ve 43).

60.8 Tam Program

Parçaları birleştirince tam program aşağıdaki gibi olur. Tek dosyadır; top-level kodun altında sınıflar tanımlanmıştır. Proje klasöründe `dotnet run` ile çalıştırılır.

envanter.cs

```
using System.Globalization;

CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;

var envanter = new Envanter();
IDepo depo = new CsvDepo("envanter.csv");
const int DusukEsik = 5;

Console.WriteLine("1 Ekle 2 Giriş 3 Çıkış 4 Liste 5 Rapor 6 Kaydet 7 Yükle 0 Bitir");
while (true)
{
    Console.Write("> ");
    string? komut = Console.ReadLine();
    if (komut is null || komut.Trim() == "0") break;
    try
    {
        switch (komut.Trim())
        {
            case "1": UrunEkle(); break;
            case "2": StokIslemi(+1); break;
            case "3": StokIslemi(-1); break;
            case "4": Listele(); break;
            case "5": Raporla(); break;
            case "6":
                depo.Kaydet(envanter.Hepsi);
                Console.WriteLine($"{envanter.Say} ürün kaydedildi.");
                break;
            case "7":
                envanter.Yukle(depo.Yukle());
                Console.WriteLine($"{envanter.Say} ürün yüklendi.");
                break;
            default:
                throw new EnvanterHatasi("Bilinmeyen komut.");
        }
    }
    catch (EnvanterHatasi e)
    {
        Console.WriteLine($"Hata: {e.Message}");
    }
}
Console.WriteLine("Güle güle.");

string Sor(string soru)
{
    Console.Write(soru);
    return Console.ReadLine() ?? "";
}

int Sayi(string s) =>
    int.TryParse(s.Trim(), out int n)
        ? n
        : throw new EnvanterHatasi($"Sayı bekleniyordu: '{s}'");

decimal Ondalik(string s) =>
    decimal.TryParse(s.Trim(), NumberStyles.Number,
                     CultureInfo.InvariantCulture, out decimal d)
        ? d
        : throw new EnvanterHatasi($"Fiyat bekleniyordu: '{s}'");

void UrunEkle()
{
    string[] p = Sor("Ürün (ad;kategori;adet;fiyat): ").Split(';');
```

```
    if (p.Length != 4)
        throw new EnvanterHatasi("Dört alan bekleniyordu.");
    var u = envanter.Ekle(p[0], p[1], Sayi(p[2]), Ondalik(p[3]));
    Console.WriteLine($"Eklendi. No: {u.No}");
}

void StokIslemi(int yon)
{
    string[] p = Sor("Ürün no ve miktar: ")
        .Split(' ', StringSplitOptions.RemoveEmptyEntries);
    if (p.Length != 2)
        throw new EnvanterHatasi("İki sayı bekleniyordu.");
    int miktar = Sayi(p[1]);
    if (miktar <= 0)
        throw new EnvanterHatasi("Miktar sıfırdan büyük olmalı.");
    var u = envanter.StokDegistir(Sayi(p[0]), yon * miktar);
    Console.WriteLine($"{u.Ad}: yeni stok {u.Adet}");
}

void Listele()
{
    Console.WriteLine($"{"No",3} {"Ad",-20} {"Kategori",-10} " +
                      $"{"Adet",5} {"Fiyat",9}");
    foreach (var u in envanter.Hepsi)
        Console.WriteLine($"{u.No,3} {u.Ad,-20} {u.Kategori,-10} " +
                          $"{u.Adet,5} {u.Fiyat,9:F2}");
}

void Raporla()
{
    foreach (var (kategori, cesit, adet, deger) in envanter.KategoriRaporu())
        Console.WriteLine($"{kategori,-10} {cesit} çeşit, {adet} adet, " +
                          $"değer {deger:F2}");
    Console.WriteLine($"Toplam değer: {envanter.ToplamDeger():F2}");
    var az = envanter.DusukStok(DusukEsik).Select(u => u.Ad).ToList();
    Console.WriteLine(az.Count == 0
        ? "Düşük stok yok."
        : $"Düşük stok (<= {DusukEsik}): {string.Join(", ", az)}");
}

record Urun(int No, string Ad, string Kategori, int Adet, decimal Fiyat)
{
    public decimal Deger => Adet * Fiyat;
}

class EnvanterHatasi : Exception
{
    public EnvanterHatasi(string mesaj) : base(mesaj) { }
}

class Envanter
{
    private readonly Dictionary<int, Urun> urunler = new();
    private int sonNo = 0;

    public int Say => urunler.Count;

    public IEnumerable<Urun> Hepsi => urunler.Values.OrderBy(u => u.No);

    public Urun Bul(int no) =>
        urunler.TryGetValue(no, out var u)
            ? u
            : throw new EnvanterHatasi($"{no} numaralı ürün yok.");
```

```
    public Urun Ekle(string ad, string kategori, int adet, decimal fiyat)
    {
        Dogrula(ad, kategori, adet, fiyat);
        var u = new Urun(++sonNo, ad.Trim(), kategori.Trim(), adet, fiyat);
        urunler[u.No] = u;
        return u;
    }

    public Urun StokDegistir(int no, int fark)
    {
        var u = Bul(no);
        if (u.Adet + fark < 0)
            throw new EnvanterHatasi(
                $"Yetersiz stok: {u.Ad} için eldeki adet {u.Adet}.");
        var yeni = u with { Adet = u.Adet + fark };
        urunler[no] = yeni;
        return yeni;
    }

    public decimal ToplamDeger() => urunler.Values.Sum(u => u.Deger);

    public IEnumerable<(string Kategori, int Cesit, int Adet, decimal Deger)>
        KategoriRaporu() =>
        urunler.Values
            .GroupBy(u => u.Kategori)
            .OrderBy(g => g.Key, StringComparer.Ordinal)
            .Select(g => (g.Key, g.Count(),
                          g.Sum(u => u.Adet), g.Sum(u => u.Deger)));

    public IEnumerable<Urun> DusukStok(int esik) =>
        Hepsi.Where(u => u.Adet <= esik);

    public void Yukle(List<Urun> liste)
    {
        var yeni = new Dictionary<int, Urun>();
        foreach (var u in liste)
        {
            Dogrula(u.Ad, u.Kategori, u.Adet, u.Fiyat);
            if (!yeni.TryAdd(u.No, u))
                throw new EnvanterHatasi($"{u.No} numarası iki kez geçiyor.");
        }
        urunler.Clear();
        foreach (var (no, u) in yeni) urunler[no] = u;
        sonNo = yeni.Count == 0 ? 0 : yeni.Keys.Max();
    }

    private static void Dogrula(string ad, string kategori,
                                int adet, decimal fiyat)
    {
        if (string.IsNullOrWhiteSpace(ad)
            || string.IsNullOrWhiteSpace(kategori))
            throw new EnvanterHatasi("Ad ve kategori boş olamaz.");
        if (ad.Contains(';') || kategori.Contains(';'))
            throw new EnvanterHatasi("Ad ve kategoride ';' kullanılamaz.");
        if (adet < 0 || fiyat < 0)
            throw new EnvanterHatasi("Adet ve fiyat eksi olamaz.");
    }
}

interface IDepo
{
    void Kaydet(IEnumerable<Urun> urunler);
    List<Urun> Yukle();
}
```

```
class CsvDepo : IDepo
{
    private readonly string yol;

    public CsvDepo(string yol) => this.yol = yol;

    public void Kaydet(IEnumerable<Urun> urunler)
    {
        var satirlar = urunler.Select(u => string.Join(';',
            u.No, u.Ad, u.Kategori, u.Adet,
            u.Fiyat.ToString(CultureInfo.InvariantCulture)));
        File.WriteAllLines(yol, satirlar);
    }

    public List<Urun> Yukle()
    {
        if (!File.Exists(yol))
            throw new EnvanterHatasi($"{yol} dosyası bulunamadı.");
        var liste = new List<Urun>();
        string[] satirlar = File.ReadAllLines(yol);
        for (int i = 0; i < satirlar.Length; i++)
            liste.Add(Ayristir(satirlar[i], i + 1));
        return liste;
    }

    private static Urun Ayristir(string satir, int sira)
    {
        string[] p = satir.Split(';');
        if (p.Length != 5
            || !int.TryParse(p[0], out int no)
            || !int.TryParse(p[3], out int adet)
            || !decimal.TryParse(p[4], NumberStyles.Number,
                   CultureInfo.InvariantCulture, out decimal fiyat))
            throw new EnvanterHatasi($"Dosyada {sira}. satır bozuk.");
        return new Urun(no, p[1], p[2], adet, fiyat);
    }
}
```

```
ÇIKTI
1 Ekle 2 Giriş 3 Çıkış 4 Liste 5 Rapor 6 Kaydet 7 Yükle 0 Bitir
> 1
Ürün (ad;kategori;adet;fiyat): Kablosuz Fare;Elektronik;25;450.50
Eklendi. No: 1
> 1
Ürün (ad;kategori;adet;fiyat): Dizüstü Bilgisayar;Elektronik;4;32500
Eklendi. No: 2
> 1
Ürün (ad;kategori;adet;fiyat): Çalışma Masası;Mobilya;3;5200.75
Eklendi. No: 3
> 1
Ürün (ad;kategori;adet;fiyat): Defter;Kırtasiye;120;35
Eklendi. No: 4
> 4
 No Ad                   Kategori    Adet     Fiyat
  1 Kablosuz Fare        Elektronik    25    450.50
  2 Dizüstü Bilgisayar   Elektronik     4  32500.00
  3 Çalışma Masası       Mobilya        3   5200.75
  4 Defter               Kırtasiye    120     35.00
> 3
Ürün no ve miktar: 2 3
Dizüstü Bilgisayar: yeni stok 1
> 3
Ürün no ve miktar: 2 3
Hata: Yetersiz stok: Dizüstü Bilgisayar için eldeki adet 1.
> 2
Ürün no ve miktar: 4 40
Defter: yeni stok 160
> 5
Elektronik 2 çeşit, 26 adet, değer 43762.50
Kırtasiye  1 çeşit, 160 adet, değer 5600.00
Mobilya    1 çeşit, 3 adet, değer 15602.25
Toplam değer: 64964.75
Düşük stok (<= 5): Dizüstü Bilgisayar, Çalışma Masası
> 6
4 ürün kaydedildi.
> 1
Ürün (ad;kategori;adet;fiyat): Kalem;Kırtasiye;abc;5
Hata: Sayı bekleniyordu: 'abc'
> 7
4 ürün yüklendi.
> 0
Güle güle.
```

Dört gözlem yapalım:

- Liste ve rapor satırları sütun genişlikleriyle (`{u.Ad,-20}` , `{u.Fiyat,9:F2}`) hizalandı (Bölüm 43). Adetler ve fiyatlar sağa, metinler sola yaslıdır.
- Dizüstü bilgisayardan üç adet çıkınca stok 1'e düştü. İkinci çıkış denemesi `StokDegistir` içinde `EnvanterHatasi` fırlattı; nesne değişmedi ve `catch` mesajı yazıp menüye döndü.
- Adet alanına `abc` yazıldığında `Sayi` yerel işlevi `FormatException` yerine kendi hatamızı üretti; ürün eklenmedi ve numara sayacı ilerlemedi.
- Rapordaki düşük stok satırı `Where` ile süzülen iki ürünü `string.Join` ile birleştirdi. Çalışma masasının değeri 3 x 5200.75 = 15602.25'tir; `decimal` olduğu için küsurat kayması yoktur.

İpucu · Envanteri ekrandan bağımsız tutun

`Envanter` sınıfında `Console` geçmez. Bu yüzden birkaç satırlık bir test kodu ürün ekleyip stok düşürerek istisnanın fırlayıp fırlamadığını menü olmadan sınayabilir. İleride JSON'a ya da veritabanına geçmek de yalnızca yeni bir `IDepo` gerçeklemesi yazmayı gerektirir.

Not · JSON ile saklama

Aynı arayüzü `System.Text.Json` ile de gerçekleyebiliriz; NuGet paketi gerekmez, .NET ile birlikte gelir. `JsonSerializer.Serialize(urunler)` `record` nesnelerini doğrudan yazar, `JsonSerializer.Deserialize<List<Urun>>(metin)` okur. CSV'yi seçtik, çünkü biçimi görmek ve elle bozup hata yolunu denemek kolaydır.

Geliştirme Önerileri

Program çalışıyor ve gereksinimleri karşılıyor. Bir uygulamayı öğrenmenin yolu onu kendi isteklerinize göre değiştirmektir. Öneriler yaklaşık artan zorluk sırasıyla verilmiştir; her birini bağımsız yapabilirsiniz.

1. **Ada göre arama.** Adında verilen sözcük geçen ürünleri listeleyen bir komut ekleyin. Büyük/küçük harf farkını yok saymak için `StringComparison.OrdinalIgnoreCase` kullanın (Bölüm 48).
2. **Sıralı listeleme.** Ürünleri ada ya da stok değerine göre sıralayan bir seçenek yazın; `OrderBy` ve `ThenBy` ile birden çok ölçüt deneyin (Bölüm 56).
3. **Ürün silme.** Silme komutu ekleyin. Stoğu sıfırdan büyük olan ürünün silinmesini bir istisnayla reddedin ya da kullanıcıdan onay isteyin.
4. **Ayarlanabilir eşik.** Düşük stok eşiğini sabitten çıkarıp ürün başına bir `EsikAdet` alanı yapın. `record`'a yeni alan eklemenin `CsvDepo` ve `Ayristir` üzerindeki etkisini izleyin.
5. **Stok hareketi günlüğü.** Her giriş ve çıkış için tarih, ürün numarası ve fark tutan bir `record` ve ayrı bir `List` ekleyin; raporda belirli bir günün hareketlerini LINQ ile gösterin.
6. **JSON deposu.** `IDepo`'yu `System.Text.Json` ile gerçekleyen bir `JsonDepo` yazın ve `CsvDepo` yerine `main` içinde bir satırla değiştirin. Envanter sınıfına dokunmanız gerekiyor mu?
7. **Tırnaklı alanlar.** Ayraç `;` olduğundan adda bu karakter kullanılamıyor. Alanları tırnak içine alan bir yazma ve okuma kuralı tasarlayın.
8. **Birim testi.** Ayrı bir test projesi açıp `Envanter` için xUnit testleri yazın: yetersiz stok, tekrarlanan numara ve bozuk dosya senaryolarını sınayın.

Bölüm Özeti

- Gereksinimlerdeki isimler sınıf adaylarını, fiiller metotları gösterir; yalnızca veri taşıyan bütünler `record` için uygundur.
- `record` değişmezdir; güncelleme `with` ile yeni bir nesne üretmek ve sözlükteki girdiyi değiştirmekle yapılır.
- Türetilebilen değerler (stok değeri) alan olarak değil, hesaplanan özellik olarak tutulur; böylece tutarsızlık doğmaz.

- Para gibi tam doğruluk isteyen değerlerde `decimal` kullanılır.
- `Dictionary` hızlı arama sağlar ama sıra vermez; listelemeden önce `OrderBy` uygulanır.
- LINQ'nun `GroupBy`, `Sum`, `Count` ve `Where` yöntemleri raporları döngü yazmadan üretir; adlandırılmış tuple'lar sonuçları okunur kılar.
- Kural ihlalleri kendi istisna sınıfımızla bildirilir; dış kütüphane istisnaları (`FormatException`, `KeyNotFoundException`) sınırda çevrilir ve tek bir `catch` ekrana mesaj yazar.
- Depolama bir arayüzün arkasına alınınca iş kuralı sınıfı biçimden bağımsız kalır; CSV yerine JSON ya da veritabanı yalnızca yeni bir gerçekleme demektir.
- Yüklemede önce yeni sözlük kurup doğrulamak, sonra eldekinin yerine koymak hata durumunda veriyi korur.
- Ondalık sayıları dosyaya yazarken `InvariantCulture` kullanmak, dosyanın her ortamda aynı okunmasını sağlar.
- Girdi-çıktı ile iş kuralını ayırmak, programı sınamayı ve başka bir arayüze taşımayı kolaylaştırır.

KISIM IV

Karşılaştırma ve Kapanış

Üç dil yan yana

Üç dili de tanıdıktan sonra onları yan yana koyacağız. Aynı problemleri üç dilde çözecek, sözdiziminden bellek yönetimine, hata yönetiminden performansa kadar farkları ve benzerlikleri tablolarla özetleyeceğiz. Son bölümde ise her dilin güçlü ve zayıf yönlerini, hangi alanda hangisinin tercih edildiğini ve bu kitaptan sonra nelerin öğrenilebileceğini ele alacağız.

BÖLÜM 61

Üç Dil Yan Yana

BU BÖLÜMDE

- Aynı problemi C, C++ ve C#'ta çözerek üç dilin yaklaşımlarını karşılaştırmayı
- Dosyadan okuma, nesne tasarımı ve sözcük sayma işlerinin her dilde nasıl göründüğünü
- Sözdizimi, tip sistemi, bellek yönetimi ve hata yönetimi gibi başlıklarda dilleri tablolarla ayırt etmeyi
- Üç dilin ortak C sözdizimi mirasını ve ortak kalıplarını
- Bir dilden diğerine geçerken düşülen tipik tuzakları tanımayı
- Bir probleme hangi dilin araçlarıyla nasıl yaklaşılacağını gerekçelendirmeyi

Kitabın bu noktasına kadar üç dili tarih sırasıyla öğrendik: Bölüm 4–20'de C'yi, Bölüm 21–40'ta C++'ı, Bölüm 41–60'ta C#'ı. Her dilin kendi içinde tutarlı bir anlatımı vardı; ama dilleri birbirinden ayıran şey çoğu zaman tek tek özelliklerde değil, **aynı işin nasıl yapıldığındadır**. Bu bölümde aynı problemleri üç dilde de tam program olarak çözeceğiz ve ardından farkları sistemli biçimde karşılaştıracağız.

Her vakada üç program da aynı girdiyle aynı çıktıyı üretir. Programlar bilerek bir dilin en gösterişli özelliklerini değil, o dilde **deyimsel** (idiomatic) sayılan çözümü kullanır: C'de dizi, işaretçi ve standart kütüphane fonksiyonları; C++'ta `vector`, `map` ve algoritmalar; C#'ta koleksiyonlar ve LINQ. Bölüm boyunca başvurduğumuz konular ilgili bölümlerde ayrıntılı işlenmiştir; burada yalnızca yan yana koyup yorumlayacağız.

> **Not · Derleme komutları**
>
> Programlar şu komutlarla derlenip çalıştırılmıştır. C# için bir proje oluşturup (`dotnet new console`) `Program.cs` içeriğini değiştirmek yeterlidir; Bölüm 41'de anlatıldığı gibi.
>
> TERMİNAL
> ```
> $ gcc -std=c17 -Wall istatistik.c -o istatistik -lm
> $ g++ -std=c++17 -Wall istatistik.cpp -o istatistik
> $ dotnet run
> ```

61.1 Vaka 1: Dosyadan Sayı Okuyup İstatistik Çıkarmak

Problem: Bir metin dosyasında boşlukla ya da satır sonuyla ayrılmış gerçek sayılar var. Sayıların adedini, toplamını, ortalamasını, medyanını, en küçüğünü, en büyüğünü ve (popülasyon) standart sapmasını yazdıralım.

Programların kendi kendine yeterli olması için her biri önce `sayilar.txt` dosyasını yazıyor, sonra aynı dosyayı okuyor. Dosyanın içeriği şudur:

```
72.5 80 65 90.5
88 79 95 60
```

Medyan için sayıların sıralanması gerekir. Eleman sayısı çiftse ortadaki iki değerin ortalaması alınır: sıralı dizi `60 65 72.5 79 80 88 90.5 95` olduğundan medyan `(79 + 80) / 2 = 79.5` olur.

C çözümü

C'de dizinin kaç elemanlı olacağını önceden bilmediğimiz için dinamik bellek kullanırız (Bölüm 13): `malloc` ile başlayıp dolduğunda `realloc` ile kapasiteyi ikiye katlarız. Dosya `fopen` / `fscanf` ile okunur (Bölüm 17), sıralama için `qsort` gerekir ve karşılaştırma fonksiyonunu biz yazarız.

istatistik.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

static int kucukse(const void *a, const void *b)
{
    double x = *(const double *)a, y = *(const double *)b;
    return (x > y) - (x < y);
}

int main(void)
{
    /* önce veri dosyasını üretelim */
    FILE *yaz = fopen("sayilar.txt", "w");
    if (yaz == NULL) { perror("sayilar.txt"); return 1; }
    fputs("72.5 80 65 90.5\n88 79 95 60\n", yaz);
    fclose(yaz);

    FILE *oku = fopen("sayilar.txt", "r");
    if (oku == NULL) { perror("sayilar.txt"); return 1; }

    size_t kapasite = 4, n = 0;
    double *dizi = malloc(kapasite * sizeof *dizi);
    if (dizi == NULL) { fclose(oku); return 1; }
    double x;
    while (fscanf(oku, "%lf", &x) == 1) {
        if (n == kapasite) {
            kapasite *= 2;
            double *yeni = realloc(dizi, kapasite * sizeof *dizi);
            if (yeni == NULL) { free(dizi); fclose(oku); return 1; }
            dizi = yeni;
        }
        dizi[n++] = x;
    }
    fclose(oku);

    double toplam = 0;
    for (size_t i = 0; i < n; i++) toplam += dizi[i];
    double ort = toplam / n;
    double kareler = 0;
    for (size_t i = 0; i < n; i++) kareler += (dizi[i] - ort) * (dizi[i] - ort);

    qsort(dizi, n, sizeof *dizi, kucukse);
    double medyan = (n % 2 == 1) ? dizi[n / 2]
                                 : (dizi[n / 2 - 1] + dizi[n / 2]) / 2;

    printf("Adet     : %zu\n", n);
```

```
    printf("Toplam   : %.2f\n", toplam);
    printf("Ortalama : %.2f\n", ort);
    printf("Medyan   : %.2f\n", medyan);
    printf("En kucuk : %.2f\n", dizi[0]);
    printf("En buyuk : %.2f\n", dizi[n - 1]);
    printf("Std sapma: %.2f\n", sqrt(kareler / n));
    free(dizi);
    return 0;
}
```

```
ÇIKTI
Adet     : 8
Toplam   : 630.00
Ortalama : 78.75
Medyan   : 79.50
En kucuk : 60.00
En buyuk : 95.00
Std sapma: 11.56
```

Dikkat edilecek noktalar: Her `fopen` sonucu denetlenir; `realloc`'un sonucu önce ayrı bir işaretçiye atanır ki başarısız olduğunda eski blok kaybolmasın; belleği `free` ile geri vermek programcının sorumluluğundadır.

C++ çözümü

C++'ta aynı iş çok daha kısadır. `vector<double>` kapasiteyi kendisi yönetir, `ifstream` dosyayı kapsam sonunda kendiliğinden kapatır, `accumulate` ve `sort` hazırdır (Bölüm 26, 36, 37). Bellek yönetimi RAII sayesinde görünmezdir (Bölüm 29).

istatistik.cpp

```
#include <algorithm>
#include <cmath>
#include <fstream>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;

int main()
{
    // önce veri dosyasını üretelim
    {
        ofstream yaz("sayilar.txt");
        yaz << "72.5 80 65 90.5\n88 79 95 60\n";
    }

    ifstream oku("sayilar.txt");
    if (!oku) {
        cerr << "sayilar.txt acilamadi\n";
        return 1;
    }
    vector<double> dizi;
    double x;
    while (oku >> x) dizi.push_back(x);

    double toplam = accumulate(dizi.begin(), dizi.end(), 0.0);
    double ort = toplam / dizi.size();
    double kareler = 0;
    for (double d : dizi) kareler += (d - ort) * (d - ort);

    sort(dizi.begin(), dizi.end());
```

```
    size_t n = dizi.size();
    double medyan = (n % 2 == 1) ? dizi[n / 2]
                                 : (dizi[n / 2 - 1] + dizi[n / 2]) / 2;

    cout.setf(ios::fixed);
    cout.precision(2);
    cout << "Adet     : " << n << '\n';
    cout << "Toplam   : " << toplam << '\n';
    cout << "Ortalama : " << ort << '\n';
    cout << "Medyan   : " << medyan << '\n';
    cout << "En kucuk : " << dizi.front() << '\n';
    cout << "En buyuk : " << dizi.back() << '\n';
    cout << "Std sapma: " << sqrt(kareler / n) << '\n';
}
```

```
ÇIKTI
Adet     : 8
Toplam   : 630.00
Ortalama : 78.75
Medyan   : 79.50
En kucuk : 60.00
En buyuk : 95.00
Std sapma: 11.56
```

Bir ayrıntı: dosyayı yazan `ofstream` iç içe bir blok içinde tanımlandı. Blok biter bitmez nesne yıkılır, tampon diske boşaltılır ve dosya kapanır; bu sayede hemen ardından okunabilir.

C# çözümü

C#'ta dosyanın tamamı tek çağrıyla (`File.ReadAllText`) okunur, metin `Split` ile parçalanır ve LINQ zinciri `Select`, `OrderBy`, `ToArray` ile diziyi üretir (Bölüm 56, 57). `Sum` gibi toplulaştırma metotları döngüyü ortadan kaldırır. `double.Parse` çağrısında `CultureInfo.InvariantCulture` vermemizin nedeni, Türkçe kültür ayarında ondalık ayracın virgül olması ve `72.5` yazısının yanlış okunmasıdır (Bölüm 43).

▸ **Istatistik.cs**

```
using System.Globalization;

// önce veri dosyasını üretelim
File.WriteAllText("sayilar.txt", "72.5 80 65 90.5\n88 79 95 60\n");

string metin = File.ReadAllText("sayilar.txt");
double[] dizi = metin
    .Split((char[]?)null, StringSplitOptions.RemoveEmptyEntries)
    .Select(s => double.Parse(s, CultureInfo.InvariantCulture))
    .OrderBy(d => d)
    .ToArray();

int n = dizi.Length;
double toplam = dizi.Sum();
double ort = toplam / n;
double kareler = dizi.Sum(d => (d - ort) * (d - ort));
double medyan = (n % 2 == 1) ? dizi[n / 2]
                             : (dizi[n / 2 - 1] + dizi[n / 2]) / 2;

Console.WriteLine($"Adet     : {n}");
Console.WriteLine($"Toplam   : {toplam:F2}");
Console.WriteLine($"Ortalama : {ort:F2}");
Console.WriteLine($"Medyan   : {medyan:F2}");
```

```
Console.WriteLine($"En kucuk : {dizi[0]:F2}");
Console.WriteLine($"En buyuk : {dizi[^1]:F2}");
Console.WriteLine($"Std sapma: {Math.Sqrt(kareler / n):F2}");
```

```
ÇIKTI
Adet     : 8
Toplam   : 630.00
Ortalama : 78.75
Medyan   : 79.50
En kucuk : 60.00
En buyuk : 95.00
Std sapma: 11.56
```

Son satırlarda `dizi[^1]` ifadesi sondan birinci eleman anlamına gelir (Bölüm 47). Çıktıdaki ondalıkları da `F2` biçim belirteci üretir.

Karşılaştırma

Konu	C	C++	C#
Dizi büyütme	`malloc` / `realloc` elle	`push_back` otomatik	LINQ ile hazır dizi
Dosya kapatma	`fclose` elle	yıkıcı otomatik	`File.ReadAllText` kendisi kapatır
Sıralama	`qsort` + karşılaştırma fonksiyonu	`std::sort`	`OrderBy` (yeni dizi üretir)
Toplam	döngü	`accumulate`	`Sum`
Hata durumu	`NULL` dönüş değeri denetimi	akış durumu (`if (!oku)`)	istisna (`FileNotFoundException`)
Yaklaşık satır sayısı	50	40	22

> ◆ **Kritik Nokta**
>
> C programı, bellek ve dosya yönetiminin tamamını programcıya bırakır; bu en çok kod gerektiren ama neyin ne zaman olduğunu en açık gösteren yaklaşımdır. C++ aynı sorumlulukları nesnelerin yaşam süresine bağlar. C# ise bellek yönetimini çöp toplayıcıya, dosya ve biçimlendirme işlerini zengin kütüphaneye devreder.

61.2 Vaka 2: Öğrenci Sınıfı

Problem: Her biri bir ad ve üç not içeren dört öğrenci olsun. Notların ortalamasını hesaplayalım, öğrencileri ortalamaya göre büyükten küçüğe sıralayıp yazdıralım, sınıf ortalamasını ve birinciyi gösterelim.

Bu vaka veriyi ve davranışı bir araya getirmenin üç yolunu karşılaştırır. Ad sütununun hizalı çıkması için örnek adlarda Türkçe karakter bilerek kullanılmadı: C'deki `printf("%-8s")` bayt sayar, karakter saymaz; UTF-8 kodlu `ş` iki bayttır ve hizayı bozar.

C çözümü

C'de veri `struct` ile, davranış ise yapıyı parametre alan **ayrı fonksiyonlarla** ifade edilir (Bölüm 14). Dizinin sıralanması için yine `qsort` ve bir karşılaştırma fonksiyonu gerekir. Fonksiyonun `const void *` parametrelerini `const Ogrenci *` işaretçisine dönüştürmeyi dilin örtük `void *` kuralı sağlar.

sinif.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char ad[20];
    int notlar[3];
} Ogrenci;

static double ortalama(const Ogrenci *o)
{
    return (o->notlar[0] + o->notlar[1] + o->notlar[2]) / 3.0;
}

static int ortalamayaGore(const void *a, const void *b)
{
    double x = ortalama(a), y = ortalama(b);
    return (x < y) - (x > y);   /* büyükten küçüğe */
}

int main(void)
{
    Ogrenci sinif[] = {
        {"Zeynep", {80, 90, 70}},
        {"Mehmet", {60, 55, 75}},
        {"Elif",   {95, 85, 90}},
        {"Can",    {70, 65, 80}},
    };
    size_t n = sizeof sinif / sizeof sinif[0];

    qsort(sinif, n, sizeof sinif[0], ortalamayaGore);

    for (size_t i = 0; i < n; i++)
        printf("%zu. %-8s %6.2f\n", i + 1, sinif[i].ad, ortalama(&sinif[i]));

    double genel = 0;
    for (size_t i = 0; i < n; i++) genel += ortalama(&sinif[i]);
    printf("Sinif ortalamasi: %.2f\n", genel / n);
    printf("Birinci: %s\n", sinif[0].ad);
    return 0;
}
```

```
ÇIKTI
1. Elif      90.00
2. Zeynep    80.00
3. Can       71.67
4. Mehmet    63.33
Sinif ortalamasi: 76.25
Birinci: Elif
```

C++ çözümü

C++'ta aynı veri ve davranış bir `class` içinde toplanır. Alanlar `private` olur, `ortalama()` bir **üye fonksiyon**dur ve `const` olması nesneyi değiştirmediğini garanti eder (Bölüm 30). Sıralama için karşılaştırma ölçütünü bir lambda olarak yerinde yazarız (Bölüm 36). Çıktının hizalanmasında `iomanip`'teki `setw`, `left` ve `right` kullanılır (Bölüm 23).

sinif.cpp

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Ogrenci {
public:
    Ogrenci(string ad, int n1, int n2, int n3)
        : ad_(move(ad)), notlar_{n1, n2, n3} {}

    const string& ad() const { return ad_; }

    double ortalama() const
    {
        return (notlar_[0] + notlar_[1] + notlar_[2]) / 3.0;
    }

private:
    string ad_;
    int notlar_[3];
};

int main()
{
    vector<Ogrenci> sinif = {
        {"Zeynep", 80, 90, 70},
        {"Mehmet", 60, 55, 75},
        {"Elif", 95, 85, 90},
        {"Can", 70, 65, 80},
    };

    sort(sinif.begin(), sinif.end(),
         [](const Ogrenci& a, const Ogrenci& b) {
             return a.ortalama() > b.ortalama();
         });

    cout << fixed << setprecision(2);
    int sira = 1;
    double genel = 0;
    for (const Ogrenci& o : sinif) {
        cout << sira++ << ". " << left << setw(8) << o.ad()
             << ' ' << right << setw(6) << o.ortalama() << '\n';
        genel += o.ortalama();
    }
    cout << "Sinif ortalamasi: " << genel / sinif.size() << '\n';
    cout << "Birinci: " << sinif.front().ad() << '\n';
}
```

```
ÇIKTI
1. Elif      90.00
2. Zeynep    80.00
3. Can       71.67
4. Mehmet    63.33
Sinif ortalamasi: 76.25
Birinci: Elif
```

C# çözümü

C# 12'nin **birincil yapıcısı** (primary constructor) ile sınıf başlığında parametreler tanımlanır; `Ad` yalnızca okunabilen bir özellik, `Ortalama` ise hesaplanan bir özelliktir (Bölüm 49). Hizalama için biçim dizesinde `{deger,-8}` ve `{deger,6:F2}` kullanıldı (Bölüm 43). LINQ'nun `OrderByDescending` ve `Average` metotları sıralama ve ortalama işini bir satıra indirir. Top-level kodun altında sınıf tanımlamak serbesttir; yalnızca tip tanımları deyimlerden sonra gelmelidir.

▸ **Sinif.cs**

```
var sinif = new List<Ogrenci>
{
    new("Zeynep", 80, 90, 70),
    new("Mehmet", 60, 55, 75),
    new("Elif", 95, 85, 90),
    new("Can", 70, 65, 80),
};

var sirali = sinif.OrderByDescending(o => o.Ortalama).ToList();

for (int i = 0; i < sirali.Count; i++)
    Console.WriteLine($"{i + 1}. {sirali[i].Ad,-8} {sirali[i].Ortalama,6:F2}");

Console.WriteLine($"Sinif ortalamasi: {sinif.Average(o => o.Ortalama):F2}");
Console.WriteLine($"Birinci: {sirali[0].Ad}");

class Ogrenci(string ad, int n1, int n2, int n3)
{
    public string Ad { get; } = ad;
    public double Ortalama => (n1 + n2 + n3) / 3.0;
}
```

```
ÇIKTI
1. Elif      90.00
2. Zeynep    80.00
3. Can       71.67
4. Mehmet    63.33
Sinif ortalamasi: 76.25
Birinci: Elif
```

Karşılaştırma

Konu	C	C++	C#
Veri + davranış	`struct` + serbest fonksiyonlar	`class` + üye fonksiyonlar	`class` + özellikler
Kapsülleme	yok (alanlar açık)	`private` alanlar	`private` alanlar, özellikler
Ortalama	`ortalama(&o)`	`o.ortalama()`	`o.Ortalama` (özellik)
Sıralama ölçütü	ayrı fonksiyon + `void *` dönüşümü	lambda	lambda (`o => o.Ortalama`)

Konu	C	C++	C#
Dizi tipi	sabit diziler, boyut elle	`vector<Ogrenci>`	`List<Ogrenci>`
Nesne ömrü	otomatik (yığın)	otomatik; yıkıcı çalışır	çöp toplayıcı karar verir

Üç Dilde · Aynı fikir, üç ifade *C · C++ · C#*

C'de "ortalama hesapla" bir fonksiyon çağrısıdır, nesne ve veri ayrıdır. C++'ta nesne kendi fonksiyonunu taşır ve çağrı noktası `nesne.fonksiyon()` olur. C#'ta ise hesap, dışarıdan bir alan okuma gibi görünen bir özellik (`o.Ortalama`) olarak sunulur; çağıran, bunun hesaplanan bir değer olduğunu bilmek zorunda değildir. Bu fark nesne yönelimli tasarımın C'den C#'a doğru artan soyutlama basamaklarını iyi gösterir.

61.3 Vaka 3: Kelime Frekansı

Problem: Boşlukla ayrılmış bir metindeki her kelimenin kaç kez geçtiğini bulalım. Sonuçları önce sayıya göre çoktan aza, eşit sayılarda alfabetik olarak sıralayıp yazdıralım. Bu vaka bir **sözlük** (dictionary) yapısının, yani anahtardan değere eşlemenin dillerdeki karşılığını gösterir.

C çözümü

C standart kütüphanesinde sözlük yoktur. Burada en basit çözümü seçtik: kelime ve sayacı tutan yapılardan oluşan bir dizi ve her yeni kelime için bu dizide **doğrusal arama**. Küçük veri için yeterlidir; büyük metinde arama süresi hızla artar ve bir sözlük (karma tablo) gerekir. Metni parçalamak için `strtok` kullanılır; bu fonksiyon metnin kendisini değiştirdiği için metin `char` dizisi olarak tanımlanmıştır, dizge sabiti olarak değil (Bölüm 11).

kelime.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define EN_COK 50

typedef struct {
    char kelime[20];
    int adet;
} Kayit;

static int karsilastir(const void *a, const void *b)
{
    const Kayit *x = a, *y = b;
    if (x->adet != y->adet) return y->adet - x->adet;   /* çoktan aza */
    return strcmp(x->kelime, y->kelime);                /* sonra alfabetik */
}

int main(void)
{
    char metin[] = "armut elma elma kiraz armut elma muz kiraz elma";
    Kayit tablo[EN_COK];
    int say = 0;

    for (char *k = strtok(metin, " "); k != NULL; k = strtok(NULL, " ")) {
        int i;
        for (i = 0; i < say; i++)
            if (strcmp(tablo[i].kelime, k) == 0) break;
```

```
        if (i == say) {                 /* yeni kelime */
            if (say == EN_COK) break;
            strncpy(tablo[say].kelime, k, sizeof tablo[say].kelime - 1);
            tablo[say].kelime[sizeof tablo[say].kelime - 1] = '\0';
            tablo[say].adet = 0;
            say++;
        }
        tablo[i].adet++;
    }

    qsort(tablo, say, sizeof tablo[0], karsilastir);
    for (int i = 0; i < say; i++)
        printf("%-6s %d\n", tablo[i].kelime, tablo[i].adet);
    return 0;
}
```

```
ÇIKTI
elma   4
armut  2
kiraz  2
muz    1
```

C++ çözümü

`std::map<string, int>` anahtara göre sıralı bir sözlüktür. `++sayac[kelime]` ifadesi anahtar yoksa onu sıfır değerle ekler, ardından artırır; bu tek satır C'deki arama ve ekleme mantığının tamamını karşılar (Bölüm 35). Sonra elemanları bir `vector`'e kopyalayıp **kararlı sıralama** (`stable_sort`) ile sayıya göre sıralarız. `map` zaten anahtarlara göre sıralı olduğundan, kararlı sıralama eşit sayıdaki kelimelerin alfabetik sırasını korur. Kapsayıcıyı dolaşırken C++17'nin yapısal bağlama (structured binding) sözdizimi kullanıldı.

kelime.cpp

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <map>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

int main()
{
    istringstream metin("armut elma elma kiraz armut elma muz kiraz elma");
    map<string, int> sayac;       // anahtara göre sıralı tutulur
    string kelime;
    while (metin >> kelime) ++sayac[kelime];

    vector<pair<string, int>> liste(sayac.begin(), sayac.end());
    stable_sort(liste.begin(), liste.end(),
                [](const auto& a, const auto& b) {
                    return a.second > b.second;   // alfabetik sıra korunur
                });

    for (const auto& [k, adet] : liste)
        cout << left << setw(6) << k << ' ' << adet << '\n';
}
```

```
ÇIKTI
elma   4
armut  2
kiraz  2
muz    1
```

C# çözümü

`Dictionary<string, int>` bir karma tablodur ve elemanları belirli bir sırada vermez; sıra bizim sorumluluğumuzdadır. `GetValueOrDefault` anahtar yoksa `0` döndürür. Sıralamayı LINQ'daki `OrderByDescending` ve `ThenBy` ile yapar, ikinci ölçütte `StringComparer.Ordinal` ile bayt sırasına göre karşılaştırırız. Böylece sonuç, bilgisayarın kültür ayarından bağımsız ve C/C++ çıktılarıyla aynı olur (Bölüm 48'deki kültür duyarlı karşılaştırma uyarısını hatırlayalım).

▸ Kelime.cs

```
string metin = "armut elma elma kiraz armut elma muz kiraz elma";

var sayac = new Dictionary<string, int>();
foreach (string kelime in metin.Split(' '))
    sayac[kelime] = sayac.GetValueOrDefault(kelime) + 1;

var sirali = sayac
    .OrderByDescending(p => p.Value)
    .ThenBy(p => p.Key, StringComparer.Ordinal);

foreach (var (kelime, adet) in sirali)
    Console.WriteLine($"{kelime,-6} {adet}");
```

```
ÇIKTI
elma   4
armut  2
kiraz  2
muz    1
```

Karşılaştırma

Konu	C	C++	C#
Sözlük yapısı	yok; elle dizi + doğrusal arama	`std::map` (ağaç, sıralı)	`Dictionary` (karma tablo)
Kelime ayırma	`strtok` (metni bozar)	`istringstream` ile `>>`	`Split(' ')`
Anahtar yoksa	kod ile ekle	`[]` otomatik ekler	`GetValueOrDefault` / `TryGetValue`
Sıralama	`qsort`, iki ölçüt tek fonksiyonda	`stable_sort` + lambda	`OrderByDescending().ThenBy()`
Dizge karşılaştırma	`strcmp`	`string::operator<`	`StringComparer.Ordinal`
Arama maliyeti	O(n)	O(log n)	ortalama O(1)

> **▲ Sık Yapılan Hata · Sözlük sırası**
>
> `std::map` anahtarları sıralı tutar, `std::unordered_map` ve C# `Dictionary` ise sıra vaadi vermez. Çıktının belirli bir sırada olması gerekiyorsa sıralamayı açıkça yapmak gerekir; bir kapsayıcının "denk geldiği" sıraya güvenilmez.

61.4 Üç Vakadan Çıkan Genel Resim

Üç vaka bir örüntü gösterir. C'de çözüm, **veri ve fonksiyonlar** üzerine kurulur; kütüphane küçüktür ve eksik parçaları (dinamik dizi, sözlük, sıralama ölçütü) programcı yazar ya da en yalın biçimde taklit eder. C++'ta aynı parçalar **standart kütüphane kapsayıcıları ve algoritmalarıdır**; programcı daha az kod yazar, ama bu kütüphanenin geniş sözlüğünü bilmesi gerekir. C#'ta ise bu parçalar bir adım daha yukarıda, **dil ve kütüphane bütünleşik** olarak sunulur: özellikler, LINQ, birincil yapıcı gibi yapılar işi daha da kısaltır.

Bu kısalık bedelsiz değildir. C programı her satırda ne olduğunu açıkça gösterir ve çalışma zamanı gerektirmez. C++ kodunda kısa bir satırın arkasında bellek ayırma, kopyalama ve yıkıcı çağrıları saklıdır. C# kodunda ise çöp toplayıcı ve çalışma zamanı kütüphanesi devrededir.

61.5 Ortak Miras: Aynı Sözdiziminin İzleri

Üç programı yan yana koyduğumuzda dikkat çeken şey, ne kadar benzer göründükleridir. Bunun nedeni Bölüm 2'de izlediğimiz soy ağacıdır: C++ doğrudan C'nin üzerine, C# ise C ve C++ sözdiziminin üzerine kurulmuştur. Aşağıdaki öğeler üç dilde de aynı yazılır ve aynı işi görür:

- Küme parantezleriyle blok, noktalı virgülle deyim sonu: `{ ... }`, `;`
- Aritmetik, karşılaştırma ve mantıksal operatörler: `+ - * / %`, `== != < > <= >=`, `&& || !`
- Bileşik atama ve artırma: `+=`, `-=`, `++`, `--`
- Koşul ve döngü deyimleri: `if` / `else`, `switch`, `for`, `while`, `do` / `while`, `break`, `continue`, `return`
- Üçlü koşul operatörü: `koşul ? a : b`
- Tek satırlık `//` ve çok satırlık `/* */` yorumlar
- Fonksiyonun tipi başa yazılan bildirimi: `double ortalama(int n)`
- Tip adları: `int`, `double`, `char`, `float`, `long`; bool için `bool` (C'de `<stdbool.h>` ile)
- Dizi indeksi `a[i]` ve sıfırdan başlayan indeks

Bu ortak yüzey sayesinde bir dili bilen biri diğerini **okuyabilir**. Farklar sözdiziminde değil, **kurallarda** yani tiplerin, belleğin ve hataların nasıl yönetildiğindedir. Aşağıdaki tablolar bu farkları başlık başlık toplar.

61.6 Karşılaştırma Tabloları

Sözdizimi ve Program Yapısı

Özellik	C	C++	C#
Giriş noktası	`int main(void)`	`int main()`	`Main` ya da top-level deyimler
Başlık dosyaları	`#include`	`#include` (C++20'den modüller de var)	yok; `using` ve ad alanı

Özellik	C	C++	C#
Ad alanı	yok	`namespace`	`namespace`
Derleme birimi	`.c` + `.h`	`.cpp` + `.h` / `.hpp`	`.cs` (bildirim ile tanım ayrılmaz)
`bool`	C99 `_Bool`, `<stdbool.h>` ile `bool`	yerleşik `bool`	yerleşik `bool`, sayı değil
String sabiti	`"..."` (`char` dizisi)	`"..."` ve `std::string`	`"..."` (`string` nesnesi)
Çıktı	`printf`	`cout <<`	`Console.WriteLine`
Satır içi biçim	`printf("%d", x)`	`cout << x`	`$"{x}"`
Önişlemci	var	var	yok (`#if` benzeri sınırlı yönergeler)

Tip Sistemi

Özellik	C	C++	C#
`int` boyutu	platforma bağlı (en az 16 bit)	platforma bağlı	her zaman 32 bit
Sabit genişlikli tipler	`<stdint.h>` : `int32_t`	`<cstdint>`	dilin kendisi: `int`, `long`, `short` …
Tip denetimi	zayıf (örtük işaretçi dönüşümleri)	daha sıkı, hâlâ geniş dönüşümler	sıkı; yalnızca güvenli dönüşümler örtük
Tip çıkarımı	yok (C23'te `auto` sınırlı)	`auto`, `decltype`	`var`
Boş değer	`NULL`	`nullptr`	`null`, `int?`, nullable referans tipleri
Değer/referans ayrımı	işaretçi ile elle	değer + referans (`&`) + işaretçi	tipin kendisi belirler: `struct` değer, `class` referans
Birleşik tipler	`struct`, `union`, `enum`	`class`, `struct`, `union`, `enum class`	`class`, `struct`, `record`, `enum`, `interface`
Dönüşüm	`(tip)x`	`static_cast<tip>(x)`	`(tip)x`, `Convert`, `Parse`

Bellek Yönetimi

Özellik	C	C++	C#
Yığın (stack)	yerel değişkenler	yerel değişkenler ve nesneler	değer tipleri, yerel referanslar
Öbek (heap)	`malloc` / `free`	`new` / `delete`, akıllı işaretçiler	`new`; çöp toplayıcı serbest bırakır
Kaynak temizliği	elle, `goto cleanup` kalıbı	RAII, yıkıcı	`using`, `IDisposable`, `Dispose`
Bellek sızıntısı riski	yüksek	akıllı işaretçilerle düşük	düşük (kaynak sızıntısı yine mümkün)
Sınır denetimi	yok	`[]` denetimsiz, `at()` denetimli	her zaman denetimli
Sarkan işaretçi	mümkün	mümkün	normal kodda mümkün değil

Özellik	C	C++	C#
İşaretçi aritmetiği	doğal	mümkün ama önerilmez	yalnızca `unsafe` bağlamında

Hata Yönetimi

Özellik	C	C++	C#
Ana mekanizma	dönüş kodu, `errno`	istisna (`try` / `catch` / `throw`)	istisna
Başlatılmamış değişken	okunabilir, değeri belirsiz	okunabilir, değeri belirsiz	okunamaz, derleme hatası
Tamsayı taşması	işaretli tipte tanımsız davranış	işaretli tipte tanımsız davranış	tanımlı: sarma ya da `checked` ile istisna
Sıfıra bölme	tanımsız davranış	tanımsız davranış	`DivideByZeroException`
Dizi sınırı aşımı	tanımsız davranış	`[]` ile tanımsız, `at()` ile istisna	`IndexOutOfRangeException`
Garanti edilen temizlik	yok	yıkıcılar (yığın çözülürken)	`finally` , `using`

Derleme ve Çalıştırma Modeli

Özellik	C	C++	C#
Çıktı	doğrudan makine kodu	doğrudan makine kodu	IL (ara dil) + JIT ya da AOT
Çalışma zamanı	yok (küçük C çalışma kitaplığı)	küçük	CLR ve temel sınıf kitaplığı
Derleme süresi	kısa	uzun (şablonlar, başlıklar)	kısa
Bağlama	statik/dinamik, bağlayıcı (linker)	statik/dinamik, bağlayıcı	derleme birimleri `.dll` , çalışma zamanı yükler
Derleyiciler	gcc, clang, MSVC	gcc, clang, MSVC	Roslyn (`dotnet`)
Paket yönetimi	standart yok (sistem paketleri, vcpkg…)	standart yok (vcpkg, Conan…)	NuGet

Standart Kütüphane

Alan	C	C++	C# (.NET)
Dinamik dizi	yok	`vector`	`List<T>`
Sözlük	yok	`map` , `unordered_map`	`Dictionary<K,V>`
Küme	yok	`set` , `unordered_set`	`HashSet<T>`
String	`char[]` + `<string.h>`	`std::string`	`string` , `StringBuilder`
Sıralama	`qsort`	`std::sort`	`Array.Sort` , `OrderBy`
Dosya	`FILE *` , `fopen`	`ifstream` , `ofstream`	`File` , `StreamReader`
Koleksiyon sorguları	yok	`<algorithm>` , aralıklar (C++20)	LINQ
Ağ, JSON, web	standartta yok	standartta yok	kütüphanede hazır

Alan	C	C++	C# (.NET)
Eşzamanlılık	C11 `<threads.h>` (sınırlı)	`std::thread`, `async`	`Task`, `async` / `await`

Nesne Yönelimi

Özellik	C	C++	C#
Sınıf	yok (`struct`)	var	var
Kapsülleme	elle (opak işaretçi)	`public` / `private` / `protected`	`public` / `private` / `protected` / `internal`
Kalıtım	yok	çoklu kalıtım	tek sınıf, çoklu arayüz
Çok biçimlilik	fonksiyon işaretçileri	`virtual`	`virtual` / `override` , arayüzler
Soyut tip	yok	saf sanal fonksiyon	`abstract` , `interface`
Yıkıcı	yok	belirli zamanda çalışır	sonlandırıcı belirsiz zamanda; `Dispose`
Operatör aşırı yükleme	yok	var	var (sınırlı)
Özellik (property)	yok	yok (get/set fonksiyonları)	var

Generic Programlama

Özellik	C	C++	C#
Mekanizma	`void *` ve makrolar	şablonlar (templates)	generics
Çözüm zamanı	yok	derleme zamanı, her tip için ayrı kod	çalışma zamanında tip bilgisi korunur
Kısıtlar	yok	kavramlar (concepts, C++20)	`where T : ...` kısıtları
Tip güvenliği	yok (`void *`)	derleme zamanında	derleme zamanında
Özelleştirme	yok	tam ve kısmi özelleştirme	yok (aşırı yükleme ile çözülür)
Hata mesajları	yok	uzun olabilir	kısa

Taşınabilirlik ve Performans

Konu	C	C++	C#
Kaynak taşınabilirliği	çok yüksek	yüksek	yüksek
Çalıştırılabilir dosya	her platform için ayrı	her platform için ayrı	IL taşınabilir; çalışma zamanı gerekir (ya da Native AOT)
Standart	ISO C (C17, C23)	ISO C++ (C++17, 20, 23)	ECMA-334 standardı; dil .NET ekibince geliştirilir
Başlangıç süresi	çok kısa	çok kısa	JIT nedeniyle daha uzun
Ham hız	çok yüksek	çok yüksek	yüksek (JIT en iyileştirmeleri yapar)
Bellek ayak izi	en düşük	düşük	çalışma zamanı ve çöp toplayıcı payı var

Konu	C	C++	C#
Kararlılık	tahmin edilebilir gecikme	tahmin edilebilir gecikme	çöp toplama duraksamaları olabilir
Düşük düzey erişim	tam	tam	sınırlı (`unsafe` , `Span<T>`)

◆ Kritik Nokta

"Hangi dil daha hızlı?" sorusunun tek bir yanıtı yoktur. C ve C++ programları genellikle daha öngörülebilir ve daha az bellekle çalışır; C# ise çoğu uygulamada yeterli hızı çok daha güvenli bir ortamda ve çok daha kısa sürede yazarak verir. Hız farkı çoğu zaman dilden çok **algoritmadan ve veri yapısından** gelir; O(n) aramayı O(1) aramaya çevirmek dil değiştirmekten daha fazla kazandırır.

Tarihçe · Aynı soyun üç dalı

C 1972'de Dennis Ritchie tarafından Bell Labs'ta geliştirildi. C++, Bjarne Stroustrup'un 1979'da başlayan "C with Classes" çalışmasından doğdu; adı 1983'te konuldu. C# ise Anders Hejlsberg liderliğindeki bir Microsoft ekibi tarafından tasarlandı, 2000'de duyuruldu ve 2002'de C# 1.0 ile .NET Framework 1.0 olarak yayımlandı. Üç dilin ortak köken izleri hâlâ görünür: `{}` blokları, `for (;;)` döngüsü ve `?:` operatörü üçünde de aynıdır.

61.7 Geçiş Tuzakları

Bir dilden diğerine geçerken en çok zarar veren şey, eski dilin alışkanlıklarını yeni dile **sessizce** taşımaktır. Aşağıdaki listeler derleyicinin hata verdiği ya da, daha kötüsü, hiç şikayet etmeden yanlış çalışan durumları toplar.

C'den C++'a geçerken

- **`malloc` dönüş değeri:** C'de `int *p = malloc(...)` geçerlidir. C++'ta `void *`'dan başka işaretçiye örtük dönüşüm yoktur, dönüşüm gerekir; zaten `new` ya da `vector` kullanmak gerekir.
- **`new` ile `delete`, `malloc` ile `free`:** İkisi karıştırılmaz. `new[]` ile ayrılan bellek `delete[]` ile bırakılır.
- **Karakter sabitinin tipi:** C'de `'a'` bir `int`'tir (`sizeof('a')` genellikle 4), C++'ta `char`'dır (1).
- **Boş parametre listesi:** C'de `void f()` parametreleri belirtilmemiş demektir (C23'ten önce), C++'ta tam olarak "parametre yok" demektir.
- **Değişken uzunluklu diziler (VLA):** C99'da `int a[n];` geçerlidir; C++ standardında yoktur, `vector` kullanılır.
- **Atanmış başlatıcılar** (`.x = 1`): C99'dan beri C'de var; C++'ta ancak C++20'den ve kısıtlı biçimde vardır.
- **Dizgi işlemleri:** `char` dizisi yerine `std::string` kullanılır; `==` ile karşılaştırma artık içeriği karşılaştırır.
- **Başlık adları:** `<stdio.h>` yerine `<cstdio>`; ama çoğu zaman `iostream` yeterlidir.

C++'tan C#'a geçerken

- **Sınıflar referans tipidir:** C++'ta `Ogrenci b = a;` kopya üretir. C#'ta `class` ise iki değişken aynı nesneyi gösterir; kopya için `struct` ya da elle çoğaltma gerekir (Bölüm 49, 52).
- **Yıkıcılar belirsiz zamanda çalışır:** Kaynakları bırakmanın RAII karşılığı `using` deyimi ve `IDisposable`'dır; sonlandırıcıya güvenilmez.

- `delete` **yoktur:** Belleği çöp toplayıcı yönetir. Ama dosya ve bağlantı gibi kaynaklar yine açıkça kapatılmalıdır.
- **Çoklu kalıtım yoktur:** Sınıf tek tabandan türer; ek davranışlar arayüzlerle gelir (Bölüm 51).
- **Şablon özelleştirmesi yoktur:** Generics, çalışma zamanında tip bilgisini taşır; her tip için ayrı kod üretmez.
- `const` **üye fonksiyon yoktur:** Bir metodun nesneyi değiştirmediği tipin sözdizimiyle belirtilemez.
- `switch` **içinde düşme yoktur:** Her `case` `break`, `return` vb. ile bitmelidir.
- **Koşul** `bool` **olmak zorundadır:** `if (x)` ya da `if (p)` yazılamaz; `if (x != 0)` ve `if (p != null)` gerekir.
- `string` **değişmezdir:** `s[0] = 'a'` atanamaz; değişiklik için `StringBuilder` kullanılır.

C#'tan C'ye dönerken

- **Sınır denetimi yoktur:** Taşan indeks bir istisna fırlatmaz, sessizce başka belleğe yazar.
- **Dizgi yoktur:** Metin, sıfırla biten `char` dizisidir; uzunluğu `strlen` hesaplar, sığacağını programcı denetler.
- **Dizi uzunluğu bilinmez:** Fonksiyona geçen diziden uzunluk çıkarılamaz (dizi işaretçiye dönüşür); uzunluk ayrıca iletilir.
- **İstisna yoktur:** Hata, dönüş değeri ya da `errno` ile bildirilir ve denetlenmesi programcıya kalır.
- **Çöp toplayıcı yoktur:** Her `malloc` bir `free` gerektirir.
- **Başlatılmamış yerel değişken okunabilir:** Derleyici engellemez; değer rastgele olabilir.
- `int` **boyutu sabit değildir:** Kesin genişlik gerekiyorsa `int32_t` gibi tipler kullanılır.
- **Koleksiyon yoktur:** Liste, sözlük ve küme kendi başımıza yazılır ya da dış kütüphaneden alınır.

> **▲ Sık Yapılan Hata · En sık yapılan üç hata**
>
> 1. C'de dizgileri `==` ile karşılaştırmak: adresleri karşılaştırır, içeriği değil. `strcmp` kullanılmalıdır.
> 2. C#'ta `class` nesnesini kopyaladığını sanmak: yalnızca ikinci bir referans oluşur.
> 3. C++'ta `vector`'ü `for (auto x : v)` ile dolaşıp elemanı değiştirmeyi denemek: `x` bir kopyadır, değişiklik `v`'ye yansımaz. `auto& x` yazılmalıdır.

61.8 Hangi Dilde Hangi Çözüm?

Üç vakadan sonra bir ölçü çıkarabiliriz. Veriyi okuyup birkaç sayı çıkaran küçük bir araç için C# ya da C++ en az kodla sonuca gider. Bellek ve zamanın doğrudan denetlendiği, ortamın kısıtlı olduğu bir iş için C doğal seçimdir. Çok sayıda nesnenin davranışını taşıyan, uzun ömürlü ve ekip çalışmasına dayalı bir uygulamada C# veya C++ nesne yönelimi ve zengin kütüphaneleriyle öne çıkar. Bu seçimin ayrıntılarını bir sonraki bölümde alanlara göre ele alacağız.

Bölüm Özeti

- Üç dil aynı soydan gelir; blok yapısı, operatörler ve kontrol deyimleri ortaktır, bu yüzden biri diğerinin kodunu okumayı kolaylaştırır.

- Aynı problem C'de veri ve fonksiyonlarla, C++'ta kapsayıcılar ve algoritmalarla, C#'ta özellikler, koleksiyonlar ve LINQ ile çözülür; kod uzunluğu aynı sırayla azalır.
- C'de dinamik dizi, sözlük ve sıralama ölçütü gibi parçaları programcı kurar; C++'ta bunlar `vector`, `map` ve `sort` olarak hazırdır; C#'ta `List`, `Dictionary` ve LINQ bunları karşılar.
- Bellek yönetimi C'de elle (`malloc` / `free`), C++'ta RAII ve akıllı işaretçilerle, C#'ta çöp toplayıcıyla yapılır.
- Hata yönetimi C'de dönüş kodlarıyla, C++ ve C#'ta istisnalarla yapılır; C# ayrıca dizi sınırını ve başlatılmamış değişkenleri denetler.
- C#'ta `int` her zaman 32 bittir; C ve C++'ta boyut platforma bağlıdır.
- C ve C++ doğrudan makine koduna derlenir; C# IL'e derlenir ve çalışma zamanı (CLR) tarafından çalıştırılır.
- C++ şablonları derleme zamanında her tip için kod üretir; C# generics çalışma zamanında tip bilgisini korur.
- C++ çoklu kalıtımı destekler; C# tek sınıf tabanı ve çoklu arayüz sunar.
- `class` C#'ta referans tipidir; C++'ta nesne değerdir ve atama kopya üretir.
- Hız farkının büyük kısmı dilden değil, algoritma ve veri yapısı seçiminden gelir.
- Dil değiştirirken en tehlikeli hatalar sessizce çalışan alışkanlıklardır: C'de `==` ile string karşılaştırma, C++'ta `auto x` ile kopya üzerinde değişiklik, C#'ta nesne kopyaladığını sanmak.

Alıştırmalar

1. Aşağıdakilerden hangisi için programcının açıkça `free` çağırması **gerekir**?

   A) C# `List<int>`
   B) C++ `vector<int>`
   C) C'de `malloc` ile ayrılmış blok
   D) C++ `std::string`
   E) C# `string`

2. C# dilinde `int` tipi hakkında hangisi doğrudur?

   A) Platforma göre 16, 32 ya da 64 bit olur.
   B) Her zaman 32 bittir.
   C) Her zaman 64 bittir.
   D) Derleyici seçeneğiyle değişir.
   E) `long` ile aynıdır.

3. Aşağıdaki C programının çıktısı nedir?

```c
int a = 7, b = 2;
printf("%d %.1f\n", a / b, (double)a / b);
```

4. Aşağıdaki C# parçasının çıktısı nedir?

```csharp
string s = new string(new[] { 'a', 'b' });
Console.WriteLine(s == "ab");
Console.WriteLine(ReferenceEquals(s, "ab"));
```

5. Aşağıdaki C kodunda mantık hatasını bulun ve düzeltin.

```c
char ad[] = "elma";
if (ad == "elma")
    printf("ayni\n");
```

6. Aşağıdaki kod C'de derlenir, C++'ta derlenmez. Neden? Düzeltmenin iki yolunu yazın.

```c
int *p = malloc(4 * sizeof(int));
```

7. `int x = 3;` iken `if (x = 5)` ifadesi hakkında hangisi doğrudur?
   - **A)** Üç dilde de derleme hatasıdır.
   - **B)** C ve C++'ta derlenir (uyarı verebilir), C#'ta derleme hatasıdır.
   - **C)** Yalnızca C'de derlenir.
   - **D)** C#'ta derlenir, C ve C++'ta hata verir.
   - **E)** Üç dilde de derlenir ve koşul yanlış olur.
8. C# `Dictionary<string, int>` koleksiyonuna sıra ve davranış bakımından en yakın C++ kapsayıcısı hangisidir?
   - **A)** `vector<int>`
   - **B)** `map<string, int>`
   - **C)** `unordered_map<string, int>`
   - **D)** `set<string>`
   - **E)** `list<int>`
9. Aşağıdaki C++ programının çıktısı nedir?

```cpp
vector<int> v = {1, 2, 3};
for (auto x : v) x *= 2;
for (auto x : v) cout << x << ' ';
```

10. C++'taki RAII fikrine C#'ta en yakın dil yapısı hangisidir?
    - **A)** `goto`
    - **B)** `using` deyimi ve `IDisposable`
    - **C)** `unsafe`
    - **D)** `checked`
    - **E)** `ref`
11. Üç dilde (C, C++, C#) ayrı ayrı: `{4, 9, 2, 7}` dizisinin en büyük elemanını bulup yazdıran birer program yazın.
12. Vaka 1'deki istatistik programının bir dilini seçin ve **açıklık** (en büyük ile en küçük arasındaki fark) değerini de yazdıracak biçimde genişletin.
13. Vaka 3'teki kelime sayma programını, `Elma` ve `elma` aynı kelime sayılacak biçimde (büyük/küçük harf duyarsız) değiştirin. İstediğiniz dili seçin.

BÖLÜM 62

Hangi Dil, Ne Zaman? Kullanım Alanları ve Yol Haritası

BU BÖLÜMDE

- C, C++ ve C#'ın güçlü ve zayıf yönlerini dengeli biçimde değerlendirmeyi
- Bir projede dil seçerken bakılacak ölçütleri
- Gömülü sistemler, işletim sistemleri, oyun, masaüstü, web, finans, bilimsel hesaplama ve mobil alanlarında hangi dilin öne çıktığını
- Sektörde bu dillerle yazılmış tanınmış yazılımları
- Bu kitaptan sonra hangi konulara ve hangi sırayla çalışılabileceğini

Bu kitapta üç dili tarih sırasıyla öğrendik. C ile belleği ve makineyi, C++ ile bu temelin üzerine kurulan soyutlama araçlarını, C# ile de çalışma zamanının sağladığı güvenliği ve verimliliği gördük. Geriye pratik bir soru kalıyor: Elimizde bir iş olduğunda hangi dili seçmeliyiz?

Bu sorunun tek ve evrensel bir cevabı yoktur. Dil seçimi; performans gereksinimine, çalışılacak platforma, ekibin bilgisine, mevcut kod tabanına ve bakım maliyetine birlikte bakılarak verilir. Bu bölümde her dilin güçlü ve zayıf yanlarını sıralayacak, kullanım alanlarını gözden geçirecek ve kitabı bitirenler için bir yol haritası çizeceğiz.

62.1 Ortak Miras, Farklı Hedefler

Üç dilin sözdizimi benzer: süslü parantezli bloklar, `for` ve `while` döngüleri, `if` ve `switch` deyimleri, aynı aritmetik ve mantıksal operatörler. Bu benzerlik tesadüf değildir; C++ C'nin üzerine, C# ise C ve C++ geleneğinin üzerine kurulmuştur. Bir dili bilen biri ötekini okuyabilir.

Buna karşılık dillerin tasarım hedefleri farklıdır:

- **C**, küçük ve sade bir dil olarak sistem programlamak için tasarlandı. Programcıya makineye yakın bir görünüm sunar ve ona güvenir.
- **C++**, C'nin makineye yakınlığını koruyup üzerine sınıfları, şablonları ve sıfır maliyetli soyutlama (zero-overhead abstraction) anlayışını ekledi.
- **C#**, yönetilen bir çalışma zamanı (CLR) üzerinde çalışan, çöp toplayıcılı, geniş bir standart kütüphaneyle gelen ve üretkenliği öne çıkaran bir dil olarak tasarlandı.

Bu hedef farkı, aşağıdaki karşılaştırmaların hemen hepsinin kökenindedir.

62.2 C'nin Güçlü ve Zayıf Yönleri

C'nin en büyük gücü **küçüklüğüdür**. Dilin tamamı kısa sürede öğrenilebilir; hangi satırın makinede hangi işlemlere karşılık geldiği büyük oranda tahmin edilebilir. Çalışma zamanı sistemi neredeyse yoktur: çöp toplayıcı, sanal makine ya da zorunlu bir kütüphane gerekmez. Bu yüzden C, küçük mikrodenetleyicilerden büyük sunuculara kadar neredeyse her platformda bir derleyiciye sahiptir.

İkinci güç **birlikte çalışabilirliktir** (interoperability). Hemen her dil, C işlevlerini çağırmak için bir mekanizma sunar; bir işletim sisteminin sistem çağrıları ve birçok kütüphanenin arayüzü C ile tanımlanır. C'nin işlev çağırma düzeni bu yüzden fiilen ortak bir dil gibi davranır. C# bu amaçla `P/Invoke`, C++ ise doğrudan `extern "C"` bildirimleri sunar.

Üçüncü güç **öngörülebilir performans ve bellek kullanımıdır**. Nesnelerin bellekte nasıl yerleştiğini, ne zaman ayrıldığını ve serbest bırakıldığını programcı belirler. Gizli bir bellek ayırma ya da beklenmedik bir duraklama yoktur.

Zayıf yönler de bu sade yapıdan doğar:

- **Bellek güvenliği programcıya aittir.** Dizi sınırı aşma, serbest bırakılmış belleği kullanma, ilk değeri verilmemiş değişkeni okuma gibi hatalar derleyici tarafından engellenmez; sonuç tanımsız davranıştır (Bölüm 10, 12 ve 13). Büyük kod tabanlarında bellek hatalarının ciddi bir güvenlik hatası sınıfı oluşturduğu bilinir.
- **Soyutlama araçları kısıtlıdır.** Sınıf, şablon, istisna, kapsayıcı yoktur. Veri yapıları elle yazılır; genel amaçlı kod için `void *` ve makrolara başvurulur, bu da tip denetimini zayıflatır (Bölüm 15 ve 19).
- **Standart kütüphane dardır.** Dizgi işlemleri, kapsayıcılar, ağ, eşzamanlılık ya da JSON gibi gündelik ihtiyaçlar ya elle yazılır ya da dış kütüphanelerle karşılanır.
- **Hata yönetimi elle yapılır.** Dönüş değerlerini her çağrıdan sonra denetlemek gerekir; unutulan bir denetim sessizce yanlış sonuca yol açar.

Özellik	C için değerlendirme
Öğrenme eğrisi	Dil küçük; ama doğru ve güvenli C yazmak deneyim ister
Performans	Çok yüksek, öngörülebilir
Bellek güvenliği	Yok; programcının sorumluluğunda
Taşınabilirlik	Çok yüksek; hemen her platformda derleyici var
Üretkenlik	Düşük; çoğu yapı elle kurulur
Çalışma zamanı yükü	Hemen hiç yok

62.3 C++'ın Güçlü ve Zayıf Yönleri

C++'ın ayırt edici gücü, **C düzeyinde denetimi yüksek düzeyli soyutlamalarla birleştirebilmesidir**. Bölüm 29'da gördüğümüz RAII sayesinde bellek, dosya ve kilit gibi kaynaklar nesnelerin ömrüne bağlanır; elle `free` çağırma ihtiyacı büyük oranda ortadan kalkar. Akıllı işaretçiler ve kapsayıcılar, güvenli ve hızlı kodu birlikte mümkün kılar.

Şablonlar (Bölüm 33) ve STL (Bölüm 35 ve 36), derleme zamanında özelleşen genel kodu sağlar. `std::vector<int>` ile elle yazılmış bir `int` dizisi arasında çoğu durumda performans farkı beklenmez; bu,

"kullanmadığın şey için ödeme yapmazsın" ilkesinin sonucudur. Taşıma semantiği, `constexpr` ve lambda ifadeleri (Bölüm 36 ve 39) modern sürümlerle birlikte bu gücü artırmıştır.

C++ ayrıca **C ile neredeyse tam uyumludur**: mevcut C kütüphaneleri doğrudan kullanılabilir. Eski kod tabanlarını bir anda yeniden yazmak yerine aşama aşama modernleştirmek mümkündür.

Zayıf yönleri de dikkate değerdir:

- **Dilin büyüklüğü ve karmaşıklığı.** Kopyalama, taşıma, şablon özelleştirme, aşırı yükleme çözümlemesi, çoklu kalıtım gibi konular birbirine bağlı kuralları olan geniş bir alan oluşturur. Dil, sürümler boyunca (C++11, 14, 17, 20, 23) büyümüştür; aynı işi yapmanın birden çok yolu bulunur.
- **Güvenli olmayan alt küme hâlâ ulaşılabilirdir.** Ham işaretçiler, sınırsız indeksleme ve `reinterpret_cast` kullanılabildiği için C'nin bellek hataları C++'ta da yapılabilir. Güvenlik, disiplinle ve modern pratiklerle sağlanır; dil bunu zorlamaz.
- **Derleme süreleri ve hata iletileri.** Şablon ağırlıklı kodda derleme uzun sürebilir, hata iletileri uzun ve okunması zor olabilir. Bu konuda sürümler arasında iyileşmeler olsa da sorun tümüyle ortadan kalkmış değildir.
- **Derleme ve paket yönetimi.** Bağımlılıkları ve derleme araçlarını yönetmek, C# ekosistemindeki hazır araçlara kıyasla daha dağınıktır; birden çok derleme sistemi ve paket yöneticisi kullanımdadır.

Özellik	C++ için değerlendirme
Öğrenme eğrisi	Dik; dil büyük ve katmanlı
Performans	Çok yüksek; soyutlamalar çoğunlukla ek maliyet getirmez
Bellek güvenliği	Kısmi; RAII ve akıllı işaretçilerle iyileşir, zorunlu değildir
Taşınabilirlik	Çok yüksek; derleyici bulunan her yerde
Üretkenlik	C'den yüksek, C#'tan düşük
Çalışma zamanı yükü	Çok düşük; sanal makine yok

62.4 C#'ın Güçlü ve Zayıf Yönleri

C#'ın en büyük avantajı **üretkenliktir**. Çöp toplayıcı bellek yönetimini üstlenir (Bölüm 41), dizi sınırları çalışma zamanında denetlenir, ilk değeri verilmemiş değişken okunamaz ve `null` ile ilgili hatalar nullable referans tipleriyle derleme aşamasında yakalanabilir (Bölüm 59). Aynı sınıf hatalarının C'de sessizce tanımsız davranışa dönüşmesine karşılık C#'ta çoğunlukla bir istisna oluşur ve hata kaynağı bellidir.

İkinci avantaj **zengin standart kütüphane ve araçlardır**. Koleksiyonlar, LINQ (Bölüm 56), `async`/`await` ile eşzamansız programlama (Bölüm 59), dosya, ağ ve JSON işlemleri, NuGet paket yöneticisi, `dotnet` komut satırı aracı ve güçlü geliştirme ortamları hazır gelir. Aynı dil ve aynı kütüphaneler masaüstü, web, bulut ve oyun gibi çok farklı alanlarda kullanılır.

Üçüncü avantaj **dilin tutarlı ve modern evrimidir**. Desen eşleme, `record` tipleri, ifade gövdeli üyeler ve top-level statements gibi özellikler gündelik kodu kısaltır (Bölüm 44 ve 52).

Zayıf yönleri şunlardır:

- **Çalışma zamanına bağımlılık.** Program, .NET çalışma zamanı üzerinde çalışır; başlangıç süresi ve bellek kullanımı C ve C++ programlarına göre genellikle daha yüksektir. Ön derleme (Native AOT) gibi seçenekler bu farkı azaltsa da her senaryoda geçerli değildir.

- **Bellek ve zamanlama üzerinde sınırlı denetim.** Çöp toplayıcı, belirsiz anlarda kısa duraklamalara yol açabilir. Çok sıkı gecikme gereksinimi olan sistemlerde bu bir sorun olabilir. `struct`, `Span<T>` ve `stackalloc` gibi araçlar bu denetimi artırır ama C düzeyine ulaşmaz.
- **Donanıma uzaklık.** Mikrodenetleyici gibi kısıtlı ortamlarda kullanımı sınırlıdır; sürücü ve çekirdek yazımı için uygun bir seçim değildir.
- **Platform ve ekosistem bağlılığı.** Dil platformlar arasıdır, ancak ekosistemin ağırlık merkezi .NET ve Microsoft araçlarıdır. Bazı kütüphaneler ve arayüz çerçeveleri yalnızca belirli platformlarda bulunur.

Özellik	C# için değerlendirme
Öğrenme eğrisi	Orta; C++'tan belirgin biçimde yumuşak
Performans	İyi; çoğu uygulama için yeterli, en uç durumlarda C ve C++'ın gerisinde
Bellek güvenliği	Yüksek; yönetilen kod varsayılandır
Taşınabilirlik	Yüksek; .NET'in desteklediği platformlarda
Üretkenlik	Yüksek
Çalışma zamanı yükü	Orta; CLR ve çöp toplayıcı gerekir

◆ Kritik Nokta

Bu tablolardaki "yüksek" ya da "düşük" ifadeleri mutlak değil, göreceli ölçülerdir. İyi yazılmış bir C# programı, kötü yazılmış bir C++ programından hızlı olabilir. Gerçek kararlar, tahmine değil ölçüme dayanmalıdır.

Üç Dilde · Aynı Güvenlik Hatası, Üç Dil *C · C++ · C#*

Bir dizinin sınırı dışına yazmak üç dilde farklı sonuç verir.

```c
int a[3];
a[5] = 1;      // C: tanımsız davranış, derleyici denetlemez
```

```cpp
std::vector<int> a(3);
a[5] = 1;      // C++: operator[] denetlemez, tanımsız davranış
a.at(5) = 1;   // C++: std::out_of_range istisnası fırlatır
```

```csharp
int[] a = new int[3];
a[5] = 1;      // C#: IndexOutOfRangeException
```

C'de güvenlik programcıdadır, C++'ta seçeneklidir, C#'ta varsayılandır.

62.5 Alanlara Göre Dil Seçimi

Aşağıdaki başlıklar, her alanda hangi dilin neden öne çıktığını özetler. Bunlar kural değil, yaygın eğilimlerdir; her alanda istisnalar ve başka diller (Rust, Go, Java, Python, Kotlin, Swift gibi) de vardır.

Gömülü Sistemler

Mikrodenetleyiciler çok az bellek ve işlem gücüne sahiptir; çoğu zaman işletim sistemi de yoktur. Bu ortamda **C** fiilî standarttır: derleyicisi hemen her yonga için bulunur, üretilen kod küçük ve öngörülebilirdir, donanım

yazmaçlarına doğrudan erişim kolaydır (Bölüm 16'daki bit işlemleri burada gündelik iştir). Yonga üreticilerinin geliştirme kitleri ve gerçek zamanlı işletim sistemleri genellikle C arayüzü sunar.

C++ da gömülü alanda yaygındır; özellikle istisnaları ve çalışma zamanı tür bilgisini kapatıp RAII, şablonlar ve `constexpr` gibi sıfır maliyetli özellikleri kullanan ekipler tercih eder. Popüler Arduino ortamında yazılan taslaklar da C++ ile derlenir. Güvenlik açısından kritik alanlarda (otomotiv, havacılık) MISRA C ve MISRA C++ gibi kodlama kuralları kümeleri kullanılır.

C# bu alanın ana akımı değildir. .NET nanoFramework gibi küçük cihazlar için geliştirilmiş projeler vardır; ancak sıkı bellek ve gecikme sınırı olan işlerde C ve C++ belirgin biçimde baskındır.

İşletim Sistemleri, Sürücüler ve Sistem Yazılımı

C, bu alanın dilidir. Unix işletim sisteminin 1973'te C ile yeniden yazılması dilin doğuşunun asıl nedenidir (Bölüm 2). Linux çekirdeği ağırlıklı olarak C ile yazılmıştır; Linux 6.1 sürümünden itibaren çekirdeğe Rust ile sürücü yazma desteği de eklenmiştir. Windows çekirdeği ve sürücüleri ağırlıklı olarak C ve C++ ile yazılır. Veritabanı sunucuları, ağ sunucuları ve sürüm kontrol sistemleri gibi altyapı yazılımlarında da C yaygındır; Git, SQLite, Redis, PostgreSQL ve Python'un standart gerçeklemesi CPython bunlara örnektir.

Çekirdek içinde C++'ın istisna ve standart kütüphane gibi özellikleri genellikle kullanılmaz; ama işletim sistemi üzerinde çalışan sistem düzeyinde uygulamalar (derleyiciler, tarayıcılar, veritabanları) sıkça C++ ile yazılır. C# işletim sistemi çekirdeği yazmak için uygun değildir; ancak sistem yönetimi araçları ve hizmetleri için kullanılabilir.

Oyun Geliştirme

Oyunlar, her karede (frame) kısıtlı bir sürede çok iş yapar; bu yüzden performans ve bellek denetimi önemlidir. **C++**, bu alanda yaygın biçimde kullanılır: Unreal Engine C++ ile yazılmıştır ve oyun kodu da C++ ile geliştirilir; birçok şirketin kendi oyun motoru da C++ tabanlıdır. Grafik ve fizik gibi sıcak yolların donanıma yakın yazılması gerektiği için bu tercih doğaldır.

C#, Unity oyun motorunun betik dilidir. Unity ile geliştirilen çok sayıda mobil, masaüstü ve bağımsız (indie) oyun C# ile yazılmıştır. Godot motoru da C# desteği sunar. Burada C#'ın üretkenliği, hızlı yineleme ve geniş bir eğitim kaynağı avantajı sağlar; oyun mantığı C# ile, motorun ağır kısmı ise C++ ile yazılır. Bu iş bölümü sık görülür.

C, oyun alanında doğrudan ürün geliştirmeden çok, düşük düzeyli kütüphanelerde (grafik, ses, pencere yönetimi) görülür; SDL gibi bazı yaygın kütüphanelerin arayüzü C'dir.

Masaüstü Uygulamaları

Windows'ta masaüstü uygulaması için **C#** ana seçeneklerden biridir: Windows Forms, WPF ve WinUI gibi arayüz çerçeveleri C# ile kullanılır. Hızlı geliştirme, görsel tasarımcılar ve zengin kütüphane bu alanda belirleyicidir. .NET MAUI ile birden çok platformu hedefleyen uygulamalar da yazılabilir.

C++, performansın ve eski kod tabanlarının önemli olduğu masaüstü yazılımında kullanılır. Qt gibi çapraz platform arayüz kütüphaneleri C++ ile yazılmıştır ve birçok masaüstü uygulaması bu kütüphaneyi kullanır. Web tarayıcıları, ofis uygulamaları ve ses veya görüntü işleme yazılımları sık görülen C++ örnekleridir; Chromium tarayıcı motoru bunlardan biridir. **C** ile masaüstü uygulaması yazmak mümkündür (GTK gibi kütüphaneler C arayüzlüdür), ancak günümüzde nadirdir.

Web Arka Uç ve Bulut Hizmetleri

Bu alanda **C#**, ASP.NET Core ile güçlü bir konumdadır: web uygulamaları, REST arayüzleri ve bulut hizmetleri yazmak için olgun bir çerçeve, `async`/`await` desteği ve Entity Framework Core gibi veri erişim araçları sunar. Kurumsal uygulamalarda yaygın olarak kullanılır.

C++ ile web arka ucu yazılabilir ve çok yüksek verim gereken hizmetlerde (örneğin yük dengeleyiciler, önbellekler, veritabanı motorları) tercih edilir; ama gündelik iş uygulamaları için üretkenlik maliyeti yüksektir. **C**, web sunucularının çekirdeğinde (nginx ve Apache HTTP Server gibi) kullanılır; ancak uygulama mantığını C ile yazmak nadirdir. Bu alanda Java, Go, Python ve JavaScript (Node.js) gibi diller de güçlü rakiplerdir.

Finans ve Bankacılık

Finans alanında iki ayrı ihtiyaç vardır. **Düşük gecikme** gereken alım satım sistemlerinde ve piyasa verisi işleyen yazılımlarda C++ yaygındır; çünkü beklenmedik duraklamalar olmaması ve bellek yerleşiminin denetlenebilmesi önemlidir. Kurumsal **iş uygulamalarında** (hesap yönetimi, raporlama, risk uygulamaları) ise C# ile .NET sıkça kullanılır. C#'ın `decimal` tipi (Bölüm 42), parasal hesaplarda ikilik kayan noktanın yuvarlama sorunlarından kaçınmayı kolaylaştırır. C bu alanda genellikle arka plandaki altyapı kütüphanelerinde bulunur.

Bilimsel Hesaplama ve Veri Bilimi

Bu alanda Python gibi yüksek düzeyli diller kullanıcı yüzüdür; ancak ağır hesap yapan kütüphaneler genellikle C, C++ ya da Fortran ile yazılmıştır. NumPy'ın çekirdeği C ile yazılmıştır; Eigen gibi doğrusal cebir kütüphaneleri ve CERN'de kullanılan ROOT çerçevesi C++ ile geliştirilmiştir. Yani **C ve C++** bilimsel yığının altında, sıcak yollarda çalışır. Şablonlar sayesinde C++ hem hızlı hem de genel nümerik kütüphaneler yazmaya elverişlidir.

C#, bu alanda sınırlı bir role sahiptir; veri analizi ve makine öğrenmesi ekosistemi Python çevresinde toplanmıştır. Yine de .NET üzerinde veri işleme ve ML.NET gibi kütüphaneler bulunur; kurumsal ortamda mevcut .NET sistemlerine analiz eklemek için kullanılabilir.

Mobil Uygulamalar

Mobil dünyada ana diller platformlara göre belirlenir: Android için Kotlin ve Java, iOS için Swift. Bu dillerde yazılan uygulamaların yanında **C# ile** .NET MAUI (ve onun öncülü Xamarin) kullanılarak birden çok platforma tek kod tabanından uygulama geliştirilebilir. Unity ile yapılan mobil oyunlar da C# kullanır.

C ve C++, mobilde arka planda performans gereken bölümler için kullanılır. Android'in yerel geliştirme kiti (NDK) C ve C++ kodunu uygulamaya dahil etmeyi sağlar; oyun motorları, görüntü ve ses işleme kütüphaneleri bu yolla mobil cihazlara taşınır. Platform arası ortak çekirdek yazmak isteyen ekipler bu kodu C++ ile bir kez yazıp iki platformda da kullanabilir.

Özet Tablo

Alan	Birinci tercih	Sık görülen diğerleri
Gömülü sistemler	C	C++
İşletim sistemi, sürücü	C	C++ (Windows tarafında), Rust (yeni)
Oyun motoru	C++	C
Oyun mantığı (Unity, Godot)	C#	C++

Alan	Birinci tercih	Sık görülen diğerleri
Windows masaüstü	C#	C++
Çapraz platform masaüstü	C++ (Qt)	C# (MAUI)
Web arka uç	C#	Java, Go, Python, JavaScript
Düşük gecikmeli finans	C++	C
Kurumsal finans uygulamaları	C#	Java
Bilimsel hesaplama çekirdeği	C++ ve C	Fortran
Mobil	Platform dili	C# (MAUI), C++ (ortak çekirdek)

▲ Sık Yapılan Hata · Dil seçiminde sık yapılan hatalar

- **Yalnızca "hız" için C++ seçmek.** Çoğu uygulamanın darboğazı disk, ağ ya da veritabanıdır; dilin hızı belirleyici olmaz. Önce ölçün, sonra karar verin.
- **Yalnızca "kolaylık" için C# seçip gecikme gereksinimini görmezden gelmek.** Çöp toplayıcı duraklamaları sıkı zaman kısıtlarında sorun yaratabilir.
- **Mevcut kod tabanını yok saymak.** Ekibin bildiği ve üzerinde yıllardır çalışılan bir dil, daha "modern" bir dile geçmenin maliyetinden çoğu zaman daha değerlidir.
- **Tek bir dili her iş için kullanmaya çalışmak.** Gerçek sistemler genellikle çok dillidir: C# arayüz ve iş mantığı, C++ hesap yoğun çekirdek, C ise işletim sistemi ile arayüz katmanıdır.

62.6 Dil Seçimi İçin Bir Karar Rehberi

Bir proje için dil seçerken aşağıdaki soruları sırasıyla sormak yararlıdır:

1. **Platform nedir?** Mikrodenetleyici ya da çekirdek içi kod ise C (belki C++). Sunucu ya da masaüstü ise üç dil de adaydır.
2. **Gecikme ve bellek kısıtı ne kadar sıkı?** Mikrosaniyelerle ölçülen gecikme ya da çok kısıtlı bellek varsa C ve C++; milisaniyeler ve yeterli bellek yeterliyse C# da uygundur.
3. **Güvenlik ve hata maliyeti nedir?** Dışarıdan gelen veriyi işleyen, ağa açık bir hizmette bellek güvenliği önceliktir; yönetilen bir dil bu açıdan avantaj sağlar.
4. **Ekip neyi biliyor?** Ekip bilgisi, geliştirme hızını ve hata sayısını doğrudan etkiler.
5. **Mevcut sistemlerle nasıl bütünleşecek?** Eldeki kütüphaneler, arayüzler ve dağıtım ortamı seçimi daraltır.
6. **Bakım ömrü ne kadar?** Uzun yıllar yaşayacak bir kod tabanında okunabilirlik ve araç desteği, ilk sürümün hızından daha çok önem taşır.

Not · Diller birbirini dışlamaz

Bu üç dilin hepsini bilmek, tek bir projede de işe yarar. C# uygulamaları yerel C ve C++ kütüphanelerini çağırabilir, C++ kütüphaneleri C arayüzü üzerinden başka dillere açılabilir. Bu kitabın üç dili birlikte öğretmesinin bir nedeni de budur: arayüzlerin nerede ve nasıl birleştiğini görmek.

62.7 Sektörde Kullanım Örnekleri

Aşağıdaki tablo, herkesçe bilinen ve genel kabul gören örnekleri bir araya getirir. Bir ürünün tamamının tek dille yazıldığı nadirdir; tabloda her ürünün baskın ya da tanınmış bileşenleri verilmiştir.

Yazılım	Ağırlıklı dil	Not
Linux çekirdeği	C	Sürücü yazımı için Rust desteği de bulunur
Git	C	Sürüm kontrol sistemi
SQLite	C	Birçok uygulamaya gömülü veritabanı motoru
CPython	C	Python dilinin standart gerçeklemesi
Redis, PostgreSQL	C	Veritabanı ve veri deposu sunucuları
nginx, Apache HTTP Server	C	Web sunucuları
LLVM/Clang	C++	Derleyici altyapısı
Chromium	C++	Web tarayıcı motoru
Unreal Engine	C++	Oyun motoru
Qt	C++	Çapraz platform arayüz kütüphanesi
MySQL, MongoDB	C++	Veritabanı sunucuları
Unity (oyun betikleri)	C#	Motorun çekirdeği C++, kullanıcı kodu C#
ASP.NET Core uygulamaları	C#	Web ve bulut hizmetleri
.NET sınıf kütüphanesi ve derleyicisi (Roslyn)	C#	C# derleyicisi büyük oranda C# ile yazılmıştır

> *Tarihçe · Kendini derleyen diller*
>
> C derleyicileri tarihsel olarak C ile yazılmıştır; C# derleyicisi Roslyn de büyük ölçüde C# ile yazılmıştır. Bir dilin kendi derleyicisini kendi diliyle yazabilmesi, dilin olgunluğunun yaygın bir göstergesi sayılır. GCC projesi ise uzun yıllar C ile geliştirildikten sonra kod tabanında C++ kullanımına geçmiştir.

Bu örneklerden iki sonuç çıkarabiliriz. Birincisi, **altyapı ne kadar aşağıdaysa C ve C++ o kadar baskındır**: çekirdekler, veritabanları, derleyiciler ve tarayıcılar. İkincisi, **uygulama ne kadar yukarıdaysa C# gibi yönetilen diller o kadar çok görülür**: iş uygulamaları, web hizmetleri ve oyun mantığı.

> **Üç Dilde · Aynı İş, Üç Dilde: Bir Dosyadaki Satırları Saymak** *C · C++ · C#*
>
> Her dilin kendi alışkanlıkları vardır; aşağıdaki kısa kesitler aynı işin üç dilde ne kadar farklı bir çabayla yazıldığını gösterir.

```c
FILE *f = fopen("veri.txt", "r");
int satir = 0, c;
while ((c = fgetc(f)) != EOF)
    if (c == '\n') satir++;
fclose(f);              // açma, denetim ve kapatma elle yapılır
```

```cpp
std::ifstream f("veri.txt");
std::string s;
int satir = 0;
while (std::getline(f, s)) satir++;   // kapatma RAII ile otomatik
```

```csharp
int satir = File.ReadLines("veri.txt").Count();
```

Satır sayısı azaldıkça denetim de azalır: C'de her ayrıntı programcıdadır, C#'ta kütüphane ayrıntıların çoğunu üstlenir. Hangisinin doğru olduğu, işin neye ihtiyaç duyduğuna bağlıdır.

62.8 Bu Kitaptan Sonra: Genel Yol Haritası

Üç dilin temellerini bitirmiş olmak, programcılığın başlangıcıdır. Bundan sonrası, dilden bağımsız konularda derinleşmek ve seçilen alanda uygulama yapmaktır. Aşağıdaki başlıklar kabaca önem sırasına göre verilmiştir.

Veri Yapıları ve Algoritmalar

Bu kitapta bağlı liste, yığın ve kuyruk gibi yapıları gördük (Bölüm 19); özyinelemeyi üç dilde işledik (Bölüm 18, 38 ve 58). Sıradaki adım; ağaçlar, ikili arama ağaçları, öbekler (heap), karma tablolar (hash table), çizgeler (graph), sıralama ve arama algoritmaları ile bunların zaman ve bellek karmaşıklığını incelemektir. Bu konular dilden bağımsızdır ve üç dilde de aynı fikirlerle uygulanır. Bir algoritmayı önce C'de elle yazıp, ardından C++ STL ve C# koleksiyonlarındaki karşılığıyla kıyaslamak çok öğreticidir.

İşletim Sistemleri

Süreç (process), iş parçacığı (thread), bellek yönetimi, sanal bellek, dosya sistemleri ve zamanlayıcılar, yazdığımız programların altında neler olduğunu anlatır. Özellikle C ve C++ ile çalışanlar için bu bilgi, işaretçi ve bellek davranışlarını daha iyi anlamayı sağlar. Linux üzerinde sistem çağrıları ile küçük araçlar yazmak iyi bir alıştırmadır.

Bilgisayar Ağları

Ağ programlama, TCP/IP, soket (socket) kavramı, istemci-sunucu modeli ve HTTP gibi protokoller günümüz yazılımlarının temelidir. C'de soket arayüzüyle düşük düzeyde, C++ ve C# ile ise daha yüksek düzeyli kütüphanelerle basit bir sohbet ya da dosya paylaşım programı yazmak iyi bir uygulamadır.

Veritabanları

İlişkisel veritabanı kavramı, SQL dili, tablo tasarımı, dizinler ve işlemler (transaction) çoğu uygulamanın veri katmanını oluşturur. SQLite, kurulum gerektirmediği için başlangıç için uygundur ve C, C++ ve C# içinden kullanılabilir. C#'ta Entity Framework Core ile nesneler ve tablolar arasındaki eşleme de öğrenilebilir.

Eşzamanlılık ve Paralel Programlama

Çok çekirdekli işlemciler yaygın olduğundan iş parçacıkları, kilitler, yarış koşulu (race condition) ve kilitlenme (deadlock) kavramlarını bilmek gerekir. C++'ta `std::thread` ve ilgili kütüphane, C#'ta `Task` ve `async`/`await` bu konunun araçlarıdır.

Sürüm Kontrolü: Git

Küçük bile olsa her proje için sürüm kontrolü kullanmak iyi bir alışkanlıktır. Git ile `commit`, dal (branch), birleştirme (merge) ve uzak depo kavramları öğrenilmelidir. Kodunuzun geçmişini tutmak, hatalı değişiklikleri geri almak ve başkalarıyla çalışmak için gereklidir.

Test Yazma ve Hata Ayıklama

Yazılan kodun doğruluğunu otomatik olarak sınamak profesyonel yazılımın temelidir. Birim testi (unit test) kavramı ve her dilin çerçevesi (C++ için Catch2 ve GoogleTest, C# için xUnit, NUnit ve MSTest gibi) öğrenilmelidir. Bunun yanında hata ayıklayıcıyı (debugger) kullanmayı, bellek hatası bulan araçları (C ve C++ için Valgrind ve AddressSanitizer) ve profil çıkarıcıları öğrenmek zaman kazandırır.

> **İpucu · Küçük ama gerçek projeler**
>
> Öğrenmeyi pekiştirmenin en etkili yolu, kendinize gerçek bir sorun seçip onu çözmektir: bir komut satırı yapılacaklar listesi, bir metin dosyası analiz aracı, basit bir oyun ya da bir kişisel bütçe takipçisi. Projeyi küçük tutun, sürüm kontrolüne alın ve önce çalışan bir sürüm çıkarın, sonra geliştirin.

62.9 Her Dil İçin Sonraki Adımlar

C için

- **Sistem programlama:** dosya tanıtıcıları, süreçler, borular (pipe) ve sinyaller; Linux'ta `man` sayfalarını okuma alışkanlığı.
- **Bellek hatalarını yakalamak:** Valgrind ve AddressSanitizer ile kendi programlarınızı sınayın.
- **Gömülü geliştirme:** düşük maliyetli bir geliştirme kartı ile LED yakmaktan başlayıp sensör okumaya ilerleyin.
- **Açık kaynak kod okumak:** SQLite ya da küçük bir Unix aracının kaynağını okumak, iyi C yazımının en iyi öğretmenidir.
- **C standardı:** C99 ve C11 ile gelen özellikleri (`stdint.h`, `_Bool`, `_Static_assert`) ve C23 ile gelen yenilikleri takip edin.

C++ için

- **Modern C++ pratikleri:** C++ Core Guidelines belgesini okuyun; ham `new` ve `delete` yerine akıllı işaretçi ve kapsayıcı kullanma alışkanlığı edinin.
- **Daha ileri konular:** C++20 ile gelen kavramlar (concepts), modüller ve aralıklar (ranges), `constexpr` ile derleme zamanı hesaplama, `std::optional`, `std::variant`.
- **Derleme sistemleri:** CMake ile çok dosyalı projeler ve bağımlılık yönetimi.
- **Kütüphaneler:** Qt ile arayüz, Boost ve Eigen gibi kütüphanelerle tanışma.
- **Grafik ve oyun:** SDL, SFML ya da bir grafik arayüzü ile küçük bir 2B oyun yazmak.

C# için

- **Web geliştirme:** ASP.NET Core ile küçük bir REST arayüzü yazın; veritabanı için Entity Framework Core kullanın.

- **Eşzamansız programlama:** `async` / `await` ve `Task` konusunda derinleşin (Bölüm 59'un devamı).
- **Test:** xUnit ile birim testi ve bağımlılık enjeksiyonu (dependency injection) kavramı.
- **Arayüz:** WPF ya da .NET MAUI ile masaüstü ya da mobil uygulama.
- **Oyun:** Unity ya da Godot ile C# betikleri yazarak oyun geliştirmeye başlayın.
- **Performans:** `Span<T>`, `struct` kullanımı ve çöp toplayıcının davranışını anlama.

Üçüncü Bir Adım: Başka Diller

Bu kitabın üç dili bir bakıma sistem programlamadan kurumsal yazılıma uzanan bir yelpazedir. Bundan sonra başka dillerle tanışmak da değerlidir. Python hızlı denemeler ve veri çalışmaları için, Java kurumsal dünya ve Android için, Rust bellek güvenliğini derleme aşamasında zorlayan bir sistem dili olarak, Go ise sade yapısı ve eşzamanlılık desteğiyle ağ hizmetleri için sık anılan seçeneklerdir. Üç dili iyi bilen biri için bunlara geçmek zor değildir; çünkü temel kavramlar (değişken, tip, akış denetimi, işlev, bellek, soyutlama) aynıdır.

Bölüm Özeti

- Dil seçimi tek bir doğru cevabı olmayan, performans, platform, ekip bilgisi ve bakım maliyetine bağlı bir mühendislik kararıdır.
- C küçüklüğü, taşınabilirliği ve öngörülebilirliği sayesinde gömülü sistemlerin ve işletim sistemi çekirdeklerinin dilidir; bellek güvenliği ve soyutlama eksikliği başlıca zayıf yanlarıdır.
- C++ C düzeyinde denetimi RAII, şablonlar ve STL gibi soyutlamalarla birleştirir; karmaşıklığı ve güvenli olmayan alt kümesi ise başlıca zorluklarıdır.
- C# yönetilen çalışma zamanı, çöp toplayıcı ve zengin kütüphanesi sayesinde yüksek üretkenlik ve güvenlik sağlar; çalışma zamanı yükü ve sınırlı düşük düzey denetim bedelidir.
- Gömülü sistem ve çekirdek geliştirmede C, oyun motoru ve düşük gecikmeli sistemlerde C++, web arka ucu ve kurumsal uygulamalarda C# öne çıkar.
- Unity betikleri C# ile, Unreal Engine ve birçok oyun motoru C++ ile, Linux çekirdeği, Git ve SQLite C ile yazılmıştır.
- Gerçek sistemler çoğunlukla çok dillidir; diller yerel kütüphane arayüzleriyle birbirine bağlanır.
- Kararlar tahmine değil ölçüme dayanmalı; mevcut kod tabanı ve ekip bilgisi dikkate alınmalıdır.
- Sonraki adımlar: veri yapıları ve algoritmalar, işletim sistemleri, ağlar, veritabanları, eşzamanlılık, Git ve test yazma.
- Dili öğrenmenin en iyi yolu, küçük ama gerçek projeler yazıp sürüm kontrolüyle geliştirmektir.

62.10 Kapanış

Bu kitap, 1972'de küçük bir sistem dili olarak doğan C'den başlayıp C++ ve C# üzerinden, aralarında kurulan bağları izleyerek ilerledi. Bir dilin neden o biçimde tasarlandığını bilmek, onu daha iyi kullanmayı sağlar; çünkü her özelliğin bir önceki dilin bir sorununa verilmiş cevap olduğunu görürüz.

Öğrendiklerimiz bir başlangıç noktasıdır. Dillerin ayrıntıları ve araçları zamanla değişecek, yeni sürümler gelecektir. Değişmeyecek olan, bellek, tip, akış denetimi, soyutlama ve hata yönetimi gibi temel fikirlerdir. Bu fikirleri sağlam kuran okur, yeni bir dile ya da yeni bir sürüme kısa sürede uyum sağlar. Sonraki adım, küçük bir proje seçip yazmaya başlamaktır.

EKLER

Ekler

Başvuru tabloları ve sözlük

EK A

ASCII Tablosu ve Sayı Sistemleri

Bu ekte iki başvuru malzemesi var: 0–127 aralığındaki ASCII karakter tablosu ve onluk, ikilik, sekizlik, onaltılık sayılar arasında çevirme tablosu. Kitabın metninde `'A'` ya da `0x1F` gibi ifadelerle karşılaştığımızda buraya dönebiliriz.

> **Tanım: ASCII**
>
> American Standard Code for Information Interchange. Her karaktere 0 ile 127 arasında bir sayısal kod veren 7 bitlik karakter kümesidir. Bilgisayar karakterleri sayı olarak saklar; `'A'` karakteri ile `65` sayısı aynı değeri ifade eder.

A.1 Sayı sistemleri kısaca

Bir sayı sisteminin tabanı, kullandığı rakam sayısıdır. Onluk sistemde 10 rakam (0–9), ikilik sistemde 2 rakam (0–1), sekizlik sistemde 8 rakam (0–7), onaltılık sistemde 16 rakam (0–9 ve A–F) vardır. Bir rakamın değeri, bulunduğu konuma göre tabanın kuvvetiyle çarpılarak bulunur.

Onluk `254` sayısını ele alalım: `2·100 + 5·10 + 4·1` eder. İkilik `101` ise `1·4 + 0·2 + 1·1`, yani onluk `5` eder.

A.2 Onluktan diğer tabanlara çevirme

Yöntem, sayıyı hedef tabana bölmek ve kalanları toplamaktır. Kalanlar son bölümden ilk bölüme doğru okunur.

Onluk `37` sayısını ikiliğe çevirelim:

Bölme	Bölüm	Kalan
37 ÷ 2	18	1
18 ÷ 2	9	0
9 ÷ 2	4	1
4 ÷ 2	2	0
2 ÷ 2	1	0
1 ÷ 2	0	1

Kalanları sondan başa okursak `100101` elde ederiz. Aynı yöntemle 8'e bölünce `37` onluk sayısı sekizlikte `45`, 16'ya bölünce onaltılıkta `25` olur.

A.3 Gruplama yoluyla çevirme

Bir sekizlik rakam tam olarak 3 ikilik bite, bir onaltılık rakam tam olarak 4 ikilik bite karşılık gelir. Bu yüzden ikilik sayıyı sağdan başlayarak 3'erli ya da 4'erli gruplara ayırıp her grubu tek rakamla yazabiliriz.

`100101` sayısını 3'erli gruplayalım: `100 101`, bu da sekizlikte `45` eder. 4'erli gruplarsak `0010 0101` olur ve onaltılıkta `25` eder. Bu iki yazım da yukarıdaki sonucu doğrular.

A.4 Önek notasyonu

C, C++ ve C#'ta sayı yazımının farklı biçimleri vardır:

- Onaltılık sayılar `0x` önekiyle yazılır: `0x1F` (C, C++, C#).
- Sekizlik sayılar başta sıfır ile yazılır: `017` (C ve C++). C#'ta sıfırla başlayan sekizlik yazım yoktur.
- İkilik sayılar `0b` önekiyle yazılır: `0b101`. C++14 ve C# 7.0 ile geldi; C23 ile de C'nin standart parçası oldu.

```
char harf = 'A';        // onluk 65, onaltılık 0x41
int onaltilik = 0x41;   // aynı değer
```

A.5 ASCII kontrol karakterleri (0–31 ve 127)

İlk 32 kod (0–31) ile 127 kodu ekrana basılmayan kontrol karakterleridir. Eski yazıcılar, teleks ve terminal donanımı için tasarlanmışlardır; bugün bazıları hâlâ kullanılır. Özellikle LF ve CR satır sonu, TAB sekme, ESC ise terminal kaçış dizileri için önemlidir.

Onluk	Onaltılık	Kısaltma	Açıklama
0	00	NUL	Boş karakter (null). Dizgilerin sonunu işaretler.
1	01	SOH	Başlık başlangıcı (start of heading).
2	02	STX	Metin başlangıcı (start of text).
3	03	ETX	Metin sonu (end of text).
4	04	EOT	İletim sonu (end of transmission).
5	05	ENQ	Sorgu (enquiry).
6	06	ACK	Onay (acknowledge).
7	07	BEL	Zil. Terminalde bip sesi çıkarır.
8	08	BS	Geri adım (backspace).
9	09	HT	Yatay sekme (horizontal tab).
10	0A	LF	Satır besleme (line feed). Unix'te satır sonu.
11	0B	VT	Dikey sekme (vertical tab).
12	0C	FF	Sayfa besleme (form feed).
13	0D	CR	Satır başı (carriage return). Windows'ta satır sonu CR LF çiftidir.
14	0E	SO	Dışarı kaydırma (shift out).
15	0F	SI	İçeri kaydırma (shift in).

Onluk	Onaltılık	Kısaltma	Açıklama
16	10	DLE	Veri bağlantısı kaçışı (data link escape).
17	11	DC1	Aygıt kontrolü 1. XON olarak kullanılır: akışı sürdürür.
18	12	DC2	Aygıt kontrolü 2.
19	13	DC3	Aygıt kontrolü 3. XOFF olarak kullanılır: akışı durdurur.
20	14	DC4	Aygıt kontrolü 4.
21	15	NAK	Olumsuz onay (negative acknowledge).
22	16	SYN	Eşzamanlama (synchronous idle).
23	17	ETB	İletim bloğu sonu (end of transmission block).
24	18	CAN	İptal (cancel).
25	19	EM	Ortam sonu (end of medium).
26	1A	SUB	Yerine koyma (substitute). Eski DOS dosyalarında dosya sonu işareti olarak kullanılırdı.
27	1B	ESC	Kaçış (escape). ANSI renk ve imleç komutları bu karakterle başlar.
28	1C	FS	Dosya ayırıcı (file separator).
29	1D	GS	Grup ayırıcı (group separator).
30	1E	RS	Kayıt ayırıcı (record separator).
31	1F	US	Birim ayırıcı (unit separator).

A.6 Kaçış dizileri

C, C++ ve C#'ta bazı kontrol karakterleri ters eğik çizgiyle başlayan kısa dizilerle yazılır. Bunlar karakter değişmezlerinde ve dizgilerde kullanılır.

Kaçış dizisi	Onluk	Anlamı
`\0`	0	Boş karakter (NUL)
`\a`	7	Zil (BEL)
`\b`	8	Geri adım (BS)
`\t`	9	Yatay sekme (HT)
`\n`	10	Satır besleme (LF)
`\v`	11	Dikey sekme (VT)
`\f`	12	Sayfa besleme (FF)
`\r`	13	Satır başı (CR)
`\\`	92	Ters eğik çizgi (`\`)
`\'`	39	Tek tırnak
`\"`	34	Çift tırnak

A.7 Boşluk ve noktalama (32–47)

Yazdırılabilir karakterler 32 ile 126 arasındadır. Bu aralığın ilk satırı boşluk karakteridir ve tırnak, parantez, operatör gibi sembolleri içerir.

Onluk	Onaltılık	Karakter	Açıklama
32	20	boşluk	Boşluk (space)
33	21	`!`	Ünlem işareti
34	22	`"`	Çift tırnak
35	23	`#`	Diyez işareti
36	24	`$`	Dolar işareti
37	25	`%`	Yüzde işareti
38	26	`&`	Ve işareti
39	27	`'`	Tek tırnak
40	28	`(`	Açılış parantezi
41	29	`)`	Kapanış parantezi
42	2A	`*`	Yıldız
43	2B	`+`	Artı
44	2C	`,`	Virgül
45	2D	`-`	Eksi işareti, kısa çizgi
46	2E	`.`	Nokta
47	2F	`/`	Eğik çizgi

A.8 Rakamlar ve ara semboller (48–64)

Rakamların kodları 48 (0) ile 57 (9) arasındadır. Rakam karakterinin değeri, sayısal değerden 48 fazladır: `'7'` karakteri 55'tir, 7 değildir.

Onluk	Onaltılık	Karakter	Açıklama
48	30	`0`	Rakam sıfır
49	31	`1`	Rakam bir
50	32	`2`	Rakam iki
51	33	`3`	Rakam üç
52	34	`4`	Rakam dört
53	35	`5`	Rakam beş
54	36	`6`	Rakam altı
55	37	`7`	Rakam yedi
56	38	`8`	Rakam sekiz
57	39	`9`	Rakam dokuz

Onluk	Onaltılık	Karakter	Açıklama
58	3A	`:`	İki nokta
59	3B	`;`	Noktalı virgül
60	3C	`<`	Küçüktür
61	3D	`=`	Eşittir
62	3E	`>`	Büyüktür
63	3F	`?`	Soru işareti
64	40	`@`	At işareti

A.9 Büyük harfler ve semboller (65–96)

Büyük harfler 65 (A) ile 90 (Z) arasındadır. Harfler alfabe sırasıyla ardışık kodlar alır; bu, `'C' - 'A'` gibi bir farkın 2 vermesini sağlar.

Onluk	Onaltılık	Karakter	Açıklama
65	41	`A`	Büyük harf A
66	42	`B`	Büyük harf B
67	43	`C`	Büyük harf C
68	44	`D`	Büyük harf D
69	45	`E`	Büyük harf E
70	46	`F`	Büyük harf F
71	47	`G`	Büyük harf G
72	48	`H`	Büyük harf H
73	49	`I`	Büyük harf I
74	4A	`J`	Büyük harf J
75	4B	`K`	Büyük harf K
76	4C	`L`	Büyük harf L
77	4D	`M`	Büyük harf M
78	4E	`N`	Büyük harf N
79	4F	`O`	Büyük harf O
80	50	`P`	Büyük harf P
81	51	`Q`	Büyük harf Q
82	52	`R`	Büyük harf R
83	53	`S`	Büyük harf S
84	54	`T`	Büyük harf T
85	55	`U`	Büyük harf U
86	56	`V`	Büyük harf V

Onluk	Onaltılık	Karakter	Açıklama
87	57	`W`	Büyük harf W
88	58	`X`	Büyük harf X
89	59	`Y`	Büyük harf Y
90	5A	`Z`	Büyük harf Z
91	5B	`[`	Köşeli açılış parantezi
92	5C	`\`	Ters eğik çizgi
93	5D	`]`	Köşeli kapanış parantezi
94	5E	`^`	Şapka işareti
95	5F	`_`	Alt çizgi
96	60	`` ` ``	Ters tırnak (geri tık)

A.10 Küçük harfler ve semboller (97–126)

Küçük harfler 97 (a) ile 122 (z) arasındadır. Büyük ve küçük harf arasındaki fark her zaman 32'dir, yani onaltılıkta `0x20`. Bu yüzden büyük harfi küçük harfe çevirmek için 5. bit (32'lik bit) açılır; `'a' - 'A'` ifadesi 32 eder.

Onluk	Onaltılık	Karakter	Açıklama
97	61	`a`	Küçük harf a
98	62	`b`	Küçük harf b
99	63	`c`	Küçük harf c
100	64	`d`	Küçük harf d
101	65	`e`	Küçük harf e
102	66	`f`	Küçük harf f
103	67	`g`	Küçük harf g
104	68	`h`	Küçük harf h
105	69	`i`	Küçük harf i
106	6A	`j`	Küçük harf j
107	6B	`k`	Küçük harf k
108	6C	`l`	Küçük harf l
109	6D	`m`	Küçük harf m
110	6E	`n`	Küçük harf n
111	6F	`o`	Küçük harf o
112	70	`p`	Küçük harf p
113	71	`q`	Küçük harf q
114	72	`r`	Küçük harf r

Onluk	Onaltılık	Karakter	Açıklama
115	73	`s`	Küçük harf s
116	74	`t`	Küçük harf t
117	75	`u`	Küçük harf u
118	76	`v`	Küçük harf v
119	77	`w`	Küçük harf w
120	78	`x`	Küçük harf x
121	79	`y`	Küçük harf y
122	7A	`z`	Küçük harf z
123	7B	`{`	Süslü açılış parantezi
124	7C	`|`	Dikey çizgi
125	7D	`}`	Süslü kapanış parantezi
126	7E	`~`	Tilde

A.11 Silme karakteri (127)

Son kod 127, silme karakteridir. Delikli kâğıt bantta bir kaydı silmek için o kaydın tüm deliklerinin açılması gerekirdi; bu da tüm bitlerin 1 olmasıyla eşdeğerdir. Bu nedenle DEL kodu 127 (`0x7F`) olarak tanımlanmıştır.

Onluk	Onaltılık	Kısaltma	Açıklama
127	7F	DEL	Silme (delete). Tüm bitleri 1 olan karakter.

A.12 Sayı sistemleri çevirme tablosu (0–32)

Aşağıdaki tablo onluk 0 ile 32 arasındaki sayıların ikilik, sekizlik ve onaltılık karşılıklarını verir. Onaltılık rakamlar 10 ile 15 için A–F harflerini kullanır.

Onluk	İkilik	Sekizlik	Onaltılık
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9

Onluk	İkilik	Sekizlik	Onaltılık
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12
19	10011	23	13
20	10100	24	14
21	10101	25	15
22	10110	26	16
23	10111	27	17
24	11000	30	18
25	11001	31	19
26	11010	32	1A
27	11011	33	1B
28	11100	34	1C
29	11101	35	1D
30	11110	36	1E
31	11111	37	1F
32	100000	40	20

EK B

Operatör Öncelik Tabloları

Bir ifadede birden fazla operatör bulunduğunda hangisinin önce uygulanacağını öncelik ve birleşme kuralları belirler. Bu ek C, C++ ve C# için ayrı tablolar veriyor, sonda da üç dil arasındaki farkları topluyor. C ve C++ tablolarında sıralama cppreference'ı, C# tablosunda sıralama Microsoft belgelerini esas alır.

> **Tanım: Öncelik ve birleşme**
>
> Öncelik, bir ifadede iki operatörden hangisinin önce işlenenlere bağlanacağını belirler. `2 + 3 * 4` ifadesinde `*` önce gelir ve sonuç 14'tür. Birleşme, aynı öncelikteki operatörlerin hangi yöne gruplanacağını gösterir: soldan sağa (`a - b - c` ifadesi `(a - b) - c`) ya da sağdan sola (`a = b = c` ifadesi `a = (b = c)`).

> **◆ Kritik Nokta**
>
> Öncelik yalnızca operatörlerin hangi işlenenlere bağlandığını belirler. Alt ifadelerin hangi sırayla hesaplanacağı (ör. `f() + g()` içinde hangisinin önce çağrılacağı) ayrı bir konudur ve bu tablolarda yer almaz.

B.1 C operatör öncelik tablosu

Aşağıdaki tabloda 1 en yüksek önceliği gösterir. Aynı satırdaki operatörler aynı önceliktedir. C'de 15 öncelik düzeyi vardır.

Öncelik	Operatörler	Açıklama	Birleşme
1	`x++` `x--` `f(...)` `a[i]` `x.y` `x->y` `(tip){liste}`	Sonek artırma ve azaltma, fonksiyon çağrısı, dizi indeksi, üye erişimi, bileşik değişmez	soldan sağa
2	`++x` `--x` `+x` `-x` `!x` `~x` `(tip)x` `*p` `&x` `sizeof` `_Alignof`	Önek artırma ve azaltma, tekli işaretler, tür dönüşümü, dolaylama, adres, boyut	sağdan sola
3	`*` `/` `%`	Çarpma, bölme, kalan	soldan sağa
4	`+` `-`	Toplama, çıkarma	soldan sağa
5	`<<` `>>`	Bit kaydırma	soldan sağa
6	`<` `<=` `>` `>=`	İlişkisel karşılaştırma	soldan sağa
7	`==` `!=`	Eşitlik karşılaştırması	soldan sağa
8	`&`	Bit düzeyinde VE	soldan sağa

Öncelik	Operatörler	Açıklama	Birleşme
9	`^`	Bit düzeyinde özel VEYA	soldan sağa
10	`|`	Bit düzeyinde VEYA	soldan sağa
11	`&&`	Mantıksal VE	soldan sağa
12	`||`	Mantıksal VEYA	soldan sağa
13	`?:`	Koşullu ifade	sağdan sola
14	`= += -= *= /= %= <<= >>= &= ^= |=`	Atama ve bileşik atama	sağdan sola
15	`,`	Virgül operatörü	soldan sağa

B.2 C++ operatör öncelik tablosu

C++, C'nin tablosuna `::`, `.*`, `->*` ve `<=>` gibi operatörler ekler; ayrıca `new`, `delete` ve `throw` ifadeleri ile C++ tür dönüşüm operatörleri (`static_cast` ve benzerleri) de bu sıralamaya girer. Tabloda `<=>` ve `co_await` C++20 ile gelen operatörlerdir.

Öncelik	Operatörler	Açıklama	Birleşme
1	`::`	Kapsam belirleme (scope resolution)	soldan sağa
2	`x++ x-- f(...) a[i] x.y x->y T(x) typeid dynamic_cast<> static_cast<> reinterpret_cast<> const_cast<>`	Sonek artırma ve azaltma, çağrı, indeks, üye erişimi, işlev tarzı dönüşüm, tür sorgusu, tür dönüşümleri	soldan sağa
3	`++x --x +x -x !x ~x (T)x *p &x sizeof alignof new new[] delete delete[] co_await`	Önek artırma ve azaltma, tekli işaretler, C tarzı dönüşüm, dolaylama, adres, boyut, bellek ayırma ve serbest bırakma	sağdan sola
4	`.* ->*`	Üye işaretçisi üzerinden erişim	soldan sağa
5	`* / %`	Çarpma, bölme, kalan	soldan sağa
6	`+ -`	Toplama, çıkarma	soldan sağa
7	`<< >>`	Bit kaydırma	soldan sağa
8	`<=>`	Üç yönlü karşılaştırma (C++20)	soldan sağa
9	`< <= > >=`	İlişkisel karşılaştırma	soldan sağa
10	`== !=`	Eşitlik karşılaştırması	soldan sağa

Öncelik	Operatörler	Açıklama	Birleşme
11	`&`	Bit düzeyinde VE	soldan sağa
12	`^`	Bit düzeyinde özel VEYA	soldan sağa
13	`|`	Bit düzeyinde VEYA	soldan sağa
14	`&&`	Mantıksal VE	soldan sağa
15	`||`	Mantıksal VEYA	soldan sağa
16	`?:` `=` `+=` `-=` `*=` `/=` `%=` `<<=` `>>=` `&=` `^=` `|=` `throw`	Koşullu ifade, atama, bileşik atama ve `throw` ifadesi	sağdan sola
17	`,`	Virgül operatörü	soldan sağa

B.3 C# operatör öncelik tablosu

C#'ta sıralama C'ye benzer, ama `?.`, `??`, `is`, `as`, `=>` ve `switch` gibi C#'a özgü operatörler kendi düzeylerinde yer alır. Tablo, Microsoft belgelerindeki kategori sırasını izler.

Öncelik	Operatörler	Açıklama	Birleşme
1	`x.y` `f(x)` `a[i]` `x?.y` `x?[i]` `x++` `x--` `x!` `x->y` `new` `typeof` `sizeof` `checked` `unchecked` `default` `nameof` `delegate` `stackalloc`	Birincil ifadeler: üye erişimi, çağrı, indeks, null koşullu erişim, sonek artırma ve azaltma, null bağışlama (`x!`), nesne oluşturma, tür ve boyut işlemleri	soldan sağa
2	`+x` `-x` `!x` `~x` `++x` `--x` `(T)x` `await` `&x` `*x` `^x`	Tekli operatörler, tür dönüşümü, `await` , adres ve dolaylama (`unsafe` bağlamda), sondan sayma (`^x`)	sağdan sola
3	`x..y`	Aralık (C# 8.0)	—
4	`switch`	Switch ifadesi (C# 8.0)	soldan sağa
5	`with`	`with` ifadesi, kayıt kopyalama (C# 9.0)	soldan sağa
6	`*` `/` `%`	Çarpma, bölme, kalan	soldan sağa
7	`+` `-`	Toplama, çıkarma	soldan sağa
8	`<<` `>>` `>>>`	Bit kaydırma. `>>>` işaretsiz sağa kaydırmadır (C# 11.0)	soldan sağa
9	`<` `<=` `>` `>=` `is` `as`	İlişkisel karşılaştırma ve tip sınama	soldan sağa
10	`==` `!=`	Eşitlik karşılaştırması	soldan sağa

Öncelik	Operatörler	Açıklama	Birleşme
11	`&`	Bit düzeyinde VE (ve mantıksal VE, bool için)	soldan sağa
12	`^`	Bit düzeyinde özel VEYA (ve mantıksal XOR, bool için)	soldan sağa
13	`|`	Bit düzeyinde VEYA (ve mantıksal VEYA, bool için)	soldan sağa
14	`&&`	Koşullu VE	soldan sağa
15	`||`	Koşullu VEYA	soldan sağa
16	`??`	Null birleştirme (C# 2.0)	sağdan sola
17	`?:`	Koşullu ifade	sağdan sola
18	`= += -= *= /= %= <<= >>= &= ^= |= ??= =>`	Atama, bileşik atama, null birleştirmeli atama (C# 8.0) ve lambda ifadesi	sağdan sola

C#'ta virgül bir operatör değildir; yalnızca `for` döngüsünün başlangıç ve artım kısımlarında kullanılır.

B.4 Üç dil arasındaki farklar

Aşağıdaki tablo, üç dilde aynı sorunun farklı çözüldüğü noktaları bir arada gösterir.

Konu	C	C++	C#
`.*` ve `->*` (üye işaretçisi)	yok	var, öncelik 4	yok
`::` kapsam belirleme	yok	var, en yüksek öncelik	yalnızca takma ad niteleyicisi olarak var
`<=>` üç yönlü karşılaştırma	yok	var (C++20)	yok
`is`, `as` tip sınama	yok	yok (`dynamic_cast` kullanılır)	var, ilişkisel düzeyde
`??`, `??=`, `?.`	yok	yok	var
`=>`	yok	yok (lambda `[]` ile yazılır)	var, atama düzeyinde
`>>>`	yok	yok	var (C# 11.0)
`new`	yok (bellek fonksiyonları çağrılır)	tekli düzeyde (öncelik 3)	birincil düzeyde
`sizeof`	tekli düzeyde	tekli düzeyde	birincil düzeyde
`throw`	yok	ifade olarak, atama düzeyinde	ifade olarak (C# 7.0)
Operand değerlendirme sırası	belirsiz	belirsiz; C++17 bazı operatörlerde sıra garantisi getirdi	işlenenler soldan sağa değerlendirilir

B.5 Sık yapılan hatalar

Öncelik kurallarını yanlış hatırlamak, okunması zor ve yanlış sonuç veren ifadelere yol açar. Dört tuzağa bakalım.

- `1 << 2 + 1` ifadesi 8 eder. `+` operatörü `<<`'den yüksek önceliklidir, yani önce `2 + 1` hesaplanır ve sonuç `1 << 3` olur. Bu davranış üç dilde de aynıdır.
- `a & 1 == 0` ifadesinde `==` operatörü `&`'den önce gelir. C ve C++'ta bu ifade `a & (1 == 0)` olarak ayrışır ve sonuç her zaman 0 olur. İstenen anlam için `(a & 1) == 0` yazılmalıdır. C#'ta ise `int` ile `bool` arasında `&` uygulanamayacağı için derleme hatası alınır.
- `!a == b` ifadesinde önce `!a` hesaplanır, yani `(!a) == b` anlamına gelir.
- `a ? b : c ? d : e` ifadesi `a ? b : (c ? d : e)` olarak gruplanır, çünkü `?:` sağdan sola birleşir. `*p++` ifadesi de `*(p++)` olarak ayrışır; yani `p` artırılır ve ifade eski adresteki değeri verir.

EK C

Anahtar Kelimeler

Anahtar kelime, derleyicinin bir yapı için ayırdığı sözcüktür. Bu sözcükleri değişken, fonksiyon ya da tip adı olarak kullanamayız. Aşağıdaki tablolarda her sözcüğün kısa Türkçe karşılığını buluyoruz. Ayrıntılı kullanımı için ilgili bölüme bakmak yeterlidir.

C ve C++ için sözcükler dilin standardına göre sabittir. C#'ta ise iki ayrı küme vardır: her bağlamda ad olarak kullanılamayan **ayrılmış** sözcükler ve yalnızca belli yerlerde anlam kazanan **bağlamsal** sözcükler.

C.1 C Anahtar Sözcükleri

C17 standardında 44 anahtar sözcük vardır. Bunların ilk 32'si C89 standardından gelir; `inline`, `restrict`, `_Bool`, `_Complex` ve `_Imaginary` C99 ile, `_Alignas`, `_Alignof`, `_Atomic`, `_Generic`, `_Noreturn`, `_Static_assert` ve `_Thread_local` ise C11 ile eklenmiştir.

Sözcük	Kısa anlamı
`auto`	Yerel değişkenin depolama süresini otomatik yapar; yerel değişkenlerin varsayılanıdır.
`break`	Döngüden ya da `switch` bloğundan çıkar.
`case`	`switch` içinde bir değer etiketi tanımlar.
`char`	Tek baytlık tamsayı tipidir; karakter tutmak için de kullanılır.
`const`	Değerin değiştirilemeyeceğini belirtir.
`continue`	Döngünün kalan kısmını atlayıp sonraki tura geçer.
`default`	`switch` içinde eşleşmeyen değerler için etiket tanımlar.
`do`	Koşulu sonda denetleyen döngüyü başlatır.
`double`	Çift duyarlıklı ondalık sayı tipidir.
`else`	`if` koşulu yanlış olduğunda çalışacak dalı belirtir.
`enum`	Adlandırılmış tamsayı sabitleri kümesi tanımlar.
`extern`	Değişken ya da fonksiyonun başka bir yerde tanımlandığını bildirir.
`float`	Tek duyarlıklı ondalık sayı tipidir.
`for`	Sayaçlı döngü başlatır.
`goto`	Etiketli bir yere koşulsuz dallanır; kullanımı sınırlıdır.
`if`	Koşula göre dal seçer.
`inline`	Fonksiyonun çağrı yerine kopyalanmasını önerir (C99).
`int`	Tamsayı tipidir.

Sözcük	Kısa anlamı
`long`	Daha geniş tamsayı tipidir.
`register`	Eskiden hızlı erişim önerisiydi; modern derleyicilerde etkisizdir.
`restrict`	İşaretçinin o bellek bölgesine tek erişim yolu olduğunu bildirir (C99).
`return`	Fonksiyondan değer döndürüp çıkar.
`short`	Kısa tamsayı tipidir.
`signed`	Tamsayının işaretli olduğunu belirtir.
`sizeof`	Bir tipin ya da ifadenin bayt cinsinden boyutunu verir.
`static`	Yerel değişkenin ömrünü uzatır; dosya dışına görünürlüğünü kapatır.
`struct`	Birden çok alanı bir arada tutan yapı tipi tanımlar.
`switch`	Bir değere göre birden çok dal seçer.
`typedef`	Mevcut bir tipe yeni ad verir.
`union`	Tüm üyeleri aynı bellek alanını paylaşan tip tanımlar.
`unsigned`	Tamsayının işaretsiz olduğunu belirtir.
`void`	Değer yokluğunu belirtir; boş işaretçi tipi olarak da kullanılır.
`volatile`	Değerin program dışından değişebileceğini derleyiciye bildirir.
`while`	Koşul doğru olduğu sürece döngüyü tekrarlar.
`_Alignas`	Değişkenin bellekteki hizalamasını belirtir (C11).
`_Alignof`	Bir tipin hizalama değerini verir (C11).
`_Atomic`	Bölünmez erişimli tip niteliği (C11).
`_Bool`	Mantıksal tiptir; `<stdbool.h>` içinde `bool` adıyla gelir (C99).
`_Complex`	Karmaşık sayı tipidir (C99).
`_Generic`	Tipe göre seçim yapan ifadedir (C11).
`_Imaginary`	Sanal sayı tipidir; isteğe bağlıdır (C99).
`_Noreturn`	Fonksiyonun geri dönmediğini bildirir (C11).
`_Static_assert`	Derleme anında koşul denetler (C11).
`_Thread_local`	Her iş parçacığı için ayrı bir değişken oluşturur (C11).

C23 ile gelenler

C23 standardı bazı sözcükleri anahtar sözcük yapmıştır. `alignas`, `alignof`, `bool`, `static_assert` ve `thread_local` daha önce başlık dosyalarında makro olarak tanımlanıyordu; artık doğrudan dilin parçasıdır. Ayrıca `constexpr`, `false`, `nullptr`, `true`, `typeof`, `typeof_unqual`, `_BitInt`, `_Decimal32`, `_Decimal64` ve `_Decimal128` yeni anahtar sözcüklerdir. Eski kodlardaki `<stdbool.h>` ve benzeri başlıklar hâlâ çalışır.

C.2 C++ Anahtar Sözcükleri

Aşağıdaki liste C++20 standardını kapsar. C++ ayrıca `and`, `or`, `not`, `xor` gibi bazı işleçlerin alternatif yazımlarını da anahtar sözcük sayar; bunlar tabloda ayrıca işaretlidir.

Sözcük	Kısa anlamı
`alignas`	Değişkenin bellekteki hizalamasını belirtir.
`alignof`	Bir tipin hizalama değerini verir.
`and`	`&&` işlecinin alternatif yazımıdır.
`and_eq`	`&=` işlecinin alternatif yazımıdır.
`asm`	Satır içi assembler kodu yazmaya yarar.
`auto`	Tipi başlatıcıdan çıkarır (C++11).
`bitand`	`&` işlecinin alternatif yazımıdır.
`bitor`	| işlecinin alternatif yazımıdır.
`bool`	Mantıksal tiptir.
`break`	Döngüden ya da `switch` bloğundan çıkar.
`case`	`switch` içinde değer etiketi tanımlar.
`catch`	Bir istisnayı yakalayan bloğu başlatır.
`char`	Tek baytlık karakter ya da tamsayı tipidir.
`char8_t`	UTF-8 karakter tipidir (C++20).
`char16_t`	UTF-16 karakter tipidir (C++11).
`char32_t`	UTF-32 karakter tipidir (C++11).
`class`	Sınıf tanımlar; üyeleri varsayılan olarak `private` olur.
`compl`	`~` işlecinin alternatif yazımıdır.
`concept`	Şablon parametreleri için koşul tanımlar (C++20).
`const`	Değişmezlik niteliğidir.
`consteval`	Yalnızca derleme zamanında çağrılabilen fonksiyon tanımlar (C++20).
`constexpr`	Derleme zamanında hesaplanabilen ifade ya da fonksiyon belirtir (C++11).
`constinit`	Değişkenin derleme zamanında başlatılmasını zorunlu kılar (C++20).
`const_cast`	Yalnızca `const` niteliğini kaldıran dönüşüm yapar.
`continue`	Döngünün sonraki turuna geçer.
`co_await`	Coroutine içinde bekleme noktası oluşturur (C++20).
`co_return`	Coroutine'den değer döndürür (C++20).
`co_yield`	Coroutine'den değer üretir (C++20).
`decltype`	Bir ifadenin tipini derleme zamanında verir (C++11).
`default`	Varsayılan üye ya da `switch` dalı belirtir.
`delete`	Dinamik belleği serbest bırakır (C++11'den beri üyeleri de silebilir).

Sözcük	Kısa anlamı
`do`	Koşulu sonda denetleyen döngüyü başlatır.
`double`	Çift duyarlıklı ondalık sayı tipidir.
`dynamic_cast`	Kalıtım hiyerarşisinde çalışma zamanı kontrollü dönüşüm yapar.
`else`	`if` koşulunun yanlış olduğu dalı belirtir.
`enum`	Sabitlerden oluşan numaralandırma tipi tanımlar.
`explicit`	Tek argümanlı kurucunun örtük dönüşüm yapmasını engeller.
`export`	Modül arayüzünde bir bildirimi dışa açar (C++20).
`extern`	Dış bağlantılı bildirim yapar.
`false`	Mantıksal yanlış değeridir.
`float`	Tek duyarlıklı ondalık sayı tipidir.
`for`	Döngü başlatır.
`friend`	Başka bir sınıf ya da fonksiyona özel erişim izni verir.
`goto`	Etiketli yere dallanır.
`if`	Koşula göre dal seçer.
`inline`	Tanımın birden çok çeviri biriminde bulunmasına izin verir.
`int`	Tamsayı tipidir.
`long`	Daha geniş tamsayı tipidir.
`mutable`	`const` nesnenin belirli bir üyesini değiştirilebilir kılar.
`namespace`	Ad alanı tanımlar.
`new`	Dinamik bellek ayırır.
`noexcept`	Fonksiyonun istisna fırlatmayacağını belirtir.
`not`	`!` işlecinin alternatif yazımıdır.
`not_eq`	`!=` işlecinin alternatif yazımıdır.
`nullptr`	Boş işaretçi değeridir (C++11).
`operator`	İşleç aşırı yüklemesi tanımlar.
`or`	|| işlecinin alternatif yazımıdır.
`or_eq`	|= işlecinin alternatif yazımıdır.
`private`	Yalnızca sınıfın kendi üyeleri erişebilir.
`protected`	Sınıf ve türediği sınıflar erişebilir.
`public`	Herkes erişebilir.
`register`	Eski bir sözcüktür; C++17'de anlamını yitirmiş ayrılmış sözcük olarak kalmıştır.
`reinterpret_cast`	Bit düzeyinde tip yeniden yorumlaması yapar.
`requires`	Şablonlar için gereksinim koşulu belirtir (C++20).
`return`	Fonksiyondan değer döndürür.
`short`	Kısa tamsayı tipidir.

Sözcük	Kısa anlamı
`signed`	Tamsayının işaretli olduğunu belirtir.
`sizeof`	Bir tipin ya da ifadenin bayt cinsinden boyutunu verir.
`static`	Statik ömürlü değişken tanımlar; sınıf üyesinde ise sınıfa aittir.
`static_assert`	Derleme anında koşul denetler.
`static_cast`	Derleme zamanında güvenli tip dönüşümü yapar.
`struct`	Varsayılan erişimi `public` olan sınıf benzeri yapı tanımlar.
`switch`	Çok dallı seçim yapar.
`template`	Şablon tanımlar.
`this`	Üye fonksiyonun çağrıldığı nesneye işaretçidir.
`thread_local`	İş parçacığına özgü depolama alanı tanımlar.
`throw`	Bir istisna fırlatır.
`true`	Mantıksal doğru değeridir.
`try`	İstisna fırlatabilecek kodu denetim altına alır.
`typedef`	Tip için takma ad tanımlar.
`typeid`	Çalışma zamanı tip bilgisini verir.
`typename`	Şablon içinde bağımlı bir adın tip olduğunu belirtir.
`union`	Üyeleri bellek alanını paylaşan tip tanımlar.
`unsigned`	Tamsayının işaretsiz olduğunu belirtir.
`using`	Ad alanı kullanımı, tip takma adı ya da kalıtımda erişim belirtir.
`virtual`	Çalışma zamanında hangi sürümün çağrılacağını sonraya bırakan fonksiyon belirtir.
`void`	Değer yokluğunu belirtir.
`volatile`	Derleyicinin optimizasyonunu sınırlayan niteliktir.
`wchar_t`	Geniş karakter tipidir.
`while`	Koşul doğru olduğu sürece döngüyü tekrarlar.
`xor`	`^` işlecinin alternatif yazımıdır.
`xor_eq`	`^=` işlecinin alternatif yazımıdır.

C.3 C# Ayrılmış Sözcükleri

Bu sözcükler, C# derleyicisinde ad olarak kullanılamaz. `int`, `string` ve `bool` gibi tip adları da bu kümededir; onları değişken adı olarak yazamayız.

Sözcük	Kısa anlamı
`abstract`	Soyut sınıf ya da üye tanımlar; doğrudan örneklenemez.
`as`	Güvenli tip dönüşümü yapar; başarısızsa `null` verir.
`base`	Üst sınıfın üyelerine erişir.

Sözcük	Kısa anlamı
`bool`	Mantıksal tiptir.
`break`	Döngüden ya da `switch` bloğundan çıkar.
`byte`	8 bitlik işaretsiz tamsayı tipidir.
`case`	`switch` içinde değer etiketi tanımlar.
`catch`	İstisnayı yakalayan bloğu başlatır.
`char`	16 bitlik UTF-16 karakter tipidir.
`checked`	Tamsayı taşmasını istisnaya çevirir.
`class`	Başvuru tipi olan sınıf tanımlar.
`const`	Derleme zamanı sabiti tanımlar.
`continue`	Döngünün sonraki turuna geçer.
`decimal`	Yüksek hassasiyetli ondalık tiptir; para hesaplarında kullanılır.
`default`	Varsayılan değer ya da `switch` dalı belirtir.
`delegate`	Metot başvurusu tipi tanımlar.
`do`	Koşulu sonda denetleyen döngüyü başlatır.
`double`	64 bitlik ondalık sayı tipidir.
`else`	`if` koşulunun yanlış olduğu dalı belirtir.
`enum`	Adlandırılmış sabitler kümesi tanımlar.
`event`	Olay bildirir.
`explicit`	Açık dönüşüm operatörü tanımlar.
`extern`	Dış derlemede tanımlı bir üyeyi bildirir.
`false`	Mantıksal yanlış değeridir.
`finally`	İstisna olsun olmasın çalışan bloğu başlatır.
`fixed`	Bellekte yeri değişmeyen işaretçi tanımlar; güvensiz bağlamda kullanılır.
`float`	32 bitlik ondalık sayı tipidir.
`for`	Sayaçlı döngü başlatır.
`foreach`	Koleksiyonun elemanlarını sırayla dolaşır.
`goto`	Etiketli yere dallanır.
`if`	Koşula göre dal seçer.
`implicit`	Örtük dönüşüm operatörü tanımlar.
`in`	`foreach` içinde koleksiyonu belirtir; parametrede salt okunur geçiş sağlar.
`int`	32 bitlik tamsayı tipidir.
`interface`	Arayüz tanımlar.
`internal`	Yalnızca aynı derlemeden erişilebilir üye belirtir.
`is`	Tip denetimi ve desen eşleme yapar.
`lock`	Bir nesne üzerinde karşılıklı dışlama sağlar.

Sözcük	Kısa anlamı
`long`	64 bitlik tamsayı tipidir.
`namespace`	Ad alanı tanımlar.
`new`	Nesne oluşturur; üye gizlemek için de kullanılır.
`null`	Başvuru tipinin boş değeridir.
`object`	`System.Object` tipinin takma adıdır.
`operator`	İşleç aşırı yüklemesi tanımlar.
`out`	Metoda çıkış değiştirgeni geçirir.
`override`	Temel sınıftaki sanal üyeyi geçersiz kılar.
`params`	Değişken sayıda argüman kabul eder.
`private`	Yalnızca içeren tip erişebilir.
`protected`	İçeren tip ve türeyen tipler erişebilir.
`public`	Her yerden erişilebilir.
`readonly`	Yalnızca kurucuda atanabilen alan tanımlar.
`ref`	Değiştirgeyi başvuru ile geçirir.
`return`	Metottan değer döndürür.
`sbyte`	8 bitlik işaretli tamsayı tipidir.
`sealed`	Kalıtılamaz sınıf ya da geçersiz kılınamaz üye belirtir.
`short`	16 bitlik tamsayı tipidir.
`sizeof`	Değer tipinin bayt boyutunu verir; güvensiz bağlamda kullanılır.
`stackalloc`	Yığında bellek ayırır.
`static`	Bir örneğe değil tipe ait üye tanımlar.
`string`	`System.String` tipinin takma adıdır.
`struct`	Değer tipi tanımlar.
`switch`	Çok dallı seçim yapar.
`this`	Geçerli nesneye başvurur.
`throw`	Bir istisna fırlatır.
`true`	Mantıksal doğru değeridir.
`try`	İstisna denetimi bloğunu başlatır.
`typeof`	Tipin `System.Type` nesnesini verir.
`uint`	32 bitlik işaretsiz tamsayı tipidir.
`ulong`	64 bitlik işaretsiz tamsayı tipidir.
`unchecked`	Tamsayı taşmasını yok sayar (varsayılan davranış).
`unsafe`	Güvensiz kod bloğu başlatır.
`ushort`	16 bitlik işaretsiz tamsayı tipidir.
`using`	Ad alanını içe aktarır ya da kaynağı otomatik kapatan blok başlatır.

Sözcük	Kısa anlamı
`virtual`	Türeyen sınıflarda geçersiz kılınabilen üye tanımlar.
`void`	Değer döndürmeyen metot belirtir.
`volatile`	Birden çok iş parçacığının erişeceği alan belirtir.
`while`	Koşullu döngü başlatır.

C.4 C# Bağlamsal Anahtar Sözcükleri

Bu sözcükler yalnızca belirli yerlerde anlam taşır; başka yerlerde değişken ya da metot adı olarak kullanılabilir. Örneğin `value` ve `get` adlarını özellik dışında serbestçe kullanabiliriz.

Sözcük	Kısa anlamı
`add`	Olay aboneliği için ekleme erişimcisidir.
`alias`	`extern alias` yönergesinde kullanılır.
`and`	Desen eşlemede "ve" bağlacıdır (C# 9).
`ascending`	LINQ sorgusunda artan sıralama belirtir.
`async`	Asenkron metot ya da lambda tanımlar.
`await`	Asenkron işlemin bitmesini bekler.
`by`	LINQ `group` yan tümcesinde anahtarı belirtir.
`descending`	LINQ sorgusunda azalan sıralama belirtir.
`dynamic`	Tipi çalışma zamanında çözülen tip belirtir.
`equals`	LINQ `join` içinde eşleşme koşulunu belirtir.
`file`	Yalnızca tanımlandığı dosyada görünen tip belirtir (C# 11).
`from`	LINQ sorgusunu başlatır.
`get`	Özelliğin okuyucu erişimcisidir.
`global`	`global::` ile kök ad alanına başvurur.
`group`	LINQ sorgusunda gruplama yapar.
`init`	Yalnızca nesne başlatma sırasında atanabilen özellik belirtir (C# 9).
`into`	LINQ sorgusunda ara sonucu yeni bir değişkene bağlar.
`join`	LINQ sorgusunda birleştirme yapar.
`let`	LINQ sorgusunda ara değişken tanımlar.
`nameof`	Bir adın metin karşılığını derleme zamanında verir.
`not`	Desen eşlemede "değil" anlamı taşır (C# 9).
`notnull`	Null olmayan tip kısıtlaması belirtir.
`on`	LINQ `join` içinde koşulu başlatır.
`or`	Desen eşlemede "veya" bağlacıdır (C# 9).
`orderby`	LINQ sorgusunda sıralama yapar.

Sözcük	Kısa anlamı
`partial`	Bir tipin tanımını birden çok dosyaya böler.
`record`	Değer eşitliği olan başvuru tipi tanımlar (C# 9).
`remove`	Olay aboneliği için kaldırma erişimcisidir.
`required`	Nesne oluşturulurken atanması zorunlu üye belirtir (C# 11).
`scoped`	Değiştirgenin yalnızca yığında yaşamasını belirtir (C# 11).
`select`	LINQ sorgusunda seçim yapar.
`set`	Özelliğin yazıcı erişimcisidir.
`unmanaged`	Yönetilmeyen tip kısıtlaması belirtir.
`value`	Özellik yazıcısında atanan değeri temsil eder.
`var`	Yerel değişkenin tipini başlatıcıdan çıkarır.
`when`	`catch` filtresi ya da desen koşulu belirtir.
`where`	Tip kısıtlaması ya da LINQ filtresi belirtir.
`with`	Kayıt (record) kopyası oluşturur (C# 9).
`yield`	Yineleyici içinde değer üretir.

EK D

Sık Görülen Derleyici ve Çalışma Zamanı Hataları

Bu ekte C, C++ ve C# ile çalışırken en sık karşılaşılan hataları topluyoruz. Her hata için önce mesajın kendisini, sonra ne anlama geldiğini ve nasıl düzeltileceğini veriyoruz.

Hataların üç türü vardır. **Derleme hataları** (error) program derlenmeden önce ortaya çıkar ve çalıştırılabilir dosya oluşmasını engeller. **Uyarılar** (warning) derlemeyi durdurmaz, ama büyük olasılıkla bir yanlışa işaret eder. **Çalışma zamanı hataları** ise program çalıştıktan sonra görülür.

Mesajların birebir metni derleyici sürümüne ve işletim sistemine göre değişebilir. Bu yüzden önemli olan, mesajdaki anahtar sözcüklerin ne anlattığını kavramaktır. Hata mesajlarını okurken her zaman ilk hatadan başlayalım; sonraki hataların çoğu ilkinin sonucudur.

D.1 C Hataları

Aşağıdaki mesajlar gcc ile üretilmiştir. `-std=c17 -Wall -Wextra` seçenekleriyle derlenmiş bir ortamda görülür.

Noktalı virgül eksik

```
printf("Merhaba\n")
return 0;
```

```
error: expected ';' before 'return'
```

Anlamı. `printf` satırının sonundaki `;` yazılmamış. Derleyici bunu bir sonraki sözcük olan `return` geldiğinde fark eder. Hata konumu bazen bir satır sonrasını gösterir; asıl eksiklik bir üst satırdadır.

Nasıl düzeltilir. İşaret edilen yere ya da bir üst satırın sonuna `;` ekleyin.

Kapanmayan blok

```
error: expected declaration or statement at end of input
```

Anlamı. Dosyanın sonuna gelindi, ama açık bir `{` hâlâ kapanmamış. Bir `if` bloğunun `}` karakteri eksikse son `}` o bloğu kapatır ve `main` açık kalır.

Nasıl düzeltilir. Her `{` için aynı girintide bir `}` olduğuna bakın. VS Code'da kodu otomatik biçimlendirmek (Shift+Alt+F), eksik ya da fazla parantezin yerini hemen gösterir.

Tanımlanmamış ad

```
error: 'Toplam' undeclared (first use in this function); did you mean 'toplam'?
```

Anlamı. Bu adla tanımlanmış bir şey yok. C büyük-küçük harfe duyarlıdır; `Toplam` ile `toplam` farklı adlardır. Bir `for` ya da `{ }` bloğu içinde tanımlanmış bir değişken dışarıda kullanılmış olabilir.

Nasıl düzeltilir. Adın yazımını kontrol edin. Değişkeni kullanıldığı yerden önce ve doğru kapsamda tanımlayın.

Başlık dosyası eksik

```c
int main(void) {
    printf("Merhaba\n");
    return 0;
}
```

```
error: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
note: include '<stdio.h>' or provide a declaration of 'printf'
```

Anlamı. `printf` bildirimi bulunamadı, çünkü `<stdio.h>` eklenmemiş. gcc 14 ve sonrasında bu durum bir hatadır; eski sürümlerde yalnızca uyarıdır, ama program yine de yanlış biçimde derlenebilir.

Nasıl düzeltilir. `#include <stdio.h>` ekleyin. `malloc` ve `free` için `<stdlib.h>`, `strlen` için `<string.h>`, `sqrt` için `<math.h>` gerekir.

Argüman sayısı uyuşmuyor

```
error: too few arguments to function 'topla'; expected 2, have 1
note: declared here
```

Anlamı. Çağrıda verilen argüman sayısı, fonksiyonun parametre sayısından azdır. Mesaj beklenen ve verilen sayıyı yazar; `declared here` notu fonksiyonun tanımını gösterir.

Nasıl düzeltilir. Argüman listesini parametre listesiyle eşleştirin. Prototipte `int topla();` yazmak yerine `int topla(int a, int b);` yazın; boş parantez C17'de argümanları denetlemez.

Dizinin adına atama

```
error: assignment to expression with array type
```

Anlamı. Dizinin adına `=` ile değer atanamaz. Dizi adı, sabit bir bellek bloğunun adresi gibi davranır.

Nasıl düzeltilir. Değeri tanımda verin (`char ad[10] = "Ali";`) ya da değişmeyen bir dizi için `strcpy` kullanın (Bölüm 11).

Yanlış çözümlenen ok işleci

```
error: 'p' is a pointer; did you mean to use '->'?
```

Anlamı. `p` bir yapı işaretçisi olduğu için üyesine nokta ile değil ok (`->`) ile erişilmelidir. Tersi durumda, yani yapının kendisine ok uygulandığında `invalid type argument of '->'` mesajı çıkar.

Nasıl düzeltilir. Değişkenin tanımına bakın. `struct Dugum d` ise `d.deger`, `struct Dugum *p` ise `p->deger` yazılır (Bölüm 14).

Tanınmayan tip: string

```
error: unknown type name 'string'
```

Anlamı. C'de `string` adında bir tip yoktur. Metin, `char` dizisi ya da `char *` ile tutulur. `bool` için de `<stdbool.h>` başlığı gerekir.

Nasıl düzeltilir. `char ad[20];` ya da `const char *ad` kullanın (Bölüm 11). `bool` kullanacaksanız `<stdbool.h>` başlığını ekleyin.

Kıvrık tırnak

```
error: stray '\342' in program
```

Anlamı. Kod, bir metin editöründen ya da web sayfasından kopyalanan kıvrık tırnak (`“` ya da `”`) içeriyor. Bu karakterler C'de dize sınırı değildir; `\342` onun UTF-8 kodunun ilk baytıdır.

Nasıl düzeltilir. Kıvrık tırnakları silip klavyeden düz `"` ve `'` yazın. Kodu düz metin olarak yapıştırmak da işe yarar.

Tamsayı ile işaretçi karışması

```
error: initialization of 'int *' from 'int' makes pointer from integer without a cast [-Wint-
conversion]
```

Anlamı. `int *p = 5;` yazıldığında tamsayı bir adres gibi kullanılmıştır. Aynı mesaj, tamsayı değişkeninin işaretçi bekleyen bir fonksiyona doğrudan verilmesinde de çıkar.

Nasıl düzeltilir. Değişkenin adresini `&` ile verin: `int *p = &sayi;` ve `artir(&sayi);` (Bölüm 12).

Biçim belirteci uyuşmuyor

```
warning: format '%d' expects argument of type 'int', but argument 2 has type 'double' [-
Wformat=]
```

Anlamı. `printf` biçim dizesindeki belirteç ile verilen değerin tipi uyuşmuyor. `%d` tamsayı içindir; `double` için `%f` kullanılır. Uyarıya rağmen program çalışır, ama ekrana çöp değer yazılır.

Nasıl düzeltilir. Belirteci tipe göre seçin: `int` için `%d`, `double` için `%f`, `char` için `%c`, metin için `%s`.

Başlatılmamış değişken

```
warning: 'toplam' is used uninitialized [-Wuninitialized]
```

Anlamı. Yerel bir değişken, değer verilmeden okunmuş. Yerel değişkenlerin ilk değeri bellekteki çöptür.

Nasıl düzeltilir. Tanımda ilk değer verin: `int toplam = 0;`.

Değer döndürmeyen yol

```
warning: control reaches end of non-void function [-Wreturn-type]
```

Anlamı. `int` döndüren fonksiyon, bazı koşul yollarında `return` görmeden bitiyor. Derleyici, o yolda dönen değerin çöp olacağını söyler.

Nasıl düzeltilir. Fonksiyonun sonuna her durumu karşılayan bir `return` ekleyin.

Eksik matematik kütüphanesi

```
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
kok.c:(.text+0x1e): undefined reference to 'sqrt'
collect2: error: ld returned 1 exit status
```

Anlamı. Derleme aşaması geçildi; ama bağlayıcı `sqrt` kodunu bulamadı. `<math.h>` yalnızca prototipi getirir; kodu ayrı bir kütüphanededir (`libm`). Bu mesaj bağlama (linker) aşamasında çıkar.

Nasıl düzeltilir. Linux'ta komutun sonuna `-lm` ekleyin: `gcc -std=c17 -Wall -Wextra kok.c -o kok -lm`. `-lm` kaynak dosyadan sonra gelmelidir.

Çalışma zamanı: Segmentation fault

```
Segmentation fault (core dumped)
```

Anlamı. Program, erişim hakkı olmayan bir bellek adresine dokundu ve işletim sistemi onu sonlandırdı. Başlıca nedenler başlatılmamış ya da `NULL` işaretçiyi kullanmak, dizi sınırını aşmak, `free` sonrası belleğe erişmek ve temel durumu olmayan özyinelemedir. Windows'ta bu hata mesajsız bir kapanma ve `-1073741819` (0xC0000005) çıkış kodu olarak görünür.

Nasıl düzeltilir. İşaretçilere tanımda değer verin, kullanmadan önce `NULL` olmadığını denetleyin. Linux'ta programı `gcc -fsanitize=address -g` ile derleyin; hatanın satırını raporlar (Bölüm 13).

Çalışma zamanı: Floating point exception

```
Floating point exception (core dumped)
```

Anlamı. Bir tamsayı sıfıra bölündü. Adı yanıltıcıdır: hata ondalık sayılardan değil, tamsayı bölmesinden gelir. `double` ile sıfıra bölmede program çökmez; sonuç `inf` ya da `nan` olur.

Nasıl düzeltilir. Bölmeden önce bölenin sıfır olmadığını denetleyin: `if (b != 0)`.

Çalışma zamanı: Double free

```
free(): double free detected in tcache 2
Aborted (core dumped)
```

Anlamı. Aynı bellek bloğu iki kez `free` edildi. Bu tanımsız davranıştır; Linux'taki glibc kütüphanesi bunu fark edip programı durdurur.

Nasıl düzeltilir. `free(p);` yazdıktan sonra `p = NULL;` atayın. `free(NULL)` zararsızdır, ikinci `free` çağrısı böylece güvenli olur (Bölüm 13).

D.2 C++ Hataları

Aşağıdaki mesajlar g++ ile üretilmiştir. Mesajlar `-std=c++17 -Wall -Wextra` ile alınmıştır. Uzun aday listeleri (candidate) kısaltılmıştır.

Kapanmayan metin

```
error: missing terminating " character
error: expected primary-expression before 'return'
```

Anlamı. Bir metin sabiti `"` ile açılmış, ama aynı satırda kapanmamış. Metin satır sonuna kadar sürer. İkinci hata sahtedir; bozuk satırdan sonra `return` beklenmemiştir. İlk hatayı düzeltince o da kaybolur.

Nasıl düzeltilir. Kapanış tırnağını ekleyin. Metnin içinde tırnak gerekiyorsa `\"` yazın (Bölüm 27).

Ad alanı eksik

```
error: 'cout' was not declared in this scope; did you mean 'std::cout'?
```

Anlamı. `cout` standart kütüphanenin `std` ad alanındadır. `using namespace std;` satırı ya da `std::` öneki eksikse bu mesaj çıkar. `string` gibi adlarda ilgili başlık da eksikse, derleyici `note` satırıyla hangi başlığı ekleyeceğini söyler.

Nasıl düzeltilir. `std::cout` yazın ya da dosyanın başına `using namespace std;` ekleyin. Başlık eşlemesi: `cout` için `<iostream>`, `string` için `<string>`, `vector` için `<vector>` (Bölüm 23).

Fonksiyon dışında komut

```
error: 'cout' does not name a type
```

Anlamı. Fonksiyonların dışında yalnızca tanımlar bulunabilir: fonksiyon, sınıf ve global değişken. `cout << ...` bir komut olduğu için bir fonksiyonun içinde olmalıdır. Fazladan bir `}` fonksiyonu erken kapatmış olabilir.

Nasıl düzeltilir. Komutu bir fonksiyonun içine alın ve süslü parantezleri sayın.

Başlık dosyası bulunamadı

```
fatal error: iostrem: No such file or directory
compilation terminated.
```

Anlamı. `#include` satırındaki başlık dosyası bulunamadı. `fatal error` derlemeyi hemen durdurur; sonraki hataları aramaz. Kendi yazdığınız `"hesap.h"` gibi bir başlıkta da aynı mesaj, dosya adı ya da klasör yanlışsa çıkar.

Nasıl düzeltilir. Adı `<iostream>` olarak düzeltin. Kendi başlığınızın `.cpp` dosyasıyla aynı klasörde olduğunu ve adının birebir aynı yazıldığını kontrol edin.

Operatör yüklenmemiş

```
error: no match for 'operator<<' (operand types are 'std::ostream' {aka
'std::basic_ostream<char>'} and 'Nokta')
```

Anlamı. `cout << nesne` yazdınız, ama `Nokta` sınıfı için `<<` operatörü tanımlanmamış. g++ yüzlerce aday satırı yazar; ilk satırdaki `operand types` kısmı eksik olanı tek başına söyler.

Nasıl düzeltilir. `ostream &operator<<(ostream &cikis, const Nokta &p)` fonksiyonunu yazın (Bölüm 32). Ya da alanları tek tek yazdırın: `cout << p.x << ' ' << p.y`.

İki metin sabiti toplanamaz

```
error: invalid operands of types 'const char [10]' and 'const char [7]' to binary 'operator+'
```

Anlamı. `"Merhaba, "` ve `"dünya"` C tarzı karakter dizileridir; `+` ile birleştirilemezler. `+` işleci, en az bir tarafı `std::string` olan ifadelerde çalışır.

Nasıl düzeltilir. Birinci tarafı `std::string` olarak kurun: `std::string("Merhaba, ") + "dünya"` (Bölüm 27).

Kapsayıcıya yanlış tipte değer

```
error: no matching function for call to 'push_back(const char [5])'
```

Anlamı. `vector<int>` yalnızca tamsayı kabul eder. `push_back` için metin sabiti verildiğinde derleyici uyan bir sürüm bulamaz; adaylar `invalid conversion from 'const char*' to 'int'` diye reddedilir.

Nasıl düzeltilir. Eleman tipine uygun değer verin ya da kapsayıcının tipini `vector<string>` yapın (Bölüm 35).

Uyarı: işaretli ve işaretsiz karşılaştırma

```
warning: comparison of integer expressions of different signedness: 'int' and
'std::vector<int>::size_type' {aka 'long unsigned int'} [-Wsign-compare]
```

Anlamı. `int` ile `size()` dönüş değeri (işaretsiz) karşılaştırıldı. Negatif sayılar bu karşılaştırmada beklenmedik büyük değerlere dönüşür.

Nasıl düzeltilir. Sayacı `size_t` yapın ya da aralık tabanlı `for` kullanın: `for (int s : sayilar)`.

Uyarı: switch dalı düşüyor

```
warning: this statement may fall through [-Wimplicit-fallthrough=]
```

Anlamı. Bir `case` bloğu `break` olmadan bitiyor ve akış bir sonraki `case`'e düşüyor. Bu çoğu zaman unutulmuş bir `break` demektir.

Nasıl düzeltilir. Her dalın sonuna `break;` ekleyin. Düşme bilerek yapılıyorsa `[[fallthrough]];` yazarak niyeti belirtin (Bölüm 24).

Uyarı: yerel değişkene referans

```
warning: reference to local variable 'sonuc' returned [-Wreturn-local-addr]
```

Anlamı. Fonksiyon bittiğinde yerel değişken yok olur. Fonksiyon ise o değişkene bir referans döndürür; çağıran taraf sarkan (dangling) bir referans elde eder. Bu referansı okumak tanımsız davranıştır (Bölüm 28).

Nasıl düzeltilir. Dönüş tipini `int` yaparak değerin kopyasını döndürün.

Bağlama: tanım eksik

```
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
hata.cpp:(.text+0x...): undefined reference to 'topla(int, int)'
collect2: error: ld returned 1 exit status
```

Anlamı. Prototip yazılmış ama gövdesi hiçbir `.o` dosyasında yok. Ya da gövdeyi içeren dosya derlemeye eklenmemiş. C++'ta mesajda parametre tipleri de yer alır, çünkü aynı adlı fonksiyonlar aşırı yükleme ile birden çok kez tanımlanabilir.

Nasıl düzeltilir. Gövdeyi yazın; parametre listesi ve tiplerinin prototiple birebir aynı olduğunu denetleyin. Birden çok dosya varsa hepsini tek komutta verin: `g++ main.cpp hesap.cpp -o program` (Bölüm 15).

Çalışma zamanı: yakalanmayan istisna

```
terminate called after throwing an instance of 'std::out_of_range'
  what():  vector::_M_range_check: __n (which is 5) >= this->size() (which is 3)
Aborted
```

Anlamı. `at()` ile sınır dışı bir indeks kullanıldı ve fırlatılan `out_of_range` istisnası hiçbir `catch` bloğunda yakalanmadı. Program `terminate` ile sonlanır. Mesajın metni derleyici sürümüne göre değişebilir; `what()` kısmındaki açıklama asıl bilgiyi taşır.

Nasıl düzeltilir. İndeksin `0` ile `size() - 1` aralığında olduğunu denetleyin ya da `try-catch` ile yakalayın (Bölüm 34). `[]` ile yapılan sınır dışı erişim ise hata vermez; bu yüzden tanımsız davranıştır.

D.3 C# Hataları

Aşağıdaki derleme mesajları C# derleyicisinin (csc, `dotnet build`) biçimindedir. Dosya adı ve konum `Program.cs(satır,sütun)` olarak yazılır; kod, mesajın sonundaki hata kodu ile aranabilir.

Noktalı virgül eksik

```
error CS1002: ; expected
```

Anlamı. Derleyici, bir deyimin bittiği yerde `;` bekledi. Hata, `;` in olması gereken yerde, yani satırın sonunda gösterilir.

Nasıl düzeltilir. Satırın sonuna `;` ekleyin. Her deyim, yeni satıra geçilse de `;` ile biter.

Tanımlanmamış ad

```
error CS0103: The name 'Toplam' does not exist in the current context
```

Anlamı. C# büyük-küçük harfe duyarlıdır; `Toplam` ile `toplam` ayrı adlardır. Başka bir olası neden, ad alanı ya da `using` yönergesinin eksik olmasıdır.

Nasıl düzeltilir. Adın yazımını ve kapsamını kontrol edin. Gerekiyorsa ilgili `using` satırını ekleyin (Bölüm 42).

Tip uyumsuzluğu

```
error CS0029: Cannot implicitly convert type 'string' to 'int'
error CS0266: Cannot implicitly convert type 'double' to 'int'. An explicit conversion exists
(are you missing a cast?)
```

Anlamı. `"19"` bir metindir ve `int`'e otomatik çevrilmez. `double` değerini `int`'e çevirmek ise veri kaybedebileceği için açık bir dönüşüm ister; mesajın sonundaki `are you missing a cast?` bunu söyler.

Nasıl düzeltilir. Metin için `int.Parse` ya da daha güvenli olan `int.TryParse` kullanın. Ondalık sayıyı tama çevirmek istiyorsanız `(int)` dönüşümünü açıkça yazın (Bölüm 42).

Atanmamış yerel değişken

```
error CS0165: Use of unassigned local variable 'harf'
```

Anlamı. `harf` yalnızca bir koşulun doğru olduğu durumda atanıyor. Derleyici her yolda değer alındığını kanıtlayamadığı için kullanımı reddeder. Yerel değişkenlerin alanların aksine varsayılan değeri yoktur.

Nasıl düzeltilir. Tanımda ilk değer verin: `string harf = "";`. Ya da her dalda atama yapın.

Tüm yollar değer döndürmüyor

```
error CS0161: 'Isaret(int)': not all code paths return a value
```

Anlamı. Metot `int` döndürmek zorunda, ama `n == 0` durumunda `return` olmadan bitiyor. Derleyici, koşulların hepsinin yanlış olabileceğini dikkate alır.

Nasıl düzeltilir. Metodun sonuna her durumu karşılayan bir `return` ekleyin ya da son `else if` yerine `else` kullanın.

Uygun aşırı yükleme yok

```
error CS1501: No overload for method 'Ortalama' takes 3 arguments
```

Anlamı. `Ortalama` iki parametre alıyor; çağrıda üç argüman var. Derleyici bu sayıda argüman kabul eden bir sürüm bulamadı.

Nasıl düzeltilir. Çağrıyı tanıma uydurun ya da üç parametreli bir aşırı yükleme yazın (Bölüm 46).

Nesne başvurusu gerekli

```
error CS0120: An object reference is required for the non-static field, method, or property
'Hesap.Topla(int, int)'
```

Anlamı. `Topla` örnek (instance) metodudur ve sınıf adıyla çağrılamaz. Hangi nesnenin `Topla`'sının çalışacağı belirsizdir.

Nasıl düzeltilir. Bir nesne oluşturup onun üzerinden çağırın: `new Hesap().Topla(2, 3)`. Metot nesneye bağlı değilse `static` yapın (Bölüm 49).

Salt okunur değer

```
error CS0200: Property or indexer 'string.this[int]' cannot be assigned to -- it is read only
```

Anlamı. `string` değişmezdir; karakterleri tek tek değiştirilemez. `ad[0]` yalnızca okunabilir bir indeksleyicidir.

Nasıl düzeltilir. Yeni bir metin üretin: `ad = "a" + ad.Substring(1);` (Bölüm 48).

Yazım hatası üye adı

```
error CS1061: 'string' does not contain a definition for 'Lenght'
```

Anlamı. `string` tipinde `Lenght` diye bir üye yoktur; doğrusu `Length`dir. Mesajın sonundaki `using directive` önerisi bu durumda yanıltıcıdır.

Nasıl düzeltilir. Nokta koyduğunuzda açılan öneri listesinden doğru adı seçin.

Uyarı: null olabilecek değer

```
warning CS8600: Converting null literal or possible null value to non-nullable type.
```

Anlamı. `Console.ReadLine()` girdi akışı kapanmışsa `null` dönebilir. Bu değeri boş olamayan bir `string` değişkene atamak bir uyarıdır.

Nasıl düzeltilir. `string ad = Console.ReadLine() ?? "";` yazın. Böylece `null` gelirse boş metin atanır.

Çalışma zamanı: FormatException

```
Unhandled exception. System.FormatException: The input string '12a' was not in a correct format.
   at Program.<Main>$(String[] args) in formatHatasi.cs:line 2
```

Anlamı. `int.Parse` metni sayıya çeviremedi. İlk satır çalışmış, ikincisinde program durmuştur. `"3.5"` ve `""` de aynı hatayı verir.

Nasıl düzeltilir. Kullanıcıdan gelen metni `int.TryParse` ile çevirin: `if (int.TryParse(metin, out int sayi)) ...`.

Çalışma zamanı: IndexOutOfRangeException

```
Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.
```

Anlamı. Dizinin geçerli indeksleri `0` ile `Length - 1` arasındadır. `i <= dizi.Length` koşulu son turda fazladan bir erişim yapar.

Nasıl düzeltilir. Koşulu `i < dizi.Length` yapın ya da `foreach` kullanın (Bölüm 47).

Çalışma zamanı: NullReferenceException

```
Unhandled exception. System.NullReferenceException: Object reference not set to an instance of
an object.
```

Anlamı. `new string[3]` üç elemanlı bir dizi oluşturur, ama elemanlar `null` olarak başlar. `null` olan bir elemanın üyesine erişilemez. Mesaj hangi değişkenin `null` olduğunu söylemez; satırı kendiniz bulmanız gerekir.

Nasıl düzeltilir. Elemanlara kullanmadan önce değer verin ya da `adlar[1]?.ToUpper()` gibi `?.` operatörünü kullanın.

Çalışma zamanı: DivideByZeroException

```
Unhandled exception. System.DivideByZeroException: Attempted to divide by zero.
```

Anlamı. Tamsayı sıfıra bölündü. `double` ile bölmede bu istisna çıkmaz; sonuç sonsuz olur.

Nasıl düzeltilir. Bölmeden önce böleni denetleyin: `if (adet != 0)`.

EK E

Terimler Sözlüğü

Bu sözlük kitapta geçen temel terimleri Türkçe alfabe sırasına göre toplar. Her terim için İngilizce karşılığını ve tek cümlelik kısa bir açıklamasını veriyoruz. Ayrıntı gerektiren terimler için ilgili bölüme bakmak yeterlidir.

E.1 A

Türkçe terim	İngilizce	Açıklama
Açık dönüşüm	Explicit conversion, cast	Dönüşümün programcı tarafından açıkça yazıldığı tip dönüşümüdür.
Ad alanı	Namespace	Adların çakışmasını önleyen adlandırma kapsamıdır.
Adres	Address	Belleğin bir baytının ya da nesnenin konumudur.
Ağaç	Tree	Düğümlerin hiyerarşik bağlarla dizildiği veri yapısıdır.
Akıllı işaretçi	Smart pointer	Nesnenin yaşam süresini otomatik yöneten işaretçi benzeri sarmalayıcıdır.
Alan	Field	Bir yapının ya da sınıfın veri üyesidir.
Algoritma	Algorithm	Bir problemi çözmek için sonlu sayıda adımdan oluşan yöntemdir.
Alt sınıf	Subclass	Kalıtımla başka bir sınıftan türetilen sınıftır.
Anahtar sözcük	Keyword	Dilin kendisi için ayırdığı ve ad olarak kullanılamayan sözcüktür.
Aralık tabanlı for	Range-based for	Bir koleksiyonun elemanlarını sırayla dolaşan döngüdür.
Arayüz	Interface	Yalnızca üye imzalarını tanımlayan sözleşme tipidir.
Argüman	Argument	Fonksiyon çağrısında parametreye verilen gerçek değerdir.
Artırma	Increment	Bir değişkenin değerini bir artırma işlemidir (`++`).
Asenkron	Asynchronous	Bir işin bitmesini beklemeden devam eden programlama biçimidir.
Assert	Assert	Bir koşulu denetleyen ve yanlışsa programı durduran ifadedir.
Aşırı yükleme	Overloading	Aynı adla farklı parametre listelerine sahip birden çok tanım yapmaktır.
Atama	Assignment	Bir değişkene değer yazma işlemidir.
Atomik	Atomic	Tek adımda ve bölünmeden yapılan işlemdir.
Ayrıştırma	Parsing	Bir metni anlamlı parçalara ayırma işlemidir.
Azaltma	Decrement	Bir değişkenin değerini bir azaltma işlemidir (`--`).

E.2 B

Türkçe terim	İngilizce	Açıklama
Bağlama	Linking	Derlenmiş parçaların birbirine bağlanması işlemidir.
Bağlayıcı	Linker	Nesne dosyalarını birleştirerek çalıştırılabilir dosya üreten araçtır.
Bağlı liste	Linked list	Düğümlerin bağlantı işaretçileriyle zincirlendiği listedir.
Başlık dosyası	Header file	Bildirimleri içeren ve `#include` ile eklenen dosyadır.
Bayrak	Flag	Bir bitin açık ya da kapalı durumunu gösteren değerdir.
Bayt	Byte	Sekiz bitten oluşan bellek birimidir.
Bellek sızıntısı	Memory leak	Ayrılan belleğin serbest bırakılmadan kaybolmasıdır.
Bit	Bit	İkilik sistemdeki en küçük veri birimidir.
Bit düzeyi işlem	Bitwise operation	Sayıların bitleri üzerinde yapılan işlemdir.
Blok	Block	Süslü parantezlerle sınırlanan kod parçasıdır.
Boolean	Boolean	Doğru ya da yanlış değerini alabilen mantıksal tiptir.
Boş işaretçi	Null pointer	Hiçbir nesneyi göstermeyen işaretçi değeridir.
Boyut	Size	Bir tipin ya da nesnenin bellekte kapladığı alandır.

E.3 Ç

Türkçe terim	İngilizce	Açıklama
Çağrı yığını	Call stack	Aktif fonksiyon çağrılarının sırasını tutan bellek bölgesidir.
Çalışma anında derleme	Just-In-Time (JIT) compilation	Kodu çalışma anında makine koduna çeviren derleme yöntemidir.
Çalışma zamanı	Runtime	Programın çalıştığı evredir; çalışma sırasında oluşan hatalar bu evreye aittir.
Çekirdek	Kernel	İşletim sisteminin donanımla doğrudan konuşan merkezi bileşenidir.
Çeviri birimi	Translation unit	Önişlemciden geçmiş tek bir kaynak dosyadır.
Çıktı	Output	Programın dışarıya ürettiği veridir.
Çift duyarlıklı	Double precision	Ondalık sayıyı 64 bitle temsil etme biçimidir.
Çoklu kalıtım	Multiple inheritance	Bir sınıfın birden fazla temel sınıftan türemesidir.
Çöp değer	Garbage value	Başlatılmamış belleğin rastgele içeriğidir.
Çöp toplayıcı	Garbage collector	Kullanılmayan nesnelerin belleğini otomatik serbest bırakan bileşendir.

E.4 D

Türkçe terim	İngilizce	Açıklama
Dal	Branch	Bir koşula bağlı olarak izlenen kod yoludur.

Türkçe terim	İngilizce	Açıklama
Değer ile geçirme	Pass by value	Argümanın kopyasının fonksiyona aktarılmasıdır.
Değer tipi	Value type	Verisi doğrudan değişkenin içinde tutulan tiptir.
Değişken	Variable	Bellekte değer tutan ve bir adla anılan alandır.
Değişmez	Literal	Kodda doğrudan yazılan sabit değerdir (ör. `42`).
Demet	Tuple	Farklı tiplerdeki değerleri tek bir yapıda bir arada tutan topluluktur.
Derleme	Compilation	Kaynak kodu makine koduna çevirme işlemidir.
Derleme zamanı	Compile time	Kodun derleyici tarafından işlendiği evredir.
Derleyici	Compiler	Kaynak kodu çalıştırılabilir ya da ara koda çeviren programdır.
Dış bağlantı	External linkage	Bir adın başka çeviri birimlerinden de erişilebilir olmasıdır.
Dinamik bellek	Dynamic memory	Program çalışırken ayrılan ve elle ya da otomatik serbest bırakılan bellektir.
Dinamik dizi	Dynamic array	Boyutu çalışma sırasında değişebilen dizidir.
Dize	String	Karakterlerden oluşan metindir.
Dizi	Array	Aynı tipten elemanları bitişik bellekte tutan veri yapısıdır.
Döngü	Loop	Bir kod parçasını koşul sağlandığı sürece tekrarlar.
Döngü değişkeni	Loop variable	Döngünün tur sayısını ya da ilerlemesini tutan değişkendir.
Dönüş değeri	Return value	Fonksiyonun `return` ile çağırana verdiği değerdir.
Düğüm	Node	Veriyi ve sonraki bağlantıyı tutan yapı birimidir.

E.5 E

Türkçe terim	İngilizce	Açıklama
Eleman	Element	Dizinin ya da koleksiyonun tek bir öğesidir.
Erişim belirleyici	Access modifier	Bir üyenin hangi kodlardan görülebileceğini belirleyen anahtar sözcüktür.
Erişimci	Accessor	Bir özelliğin `get` ya da `set` bloklarıdır.
Etiket	Label	Kodda `goto` ya da `switch` için kullanılan hedef addır.

E.6 F

Türkçe terim	İngilizce	Açıklama
Faktöriyel	Factorial	Bir sayıdan 1'e kadar olan tamsayıların çarpımıdır.
Fonksiyon	Function	Belirli bir işi yapan, adlandırılmış kod bloğudur.
Fonksiyon göstericisi	Function pointer	Bir fonksiyonun adresini tutan işaretçidir.
Format belirteci	Format specifier	`printf` biçim dizesinde değerin nasıl yazılacağını gösteren `%` ile başlayan ifadedir.

E.7 G

Türkçe terim	İngilizce	Açıklama
Genel tip	Generic	Tip parametresi alan ve farklı tiplerle kullanılabilen yapıdır.
Girdi	Input	Programın dışarıdan aldığı veridir.
Girinti	Indentation	Kodun okunabilirliği için satır başına bırakılan boşluktur.
Global değişken	Global variable	Program boyunca her yerden erişilebilen değişkendir.
Gömülü sistem	Embedded system	Belirli bir işi yapmak için donanıma yerleştirilmiş bilgisayar sistemidir.
Güvensiz kod	Unsafe code	C#'ta işaretçi kullanılabilen, derleyici denetiminin azaldığı kod bloğudur.

E.8 H

Türkçe terim	İngilizce	Açıklama
Hata ayıklayıcı	Debugger	Programı adım adım izleyip değişken değerlerini gösteren araçtır.
Heap	Heap	Dinamik bellek ayırmanın yapıldığı bellek bölgesidir.
Hedef dosya	Object file	Derlenmiş ama henüz bağlanmamış dosyadır (`.o`, `.obj`).
Hizalama	Alignment	Verinin bellekte belli bir sınırın katına yerleştirilmesidir.

E.9 İ

Türkçe terim	İngilizce	Açıklama
İçe aktarma	Import	Başka bir ad alanını ya da modülü kullanılabilir hâle getirmektir.
İkilik sayı sistemi	Binary number system	Yalnızca 0 ve 1 rakamlarını kullanan sayı sistemidir.
İlk değer	Initial value	Değişkene tanımlanırken verilen başlangıç değeridir.
İmza	Signature	Bir fonksiyonun adı ve parametre listesinden oluşan kimliktir.
İndeks	Index	Dizi elemanının sıra numarasıdır.
İstisna	Exception	Çalışma sırasında normal akışı bozan olaydır.
İşaretçi	Pointer	Başka bir nesnenin bellek adresini tutan değişkendir.
İşaretli	Signed	Negatif ve pozitif değerleri tutabilen tamsayı niteliğidir.
İşaretsiz	Unsigned	Yalnızca sıfır ve pozitif değerleri tutabilen tamsayı niteliğidir.
İşleç	Operator	`+` ya da `=` gibi işlem yapan simgedir.
İşlemci	Processor (CPU)	Komutları çalıştıran donanım birimidir.
İş parçacığı	Thread	Aynı programın içinde eşzamanlı çalışan yürütme akışıdır.

E.10 K

Türkçe terim	İngilizce	Açıklama
Kaçış dizisi	Escape sequence	`\n` gibi ters eğik çizgiyle başlayan özel karakter gösterimidir.
Kalıtım	Inheritance	Bir sınıfın üyelerinin başka bir sınıftan devralınmasıdır.
Kapsam	Scope	Bir adın geçerli olduğu kod bölgesidir.
Kapsülleme	Encapsulation	Verinin dış erişimden korunup yalnızca metotlarla değiştirilmesidir.
Karakter	Character	Tek bir harf, rakam ya da simgedir.
Kayıt	Record	Değer eşitliği olan, C# 9 ile gelen başvuru tipidir.
Koleksiyon	Collection	Birden çok öğeyi tutan nesnedir.
Komut satırı	Command line	Programın metin tabanlı olarak çalıştırıldığı arayüzdür.
Kontrol akışı	Control flow	Programın komutları hangi sırayla çalıştırdığıdır.
Koşul	Condition	Doğru ya da yanlış değer üreten ifadedir.
Kopya	Copy	Bir nesnenin özdeş bir örneğinin oluşturulmasıdır.
Kullanıcı tanımlı tip	User-defined type	Programcının yapı, sınıf ya da numaralandırma olarak tanımladığı tiptir.
Kurucu	Constructor	Nesne oluşturulurken çalışan özel metottur.
Kuyruk	Queue	İlk giren ilk çıkar mantığıyla çalışan veri yapısıdır.
Kütüphane	Library	Yeniden kullanılmak üzere derlenmiş kod topluluğudur.

E.11 L

Türkçe terim	İngilizce	Açıklama
Lambda ifadesi	Lambda expression	Adsız fonksiyon yazmaya yarayan kısa ifadedir.
LINQ	LINQ	C#'ta koleksiyonları sorgulamak için sunulan dil özelliğidir.
Liste	List	Sıralı öğelerden oluşan topluluktur.

E.12 M

Türkçe terim	İngilizce	Açıklama
Makro	Macro	Önişlemcinin metin olarak yerine koyduğu tanımdır.
Mantıksal işleç	Logical operator	`&&` ve `!` gibi koşulları birleştiren ya da değiştiren işleçtir.
Metot	Method	Bir sınıfın ya da yapının içinde tanımlanan fonksiyondur.
Modül	Module	Bağımsız olarak derlenip kullanılabilen kod bölümüdür.
Mutex	Mutex	Kaynağa aynı anda yalnızca bir iş parçacığının erişmesini sağlayan kilittir.

E.13 N

Türkçe terim	İngilizce	Açıklama
Nesne	Object	Bir tipin somut bir örneğidir.
Nesne yönelimli programlama	Object-oriented programming (OOP)	Verileri ve davranışları nesneler içinde bir araya getiren yaklaşımdır.
Nitelik	Qualifier	Bir tipe ya da değişkene ek anlam kazandıran sözcüktür (ör. `const`).
Null	Null	Hiçbir değer taşımadığını belirten özel değerdir.
Numaralandırma	Enumeration (enum)	Adlandırılmış sabitlerden oluşan tiptir.

E.14 O

Türkçe terim	İngilizce	Açıklama
Olay	Event	Bir durum oluştuğunda tetiklenen bildirimdir.

E.15 Ö

Türkçe terim	İngilizce	Açıklama
Öncelik	Precedence	İfadede hangi işlecin önce uygulanacağını belirleyen sıradır.
Önişlemci	Preprocessor	Derlemeden önce `#include` ve `#define` gibi yönergeleri işleyen araçtır.
Özellik	Property	Bir nesnenin verisine okuyucu ve yazıcı erişimcileriyle erişilmesini sağlayan üyedir.

E.16 P

Türkçe terim	İngilizce	Açıklama
Parametre	Parameter	Fonksiyon tanımında alınacak değeri temsil eden değişkendir.
Polimorfizm	Polymorphism	Aynı arayüzün farklı tiplerde farklı davranmasıdır.
Program	Program	Bilgisayara yapılacak işi adım adım anlatan koddur.
Prototip	Prototype	Fonksiyonun tanımından önce bildirilen imzasıdır.

E.17 R

Türkçe terim	İngilizce	Açıklama
RAII	Resource Acquisition Is Initialization	Kaynağın ömrünü nesnenin ömrüne bağlayan C++ deyimidir.
Rastgele erişim	Random access	Verinin sırayla değil doğrudan konumundan okunmasıdır.
Referans	Reference	Var olan bir nesneye başka bir ad üzerinden erişimdir.

E.18 S

Türkçe terim	İngilizce	Açıklama
Sabit	Constant	Programın çalışması boyunca değişmeyen değerdir.
Saf sanal	Pure virtual	Gövdesi olmayan ve türeyen sınıfta uygulanması zorunlu sanal fonksiyondur.
Sanal	Virtual	Çalışma zamanında hangi sürümün çağrılacağını belirleyen niteliktir.
Sarmalayıcı	Wrapper	Başka bir değeri ya da kaynağı çevreleyerek erişimini kolaylaştıran nesnedir.
Satır içi	Inline	Fonksiyonun çağrı yerine kopyalanmasını öneren belirteçtir.
Sayaç	Counter	Bir olayın kaç kez gerçekleştiğini tutan değişkendir.
Semantik	Semantics	Bir yapının anlamını ve davranışını tanımlayan kurallardır.
Sıralama	Sorting	Elemanları belirli bir ölçüte göre dizme işlemidir.
Sınıf	Class	Veriyi ve davranışı bir arada tanımlayan tiptir.
Sınır	Bound	Bir dizinin ya da aralığın geçerli son noktasıdır.
Sistem çağrısı	System call	Kullanıcı programının işletim sisteminden hizmet istediği arayüzdür.
Sonsuz döngü	Infinite loop	Koşulu hiç yanlış olmayan ve bu yüzden sonlanmayan döngüdür.
Soyut	Abstract	Doğrudan örneklenemeyen ve eksik üye içeren tiptir.
Sözdizimi	Syntax	Dilin yazım kurallarıdır.
Sözlük	Dictionary	Anahtar-değer çiftlerini tutan koleksiyondur.
Standart kütüphane	Standard library	Dilin kendisiyle birlikte gelen hazır kod kümesidir.
Statik	Static	Bir nesneye değil tipe ya da programın tamamına ait olan niteliktir.
Süreç	Process	Çalışan bir programın bellekteki örneğidir.

E.19 Ş

Türkçe terim	İngilizce	Açıklama
Şablon	Template	Tipi parametre olarak alan ve farklı tiplerle kullanılabilen kod kalıbıdır.
Şablon özelleştirme	Template specialization	Belirli bir tip için şablonun özel bir sürümünü yazmaktır.

E.20 T

Türkçe terim	İngilizce	Açıklama
Tampon	Buffer	Veriyi geçici olarak tutan bellek alanıdır.
Tamsayı	Integer	Ondalık kısmı olmayan sayı tipidir.
Tanım	Definition	Bir adın gövdesini ya da bellekteki yerini belirleyen bildirimdir.
Tanımsız davranış	Undefined behavior	Dilin ne yapılacağını tanımlamadığı ve sonucun öngörülemediği durumdur.
Taşıma	Move	Kaynağı kopyalamak yerine yeni nesneye devretme işlemidir (C++11).
Taşma	Overflow	Bir değerin tipinin taşıyabileceği aralığı aşmasıdır.

Türkçe terim	İngilizce	Açıklama
Temsilci	Delegate	Metot başvurusunu tutan tiptir.
Tip	Type	Bir değerin hangi türden olduğunu belirleyen niteliktir.
Tip çıkarımı	Type inference	Tipin başlatıcıdan derleyici tarafından belirlenmesidir.
Tip güvenliği	Type safety	Yanlış tipteki verinin yanlışlıkla kullanılmasını engelleme özelliğidir.
Tip takma adı	Typedef	Mevcut bir tipe başka bir ad vermektir.

E.21 U

Türkçe terim	İngilizce	Açıklama
Unicode	Unicode	Dünyadaki yazı sistemlerinin karakterlerine numara veren standarttır.
UTF-8	UTF-8	Unicode karakterlerini bir ila dört bayta kodlayan yöntemdir.
Uyarı	Warning	Derlemeyi durdurmayan, ama bir yanlışa işaret eden derleyici mesajıdır.

E.22 Ü

Türkçe terim	İngilizce	Açıklama
Üst sınıf	Base class	Kalıtımda kendisinden başka sınıfın türetildiği sınıftır.
Üye	Member	Bir sınıfın ya da yapının parçası olan alan ya da metottur.

E.23 V

Türkçe terim	İngilizce	Açıklama
Varsayılan	Default	Belirtilmediğinde kullanılan öntanımlı değerdir.
Vektör	Vector	Boyutu değişebilen dinamik diziyi temsil eden kapsayıcıdır.
Veri tipi	Data type	Bir verinin türünü ve alabileceği değerleri belirleyen tanımdır.
Veri yapısı	Data structure	Verinin bellekte düzenli biçimde tutulma yöntemidir.
Vtable	Virtual table	Sanal fonksiyonların adreslerini tutan gizli tablodur.

E.24 Y

Türkçe terim	İngilizce	Açıklama
Yan etki	Side effect	Bir ifadenin değerinin dışında programın durumunda yaptığı değişikliktir.
Yapı	Struct	Birden çok alanı bir arada tutan bileşik tiptir.
Yerel değişken	Local variable	Yalnızca tanımlandığı blok ya da fonksiyon içinde geçerli değişkendir.
Yığın	Stack	Fonksiyon çağrılarının ve yerel değişkenlerin tutulduğu bellek bölgesidir.
Yıkıcı	Destructor	Nesne yok edilirken çalışan özel metottur.

Türkçe terim	İngilizce	Açıklama
Yineleyici	Iterator	Bir kapsayıcının elemanları üzerinde dolaşmayı sağlayan nesnedir.
Yorum	Comment	Derleyici tarafından yok sayılan, okura not bırakan metindir.
Yönetilen kod	Managed code	Çalışma zamanı ortamı tarafından denetlenen koddur (ör. .NET).
Yönetilmeyen kod	Unmanaged code	Bellek yönetimini doğrudan programcının yaptığı koddur.

E.25 Z

Türkçe terim	İngilizce	Açıklama
Zaman karmaşıklığı	Time complexity	Bir algoritmanın girdi büyüdükçe harcadığı süreyi gösteren ölçüdür.
Zincirleme	Chaining	Bir işlemin sonucunun doğrudan bir sonrakine verilmesidir.

E K F

Alıştırmaların Cevapları

Bölüm 1 Bilgisayar, Program ve Programlama Dilleri

Soru 1.

Cevap: C. RAM çalışan programı ve verisini geçici tutar; hızlıdır ve elektrik kesilince içeriği silinir. A depolamanın, D işlemcinin özelliğidir. RAM depolamadan daha hızlıdır (B yanlış).

Soru 2.

Cevap: D. 8 bit $2^8 = 256$ farklı değer taşır (0 … 255 ya da işaretli yorumda -128 … 127).

Soru 3.

a) 110101 = 32 + 16 + 4 + 1 = **53**.

b) 37 = 32 + 4 + 1 olduğundan ikilik karşılığı **100101**'dir. (37 / 2 = 18 k1, 18 / 2 = 9 k0, 9 / 2 = 4 k1, 4 / 2 = 2 k0, 2 / 2 = 1 k0, 1 / 2 = 0 k1; kalanlar sondan başa: 100101.)

c) 2F = 2 × 16 + 15 = **47**.

d) 1010 = A, 1110 = E; sonuç **AE**.

e) 200 / 16 = 12 kalan 8; 12 = C olduğundan onaltılık **C8**. İkilik karşılığı C = 1100 ve 8 = 1000 olduğundan **11001000**'dir. Doğrulama: 128 + 64 + 8 = 200.

Soru 4.

Cevap: C. Derleyici çeviriyi çalışmadan önce yapar ve çalıştırılabilir dosya üretir; bu dosya derleyici olmayan bir bilgisayarda da çalışır. A ve B tanımları ters yazılmıştır, C bir yorumlayıcıyla çalışmaz, yorumlayıcı da hataları (sıra o satıra gelince) bildirir.

Soru 5.

Cevap: B. C# derleyicisi kaynak kodu IL'e çevirir; çalışma sırasında CLR içindeki JIT derleyicisi IL'i işlemcinin makine koduna çevirir.

Soru 6.

Cevap:

```
5 2
```

İzleme: `x` = 2, `y` = 5; `x ← 7`; `y ← 7 - 5 = 2`; `x ← 7 - 2 = 5`. Kod, üçüncü bir değişken kullanmadan iki değişkenin değerini takas eder.

Soru 7.

Cevap: 20. Döngü `sayac` 2, 4, 6, 8 değerleri için çalışır: 2 + 4 + 6 + 8 = 20. `sayac` 10 olunca koşul bozulur.

Soru 8.

Cevap: Hata `enBuyuk ← 0` başlangıç değeridir. Tüm sayılar negatif olduğunda hiçbiri 0'dan büyük olmaz ve algoritma listede bulunmayan 0'ı yazdırır (girdi -5, -2, -9 için sonuç 0 çıkar, doğrusu -2'dir). Düzeltme: başlangıç değeri listenin ilk elemanı olmalıdır.

```
OKU n, sayilar[1..n]
enBuyuk ← sayilar[1]
i ← 2
i <= n OLDUKÇA TEKRARLA
    EĞER sayilar[i] > enBuyuk İSE
        enBuyuk ← sayilar[i]
    i ← i + 1
YAZ enBuyuk
```

Soru 9.

Cevap: a) sözdizimi hatası (derleyici bildirir); b) çalışma zamanı hatası (hata ancak program çalışırken, değer girilince ortaya çıkar); c) mantık hatası (program çalışır ama formül yanlıştır; alan için yarıçapın karesi gerekir).

Soru 10.

Cevap: C. C sınıf içermez ve ağırlıklı olarak yordamsaldır. C++ yordamsal, nesne yönelimli, genel ve fonksiyonel öğeleri bir arada destekler. C# ağırlıklı nesne yönelimlidir. Yordamsal programlama fonksiyonlara dayanır.

Soru 11.

Örnek çözüm:

```
BAŞLA
  OKU sayi
  EĞER sayi'nin 2'ye bölümünden kalan 0 İSE
      YAZ "çift"
  DEĞİLSE
      YAZ "tek"
BİTİR
```

Bir sayı 2'ye bölündüğünde kalan 0 ise çifttir, 1 ise tektir. (Negatif sayılar için de işlem aynıdır; yalnızca kalanın 0 olup olmadığına bakıyoruz.)

Soru 12.

Örnek çözüm:

```
BAŞLA
  OKU n
  OKU sayi
  enKucuk ← sayi
  i ← 2
  i <= n OLDUKÇA TEKRARLA
      OKU sayi
      EĞER sayi < enKucuk İSE
          enKucuk ← sayi
      i ← i + 1
  YAZ enKucuk
BİTİR
```

Girdi 5, 2, 8, 2 (n = 4) için izleme:

i	okunan sayi	enKucuk (sonra)
(başlangıç)	5	5
2	2	2
3	8	2
4	2	2

Sonuç 2'dir. İlk sayı okunduktan sonra `enKucuk` ona eşitlenir; döngü yalnızca kalan sayıları denetler.

Soru 13.

Örnek çözüm:

```
BAŞLA
  toplam ← 0
  sayi ← 2
  sayi <= 100 OLDUKÇA TEKRARLA
      toplam ← toplam + sayi
      sayi ← sayi + 2
  YAZ toplam
BİTİR
```

Çift sayılar 2, 4, …, 100 olmak üzere 50 tanedir. Toplam 2 × (1 + 2 + … + 50) = 2 × 1275 = **2550** olmalıdır.

Bölüm 2 C Ailesinin Tarihi: Bell Labs'tan .NET'e

Soru 1.

Cevap: B. C dilini Dennis Ritchie, Bell Labs'ta 1971–72 yıllarında geliştirdi. Ken Thompson B dilinin, Martin Richards BCPL'in, Stroustrup C++'ın, Hejlsberg ise C#'ın tasarımcısıdır.

Soru 2.

Cevap: B. Soy zinciri CPL → BCPL → B → C biçimindedir; her dil bir öncekinin sadeleştirilmiş ya da uyarlanmış hâlidir.

Soru 3.

B tipsiz bir dildi: her şey tek boyutlu bir makine sözcüğü olarak ele alınıyordu. PDP-11 ise bayt adreslenebilir bir bellek ve kayan noktalı sayı desteği sunuyordu; tek bir bayttan oluşan karakterleri, sözcük boyutlu tamsayıları ve kayan noktalı sayıları aynı biçimde işlemek verimsiz ve yetersizdi. C, B'ye `char`, `int` gibi veri tipleri ekleyerek bu sorunu çözdü.

Soru 4.

Cevap: C. İşletim sisteminin büyük bölümü assembly yerine C ile yazıldığında, sistemi başka bir makineye taşımak için yalnızca derleyiciyi ve donanıma bağlı küçük bir kısmı uyarlamak yeterli oldu.

Soru 5.

`a` K&R tarzı, `b` ANSI tarzı tanımdır. K&R tarzında parametre tipleri parantezin dışında, ayrı bir satırda bildirilir; ANSI tarzında ise tipler parametre listesinin içindedir. İkinci biçimde (`b`) derleyici fonksiyonun kaç parametre aldığını ve tiplerini bilir, bu yüzden hatalı sayıda ya da tipte argümanla yapılan çağrıyı derleme sırasında yakalayabilir. K&R biçiminde böyle bir denetim yapılamaz.

Soru 6.

Derleme hata verir: `error: C++ style comments are not allowed in ISO C90`. `//` ile başlayan satır yorumları C99'da standarda girmiştir; C89/C90'da yalnızca `/* ... */` biçimi geçerlidir. Program `-std=c99` ya da `-std=c17` ile derlenseydi sorunsuz çalışırdı.

Soru 7.

Cevap: C. C17, C11'in hata düzeltmelerinden ibarettir ve yeni dil özelliği getirmez (yayım yılı nedeniyle C18 de denir).

Soru 8.

Simula dilinden **sınıf** (ve nesne) kavramını C'ye aktarmayı hedefliyordu; amaç Simula'nın düzenini C'nin hızı ve donanıma yakınlığıyla birleştirmekti. Dilin adı 1983'te C++ oldu (ilk ticari sürüm ve kitap 1985'te geldi).

Soru 9.

Cevap: E. `std::print` C++23 ile gelmiştir. `auto`, aralık tabanlı `for`, lambda ifadeleri ve `nullptr` C++11 yenilikleridir.

Soru 10.

Çıktı:

```
ÇIKTI
11 10
```

`x++` sonek biçimindedir: ifadenin değeri artırmadan önceki değerdir (10), `y`'ye o atanır; ardından `x` 11 olur. `printf` önce `x`'i (11), sonra `y`'yi (10) yazar.

Soru 11.

C# 1.0, 2002'de **.NET Framework 1.0** ile birlikte yayımlandı. Dilin başmimarı Anders Hejlsberg'dir (Microsoft; daha önce Turbo Pascal ve Delphi ile tanınmıştı). Dil 2000'de duyurulmuştu.

Soru 12.

.NET Framework yalnızca Windows'ta çalışır ve kapalı kaynaklıdır; .NET Core ise açık kaynaklıdır ve Windows, Linux ve macOS'ta çalışır (çapraz platform). Bu iki kol **.NET 5** (2020) ile tek bir .NET çatısı altında birleşmiştir.

Soru 13.

Cevap: B. Jenerik koleksiyonlarda eleman tipi derleme sırasında bilinir; bu yüzden `object`'ten elle dönüştürme gerekmez ve yanlış tipte eleman eklemeye çalışmak derleme hatası olur.

Soru 14.

Birinci yorum: Müzikte **diyez** (sharp), bir notayı yarım ton yükselten işarettir; "C sharp" bu yüzden "do diyez"dir ve C++'taki "bir artır" fikrine uygun bir yükseliş çağrışımı yapar. İkinci yorum: `#` işareti iki satır iki sütunluk bir `+` ızgarasına benzer, yani "C++'ın C++'ı" gibi okunabilir. Resmî açıklama müzikteki diyez üzerinedir.

Soru 15.

(a) C: Çekirdek ve sürücü yazılımı donanıma doğrudan erişim ve küçük, öngörülebilir kod ister; C'nin ana alanıdır. (b) C#: Unity oyun betikleri C# ile yazılır. (c) C++: Yüksek başarım, bellek üzerinde denetim ve zengin soyutlama birlikte gerektiği için oyun motorlarında yaygındır. (d) C#: ASP.NET Core ile kurumsal web servisleri yazmak için olgun bir araç takımı sunar; çöp toplayıcı üretkenliği artırır.

Soru 16.

Örnek cevap: C'de bellek elle yönetilir: `malloc` ile ayrılır, `free` ile serbest bırakılır; unutulursa sızıntı oluşur. C++ aynı elle yönetimi sunar, ama akıllı işaretçiler ve kapların nesne ömrüne bağlı otomatik temizliği (RAII) sayesinde çoğu durumda elle `free` ya da `delete` yazmaya gerek bırakmaz. C#'ta ise bellek çöp toplayıcı tarafından yönetilir; programcı nesneyi serbest bırakmaz, kullanılmayan nesneleri çalışma ortamı toplar. Bu fark C'ye en büyük denetimi, C#'a en büyük güvenliği verir.

Bölüm 3 Geliştirme Ortamının Kurulumu

Soru 1.

Cevap: B. MSYS2'nin derleyici paketini kurduğumuz ve gcc'yi çalıştırdığımız ortam UCRT64'tür; bu ortamın araçları `C:\msys64\ucrt64\bin` klasöründe bulunur. "MSYS2 MSYS" ortamı MSYS2'nin kendi araçları içindir. Diğer şıklardaki klasörlerde gcc yoktur.

Soru 2.

Cevap: C. PATH değişikliği yalnızca sonradan başlatılan programlara aktarılır. Açık duran terminali ve VS Code'u kapatıp yeniden açmak çoğu zaman sorunu çözer. Gerekirse ikinci adım olarak `C:\msys64\ucrt64\bin` klasöründe `gcc.exe` bulunduğu doğrulanır.

Soru 3.

Cevap: B. macOS'ta `gcc` ve `g++` komutları Apple'ın clang derleyicisini çalıştırır. Kurulum bozuk değildir ve kitabın programları bu derleyiciyle de derlenir; yalnızca uyarı ve hata mesajlarının sözcükleri gcc'den biraz farklı olabilir.

Soru 4.

Cevap: C. Runtime, derlenmiş bir .NET programını çalıştırmaya yeter. SDK ise bunun yanında C# derleyicisini, `dotnet` komutunu ve geliştirme araçlarını içerir. Program yazmak için SDK gerekir.

Soru 5.

Cevap: İki dosya oluşur. `Program.cs` kaynak kodu içerir; `Merhaba.csproj` ise projenin ayarlarını (hedef .NET sürümü, `ImplicitUsings`, `Nullable` gibi) içeren XML dosyasıdır. `dotnet build` ya da `dotnet run` çalıştırılınca ayrıca `bin` ve `obj` klasörleri oluşur.

Soru 6.

Cevap: `g++` C++ derleyicisini çalıştırır. `-std=c++17` C++17 standardını seçer. `-Wall -Wextra` önerilen uyarıların çoğunu açar. `hesap.cpp` derlenecek kaynak dosyadır. `-o hesap` çıktı dosyasının adını `hesap` yapar (Windows'ta `hesap.exe` oluşur).

Soru 7.

Cevap: `.\merhaba` (ya da `.\merhaba.exe`). PowerShell, güvenlik nedeniyle geçerli klasörü komut aramasına kendiliğinden katmaz; geçerli klasördeki programı çalıştırmak için başına `.\` yazılmalıdır. Komut İstemi'nde (cmd) yalnızca `merhaba` yeterlidir.

Soru 8.

Cevap: Komut, içinde `.csproj` dosyası bulunmayan bir klasörde çalıştırılmıştır; büyük olasılıkla proje klasörünün bir üstündedir. `cd` ile `.csproj` dosyasının bulunduğu klasöre girip `dotnet run` komutu yeniden çalıştırılır.

Soru 9.

Cevap:

```c
#include <stdio.h>

int main(void) {
    printf("Selam, Dünya!\n");
    return 0;
}
```

```
TERMİNAL
```

```
$ gcc -std=c17 -Wall -Wextra merhaba.c -o merhaba
$ ./merhaba
Selam, Dünya!
```

Önce derleme komutu çalıştırılır; hata yoksa komut sessizce biter ve `merhaba` programı oluşur. Ardından `./merhaba` ile çalıştırılır.

Soru 10.

Cevap:

```
TERMİNAL
$ dotnet new console -o Selam
$ cd Selam
```

`Program.cs` dosyasının içeriği şu tek satırla değiştirilir:

```
Console.WriteLine("Selam, Dünya!");
```

```
TERMİNAL
$ dotnet run
Selam, Dünya!
```

Proje klasörü `dotnet new` ile oluşturulur, `cd` ile içine girilir; `dotnet run` komutu da proje klasöründe çalıştırılır.

Bölüm 4 C'ye Giriş: İlk Program ve Derleme Süreci

Soru 1.

Cevap: B. `#` ile başlayan satırları önişlemci işler; `#include` satırı, başlık dosyasının içeriğiyle değiştirilir. Bu, derlemenin ilk aşamasıdır.

Soru 2.

Cevap: D. `-S` seçeneği gcc'yi assembly kodu üretildikten sonra durdurur ve `.s` dosyası oluşturur. `-E` yalnızca önişler, `-c` nesne dosyasına kadar gider, `-o` çıktı dosyasının adını belirler, `-Wall` uyarıları açar.

Soru 3.

Cevap: C. Uyarı veren kod derlenir ve çalışır, ama uyarı çoğu zaman gerçek bir hatanın işaretidir. Hata veren kod hiç derlenmez; çalıştırılabilir dosya oluşmaz. `-Wall` yalnızca uyarıları açar, kodu hızlandırmaz.

Soru 4.

Cevap: C. Ters bölü karakterini yazdırmak için `\\` yazılır. A seçeneğinde `\t` sekme ve `\n` yeni satır olarak yorumlanır. B seçeneğinde eğik çizgiler yanlış yönde ve `/n` alt satıra geçirmez. D seçeneği tek tırnak kullandığı için metin sabiti değildir. E seçeneğinde `\"` çift tırnak yazdırır.

Soru 5.

Cevap:

```
ÇIKTI
Bir
İki

ÜçDört
```

`"İki\n\n"` iki satır sonu içerir: biri `İki`'den sonra satırı bitirir, diğeri boş bir satır bırakır. `"Üç"` satır sonu içermediği için `Dört` aynı satırda devam eder.

Soru 6.

Cevap:

```
ÇIKTI
ad
%e
```

İlk satırda `//` işaretinden sonrası yorumdur; yalnızca `printf("a");` çalışır ve `a` yazılır. İkinci satırda `/* ... */` yorumu `printf("c");` çağrısını devre dışı bırakır, ardından gelen `printf("d\n");` çalışır. Bu yüzden ilk iki yazdırma aynı satırda `ad` olur. `%%` ekrana bir `%` yazdığından üçüncü satır `%e` çıkar.

Soru 7.

Cevap: C. `echo $?` son çalıştırılan programın çıkış kodunu yazar, yani `main`'in döndürdüğü `7`. `Tamam` yazısı programın çıktısıdır, çıkış kodu değildir.

Soru 8.

Cevap: C. Bağlama hatalarının işareti `undefined reference` ifadesidir. Eksik noktalı virgül derleme hatasıdır, kullanılmayan değişken bir uyarıdır, yanlış sıra bir mantık hatası, sıfıra bölme bir çalışma zamanı hatasıdır.

Soru 9.

Cevap:

a) Derleme hatası (`expected ';'`).

b) Bağlama hatası. Dosya derlenir, ama bağlayıcı `main` fonksiyonunu bulamaz (`undefined reference to 'main'`).

c) Mantık hatası. Program hatasız çalışır, ama sonuç beklenenden farklıdır.

d) Çalışma zamanı hatası.

e) Derleme hatası (gcc 14 ve sonrasında; gcc 13 ve öncesinde uyarıdır ve program yine de derlenebilir). Yani `printf` bildirimi olmadan tanınmaz; çözüm `#include <stdio.h>` eklemektir.

Soru 10.

Cevap: B. C, programcının ne yaptığını bildiğini varsayar. Dizi sınırı denetimi, kullanılmayan belleğin temizlenmesi gibi işlerin çoğu programcıya bırakılır. Bu hem hız ve esneklik sağlar hem de dikkat gerektirir.

Soru 11.

Cevap: D. `-c` seçeneği önişleme, derleme ve çevirme aşamalarını yapar; sonuçta `merhaba.o` nesne dosyası oluşur. Bağlama yapılmadığı için çalıştırılabilir dosya yoktur.

Soru 12.

Cevap: Dört sorun vardır.

1. `printf("Birinci\n")` satırının sonunda `;` yok. Derleme hatası; gcc `expected ';' before 'prinft'` der.
2. `prinft` yanlış yazılmış; doğrusu `printf`. gcc 13 yalnızca uyarı verir ve hatayı bağlamada `undefined reference to 'prinft'` olarak bildirir; gcc 14 ve sonrasında doğrudan hata olur.
3. `return 0` satırının sonunda `;` yok. Derleme hatası; gcc `expected ';' before '}' token` der.
4. `Main` yerine `main` yazılmalı. Bağlama hatası; bağlayıcı `undefined reference to 'main'` der. Bu hata ancak derleme hataları giderildikten sonra görünür.

Düzeltilmiş program:

```c
#include <stdio.h>

int main(void) {
    printf("Birinci\n");
    printf("İkinci\n");
    return 0;
}
```

Soru 13.

Cevap:

```
ÇIKTI
A\nB
%d
"Merhaba"
```

`\\n` ters bölüyü ve `n` harfini ayrı ayrı yazdırır; yeni satır oluşmaz. `%%d` ekrana `%d` yazar. `\"` çift tırnak karakterini yazdırır.

Soru 14.

Cevap:

```c
#include <stdio.h>

int main(void) {
    printf("Kitap: C Dillerinin Tarihi ve Öğrenimi\n");
    printf("\"Merhaba\" dedi ve %%100 emin oldu.\n");
    printf("Dosya: C:\\kitap\\ornek.c\n");
    return 0;
}
```

Çift tırnak için `\"`, yüzde için `%%`, ters bölü için `\\` kullanılır. `%%100` yerine `%100` yazılsaydı `printf` bunu geçersiz bir biçim belirteci sanırdı.

Soru 15.

Cevap:

```c
#include <stdio.h>

int main(void) {
    printf("*\n");
    printf("**\n");
    printf("***\n");
    printf("****\n");
    return 0;
}
```

Çıktı:

```
*
**
***
****
```

Her satırın sonunda imleci alt satıra geçirmek için `\n` gerekir. (Döngüleri öğrendiğimizde bu iş Bölüm 8'de çok daha kısa yazılacak.)

Soru 16.

Cevap:

```
int main(void) {
    return 42;
}
```

Program hiçbir şey yazdırmadığı için `#include <stdio.h>` gerekmez. Derleyip çalıştırdıktan hemen sonra çıkış kodunu Linux ve macOS'ta `echo $?`, Windows PowerShell'de `$LASTEXITCODE` (cmd'de `echo %ERRORLEVEL%`) komutuyla görürüz; ekrana `42` gelir.

Bölüm 5 Değişkenler, Veri Tipleri ve Operatörler

Soru 1.

Cevap: B. `a` ve `b` ikisi de `int` olduğu için `17 / 5` tamsayı bölmesidir ve 3 verir; bu değer `c`'ye atanırken 3.0 olur. Bölme atamadan önce yapıldığı için `c`'nin `double` olması sonucu değiştirmez.

Soru 2.

Cevap: E. Bölme sıfıra doğru kesilir (`-17 / 5` = -3), kalan `-17 - (-3 * 5)` = -2 olur. Kalanın işareti bölünenin işaretidir.

Soru 3.

Cevap: C. `_toplam2` harf, rakam ve alt çizgiden oluşur ve rakamla başlamaz. A rakamla başlıyor, B `-` içeriyor, D anahtar sözcük, E boşluk içeriyor.

Soru 4.

Cevap: C. C'de `'A'` gibi bir karakter sabiti `char` değil `int` tipindedir; bu yüzden `sizeof('A')`, `sizeof(int)` ile aynıdır ve 4 çıkar. (C++'ta aynı ifade 1 verir.)

Soru 5.

Cevap: B. Son ek `x++` ifadede eski değeri (5) verir, sonra `x` 6 olur. `y = 5 + 2 = 7`.

Soru 6.

Cevap: B. Bilgisayar ondalıklı sayıları ikilik tabanda saklar; 0,1 ve 0,2 bu tabanda sonlu basamakla yazılamaz, yaklaşık değerle tutulur. Toplam 0,30000000000000004... olur ve 0,3'ün yaklaşık değerine tam eşit değildir. Bu yüzden ondalıklı sayılar farkın küçüklüğüyle karşılaştırılır.

Soru 7.

Cevap:

```
ÇIKTI
2.0 2.5 2 -2
```

`a / b` tamsayı bölmesi olduğu için 2'dir ve `r` 2.0 olur. `(double)a / b` ondalıklı bölmedir: 2.5. `10 % 4` = 2. `-a / b` ifadesinde tekli eksi önce uygulanır: `-10 / 4` = -2 (sıfıra doğru kesme).

Soru 8.

Cevap:

```
ÇIKTI
0 1 5
```

`x != 0` yanlış olduğu için `&&` kısa devre yapar ve `++y` hiç çalışmaz; `r` = 0. `x == 0` doğru olduğu için `||` kısa devre yapar ve ikinci `++y` de çalışmaz; `s` = 1. `y` hiç artmadığından 5 kalır.

Soru 9.

Cevap:

```
ÇIKTI
c 99 A
```

`c + 2` bir `int`'tir ve değeri 99'dur; `%c` bunu `'c'` karakteri olarak, `%d` sayı olarak yazdırır. `c - 32` = 65 ve `%c` ile `'A'` olur.

Soru 10.

Cevap:

```
ÇIKTI
0 1
```

`k + 1` ifadesi `int` olarak 256 hesaplanır; `unsigned char`'a atanırken 256 mod 256 = 0 olur. `u - 5` işaretsiz aritmetiktir: 3 - 5 işlemi sıfırın altına inemez ve 4294967294 üretir; bu sıfırdan büyük olduğu için karşılaştırma 1 verir.

Soru 11.

Cevap:

```
ÇIKTI
4 10 6
```

`++a` önce `a`'yı 5 yapar; `b = 5 * 2 = 10`. Sonra `a-- + 1` ifadesi eski değer 5'i kullanır: `c = 5 + 1 = 6` ve ardından `a` 4 olur. Her satırda `a` yalnızca bir kez değiştiği için davranış tanımlıdır.

Soru 12.

Cevap: Üç hata vardır.

1. `int 2adet = 3;` ve `printf` içindeki `2adet`: ad rakamla başlayamaz. Düzeltme: `adet2` gibi bir ad.
2. `double oran = 7.5 % 2;`: `%` operatörü ondalıklı sayılarla kullanılamaz. Düzeltme: `<math.h>` eklenip `fmod(7.5, 2)` kullanmak (Bölüm 9) ya da tamsayı işlenenlerle çalışmak.
3. `N = N + 1;`: `N`, `const` ile tanımlandığı için salt okunurdur. Düzeltme: sabite atama yapmamak ya da değişmesi gerekiyorsa `const` kaldırıp `int n = 5;` yazmak.

Soru 13.

Cevap: `a + b` toplamı 15'tir ve `15 / 2` iki tamsayı olduğu için tamsayı bölmesi yapılır, sonuç 7'dir. Bu 7 sonradan `double`'a çevrilir (7.0); kesilen 0,5 geri gelmez. Düzeltmeler:

```c
double ort = (a + b) / 2.0;          // bölen ondalıklı sabit
double ort2 = (double)(a + b) / 2;   // toplam bölmeden önce double'a çevrilir
```

Her ikisi de 7.5 üretir. `(double)((a + b) / 2)` yazmak çözüm değildir, çünkü cast bölmeden sonra uygulanır.

Soru 14.

Cevap:

```c
#include <stdio.h>

int main(void) {
    const double PI = 3.14159;
    double yaricap = 5.0;

    double alan = PI * yaricap * yaricap;
    double cevre = 2 * PI * yaricap;

    printf("Alan: %.2f\n", alan);
    printf("Çevre: %.2f\n", cevre);
    return 0;
}
```

Çıktı: `Alan: 78.54` ve `Çevre: 31.42`. Yarıçap `double` tutulduğu için tüm çarpmalar ondalıklı yapılır.

Soru 15.

Cevap:

```c
#include <stdio.h>

int main(void) {
    int sayi = 472;

    int yuzler = sayi / 100;
    int onlar = sayi / 10 % 10;
    int birler = sayi % 10;

    int toplam = yuzler + onlar + birler;
    int ters = birler * 100 + onlar * 10 + yuzler;

    printf("Yüzler: %d, Onlar: %d, Birler: %d\n", yuzler, onlar, birler);
    printf("Basamak toplamı: %d\n", toplam);
    printf("Ters sayı: %d\n", ters);
    return 0;
}
```

`sayi / 100` yüzler basamağını, `sayi / 10 % 10` onlar basamağını (önce son basamak atılır, sonra kalan alınır), `sayi % 10` birler basamağını verir. 472 için çıktı: yüzler 4, onlar 7, birler 2; toplam 13; ters 274.

Soru 16.

Cevap:

```c
#include <stdio.h>

int main(void) {
    int tutar = 786;

    printf("200 TL: %d\n", tutar / 200);
    tutar %= 200;
    printf("100 TL: %d\n", tutar / 100);
    tutar %= 100;
    printf("50 TL: %d\n", tutar / 50);
    tutar %= 50;
    printf("20 TL: %d\n", tutar / 20);
    tutar %= 20;
```

```
    printf("10 TL: %d\n", tutar / 10);
    tutar %= 10;
    printf("5 TL: %d\n", tutar / 5);
    tutar %= 5;
    printf("1 TL: %d\n", tutar);
    return 0;
}
```

Her adımda bölme o banknottan kaç tane gerektiğini, `%=` ise geriye kalan tutarı verir. 786 TL için sonuç: 3 adet 200, 1 adet 100, 1 adet 50, 1 adet 20, 1 adet 10, 1 adet 5 ve 1 adet 1 TL'lik parçadır. (Tekrarlı yapıları Bölüm 8'de döngüyle kısaltacağız.)

Bölüm 6 Klavyeden Girdi ve Biçimli Çıktı

Soru 1.

Cevap: B. `%6.2f` toplam 6 karakter genişliğinde, 2 ondalıklı yazar. `3.14` dört karakterdir, soluna iki boşluk eklenir: `[ 3.14]`.

Soru 2.

Cevap: C. `scanf`, `double` için `%lf` ister (`float` için `%f`). `printf`'te ise `double` için `%f` yazılır.

Soru 3.

Cevap: B. `7` başarıyla okunur, `abc` yüzünden ikinci okuma başarısız olur. Dönüş değeri başarıyla okunan öğe sayısıdır: 1. `a` değişir, `b` 0 olarak kalır.

Soru 4.

Cevap:

```
ÇIKTI
[42   ][00042][+42]
```

`%-5d` sola yaslı ve 5 genişliğinde, `%05d` sıfırla doldurulmuş, `%+d` işaretli yazar.

Soru 5.

Cevap: C. `%d` sayıyı okuyunca durur; Enter'ın bıraktığı `'\n'` arabellekte kalır. `%c` hiçbir karakteri atlamadığı için bu `'\n'`'i okur. Çözüm: `scanf(" %c", &harf);`.

Soru 6.

Cevap: `&` işleci unutulmuş. `scanf` değişkenin adresini ister; `sayi` değişkeninin değeri (0) adres sanılır ve tanımsız davranış oluşur. Doğrusu: `scanf("%d", &sayi);`

Soru 7.

Cevap: `double` değişken `%f` ile okunuyor. `%f` 4 baytlık `float` bekler; `boy` 8 baytlık `double` olduğu için yalnızca 4 baytı yazılır ve değer bozulur. Doğrusu: `scanf("%lf", &boy);`

Soru 8.

Cevap:

```
ÇIKTI
ff FF 377 0xff
```

255 onaltılıkta `ff`, sekizlikte `377`'dir. `#` bayrağı onaltılık sayının önüne `0x` ekler.

Soru 9.

Cevap: B. `fflush(stdin)` standartta tanımsızdır, `gets` kaldırılmıştır, diğer seçeneklerde böyle bir kalıp yoktur. Karakterleri tek tek okuyup atan `while` kalıbı standarda uygundur.

Soru 10.

Cevap:

```
ÇIKTI
  7.9|ab      |12345
```

`%5.1f` 7.86'yı 7.9'a yuvarlar ve 5 genişliğine sağa yaslar (2 boşluk). `%-8s` metni sola yaslayıp 6 boşlukla 8'e tamamlar. `%3d` genişliği `12345` için yetersizdir, ama sayı kesilmez.

Soru 11.

Cevap: B. `fgets` satır sonundaki `'\n'` karakterini de diziye koyar, ardından `'\0'` ekler. Bu yüzden `satir` içeriği `Ece\n\0` olur.

Soru 12.

Cevap:

```c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0;
    printf("İki tamsayı: ");
    if (scanf("%d %d", &a, &b) != 2) {
        printf("Geçersiz girdi.\n");
        return 1;
    }
    printf("%-10s %10d\n", "Toplam", a + b);
    printf("%-10s %10d\n", "Fark", a - b);
    printf("%-10s %10d\n", "Carpim", a * b);
    if (b != 0) {
        printf("%-10s %10.2f\n", "Bolum", (double)a / b);
    } else {
        printf("%-10s %10s\n", "Bolum", "tanımsız");
    }
    return 0;
}
```

Bölümde en az bir işlenenin `double` olması gerektiği için `(double)a / b` yazdık. Etiketleri bayt sayımı yüzünden hizayı bozmasın diye ASCII yazdık. Burada `if` ile `0` denetimi için kalıbı kullandık (Bölüm 7).

Soru 13.

Cevap:

```c
#include <stdio.h>

int main(void) {
    int toplam = 0;
    printf("Saniye: ");
    if (scanf("%d", &toplam) != 1 || toplam < 0) {
        printf("Geçersiz girdi.\n");
        return 1;
    }
    int saat = toplam / 3600;
    int dakika = toplam % 3600 / 60;
```

```
    int saniye = toplam % 60;
    printf("%02d:%02d:%02d\n", saat, dakika, saniye);
    return 0;
}
```

Saat tamsayı bölmesiyle, dakika ve saniye kalan işleciyle bulunur. `%02d`, tek haneli değerlerin başına sıfır koyar.

Soru 14.

Cevap:

```
#include <stdio.h>

int main(void) {
    char satir[32];
    int not1 = 0;
    int tamam = 0;

    while (!tamam) {
        printf("Not (1-10): ");
        if (fgets(satir, sizeof satir, stdin) == NULL) {
            return 1;
        }
        if (sscanf(satir, "%d", &not1) == 1 && not1 >= 1 && not1 <= 10) {
            tamam = 1;
        } else {
            printf("Geçersiz not.\n");
        }
    }
    printf("Not: %d\n", not1);
    return 0;
}
```

Satır `fgets` ile alınır, `sscanf` ile sayıya çevrilir; hem okuma başarısı hem aralık denetlenir. Geçerli değer gelene kadar döngü sürer.

Bölüm 7 Koşullu İfadeler

Soru 1.

Cevap: C. Uç değerler dışarıda olduğu için kesin eşitsizlikler gerekir ve iki koşul `&&` ile bağlanır. A her zaman doğrudur (`10 < x` sonucu 0 ya da 1, bu da 20'den küçüktür). B her `x` için doğrudur. D uç değerleri de kapsar. E üst ucu kapsar.

Soru 2.

Cevap: C. `x != 0 && ...` ifadesinde `x` sıfırsa sol taraf yanlıştır ve `&&` sağ tarafı hiç hesaplamaz. A ve E'de bölme en solda, ilk işlenen olarak yapılır. B'de `x == 0` doğru olduğu için `&&` sağ tarafa geçer. D'de sol taraf yanlış olduğundan `||` sağ tarafı hesaplar. Yani A, B, D ve E'de sıfıra bölme gerçekleşir.

Soru 3.

Cevap: D. `case` etiketi derleme zamanında bilinen bir tamsayı sabiti olmalıdır. `'x'` (bir tamsayı değeri), `MAKS + 1` (önişlemci `10 + 1` yapar), `2 * 3` ve `-1` sabit ifadelerdir. `sinir + 1` ise bir değişken içerdiği için sabit değildir; `gcc` `case label does not reduce to an integer constant` hatası verir.

Soru 4.

Cevap: C. Ifade `n > 10 ? (n > 20 ? 3 : 2) : 1` biçiminde gruplanır. `25 > 10` doğru olduğundan iç ifade değerlendirilir; `25 > 20` doğru olduğundan sonuç 3'tür.

Soru 5.

Cevap: B. `x = 7` karşılaştırma değil atamadır: `x`'e 7 atanır ve ifadenin değeri 7, yani doğrudur. `A` yazılır ve `x` artık 7'dir. `gcc -Wall` ile uyarır ama program derlenir.

```
ÇIKTI
A
7
```

Soru 6.

Cevap:

```
ÇIKTI
C
```

`else`, girintisine rağmen en yakın `if`'e, yani `y > 5` koşuluna bağlanır. `x > 3` yanlış (2 > 3) olduğundan iç `if` ve onun `else`'i hiç değerlendirilmez. Yalnızca her zaman çalışan `printf("C\n");` yazılır.

Soru 7.

Cevap:

```
ÇIKTI
0 1 9
```

Adım adım (`a` hiç değişmez, 0'dır):

Adım	Açıklama	b	c
Başlangıç		0	0
1. if	`a > 5` yanlış, `&&` kısa devre yapar, `++b` atlanır	0	0
2. if	`a < 5` doğru, `||` kısa devre yapar, `++b` atlanır; `c += 1`	0	1
3. if	`a == 0` doğru, `&&`'nin sağı hesaplanır: `++b` ile `b` 1 olur, `1 > 0` doğru; `c += 8`	1	9
4. if	`!b` = `!1` = 0, yanlış	1	9
5. if	`a` sıfır, yanlış	1	9

Soru 8.

Cevap:

```
ÇIKTI
1105
```

`kod` 4 olduğundan program `case 4` etiketinden başlar: `toplam` 1000 olur. `break` yoktur, `case 3`'e düşülür: 1100. Yine `break` yoktur, `default`'a düşülür: 1105. `case 1` ve `case 2` hiç çalışmaz.

Soru 9.

Cevap: Koşulun kapanış parantezinden hemen sonraki `;` boş bir deyimdir; `if` yalnızca bu boş deyime bağlıdır. Alttaki `{ ... }` bloğu bağımsız bir bloktur ve koşuldan bağımsız her zaman çalışır. Bu yüzden `puan` 30 iken de "Geçtiniz" yazılır. Düzeltme:

```
if (puan >= 50) {
    printf("Geçtiniz\n");
}
```

Soru 10.

Cevap: İfade `(x == 1) || (2)` diye okunur. `2` sıfırdan farklı olduğu için doğrudur ve `||` her zaman doğru sonuç verir; `x` 7 iken de mesaj yazılır. Her karşılaştırma ayrı yazılmalıdır:

```
if (x == 1 || x == 2) {
    printf("x, 1 ya da 2\n");
}
```

Soru 11.

Cevap: İki sorun vardır. Birincisi, `switch` ifadesi tamsayı tipinde olmalıdır; `double` olamaz (`switch quantity not an integer`). İkincisi, `case 20.5:` etiketi tamsayı sabiti değildir. Ondalıklı sayılarla karar vermek için `if` / `else if` kullanılır; ondalıklı değerleri tam eşitlik yerine aralıkla ya da tolerans ile karşılaştırmak daha güvenlidir.

Soru 12.

Cevap: B. C#'ta düşme yasaktır; içinde deyim olan bir `case` bir çıkışla (`break`, `return`, `throw` vb.) bitmek zorundadır. A yanlıştır: C#'ta `switch` `string` ile çalışır. C yanlıştır: C'de `case` etiketi metin olamaz. D yanlıştır: atamanın sonucu `int`'tir, C# koşul olarak `bool` ister, bu nedenle derlenmez. E yanlıştır: `char` bir tamsayı tipidir ve `switch` ile kullanılabilir.

Soru 13.

Önce pozitif ve negatif durumlar denetlenir; ikisi de değilse sayı sıfırdır.

```
#include <stdio.h>

int main(void) {
    int sayi = 0;
    printf("Bir tamsayı girin: ");
    scanf("%d", &sayi);

    if (sayi > 0) {
        printf("pozitif\n");
    } else if (sayi < 0) {
        printf("negatif\n");
    } else {
        printf("sıfır\n");
    }
    return 0;
}
```

Soru 14.

31 ve 30 günlük aylar kendi içinde gruplanır; araya deyim girmeyen etiketler aynı kola düşer. Şubat kolunda artık yıl koşulu `?:` ile bir değer seçer. Geçersiz ayda mesaj yazılıp program bitirilir.

```
#include <stdio.h>

int main(void) {
    int ay = 0, yil = 0;
    int gun = 0;
    printf("Ay ve yıl: ");
    scanf("%d %d", &ay, &yil);
```

```
    switch (ay) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            gun = 31;
            break;
        case 4: case 6: case 9: case 11:
            gun = 30;
            break;
        case 2:
            gun = ((yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0)
                  ? 29 : 28;
            break;
        default:
            printf("Geçersiz ay!\n");
            return 0;
    }
    printf("%d/%d: %d gün\n", ay, yil, gun);
    return 0;
}
```

Soru 15.

Geçersiz (sıfır ya da negatif) değerler erken `return` ile elenir. Sınıflandırma, yukarıdan aşağıya sıralı `else if` merdivenidir; her dalda alt sınır önceki koşulla zaten sağlanmıştır.

```
#include <stdio.h>

int main(void) {
    double kilo = 0.0, boy = 0.0;
    printf("Kilo (kg) ve boy (m): ");
    scanf("%lf %lf", &kilo, &boy);

    if (boy <= 0 || kilo <= 0) {
        printf("Geçersiz değer!\n");
        return 0;
    }
    double bki = kilo / (boy * boy);
    printf("BKİ: %.2f\n", bki);

    if (bki < 18.5) {
        printf("Zayıf\n");
    } else if (bki < 25) {
        printf("Normal\n");
    } else if (bki < 30) {
        printf("Fazla kilolu\n");
    } else {
        printf("Obez\n");
    }
    return 0;
}
```

Girdi `70 1.75` için çıktı `BKİ: 22.86` ve `Normal` olur.

Soru 16.

Önce `a == 0` durumu elenir; sonra diskriminanta göre üç dal vardır. `sqrt` yalnızca diskriminant negatif olmadığında çağrılır. Derlerken `-lm` gerekebilir. Diskriminantın tam sıfır olup olmadığına bakmak, hesaplanmış ondalıklı değerlerde güvenilir değildir; katsayılar doğrudan tamsayı değerlerle girildiğinde bu örnekte sorun çıkmaz, genel durumda küçük bir toleransla karşılaştırmak daha doğrudur.

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double a = 0.0, b = 0.0, c = 0.0;
    printf("a, b ve c: ");
    scanf("%lf %lf %lf", &a, &b, &c);

    if (a == 0) {
        printf("Bu bir ikinci derece denklem değil.\n");
        return 0;
    }
    double delta = b * b - 4 * a * c;
    if (delta < 0) {
        printf("Gerçel kök yok\n");
    } else if (delta == 0) {
        printf("Tek kök: %.2f\n", -b / (2 * a));
    } else {
        double kok = sqrt(delta);
        double x1 = (-b + kok) / (2 * a);
        double x2 = (-b - kok) / (2 * a);
        printf("x1 = %.2f, x2 = %.2f\n", x1, x2);
    }
    return 0;
}
```

Girdi `1 -3 2` için çıktı `x1 = 2.00, x2 = 1.00` olur.

Bölüm 8 Döngüler

Soru 1.

Cevap: C. `i` sırasıyla 1, 4, 7, 10 değerlerini alır; sonraki değer 13 koşulu bozar. Gövde 4 kez çalışır.

Soru 2.

Cevap: E. `do-while`'da koşul gövdeden sonra denetlendiği için gövde en az bir kez çalışır. A `do-while`'ı anlatıyor, B `continue`'yu, C `break`'i anlatıyor; D yanlıştır çünkü `for` içinde tanımlanan `i` döngüyle sınırlıdır.

Soru 3.

Cevap: B. İç döngü `i = 0` için 1, `i = 1` için 2, `i = 2` için 3 kez döner: 1 + 2 + 3 = 6.

Soru 4.

Cevap: C. `scanf` başarıyla okuduğu değer sayısını döndürür. Girdi bittiğinde `EOF`, harf geldiğinde 0 döndürür; ikisinde de koşul `== 1` sağlanmadığı için döngü biter. D yanlıştır: harf girilince de döngü biter.

Soru 5.

Cevap:

```
ÇIKTI
0 1 3
```

`i = 2` için `continue` yazdırmayı atlar. `i = 4` için `break` döngüyü bitirir; 4 ve 5 hiç yazdırılmaz.

Soru 6.

Cevap:

```
ÇIKTI
10
```

İç döngü `i = 1` için 4, `i = 2` için 3, `i = 3` için 2, `i = 4` için 1 kez döner: 4 + 3 + 2 + 1 = 10.

Soru 7.

Cevap:

```
ÇIKTI
10 7 | 4
```

`n = 10` yazılır, `n` 7 olur ve 7 > 5 olduğu için döngü sürer; 7 yazılır, `n` 4 olur ve 4 > 5 yanlış olduğundan döngü biter. Döngüden sonra `n` 4'tür.

Soru 8.

Cevap:

```
ÇIKTI
3
```

`for` satırının sonundaki `;` boş gövde demektir. Döngü üç tur döner ve `i` 3 olunca biter; `printf` döngünün dışında bir kez çalışır.

Soru 9.

Cevap: `i` 0 iken çift olduğu için `continue` çalışır ve `i++` satırı atlanır; `i` hep 0 kalır, döngü sonsuza kadar sürer. Düzeltmek için `continue`'dan önce `i`'yi artırmak ya da döngüyü `for` ile yazmak gerekir:

```
for (int i = 1; i < 10; i += 2) {
    printf("%d ", i);
}
```

`for`'da `continue` adım kısmını atlamadığı için bu sorun doğmaz.

Soru 10.

Cevap: Koşul `i < 100` olduğu için 100 toplama girmiyor (off-by-one); 1'den 99'a kadar toplam 4950'dir. Doğrusu `i <= 100` yazmaktır:

```
for (int i = 1; i <= 100; i++) {
    toplam += i;
}
```

Soru 11.

Cevap: Çözüm, 3'ten başlayıp 3'er artan bir `for` döngüsüdür; her turda sayaç ve toplayıcı güncellenir.

```
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("n = ");
    scanf("%d", &n);

    int adet = 0;
    int toplam = 0;
    for (int i = 3; i <= n; i += 3) {
```

```
        adet++;
        toplam += i;
    }
    printf("Adet: %d, toplam: %d\n", adet, toplam);
    return 0;
}
```

`n = 20` için 3, 6, 9, 12, 15, 18 sayıları bulunur: adet 6, toplam 63. Aynı sonuç `i % 3 == 0` denetimiyle 1'den `n`'e giden bir döngüyle de elde edilebilir.

Soru 12.

Cevap: Sayının tersi basamak basamak hesaplanır ve orijinal sayıyla karşılaştırılır.

```
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("Pozitif bir sayı: ");
    scanf("%d", &n);

    int kalan = n;
    int ters = 0;
    while (kalan > 0) {
        ters = ters * 10 + kalan % 10;
        kalan /= 10;
    }
    if (ters == n) {
        printf("%d palindromdur.\n", n);
    } else {
        printf("%d palindrom değildir.\n", n);
    }
    return 0;
}
```

Orijinal `n` korunduğu için sonda karşılaştırma yapılabilir. 1221 için ters 1221, 1231 için 1321 çıkar.

Soru 13.

Cevap: Elmasın üst yarısı (orta satır dahil) piramit deseniyle aynıdır; alt yarı ise `n - 1`'den 1'e doğru aynı satır formülünü kullanır: `i`. satırda `n - i` boşluk ve `2 * i - 1` yıldız.

```
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("n = ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int b = 1; b <= n - i; b++) {
            printf(" ");
        }
        for (int y = 1; y <= 2 * i - 1; y++) {
            printf("*");
        }
        printf("\n");
    }
    for (int i = n - 1; i >= 1; i--) {
        for (int b = 1; b <= n - i; b++) {
            printf(" ");
        }
```

```
        for (int y = 1; y <= 2 * i - 1; y++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Soru 14.

Cevap: Sayı en küçük bölenden başlanarak bölünür. Bir bölen sayıyı birden çok kez bölebileceği için iç içe bir `while` kullanılır; döngü sonunda `kalan` 1'den büyükse kalan değer de asal çarpandır.

```
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("Sayı: ");
    scanf("%d", &n);

    int kalan = n;
    for (int bolen = 2; bolen * bolen <= kalan; bolen++) {
        while (kalan % bolen == 0) {
            printf("%d ", bolen);
            kalan /= bolen;
        }
    }
    if (kalan > 1) {
        printf("%d", kalan);
    }
    printf("\n");
    return 0;
}
```

60 için `2 2 3 5`, 84 için `2 2 3 7`, 97 (asal) için yalnızca `97` yazılır. Bölen asal olmayan bir sayıya ulaştığında sayı zaten o sayının asal çarpanlarına ayrılmış olduğundan bölme gerçekleşmez.

Bölüm 9 Fonksiyonlar

Soru 1.

Cevap: C. Prototip, tanımın gövdesiz hâlidir: dönüş tipi, ad, parametre tipleri ve sonunda `;`. Parametre adları yazılmayabilir. A bir değişken bildirimidir. B'de dönüş tipi yok. D'de dönüş tipi `void` olduğundan tanımla çakışır (`conflicting types`). E bir tanımdır, prototip değildir; gerçek tanımla birlikte fonksiyon iki kez tanımlanmış olur.

Soru 2.

Cevap: A. C'de argümanlar değerle geçirilir; fonksiyon bir kopya üzerinde çalışır. B tam tersini söylüyor (ancak adres geçirilirse olur; Bölüm 12). C yanlış: dönüş tipi bunu değiştirmez. D yanlış: değişken de ifade de argüman olabilir. E yanlış: aynı adlı parametre ile çağıran değişken farklı çerçevelerde yaşayan iki ayrı değişkendir.

Soru 3.

Cevap: E. `static int n = 10;` satırı yalnızca ilk çağrıda işlenir. `n` sırayla 10 → 15 (döner 15), 15 → 20 (döner 20), 20 → 25 (döner 25) olur. A, `static` olmasaydı çıkacak sonuçtur. B, değerin artırılmadan önce döndürüldüğünü sanmaktır. C her çağrıda 10 eklendiğini sanmaktır. D hiç değişmediğini varsayar.

```
ÇIKTI
15 20 25
```

Soru 4.

Cevap: D. Prototip ile tanım aynı sayıda ve aynı tipte parametre taşımalıdır. D'de üç parametre vardır, tanımda iki; gcc `conflicting types for 'topla'` hatası verir. A özdeştir. B'de parametre adları farklıdır, adların önemi yoktur. C'de ad hiç yazılmamıştır, o da geçerlidir.

Soru 5.

Cevap: B. `void` fonksiyonda `return;` yazılabilir ve fonksiyonu hemen bitirir. A yanlış: `void` fonksiyon değer döndüremez. C yanlış: `void value not ignored as it ought to be` hatası verir. D yanlış: istediğimiz kadar `void` fonksiyon tanımlayabiliriz. E yanlış: `void` dönüş tipidir, parametreyle ilgisi yoktur; `void bol(int pay, int payda)` geçerlidir.

Soru 6.

Cevap: C. `static` yerel değişken program boyunca yaşar, ilk değeri yalnızca bir kez verilir ve yalnızca tanımlandığı bloktan erişilir. A normal yerel değişkenin özelliğidir. B global değişkene aittir. D normal yerel değişkeni anlatır; `static` değişken çağrı yığınında değil ayrı bir bölgede durur. E yanlış: verilmezse `static` değişken 0 olur.

Soru 7.

Cevap: `toplam` `static` olduğu için çağrılar arasında birikir (2, 5, 10); `tur` ise her çağrıda 0'dan başlar, dolayısıyla yalnızca o çağrının `n` değerini taşır (2, 3, 5). Her `printf` sonuna bir boşluk koyar.

```
ÇIKTI
2/2 5/3 10/5
```

Soru 8.

Cevap: `f` ve `g` argümanın kopyasıyla çalışır; `main`'deki `a` hep 4 kalır. `f(a)`: `x = 8`, döner `9`, yani `b = 9`. `g(a)` kendi kopyasını 104 yapar ve hiçbir şey döndürmez. İlk `printf` `4 9` yazar. `f(f(a))`: içteki önce, `f(4) = 9`; sonra `f(9)`: `x = 18`, döner `19`.

```
ÇIKTI
4 9
19
```

Soru 9.

Cevap: `g` global değişkendir. `artir()` global `g`'yi artırır: 2 → 3. `gizli()` kendi yerel `g`'sini 50'den 51 yapar ve yazar; global `g` değişmez. İkinci `artir()` global `g`'yi 4 yapar. İç blokta `int g = 7;` global `g`'yi gölgeler, ama `artir()` fonksiyonu kendi kapsamında global `g`'yi görür ve onu 5 yapar; blok içindeki `printf` yerel `g`'yi, yani 7'yi yazar. Blok bitince `main`'deki `printf` global `g`'yi, yani 5'i yazar.

```
ÇIKTI
51 7 5
```

Soru 10.

Cevap: `g(f(3))`: içteki önce, `f(3) = 4`; `g(4) = f(4) * 2 = 5 * 2 = 10`. `f(g(3))`: `g(3) = f(3) * 2 = 8`; `f(8) = 9`. Toplam 10 + 9 = 19.

```
ÇIKTI
19
```

Soru 11.

Cevap: İki hata vardır.

1. `yazdir` bir `void` fonksiyondur; çağrısı `int s`'ye atanamaz (`void value not ignored as it ought to be`). `s` değişkeni ve `printf("%d\n", s);` satırı kaldırılmalı ya da `yazdir` bir değer döndüren fonksiyon olmalıdır.
2. `ucKati` prototipte bir, tanımda iki parametreyle yazılmıştır (`conflicting types for 'ucKati'`). Tanımdaki kullanılmayan `int m` parametresi silinmelidir.

Düzeltilmiş hâli:

```
#include <stdio.h>

int ucKati(int n);

void yazdir(int n) {
    printf("%d\n", n);
}

int main(void) {
    yazdir(ucKati(4));
    return 0;
}

int ucKati(int n) {
    return n * 3;
}
```

Soru 12.

Cevap: `kisim / toplam` iki `int`'in bölmesidir; `1 / 4` tamsayı bölmesi olduğu için 0 verir ve `0 * 100` sonucu 0 olur. Dönüş tipinin `double` olması bölmeyi `double` yapmaz. İki düzeltme yolu:

```
double yuzde(int kisim, int toplam) {
    return 100.0 * kisim / toplam;      // 100.0 * kisim zaten double
}

double yuzde2(int kisim, int toplam) {
    return (double)kisim / toplam * 100;   // cast ile
}
```

Her ikisinde de `yuzde(1, 4)` 25.0 verir.

Soru 13.

Cevap: Fonksiyon ilk değeri en büyük kabul eder ve diğerleriyle karşılaştırır.

```
#include <stdio.h>

int enBuyuk(int a, int b, int c) {
    int en = a;
    if (b > en) {
        en = b;
    }
    if (c > en) {
        en = c;
    }
    return en;
}

int main(void) {
```

```
    int x = 0, y = 0, z = 0;
    printf("Üç sayı: ");
    scanf("%d %d %d", &x, &y, &z);
    printf("En büyük: %d\n", enBuyuk(x, y, z));
    return 0;
}
```

`4 9 6` girişi için çıktı `En büyük: 9` olur.

Soru 14.

Cevap: `ucgenMi` kenarların pozitif olduğunu ve her birinin diğer ikisinin toplamından küçük olduğunu denetler; alan yalnızca bu denetim geçilirse hesaplanır. `sqrt` için `<math.h>` eklenir (Linux'ta gerekirse `-lm`).

```
#include <stdio.h>
#include <stdbool.h>
#include <math.h>

bool ucgenMi(double a, double b, double c) {
    return a > 0 && b > 0 && c > 0
        && a < b + c && b < a + c && c < a + b;
}

double ucgenAlani(double a, double b, double c) {
    double s = (a + b + c) / 2;
    return sqrt(s * (s - a) * (s - b) * (s - c));
}

int main(void) {
    double a = 0, b = 0, c = 0;
    printf("Kenarlar: ");
    scanf("%lf %lf %lf", &a, &b, &c);
    if (ucgenMi(a, b, c)) {
        printf("Alan: %.2f\n", ucgenAlani(a, b, c));
    } else {
        printf("Bu kenarlarla üçgen oluşmaz.\n");
    }
    return 0;
}
```

`3 4 5` için `Alan: 6.00`, `1 2 5` için `Bu kenarlarla üçgen oluşmaz.` yazılır.

Soru 15.

Cevap: `static` değişkenin ilk değeri yalnızca bir kez verilir. İlk çağrıda 1000 artırılıp 1001 döndürülsün diye başlangıç değeri 1000 alınır.

```
#include <stdio.h>

int siradakiNumara(void) {
    static int numara = 1000;
    numara++;
    return numara;
}

int main(void) {
    for (int i = 0; i < 3; i++) {
        printf("%d\n", siradakiNumara());
    }
    return 0;
}
```

Çıktı alt alta `1001`, `1002`, `1003` olur.

Soru 16.

Cevap: `asalMi` bir sayıyı denetler; `asalSayisi` ve `asalToplami` 2'den `ust`'e kadar tarayıp bu fonksiyonu kullanır. Toplam için `long long` kullanıldı, çünkü büyük `n` değerlerinde asalların toplamı `int` sınırını aşabilir.

```c
#include <stdio.h>
#include <stdbool.h>

bool asalMi(int n) {
    if (n < 2) {
        return false;
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}

int asalSayisi(int ust) {
    int adet = 0;
    for (int n = 2; n <= ust; n++) {
        if (asalMi(n)) {
            adet++;
        }
    }
    return adet;
}

long long asalToplami(int ust) {
    long long toplam = 0;
    for (int n = 2; n <= ust; n++) {
        if (asalMi(n)) {
            toplam += n;
        }
    }
    return toplam;
}

int main(void) {
    int n = 0;
    printf("n: ");
    scanf("%d", &n);
    printf("Asal sayısı: %d\n", asalSayisi(n));
    printf("Asalların toplamı: %lld\n", asalToplami(n));
    return 0;
}
```

`n = 30` için çıktı `Asal sayısı: 10` ve `Asalların toplamı: 129` olur.

Bölüm 10 Diziler

Soru 1.

Cevap: C. N elemanlı dizide geçerli indeksler 0'dan `N-1`'e kadardır; `puan[8]` dizinin dışındadır ve C bunu denetlemez. Derleyici çoğunlukla ses çıkarmaz, davranış tanımsızdır. A yanlış, indeks 0'dan başlar. B yanlış, son eleman `puan[7]`'dir. D yanlış, C'de böyle bir koruma yoktur. E yanlış, dizi $8 \times 4 = 32$ bayt kaplar.

Soru 2.

Cevap: A. İlk değer listesi yalnızca `a[0]`'a 3 verir; listenin vermediği dört eleman 0 olur: 3 + 0 + 0 + 0 + 0 = 3. B, `{3}` listesinin hepsini 3 yaptığı yanılgısıdır. E yanlış: çöp değer, ilk değer listesi hiç olmayan yerel dizilerde görülür.

Soru 3.

Cevap: D. Her adımda aday sayısı yarılanır: 100, 50, 25, 12, 6, 3, 1; yani yedi adım. Formül: $\lfloor \log_2 100 \rfloor + 1 = 6 + 1 = 7$. A (1) yalnızca en şanslı durumdur. B ve C yetmez; C, `+ 1`'i unutmanın sonucudur. E doğrusal aramanın en kötü durumudur.

Soru 4.

Cevap: B. Dizi fonksiyona adres olarak geçtiği için fonksiyon gerçek diziyi değiştirir ve değişiklik çağırana yansır. A yanlış, `b = a` derlenmez. C yanlış, fonksiyon içinde `a` bir adrestir ve `sizeof(a)` adresin boyutunu verir. D yanlış, `a == b` adresleri karşılaştırır. E yanlış, `const` parametreye yazmak derleme hatasıdır.

Soru 5.

Cevap: E. `sizeof(m)` bütün dizidir: 3 × 4 × 4 = 48 bayt. `sizeof(m[0])` bir satırın boyutudur: 4 × 4 = 16 bayt. 48 / 16 = 3, yani satır sayısı. A (4) sütun sayısıdır (`sizeof(m[0]) / sizeof(m[0][0])`), B (12) toplam eleman sayısı, C (16) bir satırın bayt sayısı, D (48) bütün dizinin bayt sayısıdır.

Soru 6.

Cevap:

```
ÇIKTI
0
2
15
4 8
```

`a = {2, 4, 0, 3, 1}`. İçten dışa çözülür: `a[a[0]]` = `a[2]` = 0; `a[a[a[0]]]` = `a[0]` = 2. `a[a[3]] = 8;` satırında `a[3]` = 3 olduğundan `a[3]` elemanına 8 yazılır ve dizi `{2, 4, 0, 8, 1}` olur; toplam 15. Son satırda `a[a[4]]` = `a[1]` = 4 ve `a[3]` = 8.

Soru 7.

Cevap:

```
ÇIKTI
1 11 2 2
```

`f(s)` ve `f(d[1])` çağrılarında değer kopyalanır: `f` kendi `x`'ini değiştirir, `s` ve `d[1]` etkilenmez. `g(d)` ise gerçek diziyi değiştirir: `d[0]` 1'den 11 olur, `d[2]` ise `d[1]`'in değerini (2) alır. `s` 1, `d[1]` 2 kalır.

Soru 8.

Cevap:

```
ÇIKTI
1 4 8 6 9
```

Dizi `6 9 8 1 4`. Tur 0'da en küçük 1'dir (indeks 3) ve 6 ile yer değiştirir: `1 9 8 6 4`. Tur 1'de kalan kısmın (`9 8 6 4`) en küçüğü 4'tür (indeks 4) ve 9 ile yer değiştirir: `1 4 8 6 9`. Dizi henüz sıralı değildir; üçüncü tur 6 ile 8'i değiştirecekti.

Soru 9.

Cevap: Döngü 1'den 5'e kadar gider; `kare[5]` dizinin dışıdır (tanımsız davranış) ve `kare[0]` hiç yazılmaz. Doğrusu indeksi 0'dan başlatıp `<` kullanmaktır:

```
int kare[5];
for (int i = 0; i < 5; i++) {
    kare[i] = (i + 1) * (i + 1);
}
```

Soru 10.

Cevap: İlk hata `kopya = dizi;` satırıdır: diziler bütün olarak atanamaz, derleyici `assignment to expression with array type` hatası verir. İkinci hata `dizi == kopya` karşılaştırmasıdır: bu, içeriği değil adresleri karşılaştırır ve iki ayrı dizi için hep yanlıştır. Düzeltme:

```
int dizi[4] = {1, 2, 3, 4};
int kopya[4];
int ayni = 1;
for (int i = 0; i < 4; i++) {
    kopya[i] = dizi[i];
}
for (int i = 0; i < 4; i++) {
    if (dizi[i] != kopya[i]) {
        ayni = 0;
    }
}
if (ayni) {
    printf("Diziler aynı\n");
}
```

Kopyalama için `memcpy(kopya, dizi, sizeof(dizi))`, karşılaştırma için `memcmp(dizi, kopya, sizeof(dizi)) == 0` de kullanılabilir.

Soru 11.

Cevap:

```
#include <stdio.h>

#define N 5

int main(void) {
    int sayilar[N];

    printf("%d sayı girin: ", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &sayilar[i]);
    }

    printf("Ters sıra:");
    for (int i = N - 1; i >= 0; i--) {
        printf(" %d", sayilar[i]);
    }
    printf("\n");
    return 0;
}
```

Sayılar sırayla okunur, ikinci döngü indeksi sondan başa doğru gezer. Diziyi gerçekten ters çevirmek gerekmez.

Soru 12.

Cevap:

```c
#include <stdio.h>

#define N 6

int main(void) {
    int notlar[N];

    printf("%d not girin: ", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &notlar[i]);
    }

    int enBuyuk = notlar[0];
    int enBuyukIndeks = 0;
    int toplam = 0;
    for (int i = 0; i < N; i++) {
        toplam += notlar[i];
        if (notlar[i] > enBuyuk) {
            enBuyuk = notlar[i];
            enBuyukIndeks = i;
        }
    }

    double ortalama = (double)toplam / N;
    int ustunde = 0;
    for (int i = 0; i < N; i++) {
        if (notlar[i] > ortalama) {
            ustunde++;
        }
    }

    printf("En büyük: %d (indeks %d)\n", enBuyuk, enBuyukIndeks);
    printf("Ortalama: %.2f\n", ortalama);
    printf("Ortalamanın üstünde: %d eleman\n", ustunde);
    return 0;
}
```

Toplam 420, ortalama 70.00; 70'in üstünde 80, 92 ve 80 olduğundan 3 eleman. Ortalama için `(double)` dönüşümü gereklidir, yoksa tamsayı bölmesi yapılırdı. Ortalamanın üstündekiler ancak ortalama bilindikten sonra, ikinci bir geçişle sayılabilir.

Soru 13.

Cevap:

```c
#include <stdio.h>

#define N 3

int main(void) {
    int m[N][N];

    printf("Matris (%dx%d): ", N, N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &m[i][j]);
        }
    }

    int ana = 0;
    int ikincil = 0;
    for (int i = 0; i < N; i++) {
```

```
        int satirToplam = 0;
        for (int j = 0; j < N; j++) {
            satirToplam += m[i][j];
            if (i == j) {
                ana += m[i][j];
            }
            if (i + j == N - 1) {
                ikincil += m[i][j];
            }
        }
        printf("%d. satır toplamı: %d\n", i + 1, satirToplam);
    }
    printf("Ana köşegen toplamı: %d\n", ana);
    printf("İkincil köşegen toplamı: %d\n", ikincil);
    return 0;
}
```

`satirToplam` her satırın başında sıfırlanır. `i == j` ana köşegene, `i + j == N - 1` ikincil köşegene eklenir. Ana köşegen 2 + 5 + 6 = 13, ikincil köşegen 1 + 5 + 3 = 9; satır toplamları 10, 18 ve 17.

Soru 14.

Cevap:

```
#include <stdio.h>

#define N 6

int main(void) {
    int sayilar[N];

    printf("%d sayı girin: ", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &sayilar[i]);
    }

    int enBuyuk = sayilar[0];
    int ikinci = sayilar[1];
    if (ikinci > enBuyuk) {
        int gecici = enBuyuk;
        enBuyuk = ikinci;
        ikinci = gecici;
    }
    for (int i = 2; i < N; i++) {
        if (sayilar[i] > enBuyuk) {
            ikinci = enBuyuk;
            enBuyuk = sayilar[i];
        } else if (sayilar[i] > ikinci) {
            ikinci = sayilar[i];
        }
    }

    printf("En büyük: %d\n", enBuyuk);
    printf("İkinci en büyük: %d\n", ikinci);
    return 0;
}
```

İlk iki eleman kendi aralarında sıralanarak başlanır. Sonraki her eleman en büyükten büyükse eski en büyük "ikinci"ye iner, yeni eleman en büyük olur; yalnızca ikinciden büyükse ikincinin yerini alır. Girdi `12 45 7 38 51 29` için en büyük 51, ikinci en büyük 45 çıkar.

Bölüm 11 Karakter Dizileri (String)

Soru 1.

Cevap: C. `"Merhaba"` 7 karakterdir; derleyici sonuna `'\0'` ekler ve dizi 8 bayt olur.

Soru 2.

Cevap: C. `'A'` kodu 65 olan bir karakter sabitidir. `"A"` ise `'A'` ve `'\0'` karakterlerinden oluşan iki baytlık bir string'dir.

Soru 3.

Cevap:

```
ÇIKTI
Mer 8
```

`s[3] = '\0'` metni üçüncü karakterden sonra bitirir, bu yüzden `%s` yalnızca `Mer` yazar. Dizinin boyutu değişmez: `sizeof s` hâlâ 8'dir.

Soru 4.

Cevap: C. `strcmp` büyük/küçük harf duyarlıdır ve karakter kodlarını karşılaştırır. `'Z'` 90, `'a'` 97 olduğundan sonuç negatiftir. Dönen değerin tam büyüklüğü (örneğin -1) standartta belirtilmemiştir; yalnızca işaretine güvenilir.

Soru 5.

Cevap: Bir diziye `=` ile sonradan metin atanamaz (`assignment to expression with array type`). Çözüm `strcpy` kullanmaktır:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char ad[10];
    strcpy(ad, "Can");
    printf("%s\n", ad);
    return 0;
}
```

Tanımlama sırasında ilk değer vermek de olur: `char ad[10] = "Can";`.

Soru 6.

Cevap: `a == b` iki dizinin içeriğini değil bellekteki başlangıç adreslerini karşılaştırır; adresler farklı olduğundan sonuç her zaman yanlıştır. Doğrusu `strcmp(a, b) == 0` olmalıdır. Ayrıca `fgets` satır sonunu (`'\n'`) tampona aldığından, karşılaştırmadan önce `strcspn` ile satır sonlarını silmek iyi olur.

Soru 7.

Cevap: C. `fgets` satır sonunu da tampona alır: `"Ali\n"` olur ve uzunluğu 4'tür.

Soru 8.

Cevap:

```
ÇIKTI
uc
3
```

`strrchr` son noktayı bulur, `p + 1` ondan sonrasını (`uc`) gösterir. `strchr` ilk noktayı bulur; bu `s[3]` konumunda olduğundan adres farkı 3'tür.

Soru 9.

Cevap: D. `scanf("%s", t)` genişlik sınırı koymadığı için 19 karakterden uzun bir girdi `t` dizisinin dışına yazar. Diğer seçeneklerde boyut sınırı vardır; E ise yalnızca ilk değer vermektir.

Soru 10.

Cevap:

```
ÇIKTI
A*B*
```

Rakamlar `*` ile değiştirilir, harfler büyük harfe çevrilir.

Soru 11.

Cevap: C. `strlen` bayt sayar: `ş` iki bayt, `e`, `k`, `e`, `r` dörder bayt değil birer bayt olduğundan toplam 2 + 4 = 6'dır.

Soru 12.

Cevap:

```
ÇIKTI
Merha 7
```

`t` altı baytlıktır: beş karakter ve `'\0'` yazılır, dolayısıyla `t` içinde `Merha` kalır. `snprintf` ise kırpılmasa yazacağı karakter sayısını döndürür: `Merhaba` 7 karakterdir. (gcc bu örnekte kırpılacağını önceden görüp uyarı verebilir.)

Soru 13.

Cevap: `kod` yalnızca 8 baytlıktır; `girdi` ise 100 baytlık bir satır tutabilir. Satır 7 karakterden uzunsa `strcpy` taşar. Hedef boyutunu aşmayacak şekilde kopyalamak gerekir:

```
snprintf(kod, sizeof kod, "%s", girdi);   // en çok 7 karakter + '\0'
```

ya da uzunluk önce `strlen(girdi) < sizeof kod` ile denetlenip yalnızca sığıyorsa `strcpy` çağrılır.

Soru 14.

Cevap:

```
#include <stdio.h>
#include <ctype.h>

int sesliSay(const char s[]) {
    int adet = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        switch (tolower((unsigned char)s[i])) {
            case 'a': case 'e': case 'i': case 'o': case 'u':
                adet++;
                break;
            default:
                break;
        }
    }
    return adet;
}
```

```
int main(void) {
    printf("%d\n", sesliSay("Merhaba Dunya"));    // 5
    return 0;
}
```

Her karakter küçük harfe çevrilir ve beş sesli harften biriyle karşılaştırılır; `"Merhaba Dunya"` için sonuç 5'tir (e, a, a, u, a).

Soru 15.

Cevap:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int palindromMu(const char s[]) {
    int bas = 0;
    int son = (int)strlen(s) - 1;

    while (bas < son) {
        while (bas < son && !isalpha((unsigned char)s[bas])) {
            bas++;                          // baştaki harf olmayanları atla
        }
        while (bas < son && !isalpha((unsigned char)s[son])) {
            son--;                          // sondaki harf olmayanları atla
        }
        if (tolower((unsigned char)s[bas]) != tolower((unsigned char)s[son])) {
            return 0;
        }
        bas++;
        son--;
    }
    return 1;
}

int main(void) {
    char satir[100];

    printf("Metin: ");
    fgets(satir, sizeof satir, stdin);
    satir[strcspn(satir, "\n")] = '\0';
    printf("%s\n", palindromMu(satir) ? "Palindrom" : "Palindrom degil");
    return 0;
}
```

İki indeks uçlardan ortaya yürür; harf olmayan karakterler atlanır, harfler küçük harfe çevrilerek karşılaştırılır.

Soru 16.

Cevap:

```
void sayiyiMetneCevir(int n, char hedef[]) {
    int i = 0;
    int negatif = n < 0;
    long long k = n;                // INT_MIN işareti için geniş tip
    if (negatif) {
        k = -k;
    }

    if (k == 0) {
        hedef[i++] = '0';
```

```
    }
    while (k > 0) {
        hedef[i++] = (char)('0' + k % 10);   // en düşük basamak önce
        k /= 10;
    }
    if (negatif) {
        hedef[i++] = '-';
    }
    hedef[i] = '\0';

    for (int a = 0, b = i - 1; a < b; a++, b--) {   // diziyi ters çevir
        char gecici = hedef[a];
        hedef[a] = hedef[b];
        hedef[b] = gecici;
    }
}
```

Basamaklar `% 10` ile sondan başa üretildiği için sonda ters çevirme gerekir. `'0' + basamak` sayıyı rakam karakterine dönüştürür. Sıfır özel durumdur, çünkü döngü hiç çalışmaz; işaret ise sona eklenip ters çevirmeyle başa gelir.

Bölüm 12 İşaretçiler (Pointers)

Soru 1.

Cevap: B. `*` yalnızca hemen yanındaki ada aittir; `p` işaretçi, `q` sıradan `int` olur. `int *p, *q;` yazılsaydı ikisi de işaretçi olurdu.

Soru 2.

Cevap: C. `p`, `a[1]`'i gösterir. `p[2]` = `a[3]` = 40 ve `*(p + 1)` = `a[2]` = 30; toplam 70.

Soru 3.

Cevap: D. `p` sabit `int`'e işaretçidir (`const` `*`'ın solunda); gösterilen değer `*p` üzerinden değiştirilemez. `p`'nin kendisi değişebilir (A), `q` sabit işaretçidir ama gösterdiği değer değişebilir (B), `a` değişkeni doğrudan değiştirilebilir (C).

Soru 4.

Cevap: A. Parametre `int a[]` yazılsa da `int *a` demektir. Dizi kopyalanmaz, adresi geçer (B yanlış); `sizeof(a)` işaretçi boyutunu verdiği için D de yanlıştır. Fonksiyon diziyi değiştirebilir (E yanlış).

Soru 5.

Cevap: D. `int *p[4]`, elemanları `int *` olan 4 elemanlı bir dizidir. 64 bitte her işaretçi 8 bayt olduğundan 4 × 8 = 32.

Soru 6.

Cevap: B. `int (*f)(int);` fonksiyon işaretçisidir. A seçeneği `int *` döndüren bir fonksiyon bildirir; parantezler anlamı belirler.

Soru 7.

Cevap:

```
ÇIKTI
15 9
```

`*p = *q + 1` ile `x` = 10 olur. `q = p` sonrası `q` de `x`'i gösterir; `*q += 5` ile `x` = 15. `y` hiç değişmez.

Soru 8.

Cevap:

```
ÇIKTI
3 3 4
```

`p - q` = 4 - 1 = 3 (eleman sayısı). `*(p - 2)` = `a[2]` = 3. `q[2]` = `a[3]` = 4.

Soru 9.

Cevap:

```
ÇIKTI
15 7 7 7
```

`*p++ += 10` ifadesinde `p++` önce `a[0]`'ı seçer, ona 10 eklenir (15) ve `p` `a[1]`'e ilerler. `(*p)++` ile `a[1]` = 7 olur. Sonuçta dizi `15 7 7`, `*p` ise `a[1]` = 7.

Soru 10.

Cevap:

```
ÇIKTI
3 2
```

`*a = *a + b` ile `x` = 3 olur. Sonra `b = 100` yerel kopyayı değiştirir; `a = &b` işaretçinin yerel kopyasını `b`'ye yöneltir ve `*a = 7` yalnızca yerel `b`'yi 7 yapar. `main`'deki `y` değişmez.

Soru 11.

Cevap: Fonksiyon, `a` ve `b` işaretçilerinin yerel kopyalarını takas ediyor; gösterdikleri değişkenlere hiç dokunmuyor. Değerleri değiştirmek için `*` ile dolaylama gerekir:

```
void takas(int *a, int *b) {
    int gecici = *a;
    *a = *b;
    *b = gecici;
}
```

Soru 12.

Cevap: Üç hata vardır.

1. `kare` yerel `s` değişkeninin adresini döndürüyor (sarkan işaretçi). Çözüm: sonucu değer olarak döndürmek (`int kare(int n) { return n * n; }`) ya da çağıranın verdiği adrese yazmak.
2. `int *p;` ilk değer verilmeden `*p = 5;` ile kullanılıyor. Çözüm: `int sayi = 0; int *p = &sayi;` gibi geçerli bir adres göstermek.
3. `char *m = "abc"; m[0] = 'X';` string sabitini değiştiriyor. Çözüm: `char m[] = "abc";` (değiştirilebilir kopya) ya da değiştirmeden `const char *m` kullanmak.

Soru 13.

Cevap:

```
#include <stdio.h>

void istatistik(const int *a, int n, double *ortalama, int *enBuyuk) {
    int toplam = 0;
    *enBuyuk = a[0];
    for (int i = 0; i < n; i++) {
        toplam += a[i];
        if (a[i] > *enBuyuk) {
```

```
            *enBuyuk = a[i];
        }
    }
    *ortalama = (double)toplam / n;
}

int main(void) {
    int dizi[] = {4, 9, 2, 7, 5};
    double ort = 0.0;
    int enB = 0;
    istatistik(dizi, 5, &ort, &enB);
    printf("Ortalama: %.1f, en buyuk: %d\n", ort, enB);
    return 0;
}
```

Çıktı: `Ortalama: 5.4, en buyuk: 9`. İki sonuç, adresi geçirilen iki değişkene yazılır. `(double)` dönüşümü tamsayı bölmesini önler.

Soru 14.

Cevap:

```
#include <stdio.h>
#include <string.h>

void ters(char *s) {
    char *bas = s;
    char *son = s + strlen(s) - 1;
    while (bas < son) {
        char gecici = *bas;
        *bas = *son;
        *son = gecici;
        bas++;
        son--;
    }
}

int main(void) {
    char metin[] = "bilgisayar";
    ters(metin);
    printf("%s\n", metin);
    return 0;
}
```

Çıktı: `rayasiglib`. İki işaretçi birbirine yaklaşırken karşılıklı harfler takas edilir. Metin değiştirileceği için `char metin[]` olarak tanımlanmalıdır; string sabiti olmaz.

Soru 15.

Cevap:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Kullanim: %s sayi1 sayi2 ...\n", argv[0]);
        return 1;
    }
    int toplam = 0;
    int enBuyuk = atoi(argv[1]);
    for (int i = 1; i < argc; i++) {
```

```
        int s = atoi(argv[i]);
        toplam += s;
        if (s > enBuyuk) {
            enBuyuk = s;
        }
    }
    printf("Toplam: %d, en buyuk: %d\n", toplam, enBuyuk);
    return 0;
}
```

`./program 5 -2 9 3` çağrısı `Toplam: 15, en buyuk: 9` yazar. Argümanlar `argv[1]` 'den başlar; `argv[0]` program adıdır. Argüman yokken `argv[1]` okunamayacağı için önce `argc` denetlenir.

Soru 16.

Cevap:

```
#include <stdio.h>

int kare(int x) {
    return x * x;
}

int ikiKati(int x) {
    return 2 * x;
}

int uygula(int *a, int n, int (*f)(int)) {
    int adet = 0;
    for (int i = 0; i < n; i++) {
        a[i] = f(a[i]);
        adet++;
    }
    return adet;
}

int main(void) {
    int d[] = {1, 2, 3, 4};
    uygula(d, 4, kare);
    printf("%d %d %d %d\n", d[0], d[1], d[2], d[3]);
    int k = uygula(d, 4, ikiKati);
    printf("%d %d %d %d (%d eleman)\n", d[0], d[1], d[2], d[3], k);
    return 0;
}
```

Çıktı: `1 4 9 16`, ardından `2 8 18 32 (4 eleman)`. Fonksiyon adı (parantezsiz) adres olarak geçirilir; `f(a[i])` çağrısı gelen fonksiyonu çalıştırır.

Bölüm 13 Dinamik Bellek Yönetimi

Soru 1.

Cevap: C. Bayt sayısı, eleman sayısı çarpı eleman boyutudur: 25 × 4 = 100. A yalnızca eleman sayısıdır. B ve D anlamsız toplamlardır (25 + 4, 100 + 4). E, bir işaretçinin boyutudur; `malloc` ise `p` için değil, `p` 'nin gösterdiği blok için yer ayırır.

Soru 2.

Cevap: B. `calloc`, ayırdığı bloğun tüm baytlarını 0 yapar; `malloc` içeriğe dokunmaz (çöp). A tersini söylüyor. C yanlış: ikisi de öbekten ayırır. D yanlış: `calloc` ile alınan blok da `free` ile bırakılır. E yanlış: ikisi de yer bulamazsa `NULL` döndürür.

Soru 3.

Cevap: E. İlk bloğun adresi `a`'da, ikincisinin adresi `b`'dedir. `a = b;` ile `a`'nın eski değeri (birinci bloğun adresi) silinir; başka hiçbir işaretçi o bloğu göstermediği için ona bir daha ulaşılamaz: sızıntı. `free(a)` ise artık ikinci bloğu bırakır (`a` ve `b` aynı bloğu gösterir), yani ikinci blok sızmaz (A ve B yanlış). Bu arada `b` sarkan işaretçi olur, ama kullanılmadığı için sorun çıkmaz. C yanlış: `free` bloğu gerçekten bırakır. D yanlış: hangi bloğun sızdığı kesin olarak bellidir.

Soru 4.

Cevap: A. `realloc` başarısız olunca `NULL` döndürür ve eski bloğa dokunmaz. Sonucu doğrudan `p`'ye atarsak `p` `NULL` olur ve eski bloğun adresi kaybolur: sızıntı. Doğrusu geçici işaretçi kullanmaktır. B yanlış: eski veri korunur. C yanlış: `realloc` büyütmek için de kullanılır. D yanlış: `realloc` `p`'yi kendisi güncellemez, dönen adresi bizim atamamız gerekir. E yanlış: yeni eklenen kısım sıfırlanmaz, çöptür.

Soru 5.

Cevap: D. `free(NULL)` hiçbir şey yapmaz, güvenlidir. A yanlış: `free` yerel işaretçiyi değiştirmez, `p = NULL;` bizim yazmamız gereken satırdır. B yanlış: C'de dönüşüm gereksizdir. C yanlış: `sizeof(p)` işaretçinin boyutunu verir. E yanlış: çifte `free` tanımsız davranıştır.

Soru 6.

Cevap:

```
ÇIKTI
24
3 0 12 0 9 24 8
```

`calloc` 5 elemanı sıfırlar: `{0, 0, 0, 0, 0}`. Sonra `p[0] = 3`, `p[2] = 12`, `p[4] = 12 - 3 = 9` olur: `{3, 0, 12, 0, 9}`, toplam 24. `realloc` ilk beş elemanı korur; `p[5] = 24` ve `p[6] = 24 / 3 = 8` yazılır. İlk satırda `toplam` (24), ikinci satırda yedi eleman yazdırılır (her elemanın ardında boşluk vardır).

Soru 7.

Cevap:

```
ÇIKTI
14 13 12 11
100 11
```

`kopyala`, `a`'yı tersten okuyup her elemana 10 ekler: `a = {1, 2, 3, 4}` için `k = {14, 13, 12, 11}`. `k` öbekte ayrı bir bloktur; `a[0] = 100;` ataması `k`'yi etkilemez. Son satırda `a[0]` = 100 ve `k[3]` = 11 yazılır.

Soru 8.

Cevap:

```
ÇIKTI
10 35
7 3
```

Satırların uzunlukları 1, 2, 3, 4'tür; toplam eleman 10. `m[i][j] = i * j + 1` değerleri: satır 0: `{1}`; satır 1: `{1, 2}`; satır 2: `{1, 3, 5}`; satır 3: `{1, 4, 7, 10}`. Toplamlar 1 + 3 + 9 + 22 = 35. `m[3][2]` = 3 × 2 + 1 = 7, `m[2][1]` = 2 × 1 + 1 = 3.

Soru 9.

Cevap: İki hata vardır. Birincisi, `malloc(strlen(s))` sonundaki `'\0'` için yer ayırmaz; `strcpy` bloğun bir bayt dışına yazar (tanımsız davranış). Doğrusu `malloc(strlen(s) + 1)`. İkincisi, `malloc`'ın dönüş değeri `NULL` için denetlenmiyor. Düzeltilmiş hâl:

```
char *buyukHarfKopya(const char *s) {
    char *k = malloc(strlen(s) + 1);
    if (k == NULL) {
        return NULL;
    }
    strcpy(k, s);
    for (int i = 0; k[i] != '\0'; i++) {
        k[i] = toupper((unsigned char)k[i]);
    }
    return k;
}
```

Dönen belleği çağıran `free` etmelidir. `toupper`'daki `(unsigned char)` dönüşümü, Bölüm 11'de gördüğümüz güvenli kullanımdır.

Soru 10.

Cevap: `a` yerel bir dizidir ve yığında yaşar; fonksiyon bitince yok olur. Döndürülen adres sarkan işaretçidir (derleyici de uyarı verir). Üstelik `n > 100` ise dizi taşar. Doğrusu öbekte yer ayırıp sahipliği çağırana devretmektir:

```
int *ilkSayilar(int n) {
    int *a = malloc(n * sizeof *a);
    if (a == NULL) {
        return NULL;
    }
    for (int i = 0; i < n; i++) {
        a[i] = i + 1;
    }
    return a;               // çağıran free etmelidir
}
```

Soru 11.

Cevap: C. `toplam > 100` olduğunda fonksiyon `free(a)` satırına gelmeden `return 100;` ile çıkar; `a` bloğu sızar. B'de `malloc` zaten başarısız olduğundan bırakılacak blok yoktur. E'de `malloc(0)` davranışı platforma bağlıdır, ama o durumda toplam 0 olur ve `free(a)` çalışır. Her `return` yolunun bloğu bıraktığından emin olmak gerekir.

Soru 12.

Cevap:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int kapasite = 2;
    int adet = 0;
    int *dizi = malloc(kapasite * sizeof *dizi);
    if (dizi == NULL) {
        return 1;
    }

    printf("Sayıları girin (0 ile biter): ");
    int x = 0;
    while (scanf("%d", &x) == 1 && x != 0) {
        if (adet == kapasite) {
            int *yeni = realloc(dizi, 2 * kapasite * sizeof *yeni);
            if (yeni == NULL) {
                free(dizi);
                return 1;
            }
```

```
            dizi = yeni;
            kapasite *= 2;
        }
        dizi[adet++] = x;
    }

    if (adet == 0) {
        printf("Sayı girilmedi.\n");
        free(dizi);
        return 0;
    }

    long toplam = 0;
    for (int i = 0; i < adet; i++) {
        toplam += dizi[i];
    }
    double ort = (double)toplam / adet;
    int ustu = 0;
    for (int i = 0; i < adet; i++) {
        if (dizi[i] > ort) {
            ustu++;
        }
    }
    printf("Ortalama: %.2f\n", ort);
    printf("Ortalamadan büyük: %d\n", ustu);

    free(dizi);
    return 0;
}
```

Dizi dolunca kapasite ikiye katlanır ve `realloc` sonucu önce geçici işaretçiye alınır. Ortalama `double` hesaplanır; karşılaştırma `int` ile `double` arasında yapılır. Hiç sayı girilmediğinde bölme yapılmaz.

Soru 13.

Cevap:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *birlestir(const char *ad, const char *soyad) {
    size_t uzunluk = strlen(ad) + 1 + strlen(soyad) + 1;   // boşluk ve '\0'
    char *s = malloc(uzunluk);
    if (s == NULL) {
        return NULL;
    }
    snprintf(s, uzunluk, "%s %s", ad, soyad);
    return s;
}

int main(void) {
    char ad[50];
    char soyad[50];
    printf("Ad Soyad: ");
    if (scanf("%49s %49s", ad, soyad) != 2) {
        return 1;
    }
    char *tam = birlestir(ad, soyad);
    if (tam == NULL) {
        printf("Bellek ayrılamadı.\n");
        return 1;
    }
```

```
    printf("Tam ad: %s (%zu karakter)\n", tam, strlen(tam));
    free(tam);
    return 0;
}
```

Gereken bayt sayısı: ad uzunluğu + 1 (boşluk) + soyad uzunluğu + 1 (`'\0'`). Sonuç sahipliği `main`'e geçtiği için `free`'yi `main` yazar.

Soru 14.

Cevap:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int satir = 0;
    int sutun = 0;
    printf("Satır ve sütun sayısı: ");
    if (scanf("%d %d", &satir, &sutun) != 2 || satir <= 0 || sutun <= 0) {
        printf("Geçersiz boyut.\n");
        return 1;
    }

    int *m = malloc(satir * sutun * sizeof *m);
    int *d = malloc(satir * sutun * sizeof *d);
    if (m == NULL || d == NULL) {
        free(m);                        // free(NULL) güvenlidir
        free(d);
        return 1;
    }

    printf("%d eleman girin: ", satir * sutun);
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            scanf("%d", &m[i * sutun + j]);
        }
    }

    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            d[j * satir + i] = m[i * sutun + j];
        }
    }

    printf("Devrik:\n");
    for (int j = 0; j < sutun; j++) {
        for (int i = 0; i < satir; i++) {
            printf("%3d", d[j * satir + i]);
        }
        printf("\n");
    }

    free(m);
    free(d);
    return 0;
}
```

`satir × sutun` boyutlu matrisin devriği `sutun × satir` boyutludur: `m`'nin `(i, j)` elemanı `d`'nin `(j, i)` elemanı olur; tek blokta bu `d[j * satir + i]` demektir. İki `malloc`'tan biri başarısız olursa diğeri bırakılır (`free(NULL)` güvenli olduğu için ayrı denetim gerekmez).

Bölüm 14 Yapılar: struct, union, enum ve typedef

Soru 1.

Cevap: B. `Kayit b = a;` yapıyı üye üye kopyalar; dizi üye `ad` de kopyalanır. Bu yüzden `b.ad` ve `b.puan` üzerindeki değişiklikler `a`'yı etkilemez. A yanlış, dizi üyesi yapı atamasında adres olarak değil, içerik olarak kopyalanır. C ve E yanlış, `a`'nın hiçbir üyesi değişmez. D yanlış, aynı tipten yapılar `=` ile atanabilir.

Soru 2.

Cevap: D. `p` bir işaretçidir; işaretçi üzerinden üyeye `.` ile erişilemez, `->` ya da `(*p).no` gerekir. Diğer şıkların hepsi geçerlidir: `(&o)->no`, `o`'nun adresi üzerinden `->` kullanır.

Soru 3.

Cevap: D. `K` = 0, `L` = 5, `M` = 6 (bir öncekinin bir fazlası), `N` = 2, `O` = 3. Yani `M + O` = 6 + 3 = 9. B yalnızca `M`'dir; E, `M` ile `L`'yi toplama (6 + 5) yanılgısıdır.

Soru 4.

Cevap: D. `a` 0. baytta, `b` 4'ün katı olan 4. baytta başlar (1-3 boş), `c` 8. baytta yer alır: toplam 9 bayt olur. Yapı boyutu en büyük hizalamanın (4) katına yuvarlanır: 12. C (9) kuyruk dolgusunu unutmaktır; A (6) boyutların toplamı gibi düşünmektir.

Soru 5.

Cevap: C. `union` en büyük üyenin boyutu kadardır: `char s[5]` 5 bayt, `int` 4 bayt. Ancak boyut en büyük hizalamanın (`int` için 4) katına yuvarlanır; 5'in 4'ün katına yuvarlanmışı 8'dir. B (5) yuvarlamayı unutmaktır; D (9) üyelerin toplamıdır.

Soru 6.

Cevap: C. `typedef struct { ... } Nokta;` yapısının etiketi yoktur; `struct Nokta` adlı bir tip tanımlı değildir, derleyici "storage size isn't known" hatası verir. Diğer satırlar geçerlidir; D'de belirlenmiş başlatıcı `x`'i sıfırlar.

Soru 7.

Cevap:

```
ÇIKTI
1 2
6 7
0 7
```

`kaydir` kopya üzerinde çalıştığı için `a` değişmez (1 2). `kaydirilmis` değiştirilmiş kopyayı döndürür; `b` = (6, 7). `c = b` bağımsız bir kopyadır; `c.x = 0` yapılınca `b` etkilenmez, `c` = (0, 7).

Soru 8.

Cevap:

```
ÇIKTI
b 25
c 30
1
65
```

`p++` işaretçiyi ikinci elemana taşır; `p->deger += 5` onu 25 yapar. `(p + 1)` üçüncü elemandır: `c` ve 30. `p - d` iki işaretçi arasındaki eleman sayısıdır: 1. Toplam: 10 + 25 + 30 = 65.

Soru 9.

Cevap:

```
ÇIKTI
5 2 3
8
4 0
```

`YESIL` = 4 olduğundan `MAVI` = 5; `SARI` = 2 verildiği için `MOR` = 3. `union U`'nun boyutu en büyük üye olan `double` kadardır: 8. `r = MOR + 1` = 4; bu değer `MAVI` (5) ile eşit değildir, karşılaştırma 0 verir. `enum` aralık denetlemediği için 4 bir sabit olmasa da geçerlidir.

Soru 10.

Cevap: Üç hata şunlardır.

1. `b.ad = "Mehmet";` — diziye `=` ile metin atanamaz. Doğrusu `strcpy(b.ad, "Mehmet");` (`<string.h>` ile).
2. `if (a == b)` — yapılar `==` ile karşılaştırılamaz. Doğrusu `strcmp(a.ad, b.ad) == 0 && a.no == b.no`.
3. `p.no` — `p` işaretçi olduğu için `p->no` ya da `(*p).no` yazılmalıdır.

Soru 11.

Cevap: Üç sorun vardır.

1. `malloc(10)` yalnızca 10 bayt ayırır; 10 öğrenci için `10 * sizeof *liste` gerekir. Döngü ayrılan bellek dışına yazar (tanımsız davranış).
2. `malloc`'un dönüş değeri `NULL` için denetlenmemiştir.
3. Ayrılan bellek `free` ile serbest bırakılmamıştır (bellek sızıntısı).

Doğrusu:

```c
Ogrenci *liste = malloc(10 * sizeof *liste);
if (liste == NULL) {
    return 1;
}
for (int i = 0; i < 10; i++) {
    liste[i].no = i;
}
free(liste);
```

Soru 12.

Cevap: Argümanlar değerle geçer; `a` ve `b` çağıran taraftaki noktaların kopyalarıdır. Takas yalnızca kopyaları değiştirir. İşaretçi almalıdır:

```c
void takas(Nokta *a, Nokta *b) {
    Nokta t = *a;
    *a = *b;
    *b = t;
}
// çağrı: takas(&p1, &p2);
```

Soru 13.

Cevap:

```c
#include <stdio.h>
#include <math.h>

typedef struct {
    double x;
    double y;
} Nokta;

double mesafe(Nokta a, Nokta b) {
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

int main(void) {
    Nokta o = {0, 0};
    Nokta p = {3, 4};
    printf("Mesafe: %.2f\n", mesafe(o, p));
    return 0;
}
```

Çıktı `Mesafe: 5.00` olur. Derlerken `-lm` bağlantı seçeneği gerekebilir. Yapı küçük olduğu için değerle geçirmek uygundur.

Soru 14.

Cevap:

```c
#include <stdio.h>

typedef struct { double x, y; } Nokta;
typedef struct {
    Nokta solAlt;
    Nokta sagUst;
} Dikdortgen;

double alan(const Dikdortgen *d) {
    double en = d->sagUst.x - d->solAlt.x;
    double boy = d->sagUst.y - d->solAlt.y;
    return en * boy;
}

double cevre(const Dikdortgen *d) {
    double en = d->sagUst.x - d->solAlt.x;
    double boy = d->sagUst.y - d->solAlt.y;
    return 2 * (en + boy);
}

int main(void) {
    Dikdortgen d = {{1, 1}, {5, 4}};
    printf("Alan: %.1f, çevre: %.1f\n", alan(&d), cevre(&d));
    return 0;
}
```

Fonksiyonlar yapıyı değiştirmediği için `const Dikdortgen *` alır; iç içe üyelere `d->sagUst.x` biçiminde erişilir. Çıktı: `Alan: 12.0, çevre: 14.0`.

Soru 15.

Cevap:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char ad[20];
    int no;
    double ortalama;
} Ogrenci;

int ortalamayaGore(const void *x, const void *y) {
    const Ogrenci *a = x;
    const Ogrenci *b = y;
    if (a->ortalama > b->ortalama) return -1;
    if (a->ortalama < b->ortalama) return 1;
    return strcmp(a->ad, b->ad);
}

int main(void) {
    int n = 0;
    if (scanf("%d", &n) != 1 || n < 1) {
        return 1;
    }
    Ogrenci *liste = malloc(n * sizeof *liste);
    if (liste == NULL) {
        return 1;
    }
    double toplam = 0;
    for (int i = 0; i < n; i++) {
        scanf("%19s %d %lf", liste[i].ad, &liste[i].no, &liste[i].ortalama);
        toplam += liste[i].ortalama;
    }
    double ort = toplam / n;
    printf("Sınıf ortalaması: %.2f\n", ort);

    qsort(liste, n, sizeof liste[0], ortalamayaGore);
    for (int i = 0; i < n; i++) {
        if (liste[i].ortalama > ort) {
            printf("%-6s %d %.2f\n", liste[i].ad, liste[i].no,
                   liste[i].ortalama);
        }
    }
    free(liste);
    return 0;
}
```

Önce ortalama hesaplanır, sonra liste büyükten küçüğe sıralanır; sıralı listede ortalamanın üstündekiler başta olur. Girdi `4`, `Ayse 101 72.5`, `Mehmet 102 91.25`, `Can 103 64`, `Elif 104 85.75` için çıktı: `Sınıf ortalaması: 78.38`, `Mehmet 102 91.25`, `Elif 104 85.75`.

Soru 16.

Cevap:

```c
#include <stdio.h>

typedef enum { DAIRE, DIKDORTGEN } Tur;

typedef struct {
    Tur tur;
    union {
        double yaricap;
```

```
        struct { double en, boy; } kenar;
    } olcu;
} Sekil;

double alan(const Sekil *s) {
    switch (s->tur) {
        case DAIRE:
            return 3.14159 * s->olcu.yaricap * s->olcu.yaricap;
        case DIKDORTGEN:
            return s->olcu.kenar.en * s->olcu.kenar.boy;
    }
    return 0.0;
}

int main(void) {
    Sekil liste[3] = {
        {DAIRE, {.yaricap = 2.0}},
        {DIKDORTGEN, {.kenar = {3.0, 4.5}}},
        {DAIRE, {.yaricap = 1.0}}
    };
    for (int i = 0; i < 3; i++) {
        printf("Şekil %d alanı: %.2f\n", i + 1, alan(&liste[i]));
    }
    return 0;
}
```

`tur` üyesi hangi `union` üyesinin geçerli olduğunu söyler; `switch` yalnızca o üyeyi okur. Çıktı: `Şekil 1 alanı: 12.57`, `Şekil 2 alanı: 13.50`, `Şekil 3 alanı: 3.14`.

Bölüm 15 Önişlemci, Makrolar ve Çok Dosyalı Programlar

Soru 1.

Cevap: B. Makro `3 + 1 * 3 + 1` olarak açılır; çarpma önce yapıldığı için sonuç 3 + 3 + 1 = 7'dir. A, parantezli makronun vereceği değerdir.

Soru 2.

Cevap: C. Önişlemci yalnızca metin üzerinde çalışır. A yanlış: komutlar `;` ile bitmez. B yanlış: önişlemci derlemeden önce çalışır. D yanlış: `#define` bellekte yer ayırmaz. E yanlış: `#define`, `#if` gibi birçok komutu tanır.

Soru 3.

Cevap: D. Makro `{ ... }` olarak açılır ve ardındaki `;` ayrı bir boş deyim olur; `if` deyimi orada biter, `else` önünde `if` bulamaz. Çözüm gövdeyi `do { ... } while (0)` içine almaktır.

Soru 4.

Cevap: B. `-D` seçeneği makroyu komut satırından tanımlar; `-DAD=değer` biçimiyle değer de verilebilir.

Soru 5.

Cevap: D. `int sayac = 0;` bir tanımdır; başlık birden çok `.c` dosyasına girince her nesne dosyası değişkeni ayrı tanımlar. A ve B bildirimdir, C makrodur, E koruyuculu bir tip tanımıdır; hiçbiri bellekte yer ayırmaz.

Soru 6.

Cevap: B. Derleme aşaması prototip sayesinde geçer, ancak `hesapla`nın gövdesi `modul.c` içinde olduğundan o dosya komuta eklenmezse bağlayıcı tanımı bulamaz. A `redefinition` hatası, D ise bir uyarı üretirdi.

Soru 7.

Cevap: A. Fonksiyon dışındaki `static` iç bağlantı verir; ad yalnızca o `.c` dosyasında görünür. B yanlış, C ve D yerel değişkenlere ilişkindir, E yanlış: değer değiştirilebilir.

Soru 8.

Cevap: B. Komut satırları TAB ile başlamalıdır; boşluk `missing separator` hatası verir.

Soru 9.

Cevap: ilk satır `2 * (3 + 1)` = 8; ikinci satır `10 * 2 + 3` = 23 (makro parantezsizdir).

```
ÇIKTI
8
23
```

Soru 10.

Cevap: `SINIR` tanımlı olduğu için `MESAJ` "tanimli" olur; `SINIR > 3` doğrudur.

```
ÇIKTI
buyuk tanimli
```

Soru 11.

Cevap: `#x` ifadeyi metne çevirir, `a##b` `xy` adını üretir.

```
ÇIKTI
xy * 2 = 14
7
```

Soru 12.

Cevap: `main.c` içinde `struct Olcu` iki kez görülüyor, çünkü başlık koruyucusu yok. Başlık şöyle düzeltilir:

```
#ifndef OLCU_H
#define OLCU_H

struct Olcu {
    int en;
    int boy;
};

#endif
```

Soru 13.

Cevap: Parametre ve gövde parantezsizdir; `DAIRE_ALANI(2 + 1)` `3.14159 * 2 + 1 * 2 + 1 * 2 + 1` olur. Doğrusu:

```
#define DAIRE_ALANI(r) (3.14159 * (r) * (r))
```

Soru 14.

```
TERMİNAL
$ gcc -std=c17 -Wall -c a.c
$ gcc -std=c17 -Wall -c b.c
$ gcc a.o b.o -o uygulama
```

Soru 15.

```c
// ucgenAlani.h
#ifndef UCGENALANI_H
#define UCGENALANI_H

double ucgenAlani(double taban, double yukseklik);

#endif
```

```c
// ucgenAlani.c
#include "ucgenAlani.h"

double ucgenAlani(double taban, double yukseklik) {
    return taban * yukseklik / 2;
}
```

```c
// main.c
#include <stdio.h>
#include "ucgenAlani.h"

int main(void) {
    printf("Alan: %.1f\n", ucgenAlani(6, 4));
    return 0;
}
```

```
TERMİNAL
$ gcc -std=c17 -Wall -Wextra main.c ucgenAlani.c -o ucgen
$ ./ucgen
Alan: 12.0
```

Başlık yalnızca prototip taşır, gövde `.c` dosyasındadır.

Soru 16.

```c
#define MIN(a, b) ((a) < (b) ? (a) : (b))
```

`MIN(i++, 5)` çağrısı `((i++) < (5) ? (i++) : (5))` olur; `i` koşul kabul edilirse iki kez artar. Argüman gövdede birden çok kez geçtiği için yan etkili ifadeler verilmemelidir. Bunun yerine `static inline int min(int a, int b)` fonksiyonu kullanılabilir; argüman çağrıdan önce bir kez hesaplanır.

Bölüm 16 Bit Düzeyinde İşlemler

Soru 1.

Cevap: B. `1100 ^ 1010 = 0110`, yani 6. XOR yalnızca bitleri farklı olan konumlarda 1 üretir.

Soru 2.

Cevap: C. `~(1u << 3)` yalnızca 3. biti 0 olan bir maske verir; `&` ile o bit silinir, diğerleri korunur. A biti açar, B yalnızca o biti süzer, D tersler.

Soru 3.

Cevap: D. `3u << 4`, $3 \times 2^4 = 48$'dir.

Soru 4.

Cevap: A. `x & 1` son biti verir; tek sayıda 1, çift sayıda 0'dır. `x && 1` yalnızca `x`'in sıfırdan farklı olup olmadığını sorar; `x | 1` her zaman sıfırdan farklıdır.

Soru 5.

Cevap: C. İşaret biti -128 ağırlığındadır: -128 + 64 + 32 + 16 + 8 + 4 + 2 = -2. Aynı sonuç `~00000010 + 1 = 11111110` ile de bulunur.

Soru 6.

Cevap: B. 40 = `101000` ve iki bit açıktır. `x &= x - 1` her adımda en sağdaki 1'i siler: 40 -> 32 -> 0, iki tur.

Soru 7.

Cevap:

```
ÇIKTI
2 7 1 1
```

`6 & 3` = `110 & 011` = 2; `6 | 3` = 7; `6 && 3` = 1 (ikisi de sıfırdan farklı); `b << 1` = 6 olduğundan karşılaştırma doğrudur ve 1 verir.

Soru 8.

Cevap:

```
ÇIKTI
12
```

Sırayla: 1, ardından 9 (bit 0 ve 3), bit 0 terslenince 8, bit 2 açılınca 12.

Soru 9.

Cevap: İlk satır yanlıştır. `==`, `&`'den önce çalıştığı için ifade `x & (1 == 0)`, yani `x & 0` olur ve her zaman sıfırdır; koşul hiç sağlanmaz. Doğrusu ikinci satırdaki `(x & 1) == 0`'dır.

Soru 10.

Cevap: `1` işaretli `int` olduğundan `1 << 31` sonucu `int`'e sığmaz ve tanımsız davranıştır. Düzeltme: `unsigned int bayrak = 1u << 31;`.

Soru 11.

Cevap: `x & (1u << 3)` sonucu 3. bit açıksa 1 değil 8'dir; `== 1` karşılaştırması yanlış olur. Düzeltme:

```
int bit3Acik = ((x & (1u << 3)) != 0);
```

Soru 12.

Cevap:

```
ÇIKTI
-16 240
```

`c` önce 32 bitlik `int`'e yükselir (`0x0000000F`), `~` bütün 32 biti çevirir (`0xFFFFFFF0` = -16). `& 0xFF` ile yalnızca alt sekiz bit tutulunca 240 (`0xF0`) elde edilir.

Soru 13.

Cevap:

```
int dusukDortBit(unsigned int x) {
    return x & 0xF;
}
```

`0xF` alt dört biti 1 olan maskedir; `&` diğer bitleri siler.

Soru 14.

Cevap:

```
#include <stdio.h>

int bitAcikMi(unsigned int x, int n) {
    return (x & (1u << n)) != 0;
}

int main(void) {
    unsigned int x = 37;     // 100101
    for (int n = 0; n <= 5; n++) {
        printf("%d%c", bitAcikMi(x, n), n < 5 ? ' ' : '\n');
    }
    return 0;
}
```

37 = `100101` olduğundan çıktı `1 0 1 0 0 1` olur.

Soru 15.

Cevap:

```
void tersBitYaz(unsigned char x) {
    for (int i = 0; i < 8; i++) {
        putchar(((x >> i) & 1) ? '1' : '0');
    }
    putchar('\n');
}
```

12 = `00001100`; en düşük bitten başlayınca `00110000` yazılır.

Soru 16.

Cevap:

```
#include <stdio.h>

#define OKU (1u << 0)
#define YAZ (1u << 1)
#define SIL (1u << 2)

void durumYaz(unsigned int izin) {
    printf("OKU=%d YAZ=%d SIL=%d\n", (izin & OKU) != 0,
           (izin & YAZ) != 0, (izin & SIL) != 0);
}

int main(void) {
    unsigned int izin = OKU | YAZ;
    durumYaz(izin);
    izin |= SIL;
    durumYaz(izin);
    izin &= ~YAZ;
```

```
    durumYaz(izin);
    return 0;
}
```

Çıktı sırasıyla `OKU=1 YAZ=1 SIL=0`, `OKU=1 YAZ=1 SIL=1`, `OKU=1 YAZ=0 SIL=1` olur. Ekleme `|=`, silme `&= ~` ile yapılır.

Bölüm 17 Dosya İşlemleri

Soru 1.

Cevap: C. `"w"` kipi dosya yoksa oluşturur, varsa içeriğini `fopen` döner dönmez siler.

Soru 2.

Cevap: C. `"a"` kipi dosyayı korur ve yazmaları hep sona ekler.

Soru 3.

Cevap: C. Koşul okuma işleminin kendisidir; A, B ve D `feof` yüzünden son öğeyi bir kez fazla işler, E ise hiç bitmez.

Soru 4.

Cevap: B. `puts` metnin sonuna `\n` ekler, `fputs` eklemez.

Soru 5.

Cevap: B. `fgetc` 0-255 arasındaki karakter kodlarını ve ayrıca -1 olan `EOF`'u döndürür. `char`'a atanırsa `EOF` bir karakterle karışabilir.

Soru 6.

Cevap: D. `fread` başarıyla okunan öğe sayısını döndürür; tek kayıt istenip kayıt tam okunamazsa 0 döner. Döngü bu yüzden `== 1` ile denetlenir.

Soru 7.

Cevap: B. Sona gidip `ftell` ile konumu okumak dosyanın bayt cinsinden boyutunu verir.

Soru 8.

Cevap: B. Windows metin kipinde `\n` (10) baytı `\r\n` olarak yazılır; ikili veri içindeki 10 değerli her bayt bozulur.

Soru 9.

Cevap: `24`

`feof` tuzağı: 3, 5, 8 okunur ve toplam 16 olur. Son satır sonu yüzünden döngü bir tur daha çalışır, `fscanf` başarısız olur, `x` 8 olarak kalır ve tekrar toplanır: 3 + 5 + 8 + 8 = 24.

Soru 10.

Cevap:

```
ÇIKTI
3
12
```

Göstergeyi 8. bayta alıp bir `int` okuyoruz: dizinin 3. elemanı olan 3. Okumadan sonra gösterge 8 + 4 = 12. bayttadır.

Soru 11.

Cevap: `fgets` başarısız olursa işlev dosyayı kapatmadan döner. Düzeltme:

```
if (fgets(tampon, boyut, f) == NULL) {
    fclose(f);
    return 0;
}
```

Her çıkış yolunda dosya kapatılmalıdır.

Soru 12.

Cevap: (1) `fopen` sonucu `NULL` için denetlenmemiş; iki açışta da gerekir. (2) `c` değişkeni `char` tanımlı; `fgetc` sonucu `int` değişkende tutulmalıdır, aksi hâlde `EOF` ile gerçek bir karakter karışabilir. Ayrıca okunan dosya hiç kapatılmamıştır.

Soru 13.

Cevap: Yapının içindeki `ad` bir işaretçidir; `fwrite` metni değil, işaretçinin değerini (bir bellek adresini) yazar. Program yeniden çalıştığında bu adres geçersizdir. Çözüm, `char ad[40]` gibi sabit boyutlu dizi kullanmak ya da metni ayrı yazıp (önce uzunluk, sonra karakterler) üyeleri tek tek yazmaktır.

Soru 14.

Cevap:

```
#include <stdio.h>

int main(void) {
    FILE *f = fopen("satirlar.txt", "w");
    if (f == NULL) {
        perror("satirlar.txt");
        return 1;
    }
    char satir[100];
    for (int i = 0; i < 3; i++) {
        if (fgets(satir, sizeof satir, stdin) == NULL) {
            break;
        }
        fputs(satir, f);
    }
    fclose(f);

    f = fopen("satirlar.txt", "r");
    if (f == NULL) {
        perror("satirlar.txt");
        return 1;
    }
    int no = 1;
    while (fgets(satir, sizeof satir, f) != NULL) {
        printf("%d: %s", no++, satir);
    }
    fclose(f);
    return 0;
}
```

Satırlar önce yazılır, dosya kapatılır, sonra okuma kipinde yeniden açılır.

Soru 15.

Cevap:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *f = fopen("ornek.txt", "w");
    if (f == NULL) {
        perror("ornek.txt");
        return 1;
    }
    fputs("kisa\nbiraz daha uzun bir satir\norta satir\n", f);
    fclose(f);

    f = fopen("ornek.txt", "r");
    if (f == NULL) {
        perror("ornek.txt");
        return 1;
    }
    char satir[200], enUzun[200] = "";
    size_t enUzunLen = 0;
    while (fgets(satir, sizeof satir, f) != NULL) {
        satir[strcspn(satir, "\n")] = '\0';
        if (strlen(satir) > enUzunLen) {
            enUzunLen = strlen(satir);
            strcpy(enUzun, satir);
        }
    }
    fclose(f);
    printf("En uzun satır (%zu): %s\n", enUzunLen, enUzun);
    return 0;
}
```

Her satırın uzunluğu şimdiye kadarki en büyükle karşılaştırılır; daha uzunsa satır saklanır.

Soru 16.

Cevap:

```c
int sil(const char *ad) {
    FILE *f = fopen(DOSYA, "rb");
    FILE *g = fopen("gecici.dat", "wb");
    if (f == NULL || g == NULL) {
        if (f != NULL) fclose(f);
        if (g != NULL) fclose(g);
        return 0;
    }
    struct Kisi k;
    while (fread(&k, sizeof k, 1, f) == 1) {
        if (strcmp(k.ad, ad) != 0) {
            fwrite(&k, sizeof k, 1, g);
        }
    }
    fclose(f);
    fclose(g);
    remove(DOSYA);
    rename("gecici.dat", DOSYA);
    return 1;
}
```

Silinecek kayıt dışındakiler geçici dosyaya kopyalanır; eski dosya silinip geçici dosya onun adını alır. `main` içinde `sil("Mehmet"); listele();` çağrısı kalan iki kaydı gösterir.

Bölüm 18 Özyineleme (Recursion)

Soru 1.

Cevap: A. Temel durum `n <= 1` olduğundan `n = 1`, `n = 0` ve negatif değerler 1 döndürerek zinciri sonlandırır; pozitif `n` için çağrı `n - 1`'e iner. B'de temel durum yok, yığın taşar. C'de `n == 1` negatif ya da 0 girdiye hiç ulaşılamaz. D'de argüman `n + 1` ile temel durumdan uzaklaşır. E'de koşul ters yazılmıştır: `n > 1` için hemen 1 döner (yanlış sonuç), `n <= 1` için ise `f(n - 1)` ile sonsuza iner.

Soru 2.

Cevap: D. Çağrı sayısı `C(n) = 1 + C(n-1) + C(n-2)`, `C(0) = C(1) = 1` ile hesaplanır: 1, 1, 3, 5, 9, 15. Yani `fib(5)` için 15 çağrı yapılır. A (8), `fib(6)`'nın değeridir; E (25) `fib(6)`'nın çağrı sayısıdır.

Soru 3.

Cevap: B. Yığın `n = 6, 5, 4, 3, 2, 1` çerçeveleriyle en derin hâline ulaşır; `n = 1` temel durumdur ve kendini çağırmadan döner. Dolayısıyla 6 çerçeve vardır. A temel durumun çerçevesini saymayı unutur, C fazladan bir çerçeve ekler (`n = 0` çağrısı yapılmaz), D sonuçtur.

Soru 4.

Cevap: C. `H(n) = 2ⁿ - 1` olduğundan `H(5) = 31`. Adım adım: `H(1) = 1`, `H(2) = 3`, `H(3) = 7`, `H(4) = 15`, `H(5) = 2 · 15 + 1 = 31`. A (15) 4 diskin, E (63) 6 diskin hamle sayısıdır; D (32) `2⁵`'tir ve "-1" unutulmuştur.

Soru 5.

Cevap: A. Bellekleme her `fib(n)` değerini bir kez hesaplatır; çağrı sayısı `2n - 1`'e iner. B yanlış: her çağrının yerel değişkenleri kendisinindir. C yanlış: temel durumsuz fonksiyon derlenir (en fazla uyarı verilir), hata çalışma zamanında yığın taşması olarak çıkar. D yanlış: her özyinelemeli çözüm, gerekirse kendi yığınımızla birlikte döngüye çevrilebilir. E yanlış: çerçeve açma maliyeti yüzünden döngü genellikle biraz daha hızlıdır.

Soru 6.

Cevap: B. `f(n - 1, b + n)` çağrısından sonra yapılacak başka işlem yoktur; sonuç `biriken` parametresinde taşınır. A, D ve E'de çağrıdan sonra toplama, çarpma ya da `1 +` işlemi yapılır; C'de iki çağrının sonucu toplanır.

Soru 7.

Cevap:

```
ÇIKTI
3 2 1 1 2 3
```

Gidişte `3 2 1` yazılır (`f(0)` temel durumdur ve hiçbir şey yazmaz). Dönüşte çerçeveler ters sırayla kapanır ve çağrıdan sonraki `printf`'ler `1 2 3` yazar.

Soru 8.

Cevap:

```
ÇIKTI
12
1
```

`g(84, 36)`: `84 % 36 = 12` olduğundan `g(36, 12)`, `36 % 12 = 0` olduğundan `g(12, 0)` ve `b == 0` için 12 döner. `g(17, 5)`: `g(5, 2)`, `g(2, 1)`, `g(1, 0)` ve sonuç 1'dir (17 ile 5 aralarında asaldır). Fonksiyon Öklid algoritmasıdır.

Soru 9.

Cevap:

```
ÇIKTI
* * * * * * * *
```

Sekiz yıldız yazılır. Yıldızlar yapraklarda yazılır: `f(0)` ve `f(1)` birer yıldız yazar. Yıldız sayısı `Y(n) = Y(n-1) + Y(n-2)` kuralıyla büyür: `Y(0) = Y(1) = 1`, `Y(2) = 2`, `Y(3) = 3`, `Y(4) = 5`, `Y(5) = 8`.

Soru 10.

Cevap: `n + topla(n - 1);` ifadesinin sonucu atılır ve fonksiyon, `n != 0` durumunda hiçbir değer döndürmeden biter. Derleyici `warning: statement with no effect` ve `warning: control reaches end of non-void function` uyarılarını verir; sonuç tanımsızdır. Düzeltme `return` eklemektir:

```c
int topla(int n) {
    if (n == 0) {
        return 0;
    }
    return n + topla(n - 1);
}
```

Soru 11.

Cevap: `say(0)`: `n == 1` hiçbir zaman sağlanmaz, çünkü `0 / 2` yine 0'dır. Argüman temel duruma yaklaşmaz, çağrılar sonsuza gider ve yığın taşar. Negatif `n` için de aynı durum vardır. Temel durum `n <= 1` yapılmalıdır (ya da girdi önceden denetlenmelidir); `say(0)` bu durumda 0 döndürür.

```c
int say(int n) {
    if (n <= 1) {
        return 0;
    }
    return 1 + say(n / 2);
}
```

Soru 12.

Cevap: Yerel dizi her çağrıda yeniden oluşturulur ve global dizinin aksine sıfırlanmaz; içinde çöp değerler bulunur. Dizi ayrıca çağrıya özeldir, çağrılar arasında paylaşılmaz; yani önceki çağrıların kaydettiği sonuçlar görünmez. Bellekleme çalışmaz, çöp değerler okunduğu için sonuç yanlış olabilir (tanımsız davranış) ve her çerçeve fazladan 728 bayt tutar. Dizi global, `static` ya da çağıranın verdiği bir parametre olmalıdır.

Soru 13.

Temel durum `n == 0`'dır (boş toplam 0). `n` için toplam, `n * n` ile `n - 1` için toplamın toplamıdır.

```c
#include <stdio.h>

int kareToplami(int n) {
    if (n <= 0) {
        return 0;
    }
    return n * n + kareToplami(n - 1);
}

int main(void) {
    int n = 0;
    printf("n: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Geçersiz sayı.\n");
        return 1;
    }
```

```
    printf("1 ile %d arası karelerin toplamı: %d\n", n, kareToplami(n));
    return 0;
}
```

`n = 4` için çıktı: `1 ile 4 arası karelerin toplamı: 30`.

Soru 14.

Temel durum `bas >= son`'dur (boş ya da tek karakterli parça zaten ters). Aksi hâlde uçlardaki karakterler yer değiştirir ve iç parça için aynı işlem yapılır.

```
#include <stdio.h>
#include <string.h>

void tersCevir(char s[], int bas, int son) {
    if (bas >= son) {
        return;
    }
    char gecici = s[bas];
    s[bas] = s[son];
    s[son] = gecici;
    tersCevir(s, bas + 1, son - 1);
}

int main(void) {
    char s[] = "ozyineleme";
    tersCevir(s, 0, (int)strlen(s) - 1);
    printf("%s\n", s);
    return 0;
}
```

Çıktı: `emeleniyzo`.

Soru 15.

Collatz dizisi her başlangıç değeri için 1'e ulaşır (bu bugüne kadar kanıtlanmış değildir, ama denenen tüm değerlerde doğrudur). Ara değerler `int`'i aşabileceği için `long long` kullanılır.

```
#include <stdio.h>

long long adim(long long n) {
    if (n == 1) {
        return 0;
    }
    if (n % 2 == 0) {
        return 1 + adim(n / 2);
    }
    return 1 + adim(3 * n + 1);
}

int main(void) {
    printf("adim(6) = %lld\n", adim(6));
    printf("adim(27) = %lld\n", adim(27));
    return 0;
}
```

Çıktı: `adim(6) = 8` (6 → 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1) ve `adim(27) = 111`.

Soru 16.

İki temel durum vardır: satırın başı ve sonu (`k == 0 || k == n`). Diğer hücreler, üstteki iki hücrenin toplamıdır.

```c
#include <stdio.h>

int c(int n, int k) {
    if (k == 0 || k == n) {
        return 1;
    }
    return c(n - 1, k - 1) + c(n - 1, k);
}

int main(void) {
    for (int n = 0; n < 5; n++) {
        for (int k = 0; k <= n; k++) {
            printf("%d ", c(n, k));
        }
        printf("\n");
    }
    return 0;
}
```

Çıktı:

```
ÇIKTI
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Bu fonksiyon da Fibonacci gibi aynı alt problemleri tekrar tekrar hesaplar; büyük `n` için bellekleme gerekir.

Bölüm 19 Veri Yapılarına Giriş: Bağlı Liste, Yığın ve Kuyruk

Soru 1.

Cevap: A. Başa eklemede yeni düğüm yaratılır, `sonraki` alanı eski başa bağlanır ve baş işaretçisi güncellenir; adım sayısı liste uzunluğuna bağlı değildir.

Soru 2.

Cevap: B. "Geri al" son yapılan işlemi ilk geri alır (LIFO). A, C ve D kuyruk (FIFO) mantığıyla çalışır; E ise bir sıralama sorusudur.

Soru 3.

Cevap: `100 9 4 1` (sonda bir boşluk bulunur, ardından satır sonu). Önce 1, 4, 9, 16 itilir; `tepe--` ile 16 çıkarılmış sayılır; 100, 16'nın yerine yazılır. Dizi `1 4 9 100` olur ve tepeden tabana yazdırılır.

Soru 4.

Cevap: `1`. `son` her turda 1, 2, 0, 1, 2, 0, 1 değerlerini alır; yedi artırma sonunda 7 % 3 = 1 olur.

Soru 5.

Cevap: `free(p)` çağrısından sonra `p->sonraki` okunur; bu, serbest bırakılmış belleği kullanmaktır (tanımsız davranış). Bir sonraki düğümün adresi `free`'den önce saklanmalıdır:

```c
void bosalt(Dugum *bas) {
    Dugum *p = bas;
    while (p != NULL) {
        Dugum *sonraki = p->sonraki;
        free(p);
```

```
        p = sonraki;
    }
}
```

Çağıran taraftaki baş işaretçisi geçersiz kalacağı için, `Dugum **` alıp sonunda `*bas = NULL;` yazmak daha güvenlidir.

Soru 6.

Cevap: Parametre `bas`, çağıran taraftaki işaretçinin bir kopyasıdır; `bas = yeni;` yalnızca kopyayı değiştirir. Üstelik yeni düğüm kimsenin gösterdiği bir yer olmadığı için bellek sızar. Çözüm, baş işaretçisinin adresini almaktır:

```
void basaEkle(Dugum **bas, int veri) {
    Dugum *yeni = malloc(sizeof(Dugum));
    if (yeni == NULL) {
        return;
    }
    yeni->veri = veri;
    yeni->sonraki = *bas;
    *bas = yeni;
}
```

Çağrı `basaEkle(&liste, 5);` biçiminde yapılır.

Soru 7.

Cevap: D. `(` `[` `]` `{` `}` `)` sırasıyla açılıp ters sırayla kapanır. A ve C'de kapatma sırası bozuk, B'de kapanmamış parantez, E'de eşi olmayan kapatan parantez vardır.

Soru 8.

Cevap:

```
2 3
1
```

`p`, `b`'yi gösterir; `b.veri` 2, `b.sonraki->veri` ise `c`'nin 3'üdür. `b.sonraki = NULL` yapılınca `a.sonraki` hâlâ `b`'dir ve `b->sonraki` artık `NULL` olduğundan karşılaştırma doğrudur (1).

Soru 9.

Cevap: Kuyruk boşken de doluyken de `bas == son` olabilir; yalnızca indekslere bakarak bu iki durum ayırt edilemez. `adet` sayacı (`0` boş, `KAPASITE` dolu) bu belirsizliği giderir. Alternatif olarak dizide bir göz boş bırakılabilir.

Soru 10.

Cevap: B. Her karşılaştırma aralığı yarıya indirir; $2^{10} = 1024$ olduğundan 1000 eleman için en çok yaklaşık 10 karşılaştırma yeter.

Soru 11.

Cevap: `bas` `NULL` olur, ama `son` serbest bırakılmış düğümü göstermeye devam eder. Sonraki `enqueue`, kuyruğun boş olduğunu `son == NULL` ile anlayamaz ve `k->son->sonraki`'ne, yani serbest bırakılmış belleğe yazar (tanımsız davranış); yeni düğüm de `bas`'a bağlanmaz, dolayısıyla kuyruk yanlış çalışır.

Soru 12.

Cevap: B. Çift yönlü listede `onceki` işaretçisi sayesinde silme için önceki düğümü aramaya gerek kalmaz. A yanlıştır (daha fazla bellek kullanır), C diziye özgüdür, D ve E hiçbir listede doğru değildir.

Soru 13.

```c
#include <stdio.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

int enBuyuk(const Dugum *bas, int *sonuc) {
    if (bas == NULL) {
        return 0;
    }
    *sonuc = bas->veri;
    for (const Dugum *p = bas->sonraki; p != NULL; p = p->sonraki) {
        if (p->veri > *sonuc) {
            *sonuc = p->veri;
        }
    }
    return 1;
}

int main(void) {
    Dugum c = {4, NULL};
    Dugum b = {9, &c};
    Dugum a = {3, &b};
    int s;
    if (enBuyuk(&a, &s)) {
        printf("En büyük: %d\n", s);   // En büyük: 9
    }
    return 0;
}
```

Boş liste ile geçerli bir sonuç ayrımı dönüş değeriyle yapıldığı için, listede sıfır ya da negatif değerler olsa da fonksiyon doğru çalışır.

Soru 14.

```c
int silHepsi(Dugum **bas, int deger) {
    int sayac = 0;
    Dugum *p = *bas;
    Dugum *onceki = NULL;
    while (p != NULL) {
        Dugum *sonraki = p->sonraki;
        if (p->veri == deger) {
            if (onceki == NULL) {
                *bas = sonraki;          // baştaki düğüm
            } else {
                onceki->sonraki = sonraki;
            }
            free(p);
            sayac++;
        } else {
            onceki = p;                  // yalnızca silmediğimizde ilerler
        }
        p = sonraki;
    }
    return sayac;
}
```

`onceki`, silinen düğümde ilerletilmez; çünkü `onceki` hâlâ listede kalan son düğümü göstermelidir. Üst üste gelen eşleşmeler de böylece doğru işlenir.

Soru 15.

```c
#include <stdio.h>

#define AZAMI 100

int main(void) {
    char kelime[AZAMI];
    char yigin[AZAMI];
    int tepe = -1;

    printf("Bir kelime girin: ");
    if (scanf("%99s", kelime) != 1) {
        return 1;
    }
    for (int i = 0; kelime[i] != '\0'; i++) {
        yigin[++tepe] = kelime[i];       // push
    }
    printf("Ters hali: ");
    while (tepe >= 0) {
        putchar(yigin[tepe--]);          // pop
    }
    printf("\n");
    return 0;
}
```

Harfler girildiği sırayla itilir, ters sırayla çıkar. `%99s` okuma sınırı, diziyi taşmaktan korur (Bölüm 11).

Soru 16.

```c
Dugum *birlestir(Dugum *a, Dugum *b) {
    Dugum baslik = {0, NULL};      // geçici başlangıç düğümü (yığında)
    Dugum *son = &baslik;

    while (a != NULL && b != NULL) {
        if (a->veri <= b->veri) {
            son->sonraki = a;
            a = a->sonraki;
        } else {
            son->sonraki = b;
            b = b->sonraki;
        }
        son = son->sonraki;
    }
    son->sonraki = (a != NULL) ? a : b;   // kalan liste olduğu gibi eklenir
    return baslik.sonraki;
}
```

Geçici `baslik` düğümü, ilk düğümün özel durumunu ortadan kaldırır. Yeni düğüm yaratılmaz; mevcut düğümlerin `sonraki` alanları yeniden bağlanır. `1 -> 4 -> 6` ile `2 -> 5` listeleri `1 -> 2 -> 4 -> 5 -> 6` olur. İki girdi liste artık tek listenin parçasıdır; birleşmiş liste bir kez serbest bırakılmalıdır.

Bölüm 21 C'den C++'a: Tarih, Felsefe ve İlk Program

Soru 1.

Cevap: C. C++ da C gibi derlenen bir dildir; doğuş nedenleri arasında yorumlama yoktur. Diğer şıklar C++'ın yanıt verdiği sorunlardır.

Soru 2.

Cevap: B. Sanal tablo işaretçisi yalnızca `virtual` fonksiyon kullanıldığında bulunur; kullanmayan sınıf bu bedeli ödemez.

Soru 3.

C++ `void *` değerini örtük olarak `double *`'a dönüştürmez. Düzeltme: `double *dizi = (double *)malloc(10 * sizeof(double));` . C++'ta daha iyisi `new double[10]` ya da `std::vector<double>` kullanmaktır (Bölüm 26 ve 29).

Soru 4.

C programı `4` , C++ programı `1` yazar. C'de karakter sabiti `int` tipindedir (genelde 4 bayt), C++'ta `char` tipindedir (1 bayt).

Soru 5.

Cevap: C. C++'ta `void islem();` ile `void islem(void);` aynıdır; `islem(3)` çağrısı "too many arguments" hatası verir. C'de ise derlenir.

Soru 6.

`cout` `std` ad alanındadır. Düzeltme: `std::cout` yazmak ya da `#include` satırından sonra `using namespace std;` eklemek.

Soru 7.

Çıktı: `10 24 1` . `x + y` = 10, `x * y` = 24, `(x > y)` doğru olduğu için `1` yazılır.

Soru 8.

İkisi de alt satıra geçer. `std::endl` bunun yanında tamponu boşaltır (hemen hedefe gönderir). Çok satırlı döngüde `'\n'` tercih edilir, çünkü her satırda boşaltma yapmak yavaşlatır.

Soru 9.

Cevap: D. `std` standart kütüphanenin ad alanıdır. A yanlıştır (`using namespace std;` bir deyimdir ve `;` ile biter), B yanlıştır (tüm standart adlar için gerekir), C yanlıştır (başlıkta kullanılmaz), E yanlıştır (`using` ile `cout` yazılabilir).

Soru 10.

`using namespace std;` yüzünden `count` , hem global değişkeni hem de `std::count` algoritmasını gösterir; derleyici `reference to 'count' is ambiguous` hatası verir. Üç çözüm: (1) `using namespace std;` satırını silip `std::cout` yazmak, (2) değişkeni `sayac` gibi başka bir adla adlandırmak, (3) global değişkene `::count` ile başvurmak.

Soru 11.

`-std=c++17` : C++17 kurallarına göre derle. `-Wall` : yaygın şüpheli durumlar için uyarıları aç. `-Wextra` : ek uyarıları aç. `-o program` : çıktı dosyasının adını `program` yap.

Soru 12.

`201703` C++17'dir (2017, mart). `201103` olsaydı program C++11 kurallarıyla derleniyor demekti.

Soru 13.

Cevap: D. `.cs` C# kaynak dosyası uzantısıdır.

Soru 14.

```cpp
#include <iostream>
#include <string>

int main() {
    std::string ad = "Elif";
    int yas = 20;
    double boy = 1.68;
    std::cout << "Ad: " << ad << '\n';
    std::cout << "Yaş: " << yas << '\n';
    std::cout << "Boy: " << boy << '\n';
    return 0;
}
```

`std::string` için `<string>` başlığı gerekir. Her değer ayrı değişkende tutulup `<<` ile zincirlenir.

Soru 15.

```cpp
#include <iostream>

int main() {
    int a = 17, b = 5;
    std::cout << "Toplam: " << a + b << '\n';
    std::cout << "Fark: " << a - b << '\n';
    std::cout << "Çarpım: " << a * b << '\n';
    std::cout << "Bölüm: " << static_cast<double>(a) / b << '\n';
    return 0;
}
```

Bölümün gerçek sayı çıkması için işlenenlerden biri `double`'a çevrilir (Bölüm 22'de `static_cast` ayrıntılı işlenecek); C'deki `(double)a / b` yazımı da çalışır.

Soru 16.

```cpp
#include <iostream>
#include <cmath>

int main() {
    double r = 2.5;
    double pi = std::acos(-1.0);
    std::cout << "Alan: " << pi * r * r << '\n';
    return 0;
}
```

`std::acos(-1.0)` pi sayısını verir. Çıktı `Alan: 19.635` olur (`cout` varsayılan olarak 6 anlamlı basamak yazar).

Bölüm 22 Tipler, Değişkenler ve Operatörler: C++'ın Getirdikleri

Soru 1.

Cevap: A. C++'ta `'A'` bir `char` sabitidir, `sizeof` 1 verir. (C'de `int` olduğu için 4 çıkardı.)

Soru 2.

Cevap: C. `{}` ile ilk değer verirken `double`'dan `int`'e dönüşüm daraltıcıdır ve hatadır. `int a = 7.5;` sessizce 7 yapar; `double d{3};` ve `char e{65};` kayıpsızdır.

Soru 3.

Cevap: D. `auto` ile `= {...}` birleşimi `std::initializer_list<int>` tipini çıkarır. `auto y{5};` ise `int` olurdu.

Soru 4.

Cevap: B. `constexpr`, değerin derleme zamanında hesaplanabilmesini zorunlu kılar; `const` yalnızca sonradan değiştirilememesini sağlar ve ilk değeri çalışma zamanında belirlenebilir.

Soru 5.

Cevap:

```
ÇIKTI
100true
```

`a` 1, `b` 0 (boş `{}` `false` yapar), `a && b` 0 yazılır. `std::boolalpha` verildikten sonra `a || b` `true` olarak yazılır.

Soru 6.

Cevap:

```
ÇIKTI
4 4 6
```

`r = y;` yeniden bağlama değil değer atamasıdır: `x` 4 olur. Sonra `y = 6;` yalnızca `y`'yi değiştirir; `r` hâlâ `x`'e bağlıdır.

Soru 7.

Cevap:

```
ÇIKTI
312
```

İç blokta `n` 3, `::n` global 1 yazar. Blok bitince `main`'deki `n` (2) yazılır.

Soru 8.

Cevap: Gerçek hata yalnızca `auto q;` satırındadır: `auto` için ilk değer zorunludur (`auto q{3};` ya da `auto q = 3;`). `const int N{5}; int dizi[N];` C++'ta geçerlidir, çünkü sabit ifadeyle ilklenmiş `const` tamsayı sabit ifade sayılır (daha güvenli yazım `constexpr int N{5};`). `NULL` derlenir, ama C++'ta `nullptr` tercih edilir.

Soru 9.

Cevap: B. `static_cast` ilgisiz işaretçi tipleri arasında dönüşüm yapmaz (`invalid 'static_cast'` hatası verir); bunun için `reinterpret_cast` gerekir. Diğer dönüşümlerin hepsi `static_cast` ile yapılabilir.

Soru 10.

Cevap: D. Kayan noktalı tiplerde `min()`, en küçük pozitif normalleşmiş değeri verir (yaklaşık 2.2e-308). En negatif değer için `lowest()` kullanılır.

Soru 11.

Cevap:

```
ÇIKTI
B 66 66
```

`std::uint8_t` genellikle `unsigned char`'dır; `cout` onu karakter olarak yazar (`B`). Tekli `+` ve `static_cast<int>` onu sayıya çevirir.

Soru 12.

Cevap:

```
ÇIKTI
3
```

`using A::f;` ile `f()` `A::f`'yi çağırır (1); `B::f()` 2 döndürür. Toplam 3.

Soru 13.

Örnek çözüm:

```
#include <iostream>

int main() {
    constexpr double PI{3.14159};
    double yaricap{2.5};
    auto alan = PI * yaricap * yaricap;
    std::cout << alan << ' ' << static_cast<int>(alan) << '\n';
    return 0;
}
```

`auto`, `double` çıkarır; `static_cast<int>` ondalık kısmı atar. Çıktı: `19.6349 19`.

Soru 14.

Örnek çözüm:

```
#include <iostream>
#include <limits>

int main() {
    int enKucuk{std::numeric_limits<int>::min()};
    int enBuyuk{std::numeric_limits<int>::max()};
    long long bir{static_cast<long long>(enBuyuk) + 1};
    std::cout << enKucuk << ' ' << enBuyuk << ' ' << bir << '\n';
    return 0;
}
```

Toplama `long long` üzerinde yapıldığından taşma olmaz. Çıktı: `-2147483648 2147483647 2147483648`.

Soru 15.

Örnek çözüm:

```
#include <iostream>

namespace Geometri {
    double alan(double kenar) { return kenar * kenar; }
    double cevre(double kenar) { return 4 * kenar; }
}

int main() {
    using Geometri::alan;
    std::cout << alan(3.5) << ' ' << Geometri::cevre(3.5) << '\n';
    return 0;
}
```

`using` bildirimi `alan`'ı öneksiz kullanılabilir yapar; `cevre` önekle çağrılır. Çıktı: `12.25 14`.

Soru 16.

Örnek çözüm:

```
#include <cmath>
#include <iostream>
#include <limits>

void takas(double& a, double& b) {
    double gecici{a};
    a = b;
    b = gecici;
}

int main() {
    double x{1.5};
    double y{2.5};
    takas(x, y);
    std::cout << x << ' ' << y << '\n';

    double t{0.1 + 0.2};
    std::cout << std::boolalpha
              << (std::abs(t - 0.3) < std::numeric_limits<double>::epsilon())
              << '\n';
    return 0;
}
```

Referans parametreler çağıranın değişkenlerini değiştirir. Çıktı: `2.5 1.5` ve `true`.

Bölüm 23 iostream ile Girdi ve Çıktı

Soru 1.

Cevap: C. `fixed` kullanılmadığında `setprecision(2)` toplam anlamlı basamak sayısını belirler; iki anlamlı basamakla `3.14159` sayısı `3.1` olarak yazılır. `3.14` yanıtı için `fixed` gerekirdi.

Soru 2.

Cevap: E. `cin >> yas` yalnızca `20`'yi okur, Enter'ın bıraktığı `'\n'` tamponda kalır. `getline` ilk karakter olarak bu satır sonunu görür, boş bir satır okur ve hemen döner. Çözüm iki okuma arasına `cin.ignore(numeric_limits<streamsize>::max(), '\n');` koymaktır.

Soru 3.

Cevap:

```
[    123]
```

`setw(5)` yalnızca hemen ardından gelen `1` için geçerlidir: dört boşluk ve `1`. `2` ve `3` genişliksiz yazılır.

Soru 4.

Cevap: B. Önce `cin.clear()` hata bayraklarını temizler; hata durumundaki akışta `ignore` çalışmayacağı için bu ilk adımdır. Sonra `cin.ignore(...)` tamponda kalan bozuk girdiyi siler. Yalnızca `clear()` yazılırsa `abc` tamponda kalır ve sonraki okuma yine takılır.

Soru 5.

Cevap:

```
8.0 7.0 12
8
1.23e+04
```

`fixed` ve `setprecision(1)` ile `7.96` yuvarlanarak `8.0`, `7.04` ise `7.0` olur; `12` tamsayı olduğu için `fixed`'ten etkilenmez. `setprecision(0)` ile `7.96` noktadan sonra hane içermeyecek biçimde `8` yazılır. `scientific` ve `setprecision(2)`, noktadan sonra iki hanelik bilimsel gösterim verir: `1.23e+04`.

Soru 6.

Cevap: `<<` işleci, `==` işlecinden önce işlenir. Derleyici ifadeyi `(cout << "x beş mi: " << x) == (5 << '\n')` biçiminde okur ve bir akışı `int` ile karşılaştırmanın anlamı olmadığı için `no match for 'operator=='` hatası verir. Düzeltme, karşılaştırmayı parantez içine almaktır:

```
cout << "x beş mi: " << (x == 5) << '\n';
```

Soru 7.

Cevap: `a=3 b=0.9`. `cin >> a` `3`'ü okuyup `.` karakterinde durur (tamsayı noktayı kabul etmez). `.9` tamponda kalır ve `double` olan `b` bunu `0.9` olarak okur. Hiçbir okuma başarısız olmadığı için `fail()` de `false`'tur; veri yine de kullanıcının amaçladığı gibi değildir.

Soru 8.

Cevap: C. `Süt` UTF-8'de 4 bayttır (`ü` iki bayt). `setw` bayt saydığı için 6 - 4 = 2 boşluk yazar; ekranda ise yalnızca 3 harf + 2 boşluk görünür.

Soru 9.

Cevap:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double a = 0.0, b = 0.0;
    cout << "İki sayı: ";
    cin >> a >> b;
    cout << fixed << setprecision(2);
    cout << "Ortalama: " << (a + b) / 2 << '\n';
    return 0;
}
```

`7` ve `10` girildiğinde `Ortalama: 8.50` yazılır. İki ondalık için `fixed` ile `setprecision(2)` birlikte kullanılır.

Soru 10.

Cevap:

```
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>
using namespace std;

int main() {
    string ad;
    int yas = 0;
```

```
    cout << "Yaş: ";
    cin >> yas;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Ad soyad: ";
    getline(cin, ad);
    cout << left << setw(15) << ad << "(" << yas << ")\n";
    return 0;
}
```

Sayıdan sonra `getline` geldiği için araya `cin.ignore(...)` konur. `left` ve `setw(15)`, adı 15 karakterlik alana sola yaslar. `Ece Kaya` ASCII olduğu için hizalama doğrudur; Türkçe harfli adlarda bayt sorunu çıkar.

Soru 11.

Cevap:

```
#include <iostream>
using namespace std;

int main() {
    int sayi = 0;
    int adet = 0;
    int toplam = 0;
    int enBuyuk = 0;

    while (cin >> sayi) {
        if (adet == 0 || sayi > enBuyuk) {
            enBuyuk = sayi;
        }
        toplam += sayi;
        ++adet;
    }
    cout << "Adet: " << adet << ", toplam: " << toplam
         << ", en büyük: " << enBuyuk << '\n';
    return 0;
}
```

`while (cin >> sayi)` girdi bitene (ya da bozuk bir değere) kadar sürer. İlk sayı en büyük sayılır; bunun için `adet == 0` koşulu kullanıldı, çünkü hepsi negatif olabilecek girdilerde `0` ile başlamak yanlış sonuç verirdi.

Soru 12.

Cevap: Çarpım tablosu:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    for (int i = 1; i <= 5; ++i) {
        for (int j = 1; j <= 5; ++j) {
            cout << setw(4) << i * j;
        }
        cout << '\n';
    }
    return 0;
}
```

Her çarpım `setw(4)` ile ayrı ayrı genişletilir; `setw` kalıcı olmadığı için her değerin önünde yazılması gerekir. Pozitif sayı okuyan döngü:

```cpp
#include <iostream>
#include <limits>
using namespace std;

int main() {
    int sayi = 0;
    while (true) {
        cout << "Pozitif tamsayı: ";
        if (cin >> sayi && sayi > 0) {
            break;
        }
        if (cin.eof()) {
            cerr << "Girdi bitti.\n";
            return 1;
        }
        cout << "Geçersiz girdi.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    cout << "Okunan: " << sayi << '\n';
    return 0;
}
```

Geçersiz girdide önce `clear()`, sonra `ignore(...)` çağrılır. Girdi biterse (`eof`) döngü sonsuza kadar dönmesin diye `cerr`'e ileti yazılıp çıkılır.

Bölüm 24 Kontrol Yapıları: Koşullar ve Döngüler

Soru 1.

Cevap: B. `<<` karşılaştırmadan önce işlenir; parantez olmadan `a > b` ifadesi `(std::cout << a) > b` olarak okunur ve derlenmez (A, C). `>>` girdi operatörüdür (D); `std::cout(...)` geçerli bir çağrı değildir (E).

Soru 2.

Cevap:

```
ÇIKTI
B T T
```

`5 % 4 = 1` için `B`; `8 % 4 = 0` ve `12 % 4 = 0` için `T` yazılır. `else` kolu hiç çalışmaz, çünkü kalan 2 ya da 3 olan bir sayı yoktur. Her çıktıdan sonra bir boşluk bırakıldığı için satır boşlukla biter.

Soru 3.

Cevap: C. Başlatıcıda tanımlanan değişken koşulda, gövdede ve bütün `else if` / `else` kollarında görünür; `if` yapısı bitince yok olur.

Soru 4.

Cevap: B. `[[fallthrough]];` içinde deyim bulunan bir `case`'in sonunda, düşmenin bilerek yapıldığını belirtir. `break`'in yerini tutmaz (A), yalnızca niyeti işaretler; derleyici uyarısını da bastırır.

Soru 5.

Cevap:

```
ÇIKTI
iki üç
```

`x` 2 olduğundan `case 2` çalışır; `break` olmadığı için `case 3`'e düşülür ve orada `break` ile çıkılır. `bir` yazılmaz, `başka` yazılmaz.

Soru 6.

Cevap: `case 2` etiketine atlanırken `case 1`'deki `int yarim = k / 2;` tanımı atlanmış olur; C++ ilk değerli bir tanımın atlanmasına izin vermez ("jump to case label ... crosses initialization"). Çözüm, `case 1`'in kodunu süslü parantez içine almaktır:

```
case 1: {
    int yarim = k / 2;
    std::cout << yarim << '\n';
    break;
}
```

Süslü parantez `yarim`'in kapsamını `case 1`'e hapseder; başka bir `case` onu atlayamaz. Alternatif olarak değişken `switch`'ten önce tanımlanabilir.

Soru 7.

Cevap:

```
ÇIKTI
2 4 6
```

İlk döngüde `x` bir kopyadır; 100 eklemek vektörü etkilemez. İkinci döngüde `auto&` elemanın kendisidir, her eleman 2 ile çarpılır.

Soru 8.

Cevap: B. Büyük bir tipi (`std::string`) yalnızca okuyacaksak `const auto&` kopyayı önler. A için `auto&`, C için indeksli `for`, D için indeksli `for` ya da başka bir yöntem gerekir (döngü sırasında boyut değiştirmek tanımsız davranıştır); E'de ise aralık tabanlı `for` hiç derlenmez.

Soru 9.

Cevap: `dizi` parametresi fonksiyona girerken `int*` olur ve boyut bilgisi kaybolur; aralık tabanlı `for` `begin`/`end` bulamadığı için derlenmez. Çözüm: indeksli döngü ya da kapsayıcı kullanmak.

```
int topla(int dizi[], int n) {
    int t = 0;
    for (int i = 0; i < n; ++i) {
        t += dizi[i];
    }
    return t;
}
```

Başka bir çözüm, parametreyi `const std::vector<int>&` yapmaktır; bu durumda aralık tabanlı `for` çalışır (Bölüm 25 ve 26).

Soru 10.

Cevap: B. `i` işaretli `int`, `v.size()` işaretsiz `std::size_t`'tir; karşılaştırma `-Wsign-compare` uyarısı verir. Çözüm: `std::size_t i`.

Soru 11.

Cevap:

```
ÇIKTI
#...
##..
###.
```

Her satırda `i` tane `#` ve `4 - i` tane `.` yazılır; satır uzunluğu hep 4'tür.

Soru 12.

Cevap: `<<`, `==` operatöründen önce işlenir; ifade `(std::cout << a) == b` olarak okunur ve akış nesnesini bir sayıyla karşılaştırmaya çalıştığı için derlenmez. Doğru yazım: `std::cout << (a == b);`.

Soru 13.

Cevap:

```
ÇIKTI
6 15
```

18 ile 24'ün EBOB'u 6'dır; 3 ile 5'in EKOK'u 15'tir.

Soru 14.

Örnek çözüm:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    for (int x{}; std::cin >> x && x != 0;) {
        v.push_back(x);
    }
    int pozitifToplam{};
    int negatifAdet{};
    for (int x : v) {
        if (x > 0) {
            pozitifToplam += x;
        } else {
            ++negatifAdet;
        }
    }
    std::cout << "Pozitiflerin toplamı: " << pozitifToplam << '\n';
    std::cout << "Negatif sayısı: " << negatifAdet << '\n';
    return 0;
}
```

Okuma koşulu hem `cin >> x` başarısını hem de 0 sentinel değerini denetler; 0 vektöre eklenmediği için ikinci döngüde yalnızca pozitif ve negatif sayılar bulunur.

Soru 15.

Örnek çözüm:

```
#include <iostream>

int main() {
    if (int n{}; std::cin >> n) {
        std::cout << n << (n % 2 == 0 ? " çift\n" : " tek\n");
    } else {
        std::cout << "Geçersiz girdi\n";
```

```
    }
    return 0;
}
```

`n` yalnızca `if` yapısında yaşar. Harf girilirse okuma başarısız olur ve `else` kolu çalışır.

Soru 16.

Örnek çözüm:

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> sehirler{"Ankara", "Van", "Eskişehir", "Rize"};
    int uzun{};
    for (const auto& s : sehirler) {
        std::cout << s << '\n';
        if (s.size() > 5) {
            ++uzun;
        }
    }
    std::cout << "5 karakterden uzun: " << uzun << '\n';
    return 0;
}
```

`const auto&` metinleri kopyalamadan dolaşır. `size()` bayt sayısını verir; `Eskişehir` içindeki `ş` UTF-8'de iki bayt olduğu için uzunluğu 10 çıkar (karakter ve bayt ayrımı Bölüm 27'de). Burada sonucu etkilemez: `Ankara` ve `Eskişehir` 5'ten uzundur, yanıt 2'dir.

Bölüm 25 Fonksiyonlar: C++'ın Getirdikleri

Soru 1.

Cevap: C. `int&` parametresi adı olan bir değişkene bağlanır. A ve B'de argüman geçici değerdir (`5`, `3 + 4`); E'de `x + 1` de geçici bir sonuçtur ve `int&`'a bağlanamaz (`cannot bind non-const lvalue reference`). D'de `&x` bir `int*` verir; `int*` tipi `int&` parametresine uymaz.

Soru 2.

Cevap: C. `const std::string&` metnin kopyalanmasını önler ve `const` olduğu için fonksiyon metni yanlışlıkla değiştiremez. A ve B `const` olmayan referansın özellikleridir ve `const &` için yanlıştır. D yanlış: referans boş olamaz. E yanlış: sabit referans geçici değerleri (metin sabitleri gibi) de kabul eder; bu `const &`'nın önemli bir avantajıdır.

Soru 3.

Cevap: D. `f(int)` ve `f(double)` farklı imzalara sahiptir; `f(1)` tam eşleşmeyle `int` sürümüne gider. A'da yalnızca dönüş tipi farklıdır, derleme hatası verir. B'de değerle geçirilen parametredeki `const` imzayı değiştirmez; iki bildirim aynı fonksiyondur ve ikisi tanımlanırsa yeniden tanımlama hatası olur. C'de `f(1)` hem `f(int)` hem `f(int, int = 0)` ile uyuşur: belirsiz çağrı. E'de `1` argümanı `long` ve `short`'a aynı derecede iyi (standart dönüşüm) uyar; belirsiz çağrı.

Soru 4.

Cevap: C. Varsayılan değerler sağdan sola verilmelidir: varsayılanı olan bir parametrenin sağındaki bütün parametrelerin de varsayılanı olmalıdır. A'da `b`, B'de `c`, D'de `b` varsayılansız kalmıştır. E'de de `int = 1` ardından varsayılansız parametreler geldiği için geçersizdir.

Soru 5.

Cevap: E. Hiçbir şey yakalamayan lambda, aynı imzaya sahip düz fonksiyon işaretçisine dönüştürülebilir. A yanlış: `constexpr` fonksiyon çalışma zamanında sıradan fonksiyon gibi de çağrılabilir. B yanlış: `return` ifadeleri farklı tip çıkarırsa `inconsistent deduction for auto return type` hatası alınır. C yanlış: yalnızca uyarı verilir. D yanlış: `inline` bir öneridir, derleyici uymak zorunda değildir.

Soru 6.

Cevap: `a` değerle geçer, `b` referansla geçer. İlk çağrıda `a` (x'in kopyası) 11 olur ama `x` 1 kalır; `b` yani `y` 11 olur. İkinci çağrı `degistir(y, x)`: `a`, `y`'nin kopyasıdır (11, 21 olur ve atılır); `b` ise `x`'e bağlıdır, 1'den 11 olur.

```
ÇIKTI
1 11
11 11
```

Soru 7.

Cevap: Her çağrıda en iyi aday seçilir. `'a'` tam eşleşmeyle `f(char)` (C). `97` tam eşleşmeyle `f(int)` (A). `97.0` tam eşleşmeyle `f(double)` (B). `2.5f` bir `float`'tır; `float` → `double` terfi, `float` → `int` ise standart dönüşümdür; terfi kazanır (B). `true` için `bool` → `int` terfisi kazanır (A).

```
ÇIKTI
CABBA
```

Soru 8.

Cevap: Varsayılan değerler prototipte yazılmıştır ve çağrılarda eksik argümanların yerine konur. `topla(1)` = `topla(1, 10, 100)` = 111. `topla(1, 2)` = `topla(1, 2, 100)` = 103. `topla(1, 2, 3)` = 6.

```
ÇIKTI
111 103 6
```

Soru 9.

Cevap: `deger` lambda'sı `n`'nin lambda yaratıldığındaki değerini (5) kopyalar; sonradan `n = 20` yapılması onu etkilemez, 10 döner. `bag` lambda'sı `n`'ye referansla bağlıdır; çağrıldığında `n` 20 olduğu için 40 döner.

```
ÇIKTI
10 40
```

Soru 10.

Cevap: `a` ve `b` değerle geçirilen parametrelerdir, yani fonksiyonun yerel kopyalarıdır. Fonksiyon bu kopyalardan birine referans döndürüyor; fonksiyon bitince kopyalar yok olur ve dönen referans ortadan kalkmış bir değişkeni gösterir (tanımsız davranış; `g++ -Wall` `reference to local variable returned` uyarısı verir). Düzeltmenin iki yolu vardır. Değerle döndürmek: `int enBuyuk(int a, int b)`. Ya da referans döndürmeyi korumak için parametreleri de referans yapmak: `int& enBuyuk(int& a, int& b)`; bu durumda dönen referans çağıranın değişkenini gösterir.

Soru 11.

Cevap: `yaz(1)` çağrısı hem `yaz(int)` hem `yaz(int, int = 0)` ile uyuşur; derleyici ikisinden birini seçemez ve `call of overloaded 'yaz(int)' is ambiguous` hatası verir. Fonksiyonların tanımı tek başına hata değildir; sorun çağrıdadır. İki düzeltme: (1) ikinci fonksiyondaki varsayılan değeri kaldırmak (`void yaz(int a, int b)`); bu durumda `yaz(1)` yalnızca birinciye uyar. (2) Fonksiyonlardan birinin adını değiştirmek (örneğin `yazCift`).

Soru 12.

Cevap: Eski `a` değerini geçici bir değişkende saklamak gerekir; yoksa ilk atamada kaybolur. Referans parametreler çağıranın değişkenlerini doğrudan değiştirir.

```
#include <iostream>

void dondur(int& a, int& b, int& c) {
    int eskiA = a;
    a = b;
    b = c;
    c = eskiA;
}

int main() {
    int x{1};
    int y{2};
    int z{3};
    dondur(x, y, z);
    std::cout << x << ' ' << y << ' ' << z << '\n';
    return 0;
}
```

```
ÇIKTI
2 3 1
```

Soru 13.

Cevap: Üç fonksiyon aynı adı taşır, imzaları farklıdır (iki `int`, üç `int`, `const std::vector<int>&`). `(a + b) / 2.0` ifadesindeki `2.0` bölmenin `double` olmasını sağlar; `vector` sürümünde ise `toplam` `double`'a çevrilir.

```
#include <iostream>
#include <vector>

double ortalama(int a, int b) {
    return (a + b) / 2.0;
}

double ortalama(int a, int b, int c) {
    return (a + b + c) / 3.0;
}

double ortalama(const std::vector<int>& sayilar) {
    int toplam{0};
    for (int s : sayilar) {
        toplam += s;
    }
    return static_cast<double>(toplam) / sayilar.size();
}

int main() {
    std::vector<int> liste{4, 8, 15, 16};
    std::cout << ortalama(3, 4) << '\n';
    std::cout << ortalama(3, 4, 8) << '\n';
    std::cout << ortalama(liste) << '\n';
    return 0;
}
```

```
ÇIKTI
3.5
5
10.75
```

Soru 14.

Cevap: `us` varsayılan olarak 2'dir; `kuvvet(3)` 3'ün karesini verir. `constexpr` fonksiyonda döngü C++14'ten beri kullanılabilir. `static_assert` koşulu derleme zamanında sınar; `kuvvet(2, 10)` sabit argümanlarla çağrıldığı için bu mümkündür.

```
#include <iostream>

constexpr double kuvvet(double taban, int us = 2) {
    double sonuc = 1.0;
    for (int i = 0; i < us; i++) {
        sonuc *= taban;
    }
    return sonuc;
}

static_assert(kuvvet(2, 10) == 1024, "kuvvet yanlis");

int main() {
    std::cout << kuvvet(3) << '\n';
    std::cout << kuvvet(2, 10) << '\n';
    std::cout << kuvvet(1.5, 3) << '\n';
    return 0;
}
```

```
ÇIKTI
9
1024
3.375
```

Soru 15.

Cevap: Parametre `std::function<int(int)>` olduğu için hem yakalamasız hem yakalamalı lambda kabul edilir. İlk çağrıda (5 + 3) + 3 = 11; ikinci çağrıda (5 × 4) × 4 = 80.

```
#include <functional>
#include <iostream>

int ikiKez(int deger, const std::function<int(int)>& islem) {
    return islem(islem(deger));
}

int main() {
    int carpan{4};
    std::cout << ikiKez(5, [](int n) { return n + 3; }) << '\n';
    std::cout << ikiKez(5, [carpan](int n) { return n * carpan; }) << '\n';
    return 0;
}
```

```
ÇIKTI
11
80
```

Bölüm 26 Diziler, std::array ve std::vector

Soru 1.

Cevap: C. Parantez ve süslü parantez farklıdır: `v{3, 3}` elemanları 3 ve 3 olan iki elemanlı vector'dür, `w(3, 3)` ise üç tane 3 tutar. Birinci tanım geçerlidir (E yanlış).

Soru 2.

Cevap: D. Fonksiyon parametresi olarak `int a[]` aslında `int*` demektir. `std::size` dizi tipinin boyutunu tipten okur; işaretçide böyle bir bilgi olmadığından derleme hatası verir. Bu, `sizeof(a) / sizeof(a[0])` kullanıldığında sessizce yanlış sonuç çıkmasından daha güvenlidir.

Soru 3.

Cevap: C. `clear()` bütün elemanları siler ve tüm yineleyicileri geçersiz kılar. `v[0] = 9`, `at`, `size` ve `front` yapıyı değiştirmez; bunlardan sonra `it` kullanılabilir.

Soru 4.

Cevap: C. Boyut `std::array`'in tipinin parçasıdır; `std::array<int, 3>` ile `std::array<int, 4>` farklı tiplerdir ve birbirine atanamaz. Derleme hatası oluşur, çalışma anına gelinmez.

Soru 5.

Cevap: C. `size()` işaretsiz `size_t` döndürür; `0 - 1` işaretsiz aritmetikte -1 değil, tipin en büyük değeridir (64 bitte 18446744073709551615). İfade geçerlidir ve istisna fırlatmaz.

Soru 6.

Adım adım: `{5, 1, 4}` -> `push_back(9)`: `{5, 1, 4, 9}` -> `insert` 1 numaralı konuma 7: `{5, 7, 1, 4, 9}` -> `erase(begin()+3)` 4'ü siler: `{5, 7, 1, 9}` -> `resize(6, 2)`: `{5, 7, 1, 9, 2, 2}`. Boyut 6, ilk 5, son 2; toplam 26; `v[2] = 1` olduğundan `v[v[2]] = v[1] = 7`.

```
ÇIKTI
6 5 2
26
7
```

Soru 7.

`a` kopyayı değiştirir, `v` `{0}` kalır; `c(v)` boyut 1 ve son eleman 0 yazar. `b` gerçek vector'e 2 ekler ve `v[0]`'ı 5 yapar: `{5, 2}`; `c(v)` bu kez `2 2` yazar. Son satır `v[0] = 5`.

```
ÇIKTI
1 0
2 2
5
```

Soru 8.

`t` önce `{1,1}, {1,1}, {1,1}`; `t[1][0] = 4` ikinci satırı `{4,1}` yapar; `t[2].push_back(9)` üçüncü satırı `{1,1,9}` yapar. Toplam: 2 + 5 + 11 = 18. `t.size()` 3, `t[2].size()` 3; `t.at(1).at(1)` 1'dir.

```
ÇIKTI
3 3
18
1
```

Soru 9.

Sıralı vector `{1, 3, 4, 8, 8}`; 8 iki kez geçer; 4'ün indeksi 2'dir. Ters çevrilince `{8, 8, 4, 3, 1}`: ilk eleman 8, en küçük 1. 5 yoktur.

```
ÇIKTI
2
2
8 1
yok
```

Soru 10.

`for (int x : v)` döngüsü arka planda `begin`/`end` yineleyicileriyle çalışır; döngü içinde `erase` çağırmak bu yineleyicileri geçersiz kılar ve tanımsız davranıştır. Üstelik silme sonrası elemanlar kaydığı için bir sonraki eleman atlanır. Doğrusu yineleyici kalıbıdır:

```
for (auto it = v.begin(); it != v.end();) {
    if (*it % 2 == 0) {
        it = v.erase(it);
    } else {
        ++it;
    }
}
```

Soru 11.

`i` işaretsizdir; `i >= 0` her zaman doğrudur, `0`'dan bir eksiltilince en büyük `size_t` değerine sarılır ve döngü hiç bitmez (`v[i]` sınır dışına çıkar). Boş vector'de `v.size() - 1` de bozuktur. Doğrusu:

```
for (std::size_t i = v.size(); i > 0; i--) {
    std::cout << v[i - 1] << ' ';
}
```

Soru 12.

İkinci satır (`int a[n];`): `n` sabit olmadığından değişken uzunluklu dizidir, standart C++'ta yoktur (g++ eklenti olarak kabul eder). Üçüncü satır: listede dizi boyutundan fazla eleman var, derleme hatası. Altıncı satır: `2.5`'in `int`'e daralması süslü parantezde hatadır. `c` ve `d` tanımları geçerlidir (`d = c` kopyalar).

Soru 13.

`std::max_element` yineleyici döndürdüğü için `*` ile açılır; ters yazdırmada `size_t` sayacı `i > 0` ve `v[i - 1]` kalıbıyla kullanılır.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    int n = 0;
    std::cout << "Kaç sayı? ";
    std::cin >> n;
    std::vector<int> v(n);
    std::cout << "Sayılar: ";
    for (int i = 0; i < n; i++) {
        std::cin >> v[i];
    }
    std::cout << "Ters:";
    for (std::size_t i = v.size(); i > 0; i--) {
```

```
        std::cout << ' ' << v[i - 1];
    }
    std::cout << '\n';
    int cift = 0;
    for (int x : v) {
        if (x % 2 == 0) {
            cift++;
        }
    }
    std::cout << "Çift sayı: " << cift << '\n';
    std::cout << "En büyük: " << *std::max_element(v.begin(), v.end()) << '\n';
    return 0;
}
```

Soru 14.

Matris `vector<vector<int>>` içinde kurulur; transpozu yazdırmak için döngüler yer değiştirir (dış döngü sütun, iç döngü satır).

```
#include <iostream>
#include <vector>

int main() {
    int satir = 0;
    int sutun = 0;
    std::cout << "Satır ve sütun: ";
    std::cin >> satir >> sutun;

    std::vector<std::vector<int>> m(satir, std::vector<int>(sutun));
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            std::cin >> m[i][j];
        }
    }

    std::cout << "Transpoz:\n";
    for (int j = 0; j < sutun; j++) {
        for (int i = 0; i < satir; i++) {
            std::cout << m[i][j] << (i + 1 < satir ? " " : "");
        }
        std::cout << '\n';
    }
    return 0;
}
```

Soru 15.

Her yeni sayı için `find`, sayının daha önce eklenip eklenmediğine bakar; `end()` dönerse sayı yenidir ve `push_back` ile eklenir. Böylece ilk geliş sırası korunur.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> benzersiz;
    int x = 0;
    std::cout << "Sayılar (0 ile biter): ";
    while (std::cin >> x && x != 0) {
        if (std::find(benzersiz.begin(), benzersiz.end(), x)
            == benzersiz.end()) {
            benzersiz.push_back(x);
```

```
        }
    }
    std::cout << "Tekrarsız:";
    for (int y : benzersiz) {
        std::cout << ' ' << y;
    }
    std::cout << '\n';
    return 0;
}
```

Bölüm 27 std::string ve Karakter İşlemleri

Soru 1.

Cevap: C. `std::toupper` `int` döndürür. İlk ifade `cout`'a `int` olarak gittiği için `'B'`'nin kodu olan 66 yazılır; ikinci ifadede sonuç `static_cast<char>` ile karaktere çevrildiği için `B` yazılır. A her iki yerde de karakter bekler. B `toupper`'ı hiç çalıştırmamış gibi düşünür. D `'b'`'nin kodudur (98), oysa `'B'` = 66. E ikinci ifadedeki dönüştürmeyi görmez.

Soru 2.

Cevap: E. `find` bulamayınca özel bir değer olan `std::string::npos` döndürür; bu, `std::size_t`'in alabileceği en büyük sayıdır. A yanlış: 0 "başta bulundu" demektir. B yanlış: değer `-1`'in işaretsiz hâlidir; ekrana yazdırılırsa `-1` değil çok büyük bir sayı çıkar. C'nin `size()` ile ilgisi yoktur. D yanlış: `find` istisna fırlatmaz (istisna fırlatan `at()` ve `substr()`'dır).

Soru 3.

Cevap: A. `"programlama"` üzerinde `erase(0, 3)` baştaki `pro`'yu siler: `gramlama` (8 karakter). `replace(0, 4, "XY")` baştaki dört karakteri (`gram`) iki karakterlik `XY` ile değiştirir: `XYlama`. Uzunluk 8 - 4 + 2 = 6. B uzunluğun değişmediğini sanır. C `erase`'i yanlış yerden yapar. D `replace`'te beş karakter değiştirilmiş gibi hesaplar. E ilk uzunluğu (11) taşır.

Soru 4.

Cevap: D. `ç` UTF-8'de 2 bayttır; `a` ve `y` birer bayt: toplam 4. `size()` karakter değil bayt sayar. A karakter sayısını verir. B ve C hiçbir hesaba uymaz. E yanlış, kod normal derlenir.

Soru 5.

Cevap: B. `std::stoi` baştaki sayıyı (42) alır ve kalan `abc` kısmını yok sayar; 42 + 1 = 43. A, 1 eklenmesini unutur. C, `stoi`'nin metin döndürdüğünü sanır; `stoi` `int` döndürür. D yanlış, kod derlenir. E, `stoi("abc")` gibi sayıyla başlamayan metinlerde olur; burada metin rakamla başlıyor.

Soru 6.

Cevap: C. `std::string("Merhaba")` bir `std::string` olduğu için `+ " dünya"` geçerlidir. A ve B'de ilk toplama iki ham string sabiti arasındadır ve derlenmez. D'de `'a' + 'b'` bir `int`'tir; `std::string` bir `int`'ten oluşturulamaz. E'de `std::string` için de ham diziler için de çıkarma işleci yoktur.

Soru 7.

Cevap:

```
ÇIKTI
1 3 4
balla 5
all
```

`banana` indeksleri: b=0, a=1, n=2, a=3, n=4, a=5. `find("an")` = 1, `rfind("an")` = 3, `find('n', 3)` = 4. `erase(1, 2)` `an` karakterlerini siler: `bana`. Şimdi `find('n')` = 2 ve `replace(2, 1, "ll")` `balla` verir (uzunluk 5). `substr(1, 3)` = `all`.

Soru 8.

Cevap:

```
ÇIKTI
6 ABC
98 b
```

Döngü rakamları değerlerine çevirip toplar: 1 + 2 + 3 = 6. Harfleri büyütüp `harfler`'e ekler: `ABC`. İkinci satırda `s[0] + 1` bir `int`'tir: `'a'` = 97, 97 + 1 = 98. `static_cast<char>` ile karaktere çevrilince `b` olur.

Soru 9.

Cevap:

```
ÇIKTI
5[] 0
Ali
```

`std::cin >> n` yalnızca `5`'i okur; Enter'ın `'\n'` karakteri tamponda kalır. İlk `getline` bu `'\n'`'i görüp hiçbir şey beklemeden boş metinle döner: `ad` boş, uzunluğu 0. `Ali` tamponda bekler ve ikinci `getline` onu okur. Düzeltmek için `std::cin >> n;` satırından sonra `std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');` yazılırdı.

Soru 10.

Cevap: `std::size_t` işaretsizdir ve hiçbir zaman negatif olamaz. `i` 0 iken `i--` onu çok büyük bir sayıya taşır; `i >= 0` her zaman doğrudur. Döngü bitmez ve `s[i]` sınırın çok dışına çıkar. Düzeltme:

```
std::string s{"abc"};
for (std::size_t i = s.size(); i > 0; i--) {
    std::cout << s[i - 1];
}
```

`i` "kaç karakter kaldı" anlamına gelir ve karaktere `i - 1` ile erişilir. Çıktı `cba` olur.

Soru 11.

Cevap: Hata `if (s.find("ab"))` satırındadır. `find` bulamazsa `npos` döner; bu sıfırdan farklı olduğu için koşul doğru çıkar ve fonksiyon `true` döner. `"ab"` metnin başındaysa 0 döner, koşul yanlış çıkar ve fonksiyon `false` döner. Yani sonuç tersine çalışır. Düzeltme:

```
bool abVarMi(const std::string& s) {
    return s.find("ab") != std::string::npos;
}
```

Soru 12.

Cevap:

```
ÇIKTI
C++ dilini ogrenmek
C++ dilini ogren
C++ dilini bil 14
dilin
```

`insert(3, " dilini")` üçüncü indeksin önüne ekleme yapar: `C++ dilini ogrenmek`. `erase(s.size() - 3)` sondaki `mek` kısmını siler: `C++ dilini ogren`. `replace(...)` `ogren` (beş karakter) yerine `bil` koyar: `C++ dilini bil`, uzunluk 14. İlk boşluk 3. indekstedir; `substr(4, 5)` `dilin` verir.

Soru 13.

Cevap:

```cpp
#include <cctype>
#include <iostream>
#include <string>

int main() {
    std::string satir;
    int harf = 0;
    int rakam = 0;
    int bosluk = 0;

    std::cout << "Bir satır yazın: ";
    std::getline(std::cin, satir);

    for (char c : satir) {
        unsigned char b = static_cast<unsigned char>(c);
        if (std::isalpha(b)) {
            harf++;
        } else if (std::isdigit(b)) {
            rakam++;
        } else if (b == ' ') {
            bosluk++;
        }
    }
    std::cout << "Harf: " << harf << '\n';
    std::cout << "Rakam: " << rakam << '\n';
    std::cout << "Boşluk: " << bosluk << '\n';
    return 0;
}
```

`getline` satır sonunu metne koymadığı için ayrıca temizlemek gerekmez. `else if` zinciri her karakteri tek bir sınıfa atar; hiçbirine uymayan (`!`) sayılmaz.

Soru 14.

Cevap:

```cpp
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

int main() {
    std::string satir;
    std::cout << "Cümle: ";
    std::getline(std::cin, satir);

    std::istringstream oku{satir};
```

```
    std::vector<std::string> kelimeler;
    std::string kelime;
    while (oku >> kelime) {
        kelimeler.push_back(kelime);
    }

    for (std::size_t i = kelimeler.size(); i > 0; i--) {
        std::cout << kelimeler[i - 1];
        if (i > 1) {
            std::cout << ' ';
        }
    }
    std::cout << '\n';
    return 0;
}
```

`oku >> kelime` ardışık boşlukları atladığı için çıktıda kelimeler tek boşlukla ayrılır. Geriye doğru döngü, 10. soruda gördüğümüz güvenli kalıptadır.

Soru 15.

Cevap:

```
#include <iostream>
#include <string>

std::string sezar(std::string metin, int kaydirma) {
    for (char& c : metin) {
        if (c >= 'a' && c <= 'z') {
            c = static_cast<char>('a' + (c - 'a' + kaydirma) % 26);
        } else if (c >= 'A' && c <= 'Z') {
            c = static_cast<char>('A' + (c - 'A' + kaydirma) % 26);
        }
    }
    return metin;
}

int main() {
    std::string gizli = sezar("Hello, World xyz", 3);
    std::cout << gizli << '\n';
    std::cout << sezar(gizli, 23) << '\n';
    return 0;
}
```

Harfin alfabedeki sırası `c - 'a'` ile bulunur, kaydırma eklenir, `% 26` ile başa döndürülür ve `'a'` eklenerek karaktere dönülür. Çıktı `Khoor, Zruog abc` ve ardından `Hello, World xyz` olur.

Soru 16.

Cevap:

```
#include <iostream>
#include <string>

std::string hepsiniDegistir(std::string metin, const std::string& eski,
                            const std::string& yeni) {
    if (eski.empty()) {
        return metin;
    }
    std::size_t konum = metin.find(eski);
    while (konum != std::string::npos) {
        metin.replace(konum, eski.size(), yeni);
```

```
        konum = metin.find(eski, konum + yeni.size());
    }
    return metin;
}

int main() {
    std::cout << hepsiniDegistir("ana ananas", "ana", "x") << '\n';
    std::cout << hepsiniDegistir("aaaa", "a", "aa") << '\n';
    std::cout << hepsiniDegistir("kedi", "z", "y") << '\n';
    return 0;
}
```

Her değişiklikten sonra arama, değiştirilen parçanın hemen sonrasından (`konum + yeni.size()`) sürer. `konum + 1` 'den sürseydik `"aaaa"` örneğinde yeni eklenen `a` 'lar yeniden bulunur ve döngü sonsuza giderdi. Çıktı `x xnas` , `aaaaaaaa` ve `kedi` olur. `eski` boş olsaydı `find` her konumda eşleşir ve döngü bitmezdi; bu yüzden baştan denetlenir.

Bölüm 28 İşaretçiler ve Referanslar

Soru 1.

Cevap: B. `r = y;` bağlama değil atamadır: `y` 'nin değeri (9) `r` üzerinden `x` 'e yazılır. A bağlamayı yeniden yapılabilir sanıyor, yanlış. C yanlış, satır geçerlidir. D yanlış, referans boş olamaz. E yanlış, `sizeof(r)` bağlandığı `int` 'in boyutunu (4) verir.

Soru 2.

Cevap: B. Tanımda ilk değer zorunluluğu referansı kısıtlar; işaretçi ilk değersiz tanımlanabilir (ama bu iyi bir alışkanlık değildir). A, C, D ve E referansın sağladığı kolaylıklardır.

Soru 3.

Cevap: D. `r` bir `const int&` 'tir; üzerinden değer atanamaz. `p = &b;` olur çünkü `p` işaretçisi sabit değil, gösterdiği değer sabittir. `*q = 5;` olur çünkü `q` sabit işaretçidir ama gösterdiği `int` sabit değildir. `a = 7;` olur, `a` sıradan değişkendir. `std::cout << r;` yalnızca okur.

Soru 4.

Cevap: C. `s` yerel değişkendir ve fonksiyon bitince yok olur; dönen referans yok olmuş nesneyi gösterir. A parametreyle gelen nesneyi, B `static` bir değişkeni döndürür; ikisi de fonksiyondan sonra yaşar. D değer döndürür, referans değildir. E'de referans çağıranın değişkenini gösterir; yalnızca çağıran geçici değer verirse sorun çıkar.

Soru 5.

Cevap: C. `const` referans hem kopyayı önler hem de fonksiyonun vektörü değiştirmesini yasaklar. A her çağrıda kopyalar. B kopyalamaz ama değiştirmeye izin verir ve geçici değerle çağrılamaz. D çalışır ama çağrıda `&` gerektirir ve boş olabilir. E yalnızca geçici değerlere bağlanır.

Soru 6.

Cevap: `2 2 2 2`

`r` ve `a` aynı nesnedir. `r = *p + 3;` ile (`p` , `b` 'yi gösterir) `a` 5 olur. `p = &a;` ile `p` artık `a` 'yı gösterir; `*p += 10;` `a` 'yı 15 yapar. `r = b;` ise `r` 'yi `b` 'ye bağlamaz, `b` 'nin değerini (2) `a` 'ya yazar. Sonuçta `a` 2, `b` 2, `r` 2 ve `*p` (yani `a`) 2'dir.

Soru 7.

Cevap: `10 50 30`

`ikinci(a)` ifadesi `a[1]` 'in kendisidir: önce 99 yazılır, sonra 1 eklenip 100 olur. `r` bu elemana bağlanır ve `r = r / 2;` ile 50 olur. Diğer elemanlar değişmez.

Soru 8.

Cevap: `L L R R R` (sonda bir boşluk bulunur)

`x` ve `r` isimli ifadelerdir, lvalue'dur (`r` türü `int&` olsa da ifade olarak lvalue'dur). `x * 2` ve `3` geçici değerlerdir. `static_cast<int&&>(x)` ifadesi `x`'i rvalue olarak işaretler, bu yüzden `int&&` sürümü seçilir. (`std::move` bu dökümün adlandırılmış biçimidir; Bölüm 39.)

Soru 9.

Yerel `mesaj` değişkeninin referansı döndürülüyor; fonksiyon bitince `mesaj` yok olur ve sarkan referans oluşur. Çözüm değer döndürmektir:

```
std::string selamla(const std::string& ad) {
    return "Merhaba " + ad;
}
```

Yeni bir metin ürettiğimiz için referans değil değer döndürmek gerekir; derleyici dönüş değerini kopyalamadan oluşturur.

Soru 10.

`ad`, `const` tanımlandığı için `std::string&` parametreye verilemez; `"Deniz"` ise geçici değer olduğundan sabit olmayan referansa bağlanamaz. `ekle` metni gerçekten değiştirdiği için çözüm, değişebilir bir değişken kullanmaktır:

```
#include <string>

void ekle(std::string& s) {
    s += "!";
}

int main() {
    std::string ad{"Ece"};
    ekle(ad);
    std::string deniz{"Deniz"};
    ekle(deniz);
    return 0;
}
```

Soru 11.

(a) Yanlış: referans tanımlanırken ilk değer almak zorundadır. (b) Doğru: `const` referans geçici değere (5) bağlanabilir ve geçicinin ömrünü uzatır. (c) Doğru: `nullptr` koşulda `false` sayılır, blok çalışmaz. (d) Yanlış: `*` yalnızca hemen sağındaki ada aittir; `q` sıradan `int`'tir.

Soru 12.

Birincisi, `std::string` parametresi her çağrıda içeriğin tamamının kopyalanmasına yol açar; uzun metinlerde bu gereksiz bir maliyettir. İkincisi, `const` fonksiyonun metni değiştirmeyeceğini söyler ve derleyici bunu denetler; `const&` geçici değerleri de kabul ettiğinden `yazdir("metin")` gibi çağrılar çalışır. `int` için bu gerekçeler geçerli değildir, çünkü bir `int` kopyalamak (4 bayt) bir referans geçirmek kadar ucuzdur; üstelik referansın dolaylama maliyeti olabilir. Küçük tipler değerle geçirilir.

Soru 13.

```
#include <iostream>

int* ilkSifir(int* d, int adet) {
    for (int i = 0; i < adet; i++) {
        if (d[i] == 0) {
            return &d[i];
```

```
        }
    }
    return nullptr;
}

int main() {
    int a[5] = {4, 7, 0, 3, 0};
    int* p{ilkSifir(a, 5)};
    if (p != nullptr) {
        *p = 1;
    } else {
        std::cout << "sıfır yok\n";
    }
    for (int i = 0; i < 5; i++) {
        std::cout << a[i] << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

Fonksiyon bulduğu elemanın adresini döndürür; bulamazsa `nullptr` ile "yok" der. `main` dönen işaretçiyi kullanmadan önce sınar. Çıktı: `4 7 1 3 0`.

Soru 14.

```
#include <iostream>

double& enBuyuk(double& a, double& b) {
    return (a > b) ? a : b;
}

int main() {
    double x{2.5};
    double y{7.25};
    enBuyuk(x, y) = 0;
    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

Fonksiyon büyük olan değişkenin kendisini döndürdüğü için atama doğrudan o değişkene yapılır. Çıktı: `2.5 0`.

Soru 15.

```
#include <iostream>
#include <vector>

void ikiyleCarp(int* d, int adet) {
    for (int i = 0; i < adet; i++) {
        d[i] *= 2;
    }
}

void ikiyleCarp(std::vector<int>& v) {
    for (int& e : v) {
        e *= 2;
    }
}

int main() {
    int a[3] = {1, 2, 3};
    ikiyleCarp(a, 3);
```

```
    std::vector<int> v{4, 5, 6};
    ikiyleCarp(v);
    for (int e : a) std::cout << e << ' ';
    for (int e : v) std::cout << e << ' ';
    std::cout << '\n';
    return 0;
}
```

İşaretçi sürümünde eleman sayısı ayrıca verilmelidir; vektör ise kendi boyutunu bilir. `int& e` elemanın kendisini verdiği için değişiklik vektöre yansır. Aynı adlı iki fonksiyon aşırı yüklemedir (Bölüm 25). Çıktı: `2 4 6 8 10 12`.

Soru 16.

```
#include <iostream>

struct Sayac {
    int deger{0};

    Sayac& artir() { ++deger; return *this; }
    Sayac& azalt() { --deger; return *this; }
    Sayac& ekle(int n) { deger += n; return *this; }
    Sayac& sifirla() { deger = 0; return *this; }
};

int main() {
    Sayac s;
    s.artir().artir().azalt().ekle(5);
    std::cout << s.deger << '\n';
    return 0;
}
```

Her üye fonksiyon `*this`'i referans olarak döndürdüğü için zincirleme çağrı mümkün olur. Çıktı: `6`.

Bölüm 29 Dinamik Bellek, RAII ve Akıllı İşaretçiler

Soru 1.

Cevap: B. `new[]` ile alınan dizi `delete[]` ile bırakılır. A yanlış: köşeli parantezsiz `delete` tanımsız davranıştır. C yanlış: `free` yalnızca `malloc` ailesi içindir. D derlenmez; `d[10]` bir `int`'tir ve `delete` işaretçi ister. E yalnızca işaretçiyi boşaltır, bloğu geri vermez; blok sızar.

Soru 2.

Cevap: C. `a = b;` ile `a`'nın eski adresi (değeri 1 olan blok) kaybolur ve o blok sızar. `delete b;` ikinci bloğu geri verir (`a` ve `b` artık sarkandır ama kullanılmıyor). `c`'nin üç elemanlık bloğu hiç `delete[]` edilmedi; o da sızar. Toplam 2 blok.

Soru 3.

Cevap: A. `std::move(p)` sahipliği `q`'ya taşır; `p` boş (`nullptr`) kalır. B yanlış: sahip tektir. C yanlış: `unique_ptr` kopyalanamaz ama taşınabilir. D tersini söylüyor. E yanlış: değer kopyalanmaz, nesne olduğu gibi devredilir.

Soru 4.

Cevap: D. `a` ile `b` iki sahip olur (2). İç bloktaki `c` ve `d` sayacı 4'e çıkarır, ama blok bitince ikisi de yok olur ve sayaç 2'ye döner. `e` ile 3, `f = b` ile 4. Sonuçta `a`, `b`, `e` ve `f` aynı nesneyi tutar. Tuzak: `c` ve `d` `cout` satırından önce yok olmuştur, sayılmaz.

Soru 5.

Cevap: E. `vector` belleği kendisi yönetir, gerektiğinde büyür, kapsam bitince bırakır. A C üslubudur, nesneler için uygun değildir. B çalışır ama elle yönetim gerektirir ve büyümez. C standart C++ değildir (VLA) ve yığını taşırabilir. D hatalıdır: `shared_ptr<int>` tek `int` için `delete` çağırır, oysa `new int[n]` için `delete[]` gerekir.

Soru 6.

Cevap: C. `weak_ptr` sayaca katılmaz (A ve E yanlış), nesneye doğrudan erişilmez, önce `lock()` ile `shared_ptr` alınır (B yanlış). Nesne yok edilmişse `lock()` boş döner. D yanlış: `weak_ptr`, `shared_ptr` ile çalışır.

Soru 7.

Çıktı:

```
ÇIKTI
+A+B|-A-B|1
+C-C
```

Adım adım: `p` için `+A`, `q` için `+B` yazılır. `r = std::move(p)` ile sahiplik `r`'ye geçer, yeni nesne oluşmaz. `|` yazılır. İç blok biterken yerel nesneler oluşturulmanın tersi sırada yok edilir: önce `r` (`-A`), sonra `q` (`-B`). Ardından `|` ve `p == nullptr` doğru olduğu için `1` yazılır. `s` için `+C`, `main` bitince `-C` yazılır; son satırda satır sonu yoktur.

Soru 8.

Çıktı: `4 3 3 2 1 8`

`x`, `v[0]`, `v[1]` ve parametre kopyası: 4. `pop_back` sonrası `x`, `v[0]` ve parametre: 3. `y = x` sonrası `x`, `v[0]`, `y`: 3. `x.reset()` sonrası `y` ve `v[0]`: 2. `x == nullptr` doğru olduğu için `1` yazılır; `*v[0]` ise 8'dir.

Soru 9.

Çıktı:

```
ÇIKTI
23
2 4 8 9 0
```

`new int[5]{2, 4}` dizisi `{2, 4, 0, 0, 0}` olur. `d[2] = 8` ile `{2, 4, 8, 0, 0}`. `e`, `d + 1`'i gösterir; `e[2]`, `d[3]`'tür ve `e[1] + 1 = d[2] + 1 = 9` olur. Dizi `{2, 4, 8, 9, 0}`, toplam 23.

Soru 10.

Üç hata vardır. Birincisi: `new[]` ile alınan blok `delete p;` ile (köşeli parantezsiz) bırakılmış; `delete[] p;` olmalıdır. İkincisi: `delete`'ten sonra `q[0]` okunuyor; `q` sarkan işaretçidir (`p`'yi `nullptr` yapmak `q`'yu korumaz). Üçüncüsü: `delete[] q;` aynı bloğu ikinci kez bırakır (çifte silme). Düzeltme: tek bir `delete[] p;` yazıp ondan sonra hiçbir işaretçiyi kullanmamak ya da hiç ham işaretçi kullanmayıp `auto p = std::make_unique<int[]>(3);` veya `std::vector<int>` tercih etmek.

Soru 11.

`unique_ptr` kopyalanamaz; `isle(a)` çağrısı parametreyi kopyalamaya çalışır ("use of deleted function"). Birinci düzeltme sahipliği devretmektir: `isle(std::move(a));` (çağrıdan sonra `a` boştur). İkinci düzeltme, işlevin sahiplik almasına gerek olmadığını fark edip parametreyi `const int&` yapmaktır: `void isle(const int& p)` ve çağrı `isle(*a);`.

Soru 12.

`e` ile `c` birbirini `shared_ptr` ile tutar; `main` bitince `e` ve `c` yok olsa bile her nesneyi diğerinin üyesi hâlâ tutar, sayaçlar 2'den 1'e iner ve 0'a hiç ulaşmaz. İki nesne de sızar. Döngünün bir yönü `weak_ptr` olmalıdır; örneğin çocuk ebeveyni gözlesin:

```
struct Cocuk {
    std::weak_ptr<Ebeveyn> ebeveyn;
    ~Cocuk() { std::cout << "Cocuk yok edildi\n"; }
};
```

Bu durumda `Ebeveyn` sayacı 1'dir; `main` biterken yok edilir, onun yok olmasıyla `Cocuk`'un sayacı da 0'a iner.

Soru 13.

```
#include <iostream>
#include <memory>

int main() {
    int n = 0;
    std::cout << "Kac sayi? ";
    if (!(std::cin >> n) || n <= 0) {
        std::cout << "Gecersiz sayi.\n";
        return 1;
    }
    auto sayilar = std::make_unique<int[]>(n);
    std::cout << "Sayilari gir: ";
    for (int i = 0; i < n; i++) {
        std::cin >> sayilar[i];
    }
    int enKucuk = sayilar[0];
    for (int i = 1; i < n; i++) {
        if (sayilar[i] < enKucuk) {
            enKucuk = sayilar[i];
        }
    }
    std::cout << "Ters: ";
    for (int i = n - 1; i >= 0; i--) {
        std::cout << sayilar[i] << ' ';
    }
    std::cout << "\nEn kucuk: " << enKucuk << '\n';
    return 0;
}
```

`make_unique<int[]>(n)` diziyi sıfırlayarak açar ve `main` bitince `delete[]`'i kendisi çağırır. Eleman sayısını bilmediği için `n`'i biz tutarız.

Soru 14.

```
#include <iostream>
#include <memory>

std::unique_ptr<int[]> kareler(int n) {
    auto dizi = std::make_unique<int[]>(n);
    for (int i = 0; i < n; i++) {
        dizi[i] = (i + 1) * (i + 1);
    }
    return dizi;                        // sahiplik çağırana taşınır
}

int main() {
    int n = 0;
    std::cout << "Kac eleman? ";
    if (!(std::cin >> n) || n <= 0) {
        std::cout << "Gecersiz sayi.\n";
        return 1;
```

```
    }
    auto k = kareler(n);
    int toplam = 0;
    for (int i = 0; i < n; i++) {
        std::cout << k[i] << ' ';
        toplam += k[i];
    }
    std::cout << "\nToplam: " << toplam << '\n';
    return 0;
}
```

Yerel `dizi`, `return` ile `std::move` yazmadan taşınır; çağıran dizinin sahibi olur.

Soru 15.

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Ogrenci {
    std::string ad;
    Ogrenci(std::string a) : ad(a) { std::cout << ad << " kaydedildi\n"; }
    ~Ogrenci() { std::cout << ad << " silindi\n"; }
};

struct Kurs {
    std::string ad;
    std::vector<std::shared_ptr<Ogrenci>> liste;
};

int main() {
    auto elif = std::make_shared<Ogrenci>("Elif");
    Kurs fizik{"Fizik", {}};
    Kurs kimya{"Kimya", {}};
    fizik.liste.push_back(elif);
    kimya.liste.push_back(elif);
    elif.reset();
    std::cout << "sayac: " << fizik.liste[0].use_count() << '\n';
    fizik.liste.clear();
    std::cout << "fizikten cikti, sayac: " << kimya.liste[0].use_count() << '\n';
    kimya.liste.clear();
    std::cout << "bitti\n";
    return 0;
}
```

Çıktı sırası: `Elif kaydedildi`, `sayac: 2`, `fizikten cikti, sayac: 1`, `Elif silindi`, `bitti`. Öğrenci, son sahibi olan `kimya` listesinden çıkarılınca silinir.

Bölüm 30 Sınıflar ve Nesneler

Soru 1.

Cevap: B. `Kutu(int)` yazıldığı için derleyici varsayılan yapıcıyı üretmez. `Kutu k{7};` argümanlı yapıcıyı çağırır ve derlenir. A, C, D (dizinin her elemanı için) ve E (`Kutu()` ifadesi) parametresiz yapıcı ister. Çözüm `Kutu() : en{0} {}` gibi bir yapıcı eklemek ya da `int en{0};` yazıp `Kutu() = default;` kullanmaktır.

Soru 2.

Cevap: C. Tek dil farkı varsayılan erişimdir: `struct` üyeleri `public`, `class` üyeleri `private` başlar. Her ikisi de yapıcı, üye fonksiyon ve kalıtım içerir. E şıkkı C#'ın özelliğidir.

Soru 3.

Cevap: C. `adet` sınıfın tek değişkenidir. `a` doğar: 1. İç blokta `b` ve `c`: 3. Blok bitince ikisinin yıkıcısı çalışır: 1. Sonra `d` (2) ve `e` (3) doğar. Yazdırma anında `a`, `d` ve `e` yaşıyor: 3.

Soru 4.

Cevap: A. `s`, `const Sicaklik&` olduğu için üzerinden yalnızca `const` üye fonksiyon çağrılabilir; `ayarla` `const` değildir. B'de `t` sabit olmayan referanstır. C ve D'de `al()` `const` üye fonksiyondur. E'de sabit olmayan nesneden `const` referans oluşturmak serbesttir.

Soru 5.

Cevap: C. Derleyicinin ürettiği kopya yapıcı işaretçi üyeyi olduğu gibi kopyalar (sığ kopya). İki nesne aynı bloğu gösterir ve her iki yıkıcı da aynı bloğa `delete[]` uygular; bu çifte silmedir ve tanımsız davranıştır. Derleme hatası oluşmaz.

Soru 6.

Cevap: B. `x` parametresi üye `x`'i gölgeler; `x = x;` parametreyi kendisine atar. Üye değişmez ve `xAl()` 0 döndürür. Doğrusu `this->x = x;` yazmaktır (ya da parametreye farklı bir ad vermek). Derleyici hata vermez.

Soru 7.

Cevap:

```
ÇIKTI
basla
P+birinci
P+ikinci
M+
M-
P-ikinci
P-birinci
bitis
```

Üyeler yapıcı listesindeki sıraya değil, sınıfta tanımlandıkları sıraya göre kurulur: önce `m1` ("birinci"), sonra `m2`. `g++` burada `-Wreorder` uyarısı verir. Sonra `Makine`'nin gövdesi çalışır (`M+`). Yıkımda sıra tersidir: önce `Makine`'nin yıkıcı gövdesi, sonra üyeler kurulumun tersi sırayla (`m2`, sonra `m1`).

Soru 8.

Cevap:

```
ÇIKTI
0 2 3
```

`kopyaArtir(x)` kopya üzerinde çalışır: `x.sayi` 0 kalır, ama `toplam` `static` olduğu için 1 olur. İki `gercekArtir(x)` çağrısı `x.sayi`'yi 2, `toplam`'ı 3 yapar.

Soru 9.

Cevap:

```
ÇIKTI
11 26 16
```

İlk döngüde `auto&` asıl elemanı verir ve herkese 1 eklenir (11, 26, 16). İkinci döngüde `auto o` kopyadır; eklenen 100 kopyalarla birlikte atılır. Son döngü 11 26 16 yazar.

Soru 10.

Cevap: `no` bir `const` üyedir; yapıcı gövdesindeki `no = n;` atamadır ve `const` üyeye atama yapılamaz (üye ayrıca ilk değer almadan doğmuş olur). Düzeltme, üyeyi üye ilk değer listesinde başlatmaktır:

```
Ogrenci(int n, int s) : no{n}, sinif{s} {}
```

Soru 11.

Cevap: `Hesap h();` bir nesne tanımı değil, "parametresiz ve `Hesap` döndüren `h` adlı fonksiyon bildirimi" olarak okunur (en can sıkıcı ayrıştırma). `h` bu yüzden bir nesne değildir ve `h.bilgiYazdir()` geçersizdir. `Hesap h;` ya da `Hesap h{};` yazılmalıdır.

Soru 12.

Cevap: B. `static` veri üyesinin sınıf içindeki satırı yalnızca bildirimdir; `int Sinif::uye{0};` tanımı hiç yazılmazsa derleme geçer, bağlama aşamasında `undefined reference` hatası alınır. A bir derleme hatasıdır. C, D ve E bağlama hatası üretmez (E ise başka sorunlara yol açar, ama bağlama hatası değildir).

Soru 13.

Yaklaşım: üyeleri sınıftaki sırayla (`sahip`, `bakiye`) listede başlatalım; `cek` önce tutarı denetleyip `bool` döndürsün.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class BankaHesabi {
private:
    string sahip;
    double bakiye;

public:
    BankaHesabi(string yeniSahip, double ilkBakiye)
        : sahip{yeniSahip}, bakiye{ilkBakiye >= 0 ? ilkBakiye : 0} {}

    void yatir(double miktar) {
        if (miktar > 0) {
            bakiye += miktar;
        }
    }

    bool cek(double miktar) {
        if (miktar <= 0 || miktar > bakiye) {
            return false;
        }
        bakiye -= miktar;
        return true;
    }

    string sahipAl() const { return sahip; }
    double bakiyeAl() const { return bakiye; }
};

int main() {
    string ad;
    double ilk{0};
    double yatirilacak{0};
```

```
    double cekilecek{0};
    cout << "Sahip, ilk bakiye, yatirilacak, cekilecek: ";
    cin >> ad >> ilk >> yatirilacak >> cekilecek;

    BankaHesabi hesap{ad, ilk};
    hesap.yatir(yatirilacak);
    if (!hesap.cek(cekilecek)) {
        cout << "Yetersiz bakiye.\n";
    }
    cout << fixed << setprecision(2);
    cout << hesap.sahipAl() << ": " << hesap.bakiyeAl() << " TL\n";
    return 0;
}
```

```
ÇIKTI
Sahip, ilk bakiye, yatirilacak, cekilecek: Ayse 500 250 900
Yetersiz bakiye.
Ayse: 750.00 TL
```

500 + 250 = 750; 900 çekilemez (`cek` `false` döndürür) ve bakiye değişmez.

Soru 14.

Yaklaşım: yapıcıda her kenarı koşullu ifadeyle düzeltelim, `emplace_back` ile ekleyelim, en büyük alan ve kare sayısını tek geçişte bulalım.

```
#include <iostream>
#include <vector>
using namespace std;

class Dikdortgen {
private:
    int en;
    int boy;

public:
    Dikdortgen(int e, int b) : en{e > 0 ? e : 1}, boy{b > 0 ? b : 1} {}

    int alan() const { return en * boy; }
    int cevre() const { return 2 * (en + boy); }
    bool kareMi() const { return en == boy; }
};

int main() {
    int n{0};
    cout << "Kac dikdortgen? ";
    cin >> n;

    vector<Dikdortgen> liste;
    for (int i{0}; i < n; i++) {
        int en{0};
        int boy{0};
        cin >> en >> boy;
        liste.emplace_back(en, boy);
    }

    int enBuyukAlan{0};
    int kareSayisi{0};
    for (size_t i{0}; i < liste.size(); i++) {
        cout << "Dikdortgen " << i + 1 << ": alan=" << liste[i].alan()
             << " cevre=" << liste[i].cevre() << '\n';
        if (liste[i].alan() > enBuyukAlan) {
```

```
            enBuyukAlan = liste[i].alan();
        }
        if (liste[i].kareMi()) {
            kareSayisi++;
        }
    }
    cout << "En buyuk alan: " << enBuyukAlan << '\n';
    cout << "Kare sayisi: " << kareSayisi << '\n';
    return 0;
}
```

```
ÇIKTI
Kac dikdortgen? 3
3 4
5 5
6 6
Dikdortgen 1: alan=12 cevre=14
Dikdortgen 2: alan=25 cevre=20
Dikdortgen 3: alan=36 cevre=24
En buyuk alan: 36
Kare sayisi: 2
```

Soru 15.

Yaklaşım: numara, `static` sayaçtan `++sonNo` ile yapıcıda atanır. `static` sayacın tanımı sınıf dışında bir kez yazılır.

```
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Ogrenci {
private:
    string ad;
    int no;
    vector<double> notlar;
    static int sonNo;

public:
    Ogrenci(string yeniAd) : ad{yeniAd}, no{++sonNo} {}

    bool notEkle(double not1) {
        if (not1 < 0 || not1 > 100) {
            return false;
        }
        notlar.push_back(not1);
        return true;
    }

    double ortalama() const {
        if (notlar.empty()) {
            return 0;
        }
        double toplam{0};
        for (double n : notlar) {
            toplam += n;
        }
        return toplam / notlar.size();
    }

    int noAl() const { return no; }
    string adAl() const { return ad; }
```

```
    static int sayiAl() { return sonNo; }
};

int Ogrenci::sonNo{0};

int main() {
    Ogrenci a{"Mehmet"};
    Ogrenci b{"Elif"};
    Ogrenci c{"Can"};
    a.notEkle(80);
    a.notEkle(70);
    b.notEkle(95.5);
    b.notEkle(150);                  // reddedilir
    cout << fixed << setprecision(2);
    for (const Ogrenci* o : {&a, &b, &c}) {
        cout << o->noAl() << " " << o->adAl() << " " << o->ortalama() << '\n';
    }
    cout << "Toplam ogrenci: " << Ogrenci::sayiAl() << '\n';
    return 0;
}
```

```
ÇIKTI
1 Mehmet 75.00
2 Elif 95.50
3 Can 0.00
Toplam ogrenci: 3
```

Elif'in 150 notu reddedilir, ortalaması yalnızca 95.5'tir; Can'ın notu olmadığı için ortalaması 0'dır.

Soru 16.

Yaklaşım: yıkıcı, kopya yapıcı ve kopya atama birlikte yazılır (üçler kuralı); kopya yapıcı yeni blok alıp elemanları kopyalar, atama önce yeni bloğu kurup sonra eskisini siler ve öz-atamayı denetler.

```
#include <iostream>
using namespace std;

class OlcumDizisi {
private:
    double* veri;
    int n;

public:
    OlcumDizisi(int boyut) : veri{new double[boyut]{}}, n{boyut} {}

    OlcumDizisi(const OlcumDizisi& diger)
        : veri{new double[diger.n]}, n{diger.n} {
        for (int i{0}; i < n; i++) {
            veri[i] = diger.veri[i];
        }
    }

    OlcumDizisi& operator=(const OlcumDizisi& diger) {
        if (this != &diger) {
            double* yeni{new double[diger.n]};
            for (int i{0}; i < diger.n; i++) {
                yeni[i] = diger.veri[i];
            }
            delete[] veri;
            veri = yeni;
            n = diger.n;
        }
```

```
        return *this;
    }

    ~OlcumDizisi() { delete[] veri; }

    void ayarla(int i, double deger) { veri[i] = deger; }
    double al(int i) const { return veri[i]; }
};

int main() {
    OlcumDizisi a{3};
    a.ayarla(0, 1.5);
    OlcumDizisi b{a};
    b.ayarla(0, 9.5);
    cout << "a[0] = " << a.al(0) << '\n';
    cout << "b[0] = " << b.al(0) << '\n';
    return 0;
}
```

```
ÇIKTI
a[0] = 1.5
b[0] = 9.5
```

`b` kopyası kendi bellek bloğuna sahip olduğu için `b.ayarla(0, 9.5)` `a`'yı etkilemez.

Bölüm 31 Kalıtım ve Polimorfizm

Soru 1.

Cevap: C. Araba bir motor değildir, motora sahiptir; bu "sahip" (has-a) ilişkisidir ve üye değişkenle kurulur. Diğer şıklarda "X bir Y'dir" cümlesi doğrudur.

Soru 2.

Cevap: A. `main` sınıfın dışıdır; yalnızca `public` üyeye erişebilir. `b` `protected`, `c` `private` olduğundan erişilemez (`c` türetilmiş sınıfın içinden de erişilemez).

Soru 3.

Cevap:

```
ÇIKTI
A B A C | ~C ~A ~B ~A
```

Önce taban zinciri (`A`, `B`), sonra üye nesne `uye` (`A`), en son `C` gövdesi çalışır. Yıkıcılar tersine işler: `~C`, üye `~A`, `~B`, taban `~A`.

Soru 4.

Cevap: `Tasit` sınıfında varsayılan yapıcı olmadığı için derleyici `Tasit()` çağrısını arar ve bulamaz. Taban yapıcısı başlatma listesinde açıkça çağrılmalıdır:

```
Otobus(int koltuk, string plaka)
    : Tasit{plaka}, koltukSayisi{koltuk} {}
```

Soru 5.

Cevap:

```
ÇIKTI
T::a U::b U::b T::b U::a
```

`a()` sanal değildir; `p->a()` işaretçinin tipine göre `T::a` çalıştırır. `b()` sanaldır; `p->b()` ve `r.b()` nesnenin gerçek tipine göre `U::b` verir. `T t = u;` nesneyi dilimler, `t` gerçekten bir `T` olduğundan `t.b()` `T::b` çalıştırır. `u.a()` doğrudan `U` nesnesi üzerinde olduğundan `U::a` verir.

Soru 6.

Cevap: `Kopek::ses()` fonksiyonunda `const` unutulmuş. İmza farklı olduğu için taban fonksiyonu ezilmez, yeni bir fonksiyon tanımlanır ve `Hayvan*` üzerinden hâlâ `Hayvan::ses` çalışır. Düzeltme: `string ses() const override { return "Hav hav"; }`. `override` yazılsaydı derleyici hatayı bildirirdi.

Soru 7.

Cevap: D. Nesnenin öbekte ya da yığında olması önemli değildir; önemli olan çağrının işaretçi ya da referans üzerinden yapılmasıdır. Diğer şıklar dinamik bağlamanın koşullarıdır (`override` yazmak zorunlu değildir ama aynı imzayla ezmek gerekir).

Soru 8.

Cevap: C. Soyut sınıftan nesne yaratılamaz ama işaretçi ve referans tanımlanabilir (A yanlış). Yapıcısı olabilir (B yanlış). Saf sanal fonksiyon `static` olamaz (D yanlış) ve bir sınıfta istenen sayıda olabilir (E yanlış). Tüm saf sanalları ezmeyen türetilmiş sınıf soyut kalır.

Soru 9.

Cevap:

```
ÇIKTI
SDD
```

`f(d)` nesneyi değerle aldığı için `D` parçası dilimlenir ve `S::ad` çalışır. `g(d)` referans aldığından, `p->ad()` de işaretçi üzerinden olduğundan `D::ad` çalışır.

Soru 10.

Cevap: `Taban` yıkıcısı `virtual` değildir. `delete p;` taban işaretçisi üzerinden yapıldığında `Alt` yıkıcısı çağrılmaz; `veri` için ayrılan bellek sızar ve davranış tanımsızdır. Düzeltme: `virtual ~Taban() {}` (ya da `virtual ~Taban() = default;`).

Soru 11.

Cevap:

```
ÇIKTI
T
U
U
```

`T` yapıcısı çalışırken `U` parçası henüz yoktur; yapıcıdan yapılan `goster()` çağrısı `T` sürümüne gider. `U` yapıcısındaki çağrı ve `main` içindeki çağrı `U::goster`'i çalıştırır.

Soru 12.

Cevap: `vector<Sekil>` her elemanı gerçek bir `Sekil` nesnesi olarak tutar; `Daire` nesnesi dilimlenir ve `Daire`'ye ait kısım kaybolur (`Sekil` soyutsa kod derlenmez bile). Doğrusu işaretçi tutmaktır:

```
vector<unique_ptr<Sekil>> sekiller;
sekiller.push_back(make_unique<Daire>(2));
```

Soru 13.

Cevap: `Amfibi` nesnesi `Canli` parçasının iki kopyasını taşır (`Kara` yoluyla ve `Su` yoluyla); `a.yas` hangisine ait olduğu belirsiz olduğundan derlenmez. Çözüm sanal kalıtımdır: `class Kara : public virtual Canli {};` ve `class Su : public virtual Canli {};`. (Alternatif olarak `a.Kara::yas` biçiminde nitelemek mümkündür ama iki ayrı kopya kalır.)

Soru 14.

Cevap:

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Hayvan {
public:
    Hayvan(string adi) : ad{adi} {}
    virtual ~Hayvan() = default;
    virtual string ses() const = 0;
    string adi() const { return ad; }
private:
    string ad;
};

class Kedi : public Hayvan {
public:
    using Hayvan::Hayvan;
    string ses() const override { return "Miyav"; }
};

class Kopek : public Hayvan {
public:
    using Hayvan::Hayvan;
    string ses() const override { return "Hav hav"; }
};

class Kus : public Hayvan {
public:
    using Hayvan::Hayvan;
    string ses() const override { return "Cik cik"; }
};

int main() {
    vector<unique_ptr<Hayvan>> hayvanlar;
    hayvanlar.push_back(make_unique<Kedi>("Pamuk"));
    hayvanlar.push_back(make_unique<Kopek>("Boncuk"));
    hayvanlar.push_back(make_unique<Kus>("Tweety"));
    for (const auto& h : hayvanlar) {
        cout << h->adi() << ": " << h->ses() << '\n';
    }
    return 0;
}
```

`Hayvan` soyut olduğundan nesnesi yaratılamaz; liste `unique_ptr` taşıdığı için dilimleme olmaz ve bellek kendiliğinden temizlenir. Çıktı: `Pamuk: Miyav`, `Boncuk: Hav hav`, `Tweety: Cik cik`.

Soru 15.

Cevap:

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Sekil {
public:
    virtual ~Sekil() = default;
    virtual double alan() const = 0;
    virtual string ad() const = 0;
};

class Kare : public Sekil {
public:
    Kare(double k) : kenar{k} {}
    double alan() const override { return kenar * kenar; }
    string ad() const override { return "Kare"; }
private:
    double kenar;
};

class Ucgen : public Sekil {
public:
    Ucgen(double t, double y) : taban{t}, yukseklik{y} {}
    double alan() const override { return taban * yukseklik / 2; }
    string ad() const override { return "Üçgen"; }
private:
    double taban, yukseklik;
};

void enBuyuguYaz(const vector<unique_ptr<Sekil>>& sekiller) {
    if (sekiller.empty()) return;
    const Sekil* enBuyuk = sekiller[0].get();
    for (const auto& s : sekiller) {
        if (s->alan() > enBuyuk->alan()) {
            enBuyuk = s.get();
        }
    }
    cout << enBuyuk->ad() << ": " << enBuyuk->alan() << '\n';
}

int main() {
    vector<unique_ptr<Sekil>> sekiller;
    sekiller.push_back(make_unique<Kare>(3));
    sekiller.push_back(make_unique<Ucgen>(4, 7));
    sekiller.push_back(make_unique<Kare>(2));
    enBuyuguYaz(sekiller);
    return 0;
}
```

Fonksiyon listeyi `const` referansla alır (kopyalama yok) ve her şeklin gerçek tipine göre `alan()` çağırır. Çıktı: `Üçgen: 14`.

Soru 16.

Cevap:

```
#include <iostream>
#include <memory>
#include <vector>
using namespace std;

class Hesap {
public:
    virtual ~Hesap() = default;
    void yatir(double miktar) { bakiye += miktar; }
    virtual bool cek(double miktar) {
        if (miktar > bakiye) return false;
        bakiye -= miktar;
        return true;
    }
    virtual void aySonu() = 0;
    double bakiyeVer() const { return bakiye; }
protected:
    double bakiye{0};
};

class VadesizHesap : public Hesap {
public:
    void aySonu() override { bakiye -= 5; }
};

class VadeliHesap : public Hesap {
public:
    bool cek(double miktar) override {
        if (miktar > bakiye / 2) return false;
        return Hesap::cek(miktar);
    }
    void aySonu() override { bakiye *= 1.02; }
};

int main() {
    vector<unique_ptr<Hesap>> hesaplar;
    hesaplar.push_back(make_unique<VadesizHesap>());
    hesaplar.push_back(make_unique<VadeliHesap>());
    for (auto& h : hesaplar) {
        h->yatir(1000);
        h->cek(100);
        h->aySonu();
        cout << h->bakiyeVer() << '\n';
    }
    return 0;
}
```

Vadesiz hesapta 1000 - 100 - 5 = 895 kalır. Vadeli hesapta 100 TL çekimi bakiyenin yarısından (500) küçük olduğundan geçer; 900 TL'ye %2 faiz eklenince 918 olur. `VadeliHesap::cek` kendi koşulunu denetledikten sonra taban sürümünü `Hesap::cek` ile çağırır. Çıktı: `895` ve `918`.

Bölüm 32 Operatör Aşırı Yükleme

Soru 1.

Cevap: C. `::` (kapsam çözümleme) aşırı yüklenemez. `[]`, `()` ve `->` yüklenebilir (ama yalnızca üye olarak), virgül de yüklenebilir.

Soru 2.

Cevap: B. `cout << nesne` ifadesinin sol işleneni `std::ostream` tipindedir; bu sınıf bizim değildir, ona üye fonksiyon ekleyemeyiz. Bu yüzden `operator<<` sınıfın dışında, iki parametreli bir fonksiyon olarak yazılır.

Soru 3.

Cevap: C. Sonek sürüm, kullanılmayan fazladan bir `int` parametre alır (`operator++(int)`). Parametre yalnızca önek sürümden ayırt etmek içindir.

Soru 4.

Cevap: B. `a + b` operandlarını değiştirmeyen, yeni bir değer üreten işlemdir. Sonuç fonksiyon içinde yerel bir nesnede oluşur; ona referans döndürmek fonksiyon bitince yok olan nesneyi göstermek, yani sarkan referans olurdu.

Soru 5.

Cevap:

```
+(1,2) +(3,3) 6
```

`+` soldan sağa birleşir: önce `a + b` (1 ve 2) hesaplanır ve `T{3}` olur, sonra bu geçici nesne `c` ile toplanır (3 ve 3).

Soru 6.

Cevap:

```
1 3 3
```

`s++` eski değerin kopyasını (1) verir ve `s` 2 olur. `++s` önce `s`'yi 3 yapar, sonra `s`'nin kendisini verir (3). Sonda `s` 3'tür.

Soru 7.

Cevap: `operator<` üye fonksiyonunun sonunda `const` yoktur. `kucukMu` fonksiyonunda `a` bir `const Kesir&` olduğundan `a < b` çağrısı `const` olmayan üyeyi çağıramaz (`passing 'const Kesir' as 'this' argument discards qualifiers`). Düzeltme:

```
bool operator<(const Kesir& d) const {
    return pay_ * d.payda_ < d.pay_ * payda_;
}
```

Soru 8.

Cevap: `operator<<` `void` döndürüyor. `std::cout << "Nokta: " << p` işleminin sonucu `void` olur ve `void << '\n'` diye bir işlem yoktur. Akışı referansla döndürmek gerekir:

```
std::ostream& operator<<(std::ostream& os, const Nokta& p) {
    os << '(' << p.x << ", " << p.y << ')';
    return os;
}
```

Soru 9.

(a) Doğru. `[]`, `()`, `=` ve `->` yalnızca üye fonksiyon olabilir. (b) Yanlış. Yeni operatör icat edilemez; `**` yoktur. (c) Yanlış. En az bir işlenen kullanıcı tanımlı tip olmalıdır; yerleşik tiplerin işlemi değiştirilemez. (d) Yanlış. Aşırı yüklenmiş `&&` sıradan bir fonksiyondur; iki argüman da hesaplanır, kısa devre yoktur.

Soru 10.

Cevap: `+`'ı `+=` üzerinden yazmak (1) hesap mantığını tek yerde tutar, bir düzeltme iki yeri etkilemez; (2) `a += b` ile `a = a + b`'nin aynı sonucu vermesini kendiliğinden sağlar. İkili operatörler üye olmayan yazılırsa sol işlenen de örtük dönüşümden yararlanır: üye `operator+` ile `a + 2` çalışırken `2 + a` çalışmaz, üye olmayan sürümle ikisi de çalışır.

Soru 11.

Cevap:

```
#include <iostream>

class Sure {
public:
    Sure(int saat = 0, int dakika = 0) : toplam_{saat * 60 + dakika} {}
    int toplamDakika() const { return toplam_; }
    Sure& operator+=(const Sure& d) {
        toplam_ += d.toplam_;
        return *this;
    }

private:
    int toplam_;
};

Sure operator+(Sure a, const Sure& b) { return a += b; }
bool operator==(const Sure& a, const Sure& b) {
    return a.toplamDakika() == b.toplamDakika();
}
bool operator<(const Sure& a, const Sure& b) {
    return a.toplamDakika() < b.toplamDakika();
}

std::ostream& operator<<(std::ostream& os, const Sure& s) {
    return os << s.toplamDakika() / 60 << " sa " << s.toplamDakika() % 60
              << " dk";
}

int main() {
    Sure ders{1, 30};
    Sure mola{0, 45};
    std::cout << ders + mola << '\n';
    std::cout << std::boolalpha << (mola < ders) << ' '
              << (ders + mola == Sure{2, 15}) << '\n';
    return 0;
}
```

Süre dakika olarak saklandığı için `+` ve karşılaştırmalar tek bir toplama ya da karşılaştırmaya iner. Çıktı `2 sa 15 dk` ve `true true` olur.

Soru 12.

Cevap:

```
#include <iostream>

class Karmasik {
public:
    Karmasik(double gercek = 0, double sanal = 0)
        : gercek_{gercek}, sanal_{sanal} {}
    double gercek() const { return gercek_; }
    double sanal() const { return sanal_; }
```

```
private:
    double gercek_;
    double sanal_;
};

Karmasik operator+(const Karmasik& a, const Karmasik& b) {
    return Karmasik{a.gercek() + b.gercek(), a.sanal() + b.sanal()};
}

Karmasik operator*(const Karmasik& a, const Karmasik& b) {
    return Karmasik{a.gercek() * b.gercek() - a.sanal() * b.sanal(),
                    a.gercek() * b.sanal() + a.sanal() * b.gercek()};
}

bool operator==(const Karmasik& a, const Karmasik& b) {
    return a.gercek() == b.gercek() && a.sanal() == b.sanal();
}

std::ostream& operator<<(std::ostream& os, const Karmasik& k) {
    os << k.gercek();
    if (k.sanal() >= 0) {
        os << '+';
    }
    return os << k.sanal() << 'i';
}

int main() {
    Karmasik a{1, 2};
    Karmasik b{3, -1};
    std::cout << a + b << '\n';
    std::cout << a * b << '\n';
    return 0;
}
```

Çıktı `4+1i` ve `5+5i` olur: `(1+2i)(3-i) = (3+2) + (-1+6)i`. Sanal kısım negatifse `-` işareti sayının kendisinden gelir, pozitifse `+` biz ekleriz.

Soru 13.

Cevap:

```
#include <iostream>

class Matris2 {
public:
    Matris2(double a, double b, double c, double d) : m_{a, b, c, d} {}
    double& operator()(int i, int j) { return m_[i * 2 + j]; }
    double operator()(int i, int j) const { return m_[i * 2 + j]; }

private:
    double m_[4];
};

Matris2 operator*(const Matris2& a, const Matris2& b) {
    Matris2 s{0, 0, 0, 0};
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 2; ++j) {
            for (int k = 0; k < 2; ++k) {
                s(i, j) += a(i, k) * b(k, j);
            }
        }
    }
    return s;
```

```
}

bool operator==(const Matris2& a, const Matris2& b) {
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 2; ++j) {
            if (a(i, j) != b(i, j)) {
                return false;
            }
        }
    }
    return true;
}

std::ostream& operator<<(std::ostream& os, const Matris2& m) {
    os << '[' << m(0, 0) << ' ' << m(0, 1) << "; " << m(1, 0) << ' '
       << m(1, 1) << ']';
    return os;
}

int main() {
    Matris2 a{1, 2, 3, 4};
    Matris2 birim{1, 0, 0, 1};
    std::cout << a * a << '\n';
    std::cout << std::boolalpha << (a * birim == a) << '\n';
    return 0;
}
```

Çıktı `[7 10; 15 22]` ve `true` olur. Elemanlar tek boyutlu dizide tutulur; `operator()` iki indeksi `i * 2 + j` ile birleştirir. `==` operatörü yalnızca küçük tamsayılı değerlerle çalışan örnekte güvenlidir; kesirli sonuçlarda `double` eşitliği için küçük bir tolerans kullanmak gerekir (Bölüm 5).

Soru 14.

Cevap:

```
#include <iostream>

class Sayac {
public:
    explicit Sayac(int deger = 0) : deger_{deger} {}
    int deger() const { return deger_; }

    Sayac& operator++() {
        ++deger_;
        return *this;
    }
    Sayac operator++(int) {
        Sayac eski{*this};
        ++deger_;
        return eski;
    }
    Sayac& operator--() {
        --deger_;
        return *this;
    }
    Sayac operator--(int) {
        Sayac eski{*this};
        --deger_;
        return eski;
    }
    Sayac& operator+=(int n) {
        deger_ += n;
        return *this;
```

```
    }

private:
    int deger_;
};

int main() {
    Sayac s{10};
    s += 5;
    Sayac eski = s--;
    std::cout << eski.deger() << ' ' << s.deger() << '\n';
    --s;
    ++s;
    std::cout << s.deger() << '\n';
    return 0;
}
```

Çıktı `15 14` ve `14` olur. `s--` sayacı 15'ten 14'e indirir ama eski değerin kopyasını (15) döndürür; `--s` ile `++s` birbirini götürür.

Bölüm 33 Şablonlar (Templates)

Soru 1.

Cevap: B. Şablon bir tariftir; derleyici kullanılan her tip kümesi için gerçek bir fonksiyon ya da sınıf üretir. Kullanılmayan şablon hiç kod üretmez.

Soru 2.

Cevap: C. İlk argümandan `T = int`, ikinciden `T = double` çıkar; derleyici ikisini uzlaştırmaz. `enBuyuk<double>(3, 4.5)` ile düzeltilir.

Soru 3.

Çıktı:

```
8 3 abab
```

`ikiKat(4)` 8, `ikiKat(1.5)` 3 (`3.0` ekrana `3` yazılır), `string` için `+` birleştirme yaptığından `abab` olur.

Soru 4.

`Kutu<T>::al()` tanımının başında `template <typename T>` başlığı yok. Doğrusu:

```
template <typename T>
T Kutu<T>::al() const { return deger_; }
```

Sınıf dışında tanımlanan her üyede şablon başlığı tekrar yazılır.

Soru 5.

Cevap: C. Derleyici `kare<int>` gibi bir örneği üretebilmek için gövdeyi görmek zorundadır. Gövde yalnızca ayrı bir `.cpp` içindeyse örnekleme yapılamaz ve bağlayıcı `undefined reference` verir.

Soru 6.

Çıktı:

```
1,1,2,3,
```

`say<int>` ve `say<char>` ayrı fonksiyonlardır, her birinin kendi `static` sayacı vardır. `say<int>` üç kez çağrıldı (1, 2, 3), `say<char>` bir kez (1).

Soru 7.

Cevap: C. Tip olmayan şablon parametresi derleme zamanında bilinen bir sabit ifade olmalıdır. `N`'nin tipi `size_t` olmak zorunda değildir ve değeri belirtilmezse (varsayılan yoksa) hata oluşur.

Soru 8.

`n` sıradan bir değişkendir, derleme zamanında sabit bir ifade değildir. Şablon argümanı olabilmesi için `constexpr int n = 5;` (ya da `const int n = 5;`) yazılmalıdır.

Soru 9.

Cevap: B. Özelleştirme, belirli bir tip (ya da tip deseni) için genel şablondan ayrı bir tanım yazmaktır.

Soru 10.

Çıktı:

```
int genel genel
```

`Ad<int>` tam özelleştirmeyi kullanır; `Ad<double>` ve `Ad<char>` genel şablondan üretilir.

Soru 11.

Cevap: B. C# generics'te `where T : ...` kısıtı açıkça yazılır; C++17 şablonlarında kısıt yoktur ve gövdede kullanılan işlemin eksikliği örnekleme sırasında hata verir. Tip olmayan parametre ve özelleştirme C++'a özgüdür; C#'ta yoktur.

Soru 12.

Cevap: B. `with T = Nokta` hangi tiple örneklendiğini, ilk `error:` satırı ise eksik olan işlemi (`operator<`) söyler. Çözüm `Nokta` için `operator<` yazmaktır.

Soru 13.

```
#include <iostream>
#include <string>
using namespace std;

template <typename T>
T enKucuk(T a, T b) {
    return (b < a) ? b : a;
}

int main() {
    cout << enKucuk(4, 2) << " " << enKucuk(2.5, 7.0) << " "
         << enKucuk(string("b"), string("a")) << '\n';   // 2 2.5 a
    return 0;
}
```

`enBuyuk` ile aynı kalıp; yalnızca karşılaştırma yönü değişir. Gövde yalnızca `<` kullandığı için `<` olan her tiple çalışır.

Soru 14.

```
#include <iostream>
#include <string>
using namespace std;

template <typename A, typename B>
```

```
struct Cift {
    A birinci;
    B ikinci;
};

template <typename A, typename B>
Cift<B, A> takas(const Cift<A, B>& c) {
    return Cift<B, A>{c.ikinci, c.birinci};
}

int main() {
    Cift<string, int> c{"x", 5};
    auto d = takas(c);                 // Cift<int, string>
    cout << d.birinci << " " << d.ikinci << '\n';   // 5 x
    return 0;
}
```

Tip çıkarımı `A = string`, `B = int` bulur; dönüş tipi `Cift<int, string>` olur.

Soru 15.

```
#include <iostream>
using namespace std;

template <typename T, int N>
class SabitYigin {
public:
    bool ekle(const T& x) {
        if (adet_ == N) {
            return false;
        }
        veri_[adet_++] = x;
        return true;
    }
    bool cikar(T& x) {
        if (adet_ == 0) {
            return false;
        }
        x = veri_[--adet_];
        return true;
    }
private:
    T veri_[N];
    int adet_ = 0;
};

int main() {
    SabitYigin<int, 2> s;
    int v;
    cout << s.ekle(1) << s.ekle(2) << s.ekle(3) << " ";   // 110
    s.cikar(v);
    cout << v << '\n';                                    // 2
    return 0;
}
```

Kapasite `N` tipin parçasıdır; dinamik bellek gerekmez. Son eleman eklenemediği için `ekle(3)` `false` döndürür, `cikar` en son eklenen 2'yi verir.

Bölüm 34 İstisna Yönetimi

Soru 1.

Cevap: C. `overflow_error`, `runtime_error` ailesindendir. Diğer dördü (`invalid_argument`, `out_of_range`, `length_error`, `domain_error`) `logic_error` ailesindendir.

Soru 2.

Cevap: C. `out_of_range` bir `logic_error`'dır. `runtime_error` ve `invalid_argument` ondan türemez (biri başka dalda, diğeri kardeş). Yukarıdan aşağıya bakıldığında uyan ilk blok `logic_error` olur; `exception` bloğu da uyar ama sırası sonradır.

Soru 3.

Cevap: B. Yakalayıcı bulunamazsa `std::terminate` çağrılır. A, C ve E yanlış; D yanlış çünkü derleyici bunu denetleyemez, durum çalışma zamanında ortaya çıkar.

Soru 4.

Cevap: E. `throw;` yakalanan nesneyi olduğu gibi yeniden fırlatır. A yanlış, imzaya yazmak zorunlu değildir. B yanlış, dilimleme olur. C yanlış, `stoi` baştaki sayıyı okuyup 12 döndürür. D yanlış, `noexcept` sınırını aşan istisna doğrudan `terminate` çağırır.

Soru 5.

Cevap: D. Yapıcısı tamamlanmayan nesnenin yıkıcısı çalışmaz, ama kurulmuş üyeler kuruluşun tersi sırada yok edilir.

Soru 6.

Cevap: `T+ A+ B+ B- A- T- yakalandi bitti`

`T`, `A` ve `B` sırayla oluşur. `throw` yığın çözülmesini başlatır: `g` içindeki `B` ve `A`, sonra `try` bloğundaki `T` ters sırada yok edilir. Bunların hepsi `catch` bloğundan önce olur. `"sonra "` hiç yazılmaz.

Soru 7.

Cevap: `tamam L I E` (sonda bir boşluk bulunur)

`i == 0` için istisna yoktur ve `tamam` yazılır. `i == 1` için `invalid_argument` bir `logic_error`'dır: `f` içinde `L` yazılır, `throw;` ile yukarı gider ve `main`'de `I` bloğu çalışır. `i == 2` için `runtime_error` `logic_error` değildir; `f`'in `catch`'ine uymaz, `main`'e ulaşır ve `invalid_argument` bloğuna uymayıp `exception` bloğunda `E` yazdırır.

Soru 8.

Cevap: `int7 diger hic` (sonda bir boşluk bulunur)

`throw 7` türü `int` olan `catch (int k)` bloğuna uyar. `throw 'x'` bir `char`'dır ve `int` ile eşleşmez (istisna eşleşmesinde dönüşüm yapılmaz); `catch (...)` yakalar. Üçüncü turda istisna yoktur.

Soru 9.

İki sorun var. Birincisi, `catch (std::exception e)` değerle yakalar; nesne dilimlenir ve `what()` gerçek iletiyi vermez. İkincisi, genel tür özelinden önce yazılmıştır; `out_of_range` bloğuna hiç ulaşılmaz. Düzeltilmiş hâli:

```
try {
    throw std::out_of_range("sınır");
} catch (const std::out_of_range& e) {
    std::cout << "özel: " << e.what() << '\n';
} catch (const std::exception& e) {
    std::cout << e.what() << '\n';
}
```

Soru 10.

`sayi`, `try` bloğunun kapsamında tanımlıdır; `catch` bloğunda ve sonrasında görünmez. Değişkeni `try`'dan önce tanımlarız:

```
int sayi = 0;
try {
    sayi = std::stoi(metin);
} catch (const std::invalid_argument&) {
    sayi = 0;
}
std::cout << sayi << '\n';
```

Soru 11.

`throw 42;` bir `int` fırlatır; `catch (const std::exception&)` `int` ile eşleşmez. İstisna yakalanmadığı için `std::terminate` çağrılır. Düzeltme: `std::exception` türevi bir sınıf fırlatmak (`throw std::runtime_error("hata");`). `int` fırlatmak gerekiyorsa `catch (int)` ya da `catch (...)` kullanılırdı, ama bu kötü stildir.

Soru 12.

```
#include <iostream>
#include <stdexcept>

double bol(double a, double b) {
    if (b == 0) {
        throw std::invalid_argument("bölen sıfır olamaz");
    }
    return a / b;
}

int main() {
    try {
        std::cout << "10 / 4 = " << bol(10, 4) << '\n';
        std::cout << "1 / 0 = " << bol(1, 0) << '\n';
    } catch (const std::invalid_argument& e) {
        std::cout << "Hata: " << e.what() << '\n';
    }
    return 0;
}
```

`b == 0` karşılaştırması `double` için burada yeterlidir; `1 / 0` satırında `"1 / 0 = "` metni, `bol` çağrılmadan önce yazıldığı için ileti aynı satıra gelir. Çıktı:

```
10 / 4 = 2.5
1 / 0 = Hata: bölen sıfır olamaz
```

Soru 13.

```
#include <iostream>
#include <stdexcept>

class PuanHatasi : public std::runtime_error {
public:
    PuanHatasi(int deger)
        : std::runtime_error("puan 0-100 aralığında olmalı"), deger_(deger) {}
    int deger() const { return deger_; }
```

```
private:
    int deger_;
};

void puanKontrol(int puan) {
    if (puan < 0 || puan > 100) {
        throw PuanHatasi(puan);
    }
}

int main() {
    for (int puan : {85, 120, -5}) {
        try {
            puanKontrol(puan);
            std::cout << puan << ": geçerli\n";
        } catch (const PuanHatasi& e) {
            std::cout << e.deger() << ": " << e.what() << '\n';
        }
    }
    return 0;
}
```

Çıktı:

```
85: geçerli
120: puan 0-100 aralığında olmalı
-5: puan 0-100 aralığında olmalı
```

Hata sınıfı, iletinin yanında geçersiz değeri de taşır; `catch` bloğu ikisini birlikte kullanır.

Soru 14.

```
#include <iostream>
#include <stdexcept>
#include <vector>

class Yigin {
public:
    void it(int x) { veri_.push_back(x); }
    int cek() {
        if (veri_.empty()) {
            throw std::underflow_error("yığın boş");
        }
        int ust = veri_.back();
        veri_.pop_back();
        return ust;
    }
    bool bos() const { return veri_.empty(); }

private:
    std::vector<int> veri_;
};

int main() {
    Yigin y;
    y.it(1);
    y.it(2);
    try {
        for (int i = 0; i < 3; i++) {
            std::cout << "çekildi: " << y.cek() << '\n';
        }
```

```
    } catch (const std::underflow_error& e) {
        std::cout << "Hata: " << e.what() << '\n';
    }
    return 0;
}
```

Çıktı:

```
çekildi: 2
çekildi: 1
çekildi: Hata: yığın boş
```

Üçüncü satır yarım kalır çünkü `"çekildi: "` metni, `cek()` çağrılmadan önce yazılmıştır.

Soru 15.

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> liste{"10", "yirmi", "30", "99999999999", "5"};
    int toplam = 0;
    int gecersiz = 0;
    for (const std::string& metin : liste) {
        try {
            toplam += std::stoi(metin);
        } catch (const std::invalid_argument&) {
            std::cout << "\"" << metin << "\" sayı değil, atlandı\n";
            gecersiz++;
        } catch (const std::out_of_range&) {
            std::cout << "\"" << metin << "\" çok büyük, atlandı\n";
            gecersiz++;
        }
    }
    std::cout << "Toplam: " << toplam << ", geçersiz: " << gecersiz << '\n';
    return 0;
}
```

Çıktı:

```
"yirmi" sayı değil, atlandı
"99999999999" çok büyük, atlandı
Toplam: 45, geçersiz: 2
```

İki ayrı `catch` hem hatanın türünü ayırt eder hem de hiçbirini sessizce yutmaz.

Bölüm 35 STL Kapsayıcıları

Soru 1.

Cevap: C. `deque` her iki uçta sabit sürede ekleme/silme yapar; `push_front` bu yüzden vardır. `vector`'ün `push_front` üyesi yoktur, başa ekleme `insert(begin(), x)` ile yapılır ve tüm elemanları kaydırır. `push_back`, `[]`, `size()` ve `clear()` her ikisinde de vardır.

Soru 2.

Cevap: C. `m["a"]` anahtarı 0 ile ekler; `m["b"] = 2` b'yi ekler; `insert({"b", 9})` b var olduğu için dokunmaz; `insert_or_assign("a", 7)` a'nın değerini 7 yapar. Sonuç `7 2 2`: iki anahtar vardır. B, `insert`'in değeri değiştirdiğini sananın; D, `m["b"]` okumasının yeni eleman eklediğini sananın cevabıdır (b zaten vardı).

Soru 3.

Cevap: A. Karma tablo ortalama durumda sabit sürede arar. O(log n) `set` / `map`, O(n) ise `vector` aramasıdır. Çok çakışma olursa en kötü durum O(n)'dir, ancak soru ortalamayı sorar.

Soru 4.

Cevap: B. Değerle `erase(3)`, 3'e eşit **tüm** elemanları siler (iki tane), geriye yalnızca `1` kalır. Yalnızca birini silmek için `m.erase(m.find(3))` yazılırdı.

Soru 5.

Cevap: D. `stack` son giren ilk çıkar düzenindedir; son yapılan işlem geri alınırken önce çıkar. `queue` ilk giren ilk çıkardır, `priority_queue` en büyüğü önce verir.

Soru 6.

Cevap:

```
ÇIKTI
4 5 9
3 4 5 9
```

Küme `{3, 5, 8, 9}` ile başlar (tekrarlar elenir). `insert(5)` etkisizdir, `insert(4)` ekler, `erase(8)` siler: `{3, 4, 5, 9}`. `lower_bound(5)` 5'e eşit ya da büyük ilk elemanı (5) gösterir; `rbegin()` en büyük elemandır (9).

Soru 7.

Cevap:

```
ÇIKTI
3 2 20
3 3 3
```

Yığına 1, 2, 3, 4 girer; `pop` 4'ü çıkarır, tepe 3 olur. Kuyruktan ilk giren 1 çıkar, ön 2 olur. Öncelikli kuyruğa 1, 20, 3, 40 girer; `pop` en büyük olan 40'ı çıkarır, `top()` 20 verir. Her birinde 4 eleman vardı, birer eleman çıktı: boyutlar 3 3 3.

Soru 8.

Cevap:

```
ÇIKTI
1z 2a 2b 3
```

`pair`'ler önce `first`'e, eşitse `second`'a göre sıralanır: `(1,z)`, `(2,a)`, `(2,b)`. İkinci `(1,z)` tekrar olduğu için eklenmez; boyut 3'tür.

Soru 9.

Cevap: `erase(it)` çağrısından sonra `it` geçersiz olur, döngü sonundaki `++it` tanımsız davranıştır. Doğru kalıp, `erase`'in döndürdüğü yineleyiciyi almak ve yalnızca silmediğimizde ilerletmektir:

```
for (auto it = m.begin(); it != m.end();) {
    if (it->second == 0) {
        it = m.erase(it);
```

```
    } else {
        ++it;
    }
}
```

Soru 10.

Cevap: `set` elemanları `const`'tır; değiştirilmeleri sıralamayı bozabileceğinden `*s.begin()` bir `const int&` verir ve atama derlenmez. Elemanı değiştirmek için eskisini `erase`, yenisini `insert` etmek gerekir.

Soru 11.

Cevap: Birinci hata: `pop()` `void` döndürür, bu yüzden `int x = s.pop();` derlenmez. Doğrusu `int x = s.top(); s.pop();`. İkinci hata: yığın boşken `top()` çağrılması tanımsız davranıştır; önce `!s.empty()` denetlenmelidir.

Soru 12.

Cevap: C. `unordered_*` kapsayıcılarda dolaşma sırası standartta belirtilmemiştir ve kütüphaneden kütüphaneye değişebilir. Ekleme sırası ya da alfabetik sıra garanti değildir. Sıralı dolaşma gerekiyorsa `map` kullanılır.

Soru 13.

Cevap: Açan parantezleri yığına atarız; kapatan parantezde yığın boşsa ya da tepe türce eşleşmiyorsa ifade dengesizdir. Satır sonunda yığın boş değilse de dengesizdir.

```
#include <iostream>
#include <stack>
#include <string>

int main() {
    std::string satir;
    while (std::getline(std::cin, satir)) {
        std::stack<char> yigin;
        bool dengeli = true;
        for (char c : satir) {
            if (c == '(' || c == '[' || c == '{') {
                yigin.push(c);
            } else if (c == ')' || c == ']' || c == '}') {
                if (yigin.empty()) {
                    dengeli = false;
                    break;
                }
                char acan = yigin.top();
                yigin.pop();
                if ((c == ')' && acan != '(') || (c == ']' && acan != '[') ||
                    (c == '}' && acan != '{')) {
                    dengeli = false;
                    break;
                }
            }
        }
        if (!yigin.empty()) {
            dengeli = false;
        }
        std::cout << satir << " -> " << (dengeli ? "dengeli" : "dengesiz")
                  << '\n';
    }
    return 0;
}
```

`(a+b)*[c-d]` ve `{[()]}` dengeli, `((a+b]` ve `())(` dengesiz çıkar.

Soru 14.

Cevap: `map<char, int>` anahtarı karakter kodlarına göre sıralı tuttuğu için çıktı alfabetik gelir. `sayac[c]++` yeni harfi 0'dan başlatıp artırır.

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::string satir;
    std::getline(std::cin, satir);
    std::map<char, int> sayac;
    for (char c : satir) {
        if (c != ' ') {
            sayac[c]++;
        }
    }
    for (const auto& [harf, adet] : sayac) {
        std::cout << harf << ": " << adet << '\n';
    }
    return 0;
}
```

"hello world" girdisinde `d: 1`, `e: 1`, `h: 1`, `l: 3`, `o: 2`, `r: 1`, `w: 1` yazılır.

Soru 15.

Cevap: Birinci vector'ü kümeye aktarırız (tekrarlar elenir, hızlı arama sağlanır); ikincideki her elemanı kümede arar ve bulunanları ikinci bir kümeye ekleriz. Sonuç kümesi hem tekrarsız hem sıralıdır.

```
#include <iostream>
#include <set>
#include <vector>

int main() {
    std::vector<int> a{4, 9, 1, 4, 7, 9};
    std::vector<int> b{9, 2, 4, 4, 8};
    std::set<int> kume(a.begin(), a.end());
    std::set<int> ortak;
    for (int x : b) {
        if (kume.count(x) > 0) {
            ortak.insert(x);
        }
    }
    for (int x : ortak) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

Bu veriyle çıktı `4 9` olur.

Soru 16.

Cevap: En küçük-önce yığın üç elemandan fazlasını tutmaz; boyut 3'ü aşınca en küçük elemanı atarız. Böylece yığında her zaman o ana kadarki en büyük üç sayı kalır. Sonunda yığın küçükten büyüğe boşalır, biz de ters sırada yazdırırız.

```
#include <functional>
#include <iostream>
#include <queue>
#include <vector>

int main() {
    std::priority_queue<int, std::vector<int>, std::greater<int>> yigin;
    int sayi;
    while (std::cin >> sayi) {
        yigin.push(sayi);
        if (yigin.size() > 3) {
            yigin.pop();
        }
    }
    std::vector<int> sonuc;
    while (!yigin.empty()) {
        sonuc.push_back(yigin.top());
        yigin.pop();
    }
    for (std::size_t i = sonuc.size(); i > 0; i--) {
        std::cout << sonuc[i - 1] << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

`7 42 3 19 8 42 1` girdisinde çıktı `42 42 19` olur. Zaman karmaşıklığı O(n log 3), yani pratikte O(n)'dir.

Bölüm 36 STL Algoritmaları, Yineleyiciler ve Lambda İfadeleri

Soru 1.

Cevap: C. `end()` son elemandan bir sonraki konumu gösterir; hiçbir elemanı göstermez ve açılamaz.

Soru 2.

Cevap: D. `std::sort` rastgele erişimli yineleyici ister; `list` çift yönlüdür. `list` için `l.sort()` kullanılır.

Soru 3.

Cevap: C. Başlangıç değeri `0` bir `int`'tir; toplam her adımda `int`'e kırpılır. Doğrusu `0.0` yazmaktır.

Soru 4.

Cevap: D. Yalnızca referansla yakalama (`[&toplam]`) asıl değişkeni değiştirir. `[toplam]` ve `[=]` kopya yakalar ve `mutable` olmadan değiştirilemez.

Soru 5.

Cevap: B. Algoritmalar yalnızca yineleyici gördüğünden kapsayıcının boyutunu değiştiremez. `remove_if` tutulacak elemanları başa kaydırır; gerçek silmeyi `erase` yapar.

Soru 6.

Cevap: B. `<=` eşit elemanlar için `true` döndürür ve katı zayıf sıralamayı bozar; `sort` tanımsız davranışla sınır dışına yazabilir ya da çökebilir. Doğrusu `a < b`'dir.

Soru 7.

Cevap: A. İkili arama sıralı aralık varsayar; sıralı değilse sonuç anlamsızdır. Tekrarsızlık, tip ya da kapsayıcı kısıtı yoktur.

Soru 8.

Cevap: B. Değerle yakalanan `sayac` lambda içinde salt okunurdur. Çözüm `mutable` yazmak (`[sayac]() mutable {...}`) ya da `[&sayac]` kullanmaktır.

Soru 9.

`find_if` 6'dan büyük ilk elemanı bulur: 8, indeks 2. Tek sayılar 5, 1, 9, 3 olduğundan `count_if` 4 verir. Azalan sıralama 9 8 5 3 2 1 yapar.

```
ÇIKTI
8 2
4
9 8 5 3 2 1
```

Soru 10.

`a` kendi kopyasını değiştirir: 11, sonra 21; dış `x` 1 kalır. `b` dış `x`'i değiştirir: 101 ve `x` de 101 olur.

```
ÇIKTI
11 21 1
101 101
```

Soru 11.

`remove` üç adet 4'ü dışarıda bırakır: başta 7 9 1 kalır, yeni son indeks 3'tür; boyut `erase`'e kadar 6 kalır. `erase`'ten sonra yalnızca 7 9 1 kalır.

```
ÇIKTI
3 6
7 9 1
```

Soru 12.

Birinci hata: `b` boş olduğu için `b.begin()`'e yazmak tanımsız davranıştır; `std::back_inserter(b)` kullanılmalı (ya da `std::vector<int> b(3);`). İkinci hata: `list` rastgele erişimli yineleyici sunmaz, `std::sort` derlenmez; `l.sort();` yazılmalıdır.

Soru 13.

Lambda yerel `k` değişkenini referansla yakalıyor; `uret` döndüğünde `k` yok olur ve lambda sarkan referans taşır (tanımsız davranış). Değerle yakalanmalı: `[k](int x) { return x + k; }`.

Soru 14.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    std::cout << "Sayılar (0 ile biter): ";
    int x;
    while (std::cin >> x && x != 0) {
        v.push_back(x);
    }
    int cift = std::count_if(v.begin(), v.end(),
                             [](int n) { return n % 2 == 0; });
    std::cout << "Çift sayı: " << cift << '\n';
    v.erase(std::remove_if(v.begin(), v.end(),
                           [](int n) { return n % 2 != 0; }),
```

```
            v.end());
    std::cout << "Kalanlar:";
    for (int n : v) {
        std::cout << ' ' << n;
    }
    std::cout << '\n';
    return 0;
}
```

Tek sayılar silme-silme deyimiyle atılır; geriye çiftler girilen sırada kalır.

Soru 15.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> k(5);
    std::cout << "Beş kelime: ";
    for (auto& s : k) {
        std::cin >> s;
    }
    std::sort(k.begin(), k.end(),
              [](const std::string& a, const std::string& b) {
                  if (a.size() != b.size()) {
                      return a.size() < b.size();
                  }
                  return a < b;
              });
    for (std::size_t i = 0; i < k.size(); i++) {
        std::cout << (i ? " " : "") << k[i];
    }
    std::cout << '\n';
    return 0;
}
```

Önce uzunluk karşılaştırılır; eşitse alfabetik sıra belirler. Eşit kelimelerde `false` döndüğü için karşılaştırma geçerlidir.

Soru 16.

```
#include <algorithm>
#include <iostream>
#include <iterator>
#include <numeric>
#include <string>
#include <vector>

struct Urun {
    std::string ad;
    double fiyat;
};

int main() {
    std::vector<Urun> urunler{{"kalem", 12.5}, {"defter", 45.0},
                              {"silgi", 8.0}, {"çanta", 350.0}};
    double esik = 10.0;
    std::vector<Urun> pahali;
    std::copy_if(urunler.begin(), urunler.end(), std::back_inserter(pahali),
                 [esik](const Urun& u) { return u.fiyat > esik; });
```

```
    std::sort(pahali.begin(), pahali.end(),
              [](const Urun& a, const Urun& b) { return a.fiyat > b.fiyat; });
    for (const Urun& u : pahali) {
        std::cout << u.ad << ' ' << u.fiyat << '\n';
    }
    double toplam = std::accumulate(pahali.begin(), pahali.end(), 0.0,
        [](double t, const Urun& u) { return t + u.fiyat; });
    std::cout << "toplam " << toplam << '\n';
    return 0;
}
```

Çıktı: `çanta 350`, `defter 45`, `kalem 12.5`, `toplam 407.5`. Toplamın başlangıç değeri `0.0` olmalıdır.

Bölüm 37 Dosya İşlemleri ve Akışlar

Soru 1.

Cevap: C. `ofstream`'in varsayılan kipi `ios::out`'tur; dosyayı açarken içeriğini siler. Hiçbir şey yazılmasa da dosya 0 bayt olur. Dosya silinmez, açılma da başarılıdır.

Soru 2.

Cevap: E. `while (dosya >> x)` okumanın kendisini koşul yapar. A, B ve C'de koşul okumadan önce sınanır; son sayı okunduktan sonra bile akış hâlâ "iyi" olduğundan bir tur daha dönülür, okuma başarısız olur ve `x`'in eski değeri (3) toplama bir kez daha eklenir (sonuç 9). D'de 100 tur dönülür ve aynı sorun 97 kez yinelenir.

Soru 3.

Cevap: B. `ios::app` kipinde her yazma, `seekp(0)` çağrılmış olsa bile dosyanın sonuna yapılır. Dosya `A`, `B` olur.

Soru 4.

Cevap: D. `seekg` bayt cinsinden konum ister. 3 numaralı kaydın konumu `3 * 20 = 60`'tır ve dosya başından ölçülür (`ios::beg`).

Soru 5.

Cevap: A. `dosya >> n` yalnızca `7`'yi okur; ilk satırın sonundaki `'\n'` akışta kalır. Ardından `getline` bu satır sonunu görüp boş bir satır okur ve `ad` boş kalır. `Ali Veli` satırı hâlâ okunmamıştır. Çare, araya `dosya.ignore(std::numeric_limits<std::streamsize>::max(), '\n');` koymaktır.

Soru 6.

Cevap:

```
5 10 20 20
toplam=55
```

`20` okunduktan sonra dosya sonu henüz görülmediği için `eof()` yanlıştır ve döngü bir tur daha döner. Bu turda `f >> x` okuyacak bir şey bulamaz, `x` değişmez (20 kalır) ve `toplam` 20 daha artar: 5 + 10 + 20 + 20 = 55.

Soru 7.

Cevap:

```
10
A
E
I 9
```

Yazdıktan sonra `tellp()` 10'dur. `seekg(0)` ile başa gidip `A` okunur (konum 1). `seekg(3, ios::cur)` konumu 4'e taşır ve `E` okunur (konum 5). `seekg(-2, ios::end)` konumu 8 yapar, `I` okunur ve `tellg()` 9 verir.

Soru 8.

Cevap:

```
2 30 1 0
abc
30
```

Döngü `10` ve `20`'yi okur (`n = 2`, `toplam = 30`), `abc` sayıya çevrilemediği için durur: `fail()` 1, `eof()` 0. `clear()` bayrakları siler; `abc` okunmamıştı, bu yüzden `string` olarak okunur. Son okuma `30`'u verir.

Soru 9.

İki hata vardır. Birincisi, `"C:\notlar.txt"` içindeki `\n` yeni satır karakteridir; yol bozulur. Her ters bölü çift yazılmalı (`"C:\\notlar.txt"`) ya da eğik çizgi ya da ham string kullanılmalıdır. İkincisi, `while (!oku.eof())` yanlış kalıptır; sonda boş bir satır yazdırılır. Ayrıca akışın açılıp açılmadığı da denetlenmemiştir. Düzeltilmiş hâli:

```
std::ifstream oku("C:/notlar.txt");
if (!oku.is_open()) {
    std::cerr << "notlar.txt açılamadı\n";
    return 1;
}
std::string satir;
while (std::getline(oku, satir)) {
    std::cout << satir << '\n';
}
```

Soru 10.

`std::string` nesnesi metnin kendisini değil, öbekteki (heap) metne giden bir işaretçiyi ve uzunluk bilgisini içerir. `write` yalnızca nesnenin baytlarını kopyalar; dosyaya metin değil, program kapandığında geçersiz kalacak bir adres yazılır. Çözüm olarak ya `char ad[32]` gibi sabit boyutlu bir dizi kullanılır ya da metin önce uzunluğuyla (`size_t` olarak), sonra karakterleriyle ayrı ayrı yazılır; ya da veri metin biçiminde (CSV gibi) saklanır.

Soru 11.

`ios::out` (`ofstream`'in varsayılanı) dosya varsa içeriğini siler, yoksa oluşturur; dosya her seferinde sıfırdan yazılır. `ios::app` dosyayı olduğu gibi bırakır ve her yazmayı sona ekler. Çıktı raporu gibi her çalışmada yenilenmesi gereken dosyalar için `ios::out`; günlük (log) gibi birikmesi gereken dosyalar için `ios::app` seçilir.

Soru 12.

```
#include <algorithm>
#include <fstream>
#include <iostream>
#include <vector>

int main() {
    std::ifstream oku("sayilar.txt");
    if (!oku.is_open()) {
        std::cerr << "sayilar.txt açılamadı\n";
        return 1;
    }
    std::vector<int> v;
    int x{0};
    while (oku >> x) {
        v.push_back(x);
```

```
    }
    if (v.empty()) {
        std::cout << "Dosya boş\n";
        return 0;
    }
    int toplam{0};
    for (int s : v) {
        toplam += s;
    }
    std::cout << "Toplam: " << toplam << '\n';
    std::cout << "En büyük: " << *std::max_element(v.begin(), v.end()) << '\n';
    return 0;
}
```

Dosya `12 -4 40 7` değerlerini içeriyorsa çıktı `Toplam: 55` ve `En büyük: 40` olur. Boş dosya için `max_element` çağrılmadan önce `v.empty()` denetlenir.

Soru 13.

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

int main() {
    const std::string ad = "yazi.txt";
    {
        std::ofstream yaz(ad);
        yaz << "Bir iki üç\ndört beş\n\naltı\n";
    }
    std::ifstream oku(ad);
    if (!oku.is_open()) {
        std::cerr << ad << " açılamadı\n";
        return 1;
    }
    int satir{0};
    int kelime{0};
    int bayt{0};
    std::string s;
    while (std::getline(oku, s)) {
        satir++;
        bayt += static_cast<int>(s.size()) + 1;   // satır sonu dahil
        std::istringstream ss{s};
        std::string k;
        while (ss >> k) {
            kelime++;
        }
    }
    std::cout << "Satır: " << satir << ", kelime: " << kelime
              << ", bayt: " << bayt << '\n';
    return 0;
}
```

Çıktı: `Satır: 4, kelime: 6, bayt: 31`. Satırı `getline` ile alıp kelimeleri `istringstream` üzerinde `>>` ile sayıyoruz. Bayt sayısına satır sonlarını (`+ 1`) ekledik; Türkçe harfler iki bayt saydığı için `size()` karakter değil bayt verir.

Soru 14.

```cpp
#include <filesystem>
#include <fstream>
#include <iostream>

int main() {
    std::ifstream giris("kaynak.txt", std::ios::binary);
    std::ofstream cikis("hedef.txt", std::ios::binary);
    if (!giris.is_open() || !cikis.is_open()) {
        std::cerr << "Dosyalar açılamadı\n";
        return 1;
    }
    char tampon[4096];
    while (giris.read(tampon, sizeof tampon) || giris.gcount() > 0) {
        cikis.write(tampon, giris.gcount());
    }
    cikis.close();
    auto a = std::filesystem::file_size("kaynak.txt");
    auto b = std::filesystem::file_size("hedef.txt");
    std::cout << a << ' ' << b << (a == b ? " aynı" : " farklı") << '\n';
    return 0;
}
```

Son blok 4096 bayttan kısa olunca `read` başarısız olur (koşulun ilk yarısı yanlış), ama `gcount()` o ana kadar okunan baytları verir; koşulun ikinci yarısı bu son parçanın da yazılmasını sağlar. `ios::binary` kipi satır sonu çevirisini engelleyerek herhangi bir dosyanın bozulmadan kopyalanmasını sağlar.

Soru 15.

```cpp
#include <algorithm>
#include <filesystem>
#include <iostream>
#include <vector>

namespace fs = std::filesystem;

int main() {
    std::vector<fs::path> liste;
    for (const auto& girdi : fs::directory_iterator(fs::current_path())) {
        if (girdi.is_regular_file() && girdi.path().extension() == ".txt") {
            liste.push_back(girdi.path());
        }
    }
    std::sort(liste.begin(), liste.end());
    for (const fs::path& p : liste) {
        std::cout << p.filename().string() << ": " << fs::file_size(p) << " bayt\n";
    }
    return 0;
}
```

`directory_iterator` sıra garantisi vermediği için yolları bir `vector`'de toplayıp `sort` ile sıralıyoruz. Klasörlerin ve diğer uzantıların dışarıda kalması için `is_regular_file()` ve `extension()` denetlenir.

Bölüm 38 Özyineleme C++'ta

Soru 1.

Cevap: A. Değer ile geçirilen vector her çağrıda tamamen kopyalanır. `n` çağrı varsa toplamda yaklaşık `n²` eleman kopyalanır. B yanlıştır: her çağrı kendi kopyasını alır. C'de `const` kopyayı önlemez; önleyen şey referanstır. D ve E yanlıştır: kopya eleman sayısından bağımsız oluşur ve fonksiyon kopyayı değiştirebilir.

Soru 2.

Cevap: C. `std::size_t` işaretsizdir; 0'dan 1 çıkarmak sarmalanarak en büyük değeri verir. Bu yüzden özyinelemede `n - 1` işlemini yalnızca `n > 0` iken yapmalıyız.

Soru 3.

Cevap: B. Değer ile geçirilen tablo her çağrıda kopyalanır; çağrının yazdığı değerler çağırana ulaşmaz. Program hatasız derlenir ve doğru sonucu verir, yalnızca tablo olmadığı için yavaşlar. Bölüm 18'de tabloyu global dizi yaptığımız için bu tuzağa düşmedik; burada tabloyu referansla geçirerek aynı güvenceyi sağlıyoruz.

Soru 4.

Cevap: D. `4! = 24` permütasyon vardır ve her biri bir satır yazdırır.

Soru 5.

Cevap: D. Her eleman için iki seçenek olduğundan dört eleman için `2^4 = 16` alt küme oluşur; her yaprak bir satır yazdırır.

Soru 6.

Cevap: B. Birleştirmeli sıralama her düzeyde toplam `n` eleman birleştirir ve `log n` düzey vardır. Girdi ne olursa olsun bu sayı aynıdır; bu yüzden en kötü durumu da `n log n`'dir.

Soru 7.

Cevap: B. Küçükten büyüğe sıralı dizide son eleman en büyüktür; her bölme pivotu bir uca koyar ve derinlik `n`'e çıkar. A ortalama durumdur. C, D ve E'de derinlik zaten küçüktür.

Soru 8.

Cevap: `ab ba ab`. Önce `f(s, 0)` içinde `ab` yazılır; ardından ilk harfler değişip `ba` yazılır; fonksiyon bitince `s` yeniden `ab` olur ve son satırda o yazılır. Çıktıda `ab` ve `ba` arasındaki boşluklar nedeniyle satır `ab ba ab` biçimindedir.

Soru 9.

Cevap: `0 1 1 2`. Sıra şöyledir: hiçbir eleman alınmadan boş küme (0), `6` alınarak (1), `5` alınarak (1), ikisi birlikte (2). Her yaprakta `secilen` içindeki eleman sayısı yazılır.

Soru 10.

Cevap: B. `son` küçük `bas`'tan küçükse fark işaretsiz olduğu için sarmalanır ve büyük bir pozitif sayı olur; `< 0` karşılaştırması hiçbir zaman doğru olmaz. Derleyici bunu uyarır ama hata vermez. Doğru yazım `son < bas` ya da `bas <= son` biçiminde olmalıdır.

Soru 11.

Cevap: `-Wsign-compare` uyarısı. `int i` ile `v.size()` (işaretsiz `std::size_t`) farklı işaretli türler olduğu için karşılaştırılır. Düzeltme: `std::size_t i{0}` kullanmak.

Soru 12.

Cevap: Sonsuz özyineleme ve yığın taşması. `r == 0` olduğunda durulur; ama `c == 0` ve `r > 0` iken `izgaraYol(r, c - 1)` çağrısı `c`'yi `-1`, `-2` diye azaltır ve temel durum hiç sağlanmaz. Temel durum `r == 0 || c == 0` olmalıdır.

Soru 13.

Cevap: Programın çekirdeği aşağıdaki gibidir. Girdi `3` için çıktı `1 2 3`, `1 3 2`, `2 1 3`, `2 3 1`, `3 2 1`, `3 1 2` olur.

```
void permutasyon(std::vector<int>& v, std::size_t k) {
    if (k == v.size()) {
        yazdir(v);                  // boşlukla yazan yardımcı fonksiyon
        return;
    }
    for (std::size_t i{k}; i < v.size(); ++i) {
        std::swap(v[k], v[i]);
        permutasyon(v, k + 1);
        std::swap(v[k], v[i]);
    }
}
```

`main` içinde `n` okunur, `1`'den `n`'e kadar sayılar bir `std::vector<int>` içine konur ve `permutasyon(v, 0)` çağrılır. Fonksiyon, Bölüm 38'deki `permutasyon` ile aynıdır; tek fark harflerin yerine sayıların kullanılmasıdır.

Soru 14.

Cevap: Birleştirme döngüsündeki karşılaştırmayı azalan düzen için seçmek yeterlidir. Parametreyi ekleyip özyinelemeye geçirin ve birleştirmede şunu kullanın:

```
bool soldaAl{azalan ? v[i] >= v[j] : v[i] <= v[j]};
```

Azalan düzende `>=` kullanmak da kararlılığı korur. Girdi `{38, 27, 43, 3, 9, 82, 10}` ve `azalan = true` için sonuç `82 43 38 27 10 9 3` olur.

Soru 15.

Cevap: Alt küme fonksiyonuna iki parametre daha ekleyin: hedef toplam `hedef` ve o ana kadar seçilenlerin toplamı `toplam`. Yaprakta `toplam == hedef` ise `secilen`'i yazdırın.

```
void toplamliAltKume(const std::vector<int>& v, std::size_t i, int hedef,
                     int toplam, std::vector<int>& secilen) {
    if (i == v.size()) {
        if (toplam == hedef) {
            yazdir(secilen);
        }
        return;
    }
    toplamliAltKume(v, i + 1, hedef, toplam, secilen);
    secilen.push_back(v[i]);
    toplamliAltKume(v, i + 1, hedef, toplam + v[i], secilen);
    secilen.pop_back();
}
```

`{1, 2, 3, 4}` ve `S = 5` için çıktı `{2, 3}` ve ardından `{1, 4}` olur. Sıra, "alınmadı" dalının önce bitmesinden gelir.

Bölüm 39 Modern C++: Taşıma Semantiği ve Ötesi

Soru 1.

Cevap: B. `std::move` hiçbir şeyi taşımaz ve kopyalamaz; ifadeyi rvalue'ya dönüştürür. Gerçek taşımayı, rvalue kabul eden taşıma yapıcısı ya da taşıma ataması yapar.

Soru 2.

Cevap: D. `std::move(s)` bir `const std::string&&` üretir. Taşıma yapıcısı `std::string&&` bekler ve `const` bir nesneden kaynak çalamaz; bu yüzden kopya yapıcısı seçilir. Derleme hatası olmaz, `s` değişmez.

Soru 3.

Cevap: `v k t v` (sondaki boşluktan sonra satır sonu). `a` varsayılan yapıcıyla (`v`), `b = a` kopya yapıcıyla (`k`), `c = std::move(a)` taşıma yapıcıyla (`t`) oluşur. `S d = S{};` ifadesinde C++17'den itibaren kopya eleme zorunludur; yalnızca varsayılan yapıcı çalışır (`v`).

Soru 4.

Cevap: B. Beşler kuralı yıkıcı, kopya yapıcı, kopya atama, taşıma yapıcı ve taşıma atamasından oluşur.

Soru 5.

`return std::move(v);` gereksizdir ve zararlıdır: yerel nesne doğrudan döndürüldüğünde derleyici kopya elemeyi (NRVO) uygulayabilir; `std::move` ifadeyi rvalue'ya çevirdiği için bu iyileştirmeyi engeller ve bir taşıma yapıcısı çağrısı bırakır. Düzeltme: `return v;`.

Soru 6.

Cevap: C. Taşıma yapıcısı `noexcept` değilse `vector` yeniden bellek ayırırken güçlü istisna güvencesini koruyabilmek için elemanları taşımak yerine kopyalayabilir. Sınıf derlenir ve doğru çalışır, yalnızca taşımanın kazancı kaybolur.

Soru 7.

Cevap: `6 6 6`. `auto y = x;` bağımsız bir kopya oluşturur, `auto& z = x;` ise `x`'in kendisine bağlanır. `y++` kopyayı 5'ten 6 yapar, `z++` ise `x`'i 5'ten 6 yapar. `z` aynı nesneyi gösterdiği için o da 6 görünür. Üç değer de 6'dır, ama `y`'nin 6 olması `x`'ten bağımsız bir artıştan gelir.

Soru 8.

Cevap: C. Boş bir `optional`'ı `*o` ile dolaylamak tanımsız davranıştır. `has_value`, `value_or`, atama ve `if (o)` güvenlidir.

Soru 9.

`std::string("gecici metin")` geçici bir nesnedir ve ifadenin sonunda yok edilir. `string_view` metne sahip olmadığı için `sv` asılı kalır (dangling); sonraki satırda okumak tanımsız davranıştır. Program tesadüfen doğru metni yazabilir, ama güvenilemez. Düzeltme: metni kalıcı bir `std::string` değişkende tutup ondan görünüm oluşturmak (`std::string s{"gecici metin"}; std::string_view sv{s};`).

Soru 10.

Cevap: `12`. Atamadan sonra `variant` ikinci alternatifi (`std::string`) taşır; `index()` 1 verir, `get<1>(v).size()` ise `"ab"` için 2 verir. İkisi bitişik yazılır.

Soru 11.

Cevap: D. `enum class` değerleri örtük olarak `int`'e dönüşmez (A ve E hatalı), `int`'ten örtük dönüşüm de yoktur (B hatalı), sabitler kapsam gerektirir (C'deki `Yesil` tek başına bulunamaz). `static_cast<int>` açık dönüşümdür ve geçerlidir.

Soru 12.

Cevap: `30`. `f(4)` = 24, `f(3)` = 6; toplam 30. `a` derleme zamanında hesaplanır, `f(3)` ise burada çalışma zamanında da hesaplanabilir; sonuç aynıdır.

Soru 13.

Cevap: D. `std::optional` C++17 ile gelmiştir. Kavramlar, aralıklar, `std::format` ve modüller C++20 özellikleridir.

Soru 14.

Örnek çözüm:

```
#include <algorithm>
#include <cstddef>
#include <iostream>
#include <utility>

class Kutu {
    std::size_t boyut{0};
    int* veri{nullptr};

public:
    explicit Kutu(std::size_t n) : boyut{n}, veri{new int[n]{}} {}
    ~Kutu() { delete[] veri; }

    Kutu(const Kutu& o) : boyut{o.boyut}, veri{new int[o.boyut]} {
        std::copy(o.veri, o.veri + boyut, veri);
    }
    Kutu& operator=(const Kutu& o) {
        if (this != &o) {
            int* yeni = new int[o.boyut];
            std::copy(o.veri, o.veri + o.boyut, yeni);
            delete[] veri;
            veri = yeni;
            boyut = o.boyut;
        }
        return *this;
    }
    Kutu(Kutu&& o) noexcept : boyut{o.boyut}, veri{o.veri} {
        o.boyut = 0;
        o.veri = nullptr;
    }
    Kutu& operator=(Kutu&& o) noexcept {
        if (this != &o) {
            delete[] veri;
            boyut = o.boyut;
            veri = o.veri;
            o.boyut = 0;
            o.veri = nullptr;
        }
        return *this;
    }
    std::size_t boyutu() const { return boyut; }
};

int main() {
    Kutu a{10};
    Kutu b = std::move(a);
    std::cout << "a: " << a.boyutu() << ", b: " << b.boyutu() << '\n';
}
```

Çıktı: `a: 0, b: 10`. Taşıma işlemleri işaretçiyi devralıp kaynağı boş duruma getirir ve `noexcept` işaretlidir; yıkıcıdaki `delete[] nullptr` zararsızdır.

Soru 15.

Örnek çözüm:

```
#include <iostream>
#include <optional>
```

```
std::optional<double> bol(double a, double b) {
    if (b == 0) {
        return std::nullopt;
    }
    return a / b;
}

int main() {
    std::cout << bol(10, 4).value_or(-1) << '\n';
    auto r = bol(1, 0);
    if (r) {
        std::cout << *r << '\n';
    } else {
        std::cout << "tanimsiz\n";
    }
}
```

Çıktı: `2.5` ve `tanimsiz`. Sonuç denetlenmeden `*r` kullanılmamalıdır.

Soru 16.

Örnek çözüm (kavram kısmı için `g++ -std=c++20` gerekir):

```
#include <concepts>
#include <iostream>

enum class Gun { Pzt, Sal, Car };

const char* adi(Gun g) {
    switch (g) {
        case Gun::Pzt: return "Pazartesi";
        case Gun::Sal: return "Sali";
        case Gun::Car: return "Carsamba";
    }
    return "?";
}

template <typename T>
concept Sayisal = std::integral<T> || std::floating_point<T>;

template <Sayisal T>
T enBuyuk(T a, T b) {
    return a > b ? a : b;
}

int main() {
    for (Gun g : {Gun::Pzt, Gun::Sal, Gun::Car}) {
        std::cout << adi(g) << '\n';
    }
    std::cout << enBuyuk(3, 8) << ' ' << enBuyuk(2.5, 1.5) << '\n';
}
```

`enum class` değerleri `switch` içinde tam adla yazılır. Kavram, `enBuyuk`'a sayısal olmayan bir tip verilmesini derleme aşamasında engeller.

Bölüm 41 .NET Dünyası ve C#'a Giriş

Soru 1.

Cevap: B. Roslyn kaynak kodu işlemciden bağımsız IL'e çevirir ve IL ile üst veriyi bir assembly'ye (`.dll` ya da `.exe`) yazar. A şıkkı C ve C++ derleyicilerinin yoludur. Başlık dosyası C#'ta yoktur (C), kaynak kod şifrelenmez (D) ve GC ayarı derleme çıktısı değildir (E).

Soru 2.

Cevap: C. IL'i makine koduna çalışma anında, CLR içindeki JIT derleyicisi çevirir. Roslyn (A) kaynak kodu IL'e çevirir ve bu iş program çalışmadan önce biter. NuGet (B) paket yöneticisidir, `dotnet new` (D) proje oluşturur, `.csproj` (E) proje ayarlarını tutar.

Soru 3.

Cevap: C. SDK, çalıştırma ortamına ek olarak derleyiciyi ve `dotnet` komut satırı araçlarını içerir; Runtime ise yalnızca hazır bir programı çalıştırmaya yeter. İkisi aynı paket değildir (A), derleyici SDK'dadır (B) ve ikisi de çapraz platformdur (D, E).

Soru 4.

Cevap: D. GC, ulaşılamayan nesnelerin belleğini kendi belirlediği bir anda geri alır. Son referans kaybolur kaybolmaz toplamaz (A), dosya ve bağlantı gibi kaynakları kapatmaz (B; bunun için `IDisposable` ve `using` vardır), `free` beklemez (C) ve her platformda çalışır (E).

Soru 5.

Cevap: B. Üst düzey deyimler tip ve ad alanı tanımlarından önce gelmelidir; aksi halde `CS8803` verilir. Bir projede yalnızca bir dosya üst düzey deyim içerebilir (A yanlış), `Main` metodunu derleyici üretir (C yanlış), `args` kullanılabilir (D yanlış) ve özellik C# 9'dan beri .NET'in yeni sürümlerinde vardır (E yanlış).

Soru 6.

Cevap: C. Bir ad alanının tipleri birden çok assembly'de bulunabilir; ad alanı mantıksal, assembly fiziksel bir gruptur (A yanlış). `using System;` yalnızca adları kısaltır, kütüphane eklemez (B). `using` satırları dosyanın başında yazılır (D). `using static` bir tipin statik üyelerini sınıf adı olmadan kullandırır (E).

Soru 7.

Cevap: D. Hedef sürüm (`TargetFramework`), `ImplicitUsings`, `Nullable` ve paket referansları `.csproj` dosyasındadır. `Program.cs` kaynak koddur, `bin` ve `obj` derleme çıktılarıdır.

Soru 8.

Cevap:

```
ÇIKTI
AB
C
D       E
"F"
```

`Write("A")` satırı açık bırakır, `WriteLine("B")` onu bitirir: `AB`. `"C\n"` yazıldıktan sonra `\n` satırı bitirir. `\t` bir sekme boşluğu bırakır. `\"` çift tırnağı yazdırır.

Soru 9.

Cevap:

```
ÇIKTI
x12
3x
x3
7x
```

İşlem soldan sağa yapılır. `"x" + 1` metin olur, ardından `+ 2` eklenir: `x12`. `1 + 2` iki sayıdır, toplanır (3), sonra `"x"` eklenir: `3x`. Parantez önce hesaplanır: `x3`. Dördüncü satırda çarpma önce yapılır (`2 * 3 = 6`), sonra `1 + 6 = 7` ve `7x` olur.

Soru 10.

Cevap:

```
ÇIKTI
5
.txt
2.5
```

`Max(4, 9) - Min(4, 9)` = `9 - 4` = 5. `GetExtension` uzantıyı noktasıyla verir. `Abs(-2.5)` 2.5 döndürür.

Soru 11.

İki hata vardır. Birincisinde `console` küçük harfle yazılmıştır; doğrusu `Console`'dır (`CS0103`). İkincisinde ilk satırın sonunda `;` eksiktir (`CS1002`). Derleyici önce sözdizimi hatasını bildirir; `;` eklendikten sonra `CS0103` ortaya çıkar.

```
Console.WriteLine("Merhaba");
Console.WriteLine("Dünya");
```

Soru 12.

Program, deyim (`Console.WriteLine`) bir tip tanımından sonra geldiği için derlenmez: `CS8803`, üst düzey deyimlerin ad alanı ve tip tanımlarından önce gelmesi gerektiğini söyler. Çözüm, deyimi sınıfın üstüne taşımaktır:

```
Console.WriteLine("Başladı");

class Kutu
{
    public int Deger = 5;
}
```

Soru 13.

`class Program` ve `Main` iskeleti yazılmaz; `args` yine kullanılabilir. `using System;` satırı gerekmez, ama yazılırsa zarar vermez.

```
Console.WriteLine("Toplam: " + (7 + 8));
Console.WriteLine("Argüman sayısı: " + args.Length);
```

Argümansız çalıştırıldığında çıktı `Toplam: 15` ve `Argüman sayısı: 0` olur.

Soru 14.

Satır sonu için `\n`, sekme için `\t`, çift tırnak için `\"` ve ters bölü için `\\` kullanılır.

```
Console.WriteLine("Kod\tAd\n\"cs\"\tC#\nC:\\proje\\cs");
```

Soru 15.

Klasik biçimde `args` dizisi `Main` metodunun parametresidir. İlk argümana `args[0]` ile ulaşılır; önce `args.Length` ile argüman olup olmadığına bakılır.

```
class Program
{
    static void Main(string[] args)
    {
        if (args.Length > 0)
        {
            Console.WriteLine("Merhaba, " + args[0] + "!");
        }
        else
        {
            Console.WriteLine("Merhaba, Dünya!");
        }
    }
}
```

`dotnet run -- Ayşe` komutu `Merhaba, Ayşe!`, argümansız `dotnet run` ise `Merhaba, Dünya!` yazdırır.

Bölüm 42 Değişkenler, Veri Tipleri ve Operatörler

Soru 1.

Cevap: C. Ad harf ya da `_` ile başlayabilir, içinde rakam bulunabilir. `_sayac1` bu kurallara uyar. `3adim` rakamla başlıyor, `toplam-tutar` ve `fiyat$` geçersiz karakter (`-` ve `$`) içeriyor, `new` ise anahtar sözcüktür.

Soru 2.

Cevap: B. Sağ taraf önce hesaplanır: `17 / 5` iki `int` değerin bölümüdür ve sonuç `3` olur (ondalık kısım kesilir). Sonra `3`, `double`'a otomatik dönüşür. C# ondalık kısmı sıfır olan bir `double`'ı `3.0` olarak değil `3` olarak yazdırır; bu yüzden C şıkkı yanlıştır. `3.4` elde etmek için `17 / 5.0` yazmak gerekirdi. `int` değer `double`'a örtük dönüştüğü için kod derlenir.

Soru 3.

Cevap: B. `2.5` bir `double` sabitidir ve `float`'a örtük olarak dönüşmez (`CS0664`); `2.5f` yazılmalıydı. A, C ve D'de `int` sabiti daha geniş bir tipe (`long`, `double`, `decimal`) örtük dönüşür. E de derlenir: `'A'` bir `char`'dır, `char` ise `int`'e örtük dönüşür ve `i` değişkeni 65 olur.

Soru 4.

Cevap: A. Soldan sağa değerlendirilir. `++a` önce `a`'yı 4 yapar ve değer olarak 4 verir. Sonra `a--` değer olarak 4 verir ve `a` 3'e iner. Çarpım `4 * 4 = 16` olur, `a` son olarak 3'tür. B şıkkı `a--` işleminin yapıldığını, E şıkkı ise çarpmanın `4 * 3` olduğunu varsayanların cevabıdır.

Soru 5.

Cevap: A. Önce `(int)9.8` cast'i ile 9 elde edilir (kesme, yuvarlama değil). Sonra `%` ve `*` soldan sağa yapılır: `23 % 6 = 5`, `5 * 2 = 10`. Son olarak `9 + 10 = 19`. B şıkkı `9.8`'i `10`'a yuvarlayanların, C şıkkı toplamayı öne alarak `(9 + 23) % 6 * 2` hesaplayanların, D şıkkı çarpmayı unutanların, E şıkkı ise cast'i hiç hesaba katmayanların cevabıdır.

Soru 6.

- `a / b` = `17 / 5` = 3 ve `a % b` = 2.
- `c`, önce tam sayı bölmesiyle 3 olur, sonra `3.0` biçimine çevrilir; ekrana `3` yazılır. `d` ise `17.0 / 5 = 3.4` olur.
- `b / a + b % a` = `5 / 17 + 5 % 17` = `0 + 5` = 5.

- `a / (double)b * 0` = `3.4 * 0` = 0.

```
ÇIKTI
3 2
3 3.4
5
0
```

Soru 7.

Değişkenleri adım adım izleyelim:

İşlem	`n`	`p`	`q`	`r`
`int n = 3;`	3			
`p = n++ + n++` (3 kullanılır, `n` = 4; 4 kullanılır, `n` = 5)	5	7		
`q = ++n * 2` (`n` = 6, 6 kullanılır)	6	7	12	
`r = n-- - --n` (6 kullanılır, `n` = 5; `n` = 4, 4 kullanılır)	4	7	12	2

```
ÇIKTI
4 7 12 2
```

Soru 8.

- `h = (char)(h + 2)`: `'a'` (97) + 2 = 99, cast ile `'c'` olur.
- `h + 1` işleminde `char` ve `int` toplanır; sonuç `int`'tir: 100.
- `h - 32` = 67 ve `(char)67` = `'C'` (büyük harf).
- `"" + h + 1`: önce `"" + h` ile `"c"`, sonra `"c1"` olur.
- `Math.Round(6.5)` tam ortada çift sayıya gider: 6. `6 + 2 = 8`.
- `Math.Round(7.5)` = 8, `Math.Round(8.5)` = 8 (çifte yuvarlanır). Okuldaki yuvarlamayı bekleyen `8 9` bekler, bu yanlıştır.

```
ÇIKTI
c
100
C
c1
8
8 8
```

Soru 9.

- `a` `null` olduğu için `a ?? 7` ifadesi 7 verir.
- `a ??= 3` değişkene 3'ü atar; `a ??= 9` ise `a` artık dolu olduğundan bir şey yapmaz. `b = a * 2` = 6.
- `c` `null` olduğu için `c + 1` de `null` olur ve `HasValue` `False` verir.
- `c == null` ifadesi `True` verir.

```
ÇIKTI
7
3 6
False
True
```

Soru 10.

`Sayac` bir sınıftır (referans tipi): `s2 = s1` ataması aynı nesneyi gösterir, bu yüzden `s2.Deger = 5` ataması `s1.Deger` değerini de değiştirir. `Cift` bir yapıdır (değer tipi): `c2 = c1` ataması kopya oluşturur, `c2.Deger = 5` ise `c1`'i etkilemez. `string` referans tipi olsa da değiştirilemezdir: `m2 += "ne"` yeni bir metin üretir ve `m2`'yi ona bağlar, `m1` ise `"kar"` olarak kalır.

```
ÇIKTI
5 5
0 5
kar karne
```

Soru 11.

Hatalı satırlar ve düzeltmeleri:

- `kucuk = kucuk + 100;` işleminin sonucu `int`'tir ve `byte`'a örtük dönüşmez (`CS0266`). Düzeltme: `kucuk = (byte)(kucuk + 100);` ya da `kucuk += 100;`. Not: `200 + 100 = 300` değeri `byte` aralığına sığmaz; sarma sonucu 44 olur. Bu nedenle bu satırda `byte` yerine `int` kullanmak daha doğru olurdu.
- `float oran = 0.5;` satırında `0.5` bir `double` sabitidir (`CS0664`). Düzeltme: `float oran = 0.5f;`.
- `decimal tutar = 12.5m * 1.18;` satırında `decimal` ile `double` bir arada kullanılamaz (`CS0019`). Düzeltme: `12.5m * 1.18m`.
- `if (sayac = 0)` koşulu `int` verir, `bool` değildir (`CS0029`). Düzeltme: `if (sayac == 0)`.

Soru 12.

`b + 5` ifadesinde `byte` işlenen önce `int`'e yükseltilir, bu yüzden sonuç `int`'tir. `int`'ten `byte`'a örtük dönüşüm olmadığı için `b = b + 5;` derlenmez (`CS0266`). Bileşik atama `b += 5` ise derleyici tarafından `b = (byte)(b + 5)` olarak ele alınır: sonuç, sol taraftaki değişkenin tipine otomatik olarak cast edilir. Bu cast veri kaybına yol açabilir; sonuç `byte` aralığına sığmazsa değer sessizce sarar.

Soru 13.

Son basamak `% 10` ile, diğer basamaklar önce `/` ile kaydırılıp sonra `% 10` ile alınır.

```
int sayi = 4821;
int toplam = sayi % 10            // 1
           + sayi / 10 % 10       // 2
           + sayi / 100 % 10      // 8
           + sayi / 1000;         // 4
Console.WriteLine(toplam);
```

```
ÇIKTI
15
```

Soru 14.

`9 / 5` yazarsak iki `int` bölündüğü için sonuç 1 olur ve yanlış çıkar. `c * 9 / 5` sıralamasında `c` bir `double` olduğundan çarpma da bölme de `double` ile yapılır.

```
double c = 37.5;
double f = c * 9 / 5 + 32;
Console.WriteLine($"{c} C = {f} F");
```

```
ÇIKTI
37.5 C = 99.5 F
```

Türkçe bölge ayarlı bir bilgisayarda ondalık ayraç virgül olarak görünür (`37,5 C = 99,5 F`).

Soru 15.

Bir saatte 3600 saniye vardır. `/ 3600` saati verir, `% 3600` saatten arta kalan saniyeyi. Kalan saniye `/ 60` ile dakikaya, `% 60` ile geriye kalan saniyeye bölünür.

```
int toplamSaniye = 5025;
int saat = toplamSaniye / 3600;
int kalan = toplamSaniye % 3600;
int dakika = kalan / 60;
int saniye = kalan % 60;
Console.WriteLine($"{saat} saat {dakika} dakika {saniye} saniye");
```

```
ÇIKTI
1 saat 23 dakika 45 saniye
```

Soru 16.

`(long)a * b` ifadesinde çarpma `long` ile yapılır. `checked(a * b)` ise çarpma `int` sınırını aştığı için `OverflowException` fırlatır; `catch` bloğu bunu yakalar.

```
int a = 100_000;
int b = 100_000;
long guvenli = (long)a * b;
Console.WriteLine($"long ile: {guvenli}");
try
{
    int carpim = checked(a * b);
    Console.WriteLine(carpim);
}
catch (OverflowException)
{
    Console.WriteLine("int sınırı aşıldı");
}
```

```
ÇIKTI
long ile: 10000000000
int sınırı aşıldı
```

Bölüm 43 Konsol Girdi/Çıktı ve Biçimlendirme

Soru 1.

Cevap: C. `ReadLine()` satırı `string?` olarak döndürür; girdi bittiğinde (EOF) `null` verebildiği için tip `string` değil `string?`'tir. Sayı döndürmez (A), dizi döndürmez, tek karakter değil satırın tamamını verir (D) ve dönüş tipi `object` değildir (E).

Soru 2.

Cevap: B. `+` soldan sağa işlenir. `"Toplam: " + 2` önce `"Toplam: 2"` olur, ardından `+ 3` metne eklenir: `Toplam: 23`. Toplam 5 olsun isteniyorsa `(2 + 3)` parantez içine alınmalıydı.

Soru 3.

Cevap: B. `D3` tam sayıyı en az üç basamak olacak şekilde başa sıfır koyarak yazar. `{7,3}` yalnızca üç karakterlik alana sağa yaslar (`  7`), `F3` ve `N3` ise `7.000` üretir, `{7,-3}` sola yaslı `7  ` verir.

Soru 4.

Cevap: B. Türkçe kültürde nokta binlik ayraçtır ve hata vermeden yok sayılır; `"1.250"` değeri `1250` olarak okunur. A şıkkı nokta ondalık ayraç olsaydı (örneğin `en-US`) doğru olurdu. Hata oluşmaz (C).

Soru 5.

Cevap: B. `??` yalnızca soldaki değer `null` ise sağdakini seçer. Kullanıcı yalnızca Enter'a basarsa `ReadLine()` `null` değil `""` döndürür (A yanlış). `ReadLine()` bir kez çağrılır (D), her çağrıda `""` vermez (C) ve EOF'ta istisna oluşmaz, `ad` boş metin olur (E).

Soru 6.

- `{a,4}` yedinin soluna üç boşluk, `{a,-4}` sağına üç boşluk koyar; `{a:D4}` `0007` verir.
- `2.0 / 3` yaklaşık `0.6667`'dir: `F3` ile `0.667`, `0.0` kalıbıyla `0.7`.
- `{0}-{1}-{0}` içinde sıfırıncı değer iki kez kullanılır.
- `{{{a}}}` yazımında dıştaki iki parantez süslü parantezin kendisini, içteki çift ise değeri verir: `{7}`.
- `N1` binlik ayraçlı bir ondalık basamak, `X2` en az iki basamaklı büyük harfli onaltılık yazar.

```
ÇIKTI
[   7][7   ][0007]
0.667 0.7
7-x-7
{7}
1,234.5 FF
```

Soru 7.

Türkçe kültürde `"2.5"` metnindeki nokta binlik ayraçtır ve atılır; `x` değeri `25` olur. `25 / 4 = 6,25`. Türkçe kültürle yazılınca ondalık ayraç virgül, değişmez kültürle yazılınca nokta olur.

```
ÇIKTI
25
6,25
6.25
```

Soru 8.

Birinci sorun derleme hatasıdır: `ReadLine()` `string?` döndürür, `int` değişkene doğrudan atanamaz (`CS0029`). Metni sayıya çevirmek gerekir: `int yas = int.Parse(Console.ReadLine() ?? "0");` ya da daha güvenli olarak `int.TryParse`. İkinci sorun mantık hatasıdır: `"Gelecek yıl: " + yas + 1` metinle birleştirdiği için örneğin `20` girilince `Gelecek yıl: 201` yazar. Doğrusu `"Gelecek yıl: " + (yas + 1)` ya da `$"Gelecek yıl: {yas + 1}"` biçimindedir.

Soru 9.

Derlenir; çünkü biçim metni ile argüman sayısının uyuşması derleme zamanında denetlenmez. Çalıştırıldığında `{2}` için değer bulunamadığından `FormatException` fırlatılır. Fazla argüman verilseydi sessizce yok sayılırdı. Dize içine değer yerleştirmede (`$"..."`) bu sorun oluşmaz.

Soru 10.

Satırın başında `$` olmadığı için metin olduğu gibi yazılır: `Merhaba {ad}!`. Derleyici hata ya da uyarı vermez. Doğrusu `Console.WriteLine($"Merhaba {ad}!");` biçimindedir.

Soru 11.

Döngü sonsuza dek döner. Dosya bitince `ReadLine()` sürekli `null` döndürür; `?? ""` bunu boş metne çevirir ve `int.TryParse("")` her seferinde `false` verir, `break`'e hiç ulaşılmaz. Düzeltme: `null` durumu ayrıca denetlenmelidir.

```
int sayi;
while (true)
{
    string? metin = Console.ReadLine();
    if (metin is null)
    {
        return;               // girdi bitti, çık
    }
    if (int.TryParse(metin, out sayi))
    {
        break;
    }
}
```

Soru 12.

Virgül noktaya çevrilir ve sayı `InvariantCulture` ile çözülür; böylece hem `3,5` hem `3.5` kabul edilir. Geçersiz girdide döngü soruyu yineler. Dönüş değeri olan bir metot kullandık (Bölüm 46).

```
using System.Globalization;

double Oku(string soru)
{
    while (true)
    {
        Console.Write(soru);
        string? s = Console.ReadLine();
        if (s is null)
        {
            return 0;
        }
        string duzgun = s.Trim().Replace(',', '.');
        if (double.TryParse(duzgun, NumberStyles.Float,
                            CultureInfo.InvariantCulture, out double d))
        {
            return d;
        }
        Console.WriteLine("Geçersiz sayı.");
    }
}

double a = Oku("Birinci sayı: ");
double b = Oku("İkinci sayı: ");
Console.WriteLine($"Toplam: {a + b:F2}");
Console.WriteLine($"Ortalama: {(a + b) / 2:F2}");
```

```
ÇIKTI
Birinci sayı: abc
Geçersiz sayı.
Birinci sayı: 3,5
İkinci sayı: 2.5
Toplam: 6.00
Ortalama: 3.00
```

Soru 13.

Başlık ve satırlar aynı genişlik değerleriyle (`3`, `6`, `6`) yazılır. Başlıktaki metinler de aynı biçimde hizalanır.

```
Console.WriteLine($"{"n",3}{"kare",6}{"küp",6}");
for (int n = 1; n <= 5; n++)
{
    Console.WriteLine($"{n,3}{n * n,6}{n * n * n,6}");
}
```

```
ÇIKTI
  n  kare   küp
  1     1     1
  2     4     8
  3     9    27
  4    16    64
  5    25   125
```

Soru 14.

Girdi bittiğinde (`null`) program `return` ile sonlanır; aksi halde `TryParse` ve aralık denetimi başarılı olana kadar soru tekrarlanır. `&&` kısa devre yaptığından aralık yalnızca çeviri başarılıysa denetlenir.

```
int sayi;
while (true)
{
    Console.Write("1-10 arası sayı: ");
    string? metin = Console.ReadLine();
    if (metin is null)
    {
        return;
    }
    if (int.TryParse(metin, out sayi) && sayi >= 1 && sayi <= 10)
    {
        break;
    }
    Console.WriteLine("Geçersiz giriş.");
}
Console.WriteLine($"Girdiğiniz sayı: {sayi}");
```

```
ÇIKTI
1-10 arası sayı: x
Geçersiz giriş.
1-10 arası sayı: 11
Geçersiz giriş.
1-10 arası sayı: 7
Girdiğiniz sayı: 7
```

Soru 15.

Para tutarları `decimal` ile hesaplanır. Virgül noktaya çevrilip değişmez kültürle okunur; çıktıda etiketler `-10`, tutarlar `12:N2` ile hizalanır. Çıktıdaki binlik ve ondalık ayraçlar kültür bağımsız (noktalı) gösterilmiştir.

```
using System.Globalization;

Console.Write("Ad soyad: ");
string ad = Console.ReadLine() ?? "";
Console.Write("Brüt maaş: ");
string metin = (Console.ReadLine() ?? "0").Trim().Replace(',', '.');
decimal brut = decimal.Parse(metin, NumberStyles.AllowDecimalPoint,
                             CultureInfo.InvariantCulture);
decimal kesinti = brut * 0.15m;
decimal net = brut - kesinti;
```

```
Console.WriteLine(ad);
Console.WriteLine($"{"Brüt",-10}{brut,12:N2}");
Console.WriteLine($"{"Kesinti",-10}{kesinti,12:N2}");
Console.WriteLine($"{"Net",-10}{net,12:N2}");
```

```
ÇIKTI
Ad soyad: Elif Su Demir
Brüt maaş: 20000,40
Elif Su Demir
Brüt         20,000.40
Kesinti       3,000.06
Net          17,000.34
```

Soru 16.

`%-8s` sola yaslı sekiz karakter (`{ad,-8}`), `%5d` sağa yaslı beş karakter (`{n,5}`), `%8.3f` sekiz karakterlik alanda üç ondalık (`{x,8:F3}`) anlamına gelir. `WriteLine` satır sonunu kendisi ekler.

```
Console.WriteLine($"{ad,-8}|{n,5}|{x,8:F3}|");
```

Bölüm 44 Koşullu İfadeler ve Desen Eşleme

Soru 1.

Cevap: C. `b` bir `bool?` olduğundan `if (b)` derlenmez (`CS0266`). `b == true` ise `bool` üretir ve geçerlidir. A, D ve E sıradan `bool` ifadelerdir; `x is > 3` de bir ilişkisel desendir ve `bool` verir.

Soru 2.

Cevap: C. İçinde deyim bulunan bir `case` bölümü, kontrolü sonraki etikete bırakamaz. C ve C++'ta bu yazım derlenir ve düşer; C#'ta `CS0163` hatasıdır. A şıkkı C davranışını anlatır.

Soru 3.

Cevap: B. Kollar yukarıdan aşağı denenir. `10 < 10` yanlıştır, `10 < 20` doğrudur; sonuç `"b"` olur. `_` kolu bulunduğu için ifade her değeri kapsar.

Soru 4.

Cevap: C. `switch` ifadesinde `break` ve düşme yoktur; ilk eşleşen kol sonucu verir. `_` kolu zorunlu değildir (yokluğu uyarı verir) ve genellikle en sona yazılır; en üste yazılırsa sonraki kollara hiç ulaşılamaz. İfade `int` ile sınırlı değildir.

Soru 5.

Cevap: A. `n is not 3` ifadesi `n != 3` anlamındadır. `n` 4 olduğundan sonuç `True` yazılır.

Soru 6.

`n` 2 olduğundan `case 2:` bölümü çalışır ve `toplam` 10 olur. `goto case 3;` ile `case 3:` bölümüne geçilir; `toplam` 110 olur ve `break` ile çıkılır.

```
ÇIKTI
110
```

Soru 7.

`(x, y)` demeti `(0, 5)` olur. İlk ifadede `(0, 0)` uymaz, `(0, _)` uyar: `"y ekseni"`. İkinci ifadede demet `(y - 5, x)` yani `(0, 0)` olur ve ilk kol eşleşir: `"orijin"`.

```
ÇIKTI
y ekseni orijin
```

Soru 8.

Elemanlar sırayla: `15` birinci kola uyar (`A`), `4.0` bir `double`'dır (`D`), `"kaf"` metin olduğundan uzunluğu `3` yazılır, `-2` hiçbir `when` kolunu sağlamaz ve `int =>` kolundan `C` çıkar, `'z'` bir `char` olduğu için `_` kolu (`?`), `8` ikinci kola uyar (`B`). Altı eleman işlendi.

```
ÇIKTI
AD3C?B 6
```

Soru 9.

`ad` `null` olduğundan `ad?.Length` `null` verir ve `?? -1` yedek değeri getirir: `-1`. `yas` `null` iken `yas ?? 18` değeri 18'dir; `>= 18` doğru olduğundan `yetişkin` yazılır. `yas ??= 12;` ile `yas` 12 olur; `12 >= 18` yanlıştır: `çocuk`. `12 is > 10 and < 20` doğrudur: `True`. `ad is null` de `True` verir.

```
ÇIKTI
-1
yetişkin
çocuk
True
True
```

Soru 10.

Döngü değişkeni sırayla 0'dan 4'e gider. `i = 0`: `default` kolu `0` yazar, `switch` biter, ardından `-` yazılır. `i = 1`: `continue` döngünün sonraki turuna geçer, `-` yazılmaz. `i = 2`: `2-`. `i = 3`: `break` yalnızca `switch`'ten çıkar, `-` yazılır. `i = 4`: `4-`.

```
ÇIKTI
0-2--4-
```

Soru 11.

`7 < 5` yanlış, `7 < 10` doğru: `s` = `"orta"`. `7`, `> 5 and < 8` desenine uyar: `t` = `"yedi civarı"`. `x is not (> 3 and < 6)`: 7 bu aralığın dışında olduğu için `True`.

```
ÇIKTI
orta yedi civarı True
```

Soru 12.

Dört sorun şunlardır:

1. `case 1:` bölümü `break` ile bitmiyor (`CS0163`). Çözüm: `Console.WriteLine("Ocak");` satırından sonra `break;` eklemek.
2. `case sinir:` etiketi sabit değil; `sinir` sıradan bir değişken (`CS9135`). Çözüm: `const int sinir = 5;` yazmak ya da doğrudan `case 5:`.
3. `if (izin)` koşulu `bool?` tipinde (`CS0266`). Çözüm: `if (izin == true)` ya da `if (izin ?? false)`.
4. `> 6 => "yaz"` kolu, önceki `>= 1` kolunun kapsadığı değerleri yakalıyor; ulaşılamaz (`CS8510`). Çözüm: kolların sırasını değiştirmek: `> 6` kolunu `>= 1` kolunun üstüne almak.

Soru 13.

`ad?.StartsWith("M")` ifadesi `ad` `null` olabileceği için `bool?` tipindedir; `if` ise `bool` ister (`CS0266`). Düzeltme: `if (ad?.StartsWith("M") == true)`. `null` durumu da ayrıca ele alınacaksa `if (ad is not null && ad.StartsWith("M"))` yazılabilir.

Soru 14.

```
Console.Write("Ay: ");
int ay = int.Parse(Console.ReadLine()!);
Console.Write("Yıl: ");
int yil = int.Parse(Console.ReadLine()!);

bool artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;

if (ay is < 1 or > 12)
{
    Console.WriteLine("Geçersiz ay");
}
else
{
    int gun = ay switch
    {
        2 when artik => 29,
        2 => 28,
        4 or 6 or 9 or 11 => 30,
        _ => 31
    };
    Console.WriteLine($"{gun} gün");
}
```

Önce ayın geçerliliğine bakılır. `2 when artik` kolu `2 =>` kolundan önce gelmelidir. `or` deseni 30 günlük dört ayı tek kolda toplar.

Soru 15.

```
double a = double.Parse(Console.ReadLine()!);
double b = double.Parse(Console.ReadLine()!);
double c = double.Parse(Console.ReadLine()!);

double[] k = { a, b, c };
Array.Sort(k);                    // k[0] en küçük, k[2] en büyük

string tur = (a, b, c) switch
{
    _ when k[0] <= 0 || k[0] + k[1] <= k[2] => "geçersiz",
    _ when a == b && b == c => "eşkenar",
    _ when a == b || b == c || a == c => "ikizkenar",
    _ => "çeşitkenar"
};
Console.WriteLine(tur);
```

Kenarlar sıralanınca "en küçük iki kenarın toplamı üçüncüden büyük olmalı" koşulu tek karşılaştırmayla yazılır. Kollar sıralıdır: geçersizlik önce elenir, sonra en özel durum (eşkenar), ardından ikizkenar denenir. `Array.Sort` ve diziler Bölüm 47'de işlenecektir.

Soru 16.

```
double x = double.Parse(Console.ReadLine()!);
double y = double.Parse(Console.ReadLine()!);
char islem = Console.ReadLine()![0];

string sonuc = (islem, y) switch
{
    ('/', 0) => "Sıfıra bölünemez",
    ('+', _) => (x + y).ToString(),
    ('-', _) => (x - y).ToString(),
    ('*', _) => (x * y).ToString(),
    ('/', _) => (x / y).ToString(),
    _ => "Geçersiz işlem"
};
Console.WriteLine(sonuc);
```

Tuple deseni işlem karakteri ile ikinci sayıyı birlikte sınar. `('/', 0)` kolu bölme kolundan önce geldiği için sıfıra bölme baştan yakalanır.

Bölüm 45 Döngüler

Soru 1.

Cevap: C. `while` koşulu `bool` olmak zorundadır. `int` değeri koşul olarak kabul edilmez ve derleyici `CS0029` hatası verir. C'de aynı kod üç kez döner; C#'ta ise hiç derlenmez.

Soru 2.

Cevap: B. `do-while` gövdeyi önce bir kez çalıştırır ve `10` yazdırır. Ardından `10 < 5` yanlış olduğu için döngü biter. Satır sonu yazılmadığı için çıktı yalnızca `10` olur.

Soru 3.

Cevap: C. `for (;;) { break; }` geçerli bir sonsuz döngüdür ve `break` ile hemen çıkar. Diğer seçenekler: A `while (1)` yüzünden `CS0029` verir; B döngüden sonra `i` kapsam dışında kaldığı için `CS0103` verir; D iterasyon değişkenine atama yaptığı için `CS1656` verir; E `int` koşulu yüzünden `CS0029` verir.

Soru 4.

Cevap: A. Dış döngü her `i` için iç döngüyü baştan başlatır. İç döngü `j == 2` olunca `break` ile biter, yani her dış turda yalnızca `j = 1` için `sayac` artar. Dört dış tur olduğu için `sayac` 4 olur.

Soru 5.

Cevap: `19`. `i` değerleri 3, 6 ve 9 `continue` ile atlanır. `i == 8` olduğunda döngü `break` ile biter. Toplanan sayılar 1, 2, 4, 5 ve 7'dir; toplamları 19'dur.

Soru 6.

Cevap: İki hata vardır.

- `p += 5;` satırında `CS1656: Cannot assign to 'p' because it is a 'foreach iteration variable'`. Düzeltme: değeri yazdırmak ya da başka bir değişkene almak gerekir. Örneğin `Console.WriteLine(p + 5);`.
- `while (say)` satırında `CS0029: Cannot implicitly convert type 'int' to 'bool'`. Düzeltme: karşılaştırma yazılır, örneğin `while (say < 3)`.

Düzeltilmiş hali:

```
int[] puanlar = { 10, 20, 30 };
foreach (int p in puanlar)
{
    Console.WriteLine(p + 5);
}
int say = 0;
while (say < 3)
{
    say++;
}
Console.WriteLine(say);
```

Soru 7.

Cevap: `11 21 22 31 32 33` (sonunda boşluk). Her `i` için `j` değeri `i`'yi geçtiğinde iç döngü `break` ile biter. `i = 1` için yalnızca `11`, `i = 2` için `21` ve `22`, `i = 3` için `31`, `32` ve `33` yazılır.

Soru 8.

Cevap: B. `string` üzerinde `foreach` dolaştığında her eleman bir `char` olur. Bu nedenle `var` ile tanımlanan `c` de `char` tipindedir.

Soru 9.

Cevap: `135`. Bu döngüde `i++` gövdenin başında olduğu için `continue` ilerlemeyi atlamaz. Çift değerler `continue` ile atlanır ve yalnızca 1, 3 ve 5 yazılır.

Soru 10.

Cevap: D. `int` tipinin en küçük değerinin pozitif karşılığı `int` sınırının dışında kaldığı için `Math.Abs` `OverflowException` fırlatır. Bu durumda çıktı üretilmez.

Soru 11.

Cevap: E. C#'ta etiketli `break` yoktur. `break dis;` satırı bir söz dizimi hatası olarak bildirilir (`CS1002: ; expected`).

Soru 12.

Örnek çözüm:

```
int toplam = 0, adet = 0;
while (true)
{
    Console.Write("Sayı: ");
    int sayi = Convert.ToInt32(Console.ReadLine());
    if (sayi == 0)
    {
        break;
    }
    if (sayi > 0)
    {
        toplam += sayi;
        adet++;
    }
}
Console.WriteLine($"Pozitif adet: {adet}, toplam: {toplam}");
```

Girdi `5`, `-3`, `12`, `0` olduğunda çıktı `Pozitif adet: 2, toplam: 17` olur. Negatif sayı sayılmaz ve toplamaya girmez.

Soru 13.

Örnek çözüm:

```
Console.Write("n: ");
int n = Convert.ToInt32(Console.ReadLine());
for (int i = n; i >= 1; i--)
{
    Console.WriteLine(new string('*', i));
}
```

Döngü `n`'den `1`'e doğru sayar; her turda `i` yıldız içeren bir satır yazılır. `n = 4` için satırlar `****`, `***`, `**` ve `*` olur.

Soru 14.

Örnek çözüm:

```
string metin = "Merhaba Dünya";
string sesliler = "aeıioöuü";
int sayi = 0;
foreach (char c in metin)
{
    if (sesliler.Contains(char.ToLower(c)))
    {
        sayi++;
    }
}
Console.WriteLine(sayi);
```

Metindeki sesliler `e`, `a`, `a`, `ü` ve `a`'dır; toplam `5` eder. Harfler önce `char.ToLower` ile küçültülür, böylece büyük harfli sesliler de sayılır.

Bölüm 46 Metotlar

Soru 1.

Cevap: C. Aşırı yüklemeyi parametre listesi (sayı, tip, sıra) ayırır. C şıkkında parametre sayısı farklıdır. A ve D yalnızca dönüş tipi bakımından, B ise yalnızca parametre adları bakımından farklıdır; imza aynı kaldığı için `CS0111` hatası verirler. E'deki `public` bir erişim belirtecidir ve imzayı değiştirmez.

Soru 2.

Cevap: C. `out` parametresine metot dönmeden önce mutlaka değer atanmalıdır (`CS0177`). A şıkkı `ref` için geçerlidir; `out` için değişkene önceden değer atanması gerekmez. `out` çağrıda da yazılır (B yanlış), bir metotta istenen sayıda `out` parametresi olabilir (D yanlış) ve değer tipleriyle de kullanılır (E yanlış).

Soru 3.

Cevap: A. `Math.Round(2.5)` tam ortada çift sayıya gider ve 2 verir. `Math.Floor(-1.5)` aşağı yuvarlar ve -2 verir. Toplam `2 + (-2) = 0` olur. Okuldaki yuvarlamayı bekleyen 3 bulup `1` cevabını seçer.

Soru 4.

Cevap: B. `Next(min, max)` alt sınırı dahil eder, üst sınırı dahil etmez: 2, 3 ve 4 üretilebilir. 5 hiçbir zaman gelmez.

Soru 5.

`x` değerle geçer; metot içinde `x`'in kopyası ikiye katlanır, `a` değişmez. `y` ise `ref` olduğu için `b` değişkeninin kendisidir; 10 olur.

```
ÇIKTI
5 10
```

Soru 6.

İlk çağrıda `x` ile `d` aynı diziyi gösterir; `x[0] = 10` ataması dışarıdan görünür. `x = new int[] {7, 7}` ise yalnızca yerel bağlantıyı değiştirir. İkinci çağrıda parametre `ref` olduğu için `x`, `d`'nin kendisidir: önce dizinin ilk elemanı 20 yapılır, sonra `d` yeni diziye bağlanır.

```
ÇIKTI
10,2,3
7,7
```

Soru 7.

Atlanan parametreler varsayılan değerlerini alır. Üçüncü çağrıda `unvan` atlanmıştır ve `sayi` adıyla verilmiştir. Dördüncü çağrıda argüman sırası değiştirilmiş, `sayi` atlanmıştır.

```
ÇIKTI
Bay Can x1
Bayan Ece x1
Bay Mert x3
Dr. Selin x1
```

Soru 8.

Derleyici, argümanı kayıpsız dönüştürebildiği adaylar arasından en yakın tipi seçer. `5` tam eşleşmeyle `int`'e, `5.0` `double`'a, `5L` `long`'a gider. `'a'` (`char`) `int`'e, `long`'a ve `double`'a örtük dönüşür, ama `int` en yakın olandır. `2.5f` (`float`) yalnızca `double`'a dönüşebilir. `"5"` metindir.

```
ÇIKTI
int
double
long
int
double
string
```

Soru 9.

`Bol(47, 5)` demetini `var (b, k)` açar: 47 / 5 = 9, 47 % 5 = 2. `int.TryParse("42")` başarılıdır, `sonuc` 42 olur. `"x2"` çevrilemez; `TryParse` başarısızlıkta `out` değerine 0 atar. Soldan sağa değerlendirme sayesinde `s` ve `t`, çağrıdan sonra okunur.

```
ÇIKTI
9 2
True 42
False 0
```

Soru 10.

İfade soldan sağa değerlendirilir. İlk `Say()` çağrısı 1 döndürür, ikincisi 2 döndürür ve çarpma 20 yapar: `1 + 20 = 21`. Yerel fonksiyon `sayac` değişkenini yakaladığı için sayaç iki kez artmıştır.

```
ÇIKTI
21
2
```

Soru 11.

1. `Takas(x, y);` — `ref` parametrelere çağrıda da `ref` yazılmalı: `Takas(ref x, ref y);` (`CS1620`).

2. `Yarim` — `n > 0` olmadığında metot değer döndürmeden biter (`CS0161`). Sona `return 0;` eklenmeli ya da `if`'in ardına bir `return` yazılmalı.
3. `Oku` — `out` parametresine hiç atama yapılmıyor (`CS0177`). Gövdeye örneğin `s = 0;` eklenmeli.
4. `Ekle(int a = 1, int b)` — varsayılan değerli parametre zorunlu parametrenin önünde olamaz (`CS1737`). Doğrusu `Ekle(int b, int a = 1)` ya da `b`'ye de varsayılan değer vermektir.

Soru 12.

Birinci fark: `ref` argümanına çağrıdan önce değer atanmış olmalıdır; `out` argümanına atanması gerekmez. İkinci fark: `out` parametresine metot dönmeden önce mutlaka değer atamak zorundadır; `ref` için böyle bir zorunluluk yoktur ve `ref` parametresi metot içinde baştan okunabilir. Çağıranın mevcut değerini metot içinde okuyup güncellemek gerekiyorsa (takas, artırma) `ref`; metot yalnızca ek bir sonuç üretiyorsa (`TryParse` gibi) `out` tercih edilir.

Soru 13.

```
Console.WriteLine(Faktoriyel(5));
Console.WriteLine(Faktoriyel(15));

static long Faktoriyel(int n)
{
    long sonuc = 1;
    for (int i = 2; i <= n; i++)
    {
        sonuc *= i;
    }
    return sonuc;
}
```

`Faktoriyel` 5 için 120, 15 için 1307674368000 verir. `15!` değeri `int` sınırını aştığı için sonuç tipi `long` seçilmiştir.

Soru 14.

```
var (enk, enb) = Aralik(7, 2, 9, 4);
Console.WriteLine($"En küçük: {enk}, en büyük: {enb}");

static (int Enk, int Enb) Aralik(params int[] sayilar)
{
    int enk = sayilar[0], enb = sayilar[0];
    foreach (int s in sayilar)
    {
        if (s < enk) enk = s;
        if (s > enb) enb = s;
    }
    return (enk, enb);
}
```

Çıktı: `En küçük: 2, en büyük: 9`. `params` argümanları diziye toplar, demet iki sonucu birlikte döndürür.

Soru 15.

```
Console.WriteLine(SicaklikCevir(100));
Console.WriteLine(SicaklikCevir(0, hedef: "K"));

static double SicaklikCevir(double deger, string hedef = "F")
{
    if (hedef == "F")
    {
        return deger * 9 / 5 + 32;
    }
```

```
    if (hedef == "K")
    {
        return deger + 273.15;
    }
    throw new ArgumentException("Geçersiz hedef birim: " + hedef);
}
```

Çıktı: `212` ve `273.15`. `hedef` verilmediğinde varsayılan `"F"` kullanılır. Metodun sonunda ayrı bir `return` gerekmez: son deyim `throw` olduğu için sona ulaşılamaz.

Soru 16.

```
Goster(7, 2);
Goster(5, 0);

static void Goster(int a, int b)
{
    if (TryBol(a, b, out double sonuc))
    {
        Console.WriteLine(sonuc);
    }
    else
    {
        Console.WriteLine("Bölme yapılamadı");
    }
}

static bool TryBol(int a, int b, out double sonuc)
{
    if (b == 0)
    {
        sonuc = 0;
        return false;
    }
    sonuc = a / (double)b;
    return true;
}
```

Çıktı: `3.5` ve `Bölme yapılamadı`. Başarısız yolda bile `sonuc`'a değer atanmıştır; `out` kuralı bunu gerektirir.

Bölüm 47 Diziler

Soru 1.

Cevap: A. `new` ile oluşturulan `double` dizisinin her elemanı `0.0` olur ve `Length` eleman sayısıdır (4). Son geçerli indeks 3'tür, ama `Length` 3 değildir. `null` yalnızca referans tipli elemanlar için varsayılandır; değer verilmemesi derleme hatası da doğurmaz.

Soru 2.

Cevap: E. Dizide de `string`'de de `Length` bir özelliktir ve parantez almaz. `Length()` diye bir metot yoktur (`CS1955`). `Count` ve `Size` dizide tanımlı değildir; `Count` koleksiyonlara aittir (Bölüm 54).

Soru 3.

Cevap: B. `b = a` ile iki değişken aynı diziyi gösterir; `b[1] = 20` bu diziyi `{ 1, 20, 3 }` yapar. Sonra `a` yeni bir diziye yöneltilir, ama `b` hâlâ eski diziyi gösterir. `b[1] + a[1]` = 20 + 8 = 28.

```
ÇIKTI
28
```

Soru 4.

Cevap: D. `d`, `a` ile aynı diziyi gösterir; `d[0] = 50` dışarıdan görünür. `n` ise `x`'in değerinin kopyasıdır; onu değiştirmek `x`'i etkilemez.

```
ÇIKTI
50 1
```

Soru 5.

Cevap: A. `a[^2]` sondan ikinci eleman olan 40'tır. `a[1..3]` 1. ve 2. indeksleri alır (3 hariç): iki eleman. `a[3..]` 3. indeksten sona kadar olduğu için `{ 40, 50 }`, yani iki eleman verir.

Soru 6.

Cevap: C. `m.Length` dış dizinin uzunluğudur: 3. `m[1].Length` = 3 (`{ 3, 4, 5 }`). `m[2][0]` = 6. Toplam 3 + 3 + 6 = 12. Kullanılan bütün indeksler geçerli olduğu için istisna oluşmaz.

Soru 7.

- İlk döngü `i = 1, 3` için çalışır: `t = a[1] + a[3] = 8 + 6 = 14`.
- İkinci döngü sondan başa her elemanı bir sağa taşır; `a[0]`'a dokunulmadığı için 3 kalır ve dizi `{ 3, 3, 8, 1, 6 }` olur. Son eleman yerine bir öncekinin kopyası geldiği için bu bir döndürme değil, öteleme sırasında bir elemanın kaybolmasıdır.
- `a[1]` artık 3'tür; 3'ten büyük elemanlar 8 ve 6'dır, yani `k = 2`.

```
ÇIKTI
14
3, 3, 8, 1, 6
2
```

Soru 8.

- `Degistir(a)`: `d[0] = d[2]` ile `a = { 3, 2, 3 }` olur. Sonraki `d = new int[] ...` yalnızca yerel `d`'yi değiştirir; `d[1] = 0` yeni diziye yazılır ve `a` etkilenmez.
- `IkiKat(a)`: `Clone()` ayrı bir kopya üretir, kopya ikiyle çarpılır: `b = { 6, 4, 6 }`. `a` değişmez.
- `c = a` takma addır; `c[1] = 7` ile `a = { 3, 7, 3 }` olur.
- `a == c` aynı diziyi gösterdikleri için `True`, `a == b` ayrı diziler olduğu için `False` verir.

```
ÇIKTI
3, 7, 3
6, 4, 6
True False
```

Soru 9.

- Ana köşegen: `m[0][0] + m[1][1] + m[2][2]` = 2 + 5 + 6 = 13. Ters köşegen: `m[0][2] + m[1][1] + m[2][0]` = 1 + 5 + 1 = 7.
- Her satırın son elemanı: 1, 3, 6 (sonda boşluk kalır).
- `ikinci = m[1]` yeni bir dizi değil, ikinci satırın kendisine ikinci bir referanstır. `ikinci[0] = 0` aslında `m[1][0] = 0` demektir. Sonuç: 0 + 3 * 1 = 3.

```
ÇIKTI
13 7
1 3 6
3
```

Soru 10.

`b = a[1..3]` yeni bir dizi oluşturur: `{ 10, 30 }`. `Array.Sort(a)` yalnızca `a`'yı değiştirir, `b` etkilenmez. Sıralı dizi `{ 10, 20, 30, 40 }` olur. 25 dizide yok; 20 ile 30 arasına, yani 2. indekse girerdi. `BinarySearch` bu durumda `~2` = -3 döndürür.

```
ÇIKTI
10, 20, 30, 40
10, 30
-3 2
```

Soru 11.

Üç hatalı satır şunlardır:

- `int sayilar[3] = { 1, 2, 3 };` : C tarzı tanım C#'ta geçersizdir. Köşeli parantez tipin yanına yazılır: `int[] sayilar = { 1, 2, 3 };`.
- `int uzunluk = b.Length();` : `Length` bir özelliktir, parantez alamaz. Doğrusu `b.Length`.
- `x = x + 1;` : `foreach` döngü değişkenine atama yapılamaz (`CS1656`). Elemanı değiştirmek için indeksli `for` kullanılmalıdır: `for (int i = 0; i < b.Length; i++) { b[i] = b[i] + 1; }`.

Soru 12.

Döngü koşulu `i <= a.Length` olduğu için son turda `i` 3 olur ve `a[3]` dizinin dışındadır; program `IndexOutOfRangeException` ile durur (önceki üç eleman yazdırılmış olur). Dizinin geçerli indeksleri 0, 1 ve 2'dir. Koşul `i < a.Length` olmalıdır. İndeks gerekmiyorsa `foreach (int x in a)` bu hatayı baştan önler.

Soru 13.

`m.Length` toplam eleman sayısıdır: 15. `m.GetLength(0)` satır sayısı 3, `m.GetLength(1)` sütun sayısı 5'tir. Dıştaki döngü `i < m.GetLength(0)`, içteki döngü `j < m.GetLength(1)` olmalıdır. İç döngüyü `j < m.Length` yazmak, 15'e kadar gitmek ve istisna almak demektir.

Soru 14.

İki değişken tutuyoruz: `birinci` (en büyük) ve `ikinci`. Yeni eleman `birinci`'den büyükse eski `birinci`, ikincilik makamına düşer. Değilse, `birinci`'den küçük olup `ikinci`'den büyükse yeni `ikinci` olur. `x < birinci` koşulu, tekrar eden en büyük değerin (45 gibi) ikinci sayılmasını engeller. Başlangıçta ikisi de dizinin kendi elemanıdır; `bulundu` bayrağı ikinci için gerçek bir değer bulunup bulunmadığını gösterir.

```
int[] a = { 12, 45, 7, 45, 30 };
int[] b = { -3, -8, -1 };
Console.WriteLine($"{string.Join(", ", a)} -> {IkinciEnBuyuk(a)}");
Console.WriteLine($"{string.Join(", ", b)} -> {IkinciEnBuyuk(b)}");

static int IkinciEnBuyuk(int[] d)
{
    int birinci = d[0];
    int ikinci = d[0];
    bool bulundu = false;           // ikinci henüz bulunmadı
    foreach (int x in d)
    {
        if (x > birinci)
        {
            ikinci = birinci;
```

```
            birinci = x;
            bulundu = true;
        }
        else if (x < birinci && (!bulundu || x > ikinci))
        {
            ikinci = x;
            bulundu = true;
        }
    }
    return ikinci;
}
```

```
ÇIKTI
12, 45, 7, 45, 30 -> 30
-3, -8, -1 -> -3
```

Soru 15.

Sonuç dizisinin `i`. elemanı, asıl dizinin `(i + k) % n` indeksli elemanıdır. Bölme kalanı, sondan taşan indeksleri başa sarar; `k` dizi uzunluğundan büyük olsa bile çalışır. Asıl dizi değiştirilmediği için son satır eski halini yazar.

```
int[] a = { 1, 2, 3, 4, 5 };
Console.WriteLine(string.Join(", ", SolaDondur(a, 2)));
Console.WriteLine(string.Join(", ", SolaDondur(a, 7)));
Console.WriteLine(string.Join(", ", a));

static int[] SolaDondur(int[] d, int k)
{
    int n = d.Length;
    int[] sonuc = new int[n];
    for (int i = 0; i < n; i++)
    {
        sonuc[i] = d[(i + k) % n];
    }
    return sonuc;
}
```

```
ÇIKTI
3, 4, 5, 1, 2
3, 4, 5, 1, 2
1, 2, 3, 4, 5
```

Soru 16.

Her satır `Split(' ')` ile parçalara ayrılır, parçalar `int.Parse` ile sayıya çevrilip iç içe iki döngüyle matrise yerleştirilir. `SimetrikMi` yalnızca köşegenin üstündeki çiftleri (`j = i + 1`'den başlayarak) `m[j, i]` ile karşılaştırır ve ilk uyumsuzlukta `false` döndürür.

```
static bool SimetrikMi(int[,] m)
{
    for (int i = 0; i < m.GetLength(0); i++)
    {
        for (int j = i + 1; j < m.GetLength(1); j++)
        {
            if (m[i, j] != m[j, i])
            {
                return false;
            }
        }
    }
    return true;
```

```
}

Console.Write("Matris boyutu: ");
int n = int.Parse(Console.ReadLine()!);
int[,] m = new int[n, n];
Console.WriteLine("Satırları girin:");
for (int i = 0; i < n; i++)
{
    string[] parcalar = Console.ReadLine()!.Split(' ');
    for (int j = 0; j < n; j++)
    {
        m[i, j] = int.Parse(parcalar[j]);
    }
}
Console.WriteLine(SimetrikMi(m) ? "Matris simetrik." : "Matris simetrik değil.");
```

```
ÇIKTI
Matris boyutu: 3
Satırları girin:
1 2 3
2 5 4
3 4 9
Matris simetrik.
```

Bölüm 48 String, StringBuilder ve Karakterler

Soru 1.

Cevap: C. İndeksler: `P0 r1 o2 g3 r4 a5 m6 l7 a8 m9 a10`. `Substring(2, 5)` ikinci indeksten başlayıp **beş karakter** alır: `o`, `g`, `r`, `a`, `m` yani `"ogram"`. A şıkkı ikinci sayıyı bitiş indeksi sanmaktan, B ve E başlangıcı yanlış almaktan, D ise yedi karakter almaktan çıkar.

Soru 2.

`merhaba8` yazılır. `s.ToUpper()` ve `s.Replace('a', 'e')` yeni metinler üretir; ama sonuçlar hiçbir değişkene atanmadığı için kaybolur ve `s` hâlâ `"merhaba"` dır. `t = s` ikisini aynı nesneye bağlar; `t + "!"` ise yeni bir metin (`"merhaba!"`, 8 karakter) üretir ve `t` ona bağlanır. `s` etkilenmez. Ekrana `"merhaba"` ile `8` birleşerek yazılır.

```
ÇIKTI
merhaba8
```

Soru 3.

Cevap: B. Parametresiz `ToLower()` geçerli kültürün kurallarını kullanır. Türkçe kültürde `I` harfinin küçüğü `ı`'dır; bu yüzden sonuç `"ısparta"` olur. `ToLowerInvariant()` (C) kültürden bağımsızdır ve `"isparta"` verir. Karşılaştırma türü (`Ordinal`, D) dönüşümün kendisini değiştirmez.

Soru 4.

Cevap: D. `"Veri"` dört karakterlidir. `Substring(2, 3)` ikinci indeksten üç karakter ister; ama sonda yalnızca iki karakter (`ri`) vardır. A geçerlidir (`'i'`). B hata vermez: başlangıç uzunluğa eşit olduğunda boş metin döner. C `"eri"` verir. E bulamadığı için `-1` döndürür, hata fırlatmaz.

Soru 5.

Cevap: C. `string` tipinde `==` içerik karşılaştırır. Aynı nesne olma denetimi için `object.ReferenceEquals` kullanılır (A). `==` metin sabitleriyle sınırlı değildir (B), büyük/küçük harfe duyarlıdır (D) ve `null` ile karşılaştırmada hata vermez (E).

Soru 6.

İlk satırda `IndexOf` bulamayınca `-1` verir. `c + 1` işleminde `char` ile `int` toplanır ve sonuç `int` olur: `98`. `(char)(c + 1)` sonucu karaktere çevirir: `b`. İkinci satırda ifade soldan sağa işlenir: `'a' + 'b'` iki karakterin **toplamıdır** (97 + 98 = 195). Sonra `"|"` ile birleştirme başlar ve `'a'`, `'b'` metne eklenir.

```
ÇIKTI
-1 98 b
195|ab
```

Soru 7.

`a` dizisi virgüllere göre bölünür: `" elma"`, `""` (iki virgül arasındaki boş parça) ve `" armut ;kiraz "`; yani üç parça. `b` dizisinde hem virgül hem noktalı virgül ayraçtır, boş parçalar atılır ve her parça kırpılır: `elma`, `armut`, `kiraz`; yani yine üç parça.

```
ÇIKTI
3 3
elma/armut/kiraz
```

Soru 8.

`sb2 = sb` aynı `StringBuilder` nesnesini gösteren ikinci bir değişken oluşturur. `Append("d")` tamponu `"abcd"` yapar, `Insert(0, 'x')` ise `"xabcd"`. İki değişken aynı nesneyi gösterdiği için ikisi de `xabcd` yazar. `sb2.Remove(1, 2)` birinci indeksten iki karakteri (`a` ve `b`) siler: `xcd`. `string` kısmında ise `s = s + "d"` yeni bir nesne üretir; `t` eski nesneyi göstermeyi sürdürür.

```
ÇIKTI
xabcd xabcd
xcd
abcd abc
```

Soru 9.

Üç sorun vardır:

- `ad[0] = 'E';` derlenmez: `string` değiştirilemez ve indeksleyici salt okunurdur (`CS0200`). Çözüm: `ad = "E" + ad.Substring(1);`.
- `ad.Trim();` sonucu atanmadığı için hiçbir etkisi yoktur (derleme hatası değil, mantık hatasıdır). Çözüm: `ad = ad.Trim();`.
- `ad.Length()` derlenmez: `Length` bir özelliktir (`CS1955`). Çözüm: `ad.Length`.

Soru 10.

`char + int` işleminin sonucu `int`'tir ve `int` değerin `char` değişkene örtük dönüşümü yoktur (`CS0266`). Açık dönüşüm gerekir:

```
char harf = 'c';
char sonraki = (char)(harf + 1);
Console.WriteLine(sonraki);
```

Bu haliyle ekrana `d` yazılır.

Soru 11.

Kapatan `"""` dört boşluk girintilidir; bu yüzden her satırdan dört boşluk silinir. İlk satır altı boşluk içerdiği için iki boşluk kalır, ikinci satırda hiç boşluk kalmaz. Metinde iki satır olduğu için `Split('\n')` iki parça verir.

```
ÇIKTI
  Merhaba Ece
Hoş geldin
2
```

Soru 12.

`t.Length` değeri 4'tür. `\U0001F600` karakterinin kod noktası 65 535'ten büyüktür; UTF-16'da iki kod birimiyle (bir yedek çiftiyle) saklanır. Dolayısıyla `a` (1) + simge (2) + `b` (1) = 4 kod birimi vardır. `EnumerateRunes()` yedek çiftini tek kod noktası olarak sayar ve sonuç 3'tür.

Soru 13.

Kelimeler `Split` ile, `RemoveEmptyEntries` seçeneğiyle ayrılır; böylece fazla boşluklar sorun olmaz. Her kelimenin ilk harfi `ToUpper(tr)`, kalanı `ToLower(tr)` ile çevrilir. Türkçe kültür verildiği için `istanbul` kelimesi `İstanbul`, `TEKNİK` kelimesi `Teknik` olur.

```
using System.Globalization;
using System.Text;

CultureInfo tr = new CultureInfo("tr-TR");

Console.Write("Kurum adı: ");
string[] kelimeler = (Console.ReadLine() ?? "")
    .Split(' ', StringSplitOptions.RemoveEmptyEntries);

StringBuilder duzgun = new StringBuilder();
StringBuilder kisa = new StringBuilder();
foreach (string k in kelimeler)
{
    string ilk = k.Substring(0, 1).ToUpper(tr);
    duzgun.Append(ilk).Append(k.Substring(1).ToLower(tr)).Append(' ');
    kisa.Append(ilk);
}
Console.WriteLine("Düzgün yazım: " + duzgun.ToString().Trim());
Console.WriteLine("Kısaltma: " + kisa);
```

Örnek çalıştırma: `istanbul TEKNİK üniversitesi` girdisi için `Düzgün yazım: İstanbul Teknik Üniversitesi` ve `Kısaltma: İTÜ` yazılır.

Soru 14.

Dış döngü her yeni harf grubunun başında durur; iç döngü aynı harf sürdükçe sayacı ve indeksi ilerletir. İç döngü koşulunda `i < s.Length` denetiminin önce yazılması şarttır: kısa devre sayesinde `i` sona ulaştığında `s[i]` değerlendirilmez ve `IndexOutOfRangeException` oluşmaz.

```
using System.Text;

Console.Write("Kelime: ");
string kelime = Console.ReadLine() ?? "";
Console.WriteLine("Sıkıştırılmış: " + Sikistir(kelime));
Console.WriteLine(Sikistir("xyz") + " " + Sikistir("kkkkkkkkkkkk"));

static string Sikistir(string s)
{
    StringBuilder sb = new StringBuilder();
    int i = 0;
    while (i < s.Length)
    {
        char c = s[i];
        int adet = 0;
```

```
        while (i < s.Length && s[i] == c)
        {
            adet++;
            i++;
        }
        sb.Append(c).Append(adet);
    }
    return sb.ToString();
}
```

`aaabccdddd` girdisi için `a3b1c2d4`, `xyz` için `x1y1z1`, on iki `k` için `k12` yazılır.

Soru 15.

Uzunlukları farklı olan kelimeler anagram olamaz. Aynı uzunluktaysa iki kelimeyi küçük harfe çevirip karakter dizilerine dönüştürür, sıralar ve eşitliğini deneriz: anagramların sıralı harf dizileri aynıdır.

```
Console.WriteLine(AnagramMi("elma", "alem"));
Console.WriteLine(AnagramMi("Kitap", "pakit"));
Console.WriteLine(AnagramMi("masa", "mass"));
Console.WriteLine(AnagramMi("ab", "abc"));

static bool AnagramMi(string a, string b)
{
    if (a.Length != b.Length)
    {
        return false;
    }

    char[] x = a.ToLowerInvariant().ToCharArray();
    char[] y = b.ToLowerInvariant().ToCharArray();
    Array.Sort(x);
    Array.Sort(y);
    return new string(x) == new string(y);
}
```

Çıktı sırasıyla `True`, `True`, `False`, `False` olur.

Soru 16.

Parolayı bir kez dolaşıp her karakter türü için bir `bool` bayrağı kaldırırız. `char.IsUpper`, `IsLower`, `IsDigit` ve `IsPunctuation` Unicode'u tanıdığı için Türkçe harfler de doğru sınıflanır. Sonra eksik koşulları tek tek yazdırırız.

```
Console.Write("Parola: ");
string parola = Console.ReadLine() ?? "";

bool buyuk = false, kucuk = false, rakam = false, noktalama = false;
foreach (char c in parola)
{
    if (char.IsUpper(c))
    {
        buyuk = true;
    }
    else if (char.IsLower(c))
    {
        kucuk = true;
    }
    else if (char.IsDigit(c))
    {
        rakam = true;
    }
    else if (char.IsPunctuation(c))
```

```
    {
        noktalama = true;
    }
}

bool gecerli = true;
if (parola.Length < 8)
{
    Console.WriteLine("En az 8 karakter olmalı.");
    gecerli = false;
}
if (!buyuk)
{
    Console.WriteLine("Büyük harf eksik.");
    gecerli = false;
}
if (!kucuk)
{
    Console.WriteLine("Küçük harf eksik.");
    gecerli = false;
}
if (!rakam)
{
    Console.WriteLine("Rakam eksik.");
    gecerli = false;
}
if (!noktalama)
{
    Console.WriteLine("Noktalama işareti eksik.");
    gecerli = false;
}
if (gecerli)
{
    Console.WriteLine("Parola geçerli");
}
```

`abc12` girdisi için "En az 8 karakter olmalı.", "Büyük harf eksik." ve "Noktalama işareti eksik." satırları yazılır.

Bölüm 49 Sınıflar ve Nesneler

Soru 1.

Cevap: C. Sınıf üyeleri için belirleyici yazılmazsa varsayılan erişim `private`'tır. `internal` varsayılanı, üye için değil sınıfın kendisi (üst düzey tip) için geçerlidir.

Soru 2.

Cevap: B. `{ get; private set; }` değeri dışarıya okutur, atamayı yalnızca sınıfın içine bırakır. A'da dışarıdan da yazılabilir, C'de (`init`) yalnızca yapıcıda ve nesne başlatıcıda atama yapılabilir, yani sınıfın sonraki metotları değiştiremez. D'de özellik dışarıya kapalıdır. E geçerli bir C# yazımı değildir.

Soru 3.

Cevap: D. `Urun` sınıfında parametreli bir yapıcı yazıldığı için derleyicinin otomatik eklediği parametresiz yapıcı yoktur; `new Urun()` çağrısı `CS7036` hatasıyla derlenmez.

Soru 4.

Cevap: B. `const` değeri derleme zamanında bilinmek zorundadır (yalnızca sayı, `bool`, `char`, `string`, `null`) ve kullanan koda gömülür. `readonly` her tipte olabilir ve tanımda ya da yapıcıda atanabilir; örnek alanı olarak nesne başına ayrı değer taşıyabilir.

Soru 5.

Cevap: B. `==` ezilmediği için iki referans karşılaştırılır. `p` ve `q` ayrı `new` ile üretilmiş farklı nesnelerdir; alan değerlerinin eşit olması sonucu değiştirmez.

Soru 6.

`b`, `a` ile aynı nesneyi gösterir; bu yüzden `b.Deger += 5` ile `a.Deger` de 6 olur. `c` ise yeni bir nesnedir: başlangıç değeri 6'dır, ikiyle çarpılınca 12 olur. `a == b` aynı nesne olduğu için `True`, `a == c` ayrı nesneler olduğu için `False` verir.

```
ÇIKTI
6 6 12
True
False
```

Soru 7.

`Ekle` aynı nesnenin `Adet` alanını değiştirir: 3. `Sifirla` ise parametreye yeni bir nesne atar; bu atama yalnızca yerel `x`'i etkiler, `s` değişmez.

```
ÇIKTI
3
```

Soru 8.

Her `new` çağrısında `static` sayaç bir artar. `n1` oluşturulurken `Adet` 1, `Id` 10; `n2` için 2 ve 20; `n3` için 3 ve 30 olur. `Nesne.Adet` son durumda 3'tür.

```
ÇIKTI
10 30 3
```

Soru 9.

`new Test()` çağrısında önce `: this(7)` ile `Test(int)` çağrılır; o da `: this(a, 0)` ile `Test(int, int)` yapıcısına gider. Gövdeler zincirin sonundan başa doğru çalışır: C, B, A. `new Test(1)` yalnızca C ve B'yi yazdırır.

```
ÇIKTI
C B A
C B
```

Soru 10.

`k` `null` iken `k?.Ad` `null` olur, `??` yedek metni verir. `k` oluşturulunca `Ad` henüz atanmadığı için `k?.Ad is null` `True` döner. `Ad`'a `"Deniz"` atanınca uzunluk 5'tir.

```
ÇIKTI
bilinmiyor
True
5
```

Soru 11.

`readonly` alana yalnızca tanımlandığı yerde ya da yapıcıda atama yapılabilir; `Yatir` metodunda `Bakiye += miktar;` `CS0191` hatası verir. Bakiye değişecekse alan `readonly` olmamalı. Dışarıdan korunması için özellik kullanılır:

```
class Hesap
{
    public decimal Bakiye { get; private set; } = 100;
```

```
    public void Yatir(decimal miktar)
    {
        Bakiye += miktar;
    }
}
```

Soru 12.

`static` bir metot belirli bir nesneye ait değildir, bu yüzden `_n` gibi nesne alanına erişemez (`CS0120`). İki çözüm vardır: metodu `static` olmaktan çıkarmak ya da alanı `static` yapmak. Her nesne kendi sayacını tutacaksa birincisi doğrudur:

```
class Sayac
{
    private int _n;

    public void Artir()
    {
        _n++;
    }
}
```

Soru 13.

`new Ogrenci[3]` yalnızca üç `null` referans içeren bir dizi açar; `sinif[1]` henüz bir nesne göstermediği için `.Ad` erişimi `NullReferenceException` fırlatır. Her eleman ayrıca oluşturulmalıdır:

```
Ogrenci[] sinif = new Ogrenci[3];
sinif[1] = new Ogrenci();
Console.WriteLine(sinif[1].Ad);
```

Soru 14.

Özellikler `set` içinde değerin pozitif olup olmadığını denetler; pozitif değilse eski değer korunur. Yapıcı özelliğe atadığı için doğrulama yapıcıda da çalışır.

```
Dikdortgen d = new Dikdortgen(3, 4);
Console.WriteLine($"{d} alan={d.Alan()} çevre={d.Cevre()}");

class Dikdortgen
{
    private double _en;
    private double _boy;

    public double En
    {
        get => _en;
        set { if (value > 0) _en = value; }
    }

    public double Boy
    {
        get => _boy;
        set { if (value > 0) _boy = value; }
    }

    public Dikdortgen(double en, double boy)
    {
        En = en;
        Boy = boy;
    }
```

```
    public double Alan() => _en * _boy;
    public double Cevre() => 2 * (_en + _boy);
    public override string ToString() => $"{_en}x{_boy}";
}
```

Çıktı: `3x4 alan=12 çevre=14`.

Soru 15.

`Fiyat` için `private set` kullanılır; `Sat` yalnızca stok yetiyorsa düşer.

```
Urun[] raf =
{
    new Urun("Kalem", 12.5m, 10),
    new Urun("Defter", 30m, 5),
    new Urun("Silgi", 4m, 20)
};

Console.WriteLine(raf[0].Sat(4));    // True
Console.WriteLine(raf[1].Sat(9));    // False

decimal toplam = 0;
foreach (Urun u in raf)
{
    toplam += u.Fiyat * u.Stok;
}
Console.WriteLine(toplam);           // 75 + 150 + 80 = 305.0

class Urun
{
    public string Ad { get; }
    public decimal Fiyat { get; private set; }
    public int Stok { get; private set; }

    public Urun(string ad, decimal fiyat, int stok)
    {
        Ad = ad;
        Fiyat = fiyat;
        Stok = stok;
    }

    public bool Sat(int adet)
    {
        if (adet <= 0 || adet > Stok)
        {
            return false;
        }
        Stok -= adet;
        return true;
    }
}
```

`Sat(4)` sonrası kalem stoğu 6 olur: 6 x 12.5 = 75, defter 5 x 30 = 150, silgi 20 x 4 = 80; toplam 305.0.

Soru 16.

Dizi boşsa `foreach` hiç dönmez ve `null` döner. Çağıran taraf `?.` ve `??` ile `null` durumunu karşılar.

```
Ogrenci[] liste =
{
    new Ogrenci("Ayşe", 80),
    new Ogrenci("Mehmet", 65),
```

```
    new Ogrenci("Elif", 92)
};
Console.WriteLine(EnIyi(liste)?.Ad ?? "kimse yok");
Console.WriteLine(EnIyi(new Ogrenci[0])?.Ad ?? "kimse yok");

static Ogrenci? EnIyi(Ogrenci[] liste)
{
    Ogrenci? enIyi = null;
    foreach (Ogrenci o in liste)
    {
        if (enIyi == null || o.Ortalama > enIyi.Ortalama)
        {
            enIyi = o;
        }
    }
    return enIyi;
}

class Ogrenci
{
    public string Ad { get; }
    public double Ortalama { get; }

    public Ogrenci(string ad, double ortalama)
    {
        Ad = ad;
        Ortalama = ortalama;
    }
}
```

Çıktı: `Elif` ve ardından `kimse yok`.

Bölüm 50 Kalıtım

Soru 1.

Cevap: D. `sealed` bir sınıftan türetilemez (`string` buna örnektir). A yanlış: sınıflarda tekli kalıtım vardır (`CS1721`). B yanlış: `private` alanlara türetilmiş sınıftan adıyla erişilemez. C yanlış: yapıcılar devralınmaz, `: base(...)` ile çağrılır. E yanlış: ezerken erişim belirleyici değiştirilemez (`CS0507`).

Soru 2.

Cevap: A. (I), taban sınıftaki `virtual Yaz(int)` ile aynı imzaya sahiptir ve `override` ile yazılmıştır; parametre adının `x` yerine `y` olması önemli değildir, bu bir ezmedir. (II) ve (III)'ün parametre listeleri farklıdır, bunlar aşırı yüklemedir ve aşırı yüklemede dönüş tipi serbesttir. Kod hatasız derlenir.

Soru 3.

Cevap: B. `Otobus` yapıcısında `: base(...)` yazılmadığı için derleyici `: base()` ekler. `Arac`'ın tek yapıcısı `string` parametresi istediğinden bu çağrının karşılığı yoktur ve `CS7036` hatası oluşur. Çözüm, `Otobus(int koltuk) : base("34 ABC 01")` gibi bir çağrı yazmak ya da `Arac`'a parametresiz yapıcı eklemektir. A ve C yanlış: program hiç derlenmez. D yanlış: `Plaka`, `Arac`'tan devralınır. E yanlış: yapıcıda özellik atamak serbesttir.

Soru 4.

Cevap: E. (1) kendi nesnemiz üzerinden, (2) `Vadeli` tipli bir değişken üzerinden erişimdir; ikisi de serbesttir. (3)'te erişim `Hesap` tipli `h` üzerinden yapılıyor; türetilmiş sınıf `protected` üyeye yalnızca kendi türünden nesneyle erişebilir (`CS1540`). (4) sınıfın dışından erişimdir (`CS0122`).

Soru 5.

Cevap: C. Nesne gerçekte bir `Kisi`'dir, `Ogrenci` değil. (1) `as` uymayınca `null` döndürür, hata fırlatmaz. (2) `is` `false` verir. (3) cast uymayınca `InvalidCastException` fırlatır. Konsolda ilk satır `True False` olur, ardından yakalanmamış istisna görünür.

Soru 6.

Çağrı ağacını çizelim. `new Taksi()` önce `this(4)` ile `Taksi(int)` yapıcısına gider; o, `base(4)` ile `Otomobil(int)` yapıcısını çağırır. `Otomobil(int)` içinde `: base` yazılmadığı için örtük `Arac()` çalışır. Gövdeler tepeden aşağı biter. `new Otomobil()` ise `base("oto")` ile `Arac(string)` yapıcısına gider; o da `this()` ile önce `Arac()` gövdesini çalıştırır.

```
ÇIKTI
Arac()
Otomobil(4)
Taksi(4)
Taksi()
---
Arac()
Arac(oto)
Otomobil()
```

Soru 7.

`Bilgi()` metodu `Tasit`'ın kodudur. İçindeki `Tur()` sanal olduğu için nesnenin gerçek sınıfına göre seçilir; `Tekerlek()` ise sanal olmadığı için her zaman `Tasit`'ın sürümüne gider. Son satırda `b.Tekerlek()` `new` ile gizlenen `Bisiklet` sürümüdür (`2`), `tb.Tekerlek()` ise değişkenin tipi `Tasit` olduğu için `4` verir.

```
ÇIKTI
Taşıt/4
Bisiklet/4
Bisiklet
4
6
```

Soru 8.

`Console.WriteLine(h)` `ToString`'i çağırır; nesne `Kedi` olduğu için `Kedi.ToString` çalışır ve `base.ToString()` ile `Hayvan` metnini ekler. `Pamuk` adı 5 harf olduğundan `k.Ad.Length > 4` doğrudur.

```
ÇIKTI
Kedi:Hayvan[Pamuk]
Kedi
True
False
True
Pamuk mırlıyor
Hayvan[Boncuk]
False
```

Soru 9.

- `class Turev : Sabit` satırı: `sealed` sınıftan türetme (`CS0509`).
- `B.Yaz()`: taban metot `virtual` olmadığı için ezilemez (`CS0506`).
- `D.Calis()`: ezerken erişim `public`'ten `protected`'a değiştirilemez (`CS0507`).
- `class E : A, C`: sınıflarda çoklu kalıtım yoktur (`CS1721`).

Soru 10.

`Daire.Ad()` metodu `override` olmadan yazıldığı için ezme değil gizlemedir (derleyici `CS0114` uyarısı verir). `s` değişkeninin tipi `Sekil` olduğundan ve gizlenmiş metotta değişkenin tipi kazandığından program `Şekil` yazar, beklenen `Daire` yazmaz. İki düzeltme: (a) `Daire.Ad()` metodunu `public override string Ad()` yapmak, böylece nesnenin gerçek sınıfı kazanır; (b) gizleme gerçekten isteniyorsa metodu `public new string Ad()` yazıp uyarıyı bilinçli hâle getirmek ve metodu `Daire` tipli bir değişkenden çağırmak.

Soru 11.

Cevap: C. Kütüphane bir kitap değildir; kütüphanenin kitapları vardır (has-a), bu bir kompozisyondur. Diğer şıklarda "X bir Y'dir" cümlesi doğrudur.

Soru 12.

```
class Urun
{
    private string ad;
    protected decimal fiyat;

    public Urun(string ad, decimal fiyat)
    {
        this.ad = ad;
        this.fiyat = fiyat;
    }

    public string Ad => ad;

    public virtual decimal KdvliFiyat() => fiyat * 1.20m;

    public override string ToString() => $"{ad} - {KdvliFiyat():F2} TL";
}

class Kitap : Urun
{
    public string Yazar { get; set; } = "";
    public Kitap(string ad, decimal fiyat) : base(ad, fiyat) { }
    public override decimal KdvliFiyat() => fiyat * 1.10m;
    public override string ToString() => base.ToString() + $" (yazar: {Yazar})";
}

class Elektronik : Urun
{
    public int GarantiYil { get; set; }
    public Elektronik(string ad, decimal fiyat) : base(ad, fiyat) { }
    public override string ToString()
        => base.ToString() + $" (garanti: {GarantiYil} yıl)";
}
```

Ana kod: `Urun[] liste = { new Urun("Kalem", 20m), new Kitap("Sefiller", 150m) { Yazar = "Victor Hugo" }, new Elektronik("Kulaklik", 1200m) { GarantiYil = 2 } };` ve bir `foreach` içinde `Console.WriteLine(u)` ile `toplam += u.KdvliFiyat()`. `Urun.ToString` içindeki `KdvliFiyat()` çağrısı sanal olduğundan `Kitap` için yüzde 10'luk sürüm kullanılır. Çıktı sorudaki gibidir ve toplam `1629.00` olur.

Soru 13.

```
Hesap[] hesaplar = { new Hesap(), new VadeliHesap() };
foreach (Hesap h in hesaplar)
{
    h.Yatir(500m);
    Console.WriteLine($"Cek 100: {h.Cek(100m)}");
```

```
}

class Hesap
{
    protected decimal bakiye;
    public decimal Bakiye => bakiye;

    public void Yatir(decimal miktar) => bakiye += miktar;

    public virtual bool Cek(decimal miktar)
    {
        if (miktar > bakiye)
        {
            return false;
        }
        bakiye -= miktar;
        return true;
    }
}

class VadeliHesap : Hesap
{
    private bool vadesiDoldu = false;

    public override bool Cek(decimal miktar)
    {
        if (!vadesiDoldu)
        {
            return false;
        }
        return base.Cek(miktar);
    }
}
```

Çıktı: `Cek 100: True` ve `Cek 100: False`. Vadeli hesap, vadesi dolmadığı için taban kurala hiç bakmadan reddeder; vadesi dolsaydı `base.Cek` ile taban sınıfın bakiye denetimine devrederdi.

Soru 14.

```
Bildirim[] liste =
{
    new Bildirim { Mesaj = "Genel duyuru" },
    new EPosta { Mesaj = "Toplantı yarın", Konu = "Toplantı" },
    new Sms { Mesaj = "Kod: 4821", Numara = "555 0100" }
};

int epostaSayisi = 0;
foreach (Bildirim b in liste)
{
    Console.WriteLine(b.Ozet());
    if (b is EPosta)
    {
        epostaSayisi++;
    }
}
Console.WriteLine($"E-posta sayısı: {epostaSayisi}");

foreach (Bildirim b in liste)
{
    Sms? s = b as Sms;
    if (s != null)
    {
```

```
        Console.WriteLine($"İlk SMS numarası: {s.Numara}");
        break;
    }
}

class Bildirim
{
    public string Mesaj { get; set; } = "";
    public virtual string Ozet() => $"Bildirim: {Mesaj}";
}

class EPosta : Bildirim
{
    public string Konu { get; set; } = "";
    public override string Ozet() => $"E-posta [{Konu}]: {Mesaj}";
}

class Sms : Bildirim
{
    public string Numara { get; set; } = "";
    public override string Ozet() => $"SMS ({Numara}): {Mesaj}";
}
```

`Ozet()` sanal olduğu için döngüde her nesne kendi özetini yazar. Türe özgü `Numara` özelliğine ise yalnızca `as` ile (ya da `is` deseniyle) aşağı dönüşüm yaptıktan sonra ulaşılır. Çıktıda sırasıyla üç özet, `E-posta sayısı: 1` ve `İlk SMS numarası: 555 0100` görünür.

Bölüm 51 Polimorfizm, Soyut Sınıflar ve Arayüzler

Soru 1.

Cevap: C. Soyut sınıftan `new` ile nesne üretilemez; `new Sekil()` derleme hatasıdır (`CS0144`). A ve D yukarı dönüşümdür, geçerlidir. E de geçerlidir: soyut sınıf tipinde (boş olabilen) bir referans tanımlamak serbesttir. B bir tuzaktır: `new Sekil[3]` soyut sınıftan nesne değil, `Sekil` referansları taşıyacak bir dizi üretir; elemanlar başlangıçta `null`'dır ve satır derlenir.

Soru 2.

Cevap: D. Önce tek sınıf, sonra arayüzler gelir; `A, IX, IY` tam bu kalıptır. A ve E iki sınıftan türemeye çalışır (`CS1721`). B ve C arayüzleri sınıftan önce yazar (`CS1722`).

Soru 3.

Cevap: D. `Cocuk`, `IKonus` arayüzünü kendi başlığında yeniden listelediği için arayüz eşlemesi yenilenir ve `k.Konus()` artık `Cocuk`'un metoduna gider: `Cocuk`. `a.Konus()` çağrısında referans tipi `Ana`'dır ve metot sanal olmadığı için `Ana` sürümü çalışır: `Ana`. `c.Konus()` referans tipi `Cocuk` olduğundan `Cocuk` yazar. Sonuç `CocukAnaCocuk`. Başlıkta arayüz yeniden listelenmeseydi `k.Konus()` de `Ana` yazar ve sonuç A şıkkındaki gibi olurdu.

Soru 4.

Cevap: E. Arayüzde yapıcı tanımlanamaz; arayüzden nesne üretilmez ve arayüzde nesne alanı yoktur. A, B ve C doğrudur. D de doğrudur: arayüz üyeleri örtük `public` olduğu için sınıfta `public` yazılmazsa program derlenmez (`CS0737`).

Soru 5.

Cevap: B. `CompareTo` içinde karşılaştırma ters yazılmış: `diger.Agirlik.CompareTo(Agirlik)`. Ağırlığı büyük olan nesne "önce gelir" sayılır ve `Array.Sort` büyükten küçüğe sıralar: 95, 70, 40. `d[0].Agirlik` 95, `d[2].Agirlik` 40; `int + string` birleştirmesi `95 40` verir. A, `CompareTo`'nun doğal (artan) hâlini sananların cevabıdır. C ve D sıralamanın hiç olmadığını varsayar. E yanlıştır: `IComparable<T>` için `Equals` ezmek zorunlu değildir.

Soru 6.

- `new Tasit()`: kendi metotları çalışır, `Taşıt 4`.
- `new Bisiklet()`: `Yaz()` `Tasit`'ten gelir, ama içindeki `Ad()` ve `Teker()` çağrıları `Bisiklet`'in sürümlerine gider: `Bisiklet 2`.
- `new Tir()`: kendi `Yaz()` metodu önce `[Yük]` yazar, sonra `base.Yaz()` ile `Tasit`'in `Yaz()` metodu çalışır. Orada `Ad()` ezilmediği için `Taşıt`, `Teker()` ise `Tir`'inki olduğu için `base.Teker() + 2 = 6` olur: `[Yük] Taşıt 6`.
- `x`, bir `Tir`'dir, `Bisiklet` değildir: `False` (C#'ta `bool` değerleri büyük harfle yazılır).
- `"Taşıt".Length` 5, `x.Teker()` 6; iki `int` toplanır: `11`.

```
ÇIKTI
Taşıt 4
Bisiklet 2
[Yük] Taşıt 6
False
11
```

Soru 7.

Her satırda üç karar vardır: hangi `Sor` seçilecek (aşırı yükleme, derleme zamanı, argümanın referans tipi), hangi `Ses` çalışacak (`override`, çalışma zamanı, nesne tipi) ve hangi `Tur` (`new` ile gizlenmiş, derleme zamanı, referans tipi). `Sor` içinde `h` parametresi her zaman `Hayvan` tipindedir, bu yüzden `H:` ile başlayan satırlarda `Tur` her zaman `Hayvan` döner.

Çağrı	Seçilen `Sor`	`Ses` (nesne tipi)	`Tur`	Sonuç
`Sor(a)`	`Hayvan`	`Kedi`	`Hayvan`	`H:Miyav/Hayvan`
`Sor(b)`	`Kedi`	`Kedi`	`Kedi`	`K:Miyav/Kedi`
`Sor(c)`	`Hayvan`	`Hayvan`	`Hayvan`	`H:.../Hayvan`
`Sor((Kedi)a)`	`Kedi`	`Kedi`	`Kedi`	`K:Miyav/Kedi`
`Sor((Hayvan)b)`	`Hayvan`	`Kedi`	`Hayvan`	`H:Miyav/Hayvan`

Son satır öğreticidir: `(Hayvan)b` cast'i nesneyi değiştirmez, yalnızca referans tipini değiştirir.

```
ÇIKTI
H:Miyav/Hayvan
K:Miyav/Kedi
H:.../Hayvan
K:Miyav/Kedi
H:Miyav/Hayvan
```

Soru 8.

- `Ogrenci("Ece")`: `Selam()` ezilmiş, `Selam, ben Ece`.
- `Ogretmen("Kaya")`: `Selam()` ezilmemiş, `Kisi`'deki sanal sürüm çalışır. İçindeki `Ad()` çağrısı dinamik bağlanır ve `Ogretmen`'in `Ad()` metodu seçilir: `Merhaba Dr. Kaya`.
- `Ogrenci("Mert")`: `Selam, ben Mert`.
- Her nesne oluşturulurken `Kisi` yapıcısı çalıştı ve `static` sayaç 3 kez arttı. `static` alan bütün nesnelerce ortak olduğu için sonuç `3`.

```
ÇIKTI
Selam, ben Ece
Merhaba Dr. Kaya
Selam, ben Mert
3
```

Soru 9.

C#'ta `Kedi` nesnesi oluşturulurken önce türetilmiş sınıfın alan başlatıcıları çalışır (`ad = "isimsiz"`), sonra `Hayvan` yapıcısı. Bu yapıcıdan çağrılan `Tanit()` sanaldır ve `Kedi`'nin sürümüne ulaşır; ama `Kedi` yapıcısının gövdesi (`this.ad = ad`) henüz çalışmadığı için `ad` hâlâ `isimsiz`'dir. Sonra `Kedi` yapıcısının gövdesi çalışır.

```
ÇIKTI
Hayvan: kedi isimsiz
Kedi: kedi Pamuk
---
Hayvan: kedi isimsiz
Kedi: kedi Tekir
```

C++'ta taban yapıcısı çalışırken nesne henüz yalnızca `Hayvan` sayıldığı için ilk satır `Hayvan: hayvan` olurdu (Bölüm 31).

Soru 10.

- `k.Ac()`: referans tipi `Kasa`; açık uygulanan üye `Kasa`'nın arayüzünde görünmez, görünen `public` metottur: `genel`.
- `s.Ac()`: `ISifreli` üzerinden çağrı, açık uygulama seçilir: `şifre`.
- `a.Ac()`: `IAnahtarli` için açık uygulama yok; `public Ac()` o arayüzü de karşılar: `genel`.
- `((ISifreli)k).Ac().Length`: `"şifre"` beş karakterdir: `5`.

```
ÇIKTI
genel
şifre
genel
5
```

Soru 11.

Cevap: C. Varsayılan arayüz metodu sınıfın üyesi olmaz; yalnızca arayüz tipli bir ifadeyle çağrılabilir. `l` değişkeninin tipi `Lazer` olduğu için `l.Baslik()` derlenmez (`CS1061`). B'de cast ile ifade `IYazici` tipine yükseltildiği için çağrı geçerlidir. A, D ve E sıradan atamalar ve özellik erişimidir.

Soru 12.

1. `Koltuk` sınıfındaki `void Tasi() => ...` metodu `public` değildir. Sınıf üyelerinin varsayılan erişimi `private` olduğu için `ITasinabilir.Tasi()` karşılanmaz (`CS0737`). Çözüm: `public void Tasi() => ...`.
2. `Masa` sınıfı somuttur, ama `Esya`'nın soyut `Ad()` metodunu ezmemiştir (`CS0534`). Çözüm: `public override string Ad() => "Masa";` yazmak ya da `Masa`'yı `abstract` yapmak. `Masa` bir arayüz uygulamadığı için `Agirlik` özelliği hata değildir.

Soru 13.

`Calisan` sınıfı `Maas()` ve `Tur()` metotlarını soyut bırakır; ortak `Satir()` metodu bu ikisini çağırır ve dinamik bağlama doğru sürümü seçer. Fazla mesai için 160 saati aşan kısım ayrıca hesaplanır: Ece için 160 × 250 + 10 × 250 × 1,5 = 40000 + 3750 = 43750.

```
Calisan[] ekip =
{
    new TamZamanli("Elif", 52000),
    new SaatUcretli("Kerem", 140, 250),
    new SaatUcretli("Ece", 170, 250),
    new TamZamanli("Selin", 41000)
};
double toplam = 0;
foreach (Calisan c in ekip)
{
    Console.WriteLine(c.Satir());
    toplam += c.Maas();
}
Console.WriteLine($"Toplam bordro: {toplam:F2} TL");

abstract class Calisan
{
    public string Ad { get; }
    protected Calisan(string ad) { Ad = ad; }
    public abstract double Maas();
    public abstract string Tur();
    public string Satir() => $"{Ad,-6} {Tur(),-14} {Maas(),9:F2}";
}

class TamZamanli : Calisan
{
    private readonly double aylik;
    public TamZamanli(string ad, double aylik) : base(ad) { this.aylik = aylik; }
    public override double Maas() => aylik;
    public override string Tur() => "Tam zamanlı";
}

class SaatUcretli : Calisan
{
    private readonly int saat;
    private readonly double ucret;

    public SaatUcretli(string ad, int saat, double ucret) : base(ad)
    {
        this.saat = saat;
        this.ucret = ucret;
    }

    public override double Maas()
    {
        if (saat <= 160)
        {
            return saat * ucret;
        }
        return 160 * ucret + (saat - 160) * ucret * 1.5;
    }

    public override string Tur() => "Saat ücretli";
}
```

```
ÇIKTI
Elif   Tam zamanlı     52000.00
Kerem  Saat ücretli    35000.00
Ece    Saat ücretli    43750.00
Selin  Tam zamanlı     41000.00
Toplam bordro: 171750.00 TL
```

Soru 14.

`CompareTo` içinde karşılaştırmayı ters çevirerek (`diger.Skor.CompareTo(Skor)`) büyük skoru öne alırız. Skorlar eşitse `string.CompareOrdinal` ile ada göre karar veririz; bu yüzden aynı skorlu `Deniz`, `Zeynep`'ten önce gelir.

```
Oyuncu[] oyuncular =
{
    new Oyuncu("Can", 70), new Oyuncu("Zeynep", 95),
    new Oyuncu("Mert", 40), new Oyuncu("Deniz", 95)
};
Array.Sort(oyuncular);
foreach (Oyuncu o in oyuncular)
{
    Console.WriteLine(o);
}
Console.WriteLine($"Lider: {oyuncular[0].Ad}, son: {oyuncular[^1].Ad}");

class Oyuncu : IComparable<Oyuncu>
{
    public string Ad { get; }
    public int Skor { get; }

    public Oyuncu(string ad, int skor)
    {
        Ad = ad;
        Skor = skor;
    }

    public int CompareTo(Oyuncu? diger)
    {
        if (diger is null)
        {
            return -1;
        }
        int fark = diger.Skor.CompareTo(Skor);        // büyük skor önce
        if (fark != 0)
        {
            return fark;
        }
        return string.CompareOrdinal(Ad, diger.Ad);    // eşitse ada göre
    }

    public override string ToString() => $"{Ad}: {Skor}";
}
```

```
ÇIKTI
Deniz: 95
Zeynep: 95
Can: 70
Mert: 40
Lider: Deniz, son: Mert
```

Soru 15.

`IEnumerable<int>` uygulamak için hem genel hem genel olmayan `GetEnumerator` gerekir; `yield return` ile sayıcıyı derleyiciye ürettiririz. `Sum()` ve `Max()` LINQ metotlarıdır ve `IEnumerable<int>` üzerinde çalışır (Bölüm 56).

```
Kareler k = new Kareler(5);
foreach (int x in k)
{
    Console.Write(x + " ");
}
```

```
Console.WriteLine();
Console.WriteLine(k.Sum());
Console.WriteLine(k.Max());

class Kareler : IEnumerable<int>
{
    private readonly int adet;
    public Kareler(int adet) { this.adet = adet; }

    public IEnumerator<int> GetEnumerator()
    {
        for (int i = 1; i <= adet; i++)
        {
            yield return i * i;
        }
    }

    System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator()
        => GetEnumerator();
}
```

```
ÇIKTI
1 4 9 16 25
55
25
```

Soru 16.

`Sepet` yalnızca `IIndirim` bilir; kural ekleme işi yeni bir sınıf yazmaktan ibarettir, `Sepet` değişmez (bağımlılığın tersine çevrilmesi ve açık/kapalı ilkesi).

```
IIndirim[] kurallar = { new YuzdeIndirim(10), new SabitIndirim(25), new IndirimYok() };
Sepet sepet = new Sepet(200);
foreach (IIndirim k in kurallar)
{
    Console.WriteLine($"{k.Ad}: {sepet.Toplam(k):F2} TL");
}

interface IIndirim
{
    string Ad { get; }
    double Uygula(double tutar);
}

class YuzdeIndirim : IIndirim
{
    private readonly double oran;
    public YuzdeIndirim(double oran) { this.oran = oran; }
    public string Ad => $"%{oran} indirim";
    public double Uygula(double tutar) => tutar * (1 - oran / 100);
}

class SabitIndirim : IIndirim
{
    private readonly double miktar;
    public SabitIndirim(double miktar) { this.miktar = miktar; }
    public string Ad => $"{miktar} TL indirim";
    public double Uygula(double tutar) => Math.Max(0, tutar - miktar);
}

class IndirimYok : IIndirim
{
    public string Ad => "İndirimsiz";
```

```
    public double Uygula(double tutar) => tutar;
}

class Sepet
{
    private readonly double tutar;
    public Sepet(double tutar) { this.tutar = tutar; }
    public double Toplam(IIndirim indirim) => indirim.Uygula(tutar);
}
```

```
ÇIKTI
%10 indirim: 180.00 TL
25 TL indirim: 175.00 TL
İndirimsiz: 200.00 TL
```

Bölüm 52 Değer Tipleri: struct, enum ve record

Soru 1.

Cevap: A. `Nokta` bir `struct` olduğu için `b = a` kopyalar; `b.X = 2` değeri `a`'yı etkilemez (`a.X` = 1). `Artir` metoduna `b` kopyalanarak geçer; metot kendi kopyasını 12 yapıp döndürür, `b` ise 2 olarak kalır. Sonuç `1 2 12` olur.

Soru 2.

Cevap: C. Bir `struct` başka bir `struct`'tan ya da `class`'tan türetilemez; yalnızca arayüz uygulayabilir. Diğer şıklar doğrudur: `struct` `null` olamaz, `new Nokta()` sıfırlanmış değer üretir ve `struct` dizisinin elemanları dizi oluşturulurken sıfırlanmış olarak hazırdır.

Soru 3.

Cevap: C. `Enum.TryParse`, sayı içeren metinleri tanımlı bir sabite denk gelmese de kabul eder; `true` döner ve `s`, değeri 7 olan (adı olmayan) bir `Seviye` olur. Böyle bir değeri `Enum.IsDefined` ile ayrıca denetlemek gerekir.

Soru 4.

Cevap: B. `record` içerik eşitliği sağlar: alanları aynı olan iki ayrı nesne için `a == b` doğrudur, ama bunlar iki ayrı nesne olduğundan `ReferenceEquals(a, b)` yanlıştır. `a with { }` bir kopya üretir, `Equals` elle yazılmak zorunda değildir ve `record` varsayılan olarak bir sınıftır.

Soru 5.

Üç bayrak eklenir (değer 7), sonra `Okuma` kaldırılır; geriye `Yazma` (2) ve `Calistirma` (4) kalır.

```
ÇIKTI
Yazma, Calistirma
6
False
```

Soru 6.

`b`, `Kredi` değeri değiştirilmiş bir kopyadır; `c` ise değişikliksiz bir kopyadır. İçerikleri `a` ile aynı olduğundan `a == c` doğrudur, ama `c` yeni bir nesne olduğundan `ReferenceEquals` yanlıştır.

```
ÇIKTI
Ders { Ad = Fizik, Kredi = 3 }
Ders { Ad = Fizik, Kredi = 4 }
True
False
```

Soru 7.

`o` ve `n` ayrı değerlerdir: kutudan çıkarma kopya oluşturur, `n++` kutuyu etkilemez. `p` ve `q` aynı kutuyu gösterdiği için `p == q` (`object` için referans karşılaştırması) doğrudur. `p.Equals(5)` kutudaki değeri karşılaştırır ve doğrudur. `o` ve `p` ise ayrı ayrı kutulandığı için `ReferenceEquals(o, p)` yanlıştır.

```
ÇIKTI
5 6
True
True
False
```

Soru 8.

Hatalı satırlar 3 ve 5'tir. `liste[0].R = 100;` satırında indeksleyici `Renk` değerinin bir kopyasını döndürür; kopyaya yazmak anlamsız olduğundan derleyici `CS1612` hatası verir (çözüm: `var r0 = liste[0]; r0.R = 100; liste[0] = r0;` ya da `readonly struct` kullanmak). `r == new Renk(1, 2, 3)` satırında `struct` için `==` operatörü tanımlı değildir (`CS0019`); `Equals` metodunu çağırabilir ya da `==` operatörünü kendimiz tanımlayabiliriz (ya da `readonly record struct` kullanırız).

Soru 9.

Cevap: C. Dizi elemanı `int` olarak doğrudan dizinin belleğindedir; atamada kutulama olmaz. A'da `double` değeri `object`'e, B'de `int` değeri `IComparable<int>` arayüzüne, D'de her sayı `object` elemanına, E'de `ArrayList`'in `object` parametresine geçerken kutulanır.

Soru 10.

C# enum ile temel tipi arasındaki açık dönüşümde değerin tanımlı bir sabite karşılık gelip gelmediğini denetlemez; bu, C'den devralınan davranıştır. Önlem olarak dışarıdan gelen değerleri `Enum.IsDefined` ile denetleriz, `Enum.TryParse` sonucunu da ayrıca `IsDefined`'dan geçiririz, `switch` ifadelerinde `_` dalında tanımsız değerleri ele alırız ve 0 değerine anlamlı bir ad (`Yok`, `Bilinmiyor`) veririz.

Soru 11.

- (a) `readonly record struct` (ya da `readonly struct`): üç sayıdan oluşan küçük, değişmez bir değerdir; kopyalanması ucuzdur.
- (b) `class`: bağlantı bir kaynağı yönetir, kimliği vardır ve paylaşılmalıdır; kopyalanması anlamsızdır.
- (c) `record` (sınıf): üç alanlı, veri taşıyan bir tiptir; boyutu küçük bir `struct` için sınırda olduğundan ve metin referansları içerdiğinden `record` sınıfı daha uygundur. `readonly record struct` de kabul edilebilir.
- (d) `readonly record struct`: iki tarihten oluşan küçük, değişmez ve eşitliği içerikle belirlenen bir değerdir.

Soru 12.

Olası farklar: - C#'ta sabite her zaman tipin adıyla erişilir (`Durum.Acik`); C'de `ACIK` doğrudan kapsama sızar. - C#'ta enum ile `int` arasında dönüşüm açık olmak zorundadır (`(int)d`, `(Durum)1`); C'de örtüktür. - C#'ta `Console.WriteLine(d)` sabitin adını yazar; C'de ad için dizi ya da `switch` gerekir. - C#'ta `Enum.Parse` / `TryParse` ile metinden enum üretilir, `Enum.GetValues` ile sabitler dolaşılır. - C#'ta temel tip `: byte` gibi bir ekle seçilebilir.

Soru 13.

```
Dikdortgen d = new Dikdortgen(3, 4);
Console.WriteLine($"Alan: {d.Alan()}");
Console.WriteLine($"Çevre: {d.Cevre()}");
Console.WriteLine($"Yeni alan: {d.GenislikIkiKat().Alan()}");

readonly struct Dikdortgen
{
    public double En { get; }
```

```
    public double Boy { get; }

    public Dikdortgen(double en, double boy)
    {
        En = en;
        Boy = boy;
    }

    public double Alan() => En * Boy;
    public double Cevre() => 2 * (En + Boy);
    public Dikdortgen GenislikIkiKat() => new Dikdortgen(En * 2, Boy);
}
```

Çıktı sırasıyla `Alan: 12`, `Çevre: 14` ve `Yeni alan: 24` olur. Yapı değişmez olduğu için `GenislikIkiKat` mevcut değeri değiştirmez, yeni bir değer döndürür.

Soru 14.

```
Console.Write("Mevsim: ");
string? metin = Console.ReadLine();

if (Enum.TryParse<Mevsim>(metin, true, out Mevsim m) && Enum.IsDefined(m))
{
    string ay = m switch
    {
        Mevsim.Ilkbahar => "Mart",
        Mevsim.Yaz => "Haziran",
        Mevsim.Sonbahar => "Eylül",
        _ => "Aralık"
    };
    Console.WriteLine($"İlk ay: {ay}");
}
else
{
    Console.WriteLine("Geçersiz mevsim");
}

enum Mevsim { Ilkbahar, Yaz, Sonbahar, Kis }
```

`TryParse`'ın ikinci parametresi (`true`) büyük/küçük harf farkını kapatır. `IsDefined` denetimi, kullanıcı `"7"` gibi bir sayı girdiğinde de program doğru çalışsın diye eklenmiştir.

Soru 15.

```
Gunler toplanti = Gunler.Sal | Gunler.Per;
toplanti |= Gunler.Cum;              // Cum ekle
toplanti &= ~Gunler.Sal;             // Sal çıkar
Console.WriteLine(toplanti);
Console.WriteLine((toplanti & Gunler.HaftaSonu) != 0);

[Flags]
enum Gunler
{
    Yok = 0,
    Pzt = 1, Sal = 2, Car = 4, Per = 8, Cum = 16, Cmt = 32, Paz = 64,
    HaftaIci = Pzt | Sal | Car | Per | Cum,
    HaftaSonu = Cmt | Paz
}
```

Çıktı `Per, Cum` ve `False` olur. Bayrak değerleri ikinin kuvvetidir; birleşik sabitler tanımlı sabitlerin `|` ile birleşimidir.

Soru 16.

```
var u1 = new Urun("Defter", 100m);
var u2 = u1 with { Fiyat = u1.Fiyat * 1.10m };
Console.WriteLine(u1);
Console.WriteLine(u2);
Console.WriteLine(u1 == u2);

record Urun(string Ad, decimal Fiyat);
```

Çıktı `Urun { Ad = Defter, Fiyat = 100 }`, `Urun { Ad = Defter, Fiyat = 110.00 }` ve `False` olur. `decimal` çarpımda ondalık basamak sayısını korur (`1.10m` iki basamaklıdır), bu yüzden `110.00` yazılır. Fiyatlar farklı olduğundan iki kayıt eşit değildir.

Bölüm 53 İstisna Yönetimi

Soru 1.

Cevap: C. C#'ta yalnızca `Exception` sınıfından türeyen nesneler fırlatılabilir. C++'ta `throw 42;` geçerliydi; C#'ta metin, sayı, `bool` ya da `char` fırlatmak derleme hatasıdır.

Soru 2.

Cevap: C. `FormatException` ve `ArgumentException` kardeş sınıflardır (ikisi de doğrudan `SystemException`'dan türer), bu yüzden sıra önemsizdir. A, B, D ve E'de üst sınıf önce yakalandığı için ikinci `catch` ulaşılamaz olur ve `CS0160` hatası verilir.

Soru 3.

Çıktı:

```
ÇIKTI
A B E F
A D E F
A C E F
```

`Test(0)`'da istisna yoktur; `B` yazılır, `catch` atlanır, `finally` çalışır. `Test(1)`'de `InvalidOperationException` yalnızca `catch (Exception)` ile uyuşur. `Test(2)`'de `FormatException` ilk `catch` ile yakalanır. Üç durumda da `finally` (E) çalışır ve ardından `F` yazılır.

Soru 4.

Cevap: D. `finally` her durumda çalışır (A ve B yanlış). İçinden `return` ile çıkmak derleme hatasıdır (`CS0157`), C yanlış. Bir `try` yapısında en fazla bir `finally` olur. `try`-`finally` ikilisi `catch` olmadan yazılabilir.

Soru 5.

Cevap: C. `throw;` yalnızca `catch` içinde yazılabilir ve istisnayı çağrı zinciriyle birlikte yeniden fırlatır. `throw ex;` geçerlidir ama zinciri o satırdan başlatır; A'daki ifade tersine çevrilmiştir.

Soru 6.

Çıktı:

```
ÇIKTI
n=3
1
```

`return n;` çalıştığında dönecek değer (1) saklanır. Ardından `finally` çalışır ve `n=3` yazar. Yerel değişkenin sonradan değişmesi dönen değeri etkilemez. Metot 1 döndürür ve `Console.WriteLine(F())` bunu yazar.

Soru 7.

Cevap: C. Gerçek sayılarda sıfıra bölme istisna doğurmaz. `payda` önce `double`'a dönüşür ve `1.0 / 0.0` işlemi `Infinity` verir. `DivideByZeroException` yalnızca tam sayı bölmesinde oluşur.

Soru 8.

Çıktı:

```
ÇIKTI
X açıldı
Y açıldı
içeride
Y kapandı
X kapandı
dışarıda
```

İç blokta `using var b` bu bloğun sonunda, dış `using (var a ...)` kendi bloğunun sonunda bırakılır. Kaynaklar açılışın tersi sırayla kapanır: önce `Y`, sonra `X`.

Soru 9.

Sorun, `catch (Exception)` her istisnayı yakaladığı için sonraki `catch (IndexOutOfRangeException)` bloğuna hiçbir zaman ulaşılamamasıdır; derleyici `CS0160` hatası verir. Düzeltme için özel olan önce, genel olan sona yazılır:

```
catch (IndexOutOfRangeException)
{
    Console.WriteLine("İndeks hatası");
}
catch (Exception)
{
    Console.WriteLine("Genel hata");
}
```

Soru 10.

Hata `CS0161`: `FormatException` yakalandığında `catch` bloğu bir değer döndürmeden biter, yani her yol değer döndürmez. İki yol vardır: `catch` bloğuna bir `return` eklemek ya da istisnayı yeniden fırlatmak.

```
catch (FormatException)
{
    Console.WriteLine("hata");
    return -1;          // 1. yol: varsayılan değer döndür
    // throw;           // 2. yol: istisnayı yukarıya ilet
}
```

Daha iyi bir tasarım, metni `int.TryParse` ile çevirmektir.

Soru 11.

Cevap: C. Yanlış yazılmış yaş sıradan, beklenen bir durumdur. `TryParse` istisna fırlatmadan başarıyı `bool` ile bildirir; kod hem daha kısa hem daha hızlıdır. A hatayı yutar, B kullanıcıyı cezalandırır, D anlamsızdır, E güvensizdir.

Soru 12.

Çıktı:

```
ÇIKTI
ok 1
B büyük: 5
A büyük: 10
```

`k = 1`'de istisna yoktur. `k = 5`'te istisna fırlar, ama `when (k > 7)` yanlış olduğundan ilk `catch` atlanır ve ikincisi çalışır. `k = 10`'da süzgeç doğrudur, ilk `catch` çalışır.

Soru 13.

```
Console.WriteLine(GuvenliBol(10, 2));
Console.WriteLine(GuvenliBol(10, 0)?.ToString() ?? "tanımsız");

static int? GuvenliBol(int a, int b)
{
    try
    {
        return a / b;
    }
    catch (DivideByZeroException)
    {
        return null;
    }
}
```

Çıktı:

```
ÇIKTI
5
tanımsız
```

Tam sayı bölmesinde sıfır payda `DivideByZeroException` fırlatır; `catch` bunu `null`'a çevirir. `int?` dönüş tipi "değer yok" durumunu ifade eder. Bölenin sıfır olması beklenen bir durumsa `if (b == 0) return null;` denetimi daha basit ve hızlıdır.

Soru 14.

```
Urun u = new Urun("Defter", 5);
foreach (int adet in new int[] { 2, 4, 3 })
{
    try
    {
        u.Cikar(adet);
        Console.WriteLine($"{adet} adet çıkarıldı, kalan: {u.Stok}");
    }
    catch (StokYetersizException ex)
    {
        Console.WriteLine($"{ex.Message} (eksik: {ex.Eksik})");
    }
}

class StokYetersizException : Exception
{
    public int Eksik { get; }

    public StokYetersizException(string urun, int eksik)
        : base($"{urun} için stok yetersiz")
    {
        Eksik = eksik;
```

```
    }
}

class Urun
{
    public string Ad { get; }
    public int Stok { get; private set; }

    public Urun(string ad, int stok)
    {
        Ad = ad;
        Stok = stok;
    }

    public void Cikar(int adet)
    {
        if (adet > Stok)
        {
            throw new StokYetersizException(Ad, adet - Stok);
        }
        Stok -= adet;
    }
}
```

Çıktı:

```
ÇIKTI
2 adet çıkarıldı, kalan: 3
Defter için stok yetersiz (eksik: 1)
3 adet çıkarıldı, kalan: 0
```

İkinci istekte stok 3, istenen 4'tür; istisna fırlatıldığı için `Stok` değişmez ve üçüncü istek başarılı olur.

Soru 15.

```
try
{
    IsYap();
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"Yakalandı: {ex.Message}");
}

static void IsYap()
{
    using Gunluk g = new Gunluk();
    g.Yaz("işlem başladı");
    throw new InvalidOperationException("işlem başarısız");
}

class Gunluk : IDisposable
{
    public Gunluk() { Console.WriteLine("Günlük açıldı"); }
    public void Yaz(string satir) { Console.WriteLine($"  kayıt: {satir}"); }
    public void Dispose() { Console.WriteLine("Günlük kapandı"); }
}
```

Çıktı:

```
ÇIKTI
Günlük açıldı
  kayıt: işlem başladı
Günlük kapandı
Yakalandı: işlem başarısız
```

İstisna `IsYap` metodundan çıkarken `using var` ile açılan nesnenin `Dispose` metodu çalışır; bu yüzden "Günlük kapandı" satırı, `catch` bloğundaki satırdan önce yazılır.

Soru 16.

```
string[] islemler = { "10 / 4", "8 / 0", "7 x 2", "on + 5" };
foreach (string islem in islemler)
{
    try
    {
        Console.WriteLine($"{islem} = {Hesapla(islem)}");
    }
    catch (InvalidOperationException ex)
    {
        string neden = ex.InnerException?.GetType().Name ?? "yok";
        Console.WriteLine($"{islem}: {ex.Message} (neden: {neden})");
    }
}

static int Hesapla(string islem)
{
    string[] parca = islem.Split(' ');
    try
    {
        int a = int.Parse(parca[0]);
        int b = int.Parse(parca[2]);
        return parca[1] switch
        {
            "+" => a + b,
            "/" => a / b,
            _ => throw new InvalidOperationException("bilinmeyen işlem")
        };
    }
    catch (FormatException ex)
    {
        throw new InvalidOperationException("sayı okunamadı", ex);
    }
    catch (DivideByZeroException ex)
    {
        throw new InvalidOperationException("sıfıra bölünemez", ex);
    }
}
```

Çıktı:

```
ÇIKTI
10 / 4 = 2
8 / 0: sıfıra bölünemez (neden: DivideByZeroException)
7 x 2: bilinmeyen işlem (neden: yok)
on + 5: sayı okunamadı (neden: FormatException)
```

`7 x 2` için işleç hatası, `switch` ifadesindeki `throw` ifadesiyle doğrudan `InvalidOperationException` olarak fırlatılır ve iç istisnası yoktur. Diğer iki durumda asıl istisna `InnerException` olarak saklanır.

Bölüm 54 Koleksiyonlar ve Generics

Soru 1.

Cevap: D. `Count`, `List<T>` gibi koleksiyonların eleman sayısı özelliğidir: `liste.Count` 3 verir. A yanlış: `List<T>`'de `Length` yoktur. B yanlış: dizinin eleman sayısı `Length`'tir. C yanlış: `string`'in uzunluğu `Length`'tir. E yanlış: `Length` özelliktir, `Length()` biçiminde çağrılamaz.

Soru 2.

Cevap: A. `Remove(2)` değeri 2 olan **ilk** elemanı siler: liste `4 0 2` olur. `RemoveAt(2)` ise 2. indeksteki elemanı (sondaki `2`) siler: sonuç `4 0`. E yanlış: `RemoveAt(2)` çağrıldığında listede üç eleman vardır, indeks geçerlidir.

Soru 3.

Cevap: E. İndeksleyiciyle atama (`d["a"] = ...`) anahtar yoksa ekler, varsa değeri değiştirir; hata vermez (A ve B yanlış). `d.Add("b", 3)` yeni bir anahtar ekler (C yanlış). İkinci `d.Add("b", 4)` var olan anahtarı eklemeye çalıştığı için `ArgumentException` fırlatır ve `Count` satırına hiç gelinmez (D yanlış).

Soru 4.

Cevap: B. "Geri" davranışı son giren ilk çıkar (LIFO) gerektirir: sayfa açılınca `Push`, Geri'ye basılınca `Pop`. `Queue` ilk girenden başlar (A yanlış). `HashSet` sıra tutmaz, `Dictionary` anahtar–değer içindir (C, D yanlış). Dizi boyutu sabittir (E yanlış).

Soru 5.

Cevap: B. `new T()` yazabilmek için `T`'nin parametresiz bir yapıcısının olduğu `new()` kısıtıyla bildirilmelidir; `class` kısıtı bunu sağlamaz (örneğin `string` bir sınıftır ama parametresiz yapıcısı yoktur). Doğrusu `where T : class, new()` biçimindedir; `new()` kısıtı kısıt listesinde en sona yazılır. E yanlış: kısıtsız `new T()` derlenmez (`CS0304`).

Soru 6.

Baştan sona indeksle silmek eleman atlatır. Adım adım:

`i`	Liste (kontrol öncesi)	`l[i]`	İşlem
0	`[2, 4, 4, 5, 6, 8]`	2	çift, sil
1	`[4, 4, 5, 6, 8]`	4	çift, sil (0. indeksteki `4` atlandı)
2	`[4, 5, 6, 8]`	6	çift, sil
3	`[4, 5, 8]`	-	`3 < 3` yanlış, döngü biter

Sonuç `[4, 5, 8]`, `Count` 3. `Insert(1, 9)` sonrası liste `[4, 9, 5, 8]`; `8`'in indeksi 3, `4` hâlâ listede.

```
ÇIKTI
4 5 8 | 3
3 True 4 9 5 8
```

Soru 7.

- Döngü sonunda Elif 30, Can 20, Bora 10 olur. `GetValueOrDefault` olmayan anahtar için 0 verir.
- `puan["Can"] -= 5` Can'ı 15 yapar. `Remove("Bora")` sonrası sözlükte Elif ve Can kalır: `Count` 2, `ContainsKey("Bora")` `False`.
- `Values` toplamı 30 + 15 = 45. Çıktı sözlük sırasından etkilenmez.

```
ÇIKTI
2 False
45
30 15
```

Soru 8.

- `k` başta `1 2 3`. `Dequeue()` 1'i alır, `Enqueue` onu sona ekler: `2 3 1`.
- `y` 'ye 1, 2, 3 itilmiştir (tepe 3). `Pop()` 3'ü alır, `3 * 10 = 30` tekrar itilir. `Stack` üstten alta dolaşıldığı için `30 2 1` yazılır.
- `h`, `{2, 3, 1}` ile başlar: `Add(2)` zaten var, `False`; `Add(5)` yeni, `True`.

```
ÇIKTI
2 3 1
30 2 1
False True
```

Soru 9.

Kısıtsız `T` hakkında yalnızca `object` işlemleri bilinir; `<` işleci her tipte tanımlı değildir, bu yüzden derleyici `CS0019` hatası verir. `T` 'nin karşılaştırılabilir olduğunu `where T : IComparable<T>` ile bildirir ve `<` yerine `CompareTo` kullanırız. `int`, `double` ve `string` bu arayüzü uygular.

```
static T EnKucuk<T>(List<T> liste) where T : IComparable<T>
{
    T enKucuk = liste[0];
    foreach (T oge in liste)
    {
        if (oge.CompareTo(enKucuk) < 0)
        {
            enKucuk = oge;
        }
    }
    return enKucuk;
}
```

`EnKucuk(new List<int> { 5, 2, 8 })` 2, `{ 2.5, 1.5 }` listesi 1.5, `{ "muz", "elma", "kiraz" }` listesi `elma` verir.

Soru 10.

`foreach` koleksiyonun sayıcısını kullanır. `Remove` listeyi dolaşma sürerken değiştirdiği için sayıcı bir sonraki adımda `InvalidOperationException` ("Collection was modified") fırlatır. İki düzeltme:

```
// Yol 1: sondan başa indeksli döngü
for (int i = sayilar.Count - 1; i >= 0; i--)
{
    if (sayilar[i] % 2 == 0)
    {
        sayilar.RemoveAt(i);
    }
}

// Yol 2: RemoveAll
sayilar.RemoveAll(s => s % 2 == 0);
```

İkisi de listeyi `3 5 7` yapar. Sondan başa dolaşmak işe yarar, çünkü silme henüz bakılmamış elemanları kaydırmaz.

Soru 11.

1. Tip güvenliği: `List<int>`'e yanlış tipte eleman eklemek derleme hatasıdır; `ArrayList`'te ancak çalışırken fark edilir.
2. Dönüşüm gerekmez: elemanlar `int` olarak okunur; `ArrayList`'te her okumada `(int)liste[i]` yazmak gerekir ve yanlış dönüşüm `InvalidCastException` verir.
3. Kutulama yoktur: `List<int>` sayıları doğrudan kendi dizisinde tutar; `ArrayList` her `int`'i öbekte bir nesneye sarar, bu da hem yavaş hem bellek açısından pahalıdır.

Soru 12.

`HashSet` kopyaları ayıklar, `SortedSet` sıralar.

```
HashSet<int> kume = new() { 5, 3, 9, 3, 5, 1, 9, 9 };
SortedSet<int> sirali = new(kume);
Console.WriteLine(string.Join(" ", sirali));
```

```
ÇIKTI
1 3 5 9
```

Soru 13.

Her harfe ait liste yoksa oluşturup sözlüğe koyarız (`TryGetValue` kalıbı). Sözlük sırası garanti olmadığı için anahtarları listeye alıp sıralarız.

```
string[] kelimeler = { "armut", "elma", "ayva", "erik", "muz", "kiraz", "kavun" };
Dictionary<char, List<string>> gruplar = new();
foreach (string k in kelimeler)
{
    char harf = k[0];
    if (!gruplar.TryGetValue(harf, out List<string>? liste))
    {
        liste = new List<string>();
        gruplar[harf] = liste;
    }
    liste.Add(k);
}
List<char> harfler = new(gruplar.Keys);
harfler.Sort();
foreach (char h in harfler)
{
    Console.WriteLine($"{h}: {string.Join(", ", gruplar[h])}");
}
```

```
ÇIKTI
a: armut, ayva
e: elma, erik
k: kiraz, kavun
m: muz
```

Soru 14.

Her eleman için sayacı artırır, o ana kadarki en büyük sayıyı ve sahibini tutarız. `notnull` kısıtı, `T`'nin sözlük anahtarı olabilmesi için gereklidir.

```
static T EnCokTekrar<T>(IEnumerable<T> kaynak) where T : notnull
{
    Dictionary<T, int> sayac = new();
    T enCokOge = default!;
```

```
    int enCok = 0;
    foreach (T oge in kaynak)
    {
        int adet = sayac.GetValueOrDefault(oge) + 1;
        sayac[oge] = adet;
        if (adet > enCok)
        {
            enCok = adet;
            enCokOge = oge;
        }
    }
    return enCokOge;
}
```

Çağrılar sırasıyla `a` ve `7` yazdırır. Eşitlik durumunda ilk ulaşan eleman kazanır.

Soru 15.

Dizi yapıcıda `kapasite` uzunluğunda ayrılır. `Push` doluyken `false` döndürür; `TryPop` boşken `out` parametresine varsayılan değeri verip `false` döndürür.

```
class SinirliYigin<T>
{
    private readonly T[] _ogeler;
    private int _sayi;

    public SinirliYigin(int kapasite)
    {
        _ogeler = new T[kapasite];
    }

    public bool Push(T oge)
    {
        if (_sayi == _ogeler.Length)
        {
            return false;
        }
        _ogeler[_sayi++] = oge;
        return true;
    }

    public bool TryPop(out T oge)
    {
        if (_sayi == 0)
        {
            oge = default!;
            return false;
        }
        oge = _ogeler[--_sayi];
        return true;
    }
}
```

`new SinirliYigin<int>(3)` ile 10, 20, 30 itilir (üçü de `True`), dördüncü `Push(40)` `False` döner; `TryPop` ile 30, 20, 10 çıkar, sonraki çağrı `False` verir.

Bölüm 55 Delegate, Event ve Lambda İfadeleri

Soru 1.

Cevap: B. `Func`'ta son tip dönüş tipidir; iki `int` parametre ve `string` dönüş için `Func<int, int, string>` gerekir.

Soru 2.

Cevap: C. `Predicate<T>`, bir `T` alıp `bool` döndürür.

Soru 3.

Cevap: C. Olaya dışarıdan yalnızca `+=` ve `-=` uygulanabilir; çağırmak ve `=` ile atamak yalnızca tanımlayan sınıfın içinde mümkündür.

Soru 4.

Cevap: `50`. Lambda `x` değişkenini kopyalamaz, değişkenin kendisini yakalar; çağrıldığında `x` artık 5'tir.

Soru 5.

Cevap: `k1 k2 m1 m2` (sonda bir boşluk, ardından satır sonu). `-=` içindeki lambda ayrı bir delegate nesnesi olduğundan listedeki ikinci lambdayı silmez ve hata da vermez.

Soru 6.

Cevap: `333`. Üç lambda da döngü sonunda 3 olan aynı `i` değişkenini yakalar.

Soru 7.

Cevap: `7` ve `2`. Çok noktaya yayında yalnızca son metodun değeri alınır; listede iki metot vardır.

Soru 8.

`Topla` tek parametre alır, `Islem` iki parametreli bir imza ister; atama derlenmez (CS0123). Düzeltme: `int Topla(int a, int b) => a + b;`.

Soru 9.

Olaya sınıfın dışından `d.Tiklandi()` ile çağrı yapılamaz (CS0070); olayı yalnızca `Dugme` sınıfı içinden tetikleyebiliriz. Sınıfa `public void Tikla() => Tiklandi?.Invoke();` ekleyip `d.Tikla()` çağırılır.

Soru 10.

İki `int` parametre alır ve `double` döndürür.

Soru 11.

Hiç abone yoksa olay `null`'dır ve doğrudan çağrı `NullReferenceException` fırlatır. `?.Invoke` bu durumda çağrıyı atlar; ayrıca delegate'i tek kez okuduğu için iş parçacığı yarışından da korur.

Soru 12.

```
Func<int, int> kareInt = x => x * x;
Func<double, double> kareDouble = x => x * x;
```

Parametre tipi atandığı delegate tipinden çıkarılır.

Soru 13.

`EventArgs` sınıfına sonradan özellik eklemek mevcut abonelerin imzasını bozmaz; `Action<double>` gibi bir olayın imzasını değiştirmek ise tüm abonelerin kodunu kırar. Ayrıca `gonderen` parametresi hangi nesnenin tetiklediğini bildirir.

Soru 14.

```
int[] sayilar = { 4, 7, 12, 3, 20, 11 };
Console.WriteLine(Uygula(sayilar, n => n % 2 == 0));   // 36
Console.WriteLine(Uygula(sayilar, n => n > 10));       // 43
```

```
int Uygula(int[] dizi, Func<int, bool> kosul)
{
    int toplam = 0;
    foreach (int n in dizi)
        if (kosul(n)) toplam += n;
    return toplam;
}
```

Koşul delegate olarak parametre alınır; hangi elemanların toplanacağını çağıran lambda belirler.

Soru 15.

```
var kutu = new MetinKutusu();
EventHandler<string> a = (g, m) => Console.WriteLine($"A: {m}");
EventHandler<string> b = (g, m) => Console.WriteLine($"B: {m}");
kutu.Eklendi += a;
kutu.Eklendi += b;
kutu.Ekle("ilk");
kutu.Eklendi -= b;
kutu.Ekle("ikinci");

class MetinKutusu
{
    private readonly List<string> metinler = new();
    public event EventHandler<string>? Eklendi;

    public void Ekle(string metin)
    {
        metinler.Add(metin);
        Eklendi?.Invoke(this, metin);
    }
}
```

Aboneliği iptal edebilmek için lambdalar değişkenlerde saklanmıştır.

Soru 16.

```
int sayac = 0;
Tekrarla(() => sayac++);
Console.WriteLine(sayac);    // 3

void Tekrarla(Action eylem)
{
    for (int i = 0; i < 3; i++)
        eylem();
}
```

Lambda `sayac` değişkenini yakalar ve her çağrıda artırır.

Bölüm 56 LINQ

Soru 1.

Cevap: A. `Where` önce 3'ten büyükleri (4, 9, 7) seçer, `Select` her birini ikiyle çarpar: 8, 18, 14. Sıra korunur.

Soru 2.

Cevap: D. `Count` sonucu hemen hesaplar. `Where`, `Select`, `OrderBy` ve `Skip` ertelenmiş yürütülür; sonuç gezilene ya da `ToList` gibi bir metotla istenene kadar çalışmazlar.

Soru 3.

Cevap: C. `Where` yalnızca bir sorgu nesnesi döndürür. Sonuç gezilmediği ya da `ToList()`, `Count()` gibi bir metotla tüketilmediği için lambda hiç çalışmaz. Liste de zaten değişmez; LINQ kaynağı değiştirmez.

Soru 4.

Cevap: D. Boş `int` dizisinde `Average`, `Max`, `First` ve `Single` `InvalidOperationException` fırlatır. `Sum` boş dizide `0` döndürür.

Soru 5.

Cevap: `30`. Sorgu `Sum()` çağrılana kadar çalışmaz. Çalıştığında liste 1, 2, 3, 4 içerir; kareler toplamı 1 + 4 + 9 + 16 = 30.

Soru 6.

Çıktı:

```
Ali,Can,Ece
2
```

Uzunluğu 3 olanlar Ece, Can, Ali; alfabetik sıra Ali, Can, Ece. `Skip(1).Take(2)` iki eleman verir, sayısı 2.

Soru 7.

Cevap: `4`. Lambda `esik`'i yakalar ve değeri sorgu çalıştığında (`Count()` çağrısında) okunur. O sırada `esik` 1'dir; 1'den büyük 2, 3, 4, 5 olmak üzere dört sayı vardır.

Soru 8.

Çıktı:

```
e:2
a:2
k:1
```

Anahtar ilk harftir. Gruplar anahtarın ilk göründüğü sırayla gelir: elma ve erik `e`, armut ve ayva `a`, kiraz `k` grubundadır.

Soru 9.

Hata, `Where`'den sonra `ThenBy` çağıran üçüncü satırdadır; `ThenBy` yalnızca `OrderBy` sonucunda çağrılabilir. Düzeltme:

```
var s = sayilar.Where(x => x > 1).OrderBy(x => x);
```

Ya da ikinci ölçüt gerçekten gerekiyorsa `Where` önce yazılır: `sayilar.Where(x => x > 1).OrderBy(x => x).ThenBy(x => -x)`.

Soru 10.

`foreach` bir sorguyu gezerken `l.Remove(x)` kaynağı değiştirir; sonraki adımda `InvalidOperationException` (koleksiyon değiştirildi) fırlar. Önce sonucun kopyası alınmalı:

```
foreach (var x in l.Where(x => x % 2 == 0).ToList())
{
    l.Remove(x);
}
```

Daha kısası `l.RemoveAll(x => x % 2 == 0);` çağrısıdır.

Soru 11.

`First`, koşulu sağlayan eleman yoksa `InvalidOperationException` fırlatır. `FirstOrDefault` tipin varsayılan değerini döndürür (`null`, `0`, `false`). `int` dizisinde varsayılan değer `0` olduğundan, "bulunamadı" sonucu dizide gerçekten bulunan bir `0` ile karışır. Bu durumda önce `Any(koşul)` ile varlık sorulabilir.

Soru 12.

İki kez çalışır; her gezme sorguyu kaynaktan baştan yeniden hesaplar, lambda her eleman için iki kez çağrılır. Sonucu `ToList()` ya da `ToArray()` ile bir kez hesaplayıp onu gezmek bunu önler.

Soru 13.

```
int[] a = { 3, -1, 4, -1, 5, 9, -2 };
var pozitif = a.Where(x => x >= 0).ToList();
Console.WriteLine($"Toplam: {pozitif.Sum()}, ortalama: {pozitif.Average()}");
```

Çıktı `Toplam: 21, ortalama: 5.25`. Liste boş olabilseydi `Average` istisna fırlatırdı; bu veride en az bir pozitif sayı vardır.

Soru 14.

```
string[] kelimeler = { "elma", "Armut", "ELMA", "kiraz", "armut", "Fig" };
var sonuc = kelimeler
    .Distinct(StringComparer.OrdinalIgnoreCase)
    .OrderBy(k => k.Length)
    .ThenBy(k => k, StringComparer.OrdinalIgnoreCase);
Console.WriteLine(string.Join(", ", sonuc));
```

Çıktı `Fig, elma, Armut, kiraz`. `Distinct` karşılaştırıcı alır; ilk görülen yazım korunur.

Soru 15.

```
var calisanlar = new List<Calisan>
{
    new("Ayşe", "Yazılım", 52000m), new("Mehmet", "Yazılım", 48000m),
    new("Elif", "Muhasebe", 39000m), new("Can", "Muhasebe", 41000m),
    new("Zeynep", "Satış", 35000m), new("Mert", "Satış", 37000m),
};

var ortalamalar = calisanlar
    .GroupBy(c => c.Departman)
    .Select(g => new { Dep = g.Key, Ort = g.Average(c => c.Maas) })
    .OrderByDescending(x => x.Ort);
foreach (var x in ortalamalar)
{
    Console.WriteLine($"{x.Dep}: {x.Ort}");
}

record Calisan(string Ad, string Departman, decimal Maas);
```

`GroupBy` departmanlara böler, `Select` her gruptan bir ortalama çıkarır, `OrderByDescending` sıralar.

Soru 16.

```
var ilk3 = calisanlar.OrderByDescending(c => c.Maas).Take(3);
Console.WriteLine(string.Join(", ", ilk3.Select(c => c.Ad)));

foreach (var g in calisanlar.GroupBy(c => c.Departman))
```

```
{
    var enAz = g.MinBy(c => c.Maas)!;
    Console.WriteLine($"{g.Key}: {enAz.Ad}");
}
```

(`Calisan` kaydı ve liste önceki soruyla aynıdır.) C++'ta ilk üç için `std::partial_sort` ya da `std::sort` ile `greater` karşılaştırıcısı kullanılır; departman gruplamasında `std::map<std::string, std::vector<Calisan>>` kurulur, her vektörde `std::min_element` ile en düşük maaşlı bulunur. Gruplama için LINQ'daki `GroupBy` gibi hazır bir algoritma yoktur.

Bölüm 57 Dosya İşlemleri

Soru 1.

Cevap: A. `WriteAllText` her çağrıda dosyayı baştan yazar; ikinci çağrı birincinin içeriğini siler.

Soru 2.

Cevap: B. `AppendAllText` dosyanın sonuna ekler. `WriteAllText` üzerine yazar, `Create` dosyayı boşaltarak açar.

Soru 3.

Cevap: D. `UnauthorizedAccessException` `SystemException`'dan türer, `IOException`'dan türemez. Diğerlerinin hepsi `IOException` soyundandır.

Soru 4.

Cevap: C. `ReadLine` dosya sonunda `null` döndürür; `Read` ise `-1` döndürür. Döngüler bu yüzden `!= null` ile yazılır.

Soru 5.

Cevap: C. `Path.Combine` işletim sisteminin ayracını kullanır. A ve B yalnızca bir sistemde doğru çalışır.

Soru 6.

Cevap: B. `using`, `Dispose` çağırır; `StreamWriter` için bu, tamponu boşaltıp dosyayı kapatmak demektir.

Soru 7.

`WriteAllLines` satırlar sonuna satır sonu koyar: `a`, `b`. `AppendAllText("c")` satır sonu eklemez, dosya `a\nb\nc` olur; `AppendAllLines` ise `d`'yi `c`'ye yapıştırır ve sonuna satır sonu koyar: `cd`. Satırlar `a`, `b`, `cd`.

```
ÇIKTI
3
bcd
```

Burada `s[1] + s[2]` ifadesi `"b" + "cd"` olduğundan `bcd` yazılır.

Soru 8.

`GetFileNameWithoutExtension` yalnızca son noktadan sonrasını uzantı sayar.

```
ÇIKTI
rapor.final
.txt
True
```

`ChangeExtension` başındaki noktayı yoksa kendisi ekler.

Soru 9.

`StreamWriter` kapatılmadığı için tamponda kalan satırlar dosyaya yazılmayabilir ve dosya kilitli kalabilir. Düzeltme: `using var w = new StreamWriter("log.txt");` ya da `using (...) { ... }` bloğu.

Soru 10.

`WriteAllText` her seferinde dosyayı ezer; yalnızca son çağrı kalır. Düzeltme: `File.AppendAllText` kullanmak (başlamadan önce eski dosyayı `File.Delete` ile silmek gerekebilir).

```
File.Delete("sayac.txt");
for (int i = 1; i <= 3; i++)
    File.AppendAllText("sayac.txt", $"Deneme {i}\n");
```

Soru 11.

`FileNotFoundException`, `IOException`'dan türediği için ilk `catch` onu da yakalar; ikinci `catch` hiç çalışmaz (derleyici bu durumda `CS0160` hatası verir). Özel istisna önce yazılmalıdır:

```
catch (FileNotFoundException) { Console.WriteLine("Dosya yok"); }
catch (IOException) { Console.WriteLine("G/Ç hatası"); }
```

Soru 12.

```
var yaslar = new List<int>();
foreach (string satir in File.ReadLines("kisiler.csv").Skip(1))
{
    string[] p = satir.Split(',');
    if (p.Length == 2 && int.TryParse(p[1], out int yas))
        yaslar.Add(yas);
}
if (yaslar.Count > 0)
    Console.WriteLine($"Ortalama yaş: {yaslar.Average():F1}");
else
    Console.WriteLine("Geçerli kayıt yok.");
```

Başlık `Skip(1)` ile atlanır, bozuk satırlar `TryParse` başarısız olunca eklenmez.

Soru 13.

```
string klasor = args.Length > 0 ? args[0] : ".";
if (!Directory.Exists(klasor))
{
    Console.WriteLine("Klasör bulunamadı.");
    return;
}
long toplam = 0;
int adet = 0;
foreach (string f in Directory.EnumerateFiles(klasor, "*.txt",
                                              SearchOption.AllDirectories))
{
    adet++;
    toplam += new FileInfo(f).Length;
}
Console.WriteLine($"{adet} dosya, {toplam} bayt");
```

Soru 14.

```
File.WriteAllText("metin.txt", "bir iki üç\ndört beş\n");
int satir = 0, kelime = 0, karakter = 0;
foreach (string s in File.ReadLines("metin.txt"))
{
    satir++;
    karakter += s.Length;
    kelime += s.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
}
Console.WriteLine($"{satir} satır, {kelime} kelime, {karakter} karakter");
```

Karakter sayısı satır sonları hariç tutulmuştur; kelimeler boşluktan ayrılır. Bu örnekte çıktı `2 satır, 5 kelime, 18 karakter` olur.

Soru 15.

```
using System.Text.Json;

var kisiler = new List<Kisi>
{
    new Kisi { Ad = "Selin", Yas = 30 },
    new Kisi { Ad = "Emre", Yas = 22 },
    new Kisi { Ad = "Ece", Yas = 27 },
};
File.WriteAllText("kisiler.json", JsonSerializer.Serialize(kisiler));

var geri = JsonSerializer.Deserialize<List<Kisi>>(
    File.ReadAllText("kisiler.json"));
foreach (Kisi k in geri!.OrderBy(k => k.Yas))
    Console.WriteLine($"{k.Ad} {k.Yas}");

class Kisi
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
}
```

Çıktı yaşa göre `Emre 22`, `Ece 27`, `Selin 30` sırasıdır. `Serialize` ve `Deserialize` genel özelliklerle çalışır, bu yüzden `{ get; set; }` gereklidir.

Bölüm 58 Özyineleme C#'ta

Soru 1.

Cevap: B. `Directory.Delete(yol, true)` klasörü, içindeki dosyaları ve alt klasörleri birlikte siler. İkinci parametre `false` olursa ve klasörde içerik varsa çağrı hata verir. Çöp kutusuna taşıma yapılmaz; silme geri alınamaz.

Soru 2.

Cevap: C. `StackOverflowException` bir sistem hatasıdır; `try`-`catch` ile yakalanamaz ve süreç sonlanır. `catch (Exception)` bloğu da çalışmaz. Sorun ancak kodu düzelterek ya da çağrı derinliğini azaltarak çözülür.

Soru 3.

Cevap: C. Referans tipi olan `List<int>` parametreye referansla geçer; `int` ise değer ile kopyalanır. Metot içinde `List<int>` üzerindeki değişiklik çağıran tarafta görünür, `int` üzerindeki değişiklik görünmez.

Soru 4.

Çıktı:

```
ÇIKTI
8
```

`F(5) = F(4) + F(3)`, `F(4) = F(3) + F(2)`, `F(3) = F(2) + F(1) = 2 + 1 = 3`, `F(2) = 2`. Buradan `F(4) = 5` ve `F(5) = 5 + 3 = 8` bulunur. Bu bir Fibonacci varyasyonudur: `n < 3` temel durumu `F(0) = 0`, `F(1) = 1`, `F(2) = 2` değerlerini verir ve sonraki değerler önceki iki değerin toplamıdır.

Soru 5.

Cevap: C. Anahtar `tutar` ile `k` birlikte bir durumu tanımlar. Yalnızca `tutar` kullanılırsa, `k` değeri farklı olan bir çağrı önceden hesaplanmış ve başka bir `k` için geçerli olan değeri alır. Program derlenir ve hata vermez, ama yanlış sonuç üretebilir.

Soru 6.

Cevap: B. Anahtarlar `(1, 2)` ve `(2, 1)` farklı olduğu için sözlükte iki kayıt vardır. `d[(1, 2)]` ise `"ilk"` değerini döndürür. Değer demetlerinde sıra önemlidir; `(1, 2)` ile `(2, 1)` aynı anahtar değildir.

Soru 7.

Cevap: B. Düğüm değeri, sol alt ağaç işlenmeden önce yazılır. Bu, ön sıra (pre-order) dolaşmasıdır ve çıktı `8 3 1 6 10 14 13` olur. Sıralı dolaşma, düğümü iki alt ağacın arasında ekleyerek elde edilir.

Soru 8.

Cevap: B. Metot `Garip(d.Sol)` çağrısının sonucunu bir yere atamaz ve döndürmez. Sonuç olarak yalnızca kökün değeri döner. Sağ alt ağaç hiç işlenmez. Bu kod derleme hatası vermez; yanlış sonuç veren bir mantık hatasıdır.

Soru 9.

Cevap: Metodun özyinelemeli çağrısı yapılıyor ama dönüş değeri kullanılmıyor. Bu yüzden metot her zaman yalnızca `d[i]` değerini döndürür ve toplam hesaplanmaz. Düzeltilmiş hâli:

```
static int Topla(int[] d, int i)
{
    if (i == d.Length) return 0;
    return d[i] + Topla(d, i + 1);
}
```

Kalan kısmın toplamını çağrının sonucuna ekleyerek özyinelemeyi doğru kurduk.

Soru 10.

Örnek çözüm:

```
static int YaprakSay(Dugum? d)
{
    if (d is null) return 0;
    if (d.Sol is null && d.Sag is null) return 1;    // yaprak
    return YaprakSay(d.Sol) + YaprakSay(d.Sag);
}

Console.WriteLine($"Yaprak: {YaprakSay(kok)}");     // 3
```

Temel durum boş ağaçtır. Yaprak, iki çocuğu da olmayan düğümdür ve 1 döner. Diğer düğümlerde iki alt ağacın yaprak sayıları toplanır. Örnek ağaçta yapraklar 1, 6 ve 13'tür; 14'ün bir çocuğu (13) olduğu için yaprak değildir.

Soru 11.

Örnek çözüm:

```
static long Yol(int n, Dictionary<int, long> hafiza)
{
    if (n == 0) return 1;                  // tam çıkıldı
    if (n < 0) return 0;                   // aşıldı, yol yok
    if (hafiza.TryGetValue(n, out long onceden)) return onceden;

    long sonuc = Yol(n - 1, hafiza) + Yol(n - 2, hafiza) + Yol(n - 3, hafiza);
    hafiza[n] = sonuc;
    return sonuc;
}

Console.WriteLine($"Merdiven 10: {Yol(10, new Dictionary<int, long>())}");
```

Çıktı: `Merdiven 10: 274`. Son adım 1, 2 ya da 3 basamaklık olabileceği için `n` basamağa gelen yolların sayısı, `n - 1`, `n - 2` ve `n - 3` basamağa gelen yolların toplamıdır. Anahtar tek bir `int` olduğu için sözlüğün tipi `Dictionary<int, long>` olur.

Soru 12.

Örnek çözüm:

```
static int TxtSay(string yol)
{
    int sayi = Directory.GetFiles(yol, "*.txt").Length;     // bu klasördeki .txt dosyaları
    foreach (string alt in Directory.GetDirectories(yol))
    {
        sayi += TxtSay(alt);                                 // alt klasörler için özyineleme
    }
    return sayi;
}

string taban = Path.Combine(Path.GetTempPath(), "txtDemo_" + Guid.NewGuid().ToString("N"));
Directory.CreateDirectory(Path.Combine(taban, "alt", "daha"));
File.WriteAllText(Path.Combine(taban, "a.txt"), "x");
File.WriteAllText(Path.Combine(taban, "b.md"), "y");
File.WriteAllText(Path.Combine(taban, "alt", "c.txt"), "z");
File.WriteAllText(Path.Combine(taban, "alt", "daha", "d.txt"), "w");

Console.WriteLine($"txt sayısı: {TxtSay(taban)}");   // 3
Directory.Delete(taban, true);
```

Çıktı: `txt sayısı: 3`. `b.md` dosyası filtreye uymadığı için sayılmaz. Her klasörde yalnızca o klasördeki dosyalar sayılır; alt klasörler yine kendi çağrılarında ele alınır.

Soru 13.

Örnek çözüm:

```
static int Derinlik(Dugum? d, int aranan)
{
    if (d is null) return -1;                  // değer yok
    if (d.Veri == aranan) return 0;            // kökte bulundu

    int alt = aranan < d.Veri ? Derinlik(d.Sol, aranan) : Derinlik(d.Sag, aranan);
    return alt == -1 ? -1 : alt + 1;           // bir düzey aşağıdan geri dönüyoruz
}
```

```
Console.WriteLine($"13 için derinlik: {Derinlik(kok, 13)}");   // 3
Console.WriteLine($"99 için derinlik: {Derinlik(kok, 99)}");   // -1
```

Ağaç bir ikili arama ağacı olduğu için her düğümde tek bir yön seçeriz; `Icerir` ile aynı yapıdır. Çağrı sonucu alt düzeyden geldiğinde bir ekleyerek derinliği yukarı taşırız. Sayaç tutmak yerine dönüş değerini kullanmak, çağrıların birbirini karıştırmadığı bir yazım sağlar.

Bölüm 59 Modern C#: Nullable Referans Tipleri, async/await ve Diğerleri

Soru 1.

Cevap: E. A'da `null` olamayan tipe `null` atanıyor (CS8600), B'de `null` olabilen değişkenin üyesine doğrudan erişiliyor (CS8602). C'de `?.` ve `??` ile `null` güvenle yönetiliyor; D'de ise değişkene gerçek bir metin atandığı için sorun yok.

Soru 2.

Cevap: B. `?.` zincirin herhangi bir halkasında `null` görürse kalan kısım çalışmaz ve ifadenin sonucu `null` olur. Sonuç tipi `c`'nin tipinin `null` alabilen biçimidir (`string?`), `c` `null` ise de hata değil `null` sonucu elde edilir.

Soru 3.

Cevap: `a1`. `x` önce `null` olduğundan `"a"` atanır; ikinci `??=` atama yapmaz. `x?.Length` 1'dir ve `"a" + 1` metni `a1` olur.

Soru 4.

Cevap: `2 3`. İlk satırda `q = 3`, `r = 2` olur. İkinci satır değerleri yer değiştirir: `q = 2`, `r = 3`.

Soru 5.

Cevap: C. `!` yalnızca derleyiciye "bu değer `null` değil" der ve uyarıyı susturur. Çalışma zamanında hiçbir denetim eklemez; değer gerçekten `null` ise `NullReferenceException` yine fırlar.

Soru 6.

Cevap: `dört ile başlıyor`. Kollar yukarıdan aşağıya denenir. Dizi boş değildir ve tek elemanlı değildir; `[4, ..]` kalıbı ilk elemanı 4 olan her diziyle eşleşir. Son kola ulaşılmaz.

Soru 7.

`async void` metodun bitişi ve hatası dışarıdan izlenemez: `Kaydet()` çağıran kod `await` edemez ve `try`/`catch` ile içindeki istisnayı yakalayamaz. İstisna, işlenmemiş istisna olarak programı sonlandırabilir. `async Task` döndüren metot ise `await` edilebilir; hata `await` noktasında yakalanabilir.

Soru 8.

Cevap: B. `async` kendiliğinden yeni iş parçacığı açmaz ve metodu hızlandırmaz. Yalnızca `await` kullanımını mümkün kılar; derleyici metodu durum makinesine çevirir. Dönüş tipi `Task`, `Task<T>` veya (olay işleyiciler için) `void` olabilir.

Soru 9.

Çıktı:

```
ÇIKTI
içeride
A
6
B
```

`Cevir(3)` çağrısı, ilk `await`'e kadar hemen çalışır; bu yüzden `içeride` önce yazılır. Görev beklerken ana kod `A`'yı yazar, ardından `await t` sonucu bekler ve `6` yazılır.

Soru 10.

Yaklaşık 600 ms; her bekleme öncekinin bitmesini bekler. Üç görevi önce başlatıp `Task.WhenAll` ile birlikte beklersek süre yaklaşık 200 ms olur:

```
await Task.WhenAll(Task.Delay(200), Task.Delay(200), Task.Delay(200));
```

Soru 11.

Cevap: C. `WhenAll` sonuçları tamamlanma sırasıyla değil, görevlerin verildiği sırayla döndürür.

Soru 12.

`Sahip` özelliği `null` olamayan `string`'tir ama yapıcıda değer almıyor; derleyici `CS8618` uyarısı verir. Üç düzeltme vardır: ilk değer vermek, `required` yapmak ya da `string?` yazmak.

```
class Hesap
{
    public string Sahip { get; set; } = "";
    public decimal Bakiye { get; set; }
}
```

Soru 13.

```
static string Temizle(string? metin) =>
    string.IsNullOrWhiteSpace(metin) ? "(isimsiz)" : metin.Trim();

Console.WriteLine(Temizle("  Deniz "));
Console.WriteLine(Temizle(null));
Console.WriteLine(Temizle("   "));
```

`IsNullOrWhiteSpace` metin `null`, boş ya da yalnızca boşluklardan oluşuyorsa `true` döndürür. Derleyici akış analizi sayesinde `:` sonrasında `metin`'in `null` olmadığını bilir. Çıktı: `Deniz`, `(isimsiz)`, `(isimsiz)`.

Soru 14.

```
static (double Ortalama, int Maks) Ozet(int[] dizi) =>
    (dizi.Average(), dizi.Max());

var (ort, maks) = Ozet([4, 8, 6, 2]);
Console.WriteLine($"Ortalama={ort} Maks={maks}");
```

Adlandırılmış tuple iki değeri birlikte döndürür, `var (ort, maks)` ise onları ayrı değişkenlere dağıtır. Çıktı: `Ortalama=5 Maks=8`.

Soru 15.

```
static async Task<int> Karesi(int n)
{
    await Task.Delay(10);
    return n * n;
}
```

```
var gorevler = Enumerable.Range(1, 5).Select(Karesi).ToArray();
int[] kareler = await Task.WhenAll(gorevler);
Console.WriteLine(kareler.Sum());
```

`ToArray()` görevlerin hepsini hemen başlatır; `WhenAll` hepsini birlikte bekler. Çıktı: `55`.

Bölüm 61 Üç Dil Yan Yana

Soru 1.

Cevap: C. Yalnızca C'de bellek elle serbest bırakılır; C++ kapsayıcıları yıkıcıyla, C# nesneleri çöp toplayıcıyla temizlenir.

Soru 2.

Cevap: B. C#'ta `int`, `System.Int32`'nin takma adıdır ve dil tanımı gereği 32 bittir.

Soru 3.

Cevap:

```
ÇIKTI
3 3.5
```

`a / b` tamsayı bölmesidir (3); ikinci ifadede `(double)` dönüşümü bölmeyi gerçek sayı bölmesine çevirir (3.5).

Soru 4.

Cevap:

```
ÇIKTI
True
False
```

C#'ta `string` için `==` içeriği karşılaştırır. `new string(...)` çalışma zamanında yeni bir nesne üretir; sabit `"ab"` ise ayrı bir nesnedir, bu yüzden `ReferenceEquals` yanlıştır.

Soru 5.

Cevap: `ad == "elma"` iki adresi karşılaştırır, içeriği değil; sonuç hemen her zaman yanlıştır. Doğrusu `if (strcmp(ad, "elma") == 0)` ve `<string.h>` eklemektir.

Soru 6.

Cevap: C++'ta `void *`'dan `int *`'a örtük dönüşüm yoktur. Çözümler: `int *p = (int *)malloc(4 * sizeof(int));` (ve `free` ile bırakmak) ya da C++ deyimi olarak `vector<int> p(4);` veya `int *p = new int[4];` (ve `delete[]`). Tercih `vector`'dür.

Soru 7.

Cevap: B. C ve C++'ta atama bir ifadedir, değeri 5'tir ve sıfırdan farklı olduğundan koşul doğrudur. C#'ta koşul `bool` olmak zorundadır, `int` atanamaz.

Soru 8.

Cevap: C. `Dictionary` karma tablodur ve sıra vaat etmez; `unordered_map` de öyledir. `map` ise anahtara göre sıralı bir ağaçtır.

Soru 9.

Cevap:

```
ÇIKTI
1 2 3
```

`auto x` her turda elemanın bir kopyasını alır; kopya değişir, vektör değişmez. Değişiklik için `auto& x` yazılmalıdır.

Soru 10.

Cevap: B. `using` bloğu bittiğinde `Dispose` çağrılır; bu, kapsam sonunda yıkıcının çalışması fikrinin karşılığıdır.

Soru 11.

C:

```c
#include <stdio.h>

int main(void)
{
    int d[] = {4, 9, 2, 7};
    int en = d[0];
    for (int i = 1; i < 4; i++)
        if (d[i] > en) en = d[i];
    printf("%d\n", en);
    return 0;
}
```

C++:

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> d = {4, 9, 2, 7};
    std::cout << *std::max_element(d.begin(), d.end()) << '\n';
}
```

C#:

```csharp
int[] d = { 4, 9, 2, 7 };
Console.WriteLine(d.Max());
```

Üçü de 9 yazar. C elle döngü kurar; C++ ve C# hazır algoritma kullanır.

Soru 12.

C# ile örnek (diğer dillerde de `dizi[0]` ve son eleman farkı yeterlidir):

```csharp
// sıralı dizi üzerinde: dizi[0] en küçük, dizi[^1] en büyük
Console.WriteLine($"Aciklik  : {dizi[^1] - dizi[0]:F2}");
```

Örnek veride sonuç `35.00` olur (95 - 60).

Soru 13.

C++ ile örnek:

```
while (metin >> kelime) {
    for (char& c : kelime)
        c = static_cast<char>(tolower(static_cast<unsigned char>(c)));
    ++sayac[kelime];
}
```

Sayaca eklemeden önce kelime küçük harfe çevrilir (`<cctype>` gerekir). C#'ta aynı işi `kelime.ToLowerInvariant()` yapar, C'de ise `tolower` ile karakter karakter dolaşılır. Sözlük oluşturulurken `StringComparer.OrdinalIgnoreCase` da kullanılabilir.